Domain Theory in Lean 4

Lars Warren Ericson[†], **Dana S. Scott**, **Vijay D'Silva**[‡], **Brian Milnes**[§]

October 2026
CMU-CS-26-140

School of Computer Science
Carnegie Mellon University
Pittsburgh, PA 15213

[†] Catskills Research Company, [‡] Google Research, [§] Unaffiliated

Keywords: Lean 4; formal verification; domain theory; continuous lattices; neighbourhood systems; information systems; Scott topology

Abstract

This document is a machine-checked Lean 4 formalization of Dana Scott's papers on domain theory: continuous lattices (1972), datatypes as lattices (1976), neighbourhood systems (1980), and information systems (1982), together with bridge theorems relating the lattice, neighbourhood, and information systems presentations.

Contents

1 Continuous Lattices (1972) 12

1.1 Introduction . . . 13
1.2 Injective spaces . . . 14
1.2.1 T0 spaces . . . 14
1.2.2 Definition 1.1: Injective Space . . . 15
1.2.3 Proposition 1.2 . . . 16
1.2.4 Proposition 1.3 . . . 16
1.2.5 Proposition 1.4 . . . 17
1.2.6 Proposition 1.5 . . . 18
1.2.7 Corollary 1.6 . . . 19
1.2.8 Corollary 1.7 . . . 19
1.2.9 Absolute retracts . . . 20
1.3 Continuous lattices . . . 20
1.3.1 Specialization . . . 22
1.3.2 Scott open . . . 24
1.3.3 Way-below relation . . . 26
1.3.4 Proposition 2.2 . . . 27
1.3.5 Definition 2.3 . . . 29
1.3.6 Proposition 2.4 . . . 30
1.3.7 Scott Maps . . . 31
1.3.8 Milner Correction . . . 38
1.3.9 Proposition 2.8 . . . 40
1.3.10 Proposition 2.9 . . . 40
1.3.11 Proposition 2.10 (flawed) . . . 43
1.3.12 Proposition 2.11 . . . 44
1.4 Function spaces . . . 46
1.4.1 Definition 3.1: Function Space . . . 46
1.4.2 Proposition 3.2 . . . 47
1.4.3 Theorem 3.3 . . . 47
1.4.4 Corollary 3.4 . . . 48
1.4.5 Proposition 3.5 . . . 49
1.4.6 Definition 3.6 . . . 49
1.4.7 Proposition 2.10 revisited . . . 51
1.4.8 Theorem 2.12 . . . 52
1.4.9 Proposition 3.7 . . . 53
1.4.10 Proposition 3.8 . . . 54
1.4.11 Lemma 3.9 . . . 56
1.4.12 Proposition 3.10 . . . 57
1.4.13 Definition 3.11: Projection space . . . 60
1.4.14 Proposition 3.12 . . . 61
1.4.15 Proposition 3.13 . . . 62
1.4.16 Proposition 3.14 . . . 64
1.5 Inverse limits . . . 66

1.5.1 Inverse limits . . . 66

2 Datatypes as Lattices (1976) 133
2.1 Introduction . . . 133
2.1.1 Historical note . . . 135
2.2 Continuous functions . . . 135
2.2.1 Introduction . . . 135
2.2.2 e . . . 137
2.2.3 pair . . . 140
2.2.4 Continuous . . . 143
2.2.5 Graph . . . 146
2.2.6 FixedPoint . . . 148
2.2.7 Embedding . . . 153
2.3 Computability and definability . . . 158
2.3.1 Historical note . . . 158
2.3.2 Meaning . . . 159
2.3.3 LAMBDA . . . 163
2.3.4 Theorem 2.1 . . . 204
2.3.5 Theorem 2.2 . . . 204
2.3.6 Theorem 2.3 . . . 208
2.3.7 Theorem 2.4 . . . 209
2.3.8 The combinator Y . . . 211
2.3.9 Theorem 2.5 . . . 215
2.3.10 The graph model . . . 221
2.3.11 Elementary definitions . . . 226
2.3.12 Sequences . . . 236
2.3.13 Distributive functions . . . 246
2.3.14 Primitive recursion . . . 253
2.3.15 Definition (computable) . . . 264
2.3.16 Computable functions . . . 264
2.3.17 Definability . . . 264
2.3.18 A law of Y . . . 268
2.4 Retracts and data types . . . 272
2.5 Closure operations and algebraic lattices . . . 297
2.6 Subsets and their classification . . . 299
2.7 Total functions and functionality . . . 301
2.8 Enumeration and degrees . . . 314

3 Neighborhood Systems (1980) 320
3.1 Introduction . . . 321
3.2 Domains given by neighbourhoods . . . 322
3.2.1 Introduction . . . 322
3.2.2 Definition 1.1 — neighbourhood system . . . 324
3.2.3 Factoid 1.1 — empty intersection is delta; recursive finite intersection . . . 325
3.2.4 Example 1.2 — two-token domain . . . 328
3.2.5 Example 1.3 — three-step chain . . . 334

3.2.6 Example 1.4 — finite binary tree . . . 340
3.2.7 Factoid 1.4a — nested or disjoint . . . 358
3.2.8 Example 1.5 — all nonempty subsets . . . 359
3.2.9 Factoid 1.5 — consistent iff nonempty; equivalent sequences same limit . . . 360
3.2.10 Definition 1.6 — ideal elements . . . 364
3.2.11 Definition 1.7 — principal filter . . . 365
3.2.12 Factoid 1.7 — principals are one-one; inclusion-reversing principals; union of principal filters . . . 366
3.2.13 Definition 1.8 — approximation and totality . . . 368
3.2.14 Factoid 1.8 — bottom element; bottom is least; finite filters are principal; prefix order; union of finite approximations . . . 369
3.2.15 Examples 1.2–1.5 (Revisited) — finite examples revisited . . . 374
3.2.16 Definition 1.9 — isomorphic domains . . . 380
3.2.17 Theorem 1.10 — basic open neighbourhoods . . . 381
3.2.18 Theorem 1.11 — intersections and directed unions . . . 389
3.2.19 Exercise 1.12 — final segments of integers . . . 393
3.2.20 Exercise 1.13 — infinite binary tree . . . 396
3.2.21 Exercise 1.14 — finite nonempty subsets . . . 398
3.2.22 Exercise 1.15 — finite elements without chains . . . 400
3.2.23 Exercise 1.16 — cofinite integer subsets . . . 404
3.2.24 Exercise 1.17 — rational open intervals . . . 408
3.2.25 Exercise 1.18 — consistency of neighbourhood families . . . 411
3.2.26 Exercise 1.19 — positive neighbourhood systems . . . 421
3.2.27 Exercise 1.20 — upward closed neighbourhoods . . . 428
3.2.28 Exercise 1.21 — complete neighbourhood systems . . . 435
3.2.29 Exercise 1.22 — topology of a domain . . . 441
3.2.30 Exercise 1.23 — decidable total elements . . . 443
3.2.31 Exercise 1.24 — extending partial elements . . . 449
3.2.32 Exercise 1.25 — final segments of ordinals . . . 450
3.2.33 Exercise 1.26 — ideals of a ring . . . 456
3.2.34 Exercise 1.27 — suprema of bounded sets . . . 461
3.3 Mappings between domains . . . 470
3.3.1 Introduction . . . 471
3.3.2 Definition 2.1 — approximable mapping . . . 471
3.3.3 Proposition 2.2 — elementwise action . . . 473
3.3.4 Example 2.3 — parity of leading zeros . . . 476
3.3.5 Example 2.4 — erase a run of ones . . . 482
3.3.6 Theorem 2.5 — category of approximable maps . . . 485
3.3.7 Proposition 2.6 — identity and composition . . . 488
3.3.8 Theorem 2.7 — isomorphisms are approximable . . . 489
3.3.9 Lecture II exercises . . . 493
3.3.10 Exercise 2.8 — determined by finite elements . . . 493
3.3.11 Exercise 2.9 — directed union formula . . . 495
3.3.12 Exercise 2.10 — intersection of two maps . . . 496
3.3.13 Exercise 2.11 — directed unions of elements . . . 497

3.3.14 Exercise 2.12 — directed unions of maps . . . 499
3.3.15 Exercise 2.13 — approximable means continuous . . . 500
3.3.16 Exercise 2.14 — isomorphisms of neighbourhoods . . . 508
3.3.17 Exercise 2.15 — opens classified by maps . . . 512
3.3.18 Exercise 2.16 — prefixing is approximable . . . 519
3.3.19 Exercise 2.17 — equations for erase-ones . . . 524
3.3.20 Exercise 2.17 — uniqueness of the erase-ones map . . . 526
3.3.21 Exercise 2.18 — doubling binary digits . . . 530
3.3.22 Exercise 2.19 — maps of two arguments . . . 538
3.3.23 Exercise 2.20 — powerset of the integers . . . 540
3.3.24 Exercise 2.21 — finite and infinite sequences . . . 547
3.3.25 Exercise 2.22 — domains from closure properties . . . 555
3.4 Domain constructs . . . 563
3.4.1 Introduction . . . 563
3.4.2 Definition 3.1 — product of systems . . . 563
3.4.3 Proposition 3.2 — elements of a product . . . 568
3.4.4 Definition 3.3 — projection mappings . . . 570
3.4.5 Proposition 3.4 — pairing and projections . . . 574
3.4.6 Theorem 3.5 — separate approximability . . . 576
3.4.7 Lemma 3.6 — constant functions . . . 578
3.4.8 Proposition 3.7 — closure under substitution . . . 580
3.4.9 Definition 3.8 — function space . . . 581
3.4.10 Proposition 3.9 — consistent step neighbourhoods . . . 586
3.4.11 Theorem 3.10 — elements are mappings . . . 589
3.4.12 Theorem 3.11 — evaluation combinator . . . 592
3.4.13 Theorem 3.12 — currying . . . 595
3.4.14 Theorem 3.13 — pointwise order of maps . . . 600
3.4.15 Exercise 3.14 — tagged product of tokens . . . 604
3.4.16 Exercise 3.15 — product isomorphisms . . . 606
3.4.17 Exercise 3.16 — infinite sequences of elements . . . 610
3.4.18 Exercise 3.17 — embedding binary sequences . . . 617
3.4.19 Exercise 3.18 — separated sum . . . 625
3.4.20 Exercise 3.19 — induced maps on constructs . . . 636
3.4.21 Exercise 3.20 — sum and product functors . . . 648
3.4.22 Exercise 3.21 — identity of step neighbourhoods . . . 650
3.4.23 Exercise 3.22 — composition combinator . . . 655
3.4.24 Exercise 3.23 — cartesian closed category . . . 660
3.4.25 Exercise 3.24 — function-space isomorphisms . . . 661
3.4.26 Exercise 3.25 — opens form a domain . . . 672
3.4.27 Exercise 3.26 — conditional combinator . . . 675
3.4.28 Exercise 3.27 — function space via closure . . . 700
3.4.29 Exercise 3.28 — minimal step function . . . 703
3.5 Fixed points and recursion . . . 706
3.5.1 Introduction . . . 706
3.5.2 Theorem 4.1 — least fixed point . . . 707

3.5.3 Theorem 4.2 — fix combinator . . . 710
3.5.4 Example 4.3 — partial functions on integers . . . 718
3.5.5 Example 4.4 — sequence domain . . . 726
3.5.6 Definition 4.5 — Peano axioms . . . 734
3.5.7 Theorem 4.6 — Peano models isomorphic . . . 737
3.5.8 Lecture IV exercises . . . 739
3.5.9 Exercise 4.7 — fixed point above a seed . . . 739
3.5.10 Exercise 4.8 — fixed-point induction . . . 744
3.5.11 Exercise 4.9 — fix as a fixed point . . . 748
3.5.12 Exercise 4.10 — domain above an element . . . 761
3.5.13 Exercise 4.11 — characterizing the fix combinator . . . 768
3.5.14 Exercise 4.12 — no greatest fixed point . . . 773
3.5.15 Exercise 4.13 — avoiding the natural numbers . . . 777
3.5.16 Exercise 4.14 — greatest powerset fixed point . . . 786
3.5.17 Exercise 4.15 — maximal fixed points . . . 788
3.5.18 Exercise 4.16 — optimal fixed point . . . 793
3.5.19 Exercise 4.17 — generated submonoid . . . 801
3.5.20 Exercise 4.18 — checking the number domain . . . 805
3.5.21 Exercise 4.19 — Peano axioms for sequences . . . 811
3.5.22 Exercise 4.20 — fix of a composite . . . 823
3.5.23 Exercise 4.21 — order as a fixed point . . . 824
3.5.24 Exercise 4.22 — inductive subset of a model . . . 830
3.5.25 Exercise 4.23 — commuting with the iterates . . . 833
3.5.26 Exercise 4.24 — Schroeder-Bernstein theorem . . . 835
3.5.27 Exercise 4.25 — unary sequence domain . . . 838
3.6 Typed λ-calculus . . . 844
3.6.1 Introduction . . . 844
3.6.2 Theorem 5.1 — terms denote approximable maps . . . 848
3.6.3 Theorem 5.2 — beta conversion . . . 850
3.6.4 Proposition 5.3 — simultaneous fixed points . . . 853
3.6.5 Proposition 5.4 — fixed point under a parameter . . . 856
3.6.6 Table 5.5 — table of combinators . . . 858
3.6.7 Theorem 5.6 — partial recursive functions . . . 864
3.6.8 Exercise 5.7 — currying the lambda notation . . . 896
3.6.9 Exercise 5.8 — combinatory completeness . . . 898
3.6.10 Exercise 5.9 — common fixed points . . . 902
3.6.11 Exercise 5.10 — smash product . . . 903
3.6.12 Exercise 5.11 — stack combinators . . . 920
3.6.13 Exercise 5.12 — while combinator . . . 924
3.6.14 Exercise 5.13 — pairing function on numbers . . . 927
3.6.15 Exercise 5.13 — powerset isomorphisms . . . 930
3.6.16 Exercise 5.14 — functions as graphs . . . 932
3.6.17 Exercise 5.15 — star of a word . . . 936
3.6.18 Exercise 5.16 — negation of sequences . . . 942
3.7 Domain equations . . . 963

3.7.1 Introduction . . . 963
3.7.2 Example 6.1 — indefinite product . . . 965
3.7.3 Example 6.2 — sequence domain equations . . . 972
3.7.4 Definition 6.3 — functor . . . 1007
3.7.5 Definition 6.4 — algebra for a functor . . . 1008
3.7.6 Definition 6.5 — initial algebra . . . 1010
3.7.7 Proposition 6.6 — uniqueness of initial algebras . . . 1012
3.7.8 Proposition 6.7 — functor preserves initiality . . . 1016
3.7.9 Definition 6.8 — continuity on maps . . . 1018
3.7.10 Theorem 6.9 — homomorphism from initial algebra . . . 1019
3.7.11 Definition 6.10 — subdomain . . . 1030
3.7.12 Proposition 6.11 — lattice of subsystems . . . 1034
3.7.13 Proposition 6.12 — projection pair . . . 1040
3.7.14 Definition 6.13 — continuity on domains . . . 1040
3.7.15 Theorem 6.14 — existence of initial algebras . . . 1049
3.7.16 Lemma 6.15 — subdomains from projection pairs . . . 1064
3.7.17 Theorem 6.16 — initial algebra is minimal . . . 1070
3.7.18 Lecture VI exercises . . . 1087
3.7.19 Exercise 6.17 — algebras for sequence domains . . . 1088
3.7.20 Exercise 6.18 — infinite product as initial . . . 1144
3.7.21 Exercise 6.19 — strict sum and product . . . 1153
3.7.22 Exercise 6.20 — tokens of a functor . . . 1168
3.7.23 Exercise 6.21 — coalesced sum and smash . . . 1177
3.7.24 Exercise 6.22 — equations for the integers . . . 1200
3.7.25 Exercise 6.23 — syntactic expression domain . . . 1223
3.7.26 Exercise 6.24 — recursive domain equations . . . 1231
3.7.27 Exercise 6.25 — projection pair laws . . . 1233
3.7.28 Exercise 6.26 — lifting a domain . . . 1252
3.7.29 Exercise 6.27 — which embeddings hold . . . 1275
3.7.30 Exercise 6.28 — finite embeddings cancel . . . 1295
3.7.31 Exercise 6.29 — infinitary sum and product . . . 1298
3.8 Computability in effectively given domains . . . 1320
3.8.1 Introduction . . . 1320
3.8.2 Definition 7.1 — computable presentation . . . 1322
3.8.3 Definition 7.2 — computable mapping . . . 1425
3.8.4 Proposition 7.3 — computable maps form a category . . . 1427
3.8.5 Theorem 7.4 — sum and product effective . . . 1429
3.8.6 Theorem 7.5 — function space effective . . . 1483
3.8.7 Theorem 7.6 — fix is computable . . . 1506
3.8.8 Proposition 7.7 — lists effectively given . . . 1539
3.8.9 Example 7.8 — powerset effectively given . . . 1555
3.8.10 Definition 7.9 — power domain . . . 1558
3.8.11 Proposition 7.10 — power domain effective . . . 1562
3.8.12 Definition 7.11 — finite sets of elements . . . 1572
3.8.13 Proposition 7.12 — finite sets are approximable . . . 1574

3.8.14 Lecture VII exercises . . . 1647
3.8.15 Exercise 7.13 — axiomatic neighbourhood structure . . . 1647
3.8.16 Exercise 7.14 — primitive recursive witnesses . . . 1656
3.8.17 Exercise 7.15 — further constructs effective . . . 1663
3.8.18 Exercise 7.16 — curry as an enumeration . . . 1702
3.8.19 Exercise 7.17 — recursion over lists . . . 1718
3.8.20 Exercise 7.18 — effective isomorphism . . . 1735
3.8.21 Exercise 7.19 — power domain functor . . . 1739
3.8.22 Exercise 7.20 — union on power domains . . . 1765
3.8.23 Exercise 7.21 — distributing the power domain . . . 1779
3.8.24 Exercise 7.22 — domain of formal languages . . . 1799
3.8.25 Exercise 7.23 — graph and fun computable . . . 1837
3.8.26 Exercise 7.24 — LUCID history domain . . . 1872
3.9 Retracts of the universal domain . . . 1921
3.9.1 Introduction . . . 1921
3.9.2 Definition 8.1 — retraction . . . 1921
3.9.3 Proposition 8.2 — retractions from subdomains . . . 1924
3.9.4 Definition 8.3 — projections . . . 1928
3.9.5 Examples 8.4 — check, fade, and smash . . . 1929
3.9.6 Theorem 8.5 — finitary projection test . . . 1931
3.9.7 Theorem 8.6 — sub combinator . . . 1934
3.9.8 Definition 8.7 — interval presentation of U . . . 1946
3.9.9 Theorem 8.8 — universality of U . . . 1950
3.9.10 Definition 8.9 — constructs on projections . . . 1956
3.9.11 Proposition 8.10 — constructs preserve projections . . . 1970
3.9.12 Lecture VIII exercises . . . 1976
3.9.13 Exercise 8.11 — retraction on intervals . . . 1976
3.9.14 Exercise 8.12 — integer presentation of U . . . 1987
3.9.15 Exercise 8.13 — Boolean-algebra filters . . . 2024
3.9.16 Exercise 8.14 — closure operators . . . 2032
3.9.17 Exercise 8.15 — domain of subsystems . . . 2041
3.9.18 Exercise 8.16 — order of projections . . . 2070
3.9.19 Exercise 8.17 — explicit embeddings into U . . . 2072
3.9.20 Exercise 8.18 — missing proofs of 8.10 . . . 2076
3.9.21 Exercise 8.19 — closure under sum and product . . . 2079
3.9.22 Exercise 8.20 — self-embeddings of a domain . . . 2085
3.9.23 Exercise 8.21 — lists of a projection . . . 2088
3.9.24 Exercise 8.22 — comparing B and C . . . 2102
3.9.25 Exercise 8.23 — initial algebra inside U . . . 2113
3.9.26 Exercise 8.24 — coupled domain equations . . . 2118
3.9.27 Exercise 8.25 — reflexive function space . . . 2127
3.9.28 Exercise 8.26 — untyped calculus in U . . . 2146
3.9.29 Exercise 8.27 — infinite cartesian products . . . 2149

4 Information Systems (1982) 2163
4.1 Introduction . . . 2163
4.2 Information systems . . . 2165
4.3 The elements of a system . . . 2174
4.4 Domains as lattices and as topological spaces . . . 2193
4.5 Approximable mappings between domains . . . 2237
4.6 Products and sums of domains . . . 2244
4.7 The function space as a domain . . . 2282
4.8 Some domain equations . . . 2306
4.8.1 Factoid 8.1 . . . 2306
4.8.2 Factoid 8.2 . . . 2358
4.8.3 Factoid 8.3 . . . 2497
4.8.4 Factoid 8.4 . . . 2532

5 Bridge Theorems 2552
5.1 Introduction . . . 2552
5.2 Catalog of bridge theorems . . . 2553
5.3 Proof notes . . . 2554
5.3.1 Continuous lattices → neighbourhood systems . . . 2554
5.3.2 Neighbourhood systems → information systems . . . 2556
5.3.3 Information systems → neighbourhood systems . . . 2556
5.3.4 Information systems → ideal completion . . . 2556
5.3.5 Algebraic complete lattices → continuous lattices . . . 2556
5.3.6 Three-presentation equivalence (`presentation_domains_equiv`) . . . 2557
5.3.7 Constructions (`infoSys_constructions_equiv`) . . . 2557
5.4 Constructivity summary . . . 2644
5.5 Worked example — S-expressions / trees . . . 2645
5.5.1 Overview . . . 2645
5.5.2 Information system (1982) . . . 2646
5.5.3 Neighbourhood system (1980) . . . 2646
5.5.4 Continuous lattice (1972) . . . 2646
5.5.5 Equivalence for this instance . . . 2647
5.5.6 Domain equation and morphisms . . . 2647
5.6 ContinuousLatticeToNeighborhood . . . 2687
5.7 InfoSysConstructions . . . 2687
5.8 InfoSysToIdealCompletion . . . 2687
5.9 InfoSysToNeighborhood . . . 2688
5.10 NeighborhoodToInfoSys . . . 2688
5.11 PresentationDomains . . . 2689

6 Acknowledgments 2690
6.1 AI-assisted development . . . 2690
6.2 Artifact availability and reproducibility . . . 2690

7 References 2691

List of Figures

1 Abstract. . . . 11
2 Continuous Lattices (1972). . . . 12
3 Injective spaces. . . . 14
4 Continuous lattices. . . . 21
5 Specialization. . . . 22
6 Scott Maps. . . . 31
7 Milner Correction. . . . 38
8 Function spaces. . . . 46
9 Inverse limits. . . . 67
10 Inverse limits. . . . 67
11 Inverse limits. . . . 69
12 Inverse limits. . . . 78
13 Inverse limits. . . . 86
14 Inverse limits. . . . 99
15 Inverse limits. . . . 102
16 Inverse limits. . . . 108
17 Datatypes as Lattices (1976). . . . 133
18 Continuous functions. . . . 136
19 Continuous. . . . 143
20 Graph. . . . 147
21 FixedPoint. . . . 148
22 Embedding. . . . 153
23 Historical note. . . . 158
24 Retracts and data types. . . . 272
25 Closure operations and algebraic lattices. . . . 298
26 Subsets and their classification. . . . 299
27 Total functions and functionality. . . . 301
28 Enumeration and degrees. . . . 314
29 Neighborhood Systems (1980). . . . 320
30 Domains given by neighbourhoods. . . . 323
31 Mappings between domains. . . . 470
32 Domain constructs. . . . 563
33 Fixed points and recursion. . . . 707
34 Typed λ-calculus. . . . 844
35 Domain equations. . . . 964
36 Computability in effectively given domains. . . . 1321
37 Retracts of the universal domain. . . . 1922
38 Information Systems (1982). . . . 2163
39 Information systems. . . . 2166
40 The elements of a system. . . . 2174
41 Domains as lattices and as topological spaces. . . . 2193
42 Approximable mappings between domains. . . . 2237
43 Products and sums of domains. . . . 2245

44 The function space as a domain. . . . 2282
45 Some domain equations. . . . 2306
46 Bridge Theorems. . . . 2552
47 Introduction. . . . 2553
48 Lean module map. . . . 2555
49 Worked example — S-expressions / trees. . . . 2645
50 ContinuousLatticeToNeighborhood. . . . 2687
51 InfoSysConstructions. . . . 2688
52 InfoSysToIdealCompletion. . . . 2688
53 InfoSysToNeighborhood. . . . 2688
54 NeighborhoodToInfoSys. . . . 2689
55 PresentationDomains. . . . 2689

Dependency diagram. The four papers stand on their own. The bridge theorems import continuous lattices, neighborhood systems, and information systems. Each box is one chapter. The arrowhead is on the chapter that imports. The number is how many direct imports. Datatypes as lattices and the references have no arrow.

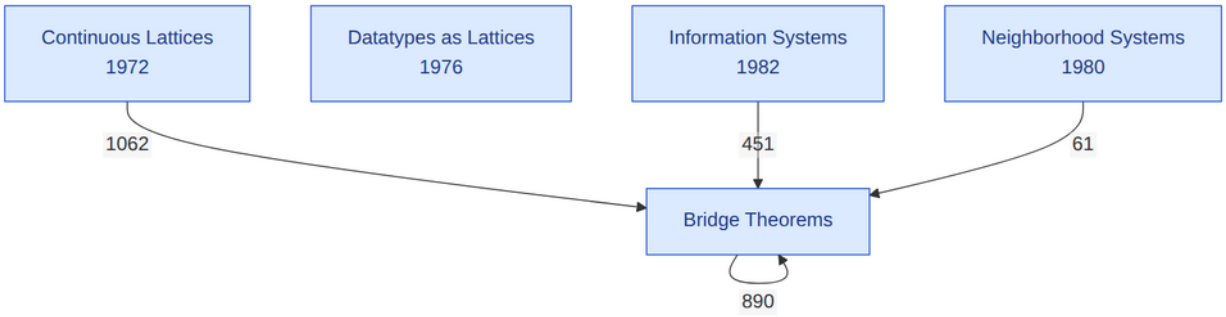


Figure 1: Abstract.

1 Continuous Lattices (1972)

Dependency diagram. Each box is one section. The arrowhead is on the section that imports. The number is how many direct imports. An arrow already implied by a longer path is left out. Section 0 has no arrow.

Figure 2: Continuous Lattices (1972).

1.1 Introduction

Starting from the topological point of view a certain wide class of T_0-spaces is introduced having a very strong extension property for continuous functions with values in these spaces. It is then shown that all such spaces are complete lattices whose lattice structure determines the topology - these are the *continuous lattices* - and every such lattice has the extension property. With this foundation the lattices are studied in detail with respect to projections, subspaces, embeddings, and constructions such as products, sums, function spaces, and inverse limits. The main result is a proof that every topological space can be embedded in a continuous lattice which is homeomorphic and isomorphic to its own function space. The function algebra of such spaces provides mathematical models for the Church-Curry λ-calculus.

Through a roundabout chain of mathematical events Scott became interested in T_0-spaces, those topological spaces satisfying the weakest separation axiom to the effect that two distinct points cannot share the same system of open neighborhoods. These spaces seem to have been originally suggested by Kolmogoroff and were introduced first in Alexandroff and Hopf (1935). Subsequent topology textbooks have dutifully recorded the definition but without much enthusiasm: mainly the idea is introduced to provide exercises. In the book Čech (1966) for example, T_0-spaces are called *feebly semi-separated spaces*, which surely is a term expressing mild contempt. Some interest has been shown in finite T_0-spaces (finite T_1-spaces are necessarily discrete), but generally topology seems to go better under at least the Hausdorff separation axiom. The reason for this is no doubt the strong motivation we get from geometry, where points *are* points and where distinct points *can* be separated.

We show from a less geometric point of view T_0-spaces can be not only interesting but also natural. The interest lies in the construction of *function spaces*, and the main result is the production of a large number of T_0-spaces D such that D and $[D \to D]$ are *homeomorphic*. Here $[D \to D]$ is the space of all continuous functions from D into D with the topology of pointwise convergence (the product topology). It will be shown that every space can be embedded in such a space D, and that D can be chosen to have quite strong extension properties for D-valued continuous functions. These properties make D most convenient for applications to logic and recursive function theory, which was Scott's original motivation. Some of the facts about these spaces seem to be most easily proved with the aid of some lattice theory, a circumstance that throws new light on the connections between topology and lattices. In fact, the required spaces are at the same time complete lattices whose topology is determined by a lattice structure which is continuous.

An announcement of this work and related investigations was first given in Scott (1970). Rather complete references and background material can be found in Scott (1971). A discussion of formal theories is to appear in Scott (1972).

The presentation of the material of the paper changed considerably after the January conference. In the first place remarks by several participants, Ernie Mannes in particular, caused Scott to rethink several points. Then the opportunity of lecturing at the Project MAC Seminar at MIT during the spring provided the opportunity of trying out some new ideas; these were then codified after lectures at the University of Southern California with the aid of several very helpful discussions on topology with James Dugundji.

The outcome of this development was that Scott found he could describe the work in purely topological terms in a simple and natural way leaving the lattices to be introduced as a special technique of analysis. This gives the presentation a much less ad hoc appearance, and relates the results to standard point-set topology in a much more understandable way. No doubt the whole idea of using completeness, inverse limits, and continuous functions could be put into a more general, more abstract categorical context.

For example, in understanding the connections of this kind of spaces with other topologies, one should consider the remarks on the topology of lattices in Birkhoff's paper in Abbott (1970). Some older papers such as Strother (1955) or Michael (1951) might also give some leads. It is curious how little there is of

interest in the literature on T_0-spaces. Concerning function spaces there ought to be some connections with the *limit spaces* of Cook and Fischer (see especially Binz and Keller (1966)) and possibly with the notion of *quasi-topology* of Spanier (1963), but these are rather vague ideas.

In a different direction note that the *algebraic lattices* of Grätzer (1968) are in fact continuous lattices in which isolated points are dense. The continuous lattices may be "higher dimensional" while algebraic lattices are "zero dimensional" - in some suitable sense. Every continuous lattice is a retract of an algebraic lattice. But does this mean anything?

1.2 Injective spaces

Dependencies. Inside Injective, each box is one Lean file. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. The arrowhead points at the file or folder that imports, and the number is how many direct imports.

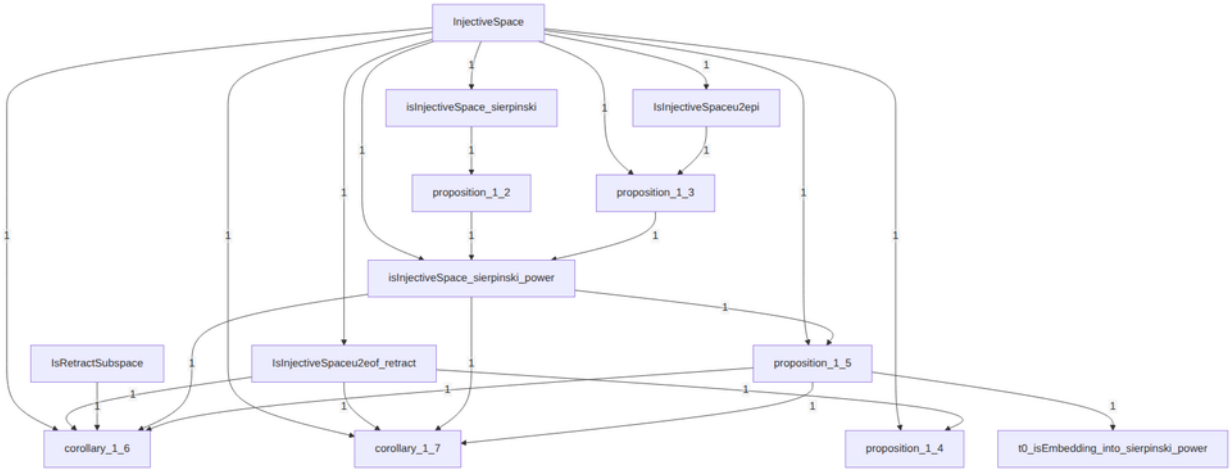


Figure 3: Injective spaces.

T_0 spaces and absolute retracts have no Lean files of their own.

1.2.1 T0 spaces

A T_0**-space** (Kolmogorov space) is a topological space in which two distinct points cannot have exactly the same system of open neighbourhoods: whenever $x \neq y$, some open set contains one of them but not the other. Equivalently, if every open neighbourhood of x is a neighbourhood of y and vice versa, then $x = y$. Scott's paper works throughout in this setting rather than assuming Hausdorff separation. Lean 4 Mathlib names this separation axiom with the typeclass `T0Space`. On a type `X` you already equip with `[TopologicalSpace X]`; `T0Space X` is the Prop-class stating that inseparable points are equal: for all `x y : X`, if `Inseparable x y` then `x = y`. The equivalence with the usual open-set formulation is `t0Space_iff_inseparable`. The class and `Inseparable` live in `Mathlib.Topology.Separation.Basic` and `Mathlib.Topology.Inseparable`. To **use** a T_0 space in a definition or theorem, add `[T0Space X]` next to `[TopologicalSpace X]` on your type variables—the pattern used in Definition 1.1 and throughout the library.

We begin by defining a class of T_0-spaces spaces to be called *injective*.

To **construct** a T_0 topology in this project:

- **Scott topology.** On a complete lattice `D`, the Scott topology is T_0. The proof is `scottTopology_t0Space`, linked from Specialization after the specialization order it uses.
- **Subspaces and embeddings.** If `f : X → Y` is an embedding and `Y` is T_0, then `X` with the subspace topology is T_0 (`Topology.IsEmbedding.t0Space`).
- **Sierpiński space.** `Prop` with the Sierpiński topology is T_0; powers and products built from it inherit T_0ness (see Proposition 1.2, where `Sierpinski` is defined and shown injective).
- **Inverse limits.** Towers of Scott spaces give a T_0 inverse-limit topology (`inverseLimit.t0Space` in the inverse-limits development—stated in that chapter, not re-proved here).

When no instance is available yet, prove `T0Space X` by applying the constructor for the class and showing `Inseparable x y → x = y`. The Scott-topology instance is the theorem `scottTopology_t0Space` in Specialization.

1.2.2 Definition 1.1: Injective Space

A T_0-space D is *injective* iff for arbitrary spaces X and Y if $X \subseteq Y$ as a subspace, then every continuous function $f : X \to D$ can be extended to a continuous function $\bar{f} : Y \to D$. As a diagram we have:

$$X \subseteq Y$$
$$X \xrightarrow{f} D$$
$$Y \xrightarrow{\bar{f}} D$$

Some people will object to this terminology because we use the subspace relationship rather than a monomorphism in the category of T_0-spaces and continuous maps. However, only the trivial 1-point space is injective in the sense of monomorphisms in that category, and so the notion is uninteresting. As we shall see these spaces have very strong retraction properties.

Injective space. `InjectiveSpace` is this definition: every continuous map into the space extends along every topological embedding. Proposition 2.11 proves a continuous lattice is injective, and Theorem 2.12 proves the two notions coincide.

`InjectiveSpace.lean`

```
module

public import Mathlib.Topology.ContinuousMap.Defs
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Order.CompleteLattice.Basic

@[expose] public section

open Topology TopologicalSpace

universe u v

class InjectiveSpace (D : Type v) [TopologicalSpace D] : Prop where
```

```
  /-- Every continuous map into `D` extends along every topological embedding. -/
  extension :
    forall  {X Y : Type u} [TopologicalSpace X] [TopologicalSpace Y] (e : X  ->  Y),
      IsEmbedding e  ->  forall  f : C(X, D), exists  g : C(Y, D), forall  x, g (e x)
    = f x
```

1.2.3 Proposition 1.2

A slightly less trivial example of an injective space is the 2-point space $\mathbb{O}$ with "points" $\bot$ and $\top$ where $\top$ is open but $\bot$ is not. (This space is sometimes called the *Sierpiński Space*.)

In Lean, **`Sierpinski`** is that space: `Prop` with the Sierpiński topology (`abbrev Sierpinski := Prop`).

1.2 Proposition. The space $\mathbb{O}$ is injective.

Proof: As is obvious, the continuous maps $f : X \to \mathbb{O}$ are in a one-one correspondence with the open subsets of X (consider $f^{-1}(\top)$). If $X \subseteq Y$ as a subspace, then an open subset of X is the restriction of some open subset of Y. Thus any $f : X \to \mathbb{O}$ can be extended to $\bar{f} : Y \to \mathbb{O}$. □

`isInjectiveSpace_sierpinski.lean`

```
module

public import DomainTheory.ContinuousLattice.Injective.InjectiveSpace
public import Mathlib.Topology.Order
import Mathlib.Topology.Maps.Basic

@[expose] public section

open Topology TopologicalSpace

/-- Scott's two-point Sierpinski space  O : `Prop` with the Sierpinski topology. -/
abbrev Sierpinski := Prop

/-- **Scott 1972, Proposition 1.2.** The Sierpinski space is injective. -/
theorem isInjectiveSpace_sierpinski : InjectiveSpace.{u} Sierpinski := by
  constructor
  intro X Y _ _ e he f
  have hfopen : IsOpen {x | f x} := continuous_Prop.1 f.continuous
  rw [he.isInducing.isOpen_iff] at hfopen
  obtain <V, hVopen, hVeq> := hfopen
  refine <<fun y => y  in  V, continuous_Prop.2 hVopen>, fun x => ?_>
  have : (e x  in  V) = (x  in  {x | f x}) := by rw [ <-  hVeq]; rfl
  simpa using this
```

1.2.4 Proposition 1.3

1.3 Proposition. The Cartesian product of any number of injective spaces is injective under the product topology.

Proof: The argument is standard. A map into the product can be projected onto each of the factors. Each of these projections can be extended. Then the separate maps can be put together again to make the required extended map into the product. □

IsInjectiveSpaceu2epi.lean

```
module

public import DomainTheory.ContinuousLattice.Injective.InjectiveSpace
public import Mathlib.Topology.Constructions

@[expose] public section

open Topology TopologicalSpace

universe v w

/-- **Scott 1972, Proposition 1.3.** Products of injective spaces are injective. -/
theorem InjectiveSpace.pi {? : Type w} (D : ?  ->  Type v) [forall  i,
    TopologicalSpace (D i)]
    (h : forall  i, InjectiveSpace.{u} (D i)) :     InjectiveSpace.{u} (forall  i, D i
    ) := by
  constructor
  intro X Y _ _ e he f
  choose g hg using fun i =>
    (h i).extension e he <fun x => f x i, (continuous_apply i).comp f.continuous>
  refine <<fun y i => g i y, continuous_pi fun i => (g i).continuous>, fun x => ?_>
  funext i
  exact hg i x
```

1.2.5 Proposition 1.4

We now have a large number of injective spaces, and further examples could be found using the next fact.

1.4 Proposition. A retract of an injective space is injective.

Proof: Let D be injective. By a retract of D we understand a subspace $D' \subseteq D$ for which there exists a retraction map $j : D \to D'$ such that

$$D' = x \in D : j(x) = x.$$

Then if $f : X \to D'$ and $X \subseteq Y$, we have $f : X \to D$ as a continuous map also. Taking $\bar{f} : Y \to D$, we have only to form

$$j \circ \bar{f} : Y \to D'$$

to show that D' is injective. □

Retract. `IsRetractSubspace` is a subspace with a continuous retraction, and this proposition uses it to show a retract of an injective space is injective. The continuous-lattice form of the same idea is Definition 3.6. The representation by an idempotent on a power of $\mathbb{O}$ is Theorem 2.12.

IsInjectiveSpaceu2eof_retract.lean

```
module

public import DomainTheory.ContinuousLattice.Injective.InjectiveSpace
import Mathlib.Topology.ContinuousMap.Basic

@[expose] public section
```

```
open Topology TopologicalSpace

universe v

/-- **Scott 1972, Proposition 1.4.** Retracts of injective spaces are injective. -/
theorem InjectiveSpace.of_retract {D : Type v} {D' : Type v}
    [TopologicalSpace D] [TopologicalSpace D'] (hD : InjectiveSpace.{u} D)
    (s : C(D', D)) (r : C(D, D')) (hrs : forall  d, r (s d) = d) :
    InjectiveSpace.{u} D' := by
  constructor
  intro X Y _ _ e he f
  obtain <g, hg> := hD.extension e he (s.comp f)
  refine <r.comp g, fun x => ?_>
  show r (g (e x)) = f x
  rw [hg x]
  exact hrs (f x)
```

1.2.6 Proposition 1.5

The relationship between arbitrary T_0-spaces and the injective spaces is given by the embedding theorem.

1.5 Proposition. Every T_0-space can be embedded in an injective space; in fact, in a Cartesian power of the 2-element space $\mathbb{O}$.

Proof: The proof is well known (cf. Čech (1966), Theorem 26B.9, p. 484.) But we give the argument for completeness sake. Let X be the given space, and let $\mathfrak{O}$ be the class of open subsets of X. Let

$$D = \mathbb{O}^{\mathfrak{O}}$$

be the Cartesian power of $\mathbb{O}$. Then D is injective by 1.3. Define the map $e : X \to D$ by:

$$e(x)(U) = \begin{cases} \top & \text{if } x \in U, \\ \bot & \text{if } x \notin U, \end{cases}$$

for $x \in X$ and $U \in \mathfrak{O}$. This map e is continuous in view of the topology given to $\mathbb{O}$ and to D. The map e is one-one, because X is T_0. Finally, if $U \subseteq X$ is open, then

$$\begin{aligned} e(U) &= \{e(x) : x \in U\} \\ &= \{e(x) : e(x)(U) = \top\} \\ &= e(X) \cap \{t \in D : t(U) = \top\}, \end{aligned}$$

which shows that the image $e(U)$ is open in the subspace $e(X) \subseteq D$. Therefore $e : X \to D$ is an embedding of X as a subspace in D. □

`isInjectiveSpace_sierpinski_power.lean`

```
module

public import DomainTheory.ContinuousLattice.Injective.InjectiveSpace
public import Mathlib.Topology.Constructions
import DomainTheory.ContinuousLattice.Injective.proposition_1_2
import DomainTheory.ContinuousLattice.Injective.proposition_1_3
```

```
@[expose] public section

open Topology TopologicalSpace

universe w

theorem isInjectiveSpace_sierpinski_power (? : Type w) :
    InjectiveSpace.{u} (?  ->  Prop) :=
  proposition_1_3 (fun (_ : ?) => Prop) (fun _ => proposition_1_2)
```

1.2.7 Corollary 1.6

1.6 Corollary. The injective spaces are exactly the retracts of the Cartesian powers of $\mathbb{O}$.

Proof: Such a retract is injective by 1.4. If D is injective, then it is (homeomorphic to) a subspace of a power of $\mathbb{O}$. But since D is injective the identity function on the subspace to itself can be extended to the whole of the power of $\mathbb{O}$ providing the required retraction. □

corollary_1_6.lean

```
module

public import DomainTheory.ContinuousLattice.Injective.IsRetractSubspace
public import DomainTheory.ContinuousLattice.Injective.InjectiveSpace
public import Mathlib.Topology.Separation.Basic
import DomainTheory.ContinuousLattice.Injective.IsInjectiveSpaceu2eof_retract
import DomainTheory.ContinuousLattice.Injective.isInjectiveSpace_sierpinski_power
import DomainTheory.ContinuousLattice.Injective.proposition_1_5

@[expose] public section

open Topology TopologicalSpace

universe u

/-- **Scott 1972, Corollary 1.6.** Injective spaces are exactly retracts of powers of
     O . -/
theorem corollary_1_6 (D : Type u) [TopologicalSpace D] [T0Space D] :
    InjectiveSpace.{u,u} D  <->  exists  (? : Type u), Nonempty (IsRetractSubspace D
    (?  ->  Prop)) := by
  constructor
  * intro hD
    obtain <r, hr> := hD.extension (productOfMemOpens D) (proposition_1_5 D).2 (
    ContinuousMap.id D)
    refine <Opens D, <<productOfMemOpens D, (proposition_1_5 D).2, r, hr>>>
  * rintro <?, <R>>
    exact InjectiveSpace.of_retract (isInjectiveSpace_sierpinski_power ?) R.section' R
    .retraction
      R.retraction_section
```

1.2.8 Corollary 1.7

1.7 Corollary. A space is injective iff it is a retract of every space of which it is a subspace.

Proof: As in the proof of 1.6, this property is obvious for injective spaces. But in view of 1.5 every such space is a retract of a power of $\mathbb{O}$ and hence is injective. □

corollary_1_7.lean

```
module

public import DomainTheory.ContinuousLattice.Injective.InjectiveSpace
public import Mathlib.Topology.Separation.Basic
import DomainTheory.ContinuousLattice.Injective.IsInjectiveSpaceu2eof_retract
import DomainTheory.ContinuousLattice.Injective.isInjectiveSpace_sierpinski_power
import DomainTheory.ContinuousLattice.Injective.proposition_1_5

@[expose] public section

open Topology TopologicalSpace

universe u

/-- **Scott 1972, Corollary 1.7.** A space is injective iff it is a retract of every
super-space of which it is a subspace. -/
theorem corollary_1_7 (D : Type u) [TopologicalSpace D] [T0Space D] :
    InjectiveSpace.{u,u} D  <->
      forall  {Y : Type u} [TopologicalSpace Y] (e : D  ->  Y), IsEmbedding e  ->
        exists  r : C(Y, D), forall  d, r (e d) = d := by
  constructor
  * intro hD Y _ e he
    obtain <r, hr> := hD.extension e he (ContinuousMap.id D)
    exact <r, hr>
  * intro hD
    obtain <r, hr> := hD (productOfMemOpens D) (proposition_1_5 D).2
    exact InjectiveSpace.of_retract (isInjectiveSpace_sierpinski_power (Opens D))
      (productOfMemOpens D) r hr
```

1.2.9 Absolute retracts

As a result of these very elementary considerations, the injective spaces could be called *absolute retracts*, if one remembers to modify the standard definitions by using *arbitrary* subspaces rather than just *closed* subspaces. Note too that it is easy to show that the only continuous maps $e : X \to Y$ for which the extension property

$$X \xrightarrow{e} YX \xrightarrow{f} \mathbb{O}Y \xrightarrow{\bar{f}} \mathbb{O}$$

could hold for *all* continuous $f : X \to \mathbb{O}$ are embeddings as subspaces.

Thus it would seem that we have a reasonably good initial grasp of the notion of injective spaces, but further constructions are considerably facilitated by the introduction of the lattice structure.

1.3 Continuous lattices

The section follows the paper from specialization through Proposition 2.11. The printed proof of Proposition 2.10 is flawed. Its corrected proof and Theorem 2.12, which calls that proof, are in section 3 with their original numbers.

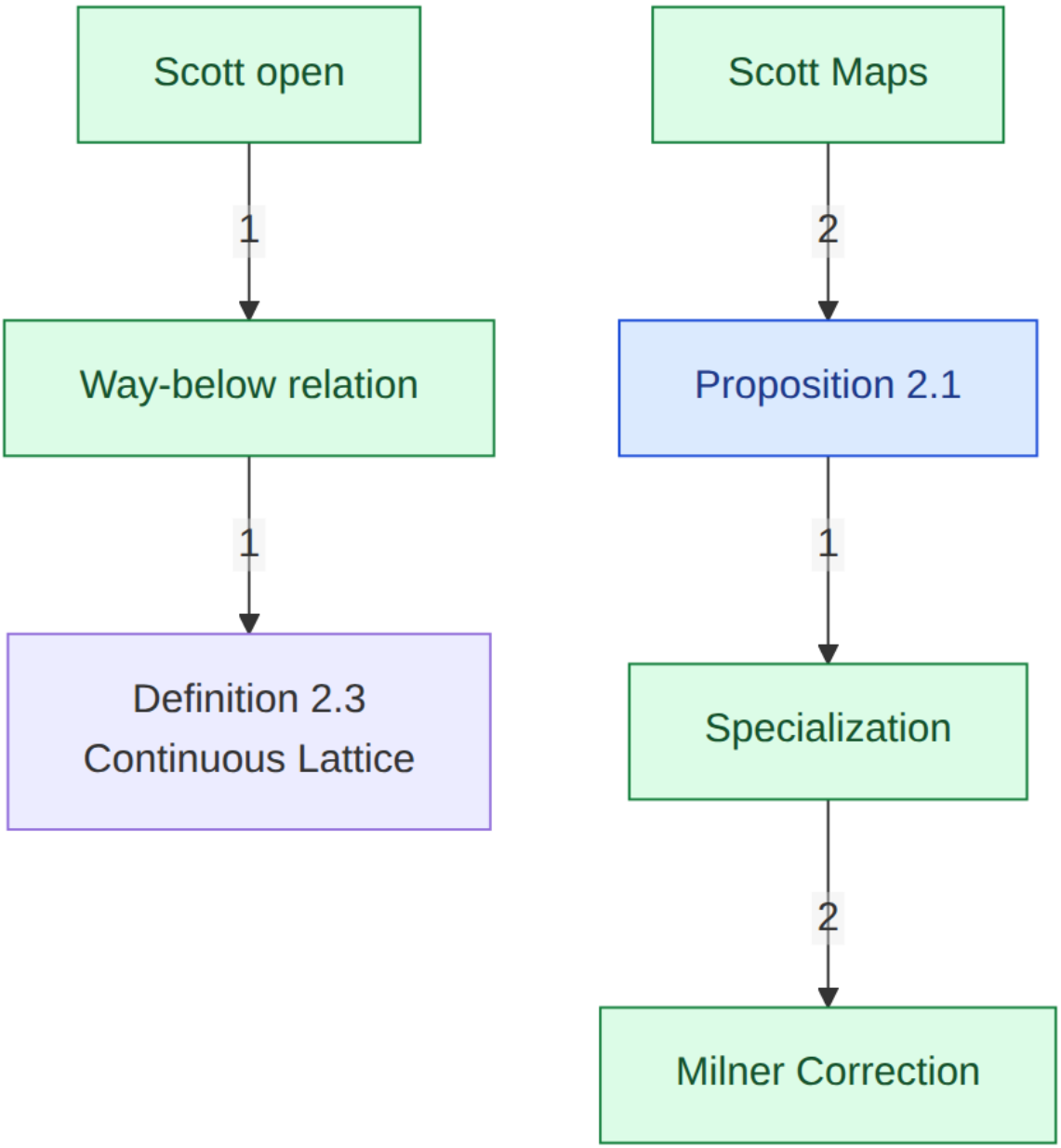


Figure 4: Continuous lattices.

The arrowhead is on the note that imports. The number is how many direct imports. An arrow already implied by a longer path is left out. The flawed proof has no Lean files, so it is not on the diagram. Theorem 2.12 is on the section 3 diagram.

1.3.1 Specialization

Dependencies. Inside Specialization, each box is one Lean file. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. The arrowhead points at the file or folder that imports, and the number is how many direct imports.

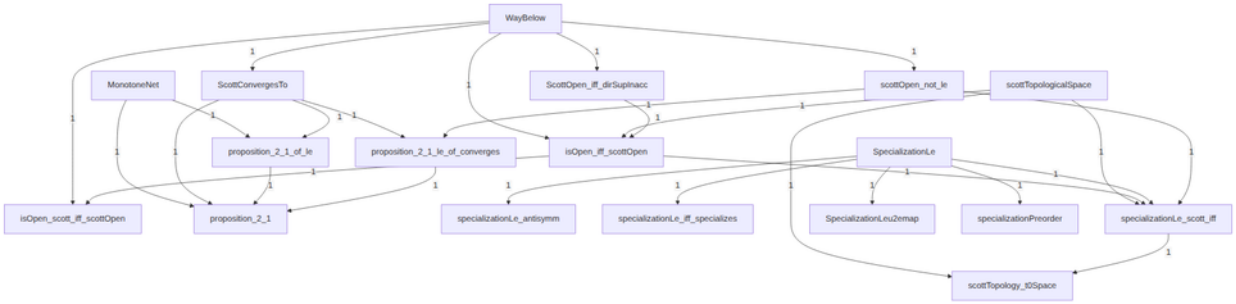


Figure 5: Specialization.

Specialization Scott assumes the T_0 separation axiom throughout; for what that means in the paper and in Lean, see T0 spaces. Here we use T_0 only as background for the specialization order and Scott topology.

Every T_0-space becomes a partially ordered set under the definition:

$$x \sqsubseteq y \text{ iff whenever } x \in U \text{ and } U \text{ is open, then } y \in U.$$

Indeed, though this relation is reflexive and transitive, the condition that it be *antisymmetric* is exactly equivalent to the T_0-axiom (formalized in Lean as `specializationLe_antisymm`).

In the converse direction, every partially ordered set $\langle X, \sqsubseteq \rangle$ can be so obtained, for we have only to define $U \subseteq X$ as being open if it satisfies the condition:

(i) whenever $x \in U$ and $x \sqsubseteq y$, then $y \in U$.

The axioms for partial order make X a T_0-space, because for any $y \in X$ the set

$$x \in X : x \not\sqsubseteq y$$

is open. This connection is not very interesting, however.

What *is* interesting in topological spaces is *convergence* and the properties of *limit points*. We shall discuss limits in terms of *nets*, in particular in terms of *monotone nets*. A monotone net in a T_0-space X is a function

$$x : I \to X$$

where $\langle I, \leq \rangle$ is a directed set and where $i \leq j$ implies $x_i \sqsubseteq x_j$ for all $i, j \in I$. In a T_1-space a monotone net is constant (hence, uninteresting) because the $\sqsubseteq$-relation is the identity. As usual (cf. Kelley (1955),

p.66) we say that a net x *converges* to an element y iff whenever U is open and $y \in U$, then for some $i \in I$ we have $x_j \in U$ for all $j \geq i$. Note that a monotone net x converges to each of its terms x_i. Suppose that a monotone net x converges to an element y which is an upper bound to all the terms of x. Then y must be the *least upper bound*, which we write as:

$$y = \bigsqcup x_i : i \in I.$$

To see this, assume that z is any other upper bound with $x_i \sqsubseteq z$ for all $i \in I$. If U is open and $y \in U$, then $x_i \in U$ for some $i \in I$. But then $z \in U$, and so $y \sqsubseteq z$ follows.

We shall find that most of the facts about the topology of the spaces we are concerned with here can be expressed in terms of least upper bounds (lubs). It is not always the case, however, that lubs are limits. Thus, for a partially ordered set X, we impose a further restriction on its topology beyond condition (i) for saying when a subset U is open:

(ii) whenever $S \subseteq X$ is *directed*, $\bigsqcup S$ exists, and $\bigsqcup S \in U$, then $S \cap U \neq \emptyset$.

By a directed subset of X we of course mean that it is directed in the sense of the partial ordering $\sqsubseteq$. Note that in this paper directed sets are always *non-empty*. The sets satisfying (i) and (ii) form the *induced topology* on a partially ordered set X, which is still a T_0-space because the sets $x \in X : x \not\sqsubseteq y$ remain open even in the sense of (ii). Obviously a directed set $S \subseteq X$ can be regarded as a net, and now in view of (ii) it follows that S converges to $\bigsqcup S$ — if this lub exists. The predicate is Scott open.

SpecializationLe.lean

```
module

public import Mathlib.Topology.Defs.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

universe u

variable {X : Type*} {D : Type u} [TopologicalSpace X] [CompleteLattice D]

/-- **Scott 1972, S2.** The *specialization order*: `x  sqsub  y` when `x  in  U` open
    implies `y  in  U`. -/
def SpecializationLe (x y : X) : Prop :=
  forall  U, IsOpen U  ->  x  in  U  ->  y  in  U

end DomainTheory.ContinuousLattice
```

Proposition 2.1 **2.1 Proposition.** *In a partially ordered set X with the induced topology, a monotone net $x : I \to X$ with a least upper bound converges to an element $y \in X$ iff*

$$y \sqsubseteq \bigsqcup x_i : i \in I.$$

□

proposition_2_1_le_of_converges.lean

```
module

public import DomainTheory.ContinuousLattice.Specialization.ScottConvergesTo
public import DomainTheory.ContinuousLattice.Specialization.scottOpen_not_le
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

universe u

variable {X : Type*} {D : Type u} [TopologicalSpace X] [CompleteLattice D]

variable {? : Type u} [Preorder ?] [IsDirected ? (*  <=  *)]

variable {x : ?  ->  D} {L y : D}

omit [IsDirected ? (*  <=  *)] in
/-- **Scott 1972, Proposition 2.1 (forward).** If a monotone net converges to `y` in
    the Scott
topology and `L` is its least upper bound, then `y  <=  L`. -/
theorem proposition_2_1_le_of_converges (hL : IsLUB (range x) L)
    (hconv : ScottConvergesTo x y) : y  <=  L := by
  by_contra hyL
  obtain <i, hi> := hconv {z : D | not  z  <=  L} (scottOpen_not_le L) hyL
  exact hi i le_rfl (hL.1 <i, rfl>)

end DomainTheory.ContinuousLattice
```

1.3.2 Scott open

For a partially ordered set X, a subset $U \subseteq X$ is open in the *induced topology* when it satisfies:

(i) whenever $x \in U$ and $x \sqsubseteq y$, then $y \in U$;

(ii) whenever $S \subseteq X$ is *directed*, $\bigsqcup S$ exists, and $\bigsqcup S \in U$, then $S \cap U \neq \emptyset$.

Directed sets are non-empty. The sets satisfying (i) and (ii) are the open sets of the induced topology.

`ScottOpen` is that predicate on a complete lattice. `ScottOpen.isUpperSet` is (i). `ScottOpen.univ`, `ScottOpen.inter`, and `ScottOpen.sUnion` are the topology laws: the universe is open, binary intersections are open, and arbitrary unions are open.

The discussion that leads to (i) and (ii) is Specialization.

ScottOpen.lean

```
module

public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

universe u

variable {D : Type u} [CompleteLattice D]

/-- **Scott 1972, S2, the induced topology.** `U` is *Scott-open* when it is an upper
    set and
is inaccessible by suprema of non-empty directed sets: if a non-empty directed `S` has
    its
supremum in `U`, then some member of `S` already lies in `U`. -/
def ScottOpen (U : Set D) : Prop :=
  letI : LE D :=
    (CompleteLattice.toCompleteSemilatticeInf (a := D)).toPartialOrder.toPreorder.toLE
  letI : SupSet D :=
    (CompleteLattice.toCompleteSemilatticeSup (a := D)).toSupSet
  IsUpperSet U  /\
    forall  ?S : Set D?, S.Nonempty  ->  DirectedOn (*  <=  *) S  ->  sSup S  in  U
    ->  (S  I  U).Nonempty

theorem ScottOpen.inter {U V : Set D} (hU : ScottOpen U) (hV : ScottOpen V) :
    ScottOpen (U  I  V) := by
  refine <hU.1.inter hV.1, fun S hS hSdir hmem => ?_>
  obtain <s_1, hs_1S, hs_1U> := hU.2 hS hSdir hmem.1
  obtain <s_2, hs_2S, hs_2V> := hV.2 hS hSdir hmem.2
  obtain <s_3, hs_3S, h_1, h_2> := hSdir s_1 hs_1S s_2 hs_2S
  exact <s_3, hs_3S, hU.1 h_1 hs_1U, hV.1 h_2 hs_2V>

theorem ScottOpen.univ : ScottOpen (Set.univ : Set D) := by
  refine <isUpperSet_univ, fun S hS _ _ => ?_>
  obtain <s, hs> := hS
  exact <s, hs, Set.mem_univ s>

theorem ScottOpen.sUnion {C : Set (Set D)} (hC : forall  U  in  C, ScottOpen U) :
    ScottOpen (?_0 C) := by
  refine <isUpperSet_sUnion fun U hU => (hC U hU).1, fun S hS hSdir hmem => ?_>
  obtain <U, hUC, hUmem> := hmem
  obtain <s, hsS, hsU> := (hC U hUC).2 hS hSdir hUmem
  exact <s, hsS, U, hUC, hsU>

end DomainTheory.ContinuousLattice
```

(Already included: `1972_Proposition_2_1.md`.)

1.3.3 Way-below relation

Our main interest will lie with those partially ordered sets in which every subset has a lub: namely, *complete lattices*. We use Mathlib's `CompleteLattice` typeclass to express that "lattices are complete".

If D is such a space we write $\bot = \bigsqcup \emptyset$ and $\top = \bigsqcup D$ for the smallest and largest elements (read: bottom and top). As is well known, greatest lower bounds must exist, for:

$$\sqcap S = \bigsqcup x \in D : x \sqsubseteq y \text{ for all } y \in S$$

gives the definition.

Given a complete lattice D we define

$$x \ll y \text{ iff } y \in \text{Int} z \in D : x \sqsubseteq z,$$

where the interior is taken in the sense of the induced topology. The relation $x \ll y$ behaves somewhat like a strict ordering relation; at least its meaning is clearly that y should be definitely larger than x in the partial ordering. Such a relation has many pleasant properties.

The primary purpose of introducing it is to provide a simple definition for the kind of spaces that are most useful to us. We first mention the most elementary features of this relation in Proposition 2.2.

`WayBelow.lean`

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

universe u

variable {D : Type u} [CompleteLattice D]

def WayBelow (x y : D) : Prop :=
  letI : LE D :=
    (CompleteLattice.toCompleteSemilatticeInf (a := D)).toPartialOrder.toPreorder.toLE
  letI : Preorder D :=
    (CompleteLattice.toCompleteSemilatticeInf (a := D)).toPartialOrder.toPreorder
  exists  U : Set D, ScottOpen U  /\  y  in  U  /\  U  subseteq  Set.Ici x

@[inherit_doc] scoped infix:50 "  <<  " => WayBelow

/-- **Scott 1972, Proposition 2.2(i).** `False  <<  x` for every `x`. -/
theorem bot_wayBelow (x : D) : (False : D)  <<  x :=
  <Set.univ, ScottOpen.univ, Set.mem_univ x, fun _ _ => bot_le>

/-- **Scott 1972, Proposition 2.2(ii).** `x  <<  z` and `y  <<  z` imply `x  sqsup  y
    <<  z`. -/
theorem WayBelow.sup {x y z : D} (hx : x  <<  z) (hy : y  <<  z) : x  sqsup  y  <<  z
    := by
  obtain <U, hU, hzU, hUsub> := hx
  obtain <V, hV, hzV, hVsub> := hy
```

```
  refine <U  I  V, hU.inter hV, <hzU, hzV>, fun w hw => ?_>
  exact Set.mem_Ici.2 (sup_le (hUsub hw.1) (hVsub hw.2))

/-- **Scott 1972, Proposition 2.2(iii).** `x  <<  y` and `y  <=  z` imply `x  <<  z` (
    monotone on the
right). -/
theorem WayBelow.trans_le {x y z : D} (h : x  <<  y) (hyz : y  <=  z) : x  <<  z := by
  obtain <U, hU, hyU, hsub> := h
  exact <U, hU, hU.1 hyz hyU, hsub>

/-- **Scott 1972, Proposition 2.2(iv).** `x  <=  y` and `y  <<  z` imply `x  <<  z` (
    monotone on the
left). -/
theorem WayBelow.le_trans {x y z : D} (hxy : x  <=  y) (h : y  <<  z) : x  <<  z := by
  obtain <U, hU, hzU, hsub> := h
  refine <U, hU, hzU, fun w hw => ?_>
  have hyw : y  <=  w := Set.mem_Ici.1 (hsub hw)
  exact Set.mem_Ici.2 (hxy.trans hyw)

/-- **Scott 1972, Proposition 2.2(v).** `x  <<  y` implies `x  <=  y`. -/
theorem WayBelow.le {x y : D} (h : x  <<  y) : x  <=  y := by
  obtain <U, _, hyU, hsub> := h
  exact hsub hyU

theorem WayBelow.self_iff_scottOpen_Ici {x : D} : x  <<  x  <->  ScottOpen (Set.Ici x)
     := by
  constructor
  * rintro <U, hU, hxU, hsub>
    -- `Ici x = U`: `Ici x  subseteq  U` since `U` is upper and `x  in  U`; `U  subseteq
      Ici x` is `hsub`.
    have hIci : Set.Ici x = U :=
      le_antisymm (fun w hw => hU.1 (Set.mem_Ici.1 hw) hxU) hsub
    rw [hIci]; exact hU
  * intro hopen
    exact <Set.Ici x, hopen, Set.self_mem_Ici, le_refl _>

theorem WayBelow.sSup_iff {x : D} {S : Set D} (hS : S.Nonempty)
    (hSdir : DirectedOn (*  <=  *) S) : x  <<  sSup S  <->  exists  y  in  S, x  <<  y
     := by
  constructor
  * rintro <U, hU, hsupU, hsub>
    obtain <s, hsS, hsU> := hU.2 hS hSdir hsupU
    exact <s, hsS, U, hU, hsU, hsub>
  * rintro <y, hyS, hxy>
    obtain <U, hU, hyU, hsub> := hxy
    exact <U, hU, hU.1 (le_sSup hyS) hyU, hsub>

end DomainTheory.ContinuousLattice
```

1.3.4 Proposition 2.2

2.2 Proposition. In a complete lattice D we have:

(i) $\bot \ll x$;

(ii) $x \ll z$ and $y \ll z$ imply $x \sqcup y \ll z$;

(iii) $x \ll y \sqsubseteq z$ implies $x \ll z$;

(iv) $x \sqsubseteq y \ll z$ implies $x \ll z$;

(v) $x \ll y$ implies $x \sqsubseteq y$;

(vi) $x \ll x$ iff $z \in D : x \sqsubseteq z$ is open;

(vii) if $S \subseteq D$ is directed, then $x \ll \bigsqcup S$ iff $x \ll y$ for some $y \in S$. □

The proofs of these statements can be safely left to the reader.

The relation is the way-below relation. `proposition_2_2` is this statement. Its proof calls `bot_wayBelow`, `WayBelow.sup`, `WayBelow.trans_le`, `WayBelow.le_trans`, `WayBelow.le`, `WayBelow.self_iff_scottOpen_Ici`, and `WayBelow.sSup_iff`, which are in `WayBelow.lean`.

`proposition_2_2.lean`

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow

@[expose] public section

namespace DomainTheory.ContinuousLattice

universe u

variable {D : Type u} [CompleteLattice D]

/-- **Scott 1972, Proposition 2.2.** In a complete lattice:

(i) `False  <<  x`;

(ii) `x  <<  z` and `y  <<  z` imply `x  sqsup  y  <<  z`;

(iii) `x  <<  y` and `y  <=  z` imply `x  <<  z`;

(iv) `x  <=  y` and `y  <<  z` imply `x  <<  z`;

(v) `x  <<  y` implies `x  <=  y`;

(vi) `x  <<  x` iff `Set.Ici x` is Scott-open;

(vii) for a nonempty directed `S`, `x  <<  sSup S` iff `x  <<  y` for some `y  in  S`.

Proved from `bot_wayBelow`, `WayBelow.sup`, `WayBelow.trans_le`, `WayBelow.le_trans`,
`WayBelow.le`, `WayBelow.self_iff_scottOpen_Ici`, and `WayBelow.sSup_iff`. -/
theorem proposition_2_2 :
    (forall  x : D, (False : D)  <<  x)  /\
    (forall  x y z : D, x  <<  z  ->  y  <<  z  ->  x  sqsup  y  <<  z)  /\
    (forall  x y z : D, x  <<  y  ->  y  <=  z  ->  x  <<  z)  /\
    (forall  x y z : D, x  <=  y  ->  y  <<  z  ->  x  <<  z)  /\
    (forall  x y : D, x  <<  y  ->  x  <=  y)  /\
```

```
    (forall  x : D, x  <<  x  <->  ScottOpen (Set.Ici x))  /\
    (forall  (x : D) (S : Set D), S.Nonempty  ->  DirectedOn (*  <=  *) S  ->
      (x  <<  sSup S  <->  exists  y  in  S, x  <<  y)) :=
  <bot_wayBelow,
    fun _ _ _ hx hy => WayBelow.sup hx hy,
    fun _ _ _ h hyz => WayBelow.trans_le h hyz,
    fun _ _ _ hxy h => WayBelow.le_trans hxy h,
    fun _ _ h => WayBelow.le h,
    fun _ => WayBelow.self_iff_scottOpen_Ici,
    fun _ _ hS hdir => WayBelow.sSup_iff hS hdir>

end DomainTheory.ContinuousLattice
```

1.3.5 Definition 2.3

Definition 2.3. A *continuous lattice* is a complete lattice D in which for every $y \in D$ we have:

$$y = \bigsqcup x \in D : x \ll y.$$

Every element is the least upper bound of the elements way below it.

Interpolation property of $\ll$. WayBelow.interpolate. In a continuous lattice, the way-below relation interpolates: `a ≪ c` implies there is some `b` with `a ≪ b ≪ c`.

The proof runs Scott's standard argument: the set `M = {m | ∃ x, m ≪ x ∧ x ≪ c}` is directed (using directedness of `{. ≪ x}` twice) and has supremum `c` (using continuity twice). Hence `a ≪ c = ⊔M` with `M` directed forces `a ≪ m` for some `m ∈ M`, say `m ≪ x ≪ c`; then `a ≪ m ≤ x` gives `a ≪ x ≪ c`, so `b := x` works.

In a continuous lattice, `WayBelow.interpolate` produces an element strictly between the two.

ContinuousLattice.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow

@[expose] public section

namespace DomainTheory

open scoped DomainTheory.ContinuousLattice

universe u

variable {D : Type u} [CompleteLattice D]

class ContinuousLattice (D : Type u) [CompleteLattice D] : Prop where
  out : letI : LE D :=
    (CompleteLattice.toCompleteSemilatticeInf (a := D)).toPartialOrder.toPreorder.toLE
    forall  y : D, IsLUB {x | x  <<  y} y
  sSup_wayBelow : forall  y : D, sSup {x | x  <<  y} = y := fun y => (out y).sSup_eq

open ContinuousLattice
```

```
theorem ContinuousLattice.WayBelow.interpolate (hD : ContinuousLattice D) {a c : D} (
    hac : a  <<  c) :
    exists  b, a  <<  b  /\  b  <<  c := by
  set M : Set D := {m | exists  x, m  <<  x  /\  x  <<  c} with hM
  have hMdir : DirectedOn (⋆  <=  ⋆) M := by
    rintro m_1 <x_1, hm_1x_1, hx_1c> m_2 <x_2, hm_2x_2, hx_2c>
    obtain <x_3, hx_3c, hx_1_3, hx_2_3> := directedOn_wayBelow c x_1 hx_1c x_2 hx_2c
    obtain <m_3, hm_3x_3, hm_1m_3, hm_2m_3> :=
      directedOn_wayBelow x_3 m_1 (hm_1x_1.trans_le hx_1_3) m_2 (hm_2x_2.trans_le
    hx_2_3)
    exact <m_3, <x_3, hm_3x_3, hx_3c>, hm_1m_3, hm_2m_3>
  have hMne : M.Nonempty := <False, a, bot_wayBelow a, hac>
  have hsupM : sSup M = c := by
    refine le_antisymm (sSup_le ?_) ?_
    ⋆ rintro m <x, hmx, hxc>
      exact hmx.le.trans hxc.le
    ⋆ rw [ <-  hD.sSup_wayBelow c]
      refine sSup_le fun x hxc => ?_
      rw [ <-  hD.sSup_wayBelow x]
      exact sSup_le fun m hmx => le_sSup <x, hmx, hxc>
  rw [ <-  hsupM] at hac
  obtain <m, <x, hmx, hxc>, ham> := (WayBelow.sSup_iff hMne hMdir).1 hac
  exact <x, ham.trans_le hmx.le, hxc>

end DomainTheory
```

1.3.6 Proposition 2.4

2.4 Proposition. A complete lattice D is continuous iff for every $y \in D$ we have:

$$y = \bigsqcup \{ \sqcap U : y \in U \},$$

where U ranges over the open subsets of D.

Proof: Suppose D is continuous. If $y \in D$ and $x \ll y$, then let $U = \{z : x \sqsubseteq z\}$, an open set. Now $y \in U$ by definition, and $U \subseteq \{z : x \sqsubseteq z\}$. Thus,

$$x \sqsubseteq \sqcap U \sqsubseteq y.$$

It easily follows by lattice theory that the equation of 2.3 implies that of 2.4.

In the converse direction we have only to note that if U is open and $y \in U$, then $\sqcap U \ll y$. The implication from 2.4 to 2.3 results at once. □

`proposition_2_4` is a synonym of `isContinuousLattice_iff_isLUB_sInf_nhds`. The comparison $\sqcap U \ll y$ is `sInf_wayBelow`, filed with the way-below relation.

`proposition_2_4.lean`

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.WayBelow.
    isContinuousLattice_iff_isLUB_sInf_nhds

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

universe u

variable {D : Type u} [CompleteLattice D]

/-- **Scott 1972, Proposition 2.4.** A complete lattice is continuous iff every
    element is the
supremum of the infima of its open neighbourhoods. Synonym of
`isContinuousLattice_iff_isLUB_sInf_nhds`. -/
theorem proposition_2_4 :
    ContinuousLattice D  <->
      forall  y : D, IsLUB {a : D | exists  U, ScottOpen U  /\  y  in  U  /\  a = sInf
     U} y :=
  isContinuousLattice_iff_isLUB_sInf_nhds

end DomainTheory.ContinuousLattice
```

1.3.7 Scott Maps

Dependencies. Inside ScottMaps, each box is one Lean file. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. The arrowhead points at the file or folder that imports, and the number is how many direct imports.

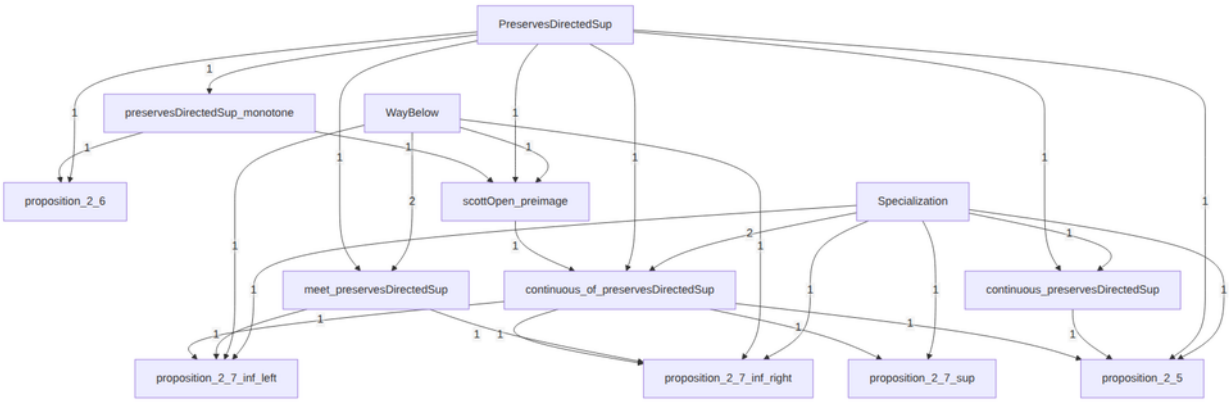


Figure 6: Scott Maps.

PreservesDirectedSup.lean

```
module
public import Mathlib.Order.ConditionallyCompleteLattice.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import Mathlib.Topology.Inseparable
public import Mathlib.Topology.Separation.Basic
public import Mathlib.Topology.Order.ScottTopology
public import Mathlib.Order.DirSupClosed
```

```
public import Mathlib.Order.ScottContinuity
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology

variable {D D' D'' : Type*} [CompleteLattice D] [CompleteLattice D'] [CompleteLattice
    D'']

def PreservesDirectedSup (f : D  ->  D') : Prop :=
  forall  ?S : Set D?, S.Nonempty  ->  DirectedOn (*  <=  *) S  ->  f (sSup S) = sSup
    (f '' S)

end DomainTheory.ContinuousLattice
```

Proposition 2.5 What is the idea of this definition? A continuous lattice is more special than a complete lattice: not only are lubs to be limits but every element must be a limit from below. This rather rough remark can be made more precise. In any complete lattice D define the *principal limit* of a net $x : I \to D$ by the formula:

$$\lim\langle x_i : i \in I\rangle = \bigsqcup \sqcap x_j : j \geq i : i \in I.$$

Then specify that x converges to $y \in D$ iff

$$y \sqsubseteq \lim\langle x_i : i \in I\rangle.$$

Having a notion of convergence, we can then say that $U \subseteq D$ is open iff every net converging to an element of U is eventually in U. This gives nothing more than what we have called the induced topology above, as is easily checked. But now being in possession of a topology, we can redefine convergence in the usual way. Question: when do the two notions of convergence agree? Answer: if and only if D is a continuous lattice.

For obviously by construction the limit definition of convergence implies the topological. Now if D is a continuous lattice and x converges to y topologically, consider an open $U \subseteq D$ with $y \in U$. For some $i \in I$ we shall have $x_j \in U$ for all $j \geq i$. Therefore

$$\sqcap U \sqsubseteq \sqcap x_j : j \geq i \sqsubseteq \lim\langle x_i : i \in I\rangle.$$

From the formula of 2.4 it at once follows that $y \sqsubseteq \lim\langle x_i : i \in I\rangle$. Thus, in continuous lattices, we have shown that the two notions of convergence are the same.

Finally, suppose that the two notions coincide for a complete lattice D. Define a directed set $I = (U, z) : y, z \in U$, where z ranges over D and U over open subsets of D. This set is directed by the relation: $(U, z) \leq (V, w)$ iff $U \supseteq V$. Let $x : I \to D$ be given by: $x(U, z) = z$. Then x is a net converging to y topologically. But

$$\lim\langle x_i : i \in I\rangle = \bigsqcup \sqcap U : y \in U.$$

In this way we see that the assumption about the two styles of convergence implies that D is a continuous lattice in view of 2.4.

In T_0-spaces continuous functions are always monotonic (i.e. $\sqsubseteq$-preserving). For continuous lattices, by virtue of the remarks we have just made about limits, we can define the continuity of $f : D \to D'$ to mean that

$$f(\lim\langle x_i : i \in I\rangle) \sqsubseteq \lim'\langle f(x_i) : i \in I\rangle$$

for all nets $x : I \to D$. This is all very fine, but general limits are messy to work with; we shall find it easier to state results in terms of lubs as in 2.5–2.7 below.

Before going any deeper, however, we should clear up another point about topologies. Suppose that D is any T_0-space which becomes a complete lattice under its induced partial ordering. Then it is evident from our definitions that every set open in the given topology is also open in the topology induced from the lattice structure. Question: when do the two topologies agree? Answer: a sufficient condition is that the equation:

$$y = \bigsqcup \sqcap U : y \in U$$

hold for all $y \in D$, where U ranges over the given open sets. Because in that case if V is open in the lattice sense and $y \in V$, then $\sqcap U \in V$ for some set U, open in the given sense, where $y \in U$. But $U \subseteq V$ follows, and so V is a union of given open sets and is itself open in the given topology. Of course this equation implies that D is a continuous lattice by virtue of 2.4. Notice that by the same token the sets of the form $y \in D : x \ll y$ will form a basis for the open sets of a continuous lattice.

hold for all $y \in D$, where U ranges over the given open sets. Because in that case if V is open in the lattice sense and $y \in V$, then $\sqcap U \in V$ for some set U, open in the given sense, where $y \in U$. But $U \subseteq V$ follows, and so V is a union of given open sets and is itself open in the given topology. Of course this equation implies that D is a continuous lattice by virtue of 2.4. Notice that by the same token the sets of the form $y \in D : x \ll y$ will form a basis for the open sets of a continuous lattice.

2.5 Proposition. *If D and D' are complete lattices with their induced topologies, then a function $f : D \to D'$ is continuous iff for all directed subsets $S \subseteq D$:*

$$f(\bigsqcup S) = \bigsqcup f(x) : x \in S.$$

Proof: If $f : D \to D'$ is continuous, the equation follows from the definition of continuous function and the fact that lubs are limits. Assume then that the equation holds for all directed sets S. Let $U' \subseteq D'$ be open in D' and let

$$U = x \in D : f(x) \in U'.$$

We must show that U is open in D. Note first that if $x \sqsubseteq y$, then $S = x, y$ is directed; hence,

$$f(x \sqcup y) = f(y) = f(x) \sqcup f(y),$$

so $f(x) \sqsubseteq f(y)$. Thus f is monotonic and so U satisfies condition (i). That U satisfies condition (ii) follows at once from the above equation. $\square$

Proposition 2.6 **2.6 Proposition.** *With functions from complete lattices to complete lattices, a function of several variables is continuous in the variables jointly iff it is continuous in the variables separately.*

Proof: It will be sufficient to discuss functions of two variables. The product $D \times D'$ of two complete lattices is a complete lattice, and it is easy to check that the induced topology is the product topology. Since projection is continuous, joint continuity implies separate continuity. To check the converse suppose that

$$f : D \times D' \to D''$$

is a map where the separate continuity holds as follows:

$$f(\bigsqcup S, y) = \bigsqcup f(x, y) : x \in S$$

and

$$f(x, \bigsqcup S') = \bigsqcup f(x, y) : y \in S'$$

where $S \subseteq D$ and $S' \subseteq D'$ are directed and $x \in D$ and $y \in D'$. Let now

$$S^* \subseteq D \times D'$$

be directed in the product. The projection of S^* to D and of S^* to D' produces directed subsets $S \subseteq D$ and $S' \subseteq D'$. Note that

$$\bigsqcup S^* = (\bigsqcup S, \bigsqcup S').$$

Thus by assumption

$$f(\bigsqcup S^*) = \bigsqcup f(x, y) : x \in S, y \in S'.$$

But since S^* is directed, $x \in S$ and $y \in S'$ implies $x \sqsubseteq u$ and $y \sqsubseteq v$ for $(u, v) \in S^*$. Thus by monotonicity of f we can show

$$f(\bigsqcup S^*) = \bigsqcup f(u, v) : (u, v) \in S^*$$

and that gives the joint continuity. □

proposition_2_6.lean

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.preservesDirectedSup_monotone
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology

variable {D D' D'' : Type*} [CompleteLattice D] [CompleteLattice D'] [CompleteLattice
    D'']
```

```
/-- **Scott 1972, Proposition 2.6.** A function of several variables between complete
    lattices is
(Scott-)continuous jointly iff it is continuous in each variable separately.
    Continuity is phrased
as preservation of directed suprema (Proposition 2.5); it suffices to treat two
    variables, the
product `D  x  D'` carrying the componentwise complete-lattice structure (whose induced
     topology is
the product topology). -/
theorem proposition_2_6 (f : D  x  D'  ->  D'') :
    PreservesDirectedSup f  <->
      (forall  y, PreservesDirectedSup fun x => f (x, y))  /\
        (forall  x, PreservesDirectedSup fun y => f (x, y)) := by
  constructor
  * -- joint continuity ? separate continuity (precompose with the continuous slice
    maps)
    intro hf
    refine <fun y S hS hSdir => ?_, fun x S hS hSdir => ?_>
    * set T : Set (D  x  D') := (fun x => (x, y)) '' S with hT
      have hTne : T.Nonempty := hS.image _
      have hTdir : DirectedOn (*  <=  *) T := by
        rintro _ <a, ha, rfl> _ <b, hb, rfl>
        obtain <c, hc, hac, hbc> := hSdir a ha b hb
        exact <(c, y), Set.mem_image_of_mem _ hc, <hac, le_rfl>, <hbc, le_rfl>>
      have hfst : Prod.fst '' T = S := by rw [hT, Set.image_image]; simp
      have hsnd : Prod.snd '' T = {y} := by rw [hT, Set.image_image]; exact hS.
    image_const y
      have hsupT : sSup T = (sSup S, y) := by
        have e1 : (sSup T).1 = sSup S := by rw [Prod.fst_sSup, hfst]
        have e2 : (sSup T).2 = y := by rw [Prod.snd_sSup, hsnd, sSup_singleton]
        exact Prod.ext_iff.mpr <e1, e2>
      have h := hf hTne hTdir
      rw [hsupT, hT, Set.image_image] at h
      simpa using h
    * set T : Set (D  x  D') := (fun y => (x, y)) '' S with hT
      have hTne : T.Nonempty := hS.image _
      have hTdir : DirectedOn (*  <=  *) T := by
        rintro _ <a, ha, rfl> _ <b, hb, rfl>
        obtain <c, hc, hac, hbc> := hSdir a ha b hb
        exact <(x, c), Set.mem_image_of_mem _ hc, <le_rfl, hac>, <le_rfl, hbc>>
      have hfst : Prod.fst '' T = {x} := by rw [hT, Set.image_image]; exact hS.
    image_const x
      have hsnd : Prod.snd '' T = S := by rw [hT, Set.image_image]; simp
      have hsupT : sSup T = (x, sSup S) := by
        have e1 : (sSup T).1 = x := by rw [Prod.fst_sSup, hfst, sSup_singleton]
        have e2 : (sSup T).2 = sSup S := by rw [Prod.snd_sSup, hsnd]
        exact Prod.ext_iff.mpr <e1, e2>
      have h := hf hTne hTdir
      rw [hsupT, hT, Set.image_image] at h
      simpa using h
  * -- separate continuity ? joint continuity (Scott 1972's directedness argument)
    rintro <h1, h2> Sstar hne hdir
    have hmono1 : forall  y, Monotone fun x => f (x, y) := fun y =>
    preservesDirectedSup_monotone (h1 y)
    have hmono2 : forall  x, Monotone fun y => f (x, y) := fun x =>
```

```
    preservesDirectedSup_monotone (h2 x)
    have hmono : Monotone f := by
      rintro <a, b> <c, d> hpq
      exact le_trans (hmono1 b hpq.1) (hmono2 c hpq.2)
    have hSne : (Prod.fst '' Sstar).Nonempty := hne.image _
    have hS'ne : (Prod.snd '' Sstar).Nonempty := hne.image _
    have hSdir : DirectedOn (⋆ ≤ ⋆) (Prod.fst '' Sstar) := hdir.fst
    have hS'dir : DirectedOn (⋆ ≤ ⋆) (Prod.snd '' Sstar) := hdir.snd
    have hsupStar : sSup Sstar = (sSup (Prod.fst '' Sstar), sSup (Prod.snd '' Sstar))
    :=
      Prod.ext_iff.mpr <Prod.fst_sSup _, Prod.snd_sSup _>
    apply le_antisymm
    · rw [hsupStar]
      have e1 : f (sSup (Prod.fst '' Sstar), sSup (Prod.snd '' Sstar))
          = sSup ((fun x => f (x, sSup (Prod.snd '' Sstar))) '' (Prod.fst '' Sstar))
    :=
        h1 (sSup (Prod.snd '' Sstar)) hSne hSdir
      rw [e1]
      apply sSup_le
      rintro _ <x, hx, rfl>
      show f (x, sSup (Prod.snd '' Sstar)) ≤ sSup (f '' Sstar)
      have e2 : f (x, sSup (Prod.snd '' Sstar))
          = sSup ((fun y => f (x, y)) '' (Prod.snd '' Sstar)) :=
        h2 x hS'ne hS'dir
      rw [e2]
      apply sSup_le
      rintro _ <y, hy, rfl>
      show f (x, y) ≤ sSup (f '' Sstar)
      obtain <p, hpS, hpfst> := hx
      obtain <q, hqS, hqsnd> := hy
      obtain <r, hrS, hpr, hqr> := hdir p hpS q hqS
      have hxr : x ≤ r.1 := hpfst ▸ hpr.1
      have hyr : y ≤ r.2 := hqsnd ▸ hqr.2
      calc f (x, y) ≤ f (r.1, r.2) := hmono <hxr, hyr>
        _ = f r := by rw [Prod.mk.eta]
        _ ≤ sSup (f '' Sstar) := le_sSup (Set.mem_image_of_mem f hrS)
    · apply sSup_le
      rintro _ <p, hp, rfl>
      exact hmono (le_sSup hp)

end DomainTheory.ContinuousLattice
```

Proposition 2.7 One of the justifications (by euphony at least) of the term *continuous lattice* is the fact that such spaces allow for so many continuous functions. One indication of this is the result:

2.7 Proposition. *In a continuous lattice D the finitary lattice operations $\sqcup$ and $\sqcap$ are continuous.*

Proof: It is trivial to show that $\sqcup$ is continuous in every complete lattice; this is not so for $\sqcap$. In view of 2.6 we need only show

$$x \sqcap \bigsqcup S = \bigsqcup x \sqcap y : y \in S$$

for every directed $S \subseteq D$. In fact it is enough to show

$$x \sqcap \bigsqcup S \sqsubseteq \bigsqcup x \sqcap y : y \in S$$

because the opposite inequality is valid in all complete lattices. In view of the fact that D is continuous, it is enough to show that

$$t \ll x \sqcap \bigsqcup S \text{ implies } t \sqsubseteq \bigsqcup x \sqcap y : y \in S.$$

So assume $t \ll x \sqcap \bigsqcup S$. Then $t \sqsubseteq x \sqcap \bigsqcup S \sqsubseteq x$. Also $t \ll \bigsqcup S$ because $x \sqcap \bigsqcup S \sqsubseteq \bigsqcup S$. Thus $t \ll y$ for some $y \in S$ since the set

$$z \in D : t \ll z$$

is open. But then $t \sqsubseteq y$, and so $t \sqsubseteq x \sqcap y$, and the result follows. □

meet_preservesDirectedSup.lean

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology

variable {D D' D'' : Type*} [CompleteLattice D] [CompleteLattice D'] [CompleteLattice
    D'']

theorem meet_preservesDirectedSup (x : D) (hD : ContinuousLattice D) :
    PreservesDirectedSup (fun z => x  sqcap  z) := by
  intro S hS hSdir
  apply le_antisymm
  * have hle : sSup {t | t  <<  x  sqcap  sSup S}  <=  sSup (Set.image (fun z => x
    sqcap  z) S) := by
      apply sSup_le
      intro t ht
      obtain <w, hwS, ht_w> := (WayBelow.sSup_iff hS hSdir).1 (ht.trans_le
    inf_le_right)
      have hmem : x  sqcap  w  in  Set.image (fun z : D => x  sqcap  z) S :=
        Set.mem_image_of_mem (fun z => x  sqcap  z) hwS
      have hbound : x  sqcap  w  <=  sSup (Set.image (fun z => x  sqcap  z) S) :=
    le_sSup hmem
      exact (le_inf (ht.le.trans inf_le_left) ht_w.le).trans hbound
    dsimp only
    rw [ <-  hD.sSup_wayBelow (x  sqcap  sSup S)]
    exact hle
  * apply sSup_le
    intro z hz
    obtain <w, hwS, rfl> := hz
    exact inf_le_inf le_rfl (le_sSup hwS)

end DomainTheory.ContinuousLattice
```

1.3.8 Milner Correction

Dependencies. Inside MilnerCorrection, each box is one Lean file. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. The arrowhead points at the file or folder that imports, and the number is how many direct imports.

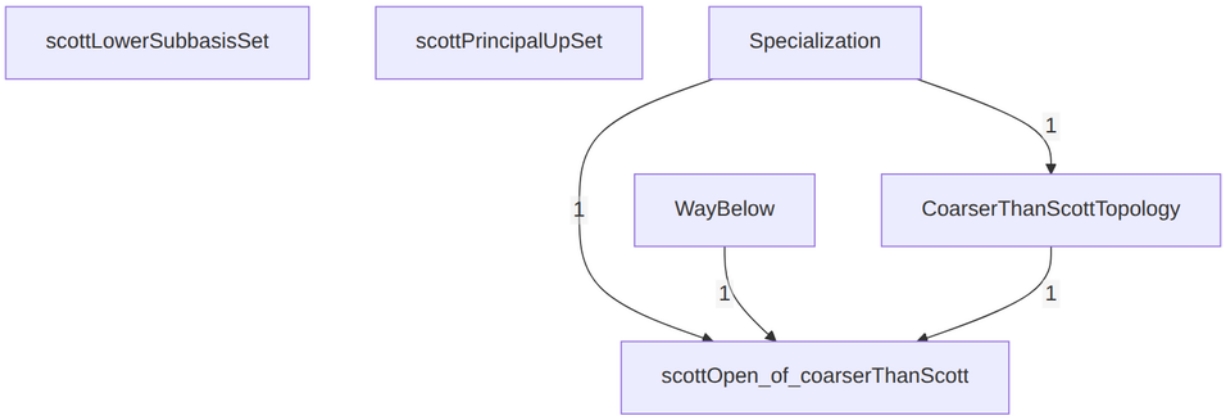


Figure 7: Milner Correction.

Correction (Added March, 1972). Robin Milner has pointed out to me that there is an error in the remark in the paragraph immediately preceding Proposition 2.5. I was mistaken in saying that if D is a T_0-space which becomes a complete lattice under its induced partial ordering, then every set open in the given topology is also open in the induced topology. There are many counterexamples to this statement. Let D be any complete lattice. There are two extreme T_0-topologies which will induce the given partial ordering. The smallest such topology has as a sub-base for its open sets those sets of the form:

$$x \in D : x \not\sqsubseteq y.$$

These sets are easily proved to be open in any T_0-topology which induces the partial ordering. At the other extreme consider sets of the form:

$$x \in D : y \sqsubseteq x.$$

Such sets will give a base for a T_0-topology that is the maximal topology inducing the given partial ordering. Clearly they need not be open in the induced lattice topology; in particular, they may well fail to satisfy conditions (ii) on open sets. To make the remark in question correct, we must thus suppose that the given T_0-topology is *contained within* the induced lattice topology. The equation given in the paragraph indicated will then be a sufficient condition for the two topologies to be identical.

The remark was employed in the proof of three different propositions: 2.9, 2.10, and 3.3. In the case of 2.9 one must verify that the product topology is contained within the lattice topology. This need only be done for the basis for the product topology, and for such basic open sets the result needed is obvious. In the case of proposition 2.10 the question concerns a relationship between the topologies of a space and a subspace; the spaces in question are also lattices. Note in passing that a lub in the subspace is generally *larger* in the partial ordering than the corresponding lub relative to the whole space. This puts the inequalities in the wrong direction, and so it is not immediate that a relativized open set for the subspace is open in the lattice

topology of the subspace. However, in this case we can appeal to the continuous retraction. Recall that the relativized open sets of the kind that we used in the proof of 2.10 are of the form:

$$U = z \in D : x < z \ .$$

Suppose then that S is a directed set, and that using the lub in the sense of D we have

$$\bigsqcup S \in U \ .$$

Referring back to the proof of 2.10 we know that ($S' = S \subseteq D$, but $\bigsqcup S'$ is the join computed in the ambient lattice D')

$$j(\bigsqcup S') = \bigsqcup S \ ,$$

which means that

$$x < j(\bigsqcup S') \ .$$

From this it follows that

$$x < j(z), \text{ some } z \in S \ .$$

Now $j(z) = z$, and we have what we need. This argument suffices only for a special type of open sets; but these open sets form a base for the topology, and so the argument is quite general.

Turning now to the proof of theorem 2.3 we note that the topology on the function space is simply the *relativized* product topology. There is no difficulty with lubs in this case, because, as we showed in the proof of that theorem, all lubs are calculated pointwise. Thus, it is easy to verify now that the sets open in the product topology are also open in the lattice topology.

CoarserThanScottTopology.lean

```
module

public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology

variable {D D' : Type*} [CompleteLattice D] [CompleteLattice D']

/-- **March 1972 correction.** The given topology `tau` is coarser than (or equal to)
    the Scott
topology: every set open in `tau` is Scott-open. Equivalently `Opens(tau)  subseteq
    Opens(Scott)`,
i.e. `scottTopologicalSpace  <=  tau` in mathlib's order on topologies. -/
def CoarserThanScottTopology (tau : TopologicalSpace D) : Prop :=
  scottTopologicalSpace  <=  tau

end DomainTheory.ContinuousLattice
```

(Already included: `1972_Proposition_2_5.md`.)
(Already included: `1972_Proposition_2_6.md`.)
(Already included: `1972_Proposition_2_7.md`.)

1.3.9 Proposition 2.8

It is now time to provide some examples of continuous lattices.

2.8 Proposition. A finite lattice is a continuous lattice. □

`proposition_2_8.lean`

```
module
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice

public import DomainTheory.ContinuousLattice.Constructions.directedOn_finite_sSup_mem
public import Mathlib.Data.Fintype.EquivFin
public import Mathlib.Basic.Finite.Defs
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

variable {D : Type*} [CompleteLattice D]

/-- **Scott 1972, Proposition 2.8.** A finite lattice is a continuous lattice. In a
    finite
lattice every principal up-set `Set.Ici y` is Scott-open (a non-empty directed set is
    finite,
so it attains its supremum), hence `y  <<  y`; therefore `y` is the supremum of `{x | x
      <<  y}`. -/
theorem proposition_2_8 [Finite D] : ContinuousLattice D := by
  refine { out := ?_ }
  intro y
  have hopen : ScottOpen (Set.Ici y) := by
    refine <isUpperSet_Ici y, fun S hSne hSdir hmem => ?_>
    exact <sSup S, directedOn_finite_sSup_mem (Set.toFinite S) hSne hSdir, hmem>
  have hyy : y  <<  y := WayBelow.self_iff_scottOpen_Ici.mpr hopen
  exact <fun x hx => hx.le, fun b hb => hb hyy>

end DomainTheory.ContinuousLattice
```

1.3.10 Proposition 2.9

Proposition. The Cartesian product of any number of continuous lattices is a continuous lattice with the induced topology agreeing with the product topology.

Proof.

Let D_i for $i \in I$ be a system of continuous lattices. The product

$$D^* = \prod_{i \in I} D_i$$

is a complete lattice in the usual way and has its induced topology. Suppose $y \in D^*$ and let $i \in I$. Then $y_i \in D_i$. Since D_i is a continuous lattice

$$y_i = \bigsqcup \{x \in D_i : x \ll y_i\}.$$

For $x \in D_i$, let $[x]^i \in D^*$ be defined by:

$$[x]^i_j = \begin{cases} x & \text{if } i = j, \\ \bot & \text{if } i \neq j. \end{cases}$$

Note that since D_i is continuous we have:

$$[y_i]^i = \bigsqcup \{[x]^i : x \ll y_i\},$$

and

$$y = \bigsqcup \{[y_i]^i : i \in I\}.$$

It follows that

$$y = \bigsqcup \bigcap \{z : z_i \in U \text{ for all } i \in I,\ y_i \in U\},$$

where i ranges over I and U over the open subsets of D_i, because

$$[x]^i \sqsubseteq \bigcap \{z : z_i \in U\}, \quad \text{where } U = \{u \in D_i : x \ll u\}.$$

But the sets $\{z : z_i \in U\}$ are open in the product sense, and so

$$y = \bigsqcup \bigcap \{U : y \in U\},$$

where U ranges now over the open subsets of the product topology on D^*. By the remark following 2.4 we conclude that D^* is continuous with the lattice-induced topology being the product topology. □

proposition_2_9.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import DomainTheory.ContinuousLattice.Constructions.Basic
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import DomainTheory.ContinuousLattice.Specialization.isOpen_iff_scottOpen
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_Ici_subset
public import Mathlib.Topology.Defs.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Topology Set

variable {D : Type*} [CompleteLattice D]

theorem proposition_2_9 {? : Type*} (E : ?  ->  Type*) [forall  i, CompleteLattice (E
    i)]
    (hE : forall  i, ContinuousLattice (E i)) :
    ContinuousLattice (forall  i, E i)  /\
      (scottTopologicalSpace : TopologicalSpace (forall  i, E i)) =
        @Pi.topologicalSpace ? E (fun _ => scottTopologicalSpace) := by
  classical
  have hprod : ContinuousLattice (forall  i, E i) := by
    refine { out := ?_ }
    intro y
    refine <fun x hx => hx.le, fun b hb => ?_>
    rw [Pi.le_def]
    intro i
    rw [ <-  (hE i).sSup_wayBelow (y i)]
    apply sSup_le
    intro a ha
    set e : (forall  j, E j) := Function.update (False : forall  j, E j) i a with he
    have hei : e i = a := by rw [he]; exact Function.update_self i a _
    have hcyl : e  <<  y := by
      obtain <U, hU, hyiU, hUsub> := ha
      refine <{z : forall  j, E j | z i  in  U}, ?_, hyiU, ?_>
      * refine <fun z w hzw hz => hU.1 (hzw i) hz, fun S hSne hSdir hmem => ?_>
        rw [Set.mem_ofPred_eq, sSup_apply_eq_sSup_image] at hmem
        have hdir' : DirectedOn (*  <=  *) (Function.eval i '' S) := by
          rintro _ <f, hf, rfl> _ <g, hg, rfl>
          obtain <h, hhS, hfh, hgh> := hSdir f hf g hg
          exact <Function.eval i h, <h, hhS, rfl>, hfh i, hgh i>
        obtain <t, htimg, htU> := hU.2 (hSne.image _) hdir' hmem
        obtain <f, hfS, rfl> := htimg
        exact <f, hfS, htU>
      * intro z hz
        rw [Set.mem_Ici, Pi.le_def]
        intro j
        rcases eq_or_ne j i with rfl | hji
        * rw [hei]; exact Set.mem_Ici.1 (hUsub hz)
        * rw [he, Function.update_of_ne hji]; exact bot_le
    have hle : e i  <=  b i := (hb hcyl) i
    rw [hei] at hle
    exact hle
  refine <hprod, ?_>
  apply le_antisymm
  * refine le_iInf fun i => ?_
    rw [ <-  continuous_iff_le_induced]
    exact continuous_of_preservesDirectedSup (fun _ _ _ => sSup_apply_eq_sSup_image)
  * intro U hU
    rw [isOpen_iff_scottOpen] at hU
    rw [@isOpen_iff_forall_mem_open _ (@Pi.topologicalSpace ? E (fun _ =>
    scottTopologicalSpace))]
    intro z hz
    obtain <a, haz, haIci> := exists_wayBelow_Ici_subset hprod hU hz
```

```
    obtain <F, hF> := wayBelow_finite_support haz
    have hproj : forall  i, a i  <<  z i := fun i => wayBelow_proj haz i
    choose V hVopen hzV hVsub using hproj
    refine <? i  in  F, (fun w : forall  j, E j => w i) ^-1' V i, ?_, ?_, ?_>
    * intro w hw
      rw [Set.mem_iInter_2] at hw
      refine haIci (Set.mem_Ici.2 fun j => ?_)
      by_cases hjF : j  in  F
      * exact Set.mem_Ici.1 (hVsub j (hw j hjF))
      * rw [hF j hjF]; exact bot_le
    * refine @isOpen_biInter_finset (forall  j, E j) ?
        (@Pi.topologicalSpace ? E (fun _ => scottTopologicalSpace)) F
        (fun i => (fun w : forall  j, E j => w i) ^-1' V i) (fun i _ => ?_)
      have hVi : @IsOpen (E i) scottTopologicalSpace (V i) := isOpen_iff_scottOpen.mpr
     (hVopen i)
      have hind : @IsOpen _
          (TopologicalSpace.induced (fun w : forall  j, E j => w i)
    scottTopologicalSpace)
          ((fun w : forall  j, E j => w i) ^-1' V i) :=
        (isOpen_induced_iff (t := scottTopologicalSpace)).mpr <V i, hVi, rfl>
      exact iInf_le
        (fun i => TopologicalSpace.induced (fun w : forall  j, E j => w i)
    scottTopologicalSpace) i _ hind
    * rw [Set.mem_iInter_2]
      exact fun i _ => hzV i

end DomainTheory.ContinuousLattice
```

1.3.11 Proposition 2.10 (flawed)

2.10 Proposition. A retract of a continuous lattice is a continuous lattice with the subspace topology agreeing with the induced topology.

Proof of 2.10

Let D' be a continuous lattice and let $D \subseteq D'$ be a subspace which is a retract. We have for a suitable $j : D' \to D$,

$$D = x \in D' : j(x) = x,$$

where of course j is continuous.

First a note of warning: though D is a subspace it is not a sublattice; that is, the partial ordering on D is the restriction of that of D', but the lubs of D are not those of D'. We shall have to distinguish operations by adding a prime ($'$) for those of D'.

Suppose $x, y \in D$. Let $z' = x \sqcup y \in D'$ (join taken in D') and define $z = j(z') \in D$. Now $x \sqsubseteq z'$ and $y \sqsubseteq z'$, and j is monotonic, so $x \sqsubseteq z$ and $y \sqsubseteq z$. Suppose $x \sqsubseteq w$ and $y \sqsubseteq w$ with $w \in D$. Then in D' we have $x \sqcup y \sqsubseteq w$; so $z \sqsubseteq w$ also. Hence we have shown that $z = x \sqcup y$ in D.

To show that D has a least element $\bot$ (which may be larger than the $\bot' \in D'$), we need a well-known lemma about monotonic functions: Every monotonic function on a complete lattice into itself has a least fixed point. (Cf. Birkhoff (1970), p. 115.) In our case j is monotonic and

$$\bot = \sqcap\, x \in D' : j(x) \sqsubseteq x$$

is the desired element in D.

Thus D is at least a semilattice with $\perp$ and $\sqcup$. To show that D is a lattice we need to show that every directed $S \subseteq D$ has a lub in D. Now we know ($S' = S$, join taken in the ambient lattice D'):

$$\bigsqcup S' \in D',$$

and this is a limit of a monotone net. So by 2.1, and the continuity of j:

$$j(\bigsqcup S') = \bigsqcup j(x) : x \in S \quad = \bigsqcup S$$

in D. In this way we now know that D is a complete lattice. We must still show that D is continuous.

Suppose $y \in D$. In D' we can write:

$$y = \bigsqcup x \in D' : x \ll y'$$

and this is the limit of a monotone net. Thus

$$j(y) = y = \bigsqcup j(x) : x \ll y, x \in D',$$

where the lub is taken in D. Note that the sets

$$U = z \in D : x \ll z$$

are open in D for each $x \in D'$. Note too that if $z \in U$, then $x \sqsubseteq z$ and so $j(x) \sqsubseteq j(z) = z$. This means that

$$j(x) \sqsubseteq \sqcap U$$

in D. We can then write in D:

$$y = \bigsqcup \sqcap U : y \in U$$

where U ranges over the open subsets of D, and so the lattice is continuous by 2.4. **Inasmuch as the open sets U just used were open in the subspace topology, it follows by the remark after 2.4 that the subspace and the lattice-induced topologies coincide.** □

This appeal is flawed. The remark before Proposition 2.5 is false, and this proof is one of the three that use it. The discussion and the repair are in the Milner correction. The corrected proof is Proposition 2.10 revisited.

1.3.12 Proposition 2.11

It would seem that the continuous lattices are starting to sound suspiciously like the injective spaces. Indeed, if we can prove the following, the circle will be complete.

Proposition. Every continuous lattice is an injective space under its induced topology.

Proof of 2.11: Let D be a continuous lattice with its induced topology, and let $X \subseteq Y$ be two T_0-spaces in the subspace relation. Suppose

$$f : X \to D$$

is continuous. Define

$$\bar{f} : Y \to D$$

by the formula:

$$\bar{f}(y) = \bigsqcup \sqcap f(x) : x \in X \cap U : y \in U,$$

where U ranges over the open subsets of Y. We need to show that $\bar{f}$ extends f and that it is continuous.

First, the continuity: Suppose that $d \in D$ and $d \ll \bar{f}(y)$; that is, $\bar{f}(y)$ belongs to a typical basic open subset of D. Since D is continuous, we can also find

$$d \ll d' \ll \bar{f}(y)$$

with $d' \in D$. From the definition of $\bar{f}$ it follows that

$$d' \sqsubseteq \sqcap f(x) : x \in X \cap U$$

for some open $U \subseteq Y$ with $y \in U$. Thus

$$d' \sqsubseteq \bar{f}(y')$$

for all $y' \in U$ by virtue of the definition of $\bar{f}$. Since $d \ll d'$, we have also

$$d \ll \bar{f}(y')$$

for all $y' \in U$; in other words, the inverse image of the open subset of D determined by d under $\bar{f}$ is indeed open in Y, and $\bar{f}$ is thus continuous.

Next, the extension property: Note that the relationship

$$\bar{f}(x') \sqsubseteq f(x')$$

for all $x' \in X$ comes directly out of the definition of $\bar{f}$. For the converse, suppose $d \ll f(x')$ where $d \in D$. By assumption $f : X \to D$ is continuous, so

$$d \ll f(x'')$$

for all $x'' \in X \cap U$ where U is a suitable open subset of Y with $x' \in U$. In particular we have:

$$d \sqsubseteq \sqcap f(x'') : x'' \in X \cap U,$$

and so $d \sqsubseteq \bar{f}(x')$. Since $d \ll f(x')$ always implies $d \sqsubseteq \bar{f}(x')$, we see that $f(x') \sqsubseteq \bar{f}(x')$ follows by the continuity of D, and the proof is complete. □

Furthermore, as we have seen, the injective property can be exhibited, as in the proof of 2.11, by an explicit formula for extending functions. That formula is `scottExtend`. Theorem 2.12 proves that every injective space arises in this way.

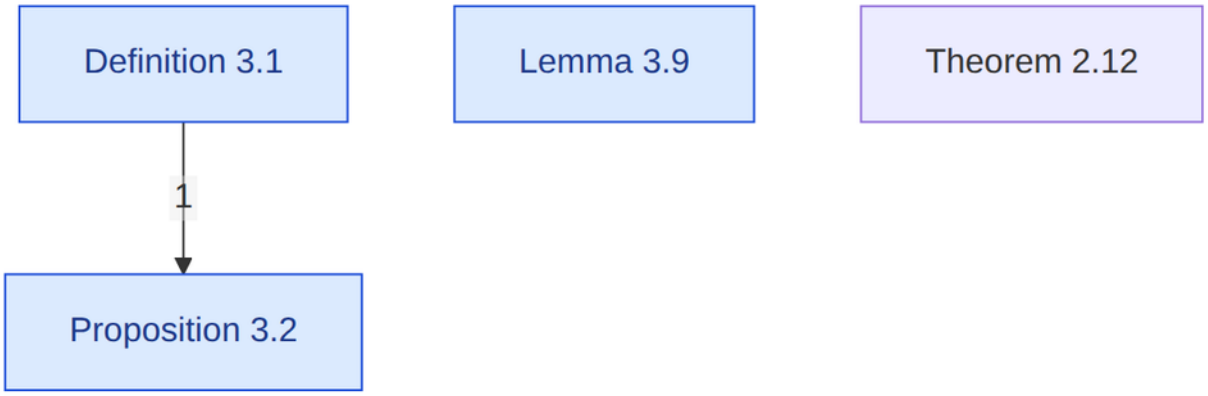


Figure 8: Function spaces.

1.4 Function spaces

The section follows the paper from Definition 3.1 through Proposition 3.14. Proposition 2.10 revisited is the corrected proof, placed after Definition 3.6. Theorem 2.12 keeps its number and follows that proof.

The arrowhead is on the note that imports. The number is how many direct imports. An arrow already implied by a longer path is left out. Definition 3.6 and Proposition 3.8 hold most of the shared code, so the large counts run through those two notes. An arrow that points up is an earlier statement using code carried on one of those notes. Definition 3.11 has no Lean files of its own.

1.4.1 Definition 3.1: Function Space

3. FUNCTION SPACES. We recall the standard definition and introduce our notation for function spaces.

3.1 Definition. For T_0-spaces X and Y we let $[X \to Y]$ be the *space of all continuous functions* $f : X \to Y$ endowed with the product topology, sometimes called the topology of pointwise convergence. This topology has as a subbase sets of the form:

$$f : f(x) \in U$$

where $x \in X$ and $U \subseteq Y$ is open.

The pointwise aspect of the topology is immediately apparent in the partial ordering.

ScottMap.lean

```
module

public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

universe u v

namespace DomainTheory.ContinuousLattice
```

```
open Set Topology TopologicalSpace

variable {X Y D D' D'' D_0 D_0' D_1 D_1' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D_0] [CompleteLattice D_0'] [CompleteLattice D_1] [
    CompleteLattice D_1']

/-- Continuous maps between complete lattices with Scott's induced topologies. -/
def ScottMap (D : Type u) (D' : Type v) [CompleteLattice D] [CompleteLattice D'] :
    Type _ :=
  { f : D  ->  D' // @Continuous D D' scottTopologicalSpace scottTopologicalSpace f }

namespace ScottMap

instance : CoeFun (ScottMap D D') (fun _ => D  ->  D') where
  coe f := f.1

@[ext]
theorem ext {f g : ScottMap D D'} (h : forall  x, f x = g x) : f = g :=
  Subtype.ext (funext h)

end ScottMap

end DomainTheory.ContinuousLattice
```

1.4.2 Proposition 3.2

3.2 Proposition. *The induced partial ordering on* $[X \to Y]$ *is such that:*

$$f \sqsubseteq g \quad \text{iff} \quad f(x) \sqsubseteq g(x) \text{ for all } x \in X,$$

where $f, g \in [X \to Y]$. □
where $f, g \in [X \to Y]$. □

1.4.3 Theorem 3.3

The first question, of course, is what kind of a partial ordering this is.

3.3 Theorem. If D and D' are continuous lattices, then so is $[D \to D']$ under the induced partial ordering with the lattice topology agreeing with the product topology.

Proof: The argument is "pointwise." Thus, the constant function with value $\perp \in D'$ is obviously continuous and is the $\perp$ of $[D \to D']$ by 3.2. Since by 2.7 the lattice operation $\sqcup$ on D' is continuous, then if $f, g \in [D \to D']$ the composition $f \sqcup g$, defined by

$$(f \sqcup g)(x) = f(x) \sqcup g(x)$$

for all $x \in D$, is also continuous and represents the lub of f, g in $[D \to D']$. (The same arguments apply to $\top$ and $\sqcap$, so $[D \to D']$ is at least a lattice.) To show that $[D \to D']$ is complete it is sufficient now to

show that lubs of directed subsets exist. So let $\mathcal{F} \subseteq [D \to D']$ be directed. Define a function from D into D' by the equation:

$$(\bigsqcup \mathcal{F})(x) = \bigsqcup f(x) : f \in \mathcal{F},$$

for all $x \in D$. If we can show that $\bigsqcup \mathcal{F}$ is continuous, then being in $[D \to D']$ it has to be the lub. Consider $U \subseteq D'$, an open subset. Taking the inverse image and remembering that $\mathcal{F}$ is directed, we find:

$$\{x : (\bigsqcup \mathcal{F})(x) \in U\} = \bigcup \{\{x : f(x) \in U\} : f \in \mathcal{F}\}.$$

This is an open set, and so $\bigsqcup \mathcal{F}$ is indeed continuous. (Warning: the infinite $\sqcap \mathcal{F}$ are not in general computed pointwise; however, it is easy to extend the above argument to show that arbitrary $\bigsqcup \mathcal{F}$ are.)

To show that $[D \to D']$ is continuous, we establish first that for $f \in [D \to D']$

$$f = \bigsqcup \overrightarrow{e}[e, e'] : e' \ll f(e),$$

where e ranges over D and e' over D', and where the function $\overrightarrow{e}[e, e']$ is defined by:

$$\overrightarrow{e}[e, e'](x) = \begin{cases} e' & \text{if } e \ll x, \\ \bot & \text{if not,} \end{cases}$$

for all $x \in D$. Call the function on the right f'. Calculate:

$$f'(x) = \bigsqcup \{\overrightarrow{e}[e, e'](x) : e' \ll f(e)\}$$

$$= \bigsqcup \{e' : \exists e \ll x,\ e' \ll f(e)\}$$

$$= \bigsqcup \{e' : e' \ll f(x)\} = f(x).$$

With the equation for f proved, note next that for all $g \in [D \to D']$, $e' \ll g(e)$ implies $\overrightarrow{e}[e, e'] \sqsubseteq g$ by an easy pointwise argument. If we let $V = \{g : e' \ll g(e)\}$, we see then that V is open in the product topology and that $\overrightarrow{e}[e, e'] \sqsubseteq \sqcap V$. We may then conclude that $f = \bigsqcup \{\sqcap V : f \in V\}$, which proves both that $[D \to D']$ is a continuous lattice and that the two topologies agree by the remark following 2.4. □

1.4.4 Corollary 3.4

The above theorem might possibly be generalized to $[X \to D]$ where X is merely a T_0-space, but I was unable to see the argument. In any case we are mostly interested in the continuous lattices. Note as a consequence of our proof:

3.4 Corollary. For continuous lattices D and D', the evaluation map:

$$\mathrm{eval} : [D \to D'] \times D \to D'$$

is continuous.

Proof: Here $\mathrm{eval}(f, x) = f(x)$. With f fixed, this is obviously continuous. With x fixed, we proved the continuity above with our calculation of $\bigsqcup \mathcal{F}$ in view of 2.5. Hence applying 5.3 and 2.6, we conclude that eval is jointly continuous. □

1.4.5 Proposition 3.5

This result gives only one example of the masses of continuous functions that are available on continuous lattices. As another fundamental example we have:

3.5 Proposition. For continuous lattices D, D', and D'', the map of functional abstraction:

$$\mathrm{lambda} : [[D \times D'] \to D''] \to [D \to [D' \to D'']]$$

is continuous.

Proof: Here lambda is defined by:

$$\mathrm{lambda}(f)(x)(y) = f(x, y)$$

where $f \in [[D \times D'] \to D'']$ and $x \in D$ and $y \in D'$. What is particularly interesting here is that by virtue of 3.3 we are making use of $[D \to [D' \to D'']]$ as a continuous lattice. The principle being stated here can be formulated more broadly in this way:

> If an expression $\mathcal{E}(x, y, z, \dots)$ is continuous in all its variables $x, y, z, \dots$ with values in D' as x ranges in D, then the expression
>
> $$\lambda x : D.\mathcal{E}(x, y, z, \dots)$$
>
> with values in $[D \to D']$ is continuous in the remaining variables $y, z, \dots$.

The λ-notation is a notation for functions, where in the above the variable after the λ is the *argument* and the expression after the . is the *value* (as a function of the argument). Thus we could write:

$$\mathrm{lambda} = \lambda f : [[D \times D'] \to D''].\lambda x : D.\lambda y : D'.f(x, y),$$

and, because $f(x, y)$ is continuous in f, x, and y, our conclusion follows. But often it is more readable not to write equations between functions but rather equations between values for definitional purposes.

The proof of the principle is easy. For let the variable y, say range over D'' and let $S \subseteq D''$ be a directed subset. Then

$$\lambda x : D.\mathcal{E}(x, \bigsqcup S, z, \dots) = \lambda x : D.\bigsqcup \mathcal{E}(x, y, z, \dots) : y \in S \quad = \bigsqcup \lambda x : D.\mathcal{E}(x, y, z, \dots) : y \in S,$$

because the lubs of functions are computed pointwise. □

1.4.6 Definition 3.6

We need not enumerate the many corollaries that follow easily now from this result. We mention, however, that composition $f \circ g$ of functions (on continuous lattices) is continuous in the two function variables, where we write

$$(f \circ g)(x) = f(g(x)).$$

What will be useful will be to return at this point to a discussion of the injective properties of continuous lattices. If one continuous lattice is a subspace of another it is of course a retract. This relationship between spaces can be given by a pair of continuous maps

$$i : D \to D' \quad \text{and} \quad j : D' \to D,$$

where

$$j \circ i = \mathrm{id}_D = \lambda x : D.x.$$

The composition $i \circ j : D' \to i(D)$ is the retraction onto the subspace corresponding to D under i. Now if we have a diagram:

$$D \underset{j}{\overset{i}{\rightleftarrows}} D' f \searrow \qquad \swarrow \tilde{f} = f \circ j D''$$

the given continuous f is at once extendable from D to D' by the obvious definition of $\tilde{f}$. This $\tilde{f}$ is not the $\bar{f}$ used in the proof of 2.11, and it will be well to sort out the connections. On one side note that if f' is any function which extends f, then we have $f = f' \circ i$. But this implies

$$\tilde{f} = f \circ j = f' \circ i \circ j,$$

which shows that $\tilde{f}$ is a "degraded" version of f'. There is one situation where this type of degrading is especially nice.

3.6 Definition. A continuous lattice D is said to be a *projection* of a continuous lattice D' iff there are a pair of continuous maps

$$i : D \to D' \quad \text{and} \quad j : D' \to D$$

such that not only

$$j \circ i = \mathrm{id}_D,$$

but also

$$i \circ j \sqsubseteq \mathrm{id}_{D'}.$$

Thus, in case our retraction is a projection, we have $\tilde{f} \sqsubseteq f'$, which means that $\tilde{f}$ is the *minimal* extension of $f \in [D \to D'']$ to a function in $[D' \to D'']$. We will discuss the nature of $\tilde{f}$ in a moment. But before we do we pause to remark that the correspondence $f \rightsquigarrow \tilde{f}$ is *continuous*, and this fact is easily extended.

Retract. `IsContinuousLatticeRetraction` and `IsContinuousLatticeProjection` are this definition: Scott maps i and j with $j \circ i = \mathrm{id}$, and, for a projection, also $i \circ j \sqsubseteq \mathrm{id}$. The function-space and inverse-limit proofs are built on them. The subspace retract used for injective spaces is Proposition 1.4, and the idempotent representation on a power of $\mathbb{O}$ is Theorem 2.12.

`IsContinuousLatticeRetraction.lean`

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section
```

```
universe u v

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D_0 D_0' D_1 D_1' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D_0] [CompleteLattice D_0'] [CompleteLattice D_1] [
    CompleteLattice D_1']

/-- **Scott 1972, Definition 3.6.** A *retraction* of continuous lattices. -/
structure IsContinuousLatticeRetraction (D : Type u) (D' : Type v)
    [CompleteLattice D] [CompleteLattice D'] where
  incl : ScottMap D D'
  retr : ScottMap D' D
  retr_incl : forall  d, retr (incl d) = d

end DomainTheory.ContinuousLattice
```

1.4.7 Proposition 2.10 revisited

2.10 Proposition. A retract of a continuous lattice is a continuous lattice with the subspace topology agreeing with the induced topology.

The printed argument is flawed. The repair is discussed in the Milner correction.

`incl_preservesDirectedSup.lean`

```
module

public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeRetraction
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D_0 D_0' D_1 D_1' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D_0] [CompleteLattice D_0'] [CompleteLattice D_1] [
    CompleteLattice D_1']

namespace IsContinuousLatticeRetraction
```

```
/-- The inclusion of a retraction preserves directed suprema (it is a Scott map). -/
theorem incl_preservesDirectedSup (R : IsContinuousLatticeRetraction D D') :
    PreservesDirectedSup (R.incl : D  ->  D') :=
  fun _ hS hSdir => R.incl.preservesDirectedSup_coe _ hS hSdir

end IsContinuousLatticeRetraction
end DomainTheory.ContinuousLattice
```

1.4.8 Theorem 2.12

2.12 Theorem. The injective spaces are exactly the continuous lattices.

This theorem is an immediate consequence of the preceding results: an injective space is a retract of a power of $\mathbb{O}$. But $\mathbb{O}$ is a finite lattice $(\bot \sqsubseteq \top)$, and so the given space is a continuous lattice under its induced topology. On the other hand a continuous lattice is injective. It remains then to prove 2.9–2.11.

The lattice approach to injective spaces gives a completely "internal" characterization of them: in the first place the lattices are complete. Next we can define lattice theoretically:

$$x \ll y \quad \text{iff} \quad \text{whenever } y \sqsubseteq \bigsqcup Z \text{ and } Z \subseteq D \text{ is directed,}$$

then $x \sqsubseteq z$ for some $z \in Z$.

Finally we assume that for all $y \in D$:

$$y = \bigsqcup \{x \in D : x \ll y\}.$$

That makes D a continuous lattice with the sets $\{y \in D : x \ll y\}$ as a basis for the topology. Such T_0-spaces are injective and every injective space can be obtained in this way with the lattice structure being uniquely determined by the topology. Furthermore, as we have seen, the injective property can be exhibited, as in the proof of 2.11, by an explicit formula for extending functions.

The retract approach to injective spaces should also be considered. The Cartesian powers $\mathbb{O}^I$ are very simple spaces; indeed, as lattices these are just the Boolean algebras of *all subsets* of I (that is, isomorphic thereto). The topology has as a basis the classes of sets containing given finite sets (the *weak topology*, cf. Nerode (1959)). A continuous function

$$j : \mathcal{P}I \to \mathcal{P}I$$

is one of "finite character" so that

$$j(X) = \bigcup \{j(F) : F \subseteq X\}$$

where $X \subseteq I$ and F ranges over *finite sets*. Such a function j is a *retraction* iff it is an *idempotent*:

$$j \circ j = j,$$

which means that the range of j is the set of fixed points of j. As we have seen

$$D = \{X \in \mathcal{P}I : j(X) = X\}$$

is a continuous lattice (under $\subseteq$ in this case), and *every* continuous lattice is isomorphic to one obtained in this way. This provides a representation theorem of sorts for continuous lattices, but it does not seem to be of too much help in proving theorems.

The idempotent map on a power of $\mathbb{O}$ in this paragraph is `IdemFix`, the fixed-point set used in the proof. A retract of an injective space, in the subspace sense, is Proposition 1.4. A retraction and a projection of continuous lattices are Definition 3.6.

Note. The paper states this theorem in section 2, before function spaces, as a consequence of Propositions 2.9–2.11. The formalized proof depends on the corrected Proposition 2.10. That dependence on the retraction filed with section 3 was observed after the paper. The statement keeps the number 2.12 and is placed immediately after that corrected proof.

`isContinuousLattice_prop.lean`

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import Mathlib.Order.CompleteLattice.Basic
public import DomainTheory.ContinuousLattice.Constructions.proposition_2_8
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

variable {D : Type*} [CompleteLattice D]

/-- The Sierpinski space `Prop` (Scott's ` O `) is a continuous lattice: it is a finite
     complete
lattice, so Proposition 2.8 applies. -/
theorem isContinuousLattice_prop : ContinuousLattice Prop :=
  proposition_2_8

end DomainTheory.ContinuousLattice
```

1.4.9 Proposition 3.7

3.7 Proposition. Suppose the two pairs of maps

$$i_n : D_n \to D'_n \quad \text{and} \quad j_n : D'_n \to D_n$$

for $n = 0, 1$ make D_n a retraction (projection) of D'_n. Then $[D_0 \to D_1]$ is also a retraction (projection) of $[D'_0 \to D'_1]$ by means of the pair of maps:

$$\vec{i}(f) = i_1 \circ f \circ j_0 \text{ , and}$$

$$\vec{j}(f') = j_1 \circ f' \circ i_0 \text{ ,}$$

where $f \in [D_0 \to D_1]$ and $f' \in [D'_0 \to D'_1]$. $\square$

proposition_3_7_projection.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMapProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D_0 D_0' D_1 D_1' : Type*}

variable [TopologicalSpace X] [TopologicalSpace Y]

variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']

variable [CompleteLattice D_0] [CompleteLattice D_0'] [CompleteLattice D_1] [
    CompleteLattice D_1']

/-- **Scott 1972, Proposition 3.7 (projection).** Function spaces inherit projections.
    -/
def proposition_3_7_projection (P_0 : IsContinuousLatticeProjection D_0 D_0')
    (P_1 : IsContinuousLatticeProjection D_1 D_1') :
    ScottMapProjection D_0 D_0' D_1 D_1' :=
  ScottMapProjection.functionSpace P_0 P_1

end DomainTheory.ContinuousLattice
```

1.4.10 Proposition 3.8

Returning now to $\bar{f}$ we can prove:

3.8 Proposition. If D is a continuous lattice and $e : X \to Y$ a subspace embedding, then for each $f : X \to D$, the function $\bar{f} : Y \to D$ given by the formula

$$\bar{f}(y) = \bigsqcup \{ \sqcap \{ f(x) : e(x) \in U : y \in U \},$$

where U ranges over open subsets of Y and x over X, is the maximal extension of f to a function in the partially ordered set $[Y \to D]$.

Proof: We are saying that $\bar{f}$ is the maximal solution to the equation

$$f = \bar{f} \circ e.$$

We already know it is a solution, so let f' be any other. We have

$$f'(y) = \bigsqcup \{ \sqcap \{ f'(z) : z \in U : y \in U \} \sqsubseteq \bigsqcup \{ \sqcap \{ f'(z) : z \in e(X) \cap U : y \in U \} = \bigsqcup \{ \sqcap \{ f'(e(x)) : e(x) \in U : y \in U \} = \bigsqcup \{ \sqcap f$$

which establishes that $f' \sqsubseteq \bar{f}$. $\square$

The reader should not forget that any space (any *given* number of spaces X, Y, ...) can be found as a *subspace* of a continuous lattice D. Since D is injective any continuous function $f : X \to Y$ can be extended to a continuous function $\bar{f} : D \to D$. Thus the continuous functions from D into D are a rich totality including *all* the structure of continuous functions on *all* the subspaces. And this remark brings us to the study of function spaces.

Subspace. There is no separate subspace type. A subspace is Mathlib's `IsEmbedding`. `IsRetractSubspace` requires its section to be an embedding, and this proposition extends a continuous map along an embedding into a continuous lattice by `scottExtend`. The function-space development writes the same formula as `scottSubspaceExtend`.

proposition_3_8.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.Specialization.scottTopologicalSpace
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_maximal
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_continuous
public import DomainTheory.ContinuousLattice.Constructions.
    scottExtend_eq_of_continuous
public import DomainTheory.ContinuousLattice.Constructions.scottExtend
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Topology Set

variable {D : Type*} [CompleteLattice D]

section InjectiveExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] {E : Type*} [
    CompleteLattice E]

/-- **Scott 1972, Proposition 3.8.** If `E` is a continuous lattice and `e : X  ->  Y`
    a subspace
embedding, then for each continuous `f : X  ->  E` the explicit formula `scottExtend e
    f` is the
*maximal extension* of `f` to `[Y  ->  E]`: it is Scott-continuous (`
    scottExtend_continuous`), it
restricts to `f` along `e` (`scottExtend_eq_of_continuous`), and it dominates every
    continuous
solution of `f'  comp  e = f` (`scottExtend_maximal`). -/
theorem proposition_3_8 (hE : ContinuousLattice E) (e : X  ->  Y) (he : IsEmbedding e)
    (f : X  ->  E) (hf : @Continuous X E _ scottTopologicalSpace f) :
    @Continuous Y E _ scottTopologicalSpace (scottExtend e f)
       /\  (forall  x, scottExtend e f (e x) = f x)
       /\  (forall  f' : Y  ->  E, @Continuous Y E _ scottTopologicalSpace f'  ->  (
    forall  x, f' (e x) = f x)  ->
          forall  y, f' y  <=  scottExtend e f y) :=
  <scottExtend_continuous hE e f,
```

```
   fun x => scottExtend_eq_of_continuous hE e he f hf x,
   fun _f' hf' h_ext y => scottExtend_maximal hE e hf' h_ext y>

end InjectiveExtension
end DomainTheory.ContinuousLattice
```

1.4.11 Lemma 3.9

By the same argument we could show that $\bar{f}$ is the maximal solution of $\bar{f} \circ e \sqsubseteq f$. An interesting question is whether the correspondence $f \rightsquigarrow \bar{f}$ is continuous. I very much doubt it, but at this moment a counterexample escapes me. It is clear that the correspondence is monotonic, for if $f \sqsubseteq g$, then the formula of 3.8 shows that $\bar{f} \sqsubseteq \bar{g}$. This gives us a neat argument for the previous remark: if $g \circ e \sqsubseteq f$, then

$$\overline{g \circ e} \sqsubseteq \bar{f}.$$

But $g \circ e = \overline{g \circ e} \circ e$, so by 3.8, $g \sqsubseteq \overline{g \circ e}$, and $\bar{f}$ is thus maximal.

In the case that the range spaces are being extended, the following lemma relating the extensions will be very useful when we consider inverse limits.

3.9 Lemma. Consider the diagram:

$$X \xrightarrow{e} Y \qquad f \downarrow \qquad g \searrow \qquad \bar{f} \swarrow \qquad \downarrow \bar{g} \qquad D \xrightarrow[j]{i} D'$$

where the upper row is a subspace embedding and the lower is a projection. If the given functions f and g are extended to $\bar{f}$ and $\bar{g}$ as in 3.8, and if $f = j \circ g$, then $\bar{f} = j \circ \bar{g}$ also.

Proof: $\bar{f}$ and $\bar{g}$ are maximal solutions of $f = \bar{f} \circ e$ and $g = \bar{g} \circ e$. Therefore since

$$f = j \circ g = j \circ \bar{g} \circ e,$$

we see that

$$j \circ \bar{g} \sqsubseteq \bar{f}.$$

Note also that

$$i \circ \bar{f} \circ e = i \circ f = i \circ j \circ g \sqsubseteq g,$$

and so by the remark following 3.8, we have

$$i \circ \bar{f} \sqsubseteq \bar{g}.$$

Therefore

$$\bar{f} = j \circ i \circ \bar{f} \sqsubseteq j \circ \bar{g},$$

which proves the equality. □

lemma_3_9_incl_inf.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono_incl_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono_incl
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtendInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D_0 D_0' D_1 D_1' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D_0] [CompleteLattice D_0'] [CompleteLattice D_1] [
    CompleteLattice D_1']

section SubspaceExtension

variable {X Y : Type*} [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y]

omit [TopologicalSpace X] [TopologicalSpace Y] [CompleteLattice Y] in
/-- **Scott 1972, Lemma 3.9 (inclusion at infimum level).** Used in the inverse-limit
argument (Theorem 4.4). -/
theorem lemma_3_9_incl_inf {e : X  ->  Y} (P : IsContinuousLatticeProjection D D')
    {f : X  ->  D} {g : X  ->  D'} (hfg : forall  x, f x = P.retr (g x)) (U : Set Y) :
    P.incl (scottSubspaceExtendInf e f U)  <=  scottSubspaceExtendInf e g U :=
  le_trans (scottSubspaceExtendInf_mono_incl_apply (e := e) (f := f) U P)
    (scottSubspaceExtendInf_mono_incl (e := e) (f := f) (g := g) P hfg U)

end SubspaceExtension
end DomainTheory.ContinuousLattice
```

1.4.12 Proposition 3.10

Whether this lemma is true for retractions in any form, I do not know. My proof seems to require the stronger projection relationship. I suspect there may be difficulties. In general projections are better behaved than retractions. By the way the word projection seems to be properly used in 3.6, for the projection $j : D \times D' \to D$ of the Cartesian product of two continuous lattices onto the first factor *is* a projection with partial inverse $i : D \to D \times D'$ defined by

$$i(x) = (x, \bot)$$

for $x \in D$.

3.10 Proposition. If the continuous lattice D is a projection of the continuous lattice D' via the pair of maps i, j; then for all $S \subseteq D$ and all $x, y \in D$ we have:

(i) $i(\bigsqcup S) = \bigsqcup\{i(x) : x \in S\}$,

(ii) $i(x) = i(y)$ implies $x = y$,

(iii) $x \ll y$ implies $i(x) \ll i(y)$.

Conversely, if a map $i : D \to D'$ satisfies (i)–(iii), then there exists a continuous $j : D' \to D$ making D a projection of D', and in fact j is uniquely determined by:

(iv) $j(x') = \bigsqcup\{x \in D : i(x) \sqsubseteq x'\}$

for all $x' \in D'$.

Proof: Equation (i) holds for directed $S \subseteq D$ because i is continuous. To have it hold for arbitrary S it is only necessary to check it for finite sets, because every lub is the directed lub of finite sublubs. (The last word of that sentence is an unfortunate accident.) Further, to check the equation for finite sets it is enough to check it for the empty set and for two element sets. Thus, $i(\bot) = \bot$, because $j(i(\bot)) = \bot$ and since $\bot \sqsubseteq i(\bot)$,

$$j(\bot) \sqsubseteq j(i(\bot)) = \bot,$$

so $j(\bot) = \bot$. Whence $i(\bot) = i(j(\bot)) \sqsubseteq \bot$. Next $i(x \sqcup y) = i(x) \sqcup i(y)$, because first

$$i(x) \sqcup i(y) \sqsubseteq i(x \sqcup y)$$

by monotonicity. Then note that

$$i(x) \sqsubseteq i(x) \sqcup i(y)$$

and so

$$x = j(i(x)) \sqsubseteq j(i(x) \sqcup i(y)).$$

Similarly

$$y \sqsubseteq j(i(x) \sqcup i(y)),$$

whence

$$x \sqcup y \sqsubseteq j(i(x) \sqcup i(y)).$$

But then

$$i(x \sqcup y) \sqsubseteq i(j(i(x) \sqcup i(y))) \sqsubseteq i(x) \sqcup i(y),$$

which completes the argument for equation (i).

Condition (ii) is obvious. For (iii) we argue as follows. Assume $x \ll y$. Since D' is continuous we can write:

$$i(y) = \bigsqcup\{z' \in D' : z' \ll i(y)\},$$

and conclude by the continuity of j that:

$$y = j(i(y)) = \bigsqcup\{j(z') : z' \ll i(y)\}.$$

But $x \ll y$, so $x \ll j(z')$ for some $z' \ll i(y)$. Now $x \sqsubseteq j(z')$ follows; therefore $i(x) \sqsubseteq i(j(z')) \sqsubseteq z'$. Thus $i(x) \ll i(y)$.

Turning now to the converse, assume of the map i that it satisfies (i)–(iii). Compute:

$$i(j(x')) = \bigsqcup\{i(x) : i(x) \sqsubseteq x'\} \sqsubseteq x'.$$

This is correct because i is continuous and the set $\{x : i(x) \sqsubseteq x'\}$ is directed in view of condition (i). Thus $i \circ j \sqsubseteq \mathrm{id}_{D'}$. Note that by virtue of (i) and (ii) it is the case that

$$i(x) \sqsubseteq i(y) \text{ implies } x \sqsubseteq y.$$

(The reason is that $x \sqsubseteq y$ is equivalent to $x \sqcup y = y$.) This remark allows us to compute:

$$\begin{aligned} j(i(y)) &= \bigsqcup\{x : i(x) \sqsubseteq i(y)\} \\ &= \bigsqcup\{x : x \sqsubseteq y\} = y. \end{aligned}$$

Hence, $j \circ i = \mathrm{id}_D$. It remains to show that j is continuous. Suppose $S' \subseteq D'$ is directed. Since j is by definition monotonic, it is sufficient to prove that

$$j(\bigsqcup S') \sqsubseteq \bigsqcup\{j(x') : x' \in S'\}.$$

Now

$$j(\bigsqcup S') = \bigsqcup\{x : i(x) \sqsubseteq \bigsqcup S'\},$$

so suppose $i(x) \sqsubseteq \bigsqcup S'$. Let $z \ll x$; whence $i(z) \ll i(x)$. Thus $i(z) \ll x'$ for some $x' \in S'$, and therefore $i(z) \sqsubseteq x'$. We obtain then $z \sqsubseteq j(x')$, which means that

$$z \sqsubseteq \bigsqcup\{j(x') : x' \in S'\}$$

holds for all $z \ll x$. By the continuity of D we conclude

$$x \sqsubseteq \bigsqcup\{j(x') : x' \in S'\}$$

holds for all x with $i(x) \sqsubseteq \bigsqcup S'$. The desired result follows. □

proposition_3_10_converse.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.converseProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Finset.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D_0 D_0' D_1 D_1' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D_0] [CompleteLattice D_0'] [CompleteLattice D_1] [
    CompleteLattice D_1']

section Proposition310

open Finset

variable {i : D  ->  D'}

/-- **Scott 1972, Proposition 3.10 (converse).** If `i : D  ->  D'` (between continuous
     lattices)
satisfies (i) preservation of arbitrary suprema, (ii) injectivity, and (iii)
    preservation of ` << `,
then there is a continuous `j` making `D` a projection of `D'` via `i`, with `j` given
    by Scott's
formula (iv) `j(x') =  sqsup  { x | i(x)  sqsub  x' }`. -/
theorem proposition_3_10_converse
    (hi : forall  S : Set D, i (sSup S) = sSup (i '' S)) (hinj : Function.Injective i)
    (hwb : forall  {x y : D}, x  <<  y  ->  i x  <<  i y) (hD : ContinuousLattice D) :
    exists  P : IsContinuousLatticeProjection D D',
      (forall  d, (P.incl : D  ->  D') d = i d)
        /\  (forall  x', (P.retr : D'  ->  D) x' = sSup {x | i x  <=  x'}) :=
  <converseProjection hi hinj hwb hD, fun _ => rfl, fun _ => rfl>

end Proposition310
end DomainTheory.ContinuousLattice
```

1.4.13 Definition 3.11: Projection space

As a corollary of 3.10 we can easily see which *subspaces* of a continuous lattice D' are projections of it. Such a subspace $D \subseteq D'$ must first be closed under $\bigsqcup$. That is, if $S \subseteq D$, then $\bigsqcup S \in D$ for

all S, where the lub is taken in the sense of D'. The identity map on D will then satisfy (i) and (ii). But this is not enough, since we would not know that D is a continuous lattice, nor whether (iii) holds. The following additional condition would be sufficient, if assumed for all $y \in D$:

$$y = \bigsqcup x \in D : x \ll y,$$

where $\ll$ is taken in the sense of D'. This implies that

$$y = \bigsqcup \sqcap (D \cap U) : y \in U$$

where U ranges over the open subsets of D' and where the $\sqcap$ is taken in the sense of D. This condition makes the subspace topology the same as the lattice topology on D and besides makes D continuous, which

is just what we need. (Another way to put it is that whenever $z \ll y$, where $y \in D$ but $z \in D'$, then $z \sqsubseteq x \ll y$, for some $x \in D$.)

It seems a bit troublesome to characterize in a simple way just which maps $j : D' \to D$ are projections. (Other than saying outright that the map $i : D \to D'$ such that for all $x \in D$:

$$i(x) = \sqcap x' \in D' : x \sqsubseteq j(x')$$

is the continuous partial inverse to j.) But we can say very easily which continuous maps $j : D' \to D'$ are projections onto *subspaces*; namely, we must have

$$j = j \circ j \sqsubseteq \mathrm{id}.$$

The subspace in question then is:

$$D = x \in D' : j(x) = x.$$

This non-empty subspace is the exact range of j and is closed under $\bigsqcup$. Let $y \in D$. Then if $x' \ll y$ in D', we find $j(x') \sqsubseteq x' \ll y$. Thus since

$$y = \bigsqcup x' \in D' : x' \ll y,$$

we see that

$$y = j(y) = \bigsqcup j(x') : x' \ll y.$$

But each $j(x') \in D$, so $y = \bigsqcup x \in D : x \ll y$, as desired.

The foregoing discussion suggests looking more closely at the space of all projections of a continuous lattice since they are so easily characterized.

3.11 Definition. Given a continuous lattice D, we let the space of projections be denoted by:

$$J_D = j \in [D \to D] : j = j \circ j \sqsubseteq \mathrm{id}.$$

1.4.14 Proposition 3.12

3.12 Proposition. For a continuous lattice D the space J_D of projections forms a complete lattice as a $\bigsqcup$-closed subspace of $[D \to D]$.

Proof: The constant function $\bot \in J_D$ obviously, so J_D contains $\bigsqcup \emptyset$. Suppose $j, k \in J_D$. We wish to show that $j \sqcup k \in J_D$. Compute:

$$(j \sqcup k)((j \sqcup k)(x)) = j(j(x) \sqcup k(x)) \sqcup k(j(x) \sqcup k(x)).$$

But note:

$$j(x) \sqsubseteq j(j(x)) \sqsubseteq j(j(x) \sqcup k(x)) \sqsubseteq j(x),$$

because $j(x) \sqcup k(x) \sqsubseteq x$. Similarly for $k(x)$. Therefore, we find that $(j \sqcup k) \circ (j \sqcup k) = j \sqcup k \sqsubseteq \mathrm{id}$.

Suppose finally that $S \subseteq J_D$ is directed. We wish to show that $\bigsqcup S \in J_D$. Clearly $\bigsqcup S \sqsubseteq \mathrm{id}$, so compute by continuity of $\circ$:

$$\bigsqcup S \circ \bigsqcup S = \bigsqcup j \circ j : j \in S = \bigsqcup j : j \in S = \bigsqcup S.$$

It follows that J_D is $\bigsqcup$-closed and hence is a complete lattice. □

proposition_3_12.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.instCompleteLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.Projections
public import DomainTheory.ContinuousLattice.FunctionSpaces.isProjection_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D_0 D_0' D_1 D_1' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D_0] [CompleteLattice D_0'] [CompleteLattice D_1] [
    CompleteLattice D_1']

section Projections

/-- **Scott 1972, Proposition 3.12.** For a (complete, in particular continuous)
    lattice `D`, the
projections `J_D` form a complete lattice as a ` sqsup `-closed subspace of `[D  ->  D
    ]`: the ` sqsup `-closure is
`isProjection_sSup`, and the complete-lattice structure is `Projections.
    instCompleteLattice`. -/
theorem proposition_3_12 :
    (forall  T : Set (ScottMap D D), (forall  j  in  T, IsProjection j)  ->
    IsProjection (sSup T))
       /\  Nonempty (CompleteLattice (Projections D)) :=
  <fun _ h => isProjection_sSup h, <Projections.instCompleteLattice>>

end Projections
end DomainTheory.ContinuousLattice
```

1.4.15 Proposition 3.13

The significance of the above result becomes clearer if we consider the connection between projections and subspaces. Let us write:

$$D(j) = x \in D : j(x) = x.$$

For $j \in J_D$, each $D(j)$ is a projection of D onto a subspace. We show first that

$$j \sqsubseteq k \quad \text{iff} \quad D(j) \subseteq D(k).$$

Because if $j \sqsubseteq k$, then $j \sqsubseteq j \circ j \sqsubseteq k \circ j \sqsubseteq \mathrm{id} \circ j = j$. Therefore if $j(x) = x$, then $k(x) = k(j(x)) = j(x) = x$, which means that $D(j) \subseteq D(k)$. On the other hand, if $D(j) \subseteq D(k)$, then since $j(D) \subseteq D(j)$,

we see that $k \circ j = j$, and so $j \sqsubseteq k \circ \mathrm{id} = k$. Hence J_D is isomorphic to the partially ordered set of subspaces of D that are projections. We thus conclude that these subspaces form a lattice. In fact, it is easy to show that

$$D(j \sqcup k) = x \sqcup y : x \in D(j), y \in D(k).$$

Similarly, if S is a directed set of J_D, then $D(\bigsqcup S)$ is the $\bigsqcup$-closure in D of the subset:

$$\bigcup D(j) : j \in S.$$

These are not very deep facts, but their proofs were very much facilitated by the introduction of J_D and the utilization of the lattice structure of $[D \to D]$. Along the same line we can define for $j, k \in J_D$ a function $(j \to k) \in [D \to D] \to [D \to D]$ by the equation

$$(j \to k)(f) = k \circ f \circ j.$$

It is very easy to show that $(j \to k) \in J_{[D \to D]}$, that $(j \to k)$ is continuous in j and k, and that $[D \to D](j \to k)$ is isomorphic to $[D(j) \to D(k)]$. There are many other interesting operations on projections corresponding to other constructs besides these. And, just as with $(j \to k)$, the operations are continuous. This makes it possible to prove existence theorems about subspaces by using results like the fixed-point theorem for continuous functions. It would be even nicer if J_D turns out to be a continuous lattice itself, but as far as I can tell this is not likely to be the case.

Before we turn to the iterated function-space construction by inverse limits, there are a couple of other connections between spaces and function spaces that are useful to know.

3.13 Proposition. Every continuous lattice D is a projection of its function space $[D \to D]$.

Proof: Consider the following pair of mappings $\mathrm{con} : D \to [D \to D]$ and $\mathrm{min} : [D \to D] \to D$ where

$$\mathrm{con}(x)(y) = x$$

and

$$\mathrm{min}(f) = f(\bot)$$

for all $x, y \in D$ and $f \in [D \to D]$. They are obviously continuous. The map con matches every element of D with the corresponding *constant* function in $[D \to D]$. The map min associates to every function in $[D \to D]$ its *minimum* value in the partial ordering. The proof that this pair forms a projection is trivial. □

proposition_3_13.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.projection
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D_0 D_0' D_1 D_1' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D_0] [CompleteLattice D_0'] [CompleteLattice D_1] [
    CompleteLattice D_1']

/-- **Scott 1972, Proposition 3.13.** Every continuous lattice `D` is a projection of
    its function
space `[D  ->  D]`, via `con`/`min` (`Proposition313.projection`). -/
theorem proposition_3_13 (_hD : ContinuousLattice D) :
    Nonempty (IsContinuousLatticeProjection D (ScottMap D D)) :=
  <Proposition313.projection>

end DomainTheory.ContinuousLattice
```

1.4.16 Proposition 3.14

The pair $\mathrm{con}, \mathrm{min}$ are not the only pair for making D a projection of $[D \to D]$. The following pair of maps were suggested by David Park:

$$\lambda x.\vec{e}[t, x] \quad \text{and} \quad \lambda f.f(t),$$

where x ranges over D, and f over $[D \to D]$, and where t is a fixed *isolated* element of D (that is, $t \ll t$ holds). The pair con and min will result if we set $t = \bot$. (Note that the expression $\vec{e}[t, x]$, though continuous in x, is not continuous—or even monotonic—in the variable t.) A lattice may very well possess a large number of isolated

elements, whence a large number of projections. And furthermore this is the only way the function $j = \lambda f.f(t)$ can be a projection. For assume the existence of an inverse $i : D \to [D \to D]$ satisfying the proper conditions. Then it would be the case that

$$i(x)(t) = x$$

and

$$i(f(t))(y) \sqsubseteq f(y)$$

for all $x, y \in D$ and all $f \in [D \to D]$. We can prove for all $u \in D$, if $t \not\sqsubseteq u$, then

$$i(x)(u) = \bot$$

by substituting $\vec{e}[v, x]$ for f in the second equation above, where v is chosen so that $v \ll t$ but not $v \ll u$. But then note that

$$i(x)(t) = \bigsqcup i(x)(u) : u \ll t.$$

If not $t \ll t$, then $u \ll t$ implies $t \not\sqsubseteq u$, which leads to absurdity. Hence t must be isolated, and, as we noted earlier, the function i is uniquely determined as being the one we already knew. Aside from these pairs of projections one could obtain others by combinations with automorphisms. I was unable to determine whether there are further pairs of an essentially different nature.

The topic of projections in these spaces is rather interesting since one has in a way more freedom in T_0-spaces (particularly in injective spaces) than in ordinary spaces for defining functions. As another example, consider the Cartesian square $D \times D$. Aside from the two obvious projections onto D, there is also the "diagonal" system given by the pair:

$$\lambda x.(x, x) \qquad \text{and} \qquad \lambda(x, y).x \sqcap y$$

We shall note in the next section how the choice of an initial projection effects the construction of an inverse limit.

The projections are not the only useful functions in $[D \to D] \to D$. As a final example of what can be done in function spaces we mention the fixed-point operator.

3.14 Proposition. *For a continuous lattice D there is a uniquely determined continuous mapping*

$$\mathrm{fix} : [D \to D] \to D$$

where for all $f \in [D \to D]$ and $x \in D$

$$f(\mathrm{fix}(f)) = \mathrm{fix}(f)$$

and whenever $f(x) \sqsubseteq x$, then $\mathrm{fix}(f) \sqsubseteq x$.
and whenever $f(x) \sqsubseteq x$, then

$$\mathrm{fix}(f) \sqsubseteq x.$$

Proof: The proof of the existence of minimal fixed points in complete lattices is well known, as was mentioned in the proof of 2.10.

To establish the continuity, it is sufficient to remark that since all functions $f \in [D \to D]$ are continuous, we have

$$\mathrm{fix}(f) = \bigsqcup_{n=0}^{\infty} f^n(\bot),$$

where $f^n(x) = f(f(\ldots f(x)\ldots))$ (n times). Thus fix is the point-wise lub of continuous functions on $[D \to D]$ and is thus itself continuous. □

proposition_3_14.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_unique
public import DomainTheory.ContinuousLattice.FunctionSpaces.fixMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_le
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_eq
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Data.Set.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set Topology TopologicalSpace

variable {X Y D D' D'' D_0 D_0' D_1 D_1' : Type*}
variable [TopologicalSpace X] [TopologicalSpace Y]
variable [CompleteLattice D] [CompleteLattice D'] [CompleteLattice D'']
variable [CompleteLattice D_0] [CompleteLattice D_0'] [CompleteLattice D_1] [
    CompleteLattice D_1']

/-- **Scott 1972, Proposition 3.14.** For a continuous lattice `D` there is a uniquely
    determined
continuous mapping `fix : [D  ->  D]  ->  D` such that `f (fix f) = fix f` for all `f`,
     and `fix f  sqsub  x`
whenever `f x  sqsub  x`. Existence and continuity are `Proposition314.fixMap`; the
    defining equations are
`fix_eq`/`fix_le`; uniqueness (any operator with these two properties agrees with `fix
    `) is
`fix_unique`. -/
theorem proposition_3_14 (_hD : ContinuousLattice D) :
    exists  Fix : ScottMap (ScottMap D D) D,
      (forall  f : ScottMap D D, (f : D  ->  D) (Fix f) = Fix f)
         /\  (forall  (f : ScottMap D D) (x : D), (f : D  ->  D) x  <=  x  ->  (Fix f
    : D)  <=  x)
         /\  (forall  g : ScottMap D D  ->  D,
            (forall  f : ScottMap D D, (f : D  ->  D) (g f) = g f)  ->
            (forall  (f : ScottMap D D) (x : D), (f : D  ->  D) x  <=  x  ->  g f  <=
     x)  ->
            forall  f : ScottMap D D, g f = Fix f) :=
  <Proposition314.fixMap, fun f => Proposition314.fix_eq f,
    fun _ _ h => Proposition314.fix_le h,
    fun _ hfix hleast f => Proposition314.fix_unique (hfix f) (fun x hx => hleast f x
    hx)>

end DomainTheory.ContinuousLattice
```

1.5 Inverse limits

Dependency diagram. The arrowhead is on the note that imports. The number is how many direct imports. An arrow already implied by a longer path is left out.

1.5.1 Inverse limits

Scott's section begins with an arbitrary inverse system and then specializes to continuous lattices and projections. The construction is divided into the concepts below. The numbered results follow in his order, and the rest of the section is the discussion after those results.

Dependency diagram. Each box is one of the construction notes. The arrowhead is on the note that imports. The number is how many direct imports. An arrow already implied by a longer path is left out.

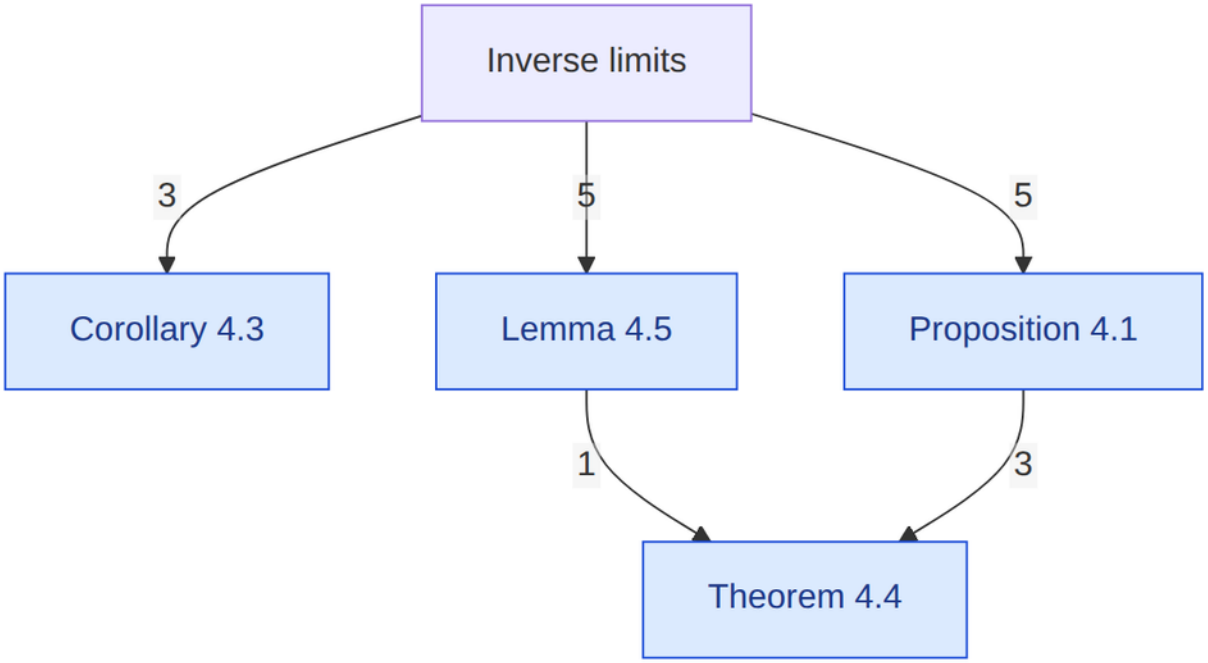


Figure 9: Inverse limits.

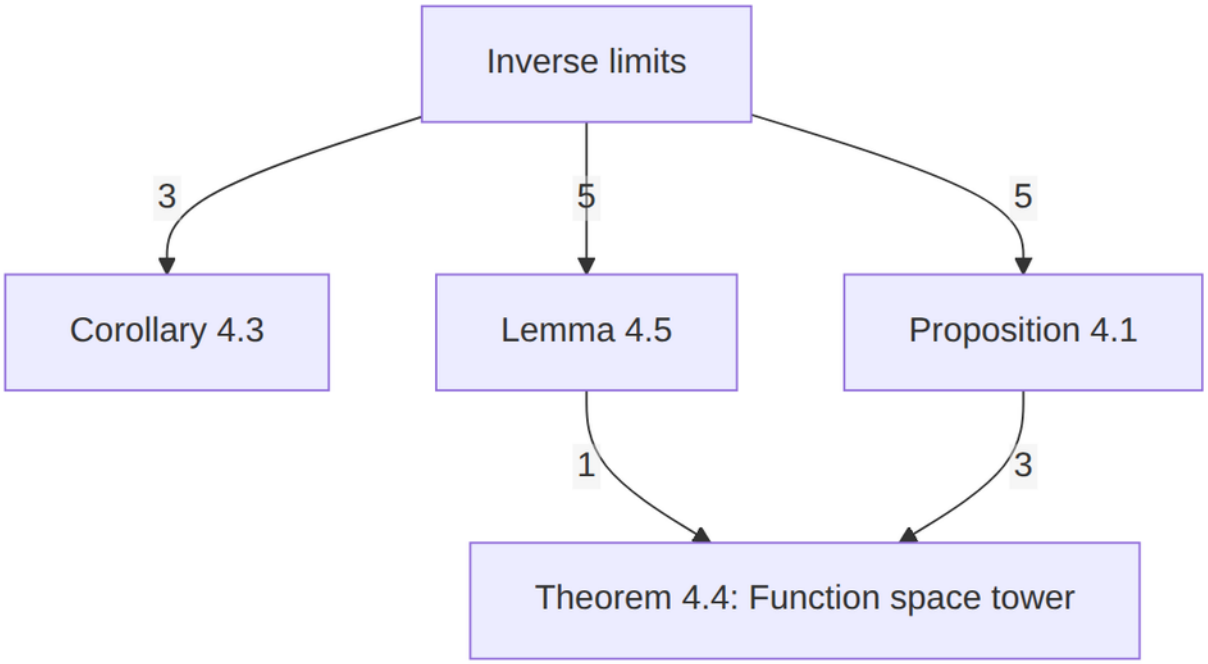


Figure 10: Inverse limits.

The inverse system `Compatible` is a thread through the stages, and `InverseLimit` is the space of those threads.

4. INVERSE LIMITS. By an *inverse system of spaces* we understand as usual a sequence

$$\langle X_n, j_n \rangle_{n=0}^{\infty}$$

of T_0-spaces and continuous maps $j_n : X_{n+1} \to X_n$. The space X_∞, called the *inverse limit* of the sequence, is constructed in the familiar way as that subspace of the product space consisting of exactly those infinite sequences

$$x = \langle x_n \rangle_{n=0}^{\infty},$$

where for each n we have $x_n \in X_n$, and

$$j_n(x_{n+1}) = x_n.$$

Dependency diagram. Each box is one of these notes. The arrowhead is on the note that imports. The number is how many direct imports. An arrow already implied by a longer path is left out.

Compatible `Compatible` is a thread through the stages: each projection sends the next term to the current term.

Compatible.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : Nat  ->  Type u) [forall  n, CompleteLattice (D n)]

variable (P : forall  n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Compatibility of a sequence: `j_n(x_{n+1}) = x_n` for all `n`. -/
def Compatible (x : forall  n, D n) : Prop := forall  n, (P n).retr (x (n + 1)) = x n

end InverseLimit

end DomainTheory.ContinuousLattice
```

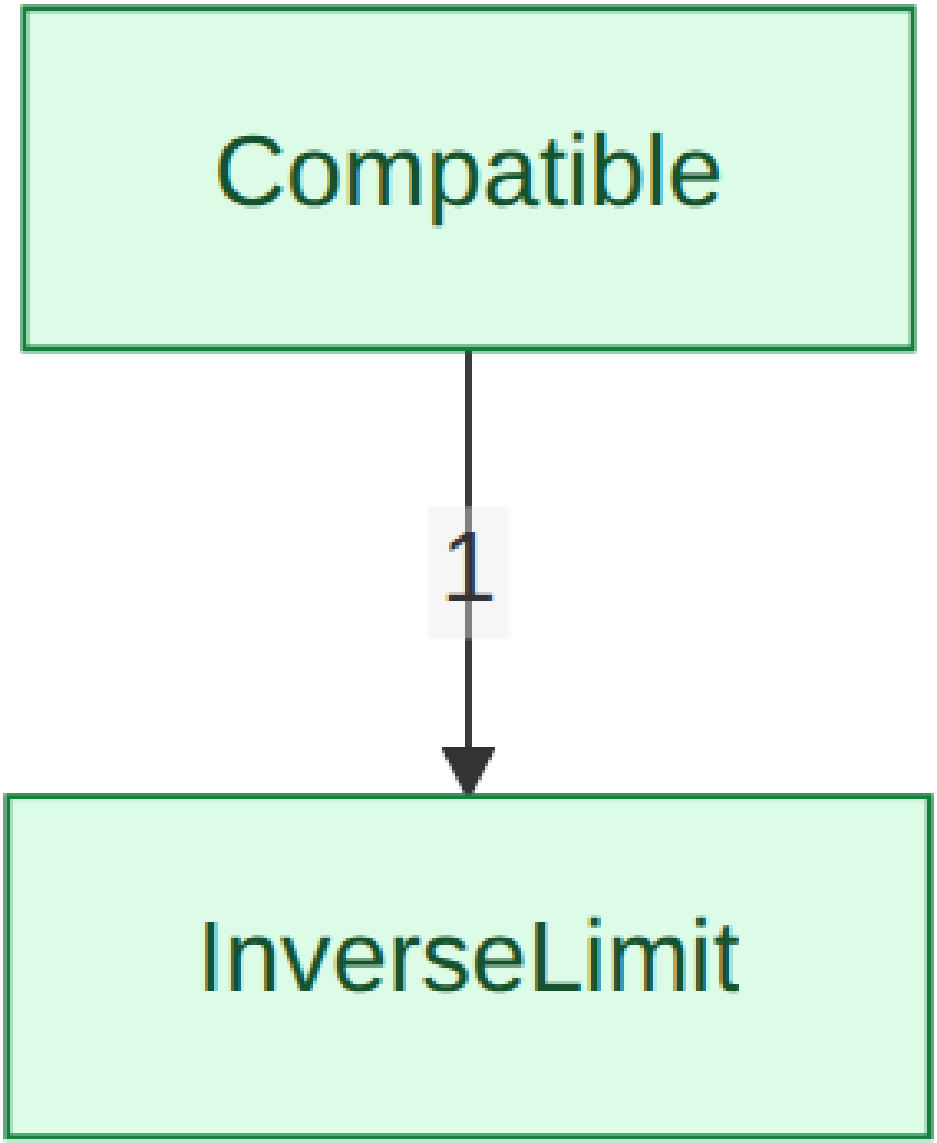


Figure 11: Inverse limits.

InverseLimit `InverseLimit` is the space of Compatible threads.

`InverseLimit.lean`

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.Compatible
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : Nat  ->  Type u) [forall  n, CompleteLattice (D n)]

variable (P : forall  n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- **Scott 1972, S4.** The inverse limit `D_ inf ` as the subspace of compatible
    sequences. -/
abbrev InverseLimit : Type u := {x : forall  n, D n // Compatible D P x}

end InverseLimit

end DomainTheory.ContinuousLattice
```

projLE `projLE` is the composite of the bonding projections from a later stage down to an earlier one. The lemmas are `projLE.self`, `projLE.succ`, `projLE.mono`, `projLE.compatible`, `projLE.retr`, and `projLE.preservesDirectedSup`.

`projLE.lean`

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.comp
public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u
```

```
section InverseLimit

variable (D : Nat  ->  Type u) [forall  n, CompleteLattice (D n)]

variable (P : forall  n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Descend the tower of projections: for `m  <=  n`, `projLE h = j_m  comp  ...  comp
    j_{n-1} : D_n  ->  D?`. -/
def projLE {m n : Nat} (h : m  <=  n) (x : D n) : D m :=
  Nat.leRecOn (C := fun k => D k  ->  D m) h
    (fun {k} (g : D k  ->  D m) (w : D (k + 1)) => g ((P k).retr w)) id x

namespace projLE

theorem self {m : Nat} (h : m  <=  m) (x : D m) : projLE D P h x = x := by
  simp only [projLE, Nat.leRecOn_self, id_eq]

theorem succ {m n : Nat} (h1 : m  <=  n) (h2 : m  <=  n + 1) (z : D (n + 1)) :
    projLE D P h2 z = projLE D P h1 ((P n).retr z) := by
  simp only [projLE]
  rw [Nat.leRecOn_succ (C := fun k => D k  ->  D m) h1]

theorem mono {m n : Nat} (h : m  <=  n) {x y : D n} (hxy : x  <=  y) :
    projLE D P h x  <=  projLE D P h y := by
  induction n, h using Nat.le_induction with
  | base =>
      rw [self, self]
      exact hxy
  | succ k hk ih =>
      rw [succ D P hk (Nat.le_succ_of_le hk), succ D P hk (Nat.le_succ_of_le hk)]
      exact ih ((P k).retr.monotone hxy)

/-- For a compatible sequence `y`, descending from level `n` to `m  <=  n` recovers `y
    ?`. -/
theorem compatible {m : Nat} (y : InverseLimit D P) :
    forall  {n : Nat} (h : m  <=  n), projLE D P h (y.1 n) = y.1 m := by
  intro n h
  induction n, h using Nat.le_induction with
  | base => rw [self]
  | succ k hk ih => rw [succ D P hk (Nat.le_succ_of_le hk), y.2 k, ih]

/-- Peeling the last projection: `(P m).retr  comp  projLE (m+1  <=  n) = projLE (m  <=
     n)`. -/
theorem retr {m : Nat} : forall  {n : Nat} (h : m + 1  <=  n) (x : D n),
    (P m).retr (projLE D P h x) = projLE D P (Nat.le_of_succ_le h) x := by
  intro n h
  induction n, h using Nat.le_induction with
  | base =>
      intro x
      rw [succ D P (le_refl m) (Nat.le_of_succ_le (le_refl (m + 1))) x]
      simp only [self]
  | succ k hk ih =>
      intro x
      rw [succ D P hk (Nat.le_succ_of_le hk) x, ih ((P k).retr x),
        succ D P (Nat.le_of_succ_le hk) (Nat.le_of_succ_le (Nat.le_succ_of_le hk)) x]
```

```
/-- The tower of projections is Scott-continuous (a composite of the `j?`). -/
theorem preservesDirectedSup {m n : Nat} (h : m  <=  n) :
    PreservesDirectedSup (projLE D P h) := by
  induction n, h using Nat.le_induction with
  | base =>
      have hid : projLE D P (le_refl m) = id := funext (fun x => self D P (le_refl m)
    x)
      rw [hid]; intro S _ _; simp
  | succ k hk ih =>
      have hfun : projLE D P (Nat.le_succ_of_le hk)
          = projLE D P hk  comp  (fun z : D (k + 1) => (P k).retr z) :=
        funext (fun x => succ D P hk (Nat.le_succ_of_le hk) x)
      rw [hfun]
      exact ScottMap.preservesDirectedSup_comp ih
        (fun S hS hd => (P k).retr.preservesDirectedSup_coe S hS hd)

end projLE

end InverseLimit

end DomainTheory.ContinuousLattice
```

projection `projection_galoisConnection` is the adjunction between one bonding embedding and its projection: $i_n x \le y$ if and only if $x \le j_n y$. The projection, as the upper adjoint, preserves arbitrary infima.

projection_galoisConnection.lean

```
module
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : Nat  ->  Type u) [forall  n, CompleteLattice (D n)]

variable (P : forall  n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The embedding-projection pair of a projection is a Galois connection `i_n  dashv
    j_n`: from
`j_n  comp  i_n = id` and `i_n  comp  j_n  sqsub  id` we get `i_n x  sqsub  y  <->  x
    sqsub  j_n y`. In particular `j_n` (the upper
adjoint) preserves arbitrary infima. -/
theorem projection_galoisConnection (n : Nat) :
```

```
    GaloisConnection ((P n).incl : D n  ->  D (n + 1)) ((P n).retr : D (n + 1)  ->  D
    n) := by
  intro x y
  constructor
  * intro h
    have h' := (P n).retr.monotone h
    rwa [(P n).retr_incl] at h'
  * intro h
    exact le_trans ((P n).incl.monotone h) ((P n).incl_retr_le y)

end InverseLimit

end DomainTheory.ContinuousLattice
```

Projections from the limit `projInfFun` reads a coordinate of a thread, `projInfFun.preservesDirectedSup` is preservation of directed suprema, and `projInf` is that map as a Scott map. These are the maps $j_{\infty n}$.

The space X_∞ is given the product topology, and the maps $j_{\infty n} : X_\infty \to X_n$ such that

$$j_{\infty n}(x) = x_n$$

are of course continous and satisfy the recursion equation:

$$j_{\infty n} = j_n \circ j_{\infty(n+1)}.$$

projInf.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.coe_sSup_of_directed
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : Nat  ->  Type u) [forall  n, CompleteLattice (D n)]

variable (P : forall  n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The projection `j_{ inf n} : D_ inf   ->  D_n` as a bare function. -/
def projInfFun (n : Nat) (y : InverseLimit D P) : D n := y.1 n

namespace projInfFun
```

```
theorem preservesDirectedSup (n : Nat) : PreservesDirectedSup (projInfFun D P n) := by
  intro S hS hSdir
  have hL : (sSup S : InverseLimit D P).1 n = sSup ((fun y : InverseLimit D P => y.1 n
    ) '' S) := by
    rw [coe_sSup_of_directed D P S hS hSdir, sSup_apply_eq_sSup_image]
    congr 1
    rw [Set.image_image]
  exact hL

end projInfFun

/-- The projection `j_{ inf n} : D_ inf   ->  D_n`, Scott-continuous. -/
noncomputable def projInf (n : Nat) : ScottMap (InverseLimit D P) (D n) :=
  <projInfFun D P n, continuous_of_preservesDirectedSup (projInfFun.
    preservesDirectedSup D P n)>

end InverseLimit

end DomainTheory.ContinuousLattice
```

Embeddings along the tower The partial inverses of the bonding projections j_n are the embeddings i_n. `embLE` climbs from an earlier stage to a later one by composing those embeddings. The lemmas are `embLE_self`, `embLE_succ`, `embLE_succ_left`, `embLE_le`, `embLE_mono`, and `embLE_preservesDirectedSup`.

embLE.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.comp
public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : Nat  ->  Type u) [forall  n, CompleteLattice (D n)]

variable (P : forall  n, IsContinuousLatticeProjection (D n) (D (n + 1)))
```

```
/-- Climb the tower of embeddings: for `n <= m`, `embLE h = i_{m-1} comp ... comp
    i_n : D_n -> D_m`. -/
def embLE {n m : Nat} (h : n <= m) (x : D n) : D m :=
  Nat.leRecOn h (fun {k} (y : D k) => (P k).incl y) x

theorem embLE_self {n : Nat} (h : n <= n) (x : D n) : embLE D P h x = x := by
  simp only [embLE, Nat.leRecOn_self]

theorem embLE_succ {n m : Nat} (h1 : n <= m) (h2 : n <= m + 1) (x : D n) :
    embLE D P h2 x = (P m).incl (embLE D P h1 x) := by
  simp only [embLE]
  rw [Nat.leRecOn_succ h1]

theorem embLE_succ_left {n m : Nat} (h1 : n <= m) (h2 : n + 1 <= m) (x : D n) :
    embLE D P h2 ((P n).incl x) = embLE D P h1 x := by
  simp only [embLE]
  exact Nat.leRecOn_succ_left x h1 h2

/-- For a compatible sequence `y`, climbing `y_n` up to level `m >= n` stays below `y
    ?`. -/
theorem embLE_le {n : Nat} (y : InverseLimit D P) {m : Nat} (h : n <= m) :
    embLE D P h (y.1 n) <= y.1 m := by
  induction m, h using Nat.le_induction with
  | base => exact le_of_eq (embLE_self D P _ _)
  | succ k hk ih =>
      rw [embLE_succ D P hk (Nat.le_succ_of_le hk)]
      calc (P k).incl (embLE D P hk (y.1 n))
           <= (P k).incl (y.1 k) := (P k).incl.monotone ih
        _ = (P k).incl ((P k).retr (y.1 (k + 1))) := by rw [y.2 k]
        _ <= y.1 (k + 1) := (P k).incl_retr_le _

theorem embLE_mono {n m : Nat} (h : n <= m) {x y : D n} (hxy : x <= y) :
    embLE D P h x <= embLE D P h y := by
  induction m, h using Nat.le_induction with
  | base =>
      rw [embLE_self, embLE_self]
      exact hxy
  | succ k hk ih =>
      rw [embLE_succ D P hk (Nat.le_succ_of_le hk), embLE_succ D P hk (Nat.
    le_succ_of_le hk)]
      exact (P k).incl.monotone ih

/-- The tower of embeddings is Scott-continuous (a composite of the `i?`). -/
theorem embLE_preservesDirectedSup {n m : Nat} (h : n <= m) :
    PreservesDirectedSup (embLE D P h) := by
  induction m, h using Nat.le_induction with
  | base =>
      have hid : embLE D P (le_refl n) = id := funext (fun x => embLE_self D P (
    le_refl n) x)
      rw [hid]; intro S _ _; simp
  | succ k hk ih =>
      have hfun : embLE D P (Nat.le_succ_of_le hk)
          = (fun y : D k => (P k).incl y) comp embLE D P hk :=
        funext (fun x => embLE_succ D P hk (Nat.le_succ_of_le hk) x)
      rw [hfun]
      exact ScottMap.preservesDirectedSup_comp
```

```
        (fun S hS hd => (P k).incl.preservesDirectedSup_coe S hS hd) ih

end InverseLimit

end DomainTheory.ContinuousLattice
```

Coordinates of an embedding `iComp` is one coordinate of the embedding $i_{n\infty}$ of a stage into the limit: it climbs by `embLE` at later stages and descends by `projLE` at earlier ones. The lemmas are `iComp_of_le`, `iComp_of_ge`, `iComp_self`, `iComp_incl_le`, `iComp_preservesDirectedSup`, `iComp_monotone`, and `iComp_compatible`.

iComp.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.embLE
public import DomainTheory.ContinuousLattice.InverseLimits.projLE
public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.InverseLimits.Compatible
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import DomainTheory.ContinuousLattice.ScottMaps.preservesDirectedSup_monotone
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : Nat  ->  Type u) [forall  n, CompleteLattice (D n)]

variable (P : forall  n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Scott's embedding component `i_{n inf }(x)_m`: climb for `m  >=  n`, descend for `m
     < n`. -/
def iComp (n : Nat) (x : D n) (m : Nat) : D m :=
  if h : n  <=  m then embLE D P h x else projLE D P (le_of_lt (not_le.mp h)) x

theorem iComp_of_le {n m : Nat} (h : n  <=  m) (x : D n) : iComp D P n x m = embLE D P
     h x :=
  dite_eq_left h

theorem iComp_of_ge {n m : Nat} (h : not  n  <=  m) (x : D n) :
    iComp D P n x m = projLE D P (le_of_lt (not_le.mp h)) x :=
  dite_eq_right h

theorem iComp_self (n : Nat) (x : D n) : iComp D P n x n = x := by
  rw [iComp_of_le D P (le_refl n), embLE_self]
```

```
/-- `i_{n inf }(y_n)  sqsub  y` coordinatewise (the heart of `incl_retr_le` for `j_{
    inf n}`). -/
theorem iComp_incl_le (n : Nat) (y : InverseLimit D P) (m : Nat) :
    iComp D P n (y.1 n) m  <=  y.1 m := by
  by_cases h : n  <=  m
  * rw [iComp_of_le D P h]; exact embLE_le D P y h
  * rw [iComp_of_ge D P h]
    exact le_of_eq (projLE.compatible D P y (le_of_lt (not_le.mp h)))

theorem iComp_preservesDirectedSup (n m : Nat) :
    PreservesDirectedSup (fun x : D n => iComp D P n x m) := by
  by_cases h : n  <=  m
  * have hfun : (fun x : D n => iComp D P n x m) = embLE D P h :=
      funext (fun x => iComp_of_le D P h x)
    rw [hfun]; exact embLE_preservesDirectedSup D P h
  * have hfun : (fun x : D n => iComp D P n x m) = projLE D P (le_of_lt (not_le.mp h))
     :=
      funext (fun x => iComp_of_ge D P h x)
    rw [hfun]; exact projLE.preservesDirectedSup D P _

theorem iComp_monotone (n m : Nat) : Monotone (fun x : D n => iComp D P n x m) :=
  preservesDirectedSup_monotone (iComp_preservesDirectedSup D P n m)

/-- The sequence `i_{n inf }(x)` is compatible, hence a genuine point of `D_ inf `. -/
theorem iComp_compatible (n : Nat) (x : D n) : Compatible D P (iComp D P n x) := by
  intro m
  by_cases h1 : n  <=  m
  * rw [iComp_of_le D P h1, iComp_of_le D P (Nat.le_succ_of_le h1),
      embLE_succ D P h1 (Nat.le_succ_of_le h1), (P m).retr_incl]
  * by_cases h2 : n  <=  m + 1
    * have hn : n = m + 1 := le_antisymm h2 (not_le.mp h1)
      subst hn
      rw [iComp_of_le D P (le_refl (m + 1)), embLE_self, iComp_of_ge D P h1,
        projLE.succ D P (le_refl m) (le_of_lt (not_le.mp h1)) x, projLE.self]
    * rw [iComp_of_ge D P h2, iComp_of_ge D P h1,
        projLE.retr D P (le_of_lt (not_le.mp h2)) x]

end InverseLimit

end DomainTheory.ContinuousLattice
```

Embeddings into the limit `embInfFun` is the thread $i_{n\infty}(x)$, and `embInf` is that map as a Scott map. The application formula in the application note is the supremum of these embeddings.

Dependency diagram. Each box is one of these notes. The arrowhead is on the note that imports. The number is how many direct imports. An arrow already implied by a longer path is left out.

embInfFun `embInfFun` is the thread $i_{n\infty}(x)$. `embInfFun_coe` is its underlying sequence, `embInf_monotone` is monotonicity, and `embInf_preservesDirectedSup` is preservation of directed suprema. The Scott map is embInf.

`embInfFun.lean`

```
module
```

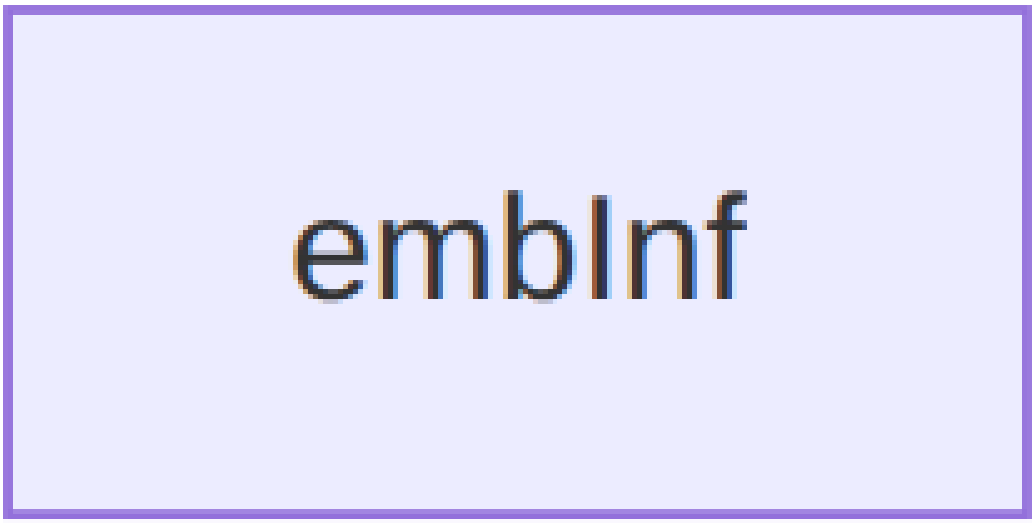


Figure 12: Inverse limits.

```
public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.InverseLimits.iComp
public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.coe_sSup_of_directed
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : Nat  ->  Type u) [forall  n, CompleteLattice (D n)]

variable (P : forall  n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The embedding `i_{n inf } : D_n  ->  D_ inf ` as a bare function into the inverse
    limit. -/
def embInfFun (n : Nat) (x : D n) : InverseLimit D P := <iComp D P n x,
    iComp_compatible D P n x>

@[simp] theorem embInfFun_coe (n : Nat) (x : D n) : (embInfFun D P n x).1 = iComp D P
```

```
    n x := rfl

theorem embInf_monotone (n : Nat) : Monotone (embInfFun D P n) := by
  intro x x' hxx
  rw [ <-  Subtype.coe_le_coe]
  intro m
  exact iComp_monotone D P n m hxx

theorem embInf_preservesDirectedSup (n : Nat) : PreservesDirectedSup (embInfFun D P n)
     := by
  intro S hS hSdir
  have hTne : (embInfFun D P n '' S).Nonempty := hS.image _
  have hTdir : DirectedOn (*  <=  *) (embInfFun D P n '' S) := by
    rintro _ <a, ha, rfl> _ <b, hb, rfl>
    obtain <c, hc, hac, hbc> := hSdir a ha b hb
    exact <embInfFun D P n c, Set.mem_image_of_mem _ hc,
      embInf_monotone D P n hac, embInf_monotone D P n hbc>
  apply Subtype.ext
  rw [coe_sSup_of_directed D P (embInfFun D P n '' S) hTne hTdir]
  funext m
  have heq : Function.eval m '' (Subtype.val '' (embInfFun D P n '' S))
      = (fun x => iComp D P n x m) '' S := by
    simp only [Set.image_image, embInfFun_coe, Function.eval]
  rw [sSup_apply_eq_sSup_image, heq]
  exact iComp_preservesDirectedSup D P n m hS hSdir

end InverseLimit

end DomainTheory.ContinuousLattice
```

embInf `embInf` is embInfFun as a Scott map. `embInf_succ` is the recursion $i_{n\infty} = i_{(n+1)\infty} \circ i_n$. `embInf_le_succ` and `embInf_family_directed` say that the embeddings of the coordinates of a thread form a monotone, hence directed, family.

embInf.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.embInfFun
public import DomainTheory.ContinuousLattice.InverseLimits.embLE
public import DomainTheory.ContinuousLattice.InverseLimits.iComp
public import DomainTheory.ContinuousLattice.InverseLimits.projLE
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u
```

```
section InverseLimit

variable (D : Nat  ->  Type u) [forall  n, CompleteLattice (D n)]

variable (P : forall  n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The embedding `i_{n inf } : D_n  ->  D_ inf `, Scott-continuous. -/
noncomputable def embInf (n : Nat) : ScottMap (D n) (InverseLimit D P) :=
  <embInfFun D P n, continuous_of_preservesDirectedSup (embInf_preservesDirectedSup D
    P n)>

/-- **Scott 1972, S4 (recursion equation).** `i_{n inf } = i_{(n+1) inf }  comp  i_n`.
    -/
theorem embInf_succ (n : Nat) (x : D n) :
    embInf D P (n + 1) ((P n).incl x) = embInf D P n x := by
  apply Subtype.ext
  funext m
  show iComp D P (n + 1) ((P n).incl x) m = iComp D P n x m
  by_cases h1 : n  <=  m
  * by_cases h2 : n + 1  <=  m
    * rw [iComp_of_le D P h2, iComp_of_le D P h1, embLE_succ_left D P h1 h2]
    * have hmn : m = n := le_antisymm (Nat.lt_succ_iff.mp (not_le.mp h2)) h1
      subst hmn
      rw [iComp_self, iComp_of_ge D P h2,
        projLE.succ D P (le_refl m) (le_of_lt (not_le.mp h2)) ((P m).incl x),
        projLE.self, (P m).retr_incl]
  * have h2 : not  n + 1  <=  m := fun h => h1 (le_trans (Nat.le_succ n) h)
    rw [iComp_of_ge D P h2, iComp_of_ge D P h1,
      projLE.succ D P (le_of_lt (not_le.mp h1)) (le_of_lt (not_le.mp h2)) ((P n).incl
    x),
      (P n).retr_incl]

/-- The family `n  |->  i_{n inf }(x_n)` is monotone (the lub defining `x` is monotone)
    . -/
theorem embInf_le_succ (x : InverseLimit D P) (n : Nat) :
    embInf D P n (x.1 n)  <=  embInf D P (n + 1) (x.1 (n + 1)) := by
  rw [ <-  embInf_succ D P n (x.1 n)]
  exact embInf_monotone D P (n + 1)
    (by calc (P n).incl (x.1 n) = (P n).incl ((P n).retr (x.1 (n + 1))) := by rw [x.2
    n]
          _  <=  x.1 (n + 1) := (P n).incl_retr_le _)

theorem embInf_family_directed (x : InverseLimit D P) :
    DirectedOn (*  <=  *) (Set.range (fun n => embInf D P n (x.1 n))) :=
  directedOn_range.2 (monotone_nat_of_le_succ (embInf_le_succ D P x)).directed_le

end InverseLimit

end DomainTheory.ContinuousLattice
```

The mediating map `coconeInf` assembles a compatible family of maps into the stages into one map out of the limit. The lemmas are `coconeInf_apply`, `coconeInf_climb`, `coconeInf_descend`, `coconeInf_comp_embInf`, and `coconeInf_preservesDirectedSup`.

Besides this we have the expected extension property for any system of continuous maps

$$f_n : Y \to X_n$$

where for each n

$$f_n = j_n \circ f_{n+1}.$$

Because, we can define

$$f_\infty : Y \to X_\infty$$

by the equation

$$f_\infty(y) = \langle f_n(y) \rangle_{n=0}^{\infty}$$

for all $y \in Y$; whence

$$f_n = j_{\infty n} \circ f_\infty$$

holds. So much for a review of inverse limits. In this paper our interest will center on rather special inverse systems and their limits.

coconeInf.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.InverseLimits.embLE
public import DomainTheory.ContinuousLattice.InverseLimits.projLE
public import DomainTheory.ContinuousLattice.InverseLimits.embInf
public import DomainTheory.ContinuousLattice.InverseLimits.iComp
public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.projInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.ScottMaps.PreservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : Nat  ->  Type u) [forall  n, CompleteLattice (D n)]

variable (P : forall  n, IsContinuousLatticeProjection (D n) (D (n + 1)))

variable {D' : Type*} [CompleteLattice D']
```

```
/-- The mediating map of Corollary 4.3: `f inf (x) =  bigcup _n f_n(x_n)`. -/
def coconeInf (f : forall  n, ScottMap (D n) D') (x : InverseLimit D P) : D' :=
   bigcup  n, f n (x.1 n)

theorem coconeInf_apply (f : forall  n, ScottMap (D n) D') (x : InverseLimit D P) :
    coconeInf D P f x =  bigcup  n, f n (x.1 n) := rfl

/-- Climbing then applying `f` is constant: `f_m(i_{m-1}...i_n x) = f_n(x)`. -/
theorem coconeInf_climb (f : forall  n, ScottMap (D n) D')
    (hf : forall  n x, f n x = f (n + 1) ((P n).incl x)) {n : Nat} (x : D n) :
    forall  {m : Nat} (h : n  <=  m), f m (embLE D P h x) = f n x := by
  intro m h
  induction m, h using Nat.le_induction with
  | base => rw [embLE_self]
  | succ k hk ih => rw [embLE_succ D P hk (Nat.le_succ_of_le hk),  <-  hf k, ih]

/-- Descending then applying `f` only decreases: `f_m(j_m...j_{n-1} x)  sqsub  f_n(x)`.
    -/
theorem coconeInf_descend (f : forall  n, ScottMap (D n) D')
    (hf : forall  n x, f n x = f (n + 1) ((P n).incl x)) {m : Nat} :
    forall  {n : Nat} (h : m  <=  n) (x : D n), f m (projLE D P h x)  <=  f n x := by
  intro n h
  induction n, h using Nat.le_induction with
  | base => intro x; exact le_of_eq (congrArg (f m) (projLE.self D P (le_refl m) x))
  | succ k hk ih =>
      intro x
      rw [projLE.succ D P hk (Nat.le_succ_of_le hk)]
      calc f m (projLE D P hk ((P k).retr x))  <=  f k ((P k).retr x) := ih ((P k).
    retr x)
        _ = f (k + 1) ((P k).incl ((P k).retr x)) := hf k _
        _  <=  f (k + 1) x := (f (k + 1)).monotone ((P k).incl_retr_le x)

/-- **Scott 1972, Corollary 4.3 (factorization).** `f_n = f inf   comp  i_{n inf }`. -/
theorem coconeInf_comp_embInf (f : forall  n, ScottMap (D n) D')
    (hf : forall  n x, f n x = f (n + 1) ((P n).incl x)) (n : Nat) (x : D n) :
    coconeInf D P f (embInf D P n x) = f n x := by
  apply le_antisymm
  * show ( bigcup  m, f m ((embInf D P n x).1 m))  <=  f n x
    refine iSup_le fun m => ?_
    show f m (iComp D P n x m)  <=  f n x
    by_cases h : n  <=  m
    * rw [iComp_of_le D P h]; exact le_of_eq (coconeInf_climb D P f hf x h)
    * rw [iComp_of_ge D P h]; exact coconeInf_descend D P f hf (le_of_lt (not_le.mp h)
    ) x
  * refine le_trans (le_of_eq ?_) (le_iSup (fun m => f m ((embInf D P n x).1 m)) n)
    rw [show (embInf D P n x).1 n = x from iComp_self D P n x]

/-- `f inf ` is Scott-continuous: `f inf ( sqsup S) =  sqsup  f inf (S)` for directed `
    S` (each `f_n` is continuous and a
double ` bigcup ` over `Nat  x  S` commutes). -/
theorem coconeInf_preservesDirectedSup (f : forall  n, ScottMap (D n) D') :
    PreservesDirectedSup (coconeInf D P f) := by
  intro S hS hSdir
  have hev : forall  n, DirectedOn (*  <=  *) ((fun s : InverseLimit D P => s.1 n) ''
    S) := by
    intro n
```

```
    rintro _ <a, ha, rfl> _ <b, hb, rfl>
    obtain <c, hc, hac, hbc> := hSdir a ha b hb
    exact <c.1 n, <c, hc, rfl>, hac n, hbc n>
  have key : forall  n, f n ((sSup S).1 n) = sSup ((fun s : InverseLimit D P => f n (s
    .1 n)) '' S) := by
    intro n
    have h1 : (sSup S).1 n = sSup ((fun s : InverseLimit D P => s.1 n) '' S) :=
      projInfFun.preservesDirectedSup D P n hS hSdir
    rw [h1, (f n).preservesDirectedSup_coe _ (hS.image _) (hev n), Set.image_image]
  show ( bigcup  n, f n ((sSup S).1 n)) = sSup (coconeInf D P f '' S)
  simp_rw [key, sSup_image', coconeInf_apply]
  rw [iSup_comm]

end InverseLimit

end DomainTheory.ContinuousLattice
```

inverseLimitLE `inverseLimitLE` is the coordinatewise order on threads: every coordinate of the first lies below the corresponding coordinate of the second. `inverseLimitLE.refl`, `inverseLimitLE.trans`, and `inverseLimitLE.antisymm` are the proofs the partial order uses.

`inverseLimitLE.lean`

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : Nat  ->  Type u) [forall  n, CompleteLattice (D n)]

variable (P : forall  n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Coordinatewise order on compatible sequences. Kept reducible so existing
subtype-order lemmas see the underlying pointwise relation definitionally. -/
abbrev inverseLimitLE (x y : InverseLimit D P) : Prop :=
  forall  n,
    @LE.le (D n)
      (ChainCompletePartialOrder.instOfCompleteLattice
        (a := D n)).toPartialOrder.toPreorder.toLE
      (x.1 n) (y.1 n)

namespace inverseLimitLE
```

```
/-- Reflexivity of the coordinatewise inverse-limit order. -/
theorem refl (x : InverseLimit D P) :
    inverseLimitLE D P x x :=
  fun n => le_refl (x.1 n)

/-- Transitivity of the coordinatewise inverse-limit order. -/
theorem trans (x y z : InverseLimit D P)
    (hxy : inverseLimitLE D P x y) (hyz : inverseLimitLE D P y z) :
    inverseLimitLE D P x z :=
  fun n => le_trans (hxy n) (hyz n)

/-- Antisymmetry of the coordinatewise inverse-limit order. -/
theorem antisymm (x y : InverseLimit D P)
    (hxy : inverseLimitLE D P x y) (hyx : inverseLimitLE D P y x) :
    x = y :=
  Subtype.ext (funext fun n => le_antisymm (hxy n) (hyx n))

end inverseLimitLE

end InverseLimit

end DomainTheory.ContinuousLattice
```

inverseLimitLT `inverseLimitLT` is the strict coordinatewise order. `inverseLimitLT.iff_LE_not_GE` says it is `inverseLimitLE` in one direction and the failure of `inverseLimitLE` in the other, so a thread lies strictly below another when it is below at every stage and strictly below at one stage.

`inverseLimitLT.lean`

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitLE
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : Nat  ->  Type u) [forall  n, CompleteLattice (D n)]

variable (P : forall  n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- Strict coordinatewise order induced by `inverseLimitLE`. -/
abbrev inverseLimitLT (x y : InverseLimit D P) : Prop :=
  inverseLimitLE D P x y  /\  not  inverseLimitLE D P y x

namespace inverseLimitLT
```

```
/-- The chosen strict inverse-limit order is ` <=   /\  not   >= `. -/
theorem iff_LE_not_GE (x y : InverseLimit D P) :
    inverseLimitLT D P x y  <->
      inverseLimitLE D P x y  /\  not  inverseLimitLE D P y x :=
  Iff.rfl

end inverseLimitLT

end InverseLimit

end DomainTheory.ContinuousLattice
```

The limit lattice The limit becomes a complete lattice in the steps below. `inverseLimitPartialOrder` is the coordinatewise order. `inverseLimitSInfCoe` is the coordinatewise infimum of a set of threads. `instInfSet` packages that infimum once it is shown to be a thread, and records that it is a greatest lower bound. `instCompleteLattice` obtains the complete lattice from that order and those infima. `coe_sSup_of_directed` is the pointwise formula for a nonempty directed supremum.

Dependency diagram. Each box is one of these notes. The arrowhead is on the note that imports. The number is how many direct imports. An arrow already implied by a longer path is left out.

inverseLimitPartialOrder `inverseLimitPartialOrder` is the partial order whose `le` is inverseLimitLE and whose `lt` is inverseLimitLT.

`inverseLimitPartialOrder.lean`

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitLE
public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitLT
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : Nat  ->  Type u) [forall  n, CompleteLattice (D n)]

variable (P : forall  n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The inverse limit carries the coordinatewise order inherited from its
ambient product. This declaration makes that order explicit instead of
leaving typeclass search to choose an equivalent route. -/
instance inverseLimitPartialOrder : PartialOrder (InverseLimit D P) where
  le := inverseLimitLE D P
  lt := inverseLimitLT D P
  le_refl := inverseLimitLE.refl D P
```

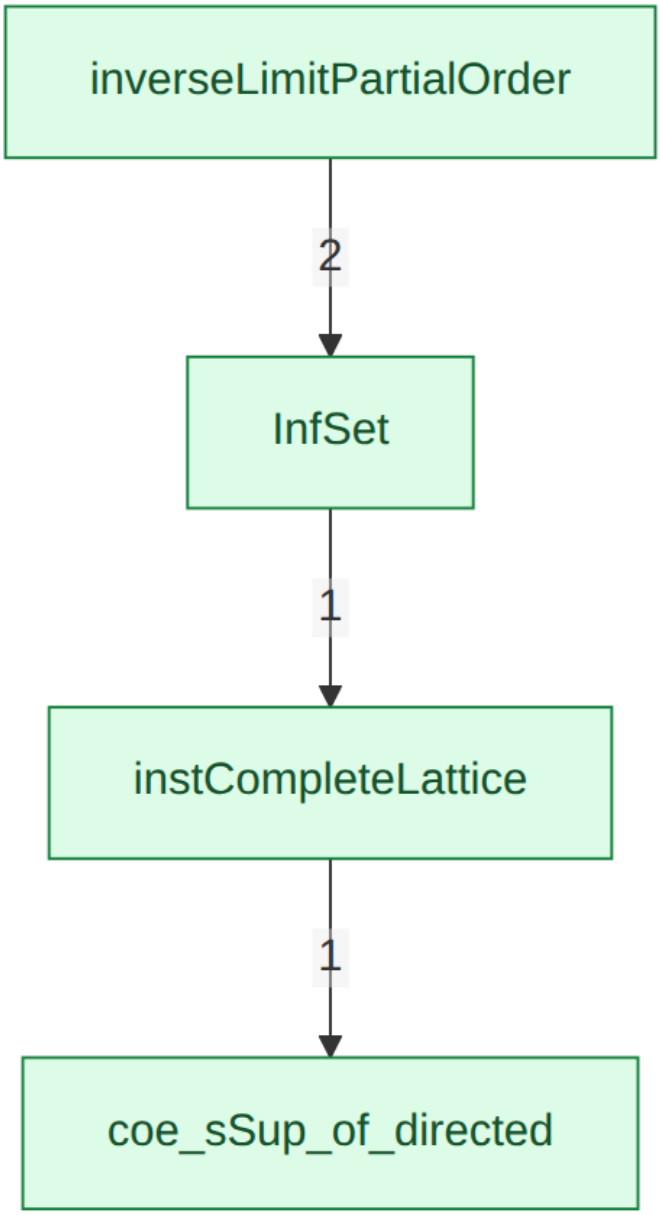


Figure 13: Inverse limits.

```
  le_trans := inverseLimitLE.trans D P
  lt_iff_le_not_ge := inverseLimitLT.iff_LE_not_GE D P
  le_antisymm := inverseLimitLE.antisymm D P

end InverseLimit

end DomainTheory.ContinuousLattice
```

inverseLimitSInfCoe `inverseLimitSInfCoe` is the coordinatewise infimum of a set of threads, before InfSet packages the compatibility proof.

`inverseLimitSInfCoe.lean`

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : Nat  ->  Type u) [forall  n, CompleteLattice (D n)]

variable (P : forall  n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- The coordinatewise infimum of a family of compatible sequences, before
packaging the compatibility proof. -/
noncomputable def inverseLimitSInfCoe (S : Set (InverseLimit D P)) : forall  n, D n :=
  fun n => sInf ((fun x : InverseLimit D P => x.1 n) '' S)

end InverseLimit

end DomainTheory.ContinuousLattice
```

InfSet `instInfSet` is the infimum of a set of threads. `retr_sInf` and `compatible_sInf` show that the pointwise infimum is a thread, `coe_sInf` is the underlying sequence of that infimum, and `isGLB_sInf'` says it is a greatest lower bound.

`instInfSet.lean`

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitPartialOrder
public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitSInfCoe
public import DomainTheory.ContinuousLattice.InverseLimits.projection_galoisConnection
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : Nat  ->  Type u) [forall  n, CompleteLattice (D n)]

variable (P : forall  n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- `j_n` preserves arbitrary infima (it is the upper adjoint of `i_n`). -/
theorem retr_sInf (n : Nat) (A : Set (D (n + 1))) :
    (P n).retr (sInf A) = sInf ((P n).retr '' A) := by
  rw [(projection_galoisConnection D P n).u_sInf, sInf_image]

/-- The pointwise infimum of compatible sequences is compatible. -/
theorem compatible_sInf (S : Set (InverseLimit D P)) :
    Compatible D P (inverseLimitSInfCoe D P S) := by
  intro n
  rw [inverseLimitSInfCoe, inverseLimitSInfCoe, retr_sInf]
  congr 1
  rw [Set.image_image]
  exact Set.image_congr (by rintro x hx; exact x.2 n)

noncomputable instance instInfSet : InfSet (InverseLimit D P) :=
  <fun S => <inverseLimitSInfCoe D P S, compatible_sInf D P S>>

theorem coe_sInf (S : Set (InverseLimit D P)) :
    ((sInf S : InverseLimit D P) : forall  n, D n) = inverseLimitSInfCoe D P S := rfl

theorem isGLB_sInf' (S : Set (InverseLimit D P)) :
    @IsGLB (InverseLimit D P)
      (inverseLimitPartialOrder D P).toPreorder.toLE S (sInf S) := by
  constructor
  * intro x hx n
    exact sInf_le (Set.mem_image_of_mem _ hx)
  * intro b hb n
    refine le_sInf ?_
    rintro _ <x, hx, rfl>
    exact hb hx n

end InverseLimit

end DomainTheory.ContinuousLattice
```

instCompleteLattice `instCompleteLattice` is the complete lattice obtained from inverseLimitPartialOrder by taking the infima of InfSet to be greatest lower bounds.

`instCompleteLattice.lean`

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.instInfSet
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : Nat  ->  Type u) [forall  n, CompleteLattice (D n)]

variable (P : forall  n, IsContinuousLatticeProjection (D n) (D (n + 1)))

noncomputable instance instCompleteLattice : CompleteLattice (InverseLimit D P) :=
  @completeLatticeOfInf (InverseLimit D P) (inverseLimitPartialOrder D P)
    inferInstance (isGLB_sInf' D P)

end InverseLimit

end DomainTheory.ContinuousLattice
```

coe_sSup_of_directed `coe_sSup_of_directed` says a nonempty directed supremum in the limit is computed pointwise. The identities that express a thread as a directed supremum of embeddings are directed suprema.

`coe_sSup_of_directed.lean`

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit
```

```
variable (D : Nat  ->  Type u) [forall  n, CompleteLattice (D n)]

variable (P : forall  n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- For a directed, nonempty family, the inverse-limit supremum is computed pointwise
    (each `j_n`
is Scott-continuous, so the pointwise sup of compatible sequences is compatible and is
     the least
upper bound in `D_ inf `). -/
theorem coe_sSup_of_directed (S : Set (InverseLimit D P)) (hSne : S.Nonempty)
    (hSdir : DirectedOn (*  <=  *) S) :
    ((sSup S : InverseLimit D P) : forall  n, D n) = sSup (Subtype.val '' S) := by
  have hcompat : Compatible D P (sSup (Subtype.val '' S)) := by
    intro n
    rw [sSup_apply_eq_sSup_image, sSup_apply_eq_sSup_image]
    set A : Set (D (n + 1)) := (fun f : forall  m, D m => f (n + 1)) '' (Subtype.val
    '' S) with hA
    have hAne : A.Nonempty := (hSne.image _).image _
    have hAdir : DirectedOn (*  <=  *) A := by
      rintro _ <_, <x, hxS, rfl>, rfl> _ <_, <y, hyS, rfl>, rfl>
      obtain <z, hzS, hxz, hyz> := hSdir x hxS y hyS
      exact <z.1 (n + 1), <z.1, <z, hzS, rfl>, rfl>, hxz (n + 1), hyz (n + 1)>
    rw [(P n).retr.preservesDirectedSup_coe A hAne hAdir]
    congr 1
    rw [hA, Set.image_image]
    exact Set.image_congr (by rintro _ <x, _, rfl>; exact x.2 n)
  set p : InverseLimit D P := <sSup (Subtype.val '' S), hcompat> with hp
  have hlub : IsLUB S p := by
    constructor
    * intro x hx
      refine Subtype.coe_le_coe.mp ?_
      exact le_sSup (Set.mem_image_of_mem _ hx)
    * intro b hb
      refine Subtype.coe_le_coe.mp ?_
      refine sSup_le ?_
      rintro _ <x, hx, rfl>
      exact Subtype.coe_le_coe.mpr (hb hx)
  rw [(isLUB_sSup S).unique hlub]

end InverseLimit

end DomainTheory.ContinuousLattice
```

Directed suprema `inverseLimit.eq_iSup` says each thread is the directed supremum of its embeddings from the stages, $x = \bigsqcup_n i_{n\infty}(x_n)$. `idInf_eq_iSup` says the identity on the limit is the supremum of the approximating projections $i_{n\infty} \circ j_{\infty n}$.

`idInf_eq_iSup.lean`

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.embInf
public import DomainTheory.ContinuousLattice.InverseLimits.projInf
public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitTopology
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.idMap_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst30
public import DomainTheory.ContinuousLattice.FunctionSpaces.idMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.comp
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : Nat  ->  Type u) [forall  n, CompleteLattice (D n)]
variable (P : forall  n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- **Scott 1972, S4 (functional equation, "the remark following 4.2").** The identity
     of `D_ inf ` is
the directed lub of the approximating projections `r_n = i_{n inf }  comp  j_{ inf n}`:
`id =  bigcup _n i_{n inf }  comp  j_{ inf n}`. This is the algebraic identity at the
    heart of Theorem 4.4. -/
theorem idInf_eq_iSup :
    (ScottMap.idMap : ScottMap (InverseLimit D P) (InverseLimit D P))
      =  bigcup  n, (embInf D P n).comp (projInf D P n) := by
  apply ScottMap.ext
  intro x
  rw [show ( bigcup  n, (embInf D P n).comp (projInf D P n))
        = sSup (Set.range fun n => (embInf D P n).comp (projInf D P n)) from
    sSup_range.symm,
    ScottMap.sSup_apply, ScottMap.idMap_apply,  <-  Set.range_comp, sSup_range]
  exact inverseLimit.eq_iSup D P x

end InverseLimit
end DomainTheory.ContinuousLattice
```

inverseLimitTopology `inverseLimitTopology` is the subspace topology induced on threads by the product of the stage Scott topologies. The same file then opens `namespace inverseLimit` for the properties of the limit in that topology, and `namespace inverseLimitTopology` for continuity of the stage embedding, the limit projection, and the mediating map. Those continuity theorems call `inverseLimit.coordinate_continuous` and `inverseLimit.scottTopology_eq_induced_pi`, so the two namespaces are one module.

`inverseLimitTopology.lean`

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.InverseLimits.projInf
public import DomainTheory.ContinuousLattice.InverseLimits.embInf
```

```
public import DomainTheory.ContinuousLattice.InverseLimits.coconeInf
public import DomainTheory.ContinuousLattice.Specialization.scottTopology_t0Space
public import DomainTheory.ContinuousLattice.Specialization.specializationLe_scott_iff
public import DomainTheory.ContinuousLattice.Specialization.
    specializationLe_iff_specializes
public import DomainTheory.ContinuousLattice.Specialization.SpecializationLe
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.ContinuousLattices.lemma_3_9
public import DomainTheory.ContinuousLattice.ContinuousLattices.theorem_2_12
public import DomainTheory.ContinuousLattice.ContinuousLattices.theorem_2_12_more
public import DomainTheory.ContinuousLattice.Constructions.scottExtend_continuous
public import DomainTheory.ContinuousLattice.Constructions.
    scottExtend_eq_of_continuous
public import DomainTheory.ContinuousLattice.Injective.InjectiveSpace
public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set
universe u

section InverseLimit

variable (D : Nat  ->  Type u) [forall  n, CompleteLattice (D n)]

variable (P : forall  n, IsContinuousLatticeProjection (D n) (D (n + 1)))

@[reducible] noncomputable def inverseLimitTopology : TopologicalSpace (InverseLimit D
     P) :=
  TopologicalSpace.induced (Subtype.val : InverseLimit D P  ->  forall  n, D n)
    (@Pi.topologicalSpace Nat D (fun _ => scottTopologicalSpace))

namespace inverseLimit
/-- Every coordinate projection from the explicit product/subspace inverse-limit
    topology is
continuous. -/
theorem coordinate_continuous (n : Nat) :
    @Continuous (InverseLimit D P) (D n) (inverseLimitTopology D P)
      scottTopologicalSpace (fun x => x.1 n) := by
  let : forall  n, TopologicalSpace (D n) := fun _ => scottTopologicalSpace
  rw [continuous_def]
  intro U hU
  rw [isOpen_induced_iff]
  exact <{x : forall  n, D n | x n  in  U},
    (@continuous_apply Nat D (fun _ => scottTopologicalSpace) n).isOpen_preimage U hU,
     rfl>

/-- The explicit inverse-limit topology is `T_0`, since the inclusion into the product
    of the
stage Scott spaces is an embedding. -/
theorem t0Space :
```

```
    @T0Space (InverseLimit D P) (inverseLimitTopology D P) := by
  let : forall  n, TopologicalSpace (D n) := fun _ => scottTopologicalSpace
  let : forall  n, T0Space (D n) := fun _ => scottTopology_t0Space
  let : TopologicalSpace (InverseLimit D P) := inverseLimitTopology D P
  have hval : Topology.IsEmbedding (Subtype.val : InverseLimit D P  ->  forall  n, D n
    ) :=
    <<rfl>, Subtype.val_injective>
  exact hval.t0Space

/-- The specialization order of the explicit inverse-limit topology is the
    componentwise order. -/
theorem specializationLe_iff (x y : InverseLimit D P) :
    @SpecializationLe (InverseLimit D P) (inverseLimitTopology D P) x y  <->  x  <=  y
     := by
  let stageOrder : forall  n, Preorder (D n) := fun _ => inferInstance
  let limitOrder : Preorder (InverseLimit D P) := inferInstance
  let : forall  n, TopologicalSpace (D n) := fun _ => scottTopologicalSpace
  let : TopologicalSpace (InverseLimit D P) := inverseLimitTopology D P
  let : forall  n, Preorder (D n) := stageOrder
  let : Preorder (InverseLimit D P) := limitOrder
  have hval : Topology.IsEmbedding (Subtype.val : InverseLimit D P  ->  forall  n, D n
    ) :=
    <<rfl>, Subtype.val_injective>
  rw [specializationLe_iff_specializes,  <-  hval.isInducing.specializes_iff,
    specializes_pi]
  constructor
  * intro h n
    rw [ <-  specializationLe_scott_iff, specializationLe_iff_specializes]
    exact h n
  * intro h n
    rw [ <-  specializationLe_iff_specializes, specializationLe_scott_iff]
    exact h n

/-- **Scott 1972, Proposition 4.1 (injectivity step).** Coordinatewise maximal
    extensions
assemble to an extension into the inverse limit. Lemma 3.9 is exactly what supplies
    compatibility
of adjacent coordinates. -/
theorem isInjectiveSpace (hD : forall  n, ContinuousLattice (D n)) :
    @InjectiveSpace.{u, u} (InverseLimit D P) (inverseLimitTopology D P) := by
  let stageOrder : forall  n, Preorder (D n) := fun _ => inferInstance
  let limitOrder : Preorder (InverseLimit D P) := inferInstance
  let : forall  n, TopologicalSpace (D n) := fun _ => scottTopologicalSpace
  let : TopologicalSpace (InverseLimit D P) := inverseLimitTopology D P
  let : forall  n, Preorder (D n) := stageOrder
  let : Preorder (InverseLimit D P) := limitOrder
  constructor
  intro X Y _ _ e he f
  let fcoord (n : Nat) (x : X) : D n := (f x).1 n
  have hfcoord (n : Nat) : @Continuous X (D n) _ scottTopologicalSpace (fcoord n) :=
    (coordinate_continuous D P n).comp f.continuous
  let fbar (n : Nat) : Y  ->  D n := scottExtend e (fcoord n)
  have hfbar_cont (n : Nat) :
      @Continuous Y (D n) _ scottTopologicalSpace (fbar n) :=
    scottExtend_continuous (hD n) e (fcoord n)
  have hfbar_compat (y : Y) (n : Nat) :
```

```
      (P n).retr (fbar (n + 1) y) = fbar n y := by
    symm
    exact lemma_3_9 (hD n) (hD (n + 1)) (P n) e he
      (hfcoord n) (hfcoord (n + 1)) (fun x => (f x).2 n |>.symm) y
  let fbarInf (y : Y) : InverseLimit D P :=
    <fun n => fbar n y, hfbar_compat y>
  have hfbarInf_cont :
      @Continuous Y (InverseLimit D P) _ (inverseLimitTopology D P) fbarInf := by
    change @Continuous Y (InverseLimit D P) _
      (TopologicalSpace.induced (Subtype.val : InverseLimit D P  ->  forall  n, D n)
        (@Pi.topologicalSpace Nat D (fun _ => scottTopologicalSpace))) fbarInf
    rw [continuous_induced_rng]
    exact continuous_pi hfbar_cont
  refine <<fbarInf, hfbarInf_cont>, fun x => ?_>
  apply Subtype.ext
  funext n
  exact scottExtend_eq_of_continuous (hD n) e he (fcoord n) (hfcoord n) x

/-- The Scott lattice topology constructed on `D inf ` agrees with Scott's literal
    inverse-limit
topology: the subspace topology inherited from the Pi product of the stage Scott
    topologies. -/
theorem scottTopology_eq_induced_pi (hD : forall  n, ContinuousLattice (D n)) :
    (scottTopologicalSpace : TopologicalSpace (InverseLimit D P)) =
      inverseLimitTopology D P := by
  let : TopologicalSpace (InverseLimit D P) := inverseLimitTopology D P
  let : T0Space (InverseLimit D P) := t0Space D P
  exact (theorem_2_12.backward_exact (isInjectiveSpace D P hD)
    (specializationLe_iff D P)).2

/-- **Scott 1972, S4.** Each `x  in  D_ inf ` is the (monotone) lub of its projections:
`x =  bigcup _n i_{n inf }(x_n)`. -/
theorem eq_iSup (x : InverseLimit D P) :
    x =  bigcup  n, embInf D P n (x.1 n) := by
  have hdir := embInf_family_directed D P x
  have hne : (Set.range (fun n => embInf D P n (x.1 n))).Nonempty := Set.
    range_nonempty _
  refine le_antisymm ?_ ?_
  * rw [ <-  Subtype.coe_le_coe]
    have hcoe : (( bigcup  n, embInf D P n (x.1 n) : InverseLimit D P)).1
        = sSup (Subtype.val '' Set.range (fun n => embInf D P n (x.1 n))) := by
      rw [ <-  sSup_range]
      exact coe_sSup_of_directed D P _ hne hdir
    rw [hcoe]
    intro m
    have hmem : (embInf D P m (x.1 m)).1
         in  Subtype.val '' Set.range (fun n => embInf D P n (x.1 n)) :=
      <embInf D P m (x.1 m), <m, rfl>, rfl>
    have hle := (le_sSup hmem) m
    rwa [show (embInf D P m (x.1 m)).1 m = x.1 m from iComp_self D P m (x.1 m)] at hle
  * exact iSup_le fun n => by
      rw [ <-  Subtype.coe_le_coe]; intro m; exact iComp_incl_le D P n x m

end inverseLimit
namespace inverseLimitTopology
/-- Proposition 4.2's projection `j_{ inf n}` is a continuous coordinate map for the
```

```
    explicit
product/subspace inverse-limit topology. -/
theorem projInf_continuous (n : Nat) :
    @Continuous (InverseLimit D P) (D n) (inverseLimitTopology D P)
      scottTopologicalSpace (projInf D P n) :=
  inverseLimit.coordinate_continuous D P n

/-- Proposition 4.2's embedding `i_{n inf }` is also continuous for Scott's literal
    inverse-limit
topology, namely the subspace topology inherited from the product `?? D?`. -/
theorem embInf_continuous (hD : forall  n, ContinuousLattice (D n)) (n : Nat) :
    @Continuous (D n) (InverseLimit D P) scottTopologicalSpace
      (inverseLimitTopology D P) (embInf D P n) := by
  rw [ <-  inverseLimit.scottTopology_eq_induced_pi D P hD]
  exact (embInf D P n).continuous

/-- Corollary 4.3's formula `f inf (x) =  bigcup _n f_n(x_n)` is continuous not only
    for the reconstructed Scott
topology on `D inf `, but explicitly for Scott's product/subspace inverse-limit
    topology. -/
theorem coconeInf_continuous (hD : forall  n, ContinuousLattice (D n))
    {D' : Type*} [CompleteLattice D']
    (f : forall  n, ScottMap (D n) D') :
    @Continuous (InverseLimit D P) D' (inverseLimitTopology D P)
      scottTopologicalSpace (coconeInf D P f) := by
  rw [ <-  inverseLimit.scottTopology_eq_induced_pi D P hD]
  exact continuous_of_preservesDirectedSup (coconeInf_preservesDirectedSup D P f)

end inverseLimitTopology

end InverseLimit

end DomainTheory.ContinuousLattice
```

inverseLimit `namespace inverseLimit` is the coordinate continuity, the specialization order, the T_0 separation, the injectivity, the agreement with the Scott topology, and each thread as the supremum of its stage embeddings. The declarations are `inverseLimit.coordinate_continuous`, `inverseLimit.specializationLe_iff`, `inverseLimit.t0Space`, `inverseLimit.isInjectiveSpac` `inverseLimit.scottTopology_eq_induced_pi`, and `inverseLimit.eq_iSup`, in inverseLimitTopology.lean.

Proposition 4.1 **4.1 Proposition.** Let $\langle D_n, j_n\rangle_{n=0}^{\infty}$ be an inverse system of continuous lattices where each $j_n : D_{n+1} \to D_n$ is a projection. Then the inverse limit space D_∞ is also a continuous lattice.

Proof: We need only show that D_∞ as a T_0-space is injective. So suppose $f_\infty : X \to D_\infty$ is given and $X \subseteq Y$. Define $f_n : X \to D_n$ by $f_n = j_{\infty n} \circ f_\infty$. Let $\bar{f} * n : Y \to D_n$ be the maximal extension of f_n according to 3.8. Now we can see the point of Lemma 3.9: by this construction we guarantee that $\bar{f}n = j_n \circ \bar{f}n + 1$. Hence the required $\bar{f} * \infty : Y \to D_\infty$ exists. □

proposition_4_1.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.ContinuousLattices.theorem_2_12
public import DomainTheory.ContinuousLattice.ContinuousLattices.theorem_2_12_more
public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitTopology
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : Nat  ->  Type u) [forall  n, CompleteLattice (D n)]
variable (P : forall  n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- **Scott 1972, Proposition 4.1.** The inverse limit is injective by the
    coordinatewise
Proposition 3.8/Lemma 3.9 construction; Theorem 2.12 then gives the continuous-lattice
     structure
on the existing componentwise complete lattice. -/
theorem proposition_4_1 (hD : forall  n, ContinuousLattice (D n)) :
    ContinuousLattice (InverseLimit D P) := by
  let : TopologicalSpace (InverseLimit D P) := inverseLimitTopology D P
  let : T0Space (InverseLimit D P) := inverseLimit.t0Space D P
  exact (theorem_2_12.backward_exact (inverseLimit.isInjectiveSpace D P hD)
    (inverseLimit.specializationLe_iff D P)).1

end InverseLimit
end DomainTheory.ContinuousLattice
```

Proposition 4.2 I do not know at the time of writing whether this theorem on inverse limits of continuous lattices extends to sequences where, say, the j_n are only retractions. Fortunately, sufficiently many projections exist to make this construction useful. Note that by reference to the product space construction of D_∞, its lattice ordering is given simply by the relation:

$$x \sqsubseteq y \quad \text{iff} \quad x_n \sqsubseteq y_n \text{ for all } n.$$

4.2 Proposition. Let $\langle D_n, j_n \rangle_{n=0}^{\infty}$ and D_∞ be as in 4.1. Then the maps $j_{\infty n} : D_\infty \to D_n$ are projections.

Proof: The projections $j_n : D_{n+1} \to D_n$, as we know, have their uniquely determined inverses $i_n : D_n \to D_{n+1}$. We can define $i_{n\infty} : D_n \to D_\infty$ by the equation:

$$i_{n\infty}(x) = \langle y_m \rangle_{m=0}^{\infty}$$

where

$$y_m = \begin{cases} j_m(y_{m+1}) & \text{if } m < n, \\ x & \text{if } m = n, \\ i_m(y_{m-1}) & \text{if } m > n. \end{cases}$$

The proof that $i_{n\infty}$ and $j_{\infty n}$ form a projection is now an easy computation. □

proposition_4_2.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.iComp
public import DomainTheory.ContinuousLattice.InverseLimits.embInf
public import DomainTheory.ContinuousLattice.InverseLimits.projInf
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : Nat  ->  Type u) [forall  n, CompleteLattice (D n)]

variable (P : forall  n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- **Scott 1972, Proposition 4.2.** Each `j_{ inf n} : D_ inf   ->  D_n` is a
    projection of continuous
lattices, with embedding `i_{n inf } = embInf n`. -/
noncomputable def proposition_4_2 (n : Nat) :
    IsContinuousLatticeProjection (D n) (InverseLimit D P) where
  incl := embInf D P n
  retr := projInf D P n
  retr_incl x := iComp_self D P n x
  incl_retr_le y := by
    rw [ <-  Subtype.coe_le_coe]
    intro m
    exact iComp_incl_le D P n y m

end InverseLimit

end DomainTheory.ContinuousLattice
```

Corollary 4.3 One should note also the recursion equation:

$$i_{n\infty} = i_{(n+1)\infty} \circ i_n.$$

These maps also make it possible to state this useful equation:

$$x = \bigsqcup_{n=0}^{\infty} i_{n\infty}(x_n),$$

where $x \in D_\infty$. It is easy to check that this is a monotone lub, and so we can say each $x \in D_\infty$ is the limit of its projections x_n. In fact, from what we know about projections, x_n is the best possible approximation to x in the space D_n.

4.3 Corollary. Let the spaces be as in 4.1 and 4.2. Let D' be any complete lattice and suppose continuous functions $f_n : D_n \to D'$ are given so that $f_n = f_{n+1} \circ i_n$. Then we can define $f_\infty : D_\infty \to D'$ by the equation:

$$f_\infty(x) = \bigsqcup_{n=0}^{\infty} f_n(x_n)$$

for $x \in D_\infty$, and we have $f_n = f_\infty \circ i_{n\infty}$. □

corollary_4_3.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.embInf
public import DomainTheory.ContinuousLattice.InverseLimits.coconeInf
public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitTopology
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : Nat  ->  Type u) [forall  n, CompleteLattice (D n)]
variable (P : forall  n, IsContinuousLatticeProjection (D n) (D (n + 1)))

variable {D' : Type*} [CompleteLattice D']

/-- **Scott 1972, Corollary 4.3.** Within complete lattices, `D_ inf ` is also the *
    direct* limit of
the `D_n` along the embeddings `i_n`: for every compatible cocone `f_n : D_n  ->  D'` (
    continuous, with
`f_n = f_{n+1}  comp  i_n`) there is a **unique** continuous `f inf  : D_ inf   ->  D'`
     with `f_n = f inf   comp  i_{n inf }`,
namely `f inf (x) =  bigcup _n f_n(x_n)`. -/
theorem corollary_4_3 (f : forall  n, ScottMap (D n) D')
    (hf : forall  n x, f n x = f (n + 1) ((P n).incl x)) :
    exists ! g : ScottMap (InverseLimit D P) D', forall  n x, f n x = g (embInf D P n
    x) := by
```

```
  refine <<coconeInf D P f,
    continuous_of_preservesDirectedSup (coconeInf_preservesDirectedSup D P f)>, ?_, ?_
    >
  * exact fun n x => (coconeInf_comp_embInf D P f hf n x).symm
  * intro g hg
    ext x
    -- `g x = f inf  x`, since `x =  bigcup _n i_{n inf }(x_n)` and `g` is continuous
    on this directed family
    calc g x = g ( bigcup  n, embInf D P n (x.1 n)) := by rw [ <-  inverseLimit.
    eq_iSup D P x]
      _ = g (sSup (Set.range fun n => embInf D P n (x.1 n))) := by rw [sSup_range]
      _ = sSup (g '' Set.range fun n => embInf D P n (x.1 n)) :=
          g.preservesDirectedSup_coe _ (Set.range_nonempty _) (embInf_family_directed
    D P x)
      _ =  bigcup  n, g (embInf D P n (x.1 n)) := by rw [ <-  Set.range_comp,
    sSup_range]; rfl
      _ =  bigcup  n, f n (x.1 n) := by simp_rw [ <-  hg]

end InverseLimit
end DomainTheory.ContinuousLattice
```

Theorem 4.4 Scott builds the tower in this order. Theorem 4.4 is the statement that the limit is a continuous lattice and is homeomorphic to its own function space. The functorial reading of the same argument is the functorial form.

Dependency diagram. Each box is one of these notes. The arrowhead is on the note that imports. The number is how many direct imports. An arrow already implied by a longer path is left out.

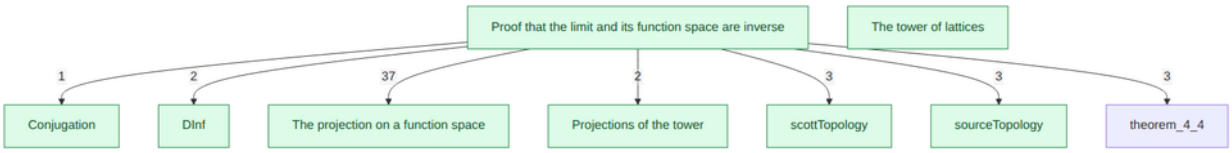


Figure 14: Inverse limits.

The tower of lattices The levels of the function-space tower are D_0 and $D_{n+1} = [D_n \to D_n]$. `CLat` packages a complete lattice so the tower can be defined by recursion, and `towerType` is that recursion. `towerType_zero` and `towerType_succ` are the two levels, and `towerToMap` with `towerCoeFun` applies a successor level as a Scott map.

`CLat.lean`

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.ContinuousLattice.FunctionSpaces.BasicInst33

@[expose] public section

universe u
```

```
namespace DomainTheory.ContinuousLattice

/-- A complete lattice bundled with its instance, used to define the function-space
    tower by
recursion on `Nat` (the type at level `n+1` depends on the lattice structure at level `
    n`). -/
structure CLat : Type (u + 1) where
  carrier : Type u
  [str : CompleteLattice carrier]

attribute [instance] CLat.str

/-- The tower `D_0, [D_0 -> D_0], [[D_0 -> D_0] -> [D_0 -> D_0]], ...` as bundled
    complete lattices. -/
noncomputable def towerCLat (D_0 : CLat.{u}) : Nat  ->  CLat.{u}
  | 0 => D_0
  | (n + 1) => <ScottMap (towerCLat D_0 n).carrier (towerCLat D_0 n).carrier>

/-- The carrier `D_n` of the function-space tower. -/
def towerType (D_0 : CLat.{u}) (n : Nat) : Type u := (towerCLat D_0 n).carrier

noncomputable instance towerCompleteLattice (D_0 : CLat.{u}) (n : Nat) :
    CompleteLattice (towerType D_0 n) := (towerCLat D_0 n).str

/-- `D_{n+1}` is *definitionally* the function space `[D_n  ->  D_n]`. -/
theorem towerType_succ (D_0 : CLat.{u}) (n : Nat) :
    towerType D_0 (n + 1) = ScottMap (towerType D_0 n) (towerType D_0 n) := rfl

@[simp] theorem towerType_zero (D_0 : CLat.{u}) : towerType D_0 0 = D_0.carrier := rfl

/-- View an element of `D_{n+1}` as the Scott map `[D_n  ->  D_n]` it definitionally is
    . -/
def towerToMap {D_0 : CLat.{u}} {n : Nat} (f : towerType D_0 (n + 1)) :
    ScottMap (towerType D_0 n) (towerType D_0 n) := f

/-- Apply an element of `D_{n+1}` as a function `D_n  ->  D_n` (definitional via `
    towerType_succ`). -/
instance towerCoeFun {D_0 : CLat.{u}} {n : Nat} :
    CoeFun (towerType D_0 (n + 1)) (fun _ => towerType D_0 n  ->  towerType D_0 n)
    where
  coe f := towerToMap f

@[simp] theorem towerToMap_coe {D_0 : CLat.{u}} {n : Nat} (f : towerType D_0 (n + 1))
    (x : towerType D_0 n) : towerToMap f x = f x := rfl

end DomainTheory.ContinuousLattice
```

Conjugation `conjMap` sends a map f to $j' \circ f \circ i$, the conjugate of f along a projection. `conjMapFun` is the underlying function, `conjMap.fun_apply` and `conjMap.apply` are the two evaluations, and `conjMap.preservesDirectedSup` is preservation of directed suprema.

`conjMap.lean`

```
module
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.CLat
public import DomainTheory.ContinuousLattice.FunctionSpaces.comp
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.ScottMaps.
    continuous_of_preservesDirectedSup
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

universe u
attribute [instance] CLat.str

section Conj

open Set

variable {Y Z W : Type u} [CompleteLattice Y] [CompleteLattice Z] [CompleteLattice W]

/-- Conjugation `f |-> post comp f comp pre` as a bare function `[Y -> Y] ->
    [W -> Z]`. -/
def conjMapFun (post : ScottMap Y Z) (pre : ScottMap W Y) (f : ScottMap Y Y) :
    ScottMap W Z :=
  post.comp (f.comp pre)

namespace conjMap

theorem fun_apply (post : ScottMap Y Z) (pre : ScottMap W Y) (f : ScottMap Y Y) (x : W
    ) :
    conjMapFun post pre f x = post (f (pre x)) := rfl

theorem preservesDirectedSup (post : ScottMap Y Z) (pre : ScottMap W Y) :
    PreservesDirectedSup (conjMapFun post pre) := by
  intro F hF hFdir
  apply ScottMap.ext
  intro x
  have hdir : DirectedOn (* <= *) ((fun f : ScottMap Y Y => f (pre x)) '' F) := by
    rintro _ <a, ha, rfl> _ <b, hb, rfl>
    obtain <c, hc, hac, hbc> := hFdir a ha b hb
    exact <c (pre x), <c, hc, rfl>, hac (pre x), hbc (pre x)>
  show post ((sSup F : ScottMap Y Y) (pre x)) = (sSup (conjMapFun post pre '' F) :
    ScottMap W Z) x
  rw [ScottMap.sSup_apply F (pre x), post.preservesDirectedSup_coe _ (hF.image _) hdir
    ,
    ScottMap.sSup_apply, Set.image_image, Set.image_image]
  rfl

end conjMap

/-- Conjugation `f |-> post comp f comp pre` as a Scott map `[Y -> Y] -> [W
    -> Z]`. -/
noncomputable def conjMap (post : ScottMap Y Z) (pre : ScottMap W Y) :
    ScottMap (ScottMap Y Y) (ScottMap W Z) :=
```

```
  <conjMapFun post pre, continuous_of_preservesDirectedSup (conjMap.
    preservesDirectedSup post pre)>

namespace conjMap

@[simp] theorem apply (post : ScottMap Y Z) (pre : ScottMap W Y) (f : ScottMap Y Y) (x
     : W) :
    conjMap post pre f x = post (f (pre x)) := rfl

end conjMap

end Conj

end DomainTheory.ContinuousLattice
```

The projection on a function space A projection of D in D' induces a projection of $[D \to D]$ in $[D' \to D']$. These are the inclusion, the retraction, and the projection they form.

Dependency diagram. Each box is one of these notes. The arrowhead is on the note that imports. The number is how many direct imports. An arrow already implied by a longer path is left out.



Figure 15: Inverse limits.

functionSpaceInclFun `functionSpaceInclFun` is the inclusion of function spaces induced by a projection, $f \mapsto i \circ f \circ j$. `functionSpaceInclFun.continuous` is continuity of the underlying map, and `functionSpaceInclFun.outer_continuous` is continuity of the assignment $f \mapsto i \circ f \circ j$.

`functionSpaceInclFun.lean`

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaceTower.conjMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection

@[expose] public section
```

```
namespace DomainTheory.ContinuousLattice

universe u
attribute [instance] CLat.str

namespace functionSpaceInclFun

/-- Continuity of the inner map `i  comp  f  comp  j`. -/
theorem continuous {D D' : Type u}
    [CompleteLattice D] [CompleteLattice D']
    (P : IsContinuousLatticeProjection D D') (f : ScottMap D D) :
    @Continuous D' D' scottTopologicalSpace scottTopologicalSpace
      (fun x => P.incl (f (P.retr x))) :=
  @Continuous.congr D' D' scottTopologicalSpace scottTopologicalSpace
    (P.incl.comp (f.comp P.retr)).1 (fun x => P.incl (f (P.retr x)))
    (P.incl.comp (f.comp P.retr)).continuous (fun _ => rfl)

end functionSpaceInclFun

/-- The inclusion part of `[P  ->  P]`, with its underlying function explicit. -/
def functionSpaceInclFun {D D' : Type u} [CompleteLattice D] [CompleteLattice D']
    (P : IsContinuousLatticeProjection D D') (f : ScottMap D D) : ScottMap D' D' :=
  <fun x => P.incl (f (P.retr x)), functionSpaceInclFun.continuous P f>

namespace functionSpaceInclFun

/-- Continuity, for the lattice Scott topologies on the function spaces, of
`f  |->  i  comp  f  comp  j`. -/
theorem outer_continuous {D D' : Type u}
    [CompleteLattice D] [CompleteLattice D']
    (P : IsContinuousLatticeProjection D D') :
    @Continuous (ScottMap D D) (ScottMap D' D')
      scottTopologicalSpace scottTopologicalSpace (functionSpaceInclFun P) := by
  exact @Continuous.congr (ScottMap D D) (ScottMap D' D')
    scottTopologicalSpace scottTopologicalSpace (conjMap P.incl P.retr).1
    (functionSpaceInclFun P) (conjMap P.incl P.retr).continuous
    (fun f => ScottMap.ext fun _ => rfl)

end functionSpaceInclFun

end DomainTheory.ContinuousLattice
```

functionSpaceRetrFun `functionSpaceRetrFun` is the retraction of function spaces induced by a projection, $g \mapsto j \circ g \circ i$. `functionSpaceRetrFun.continuous` is continuity of the underlying map, and `functionSpaceRetrFun.outer_continuous` is continuity of the assignment $g \mapsto j \circ g \circ i$.

functionSpaceRetrFun.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaceTower.conjMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    IsContinuousLatticeProjection
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

universe u
attribute [instance] CLat.str

namespace functionSpaceRetrFun

/-- Continuity of the inner map `j ∘ g ∘ i`. -/
theorem continuous {D D' : Type u}
    [CompleteLattice D] [CompleteLattice D']
    (P : IsContinuousLatticeProjection D D') (g : ScottMap D' D') :
    @Continuous D D scottTopologicalSpace scottTopologicalSpace
      (fun x => P.retr (g (P.incl x))) :=
  @Continuous.congr D D scottTopologicalSpace scottTopologicalSpace
    (P.retr.comp (g.comp P.incl)).1 (fun x => P.retr (g (P.incl x)))
    (P.retr.comp (g.comp P.incl)).continuous (fun _ => rfl)

end functionSpaceRetrFun

/-- The retraction part of `[P → P]`, with its underlying function explicit. -/
def functionSpaceRetrFun {D D' : Type u} [CompleteLattice D] [CompleteLattice D']
    (P : IsContinuousLatticeProjection D D') (g : ScottMap D' D') : ScottMap D D :=
  <fun x => P.retr (g (P.incl x)), functionSpaceRetrFun.continuous P g>

namespace functionSpaceRetrFun

/-- Continuity, for the lattice Scott topologies on the function spaces, of
`g ↦ j ∘ g ∘ i`. -/
theorem outer_continuous {D D' : Type u}
    [CompleteLattice D] [CompleteLattice D']
    (P : IsContinuousLatticeProjection D D') :
    @Continuous (ScottMap D' D') (ScottMap D D)
      scottTopologicalSpace scottTopologicalSpace (functionSpaceRetrFun P) := by
  exact @Continuous.congr (ScottMap D' D') (ScottMap D D)
    scottTopologicalSpace scottTopologicalSpace (conjMap P.retr P.incl).1
    (functionSpaceRetrFun P) (conjMap P.retr P.incl).continuous
    (fun g => ScottMap.ext fun _ => rfl)

end functionSpaceRetrFun

end DomainTheory.ContinuousLattice
```

functionSpaceProjection `functionSpaceProjection` is the projection of $[D \to D]$ in $[D' \to D']$ induced by a projection of D in D'. `functionSpaceProjection.retrIncl` says the retraction after the inclusion is the identity, and `functionSpaceProjection.inclRetrLe` says the inclusion after the retraction lies below the identity. The two maps are functionSpaceInclFun and functionSpaceRetrFun.

functionSpaceProjection.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.functionSpaceInclFun
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.functionSpaceRetrFun
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_3_7_projection
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def

@[expose] public section

namespace DomainTheory.ContinuousLattice

universe u

attribute [instance] CLat.str

namespace functionSpaceProjection

/-- Projection inequality for the diagonal Proposition 3.7 construction. -/
theorem inclRetrLe {D D' : Type u} [CompleteLattice D] [CompleteLattice D']
    (P : IsContinuousLatticeProjection D D') :
    forall  g, functionSpaceInclFun P (functionSpaceRetrFun P g)  <=  g := by
  intro g
  exact ScottMap.le_def.mpr ((proposition_3_7_projection P P).incl_retr_le g)

/-- Retraction law for the diagonal Proposition 3.7 construction. -/
theorem retrIncl {D D' : Type u} [CompleteLattice D] [CompleteLattice D']
    (P : IsContinuousLatticeProjection D D') :
    forall  f, functionSpaceRetrFun P (functionSpaceInclFun P f) = f :=
  (proposition_3_7_projection P P).retr_incl

end functionSpaceProjection

/-- Scott's Proposition 3.7 construction specialized diagonally. -/
noncomputable def functionSpaceProjection
    {D D' : Type u} [CompleteLattice D] [CompleteLattice D']
    (P : IsContinuousLatticeProjection D D') :
    IsContinuousLatticeProjection (ScottMap D D) (ScottMap D' D') where
  incl := <functionSpaceInclFun P, functionSpaceInclFun.outer_continuous P>
  retr := <functionSpaceRetrFun P, functionSpaceRetrFun.outer_continuous P>
  retr_incl := functionSpaceProjection.retrIncl P
  incl_retr_le := functionSpaceProjection.inclRetrLe P

end DomainTheory.ContinuousLattice
```

Projections of the tower `towerProj` is the projection from D_{n+1} onto D_n in the function-space tower. `towerProj.succ` is the successor step, `towerProj.succ_incl_apply` and `towerProj.succ_retr_apply` are its inclusion and retraction, and `towerProj.incl_apply` is preservation of application.

towerProj.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaceTower.
    functionSpaceProjection
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.conjMap

@[expose] public section

namespace DomainTheory.ContinuousLattice

attribute [instance] CLat.str

/-- The projection tower from Scott's recursion immediately before Theorem 4.4:
`j_{n+1} = [j_n  ->  j_n]`, anchored at `j_0 : [D_0  ->  D_0]  ->  D_0`.

At each successor, the maps and projection laws are the diagonal specialization
`proposition_3_7_projection P P`.  `conjMap` supplies the outer Scott-continuity needed
     to bundle
those canonical maps as an `IsContinuousLatticeProjection`. -/
noncomputable def towerProj (D_0 : CLat.{u})
    (j_0 : IsContinuousLatticeProjection D_0.carrier (ScottMap D_0.carrier D_0.carrier
    )) :
    forall  n, IsContinuousLatticeProjection (towerType D_0 n) (towerType D_0 (n + 1))
  | 0 => j_0
  | (n + 1) => functionSpaceProjection (towerProj D_0 j_0 n)

namespace towerProj

theorem succ (D_0 : CLat.{u})
    (j_0 : IsContinuousLatticeProjection D_0.carrier (ScottMap D_0.carrier D_0.carrier
    )) (n : Nat) :
    let P := towerProj D_0 j_0 n
    (towerProj D_0 j_0 (n + 1)).incl = conjMap P.incl P.retr  /\
      (towerProj D_0 j_0 (n + 1)).retr = conjMap P.retr P.incl := by
  exact <rfl, rfl>

section Tower

variable (D_0 : CLat.{u})
  (j_0 : IsContinuousLatticeProjection D_0.carrier (ScottMap D_0.carrier D_0.carrier))

/-- **Scott 1972, recursion before Theorem 4.4 (from Proposition 3.7).**
`i_{n+1}(x) = i_n  comp  x  comp  j_n`. -/
theorem succ_incl_apply (n : Nat) (x : towerType D_0 (n + 1)) (y : towerType D_0 (n +
    1)) :
    ((towerProj D_0 j_0 (n + 1)).incl x) y
      = (towerProj D_0 j_0 n).incl (x ((towerProj D_0 j_0 n).retr y)) := rfl

/-- **Scott 1972, recursion before Theorem 4.4 (from Proposition 3.7).**
`j_{n+1}(x') = j_n  comp  x'  comp  i_n`. -/
theorem succ_retr_apply (n : Nat) (x' : towerType D_0 (n + 2)) (y : towerType D_0 n) :
    ((towerProj D_0 j_0 (n + 1)).retr x') y
      = (towerProj D_0 j_0 n).retr (x' ((towerProj D_0 j_0 n).incl y)) := rfl

/-- **Scott 1972, S4 (application is preserved).** `i_n(f(x)) = i_{n+1}(f)(i_n(x))` for
     `f  in  D_{n+1}`,
`x  in  D_n`. The embeddings turn functional application into application one level up.
     -/
theorem incl_apply (n : Nat) (f : towerType D_0 (n + 1)) (x : towerType D_0 n) :
    (towerProj D_0 j_0 n).incl (f x)
```

```
      = ((towerProj D_0 j_0 (n + 1)).incl f) ((towerProj D_0 j_0 n).incl x) := by
  rw [succ_incl_apply, (towerProj D_0 j_0 n).retr_incl]

end Tower

end towerProj

end DomainTheory.ContinuousLattice
```

DInf `DInf` is the inverse limit of the function-space tower along its projections.

DInf.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.InverseLimit
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.towerProj

@[expose] public section

namespace DomainTheory.ContinuousLattice

universe u

attribute [instance] CLat.str

section LimitMaps

variable (D_0 : CLat.{u})
  (j_0 : IsContinuousLatticeProjection D_0.carrier (ScottMap D_0.carrier D_0.carrier))

/-- The inverse limit `D_ inf ` of the function-space tower `<D_n, j_n>`. -/
abbrev DInf : Type u := InverseLimit (towerType D_0) (towerProj D_0 j_0)

end LimitMaps

end DomainTheory.ContinuousLattice
```

DInfFn `DInfFn` is the function space of DInf, the Scott maps from that limit to itself.

DInfFn.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.instCompleteLattice
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.DInf

@[expose] public section

namespace DomainTheory.ContinuousLattice

universe u
```

```
attribute [instance] CLat.str

section LimitMaps

variable (D_0 : CLat.{u})
  (j_0 : IsContinuousLatticeProjection D_0.carrier (ScottMap D_0.carrier D_0.carrier))

/-- The function space `[D_ inf   ->  D_ inf ]`. -/
abbrev DInfFn : Type u := ScottMap (DInf D_0 j_0) (DInf D_0 j_0)

end LimitMaps

end DomainTheory.ContinuousLattice
```

Proof that the limit and its function space are inverse These notes construct the embedding of the limit into its function space and the projection back, and show that the two composites are the identity.

Dependency diagram. Each box is one of these notes. The arrowhead is on the note that imports. The number is how many direct imports. An arrow already implied by a longer path is left out.

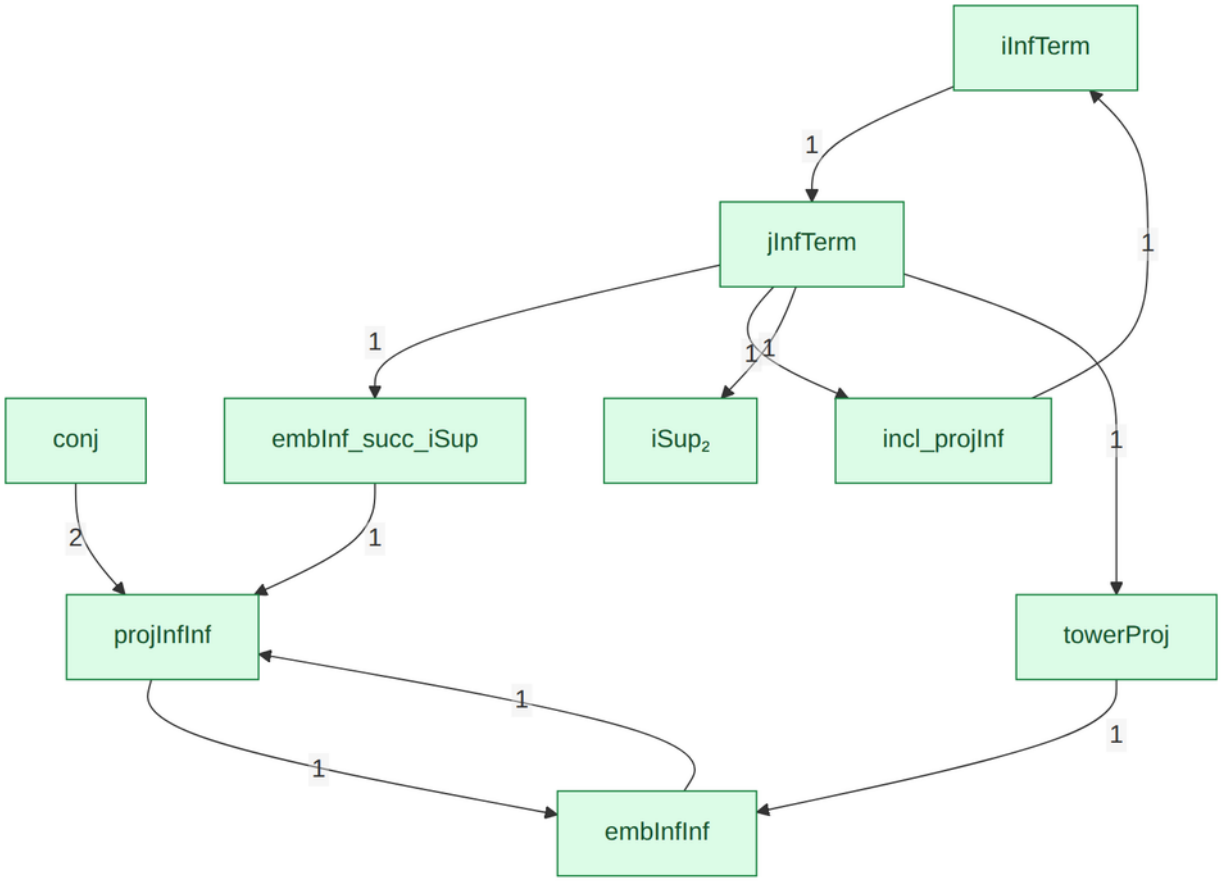


Figure 16: Inverse limits.

iInfTerm `iInfTerm` is the n-th summand of the embedding of DInfFn's domain into its function space, $x \mapsto i_{n\infty} \circ x_{n+1} \circ j_{\infty n}$. `iInfTerm.apply` is that formula.

`iInfTerm.lean`

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.embInf
public import DomainTheory.ContinuousLattice.InverseLimits.projInf
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.DInfFn

@[expose] public section

namespace DomainTheory.ContinuousLattice

attribute [instance] CLat.str

section LimitMaps

variable (D_0 : CLat.{u})
  (j_0 : IsContinuousLatticeProjection D_0.carrier (ScottMap D_0.carrier D_0.carrier))

/-- The `n`-th summand of `i_ inf `: the Scott map `x  |->  i_{n inf }  comp  x_{n+1}
    comp  j_{ inf n}`, where `x_{n+1}` is
the `(n+1)`-st component of `x  in  D_ inf `. As a map `D_ inf   ->  [D_ inf   ->  D_
    inf ]` it is the composite of the
component projection `j_{ inf (n+1)}` with conjugation by `(i_{n inf }, j_{ inf n})`.
    -/
noncomputable def iInfTerm (n : Nat) : ScottMap (DInf D_0 j_0) (DInfFn D_0 j_0) :=
  (conjMap (embInf (towerType D_0) (towerProj D_0 j_0) n)
           (projInf (towerType D_0) (towerProj D_0 j_0) n)).comp
    (projInf (towerType D_0) (towerProj D_0 j_0) (n + 1))

namespace iInfTerm

@[simp] theorem apply (n : Nat) (x z : DInf D_0 j_0) :
    (iInfTerm D_0 j_0 n x) z
      = embInf (towerType D_0) (towerProj D_0 j_0) n
          ((x.1 (n + 1)) ((projInf (towerType D_0) (towerProj D_0 j_0) n) z)) := rfl

end iInfTerm

end LimitMaps

end DomainTheory.ContinuousLattice
```

embInfInf `embInfInf` is the embedding of the tower limit into its function space, $i_\infty(x) = \bigsqcup_n i_{n\infty} \circ x_{n+1} \circ j_{\infty n}$. `embInfInf.apply` is that formula and `embInfInf.preservesDirectedSup` is preservation of directed suprema. The comparison with the projection back is projInfInf and embInfInf_projInfInf.

`embInfInf.lean`

```
module
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.iInfTerm
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_5
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

attribute [instance] CLat.str

section LimitMaps

variable (D_0 : CLat.{u})
  (j_0 : IsContinuousLatticeProjection D_0.carrier (ScottMap D_0.carrier D_0.carrier))

/-- **Scott 1972, S4 (Theorem 4.4).** The embedding `i_ inf  : D_ inf   ->  [D_ inf
    ->  D_ inf ]`,
`i_ inf (x) =  bigcup _n i_{n inf }  comp  x_{n+1}  comp  j_{ inf n}`. It is Scott-
    continuous because it is a supremum of the
Scott maps `iInfTerm n` (suprema in `[D_ inf   ->  [D_ inf   ->  D_ inf ]]` are
    computed pointwise). -/
noncomputable def embInfInf : ScottMap (DInf D_0 j_0) (DInfFn D_0 j_0) :=
   bigcup  n, iInfTerm D_0 j_0 n

namespace embInfInf

/-- `i_ inf ` evaluated at `x` is the pointwise supremum of the summands `iInfTerm n x
    `. -/
theorem apply (x : DInf D_0 j_0) :
    embInfInf D_0 j_0 x =  bigcup  n, iInfTerm D_0 j_0 n x := by
  show (sSup (Set.range (iInfTerm D_0 j_0)) : ScottMap _ _) x = _
  rw [ScottMap.sSup_apply,  <-  Set.range_comp, sSup_range]
  rfl

/-- `i_ inf ` is Scott-continuous (it is a bundled `ScottMap`). -/
theorem preservesDirectedSup :
    PreservesDirectedSup (embInfInf D_0 j_0 : DInf D_0 j_0  ->  DInfFn D_0 j_0) :=
  (proposition_2_5 _).mp (embInfInf D_0 j_0).continuous

end embInfInf

end LimitMaps

end DomainTheory.ContinuousLattice
```

jInfTerm `jInfTerm` is the n-th summand of the projection back onto the limit, $f \mapsto i_{(n+1)\infty}(j_{\infty n} \circ f \circ i_{n\infty})$. `jInfTerm.apply` is that formula. The summands of the embedding are iInfTerm.

jInfTerm.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.iInfTerm

@[expose] public section

namespace DomainTheory.ContinuousLattice

attribute [instance] CLat.str

section LimitMaps

variable (D_0 : CLat.{u})
  (j_0 : IsContinuousLatticeProjection D_0.carrier (ScottMap D_0.carrier D_0.carrier))

/-- The `n`-th summand of `j_ inf `: the Scott map `f  |->  i_{(n+1) inf }(j_{ inf n}
    comp  f  comp  i_{n inf })`. As a map
`[D_ inf   ->  D_ inf ]  ->  D_ inf ` it is conjugation by `(j_{ inf n}, i_{n inf })` (
    landing in `D_{n+1}`) followed by the
embedding `i_{(n+1) inf }`. -/
noncomputable def jInfTerm (n : Nat) : ScottMap (DInfFn D_0 j_0) (DInf D_0 j_0) :=
  (embInf (towerType D_0) (towerProj D_0 j_0) (n + 1)).comp
    (conjMap (projInf (towerType D_0) (towerProj D_0 j_0) n)
             (embInf (towerType D_0) (towerProj D_0 j_0) n))

namespace jInfTerm

@[simp] theorem apply (n : Nat) (f : DInfFn D_0 j_0) :
    jInfTerm D_0 j_0 n f
      = embInf (towerType D_0) (towerProj D_0 j_0) (n + 1)
          (conjMap (projInf (towerType D_0) (towerProj D_0 j_0) n)
                   (embInf (towerType D_0) (towerProj D_0 j_0) n) f) := rfl

end jInfTerm

end LimitMaps

end DomainTheory.ContinuousLattice
```

incl_projInf `incl_projInf_le_projInf_succ` says that embedding the n-th component of a thread one level up stays below the next component.

incl_projInf_le_projInf_succ.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaceTower.jInfTerm

@[expose] public section

namespace DomainTheory.ContinuousLattice

attribute [instance] CLat.str

section LimitMaps
```

```
variable (D_0 : CLat.{u})
  (j_0 : IsContinuousLatticeProjection D_0.carrier (ScottMap D_0.carrier D_0.carrier))

section Thm44b

variable (D_0 : CLat.{u})
  (j_0 : IsContinuousLatticeProjection D_0.carrier (ScottMap D_0.carrier D_0.carrier))

/-- `i_{n inf }(y_n)  sqsub  y_{n+1}`: climbing one level and embedding stays below the
    next component. -/
theorem incl_projInf_le_projInf_succ (n : Nat) (w : DInf D_0 j_0) :
    (towerProj D_0 j_0 n).incl (projInf (towerType D_0) (towerProj D_0 j_0) n w)
      <=  projInf (towerType D_0) (towerProj D_0 j_0) (n + 1) w := by
  have h := (towerProj D_0 j_0 n).incl_retr_le
    (projInf (towerType D_0) (towerProj D_0 j_0) (n + 1) w)
  rwa [show (towerProj D_0 j_0 n).retr (projInf (towerType D_0) (towerProj D_0 j_0) (n
    + 1) w)
      = projInf (towerType D_0) (towerProj D_0 j_0) n w from w.2 n] at h

end Thm44b

end LimitMaps

end DomainTheory.ContinuousLattice
```

iInfTerm_monotone `iInfTerm_le_succ` says the summands of iInfTerm increase with the index, and `iInfTerm_monotone` is that sequence. The successor step uses incl_projInf.

`iInfTerm_monotone.lean`

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaceTower.towerProj
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.
    incl_projInf_le_projInf_succ
public import DomainTheory.ContinuousLattice.InverseLimits.embInf
public import DomainTheory.ContinuousLattice.InverseLimits.embInfFun
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone

@[expose] public section

namespace DomainTheory.ContinuousLattice

attribute [instance] CLat.str

section LimitMaps

variable (D_0 : CLat.{u})
  (j_0 : IsContinuousLatticeProjection D_0.carrier (ScottMap D_0.carrier D_0.carrier))

section Thm44b
```

```
variable (D_0 : CLat.{u})
  (j_0 : IsContinuousLatticeProjection D_0.carrier (ScottMap D_0.carrier D_0.carrier))

/-- `i_ inf ` is monotone in its summand index. -/
theorem iInfTerm_le_succ (x : DInf D_0 j_0) (m : Nat) :
    iInfTerm D_0 j_0 m x  <=  iInfTerm D_0 j_0 (m + 1) x := by
  rw [ScottMap.le_def]
  intro z
  rw [iInfTerm.apply, iInfTerm.apply,
     <-  embInf_succ (towerType D_0) (towerProj D_0 j_0) m
      (x.1 (m + 1) (projInf (towerType D_0) (towerProj D_0 j_0) m z))]
  refine embInf_monotone (towerType D_0) (towerProj D_0 j_0) (m + 1) ?_
  have hab : (towerProj D_0 j_0 m).incl (projInf (towerType D_0) (towerProj D_0 j_0) m
     z)
       <=  projInf (towerType D_0) (towerProj D_0 j_0) (m + 1) z :=
    incl_projInf_le_projInf_succ D_0 j_0 m z
  calc (towerProj D_0 j_0 m).incl
          (x.1 (m + 1) (projInf (towerType D_0) (towerProj D_0 j_0) m z))
      = (towerProj D_0 j_0 m).incl
          (((towerProj D_0 j_0 (m + 1)).retr (x.1 (m + 2)))
            (projInf (towerType D_0) (towerProj D_0 j_0) m z)) := by
        rw [show ((towerProj D_0 j_0 (m + 1)).retr (x.1 (m + 2))) = x.1 (m + 1) from x
    .2 (m + 1)]
    _ = (towerProj D_0 j_0 m).incl
          ((towerProj D_0 j_0 m).retr
            (x.1 (m + 2) ((towerProj D_0 j_0 m).incl
              (projInf (towerType D_0) (towerProj D_0 j_0) m z)))) := by
        rw [towerProj.succ_retr_apply]
    _  <=  x.1 (m + 2) ((towerProj D_0 j_0 m).incl
          (projInf (towerType D_0) (towerProj D_0 j_0) m z)) :=
        (towerProj D_0 j_0 m).incl_retr_le _
    _  <=  x.1 (m + 2) (projInf (towerType D_0) (towerProj D_0 j_0) (m + 1) z) :=
        ScottMap.monotone (x.1 (m + 2)) hab

theorem iInfTerm_monotone (x : DInf D_0 j_0) : Monotone (fun m => iInfTerm D_0 j_0 m x
    ) :=
  monotone_nat_of_le_succ (iInfTerm_le_succ D_0 j_0 x)

end Thm44b

end LimitMaps

end DomainTheory.ContinuousLattice
```

embInf_succ_iSup `embInf_succ_iSup` says the embedding of level $n + 1$ sends the supremum of a monotone sequence to the supremum of the embedded terms.

embInf_succ_iSup.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaceTower.jInfTerm
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import Mathlib.Data.Set.Basic
```

```
@[expose] public section

namespace DomainTheory.ContinuousLattice

attribute [instance] CLat.str

section LimitMaps

variable (D_0 : CLat.{u})
  (j_0 : IsContinuousLatticeProjection D_0.carrier (ScottMap D_0.carrier D_0.carrier))

section Thm44b

variable (D_0 : CLat.{u})
  (j_0 : IsContinuousLatticeProjection D_0.carrier (ScottMap D_0.carrier D_0.carrier))

/-- The inverse-limit embedding commutes with a supremum in `D_{n+1}` (monotone  =>
    directed). -/
theorem embInf_succ_iSup (n : Nat) (f : Nat  ->  towerType D_0 (n + 1)) (hf : Monotone
     f) :
    embInf (towerType D_0) (towerProj D_0 j_0) (n + 1) ( bigcup  m, f m) =
       bigcup  m, embInf (towerType D_0) (towerProj D_0 j_0) (n + 1) (f m) := by
  have hdir : DirectedOn (*  <=  *) (Set.range f) :=
    directedOn_range.2 fun a b => <max a b, hf (le_max_left a b), hf (le_max_right a b
    )>
  have hne := Set.range_nonempty f
  rw [show ( bigcup  m, f m) = sSup (Set.range f) from sSup_range.symm,
    (embInf (towerType D_0) (towerProj D_0 j_0) (n + 1)).preservesDirectedSup_coe
      (Set.range f) hne hdir,  <-  Set.range_comp]
  rfl

end Thm44b

end LimitMaps

end DomainTheory.ContinuousLattice
```

iSup2 `iSup2_monotone_eq_diagonal` says a double supremum over $\mathbb{N} \times \mathbb{N}$, monotone in each argument, equals the supremum of the diagonal.

iSup2_monotone_eq_diagonal.lean

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaceTower.jInfTerm

@[expose] public section

namespace DomainTheory.ContinuousLattice

attribute [instance] CLat.str

section LimitMaps
```

```
variable (D_0 : CLat.{u})
  (j_0 : IsContinuousLatticeProjection D_0.carrier (ScottMap D_0.carrier D_0.carrier))

section Thm44b

variable (D_0 : CLat.{u})
  (j_0 : IsContinuousLatticeProjection D_0.carrier (ScottMap D_0.carrier D_0.carrier))

/-- A monotone double `iSup` over `Nat  x  Nat` equals the diagonal `iSup`. -/
theorem iSup_2_monotone_eq_diagonal {a : Type*} [CompleteLattice a] (f : Nat  ->  Nat
    ->  a)
    (hfm : forall  n, Monotone (f n)) (hfn : forall  m, Monotone (fun n => f n m)) :
     bigcup  n,  bigcup  m, f n m =  bigcup  n, f n n := by
  apply le_antisymm
  * refine iSup_le fun n => iSup_le fun m => ?_
    have hk : n  <=  n  sqsup  m := le_sup_left
    have hk' : m  <=  n  sqsup  m := le_sup_right
    calc f n m  <=  f (n  sqsup  m) m := hfn m hk
      _  <=  f (n  sqsup  m) (n  sqsup  m) := hfm (n  sqsup  m) hk'
      _  <=   bigcup  n', f n' n' := le_iSup (fun n' => f n' n') (n  sqsup  m)
  * refine iSup_le fun n => le_trans (le_iSup (f n) n) (le_iSup (fun n' =>  bigcup  m,
    f n' m) n)

end Thm44b

end LimitMaps

end DomainTheory.ContinuousLattice
```

conjMap `conjMap_iSup` says conjugation commutes with a supremum indexed by the natural numbers. `conjMap_incl_le_conjMap_succ` says conjugation climbs one step of the tower. The general conjugate is conjugation.

`conjMap_iSup.lean`

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaceTower.jInfTerm
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.
    incl_projInf_le_projInf_succ
public import DomainTheory.ContinuousLattice.InverseLimits.embInf
public import DomainTheory.ContinuousLattice.InverseLimits.embInfFun
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

attribute [instance] CLat.str
```

```
section LimitMaps

variable (D_0 : CLat.{u})
  (j_0 : IsContinuousLatticeProjection D_0.carrier (ScottMap D_0.carrier D_0.carrier))

/-- Conjugation commutes with a supremum indexed by `Nat` (the range is directed when `
    f` is monotone).
Since `conjMap post pre` is itself a Scott map, this is just preservation of directed
    suprema. -/
theorem conjMap_iSup (n : Nat) (post : ScottMap (DInf D_0 j_0) (towerType D_0 n))
    (pre : ScottMap (towerType D_0 n) (DInf D_0 j_0))
    (f : Nat  ->  ScottMap (DInf D_0 j_0) (DInf D_0 j_0)) (hf : Monotone f) :
    conjMap post pre ( bigcup  m, f m) =  bigcup  m, conjMap post pre (f m) := by
  have hdir : DirectedOn (*  <=  *) (Set.range f) :=
    directedOn_range.2 fun a b => <max a b, hf (le_max_left a b), hf (le_max_right a b
    )>
  have hne := Set.range_nonempty f
  rw [show ( bigcup  m, f m) = sSup (Set.range f) from sSup_range.symm,
    (conjMap post pre).preservesDirectedSup_coe (Set.range f) hne hdir,  <-  Set.
    range_comp]
  rfl

/-- **Conjugation climbs the tower (one step).** Lifting `conjMap (j_{ inf n}, i_{n inf
     }) f` along `i_{n+1}`
stays below `conjMap (j_{ inf (n+1)}, i_{(n+1) inf }) f`. Both sides live in `D_{n+2}`;
     the inequality is the
algebraic content that makes the double sup defining `j_ inf   comp  i_ inf ` monotone
    in the level index `n`. -/
theorem conjMap_incl_le_conjMap_succ (n : Nat) (f : DInfFn D_0 j_0) :
    (towerProj D_0 j_0 (n + 1)).incl
        (conjMap (projInf (towerType D_0) (towerProj D_0 j_0) n)
                 (embInf (towerType D_0) (towerProj D_0 j_0) n) f)
       <=  conjMap (projInf (towerType D_0) (towerProj D_0 j_0) (n + 1))
                (embInf (towerType D_0) (towerProj D_0 j_0) (n + 1)) f := by
  refine ScottMap.le_def.mpr fun y => ?_
  calc (towerProj D_0 j_0 n).incl
          (projInf (towerType D_0) (towerProj D_0 j_0) n
            (f (embInf (towerType D_0) (towerProj D_0 j_0) n ((towerProj D_0 j_0 n).
    retr y))))
       <=  (towerProj D_0 j_0 n).incl
          (projInf (towerType D_0) (towerProj D_0 j_0) n
            (f (embInf (towerType D_0) (towerProj D_0 j_0) (n + 1) y))) := by
        refine (towerProj D_0 j_0 n).incl.monotone
          ((projInf (towerType D_0) (towerProj D_0 j_0) n).monotone (f.monotone ?_))
        rw [ <-  embInf_succ (towerType D_0) (towerProj D_0 j_0) n ((towerProj D_0 j_0
     n).retr y)]
        exact embInf_monotone (towerType D_0) (towerProj D_0 j_0) (n + 1)
          ((towerProj D_0 j_0 n).incl_retr_le y)
    _  <=  projInf (towerType D_0) (towerProj D_0 j_0) (n + 1)
          (f (embInf (towerType D_0) (towerProj D_0 j_0) (n + 1) y)) :=
        incl_projInf_le_projInf_succ D_0 j_0 n
          (f (embInf (towerType D_0) (towerProj D_0 j_0) (n + 1) y))

end LimitMaps

end DomainTheory.ContinuousLattice
```

conj `conj_iInfTerm_eq` says conjugating a summand of i_∞ by the projection and embedding of that level recovers the next component of the thread.

`conj_iInfTerm_eq.lean`

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaceTower.jInfTerm
public import DomainTheory.ContinuousLattice.InverseLimits.proposition_4_2
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_3_7_projection

@[expose] public section

namespace DomainTheory.ContinuousLattice

attribute [instance] CLat.str

section LimitMaps

variable (D_0 : CLat.{u})
  (j_0 : IsContinuousLatticeProjection D_0.carrier (ScottMap D_0.carrier D_0.carrier))

section Thm44b

variable (D_0 : CLat.{u})
  (j_0 : IsContinuousLatticeProjection D_0.carrier (ScottMap D_0.carrier D_0.carrier))

/-- **Diagonal simplification.** `conjMap (j_{ inf n}, i_{n inf })` applied to the `n`-
    th summand of `i_ inf `
recovers the component `x_{n+1}`. This is Proposition 3.7's equation
`j_{[*]}  comp  i_{[*]} = id`, diagonally specialized to the projection from
    Proposition 4.2. -/
theorem conj_iInfTerm_eq (n : Nat) (x : DInf D_0 j_0) :
    conjMap (projInf (towerType D_0) (towerProj D_0 j_0) n)
            (embInf (towerType D_0) (towerProj D_0 j_0) n)
            (iInfTerm D_0 j_0 n x) = x.1 (n + 1) :=
  (proposition_3_7_projection
      (proposition_4_2 (towerType D_0) (towerProj D_0 j_0) n)
      (proposition_4_2 (towerType D_0) (towerProj D_0 j_0) n)).retr_incl
    (projInf (towerType D_0) (towerProj D_0 j_0) (n + 1) x)

end Thm44b

end LimitMaps

end DomainTheory.ContinuousLattice
```

towerProj `towerProj_retr_conjMap_succ` says the tower retraction sends the conjugate at level $n+1$ to the conjugate at level n. The tower projections themselves are projections of the tower.

`towerProj_retr_conjMap_succ.lean`

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaceTower.jInfTerm
```

```
public import DomainTheory.ContinuousLattice.InverseLimits.embInf

@[expose] public section

namespace DomainTheory.ContinuousLattice

attribute [instance] CLat.str

section LimitMaps

variable (D_0 : CLat.{u})
  (j_0 : IsContinuousLatticeProjection D_0.carrier (ScottMap D_0.carrier D_0.carrier))

section Thm44b

variable (D_0 : CLat.{u})
  (j_0 : IsContinuousLatticeProjection D_0.carrier (ScottMap D_0.carrier D_0.carrier))

/-- **Step 1 (Lemma-4.5 recursion).** `j_{n+1}(j_{ inf (n+1)}  comp  f  comp  i_{(n+1)
    inf }) = j_{ inf n}  comp  f  comp  i_{n inf }`.
This is the equality counterpart of `conjMap_incl_le_conjMap_succ`; it is the
    hypothesis Lemma 4.5
needs to recognize `j_ inf (f)` from its restrictions. -/
theorem towerProj_retr_conjMap_succ (n : Nat) (f : DInfFn D_0 j_0) :
    (towerProj D_0 j_0 (n + 1)).retr
        (conjMap (projInf (towerType D_0) (towerProj D_0 j_0) (n + 1))
                 (embInf (towerType D_0) (towerProj D_0 j_0) (n + 1)) f)
      = conjMap (projInf (towerType D_0) (towerProj D_0 j_0) n)
                (embInf (towerType D_0) (towerProj D_0 j_0) n) f := by
  apply ScottMap.ext
  intro y
  show (towerProj D_0 j_0 n).retr
      (projInf (towerType D_0) (towerProj D_0 j_0) (n + 1)
        (f (embInf (towerType D_0) (towerProj D_0 j_0) (n + 1)
          ((towerProj D_0 j_0 n).incl y))))
    = projInf (towerType D_0) (towerProj D_0 j_0) n
        (f (embInf (towerType D_0) (towerProj D_0 j_0) n y))
  rw [embInf_succ (towerType D_0) (towerProj D_0 j_0) n y]
  exact (f (embInf (towerType D_0) (towerProj D_0 j_0) n y)).2 n

end Thm44b

end LimitMaps

end DomainTheory.ContinuousLattice
```

projInfInf `projInfInf` is the projection from the function space back onto the tower limit, $j_\infty(f) = \bigsqcup_n i_{(n+1)\infty}(j_{\infty n} \circ f \circ i_{n\infty})$. `projInfInf.apply` is that formula, `projInfInf.preservesDirectedSup` is preservation of directed suprema, and `projInfInf.comp_embInfInf` says $j_\infty \circ i_\infty = \mathrm{id}$. The other composite is embInfInf.

projInfInf.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.jInfTerm
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.embInfInf
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.conjMap_iSup
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.embInf_succ_iSup
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.conj_iInfTerm_eq
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.iInfTerm_monotone
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.
    iSup_2_monotone_eq_diagonal
public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitTopology
public import DomainTheory.ContinuousLattice.InverseLimits.embInf
public import DomainTheory.ContinuousLattice.InverseLimits.embInfFun
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.le_def
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.FunctionSpaces.comp_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.idMap_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.idMap
public import DomainTheory.ContinuousLattice.ScottMaps.proposition_2_5
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

attribute [instance] CLat.str

section LimitMaps

variable (D_0 : CLat.{u})
  (j_0 : IsContinuousLatticeProjection D_0.carrier (ScottMap D_0.carrier D_0.carrier))

/-- **Scott 1972, S4 (Theorem 4.4).** The projection `j_ inf  : [D_ inf   ->  D_ inf ]
     ->  D_ inf `,
`j_ inf (f) =  bigcup _n i_{(n+1) inf }(j_{ inf n}  comp  f  comp  i_{n inf })`. Scott-
    continuous as a supremum of the Scott maps
`jInfTerm n`. -/
noncomputable def projInfInf : ScottMap (DInfFn D_0 j_0) (DInf D_0 j_0) :=
   bigcup  n, jInfTerm D_0 j_0 n

namespace projInfInf

/-- `j_ inf ` evaluated at `f` is the supremum of the summands `jInfTerm n f`. -/
theorem apply (f : DInfFn D_0 j_0) :
    projInfInf D_0 j_0 f =  bigcup  n, jInfTerm D_0 j_0 n f := by
  show (sSup (Set.range (jInfTerm D_0 j_0)) : ScottMap _ _) f = _
  rw [ScottMap.sSup_apply,  <-  Set.range_comp, sSup_range]
  rfl

/-- `j_ inf ` is Scott-continuous (it is a bundled `ScottMap`). -/
theorem preservesDirectedSup :
    PreservesDirectedSup (projInfInf D_0 j_0 : DInfFn D_0 j_0  ->  DInf D_0 j_0) :=
  (proposition_2_5 _).mp (projInfInf D_0 j_0).continuous

/-- `j_ inf (i_ inf (x)) =  bigcup _n i_{(n+1) inf }(x_{n+1})`. -/
```

```
theorem embInfInf_eq (x : DInf D_0 j_0) :
    projInfInf D_0 j_0 (embInfInf D_0 j_0 x) =  bigcup  n, embInf (towerType D_0) (
    towerProj D_0 j_0) (n + 1) (x.1 (n + 1)) := by
  rw [apply]
  set g := fun n m =>
    embInf (towerType D_0) (towerProj D_0 j_0) (n + 1)
      (conjMap (projInf (towerType D_0) (towerProj D_0 j_0) n)
               (embInf (towerType D_0) (towerProj D_0 j_0) n)
               (iInfTerm D_0 j_0 m x)) with hg
  have hmono (n : Nat) : Monotone (fun m =>
      conjMap (projInf (towerType D_0) (towerProj D_0 j_0) n)
              (embInf (towerType D_0) (towerProj D_0 j_0) n)
              (iInfTerm D_0 j_0 m x)) := fun a b hab =>
    (conjMap (projInf (towerType D_0) (towerProj D_0 j_0) n)
             (embInf (towerType D_0) (towerProj D_0 j_0) n)).monotone
      (ScottMap.le_def.mpr (iInfTerm_monotone D_0 j_0 x hab))
  have hinner (n : Nat) : jInfTerm D_0 j_0 n (embInfInf D_0 j_0 x) =  bigcup  m, g n m
     :=
    calc jInfTerm D_0 j_0 n (embInfInf D_0 j_0 x)
        = embInf (towerType D_0) (towerProj D_0 j_0) (n + 1)
            ( bigcup  m, conjMap (projInf (towerType D_0) (towerProj D_0 j_0) n)
                         (embInf (towerType D_0) (towerProj D_0 j_0) n)
                         (iInfTerm D_0 j_0 m x)) := by
          rw [jInfTerm.apply, embInfInf.apply,
            conjMap_iSup D_0 j_0 n (projInf (towerType D_0) (towerProj D_0 j_0) n)
              (embInf (towerType D_0) (towerProj D_0 j_0) n)
              (fun m => iInfTerm D_0 j_0 m x) (iInfTerm_monotone D_0 j_0 x)]
          rfl
      _ =  bigcup  m, embInf (towerType D_0) (towerProj D_0 j_0) (n + 1)
             (conjMap (projInf (towerType D_0) (towerProj D_0 j_0) n)
                      (embInf (towerType D_0) (towerProj D_0 j_0) n)
                      (iInfTerm D_0 j_0 m x)) :=
          embInf_succ_iSup D_0 j_0 n _ (hmono n)
      _ =  bigcup  m, g n m := by simp only [hg]
  have g_mono_m (n : Nat) : Monotone (g n) := by
    intro a b hab
    rw [hg]
    exact (embInf (towerType D_0) (towerProj D_0 j_0) (n + 1)).monotone
      ((conjMap (projInf (towerType D_0) (towerProj D_0 j_0) n)
                (embInf (towerType D_0) (towerProj D_0 j_0) n)).monotone
        (ScottMap.le_def.mpr (iInfTerm_monotone D_0 j_0 x hab)))
  have g_mono_n_succ (m n : Nat) : g n m  <=  g (n + 1) m := by
    rw [hg]
    dsimp only
    rw [ <-  embInf_succ (towerType D_0) (towerProj D_0 j_0) (n + 1)
        (conjMap (projInf (towerType D_0) (towerProj D_0 j_0) n)
                 (embInf (towerType D_0) (towerProj D_0 j_0) n)
                 (iInfTerm D_0 j_0 m x))]
    exact embInf_monotone (towerType D_0) (towerProj D_0 j_0) (n + 2)
      (conjMap_incl_le_conjMap_succ D_0 j_0 n (iInfTerm D_0 j_0 m x))
  have g_mono_n (m : Nat) : Monotone (fun n => g n m) :=
    monotone_nat_of_le_succ (g_mono_n_succ m)
  have hin : ( bigcup  n, jInfTerm D_0 j_0 n (embInfInf D_0 j_0 x)) =  bigcup  n,
    bigcup  m, g n m := by
    congr 1
    funext n
```

```
    exact hinner n
  rw [hin, iSup_2_monotone_eq_diagonal g g_mono_m g_mono_n]
  congr 1
  funext n
  rw [hg]
  dsimp only
  rw [conj_iInfTerm_eq D_0 j_0 n x]

/-- **Scott 1972, S4 (Theorem 4.4, first half).** `j_ inf   comp  i_ inf  = id` on `D_
    inf `. -/
theorem comp_embInfInf :
    (projInfInf D_0 j_0).comp (embInfInf D_0 j_0) = ScottMap.idMap := by
  apply ScottMap.ext
  intro x
  have hmono : Monotone (fun k => embInf (towerType D_0) (towerProj D_0 j_0) k (x.1 k)
    ) :=
    monotone_nat_of_le_succ (embInf_le_succ (towerType D_0) (towerProj D_0 j_0) x)
  rw [ScottMap.comp_apply, ScottMap.idMap_apply, embInfInf_eq D_0 j_0 x,
    Monotone.iSup_nat_add hmono 1]
  exact (inverseLimit.eq_iSup (towerType D_0) (towerProj D_0 j_0) x).symm

theorem embInfInf (x : DInf D_0 j_0) :
    projInfInf D_0 j_0 (embInfInf D_0 j_0 x) = x := by
  have h := congrArg (fun g => g x) (comp_embInfInf D_0 j_0)
  simpa [ScottMap.comp_apply, ScottMap.idMap_apply] using h

end projInfInf

end LimitMaps

end DomainTheory.ContinuousLattice
```

embInfInf_projInfInf `embInfInf_comp_projInfInf` says $i_\infty \circ j_\infty = \mathrm{id}$. `embInfInf_projInfInf` is that identity at a map, and `embInfInf_le_iff` is the order reflection of embInfInf. The other composite is projInfInf.

`embInfInf_projInfInf.lean`

```
module

public import DomainTheory.ContinuousLattice.FunctionSpaceTower.embInfInf
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.projInfInf
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.
    iSup_2_monotone_eq_diagonal
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.
    towerProj_retr_conjMap_succ
public import DomainTheory.ContinuousLattice.InverseLimits.lemma_4_5
public import DomainTheory.ContinuousLattice.InverseLimits.inverseLimitTopology
public import DomainTheory.ContinuousLattice.InverseLimits.embInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.comp_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.idMap_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.idMap
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

attribute [instance] CLat.str

section LimitMaps

variable (D_0 : CLat.{u})
  (j_0 : IsContinuousLatticeProjection D_0.carrier (ScottMap D_0.carrier D_0.carrier))

/-- **Scott 1972, S4 (Theorem 4.4, second half).** `i_ inf   comp  j_ inf  = id` on `[
    D_ inf   ->  D_ inf ]`. -/
theorem embInfInf_comp_projInfInf :
    (embInfInf D_0 j_0).comp (projInfInf D_0 j_0) = ScottMap.idMap := by
  apply ScottMap.ext
  intro f
  rw [ScottMap.comp_apply, ScottMap.idMap_apply]
  apply ScottMap.ext
  intro z
  -- `r_n = i_{n inf }  comp  j_{ inf n}`, the Proposition-4.2 approximation of the
    identity on `D_ inf `.
  set r : Nat  ->  ScottMap (DInf D_0 j_0) (DInf D_0 j_0) :=
    fun n => (embInf (towerType D_0) (towerProj D_0 j_0) n).comp
               (projInf (towerType D_0) (towerProj D_0 j_0) n) with hr
  have hrw : forall  n (w : DInf D_0 j_0), r n w
      = embInf (towerType D_0) (towerProj D_0 j_0) n
          (projInf (towerType D_0) (towerProj D_0 j_0) n w) := by
    intro n w
    simp only [hr, ScottMap.comp_apply]
  have hr_mono : forall  (w : DInf D_0 j_0), Monotone (fun m => r m w) := by
    intro w
    refine monotone_nat_of_le_succ (fun m => ?_)
    show r m w  <=  r (m + 1) w
    rw [hrw, hrw]
    exact embInf_le_succ (towerType D_0) (towerProj D_0 j_0) w m
  -- The functional equation `id =  bigcup _n r_n` (remark following Proposition 4.2),
    pointwise.
  have hA : forall  (w : DInf D_0 j_0), w =  bigcup  m, r m w := by
    intro w
    have h1 : ( bigcup  m, r m w)
        =  bigcup  m, embInf (towerType D_0) (towerProj D_0 j_0) m
                 (projInf (towerType D_0) (towerProj D_0 j_0) m w) :=
      iSup_congr (fun m => hrw m w)
    rw [h1]
    exact inverseLimit.eq_iSup (towerType D_0) (towerProj D_0 j_0) w
  -- Evaluating a supremum of Scott maps `D_ inf   ->  D_ inf ` at a point is pointwise
    .
  have hsup_apply : forall  (g : Nat  ->  ScottMap (DInf D_0 j_0) (DInf D_0 j_0)) (w :
     DInf D_0 j_0),
      ( bigcup  n, g n) w =  bigcup  n, g n w := by
    intro g w
    rw [show ( bigcup  n, g n) = sSup (Set.range g) from sSup_range.symm,
```

```
    ScottMap.sSup_apply,  <-  Set.range_comp, sSup_range]
  rfl
-- A Scott map commutes with a monotone `Nat`-indexed supremum.
have hcont : forall  (g : ScottMap (DInf D_0 j_0) (DInf D_0 j_0)) (a : Nat  ->  DInf
   D_0 j_0),
    Monotone a  ->  g ( bigcup  m, a m) =  bigcup  m, g (a m) := by
  intro g a ha
  have hdir : DirectedOn (*  <=  *) (Set.range a) :=
    directedOn_range.2 fun i j => <max i j, ha (le_max_left i j), ha (le_max_right i
   j)>
  rw [show ( bigcup  m, a m) = sSup (Set.range a) from sSup_range.symm,
    g.preservesDirectedSup_coe (Set.range a) (Set.range_nonempty a) hdir,
     <-  Set.range_comp, sSup_range]
  rfl
-- `j_ inf (f) =  bigcup ? i_{(k+1) inf }(j_{ inf k}  comp  f  comp  i_{k inf })`.
have hpi : projInfInf D_0 j_0 f
    =  bigcup  k, embInf (towerType D_0) (towerProj D_0 j_0) (k + 1)
        (conjMap (projInf (towerType D_0) (towerProj D_0 j_0) k)
                 (embInf (towerType D_0) (towerProj D_0 j_0) k) f) := by
  rw [projInfInf.apply]
  exact iSup_congr (fun n => jInfTerm.apply D_0 j_0 n f)
-- Lemma 4.5: the `(n+1)`-st component of `j_ inf (f)` is the restriction `j_{ inf n}
    comp  f  comp  i_{n inf }`.
have hcoord : forall  n, (projInfInf D_0 j_0 f).1 (n + 1)
    = conjMap (projInf (towerType D_0) (towerProj D_0 j_0) n)
              (embInf (towerType D_0) (towerProj D_0 j_0) n) f := by
  intro n
  rw [hpi]
  exact lemma_4_5 (towerType D_0) (towerProj D_0 j_0)
    (fun k => conjMap (projInf (towerType D_0) (towerProj D_0 j_0) k)
                      (embInf (towerType D_0) (towerProj D_0 j_0) k) f)
    (fun m => towerProj_retr_conjMap_succ D_0 j_0 m f) n
-- Evaluate `i_ inf (j_ inf (f))` pointwise as a sup of summands.
have hev : embInfInf D_0 j_0 (projInfInf D_0 j_0 f) z
    =  bigcup  n, (iInfTerm D_0 j_0 n (projInfInf D_0 j_0 f)) z := by
  rw [embInfInf.apply]
  exact hsup_apply (fun n => iInfTerm D_0 j_0 n (projInfInf D_0 j_0 f)) z
-- Each summand is `r_n (f (r_n z))` (using `hcoord` and `conjMap`).
have hterm : forall  n, (iInfTerm D_0 j_0 n (projInfInf D_0 j_0 f)) z = r n (f (r n
  z)) := by
  intro n
  rw [hrw, hrw, iInfTerm.apply, hcoord n]
  rfl
have hmono_frz : Monotone (fun m => f (r m z)) :=
  fun a b hab => f.monotone (hr_mono z hab)
have hfm : forall  n, Monotone (fun m => r n (f (r m z))) :=
  fun n _ _ hab => (r n).monotone (f.monotone (hr_mono z hab))
have hfn : forall  m, Monotone (fun n => r n (f (r m z))) :=
  fun m => hr_mono (f (r m z))
-- The analytic step (Scott ~1326-1334): confine the lub via continuity of `f`, then
  collapse the
-- monotone double sup to its diagonal.
have hfz : f z =  bigcup  n, r n (f (r n z)) :=
  calc f z =  bigcup  k, r k (f z) := hA (f z)
    _ =  bigcup  k, r k (f ( bigcup  m, r m z)) := by
          refine iSup_congr (fun k => ?_)
```

```
            rw [ <-  hA z]
      _ =  bigcup  k, r k ( bigcup  m, f (r m z)) := by
            refine iSup_congr (fun k => ?_)
            rw [hcont f (fun m => r m z) (hr_mono z)]
      _ =  bigcup  k,  bigcup  m, r k (f (r m z)) :=
            iSup_congr (fun k => hcont (r k) (fun m => f (r m z)) hmono_frz)
      _ =  bigcup  n, r n (f (r n z)) :=
            iSup_2_monotone_eq_diagonal (fun n m => r n (f (r m z))) hfm hfn
  calc embInfInf D_0 j_0 (projInfInf D_0 j_0 f) z
      =  bigcup  n, (iInfTerm D_0 j_0 n (projInfInf D_0 j_0 f)) z := hev
    _ =  bigcup  n, r n (f (r n z)) := iSup_congr hterm
    _ = f z := hfz.symm

theorem embInfInf_projInfInf (f : DInfFn D_0 j_0) :
    embInfInf D_0 j_0 (projInfInf D_0 j_0 f) = f := by
  have h := congrArg (fun g => g f) (embInfInf_comp_projInfInf D_0 j_0)
  simpa [ScottMap.comp_apply, ScottMap.idMap_apply] using h

theorem embInfInf_le_iff (x y : DInf D_0 j_0) :
    embInfInf D_0 j_0 x  <=  embInfInf D_0 j_0 y  <->  x  <=  y := by
  constructor
  * intro h
    have := (projInfInf D_0 j_0).monotone h
    rwa [projInfInf.embInfInf, projInfInf.embInfInf] at this
  * intro h; exact (embInfInf D_0 j_0).monotone h

end LimitMaps

end DomainTheory.ContinuousLattice
```

scottTopology `theorem_4_4.scottTopology_homeomorph` is the pair i_∞, j_∞ as a homeomorphism when both sides carry the Scott topology. The source topologies are sourceTopology.

theorem_4_4.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.projInfInf
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.embInfInf_projInfInf
public import DomainTheory.ContinuousLattice.FunctionSpaces.monotone
public import DomainTheory.ContinuousLattice.InverseLimits.proposition_4_1
public import DomainTheory.ContinuousLattice.FunctionSpaces.theorem_3_3
public import DomainTheory.ContinuousLattice.FunctionSpaces.theorem_3_3_more
public import DomainTheory.ContinuousLattice.FunctionSpaceTower.theorem_4_4_more

@[expose] public section

namespace DomainTheory.ContinuousLattice

attribute [instance] CLat.str

section LimitMaps
```

```
variable (D_0 : CLat.{u})
  (j_0 : IsContinuousLatticeProjection D_0.carrier (ScottMap D_0.carrier D_0.carrier))

section Thm44d

variable (D_0 : CLat.{u})
  (j_0 : IsContinuousLatticeProjection D_0.carrier (ScottMap D_0.carrier D_0.carrier))

/-- The inverse pair Scott writes down in the proof of Theorem 4.4:
`i_ inf  = embInfInf` and `j_ inf  = projInfInf` satisfy `j_ inf   comp  i_ inf  = id`
    and `i_ inf   comp  j_ inf  = id`. -/
theorem theorem_4_4_inverses :
    (projInfInf D_0 j_0).comp (embInfInf D_0 j_0) = ScottMap.idMap  /\
    (embInfInf D_0 j_0).comp (projInfInf D_0 j_0) = ScottMap.idMap :=
  <projInfInf.comp_embInfInf D_0 j_0, embInfInf_comp_projInfInf D_0 j_0>

/-- Lattice isomorphism along the same pair `i_ inf `, `j_ inf `. Scott's abstract also
     says the spaces
are isomorphic; this is not the statement of Theorem 4.4 (that statement is the
    homeomorphism). -/
noncomputable def theorem_4_4_orderIso : OrderIso (DInf D_0 j_0) (DInfFn D_0 j_0) :=
  (Equiv.mk (embInfInf D_0 j_0) (projInfInf D_0 j_0)
      (projInfInf.embInfInf D_0 j_0) (embInfInf_projInfInf D_0 j_0)).toOrderIso
    (embInfInf D_0 j_0).monotone (projInfInf D_0 j_0).monotone

/-- **Scott 1972, Theorem 4.4** (`sources/ScottContinLatt1972.md`):
*The inverse limit $D_\infty$ of the recursively defined sequence
$\langle D_n, j_n \rangle_{n=0}^{\infty}$ of function spaces is not only a
continuous lattice, but it is also homeomorphic to its own function space
$[D_\infty \to D_\infty]$.*  Here ?inverse limit? carries Scott's subspace
topology from the Pi product, while the function space carries Definition 3.1's
pointwise/Pi-induced topology. -/
theorem theorem_4_4 (h_0 : ContinuousLattice D_0.carrier) :
    @ContinuousLattice (DInf D_0 j_0) inferInstance  /\
      Nonempty
        (@Homeomorph (DInf D_0 j_0) (DInfFn D_0 j_0)
          (inverseLimitTopology (towerType D_0) (towerProj D_0 j_0))
          (scottMapInducedPiTopology (DInf D_0 j_0) (DInf D_0 j_0))) := by
  have hTower : forall  n, ContinuousLattice (towerType D_0 n) := by
    intro n
    induction n with
    | zero => exact h_0
    | succ n ih => exact theorem_3_3.isContinuousLattice ih ih
  refine <proposition_4_1 (towerType D_0) (towerProj D_0 j_0) hTower, ?_>
  exact <theorem_4_4.sourceTopology_homeomorph D_0 j_0 hTower>

end Thm44d
end LimitMaps

end DomainTheory.ContinuousLattice
```

sourceTopology `theorem_4_4.sourceTopology_homeomorph` is the homeomorphism of Theorem 4.4 for the inverse-limit subspace topology and the pointwise topology. It transports scottTopology. The statement that calls it is theorem_4_4.

theorem_4_4 `theorem_4_4_inverses` is the inverse pair in the proof of Theorem 4.4. `theorem_4_4_orderIso` is that pair as an order isomorphism. `theorem_4_4` is the statement: the limit is a continuous lattice and is homeomorphic to its function space. The homeomorphism it calls is sourceTopology.

The import of this last result is that within the category of complete lattices, the space D_∞ is not only the inverse limit of the D_n, but it is also the *direct* limit. (One system of spaces here uses the j_n as connecting maps, the other the i_n.) This is the algebraic result that lies at the heart of our main result about inverse limits of function spaces.

Turning to function spaces, let $D = D_0$ be a given continuous lattice. As we have seen in 3.13, there are many ways of making D_0 a projection of $D_1 = [D_0 \to D_0]$. Choose one such given by a pair i_0, j_0. Define recursively:

$$D_{n+1} = [D_n \to D_n]$$

and introduce the pairs i_{n+1}, j_{n+1} making D_{n+1} a projection of D_{n+2} by the method of 3.7. Specifically we shall have for $x \in D_{n+1}$ and $x' \in D_{n+2}$:

$$i_{n+1}(x) = i_n \circ x \circ j_n,$$

$$j_{n+1}(x') = j_n \circ x' \circ i_n.$$

Since these spaces are more than continuous lattices being function spaces, it is interesting to note that the maps i_n preserve function value as an algebraic operation as follows:

$$i_n(f(x)) = i_{n+1}(f)(i_n(x)),$$

where $x \in D_n$ and $f \in D_{n+1}$. Thus in passing to the limit space D_∞ something of functional application must also be preserved. The precise result shows that indeed D_∞ becomes its own function space.

4.4 Theorem. *The inverse limit D_∞ of the recursively defined sequence $\langle D_n, j_n\rangle_{n=0}^{\infty}$ of function spaces is not only a continuous lattice, but it is also homeomorphic to its own function space $[D_\infty \to D_\infty]$.*

Proof: We can write down directly the pair of maps i_∞, j_∞ that provide the homeomorphism:

$$i_\infty(x) = \bigsqcup_{n=0}^{\infty} (i_{n\infty} \circ x_{n+1} \circ j_{\infty n}),$$

$$j_\infty(f) = \bigsqcup_{n=0}^{\infty} i_{(n+1)\infty}(j_{\infty n} \circ f \circ i_{n\infty}).$$

Note that these formulae are simply generalizations at the limit for the formulae we used to define i_n, j_n in the first place. Thus it is not surprising that they would provide a projection of $[D_\infty \to D_\infty]$ upon D_∞. Indeed we can compute out $j_\infty(i_\infty(x))$, noting that all the lubs are monotone and that a double monotone limit can always be replaced by a single one in view of the continuity of the operations involved, obtaining

$$j_\infty(i_\infty(x)) = \bigsqcup_{n=0}^{\infty} i_{(n+1)\infty}(j_{\infty n} \circ i_{n\infty} \circ x_{n+1} \circ j_{\infty n} \circ i_{n\infty}) \quad = \bigsqcup_{n=0}^{\infty} i_{(n+1)\infty}(x_{n+1}) = x.$$

In the converse order the calculation is only a bit more complicated. The idea is that since all the functions f are continuous and since the elements x are the limits of their approximations, then each f is actually completely determined by its

In the converse order the calculation is only a bit more complicated. The idea is that since all the functions f are continuous and since the elements x are the limits of their approximations, then each f is actually completely determined by its sequence of restrictions $j_{\infty n} \circ f \circ i_{n\infty} \in D_{n+1}$. This simple idea can be made more precise with the aid of a lemma about D_∞, which allows us in certain cases to recognize projections from limits.

Proof of 4.4 concluded: The lemma applies at once to our calculation, for we find:

$$i_\infty(j_\infty(f)) = \bigsqcup_{n=0}^{\infty}(i_{n\infty} \circ j_{\infty n} \circ f \circ i_{n\infty} \circ j_{\infty n}) \quad = \bigsqcup_{n=0}^{\infty}(i_{n\infty} \circ j_{\infty n}) \circ f \circ \bigsqcup_{n=0}^{\infty}(i_{n\infty} \circ j_{\infty n}).$$

Here we have just applied the continuity of f to be able to confine the lub on the right. But now by the remark following 4.2, we note the functional equation:

$$\mathrm{id} = \bigsqcup_{n=0}^{\infty}(i_{n\infty} \circ j_{\infty n}),$$

and the proof that i_∞ and j_∞ are inverse to one another is complete. □

Lemma 4.5 **4.5 Lemma.** Suppose for each n we have $u_{(n+1)} \in D_{n+1}$ and we let:

$$u = \bigsqcup_{n=0}^{\infty} i_{(n+1)\infty}(u_{(n+1)}).$$

Then if

$$j_{n+1}(u_{(n+2)}) = u_{(n+1)}$$

for each n, we can conclude that:

$$j_{\infty(n+1)}(u) = u_{(n+1)}.$$

Proof: If the sequence $u_{(n+1)}$ satisfies the recursion, then the limit defining u is monotonic. Therefore by continuity of projection it suffices to prove that

$$j_{\infty(n+1)}(i_{(m+1)\infty}(u_{(m+1)})) = u_{(n+1)}$$

for all $m \geqslant n$. This is obvious for $m = n$, and it can be readily proved by induction for larger m using the various recursion equations. (Properly speaking the induction is done on the quantity $(m - n)$ using both n and m as variables.) □

lemma_4_5.lean

```
module

public import DomainTheory.ContinuousLattice.InverseLimits.embInf
public import DomainTheory.ContinuousLattice.InverseLimits.embInfFun
```

```
public import DomainTheory.ContinuousLattice.InverseLimits.iComp
public import DomainTheory.ContinuousLattice.InverseLimits.projInf
public import DomainTheory.ContinuousLattice.InverseLimits.projLE
public import DomainTheory.ContinuousLattice.FunctionSpaces.preservesDirectedSup_coe
public import Mathlib.Data.Set.Basic

@[expose] public section

namespace DomainTheory.ContinuousLattice

open Set

universe u

section InverseLimit

variable (D : Nat  ->  Type u) [forall  n, CompleteLattice (D n)]
variable (P : forall  n, IsContinuousLatticeProjection (D n) (D (n + 1)))

/-- **Scott 1972, Lemma 4.5.** A criterion for recognizing projections out of the
    limit: if a
sequence `u : forall  n, D_{n+1}` satisfies the (shifted) recursion `j_{n+1}(u_{n+2}) =
     u_{n+1}`, then the
monotone limit `u_ inf  =  bigcup _n i_{(n+1) inf }(u_n)` has `j_{ inf (n+1)}(u_ inf )
    = u_n`.

The proof follows Scott's argument at source lines 1316-1320.  First the defining
    family is
monotone.  Continuity of `j_{ inf (n+1)}` therefore moves the projection through its
    directed
supremum.  Finally, for every `m  >=  n`, the identity
`j_{ inf (n+1)}(i_{(m+1) inf }(u_m)) = u_n` is proved by induction on `m - n`, peeling
    one projection with
`projLE.succ` and then applying the recursion hypothesis `j_{m+1}(u_{m+1}) = u_m`. -/
theorem lemma_4_5 (u : forall  n, D (n + 1))
    (hu : forall  n, (P (n + 1)).retr (u (n + 1)) = u n) (n : Nat) :
    ( bigcup  k, embInf D P (k + 1) (u k) : InverseLimit D P).1 (n + 1) = u n := by
  let a : Nat  ->  InverseLimit D P := fun k => embInf D P (k + 1) (u k)
  have ha_succ (k : Nat) : a k  <=  a (k + 1) := by
    rw [show a k = embInf D P (k + 1) (u k) from rfl,
      show a (k + 1) = embInf D P (k + 2) (u (k + 1)) from rfl,
       <-  embInf_succ D P (k + 1) (u k)]
    exact embInf_monotone D P (k + 2)
      (by calc (P (k + 1)).incl (u k)
            = (P (k + 1)).incl ((P (k + 1)).retr (u (k + 1))) := by rw [hu k]
          _  <=  u (k + 1) := (P (k + 1)).incl_retr_le _)
  have ha : Monotone a := monotone_nat_of_le_succ ha_succ
  have hprojLE : forall  d n : Nat,
      projLE D P (show n + 1  <=  n + d + 1 by omega) (u (n + d)) = u n := by
    intro d
    induction d with
    | zero =>
        intro n
        simpa only [Nat.add_zero] using
          (projLE.self D P (le_refl (n + 1)) (u n))
    | succ d ih =>
```

```
      intro n
      simp only [Nat.add_succ]
      rw [projLE.succ D P
        (show n + 1  <=  n + d + 1 by omega)
        (show n + 1  <=  (n + d + 1) + 1 by omega)]
      rw [hu (n + d)]
      exact ih n
  have hprojection : forall  d n : Nat,
      (embInf D P (n + d + 1) (u (n + d))).1 (n + 1) = u n := by
    intro d n
    cases d with
    | zero =>
      change iComp D P (n + 1) (u n) (n + 1) = u n
      exact iComp_self D P (n + 1) (u n)
    | succ d =>
      change iComp D P (n + (d + 1) + 1) (u (n + (d + 1))) (n + 1) = u n
      rw [iComp_of_ge D P (by omega)]
      exact hprojLE (d + 1) n
  have hdir : DirectedOn (*  <=  *) (Set.range a) :=
    directedOn_range.2 ha.directed_le
  have hcont := (projInf D P (n + 1)).preservesDirectedSup_coe
    (Set.range a) (Set.range_nonempty a) hdir
  rw [show ( bigcup  k, embInf D P (k + 1) (u k) : InverseLimit D P)
      = sSup (Set.range a) from sSup_range.symm]
  change (projInf D P (n + 1)) (sSup (Set.range a)) = u n
  rw [hcont,  <-  Set.range_comp, sSup_range]
  apply le_antisymm
  * exact iSup_le fun m => by
      by_cases hmn : n  <=  m
      * obtain <d, rfl> := Nat.exists_eq_add_of_le hmn
        exact le_of_eq (hprojection d n)
      * exact le_trans ((ha (Nat.le_of_not_ge hmn)) (n + 1))
          (le_of_eq (hprojection 0 n))
  * exact le_trans (le_of_eq (hprojection 0 n).symm) (le_iSup (fun m => (a m).1 (n +
    1)) n)

end InverseLimit
end DomainTheory.ContinuousLattice
```

The functorial form We can explain the idea of this proof in other terms using a suggestion made to me by F. W. Lawvere. Consider the category of continuous lattices and projections. In that category our D_∞ is, as we have remarked, both a *direct* and an *inverse* limit. Note too that with regard to projections $[D \to D']$ is a *functor*, for we can also define $[j \to j']$ when the maps are projections. In this language our particular inverse system is defined by the recursion:

$$D_{n+1} = [D_n \to D_n] \quad \text{and} \quad j_{n+1} = [j_n \to j_n],$$

where D_0 and j_0 are given in advance. Now the function space construction is continuous in its two arguments, turning an inverse limit on the right into an inverse limit and a direct limit on the *left* also into an *inverse* limit. A repeated limit can be made into a simple limit, so we can write:

$$D_\infty = \varprojlim \langle D_n, j_n \rangle_{n=0}^{\infty} \quad = \varinjlim \langle D_n, i_n \rangle_{n=0}^{\infty}$$

and

$$[D_\infty \to D_\infty] = \varprojlim \langle [D_n \to D_n], [j_n \to j_n] \rangle_{n=0}^{\infty} \quad = \varprojlim \langle D_{n+1}, j_{n+1} \rangle_{n=0}^{\infty} = D_\infty \quad \textit{(up to isomorphism).}$$

A full checking of the details involved would not make the argument appreciably simpler over the more "element-by-element" argument I have presented. In fact, the proofs are actually the same. But thinking of the result in terms of properties of functors does seem to isolate very well the essential idea and to show how simple it is.

One must only add here a note of caution: the proper choice of category must be done with care. Thus it seems to me that the use of projections rather than arbitrary continuous maps is necessary.

Inasmuch as I have not checked all details in this form, I hope what I say is correct.

The element-by-element construction of this isomorphism is Theorem 4.4.

Application Since we have shown $[D_\infty \to D_\infty]$ to be homeomorphic to D_∞, we can begin to regard them as the same. In particular there ought to be some function space structure to transfer to D_∞. This can be done by defining functional application for any elements $x, y \in D_\infty$ by the equation:

$$x(y) = \bigsqcup_{n=0}^{\infty} i_{n\infty}(x_{n+1}(y_n)).$$

Similarly we can define λ-abstraction on continuous expressions:

$$\lambda x.[\ldots x \ldots] = j_\infty(\lambda x : D_\infty.[\ldots x \ldots]),$$

and in this way D_∞ becomes a model for the λ-calculus of Church and Curry. The model-theoretic and proof-theoretic aspects of this result will be explained in another paper (Scott (1972)).

The embeddings $i_{n\infty}$ in the application formula are embInf.

Two projections Suppose we were to start with the least, non-trivial lattice $0 = \top, \bot$ for D_0. Now $D_1 = [0 \to 0]$ has exactly three elements and there are just two ways of defining a projection $j_0 : D_1 \to D_0$. They are illustrated in the figure:

Left projection

```
 top    <- -- j_0 --  [ top , top ]
False   <- -- j_0 --  [False, top ]
False   <- -- j_0 --  [False,False]
```

Right projection

```
 top    <- -- j_0 --  [ top , top ]
 top    <- -- j_0 --  [False, top ]
False   <- -- j_0 --  [False,False]
```

Hence our construction gives two limit spaces D_∞ and $D' * \infty$. Are they the same? No, they are not. It can be shown, for example, that the $\top$ of $D * \infty$ is *isolated* (that is, $\top \ll \top$), while the same is not true of $D' * \infty$. More interestingly, David Park has proved that the fixed-point operator fix mentioned in 3.14 has algebraic properties in $D * \infty$ quite different from those in $D' * \infty$. By algebraic here, we of course have reference to the functional algebra embodied in the application operation $x(y)$ defined on these limit spaces. Note, by the way, that in view of our isomorphism result we can regard fix (or any other similar continuous function for that matter) as an element of $D * \infty$. This makes the "algebra" of D_∞ quite a rich field for study.

Subspaces of a limit The reader will have surely remarked that by virtue of 1.5, every T_0-space X whatsoever can be embedded as a subspace in a D_∞. Besides this all the continuous functions on X (oh, into D_∞, say) can be extended to D_∞; whence they can be regarded as *elements* of D_∞. Thus we have been able to embed not only the topology of X into D_∞ but also all of the continuous function theory over X. So far this is only a "logical" construction. For more interesting "mathematical" results we shall have to investigate whether any useful theorems about the usual function spaces $[X \to X]$ can be obtained with the aid of D_∞. This method can easily be employed for real- or complex-valued continuous functions, though it seems more oriented toward pointwise convergence than anything else. Still, there seems to be a chance it might be useful—especially if one wished to consider continuous *operators* on function spaces.

Other functors The idea of forming the limit space can also be applied to *other* functors besides $[D \to D]$. Thus instead of solving the "equation"

$$D = [D \to D]$$

as we have done with the D_∞ construction, we could also solve:

$$V = T + [V \times V] + [V + V] + [V \to V],$$

for example. Here $T = \bot, 0, 1, \top$ is the four-element lattice with 0 and 1 as incomparable elements. By $[V \times V]$ and $[V \to V]$ we understand the usual Cartesian product and function space construction. The $+$ operator, on the other hand works only in the category of lattices with $\top$ as an isolated element. It is defined so as to make:

$$\begin{array}{ccc} D + D' & \longleftarrow & D' \\ \uparrow & & \uparrow \\ D & \longleftarrow & 0 \end{array}$$

a push-out diagram, where the maps from 0 are meant to match $\bot$ with $\bot$ and $\top$ with $\top$. The point of requiring $\top$ to be isolated is that both D and D' become projections of $D + D'$. This construction, though not quite a disjoint union, has many properties in common with that operation on spaces. In particular, if we consider the category with projections as maps, the construction

$$\mathbb{F}(D) = T + [D \times D] + [D + D] + [D \to D]$$

is a *functor*. Furthermore, we can project $\mathbb{F}(T)$ onto T in an obvious way, thereby setting things up for an inverse limit construction:

$$V = \varprojlim \langle \mathbb{F}^n(T), j_n \rangle_{n=0}^{\infty}.$$

The resulting continuous lattice satisfies the desired equation up to isomorphism.

The space V constructed in the way just indicated is very rich in subspaces. To see this, consider the space $\mathcal{J}_V$ of proper projections j where $j(\top) = \top$. As in 3.12 this is a complete lattice. Now that $[V \times V]$ and $[V + V]$ and $[V \to V]$ are regarded as *subspaces* of this "universe" V itself, we can easily define *continuous operations*

$$(j \times k)\ ,\ (j + k)\ ,\ \text{and}\ (j \to k)$$

on the projections obtaining again elements of $\mathcal{J}_V$. The projections so obtained correspond to the indicated constructions of subspaces, of course. (Indeed, if we had the time and space, we could show that $\mathcal{J}_V$ becomes a very interesting category). There will be a particular

projection t corresponding to T, and reason for doing all this is to show that the existence of subspaces of V can now be established by solving equations in $\mathcal{J}_V$. For example, by the fixed-point construction we could find a $j \in \mathcal{J}_V$ such that

$$j = t + (t \times j) + ((j \times j) \to j).$$

The range of j would then be a subspace $W \subseteq V$ such that W solves the equation:

$$W = T + [T \times W] + [[W \times W] \to W].$$

And these are only a few examples: simultaneous equations are possible, and many other operators are waiting for discovery and application.

(Already included: `1972_Proposition_4_1.md`.)
(Already included: `1972_Proposition_4_2.md`.)
(Already included: `1972_Corollary_4_3.md`.)
(Already included: `1972_Function_space_tower.md`.)
(Already included: `1972_Lemma_4_5.md`.)

2 Datatypes as Lattices (1976)

Dependencies. Each box is one section. The arrowhead is on the section that imports. The number is how many direct imports. An arrow already implied by a longer path is left out.

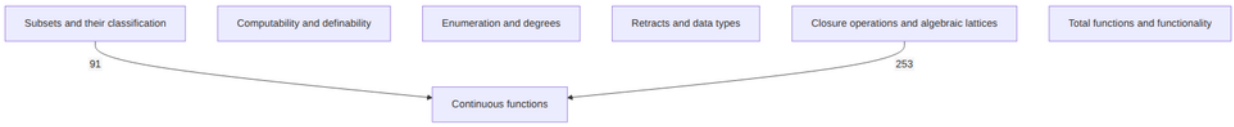


Figure 17: Datatypes as Lattices (1976).

2.1 Introduction

To the Memory of Christopher Strachey, 1916–1975

The meaning of many kinds of expressions in programming languages can be taken as elements of certain spaces of "partial" objects. In this report these spaces are modeled in one universal domain $\mathbf{P}\omega$, the set of all subsets of the integers. This domain renders the connection of this semantic theory with the ordinary theory of number theoretic (especially general recursive) functions clear and straightforward.

Investigations begun in 1969 with Christopher Strachey led to the idea that the denotations of many kinds of expressions in programming languages could be taken as elements of certain kinds of spaces of "partial" objects. As these spaces could be treated as function spaces, their structure at first seemed excessively complicated—even impossible. But then the author discovered that there were many more spaces than we had first imagined—even wanted. They could be presented as lattices (or as some prefer, semilattices), and the main technique was to employ topological ideas, in particular the notion of a continuous function. This approach and its applications have been presented in a number of publications, but that part of the foundation concerned with *computability* (in the sense of recursion theory) was never before adequately exposed. The purpose of the present paper is to provide such a foundation and to simplify the whole presentation by a de-emphasis of abstract ideas.

The main innovation in this report is to model everything within one "universal" domain $\mathbf{P}\omega = \{x \mid x \subseteq \omega\}$, the domain of all subsets of the set ω of nonnegative integers. The advantages are many: by the most elementary considerations $\mathbf{P}\omega$ is recognized to be a lattice and a topological space. In fact, $\mathbf{P}\omega$ is a *continuous lattice*, even an *algebraic lattice*, but in the beginning we do not even need to define such an "advanced" concept; we can save these ideas for an *analysis* of what has been done here in a more direct way. Next by taking the set of integers as the basis of the construction, the connection with the ordinary theory of number-theoretic (especially, general recursive) functions can be made clear and straightforward.

The model $\mathbf{P}\omega$ can be intuitively viewed as the domain of *multiple-valued* integers; what is new in the presentation is that functions are not only multiple-valued but also "multiple-argumented". This remark is given a precise sense in § 2 below, but the upshot of the idea is that multiple-valued integers are regarded as objects in themselves—possibly infinite—and as something more than just the collection of single integers contained in them. This combination of the finite with the infinite into a single domain, together with the idea that a continuous function can be reduced to its *graph* (in the end, a set of integers), makes it possible to view an $x \in \mathbf{P}\omega$ at one time as a value, at another as an argument, then as an integer, then as a function, and still later as a functional (or combinator). The "paradox" of self-application (as in $x(x)$) is solved by allowing the *same* x to be used in two *different* ways. This is done in ordinary recursion theory via Gödel

numbers (as in $\{e\}(e)$), but the advantage of the present theory is that not only is the function concept the *extensional* one, but it includes *arbitrary* continuous functions and not just the computable ones.

Section 1 introduces the elementary ideas on the topology of $\mathbf{P}\omega$ and the continuous functions including the fixed-point theorem. Section 2 has to do with computability and definability. The language LAMBDA is introduced as an extension of the pure λ-calculus by four arithmetical combinators; in fact, it is indicated in § 3 how the whole system could be based on *one* combinator. What is shown is that computability in $\mathbf{P}\omega$ according to the natural definition (which assumes that we already know what a recursively enumerable set of integers is) is equivalent to LAMBDA-definability. The main tool is, not surprisingly, the first recursion theorem formulated with the aid of the so-called *paradoxical combinator* $\mathbf{Y}$. The plan is hardly original, but the point is to work out what it all means in the model.

Along the way we have to show how to give every λ-term a denotation in $\mathbf{P}\omega$; the resulting principles of λ-calculus that are thereby verified are summarized in Table 1. Of these the first three, (α), (β), and (ξ), are indeed valid in the model; however, rule (η), which is a stronger version of extensionality, *fails* in the $\mathbf{P}\omega$ model. This should not be regarded as a disadvantage since the import of (η) is to suppose every object is a function. A quick construction of these special models is indicated at the end of § 5. Since $\mathbf{P}\omega$ is partially ordered by $\subseteq$, there are also laws involving this relation. Law (ξ^*) is an improvement of (ξ); while (μ) is a form of monotonicity for application.

TABLE 1

Some laws of λ-calculus

(α)	$\lambda x.\tau = \lambda y.\tau[y/x]$
(β)	$(\lambda x.\tau)(y) = \tau[y/x]$
(ξ)	$\lambda x.\tau = \lambda x.\sigma \quad \text{if} \quad \forall x.\, \tau = \sigma$
(η)	$y = \lambda x.y(x)$
(ξ^*)	$\lambda x.\tau \subseteq \lambda x.\sigma \quad \text{if} \quad \forall x.\, \tau \subseteq \sigma$
(μ)	$x \subseteq y$ and $u \subseteq v$ imply $u(x) \subseteq v(y)$

Section 3 has to do with enumeration and degrees. Gödel numbers for LAMBDA are defined in a very easy way which takes advantage of the notation of combinators. This leads to the second recursion theorem, and results on incompleteness and undecidability follow along standard lines. *Relative recursiveness* is also very easy to define in the system, and we make the tie-in with *enumeration degrees* which correspond to finitely generated combinatory subalgebras of $\mathbf{P}\omega$. Finally a theorem of Myhill and Shepherdson is interpreted as a most satisfactory completeness property for definability in the system.

Sections 4 and 5 show how a calculus of *retracts* leads to quite simple definitions of a host of useful domains (as lattices). Section 6 investigates the classification of other subsets (*nonlattices*) of $\mathbf{P}\omega$; while § 7 contrasts partial (multiple-valued) functions with total functions, and interprets various theories of functionality. Connections with category theory are mentioned.

What is demonstrated in this work is how the language LAMBDA, together with its interpretation in $\mathbf{P}\omega$, is an extremely convenient vehicle for *definitions* of computable functions on complex structures (all taken as subdomains of $\mathbf{P}\omega$). It is a "high-level" programming language for recursion theory. It is *applied* combinatory logic, which in usefulness goes far beyond anything envisioned in the standard literature. What has been shown is how many interesting predicates can be *expressed* as equations between continuous functions. What is needed next is a development of the *proof theory* of the system along the lines of the work of Milner, which incorporates the author's extension of McCarthy's rule of recursion induction to this high-level language. Then we will have a flexible and practical "mathematical" theory of computation.

2.1.1 Historical note

The paper comes out of work with Christopher Strachey begun in 1969. He treated the λ-calculus as ordinary algebra. Scott's first answer was a typed theory of function spaces, later developed as LCF by Robin Milner and his collaborators. Those function spaces, which he had met in the recursion theory of Nerode and Platek, led to the inverse-limit construction of function spaces of infinite type: models of the type-free λ-calculus, written up in Scott (1971) and Scott (1973).

Plotkin (1972) had proposed a set-theoretic model. This paper is the form of that idea in which the connection with ordinary recursion theory is immediate, because the elements are sets of integers. Scott presented it at Oberwolfach at Easter 1973, and in a shorter version at the Third Scandinavian Logic Symposium (Scott (1975)). Lectures at Kiel in July 1974 covered most of the paper; the text, without the acknowledgments or the references, appeared in Springer Lecture Notes in Mathematics, volume 499. Once the definitions had settled, the model was Plotkin's, and it was already present in the enumeration operators of Myhill and Shepherdson (1955) and Friedberg and Rogers (1959).

At the end of §2 of *Continuous lattices* (Scott (1972)) he had written that representing a continuous-function space as a computable retract "does not seem to be of too much help in proving theorems." That representation is the construction of this paper. Attributions for the separate sections are recorded on those sections.

2.2 Continuous functions

The constructions below are the enumeration of finite subsets, the pairing of natural numbers, the positive topology, the graph of a continuous function, the least fixed point, and the embedding of a continuous function.

Dependency diagram. Each box is one construction. The arrowhead is on the note that imports. The number is how many direct imports. An arrow already implied by a longer path is left out.

2.2.1 Introduction

The domain $\mathbf{P}\omega$ of all subsets of the set ω of nonnegative integers is a complete lattice under the partial ordering $\subseteq$ of set inclusion, as is well known. We use the usual symbols $\cup$, $\cap$, $\bigcup$, $\bigcap$ for the finite and infinite lattice operations of union and intersection. $\mathbf{P}\omega$ is of course also a Boolean algebra; and for complements we write

$$\sim x = \{n \mid n \notin x\}$$

where it is understood that such variables as i, j, k, l, m, n range over *integers in* ω, while u, v, w, x, y, z range over *subsets of* ω.

The domain $\mathbf{P}\omega$ can also be made into a topological space—in many ways. A common method is to match each $x \subseteq \omega$ with the corresponding characteristic function in $\{0, 1\}^\omega$, and to take the induced product topology. In this way $\mathbf{P}\omega$ is a totally disconnected compact Hausdorff space homeomorphic to the Cantor "middle-third" set. This is *not* the topology we want; it is a *positive-and-negative* topology which makes the function $\sim x$ continuous. We want a weaker topology: the topology of positive "information", which has the advantage that all continuous functions possess fixed points. (The equation $x = \sim x$ is impossible.) The topology that we *do* want is exactly that appropriate to considering $\mathbf{P}\omega$ to be a continuous lattice. But all this terminology of abstract mathematics is quite unnecessary, since the required definitions can be given in very elementary terms.

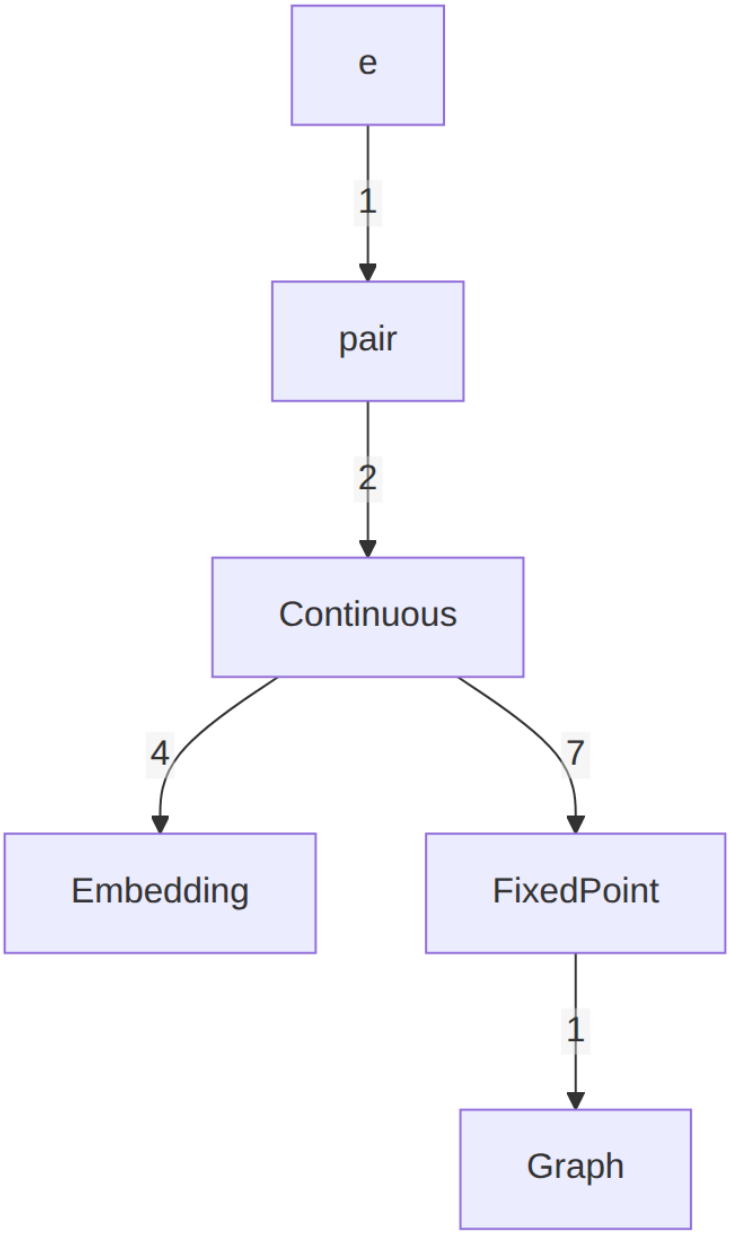


Figure 18: Continuous functions.

To make the topology "visible", we introduce a standard enumeration $\{e_n \mid n \in \omega\}$ of all *finite* subsets of ω. Specifically we set

$$e_n = \{k_0, k_1, \cdots, k_{m-1}\},$$

provided that $k_0 < k_1 < \cdots < k_{m-1}$ and $n = \sum_{i<m} 2^{k_i}$. Thus n is the code number for e_n, and the elements of e_n are the exponents in the binary expansion of the integer n. This is a one-to-one enumeration of finite subsets, where $k \in e_n$ always implies $k < n$, the function $\max(e_n)$ is (primitive) recursive in n, and the relations $k \in e_n$, $e_n \subseteq e_m$, $e_n = e_m \cup e_k$ are all (primitive) recursive in k, m, n.

Topologically speaking the finite sets e_n are *dense* in the space $\mathbf{P}\omega$, for each $x \in \mathbf{P}\omega$ is the "limit" of its finite subsets in the sense that

$$x = \bigcup \{e_n \mid e_n \subseteq x\}.$$

To make this precise we need a rigorous definition of *open subset* of $\mathbf{P}\omega$.

Motivation Give $\{\bot, \top\}$ the weak T_0-topology with open sets $\varnothing$, $\{\top\}$, and $\{\bot, \top\}$. That is the Sierpinski space, and the product $\{\bot, \top\}^\omega$ in the product topology is $\mathbf{P}\omega$. The finite sets e_n are the basic open sets of the product. Theorem 1.1 is that observation written out in elementary terms.

On a subspace $X \subseteq \mathbf{P}\omega$ a finite set e_n need not lie in X, so continuity of $f : X \to \mathbf{P}\omega$ cannot be written by applying f to e_n. Theorem 1.5 is the substitute. Such an f is continuous exactly when it is the restriction of its extension $\bar{f}$, and for $x \in X$ that reads

$$m \in f(x) \quad \text{iff} \quad \exists e_n \subseteq x \; \forall x' \in X \; [e_n \subseteq x' \text{ implies } m \in f(x')].$$

The extension $\bar{f} : \mathbf{P}\omega \to \mathbf{P}\omega$ is continuous for an arbitrary f. It extends f precisely when f is continuous.

The fixed-point formula uses the same fact in the form Scott recorded for an increasing sequence: if $x_n \subseteq x_{n+1}$ and f is continuous, then f preserves the union of the x_n. Every fixed point lies above every iterate of $\varnothing$, so the union is the least one.

Historical note The positive topology came to Scott's attention through Nerode (1959). Continuous functionals had been studied by Kleene, Platek, and others, and Platek (1964) is where monotonicity is stressed. Graphs and application are used in the same way by Plotkin (1972) and by Rogers (1967, p. 147). The fixed-point theorem is Tarski's (1955). The extension theorem is Scott's, in the spirit of point-set topology, and it is the characterization of continuous lattices in Scott (1972). The embedding theorem is the standard one; Čech (1966) is a reference.

2.2.2 e

The powerset of the natural numbers, and the enumeration of its finite subsets by the binary code `e`. The declarations named `e_` are the facts about that code. The others are the domain, the membership tests, and the coding of finite lists and finite sets that those facts use.

`e.lean`

```
module

public import Mathlib.Data.Set.Finite.Basic
public import Mathlib.Algebra.GroupWithZero.Nat
public import Mathlib.Order.SetNotation

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, S1.** `P? = {x | x  subseteq  ?}`. -/
abbrev Pomega := Set Nat

/-- **Scott 1976, after (2.2).** `False = {}`. -/
def botElem : Pomega := {}

/-- **Scott 1976, after (2.2).** ` top  = ?`. -/
def topElem : Pomega := Set.univ

@[simp] theorem mem_botElem (n : Nat) : n  in  botElem  <->  False := by simp [botElem
    ]

@[simp] theorem mem_topElem (n : Nat) : n  in  topElem := Set.mem_univ n

/-- **Scott 1976, S1.** Finite set coded by the binary expansion of `n`. -/
def e (n : Nat) : Pomega := {k | n.testBit k}

@[simp] theorem mem_e {n k : Nat} : k  in  e n  <->  n.testBit k := Iff.rfl

theorem le_two_pow (k : Nat) : k  <=  2 ^ k := by
  induction k with
  | zero => decide
  | succ k ih =>
    calc
      k + 1  <=  2 ^ k + 1 := Nat.succ_le_succ ih
      _  <=  2 ^ k + 2 ^ k := Nat.add_le_add_left Nat.one_le_two_pow _
      _ = 2 ^ k * 2 := (Nat.mul_two (2 ^ k)).symm
      _ = 2 ^ (k + 1) := (Nat.pow_succ 2 k).symm

theorem two_pow_le_of_testBit {n k : Nat} (h : n.testBit k) : 2 ^ k  <=  n := by
  have hmod : (n >>> k) % 2 = 1 := by simpa [Nat.testBit] using h
  have hpos : 0 < n >>> k := Nat.pos_of_ne_zero fun hz => by simp [hz] at hmod
  have hdiv : 1  <=  n / 2 ^ k := by
    rwa [Nat.shiftRight_eq_div_pow] at hpos
  simpa using (Nat.le_div_iff_mul_le (Nat.two_pow_pos k)).mp hdiv

theorem mem_e_le {n k : Nat} (h : k  in  e n) : k  <=  n :=
  (le_two_pow k).trans (two_pow_le_of_testBit h)

instance (n k : Nat) : Decidable (k  in  e n) :=
  inferInstanceAs (Decidable (n.testBit k))

instance (n m : Nat) : Decidable (e n  subseteq  e m) :=
  decidable_of_iff (forall  k  <=  n, k  in  e n  ->  k  in  e m) <
    fun h k hk => h k (mem_e_le hk) hk,
    fun h _k _ hk => h hk>
```

```
/-- Bit-mask code of a finite list of naturals. -/
def codeList : List Nat  ->  Nat
  | [] => 0
  | k :: ks => 2 ^ k ||| codeList ks

noncomputable def codeFinset (s : Finset Nat) : Nat := codeList s.val.toList

/-- Finite pieces of `x` used in directed unions. -/
def eBelow (x : Pomega) (n : Nat) : Pomega :=
  {k | e n  subseteq  x  /\  k  in  e n}

theorem e_zero : e 0 = {} := by
  ext k
  simp [e]

theorem e_pow2 (k : Nat) : e (2 ^ k) = {k} := by
  ext i
  simp [e, Nat.testBit_two_pow]

theorem e_finite (n : Nat) : (e n).Finite :=
  (Finset.range (n + 1)).finite_toSet.subset fun _k hk =>
    Finset.mem_range.mpr (Nat.lt_succ_of_le (mem_e_le hk))

theorem e_or (n m : Nat) : e (n ||| m) = e n  U  e m := by
  ext k
  simp [e, Nat.testBit_or]

theorem e_or_of_subset {n m : Nat} {x : Pomega} (hn : e n  subseteq  x) (hm : e m
    subseteq  x) :
    e (n ||| m)  subseteq  x := by
  rw [e_or]
  exact Set.union_subset hn hm

theorem lt_two_pow' (k : Nat) : k < 2 ^ k := by
  induction k with
  | zero => decide
  | succ k ih =>
    calc
      k + 1  <=  2 ^ k := Nat.succ_le_of_lt ih
      _ < 2 ^ k + 2 ^ k := Nat.lt_add_of_pos_right (Nat.two_pow_pos k)
      _ = 2 ^ (k + 1) := by rw [ <-  Nat.two_mul, Nat.mul_comm, Nat.pow_succ]

/-- **Scott 1976, S1.** `k  in  e_n` implies `k < n`. -/
theorem mem_e_lt {n k : Nat} (h : k  in  e n) : k < n := by
  have hle := mem_e_le h
  rcases eq_or_lt_of_le hle with rfl | hlt
  * exact (lt_irrefl k ((lt_two_pow' k).trans_le (two_pow_le_of_testBit h))).elim
  * exact hlt

theorem singleton_eq_e (k : Nat) : ({k} : Pomega) = e (2 ^ k) := (e_pow2 k).symm

theorem exists_e_mem {x : Pomega} {k : Nat} (hk : k  in  x) :
    exists  n, e n  subseteq  x  /\  k  in  e n :=
  <2 ^ k, by simp [e_pow2, Set.singleton_subset_iff, hk], by simp [e_pow2]>
```

```
theorem union_e_eq (x : Pomega) : (? n, eBelow x n) = x := by
  ext k
  constructor
  * intro hk
    obtain <n, hn, hke> := Set.mem_iUnion.mp hk
    exact hn hke
  * intro hk
    refine Set.mem_iUnion.mpr <2 ^ k, ?_>
    simp [eBelow, e_pow2, Set.singleton_subset_iff, hk]

theorem e_codeList (ks : List Nat) : e (codeList ks) = {k | k  in  ks} := by
  induction ks with
  | nil => simp [codeList, e_zero]
  | cons k ks ih =>
    ext i
    simp [codeList, e_or, e_pow2, ih]

theorem e_codeFinset (s : Finset Nat) : e (codeFinset s) = (s : Set Nat) := by
  ext k
  simp [codeFinset, e_codeList]

theorem exists_code (s : Finset Nat) : exists  n, e n = (s : Set Nat) :=
  <codeFinset s, e_codeFinset s>

theorem exists_code_of_finite {s : Pomega} (hs : s.Finite) :
    exists  n, e n = s := by
  obtain <t, ht> := hs.exists_finset_coe
  exact <codeFinset t, ht  |>  e_codeFinset t>

end DomainTheory.DataTypesAsLattices
```

2.2.3 pair

The Cantor pairing of two natural numbers into one, its inverse, and the triangular-number facts that show the pairing is injective. This imports the enumeration.

pair.lean

```
module

public import Mathlib.Data.Nat.Bitwise
public import Mathlib.Data.Set.Finite.Basic
public import Mathlib.Data.Set.Lattice.Bounded
public import Mathlib.Data.Set.Lattice.Disjoint
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, S1.** Diagonal pairing `(n, m) = ?(n+m)(n+m+1) + m`. -/
def pair (n m : Nat) : Nat := (n + m) * (n + m + 1) / 2 + m

def triangle (w : Nat) : Nat := w * (w + 1) / 2
```

```
theorem pair_eq_triangle (n m : Nat) : pair n m = triangle (n + m) + m := rfl

theorem triangle_succ (w : Nat) : triangle (w + 1) = triangle w + (w + 1) := by
  have hex : (w + 1) * (w + 2) = w * (w + 1) + 2 * (w + 1) := by
    calc
      (w + 1) * (w + 2) = (w + 1) * w + (w + 1) * 2 := Nat.mul_add _ _ _
      _ = w * (w + 1) + 2 * (w + 1) := by
        rw [Nat.mul_comm (w + 1) w, Nat.mul_comm (w + 1) 2]
  simp only [triangle]
  rw [show (w + 1) * ((w + 1) + 1) = (w + 1) * (w + 2) from rfl, hex]
  exact Nat.add_mul_div_left _ _ (by decide : 0 < 2)

theorem exists_triangle_bucket (k : Nat) :
    exists  w, triangle w  <=  k  /\  k < triangle (w + 1) := by
  let P := fun w : Nat => k < triangle (w + 1)
  have hP : exists  w, P w := <k, by
    change k < triangle (k + 1)
    rw [triangle_succ]
    exact lt_of_lt_of_le (Nat.lt_succ_self k) (Nat.le_add_left (k + 1) _)>
  let w := Nat.find hP
  refine <w, ?_, Nat.find_spec hP>
  cases hw : w with
  | zero => exact Nat.zero_le k
  | succ w' =>
    have hmin : not  P w' :=
      Nat.find_min hP (Nat.lt_of_succ_le (by
        have : w = Nat.find hP := rfl
        rw [ <-  this, hw]))
    exact Nat.le_of_not_gt hmin

theorem exists_pair (k : Nat) : exists  n m, pair n m = k := by
  obtain <w, hle, hlt> := exists_triangle_bucket k
  refine <w - (k - triangle w), k - triangle w, ?_>
  have hlt' : k < triangle w + w + 1 := by
    have : k < triangle w + (w + 1) := by rwa [triangle_succ] at hlt
    exact Nat.add_assoc (triangle w) w 1  |>  this
  have hk : k  <=  triangle w + w := Nat.lt_succ_iff.mp hlt'
  have hm : k - triangle w  <=  w :=
    Nat.le_of_add_le_add_left ((Nat.add_sub_of_le hle).symm  |>  hk)
  have : w - (k - triangle w) + (k - triangle w) = w := Nat.sub_add_cancel hm
  rw [pair_eq_triangle, this, Nat.add_sub_of_le hle]

/-- Inverse of Cantor pairing. -/
noncomputable def unpair (k : Nat) : Nat  x  Nat :=
  ((exists_pair k).choose, (exists_pair k).choose_spec.choose)

theorem pair_le_left (n m : Nat) : n  <=  pair n m := by
  have htri : n  <=  triangle (n + m) := by
    cases n with
    | zero => exact Nat.zero_le _
    | succ n =>
      have hmul : n.succ * 2  <=  (n.succ + m) * (n.succ + m + 1) :=
        Nat.mul_le_mul
          (Nat.le_add_right n.succ m)
          (Nat.succ_le_succ
            (Nat.le_trans (Nat.succ_le_succ (Nat.zero_le n)) (Nat.le_add_right n.succ
```

```
    m)))
      exact (Nat.le_div_iff_mul_le (by decide : 0 < 2)).mpr hmul
  exact htri.trans (Nat.le_add_right _ _)

theorem pair_lt_left (n m : Nat) : n < pair n m + 1 :=
  Nat.lt_succ_of_le (pair_le_left n m)

theorem pair_le_right (n m : Nat) : m  <=  pair n m :=
  Nat.le_add_left _ _

theorem pair_lt_right (n m : Nat) : m < pair n m + 1 :=
  Nat.lt_succ_of_le (pair_le_right n m)

theorem pair_unpair (k : Nat) : pair (unpair k).1 (unpair k).2 = k :=
  (exists_pair k).choose_spec.choose_spec

theorem triangle_le_of_le {a b : Nat} (h : a  <=  b) : triangle a  <=  triangle b :=
    by
  induction h with
  | refl => exact le_rfl
  | step h ih =>
    rw [triangle_succ]
    exact le_trans ih (Nat.le_add_right _ _)

theorem triangle_le_pair (n m : Nat) :
    triangle (n + m)  <=  pair n m  /\  pair n m < triangle (n + m + 1) := by
  constructor
  * exact Nat.le_add_right _ _
  * rw [triangle_succ]
    exact Nat.add_lt_add_left (Nat.lt_succ_of_le (Nat.le_add_left m n)) _

theorem triangle_lt_of_lt {a b : Nat} (h : a < b) : triangle a < triangle b := by
  have : a + 1  <=  b := Nat.succ_le_of_lt h
  calc
    triangle a < triangle a + (a + 1) := Nat.lt_add_of_pos_right (Nat.succ_pos _)
    _ = triangle (a + 1) := (triangle_succ a).symm
    _  <=  triangle b := triangle_le_of_le this

theorem pair_inj {n_1 m_1 n_2 m_2 : Nat} (h : pair n_1 m_1 = pair n_2 m_2) :
    n_1 = n_2  /\  m_1 = m_2 := by
  have h1 := triangle_le_pair n_1 m_1
  have h2 := triangle_le_pair n_2 m_2
  have hw : n_1 + m_1 = n_2 + m_2 := by
    rcases lt_trichotomy (n_1 + m_1) (n_2 + m_2) with hlt | heq | hgt
    * have hle : triangle (n_1 + m_1 + 1)  <=  triangle (n_2 + m_2) :=
        triangle_le_of_le (Nat.succ_le_of_lt hlt)
      have : pair n_1 m_1 < pair n_1 m_1 :=
        calc
          pair n_1 m_1 < triangle (n_1 + m_1 + 1) := h1.2
          _  <=  triangle (n_2 + m_2) := hle
          _  <=  pair n_2 m_2 := h2.1
          _ = pair n_1 m_1 := h.symm
      exact (lt_irrefl _ this).elim
    * exact heq
    * have hle : triangle (n_2 + m_2 + 1)  <=  triangle (n_1 + m_1) :=
        triangle_le_of_le (Nat.succ_le_of_lt hgt)
```

```
      have : pair n_2 m_2 < pair n_2 m_2 :=
        calc
          pair n_2 m_2 < triangle (n_2 + m_2 + 1) := h2.2
          _  <=  triangle (n_1 + m_1) := hle
          _  <=  pair n_1 m_1 := h1.1
          _ = pair n_2 m_2 := h
      exact (lt_irrefl _ this).elim
  have hm : m_1 = m_2 := by
    have : triangle (n_1 + m_1) + m_1 = triangle (n_2 + m_2) + m_2 := h
    rw [hw] at this
    exact Nat.add_left_cancel this
  have hn : n_1 = n_2 := Nat.add_right_cancel (hw.trans (by rw [hm]))
  exact <hn, hm>

theorem unpair_pair (n m : Nat) : unpair (pair n m) = (n, m) := by
  have h := pair_unpair (pair n m)
  exact Prod.ext (pair_inj h).1 (pair_inj h).2

end DomainTheory.DataTypesAsLattices
```

2.2.4 Continuous

Dependencies. Inside Continuous, each box is one Lean file. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. The arrowhead points at the file or folder that imports, and the number is how many direct imports.

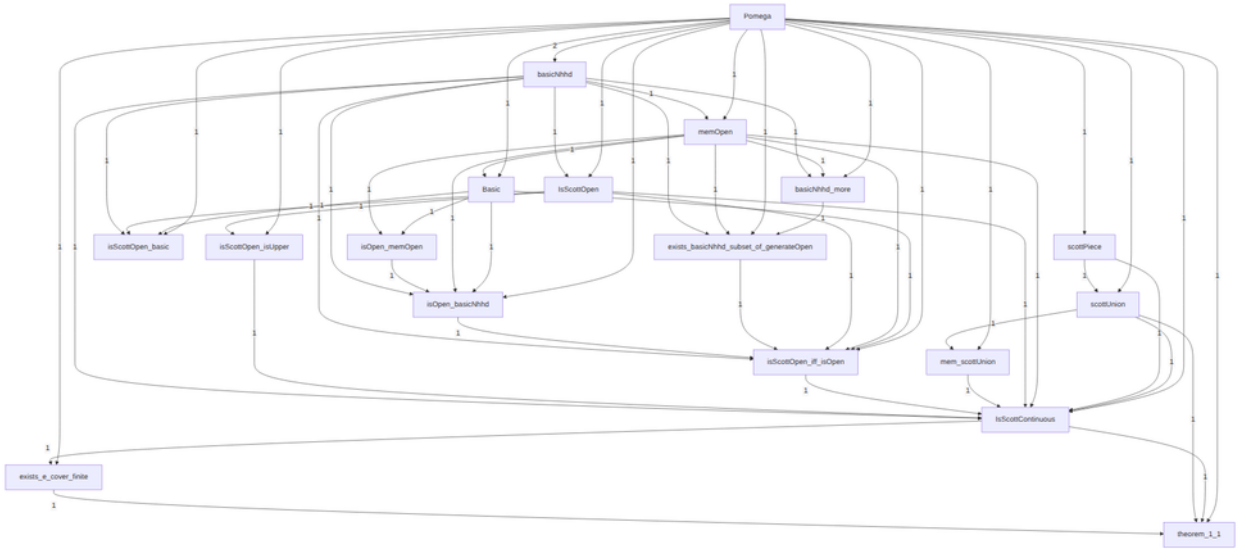


Figure 19: Continuous.

basicNhhd.lean

```
module

public import Mathlib.Topology.Bases
```

```
public import Mathlib.Topology.Order
public import Mathlib.Data.Nat.Bitwise
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Pomega.pair

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

/-- **Scott 1976, S1, Definition.** A basic neighbourhood is
`{x  in  P? | e n  subseteq  x}` for some finite code `n`. -/
def basicNhhd (n : Nat) : Set Pomega := {x | e n  subseteq  x}

namespace basicNhhd

theorem inter (n m : Nat) :
    basicNhhd n  I  basicNhhd m = basicNhhd (n ||| m) := by
  ext x
  simp [basicNhhd, e_or, Set.union_subset_iff]

theorem zero : basicNhhd 0 = Set.univ := by
  ext x
  simp [basicNhhd, e_zero]

theorem mono {n m : Nat} (h : e n  subseteq  e m) : basicNhhd m  subseteq  basicNhhd n
    := by
  intro x hx
  exact h.trans hx

end basicNhhd

end DomainTheory.DataTypesAsLattices
```

Definition (basis) **DEFINITION.** A *basis* for the neighborhoods of $\mathbf{P}\omega$ consists of those sets of the form:

$$\{x \in \mathbf{P}\omega \mid e_n \subseteq x\},$$

for a given e_n. An arbitrary *open subset* is then a union of basic neighborhoods.

It is easy to prove that an open subset $U \subseteq \mathbf{P}\omega$ is just a set of "finite character"; that is, a set such that for all $x \in \mathbf{P}\omega$ we have $x \in U$ if and only if some finite subset of x also belongs to U. An alternate approach would define directly what we mean by a continuous function using the idea that such functions must preserve limits.

Definition (continuous) **DEFINITION.** A function $f : \mathbf{P}\omega \to \mathbf{P}\omega$ is *continuous* iff for all $x \in \mathbf{P}\omega$ we have:

$$f(x) = \bigcup\{f(e_n) \mid e_n \subseteq x\}.$$

Again it is an easy exercise to prove that a function is continuous in the sense of this definition iff it is continuous in the usual topological sense (namely: inverse images of open sets are open). For giving proofs

it is even more convenient to have the usual ε-δ formulation of continuity.

Theorem 1.1 **THEOREM 1.1** (The characterization theorem). A function $f : \mathbf{P}\omega \to \mathbf{P}\omega$ is *continuous* iff for all $x \in \mathbf{P}\omega$ and all e_m we have:

$$e_m \subseteq f(x) \quad \text{iff} \quad \exists e_n \subseteq x.\ e_m \subseteq f(e_n).$$

Note that open sets and continuous functions have a *monotonicity property*:
whenever $x \subseteq y$ *and* $x \in U$, *then* $y \in U$; *and*
whenever $x \subseteq y$, *then* $f(x) \subseteq f(y)$.

This gives a precise expression to the "positive" character of our topology. However, note too that openness and continuity mean rather more than just monotonicity. In particular, a continuous function is completely determined by the pairs of integers n, m such that $m \in f(e_n)$, as can be seen from the definition. (Hence, there are only a continuum number of continuous functions, but more than a continuum number of monotonic functions.) This brings us to the definition of the *graph* of a continuous function.

To formulate the definition, we introduce a standard enumeration (n, m) of all pairs of integers. Specifically we set

$$(n, m) = \tfrac{1}{2}(n + m)(n + m + 1) + m.$$

This is the enumeration along the "little diagonals" going from left to right, and it produces the ordering:

$$(0,0), (1,0), (0,1), (2,0), (1,1), (0,2), (3,0), (2,1), \ldots.$$

Note that $n \leq (n, m)$ and $m \leq (n, m)$ with equality possible only in the cases of $(0, 0)$ and $(1, 0)$. This is a one-to-one enumeration, and the inverse functions are (primitive) recursive—but we do not require at the present any notation for them.

theorem_1_1.lean

```
module

public import Mathlib.Topology.Bases
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Continuous.exists_e_cover_finite
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Continuous.scottUnion

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

/-- **Scott 1976, Theorem 1.1 (The characterization theorem).**
`f` is continuous iff `e m  subseteq  f(x)` just when some finite `e n  subseteq  x`
    already
satisfies `e m  subseteq  f(e n)`. -/
theorem theorem_1_1 (f : Pomega  ->  Pomega) :
    IsScottContinuous f  <->
      forall  x m, e m  subseteq  f x  <->  exists  n, e n  subseteq  x  /\  e m
    subseteq  f (e n) := by
```

```
  constructor
  * intro hf x m
    constructor
    * intro hmf
      have : forall  k  in  e m, exists  n, e n  subseteq  x  /\  k  in  f (e n) := by
        intro k hk
        have : k  in  scottUnion f x := by
          rw [ <-  hf.out x]; exact hmf hk
        exact mem_scottUnion.mp this
      obtain <s, hs> := (e_finite m).exists_finset_coe
      obtain <N, hN, hsf> := exists_e_cover_finite hf s (fun k hk =>
        this k (by
          have : k  in  (s : Set Nat) := hk
          rwa [hs] at this))
      exact <N, hN, by simpa [hs] using hsf>
    * intro <n, hn, hmf>
      exact subset_trans hmf (IsScottContinuous.monotone hf hn)
  * intro h
    constructor
    intro x
    ext k
    constructor
    * intro hk
      have : e (2 ^ k)  subseteq  f x := by
        simp [e_pow2, Set.singleton_subset_iff, hk]
      obtain <n, hn, hkn> := (h x (2 ^ k)).mp this
      refine mem_scottUnion.mpr <n, hn, ?_>
      simpa [e_pow2, Set.singleton_subset_iff] using hkn
    * intro hk
      obtain <n, hn, hkn> := mem_scottUnion.mp hk
      have : e (2 ^ k)  subseteq  f (e n) := by
        simp [e_pow2, Set.singleton_subset_iff, hkn]
      have : e (2 ^ k)  subseteq  f x := (h x (2 ^ k)).mpr <n, hn, this>
      simpa [e_pow2, Set.singleton_subset_iff] using this

end DomainTheory.DataTypesAsLattices
```

2.2.5 Graph

Dependencies. Inside Graph, each box is one subfolder, or one Lean file sitting directly in the folder. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. The arrowhead points at the file or folder that imports, and the number is how many direct imports.

IsGraph.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.Basic.notation3

@[expose] public section
```

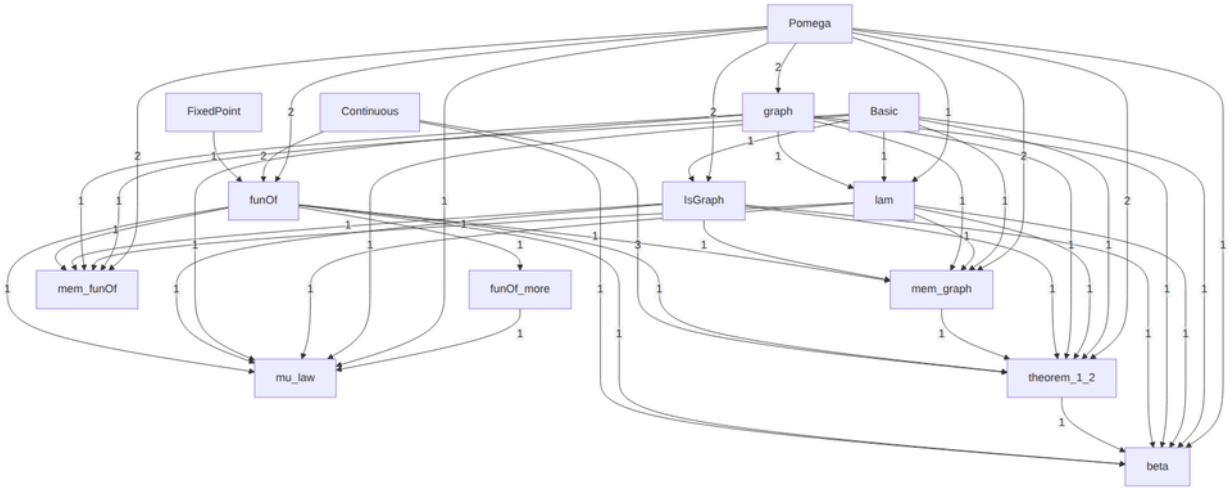


Figure 20: Graph.

```
namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 1.2 (iii).** Equality in (ii) iff `u` is downward
closed in the finite-set argument: `(k, m)  in  u` and `e k  subseteq  e n` imply `(n,
    m)  in  u`. -/
class IsGraph (u : Pomega) : Prop where
  out : forall  ?k m n?, pair k m  in  u  ->  e k  subseteq  e n  ->  pair n m  in  u

end DomainTheory.DataTypesAsLattices
```

Definition (graph) **DEFINITION.** The graph of a continuous function $f : \mathbf{P}\omega \to \mathbf{P}\omega$ is defined by the equation:

$$\mathbf{graph}(f) = \{(n, m) \mid m \in f(e_n)\};$$

while the function determined by any set $u \subseteq \omega$ is defined by the equation:

$$\mathbf{fun}(u)(x) = \{m \mid \exists e_n \subseteq x.\ (n, m) \in u\}.$$

Theorem 1.2 **THEOREM 1.2** (The graph theorem). *Every continuous function is uniquely determined by its graph in the sense that:*

(i) $\mathbf{fun}(\mathbf{graph}(f)) = f$.

Conversely, every set of integers determines a continuous function and we have:

(ii) $u \subseteq \mathbf{graph}(\mathbf{fun}(u))$,

where equality holds just in case u satisfies:

(iii) *whenever $(k, m) \in u$ and $e_k \subseteq e_n$, then $(n, m) \in u$.*

Besides functions of one variable we need to consider also functions of several variables. The official definition for one variable given above can be extended simply by saying $f(x, y, \dots)$ is continuous iff it is continuous in each of $x, y, \dots$. Those familiar with the product topology can prove that for our special positive topology on $\mathbf{P}\omega$ this is equivalent to being continuous on the product space (continuous in the variables jointly). Those interested only in elementary proofs can calculate out directly from the definition (with the aid of 1.1) that continuity behaves under combinations by substitution [as in: $f(g(x, y), h(y, x, y))$].

2.2.6 FixedPoint

Dependencies. Inside FixedPoint, each box is one Lean file. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. The arrowhead points at the file or folder that imports, and the number is how many direct imports.

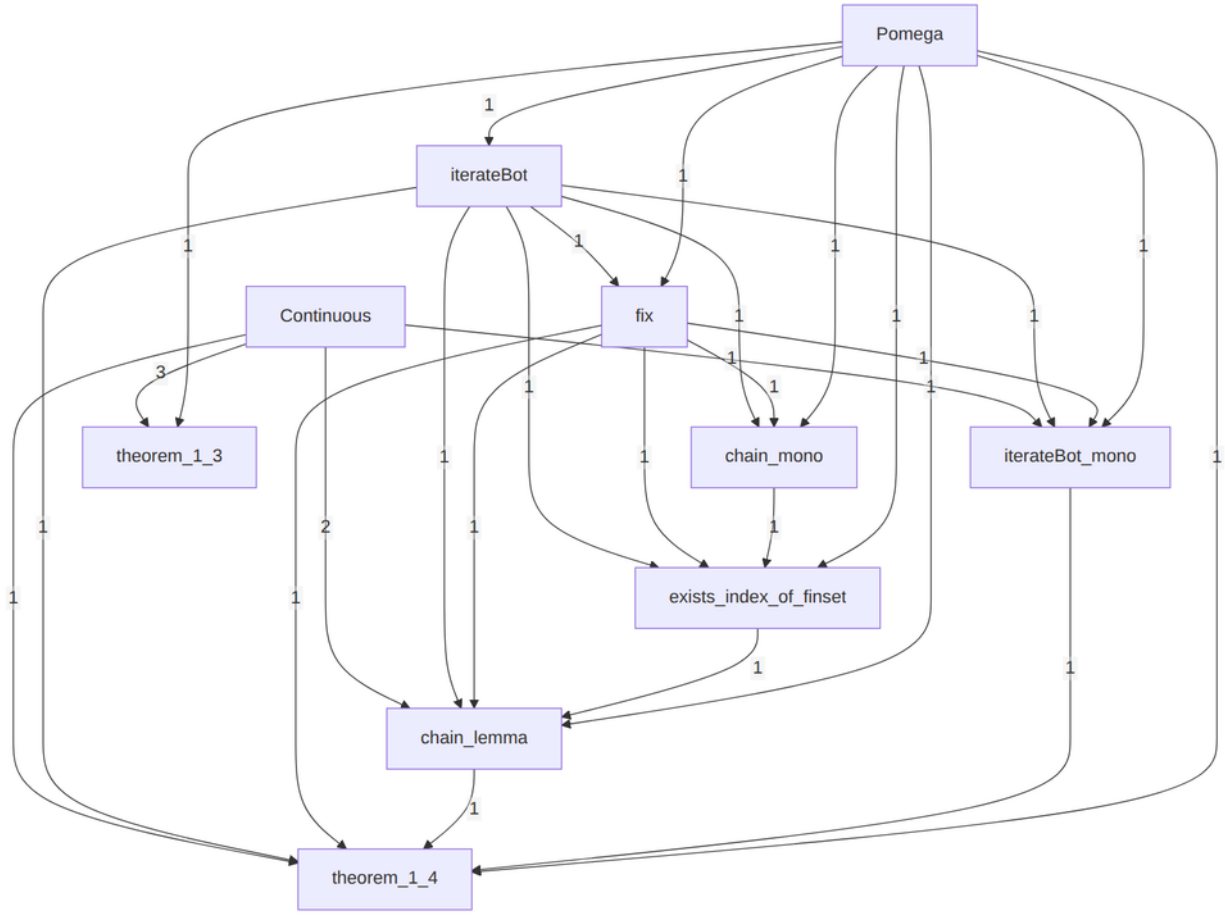


Figure 21: FixedPoint.

chain_mono.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
```

```
public import DomainTheory.DataTypesAsLattices.FixedPoint.iterateBot
public import DomainTheory.DataTypesAsLattices.FixedPoint.fix

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem chain_mono (xs : Nat  ->  Pomega) (hmono : forall  n, xs n  subseteq  xs (n +
    1)) :
    forall  a b, a  <=  b  ->  xs a  subseteq  xs b := by
  intro a b hab
  obtain <d, hd> := Nat.exists_eq_add_of_le hab
  subst hd
  clear hab
  induction d with
  | zero => simp
  | succ d ih => exact subset_trans ih (hmono _)

end DomainTheory.DataTypesAsLattices
```

Theorem 1.3 THEOREM 1.3 (The substitution theorem). *Continuous functions of several variables on* $\mathbf{P}\omega$ *are closed under substitution.*

theorem_1_3.lean

```
module

public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Continuous.theorem_1_1
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Continuous.scottUnion

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 1.3, composition.** -/
theorem theorem_1_3.comp {f g : Pomega  ->  Pomega}
    (hf : IsScottContinuous f) (hg : IsScottContinuous g) :
    IsScottContinuous (f  comp  g) := by
  constructor
  intro x
  ext k
  constructor
  * intro hk
    have hk' : k  in  f (g x) := hk
    have := (theorem_1_1 f).mp hf (g x) (2 ^ k)
    have hfin : e (2 ^ k)  subseteq  f (g x) := by
      simpa [e_pow2, Set.singleton_subset_iff] using hk'
    obtain <m, hm, hkm> := this.mp hfin
    have := (theorem_1_1 g).mp hg x m
    obtain <n, hn, hmn> := this.mp hm
    refine mem_scottUnion.mpr <n, hn, ?_>
    have : e (2 ^ k)  subseteq  f (g (e n)) :=
```

```
      subset_trans hkm (IsScottContinuous.monotone hf hmn)
    simpa [e_pow2, Set.singleton_subset_iff] using this
  * intro hk
    obtain <n, hn, hkn> := mem_scottUnion.mp hk
    exact IsScottContinuous.monotone hf (IsScottContinuous.monotone hg hn) hkn

/-- **Scott 1976, Theorem 1.3 (The substitution theorem), binary case.**
If `f` is continuous in each argument separately, then `x  |->  f(x,x)` is
continuous. -/
theorem theorem_1_3.diag {f : Pomega  ->  Pomega  ->  Pomega}
    (hfx : forall  y, IsScottContinuous (fun x => f x y))
    (hfy : forall  x, IsScottContinuous (fun y => f x y))
    {x : Pomega} :
    f x x = ? n, {k | e n  subseteq  x  /\  k  in  f (e n) (e n)} := by
  apply subset_antisymm
  * intro k hk
    have : k  in  scottUnion (fun z => f z x) x := by
      rw [ <-  (hfx x).out x]; exact hk
    obtain <n, hn, hkn> := mem_scottUnion.mp this
    have : k  in  scottUnion (fun y => f (e n) y) x := by
      rw [ <-  (hfy (e n)).out x]; exact hkn
    obtain <m, hm, hkm> := mem_scottUnion.mp this
    refine Set.mem_iUnion.mpr <n ||| m, ?_>
    refine <e_or_of_subset hn hm, ?_>
    have hle : e n  subseteq  e (n ||| m) := by rw [e_or]; exact Set.subset_union_left
    have hre : e m  subseteq  e (n ||| m) := by rw [e_or]; exact Set.
    subset_union_right
    exact IsScottContinuous.monotone (hfy (e (n ||| m))) hre
      (IsScottContinuous.monotone (hfx (e m)) hle hkm)
  * intro k hk
    obtain <n, hn, hkn> := Set.mem_iUnion.mp hk
    exact IsScottContinuous.monotone (hfy x) hn
      (IsScottContinuous.monotone (hfx (e n)) hn hkn)

/-- **Scott 1976, Theorem 1.3, binary substitution.** If `f` is continuous
in each argument and `g, h` are continuous, then `x  |->  f(g x, h x)` is
continuous. -/
theorem theorem_1_3.tuple {f : Pomega  ->  Pomega  ->  Pomega}
    (hfx : forall  y, IsScottContinuous (fun x => f x y))
    (hfy : forall  x, IsScottContinuous (fun y => f x y))
    {g h : Pomega  ->  Pomega} (hg : IsScottContinuous g) (hh : IsScottContinuous h) :
    IsScottContinuous (fun x => f (g x) (h x)) := by
  constructor
  intro x
  have H := theorem_1_3.diag (x := x)
    (f := fun u v => f (g u) (h v))
    (fun v => theorem_1_3.comp (hfx (h v)) hg)
    (fun u => theorem_1_3.comp (hfy (g u)) hh)
  ext k
  constructor
  * intro hk
    have : k  in  ? n, {k | e n  subseteq  x  /\  k  in  f (g (e n)) (h (e n))} := by
    rwa [ <-  H]
    obtain <n, hn, hkn> := Set.mem_iUnion.mp this
    exact mem_scottUnion.mpr <n, hn, hkn>
  * intro hk
```

```
    obtain <n, hn, hkn> := mem_scottUnion.mp hk
    have : k  in  ? n, {k | e n  subseteq  x  /\  k  in  f (g (e n)) (h (e n))} :=
      Set.mem_iUnion.mpr <n, hn, hkn>
    rwa [ <-  H] at this

/-- **Scott 1976, Theorem 1.3, ternary substitution.** -/
theorem theorem_1_3.nary {f : Pomega  ->  Pomega  ->  Pomega  ->  Pomega}
    (hf0 : forall  y z, IsScottContinuous (fun x => f x y z))
    (hf1 : forall  x z, IsScottContinuous (fun y => f x y z))
    (hf2 : forall  x y, IsScottContinuous (fun z => f x y z))
    {g h i : Pomega  ->  Pomega}
    (hg : IsScottContinuous g) (hh : IsScottContinuous h) (hi : IsScottContinuous i) :
    IsScottContinuous (fun x => f (g x) (h x) (i x)) := by
  constructor
  intro x
  have H := theorem_1_3.diag (x := x)
    (f := fun u v => f (g u) (h u) (i v))
    (fun v => theorem_1_3.tuple (fun y => hf0 y (i v)) (fun x => hf1 x (i v)) hg hh)
    (fun u => theorem_1_3.comp (hf2 (g u) (h u)) hi)
  ext k
  constructor
  * intro hk
    have : k  in  ? n, {k | e n  subseteq  x  /\  k  in  f (g (e n)) (h (e n)) (i (e n
    ))} := by
      rwa [ <-  H]
    obtain <n, hn, hkn> := Set.mem_iUnion.mp this
    exact mem_scottUnion.mpr <n, hn, hkn>
  * intro hk
    obtain <n, hn, hkn> := mem_scottUnion.mp hk
    have : k  in  ? n, {k | e n  subseteq  x  /\  k  in  f (g (e n)) (h (e n)) (i (e n
    ))} :=
      Set.mem_iUnion.mpr <n, hn, hkn>
    rwa [ <-  H] at this

/-- **Scott 1976, Theorem 1.3 (The substitution theorem).** Continuous
functions of several variables on `P?` are closed under substitution. -/
theorem theorem_1_3 {f g : Pomega  ->  Pomega}
    (hf : IsScottContinuous f) (hg : IsScottContinuous g) :
    IsScottContinuous (fun x => f (g x)) :=
  theorem_1_3.comp hf hg

end DomainTheory.DataTypesAsLattices
```

Theorem 1.4 The other general fact about continuous functions that we shall use constantly concerns fixed points, whose existence can be proved using a well-known method.

THEOREM 1.4 (The fixed-point theorem). *Every continuous function $f : \mathbf{P}\omega \to \mathbf{P}\omega$ has a least fixed point given by the formula:*

$$\mathbf{fix}(f) = \bigcup\{f^n(\varnothing) \mid n \in \omega\},$$

where $\varnothing$ is the empty set and f^n is the n-fold composition of f with itself.

Actually **fix** is a *functional* with continuity properties of its own. We shall not give the required definitions here because they can be more easily derived from the construction of the model given in the next

section.

For those familiar with the abstract theory of topological spaces we give in conclusion two general facts about continuous functions with values in $\mathbf{P}\omega$ which indicate the scope and generality of our method.

theorem_1_4.lean

```
module

public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Order
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.FixedPoint.iterateBot
public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
public import DomainTheory.DataTypesAsLattices.FixedPoint.chain_lemma
public import DomainTheory.DataTypesAsLattices.FixedPoint.iterateBot_mono

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 1.4 (The fixed-point theorem).** -/
theorem theorem_1_4 {f : Pomega  ->  Pomega} (hf : IsScottContinuous f) :
    f (fix f) = fix f  /\  forall  x, f x = x  ->  fix f  subseteq  x := by
  constructor
  * -- f(fix f) = ? f(f?({})) = ? f?^+1({}) = fix f
    have hchain := chain_lemma hf (iterateBot f) (iterateBot_mono hf)
    unfold fix
    rw [hchain]
    ext k
    constructor
    * intro hk
      obtain <n, hkn> := Set.mem_iUnion.mp hk
      exact Set.mem_iUnion.mpr <n + 1, hkn>
    * intro hk
      obtain <n, hkn> := Set.mem_iUnion.mp hk
      cases n with
      | zero => simp [iterateBot, botElem] at hkn
      | succ n => exact Set.mem_iUnion.mpr <n, hkn>
  * intro x hx k hk
    obtain <n, hkn> := Set.mem_iUnion.mp hk
    have hsub : forall  n, iterateBot f n  subseteq  x := by
      intro n
      induction n with
      | zero => simp [iterateBot, botElem]
      | succ n ih =>
        simpa [iterateBot, hx] using IsScottContinuous.monotone hf ih
    exact hsub n hkn

end DomainTheory.DataTypesAsLattices
```

2.2.7 Embedding

Dependencies. Inside Embedding, each box is one Lean file. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. The arrowhead points at the file or folder that imports, and the number is how many direct imports.

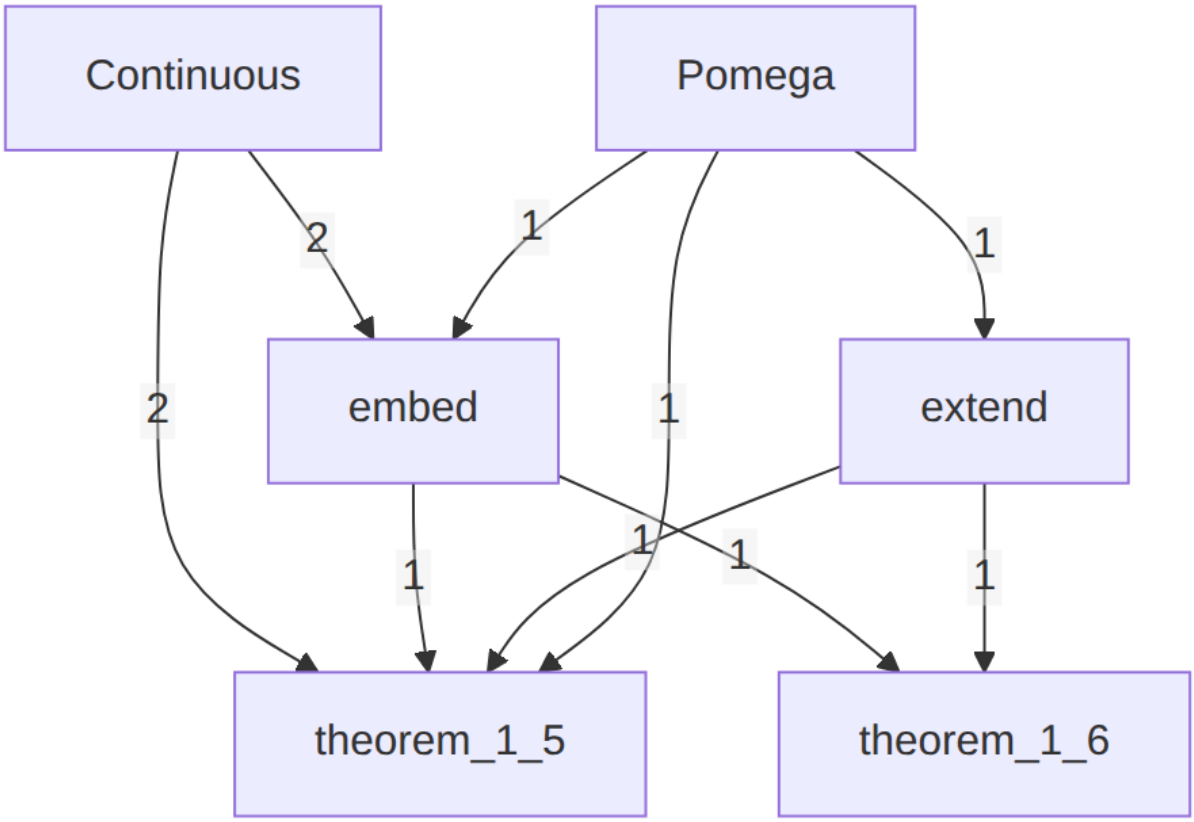


Figure 22: Embedding.

embed.lean

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.memOpen
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Continuous.Basic
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace Topology Function
```

```
/-- **Scott 1976, Theorem 1.6.** The embedding `?(x) = {n | x  in  U n}`. -/
def embed {X : Type*} (U : Nat  ->  Set X) (x : X) : Pomega :=
  {n | x  in  U n}

namespace embed

theorem continuous {X : Type*} [TopologicalSpace X]
    (U : Nat  ->  Set X) (hbasis : IsTopologicalBasis (Set.range U)) :
    Continuous (embed U) :=
  continuous_generateFrom_iff.2 fun t <n, hn> => by
    subst hn
    change IsOpen (U n)
    exact hbasis.isOpen <n, rfl>

theorem inducing {X : Type*} [TopologicalSpace X]
    (U : Nat  ->  Set X) (hbasis : IsTopologicalBasis (Set.range U)) :
    IsInducing (embed U) := by
  refine <?_>
  rw [hbasis.eq_generateFrom]
  change generateFrom (Set.range U) =
    (generateFrom (Set.range memOpen)).induced (embed U)
  rw [induced_generateFrom_eq]
  congr 1
  ext V
  constructor
  * rintro <n, rfl>
    exact <memOpen n, <n, rfl>, rfl>
  * rintro <_, <n, rfl>, rfl>
    exact <n, rfl>

theorem injective {X : Type*} [TopologicalSpace X] [T0Space X]
    (U : Nat  ->  Set X) (hbasis : IsTopologicalBasis (Set.range U)) :
    Injective (embed U) := by
  intro x y hxy
  have hmem : forall  n, x  in  U n  <->  y  in  U n := fun n =>
    show n  in  embed U x  <->  n  in  embed U y by rw [hxy]
  refine Inseparable.eq ?_
  refine inseparable_iff_forall_isOpen.mpr ?_
  intro V hV
  constructor
  * intro hx
    obtain <t, <n, rfl>, hxt, htV> :=
      hbasis.mem_nhds_iff.mp (hV.mem_nhds hx)
    exact htV ((hmem n).mp hxt)
  * intro hy
    obtain <t, <n, rfl>, hyt, htV> :=
      hbasis.mem_nhds_iff.mp (hV.mem_nhds hy)
    exact htV ((hmem n).mpr hyt)

end embed

end DomainTheory.DataTypesAsLattices
```

extend.lean

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Data.Set.Lattice.Indexed
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace Topology Function

/-- **Scott 1976, Theorem 1.5.** Maximal continuous extension of
`f : X  ->  P?` along a subspace inclusion `X  subseteq  Y`:
`f(y) = ? { ? {f(x) | x  in  X  I  U} | y  in  U, U open in Y }`. -/
def extend {Y : Type*} [TopologicalSpace Y] {X : Set Y}
    (f : X  ->  Pomega) (y : Y) : Pomega :=
  ?_0 { t | exists  U : Set Y, IsOpen U  /\  y  in  U  /\  t = ?_0 (f '' {x : X | x
    in  U}) }

namespace extend

theorem mem {Y : Type*} [TopologicalSpace Y] {X : Set Y}
    (f : X  ->  Pomega) {y : Y} {k : Nat} :
    k  in  extend (Y := Y) (X := X) f y  <->
      exists  U : Set Y, IsOpen U  /\  y  in  U  /\  forall  x : X, x  in  U  ->  k
    in  f x := by
  constructor
  * intro hk
    rcases Set.mem_sUnion.mp hk with <t, <U, hU, hy, rfl>, hkt>
    refine <U, hU, hy, ?_>
    intro x hx
    exact Set.mem_sInter.mp hkt (f x) <x, hx, rfl>
  * intro <U, hU, hy, hall>
    refine Set.mem_sUnion.mpr <?_0 (f '' {x : X | x  in  U}), <U, hU, hy, rfl>, ?_>
    intro t ht
    rcases ht with <x, hx, rfl>
    exact hall x hx

theorem isOpen_mem {Y : Type*} [TopologicalSpace Y] {X : Set Y}
    (f : X  ->  Pomega) (k : Nat) :
    IsOpen {y : Y | k  in  extend (Y := Y) (X := X) f y} := by
  have : {y : Y | k  in  extend (Y := Y) (X := X) f y} =
      ? U  in  {U : Set Y | IsOpen U  /\  forall  x : X, x  in  U  ->  k  in  f x}, U
    := by
    ext y
    constructor
    * intro hy
      obtain <U, hU, hyU, hall> := (mem f).mp hy
      exact Set.mem_iUnion_2.mpr <U, <hU, hall>, hyU>
    * intro hy
      obtain <U, hU, hyU> := Set.mem_iUnion_2.mp hy
      exact (mem f).mpr <U, hU.1, hyU, hU.2>
  rw [this]
```

```
  exact isOpen_biUnion fun _ hU => hU.1

end extend

end DomainTheory.DataTypesAsLattices
```

Theorem 1.5 **THEOREM 1.5** (The extension theorem). *Let X and Y be arbitrary topological spaces where $X \subseteq Y$ as a subspace. Then every continuous function $f : X \to \mathbf{P}\omega$ can be extended to a continuous function $\bar{f} : Y \to \mathbf{P}\omega$ defined by the equation:*

$$\bar{f}(y) = \bigcup\{\bigcap\{f(x) \mid x \in X \cap U\} \mid y \in U\},$$

where $y \in Y$, and U ranges over the open subsets of Y.

theorem_1_5.lean

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.isOpen_memOpen
public import DomainTheory.DataTypesAsLattices.Continuous.memOpen
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Embedding.extend
public import DomainTheory.DataTypesAsLattices.Embedding.embed

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace Topology Function

/-- **Scott 1976, Theorem 1.5 (The extension theorem), continuity.** -/
theorem theorem_1_5 {Y : Type*} [TopologicalSpace Y] {X : Set Y}
    (f : X  ->  Pomega) : Continuous (extend (Y := Y) (X := X) f) := by
  refine continuous_generateFrom_iff.2 ?_
  rintro _ <k, rfl>
  exact extend.isOpen_mem f k

/-- **Scott 1976, Theorem 1.5**, extension property when `f` is continuous. -/
theorem theorem_1_5.extension {Y : Type*} [TopologicalSpace Y] {X : Set Y}
    (f : X  ->  Pomega) (hf : Continuous f) (x : X) :
    extend (Y := Y) (X := X) f x = f x := by
  apply subset_antisymm
  * intro k hk
    obtain <U, _, hxU, hall> := (extend.mem f).mp hk
    exact hall x hxU
  * intro k hk
    have hpre : IsOpen (f ^-1' memOpen k) :=
      IsOpen.preimage hf (isOpen_memOpen k)
    obtain <U, hU, hrel> := isOpen_induced_iff.mp hpre
    refine (extend.mem f).mpr <U, hU, ?_, ?_>
```

```
    * have : x  in  Subtype.val ^-1' U := by
        rwa [hrel]
      exact this
    * intro z hz
      have : z  in  f ^-1' memOpen k := by
        rwa [ <-  hrel]
      exact this

end DomainTheory.DataTypesAsLattices
```

Theorem 1.6 **THEOREM 1.6** (The embedding theorem). *Every T_0-space X with a countable basis $\{U_n \mid n \in \omega\}$ for its topology can be embedded in $\mathbf{P}\omega$ by the continuous function $\varepsilon : X \to \mathbf{P}\omega$ defined by the equation:*

$$\varepsilon(x) = \{n \mid x \in U_n\}.$$

Technically the T_0-hypothesis is what is needed to show that ε is one-to-one. The upshot of these two theorems is that in looking for (reasonable) topological structures we can confine attention to the subspaces of $\mathbf{P}\omega$ and to continuous functions defined on *all* of $\mathbf{P}\omega$. Thus the emphasis on a single space is justified structurally. What we shall see in the remainder of this work is that the use of a single space is also justified practically because the required subspaces and functions can be *defined* in very simple ways by a natural method of equations.

theorem_1_6.lean

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Embedding.extend
public import DomainTheory.DataTypesAsLattices.Embedding.embed

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace Topology Function

/-- **Scott 1976, Theorem 1.6 (The embedding theorem).**
Every T_0 space with a countable basis embeds in `P?`. -/
theorem theorem_1_6 {X : Type*} [TopologicalSpace X] [T0Space X]
    (U : Nat  ->  Set X) (hbasis : IsTopologicalBasis (Set.range U)) :
    IsEmbedding (embed U) where
  toIsInducing := embed.inducing U hbasis
  injective := embed.injective U hbasis

theorem theorem_1_6.injective_continuous {X : Type*} [TopologicalSpace X]
    [T0Space X] (U : Nat  ->  Set X) (hbasis : IsTopologicalBasis (Set.range U)) :
    Injective (embed U)  /\  Continuous (embed U) :=
  <embed.injective U hbasis, embed.continuous U hbasis>
```

```
end DomainTheory.DataTypesAsLattices
```

2.3 Computability and definability

2.3.1 Historical note

The language LAMBDA is Scott's. Plotkin and Rogers already used graphs, application, and conversion, and the combinators they had would have defined abstraction if anyone had asked; the abstraction operation is what this section supplies. The reduction theorem and the combinator theorem are the classical ones from combinatory logic. The first recursion theorem is classical as well. The argument that Curry's combinator $\mathbf{Y}$ is the least fixed point in this model is David Park's, from an unpublished note of 1970, and it is recorded with Theorem 2.5. The equation $\mathbf{Y}(u) = u(\mathbf{Y}(u))$ is the usual calculation from (2.8). Leastness uses the pairing (n, m) and the finite sets e_l, and does not appeal to Theorem 1.4. The definition of computability and Theorem 2.6 identify the LAMBDA-definable functions with the enumeration operators.

Dependency diagram. Each box is one note that contains Lean. The arrowhead is on the note that imports. The number is how many direct imports. An arrow already implied by a longer path is left out.

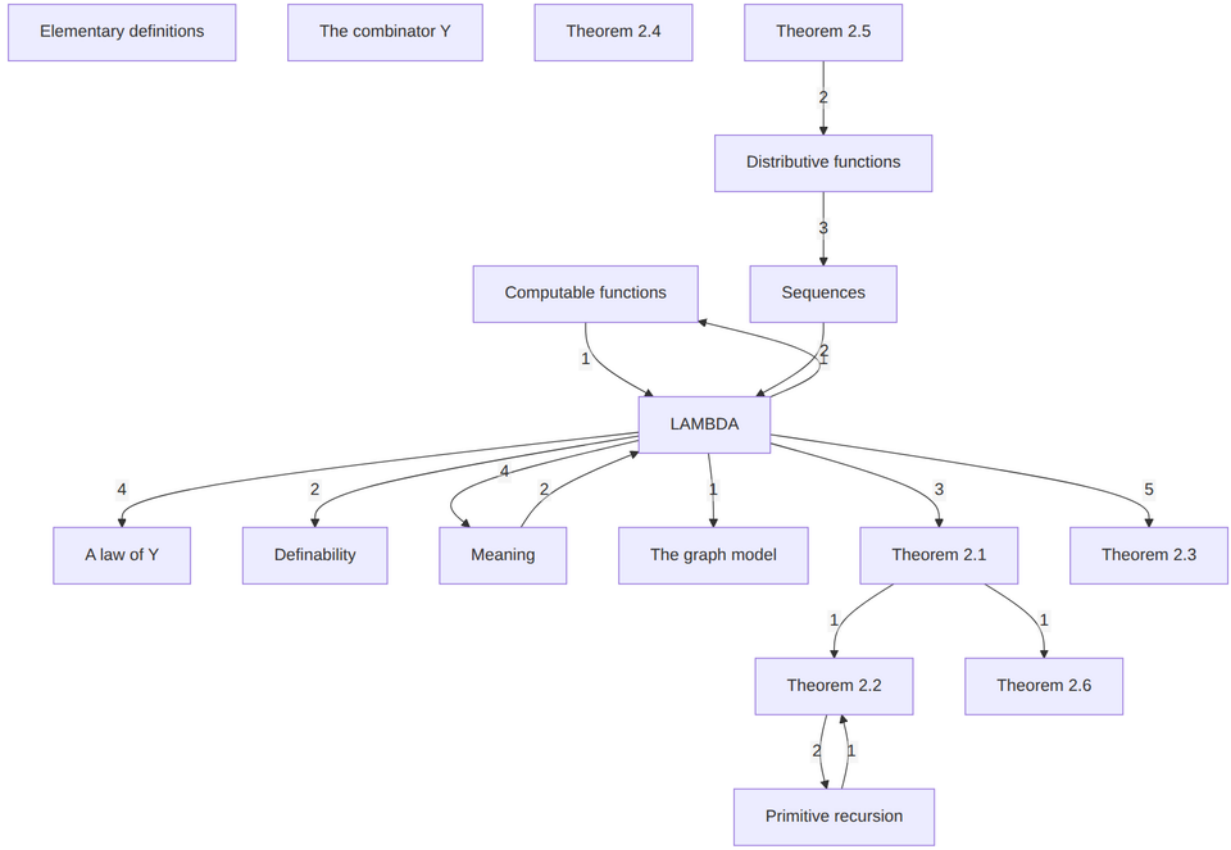


Figure 23: Historical note.

2.3.2 Meaning

The purpose of the first section was to introduce in a simple-minded way the basic notions about the topology of $\mathbf{P}\omega$ and its continuous functions. In this section we wish to present the details of a powerful language for defining *particular* functions—especially computable functions—and initiate the study of the *use* of these functions. This study is then extended in different ways in the following sections.

Before looking at the language, a short discussion of the "meaning" of the elements $x \in \mathbf{P}\omega$ will be helpful from the point of view of motivation. Now in itself $x \in \mathbf{P}\omega$ is a set, but this does not reveal its *meaning*. Actually x has no "fixed" meaning, because it can be *used* in strikingly different ways; we look for meaning here solely in terms of use. Nevertheless it is possible to give some coherent guidelines.

In the first place it is convenient to let the singleton subsets $\{n\} \in \mathbf{P}\omega$ stand for the corresponding integers. In fact, we shall enforce by convention the equation $n = \{n\}$ as a way of simplifying notation. In this way, $\omega \subseteq \mathbf{P}\omega$ as a subset. (Note that our convention conflicts with such set-theoretical equations as $5 = \{0, 1, 2, 3, 4\}$. What we have done is to abandon the usual set-theoretical conventions in favor of a slight redefinition of *set of integers* which produces a more helpful convention for present purposes.) So, if we choose, a singleton "means" a single integer. The next question is what a "large" set $x \in \mathbf{P}\omega$ could mean. Here is an answer: if we write:

$$x = \{0, 5, 17\} = 0 \cup 5 \cup 17,$$

we are thinking of x as a *multiple integer*. This is especially useful in the case of multiple-valued function where we can write:

$$f(a) = 0 \cup 5 \cup 17.$$

Then "$m \in f(a)$" can be interpreted as "m is one value of f at a." Now $a \in \mathbf{P}\omega$, too, and so it is a multiple integer also. This brings us to an important point.

A multiple integer is (often) *more* than just the (random) collection of its elements. From the definition of continuity, $m \in f(a)$ is equivalent to $m \in f(e_n)$ with $e_n \subseteq a$. We may not be able to reduce this to $m \in f(\{n\})$ with $n \in a$ without additional assumptions on f. Indeed we shall take advantage of the feature of continuous functions whereby the elements of an argument a can join in *cooperation* in determining $f(a)$. Needless to say, continuity implies that the cooperation cannot extend beyond *finite* configurations, and so we can say that a is the union (or limit) of its finite subsets. However, finitary cooperation will be found to be quite a powerful notion.

Where does this interpretation leave the empty set $\varnothing$? When we write "$f(a) = \varnothing$" we can read this as "f has no value at a", or "f is undefined at a". In this case $f(a)$ exists (as a set), but it is not "defined" as an integer. Single- (or singleton-) valued functions are "well-defined", but multiple-valued functions are rather "over-defined".

How does this interpretation fit in with monotonicity? In case $a \subseteq b$ and $m \in f(a)$, then we must have $m \in f(b)$. We can read "$a \subseteq b$" as "b is an *improvement of* a" or "b is *better-defined than* a." The point of monotonicity is that the better we define an argument, the better we define a value. "Better" does not imply "well" (that is, singleton-valuedness), and overdefinedness may well creep in. This is not the fault of the function; it is our fault for not choosing a different function.

As a subspace $\omega \subseteq \mathbf{P}\omega$ is discrete. This implies that arbitrary functions $p : \omega \to \omega$ are continuous. Note that $p : \omega \to \mathbf{P}\omega$ as well, because $\omega \subseteq \mathbf{P}\omega$. By Theorem 1.5 we can extend p continuously to $\bar{p} : \mathbf{P}\omega \to \mathbf{P}\omega$. The formula given produces this function:

$$(2.1)\quad \bar{p}(x) = \begin{cases} \bigcap\{p(n) \mid n \in \omega\} & \text{if } x = \varnothing; \\ p(n) & \text{if } x = n \in \omega; \\ \omega & \text{otherwise.} \end{cases}$$

This is a rather abrupt extension of p (the *maximal extension*); a more gradual, continuous extension (the *minimal extension*) is determined by this equation:

$$(2.2)\quad \hat{p}(x) = \bigcup\{p(n) \mid n \in x\}.$$

The same formulae work for *all* multiple-valued functions $p : \omega \to \mathbf{P}\omega$. Functions like $f = \hat{p}$ are exactly characterized as being those continuous functions $f : \mathbf{P}\omega \to \mathbf{P}\omega$ which in addition are *distributive* in the sense of these equations:

$$f(x \cup y) = f(x) \cup f(y) \quad \text{and} \quad f(\varnothing) = \varnothing.$$

The sets $\varnothing$ and ω play special roles. When we consider them as elements of $\mathbf{P}\omega$ we shall employ the notation:

$$\bot = \varnothing \quad \text{and} \quad \top = \omega$$

The element $\bot$ is the most "undefined" integer, and $\top$ is the most "overdefined". All others are in between.

One last general point on meaning: suppose $x \in \mathbf{P}\omega$ and $k \in x$. Then $k = (n, m)$ for suitable (uniquely determined) integers n and m. That is to say, every element of x can be regarded as an ordered pair; thus, x can be used as a relation. Such an operation as

$$(2.3)\quad x; y = \{(n, l) \mid \exists m.\, (n, m) \in x,\, (m, l) \in y\}$$

is then a continuous function of two variables that treats both x and y as relations. On the other hand we could define a quite different continuous function such as

$$(2.4)\quad x + y = \{n + m \mid n \in x,\, m \in y\}$$

which treats both x and y arithmetically. The only reason we shall probably never write $(x + y); x$ again is that the values of this perfectly well-defined continuous function are, for the most part, quite uninteresting. There is, however, no theoretical reason why we cannot use the same set with several different "meanings" in the same formula. Of course if we do so, it is to be expected that we will show the point of doing this in the special case. We turn now to the definition of the general language for defining all such functions.

eq_2_1_empty.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.maxExtend
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section
```

```
namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.1).** Maximal extension by cases. -/
theorem eq_2_1_empty (p : Nat  ->  Pomega) :
    maxExtend p botElem = ? n, p n := by
  ext k
  constructor
  * intro hk
    rcases hk with <_, hall> | h | h
    * exact Set.mem_iInter.mpr hall
    * rcases h with <n, hn, _>
      have : n  in  ({} : Pomega) := by
        have : ({} : Pomega) = ofNat n := hn
        rw [this]; simp [ofNat]
      exact this.elim
    * exact (h.1 rfl).elim
  * intro hk
    exact Or.inl <rfl, fun n => Set.mem_iInter.mp hk n>

end DomainTheory.DataTypesAsLattices
```

minExtend.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Data.Set.Lattice.Indexed
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.2).** Minimal (distributive) extension `p(x) = ? {p(n) | n  in  x
    }`. -/
def minExtend (p : Nat  ->  Pomega) (x : Pomega) : Pomega :=
  ? n  in  x, p n

namespace minExtend

theorem bot (p : Nat  ->  Pomega) : minExtend p botElem = botElem := by
  simp [minExtend, botElem]

theorem isScottContinuous (p : Nat  ->  Pomega) :
    IsScottContinuous (minExtend p) := by
  constructor
  intro x
  ext k
  constructor
  * intro hk
    obtain <n, hn, hkn> := Set.mem_iUnion_2.mp hk
    exact mem_scottUnion.mpr <2 ^ n, by simp [e_pow2, Set.singleton_subset_iff, hn],
      by simp [minExtend, e_pow2, hkn]>
  * intro hk
    obtain <m, hm, hkm> := mem_scottUnion.mp hk
```

```
    obtain <n, hn, hkn> := Set.mem_iUnion_2.mp hkm
    exact Set.mem_iUnion_2.mpr <n, hm hn, hkn>

theorem union (p : Nat  ->  Pomega) (x y : Pomega) :
    minExtend p (x  U  y) = minExtend p x  U  minExtend p y := by
  ext k
  simp [minExtend, or_and_right, exists_or]

end minExtend

end DomainTheory.DataTypesAsLattices
```

maxExtend.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.1).** Maximal extension, written without a decidability side
    condition. -/
def maxExtend (p : Nat  ->  Pomega) (x : Pomega) : Pomega :=
  {k | (x = {}  /\  forall  n, k  in  p n)  \/
    (exists  n, x = ofNat n  /\  k  in  p n)  \/
    (x  !=  {}  /\  forall  n, x  !=  ofNat n)}

end DomainTheory.DataTypesAsLattices
```

relComp.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.3).** Relational composition. -/
def relComp (x y : Pomega) : Pomega :=
  {p | exists  n m l, p = pair n l  /\  pair n m  in  x  /\  pair m l  in  y}

end DomainTheory.DataTypesAsLattices
```

setSum.lean

```
module
```

```
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.4).** Set-sum. -/
def setSum (x y : Pomega) : Pomega :=
  {k | exists  n m, n  in  x  /\  m  in  y  /\  k = n + m}

end DomainTheory.DataTypesAsLattices
```

2.3.3 LAMBDA

The syntax and semantics of the language LAMBDA are set out in Table 2. The syntax is indicated on the left, and the meanings of the combinations are shown on the right as subsets of $\mathbf{P}\omega$. This is the basic language and could have been given (less understandably) in terms of combinators (see Theorem 2.4). It is, however, a very primitive language, and we shall require many definitions before we can see why such functions as in (2.3) and (2.4) are themselves definable.

TABLE 2

The language LAMBDA

$$0 = \{0\}$$

$$x + 1 = \{n + 1 \mid n \in x\}$$

$$x - 1 = \{n \mid n + 1 \in x\}$$

$$z \supset x, y = \{n \in x \mid 0 \in z\} \cup \{m \in y \mid \exists k.\, k + 1 \in z\}$$

$$u(x) = \{m \mid \exists e_n \subseteq x.\, (n, m) \in u\}$$

$$\lambda x.\, \tau = \{(n, m) \mid m \in \tau[e_n/x]\}$$

The definition has been left somewhat informal in hopes that it will be more understandable. In the above, τ is any term of the language. LAMBDA is type-free and allows any combination to be made by substitution into the given functions. There is one primitive constant (0); there are two unary functions $(x + 1,\ x - 1)$; there is one binary function $(u(x))$ and one ternary function $(z \supset x, y)$; finally there is one variable binding operator $(\lambda x.\, \tau)$. The first three equations have obvious sense. In the fourth, $z \supset x, y$ is McCarthy's conditional expression (a test for zero). Next $u(x)$ defines *application* (u is treated as a graph and x as a set), and $\lambda x.\, \tau$ is *functional abstraction* (compare the definition of **fun**). In defining $\lambda x.\, \tau$, we use $\tau[e_n/x]$ as a shorthand for evaluating the term τ when the variable x is given the value e_n.

Note that the functions are all multiple-valued. Thus we have such a result as:

$$(2.5) \quad (6 \cup 10) + 1 = 7 \cup 11.$$

The partial character of subtraction has expression as:

$$(2.6) \quad 0 - 1 = \bot.$$

We shall see how to define $+$ and $-$ in LAMBDA later. The conditional could also have been defined by cases:

$$(2.7) \quad z \supset x, y = \begin{cases} \bot & \text{if } z = \bot; \\ x & \text{if } z = 0; \\ y & \text{if } 0 \notin z \neq \bot; \\ x \cup y & \text{if } 0 \in z \neq 0. \end{cases}$$

We say that a LAMBDA-term τ defines a function of its free variables (at least).

combinatory_isLambdaDefinable.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.envSet
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Lambda.sucC
public import DomainTheory.DataTypesAsLattices.Lambda.predC
public import DomainTheory.DataTypesAsLattices.Lambda.condC
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.LambdaDefinable

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

/-- Combinatory elements are LAMBDA-definable (converse of Theorem 2.4). -/
theorem combinatory_isLambdaDefinable {u : Pomega} (hu : IsCombinatory u) :
    LambdaDefinable u := by
  induction hu with
  | zero => exact <<.zero, rfl>>
  | suc =>
    refine <<.lam 0 (.succ (.var 0)), ?_>>
    simp [interp, sucC, envSet, Function.update]
  | pred =>
    refine <<.lam 0 (.pred (.var 0)), ?_>>
    simp [interp, predC, envSet, Function.update]
  | cond =>
    refine <<.lam 0 (.lam 1 (.lam 2
        (.cond (.var 2) (.var 0) (.var 1)))), ?_>>
    simp [interp, condC, envSet, Function.update]
  | K =>
    refine <<.lam 0 (.lam 1 (.var 0)), ?_>>
```

```
    simp [interp, Kcomb, envSet, Function.update]
  | S =>
    refine <<.lam 0 (.lam 1 (.lam 2
        (.app (.app (.var 0) (.var 2)) (.app (.var 1) (.var 2))))), ?_>>
    simp [interp, Scomb, envSet, Function.update]
  | app _ _ iu iv =>
    obtain <t, ht> := iu.out
    obtain <s, hs> := iv.out
    exact <<.app t s, by simp [interp, ht, hs]>>

end DomainTheory.DataTypesAsLattices
```

Comb.lean

```
module

public import Mathlib.Data.Nat.Basic

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Combinatory expressions generated from the six constants and variables. -/
inductive Comb where
  | var : Nat  ->  Comb
  | zero | suc | pred | cond | K | S
  | app : Comb  ->  Comb  ->  Comb

end DomainTheory.DataTypesAsLattices
```

Kcomb.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def Kcomb : Pomega := graph (fun x => graph (fun _ => x))

end DomainTheory.DataTypesAsLattices
```

Scomb.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def Scomb : Pomega :=
  graph (fun u => graph (fun v => graph (fun x => funOf (funOf u x) (funOf v x))))

end DomainTheory.DataTypesAsLattices
```

Term.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.lam
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Table 2.** Syntax of LAMBDA. -/
inductive Term where
  | var : Nat  ->  Term
  | zero : Term
  | succ : Term  ->  Term
  | pred : Term  ->  Term
  | cond : Term  ->  Term  ->  Term  ->  Term
  | app : Term  ->  Term  ->  Term
  | lam : Nat  ->  Term  ->  Term

end DomainTheory.DataTypesAsLattices
```

condSet.lean

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import Mathlib.Data.Nat.Bitwise

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Table 2.** McCarthy conditional. -/
def condSet (z x y : Pomega) : Pomega :=
  {n | n  in  x  /\  0  in  z}  U  {m | m  in  y  /\  exists  k, k + 1  in  z}

namespace condSet
```

```
theorem bot (x y : Pomega) : condSet botElem x y = botElem := by
  ext n
  simp [condSet, botElem]

theorem ofNat_zero (x y : Pomega) : condSet (ofNat 0) x y = x := by
  ext n
  simp [condSet, ofNat]

theorem ofNat_one (x y : Pomega) : condSet (ofNat 1) x y = y := by
  ext n
  simp [condSet, ofNat]

theorem ofNat_succ (n : Nat) (x y : Pomega) :
    condSet (ofNat (n + 1)) x y = y := by
  ext k
  simp [condSet, ofNat]

theorem same_bot (x : Pomega) : condSet botElem x x = botElem :=
  bot x x

theorem same_of_mem {z x : Pomega} {k : Nat} (hk : k  in  z) :
    condSet z x x = x := by
  ext n
  constructor
  * intro hn
    rcases hn with <hnx, _> | <hnx, _>
    * exact hnx
    * exact hnx
  * intro hnx
    cases k with
    | zero => exact Or.inl <hnx, hk>
    | succ t => exact Or.inr <hnx, t, hk>

theorem same (z x : Pomega) (hz : z  !=  botElem) : condSet z x x = x := by
  obtain <k, hk> : exists  k, k  in  z := by
    by_contra h
    push Not at h
    exact hz (Set.ext fun n => by simp [botElem, h n])
  exact same_of_mem hk

theorem isScottContinuous_left (x y : Pomega) :
    IsScottContinuous (fun z => condSet z x y) := by
  constructor
  intro z
  ext k
  constructor
  * intro hk
    rcases hk with <hkx, h0> | <hky, <t, ht>>
    * have e1 : e 1 = ({0} : Pomega) := by simpa using e_pow2 0
      refine mem_scottUnion.mpr <1, ?_, Or.inl <hkx, ?_>>
      * simpa [e1, Set.singleton_subset_iff] using h0
      * simp [e1]
    * refine mem_scottUnion.mpr <2 ^ (t + 1), ?_, Or.inr <hky, t, ?_>>
      * simp [e_pow2, Set.singleton_subset_iff, ht]
      * simp [e_pow2]
  * intro hk
```

```
    obtain <m, hm, hkm> := mem_scottUnion.mp hk
    rcases hkm with <hkx, h0> | <hky, <t, ht>>
    * exact Or.inl <hkx, hm h0>
    * exact Or.inr <hky, t, hm ht>

theorem isScottContinuous_mid (z y : Pomega) :
    IsScottContinuous (fun x => condSet z x y) := by
  constructor
  intro x
  ext k
  constructor
  * intro hk
    rcases hk with <hkx, h0> | hky
    * refine mem_scottUnion.mpr <2 ^ k, by simp [e_pow2, Set.singleton_subset_iff, hkx
    ],
        Or.inl <by simp [e_pow2], h0>>
    * refine mem_scottUnion.mpr <0, by simp [e_zero], Or.inr hky>
  * intro hk
    obtain <m, hm, hkm> := mem_scottUnion.mp hk
    rcases hkm with <hkx, h0> | hky
    * exact Or.inl <hm hkx, h0>
    * exact Or.inr hky

theorem isScottContinuous_right (z x : Pomega) :
    IsScottContinuous (fun y => condSet z x y) := by
  constructor
  intro y
  ext k
  constructor
  * intro hk
    rcases hk with hzx | <hky, ht>
    * refine mem_scottUnion.mpr <0, by simp [e_zero], Or.inl hzx>
    * refine mem_scottUnion.mpr <2 ^ k, by simp [e_pow2, Set.singleton_subset_iff, hky
    ],
        Or.inr <by simp [e_pow2], ht>>
  * intro hk
    obtain <m, hm, hkm> := mem_scottUnion.mp hk
    rcases hkm with hzx | <hky, ht>
    * exact Or.inl hzx
    * exact Or.inr <hm hky, ht>

private theorem exists_pos_bit {n : Nat} (hn : 2  <=  n) :
    exists  k, n.testBit (k + 1) = true := by
  by_contra hex
  push Not at hex
  have hfalse : forall  k, n.testBit (k + 1) = false := fun k =>
    Bool.eq_false_iff.2 (hex k)
  have : n = 0  \/  n = 1 := by
    cases h0 : n.testBit 0 with
    | false =>
      exact Or.inl <| Nat.zero_of_testBit_eq_false fun i => by
        cases i with
        | zero => exact h0
        | succ k => exact hfalse k
    | true =>
      exact Or.inr <| Nat.eq_of_testBit_eq fun i => by
```

```
        cases i with
        | zero => simp [h0]
        | succ k =>
          rw [hfalse k]
          exact (Nat.testBit_eq_false_of_lt
            (Nat.one_lt_two_pow (Nat.succ_ne_zero k))).symm
  omega

theorem e_iff (n0 n1 n2 m : Nat) :
    m  in  condSet (e n2) (e n0) (e n1)  <->
      (n0.testBit m = true  /\  n2.testBit 0 = true)  \/
        (n1.testBit m = true  /\  2  <=  n2) := by
  constructor
  * intro h
    rcases h with <hm, h0> | <hm, <k, hk>>
    * exact Or.inl <mem_e.mp hm, mem_e.mp h0>
    * exact Or.inr <mem_e.mp hm,
        (Nat.pow_le_pow_right (by decide : 0 < (2 : Nat))
          (Nat.le_add_left 1 k) : (2 : Nat) ^ 1  <=  2 ^ (k + 1)).trans
          (two_pow_le_of_testBit (mem_e.mp hk))>
  * intro h
    rcases h with <hm, h0> | <hm, hn2>
    * exact Or.inl <mem_e.mpr hm, mem_e.mpr h0>
    * obtain <k, hk> := exists_pos_bit hn2
      exact Or.inr <mem_e.mpr hm, k, mem_e.mpr hk>

end condSet

end DomainTheory.DataTypesAsLattices
```

condC.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def condC : Pomega :=
  graph (fun x => graph (fun y => graph (fun z => condSet z x y)))

end DomainTheory.DataTypesAsLattices
```

const_isScottContinuous.lean

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
```

```
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem const_isScottContinuous (c : Pomega) :
    IsScottContinuous (fun _ : Pomega => c) := by
  constructor
  intro x
  ext k
  constructor
  * intro hk
    exact mem_scottUnion.mpr <0, by simp [e_zero], hk>
  * intro hk
    obtain <_, _, hkn> := mem_scottUnion.mp hk
    exact hkn

end DomainTheory.DataTypesAsLattices
```

dcondSet.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.Comb
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.4).** Doubly strict conditional
`z ? x, y = z ? (z ? x,  top ), (z ?  top , y)`. -/
def dcondSet (z x y : Pomega) : Pomega :=
  condSet z (condSet z x topElem) (condSet z topElem y)

end DomainTheory.DataTypesAsLattices
```

e_one.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem e_one : e 1 = ({0} : Pomega) := by
```

```
  simpa using e_pow2 0

end DomainTheory.DataTypesAsLattices
```

envSet.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
def envSet (? : Nat  ->  Pomega) (i : Nat) (v : Pomega) : Nat  ->  Pomega :=
  Function.update ? i v

namespace envSet

theorem comm {? : Nat  ->  Pomega} {i j : Nat} (hij : i  !=  j) (v w : Pomega) :
    envSet (envSet ? i v) j w = envSet (envSet ? j w) i v := by
  funext k
  unfold envSet
  by_cases hj : k = j
  * subst hj
    simp [Function.update, hij.symm]
  * by_cases hi : k = i
    * subst hi
      simp [Function.update, hij]
    * simp [Function.update, hi, hj]

end envSet

end DomainTheory.DataTypesAsLattices
```

eq_2_1_else.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.maxExtend
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_1_else (p : Nat  ->  Pomega) {x : Pomega}
    (hne : x  !=  botElem) (hsing : forall  n, x  !=  ofNat n) :
    maxExtend p x = topElem := by
  ext k
  constructor
  * intro; trivial
  * intro
    exact Or.inr (Or.inr <hne, hsing>)
```

```
end DomainTheory.DataTypesAsLattices
```

eq_2_1_singleton.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.maxExtend
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_1_singleton (p : Nat  ->  Pomega) (n : Nat) :
    maxExtend p (ofNat n) = p n := by
  ext k
  constructor
  * intro hk
    rcases hk with <he, _> | <m, hm, hkm> | <_, hall>
    * have hne : ofNat n  !=  {} := Set.Nonempty.ne_empty <n, (rfl : n  in  ofNat n)>
      exact (hne he).elim
    * have : n = m := by
        have hn : n  in  ofNat m := hm  |>  (rfl : n  in  ofNat n)
        simpa [ofNat] using hn
      subst this; exact hkm
    * exact (hall n rfl).elim
  * intro hk
    exact Or.inr (Or.inl <n, rfl, hk>)

end DomainTheory.DataTypesAsLattices
```

eq_2_5.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.succSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.5).** `(6  U  10) + 1 = 7  U  11`. -/
theorem eq_2_5 : succSet (ofNat 6  U  ofNat 10) = ofNat 7  U  ofNat 11 := by
  ext k
  constructor
  * intro <n, hn, hk>
    rcases hn with h6 | h10
    * simp [ofNat] at h6; subst h6; simp [ofNat, hk]
    * simp [ofNat] at h10; subst h10; simp [ofNat, hk]
  * intro hk
```

```
    rcases hk with h7 | h11
    * simp [ofNat] at h7; exact <6, Or.inl (by simp [ofNat]), by simp [h7]>
    * simp [ofNat] at h11; exact <10, Or.inr (by simp [ofNat]), by simp [h11]>

end DomainTheory.DataTypesAsLattices
```

eq_2_6.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.predSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.6).** `0 ? 1 = False`. -/
theorem eq_2_6 : predSet (ofNat 0) = botElem := by
  ext k
  simp [predSet, ofNat, botElem]

end DomainTheory.DataTypesAsLattices
```

eq_2_7_bot.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.7).** Conditional cases. -/
theorem eq_2_7_bot (x y : Pomega) : condSet botElem x y = botElem := by
  ext n
  simp [condSet, botElem]

end DomainTheory.DataTypesAsLattices
```

eq_2_7_mix.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section
```

```
namespace DomainTheory.DataTypesAsLattices

theorem eq_2_7_mix (x y z : Pomega) (h0 : 0  in  z) (hne : z  !=  ofNat 0) :
    condSet z x y = x  U  y := by
  have hpos : exists  t, t + 1  in  z := by
    by_contra h
    apply hne
    ext k
    constructor
    * intro hk
      have : k = 0 := by
        by_contra hk0
        exact h <k - 1, by
          have : k = (k - 1) + 1 := (Nat.succ_pred_eq_of_ne_zero hk0).symm
          rwa [ <-  this]>
      simp [this, ofNat]
    * intro hk
      simp [ofNat] at hk
      subst hk
      exact h0
  ext n
  constructor
  * intro hn
    rcases hn with <hnx, _> | <hny, _>
    * exact Or.inl hnx
    * exact Or.inr hny
  * intro hn
    rcases hn with hnx | hny
    * exact Or.inl <hnx, h0>
    * exact Or.inr <hny, hpos>

end DomainTheory.DataTypesAsLattices
```

eq_2_7_pos.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.7), remaining cases.** -/
theorem eq_2_7_pos (x y z : Pomega) (h0 : 0  notin  z) (hne : z  !=  botElem) :
    condSet z x y = y := by
  have hpos : exists  t, t + 1  in  z := by
    obtain <k, hk> := Set.nonempty_iff_ne_empty.mpr hne
    have : k  !=  0 := fun h => h0 (h  |>  hk)
    exact <k - 1, by
      have : k = (k - 1) + 1 := (Nat.succ_pred_eq_of_ne_zero this).symm
      rwa [ <-  this]>
  ext n
```

```
  constructor
  * intro hn
    rcases hn with <_, hz> | <hny, _>
    * exact (h0 hz).elim
    * exact hny
  * intro hny
    exact Or.inr <hny, hpos>

end DomainTheory.DataTypesAsLattices
```

eq_2_7_succ.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_7_succ (x y : Pomega) (k : Nat) :
    condSet (ofNat (k + 1)) x y = y := by
  ext n
  simp [condSet, ofNat]

end DomainTheory.DataTypesAsLattices
```

eq_2_7_zero.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_7_zero (x y : Pomega) : condSet (ofNat 0) x y = x := by
  ext n
  simp [condSet, ofNat]

end DomainTheory.DataTypesAsLattices
```

exists_least_mem.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

lemma exists_least_mem {a : Pomega} (hex : exists  k, k  in  a) :
    exists  k, k  in  a  /\  forall  i < k, i  notin  a := by
  have : forall  n, n  in  a  ->  exists  k  <=  n, k  in  a  /\  forall  i < k, i
    notin  a := by
    intro n
    induction n using Nat.strong_induction_on with
    | h n ih =>
      intro hn
      rcases Classical.em (forall  i < n, i  notin  a) with hmin | hmin
      * exact <n, le_rfl, hn, hmin>
      * obtain <i, hi> := not_forall.mp hmin
        have hi' : i < n := (Classical.not_imp.mp hi).1
        have hia : i  in  a := not_not.mp (Classical.not_imp.mp hi).2
        obtain <k, hki, hk, hall> := ih i hi' hia
        exact <k, hki.trans hi'.le, hk, hall>
  obtain <n, hn> := hex
  obtain <k, _, hk, hall> := this n hn
  exact <k, hk, hall>

end DomainTheory.DataTypesAsLattices
```

exists_least_not_mem.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

lemma exists_least_not_mem {a : Pomega} (hex : exists  k, k  notin  a) :
    exists  k, k  notin  a  /\  forall  i < k, i  in  a := by
  have : forall  n, n  notin  a  ->  exists  k  <=  n, k  notin  a  /\  forall  i < k,
     i  in  a := by
    intro n
    induction n using Nat.strong_induction_on with
    | h n ih =>
      intro hn
      rcases Classical.em (forall  i < n, i  in  a) with hmin | hmin
      * exact <n, le_rfl, hn, hmin>
      * obtain <i, hi> := not_forall.mp hmin
        have hi' : i < n := (Classical.not_imp.mp hi).1
        have hia : i  notin  a := (Classical.not_imp.mp hi).2
        obtain <k, hki, hk, hall> := ih i hi' hia
        exact <k, hki.trans hi'.le, hk, hall>
  obtain <n, hn> := hex
  obtain <k, _, hk, hall> := this n hn
  exact <k, hk, hall>
```

```
end DomainTheory.DataTypesAsLattices
```

Scomb_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace Scomb

theorem inner_cont (u v : Pomega) :
    IsScottContinuous (fun x => funOf (funOf u x) (funOf v x)) :=
  theorem_1_3.tuple
    (fun y => funOf.isScottContinuous_left y)
    (fun t => funOf.isScottContinuous t)
    (funOf.isScottContinuous u)
    (funOf.isScottContinuous v)

theorem map_cont :
    IsScottContinuous (fun u => graph (fun v =>
      graph (fun x => funOf (funOf u x) (funOf v x)))) :=
  graph_const_isScottContinuous
    (fun u v => graph (fun x => funOf (funOf u x) (funOf v x)))
    (fun v => graph_const_isScottContinuous
      (fun u x => funOf (funOf u x) (funOf v x))
      (fun x => theorem_1_3
        (f := fun t => funOf t (funOf v x))
        (g := fun u => funOf u x)
        (funOf.isScottContinuous_left (funOf v x))
        (funOf.isScottContinuous_left x)))

theorem mid_cont (u : Pomega) :
    IsScottContinuous (fun v =>
      graph (fun x => funOf (funOf u x) (funOf v x))) :=
  graph_const_isScottContinuous
    (fun v x => funOf (funOf u x) (funOf v x))
    (fun x => theorem_1_3 (funOf.isScottContinuous (funOf u x))
      (funOf.isScottContinuous_left x))
```

```
theorem beta (f : Pomega) :
    funOf Scomb f =
      graph (fun v => graph (fun x => funOf (funOf f x) (funOf v x))) :=
  _root_.DomainTheory.DataTypesAsLattices.beta Scomb.map_cont f

theorem beta2 (f g : Pomega) :
    funOf (funOf Scomb f) g =
      graph (fun x => funOf (funOf f x) (funOf g x)) := by
  rw [Scomb.beta]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (Scomb.mid_cont f) g

theorem beta3 (f g x : Pomega) :
    funOf (funOf (funOf Scomb f) g) x = funOf (funOf f x) (funOf g x) := by
  rw [Scomb.beta2]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (Scomb.inner_cont f g) x

end Scomb

end DomainTheory.DataTypesAsLattices
```

graph_const_isScottContinuous.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Continuous.scottUnion
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- `graph (fun y => g x y)` is continuous in `x` when `g` is continuous in `x`. -/
theorem graph_const_isScottContinuous
    (g : Pomega  ->  Pomega  ->  Pomega)
    (hg : forall  y, IsScottContinuous (fun x => g x y)) :
    IsScottContinuous (fun x => graph (fun y => g x y)) := by
  constructor
  intro x
  ext k
  constructor
  * intro hk
    rcases hk with <n, m, rfl, hm>
    have : m  in  scottUnion (fun z => g z (e n)) x := by
      rw [ <-  (hg (e n)).out x]; exact hm
    obtain <p, hp, hmp> := mem_scottUnion.mp this
    exact mem_scottUnion.mpr <p, hp, n, m, rfl, hmp>
  * intro hk
    obtain <p, hp, n, m, rfl, hm> := mem_scottUnion.mp hk
    exact <n, m, rfl, IsScottContinuous.monotone (hg (e n)) hp hm>

end DomainTheory.DataTypesAsLattices
```

condC_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.condC
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace condC

theorem map_cont :
    IsScottContinuous (fun x => graph (fun y => graph (fun z => condSet z x y))) :=
  graph_const_isScottContinuous
    (fun x y => graph (fun z => condSet z x y))
    (fun y => graph_const_isScottContinuous (fun x z => condSet z x y)
      (fun z => condSet.isScottContinuous_mid z y))

theorem beta (x : Pomega) :
    funOf condC x = graph (fun y => graph (fun z => condSet z x y)) :=
  _root_.DomainTheory.DataTypesAsLattices.beta condC.map_cont x

theorem beta2 (x y : Pomega) :
    funOf (funOf condC x) y = graph (fun z => condSet z x y) := by
  rw [condC.beta]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (graph_const_isScottContinuous (
    fun y z => condSet z x y)
    (fun z => condSet.isScottContinuous_right z x)) y

theorem beta3 (x y z : Pomega) :
    funOf (funOf (funOf condC x) y) z = condSet z x y := by
  rw [condC.beta2]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (condSet.isScottContinuous_left x
     y) z

end condC

end DomainTheory.DataTypesAsLattices
```

curry2.lean

```
module
```

```
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
def curry2 (f : Pomega  ->  Pomega  ->  Pomega)
    (_hfx : forall  y, IsScottContinuous (fun x => f x y))
    (_hfy : forall  x, IsScottContinuous (fun y => f x y)) : Pomega :=
  graph (fun x => graph (fun y => f x y))

namespace curry2

theorem app (f : Pomega  ->  Pomega  ->  Pomega)
    (hfx : forall  y, IsScottContinuous (fun x => f x y))
    (hfy : forall  x, IsScottContinuous (fun y => f x y)) (x y : Pomega) :
    funOf (funOf (curry2 f hfx hfy) x) y = f x y := by
  have h1 : funOf (curry2 f hfx hfy) x = graph (fun y => f x y) :=
    beta (graph_const_isScottContinuous f hfx) x
  rw [h1]
  exact beta (hfy x) y

end curry2

end DomainTheory.DataTypesAsLattices
```

curry3.lean

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
def curry3 (f : Pomega  ->  Pomega  ->  Pomega  ->  Pomega)
    (_hf0 : forall  y z, IsScottContinuous (fun x => f x y z))
    (_hf1 : forall  x z, IsScottContinuous (fun y => f x y z))
```

```
    (_hf2 : forall  x y, IsScottContinuous (fun z => f x y z)) : Pomega :=
  graph (fun x => graph (fun y => graph (fun z => f x y z)))

namespace curry3

theorem app (f : Pomega  ->  Pomega  ->  Pomega  ->  Pomega)
    (hf0 : forall  y z, IsScottContinuous (fun x => f x y z))
    (hf1 : forall  x z, IsScottContinuous (fun y => f x y z))
    (hf2 : forall  x y, IsScottContinuous (fun z => f x y z)) (x y z : Pomega) :
    funOf (funOf (funOf (curry3 f hf0 hf1 hf2) x) y) z = f x y z := by
  have h1 : funOf (curry3 f hf0 hf1 hf2) x =
      graph (fun y => graph (fun z => f x y z)) :=
    beta (graph_const_isScottContinuous
      (fun x y => graph (fun z => f x y z))
      (fun y => graph_const_isScottContinuous (fun x z => f x y z)
        (fun z => hf0 y z))) x
  have h2 : funOf (funOf (curry3 f hf0 hf1 hf2) x) y =
      graph (fun z => f x y z) := by
    rw [h1]
    exact beta (graph_const_isScottContinuous (fun y z => f x y z)
      (fun z => hf1 x z)) y
  rw [h2]
  exact beta (hf2 x y) z

end curry3

end DomainTheory.DataTypesAsLattices
```

curryFin.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.ScottContinuousFin
public import DomainTheory.DataTypesAsLattices.Lambda.nestApplyFin
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.beta

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
/-- Nested-graph representative of an n-ary continuous map. -/
def curryFin : forall  {n : Nat}, ((Fin n  ->  Pomega)  ->  Pomega)  ->  Pomega
  | 0, f => f Fin.elim0
  | _n + 1, f => graph (fun x => curryFin (fun xs => f (Fin.cons x xs)))

namespace curryFin
```

```
theorem isScottContinuous {n : Nat} :
    forall  {f : (Fin (n + 1)  ->  Pomega)  ->  Pomega},
      ScottContinuousFin f  ->
      IsScottContinuous (fun x => curryFin (fun xs : Fin n  ->  Pomega => f (Fin.cons
    x xs))) := by
  induction n with
  | zero =>
    intro f hf
    exact hf.out.1 Fin.elim0
  | succ n ih =>
    intro f hf
    refine graph_const_isScottContinuous
      (fun x y => curryFin (fun zs : Fin n  ->  Pomega =>
        f (Fin.cons x (Fin.cons y zs)))) (fun y =>
      ih (f := fun xs : Fin (n + 1)  ->  Pomega =>
          f (Fin.cons (xs 0) (Fin.cons y (Fin.tail xs)))) <<?_, ?_>>)
    * intro zs
      simpa [Fin.cons_zero, Fin.cons_succ] using hf.out.1 (Fin.cons y zs)
    * intro x
      simpa [Fin.cons_zero, Fin.cons_succ] using (hf.out.2 x).2 y

theorem app {n : Nat} {f : (Fin n  ->  Pomega)  ->  Pomega}
    (hf : ScottContinuousFin f) (xs : Fin n  ->  Pomega) :
    nestApplyFin (curryFin f) xs = f xs := by
  induction n with
  | zero =>
    change curryFin f = f xs
    exact congrArg f (funext fun i => nomatch i)
  | succ n ih =>
    have hb : funOf (curryFin f) (xs 0) =
        curryFin (fun ys => f (Fin.cons (xs 0) ys)) :=
      beta (curryFin.isScottContinuous hf) (xs 0)
    change nestApplyFin (funOf (curryFin f) (xs 0)) (Fin.tail xs) = f xs
    rw [hb, ih <hf.out.2 (xs 0)> (Fin.tail xs)]
    exact congrArg f (Fin.cons_self_tail xs)

end curryFin

end DomainTheory.DataTypesAsLattices
```

id_isScottContinuous.lean

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
```

```
theorem id_isScottContinuous : IsScottContinuous (fun x : Pomega => x) := by
  constructor
  intro x
  ext k
  constructor
  * intro hk
    exact mem_scottUnion.mpr <2 ^ k, by simp [e_pow2, Set.singleton_subset_iff, hk],
      by simp [e_pow2]>
  * intro hk
    obtain <n, hn, hkn> := mem_scottUnion.mp hk
    exact hn hkn

end DomainTheory.DataTypesAsLattices
```

Kcomb_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.id_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.const_isScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace Kcomb

theorem beta (x : Pomega) :
    funOf Kcomb x = graph (fun _ => x) :=
  _root_.DomainTheory.DataTypesAsLattices.beta
    (graph_const_isScottContinuous (fun x _ => x)
      (fun _ => id_isScottContinuous)) x

theorem beta2 (x y : Pomega) : funOf (funOf Kcomb x) y = x := by
  rw [Kcomb.beta]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (const_isScottContinuous x) y

end Kcomb

end DomainTheory.DataTypesAsLattices
```

SKK_eq_I.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
```

```
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb_more
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb_more
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem SKK_eq_I : funOf (funOf Scomb Kcomb) Kcomb = graph (fun x => x) := by
  rw [Scomb.beta2]
  apply congrArg graph
  funext x
  simp [Kcomb.beta2]

end DomainTheory.DataTypesAsLattices
```

dcondSet_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.id_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Lambda.predSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace dcondSet

theorem isScottContinuous_left (x y : Pomega) :
    IsScottContinuous (fun z => dcondSet z x y) :=
  theorem_1_3.nary
    (fun y z => condSet.isScottContinuous_left y z)
    (fun x z => condSet.isScottContinuous_mid x z)
    (fun x y => condSet.isScottContinuous_right x y)
    id_isScottContinuous
    (condSet.isScottContinuous_left x topElem)
    (condSet.isScottContinuous_left topElem y)

theorem bot (x y : Pomega) : dcondSet botElem x y = botElem := by
  ext n
  simp [dcondSet, condSet, botElem]
```

```
theorem cases (z : Pomega) :
    (0  in  z  /\  not  exists  k, k + 1  in  z)  \/
      (0  notin  z  /\  exists  k, k + 1  in  z)  \/
        (0  in  z  /\  exists  k, k + 1  in  z)  \/
          (0  notin  z  /\  not  exists  k, k + 1  in  z) := by
  by_cases h0 : 0  in  z
  * by_cases hp : exists  k, k + 1  in  z
    * exact Or.inr (Or.inr (Or.inl <h0, hp>))
    * exact Or.inl <h0, hp>
  * by_cases hp : exists  k, k + 1  in  z
    * exact Or.inr (Or.inl <h0, hp>)
    * exact Or.inr (Or.inr (Or.inr <h0, hp>))

theorem isScottContinuous_mid (z y : Pomega) :
    IsScottContinuous (fun x => dcondSet z x y) :=
  theorem_1_3 (f := fun w => condSet z w (condSet z topElem y))
    (g := fun x => condSet z x topElem)
    (condSet.isScottContinuous_mid z (condSet z topElem y))
    (condSet.isScottContinuous_mid z topElem)

theorem isScottContinuous_right (z x : Pomega) :
    IsScottContinuous (fun y => dcondSet z x y) :=
  theorem_1_3 (f := fun w => condSet z (condSet z x topElem) w)
    (g := fun y => condSet z topElem y)
    (condSet.isScottContinuous_right z (condSet z x topElem))
    (condSet.isScottContinuous_right z topElem)

theorem of_empty {z x y : Pomega}
    (h0 : 0  notin  z) (hp : not  exists  k, k + 1  in  z) :
    dcondSet z x y = botElem := by
  ext n
  constructor
  * intro hn
    rcases hn with <_, hz> | <_, ht>
    * exact (h0 hz).elim
    * exact (hp ht).elim
  * intro hn
    simp [botElem] at hn

theorem of_mixed {z x y : Pomega}
    (h0 : 0  in  z) (hp : exists  k, k + 1  in  z) :
    dcondSet z x y = topElem := by
  have hL : condSet z x topElem = topElem := by
    ext n
    constructor
    * intro _; exact Set.mem_univ n
    * intro _; exact Or.inr <Set.mem_univ n, hp>
  have hR : condSet z topElem y = topElem := by
    ext n
    constructor
    * intro _; exact Set.mem_univ n
    * intro _; exact Or.inl <Set.mem_univ n, h0>
  rw [dcondSet, hL, hR]
  ext n
  constructor
  * intro _; exact Set.mem_univ n
```

```
  * intro _; exact Or.inl <Set.mem_univ n, h0>

theorem of_pos_not_zero {z x y : Pomega}
    (h0 : 0  notin  z) (hp : exists  k, k + 1  in  z) :
    dcondSet z x y = y := by
  have hL : condSet z x topElem = topElem := by
    ext n
    constructor
    * intro _; exact Set.mem_univ n
    * intro _; exact Or.inr <Set.mem_univ n, hp>
  have hR : condSet z topElem y = y := by
    ext n
    constructor
    * intro hn
      rcases hn with <_, hz> | <hy, _>
      * exact (h0 hz).elim
      * exact hy
    * intro hy
      exact Or.inr <hy, hp>
  rw [dcondSet, hL, hR]
  ext n
  constructor
  * intro hn
    rcases hn with <_, hz> | <hy, _>
    * exact (h0 hz).elim
    * exact hy
  * intro hy
    exact Or.inr <hy, hp>

theorem of_zero_not_pos {z x y : Pomega}
    (h0 : 0  in  z) (hp : not  exists  k, k + 1  in  z) :
    dcondSet z x y = x := by
  have hL : condSet z x topElem = x := by
    ext n
    constructor
    * intro hn
      rcases hn with <hx, _> | <_, ht>
      * exact hx
      * exact (hp ht).elim
    * intro hx
      exact Or.inl <hx, h0>
  have hR : condSet z topElem y = topElem := by
    ext n
    constructor
    * intro _; exact Set.mem_univ n
    * intro _; exact Or.inl <Set.mem_univ n, h0>
  rw [dcondSet, hL, hR]
  ext n
  constructor
  * intro hn
    rcases hn with <hx, _> | <_, ht>
    * exact hx
    * exact (hp ht).elim
  * intro hx
    exact Or.inl <hx, h0>
```

```
theorem ofNat_one (x y : Pomega) : dcondSet (ofNat 1) x y = y := by
  ext n
  simp [dcondSet, condSet, ofNat, topElem]

theorem ofNat_succ (n : Nat) (x y : Pomega) :
    dcondSet (ofNat (n + 1)) x y = y :=
  dcondSet.of_pos_not_zero (by simp [ofNat]) <n, by simp [ofNat]>

theorem ofNat_zero (x y : Pomega) : dcondSet (ofNat 0) x y = x := by
  ext n
  simp [dcondSet, condSet, ofNat, topElem]

theorem top (x y : Pomega) : dcondSet topElem x y = topElem := by
  ext n
  simp [dcondSet, condSet, topElem]

end dcondSet

end DomainTheory.DataTypesAsLattices
```

ofComb.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.Comb
public import DomainTheory.DataTypesAsLattices.Lambda.zeroC
public import DomainTheory.DataTypesAsLattices.Lambda.sucC
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Lambda.predC
public import DomainTheory.DataTypesAsLattices.Lambda.condC
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.envSet
public import DomainTheory.DataTypesAsLattices.Lambda.id_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
def ofComb : Comb  ->  (Nat  ->  Pomega)  ->  Pomega
  | .var i, ? => ? i
  | .zero, _ => zeroC
  | .suc, _ => sucC
  | .pred, _ => predC
  | .cond, _ => condC
  | .K, _ => Kcomb
```

```
  | .S, _ => Scomb
  | .app u v, ? => funOf (ofComb u ?) (ofComb v ?)

namespace ofComb

theorem update_isScottContinuous (c : Comb) (? : Nat  ->  Pomega) (i : Nat) :
    IsScottContinuous (fun v => ofComb c (envSet ? i v)) := by
  induction c generalizing ? i with
  | var j =>
    constructor
    intro x
    by_cases hj : j = i
    * subst hj; simpa [ofComb, envSet, Function.update] using id_isScottContinuous.out
     x
    * simpa [ofComb, envSet, Function.update, hj] using (const_isScottContinuous (? j)
    ).out x
  | zero => constructor; intro x; simpa [ofComb] using (const_isScottContinuous zeroC)
    .out x
  | suc => constructor; intro x; simpa [ofComb] using (const_isScottContinuous sucC).
    out x
  | pred => constructor; intro x; simpa [ofComb] using (const_isScottContinuous predC)
    .out x
  | cond => constructor; intro x; simpa [ofComb] using (const_isScottContinuous condC)
    .out x
  | K => constructor; intro x; simpa [ofComb] using (const_isScottContinuous Kcomb).
    out x
  | S => constructor; intro x; simpa [ofComb] using (const_isScottContinuous Scomb).
    out x
  | app u v iu iv =>
    exact theorem_1_3.tuple
      (fun y => funOf.isScottContinuous_left y)
      (fun t => funOf.isScottContinuous t)
      (iu ? i) (iv ? i)

theorem combinatory (c : Comb) (? : Nat  ->  Pomega)
    (h? : forall  i, IsCombinatory (? i)) :
    IsCombinatory (ofComb c ?) := by
  induction c with
  | var i => exact h? i
  | zero => exact .zero
  | suc => exact .suc
  | pred => exact .pred
  | cond => exact .cond
  | K => exact .K
  | S => exact .S
  | app _ _ iu iv => exact .app iu iv

end ofComb

end DomainTheory.DataTypesAsLattices
```

abs.lean

```
module
```

```
public import DomainTheory.DataTypesAsLattices.Lambda.Comb
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.theorem_1_2
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.envSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofComb
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Lambda.SKK_eq_I
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb_more
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def abs (i : Nat) : Comb  ->  Comb
  | .var j => if j = i then .app (.app .S .K) .K else .app .K (.var j)
  | .zero => .app .K .zero
  | .suc => .app .K .suc
  | .pred => .app .K .pred
  | .cond => .app .K .cond
  | .K => .app .K .K
  | .S => .app .K .S
  | .app u v => .app (.app .S (abs i u)) (abs i v)

namespace abs

theorem correct (i : Nat) (c : Comb) (? : Nat  ->  Pomega) :
    ofComb (abs i c) ? = graph (fun v => ofComb c (envSet ? i v)) := by
  induction c generalizing ? with
  | var j =>
    simp only [abs, ofComb]
    by_cases hj : j = i
    * rw [ite_eq_left hj]
      simp only [ofComb]
      convert SKK_eq_I
      funext v
      simp [envSet, Function.update, hj]
    * rw [ite_eq_right hj]
      simp [ofComb, Kcomb.beta, envSet, Function.update, hj]
  | zero => simp [abs, ofComb, Kcomb.beta]
  | suc => simp [abs, ofComb, Kcomb.beta]
  | pred => simp [abs, ofComb, Kcomb.beta]
  | cond => simp [abs, ofComb, Kcomb.beta]
  | K => simp [abs, ofComb, Kcomb.beta]
  | S => simp [abs, ofComb, Kcomb.beta]
  | app u v iu iv =>
    simp only [abs, ofComb]
    rw [Scomb.beta2, iu, iv]
    congr 1
    funext x
    rw [theorem_1_2.i (ofComb.update_isScottContinuous u ? i)]
    rw [theorem_1_2.i (ofComb.update_isScottContinuous v ? i)]
```

```
end abs

end DomainTheory.DataTypesAsLattices
```

ofNat.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Scott's integer convention: `n = {n}` as an element of `P?`. -/
def ofNat (n : Nat) : Pomega := {n}

end DomainTheory.DataTypesAsLattices
```

pair_one_zero.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.pair

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem pair_one_zero : pair 1 0 = 1 := by simp [pair]

end DomainTheory.DataTypesAsLattices
```

pair_zero_zero.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.pair

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem pair_zero_zero : pair 0 0 = 0 := by simp [pair]
end DomainTheory.DataTypesAsLattices
```

predSet.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Table 2.** Predecessor `x ? 1 = {n | n + 1  in  x}`. -/
def predSet (x : Pomega) : Pomega :=
  {k | k + 1  in  x}

end DomainTheory.DataTypesAsLattices
```

predC.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.predSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def predC : Pomega := graph predSet

end DomainTheory.DataTypesAsLattices
```

predSet_more.lean

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace predSet

theorem isScottContinuous : IsScottContinuous predSet := by
  constructor
  intro x
  ext k
  constructor
  * intro hk
    refine mem_scottUnion.mpr <2 ^ (k + 1), ?_, ?_>
    * simpa [e_pow2, Set.singleton_subset_iff, predSet] using hk
    * simp [predSet, e_pow2]
```

```
  * intro hk
    obtain <m, hm, hkm> := mem_scottUnion.mp hk
    exact hm hkm

theorem ofNat_succ (j : Nat) : predSet (ofNat (j + 1)) = ofNat j := by
  ext k
  simp [predSet, ofNat]

end predSet

end DomainTheory.DataTypesAsLattices
```

predC_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.predC
public import DomainTheory.DataTypesAsLattices.Lambda.predSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace predC

theorem app (x : Pomega) : funOf predC x = predSet x :=
  beta predSet.isScottContinuous x

end predC

end DomainTheory.DataTypesAsLattices
```

succSet.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Table 2.** Successor `x + 1 = {n + 1 | n  in  x}`. -/
def succSet (x : Pomega) : Pomega :=
  {k | exists  n  in  x, k = n + 1}

end DomainTheory.DataTypesAsLattices
```

sucC.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.succSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def sucC : Pomega := graph succSet

end DomainTheory.DataTypesAsLattices
```

succSet_more.lean

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.succSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace succSet

theorem isScottContinuous : IsScottContinuous succSet := by
  constructor
  intro x
  ext k
  constructor
  * intro <n, hn, hk>
    refine mem_scottUnion.mpr <2 ^ n, by simp [e_pow2, Set.singleton_subset_iff, hn],
    ?_>
    exact <n, by simp [e_pow2], hk>
  * intro hk
    obtain <m, hm, n, hn, hk> := mem_scottUnion.mp hk
    exact <n, hm hn, hk>

theorem ofNat (j : Nat) : succSet (ofNat j) = ofNat (j + 1) := by
  ext k
  simp [succSet]

end succSet

end DomainTheory.DataTypesAsLattices
```

interp.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.envSet
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.succSet
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Continuous.exists_e_cover_finite
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Continuous.scottUnion
public import DomainTheory.DataTypesAsLattices.Lambda.id_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.predSet_more
public import DomainTheory.DataTypesAsLattices.Lambda.succSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Table 2.** Semantics of a LAMBDA term. -/
def interp : Term  ->  (Nat  ->  Pomega)  ->  Pomega
  | .var i, ? => ? i
  | .zero, _ => ofNat 0
  | .succ t, ? => succSet (interp t ?)
  | .pred t, ? => predSet (interp t ?)
  | .cond z x y, ? => condSet (interp z ?) (interp x ?) (interp y ?)
  | .app u x, ? => funOf (interp u ?) (interp x ?)
  | .lam i body, ? => graph (fun v => interp body (envSet ? i v))

namespace interp

/-- Appendix lemma for Theorem 2.1: abstraction preserves continuity in the other
    variables. -/
theorem var_isScottContinuous (t : Term) (? : Nat  ->  Pomega) (i : Nat) :
    IsScottContinuous (fun v => interp t (envSet ? i v)) := by
  induction t generalizing ? i with
  | var j =>
    constructor
    intro x
```

```
  by_cases hj : j = i
  * subst hj
    simpa [interp, envSet, Function.update] using id_isScottContinuous.out x
  * simpa [interp, envSet, Function.update, hj] using (const_isScottContinuous (? j)
  ).out x
| zero =>
  constructor
  intro x
  simpa [interp] using (const_isScottContinuous (ofNat 0)).out x
| succ t ih =>
  exact theorem_1_3 succSet.isScottContinuous (ih ? i)
| pred t ih =>
  exact theorem_1_3 predSet.isScottContinuous (ih ? i)
| cond z x y iz ix iy =>
  constructor
  intro w
  ext n
  simp only [interp]
  constructor
  * intro hn
    rcases hn with <hnx, h0> | <hny, t, ht>
    * have : n  in  scottUnion (fun v => interp x (envSet ? i v)) w := by
        rw [ <-  (ix ? i).out w]; exact hnx
      obtain <m, hm, hnm> := mem_scottUnion.mp this
      have : 0  in  scottUnion (fun v => interp z (envSet ? i v)) w := by
        rw [ <-  (iz ? i).out w]; exact h0
      obtain <m', hm', h0m> := mem_scottUnion.mp this
      refine mem_scottUnion.mpr <m ||| m', e_or_of_subset hm hm', Or.inl <?_, ?_>>
      * exact IsScottContinuous.monotone (ix ? i)
          (by rw [e_or]; exact Set.subset_union_left) hnm
      * exact IsScottContinuous.monotone (iz ? i)
          (by rw [e_or]; exact Set.subset_union_right) h0m
    * have : n  in  scottUnion (fun v => interp y (envSet ? i v)) w := by
        rw [ <-  (iy ? i).out w]; exact hny
      obtain <m, hm, hnm> := mem_scottUnion.mp this
      have : t + 1  in  scottUnion (fun v => interp z (envSet ? i v)) w := by
        rw [ <-  (iz ? i).out w]; exact ht
      obtain <m', hm', htm> := mem_scottUnion.mp this
      refine mem_scottUnion.mpr <m ||| m', e_or_of_subset hm hm', Or.inr <?_, t, ?_
 >>
      * exact IsScottContinuous.monotone (iy ? i)
          (by rw [e_or]; exact Set.subset_union_left) hnm
      * exact IsScottContinuous.monotone (iz ? i)
          (by rw [e_or]; exact Set.subset_union_right) htm
  * intro hn
    obtain <m, hm, hnm> := mem_scottUnion.mp hn
    rcases hnm with <hnx, h0> | <hny, t, ht>
    * exact Or.inl <IsScottContinuous.monotone (ix ? i) hm hnx,
        IsScottContinuous.monotone (iz ? i) hm h0>
    * exact Or.inr <IsScottContinuous.monotone (iy ? i) hm hny, t,
        IsScottContinuous.monotone (iz ? i) hm ht>
| app u x iu ix =>
  constructor
  intro w
  ext n
  simp only [interp]
```

```
    constructor
    * intro <p, hp, hpair>
      have : pair p n  in  scottUnion (fun v => interp u (envSet ? i v)) w := by
        rw [ <-  (iu ? i).out w]; exact hpair
      obtain <m, hm, hpm> := mem_scottUnion.mp this
      obtain <s, hs> := (e_finite p).exists_finset_coe
      have hall : forall  t  in  s, exists  q, e q  subseteq  w  /\  t  in  interp x (
    envSet ? i (e q)) := by
        intro t ht
        have ht' : t  in  e p := by
          have : t  in  (s : Set Nat) := ht
          rwa [hs] at this
        have : t  in  interp x (envSet ? i w) := hp ht'
        have : t  in  scottUnion (fun v => interp x (envSet ? i v)) w := by
          rwa [ <-  (ix ? i).out w]
        exact mem_scottUnion.mp this
      obtain <N, hN, hsN> := exists_e_cover_finite (ix ? i) s hall
      refine mem_scottUnion.mpr <m ||| N, e_or_of_subset hm hN, p, ?_, ?_>
      * have : e p  subseteq  interp x (envSet ? i (e (m ||| N))) := by
          rw [ <-  hs]
          intro t ht
          exact IsScottContinuous.monotone (ix ? i)
            (by rw [e_or]; exact Set.subset_union_right) (hsN ht)
        exact this
      * exact IsScottContinuous.monotone (iu ? i)
          (by rw [e_or]; exact Set.subset_union_left) hpm
    * intro hn
      obtain <m, hm, p, hp, hpair> := mem_scottUnion.mp hn
      exact <p, subset_trans hp (IsScottContinuous.monotone (ix ? i) hm),
        IsScottContinuous.monotone (iu ? i) hm hpair>
  | lam j body ih =>
    by_cases hj : j = i
    * subst hj
      constructor
      intro w
      simpa [interp, envSet, Function.update] using
        (const_isScottContinuous (graph (fun v => interp body (envSet ? j v)))).out w
    * have hmap : IsScottContinuous (fun w =>
          graph (fun v => interp body (envSet (envSet ? i w) j v))) := by
        refine graph_const_isScottContinuous
          (fun w v => interp body (envSet (envSet ? i w) j v)) ?_
        intro v
        have : (fun w => interp body (envSet (envSet ? i w) j v)) =
            (fun w => interp body (envSet (envSet ? j v) i w)) := by
          funext w; rw [envSet.comm hj]
        rw [this]
        exact ih (envSet ? j v) i
      constructor
      intro w
      simpa [interp] using hmap.out w

end interp

end DomainTheory.DataTypesAsLattices
```

sucC_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.sucC
public import DomainTheory.DataTypesAsLattices.Lambda.succSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace sucC

theorem app (x : Pomega) : funOf sucC x = succSet x :=
  beta succSet.isScottContinuous x

end sucC

end DomainTheory.DataTypesAsLattices
```

erase.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Lambda.Comb
public import DomainTheory.DataTypesAsLattices.Lambda.abs
public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.envSet
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Lambda.ofComb
public import DomainTheory.DataTypesAsLattices.Lambda.sucC
public import DomainTheory.DataTypesAsLattices.Lambda.predC
public import DomainTheory.DataTypesAsLattices.Lambda.condC
public import DomainTheory.DataTypesAsLattices.Lambda.zeroC
public import DomainTheory.DataTypesAsLattices.Lambda.condC_more
public import DomainTheory.DataTypesAsLattices.Lambda.predC_more
public import DomainTheory.DataTypesAsLattices.Lambda.sucC_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def erase : Term  ->  Comb
  | .var i => .var i
  | .zero => .zero
  | .succ t => .app .suc (erase t)
  | .pred t => .app .pred (erase t)
  | .cond z x y => .app (.app (.app .cond (erase x)) (erase y)) (erase z)
  | .app u x => .app (erase u) (erase x)
```

```
  | .lam i body => abs i (erase body)

namespace erase

theorem correct (t : Term) (? : Nat  ->  Pomega) :
    ofComb (erase t) ? = interp t ? := by
  induction t generalizing ? with
  | var i => simp [erase, ofComb, interp]
  | zero => simp [erase, ofComb, interp, zeroC]
  | succ t ih => simp [erase, ofComb, interp, sucC.app, ih]
  | pred t ih => simp [erase, ofComb, interp, predC.app, ih]
  | cond z x y iz ix iy =>
    simp [erase, ofComb, interp, condC.beta3, iz, ix, iy]
  | app u x iu ix => simp [erase, ofComb, interp, iu, ix]
  | lam i body ih =>
    simp only [erase, interp]
    rw [abs.correct]
    congr 1
    funext v
    exact ih (envSet ? i v)

end erase

end DomainTheory.DataTypesAsLattices
```

ofNat_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.zeroC
public import DomainTheory.DataTypesAsLattices.Lambda.sucC_more
public import DomainTheory.DataTypesAsLattices.Lambda.succSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace ofNat

theorem combinatory : forall  n, IsCombinatory (ofNat n)
  | 0 => by
    simpa [zeroC] using IsCombinatory.zero
  | n + 1 => by
    have h := IsCombinatory.app IsCombinatory.suc (combinatory n)
    simpa [sucC.app, succSet.ofNat] using h

end ofNat

end DomainTheory.DataTypesAsLattices
```

termShift.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Lambda.seqShift
public import DomainTheory.DataTypesAsLattices.Lambda.succSet
public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Lambda.succSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Term-level shift `fun t. u(t+1)`, which agrees with `seqShift` on numerals. -/
def termShift (u : Pomega) : Pomega :=
  graph (fun t => funOf u (succSet t))

namespace termShift

theorem isScottContinuous (u : Pomega) :
    IsScottContinuous (fun t => funOf u (succSet t)) :=
  theorem_1_3 (funOf.isScottContinuous u) succSet.isScottContinuous

theorem app (u t : Pomega) :
    funOf (termShift u) t = funOf u (succSet t) :=
  _root_.DomainTheory.DataTypesAsLattices.beta (termShift.isScottContinuous u) t

theorem left_isScottContinuous : IsScottContinuous termShift :=
  graph_const_isScottContinuous (fun u t => funOf u (succSet t))
    (fun t => funOf.isScottContinuous_left (succSet t))

theorem ofNat (u : Pomega) (i : Nat) :
    funOf (termShift u) (ofNat i) = funOf u (ofNat (i + 1)) := by
  rw [termShift.app, succSet.ofNat]

end termShift

end DomainTheory.DataTypesAsLattices
```

triangle_eq_zero.lean

```
module
```

```
public import DomainTheory.DataTypesAsLattices.Pomega.pair

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem triangle_eq_zero {w : Nat} (h : triangle w = 0) : w = 0 := by
  match w with
  | 0 => rfl
  | n + 1 =>
    have h1 : 1  <=  n + 1 := Nat.succ_le_succ (Nat.zero_le _)
    have h2 : 2  <=  (n + 1) * (n + 2) :=
      calc 2  <=  1 * 2 := by decide
           _  <=  (n + 1) * (n + 2) := Nat.mul_le_mul h1 (Nat.succ_le_succ h1)
    have : 0 < (n + 1) * (n + 2) / 2 := Nat.div_pos h2 (by decide)
    exact (Nat.ne_of_gt this h).elim

end DomainTheory.DataTypesAsLattices
```

pair_eq_right.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Lambda.triangle_eq_zero

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem pair_eq_right {n m : Nat} (h : pair n m = m) : n = 0  /\  m = 0 := by
  have : triangle (n + m) = 0 := by
    have h' : triangle (n + m) + m = m := by simpa [pair_eq_triangle] using h
    omega
  have : n + m = 0 := triangle_eq_zero this
  omega

end DomainTheory.DataTypesAsLattices
```

zeroC.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def zeroC : Pomega := ofNat 0

end DomainTheory.DataTypesAsLattices
```

IsCombinatory.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.zeroC
public import DomainTheory.DataTypesAsLattices.Lambda.sucC
public import DomainTheory.DataTypesAsLattices.Lambda.predC
public import DomainTheory.DataTypesAsLattices.Lambda.condC
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Combinatory closure of the six constants of Theorem 2.4. -/
inductive IsCombinatory : Pomega  ->  Prop
  | zero : IsCombinatory zeroC
  | suc : IsCombinatory sucC
  | pred : IsCombinatory predC
  | cond : IsCombinatory condC
  | K : IsCombinatory Kcomb
  | S : IsCombinatory Scomb
  | app {u v} : IsCombinatory u  ->  IsCombinatory v  ->  IsCombinatory (funOf u v)

end DomainTheory.DataTypesAsLattices
```

mem_ofNat.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

@[simp] theorem mem_ofNat {n k : Nat} : k  in  ofNat n  <->  k = n := by simp [ofNat]

end DomainTheory.DataTypesAsLattices
```

Fofg.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section
```

```
namespace DomainTheory.DataTypesAsLattices

def Fofg (g : Pomega  ->  Pomega  ->  Pomega) (f : Pomega) : Pomega :=
  graph (fun x => g x (funOf f x))

end DomainTheory.DataTypesAsLattices
```

ScottContinuousFin.lean

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- The recursive content of finite Scott continuity, one argument at a time. -/
def ScottContinuousFin.Body : forall  {n : Nat}, ((Fin n  ->  Pomega)  ->  Pomega)  ->
      Prop
  | 0, _ => True
  | n + 1, f =>
      (forall  xs : Fin n  ->  Pomega, IsScottContinuous (fun x => f (Fin.cons x xs)))
      /\
      (forall  x : Pomega, ScottContinuousFin.Body (fun xs : Fin n  ->  Pomega => f (
    Fin.cons x xs)))

/-- Continuity of an n-ary map, taken one argument at a time from the left. -/
class ScottContinuousFin {n : Nat} (f : (Fin n  ->  Pomega)  ->  Pomega) : Prop where
  out : ScottContinuousFin.Body f

end DomainTheory.DataTypesAsLattices
```

Occurs.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Lambda.Term

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- `i` occurs anywhere in `t` (free or bound). Freshness for (a). -/
def Occurs (i : Nat) : Term  ->  Prop
  | .var j => j = i
  | .zero => False
```

```
  | .succ t => Occurs i t
  | .pred t => Occurs i t
  | .cond z x y => Occurs i z  \/  Occurs i x  \/  Occurs i y
  | .app u x => Occurs i u  \/  Occurs i x
  | .lam k body => k = i  \/  Occurs i body

end DomainTheory.DataTypesAsLattices
```

nestApplyFin.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.graph

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Iterated application of a nested-graph representative. -/
def nestApplyFin (u : Pomega) : forall  {n : Nat}, (Fin n  ->  Pomega)  ->  Pomega
  | 0, _ => u
  | _n + 1, xs => nestApplyFin (funOf u (xs 0)) (Fin.tail xs)

end DomainTheory.DataTypesAsLattices
```

rename.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Rename free `i` to a completely fresh `j`. -/
def rename (i j : Nat) : Term  ->  Term
  | .var k => if k = i then .var j else .var k
  | .zero => .zero
  | .succ t => .succ (rename i j t)
  | .pred t => .pred (rename i j t)
  | .cond z x y => .cond (rename i j z) (rename i j x) (rename i j y)
  | .app u x => .app (rename i j u) (rename i j x)
  | .lam k body => if k = i then .lam k body else .lam k (rename i j body)

end DomainTheory.DataTypesAsLattices
```

2.3.4 Theorem 2.1

THEOREM 2.1 (The continuity theorem). *All LAMBDA definable functions are continuous.*

theorem_2_1.lean

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Lambda.envSet
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 2.1 (The continuity theorem).** -/
theorem theorem_2_1 (t : Term) (? : Nat  ->  Pomega) (i : Nat) :
    IsScottContinuous (fun v => interp t (envSet ? i v)) :=
  interp.var_isScottContinuous t ? i

end DomainTheory.DataTypesAsLattices
```

2.3.5 Theorem 2.2

Once that is proved, we can use Theorem 1.2 to establish:

THEOREM 2.2 (The conversion theorem). *The three basic principles* (α), (β), (ξ) *of* λ*-conversion are all valid in the model.*

primRecStepTerm.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.envSet
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Lambda.theorem_2_2
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.predSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
```

```
/-- Combinatory primitive-recursion step `fun a fun f fun u fun n. n ? a, f(n?1)(u(n?1)
    )`. -/
def primRecStepTerm : Term :=
  .lam 0 (.lam 1 (.lam 2 (.lam 3
    (.cond (.var 3) (.var 0)
      (.app (.app (.var 1) (.pred (.var 3)))
        (.app (.var 2) (.pred (.var 3))))))))

namespace primRecStepTerm

theorem interp (a f u n : Pomega) :
    funOf (funOf (funOf (funOf (interp primRecStepTerm (fun _ => botElem)) a) f) u) n
    =
      condSet n a (funOf (funOf f (predSet n)) (funOf u (predSet n))) := by
  change funOf (funOf (funOf (funOf (DomainTheory.DataTypesAsLattices.interp (Term.lam
    0 _) (fun _ => botElem)) a) f) u) n = _
  rw [theorem_2_2.beta]
  rw [theorem_2_2.beta]
  rw [theorem_2_2.beta]
  rw [theorem_2_2.beta]
  simp [_root_.DomainTheory.DataTypesAsLattices.interp, envSet, Function.update]

end primRecStepTerm

end DomainTheory.DataTypesAsLattices
```

primRecStep.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.envSet
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Lambda.primRecStepTerm
public import DomainTheory.DataTypesAsLattices.Lambda.id_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.theorem_2_2
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Lambda.predSet_more

@[expose] public section
```

```
namespace DomainTheory.DataTypesAsLattices

def primRecStep (a f u : Pomega) : Pomega :=
  graph (fun n => condSet n a (funOf (funOf f (predSet n)) (funOf u (predSet n))))

namespace primRecStep

theorem body_isScottContinuous_n (a f u : Pomega) :
    IsScottContinuous (fun n =>
      condSet n a (funOf (funOf f (predSet n)) (funOf u (predSet n)))) :=
  theorem_1_3.tuple
    (fun y => condSet.isScottContinuous_left a y)
    (fun n => condSet.isScottContinuous_right n a)
    id_isScottContinuous
    (theorem_1_3.tuple
      (fun y => funOf.isScottContinuous_left y)
      (fun x => funOf.isScottContinuous x)
      (theorem_1_3 (funOf.isScottContinuous f) predSet.isScottContinuous)
      (theorem_1_3 (funOf.isScottContinuous u) predSet.isScottContinuous))

theorem graph_of (a f : Pomega) :
    graph (primRecStep a f) =
      funOf (funOf (interp primRecStepTerm (fun _ => botElem)) a) f := by
  change graph (primRecStep a f) =
    funOf (funOf (DomainTheory.DataTypesAsLattices.interp (Term.lam 0 (.lam 1 (.lam 2
    (.lam 3
      (.cond (.var 3) (.var 0)
        (.app (.app (.var 1) (.pred (.var 3)))
          (.app (.var 2) (.pred (.var 3))))))))) (fun _ => botElem)) a) f
  rw [theorem_2_2.beta, theorem_2_2.beta]
  apply congrArg graph
  funext u
  apply congrArg graph
  funext n
  simp [DomainTheory.DataTypesAsLattices.interp, envSet, Function.update]

theorem isScottContinuous (a f : Pomega) :
    IsScottContinuous (primRecStep a f) :=
  graph_const_isScottContinuous
    (fun u n => condSet n a (funOf (funOf f (predSet n)) (funOf u (predSet n))))
    (fun n => theorem_1_3 (condSet.isScottContinuous_right n a)
      (theorem_1_3 (funOf.isScottContinuous (funOf f (predSet n)))
        (funOf.isScottContinuous_left (predSet n))))

theorem app (a f u n : Pomega) :
    funOf (primRecStep a f u) n =
      condSet n a (funOf (funOf f (predSet n)) (funOf u (predSet n))) :=
  beta (primRecStep.body_isScottContinuous_n a f u) n

end primRecStep

end DomainTheory.DataTypesAsLattices
```

theorem_2_2.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Lambda.envSet
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Lambda.rename

public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids
public import DomainTheory.DataTypesAsLattices.Lambda.theorem_2_1

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
/-- **Scott 1976, Table 1 (a).** Bound variables may be renamed when the
bodies agree after the rename (Scott: "`x` is a bound variable"). -/
theorem theorem_2_2.alpha (i j : Nat) (body : Term) (? : Nat  ->  Pomega)
    (h : forall  v, interp body (envSet ? i v) =
            interp (rename i j body) (envSet ? j v)) :
    interp (.lam i body) ? = interp (.lam j (rename i j body)) ? := by
  unfold interp
  exact congrArg graph (funext h)

/-- **Scott 1976, Theorem 2.2 (?).** -/
theorem theorem_2_2.xi {i : Nat} {tau sigma : Term} {? : Nat  ->  Pomega}
    (h : forall  v, interp tau (envSet ? i v) = interp sigma (envSet ? i v)) :
    interp (.lam i tau) ? = interp (.lam i sigma) ? := by
  simp [interp, h]

/-- **Scott 1976, Theorem 2.2 (b).** -/
theorem theorem_2_2.beta (i : Nat) (body : Term) (? : Nat  ->  Pomega) (y : Pomega) :
    funOf (interp (.lam i body) ?) y = interp body (envSet ? i y) :=
  DomainTheory.DataTypesAsLattices.beta (theorem_2_1 body ? i) y

/-- **Scott 1976, Theorem 2.2 (The conversion theorem).**
The three basic principles (a), (b), (?) are valid in the model. -/
theorem theorem_2_2 (i : Nat) (body : Term) (? : Nat  ->  Pomega) (y : Pomega) :
    (forall  j, (forall  v, interp body (envSet ? i v) =
            interp (rename i j body) (envSet ? j v))  ->
        interp (.lam i body) ? = interp (.lam j (rename i j body)) ?)  /\
      funOf (interp (.lam i body) ?) y = interp body (envSet ? i y)  /\
        forall  sigma, (forall  v, interp body (envSet ? i v) = interp sigma (envSet ?
     i v))  ->
          interp (.lam i body) ? = interp (.lam i sigma) ? :=
  <fun j h => theorem_2_2.alpha i j body ? h,
    theorem_2_2.beta i body ? y,
    fun _sigma h => theorem_2_2.xi h>

end DomainTheory.DataTypesAsLattices
```

2.3.6 Theorem 2.3

By "model" here we of course understand the interpretation of the language where the semantics gives terms denotations in $\mathbf{P}\omega$ according to the stated definition. Through this interpretation, more properly speaking, $\mathbf{P}\omega$ becomes a model for the axioms (α), (β), (ξ). Two well-known results of the calculus of λ-conversion allow the reduction of functions of several variables to those of *one*, and the reduction of all the primitives to *combinators* (constants)—all this with the aid of the binary operation of application.

THEOREM 2.3 (The reduction theorem). *Any continuous function of k-variables can be written as*

$$f(x_0, x_1, \cdots, x_{k-1}) = u(x_0)(x_1)\cdots(x_{k-1}),$$

where u is a suitably chosen element of $\mathbf{P}\omega$.

theorem_2_3.lean

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.theorem_1_2
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.ScottContinuousFin
public import DomainTheory.DataTypesAsLattices.Lambda.curryFin
public import DomainTheory.DataTypesAsLattices.Lambda.nestApplyFin
public import DomainTheory.DataTypesAsLattices.Lambda.curry2
public import DomainTheory.DataTypesAsLattices.Lambda.curry3
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 2.3 (The reduction theorem).**
Any continuous function of `k` variables is represented by an element
`u  in  P?` via iterated application. -/
theorem theorem_2_3 {n : Nat} {f : (Fin n  ->  Pomega)  ->  Pomega}
    (hf : ScottContinuousFin f) :
    exists  u, forall  xs, nestApplyFin u xs = f xs :=
  <curryFin f, curryFin.app hf>

/-- **Scott 1976, Theorem 2.3, unary case.** -/
theorem theorem_2_3.unary {f : Pomega  ->  Pomega} (hf : IsScottContinuous f) :
    exists  u, forall  x, funOf u x = f x :=
  <graph f, fun x => by rw [theorem_1_2.i hf]>

/-- **Scott 1976, Theorem 2.3 (The reduction theorem), binary case.** -/
theorem theorem_2_3.binary {f : Pomega  ->  Pomega  ->  Pomega}
    (hfx : forall  y, IsScottContinuous (fun x => f x y))
    (hfy : forall  x, IsScottContinuous (fun y => f x y)) :
    exists  u, forall  x y, funOf (funOf u x) y = f x y :=
  <curry2 f hfx hfy, curry2.app f hfx hfy>
```

```
/-- **Scott 1976, Theorem 2.3, ternary case** (the usual reduction of `k`). -/
theorem theorem_2_3.ternary {f : Pomega  ->  Pomega  ->  Pomega  ->  Pomega}
    (hf0 : forall  y z, IsScottContinuous (fun x => f x y z))
    (hf1 : forall  x z, IsScottContinuous (fun y => f x y z))
    (hf2 : forall  x y, IsScottContinuous (fun z => f x y z)) :
    exists  u, forall  x y z, funOf (funOf (funOf u x) y) z = f x y z :=
  <curry3 f hf0 hf1 hf2, curry3.app f hf0 hf1 hf2>

end DomainTheory.DataTypesAsLattices
```

2.3.7 Theorem 2.4

THEOREM 2.4 (The combinator theorem). *The LAMBDA-definable functions* can be generated (*from variables*) by iterated application with the aid of these six constants:

$$\mathbf{0} = 0$$

$$\mathbf{suc} = \lambda x.\, x + 1$$

$$\mathbf{pred} = \lambda x.\, x - 1$$

$$\mathbf{cond} = \lambda x \lambda y \lambda z.\, z \supset x, y$$

$$\mathbf{K} = \lambda x \lambda y.\, x$$

$$\mathbf{S} = \lambda u \lambda v \lambda x.\, u(x)(v(x))$$

But the result that makes all this model building and combinatory logic mathematically interesting concerns the so-called *paradoxical combinator* defined by the equation:

$$(2.8) \quad \mathbf{Y} = \lambda u.\, (\lambda x.\, u(x(x)))(\lambda x.\, u(x(x))).$$

theorem_2_4.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.zeroC
public import DomainTheory.DataTypesAsLattices.Lambda.sucC
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Lambda.predC
public import DomainTheory.DataTypesAsLattices.Lambda.condC
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Lambda.ofComb
```

```
public import DomainTheory.DataTypesAsLattices.Lambda.erase
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_6
public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids
public import DomainTheory.DataTypesAsLattices.Lambda.predC_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 2.4 (The combinator theorem).** -/
theorem theorem_2_4 : IsCombinatory zeroC  /\  IsCombinatory sucC  /\  IsCombinatory
    predC  /\
    IsCombinatory condC  /\  IsCombinatory Kcomb  /\  IsCombinatory Scomb :=
  <.zero, .suc, .pred, .cond, .K, .S>

/-- **Scott 1976, Theorem 2.4 (The combinator theorem).** Every LAMBDA
term equals an applicative combination of the six constants and its
free variables. -/
theorem theorem_2_4.complete (t : Term) (? : Nat  ->  Pomega) :
    interp t ? = ofComb (erase t) ? :=
  (erase.correct t ?).symm

theorem theorem_2_4.closed (t : Term) :
    IsCombinatory (interp t (fun _ => botElem)) := by
  rw [theorem_2_4.complete]
  refine ofComb.combinatory _ _ fun _ => ?_
  have hbot : funOf predC zeroC = botElem := by
    rw [predC.app]; exact eq_2_6
  exact hbot  |>  IsCombinatory.app IsCombinatory.pred IsCombinatory.zero

end DomainTheory.DataTypesAsLattices
```

Ycomb_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.omegaComb
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Lambda.Yterm
public import DomainTheory.DataTypesAsLattices.Lambda.diagApp_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.theorem_2_4

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace Ycomb
```

```
theorem app (u : Pomega) :
    funOf Ycomb u = funOf (omegaComb u) (omegaComb u) :=
  beta (theorem_1_3 diagApp_isScottContinuous omegaComb.isScottContinuous) u

theorem combinatory : IsCombinatory Ycomb := by
  simpa [Yterm.interp] using theorem_2_4.closed Yterm

/-- **Scott 1976, (2.8) unfolding.** `Y(u) = u(Y(u))`. -/
theorem unfold (u : Pomega) :
    funOf Ycomb u = funOf u (funOf Ycomb u) := by
  calc
    funOf Ycomb u = funOf (omegaComb u) (omegaComb u) := Ycomb.app u
    _ = funOf u (funOf (omegaComb u) (omegaComb u)) :=
      omegaComb.app u (omegaComb u)
    _ = funOf u (funOf Ycomb u) := by rw [Ycomb.app]

end Ycomb

end DomainTheory.DataTypesAsLattices
```

2.3.8 The combinator Y

But the result that makes all this model building and combinatory logic mathematically interesting concerns the so-called *paradoxical combinator* defined by the equation:

$$(2.8) \quad \mathbf{Y} = \lambda u.\,(\lambda x.\,u(x(x)))(\lambda x.\,u(x(x))).$$

Yterm.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Lambda.envSet
public import DomainTheory.DataTypesAsLattices.Lambda.omegaComb
public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Closed term for `Y`. -/
def Yterm : Term :=
  .lam 0 (.app
    (.lam 1 (.app (.var 0) (.app (.var 1) (.var 1))))
    (.lam 1 (.app (.var 0) (.app (.var 1) (.var 1)))))

namespace Yterm

theorem interp :
```

```
    _root_.DomainTheory.DataTypesAsLattices.interp Yterm (fun _ => botElem) =
      Ycomb := by
  simp [Yterm, _root_.DomainTheory.DataTypesAsLattices.interp, Ycomb, omegaComb,
    envSet, Function.update]

end Yterm

end DomainTheory.DataTypesAsLattices
```

diagApp_isScottContinuous.lean

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem diagApp_isScottContinuous : IsScottContinuous (fun x => funOf x x) := by
  constructor
  intro x
  have h := theorem_1_3.diag (x := x)
    (f := fun u y => funOf u y)
    (fun y => funOf.isScottContinuous_left y)
    (fun u => funOf.isScottContinuous u)
  ext k
  constructor
  * intro hk
    have : k  in  ? n, {k | e n  subseteq  x  /\  k  in  funOf (e n) (e n)} := by rwa
    [ <-  h]
    obtain <n, hn, hkn> := Set.mem_iUnion.mp this
    exact mem_scottUnion.mpr <n, hn, hkn>
  * intro hk
    obtain <n, hn, hkn> := mem_scottUnion.mp hk
    have : k  in  ? n, {k | e n  subseteq  x  /\  k  in  funOf (e n) (e n)} :=
      Set.mem_iUnion.mpr <n, hn, hkn>
    rwa [ <-  h] at this

end DomainTheory.DataTypesAsLattices
```

omegaComb.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
```

```
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.diagApp_isScottContinuous
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
/-- **Scott 1976, (2.8).** `?(u) = fun  x. u(x(x))`. -/
def omegaComb (u : Pomega) : Pomega :=
  graph (fun x => funOf u (funOf x x))

namespace omegaComb

theorem app (u x : Pomega) :
    funOf (omegaComb u) x = funOf u (funOf x x) :=
  beta (theorem_1_3 (funOf.isScottContinuous u) diagApp_isScottContinuous) x

theorem isScottContinuous : IsScottContinuous omegaComb :=
  graph_const_isScottContinuous
    (fun u x => funOf u (funOf x x))
    (fun x => funOf.isScottContinuous_left (funOf x x))

end omegaComb

end DomainTheory.DataTypesAsLattices
```

Ycomb.lean

```
module
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.Comb
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.omegaComb

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.8).** `Y = fun  u. ?(u)(?(u))`. -/
def Ycomb : Pomega :=
  graph (fun u => funOf (omegaComb u) (omegaComb u))
```

```
end DomainTheory.DataTypesAsLattices
```

omega_first_recursion.lean

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_4
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.theorem_1_2
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.omegaComb
public import DomainTheory.DataTypesAsLattices.Graph.Basic.notation3
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Scott's core induction proving that `?(graph f)(?(graph f))` is the
least fixed point of `f`. -/
theorem omega_first_recursion {f : Pomega  ->  Pomega} (hf : IsScottContinuous f) :
    funOf (omegaComb (graph f)) (omegaComb (graph f)) = fix f := by
  let d := omegaComb (graph f)
  have hd : forall  x, funOf d x = f (funOf x x) := by
    intro x
    simp [d, omegaComb.app, theorem_1_2.i hf]
  have h? : funOf d d = f (funOf d d) := hd d
  refine subset_antisymm ?_ ((theorem_1_4 hf).2 _ h?.symm)
  intro m hm
  -- Scott's argument: reduce to `e_l  subseteq  d  ->  e_l ? e_l  subseteq  a` with `a
     = fix f`.
  let a := fix f
  have ha : f a = a := (theorem_1_4 hf).1
  have key : forall  l, e l  subseteq  d  ->  funOf (e l) (e l)  subseteq  a := by
    intro l
    induction l using Nat.strongRecOn with
    | ind l ih =>
      intro hld k hk
      obtain <n, hn, hp> := hk
      have hnlt : n < l := (pair_le_left n k).trans_lt (mem_e_lt hp)
      have hnd : e n  subseteq  d := subset_trans hn hld
      have hna : funOf (e n) (e n)  subseteq  a := ih n hnlt hnd
      -- `(n,k)  in  e_l  subseteq  d = fun x. f(x x)`, so `k  in  f(e n ? e n)`
      have : k  in  funOf d (e n) := <n, subset_rfl, hld hp>
      have : k  in  f (funOf (e n) (e n)) := by
        rwa [hd] at this
      have : k  in  f a := IsScottContinuous.monotone hf hna this
      rwa [ha] at this
```

```
  rw [funOf.diag_iUnion] at hm
  obtain <l, hl, hml> := Set.mem_iUnion.mp hm
  exact key l hl hml

end DomainTheory.DataTypesAsLattices
```

2.3.9 Theorem 2.5

THEOREM 2.5 (The first recursion theorem). *If u is the graph of a continuous function f, then $\mathbf{Y}(u) = \mathbf{fix}(f)$, the least fixed point of f.*

There are two points to note here: the fixed point is LAMBDA-definable if f is; and $\mathbf{Y}$ defines a continuous operator. The word "recursion" is attached to the theorem because fixed points are employed to solve recursion equations. It would not be correct to call the fixed-point theorem (Theorem 1.4) the recursion theorem since it only shows that fixed points exist and not how they are definable in a language. The *second* recursion theorem (in Kleene's terminology) is related, but it involves Gödel numbers as introduced in § 3.

From this point on we see no need to distinguish continuous functions from elements of $\mathbf{P}\omega$; a continuous function will be *identified* with its graph. Note that u is a graph iff $u = \lambda x.\, u(x)$, which is equivalent to Theorem 1.2 (iii). For this reason (functions are graphs) we propose the name *Graph Model* for this model of the λ-calculus. (There is more to LAMBDA than just λ, however.)

The identification of functions with graphs entails that the function space of all continuous functions from $\mathbf{P}\omega$ into $\mathbf{P}\omega$ is to be identified (one-to-one) with the subspace

$$\mathbf{FUN} = \{u \mid u = \lambda x.\, u(x)\} \subseteq \mathbf{P}\omega.$$

The identification is *topological* in that the subspace topology agree with the product topology on the function space. This is the topology of pointwise convergence and is closely connected with the lattice structure on the function space which is also defined pointwise (that is, argumentwise). In the notation of λ-abstraction we can express this as the extension of the axiom of extensionality called (ξ^*) in Table 1 of the Introduction. The laws in Table 1 are not the only ones valid in the model, however. We may also note such argumentwise distribution laws as:

$$(2.9) \quad (f \cup g)(x) = f(x) \cup g(x);$$

$$(2.10) \quad (\lambda x.\, \tau) \cup (\lambda x.\, \sigma) = \lambda x.\, (\tau \cup \sigma);$$

$$(2.11) \quad (f \cap g)(x) = f(x) \cap g(x);$$

$$(2.12) \quad (\lambda x.\, \tau) \cap (\lambda x.\, \sigma) = \lambda x.\, (\tau \cap \sigma).$$

In the above f and g must be graphs. It is also true that if $\mathcal{F} \subseteq \mathbf{P}\omega$, then

$$(2.13) \quad \bigcup\{f \mid f \in \mathcal{F}\}(x) = \bigcup\{f(x) \mid f \in \mathcal{F}\},$$

but the same does not hold for $\cap$.

We state now a sequence of minor results which show why some simple functions and constants are LAMBDA-definable.

$$(2.14)\quad \bot = (\lambda x.\, x(x))(\lambda x.\, x(x));$$

$$(2.15)\quad x \cup y = (\lambda z.\, 0) \supset x, y; \quad \text{(Hint: } 0, 1 \in \lambda z.\, 0)$$

$$(2.16)\quad \top = \mathbf{Y}(\lambda x.\, 0 \cup (x+1));$$

$$(2.17)\quad x \cap y = \mathbf{Y}(\lambda f \lambda x \lambda y.\, x \supset (y \supset 0, \bot), f(x-1)(y-1)+1)(x)(y).$$

The elements $\bot$ and $\top$ are graphs, by the way, and we can characterize them as the only fixed points of the combinator $\mathbf{K}$:

$$(2.18)\quad a = \lambda x.\, a \quad \text{iff} \quad a = \bot \quad \text{or} \quad a = \top.$$

Next we use the notation $\langle x_0, x_1, \ldots, x_{n-1}\rangle$ for the function $\hat{p}$ where $p : \omega \to \mathbf{P}\omega$ is defined by:

$$p(i) = \begin{cases} x_i & \text{if } i < n; \\ \bot & \text{if } i \geq n. \end{cases}$$

$$(2.19)\quad \langle\rangle = \bot$$

$$(2.20)\quad \langle x\rangle = \lambda z.\, z \supset x, \bot$$

$$(2.21)\quad \langle x, y\rangle = \lambda z.\, z \supset x, (z-1 \supset y, \bot)$$

$$(2.22)\quad \langle x_0, x_1, \ldots, x_n\rangle = \lambda z.\, z \supset x_0, \langle x_1, \ldots, x_n\rangle(z-1).$$

Obviously we should formalize the subscript notation so that $u_x = u(x)$; then we find:

$$(2.23)\quad \langle x_0, x_1, \ldots, x_n\rangle_i = \begin{cases} x_i & \text{if } i < n; \\ \bot & \text{if } i \geq n. \end{cases}$$

This gives us the method of LAMBDA-defining *finite sequences* (in a quite natural way), and the next step is to consider *infinite sequences*. But these are just the functions $\hat{p}$ where $p : \omega \to \mathbf{P}\omega$ is arbitrary. What we need then is a condition expressible in the language equivalent to saying $u = \hat{p}$ for some p. This is the same as

$$u = \lambda x. \bigcup\{u_i \mid i \in x\}.$$

but the $\cup$- and set-notation is not part of LAMBDA. We are forced into a recursive definition:

$$(2.24)\quad \$ = \mathbf{Y}(\lambda s \lambda u \lambda z.\, z \supset u_0,\ s(\lambda t.\, u_{t+1})(z-1)).$$

This equation generalizes (2.22) and we have:

$$(2.25)\quad \$(u) = \lambda x. \bigcup\{u_i \mid i \in x\}.$$

Thus the combinator \$ "revalues" an element as a distributive function. This suggests introducing the λ-notation for such functions by the equation:

$$(2.26) \quad \lambda n \in \omega.\, \tau = \$(\lambda z.\, \tau[z/n]).$$

With all these conventions LAMBDA-notation becomes very much like ordinary mathematical notation without too much strain.

Suppose that f is any continuous function and $a \in \mathbf{P}\omega$. We can define $p : \omega \to \mathbf{P}\omega$ in the ordinary way by primitive recursion where:

$$p(0) = a;$$

$$p(n+1) = f(n)(p(n)).$$

The question is: can we give a LAMBDA-definition for $\hat{p}$ (in terms of f and a as constants, say)? The answer is clear, for we can prove:

$$(2.27) \quad \hat{p} = \mathbf{Y}(\lambda u \lambda n \in \omega.\, n \supset a,\, f(n-1)(u(n-1))).$$

This already shows that a large part of the definitions of recursion theory can be given in this special language. Of course, simultaneous (primitive) recursions can be transcribed into LAMBDA with the aid of the ordered tuples of (2.22), (2.23) above. But we can go further and connect with partial (and general) recursive functions. We state first a definition.

theorem_2_5.lean

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb
public import DomainTheory.DataTypesAsLattices.Lambda.omega_first_recursion
public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 2.5 (The first recursion theorem).**
If `u` is the graph of a continuous `f`, then `Y(u) = fix(f)`. -/
theorem theorem_2_5 {f : Pomega  ->  Pomega} (hf : IsScottContinuous f) :
    funOf Ycomb (graph f) = fix f := by
  rw [Ycomb.app]
  exact omega_first_recursion hf

end DomainTheory.DataTypesAsLattices
```

dollarC.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.seqFun
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb
public import DomainTheory.DataTypesAsLattices.Lambda.dollarStep
public import DomainTheory.DataTypesAsLattices.Lambda.dollarStepTerm
public import DomainTheory.DataTypesAsLattices.Lambda.termDollarStepBody
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.termDollarStep
public import DomainTheory.DataTypesAsLattices.Lambda.theorem_2_5
public import DomainTheory.DataTypesAsLattices.Lambda.theorem_2_4
public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_4
public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb_more
public import DomainTheory.DataTypesAsLattices.Lambda.seqFun_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
def dollarC : Pomega :=
  graph (fun u => graph (fun x => seqFun u x))

namespace dollarC
theorem app (u : Pomega) :
    funOf dollarC u = graph (fun x => seqFun u x) :=
  beta (graph_const_isScottContinuous (fun u x => seqFun u x)
    (fun x => seqFun.left_isScottContinuous x)) u

theorem eq_fix_termDollarStep : dollarC = fix termDollarStep := by
  have hfix : termDollarStep (fix termDollarStep) = fix termDollarStep :=
    (theorem_1_4 termDollarStep.isScottContinuous).1
  have hbody : forall  u z, termDollarStepBody (fix termDollarStep) u z = seqFun u z
    := by
    intro u z
    calc termDollarStepBody (fix termDollarStep) u z
        = funOf (funOf (termDollarStep (fix termDollarStep)) u) z :=
          (termDollarStep.app2 _ _ _).symm
      _ = funOf (funOf (fix termDollarStep) u) z := by rw [hfix]
      _ = seqFun u z := termDollarStep.fix_app u z
  have : termDollarStep (fix termDollarStep) =
      graph (fun u => graph (fun z => seqFun u z)) := by
    apply congrArg graph
    funext u
    apply congrArg graph
    funext z
    exact hbody u z
  exact this.symm.trans hfix
```

```
/-- **Scott 1976, (2.24).** `$ is combinatory: $ = Y(fun s fun u fun z. z ? u_0, s(fun
    t. u_{t+1})(z?1))`. -/
theorem combinatory : IsCombinatory dollarC := by
  have hY : funOf Ycomb (interp dollarStepTerm (fun _ => botElem)) = dollarC := by
    rw [dollarStepTerm.interp, theorem_2_5 termDollarStep.isScottContinuous,
      dollarC.eq_fix_termDollarStep]
  exact hY  |>  IsCombinatory.app Ycomb.combinatory (theorem_2_4.closed dollarStepTerm
    )

end dollarC

end DomainTheory.DataTypesAsLattices
```

eq_2_24.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb
public import DomainTheory.DataTypesAsLattices.Lambda.seqFun
public import DomainTheory.DataTypesAsLattices.Lambda.dollarC
public import DomainTheory.DataTypesAsLattices.Lambda.seqShift
public import DomainTheory.DataTypesAsLattices.Lambda.dollarStepBody
public import DomainTheory.DataTypesAsLattices.Lambda.dollarStep
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import Mathlib.Data.Set.Lattice.Indexed
public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.succSet
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Lambda.theorem_2_5
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_4
public import DomainTheory.DataTypesAsLattices.Lambda.seqShift_more
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.24).** `seq(u)(x) = x ? u_0, seq(fun t. u_{t+1})(x?1)`. -/
theorem eq_2_24 (u x : Pomega) :
    seqFun u x =
      condSet x (funOf u (ofNat 0))
        (seqFun (seqShift u) (predSet x)) := by
  ext k
  constructor
  * intro hk
    obtain <i, hi, hki> := Set.mem_iUnion_2.mp hk
    cases i with
    | zero => exact Or.inl <hki, hi>
    | succ i =>
      refine Or.inr <?_, i, hi>
      refine Set.mem_iUnion_2.mpr <i, hi, ?_>
      have : funOf (seqShift u) (ofNat i) = funOf u (ofNat (i + 1)) := by
```

```
        rw [seqShift.app]
        ext m
        constructor
        * intro hm
          obtain <j, hj, hmj> := Set.mem_iUnion_2.mp hm
          simp [succSet, ofNat] at hj; subst hj; exact hmj
        * intro hm
          exact Set.mem_iUnion_2.mpr <i + 1, <i, rfl, rfl>, hm>
      simpa [this] using hki
  * intro hk
    rcases hk with <hk0, h0x> | <hpos, t, ht>
    * exact Set.mem_iUnion_2.mpr <0, h0x, hk0>
    * obtain <i, hi, hki> := Set.mem_iUnion_2.mp hpos
      have : funOf (seqShift u) (ofNat i) = funOf u (ofNat (i + 1)) := by
        rw [seqShift.app]
        ext m
        constructor
        * intro hm
          obtain <j, hj, hmj> := Set.mem_iUnion_2.mp hm
          simp [succSet, ofNat] at hj; subst hj; exact hmj
        * intro hm
          exact Set.mem_iUnion_2.mpr <i + 1, <i, rfl, rfl>, hm>
      refine Set.mem_iUnion_2.mpr <i + 1, hi, ?_>
      simpa [this] using hki

namespace eq_2_24

/-- **Scott 1976, (2.24).** `$ = Y(fun s fun u fun z. z ? u_0, s(fun t. u_{t+1})(z?1))
    `. -/
theorem Y : dollarC = fix dollarStep := by
  have hfix : dollarStep (fix dollarStep) = fix dollarStep :=
    (theorem_1_4 dollarStep.isScottContinuous).1
  have hbody : forall  u z, dollarStepBody (fix dollarStep) u z = seqFun u z := by
    intro u z
    calc dollarStepBody (fix dollarStep) u z
        = funOf (funOf (dollarStep (fix dollarStep)) u) z :=
          (dollarStep.app2 _ _ _).symm
      _ = funOf (funOf (fix dollarStep) u) z := by rw [hfix]
      _ = seqFun u z := dollarStep.fix_app u z
  have : dollarStep (fix dollarStep) =
      graph (fun u => graph (fun z => seqFun u z)) := by
    apply congrArg graph
    funext u
    apply congrArg graph
    funext z
    exact hbody u z
  exact this.symm.trans hfix

/-- **Scott 1976, (2.24).** `$ = Y(step)` as a combinator application. -/
theorem Ycomb : funOf Ycomb (graph dollarStep) = dollarC := by
  rw [theorem_2_5 dollarStep.isScottContinuous, eq_2_24.Y]

end eq_2_24

end DomainTheory.DataTypesAsLattices
```

2.3.10 The graph model

From this point on we see no need to distinguish continuous functions from elements of $\mathbf{P}\omega$; a continuous function will be *identified* with its graph. Note that u is a graph iff $u = \lambda x.\, u(x)$, which is equivalent to Theorem 1.2 (iii). For this reason (functions are graphs) we propose the name *Graph Model* for this model of the λ-calculus. (There is more to LAMBDA than just λ, however.)

The identification of functions with graphs entails that the function space of all continuous functions from $\mathbf{P}\omega$ into $\mathbf{P}\omega$ is to be identified (one-to-one) with the subspace

$$\mathbf{FUN} = \{u \mid u = \lambda x.\, u(x)\} \subseteq \mathbf{P}\omega.$$

The identification is *topological* in that the subspace topology agree with the product topology on the function space. This is the topology of pointwise convergence and is closely connected with the lattice structure on the function space which is also defined pointwise (that is, argumentwise). In the notation of λ-abstraction we can express this as the extension of the axiom of extensionality called (ξ^*) in Table 1 of the Introduction. The laws in Table 1 are not the only ones valid in the model, however. We may also note such argumentwise distribution laws as:

$$(2.9) \quad (f \cup g)(x) = f(x) \cup g(x);$$

$$(2.10) \quad (\lambda x.\, \tau) \cup (\lambda x.\, \sigma) = \lambda x.\, (\tau \cup \sigma);$$

$$(2.11) \quad (f \cap g)(x) = f(x) \cap g(x);$$

$$(2.12) \quad (\lambda x.\, \tau) \cap (\lambda x.\, \sigma) = \lambda x.\, (\tau \cap \sigma).$$

In the above f and g must be graphs. It is also true that if $\mathcal{F} \subseteq \mathbf{P}\omega$, then

$$(2.13) \quad \bigcup\{f \mid f \in \mathcal{F}\}(x) = \bigcup\{f(x) \mid f \in \mathcal{F}\},$$

but the same does not hold for $\cap$.

eq_2_10.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.10).** Abstraction distributes over union. -/
theorem eq_2_10 (tau sigma : Pomega  ->  Pomega) :
    graph tau  U  graph sigma = graph (fun x => tau x  U  sigma x) := by
  ext p
  constructor
  * intro hp
    rcases hp with h | h
```

```
    * rcases h with <n, m, hp, hm>
      exact <n, m, hp, Or.inl hm>
    * rcases h with <n, m, hp, hm>
      exact <n, m, hp, Or.inr hm>
  * intro h
    rcases h with <n, m, hp, hm>
    rcases hm with hm | hm
    * exact Or.inl <n, m, hp, hm>
    * exact Or.inr <n, m, hp, hm>

end DomainTheory.DataTypesAsLattices
```

eq_2_11.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.IsGraph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Pomega.pair

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.11).** One inclusion; equality needs both arguments to be graphs.
    -/
theorem eq_2_11 (f g x : Pomega) :
    funOf (f  I  g) x  subseteq  funOf f x  I  funOf g x := by
  intro m <n, hn, hp, hq>
  exact <<n, hn, hp>, <n, hn, hq>>

namespace eq_2_11

/-- **Scott 1976, (2.11).** Equality when both arguments are graphs. -/
theorem graphs {f g : Pomega} (hf : IsGraph f) (hg : IsGraph g)
    (x : Pomega) : funOf (f  I  g) x = funOf f x  I  funOf g x := by
  apply subset_antisymm (_root_.DomainTheory.DataTypesAsLattices.eq_2_11 f g x)
  intro m <hf', hg'>
  obtain <k, hk, hfk> := hf'
  obtain <l, hl, hgl> := hg'
  refine <k ||| l, e_or_of_subset hk hl, <?_, ?_>>
  * exact hf.out hfk (by rw [e_or]; exact Set.subset_union_left)
  * exact hg.out hgl (by rw [e_or]; exact Set.subset_union_right)

end eq_2_11

end DomainTheory.DataTypesAsLattices
```

eq_2_12.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.pair
```

```
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.12).** Abstraction distributes over intersection. -/
theorem eq_2_12 (tau sigma : Pomega  ->  Pomega) :
    graph tau  I  graph sigma = graph (fun x => tau x  I  sigma x) := by
  ext p
  constructor
  * intro <htau, hsigma>
    rcases htau with <n, m, hptau, hmtau>
    rcases hsigma with <n', m', hpsigma, hmsigma>
    have hp' : pair n m = pair n' m' := hptau.symm.trans hpsigma
    obtain <rfl, rfl> := pair_inj hp'
    exact <n, m, hptau, hmtau, hmsigma>
  * intro h
    rcases h with <n, m, hp, hmtau, hmsigma>
    exact <<n, m, hp, hmtau>, <n, m, hp, hmsigma>>

end DomainTheory.DataTypesAsLattices
```

eq_2_13.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Data.Set.Lattice.Indexed

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.13).** Application distributes over arbitrary unions. -/
theorem eq_2_13 (F : Set Pomega) (x : Pomega) :
    funOf (?_0 F) x = ? u  in  F, funOf u x := by
  ext m
  constructor
  * intro <n, hn, hp>
    obtain <u, hu, hpu> := Set.mem_sUnion.mp hp
    exact Set.mem_iUnion_2.mpr <u, hu, n, hn, hpu>
  * intro hm
    obtain <u, hu, n, hn, hp> := Set.mem_iUnion_2.mp hm
    exact <n, hn, <u, hu, hp>>

end DomainTheory.DataTypesAsLattices
```

eq_2_9.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.9).** Application distributes over unions. -/
theorem eq_2_9 (f g x : Pomega) :
    funOf (f  U  g) x = funOf f x  U  funOf g x := by
  ext m
  constructor
  * intro <n, hn, hp>
    rcases hp with hp | hp
    * exact Or.inl <n, hn, hp>
    * exact Or.inr <n, hn, hp>
  * intro h
    rcases h with <n, hn, hp> | <n, hn, hp>
    * exact <n, hn, Or.inl hp>
    * exact <n, hn, Or.inr hp>

end DomainTheory.DataTypesAsLattices
```

eta_fails.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.IsGraph
public import DomainTheory.DataTypesAsLattices.Graph.theorem_1_2
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.e_one

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Table 1 (?) fails.** Not every element is a graph. -/
theorem eta_fails : exists  u, graph (funOf u)  !=  u := by
  refine <{pair 1 0}, ?_>
  intro h
  have hG : IsGraph {pair 1 0} := (theorem_1_2.iii _).mp h
  have hsub : e 1  subseteq  e 3 := by
    intro k hk
    have : k = 0 := by
      simpa [e_one] using hk
    subst this
    decide
  have : pair 3 0  in  ({pair 1 0} : Pomega) :=
    hG.out (by simp) hsub
  have : pair 3 0 = pair 1 0 := this
  exact (by simp [pair] : pair 3 0  !=  pair 1 0) this

end DomainTheory.DataTypesAsLattices
```

table1_mu.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.mu_law
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Table 1 (?).** Application is monotone. -/
theorem table1_mu {u v x y : Pomega} (hu : u  subseteq  v) (hx : x  subseteq  y) :
    funOf u x  subseteq  funOf v y :=
  mu_law hu hx

end DomainTheory.DataTypesAsLattices
```

xi_star.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Table 1 (?*).** Abstraction is monotone. -/
theorem xi_star {tau sigma : Pomega  ->  Pomega} (h : forall  x, tau x  subseteq
    sigma x) :
    graph tau  subseteq  graph sigma := by
  intro p hp
  rcases hp with <n, m, rfl, hm>
  exact <n, m, rfl, h _ hm>

end DomainTheory.DataTypesAsLattices
```

FUN.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, S2.** The subspace `FUN` of graphs. -/
def FUN : Set Pomega := {u | u = graph (funOf u)}
```

```
end DomainTheory.DataTypesAsLattices
```

2.3.11 Elementary definitions

We state now a sequence of minor results which show why some simple functions and constants are LAMBDA-definable.

$$(2.14) \quad \bot = (\lambda x.\, x(x))(\lambda x.\, x(x));$$

$$(2.15) \quad x \cup y = (\lambda z.\, 0) \supset x, y; \quad \text{(Hint: } 0, 1 \in \lambda z.\, 0)$$

$$(2.16) \quad \top = \mathbf{Y}(\lambda x.\, 0 \cup (x+1));$$

$$(2.17) \quad x \cap y = \mathbf{Y}(\lambda f \lambda x \lambda y.\, x \supset (y \supset 0, \bot), f(x-1)(y-1)+1)(x)(y).$$

The elements $\bot$ and $\top$ are graphs, by the way, and we can characterize them as the only fixed points of the combinator $\mathbf{K}$:

$$(2.18) \quad a = \lambda x.\, a \quad \text{iff} \quad a = \bot \quad \text{or} \quad a = \top.$$

topRecTerm.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Lambda.envSet
public import DomainTheory.DataTypesAsLattices.Lambda.topRec
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_15

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

/-- **Scott 1976, (2.16).** Combinator for `fun x. 0  U  (x+1)`. -/
```

```
def topRecTerm : Term :=
  .lam 0 (.cond (.lam 1 .zero) .zero (.succ (.var 0)))

namespace topRecTerm

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem interp :
    interp topRecTerm (fun _ => botElem) = graph topRec := by
  apply congrArg graph
  funext v
  simp[topRec, _root_.DomainTheory.DataTypesAsLattices.interp, envSet, Function.update
    , _root_.DomainTheory.DataTypesAsLattices.eq_2_15]

end topRecTerm

end DomainTheory.DataTypesAsLattices
```

eq_2_16.lean

```
module

public import DomainTheory.DataTypesAsLattices.FixedPoint.iterateBot
public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.topRec
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_16 : fix topRec = topElem := by
  apply subset_antisymm
  * intro k _; trivial
  * intro k _
    have hiter : forall  n, n  in  iterateBot topRec (n + 1) := by
      intro n
      induction n with
      | zero => simp [iterateBot, topRec, ofNat]
      | succ n ih =>
        refine Or.inr <n, ?_, rfl>
        simpa [iterateBot] using ih
    exact Set.mem_iUnion.mpr <k + 1, hiter k>

end DomainTheory.DataTypesAsLattices
```

eq_2_17.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
```

```
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.interC
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.succSet
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.17).** `x  I  y = Y(fun f fun x fun y. x ? (y ? 0, False), f(x?1)(
    y?1)+1)(x)(y)`. -/
theorem eq_2_17 (x y : Pomega) :
    funOf (funOf interC x) y = x  I  y := by
  have hmem : forall  n x y, n  in  funOf (funOf interC x) y  <->  n  in  x  I  y :=
    by
    intro n
    induction n using Nat.strong_induction_on with
    | h n ih =>
      intro x y
      rw [interC.app]
      constructor
      * intro hn
        rcases hn with <hL, h0x> | <<k, hk, rfl>, _t, _ht>
        * rcases hL with <hk0, h0y> | <hbot, _>
          * simp [ofNat] at hk0; subst hk0
            exact <h0x, h0y>
          * simp [botElem] at hbot
        * have hk' : k  in  predSet x  I  predSet y :=
            (ih k (Nat.lt_succ_self k) (predSet x) (predSet y)).mp hk
          exact <hk'.1, hk'.2>
      * intro <hnx, hny>
        cases n with
        | zero => exact Or.inl <Or.inl <rfl, hny>, hnx>
        | succ k =>
          refine Or.inr <<k, ?_, rfl>, k, hnx>
          exact (ih k (Nat.lt_succ_self k) (predSet x) (predSet y)).mpr <hnx, hny>
  ext n
  exact hmem n x y

namespace eq_2_17

/-- **Scott 1976, (2.17), fundamental equation.** -/
theorem step (x y : Pomega) :
    x  I  y =
      condSet x (condSet y (ofNat 0) botElem)
        (succSet (predSet x  I  predSet y)) := by
  ext n
  constructor
  * intro <hnx, hny>
    cases n with
    | zero =>
      exact Or.inl <Or.inl <rfl, hny>, hnx>
    | succ n =>
      refine Or.inr <<n, <hnx, hny>, rfl>, n, hnx>
```

```
  * intro hn
    rcases hn with <hL, h0x> | <<k, hk, rfl>, _t, _ht>
    * rcases hL with <hk0, h0y> | <hbot, _>
      * simp [ofNat] at hk0; subst hk0
        exact <h0x, h0y>
      * simp [botElem] at hbot
    * exact <hk.1, hk.2>

end eq_2_17

end DomainTheory.DataTypesAsLattices
```

eq_2_18.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.pair_eq_right
public import DomainTheory.DataTypesAsLattices.Lambda.exists_least_not_mem
public import DomainTheory.DataTypesAsLattices.Lambda.exists_least_mem

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace eq_2_18

/-- **Scott 1976, (2.18).** Constants as graphs of constant maps. -/
theorem bot : botElem = graph (fun _ => botElem) := by
  ext p
  constructor
  * intro hp; exact hp.elim
  * intro hp
    rcases hp with <_, _, _, hm>
    simp [botElem] at hm

theorem top : topElem = graph (fun _ => topElem) := by
  ext p
  constructor
  * intro _
    obtain <n, m, rfl> := exists_pair p
    exact <n, m, rfl, Set.mem_univ m>
  * intro; trivial

end eq_2_18

/-- **Scott 1976, (2.18).** `a = fun x. a` iff `a = False` or `a =  top `. -/
theorem eq_2_18 (a : Pomega) :
    a = graph (fun _ => a)  <->  a = botElem  \/  a = topElem := by
  constructor
  * intro ha
    by_cases htop : a = topElem
    * exact Or.inr htop
```

```
    * have hex : exists  k, k  notin  a := by
        by_contra h
        apply htop
        ext k
        constructor
        * intro; trivial
        * intro
          exact not_not.mp (not_exists.mp h k)
      obtain <k, hk, hmin> := exists_least_not_mem hex
      obtain <n, m, hnm> := exists_pair k
      have hm : m  notin  a := by
        intro hma
        have : pair n m  in  a :=
          ha.symm  |>  (<n, m, rfl, hma> : pair n m  in  graph (fun _ => a))
        exact hk (by rwa [hnm] at this)
      have : m = k :=
        le_antisymm (by rw [ <-  hnm]; exact pair_le_right n m)
          (le_of_not_gt fun hlt => hm (hmin m hlt))
      have hnmk : pair n k = k := by simpa [this] using hnm
      have <_, hk0> := pair_eq_right hnmk
      have h0 : 0  notin  a := by simpa [hk0] using hk
      by_cases hbot : a = botElem
      * exact Or.inl hbot
      * obtain <l, hl, hlmin> := exists_least_mem (Set.nonempty_iff_ne_empty.mpr hbot)
        obtain <i, j, hij> := exists_pair l
        have hj : j  in  a := by
          have : pair i j  in  graph (fun _ => a) :=
            ha  |>  (by rwa [hij] : pair i j  in  a)
          rcases this with <i', j', hp, hj'>
          obtain <rfl, rfl> := pair_inj hp
          exact hj'
        have : j = l :=
          le_antisymm (by rw [ <-  hij]; exact pair_le_right i j)
            (le_of_not_gt fun hlt => hlmin j hlt hj)
        have hil : pair i l = l := by simpa [this] using hij
        have <_, hl0> := pair_eq_right hil
        exact (h0 (by simpa [hl0] using hl)).elim
  * intro h
    rcases h with rfl | rfl
    * exact eq_2_18.bot
    * exact eq_2_18.top

end DomainTheory.DataTypesAsLattices
```

eq_2_14.lean

```
module

public import DomainTheory.DataTypesAsLattices.FixedPoint.iterateBot
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.theorem_1_2
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.omegaComb
public import DomainTheory.DataTypesAsLattices.Lambda.id_isScottContinuous
```

```
public import DomainTheory.DataTypesAsLattices.Lambda.omega_first_recursion

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.14).** `False = (fun x. x(x))(fun x. x(x))`. -/
public theorem eq_2_14 :
    funOf (graph (fun x => funOf x x)) (graph (fun x => funOf x x)) = botElem := by
  have h? : omegaComb (graph fun x => x) = graph (fun x => funOf x x) := by
    ext p
    constructor
    * intro hp
      rcases hp with <n, m, rfl, hm>
      rw [theorem_1_2.i id_isScottContinuous] at hm
      exact <n, m, rfl, hm>
    * intro hp
      rcases hp with <n, m, rfl, hm>
      refine <n, m, rfl, ?_>
      rw [theorem_1_2.i id_isScottContinuous]
      exact hm
  rw [ <-  h?, omega_first_recursion id_isScottContinuous]
  apply subset_antisymm
  * intro k hk
    obtain <n, hkn> := Set.mem_iUnion.mp hk
    have : forall  n, iterateBot (fun x => x) n = botElem := by
      intro n
      induction n with
      | zero => rfl
      | succ n ih => simp [iterateBot, ih]
    simp [this n] at hkn
  * intro k hk
    exact hk.elim

end DomainTheory.DataTypesAsLattices
```

eq_2_15.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.pair_zero_zero
public import DomainTheory.DataTypesAsLattices.Lambda.pair_one_zero
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.15).** `x  U  y = (fun z. 0) ? x, y`. -/
theorem eq_2_15 (x y : Pomega) :
    condSet (graph (fun _ => ofNat 0)) x y = x  U  y := by
```

```
  have h0 : 0  in  graph (fun _ => ofNat 0) :=
    <0, 0, pair_zero_zero.symm, by simp [ofNat]>
  have h1 : 1  in  graph (fun _ => ofNat 0) :=
    <1, 0, pair_one_zero.symm, by simp [ofNat]>
  ext n
  constructor
  * intro hn
    rcases hn with <hnx, _> | <hny, _>
    * exact Or.inl hnx
    * exact Or.inr hny
  * intro hn
    rcases hn with hnx | hny
    * exact Or.inl <hnx, h0>
    * exact Or.inr <hny, 0, h1>

end DomainTheory.DataTypesAsLattices
```

interC.lean

```
module

public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.interStep
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_4
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.id_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.succSet
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Lambda.predSet_more
public import DomainTheory.DataTypesAsLattices.Lambda.succSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def interC : Pomega := fix interStep

namespace interC

theorem else_isScottContinuous (y : Pomega) :
    IsScottContinuous (fun x =>
      succSet (funOf (funOf interC (predSet x)) (predSet y))) := by
```

```
  refine theorem_1_3 (f := succSet) (g := fun x =>
      funOf (funOf interC (predSet x)) (predSet y))
    succSet.isScottContinuous ?_
  refine theorem_1_3
    (f := fun t => funOf t (predSet y))
    (g := fun x => funOf interC (predSet x))
    (funOf.isScottContinuous_left (predSet y)) ?_
  exact theorem_1_3 (funOf.isScottContinuous interC) predSet.isScottContinuous

theorem then_else_isScottContinuous (x : Pomega) :
    IsScottContinuous (fun y =>
      condSet x (condSet y (ofNat 0) botElem)
        (succSet (funOf (funOf interC (predSet x)) (predSet y)))) :=
  theorem_1_3.tuple
    (fun els => condSet.isScottContinuous_mid x els)
    (fun th => condSet.isScottContinuous_right x th)
    (condSet.isScottContinuous_left (ofNat 0) botElem)
    (theorem_1_3 succSet.isScottContinuous
      (theorem_1_3 (funOf.isScottContinuous (funOf interC (predSet x)))
        predSet.isScottContinuous))

theorem body_isScottContinuous :
    IsScottContinuous (fun x =>
      graph (fun y =>
        condSet x (condSet y (ofNat 0) botElem)
          (succSet (funOf (funOf interC (predSet x)) (predSet y))))) :=
  graph_const_isScottContinuous
    (fun x y =>
      condSet x (condSet y (ofNat 0) botElem)
        (succSet (funOf (funOf interC (predSet x)) (predSet y))))
    (fun y =>
      theorem_1_3.tuple
        (fun els => condSet.isScottContinuous_left
          (condSet y (ofNat 0) botElem) els)
        (fun tes => condSet.isScottContinuous_right tes
          (condSet y (ofNat 0) botElem))
        id_isScottContinuous
        (interC.else_isScottContinuous y))

theorem unfold : interC = interStep interC :=
  (theorem_1_4 interStep.isScottContinuous).1.symm

theorem app_graph (x : Pomega) :
    funOf interC x =
      graph (fun y =>
        condSet x (condSet y (ofNat 0) botElem)
          (succSet (funOf (funOf interC (predSet x)) (predSet y)))) := by
  nth_rw 1 [interC.unfold]
  exact _root_.DomainTheory.DataTypesAsLattices.beta interC.body_isScottContinuous x

theorem app (x y : Pomega) :
    funOf (funOf interC x) y =
      condSet x (condSet y (ofNat 0) botElem)
        (succSet (funOf (funOf interC (predSet x)) (predSet y))) := by
  rw [interC.app_graph]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (interC.
```

```
    then_else_isScottContinuous x) y

end interC

end DomainTheory.DataTypesAsLattices
```

interStep.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.succSet
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Lambda.succSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
/-- Least operator solving the (2.17) recursion. -/
def interStep (f : Pomega) : Pomega :=
  graph (fun x => graph (fun y =>
    condSet x (condSet y (ofNat 0) botElem)
      (succSet (funOf (funOf f (predSet x)) (predSet y)))))

namespace interStep

theorem isScottContinuous :
    IsScottContinuous interStep := by
  refine graph_const_isScottContinuous
    (fun f x => graph (fun y =>
      condSet x (condSet y (ofNat 0) botElem)
        (succSet (funOf (funOf f (predSet x)) (predSet y))))) ?_
  intro x
  refine graph_const_isScottContinuous
    (fun f y =>
      condSet x (condSet y (ofNat 0) botElem)
        (succSet (funOf (funOf f (predSet x)) (predSet y)))) ?_
  intro y
  refine theorem_1_3
    (condSet.isScottContinuous_right x (condSet y (ofNat 0) botElem)) ?_
  refine theorem_1_3 succSet.isScottContinuous ?_
  exact theorem_1_3
```

```
    (f := fun t => funOf t (predSet y))
    (g := fun f => funOf f (predSet x))
    (funOf.isScottContinuous_left (predSet y))
    (funOf.isScottContinuous_left (predSet x))

end interStep

end DomainTheory.DataTypesAsLattices
```

topRec.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.succSet
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.16).** ` top  = Y(fun x. 0  U  (x + 1))`. -/
def topRec (x : Pomega) : Pomega := ofNat 0  U  succSet x

namespace topRec

theorem isScottContinuous : IsScottContinuous topRec := by
  constructor
  intro x
  ext k
  constructor
  * intro hk
    rcases hk with hk | <n, hn, rfl>
    * exact mem_scottUnion.mpr <0, by simp [e_zero], Or.inl hk>
    * exact mem_scottUnion.mpr <2 ^ n, by simp [e_pow2, Set.singleton_subset_iff, hn],
        Or.inr <n, by simp [e_pow2], rfl>>
  * intro hk
    obtain <m, hm, hkm> := mem_scottUnion.mp hk
    rcases hkm with hk | <n, hn, rfl>
    * exact Or.inl hk
    * exact Or.inr <n, hm hn, rfl>

end topRec

end DomainTheory.DataTypesAsLattices
```

union_via_cond.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_15
```

```
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Lambda.condC
public import DomainTheory.DataTypesAsLattices.Lambda.zeroC
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb_more
public import DomainTheory.DataTypesAsLattices.Lambda.condC_more
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.15)** as a combinator: `cond(x)(y)(K(0)) = x  U  y`. -/
theorem union_via_cond (x y : Pomega) :
    funOf (funOf (funOf condC x) y) (funOf Kcomb zeroC) = x  U  y := by
  rw [condC.beta3, Kcomb.beta]
  exact eq_2_15 x y

end DomainTheory.DataTypesAsLattices
```

2.3.12 Sequences

Next we use the notation $\langle x_0, x_1, \ldots, x_{n-1}\rangle$ for the function $\hat{p}$ where $p : \omega \to \mathbf{P}\omega$ is defined by:

$$p(i) = \begin{cases} x_i & \text{if } i < n; \\ \bot & \text{if } i \geq n. \end{cases}$$

$$\langle\rangle = \bot \tag{2.19}$$

$$\langle x\rangle = \lambda z.\, z \supset x, \bot \tag{2.20}$$

$$\langle x, y\rangle = \lambda z.\, z \supset x, (z - 1 \supset y, \bot) \tag{2.21}$$

$$\langle x_0, x_1, \ldots, x_n\rangle = \lambda z.\, z \supset x_0, \langle x_1, \ldots, x_n\rangle(z - 1). \tag{2.22}$$

Obviously we should formalize the subscript notation so that $u_x = u(x)$; then we find:

$$\langle x_0, x_1, \ldots, x_n\rangle_i = \begin{cases} x_i & \text{if } i < n; \\ \bot & \text{if } i \geq n. \end{cases} \tag{2.23}$$

This gives us the method of LAMBDA-defining *finite sequences* (in a quite natural way), and the next step is to consider *infinite sequences*. But these are just the functions $\hat{p}$ where $p : \omega \to \mathbf{P}\omega$ is arbitrary. What we need then is a condition expressible in the language equivalent to saying $u = \hat{p}$ for some p. This is the same as

$$u = \lambda x. \bigcup\{u_i \mid i \in x\}.$$

mem_seqFunLt.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.seqFunLt
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import Mathlib.Data.Set.Lattice.Indexed
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem mem_seqFunLt {u x : Pomega} {n k : Nat} :
    k  in  seqFunLt u x n  <->  exists  i  in  x, i < n  /\  k  in  funOf u (ofNat i)
    := by
  constructor
  * intro hk
    obtain <i, hi, hrest> := Set.mem_iUnion_2.mp hk
    obtain <hlt, hki> := Set.mem_iUnion.mp hrest
    exact <i, hi, hlt, hki>
  * intro <i, hi, hlt, hki>
    exact Set.mem_iUnion_2.mpr <i, hi, Set.mem_iUnion.mpr <hlt, hki>>

end DomainTheory.DataTypesAsLattices
```

seq2.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.scottUnion
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import Mathlib.Data.Set.Lattice.Indexed
public import DomainTheory.DataTypesAsLattices.Lambda.predSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def seq2 (x y : Pomega) : Pomega :=
  graph (fun z => condSet z x (condSet (predSet z) y botElem))

namespace seq2

theorem body_isScottContinuous (x y : Pomega) :
```

```
    IsScottContinuous (fun z => condSet z x (condSet (predSet z) y botElem)) := by
  constructor
  intro z
  have H := theorem_1_3.diag (x := z)
    (f := fun z w => condSet z x (condSet (predSet w) y botElem))
    (fun _w => condSet.isScottContinuous_left x _)
    (fun _z =>
      theorem_1_3 (condSet.isScottContinuous_right _z x)
        (theorem_1_3 (condSet.isScottContinuous_left y botElem)
          predSet.isScottContinuous))
  ext k
  constructor
  * intro hk
    have : k  in  ? n, {k | e n  subseteq  z  /\
        k  in  condSet (e n) x (condSet (predSet (e n)) y botElem)} := by
      rwa [ <-  H]
    obtain <n, hn, hkn> := Set.mem_iUnion.mp this
    exact mem_scottUnion.mpr <n, hn, hkn>
  * intro hk
    obtain <n, hn, hkn> := mem_scottUnion.mp hk
    have : k  in  ? n, {k | e n  subseteq  z  /\
        k  in  condSet (e n) x (condSet (predSet (e n)) y botElem)} :=
      Set.mem_iUnion.mpr <n, hn, hkn>
    rwa [ <-  H] at this

theorem app (x y z : Pomega) :
    funOf (seq2 x y) z = condSet z x (condSet (predSet z) y botElem) :=
  beta (seq2.body_isScottContinuous x y) z

theorem one (x y : Pomega) : funOf (seq2 x y) (ofNat 1) = y := by
  rw [app]
  ext n
  simp [condSet, predSet, ofNat, botElem]

theorem zero (x y : Pomega) : funOf (seq2 x y) (ofNat 0) = x := by
  rw [app]
  ext n
  simp [condSet, predSet, ofNat, botElem]

theorem tag_ne (x y : Pomega) :
    seq2 (ofNat 0) x  !=  seq2 (ofNat 1) y := by
  intro h
  have hfun := congrArg (fun w => funOf w (ofNat 0)) h
  have hx : (0 : Nat)  in  funOf (seq2 (ofNat 0) x) (ofNat 0) :=
    <1, fun k hk =>
      match k, hk with
      | 0, _ => rfl
      | k + 1, hk =>
        False.elim (Bool.false_ne_true
          (((Nat.testBit_succ 1 k).trans (Nat.zero_testBit k)).symm.trans hk)),
      <1, 0, rfl, Or.inl <rfl, rfl>>>
  have hy : (0 : Nat)  notin  funOf (seq2 (ofNat 1) y) (ofNat 0) := by
    intro <k, hk, hp>
    rcases hp with <k', m, heq, hm>
    obtain <rfl, rfl> := pair_inj heq
    rcases hm with <h1, _> | <_, <t, ht>>
```

```
    * exact Nat.zero_ne_one h1
    * exact Nat.succ_ne_zero t (hk ht)
  exact hy (hfun  |>  hx)

end seq2

end DomainTheory.DataTypesAsLattices
```

eq_2_23_ge.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.seq2
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_23_ge (x y : Pomega) (n : Nat) :
    funOf (seq2 x y) (ofNat (n + 2)) = botElem := by
  rw [seq2.app]
  ext k
  simp [condSet, predSet, ofNat, botElem]

end DomainTheory.DataTypesAsLattices
```

eq_2_23_one.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.seq2
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_23_one (x y : Pomega) : funOf (seq2 x y) (ofNat 1) = y :=
  seq2.one x y

end DomainTheory.DataTypesAsLattices
```

eq_2_23_zero.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
```

```
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.seq2
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.23).** Projection of a binary sequence. -/
theorem eq_2_23_zero (x y : Pomega) : funOf (seq2 x y) (ofNat 0) = x :=
  seq2.zero x y

end DomainTheory.DataTypesAsLattices
```

seqCons.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.id_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Lambda.predSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
/-- **Scott 1976, (2.22).** Longer sequences by cons. -/
def seqCons (x xs : Pomega) : Pomega :=
  graph (fun z => condSet z x (funOf xs (predSet z)))

namespace seqCons

theorem body (x xs : Pomega) :
    IsScottContinuous (fun z => condSet z x (funOf xs (predSet z))) :=
  theorem_1_3.tuple
    (fun _ => condSet.isScottContinuous_left x _)
    (fun z => condSet.isScottContinuous_right z x)
    id_isScottContinuous
    (theorem_1_3 (funOf.isScottContinuous xs) predSet.isScottContinuous)

end seqCons

end DomainTheory.DataTypesAsLattices
```

eq_2_22.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.seqCons
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_22 (x xs z : Pomega) :
    funOf (seqCons x xs) z = condSet z x (funOf xs (predSet z)) :=
  beta (seqCons.body x xs) z

end DomainTheory.DataTypesAsLattices
```

seqFunLt_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.seqFunLt
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.seqShift
public import DomainTheory.DataTypesAsLattices.Lambda.termShift
public import DomainTheory.DataTypesAsLattices.Lambda.mem_seqFunLt
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Lambda.seqShift_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace seqFunLt

theorem succ (u z : Pomega) (n : Nat) :
    condSet z (funOf u (ofNat 0)) (seqFunLt (seqShift u) (predSet z) n) =
      seqFunLt u z (n + 1) := by
  ext k
  constructor
  * intro hk
    rcases hk with <hk0, h0z> | <hpos, t, ht>
    * exact (mem_seqFunLt).mpr <0, h0z, Nat.succ_pos n, hk0>
    * obtain <i, hi, hlt, hki> := (mem_seqFunLt).mp hpos
      refine (mem_seqFunLt).mpr <i + 1, hi, Nat.succ_lt_succ hlt, ?_>
      simpa [seqShift.ofNat] using hki
  * intro hk
    obtain <i, hi, hlt, hki> := (mem_seqFunLt).mp hk
    cases i with
    | zero => exact Or.inl <hki, hi>
```

```
    | succ i =>
      refine Or.inr <?_, i, hi>
      refine (mem_seqFunLt).mpr <i, hi, Nat.lt_of_succ_lt_succ hlt, ?_>
      simpa [seqShift.ofNat] using hki

theorem term (u z : Pomega) (n : Nat) :
    condSet z (funOf u (ofNat 0)) (seqFunLt (termShift u) (predSet z) n) =
      seqFunLt u z (n + 1) := by
  ext k
  constructor
  * intro hk
    rcases hk with <hk0, h0z> | <hpos, t, ht>
    * exact (mem_seqFunLt).mpr <0, h0z, Nat.succ_pos n, hk0>
    * obtain <i, hi, hlt, hki> := (mem_seqFunLt).mp hpos
      refine (mem_seqFunLt).mpr <i + 1, hi, Nat.succ_lt_succ hlt, ?_>
      simpa [termShift.ofNat] using hki
  * intro hk
    obtain <i, hi, hlt, hki> := (mem_seqFunLt).mp hk
    cases i with
    | zero => exact Or.inl <hki, hi>
    | succ i =>
      refine Or.inr <?_, i, hi>
      refine (mem_seqFunLt).mpr <i, hi, Nat.lt_of_succ_lt_succ hlt, ?_>
      simpa [termShift.ofNat] using hki

theorem zero (u z : Pomega) : seqFunLt u z 0 = botElem := by
  ext k
  simp [mem_seqFunLt, botElem]

end seqFunLt

end DomainTheory.DataTypesAsLattices
```

seqFun_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.seqFun
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.seqFunLt
public import DomainTheory.DataTypesAsLattices.Lambda.mem_seqFunLt
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Continuous.scottUnion
public import Mathlib.Data.Set.Lattice.Indexed

@[expose] public section
```

```
namespace DomainTheory.DataTypesAsLattices

namespace seqFun

theorem eq_iUnion_lt (u z : Pomega) :
    seqFun u z = ? n, seqFunLt u z n := by
  ext k
  constructor
  * intro hk
    obtain <i, hi, hki> := Set.mem_iUnion_2.mp hk
    exact Set.mem_iUnion.mpr <i + 1, (mem_seqFunLt).mpr <i, hi, Nat.lt_succ_self i,
    hki>>
  * intro hk
    obtain <n, hn> := Set.mem_iUnion.mp hk
    obtain <i, hi, _, hki> := (mem_seqFunLt).mp hn
    exact Set.mem_iUnion_2.mpr <i, hi, hki>

theorem left_isScottContinuous (x : Pomega) :
    IsScottContinuous (fun u => seqFun u x) := by
  constructor
  intro u
  ext k
  constructor
  * intro hk
    obtain <i, hi, hki> := Set.mem_iUnion_2.mp hk
    have : k  in  scottUnion (fun v => funOf v (ofNat i)) u := by
      rw [ <-  (funOf.isScottContinuous_left (ofNat i)).out u]; exact hki
    obtain <n, hn, hkn> := mem_scottUnion.mp this
    exact mem_scottUnion.mpr <n, hn, Set.mem_iUnion_2.mpr <i, hi, hkn>>
  * intro hk
    obtain <n, hn, hkn> := mem_scottUnion.mp hk
    obtain <i, hi, hki> := Set.mem_iUnion_2.mp hkn
    exact Set.mem_iUnion_2.mpr <i, hi,
      IsScottContinuous.monotone (funOf.isScottContinuous_left (ofNat i)) hn hki>

theorem right_isScottContinuous (u : Pomega) :
    IsScottContinuous (fun x => seqFun u x) := by
  constructor
  intro x
  ext k
  constructor
  * intro hk
    obtain <i, hi, hki> := Set.mem_iUnion_2.mp hk
    exact mem_scottUnion.mpr <2 ^ i, by simp [e_pow2, Set.singleton_subset_iff, hi],
      Set.mem_iUnion_2.mpr <i, by simp [e_pow2], hki>>
  * intro hk
    obtain <n, hn, hkn> := mem_scottUnion.mp hk
    obtain <i, hi, hki> := Set.mem_iUnion_2.mp hkn
    exact Set.mem_iUnion_2.mpr <i, hn hi, hki>

end seqFun

end DomainTheory.DataTypesAsLattices
```

seqShift_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.seqShift
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.seqFun
public import DomainTheory.DataTypesAsLattices.Lambda.succSet
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import Mathlib.Data.Set.Lattice.Indexed
public import DomainTheory.DataTypesAsLattices.Lambda.seqFun_more
public import DomainTheory.DataTypesAsLattices.Lambda.succSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace seqShift

theorem isScottContinuous (u : Pomega) :
    IsScottContinuous (fun z => seqFun u (succSet z)) :=
  theorem_1_3 (seqFun.right_isScottContinuous u) succSet.isScottContinuous

theorem app (u z : Pomega) :
    funOf (seqShift u) z = seqFun u (succSet z) :=
  beta (seqShift.isScottContinuous u) z

theorem left_isScottContinuous : IsScottContinuous seqShift :=
  graph_const_isScottContinuous (fun u z => seqFun u (succSet z))
    (fun z => seqFun.left_isScottContinuous (succSet z))

theorem ofNat (u : Pomega) (i : Nat) :
    funOf (seqShift u) (ofNat i) = funOf u (ofNat (i + 1)) := by
  rw [seqShift.app]
  ext m
  constructor
  * intro hm
    obtain <j, hj, hmj> := Set.mem_iUnion_2.mp hm
    simp [succSet] at hj; subst hj; exact hmj
  * intro hm
    exact Set.mem_iUnion_2.mpr <i + 1, <i, rfl, rfl>, hm>

end seqShift

end DomainTheory.DataTypesAsLattices
```

seq0.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.19)-(2.21).** Finite sequences as distributive functions. -/
def seq0 : Pomega := botElem

end DomainTheory.DataTypesAsLattices
```

seq1.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def seq1 (x : Pomega) : Pomega :=
  graph (fun z => condSet z x botElem)

end DomainTheory.DataTypesAsLattices
```

seqFunLt.lean

```
module
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.Comb
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
/-- Finite stage of the `$` iteration: `? { u_i | i  in  x, i < n }`. -/
def seqFunLt (u x : Pomega) (n : Nat) : Pomega :=
  ? i  in  x, ? _ : i < n, funOf u (ofNat i)

end DomainTheory.DataTypesAsLattices
```

seqShift.lean

```
module
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.Comb
```

```
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.seqFun

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
/-- Shift of a sequence: `(fun t. u_{t+1})`. -/
def seqShift (u : Pomega) : Pomega :=
  graph (fun z => seqFun u (succSet z))

end DomainTheory.DataTypesAsLattices
```

2.3.13 Distributive functions

The $\cup$- and set-notation is not part of LAMBDA. We are forced into a recursive definition:

$$(2.24) \quad \$ = \mathbf{Y}(\lambda s \lambda u \lambda z.\, z \supset u_0,\, s(\lambda t.\, u_{t+1})(z-1)).$$

This equation generalizes (2.22) and we have:

$$(2.25) \quad \$(u) = \lambda x.\, \bigcup \{u_i \mid i \in x\}.$$

Thus the combinator \$ "revalues" an element as a distributive function. This suggests introducing the λ-notation for such functions by the equation:

$$(2.26) \quad \lambda n \in \omega.\, \tau = \$(\lambda z.\, \tau[z/n]).$$

With all these conventions LAMBDA-notation becomes very much like ordinary mathematical notation without too much strain.

dollarStepTerm.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.envSet
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Lambda.termShift
public import DomainTheory.DataTypesAsLattices.Lambda.termDollarStepBody
public import DomainTheory.DataTypesAsLattices.Lambda.termDollarStep
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.succSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
/-- Paper combinator for `$`: `fun s fun u fun z. z ? u(0), s(fun t. u(t+1))(z?1)`. -/
def dollarStepTerm : Term :=
  .lam 0 (.lam 1 (.lam 2
```

```
    (.cond (.var 2)
      (.app (.var 1) .zero)
      (.app (.app (.var 0) (.lam 3 (.app (.var 1) (.succ (.var 3)))))
        (.pred (.var 2))))))

namespace dollarStepTerm

theorem interp :
    interp dollarStepTerm (fun _ => botElem) = graph termDollarStep := by
  unfold dollarStepTerm termDollarStep termDollarStepBody termShift DomainTheory.
    DataTypesAsLattices.interp
  apply congrArg graph
  funext s
  simp only[envSet]
  apply congrArg graph
  funext u
  simp only[envSet]
  apply congrArg graph
  funext z
  simp[envSet, condSet, succSet]
  rfl

end dollarStepTerm

end DomainTheory.DataTypesAsLattices
```

dollarStepBody.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.seqShift
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.id_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Lambda.predSet_more
public import DomainTheory.DataTypesAsLattices.Lambda.seqShift_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
/-- **Scott 1976, (2.24).** The step of `$ = Y(fun s fun u fun z. z ? u_0, s(fun t. u_{
    t+1})(z?1))`. -/
def dollarStepBody (s u z : Pomega) : Pomega :=
  condSet z (funOf u (ofNat 0))
    (funOf (funOf s (seqShift u)) (predSet z))

namespace dollarStepBody
```

```
theorem isScottContinuous_s (u z : Pomega) :
    IsScottContinuous (fun s => dollarStepBody s u z) := by
  have hinner : IsScottContinuous
      (fun s => funOf (funOf s (seqShift u)) (predSet z)) :=
    theorem_1_3 (f := fun w => funOf w (predSet z))
      (g := fun s => funOf s (seqShift u))
      (funOf.isScottContinuous_left (predSet z))
      (funOf.isScottContinuous_left (seqShift u))
  exact theorem_1_3 (condSet.isScottContinuous_right z (funOf u (ofNat 0))) hinner

theorem isScottContinuous_u (s z : Pomega) :
    IsScottContinuous (fun u => dollarStepBody s u z) := by
  have helse : IsScottContinuous
      (fun u => funOf (funOf s (seqShift u)) (predSet z)) :=
    theorem_1_3 (f := fun w => funOf w (predSet z))
      (g := fun u => funOf s (seqShift u))
      (funOf.isScottContinuous_left (predSet z))
      (theorem_1_3 (funOf.isScottContinuous s) seqShift.left_isScottContinuous)
  exact theorem_1_3.tuple
    (fun y => condSet.isScottContinuous_mid z y)
    (fun x => condSet.isScottContinuous_right z x)
    (funOf.isScottContinuous_left (ofNat 0)) helse

theorem isScottContinuous_z (s u : Pomega) :
    IsScottContinuous (fun z => dollarStepBody s u z) :=
  theorem_1_3.tuple
    (fun y => condSet.isScottContinuous_left (funOf u (ofNat 0)) y)
    (fun z => condSet.isScottContinuous_right z (funOf u (ofNat 0)))
    id_isScottContinuous
    (theorem_1_3 (funOf.isScottContinuous (funOf s (seqShift u)))
      predSet.isScottContinuous)

end dollarStepBody

end DomainTheory.DataTypesAsLattices
```

dollarStep.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.dollarStepBody
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Lambda.seqFun
public import DomainTheory.DataTypesAsLattices.Lambda.seqFunLt
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.FixedPoint.iterateBot
public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
```

```
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Lambda.seqFunLt_more
public import DomainTheory.DataTypesAsLattices.Lambda.seqFun_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def dollarStep (s : Pomega) : Pomega :=
  graph (fun u => graph (fun z => dollarStepBody s u z))

namespace dollarStep

theorem isScottContinuous : IsScottContinuous dollarStep :=
  graph_const_isScottContinuous
    (fun s u => graph (fun z => dollarStepBody s u z))
    (fun u => graph_const_isScottContinuous
      (fun s z => dollarStepBody s u z)
      (fun z => dollarStepBody.isScottContinuous_s u z))

theorem app (s u : Pomega) :
    funOf (dollarStep s) u = graph (fun z => dollarStepBody s u z) :=
  beta (graph_const_isScottContinuous (fun u z => dollarStepBody s u z)
    (fun z => dollarStepBody.isScottContinuous_u s z)) u

theorem app2 (s u z : Pomega) :
    funOf (funOf (dollarStep s) u) z = dollarStepBody s u z := by
  rw [dollarStep.app]
  exact beta (dollarStepBody.isScottContinuous_z s u) z

theorem iterate (n : Nat) (u z : Pomega) :
    funOf (funOf (iterateBot dollarStep n) u) z = seqFunLt u z n := by
  induction n generalizing u z with
  | zero =>
    simp [iterateBot, funOf.bot, seqFunLt.zero]
  | succ n ih =>
    rw [iterateBot, dollarStep.app2, dollarStepBody, ih, seqFunLt.succ]

theorem fix_app (u z : Pomega) :
    funOf (funOf (fix dollarStep) u) z = seqFun u z := by
  have : funOf (funOf (fix dollarStep) u) z =
      ? n, funOf (funOf (iterateBot dollarStep n) u) z := by
    simp [fix, funOf.iUnion]
  rw [this, seqFun.eq_iUnion_lt]
  ext k
  constructor
  * intro hk
    obtain <n, hn> := Set.mem_iUnion.mp hk
    exact Set.mem_iUnion.mpr <n, by simpa [dollarStep.iterate] using hn>
  * intro hk
    obtain <n, hn> := Set.mem_iUnion.mp hk
    exact Set.mem_iUnion.mpr <n, by simpa [dollarStep.iterate] using hn>

end dollarStep

end DomainTheory.DataTypesAsLattices
```

eq_2_25.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.seqFun
public import DomainTheory.DataTypesAsLattices.Lambda.dollarC
public import DomainTheory.DataTypesAsLattices.Lambda.seqFun_more
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.25).** `$ (u) = fun x. ? {u_i | i  in  x}`. -/
theorem eq_2_25 (u x : Pomega) :
    funOf (funOf dollarC u) x = seqFun u x := by
  rw [dollarC.app]
  exact beta (seqFun.right_isScottContinuous u) x

end DomainTheory.DataTypesAsLattices
```

termDollarStepBody.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.termShift
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.dollarStepBody
public import DomainTheory.DataTypesAsLattices.Lambda.id_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Lambda.predSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
def termDollarStepBody (s u z : Pomega) : Pomega :=
  condSet z (funOf u (ofNat 0))
    (funOf (funOf s (termShift u)) (predSet z))

namespace termDollarStepBody

theorem isScottContinuous_u (s z : Pomega) :
    IsScottContinuous (fun u => termDollarStepBody s u z) := by
  have helse : IsScottContinuous
```

```
      (fun u => funOf (funOf s (termShift u)) (predSet z)) :=
    theorem_1_3 (f := fun w => funOf w (predSet z))
      (g := fun u => funOf s (termShift u))
      (funOf.isScottContinuous_left (predSet z))
      (theorem_1_3 (funOf.isScottContinuous s) termShift.left_isScottContinuous)
  exact theorem_1_3.tuple
    (fun y => condSet.isScottContinuous_mid z y)
    (fun x => condSet.isScottContinuous_right z x)
    (funOf.isScottContinuous_left (ofNat 0)) helse

theorem isScottContinuous_z (s u : Pomega) :
    IsScottContinuous (fun z => termDollarStepBody s u z) :=
  theorem_1_3.tuple
    (fun y => condSet.isScottContinuous_left (funOf u (ofNat 0)) y)
    (fun z => condSet.isScottContinuous_right z (funOf u (ofNat 0)))
    id_isScottContinuous
    (theorem_1_3 (funOf.isScottContinuous (funOf s (termShift u)))
      predSet.isScottContinuous)

theorem isScottContinuous_s (u z : Pomega) :
    IsScottContinuous (fun s => termDollarStepBody s u z) := by
  have hinner : IsScottContinuous
      (fun s => funOf (funOf s (termShift u)) (predSet z)) :=
    theorem_1_3 (f := fun w => funOf w (predSet z))
      (g := fun s => funOf s (termShift u))
      (funOf.isScottContinuous_left (predSet z))
      (funOf.isScottContinuous_left (termShift u))
  exact theorem_1_3 (condSet.isScottContinuous_right z (funOf u (ofNat 0))) hinner

end termDollarStepBody

end DomainTheory.DataTypesAsLattices
```

seqFun.lean

```
module
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.Comb
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.24)-(2.25).** Revaluation as a distributive sequence. -/
def seqFun (u x : Pomega) : Pomega :=
  ? i  in  x, funOf u (ofNat i)

end DomainTheory.DataTypesAsLattices
```

lamOmega.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.dollarC

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.26).** `fun n in ?. tau` is `$` applied to a graph. -/
def lamOmega (tau : Nat  ->  Pomega) : Pomega :=
  funOf dollarC (graph (fun z => ? n  in  z, tau n))

end DomainTheory.DataTypesAsLattices
```

termDollarStep.lean

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.iterateBot
public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.termDollarStepBody
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.seqFunLt
public import DomainTheory.DataTypesAsLattices.Lambda.seqFun
public import DomainTheory.DataTypesAsLattices.Lambda.dollarStep
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Lambda.seqFunLt_more
public import DomainTheory.DataTypesAsLattices.Lambda.seqFun_more
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def termDollarStep (s : Pomega) : Pomega :=
  graph (fun u => graph (fun z => termDollarStepBody s u z))

namespace termDollarStep

theorem isScottContinuous : IsScottContinuous termDollarStep :=
  graph_const_isScottContinuous
    (fun s u => graph (fun z => termDollarStepBody s u z))
    (fun u => graph_const_isScottContinuous
      (fun s z => termDollarStepBody s u z)
      (fun z => termDollarStepBody.isScottContinuous_s u z))
```

```
theorem app (s u : Pomega) :
    funOf (termDollarStep s) u = graph (fun z => termDollarStepBody s u z) :=
  beta (graph_const_isScottContinuous (fun u z => termDollarStepBody s u z)
    (fun z => termDollarStepBody.isScottContinuous_u s z)) u

theorem app2 (s u z : Pomega) :
    funOf (funOf (termDollarStep s) u) z = termDollarStepBody s u z := by
  rw [app]
  exact beta (termDollarStepBody.isScottContinuous_z s u) z

theorem iterate (n : Nat) (u z : Pomega) :
    funOf (funOf (iterateBot termDollarStep n) u) z = seqFunLt u z n := by
  induction n generalizing u z with
  | zero =>
    simp [iterateBot, funOf.bot, seqFunLt.zero]
  | succ n ih =>
    rw [iterateBot, app2, termDollarStepBody, ih, seqFunLt.term]

theorem fix_app (u z : Pomega) :
    funOf (funOf (fix termDollarStep) u) z = seqFun u z := by
  have : funOf (funOf (fix termDollarStep) u) z =
      ? n, funOf (funOf (iterateBot termDollarStep n) u) z := by
    simp [fix, funOf.iUnion]
  rw [this, seqFun.eq_iUnion_lt]
  ext k
  constructor
  * intro hk
    obtain <n, hn> := Set.mem_iUnion.mp hk
    exact Set.mem_iUnion.mpr <n, by simpa [iterate] using hn>
  * intro hk
    obtain <n, hn> := Set.mem_iUnion.mp hk
    exact Set.mem_iUnion.mpr <n, by simpa [iterate] using hn>

end termDollarStep

end DomainTheory.DataTypesAsLattices
```

2.3.14 Primitive recursion

Suppose that f is any continuous function and $a \in \mathbf{P}\omega$. We can define $p : \omega \to \mathbf{P}\omega$ in the ordinary way by primitive recursion where:

$$p(0) = a;$$

$$p(n+1) = f(n)(p(n)).$$

The question is: can we give a LAMBDA-definition for $\hat{p}$ (in terms of f and a as constants, say)? The answer is clear, for we can prove:

$$\hat{p} = \mathbf{Y}(\lambda u \lambda n \in \omega.\, n \supset a,\ f(n-1)(u(n-1))). \tag{2.27}$$

This already shows that a large part of the definitions of recursion theory can be given in this special language. Of course, simultaneous (primitive) recursions can be transcribed into LAMBDA with the aid of the ordered tuples of (2.22), (2.23) above. But we can go further and connect with partial (and general) recursive functions. We state first a definition.

primRecComb.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb
public import DomainTheory.DataTypesAsLattices.Lambda.dollarC
public import DomainTheory.DataTypesAsLattices.Lambda.primRecStepTerm
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Lambda.primRecHat
public import DomainTheory.DataTypesAsLattices.Lambda.primRecVal
public import DomainTheory.DataTypesAsLattices.Lambda.primRecStep
public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb_more
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_27
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

/-- Scott (2.27) packaged as a combinator. -/
def primRecComb (a f : Pomega) : Pomega :=
  funOf dollarC
    (funOf Ycomb
      (funOf (funOf (interp primRecStepTerm (fun _ => botElem)) a) f))

namespace primRecComb

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem combinatory {a f : Pomega}
    (ha : IsCombinatory a) (hf : IsCombinatory f) :
    IsCombinatory (primRecComb a f) :=
  .app dollarC.combinatory <|
    .app Ycomb.combinatory <|
      .app (.app (theorem_2_4.closed primRecStepTerm) ha) hf

theorem ofNat (a f : Pomega) (n : Nat) :
    funOf (primRecComb a f) (_root_.DomainTheory.DataTypesAsLattices.ofNat n) =
```

```
      primRecVal a f n := by
  have hgraph := primRecStep.graph_of a f
  have hfix := theorem_2_5 (primRecStep.isScottContinuous a f)
  simp only [primRecComb]
  rw [ <-  hgraph, hfix,  <-  _root_.DomainTheory.DataTypesAsLattices.eq_2_27,
    primRecHat.ofNat]

end primRecComb

end DomainTheory.DataTypesAsLattices
```

primRecOpC.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.primRecOpEnv
public import DomainTheory.DataTypesAsLattices.Computability.primRecOpTerm
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Graph.Basic.notation3
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Computability.primRecComb
public import DomainTheory.DataTypesAsLattices.Lambda.envSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofComb

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

def primRecOpC : Pomega := interp primRecOpTerm primRecOpEnv

namespace primRecOpC

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem app (a f : Pomega) :
    (primRecOpC ? a) ? f = primRecComb a f := by
  rw [primRecOpC, primRecOpTerm, theorem_2_2.beta, theorem_2_2.beta]
  simp [interp, envSet, Function.update, primRecOpEnv, primRecComb]

theorem combinatory : IsCombinatory primRecOpC := by
  rw [primRecOpC, theorem_2_4.complete]
  exact ofComb.combinatory _ _ primRecOpEnv.combinatory
```

```
end primRecOpC

end DomainTheory.DataTypesAsLattices
```

primRecOpEnv.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb
public import DomainTheory.DataTypesAsLattices.Lambda.dollarC
public import DomainTheory.DataTypesAsLattices.Lambda.primRecStepTerm
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Computability.primRecComb
public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb_more
public import DomainTheory.DataTypesAsLattices.Computability.botElem_combinatory
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

/-- Curried operator `fun a f. primRecComb a f`. -/
def primRecOpEnv : Nat  ->  Pomega
  | 2 => dollarC
  | 3 => Ycomb
  | 4 => interp primRecStepTerm (fun _ => botElem)
  | _ => botElem

namespace primRecOpEnv

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem combinatory : forall  i, IsCombinatory (primRecOpEnv i)
  | 2 => dollarC.combinatory
  | 3 => Ycomb.combinatory
  | 4 => theorem_2_4.closed primRecStepTerm
  | 0 | 1 => botElem_combinatory
  | _ + 5 => botElem_combinatory

end primRecOpEnv

end DomainTheory.DataTypesAsLattices
```

primRecOpTerm.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

def primRecOpTerm : Term :=
  .lam 0 (.lam 1
    (.app (.var 2)
      (.app (.var 3) (.app (.app (.var 4) (.var 0)) (.var 1)))))

end DomainTheory.DataTypesAsLattices
```

primRecVal_add.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.primRecVal
public import DomainTheory.DataTypesAsLattices.Lambda.sucC
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Lambda.succSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Computability.addStepC
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb_more
public import DomainTheory.DataTypesAsLattices.Lambda.sucC_more
public import DomainTheory.DataTypesAsLattices.Lambda.succSet_more

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem primRecVal_add (k : Nat) : forall  n,
    primRecVal (ofNat k) addStepC n = ofNat (k + n)
```

```
  | 0 => rfl
  | n + 1 => by
    calc primRecVal (ofNat k) addStepC (n + 1)
        = funOf (funOf addStepC (ofNat n))
            (primRecVal (ofNat k) addStepC n) := rfl
      _ = funOf sucC (primRecVal (ofNat k) addStepC n) := by
          simp [addStepC, Kcomb.beta2]
      _ = funOf sucC (ofNat (k + n)) := by rw [primRecVal_add k n]
      _ = ofNat (k + n + 1) := by rw [sucC.app, succSet.ofNat]

end DomainTheory.DataTypesAsLattices
```

primRecVal_mul.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.primRecVal
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Computability.addC
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb_more
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem primRecVal_mul (k : Nat) : forall  n,
    primRecVal (ofNat 0) (funOf Kcomb (funOf addC (ofNat k))) n = ofNat (k * n)
  | 0 => by simp [primRecVal]
  | n + 1 => by
    calc primRecVal (ofNat 0) (funOf Kcomb (funOf addC (ofNat k))) (n + 1)
        = funOf (funOf (funOf Kcomb (funOf addC (ofNat k))) (ofNat n))
            (primRecVal (ofNat 0) (funOf Kcomb (funOf addC (ofNat k))) n) :=
          rfl
      _ = funOf (funOf addC (ofNat k))
            (ofNat (k * n)) := by
          rw [Kcomb.beta2, primRecVal_mul k n]
      _ = ofNat (k + k * n) := addC.realizes2 k (k * n)
      _ = ofNat (k * (n + 1)) := by rw [Nat.mul_succ, Nat.add_comm]

end DomainTheory.DataTypesAsLattices
```

primRecZeroC.lean

```
module

public import Mathlib.Computability.RE
```

```
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb
public import DomainTheory.DataTypesAsLattices.Lambda.dollarC
public import DomainTheory.DataTypesAsLattices.Lambda.primRecStepTerm
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.Basic.notation3
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Computability.primRecComb
public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb_more
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat_more
public import DomainTheory.DataTypesAsLattices.Computability.SK_comp
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

/-- Primitive recursion with zero initial value, viewed as an operator on its step. -/
def primRecZeroC : Pomega :=
  Scomb ? (Kcomb ? dollarC) ?
    (Scomb ? (Kcomb ? Ycomb) ?
      (interp primRecStepTerm (fun _ => botElem) ? ofNat 0))

namespace primRecZeroC

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem app (f : Pomega) :
    primRecZeroC ? f = primRecComb (ofNat 0) f := by
  rw [primRecZeroC, SK_comp, SK_comp]
  rfl

theorem combinatory : IsCombinatory primRecZeroC :=
  .app (.app .S (.app .K dollarC.combinatory))
    (.app (.app .S (.app .K Ycomb.combinatory))
      (.app (theorem_2_4.closed primRecStepTerm) (ofNat.combinatory 0)))

end primRecZeroC

end DomainTheory.DataTypesAsLattices
```

primRecVal_sub.lean

```
module
```

```
public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.primRecVal
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Computability.safePredC
public import DomainTheory.DataTypesAsLattices.Computability.subStepC
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb_more
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem primRecVal_sub (k : Nat) : forall  n,
    primRecVal (ofNat k) subStepC n = ofNat (k - n)
  | 0 => by simp [primRecVal]
  | n + 1 => by
    calc primRecVal (ofNat k) subStepC (n + 1)
        = funOf (funOf subStepC (ofNat n))
            (primRecVal (ofNat k) subStepC n) := rfl
      _ = funOf safePredC (primRecVal (ofNat k) subStepC n) := by
          simp [subStepC, Kcomb.beta2]
      _ = funOf safePredC (ofNat (k - n)) := by rw [primRecVal_sub k n]
      _ = ofNat (k - (n + 1)) := by
          cases h : k - n with
          | zero =>
            have : k - (n + 1) = 0 := by omega
            simp [safePredC.zero, this]
          | succ k' =>
            have : k - (n + 1) = k' := by omega
            simp [safePredC.succ, this]

end DomainTheory.DataTypesAsLattices
```

primRecVal_sqrt.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.primRecVal
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Computability.pack4
public import DomainTheory.DataTypesAsLattices.Computability.sqrtIter
public import DomainTheory.DataTypesAsLattices.Computability.sqrtStepC
public import DomainTheory.DataTypesAsLattices.Graph.Basic.notation3
@[expose] public section

set_option maxHeartbeats 2000000
```

```
namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem primRecVal_sqrt (n : Nat) : forall  k,
    primRecVal (ofNat 0) (sqrtStepC ? ofNat n) k = ofNat (sqrtIter n k)
  | 0 => rfl
  | k + 1 => by
      rw [show primRecVal (ofNat 0) (sqrtStepC ? ofNat n) (k + 1) =
        ((sqrtStepC ? ofNat n) ? ofNat k) ?
          primRecVal (ofNat 0) (sqrtStepC ? ofNat n) k by rfl,
        primRecVal_sqrt n k, sqrtStepC.app]
      rfl

end DomainTheory.DataTypesAsLattices
```

primRecHat.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.primRecVal
public import Mathlib.Data.Set.Lattice.Indexed
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def primRecHat (a f x : Pomega) : Pomega :=
  ? n  in  x, primRecVal a f n

namespace primRecHat

theorem zero (a f : Pomega) : primRecHat a f (ofNat 0) = a := by
  ext k
  constructor
  * intro hk
    obtain <n, hn, hkn> := Set.mem_iUnion_2.mp hk
    subst hn; simpa [primRecVal] using hkn
  * intro hk
    exact Set.mem_iUnion_2.mpr <0, rfl, hk>

/-- **Scott 1976, (2.27).** The distributive extension agrees with the
integer recursion on singletons. -/
theorem ofNat (a f : Pomega) : forall  n,
    primRecHat a f (ofNat n) = primRecVal a f n
  | 0 => primRecHat.zero a f
  | n + 1 => by
    ext k
    constructor
```

```
    * intro hk
      obtain <m, hm, hkm> := Set.mem_iUnion_2.mp hk
      subst hm; simpa [primRecVal] using hkm
    * intro hk
      exact Set.mem_iUnion_2.mpr <n + 1, rfl, hk>

end primRecHat

end DomainTheory.DataTypesAsLattices
```

primRecVal.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_4
public import DomainTheory.DataTypesAsLattices.Lambda.primRecStep
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_7_zero
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_7_succ
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.predSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.27).** Primitive recursion as a distributive function. -/
def primRecVal (a f : Pomega) : Nat  ->  Pomega
  | 0 => a
  | n + 1 => funOf (funOf f (ofNat n)) (primRecVal a f n)

namespace primRecVal

/-- **Scott 1976, (2.27).** The fixed point of the primrec step agrees
with integer recursion. -/
theorem fix (a f : Pomega) : forall  n,
    funOf (DomainTheory.DataTypesAsLattices.fix (primRecStep a f)) (ofNat n) =
      primRecVal a f n
  | 0 => by
    have hfix := (theorem_1_4 (primRecStep.isScottContinuous a f)).1
    calc funOf (DomainTheory.DataTypesAsLattices.fix (primRecStep a f)) (ofNat 0)
        = funOf (primRecStep a f
            (DomainTheory.DataTypesAsLattices.fix (primRecStep a f))) (ofNat 0) := by
          rw [hfix]
      _ = condSet (ofNat 0) a
            (funOf (funOf f (predSet (ofNat 0)))
              (funOf (DomainTheory.DataTypesAsLattices.fix (primRecStep a f))
                (predSet (ofNat 0)))) :=
          primRecStep.app a f _ _
      _ = a := eq_2_7_zero _ _
```

```
  | n + 1 => by
    have hfix := (theorem_1_4 (primRecStep.isScottContinuous a f)).1
    calc funOf (DomainTheory.DataTypesAsLattices.fix (primRecStep a f)) (ofNat (n + 1))
    )
        = funOf (primRecStep a f
            (DomainTheory.DataTypesAsLattices.fix (primRecStep a f))) (ofNat (n + 1))
    := by
          rw [hfix]
      _ = condSet (ofNat (n + 1)) a
            (funOf (funOf f (predSet (ofNat (n + 1))))
              (funOf (DomainTheory.DataTypesAsLattices.fix (primRecStep a f))
                (predSet (ofNat (n + 1))))) :=
            primRecStep.app a f _ _
      _ = funOf (funOf f (ofNat n))
            (funOf (DomainTheory.DataTypesAsLattices.fix (primRecStep a f)) (ofNat n))
     := by
          rw [eq_2_7_succ, predSet.ofNat_succ]
      _ = funOf (funOf f (ofNat n)) (primRecVal a f n) := by
          rw [?fix? a f n]

end primRecVal

end DomainTheory.DataTypesAsLattices
```

eq_2_27.lean

```
module

public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.dollarC
public import DomainTheory.DataTypesAsLattices.Lambda.primRecVal
public import DomainTheory.DataTypesAsLattices.Lambda.primRecHat
public import DomainTheory.DataTypesAsLattices.Lambda.primRecStep
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_25
public import Mathlib.Data.Set.Lattice.Indexed
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.27).** `p = Y(fun u fun n in ?. n ? a, f(n?1)(u(n?1)))`. -/
theorem eq_2_27 (a f x : Pomega) :
    primRecHat a f x = funOf (funOf dollarC (fix (primRecStep a f))) x := by
  rw [eq_2_25]
  ext k
  constructor
  * intro hk
    obtain <n, hn, hkn> := Set.mem_iUnion_2.mp hk
    refine Set.mem_iUnion_2.mpr <n, hn, ?_>
    simpa [primRecVal.fix] using hkn
  * intro hk
    obtain <n, hn, hkn> := Set.mem_iUnion_2.mp hk
    refine Set.mem_iUnion_2.mpr <n, hn, ?_>
    simpa [primRecVal.fix] using hkn
```

```
end DomainTheory.DataTypesAsLattices
```

2.3.15 Definition (computable)

DEFINITION. A continuous function f of k-variables is *computable* iff the relationship

$$m \in f(e_{n_0})(e_{n_1}) \cdots (e_{n_{k-1}})$$

is recursively enumerable (r.e.) in the integer variables $m, n_0, n_1, \ldots, n_{k-1}$.

If q is a partial recursive function in the usual sense, then we can regard it as a mapping $q : \omega \to \omega \cup \{\bot\}$, where $q(n) = \bot$ means that q is undefined at n. Saying that q is partial recursive is just to say that $m \in q(n)$ is r.e. as a relation in n and m. It is easy to see that this is in turn equivalent to the recursive enumerability of the relationship $m \in \hat{q}(e_n)$; and so our definition is formally a generalization of the usual one. But it is also intuitively reasonable. To "compute" $y = f(x)$, in the one-variable case, we proceed by enumeration. First we begin the enumeration of all finite subsets $e_n \subseteq x$. For each of these f starts up an enumeration of the set $f(e_n)$; so we sit back and observe which $m \in f(e_n)$ by enumeration. The totality of all such m for all $e_n \subseteq x$ forms in the end the set y.

2.3.16 Computable functions

`LambdaDefinable.lean`

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

/-- Closed LAMBDA-definable elements (empty environment). -/
class LambdaDefinable (u : Pomega) : Prop where
  out : exists  t : Term, interp t (fun _ => botElem) = u

end DomainTheory.DataTypesAsLattices
```

(Already included: `1976_Definition_computable.md`.)

2.3.17 Definability

As a hint for the proof we may note that the method of (2.27) shows that all primitive recursive functions p have the corresponding $\hat{p}$ LAMBDA-definable. Next we remark that a nonempty r.e. set is the range of a primitive recursive function; but the range of p is $\hat{p}(\top)$, which is clearly LAMBDA-definable. That any LAMBDA-definable set (graph) is r.e. is obvious from the definition of the language itself.

Realizes.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

/-- A combinator realizes a number-theoretic function on numerals. -/
def Realizes (u : Pomega) (p : Nat  ->  Nat) : Prop :=
  forall  n, funOf u (ofNat n) = ofNat (p n)

end DomainTheory.DataTypesAsLattices
```

unpairSndTerm.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Computability.unpairSqrtTerm
public import DomainTheory.DataTypesAsLattices.Computability.unpairDeltaTerm
public import DomainTheory.DataTypesAsLattices.Computability.unpairTestTerm
public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

def unpairSndTerm : Term :=
  .lam 0 (.cond unpairTestTerm
    (.app (.app (.var 2) unpairDeltaTerm) unpairSqrtTerm)
    unpairSqrtTerm)

end DomainTheory.DataTypesAsLattices
```

Theorem 2.6 **THEOREM 2.6 (The definability theorem).** *For a k-ary continuous function f, the following are equivalent:*

(i) f is computable;

(ii) $\lambda x_0 \lambda x_1 \cdots \lambda x_{k-1}.\, f(x_0)(x_1)\cdots(x_{k-1})$ as a set is r.e.;

(iii) $\lambda x_0 \lambda x_1 \cdots \lambda x_{k-1}.\, f(x_0)(x_1)\cdots(x_{k-1})$ is LAMBDA-definable.

As a hint for the proof we may note that the method of (2.27) shows that all primitive recursive functions p have the corresponding $\hat{p}$ LAMBDA-definable. Next we remark that a nonempty r.e. set is the range of a primitive recursive function; but the range of p is $\hat{p}(\top)$, which is clearly LAMBDA-definable. That any LAMBDA-definable set (graph) is r.e. is obvious from the definition of the language itself.

We may draw some interesting conclusions from the definability theorem. In the first place, we see that the countable collection $\mathbf{RE} \subseteq \mathbf{P}\omega$ of r.e. sets is closed under application and LAMBDA-definability. Indeed it forms a model for the λ-calculus (axioms (α), (β), and (ξ^*) at least) and it also contains the arithmetical combinators. (Clearly there will be many intermediate submodels.) In the second place, we can see now how very easy it is to interpret λ-calculus in ordinary arithmetical recursion theory by means of quite elementary operations on r.e. sets. Thus the equivalence of λ-definability with partial recursiveness seems not to be all that good a piece of evidence for Church's Thesis. In his 1936 paper (a footnote on p. 346) Church says about λ-definability:

> The fact, however, that two such widely different and (in the opinion of the author) equally natural definitions of effective calculability turn out to be equivalent adds to the strength of the reasons adduced below for believing that they constitute as general a characterization of this notion as is consistent with the usual intuitive understanding of it.

The point never struck the present author as an especially telling one, and the reduction of λ-calculus to r.e. theory shows that the divergence between the theories is not at all wide. Of course it is a pleasant surprise to see how many complicated things can be defined in pure λ-calculus (without arithmetical combinators), but this fact cuts the wrong way as evidence for the thesis (we want stronger theories, not weaker ones). Post systems (or even first-order theories) are much better to mention in this connection, since they are obviously more inclusive in giving enumerations than Turing machines or Herbrand–Gödel recursion equations. But the equivalence proofs are all so easy! What one would like to see is a "natural" definition where the equivalence with r.e. is not just a mechanical exercise involving a few tricks of coding.

In the course of the development in this section we have stated many equations which are not found in Table 1, and which involve new combinators. In conclusion we would like to mention an equation about $\mathbf{Y}$ which holds in the model, which can be stated in pure λ-calculus, and which cannot be proved by ordinary reduction (though we shall not try to justify this last statement here). In order to shorten the calculations, we note from definition (2.8) that $\mathbf{Y}(u) = \mathbf{Y}(\lambda y.\, u(y))$; so by Theorem 2.5 this also equals $u(\mathbf{Y}(u))$.

$$(2.28) \quad \mathbf{Y}(\lambda f \lambda x.\, g(x)(f(x))) = \lambda x.\, \mathbf{Y}(g(x)).$$

Call the left side f' and the right f''. Now $f'' = \lambda x.\, g(x)(\mathbf{Y}(g(x))) = \lambda x.\, g(x)(f''(x))$, thus $f' \subseteq f''$, because f' is a least fixed point. On the other hand $f' = \lambda x.\, g(x)(f'(x))$, so $f'(x) = g(x)(f'(x))$. Thus $f''(x) \subseteq f'(x)$, because $f''(x)$ is a least fixed point. As this holds for all x, we see that $f'' \subseteq f'$; and so they are equal. There must be many other such equations.

Motivation The three conditions meet as follows. Passing from a computable f to its iterated graph replaces $m \in f(e_{n_0})(e_{n_1})\cdots(e_{n_{k-1}})$ by membership of the iterated pair $(n_0, (n_1, (\cdots, (n_{k-1}, m)\cdots)))$

in that graph, so the function is computable exactly when the graph is r.e. The hint above shows that an r.e. graph is LAMBDA-definable. In the other direction each of the six combinators of Theorem 2.4 is r.e., and application preserves r.e. sets: $m \in u(x)$ is an existential quantification over a finite $e_n \subseteq x$ together with a bounded universal quantification, and membership in u and in x occurs positively.

theorem_2_6.lean

```
module

public import Mathlib.Computability.RE
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.theorem_2_4
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Computability.IsRE
public import DomainTheory.DataTypesAsLattices.Computability.IsComputable
public import DomainTheory.DataTypesAsLattices.Computability.LambdaDefinable
public import DomainTheory.DataTypesAsLattices.Computability.ComputableFin
public import DomainTheory.DataTypesAsLattices.Computability.
    combinatory_isLambdaDefinable

public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Lambda.zeroC
public import DomainTheory.DataTypesAsLattices.Lambda.ScottContinuousFin
public import DomainTheory.DataTypesAsLattices.Lambda.curryFin
public import DomainTheory.DataTypesAsLattices.Lambda.envSet
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Lambda.theorem_2_1
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Enumeration.Avoids
public import DomainTheory.DataTypesAsLattices.Enumeration.fin
public import DomainTheory.DataTypesAsLattices.Computability.IsREu2eiff_combinatory
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, Theorem 2.6 (The definability theorem), unary case.**
For a continuous `f`, computability, recursive enumerability of the graph,
combinatory definability, and closed LAMBDA-definability agree. -/
theorem theorem_2_6 {f : Pomega  ->  Pomega} (hf : IsScottContinuous f) :
    (IsComputable f  <->  IsRE (graph f))  /\
      (IsRE (graph f)  <->  IsCombinatory (graph f))  /\
      (LambdaDefinable (graph f)  <->  IsCombinatory (graph f)) := by
  refine <?_, IsRE.iff_combinatory, ?_>
  * exact <fun h => h.2, fun h => <hf, h>>
  * exact <fun h => by
      obtain <t, ht> := h.out
      simpa [ht] using theorem_2_4.closed t,
      combinatory_isLambdaDefinable>
```

```
namespace theorem_2_6

/-- Combinatory elements are generated from the six constants, matching
the LAMBDA-definable closed terms of Theorem 2.4. -/
theorem combinatory_zero : LambdaDefinable zeroC :=
  <<Term.zero, rfl>>

/-- **Scott 1976, Theorem 2.6 (The definability theorem).**
For a `k`-ary continuous `f`, computability of the finite-argument relation,
recursive enumerability of the nested graph, combinatory definability, and
closed LAMBDA-definability agree. -/
theorem fin {n : Nat} {f : (Fin n  ->  Pomega)  ->  Pomega}
    (hf : ScottContinuousFin f) :
    (ComputableFin f  <->  IsRE (curryFin f))  /\
      (IsRE (curryFin f)  <->  IsCombinatory (curryFin f))  /\
      (LambdaDefinable (curryFin f)  <->  IsCombinatory (curryFin f)) := by
  refine <?_, IsRE.iff_combinatory, ?_>
  * exact <fun h => h.out.2, fun h => <<hf, h>>>
  * exact <fun h => by
      obtain <t, ht> := h.out
      simpa [ht] using theorem_2_4.closed t,
      combinatory_isLambdaDefinable>

/-- LAMBDA-definable maps (in one free variable) are continuous, so
clause (iii) of Theorem 2.6 lands in the setting of (i)-(ii). -/
theorem lambda_continuous (t : Term) :
    IsScottContinuous (fun v => interp t (envSet (fun _ => botElem) 0 v)) :=
  theorem_2_1 t (fun _ => botElem) 0

/-- **Scott 1976, Theorem 2.6, r.e. direction for numerals.** -/
theorem ofNat_isRE (n : Nat) : IsCombinatory (ofNat n)  ->  IsRE (ofNat n) :=
  fun _ => IsRE.numeral n

end theorem_2_6

end DomainTheory.DataTypesAsLattices
```

(Already included: `1976_Theorem_2_6.md`.)

2.3.18 A law of Y

In the course of the development in this section we have stated many equations which are not found in Table 1, and which involve new combinators. In conclusion we would like to mention an equation about $\mathbf{Y}$ which holds in the model, which can be stated in pure λ-calculus, and which cannot be proved by ordinary reduction (though we shall not try to justify this last statement here). In order to shorten the calculations, we note from definition (2.8) that $\mathbf{Y}(u) = \mathbf{Y}(\lambda y.\, u(y))$; so by Theorem 2.5 this also equals $u(\mathbf{Y}(u))$.

$$(2.28) \quad \mathbf{Y}(\lambda f \lambda x.\, g(x)(f(x))) = \lambda x.\, \mathbf{Y}(g(x)).$$

Call the left side f' and the right f''. Now $f'' = \lambda x.\, g(x)(\mathbf{Y}(g(x))) = \lambda x.\, g(x)(f''(x))$, thus $f' \subseteq f''$, because f' is a least fixed point. On the other hand $f' = \lambda x.\, g(x)(f'(x))$, so $f'(x) = g(x)(f'(x))$. Thus $f''(x) \subseteq f'(x)$, because $f''(x)$ is a least fixed point. As this holds for all x, we see that $f'' \subseteq f'$; and so they are equal. There must be many other such equations.

eq_2_28.lean

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_4
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.Ymap
public import DomainTheory.DataTypesAsLattices.Lambda.Fofg
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.FixedPoint.iterateBot
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.lam
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.28).** Least-fixed-point unfolding `Y(g(x)) = g(x)(Y(g(x)))`. -/
theorem eq_2_28 {g : Pomega  ->  Pomega  ->  Pomega}
    (hgy : forall  x, IsScottContinuous (fun y => g x y)) (x : Pomega) :
    fix (g x) = g x (fix (g x)) :=
  (theorem_1_4 (hgy x)).1.symm

namespace eq_2_28

/-- **Scott 1976, (2.28).** `Y(fun f fun x. g(x)(f(x))) = fun x. Y(g(x))`. -/
theorem commute {g : Pomega  ->  Pomega  ->  Pomega}
    (hgx : forall  y, IsScottContinuous (fun x => g x y))
    (hgy : forall  x, IsScottContinuous (fun y => g x y)) :
    fix (Fofg g) = Ymap g := by
  have hF : IsScottContinuous (Fofg g) :=
    graph_const_isScottContinuous (fun f x => g x (funOf f x))
      (fun x => theorem_1_3 (hgy x) (funOf.isScottContinuous_left x))
  have hfix : Fofg g (Ymap g) = Ymap g := by
    apply congrArg graph
    funext x
    rw [Ymap.app hgx hgy]
    exact (theorem_1_4 (hgy x)).1
  have hle : fix (Fofg g)  subseteq  Ymap g := (theorem_1_4 hF).2 (Ymap g) hfix
  refine subset_antisymm hle ?_
  have hiter : forall  n x, funOf (iterateBot (Fofg g) n) x = iterateBot (g x) n := by
    intro n
    induction n with
    | zero =>
      intro x
      ext m
```

```
      simp [iterateBot, funOf, botElem]
    | succ n ih =>
      intro x
      have hcont : IsScottContinuous
          (fun z => g z (funOf (iterateBot (Fofg g) n) z)) :=
        theorem_1_3.tuple hgx hgy id_isScottContinuous
          (funOf.isScottContinuous (iterateBot (Fofg g) n))
      change funOf (lam (fun z => g z (funOf (iterateBot (Fofg g) n) z))) x =
        g x (iterateBot (g x) n)
      rw [beta hcont, ih]
  intro p hp
  rcases hp with <n, m, rfl, hm>
  obtain <k, hkm> := Set.mem_iUnion.mp hm
  cases k with
  | zero =>
    simp [iterateBot, botElem] at hkm
  | succ k =>
    have : pair n m  in  iterateBot (Fofg g) (k + 1) :=
      <n, m, rfl, by
        change m  in  g (e n) (funOf (iterateBot (Fofg g) k) (e n))
        rw [hiter]
        exact hkm>
    exact Set.mem_iUnion.mpr <k + 1, this>

end eq_2_28

end DomainTheory.DataTypesAsLattices
```

iterateBot_param_isScottContinuous.lean

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.iterateBot
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.id_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.const_isScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem iterateBot_param_isScottContinuous
    {g : Pomega  ->  Pomega  ->  Pomega}
    (hgx : forall  y, IsScottContinuous (fun x => g x y))
    (hgy : forall  x, IsScottContinuous (fun y => g x y)) :
    forall  n, IsScottContinuous (fun x => iterateBot (g x) n)
  | 0 => const_isScottContinuous botElem
  | n + 1 =>
    theorem_1_3.tuple (hgx) (hgy) id_isScottContinuous
```

```
      (iterateBot_param_isScottContinuous hgx hgy n)

end DomainTheory.DataTypesAsLattices
```

fix_param_isScottContinuous.lean

```
module

public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.iterateBot
public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Continuous.scottUnion
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.
    iterateBot_param_isScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem fix_param_isScottContinuous
    {g : Pomega  ->  Pomega  ->  Pomega}
    (hgx : forall  y, IsScottContinuous (fun x => g x y))
    (hgy : forall  x, IsScottContinuous (fun y => g x y)) :
    IsScottContinuous (fun x => fix (g x)) := by
  constructor
  intro x
  ext k
  constructor
  * intro hk
    obtain <n, hkn> := Set.mem_iUnion.mp hk
    have := (iterateBot_param_isScottContinuous hgx hgy n).out x
    have : k  in  scottUnion (fun z => iterateBot (g z) n) x := by
      rwa [ <-  this]
    obtain <m, hm, hkm> := mem_scottUnion.mp this
    exact mem_scottUnion.mpr <m, hm, Set.mem_iUnion.mpr <n, hkm>>
  * intro hk
    obtain <m, hm, hkm> := mem_scottUnion.mp hk
    obtain <n, hkn> := Set.mem_iUnion.mp hkm
    exact Set.mem_iUnion.mpr <n,
      IsScottContinuous.monotone (iterateBot_param_isScottContinuous hgx hgy n) hm hkn
    >

end DomainTheory.DataTypesAsLattices
```

Ymap.lean

```
module
```

```
public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.fix_param_isScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
def Ymap (g : Pomega  ->  Pomega  ->  Pomega) : Pomega :=
  graph (fun x => fix (g x))

namespace Ymap

theorem app {g : Pomega  ->  Pomega  ->  Pomega}
    (hgx : forall  y, IsScottContinuous (fun x => g x y))
    (hgy : forall  x, IsScottContinuous (fun y => g x y)) (x : Pomega) :
    funOf (Ymap g) x = fix (g x) :=
  beta (fix_param_isScottContinuous hgx hgy) x

end Ymap

end DomainTheory.DataTypesAsLattices
```

2.4 Retracts and data types

Historical note. A retract is an ordinary notion of topology. The use of computable retracts as data types, the operations on those retracts, and the recursive definitions by fixed-point equations are Scott's. A continuous lattice, in the sense of Scott (1972), has continuous meet and join, and as a space it is a T_0-space satisfying the extension theorem. The connection between lattices and fixed points was already known. The set of all retracts is a complete lattice by Theorem 4.1, and it is not itself a continuous lattice. Ershov solved that question after the body of the paper was written; Hosono and Sato (1975) found essentially the same argument independently. The argument is given with the ordering of retracts. The correspondence was Scott's introduction to Ershov's numeration theory. Applications to the semantics of flow diagrams are in Scott (1971), and were taken up by Goguen et al. (1975) and by Reynolds (1974).

Dependencies. Inside Retracts, each box is one Lean file. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. The arrowhead points at the file or folder that imports, and the number is how many direct imports.

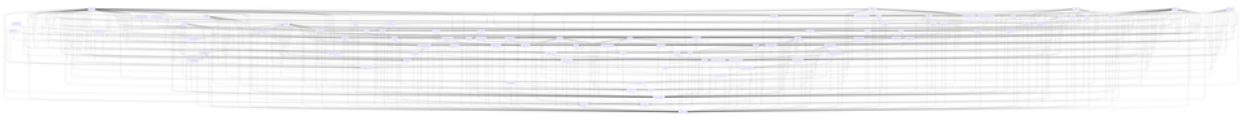


Figure 24: Retracts and data types.

Data can be structured in many ways: ordered tuples, lists, arrays, trees, streams, and even operations and functions. The last point becomes clear if one thinks of *parameters*. We would normally hardly consider the pairing function $\lambda x \lambda y.\, \langle x, y \rangle$ as being in itself a piece of data. But if we treat the first variable as a *parameter*, then it can be specialized to a *fixed value*, say the element a, producing the function $\lambda y.\, \langle a, y \rangle$. This function is more likely to be the output of some process and *in itself* can be considered as a datum. It is rather like one *whole row* of a matrix. If we were to regard a two-argument function f as being a matrix, then its ath row would be exactly $\lambda y.\, f(a)(y)$. If s were a selection function, then, for example, $\lambda y.\, f(s(y))(y)$ would represent the selection of one element out of each column of the matrix. This selection could be taken as a specialization of parameters in the operator $\lambda u \lambda v \lambda y.\, u(v(y))(y)$. We have not been very definite here about the exact nature of the fixed a, f, or s, or the range of the variable y or the range of values of the function f. The point is only to recall a few elements of structure and to suggest an abstract view of data going beyond the usual iterated arrays and trees.

What then is a *data type*? Answer: a type of data. That is to say, a collection of data that have been grouped together for reasons of similarity of structure or perhaps mere convenience. Thus the collection may very well be a mixed bag, but more often than not canons of taste or demands of simplicity dictate an adherence to regularity. The grouping may be formed to eliminate irrelevant objects and focus the attention in other ways. It is frequently a matter of good organization that aids the understanding of complex definitions. In programming languages, one of the major reasons for making an explicit declaration of a data type (that is, the restriction of certain variables to certain *"modes"*) is that the computed objects of that type can enjoy a special *representation* in the machine that allows the manipulation of these objects via the chosen representation to be reasonably efficient. This is a very critical matter for good language design and good compiler writing. In this report, however, we cannot discuss the problems of representation, important as they may be. Our objective here is conceptual organization, and we wish to show how such ideas, in the language for computable functions used here, can find the proper expression.

Which are the data types that can be defined in LAMBDA? No final answer can be given since the number is infinite and inexhaustible. From one point of view, however, there is only one: $\mathbf{P}\omega$ itself. It is the universal type and all other types are subtypes of it; so $\mathbf{P}\omega$ plays a primary role in this exposition. But in a way it is too big, or at least too complex, since each of its elements can be used in so many different ways. When we specify a subtype the intention is to restrict attention to a special use. But even the various subtypes overlap, and so the same elements still get different uses. Style in writing definitions will usually make the differentiation clear though. The main innovation to be described in this section is the use of LAMBDA expressions to define types *as well as* elements. Certain expressions define *retracts* (or better: retraction mappings), and it is the *ranges* (or as we shall see: *sets of finite points*) of such retracts that form the groupings into types. Thus LAMBDA provides a calculus of type definitions including *recursive* type definitions. Examples will be explained both here and in the following sections. Note that types as retracts turn out to be types as *lattices*, that is, types of partial and many-valued objects. The problem of cutting these lattice types down to the perfect or complete objects is discussed in § 6. Another view of types and functionality of mappings is presented in § 7.

The notion of a retract comes from (analytic) topology, but it seems almost an accident that the idea can be applied in the present context. The word is employed not because there is some deep tie-up with topology but because it is short and rather descriptive. Three easy examples will motivate the general plan:

$$(4.1) \quad \mathbf{fun} = \lambda u \lambda x.\, u(x);$$

$$(4.2) \quad \mathbf{pair} = \lambda u.\, \langle u_0, u_1 \rangle;$$

$$(4.3) \quad \mathbf{bool} = \lambda u.\, u \sqsupset 0, 1.$$

Here $\sqsupset$ is the *doubly strict conditional* defined by

$$(4.4) \quad z \sqsupset x, y = z \supset (z \supset x, \top), (z \supset \top, y),$$

which has the property that if z is both zero and positive, then it takes the value $\top$ instead of the value $x \cup y$.

- Definition (retract)
- Definition (membership)
- Theorem 4.1
- Definition (ordering)
- Theorem 4.2
- Theorem 4.3
- Theorem 4.4
- Theorem 4.5
- Theorem 4.6

DirectedSet.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- A nonempty directed subset of a preorder. -/
class DirectedSet {a : Type u} [Preorder a] (s : Set a) : Prop where
  out : s.Nonempty  /\
    forall  ?x?, x  in  s  ->  forall  ?y?, y  in  s  ->
      exists  z  in  s, x  <=  z  /\  y  <=  z

end DomainTheory.DataTypesAsLattices
```

Icomb.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
```

```
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.id_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Retracts.Retract

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- The identity combinator `I = fun u. u`, the largest retract. -/
def Icomb : Pomega := graph (fun x => x)

namespace Icomb

theorem app (x : Pomega) : funOf Icomb x = x :=
  beta id_isScottContinuous x

theorem isRetract : Retract Icomb := by
  constructor
  change Icomb = graph (fun x => funOf Icomb (funOf Icomb x))
  have : (fun x => funOf Icomb (funOf Icomb x)) = fun x => x := by
    funext x
    simp [app]
  rw [this]
  rfl

end Icomb

end DomainTheory.DataTypesAsLattices
```

DirectedSubset.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.Fixpoints
public import DomainTheory.DataTypesAsLattices.Retracts.vals

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Directedness of fixed points under subset, without a lattice instance. -/
class DirectedSubset {f : Pomega  ->  Pomega} (s : Set (Fixpoints f)) : Prop where
  out : s.Nonempty  /\
    forall  ?x?, x  in  s  ->  forall  ?y?, y  in  s  ->
      exists  z  in  s, (x : Pomega)  subseteq  z  /\  (y : Pomega)  subseteq  z

end DomainTheory.DataTypesAsLattices
```

boolR.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet
```

```
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Retracts.dcondSet_bool_idem
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.3).** Boolean retract via the doubly strict conditional. -/
def boolR : Pomega :=
  graph (fun u => dcondSet u (ofNat 0) (ofNat 1))

namespace boolR

theorem app (u : Pomega) :
    funOf boolR u = dcondSet u (ofNat 0) (ofNat 1) :=
  beta (dcondSet.isScottContinuous_left (ofNat 0) (ofNat 1)) u

theorem isRetract : Retract boolR := by
  constructor
  apply graph_ext
  intro u
  rw [app, app, dcondSet_bool_idem]

end boolR

end DomainTheory.DataTypesAsLattices
```

comp.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.5).** Composition `u  comp  v = fun x. u(v(x))`. -/
def comp (u v : Pomega) : Pomega :=
  graph (fun x => funOf u (funOf v x))
```

```
namespace comp

theorem app (u v x : Pomega) :
    funOf (comp u v) x = funOf u (funOf v x) :=
  beta (theorem_1_3 (funOf.isScottContinuous u) (funOf.isScottContinuous v)) x

theorem isScottContinuous_right (u : Pomega) :
    IsScottContinuous (fun v => comp u v) :=
  graph_const_isScottContinuous (fun v x => funOf u (funOf v x))
    (fun x => theorem_1_3 (funOf.isScottContinuous u) (funOf.isScottContinuous_left x)
    )

theorem right_const (a : Pomega) :
    IsScottContinuous (fun u => comp u a) :=
  graph_const_isScottContinuous (fun u x => funOf u (funOf a x))
    (fun x => funOf.isScottContinuous_left (funOf a x))

end comp

end DomainTheory.DataTypesAsLattices
```

arrowR.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.IsStrict
public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.funOf_botElem
public import DomainTheory.DataTypesAsLattices.Retracts.graph_const_bot
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def arrowR (a b : Pomega) : Pomega :=
  graph (fun u => comp b (comp u a))

namespace arrowR

theorem map_isScottContinuous (a b : Pomega) :
    IsScottContinuous (fun u => comp b (comp u a)) :=
  theorem_1_3 (comp.isScottContinuous_right b) (comp.right_const a)

theorem app (a b u : Pomega) :
    funOf (arrowR a b) u = comp b (comp u a) :=
  beta (map_isScottContinuous a b) u
```

```
theorem isStrict {a b : Pomega} (hb : IsStrict b) :
    IsStrict (arrowR a b) := by
  change funOf (arrowR a b) botElem = botElem
  rw [app]
  have hbot : comp botElem a = botElem := by
    unfold comp
    simp [funOf_botElem, graph_const_bot]
  rw [hbot]
  unfold comp
  simp only [funOf_botElem]
  exact (congrArg graph (funext fun _ => hb)).trans graph_const_bot

theorem typed_of {a b u : Pomega} (hu : typed u (arrowR a b)) :
    u = comp b (comp u a) := by
  simpa [typed, app] using hu

end arrowR

end DomainTheory.DataTypesAsLattices
```

arrowR_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.IsStrict
public import DomainTheory.DataTypesAsLattices.Retracts.funOf_botElem
public import DomainTheory.DataTypesAsLattices.Retracts.graph_const_bot
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Retracts.retractLe
public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.eq_4_11
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Retracts.theorem_4_3_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace arrowR
```

```
theorem isRetract {a b : Pomega} (ha : Retract a) (hb : Retract b) :
    Retract (arrowR a b) := by
  constructor
  change arrowR a b =
    graph (fun x => funOf (arrowR a b) (funOf (arrowR a b) x))
  apply graph_ext
  intro u
  rw [theorem_4_3.i ha hb, app]

theorem mono {a b a' b' : Pomega}
    (hab : _root_.DomainTheory.DataTypesAsLattices.retractLe a a')
    (hbb : _root_.DomainTheory.DataTypesAsLattices.retractLe b b') :
    _root_.DomainTheory.DataTypesAsLattices.retractLe (arrowR a b) (arrowR a' b') :=
    by
  constructor
  * have h : arrowR (comp a' a) (comp b b') = arrowR a b := by
      rw [ <-  hab.2,  <-  hbb.1]
    exact (h.symm.trans (eq_4_11.comp a b a' b').symm)
  * have h : arrowR (comp a a') (comp b' b) = arrowR a b := by
      rw [ <-  hab.1,  <-  hbb.2]
    exact (h.symm.trans (eq_4_11.comp a' b' a b).symm)

theorem functor {a b a' b' f f' : Pomega}
    (hf : typed f (arrowR a b)) (hf' : typed f' (arrowR a' b')) :
    typed (arrowR f f') (arrowR (arrowR b a') (arrowR a b')) := by
  change arrowR f f' =
    funOf (arrowR (arrowR b a') (arrowR a b')) (arrowR f f')
  rw [app]
  have inner := eq_4_11.comp f f' b a'
  have outer := eq_4_11.comp a b' (comp b f) (comp f' a')
  have hf1 := typed_of hf
  have hf2 := typed_of hf'
  apply Eq.symm
  calc
    comp (arrowR a b') (comp (arrowR f f') (arrowR b a'))
        = comp (arrowR a b') (arrowR (comp b f) (comp f' a')) := by rw [inner]
    _ = arrowR (comp (comp b f) a) (comp b' (comp f' a')) := outer
    _ = arrowR f f' := by
      apply graph_ext
      intro u
      apply graph_ext
      intro x
      have hfx : funOf f x = funOf b (funOf f (funOf a x)) := by
        simpa [comp.app] using congrArg (fun w => funOf w x) hf1
      have hf'y : forall  y, funOf f' y = funOf b' (funOf f' (funOf a' y)) := by
        intro y
        simpa [comp.app] using congrArg (fun w => funOf w y) hf2
      simp only [comp.app]
      rw [hfx]
      exact (hf'y _).symm

end arrowR

end DomainTheory.DataTypesAsLattices
```

dcondSet_bool_idem.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet_more
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem dcondSet_bool_idem (u : Pomega) :
    dcondSet (dcondSet u (ofNat 0) (ofNat 1)) (ofNat 0) (ofNat 1) =
      dcondSet u (ofNat 0) (ofNat 1) := by
  rcases dcondSet.cases u with h | h | h | h
  * rw [dcondSet.of_zero_not_pos h.1 h.2, dcondSet.ofNat_zero]
  * rw [dcondSet.of_pos_not_zero h.1 h.2, dcondSet.ofNat_one]
  * rw [dcondSet.of_mixed h.1 h.2, dcondSet.top]
  * rw [dcondSet.of_empty h.1 h.2, dcondSet.bot]

end DomainTheory.DataTypesAsLattices
```

gfpBelow.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.Fixpoints
public import DomainTheory.DataTypesAsLattices.Retracts.vals

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Greatest postfixed point of `z  |->  f z  I  b` (Knaster-Tarski inf below `b`). -/
def gfpBelow (f : Pomega  ->  Pomega) (b : Pomega) : Pomega :=
  ?_0 {x | x  subseteq  f x  I  b}

namespace gfpBelow

theorem subset (f : Pomega  ->  Pomega) (b : Pomega) :
    gfpBelow f b  subseteq  b := by
  intro k hk
  obtain <x, hx, hkx> := Set.mem_sUnion.mp hk
  exact (hx hkx).2

theorem postfixed {f : Pomega  ->  Pomega} (hf : IsScottContinuous f)
    (b : Pomega) : gfpBelow f b  subseteq  f (gfpBelow f b)  I  b := by
  intro k hk
```

```
  obtain <x, hx, hkx> := Set.mem_sUnion.mp hk
  have hxz : x  subseteq  gfpBelow f b := fun _ hm => Set.mem_sUnion.mpr <x, hx, hm>
  exact <IsScottContinuous.monotone hf hxz (hx hkx).1, (hx hkx).2>

theorem le {f : Pomega  ->  Pomega} (_hf : IsScottContinuous f) (b : Pomega)
    (_hb : f b  subseteq  b) : gfpBelow f b  subseteq  b :=
  subset f b

theorem fixed_inter {f : Pomega  ->  Pomega} (hf : IsScottContinuous f)
    (b : Pomega) : f (gfpBelow f b)  I  b = gfpBelow f b := by
  apply subset_antisymm
  * intro k hk
    refine Set.mem_sUnion.mpr <f (gfpBelow f b)  I  b, ?_, hk>
    have hle : gfpBelow f b  subseteq  f (gfpBelow f b)  I  b := postfixed hf b
    have : f (gfpBelow f b)  subseteq  f (f (gfpBelow f b)  I  b) :=
      IsScottContinuous.monotone hf hle
    intro m hm
    exact <this hm.1, hm.2>
  * exact postfixed hf b

theorem isGLB {f : Pomega  ->  Pomega} (_hf : IsScottContinuous f)
    {A : Set Pomega} (_hA : A  subseteq  Fixpoints f) {z : Pomega}
    (hz : f z = z) (hlb : forall  x  in  A, z  subseteq  x) :
    z  subseteq  gfpBelow f (?_0 A) := by
  have hzI : z  subseteq  ?_0 A := fun k hk y hy => hlb y hy hk
  have hpost : z  subseteq  f z  I  ?_0 A := fun k hk => <by rwa [hz], hzI hk>
  exact fun k hk => Set.mem_sUnion.mpr <z, hpost, hk>

end gfpBelow

end DomainTheory.DataTypesAsLattices
```

graph_ext.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem graph_ext {f g : Pomega  ->  Pomega} (h : forall  x, f x = g x) :
    graph f = graph g := by
  simp [graph, h]

end DomainTheory.DataTypesAsLattices
```

eq_4_11.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.comp
```

```
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.11).** Action of composed function-space retracts. -/
theorem eq_4_11 (a b a' b' u : Pomega) :
    funOf (arrowR a b) (funOf (arrowR a' b') u) =
      comp b (comp (comp b' (comp u a')) a) := by
  rw [arrowR.app, arrowR.app]

namespace eq_4_11

/-- **Scott 1976, (4.11).** Composition of function-space retracts. -/
theorem comp (a b a' b' : Pomega) :
    DomainTheory.DataTypesAsLattices.comp (arrowR a b) (arrowR a' b') =
      arrowR (DomainTheory.DataTypesAsLattices.comp a' a)
        (DomainTheory.DataTypesAsLattices.comp b b') := by
  apply graph_ext
  intro u
  calc
    funOf (arrowR a b) (funOf (arrowR a' b') u)
        = DomainTheory.DataTypesAsLattices.comp b
            (DomainTheory.DataTypesAsLattices.comp (funOf (arrowR a' b') u) a) :=
          arrowR.app _ _ _
    _ = DomainTheory.DataTypesAsLattices.comp b
            (DomainTheory.DataTypesAsLattices.comp
              (DomainTheory.DataTypesAsLattices.comp b'
                (DomainTheory.DataTypesAsLattices.comp u a')) a) := by
          rw [arrowR.app]
    _ = DomainTheory.DataTypesAsLattices.comp
            (DomainTheory.DataTypesAsLattices.comp b b')
            (DomainTheory.DataTypesAsLattices.comp u
              (DomainTheory.DataTypesAsLattices.comp a' a)) := by
          apply graph_ext
          intro x
          simp only [DomainTheory.DataTypesAsLattices.comp.app]

end eq_4_11

end DomainTheory.DataTypesAsLattices
```

funRetract.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
```

```
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.1).** The function-space retract `fun = fun u fun x. u(x)`. -/
def funRetract : Pomega :=
  graph (fun u => graph (fun x => funOf u x))

namespace funRetract

theorem app (u : Pomega) :
    funOf funRetract u = graph (fun x => funOf u x) :=
  beta (graph_const_isScottContinuous (fun u x => funOf u x)
    (fun x => funOf.isScottContinuous_left x)) u

theorem isRetract : Retract funRetract := by
  constructor
  apply graph_ext
  intro u
  rw [funRetract.app, funRetract.app]
  apply graph_ext
  intro x
  exact (beta (funOf.isScottContinuous u) x).symm

end funRetract

end DomainTheory.DataTypesAsLattices
```

intF.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.id_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet_more
public import DomainTheory.DataTypesAsLattices.Lambda.predSet_more
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.7).** The integer functional
`int(u) = u ? 0, (int(u?1) ? u, u)`. -/
def intF (f : Pomega) : Pomega :=
  graph (fun u =>
    dcondSet u (ofNat 0) (dcondSet (funOf f (predSet u)) u u))

namespace intF

theorem map_isScottContinuous (f : Pomega) :
    IsScottContinuous (fun u =>
      dcondSet u (ofNat 0) (dcondSet (funOf f (predSet u)) u u)) :=
  theorem_1_3.nary
    (fun y z => dcondSet.isScottContinuous_left y z)
    (fun x z => dcondSet.isScottContinuous_mid x z)
    (fun x y => dcondSet.isScottContinuous_right x y)
    id_isScottContinuous
    (const_isScottContinuous (ofNat 0))
    (theorem_1_3.nary
      (fun y z => dcondSet.isScottContinuous_left y z)
      (fun x z => dcondSet.isScottContinuous_mid x z)
      (fun x y => dcondSet.isScottContinuous_right x y)
      (theorem_1_3 (funOf.isScottContinuous f) predSet.isScottContinuous)
      id_isScottContinuous
      id_isScottContinuous)

theorem isScottContinuous : IsScottContinuous intF :=
  graph_const_isScottContinuous
    (fun f u => dcondSet u (ofNat 0) (dcondSet (funOf f (predSet u)) u u))
    (fun u =>
      theorem_1_3
        (f := fun z => dcondSet u (ofNat 0) z)
        (g := fun f => dcondSet (funOf f (predSet u)) u u)
        (dcondSet.isScottContinuous_right u (ofNat 0))
        (theorem_1_3
          (f := fun z => dcondSet z u u)
          (g := fun f => funOf f (predSet u))
          (dcondSet.isScottContinuous_left u u)
          (funOf.isScottContinuous_left (predSet u))))

theorem app (f u : Pomega) :
    funOf (intF f) u =
      dcondSet u (ofNat 0) (dcondSet (funOf f (predSet u)) u u) :=
  beta (map_isScottContinuous f) u

end intF

end DomainTheory.DataTypesAsLattices
```

intR.lean

```
module
```

```
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.intF
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_4
public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Retracts.dcondSet_same_ofNat
public import DomainTheory.DataTypesAsLattices.Retracts.predSet_eq_ofNat_of_no_zero
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Lambda.dcondSet_more
public import DomainTheory.DataTypesAsLattices.Lambda.predSet_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.7).** Integer retract as the least fixed point. -/
def intR : Pomega := fix intF

namespace intR

theorem eq_intF : intF intR = intR :=
  (theorem_1_4 intF.isScottContinuous).1

theorem app (u : Pomega) :
    funOf intR u =
      dcondSet u (ofNat 0) (dcondSet (funOf intR (predSet u)) u u) := by
  have h := congrArg (fun a => funOf a u) eq_intF
  simpa [intF.app] using h.symm

theorem bot : funOf intR botElem = botElem := by
  rw [app, dcondSet.bot]

theorem app_ofNat : forall  n, funOf intR (ofNat n) = ofNat n
  | 0 => by
    rw [app, dcondSet.ofNat_zero]
  | n + 1 => by
    rw [app, predSet.ofNat_succ, app_ofNat n, dcondSet_same_ofNat,
      dcondSet.ofNat_succ]

theorem top_of_nonsingleton :
    forall  n u, n  in  u  ->  (forall  k  in  u, n  <=  k)  ->  u  !=  botElem  ->  (
    forall  k, u  !=  ofNat k)  ->
      funOf intR u = topElem := by
  intro n
  induction n using Nat.strongRecOn with
  | ind n ih =>
    intro u hn hmin hne hns
    rw [app]
    by_cases h0 : 0  in  u
    * have hp : exists  t, t + 1  in  u := by
```

```
        by_contra hp
        apply hns 0
        ext m
        constructor
        * intro hm
          have : m = 0 := by
            by_contra hm0
            obtain <t, rfl> := Nat.exists_eq_succ_of_ne_zero hm0
            exact hp <t, hm>
          simp [ofNat, this]
        * intro hm
          simpa [ofNat] using hm.symm  |>  h0
      rw [dcondSet.of_mixed h0 hp]
    * have hn0 : n  !=  0 := fun h => h0 (by simpa [h] using hn)
      obtain <n', rfl> := Nat.exists_eq_succ_of_ne_zero hn0
      have hn' : n'  in  predSet u := hn
      have hmin' : forall  k  in  predSet u, n'  <=  k := by
        intro k hk
        have : n' + 1  <=  k + 1 := hmin (k + 1) hk
        omega
      have hne' : predSet u  !=  botElem := by
        intro hempty
        have : n'  in  predSet u := hn'
        rw [hempty] at this
        simp [botElem] at this
      have hns' : forall  k, predSet u  !=  ofNat k := by
        intro k hk
        exact hns (k + 1) (predSet_eq_ofNat_of_no_zero h0 hk)
      have hpred : funOf intR (predSet u) = topElem :=
        ih n' (Nat.lt_succ_self n') (predSet u) hn' hmin' hne' hns'
      rw [hpred, dcondSet.top, dcondSet.of_pos_not_zero h0 <n', hn>]

theorem top : funOf intR topElem = topElem :=
  top_of_nonsingleton 0 topElem (Set.mem_univ 0)
    (fun _ _ => Nat.zero_le _)
    (by
      intro h
      have : (0 : Nat)  in  (botElem : Pomega) := by
        rw [ <-  h]
        exact Set.mem_univ 0
      simp [botElem] at this)
    (fun n h => by
      have h0 : (0 : Nat)  in  ofNat n := by
        have : (0 : Nat)  in  topElem := Set.mem_univ 0
        rwa [h] at this
      have h1 : (1 : Nat)  in  ofNat n := by
        have : (1 : Nat)  in  topElem := Set.mem_univ 1
        rwa [h] at this
      simp [ofNat] at h0 h1
      omega)

theorem idem (x : Pomega) : funOf intR (funOf intR x) = funOf intR x := by
  by_cases hbot : x = botElem
  * subst hbot
    rw [bot, bot]
  * by_cases hsing : exists  n, x = ofNat n
```

```
    * obtain <n, rfl> := hsing
      rw [app_ofNat, app_ofNat]
    * have hx : funOf intR x = topElem := by
        have hne : x.Nonempty := by
          rw [Set.nonempty_iff_ne_empty]
          simpa [botElem] using hbot
        exact top_of_nonsingleton (sInf x) x (Nat.sInf_mem hne)
          (fun k hk => Nat.sInf_le hk) hbot (fun n hn => hsing <n, hn>)
      rw [hx, top]

theorem isGraph : intR = graph (funOf intR) := by
  refine eq_intF.symm.trans ?_
  apply graph_ext
  intro u
  exact (app u).symm

theorem isRetract : Retract intR := by
  constructor
  change intR = graph (fun x => funOf intR (funOf intR x))
  refine isGraph.trans ?_
  apply graph_ext
  intro x
  exact (idem x).symm

end intR

end DomainTheory.DataTypesAsLattices
```

inter_const_isScottContinuous.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Continuous.scottUnion
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem inter_const_isScottContinuous {f : Pomega  ->  Pomega}
    (hf : IsScottContinuous f) (b : Pomega) :
    IsScottContinuous (fun z => f z  I  b) := by
  constructor
  intro x
  ext k
  constructor
  * intro <hkf, hkb>
    have : k  in  scottUnion f x := by rwa [ <-  hf.out x]
```

```
    obtain <n, hn, hkn> := mem_scottUnion.mp this
    exact mem_scottUnion.mpr <n, hn, hkn, hkb>
  * intro hk
    obtain <n, hn, hkn, hkb> := mem_scottUnion.mp hk
    exact <IsScottContinuous.monotone hf hn hkn, hkb>

end DomainTheory.DataTypesAsLattices
```

iterateFrom.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.FixedPoint.iterateBot

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Directed iterates of `f` starting from `x` (Scott's sup construction). -/
def iterateFrom (f : Pomega  ->  Pomega) (x : Pomega) : Nat  ->  Pomega
  | 0 => x
  | n + 1 => f (iterateFrom f x n)

namespace iterateFrom

theorem bot (f : Pomega  ->  Pomega) (n : Nat) :
    iterateFrom f botElem n = iterateBot f n := by
  induction n with
  | zero => rfl
  | succ n ih => simp [iterateFrom, iterateBot, ih]

theorem mono {f : Pomega  ->  Pomega} (hf : IsScottContinuous f)
    {x : Pomega} (hx : x  subseteq  f x) :
    forall  n, iterateFrom f x n  subseteq  iterateFrom f x (n + 1)
  | 0 => hx
  | n + 1 => IsScottContinuous.monotone hf (mono hf hx n)

end iterateFrom

end DomainTheory.DataTypesAsLattices
```

lfpAbove.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
```

```
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Embedding.extend
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.iterateFrom
public import DomainTheory.DataTypesAsLattices.Retracts.Fixpoints
public import DomainTheory.DataTypesAsLattices.Retracts.sUnion_Fixpoints_le_image
public import DomainTheory.DataTypesAsLattices.FixedPoint.chain_lemma
public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
public import DomainTheory.DataTypesAsLattices.Retracts.vals

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Least upper bound of fixed points above `x`: `?_n f?(x)`. -/
def lfpAbove (f : Pomega  ->  Pomega) (x : Pomega) : Pomega :=
  ? n, iterateFrom f x n

namespace lfpAbove

theorem bot (f : Pomega  ->  Pomega) : lfpAbove f botElem = fix f := by
  simp [lfpAbove, fix, iterateFrom.bot]

theorem ub {f : Pomega  ->  Pomega} {A : Set Pomega} :
    forall  x  in  A, x  subseteq  lfpAbove f (?_0 A) := by
  intro x hx k hk
  exact Set.mem_iUnion.mpr <0, Set.subset_sUnion_of_mem hx hk>

theorem fixed {f : Pomega  ->  Pomega} (hf : IsScottContinuous f)
    {x : Pomega} (hx : x  subseteq  f x) : f (lfpAbove f x) = lfpAbove f x := by
  have hchain := chain_lemma hf (iterateFrom f x) (iterateFrom.mono hf hx)
  unfold lfpAbove
  rw [hchain]
  ext k
  constructor
  * intro hk
    obtain <n, hkn> := Set.mem_iUnion.mp hk
    exact Set.mem_iUnion.mpr <n + 1, hkn>
  * intro hk
    obtain <n, hkn> := Set.mem_iUnion.mp hk
    cases n with
    | zero => exact Set.mem_iUnion.mpr <0, hx hkn>
    | succ n => exact Set.mem_iUnion.mpr <n, hkn>

theorem le_of_ub {f : Pomega  ->  Pomega} (hf : IsScottContinuous f)
    {A : Set Pomega} {z : Pomega} (hz : f z = z) (hub : forall  x  in  A, x  subseteq
     z) :
    lfpAbove f (?_0 A)  subseteq  z := by
  have h0 : ?_0 A  subseteq  z := by
    intro k hk
    obtain <x, hxA, hkx> := Set.mem_sUnion.mp hk
    exact hub x hxA hkx
  have hiter : forall  n, iterateFrom f (?_0 A) n  subseteq  z := by
    intro n
```

```
    induction n with
    | zero => simpa [iterateFrom] using h0
    | succ n ih => simpa [iterateFrom, hz] using IsScottContinuous.monotone hf ih
  intro k hk
  obtain <n, hkn> := Set.mem_iUnion.mp hk
  exact hiter n hkn

theorem mem {f : Pomega  ->  Pomega} (hf : IsScottContinuous f)
    {A : Set Pomega} (hA : A  subseteq  Fixpoints f) :
    lfpAbove f (?_0 A)  in  Fixpoints f :=
  fixed hf (sUnion_Fixpoints_le_image hf hA)

end lfpAbove

end DomainTheory.DataTypesAsLattices
```

pairSeq.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.seq2
public import DomainTheory.DataTypesAsLattices.Pomega.e
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.21) / (4.2).** Pairing as a distributive sequence. -/
def pairSeq (x y : Pomega) : Pomega := seq2 x y

end DomainTheory.DataTypesAsLattices
```

predSet_ofNat_zero.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem predSet_ofNat_zero : predSet (ofNat 0) = botElem := by
  ext k
  simp [predSet, ofNat, botElem]

end DomainTheory.DataTypesAsLattices
```

retractBasis.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
```

```
public import DomainTheory.DataTypesAsLattices.FixedLattices.WayBelow
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.Fixpoints
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.retract_app
public import DomainTheory.DataTypesAsLattices.Retracts.retract_interpolation
public import DomainTheory.DataTypesAsLattices.Retracts.sSup_fixed_eq_sUnion
public import DomainTheory.DataTypesAsLattices.Retracts.directed_finite_cover
public import DomainTheory.DataTypesAsLattices.FixedLattices.DirectedSet
public import DomainTheory.DataTypesAsLattices.Retracts.DirectedSubset
public import DomainTheory.DataTypesAsLattices.Retracts.vals
public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Retracts.BasicInst36
public import DomainTheory.DataTypesAsLattices.Retracts.fixpointsCompleteLattice
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- The canonical finite-image basis of the fixed-point lattice of a retract. -/
def retractBasis (a : Pomega) (x : Fixpoints (funOf a)) :
    Set (Fixpoints (funOf a)) :=
  {y | exists  n, e n  subseteq  x  /\  (y : Pomega) = funOf a (e n)}

namespace retractBasis

theorem directed {a : Pomega} (ha : Retract a)
    (x : Fixpoints (funOf a)) :
    DirectedSet (retractBasis a x) := by
  constructor
  constructor
  * let y : Fixpoints (funOf a) :=
      <funOf a (e 0), retract_app ha (e 0)>
    exact <y, 0, by simp [e_zero], rfl>
  * intro y hy z hz
    obtain <n, hnx, hyn> := hy
    obtain <m, hmx, hzm> := hz
    let w : Fixpoints (funOf a) :=
      <funOf a (e (n ||| m)), retract_app ha (e (n ||| m))>
    refine <w, <n ||| m, e_or_of_subset hnx hmx, rfl>, ?_, ?_>
    * change (y : Pomega)  subseteq  (w : Pomega)
      rw [hyn]
      change funOf a (e n)  subseteq  funOf a (e (n ||| m))
      rw [e_or]
      exact IsScottContinuous.monotone (funOf.isScottContinuous a)
        Set.subset_union_left
    * change (z : Pomega)  subseteq  (w : Pomega)
      rw [hzm]
      change funOf a (e m)  subseteq  funOf a (e (n ||| m))
      rw [e_or]
```

```
      exact IsScottContinuous.monotone (funOf.isScottContinuous a)
        Set.subset_union_right

theorem directedSubset {a : Pomega} (ha : Retract a)
    (x : Fixpoints (funOf a)) :
    DirectedSubset (retractBasis a x) := by
  have h := directed ha x
  refine <<h.out.1, ?_>>
  intro y hy z hz
  obtain <w, hw, hyw, hzw> := h.out.2 hy hz
  exact <w, hw, hyw, hzw>

theorem eq_sSup {a : Pomega} (ha : Retract a)
    (x : Fixpoints (funOf a)) :
    x = sSup (retractBasis a x) := by
  apply Subtype.ext
  rw [sSup_fixed_eq_sUnion (funOf.isScottContinuous a)
    (directed ha x)]
  have hx : typed (x : Pomega) a := x.property.symm
  rw [retract_interpolation ha hx]
  ext k
  constructor
  * intro hk
    obtain <n, hnx, hkn> := Set.mem_iUnion.mp hk
    let y : Fixpoints (funOf a) :=
      <funOf a (e n), retract_app ha (e n)>
    exact Set.mem_sUnion.mpr
      <y, <y, <n, hnx, rfl>, rfl>, hkn>
  * intro hk
    obtain <_, <y, <n, hnx, hyn>, rfl>, hky> :=
      Set.mem_sUnion.mp hk
    exact Set.mem_iUnion.mpr <n, hnx, hyn  |>  hky>

theorem eq_sUnion {a : Pomega} (ha : Retract a)
    (x : Fixpoints (funOf a)) :
    (x : Pomega) = ?_0 (Subtype.val '' retractBasis a x) := by
  have h := eq_sSup ha x
  have hsu := sSup_fixed_eq_sUnion (funOf.isScottContinuous a)
    (directed ha x)
  have : (x : Pomega) = (sSup (retractBasis a x) : Fixpoints (funOf a)) :=
    congrArg Subtype.val h
  exact this.trans hsu

theorem wayBelow {a : Pomega} (_ha : Retract a)
    {x y : Fixpoints (funOf a)} (hy : y  in  retractBasis a x) :
    WayBelow y x := by
  intro A hA hxA
  obtain <n, hnx, hyn> := hy
  have hsup :
      ((sSup A : Fixpoints (funOf a)) : Pomega) =
        ?_0 (Fixpoints.vals A) :=
    sSup_fixed_eq_sUnion (funOf.isScottContinuous a) hA
  have hfinite :
      e n  subseteq  ?_0 (Fixpoints.vals A) := by
    rw [ <-  hsup]
    exact hnx.trans hxA
```

```
  obtain <z, hzA, hnz> :=
    directed_finite_cover hA (e_finite n) hfinite
  refine <z, hzA, ?_>
  change (y : Pomega)  subseteq  z
  rw [hyn]
  have := IsScottContinuous.monotone (funOf.isScottContinuous a) hnz
  rwa [z.property] at this

end retractBasis

end DomainTheory.DataTypesAsLattices
```

retractLe.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.Retract

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, S4, Definition.** `a  sqsub  b` iff `a = a  comp  b = b  comp  a`. -/
def retractLe (a b : Pomega) : Prop :=
  a = comp a b  /\  a = comp b a

namespace retractLe

theorem refl {a : Pomega} (ha : Retract a) : retractLe a a :=
  <ha.out, ha.out>

theorem trans {a b c : Pomega}
    (hab : retractLe a b) (hbc : retractLe b c) :
    retractLe a c := by
  rcases hab with <hab_1, hab_2>
  rcases hbc with <hbc_1, hbc_2>
  constructor
  * apply Eq.trans hab_1
    apply Eq.symm
    calc
      comp a c = graph (fun x => funOf a (funOf c x)) := rfl
      _ = graph (fun x => funOf (comp a b) (funOf c x)) := by simp [ <-  hab_1]
      _ = graph (fun x => funOf a (funOf b (funOf c x))) := by simp [comp.app]
      _ = graph (fun x => funOf a (funOf (comp b c) x)) := by simp [comp.app]
      _ = graph (fun x => funOf a (funOf b x)) := by simp [ <-  hbc_1]
      _ = comp a b := rfl
  * apply Eq.trans hab_2
    apply Eq.symm
    calc
      comp c a = graph (fun x => funOf c (funOf a x)) := rfl
      _ = graph (fun x => funOf c (funOf (comp b a) x)) := by simp [ <-  hab_2]
```

```
      _ = graph (fun x => funOf c (funOf b (funOf a x))) := by simp [comp.app]
      _ = graph (fun x => funOf (comp c b) (funOf a x)) := by simp [comp.app]
      _ = graph (fun x => funOf b (funOf a x)) := by simp [ <-  hbc_2]
      _ = comp b a := rfl

end retractLe

end DomainTheory.DataTypesAsLattices
```

retract_interpolation.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Continuous.scottUnion
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.typed
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Continuous-lattice interpolation: `x = ? { a(e n) | e n  subseteq  x }`. -/
theorem retract_interpolation {a x : Pomega} (_ha : Retract a) (hx : typed x a) :
    x = ? n, {k | e n  subseteq  x  /\  k  in  funOf a (e n)} := by
  have hx' : funOf a x = x := hx.symm
  have hsc := (funOf.isScottContinuous a).out x
  apply subset_antisymm
  * intro k hk
    have : k  in  funOf a x := by rwa [hx']
    have : k  in  scottUnion (funOf a) x := by rwa [ <-  hsc]
    obtain <n, hn, hkn> := mem_scottUnion.mp this
    exact Set.mem_iUnion.mpr <n, hn, hkn>
  * intro k hk
    obtain <n, hn, hkn> := Set.mem_iUnion.mp hk
    have : k  in  funOf a x :=
      IsScottContinuous.monotone (funOf.isScottContinuous a) hn hkn
    rwa [hx'] at this

end DomainTheory.DataTypesAsLattices
```

sUnion_Fixpoints_le_image.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
```

```
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.Fixpoints
public import DomainTheory.DataTypesAsLattices.Retracts.vals

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem sUnion_Fixpoints_le_image {f : Pomega  ->  Pomega} (hf : IsScottContinuous f)
    {A : Set Pomega} (hA : A  subseteq  Fixpoints f) :
    ?_0 A  subseteq  f (?_0 A) := by
  intro k hk
  obtain <x, hxA, hkx> := Set.mem_sUnion.mp hk
  have hx : f x = x := hA hxA
  have : k  in  f x := by rwa [hx]
  exact IsScottContinuous.monotone hf (Set.subset_sUnion_of_mem hxA) this

end DomainTheory.DataTypesAsLattices
```

sndC.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def sndC : Pomega := graph (fun u => funOf u (ofNat 1))

namespace sndC

theorem app (u : Pomega) : funOf sndC u = funOf u (ofNat 1) :=
  beta (funOf.isScottContinuous_left (ofNat 1)) u

end sndC

end DomainTheory.DataTypesAsLattices
```

typed_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.typed
```

```
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.Retract
public import DomainTheory.DataTypesAsLattices.Retracts.theorem_4_3_more
public import DomainTheory.DataTypesAsLattices.Retracts.retract_app
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import DomainTheory.DataTypesAsLattices.Graph.graph

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace typed

theorem arrowR_range {a b u : Pomega}
    (hb : Retract b) (hu : typed u (arrowR a b)) :
    u = comp b u := by
  have hu' := arrowR.typed_of hu
  rw [hu']
  apply graph_ext
  intro x
  simp only [comp.app]
  rw [retract_app hb]

theorem arrowR_restrict {a b u : Pomega}
    (ha : Retract a) (hu : typed u (arrowR a b)) :
    u = comp u a := by
  have hu' := arrowR.typed_of hu
  have h : comp (comp b (comp u a)) a = comp b (comp u a) := by
    apply graph_ext
    intro x
    simp only [comp.app]
    rw [retract_app ha]
  have : comp u a = comp (comp b (comp u a)) a := by
    conv_lhs => rw [hu']
  exact ((this.trans h).trans hu'.symm).symm

theorem arrowR_iff {a b u : Pomega} (ha : Retract a) (_hb : Retract b) :
    typed u (arrowR a b)  <->
      u = comp u a  /\  forall  x, typed x a  ->  typed (funOf u x) b := by
  constructor
  * intro hu
    exact <typed.arrowR_restrict ha hu, fun x hx => theorem_4_3.ii hu hx>
  * intro <hrest, hval>
    change u = funOf (arrowR a b) u
    rw [arrowR.app]
    apply Eq.trans hrest
    apply graph_ext
    intro x
    have hx : typed (funOf a x) a := (retract_app ha x).symm
    have hbx : funOf u (funOf a x) = funOf b (funOf u (funOf a x)) :=
      hval (funOf a x) hx
    simp only [comp.app]
    exact hbx
```

```
theorem comp {a b c f f' : Pomega}
    (ha : Retract a) (hb : Retract b) (_hc : Retract c)
    (hf : typed f (arrowR a b)) (hf' : typed f' (arrowR b c)) :
    typed (DomainTheory.DataTypesAsLattices.comp f' f) (arrowR a c) := by
  have hf1 := arrowR.typed_of hf
  have hf2 := arrowR.typed_of hf'
  change DomainTheory.DataTypesAsLattices.comp f' f =
      funOf (arrowR a c) (DomainTheory.DataTypesAsLattices.comp f' f)
  rw [arrowR.app]
  apply graph_ext
  intro x
  have hfx : funOf f x = funOf b (funOf f (funOf a x)) := by
    simpa [DomainTheory.DataTypesAsLattices.comp.app] using
      congrArg (fun w => funOf w x) hf1
  have hrest := typed.arrowR_restrict ha hf
  have hfa : funOf f (funOf a x) = funOf f x := by
    simpa [DomainTheory.DataTypesAsLattices.comp.app] using
      (congrArg (fun w => funOf w x) hrest).symm
  have hf'fx : funOf f' (funOf f x) =
      funOf c (funOf f' (funOf b (funOf f x))) := by
    simpa [DomainTheory.DataTypesAsLattices.comp.app] using
      congrArg (fun w => funOf w (funOf f x)) hf2
  have hrange := typed.arrowR_range hb hf
  have hbval : funOf b (funOf f (funOf a x)) = funOf f (funOf a x) := by
    simpa [DomainTheory.DataTypesAsLattices.comp.app] using
      (congrArg (fun w => funOf w (funOf a x)) hrange).symm
  simp only [DomainTheory.DataTypesAsLattices.comp.app]
  rw [hf'fx,  <-  hfa, hbval, hfa]

end typed

end DomainTheory.DataTypesAsLattices
```

typed.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, S4, Definition.** `u : a` means `u = a(u)`. -/
def typed (u a : Pomega) : Prop :=
  u = funOf a u

end DomainTheory.DataTypesAsLattices
```

2.5 Closure operations and algebraic lattices

Historical note. Algebraic lattices and closure operations predate the paper: Grätzer (1968) for the lattices, Tarski (1930) for the closures. Per Martin-Löf and Peter Hancock suggested that the continuous closures

form a universe, and the construction of $\mathbf{V}$ is essentially theirs. The limit theorem is Scott's.

Dependencies. Inside Closures, each box is one Lean file. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. The arrowhead points at the file or folder that imports, and the number is how many direct imports.

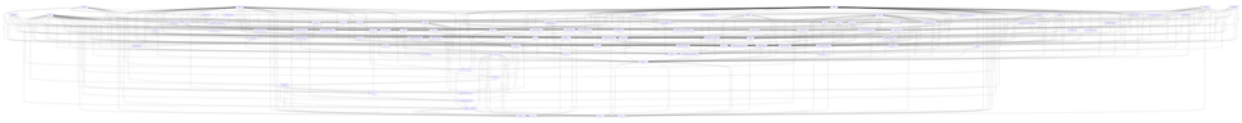


Figure 25: Closure operations and algebraic lattices.

Given any family of (finitary) operations on a set (say, ω) there is a closure operation defined on the subsets of that set obtained by forming the *least subset* including the given elements and *closed* under the operations. Examples are very familiar from algebra: the subgroup generated by a set of elements, the subspace spanned by a set of vectors, the convex hull of a set of geometric points. We simplify matters here by restricting attention to closures operating on sets in $\mathbf{P}\omega$, but the idea is quite general. The main point about these "algebraic" closure operations—as distinguished from topological closure operations—is that they are *continuous*. Thus, in the case of subgroups, if an element belongs to the subgroup generated by some elements, then it also belongs to the subgroup generated by *finitely* many of them. In the context of $\mathbf{P}\omega$ we can state the characteristic condition very simply.

- Definition (closure)
- Theorem 5.1
- Theorem 5.2
- Theorem 5.3
- Theorem 5.4
- Theorem 5.5
- Theorem 5.6

`Icomb_isClosure.lean`

```
module

public import Mathlib.Algebra.Ring.Parity
public import DomainTheory.DataTypesAsLattices.Retracts.Icomb
public import DomainTheory.DataTypesAsLattices.Closures.Closure
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem Icomb_isClosure : Closure Icomb :=
  <<subset_rfl, Icomb.isRetract>>

end DomainTheory.DataTypesAsLattices
```

2.6 Subsets and their classification

Historical note. The classification of these subsets by notions from descriptive set theory, and the connection with Wadge reducibility, are in the work of Adrian Tang. The normal forms for the classes, such as Table 3, are Scott's.

Dependencies. Inside Classification, each box is one Lean file. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. The arrowhead points at the file or folder that imports, and the number is how many direct imports.



Figure 26: Subsets and their classification.

Retracts produce very special subsets of $\mathbf{P}\omega$: a retract always has a nonempty range which forms a lattice under $\sqsubseteq$. For example, the range of **int** is $\{\bot, \top\} \cup \omega$. We often wish to eliminate $\bot$ and $\top$; and with a retract like **tree** the situation is more complex, since combinations like $\langle 1, \langle\langle 1, \langle \bot, \langle 0\rangle\rangle\rangle, \top\rangle\rangle$ might require elimination. In these two cases the method is simple.

Consider these two functions:

$$(6.1) \quad \mathbf{mid} = \lambda x : \mathbf{int}.\, x \supset 0, 0$$

$$(6.2) \quad \mathbf{perf} = \lambda u : \mathbf{tree}.\, \mathbf{which}(u) \supset 0,\, \Delta(\mathbf{perf}(\mathbf{fst}(\mathbf{out}(u))))(\mathbf{perf}(\mathbf{snd}(\mathbf{out}(u))))$$

where Δ is a special combinator:

$$(6.3) \quad \Delta = \lambda x \lambda y.\, (x \supset (y \supset 0, \top), \top) \cup (y \supset (x \supset 0, \top), \top).$$

We find that $\omega = \{x : \mathbf{int} \mid \mathbf{mid}(x) = 0\}$. In the case of trees, note first this behavior of Δ:

Δ	$\bot$	0	$\top$
$\bot$	$\bot$	$\bot$	$\top$
0	$\bot$	0	$\top$
$\top$	$\top$	$\top$	$\top$

The question is: what subset is $\{u : \mathbf{tree} \mid \mathbf{perf}(u) = 0\}$?

Now **perf** is defined recursively. We can see that

$$\mathbf{perf}(\bot) = \bot, \quad \mathbf{perf}(\top) = \top, \quad \mathbf{perf}(\langle 0\rangle) = 0,$$

and

$$\mathbf{perf}(\langle 1, \langle x, y\rangle\rangle) = \Delta(\mathbf{perf}(x))(\mathbf{perf}(y))$$

when $x, y : \mathbf{tree}$. Every tree, aside from $\top$ or $\bot$, is either *atomic* or a pair of trees. The atomic tree is "perfect" (that is, $\mathbf{perf}(\langle 0\rangle) = 0$). A finite tree which does not contain $\bot$ or $\top$ is perfect—as we can see

inductively using the table above for Δ. An infinite tree is never perfect: either some branch ends in $\top$ and **perf** maps it to $\top$, or $\top$ is never reached and **perf** maps it to $\bot$. Thus the subset in question is then seen to be the set of *finite* trees generated from the atom by pairing. This is clearly a desirable subset, and it is sorted out by a function with a simple recursive definition. The general question is: what subsets can be characterized by equations? The answer can be given by reference to the topology of $\mathbf{P}\omega$.

- Definition (classes)
- Theorem 6.1
- Theorem 6.2
- Theorem 6.3
- Theorem 6.4
- Theorem 6.5
- Theorem 6.6
- Theorem 6.7

IsBooleanOpen.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottOpen
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import Mathlib.Data.Set.Finite.Powerset

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

open TopologicalSpace

inductive IsBooleanOpen : Set Pomega  ->  Prop
  | ofOpen {U} : IsScottOpen U  ->  IsBooleanOpen U
  | union {U V} : IsBooleanOpen U  ->  IsBooleanOpen V  ->  IsBooleanOpen (U  U  V)
  | inter {U V} : IsBooleanOpen U  ->  IsBooleanOpen V  ->  IsBooleanOpen (U  I  V)
  | compl {U} : IsBooleanOpen U  ->  IsBooleanOpen U?

end DomainTheory.DataTypesAsLattices
```

2.7 Total functions and functionality

Historical note. Functionality had long been studied in combinatory logic. Hindley, Lercher, and Seldin (1972) is the introduction Scott used. This section is his reading of that theory in $\mathbf{P}\omega$. The connection with the functional interpretation of intuitionistic logic goes back to Curry and Kleene, and Theorem 7.3 is the standard result of that interpretation. Theorem 7.4, that the obvious typing characterizes the iterators, is due to Gordon Plotkin.

Dependencies. Inside Functionality, each box is one Lean file. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. The arrowhead points at the file or folder that imports, and the number is how many direct imports.



Figure 27: Total functions and functionality.

There is an inevitable conflict between the concepts of *total* and *partial* functions: we desire the former, but it is the latter we usually get. Total functions are better because they are "well-defined" at all their arguments, but the rub is that there is no general way of deciding when a *definition* is going to be well-defined in all its uses. In analysis we have singularities, and in recursion theory we have endless, nonfinishing computations. In the present theory we have in effect evaded the question in two ways. First we have embraced the partial function as the norm. But secondly, and possibly confusingly, the multiple-valued functions are normal, total functions from $\mathbf{P}\omega$ into $\mathbf{P}\omega$. The point, of course, is that we are making a *model* of the partial functions in terms of ordinary mathematical functions. But note that the success of the *model* lies in *not* using arbitrary functions: it is only the continuous functions that correspond to the kind of partial functions we wanted to study. It would be a mistake to think of the variables in λ-calculus as ranging over arbitrary functions—and this mistake was made by both Church and Curry. The fixed-point operator $\mathbf{Y}$ shows that we must restrict attention to functions which do have fixed points. It is certainly the case that $\mathbf{P}\omega$ is not the only model for the λ-calculus, but it is a very satisfactory model and is rich enough to illustrate what can and what cannot be done with partial functions.

Whatever the pleasures of partial functions (and the multiple-valued ones, too), the desire for total functions remains. Take the integers. We are more interested in ω than $\omega \cup \{\bot, \top\}$. Since the multiple values $\bot$ and $\top$ are but two in number, it is easy to avoid them. The problem becomes tiresome in considering functions, however. The lattice represented by the retract $\mathbf{int} \circ \to \mathbf{int}$ is much too large, in that there are as many nontotal functions in this domain as total ones. The aim of the present section is to introduce an interpretation of a theory of functionality in the model $\mathbf{P}\omega$ that provides a convenient way of restricting attention to the functions (or other objects) that are total in the desired sense. The theory of functionality is rather like proposals of Curry, but not quite the same for important reasons as we will see.

In the theory of retracts of §§ 5 and 6, the plan of "restricting attention" was the very simple one of restricting to a *subset*. It was made notationally simple as the subsets in question could be parameterized by continuous functions. The retraction mappings stand in for their ranges. Even better, certain continuous functions act on these retractions as space-forming functors (such as $\oplus$ and $\circ \to$), which gives greater notational simplicity because one language is able to serve for several tasks. When we pass to the theory of total functions, this same kind of simplicity is no longer possible owing to an increase in quantifier complexity in the necessary definitions. (This remark is made definite below.) Another point where there is some loss of simplicity concerns the representation of entities in $\mathbf{P}\omega$: subsets will no longer be enough, since we will need *quotients* of subsets. This is not a very startling point. Many constructions are affected

in a natural way via equivalence classes. An equivalence relation makes you blind to certain distinctions. It may be easier also to remain a bit blind than to search for the most beautiful representative of an equivalence class: there may be nothing to choose between several candidates, and it can cost too much effort to attempt a choice. Thus our first agreement is that for many purposes a *kind* of object can be taken as a set of equivalence classes for an equivalence relation on a subset of $\mathbf{P}\omega$.

Because $\mathbf{P}\omega$ is closed under the pairing function $\lambda x \lambda y.\, \langle x, y\rangle$, we shall construe relations on subsets of $\mathbf{P}\omega$ as subsets of $\mathbf{P}\omega$ all of whose elements are ordered pairs. That is, a relation A satisfies this inclusion:

$$(7.1) \quad A \subseteq \{\langle x, y\rangle \mid x, y \in \mathbf{P}\omega\}.$$

- Definition (equivalence)
- Theorem 7.1
- Theorem 7.2
- Theorem 7.3
- Theorem 7.4

Hcomb.lean

```
module

public import DomainTheory.DataTypesAsLattices.Functionality.Hfun
public import DomainTheory.DataTypesAsLattices.Functionality.envR_isRetract
public import DomainTheory.DataTypesAsLattices.Functionality.expR_isRetract
public import DomainTheory.DataTypesAsLattices.Functionality.lambR_isRetract
public import DomainTheory.DataTypesAsLattices.Functionality.typed_apply_retract
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import DomainTheory.DataTypesAsLattices.Retracts.arrowR_more
public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.envR
public import DomainTheory.DataTypesAsLattices.Retracts.expR
public import DomainTheory.DataTypesAsLattices.Retracts.lambR
public import DomainTheory.DataTypesAsLattices.Retracts.typed

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

def Hcomb : Pomega :=
  funOf (arrowR expR (arrowR envR lambR)) Hfun

namespace Hcomb

theorem app (e : Pomega) :
    funOf Hcomb e =
      funOf (arrowR envR lambR) (funOf Hfun (funOf expR e)) := by
  rw [Hcomb, arrowR.app]
  simp only [comp.app]
```

```
theorem typed :
    typed Hcomb (arrowR expR (arrowR envR lambR)) :=
  typed_apply_retract
    (arrowR.isRetract expR_isRetract
      (arrowR.isRetract envR_isRetract lambR_isRetract))

end Hcomb

end DomainTheory.DataTypesAsLattices
```

Hcomb_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Functionality.Hcomb
public import DomainTheory.DataTypesAsLattices.Functionality.LambTerm
public import DomainTheory.DataTypesAsLattices.Functionality.encodeExp
public import DomainTheory.DataTypesAsLattices.Functionality.eq_4_45
public import DomainTheory.DataTypesAsLattices.Functionality.Htyped
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace Hcomb

theorem decode (tau : LambTerm) :
    funOf Hcomb (encodeExp tau) = Htyped tau :=
  (DomainTheory.DataTypesAsLattices.eq_4_45 tau).2.2

end Hcomb

end DomainTheory.DataTypesAsLattices
```

Z.lean

```
module

public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Functionality.RestrictedEquiv
public import DomainTheory.DataTypesAsLattices.Functionality.continuous_tuple
public import DomainTheory.DataTypesAsLattices.Functionality.mem
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Lambda.const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.id_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Pomega.pair
@[expose] public section
```

```
namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (7.15)-(7.16).** Iterators `Z_n`. -/
def Z : Nat  ->  Pomega  ->  Pomega  ->  Pomega
  | 0, _, x => x
  | n + 1, f, x => funOf f (Z n f x)

namespace Z

theorem rel (A : RestrictedEquiv) (n : Nat) {f f' x y : Pomega}
    (hf : forall  a b, A.rel a b  ->  A.rel (funOf f a) (funOf f' b))
    (hxy : A.rel x y) :
    A.rel (Z n f x) (Z n f' y) := by
  induction n with
  | zero => simpa [Z] using hxy
  | succ n ih =>
    simpa [Z] using hf _ _ ih

theorem finite_bound (p q m : Nat) :
    forall  k  in  Z m (e p) (e q), k  <=  max p q := by
  induction m with
  | zero =>
      intro k hk
      exact (mem_e_le hk).trans (Nat.le_max_right p q)
  | succ m ih =>
      intro k hk
      obtain <r, _hr, hpair> := hk
      exact (pair_le_right r k).trans
        ((mem_e_le hpair).trans (Nat.le_max_left p q))

theorem finite_no_tag (p q m k : Nat) :
    pair (max p q + 1) k  notin  Z m (e p) (e q) := by
  intro hk
  have hle := Z.finite_bound p q m _ hk
  exact (Nat.not_succ_le_self (max p q))
    ((pair_le_left (max p q + 1) k).trans hle)

theorem graph_funOf (n : Nat) (f x : Pomega) :
    Z n (graph (fun y => funOf f y)) x = Z n f x := by
  induction n with
  | zero => simp [Z]
  | succ n ih =>
    simp [Z, ih]
    have : funOf (graph (fun y => funOf f y)) (Z n f x) = funOf f (Z n f x) :=
      beta (funOf.isScottContinuous f) (Z n f x)
    exact this

theorem isScottContinuous_left (n : Nat) (x : Pomega) :
    IsScottContinuous (fun f => Z n f x) := by
  induction n with
  | zero => simpa [Z] using const_isScottContinuous x
  | succ n ih =>
    simpa [Z] using
      continuous_tuple
        (fun y => funOf.isScottContinuous_left y)
        (fun u => funOf.isScottContinuous u)
```

```
        id_isScottContinuous ih

theorem isScottContinuous_right (n : Nat) (f : Pomega) :
    IsScottContinuous (fun x => Z n f x) := by
  induction n with
  | zero => simpa [Z] using id_isScottContinuous
  | succ n ih =>
    simpa [Z] using theorem_1_3 (funOf.isScottContinuous f) ih

end Z

end DomainTheory.DataTypesAsLattices
```

Zcomb.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.Z
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.beta

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (7.15)-(7.16).** Combinators `Z_n = fun f fun x. f?(x)`. -/
def Zcomb (n : Nat) : Pomega :=
  graph (fun f => graph (fun x => Z n f x))

namespace Zcomb

theorem map_isScottContinuous (n : Nat) :
    IsScottContinuous (fun f => graph (fun x => Z n f x)) :=
  graph_const_isScottContinuous (fun f x => Z n f x)
    (fun x => Z.isScottContinuous_left n x)

theorem app (n : Nat) (f : Pomega) :
    funOf (Zcomb n) f = graph (fun x => Z n f x) :=
  beta (Zcomb.map_isScottContinuous n) f

theorem app2 (n : Nat) (f x : Pomega) :
    funOf (funOf (Zcomb n) f) x = Z n f x := by
  rw [Zcomb.app]
  exact beta (Z.isScottContinuous_right n f) x
```

```
theorem isGraph (n : Nat) :
    Zcomb n = graph (fun f => funOf (Zcomb n) f) := by
  apply graph_ext
  intro f
  rw [Zcomb.app]

theorem app_isGraph (n : Nat) (f : Pomega) :
    funOf (Zcomb n) f = graph (fun x => funOf (funOf (Zcomb n) f) x) := by
  rw [Zcomb.app]
  apply graph_ext
  intro x
  exact (beta (Z.isScottContinuous_right n f) x).symm

end Zcomb

end DomainTheory.DataTypesAsLattices
```

arrowR_diag_isScottContinuous.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.arrowR
public import DomainTheory.DataTypesAsLattices.Retracts.comp
public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Lambda.id_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Functionality.continuous_tuple
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem arrowR_diag_isScottContinuous :
    IsScottContinuous (fun z => arrowR z z) := by
  apply graph_const_isScottContinuous
  intro u
  have h :
      IsScottContinuous (fun z =>
        graph (fun x => funOf z (funOf u (funOf z x)))) :=
    graph_const_isScottContinuous
      (fun z x => funOf z (funOf u (funOf z x)))
      (fun x =>
        continuous_tuple
          (fun y => funOf.isScottContinuous_left (funOf u (funOf y x)))
          (fun z =>
```

```
            theorem_1_3 (funOf.isScottContinuous z)
              (theorem_1_3 (funOf.isScottContinuous u)
                (funOf.isScottContinuous_left x)))
          id_isScottContinuous
          id_isScottContinuous)
  convert h using 1
  funext z
  apply graph_ext
  intro x
  simp only [comp.app]

end DomainTheory.DataTypesAsLattices
```

sigmaJ.lean

```
module

public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.theorem_1_2
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (7.17).** Independent successor functions. -/
def sigmaJ (j : Nat) (x : Pomega) : Pomega :=
  {p | exists  k, pair j (k + 1) = p  /\  pair j k  in  x}

namespace sigmaJ

theorem isScottContinuous (j : Nat) : IsScottContinuous (sigmaJ j) := by
  constructor
  intro x
  ext p
  constructor
  * intro <k, heq, hp>
    refine mem_scottUnion.mpr <2 ^ pair j k, ?_, k, heq, ?_>
    * simpa [e_pow2, Set.singleton_subset_iff] using hp
    * simp [e_pow2]
  * intro hp
    obtain <n, hn, k, heq, hpk> := mem_scottUnion.mp hp
    exact <k, heq, hn hpk>

theorem app (j : Nat) (x : Pomega) :
    funOf (graph (sigmaJ j)) x = sigmaJ j x := by
```

```
  simpa using congrArg (fun g => g x) (theorem_1_2.i (isScottContinuous j))

theorem bot (j : Nat) : sigmaJ j botElem = botElem := by
  ext p
  simp [sigmaJ, botElem]

theorem of_no_tag {j : Nat} {u : Pomega}
    (hu : forall  k, pair j k  notin  u) :
    sigmaJ j u = botElem := by
  ext p
  constructor
  * intro <k, _hp, hk>
    exact (hu k hk).elim
  * simp [botElem]

theorem pair_union (j j' m : Nat) (hne : j  !=  j') :
    sigmaJ j (({pair j m} : Pomega)  U  {pair j' m}) =
      ({pair j (m + 1)} : Pomega) := by
  ext p
  constructor
  * intro <k, hp, hk>
    rcases hk with hk | hk
    * have hkm : k = m := (pair_inj hk).2
      subst k
      simpa using hp.symm
    * have hj : j = j' := (pair_inj hk).1
      exact (hne hj).elim
  * intro hp
    exact <m, hp.symm, Or.inl (by simp)>

theorem succ_ne (j j' m : Nat) (hne : j  !=  j') :
    sigmaJ j {pair j' m} = botElem := by
  ext p
  constructor
  * intro <k, _hk, hp>
    have : pair j k = pair j' m := by
      simpa [Set.mem_singleton_iff] using hp
    exact (hne (pair_inj this).1).elim
  * intro hp
    simp [botElem] at hp

theorem succ_singleton (j m : Nat) :
    sigmaJ j {pair j m} = ({pair j (m + 1)} : Pomega) := by
  ext p
  constructor
  * intro <k, heq, hp>
    have : k = m := (pair_inj hp).2
    subst this
    simp [heq]
  * intro hp
    exact <m, hp.symm, by simp>

theorem union_fresh {j m : Nat} {u : Pomega}
    (hu : forall  k, pair j k  notin  u) :
    sigmaJ j (u  U  {pair j m}) = ({pair j (m + 1)} : Pomega) := by
  rw [show u  U  {pair j m} = ({pair j m} : Pomega)  U  u by
```

```
    exact Set.union_comm _ _]
  ext p
  constructor
  * intro <k, hp, hk>
    rcases hk with hk | hk
    * have hkm : k = m := (pair_inj hk).2
      subst k
      simpa using hp.symm
    * exact (hu k hk).elim
  * intro hp
    exact <m, hp.symm, Or.inl (by simp)>

end sigmaJ

end DomainTheory.DataTypesAsLattices
```

finite_sigma_plain_app.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_9
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.Z
public import DomainTheory.DataTypesAsLattices.Functionality.sigmaJ
public import DomainTheory.DataTypesAsLattices.Functionality.normalized_finite_app

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem finite_sigma_plain_app (p q m : Nat) :
    funOf
        (graph (fun y => funOf (e p) y)  U
          graph (sigmaJ (max p q + 1)))
        (Z m (e p) (e q)) =
      Z (m + 1) (e p) (e q) := by
  rw [eq_2_9, normalized_finite_app, sigmaJ.app,
    sigmaJ.of_no_tag (fun k => Z.finite_no_tag p q m k)]
  simp [botElem, Z]

end DomainTheory.DataTypesAsLattices
```

eq_7_18_ne.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.Z
public import DomainTheory.DataTypesAsLattices.Functionality.sigmaJ
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (7.18).** Distinct tags: `sigma_j^{m+1}({(j',0)}) = False`. -/
theorem eq_7_18_ne (j j' m : Nat) (hne : j  !=  j') :
    Z (m + 1) (graph (sigmaJ j)) {pair j' 0} = botElem := by
  induction m with
  | zero =>
    calc
      Z 1 (graph (sigmaJ j)) {pair j' 0}
        = funOf (graph (sigmaJ j)) {pair j' 0} := rfl
      _ = sigmaJ j {pair j' 0} := sigmaJ.app _ _
      _ = botElem := sigmaJ.succ_ne j j' 0 hne
  | succ m ih =>
    calc
      Z (m + 1 + 1) (graph (sigmaJ j)) {pair j' 0}
        = funOf (graph (sigmaJ j))
            (Z (m + 1) (graph (sigmaJ j)) {pair j' 0}) := rfl
      _ = sigmaJ j (Z (m + 1) (graph (sigmaJ j)) {pair j' 0}) :=
          sigmaJ.app _ _
      _ = sigmaJ j botElem := by rw [ih]
      _ = botElem := sigmaJ.bot j

end DomainTheory.DataTypesAsLattices
```

eq_7_18_eq.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.Z
public import DomainTheory.DataTypesAsLattices.Functionality.sigmaJ

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (7.18).** Equal tags: `sigma_j^m({(j,0)}) = {(j,m)}`. -/
theorem eq_7_18_eq (j m : Nat) :
    Z m (graph (sigmaJ j)) {pair j 0} = ({pair j m} : Pomega) := by
  induction m with
  | zero => rfl
  | succ m ih =>
    calc
      Z (m + 1) (graph (sigmaJ j)) {pair j 0}
        = funOf (graph (sigmaJ j)) (Z m (graph (sigmaJ j)) {pair j 0}) := rfl
      _ = sigmaJ j (Z m (graph (sigmaJ j)) {pair j 0}) := sigmaJ.app _ _
      _ = sigmaJ j {pair j m} := by rw [ih]
      _ = {pair j (m + 1)} := sigmaJ.succ_singleton j m

end DomainTheory.DataTypesAsLattices
```

finite_sigma_union_app.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_9
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.pair
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.Z
public import DomainTheory.DataTypesAsLattices.Functionality.sigmaJ
public import DomainTheory.DataTypesAsLattices.Functionality.
    normalized_finite_fresh_app

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem finite_sigma_union_app (p q m : Nat) :
    funOf
        (graph (fun y => funOf (e p) y)  U
          graph (sigmaJ (max p q + 1)))
        (Z m (e p) (e q)  U  {pair (max p q + 1) m}) =
      Z (m + 1) (e p) (e q)  U  {pair (max p q + 1) (m + 1)} := by
  rw [eq_2_9, normalized_finite_fresh_app, sigmaJ.app,
    sigmaJ.union_fresh (fun k => Z.finite_no_tag p q m k)]

end DomainTheory.DataTypesAsLattices
```

sumMed.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.graph_ext
public import Mathlib.Order.CompleteLattice.Defs
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Embedding.extend
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Functionality.RestrictedEquiv
public import DomainTheory.DataTypesAsLattices.Functionality.pairElem
public import DomainTheory.DataTypesAsLattices.Functionality.arrowE
public import DomainTheory.DataTypesAsLattices.Functionality.sumE
public import DomainTheory.DataTypesAsLattices.Functionality.continuous_nary
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Functionality.mem
public import DomainTheory.DataTypesAsLattices.Lambda.interC
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- Coproduct mediator in the category of restricted equivalences. -/
def sumMed (f g : Pomega) : Pomega :=
  graph (fun u =>
    condSet (funOf u (ofNat 0))
      (funOf f (funOf u (ofNat 1)))
      (funOf g (funOf u (ofNat 1))))

namespace sumMed

theorem body_isScottContinuous (f g : Pomega) :
    IsScottContinuous (fun u =>
      condSet (funOf u (ofNat 0))
        (funOf f (funOf u (ofNat 1)))
        (funOf g (funOf u (ofNat 1)))) :=
  continuous_nary
    (fun y z => condSet.isScottContinuous_left y z)
    (fun x z => condSet.isScottContinuous_mid x z)
    (fun x y => condSet.isScottContinuous_right x y)
    (funOf.isScottContinuous_left (ofNat 0))
    (theorem_1_3 (funOf.isScottContinuous f)
      (funOf.isScottContinuous_left (ofNat 1)))
    (theorem_1_3 (funOf.isScottContinuous g)
      (funOf.isScottContinuous_left (ofNat 1)))

theorem app (f g u : Pomega) :
    funOf (sumMed f g) u =
      condSet (funOf u (ofNat 0))
        (funOf f (funOf u (ofNat 1)))
        (funOf g (funOf u (ofNat 1))) :=
  beta (sumMed.body_isScottContinuous f g) u

theorem app_inl (f g x : Pomega) :
    funOf (sumMed f g) (pairElem (ofNat 0) x) = funOf f x := by
  rw [sumMed.app, pairElem.fst, pairElem.snd, condSet.ofNat_zero]

theorem app_inr (f g x : Pomega) :
    funOf (sumMed f g) (pairElem (ofNat 1) x) = funOf g x := by
  rw [sumMed.app, pairElem.fst, pairElem.snd, condSet.ofNat_one]

theorem isGraph (f g : Pomega) :
    sumMed f g = graph (fun u => funOf (sumMed f g) u) := by
  apply graph_ext
  intro u
  exact (sumMed.app f g u).symm

theorem rel {A B C : RestrictedEquiv} {f g : Pomega}
    (hf : (arrowE A C).mem f) (hg : (arrowE B C).mem g) :
    (arrowE (sumE A B) C).mem (sumMed f g) := by
  refine <sumMed.isGraph f g, sumMed.isGraph f g, ?_>
  intro u v huv
  rcases huv with <hu, hv, hA> | <hu, hv, hB>
```

```
  * have hmu : funOf (sumMed f g) u = funOf f (funOf u (ofNat 1)) :=
      (congrArg (fun w => funOf (sumMed f g) w) hu).trans
        (sumMed.app_inl f g _)
    have hmv : funOf (sumMed f g) v = funOf f (funOf v (ofNat 1)) :=
      (congrArg (fun w => funOf (sumMed f g) w) hv).trans
        (sumMed.app_inl f g _)
    rw [hmu, hmv]
    exact hf.2.2 (funOf u (ofNat 1)) (funOf v (ofNat 1)) hA
  * have hmu : funOf (sumMed f g) u = funOf g (funOf u (ofNat 1)) :=
      (congrArg (fun w => funOf (sumMed f g) w) hu).trans
        (sumMed.app_inr f g _)
    have hmv : funOf (sumMed f g) v = funOf g (funOf v (ofNat 1)) :=
      (congrArg (fun w => funOf (sumMed f g) w) hv).trans
        (sumMed.app_inr f g _)
    rw [hmu, hmv]
    exact hg.2.2 (funOf u (ofNat 1)) (funOf v (ofNat 1)) hB

theorem rel_unique {A B C : RestrictedEquiv} {f g h : Pomega}
    (hh : (arrowE (sumE A B) C).mem h)
    (hinl : forall  x y, A.rel x y  ->
      C.rel (funOf h (pairElem (ofNat 0) x)) (funOf f y))
    (hinr : forall  x y, B.rel x y  ->
      C.rel (funOf h (pairElem (ofNat 1) x)) (funOf g y)) :
    (arrowE (sumE A B) C).rel h (sumMed f g) := by
  refine <hh.1, sumMed.isGraph f g, ?_>
  intro u v huv
  rcases huv with <hu, hv, hA> | <hu, hv, hB>
  * have hhu :
        funOf h u = funOf h (pairElem (ofNat 0) (funOf u (ofNat 1))) :=
      congrArg (fun w => funOf h w) hu
    have hmv : funOf (sumMed f g) v = funOf f (funOf v (ofNat 1)) :=
      (congrArg (fun w => funOf (sumMed f g) w) hv).trans
        (sumMed.app_inl f g _)
    rw [hhu, hmv]
    exact hinl (funOf u (ofNat 1)) (funOf v (ofNat 1)) hA
  * have hhu :
        funOf h u = funOf h (pairElem (ofNat 1) (funOf u (ofNat 1))) :=
      congrArg (fun w => funOf h w) hu
    have hmv : funOf (sumMed f g) v = funOf g (funOf v (ofNat 1)) :=
      (congrArg (fun w => funOf (sumMed f g) w) hv).trans
        (sumMed.app_inr f g _)
    rw [hhu, hmv]
    exact hinr (funOf u (ofNat 1)) (funOf v (ofNat 1)) hB

end sumMed

end DomainTheory.DataTypesAsLattices
```

tagInj_isScottContinuous.lean

```
module

public import DomainTheory.DataTypesAsLattices.Retracts.tagInj
public import Mathlib.Order.CompleteLattice.Defs
```

```
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Functionality.
    pairSeq_isScottContinuous_right

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem tagInj_isScottContinuous (i : Nat) :
    IsScottContinuous (tagInj i) :=
  pairSeq_isScottContinuous_right (ofNat i)

end DomainTheory.DataTypesAsLattices
```

2.8 Enumeration and degrees

Historical note. Reduction to a few combinators is old in combinatory logic. The device special to this model is (3.1), which folds the arithmetical combinators into the one combinator $\mathbf{G}$. The Gödel numbering and the second recursion theorem are the standard constructions, written so that arithmetic and the type-free combinators share a notation, and the incompleteness theorem is the standard one in that notation. Theorem 3.5 is the argument of Myhill and Shepherdson (1955); Martin Hyland pointed out the connection. Theorem 3.6 restates enumeration degrees, as in Rogers (1967), Sasso (1975), and Gutteridge (1971). Scott notes that this area was underdeveloped beside the theory of Turing degrees. Semigroups of combinators had been discussed by Church and by Böhm, and Theorem 3.7 reads that discussion inside $\mathbf{RE} \cap \mathbf{FUN}$.

Dependencies. Inside Enumeration, each box is one Lean file. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. The arrowhead points at the file or folder that imports, and the number is how many direct imports.



Figure 28: Enumeration and degrees.

A great advantage of the combinators from the formal point of view is that (bound) variables are eliminated in favor of "algebraic" combinations. The disadvantage is that the algebra is not all that pretty, as the combinations tend to get rather long and general laws are rather few. Nevertheless as a technical device it is mildly remarkable that we can have a notation for all r.e. sets requiring so few primitives. In the model defined here the reduction to one combinator rests on a lemma about conditionals:

$$(3.1) \quad \mathbf{cond}(x)(y)(\mathbf{cond}(x)(y)) = y.$$

Recall that $\mathbf{cond}$ (or $\supset$) is a test for zero, so that:

$$(3.2) \quad \mathbf{cond}(x)(y)(0) = x.$$

This suggests that we lump all combinators of Theorem 2.4 into this one:

$$(3.3) \quad \mathbf{G} = \mathbf{cond}(\langle \mathbf{suc}, \mathbf{pred}, \mathbf{cond}, \mathbf{K}, \mathbf{S} \rangle)(0).$$

We can then readily prove:

- Theorem 3.1
- Theorem 3.2
- Theorem 3.3
- Theorem 3.4
- Theorem 3.5
- Theorem 3.6
- Theorem 3.7

Deg.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Enumeration.Icomb_SK
public import DomainTheory.DataTypesAsLattices.Enumeration.Gcomb
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Enumeration.Subalgebra
public import DomainTheory.DataTypesAsLattices.Enumeration.Gcomb_more
public import DomainTheory.DataTypesAsLattices.Lambda.Kcomb_more
public import DomainTheory.DataTypesAsLattices.Lambda.Scomb_more

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable

open Nat.Partrec (Code)

open Nat.Partrec.Code

/-- **Scott 1976, Definition.** Enumeration degree of `a`. -/
def Deg (a : Pomega) : Set Pomega :=
  {u | exists  r, IsCombinatory r  /\  u = funOf r a}

namespace Deg
```

```
open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem isSubalgebra (a : Pomega) : Subalgebra (Deg a) := by
  constructor
  constructor
  * refine <funOf Kcomb Gcomb, IsCombinatory.app .K Gcomb.combinatory, ?_>
    rw [Kcomb.beta2]
  * intro x y hx hy
    obtain <r, hr, rfl> := hx
    obtain <s, hs, rfl> := hy
    refine <funOf (funOf Scomb r) s, .app (.app .S hr) hs, ?_>
    rw [Scomb.beta3]

theorem self (a : Pomega) : a  in  Deg a :=
  <Icomb_SK, .app (.app .S .K) .K, (Icomb_SK.app a).symm>

end Deg

end DomainTheory.DataTypesAsLattices
```

Rcomb.lean

```
module

public import DomainTheory.DataTypesAsLattices.Enumeration.
    seq2_right_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Lambda.seq2
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Lambda.FUN
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Enumeration.graph_mem_FUN

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (3.13)-(3.14).** Semigroup generators. -/
def Rcomb : Pomega := graph (fun x => seq2 (ofNat 0) x)

namespace Rcomb

theorem app (x : Pomega) : funOf Rcomb x = seq2 (ofNat 0) x :=
  beta (seq2_right_isScottContinuous (ofNat 0)) x

theorem app_zero (x : Pomega) :
    funOf (funOf Rcomb x) (ofNat 0) = ofNat 0 := by
  rw [app, seq2.zero]
```

```
theorem mem_FUN : Rcomb  in  FUN :=
  graph_mem_FUN (seq2_right_isScottContinuous (ofNat 0))

end Rcomb

end DomainTheory.DataTypesAsLattices
```

barPos.lean

```
module

public import DomainTheory.DataTypesAsLattices.Enumeration.
    seqCons_right_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Enumeration.seq2_left_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Enumeration.
    seq2_right_isScottContinuous
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_22
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_7_succ
public import DomainTheory.DataTypesAsLattices.Lambda.eq_2_7_zero
public import DomainTheory.DataTypesAsLattices.Lambda.seq2
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.seqCons
public import DomainTheory.DataTypesAsLattices.Lambda.dollarStepBody
public import DomainTheory.DataTypesAsLattices.Lambda.predSet
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (3.15).** Positive branch of `u`: `<1, u, x_1>`. -/
def barPos (u x : Pomega) : Pomega :=
  seqCons (ofNat 1) (seq2 u (funOf x (ofNat 1)))

namespace barPos

theorem isScottContinuous_u (x : Pomega) :
    IsScottContinuous (fun u => barPos u x) :=
  theorem_1_3 (seqCons_right_isScottContinuous (ofNat 1))
    (seq2_left_isScottContinuous (funOf x (ofNat 1)))

theorem isScottContinuous_x (u : Pomega) :
    IsScottContinuous (fun x => barPos u x) :=
  theorem_1_3 (f := fun xs => seqCons (ofNat 1) xs)
    (g := fun x => seq2 u (funOf x (ofNat 1)))
    (seqCons_right_isScottContinuous (ofNat 1))
    (theorem_1_3 (seq2_right_isScottContinuous u)
      (funOf.isScottContinuous_left (ofNat 1)))

theorem zero (u x : Pomega) :
```

```
    funOf (barPos u x) (ofNat 0) = ofNat 1 := by
  simp [barPos, _root_.DomainTheory.DataTypesAsLattices.eq_2_22, eq_2_7_zero]

theorem one (u x : Pomega) :
    funOf (barPos u x) (ofNat 1) = u := by
  rw [barPos, _root_.DomainTheory.DataTypesAsLattices.eq_2_22]
  have hpred : predSet (ofNat 1) = ofNat 0 := by
    ext k; simp [predSet, ofNat]
  rw [hpred, eq_2_7_succ, seq2.zero]

theorem two (u x : Pomega) :
    funOf (barPos u x) (ofNat 2) = funOf x (ofNat 1) := by
  rw [barPos, _root_.DomainTheory.DataTypesAsLattices.eq_2_22]
  have hpred : predSet (ofNat 2) = ofNat 1 := by
    ext k; simp [predSet, ofNat]
  rw [hpred, eq_2_7_succ, seq2.one]

end barPos

end DomainTheory.DataTypesAsLattices
```

barComb.lean

```
module

public import DomainTheory.DataTypesAsLattices.Enumeration.barStep
public import DomainTheory.DataTypesAsLattices.FixedPoint.fix
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb
public import DomainTheory.DataTypesAsLattices.Lambda.interp
public import DomainTheory.DataTypesAsLattices.Enumeration.barStepTerm
public import DomainTheory.DataTypesAsLattices.Lambda.Ycomb_more
public import DomainTheory.DataTypesAsLattices.Lambda.theorem_2_5

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (3.15).** The bar combinator as a least fixed point. -/
def barComb : Pomega := fix barStep

namespace barComb

theorem combinatory : IsCombinatory barComb := by
  have hY : funOf Ycomb (interp barStepTerm (fun _ => botElem)) = barComb := by
    rw [barStepTerm.interp, theorem_2_5 barStep.isScottContinuous]
    rfl
  exact hY  |>  IsCombinatory.app Ycomb.combinatory (theorem_2_4.closed barStepTerm)

end barComb

end DomainTheory.DataTypesAsLattices
```

Gbar.lean

```
module

public import Mathlib.Computability.Halting
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Enumeration.barComb
public import DomainTheory.DataTypesAsLattices.Enumeration.eq_3_15
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.theorem_1_2
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import Mathlib.Order.CompleteLattice.Defs
public import DomainTheory.DataTypesAsLattices.Lambda.FUN
public import DomainTheory.DataTypesAsLattices.Enumeration.barStepBody
public import DomainTheory.DataTypesAsLattices.Enumeration.Gcomb
public import DomainTheory.DataTypesAsLattices.Enumeration.Gcomb_more

@[expose] public section

set_option maxHeartbeats 2000000

namespace DomainTheory.DataTypesAsLattices

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

/-- Scott's `G`. -/
def Gbar : Pomega := funOf barComb Gcomb

namespace Gbar

open Encodable
open Nat.Partrec (Code)
open Nat.Partrec.Code

theorem combinatory : IsCombinatory Gbar :=
  .app barComb.combinatory Gcomb.combinatory

theorem mem_FUN : Gbar  in  FUN := by
  change Gbar = graph (funOf Gbar)
  have h : Gbar = graph (fun x => barStepBody barComb Gcomb x) :=
    _root_.DomainTheory.DataTypesAsLattices.eq_3_15 Gcomb
  have hf : IsScottContinuous (fun x => barStepBody barComb Gcomb x) :=
    barStepBody.isScottContinuous_x barComb Gcomb
  rw [h, theorem_1_2.i hf]

end Gbar

end DomainTheory.DataTypesAsLattices
```

3 Neighborhood Systems (1980)

Dependency diagram. Each box is one section. The arrowhead is on the section that imports, and the number on it is how many import lines run that way. An arrow already implied by a longer path is left out. The introduction has no Lean file, so it has no arrow.

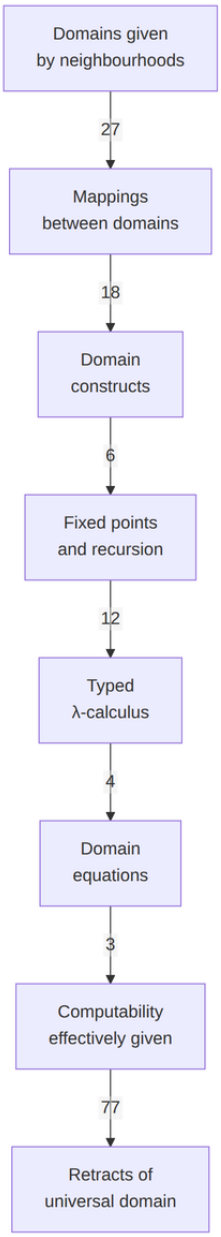


Figure 29: Neighborhood Systems (1980).

3.1 Introduction

These notes were written in conjunction with the lectures delivered by me for the Semantics of Programming Languages sequence during Michaelmas Term 1980 at Oxford. I started writing around the first week of October and finished at the end of November. The purpose of the course was to provide the foundations needed for the method of denotational semantics; in particular, I wanted to make the connections with recursive function theory more definite and to show explicit, effectively given solutions to domain equations. Roughly, these chapters cover the first half of the book of J.E. Stoy. I plan soon to expand the notes into a book by adding additional chapters on other theoretical topics that time did not permit me to cover in one eight-week term.

When I started writing Lecture I in October, I did not know what the later lectures would contain: I could see no further ahead than part of Lecture III in the beginning. The lectures had to be typed in advance of the class meetings, however, so there was at the time of composition no opportunity for second thoughts of any major proportions: I had to write the text straight through. As a consequence there are many remarks I would like to transpose and many additional points of explanation I see are needed; further worked examples and easier exercises are also required. During the spring, after receiving many helpful comments, I was able to introduce a few changes in the text and make some necessary corrections. However, a complete retyping was impossible. Nevertheless, this preliminary version of the book seems to provide a quite detailed introduction and is sufficient to exhibit the scope of the approach and several applications.

The idea of using neighbourhood systems to give set-theoretical representations of domains had been in the back of

my mind for some time in connection with specific examples. But the thought that a systematic development along these lines might be easier to follow than the more abstract lattice-theoretic and topological approach used by myself and others in many publications only came to me during the IFIP Working Group 2.2 meeting in Copenhagen in mid-June 1980. I gave a brief public presentation at ICALP '80 in Holland in mid-July.

One large mistake I have made is to de-emphasize partial orderings too much, since at the right point the concepts and the language are in fact helpful. The basic plan is that, instead of axiomatizing the theory using partial orderings, the necessary facts come out as *theorems*. For a neighbourhood system $\mathcal{D}$, the set of elements $|\mathcal{D}|$, which consists of filters, is naturally partially ordered. And approximable mappings naturally *preserve* the ordering. And so on. The advantage I see from the point of view of exposition is that properties can be brought out one at a time instead of having to put them down all in advance of any experience with the ideas. My own feeling after writing these chapters is that the plan has worked out far better than I could have dared to hope. I was especially glad that I could generate so many *exercises*, and I hope eventually to provide many more. One interesting place at which partial orderings prove their usefulness is in visualizing domains. As it stands now the text does not contain enough in the way of pictures. This will have to be remedied in a future version. Undoubtedly to include enough explanation, several of the lectures will have to be sub-divided into separate chapters.

One major improvement is needed: to bring Exercise 2.22 into the main text. I did not know in advance how often I would need this result for giving (easy) set-theoretical characterizations of domains and structure on them. This will be an easy repair, but it will cause quite a bit of rewriting. Clearly

much more has to be said about the interplay between elements and neighbourhoods. In particular, the character of the elements of a domain, like the power set of a set, has not been sufficiently illustrated, and quite a bit of expansion on this topic is also needed.

Finally I have to explain that I had no time whatsoever to put in references and a bibliography. The ideas I have used have occurred to many, many people who have worked on domains, and I do not wish to claim

originality. I *am* claiming some advantage to my style of representation, but I fully realize that a published version will have to have detailed historical references and notes at the ends of the lectures. Needless to say I should very much appreciate any advice or criticism from readers of this preliminary version.

I would like to give a warm word of thanks to the many people who have already commented on the preliminary text both at Oxford and in Boston, where I gave lectures. Very special thanks are due to Steve Comer and Steve Brookes, who spent many hours proof reading the typescripts. The biggest word of thanks, however, is reserved for Elsie Hinkes who, under very considerable pressure, did a wonderful job of typing.

Dana S. Scott
Merton College
Oxford
May 1981

3.2 Domains given by neighbourhoods

Dependency diagram. Each box is one item. The arrowhead is on the item that imports. An arrow already implied by a longer path is left out. The introduction has no Lean file, so it is not shown. Blue is a definition, green a factoid, amber an example, red a theorem, and indigo an exercise.

3.2.1 Introduction

LECTURE I DOMAINS GIVEN BY NEIGHBOURHOODS

Often an object (or element) can be determined by a selection of its properties. Often it is also the case that it is easier (more convenient, more elementary) to think of these properties than it is to think of the elements themselves. Let us term the properties under consideration *neighbourhoods*, the family of those allowed a *neighbourhood system*.

Generally, the collection of these neighbourhoods is, for one reason or another, somewhat restricted; that is, a completely arbitrary property may not be allowable as a neighbourhood. Therefore, the elements determined by selections of neighbourhoods may not be as separable into the discrete objects common to the classical view of set theory. This is particularly true in working with infinite objects: it is hard to specify an infinite element completely. The theory of elements to be studied here, then, is going to permit *partial elements* as well as *total elements*, and each neighbourhood system will define a *domain* of such elements.

Since we may wish to use a neighbourhood system to introduce elements not previously investigated, the neighbourhoods do not have to be regarded as sets of the as-yet-to-be-defined objects. We can take a non-empty set Δ of *tokens* (or "traces") that function as "parts" of elements — or even as parts of "descriptions" of elements. Then a neighbourhood is a subset $X \subseteq \Delta$ containing all those tokens that provide sufficient information when taken together to "approximate" a possible element up to a certain "degree". All these words in inverted commas are vague, and in any case we shall have at the start only a *qualitative* theory of "degree of approximation". A token should be considered as a very "rough" representative of an element, and a neighbourhood should be regarded as "smoothing out" irrelevant details by grouping together *all* those representatives sharing some common feature. One neighbourhood, then,

may be only a very incomplete specification of an (ideal) element; fuller specifications can be secured by taking "convergent" sequences of neighbourhoods. Even then convergence need not be to a total element.

Let us call the family of allowed neighbourhoods $\mathcal{D}$; it is a family of subsets of the set Δ. An obvious first question is: when are two neighbourhoods $X, Y \in \mathcal{D}$ neighbourhoods of the "same" element? This question of course generalizes to a (finite) sequence of neighbourhoods. This property we will call the *consistency* of the sequence of neighbourhoods. By definition this will mean that the given neighbourhoods

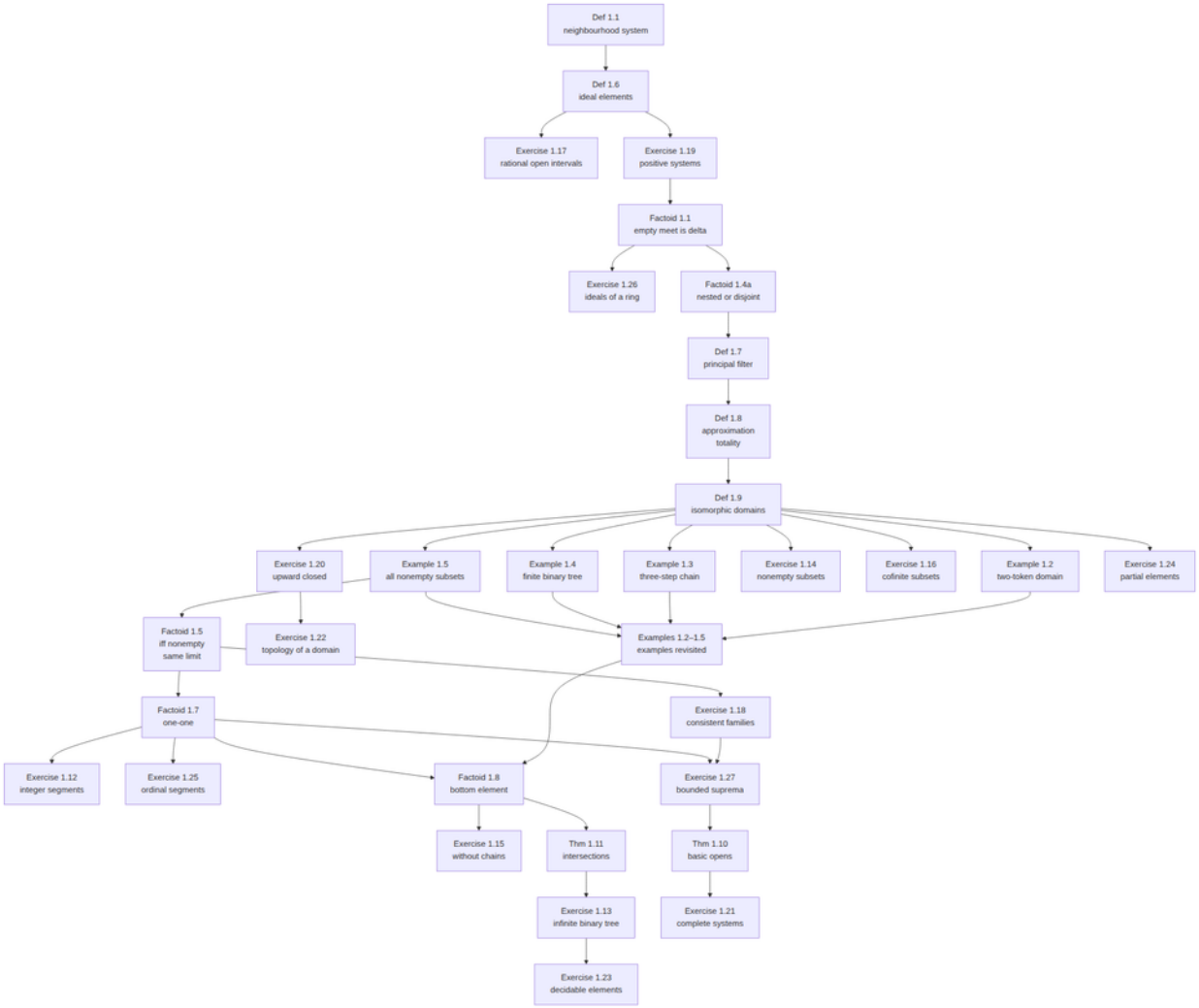


Figure 30: Domains given by neighbourhoods.

all contain a common neighbourhood in $\mathcal{D}$. That is, for X, Y to be consistent, there must be a $Z \in \mathcal{D}$ with $Z \subseteq X$ and $Z \subseteq Y$. This is not a very informative definition, but it has something of the flavour of a notion of consistency insofar as it can be expressed within $\mathcal{D}$. When consistency holds it seems reasonable enough at first glance to say that the intersection $X \cap Y$ is also an approximation to this common element. If this is reasonable, then $X \cap Y$ should also be regarded as a neighbourhood. This assumption has many consequences, but as a preliminary theory of approximation we will find it quite workable with many natural instances. Taking intersections just means taking more and more properties of the element and putting them together "conjunctively". It is something we do all the time. We therefore accept the idea for the present for giving our first principal definition.

3.2.2 Definition 1.1 — neighbourhood system

DEFINITION 1.1. A family $\mathcal{D}$ of subsets of a given set Δ is called a *neighbourhood system* (over Δ) iff it is a non-empty family closed under the intersection of finite consistent sequences of neighbourhoods. That is to say, $\mathcal{D}$ must fulfill these two conditions:

(i) $\Delta \in \mathcal{D}$;

(ii) whenever $X, Y, Z \in \mathcal{D}$ and $Z \subseteq X \cap Y$, then $X \cap Y \in \mathcal{D}$. □

We remark that by convention Δ corresponds to the intersection of an empty sequence of neighbourhoods; in particular, see Factoid 1.1.

NeighborhoodSystem.lean

```
module

public import Mathlib.Algebra.Order.Archimedean.Real.Basic
public import Mathlib.Basic.Countable.Defs
public import Mathlib.Basic.Real.Basic
public import Mathlib.Data.Fintype.Powerset
public import Mathlib.Data.List.Infix
public import Mathlib.Data.Rat.Cast.Order
public import Mathlib.Data.Set.Basic
public import Mathlib.Data.Set.Finite.Basic
public import Mathlib.Order.Bounds.Basic
public import Mathlib.Order.Hom.Basic
public import Mathlib.Order.Monotone.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Zorn
public import Mathlib.RingTheory.Ideal.Span
public import Mathlib.Tactic
public import Mathlib.Topology.Inseparable

@[expose] public section

section
-- NeighborhoodSystem.lean
namespace DomainTheory.NeighbourhoodSystems

/- concordance: source L120 DEFINITION 1.1 neighbourhood system -/
/-- **Definition 1.1 (Scott 1981, PRG-19).** A *neighbourhood system* over a token
    type
```

```
`a`. `mem X` means "`X` is a neighbourhood" (`X  in  ?`), and `master` is Scott's least
informative neighbourhood `Delta` (the whole token set, "ask me no questions").

The two conditions are exactly Scott's:

* (i)  `Delta  in  ?`                                          -- `master_mem`;
* (ii) whenever `X, Y, Z  in  ?` and `Z  subseteq  X  I  Y`, then `X  I  Y  in  ?` -- `
    inter_mem`.

We keep `master` as a field (rather than hard-wiring `Set.univ`) to stay faithful to
Scott's `Delta` notation, and record Scott's standing assumption `?  subseteq  ?(Delta)
    ` as the field
`sub_master` (every neighbourhood is a subset of `Delta`). Scott also assumes from the
    outset
that the token set `Delta` is non-empty; `master_nonempty` records this standing
    assumption
explicitly. The subset condition is what makes the principal filter `X` (Definition
    1.7)
contain `Delta`, and underlies the least element `False = Delta`. -/
structure NeighborhoodSystem (a : Type*) where
  /-- `mem X` holds iff `X` is a neighbourhood of the system (`X  in  ?`). -/
  mem : Set a  ->  Prop
  /-- Scott's distinguished least-informative neighbourhood `Delta`. -/
  master : Set a
  /-- Scott's standing assumption that the token set `Delta` is non-empty. -/
  master_nonempty : master.Nonempty
  /-- (i) `Delta  in  ?`. -/
  master_mem : mem master
  /-- (ii) Closure under intersection of a *consistent* pair: if `X, Y, Z  in  ?` with
    the
  witness `Z  subseteq  X  I  Y`, then `X  I  Y  in  ?`. -/
  inter_mem : forall  {X Y Z : Set a}, mem X  ->  mem Y  ->  mem Z  ->  Z  subseteq  X
      I  Y  ->  mem (X  I  Y)
  /-- Scott's `?  subseteq  ?(Delta)`: every neighbourhood is a subset of the master
    neighbourhood `Delta`. -/
  sub_master : forall  {X : Set a}, mem X  ->  X  subseteq  master

end DomainTheory.NeighbourhoodSystems
end
```

3.2.3 Factoid 1.1 — empty intersection is delta; recursive finite intersection

Factoid 1.1a — empty intersection is delta

$$\bigcap_{i<n} X_i = \Delta, \text{ if } n = 0;$$

Factoid 1.1b — recursive finite intersection

$$= \left(\bigcap_{i<n-1} X_i \right) \cap X_{n-1}, \text{ if } n > 0.$$

Of course, from (ii), we can extend the intersection property to any finite sequence. Consequently, we can say $X_0, \ldots, X_{n-1}$ is consistent in $\mathcal{D}$ iff

$$\bigcap_{i<n} X_i \in \mathcal{D}.$$

Some examples will help us understand the notions better.

Factoid_1_1.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_19

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem
section

variable {a : Type*} (V : NeighborhoodSystem a)

/-- The finite intersection `?_{i < n} X_i` of the first `n` terms of a sequence of
/- concordance: source L131 FACTOID 1.1a empty intersection is delta -/
neighbourhoods, defined by Scott's recursive convention (**Factoid 1.1a / 1.1b**):

* `n = 0` : the empty intersection is `Delta` (`master`);
* `n + 1` : `(?_{i < n} X_i)  I  X_n`.

(See `interUpTo_zero` and `interUpTo_succ` for the two defining equations as lemmas.)
    -/
def interUpTo (V : NeighborhoodSystem a) (X : Nat  ->  Set a) : Nat  ->  Set a
  | 0 => V.master
  | (n + 1) => interUpTo V X n  I  X n

/-- **Factoid 1.1a.** The intersection of the empty sequence of neighbourhoods is `
    Delta`:
`?_{i < 0} X_i = Delta`. -/
@[simp] theorem interUpTo_zero (X : Nat  ->  Set a) : V.interUpTo X 0 = V.master :=
    rfl

/- concordance: source L135 FACTOID 1.1b recursive finite intersection -/
/-- **Factoid 1.1b.** The intersection of the first `n + 1` neighbourhoods peels off
    the
last factor: `?_{i < n+1} X_i = (?_{i < n} X_i)  I  X_n`. -/
@[simp] theorem interUpTo_succ (X : Nat  ->  Set a) (n : Nat) :
    V.interUpTo X (n + 1) = V.interUpTo X n  I  X n := rfl

/-- A finite sequence `X_0, ..., X_n?_1` of neighbourhoods is *consistent in* `?` when
    it has a
common lower bound inside `?`: some `Z  in  ?` contained in the intersection `?_{i < n}
     X_i`
(equivalently, contained in every `X?`, `j < n`). This is Scott's notion of consistency
    ,
generalized from pairs to finite sequences. -/
def Consistent (X : Nat  ->  Set a) (n : Nat) : Prop :=
  exists  Z, V.mem Z  /\  Z  subseteq  V.interUpTo X n
```

```
/-- Filter-inclusion order on elements (Scott's approximation order, Definition 1.8).
Named so Palomar can lock the `PartialOrder` relation without inlining it. -/
def element_le (x y : V.Element) : Prop :=
  forall  X, x.mem X  ->  y.mem X

/-- Reflexivity of the filter-inclusion order. Named so Palomar can lock the
`PartialOrder` instance without a generated `._proof_N`. -/
theorem element_le_refl (x : V.Element) : forall  X, x.mem X  ->  x.mem X :=
  fun _ h => h

/-- Transitivity of the filter-inclusion order. -/
theorem element_le_trans (x y z : V.Element)
    (hxy : forall  X, x.mem X  ->  y.mem X) (hyz : forall  X, y.mem X  ->  z.mem X) :
    forall  X, x.mem X  ->  z.mem X :=
  fun X h => hyz X (hxy X h)

/-- Two elements with the same membership predicate are equal (the remaining fields
    are `Prop`s). -/
theorem Element.ext {x y : V.Element} (h : forall  X, x.mem X  <->  y.mem X) : x = y
    := by
  rcases x with <xmem, _, _, _, _>
  rcases y with <ymem, _, _, _, _>
  have hmem : xmem = ymem := funext fun X => propext (h X)
  subst hmem
  rfl

/-- Antisymmetry of the filter-inclusion order. -/
theorem element_le_antisymm (x y : V.Element)
    (hxy : forall  X, x.mem X  ->  y.mem X) (hyx : forall  X, y.mem X  ->  x.mem X) :
    x = y :=
  @Element.ext a V x y fun X => <hxy X, hyx X>

/-- Elements are ordered by inclusion of their membership predicates (Scott's
    approximation
order, Definition 1.8). -/
instance : PartialOrder V.Element where
  le := element_le V
  le_refl := element_le_refl V
  le_trans := element_le_trans V
  le_antisymm := element_le_antisymm V

/-- Two sequences of neighbourhoods are **equivalent** ("each goes equally deep as the
     other"):
for every `Y?` some `X_n  subseteq  Y?`, and for every `X_n` some `Y?  subseteq  X_n`.
    -/
def SeqEquiv (X Y : Nat  ->  Set a) : Prop :=
  (forall  m, exists  n, X n  subseteq  Y m)  /\  (forall  n, exists  m, Y m  subseteq
     X n)

/-- The **limit family** of a sequence of neighbourhoods (Scott, the prose before
    Definition
1.6): `x = {Z  in  ? ? X_n  subseteq  Z for some n}` -- the family of all
    neighbourhoods eventually reached
by `<X_n>`. This is the construction Scott uses to motivate the (ideal) elements of
    `|?|`. -/
```

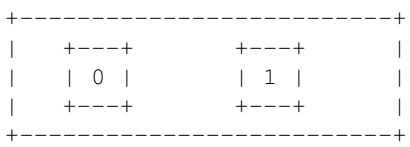
```
def limitFamily (X : Nat  ->  Set a) : Set (Set a) := {Z | V.mem Z  /\  exists  n, X n
      subseteq  Z}
end

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
end
```

3.2.4 Example 1.2 — two-token domain

EXAMPLE 1.2. Let $\Delta = \{0, 1\}$ and let $\mathcal{D} = \{\{0, 1\}, \{0\}, \{1\}\}$.

In pictures we have:



The intention is that 0 and 1 can be completely specified and that they can be identified with the total elements. As we shall see, there is only one partial element: either we give no information (the neighbourhood $\{0, 1\}$), or we decide between 0 and 1 (by giving $\{0\}$ or $\{1\}$). □

Example_1_2.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.DomainIso

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems.Example12
/-- Tokens for Example 1.2: `Delta = {0, 1}`. -/
abbrev Token := Fin 2

/-- The master neighbourhood `Delta = {0, 1}`. -/
def master : Set Token := Set.univ

theorem inter_master (A : Set Token) : A  I  master = A := by
  rw [master]; exact Set.inter_univ A

theorem master_inter (A : Set Token) : master  I  A = A := by
  rw [master]; exact Set.univ_inter A

/-- The neighbourhood `{1}`. -/
def one : Set Token := {1}

theorem master_not_subset_one : not master  subseteq  one := by
  intro h
  have : (0 : Token)  in  one := h (by simp [master])
  simp [one] at this

/-- The neighbourhood `{0}`. -/
def zero : Set Token := {0}
```

```
theorem master_not_subset_zero : not master  subseteq  zero := by
  intro h
  have : (1 : Token)  in  zero := h (by simp [master])
  simp [zero] at this

/-- The three neighbourhoods of Example 1.2. -/
def memSet : Set (Set Token) := {master, zero, one}

/-- Membership in the neighbourhood system `?` of Example 1.2. -/
def mem (X : Set Token) : Prop := X  in  memSet

theorem mem_master : mem master := by simp [mem, memSet, master, zero, one]

theorem mem_one : mem one := by simp [mem, memSet, master, zero, one]

theorem mem_zero : mem zero := by simp [mem, memSet, master, zero, one]

/-- A neighbourhood of Example 1.2 is exactly one of `Delta`, `{0}`, or `{1}`. -/
theorem mem_iff (X : Set Token) : mem X  <->  X = master  \/  X = zero  \/  X = one :=
     by
  constructor
  * intro h
    simp [mem, memSet, master, zero, one] at h
    rcases h with rfl | rfl | rfl
    * exact Or.inl rfl
    * exact Or.inr (Or.inl rfl)
    * exact Or.inr (Or.inr rfl)
  * intro h
    rcases h with rfl | rfl | rfl
    * exact mem_master
    * exact mem_zero
    * exact mem_one

theorem eq_of_master_subset {Y : Set Token} (h : mem Y) (hsub : master  subseteq  Y) :
     Y = master := by
  rcases (mem_iff Y).mp h with rfl | hzero | hone
  * rfl
  * exact absurd hzero (fun h' => master_not_subset_zero (h'  |>  hsub))
  * exact absurd hone (fun h' => master_not_subset_one (h'  |>  hsub))

theorem not_mem_empty : not mem ({} : Set Token) := by
  intro h
  rcases (mem_iff ({} : Set Token)).mp h with h | h | h
  * rw [master] at h; exact Set.empty_ne_univ h
  * simp [zero] at h
  * simp [one] at h

theorem one_inter_zero : one  I  zero = ({} : Set Token) := by
  ext t; fin_cases t <;> simp [zero, one]

theorem one_not_subset_zero : not one  subseteq  zero := by
  intro h
  have : (1 : Token)  in  zero := h (by simp [one])
  simp [zero] at this
```

```
theorem zero_inter_one : zero  I  one = ({} : Set Token) := by
  ext t; fin_cases t <;> simp [zero, one]

theorem inter_eq (X Y : Set Token) (h : mem X) (h' : mem Y) :
    X  I  Y = master  \/  X  I  Y = zero  \/  X  I  Y = one  \/  X  I  Y = ({} : Set
    Token) := by
  rcases (mem_iff X).mp h with rfl | rfl | rfl <;>
    rcases (mem_iff Y).mp h' with rfl | rfl | rfl
  * exact Or.inl (master_inter _)
  * exact Or.inr (Or.inl (master_inter _))
  * exact Or.inr (Or.inr (Or.inl (master_inter _)))
  * exact Or.inr (Or.inl (inter_master _))
  * exact Or.inr (Or.inl (Set.inter_self _))
  * exact Or.inr (Or.inr (Or.inr zero_inter_one))
  * exact Or.inr (Or.inr (Or.inl (inter_master _)))
  * exact Or.inr (Or.inr (Or.inr one_inter_zero))
  * exact Or.inr (Or.inr (Or.inl (Set.inter_self _)))

/- concordance: source L146 EXAMPLE 1.2 two-token domain -/
/-- **Example 1.2.** The neighbourhood system on `Delta = {0, 1}`. -/
def neighborhoodSystem : NeighborhoodSystem Token where
  mem
  master
  master_nonempty := Set.univ_nonempty
  master_mem := mem_master
  sub_master := fun _ => Set.subset_univ _
  inter_mem := by
    intro X Y Z hX hY hZ hZsub
    rcases inter_eq X Y hX hY with h | h | h | h
    * rw [h]; exact mem_master
    * rw [h]; exact mem_zero
    * rw [h]; exact mem_one
    * rw [h] at hZsub
      have hz : Z = ({} : Set Token) := Set.subset_empty_iff.mp hZsub
      subst hz
      exact absurd hZ not_mem_empty

namespace neighborhoodSystem
section

open NeighborhoodSystem

/-- The bottom element `False = {Delta}`. -/
def bot : neighborhoodSystem.Element where
  mem X := X = master
  sub h := by rw [h]; exact mem_master
  master_mem := rfl
  inter_mem := by
    intro X Y hX hY
    rw [hX, hY, master_inter]
  up_mem := by
    intro X Y hX hY hXY
    rw [hX] at hXY
    exact eq_of_master_subset hY hXY

/-- The total element determined by `{1}`. -/
```

```
def elemOne : neighborhoodSystem.Element where
  mem X := X = master  \/  X = one
  sub h := by
    rcases h with rfl | rfl
    * exact mem_master
    * exact mem_one
  master_mem := Or.inl rfl
  inter_mem := by
    intro X Y hX hY
    rcases hX with rfl | rfl <;> rcases hY with rfl | rfl
    * exact Or.inl (master_inter _)
    * exact Or.inr (master_inter _)
    * exact Or.inr (inter_master _)
    * exact Or.inr (Set.inter_self _)
  up_mem := by
    intro X Y hX hY hXY
    rcases hX with rfl | rfl
    * exact Or.inl (eq_of_master_subset hY hXY)
    * rcases (mem_iff Y).mp hY with rfl | rfl | rfl
      * exact Or.inl rfl
      * exact absurd hXY one_not_subset_zero
      * exact Or.inr rfl
end

end neighborhoodSystem
theorem zero_not_subset_one : not zero  subseteq  one := by
  intro h
  have : (0 : Token)  in  one := h (by simp [zero])
  simp [one] at this

namespace neighborhoodSystem
section

open NeighborhoodSystem

/-- The total element determined by `{0}`. -/
def elemZero : neighborhoodSystem.Element where
  mem X := X = master  \/  X = zero
  sub h := by
    rcases h with rfl | rfl
    * exact mem_master
    * exact mem_zero
  master_mem := Or.inl rfl
  inter_mem := by
    intro X Y hX hY
    rcases hX with rfl | rfl <;> rcases hY with rfl | rfl
    * exact Or.inl (master_inter _)
    * exact Or.inr (master_inter _)
    * exact Or.inr (inter_master _)
    * exact Or.inr (Set.inter_self _)
  up_mem := by
    intro X Y hX hY hXY
    rcases hX with rfl | rfl
    * exact Or.inl (eq_of_master_subset hY hXY)
    * rcases (mem_iff Y).mp hY with rfl | rfl | rfl
      * exact Or.inl rfl
```

```
      * exact Or.inr rfl
      * exact absurd hXY zero_not_subset_one
end

end neighborhoodSystem
theorem zero_ne_master : zero  !=  master := by
  intro h
  have : (1 : Token)  in  zero := h  |>  (by simp [master])
  simp [zero] at this

theorem one_ne_master : one  !=  master := by
  intro h
  have : (0 : Token)  in  one := h  |>  (by simp [master])
  simp [one] at this

namespace neighborhoodSystem

open NeighborhoodSystem

theorem mem_zero_of_mem (x : neighborhoodSystem.Element) (h : x.mem zero) :
    x = elemZero := by
  apply Element.ext
  intro X
  constructor
  * intro hx
    rcases (mem_iff X).mp (x.sub hx) with rfl | hzero | hone
    * exact Or.inl rfl
    * exact Or.inr hzero
    * have hxone : x.mem one := hone  |>  hx
      have := x.inter_mem h hxone
      rw [zero_inter_one] at this
      exact absurd (x.sub this) not_mem_empty
  * intro hx
    rcases hx with rfl | hx
    * exact x.master_mem
    * rw [hx]; exact h

theorem mem_one_of_mem (x : neighborhoodSystem.Element) (h : x.mem one) :
    x = elemOne := by
  apply Element.ext
  intro X
  constructor
  * intro hx
    rcases (mem_iff X).mp (x.sub hx) with rfl | hzero | hone
    * exact Or.inl rfl
    * have hxzero : x.mem zero := hzero  |>  hx
      have := x.inter_mem hxzero h
      rw [zero_inter_one] at this
      exact absurd (x.sub this) not_mem_empty
    * exact Or.inr hone
  * intro hx
    rcases hx with rfl | hx
    * exact x.master_mem
    * rw [hx]; exact h

theorem zero_ne_one : zero  !=  (one : Set Token) := by
```

```
  intro h
  have h0 : (0 : Token)  in  zero := by simp [zero]
  rw [h] at h0
  simp [one] at h0

section

open NeighborhoodSystem

theorem bot_lt_elemOne : bot < elemOne := by
  constructor
  * intro X hx; exact Or.inl hx
  * intro h
    have : bot.mem one := h one (Or.inr rfl)
    have hm : one = master := this
    exact one_ne_master hm

/-- The bottom filter is the only partial element: it is strictly below both total
    elements. -/
theorem bot_lt_elemZero : bot < elemZero := by
  constructor
  * intro X hx; exact Or.inl hx
  * intro h
    have : bot.mem zero := h zero (Or.inr rfl)
    have hm : zero = master := this
    exact zero_ne_master hm

/-- Every element of Example 1.2 is one of the three filters `False`, `{0}`-total,
    `{1}`-total. -/
theorem element_classification (x : neighborhoodSystem.Element) :
    x = bot  \/  x = elemZero  \/  x = elemOne := by
  by_cases h0 : x.mem zero
  * exact Or.inr (Or.inl (mem_zero_of_mem x h0))
  by_cases h1 : x.mem one
  * exact Or.inr (Or.inr (mem_one_of_mem x h1))
  apply Or.inl
  apply Element.ext
  intro X
  constructor
  * intro hx
    rcases (mem_iff X).mp (x.sub hx) with rfl | hzero | hone
    * rfl
    * exact absurd (hzero  |>  hx) h0
    * exact absurd (hone  |>  hx) h1
  * intro hx
    rw [hx]
    exact x.master_mem

theorem bot_is_unique_partial (x : neighborhoodSystem.Element) :
    x  !=  elemZero  ->  x  !=  elemOne  ->  x = bot := by
  intro hne0 hne1
  rcases element_classification x with hx | hx | hx
  * exact hx
  * exact (hne0 hx).elim
  * exact (hne1 hx).elim
```

```
theorem not_elemOne_le_elemZero : not  elemOne  <=  elemZero := by
  intro h
  rcases h one (Or.inr rfl) with h' | h'
  * exact one_ne_master h'
  * exact zero_ne_one h'.symm

/-- **`elemZero` and `elemOne` are incomparable.** Neither total element approximates
    the other:
if it did, its finite neighbourhood `{i}` would have to belong to the other's filter
    too, which
is neither `Delta` nor its own singleton `{1-i}`. Used to show `False` is the *only*
    common lower bound
of `elemZero` and `elemOne` (Exercise 2.16's uniqueness argument for the parity map).
    -/
theorem not_elemZero_le_elemOne : not  elemZero  <=  elemOne := by
  intro h
  rcases h zero (Or.inr rfl) with h' | h'
  * exact zero_ne_master h'
  * exact zero_ne_one h'

/-- **`False` is the unique common lower bound of `elemZero` and `elemOne`.** Any
    element approximated
by *both* total elements is exactly `False`: `element_classification` puts it at `bot`,
     `elemZero`, or
`elemOne`, and the latter two are excluded by incomparability. This is the order-
    theoretic
ingredient (continuity/flatness of `?`) behind the uniqueness half of Exercise 2.16. -/
theorem eq_bot_of_le_elemZero_of_le_elemOne {a : neighborhoodSystem.Element}
    (h0 : a  <=  elemZero) (h1 : a  <=  elemOne) : a = bot := by
  rcases element_classification a with rfl | rfl | rfl
  * rfl
  * exact absurd h1 not_elemZero_le_elemOne
  * exact absurd h0 not_elemOne_le_elemZero
end

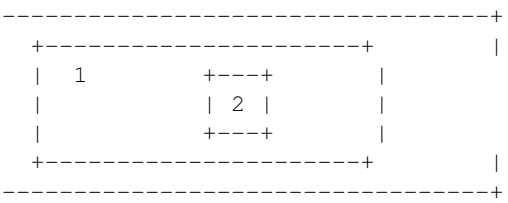
end neighborhoodSystem

end DomainTheory.NeighbourhoodSystems.Example12
end
```

3.2.5 Example 1.3 — three-step chain

EXAMPLE 1.3. Let $\Delta = \{0, 1, 2\}$ and let $\mathcal{D} = \{\{0, 1, 2\}, \{1, 2\}, \{2\}\}$.

In pictures we have:



Instead of stepping to the total element (here represented by 2) in one big step, the passage is divided into two steps. (Note 0 and 1 cannot be taken as representing total elements.) This example is not very

interesting because the direction of approximation is unique. We need an example with some choice. □

Example_1_3.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.DomainIso

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems.Example13
/-- Tokens for Example 1.3: `Delta = {0, 1, 2}`. -/
abbrev Token := Fin 3

/-- The master neighbourhood `Delta = {0, 1, 2}`. -/
def master : Set Token := Set.univ

theorem inter_master (A : Set Token) : A  I  master = A := by
  rw [master]; exact Set.inter_univ A

theorem master_inter (A : Set Token) : master  I  A = A := by
  rw [master]; exact Set.univ_inter A

/-- The neighbourhood `{1, 2}`. -/
def twelve : Set Token := (insert 2 {1} : Set Token)

theorem master_not_subset_twelve : not master  subseteq  twelve := by
  intro h
  have : (0 : Token)  in  twelve := h (by simp [master])
  simp [twelve] at this

/-- The neighbourhood `{2}`. -/
def two : Set Token := {2}

theorem master_not_subset_two : not master  subseteq  two := by
  intro h
  have : (0 : Token)  in  two := h (by simp [master])
  simp [two] at this

/-- The three neighbourhoods of Example 1.3. -/
def memSet : Set (Set Token) := {master, twelve, two}

/-- Membership in the neighbourhood system `?` of Example 1.3. -/
def mem (X : Set Token) : Prop := X  in  memSet

theorem mem_master : mem master := by simp [mem, memSet, master, twelve, two]

theorem mem_twelve : mem twelve := by simp [mem, memSet, master, twelve, two]

theorem mem_two : mem two := by simp [mem, memSet, master, twelve, two]

/-- A neighbourhood of Example 1.3 is exactly one of `Delta`, `{1,2}`, or `{2}`. -/
theorem mem_iff (X : Set Token) : mem X  <->  X = master  \/  X = twelve  \/  X = two
    := by
  constructor
```

```
  * intro h
    simp [mem, memSet, master, twelve, two] at h
    rcases h with rfl | rfl | rfl
    * exact Or.inl rfl
    * exact Or.inr (Or.inl rfl)
    * exact Or.inr (Or.inr rfl)
  * intro h
    rcases h with rfl | rfl | rfl
    * exact mem_master
    * exact mem_twelve
    * exact mem_two

theorem eq_of_master_subset {Y : Set Token} (h : mem Y) (hsub : master  subseteq  Y) :
     Y = master := by
  rcases (mem_iff Y).mp h with rfl | htwelve | htwo
  * rfl
  * exact absurd htwelve (fun h' => master_not_subset_twelve (h'  |>  hsub))
  * exact absurd htwo (fun h' => master_not_subset_two (h'  |>  hsub))

theorem twelve_inter_two : twelve  I  two = two := by
  ext t
  fin_cases t <;> simp [twelve, two]

theorem twelve_not_subset_two : not twelve  subseteq  two := by
  intro h
  have : (1 : Token)  in  two := h (by simp [twelve])
  simp [two] at this

theorem two_inter_twelve : two  I  twelve = two := by
  ext t
  fin_cases t <;> simp [twelve, two]

theorem inter_eq (X Y : Set Token) (h : mem X) (h' : mem Y) :
    X  I  Y = master  \/  X  I  Y = twelve  \/  X  I  Y = two := by
  rcases (mem_iff X).mp h with rfl | rfl | rfl <;>
    rcases (mem_iff Y).mp h' with rfl | rfl | rfl
  * exact Or.inl (master_inter _)
  * exact Or.inr (Or.inl (master_inter _))
  * exact Or.inr (Or.inr (master_inter _))
  * exact Or.inr (Or.inl (inter_master _))
  * exact Or.inr (Or.inl (Set.inter_self _))
  * exact Or.inr (Or.inr (twelve_inter_two))
  * exact Or.inr (Or.inr (inter_master _))
  * exact Or.inr (Or.inr (two_inter_twelve))
  * exact Or.inr (Or.inr (Set.inter_self _))

/- concordance: source L160 EXAMPLE 1.3 three-step chain -/
/-- **Example 1.3.** The neighbourhood system on `Delta = {0, 1, 2}`. -/
def neighborhoodSystem : NeighborhoodSystem Token where
  mem
  master
  master_nonempty := Set.univ_nonempty
  master_mem := mem_master
  sub_master := fun _ => Set.subset_univ _
  inter_mem := by
    intro X Y Z hX hY hZ hZsub
```

```
    rcases inter_eq X Y hX hY with h | h | h
    * rw [h]; exact mem_master
    * rw [h]; exact mem_twelve
    * rw [h]; exact mem_two

namespace neighborhoodSystem
section

open NeighborhoodSystem

/-- The bottom element `False = {Delta}`. -/
def bot : neighborhoodSystem.Element where
  mem X := X = master
  sub h := by rw [h]; exact mem_master
  master_mem := rfl
  inter_mem := by
    intro X Y hX hY
    rw [hX, hY, master_inter]
  up_mem := by
    intro X Y hX hY hXY
    rw [hX] at hXY
    exact eq_of_master_subset hY hXY

/-- The intermediate partial element `{Delta, {1,2}}`. -/
def elemTwelve : neighborhoodSystem.Element where
  mem X := X = master  \/  X = twelve
  sub h := by
    rcases h with rfl | rfl
    * exact mem_master
    * exact mem_twelve
  master_mem := Or.inl rfl
  inter_mem := by
    intro X Y hX hY
    rcases hX with rfl | rfl <;> rcases hY with rfl | rfl
    * exact Or.inl (master_inter _)
    * exact Or.inr (master_inter _)
    * exact Or.inr (inter_master _)
    * exact Or.inr (Set.inter_self _)
  up_mem := by
    intro X Y hX hY hXY
    rcases hX with rfl | rfl
    * exact Or.inl (eq_of_master_subset hY hXY)
    * rcases (mem_iff Y).mp hY with rfl | rfl | rfl
      * exact Or.inl rfl
      * exact Or.inr rfl
      * exact absurd hXY twelve_not_subset_two

/-- The total element `{Delta, {1,2}, {2}}`. -/
def elemTwo : neighborhoodSystem.Element where
  mem X := X = master  \/  X = twelve  \/  X = two
  sub h := by
    rcases h with rfl | rfl | rfl
    * exact mem_master
    * exact mem_twelve
    * exact mem_two
  master_mem := Or.inl rfl
```

```
  inter_mem := by
    intro X Y hX hY
    rcases hX with rfl | rfl | rfl <;> rcases hY with rfl | rfl | rfl
    * exact Or.inl (master_inter _)
    * exact Or.inr (Or.inl (master_inter _))
    * exact Or.inr (Or.inr (master_inter _))
    * exact Or.inr (Or.inl (inter_master _))
    * exact Or.inr (Or.inl (Set.inter_self _))
    * exact Or.inr (Or.inr (twelve_inter_two))
    * exact Or.inr (Or.inr (inter_master _))
    * exact Or.inr (Or.inr (two_inter_twelve))
    * exact Or.inr (Or.inr (Set.inter_self _))
  up_mem := by
    intro X Y hX hY hXY
    rcases hX with rfl | rfl | rfl
    * exact Or.inl (eq_of_master_subset hY hXY)
    * rcases (mem_iff Y).mp hY with rfl | rfl | rfl
      * exact Or.inl rfl
      * exact Or.inr (Or.inl rfl)
      * exact Or.inr (Or.inr rfl)
    * rcases (mem_iff Y).mp hY with rfl | rfl | rfl
      * exact Or.inl rfl
      * exact Or.inr (Or.inl rfl)
      * exact Or.inr (Or.inr rfl)
end

end neighborhoodSystem
theorem twelve_ne_master : twelve  !=  master := by
  intro h
  have hmem : (0 : Token)  in  twelve := by
    rw [h]
    simp [master]
  simp [twelve] at hmem

theorem two_ne_master : two  !=  master := by
  intro h
  have hmem : (0 : Token)  in  two := by
    rw [h]
    simp [master]
  simp [two] at hmem

theorem two_ne_twelve : two  !=  twelve := by
  intro h
  have hmem : (1 : Token)  in  two := by
    rw [h]
    simp [twelve]
  simp [two] at hmem

namespace neighborhoodSystem

open NeighborhoodSystem

theorem two_subset_twelve : two  subseteq  twelve := by
  intro t ht
  fin_cases t <;> simp [two, twelve] at ht  |-
```

```
theorem mem_two_of_mem (x : neighborhoodSystem.Element) (h : x.mem two) :
    x = elemTwo := by
  apply Element.ext
  intro X
  constructor
  * intro hx
    rcases (mem_iff X).mp (x.sub hx) with rfl | hX12 | htwo
    * exact Or.inl rfl
    * rw [hX12]; exact Or.inr (Or.inl rfl)
    * exact Or.inr (Or.inr htwo)
  * intro hx
    rcases hx with rfl | hx | hx
    * exact x.master_mem
    * rw [hx]; exact x.up_mem h mem_twelve two_subset_twelve
    * rw [hx]; exact h

theorem mem_twelve_of_mem (x : neighborhoodSystem.Element) (htwelve : x.mem twelve)
    (htwo : not x.mem two) : x = elemTwelve := by
  apply Element.ext
  intro X
  constructor
  * intro hx
    rcases (mem_iff X).mp (x.sub hx) with rfl | hX12 | hX2
    * exact Or.inl rfl
    * exact Or.inr hX12
    * have hxo : x.mem two := hX2  |>  hx
      have := x.inter_mem htwelve hxo
      rw [twelve_inter_two] at this
      exact absurd this htwo
  * intro hx
    rcases hx with rfl | hx
    * exact x.master_mem
    * rw [hx]; exact htwelve

section

open NeighborhoodSystem

theorem bot_lt_elemTwelve : bot < elemTwelve := by
  constructor
  * intro X hx
    rw [hx]
    exact Or.inl rfl
  * intro h
    have : bot.mem twelve := h twelve (Or.inr rfl)
    exact twelve_ne_master this

theorem elemTwelve_lt_elemTwo : elemTwelve < elemTwo := by
  constructor
  * intro X hx
    rcases hx with rfl | hX12
    * exact Or.inl rfl
    * exact Or.inr (Or.inl hX12)
  * intro h
    exfalso
    have ht := h two (Or.inr (Or.inr rfl))
```

```
    rcases ht with h1 | h2
    * exact absurd h1 two_ne_master
    * exact two_ne_twelve h2

/-- Every element of Example 1.3 is one of the three filters in the linear chain. -/
theorem element_classification (x : neighborhoodSystem.Element) :
    x = bot  \/  x = elemTwelve  \/  x = elemTwo := by
  by_cases htwo : x.mem two
  * exact Or.inr (Or.inr (mem_two_of_mem x htwo))
  by_cases htwelve : x.mem twelve
  * exact Or.inr (Or.inl (mem_twelve_of_mem x htwelve htwo))
  apply Or.inl
  apply Element.ext
  intro X
  constructor
  * intro hx
    rcases (mem_iff X).mp (x.sub hx) with rfl | htwelve' | htwo'
    * rfl
    * exact absurd (htwelve'  |>  hx) htwelve
    * exact absurd (htwo'  |>  hx) htwo
  * intro hx
    rw [hx]
    exact x.master_mem

theorem elemTwo_maximal (x : neighborhoodSystem.Element) (h : elemTwo  <=  x) : x =
    elemTwo := by
  rcases element_classification x with rfl | rfl | rfl
  * exfalso
    have : bot.mem twelve := h twelve (Or.inr (Or.inl rfl))
    exact twelve_ne_master this
  * exfalso
    have ht := h two (Or.inr (Or.inr rfl))
    rcases ht with h1 | h2
    * exact absurd h1 two_ne_master
    * exact two_ne_twelve h2
  * rfl
end

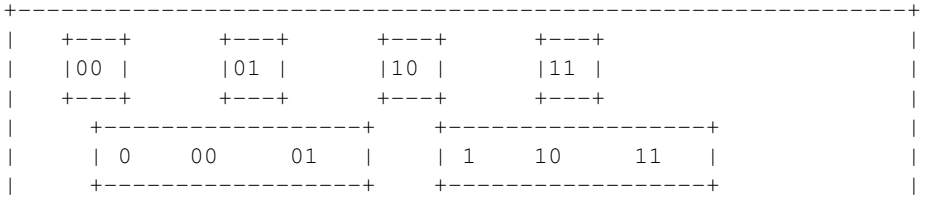
end neighborhoodSystem

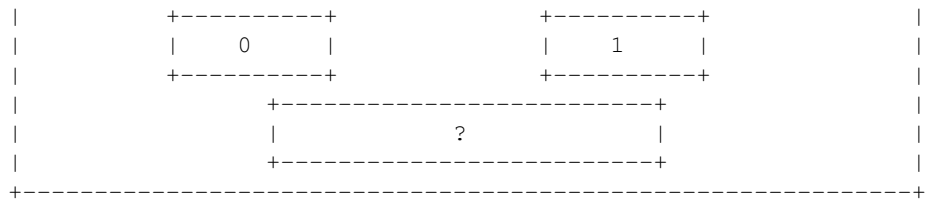
end DomainTheory.NeighbourhoodSystems.Example13
end
```

3.2.6 Example 1.4 — finite binary tree

EXAMPLE 1.4. Let $\Delta = \{\Lambda, 0, 1, 00, 01, 10, 11\}$ and let $\mathcal{D} = \{\Delta, \{0, 00, 01\}, \{1, 10, 11\}, \{00\}, \{01\}, \{10\}, \{11\}\}$.

Or more understandably in pictures:





The tokens are finite sequences of 0's and 1's (up to length 2) with Λ the empty sequence; they form — in the picture — the binary tree with the sequences as the nodes. The neighbourhoods are the subtrees of all nodes above a given node. Obviously this can be generalized to sequences of any length (and to trees less regular than the binary tree). The total elements of the example correspond to the top nodes 00, 01, 10, 11 and the lower nodes to the partial elements. When we are not at a top node we have only partially determined a sequence, and the branching indicates that we have some choice as to how the sequence can be extended. □

It should be noted that, in these three examples, the reason that we have a neighbourhood system is a simple consequence of a

Resolutions from the image: "is unique" (not "in unique"); $\mathcal{D}$ begins with Δ, not Λ; Pass 1's nested-box diagram matches the neighborhood boundaries in the scan; em dashes around "in the picture" as in the source.

Example_1_4.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.DomainIso

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems.Example14
/-- Tokens for Example 1.4: the binary tree `Delta = {?,0,1,00,01,10,11}`, encoded as `
    Fin 7`
(`?=0, 0=1, 1=2, 00=3, 01=4, 10=5, 11=6`). -/
abbrev Token := Fin 7

/-- The leaf `{00} = {3}`. -/
def leaf00 : Set Token := {3}

/-- The leaf `{01} = {4}`. -/
def leaf01 : Set Token := {4}

theorem leaf00_inter_leaf01 : leaf00  I  leaf01 = ({} : Set Token) := by
  ext t; fin_cases t <;> simp [leaf00, leaf01]

theorem leaf01_inter_leaf00 : leaf01  I  leaf00 = ({} : Set Token) := by
  rw [Set.inter_comm]; exact leaf00_inter_leaf01

/-- The leaf `{10} = {5}`. -/
def leaf10 : Set Token := {5}

theorem leaf00_inter_leaf10 : leaf00  I  leaf10 = ({} : Set Token) := by
  ext t; fin_cases t <;> simp [leaf00, leaf10]
```

```
theorem leaf01_inter_leaf10 : leaf01  I  leaf10 = ({} : Set Token) := by
  ext t; fin_cases t <;> simp [leaf01, leaf10]

theorem leaf10_inter_leaf00 : leaf10  I  leaf00 = ({} : Set Token) := by
  rw [Set.inter_comm]; exact leaf00_inter_leaf10

theorem leaf10_inter_leaf01 : leaf10  I  leaf01 = ({} : Set Token) := by
  rw [Set.inter_comm]; exact leaf01_inter_leaf10

/-- The leaf `{11} = {6}`. -/
def leaf11 : Set Token := {6}

theorem leaf00_inter_leaf11 : leaf00  I  leaf11 = ({} : Set Token) := by
  ext t; fin_cases t <;> simp [leaf00, leaf11]

theorem leaf01_inter_leaf11 : leaf01  I  leaf11 = ({} : Set Token) := by
  ext t; fin_cases t <;> simp [leaf01, leaf11]

theorem leaf10_inter_leaf11 : leaf10  I  leaf11 = ({} : Set Token) := by
  ext t; fin_cases t <;> simp [leaf10, leaf11]

theorem leaf11_inter_leaf00 : leaf11  I  leaf00 = ({} : Set Token) := by
  rw [Set.inter_comm]; exact leaf00_inter_leaf11

theorem leaf11_inter_leaf01 : leaf11  I  leaf01 = ({} : Set Token) := by
  rw [Set.inter_comm]; exact leaf01_inter_leaf11

theorem leaf11_inter_leaf10 : leaf11  I  leaf10 = ({} : Set Token) := by
  rw [Set.inter_comm]; exact leaf10_inter_leaf11

/-- The left subtree `{0, 00, 01} = {1, 3, 4}`. -/
def left : Set Token := {1, 3, 4}

theorem left_inter_leaf00 : left  I  leaf00 = leaf00 := by
  ext t; fin_cases t <;> simp [left, leaf00]

theorem leaf00_inter_left : leaf00  I  left = leaf00 := by
  rw [Set.inter_comm]; exact left_inter_leaf00

theorem left_inter_leaf01 : left  I  leaf01 = leaf01 := by
  ext t; fin_cases t <;> simp [left, leaf01]

theorem leaf01_inter_left : leaf01  I  left = leaf01 := by
  rw [Set.inter_comm]; exact left_inter_leaf01

theorem left_inter_leaf10 : left  I  leaf10 = ({} : Set Token) := by
  ext t; fin_cases t <;> simp [left, leaf10]

theorem leaf10_inter_left : leaf10  I  left = ({} : Set Token) := by
  rw [Set.inter_comm]; exact left_inter_leaf10

theorem left_inter_leaf11 : left  I  leaf11 = ({} : Set Token) := by
  ext t; fin_cases t <;> simp [left, leaf11]

theorem leaf11_inter_left : leaf11  I  left = ({} : Set Token) := by
  rw [Set.inter_comm]; exact left_inter_leaf11
```

```
/-- The master neighbourhood `Delta` (the whole tree). -/
def master : Set Token := Set.univ

theorem inter_master (A : Set Token) : A  I  master = A := by
  rw [master]; exact Set.inter_univ A

theorem master_inter (A : Set Token) : master  I  A = A := by
  rw [master]; exact Set.univ_inter A

/-- The right subtree `{1, 10, 11} = {2, 5, 6}`. -/
def right : Set Token := {2, 5, 6}

theorem left_inter_right : left  I  right = ({} : Set Token) := by
  ext t; fin_cases t <;> simp [left, right]

/-- The seven neighbourhoods of Example 1.4. -/
def memSet : Set (Set Token) := {master, left, right, leaf00, leaf01, leaf10, leaf11}

/-- Membership in the neighbourhood system `?` of Example 1.4. -/
def mem (X : Set Token) : Prop := X  in  memSet

theorem mem_leaf00 : mem leaf00 := by
  simp [mem, memSet, master, left, right, leaf00, leaf01, leaf10, leaf11]

theorem mem_leaf01 : mem leaf01 := by
  simp [mem, memSet, master, left, right, leaf00, leaf01, leaf10, leaf11]

theorem mem_leaf10 : mem leaf10 := by
  simp [mem, memSet, master, left, right, leaf00, leaf01, leaf10, leaf11]

theorem mem_leaf11 : mem leaf11 := by
  simp [mem, memSet, master, left, right, leaf00, leaf01, leaf10, leaf11]

theorem mem_left : mem left := by
  simp [mem, memSet, master, left, right, leaf00, leaf01, leaf10, leaf11]

theorem mem_master : mem master := by
  simp [mem, memSet, master, left, right, leaf00, leaf01, leaf10, leaf11]

theorem mem_right : mem right := by
  simp [mem, memSet, master, left, right, leaf00, leaf01, leaf10, leaf11]

/-- A neighbourhood of Example 1.4 is exactly one of the seven. -/
theorem mem_iff (X : Set Token) :
    mem X  <->  X = master  \/  X = left  \/  X = right  \/  X = leaf00  \/  X =
    leaf01  \/  X = leaf10  \/  X = leaf11 := by
  constructor
  * intro h
    simpa [mem, memSet] using h
  * intro h
    rcases h with rfl | rfl | rfl | rfl | rfl | rfl | rfl
    * exact mem_master
    * exact mem_left
    * exact mem_right
    * exact mem_leaf00
```

```
    * exact mem_leaf01
    * exact mem_leaf10
    * exact mem_leaf11

theorem eq_of_master_subset {Y : Set Token} (hY : mem Y) (h : master  subseteq  Y) : Y
     = master := by
  rcases (mem_iff Y).mp hY with rfl | rfl | rfl | rfl | rfl | rfl | rfl
  * rfl
  * exact absurd (h (show (0 : Token)  in  master by simp [master])) (by simp [left])
  * exact absurd (h (show (0 : Token)  in  master by simp [master])) (by simp [right])
  * exact absurd (h (show (0 : Token)  in  master by simp [master])) (by simp [leaf00
    ])
  * exact absurd (h (show (0 : Token)  in  master by simp [master])) (by simp [leaf01
    ])
  * exact absurd (h (show (0 : Token)  in  master by simp [master])) (by simp [leaf10
    ])
  * exact absurd (h (show (0 : Token)  in  master by simp [master])) (by simp [leaf11
    ])

theorem leaf00_subset_cases {Y : Set Token} (hY : mem Y) (h : leaf00  subseteq  Y) :
    Y = master  \/  Y = left  \/  Y = leaf00 := by
  rcases (mem_iff Y).mp hY with rfl | rfl | rfl | rfl | rfl | rfl | rfl
  * exact Or.inl rfl
  * exact Or.inr (Or.inl rfl)
  * exact absurd (h (show (3 : Token)  in  leaf00 by simp [leaf00])) (by simp [right])
  * exact Or.inr (Or.inr rfl)
  * exact absurd (h (show (3 : Token)  in  leaf00 by simp [leaf00])) (by simp [leaf01
    ])
  * exact absurd (h (show (3 : Token)  in  leaf00 by simp [leaf00])) (by simp [leaf10
    ])
  * exact absurd (h (show (3 : Token)  in  leaf00 by simp [leaf00])) (by simp [leaf11
    ])

theorem leaf01_subset_cases {Y : Set Token} (hY : mem Y) (h : leaf01  subseteq  Y) :
    Y = master  \/  Y = left  \/  Y = leaf01 := by
  rcases (mem_iff Y).mp hY with rfl | rfl | rfl | rfl | rfl | rfl | rfl
  * exact Or.inl rfl
  * exact Or.inr (Or.inl rfl)
  * exact absurd (h (show (4 : Token)  in  leaf01 by simp [leaf01])) (by simp [right])
  * exact absurd (h (show (4 : Token)  in  leaf01 by simp [leaf01])) (by simp [leaf00
    ])
  * exact Or.inr (Or.inr rfl)
  * exact absurd (h (show (4 : Token)  in  leaf01 by simp [leaf01])) (by simp [leaf10
    ])
  * exact absurd (h (show (4 : Token)  in  leaf01 by simp [leaf01])) (by simp [leaf11
    ])

theorem leaf10_subset_cases {Y : Set Token} (hY : mem Y) (h : leaf10  subseteq  Y) :
    Y = master  \/  Y = right  \/  Y = leaf10 := by
  rcases (mem_iff Y).mp hY with rfl | rfl | rfl | rfl | rfl | rfl | rfl
  * exact Or.inl rfl
  * exact absurd (h (show (5 : Token)  in  leaf10 by simp [leaf10])) (by simp [left])
  * exact Or.inr (Or.inl rfl)
  * exact absurd (h (show (5 : Token)  in  leaf10 by simp [leaf10])) (by simp [leaf00
    ])
  * exact absurd (h (show (5 : Token)  in  leaf10 by simp [leaf10])) (by simp [leaf01
```

```
    ])
  * exact Or.inr (Or.inr rfl)
  * exact absurd (h (show (5 : Token)  in  leaf10 by simp [leaf10])) (by simp [leaf11
    ])

theorem leaf11_subset_cases {Y : Set Token} (hY : mem Y) (h : leaf11  subseteq  Y) :
    Y = master  \/  Y = right  \/  Y = leaf11 := by
  rcases (mem_iff Y).mp hY with rfl | rfl | rfl | rfl | rfl | rfl | rfl
  * exact Or.inl rfl
  * exact absurd (h (show (6 : Token)  in  leaf11 by simp [leaf11])) (by simp [left])
  * exact Or.inr (Or.inl rfl)
  * exact absurd (h (show (6 : Token)  in  leaf11 by simp [leaf11])) (by simp [leaf00
    ])
  * exact absurd (h (show (6 : Token)  in  leaf11 by simp [leaf11])) (by simp [leaf01
    ])
  * exact absurd (h (show (6 : Token)  in  leaf11 by simp [leaf11])) (by simp [leaf10
    ])
  * exact Or.inr (Or.inr rfl)

theorem left_subset_cases {Y : Set Token} (hY : mem Y) (h : left  subseteq  Y) :
    Y = master  \/  Y = left := by
  rcases (mem_iff Y).mp hY with rfl | rfl | rfl | rfl | rfl | rfl | rfl
  * exact Or.inl rfl
  * exact Or.inr rfl
  * exact absurd (h (show (1 : Token)  in  left by simp [left])) (by simp [right])
  * exact absurd (h (show (1 : Token)  in  left by simp [left])) (by simp [leaf00])
  * exact absurd (h (show (1 : Token)  in  left by simp [left])) (by simp [leaf01])
  * exact absurd (h (show (1 : Token)  in  left by simp [left])) (by simp [leaf10])
  * exact absurd (h (show (1 : Token)  in  left by simp [left])) (by simp [leaf11])

theorem not_mem_empty : not mem ({} : Set Token) := by
  intro h
  rcases (mem_iff ({} : Set Token)).mp h with h | h | h | h | h | h | h
  * have : (0 : Token)  in  ({} : Set Token) := by rw [h]; simp [master]
    simp at this
  * have : (1 : Token)  in  ({} : Set Token) := by rw [h]; simp [left]
    simp at this
  * have : (2 : Token)  in  ({} : Set Token) := by rw [h]; simp [right]
    simp at this
  * have : (3 : Token)  in  ({} : Set Token) := by rw [h]; simp [leaf00]
    simp at this
  * have : (4 : Token)  in  ({} : Set Token) := by rw [h]; simp [leaf01]
    simp at this
  * have : (5 : Token)  in  ({} : Set Token) := by rw [h]; simp [leaf10]
    simp at this
  * have : (6 : Token)  in  ({} : Set Token) := by rw [h]; simp [leaf11]
    simp at this

theorem right_inter_leaf00 : right  I  leaf00 = ({} : Set Token) := by
  ext t; fin_cases t <;> simp [right, leaf00]

theorem leaf00_inter_right : leaf00  I  right = ({} : Set Token) := by
  rw [Set.inter_comm]; exact right_inter_leaf00

theorem right_inter_leaf01 : right  I  leaf01 = ({} : Set Token) := by
  ext t; fin_cases t <;> simp [right, leaf01]
```

```
theorem leaf01_inter_right : leaf01  I  right = ({} : Set Token) := by
  rw [Set.inter_comm]; exact right_inter_leaf01

theorem right_inter_leaf10 : right  I  leaf10 = leaf10 := by
  ext t; fin_cases t <;> simp [right, leaf10]

theorem leaf10_inter_right : leaf10  I  right = leaf10 := by
  rw [Set.inter_comm]; exact right_inter_leaf10

theorem right_inter_leaf11 : right  I  leaf11 = leaf11 := by
  ext t; fin_cases t <;> simp [right, leaf11]

theorem leaf11_inter_right : leaf11  I  right = leaf11 := by
  rw [Set.inter_comm]; exact right_inter_leaf11

theorem right_inter_left : right  I  left = ({} : Set Token) := by
  rw [Set.inter_comm]; exact left_inter_right

/-- Every binary intersection of two neighbourhoods is again a neighbourhood, or is
    empty
(the tree's "nested-or-disjoint" property). -/
theorem inter_eq (X Y : Set Token) (hX : mem X) (hY : mem Y) :
    X  I  Y = master  \/  X  I  Y = left  \/  X  I  Y = right  \/  X  I  Y = leaf00
    \/
      X  I  Y = leaf01  \/  X  I  Y = leaf10  \/  X  I  Y = leaf11  \/  X  I  Y = ({}
    : Set Token) := by
  rcases (mem_iff X).mp hX with rfl | rfl | rfl | rfl | rfl | rfl | rfl <;>
    rcases (mem_iff Y).mp hY with rfl | rfl | rfl | rfl | rfl | rfl | rfl <;>
    simp only [master_inter, inter_master, Set.inter_self,
      left_inter_right, right_inter_left, left_inter_leaf10, leaf10_inter_left,
      left_inter_leaf11, leaf11_inter_left, right_inter_leaf00, leaf00_inter_right,
      right_inter_leaf01, leaf01_inter_right, leaf00_inter_leaf01, leaf01_inter_leaf00
    ,
      leaf00_inter_leaf10, leaf10_inter_leaf00, leaf00_inter_leaf11,
    leaf11_inter_leaf00,
      leaf01_inter_leaf10, leaf10_inter_leaf01, leaf01_inter_leaf11,
    leaf11_inter_leaf01,
      leaf10_inter_leaf11, leaf11_inter_leaf10,
      left_inter_leaf00, leaf00_inter_left, left_inter_leaf01, leaf01_inter_left,
      right_inter_leaf10, leaf10_inter_right, right_inter_leaf11, leaf11_inter_right,
      true_or, or_true]

/- concordance: source L177 EXAMPLE 1.4 finite binary tree -/
/-- **Example 1.4.** The binary-tree neighbourhood system on `Delta =
    {?,0,1,00,01,10,11}`. -/
def neighborhoodSystem : NeighborhoodSystem Token where
  mem
  master
  master_nonempty := Set.univ_nonempty
  master_mem := mem_master
  sub_master := fun _ => Set.subset_univ _
  inter_mem := by
    intro X Y Z hX hY hZ hZsub
    rcases inter_eq X Y hX hY with h | h | h | h | h | h | h | h
    * rw [h]; exact mem_master
```

```
    * rw [h]; exact mem_left
    * rw [h]; exact mem_right
    * rw [h]; exact mem_leaf00
    * rw [h]; exact mem_leaf01
    * rw [h]; exact mem_leaf10
    * rw [h]; exact mem_leaf11
    * rw [h] at hZsub
      have hz : Z = ({} : Set Token) := Set.subset_empty_iff.mp hZsub
      subst hz
      exact absurd hZ not_mem_empty

namespace neighborhoodSystem
section

open NeighborhoodSystem

/-- The bottom element `False = {Delta}` (node `?`). -/
def bot : neighborhoodSystem.Element where
  mem X := X = master
  sub h := by rw [h]; exact mem_master
  master_mem := rfl
  inter_mem := by
    intro X Y hX hY
    rw [hX, hY, master_inter]
  up_mem := by
    intro X Y hX hY hXY
    rw [hX] at hXY
    exact eq_of_master_subset hY hXY

/-- The total element `{Delta, left, leaf00}` (leaf `00`). -/
def elem00 : neighborhoodSystem.Element where
  mem X := X = master  \/  X = left  \/  X = leaf00
  sub h := by
    rcases h with rfl | rfl | rfl
    * exact mem_master
    * exact mem_left
    * exact mem_leaf00
  master_mem := Or.inl rfl
  inter_mem := by
    intro X Y hX hY
    rcases hX with rfl | rfl | rfl <;> rcases hY with rfl | rfl | rfl
    * exact Or.inl (master_inter _)
    * exact Or.inr (Or.inl (master_inter _))
    * exact Or.inr (Or.inr (master_inter _))
    * exact Or.inr (Or.inl (inter_master _))
    * exact Or.inr (Or.inl (Set.inter_self _))
    * exact Or.inr (Or.inr left_inter_leaf00)
    * exact Or.inr (Or.inr (inter_master _))
    * exact Or.inr (Or.inr leaf00_inter_left)
    * exact Or.inr (Or.inr (Set.inter_self _))
  up_mem := by
    intro X Y hX hY hXY
    rcases hX with rfl | rfl | rfl
    * exact Or.inl (eq_of_master_subset hY hXY)
    * rcases left_subset_cases hY hXY with h | h
      * exact Or.inl h
```

```
      * exact Or.inr (Or.inl h)
    * rcases leaf00_subset_cases hY hXY with h | h | h
      * exact Or.inl h
      * exact Or.inr (Or.inl h)
      * exact Or.inr (Or.inr h)

/-- The total element `{Delta, left, leaf01}` (leaf `01`). -/
def elem01 : neighborhoodSystem.Element where
  mem X := X = master  \/  X = left  \/  X = leaf01
  sub h := by
    rcases h with rfl | rfl | rfl
    * exact mem_master
    * exact mem_left
    * exact mem_leaf01
  master_mem := Or.inl rfl
  inter_mem := by
    intro X Y hX hY
    rcases hX with rfl | rfl | rfl <;> rcases hY with rfl | rfl | rfl
    * exact Or.inl (master_inter _)
    * exact Or.inr (Or.inl (master_inter _))
    * exact Or.inr (Or.inr (master_inter _))
    * exact Or.inr (Or.inl (inter_master _))
    * exact Or.inr (Or.inl (Set.inter_self _))
    * exact Or.inr (Or.inr left_inter_leaf01)
    * exact Or.inr (Or.inr (inter_master _))
    * exact Or.inr (Or.inr leaf01_inter_left)
    * exact Or.inr (Or.inr (Set.inter_self _))
  up_mem := by
    intro X Y hX hY hXY
    rcases hX with rfl | rfl | rfl
    * exact Or.inl (eq_of_master_subset hY hXY)
    * rcases left_subset_cases hY hXY with h | h
      * exact Or.inl h
      * exact Or.inr (Or.inl h)
    * rcases leaf01_subset_cases hY hXY with h | h | h
      * exact Or.inl h
      * exact Or.inr (Or.inl h)
      * exact Or.inr (Or.inr h)

/-- The partial element `{Delta, left}` (node `0`): the left branch is decided. -/
def elemZero : neighborhoodSystem.Element where
  mem X := X = master  \/  X = left
  sub h := by
    rcases h with rfl | rfl
    * exact mem_master
    * exact mem_left
  master_mem := Or.inl rfl
  inter_mem := by
    intro X Y hX hY
    rcases hX with rfl | rfl <;> rcases hY with rfl | rfl
    * exact Or.inl (master_inter _)
    * exact Or.inr (master_inter _)
    * exact Or.inr (inter_master _)
    * exact Or.inr (Set.inter_self _)
  up_mem := by
    intro X Y hX hY hXY
```

```
    rcases hX with rfl | rfl
    * exact Or.inl (eq_of_master_subset hY hXY)
    * rcases left_subset_cases hY hXY with h | h

      * exact Or.inl h
      * exact Or.inr h
end

end neighborhoodSystem
theorem right_subset_cases {Y : Set Token} (hY : mem Y) (h : right  subseteq  Y) :
    Y = master  \/  Y = right := by
  rcases (mem_iff Y).mp hY with rfl | rfl | rfl | rfl | rfl | rfl | rfl
  * exact Or.inl rfl
  * exact absurd (h (show (2 : Token)  in  right by simp [right])) (by simp [left])
  * exact Or.inr rfl
  * exact absurd (h (show (2 : Token)  in  right by simp [right])) (by simp [leaf00])
  * exact absurd (h (show (2 : Token)  in  right by simp [right])) (by simp [leaf01])
  * exact absurd (h (show (2 : Token)  in  right by simp [right])) (by simp [leaf10])
  * exact absurd (h (show (2 : Token)  in  right by simp [right])) (by simp [leaf11])

namespace neighborhoodSystem
section

open NeighborhoodSystem

/-- The total element `{Delta, right, leaf10}` (leaf `10`). -/
def elem10 : neighborhoodSystem.Element where
  mem X := X = master  \/  X = right  \/  X = leaf10
  sub h := by
    rcases h with rfl | rfl | rfl
    * exact mem_master
    * exact mem_right
    * exact mem_leaf10
  master_mem := Or.inl rfl
  inter_mem := by
    intro X Y hX hY
    rcases hX with rfl | rfl | rfl <;> rcases hY with rfl | rfl | rfl
    * exact Or.inl (master_inter _)
    * exact Or.inr (Or.inl (master_inter _))
    * exact Or.inr (Or.inr (master_inter _))
    * exact Or.inr (Or.inl (inter_master _))
    * exact Or.inr (Or.inl (Set.inter_self _))
    * exact Or.inr (Or.inr right_inter_leaf10)
    * exact Or.inr (Or.inr (inter_master _))
    * exact Or.inr (Or.inr leaf10_inter_right)
    * exact Or.inr (Or.inr (Set.inter_self _))
  up_mem := by
    intro X Y hX hY hXY
    rcases hX with rfl | rfl | rfl
    * exact Or.inl (eq_of_master_subset hY hXY)
    * rcases right_subset_cases hY hXY with h | h
      * exact Or.inl h
      * exact Or.inr (Or.inl h)
    * rcases leaf10_subset_cases hY hXY with h | h | h
      * exact Or.inl h
      * exact Or.inr (Or.inl h)
```

```
      * exact Or.inr (Or.inr h)

/-- The total element `{Delta, right, leaf11}` (leaf `11`). -/
def elem11 : neighborhoodSystem.Element where
  mem X := X = master  \/  X = right  \/  X = leaf11
  sub h := by
    rcases h with rfl | rfl | rfl
    * exact mem_master
    * exact mem_right
    * exact mem_leaf11
  master_mem := Or.inl rfl
  inter_mem := by
    intro X Y hX hY
    rcases hX with rfl | rfl | rfl <;> rcases hY with rfl | rfl | rfl
    * exact Or.inl (master_inter _)
    * exact Or.inr (Or.inl (master_inter _))
    * exact Or.inr (Or.inr (master_inter _))
    * exact Or.inr (Or.inl (inter_master _))
    * exact Or.inr (Or.inl (Set.inter_self _))
    * exact Or.inr (Or.inr right_inter_leaf11)
    * exact Or.inr (Or.inr (inter_master _))
    * exact Or.inr (Or.inr leaf11_inter_right)
    * exact Or.inr (Or.inr (Set.inter_self _))
  up_mem := by
    intro X Y hX hY hXY
    rcases hX with rfl | rfl | rfl
    * exact Or.inl (eq_of_master_subset hY hXY)
    * rcases right_subset_cases hY hXY with h | h
      * exact Or.inl h
      * exact Or.inr (Or.inl h)
    * rcases leaf11_subset_cases hY hXY with h | h | h
      * exact Or.inl h
      * exact Or.inr (Or.inl h)
      * exact Or.inr (Or.inr h)

/-- The partial element `{Delta, right}` (node `1`): the right branch is decided. -/
def elemOne : neighborhoodSystem.Element where
  mem X := X = master  \/  X = right
  sub h := by
    rcases h with rfl | rfl
    * exact mem_master
    * exact mem_right
  master_mem := Or.inl rfl
  inter_mem := by
    intro X Y hX hY
    rcases hX with rfl | rfl <;> rcases hY with rfl | rfl
    * exact Or.inl (master_inter _)
    * exact Or.inr (master_inter _)
    * exact Or.inr (inter_master _)
    * exact Or.inr (Set.inter_self _)
  up_mem := by
    intro X Y hX hY hXY
    rcases hX with rfl | rfl
    * exact Or.inl (eq_of_master_subset hY hXY)
    * rcases right_subset_cases hY hXY with h | h
      * exact Or.inl h
```

```
      * exact Or.inr h
end

end neighborhoodSystem
theorem left_ne_master : left  !=  master := by
  intro h
  have hmem : (0 : Token)  in  left := by rw [h]; simp [master]
  simp [left] at hmem

theorem right_ne_master : right  !=  master := by
  intro h
  have hmem : (0 : Token)  in  right := by rw [h]; simp [master]
  simp [right] at hmem

theorem leaf00_ne_master : leaf00  !=  master := by
  intro h
  have hmem : (0 : Token)  in  leaf00 := by rw [h]; simp [master]
  simp [leaf00] at hmem

theorem leaf01_ne_master : leaf01  !=  master := by
  intro h
  have hmem : (0 : Token)  in  leaf01 := by rw [h]; simp [master]
  simp [leaf01] at hmem

theorem leaf10_ne_master : leaf10  !=  master := by
  intro h
  have hmem : (0 : Token)  in  leaf10 := by rw [h]; simp [master]
  simp [leaf10] at hmem

theorem leaf11_ne_master : leaf11  !=  master := by
  intro h
  have hmem : (0 : Token)  in  leaf11 := by rw [h]; simp [master]
  simp [leaf11] at hmem

theorem leaf00_ne_left : leaf00  !=  left := by
  intro h
  have hmem : (1 : Token)  in  leaf00 := by rw [h]; simp [left]
  simp [leaf00] at hmem

theorem leaf01_ne_left : leaf01  !=  left := by
  intro h
  have hmem : (1 : Token)  in  leaf01 := by rw [h]; simp [left]
  simp [leaf01] at hmem

theorem leaf10_ne_right : leaf10  !=  right := by
  intro h
  have hmem : (2 : Token)  in  leaf10 := by rw [h]; simp [right]
  simp [leaf10] at hmem

theorem leaf11_ne_right : leaf11  !=  right := by
  intro h
  have hmem : (2 : Token)  in  leaf11 := by rw [h]; simp [right]
  simp [leaf11] at hmem

theorem leaf00_subset_left : leaf00  subseteq  left := by
  intro t ht; fin_cases t <;> simp [leaf00, left] at ht  |-
```

```
private theorem leaf01_subset_left : leaf01  subseteq  left := by
  intro t ht; fin_cases t <;> simp [leaf01, left] at ht  |-

private theorem leaf10_subset_right : leaf10  subseteq  right := by
  intro t ht; fin_cases t <;> simp [leaf10, right] at ht  |-

private theorem leaf11_subset_right : leaf11  subseteq  right := by
  intro t ht; fin_cases t <;> simp [leaf11, right] at ht  |-

namespace neighborhoodSystem

open NeighborhoodSystem

theorem mem_leaf00_imp (x : neighborhoodSystem.Element) (h : x.mem leaf00) :
    x = elem00 := by
  have hleft : x.mem left := x.up_mem h mem_left leaf00_subset_left
  apply Element.ext
  intro X
  constructor
  * intro hx
    rcases (mem_iff X).mp (x.sub hx) with rfl | rfl | rfl | rfl | rfl | rfl | rfl
    * exact Or.inl rfl
    * exact Or.inr (Or.inl rfl)
    * exfalso
      have hi := x.inter_mem hx h
      rw [show (right  I  leaf00 : Set Token) = {} from by ext t; fin_cases t <;> simp
     [right, leaf00]] at hi
      exact not_mem_empty (x.sub hi)
    * exact Or.inr (Or.inr rfl)
    * exfalso
      have hi := x.inter_mem hx h
      rw [show (leaf01  I  leaf00 : Set Token) = {} from by ext t; fin_cases t <;>
    simp [leaf01, leaf00]] at hi
      exact not_mem_empty (x.sub hi)
    * exfalso
      have hi := x.inter_mem hx h
      rw [show (leaf10  I  leaf00 : Set Token) = {} from by ext t; fin_cases t <;>
    simp [leaf10, leaf00]] at hi
      exact not_mem_empty (x.sub hi)
    * exfalso
      have hi := x.inter_mem hx h
      rw [show (leaf11  I  leaf00 : Set Token) = {} from by ext t; fin_cases t <;>
    simp [leaf11, leaf00]] at hi
      exact not_mem_empty (x.sub hi)
  * intro hx
    rcases hx with rfl | rfl | rfl
    * exact x.master_mem
    * exact hleft
    * exact h

theorem mem_leaf01_imp (x : neighborhoodSystem.Element) (h : x.mem leaf01) :
    x = elem01 := by
  have hleft : x.mem left := x.up_mem h mem_left leaf01_subset_left
  apply Element.ext
  intro X
```

```
  constructor
  * intro hx
    rcases (mem_iff X).mp (x.sub hx) with rfl | rfl | rfl | rfl | rfl | rfl | rfl
    * exact Or.inl rfl
    * exact Or.inr (Or.inl rfl)
    * exfalso
      have hi := x.inter_mem hx h
      rw [show (right  I  leaf01 : Set Token) = {} from by ext t; fin_cases t <;> simp
     [right, leaf01]] at hi
      exact not_mem_empty (x.sub hi)
    * exfalso
      have hi := x.inter_mem hx h
      rw [show (leaf00  I  leaf01 : Set Token) = {} from by ext t; fin_cases t <;>
    simp [leaf00, leaf01]] at hi
      exact not_mem_empty (x.sub hi)
    * exact Or.inr (Or.inr rfl)
    * exfalso
      have hi := x.inter_mem hx h
      rw [show (leaf10  I  leaf01 : Set Token) = {} from by ext t; fin_cases t <;>
    simp [leaf10, leaf01]] at hi
      exact not_mem_empty (x.sub hi)
    * exfalso
      have hi := x.inter_mem hx h
      rw [show (leaf11  I  leaf01 : Set Token) = {} from by ext t; fin_cases t <;>
    simp [leaf11, leaf01]] at hi
      exact not_mem_empty (x.sub hi)
  * intro hx
    rcases hx with rfl | rfl | rfl
    * exact x.master_mem
    * exact hleft
    * exact h

theorem mem_leaf10_imp (x : neighborhoodSystem.Element) (h : x.mem leaf10) :
    x = elem10 := by
  have hright : x.mem right := x.up_mem h mem_right leaf10_subset_right
  apply Element.ext
  intro X
  constructor
  * intro hx
    rcases (mem_iff X).mp (x.sub hx) with rfl | rfl | rfl | rfl | rfl | rfl | rfl
    * exact Or.inl rfl
    * exfalso
      have hi := x.inter_mem hx h
      rw [show (left  I  leaf10 : Set Token) = {} from by ext t; fin_cases t <;> simp
    [left, leaf10]] at hi
      exact not_mem_empty (x.sub hi)
    * exact Or.inr (Or.inl rfl)
    * exfalso
      have hi := x.inter_mem hx h
      rw [show (leaf00  I  leaf10 : Set Token) = {} from by ext t; fin_cases t <;>
    simp [leaf00, leaf10]] at hi
      exact not_mem_empty (x.sub hi)
    * exfalso
      have hi := x.inter_mem hx h
      rw [show (leaf01  I  leaf10 : Set Token) = {} from by ext t; fin_cases t <;>
    simp [leaf01, leaf10]] at hi
```

```
      exact not_mem_empty (x.sub hi)
    * exact Or.inr (Or.inr rfl)
    * exfalso
      have hi := x.inter_mem hx h
      rw [show (leaf11  I  leaf10 : Set Token) = {} from by ext t; fin_cases t <;>
    simp [leaf11, leaf10]] at hi
      exact not_mem_empty (x.sub hi)
  * intro hx
    rcases hx with rfl | rfl | rfl
    * exact x.master_mem
    * exact hright
    * exact h

theorem mem_leaf11_imp (x : neighborhoodSystem.Element) (h : x.mem leaf11) :
    x = elem11 := by
  have hright : x.mem right := x.up_mem h mem_right leaf11_subset_right
  apply Element.ext
  intro X
  constructor
  * intro hx
    rcases (mem_iff X).mp (x.sub hx) with rfl | rfl | rfl | rfl | rfl | rfl | rfl
    * exact Or.inl rfl
    * exfalso
      have hi := x.inter_mem hx h
      rw [show (left  I  leaf11 : Set Token) = {} from by ext t; fin_cases t <;> simp
    [left, leaf11]] at hi
      exact not_mem_empty (x.sub hi)
    * exact Or.inr (Or.inl rfl)
    * exfalso
      have hi := x.inter_mem hx h
      rw [show (leaf00  I  leaf11 : Set Token) = {} from by ext t; fin_cases t <;>
    simp [leaf00, leaf11]] at hi
      exact not_mem_empty (x.sub hi)
    * exfalso
      have hi := x.inter_mem hx h
      rw [show (leaf01  I  leaf11 : Set Token) = {} from by ext t; fin_cases t <;>
    simp [leaf01, leaf11]] at hi
      exact not_mem_empty (x.sub hi)
    * exfalso
      have hi := x.inter_mem hx h
      rw [show (leaf10  I  leaf11 : Set Token) = {} from by ext t; fin_cases t <;>
    simp [leaf10, leaf11]] at hi
      exact not_mem_empty (x.sub hi)
    * exact Or.inr (Or.inr rfl)
  * intro hx
    rcases hx with rfl | rfl | rfl
    * exact x.master_mem
    * exact hright
    * exact h

theorem mem_left_imp (x : neighborhoodSystem.Element) (hl : x.mem left)
    (h00 : not x.mem leaf00) (h01 : not x.mem leaf01) : x = elemZero := by
  apply Element.ext
  intro X
  constructor
  * intro hx
```

```
    rcases (mem_iff X).mp (x.sub hx) with rfl | rfl | rfl | rfl | rfl | rfl | rfl
    * exact Or.inl rfl
    * exact Or.inr rfl
    * exfalso
      have hi := x.inter_mem hl hx
      rw [show (left  I  right : Set Token) = {} from by ext t; fin_cases t <;> simp [
    left, right]] at hi
      exact not_mem_empty (x.sub hi)
    * exact absurd hx h00
    * exact absurd hx h01
    * exfalso
      have hi := x.inter_mem hl hx
      rw [show (left  I  leaf10 : Set Token) = {} from by ext t; fin_cases t <;> simp
    [left, leaf10]] at hi
      exact not_mem_empty (x.sub hi)
    * exfalso
      have hi := x.inter_mem hl hx
      rw [show (left  I  leaf11 : Set Token) = {} from by ext t; fin_cases t <;> simp
    [left, leaf11]] at hi
      exact not_mem_empty (x.sub hi)
  * intro hx
    rcases hx with rfl | rfl
    * exact x.master_mem
    * exact hl

theorem mem_right_imp (x : neighborhoodSystem.Element) (hr : x.mem right)
    (h10 : not x.mem leaf10) (h11 : not x.mem leaf11) : x = elemOne := by
  apply Element.ext
  intro X
  constructor
  * intro hx
    rcases (mem_iff X).mp (x.sub hx) with rfl | rfl | rfl | rfl | rfl | rfl | rfl
    * exact Or.inl rfl
    * exfalso
      have hi := x.inter_mem hr hx
      rw [show (right  I  left : Set Token) = {} from by ext t; fin_cases t <;> simp [
    right, left]] at hi
      exact not_mem_empty (x.sub hi)
    * exact Or.inr rfl
    * exfalso
      have hi := x.inter_mem hr hx
      rw [show (right  I  leaf00 : Set Token) = {} from by ext t; fin_cases t <;> simp
     [right, leaf00]] at hi
      exact not_mem_empty (x.sub hi)
    * exfalso
      have hi := x.inter_mem hr hx
      rw [show (right  I  leaf01 : Set Token) = {} from by ext t; fin_cases t <;> simp
     [right, leaf01]] at hi
      exact not_mem_empty (x.sub hi)
    * exact absurd hx h10
    * exact absurd hx h11
  * intro hx
    rcases hx with rfl | rfl
    * exact x.master_mem
    * exact hr
```

```
section

open NeighborhoodSystem

theorem bot_lt_elemOne : bot < elemOne := by
  constructor
  * intro X hx; rw [hx]; exact Or.inl rfl
  * intro h
    exact right_ne_master (h right (Or.inr rfl))

theorem bot_lt_elemZero : bot < elemZero := by
  constructor
  * intro X hx; rw [hx]; exact Or.inl rfl
  * intro h
    exact left_ne_master (h left (Or.inr rfl))

theorem elem00_maximal (x : neighborhoodSystem.Element) (h : elem00  <=  x) : x =
    elem00 :=
  mem_leaf00_imp x (h leaf00 (Or.inr (Or.inr rfl)))

theorem elem01_maximal (x : neighborhoodSystem.Element) (h : elem01  <=  x) : x =
    elem01 :=
  mem_leaf01_imp x (h leaf01 (Or.inr (Or.inr rfl)))

theorem elem10_maximal (x : neighborhoodSystem.Element) (h : elem10  <=  x) : x =
    elem10 :=
  mem_leaf10_imp x (h leaf10 (Or.inr (Or.inr rfl)))

theorem elem11_maximal (x : neighborhoodSystem.Element) (h : elem11  <=  x) : x =
    elem11 :=
  mem_leaf11_imp x (h leaf11 (Or.inr (Or.inr rfl)))

theorem elemOne_lt_elem10 : elemOne < elem10 := by
  constructor
  * intro X hx
    rcases hx with rfl | rfl
    * exact Or.inl rfl
    * exact Or.inr (Or.inl rfl)


  * intro h
    rcases h leaf10 (Or.inr (Or.inr rfl)) with h1 | h1
    * exact leaf10_ne_master h1
    * exact leaf10_ne_right h1

theorem elemOne_lt_elem11 : elemOne < elem11 := by
  constructor
  * intro X hx
    rcases hx with rfl | rfl
    * exact Or.inl rfl
    * exact Or.inr (Or.inl rfl)
  * intro h
    rcases h leaf11 (Or.inr (Or.inr rfl)) with h1 | h1
    * exact leaf11_ne_master h1
    * exact leaf11_ne_right h1

theorem elemZero_lt_elem00 : elemZero < elem00 := by
```

```
  constructor
  * intro X hx
    rcases hx with rfl | rfl
    * exact Or.inl rfl
    * exact Or.inr (Or.inl rfl)
  * intro h
    rcases h leaf00 (Or.inr (Or.inr rfl)) with h1 | h1
    * exact leaf00_ne_master h1
    * exact leaf00_ne_left h1

theorem elemZero_lt_elem01 : elemZero < elem01 := by
  constructor
  * intro X hx
    rcases hx with rfl | rfl
    * exact Or.inl rfl
    * exact Or.inr (Or.inl rfl)
  * intro h
    rcases h leaf01 (Or.inr (Or.inr rfl)) with h1 | h1
    * exact leaf01_ne_master h1
    * exact leaf01_ne_left h1

/-- **Example 1.4 classification.** Every filter is one of the seven: the bottom `False
    `, the two
branch partials `elemZero`/`elemOne`, or one of the four total leaf elements. -/
theorem element_classification (x : neighborhoodSystem.Element) :
    x = bot  \/  x = elemZero  \/  x = elemOne  \/  x = elem00  \/  x = elem01  \/  x
    = elem10  \/  x = elem11 := by
  by_cases h00 : x.mem leaf00
  * exact Or.inr (Or.inr (Or.inr (Or.inl (mem_leaf00_imp x h00))))
  by_cases h01 : x.mem leaf01
  * exact Or.inr (Or.inr (Or.inr (Or.inr (Or.inl (mem_leaf01_imp x h01)))))
  by_cases h10 : x.mem leaf10
  * exact Or.inr (Or.inr (Or.inr (Or.inr (Or.inr (Or.inl (mem_leaf10_imp x h10))))))
  by_cases h11 : x.mem leaf11
  * exact Or.inr (Or.inr (Or.inr (Or.inr (Or.inr (Or.inr (mem_leaf11_imp x h11))))))
  by_cases hl : x.mem left
  * exact Or.inr (Or.inl (mem_left_imp x hl h00 h01))
  by_cases hr : x.mem right
  * exact Or.inr (Or.inr (Or.inl (mem_right_imp x hr h10 h11)))
  apply Or.inl
  apply Element.ext
  intro X
  constructor
  * intro hx
    rcases (mem_iff X).mp (x.sub hx) with rfl | rfl | rfl | rfl | rfl | rfl | rfl
    * rfl
    * exact absurd hx hl
    * exact absurd hx hr
    * exact absurd hx h00
    * exact absurd hx h01
    * exact absurd hx h10
    * exact absurd hx h11
  * intro hx
    rw [hx]
    exact x.master_mem
end
```

```
end neighborhoodSystem

end DomainTheory.NeighbourhoodSystems.Example14
end
```

3.2.7 Factoid 1.4a — nested or disjoint

very special circumstance: in these systems two neighbourhoods are either disjoint or one is included in the other. This arrangement of neighbourhoods is by no means necessary.

Factoid_1_4.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_1

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
/- concordance: source L206 FACTOID 1.4a nested or disjoint -/
/-- **Factoid 1.4a (Scott 1981, PRG-19).** "In these systems two neighbourhoods are
    either
disjoint or one is included in the other": a family containing `Delta` whose members
    are pairwise
nested-or-disjoint **is** a neighbourhood system. This uniformly explains why Examples
     1.2,
1.3 and 1.4 satisfy Definition 1.1.

The verification of condition (ii) needs no choice: if `X, Y` are nested then `X  I  Y`
     is the
smaller (already in `?`); if they are disjoint then the consistency witness `Z
    subseteq  X  I  Y = {}`
forces `Z = {}`, whence `X  I  Y = {} = Z  in  ?`. The caller supplies `master_nonempty
    `
(Scott's standing `Delta  !=  {}` assumption) and `sub_master` (`?  subseteq  ?(Delta)
    `) directly. -/
def NeighborhoodSystem.ofNestedOrDisjoint {a : Type*} (mem : Set a  ->  Prop) (master
    : Set a)
    (master_nonempty : master.Nonempty) (master_mem : mem master) (hnd :
    NestedOrDisjoint mem)
    (sub_master : forall  {X : Set a}, mem X  ->  X  subseteq  master) :
    NeighborhoodSystem a where
  mem
  master
  master_nonempty
  master_mem
  sub_master
  inter_mem := by
    intro X Y Z hX hY hZ hZsub
    rcases hnd hX hY with h | h | h
    * rwa [Set.inter_eq_left.mpr h]
    * rwa [Set.inter_eq_right.mpr h]
```

```
    * rw [h]
      rw [h] at hZsub
      rwa [ <-  Set.subset_empty_iff.mp hZsub]

end DomainTheory.NeighbourhoodSystems
end
```

3.2.8 Example 1.5 — all nonempty subsets

EXAMPLE 1.5. Let $\Delta = \{0, 1, 2, 3\}$ and let $\mathcal{D}$ be the family of all non-empty subsets of Δ.

Example_1_5.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.DomainIso

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems.Example15
section

open NeighborhoodSystem

/-- Tokens for Example 1.5: `Delta = {0, 1, 2, 3}`. -/
abbrev Token := Fin 4

/-- Membership: the non-empty subsets of `Delta`. -/
def P4Mem (X : Set Token) : Prop := X.Nonempty

/-- A non-empty witness inside `X  I  Y` shows the intersection is a neighbourhood. -/
theorem P4_inter_mem {X Y Z : Set Token} (_hX : P4Mem X) (_hY : P4Mem Y)
    (hZ : P4Mem Z) (hZsub : Z  subseteq  X  I  Y) : P4Mem (X  I  Y) :=
  hZ.mono hZsub

/-- The master neighbourhood `Delta = {0, 1, 2, 3}`. -/
def master : Set Token := Set.univ

/-- `Delta` is non-empty. -/
theorem P4_master_mem : P4Mem master := by
  rw [master]; exact Set.univ_nonempty

/-- The master token set itself is non-empty. -/
theorem P4_master_nonempty : master.Nonempty := by
  exact <0, Set.mem_univ 0>

/-- Every neighbourhood is a subset of `Delta`. -/
theorem P4_sub_master {X : Set Token} (_h : P4Mem X) : X  subseteq  master :=
  Set.subset_univ _

/- concordance: source L208 EXAMPLE 1.5 all nonempty subsets -/
/-- **Example 1.5.** The neighbourhood system of all non-empty subsets of `Delta =
    {0,1,2,3}`. -/
def neighborhoodSystem : NeighborhoodSystem Token where
```

```
  mem := P4Mem
  master
  master_nonempty := P4_master_nonempty
  master_mem := P4_master_mem
  inter_mem := P4_inter_mem
  sub_master := P4_sub_master

/-- The neighbourhoods of Example 1.5 are a finite family, hence countable. -/
instance P4_countable : Countable {S : Set Token // neighborhoodSystem.mem S} :=
  inferInstance
end

end DomainTheory.NeighbourhoodSystems.Example15
end
```

3.2.9 Factoid 1.5 — consistent iff nonempty; equivalent sequences same limit

Factoid 1.5a — consistent iff nonempty This system is a direct generalization of Example 1.2., which was special owing to the small number of tokens. (The other examples were special by virtue of the choice of neighbourhoods.) The verification that the present $\mathcal{D}$ is a neighbourhood system rests on nothing more than the remark that sets are consistent in $\mathcal{D}$ iff they have a non-empty intersection. Clearly the arrangement of neighbourhoods in $\mathcal{D}$ can be as varied as a four-element set will allow; if Δ were made larger, the possible combinations of neighbourhoods could be made as complex as you wish. □

Having some idea now of the variety of neighbourhood systems, we have to discuss what it is they do. As stressed before, the tokens do not have to correspond directly to the elements; but where, we ask, do the elements come from? One obvious suggestion for determining an element is to produce a sequence of "better and better" neighbourhoods:

$$X_0 \supseteq X_1 \supseteq \cdots \supseteq X_n \supseteq \cdots$$

Trivially, any finite initial segment of this sequence is consistent, and so each X_n is a partial approximation to the "limit". If $\mathcal{D}$ were always to be taken as *finite*, of course, there would be no point in discussing limits since any such sequence would eventually be constant. The elements in the finite case would therefore be completely represented by neighbourhoods with the *minimal* neighbourhoods corresponding to the total elements. But there are many reasons to go beyond the finite (though perhaps not too far beyond).

Suppose $\langle Y_n \rangle_{n=0}^{\infty}$ is another "convergent" sequence with

$Y_{n+1} \subseteq Y_n$ for all indices: when do the two sequences of neighbourhoods determine the *same limit*? The two sequences can surely be different; for example, $\langle Y_n \rangle_{n=0}^{\infty}$ could be a subsequence of $\langle X_n \rangle_{n=0}^{\infty}$, say, $Y_n = X_{2n}$. Still we would want to say that the same limit is obtained. Without being given any further structure on the neighbourhoods, a simple answer is just to say that each sequence goes *equally deep* as the other:

for each m there is an n with $X_n \subseteq Y_m$, and

for each n there is an m with $Y_m \subseteq X_n$.

This definition obviously puts sequences into equivalence classes, and so elements could be identified with these. But such a definition is clumsy for two reasons: it is always tiresome to work with equivalence classes, and there is no reason to think that simple infinite sequences are adequate for determining elements without some rather drastic assumptions on $\mathcal{D}$. Nevertheless, the idea is *suggestive*; we just have to find some construct to represent elements in a unique way and to phrase it in a general enough manner.

Start with $\langle X_n \rangle_{n=0}^{\infty}$ again, which "converges" as before. Think of all the other sequences equivalent to this one in the sense just defined. We can define the class of all terms of all such sequences very easily as being the family:

$$x = \{Z \in \mathcal{D} \mid X_n \subseteq Z \text{ for some } n\}.$$

Factoid 1.5b — equivalent sequences same limit It is easy to prove that if we form the analogous class for $\langle Y_n \rangle_{n=0}^{\infty}$, then the two families are *equal* if and only if the sequences are *equivalent*. Thus, we seem justified in letting x represent the limit of $\langle X_n \rangle_{n=0}^{\infty}$. All we have to do now is to remark on what sort of class x is as a subfamily of $\mathcal{D}$; what we abstract from the construction, however, will be just a bit more general than taking those x that result from sequences.

Factoid_1_5.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Example_1_5

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
/-- The system built by `ofPositive` is indeed positive. -/
theorem NeighborhoodSystem.ofPositive_isPositive {a : Type*} (mem : Set a  ->  Prop)
    (master : Set a) (master_nonempty : master.Nonempty) (master_mem : mem master)
    (sub_master : forall  {X : Set a}, mem X  ->  X  subseteq  master)
    (pos : forall  ?X Y : Set a?, mem X  ->  mem Y  ->  (mem (X  I  Y)  <->  (X  I  Y)
    .Nonempty)) :
    (NeighborhoodSystem.ofPositive mem master master_nonempty master_mem sub_master
    pos).IsPositive :=
  pos

namespace NeighborhoodSystem

variable {a : Type*} (V : NeighborhoodSystem a)

/-- The finite intersection is contained in each of its factors: `?_{i < n} X_i
    subseteq  X?` for
`j < n`. (Supporting lemma: this is what makes `?_{i < n} X_i` a common lower bound of
    the
sequence, the intuition behind consistency.) -/
theorem interUpTo_subset (X : Nat  ->  Set a) :
    forall  {n j : Nat}, j < n  ->  V.interUpTo X n  subseteq  X j := by
  intro n
  induction n with
  | zero => intro j h; exact absurd h (Nat.not_lt_zero j)
  | succ n ih =>
    intro j h
    rw [interUpTo_succ]
    rcases Nat.eq_or_lt_of_le (Nat.lt_succ_iff.mp h) with h' | h'
    * subst h'; exact Set.inter_subset_right
    * exact Set.inter_subset_left.trans (ih h')
```

```
/-- **Theorem 1.1c (extension of the intersection property).** Scott: "from (ii), we
    can
extend the intersection property to any finite sequence." If `X_i  in  ?` for every `i
    < n`
and the sequence is consistent, then the finite intersection `?_{i < n} X_i` is again a
neighbourhood (` in  ?`). Proved by induction on `n`; the inductive step is one
    application of
condition (ii). -/
theorem interUpTo_mem (X : Nat  ->  Set a) :
    forall  {n : Nat}, (forall  i, i < n  ->  V.mem (X i))  ->  V.Consistent X n  ->
      V.mem (V.interUpTo X n) := by
  intro n
  induction n with
  | zero => intro _ _; exact V.master_mem
  | succ n ih =>
    intro hX hcons
    obtain <Z, hZmem, hZsub> := hcons
    have hZsub' : Z  subseteq  V.interUpTo X n  I  X n := by rwa [interUpTo_succ] at
    hZsub
    -- The same witness `Z` shows the length-`n` prefix is consistent.
    have hconsn : V.Consistent X n :=
      <Z, hZmem, hZsub'.trans Set.inter_subset_left>
    have hmemn : V.mem (V.interUpTo X n) :=
      ih (fun i hi => hX i (Nat.lt_succ_of_lt hi)) hconsn
    have hXn : V.mem (X n) := hX n (Nat.lt_succ_self n)
    rw [interUpTo_succ]
    exact V.inter_mem hmemn hXn hZmem hZsub'

/-- **Theorem 1.1c (consistency characterization).** "Consequently, `X_0, ..., X_n?_1`
    is
consistent in `?` iff `?_{i < n} X_i  in  ?`." (Given `X_i  in  ?` for all `i < n`.)

* ` -> ` is the extension property `interUpTo_mem`;
* ` <- ` is immediate: the intersection is its own common lower bound. -/
theorem consistent_iff_interUpTo_mem (X : Nat  ->  Set a) {n : Nat}
    (hX : forall  i, i < n  ->  V.mem (X i)) :
    V.Consistent X n  <->  V.mem (V.interUpTo X n) := by
  constructor
  * exact V.interUpTo_mem X hX
  * intro h; exact <V.interUpTo X n, h, Set.Subset.refl _>

/-- A filter (`Element`) is closed under the finite intersection `?_{i<n} X_i`: if
    every factor
`X_i` (`i < n`) lies in the filter `x`, so does `interUpTo X n`. Used in Exercises 1.18
     and 1.21.
Base case `x.master_mem`; inductive step one `x.inter_mem`. -/
theorem Element.mem_interUpTo {a : Type*} {V : NeighborhoodSystem a} (x : V.Element)
    (X : Nat  ->  Set a) :
    forall  {n : Nat}, (forall  i, i < n  ->  x.mem (X i))  ->  x.mem (V.interUpTo X n
    ) := by
  intro n
  induction n with
  | zero => intro _; exact x.master_mem
  | succ n ih =>
    intro h
    rw [interUpTo_succ]
```

```
    exact x.inter_mem (ih (fun i hi => h i (Nat.lt_succ_of_lt hi))) (h n (Nat.
    lt_succ_self n))

/-- Membership of the finite intersection in a filter, as a biconditional (given all
    factors
are neighbourhoods). ` -> ` is upward closure along `interUpTo X n  subseteq  X_i` (`
    interUpTo_subset`); ` <- `
is `Element.mem_interUpTo`. -/
theorem Element.mem_interUpTo_iff {a : Type*} {V : NeighborhoodSystem a} (x : V.
    Element)
    (X : Nat  ->  Set a) {n : Nat} (hX : forall  i, i < n  ->  V.mem (X i)) :
    x.mem (V.interUpTo X n)  <->  forall  i, i < n  ->  x.mem (X i) := by
  constructor
  * intro h i hi
    exact x.up_mem h (hX i hi) (V.interUpTo_subset X hi)
  * exact x.mem_interUpTo X

/- concordance: source L238 FACTOID 1.5b equivalent sequences same limit -/
/-- **Factoid 1.5b (Scott 1981, PRG-19).** "It is easy to prove that ... the two
    families are
*equal* if and only if the sequences are *equivalent*." Given that every term of each
    sequence
is a neighbourhood, the limit families coincide exactly when the sequences are
    equivalent. -/
theorem limitFamily_eq_iff (X Y : Nat  ->  Set a)
    (hX : forall  n, V.mem (X n)) (hY : forall  m, V.mem (Y m)) :
    V.limitFamily X = V.limitFamily Y  <->  SeqEquiv X Y := by
  constructor
  * intro hEq
    refine <fun m => ?_, fun n => ?_>
    * have hmem : Y m  in  V.limitFamily Y := <hY m, m, subset_rfl>
      rw [ <-  hEq] at hmem
      obtain <_, n, hn> := hmem
      exact <n, hn>
    * have hmem : X n  in  V.limitFamily X := <hX n, n, subset_rfl>
      rw [hEq] at hmem
      obtain <_, m, hm> := hmem
      exact <m, hm>
  * rintro <h1, h2>
    apply Set.ext
    intro Z
    constructor
    * rintro <hZ, n, hn>
      obtain <m, hm> := h2 n
      exact <hZ, m, hm.trans hn>
    * rintro <hZ, m, hm>
      obtain <n, hn> := h1 m
      exact <hZ, n, hn.trans hm>

end NeighborhoodSystem

end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.Example15
section

open NeighborhoodSystem
```

```
/- concordance: source L210 FACTOID 1.5a consistent iff nonempty -/
/-- **Factoid 1.5a (Scott 1981, PRG-19).** In Example 1.5, "sets are consistent in `?`
    iff they
have a non-empty intersection": a finite prefix `X_0, ..., X_n?_1` is consistent
    exactly when its
intersection `?_{i<n} X_i` is non-empty. -/
theorem consistent_iff_inter_nonempty (X : Nat  ->  Set Token) (n : Nat) :
    neighborhoodSystem.Consistent X n  <->  (neighborhoodSystem.interUpTo X n).
    Nonempty := by
  constructor
  * rintro <Z, hZ, hsub>
    obtain <z, hz> := hZ
    exact <z, hsub hz>
  * intro h
    exact <_, h, subset_rfl>

/-- The neighbourhoods of Example 1.5 are exactly the non-empty subsets. -/
theorem mem_iff_nonempty (X : Set Token) : neighborhoodSystem.mem X  <->  X.Nonempty
    := Iff.rfl
end

end DomainTheory.NeighbourhoodSystems.Example15
end
```

3.2.10 Definition 1.6 — ideal elements

DEFINITION 1.6. The (ideal) elements of a neighbourhood system $\mathcal{D}$ are those subfamilies $x \subseteq \mathcal{D}$ where:

(i) $\Delta \in x$;

(ii) $X, Y \in x$ always implies $X \cap Y \in x$; and

(iii) whenever $X \in x$ and $X \subseteq Y \in \mathcal{D}$, then $Y \in x$.

The *domain* of all such elements is written as $|\mathcal{D}|$. □

The idea of 1.6 is a well-known mathematical device: the families x satisfying (i)–(iii) are usually called *filters*. Most frequently the emphasis is put on the *maximal* filters, and these would be our *total* elements; however, in general, the proof that maximal filters exist is non-constructive, so for our purposes it is better not to neglect the partial filters. When maximal filters can be found, well and good, but we do not have to insist on them. Note that the generality of 1.6 is achieved by not requiring that there is a sequence of neighbourhoods that "generates" the filter x. (See Exercise 1.22.)

We have often said that neighbourhoods determine partial elements by themselves; we now make this remark precise.

`Element.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NeighborhoodSystem
```

```
@[expose] public section

section
-- Element.lean
namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem

variable {a : Type*} (V : NeighborhoodSystem a)

/- concordance: source L240 DEFINITION 1.6 ideal elements -/
/-- **Definition 1.6 (Scott 1981, PRG-19).** An (ideal) *element* of a neighbourhood
    system:
a subfamily `x  subseteq  ?` that is a *filter* -- (i) `Delta  in  x`, (ii) closed
    under intersection, (iii)
upward closed within `?`. The domain is the type `Element` of all such filters, ordered
     by
inclusion. -/
structure Element where
  /-- `mem X` holds iff the neighbourhood `X` belongs to the filter `x`. -/
  mem : Set a  ->  Prop
  /-- `x` is a subfamily of `?`. -/
  sub : forall  {X}, mem X  ->  V.mem X
  /-- (i) `Delta  in  x`. -/
  master_mem : mem V.master
  /-- (ii) `X, Y  in  x ? X  I  Y  in  x`. -/
  inter_mem : forall  {X Y}, mem X  ->  mem Y  ->  mem (X  I  Y)
  /-- (iii) `X  in  x` and `X  subseteq  Y  in  ? ? Y  in  x`. -/
  up_mem : forall  {X Y}, mem X  ->  V.mem Y  ->  X  subseteq  Y  ->  mem Y

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
end
```

3.2.11 Definition 1.7 — principal filter

DEFINITION 1.7. For $X \in \mathcal{D}$, the *principal filter* determined by X is defined by:

$$\uparrow X = \{Y \in \mathcal{D} \mid X \subseteq Y\}.$$

The principal filters form what we shall call the *finite elements* of the domain $|\mathcal{D}|$. □

principal.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_4

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem
section
```

```
variable {a : Type*} (V : NeighborhoodSystem a)

/- concordance: source L256 DEFINITION 1.7 principal filter -/
/-- **Definition 1.7 (Scott 1981, PRG-19).** The *principal filter* `X` determined by a
neighbourhood `X  in  ?`:

`X = {Y  in  ? ? X  subseteq  Y}`.

These are Scott's *finite elements* of `|?|`. The four filter conditions:

* `sub` is the first projection (`Y  in  X ? Y  in  ?`);
* `master_mem` needs `X  subseteq  Delta`, supplied by `V.sub_master` (Scott's `?
    subseteq  ?(Delta)`);
* `inter_mem` uses `Set.subset_inter` (from `X  subseteq  Y_1`, `X  subseteq  Y_2`)
    with `X` itself as the
  consistency witness for `V.inter_mem`;
* `up_mem` is transitivity of ` subseteq `. -/
def principal {X : Set a} (hX : V.mem X) : V.Element where
  mem Y := V.mem Y  /\  X  subseteq  Y
  sub h := h.1
  master_mem := <V.master_mem, V.sub_master hX>
  inter_mem h1 h2 :=
    <V.inter_mem h1.1 h2.1 hX (Set.subset_inter h1.2 h2.2), Set.subset_inter h1.2 h2
    .2>
  up_mem h hY hsub := <hY, h.2.trans hsub>

@[simp] theorem mem_principal {X Y : Set a} (hX : V.mem X) :
    (V.principal hX).mem Y  <->  V.mem Y  /\  X  subseteq  Y := Iff.rfl
end

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
end
```

3.2.12 Factoid 1.7 — principals are one-one; inclusion-reversing principals; union of principal filters

Factoid 1.7a1 — principals are one-one It is obvious that the correspondence between X and $\uparrow X$ is one-one and inclusion *reversing*, in the sense that

Factoid 1.7a2 — inclusion-reversing principals

$$X \subseteq Y \quad \text{iff} \quad \uparrow Y \subseteq \uparrow X$$

for all $X, Y \in \mathcal{D}$. But, except in very special cases, there is much more to $|\mathcal{D}|$ than just the finite elements. Much of our investigation will be concerned with finding out how much more. The finite elements are, in a certain sense, "dense" in $|\mathcal{D}|$, however, because it is also obvious from the definitions that for each $x \in |\mathcal{D}|$

Factoid 1.7b — union of principal filters

$$x = \bigcup \{\uparrow X \mid X \in x\}.$$

That is, every element is a certain type of "limit" of finite elements. (This statement is made more precise in Exercise 1.21)

We note that we have now had several occasions to use inclusion relationships between elements; this is an important relationship, and we give it a special name.

Factoid_1_7.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_5

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems

namespace NeighborhoodSystem

variable {a : Type*} (V : NeighborhoodSystem a)

/- concordance: source L267 FACTOID 1.7a2 inclusion-reversing principals -/
/- concordance: source L264 FACTOID 1.7a1 principals are one-one -/
/-- **Factoid 1.7a (Scott 1981, PRG-19) -- inclusion-*reversing*.** "It is obvious
    that the
correspondence between `X` and `X` is one-one and inclusion *reversing*." The order on
    ``:
`X  sqsub  Y  <->  Y  subseteq  X` (equivalently Scott's `X  subseteq  Y  <->  Y  sqsub
     X`).

` -> ` tests at `Z = X` (`X  in  X` since `X  subseteq  X`), reading off `Y  subseteq  X
    ` from `X  in  Y`; ` <- ` chains
`Y  subseteq  X  subseteq  Z`. -/
theorem principal_le_iff {X Y : Set a} (hX : V.mem X) (hY : V.mem Y) :
    V.principal hX  <=  V.principal hY  <->  Y  subseteq  X := by
  constructor
  * intro h
    exact (h X <hX, subset_rfl>).2
  * intro hYX Z hZ
    exact <hZ.1, hYX.trans hZ.2>

/-- **Factoid 1.7a (Scott 1981, PRG-19) -- one-one.** The correspondence `X  |->  X` is
     injective:
`X = Y ? X = Y`. Antisymmetry applied to `principal_le_iff` in both directions. -/
theorem principal_injective {X Y : Set a} (hX : V.mem X) (hY : V.mem Y)
    (h : V.principal hX = V.principal hY) : X = Y := by
  have hYX : Y  subseteq  X := (V.principal_le_iff hX hY).mp (le_of_eq h)
  have hXY : X  subseteq  Y := (V.principal_le_iff hY hX).mp (le_of_eq h.symm)
  exact Set.Subset.antisymm hXY hYX

/- concordance: source L273 FACTOID 1.7b union of principal filters -/
/-- **Factoid 1.7b (Scott 1981, PRG-19).** "It is also obvious from the definitions
    that for each
`x  in  |?|`, `x = ? {X ? X  in  x}`." In membership form (the union over a `Set (Set a
    )` made
concrete): a neighbourhood `Z` is in `x` iff `Z` lies in the principal filter `X` of *
```

```
    some*
member `X` of `x`.

` -> ` uses `X = Z` (`Z  in  Z` as `Z  subseteq  Z`); ` <- ` is upward closure `up_mem`
    (`X  subseteq  Z`, `Z  in  ?`). -/
theorem eq_iUnion_principal (x : V.Element) {Z : Set a} :
    x.mem Z  <->  exists  X, exists  hX : x.mem X, (V.principal (x.sub hX)).mem Z :=
    by
  constructor
  * intro hZ
    exact <Z, hZ, x.sub hZ, subset_rfl>
  * rintro <X, hX, hVZ, hXZ>
    exact x.up_mem hX hVZ hXZ

end NeighborhoodSystem

end DomainTheory.NeighbourhoodSystems
end
```

3.2.13 Definition 1.8 — approximation and totality

DEFINITION 1.8. For $x, y \in |\mathcal{D}|$, we say that x *approximates* y iff $x \subseteq y$. The element that approximates all others, $\{\Delta\}$, is called $\bot$ (read: *bottom*); it is the "least defined" element, or the "most partial" element. Elements maximal with respect to the approximation relation are called *total elements*. □

bot.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.principal

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem
section

variable {a : Type*} (V : NeighborhoodSystem a)

/- concordance: source L282 DEFINITION 1.8 approximation and totality -/
/-- **Definition 1.8 (Scott 1981, PRG-19) -- `False`.** The least defined element `
    False = {Delta}`,
"read: *bottom*". It is the principal filter of the master neighbourhood `Delta`: `
    False = Delta`. -/
def bot : V.Element := V.principal V.master_mem

/-- **Definition 1.8 -- `False = {Delta}` literally.** Scott's `False` is the *
    singleton* `{Delta}`: a
neighbourhood `Y` belongs to `False` iff `Y = Delta`.

` -> `: `Y  in  False = Delta` gives `Y  in  ?` and `Delta  subseteq  Y`; `V.sub_master`
     gives the reverse `Y  subseteq  Delta`, so
`Y = Delta` by antisymmetry. ` <- `: `Delta  in  ?` and `Delta  subseteq  Delta`. -/
```

```
@[simp] theorem mem_bot {Y : Set a} : V.bot.mem Y  <->  Y = V.master := by
  constructor
  * rintro <hY, hDeltaY>
    exact Set.Subset.antisymm (V.sub_master hY) hDeltaY
  * rintro rfl
    exact <V.master_mem, subset_rfl>
end

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
end
```

3.2.14 Factoid 1.8 — bottom element; bottom is least; finite filters are principal; prefix order; union of finite approximations

Factoid 1.8a1 — bottom element DEFINITION 1.8. For $x, y \in |\mathcal{D}|$, we say that x *approximates* y iff $x \subseteq y$. The element that approximates all others, $\{\Delta\}$, is called $\perp$ (read: *bottom*); it is the "least defined" element, or the "most partial" element. Elements maximal with respect to the approximation relation are called *total elements*. □

Factoid 1.8a2 — bottom is least DEFINITION 1.8. For $x, y \in |\mathcal{D}|$, we say that x *approximates* y iff $x \subseteq y$. The element that approximates all others, $\{\Delta\}$, is called $\perp$ (read: *bottom*); it is the "least defined" element, or the "most partial" element. Elements maximal with respect to the approximation relation are called *total elements*. □

Factoid 1.8b — finite filters are principal EXAMPLES 1.2 – 1.5 (Revisited). The examples as given were all finite, so any explicitly given filter x is principal, the element is finite, the minimal $X \in x$ tells us all we need to know. In such simple situations there is essentially no difference between elements and neighbourhoods — except for the reversal of the order as noted. This (necessary) reversal should not, however, become a matter of confusion: the smaller the neighbourhood has become, the more it has "converged", and so the better defined the element has become. In the approximation relation the "poorer" elements are placed below the "better" with the total up at the top. This will become clearer in discussing "infinite" elements.

Example 1.3 will be generalized in Exercise 1.12. Let us here generalize first 1.4. We let

$$\Delta = \Sigma^*,$$

where $\Sigma = \{0, 1\}$ and Σ^* means the set of all finite sequences of 0's and 1's, with Λ being the empty sequence. We write $\sigma\tau$ for the *concatenation* or *juxtaposition* of two sequences $\sigma, \tau \in \Sigma^*$. Define

$$B = \{\sigma\Sigma^* \mid \sigma \in \Sigma^*\}, \text{ where}$$

$$\sigma X = \{\sigma\tau \mid \tau \in X\},$$

for an arbitrary set $X \subseteq \Sigma^*$. In other words, a neighbourhood in B consists of all *extensions* of a given sequence σ. (Refer back to the finite version of 1.4.) We use the letter "B" to remind us of "binary", and this is an example we shall refer to many times. The proof (if it is not obvious) that B is a neighbourhood system should be done as an exercise.

What do we find in $|B|$? Of course $\perp = \{\Delta\} \in |B|$. For any $x \in |B|$ and $\sigma \in \Sigma^*$ define

$$\sigma x = \{Y \mid \sigma X \subseteq Y \text{ some } X \in x\}.$$

Factoid 1.8c — prefix order Again there is an exercise here to show $\sigma x \in |B|$. In particular $\sigma \perp \in |B|$ for all $\sigma \in \Sigma^*$, and these are just the finite elements. The minimal element of $\sigma \perp$ is $\sigma\Delta$. Note that $\sigma_0 \perp \subseteq \sigma_1 \perp$ if and only if σ_0 is an *initial segment* of the sequence σ_1.

If now $x \in |B|$ is any explicitly given element (that is, if we know for any $X \in B$ whether or not $X \in x$), we have but to work out from these definitions that

Factoid 1.8d — union of finite approximations

$$x = \bigcup_{n=0}^{\infty} \sigma_n \perp,$$

where the $\sigma_n \in \Sigma^*$ and each σ_n is an initial segment of the next σ_{n+1}. *In general, in any domain, an element is uniquely determined by its finite approximations, and we are just making this explicit in* $|B|$. When we have complete knowledge of x, then there are two cases: either the approximations $\sigma_n \perp$ become constant from some point on (where $n > n_0$), or not. In the first case x is finite and equal to $\sigma_{n_0} \perp$; in the second case x is infinite and the σ_n fill out an infinite (one-way) sequence.

The generalization of 1.5 to the infinite case where $\Delta = \mathbf{N} = \{0, 1, 2, 3, \ldots, n, \ldots\}$ can be made in more than one way: for instance either we use as neighbourhoods *all* non-empty subsets of Δ or just those omitting but a finite number of integers. And, as will become apparent, there are other choices giving domains of quite different characters. □

Many constructions (choices of $\mathcal{D}$) lead to the "same" domain; "sameness" is an important notion and it is to be defined in terms of "isomorphism", which in turn is to be defined in terms of approximation preserving correspondences.

Factoid_1_8.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_7
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Examples_1_2_1_5_revisited

@[expose] public section

/- concordance: source L282 FACTOID 1.8a1 bottom element -/

section
namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem
section

variable {a : Type*} (V : NeighborhoodSystem a)

/- concordance: source L282 FACTOID 1.8a2 bottom is least -/
/-- **Factoid 1.8a (Scott 1981, PRG-19).** "The element that approximates all others,
    `{Delta}`,
```

```
is called `False`": `False` is the least element of `|?|`, `False  sqsub  x` for every `
    x`.

Given `Y  in  False`, i.e. `Y = Delta`, membership `Delta  in  x` is filter condition (
    i) (`x.master_mem`). -/
theorem bot_le (x : V.Element) : V.bot  <=  x := by
  intro Y hY
  rw [mem_bot] at hY
  subst hY
  exact x.master_mem

/-- **Factoid 1.8a, packaged.** `False` is an `OrderBot` for the approximation order,
    so the `False`
notation refers to `{Delta}`. Constructive (`bot_le` is `[propext, Quot.sound]`). -/
instance : OrderBot V.Element where
  bot := V.bot
  bot_le := V.bot_le

/- concordance: source L284 FACTOID 1.8b finite filters are principal -/
/- concordance: source L284 EXAMPLES 1.2-1.5 finite examples revisited -/
/-- **Factoid 1.8b (Scott 1981, PRG-19) -- "Examples 1.2-1.5 revisited".** "Any
    explicitly given
filter `x` is principal ... the minimal `X  in  x` tells us all we need to know."
    Stated honestly: if
the filter `x` has a ` subseteq `-minimum member `X` (one contained in every member of
    `x`), then `x` is
exactly the principal filter `X`. In a *finite* system every filter has such a minimum
    (the
intersection of its finitely many members, itself in `x` by closure), so every element
    is
principal; that finiteness step is the only classical ingredient and is left implicit
    here -- this
constructive core captures the content.

` subseteq `: any `Z  in  x` satisfies `X  subseteq  Z` by minimality, so `Z  in  X`. `
    supseteq `: `Z  in  X` means `Z  in  ?` and
`X  subseteq  Z`, so `Z  in  x` by upward closure from `X  in  x`. -/
theorem eq_principal_of_isMin (x : V.Element) {X : Set a} (hX : x.mem X)
    (hmin : forall  Y, x.mem Y  ->  X  subseteq  Y) : x = V.principal (x.sub hX) := by
  apply Element.ext
  intro Z
  constructor
  * intro hZ
    exact <x.sub hZ, hmin Z hZ>
  * rintro <hZmem, hXZ>
    exact x.up_mem hX hZmem hXZ
end

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.ExampleB
section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem

/- concordance: source L317 FACTOID 1.8d union of finite approximations -/
```

```
/-- **Factoid (Scott 1981, PRG-19).** "`x = ?_n sigma_nFalse`": every `x  in  |B|` is
    the union of the finite
elements `sigmaFalse` whose generating cone `sigmaSigma*` lies in `x`. In membership
    form, a neighbourhood `Z`
belongs to `x` iff `Z` lies in some `sigmaFalse = (sigmaSigma*)` with `sigmaSigma*  in
     x`.

This is the concrete "an element is uniquely determined by its finite approximations"
    in `|B|`
(`Basic.eq_iUnion_principal` specialized to `B`, where every neighbourhood is a cone).
    The further
arrangement of the `sigma` into a single increasing chain `sigma_0 ? sigma_1 ? ...`
    requires choice/enumeration and
is left to Scott's prose. -/
theorem mem_iff_exists_sigmaBot (x : B.Element) {Z : Set Str} :
    x.mem Z  <->  exists  sigma, x.mem (cone sigma)  /\  (sigmaBot sigma).mem Z := by
  constructor
  * intro hZ
    obtain <sigma, rfl> := x.sub hZ
    exact <sigma, hZ, x.sub hZ, subset_rfl>
  * rintro <sigma, hcone, hZmem, hsub>
    exact x.up_mem hcone hZmem hsub

/-- **Prepending is associative.** `sigma(tauX) = (sigmatau)X`: prepending `sigma`
    after `tau` is the same as
prepending the concatenation `sigma ++ tau` in one step (used to compose `sigmaElt`, `
    Exercise216`). -/
theorem prepend_append (sigma tau : Str) (X : Set Str) :
    prepend sigma (prepend tau X) = prepend (sigma ++ tau) X := by
  ext w
  simp only [mem_prepend]
  constructor
  * rintro <?, <?', h?', rfl>, rfl>
    exact <?', h?', by rw [List.append_assoc]>
  * rintro <?, h?, rfl>
    exact <tau ++ ?, <?, h?, rfl>, by rw [List.append_assoc]>

/-- **Prepending is monotone.** `X'  subseteq  X  ->  sigmaX'  subseteq  sigmaX` (
    Exercise 2.16's `sigmaMap` uses this for
its `mono` clause; `sigmaElt_append` uses it to transport the up-closure through
    composition). -/
theorem prepend_mono (sigma : Str) {X X' : Set Str} (h : X'  subseteq  X) : prepend
    sigma X'  subseteq  prepend sigma X := by
  rintro w <tau, htau, rfl>
  exact <tau, h htau, rfl>

/-- `sigmaSigma* = sigma*Sigma*`: prepending `sigma` to the whole space recovers the
    cone of `sigma`. -/
theorem prepend_univ (sigma : Str) : prepend sigma Set.univ = cone sigma := by
  ext w
  simp only [mem_prepend, Set.mem_univ, true_and, mem_cone]
  constructor
  * rintro <tau, rfl>
    exact List.prefix_append sigma tau
  * rintro <t, ht>
    exact <t, ht.symm>
```

```
/- concordance: source L312 FACTOID 1.8c prefix order -/
/-- **Factoid (Scott 1981, PRG-19).** "`sigma_0False  subseteq  sigma_1False` if and
    only if `sigma_0` is an *initial segment* of
the sequence `sigma_1`." The approximation order on finite elements is exactly the
    prefix order:
`sigma_0False  sqsub  sigma_1False  <->  sigma_0 <+: sigma_1`. (Via `principal_le_iff`
    -- reversal -- composed with `cone_subset_cone` --
reversal again -- which cancel to give the prefix order directly.) -/
theorem sigmaBot_le_iff (sigma_0 sigma_1 : Str) :
    sigmaBot sigma_0  <=  sigmaBot sigma_1  <->  sigma_0 <+: sigma_1 := by
  rw [sigmaBot, sigmaBot, B.principal_le_iff, cone_subset_cone]

/-- **`sigma` and `tau` prepend in sequence, like concatenation.** `(sigmatau)x = sigma
    (taux)`: applying the
prefix `sigma ++ tau` in one step agrees with applying `tau` first and then `sigma`. (
    Used in Exercise 2.16's
second half -- the uniqueness of the parity map -- to unfold `f(sigma_1sigma_2 ...
    sigma_nx)` one token at a time.) -/
theorem sigmaElt_append (sigma tau : Str) (x : B.Element) :
    sigmaElt (sigma ++ tau) x = sigmaElt sigma (sigmaElt tau x) := by
  apply Element.ext
  intro Y
  constructor
  * rintro <hY, X, hX, hsub>
    refine <hY, prepend tau X, <memB_prepend tau (x.sub hX), X, hX, subset_rfl>, ?_>
    rw [prepend_append]; exact hsub
  * rintro <hY, X, <_, X', hX', hsub'>, hsub>
    refine <hY, X', hX', ?_>
    rw [ <-  prepend_append]
    exact (prepend_mono sigma hsub').trans hsub

/-- **`sigmaFalse` really is `sigma` applied to `False`.** `sigmaElt sigma False =
    sigmaBot sigma`, justifying the `sigmaFalse`
notation: applying `sigma` to the least element produces the finite element `(
    sigmaSigma*)`. -/
theorem sigmaElt_bot (sigma : Str) : sigmaElt sigma B.bot = sigmaBot sigma := by
  apply Element.ext
  intro Y
  constructor
  * rintro <hY, X, hX, hsub>
    rw [B.mem_bot] at hX
    subst hX
    rw [B_master, prepend_univ] at hsub
    exact <hY, hsub>
  * rintro <hY, hcone>
    refine <hY, B.master, B.mem_bot.mpr rfl, ?_>
    rw [B_master, prepend_univ]
    exact hcone

/-- **Prepending the empty prefix fixes every element.** `?x = x` -- the identity case
    of
`sigmaElt`, matching `prepend_nil` on neighbourhoods. -/
theorem sigmaElt_nil (x : B.Element) : sigmaElt [] x = x := by
  apply Element.ext
  intro Y
```

```
  constructor
  * rintro <hY, X, hX, hsub>
    rw [prepend_nil] at hsub
    exact x.up_mem hX hY hsub
  * intro hY
    exact <x.sub hY, Y, hY, by rw [prepend_nil]>
end

end DomainTheory.NeighbourhoodSystems.ExampleB
end
```

3.2.15 Examples 1.2–1.5 (Revisited) — finite examples revisited

EXAMPLES 1.2 – 1.5 (Revisited). The examples as given were all finite, so any explicitly given filter x is principal, the element is finite, the minimal $X \in x$ tells us all we need to know. In such simple situations there is essentially no difference between elements and neighbourhoods — except for the reversal of the order as noted. This (necessary) reversal should not, however, become a matter of confusion: the smaller the neighbourhood has become, the more it has "converged", and so the better defined the element has become. In the approximation relation the "poorer" elements are placed below the "better" with the total up at the top. This will become clearer in discussing "infinite" elements.

Example 1.3 will be generalized in Exercise 1.12. Let us here generalize first 1.4. We let

$$\Delta = \Sigma^*,$$

where $\Sigma = \{0, 1\}$ and Σ^* means the set of all finite sequences of 0's and 1's, with Λ being the empty sequence. We write $\sigma\tau$ for the *concatenation* or *juxtaposition* of two sequences $\sigma, \tau \in \Sigma^*$. Define

$$B = \{\sigma\Sigma^* \mid \sigma \in \Sigma^*\}, \text{ where}$$

$$\sigma X = \{\sigma\tau \mid \tau \in X\},$$

for an arbitrary set $X \subseteq \Sigma^*$. In other words, a neighbourhood in B consists of all *extensions* of a given sequence σ. (Refer back to the finite version of 1.4.) We use the letter "B" to remind us of "binary", and this is an example we shall refer to many times. The proof (if it is not obvious) that B is a neighbourhood system should be done as an exercise.

What do we find in $|B|$? Of course $\perp = \{\Delta\} \in |B|$. For any $x \in |B|$ and $\sigma \in \Sigma^*$ define

$$\sigma x = \{Y \mid \sigma X \subseteq Y \text{ some } X \in x\}.$$

Examples_1_2_1_5_revisited.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Example_1_2
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Example_1_3
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Example_1_4
```

```
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Example_1_5

@[expose] public section

/- concordance: source L284 EXAMPLES 1.2-1.5 finite examples revisited -/

section
namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem
section

variable {a : Type*} (V : NeighborhoodSystem a)

/-- **Examples 1.2-1.5 revisited.** A filter with a ` subseteq `-least neighbourhood `X
    ` is the principal
filter `X`: the least neighbourhood determines the element. -/
theorem eq_principal_of_minimum (x : V.Element) {X : Set a} (hX : x.mem X)
    (hmin : forall  Y, x.mem Y  ->  X  subseteq  Y) : x = V.principal (x.sub hX) := by
  apply Element.ext
  intro Z
  constructor
  * intro hZ
    exact <x.sub hZ, hmin Z hZ>
  * rintro <hZ, hXZ>
    exact x.up_mem hX hZ hXZ

/-- On a finite token set every element has a ` subseteq `-least neighbourhood: descend
     on cardinality
from `Delta`, and close the least candidate under intersection with an arbitrary member
    . -/
theorem exists_minimum_of_fintype [Fintype a] (x : V.Element) :
    exists  X, x.mem X  /\  forall  Y, x.mem Y  ->  X  subseteq  Y := by
  classical
  suffices aux : forall  n (A : Set a), A.ncard = n  ->  x.mem A  ->
      exists  X, x.mem X  /\  X  subseteq  A  /\  forall  Y, x.mem Y  ->  Y  subseteq
     A  ->  X  subseteq  Y from by
    obtain <X, hX, _, hmin> := aux V.master.ncard V.master rfl x.master_mem
    exact <X, hX, fun Y hY => hmin Y hY (V.sub_master (x.sub hY))>
  intro n
  induction n using Nat.strong_induction_on with
  | h n ih =>
    intro A hn hA
    by_cases hleast : forall  Y, x.mem Y  ->  Y  subseteq  A  ->  A  subseteq  Y
    * exact <A, hA, subset_rfl, hleast>
    * push Not at hleast
      obtain <Y, hY, hYA, hAY> := hleast
      have hss : Y ? A := <hYA, hAY>
      have hlt : Y.ncard < n := by
        rw [ <-  hn]
        exact Set.ncard_lt_ncard hss
      obtain <X, hX, hXsub, hmin> := ih Y.ncard hlt Y rfl hY
      refine <X, hX, hXsub.trans hYA, ?_>
      intro Z hZ hZA
      have hcap : x.mem (Z  I  Y) := x.inter_mem hZ hY
      exact (hmin (Z  I  Y) hcap Set.inter_subset_right).trans Set.inter_subset_left
```

```
/-- **Examples 1.2–1.5 revisited.** "The examples as given were all finite, so any
    explicitly
given filter `x` is principal ... the minimal `X  in  x` tells us all we need to know."
     -/
theorem element_eq_principal_of_fintype [Fintype a] (x : V.Element) :
    exists  X, exists  hX : V.mem X, x = V.principal hX := by
  obtain <X, hX, hmin> := V.exists_minimum_of_fintype x
  exact <X, x.sub hX, V.eq_principal_of_minimum x hX hmin>

/-- **Examples 1.2–1.5 revisited.** The order reverses: a smaller neighbourhood
    determines a
better element. `X  subseteq  Y` means `Y  sqsub  X`. -/
theorem smaller_neighbourhood_le {X Y : Set a} (hX : V.mem X) (hY : V.mem Y) :
    V.principal hY  <=  V.principal hX  <->  X  subseteq  Y := by
  constructor
  * intro h
    exact (h Y <hY, subset_rfl>).2
  * intro hXY Z hZ
    exact <hZ.1, hXY.trans hZ.2>
end

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.Example12
/-- Every element of Example 1.2 is the principal filter of its least neighbourhood.
    -/
theorem element_eq_principal (x : neighborhoodSystem.Element) :
    exists  X, exists  hX : neighborhoodSystem.mem X, x = neighborhoodSystem.principal
     hX :=
  NeighborhoodSystem.element_eq_principal_of_fintype neighborhoodSystem x

end DomainTheory.NeighbourhoodSystems.Example12
namespace DomainTheory.NeighbourhoodSystems.Example13
/-- Every element of Example 1.3 is the principal filter of its least neighbourhood.
    -/
theorem element_eq_principal (x : neighborhoodSystem.Element) :
    exists  X, exists  hX : neighborhoodSystem.mem X, x = neighborhoodSystem.principal
     hX :=
  NeighborhoodSystem.element_eq_principal_of_fintype neighborhoodSystem x

end DomainTheory.NeighbourhoodSystems.Example13
namespace DomainTheory.NeighbourhoodSystems.Example14
/-- Every element of the finite tree (Example 1.4) is the principal filter of its
    least
neighbourhood. The infinite system `B` below is a different example. -/
theorem element_eq_principal (x : neighborhoodSystem.Element) :
    exists  X, exists  hX : neighborhoodSystem.mem X, x = neighborhoodSystem.principal
     hX :=
  NeighborhoodSystem.element_eq_principal_of_fintype neighborhoodSystem x

end DomainTheory.NeighbourhoodSystems.Example14
namespace DomainTheory.NeighbourhoodSystems.Example15
/-- Every element of Example 1.5 is the principal filter of its least neighbourhood.
    -/
theorem element_eq_principal (x : neighborhoodSystem.Element) :
```

```
    exists  X, exists  hX : neighborhoodSystem.mem X, x = neighborhoodSystem.principal
     hX :=
  NeighborhoodSystem.element_eq_principal_of_fintype neighborhoodSystem x

end DomainTheory.NeighbourhoodSystems.Example15
namespace DomainTheory.NeighbourhoodSystems.ExampleB
section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem

/-- The token type `Sigma* = List Bool` (finite binary strings); `? = []`. -/
abbrev Str := List Bool

/-- The neighbourhood `sigmaSigma*`: all *extensions* of `sigma` (sequences with `sigma
    ` as an initial segment). -/
def cone (sigma : Str) : Set Str := {w | sigma <+: w}

@[simp] theorem mem_cone {sigma w : Str} : w  in  cone sigma  <->  sigma <+: w := Iff.
    rfl

/-- `?Sigma* = Sigma*`: the cone of the empty string is everything (Scott's `Delta`).
    -/
theorem cone_nil : cone [] = Set.univ := by
  ext w; simp [cone]

/-- **Cones reverse the prefix order.** `sigmaSigma*  subseteq  tauSigma*` iff `tau` is
     an initial segment of `sigma`: a longer
prefix carves out a *smaller* cone. (` -> ` tests at `sigma  in  sigmaSigma*`; ` <- `
    is transitivity of `<+:`.) -/
theorem cone_subset_cone {sigma tau : Str} : cone sigma  subseteq  cone tau  <->  tau
    <+: sigma := by
  constructor
  * intro h
    exact h (show sigma  in  cone sigma from List.prefix_rfl)
  * intro htausigma w hw
    exact htausigma.trans hw

/-- **Prefix trichotomy for cones.** Any two cones are nested-or-disjoint: either one
    contains the
other, or they are disjoint (incomparable prefixes have no common extension). Choice-
    free: the
prefix relation on `List Bool` is decidable. -/
theorem cone_trichotomy (sigma tau : Str) :
    cone sigma  subseteq  cone tau  \/  cone tau  subseteq  cone sigma  \/  cone sigma
      I  cone tau = {} :=
  if hsigmatau : sigma <+: tau then Or.inr (Or.inl (cone_subset_cone.mpr hsigmatau))
  else if htausigma : tau <+: sigma then Or.inl (cone_subset_cone.mpr htausigma)
  else Or.inr (Or.inr (by
    ext w
    simp only [Set.mem_inter_iff, mem_cone, Set.mem_empty_iff_false, iff_false,
    not_and]
    intro h1 h2
    rcases List.prefix_or_prefix_of_prefix h1 h2 with h | h
    * exact hsigmatau h
    * exact htausigma h))
```

```
/-- Membership in Scott's binary neighbourhood system `B`: `X  in  B` iff `X =
    sigmaSigma*` for some `sigma`. -/
def memB (X : Set Str) : Prop := exists  sigma, X = cone sigma

/-- **Exercise ("`B` is a neighbourhood system").** The family `B = {sigmaSigma* ?
    sigma  in  Sigma*}` is pairwise
nested-or-disjoint, by `cone_trichotomy`. -/
theorem nestedOrDisjoint : NestedOrDisjoint memB := by
  rintro X Y <sigma, rfl> <tau, rfl>
  exact cone_trichotomy sigma tau

/-- **Examples 1.2–1.5 revisited -- the infinite form of Example 1.4.**
Scott's `B` on `Delta = Sigma*`. This is a new system: Example 1.4 remains the finite
    tree of sequences
of length at most 2. -/
def B : NeighborhoodSystem Str :=
  NeighborhoodSystem.ofNestedOrDisjoint memB Set.univ
    <[], Set.mem_univ _> <[], cone_nil.symm> nestedOrDisjoint
    (fun _ => Set.subset_univ _)

@[simp] theorem B_mem {X : Set Str} : B.mem X  <->  memB X := Iff.rfl

@[simp] theorem B_master : B.master = Set.univ := rfl

/-- Every cone is a neighbourhood of `B`. -/
theorem memB_cone (sigma : Str) : B.mem (cone sigma) := <sigma, rfl>

/-- Scott's `sigmaX = {sigmatau ? tau  in  X}` (prepend the prefix `sigma` to every
    member of `X`). -/
def prepend (sigma : Str) (X : Set Str) : Set Str := {w | exists  tau, tau  in  X  /\
     w = sigma ++ tau}

@[simp] theorem mem_prepend {sigma : Str} {X : Set Str} {w : Str} :
    w  in  prepend sigma X  <->  exists  tau, tau  in  X  /\  w = sigma ++ tau := Iff.
    rfl

/-- **`sigma(tauSigma*) = (sigmatau)Sigma*`.** Prepending `sigma` to a cone yields the
    cone of the concatenation -- this is
why `sigmax` lands back in `B` and why `sigmaFalse` is again a finite element. -/
theorem prepend_cone (sigma ? : Str) : prepend sigma (cone ?) = cone (sigma ++ ?) :=
    by
  ext w
  simp only [mem_prepend, mem_cone]
  constructor
  * rintro <tau, htau, rfl>
    exact (List.prefix_append_right_inj sigma).mpr htau
  * rintro <t, ht>
    exact <? ++ t, List.prefix_append ? t, by rw [ <-  ht, List.append_assoc]>

/-- Prepending preserves membership in `B` (`sigma` applied to a cone is a cone). -/
theorem memB_prepend (sigma : Str) {X : Set Str} (hX : B.mem X) : B.mem (prepend sigma
     X) := by
  obtain <?, rfl> := hX
  exact <sigma ++ ?, prepend_cone sigma ?>

@[simp] theorem prepend_nil (X : Set Str) : prepend [] X = X := by
```

```
  ext w; simp [mem_prepend]

/-- **`sigmaFalse`, a finite element of `|B|`.** The principal filter `(sigmaSigma*)`
    of the cone of `sigma`; its
minimal neighbourhood is `sigmaDelta = sigmaSigma*` (Scott). These are exactly the
    finite elements of `|B|`. -/
def sigmaBot (sigma : Str) : B.Element := B.principal (memB_cone sigma)

/-- **Exercise (`sigmax in |B|`).** For `x in |B|` and `sigma in Sigma*`, Scott's
`sigmax = {Y ? sigmaX subseteq Y for some X in x}` is again an element of `|B|`.

The filter laws: `master` uses `X = Delta in x` (`sigmaDelta subseteq Delta`
    trivially); `inter` takes `X_1 I X_2 in x` and
the consistency witness `sigma(X_1 I X_2)`, which is a *cone* (hence in `B`, by `
    memB_prepend`) contained in
both `Y_1` and `Y_2`; `up` reuses the same `X`. -/
def sigmaElt (sigma : Str) (x : B.Element) : B.Element where
  mem Y := B.mem Y /\ exists X, x.mem X /\ prepend sigma X subseteq Y
  sub h := h.1
  master_mem := <B.master_mem, B.master, x.master_mem, Set.subset_univ _>
  inter_mem := by
    intro Y_1 Y_2 h1 h2
    obtain <hY_1, X_1, hX_1, hsub_1> := h1
    obtain <hY_2, X_2, hX_2, hsub_2> := h2
    have hXinter : x.mem (X_1 I X_2) := x.inter_mem hX_1 hX_2
    have hsub : prepend sigma (X_1 I X_2) subseteq Y_1 I Y_2 := by
      rintro w <tau, <htau_1, htau_2>, rfl>
      exact <hsub_1 <tau, htau_1, rfl>, hsub_2 <tau, htau_2, rfl>>
    have hZmem : B.mem (prepend sigma (X_1 I X_2)) := memB_prepend sigma (x.sub
    hXinter)
    exact <B.inter_mem hY_1 hY_2 hZmem hsub, X_1 I X_2, hXinter, hsub>
  up_mem := by
    intro X Y hX hY hXY
    obtain <_, X', hX', hsub'> := hX
    exact <hY, X', hX', hsub'.trans hXY>

/-- **Cones are one-one in the prefix.** `cone sigma = cone tau ? sigma = tau`: from
    the two inclusions we get
`tau <+: sigma` and `sigma <+: tau`, and a prefix-antisymmetry (equal lengths) finishes
    . Used by the approximable
maps `B -> T` / `B -> B` (Examples 2.3, 2.4) to read off the unique generating
    prefix of a cone. -/
theorem cone_injective {sigma tau : Str} (h : cone sigma = cone tau) : sigma = tau :=
    by
  have h1 : tau <+: sigma := cone_subset_cone.mp (le_of_eq h)
  have h2 : sigma <+: tau := cone_subset_cone.mp (le_of_eq h.symm)
  exact h2.eq_of_length (h2.length_le.antisymm h1.length_le)

/-- **`False` sits only in the cone of the empty prefix.** `False.mem(sigmaSigma*) <->
      sigma = ?`: `False`'s only
neighbourhood is `Delta = ?Sigma*` (`cone_nil`), and a cone determines its prefix (`
    cone_injective`).
(Used by Exercise 2.17's base case, where `g(1)=False` is stated only at the input `
    False`.) -/
theorem bot_mem_cone_iff {sigma : Str} : B.bot.mem (cone sigma) <-> sigma = [] := by
  rw [mem_bot, B_master, <- cone_nil]
```

```
  constructor
  * exact cone_injective
  * intro h; rw [h]
end

end DomainTheory.NeighbourhoodSystems.ExampleB
end
```

3.2.16 Definition 1.9 — isomorphic domains

DEFINITION 1.9. Two neighbourhood systems $\mathcal{D}_0$ and $\mathcal{D}_1$ determine *isomorphic domains* iff there is a one-one correspondence between $|\mathcal{D}_0|$ and $|\mathcal{D}_1|$ which preserves inclusion between the elements of the domains. In symbols we write $\mathcal{D}_0 \cong \mathcal{D}_1$. □

It is certain that the property of 1.9 is necessary, but it may not be so clear that it is sufficient. We shall in fact prove in the next lecture that an isomorphism between domains always maps finite elements to finite elements, so it always results from a one-one inclusion-preserving correspondence between neighbourhoods. This is surely as strong as could be hoped. This general result is not needed to see that particular domains are isomorphic.

In some of the examples tokens corresponded to total elements and in some to partial elements; it is not difficult to see (ex post facto) that every domain can be presented with tokens exactly corresponding to partial elements.

DomainIso.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.bot

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
/- concordance: source L326 DEFINITION 1.9 isomorphic domains -/
/-- **Definition 1.9 (Scott 1981, PRG-19).** Two neighbourhood systems `?_0` and `?_1`
    (over possibly
*different* token types) *determine isomorphic domains* iff there is a one-one,
    inclusion-preserving
correspondence between `|?_0|` and `|?_1|`. We package "one-one + preserves inclusion (
    both ways)" as
mathlib's order-isomorphism `?o`: an `OrderIso` is automatically a bijection that *
    reflects* as well
as preserves ` sqsub ` (`map_rel_iff`), which is exactly Scott's requirement. -/
abbrev DomainIso {a b : Type*} (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem
    b) : Type _ :=
  V_0.Element ?o V_1.Element

namespace NeighborhoodSystem
section

variable {a : Type*} (V : NeighborhoodSystem a)

/-- **Definition 1.8 (Scott 1981, PRG-19) -- *total* elements.** "Elements maximal
    with respect
```

```
to the approximation relation are called *total elements*." `x` is total iff it is
    maximal: any
`y` it approximates approximates it back. This is the *predicate* only; the *existence*
     of total
elements above a given `x` (Exercise 1.24) is choice-dependent and out of scope here.
    -/
def IsTotal (x : V.Element) : Prop := forall  y, x  <=  y  ->  y  <=  x
end

end NeighborhoodSystem
/-- Scott's `?_0  ~=  ?_1`: the domains are isomorphic (there *exists* a `DomainIso`).
    -/
def Isomorphic {a b : Type*} (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b)
     : Prop :=
  Nonempty (DomainIso V_0 V_1)

@[inherit_doc] infix:25 "  ~= ? " => Isomorphic

/-- ` ~= ?` is reflexive (`OrderIso.refl`). -/
theorem Isomorphic.refl {a : Type*} (V : NeighborhoodSystem a) : V  ~= ? V :=
  <OrderIso.refl _>

/-- ` ~= ?` is symmetric (`OrderIso.symm`). -/
theorem Isomorphic.symm {a b : Type*} {V_0 : NeighborhoodSystem a} {V_1 :
    NeighborhoodSystem b}
    (h : V_0  ~= ? V_1) : V_1  ~= ? V_0 :=
  h.elim fun e => <e.symm>

/-- ` ~= ?` is transitive (`OrderIso.trans`). -/
theorem Isomorphic.trans {a b ? : Type*} {V_0 : NeighborhoodSystem a} {V_1 :
    NeighborhoodSystem b}
    {V_2 : NeighborhoodSystem ?} (h_0 : V_0  ~= ? V_1) (h_1 : V_1  ~= ? V_2) : V_0  ~=
     ? V_2 :=
  h_0.elim fun e_0 => h_1.elim fun e_1 => <e_0.trans e_1>

end DomainTheory.NeighbourhoodSystems
end
```

3.2.17 Theorem 1.10 — basic open neighbourhoods

THEOREM 1.10. Given any neighbourhood system $\mathcal{D}$, define for $X \in \mathcal{D}$

$$[X] = \{x \in |\mathcal{D}| \mid X \in x\}.$$

The subsets $[X] \subseteq |\mathcal{D}|$ for $X \in \mathcal{D}$ form a neighbourhood system over $|\mathcal{D}|$ which determines a domain isomorphic to $|\mathcal{D}|$.

Proof: We note first that

(1) $[\Delta] = |\mathcal{D}|$.

Next note that

(2) X, Y are consistent in $\mathcal{D}$ iff $[X] \cap [Y] \neq \emptyset$;

and that for $X, Y \in \mathcal{D}$

(3) $[X] \cap [Y] = [X \cap Y]$ if $X \cap Y \in \mathcal{D}$.

Inasmuch as

(4) $\uparrow X \in [X]$ for all $X \in \mathcal{D}$,

it easily follows that $\mathcal{D}$ and the family

$$\{[X] \mid X \in \mathcal{D}\}$$

are in a one-one, inclusion-preserving correspondence. Thus, we can induce the desired one-one correspondence between the elements of the two systems. □

The import of 1.10 is that the original tokens in Δ can be replaced by the elements of $|\mathcal{D}|$. This process replaces the neighbourhood $X \subseteq \Delta$ by the subset $[X] \subseteq |\mathcal{D}|$. As the passage is inclusion preserving, the domain has not really changed, only its presentation. Though of some theoretical charm, the theorem is not of much use since we still have to get $\mathcal{D}$ from somewhere. It does emphasize, though, that the rôle of the tokens is simply to keep the inclusions (and intersections) of neighbourhoods sorted out. It is *not* always true that the tokens can be identified with the total elements.

The last theorem in this lecture is a result on *closure properties* of a domain with respect to set-theoretical operations which have interesting meanings with respect to approximation.

Theorem_1_10.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_27
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.DomainIso
import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.bot
public import Mathlib.Algebra.Group.End

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem
section

variable {a : Type*} (V : NeighborhoodSystem a)

/- concordance: source L334 THEOREM 1.10 basic open neighbourhoods -/
/-- **Theorem 1.10.** The *element-token system* `{[X] ? X  in  ?}` over the tokens
    `|?|`. The two
laws reduce to facts about `[*]`: the master `[Delta] = |?|` is the whole space; the
    consistency
witness `[W]` for `[X]  I  [Y]` yields `W  subseteq  X  I  Y` (via `W`), so `X  I  Y  in
     ?` and
`[X]  I  [Y] = [X  I  Y]`. -/
def tokenSystem : NeighborhoodSystem V.Element where
  mem S := exists  X, V.mem X  /\  S = V.bracket X
  master := Set.univ
```

```
  master_nonempty := <V.bot, Set.mem_univ _>
  master_mem := <V.master, V.master_mem, V.bracket_master.symm>
  inter_mem := by
    rintro S T W <X, hX, rfl> <Y, hY, rfl> <Z, hZ, rfl> hWsub
    have hpz := hWsub (V.principal_mem_bracket hZ)
    obtain <hzX, hzY> := hpz
    have hXY : V.mem (X  I  Y) := V.inter_mem hX hY hZ (Set.subset_inter hzX.2 hzY.2)
    exact <X  I  Y, hXY, V.bracket_inter hX hY>
  sub_master := fun _ => Set.subset_univ _

@[simp] theorem tokenSystem_mem {S : Set V.Element} :
    V.tokenSystem.mem S  <->  exists  X, V.mem X  /\  S = V.bracket X := Iff.rfl

/-- The element of `|?|` corresponding to `y  in  |{[X]}|`: the filter `{X ? [X]  in  y
    }`. -/
def ofToken (y : V.tokenSystem.Element) : V.Element where
  mem X := V.mem X  /\  y.mem (V.bracket X)
  sub h := h.1
  master_mem := <V.master_mem, by rw [V.bracket_master]; exact y.master_mem>
  inter_mem := by
    rintro X Y <hX, hyX> <hY, hyY>
    have hyInter : y.mem (V.bracket X  I  V.bracket Y) := y.inter_mem hyX hyY
    obtain <W, hW, hWeq> := y.sub hyInter
    have hpw := V.principal_mem_bracket hW
    rw [ <-  hWeq] at hpw
    obtain <hwX, hwY> := hpw
    have hXY : V.mem (X  I  Y) := V.inter_mem hX hY hW (Set.subset_inter hwX.2 hwY.2)
    refine <hXY, ?_>
    rw [ <-  V.bracket_inter hX hY]
    exact hyInter
  up_mem := by
    rintro X Y <hX, hyX> hY hXY
    exact <hY, y.up_mem hyX <Y, hY, rfl> ((V.bracket_subset_iff hX hY).mpr hXY)>

/-- The element of `|{[X]}|` corresponding to `x  in  |?|`: the filter `{[X] ? X  in  x
    }`. -/
def toToken (x : V.Element) : V.tokenSystem.Element where
  mem S := exists  X, x.mem X  /\  S = V.bracket X
  sub := by rintro S <X, hX, rfl>; exact <X, x.sub hX, rfl>
  master_mem := <V.master, x.master_mem, V.bracket_master.symm>
  inter_mem := by
    rintro S T <X, hX, rfl> <Y, hY, rfl>
    exact <X  I  Y, x.inter_mem hX hY, V.bracket_inter (x.sub hX) (x.sub hY)>
  up_mem := by
    rintro S T <X, hX, rfl> hT hsub
    obtain <Y, hYmem, rfl> := hT
    exact <Y, x.up_mem hX hYmem ((V.bracket_subset_iff (x.sub hX) hYmem).mp hsub), rfl
    >

/-- **Theorem 1.10 (the isomorphism).** `X  |->  [X]` induces an order-isomorphism `|?|
     ?o |{[X]}|`:
`toToken` and `ofToken` are mutually inverse and preserve/reflect ` sqsub `. -/
def tokenIso : V.Element ?o V.tokenSystem.Element where
  toFun := V.toToken
  invFun := V.ofToken
  left_inv := by
```

```
    intro x
    apply Element.ext
    intro X
    constructor
    * rintro <hX, X', hX', heq>
      rw [V.bracket_injective hX (x.sub hX') heq]; exact hX'
    * intro hx
      exact <x.sub hx, X, hx, rfl>
  right_inv := by
    intro y
    apply Element.ext
    intro S
    constructor
    * rintro <X, <_, hyX>, rfl>
      exact hyX
    * intro hyS
      obtain <X, hX, rfl> := y.sub hyS
      exact <X, <hX, hyS>, rfl>
  map_rel_iff' := by
    intro x x'
    constructor
    * intro h X hx
      have hmem : (V.toToken x).mem (V.bracket X) := <X, hx, rfl>
      obtain <X', hX', heq> := h (V.bracket X) hmem
      rw [V.bracket_injective (x.sub hx) (x'.sub hX') heq]; exact hX'
    * intro h S hS
      obtain <X, hX, rfl> := hS
      exact <X, h X hX, rfl>

/-- **Theorem 1.10 (2).** `X, Y` are consistent in `?` (have a common lower bound `Z
    in  ?`,
`Z  subseteq  X  I  Y`) iff `[X]  I  [Y]  !=  {}`. ` -> ` uses `Z` (which lies in both
    `[X]` and `[Y]`); ` <- ` reads a
witness off any common element. -/
theorem bracket_inter_nonempty_iff {X Y : Set a} (hX : V.mem X) (hY : V.mem Y) :
    (V.bracket X  I  V.bracket Y).Nonempty  <->  exists  Z, V.mem Z  /\  Z  subseteq
    X  I  Y := by
  constructor
  * rintro <x, hxX, hxY>
    exact <X  I  Y, x.sub (x.inter_mem hxX hxY), subset_rfl>
  * rintro <Z, hZ, hZsub>
    refine <V.principal hZ, ?_, ?_>
    * exact <hX, hZsub.trans Set.inter_subset_left>
    * exact <hY, hZsub.trans Set.inter_subset_right>

/-- **Theorem 1.10 (statement).** Any neighbourhood system is isomorphic to its
    element-token
system: `?  ~= ? {[X]}`. -/
theorem isomorphic_tokenSystem : V  ~= ? V.tokenSystem := <V.tokenIso>
end

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
section
```

```
set_option linter.unusedVariables false

theorem theorem_110.{u_1} : And
  (And
    (And
      (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem a) {X Y : Set a},
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X  ->
          @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V Y  ->
            Iff
              (@Set.Nonempty (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    Element a V)
                (@Inter.inter (Set (@DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.Element a V))
                  (@Set.instInter (@DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.Element a V))
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bracket a V X
    )
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bracket a V Y
    )))
              (exists  (Z : Set a),
                And (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V Z)
                  (@LE.le (Set a) (@Set.instLE a) Z (@Inter.inter (Set a) (@Set.
    instInter a) X Y))))
      (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem a) {X Y : Set a},
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X  ->
          @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V Y  ->
            @Eq (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
    V))
              (@Inter.inter (Set (@DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.Element a V))
                (@Set.instInter (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
    .Element a V))
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bracket a V X)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bracket a V Y))
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bracket a V
                (@Inter.inter (Set a) (@Set.instInter a) X Y))))
     (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
    a),
       @DomainTheory.NeighbourhoodSystems.Isomorphic a
         (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V) V
         (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.tokenSystem a V)))
  (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a)
     {X : Set a}
    (hX : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X),
    @Membership.mem (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V
    )
      (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V))
      (@Set.instMembership (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    Element a V))
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bracket a V X)
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.principal a V X hX)) :=
  @And.intro
  (And
    (And
```

```
   (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem a) {X Y : Set a},
     @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X  ->
       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V Y  ->
         Iff
           (@Set.Nonempty (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 Element a V)
             (@Inter.inter (Set (@DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem.Element a V))
               (@Set.instInter (@DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem.Element a V))
               (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bracket a V X
 )
               (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bracket a V Y
 )))
           (exists  (Z : Set a),
             And (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V Z)
               (@LE.le (Set a) (@Set.instLE a) Z (@Inter.inter (Set a) (@Set.
 instInter a) X Y))))
   (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem a) {X Y : Set a},
     @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X  ->
       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V Y  ->
         @Eq (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
 V))
           (@Inter.inter (Set (@DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem.Element a V))
             (@Set.instInter (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
 .Element a V))
             (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bracket a V X)
             (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bracket a V Y))
           (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bracket a V
             (@Inter.inter (Set a) (@Set.instInter a) X Y))))
  (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
  a),
    @DomainTheory.NeighbourhoodSystems.Isomorphic a
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V) V
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.tokenSystem a V)))
(forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a)
   {X : Set a}
  (hX : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X),
  @Membership.mem (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V
  )
    (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V))
    (@Set.instMembership (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 Element a V))
    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bracket a V X)
    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.principal a V X hX))
(@And.intro
  (And
    (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem a) {X Y : Set a},
     @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X  ->
       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V Y  ->
         Iff
           (@Set.Nonempty (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
```

```
Element a V)
             (@Inter.inter (Set (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V))
               (@Set.instInter (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V))
               (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bracket a V X
)
               (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bracket a V Y
)))
           (exists  (Z : Set a),
             And (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V Z)
               (@LE.le (Set a) (@Set.instLE a) Z (@Inter.inter (Set a) (@Set.
instInter a) X Y))))
   (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a) {X Y : Set a},
     @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X  ->
       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V Y  ->
         @Eq (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
V))
           (@Inter.inter (Set (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V))
             (@Set.instInter (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
.Element a V))
             (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bracket a V X)
             (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bracket a V Y))
           (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bracket a V
             (@Inter.inter (Set a) (@Set.instInter a) X Y))))
(forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
a),
  @DomainTheory.NeighbourhoodSystems.Isomorphic a
    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V) V
    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.tokenSystem a V))
(@And.intro
  (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a) {X Y : Set a},
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X  ->
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V Y  ->
        Iff
          (@Set.Nonempty (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a V)
            (@Inter.inter (Set (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V))
              (@Set.instInter (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V))
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bracket a V X
)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bracket a V Y
)))
          (exists  (Z : Set a),
            And (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V Z)
              (@LE.le (Set a) (@Set.instLE a) Z (@Inter.inter (Set a) (@Set.
instInter a) X Y))))
  (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a) {X Y : Set a},
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X  ->
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V Y  ->
```

```
          @Eq (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
    V))
            (@Inter.inter (Set (@DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.Element a V))
              (@Set.instInter (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
    .Element a V))
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bracket a V X)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bracket a V Y))
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bracket a V
              (@Inter.inter (Set a) (@Set.instInter a) X Y)))
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bracket_inter_nonempty_iff
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bracket_inter)
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.isomorphic_tokenSystem)
  @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.principal_mem_bracket
end

end DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem
section

variable {a : Type*} (V : NeighborhoodSystem a)

/-- A neighbourhood system `?'` over `b` is **complete** when every filter `y  in
    |?'|` is *fixed
by a unique point* `b  in  b`: a neighbourhood `S` lies in `y` iff `b  in  S`. (Scott:
    "*a filter is
fixed by a point iff it is the filter of all neighbourhoods containing that point*".)
    -/
def IsComplete {b : Type*} (V' : NeighborhoodSystem b) : Prop :=
  forall  y : V'.Element, exists ! b : b, forall  S, V'.mem S  ->  (y.mem S  <->  b
    in  S)

/-- The filter of `{[X]}` *fixed by* the point `x  in  |?|` is `toToken x` (the filter
    of all
brackets containing `x`). By Theorem 1.10 this is a bijection `|?| ? |{[X]}|`, so
    tokens and
(partial) elements are identified. -/
def filterFixedBy (x : V.Element) : V.tokenSystem.Element := V.toToken x

/-- `[X]  !=  {} ? X  in  ?`. ` -> ` reads `X  in  ?` off any element of `[X]`; ` <- `
    exhibits `X`. -/
theorem bracket_nonempty_iff {X : Set a} : (V.bracket X).Nonempty  <->  V.mem X := by
  constructor
  * rintro <x, hx>; exact x.sub hx
  * intro hX; exact <V.principal hX, V.principal_mem_bracket hX>
end

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
end
```

3.2.18 Theorem 1.11 — intersections and directed unions

THEOREM 1.11. If $\mathcal{D}$ is a neighbourhood system and $x_n \in |\mathcal{D}|$ for $n = 0, 1, 2, \ldots$, then

(i) $\bigcap_{n=0}^{\infty} x_n \in |\mathcal{D}|$; and

(ii) $\bigcup_{n=0}^{\infty} x_n \in |\mathcal{D}|$, provided

$$x_0 \subseteq x_1 \subseteq x_2 \subseteq \cdots \subseteq x_n \subseteq x_{n+1} \subseteq \cdots .$$

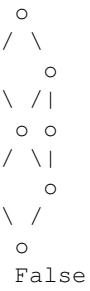
Proof: The conditions of 1.6 have to be checked. For the case of intersection, all of 1.6(i)–(iii) are quite obvious. For the case of union, only 1.6(ii) gives pause and it requires the proviso. If X and Y belong to the union, then $X \in x_n$, say, and $Y \in x_m$. But, either $n < m$ or $m < n$, and if $k = \max(n, m)$, then $X, Y \in x_k$. Since $x_k \in |\mathcal{D}|$, we have $X \cap Y \in x_k$; thus, $X \cap Y$ belongs to the union. This proves (ii). □

In words, the intersection is the best element that is at the same time an approximation to all of the elements x_n; the intersection is exactly what is common to all the given elements. The union on the other hand is just what the (increas-

The union on the other hand is just what the (increasing sequence of the) x_n approximates; the union combines contributions from all the x_n into a "better" element – but no more than that.

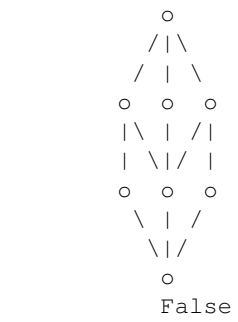
In thinking about domains a rough diagram of the partial-ordering relation $\subseteq$ between elements is often helpful. The picture of 1.4 is an example where the nodes represent the elements. Any finite tree growing up from a root node would also be an example. Indeed, any finite partially ordered set with least element would be an example. (Here no distinction between tokens and elements is necessary.) A lattice diagram is also illustrated.

A TREE



A ROUGH PICTURE

A LATTICE



The root node is the element $\bot$ of $|\mathcal{D}|$; there need be no top node $\top$. *Approximation* is represented by a passage from a lower node to a higher node along the rising lines. The system $\mathcal{D}$ of neighbourhoods is the collection of sets each of which is all

the nodes above a given node. For infinite examples, however, care must be given to introduce limit nodes. The first few exercises should be provided with pictures to illustrate the structure.

Theorem_1_11.lean

```
module

import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.Factoid_1_8
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Examples_1_2_1_5_revisited

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem
section

variable {a : Type*} (V : NeighborhoodSystem a)

/- concordance: source L372 THEOREM 1.11 intersections and directed unions -/
/-- **Theorem 1.11 (i).** The intersection `?_n x_n = {X ? forall  n, X  in  x_n}` of a
     sequence of elements is
again an element of `|?|`. (No proviso: all of 1.6(i)-(iii) are pointwise.) -/
def iInter (x : Nat  ->  V.Element) : V.Element where
  mem X := forall  n, (x n).mem X
  sub h := (x 0).sub (h 0)
  master_mem n := (x n).master_mem
  inter_mem h1 h2 n := (x n).inter_mem (h1 n) (h2 n)
  up_mem h hY hXY n := (x n).up_mem (h n) hY hXY

@[simp] theorem mem_iInter (x : Nat  ->  V.Element) {X : Set a} :
    (V.iInter x).mem X  <->  forall  n, (x n).mem X := Iff.rfl

/-- **Theorem 1.11 (ii).** For an **ascending** sequence `x_0  sqsub  x_1  sqsub  ?`,
    the union
`?_n x_n = {X ? exists  n, X  in  x_n}` is again an element of `|?|`. The proviso (`
    Monotone x`) is used only in
the intersection law: `X  in  x_n`, `Y  in  x?` ? both in `x_{max n m}`. -/
def iUnion (x : Nat  ->  V.Element) (hmono : Monotone x) : V.Element where
  mem X := exists  n, (x n).mem X
```

```
  sub := by rintro X <n, hn>; exact (x n).sub hn
  master_mem := <0, (x 0).master_mem>
  inter_mem := by
    rintro X Y <n, hn> <m, hm>
    refine <max n m, (x (max n m)).inter_mem ?_ ?_>
    * exact hmono (le_max_left n m) X hn
    * exact hmono (le_max_right n m) Y hm
  up_mem := by
    rintro X Y <n, hn> hY hXY
    exact <n, (x n).up_mem hn hY hXY>

@[simp] theorem mem_iUnion (x : Nat  ->  V.Element) (hmono : Monotone x) {X : Set a} :
    (V.iUnion x hmono).mem X  <->  exists  n, (x n).mem X := Iff.rfl

/-- `?_n x_n  sqsub  x_n`: the intersection approximates every member of the sequence.
    -/
theorem iInter_le (x : Nat  ->  V.Element) (n : Nat) : V.iInter x  <=  x n :=
  fun _ h => h n

/-- `?_n x_n` is the **least** upper bound: anything approximated by all `x_n`
    approximates the
union from above. -/
theorem iUnion_le (x : Nat  ->  V.Element) (hmono : Monotone x) (y : V.Element)
    (h : forall  n, x n  <=  y) : V.iUnion x hmono  <=  y := by
  rintro X <n, hn>
  exact h n X hn
end

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
section

set_option linter.unusedVariables false

theorem theorem_111.{u_1} : forall  {a : Type u_1} (V : DomainTheory.
    NeighbourhoodSystems.NeighborhoodSystem a)
  (x : Nat  ->  @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
  (hmono :
    @Monotone Nat (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
    Nat.instPreorder
      (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
    .Element a V)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.instPartialOrderElement
     a V))
      x)
  (y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V),
  (forall  (n : Nat),
      @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
        (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
     a V)
          (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.Element a V)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    instPartialOrderElement a V)))
        (x n) y)  ->
```

```
    @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
      (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
     V)
        (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.Element a V)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    instPartialOrderElement a V)))
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.iUnion a V x hmono) y :=
  @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.iUnion_le
end

end DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
namespace DomainTheory.NeighbourhoodSystems.Exercise113
section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ExampleB

/-- The length-`n` initial segment `p ? n = <p 0, ..., p (n-1)>` of an infinite path `p
    `. -/
def prefixSeq (p : Nat  ->  Bool) : Nat  ->  Str
  | 0 => []
  | (n + 1) => prefixSeq p n ++ [p n]

@[simp] theorem prefixSeq_length (p : Nat  ->  Bool) (n : Nat) : (prefixSeq p n).
    length = n := by
  induction n with
  | zero => rfl
  | succ k ih => simp [prefixSeq, ih]

/-- The finite approximations `sigma_nFalse = (p ? n)False` along the path `p`. -/
def branchSeq (p : Nat  ->  Bool) (n : Nat) : B.Element := sigmaBot (prefixSeq p n)

theorem prefixSeq_prefix_succ (p : Nat  ->  Bool) (n : Nat) :
    prefixSeq p n <+: prefixSeq p (n + 1) := <[p n], rfl>

theorem prefixSeq_mono (p : Nat  ->  Bool) {n m : Nat} (h : n  <=  m) :
    prefixSeq p n <+: prefixSeq p m := by
  induction m, h using Nat.le_induction with
  | base => exact List.prefix_rfl
  | succ k _ ih => exact ih.trans (prefixSeq_prefix_succ p k)

theorem branchSeq_mono (p : Nat  ->  Bool) : Monotone (branchSeq p) := by
  intro n m h
  exact (sigmaBot_le_iff (prefixSeq p n) (prefixSeq p m)).mpr (prefixSeq_mono p h)

/-- **The limit/total element of an infinite path** `p`: the ascending union `?_n (p ?
    n)False`
(Theorem 1.11(ii)). These are the "limit nodes all along the top" of Scott's picture.
    -/
def branch (p : Nat  ->  Bool) : B.Element := B.iUnion (branchSeq p) (branchSeq_mono p
    )

/-- `cone sigma  in  branch p` iff `sigma` is an initial segment of some `p ? n`, i.e.
    `sigma` lies on the path. -/
theorem branch_mem_iff (p : Nat  ->  Bool) {sigma : Str} :
    (branch p).mem (cone sigma)  <->  exists  n, sigma <+: prefixSeq p n := by
```

```
  constructor
  * intro h
    obtain <n, hn> := (B.mem_iUnion (branchSeq p) (branchSeq_mono p)).mp h
    exact <n, cone_subset_cone.mp hn.2>
  * rintro <n, hn>
    exact (B.mem_iUnion (branchSeq p) (branchSeq_mono p)).mpr
      <n, memB_cone sigma, cone_subset_cone.mpr hn>

/-- Two neighbourhoods present in a common element of `|B|` have comparable generating
    prefixes
(their cones cannot be disjoint, since the intersection is a nonempty cone). -/
theorem prefix_comparable_of_mem {sigma tau : Str} (y : B.Element)
    (hsigma : y.mem (cone sigma)) (htau : y.mem (cone tau)) : sigma <+: tau  \/  tau
    <+: sigma := by
  have hi : y.mem (cone sigma  I  cone tau) := y.inter_mem hsigma htau
  obtain <?, h?> := y.sub hi
  rcases cone_trichotomy sigma tau with h | h | h
  * exact Or.inr (cone_subset_cone.mp h)
  * exact Or.inl (cone_subset_cone.mp h)
  * exfalso
    have hmem : ?  in  cone ? := mem_cone.mpr List.prefix_rfl
    rw [ <-  h?, h] at hmem
    simp at hmem
end

end DomainTheory.NeighbourhoodSystems.Exercise113
end
```

3.2.19 Exercise 1.12 — final segments of integers

Let $\Delta = \mathbb{N} = \{0, 1, 2, \ldots, n, \ldots\}$ be the set of non-negative integers. Use as neighbourhoods final segments:

$$\{m \in \mathbb{N} \mid m > n\}$$

for $n \in \mathbb{N}$. Verify that this is a neighbourhood system. What are the total elements? What are the finite elements? Draw a picture of the approximation relation in this domain. (Hint: there is only one limit element.)

Exercise_1_12.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_7

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.Exercise112

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem

/-- The final segment (tail) `tail n = {m ? n  <=  m}`; the neighbourhood "all integers
     ` >=  n`". -/
```

```
def tail (n : Nat) : Set Nat := {m | n  <=  m}

@[simp] theorem mem_tail {n m : Nat} : m  in  tail n  <->  n  <=  m := Iff.rfl

/-- Membership in the final-segment system: `X` is a neighbourhood iff `X = tail n` for
     some `n`. -/
def mem (X : Set Nat) : Prop := exists  n, X = tail n

/-- `tail n  I  tail m = tail (max n m)` (the intersection of two tails is the shorter
    one). -/
theorem tail_inter (n m : Nat) : tail n  I  tail m = tail (max n m) := by
  ext k
  simp only [Set.mem_inter_iff, mem_tail]
  omega

/-- A longer tail is a *smaller* set: `tail n  subseteq  tail m  <->  m  <=  n`. -/
theorem tail_subset_iff {n m : Nat} : tail n  subseteq  tail m  <->  m  <=  n := by
  constructor
  * intro h
    exact h (show n  in  tail n from le_refl n)
  * intro hmn k hk
    exact le_trans hmn hk

theorem nestedOrDisjoint : NestedOrDisjoint mem := by
  rintro X Y <n, rfl> <m, rfl>
  rcases le_total n m with h | h
  * exact Or.inr (Or.inl (tail_subset_iff.mpr h))
  * exact Or.inl (tail_subset_iff.mpr h)

/- concordance: source L446 EXERCISE 1.12 final segments of integers -/
/-- **Exercise 1.12.** The final-segment neighbourhood system on `Nat` (master `Delta =
     tail 0 = Nat`). -/
def neighborhoodSystem : NeighborhoodSystem Nat :=
  ofNestedOrDisjoint mem (tail 0) <0, Nat.zero_le 0> <0, rfl> nestedOrDisjoint
    (fun {X} hX => by obtain <n, rfl> := hX; exact tail_subset_iff.mpr (Nat.zero_le n)
    )

@[simp] theorem ns_mem {X : Set Nat} : neighborhoodSystem.mem X  <->  exists  n, X =
    tail n := Iff.rfl

theorem mem_tail_nbhd (n : Nat) : neighborhoodSystem.mem (tail n) := <n, rfl>

/-- The finite element `fin n = (tail n)` (principal filter of the tail). -/
def fin (n : Nat) : neighborhoodSystem.Element := neighborhoodSystem.principal (
    mem_tail_nbhd n)

/-- The **limit element** `top`: the filter of *all* tails (Scott's single limit node).
     -/
def top : neighborhoodSystem.Element where
  mem X := neighborhoodSystem.mem X
  sub h := h
  master_mem := neighborhoodSystem.master_mem
  inter_mem := by
    rintro X Y <n, rfl> <m, rfl>
    exact <max n m, tail_inter n m>
  up_mem _ hY _ := hY
```

```
@[simp] theorem mem_top {X : Set Nat} : top.mem X  <->  neighborhoodSystem.mem X :=
    Iff.rfl

/-- **Exercise 1.12 (classification).** Every element is either a finite element `fin n
    ` or the
single limit element `top`. *Classical*: it decides whether the indices present in `x`
    are bounded.
(Hint realized: "there is only one limit element.") -/
theorem element_eq (x : neighborhoodSystem.Element) :
    (exists  n, x = fin n)  \/  x = top := by
  classical
  by_cases h : forall  n, x.mem (tail n)
  * refine Or.inr ?_
    apply Element.ext
    intro X
    constructor
    * intro hx; exact x.sub hx
    * rintro <n, rfl>; exact h n
  * refine Or.inl ?_
    have h' : exists  n, not  x.mem (tail n) := not_forall.mp h
    have hpos : forall  n, not  x.mem (tail n)  ->  0 < n := by
      intro n hn
      rcases Nat.eq_zero_or_pos n with rfl | hp
      * exact absurd x.master_mem hn
      * exact hp
    have hspec : not  x.mem (tail (Nat.find h')) := Nat.find_spec h'
    have hmin : forall  j, j < Nat.find h'  ->  x.mem (tail j) := by
      intro j hj
      by_contra hc
      exact Nat.find_min h' hj hc
    have hn_0 : 0 < Nat.find h' := hpos _ hspec
    refine <Nat.find h' - 1, ?_>
    apply Element.ext
    intro X
    have key : forall  j, x.mem (tail j)  <->  j  <=  Nat.find h' - 1 := by
      intro j
      constructor
      * intro hj
        by_contra hc
        have hle : Nat.find h'  <=  j := by omega
        exact hspec (x.up_mem hj (mem_tail_nbhd (Nat.find h')) (tail_subset_iff.mpr
    hle))
      * intro hj
        exact hmin j (by omega)
    constructor
    * intro hx
      obtain <j, rfl> := x.sub hx
      exact <mem_tail_nbhd j, tail_subset_iff.mpr ((key j).mp hx)>
    * rintro <hX, hsub>
      obtain <j, rfl> := hX
      exact (key j).mpr (tail_subset_iff.mp hsub)

/-- The finite elements form an ascending chain: `fin n  sqsub  fin m  <->  n  <=  m`.
    -/
theorem fin_le_iff (n m : Nat) : fin n  <=  fin m  <->  n  <=  m := by
```

```
  rw [fin, fin, neighborhoodSystem.principal_le_iff, tail_subset_iff]

/-- The chain `fin 0 ? fin 1 ? ?` is strictly increasing. -/
theorem fin_strictMono : StrictMono fin := by
  intro n m hnm
  refine lt_of_le_of_ne ((fin_le_iff n m).mpr hnm.le) ?_
  intro heq
  have : m  <=  n := (fin_le_iff m n).mp (le_of_eq heq.symm)
  exact absurd (le_antisymm hnm.le this) (Nat.ne_of_lt hnm)

/-- `top` is the greatest element of `|?|`: every element approximates it. -/
theorem le_top (x : neighborhoodSystem.Element) : x  <=  top :=
  fun _ hX => x.sub hX

/-- `top` is a total (maximal) element. -/
theorem top_isTotal : neighborhoodSystem.IsTotal top :=
  fun y _ => le_top y

/-- `top` is the **unique** total element: an element is total iff it equals `top`. -/
theorem isTotal_iff_top (x : neighborhoodSystem.Element) :
    neighborhoodSystem.IsTotal x  <->  x = top := by
  constructor
  * intro h
    exact le_antisymm (le_top x) (h top (le_top x))
  * rintro rfl; exact top_isTotal

end DomainTheory.NeighbourhoodSystems.Exercise112
```

3.2.20 Exercise 1.13 — infinite binary tree

Verify all the assertions made about the system B defined as the infinite generalization of Example 1.4. Draw a picture similar to that given in the text which includes nodes for all $\sigma \in \Sigma^*$. Show the neighbourhoods, how the approximation relation behaves, and where the total elements lie. (The picture is closely related to the "binary tree", but has to have limit nodes all along the top.)

Exercise_1_13.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Theorem_1_11
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.DomainIso

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems.Exercise113
section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ExampleB

/- concordance: source L454 EXERCISE 1.13 infinite binary tree -/
/-- **Exercise 1.13 (total elements).** Each infinite path `p` gives a *total* (maximal
    ) element
```

```
`branch p`: any `y` it approximates approximates it back. (These maximal elements are
    exactly the
limit nodes along the top of the binary-tree picture.) -/
theorem branch_isTotal (p : Nat  ->  Bool) : B.IsTotal (branch p) := by
  intro y hy X hX
  obtain <sigma, rfl> := y.sub hX
  have hk : y.mem (cone (prefixSeq p sigma.length)) :=
    hy _ ((branch_mem_iff p).mpr <sigma.length, List.prefix_rfl>)
  rcases prefix_comparable_of_mem y hX hk with h | h
  * exact (branch_mem_iff p).mpr <sigma.length, h>
  * have hlen : (prefixSeq p sigma.length).length = sigma.length := prefixSeq_length p
     sigma.length
    have hsigma : prefixSeq p sigma.length = sigma := h.eq_of_length hlen
    exact (branch_mem_iff p).mpr <sigma.length, by rw [hsigma]>

section
set_option linter.unusedVariables false

theorem example_113 : forall  (p : Nat  ->  Bool),
  @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsTotal DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
    DomainTheory.NeighbourhoodSystems.ExampleB.B (DomainTheory.NeighbourhoodSystems.
    Exercise113.branch p) :=
  DomainTheory.NeighbourhoodSystems.Exercise113.branch_isTotal
end
end

end DomainTheory.NeighbourhoodSystems.Exercise113
namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem
section

variable {a : Type*} (V : NeighborhoodSystem a)

/-- `?_n x_n` is the **greatest** lower bound: anything approximating all `x_n`
    approximates `?_n x_n`. -/
theorem le_iInter (x : Nat  ->  V.Element) (y : V.Element) (h : forall  n, y  <=  x n)
     :
    y  <=  V.iInter x :=
  fun X hX n => h n X hX

/-- `x_n  sqsub  ?_n x_n`: every member of the sequence approximates the union. -/
theorem le_iUnion (x : Nat  ->  V.Element) (hmono : Monotone x) (n : Nat) :
    x n  <=  V.iUnion x hmono :=
  fun _ hX => <n, hX>
end

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.Exercise113
section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ExampleB

/-- Every finite approximation `(p ? n)False` approximates the limit element `branch p
    `. -/
```

```
theorem branchSeq_le_branch (p : Nat  ->  Bool) (n : Nat) : branchSeq p n  <=  branch
    p :=
  B.le_iUnion (branchSeq p) (branchSeq_mono p) n
end

end DomainTheory.NeighbourhoodSystems.Exercise113
namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem
section

variable {a : Type*} (V : NeighborhoodSystem a)

/-- The finite intersection is antitone in its length: a longer prefix is contained in
     a shorter
one. -/
theorem interUpTo_antitone (X : Nat  ->  Set a) {j k : Nat} (h : j  <=  k) :
    V.interUpTo X k  subseteq  V.interUpTo X j := by
  induction k with
  | zero => rw [Nat.le_zero.mp h]
  | succ k ih =>
    rcases Nat.eq_or_lt_of_le h with h' | h'
    * rw [h']
    * exact Set.inter_subset_left.trans (ih (Nat.lt_succ_iff.mp h'))
end

end NeighborhoodSystem

end DomainTheory.NeighbourhoodSystems
end
```

3.2.21 Exercise 1.14 — finite nonempty subsets

Let $\Delta = \mathbb{N}$ and let $\mathcal{V}$ be the family of finite non-empty subsets of Δ plus the set Δ. Show that this is a neighbourhood system. What are the total elements? What are the finite elements? Draw a picture.

Exercise_1_14.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.DomainIso

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems.Exercise114
section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem

/-- Membership: `X` is a neighbourhood iff `X = Nat` (the master `Delta`) or `X` is
    finite and non-empty. -/
def mem (X : Set Nat) : Prop := X = Set.univ  \/  (X.Finite  /\  X.Nonempty)

theorem nonempty_of_mem {X : Set Nat} (h : mem X) : X.Nonempty := by
  rcases h with rfl | h
```

```
  * exact Set.univ_nonempty
  * exact h.2

/- concordance: source L456 EXERCISE 1.14 finite nonempty subsets -/
/-- **Exercise 1.14.** The neighbourhood system of finite non-empty subsets of `Nat` (
    plus `Delta = Nat`). -/
def neighborhoodSystem : NeighborhoodSystem Nat where
  mem
  master := Set.univ
  master_nonempty := Set.univ_nonempty
  master_mem := Or.inl rfl
  inter_mem := by
    intro X Y Z hX hY hZ hZsub
    have hne : (X  I  Y).Nonempty := (nonempty_of_mem hZ).mono hZsub
    rcases hX with hX | hX
    * rw [hX, Set.univ_inter]; exact hY
    * rcases hY with hY | hY
      * rw [hY, Set.inter_univ]; exact Or.inr hX
      * exact Or.inr <hX.1.inter_of_left Y, hne>
  sub_master := fun _ => Set.subset_univ _

@[simp] theorem ns_mem {X : Set Nat} : neighborhoodSystem.mem X  <->  mem X := Iff.rfl

/-- `False = Delta = Nat`, the least element. -/
def bot : neighborhoodSystem.Element := neighborhoodSystem.bot

/-- The finite element `X` for a neighbourhood `X` (Scott's finite elements). -/
def fin {X : Set Nat} (h : neighborhoodSystem.mem X) : neighborhoodSystem.Element :=
  neighborhoodSystem.principal h

theorem empty_not_mem : not  mem ({} : Set Nat) := by
  rintro (h | <_, h>)
  * exact Set.empty_ne_univ h
  * exact Set.not_nonempty_empty h

theorem mem_singleton (n : Nat) : mem {n} :=
  Or.inr <Set.finite_singleton n, Set.singleton_nonempty n>

/-- **Exercise 1.14 (total elements).** The principal filter of a *singleton* `{n}` is
    a total
(maximal) element: any `y` it approximates approximates it back. (A `y ? {n}` would
    contain some
`W ? n`; then `{n}  I  W = {}  in  y  subseteq  ?`, impossible.) These are exactly the
    total elements. -/
theorem singleton_isTotal (n : Nat) :
    neighborhoodSystem.IsTotal (neighborhoodSystem.principal (mem_singleton n)) := by
  intro y hy W hW
  have hn : y.mem {n} := hy {n} <mem_singleton n, subset_rfl>
  refine <y.sub hW, ?_>
  by_contra hc
  rw [Set.singleton_subset_iff] at hc
  have hempty : ({n} : Set Nat)  I  W = ({} : Set Nat) := by
    ext k
    simp only [Set.mem_inter_iff, Set.mem_singleton_iff, Set.mem_empty_iff_false,
    iff_false]
    rintro <rfl, hk>
```

```
    exact hc hk
  have hi := y.inter_mem hn hW
  rw [hempty] at hi
  exact empty_not_mem (y.sub hi)

section
set_option linter.unusedVariables false

theorem example_114 : forall  (n : Nat),
  @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsTotal Nat
    DomainTheory.NeighbourhoodSystems.Exercise114.neighborhoodSystem
    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.principal Nat
      DomainTheory.NeighbourhoodSystems.Exercise114.neighborhoodSystem
      (@Singleton.singleton Nat (Set Nat) (@Set.instSingletonSet Nat) n)
      (DomainTheory.NeighbourhoodSystems.Exercise114.mem_singleton n)) :=
  DomainTheory.NeighbourhoodSystems.Exercise114.singleton_isTotal
end
end

end DomainTheory.NeighbourhoodSystems.Exercise114
end
```

3.2.22 Exercise 1.15 — finite elements without chains

Construct non-isomorphic infinite domains where all elements are finite but where there are no infinite chains $\langle x_n \rangle_{n=0}^{\infty}$ of elements with $x_n \sqsubseteq x_{n+1}$ but $x_n \neq x_{n+1}$ for all n.

Exercise_1_15.lean

```
module

import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.Factoid_1_8
import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.Factoid_1_7
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.DomainIso
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.bot
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_4

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems.Exercise115
section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem

/-- Neighbourhoods of the flat domain: `Delta = Nat` together with all singletons. -/
def flatMem (X : Set Nat) : Prop := X = Set.univ  \/  exists  n, X = {n}

/-- `{n}  I  {m} = {}` for `n  !=  m`. -/
theorem singleton_disjoint {n m : Nat} (h : n  !=  m) : ({n} : Set Nat)  I  {m} = {}
    := by
  ext k
  simp only [Set.mem_inter_iff, Set.mem_singleton_iff, Set.mem_empty_iff_false,
    iff_false]
```

```
  rintro <rfl, h2>
  exact h h2

theorem flatNOD : NestedOrDisjoint flatMem := by
  rintro X Y (rfl | <n, rfl>) (rfl | <m, rfl>)
  * exact Or.inl (subset_refl _)
  * exact Or.inr (Or.inl (Set.subset_univ _))
  * exact Or.inl (Set.subset_univ _)
  * rcases eq_or_ne n m with rfl | h
    * exact Or.inl (subset_refl _)
    * exact Or.inr (Or.inr (singleton_disjoint h))

/-- **Exercise 1.15 -- the flat domain.** `? = {Nat}  U  {{n}}`. -/
def flat : NeighborhoodSystem Nat :=
  ofNestedOrDisjoint flatMem Set.univ Set.univ_nonempty
    (Or.inl rfl) flatNOD (fun _ => Set.subset_univ _)

@[simp] theorem flat_mem {X : Set Nat} : flat.mem X  <->  X = Set.univ  \/  exists  n,
     X = {n} := Iff.rfl

@[simp] theorem flat_master : flat.master = Set.univ := rfl

/-- Neighbourhoods of the stem domain: `Delta = Nat`, the pair `{0,1}`, and all
    singletons. -/
def stemMem (X : Set Nat) : Prop := X = Set.univ  \/  X = {0, 1}  \/  exists  n, X = {
    n}

theorem stemNOD : NestedOrDisjoint stemMem := by
  have pair_single : forall  m : Nat, ({m} : Set Nat)  subseteq  {0, 1}  \/  ({0, 1} :
     Set Nat)  I  {m} = {} := by
    intro m
    by_cases h : m = 0  \/  m = 1
    * refine Or.inl ?_
      rcases h with rfl | rfl <;> intro k hk <;> simp_all
    * refine Or.inr ?_
      ext k
      simp only [Set.mem_inter_iff, Set.mem_insert_iff, Set.mem_singleton_iff,
        Set.mem_empty_iff_false, iff_false, not_and]
      rintro (rfl | rfl) rfl <;> simp_all
  rintro X Y (rfl | rfl | <n, rfl>) (rfl | rfl | <m, rfl>)
  * exact Or.inl (subset_refl _)
  * exact Or.inr (Or.inl (Set.subset_univ _))
  * exact Or.inr (Or.inl (Set.subset_univ _))
  * exact Or.inl (Set.subset_univ _)
  * exact Or.inl (subset_refl _)
  * rcases pair_single m with h | h
    * exact Or.inr (Or.inl h)
    * exact Or.inr (Or.inr h)
  * exact Or.inl (Set.subset_univ _)
  * rcases pair_single n with h | h
    * exact Or.inl h
    * exact Or.inr (Or.inr (by rw [Set.inter_comm]; exact h))
  * rcases eq_or_ne n m with rfl | h
    * exact Or.inl (subset_refl _)
    * exact Or.inr (Or.inr (singleton_disjoint h))
```

```
/-- **Exercise 1.15 -- the stem domain.** `? = {Nat, {0,1}}  U  {{n}}`. -/
def stem : NeighborhoodSystem Nat :=
  ofNestedOrDisjoint stemMem Set.univ Set.univ_nonempty
    (Or.inl rfl) stemNOD (fun _ => Set.subset_univ _)

@[simp] theorem stem_mem {X : Set Nat} :
    stem.mem X  <->  X = Set.univ  \/  X = {0, 1}  \/  exists  n, X = {n} := Iff.rfl

theorem flat_empty_not_mem : not  flat.mem ({} : Set Nat) := by
  rintro (h | <n, h>)
  * exact Set.empty_ne_univ h
  * exact absurd h.symm (Set.singleton_ne_empty n)

/-- The atom `{n}` (a finite element). -/
theorem flat_mem_singleton (n : Nat) : flat.mem {n} := Or.inr <n, rfl>

/-- **Flat classification.** Every element is `False` or an atom `{n}`. *Classical.* -/
theorem flat_classify (x : flat.Element) :
    x = flat.bot  \/  exists  n, x = flat.principal (flat_mem_singleton n) := by
  classical
  by_cases h : exists  n, x.mem {n}
  * obtain <n, hn> := h
    refine Or.inr <n, ?_>
    have huniq : forall  m, x.mem {m}  ->  m = n := by
      intro m hm
      by_contra hmn
      have hi : x.mem (({m} : Set Nat)  I  {n}) := x.inter_mem hm hn
      rw [singleton_disjoint hmn] at hi
      exact flat_empty_not_mem (x.sub hi)
    apply Element.ext
    intro W
    constructor
    * intro hx
      rcases (x.sub hx) with rfl | <k, rfl>
      * exact <Or.inl rfl, Set.subset_univ _>
      * obtain rfl := huniq k hx
        exact <flat_mem_singleton k, subset_rfl>
    * rintro <hWmem, hsub>
      exact x.up_mem hn hWmem hsub
  * refine Or.inl ?_
    apply Element.ext
    intro W
    rw [flat.mem_bot]
    constructor
    * intro hx
      rcases (x.sub hx) with rfl | <k, rfl>
      * rfl
      * exact absurd <k, hx> h
    * rintro rfl
      exact x.master_mem

/-- **All elements of `flat` are finite** (principal). -/
theorem flat_all_finite (x : flat.Element) : exists  (X : Set Nat) (h : flat.mem X), x
     = flat.principal h := by
  rcases flat_classify x with rfl | <n, rfl>
  * exact <flat.master, flat.master_mem, rfl>
```

```
  * exact <{n}, flat_mem_singleton n, rfl>

/-- **Flat: every atom is maximal (total).** -/
theorem flat_atom_maximal (n : Nat) (y : flat.Element)
    (hle : flat.principal (flat_mem_singleton n)  <=  y) :
    y = flat.principal (flat_mem_singleton n) := by
  rcases flat_classify y with rfl | <m, rfl>
  * exfalso
    have hmem : flat.bot.mem {n} := hle {n} <flat_mem_singleton n, subset_rfl>
    rw [flat.mem_bot, flat_master] at hmem
    have : (n + 1)  in  ({n} : Set Nat) := by rw [hmem]; exact Set.mem_univ _
    simp at this
  * have hsub : ({m} : Set Nat)  subseteq  {n} :=
      (flat.principal_le_iff (flat_mem_singleton n) (flat_mem_singleton m)).mp hle
    obtain rfl : m = n := Set.mem_singleton_iff.mp (Set.singleton_subset_iff.mp hsub)
    rfl

/-- `flat` is infinite: the atoms `{n}` are pairwise distinct. -/
theorem flat_infinite : Function.Injective (fun n => flat.principal (
    flat_mem_singleton n)) := by
  intro n m h
  have := flat.principal_injective (flat_mem_singleton n) (flat_mem_singleton m) h
  exact Set.singleton_injective this

/-- **Flat has no strict 3-chain** (`a ? b ? c` is impossible): `False` is least and
    atoms are maximal. -/
theorem flat_no_three_chain : not  exists  a b c : flat.Element, a < b  /\  b < c :=
    by
  rintro <a, b, c, hab, hbc>
  rcases flat_classify b with rfl | <n, rfl>
  * exact absurd (le_antisymm hab.le (flat.bot_le a)) (ne_of_lt hab)
  * exact (ne_of_lt hbc) (flat_atom_maximal n c hbc.le).symm

/-- **Flat has no infinite ascending chain.** -/
theorem flat_no_infinite_chain : not  exists  f : Nat  ->  flat.Element, StrictMono f
    := by
  rintro <f, hf>
  exact flat_no_three_chain
    <f 0, f 1, f 2, hf (by norm_num), hf (by norm_num)>

theorem stem_mem_pair : stem.mem {0, 1} := Or.inr (Or.inl rfl)

theorem stem_mem_univ : stem.mem Set.univ := Or.inl rfl

theorem stem_mem_zero : stem.mem {0} := Or.inr (Or.inr <0, rfl>)

/-- A strict step between principal filters of `stem`: `Y ? X` (proper) gives `X ? Y`.
    -/
theorem stem_principal_lt {X Y : Set Nat} (hX : stem.mem X) (hY : stem.mem Y)
    (hYX : Y  subseteq  X) (hne : X  !=  Y) : stem.principal hX < stem.principal hY :=
     by
  rw [lt_iff_le_and_ne]
  refine <(stem.principal_le_iff hX hY).mpr hYX, ?_>
  intro he
  exact hne (stem.principal_injective hX hY he)
```

```
/-- **Stem contains a strict 3-chain** `False = Nat ? {0,1} ? {0}` -- the structural
    feature that
distinguishes it from `flat`. -/
theorem stem_three_chain : exists  a b c : stem.Element, a < b  /\  b < c := by
  refine <stem.principal stem_mem_univ, stem.principal stem_mem_pair,
    stem.principal stem_mem_zero, ?_, ?_>
  * refine stem_principal_lt stem_mem_univ stem_mem_pair (Set.subset_univ _) ?_
    intro h
    have : (2 : Nat)  in  ({0, 1} : Set Nat) := h  |>  Set.mem_univ 2
    simp at this
  * refine stem_principal_lt stem_mem_pair stem_mem_zero
      (Set.singleton_subset_iff.mpr (by simp)) ?_
    intro h
    have : (1 : Nat)  in  ({0} : Set Nat) := h  |>  (by simp : (1 : Nat)  in  ({0, 1}
    : Set Nat))
    simp at this
end

end DomainTheory.NeighbourhoodSystems.Exercise115

namespace DomainTheory.NeighbourhoodSystems.Exercise115
section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem

/- concordance: source L458 EXERCISE 1.15 finite elements without chains -/
/-- **Exercise 1.15.** `flat` and `stem` are *not* isomorphic: an order-isomorphism
    would transport
`stem`'s strict 3-chain `False ? {0,1} ? {0}` back to a strict 3-chain in `flat`, which
     has none. -/
theorem not_isomorphic : not  (flat  ~= ? stem) := by
  rintro <e>
  obtain <a, b, c, hab, hbc> := stem_three_chain
  exact flat_no_three_chain
    <e.symm a, e.symm b, e.symm c, e.symm.lt_iff_lt.mpr hab, e.symm.lt_iff_lt.mpr hbc>

section
set_option linter.unusedVariables false

theorem example_115 : Not
  (@DomainTheory.NeighbourhoodSystems.Isomorphic Nat Nat DomainTheory.
    NeighbourhoodSystems.Exercise115.flat
    DomainTheory.NeighbourhoodSystems.Exercise115.stem) :=
  DomainTheory.NeighbourhoodSystems.Exercise115.not_isomorphic
end
end

end DomainTheory.NeighbourhoodSystems.Exercise115
end
```

3.2.23 Exercise 1.16 — cofinite integer subsets

Let $\Delta = \mathbf{N}$ and let $\mathcal{D}$ be the family of *cofinite* subsets of $\mathbf{N}$. Show that $|\mathcal{D}|$ is isomorphic to the partially ordered set of *all* subsets of $\mathbf{N}$ under inclusion. Construct some other neighbourhood systems where $\mathcal{D}$ is

closed under finite intersection. What happens to the total elements in such systems?

Exercise_1_16.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.DomainIso
public import Mathlib.Algebra.Group.End
public import Mathlib.Basic.Finite.Defs

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
namespace Cofinite
section

open NeighborhoodSystem

/- concordance: source L462 EXERCISE 1.16 cofinite integer subsets -/
/-- **Exercise 1.16 -- the cofinite neighbourhood system on `Nat`.** `X  in  ? ? X?`
    finite;
closed under finite intersection since `(X I Y)? = X?  U  Y?`. -/
def cofiniteSystem : NeighborhoodSystem Nat where
  mem X := X?.Finite
  master := Set.univ
  master_nonempty := Set.univ_nonempty
  master_mem := by rw [Set.compl_univ]; exact Set.finite_empty
  inter_mem := by
    intro X Y _ hX hY _ _
    rw [Set.compl_inter]
    exact hX.union hY
  sub_master := fun _ => Set.subset_univ _

@[simp] theorem cofiniteSystem_mem {X : Set Nat} : cofiniteSystem.mem X  <->  X?.
    Finite := Iff.rfl

/-- The filter generated by an arbitrary set `E  subseteq  Nat` of "excluded points":
`ofExcluded E = {Y cofinite ? Y?  subseteq  E}`. This is a filter for *every* `E`. -/
def ofExcluded (E : Set Nat) : cofiniteSystem.Element where
  mem Y := Y?.Finite  /\  Y?  subseteq  E
  sub h := h.1
  master_mem := by
    show (Set.univ : Set Nat)?.Finite  /\  (Set.univ : Set Nat)?  subseteq  E
    rw [Set.compl_univ]
    exact <Set.finite_empty, Set.empty_subset E>
  inter_mem := by
    rintro Y1 Y2 <hf1, hs1> <hf2, hs2>
    rw [Set.compl_inter]
    exact <hf1.union hf2, Set.union_subset hs1 hs2>
  up_mem := by
    rintro Y1 Y2 <_, hs1> hY2 hsub
    exact <hY2, (Set.compl_subset_compl.mpr hsub).trans hs1>

@[simp] theorem mem_ofExcluded {E Y : Set Nat} :
    (ofExcluded E).mem Y  <->  Y?.Finite  /\  Y?  subseteq  E := Iff.rfl
```

```
/-- The set of points *excluded* by a filter `x`: `toExcluded x = {n ? {n}?  in  x}`.
    -/
def toExcluded (x : cofiniteSystem.Element) : Set Nat := {n | x.mem ({n}?)}

@[simp] theorem mem_toExcluded {x : cofiniteSystem.Element} {n : Nat} :
    n  in  toExcluded x  <->  x.mem ({n}?) := Iff.rfl

/-- **Exercise 1.16 (second example).** All subsets of `Nat` form a neighbourhood
    system, trivially
closed under *all* finite intersections. Unlike `cofiniteSystem` it is **not** positive
     (`{}` is a
neighbourhood). Every neighbourhood is a finite element via `principal`; its total
    elements are
the principal filters of singletons (the system is "flat"). -/
def fullSystem : NeighborhoodSystem Nat where
  mem _ := True
  master := Set.univ
  master_nonempty := Set.univ_nonempty
  master_mem := trivial
  inter_mem := fun _ _ _ _ => trivial
  sub_master := fun _ => Set.subset_univ _

/-- **Reconstruction lemma.** For finite `F` of points each of whose single-deletion `{
    n}?`
is in the filter `x`, the intersection `?_{n in F} {n}? = F?` is also in `x`. -/
theorem mem_compl_of_finite (x : cofiniteSystem.Element) {F : Set Nat} (hF : F.Finite)
     :
    (forall  n  in  F, x.mem ({n}?))  ->  x.mem (F?) := by
  induction F, hF using Set.Finite.induction_on with
  | empty => intro _; rw [Set.compl_empty]; exact x.master_mem
  | @insert a s _ _ ih =>
    intro h
    rw [Set.insert_eq, Set.compl_union]
    exact x.inter_mem (h a (Set.mem_insert a s))
      (ih (fun n hn => h n (Set.mem_insert_of_mem a hn)))

/-- Each `{n}?` is a neighbourhood (its complement `{n}` is finite). -/
theorem mem_compl_singleton (n : Nat) : cofiniteSystem.mem (({n} : Set Nat)?) := by
  rw [cofiniteSystem_mem, compl_compl]
  exact Set.finite_singleton n

/-- For `n  notin  Y`, the neighbourhood `Y` is contained in `{n}?`. -/
theorem subset_compl_singleton {Y : Set Nat} {n : Nat} (hn : n  in  Y?) : Y  subseteq
     ({n} : Set Nat)? := by
  intro m hm
  rw [Set.mem_compl_iff, Set.mem_singleton_iff]
  rintro rfl
  exact hn hm

theorem ofExcluded_toExcluded (x : cofiniteSystem.Element) :
    ofExcluded (toExcluded x) = x := by
  apply Element.ext
  intro Y
  rw [mem_ofExcluded]
  constructor
```

```
  * rintro <hfin, hsub>
    have h := mem_compl_of_finite x hfin (fun n hn => mem_toExcluded.mp (hsub hn))
    rwa [compl_compl] at h
  * intro hxY
    refine <x.sub hxY, ?_>
    intro n hn
    rw [mem_toExcluded]
    exact x.up_mem hxY (mem_compl_singleton n) (subset_compl_singleton hn)

theorem toExcluded_ofExcluded (E : Set Nat) : toExcluded (ofExcluded E) = E := by
  ext n
  rw [mem_toExcluded, mem_ofExcluded]
  simp [compl_compl, Set.finite_singleton, Set.singleton_subset_iff]

/-- **Exercise 1.16 -- `|?|  ~=  (?(Nat),  subseteq )`.** The filter `x` corresponds to
     its excluded-point
set `toExcluded x`; inclusion of filters matches inclusion of excluded sets. -/
def cofiniteIso : cofiniteSystem.Element ?o Set Nat where
  toFun := toExcluded
  invFun := ofExcluded
  left_inv := ofExcluded_toExcluded
  right_inv := toExcluded_ofExcluded
  map_rel_iff' := by
    intro x x'
    constructor
    * intro h Y hY
      have hfin : Y?.Finite := x.sub hY
      have hsub : Y?  subseteq  toExcluded x' := by
        intro n hn
        have hx : x.mem (({n} : Set Nat)?) :=
          x.up_mem hY (mem_compl_singleton n) (subset_compl_singleton hn)
        exact h (mem_toExcluded.mpr hx)
      have h2 := mem_compl_of_finite x' hfin (fun n hn => mem_toExcluded.mp (hsub hn))
      rwa [compl_compl] at h2
    * intro h n hn
      exact mem_toExcluded.mpr (h _ (mem_toExcluded.mp hn))

/-- Every filter is ` sqsub  ofExcluded Nat` (the whole family `?`; every cofinite `Y`
    has `Y?  subseteq  Nat`). -/
theorem le_ofExcluded_univ (x : cofiniteSystem.Element) :
    x  <=  ofExcluded Set.univ := by
  intro Y hY
  exact <x.sub hY, Set.subset_univ _>

/-- **Exercise 1.16 -- the unique total element.** `ofExcluded Nat` (matching the
    greatest
element `Nat  in  ?(Nat)`) is total: it is ` sqsub `-maximal. -/
theorem ofExcluded_univ_isTotal : cofiniteSystem.IsTotal (ofExcluded Set.univ) :=
  fun y _ => le_ofExcluded_univ y
end

end Cofinite
end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.Cofinite
section
```

```
set_option linter.unusedVariables false

theorem example_116 : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsTotal
    Nat
  DomainTheory.NeighbourhoodSystems.Cofinite.cofiniteSystem
  (DomainTheory.NeighbourhoodSystems.Cofinite.ofExcluded (@Set.univ Nat)) :=
  DomainTheory.NeighbourhoodSystems.Cofinite.ofExcluded_univ_isTotal
end

end DomainTheory.NeighbourhoodSystems.Cofinite
end
```

3.2.24 Exercise 1.17 — rational open intervals

Let $\Delta = \mathbf{R}$ be the real line. Let $\mathcal{D}$ be the set of non-empty open intervals with rational end points plus the set Δ. Show that this is a neighbourhood system. For any real $t \in \mathbf{R}$, show that $\{X \in \mathcal{D} \mid t \in X\}$ is a filter. Is it always total? What are the total elements of $|\mathcal{D}|$? (Hint: When t is rational consider all intervals with t as a right-hand end point.)

Exercise_1_17.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.Element
public import Mathlib.Basic.Real.Basic

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
namespace RatInterval
section

open NeighborhoodSystem

/-- A neighbourhood of `ratIntervalSystem`: either `Delta = ?`, or a non-empty open
    interval with
rational endpoints. -/
def ratIntervalMem (X : Set ?) : Prop :=
  X = Set.univ  \/  exists  a b : ?, a < b  /\  X = Set.Ioo (a : ?) (b : ?)

/- concordance: source L464 EXERCISE 1.17 rational open intervals -/
/-- **Exercise 1.17 -- intersections.** The intersection of two neighbourhoods that
    share a point
is again a neighbourhood: `Ioo a b  I  Ioo c d = Ioo (max a c) (min b d)`, with
    rational endpoints,
non-empty because it contains the shared point. -/
theorem inter_mem' {X Y : Set ?} (hX : ratIntervalMem X) (hY : ratIntervalMem Y)
    (hne : (X  I  Y).Nonempty) : ratIntervalMem (X  I  Y) := by
  rcases hX with rfl | <a, b, hab, rfl>
  * rw [Set.univ_inter]; exact hY
  * rcases hY with rfl | <c, d, hcd, rfl>
    * rw [Set.inter_univ]; exact Or.inr <a, b, hab, rfl>
    * obtain <z, <hza, hzb>, hzc, hzd> := hne
```

```
      have had : a < d := by exact_mod_cast hza.trans hzd
      have hcb : c < b := by exact_mod_cast hzc.trans hzb
      have hlt : max a c < min b d := by
        simp only [max_lt_iff, lt_min_iff]
        exact <<hab, hcb>, had, hcd>
      have hset : Set.Ioo (a : ?) b  I  Set.Ioo (c : ?) d
          = Set.Ioo ((max a c : ?) : ?) ((min b d : ?) : ?) := by
        rw [Set.Ioo_inter_Ioo, Rat.cast_max, Rat.cast_min]
      exact Or.inr <max a c, min b d, hlt, hset>

/-- Every neighbourhood is non-empty (`Delta`, or `Ioo a b` with `a < b`). -/
theorem ratIntervalMem_nonempty {X : Set ?} (hX : ratIntervalMem X) : X.Nonempty := by
  rcases hX with rfl | <a, b, hab, rfl>
  * exact <0, Set.mem_univ 0>
  * rw [Set.nonempty_Ioo]; exact_mod_cast hab

/-- **Exercise 1.17 -- the rational-interval neighbourhood system over `?`.** -/
def ratIntervalSystem : NeighborhoodSystem ? where
  mem := ratIntervalMem
  master := Set.univ
  master_nonempty := Set.univ_nonempty
  master_mem := Or.inl rfl
  inter_mem := by
    intro X Y Z hX hY hZ hZsub
    exact inter_mem' hX hY ((ratIntervalMem_nonempty hZ).mono hZsub)
  sub_master := fun _ => Set.subset_univ _

@[simp] theorem ratIntervalSystem_mem {X : Set ?} :
    ratIntervalSystem.mem X  <->  ratIntervalMem X := Iff.rfl

/-- **Exercise 1.17 -- the point filter `{X  in  ? ? t  in  X}`.** For any real `t`
    this is a filter:
closure under ` I ` uses `inter_mem'` with `t` itself as the shared point. -/
def filterAt (t : ?) : ratIntervalSystem.Element where
  mem X := ratIntervalMem X  /\  t  in  X
  sub h := h.1
  master_mem := <Or.inl rfl, Set.mem_univ t>
  inter_mem := by
    rintro X Y <hX, htX> <hY, htY>
    exact <inter_mem' hX hY <t, htX, htY>, htX, htY>
  up_mem := by
    rintro X Y <_, htX> hY hXY
    exact <hY, hXY htX>

@[simp] theorem mem_filterAt {t : ?} {X : Set ?} :
    (filterAt t).mem X  <->  ratIntervalMem X  /\  t  in  X := Iff.rfl

/-- **Exercise 1.17 -- `? ? |?|`.** Distinct reals give distinct point filters: between
     any two
reals lies a rational interval separating them. -/
theorem filterAt_injective {s t : ?} (h : filterAt s = filterAt t) : s = t := by
  by_contra hst
  rcases lt_or_gt_of_ne hst with hlt | hlt
  * obtain <a, ha1, ha2> := exists_rat_btwn hlt
    obtain <b, hb> := exists_rat_gt t
    have hint : ratIntervalMem (Set.Ioo (a : ?) (b : ?)) :=
```

```
      Or.inr <a, b, by exact_mod_cast ha2.trans hb, rfl>
    have htmem : (filterAt t).mem (Set.Ioo (a : ?) (b : ?)) := <hint, ha2, hb>
    rw [ <-  h] at htmem
    exact absurd htmem.2.1 (not_lt.mpr ha1.le)
  * obtain <a, ha1, ha2> := exists_rat_btwn hlt
    obtain <b, hb> := exists_rat_gt s
    have hint : ratIntervalMem (Set.Ioo (a : ?) (b : ?)) :=
      Or.inr <a, b, by exact_mod_cast ha2.trans hb, rfl>
    have hsmem : (filterAt s).mem (Set.Ioo (a : ?) (b : ?)) := <hint, ha2, hb>
    rw [h] at hsmem
    exact absurd hsmem.2.1 (not_lt.mpr ha1.le)
end

end RatInterval
end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.RatInterval
section

set_option linter.unusedVariables false

theorem example_117 : And
  (forall  {s t : Real},
    @Eq
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element Real
          DomainTheory.NeighbourhoodSystems.RatInterval.ratIntervalSystem)
        (DomainTheory.NeighbourhoodSystems.RatInterval.filterAt s)
        (DomainTheory.NeighbourhoodSystems.RatInterval.filterAt t)  ->
      @Eq Real s t)
  (forall  {X Y : Set Real},
    DomainTheory.NeighbourhoodSystems.RatInterval.ratIntervalMem X  ->
      DomainTheory.NeighbourhoodSystems.RatInterval.ratIntervalMem Y  ->
        @Set.Nonempty Real (@Inter.inter (Set Real) (@Set.instInter Real) X Y)  ->
          DomainTheory.NeighbourhoodSystems.RatInterval.ratIntervalMem
            (@Inter.inter (Set Real) (@Set.instInter Real) X Y)) :=
  @And.intro
  (forall  {s t : Real},
    @Eq
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element Real
          DomainTheory.NeighbourhoodSystems.RatInterval.ratIntervalSystem)
        (DomainTheory.NeighbourhoodSystems.RatInterval.filterAt s)
        (DomainTheory.NeighbourhoodSystems.RatInterval.filterAt t)  ->
      @Eq Real s t)
  (forall  {X Y : Set Real},
    DomainTheory.NeighbourhoodSystems.RatInterval.ratIntervalMem X  ->
      DomainTheory.NeighbourhoodSystems.RatInterval.ratIntervalMem Y  ->
        @Set.Nonempty Real (@Inter.inter (Set Real) (@Set.instInter Real) X Y)  ->
          DomainTheory.NeighbourhoodSystems.RatInterval.ratIntervalMem
            (@Inter.inter (Set Real) (@Set.instInter Real) X Y))
  @DomainTheory.NeighbourhoodSystems.RatInterval.filterAt_injective
  @DomainTheory.NeighbourhoodSystems.RatInterval.inter_mem'
end

end DomainTheory.NeighbourhoodSystems.RatInterval
end
```

3.2.25 Exercise 1.18 — consistency of neighbourhood families

Let $\mathcal{D}$ be a neighbourhood system. Call a subset $C \subseteq \mathcal{D}$ *consistent* iff every finite subset of C is consistent in $\mathcal{D}$. Give an example where C is a subset with more than two elements, every pair of neighbourhoods in C is consistent, but C is *not* consistent. Show that if C is consistent, then there is a *least* filter $x \in |\mathcal{D}|$ with $C \subseteq x$. Show generally that the *intersection* of any non-empty collection of filters is again a filter.

Exercise_1_18.lean

```
module

import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.Factoid_1_5
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_1
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_19

@[expose] public section

section
-- Exercise118/FinitelyConsistent.lean
set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems

namespace NeighborhoodSystem

variable {a : Type*} (V : NeighborhoodSystem a)

/- concordance: source L466 EXERCISE 1.18 consistency of neighbourhood families -/
/-- **Exercise 1.18 -- consistent subset.** `C  subseteq  ?` is *finitely consistent*
    iff every finite
sequence drawn from `C` is `Consistent` in `?`. -/
def FinitelyConsistent (C : Set (Set a)) : Prop :=
  forall  (n : Nat) (X : Nat  ->  Set a), (forall  i, i < n  ->  X i  in  C)  ->  V.
    Consistent X n

end NeighborhoodSystem

end DomainTheory.NeighbourhoodSystems
end

section
-- Exercise118/appendSeq.lean
set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems

/-- Concatenation of two finite sequences: the first `n1` entries are `X1 0, ..., X1 (
    n1-1)`,
then `X2 0, X2 1, ...`. -/
def appendSeq {a : Type*} (X1 : Nat  ->  Set a) (n1 : Nat) (X2 : Nat  ->  Set a) : Nat
      ->  Set a :=
  fun i => if i < n1 then X1 i else X2 (i - n1)

end DomainTheory.NeighbourhoodSystems
```

```
end

section
-- Exercise118/appendSeq_mem.lean
set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems

/-- Each entry of `appendSeq X1 n1 X2` below `n1 + n2` is drawn from `C`. -/
theorem appendSeq_mem {a : Type*} {C : Set (Set a)} {X1 : Nat  ->  Set a} {n1 : Nat}
    {X2 : Nat  ->  Set a} {n2 : Nat} (h1 : forall  i, i < n1  ->  X1 i  in  C) (h2 :
    forall  i, i < n2  ->  X2 i  in  C) :
    forall  i, i < n1 + n2  ->  appendSeq X1 n1 X2 i  in  C := by
  intro i hi
  simp only [appendSeq]
  by_cases h : i < n1
  * rw [ite_eq_left h]; exact h1 i h
  * rw [ite_eq_right h]; exact h2 (i - n1) (by omega)

end DomainTheory.NeighbourhoodSystems
end

section
-- Exercise118/A.lean
set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems

namespace triSys

/-- `A = {0,1}`. -/
def A : Set (Fin 3) := {0, 1}

end triSys

end DomainTheory.NeighbourhoodSystems
end

section
-- Exercise118/B.lean
set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems

namespace triSys

/-- `B = {1,2}`. -/
def B : Set (Fin 3) := {1, 2}

end triSys

end DomainTheory.NeighbourhoodSystems
end

section
-- Exercise118/Cc.lean
```

```
set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems

namespace triSys

/-- `Cc = {0,2}`. -/
def Cc : Set (Fin 3) := {0, 2}

end triSys

end DomainTheory.NeighbourhoodSystems
end

section
-- Exercise118/family.lean
set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems

namespace triSys

/-- The three-member family `C = {A, B, Cc}`. -/
def family : Set (Set (Fin 3)) := {A, B, Cc}

end triSys

end DomainTheory.NeighbourhoodSystems
end

section
-- Exercise118/interUpTo_appendSeq_left.lean
set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems

namespace NeighborhoodSystem

variable {a : Type*} (V : NeighborhoodSystem a)

/-- For a prefix length `k  <=  n1`, `interUpTo` of the appended sequence agrees with `
    interUpTo`
of the first sequence. -/
theorem interUpTo_appendSeq_left (X1 : Nat  ->  Set a) (n1 : Nat) (X2 : Nat  ->  Set a
    ) :
    forall  {k : Nat}, k  <=  n1  ->  V.interUpTo (appendSeq X1 n1 X2) k = V.interUpTo
     X1 k := by
  intro k
  induction k with
  | zero => intro _; rfl
  | succ k ih =>
    intro hk
    rw [interUpTo_succ, interUpTo_succ, ih (Nat.le_of_succ_le hk)]
    have happ : appendSeq X1 n1 X2 k = X1 k := by
      simp only [appendSeq, ite_eq_left (Nat.lt_of_succ_le hk)]
    rw [happ]
```

```
end NeighborhoodSystem

end DomainTheory.NeighbourhoodSystems
end

section
-- Exercise118/interUpTo_subset_master.lean
set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems

namespace NeighborhoodSystem

variable {a : Type*} (V : NeighborhoodSystem a)

/-- The finite intersection is contained in the master neighbourhood (its first factor
    ). -/
theorem interUpTo_subset_master (X : Nat  ->  Set a) : forall  n, V.interUpTo X n
    subseteq  V.master := by
  intro n
  induction n with
  | zero => exact subset_rfl
  | succ n ih => exact Set.inter_subset_left.trans ih

end NeighborhoodSystem

end DomainTheory.NeighbourhoodSystems
end

section
-- Exercise118/interUpTo_appendSeq.lean
set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems

namespace NeighborhoodSystem

variable {a : Type*} (V : NeighborhoodSystem a)

/-- The key identity: `?_{i<n1+n2} (X1 ? X2)_i = (?_{i<n1} X1_i)  I  (?_{i<n2} X2_i)`.
    -/
theorem interUpTo_appendSeq (X1 : Nat  ->  Set a) (n1 : Nat) (X2 : Nat  ->  Set a) :
    forall  {j : Nat}, V.interUpTo (appendSeq X1 n1 X2) (n1 + j)
      = V.interUpTo X1 n1  I  V.interUpTo X2 j := by
  intro j
  induction j with
  | zero =>
    rw [Nat.add_zero, V.interUpTo_appendSeq_left X1 n1 X2 (le_refl n1), interUpTo_zero
    ]
    exact (Set.inter_eq_left.mpr (V.interUpTo_subset_master X1 n1)).symm
  | succ j ih =>
    rw [Nat.add_succ, interUpTo_succ, ih]
    have happ : appendSeq X1 n1 X2 (n1 + j) = X2 j := by
      simp only [appendSeq, ite_eq_right (by omega : not  n1 + j < n1)]
      congr 1
```

```
      omega
    rw [happ, interUpTo_succ, Set.inter_assoc]

end NeighborhoodSystem

end DomainTheory.NeighbourhoodSystems
end

section
-- Exercise118/leastFilter.lean
set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems

namespace NeighborhoodSystem

variable {a : Type*} (V : NeighborhoodSystem a)

/-- **Exercise 1.18 -- the least filter containing a consistent `C`.**
`leastFilter C = {Y  in  ? ? ?_{i<n} X_i  subseteq  Y for some finite sequence <X_i>
    from C}`. The
intersection law concatenates two finite sequences (`interUpTo_appendSeq`) and uses
    finite
consistency to keep their combined intersection in `?`. -/
def leastFilter (C : Set (Set a)) (hCsub : forall  X  in  C, V.mem X)
    (hC : V.FinitelyConsistent C) : V.Element where
  mem Y := V.mem Y  /\  exists  (n : Nat) (X : Nat  ->  Set a), (forall  i, i < n  ->
     X i  in  C)  /\  V.interUpTo X n  subseteq  Y
  sub h := h.1
  master_mem :=
    <V.master_mem, 0, (fun _ => V.master), (fun i hi => absurd hi (Nat.not_lt_zero i))
    ,
      V.interUpTo_subset_master _ 0>
  inter_mem := by
    rintro X Y <hXmem, n1, X1, hX1C, hX1sub> <hYmem, n2, X2, hX2C, hX2sub>
    have hmemC : forall  i, i < n1 + n2  ->  appendSeq X1 n1 X2 i  in  C :=
    appendSeq_mem hX1C hX2C
    have hintermem : V.mem (V.interUpTo (appendSeq X1 n1 X2) (n1 + n2)) :=
      (V.consistent_iff_interUpTo_mem _ (fun i hi => hCsub _ (hmemC i hi))).mp
        (hC _ _ hmemC)
    have hsub : V.interUpTo (appendSeq X1 n1 X2) (n1 + n2)  subseteq  X  I  Y := by
      rw [V.interUpTo_appendSeq X1 n1 X2]
      exact Set.inter_subset_inter hX1sub hX2sub
    exact <V.inter_mem hXmem hYmem hintermem hsub, n1 + n2, appendSeq X1 n1 X2, hmemC,
     hsub>
  up_mem := by
    rintro X Y <_, n, Xs, hXC, hsub> hY hXY
    exact <hY, n, Xs, hXC, hsub.trans hXY>

end NeighborhoodSystem

end DomainTheory.NeighbourhoodSystems
end

section
-- Exercise118/sInf.lean
```

```
set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems

namespace NeighborhoodSystem

variable {a : Type*} (V : NeighborhoodSystem a)

/-- **Exercise 1.18 -- the intersection of a non-empty family of filters is a filter
    .**
`sInf F = {X ? forall  x  in  F, X  in  x}`. -/
def sInf (F : Set V.Element) (hF : F.Nonempty) : V.Element where
  mem X := forall  x  in  F, x.mem X
  sub h := hF.elim (fun x hx => x.sub (h x hx))
  master_mem := fun x _ => x.master_mem
  inter_mem h1 h2 := fun x hx => x.inter_mem (h1 x hx) (h2 x hx)
  up_mem h hY hXY := fun x hx => x.up_mem (h x hx) hY hXY

end NeighborhoodSystem

end DomainTheory.NeighbourhoodSystems
end

section
-- Exercise118/triSys.lean
set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems

/-- All non-empty subsets of `Delta = {0,1,2}` (a positive neighbourhood system). -/
def triSys : NeighborhoodSystem (Fin 3) :=
  NeighborhoodSystem.ofPositive (fun X => X.Nonempty) Set.univ
    (<0, Set.mem_univ 0>) (<0, Set.mem_univ 0>)
    (fun {_} _ => Set.subset_univ _) (fun _ _ _ _ => Iff.rfl)

end DomainTheory.NeighbourhoodSystems
end

section
-- Exercise118/triple.lean
set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems

namespace triSys

/-- The triple `A, B, Cc` as a finite sequence. -/
def triple : Nat  ->  Set (Fin 3) := fun i => if i = 0 then A else if i = 1 then B
    else Cc

end triSys

end DomainTheory.NeighbourhoodSystems
end

section
```

```
-- Exercise118/family_pairwise_nonempty.lean
set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems

namespace triSys

/-- Every pair of members of `family` has non-empty intersection, hence (in the all-non
    -empty
system `triSys`) every pair is consistent. -/
theorem family_pairwise_nonempty :
    forall  X  in  family, forall  Y  in  family, (X  I  Y).Nonempty := by
  intro X hX Y hY
  simp only [family, Set.mem_insert_iff, Set.mem_singleton_iff] at hX hY
  rcases hX with rfl | rfl | rfl <;> rcases hY with rfl | rfl | rfl
  * exact <0, by simp [A, B, Cc]>
  * exact <1, by simp [A, B, Cc]>
  * exact <0, by simp [A, B, Cc]>
  * exact <1, by simp [A, B, Cc]>
  * exact <1, by simp [A, B, Cc]>
  * exact <2, by simp [A, B, Cc]>
  * exact <0, by simp [A, B, Cc]>
  * exact <2, by simp [A, B, Cc]>
  * exact <0, by simp [A, B, Cc]>

end triSys
end DomainTheory.NeighbourhoodSystems
end

section
-- Exercise118/le_sInf.lean
set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems

namespace NeighborhoodSystem

variable {a : Type*} (V : NeighborhoodSystem a)

/-- `sInf F` is the **greatest** lower bound of `F`. -/
theorem le_sInf (F : Set V.Element) (hF : F.Nonempty) (y : V.Element) (h : forall  x
    in  F, y  <=  x) :
    y  <=  V.sInf F hF :=
  fun _ hX x hx => h x hx _ hX

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
end

section
-- Exercise118/leastFilter_le.lean
set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems

namespace NeighborhoodSystem
```

```
variable {a : Type*} (V : NeighborhoodSystem a)

/-- **Exercise 1.18 -- `leastFilter` is least.** Any filter `z` with `C  subseteq  z`
    contains
`leastFilter C`. -/
theorem leastFilter_le (C : Set (Set a)) (hCsub : forall  X  in  C, V.mem X)
    (hC : V.FinitelyConsistent C) (z : V.Element) (hz : forall  W  in  C, z.mem W) :
    V.leastFilter C hCsub hC  <=  z := by
  rintro Y <hYmem, n, X, hXC, hsub>
  exact z.up_mem (z.mem_interUpTo X (fun i hi => hz (X i) (hXC i hi))) hYmem hsub

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
end

section
-- Exercise118/sInf_le.lean
set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems

namespace NeighborhoodSystem

variable {a : Type*} (V : NeighborhoodSystem a)

/-- `sInf F  sqsub  x` for every `x  in  F`. -/
theorem sInf_le (F : Set V.Element) (hF : F.Nonempty) {x : V.Element} (hx : x  in  F)
    :
    V.sInf F hF  <=  x :=
  fun _ h => h x hx

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
end

section
-- Exercise118/subset_leastFilter.lean
set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems

namespace NeighborhoodSystem


variable {a : Type*} (V : NeighborhoodSystem a)

/-- `C  subseteq  leastFilter C`: every member of `C` is in the least filter. -/
theorem subset_leastFilter (C : Set (Set a)) (hCsub : forall  X  in  C, V.mem X)
    (hC : V.FinitelyConsistent C) {W : Set a} (hW : W  in  C) :
    (V.leastFilter C hCsub hC).mem W := by
  refine <hCsub W hW, 1, (fun _ => W), (fun _ _ => hW), ?_>
  rw [interUpTo_succ, interUpTo_zero]
  exact Set.inter_subset_right

end NeighborhoodSystem
```

```
end DomainTheory.NeighbourhoodSystems
end

section
-- Exercise118/triSys_master.lean
set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems

theorem triSys_master : triSys.master = (Set.univ : Set (Fin 3)) := rfl

end DomainTheory.NeighbourhoodSystems
end

section
-- Exercise118/triple_interUpTo_empty.lean
set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems

namespace triSys

theorem triple_interUpTo_empty : triSys.interUpTo triple 3 = ({} : Set (Fin 3)) := by
  simp only [NeighborhoodSystem.interUpTo_succ, NeighborhoodSystem.interUpTo_zero,
    triSys_master]
  ext x
  fin_cases x <;> simp [triple, A, B, Cc]

end triSys
end DomainTheory.NeighbourhoodSystems
end

section
-- Exercise118/triple_mem.lean
set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems

namespace triSys

theorem triple_mem : forall  i, i < 3  ->  triple i  in  family := by
  intro i hi
  simp only [family, Set.mem_insert_iff, Set.mem_singleton_iff]
  interval_cases i
  * exact Or.inl rfl
  * exact Or.inr (Or.inl rfl)
  * exact Or.inr (Or.inr rfl)

end triSys
end DomainTheory.NeighbourhoodSystems
end

section
-- Exercise118/not_finitelyConsistent.lean
set_option linter.unusedSimpArgs false
```

```
namespace DomainTheory.NeighbourhoodSystems

namespace triSys

/-- **Exercise 1.18 -- `family` is pairwise consistent but not consistent.** Its full
    triple has
empty intersection, so no `Z  in  ?` (i.e. no non-empty `Z`) lies below it. -/
theorem not_finitelyConsistent : not  triSys.FinitelyConsistent family := by
  intro h
  obtain <Z, hZmem, hZsub> := h 3 triple triple_mem
  have hZne : Z.Nonempty := hZmem
  rw [triple_interUpTo_empty, Set.subset_empty_iff] at hZsub
  rw [hZsub] at hZne
  exact Set.not_nonempty_empty hZne

end triSys
end DomainTheory.NeighbourhoodSystems
end

section
-- Exercise118.lean
namespace DomainTheory.NeighbourhoodSystems

set_option linter.unusedVariables false

theorem example_118.{u_1} : And
  (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a)
     (C : Set (Set a))
     (hCsub :
       forall  (X : Set a),
         @Membership.mem (Set a) (Set (Set a)) (@Set.instMembership (Set a)) C X  ->
           @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X)
     (hC : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.FinitelyConsistent a V
     C)
     (z : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V),
     (forall  (W : Set a),
         @Membership.mem (Set a) (Set (Set a)) (@Set.instMembership (Set a)) C W  ->
           @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem a V z W)
     ->
       @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
         (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
     a V)
           (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.Element a V)
             (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    instPartialOrderElement a V)))
         (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.leastFilter a V C hCsub
     hC) z)
  (Not
    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.FinitelyConsistent
      (Fin (@OfNat.ofNat Nat (nat_lit 3) (instOfNatNat (nat_lit 3)))) DomainTheory.
    NeighbourhoodSystems.triSys
      DomainTheory.NeighbourhoodSystems.triSys.family)) :=
  @And.intro
  (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a)
     (C : Set (Set a))
```

```
    (hCsub :
      forall  (X : Set a),
        @Membership.mem (Set a) (Set (Set a)) (@Set.instMembership (Set a)) C X  ->
          @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X)
    (hC : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.FinitelyConsistent a V
    C)
    (z : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V),
    (forall  (W : Set a),
        @Membership.mem (Set a) (Set (Set a)) (@Set.instMembership (Set a)) C W  ->
          @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem a V z W)
   ->
     @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
       (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
    a V)
         (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
   NeighborhoodSystem.Element a V)
           (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
   instPartialOrderElement a V)))
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.leastFilter a V C hCsub
    hC) z)
  (Not
    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.FinitelyConsistent
      (Fin (@OfNat.ofNat Nat (nat_lit 3) (instOfNatNat (nat_lit 3)))) DomainTheory.
    NeighbourhoodSystems.triSys
      DomainTheory.NeighbourhoodSystems.triSys.family))
  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.leastFilter_le)
  DomainTheory.NeighbourhoodSystems.triSys.not_finitelyConsistent

end DomainTheory.NeighbourhoodSystems
end
```

3.2.26 Exercise 1.19 — positive neighbourhood systems

Define a *positive neighbourhood system* to be a family $\mathcal{D}$ where (ii) of 1.1 is replaced by **(ii')** whenever $X, Y \in \mathcal{D}$, then $X \cap Y \neq \emptyset$ iff $X \cap Y \in \mathcal{D}$. Prove that a positive neighbourhood system is indeed a neighbourhood system in the sense of the earlier definition. Give an example of a neighbourhood system that is *not* positive. (Hint: (suggested by C.A.R. Hoare). Let $\Delta = \mathbf{N} \times \mathbf{N}$, in the plane. Let $\mathcal{D}$ be the family of subsets $X \subseteq \mathbf{N} \times \mathbf{N}$ where all but a finite number of places the *vertical* sections of X are the whole of $\mathbf{N}$ but at the other places the sections are finite and nonempty. Smaller examples are of course possible.)

Exercise_1_19.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.Element

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
/- concordance: source L468 EXERCISE 1.19 positive neighbourhood systems -/
/-- **Exercise 1.19 (Scott 1981, PRG-19) -- positivity, condition (ii').** A
    neighbourhood
```

```
system is *positive* when Scott's (ii) is strengthened to the biconditional **(ii')**:
     for
`X, Y  in  ?`, the intersection `X  I  Y` is a neighbourhood **iff** it is non-empty.
    -/
def NeighborhoodSystem.IsPositive {a : Type*} (V : NeighborhoodSystem a) : Prop :=
  forall  ?X Y : Set a?, V.mem X  ->  V.mem Y  ->  (V.mem (X  I  Y)  <->  (X  I  Y).
    Nonempty)

/-- **Difference-closure**: the set-difference of two neighbourhoods is again a
    neighbourhood, or
empty. -/
def NeighborhoodSystem.DiffClosed {a : Type*} (D : NeighborhoodSystem a) : Prop :=
  forall  {X Y : Set a}, D.mem X  ->  D.mem Y  ->  X \ Y = {}  \/  D.mem (X \ Y)

/-- **Atomlessness** ("no minimal neighbourhoods"): every neighbourhood splits into
    two disjoint,
non-empty neighbourhood pieces whose union recovers it. -/
def NeighborhoodSystem.NoMinimal {a : Type*} (D : NeighborhoodSystem a) : Prop :=
  forall  {X : Set a}, D.mem X  ->
    exists  Y Z : Set a, D.mem Y  /\  D.mem Z  /\  Y.Nonempty  /\  Z.Nonempty  /\  Y
    I  Z = {}  /\  Y  U  Z = X

/-- Scott's *"very special circumstance"* (the prose after Examples 1.2-1.4): a family
     `?`
is **nested-or-disjoint** when any two of its members are either nested (one included
    in the
other) or disjoint. -/
def NestedOrDisjoint {a : Type*} (mem : Set a  ->  Prop) : Prop :=
  forall  ?X Y : Set a?, mem X  ->  mem Y  ->  X  subseteq  Y  \/  Y  subseteq  X  \/
     X  I  Y = {}

/-- **Exercise 1.19 -- a positive system is a neighbourhood system.** Scott: "*prove
    that a
positive neighbourhood system is indeed a neighbourhood system*". From the raw data --
     (i)
`Delta  in  ?`, `?  subseteq  ?(Delta)`, and the positivity axiom (ii') -- condition (
    ii) follows: a consistency
witness `Z  subseteq  X  I  Y` with `Z  in  ?` is itself non-empty (apply (ii') to `Z
    I  Z = Z`), so
`X  I  Y  supseteq  Z` is non-empty, whence `X  I  Y  in  ?` by (ii'). Choice-free. -/
def NeighborhoodSystem.ofPositive {a : Type*} (mem : Set a  ->  Prop) (master : Set a)
    (master_nonempty : master.Nonempty) (master_mem : mem master)
    (sub_master : forall  {X : Set a}, mem X  ->  X  subseteq  master)
    (pos : forall  ?X Y : Set a?, mem X  ->  mem Y  ->  (mem (X  I  Y)  <->  (X  I  Y)
    .Nonempty)) :
    NeighborhoodSystem a where
  mem
  master
  master_nonempty
  master_mem
  sub_master
  inter_mem := by
    intro X Y Z hX hY hZ hZsub
    have hZZ : mem (Z  I  Z) := by rwa [Set.inter_self]
    have hZne : (Z  I  Z).Nonempty := (pos hZ hZ).mp hZZ
    rw [Set.inter_self] at hZne
```

```
    exact (pos hX hY).mpr (hZne.mono hZsub)

end DomainTheory.NeighbourhoodSystems
end

section
-- Exercise119/Tok.lean
set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems.Exercise119

open DomainTheory.NeighbourhoodSystems

/-- Tokens `Delta = {0, 1, 2}`. -/
abbrev Tok := Fin 3

end DomainTheory.NeighbourhoodSystems.Exercise119
end

section
-- Exercise119/a.lean
set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems.Exercise119

open DomainTheory.NeighbourhoodSystems

/-- The neighbourhood `{0, 1}`. -/
def a : Set Tok := {0, 1}

end DomainTheory.NeighbourhoodSystems.Exercise119
end

section
-- Exercise119/b.lean
set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems.Exercise119

open DomainTheory.NeighbourhoodSystems

/-- The neighbourhood `{1, 2}`. -/
def b : Set Tok := {1, 2}

end DomainTheory.NeighbourhoodSystems.Exercise119
end

section
-- Exercise119/master.lean
set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems.Exercise119

open DomainTheory.NeighbourhoodSystems

/-- The master neighbourhood `Delta`. -/
```

```
def master : Set Tok := Set.univ

end DomainTheory.NeighbourhoodSystems.Exercise119
end

section
-- Exercise119/memSet.lean
set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems.Exercise119

open DomainTheory.NeighbourhoodSystems

/-- `? = {Delta, {0,1}, {1,2}}`. -/
def memSet : Set (Set Tok) := {master, a, b}

end DomainTheory.NeighbourhoodSystems.Exercise119
end

section
-- Exercise119/mem.lean
set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems.Exercise119

open DomainTheory.NeighbourhoodSystems

/-- Membership in `?`. -/
def mem (X : Set Tok) : Prop := X  in  memSet

end DomainTheory.NeighbourhoodSystems.Exercise119
end

section
-- Exercise119/mem_iff.lean
set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems.Exercise119

open DomainTheory.NeighbourhoodSystems

theorem mem_iff (X : Set Tok) : mem X  <->  X = master  \/  X = a  \/  X = b := by
  simp only [mem, memSet, Set.mem_insert_iff, Set.mem_singleton_iff]

end DomainTheory.NeighbourhoodSystems.Exercise119
end

section
-- Exercise119/inter_eq.lean
set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems.Exercise119

open DomainTheory.NeighbourhoodSystems

/-- Classification of binary intersections in `?`. -/
```

```
theorem inter_eq (X Y : Set Tok) (hX : mem X) (hY : mem Y) :
    X  I  Y = master  \/  X  I  Y = a  \/  X  I  Y = b  \/  X  I  Y = ({1} : Set Tok)
    := by
  rcases (mem_iff X).mp hX with rfl | rfl | rfl
  * rcases (mem_iff Y).mp hY with rfl | rfl | rfl
    * exact Or.inl (by ext x; fin_cases x <;> simp [master, a, b])
    * exact Or.inr (Or.inl (by ext x; fin_cases x <;> simp [master, a, b]))
    * exact Or.inr (Or.inr (Or.inl (by ext x; fin_cases x <;> simp [master, a, b])))
  * rcases (mem_iff Y).mp hY with rfl | rfl | rfl
    * exact Or.inr (Or.inl (by ext x; fin_cases x <;> simp [master, a, b]))
    * exact Or.inr (Or.inl (by ext x; fin_cases x <;> simp [master, a, b]))
    * exact Or.inr (Or.inr (Or.inr (by ext x; fin_cases x <;> simp [master, a, b])))
  * rcases (mem_iff Y).mp hY with rfl | rfl | rfl
    * exact Or.inr (Or.inr (Or.inl (by ext x; fin_cases x <;> simp [master, a, b])))
    * exact Or.inr (Or.inr (Or.inr (by ext x; fin_cases x <;> simp [master, a, b])))
    * exact Or.inr (Or.inr (Or.inl (by ext x; fin_cases x <;> simp [master, a, b])))

end DomainTheory.NeighbourhoodSystems.Exercise119
end

section
-- Exercise119/mem_a.lean
set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems.Exercise119

open DomainTheory.NeighbourhoodSystems

theorem mem_a : mem a := (mem_iff _).mpr (Or.inr (Or.inl rfl))

end DomainTheory.NeighbourhoodSystems.Exercise119
end

section
-- Exercise119/mem_b.lean
set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems.Exercise119

open DomainTheory.NeighbourhoodSystems

theorem mem_b : mem b := (mem_iff _).mpr (Or.inr (Or.inr rfl))

end DomainTheory.NeighbourhoodSystems.Exercise119
end

section
-- Exercise119/mem_master.lean
set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems.Exercise119

open DomainTheory.NeighbourhoodSystems

theorem mem_master : mem master := (mem_iff _).mpr (Or.inl rfl)
```

```
end DomainTheory.NeighbourhoodSystems.Exercise119
end

section
-- Exercise119/not_mem_sub_one.lean
set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems.Exercise119

open DomainTheory.NeighbourhoodSystems

/-- No neighbourhood is contained in `{1}` (each contains a token other than `1`). This
     is why
the overlapping pair `{0,1}`, `{1,2}` never triggers condition (ii). -/
theorem not_mem_sub_one {Z : Set Tok} (hZ : mem Z) (hsub : Z  subseteq  ({1} : Set Tok
    )) : False := by
  rcases (mem_iff Z).mp hZ with rfl | rfl | rfl
  * exact absurd (hsub (show (0 : Tok)  in  master by simp [master])) (by simp)
  * exact absurd (hsub (show (0 : Tok)  in  a by simp [a])) (by simp)
  * exact absurd (hsub (show (2 : Tok)  in  b by simp [b])) (by simp)

end DomainTheory.NeighbourhoodSystems.Exercise119
end

section
-- Exercise119/notPositiveSystem.lean
set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems.Exercise119

open DomainTheory.NeighbourhoodSystems

/-- **Exercise 1.19 -- the non-positive example is a neighbourhood system.** The only
    intersection
that is not already a neighbourhood is `{0,1}  I  {1,2} = {1}`, and that case is
    impossible: a
consistency witness `Z  subseteq  {1}` with `Z  in  ?` does not exist (`not_mem_sub_one
    `). -/
def notPositiveSystem : NeighborhoodSystem Tok where
  mem
  master
  master_nonempty := Set.univ_nonempty
  master_mem := mem_master
  sub_master := fun _ => Set.subset_univ _
  inter_mem := by
    intro X Y Z hX hY hZ hZsub
    rcases inter_eq X Y hX hY with h | h | h | h
    * rw [h]; exact mem_master
    * rw [h]; exact mem_a
    * rw [h]; exact mem_b
    * rw [h] at hZsub; exact (not_mem_sub_one hZ hZsub).elim

end DomainTheory.NeighbourhoodSystems.Exercise119
end

section
```

```
-- Exercise119/positiveExample.lean
set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems.Exercise119

open DomainTheory.NeighbourhoodSystems

/-- All non-empty subsets of `Fin 2` form a *positive* neighbourhood system: there `X
    I  Y  in  ?`
literally **is** `(X  I  Y).Nonempty`, so (ii') is `Iff.rfl`. -/
def positiveExample : NeighborhoodSystem (Fin 2) :=
  NeighborhoodSystem.ofPositive (fun X => X.Nonempty) Set.univ
    (by exact <(0 : Fin 2), Set.mem_univ 0>)
    (by exact <(0 : Fin 2), Set.mem_univ 0>) (by intro X _; exact Set.subset_univ X)
    (by intro X Y _ _; exact Iff.rfl)

end DomainTheory.NeighbourhoodSystems.Exercise119
end

section
-- Exercise119/a_inter_b.lean
set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems.Exercise119

open DomainTheory.NeighbourhoodSystems

/-- `{0,1}  I  {1,2} = {1}`. -/
theorem a_inter_b : a  I  b = ({1} : Set Tok) := by ext x; fin_cases x <;> simp [a, b]

end DomainTheory.NeighbourhoodSystems.Exercise119
end

section
-- Exercise119/one_not_mem.lean
set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems.Exercise119

open DomainTheory.NeighbourhoodSystems

/-- `{1}` is not a neighbourhood (it is neither `Delta`, `{0,1}`, nor `{1,2}`). -/
theorem one_not_mem : not  mem ({1} : Set Tok) := by
  intro h
  rcases (mem_iff _).mp h with h | h | h
  * have h0 : (0 : Tok)  in  master := by simp [master]
    rw [ <-  h] at h0; simp at h0
  * have h0 : (0 : Tok)  in  a := by simp [a]
    rw [ <-  h] at h0; simp at h0
  * have h2 : (2 : Tok)  in  b := by simp [b]
    rw [ <-  h] at h2; simp at h2

end DomainTheory.NeighbourhoodSystems.Exercise119
end

section
```

```
-- Exercise119/not_isPositive.lean
set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems.Exercise119

open DomainTheory.NeighbourhoodSystems

/-- **Exercise 1.19 -- the example is not positive.** `{0,1}` and `{1,2}` are
    neighbourhoods whose
intersection `{1}` is non-empty but is *not* a neighbourhood, contradicting (ii'). -/
theorem not_isPositive : not  notPositiveSystem.IsPositive := by
  intro h
  have hab := h (X := a) (Y := b) mem_a mem_b
  rw [a_inter_b] at hab
  exact one_not_mem (hab.mpr <1, rfl>)

end DomainTheory.NeighbourhoodSystems.Exercise119
end

section
-- Exercise119.lean
namespace DomainTheory.NeighbourhoodSystems.Exercise119

set_option linter.unusedVariables false

theorem example_119 : Not
  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsPositive DomainTheory.
    NeighbourhoodSystems.Exercise119.Tok
    DomainTheory.NeighbourhoodSystems.Exercise119.notPositiveSystem) :=
  DomainTheory.NeighbourhoodSystems.Exercise119.not_isPositive

end DomainTheory.NeighbourhoodSystems.Exercise119
end

section
-- Exercise119/positiveExample_isPositive.lean
set_option linter.unusedSimpArgs false

namespace DomainTheory.NeighbourhoodSystems.Exercise119

open DomainTheory.NeighbourhoodSystems

theorem positiveExample_isPositive : positiveExample.IsPositive := by
  intro X Y _ _
  exact Iff.rfl

end DomainTheory.NeighbourhoodSystems.Exercise119
end
```

3.2.27 Exercise 1.20 — upward closed neighbourhoods

Let $\mathcal{D}$ be any neighbourhood system over a set Δ. Let $\Delta' = \mathcal{D}$ and define

$$\mathcal{D}' = \{\uparrow X \mid X \in \mathcal{D}\},$$

where

$$\uparrow X = \{Y \in \mathcal{D} \mid Y \subseteq X\}.$$

Show that $\mathcal{D}'$ is a positive neighbourhood system and that $|\mathcal{D}|$ and $|\mathcal{D}'|$ are isomorphic. Note that for $\mathcal{D}'$ finite elements and tokens are in a one-one correspondence.

Exercise_1_20.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.DomainIso
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.principal
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_19
public import Mathlib.Topology.Defs.Basic
public import Mathlib.Algebra.Group.End

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem
section

variable {a : Type*} (V : NeighborhoodSystem a)

/-- Exercise 1.20's `X = {Y  in  ? ? Y  subseteq  X}` (the up-set of `X` *inside* `?`).
     -/
def upSet (X : Set a) : Set (Set a) := {Y | V.mem Y  /\  Y  subseteq  X}

@[simp] theorem mem_upSet {X Y : Set a} : Y  in  V.upSet X  <->  V.mem Y  /\  Y
    subseteq  X := Iff.rfl

/-- `X  I  Y = (X  I  Y)` -- a set identity (no membership hypotheses). -/
theorem upSet_inter (X Y : Set a) : V.upSet X  I  V.upSet Y = V.upSet (X  I  Y) := by
  ext Z
  simp only [Set.mem_inter_iff, mem_upSet, Set.subset_inter_iff]
  tauto

/-- `X  |->  X` is inclusion-**preserving**: `X  subseteq  Y  <->  X  subseteq  Y` (for
     `X  in  ?`).
` -> ` tests at `X  in  X`; ` <- ` is transitivity of ` subseteq `. -/
theorem upSet_subset_iff {X Y : Set a} (hX : V.mem X) : V.upSet X  subseteq  V.upSet Y
      <->  X  subseteq  Y := by
  constructor
  * intro h
    exact (h <hX, subset_rfl>).2
  * intro hXY Z hZ
    exact <hZ.1, hZ.2.trans hXY>
end

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
end
```

```
section
namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem
section

variable {a : Type*} (V : NeighborhoodSystem a)

/- concordance: source L472 EXERCISE 1.20 upward closed neighbourhoods -/
/-- **Exercise 1.20 -- the power system `?' = {X ? X  in  ?}`** over the tokens `Delta'
    = ?`. The
master is `Delta`; the intersection law uses the consistency witness `Z  in  Z
   subseteq  X  I  Y` to get
`Z  subseteq  X  I  Y`, hence `X  I  Y  in  ?` and `X  I  Y = (X I Y)`. -/
def powerSystem : NeighborhoodSystem (Set a) where
  mem S := exists  X, V.mem X  /\  S = V.upSet X
  master := V.upSet V.master
  master_nonempty := <V.master, V.master_mem, subset_rfl>
  master_mem := <V.master, V.master_mem, rfl>
  inter_mem := by
    rintro S T W <X, hX, rfl> <Y, hY, rfl> <Z, hZ, rfl> hWsub
    have hZin : Z  in  V.upSet X  I  V.upSet Y := hWsub <hZ, subset_rfl>
    obtain <<_, hZX>, _, hZY> := hZin
    have hXY : V.mem (X  I  Y) := V.inter_mem hX hY hZ (Set.subset_inter hZX hZY)
    exact <X  I  Y, hXY, V.upSet_inter X Y>
  sub_master := by
    intro S hS
    obtain <X, hX, rfl> := hS
    exact (V.upSet_subset_iff hX).mpr (V.sub_master hX)

@[simp] theorem powerSystem_mem {S : Set (Set a)} :
    V.powerSystem.mem S  <->  exists  X, V.mem X  /\  S = V.upSet X := Iff.rfl

/-- The element of `|?|` corresponding to `y  in  |?'|`: the filter `{W ? W  in  y}`.
   -/
def ofPower (y : V.powerSystem.Element) : V.Element where
  mem W := V.mem W  /\  y.mem (V.upSet W)
  sub h := h.1
  master_mem := <V.master_mem, y.master_mem>
  inter_mem := by
    rintro X Y <hX, hyX> <hY, hyY>
    have hyInter : y.mem (V.upSet X  I  V.upSet Y) := y.inter_mem hyX hyY
    rw [V.upSet_inter] at hyInter
    obtain <W, hW, hWeq> := y.sub hyInter
    have hWin : W  in  V.upSet (X  I  Y) := by rw [hWeq]; exact <hW, subset_rfl>
    exact <V.inter_mem hX hY hW hWin.2, hyInter>
  up_mem := by
    rintro X Y <hX, hyX> hY hXY
    exact <hY, y.up_mem hyX <Y, hY, rfl> ((V.upSet_subset_iff hX).mpr hXY)>

/-- The element of `|?'|` corresponding to `x  in  |?|`: the filter `{W ? W  in  x}`.
   -/
def toPower (x : V.Element) : V.powerSystem.Element where
  mem S := exists  W, x.mem W  /\  S = V.upSet W
  sub := by rintro S <W, hW, rfl>; exact <W, x.sub hW, rfl>
  master_mem := <V.master, x.master_mem, rfl>
  inter_mem := by
```

```
    rintro S T <W1, hW1, rfl> <W2, hW2, rfl>
    exact <W1  I  W2, x.inter_mem hW1 hW2, V.upSet_inter W1 W2>
  up_mem := by
    rintro S T <W, hW, rfl> hT hsub
    obtain <W', hW', rfl> := hT
    exact <W', x.up_mem hW hW' ((V.upSet_subset_iff (x.sub hW)).mp hsub), rfl>

/-- `X  |->  X` is one-one on `?`. -/
theorem upSet_injective {X Y : Set a} (hX : V.mem X) (hY : V.mem Y)
    (h : V.upSet X = V.upSet Y) : X = Y :=
  Set.Subset.antisymm ((V.upSet_subset_iff hX).mp h.subset)
    ((V.upSet_subset_iff hY).mp h.superset)

/-- **Exercise 1.20 (the isomorphism).** `toPower`/`ofPower` are mutually inverse and
preserve/reflect ` sqsub `, giving `|?| ?o |?'|`. -/
def powerIso : V.Element ?o V.powerSystem.Element where
  toFun := V.toPower
  invFun := V.ofPower
  left_inv := by
    intro x
    apply Element.ext
    intro W
    constructor
    * rintro <hW, W', hW', heq>
      rw [V.upSet_injective hW (x.sub hW') heq]; exact hW'
    * intro hx
      exact <x.sub hx, W, hx, rfl>
  right_inv := by
    intro y
    apply Element.ext
    intro S
    constructor
    * rintro <W, <_, hyW>, rfl>
      exact hyW
    * intro hyS
      obtain <W, hW, rfl> := y.sub hyS
      exact <W, <hW, hyS>, rfl>
  map_rel_iff' := by
    intro x x'
    constructor
    * intro h W hx
      have hmem : (V.toPower x).mem (V.upSet W) := <W, hx, rfl>
      obtain <W', hW', heq> := h (V.upSet W) hmem
      rw [V.upSet_injective (x.sub hx) (x'.sub hW') heq]; exact hW'
    * intro h S hS
      obtain <W, hW, rfl> := hS
      exact <W, h W hW, rfl>

/-- **Exercise 1.20 (statement).** `?` and its power system `?'` are isomorphic. -/
theorem isomorphic_powerSystem : V  ~= ? V.powerSystem := <V.powerIso>

/-- **Exercise 1.20 -- `?'` is positive.** `X  I  Y = (X I Y)` is a neighbourhood iff
    it is
non-empty: a witness `Z  subseteq  X  I  Y` gives `X  I  Y  in  ?`; conversely `W`
    always contains `W`. -/
theorem powerSystem_isPositive : V.powerSystem.IsPositive := by
```

```
  rintro S T <X, hX, rfl> <Y, hY, rfl>
  rw [V.upSet_inter]
  constructor
  * rintro <W, hW, hWeq>
    rw [hWeq]
    exact <W, hW, subset_rfl>
  * rintro <Z, hZmem, hZsub>
    exact <X  I  Y, V.inter_mem hX hY hZmem hZsub, rfl>

/-- **Exercise 1.20 -- tokens  <->  finite elements.** The iso carries the finite
    element `X` of
`|?|` (Def 1.7) to the finite element of `|?'|` generated by the token `X`, i.e. the
    principal
filter of `X` in `?'`. Since `X  |->  X` is one-one (`upSet_injective`), tokens of `?'`
    and its
finite elements are in one-one correspondence. -/
theorem toPower_principal {X : Set a} (hX : V.mem X) :
    V.toPower (V.principal hX) = V.powerSystem.principal <X, hX, rfl> := by
  apply Element.ext
  intro S
  constructor
  * rintro <W, <hWmem, hXW>, rfl>
    exact <<W, hWmem, rfl>, (V.upSet_subset_iff hX).mpr hXW>
  * rintro <<W, hWmem, rfl>, hsub>
    exact <W, <hWmem, (V.upSet_subset_iff hX).mp hsub>, rfl>
end

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
section

set_option linter.unusedVariables false

theorem example_120.{u_1} : And
  (And
    (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
    a),
      @DomainTheory.NeighbourhoodSystems.Isomorphic a (Set a) V
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.powerSystem a V))
    (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
    a),
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsPositive (Set a)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.powerSystem a V)))
  (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a)
     {X : Set a}
    (hX : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X),
    @Eq
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Set a)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.powerSystem a V))
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.toPower a V
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.principal a V X hX))
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.principal (Set a)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.powerSystem a V)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.upSet a V X)
        (@Exists.intro (Set a)
```

```
       (fun (X_1 : Set a) =>
         And (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X_1)
           (@Eq (Set (Set a)) (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.upSet a V X)
             (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.upSet a V X_1))
  )
       X
       (@And.intro (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X
  )
         (@Eq (Set (Set a)) (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  upSet a V X)
           (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.upSet a V X))
         hX (@rfl (Set (Set a)) (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.upSet a V X))))))) :=
@And.intro
(And
  (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
  a),
    @DomainTheory.NeighbourhoodSystems.Isomorphic a (Set a) V
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.powerSystem a V))
  (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
  a),
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsPositive (Set a)
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.powerSystem a V)))
(forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a)
   {X : Set a}
  (hX : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X),
  @Eq
    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Set a)
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.powerSystem a V))
    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.toPower a V
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.principal a V X hX))
    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.principal (Set a)
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.powerSystem a V)
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.upSet a V X)
      (@Exists.intro (Set a)
        (fun (X_1 : Set a) =>
          And (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X_1)
            (@Eq (Set (Set a)) (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.upSet a V X)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.upSet a V X_1))
  )
       X
       (@And.intro (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X
  )
         (@Eq (Set (Set a)) (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  upSet a V X)
           (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.upSet a V X))
         hX (@rfl (Set (Set a)) (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.upSet a V X))))))
(@And.intro
  (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
  a),
    @DomainTheory.NeighbourhoodSystems.Isomorphic a (Set a) V
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.powerSystem a V))
  (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
```

```
    a),
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsPositive (Set a)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.powerSystem a V))
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.isomorphic_powerSystem
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.powerSystem_isPositive)
  @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.toPower_principal
end

end DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem
section

variable {a : Type*} (V : NeighborhoodSystem a)

/-- Scott's `[X] = {x  in  |?| ? X  in  x}` (Theorem 1.10 notation): the set of
    elements of the domain
`|?|` that contain the neighbourhood `X`. These sets are the basic opens of the
    topology of
Exercise 1.22. -/
def basicOpen (X : Set a) : Set V.Element := {x | x.mem X}

@[simp] theorem mem_basicOpen {X : Set a} {x : V.Element} :
    x  in  V.basicOpen X  <->  x.mem X := Iff.rfl

/-- A set `?  subseteq  |?|` is *open* (Exercise 1.22, condition (ii)) when every point
     `x  in  ?` has a
basic neighbourhood `[X]` with `X  in  x` contained in `?`. -/
class OpenFilter (U : Set V.Element) : Prop where
  out : forall  x  in  U, exists  X, x.mem X  /\  V.basicOpen X  subseteq  U

/-- `[X  I  Y]  subseteq  [X]` whenever `X  in  ?`: a filter containing `X  I  Y`
    contains `X` (upward closure).
This (with the symmetric version) is the closure of the basic opens under finite
    intersection,
i.e. `[X]  I  [Y] = [X  I  Y]`, the base condition behind the topology. -/
theorem basicOpen_inter_subset_left {X Y : Set a} (hX : V.mem X) :
    V.basicOpen (X  I  Y)  subseteq  V.basicOpen X :=
  fun z hz => z.up_mem hz hX Set.inter_subset_left

/-- `[X  I  Y]  subseteq  [Y]` whenever `Y  in  ?`. -/
theorem basicOpen_inter_subset_right {X Y : Set a} (hY : V.mem Y) :
    V.basicOpen (X  I  Y)  subseteq  V.basicOpen Y :=
  fun z hz => z.up_mem hz hY Set.inter_subset_right

/-- **Exercise 1.22 (the space).** The basic opens `[X]` (`X  in  ?`) generate a
    topology on `|?|`:
`?` is open iff it is a union of basic opens (condition (ii)). The three axioms hold
    because the
base is closed under finite intersection (`basicOpen_inter_subset_left/right`, using
    that filters
are ` I `-closed and upward closed) with `[Delta] = |?|` covering the space. -/
instance instTopologicalSpaceElement : TopologicalSpace V.Element where
  IsOpen := V.OpenFilter
  isOpen_univ := <fun x _ => <V.master, x.master_mem, Set.subset_univ _>>
  isOpen_inter := by
```

```
    intro U W hU hW
    constructor
    intro x hx
    obtain <hxU, hxW> := hx
    obtain <X, hX, hXU> := hU.out x hxU
    obtain <Y, hY, hYW> := hW.out x hxW
    refine <X  I  Y, x.inter_mem hX hY, fun z hz => <hXU ?_, hYW ?_>>
    * exact V.basicOpen_inter_subset_left (x.sub hX) hz
    * exact V.basicOpen_inter_subset_right (x.sub hY) hz
  isOpen_sUnion := by
    intro S hS
    constructor
    intro x hx
    obtain <t, htS, hxt> := hx
    obtain <X, hX, hXt> := (hS t htS).out x hxt
    exact <X, hX, hXt.trans fun _ ha => <t, htS, ha>>
end

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
end
```

3.2.28 Exercise 1.21 — complete neighbourhood systems

Work out in greater detail the proof of 1.10. Remark that the neighbourhood system over $|\mathcal{D}|$ so constructed is positive, thereby obtaining in a different way the same kind of conclusion as in 1.20. Show also that the system over $|\mathcal{D}|$ is complete in the sense that every filter is fixed by a unique member of the underlying set. (A filter is *fixed by a point* iff it is the filter of *all* neighbourhoods containing that point.) Remark that a complete system is one where tokens and (partial) elements can always be identified (under a suitable one-one correspondence). Show also that consistency of a set $\{X_i \mid i < n\}$ of neighbourhoods in $\mathcal{D}$ is equivalent to saying

$$\bigcap_{i<n} [X_i] \neq \emptyset.$$

Exercise_1_21.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Theorem_1_10
import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.Factoid_1_5

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem
section

variable {a : Type*} (V : NeighborhoodSystem a)

/- concordance: source L486 EXERCISE 1.21 complete neighbourhood systems -/
```

```
/-- **Exercise 1.21 -- consistency via the intersection `[?]`.** For `X_0, ..., X_n?_1
     in  ?`,
`<X_i>` is consistent iff `[?_{i<n} X_i]  !=  {}`. Combines Theorem 1.1c
(`consistent_iff_interUpTo_mem`) with `bracket_nonempty_iff`. -/
theorem consistent_iff_bracket_interUpTo_nonempty (X : Nat  ->  Set a) {n : Nat}
    (hX : forall  i, i < n  ->  V.mem (X i)) :
    V.Consistent X n  <->  (V.bracket (V.interUpTo X n)).Nonempty := by
  rw [V.consistent_iff_interUpTo_mem X hX]
  exact V.bracket_nonempty_iff.symm

/-- **Exercise 1.21 -- consistency `? ?_{i<n} [X_i]  !=  {}`.** The literal form of
    Scott's
equivalence: `<X_i>` is consistent iff there is a filter lying in every `[X_i]` (`i < n
    `). ` -> `
uses upward closure along `?_{i<n} X_i  subseteq  X_i`; ` <- ` uses `Element.
    mem_interUpTo`. -/
theorem consistent_iff_iInter_bracket_nonempty (X : Nat  ->  Set a) {n : Nat}
    (hX : forall  i, i < n  ->  V.mem (X i)) :
    V.Consistent X n  <->  exists  x : V.Element, forall  i, i < n  ->  x  in  V.
    bracket (X i) := by
  rw [V.consistent_iff_bracket_interUpTo_nonempty X hX]
  constructor
  * rintro <x, hx>
    exact <x, fun i hi => x.up_mem hx (hX i hi) (V.interUpTo_subset X hi)>
  * rintro <x, hx>
    exact <x, x.mem_interUpTo X (fun i hi => hx i hi)>

/-- **Exercise 1.21 -- the token system is complete.** Every filter `y` of `{[X]}` is
    fixed by
the unique point `ofToken y  in  |?|`: `[W]  in  y  <->  ofToken y  in  [W]`.
    Uniqueness is `Element.ext`
applied through the brackets. -/
theorem tokenSystem_complete : V.tokenSystem.IsComplete := by
  intro y
  refine <V.ofToken y, ?_, ?_>
  * rintro S <W, hW, rfl>
    constructor
    * intro hyW; exact <hW, hyW>
    * rintro <_, hyW>; exact hyW
  * intro b hb
    apply Element.ext
    intro W
    by_cases hW : V.mem W
    * have hb' := hb (V.bracket W) <W, hW, rfl>
      constructor
      * intro hbW; exact <hW, hb'.mpr hbW>
      * intro hxW; exact hb'.mp hxW.2
    * constructor
      * intro hbW; exact absurd (b.sub hbW) hW
      * intro hxW; exact absurd ((V.ofToken y).sub hxW) hW

/-- **Exercise 1.21 -- the token system is positive.** `[X]  I  [Y] = [X  I  Y]` is a
    neighbourhood
iff non-empty: a common element `x` gives `X  I  Y  in  ?` (via `x.sub (x.inter_mem
    ...)`); conversely
`[W]` always contains the principal filter `W`. -/
```

```
theorem tokenSystem_isPositive : V.tokenSystem.IsPositive := by
  rintro S T <X, hX, rfl> <Y, hY, rfl>
  rw [V.bracket_inter hX hY]
  constructor
  * rintro <W, hW, hWeq>
    rw [hWeq]
    exact <V.principal hW, V.principal_mem_bracket hW>
  * rintro <x, hx>
    exact <X  I  Y, x.sub hx, rfl>

/-- **Exercise 1.21 -- tokens  <->  elements.** `toToken : |?|  ->  |{[X]}|` is a
    bijection (it is the
underlying map of the order-iso `tokenIso`), so a complete system identifies tokens
    with
(partial) elements under a one-one correspondence. -/
theorem tokenSystem_toToken_bijective : Function.Bijective V.toToken :=
  V.tokenIso.bijective
end

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
section

set_option linter.unusedVariables false

theorem example_121.{u_1} : And
  (And
    (And
      (And
        (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem a) (X : Nat  ->  Set a) {n : Nat},
          (forall  (i : Nat),
              @LT.lt Nat instLTNat i n  ->  @DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.mem a V (X i))  ->
            Iff (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Consistent a V
    X n)
              (@Set.Nonempty (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    Element a V)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bracket a V
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.interUpTo a V
     X n))))
        (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem a) (X : Nat  ->  Set a) {n : Nat},
          (forall  (i : Nat),
              @LT.lt Nat instLTNat i n  ->  @DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.mem a V (X i))  ->
            Iff (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Consistent a V
    X n)
              (exists  (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    Element a V),
                forall  (i : Nat),
                  @LT.lt Nat instLTNat i n  ->
                    @Membership.mem (@DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.Element a V)
                      (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
```

```
  Element a V))
                    (@Set.instMembership (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V))
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bracket a
   V (X i)) x)))
     (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem a),
       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsComplete
         (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
         (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.tokenSystem a V)))
   (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
  a),
     @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsPositive
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.tokenSystem a V)))
(forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a)
  ,
  @Function.Bijective (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
   a V)
     (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.tokenSystem a V))
     (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.toToken a V)) :=
@And.intro
(And
  (And
    (And
      (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem a) (X : Nat  ->  Set a) {n : Nat},
        (forall  (i : Nat),
           @LT.lt Nat instLTNat i n  ->  @DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.mem a V (X i))  ->
         Iff (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Consistent a V
  X n)
           (@Set.Nonempty (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  Element a V)
             (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bracket a V
               (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.interUpTo a V
   X n))))
      (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem a) (X : Nat  ->  Set a) {n : Nat},
        (forall  (i : Nat),
           @LT.lt Nat instLTNat i n  ->  @DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.mem a V (X i))  ->
         Iff (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Consistent a V
  X n)
           (exists  (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  Element a V),
             forall  (i : Nat),
               @LT.lt Nat instLTNat i n  ->
                 @Membership.mem (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
                   (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  Element a V))
                   (@Set.instMembership (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V))
```

```
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bracket a
   V (X i)) x)))
     (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem a),
       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsComplete
         (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
         (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.tokenSystem a V)))
   (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
  a),
     @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsPositive
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.tokenSystem a V)))
(forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a)
  ,
  @Function.Bijective (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
    a V)
     (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.tokenSystem a V))
     (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.toToken a V))
(@And.intro
   (And
     (And
       (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem a) (X : Nat  ->  Set a) {n : Nat},
         (forall  (i : Nat),
             @LT.lt Nat instLTNat i n  ->  @DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.mem a V (X i))  ->
           Iff (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Consistent a V
  X n)
             (@Set.Nonempty (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  Element a V)
               (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bracket a V
                 (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.interUpTo a V
    X n))))
       (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem a) (X : Nat  ->  Set a) {n : Nat},
         (forall  (i : Nat),
             @LT.lt Nat instLTNat i n  ->  @DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.mem a V (X i))  ->
           Iff (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Consistent a V
  X n)
             (exists  (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  Element a V),
               forall  (i : Nat),
                 @LT.lt Nat instLTNat i n  ->
                   @Membership.mem (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
                     (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  Element a V))
                     (@Set.instMembership (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V))
                     (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bracket a
    V (X i)) x)))
     (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem a),
```

```
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsComplete
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.tokenSystem a V)))
(forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
a),
  @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsPositive
    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.tokenSystem a V))
(@And.intro
  (And
    (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a) (X : Nat  ->  Set a) {n : Nat},
      (forall  (i : Nat),
          @LT.lt Nat instLTNat i n  ->  @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem a V (X i))  ->
        Iff (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Consistent a V
X n)
          (@Set.Nonempty (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a V)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bracket a V
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.interUpTo a V
 X n))))
    (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a) (X : Nat  ->  Set a) {n : Nat},
      (forall  (i : Nat),
          @LT.lt Nat instLTNat i n  ->  @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem a V (X i))  ->
        Iff (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Consistent a V
X n)
          (exists  (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a V),
            forall  (i : Nat),
              @LT.lt Nat instLTNat i n  ->
                @Membership.mem (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                  (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a V))
                  (@Set.instMembership (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V))
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bracket a
 V (X i)) x)))
  (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a),
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsComplete
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.tokenSystem a V))
  (@And.intro
    (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a) (X : Nat  ->  Set a) {n : Nat},
      (forall  (i : Nat),
          @LT.lt Nat instLTNat i n  ->  @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem a V (X i))  ->
        Iff (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Consistent a V
X n)
          (@Set.Nonempty (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a V)
```

```
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bracket a V
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.interUpTo a V
     X n))))
         (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem a) (X : Nat  ->  Set a) {n : Nat},
           (forall  (i : Nat),
              @LT.lt Nat instLTNat i n  ->  @DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.mem a V (X i))  ->
            Iff (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Consistent a V
    X n)
             (exists  (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    Element a V),
               forall  (i : Nat),
                 @LT.lt Nat instLTNat i n  ->
                   @Membership.mem (@DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.Element a V)
                     (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    Element a V))
                     (@Set.instMembership (@DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.Element a V))
                     (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bracket a
     V (X i)) x))
         @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    consistent_iff_bracket_interUpTo_nonempty
         @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    consistent_iff_iInter_bracket_nonempty)
       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.tokenSystem_complete)
     @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.tokenSystem_isPositive)
  @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.tokenSystem_toToken_bijective
end

end DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
end
```

3.2.29 Exercise 1.22 — topology of a domain

(For topologists). Show that the neighbourhoods $[X]$ for $X \in \mathcal{D}$ make $|\mathcal{D}|$ into a topological space where the open subsets $\mathcal{U} \subseteq |\mathcal{D}|$ can be characterized by the following two conditions:

(i) whenever $x \in \mathcal{U}$ and $x \sqsubseteq y \in |\mathcal{D}|$, then $y \in \mathcal{U}$; and

(ii) whenever $x \in \mathcal{U}$, then $[X] \in \mathcal{U}$ for some $X \in x$.

Prove also that the inclusion relation on $|\mathcal{D}|$ can be defined topologically as:

(iii) $x \sqsubseteq y$ iff for all open $\mathcal{U} \subseteq |\mathcal{D}|$, if $x \in \mathcal{U}$ then $y \in \mathcal{U}$.

Is $|D|$ ever a Hausdorff space?
Show that if $(x_n)_{n=0}^{\infty}$ is a sequence of elements of $|D|$ with $x_n \subseteq x_{n+1}$ for all n, then

$$\bigcup_{n=0}^{\infty} x_n$$

is not only in $|D|$ but is a topological limit point of the sequence. Show that any element x is a limit point of the set $\{\uparrow X \mid X \in x\}$. Are there other limit points?

Exercise_1_22.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_20
public import Mathlib.Topology.Defs.Filter

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem
section

variable {a : Type*} (V : NeighborhoodSystem a)

/- concordance: source L492 EXERCISE 1.22 topology of a domain -/
/-- **Exercise 1.22.** Each basic neighbourhood `[X]` is open. -/
theorem isOpen_basicOpen (X : Set a) : IsOpen (V.basicOpen X) :=
  <fun _ hx => <X, hx, subset_rfl>>

/-- `IsOpen` for `|?|` is exactly Scott's condition (ii). -/
theorem isOpen_iff_isOpenFilter (U : Set V.Element) : IsOpen U  <->  V.OpenFilter U :=
  <fun h => h, fun h => h>

/-- **Exercise 1.22, condition (i).** Every open set is upward closed under the
    approximation order
` sqsub `: if `x  in  ?` and `x  sqsub  y` then `y  in  ?`. (This is a *consequence* of
     (ii): the basic
neighbourhood `[X]  subseteq  ?` witnessing `x  in  ?` also contains every `y ? x`.) -/
theorem isOpen_isUpperSet {U : Set V.Element} (hU : IsOpen U) :
    forall  ?x y : V.Element?, x  in  U  ->  x  <=  y  ->  y  in  U := by
  intro x y hxU hxy
  obtain <X, hX, hXU> := hU.out x hxU
  exact hXU (hxy X hX)

/-- **Exercise 1.22 (characterization of open sets).** A subset `?  subseteq  |?|` is
    open iff
(i) it is upward closed under ` sqsub `, and (ii) every point of `?` has a basic
    neighbourhood `[X]`
(`X  in  x`) contained in `?`. -/
theorem isOpen_iff_upper_basic (U : Set V.Element) :
    IsOpen U  <->
      (forall  ?x y : V.Element?, x  in  U  ->  x  <=  y  ->  y  in  U)  /\
        (forall  x  in  U, exists  X, x.mem X  /\  V.basicOpen X  subseteq  U) := by
  constructor
  * intro hU
    exact <V.isOpen_isUpperSet hU, hU.out>
  * rintro <_, h2>
    exact <h2>

/-- **Exercise 1.22, condition (iii).** The approximation order is recovered from the
```

```
    topology:
`x  sqsub  y` iff every open set containing `x` also contains `y`.

* ` -> ` is upward closure of opens (`isOpen_isUpperSet`);
* ` <- ` tests against the open basic neighbourhood `[X]` for each `X  in  x`. -/
theorem le_iff_isOpen_imp (x y : V.Element) :
    x  <=  y  <->  forall  U : Set V.Element, IsOpen U  ->  x  in  U  ->  y  in  U :=
    by
  constructor
  * intro hxy U hU hxU
    exact V.isOpen_isUpperSet hU hxU hxy
  * intro h
    exact fun X hX => h (V.basicOpen X) (V.isOpen_basicOpen X) hX

/-- **Exercise 1.22 (assembled).** Open sets are upward closed; basic opens are open;
    opens are
exactly the upper sets covered by basic opens; and ` sqsub ` is specialization in the
    Scott topology. -/
example :
    (forall  {U : Set V.Element}, IsOpen U  ->  forall  ?x y : V.Element?, x  in  U
    ->  x  <=  y  ->  y  in  U)  /\
      (forall  X : Set a, IsOpen (V.basicOpen X))  /\
      (forall  U : Set V.Element,
        IsOpen U  <->
          (forall  ?x y : V.Element?, x  in  U  ->  x  <=  y  ->  y  in  U)  /\
            (forall  x  in  U, exists  X, x.mem X  /\  V.basicOpen X  subseteq  U))
    /\
      (forall  x y : V.Element, x  <=  y  <->  forall  U : Set V.Element, IsOpen U  ->
      x  in  U  ->  y  in  U) :=
  <V.isOpen_isUpperSet, V.isOpen_basicOpen, V.isOpen_iff_upper_basic, V.
    le_iff_isOpen_imp>

/-- The approximation order ` sqsub ` is the opposite of Mathlib's specialization
    preorder `?`:
`y ? x  <->  x  sqsub  y`. (Scott's (iii) says exactly that ` sqsub ` is the
    specialization order of this
topology.) -/
theorem specializes_iff_le (x y : V.Element) : y ? x  <->  x  <=  y := by
  rw [specializes_iff_forall_open]
  exact (V.le_iff_isOpen_imp x y).symm
end

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
end
```

3.2.30 Exercise 1.23 — decidable total elements

Suppose that the neighbourhood system D is countable, say,

$$D = \{X_0, X_1, X_2, \ldots, X_n, \ldots\}.$$

Suppose further that the property of consistency of finite sequences of neighbourhoods is decidable (or "effectively known"). Then the following sequence is well defined:

$$
\begin{aligned}
Y_0 &= X_0 \\
Y_{n+1} &= X_{n+1}, \text{ if this set is consistent with } Y_0, Y_1, \ldots, Y_n; \\
&= Y_n, \text{ if not.}
\end{aligned}
$$

Show that $\{Y_0, Y_1, \ldots, Y_n, \ldots\}$ is a total element of $|D|$. (Hint: Show first that $Y_0, Y_1, \ldots, Y_{n-1}$ is consistent for all n.) In such a system show that all filters can be determined by sequences.

Exercise_1_23.lean

```
module

import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.Exercise_1_13
import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.Factoid_1_5
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.DomainIso

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.Exercise123
section

open NeighborhoodSystem
variable {a : Type*} (V : NeighborhoodSystem a) (enum : Nat  ->  Set a) [DecidablePred
    V.mem]

/-- Greedy *state*: `state n = (Y_n, ?_{i <= n} Y_i)`. The running intersection in the
    second
component lets the consistency test `V.mem (acc  I  enum (n+1))` be self-contained (
    Theorem 1.1c:
the prefix `Y_0,...,Y_n,enum (n+1)` is consistent iff this intersection is a
    neighbourhood). -/
def state : Nat  ->  Set a  x  Set a
  | 0 => (enum 0, V.master  I  enum 0)
  | (n + 1) =>
      if V.mem ((state n).2  I  enum (n + 1)) then
        (enum (n + 1), (state n).2  I  enum (n + 1))
      else
        state n

/-- The greedy sequence `Y_n = (state n).1`. -/
def Y (n : Nat) : Set a := (state V enum n).1

@[simp] theorem Y_zero : Y V enum 0 = enum 0 := rfl

/-- The running intersection `acc n = ?_{i <= n} Y_i = (state n).2`. -/
def acc (n : Nat) : Set a := (state V enum n).2

@[simp] theorem acc_zero : acc V enum 0 = V.master  I  enum 0 := rfl

/-- The consistency test at stage `n+1`: is `enum (n+1)` consistent with the prefix `
    Y_0,...,Y_n`?
An `abbrev` so the `[DecidablePred V.mem]` instance is found through it. -/
abbrev cond (n : Nat) : Prop := V.mem (acc V enum n  I  enum (n + 1))

/-- The defining equation of `state` at a successor, written with the named `cond`/`acc
    `. -/
```

```
theorem state_succ (n : Nat) :
    state V enum (n + 1) =
      if cond V enum n then (enum (n + 1), acc V enum n  I  enum (n + 1)) else state V
     enum n :=
  rfl

theorem Y_succ_neg {n : Nat} (h : not  cond V enum n) : Y V enum (n + 1) = Y V enum n
    := by
  show (state V enum (n + 1)).1 = (state V enum n).1
  rw [state_succ, ite_eq_right h]

theorem Y_succ_pos {n : Nat} (h : cond V enum n) : Y V enum (n + 1) = enum (n + 1) :=
    by
  show (state V enum (n + 1)).1 = enum (n + 1)
  rw [state_succ, ite_eq_left h]

theorem acc_succ_neg {n : Nat} (h : not  cond V enum n) :
    acc V enum (n + 1) = acc V enum n := by
  show (state V enum (n + 1)).2 = (state V enum n).2
  rw [state_succ, ite_eq_right h]

theorem acc_succ_pos {n : Nat} (h : cond V enum n) :
    acc V enum (n + 1) = acc V enum n  I  enum (n + 1) := by
  show (state V enum (n + 1)).2 = acc V enum n  I  enum (n + 1)
  rw [state_succ, ite_eq_left h]

/-- `acc n = ?_{i<n+1} Y_i`: the carried intersection is exactly the prefix
    intersection. -/
theorem acc_eq_interUpTo : forall  n, acc V enum n = V.interUpTo (Y V enum) (n + 1) :=
     by
  intro n
  induction n with
  | zero =>
    rw [acc_zero, interUpTo_succ, interUpTo_zero, Y_zero]
  | succ n ih =>
    rw [interUpTo_succ,  <-  ih]
    by_cases h : cond V enum n
    * rw [acc_succ_pos V enum h, Y_succ_pos V enum h]
    * rw [acc_succ_neg V enum h, Y_succ_neg V enum h]
      have hsub : acc V enum n  subseteq  Y V enum n := by
        rw [ih]; exact V.interUpTo_subset (Y V enum) (Nat.lt_succ_self n)
      rw [Set.inter_eq_left.mpr hsub]

section
variable (henum : forall  n, V.mem (enum n))

include henum

/-- The running intersection `acc n` is always a neighbourhood -- the greedy choice
    guarantees it.
This is the content of Scott's hint (consistency of the prefix). -/
theorem acc_mem : forall  n, V.mem (acc V enum n) := by
  intro n
  induction n with
  | zero =>
    rw [acc_zero, Set.inter_eq_right.mpr (V.sub_master (henum 0))]
```

```
    exact henum 0
  | succ n ih =>
    by_cases h : cond V enum n
    * rw [acc_succ_pos V enum h]; exact h
    * rw [acc_succ_neg V enum h]; exact ih

include henum

/-- `?_{i<n} Y_i` is always a neighbourhood. -/
theorem mem_interUpTo_Y : forall  n, V.mem (V.interUpTo (Y V enum) n) := by
  intro n
  cases n with
  | zero => rw [interUpTo_zero]; exact V.master_mem
  | succ n => rw [ <-  acc_eq_interUpTo V enum]; exact acc_mem V enum henum n
end
end

end DomainTheory.NeighbourhoodSystems.Exercise123

section
namespace DomainTheory.NeighbourhoodSystems
namespace Exercise123
section

open NeighborhoodSystem
variable {a : Type*} (V : NeighborhoodSystem a) (enum : Nat  ->  Set a) [DecidablePred
     V.mem]
variable (henum : forall  n, V.mem (enum n))

include henum

/- concordance: source L514 EXERCISE 1.23 decidable total elements -/
/-- **Exercise 1.23 -- the greedy element.** The filter generated by all `Y_n`:
`{X  in  ? ? ?_{i<n} Y_i  subseteq  X for some n}`. The prefix intersections `?_{i<n}
    Y_i` are a descending
chain of neighbourhoods, so this is a genuine filter. -/
def greedyElement : V.Element where
  mem X := V.mem X  /\  exists  n, V.interUpTo (Y V enum) n  subseteq  X
  sub h := h.1
  master_mem := <V.master_mem, 0, by rw [interUpTo_zero]>
  inter_mem := by
    rintro X X' <hX, n, hn> <hX', m, hm>
    have hsub : V.interUpTo (Y V enum) (max n m)  subseteq  X  I  X' :=
      Set.subset_inter
        ((V.interUpTo_antitone (Y V enum) (le_max_left n m)).trans hn)
        ((V.interUpTo_antitone (Y V enum) (le_max_right n m)).trans hm)
    exact <V.inter_mem hX hX' (mem_interUpTo_Y V enum henum (max n m)) hsub, max n m,
    hsub>
  up_mem := by
    rintro X X' <_, n, hn> hX' hsub
    exact <hX', n, hn.trans hsub>
end

open NeighborhoodSystem

variable {a : Type*} (V : NeighborhoodSystem a) (enum : Nat  ->  Set a) [DecidablePred
```

```
     V.mem]

variable (henum : forall  n, V.mem (enum n))
include henum

/-- Each `Y_n` is a neighbourhood. -/
theorem Y_mem : forall  n, V.mem (Y V enum n) := by
  intro n
  induction n with
  | zero => rw [Y_zero]; exact henum 0
  | succ n ih =>
    by_cases h : cond V enum n
    * rw [Y_succ_pos V enum h]; exact henum (n + 1)
    * rw [Y_succ_neg V enum h]; exact ih

/-- **Exercise 1.23 -- the hint.** For every `n`, the prefix `Y_0, ..., Y_{n-1}` is
    consistent in
`?`: its finite intersection is a neighbourhood, so it is its own common lower bound.
    -/
theorem Y_prefix_consistent (n : Nat) : V.Consistent (Y V enum) n :=
  <V.interUpTo (Y V enum) n, mem_interUpTo_Y V enum henum n, subset_rfl>

/-- Every `Y_n` belongs to the greedy element. -/
theorem greedyElement_mem_Y (n : Nat) : (greedyElement V enum henum).mem (Y V enum n)
    :=
  <Y_mem V enum henum n, n + 1, V.interUpTo_subset (Y V enum) (Nat.lt_succ_self n)>

/-- The running intersection `acc n` belongs to the greedy element. -/
theorem greedyElement_mem_acc (n : Nat) : (greedyElement V enum henum).mem (acc V enum
     n) := by
  refine <acc_mem V enum henum n, n + 1, ?_>
  rw [acc_eq_interUpTo V enum]

variable (hsurj : forall  {X : Set a}, V.mem X  ->  exists  n, enum n = X)
include hsurj

/-- **Exercise 1.23 -- `{Y_n}` is a total element.** Maximality: if `z` extends the
    greedy element
and `W  in  z`, write `W = enum k` (surjectivity). For `k = 0`, `W = Y_0` is already in
     the greedy
element. For `k = m+1`: either the greedy step *accepted* `enum k` (then `W = Y_k` is
    in the
element), or it *rejected* it -- but then `acc m  I  enum k  notin  ?`, contradicting
    that both `acc m`
(in the greedy element, hence in `z`) and `W` lie in the filter `z`. -/
theorem greedyElement_isTotal : V.IsTotal (greedyElement V enum henum) := by
  intro z hz W hW
  obtain <k, rfl> := hsurj (z.sub hW)
  cases k with
  | zero =>
    have : Y V enum 0 = enum 0 := Y_zero V enum
    rw [ <-  this]
    exact greedyElement_mem_Y V enum henum 0
  | succ m =>
    by_cases h : cond V enum m
    * have hYk : Y V enum (m + 1) = enum (m + 1) := Y_succ_pos V enum h
```

```
      rw [ <-  hYk]
      exact greedyElement_mem_Y V enum henum (m + 1)
    * exfalso
      apply h
      have hacc_z : z.mem (acc V enum m) := hz _ (greedyElement_mem_acc V enum henum m
    )
      have : z.mem (acc V enum m  I  enum (m + 1)) := z.inter_mem hacc_z hW
      exact z.sub this

omit [DecidablePred V.mem] in
/-- **Exercise 1.23 -- "all filters can be determined by sequences".** In a countable
    system, every
filter `x` is the *limit family* of a sequence of neighbourhoods: take `s_n = enum n`
    whenever
`enum n  in  x` and `s_n = Delta` otherwise. Every member of `x` is some `enum k` (
    surjectivity), so it is
caught by `s`; conversely each `s_n  in  x`, so anything above some `s_n` is in `x`. (
    Classical: it
selects the sequence by deciding membership in `x`.) -/
theorem filters_sequence_determined (x : V.Element) :
    exists  s : Nat  ->  Set a, (forall  n, V.mem (s n))  /\  forall  Z, x.mem Z  <->
     (V.mem Z  /\  exists  n, s n  subseteq  Z) := by
  classical
  refine <fun i => if x.mem (enum i) then enum i else V.master, ?_, ?_>
  * intro n
    by_cases h : x.mem (enum n)
    * simp only [ite_eq_left h]; exact henum n
    * simp only [ite_eq_right h]; exact V.master_mem
  * intro Z
    constructor
    * intro hZ
      obtain <k, hk> := hsurj (x.sub hZ)
      have hxk : x.mem (enum k) := by rw [hk]; exact hZ
      refine <x.sub hZ, k, ?_>
      show (if x.mem (enum k) then enum k else V.master)  subseteq  Z
      rw [ite_eq_left hxk]; exact hk.subset
    * rintro <hZmem, n, hn>
      have hsx : x.mem (if x.mem (enum n) then enum n else V.master) := by
        by_cases h : x.mem (enum n)
        * simp only [ite_eq_left h]; exact h
        * simp only [ite_eq_right h]; exact x.master_mem
      exact x.up_mem hsx hZmem hn

end Exercise123

end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.Exercise123
section

set_option linter.unusedVariables false

theorem example_123.{u_1} : forall  {a : Type u_1} (V : DomainTheory.
    NeighbourhoodSystems.NeighborhoodSystem a) (enum : Nat  ->  Set a)
  [inst : @DecidablePred (Set a) (@DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.mem a V)],
  (forall  (n : Nat), @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V (
```

```
    enum n))  ->
    forall  (n : Nat),
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Consistent a V
        (@DomainTheory.NeighbourhoodSystems.Exercise123.Y a V enum inst) n :=
  @DomainTheory.NeighbourhoodSystems.Exercise123.Y_prefix_consistent
end

end DomainTheory.NeighbourhoodSystems.Exercise123
end
```

3.2.31 Exercise 1.24 — extending partial elements

(For set theorists). Prove, using the Axiom of Choice, or some equivalent principle, that in every domain a partial element can always be extended to a total element. Is this assertion equivalent to the Axiom of Choice? (Hint: Remember to prove that the union of every (transfinite) chain of filters is again a filter.)

Exercise_1_24.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.DomainIso
public import Mathlib.Order.Preorder.Chain

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem
section

variable {a : Type*} (V : NeighborhoodSystem a)

/- concordance: source L532 EXERCISE 1.24 extending partial elements -/
/-- **Exercise 1.24 -- the union of a chain of filters is a filter.** For a non-empty
    chain `C` of
filters, `? C = {X ? exists  x  in  C, X  in  x}` is again a filter. The intersection
    axiom uses chain
totality: given `X  in  x` and `Y  in  y` with `x, y  in  C`, one of `x  sqsub  y`, `y
    sqsub  x` holds, and the larger
filter contains both `X` and `Y`, hence `X  I  Y`. -/
def chainUnion (C : Set V.Element) (hne : C.Nonempty) (hchain : IsChain (*  <=  *) C)
    : V.Element where
  mem X := exists  x  in  C, x.mem X
  sub := by rintro X <x, _, hxX>; exact x.sub hxX
  master_mem := by obtain <x, hxC> := hne; exact <x, hxC, x.master_mem>
  inter_mem := by
    rintro X Y <x, hxC, hxX> <y, hyC, hyY>
    rcases hchain.total hxC hyC with hxy | hyx
    * exact <y, hyC, y.inter_mem (hxy X hxX) hyY>
    * exact <x, hxC, x.inter_mem hxX (hyx Y hyY)>
  up_mem := by
    rintro X Y <x, hxC, hxX> hY hXY
    exact <x, hxC, x.up_mem hxX hY hXY>
```

```
/-- Every member of the chain approximates the union `? C`. -/
theorem le_chainUnion (C : Set V.Element) (hne : C.Nonempty) (hchain : IsChain (*  <=
     *) C)
    {x : V.Element} (hx : x  in  C) : x  <=  V.chainUnion C hne hchain :=
  fun _ hX => <x, hx, hX>

/-- **Exercise 1.24 -- partial elements extend to total ones (AC).** Using Zorn's
    lemma, every
element `x` admits a *total* element `t` with `x  sqsub  t`. The upper bound of any non
    -empty chain in
`Ici x` is its `chainUnion`, so a maximal element exists; maximality is exactly
    totality
(`IsMax = IsTotal`). -/
theorem exists_total_ge (x : V.Element) : exists  t, V.IsTotal t  /\  x  <=  t := by
  obtain <m, hxm, hmax> :=
    zorn_le_nonempty_Ici_0 x
      (fun c _ hchain y hy =>
        <V.chainUnion c <y, hy> hchain, fun z hz => V.le_chainUnion c <y, hy> hchain
    hz>)
      x le_rfl
  exact <m, fun y hy => hmax hy, hxm>
end

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
section

set_option linter.unusedVariables false

theorem example_124.{u_1} : forall  {a : Type u_1} (V : DomainTheory.
    NeighbourhoodSystems.NeighborhoodSystem a)
  (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V),
  exists  (t : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V),
    And (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsTotal a V t)
      (@LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
        (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
     a V)
          (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.Element a V)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    instPartialOrderElement a V)))
        x t) :=
  @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.exists_total_ge
end

end DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
end
```

3.2.32 Exercise 1.25 — final segments of ordinals

(For set theorists). Let Δ be any well-ordered set (ordinal). (Even small ordinals like $\omega \cdot 3$ or ω^5 are interesting.) Let $\mathcal{D}$ be the family of non-empty *final* segments of Δ. What is $|\mathcal{D}|$? Are all elements finite? Is every approximation to a finite element finite?

Exercise_1_25.lean

```
module

import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.Factoid_1_7
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.DomainIso
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.principal
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_4
public import Mathlib.Algebra.Group.End
public import Mathlib.Order.UpperLower.Basic

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
namespace FinalSegment
section

open NeighborhoodSystem

/-- In a linear order, any two upper sets are nested. (Hence final segments are
*nested-or-disjoint* -- in fact always nested.) -/
theorem upperSet_subset_or {Delta : Type*} [LinearOrder Delta] {X Y : Set Delta}
    (hX : IsUpperSet X) (hY : IsUpperSet Y) : X  subseteq  Y  \/  Y  subseteq  X := by
  by_cases h : X  subseteq  Y
  * exact Or.inl h
  * right
    rw [Set.not_subset] at h
    obtain <a, haX, haY> := h
    intro b hbY
    rcases le_total a b with hab | hba
    * exact hX hab haX
    * exact absurd (hY hba hbY) haY

/- concordance: source L536 EXERCISE 1.25 final segments of ordinals -/
/-- **Exercise 1.25 -- the final-segment neighbourhood system.** Over a (non-empty)
    linear order
`Delta`, the non-empty upper sets (final segments) form a neighbourhood system. Built
    via
`ofNestedOrDisjoint` since any two upper sets are nested (`upperSet_subset_or`). -/
def finalSegmentSystem (Delta : Type*) [LinearOrder Delta] [Nonempty Delta] :
    NeighborhoodSystem Delta :=
  NeighborhoodSystem.ofNestedOrDisjoint
    (fun X => X.Nonempty  /\  IsUpperSet X)
    Set.univ
    Set.univ_nonempty
    <Set.univ_nonempty, isUpperSet_univ>
    (fun _ _ hX hY => (upperSet_subset_or hX.2 hY.2).imp_right Or.inl)
    (fun _ => Set.subset_univ _)

section
variable {Delta : Type*} [LinearOrder Delta] [Nonempty Delta]

@[simp] theorem mem_def {X : Set Delta} :
    (finalSegmentSystem Delta).mem X  <->  X.Nonempty  /\  IsUpperSet X := Iff.rfl
```

```
@[simp] theorem master_eq : (finalSegmentSystem Delta).master = Set.univ := rfl

/-- Each `Set.Ici a` is a (non-empty, upper) neighbourhood. -/
theorem Ici_mem (a : Delta) : (finalSegmentSystem Delta).mem (Set.Ici a) :=
  <<a, le_refl a>, isUpperSet_Ici a>
end

section
variable {Delta : Type*} [LinearOrder Delta] [Nonempty Delta]
variable [WellFoundedLT Delta]

/-- **Well-order key lemma.** In a well-order, every non-empty upper set is `Set.Ici a
    `, where `a`
is its least element. -/
theorem exists_Ici_of_mem {X : Set Delta} (hX : (finalSegmentSystem Delta).mem X) :
    exists  a, X = Set.Ici a := by
  obtain <a, haX, hmin> := wellFounded_lt.has_min X hX.1
  refine <a, Set.ext fun b => <fun hb => ?_, fun hb => ?_>>
  * rcases lt_or_ge b a with hlt | hge
    * exact absurd hlt (hmin b hb)
    * exact hge
  * exact hX.2 (Set.mem_Ici.mp hb) haX

/-- The trace `lowerSetOf x = {a ? Ici a  in  x}` of a filter `x`. -/
def lowerSetOf (x : (finalSegmentSystem Delta).Element) : Set Delta :=
  {a : Delta | x.mem (Set.Ici a)}

omit [WellFoundedLT Delta] in
theorem lowerSetOf_isLowerSet (x : (finalSegmentSystem Delta).Element) :
    IsLowerSet (lowerSetOf x) := by
  intro a b hba ha
  exact x.up_mem ha (Ici_mem b) (Set.Ici_subset_Ici.mpr hba)

theorem lowerSetOf_nonempty (x : (finalSegmentSystem Delta).Element) : (lowerSetOf x).
    Nonempty := by
  obtain <a, ha> := exists_Ici_of_mem (X := (Set.univ : Set Delta)) <Set.univ_nonempty
    , isUpperSet_univ>
  refine <a, ?_>
  have hu : x.mem (Set.univ) := x.master_mem
  rw [ha] at hu
  exact hu

/-- The filter built from a non-empty lower set `S`: the neighbourhoods that contain
    some
`Ici a` with `a  in  S`. -/
def ofLowerSet (S : Set Delta) (hSne : S.Nonempty) (_hS : IsLowerSet S) :
    (finalSegmentSystem Delta).Element where
  mem X := (finalSegmentSystem Delta).mem X  /\  exists  a  in  S, Set.Ici a  subseteq
      X
  sub h := h.1
  master_mem := by
    obtain <a, ha> := hSne
    exact <<Set.univ_nonempty, isUpperSet_univ>, a, ha, Set.subset_univ _>
  inter_mem := by
    rintro X Y <hX, a, haS, haX> <hY, b, hbS, hbY>
```

```
    have hsub : Set.Ici (max a b)  subseteq  X  I  Y :=
      Set.subset_inter
        ((Set.Ici_subset_Ici.mpr (le_max_left a b)).trans haX)
        ((Set.Ici_subset_Ici.mpr (le_max_right a b)).trans hbY)
    refine <(finalSegmentSystem Delta).inter_mem hX hY (Ici_mem (max a b)) hsub, max a
     b, ?_, hsub>
    rcases max_choice a b with h | h
    * rw [h]; exact haS
    * rw [h]; exact hbS
  up_mem := by
    rintro X Y <_, a, haS, haX> hY hXY
    exact <hY, a, haS, haX.trans hXY>

omit [WellFoundedLT Delta] in
theorem lowerSetOf_ofLowerSet (S : Set Delta) (hSne : S.Nonempty) (hS : IsLowerSet S)
    :
    lowerSetOf (ofLowerSet S hSne hS) = S := by
  ext a
  constructor
  * rintro <_, b, hbS, hba>
    exact hS (Set.Ici_subset_Ici.mp hba) hbS
  * intro ha
    exact <Ici_mem a, a, ha, subset_rfl>

theorem ofLowerSet_lowerSetOf (x : (finalSegmentSystem Delta).Element) :
    ofLowerSet (lowerSetOf x) (lowerSetOf_nonempty x) (lowerSetOf_isLowerSet x) = x :=
     by
  apply NeighborhoodSystem.Element.ext
  intro X
  constructor
  * rintro <hX, a, haS, haX>
    exact x.up_mem haS hX haX
  * intro hX
    obtain <a, rfl> := exists_Ici_of_mem (x.sub hX)
    exact <x.sub hX, a, hX, subset_rfl>

/-- **Exercise 1.25 -- classification of `|?|`.** For a well-order `Delta`, the domain
    of non-empty
final segments is order-isomorphic to the poset of **non-empty initial segments** (
    lower sets) of
`Delta` under inclusion: `x  |->  {a ? Ici a  in  x}`. (So `|?|` is the lattice of cuts
     of `Delta`.) -/
def finalSegmentClassify :
    (finalSegmentSystem Delta).Element ?o {S : Set Delta // S.Nonempty  /\  IsLowerSet
     S} where
  toFun x := <lowerSetOf x, lowerSetOf_nonempty x, lowerSetOf_isLowerSet x>
  invFun S := ofLowerSet S.1 S.2.1 S.2.2
  left_inv x := ofLowerSet_lowerSetOf x
  right_inv S := Subtype.ext (lowerSetOf_ofLowerSet S.1 S.2.1 S.2.2)
  map_rel_iff' := by
    intro x y
    constructor
    * intro h X hX
      obtain <a, rfl> := exists_Ici_of_mem (x.sub hX)
      exact h hX
    * intro h a ha
```

```
      exact h _ ha

omit [WellFoundedLT Delta]

/-- The greatest element: the filter of **all** neighbourhoods. -/
def topElement : (finalSegmentSystem Delta).Element where
  mem X := (finalSegmentSystem Delta).mem X
  sub h := h
  master_mem := <Set.univ_nonempty, isUpperSet_univ>
  inter_mem := by
    rintro X Y hX hY
    rcases upperSet_subset_or hX.2 hY.2 with h | h
    * rw [Set.inter_eq_left.mpr h]; exact hX
    * rw [Set.inter_eq_right.mpr h]; exact hY
  up_mem := fun _ hY _ => hY

omit [WellFoundedLT Delta]

theorem le_topElement (x : (finalSegmentSystem Delta).Element) : x  <=  topElement :=
  fun _ hX => x.sub hX

/-- **Exercise 1.25 -- the unique total element.** The filter of all neighbourhoods is
     total
(` sqsub `-maximal). -/
theorem topElement_isTotal : (finalSegmentSystem Delta).IsTotal (topElement (Delta :=
    Delta)) :=
  fun y _ => le_topElement y

/-- It is the *unique* total element: any total `x` equals `topElement`. -/
theorem eq_topElement_of_isTotal {x : (finalSegmentSystem Delta).Element}
    (hx : (finalSegmentSystem Delta).IsTotal x) : x = topElement :=
  le_antisymm (le_topElement x) (hx topElement (le_topElement x))

/-- **Exercise 1.25 -- not all elements are finite.** If `Delta` has *no greatest
    element* (e.g.
`Delta = Nat = ?`), the total element `topElement` is not a principal filter `(Ici a)`:
     pick `b > a`,
then `Ici b  in  topElement` but `Ici a ? Ici b`. So `|?|` has a non-finite (limit)
    element. -/
theorem topElement_not_principal_of_noMax (hnomax : forall  a : Delta, exists  b, a <
    b) (a : Delta) :
    topElement  !=  (finalSegmentSystem Delta).principal (Ici_mem a) := by
  intro h
  obtain <b, hab> := hnomax a
  have hmem : ((finalSegmentSystem Delta).principal (Ici_mem a)).mem (Set.Ici b) := by
    rw [ <-  h]; exact Ici_mem b
  have hsub : Set.Ici a  subseteq  Set.Ici b := hmem.2
  exact absurd (Set.Ici_subset_Ici.mp hsub) (not_le.mpr hab)
end
end

end FinalSegment
end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.FinalSegment
section
```

```
set_option linter.unusedVariables false

theorem example_125.{u_1} : And
  (forall  {Delta : Type u_1} [inst : LinearOrder Delta] [inst_1 : Nonempty Delta],
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsTotal Delta
      (@DomainTheory.NeighbourhoodSystems.FinalSegment.finalSegmentSystem Delta inst
    inst_1)
      (@DomainTheory.NeighbourhoodSystems.FinalSegment.topElement Delta inst inst_1))
  (forall  {Delta : Type u_1} [inst : LinearOrder Delta] [inst_1 : Nonempty Delta],
    (forall  (a : Delta),
        exists  (b : Delta),
          @LT.lt Delta
            (@Preorder.toLT Delta
              (@PartialOrder.toPreorder Delta
                (@SemilatticeInf.toPartialOrder Delta
                  (@Lattice.toSemilatticeInf Delta
                    (@DistribLattice.toLattice Delta (@instDistribLatticeOfLinearOrder
    Delta inst))))))
            a b)  ->
      forall  (a : Delta),
        @Ne
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element Delta
            (@DomainTheory.NeighbourhoodSystems.FinalSegment.finalSegmentSystem Delta
    inst inst_1))
          (@DomainTheory.NeighbourhoodSystems.FinalSegment.topElement Delta inst
    inst_1)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.principal Delta
            (@DomainTheory.NeighbourhoodSystems.FinalSegment.finalSegmentSystem Delta
    inst inst_1)
            (@Set.Ici Delta
              (@PartialOrder.toPreorder Delta
                (@SemilatticeInf.toPartialOrder Delta
                  (@Lattice.toSemilatticeInf Delta
                    (@DistribLattice.toLattice Delta (@instDistribLatticeOfLinearOrder
    Delta inst)))))
              a)
            (@DomainTheory.NeighbourhoodSystems.FinalSegment.Ici_mem Delta inst inst_1
    a))) :=
  @And.intro
  (forall  {Delta : Type u_1} [inst : LinearOrder Delta] [inst_1 : Nonempty Delta],
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsTotal Delta
      (@DomainTheory.NeighbourhoodSystems.FinalSegment.finalSegmentSystem Delta inst
    inst_1)
      (@DomainTheory.NeighbourhoodSystems.FinalSegment.topElement Delta inst inst_1))
  (forall  {Delta : Type u_1} [inst : LinearOrder Delta] [inst_1 : Nonempty Delta],
    (forall  (a : Delta),
        exists  (b : Delta),
          @LT.lt Delta
            (@Preorder.toLT Delta
              (@PartialOrder.toPreorder Delta
                (@SemilatticeInf.toPartialOrder Delta
                  (@Lattice.toSemilatticeInf Delta
                    (@DistribLattice.toLattice Delta (@instDistribLatticeOfLinearOrder
    Delta inst))))))
            a b)  ->
      forall  (a : Delta),
```

```
        @Ne
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element Delta
            (@DomainTheory.NeighbourhoodSystems.FinalSegment.finalSegmentSystem Delta
    inst inst_1))
          (@DomainTheory.NeighbourhoodSystems.FinalSegment.topElement Delta inst
    inst_1)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.principal Delta
            (@DomainTheory.NeighbourhoodSystems.FinalSegment.finalSegmentSystem Delta
    inst inst_1)
            (@Set.Ici Delta
              (@PartialOrder.toPreorder Delta
                (@SemilatticeInf.toPartialOrder Delta
                  (@Lattice.toSemilatticeInf Delta
                    (@DistribLattice.toLattice Delta (@instDistribLatticeOfLinearOrder
     Delta inst)))))
              a)
            (@DomainTheory.NeighbourhoodSystems.FinalSegment.Ici_mem Delta inst inst_1
     a)))
  @DomainTheory.NeighbourhoodSystems.FinalSegment.topElement_isTotal
  @DomainTheory.NeighbourhoodSystems.FinalSegment.topElement_not_principal_of_noMax
end

end DomainTheory.NeighbourhoodSystems.FinalSegment
namespace DomainTheory.NeighbourhoodSystems
namespace FinalSegment
section

open NeighborhoodSystem
variable {Delta : Type*} [LinearOrder Delta] [Nonempty Delta]

/-- The principal finite elements `(Ici a)`; the correspondence `a  |->  (Ici a)` is
order-*reversing* (`Ici a  subseteq  Ici b  <->  b  <=  a`), one of Scott's "finite
    elements". -/
theorem principal_Ici_le_iff (a b : Delta) :
    (finalSegmentSystem Delta).principal (Ici_mem a)  <=  (finalSegmentSystem Delta).
    principal (Ici_mem b)
       <->  a  <=  b := by
  rw [NeighborhoodSystem.principal_le_iff, Set.Ici_subset_Ici]
end

end FinalSegment
end DomainTheory.NeighbourhoodSystems
end
```

3.2.33 Exercise 1.26 — ideals of a ring

(For algebraists). Let A be a commutative ring with unit. Let Δ be the set of finite subsets $F \subseteq A$. Define

$$I(F) = \{G \in \Delta \mid F \subseteq \text{the ideal generated by } G\}.$$

Prove that the sets of the form $I(F)$ form a neighbourhood system, and that the corresponding domain is isomorphic to the set of ring-theoretic ideals of A partially ordered by inclusion. What would happen if we excluded from Δ all F with $I(F) = I(\{1\})$, where 1 is the unit of A?

Exercise_1_26.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_1
public import Mathlib.RingTheory.Ideal.Span

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
namespace Ring
section

open NeighborhoodSystem
variable {A : Type*} [CommRing A]

/- concordance: source L538 EXERCISE 1.26 ideals of a ring -/
/-- **Exercise 1.26 -- the basic neighbourhood `I(F)`.** `I(F) = {G ? F  subseteq  <G
    >}`, the finite sets `G`
whose generated ideal contains `F`. -/
def IFamily (F : Finset A) : Set (Finset A) := {G | (F : Set A)  subseteq  Ideal.span
    (G : Set A)}

@[simp] theorem mem_IFamily {F G : Finset A} :
    G  in  IFamily F  <->  (F : Set A)  subseteq  Ideal.span (G : Set A) := Iff.rfl

section
variable [DecidableEq A]

/-- `I(F_1)  I  I(F_2) = I(F_1  U  F_2)`: the system is closed under all binary
    intersections. -/
theorem IFamily_inter (F_1 F_2 : Finset A) :
    IFamily F_1  I  IFamily F_2 = IFamily (F_1  U  F_2) := by
  ext G
  simp only [IFamily, Set.mem_inter_iff, Set.mem_ofPred_eq, Finset.coe_union, Set.
    union_subset_iff]
end

/-- Inclusion of neighbourhoods reversed to ideal generation: `I(F_1)  subseteq  I(F_2)
      <->  F_2  subseteq  <F_1>`. -/
theorem IFamily_subset_iff (F_1 F_2 : Finset A) :
    IFamily F_1  subseteq  IFamily F_2  <->  (F_2 : Set A)  subseteq  Ideal.span (F_1
    : Set A) := by
  constructor
  * intro h
    have hF_1 : F_1  in  IFamily F_1 := mem_IFamily.mpr Ideal.subset_span
    exact (h hF_1)
  * intro h G hG
    rw [mem_IFamily] at hG  |-
    exact h.trans (SetLike.coe_subset_coe.mpr (Ideal.span_le.mpr hG))

section
variable [DecidableEq A]
```

```
/-- **Exercise 1.26 -- the neighbourhood system of a commutative ring.** Tokens are
    finite subsets
`F  subseteq  A`; neighbourhoods are the sets `I(F)`. The master is `I({})` (all of `
    Delta`), and intersections
`I(F_1)  I  I(F_2) = I(F_1  U  F_2)` are always neighbourhoods. -/
def ringSystem : NeighborhoodSystem (Finset A) where
  mem S := exists  F, S = IFamily F
  master := IFamily {}
  master_nonempty := <{}, by simp [IFamily]>
  master_mem := <{}, rfl>
  inter_mem := by
    rintro X Y Z <F_1, rfl> <F_2, rfl> _ _
    exact <F_1  U  F_2, IFamily_inter F_1 F_2>
  sub_master := by
    rintro X <F, rfl>
    exact (IFamily_subset_iff F {}).mpr (by rw [Finset.coe_empty]; exact Set.
    empty_subset _)

@[simp] theorem ringSystem_mem {S : Set (Finset A)} :
    (ringSystem (A := A)).mem S  <->  exists  F, S = IFamily F := Iff.rfl

theorem IFamily_mem (F : Finset A) : (ringSystem (A := A)).mem (IFamily F) := <F, rfl>

/-- **Exercise 1.26 -- the ideal of a filter.** `idealOf x = {a ? I({a})  in  x}`. The
    closure
properties of an ideal come from the filter axioms: `0` from the master, `+` from ` I
    `-closure
(`I({a})  I  I({b}) = I({a,b})  subseteq  I({a+b})`), and `*` from upward closure (`I({
    a})  subseteq  I({c*a})`). -/
def idealOf (x : (ringSystem (A := A)).Element) : Ideal A where
  carrier := {a | x.mem (IFamily {a})}
  zero_mem' := by
    show x.mem (IFamily {(0 : A)})
    refine x.up_mem x.master_mem (IFamily_mem {(0 : A)}) ?_
    intro G _
    rw [mem_IFamily, Finset.coe_singleton, Set.singleton_subset_iff]
    exact Submodule.zero_mem _
  add_mem' := by
    intro a b ha hb
    show x.mem (IFamily {a + b})
    have h := x.inter_mem ha hb
    rw [IFamily_inter] at h
    refine x.up_mem h (IFamily_mem {a + b}) ?_
    apply (IFamily_subset_iff _ _).mpr
    rw [Finset.coe_singleton, Set.singleton_subset_iff, Finset.coe_union,
      Finset.coe_singleton, Finset.coe_singleton]
    exact Ideal.add_mem _ (Ideal.subset_span (by simp)) (Ideal.subset_span (by simp))
  smul_mem' := by
    intro c a ha
    show x.mem (IFamily {c * a})
    refine x.up_mem ha (IFamily_mem {c * a}) ?_
    apply (IFamily_subset_iff _ _).mpr
    rw [Finset.coe_singleton, Set.singleton_subset_iff, Finset.coe_singleton]
    exact Submodule.smul_mem _ c (Ideal.subset_span (Set.mem_singleton a))

@[simp] theorem mem_idealOf {x : (ringSystem (A := A)).Element} {a : A} :
```

```
    a  in  idealOf x  <->  x.mem (IFamily {a}) := Iff.rfl

/-- A filter contains `I(F)` whenever it contains `I({a})` for every `a  in  F` (by ` I
    `-closure). -/
theorem mem_IFamily_of_forall (x : (ringSystem (A := A)).Element) (F : Finset A)
    (h : forall  a  in  F, x.mem (IFamily {a})) : x.mem (IFamily F) := by
  classical
  induction F using Finset.induction with
  | empty => exact x.master_mem
  | @insert a F' _ ih =>
    have hins : (insert a F' : Finset A) = {a}  U  F' := by
      ext y; simp [Finset.mem_insert]
    rw [hins,  <-  IFamily_inter]
    exact x.inter_mem (h a (by simp)) (ih (fun b hb => h b (by simp [hb])))

/-- **Exercise 1.26 -- the filter of an ideal.** `ofIdeal J = {I(F) ? F  subseteq  J}`.
    -/
def ofIdeal (J : Ideal A) : (ringSystem (A := A)).Element where
  mem S := exists  F, S = IFamily F  /\  Ideal.span (F : Set A)  <=  J
  sub := by rintro S <F, rfl, _>; exact <F, rfl>
  master_mem := by
    refine <{}, rfl, ?_>
    rw [Finset.coe_empty, Ideal.span_empty]
    exact bot_le
  inter_mem := by
    rintro X Y <F_1, rfl, h_1> <F_2, rfl, h_2>
    refine <F_1  U  F_2, IFamily_inter F_1 F_2, ?_>
    rw [Finset.coe_union, Ideal.span_union]
    exact sup_le h_1 h_2
  up_mem := by
    rintro X Y <F, rfl, hF> <F', rfl> hXY
    refine <F', rfl, ?_>
    have hsub : (F' : Set A)  subseteq  Ideal.span (F : Set A) := (IFamily_subset_iff
    F F').mp hXY
    exact (Ideal.span_le.mpr hsub).trans hF

theorem ofIdeal_idealOf (x : (ringSystem (A := A)).Element) : ofIdeal (idealOf x) = x
    := by
  apply NeighborhoodSystem.Element.ext
  intro S
  constructor
  * rintro <F, rfl, hF>
    refine mem_IFamily_of_forall x F (fun a ha => ?_)
    have haspan : a  in  Ideal.span (F : Set A) := Ideal.subset_span (by simpa using
    ha)
    exact mem_idealOf.mp (hF haspan)
  * intro hS
    obtain <F, rfl> := x.sub hS
    refine <F, rfl, ?_>
    rw [Ideal.span_le]
    intro a ha
    rw [SetLike.mem_coe, mem_idealOf]
    refine x.up_mem hS (IFamily_mem {a}) ?_
    apply (IFamily_subset_iff _ _).mpr
    rw [Finset.coe_singleton, Set.singleton_subset_iff]
    exact Ideal.subset_span ha
```

```
end

theorem span_eq_of_IFamily_eq {F_1 F_2 : Finset A} (h : IFamily F_1 = IFamily F_2) :
    Ideal.span (F_1 : Set A) = Ideal.span (F_2 : Set A) := by
  have h12 : (F_2 : Set A)  subseteq  Ideal.span (F_1 : Set A) :=
    (IFamily_subset_iff F_1 F_2).mp h.le
  have h21 : (F_1 : Set A)  subseteq  Ideal.span (F_2 : Set A) :=
    (IFamily_subset_iff F_2 F_1).mp h.ge
  exact le_antisymm (Ideal.span_le.mpr h21) (Ideal.span_le.mpr h12)

section
variable [DecidableEq A]

theorem mem_ofIdeal_IFamily (J : Ideal A) (F : Finset A) :
    (ofIdeal J).mem (IFamily F)  <->  Ideal.span (F : Set A)  <=  J := by
  constructor
  * rintro <F', hF', hle>
    rw [span_eq_of_IFamily_eq hF']
    exact hle
  * intro h
    exact <F, rfl, h>

theorem idealOf_ofIdeal (J : Ideal A) : idealOf (ofIdeal J) = J := by
  apply SetLike.ext
  intro a
  rw [mem_idealOf, mem_ofIdeal_IFamily, Finset.coe_singleton, Ideal.
    span_singleton_le_iff_mem]

/-- **Exercise 1.26 -- `|?|  ~=  (Ideals A,  subseteq )`.** The domain of the ring's
    neighbourhood system is
order-isomorphic to the ideals of `A` ordered by inclusion, via `x  |->  {a ? I({a})
    in  x}`. -/
def ringIso : (ringSystem (A := A)).Element ?o Ideal A where
  toFun := idealOf
  invFun := ofIdeal
  left_inv := ofIdeal_idealOf
  right_inv := idealOf_ofIdeal
  map_rel_iff' := by
    intro x y
    constructor
    * intro h S hS
      obtain <F, rfl> := x.sub hS
      refine mem_IFamily_of_forall y F (fun a ha => ?_)
      have hax : a  in  idealOf x := by
        rw [mem_idealOf]
        exact x.up_mem hS (IFamily_mem {a}) <| (IFamily_subset_iff _ _).mpr <| by
          rw [Finset.coe_singleton, Set.singleton_subset_iff]
          exact Ideal.subset_span (by simpa using ha)
      exact mem_idealOf.mp (h hax)
    * intro h a ha
      exact h (IFamily {a}) ha
end

theorem IFamily_eq_of_span_eq {F_1 F_2 : Finset A}
    (h : Ideal.span (F_1 : Set A) = Ideal.span (F_2 : Set A)) : IFamily F_1 = IFamily
    F_2 := by
```

```
  apply Set.Subset.antisymm
  * exact (IFamily_subset_iff F_1 F_2).mpr (by rw [h]; exact Ideal.subset_span)
  * exact (IFamily_subset_iff F_2 F_1).mpr (by rw [ <-  h]; exact Ideal.subset_span)
end

end Ring
end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.Ring
section

set_option linter.unusedVariables false

theorem example_126.{u_1} : forall  {A : Type u_1} [inst : CommRing A] {F_1 F_2 :
    Finset A},
  @Eq (@Ideal A (@CommSemiring.toSemiring A (@CommRing.toCommSemiring A inst)))
      (@Ideal.span A (@CommSemiring.toSemiring A (@CommRing.toCommSemiring A inst))
        (@SetLike.coe (Finset A) A (@Finset.instSetLike A) F_1))
      (@Ideal.span A (@CommSemiring.toSemiring A (@CommRing.toCommSemiring A inst))
        (@SetLike.coe (Finset A) A (@Finset.instSetLike A) F_2))  ->
    @Eq (Set (Finset A)) (@DomainTheory.NeighbourhoodSystems.Ring.IFamily A inst F_1)
      (@DomainTheory.NeighbourhoodSystems.Ring.IFamily A inst F_2) :=
  @DomainTheory.NeighbourhoodSystems.Ring.IFamily_eq_of_span_eq
end

end DomainTheory.NeighbourhoodSystems.Ring
end
```

3.2.34 Exercise 1.27 — suprema of bounded sets

Further closure properties of domains can be proved for bounded sets. We say $X \subseteq |\mathcal{D}|$ is *bounded* iff for some $y \in |\mathcal{D}|$ we have $x \subseteq y$ for all $x \in X$. This y is called an *upper bound*. We let

$$\bigsqcup X = \bigcap \{y \in |\mathcal{D}| \mid x \subseteq y \text{ all } x \in X\}.$$

Prove that if X is bounded, then $\bigsqcup X$ is the *least upper bound* for X in $|\mathcal{D}|$. Prove also: if $U, V \in \mathcal{D}$ are neighbourhoods, then $\{U, V\}$ is consistent in $\mathcal{D}$ iff $\{\uparrow U, \uparrow V\}$ is bounded in $|\mathcal{D}|$. (That is, boundedness is for elements what consistency is for neighbourhoods.) Prove finally with the aid of 1.18 that $X \subseteq |\mathcal{D}|$ is bounded iff every finite subset of X is bounded.

Exercise_1_27.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_18
import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.Factoid_1_7
import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.Factoid_1_5
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.principal
public import Mathlib.Basic.Finite.Defs

@[expose] public section

section
```

```
namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem
section

variable {a : Type*} (V : NeighborhoodSystem a)

/- concordance: source L546 EXERCISE 1.27 suprema of bounded sets -/
/-- **Exercise 1.27 -- bounded set of elements.** `X  subseteq  |?|` is *bounded* iff
    it has an upper
bound `y  in  |?|`: `x  sqsub  y` for all `x  in  X`. -/
def Bounded (X : Set V.Element) : Prop := exists  y : V.Element, forall  x  in  X, x
    <=  y

/-- The two-term sequence `<U, W>` as a function `Nat  ->  Set a` (used to phrase `
    Consistent`). -/
def pairSeq (U W : Set a) : Nat  ->  Set a := fun i => if i = 0 then U else W

/-- The family of upper bounds of `X`: `{y ? x  sqsub  y for all x  in  X}`. -/
def upperBounds (X : Set V.Element) : Set V.Element := {y | forall  x  in  X, x  <=  y
    }

/-- If `X` is bounded then its upper-bound family is non-empty (it contains the witness
    ). -/
theorem upperBounds_nonempty {X : Set V.Element} (hX : V.Bounded X) :
    (V.upperBounds X).Nonempty :=
  hX.imp fun _ hy => hy

/-- **Exercise 1.27 -- ` sqsup X`.** The least upper bound of a bounded `X`, defined a
    la Scott as the
intersection of all upper bounds: ` sqsup X = ? {y ? x  sqsub  y all x in X}`. Reusing
    `sInf` from Exercise
1.18 on the (non-empty, because bounded) family of upper bounds. -/
def sSup (X : Set V.Element) (hX : V.Bounded X) : V.Element :=
  V.sInf (V.upperBounds X) (V.upperBounds_nonempty hX)

/-- **Exercise 1.27 -- boundedness is finitary.** `X  subseteq  |?|` is bounded iff
    every *finite* subset
of `X` is bounded. The forward direction reuses any global bound. The reverse direction
     is the
content: assemble `C = ?_{x in X} x` (the neighbourhoods of all members of `X`); `C` is
     finitely
consistent because any finite sequence drawn from `C` comes from finitely many members
    of `X`,
which form a finite subset, hence bounded -- that bound's filter contains the whole
    finite
sequence, so its intersection lies in `?`. The least filter `leastFilter C` (Exercise
    1.18) is
then an upper bound of `X`. -/
theorem bounded_iff_finite_bounded (X : Set V.Element) :
    V.Bounded X  <->  forall  S : Set V.Element, S  subseteq  X  ->  S.Finite  ->  V.
    Bounded S := by
  constructor
  * rintro <y, hy> S hS _
    exact <y, fun x hx => hy x (hS hx)>
  * intro hfin
    set C : Set (Set a) := {Z | exists  x : V.Element, x  in  X  /\  x.mem Z} with
```

```
    hCdef
    have hCsub : forall  Z  in  C, V.mem Z := by rintro Z <x, _, hxZ>; exact x.sub hxZ
    have hCcons : V.FinitelyConsistent C := by
      intro n seq hseqC
      choose g hgX hgmem using hseqC
      set S : Set V.Element := Set.range (fun i : Fin n => g i.1 i.2) with hSdef
      have hSfin : S.Finite := Set.finite_range _
      have hSsub : S  subseteq  X := by
        rintro _ <i, rfl>
        exact hgX i.1 i.2
      obtain <y, hy> := hfin S hSsub hSfin
      have hseqy : forall  i, i < n  ->  y.mem (seq i) := by
        intro i hi
        have hle : g i hi  <=  y := hy _ <<i, hi>, rfl>
        exact hle (seq i) (hgmem i hi)
      have hmem : y.mem (V.interUpTo seq n) := y.mem_interUpTo seq hseqy
      exact <V.interUpTo seq n, y.sub hmem, subset_rfl>
    refine <V.leastFilter C hCsub hCcons, ?_>
    intro x hx Z hZ
    exact V.subset_leastFilter C hCsub hCcons <x, hx, hZ>

theorem interUpTo_pairSeq (U W : Set a) :
    V.interUpTo (pairSeq U W) 2 = V.master  I  U  I  W := by
  simp only [interUpTo_succ, interUpTo_zero, pairSeq]
  norm_num

/-- **Exercise 1.27 -- "boundedness is for elements what consistency is for
    neighbourhoods".**
For `U, W  in  ?`, the pair `<U, W>` is consistent in `?` iff `{U, W}` is bounded in
    `|?|`.

` -> ` packages the consistency witness `Z` into the principal filter `Z`, which lies
    above both
`U` and `W` (via `principal_le_iff`). ` <- ` uses that any bound `y` contains both `U`
    and `W`,
hence `U  I  W  in  y  subseteq  ?`, giving `U  I  W` as the consistency witness. -/
theorem consistent_pair_iff_bounded {U W : Set a} (hU : V.mem U) (hW : V.mem W) :
    V.Consistent (pairSeq U W) 2  <->  V.Bounded {V.principal hU, V.principal hW} :=
    by
  constructor
  * rintro <Z, hZmem, hZsub>
    rw [V.interUpTo_pairSeq] at hZsub
    have hZU : Z  subseteq  U := hZsub.trans (Set.inter_subset_left.trans Set.
    inter_subset_right)
    have hZW : Z  subseteq  W := hZsub.trans Set.inter_subset_right
    refine <V.principal hZmem, ?_>
    intro x hx
    rcases hx with rfl | rfl
    * exact (V.principal_le_iff hU hZmem).mpr hZU
    * exact (V.principal_le_iff hW hZmem).mpr hZW
  * rintro <y, hy>
    have hyU : y.mem U :=
      hy _ (Or.inl rfl) U <hU, subset_rfl>
    have hyW : y.mem W :=
      hy _ (Or.inr rfl) W <hW, subset_rfl>
    have hyUW : y.mem (U  I  W) := y.inter_mem hyU hyW
```

```
    refine <U  I  W, y.sub hyUW, ?_>
    rw [V.interUpTo_pairSeq]
    intro z hz
    exact <<V.sub_master hU hz.1, hz.1>, hz.2>

/-- **Exercise 1.27 -- ` sqsup X` is an upper bound.** Each `x  in  X` satisfies `x
    sqsub   sqsup X`. -/
theorem le_sSup (X : Set V.Element) (hX : V.Bounded X) {x : V.Element} (hx : x  in  X)
     :
    x  <=  V.sSup X hX :=
  V.le_sInf (V.upperBounds X) (V.upperBounds_nonempty hX) x (fun _ hy => hy x hx)

/-- **Exercise 1.27 -- ` sqsup X` is least.** Any upper bound `z` of `X` satisfies `
    sqsup X  sqsub  z`. -/
theorem sSup_le (X : Set V.Element) (hX : V.Bounded X) {z : V.Element}
    (hz : forall  x  in  X, x  <=  z) : V.sSup X hX  <=  z :=
  V.sInf_le (V.upperBounds X) (V.upperBounds_nonempty hX) hz
end

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
section

set_option linter.unusedVariables false

theorem example_127.{u_1} : And
  (And
    (And
      (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem a)
        (X : Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)),
        Iff (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Bounded a V X)
          (forall  (S : Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    Element a V)),
            @LE.le (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
     a V))
                (@Set.instLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    Element a V)) S X  ->
              @Set.Finite (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    Element a V) S  ->
                @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Bounded a V S))
      (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem a) {U W : Set a}
        (hU : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V U)
        (hW : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V W),
        Iff
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Consistent a V
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.pairSeq a U W)
            (@OfNat.ofNat Nat (nat_lit 2) (instOfNatNat (nat_lit 2))))
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Bounded a V
            (@Insert.insert (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    Element a V)
              (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
    )
              (@Set.instInsert (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
```

```
  Element a V))
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.principal a V U
  hU)
            (@Singleton.singleton (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
              (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
  V))
              (@Set.instSingletonSet (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V))
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.principal a V W
   hW))))))
   (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
  a)
     (X : Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V))
     (hX : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Bounded a V X)
     {x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V},
     @Membership.mem (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
    V)
          (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V))
          (@Set.instMembership (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  Element a V)) X x  ->
       @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
         (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  Element a V)
           (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
             (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement a V)))
          x (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.sSup a V X hX)))
(forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a)
  (X : Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V))
  (hX : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Bounded a V X)
  {z : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V},
  (forall  (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V),
     @Membership.mem (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
   a V)
           (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V))
           (@Set.instMembership (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)) X x  ->
         @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
           (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  Element a V)
             (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
               (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement a V)))
           x z)  ->
    @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
      (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
   a V)
        (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement a V)))
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.sSup a V X hX) z) :=
@And.intro
```

```
(And
  (And
    (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem a)
      (X : Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)),
      Iff (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Bounded a V X)
        (forall  (S : Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 Element a V)),
          @LE.le (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
  a V))
             (@Set.instLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 Element a V)) S X  ->
           @Set.Finite (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 Element a V) S  ->
             @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Bounded a V S))
    (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem a) {U W : Set a}
      (hU : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V U)
      (hW : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V W),
      Iff
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Consistent a V
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.pairSeq a U W)
          (@OfNat.ofNat Nat (nat_lit 2) (instOfNatNat (nat_lit 2))))
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Bounded a V
          (@Insert.insert (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 Element a V)
            (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
 )
            (@Set.instInsert (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 Element a V))
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.principal a V U
 hU)
            (@Singleton.singleton (@DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem.Element a V)
              (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
 V))
              (@Set.instSingletonSet (@DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem.Element a V))
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.principal a V W
  hW))))))
  (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
 a)
    (X : Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V))
    (hX : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Bounded a V X)
    {x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V},
    @Membership.mem (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
  V)
      (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V))
      (@Set.instMembership (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 Element a V)) X x  ->
    @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
      (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 Element a V)
        (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem.Element a V)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
```

```
 instPartialOrderElement a V)))
       x (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.sSup a V X hX)))
(forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a)
  (X : Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V))
  (hX : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Bounded a V X)
  {z : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V},
  (forall  (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V),
     @Membership.mem (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
   a V)
         (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V))
         (@Set.instMembership (@DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem.Element a V)) X x  ->
       @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
         (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 Element a V)
           (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem.Element a V)
             (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 instPartialOrderElement a V)))
         x z)  ->
    @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
      (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
   a V)
       (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem.Element a V)
         (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 instPartialOrderElement a V)))
     (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.sSup a V X hX) z)
(@And.intro
  (And
    (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem a)
     (X : Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)),
     Iff (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Bounded a V X)
       (forall  (S : Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 Element a V)),
         @LE.le (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
   a V))
             (@Set.instLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 Element a V)) S X  ->
           @Set.Finite (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 Element a V) S  ->
             @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Bounded a V S))
    (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem a) {U W : Set a}
     (hU : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V U)
     (hW : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V W),
     Iff
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Consistent a V
         (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.pairSeq a U W)
         (@OfNat.ofNat Nat (nat_lit 2) (instOfNatNat (nat_lit 2))))
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Bounded a V
         (@Insert.insert (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 Element a V)
           (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
 )
```

```
          (@Set.instInsert (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a V))
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.principal a V U
hU)
          (@Singleton.singleton (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
            (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
V))
            (@Set.instSingletonSet (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V))
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.principal a V W
 hW))))))
(forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
a)
  (X : Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V))
  (hX : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Bounded a V X)
  {x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V},
  @Membership.mem (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
 V)
      (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V))
      (@Set.instMembership (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a V)) X x  ->
    @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
      (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a V)
        (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
instPartialOrderElement a V)))
      x (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.sSup a V X hX))
(@And.intro
  (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
    (X : Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)),
    Iff (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Bounded a V X)
      (forall  (S : Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a V)),
        @LE.le (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
 a V))
          (@Set.instLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a V)) S X  ->
          @Set.Finite (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a V) S  ->
            @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Bounded a V S))
  (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a) {U W : Set a}
    (hU : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V U)
    (hW : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V W),
    Iff
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Consistent a V
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.pairSeq a U W)
        (@OfNat.ofNat Nat (nat_lit 2) (instOfNatNat (nat_lit 2))))
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Bounded a V
        (@Insert.insert (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a V)
          (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
```

```
    )
              (@Set.instInsert (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    Element a V))
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.principal a V U
    hU)
              (@Singleton.singleton (@DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.Element a V)
                (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
    V))
                (@Set.instSingletonSet (@DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.Element a V))
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.principal a V W
     hW)))))
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bounded_iff_finite_bounded
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    consistent_pair_iff_bounded)
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.le_sSup)
  @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.sSup_le
end

end DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem
section

variable {a : Type*} (V : NeighborhoodSystem a)

/-- Scott's `[X] = {x  in  |?| ? X  in  x}`: the elements of `|?|` that contain the
    neighbourhood `X`.
(This is the `basicOpen X` of Exercise 1.22, repeated here to avoid the topology
    dependency.) -/
def bracket (X : Set a) : Set V.Element := {x | x.mem X}

@[simp] theorem mem_bracket {X : Set a} {x : V.Element} : x  in  V.bracket X  <->  x.
    mem X := Iff.rfl

@[simp] theorem bracket_master : V.bracket V.master = Set.univ := by
  ext x; simp only [mem_bracket, Set.mem_univ, iff_true]; exact x.master_mem

/-- **Theorem 1.10 (3).** `[X]  I  [Y] = [X  I  Y]` for `X, Y  in  ?`. (` subseteq ` is
     filter closure under ` I `;
` supseteq ` is upward closure along `X  I  Y  subseteq  X`, `X  I  Y  subseteq  Y`.)
    -/
theorem bracket_inter {X Y : Set a} (hX : V.mem X) (hY : V.mem Y) :
    V.bracket X  I  V.bracket Y = V.bracket (X  I  Y) := by
  ext x
  simp only [Set.mem_inter_iff, mem_bracket]
  constructor
  * rintro <hx, hy>
    exact x.inter_mem hx hy
  * intro hxy
    exact <x.up_mem hxy hX Set.inter_subset_left, x.up_mem hxy hY Set.
    inter_subset_right>

/-- **Theorem 1.10 (4).** `X  in  [X]`: the principal filter of `X` contains `X`. -/
theorem principal_mem_bracket {X : Set a} (hX : V.mem X) : V.principal hX  in  V.
```

```
    bracket X :=
  <hX, subset_rfl>

/-- **The correspondence `X |-> [X]` is inclusion-preserving.** `[X] subseteq [Y]
    <-> X subseteq Y` (for `X,Y in ?`).
` -> ` tests at `X in [X]`, reading `X subseteq Y` off `X in [Y]`; ` <- ` is
    upward closure. -/
theorem bracket_subset_iff {X Y : Set a} (hX : V.mem X) (hY : V.mem Y) :
    V.bracket X subseteq V.bracket Y <-> X subseteq Y := by
  constructor
  * intro h
    exact (h (V.principal_mem_bracket hX)).2
  * intro hXY x hx
    exact x.up_mem hx hY hXY

/-- **The correspondence `X |-> [X]` is one-one.** `[X] = [Y] ? X = Y` (for `X,Y in
    ?`). -/
theorem bracket_injective {X Y : Set a} (hX : V.mem X) (hY : V.mem Y)
    (h : V.bracket X = V.bracket Y) : X = Y :=
  Set.Subset.antisymm ((V.bracket_subset_iff hX hY).mp h.subset)
    ((V.bracket_subset_iff hY hX).mp h.superset)
end

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
end
```

3.3 Mappings between domains

Dependency diagram. Each box is one item. The arrowhead is on the item that imports. An arrow already implied by a longer path is left out. The introduction has no Lean file, so it is not shown. Blue is a definition, green a proposition, amber an example, red a theorem, and indigo an exercise.

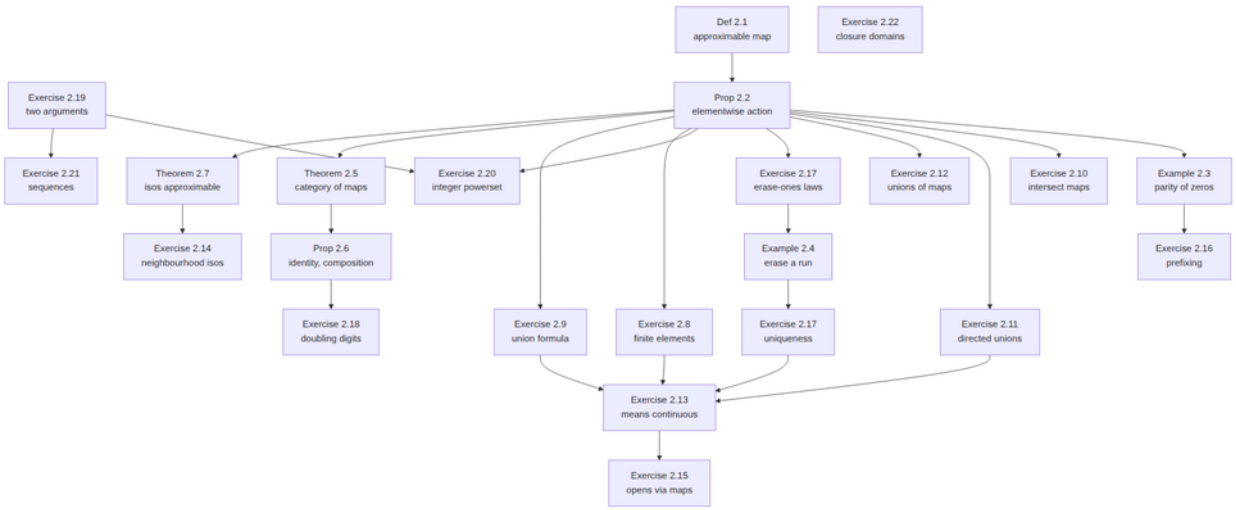


Figure 31: Mappings between domains.

3.3.1 Introduction

LECTURE II MAPPINGS BETWEEN DOMAINS

The elements of a domain are regarded as being specified by approximations: the neighbourhoods. With the idea of approximation as the dominant notion, therefore, it is natural to look for a concept of mapping (transformation of domains) that in some suitable sense preserves the spirit of the approximations. In a 'theory of computability,' where the (finite) approximations to the elements are all we can ever know at one time, the only mappings that can be computed are those that proceed by approximation, somehow passing from the neighbourhoods of one domain over to the neighbourhoods of the other.

Suppose $X \in \mathcal{D}_0$ is given - it is an approximation to certain elements of $|\mathcal{D}_0|$. (More precisely $\uparrow X$ is the approximation in the domain, but it is easier to speak of the neighbourhood X.) What can be said about the approximations of the images of these elements under the mapping we will call f? If X is not a very sharp approximation, then not very much can be said about the image in the other domain $|\mathcal{D}_1|$. Trivially, of course, we can say that Δ_1 is an approximation - because it approximates everything in its domain. Suppose, however, that we could say more. Suppose we could say that both Y and Y' approximate the image of X. If the mapping f is coherent, then it is reasonable to suppose that such a statement would imply that Y and Y' are *consistent* in $\mathcal{D}_1$. But if this is so, then since the two neighbourhoods are meant to cluster around the same images, we can feel some confidence in saying that $Y \cap Y'$ approximates these images. In other words to specify f we do not supply a unique image of X, but we say which of the $Y \in \mathcal{D}_1$ approximate the (ideal) image. To make this idea work a *monotonicity condition* is also needed since we are trying to express the idea that "if we give at least X as an approximate input to f, then we can expect at least Y as output." Thus,

a mapping is taken as a kind of relation between neighbourhoods.

3.3.2 Definition 2.1 — approximable mapping

DEFINITION 2.1. An *approximable mapping* $f : D_0 \to D_1$ between domains is a binary relation $f \subseteq D_0 \times D_1$ between neighbourhoods such that

(i) $\Delta_0 f \Delta_1$;

(ii) $X f Y$ and $X f Y'$ always imply $X f (Y \cap Y')$;

(iii) $X f Y$, $X' \subseteq X$, and $Y \subseteq Y'$ always imply $X' f Y'$. □

Condition (i) we have already discussed; in a sense it means "ask me no questions and I shall tell you no lies." In other words "zero input can expect at least zero output." The other conditions are compatible with having

$$f = \{\langle X, \Delta_1 \rangle \mid X \in D_0\};$$

that is, f might be the least informative relation and nothing more. But if it is more, then (ii) is, as we explained, a consistency condition. To explain monotonicity in (iii), suppose a mapping relationship is already known, $X f Y$, say. If we *improve* the accuracy of X to $X' \subseteq X$ and if we *degrade* the accuracy of Y to $Y' \supseteq Y$, then we can still assert $X' f Y'$ since this relationship is *less informative* than the former relationship, which was already known. Thus, we see that conditions (i)–(iii) are all reasonably argued as necessary.

One indication that the conditions of 2.1 are sufficient for a definition is that they are exactly what we need to show that f as a neighbourhood relation determines an equivalent elementwise mapping from $|D_0|$ into $|D_1|$. (Owing to the equivalence, we use the same symbol f for both.)

Definition_2_1.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_1
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem

section
variable {a b ? ? : Type*}

/- concordance: source L568 DEFINITION 2.1 approximable mapping -/
/-- **Definition 2.1 (Scott 1981, PRG-19).** An *approximable mapping* `f : ?_0  ->  ?
    _1` is a relation
`rel` between neighbourhoods (`rel X Y`, Scott's `X f Y`) confined to `?_0  x  ?_1` and
     satisfying
Scott's three conditions:

* (i)   `Delta_0 f Delta_1`                                    -- `master_rel`;
* (ii)  `X f Y` and `X f Y'` imply `X f (Y  I  Y')`   -- `inter_right`;
* (iii) `X f Y`, `X'  subseteq  X`, `Y  subseteq  Y'` imply `X' f Y'` -- `mono` (the
    targets `X'`, `Y'` must be
  neighbourhoods, as Scott's relation lives on `?_0  x  ?_1`). -/
structure ApproximableMap (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b)
    where
  /-- The underlying neighbourhood relation `X f Y`. -/
  rel : Set a  ->  Set b  ->  Prop
  /-- `f  subseteq  ?_0  x  ?_1` (domain side): related inputs are neighbourhoods. -/
  rel_dom : forall  {X Y}, rel X Y  ->  V_0.mem X
  /-- `f  subseteq  ?_0  x  ?_1` (codomain side): related outputs are neighbourhoods.
    -/
  rel_cod : forall  {X Y}, rel X Y  ->  V_1.mem Y
  /-- (i) `Delta_0 f Delta_1`. -/
  master_rel : rel V_0.master V_1.master
  /-- (ii) intersectivity on the output: `X f Y  ->  X f Y'  ->  X f (Y  I  Y')`. -/
  inter_right : forall  {X Y Y'}, rel X Y  ->  rel X Y'  ->  rel X (Y  I  Y')
  /-- (iii) monotonicity: a sharper input `X'  subseteq  X` with a blunter output `Y
    subseteq  Y'` is still related,
  provided `X'`, `Y'` are neighbourhoods. -/
  mono : forall  {X X' Y Y'}, rel X Y  ->  X'  subseteq  X  ->  Y  subseteq  Y'  ->
    V_0.mem X'  ->  V_1.mem Y'  ->  rel X' Y'

namespace ApproximableMap

variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b} {V_2 :
    NeighborhoodSystem ?}

/-- **Extensionality for the relation.** Two approximable maps with the same
    neighbourhood relation
are equal (the remaining fields are propositions). -/
theorem ext {f g : ApproximableMap V_0 V_1} (h : forall  X Y, f.rel X Y  <->  g.rel X
    Y) : f = g := by
  obtain <rf, _, _, _, _, _> := f
```

```
  obtain <rg, _, _, _, _, _> := g
  have : rf = rg := by funext X Y; exact propext (h X Y)
  subst this; rfl

end ApproximableMap

namespace NeighborhoodSystem

/-- **Directed union of filters.** The union `? S = {Z ? exists  s  in  S, Z  in  s}`
    of a *non-empty directed*
family `S` of elements (any two members have an upper bound in `S`) is again an element
    . The only
non-trivial law is `inter_mem`: given `Z  in  a` and `Z'  in  b`, an upper bound `c
    supseteq  a, b` contains both,
hence `Z  I  Z'  in  c`. (Generalizes `chainUnion` of Exercise 1.24 from chains to
    directed sets; this is
the construction behind Exercise 2.11 and Scott's finiteness argument in Theorem 2.7.)
     -/
def sSupDirected (V : NeighborhoodSystem a) (S : Set V.Element) (hne : S.Nonempty)
    (hdir : forall  a  in  S, forall  b  in  S, exists  c  in  S, a  <=  c  /\  b  <=
     c) : V.Element where
  mem Z := exists  s  in  S, s.mem Z
  sub := fun <s, _, hs> => s.sub hs
  master_mem := by obtain <s, hs> := hne; exact <s, hs, s.master_mem>
  inter_mem := by
    rintro Z Z' <a, haS, haZ> <b, hbS, hbZ>
    obtain <c, hcS, hac, hbc> := hdir a haS b hbS
    exact <c, hcS, c.inter_mem (hac Z haZ) (hbc Z' hbZ)>
  up_mem := by
    rintro Z Z' <a, haS, haZ> hZ' hZZ'
    exact <a, haS, a.up_mem haZ hZ' hZZ'>

end NeighborhoodSystem
end

end DomainTheory.NeighbourhoodSystems
```

3.3.3 Proposition 2.2 — elementwise action

PROPOSITION 2.2. Given neighbourhood systems D_0 and D_1, an approximable mapping $f : D_0 \to D_1$ always determines a function $f : |D_0| \to |D_1|$ between domains by virtue of the formula:

(i) $f(x) = \{Y \in D_1 \mid \exists X \in x.\, X f Y\}$

for all $x \in |D_0|$. Conversely, this function uniquely determines the original relation by the equivalence:

(ii) $X f Y$ iff $Y \in f(\uparrow X)$

for all $X \in D_0$ and $Y \in D_1$. Approximable functions are always monotone in the following sense:

(iii) $x \subseteq y$ always implies $f(x) \subseteq f(y)$,

for $x, y \in |D_0|$; moreover two approximable functions $f : D_0 \to D_1$ and $g : D_0 \to D_1$ are identical as relations iff

(iv) $f(x) = g(x)$, for all $x \in |D_0|$.

Proof: The argument that formula (i) always gives us $f(x) \in |D_1|$ when $x \in |D_0|$ can be safely left to the reader. Note, however, that all the conditions of 2.1 are required to show this. As for (ii), the implication from left to right follows directly from (i) because $X \in\uparrow X$. In the other direction $Y \in f(\uparrow X)$ means that ZfY holds for some $Z \in\uparrow X$. But from $X \subseteq Z$ it follows that XfY, as we wished.

To prove monotonicity, assume $x \subseteq y$. Now $X \in x$ and XfY always imply $X \in y$ and XfY. This means $Y \in f(x)$ always implies $Y \in f(y)$; that is, $f(x) \subseteq f(y)$.

Finally, to check that (iv) means $f = g$ as relations, all that has to be remarked that this follows from formulae (i) and (ii). □

Note that the right-hand side of (ii) can be written:

$$\uparrow Y \subseteq f(\uparrow X),$$

which can be read as saying that the partial element determined by the neighbourhood Y approximates the function value at the element determined by X. This precise relationship of course fits the informal discussion of mapping given earlier. Indeed whenever $x \in [X]$ and XfY hold, then $f(x) \in [Y]$ always follows, which is another way to construe the mapping character of f. Some examples of mappings are now called for.

Proposition_2_2.lean

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Definition_2_1

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.principal
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem

namespace ApproximableMap

section
variable {a b ? ? : Type*}

variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b} {V_2 :
    NeighborhoodSystem ?}

/- concordance: source L586 PROPOSITION 2.2 elementwise action -/
/-- **Proposition 2.2(i) (Scott 1981, PRG-19).** The elementwise function determined
    by an
approximable mapping: `f(x) = {Y  in  ?_1 ? exists  X  in  x, X f Y}`. The four filter
    laws use *all* of
Definition 2.1: `master_mem` uses (i); `inter_mem` uses (ii) together with (iii) (to
    pull both
outputs back along the common input `X  I  X'`); `up_mem` uses (iii). -/
def toElementMap (f : ApproximableMap V_0 V_1) (x : V_0.Element) : V_1.Element where
  mem Y := exists  X, x.mem X  /\  f.rel X Y
```

```
  sub := fun <_, _, hXY> => f.rel_cod hXY
  master_mem := <V_0.master, x.master_mem, f.master_rel>
  inter_mem := by
    rintro Y Y' <X, hX, hXY> <X', hX', hX'Y'>
    have hXX'mem : x.mem (X  I  X') := x.inter_mem hX hX'
    have hXX' : V_0.mem (X  I  X') := x.sub hXX'mem
    refine <X  I  X', hXX'mem, ?_>
    have h1 : f.rel (X  I  X') Y :=
      f.mono hXY Set.inter_subset_left subset_rfl hXX' (f.rel_cod hXY)
    have h2 : f.rel (X  I  X') Y' :=
      f.mono hX'Y' Set.inter_subset_right subset_rfl hXX' (f.rel_cod hX'Y')
    exact f.inter_right h1 h2
  up_mem := by
    rintro Y Y' <X, hX, hXY> hY' hYY'
    exact <X, hX, f.mono hXY subset_rfl hYY' (x.sub hX) hY'>

@[simp] theorem mem_toElementMap (f : ApproximableMap V_0 V_1) (x : V_0.Element) {Y :
    Set b} :
    (f.toElementMap x).mem Y  <->  exists  X, x.mem X  /\  f.rel X Y := Iff.rfl

/-- **Proposition 2.2(ii) (Scott 1981, PRG-19).** The relation is recovered from the
    elementwise
map: for `X  in  ?_0`, `X f Y  <->  Y  in  f(X)`. (` -> ` since `X  in  X`; ` <- ` since
     any `Z  in  X` has `X  subseteq  Z`,
so `Z f Y` monotonically yields `X f Y`.) -/
theorem rel_iff_mem_principal (f : ApproximableMap V_0 V_1) {X : Set a} (hX : V_0.mem
    X) {Y : Set b} :
    f.rel X Y  <->  (f.toElementMap (V_0.principal hX)).mem Y := by
  constructor
  * intro hXY
    exact <X, <hX, subset_rfl>, hXY>
  * rintro <Z, <_, hXZ>, hZY>
    exact f.mono hZY hXZ subset_rfl hX (f.rel_cod hZY)

/-- **Proposition 2.2(iv) (Scott 1981, PRG-19).** Two approximable maps are *identical
     as relations*
iff they induce the same elementwise function: `(forall  x, f(x) = g(x)) ? f = g`. For
    neighbourhoods `X`
the relation is read off `f(X)` (`rel_iff_mem_principal`); off `?_0` both relations are
     empty. -/
theorem ext_of_toElementMap {f g : ApproximableMap V_0 V_1}
    (h : forall  x, f.toElementMap x = g.toElementMap x) : f = g := by
  apply ApproximableMap.ext
  intro X Y
  by_cases hX : V_0.mem X
  * rw [f.rel_iff_mem_principal hX, g.rel_iff_mem_principal hX, h]
  * constructor
    * intro hr; exact absurd (f.rel_dom hr) hX
    * intro hr; exact absurd (g.rel_dom hr) hX

/-- An approximable map relates any input neighbourhood to the master output `Delta_1`.
     -/
theorem rel_master (f : ApproximableMap V_0 V_1) {X : Set a} (hX : V_0.mem X) :
    f.rel X V_1.master :=
  f.mono f.master_rel (V_0.sub_master hX) subset_rfl hX V_1.master_mem
end
```

```
end ApproximableMap

section
variable {a b ? ? : Type*}

namespace NeighborhoodSystem

/-- The directed union is the least upper bound: an upper bound of every member
    dominates it. -/
theorem sSupDirected_le (V : NeighborhoodSystem a) (S : Set V.Element) (hne : S.
    Nonempty)
    (hdir : forall a in S, forall b in S, exists c in S, a <= c /\ b <=
     c) {y : V.Element}
    (hy : forall s in S, s <= y) : V.sSupDirected S hne hdir <= y := by
  rintro Z <s, hs, hsZ>
  exact hy s hs Z hsZ

/-- Each member of a directed family approximates the directed union. -/
theorem le_sSupDirected (V : NeighborhoodSystem a) (S : Set V.Element) (hne : S.
    Nonempty)
    (hdir : forall a in S, forall b in S, exists c in S, a <= c /\ b <=
     c) {a : V.Element} (ha : a in S) :
    a <= V.sSupDirected S hne hdir :=
  fun _ hZ => <a, ha, hZ>

end NeighborhoodSystem

namespace ApproximableMap

variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b} {V_2 :
    NeighborhoodSystem ?}

/-- **Proposition 2.2(iii) (Scott 1981, PRG-19).** Approximable maps are monotone on
    elements:
`x sqsub y ? f(x) sqsub f(y)`. -/
theorem toElementMap_mono (f : ApproximableMap V_0 V_1) {x y : V_0.Element} (hxy : x
    <= y) :
    f.toElementMap x <= f.toElementMap y := by
  rintro Y <X, hX, hXY>
  exact <X, hxy X hX, hXY>

end ApproximableMap
end

end DomainTheory.NeighbourhoodSystems
```

3.3.4 Example 2.3 — parity of leading zeros

EXAMPLE 2.3. Let T be the neighbourhood system of the two-token domain of Example 1.2. To avoid confusion with some other domains, we will call the two total elements of $|T|$ respectively true and false. There is only one other finite element here, namely $\perp =$ undefined. We often use these elements as indicators of results: true indicates a positive outcome; false, a negative outcome; and $\perp$ indicates that there is not enough information to decide the outcome totally.

Let B be the system for the binary tree as in the last chapter. What we wish to define is an approximable mapping $f : B \to T$. The intuitive idea of the mapping we have in mind is that the binary sequence is being read from left to right, and we are counting the number of 0's seen before the first 1 is encountered. We then test the parity of this count; if it is even, the output is true; if not, false. Using a suggestive informal notation with three dots, some results of the function that does the counting and testing can be written as:

$$f(0000101\ldots) = \text{true}$$
$$f(1101110\ldots) = \text{true}$$
$$f(0111011\ldots) = \text{false}$$
$$f(0000000\ldots) = \perp .$$

The last equation is necessary, because 0000000 as a partial element cannot be counted as either even or odd since it can have inconsistent extensions:

$$0000000\ \perp \sqsubseteq 00000001\ \perp$$
$$0000000\ \perp \sqsubseteq 000000000001\ \perp .$$

So, as far as f is concerned, a plain string of 0's is indefinite. The same answer holds if the 0's go on infinitely.

To be more precise we want

$$f(0^n1\ \perp) = \text{true} \quad \text{if } n \text{ is even;}$$
$$= \text{false} \quad \text{if } n \text{ is odd.}$$

As a binary relation $f \subseteq B \times T$ we will have

XfY iff $Y \in \perp$ or $X \sqsubseteq 0^n1\Delta$ for some $n \in \mathbf{N}$ and either n is even and $Y \in$ true or n is odd and $Y \in$ false.

It should be checked that 2.1(i)–(ii) are satisfied. □

Example_2_3.lean

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_2
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Examples_1_2_1_5_revisited
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.Example23
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ExampleB

section
/-- The two-token codomain `T` of Example 1.2. -/
abbrev T : NeighborhoodSystem Example12.Token := Example12.neighborhoodSystem
end

section
/-- Scott's `False`: the unique partial element `{Delta}` of `T`. -/
def botElt : T.Element := Example12.neighborhoodSystem.bot
end
```

```
section
/-- `False` approximates every element of `T` (the local `{Delta}` form, proved
    directly). -/
theorem botElt_le (x : T.Element) : botElt  <=  x := by
  intro Z hZ
  have : Z = Example12.master := hZ
  subst this
  exact x.master_mem
end

section
/-- Scott's `false`: the total element `{Delta, {1}}` of `T`. -/
def falseElt : T.Element := Example12.neighborhoodSystem.elemOne
end

section
/-- **The parity scanner.** `scan sigma = none` while `sigma` is an unbroken run of
    `0`'s; once a `1`
appears, `scan sigma = some b` with `b = true` iff an even number of `0`'s preceded it.
     A leading `0`
flips the parity of the rest; a leading `1` fixes parity `true` (zero preceding zeros).
     -/
def scan : Str  ->  Option Bool
  | [] => none
  | true :: _ => some true
  | false :: t => (scan t).map (!*)
end

section
@[simp] theorem scan_nil : scan [] = none := rfl
end

section
@[simp] theorem scan_true (t : Str) : scan (true :: t) = some true := rfl
end

section
@[simp] theorem scan_false (t : Str) : scan (false :: t) = (scan t).map (!*) := rfl
end

section
/-- **Stability of the scan under extension.** Once `scan sigma` has committed to a
    parity `some b`,
every extension `sigma ++ t` keeps that value. This is the engine of monotonicity for `
    parityMap`. -/
theorem scan_append {sigma : Str} {b : Bool} (h : scan sigma = some b) (t : Str) :
    scan (sigma ++ t) = some b := by
  induction sigma generalizing b with
  | nil => simp at h
  | cons c sigma_0 ih =>
    cases c with
    | true => simp only [scan_true] at h  |- ; exact h
    | false =>
      simp only [List.cons_append, scan_false] at h  |-
      rw [Option.map_eq_some_iff] at h
```

```
      obtain <a, ha, rfl> := h
      rw [ih ha]
      rfl
end

section
/-- Scott's `true`: the total element `{Delta, {0}}` of `T`. -/
def trueElt : T.Element := Example12.neighborhoodSystem.elemZero
end

section
/-- The codomain element selected by a parity reading: `none  |->  False`, `some true
    |->  true`,
`some false  |->  false`. -/
def valElt : Option Bool  ->  T.Element
  | none => botElt
  | some true => trueElt
  | some false => falseElt
end

section
/-- **Monotonicity of the parity value.** A longer prefix `sigma <+: sigma'` yields a (
    weakly) more defined
value: `valElt (scan sigma)  sqsub  valElt (scan sigma')`. If `scan sigma = none` then
    `valElt = False  sqsub  _`; otherwise the
value is fixed by `scan_append`. -/
theorem valElt_scan_mono {sigma sigma' : Str} (h : sigma <+: sigma') :
    valElt (scan sigma)  <=  valElt (scan sigma') := by
  obtain <t, rfl> := h
  cases hsigma : scan sigma with
  | none => simpa [valElt, hsigma] using botElt_le _
  | some b => rw [scan_append hsigma t]
end

section
/- concordance: source L620 EXAMPLE 2.3 parity of leading zeros -/
/-- **Example 2.3 -- the parity mapping `f : B  ->  T`.** `X f Y` iff `X` is the cone `
    sigmaSigma*` of some
prefix `sigma` and `Y` is approximated by the parity verdict `valElt (scan sigma)`.
    Definition 2.1:
(i) the empty prefix scans to `none = False`, and `False` contains `Delta_T`; (ii) for
    a fixed cone the verdict
is a *single* filter (cones have a unique prefix), closed under ` I `; (iii) extending
    the prefix only
sharpens the verdict (`valElt_scan_mono`). -/
def parityMap : ApproximableMap B T where
  rel X Y := exists  sigma, X = cone sigma  /\  (valElt (scan sigma)).mem Y
  rel_dom := fun <sigma, hX, _> => <sigma, hX>
  rel_cod := fun <_, _, hY> => (valElt _).sub hY
  master_rel := by
    refine <[], cone_nil.symm, ?_>
    show (botElt).mem Example12.master
    rfl
  inter_right := by
    rintro X Y Y' <sigma, hX, hY> <sigma', hX', hY'>
    have hsigma : sigma = sigma' := cone_injective (hX  |>  hX')
```

```
    subst hsigma
    exact <sigma, hX, (valElt (scan sigma)).inter_mem hY hY'>
  mono := by
    rintro X X' Y Y' <sigma, hX, hY> hX'X hYY' hX'mem hY'mem
    obtain <sigma', hX'cone> := hX'mem
    have hpre : sigma <+: sigma' := by
      apply cone_subset_cone.mp
      rw [ <-  hX'cone,  <-  hX]; exact hX'X
    have hYmem' : (valElt (scan sigma)).mem Y' := (valElt (scan sigma)).up_mem hY hY'
    mem hYY'
    exact <sigma', hX'cone, valElt_scan_mono hpre Y' hYmem'>
end

section
/-- **`False` is the unique common lower bound of `true` and `false`.** Any `a : |?|`
    approximated by
*both* total elements is `False` -- `?`'s flatness. (Exercise 2.16's uniqueness
    argument: reading no
tokens yet is the only state consistent with committing to either verdict.) -/
theorem eq_botElt_of_le {a : T.Element} (h0 : a  <=  trueElt) (h1 : a  <=  falseElt) :
     a = botElt :=
  Example12.neighborhoodSystem.eq_bot_of_le_elemZero_of_le_elemOne h0 h1
end

section
/-- **`parityMap`'s relation on a cone, read off.** `cone sigma f Y  <->  Y  in  valElt
    (scan sigma)` -- the
defining existential collapses since a cone has a *unique* generating prefix (`
    cone_injective`).
(Used throughout Exercise 2.16's uniqueness argument.) -/
theorem parityMap_rel_cone (sigma : Str) (Y : Set Example12.Token) :
    parityMap.rel (cone sigma) Y  <->  (valElt (scan sigma)).mem Y := by
  constructor
  * rintro <?, h?, hval>
    rw [cone_injective h?]
    exact hval
  * intro hval
    exact <sigma, rfl, hval>
end

section
/-- **`parityMap`'s elementwise value, read off cone-by-cone.** `f(x).mem Y` iff *some*
     cone
`sigmaSigma*  in  x` already witnesses `Y  in  valElt(scan sigma)` -- every
    neighbourhood of `x` is a cone
(`Element.sub`), and `parityMap`'s relation is exactly `parityMap_rel_cone`. -/
theorem parityMap_toElementMap_mem (x : B.Element) (Y : Set Example12.Token) :
    (parityMap.toElementMap x).mem Y  <->  exists  tau, x.mem (cone tau)  /\  (valElt
    (scan tau)).mem Y := by
  constructor
  * rintro <X, hX, hrel>
    obtain <tau, rfl> := x.sub hX
    exact <tau, hX, (parityMap_rel_cone tau Y).mp hrel>
  * rintro <tau, htau, hval>
    exact <cone tau, htau, (parityMap_rel_cone tau Y).mpr hval>
end
```

```
section
/-- **The "shift formula" for `parityMap`.** Reading off the parity of `sigmax` depends
     on `x` only
through *some* cone `cone tau  in  x`, via the scan of the concatenation `sigma ++ tau
    `: any deeper cone in
`sigmaElt`'s built-in up-closure is subsumed by monotonicity (`valElt_scan_mono`), so
    the longest
one -- `sigma ++ tau` itself -- already witnesses the answer. This is the key
    computation behind Exercise
2.16's second half: `parityMap` genuinely satisfies its own defining equations. -/
theorem parityMap_toElementMap_sigmaElt (sigma : Str) (x : B.Element) (Y : Set
    Example12.Token) :
    (parityMap.toElementMap (sigmaElt sigma x)).mem Y  <->
      exists  tau, x.mem (cone tau)  /\  (valElt (scan (sigma ++ tau))).mem Y := by
  constructor
  * rintro <Z, <_, X, hX, hsub>, ?, rfl, hval>
    obtain <tau, rfl> := x.sub hX
    rw [prepend_cone] at hsub
    exact <tau, hX, valElt_scan_mono (cone_subset_cone.mp hsub) Y hval>
  * rintro <tau, htau, hval>
    exact <cone (sigma ++ tau), <memB_cone _, cone tau, htau, (prepend_cone sigma tau)
    .le>, sigma ++ tau, rfl, hval>
end

section
/-- **Two leading `0`'s cancel.** `scan([0,0]++tau) = scan tau`: reading two extra
    zeros flips the
parity twice, i.e. not at all (`Bool.not_not`). (Exercise 2.16's third equation, at the
     token
level -- the recursive step behind the uniqueness argument for the parity map.) -/
theorem scan_append_falseFalse (tau : Str) : scan ([false, false] ++ tau) = scan tau
    := by
  show scan (false :: false :: tau) = scan tau
  cases h : scan tau with
  | none => simp [scan_false, h]
  | some b => simp [scan_false, h, Bool.not_not]
end

section
/-- **A leading `01` always commits to `false`, whatever follows.** `scan([0,1]++tau) =
     false`, for
*every* tail `tau`. (Exercise 2.16's second equation, at the token level.) -/
theorem scan_append_falseTrue (tau : Str) : scan ([false, true] ++ tau) = some false
    := by
  show scan (false :: true :: tau) = some false
  simp [scan_false, scan_true]
end

section
/-- **A leading `1` always commits to `true`, whatever follows.** `scan([1]++tau) =
    true`, for
*every* tail `tau`. (Exercise 2.16's first equation, at the token level.) -/
theorem scan_append_true (tau : Str) : scan ([true] ++ tau) = some true :=
  scan_true tau
end
```

```
end DomainTheory.NeighbourhoodSystems.Example23
```

3.3.5 Example 2.4 — erase a run of ones

EXAMPLE 2.4. Let us briefly describe an approximable mapping $g : B \to B$. Informally, g can be said to "read a sequence from left to right and eliminate the first consecutive run of 1's while copying all the other digits as read." We will have

$$g(0^n 1^k 0x) = 0^{n+1}x$$

provided $k > 0$. (Here 1^k means a string of 1's of length k.) However, if 1^∞ is the infinite sequences of 1's, then

$$g(1^\infty) = \bot, \text{ and}$$
$$g(0^n 1^\infty) = 0^n.$$

This example is instructive, since it shows that a non-trivial mapping can transform a total element into a partial element. □

Aside from our being able to define particular functions outright, we can combine functions in many different ways; the idea of composition is probably the most basic scheme of combination, and there is a technical name for a family of structures with mappings that can be so combined.

Example_2_4.lean

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Exercise_2_17

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_8
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.Example24
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ExampleB

section
/- concordance: source L663 EXAMPLE 2.4 erase a run of ones -/
/-- **Example 2.4 -- the run-eliminating mapping `g : B  ->  B`.** `X g Y` iff `X =
    sigmaSigma*` and the cone of
the guaranteed output `(out sigma)Sigma*` approximates `Y`. Definition 2.1: (i) `out []
     = []` so `Delta g Delta`;
(ii) a fixed cone has a unique prefix, and the value is the principal filter `((out
    sigma)Sigma*)`, closed
under ` I `; (iii) `out_mono` shrinks the output cone as the input cone shrinks. -/
def runMap : ApproximableMap B B where
  rel X Y := exists  sigma, X = cone sigma  /\  B.mem Y  /\  cone (out sigma)
    subseteq  Y
  rel_dom := fun <sigma, hX, _, _> => <sigma, hX>
  rel_cod := fun <_, _, hYmem, _> => hYmem
  master_rel := by
    refine <[], (B_master).trans cone_nil.symm, B.master_mem, ?_>
```

```
    rw [out_nil, B_master, cone_nil]
  inter_right := by
    rintro X Y Y' <sigma, hX, hYmem, hYsub> <sigma', hX', hY'mem, hY'sub>
    have hsigma : sigma = sigma' := cone_injective (hX  |>  hX')
    subst hsigma
    have hsub : cone (out sigma)  subseteq  Y  I  Y' := Set.subset_inter hYsub hY'sub
    exact <sigma, hX, B.inter_mem hYmem hY'mem (memB_cone (out sigma)) hsub, hsub>
  mono := by
    rintro X X' Y Y' <sigma, hX, _, hYsub> hX'X hYY' hX'mem hY'mem
    obtain <sigma', hX'cone> := hX'mem
    have hpre : sigma <+: sigma' := by
      apply cone_subset_cone.mp
      rw [ <-  hX'cone,  <-  hX]; exact hX'X
    have hcone : cone (out sigma')  subseteq  cone (out sigma) := cone_subset_cone.mpr
     (out_mono hpre)
    exact <sigma', hX'cone, hY'mem, (hcone.trans hYsub).trans hYY'>
end

section
/-- **A leading `0` always emits it and continues copying.** `out([0]++tau) = 0::out
    tau`, for every
tail `tau`. (Exercise 2.17's first equation, at the token level.) -/
theorem out_append_false (tau : Str) : out ([false] ++ tau) = false :: out tau := rfl
end

section
/-- **`10` terminates the run and emits the `0`, whatever follows.** `out([1,0]++tau) =
     0::tau`, for
every tail `tau`. (Exercise 2.17's third equation, at the token level.) -/
theorem out_append_trueFalse (tau : Str) : out ([true, false] ++ tau) = false :: tau
    := rfl
end

section
/-- **A second consecutive `1` changes nothing.** `out([1,1]++tau) = out([1]++tau)`:
    once inside the
run of `1`'s, an extra `1` is simply swallowed by `del`. (Exercise 2.17's second
    equation, at the
token level -- the recursive step behind the uniqueness argument for `runMap`.) -/
theorem out_append_trueTrue (tau : Str) : out ([true, true] ++ tau) = out ([true] ++
    tau) := rfl
end

section
/-- Cons-form restatement of `out_append_trueFalse`, for peeling one token at a time in
     `key`. -/
theorem out_true_false (t : Str) : out (true :: false :: t) = false :: t := rfl
end

section
/-- Cons-form restatement of `out_append_trueTrue`, for peeling one token at a time in
    `key`. -/
theorem out_true_true (t : Str) : out (true :: true :: t) = out (true :: t) := rfl
end

section
```

```
/-- **The recursive core of uniqueness.** Any `g` satisfying the four equations agrees
    with
`runMap` on every principal (finite) element `sigmaFalse`. The recursion peels *one*
    token at a time,
matching `out`'s own case split: a leading `0` recurses via `hg.zero` (`Basic
    decreasing case`);
inside a run of `1`'s, a further `1` recurses via `hg.oneOne` (peeling to `1tau`, still
     one token
shorter); a `1` followed by `0` is pinned *directly* by `hg.oneZero` for any tail (no
    recursion);
the lone token `1` (nothing more) is pinned directly by `hg.one`; and the base case `
    sigma = []` is
the one place order theory enters: `False  <=  1False` forces `g(False)  <=  g(1False)
    = False` by monotonicity, and `False` is
already the least element, so `g(False) = False`. -/
theorem key (g : ApproximableMap B B) (hg : SatisfiesRunEquations g) :
    forall  sigma : Str, g.toElementMap (sigmaElt sigma B.bot) = sigmaElt (out sigma)
    B.bot
  | [] => by
      rw [show out ([] : Str) = ([] : Str) from rfl, sigmaElt_nil]
      have hm := g.toElementMap_mono (B.bot_le (sigmaElt [true] B.bot))
      rw [hg.one] at hm
      exact le_antisymm hm (B.bot_le _)
  | [true] => by
      rw [show out [true] = ([] : Str) from rfl, sigmaElt_nil]
      exact hg.one
  | true :: false :: t => by
      rw [out_true_false, show (true :: false :: t : Str) = [true, false] ++ t from
    rfl,
        sigmaElt_append, hg.oneZero,  <-  sigmaElt_append]
      rfl
  | true :: true :: t => by
      rw [out_true_true, show (true :: true :: t : Str) = [true, true] ++ t from rfl,
        sigmaElt_append, hg.oneOne,  <-  sigmaElt_append]
      exact key g hg (true :: t)
  | false :: t => by
      rw [out_false, show (false :: t : Str) = [false] ++ t from rfl, sigmaElt_append,
     hg.zero,
        key g hg t, show (false :: out t : Str) = [false] ++ out t from rfl,
    sigmaElt_append]
end

section
/-- **`runMap`'s relation on a cone, read off.** `sigmaSigma* g Y  <->  Y  in  B  /\  (
    out sigma)Sigma*  subseteq  Y` -- the defining
existential collapses since a cone has a *unique* generating prefix (`cone_injective`).
     -/
theorem runMap_rel_cone (sigma : Str) (Y : Set Str) :
    runMap.rel (cone sigma) Y  <->  B.mem Y  /\  cone (out sigma)  subseteq  Y := by
  constructor
  * rintro <?, h?, hYmem, hsub>
    obtain rfl : ? = sigma := (cone_injective h?).symm
    exact <hYmem, hsub>
  * rintro <hYmem, hsub>
    exact <sigma, rfl, hYmem, hsub>
end
```

```
end DomainTheory.NeighbourhoodSystems.Example24
```

3.3.6 Theorem 2.5 — category of approximable maps

THEOREM 2.5. The class of neighbourhood systems and approximable mappings form a *category*, where the identity mapping $I_D : D \to D$ relates $X, Y \in D$ as follows:

(i) $X\ I_D\ Y$ iff $X \subseteq Y$.

If $f : D_0 \to D_1$ and $g : D_1 \to D_2$ are given, then the *composition* $g \circ f : D_0 \to D_2$ relates $X \in D_0$ and $Z \in D_2$ as follows:

(ii) $X\ g \circ f\ Z$ iff $\exists Y \in D_1.\ X\ f\ Y$ and $Y\ g\ Z$.

Proof: (We may use MacLane [1971] as the standard reference on category theory, but we require hardly more than the basic definitions at this stage.) To check that we have a category, we need to know that the identity and composition maps really are maps in the category and that certain identity and associative laws hold. Now it is obvious that I_D satisfies 2.1 (i)-(iii). Moreover if $f : D_0 \to D_1$, all we have to prove is:

$$f \circ I_{D_0} = I_{D_1} \circ f = f$$

Checking one of these equations is enough. Thus, for $X \in D_0$ and $Z \in D_1$ we find

$$\begin{aligned} X\ f \circ I_{D_0}\ Z &\text{ iff } \exists Y \in D_0.\ X \subseteq Y \text{ and } Y\ f\ Z \\ &\text{iff } X\ f\ Z. \end{aligned}$$

So, f and $f \circ I_{D_0}$ are the same mapping.

Suppose now that $f : D_0 \to D_1$ and $g : D_1 \to D_2$. We have to verify that $g \circ f$ is an approximable mapping. First off, there is no trouble in seeing that $\Delta_0\ g \circ f\ \Delta_2$ holds. Next, suppose that $X\ g \circ f\ Z$ and $X\ g \circ f\ Z'$ hold. Then we have $X\ f\ Y$ and $Y\ g\ Z$ for some choice of $Y \in D_1$. Also $X\ f\ Y'$ and $Y'\ g\ Z'$ hold for some choice of $Y' \in D_1$. By 2.1 (ii) it follows that $X\ f\ (Y \cap Y')$. Since $Y \cap Y' \subseteq Y$, we conclude $(Y \cap Y')\ g\ Z$ by 2.1 (iii); similarly $(Y \cap Y')\ g\ Z'$. Invoking 2.1 (ii) again, we obtain $(Y \cap Y')\ g\ (Z \cap Z')$, and $X\ g \circ f\ (Z \cap Z')$ is proved.

Suppose finally that $X' \subseteq X$, $X\ g \circ f\ Z$, and $Z \subseteq Z'$. Now $X\ f\ Y$ and $Y\ g\ Z$ for some $Y \in D_1$. But then $X'\ f\ Y$ holds; for a similar reason $Y\ g\ Z'$ holds also. Therefore, $X'\ g \circ f\ Z'$ is established, which means that we have checked 2.1 (iii) for $g \circ f$ and have completed the proof that $g \circ f : D_0 \to D_2$.

The verification of associativity is a purely logical deduction. Thus suppose that in addition to f and g we have $h : D_2 \to D_3$. If $X \in D_0$ and $W \in D_3$ we find

$$\begin{aligned} X\ h \circ (g \circ f)\ W &\text{ iff } \exists Z \in D_2.\ X\ g \circ f\ Z \text{ and } Z\ h\ W \\ &\text{iff } \exists Z \in D_2\ \exists Y \in D_1.\ X\ f\ Y \text{ and } Y\ g\ Z \text{ and } Z\ h\ W \\ &\text{iff } \exists Y \in D_1\ \exists Z \in D_2.\ X\ f\ Y \text{ and } Y\ g\ Z \text{ and } Z\ h\ W \\ &\text{iff } \exists Y \in D_1.\ X\ f\ Y \text{ and } Y\ (h \circ g)\ W \\ &\text{iff } X\ (h \circ g) \circ f\ W. \end{aligned}$$

So, as relations, $h \circ (g \circ f) = (h \circ g) \circ f$. $\square$

It may seem as though we have, in the definition of composition, written things backwards. But the reason is that when mappings are taken as elementwise functions, then the order is preserved in expressions involving the usual function value notation. We have, for example:

Theorem_2_5.lean

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_2

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem

namespace ApproximableMap

section
variable {a b ? ? : Type*}

variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b} {V_2 :
    NeighborhoodSystem ?}

/-- **Theorem 2.5(i) (Scott 1981, PRG-19) -- the identity mapping `I_D`.** `X I_D Y
    <->  X  subseteq  Y`
(confined to `?  x  ?`). It is approximable: (i) `Delta  subseteq  Delta`; (ii) `X
    subseteq  Y`, `X  subseteq  Y'` give `X  subseteq  Y  I  Y'`
with witness `X`; (iii) is transitivity `X'  subseteq  X  subseteq  Y  subseteq  Y'`.
    -/
def idMap (V : NeighborhoodSystem a) : ApproximableMap V V where
  rel X Y := V.mem X  /\  V.mem Y  /\  X  subseteq  Y
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := <V.master_mem, V.master_mem, subset_rfl>
  inter_right := by
    rintro X Y Y' <hX, hY, hXY> <_, hY', hXY'>
    exact <hX, V.inter_mem hY hY' hX (Set.subset_inter hXY hXY'), Set.subset_inter hXY
     hXY'>
  mono := by
    rintro X X' Y Y' <_, _, hXY> hX'X hYY' hX' hY'
    exact <hX', hY', (hX'X.trans hXY).trans hYY'>

@[simp] theorem idMap_rel {V : NeighborhoodSystem a} {X Y : Set a} :
    (idMap V).rel X Y  <->  V.mem X  /\  V.mem Y  /\  X  subseteq  Y := Iff.rfl

/-- **Theorem 2.5(ii) (Scott 1981, PRG-19) -- composition `g  comp  f`.** `X (g comp f)
     Z  <->  exists  Y, X f Y  /\  Y g Z`.
Approximability is Scott's verification: (i) use `Y = Delta_1`; (ii) intersect both
    witnesses via
`f.inter_right` then `g.inter_right` (narrowing the inner neighbourhood with `g.mono`);
     (iii) narrow
the input with `f.mono` and widen the output with `g.mono`, keeping the same witness.
    -/
def comp (g : ApproximableMap V_1 V_2) (f : ApproximableMap V_0 V_1) : ApproximableMap
     V_0 V_2 where
  rel X Z := exists  Y, f.rel X Y  /\  g.rel Y Z
```

```
  rel_dom := fun <_, hXY, _> => f.rel_dom hXY
  rel_cod := fun <_, _, hYZ> => g.rel_cod hYZ
  master_rel := <V_1.master, f.master_rel, g.master_rel>
  inter_right := by
    rintro X Z Z' <Y, hXY, hYZ> <Y', hXY', hY'Z'>
    refine <Y  I  Y', f.inter_right hXY hXY', ?_>
    have hYY'mem : V_1.mem (Y  I  Y') := f.rel_cod (f.inter_right hXY hXY')
    have h1 : g.rel (Y  I  Y') Z :=
      g.mono hYZ Set.inter_subset_left subset_rfl hYY'mem (g.rel_cod hYZ)
    have h2 : g.rel (Y  I  Y') Z' :=
      g.mono hY'Z' Set.inter_subset_right subset_rfl hYY'mem (g.rel_cod hY'Z')
    exact g.inter_right h1 h2
  mono := by
    rintro X X' Z Z' <Y, hXY, hYZ> hX'X hZZ' hX' hZ'
    refine <Y, f.mono hXY hX'X subset_rfl hX' (f.rel_cod hXY), ?_>
    exact g.mono hYZ subset_rfl hZZ' (g.rel_dom hYZ) hZ'

@[simp] theorem comp_rel {g : ApproximableMap V_1 V_2} {f : ApproximableMap V_0 V_1} {
    X : Set a}
    {Z : Set ?} : (g.comp f).rel X Z  <->  exists  Y, f.rel X Y  /\  g.rel Y Z := Iff.
    rfl

@[simp] theorem toElementMap_idMap (x : V_0.Element) : (idMap V_0).toElementMap x = x
    := by
  apply Element.ext
  intro Y
  constructor
  * rintro <X, hXx, _, hY, hXY>
    exact x.up_mem hXx hY hXY
  * intro hY
    exact <Y, hY, x.sub hY, x.sub hY, subset_rfl>

/- concordance: source L682 THEOREM 2.5 category of approximable maps -/
/-- **Theorem 2.5 -- associativity.** `h  comp  (g  comp  f) = (h  comp  g)  comp  f`.
    Pure reassociation of the
existential witnesses. -/
theorem comp_assoc {V_3 : NeighborhoodSystem ?} (h : ApproximableMap V_2 V_3)
    (g : ApproximableMap V_1 V_2) (f : ApproximableMap V_0 V_1) :
    (h.comp g).comp f = h.comp (g.comp f) := by
  apply ApproximableMap.ext
  intro X W
  constructor
  * rintro <Y, hXY, Z, hYZ, hZW>
    exact <Z, <Y, hXY, hYZ>, hZW>
  * rintro <Z, <Y, hXY, hYZ>, hZW>
    exact <Y, hXY, Z, hYZ, hZW>

/-- **Theorem 2.5 -- right identity law.** `f  comp  I_{D_0} = f`. (` -> `: a witness `
    X  subseteq  Y` sharpens the
input of `f` by `f.mono`; ` <- `: take `Y = X`.) -/
theorem comp_idMap (f : ApproximableMap V_0 V_1) : f.comp (idMap V_0) = f := by
  apply ApproximableMap.ext
  intro X Z
  constructor
  * rintro <Y, <hX, _, hXY>, hYZ>
    exact f.mono hYZ hXY subset_rfl hX (f.rel_cod hYZ)
```

```
  * intro hXZ
    exact <X, <f.rel_dom hXZ, f.rel_dom hXZ, subset_rfl>, hXZ>

/-- **Theorem 2.5 -- left identity law.** `I_{D_1}  comp  f = f`. (` -> `: a witness `Y
      subseteq  Z` widens the output
of `f` by `f.mono`; ` <- `: take `Y = Z`.) -/
theorem idMap_comp (f : ApproximableMap V_0 V_1) : (idMap V_1).comp f = f := by
  apply ApproximableMap.ext
  intro X Z
  constructor
  * rintro <Y, hXY, _, hZ, hYZ>
    exact f.mono hXY subset_rfl hYZ (f.rel_dom hXY) hZ
  * intro hXZ
    exact <Z, hXZ, f.rel_cod hXZ, f.rel_cod hXZ, subset_rfl>
end

end ApproximableMap

end DomainTheory.NeighbourhoodSystems
```

3.3.7 Proposition 2.6 — identity and composition

PROPOSITION 2.6. Given $f : D_0 \to D_1$ and $g : D_1 \to D_2$, the following equations hold:

(i) $I_{D_0}(x) = x$, and

(ii) $(g \circ f)(x) = g(f(x))$,

for all $x \in |D_0|$. □

The proof is not troublesome and is left as an exercise. In technical language the result shows that the category defined in Theorem 2.5 is equivalent to a "concrete category" of sets and functions, namely the domains and elementwise transformations of 2.2.

Toward the end of the last lecture (see 1.9) we promised to show that isomorphisms of domains always come from approximable mappings, and this we now do. It means that the category contains all the isomorphisms it should have.

Proposition_2_6.lean

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Theorem_2_5
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem

section
variable {a b ? ? : Type*}

namespace ApproximableMap

variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b} {V_2 :
    NeighborhoodSystem ?}
```

```
/- concordance: source L731 PROPOSITION 2.6 identity and composition -/
/-- **Proposition 2.6(ii) (Scott 1981, PRG-19).** Composition of approximable mappings
     becomes
composition of the elementwise functions: `(g  comp  f)(x) = g(f(x))`. Both sides
    unfold to
`exists  Y X, x.mem X  /\  X f Y  /\  Y g Z`; the proof is a reassociation of
    existentials. -/
theorem toElementMap_comp (g : ApproximableMap V_1 V_2) (f : ApproximableMap V_0 V_1)
    (x : V_0.Element) :
    (g.comp f).toElementMap x = g.toElementMap (f.toElementMap x) := by
  apply Element.ext
  intro Z
  constructor
  * rintro <X, hXx, Y, hXY, hYZ>
    exact <Y, <X, hXx, hXY>, hYZ>
  * rintro <Y, <X, hXx, hXY>, hYZ>
    exact <X, hXx, Y, hXY, hYZ>

/-- Commutation `f  comp  g = g  comp  f` read elementwise: `f(g(x)) = g(f(x))`. -/
theorem commute_apply {V : NeighborhoodSystem a} {f g : ApproximableMap V V}
    (hcomm : f.comp g = g.comp f) (x : V.Element) :
    f.toElementMap (g.toElementMap x) = g.toElementMap (f.toElementMap x) := by
  have h1 : (f.comp g).toElementMap x = (g.comp f).toElementMap x := by rw [hcomm]
  rwa [toElementMap_comp, toElementMap_comp] at h1

end ApproximableMap
end

end DomainTheory.NeighbourhoodSystems
```

3.3.8 Theorem 2.7 — isomorphisms are approximable

THEOREM 2.7. Every isomorphism between domains results from an approximable mapping between the neighbourhood systems. Moreover, finite elements are always transformed into finite elements.

Proof: Suppose that $f : |D_0| \to |D_1|$ is a one-one, inclusion-preserving function defined on elements, where the range of the function is the whole of $|D_1|$, of course. Taking the hint from 2.2, there is only one way we could define a neighbourhood mapping; namely, we consider the relation $Y \in f(\uparrow X)$ for $X \in D_0$ and $Y \in D_1$. What has to be shown is that this is an approximable mapping which determines the original function via the formula 2.2 (i).

The first part is easy; indeed, there is a general result that monotone functions on finite elements of one domain to arbitrary elements of another domain always determine approximable mappings (cf. Exercise 2.8). What remains, then, is to show that the relation re-defines the function. This comes down to showing that for $x \in |D_0|$

$$f(x) = \{Y \in D_1 \mid \exists X \in x.\, Y \in f(\uparrow X)\}.$$

Consider the right-hand side of this equation: it is a filter. (This either can be proved directly or Exercise 2.11 can be used.) Because f is an onto-function, we can call the right-hand side $f(x')$ for some $x' \in |D_0|$. But since $X \in x$ implies $\uparrow X \subseteq x$ and $f(\uparrow X) \subseteq f(x)$, the right-hand side is included in the left-hand side. In other words $f(x') \subseteq f(x)$. But, since f is an isomorphism, $x' \subseteq x$ follows.

In the other direction, if $X \in x$, then $f(\uparrow X) \subseteq f(x')$ holds by definition, so $\uparrow X \subseteq x'$. This implies $X \in x'$; and, as X is arbitrary, $x \subseteq x'$. So $x = x'$, and $f(x) = f(x')$ as desired.

Finally, consider any finite element $\uparrow X \in |D_0|$, where $X \in D_0$. What we have to show is that $f(\uparrow X)$ is finite in $|D_1|$. Because f is an isomorphism, we can associate uniquely to every $Y \in f(\uparrow X)$ an element $y_Y \subseteq\uparrow X$ in $|D_0|$ where $f(y_Y) =\uparrow Y$. (Just apply the inverse of the function f.) Define

$$z = \bigcup\{y_Y \mid Y \in f(\uparrow X)\}.$$

Because $Y' \subseteq Y$ always implies $y_{Y'} \subseteq y_Y$ and each $y_Y \in |D_0|$, it is easy to show z is a filter and hence is in $|D_0|$ also (cf. Exercise 2.11). Because each $y_Y \subseteq\uparrow X$, then $z \subseteq\uparrow X$, too. But each $y_Y \subseteq z$, so $\uparrow Y = f(y_Y) \subseteq f(z)$ and hence $Y \in f(z)$. As this holds for all $Y \in f(\uparrow X)$, the inclusion $f(\uparrow X) \subseteq f(z)$ follows, as well as $\uparrow X \subseteq z$. Therefore, $z =\uparrow X$ and so $X \in z$. But then $X \in y_Y$ for some $Y \in f(\uparrow X)$, by definition of z. Since $\uparrow X \subseteq y_Y$, we obtain $f(\uparrow X) \subseteq\uparrow Y$, but of course the opposite inclusion is also true from the choice of Y. This means that $f(\uparrow X) =\uparrow Y$ is finite in $|D_1|$ as claimed. We can apply the same argument to the inverse function; and, thus, the finite elements of $|D_0|$ and $|D_1|$ are in a one-one inclusion-preserving correspondence under the isomorphism. □

Theorem_2_7.lean

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_2
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_7
public import Mathlib.Order.Hom.Basic
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem

namespace ApproximableMap

section
variable {a b ? ? : Type*}

variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b} {V_2 :
    NeighborhoodSystem ?}

/- concordance: source L743 THEOREM 2.7 isomorphisms are approximable -/
/-- **Theorem 2.7 (Scott 1981, PRG-19) -- the approximable map of an isomorphism.**
    Given a domain
isomorphism `e : |?_0| ?o |?_1|` (Definition 1.9), Scott's "only way to define a
    neighbourhood
mapping" is the relation `X f Y  <->  Y  in  e(X)`. The conditions of 2.1 hold because
    `e` is monotone:
(i) `Delta_1  in  e(False_0)` is `master_mem`; (ii) is `inter_mem` of the filter `e(X)
    `; (iii) sharpening `X'  subseteq  X`
means `X  sqsub  X'`, so `e(X)  sqsub  e(X')` and the output transports along, then
    widens by `up_mem`. -/
def ofIso (e : V_0.Element ?o V_1.Element) : ApproximableMap V_0 V_1 where
  rel X Y := exists  _ : V_0.mem X, (e (V_0.principal ?V_0.mem X?)).mem Y
  rel_dom := fun h => h.1
  rel_cod := fun h => (e _).sub h.2
```

```
  master_rel := <V_0.master_mem, (e _).master_mem>
  inter_right := by
    rintro X Y Y' <hX, hY> <_, hY'>
    exact <hX, (e (V_0.principal hX)).inter_mem hY hY'>
  mono := by
    rintro X X' Y Y' <hX, hY> hX'X hYY' hX' hY'
    refine <hX', ?_>
    have hle : V_0.principal hX  <=  V_0.principal hX' := (V_0.principal_le_iff hX hX
    ').mpr hX'X
    have hmem : (e (V_0.principal hX')).mem Y := (e.monotone hle) Y hY
    exact (e (V_0.principal hX')).up_mem hmem hY' hYY'

/-- **Theorem 2.7 (Scott 1981, PRG-19) -- finite elements go to finite elements.** A
    domain
isomorphism `e` carries the finite (principal) element `X` to a finite element `Y` of
    the other
domain. Following Scott: with `w = e(X)`, the set `S = {e^-1(Y) ? Y  in  w}` is
    directed (intersections
of members of `w` give upper bounds), so its union `z = ? S` is an element (`
    sSupDirected`). One shows
`z = X` (each `e^-1(Y)  sqsub  e^-1(w) = X`, and conversely `w  sqsub  e(z)` forces `X
    = e^-1(w)  sqsub  z`); then
`X  in  z` lands in some `e^-1(Y)`, giving `w  sqsub  Y`, while `Y  sqsub  w` is
    automatic -- so `w = Y`. -/
theorem exists_principal_eq_apply_principal (e : V_0.Element ?o V_1.Element)
    {X : Set a} (hX : V_0.mem X) :
    exists  (Y : Set b) (hY : V_1.mem Y), e (V_0.principal hX) = V_1.principal hY :=
    by
  -- `w = e(X)`, and the directed family `S` of inverse images of principals of members
     of `w`.
  set w : V_1.Element := e (V_0.principal hX) with hw
  set S : Set V_0.Element :=
    {z | exists  (Y : Set b) (hY : V_1.mem Y), w.mem Y  /\  z = e.symm (V_1.principal
    hY)} with hS
  -- `e^-1(w) = X`.
  have hsymm_w : e.symm w = V_0.principal hX := by rw [hw, e.symm_apply_apply]
  -- For `Y  in  w`, `Y  sqsub  w`.
  have hprin_le_w : forall  {Y : Set b} (hY : V_1.mem Y), w.mem Y  ->  V_1.principal
    hY  <=  w :=
    fun hY hYw Z hZ => w.up_mem hYw hZ.1 hZ.2
  -- `S` is non-empty (use `Y = Delta_1`).
  have hne : S.Nonempty :=
    <e.symm (V_1.principal V_1.master_mem), V_1.master, V_1.master_mem, w.master_mem,
    rfl>
  -- `S` is directed: intersect the two members of `w`.
  have hdir : forall  a  in  S, forall  b  in  S, exists  c  in  S, a  <=  c  /\  b
    <=  c := by
    rintro a <Y, hY, hYw, rfl> b <Y', hY', hY'w, rfl>
    have hYY'w : w.mem (Y  I  Y') := w.inter_mem hYw hY'w
    have hYY' : V_1.mem (Y  I  Y') := w.sub hYY'w
    refine <e.symm (V_1.principal hYY'), <Y  I  Y', hYY', hYY'w, rfl>, ?_, ?_>
    * exact e.symm.monotone ((V_1.principal_le_iff hY hYY').mpr Set.inter_subset_left)
    * exact e.symm.monotone ((V_1.principal_le_iff hY' hYY').mpr Set.
    inter_subset_right)
  -- The directed union `z = ? S`.
  set z : V_0.Element := V_0.sSupDirected S hne hdir with hz
```

```
  -- `z  sqsub  X`: every member `e^-1(Y)  sqsub  e^-1(w) = X`.
  have hz_le : z  <=  V_0.principal hX := by
    apply V_0.sSupDirected_le
    rintro s <Y, hY, hYw, rfl>
    have : e.symm (V_1.principal hY)  <=  e.symm w := e.symm.monotone (hprin_le_w hY
    hYw)
    rwa [hsymm_w] at this
  -- `X  sqsub  z`: show `w  sqsub  e(z)`, then `X = e^-1(w)  sqsub  z`.
  have hw_le_ez : w  <=  e z := by
    intro Y hYw
    have hY : V_1.mem Y := w.sub hYw
    have hmem_S : e.symm (V_1.principal hY)  in  S := <Y, hY, hYw, rfl>
    have h1 : e.symm (V_1.principal hY)  <=  z := V_0.le_sSupDirected S hne hdir
    hmem_S
    have h2 : V_1.principal hY  <=  e z := by
      have := e.monotone h1
      rwa [e.apply_symm_apply] at this
    exact h2 Y <hY, subset_rfl>
  have hX_le_z : V_0.principal hX  <=  z := by
    have : e.symm w  <=  e.symm (e z) := e.symm.monotone hw_le_ez
    rwa [hsymm_w, e.symm_apply_apply] at this
  -- Hence `z = X`, so `X  in  z` lands in some member `e^-1(Y)`.
  have hz_eq : z = V_0.principal hX := le_antisymm hz_le hX_le_z
  have hXz : z.mem X := hz_eq  |>  <hX, subset_rfl>
  obtain <s, <Y, hY, hYw, rfl>, hXs> := hXz
  -- `X  sqsub  e^-1(Y)` (it contains `X`), so `w = e(X)  sqsub  Y`; with `Y  sqsub  w`
     we get `w = Y`.
  refine <Y, hY, ?_>
  have hprinX_le : V_0.principal hX  <=  e.symm (V_1.principal hY) :=
    fun Z hZ => (e.symm (V_1.principal hY)).up_mem hXs hZ.1 hZ.2
  have hw_le_prinY : w  <=  V_1.principal hY := by
    have := e.monotone hprinX_le
    rw [e.apply_symm_apply] at this
    rwa [ <-  hw] at this
  exact le_antisymm hw_le_prinY (hprin_le_w hY hYw)

/-- **Theorem 2.7 -- the relation re-defines the function.** The elementwise map of `
    ofIso e` is `e`
itself: `(ofIso e)(x) = e(x)` for every `x`. The forward inclusion uses that `X  in  x`
     implies
`X  sqsub  x`, hence `e(X)  sqsub  e(x)`; the reverse uses surjectivity of `e` (via `e.
    symm`) exactly as in
Scott's proof -- one shows `x = e^-1((ofIso e)(x))` by antisymmetry. -/
theorem toElementMap_ofIso (e : V_0.Element ?o V_1.Element) (x : V_0.Element) :
    (ofIso e).toElementMap x = e x := by
  have hgxle : (ofIso e).toElementMap x  <=  e x := by
    rintro Y <X, hXx, hX, hY>
    have hpx : V_0.principal hX  <=  x := fun Z hZ => x.up_mem hXx hZ.1 hZ.2
    exact (e.monotone hpx) Y hY
  have key : x = e.symm ((ofIso e).toElementMap x) := by
    apply le_antisymm
    * intro X hXx
      have hX : V_0.mem X := x.sub hXx
      have hsub : e (V_0.principal hX)  <=  (ofIso e).toElementMap x :=
        fun Y hY => <X, hXx, hX, hY>
      have hple : V_0.principal hX  <=  e.symm ((ofIso e).toElementMap x) := by
```

```
        have h := e.symm.monotone hsub
        rwa [e.symm_apply_apply] at h
      exact hple X <hX, subset_rfl>
    * have h := e.symm.monotone hgxle
      rwa [e.symm_apply_apply] at h
  have h1 : e x = e (e.symm ((ofIso e).toElementMap x)) := congrArg e key
  rw [e.apply_symm_apply] at h1
  exact h1.symm

/-- **Theorem 2.7 (statement) (Scott 1981, PRG-19).** "Every isomorphism between
    domains results from
an approximable mapping." For any domain isomorphism `e`, there is an approximable
    mapping whose
elementwise action is exactly `e`. -/
theorem exists_approximable_of_iso (e : V_0.Element ?o V_1.Element) :
    exists  f : ApproximableMap V_0 V_1, forall  x, f.toElementMap x = e x :=
  <ofIso e, toElementMap_ofIso e>
end

end ApproximableMap

end DomainTheory.NeighbourhoodSystems
```

3.3.9 Lecture II exercises

EXERCISES

3.3.10 Exercise 2.8 — determined by finite elements

With reference to the proof of 2.2 show that an approximable mapping is uniquely determined by its elementwise effect on finite elements. Moreover any arbitrary monotone function on finite elements of $|D_0|$ with values in $|D_1|$ comes from an approximable $f : D_0 \to D_1$.

Exercise_2_8.lean

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_2
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem

namespace ApproximableMap

section
variable {a b ? : Type*}

variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b} {V_2 :
    NeighborhoodSystem ?}

/- concordance: source L769 EXERCISE 2.8 determined by finite elements -/
/- concordance: source L767 EXERCISES -- Lecture II exercises -/
```

```
/-- **Exercise 2.8 (extension).** *Any* monotone function on finite elements comes
    from an
approximable map. Here a "monotone function on finite elements" is a map `m` sending
    each
neighbourhood `X` (a finite element `X`) to an element `m X hX : |?_1|`, monotone in
    the sense
`X'  subseteq  X  ->  m X hX  <=  m X' hX'` (i.e. `X  sqsub  X' ? m(X)  sqsub  m(X')`).
     The induced relation is
`X f Y  <->  Y  in  m(X)`. -/
def ofMono (m : (X : Set a)  ->  V_0.mem X  ->  V_1.Element)
    (hmono : forall  (X X' : Set a) (hX : V_0.mem X) (hX' : V_0.mem X'), X'  subseteq
     X  ->  m X hX  <=  m X' hX') :
    ApproximableMap V_0 V_1 where
  rel X Y := exists  hX : V_0.mem X, (m X hX).mem Y
  rel_dom := fun <hX, _> => hX
  rel_cod := fun <hX, hY> => (m _ hX).sub hY
  master_rel := <V_0.master_mem, (m _ V_0.master_mem).master_mem>
  inter_right := by
    rintro X Y Y' <hX, hY> <_, hY'>
    exact <hX, (m X hX).inter_mem hY hY'>
  mono := by
    rintro X X' Y Y' <hX, hY> hX'X hYY' hX' hY'
    have hle : m X hX  <=  m X' hX' := hmono X X' hX hX' hX'X
    exact <hX', (m X' hX').up_mem (hle Y hY) hY' hYY'>

/-- **Exercise 2.8 (uniqueness).** An approximable mapping is *uniquely determined by
    its elementwise
effect on finite elements*: if `f(X) = g(X)` for every neighbourhood `X`, then `f = g`.
     (Off `?_0`
both relations are empty; on `?_0` use `rel_iff_mem_principal`.) -/
theorem eq_of_toElementMap_principal {f g : ApproximableMap V_0 V_1}
    (h : forall  (X : Set a) (hX : V_0.mem X),
      f.toElementMap (V_0.principal hX) = g.toElementMap (V_0.principal hX)) : f = g
    := by
  apply ApproximableMap.ext
  intro X Y
  by_cases hX : V_0.mem X
  * rw [f.rel_iff_mem_principal hX, g.rel_iff_mem_principal hX, h X hX]
  * constructor
    * intro hr; exact absurd (f.rel_dom hr) hX
    * intro hr; exact absurd (g.rel_dom hr) hX

/-- **Exercise 2.8 (extension, computed).** The map `ofMono m` realizes `m` on finite
    elements:
`(ofMono m)(X) = m(X)`. -/
theorem toElementMap_ofMono_principal
    (m : (X : Set a)  ->  V_0.mem X  ->  V_1.Element)
    (hmono : forall  (X X' : Set a) (hX : V_0.mem X) (hX' : V_0.mem X'), X'  subseteq
     X  ->  m X hX  <=  m X' hX')
    (X : Set a) (hX : V_0.mem X) :
    (ofMono m hmono).toElementMap (V_0.principal hX) = m X hX := by
  apply Element.ext
  intro Y
  constructor
  * rintro <Z, <hZmem, hXZ>, hZ', hmY>
    have hle : m Z hZ'  <=  m X hX := hmono Z X hZ' hX hXZ
```

```
    exact hle Y hmY
  * intro hmY
    exact <X, <hX, subset_rfl>, hX, hmY>
end

end ApproximableMap

end DomainTheory.NeighbourhoodSystems
```

3.3.11 Exercise 2.9 — directed union formula

Prove that if $f : D_0 \to D_1$ is an approximable mapping, then the elementwise mapping $f : |D_0| \to |D_1|$ satisfies the equation

$$f(x) = \bigcup \{ f(\uparrow X) \mid X \in x \}$$

for all $x \in |D_0|$. Conversely, show that every elementwise function satisfying this equation comes from an approximable mapping as defined in 2.2.

Exercise_2_9.lean

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_2
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem

section
variable {a b ? : Type*}

namespace ApproximableMap

variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b} {V_2 :
    NeighborhoodSystem ?}

/- concordance: source L773 EXERCISE 2.9 directed union formula -/
/-- **Exercise 2.9 (Scott 1981, PRG-19).** `f(x) = ? {f(X) ? X  in  x}`: a
    neighbourhood `Y` lies in
`f(x)` iff it lies in `f(X)` for some `X  in  x`. (Immediate from `
    rel_iff_mem_principal`.) -/
theorem toElementMap_mem_iff_principal (f : ApproximableMap V_0 V_1) (x : V_0.Element)
     {Y : Set b} :
    (f.toElementMap x).mem Y  <->
      exists  (X : Set a) (hx : x.mem X), (f.toElementMap (V_0.principal (x.sub hx))).
    mem Y := by
  rw [mem_toElementMap]
  constructor
  * rintro <X, hxX, hrel>
    exact <X, hxX, (f.rel_iff_mem_principal (x.sub hxX)).mp hrel>
  * rintro <X, hxX, hmem>
    exact <X, hxX, (f.rel_iff_mem_principal (x.sub hxX)).mpr hmem>
```

```
end ApproximableMap
end

end DomainTheory.NeighbourhoodSystems
```

3.3.12 Exercise 2.10 — intersection of two maps

Carry out the proof of Proposition 2.6; and in addition show that, if $f, g : D_0 \to D_1$ are two approximable mappings, there exists $h : D_0 \to D_1$ such that

$$h(x) = f(x) \cap g(x)$$

for all $x \in |D_0|$.

Exercise_2_10.lean

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_2
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem

namespace ApproximableMap

section
variable {a b ? : Type*}

variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b} {V_2 :
    NeighborhoodSystem ?}

/-- **Exercise 2.10 (Scott 1981, PRG-19).** The pointwise **intersection** `h` of two
    approximable
maps: `X h Z  <->  X f Z  /\  X g Z`. It is approximable, and (`
    mem_toElementMap_interMap`)
`h(x) = f(x)  I  g(x)`. -/
def interMap (f g : ApproximableMap V_0 V_1) : ApproximableMap V_0 V_1 where
  rel X Z := f.rel X Z  /\  g.rel X Z
  rel_dom h := f.rel_dom h.1
  rel_cod h := f.rel_cod h.1
  master_rel := <f.master_rel, g.master_rel>
  inter_right := fun <hf, hg> <hf', hg'> => <f.inter_right hf hf', g.inter_right hg hg
    '>
  mono := fun <hf, hg> hX'X hZZ' hX' hZ' =>
    <f.mono hf hX'X hZZ' hX' hZ', g.mono hg hX'X hZZ' hX' hZ'>

/- concordance: source L781 EXERCISE 2.10 intersection of two maps -/
/-- **Exercise 2.10.** `h(x) = f(x)  I  g(x)` (the meet in `|?_1|`). The non-trivial
    direction combines
witnesses `X  in  x` (for `f`) and `X'  in  x` (for `g`) through `X  I  X'  in  x` using
     `mono`. -/
theorem mem_toElementMap_interMap (f g : ApproximableMap V_0 V_1) (x : V_0.Element) {Z
     : Set b} :
```

```
    ((interMap f g).toElementMap x).mem Z  <->
      (f.toElementMap x).mem Z  /\  (g.toElementMap x).mem Z := by
  constructor
  * rintro <X, hxX, hf, hg>
    exact <<X, hxX, hf>, <X, hxX, hg>>
  * rintro <<X, hxX, hf>, <X', hxX', hg>>
    have hxXX' : x.mem (X  I  X') := x.inter_mem hxX hxX'
    have hXX' : V_0.mem (X  I  X') := x.sub hxXX'
    refine <X  I  X', hxXX', ?_, ?_>
    * exact f.mono hf Set.inter_subset_left subset_rfl hXX' (f.rel_cod hf)
    * exact g.mono hg Set.inter_subset_right subset_rfl hXX' (g.rel_cod hg)
end

end ApproximableMap

end DomainTheory.NeighbourhoodSystems
```

3.3.13 Exercise 2.11 — directed unions of elements

Let $(I, \leq)$ be a non-empty abstract partially ordered set; suppose it is directed in the sense that whenever $i, j \in I$, then $i \leq k$ and $j \leq k$ for some $k \in I$. Suppose that $a : I \to |D|$ is such that

$$i \leq j \text{ implies } a_i \subseteq a_j$$

for all $i, j \in I$. Prove that

$$\bigcup\{a_i \mid i \in I\}$$

is always a filter in $|D|$. (Note the ways this lemma could be used in the proof of 2.7; but be careful in defining the partially ordered set and do not confuse $\subseteq$ and $\supseteq$.) In words we could say that the domain of filters is closed under directed unions. Prove also that if $f : D \to D'$ is an approximable mapping, then for any directed union

$$f(\bigcup\{a_i \mid i \in I\}) = \bigcup\{f(a_i) \mid i \in I\};$$

that is, approximable mappings always preserve directed unions. If an elementwise function preserves directed unions, must it come from an approximable mapping? (Hint: Invoke 2.9.)

Exercise_2_11.lean

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_2
public import Mathlib.Data.Set.Operations
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem

namespace NeighborhoodSystem

section
variable {a b ? : Type*}
```

```
/- concordance: source L789 EXERCISE 2.11 directed unions of elements -/
/-- **Exercise 2.11 -- directed union (indexed form).** For a directed family `a : I
    ->  |?|` (any two
`a i, a j` have a common upper bound `a k`), the union `?_i a i` is again an element of
     `|?|`. Built
on `sSupDirected` over the range. -/
def iSupDirected {a : Type*} {V : NeighborhoodSystem a} {I : Type*} [Nonempty I]
    (a : I  ->  V.Element) (hdir : forall  i j, exists  k, a i  <=  a k  /\  a j  <=
    a k) : V.Element :=
  V.sSupDirected (Set.range a) (Set.range_nonempty a) (by
    rintro _ <i, rfl> _ <j, rfl>
    obtain <k, hik, hjk> := hdir i j
    exact <a k, <k, rfl>, hik, hjk>)

theorem mem_iSupDirected {a : Type*} {V : NeighborhoodSystem a} {I : Type*} [Nonempty
    I]
    (a : I  ->  V.Element) (hdir : forall  i j, exists  k, a i  <=  a k  /\  a j  <=
    a k) {Z : Set a} :
    (iSupDirected a hdir).mem Z  <->  exists  i, (a i).mem Z := by
  constructor
  * rintro <s, <i, rfl>, hsZ>; exact <i, hsZ>
  * rintro <i, hi>; exact <a i, <i, rfl>, hi>

theorem iSupDirected_le {a : Type*} {V : NeighborhoodSystem a} {I : Type*} [Nonempty I
    ]
    (a : I  ->  V.Element) (hdir : forall  i j, exists  k, a i  <=  a k  /\  a j  <=
    a k) {y : V.Element}
    (hy : forall  i, a i  <=  y) : iSupDirected a hdir  <=  y := by
  intro Z hZ
  obtain <i, hi> := (mem_iSupDirected a hdir).mp hZ
  exact hy i Z hi

theorem le_iSupDirected {a : Type*} {V : NeighborhoodSystem a} {I : Type*} [Nonempty I
    ]
    (a : I  ->  V.Element) (hdir : forall  i j, exists  k, a i  <=  a k  /\  a j  <=
    a k) (i : I) :
    a i  <=  iSupDirected a hdir :=
  fun _ hZ => (mem_iSupDirected a hdir).mpr <i, hZ>
end

end NeighborhoodSystem

section
variable {a b ? : Type*}

namespace ApproximableMap

variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b} {V_2 :
    NeighborhoodSystem ?}

/-- **Exercise 2.11 (Scott 1981, PRG-19).** Approximable mappings *preserve directed
    unions*:
`f(?_i a i) = ?_i f(a i)`. Both sides have member `Y` iff `exists  i X, X  in  a i  /\
     X f Y`. -/
theorem toElementMap_iSupDirected (f : ApproximableMap V_0 V_1) {I : Type*} [Nonempty
```

```
    I]
    (a : I  ->  V_0.Element) (hdir : forall  i j, exists  k, a i  <=  a k  /\  a j  <=
      a k) :
    f.toElementMap (NeighborhoodSystem.iSupDirected a hdir) =
      NeighborhoodSystem.iSupDirected (fun i => f.toElementMap (a i))
        (fun i j => by
          obtain <k, hik, hjk> := hdir i j
          exact <k, f.toElementMap_mono hik, f.toElementMap_mono hjk>) := by
  apply Element.ext
  intro Y
  rw [mem_toElementMap, NeighborhoodSystem.mem_iSupDirected]
  constructor
  * rintro <X, hX, hrel>
    obtain <i, hi> := (NeighborhoodSystem.mem_iSupDirected a hdir).mp hX
    exact <i, X, hi, hrel>
  * rintro <i, X, hi, hrel>
    exact <X, (NeighborhoodSystem.mem_iSupDirected a hdir).mpr <i, hi>, hrel>

end ApproximableMap
end

end DomainTheory.NeighbourhoodSystems
```

3.3.14 Exercise 2.12 — directed unions of maps

Suppose $(I, \leq)$ is a directed, partially ordered set and $f_i : D_0 \to D_1$ is a family of approximable mappings indexed by $i \in I$, where we assume

$$i \leq j \text{ implies } f_i(x) \subseteq f_j(x)$$

for all $i, j \in I$ and all $x \in |D_0|$. Prove that there is an approximable mapping $g : D_0 \to D_1$ where

$$g(x) = \bigcup \{f_i(x) \mid i \in I\}$$

for all $x \in |D_0|$.

Exercise_2_12.lean

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_2
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem

namespace ApproximableMap

section
variable {a b ? : Type*}

variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b} {V_2 :
    NeighborhoodSystem ?}
```

```
/-- **Exercise 2.12 (Scott 1981, PRG-19).** The pointwise union of a *directed* family
     of approximable
maps is approximable. Directedness is stated on the relations: any two `f i, f j` are
    dominated by
some `f k`. The union relation is `X g Z  <->  exists  i, X (f i) Z`. -/
def iSupMap {I : Type*} [Nonempty I] (f : I  ->  ApproximableMap V_0 V_1)
    (hdir : forall  i j, exists  k, (forall  X Y, (f i).rel X Y  ->  (f k).rel X Y)
    /\
      (forall  X Y, (f j).rel X Y  ->  (f k).rel X Y)) : ApproximableMap V_0 V_1 where
  rel X Z := exists  i, (f i).rel X Z
  rel_dom := fun <i, h> => (f i).rel_dom h
  rel_cod := fun <i, h> => (f i).rel_cod h
  master_rel := by obtain <i> := (inferInstance : Nonempty I); exact <i, (f i).
    master_rel>
  inter_right := by
    rintro X Z Z' <i, hi> <j, hj>
    obtain <k, hik, hjk> := hdir i j
    exact <k, (f k).inter_right (hik X Z hi) (hjk X Z' hj)>
  mono := by
    rintro X X' Z Z' <i, hi> hX'X hZZ' hX' hZ'
    exact <i, (f i).mono hi hX'X hZZ' hX' hZ'>

/- concordance: source L811 EXERCISE 2.12 directed unions of maps -/
/-- **Exercise 2.12.** The induced elementwise map is the pointwise union: `g(x) = ?_i
    f i (x)`. -/
theorem mem_toElementMap_iSupMap {I : Type*} [Nonempty I] (f : I  ->  ApproximableMap
    V_0 V_1)
    (hdir : forall  i j, exists  k, (forall  X Y, (f i).rel X Y  ->  (f k).rel X Y)
    /\
      (forall  X Y, (f j).rel X Y  ->  (f k).rel X Y)) (x : V_0.Element) {Y : Set b} :
    ((iSupMap f hdir).toElementMap x).mem Y  <->  exists  i, ((f i).toElementMap x).
    mem Y := by
  constructor
  * rintro <X, hxX, i, hrel>
    exact <i, X, hxX, hrel>
  * rintro <i, X, hxX, hrel>
    exact <X, hxX, i, hrel>
end

end ApproximableMap

end DomainTheory.NeighbourhoodSystems
```

3.3.15 Exercise 2.13 — approximable means continuous

(For topologists.) Recall Exercise 1.22 where it was shown that any domain $|D|$ is a topological space. Prove from Exercise 2.9 that the functions $f : |D_0| \to |D_1|$ determined by approximable mappings are exactly *the continuous functions between these spaces.* (Hint: To prove continuity, remark that by 2.9

$$f^{-1}[Y] = \bigcup\{[X] \mid Y \in f(\uparrow X)\};$$

hence, the inverse image of any open set is open. In the other direction, suppose that $f : |D_0| \to |D_1|$ is topologically continuous. Argue that for all $x \in |D_0|$ and all open subsets $U \subseteq |D_1|$ we have

$$f(x) \in U \text{ iff } \exists X \in x.\ f(\uparrow X) \in U.$$

This holds because an open subset of $|D_0|$ is always a union of basic open subsets of the form $[X']$ for $X \in D_0$ and because

$$x = \bigcup \{\uparrow X \mid X \in x\}$$

for all $x \in |D_0|$.)

Exercise_2_13.lean

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Exercise_2_17b

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Exercise_2_9
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Exercise_2_8
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Exercise_2_11
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_22
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_7
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

namespace ApproximableMap

section
open NeighborhoodSystem
universe u
variable {a : Type u} {b : Type*} {V_0 : NeighborhoodSystem a} {V_1 :
    NeighborhoodSystem b}

/- concordance: source L825 EXERCISE 2.13 approximable means continuous -/
/-- **Exercise 2.13 (reverse).** The approximable mapping determined by a continuous
    function `c`:
its action on the finite element `X` is the value `c(X)`, extended to all of `?_0` via
    `ofMono`.
Monotonicity of `X  |->  c(X)` is `continuous_monotone` together with the inclusion-
    reversal
`X'  subseteq  X  <->  X  sqsub  X'`. -/
def ofContinuous (c : V_0.Element  ->  V_1.Element) (hc : Continuous c) :
    ApproximableMap V_0 V_1 :=
  ofMono (fun _X hX => c (V_0.principal hX))
    (fun _X _X' hX hX' hX'X =>
      NeighborhoodSystem.continuous_monotone hc ((V_0.principal_le_iff hX hX').mpr hX'
    X))

/-- **Exercise 2.13 (forward).** The elementwise function of an approximable mapping
    is continuous.
For an open `U` and `x` with `f(x)  in  U`, openness gives `Y` with `Y  in  f(x)` and `[
    Y]  subseteq  U`; unfolding
`Y  in  f(x)` produces `X  in  x` with `X f Y`; then `[X]  subseteq  f^-1U`, since any `
    x'  in  [X]` has `Y  in  f(x')`
```

```
(via `X f Y`). -/
theorem continuous_toElementMap (f : ApproximableMap V_0 V_1) :
    Continuous (fun x => f.toElementMap x) := by
  rw [continuous_def]
  intro U hU
  show V_0.OpenFilter _
  constructor
  intro x hx
  obtain <Y, hY, hYU> := hU.out (f.toElementMap x) hx
  obtain <X, hxX, hrel> := hY
  exact <X, hxX, fun x' hx' => hYU <X, hx', hrel>>

end

end ApproximableMap

namespace NeighborhoodSystem

section
open NeighborhoodSystem
universe u
variable {a : Type u} {b : Type*} {V_0 : NeighborhoodSystem a} {V_1 :
    NeighborhoodSystem b}

/-- **Exercise 2.13 -- Scott's union formula for a continuous map.** For continuous `c`
     and any
`x  in  |?_0|`: `Y  in  c(x)  <->  exists  X  in  x, Y  in  c(X)`.

* ` -> ` : `c^-1[[Y]]` is open and contains `x`, so it contains a basic neighbourhood
    `[X]` with `X  in  x`;
  since `X  in  [X]`, `Y  in  c(X)`.
* ` <- ` : `X  sqsub  x` and `c` monotone (`continuous_monotone`) give `c(X)  sqsub  c(x
    )`, transporting `Y`. -/
theorem mem_iff_principal_of_continuous {c : V_0.Element  ->  V_1.Element} (hc :
    Continuous c)
    (x : V_0.Element) {Y : Set b} :
    (c x).mem Y  <->  exists  (X : Set a) (hx : x.mem X), (c (V_0.principal (x.sub hx)
    )).mem Y := by
  constructor
  * intro hY
    have hxpre : x  in  c ^-1' V_1.basicOpen Y := hY
    have hopen : IsOpen (c ^-1' V_1.basicOpen Y) := (V_1.isOpen_basicOpen Y).preimage
    hc
    obtain <X, hxX, hXU> := hopen.out x hxpre
    refine <X, hxX, ?_>
    exact hXU (show V_0.principal (x.sub hxX)  in  V_0.basicOpen X from <x.sub hxX,
    subset_rfl>)
  * rintro <X, hxX, hY>
    have hple : V_0.principal (x.sub hxX)  <=  x := fun Z hZ => x.up_mem hxX hZ.1 hZ.2
    exact (continuous_monotone hc hple) Y hY

theorem principalFamily_directed (x : V_0.Element) :
    forall  i j : {X : Set a // x.mem X}, exists  k : {X : Set a // x.mem X},
      V_0.principal (x.sub i.2)  <=  V_0.principal (x.sub k.2)  /\
        V_0.principal (x.sub j.2)  <=  V_0.principal (x.sub k.2) :=
  fun i j => <<i.1  I  j.1, x.inter_mem i.2 j.2>,
```

```
    (V_0.principal_le_iff (x.sub i.2) (x.sub (x.inter_mem i.2 j.2))).mpr Set.
    inter_subset_left,
    (V_0.principal_le_iff (x.sub j.2) (x.sub (x.inter_mem i.2 j.2))).mpr Set.
    inter_subset_right>

theorem eq_iSupDirected_principal (x : V_0.Element) :
    x = iSupDirected (fun i : {X : Set a // x.mem X} => V_0.principal (x.sub i.2))
      (principalFamily_directed x) := by
  apply Element.ext
  intro Z
  rw [mem_iSupDirected]
  constructor
  * intro hZ; exact <<Z, hZ>, (V_0.mem_principal _).mpr <x.sub hZ, subset_rfl>>
  * rintro <<X, hX>, hZ'>
    obtain <hZmem, hXZ> := (V_0.mem_principal _).mp hZ'
    exact x.up_mem hX hZmem hXZ

/-- **The converse of `continuous_toElementMap`/`continuous_monotone`, in domain-
    theoretic form.**
A monotone function that also preserves directed unions is topologically continuous --
     the standard
"Scott continuity ? order-theoretic continuity" bridge, proved directly from
    algebraicity rather
than through a general topological-basis argument: given `x  in  c^-1U`, decompose `x`
    as the directed
union of its principal approximants (`eq_iSupDirected_principal`); `c` preserving the
    union puts
`c x` there too, so openness of `U` finds a witness `Y` in *some* `c(X)` (`X  in  x`);
    monotonicity of
`c` then transfers `Y  in  c(X)` up to every `x'  in  [X]`, giving `[X]  subseteq  c^-1
    U`. -/
theorem continuous_of_monotone_iSupDirected {c : V_0.Element  ->  V_1.Element} (hmono
    : Monotone c)
    (hsup : forall  {I : Type u} [Nonempty I] (d : I  ->  V_0.Element)
      (hdir : forall  i j, exists  k, d i  <=  d k  /\  d j  <=  d k)
      (hdir' : forall  i j, exists  k, c (d i)  <=  c (d k)  /\  c (d j)  <=  c (d k))
    ,
      c (iSupDirected d hdir) = iSupDirected (fun i => c (d i)) hdir') :
    Continuous c := by
  rw [continuous_def]
  intro U hU
  constructor
  intro x hx
  rw [Set.mem_preimage] at hx
  set fam : {X : Set a // x.mem X}  ->  V_0.Element := fun i => V_0.principal (x.sub i
    .2) with hfam
  have hdir : forall  i j : {X : Set a // x.mem X}, exists  k, fam i  <=  fam k  /\
    fam j  <=  fam k :=
    principalFamily_directed x
  have hdir' : forall  i j : {X : Set a // x.mem X}, exists  k, c (fam i)  <=  c (fam
    k)  /\  c (fam j)  <=  c (fam k) :=
    fun i j => by obtain <k, hik, hjk> := hdir i j; exact <k, hmono hik, hmono hjk>
  have hxeq : c x = iSupDirected (fun i => c (fam i)) hdir' :=
    (congrArg c (eq_iSupDirected_principal x)).trans
      (hsup (I := {X : Set a // x.mem X}) fam hdir hdir')
  rw [hxeq] at hx
```

```
  obtain <Y, hY, hYU> := hU.out _ hx
  obtain <i, hi> := (mem_iSupDirected _ hdir').mp hY
  refine <i.1, i.2, fun x' hx' => hYU ?_>
  have hple : fam i  <=  x' := fun Z hZ => x'.up_mem hx' hZ.1 hZ.2
  exact (hmono hple) Y hi

end

end NeighborhoodSystem

section
open NeighborhoodSystem
universe u
variable {a : Type u} {b : Type*} {V_0 : NeighborhoodSystem a} {V_1 :
    NeighborhoodSystem b}

namespace ApproximableMap

/-- **Exercise 2.13 -- the round trip.** `ofContinuous c hc` induces exactly `c`:
`(ofContinuous c hc)(x) = c(x)` for all `x`. Exercise 2.9 reduces `f(x)` to a union
    over finite
approximants `X` (`X  in  x`), where `ofMono` evaluates to `c(X)`; the union formula
`mem_iff_principal_of_continuous` then re-assembles `c(x)`. -/
theorem toElementMap_ofContinuous (c : V_0.Element  ->  V_1.Element) (hc : Continuous
    c)
    (x : V_0.Element) : (ofContinuous c hc).toElementMap x = c x := by
  apply Element.ext
  intro Y
  rw [toElementMap_mem_iff_principal, NeighborhoodSystem.
    mem_iff_principal_of_continuous hc]
  constructor
  * rintro <X, hxX, hmem>
    refine <X, hxX, ?_>
    rwa [ofContinuous, toElementMap_ofMono_principal] at hmem
  * rintro <X, hxX, hmem>
    refine <X, hxX, ?_>
    rwa [ofContinuous, toElementMap_ofMono_principal]

end ApproximableMap
end

section
set_option linter.unusedVariables false

theorem example_213.{u, u_1} : And
  (And
    (forall  {a : Type u} {b : Type u_1} {V_0 : DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem a}
      {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
      (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1),
      @Continuous (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V_0
    )
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element b V_1)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    instTopologicalSpaceElement a V_0)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
```

```
  instTopologicalSpaceElement b V_1)
      fun (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V_0)
  =>
      @DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a b V_0 V_1 f
  x)
   (forall  {a : Type u} {b : Type u_1} {V_0 : DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem a}
     {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
     {c :
       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V_0  ->
         @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element b V_1},
     @Continuous (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V_0
  )
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element b V_1)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instTopologicalSpaceElement a V_0)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instTopologicalSpaceElement b V_1) c  ->
      forall  (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
  V_0) {Y : Set b},
        Iff (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem b V_1
   (c x) Y)
          (exists  (X : Set a) (hx : @DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element.mem a V_0 x X),
            @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem b V_1
              (c
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.principal a
  V_0 X
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.sub
   a V_0 x X hx)))
              Y)))
(forall  {a : Type u} {b : Type u_1} {V_0 : DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem a}
  {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
  (c :
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V_0  ->
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element b V_1)
  (hc :
    @Continuous (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V_0
  )
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element b V_1)
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instTopologicalSpaceElement a V_0)
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instTopologicalSpaceElement b V_1) c)
  (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V_0),
  @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element b V_1)
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a b V_0 V_1
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.ofContinuous a b V_0 V_1 c
   hc) x)
    (c x)) :=
@And.intro
(And
  (forall  {a : Type u} {b : Type u_1} {V_0 : DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem a}
    {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
```

```
    (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1),
    @Continuous (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V_0
 )
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element b V_1)
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 instTopologicalSpaceElement a V_0)
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 instTopologicalSpaceElement b V_1)
      fun (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V_0)
 =>
      @DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a b V_0 V_1 f
 x)
  (forall  {a : Type u} {b : Type u_1} {V_0 : DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem a}
    {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
    {c :
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V_0  ->
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element b V_1},
    @Continuous (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V_0
 )
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element b V_1)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 instTopologicalSpaceElement a V_0)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 instTopologicalSpaceElement b V_1) c  ->
      forall  (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
 V_0) {Y : Set b},
        Iff (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem b V_1
  (c x) Y)
          (exists  (X : Set a) (hx : @DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem.Element.mem a V_0 x X),
            @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem b V_1
              (c
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.principal a
 V_0 X
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.sub
  a V_0 x X hx)))
              Y)))
(forall  {a : Type u} {b : Type u_1} {V_0 : DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem a}
 {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
 (c :
   @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V_0  ->
     @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element b V_1)
 (hc :
   @Continuous (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V_0
 )
     (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element b V_1)
     (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 instTopologicalSpaceElement a V_0)
     (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 instTopologicalSpaceElement b V_1) c)
 (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V_0),
 @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element b V_1)
   (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a b V_0 V_1
     (@DomainTheory.NeighbourhoodSystems.ApproximableMap.ofContinuous a b V_0 V_1 c
```

```
     hc) x)
       (c x))
   (@And.intro
     (forall  {a : Type u} {b : Type u_1} {V_0 : DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem a}
       {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
       (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1),
       @Continuous (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V_0
    )
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element b V_1)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    instTopologicalSpaceElement a V_0)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    instTopologicalSpaceElement b V_1)
          fun (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V_0)
    =>
          @DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a b V_0 V_1 f
    x)
     (forall  {a : Type u} {b : Type u_1} {V_0 : DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem a}
       {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
       {c :
         @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V_0  ->
           @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element b V_1},
       @Continuous (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V_0
    )
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element b V_1)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    instTopologicalSpaceElement a V_0)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    instTopologicalSpaceElement b V_1) c  ->
          forall  (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
    V_0) {Y : Set b},
          Iff (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem b V_1
     (c x) Y)
            (exists  (X : Set a) (hx : @DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.Element.mem a V_0 x X),
              @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem b V_1
                (c
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.principal a
    V_0 X
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.sub
     a V_0 x X hx)))
                Y))
     @DomainTheory.NeighbourhoodSystems.ApproximableMap.continuous_toElementMap
     @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    mem_iff_principal_of_continuous)
   @DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap_ofContinuous
end

end DomainTheory.NeighbourhoodSystems
```

3.3.16 Exercise 2.14 — isomorphisms of neighbourhoods

Let $f : |D_0| \to |D_1|$ be an isomorphism between domains. Let $\varphi : D_0 \to D_1$ be the one-one correspondence between neighbourhoods provided by Theorem 2.7 where

$$f(\uparrow X) = \uparrow \varphi(X)$$

for all $X \in D_0$. Show that the approximable mapping determined by f is just the relationship $\varphi(X) \subseteq Y$. In addition prove that if $X, X' \in D_0$ are consistent, then

$$\varphi(X \cap X') = \varphi(X) \cap \varphi(X').$$

Remark that the isomorphisms between domains correspond exactly to the isomorphisms between neighbourhood systems (in the sense of one-one inclusion preserving correspondences).

Exercise_2_14.lean

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Theorem_2_7
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_7
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

section
open NeighborhoodSystem ApproximableMap
variable {a b : Type*} {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}

/- concordance: source L845 EXERCISE 2.14 isomorphisms of neighbourhoods -/
/-- **Exercise 2.14 -- the correspondence `phi`.** For a domain isomorphism `e` and `X
     in  ?_0`, `phi(X)` is
the neighbourhood with `e(X) = phi(X)` provided by Theorem 2.7. -/
noncomputable def phi (e : V_0.Element ?o V_1.Element) {X : Set a} (hX : V_0.mem X) :
    Set b :=
  Classical.choose (exists_principal_eq_apply_principal e hX)

/-- `phi(X)  in  ?_1`. -/
theorem phi_mem (e : V_0.Element ?o V_1.Element) {X : Set a} (hX : V_0.mem X) :
    V_1.mem (phi e hX) :=
  Classical.choose (Classical.choose_spec (exists_principal_eq_apply_principal e hX))

/-- **Theorem 2.7 defining equation: `e(X) = phi(X)`.** -/
theorem phi_spec (e : V_0.Element ?o V_1.Element) {X : Set a} (hX : V_0.mem X) :
    e (V_0.principal hX) = V_1.principal (phi_mem e hX) :=
  Classical.choose_spec (Classical.choose_spec (exists_principal_eq_apply_principal e
    hX))

/-- **Exercise 2.14 -- `phi` preserves consistent intersections.** If `X, X'` are
    consistent
(`X  I  X'  in  ?_0`), then `phi(X  I  X') = phi(X)  I  phi(X')`.

`(X  I  X')` is the join of `X, X'` in `|?_0|` (the order is inclusion-reversed). `e`/`e
    .symm`
```

```
preserve this least upper bound, and the join of the consistent principals `phi(X), phi
    (X')` is
`(phi(X)  I  phi(X'))`. -/
theorem phi_inter (e : V_0.Element ?o V_1.Element) {X X' : Set a}
    (hX : V_0.mem X) (hX' : V_0.mem X') (hXX' : V_0.mem (X  I  X')) :
    phi e hXX' = phi e hX  I  phi e hX' := by
  have hpX_le : V_0.principal hX  <=  V_0.principal hXX' :=
    (V_0.principal_le_iff hX hXX').mpr Set.inter_subset_left
  have hpX'_le : V_0.principal hX'  <=  V_0.principal hXX' :=
    (V_0.principal_le_iff hX' hXX').mpr Set.inter_subset_right
  have hleast : forall  w : V_0.Element, V_0.principal hX  <=  w  ->  V_0.principal hX
    '  <=  w  ->
      V_0.principal hXX'  <=  w := by
    intro w hXw hX'w Z hZ
    obtain <hZmem, hsubZ> := hZ
    have hwX : w.mem X := hXw X <hX, subset_rfl>
    have hwX' : w.mem X' := hX'w X' <hX', subset_rfl>
    exact w.up_mem (w.inter_mem hwX hwX') hZmem hsubZ
  have hqX : e (V_0.principal hX) = V_1.principal (phi_mem e hX) := phi_spec e hX
  have hqX' : e (V_0.principal hX') = V_1.principal (phi_mem e hX') := phi_spec e hX'
  have hqXX' : e (V_0.principal hXX') = V_1.principal (phi_mem e hXX') := phi_spec e
    hXX'
  have hsub1 : phi e hXX'  subseteq  phi e hX := by
    have h : e (V_0.principal hX)  <=  e (V_0.principal hXX') := e.monotone hpX_le
    rw [hqX, hqXX'] at h
    exact (V_1.principal_le_iff (phi_mem e hX) (phi_mem e hXX')).mp h
  have hsub1' : phi e hXX'  subseteq  phi e hX' := by
    have h : e (V_0.principal hX')  <=  e (V_0.principal hXX') := e.monotone hpX'_le
    rw [hqX', hqXX'] at h
    exact (V_1.principal_le_iff (phi_mem e hX') (phi_mem e hXX')).mp h
  have hsubInter : phi e hXX'  subseteq  phi e hX  I  phi e hX' := Set.subset_inter
    hsub1 hsub1'
  have hinter : V_1.mem (phi e hX  I  phi e hX') :=
    V_1.inter_mem (phi_mem e hX) (phi_mem e hX') (phi_mem e hXX') hsubInter
  have hsub2 : phi e hX  I  phi e hX'  subseteq  phi e hXX' := by
    have hqXleI : e (V_0.principal hX)  <=  V_1.principal hinter := by
      rw [hqX]; exact (V_1.principal_le_iff (phi_mem e hX) hinter).mpr Set.
    inter_subset_left
    have hqX'leI : e (V_0.principal hX')  <=  V_1.principal hinter := by
      rw [hqX']; exact (V_1.principal_le_iff (phi_mem e hX') hinter).mpr Set.
    inter_subset_right
    have hpXleI : V_0.principal hX  <=  e.symm (V_1.principal hinter) := by
      have h := e.symm.monotone hqXleI; rwa [e.symm_apply_apply] at h
    have hpX'leI : V_0.principal hX'  <=  e.symm (V_1.principal hinter) := by
      have h := e.symm.monotone hqX'leI; rwa [e.symm_apply_apply] at h
    have hpXX'leI : V_0.principal hXX'  <=  e.symm (V_1.principal hinter) :=
      hleast _ hpXleI hpX'leI
    have h : e (V_0.principal hXX')  <=  V_1.principal hinter := by
      have h := e.monotone hpXX'leI; rwa [e.apply_symm_apply] at h
    rw [hqXX'] at h
    exact (V_1.principal_le_iff (phi_mem e hXX') hinter).mp h
  exact Set.Subset.antisymm hsubInter hsub2

/-- **Exercise 2.14 -- the approximable mapping of `e` is `phi(X)  subseteq  Y`.** For
    `X  in  ?_0`,
`(ofIso e).rel X Y  <->  Y  in  ?_1  /\  phi(X)  subseteq  Y`. (Immediate from `
```

```
    phi_spec` and `mem_principal`.) -/
theorem rel_ofIso_iff (e : V_0.Element ?o V_1.Element) {X : Set a} (hX : V_0.mem X) {Y
     : Set b} :
    (ofIso e).rel X Y  <->  V_1.mem Y  /\  phi e hX  subseteq  Y := by
  constructor
  * rintro <_, hmem>
    have h : (e (V_0.principal hX)).mem Y := hmem
    rw [phi_spec e hX] at h
    exact h
  * intro h
    refine <hX, ?_>
    rw [phi_spec e hX]
    exact h

end

section
set_option linter.unusedVariables false

theorem example_214.{u_1, u_2} : And
  (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem a}
    {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
    (e :
      @OrderIso (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V_0)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element b V_1)
        (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
    a V_0)
          (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.Element a V_0)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    instPartialOrderElement a V_0)))
        (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
    b V_1)
          (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.Element b V_1)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    instPartialOrderElement b V_1))))
    {X X' : Set a} (hX : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a
    V_0 X)
    (hX' : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V_0 X')
    (hXX' :
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V_0 (@Inter.inter (
    Set a) (@Set.instInter a) X X')),
    @Eq (Set b) (@DomainTheory.NeighbourhoodSystems.phi a b V_0 V_1 e (@Inter.inter (
    Set a) (@Set.instInter a) X X') hXX')
      (@Inter.inter (Set b) (@Set.instInter b) (@DomainTheory.NeighbourhoodSystems.phi
    a b V_0 V_1 e X hX)
        (@DomainTheory.NeighbourhoodSystems.phi a b V_0 V_1 e X' hX')))
  (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem a}
    {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
    (e :
      @OrderIso (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V_0)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element b V_1)
        (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
```

```
   a V_0)
         (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V_0)
           (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement a V_0)))
       (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
   b V_1)
         (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element b V_1)
           (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement b V_1))))
  {X : Set a} (hX : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V_0
  X) {Y : Set b},
  Iff
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel a b V_0 V_1
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.ofIso a b V_0 V_1 e) X Y)
    (And (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b V_1 Y)
      (@LE.le (Set b) (@Set.instLE b) (@DomainTheory.NeighbourhoodSystems.phi a b
  V_0 V_1 e X hX) Y))) :=
@And.intro
(forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem a}
  {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
  (e :
    @OrderIso (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V_0)
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element b V_1)
      (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
   a V_0)
         (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V_0)
           (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement a V_0)))
       (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
   b V_1)
         (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element b V_1)
           (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement b V_1))))
  {X X' : Set a} (hX : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a
  V_0 X)
  (hX' : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V_0 X')
  (hXX' :
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V_0 (@Inter.inter (
  Set a) (@Set.instInter a) X X')),
  @Eq (Set b) (@DomainTheory.NeighbourhoodSystems.phi a b V_0 V_1 e (@Inter.inter (
  Set a) (@Set.instInter a) X X') hXX')
    (@Inter.inter (Set b) (@Set.instInter b) (@DomainTheory.NeighbourhoodSystems.phi
   a b V_0 V_1 e X hX)
      (@DomainTheory.NeighbourhoodSystems.phi a b V_0 V_1 e X' hX')))
(forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem a}
  {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
  (e :
    @OrderIso (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V_0)
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element b V_1)
      (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
```

```
     a V_0)
           (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.Element a V_0)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    instPartialOrderElement a V_0)))
        (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
     b V_1)
           (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.Element b V_1)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    instPartialOrderElement b V_1))))
    {X : Set a} (hX : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V_0
    X) {Y : Set b},
    Iff
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel a b V_0 V_1
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.ofIso a b V_0 V_1 e) X Y)
      (And (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b V_1 Y)
        (@LE.le (Set b) (@Set.instLE b) (@DomainTheory.NeighbourhoodSystems.phi a b
    V_0 V_1 e X hX) Y)))
  @DomainTheory.NeighbourhoodSystems.phi_inter @DomainTheory.NeighbourhoodSystems.
    rel_ofIso_iff
end

end DomainTheory.NeighbourhoodSystems
```

3.3.17 Exercise 2.15 — opens classified by maps

(For topologists.) Consider the one-token system with

$$\mathcal{O} = \{\{0\}, \emptyset\}.$$

We can regard $|\mathcal{O}|$ as having just two finite elements $\bot$ (bottom) and $\top$ (top), where $\bot \sqsubseteq \top$. For any system D, show that the open subsets U of $|D|$ are in a one-one correspondence with the approximable mappings $f : D \to \mathcal{O}$, where the correspondence is given by the equation

$$U = \{x \in |D| \mid f(x) = \top\}.$$

What are the open subsets of $|\mathcal{O}|$? of $|\mathbb{T}|$? of $|\mathbb{B}|$?

Exercise_2_15.lean

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Exercise_2_13
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Exercise_2_9
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_2
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_22
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_20
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_8
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_7
```

```
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.bot
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_4
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.Exercise215
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem

section
/-- Membership for `?`: a neighbourhood is the master `Delta = univ` or the empty set.
    -/
def memO (X : Set (Fin 1)) : Prop := X = Set.univ  \/  X = {}
end

section
theorem nestedOrDisjoint_O : NestedOrDisjoint memO := by
  rintro X Y (rfl | rfl) (rfl | rfl)
  * exact Or.inl subset_rfl
  * exact Or.inr (Or.inl (Set.empty_subset _))
  * exact Or.inl (Set.empty_subset _)
  * exact Or.inl subset_rfl
end

section
theorem subMaster_O : forall  {X : Set (Fin 1)}, memO X  ->  X  subseteq  Set.univ :=
    fun _ => Set.subset_univ _
end

section
/- concordance: source L861 EXERCISE 2.15 opens classified by maps -/
/-- **Exercise 2.15 -- the one-token system `? = {Delta, {}}`.** -/
def O : NeighborhoodSystem (Fin 1) :=
  NeighborhoodSystem.ofNestedOrDisjoint memO Set.univ Set.univ_nonempty
    (Or.inl rfl) nestedOrDisjoint_O subMaster_O
end

section
@[simp] theorem O_mem {X : Set (Fin 1)} : O.mem X  <->  memO X := Iff.rfl
end

section
@[simp] theorem O_master : O.master = (Set.univ : Set (Fin 1)) := rfl
end

section
theorem O_mem_empty : O.mem ({} : Set (Fin 1)) := Or.inr rfl
end

section
/-- `?` is closed under intersection (used to verify the relation of `openToMap`). -/
theorem O_inter_mem {Y Y' : Set (Fin 1)} (hY : O.mem Y) (hY' : O.mem Y') : O.mem (Y  I
     Y') := by
  rcases hY with rfl | rfl
  * rw [Set.univ_inter]; exact hY'
  * rw [Set.empty_inter]; exact O_mem_empty
end
```

```
section
/-- The bottom element `False = Delta = {Delta}` of `|?|`. -/
def botElt : O.Element := O.bot
end

section
/-- `Y  in  False` iff `Y = Delta = univ`. -/
theorem mem_botElt {Y : Set (Fin 1)} : botElt.mem Y  <->  Y = (Set.univ : Set (Fin 1))
     := by
  rw [botElt, O.mem_bot, O_master]
end

section
/-- The top element ` top  = {}` of `|?|`. -/
def topElt : O.Element := O.principal O_mem_empty
end

section
variable {? : Type*} (V : NeighborhoodSystem ?)

/-- **Exercise 2.15 -- the open set of an approximable map.** `mapToOpen f = {x ? f(x)
    =  top }`. -/
def mapToOpen (f : ApproximableMap V O) : Set V.Element := {x | f.toElementMap x =
    topElt}
end

section
/-- **The defining property of ` top `.** An element `z` of `|?|` is ` top ` iff it
    contains `{}`: `{}` is
the sharpest neighbourhood, so `{}  in  z` forces `z` to be the principal filter `{} =
     top `. -/
theorem mem_empty_iff_eq_top (z : O.Element) : z.mem {}  <->  z = topElt := by
  constructor
  * intro hz
    exact O.eq_principal_of_isMin z hz (fun Y _ => Set.empty_subset Y)
  * rintro rfl
    exact <O_mem_empty, subset_rfl>
end

section
/-- **Two elements.** Every element of `|?|` is `False` or ` top `. -/
theorem element_eq_bot_or_top (z : O.Element) : z = botElt  \/  z = topElt := by
  by_cases hz : z.mem {}
  * exact Or.inr ((mem_empty_iff_eq_top z).mp hz)
  * refine Or.inl ?_
    apply Element.ext
    intro Y
    constructor
    * intro hY
      rcases (O_mem.mp (z.sub hY)) with rfl | rfl
      * exact mem_botElt.mpr rfl
      * exact absurd hY hz
    * intro hY
      rw [mem_botElt.mp hY,  <-  O_master]
      exact z.master_mem
```

```
end

section
/-- **Extensionality for `|?|`.** Two elements agreeing on membership of `{}` are equal
     (it
distinguishes `False` from ` top `). -/
theorem O_ext {z w : O.Element} (h : z.mem {}  <->  w.mem {}) : z = w := by
  by_cases hz : z.mem {}
  * rw [(mem_empty_iff_eq_top z).mp hz, (mem_empty_iff_eq_top w).mp (h.mp hz)]
  * have ez : z = botElt :=
      (element_eq_bot_or_top z).resolve_right (fun c => hz ((mem_empty_iff_eq_top z).
    mpr c))
    have ew : w = botElt :=
      (element_eq_bot_or_top w).resolve_right
        (fun c => (fun c' => hz (h.mpr c')) ((mem_empty_iff_eq_top w).mpr c))
    rw [ez, ew]
end

section
variable {? : Type*} (V : NeighborhoodSystem ?)

/-- `mapToOpen f` is open: it is the preimage of the open point `{ top } = [{}]` under
    the continuous
`x  |->  f(x)` (Exercise 2.13). -/
theorem isOpen_mapToOpen (f : ApproximableMap V O) : IsOpen (mapToOpen V f) := by
  have heq : mapToOpen V f = (fun x => f.toElementMap x) ^-1' O.basicOpen {} := by
    ext x
    simp only [mapToOpen, Set.mem_ofPred_eq, Set.mem_preimage, mem_basicOpen]
    exact (mem_empty_iff_eq_top (f.toElementMap x)).symm
  rw [heq]
  exact (O.isOpen_basicOpen {}).preimage f.continuous_toElementMap
end

section
theorem univ_ne_empty : (Set.univ : Set (Fin 1))  !=  {} := by
  intro h
  have h0 : (0 : Fin 1)  in  ({} : Set (Fin 1)) := h  |>  Set.mem_univ 0
  simp at h0
end

section
variable {? : Type*} (V : NeighborhoodSystem ?)

/-- **Exercise 2.15 -- the approximable map of an open set.** For open `U  subseteq
    |?|`, the relation
`X f Y  <->  X  in  ?  /\  Y  in  ?  /\  (Y = {}  ->  X  in  U)`. The single nontrivial
     obligation is monotonicity:
sharpening the input `X'  subseteq  X` means `X  sqsub  X'`, and `U` open (an upper set
    ) carries `X  in  U` up to
`X'  in  U`. Choice-free. -/
def openToMap (U : Set V.Element) (hU : IsOpen U) : ApproximableMap V O where
  rel X Y := V.mem X  /\  O.mem Y  /\  (Y = {}  ->  exists  hX : V.mem X, V.principal
    hX  in  U)
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := <V.master_mem, O.master_mem, fun heq => (univ_ne_empty (O_master  |>
```

```
    heq)).elim>
  inter_right := by
    rintro X Y Y' <hX, hY, fY> <_, hY', fY'>
    refine <hX, O_inter_mem hY hY', ?_>
    intro h
    rcases O_mem.mp hY with rfl | rfl
    * rw [Set.univ_inter] at h; exact fY' h
    * exact fY rfl
  mono := by
    rintro X X' Y Y' <hX, _, fY> hX'X hYY' hX' hY'
    refine <hX', hY', ?_>
    intro hY'eq
    have hYeq : Y = {} := Set.subset_empty_iff.mp (hY'eq  |>  hYY')
    obtain <hX0, hmem> := fY hYeq
    have hmem' : V.principal hX  in  U := hmem
    have hle : V.principal hX  <=  V.principal hX' := (V.principal_le_iff hX hX').mpr
    hX'X
    exact <hX', V.isOpen_isUpperSet hU hmem' hle>

/-- The value of `openToMap U hU` on the finite element `X` contains `{}` iff `X  in  U
    `; i.e.
`(openToMap U hU)(X) =  top   <->  X  in  U`. -/
theorem toElementMap_openToMap_principal_mem_empty (U : Set V.Element) (hU : IsOpen U)
    {X : Set ?} (hX : V.mem X) :
    ((openToMap V U hU).toElementMap (V.principal hX)).mem {}  <->  V.principal hX  in
      U := by
  rw [ <-  (openToMap V U hU).rel_iff_mem_principal hX]
  constructor
  * rintro <_, _, fimpl>
    obtain <_, hmem> := fimpl rfl
    exact hmem
  * intro h
    exact <hX, O_mem_empty, fun _ => <hX, h>>

/-- **Exercise 2.15 -- round trip (open  ->  map  ->  open).** `mapToOpen (openToMap U
    hU) = U`. -/
theorem mem_mapToOpen_openToMap (U : Set V.Element) (hU : IsOpen U) (x : V.Element) :
    x  in  mapToOpen V (openToMap V U hU)  <->  x  in  U := by
  rw [mapToOpen, Set.mem_ofPred_eq,  <-  mem_empty_iff_eq_top,
    ApproximableMap.toElementMap_mem_iff_principal]
  constructor
  * rintro <X, hxX, hmem>
    rw [toElementMap_openToMap_principal_mem_empty] at hmem
    have hple : V.principal (x.sub hxX)  <=  x := fun Z hZ => x.up_mem hxX hZ.1 hZ.2
    exact V.isOpen_isUpperSet hU hmem hple
  * intro hxU
    obtain <X, hxX, hXU> := hU.out x hxU
    refine <X, hxX, ?_>
    rw [toElementMap_openToMap_principal_mem_empty]
    exact hXU (show V.principal (x.sub hxX)  in  V.basicOpen X from <x.sub hxX,
    subset_rfl>)

/-- **Exercise 2.15 -- round trip (map  ->  open  ->  map).** `openToMap (mapToOpen f)
    = f`. -/
theorem openToMap_mapToOpen (f : ApproximableMap V O) :
    openToMap V (mapToOpen V f) (isOpen_mapToOpen V f) = f := by
```

```
  apply ApproximableMap.eq_of_toElementMap_principal
  intro X hX
  apply O_ext
  rw [toElementMap_openToMap_principal_mem_empty, mapToOpen, Set.mem_ofPred_eq,
     <-  mem_empty_iff_eq_top]
end

section
set_option linter.unusedVariables false

theorem example_215.{u_1} : And
  (forall  {? : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem ?)
    (U : Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element ? V))
    (hU :
      @IsOpen (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element ? V)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    instTopologicalSpaceElement ? V) U)
    (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element ? V),
    Iff
      (@Membership.mem (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
    ? V)
        (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element ? V))
        (@Set.instMembership (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    Element ? V))
        (@DomainTheory.NeighbourhoodSystems.Exercise215.mapToOpen ? V
          (@DomainTheory.NeighbourhoodSystems.Exercise215.openToMap ? V U hU))
        x)
      (@Membership.mem (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
    ? V)
        (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element ? V))
        (@Set.instMembership (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    Element ? V)) U x))
  (forall  {? : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem ?)
    (f :
      @DomainTheory.NeighbourhoodSystems.ApproximableMap ?
        (Fin (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))) V
        DomainTheory.NeighbourhoodSystems.Exercise215.O),
    @Eq
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap ?
        (Fin (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))) V
        DomainTheory.NeighbourhoodSystems.Exercise215.O)
      (@DomainTheory.NeighbourhoodSystems.Exercise215.openToMap ? V
        (@DomainTheory.NeighbourhoodSystems.Exercise215.mapToOpen ? V f)
        (@DomainTheory.NeighbourhoodSystems.Exercise215.isOpen_mapToOpen ? V f))
      f) :=
  @And.intro
  (forall  {? : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem ?)
    (U : Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element ? V))
    (hU :
      @IsOpen (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element ? V)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    instTopologicalSpaceElement ? V) U)
    (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element ? V),
    Iff
      (@Membership.mem (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
    ? V)
```

```
        (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element ? V))
        (@Set.instMembership (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    Element ? V))
        (@DomainTheory.NeighbourhoodSystems.Exercise215.mapToOpen ? V
          (@DomainTheory.NeighbourhoodSystems.Exercise215.openToMap ? V U hU))
        x)
      (@Membership.mem (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
    ? V)
        (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element ? V))
        (@Set.instMembership (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    Element ? V)) U x))
  (forall  {? : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem ?)
    (f :
      @DomainTheory.NeighbourhoodSystems.ApproximableMap ?
        (Fin (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))) V
        DomainTheory.NeighbourhoodSystems.Exercise215.O),
    @Eq
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap ?
        (Fin (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))) V
        DomainTheory.NeighbourhoodSystems.Exercise215.O)
      (@DomainTheory.NeighbourhoodSystems.Exercise215.openToMap ? V
        (@DomainTheory.NeighbourhoodSystems.Exercise215.mapToOpen ? V f)
        (@DomainTheory.NeighbourhoodSystems.Exercise215.isOpen_mapToOpen ? V f))
      f)
  @DomainTheory.NeighbourhoodSystems.Exercise215.mem_mapToOpen_openToMap
  @DomainTheory.NeighbourhoodSystems.Exercise215.openToMap_mapToOpen
end

section
variable {? : Type*} (V : NeighborhoodSystem ?)

/-- **Exercise 2.15 (main).** The open subsets of `|?|` are in one-one correspondence
    with the
approximable mappings `?  ->  ?`, via `U  |->  openToMap U` and `f  |->  {x ? f(x) =
    top }`. -/
def openSet_equiv_map : {U : Set V.Element // IsOpen U} ? ApproximableMap V O where
  toFun U := openToMap V U.1 U.2
  invFun f := <mapToOpen V f, isOpen_mapToOpen V f>
  left_inv := by
    rintro <U, hU>
    apply Subtype.ext
    ext x
    exact mem_mapToOpen_openToMap V U hU x
  right_inv f := openToMap_mapToOpen V f
end

section
theorem O_mem_univ : O.mem (Set.univ : Set (Fin 1)) := Or.inl rfl
end

section
/-- The open point `{ top }` is the basic open `[{}]`. -/
theorem basicOpen_empty_eq_singleton_top : O.basicOpen {} = {topElt} := by
  ext x
  rw [mem_basicOpen, mem_empty_iff_eq_top]
  exact Set.mem_singleton_iff.symm
```

```
end

section
/-- `False  sqsub   top `. -/
theorem botElt_le_topElt : botElt  <=  topElt := O.bot_le _
end

section
/-- `False  !=   top ` (so `False ?  top `). -/
theorem botElt_ne_topElt : botElt  !=  topElt := by
  intro h
  have htop : topElt.mem {} := <O_mem_empty, subset_rfl>
  rw [ <-  h, mem_botElt] at htop
  exact univ_ne_empty htop.symm
end

section
/-- **The opens of `|?|`.** `{False}` is not open: `False` is the generic point. (The
    basic neighbourhood of
`False` is `[Delta] = |?|`, which is not contained in `{False}` because it also
    contains ` top `.) -/
theorem not_isOpen_singleton_botElt : not  IsOpen ({botElt} : Set O.Element) := by
  intro h
  obtain <X, hX, hsub> := h.out botElt rfl
  rw [mem_botElt] at hX
  subst hX
  have htop : topElt  in  ({botElt} : Set O.Element) :=
    hsub (show topElt  in  O.basicOpen Set.univ from (O_master  |>  topElt.master_mem)
    )
  exact botElt_ne_topElt (Set.mem_singleton_iff.mp htop).symm
end

end DomainTheory.NeighbourhoodSystems.Exercise215
```

3.3.18 Exercise 2.16 — prefixing is approximable

In the discussion of $\mathbb{B}$ in Chapter 1 we defined a mapping $x \mapsto \sigma x$ for any given $\sigma \in \Sigma^*$. Is this (element-wise) mapping approximable? Show in addition that the mapping $f : \mathbb{B} \to \mathbb{T}$ of 2.3 is uniquely determined among approximable mappings by the equations:

$$f(1x) = \text{true},$$
$$f(01x) = \text{false, and}$$
$$f(00x) = f(x).$$

Exercise_2_16.lean

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Example_2_3
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_8
@[expose] public section
```

```
namespace DomainTheory.NeighbourhoodSystems.Exercise216
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ExampleB

section
/-- **The three equations of Exercise 2.16.** `f(1x) = true`, `f(01x) = false`, `f(00x)
     = f(x)`,
for *every* `x : |B|`. -/
structure SatisfiesParityEquations (g : ApproximableMap B Example23.T) : Prop where
  one : forall  x : B.Element, g.toElementMap (sigmaElt [true] x) = Example23.trueElt
  zeroOne : forall  x : B.Element, g.toElementMap (sigmaElt [false, true] x) =
    Example23.falseElt
  zeroZero : forall  x : B.Element, g.toElementMap (sigmaElt [false, false] x) = g.
    toElementMap x
end

section
/-- **Exercise 2.16 -- `x  |->  sigmax` is approximable.** The neighbourhood relation `
    X f Y  <->  sigmaX  subseteq  Y`
(confined to `B  x  B`). Definition 2.1: (i) `sigmaDelta  subseteq  Delta`; (ii) the
    prefixed cone `sigmaX` is a common lower
bound of `Y, Y'`, witnessing `Y  I  Y'  in  B`; (iii) `prepend_mono` shrinks `sigmaX'`
    as `X'  subseteq  X`. -/
def sigmaMap (sigma : Str) : ApproximableMap B B where
  rel X Y := B.mem X  /\  B.mem Y  /\  prepend sigma X  subseteq  Y
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := <B.master_mem, B.master_mem, Set.subset_univ _>
  inter_right := by
    rintro X Y Y' <hX, hY, hsub> <_, hY', hsub'>
    have hpre : B.mem (prepend sigma X) := memB_prepend sigma hX
    have hsubInter : prepend sigma X  subseteq  Y  I  Y' := Set.subset_inter hsub hsub
    '
    exact <hX, B.inter_mem hY hY' hpre hsubInter, hsubInter>
  mono := by
    rintro X X' Y Y' <_, _, hsub> hX'X hYY' hX' hY'
    exact <hX', hY', ((prepend_mono sigma hX'X).trans hsub).trans hYY'>
end

section
/-- **The recursive core of uniqueness.** Any `g` satisfying the three equations agrees
     with
`parityMap` on every principal (finite) element `sigmaFalse`. Recursion peels *two*
    tokens at a time
(mirroring `scan`'s own recursion `scan(00tau)=scantau`): `1tau` and `01tau` are pinned
     directly by
`hg.one`/`hg.zeroOne` for *any* tail (no recursion needed there); `00tau` reduces to `
    tau` by
`hg.zeroZero`, decreasing structurally. The base case `sigma = []` and the leftover
    singleton
`sigma = [false]` (an odd trailing zero, never reached by the two-at-a-time step) are
    the two places
genuine order theory enters: `False  <=  1False` and `False  <=  01False` force `g(
    False)  <=  true` and `g(False)  <=  false`, and
`False` is the *only* element below both (`eq_botElt_of_le`, `?`'s flatness); the
    singleton then
```

```
follows by squeezing `g(0False)` between `False` and `g(00False) = g(False)` via
    monotonicity. -/
theorem key (g : ApproximableMap B Example23.T) (hg : SatisfiesParityEquations g) :
    forall  sigma : Str, g.toElementMap (sigmaElt sigma B.bot) = Example23.valElt (
    Example23.scan sigma)
  | [] => by
      rw [sigmaElt_nil]
      have h1 : g.toElementMap B.bot  <=  Example23.trueElt := by
        have hm := g.toElementMap_mono (B.bot_le (sigmaElt [true] B.bot))
        rwa [hg.one] at hm
      have h2 : g.toElementMap B.bot  <=  Example23.falseElt := by
        have hm := g.toElementMap_mono (B.bot_le (sigmaElt [false, true] B.bot))
        rwa [hg.zeroOne] at hm
      exact Example23.eq_botElt_of_le h1 h2
  | [false] => by
      have hbase : g.toElementMap B.bot = Example23.botElt := by
        have h0 := key g hg []
        rwa [sigmaElt_nil] at h0
      have hb : sigmaElt [false] B.bot  <=  sigmaElt [false, false] B.bot := by
        rw [sigmaElt_bot, sigmaElt_bot]
        exact (sigmaBot_le_iff _ _).mpr <[false], rfl>
      have hm : g.toElementMap (sigmaElt [false] B.bot)  <=  Example23.botElt := by
        have := g.toElementMap_mono hb
        rwa [hg.zeroZero, hbase] at this
      have hm2 : Example23.botElt  <=  g.toElementMap (sigmaElt [false] B.bot) :=
        Example23.botElt_le _
      exact (le_antisymm hm hm2).trans rfl
  | (true :: t) => by
      rw [show (true :: t : Str) = [true] ++ t from rfl, sigmaElt_append, hg.one,
        Example23.scan_append_true]
      rfl
  | (false :: true :: t) => by
      rw [show (false :: true :: t : Str) = [false, true] ++ t from rfl,
    sigmaElt_append,
        hg.zeroOne, Example23.scan_append_falseTrue]
      rfl
  | (false :: false :: t) => by
      rw [show (false :: false :: t : Str) = [false, false] ++ t from rfl,
    sigmaElt_append,
        hg.zeroZero, key g hg t, Example23.scan_append_falseFalse]
end

section
/-- **Exercise 2.16, second half.** `parityMap` is the *unique* approximable map `B  ->
     T`
satisfying `f(1x)=true`, `f(01x)=false`, `f(00x)=f(x)`. Every neighbourhood of `B` is a
     cone
(`memB_cone`), so `ApproximableMap.ext` reduces `g = parityMap` to matching relations
    on cones,
which is exactly `key` read through `rel_iff_mem_principal`/`parityMap_rel_cone`. -/
theorem eq_parityMap_of_satisfies {g : ApproximableMap B Example23.T}
    (hg : SatisfiesParityEquations g) : g = Example23.parityMap := by
  apply ApproximableMap.ext
  intro X Y
  by_cases hX : B.mem X
  * obtain <sigma, rfl> := hX
```

```
    rw [g.rel_iff_mem_principal (memB_cone sigma)]
    show (g.toElementMap (sigmaBot sigma)).mem Y  <->  Example23.parityMap.rel (cone
    sigma) Y
    rw [ <-  sigmaElt_bot, key g hg sigma, Example23.parityMap_rel_cone]
  * exact <fun hr => absurd (g.rel_dom hr) hX,
      fun hr => absurd (Example23.parityMap.rel_dom hr) hX>
end

section
/-- **`parityMap` satisfies its own defining equations.** Each is a direct instance of
    the
"shift formula" `Example23.parityMap_toElementMap_sigmaElt`, using that a leading `1` (
    resp. `01`,
`00`) fixes the scan of the concatenation `sigma ++ tau` regardless of the tail `tau`
(`scan_append_true`/`scan_append_falseTrue`/`scan_append_falseFalse`). -/
theorem parityMap_satisfies : SatisfiesParityEquations Example23.parityMap where
  one x := by
    apply Element.ext
    intro Y
    rw [Example23.parityMap_toElementMap_sigmaElt]
    constructor
    * rintro <tau, _, hval>
      rwa [Example23.scan_append_true] at hval
    * intro hval
      exact <[], by rw [cone_nil]; exact x.master_mem, by rwa [Example23.
    scan_append_true]>
  zeroOne x := by
    apply Element.ext
    intro Y
    rw [Example23.parityMap_toElementMap_sigmaElt]
    constructor
    * rintro <tau, _, hval>
      rwa [Example23.scan_append_falseTrue] at hval
    * intro hval
      exact <[], by rw [cone_nil]; exact x.master_mem, by rwa [Example23.
    scan_append_falseTrue]>
  zeroZero x := by
    apply Element.ext
    intro Y
    rw [Example23.parityMap_toElementMap_sigmaElt, Example23.
    parityMap_toElementMap_mem]
    constructor
    * rintro <tau, htau, hval>
      exact <tau, htau, by rwa [Example23.scan_append_falseFalse] at hval>
    * rintro <tau, htau, hval>
      exact <tau, htau, by rwa [Example23.scan_append_falseFalse]>
end

section
/- concordance: source L871 EXERCISE 2.16 prefixing is approximable -/
/-- **Exercise 2.16.** The elementwise action of `sigmaMap sigma` is Scott's `sigmax`:
    `(sigmaMap sigma)(x) =
sigmax` for every `x  in  |B|`. -/
theorem toElementMap_sigmaMap (sigma : Str) (x : B.Element) :
    (sigmaMap sigma).toElementMap x = sigmaElt sigma x := by
  apply Element.ext
```

```
  intro Y
  constructor
  * rintro <X, hxX, _, hY, hsub>
    exact <hY, X, hxX, hsub>
  * rintro <hY, X, hxX, hsub>
    exact <X, hxX, x.sub hxX, hY, hsub>
end

section
set_option linter.unusedVariables false

theorem example_216 : And
  (forall
    {g :
      @DomainTheory.NeighbourhoodSystems.ApproximableMap DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
        DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
    NeighbourhoodSystems.ExampleB.B
        DomainTheory.NeighbourhoodSystems.Example23.T},
    DomainTheory.NeighbourhoodSystems.Exercise216.SatisfiesParityEquations g  ->
      @Eq
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
          DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
    NeighbourhoodSystems.ExampleB.B
          DomainTheory.NeighbourhoodSystems.Example23.T)
        g DomainTheory.NeighbourhoodSystems.Example23.parityMap)
  (forall  (sigma : DomainTheory.NeighbourhoodSystems.ExampleB.Str)
    (x :
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
        DomainTheory.NeighbourhoodSystems.ExampleB.B),
    @Eq
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
        DomainTheory.NeighbourhoodSystems.ExampleB.B)
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
        DomainTheory.NeighbourhoodSystems.ExampleB.Str DomainTheory.
    NeighbourhoodSystems.ExampleB.B
        DomainTheory.NeighbourhoodSystems.ExampleB.B (DomainTheory.
    NeighbourhoodSystems.Exercise216.sigmaMap sigma) x)
      (DomainTheory.NeighbourhoodSystems.ExampleB.sigmaElt sigma x)) :=
  @And.intro
  (forall
    {g :
      @DomainTheory.NeighbourhoodSystems.ApproximableMap DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
        DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
    NeighbourhoodSystems.ExampleB.B
        DomainTheory.NeighbourhoodSystems.Example23.T},
    DomainTheory.NeighbourhoodSystems.Exercise216.SatisfiesParityEquations g  ->
      @Eq
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
          DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
```

```
    NeighbourhoodSystems.ExampleB.B
          DomainTheory.NeighbourhoodSystems.Example23.T)
        g DomainTheory.NeighbourhoodSystems.Example23.parityMap)
  (forall  (sigma : DomainTheory.NeighbourhoodSystems.ExampleB.Str)
    (x :
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
        DomainTheory.NeighbourhoodSystems.ExampleB.B),
    @Eq
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
        DomainTheory.NeighbourhoodSystems.ExampleB.B)
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
        DomainTheory.NeighbourhoodSystems.ExampleB.Str DomainTheory.
    NeighbourhoodSystems.ExampleB.B
        DomainTheory.NeighbourhoodSystems.ExampleB.B (DomainTheory.
    NeighbourhoodSystems.Exercise216.sigmaMap sigma) x)
      (DomainTheory.NeighbourhoodSystems.ExampleB.sigmaElt sigma x))
  (@DomainTheory.NeighbourhoodSystems.Exercise216.eq_parityMap_of_satisfies)
  DomainTheory.NeighbourhoodSystems.Exercise216.toElementMap_sigmaMap
end

end DomainTheory.NeighbourhoodSystems.Exercise216
```

3.3.19 Exercise 2.17 — equations for erase-ones

Establish in detail that the mapping $g : \mathbb{B} \to \mathbb{B}$ of Exercise 2.4 is approximable. Is it uniquely determined by these equations:

$$g(0x) = 0g(x),$$
$$g(11x) = g(1x),$$
$$g(10x) = 0x,$$
$$g(1) = \bot,$$

or are some missing?

Exercise_2_17.lean

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_2
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Examples_1_2_1_5_revisited
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.Example24
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ExampleB

section
/- concordance: source L877 EXERCISE 2.17 equations for erase-ones -/
```

```
/-- **The four equations of Exercise 2.17.** `g(0x) = 0g(x)`, `g(11x) = g(1x)`, `g(10x)
     = 0x` for
*every* `x : |B|`, plus the single value equation `g(1) = False` (this last one is *not
    * quantified
over `x`: it pins down `g` at the single finite element `1False`, matching Scott's
    literal statement
"`g(1) = False`"). -/
structure SatisfiesRunEquations (g : ApproximableMap B B) : Prop where
  zero : forall  x : B.Element, g.toElementMap (sigmaElt [false] x) = sigmaElt [false]
     (g.toElementMap x)
  oneOne : forall  x : B.Element, g.toElementMap (sigmaElt [true, true] x) = g.
    toElementMap (sigmaElt [true] x)
  oneZero : forall  x : B.Element, g.toElementMap (sigmaElt [true, false] x) =
    sigmaElt [false] x
  one : g.toElementMap (sigmaElt [true] B.bot) = B.bot
end

section
/-- Inside the first run of `1`'s: swallow `1`'s; on the first `0`, emit it and copy
    the rest. -/
def del : Str  ->  Str
  | [] => []
  | true :: t => del t
  | false :: t => false :: t
end

section
@[simp] theorem del_nil : del [] = [] := rfl
end

section
@[simp] theorem del_true (t : Str) : del (true :: t) = del t := rfl
end

section
@[simp] theorem del_false (t : Str) : del (false :: t) = false :: t := rfl
end

section
/-- The guaranteed output prefix: copy leading `0`'s; on the first `1`, eliminate the
    run via `del`.
-/
def out : Str  ->  Str
  | [] => []
  | false :: t => false :: out t
  | true :: t => del t
end

section
@[simp] theorem out_nil : out [] = [] := rfl
end

section
@[simp] theorem out_false (t : Str) : out (false :: t) = false :: out t := rfl
end
```

```
section
@[simp] theorem out_true (t : Str) : out (true :: t) = del t := rfl
end

section
/-- `del` only grows under extension: `del s <+: del (s ++ t)`. -/
theorem del_append (s t : Str) : del s <+: del (s ++ t) := by
  induction s with
  | nil => simp
  | cons c s_0 ih =>
    cases c with
    | true => simpa using ih
    | false => exact <t, rfl>
end

section
/-- `out` only grows under extension: `out sigma <+: out (sigma ++ t)`. -/
theorem out_append (sigma t : Str) : out sigma <+: out (sigma ++ t) := by
  induction sigma with
  | nil => simp
  | cons c sigma_0 ih =>
    cases c with
    | true => simpa using del_append sigma_0 t
    | false =>
      simp only [List.cons_append, out_false]
      obtain <u, hu> := ih
      exact <u, by rw [List.cons_append, hu]>
end

section
/-- **Prefix-monotonicity of the output.** Extending the input prefix can only extend
    the guaranteed
output: `sigma <+: sigma'  ->  out sigma <+: out sigma'`. This is the heart of
    Definition 2.1(iii) for `g`. -/
theorem out_mono {sigma sigma' : Str} (h : sigma <+: sigma') : out sigma <+: out sigma
    ' := by
  obtain <t, rfl> := h
  exact out_append sigma t
end

end DomainTheory.NeighbourhoodSystems.Example24
```

3.3.20 Exercise 2.17 — uniqueness of the erase-ones map

Establish in detail that the mapping $g : \mathbb{B} \to \mathbb{B}$ of Exercise 2.4 is approximable. Is it uniquely determined by these equations:

$$g(0x) = 0g(x),$$
$$g(11x) = g(1x),$$
$$g(10x) = 0x,$$
$$g(1) = \bot,$$

or are some missing?

Exercise_2_17b.lean

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Example_2_4

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_22
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_20
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_8
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.Example24
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ExampleB

section
/-- **Exercise 2.17, second half.** `runMap` is the *unique* approximable map `B  ->  B
    ` satisfying
`g(0x) = 0g(x)`, `g(11x) = g(1x)`, `g(10x) = 0x`, `g(1) = False`. Every neighbourhood
    of `B` is a cone
(`memB_cone`), so `ApproximableMap.ext` reduces `g = runMap` to matching relations on
    cones, which
is exactly `key` read through `rel_iff_mem_principal`/`runMap_rel_cone`. -/
theorem eq_runMap_of_satisfies {g : ApproximableMap B B} (hg : SatisfiesRunEquations g
    ) :
    g = runMap := by
  apply ApproximableMap.ext
  intro X Y
  by_cases hX : B.mem X
  * obtain <sigma, rfl> := hX
    rw [g.rel_iff_mem_principal (memB_cone sigma)]
    show (g.toElementMap (sigmaBot sigma)).mem Y  <->  runMap.rel (cone sigma) Y
    rw [ <-  sigmaElt_bot, key g hg sigma, sigmaElt_bot, runMap_rel_cone]
    show (B.principal (memB_cone (out sigma))).mem Y  <->  B.mem Y  /\  cone (out
    sigma)  subseteq  Y
    exact B.mem_principal (memB_cone (out sigma))
  * exact <fun hr => absurd (g.rel_dom hr) hX, fun hr => absurd (runMap.rel_dom hr) hX
    >
end

section
/-- **`runMap`'s elementwise value, read off cone-by-cone.** `g(x).mem Y` iff *some*
    cone
`sigmaSigma*  in  x` already witnesses `Y  in  B` and `(out sigma)Sigma*  subseteq  Y`.
     -/
theorem runMap_toElementMap_mem (x : B.Element) (Y : Set Str) :
    (runMap.toElementMap x).mem Y  <->  exists  tau, x.mem (cone tau)  /\  B.mem Y  /\
      cone (out tau)  subseteq  Y := by
  constructor
  * rintro <X, hX, hrel>
    obtain <tau, rfl> := x.sub hX
    exact <tau, hX, (runMap_rel_cone tau Y).mp hrel>
```

```
  * rintro <tau, htau, hYmem, hsub>
    exact <cone tau, htau, (runMap_rel_cone tau Y).mpr <hYmem, hsub>>
end

section
/-- **The "shift formula" for `runMap` (input side).** Reading off `g(sigmax)` depends
    on `x` only
through *some* cone `cone tau  in  x`, via the output of the concatenation `sigma ++
    tau`: any deeper cone in
`sigmaElt`'s built-in up-closure is subsumed by monotonicity (`out_mono`), so the
    longest one --
`sigma ++ tau` itself -- already witnesses the answer. -/
theorem runMap_toElementMap_sigmaElt (sigma : Str) (x : B.Element) (Y : Set Str) :
    (runMap.toElementMap (sigmaElt sigma x)).mem Y  <->
      exists  tau, x.mem (cone tau)  /\  B.mem Y  /\  cone (out (sigma ++ tau))
    subseteq  Y := by
  constructor
  * rintro <Z, <_, X, hX, hsub>, ?, rfl, hYmem, hsubOut>
    obtain <tau, rfl> := x.sub hX
    rw [prepend_cone] at hsub
    exact <tau, hX, hYmem, (cone_subset_cone.mpr (out_mono (cone_subset_cone.mp hsub))
    ).trans hsubOut>
  * rintro <tau, htau, hYmem, hsub>
    exact <cone (sigma ++ tau), <memB_cone _, cone tau, htau, (prepend_cone sigma tau)
    .le>,
      (runMap_rel_cone (sigma ++ tau) Y).mpr <hYmem, hsub>>
end

section
/-- **The "shift formula" for `runMap` (output side).** Since `g(x)`'s membership
    predicate is
itself a cone-monotone family (`runMap_toElementMap_mem`), prepending `tau_0` to `g(x)`
     collapses the
same way: the up-closure of neighbourhoods `Z  supseteq  (out tau)Sigma*` is absorbed
    because `(out tau)Sigma*` itself
is the *deepest* (hence hardest-to-satisfy, so witness-minimal) choice. -/
theorem sigmaElt_runMap_toElementMap (tau_0 : Str) (x : B.Element) (Y : Set Str) :
    (sigmaElt tau_0 (runMap.toElementMap x)).mem Y  <->
      exists  tau, x.mem (cone tau)  /\  B.mem Y  /\  cone (tau_0 ++ out tau)
    subseteq  Y := by
  constructor
  * rintro <hYmem, Z, hZ, hsub>
    obtain <tau, htau, _, hZsub> := (runMap_toElementMap_mem x Z).mp hZ
    refine <tau, htau, hYmem, ?_>
    rw [ <-  prepend_cone]
    exact (prepend_mono tau_0 hZsub).trans hsub
  * rintro <tau, htau, hYmem, hsub>
    refine <hYmem, cone (out tau),
      (runMap_toElementMap_mem x _).mpr <tau, htau, memB_cone _, subset_rfl>, ?_>
    rw [prepend_cone]
    exact hsub
end

section
/-- **`runMap` satisfies its own defining equations.** `zero`/`oneOne`/`oneZero` are
    direct
```

```
instances of the two "shift formulas" (`runMap_toElementMap_sigmaElt` on the input side
    ,
`sigmaElt_runMap_toElementMap` on the output side), using that the output of a leading
    `0`
(resp. `11`, `10`) is fixed by `out_append_false`/`out_append_trueTrue`/`
    out_append_trueFalse`
regardless of the tail. `one` reads off the unique cone (`?`) that `False` sits in
(`bot_mem_cone_iff`), where `out [true] = []`. -/
theorem runMap_satisfies : SatisfiesRunEquations runMap where
  zero x := by
    apply Element.ext
    intro Y
    rw [runMap_toElementMap_sigmaElt, sigmaElt_runMap_toElementMap]
    constructor
    * rintro <tau, htau, hYmem, hsub>
      exact <tau, htau, hYmem, by rwa [out_append_false] at hsub>
    * rintro <tau, htau, hYmem, hsub>
      exact <tau, htau, hYmem, by rwa [out_append_false]>
  oneOne x := by
    apply Element.ext
    intro Y
    rw [runMap_toElementMap_sigmaElt, runMap_toElementMap_sigmaElt]
    constructor
    * rintro <tau, htau, hYmem, hsub>
      exact <tau, htau, hYmem, by rwa [out_append_trueTrue] at hsub>
    * rintro <tau, htau, hYmem, hsub>
      exact <tau, htau, hYmem, by rwa [ <-  out_append_trueTrue] at hsub>
  oneZero x := by
    apply Element.ext
    intro Y
    rw [runMap_toElementMap_sigmaElt]
    constructor
    * rintro <tau, htau, hYmem, hsub>
      rw [out_append_trueFalse] at hsub
      exact <hYmem, cone tau, htau, by rw [prepend_cone]; exact hsub>
    * rintro <hYmem, X, hX, hsub>
      obtain <tau, rfl> := x.sub hX
      rw [prepend_cone] at hsub
      exact <tau, hX, hYmem, by rwa [out_append_trueFalse]>
  one := by
    apply Element.ext
    intro Y
    rw [runMap_toElementMap_sigmaElt]
    constructor
    * rintro <tau, htau, hYmem, hsub>
      obtain rfl : tau = [] := bot_mem_cone_iff.mp htau
      rw [show out ([true] ++ ([] : Str)) = ([] : Str) from rfl, cone_nil] at hsub
      rw [mem_bot]
      exact Set.eq_univ_of_univ_subset hsub
    * intro hY
      rw [mem_bot] at hY
      refine <[], bot_mem_cone_iff.mpr rfl, hY  |>  B.master_mem, ?_>
      rw [show out ([true] ++ ([] : Str)) = ([] : Str) from rfl, cone_nil, hY,
    B_master]
end
```

```
end DomainTheory.NeighbourhoodSystems.Example24

namespace DomainTheory.NeighbourhoodSystems

namespace NeighborhoodSystem

section
open NeighborhoodSystem
universe u
variable {a : Type u} {b : Type*} {V_0 : NeighborhoodSystem a} {V_1 :
    NeighborhoodSystem b}

/-- **Algebraicity.** Every element `x` is the directed union of its own principal
("finite"/compact) approximants: `x = ? {X ? X  in  x}`, literally as an `iSupDirected
    `. (Used below
to reduce continuity checks to principal elements; a duplicate of `Theorem85.lean`'s
    own copy,
kept local here to avoid a heavy import for this early file.) -/
instance instNonemptyMemSubtype (x : V_0.Element) : Nonempty {X : Set a // x.mem X} :=
  <<V_0.master, x.master_mem>>

/-- A continuous function between domains is monotone for the approximation order `
    sqsub `: this is
because ` sqsub ` is recoverable from the topology (`le_iff_isOpen_imp`) and continuous
     preimages of opens
are open and upward closed (`isOpen_isUpperSet`). -/
theorem continuous_monotone {c : V_0.Element  ->  V_1.Element} (hc : Continuous c) :
    Monotone c := by
  intro x y hxy
  rw [V_1.le_iff_isOpen_imp]
  intro U hU hxU
  exact V_0.isOpen_isUpperSet (hU.preimage hc) hxU hxy

end

end NeighborhoodSystem

end DomainTheory.NeighbourhoodSystems
```

3.3.21 Exercise 2.18 — doubling binary digits

What is the meaning in words of the approximable mapping $h : \mathbb{B} \to \mathbb{B}$, where

$$h(0x) = 00h(x), \text{ and}$$
$$h(1x) = 10h(x),$$

for all elements $x \in |\mathbb{B}|$? Is h an isomorphism? Does there exist a map $k : \mathbb{B} \to \mathbb{B}$ where

$$k \circ h = I_{\mathbb{B}},$$

and is k one-one?

Exercise_2_18.lean

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Theorem_2_5
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Examples_1_2_1_5_revisited
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_7
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.Exercise218
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ExampleB ApproximableMap

section
/-- The guaranteed output of `h` on a finite prefix: each symbol `b` becomes the block
    `b0`. -/
def hOut : Str  ->  Str
  | [] => []
  | b :: t => b :: false :: hOut t
end

section
@[simp] theorem hOut_nil : hOut [] = [] := rfl
end

section
@[simp] theorem hOut_cons (b : Bool) (t : Str) : hOut (b :: t) = b :: false :: hOut t
    := rfl
end

section
/-- `hOut` grows under extension: `hOut sigma <+: hOut (sigma ++ t)`. -/
theorem hOut_append (sigma t : Str) : hOut sigma <+: hOut (sigma ++ t) := by
  induction sigma with
  | nil => simp
  | cons b s ih =>
    simp only [List.cons_append, hOut_cons]
    obtain <u, hu> := ih
    exact <u, by rw [List.cons_append, List.cons_append, hu]>
end

section
/-- **Prefix-monotonicity of `hOut`.** `sigma <+: sigma'  ->  hOut sigma <+: hOut sigma
    '`. -/
theorem hOut_mono {sigma sigma' : Str} (h : sigma <+: sigma') : hOut sigma <+: hOut
    sigma' := by
  obtain <t, rfl> := h
  exact hOut_append sigma t
end

section
/-- The guaranteed output of `k` on a finite prefix: keep the first bit of every *
    complete* pair,
discard the inserted `0`. A trailing odd bit is an incomplete pair, hence dropped. -/
def kOut : Str  ->  Str
  | [] => []
```

```
  | [_] => []
  | b :: _ :: t => b :: kOut t
end

section
@[simp] theorem kOut_nil : kOut [] = [] := rfl
end

section
@[simp] theorem kOut_single (b : Bool) : kOut [b] = [] := rfl
end

section
@[simp] theorem kOut_cons (b c : Bool) (t : Str) : kOut (b :: c :: t) = b :: kOut t :=
     rfl
end

section
/- concordance: source L888 EXERCISE 2.18 doubling binary digits -/
/-- **Exercise 2.18 -- the spacing map `h : ?  ->  ?`.** `X h Y  <->  exists  sigma, X
    = sigmaSigma*  /\  Y  in  ?  /\  (hOut sigma)Sigma*  subseteq  Y`.
Same shape as `Example24.runMap`, with the doubling `hOut` (`hOut_mono` gives
    Definition 2.1(iii)). -/
def hMap : ApproximableMap B B where
  rel X Y := exists  sigma, X = cone sigma  /\  B.mem Y  /\  cone (hOut sigma)
    subseteq  Y
  rel_dom := fun <sigma, hX, _, _> => <sigma, hX>
  rel_cod := fun <_, _, hYmem, _> => hYmem
  master_rel := by
    refine <[], (B_master).trans cone_nil.symm, B.master_mem, ?_>
    rw [hOut_nil, B_master, cone_nil]
  inter_right := by
    rintro X Y Y' <sigma, hX, hYmem, hYsub> <sigma', hX', hY'mem, hY'sub>
    have hsigma : sigma = sigma' := cone_injective (hX  |>  hX')
    subst hsigma
    have hsub : cone (hOut sigma)  subseteq  Y  I  Y' := Set.subset_inter hYsub hY'sub
    exact <sigma, hX, B.inter_mem hYmem hY'mem (memB_cone (hOut sigma)) hsub, hsub>
  mono := by
    rintro X X' Y Y' <sigma, hX, _, hYsub> hX'X hYY' hX'mem hY'mem
    obtain <sigma', hX'cone> := hX'mem
    have hpre : sigma <+: sigma' := by
      apply cone_subset_cone.mp; rw [ <-  hX'cone,  <-  hX]; exact hX'X
    have hcone : cone (hOut sigma')  subseteq  cone (hOut sigma) := cone_subset_cone.
    mpr (hOut_mono hpre)
    exact <sigma', hX'cone, hY'mem, (hcone.trans hYsub).trans hYY'>
end

section
/-- `kOut` grows under extension: `kOut sigma <+: kOut (sigma ++ t)`. -/
theorem kOut_append (sigma t : Str) : kOut sigma <+: kOut (sigma ++ t) := by
  induction sigma using kOut.induct with
  | case1 => simp
  | case2 _ => simp
  | case3 b c t' ih =>
    simp only [List.cons_append, kOut_cons]
    obtain <u, hu> := ih
```

```
    exact <u, by rw [List.cons_append, hu]>
end

section
/-- **Prefix-monotonicity of `kOut`.** `sigma <+: sigma'  ->  kOut sigma <+: kOut sigma
    '`. -/
theorem kOut_mono {sigma sigma' : Str} (h : sigma <+: sigma') : kOut sigma <+: kOut
    sigma' := by
  obtain <t, rfl> := h
  exact kOut_append sigma t
end

section
/-- **Exercise 2.18 -- the left inverse `k : ?  ->  ?`.** `X k Y  <->  exists  sigma, X
     = sigmaSigma*  /\  Y  in  ?  /\  (kOut sigma)Sigma*  subseteq  Y`. -/
def kMap : ApproximableMap B B where
  rel X Y := exists  sigma, X = cone sigma  /\  B.mem Y  /\  cone (kOut sigma)
    subseteq  Y
  rel_dom := fun <sigma, hX, _, _> => <sigma, hX>
  rel_cod := fun <_, _, hYmem, _> => hYmem
  master_rel := by
    refine <[], (B_master).trans cone_nil.symm, B.master_mem, ?_>
    rw [kOut_nil, B_master, cone_nil]
  inter_right := by
    rintro X Y Y' <sigma, hX, hYmem, hYsub> <sigma', hX', hY'mem, hY'sub>
    have hsigma : sigma = sigma' := cone_injective (hX  |>  hX')
    subst hsigma
    have hsub : cone (kOut sigma)  subseteq  Y  I  Y' := Set.subset_inter hYsub hY'sub
    exact <sigma, hX, B.inter_mem hYmem hY'mem (memB_cone (kOut sigma)) hsub, hsub>
  mono := by
    rintro X X' Y Y' <sigma, hX, _, hYsub> hX'X hYY' hX'mem hY'mem
    obtain <sigma', hX'cone> := hX'mem
    have hpre : sigma <+: sigma' := by
      apply cone_subset_cone.mp; rw [ <-  hX'cone,  <-  hX]; exact hX'X
    have hcone : cone (kOut sigma')  subseteq  cone (kOut sigma) := cone_subset_cone.
    mpr (kOut_mono hpre)
    exact <sigma', hX'cone, hY'mem, (hcone.trans hYsub).trans hYY'>
end

section
/-- **`k` inverts `h` on prefixes.** `kOut (hOut sigma) = sigma`. -/
theorem kOut_hOut (sigma : Str) : kOut (hOut sigma) = sigma := by
  induction sigma with
  | nil => rfl
  | cons b t ih => simp only [hOut_cons, kOut_cons, ih]
end

section
/-- **Exercise 2.18 -- `k  comp  h = I_?`.** The composite recovers the identity:
    reading `kOut (hOut sigma) = sigma`
through the cone relations. -/
theorem kMap_comp_hMap : kMap.comp hMap = idMap B := by
  apply ApproximableMap.ext
  intro X Z
  rw [comp_rel, idMap_rel]
  constructor
```

```
  * rintro <Y, <sigma, hXcone, _, hYsub>, tau, hYcone, hZmem, hZsub>
    refine <<sigma, hXcone>, hZmem, ?_>
    have htau : tau <+: hOut sigma := cone_subset_cone.mp (hYcone  |>  hYsub)
    have htausigma : kOut tau <+: sigma := by have := kOut_mono htau; rwa [kOut_hOut]
    at this
    calc X = cone sigma := hXcone
      _  subseteq  cone (kOut tau) := cone_subset_cone.mpr htausigma
      _  subseteq  Z := hZsub
  * rintro <<sigma, hXcone>, hZmem, hXZ>
    refine <cone (hOut sigma), <sigma, hXcone, memB_cone (hOut sigma), subset_rfl>,
    hOut sigma, rfl, hZmem, ?_>
    rw [kOut_hOut, hXcone] at *
    exact hXZ
end

section
/-- The elementwise value of `hMap` on the finite element `(sigmaSigma*)` is `((hOut
    sigma)Sigma*)`. -/
theorem toElementMap_hMap_cone (sigma : Str) :
    hMap.toElementMap (B.principal (memB_cone sigma)) = B.principal (memB_cone (hOut
    sigma)) := by
  apply Element.ext
  intro Y
  rw [ <-  hMap.rel_iff_mem_principal (memB_cone sigma), mem_principal]
  constructor
  * rintro <?, hcone, hYmem, hsub>
    have : sigma = ? := cone_injective hcone
    subst this
    exact <hYmem, hsub>
  * rintro <hYmem, hsub>
    exact <sigma, rfl, hYmem, hsub>
end

section
/-- The elementwise value of `kMap` on the finite element `(sigmaSigma*)` is `((kOut
    sigma)Sigma*)`. -/
theorem toElementMap_kMap_cone (sigma : Str) :
    kMap.toElementMap (B.principal (memB_cone sigma)) = B.principal (memB_cone (kOut
    sigma)) := by
  apply Element.ext
  intro Y
  rw [ <-  kMap.rel_iff_mem_principal (memB_cone sigma), mem_principal]
  constructor
  * rintro <?, hcone, hYmem, hsub>
    have : sigma = ? := cone_injective hcone
    subst this
    exact <hYmem, hsub>
  * rintro <hYmem, hsub>
    exact <sigma, rfl, hYmem, hsub>
end

section
/-- **Exercise 2.18 -- `h` is not an isomorphism.** It is not surjective: `(01Sigma*)`
    is not in the
range (any preimage `x` satisfies `x = k(h(x))`, forcing `h(x) = (00Sigma*)  !=  (01
    Sigma*)`). -/
```

```
theorem hMap_not_surjective : not  Function.Surjective hMap.toElementMap := by
  intro hsurj
  obtain <x, hx> := hsurj (B.principal (memB_cone [false, true]))
  have hkhx : kMap.toElementMap (hMap.toElementMap x) = x := by
    rw [ <-  toElementMap_comp, kMap_comp_hMap, toElementMap_idMap]
  rw [hx, toElementMap_kMap_cone] at hkhx
  rw [ <-  hkhx, toElementMap_hMap_cone] at hx
  have hcone := cone_injective (B.principal_injective (memB_cone _) (memB_cone _) hx)
  simp at hcone
end

section
/-- **Exercise 2.18 -- `k` is not one-one.** `(0Sigma*)` and `(1Sigma*)` are distinct
    finite elements that
`k` collapses to `False` (a length-1 prefix is an incomplete pair, so its bit is
    dropped). -/
theorem kMap_not_injective : not  Function.Injective kMap.toElementMap := by
  intro hinj
  have h1 : kMap.toElementMap (B.principal (memB_cone [false]))
      = kMap.toElementMap (B.principal (memB_cone [true])) := by
    rw [toElementMap_kMap_cone, toElementMap_kMap_cone]; rfl
  have h2 := hinj h1
  have hcone := cone_injective (B.principal_injective (memB_cone _) (memB_cone _) h2)
  simp at hcone
end

section
set_option linter.unusedVariables false

theorem example_218 : And
  (And
    (Not
      (@Function.Surjective
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
          DomainTheory.NeighbourhoodSystems.ExampleB.B)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
          DomainTheory.NeighbourhoodSystems.ExampleB.B)
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
          DomainTheory.NeighbourhoodSystems.ExampleB.Str DomainTheory.
    NeighbourhoodSystems.ExampleB.B
          DomainTheory.NeighbourhoodSystems.ExampleB.B DomainTheory.
    NeighbourhoodSystems.Exercise218.hMap)))
    (@Eq
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
        DomainTheory.NeighbourhoodSystems.ExampleB.Str DomainTheory.
    NeighbourhoodSystems.ExampleB.B
        DomainTheory.NeighbourhoodSystems.ExampleB.B)
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
        DomainTheory.NeighbourhoodSystems.ExampleB.Str DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
        DomainTheory.NeighbourhoodSystems.ExampleB.B DomainTheory.NeighbourhoodSystems
```

```
  .ExampleB.B
       DomainTheory.NeighbourhoodSystems.ExampleB.B DomainTheory.NeighbourhoodSystems
  .Exercise218.kMap
       DomainTheory.NeighbourhoodSystems.Exercise218.hMap)
     (@DomainTheory.NeighbourhoodSystems.ApproximableMap.idMap DomainTheory.
  NeighbourhoodSystems.ExampleB.Str
       DomainTheory.NeighbourhoodSystems.ExampleB.B)))
(Not
  (@Function.Injective
     (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element DomainTheory.
  NeighbourhoodSystems.ExampleB.Str
       DomainTheory.NeighbourhoodSystems.ExampleB.B)
     (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element DomainTheory.
  NeighbourhoodSystems.ExampleB.Str
       DomainTheory.NeighbourhoodSystems.ExampleB.B)
     (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap DomainTheory.
  NeighbourhoodSystems.ExampleB.Str
       DomainTheory.NeighbourhoodSystems.ExampleB.Str DomainTheory.
  NeighbourhoodSystems.ExampleB.B
       DomainTheory.NeighbourhoodSystems.ExampleB.B DomainTheory.NeighbourhoodSystems
  .Exercise218.kMap))) :=
@And.intro
(And
  (Not
    (@Function.Surjective
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element DomainTheory.
  NeighbourhoodSystems.ExampleB.Str
        DomainTheory.NeighbourhoodSystems.ExampleB.B)
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element DomainTheory.
  NeighbourhoodSystems.ExampleB.Str
        DomainTheory.NeighbourhoodSystems.ExampleB.B)
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap DomainTheory.
  NeighbourhoodSystems.ExampleB.Str
        DomainTheory.NeighbourhoodSystems.ExampleB.Str DomainTheory.
  NeighbourhoodSystems.ExampleB.B
        DomainTheory.NeighbourhoodSystems.ExampleB.B DomainTheory.
  NeighbourhoodSystems.Exercise218.hMap)))
  (@Eq
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap DomainTheory.
  NeighbourhoodSystems.ExampleB.Str
       DomainTheory.NeighbourhoodSystems.ExampleB.Str DomainTheory.
  NeighbourhoodSystems.ExampleB.B
       DomainTheory.NeighbourhoodSystems.ExampleB.B)
     (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp DomainTheory.
  NeighbourhoodSystems.ExampleB.Str
       DomainTheory.NeighbourhoodSystems.ExampleB.Str DomainTheory.
  NeighbourhoodSystems.ExampleB.Str
       DomainTheory.NeighbourhoodSystems.ExampleB.B DomainTheory.NeighbourhoodSystems
  .ExampleB.B
       DomainTheory.NeighbourhoodSystems.ExampleB.B DomainTheory.NeighbourhoodSystems
  .Exercise218.kMap
       DomainTheory.NeighbourhoodSystems.Exercise218.hMap)
     (@DomainTheory.NeighbourhoodSystems.ApproximableMap.idMap DomainTheory.
  NeighbourhoodSystems.ExampleB.Str
       DomainTheory.NeighbourhoodSystems.ExampleB.B)))
(Not
```

```
    (@Function.Injective
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
         DomainTheory.NeighbourhoodSystems.ExampleB.B)
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
         DomainTheory.NeighbourhoodSystems.ExampleB.B)
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
         DomainTheory.NeighbourhoodSystems.ExampleB.Str DomainTheory.
    NeighbourhoodSystems.ExampleB.B
         DomainTheory.NeighbourhoodSystems.ExampleB.B DomainTheory.NeighbourhoodSystems
    .Exercise218.kMap)))
  (@And.intro
    (Not
      (@Function.Surjective
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
          DomainTheory.NeighbourhoodSystems.ExampleB.B)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
          DomainTheory.NeighbourhoodSystems.ExampleB.B)
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
          DomainTheory.NeighbourhoodSystems.ExampleB.Str DomainTheory.
    NeighbourhoodSystems.ExampleB.B
          DomainTheory.NeighbourhoodSystems.ExampleB.B DomainTheory.
    NeighbourhoodSystems.Exercise218.hMap)))
    (@Eq
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
         DomainTheory.NeighbourhoodSystems.ExampleB.Str DomainTheory.
    NeighbourhoodSystems.ExampleB.B
         DomainTheory.NeighbourhoodSystems.ExampleB.B)
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
         DomainTheory.NeighbourhoodSystems.ExampleB.Str DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
         DomainTheory.NeighbourhoodSystems.ExampleB.B DomainTheory.NeighbourhoodSystems
    .ExampleB.B
         DomainTheory.NeighbourhoodSystems.ExampleB.B DomainTheory.NeighbourhoodSystems
    .Exercise218.kMap
         DomainTheory.NeighbourhoodSystems.Exercise218.hMap)
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.idMap DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
         DomainTheory.NeighbourhoodSystems.ExampleB.B))
    DomainTheory.NeighbourhoodSystems.Exercise218.hMap_not_surjective
    DomainTheory.NeighbourhoodSystems.Exercise218.kMap_comp_hMap)
  DomainTheory.NeighbourhoodSystems.Exercise218.kMap_not_injective
end

end DomainTheory.NeighbourhoodSystems.Exercise218
```

3.3.22 Exercise 2.19 — maps of two arguments

Generalize Definition 2.1 in an appropriate way in order to define the concept of *an approximable mapping*

$$f : \mathcal{D}_0 \times \mathcal{D}_1 \to \mathcal{D}_2$$

of two variables. (Hint: f can be taken to be a certain kind of ternary relation

$$f \subseteq \mathcal{D}_0 \times \mathcal{D}_1 \times \mathcal{D}_2,$$

where we can write

$$X, Y \; f \; Z$$

for the relationship among neighbourhoods.) What is the corresponding version of Proposition 2.2 for functions of two variables?

Exercise_2_19.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.principal
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_1
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem

section
variable {a b ? : Type*}

/- concordance: source L899 EXERCISE 2.19 maps of two arguments -/
/-- **Exercise 2.19 (Scott 1981, PRG-19).** An *approximable mapping of two variables*
`f : ?_0  x  ?_1  ->  ?_2` is a ternary relation `X, Y f Z` confined to `?_0  x  ?_1  x
      ?_2` with the natural
generalization of Definition 2.1: (i) `Delta_0, Delta_1 f Delta_2`; (ii) intersectivity
     on the output; (iii)
monotonicity jointly in both inputs (sharper) and the output (blunter). -/
structure ApproximableMap_2 (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b)
    (V_2 : NeighborhoodSystem ?) where
  /-- The underlying ternary relation `X, Y f Z`. -/
  rel : Set a  ->  Set b  ->  Set ?  ->  Prop
  rel_dom_0 : forall  {X Y Z}, rel X Y Z  ->  V_0.mem X
  rel_dom_1 : forall  {X Y Z}, rel X Y Z  ->  V_1.mem Y
  rel_cod : forall  {X Y Z}, rel X Y Z  ->  V_2.mem Z
  /-- (i) `Delta_0, Delta_1 f Delta_2`. -/
  master_rel : rel V_0.master V_1.master V_2.master
  /-- (ii) intersectivity on the output. -/
  inter_right : forall  {X Y Z Z'}, rel X Y Z  ->  rel X Y Z'  ->  rel X Y (Z  I  Z')
  /-- (iii) joint monotonicity: sharper inputs `X'  subseteq  X`, `Y'  subseteq  Y`;
    blunter output `Z  subseteq  Z'`. -/
  mono : forall  {X X' Y Y' Z Z'}, rel X Y Z  ->  X'  subseteq  X  ->  Y'  subseteq  Y
      ->  Z  subseteq  Z'  ->
    V_0.mem X'  ->  V_1.mem Y'  ->  V_2.mem Z'  ->  rel X' Y' Z'
```

```
namespace ApproximableMap_2

variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b} {V_2 :
    NeighborhoodSystem ?}

/-- **Exercise 2.19 (Proposition 2.2 analogue).** A two-variable approximable mapping
    determines an
elementwise function of two arguments: `f(x, y) = {Z ? exists  X  in  x, exists  Y  in
     y, X, Y f Z}`. The filter
laws use all three conditions: `inter_mem` pulls both outputs back to `(X  I  X', Y  I
     Y')` via `mono`
then `inter_right`. -/
def toElementMap_2 (f : ApproximableMap_2 V_0 V_1 V_2) (x : V_0.Element) (y : V_1.
    Element) : V_2.Element where
  mem Z := exists  X Y, x.mem X  /\  y.mem Y  /\  f.rel X Y Z
  sub := fun <_, _, _, _, hrel> => f.rel_cod hrel
  master_mem := <V_0.master, V_1.master, x.master_mem, y.master_mem, f.master_rel>
  inter_mem := by
    rintro Z Z' <X, Y, hX, hY, hrel> <X', Y', hX', hY', hrel'>
    have hXX' : x.mem (X  I  X') := x.inter_mem hX hX'
    have hYY' : y.mem (Y  I  Y') := y.inter_mem hY hY'
    have hXX'm : V_0.mem (X  I  X') := x.sub hXX'
    have hYY'm : V_1.mem (Y  I  Y') := y.sub hYY'
    refine <X  I  X', Y  I  Y', hXX', hYY', ?_>
    have h1 : f.rel (X  I  X') (Y  I  Y') Z :=
      f.mono hrel Set.inter_subset_left Set.inter_subset_left subset_rfl hXX'm hYY'm (
    f.rel_cod hrel)
    have h2 : f.rel (X  I  X') (Y  I  Y') Z' :=
      f.mono hrel' Set.inter_subset_right Set.inter_subset_right subset_rfl hXX'm hYY'
    m
        (f.rel_cod hrel')
    exact f.inter_right h1 h2
  up_mem := by
    rintro Z Z' <X, Y, hX, hY, hrel> hZ' hZZ'
    exact <X, Y, hX, hY, f.mono hrel subset_rfl subset_rfl hZZ' (x.sub hX) (y.sub hY)
    hZ'>

end ApproximableMap_2

namespace NeighborhoodSystem

end NeighborhoodSystem

namespace ApproximableMap

variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b} {V_2 :
    NeighborhoodSystem ?}

end ApproximableMap

namespace ApproximableMap_2

variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b} {V_2 :
    NeighborhoodSystem ?}
```

```
@[simp] theorem mem_toElementMap_2 (f : ApproximableMap_2 V_0 V_1 V_2) (x : V_0.
    Element) (y : V_1.Element)
    {Z : Set ?} : (f.toElementMap_2 x y).mem Z  <->  exists  X Y, x.mem X  /\  y.mem Y
      /\  f.rel X Y Z := Iff.rfl
end ApproximableMap_2

namespace ApproximableMap_2

variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b} {V_2 :
    NeighborhoodSystem ?}

/-- **Exercise 2.19 (recovery of the relation).** `X, Y f Z  <->  Z  in  f(X, Y)`, the
    two-variable
analogue of Proposition 2.2(ii). -/
theorem rel_2_iff_mem_principal (f : ApproximableMap_2 V_0 V_1 V_2) {X : Set a} (hX :
    V_0.mem X)
    {Y : Set b} (hY : V_1.mem Y) {Z : Set ?} :
    f.rel X Y Z  <->  (f.toElementMap_2 (V_0.principal hX) (V_1.principal hY)).mem Z
    := by
  constructor
  * intro hrel
    exact <X, Y, <hX, subset_rfl>, <hY, subset_rfl>, hrel>
  * rintro <X', Y', <_, hXX'>, <_, hYY'>, hrel>
    exact f.mono hrel hXX' hYY' subset_rfl hX hY (f.rel_cod hrel)

end ApproximableMap_2

namespace ApproximableMap_2

variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b} {V_2 :
    NeighborhoodSystem ?}

/-- **Exercise 2.19 (monotonicity).** The two-variable elementwise map is monotone in
    each argument
jointly: `x  sqsub  x'`, `y  sqsub  y'` imply `f(x, y)  sqsub  f(x', y')`. -/
theorem toElementMap_2_mono (f : ApproximableMap_2 V_0 V_1 V_2) {x x' : V_0.Element} {
    y y' : V_1.Element}
    (hx : x  <=  x') (hy : y  <=  y') : f.toElementMap_2 x y  <=  f.toElementMap_2 x'
    y' := by
  rintro Z <X, Y, hX, hY, hrel>
  exact <X, Y, hx X hX, hy Y hY, hrel>

end ApproximableMap_2
end

end DomainTheory.NeighbourhoodSystems
```

3.3.23 Exercise 2.20 — powerset of the integers

Discuss again the example of Exercise 1.15 where the domain turns out to be the powerset (set of all subsets) of $\mathbf{N}$. Show how the finite elements can be taken to be the finite subsets of $\mathbf{N}$ and can be identified with the tokens of a suitable neighbourhood system $\mathcal{P}$. (Hint: Define $\uparrow F$ for finite sets $F \subseteq \mathbf{N}$.) Show that both union and intersection ($x \cup y$ and $x \cap y$) are functions on $|\mathcal{P}|$ that are approximable in the sense of Exercise

2.19. (The elements of $|\mathcal{P}|$ are being identified with arbitrary sets $x \subseteq \mathbf{N}$.) Show also that the following transformations are approximable:

$$x + 1 = \{n + 1 \mid n \in x\}, \text{ and}$$
$$x - 1 = \{n \mid n + 1 \in x\}.$$

Exercise_2_20.lean

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Exercise_2_19
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_2
public import Mathlib.Algebra.Group.End
public import Mathlib.Basic.Finite.Defs
public import Mathlib.Order.Hom.Basic
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.Exercise220
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem

section
/- concordance: source L913 EXERCISE 2.20 powerset of the integers -/
/-- **Exercise 2.20 -- the powerset system `?`.** Tokens `Nat`; the neighbourhoods are
    the cofinite
sets, closed under finite intersection (`(X  I  Y)? = X?  U  Y?`). -/
def powerSet : NeighborhoodSystem Nat where
  mem X := X?.Finite
  master := Set.univ
  master_nonempty := Set.univ_nonempty
  master_mem := by rw [Set.compl_univ]; exact Set.finite_empty
  inter_mem := by
    intro X Y _ hX hY _ _
    rw [Set.compl_inter]; exact hX.union hY
  sub_master := fun _ => Set.subset_univ _
end

section
@[simp] theorem mem_powerSet {X : Set Nat} : powerSet.mem X  <->  X?.Finite := Iff.rfl
end

section
@[simp] theorem powerSet_master : powerSet.master = (Set.univ : Set Nat) := rfl
end

section
/-- The complement of a singleton is a neighbourhood (it represents the finite element
     `{n}`). -/
theorem mem_compl_singleton (n : Nat) : powerSet.mem ({n}?) := by
  rw [mem_powerSet, compl_compl]; exact Set.finite_singleton n
end

section
/-- Cofinite sets are closed under intersection (witness-free form). -/
theorem powerSet_inter_mem {A B : Set Nat} (hA : powerSet.mem A) (hB : powerSet.mem B)
     :
```

```
    powerSet.mem (A  I  B) := by
  rw [mem_powerSet, Set.compl_inter]; exact hA.union hB
end

section
/-- The subset of `Nat` represented by an element: `n  in  toSet x  <->  {n}?  in  x`.
    -/
def toSet (x : powerSet.Element) : Set Nat := {n | x.mem ({n}?)}
end

section
@[simp] theorem mem_toSet {x : powerSet.Element} {n : Nat} : n  in  toSet x  <->  x.
    mem ({n}?) := Iff.rfl
end

section
/-- **Filters are determined on cofinite sets by their singleton-complements.** For
    cofinite `Z`,
`Z  in  x  <->  forall  n  in  Z?, {n}?  in  x` (since `Z = ?_{n  in  Z?} {n}?`, a *
    finite* intersection). -/
theorem mem_compl_of_finite (x : powerSet.Element) {t : Set Nat} (ht : t.Finite)
    (h : forall  n  in  t, x.mem ({n}?)) : x.mem (t?) := by
  revert h
  induction t, ht using Set.Finite.induction_on with
  | empty => intro _; rw [Set.compl_empty]; exact x.master_mem
  | @insert a s _ha _hsfin ih =>
    intro h
    have hcompl : (insert a s)? = ({a}?)  I  (s?) := by rw [Set.insert_eq, Set.
    compl_union]
    rw [hcompl]
    exact x.inter_mem (h a (Set.mem_insert a s))
      (ih (fun n hn => h n (Set.mem_insert_of_mem a hn)))
end

section
/-- The filter of a subset `S  subseteq  Nat`: the cofinite `Z` with `Z?  subseteq  S`.
    -/
def ofSet (S : Set Nat) : powerSet.Element where
  mem Z := powerSet.mem Z  /\  Z?  subseteq  S
  sub h := h.1
  master_mem := <powerSet.master_mem, by rw [powerSet_master, Set.compl_univ]; exact
    Set.empty_subset S>
  inter_mem := by
    rintro Z W <hZ, hZS> <hW, hWS>
    refine <powerSet_inter_mem hZ hW, ?_>
    rw [Set.compl_inter]; exact Set.union_subset hZS hWS
  up_mem := by
    rintro Z W <hZ, hZS> hW hZW
    exact <hW, (Set.compl_subset_compl.mpr hZW).trans hZS>
end

section
/-- `ofSet` is monotone in the set. -/
theorem ofSet_mono {S S' : Set Nat} (h : S  subseteq  S') : ofSet S  <=  ofSet S' :=
  fun _ hZ => <hZ.1, hZ.2.trans h>
end
```

```
section
theorem ofSet_toSet (x : powerSet.Element) : ofSet (toSet x) = x := by
  apply Element.ext
  intro Z
  constructor
  * rintro <hZ, hZsub>
    have hmem := mem_compl_of_finite x hZ (fun n hn => hZsub hn)
    rwa [compl_compl] at hmem
  * intro hZ
    refine <x.sub hZ, ?_>
    intro n hn
    rw [mem_toSet]
    exact x.up_mem hZ (mem_compl_singleton n) (Set.subset_compl_singleton_iff.mpr hn)
end

section
/-- `S ? 1 = {n ? n + 1  in  S}`. -/
def predSet (S : Set Nat) : Set Nat := {n | n + 1  in  S}
end

section
@[simp] theorem mem_predSet {S : Set Nat} {n : Nat} : n  in  predSet S  <->  n + 1  in
      S := Iff.rfl
end

section
/-- **Exercise 2.20 -- `x  I  y` is approximable (two variables).** `X, Y f Z  <->  X
    U  Y  subseteq  Z`
(`(X?  I  Y?)? = X  U  Y`). -/
def interMap_2 : ApproximableMap_2 powerSet powerSet powerSet where
  rel X Y Z := powerSet.mem X  /\  powerSet.mem Y  /\  powerSet.mem Z  /\  X  U  Y
    subseteq  Z
  rel_dom_0 h := h.1
  rel_dom_1 h := h.2.1
  rel_cod h := h.2.2.1
  master_rel := <powerSet.master_mem, powerSet.master_mem, powerSet.master_mem,
    Set.union_subset (Set.subset_univ _) (Set.subset_univ _)>
  inter_right := by
    rintro X Y Z Z' <hX, hY, hZ, hsub> <_, _, hZ', hsub'>
    exact <hX, hY, powerSet_inter_mem hZ hZ', Set.subset_inter hsub hsub'>
  mono := by
    rintro X X' Y Y' Z Z' <_, _, _, hsub> hX'X hY'Y hZZ' hX' hY' hZ'
    exact <hX', hY', hZ', (Set.union_subset_union hX'X hY'Y).trans (hsub.trans hZZ')>
end

section
theorem predSet_mono {S T : Set Nat} (h : S  subseteq  T) : predSet S  subseteq
    predSet T := fun _ hn => h hn
end

section
/-- **Exercise 2.20 -- `x ? 1` is approximable.** `X f Z  <->  Z?  subseteq  (X? ? 1)`.
     -/
def predMap : ApproximableMap powerSet powerSet where
  rel X Z := powerSet.mem X  /\  powerSet.mem Z  /\  Z?  subseteq  predSet X?
```

```
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := by
    refine <powerSet.master_mem, powerSet.master_mem, ?_>
    rw [powerSet_master, Set.compl_univ]; exact Set.empty_subset _
  inter_right := by
    rintro X Z Z' <hX, hZ, hsub> <_, hZ', hsub'>
    refine <hX, powerSet_inter_mem hZ hZ', ?_>
    rw [Set.compl_inter]; exact Set.union_subset hsub hsub'
  mono := by
    rintro X X' Z Z' <_, _, hsub> hX'X hZZ' hX' hZ'
    refine <hX', hZ', ?_>
    exact (Set.compl_subset_compl.mpr hZZ').trans
      (hsub.trans (predSet_mono (Set.compl_subset_compl.mpr hX'X)))
end

section
/-- `S + 1 = {n + 1 ? n  in  S}`. -/
def succSet (S : Set Nat) : Set Nat := (* + 1) '' S
end

section
theorem succSet_mono {S T : Set Nat} (h : S  subseteq  T) : succSet S  subseteq
    succSet T := Set.image_mono h
end

section
/-- **Exercise 2.20 -- `x + 1` is approximable.** `X f Z  <->  Z?  subseteq  (X? + 1)`.
    -/
def succMap : ApproximableMap powerSet powerSet where
  rel X Z := powerSet.mem X  /\  powerSet.mem Z  /\  Z?  subseteq  succSet X?
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := by
    refine <powerSet.master_mem, powerSet.master_mem, ?_>
    rw [powerSet_master, Set.compl_univ]; exact Set.empty_subset _
  inter_right := by
    rintro X Z Z' <hX, hZ, hsub> <_, hZ', hsub'>
    refine <hX, powerSet_inter_mem hZ hZ', ?_>
    rw [Set.compl_inter]; exact Set.union_subset hsub hsub'
  mono := by
    rintro X X' Z Z' <_, _, hsub> hX'X hZZ' hX' hZ'
    refine <hX', hZ', ?_>
    exact (Set.compl_subset_compl.mpr hZZ').trans
      (hsub.trans (succSet_mono (Set.compl_subset_compl.mpr hX'X)))
end

section
theorem toSet_ofSet (S : Set Nat) : toSet (ofSet S) = S := by
  ext n
  simp only [mem_toSet]
  constructor
  * rintro <_, hsub>
    rw [compl_compl] at hsub
    exact hsub (Set.mem_singleton n)
  * intro hn
```

```
    exact <mem_compl_singleton n, by rw [compl_compl]; exact Set.singleton_subset_iff.
    mpr hn>
end

section
/-- **Exercise 2.20 -- the identification `|?|  ~=  (Set Nat,  subseteq )`.** -/
def equivSetNat : powerSet.Element ?o Set Nat where
  toFun := toSet
  invFun := ofSet
  left_inv := ofSet_toSet
  right_inv := toSet_ofSet
  map_rel_iff' := by
    intro x y
    show toSet x  <=  toSet y  <->  x  <=  y
    constructor
    * intro h
      have hmono := ofSet_mono h
      rwa [ofSet_toSet, ofSet_toSet] at hmono
    * intro h n hn
      exact h _ hn
end

section
/-- **Exercise 2.20 -- `x  U  y` is approximable (two variables).** `X, Y f Z  <->  X
    I  Y  subseteq  Z`
(`(X?  U  Y?)? = X  I  Y`). -/
def unionMap : ApproximableMap_2 powerSet powerSet powerSet where
  rel X Y Z := powerSet.mem X  /\  powerSet.mem Y  /\  powerSet.mem Z  /\  X  I  Y
    subseteq  Z
  rel_dom_0 h := h.1
  rel_dom_1 h := h.2.1
  rel_cod h := h.2.2.1
  master_rel := <powerSet.master_mem, powerSet.master_mem, powerSet.master_mem, Set.
    inter_subset_left>
  inter_right := by
    rintro X Y Z Z' <hX, hY, hZ, hsub> <_, _, hZ', hsub'>
    exact <hX, hY, powerSet_inter_mem hZ hZ', Set.subset_inter hsub hsub'>
  mono := by
    rintro X X' Y Y' Z Z' <_, _, _, hsub> hX'X hY'Y hZZ' hX' hY' hZ'
    exact <hX', hY', hZ', (Set.inter_subset_inter hX'X hY'Y).trans (hsub.trans hZZ')>
end

section
theorem toSet_interMap_2 (x y : powerSet.Element) :
    toSet (interMap_2.toElementMap_2 x y) = toSet x  I  toSet y := by
  ext k
  simp only [mem_toSet, ApproximableMap_2.mem_toElementMap_2, Set.mem_inter_iff]
  constructor
  * rintro <X, Y, hxX, hyY, _, _, _, hsub>
    have hkX : k  notin  X := fun h => (hsub (Or.inl h)) rfl
    have hkY : k  notin  Y := fun h => (hsub (Or.inr h)) rfl
    exact <x.up_mem hxX (mem_compl_singleton k) (Set.subset_compl_singleton_iff.mpr
    hkX),
      y.up_mem hyY (mem_compl_singleton k) (Set.subset_compl_singleton_iff.mpr hkY)>
  * rintro <hkx, hky>
    exact <{k}?, {k}?, hkx, hky, mem_compl_singleton k, mem_compl_singleton k,
```

```
      mem_compl_singleton k, Set.union_subset subset_rfl subset_rfl>
end

section
theorem toSet_predMap (x : powerSet.Element) :
    toSet (predMap.toElementMap x) = predSet (toSet x) := by
  ext k
  simp only [mem_toSet, ApproximableMap.mem_toElementMap, mem_predSet]
  constructor
  * rintro <X, hxX, _, _, hsub>
    rw [compl_compl] at hsub
    have hk : k + 1  in  X? := hsub (Set.mem_singleton k)
    exact x.up_mem hxX (mem_compl_singleton (k + 1)) (Set.subset_compl_singleton_iff.
    mpr hk)
  * intro hk
    refine <{k + 1}?, hk, mem_compl_singleton (k + 1), mem_compl_singleton k, ?_>
    rw [compl_compl]
    intro a ha
    rw [Set.mem_singleton_iff] at ha; subst ha
    rw [mem_predSet, compl_compl]
    exact Set.mem_singleton (a + 1)
end

section
theorem toSet_succMap (x : powerSet.Element) :
    toSet (succMap.toElementMap x) = succSet (toSet x) := by
  ext k
  simp only [mem_toSet, ApproximableMap.mem_toElementMap]
  constructor
  * rintro <X, hxX, _, _, hsub>
    rw [compl_compl] at hsub
    obtain <m, hm, rfl> := hsub (Set.mem_singleton k)
    exact <m, x.up_mem hxX (mem_compl_singleton m) (Set.subset_compl_singleton_iff.mpr
     hm), rfl>
  * rintro <m, hm, rfl>
    refine <{m}?, hm, mem_compl_singleton m, mem_compl_singleton (m + 1), ?_>
    rw [compl_compl]
    intro a ha
    rw [Set.mem_singleton_iff] at ha; subst ha
    exact <m, by rw [compl_compl]; exact Set.mem_singleton m, rfl>
end

section
theorem toSet_unionMap (x y : powerSet.Element) :
    toSet (unionMap.toElementMap_2 x y) = toSet x  U  toSet y := by
  ext k
  simp only [mem_toSet, ApproximableMap_2.mem_toElementMap_2, Set.mem_union]
  constructor
  * rintro <X, Y, hxX, hyY, _, _, _, hsub>
    by_cases hkX : k  in  X
    * refine Or.inr ?_
      have hkY : k  notin  Y := fun hkY => (hsub <hkX, hkY>) rfl
      exact y.up_mem hyY (mem_compl_singleton k) (Set.subset_compl_singleton_iff.mpr
    hkY)
    * exact Or.inl (x.up_mem hxX (mem_compl_singleton k) (Set.
    subset_compl_singleton_iff.mpr hkX))
```

```
  * rintro (hkx | hky)
    * exact <{k}?, Set.univ, hkx, y.master_mem, mem_compl_singleton k, powerSet.
    master_mem,
        mem_compl_singleton k, Set.inter_subset_left>
    * exact <Set.univ, {k}?, x.master_mem, hky, powerSet.master_mem,
    mem_compl_singleton k,
        mem_compl_singleton k, Set.inter_subset_right>
end

section
set_option linter.unusedVariables false

theorem example_220 : forall
  (x y :
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element Nat
      DomainTheory.NeighbourhoodSystems.Exercise220.powerSet),
  @Eq (Set Nat)
    (DomainTheory.NeighbourhoodSystems.Exercise220.toSet
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap_2.toElementMap_2 Nat Nat Nat
        DomainTheory.NeighbourhoodSystems.Exercise220.powerSet DomainTheory.
    NeighbourhoodSystems.Exercise220.powerSet
        DomainTheory.NeighbourhoodSystems.Exercise220.powerSet DomainTheory.
    NeighbourhoodSystems.Exercise220.unionMap x
        y))
    (@Union.union (Set Nat) (@Set.instUnion Nat) (DomainTheory.NeighbourhoodSystems.
    Exercise220.toSet x)
      (DomainTheory.NeighbourhoodSystems.Exercise220.toSet y)) :=
  DomainTheory.NeighbourhoodSystems.Exercise220.toSet_unionMap
end

end DomainTheory.NeighbourhoodSystems.Exercise220
```

3.3.24 Exercise 2.21 — finite and infinite sequences

The system $\mathcal{B}$ of 2.3 has as its total elements only the infinite sequences. Modify the construction of $\mathcal{B}$ to another neighbourhood system $\mathcal{C}$ which has *both* the finite and infinite sequences as total elements. (Hint: $\mathcal{B} \subseteq \mathcal{C}$.) Show that there is an approximable map xy on elements naturally extending ordinary juxtaposition of sequences. (Hint: Write 01001 for a *total* finite sequence and $01001\ \bot$ for the corresponding finite partial element. Remember to distinguish between Λ (the total empty sequence) and $\bot$ (the undefined sequence). The definition should work out so that if x is an infinite sequence (hence, total), then $xy = x$ for all y. What will xy equal if x is not total? In other words, the construction possesses a rather strong left-to-right bias.)

Exercise_2_21.lean

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Exercise_2_19
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_8
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Examples_1_2_1_5_revisited
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.DomainIso
@[expose] public section
```

```
section
/- concordance: source L920 EXERCISE 2.21 finite and infinite sequences -/
-- Exercise221/cone_not_subset_singleton.lean
namespace DomainTheory.NeighbourhoodSystems.Exercise221

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ExampleB

/-- A singleton is never a cone (a cone has at least two elements `sigma` and `sigma0`)
    . -/
theorem cone_not_subset_singleton (sigma tau : Str) : not  cone sigma  subseteq  ({tau
    } : Set Str) := by
  intro h
  have h1 : sigma  in  ({tau} : Set Str) := h (mem_cone.mpr List.prefix_rfl)
  have h2 : (sigma ++ [false])  in  ({tau} : Set Str) := h (mem_cone.mpr (List.
    prefix_append sigma [false]))
  rw [Set.mem_singleton_iff] at h1 h2
  rw [ <-  h1] at h2
  have hlen := congrArg List.length h2
  simp only [List.length_append, List.length_cons, List.length_nil] at hlen
  omega

end DomainTheory.NeighbourhoodSystems.Exercise221
end

namespace DomainTheory.NeighbourhoodSystems.Exercise221
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ExampleB

section
/-- Cones and singletons are distinct neighbourhoods. -/
theorem cone_ne_singleton (sigma tau : Str) : cone sigma  !=  ({tau} : Set Str) := by
  intro h
  refine cone_not_subset_singleton sigma tau ?_
  intro x hx
  rw [ <-  h]; exact hx
end

section
/-- Nested-or-disjoint for a cone vs. a terminator singleton: `{tau}  subseteq
    sigmaSigma*` iff `sigma ? tau`, else
disjoint. Choice-free (`sigma <+: tau` is decidable). -/
theorem cone_singleton_nd (sigma tau : Str) :
    cone sigma  subseteq  ({tau} : Set Str)  \/  ({tau} : Set Str)  subseteq  cone
    sigma  \/  cone sigma  I  ({tau} : Set Str) = {} :=
  if h : sigma <+: tau then
    Or.inr (Or.inl (Set.singleton_subset_iff.mpr (mem_cone.mpr h)))
  else
    Or.inr (Or.inr (by
      ext w
      simp only [Set.mem_inter_iff, mem_cone, Set.mem_singleton_iff, Set.
    mem_empty_iff_false,
        iff_false, not_and]
      rintro hw rfl
      exact h hw))
end
```

```
section
/-- Membership in `?`: `X` is either a cone `sigmaSigma*` (as in `?`) or a terminator
    singleton `{sigma}`. -/
def memC (X : Set Str) : Prop := (exists  sigma, X = cone sigma)  \/  (exists  sigma,
    X = {sigma})
end

section
/-- Nested-or-disjoint for a terminator singleton vs. a cone (the mirror of `
    cone_singleton_nd`). -/
theorem singleton_cone_nd (sigma tau : Str) :
    ({sigma} : Set Str)  subseteq  cone tau  \/  cone tau  subseteq  ({sigma} : Set
    Str)  \/  ({sigma} : Set Str)  I  cone tau = {} :=
  if h : tau <+: sigma then
    Or.inl (Set.singleton_subset_iff.mpr (mem_cone.mpr h))
  else
    Or.inr (Or.inr (by
      ext w
      simp only [Set.mem_inter_iff, Set.mem_singleton_iff, mem_cone, Set.
    mem_empty_iff_false,
        iff_false, not_and]
      rintro rfl hw
      exact h hw))
end

section
/-- Nested-or-disjoint for two terminator singletons: equal or disjoint. -/
theorem singleton_singleton_nd (sigma tau : Str) :
    ({sigma} : Set Str)  subseteq  ({tau} : Set Str)  \/  ({tau} : Set Str)  subseteq
     ({sigma} : Set Str)  \/
      ({sigma} : Set Str)  I  ({tau} : Set Str) = {} := by
  by_cases h : sigma = tau
  * subst h; exact Or.inl subset_rfl
  * refine Or.inr (Or.inr ?_)
    ext w
    simp only [Set.mem_inter_iff, Set.mem_singleton_iff, Set.mem_empty_iff_false,
    iff_false,
      not_and]
    rintro rfl h2
    exact h h2
end

section
/-- **`?` is pairwise nested-or-disjoint.** Cone/cone is the `?` trichotomy; the mixed
    and
singleton/singleton cases are the three lemmas above. -/
theorem nestedOrDisjoint_C : NestedOrDisjoint memC := by
  rintro X Y hX hY
  rcases hX with <sigma, rfl> | <sigma, rfl> <;> rcases hY with <tau, rfl> | <tau, rfl
    >
  * exact cone_trichotomy sigma tau
  * exact cone_singleton_nd sigma tau
  * exact singleton_cone_nd sigma tau
  * exact singleton_singleton_nd sigma tau
end
```

```
section
/-- **The system `?`** on `Delta = Sigma*`: cones and terminator singletons. -/
def C : NeighborhoodSystem Str :=
  ofNestedOrDisjoint memC Set.univ <[], Set.mem_univ _>
    (Or.inl <[], cone_nil.symm>) nestedOrDisjoint_C
    (by rintro X (<sigma, rfl> | <sigma, rfl>) <;> exact Set.subset_univ _)
end

section
@[simp] theorem C_mem {X : Set Str} : C.mem X  <->  memC X := Iff.rfl
end

section
@[simp] theorem C_master : C.master = Set.univ := rfl
end

section
/-- Every cone is a neighbourhood of `?` -- this is the inclusion `?  subseteq  ?`. -/
theorem memC_cone (sigma : Str) : C.mem (cone sigma) := Or.inl <sigma, rfl>
end

section
/-- Every terminator singleton `{sigma}` is a neighbourhood of `?`. -/
theorem memC_singleton (sigma : Str) : C.mem ({sigma} : Set Str) := Or.inr <sigma, rfl
    >
end

section
/-- Prepending is monotone in the suffix set. -/
theorem prepend_mono (sigma : Str) {Y Y' : Set Str} (h : Y'  subseteq  Y) : prepend
    sigma Y'  subseteq  prepend sigma Y := by
  rintro w <tau, htau, rfl>
  exact <tau, h htau, rfl>
end

section
/-- `sigma{?} = {sigma?}`: prepending `sigma` to a terminator singleton. -/
theorem prepend_singleton (sigma ? : Str) : prepend sigma ({?} : Set Str) = ({sigma ++
     ?} : Set Str) := by
  ext w
  simp only [mem_prepend, Set.mem_singleton_iff]
  constructor
  * rintro <tau, rfl, rfl>; rfl
  * rintro rfl; exact <?, rfl, rfl>
end

section
/-- Prepending preserves `?`-membership (cone  |->  cone, singleton  |->  singleton).
    -/
theorem memC_prepend (sigma : Str) {Y : Set Str} (hY : C.mem Y) : C.mem (prepend sigma
     Y) := by
  rcases hY with <?, rfl> | <?, rfl>
  * rw [prepend_cone]; exact memC_cone (sigma ++ ?)
  * rw [prepend_singleton]; exact memC_singleton (sigma ++ ?)
end
```

```
section
/-- `sigmaY  subseteq  sigmaSigma* = cone sigma`: prepending `sigma` always lands
    inside the cone of `sigma`. -/
theorem prepend_subset_cone (sigma : Str) (Y : Set Str) : prepend sigma Y  subseteq
    cone sigma := by
  rintro w <tau, _, rfl>
  exact mem_cone.mpr (List.prefix_append sigma tau)
end

section
/-- The **total finite sequence** `sigma` as an element of `|?|`: the principal filter
    `{sigma}`. -/
def singletonElt (sigma : Str) : C.Element := C.principal (memC_singleton sigma)
end

section
/-- `?`, the **total empty sequence** (terminated): `{[]}`. -/
def Lambda : C.Element := singletonElt []
end

section
/-- A nonempty `?`-neighbourhood contained in a singleton *is* that singleton. -/
theorem subset_singleton_eq {W : Set Str} {sigma : Str} (hW : C.mem W) (hsub : W
    subseteq  ({sigma} : Set Str)) :
    W = ({sigma} : Set Str) := by
  rcases hW with <?, rfl> | <?, rfl>
  * exact absurd hsub (cone_not_subset_singleton ? sigma)
  * rw [Set.singleton_subset_iff, Set.mem_singleton_iff] at hsub
    rw [hsub]
end

section
/-- **Juxtaposition `xy`** as an approximable map `?  x  ?  ->  ?`.

The relation `X, Y j Z` reads off the *left-to-right bias*: if the first input
    neighbourhood `X` is
a cone `sigmaSigma*` (`x` extends `sigma` but is undetermined), the output only commits
     to the prefix `sigma`
(`sigmaSigma*  subseteq  Z`), ignoring `Y`; if `X` is a terminator `{sigma}` (`x` is the
     total finite sequence `sigma`), the
output prepends `sigma` to the second input `Y` (`sigmaY  subseteq  Z`). -/
def juxtapose : ApproximableMap_2 C C C where
  rel X Y Z := C.mem Y  /\  C.mem Z  /\
    ((exists  sigma, X = cone sigma  /\  cone sigma  subseteq  Z)  \/  (exists  sigma,
     X = ({sigma} : Set Str)  /\  prepend sigma Y  subseteq  Z))
  rel_dom_0 := by
    rintro X Y Z <_, _, (<sigma, rfl, _> | <sigma, rfl, _>)>
    * exact memC_cone sigma
    * exact memC_singleton sigma
  rel_dom_1 := fun <hY, _, _> => hY
  rel_cod := fun <_, hZ, _> => hZ
  master_rel := <C.master_mem, C.master_mem,
    Or.inl <[], C_master.trans cone_nil.symm, by rw [C_master]; exact Set.subset_univ
    _>>
  inter_right := by
    rintro X Y Z Z' <hY, hZ, hd> <_, hZ', hd'>
```

```
    rcases hd with <sigma, rfl, hsub> | <sigma, rfl, hsub>
    * rcases hd' with <sigma', hX', hsub'> | <sigma', hX', _>
      * obtain rfl : sigma = sigma' := cone_injective hX'
        exact <hY, C.inter_mem hZ hZ' (memC_cone sigma) (Set.subset_inter hsub hsub'),
          Or.inl <sigma, rfl, Set.subset_inter hsub hsub'>>
      * exact absurd hX' (cone_ne_singleton sigma sigma')
    * rcases hd' with <sigma', hX', _> | <sigma', hX', hsub'>
      * exact absurd hX'.symm (cone_ne_singleton sigma' sigma)
      * obtain rfl : sigma = sigma' := by
          have := hX'; rw [Set.singleton_eq_singleton_iff] at this; exact this
        exact <hY, C.inter_mem hZ hZ' (memC_prepend sigma hY) (Set.subset_inter hsub
    hsub'),
          Or.inr <sigma, rfl, Set.subset_inter hsub hsub'>>
  mono := by
    rintro X X' Y Y' Z Z' <hY, _, hd> hX'X hY'Y hZZ' hX' hY' hZ'
    refine <hY', hZ', ?_>
    rcases hd with <sigma, rfl, hsub> | <sigma, rfl, hsub>
    * rcases hX' with <sigma', rfl> | <sigma', rfl>
      * exact Or.inl <sigma', rfl, (hX'X.trans hsub).trans hZZ'>
      * refine Or.inr <sigma', rfl, ?_>
        have hsigmasigma' : sigma <+: sigma' := mem_cone.mp (Set.singleton_subset_iff.
    mp hX'X)
        have h2 : cone sigma'  subseteq  cone sigma := cone_subset_cone.mpr
    hsigmasigma'
        exact ((prepend_subset_cone sigma' Y').trans (h2.trans hsub)).trans hZZ'
    * obtain rfl : X' = ({sigma} : Set Str) := subset_singleton_eq hX' hX'X
      exact Or.inr <sigma, rfl, ((prepend_mono sigma hY'Y).trans hsub).trans hZZ'>
end

section
/-- **`False ? ?`.** The undefined sequence `False = Delta` is strictly below the total
     empty sequence
`? = {[]}`: they are distinct because `False` contains only `Delta`, whereas `?`
    contains `{[]}  !=  Delta`. -/
theorem bot_lt_Lambda : C.bot < Lambda := by
  refine lt_of_le_of_ne (C.bot_le _) ?_
  intro h
  have hmem : C.bot.mem ({([] : Str)} : Set Str) := by
    rw [h]; exact <memC_singleton [], subset_rfl>
  rw [C.mem_bot, C_master] at hmem
  have h2 : ([true] : Str)  in  ({([] : Str)} : Set Str) := by rw [hmem]; trivial
  rw [Set.mem_singleton_iff] at h2
  exact absurd h2 (by decide)
end

section
/-- **Total finite sequences.** `{sigma}` is a total (maximal) element of `|?|`: any
    element above it
must equal it, because `{sigma}` is a minimal nonempty neighbourhood. This is what was
    *false* in `?`,
where finite sequences gave only the partial `sigmaFalse`. -/
theorem isTotal_singletonElt (sigma : Str) : C.IsTotal (singletonElt sigma) := by
  intro y hy Z hZ
  have hsigmay : y.mem ({sigma} : Set Str) := hy {sigma} <memC_singleton sigma,
    subset_rfl>
  have hinter : y.mem (({sigma} : Set Str)  I  Z) := y.inter_mem hsigmay hZ
```

```
  have hmemC : C.mem (({sigma} : Set Str)  I  Z) := y.sub hinter
  have heq : ({sigma} : Set Str)  I  Z = ({sigma} : Set Str) :=
    subset_singleton_eq hmemC Set.inter_subset_left
  exact <y.sub hZ, by rw [ <-  heq]; exact Set.inter_subset_right>
end

section
/-- **Left-to-right bias / "`x` infinite ? `xy = x`".** On a partial input `sigmaFalse
    = (sigmaSigma*)` the
juxtaposition returns `sigmaFalse` itself, *independently of `y`*. Since an infinite (
    total) sequence is the
directed union of its finite approximants `sigmaFalse`, this says `xy = x` for total
    infinite `x`. -/
theorem juxtapose_cone (sigma : Str) (y : C.Element) :
    juxtapose.toElementMap_2 (C.principal (memC_cone sigma)) y = C.principal (
    memC_cone sigma) := by
  apply Element.ext
  intro Z
  constructor
  * rintro <X, Y, <_, hconeX>, _, hY, hZ, hd>
    rcases hd with <?, rfl, hsub> | <?, rfl, _>
    * exact <hZ, hconeX.trans hsub>
    * exact absurd hconeX (cone_not_subset_singleton sigma ?)
  * rintro <hZ, hconesigmaZ>
    exact <cone sigma, C.master, <memC_cone sigma, subset_rfl>, y.master_mem, C.
    master_mem, hZ,
      Or.inl <sigma, rfl, hconesigmaZ>>
end

section
/-- **`sigmay` for a total finite `x = {sigma}`.** When the first argument is the total
     finite sequence
`sigma`, juxtaposition prepends `sigma` to `y`: `Z  in  ({sigma})y` iff `Z` contains `
    sigmaY'` for some `Y'  in  y`. The
coarser cone witnesses `sigmaSigma*  in  {sigma}` add nothing, since `sigmaDelta =
    sigmaSigma*` is itself of the form `sigmaY'`. -/
theorem juxtapose_singleton_mem (sigma : Str) (y : C.Element) {Z : Set Str} :
    (juxtapose.toElementMap_2 (singletonElt sigma) y).mem Z  <->
      C.mem Z  /\  exists  Y', y.mem Y'  /\  prepend sigma Y'  subseteq  Z := by
  constructor
  * rintro <X, Y, <_, hsingleX>, hYmem, _, hZ, hd>
    refine <hZ, ?_>
    rcases hd with <?, rfl, hsub> | <?, rfl, hsub>
    * have h?sigma : ? <+: sigma := mem_cone.mp (Set.singleton_subset_iff.mp hsingleX)
      refine <C.master, y.master_mem, ?_>
      rw [C_master, prepend_univ]
      exact (cone_subset_cone.mpr h?sigma).trans hsub
    * obtain rfl : sigma = ? := by
        have := Set.singleton_subset_iff.mp hsingleX
        rwa [Set.mem_singleton_iff] at this
      exact <Y, hYmem, hsub>
  * rintro <hZ, Y', hY'mem, hsub>
    exact <{sigma}, Y', <memC_singleton sigma, subset_rfl>, hY'mem, y.sub hY'mem, hZ,
      Or.inr <sigma, rfl, hsub>>
end
```

```
section
set_option linter.unusedVariables false

theorem example_221 : forall  (sigma : DomainTheory.NeighbourhoodSystems.ExampleB.Str)
  (y :
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
      DomainTheory.NeighbourhoodSystems.Exercise221.C)
  {Z : Set DomainTheory.NeighbourhoodSystems.ExampleB.Str},
  Iff
    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
      DomainTheory.NeighbourhoodSystems.Exercise221.C
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap_2.toElementMap_2
    DomainTheory.NeighbourhoodSystems.ExampleB.Str
        DomainTheory.NeighbourhoodSystems.ExampleB.Str DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
        DomainTheory.NeighbourhoodSystems.Exercise221.C DomainTheory.
    NeighbourhoodSystems.Exercise221.C
        DomainTheory.NeighbourhoodSystems.Exercise221.C DomainTheory.
    NeighbourhoodSystems.Exercise221.juxtapose
         (DomainTheory.NeighbourhoodSystems.Exercise221.singletonElt sigma) y)
      Z)
    (And
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
        DomainTheory.NeighbourhoodSystems.Exercise221.C Z)
      (exists  (Y' : Set DomainTheory.NeighbourhoodSystems.ExampleB.Str),
        And
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem
            DomainTheory.NeighbourhoodSystems.ExampleB.Str DomainTheory.
    NeighbourhoodSystems.Exercise221.C y Y')
          (@LE.le (Set DomainTheory.NeighbourhoodSystems.ExampleB.Str)
            (@Set.instLE DomainTheory.NeighbourhoodSystems.ExampleB.Str)
            (DomainTheory.NeighbourhoodSystems.ExampleB.prepend sigma Y') Z))) :=
  DomainTheory.NeighbourhoodSystems.Exercise221.juxtapose_singleton_mem
end

section
/-- **`?  subseteq  ?`.** Every neighbourhood of `?` is a neighbourhood of `?`. -/
theorem memB_imp_memC {X : Set Str} (hX : B.mem X) : C.mem X := by
  obtain <sigma, rfl> := hX; exact memC_cone sigma
end

section
/-- Every neighbourhood of `?` is nonempty (there is no empty token). -/
theorem memC_nonempty {W : Set Str} (hW : C.mem W) : W.Nonempty := by
  rcases hW with <sigma, rfl> | <sigma, rfl>
  * exact <sigma, mem_cone.mpr List.prefix_rfl>
  * exact <sigma, rfl>
end

end DomainTheory.NeighbourhoodSystems.Exercise221
```

3.3.25 Exercise 2.22 — domains from closure properties

(For set theorists). We have remarked in Exercise 1.18 and in Exercise 2.11 that any domain $|D|$, as a family of sets (in fact, a family of subsets of the set D itself), is closed under the intersection of an arbitrary non-empty sub family and under the union of any directed sub family. For those familiar with the subject matter, the example of the (proper) ideals of a commutative ring (with unit) is also seen to be such a family. What is the abstract situation? Let $\mathbf{C}$ be *any* family of sets with these closure properties. It is to be shown that $\mathbf{C}$ is inclusion-isomorphic to a domain. (Hint: Let Δ be the set of finite sets included in sets in $\mathbf{C}$. For $F \in \Delta$, define its "closure" by the equation:

$$\overline{F} = \bigcap \{X \in \mathbf{C} \mid F \subseteq X\}.$$

Every $\overline{F} \in \mathbf{C}$, and these will prove to be the "finite" elements of $\mathbf{C}$. The neighbourhood system $\mathcal{D}$ over Δ can be taken to be the sets of the form

$$C(F) = \{G \in \Delta \mid F \subseteq \overline{G}\}$$

for $F \in \Delta$. Notice that for all $X \in \mathbf{C}$

$$X = \bigcup \{\overline{F} \mid F \subseteq X \text{ and } F \in \Delta\}.$$

)

Check that approximable functions on these families are just those preserving directed unions.

LECTURE III

DOMAIN CONSTRUCTS

Having now seen a number of domains presented through their neighbourhood systems, we need next to introduce general constructs for forming new domains from old. There are an unlimited number of such constructs (technically called *functors*), but we have time only to single out a few of the more important ones. Outstanding among all of them is the notion of product of systems, which in our chosen category has all the expected properties. For the time being in order to simplify notation we assume of the underlying sets Δ_0 and Δ_1 of systems $\mathcal{D}_0$ and $\mathcal{D}_1$ that they are disjoint. There is no loss of generality as $\mathcal{D}_1$ can always be replaced by an isomorphic system disjoint from $\mathcal{D}_0$ in the required sense.

Exercise_2_22.lean

```
module

public import Mathlib.Order.Directed

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_1
public import Mathlib.Algebra.Group.End
public import Mathlib.Basic.Finite.Defs
public import Mathlib.Order.SetNotation
public import Mathlib.Order.Hom.Basic
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.Exercise222
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem Set

section
```

```
variable {tau : Type*} (C : Set (Set tau))

/-- The **closure** `F = ? {X  in  C ? F  subseteq  X}` of a set `F` relative to the
    family `C`. -/
def Cl (F : Set tau) : Set tau := ?_0 {X | X  in  C  /\  F  subseteq  X}
end

section
variable {tau : Type*} (C : Set (Set tau))
variable (hInter : forall  ? : Set (Set tau), ?.Nonempty  ->  ?  subseteq  C  ->  ?_0
    ?  in  C)

include hInter in
/-- Every closure `F` of a token lies in `C` (closure under non-empty intersection). -/
theorem Cl_mem {F : Set tau} (h : exists  X  in  C, F  subseteq  X) : Cl C F  in  C :=
     by
  show ?_0 {X | X  in  C  /\  F  subseteq  X}  in  C
  apply hInter
  * obtain <X, hX, hFX> := h
    exact <X, hX, hFX>
  * intro X hX
    exact hX.1
end

section
variable {tau : Type*} (C : Set (Set tau))

/-- `F  subseteq  X` whenever `X  in  C` contains `F`: the closure is the *smallest*
    member of `C` over `F`. -/
theorem Cl_subset {F X : Set tau} (hX : X  in  C) (hFX : F  subseteq  X) : Cl C F
    subseteq  X := by
  intro t ht
  exact (Set.mem_sInter.mp ht) X <hX, hFX>

/-- A token: a finite set included in some member of `C`. -/
class IsTok (F : Set tau) : Prop where
  out : F.Finite  /\  exists  X  in  C, F  subseteq  X

/-- Scott's `Delta`: the type of tokens (finite sets included in members of `C`). -/
abbrev Tok := {F : Set tau // IsTok C F}
end

section
variable {tau : Type*} (C : Set (Set tau))
variable (hInter : forall  ? : Set (Set tau), ?.Nonempty  ->  ?  subseteq  C  ->  ?_0
    ?  in  C)

/-- Scott's neighbourhood `C(F) = {G  in  Delta ? F  subseteq  G}`. -/
def nbhd (F : Set tau) : Set (Tok C) := {G | F  subseteq  Cl C G.1}

@[simp] theorem mem_nbhd {F : Set tau} {G : Tok C} : G  in  nbhd C F  <->  F  subseteq
     Cl C G.1 := Iff.rfl
end

section
variable {tau : Type*} (C : Set (Set tau))
```

```
variable (hInter : forall  ? : Set (Set tau), ?.Nonempty  ->  ?  subseteq  C  ->  ?_0
    ?  in  C)
variable (hne : C.Nonempty)

include hne in
/-- The empty token `{}  in  Delta` (needs `C` non-empty for `{}` to lie in some member
    ). -/
def botTok : Tok C := <{}, <<Set.finite_empty, hne.choose, hne.choose_spec, Set.
    empty_subset _>>>
end

section
variable {tau : Type*} (C : Set (Set tau))
variable (hInter : forall  ? : Set (Set tau), ?.Nonempty  ->  ?  subseteq  C  ->  ?_0
    ?  in  C)

/-- `C({}) = Delta`: the empty token's neighbourhood is everything (the master). -/
theorem nbhd_empty : nbhd C ({} : Set tau) = Set.univ := by
  ext G
  simp [nbhd]
end

section
variable {tau : Type*} (C : Set (Set tau))

/-- `F  subseteq  F`: a set is contained in its closure. -/
theorem subset_Cl (F : Set tau) : F  subseteq  Cl C F := by
  intro t ht
  rw [Cl, Set.mem_sInter]
  intro X hX
  exact hX.2 ht
end

section
variable {tau : Type*} (C : Set (Set tau))
variable (hInter : forall  ? : Set (Set tau), ?.Nonempty  ->  ?  subseteq  C  ->  ?_0
    ?  in  C)

include hInter in
/-- `C(F)  subseteq  C(G)  <->  G  subseteq  F`: comparison of neighbourhoods reduces
    to inclusion against closures. -/
theorem nbhd_subset_iff {F G : Set tau} (hF : IsTok C F) :
    nbhd C F  subseteq  nbhd C G  <->  G  subseteq  Cl C F := by
  constructor
  * intro h
    have hmem : (<F, hF> : Tok C)  in  nbhd C F := subset_Cl C F
    exact h hmem
  * intro hG H hH
    exact hG.trans (Cl_subset C (Cl_mem C hInter H.2.out.2) hH)
end

section
variable {tau : Type*} (C : Set (Set tau))
variable (hInter : forall  ? : Set (Set tau), ?.Nonempty  ->  ?  subseteq  C  ->  ?_0
    ?  in  C)
variable (hne : C.Nonempty)
```

```
include hInter hne in
/-- **The representation neighbourhood system** `reprSystem` over `Delta = Tok C`:
    neighbourhoods are
exactly the `C(F)` for tokens `F`, the master is all of `Delta = C({})`. -/
def reprSystem : NeighborhoodSystem (Tok C) where
  mem N := exists  F : Tok C, N = nbhd C F.1
  master := Set.univ
  master_nonempty := <botTok C hne, Set.mem_univ _>
  master_mem := <botTok C hne, (nbhd_empty C).symm>
  inter_mem := by
    rintro X Y Z <F, rfl> <F', rfl> <F'', rfl> hZsub
    have hF''mem : (F'' : Tok C)  in  nbhd C F''.1 := subset_Cl C F''.1
    have h := hZsub hF''mem
    have hClF'' : Cl C F''.1  in  C := Cl_mem C hInter F''.2.out.2
    have hunionsub : F.1  U  F'.1  subseteq  Cl C F''.1 := Set.union_subset h.1 h.2
    refine <<F.1  U  F'.1, <<F.2.out.1.union F'.2.out.1, Cl C F''.1, hClF'', hunionsub
    >>>, ?_>
    ext G
    simp only [mem_nbhd, Set.mem_inter_iff, Set.union_subset_iff]
  sub_master := fun _ => Set.subset_univ _

@[simp] theorem mem_reprSystem {N : Set (Tok C)} :
    (reprSystem C hInter hne).mem N  <->  exists  F : Tok C, N = nbhd C F.1 := Iff.rfl

@[simp] theorem reprSystem_master : (reprSystem C hInter hne).master = Set.univ := rfl

/-- **Directedness step.** If `C(F), C(F')  in  x` then there is a token `F_3` with `C(
    F_3)  in  x` whose
closure dominates both `F` and `F'`. (From the filter intersection `C(F)  I  C(F')  in
    x`, which is
some `C(F_3)`.) -/
theorem directed_step (x : (reprSystem C hInter hne).Element) {F F' : Tok C}
    (hF : x.mem (nbhd C F.1)) (hF' : x.mem (nbhd C F'.1)) :
    exists  F_3 : Tok C, x.mem (nbhd C F_3.1)  /\  Cl C F.1  subseteq  Cl C F_3.1  /\
    Cl C F'.1  subseteq  Cl C F_3.1 := by
  have hinter : x.mem (nbhd C F.1  I  nbhd C F'.1) := x.inter_mem hF hF'
  have hmemD : (reprSystem C hInter hne).mem (nbhd C F.1  I  nbhd C F'.1) := x.sub
    hinter
  obtain <F_3, hF_3eq> := hmemD
  rw [hF_3eq] at hinter
  have hF_3self : (F_3 : Tok C)  in  nbhd C F_3.1 := subset_Cl C F_3.1
  rw [ <-  hF_3eq] at hF_3self
  refine <F_3, hinter, ?_, ?_>
  * exact Cl_subset C (Cl_mem C hInter F_3.2.out.2) hF_3self.1
  * exact Cl_subset C (Cl_mem C hInter F_3.2.out.2) hF_3self.2

/-- The family `{F ? C(F)  in  x}` of finite-element closures present in the ideal
    element `x`. -/
def famC (x : (reprSystem C hInter hne).Element) : Set (Set tau) :=
  {Y | exists  F : Tok C, x.mem (nbhd C F.1)  /\  Y = Cl C F.1}

/-- The empty token's neighbourhood `C({}) = Delta` is in every element (it is the
    master). -/
theorem mem_nbhd_bot (x : (reprSystem C hInter hne).Element) :
    x.mem (nbhd C (botTok C hne).1) := by
```

```
  have h : (botTok C hne).1 = ({} : Set tau) := rfl
  rw [h, nbhd_empty C]
  exact x.master_mem

/-- The ideal element generated by `X  in  C`: it contains the neighbourhoods above
    some `C(F)` with
`F  subseteq  X`. -/
def ofC (X : Set tau) (hX : X  in  C) : (reprSystem C hInter hne).Element where
  mem N := (reprSystem C hInter hne).mem N  /\  exists  F : Tok C, F.1  subseteq  X
    /\  nbhd C F.1  subseteq  N
  sub h := h.1
  master_mem :=
    <(reprSystem C hInter hne).master_mem, botTok C hne, Set.empty_subset X, Set.
    subset_univ _>
  inter_mem := by
    rintro N M <hN, F, hFX, hFN> <hM, F', hF'X, hF'M>
    have htok : IsTok C (F.1  U  F'.1) :=
      <<F.2.out.1.union F'.2.out.1, X, hX, Set.union_subset hFX hF'X>>
    have hsub : nbhd C (F.1  U  F'.1)  subseteq  N  I  M := by
      intro G hG
      rw [mem_nbhd, Set.union_subset_iff] at hG
      exact <hFN hG.1, hF'M hG.2>
    exact <(reprSystem C hInter hne).inter_mem hN hM <<_, htok>, rfl> hsub,
      <F.1  U  F'.1, htok>, Set.union_subset hFX hF'X, hsub>
  up_mem := by
    rintro N M <_, F, hFX, hFN> hM hNM
    exact <hM, F, hFX, hFN.trans hNM>

/-- Scott's `? {F ? C(F)  in  x}`: the set of `C` represented by the ideal element `x`.
     -/
def toC (x : (reprSystem C hInter hne).Element) : Set tau := ?_0 famC C hInter hne x

theorem mem_toC (x : (reprSystem C hInter hne).Element) {t : tau} :
    t  in  toC C hInter hne x  <->  exists  F : Tok C, x.mem (nbhd C F.1)  /\  t  in
    Cl C F.1 := by
  simp only [toC, famC, Set.mem_sUnion, Set.mem_ofPred_eq]
  constructor
  * rintro <Y, <F, hF, rfl>, ht>
    exact <F, hF, ht>
  * rintro <F, hF, ht>
    exact <Cl C F.1, <F, hF, rfl>, ht>

/-- **Finite sets are captured by a single token.** A finite `s  subseteq  toC x` is
    contained in some `F`
with `C(F)  in  x` -- the finite-subcover property of the directed union `toC x`. -/
theorem exists_tok_of_finite_subset (x : (reprSystem C hInter hne).Element) {s : Set
    tau}
    (hs : s.Finite) (hsub : s  subseteq  toC C hInter hne x) :
    exists  F : Tok C, x.mem (nbhd C F.1)  /\  s  subseteq  Cl C F.1 := by
  revert hsub
  induction s, hs using Set.Finite.induction_on with
  | empty =>
    intro _
    exact <botTok C hne, mem_nbhd_bot C hInter hne x, Set.empty_subset _>
  | @insert a s _ha _hsfin ih =>
    intro hsub
```

```
    have ha' : a  in  toC C hInter hne x := hsub (Set.mem_insert a s)
    rw [mem_toC] at ha'
    obtain <Fa, hFa, haCl> := ha'
    obtain <Fs, hFs, hsCl> := ih (fun y hy => hsub (Set.mem_insert_of_mem a hy))
    obtain <F_3, hF_3, hFaF_3, hFsF_3> := directed_step C hInter hne x hFa hFs
    refine <F_3, hF_3, ?_>
    rw [Set.insert_subset_iff]
    exact <hFaF_3 haCl, hsCl.trans hFsF_3>

/-- **Key membership identity.** `C(G)  in  x  <->  G  subseteq  toC x`. Forward: `G
    subseteq  G  subseteq  toC x`. Reverse: `G` is
finite, so the finite-subcover lemma gives `F` with `C(F)  in  x` and `G  subseteq  F`,
     whence `C(F)  subseteq  C(G)`
and upward closure gives `C(G)  in  x`. -/
theorem mem_nbhd_iff (x : (reprSystem C hInter hne).Element) (G : Tok C) :
    x.mem (nbhd C G.1)  <->  G.1  subseteq  toC C hInter hne x := by
  constructor
  * intro hx t ht
    rw [mem_toC]
    exact <G, hx, subset_Cl C G.1 ht>
  * intro hsub
    obtain <F, hF, hGF> := exists_tok_of_finite_subset C hInter hne x G.2.out.1 hsub
    have hnsub : nbhd C F.1  subseteq  nbhd C G.1 := (nbhd_subset_iff C hInter F.2).
    mpr hGF
    exact x.up_mem hF <G, rfl> hnsub
end

section
variable {tau : Type*} (C : Set (Set tau))
variable (hInter : forall  ? : Set (Set tau), ?.Nonempty  ->  ?  subseteq  C  ->  ?_0
    ?  in  C)
variable (hne : C.Nonempty)
variable (hDir : forall  ? : Set (Set tau), ?.Nonempty  ->  ?  subseteq  C  ->
    DirectedOn (*  subseteq  *) ?  ->  ?_0 ?  in  C)

include hDir in
/-- `toC x  in  C`: the represented set lies in the family (closure under directed
    unions). -/
theorem toC_mem (x : (reprSystem C hInter hne).Element) : toC C hInter hne x  in  C :=
     by
  show ?_0 famC C hInter hne x  in  C
  apply hDir (famC C hInter hne x)
  * exact <Cl C (botTok C hne).1, botTok C hne, mem_nbhd_bot C hInter hne x, rfl>
  * rintro Y <F, _, rfl>
    exact Cl_mem C hInter F.2.out.2
  * rintro Y <F, hF, rfl> Y' <F', hF', rfl>
    obtain <F_3, hF_3, hFF_3, hF'F_3> := directed_step C hInter hne x hF hF'
    exact <Cl C F_3.1, <F_3, hF_3, rfl>, hFF_3, hF'F_3>

/-- The other round-trip `ofC (toC x) = x`. -/
theorem ofC_toC (x : (reprSystem C hInter hne).Element) :
    ofC C hInter hne (toC C hInter hne x) (toC_mem C hInter hne hDir x) = x := by
  apply Element.ext
  intro N
  constructor
  * rintro <hN, F, hFtoC, hFN>
```

```
    have hxF : x.mem (nbhd C F.1) := (mem_nbhd_iff C hInter hne x F).mpr hFtoC
    exact x.up_mem hxF hN hFN
  * intro hxN
    have hN : (reprSystem C hInter hne).mem N := x.sub hxN
    obtain <G, rfl> := hN
    have hGtoC : G.1  subseteq  toC C hInter hne x := (mem_nbhd_iff C hInter hne x G).
    mp hxN
    exact <<G, rfl>, G, hGtoC, subset_rfl>
end

section
set_option linter.unusedVariables false

theorem example_222.{u_1} : forall  {tau : Type u_1} (C : Set (Set tau))
  (hInter :
    forall  (? : Set (Set tau)),
      @Set.Nonempty (Set tau) ?  ->
        @LE.le (Set (Set tau)) (@Set.instLE (Set tau)) ? C  ->
          @Membership.mem (Set tau) (Set (Set tau)) (@Set.instMembership (Set tau)) C
    (@Set.sInter tau ?))
  (hne : @Set.Nonempty (Set tau) C)
  (hDir :
    forall  (? : Set (Set tau)),
      @Set.Nonempty (Set tau) ?  ->
        @LE.le (Set (Set tau)) (@Set.instLE (Set tau)) ? C  ->
          @DirectedOn (Set tau) (fun (x1 x2 : Set tau) => @LE.le (Set tau) (@Set.
    instLE tau) x1 x2) ?  ->
            @Membership.mem (Set tau) (Set (Set tau)) (@Set.instMembership (Set tau))
    C (@Set.sUnion tau ?))
  (x :
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
      (@DomainTheory.NeighbourhoodSystems.Exercise222.Tok tau C)
      (@DomainTheory.NeighbourhoodSystems.Exercise222.reprSystem tau C hInter hne)),
  @Eq
    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
      (@DomainTheory.NeighbourhoodSystems.Exercise222.Tok tau C)
      (@DomainTheory.NeighbourhoodSystems.Exercise222.reprSystem tau C hInter hne))
    (@DomainTheory.NeighbourhoodSystems.Exercise222.ofC tau C hInter hne
      (@DomainTheory.NeighbourhoodSystems.Exercise222.toC tau C hInter hne x)
      (@DomainTheory.NeighbourhoodSystems.Exercise222.toC_mem tau C hInter hne hDir x)
    )
    x :=
  @DomainTheory.NeighbourhoodSystems.Exercise222.ofC_toC
end

section
variable {tau : Type*} (C : Set (Set tau))
variable (hInter : forall  ? : Set (Set tau), ?.Nonempty  ->  ?  subseteq  C  ->  ?_0
    ?  in  C)
variable (hne : C.Nonempty)

/-- **Scott's identity** `X = ? {F ? F  subseteq  X, F  in  Delta}` in round-trip form:
     `toC (ofC X) = X`. -/
theorem toC_ofC (X : Set tau) (hX : X  in  C) : toC C hInter hne (ofC C hInter hne X
    hX) = X := by
  apply Set.Subset.antisymm
```

```
  * intro t ht
    rw [mem_toC] at ht
    obtain <F, hF, htCl> := ht
    obtain <_, F', hF'X, hF'F> := hF
    have h1 : F.1  subseteq  Cl C F'.1 := (nbhd_subset_iff C hInter F'.2).mp hF'F
    have hFX : F.1  subseteq  X := h1.trans (Cl_subset C hX hF'X)
    exact Cl_subset C hX hFX htCl
  * intro t ht
    have hst : IsTok C {t} := <<Set.finite_singleton t, X, hX, Set.
    singleton_subset_iff.mpr ht>>
    rw [mem_toC]
    refine <<{t}, hst>, ?_, subset_Cl C {t} rfl>
    exact <<<{t}, hst>, rfl>, <{t}, hst>, Set.singleton_subset_iff.mpr ht, subset_rfl>
end

section
variable {tau : Type*} (C : Set (Set tau))
variable (hInter : forall  ? : Set (Set tau), ?.Nonempty  ->  ?  subseteq  C  ->  ?_0
    ?  in  C)
variable (hne : C.Nonempty)
variable (hDir : forall  ? : Set (Set tau), ?.Nonempty  ->  ?  subseteq  C  ->
    DirectedOn (*  subseteq  *) ?  ->  ?_0 ?  in  C)

include hDir in
/- concordance: source L922 EXERCISE 2.22 domains from closure properties -/
/-- **Exercise 2.22 -- the representation theorem.** `C`, ordered by inclusion, is
    order-isomorphic
to the domain `|reprSystem|`. The isomorphism sends `x  |->  ? {F ? C(F)  in  x}` (`toC
    `) and
`X  |->  {C(F) ? F  subseteq  X}` (`ofC`). -/
def reprIso : (reprSystem C hInter hne).Element ?o {X : Set tau // X  in  C} where
  toFun x := <toC C hInter hne x, toC_mem C hInter hne hDir x>
  invFun X := ofC C hInter hne X.1 X.2
  left_inv x := ofC_toC C hInter hne hDir x
  right_inv X := Subtype.ext (toC_ofC C hInter hne X.1 X.2)
  map_rel_iff' := by
    intro a b
    show toC C hInter hne a  subseteq  toC C hInter hne b  <->  a  <=  b
    constructor
    * intro hsub N hN
      have hmemD : (reprSystem C hInter hne).mem N := a.sub hN
      obtain <G, rfl> := hmemD
      have hGa : G.1  subseteq  toC C hInter hne a := (mem_nbhd_iff C hInter hne a G).
    mp hN
      exact (mem_nbhd_iff C hInter hne b G).mpr (hGa.trans hsub)
    * intro hab t ht
      rw [mem_toC] at ht  |-
      obtain <F, hF, htCl> := ht
      exact <F, hab _ hF, htCl>
end

end DomainTheory.NeighbourhoodSystems.Exercise222
```

3.4 Domain constructs

Dependency diagram. Each box is one item. The arrowhead is on the item that imports. An arrow already implied by a longer path is left out. The introduction has no Lean file, so it is not shown. Blue is a definition, green a proposition or lemma, amber an example, red a theorem, and indigo an exercise.

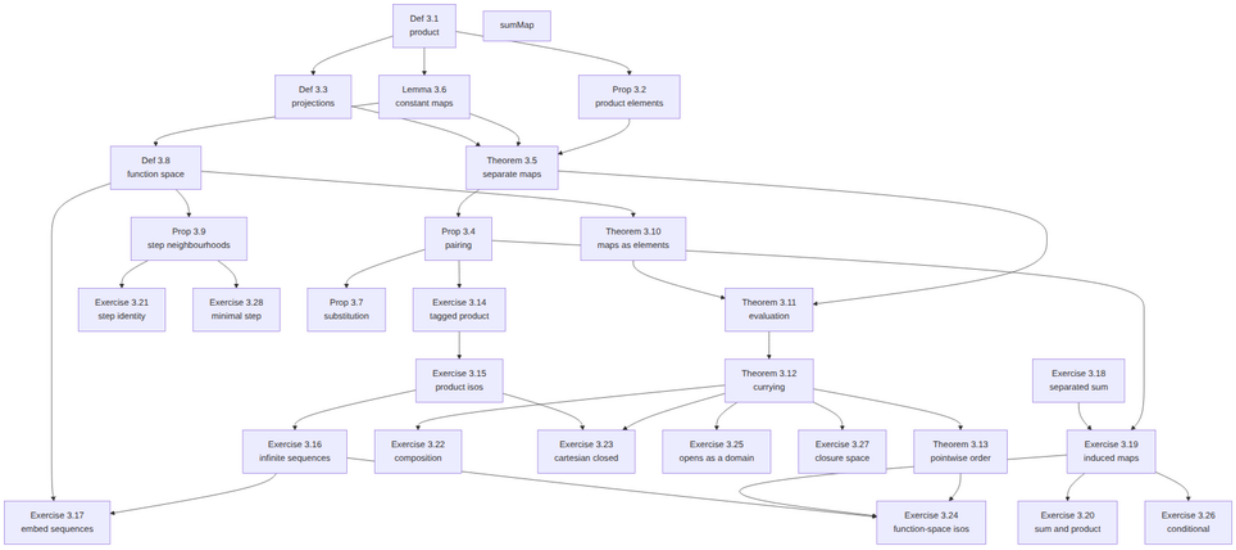


Figure 32: Domain constructs.

3.4.1 Introduction

LECTURE III

DOMAIN CONSTRUCTS

Having now seen a number of domains presented through their neighbourhood systems, we need next to introduce general constructs for forming new domains from old. There are an unlimited number of such constructs (technically called *functors*), but we have time only to single out a few of the more important ones. Outstanding among all of them is the notion of product of systems, which in our chosen category has all the expected properties. For the time being in order to simplify notation we assume of the underlying sets Δ_0 and Δ_1 of systems $\mathcal{D}_0$ and $\mathcal{D}_1$ that they are disjoint. There is no loss of generality as $\mathcal{D}_1$ can always be replaced by an isomorphic system disjoint from $\mathcal{D}_0$ in the required sense.

3.4.2 Definition 3.1 — product of systems

DEFINITION 3.1. Let neighbourhood systems $\mathcal{D}_0$ and $\mathcal{D}_1$ be given over disjoint sets Δ_0 and Δ_1. The *product system* over $\Delta_0 \cup \Delta_1$ is defined by:

$$\mathcal{D}_0 \times \mathcal{D}_1 = \{X \cup Y \mid X \in \mathcal{D}_0 \text{ and } Y \in \mathcal{D}_1\}.$$

For elements $x \in |\mathcal{D}_0|$ and $y \in |\mathcal{D}_1|$ we also define:

$$\langle x, y \rangle = \{X \cup Y \mid X \in x \text{ and } Y \in y\}.$$

□

Definition_3_1.lean

```
module
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_1

public import Mathlib.Data.Set.Operations
@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem
variable {a b ? ? : Type*}

/-- The product neighbourhood `X  U  Y` over the disjoint union `Delta_0  U  Delta_1`,
    modelled on `a ? b` as
`Sum.inl '' X  U  Sum.inr '' Y`. -/
def prodNbhd (X : Set a) (Y : Set b) : Set (a ? b) := Sum.inl '' X  U  Sum.inr '' Y

@[simp] theorem mem_prodNbhd_inl {X : Set a} {Y : Set b} {a : a} :
    (Sum.inl a : a ? b)  in  prodNbhd X Y  <->  a  in  X := by
  simp [prodNbhd]

@[simp] theorem mem_prodNbhd_inr {X : Set a} {Y : Set b} {b : b} :
    (Sum.inr b : a ? b)  in  prodNbhd X Y  <->  b  in  Y := by
  simp [prodNbhd]

@[simp] theorem inl_preimage_prodNbhd (X : Set a) (Y : Set b) :
    Sum.inl ^-1' prodNbhd X Y = X := by ext a; simp

@[simp] theorem inr_preimage_prodNbhd (X : Set a) (Y : Set b) :
    Sum.inr ^-1' prodNbhd X Y = Y := by ext b; simp

/-- The representation `X  U  Y` is unique (choice-free, via the preimage projections).
     -/
theorem prodNbhd_injective {X X' : Set a} {Y Y' : Set b}
    (h : prodNbhd X Y = prodNbhd X' Y') : X = X'  /\  Y = Y' := by
  refine <?_, ?_>
  * rw [ <-  inl_preimage_prodNbhd X Y,  <-  inl_preimage_prodNbhd X' Y', h]
  * rw [ <-  inr_preimage_prodNbhd X Y,  <-  inr_preimage_prodNbhd X' Y', h]

/-- Scott's (2): the product nbhds intersect componentwise. -/
theorem prodNbhd_inter (X X' : Set a) (Y Y' : Set b) :
    prodNbhd X Y  I  prodNbhd X' Y' = prodNbhd (X  I  X') (Y  I  Y') := by
  ext (a | b) <;> simp [Set.mem_inter_iff]

/-- Scott's (1): inclusion of product nbhds is componentwise (uses disjointness of `
    Delta_0`, `Delta_1`). -/
theorem prodNbhd_subset_iff {X X' : Set a} {Y Y' : Set b} :
    prodNbhd X Y  subseteq  prodNbhd X' Y'  <->  X  subseteq  X'  /\  Y  subseteq  Y'
    := by
  constructor
  * intro h
```

```
    refine <fun a ha => ?_, fun b hb => ?_>
    * have : (Sum.inl a : a ? b)  in  prodNbhd X' Y' := h (by simpa using ha)
      simpa using this
    * have : (Sum.inr b : a ? b)  in  prodNbhd X' Y' := h (by simpa using hb)
      simpa using this
  * rintro <hX, hY> (a | b) hs
    * simp only [mem_prodNbhd_inl] at hs  |- ; exact hX hs
    * simp only [mem_prodNbhd_inr] at hs  |- ; exact hY hs

/- concordance: source L944 DEFINITION 3.1 product of systems -/
/-- **Definition 3.1 (Scott 1981, PRG-19).** The *product system* `?_0  x  ?_1`:
    neighbourhoods are
`X  U  Y` with `X  in  ?_0`, `Y  in  ?_1`. Closure under consistent intersection is
    Scott's (2)
(`prodNbhd_inter`) together with the factors' closure; the consistency witness `Z
    subseteq  (X U Y)  I  (X' U Y')`
splits into witnesses `Z_0  subseteq  X  I  X'`, `Z_1  subseteq  Y  I  Y'` by `
    prodNbhd_subset_iff`. -/
def prod (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) :
    NeighborhoodSystem (a ? b) where
  mem W := exists  X Y, V_0.mem X  /\  V_1.mem Y  /\  W = prodNbhd X Y
  master := prodNbhd V_0.master V_1.master
  master_nonempty := by
    obtain <a, ha> := V_0.master_nonempty
    exact <Sum.inl a, mem_prodNbhd_inl.mpr ha>
  master_mem := <V_0.master, V_1.master, V_0.master_mem, V_1.master_mem, rfl>
  inter_mem := by
    rintro W W' Z <X, Y, hX, hY, rfl> <X', Y', hX', hY', rfl> <Z_0, Z_1, hZ_0, hZ_1,
    rfl> hsub
    rw [prodNbhd_inter] at hsub  |-
    obtain <hsub_0, hsub_1> := prodNbhd_subset_iff.mp hsub
    exact <X  I  X', Y  I  Y', V_0.inter_mem hX hX' hZ_0 hsub_0, V_1.inter_mem hY hY'
    hZ_1 hsub_1, rfl>
  sub_master := by
    rintro W <X, Y, hX, hY, rfl>
    exact prodNbhd_subset_iff.mpr <V_0.sub_master hX, V_1.sub_master hY>
end

section
open NeighborhoodSystem
variable {a b ? ? : Type*}
variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}

@[simp] theorem prod_mem_iff {W : Set (a ? b)} :
    (prod V_0 V_1).mem W  <->  exists  X Y, V_0.mem X  /\  V_1.mem Y  /\  W = prodNbhd
     X Y := Iff.rfl

@[simp] theorem prod_master : (prod V_0 V_1).master = prodNbhd V_0.master V_1.master
    := rfl

theorem prod_mem_prodNbhd {X : Set a} {Y : Set b} (hX : V_0.mem X) (hY : V_1.mem Y) :
    (prod V_0 V_1).mem (prodNbhd X Y) := <X, Y, hX, hY, rfl>

theorem prod_mem_prodNbhd_iff {X : Set a} {Y : Set b} :
    (prod V_0 V_1).mem (prodNbhd X Y)  <->  V_0.mem X  /\  V_1.mem Y := by
  constructor
```

```
  * rintro <A, B, hA, hB, heq>
    obtain <rfl, rfl> := prodNbhd_injective heq
    exact <hA, hB>
  * rintro <hX, hY>; exact prod_mem_prodNbhd hX hY

/-- Every product neighbourhood is `(inl^-1 W)  U  (inr^-1 W)`. -/
theorem prodNbhd_preimage {W : Set (a ? b)} (hW : (prod V_0 V_1).mem W) :
    W = prodNbhd (Sum.inl ^-1' W) (Sum.inr ^-1' W) := by
  obtain <X, Y, _, _, rfl> := hW
  rw [inl_preimage_prodNbhd, inr_preimage_prodNbhd]

/-- Scott's `z_0 = {X  in  ?_0 ? X  U  Delta_1  in  z}`: the first component of a
    product element. -/
def NeighborhoodSystem.Element.fst (z : (prod V_0 V_1).Element) : V_0.Element where
  mem X := V_0.mem X  /\  z.mem (prodNbhd X V_1.master)
  sub h := h.1
  master_mem := <V_0.master_mem, z.master_mem>
  inter_mem := by
    rintro X X' <_, hzX> <_, hzX'>
    have hz := z.inter_mem hzX hzX'
    rw [prodNbhd_inter, Set.inter_self] at hz
    obtain <A, B, hA, _, heq> := z.sub hz
    obtain <rfl, rfl> := prodNbhd_injective heq
    exact <hA, hz>
  up_mem := by
    rintro X X' <_, hzX> hX' hXX'
    refine <hX', z.up_mem hzX (prod_mem_prodNbhd hX' V_1.master_mem) ?_>
    exact prodNbhd_subset_iff.mpr <hXX', subset_rfl>

@[simp] theorem mem_fst {z : (prod V_0 V_1).Element} {X : Set a} :
    z.fst.mem X  <->  V_0.mem X  /\  z.mem (prodNbhd X V_1.master) := Iff.rfl

/-- Scott's `z_1 = {Y  in  ?_1 ? Delta_0  U  Y  in  z}`: the second component of a
    product element. -/
def NeighborhoodSystem.Element.snd (z : (prod V_0 V_1).Element) : V_1.Element where
  mem Y := V_1.mem Y  /\  z.mem (prodNbhd V_0.master Y)
  sub h := h.1
  master_mem := <V_1.master_mem, z.master_mem>
  inter_mem := by
    rintro Y Y' <_, hzY> <_, hzY'>
    have hz := z.inter_mem hzY hzY'
    rw [prodNbhd_inter, Set.inter_self] at hz
    obtain <A, B, _, hB, heq> := z.sub hz
    obtain <rfl, rfl> := prodNbhd_injective heq
    exact <hB, hz>
  up_mem := by
    rintro Y Y' <_, hzY> hY' hYY'
    refine <hY', z.up_mem hzY (prod_mem_prodNbhd V_0.master_mem hY') ?_>
    exact prodNbhd_subset_iff.mpr <subset_rfl, hYY'>

@[simp] theorem mem_snd {z : (prod V_0 V_1).Element} {Y : Set b} :
    z.snd.mem Y  <->  V_1.mem Y  /\  z.mem (prodNbhd V_0.master Y) := Iff.rfl

/-- **Definition 3.1 (Scott 1981, PRG-19).** The element pairing `<x, y> = {X  U  Y ? X
     in  x, Y  in  y}`. -/
def pair (x : V_0.Element) (y : V_1.Element) : (prod V_0 V_1).Element where
```

```
  mem W := exists  X Y, x.mem X  /\  y.mem Y  /\  W = prodNbhd X Y
  sub := by rintro W <X, Y, hX, hY, rfl>; exact prod_mem_prodNbhd (x.sub hX) (y.sub hY
    )
  master_mem := <V_0.master, V_1.master, x.master_mem, y.master_mem, rfl>
  inter_mem := by
    rintro W W' <X, Y, hX, hY, rfl> <X', Y', hX', hY', rfl>
    exact <X  I  X', Y  I  Y', x.inter_mem hX hX', y.inter_mem hY hY', prodNbhd_inter
    X X' Y Y'>
  up_mem := by
    rintro W W' <X, Y, hX, hY, rfl> <X', Y', hX', hY', rfl> hsub
    obtain <hXX', hYY'> := prodNbhd_subset_iff.mp hsub
    exact <X', Y', x.up_mem hX hX' hXX', y.up_mem hY hY' hYY', rfl>

@[simp] theorem mem_pair {x : V_0.Element} {y : V_1.Element} {W : Set (a ? b)} :
    (pair x y).mem W  <->  exists  X Y, x.mem X  /\  y.mem Y  /\  W = prodNbhd X Y :=
    Iff.rfl

theorem mem_pair_prodNbhd {x : V_0.Element} {y : V_1.Element} {X : Set a} {Y : Set b}
    :
    (pair x y).mem (prodNbhd X Y)  <->  x.mem X  /\  y.mem Y := by
  constructor
  * rintro <X', Y', hX', hY', heq>
    obtain <rfl, rfl> := prodNbhd_injective heq
    exact <hX', hY'>
  * rintro <hx, hy>; exact <X, Y, hx, hy, rfl>

@[simp] theorem fst_pair (x : V_0.Element) (y : V_1.Element) : (pair x y).fst = x :=
    by
  apply Element.ext
  intro X
  constructor
  * rintro <hX, hmem>
    exact (mem_pair_prodNbhd.mp hmem).1
  * intro hX
    exact <x.sub hX, mem_pair_prodNbhd.mpr <hX, y.master_mem>>

/-- The key splitting (Scott's (3)): for a product element `z` and neighbourhoods `X
    in  ?_0`,
`Y  in  ?_1`, membership of `X  U  Y` in `z` is equivalent to membership of its two "
    slices". -/
theorem prod_mem_split {z : (prod V_0 V_1).Element} {X : Set a} {Y : Set b}
    (hX : V_0.mem X) (hY : V_1.mem Y) :
    z.mem (prodNbhd X Y)  <->  z.mem (prodNbhd X V_1.master)  /\  z.mem (prodNbhd V_0.
    master Y) := by
  constructor
  * intro h
    refine <z.up_mem h (prod_mem_prodNbhd hX V_1.master_mem) ?_,
            z.up_mem h (prod_mem_prodNbhd V_0.master_mem hY) ?_>
    * exact prodNbhd_subset_iff.mpr <subset_rfl, V_1.sub_master hY>
    * exact prodNbhd_subset_iff.mpr <V_0.sub_master hX, subset_rfl>
  * rintro <h1, h2>
    have := z.inter_mem h1 h2
    rwa [prodNbhd_inter, Set.inter_eq_left.mpr (V_0.sub_master hX),
      Set.inter_eq_right.mpr (V_1.sub_master hY)] at this

/-- `z = <z_0, z_1>`: every product element is the pairing of its two components. -/
```

```
theorem pair_fst_snd (z : (prod V_0 V_1).Element) : pair z.fst z.snd = z := by
  apply Element.ext
  intro W
  constructor
  * rintro <X, Y, <hX, hzX>, <hY, hzY>, rfl>
    exact (prod_mem_split hX hY).mpr <hzX, hzY>
  * intro hW
    obtain <X, Y, hX, hY, rfl> := z.sub hW
    obtain <h1, h2> := (prod_mem_split hX hY).mp hW
    exact <X, Y, <hX, h1>, <hY, h2>, rfl>

@[simp] theorem snd_pair (x : V_0.Element) (y : V_1.Element) : (pair x y).snd = y :=
    by
  apply Element.ext
  intro Y
  constructor
  * rintro <hY, hmem>
    exact (mem_pair_prodNbhd.mp hmem).2
  * intro hY
    exact <y.sub hY, mem_pair_prodNbhd.mpr <x.master_mem, hY>>

theorem prod_mem_inl {W : Set (a ? b)} (hW : (prod V_0 V_1).mem W) : V_0.mem (Sum.inl
    ^-1' W) := by
  obtain <X, Y, hX, _, rfl> := hW; rwa [inl_preimage_prodNbhd]

theorem prod_mem_inr {W : Set (a ? b)} (hW : (prod V_0 V_1).mem W) : V_1.mem (Sum.inr
    ^-1' W) := by
  obtain <X, Y, _, hY, rfl> := hW; rwa [inr_preimage_prodNbhd]

end
end DomainTheory.NeighbourhoodSystems
end
```

3.4.3 Proposition 3.2 — elements of a product

PROPOSITION 3.2. The construct $\mathcal{D}_0 \times \mathcal{D}_1$ always gives a neighbourhood system where for elements $x, x' \in |\mathcal{D}_0|$ and $y, y' \in |\mathcal{D}_1|$ we have

(i) $\langle x, y\rangle \subseteq \langle x', y'\rangle$ iff $x \subseteq x'$ and $y \subseteq y'$.

Moreover, there is a one-one correspondence between the elements of $|\mathcal{D}_0 \times \mathcal{D}_1|$ and pairs of elements of $|\mathcal{D}_0|$ and $|\mathcal{D}_1|$ since all elements of $|\mathcal{D}_0 \times \mathcal{D}_1|$ are of the form $\langle x, y\rangle$.

Proof: Owing to the disjointness of Δ_0 and Δ_1, we note that for $X, X' \in \mathcal{D}_0$ and $Y, Y' \in \mathcal{D}_1$ we have

$$(1)\ X \cup Y \subseteq X' \cup Y' \text{ iff } X \subseteq X' \text{ and } Y \subseteq Y'.$$

Thus, $\{X \cup Y, X' \cup Y'\}$ is consistent in $\mathcal{D}_0 \times \mathcal{D}_1$ iff $\{X, X'\}$ is consistent in $\mathcal{D}_0$ and $\{Y, Y'\}$ is consistent in $\mathcal{D}_1$. In the consistent case we find

$$(2) \qquad (X \cup Y) \cap (X' \cup Y') = (X \cap X') \cup (Y \cap Y'),$$

and so $\mathcal{D}_0 \times \mathcal{D}_1$ is closed under consistent intersection. As $\Delta_0 \cup \Delta_1 \in \mathcal{D}_0 \times \mathcal{D}_1$, it is certainly a neighbourhood system.

It is easy to check by the previous calculations that $\langle x, y\rangle \in |\mathcal{D}_0 \times \mathcal{D}_1|$ if $x \in |\mathcal{D}_0|$ and $y \in |\mathcal{D}_1|$. The proof of 3.2(i) follows directly from the definition and (1).

Suppose $z \in |\mathcal{D}_0 \times \mathcal{D}_1|$. Define as a temporary notation:

$$z_0 = \{X \in \mathcal{D}_0 \mid X \cup \Delta_1 \in z\}, \quad \text{and} \quad z_1 = \{Y \in \mathcal{D}_1 \mid \Delta_0 \cup Y \in z\}.$$

Clearly, both $z_0 \in |\mathcal{D}_0|$ and $z_1 \in |\mathcal{D}_1|$. In view of the formula

$$(3) \qquad (X \cup \Delta_1) \cap (\Delta_0 \cup Y) = X \cup Y,$$

we can calculate that

$$z = \langle z_0, z_1\rangle.$$

Moreover, if $z = \langle x, y\rangle$, then $\langle x, y\rangle_0 = x$ and $\langle x, y\rangle_1 = y$.

The one-one correspondence required is thus established. □

There is more going on in the proof of 3.2 than just a one-one correspondence between elements and pairs. The extra information is best formalized by introducing a notation for mappings.

Proposition_3_2.lean

```
module
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Definition_3_1

public import Mathlib.Algebra.Group.End
public import Mathlib.Order.Hom.Basic
@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem
variable {a b ? ? : Type*}
variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
/- concordance: source L958 PROPOSITION 3.2 elements of a product -/
/-- **Proposition 3.2 (Scott 1981, PRG-19).** The order-isomorphism `|?_0  x  ?_1| ?o
    |?_0|  x  |?_1|`. -/
def prodEquiv (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) :
    (prod V_0 V_1).Element ?o V_0.Element  x  V_1.Element where
  toFun z := (z.fst, z.snd)
  invFun p := pair p.1 p.2
  left_inv z := pair_fst_snd z
  right_inv p := by simp
  map_rel_iff' := by
    intro z z'
    constructor
    * rintro <h1, h2> W hW
      obtain <X, Y, hX, hY, rfl> := z.sub hW
      obtain <ha, hb> := (prod_mem_split hX hY).mp hW
      have hX' : z'.fst.mem X := h1 X <hX, ha>
      have hY' : z'.snd.mem Y := h2 Y <hY, hb>
      exact (prod_mem_split hX hY).mpr <hX'.2, hY'.2>
    * intro h
      exact <fun X <hX, hzX> => <hX, h _ hzX>, fun Y <hY, hzY> => <hY, h _ hzY>>
```

```
@[simp] theorem prodEquiv_apply (z : (prod V_0 V_1).Element) :
    prodEquiv V_0 V_1 z = (z.fst, z.snd) := rfl

@[simp] theorem prodEquiv_symm_apply (p : V_0.Element  x  V_1.Element) :
    (prodEquiv V_0 V_1).symm p = pair p.1 p.2 := rfl

/-- **Proposition 3.2(i) (Scott 1981, PRG-19).** `<x, y>  sqsub  <x', y'>  <->  x
    sqsub  x'  /\  y  sqsub  y'`. -/
theorem pair_le_pair_iff {x x' : V_0.Element} {y y' : V_1.Element} :
    pair x y  <=  pair x' y'  <->  x  <=  x'  /\  y  <=  y' := by
  constructor
  * intro h
    refine <fun X hX => ?_, fun Y hY => ?_>
    * obtain <X', Y', hX', hY', heq> :=
        h (prodNbhd X V_1.master) <X, V_1.master, hX, y.master_mem, rfl>
      obtain <rfl, _> := prodNbhd_injective heq
      exact hX'
    * obtain <X', Y', hX', hY', heq> :=
        h (prodNbhd V_0.master Y) <V_0.master, Y, x.master_mem, hY, rfl>
      obtain <_, rfl> := prodNbhd_injective heq
      exact hY'
  * rintro <hx, hy> W <X, Y, hX, hY, rfl>
    exact <X, Y, hx X hX, hy Y hY, rfl>
end
end DomainTheory.NeighbourhoodSystems
end
```

3.4.4 Definition 3.3 — projection mappings

DEFINITION 3.3. *Projection mappings*

$$p_0 : \mathcal{D}_0 \times \mathcal{D}_1 \to \mathcal{D}_0 \quad \text{and} \quad p_1 : \mathcal{D}_0 \times \mathcal{D}_1 \to \mathcal{D}_1$$

are defined as relations where

$$(X \cup Y)\ p_0\ X' \text{ iff } X \subseteq X', \quad \text{and} \quad (X \cup Y)\ p_1\ Y' \text{ iff } Y \subseteq Y'$$

hold for all $X, X' \in \mathcal{D}_0$ and $Y, Y' \in \mathcal{D}_1$. Given $f : \mathcal{D}_2 \to \mathcal{D}_0$ and $g : \mathcal{D}_2 \to \mathcal{D}_1$, the *paired mapping*

$$\langle f, g \rangle : \mathcal{D}_2 \to \mathcal{D}_0 \times \mathcal{D}_1$$

is defined as a relation where

$$Z\ \langle f, g \rangle\ (X \cup Y) \text{ iff } Z\ f\ X \text{ and } Z\ g\ Y$$

holds for all $X \in \mathcal{D}_0$, $Y \in \mathcal{D}_1$, and $Z \in \mathcal{D}_2$. □

Definition_3_3.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Definition_3_1
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
```

```
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_2
@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem
variable {a b ? ? : Type*}
variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
variable {V_2 : NeighborhoodSystem ?}
open ApproximableMap
/- concordance: source L1008 DEFINITION 3.3 projection mappings -/
/-- **Definition 3.3 (Scott 1981, PRG-19).** The paired mapping `<f, g> : ?_2  ->  ?_0
     x  ?_1`,
`Z <f, g> (X  U  Y)  <->  Z f X  /\  Z g Y`. -/
def paired (f : ApproximableMap V_2 V_0) (g : ApproximableMap V_2 V_1) :
    ApproximableMap V_2 (prod V_0 V_1) where
  rel Z P := (prod V_0 V_1).mem P  /\  f.rel Z (Sum.inl ^-1' P)  /\  g.rel Z (Sum.inr
    ^-1' P)
  rel_dom h := f.rel_dom h.2.1
  rel_cod h := h.1
  master_rel := by
    refine <(prod V_0 V_1).master_mem, ?_, ?_>
    * simpa using f.master_rel
    * simpa using g.master_rel
  inter_right := by
    rintro Z P P' <hP, hfP, hgP> <hP', hfP', hgP'>
    have hfX := f.inter_right hfP hfP'
    have hgY := g.inter_right hgP hgP'
    refine <?_, ?_, ?_>
    * rw [prodNbhd_preimage hP, prodNbhd_preimage hP', prodNbhd_inter]
      exact prod_mem_prodNbhd (f.rel_cod hfX) (g.rel_cod hgY)
    * rw [Set.preimage_inter]; exact hfX
    * rw [Set.preimage_inter]; exact hgY
  mono := by
    rintro Z Z_2 P P_2 <_, hfP, hgP> hZ_2Z hPP_2 hZ_2 hP_2
    obtain <A, B, hA, hB, rfl> := hP_2
    have hinl : Sum.inl ^-1' P  subseteq  A := by
      have := Set.preimage_mono (f := Sum.inl) hPP_2; rwa [inl_preimage_prodNbhd] at
    this
    have hinr : Sum.inr ^-1' P  subseteq  B := by
      have := Set.preimage_mono (f := Sum.inr) hPP_2; rwa [inr_preimage_prodNbhd] at
    this
    refine <<A, B, hA, hB, rfl>, ?_, ?_>
    * rw [inl_preimage_prodNbhd]; exact f.mono hfP hZ_2Z hinl hZ_2 hA
    * rw [inr_preimage_prodNbhd]; exact g.mono hgP hZ_2Z hinr hZ_2 hB

/-- **Definition 3.3 (Scott 1981, PRG-19).** The projection `p_0 : ?_0  x  ?_1  ->  ?_0
    `,
`(X  U  Y) p_0 X'  <->  X  subseteq  X'`. -/
def proj_0 (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) :
    ApproximableMap (prod V_0 V_1) V_0 where
  rel W X' := (prod V_0 V_1).mem W  /\  V_0.mem X'  /\  Sum.inl ^-1' W  subseteq  X'
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := <(prod V_0 V_1).master_mem, V_0.master_mem, by simp>
```

```
  inter_right := by
    rintro W X' X'' <hW, hX', hsub> <_, hX'', hsub'>
    obtain <A, B, hA, _, rfl> := hW
    rw [inl_preimage_prodNbhd] at hsub hsub'  |-
    exact <<A, B, hA, ?_?, rfl>, V_0.inter_mem hX' hX'' hA (Set.subset_inter hsub hsub
    '),
      Set.subset_inter hsub hsub'>
  mono := by
    rintro W W_2 X' X_2' <_, _, hsub> hW_2W hX'X_2' hW_2 hX_2'
    exact <hW_2, hX_2', ((Set.preimage_mono hW_2W).trans hsub).trans hX'X_2'>

@[simp] theorem proj_0_rel {W : Set (a ? b)} {X' : Set a} :
    (proj_0 V_0 V_1).rel W X'  <->  (prod V_0 V_1).mem W  /\  V_0.mem X'  /\  Sum.inl
    ^-1' W  subseteq  X' := Iff.rfl

/-- **Definition 3.3 (Scott 1981, PRG-19).** The projection `p_1 : ?_0  x  ?_1  ->  ?_1
    `,
`(X  U  Y) p_1 Y'  <->  Y  subseteq  Y'`. -/
def proj_1 (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) :
    ApproximableMap (prod V_0 V_1) V_1 where
  rel W Y' := (prod V_0 V_1).mem W  /\  V_1.mem Y'  /\  Sum.inr ^-1' W  subseteq  Y'
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := <(prod V_0 V_1).master_mem, V_1.master_mem, by simp>
  inter_right := by
    rintro W Y' Y'' <hW, hY', hsub> <_, hY'', hsub'>
    obtain <A, B, _, hB, rfl> := hW
    rw [inr_preimage_prodNbhd] at hsub hsub'  |-
    exact <<A, B, ?_?, hB, rfl>, V_1.inter_mem hY' hY'' hB (Set.subset_inter hsub hsub
    '),
      Set.subset_inter hsub hsub'>
  mono := by
    rintro W W_2 Y' Y_2' <_, _, hsub> hW_2W hY'Y_2' hW_2 hY_2'
    exact <hW_2, hY_2', ((Set.preimage_mono hW_2W).trans hsub).trans hY'Y_2'>

@[simp] theorem proj_1_rel {W : Set (a ? b)} {Y' : Set b} :
    (proj_1 V_0 V_1).rel W Y'  <->  (prod V_0 V_1).mem W  /\  V_1.mem Y'  /\  Sum.inr
    ^-1' W  subseteq  Y' := Iff.rfl

@[simp] theorem paired_rel {f : ApproximableMap V_2 V_0} {g : ApproximableMap V_2 V_1}
    {Z : Set ?} {P : Set (a ? b)} :
    (paired f g).rel Z P  <->
      (prod V_0 V_1).mem P  /\  f.rel Z (Sum.inl ^-1' P)  /\  g.rel Z (Sum.inr ^-1' P)
     := Iff.rfl

@[simp] theorem toElementMap_proj_0 (z : (prod V_0 V_1).Element) :
    (proj_0 V_0 V_1).toElementMap z = z.fst := by
  apply Element.ext
  intro X'
  constructor
  * rintro <W, hzW, hW, hX', hsub>
    obtain <A, B, _, hB, rfl> := hW
    rw [inl_preimage_prodNbhd] at hsub
    refine <hX', z.up_mem hzW (prod_mem_prodNbhd hX' V_1.master_mem) ?_>
    exact prodNbhd_subset_iff.mpr <hsub, V_1.sub_master hB>
  * rintro <hX', hz>
```

```
    exact <prodNbhd X' V_1.master, hz, prod_mem_prodNbhd hX' V_1.master_mem, hX', by
    simp>

@[simp] theorem toElementMap_proj_1 (z : (prod V_0 V_1).Element) :
    (proj_1 V_0 V_1).toElementMap z = z.snd := by
  apply Element.ext
  intro Y'
  constructor
  * rintro <W, hzW, hW, hY', hsub>
    obtain <A, B, hA, _, rfl> := hW
    rw [inr_preimage_prodNbhd] at hsub
    refine <hY', z.up_mem hzW (prod_mem_prodNbhd V_0.master_mem hY') ?_>
    exact prodNbhd_subset_iff.mpr <V_0.sub_master hA, hsub>
  * rintro <hY', hz>
    exact <prodNbhd V_0.master Y', hz, prod_mem_prodNbhd V_0.master_mem hY', hY', by
    simp>

theorem toElementMap_proj_0_pair (x : V_0.Element) (y : V_1.Element) :
    (proj_0 V_0 V_1).toElementMap (pair x y) = x := by
  rw [toElementMap_proj_0, fst_pair]

theorem toElementMap_proj_1_pair (x : V_0.Element) (y : V_1.Element) :
    (proj_1 V_0 V_1).toElementMap (pair x y) = y := by
  rw [toElementMap_proj_1, snd_pair]

/-- Elementwise action of pairing: `<f, g>(w) = <f(w), g(w)>`. -/
theorem toElementMap_paired (f : ApproximableMap V_2 V_0) (g : ApproximableMap V_2 V_1
    )
    (w : V_2.Element) :
    (paired f g).toElementMap w = pair (f.toElementMap w) (g.toElementMap w) := by
  apply Element.ext
  intro P
  constructor
  * rintro <Z, hwZ, hP, hfZ, hgZ>
    exact <Sum.inl ^-1' P, Sum.inr ^-1' P, <Z, hwZ, hfZ>, <Z, hwZ, hgZ>,
    prodNbhd_preimage hP>
  * rintro <X, Y, <Z_1, hwZ_1, hfZ_1>, <Z_2, hwZ_2, hgZ_2>, rfl>
    refine <Z_1  I  Z_2, w.inter_mem hwZ_1 hwZ_2, prod_mem_prodNbhd (f.rel_cod hfZ_1)
    (g.rel_cod hgZ_2),
      ?_, ?_>
    * rw [inl_preimage_prodNbhd]
      exact f.mono hfZ_1 Set.inter_subset_left subset_rfl (w.sub (w.inter_mem hwZ_1
    hwZ_2))
        (f.rel_cod hfZ_1)
    * rw [inr_preimage_prodNbhd]
      exact g.mono hgZ_2 Set.inter_subset_right subset_rfl (w.sub (w.inter_mem hwZ_1
    hwZ_2))
        (g.rel_cod hgZ_2)

/-- Every map into a product is the pairing of its projections: `h = <p_0  comp  h, p_1
      comp  h>`. -/
theorem paired_proj (h : ApproximableMap V_2 (prod V_0 V_1)) :
    paired ((proj_0 V_0 V_1).comp h) ((proj_1 V_0 V_1).comp h) = h := by
  apply ext_of_toElementMap
  intro w
  rw [toElementMap_paired, toElementMap_comp, toElementMap_comp, toElementMap_proj_0,
```

```
    toElementMap_proj_1, pair_fst_snd]

end
end DomainTheory.NeighbourhoodSystems
end
```

3.4.5 Proposition 3.4 — pairing and projections

PROPOSITION 3.4. The mappings p_0, p_1 and $\langle f, g\rangle$ are approximable mappings, provided f and g are, and we have:

(i) $p_0 \circ \langle f, g\rangle = f$ and $p_1 \circ \langle f, g\rangle = g$.

Moreover, for $z \in |\mathcal{D}_0 \times \mathcal{D}_1|$, we have:

(ii) $p_0(z) = z_0$ and $p_1(z) = z_1$,

in the notation of the proof of 3.2. Further if $h : \mathcal{D}_2 \to \mathcal{D}_0 \times \mathcal{D}_1$ is any approximable mapping, then

(iii) $h = \langle p_0 \circ h, p_1 \circ h\rangle$.

Moreover, for all $w \in |\mathcal{D}_2|$, we have:

(iv) $\langle f, g\rangle(w) = \langle f(w), g(w)\rangle$,

where again on the right-hand side the notation of the proof of 3.2 is used. □

The proof of this result is left as an exercise. Note the consequence that there is a one-one correspondence between pairs of approximable mappings $f : \mathcal{D}_2 \to \mathcal{D}_0$ and $g : \mathcal{D}_2 \to \mathcal{D}_1$ and mappings $h : \mathcal{D}_2 \to \mathcal{D}_0 \times \mathcal{D}_1$. It is clear that we generalize all this to products

$$\mathcal{D}_0 \times \mathcal{D}_1 \times \cdots \times \mathcal{D}_{n-1}$$

of several systems.

The product construct also neatly explains functions of several variables. In Exercise 2.19 we used the informal notation

$$f : \mathcal{D}_0 \times \mathcal{D}_1 \to \mathcal{D}_2$$

and suggested regarding f as a ternary relation

$$X, Y\ f\ Z.$$

But now with $\mathcal{D}_0 \times \mathcal{D}_1$ given an independent meaning, all we have to do is to regard f as a binary relation with

$$(X, Y)\ f\ Z$$

equivalent to the old relationship. We can also employ an element-wise notation as in $f(\langle x, y\rangle)$, which can more easily be written $f(x, y)$. Similar remarks apply to functions of more than two arguments.

We have discussed several times what it means for a function $f(x)$ to come from an approximable mapping. It is interesting to ask the analogous question for functions of several arguments.

Proposition_3_4.lean

```
module
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_5

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem
variable {a b ? ? : Type*}
variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
variable {V_2 : NeighborhoodSystem ?}
open ApproximableMap
/- concordance: source L1036 PROPOSITION 3.4 pairing and projections -/
/-- **Proposition 3.4(i) (Scott 1981, PRG-19).** `p_0  comp  <f, g> = f`. -/
theorem proj_0_comp_paired (f : ApproximableMap V_2 V_0) (g : ApproximableMap V_2 V_1)
     :
    (proj_0 V_0 V_1).comp (paired f g) = f := by
  apply ext_of_toElementMap
  intro w
  rw [toElementMap_comp, toElementMap_paired, toElementMap_proj_0, fst_pair]

/-- **Proposition 3.4(i) (Scott 1981, PRG-19).** `p_1  comp  <f, g> = g`. -/
theorem proj_1_comp_paired (f : ApproximableMap V_2 V_0) (g : ApproximableMap V_2 V_1)
     :
    (proj_1 V_0 V_1).comp (paired f g) = g := by
  apply ext_of_toElementMap
  intro w
  rw [toElementMap_comp, toElementMap_paired, toElementMap_proj_1, snd_pair]

/-- **Proposition 3.4 (Scott 1981, PRG-19).** Pairing and projections: `p_0  comp  <f,
    g> = f`,
`p_1  comp  <f, g> = g`, `h = <p_0  comp  h, p_1  comp  h>`, and `<f, g>(w) = <f(w), g(
    w)>`. -/
theorem proposition_3_4 :
    (forall  (f : ApproximableMap V_2 V_0) (g : ApproximableMap V_2 V_1),
      (proj_0 V_0 V_1).comp (paired f g) = f  /\  (proj_1 V_0 V_1).comp (paired f g) =
     g)  /\
    (forall  (h : ApproximableMap V_2 (prod V_0 V_1)),
      paired ((proj_0 V_0 V_1).comp h) ((proj_1 V_0 V_1).comp h) = h)  /\
    (forall  (f : ApproximableMap V_2 V_0) (g : ApproximableMap V_2 V_1) (w : V_2.
    Element),
      (paired f g).toElementMap w = pair (f.toElementMap w) (g.toElementMap w)) :=
  <fun f g => <proj_0_comp_paired f g, proj_1_comp_paired f g>, paired_proj,
    toElementMap_paired>

end
end DomainTheory.NeighbourhoodSystems
end
```

3.4.6 Theorem 3.5 — separate approximability

THEOREM 3.5. An elementwise function

$$f : |D_0 \times D_1| \to |D_2|$$

of two arguments comes from an approximable mapping iff for each fixed $a \in |D_0|$ and each fixed $b \in |D_1|$ the transformations

$$x \mapsto f(x, b) \quad \text{and} \quad y \mapsto f(a, y)$$

come from approximable mappings of one argument.

Proof: As this is the first time we have had to deal with constants in functions, a lemma is useful.

Theorem_3_5.lean

```
module
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Exercise_2_19

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Lemma_3_6
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Definition_3_3
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_2
@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem
variable {a b ? ? : Type*}
variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
variable {V_2 : NeighborhoodSystem ?}
open ApproximableMap
/- concordance: source L1086 THEOREM 3.5 separate approximability -/
/-- **Theorem 3.5 ( <- ) (Scott 1981, PRG-19).** A two-variable mapping induces a
    joint mapping. -/
def ofMap_2 (f : ApproximableMap_2 V_0 V_1 V_2) : ApproximableMap (prod V_0 V_1) V_2
    where
  rel W Z := (prod V_0 V_1).mem W  /\  f.rel (Sum.inl ^-1' W) (Sum.inr ^-1' W) Z
  rel_dom h := h.1
  rel_cod h := f.rel_cod h.2
  master_rel := by
    refine <(prod V_0 V_1).master_mem, ?_>
    simpa using f.master_rel
  inter_right := by rintro W Z Z' <hW, hrel> <_, hrel'>; exact <hW, f.inter_right hrel
     hrel'>
  mono := by
    rintro W W_2 Z Z' <_, hrel> hW_2W hZZ' hW_2 hZ'
    have hinl : Sum.inl ^-1' W_2  subseteq  Sum.inl ^-1' W := Set.preimage_mono hW_2W
    have hinr : Sum.inr ^-1' W_2  subseteq  Sum.inr ^-1' W := Set.preimage_mono hW_2W
    obtain <A, B, hA, hB, rfl> := hW_2
    rw [inl_preimage_prodNbhd] at hinl
    rw [inr_preimage_prodNbhd] at hinr
    refine <<A, B, hA, hB, rfl>, ?_>
    rw [inl_preimage_prodNbhd, inr_preimage_prodNbhd]
    exact f.mono hrel hinl hinr hZZ' hA hB hZ'
```

```
/-- **Theorem 3.5 ( -> ) (Scott 1981, PRG-19).** A joint approximable mapping `?_0  x
    ?_1  ->  ?_2`
restricts to a two-variable mapping `X, Y f Z  <->  (X  U  Y) f Z`. -/
def toMap_2 (f : ApproximableMap (prod V_0 V_1) V_2) : ApproximableMap_2 V_0 V_1 V_2
    where
  rel X Y Z := f.rel (prodNbhd X Y) Z
  rel_dom_0 h := (prod_mem_prodNbhd_iff.mp (f.rel_dom h)).1
  rel_dom_1 h := (prod_mem_prodNbhd_iff.mp (f.rel_dom h)).2
  rel_cod h := f.rel_cod h
  master_rel := f.master_rel
  inter_right h h' := f.inter_right h h'
  mono := by
    rintro X X' Y Y' Z Z' hrel hX'X hY'Y hZZ' hX' hY' hZ'
    exact f.mono hrel (prodNbhd_subset_iff.mpr <hX'X, hY'Y>) hZZ'
      (prod_mem_prodNbhd hX' hY') hZ'

theorem ofMap_2_toMap_2 (f : ApproximableMap (prod V_0 V_1) V_2) : ofMap_2 (toMap_2 f)
     = f := by
  apply ApproximableMap.ext
  intro W Z
  show (prod V_0 V_1).mem W  /\  f.rel (prodNbhd (Sum.inl ^-1' W) (Sum.inr ^-1' W)) Z
     <->  _
  constructor
  * rintro <hW, hrel>; rwa [ <-  prodNbhd_preimage hW] at hrel
  * intro hrel
    exact <f.rel_dom hrel, by rwa [ <-  prodNbhd_preimage (f.rel_dom hrel)]>

/-- Extensionality for two-variable approximable mappings. -/
theorem ApproximableMap_2.ext {f g : ApproximableMap_2 V_0 V_1 V_2}
    (h : forall  X Y Z, f.rel X Y Z  <->  g.rel X Y Z) : f = g := by
  obtain <rf, _, _, _, _, _, _> := f
  obtain <rg, _, _, _, _, _, _> := g
  have : rf = rg := by funext X Y Z; exact propext (h X Y Z)
  subst this; rfl

theorem toMap_2_ofMap_2 (f : ApproximableMap_2 V_0 V_1 V_2) : toMap_2 (ofMap_2 f) = f
    := by
  apply ApproximableMap_2.ext
  intro X Y Z
  show (prod V_0 V_1).mem (prodNbhd X Y)  /\  f.rel _ _ Z  <->  _
  rw [inl_preimage_prodNbhd, inr_preimage_prodNbhd]
  constructor
  * rintro <_, hrel>; exact hrel
  * intro hrel
    exact <prod_mem_prodNbhd (f.rel_dom_0 hrel) (f.rel_dom_1 hrel), hrel>

/-- **Theorem 3.5 (Scott 1981, PRG-19).** The bijection between joint approximable
    mappings
`?_0  x  ?_1  ->  ?_2` and two-variable mappings `?_0, ?_1  ->  ?_2`: a function of two
     arguments comes from
an approximable mapping iff it is separately approximable. -/
def map_2Equiv (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) (V_2 :
    NeighborhoodSystem ?) :
    ApproximableMap (prod V_0 V_1) V_2 ? ApproximableMap_2 V_0 V_1 V_2 where
  toFun := toMap_2
  invFun := ofMap_2
```

```
  left_inv := ofMap_2_toMap_2
  right_inv := toMap_2_ofMap_2

/-- **Theorem 3.5 (elementwise) (Scott 1981, PRG-19).** The two-variable elementwise
    map of
`toMap_2 f` is `f` evaluated at the pairing: `(toMap_2 f)(x, y) = f(<x, y>)`. -/
theorem toElementMap_2_toMap_2 (f : ApproximableMap (prod V_0 V_1) V_2) (x : V_0.
    Element) (y : V_1.Element) :
    (toMap_2 f).toElementMap_2 x y = f.toElementMap (pair x y) := by
  apply Element.ext
  intro Z
  constructor
  * rintro <X, Y, hX, hY, hrel>
    exact <prodNbhd X Y, <X, Y, hX, hY, rfl>, hrel>
  * rintro <W, <X, Y, hX, hY, rfl>, hrel>
    exact <X, Y, hX, hY, hrel>

end
end DomainTheory.NeighbourhoodSystems
end
```

3.4.7 Lemma 3.6 — constant functions

LEMMA 3.6. Given $b \in |D_1|$, the constant function

$$b : |D_0| \to |D_1|$$

where $b(x) = b$ for all $x \in |D_0|$, comes from the approximable mapping such that

$$X \; b \; Y \quad \text{iff} \quad Y \in b,$$

for all $X \in D_0$ and $Y \in D_1$. □

There is no real confusion here in using "b" both for function and value. Returning, then, to the proof of 3.5, we see that the reason that $x \mapsto f(x, b)$ comes from an approximable mapping is that the mapping in question is the composition of two approximable mappings, namely $f \circ \langle I_{D_0}, b \rangle$. Clearly we can interchange the rôles of D_0 and D_1 to get at $y \mapsto f(a, y)$.

Conversely, assume that both these functions come from approximable mappings no matter the choice of a and b. Clearly the mapping to determine f is the relation from $X \cup Y$ to Z where

$$Z \in f(\uparrow X, \uparrow Y) = f(\uparrow (X \cup Y)).$$

To prove that this determines f we calculate by the formula of Exercise 2.9:

$$\begin{aligned} f(x, y) &= \bigcup \{ f(\uparrow X, y) \mid X \in x \} \\ &= \bigcup \{ \bigcup \{ f(\uparrow X, \uparrow Y) \mid Y \in y \} \mid X \in x \} \\ &= \bigcup \{ f(\uparrow X, \uparrow Y) \mid X \in x \text{ and } Y \in y \} \\ &= \bigcup \{ f(\uparrow (X \cup Y)) \mid (X \cup Y) \in \langle x, y \rangle \}. \end{aligned}$$

And, again by 2.9, this is what was needed. □

Said more informarily, a function of several arguments is approximable in all the variables *jointly* if it is approximable in each of the variables *separately*.

The type of argument used in 3.5 in the first half of the proof also provides a generalization of 2.6 to functions of several arguments. When we form a function like

$$f(g(x,z,\ldots),h(y,x,\ldots),k(z,w,\ldots),\ldots)$$

from given functions $f,g,h,k,\ldots$; we call the process *substitution*.

Lemma_3_6.lean

```
module
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_2

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Definition_3_1
@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem
variable {a b ? ? : Type*}
variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
open ApproximableMap
/- concordance: source L1094 LEMMA 3.6 constant functions -/
/-- **Lemma 3.6 (Scott 1981, PRG-19).** The constant map at `b : |?_1|`: `X b Y  <->  Y
      in  b`. -/
def constMap (V_0 : NeighborhoodSystem a) (b : V_1.Element) : ApproximableMap V_0 V_1
    where
  rel X Y := V_0.mem X  /\  b.mem Y
  rel_dom h := h.1
  rel_cod h := b.sub h.2
  master_rel := <V_0.master_mem, b.master_mem>
  inter_right := by rintro X Y Y' <hX, hY> <_, hY'>; exact <hX, b.inter_mem hY hY'>
  mono := by
    rintro X X' Y Y' <_, hY> _ hYY' hX' hY'
    exact <hX', b.up_mem hY hY' hYY'>

@[simp] theorem constMap_rel {b : V_1.Element} {X : Set a} {Y : Set b} :
    (constMap V_0 b).rel X Y  <->  V_0.mem X  /\  b.mem Y := Iff.rfl

@[simp] theorem toElementMap_constMap (b : V_1.Element) (x : V_0.Element) :
    (constMap V_0 b).toElementMap x = b := by
  apply Element.ext
  intro Y
  constructor
  * rintro <X, _, _, hbY>; exact hbY
  * intro hbY; exact <V_0.master, x.master_mem, V_0.master_mem, hbY>

end
end DomainTheory.NeighbourhoodSystems
end
```

3.4.8 Proposition 3.7 — closure under substitution

PROPOSITION 3.7. The functions of several arguments between domains coming from approximable mappings are closed under substitution.

Proof: An example will establish the method. Suppose there are four variables involved taking values in domains provided by systems D_0, D_1, D_2, D_3. We might have a substitution like:

$$f(g(x_0, x_1), h(x_1, x_2), k(x_3, x_0, x_2)).$$

Here it might be that the values of the functions inside come from quite other systems; for instance,

$$k : D_3 \times D_0 \times D_2 \to D_4$$

might be possible. By using projections

$$p_i : D_0 \times D_1 \times D_2 \times D_3 \to D_i,$$

where $i < 4$, we can assure that we have several functions all on the same product; thus,

$$k \cdot \langle p_3, p_0, p_2 \rangle : D_0 \times D_1 \times D_2 \times D_3 \to D_4.$$

Now no matter on what domains f is defined, the following composition makes sense:

$$f \circ \langle g \circ \langle p_0, p_1 \rangle, h \circ \langle p_1, p_2 \rangle, k \circ \langle p_3, p_0, p_2 \rangle \rangle;$$

and in fact this is the desired function. Writing it this way makes it clear that the function comes from an approximable mapping: we apply 3.3 (generalized, of course, to products with several terms) to construe the parts between brackets $\langle$ and $\rangle$ as approximable mappings, and then by this trick the composition $\circ$ is the ordinary composition of 2.6. □

It has to be admitted that there is a slight point overlooked in forming products like $D \times D$ with two identical domains. This is discussed in Exercise 3.14, invoking explicit isomorphisms.

The construct that makes the whole theory of domains work so smoothly is the function-space construct: it is possible to regard functions as *objects* which form a domain. Look back at Definition 2.1 and compare it with the original definition of element in 1.6. There are obvious formal similarities, except that filters are sets of neighbourhoods and mappings are sets of pairs of neighbourhoods (relations). But as we saw in 1.10 it is possible to turn the filters into tokens via a simple definition of neighbourhood. We apply the same kind of definition to the mappings.

Proposition_3_7.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_4
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_5
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem
variable {a b ? ? : Type*}
```

```
variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
variable {V_2 : NeighborhoodSystem ?}
open ApproximableMap
variable {V_3 : NeighborhoodSystem ?}
/- concordance: source L1129 PROPOSITION 3.7 closure under substitution -/
/-- **Proposition 3.7 (Scott 1981, PRG-19).** Multivariate approximable functions are
    closed under
substitution: substituting approximable maps `a, b : ?_3  ->  ?_i` into a two-variable
    approximable map
`F : ?_0  x  ?_1  ->  ?_2` yields the approximable map `F  comp  <a, b>`, whose value
    is `F(a(w), b(w))`. The
building blocks are exactly Definition 3.3's `paired` and Theorem 2.5's `comp`. -/
theorem substitution_toElementMap (F : ApproximableMap (prod V_0 V_1) V_2)
    (a : ApproximableMap V_3 V_0) (b : ApproximableMap V_3 V_1) (w : V_3.Element) :
    (F.comp (paired a b)).toElementMap w
      = (toMap_2 F).toElementMap_2 (a.toElementMap w) (b.toElementMap w) := by
  rw [toElementMap_comp, toElementMap_paired, toElementMap_2_toMap_2]
end

end DomainTheory.NeighbourhoodSystems
end
```

3.4.9 Definition 3.8 — function space

DEFINITION 3.8. Given neighbourhood systems D_0 and D_1, the *function space* $(D_0 \to D_1)$ is the system whose set of tokens is the set of approximable mappings of Definition 2.1 and whose neighbourhoods are finite non-empty intersections of sets of the form

$$[X, Y] = \{f : D_0 \to D_1 \mid X\, f\, Y\},$$

where $X \in D_0$ and $Y \in D_1$. □

We have been calling our mappings "approximable" for a long time now without saying exactly how they can be approximated! Definition 3.8 supplies the missing key, because once a domain has been defined, then the general theory gives an explicit meaning to the word approximation. We still have to verify, however, that the mappings do correspond to the elements of the domain.

Definition_3_8.lean

```
module
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.bot

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Lemma_3_6
@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem ApproximableMap
variable {a b ? : Type*} {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}

/-- Approximable maps are ordered by inclusion of their relations (Scott's
    approximation order on
```

```
`|?_0  ->  ?_1|`). Antisymmetry is `ApproximableMap.ext`. -/
instance : PartialOrder (ApproximableMap V_0 V_1) where
  le f g := forall  X Y, f.rel X Y  ->  g.rel X Y
  le_refl _ _ _ h := h
  le_trans _ _ _ h1 h2 X Y h := h2 X Y (h1 X Y h)
  le_antisymm f g h1 h2 := ApproximableMap.ext fun X Y => <h1 X Y, h2 X Y>

/-- Scott's step set `[X, Y] = {f ? X f Y}`. -/
def step (X : Set a) (Y : Set b) : Set (ApproximableMap V_0 V_1) := {f | f.rel X Y}

@[simp] theorem mem_step {X : Set a} {Y : Set b} {f : ApproximableMap V_0 V_1} :
    f  in  step X Y  <->  f.rel X Y := Iff.rfl

/-- A finite intersection of step sets, indexed by a list of `(X, Y)` pairs. -/
def stepFun (L : List (Set a  x  Set b)) : Set (ApproximableMap V_0 V_1) :=
  {f | forall  p  in  L, f.rel p.1 p.2}

@[simp] theorem mem_stepFun {L : List (Set a  x  Set b)} {f : ApproximableMap V_0 V_1}
    :
    f  in  stepFun L  <->  forall  p  in  L, f.rel p.1 p.2 := Iff.rfl

@[simp] theorem stepFun_nil : (stepFun [] : Set (ApproximableMap V_0 V_1)) = Set.univ
    := by
  ext f; simp

theorem stepFun_append (L L' : List (Set a  x  Set b)) :
    (stepFun (L ++ L') : Set (ApproximableMap V_0 V_1)) = stepFun L  I  stepFun L' :=
    by
  ext f
  simp only [mem_stepFun, List.mem_append, Set.mem_inter_iff]
  constructor
  * intro h; exact <fun p hp => h p (Or.inl hp), fun p hp => h p (Or.inr hp)>
  * rintro <hL, hL'> p (hp | hp)
    * exact hL p hp
    * exact hL' p hp

theorem stepFun_cons (p : Set a  x  Set b) (L : List (Set a  x  Set b)) :
    (stepFun (p :: L) : Set (ApproximableMap V_0 V_1)) = step p.1 p.2  I  stepFun L :=
    by
  ext f
  simp only [mem_stepFun, List.mem_cons, Set.mem_inter_iff, mem_step]
  constructor
  * intro h; exact <h p (Or.inl rfl), fun q hq => h q (Or.inr hq)>
  * rintro <hp, hrest> q (rfl | hq)
    * exact hp
    * exact hrest q hq

theorem stepFun_singleton (X : Set a) (Y : Set b) :
    (stepFun [(X, Y)] : Set (ApproximableMap V_0 V_1)) = step X Y := by
  rw [stepFun_cons, stepFun_nil, Set.inter_univ]

@[simp] theorem step_master_eq : (step V_0.master V_1.master : Set (ApproximableMap
    V_0 V_1)) = Set.univ := by
  ext f; simpa using f.master_rel

/-- Step neighbourhoods are *up-closed* under the map order: if `f  in  stepFun L` and
```

```
    `f  sqsub  f'`, then
`f'  in  stepFun L`. -/
theorem stepFun_up_closed {L : List (Set a  x  Set b)} {f f' : ApproximableMap V_0 V_1
    }
    (hf : f  in  stepFun L) (hff' : f  <=  f') : f'  in  stepFun L := by
  intro p hp
  exact hff' p.1 p.2 (hf p hp)

/-- Scott's intersection `? {Y_i ? X  subseteq  X_i}` of the outputs whose input is
    coarser than `X`, taken
inside the master neighbourhood `Delta_1` (so the empty intersection is `Delta_1`, per
    the convention 1.1a).
Indexed by the list `L` of `(X_i, Y_i)` pairs. -/
def interYs (m : Set b) : List (Set a  x  Set b)  ->  Set a  ->  Set b
  | [], _ => m
  | p :: L, X => {z | X  subseteq  p.1  ->  z  in  p.2}  I  interYs m L X

@[simp] theorem interYs_nil (m : Set b) (X : Set a) : interYs m [] X = m := rfl

theorem interYs_cons (m : Set b) (p : Set a  x  Set b) (L : List (Set a  x  Set b)) (X
     : Set a) :
    interYs m (p :: L) X = {z | X  subseteq  p.1  ->  z  in  p.2}  I  interYs m L X :=
     rfl

/-- Membership in `interYs`: `z  in  ?{Y_i ? X  subseteq  X_i}` iff `z  in  Delta_1`
    and `z  in  Y_i` for every `i` with
`X  subseteq  X_i`. -/
theorem mem_interYs {m : Set b} {L : List (Set a  x  Set b)} {X : Set a} {z : b} :
    z  in  interYs m L X  <->  z  in  m  /\  forall  p  in  L, X  subseteq  p.1  ->  z
      in  p.2 := by
  induction L with
  | nil => simp
  | cons p L ih =>
    rw [interYs_cons]
    simp only [Set.mem_inter_iff, Set.mem_ofPred_eq, ih, List.mem_cons]
    constructor
    * rintro <hp, hm, hL>
      refine <hm, ?_>
      rintro q (rfl | hq) hXq
      * exact hp hXq
      * exact hL q hq hXq
    * rintro <hm, hall>
      exact <fun hXp => hall p (Or.inl rfl) hXp, hm,
        fun q hq hXq => hall q (Or.inr hq) hXq>

/-- `interYs` is antitone in the input `X`: a sharper input intersects over more
    outputs. -/
theorem interYs_antitone {m : Set b} {L : List (Set a  x  Set b)} {X X' : Set a} (h :
    X'  subseteq  X) :
    interYs m L X'  subseteq  interYs m L X := by
  intro z hz
  rw [mem_interYs] at hz  |-
  exact <hz.1, fun p hp hXp => hz.2 p hp (h.trans hXp)>

/-- `interYs` is contained in the master neighbourhood. -/
theorem interYs_subset_master {m : Set b} {L : List (Set a  x  Set b)} {X : Set a} :
```

```
    interYs m L X  subseteq  m := fun _ hz => (mem_interYs.mp hz).1

/-- `interYs` contains `Y?` whenever `X?  subseteq  X`-indexed: in particular `interYs
    m L X?  subseteq  Y?` for
`(X?, Y?)  in  L`. -/
theorem interYs_subset_of_mem {m : Set b} {L : List (Set a  x  Set b)} {p : Set a  x
    Set b}
    (hp : p  in  L) : interYs m L p.1  subseteq  p.2 :=
  fun _ hz => (mem_interYs.mp hz).2 p hp subset_rfl

/-- The *least* approximable mapping `f_0` belonging to the neighbourhood `? [X_i, Y_i
    ]`, defined by
`X f_0 Y  <->  ?{Y_i ? X  subseteq  X_i}  subseteq  Y`. Well-definedness uses
    consistency in the operational form `hcons`:
for every neighbourhood `X`, the outputs `{Y_i ? X  subseteq  X_i}` have their
    intersection again a
neighbourhood. -/
def leastMap (L : List (Set a  x  Set b)) (_hL : forall  p  in  L, V_0.mem p.1  /\
    V_1.mem p.2)
    (hcons : forall  {X}, V_0.mem X  ->  V_1.mem (interYs V_1.master L X)) :
    ApproximableMap V_0 V_1 where
  rel X Y := V_0.mem X  /\  V_1.mem Y  /\  interYs V_1.master L X  subseteq  Y
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := <V_0.master_mem, V_1.master_mem, interYs_subset_master>
  inter_right := by
    rintro X Y Y' <hX, hY, hsub> <_, hY', hsub'>
    exact <hX, V_1.inter_mem hY hY' (hcons hX) (Set.subset_inter hsub hsub'),
      Set.subset_inter hsub hsub'>
  mono := by
    rintro X X' Y Y' <_, _, hsub> hX'X hYY' hX' hY'
    exact <hX', hY', (interYs_antitone hX'X).trans (hsub.trans hYY')>

@[simp] theorem leastMap_rel {L : List (Set a  x  Set b)}
    {hL : forall  p  in  L, V_0.mem p.1  /\  V_1.mem p.2}
    {hcons : forall  {X}, V_0.mem X  ->  V_1.mem (interYs V_1.master L X)} {X : Set a}
     {Y : Set b} :
    (leastMap L hL hcons).rel X Y  <->  V_0.mem X  /\  V_1.mem Y  /\  interYs V_1.
    master L X  subseteq  Y := Iff.rfl

/- concordance: source L1169 DEFINITION 3.8 function space -/
/-- **Definition 3.8 (Scott 1981, PRG-19).** The *function space* `(?_0  ->  ?_1)`:
    tokens are
approximable maps, neighbourhoods are non-empty finite intersections of step sets. -/
def funSpace (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) :
    NeighborhoodSystem (ApproximableMap V_0 V_1) where
  mem W := (exists  L : List (Set a  x  Set b), (forall  p  in  L, V_0.mem p.1  /\
    V_1.mem p.2)  /\  W = stepFun L)
     /\  W.Nonempty
  master := Set.univ
  master_nonempty := <constMap V_0 V_1.bot, Set.mem_univ _>
  master_mem := <<[], by simp, stepFun_nil.symm>, <constMap V_0 V_1.bot, Set.mem_univ
    _>>
  inter_mem := by
    rintro W W' Z <<L, hL, rfl>, _> <<L', hL', rfl>, _> <_, hZne> hZsub
    refine <<L ++ L', ?_, (stepFun_append _ _).symm>, hZne.mono hZsub>
```

```
    intro p hp
    rcases List.mem_append.mp hp with h | h
    * exact hL p h
    * exact hL' p h
  sub_master := fun _ => Set.subset_univ _

@[simp] theorem funSpace_master : (funSpace V_0 V_1).master = Set.univ := rfl

theorem funSpace_mem_iff {W : Set (ApproximableMap V_0 V_1)} :
    (funSpace V_0 V_1).mem W  <->
      (exists  L : List (Set a  x  Set b), (forall  p  in  L, V_0.mem p.1  /\  V_1.mem
     p.2)  /\  W = stepFun L)
         /\  W.Nonempty := Iff.rfl

/-- A function-space neighbourhood is up-closed under the map order. -/
theorem funSpace_mem_up_closed {W : Set (ApproximableMap V_0 V_1)} (hW : (funSpace V_0
     V_1).mem W)
    {f f' : ApproximableMap V_0 V_1} (hf : f  in  W) (hff' : f  <=  f') : f'  in  W :=
     by
  obtain <<L, _, rfl>, _> := hW
  exact stepFun_up_closed hf hff'

/-- `[X, Y]  I  [X, Y'] = [X, Y  I  Y']` (intersectivity in the output). -/
theorem step_inter_right {X : Set a} {Y Y' : Set b} (hY : V_1.mem Y) (hY' : V_1.mem Y
    ') :
    (step X Y  I  step X Y' : Set (ApproximableMap V_0 V_1)) = step X (Y  I  Y') := by
  ext f
  simp only [Set.mem_inter_iff, mem_step]
  constructor
  * rintro <h, h'>; exact f.inter_right h h'
  * intro h
    exact <f.mono h subset_rfl Set.inter_subset_left (f.rel_dom h) hY,
           f.mono h subset_rfl Set.inter_subset_right (f.rel_dom h) hY'>

/-- A step neighbourhood `[X, Y]` is a neighbourhood of the function space (non-empty:
    it contains
the constant map `constMap V_0 (Y)`). -/
theorem step_mem {X : Set a} {Y : Set b} (hX : V_0.mem X) (hY : V_1.mem Y) :
    (funSpace V_0 V_1).mem (step X Y) := by
  refine <<[(X, Y)], ?_, (stepFun_singleton X Y).symm>,
    <constMap V_0 (V_1.principal hY), ?_>>
  * intro p hp; rw [List.mem_singleton] at hp; subst hp; exact <hX, hY>
  * show (constMap V_0 (V_1.principal hY)).rel X Y
    exact <hX, hY, subset_rfl>

/-- The "generation" lemma: a filter contains the intersection `stepFun L` iff it
    contains each
step `[X_i, Y_i]`. (The step sets `[X, Y]  in  phi` generate the filter `phi`.) -/
theorem mem_stepFun_iff (phi : (funSpace V_0 V_1).Element) {L : List (Set a  x  Set b)
    }
    (hL : forall  p  in  L, V_0.mem p.1  /\  V_1.mem p.2) :
    phi.mem (stepFun L)  <->  forall  p  in  L, phi.mem (step p.1 p.2) := by
  induction L with
  | nil => simp only [stepFun_nil, List.not_mem_nil, IsEmpty.forall_iff, implies_true,
     iff_true]
           exact phi.master_mem
```

```
  | cons p L ih =>
    rw [stepFun_cons]
    have hp := hL p (List.mem_cons.mpr (Or.inl rfl))
    have hLtail : forall  q  in  L, V_0.mem q.1  /\  V_1.mem q.2 :=
      fun q hq => hL q (List.mem_cons.mpr (Or.inr hq))
    constructor
    * intro hmem
      have hstep : phi.mem (step p.1 p.2) :=
        phi.up_mem hmem (step_mem hp.1 hp.2) Set.inter_subset_left
      have hne : (step p.1 p.2  I  stepFun L).Nonempty := (phi.sub hmem).2
      have htail : phi.mem (stepFun L) :=
        phi.up_mem hmem <<L, hLtail, rfl>, hne.mono Set.inter_subset_right> Set.
    inter_subset_right
      intro q hq
      rcases List.mem_cons.mp hq with rfl | hq
      * exact hstep
      * exact (ih hLtail).mp htail q hq
    * intro hall
      have hstep : phi.mem (step p.1 p.2) := hall p (List.mem_cons.mpr (Or.inl rfl))
      have htail : phi.mem (stepFun L) :=
        (ih hLtail).mpr (fun q hq => hall q (List.mem_cons.mpr (Or.inr hq)))
      exact phi.inter_mem hstep htail

/-- `X'  subseteq  X` and `Y  subseteq  Y'` imply `[X, Y]  subseteq  [X', Y']`. -/
theorem step_subset {X X' : Set a} {Y Y' : Set b} (hX' : V_0.mem X') (hY' : V_1.mem Y
    ')
    (hX'X : X'  subseteq  X) (hYY' : Y  subseteq  Y') : (step X Y : Set (
    ApproximableMap V_0 V_1))  subseteq  step X' Y' := by
  intro f hf
  exact f.mono hf hX'X hYY' hX' hY'

end
end DomainTheory.NeighbourhoodSystems
end
```

3.4.10 Proposition 3.9 — consistent step neighbourhoods

PROPOSITION 3.9. Let neighbourhoods $X_i \in D_0$ and $Y_i \in D_1$ be given for $i < n$. Then the set of $[X_i, Y_i]$ for $i < n$ is consistent in $(D_0 \to D_1)$ iff the following condition holds:

(i) whenever $I \subseteq \{0, 1, \ldots, n-1\}$ and $\{X_i \mid i \in I\}$ is consistent in D_0, then $\{Y_i \mid i \in I\}$ must be consistent in D_1.

Moreover, when consistency holds, the least approximable mapping f_0 belonging to the intersection of the $[X_i, Y_i]$ is defined by:

(ii) $X\, f_0 Y$ iff $\bigcap\{Y_i \mid X \subseteq X_i\} \subseteq Y$ for $X \in D_0$ and $Y \in D_1$.

Proof: Suppose the $[X_i, Y_i]$ are consistent in $(D_0 \to D_1)$. Since the function space is being defined outright as a positive system, consistency means

$$f \in \bigcap\{[X_i, Y_i] \mid i < n\}$$

for some $f : D_0 \to D_1$. Now, with f in hand, let us check condition (i). Suppose $\{X_i \mid i \in I\}$ is consistent. This means

$$x \in \bigcap\{[X_i] \mid i \in I\}$$

for some $x \in |D_0|$. Suppose $i \in I$, so $x \in [X_i]$. Since $X_i f Y_i$ holds, $f(x) \in [Y_i]$. This means, therefore, that

$$f(x) \in \bigcap\{[Y_i] \mid i \in I\},$$

and so $\{Y_i \mid i \in I\}$ is consistent.

For the converse, suppose (i) is the case. We take (ii) as the definition of a mapping and remark that for an arbitrary $X \in D_0$, the set $\{X_i \mid X \subseteq X_i\}$ is automatically consistent in D_0. By our assumption, the set $\{Y_i \mid X \subseteq X_i\}$ is therefore consistent. This means that

$$\bigcap\{Y_i \mid X \subseteq X_i\} \in D_1.$$

(Keep in mind that i is restricted to those $i < n$, and there are only finitely many neighbourhoods being considered here.) It is thus almost immediate that the relation f_0 defined by (ii) satisfies conditions of 2.1 and so is an approximable mapping $f_0 : D_0 \to D_1$. By construction

$$X_i f_0 Y_i$$

holds trivially for all $i < n$; therefore,

$$f_0 \in \bigcap\{[X_i, Y_i] \mid i < n\}$$

and the desired consistency is established.

Finally suppose that f is any mapping in the neighbourhood under discussion; this means $X_i f Y_i$ holds for all $i < n$. Suppose $X f_0 Y$ holds. We have for $X \subseteq X_i$, $X f Y_i$; so

$$X f \bigcap\{Y_i \mid X \subseteq X_i\} \subseteq Y.$$

Thus, $X f Y$ follows; hence, as relations, $f_0 \subseteq f$. In other words f_0 is the minimal element of the neighbourhood. □

We note that, as a consequence of what we have just proved, when the neighbourhood is consistent, then

$$\bigcap\{[X_i, Y_i] \mid i < n\} \subseteq [X, Y]$$

is exactly equivalent to

$$\bigcap\{Y_i \mid X \subseteq X_i\} \subseteq Y.$$

Note also that a single neighbourhood $[X_0, Y_0]$ is always consistent since it contains the *constant mapping* k where

$$X k Y \text{ iff } Y_0 \subseteq Y,$$

for all $X \in \mathcal{D}_0$ and $Y \in \mathcal{D}_1$. Some other simple observations about these neighbourhoods are just translations of the conditions of Definition 2.1:

$$[\Delta_0, \Delta_1] = |\mathcal{D}_0 \to \mathcal{D}_1|;$$

$$[X, Y] \cap [X, Y'] = [X, Y \cap Y']; \text{ and}$$

$$X' \subseteq X \text{ and } Y \subseteq Y' \text{ imply } [X, Y] \subseteq [X', Y'],$$

for all $X, X' \in \mathcal{D}_0$ and $Y, Y' \in \mathcal{D}_1$. We are now ready to prove the main result about the construct.

Proposition_3_9.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Definition_3_8
@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem ApproximableMap
variable {a b ? : Type*} {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
/- concordance: source L1181 PROPOSITION 3.9 consistent step neighbourhoods -/
/-- **Proposition 3.9 (Scott 1981, PRG-19).** The least map `f_0` belongs to the
    neighbourhood:
`X_i f_0 Y_i` for every `(X_i, Y_i)  in  L`. -/
theorem leastMap_mem_stepFun {L : List (Set a  x  Set b)} (hL : forall  p  in  L, V_0.
    mem p.1  /\  V_1.mem p.2)
    (hcons : forall  {X}, V_0.mem X  ->  V_1.mem (interYs V_1.master L X)) : leastMap
    L hL hcons  in  stepFun L := by
  intro p hp
  exact <(hL p hp).1, (hL p hp).2, interYs_subset_of_mem hp>

/-- The relation `X f Y` holds for `f` in the neighbourhood `stepFun L` at the master
    output, and
more importantly `f` relates `X` to the whole intersection `interYs Delta_1 L X` (
    finite intersectivity
over the relevant outputs). The case split deciding `X  subseteq  X_i` is a documented
    classical step. -/
theorem rel_interYs {L : List (Set a  x  Set b)} {f : ApproximableMap V_0 V_1} (hf : f
     in  stepFun L)
    {X : Set a} (hX : V_0.mem X) : f.rel X (interYs V_1.master L X) := by
  induction L with
  | nil =>
    rw [interYs_nil]
    exact f.mono f.master_rel (V_0.sub_master hX) subset_rfl hX V_1.master_mem
  | cons p L ih =>
    have hftail : f  in  stepFun L := fun q hq => hf q (List.mem_cons.mpr (Or.inr hq))
    have htail := ih hftail
    by_cases hXp : X  subseteq  p.1
    * have hp : f.rel p.1 p.2 := hf p (List.mem_cons.mpr (Or.inl rfl))
      have hXp2 : f.rel X p.2 := f.mono hp hXp subset_rfl hX (f.rel_cod hp)
      have heq : interYs V_1.master (p :: L) X = p.2  I  interYs V_1.master L X := by
        rw [interYs_cons]; ext z
        simp only [Set.mem_inter_iff, Set.mem_ofPred_eq]
```

```
        exact <fun <h1, h2> => <h1 hXp, h2>, fun <h1, h2> => <fun _ => h1, h2>>
      rw [heq]; exact f.inter_right hXp2 htail
    * have heq : interYs V_1.master (p :: L) X = interYs V_1.master L X := by
        rw [interYs_cons]; ext z
        simp only [Set.mem_inter_iff, Set.mem_ofPred_eq]
        exact <fun h => h.2, fun h => <fun hc => absurd hc hXp, h>>
      rw [heq]; exact htail

/-- **Proposition 3.9 (Scott 1981, PRG-19).** `f_0` is the *minimal* element of the
    neighbourhood:
any `f` with `X_i f Y_i` for all `i` satisfies `f_0  subseteq  f`. -/
theorem leastMap_le {L : List (Set a  x  Set b)} (hL : forall  p  in  L, V_0.mem p.1
    /\  V_1.mem p.2)
    (hcons : forall  {X}, V_0.mem X  ->  V_1.mem (interYs V_1.master L X)) {f :
    ApproximableMap V_0 V_1}
    (hf : f  in  stepFun L) : leastMap L hL hcons  <=  f := by
  rintro X Y <hX, hY, hsub>
  exact f.mono (rel_interYs hf hX) subset_rfl hsub hX hY

/-- **Remark after Proposition 3.9 (Scott 1981, PRG-19).** When the neighbourhood is
    consistent,
`? [X_i, Y_i]  subseteq  [X, Y]` iff `?{Y_i ? X  subseteq  X_i}  subseteq  Y`. This is
    the form used to check that `curry` is
monotone (and hence approximable). -/
theorem stepFun_subset_step_iff {L : List (Set a  x  Set b)} (hL : forall  p  in  L,
    V_0.mem p.1  /\  V_1.mem p.2)
    (hcons : forall  {X}, V_0.mem X  ->  V_1.mem (interYs V_1.master L X)) {X : Set a}
     {Y : Set b}
    (hX : V_0.mem X) (hY : V_1.mem Y) :
    (stepFun L : Set (ApproximableMap V_0 V_1))  subseteq  step X Y  <->  interYs V_1.
    master L X  subseteq  Y := by
  constructor
  * intro hsub
    have := hsub (leastMap_mem_stepFun hL hcons)
    exact (mem_step.mp this).2.2
  * intro hsub f hf
    exact f.mono (rel_interYs hf hX) subset_rfl hsub hX hY

end
end DomainTheory.NeighbourhoodSystems
end
```

3.4.11 Theorem 3.10 — elements are mappings

THEOREM 3.10. Given neighbourhood systems $\mathcal{D}_0$ and $\mathcal{D}_1$, the function space system $(\mathcal{D}_0 \to \mathcal{D}_1)$ is complete in the sense that every filter in $|\mathcal{D}_0 \to \mathcal{D}_1|$ is fixed by a unique approximable mapping.

Proof: Let $f : \mathcal{D}_0 \to \mathcal{D}_1$ be an approximable mapping. By the very definition of $(\mathcal{D}_0 \to \mathcal{D}_1)$ it determines a filter by the definition:

$$\hat{f} = \{F \in (D_0 \to D_1) \mid f \in F\}.$$

Trivially $[X, Y] \in \hat{f}$ iff $f \in [X, Y]$ iff $X f Y$; so this filter uniquely determines the relation f. What we have to show is that every filter in $|D_0 \to D_1|$ is of this form.

Suppose $\varphi \in |D_0 \to D_1|$ is any filter. A relation can be defined at once by

$$X \hat{\varphi} Y \quad \text{iff} \quad [X, Y] \in \varphi.$$

In view of the remarks we made just before stating this theorem, there is no problem in showing that $\hat{\varphi}$ is an approximable mapping. Since the neighbourhoods of the function space are in any case finite intersections of sets like $[X, Y]$, those $[X, Y] \in \varphi$ generate φ. This means that $\hat{\hat{\varphi}} = \varphi$. By definition $\hat{\hat{f}} = f$, so there is a one-one correspondence between mappings and filters. (This correspondence is obviously inclusion preserving, too.) □

We now know just about everything about $|D_0 \to D_1|$ as a domain: the elements correspond isomorphically to the approximable mappings; the finite elements are explained completely by 3.9; and we have seen how to calculate with neighbourhoods. The final step is to relate the function space to other domains by appropriate mappings. In doing this we shall freely construe elements of $|D_0 \to D_1|$ as approximable mappings in view of 3.10.

Theorem_3_10.lean

```
module
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Definition_3_8

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem ApproximableMap
variable {a b ? : Type*} {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
/- concordance: source L1273 THEOREM 3.10 elements are mappings -/
/-- **Theorem 3.10 (Scott 1981, PRG-19).** The relation `X phi Y  <->  [X, Y]  in  phi`
    of a filter `phi`.
Intersectivity is the payoff of positivity (`[X,Y] I [X,Y'] = [X,Y I Y']` is non-empty,
    so `Y I Y'  in  ?_1`). -/
def toApproxMap (phi : (funSpace V_0 V_1).Element) : ApproximableMap V_0 V_1 where
  rel X Y := phi.mem (step X Y)
  rel_dom := by intro X Y h; obtain <f, hf> := (phi.sub h).2; exact f.rel_dom hf
  rel_cod := by intro X Y h; obtain <f, hf> := (phi.sub h).2; exact f.rel_cod hf
  master_rel := by show phi.mem (step V_0.master V_1.master); rw [step_master_eq];
    exact phi.master_mem
  inter_right := by
    intro X Y Y' h h'
    obtain <f, hf> := (phi.sub h).2
    obtain <f', hf'> := (phi.sub h').2
    have hY : V_1.mem Y := f.rel_cod hf
    have hY' : V_1.mem Y' := f'.rel_cod hf'
    show phi.mem (step X (Y  I  Y'))
    rw [ <-  step_inter_right hY hY']
    exact phi.inter_mem h h'
  mono := by
    intro X X' Y Y' h hX'X hYY' hX' hY'
    show phi.mem (step X' Y')
    exact phi.up_mem h (step_mem hX' hY') (step_subset hX' hY' hX'X hYY')

@[simp] theorem toApproxMap_rel {phi : (funSpace V_0 V_1).Element} {X : Set a} {Y :
    Set b} :
```

```
    (toApproxMap phi).rel X Y  <->  phi.mem (step X Y) := Iff.rfl

/-- **Theorem 3.10 (Scott 1981, PRG-19).** The filter `f-hat = {F ? f  in  F}` of an
    approximable map. -/
def toFilter (f : ApproximableMap V_0 V_1) : (funSpace V_0 V_1).Element where
  mem W := (funSpace V_0 V_1).mem W  /\  f  in  W
  sub h := h.1
  master_mem := <(funSpace V_0 V_1).master_mem, Set.mem_univ f>
  inter_mem := by
    rintro W W' <hW, hfW> <hW', hfW'>
    obtain <<L, hL, rfl>, _> := hW
    obtain <<L', hL', rfl>, _> := hW'
    refine <<<L ++ L', ?_, (stepFun_append _ _).symm>, <f, ?_>>, ?_>
    * intro p hp; rcases List.mem_append.mp hp with h | h
      * exact hL p h
      * exact hL' p h
    * exact Set.mem_inter hfW hfW'
    * exact Set.mem_inter hfW hfW'
  up_mem := by rintro W W' <hW, hfW> hW' hWW'; exact <hW', hWW' hfW>

@[simp] theorem mem_toFilter {f : ApproximableMap V_0 V_1} {W : Set (ApproximableMap
    V_0 V_1)} :
    (toFilter f).mem W  <->  (funSpace V_0 V_1).mem W  /\  f  in  W := Iff.rfl

/-- **Theorem 3.10 (Scott 1981, PRG-19).** The function space is *complete*: every
    filter is fixed
by a unique approximable mapping, inclusion-preservingly. -/
def funSpaceEquiv (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) :
    (funSpace V_0 V_1).Element ?o ApproximableMap V_0 V_1 where
  toFun := toApproxMap
  invFun := toFilter
  left_inv phi := by
    apply Element.ext
    intro W
    constructor
    * rintro <hWmem, hfW>
      obtain <<L, hL, rfl>, _> := hWmem
      exact (mem_stepFun_iff phi hL).mpr (fun p hp => hfW p hp)
    * intro hW
      refine <phi.sub hW, ?_>
      obtain <<L, hL, rfl>, _> := phi.sub hW
      intro p hp
      exact (mem_stepFun_iff phi hL).mp hW p hp
  right_inv f := by
    apply ApproximableMap.ext
    intro X Y
    constructor
    * rintro <_, hf>; exact hf
    * intro hf; exact <step_mem (f.rel_dom hf) (f.rel_cod hf), hf>
  map_rel_iff' := by
    intro phi phi'
    constructor
    * intro h W hW
      obtain <<L, hL, rfl>, _> := phi.sub hW
      refine (mem_stepFun_iff phi' hL).mpr (fun p hp => ?_)
      exact h p.1 p.2 ((mem_stepFun_iff phi hL).mp hW p hp)
```

```
    * intro h X Y hrel
      exact h _ hrel

@[simp] theorem funSpaceEquiv_apply (phi : (funSpace V_0 V_1).Element) :
    funSpaceEquiv V_0 V_1 phi = toApproxMap phi := rfl

@[simp] theorem funSpaceEquiv_symm_apply (f : ApproximableMap V_0 V_1) :
    (funSpaceEquiv V_0 V_1).symm f = toFilter f := rfl

/-- Intersection of two function-space neighbourhoods, when non-empty, is again one.
    -/
theorem funSpace_mem_inter {W W' : Set (ApproximableMap V_0 V_1)}
    (hW : (funSpace V_0 V_1).mem W) (hW' : (funSpace V_0 V_1).mem W') (hne : (W  I  W
    ').Nonempty) :
    (funSpace V_0 V_1).mem (W  I  W') := by
  obtain <<L, hL, rfl>, _> := hW
  obtain <<L', hL', rfl>, _> := hW'
  refine <<L ++ L', ?_, (stepFun_append _ _).symm>, hne>
  intro p hp
  rcases List.mem_append.mp hp with h | h
  * exact hL p h
  * exact hL' p h

end
end DomainTheory.NeighbourhoodSystems
end
```

3.4.12 Theorem 3.11 — evaluation combinator

THEOREM 3.11. Given neighbourhood systems D_1 and D_2, there is a uniquely determined approximable mapping

$$\text{eval} : (D_1 \to D_2) \times D_1 \to D_2,$$

where for all $f : D_1 \to D_2$ and all $x \in |D_1|$ we have

(i) $\text{eval}(f, x) = f(x)$.

Proof: For $F \in (D_1 \to D_2)$ and $X \in D_1$ and $Y \in D_2$ define eval as a relation by:

$$F, X \text{ eval } Y \quad \text{iff} \quad X f Y \text{ for all } f \in F.$$

Remember that neighbourhoods in the function space are sets of approximable mappings. It is easily checked that this definition makes eval approximable. We now calculate the function values by the formula of 2.2 (i):

$$\text{eval}(f, x) = \{Y \in D_2 \mid \exists F \in (D_1 \to D_2)\, \exists X \in x.\ f \in F \text{ and } F \cup X \text{ eval } Y\}$$

Because, again by 2.2 (i), we have

$$f(x) = \{Y \in D_2 \mid \exists X \in x.\ X f Y\},$$

we can see from the definition of eval that $\text{eval}(f,x) \subseteq f(x)$. Suppose that $Y \in f(x)$. Then $X\,f\,Y$ holds for some $X \in x$. We can write $f \in [X,Y] \in (D_1 \to D_2)$ and it is clear that

$$[X,Y] \cup X \text{ eval } Y$$

holds by definition. Therefore, $Y \in \text{eval}(f,x)$, and so $f(x) \subseteq \text{eval}(f,x)$. □

This theorem is essential for our programme: it shows that in taking functions as objects the very basic operation of forming the function value is an approximable mapping. In other words we can treat the expression $f(x)$ not just as a function of x, as we have done from the start, but also as a function of f as well. The result also indicates that there are useful maps defined on domains that themselves are function spaces; we shall meet many more of these. The next theorem provides further examples.

Theorem_3_11.lean

```
module
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_10

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_5
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Lemma_3_6
@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem ApproximableMap
variable {a b ? : Type*} {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
section
variable {V_2 : NeighborhoodSystem ?}
/- concordance: source L1291 THEOREM 3.11 evaluation combinator -/
/-- **Theorem 3.11 (Scott 1981, PRG-19).** The two-variable evaluation map
`eval : (?_1  ->  ?_2)  x  ?_1  ->  ?_2`, `F, X eval Y  <->  X f Y for all f  in  F`.
    -/
def eval (V_1 : NeighborhoodSystem b) (V_2 : NeighborhoodSystem ?) :
    ApproximableMap_2 (funSpace V_1 V_2) V_1 V_2 where
  rel F X Y := (funSpace V_1 V_2).mem F  /\  V_1.mem X  /\  V_2.mem Y  /\  forall  f
    in  F, f.rel X Y
  rel_dom_0 h := h.1
  rel_dom_1 h := h.2.1
  rel_cod h := h.2.2.1
  master_rel := <(funSpace V_1 V_2).master_mem, V_1.master_mem, V_2.master_mem,
    fun f _ => f.master_rel>
  inter_right := by
    rintro F X Y Y' <hF, hX, hY, hrel> <_, _, hY', hrel'>
    obtain <f, hf> := (funSpace_mem_iff.mp hF).2
    refine <hF, hX, ?_, fun g hg => g.inter_right (hrel g hg) (hrel' g hg)>
    exact f.rel_cod (f.inter_right (hrel f hf) (hrel' f hf))
  mono := by
    rintro F F' X X' Y Y' <hF, hX, hY, hrel> hF'F hX'X hYY' hF' hX' hY'
    exact <hF', hX', hY', fun f hf => f.mono (hrel f (hF'F hf)) hX'X hYY' hX' hY'>

/-- **Theorem 3.11 (Scott 1981, PRG-19).** Evaluation as a single approximable map out
     of the
product `(?_1  ->  ?_2)  x  ?_1  ->  ?_2`. -/
def evalMap (V_1 : NeighborhoodSystem b) (V_2 : NeighborhoodSystem ?) :
    ApproximableMap (prod (funSpace V_1 V_2) V_1) V_2 := ofMap_2 (eval V_1 V_2)
```

```
/-- The `X`-section of a two-variable map `g : ?_0  x  ?_1  ->  ?_2`: the map `?_1  ->
     ?_2` with
`Y (gSection g X) Z  <->  X  U  Y g Z`. -/
def gSection (g : ApproximableMap (prod V_0 V_1) V_2) {X : Set a} (hX : V_0.mem X) :
    ApproximableMap V_1 V_2 where
  rel Y Z := g.rel (prodNbhd X Y) Z
  rel_dom h := (prod_mem_prodNbhd_iff.mp (g.rel_dom h)).2
  rel_cod h := g.rel_cod h
  master_rel := g.rel_master (prod_mem_prodNbhd hX V_1.master_mem)
  inter_right h h' := g.inter_right h h'
  mono := by
    intro Y Y' Z Z' h hY'Y hZZ' hY' hZ'
    exact g.mono h (prodNbhd_subset_iff.mpr <subset_rfl, hY'Y>) hZZ'
      (prod_mem_prodNbhd hX hY') hZ'

@[simp] theorem gSection_rel {g : ApproximableMap (prod V_0 V_1) V_2} {X : Set a} (hX
    : V_0.mem X)
    {Y : Set b} {Z : Set ?} : (gSection g hX).rel Y Z  <->  g.rel (prodNbhd X Y) Z :=
    Iff.rfl

theorem gSection_le {g : ApproximableMap (prod V_0 V_1) V_2} {X X' : Set a}
    (hX : V_0.mem X) (hX' : V_0.mem X') (hX'X : X'  subseteq  X) : gSection g hX  <=
    gSection g hX' := by
  intro Y Z h
  have hY := (prod_mem_prodNbhd_iff.mp (g.rel_dom h)).2
  exact g.mono h (prodNbhd_subset_iff.mpr <hX'X, subset_rfl>) subset_rfl
    (prod_mem_prodNbhd hX' hY) (g.rel_cod h)

/-- **Theorem 3.11(i) (Scott 1981, PRG-19).** `eval(f, x) = f(x)` (with the filter `phi
    ` viewed as
the map `toApproxMap phi` via Theorem 3.10). -/
theorem toElementMap_2_eval (phi : (funSpace V_1 V_2).Element) (x : V_1.Element) :
    (eval V_1 V_2).toElementMap_2 phi x = (toApproxMap phi).toElementMap x := by
  apply Element.ext
  intro Y
  constructor
  * rintro <F, X, hphiF, hxX, _, hX, hY, hrel>
    refine <X, hxX, ?_>
    show phi.mem (step X Y)
    exact phi.up_mem hphiF (step_mem hX hY) (fun f hf => hrel f hf)
  * rintro <X, hxX, hrel>
    have hstep : (funSpace V_1 V_2).mem (step X Y) := phi.sub hrel
    obtain <f, hf> := (funSpace_mem_iff.mp hstep).2
    exact <step X Y, X, hrel, hxX, hstep, f.rel_dom hf, f.rel_cod hf, fun g hg => hg>

/-- **Theorem 3.11(i) (Scott 1981, PRG-19).** `eval(<f, x>) = f(x)`. -/
theorem evalMap_apply (phi : (funSpace V_1 V_2).Element) (x : V_1.Element) :
    (evalMap V_1 V_2).toElementMap (pair phi x) = (toApproxMap phi).toElementMap x :=
    by
  rw [evalMap,  <-  toElementMap_2_toMap_2, toMap_2_ofMap_2, toElementMap_2_eval]

/-- `(eval  comp  <F, G>)(w) = F(w)(G(w))`. -/
theorem toElementMap_evalMap_comp_paired (F : ApproximableMap V_0 (funSpace V_1 V_2))
    (G : ApproximableMap V_0 V_1) (w : V_0.Element) :
    ((evalMap V_1 V_2).comp (paired F G)).toElementMap w =
```

```
      (toApproxMap (F.toElementMap w)).toElementMap (G.toElementMap w) := by
  rw [toElementMap_comp, toElementMap_paired, evalMap_apply]
end

end
end DomainTheory.NeighbourhoodSystems
end
```

3.4.13 Theorem 3.12 — currying

THEOREM 3.12. Given neighbourhood systems D_0, D_1, D_2 there is associated with every approximable mapping $g : D_0 \times D_1 \to D_2$ a uniquely determined approximable mapping

$$\text{curry}(g) : D_0 \to (D_1 \to D_2)$$

such that for $x \in |D_0|$ and $y \in |D_1|$

(i) $\text{curry}(g)(x)(y) = g(x, y)$.

Moreover we have these functional equations:

(ii) $\text{eval} \circ \langle \text{curry}(g) \circ p_0,\ p_1 \rangle = g$, and

(iii) $\text{curry}(\text{eval} \circ \langle h \circ p_0,\ p_1 \rangle) = h$,

where the $p_i : \mathcal{D}_0 \times \mathcal{D}_1 \to \mathcal{D}_i$ are the projection mappings and $h : \mathcal{D}_0 \to (\mathcal{D}_1 \to \mathcal{D}_2)$ is any approximable mapping. This provides an isomorphism between the domains $|\mathcal{D}_0 \times \mathcal{D}_1 \to \mathcal{D}_2|$ and $|\mathcal{D}_0 \to (\mathcal{D}_1 \to \mathcal{D}_2)|$ and so we can regard

$$\text{curry} : (\mathcal{D}_0 \times \mathcal{D}_1 \to \mathcal{D}_2) \to (\mathcal{D}_0 \to (\mathcal{D}_1 \to \mathcal{D}_2))$$

as itself being an approximable mapping.

Proof: Given g as indicated, we can define $\text{curry}(g)$ as a relation and as an approximable mapping by:

$$X\ \text{curry}(g)\ [Y, Z] \text{ iff } X \cup Y\ g\ Z \quad \text{(but see Ex. 3.21)}$$

for all $X \in \mathcal{D}_0$, $Y \in \mathcal{D}_1$, $Z \in \mathcal{D}_2$. This is sufficient because an approximable mapping is intersective in the right-hand neighbourhood, so we know from the above exactly what $X\ \text{curry}(g)\ \bigcap\{[Y_i, Z_i] \mid i < n\}$ means for all finite intersections. The remark after 3.9 is then helpful in checking that by this definition $\text{curry}(g)$ satisfies the monotonicity condition and so is indeed approximable. We now calculate:

$$\begin{aligned}
\text{curry}(g)(x)(y) &= \{Z \in \mathcal{D}_2 \mid \exists Y \in y.\ Y\ \text{curry}(g)\ (x)\ Z\} \\
&= \{Z \in \mathcal{D}_2 \mid \exists Y \in y\ \exists X \in x.\ X\ \text{curry}(g)\ [Y, Z]\} \\
&= \{Z \in \mathcal{D}_2 \mid \exists Y \in y\ \exists X \in x.\ X \cup Y\ g\ Z\} \\
&= \{Z \in \mathcal{D}_2 \mid \exists W \in \langle x, y \rangle.\ W\ g\ Z\} \\
&= g(\langle x, y \rangle) = g(x, y).
\end{aligned}$$

This proves (i). We also see, that if we take the left-hand side of (ii) and apply it to a pair $\langle x, y \rangle$, it reduces to $g(x, y)$ by virtue of (i). Thus, the two functions in (ii) are the same.

Turning to (iii), call the left-hand side k. Using (i) again, we find

$$\begin{aligned} k(x)(y) &= \mathrm{eval} \circ \langle h \circ p_0, p_1 \rangle (\langle x, y \rangle) \\ &= \mathrm{eval}(\langle h \circ p_0(\langle x, y \rangle), p_1(\langle x, y \rangle)\rangle) \\ &= \mathrm{eval}(\langle h(x), y \rangle) \\ &= h(x)(y). \end{aligned}$$

As this is true for all $y \in |\mathcal{D}_1|$, then $k(x) = h(x)$ follows. As this is true for all $x \in |\mathcal{D}_0|$, then $k = h$ follows, and (iii) is proved.

Taking (ii) and (iii) together, it is clear that the domains $|D_0 \times D_1 \to D_2|$ and $|D_0 \to (D_1 \to D_2)|$ are in a one-one correspondence.

$$\mathrm{curry}(g) \sqsubseteq \mathrm{curry}(g') \text{ iff } g \sqsubseteq g'.$$

Hence, curry is an isomorphism, and we can invoke 2.7 to conclude that it comes from an approximable mapping. □

We close this lecture with some order-theoretic properties of function spaces that characterize inclusion and upper bounds of functions in a "pointwise" manner.

Theorem_3_12.lean

```
module
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_11

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem ApproximableMap
variable {a b ? : Type*} {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
section
variable {V_2 : NeighborhoodSystem ?}
/- concordance: source L1327 THEOREM 3.12 currying -/
/-- **Theorem 3.12 (Scott 1981, PRG-19).** `curry(g) : ?_0  ->  (?_1  ->  ?_2)`, where
`X curry(g) W  <->  (the X-section of g)  in  W` (for `W = [Y, Z]` this is `X  U  Y g Z
    `). -/
def curry (g : ApproximableMap (prod V_0 V_1) V_2) : ApproximableMap V_0 (funSpace V_1
     V_2) where
  rel X W := exists  hX : V_0.mem X, (funSpace V_1 V_2).mem W  /\  gSection g hX  in
    W
  rel_dom := fun <hX, _> => hX
  rel_cod := fun <_, hW, _> => hW
  master_rel := <V_0.master_mem, (funSpace V_1 V_2).master_mem, Set.mem_univ _>
  inter_right := by
    rintro X W W' <hX, hW, hmem> <_, hW', hmem'>
    exact <hX, funSpace_mem_inter hW hW' <gSection g hX, hmem, hmem'>,
      Set.mem_inter hmem hmem'>
  mono := by
    rintro X X' W W' <hX, hW, hmem> hX'X hWW' hX' hW'
    exact <hX', hW', funSpace_mem_up_closed hW' (hWW' hmem) (gSection_le hX hX' hX'X)>

@[simp] theorem curry_rel {g : ApproximableMap (prod V_0 V_1) V_2} {X : Set a}
```

```
    {W : Set (ApproximableMap V_1 V_2)} :
    (curry g).rel X W  <->  exists  hX : V_0.mem X, (funSpace V_1 V_2).mem W  /\
    gSection g hX  in  W := Iff.rfl

/-- The relational generation lemma for maps `h : ?_0  ->  (?_1  ->  ?_2)`: `X h (?_i [
    Y_i,Z_i])` iff
`X h [Y_i,Z_i]` for all `i`. -/
theorem rel_stepFun_iff (h : ApproximableMap V_0 (funSpace V_1 V_2)) {X : Set a} (hX :
     V_0.mem X)
    {L : List (Set b  x  Set ?)} (hL : forall  p  in  L, V_1.mem p.1  /\  V_2.mem p.2)
     :
    h.rel X (stepFun L)  <->  forall  p  in  L, h.rel X (step p.1 p.2) := by
  induction L with
  | nil =>
    simp only [stepFun_nil, List.not_mem_nil, IsEmpty.forall_iff, implies_true,
    iff_true]
    show h.rel X (funSpace V_1 V_2).master
    exact h.rel_master hX
  | cons p L ih =>
    rw [stepFun_cons]
    have hp := hL p (List.mem_cons.mpr (Or.inl rfl))
    have hLtail : forall  q  in  L, V_1.mem q.1  /\  V_2.mem q.2 :=
      fun q hq => hL q (List.mem_cons.mpr (Or.inr hq))
    constructor
    * intro hmem
      have hne : (step p.1 p.2  I  stepFun L).Nonempty := (h.rel_cod hmem).2
      have hstep : h.rel X (step p.1 p.2) :=
        h.mono hmem subset_rfl Set.inter_subset_left hX (step_mem hp.1 hp.2)
      have htail : h.rel X (stepFun L) :=
        h.mono hmem subset_rfl Set.inter_subset_right hX
          <<L, hLtail, rfl>, hne.mono Set.inter_subset_right>
      intro q hq
      rcases List.mem_cons.mp hq with rfl | hq
      * exact hstep
      * exact (ih hLtail).mp htail q hq
    * intro hall
      have hstep : h.rel X (step p.1 p.2) := hall p (List.mem_cons.mpr (Or.inl rfl))
      have htail : h.rel X (stepFun L) :=
        (ih hLtail).mpr (fun q hq => hall q (List.mem_cons.mpr (Or.inr hq)))
      exact h.inter_right hstep htail

/-- **Theorem 3.12 (Scott 1981, PRG-19).** Uncurrying `h : ?_0  ->  (?_1  ->  ?_2)` to
    `?_0  x  ?_1  ->  ?_2`:
`X  U  Y (uncurry h) Z  <->  X h [Y, Z]`. -/
def uncurry (h : ApproximableMap V_0 (funSpace V_1 V_2)) : ApproximableMap (prod V_0
    V_1) V_2 where
  rel W Z := (prod V_0 V_1).mem W  /\  h.rel (Sum.inl ^-1' W) (step (Sum.inr ^-1' W) Z
    )
  rel_dom h' := h'.1
  rel_cod := by
    rintro W Z <_, hrel>
    obtain <f, hf> := (funSpace_mem_iff.mp (h.rel_cod hrel)).2
    exact f.rel_cod hf
  master_rel := by
    refine <(prod V_0 V_1).master_mem, ?_>
    rw [show (prod V_0 V_1).master = prodNbhd V_0.master V_1.master from rfl,
```

```
      inl_preimage_prodNbhd, inr_preimage_prodNbhd, step_master_eq]
    exact h.master_rel
  inter_right := by
    rintro W Z Z' <hW, hrel> <_, hrel'>
    obtain <f, hf> := (funSpace_mem_iff.mp (h.rel_cod hrel)).2
    obtain <f', hf'> := (funSpace_mem_iff.mp (h.rel_cod hrel')).2
    refine <hW, ?_>
    rw [ <-  step_inter_right (f.rel_cod hf) (f'.rel_cod hf')]
    exact h.inter_right hrel hrel'
  mono := by
    rintro W W_2 Z Z' <_, hrel> hW_2W hZZ' hW_2 hZ'
    have hinl : Sum.inl ^-1' W_2  subseteq  Sum.inl ^-1' W := Set.preimage_mono hW_2W
    have hinr : Sum.inr ^-1' W_2  subseteq  Sum.inr ^-1' W := Set.preimage_mono hW_2W
    obtain <A, B, hA, hB, rfl> := hW_2
    rw [inl_preimage_prodNbhd] at hinl  |-
    rw [inr_preimage_prodNbhd] at hinr  |-
    refine <prod_mem_prodNbhd hA hB, ?_>
    exact h.mono hrel hinl (step_subset hB hZ' hinr hZZ') hA (step_mem hB hZ')

@[simp] theorem uncurry_rel {h : ApproximableMap V_0 (funSpace V_1 V_2)}
    {W : Set (a ? b)} {Z : Set ?} :
    (uncurry h).rel W Z  <->
      (prod V_0 V_1).mem W  /\  h.rel (Sum.inl ^-1' W) (step (Sum.inr ^-1' W) Z) :=
    Iff.rfl

/-- **Theorem 3.12 (Scott 1981, PRG-19).** `curry (uncurry h) = h`. -/
theorem curry_uncurry (h : ApproximableMap V_0 (funSpace V_1 V_2)) : curry (uncurry h)
     = h := by
  apply ApproximableMap.ext
  intro X W
  constructor
  * rintro <hX, hW, hmem>
    obtain <<L, hL, rfl>, _> := hW
    refine (rel_stepFun_iff h hX hL).mpr (fun p hp => ?_)
    have := hmem p hp
    -- `gSection (uncurry h) hX  in  step p.1 p.2` means `(uncurry h).rel (prodNbhd X p
    .1) p.2`
    have hrel : (uncurry h).rel (prodNbhd X p.1) p.2 := this
    obtain <_, hrel'> := hrel
    rw [inl_preimage_prodNbhd, inr_preimage_prodNbhd] at hrel'
    exact hrel'
  * intro hrel
    have hX : V_0.mem X := h.rel_dom hrel
    have hW : (funSpace V_1 V_2).mem W := h.rel_cod hrel
    refine <hX, hW, ?_>
    obtain <<L, hL, rfl>, _> := hW
    intro p hp
    show (uncurry h).rel (prodNbhd X p.1) p.2
    refine <prod_mem_prodNbhd hX (hL p hp).1, ?_>
    rw [inl_preimage_prodNbhd, inr_preimage_prodNbhd]
    exact (rel_stepFun_iff h hX hL).mp hrel p hp

/-- **Theorem 3.12 (Scott 1981, PRG-19).** `uncurry (curry g) = g`. -/
theorem uncurry_curry (g : ApproximableMap (prod V_0 V_1) V_2) : uncurry (curry g) = g
     := by
  apply ApproximableMap.ext
```

```
  intro W Z
  constructor
  * rintro <hW, _, _, hmem>
    rw [prodNbhd_preimage hW]; exact hmem
  * intro hrel
    have hW := g.rel_dom hrel
    refine <hW, prod_mem_inl hW, step_mem (prod_mem_inr hW) (g.rel_cod hrel), ?_>
    show g.rel (prodNbhd (Sum.inl ^-1' W) (Sum.inr ^-1' W)) Z
    rwa [ <-  prodNbhd_preimage hW]

/-- **Theorem 3.12 (Scott 1981, PRG-19).** `curry` is an order-isomorphism between
`|?_0  x  ?_1  ->  ?_2|` and `|?_0  ->  (?_1  ->  ?_2)|`. -/
def curryEquiv (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b)
    (V_2 : NeighborhoodSystem ?) :
    ApproximableMap (prod V_0 V_1) V_2 ?o ApproximableMap V_0 (funSpace V_1 V_2) where
  toFun := curry
  invFun := uncurry
  left_inv := uncurry_curry
  right_inv := curry_uncurry
  map_rel_iff' := by
    intro g g'
    constructor
    * intro hcurry W Z hrel
      have h1 : (curry g).rel (Sum.inl ^-1' W) (step (Sum.inr ^-1' W) Z) := by
        have hu : (uncurry (curry g)).rel W Z := by rw [uncurry_curry]; exact hrel
        exact hu.2
      have h2 := hcurry _ _ h1
      have hu' : (uncurry (curry g')).rel W Z := <g.rel_dom hrel, h2>
      rw [uncurry_curry] at hu'
      exact hu'
    * intro hg X W hrel
      obtain <hX, hW, hmem> := hrel
      refine <hX, hW, ?_>
      obtain <<L, hL, rfl>, _> := hW
      intro p hp
      have hgrel : g.rel (prodNbhd X p.1) p.2 := hmem p hp
      exact hg _ _ hgrel

/-- **Theorem 3.12(i) (Scott 1981, PRG-19).** `curry(g)(x)(y) = g(x, y)`. -/
theorem toElementMap_curry_apply (g : ApproximableMap (prod V_0 V_1) V_2)
    (x : V_0.Element) (y : V_1.Element) :
    (toApproxMap ((curry g).toElementMap x)).toElementMap y = g.toElementMap (pair x y
    ) := by
  apply Element.ext
  intro Z
  constructor
  * rintro <Y, hyY, X, hxX, hX, _, hrel>
    exact <prodNbhd X Y, <X, Y, hxX, hyY, rfl>, hrel>
  * rintro <W, <X, Y, hxX, hyY, rfl>, hrel>
    exact <Y, hyY, X, hxX, x.sub hxX, step_mem (y.sub hyY) (g.rel_cod hrel), hrel>

/-- `uncurry` is the composition `eval  comp  <h  comp  p_0, p_1>`. -/
theorem uncurry_eq (h : ApproximableMap V_0 (funSpace V_1 V_2)) :
    uncurry h = (evalMap V_1 V_2).comp (paired (h.comp (proj_0 V_0 V_1)) (proj_1 V_0
    V_1)) := by
  apply ext_of_toElementMap
```

```
  intro w
  rw [toElementMap_comp, toElementMap_paired, toElementMap_comp, toElementMap_proj_0,
    toElementMap_proj_1, evalMap_apply]
  apply Element.ext
  intro Z
  constructor
  * rintro <W_0, hwW_0, hW_0, hrel>
    obtain <X, Y, hX, hY, rfl> := hW_0
    rw [inl_preimage_prodNbhd] at hrel
    rw [inr_preimage_prodNbhd] at hrel
    obtain <hwX, hwY> := (prod_mem_split hX hY).mp hwW_0
    exact <Y, <hY, hwY>, X, <hX, hwX>, hrel>
  * rintro <Y, <hY, hwY>, X, <hX, hwX>, hrel>
    refine <prodNbhd X Y, (prod_mem_split hX hY).mpr <hwX, hwY>, prod_mem_prodNbhd hX
    hY, ?_>
    rw [inl_preimage_prodNbhd, inr_preimage_prodNbhd]
    exact hrel

/-- **Theorem 3.12(iii) (Scott 1981, PRG-19).** `curry (eval  comp  <h  comp  p_0, p_1
    >) = h`. -/
theorem curry_eval_comp (h : ApproximableMap V_0 (funSpace V_1 V_2)) :
    curry ((evalMap V_1 V_2).comp (paired (h.comp (proj_0 V_0 V_1)) (proj_1 V_0 V_1)))
     = h := by
  rw [ <-  uncurry_eq, curry_uncurry]

/-- **Theorem 3.12(ii) (Scott 1981, PRG-19).** `eval  comp  <curry(g)  comp  p_0, p_1>
    = g`. -/
theorem eval_comp_curry (g : ApproximableMap (prod V_0 V_1) V_2) :
    (evalMap V_1 V_2).comp (paired ((curry g).comp (proj_0 V_0 V_1)) (proj_1 V_0 V_1))
     = g := by
  rw [ <-  uncurry_eq]; exact uncurry_curry g
end

theorem ApproximableMap.le_iff {f g : ApproximableMap V_0 V_1} :
    f  <=  g  <->  forall  X Y, f.rel X Y  ->  g.rel X Y := Iff.rfl

end
end DomainTheory.NeighbourhoodSystems
end
```

3.4.14 Theorem 3.13 — pointwise order of maps

THEOREM 3.13. For approximable functions $f, g : D_0 \to D_1$ we have

(i) $f \sqsubseteq g$ iff $f(x) \sqsubseteq g(x)$ for all $x \in |D_0|$.

For subsets $F \subseteq |D_0 \to D_1|$ we have

(ii) F is bounded in $|D_0 \to D_1|$ iff $\{f(x) \mid f \in F\}$ is bounded in $|D_1|$ for each $x \in |D_0|$;

and in that case for all $x \in |D_0|$:

(iii) $(\bigsqcup F)(x) = \bigsqcup\{f(x) \mid f \in F\}$.

Proof. The implication in (i) from left to right follows because evaluation is monotone in the function as well as the argument. The converse implication is a consequence of 2.2(ii).

For the proof of (ii) and (iii) we see that by (i) if F is bounded, so is every set $\{f(x) \mid f \in F\}$. For the converse direction, it is clear that (iii) defines *some* pointwise mapping; we have only to prove that it is *approximable*. The calculation that $\sqcup F$ preserves directed unions (see 2.9 and 2.11) is probably the simplest way to reach the conclusion. □

Theorem_3_13.lean

```
module
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Exercise_2_8
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Exercise_2_9
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_27
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_7
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_12

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem ApproximableMap
variable {a b ? : Type*} {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
/- concordance: source L1390 THEOREM 3.13 pointwise order of maps -/

/-- A set `F` of approximable maps is *bounded* when it has an upper bound in the map
    order. -/
def MapsBounded (F : Set (ApproximableMap V_0 V_1)) : Prop := exists  h, forall  f  in
      F, f  <=  h

/-- `F` is *pointwise bounded* when `{f(x) ? f  in  F}` is bounded in `|?_1|` for every
     `x`. -/
def PointwiseBounded (F : Set (ApproximableMap V_0 V_1)) : Prop :=
  forall  x : V_0.Element, V_1.Bounded (Set.image (fun f => f.toElementMap x) F)

theorem mapsBounded_principal {F : Set (ApproximableMap V_0 V_1)} (hF :
    PointwiseBounded F)
    {X : Set a} (hX : V_0.mem X) :
    V_1.Bounded (Set.image (fun f => f.toElementMap (V_0.principal hX)) F) :=
  hF (V_0.principal hX)

/-- The sup of `{f(X) ? f  in  F}` on principal inputs, used to build `sSupMaps`. -/
def supOnPrincipal (F : Set (ApproximableMap V_0 V_1)) (hF : PointwiseBounded F)
    (X : Set a) (hX : V_0.mem X) : V_1.Element :=
  V_1.sSup (Set.image (fun f => f.toElementMap (V_0.principal hX)) F) (
    mapsBounded_principal hF hX)

theorem supOnPrincipal_mono (F : Set (ApproximableMap V_0 V_1)) (hF : PointwiseBounded
     F)
    (X X' : Set a) (hX : V_0.mem X) (hX' : V_0.mem X') (hX'X : X'  subseteq  X) :
    supOnPrincipal F hF X hX  <=  supOnPrincipal F hF X' hX' :=
  V_1.sSup_le _ (mapsBounded_principal hF hX) fun s hs => by
    obtain <f, hf, rfl> := hs
```

```
    exact (toElementMap_mono f ((V_0.principal_le_iff hX hX').mpr hX'X)).trans
      (V_1.le_sSup _ (mapsBounded_principal hF hX') <f, hf, rfl>)

/-- **Theorem 3.13(iii) (Scott 1981, PRG-19).** The least upper bound of a pointwise-
    bounded set
`F`, defined on principal inputs by `supOnPrincipal` and extended via Exercise 2.8 (`
    ofMono`). -/
def sSupMaps (F : Set (ApproximableMap V_0 V_1)) (hF : PointwiseBounded F) :
    ApproximableMap V_0 V_1 :=
  ofMono (fun X hX => supOnPrincipal F hF X hX) (supOnPrincipal_mono F hF)

/-- **Theorem 3.13(i) (Scott 1981, PRG-19).** `f  sqsub  g  <->  forall  x, f(x)  sqsub
      g(x)`. -/
theorem le_iff_toElementMap_le {f g : ApproximableMap V_0 V_1} :
    f  <=  g  <->  forall  x, f.toElementMap x  <=  g.toElementMap x := by
  constructor
  * intro h x Y <X, hXx, hrel>
    exact <X, hXx, h X Y hrel>
  * intro h X Y hrel
    have hX : V_0.mem X := f.rel_dom hrel
    rw [f.rel_iff_mem_principal hX] at hrel
    rw [g.rel_iff_mem_principal hX]
    exact h (V_0.principal hX) Y hrel

theorem le_sSupMaps {F : Set (ApproximableMap V_0 V_1)} (hF : PointwiseBounded F)
    {f : ApproximableMap V_0 V_1} (hf : f  in  F) : f  <=  sSupMaps F hF := by
  intro X Y hrel
  have hX : V_0.mem X := f.rel_dom hrel
  have hmem : (f.toElementMap (V_0.principal hX)).mem Y := (f.rel_iff_mem_principal hX
    ).mp hrel
  exact <hX, (V_1.le_sSup _ (mapsBounded_principal hF hX) <f, hf, rfl>) Y hmem>

theorem sSupMaps_le {F : Set (ApproximableMap V_0 V_1)} (hF : PointwiseBounded F)
    {h : ApproximableMap V_0 V_1} (hh : forall  f  in  F, f  <=  h) : sSupMaps F hF
    <=  h := by
  intro X Y hrel
  obtain <hX, hYmem> := hrel
  have hle : supOnPrincipal F hF X hX  <=  h.toElementMap (V_0.principal hX) :=
    V_1.sSup_le _ (mapsBounded_principal hF hX) fun s hs => by
      obtain <f, hf, rfl> := hs
      exact (le_iff_toElementMap_le.mp (hh f hf)) (V_0.principal hX)
  exact (h.rel_iff_mem_principal hX).mpr (hle Y hYmem)

/-- **Theorem 3.13(ii) (Scott 1981, PRG-19).** `F` is bounded in `|?_0  ->  ?_1|` iff
    `{f(x) ? f  in  F}` is
bounded in `|?_1|` for each `x  in  |?_0|`. The forward direction is `
    le_iff_toElementMap_le` (3.13(i))
applied pointwise; the backward direction builds the bound `sSupMaps F`. -/
theorem mapsBounded_iff_pointwiseBounded {F : Set (ApproximableMap V_0 V_1)} :
    MapsBounded F  <->  PointwiseBounded F := by
  constructor
  * intro <h, hh> x
    refine <h.toElementMap x, fun z hz => ?_>
    obtain <f, hf, rfl> := hz
    exact (le_iff_toElementMap_le.mp (hh f hf)) x
  * intro hpb
```

```
    refine <sSupMaps F hpb, fun f hf X Y hrel => ?_>
    have hX : V_0.mem X := f.rel_dom hrel
    have hmem : (f.toElementMap (V_0.principal hX)).mem Y := (f.rel_iff_mem_principal
    hX).mp hrel
    exact <hX, (V_1.le_sSup _ (mapsBounded_principal hpb hX) <f, hf, rfl>) Y hmem>

theorem toElementMap_sSupMaps_principal {F : Set (ApproximableMap V_0 V_1)} (hF :
    PointwiseBounded F)
    {X : Set a} (hX : V_0.mem X) :
    (sSupMaps F hF).toElementMap (V_0.principal hX) = supOnPrincipal F hF X hX :=
  toElementMap_ofMono_principal _ (supOnPrincipal_mono F hF) X hX

theorem toElementMap_sSupMaps {F : Set (ApproximableMap V_0 V_1)} (hF :
    PointwiseBounded F)
    (x : V_0.Element) :
    (sSupMaps F hF).toElementMap x =
      V_1.sSup (Set.image (fun f => f.toElementMap x) F) (hF x) := by
  apply le_antisymm
  * -- `( sqsup F)(x)  sqsub   sqsup {f(x)}`: read `( sqsup F)(x)` off some principal `
    X` (Ex 2.9), where the
    -- principal value `( sqsup F)(X) =  sqsup {f(X)}` is bounded above by ` sqsup {f(x
    )}` (monotonicity, `X  sqsub  x`).
    intro Y hY
    rw [toElementMap_mem_iff_principal (sSupMaps F hF) x] at hY
    obtain <X, hxX, hY'> := hY
    have hX : V_0.mem X := x.sub hxX
    rw [toElementMap_sSupMaps_principal hF] at hY'
    have hprinc : V_0.principal hX  <=  x := fun Z hZ => x.up_mem hxX hZ.1 hZ.2
    have hsub : supOnPrincipal F hF X hX  <=  V_1.sSup (Set.image (fun f => f.
    toElementMap x) F) (hF x) :=
      V_1.sSup_le _ (mapsBounded_principal hF hX) fun s hs => by
        obtain <f, hf, rfl> := hs
        exact (toElementMap_mono f hprinc).trans (V_1.le_sSup _ (hF x) <f, hf, rfl>)
    exact hsub Y hY'
  * -- ` sqsup {f(x)}  sqsub  ( sqsup F)(x)`: `( sqsup F)(x)` is an upper bound of
    every `f(x)` since `f  sqsub   sqsup F` (3.13(i)).
    refine V_1.sSup_le _ (hF x) fun s hs => ?_
    obtain <f, hf, rfl> := hs
    exact le_iff_toElementMap_le.mp (le_sSupMaps hF hf) x

/-- **Theorem 3.13(iii) (Scott 1981, PRG-19).** When `F` is bounded, `( sqsup F)(x) =
    sqsup {f(x) ? f  in  F}`
(stated with the boundedness hypothesis in Scott's `MapsBounded` form). -/
theorem toElementMap_sSupMaps' {F : Set (ApproximableMap V_0 V_1)} (hF : MapsBounded F
    ) (x : V_0.Element) :
    (sSupMaps F (mapsBounded_iff_pointwiseBounded.mp hF)).toElementMap x =
      V_1.sSup (Set.image (fun f => f.toElementMap x) F)
        (mapsBounded_iff_pointwiseBounded.mp hF x) :=
  toElementMap_sSupMaps (mapsBounded_iff_pointwiseBounded.mp hF) x

theorem toFilter_le_iff {f g : ApproximableMap V_0 V_1} : toFilter f  <=  toFilter g
    <->  f  <=  g :=
  (funSpaceEquiv V_0 V_1).symm.map_rel_iff'

theorem mapsBounded_to_filters {F : Set (ApproximableMap V_0 V_1)} (h : MapsBounded F)
     :
```

```
    (funSpace V_0 V_1).Bounded (Set.image toFilter F) := by
  obtain <h, hh> := h
  refine <toFilter h, fun phi hphi => ?_>
  obtain <f, hf, rfl> := hphi
  exact (toFilter_le_iff).mpr (hh f hf)

end
end DomainTheory.NeighbourhoodSystems
end
```

3.4.15 Exercise 3.14 — tagged product of tokens

For the most part we can assume that there is at most a countable number of tokens; thus, without loss of generality the underlying sets Δ_i of given systems $\mathcal{D}_i$ could be assumed to be subsets of Σ^* where $\Sigma = \{0, 1\}$. (Any denumerable set would do.) Show that the product $\mathcal{D}_0 \times \mathcal{D}_1$ could be defined as the system over the set $0\Delta_0 \cup 1\Delta_1$ where

$$\mathcal{D}_0 \times \mathcal{D}_1 = \{0X \cup 1Y \mid X \in \mathcal{D}_0 \text{ and } Y \in \mathcal{D}_1\}.$$

In other words, the assumption of the disjointness of Δ_0 and Δ_1 is unnecessary. Give, therefore, the revised definition of $\langle x, y\rangle$ for elements, and prove that for a single system $\mathcal{D}$, there exists an approximable mapping

$$\mathrm{diag} : \mathcal{D} \to \mathcal{D} \times \mathcal{D}$$

where $\mathrm{diag}(x) = \langle x, x\rangle$ for all $x \in |\mathcal{D}|$. Also extend the definition to a product of n-factors

$$\mathcal{D}_0 \times \mathcal{D}_1 \times \cdots \times \mathcal{D}_{n-1}$$

which will be a system over the set

$$\bigcup_{i<n} 1^i 0 \Delta_i.$$

Note that for a 2-termed product we simplify $10\Delta_1$ to $1\Delta_1$.

Exercise_3_14.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_4
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Definition_3_3
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_2
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.bot
@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap
variable {a : Type*} (V : NeighborhoodSystem a)
/- concordance: source L1410 EXERCISE 3.14 tagged product of tokens -/
/-- **Exercise 3.14 (Scott 1981, PRG-19).** The *diagonal* approximable mapping
```

```
`diag : ?  ->  ?  x  ?`, defined as the paired map `<I_?, I_?>` of the identity with
    itself. -/
def diag : ApproximableMap V (prod V V) := paired (idMap V) (idMap V)

end DomainTheory.NeighbourhoodSystems
end

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem ApproximableMap
variable {a : Type*} (V : NeighborhoodSystem a)

@[simp] theorem diag_rel {Z : Set a} {P : Set (a ? a)} :
    (diag V).rel Z P  <->
      (prod V V).mem P  /\  (idMap V).rel Z (Sum.inl ^-1' P)  /\  (idMap V).rel Z (Sum
    .inr ^-1' P) :=
  Iff.rfl

@[simp] theorem toElementMap_diag (x : V.Element) :
    (diag V).toElementMap x = pair x x := by
  rw [diag, toElementMap_paired, toElementMap_idMap]

/-- **Exercise 3.14 (Scott 1981, PRG-19).** Post-composing the diagonal with the two
    projections
returns the identity: `p_0  comp  diag = I` and `p_1  comp  diag = I`. -/
theorem proj_0_comp_diag : (proj_0 V V).comp (diag V) = idMap V :=
  proj_0_comp_paired (idMap V) (idMap V)
end

section
set_option linter.unusedVariables false

theorem example_314.{u_1} : forall  {a : Type u_1} (V : DomainTheory.
    NeighbourhoodSystems.NeighborhoodSystem a),
  @Eq (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V)
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp a (Sum a a) a V
      (@DomainTheory.NeighbourhoodSystems.prod a a V V) V (@DomainTheory.
    NeighbourhoodSystems.proj_0 a a V V)
      (@DomainTheory.NeighbourhoodSystems.diag a V))
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.idMap a V) :=
  @DomainTheory.NeighbourhoodSystems.proj_0_comp_diag
end

section
open NeighborhoodSystem ApproximableMap
variable {a : Type*} (V : NeighborhoodSystem a)

theorem proj_1_comp_diag : (proj_1 V V).comp (diag V) = idMap V :=
  proj_1_comp_paired (idMap V) (idMap V)
end
end DomainTheory.NeighbourhoodSystems
end
```

3.4.16 Exercise 3.15 — product isomorphisms

Establish the usual isomorphisms:

(i) $\mathcal{D}_0 \times \mathcal{D}_1 \cong \mathcal{D}_1 \times \mathcal{D}_0$;

(ii) $\mathcal{D}_0 \times (\mathcal{D}_1 \times \mathcal{D}_2) \cong (\mathcal{D}_0 \times \mathcal{D}_1) \times \mathcal{D}_2 \cong \mathcal{D}_0 \times \mathcal{D}_1 \times \mathcal{D}_2$.

How does the product of no factors fit in? Prove also:

(iii) $\mathcal{D}_0 \cong \mathcal{D}_0'$ and $\mathcal{D}_1 \cong \mathcal{D}_1'$ imply $\mathcal{D}_0 \times \mathcal{D}_1 \cong \mathcal{D}_0' \times \mathcal{D}_1'$.

Exercise_3_15.lean

```
module
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Exercise_3_14

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.DomainIso
@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem
variable {a b ? a' b' : Type*}
variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b} {V_2 :
    NeighborhoodSystem ?}
variable {V_0' : NeighborhoodSystem a'} {V_1' : NeighborhoodSystem b'}
/- concordance: source L1436 EXERCISE 3.15 product isomorphisms -/
/-- **Exercise 3.15(i) (Scott 1981, PRG-19).** The commutativity order-isomorphism
`|?_0  x  ?_1| ?o |?_1  x  ?_0|`, factored through Proposition 3.2 and the cartesian
    swap. -/
def prodCommD (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) :
    (prod V_0 V_1).Element ?o (prod V_1 V_0).Element :=
  (prodEquiv V_0 V_1).trans (OrderIso.prodComm.trans (prodEquiv V_1 V_0).symm)

/-- The product of two order isomorphisms, as an order isomorphism. -/
def prodCongrOrderIso {A B C D : Type*} [Preorder A] [Preorder B] [Preorder C] [
    Preorder D]
    (e_0 : A ?o B) (e_1 : C ?o D) : A  x  C ?o B  x  D where
  toFun p := (e_0 p.1, e_1 p.2)
  invFun q := (e_0.symm q.1, e_1.symm q.2)
  left_inv p := by simp
  right_inv q := by simp
  map_rel_iff' := by
    rintro <a, c> <a', c'>
    show (e_0 a, e_1 c)  <=  (e_0 a', e_1 c')  <->  (a, c)  <=  (a', c')
    rw [Prod.mk_le_mk, Prod.mk_le_mk, e_0.le_iff_le, e_1.le_iff_le]

/-- **Exercise 3.15(ii) (Scott 1981, PRG-19).** The associativity order-isomorphism
`|?_0  x  (?_1  x  ?_2)| ?o |(?_0  x  ?_1)  x  ?_2|`. -/
def prodAssocD (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) (V_2 :
    NeighborhoodSystem ?) :
    (prod V_0 (prod V_1 V_2)).Element ?o (prod (prod V_0 V_1) V_2).Element :=
  (prodEquiv V_0 (prod V_1 V_2)).trans <|
```

```
    (prodCongrOrderIso (OrderIso.refl V_0.Element) (prodEquiv V_1 V_2)).trans <|
      (OrderIso.prodAssoc V_0.Element V_1.Element V_2.Element).symm.trans <|
        (prodCongrOrderIso (prodEquiv V_0 V_1).symm (OrderIso.refl V_2.Element)).trans
          (prodEquiv (prod V_0 V_1) V_2).symm

/-- For a `Unique` second factor, `A  x  C ?o A` (forget the constant component). -/
def prodUniqueOrderIso (A C : Type*) [Preorder A] [Preorder C] [Unique C] : A  x  C ?o
     A where
  toFun p := p.1
  invFun a := (a, default)
  left_inv p := by
    have : (default : C) = p.2 := Subsingleton.elim _ _
    simp [this]
  right_inv _ := rfl
  map_rel_iff' := by
    rintro <a, c> <a', c'>
    simp only [Prod.mk_le_mk]
    exact <fun h => <h, le_of_eq (Subsingleton.elim c c')>, And.left>

/-- For a `Unique` first factor, `C  x  A ?o A` (forget the constant component). -/
def uniqueProdOrderIso (A C : Type*) [Preorder A] [Preorder C] [Unique C] : C  x  A ?o
     A where
  toFun p := p.2
  invFun a := (default, a)
  left_inv p := by
    have : (default : C) = p.1 := Subsingleton.elim _ _
    simp [this]
  right_inv _ := rfl
  map_rel_iff' := by
    rintro <c, a> <c', a'>
    simp only [Prod.mk_le_mk]
    exact <fun h => <le_of_eq (Subsingleton.elim c c'), h>, And.right>

/-- The **terminal domain** `?`: the neighbourhood system over `Unit` with the single
neighbourhood `Delta = univ`. Its domain `|?|` has exactly one element (`False = {Delta
    }`), so `?` is the
*product of no factors*. -/
def unitSys : NeighborhoodSystem Unit where
  mem X := X = Set.univ
  master := Set.univ
  master_nonempty := <(), Set.mem_univ _>
  master_mem := rfl
  inter_mem := by rintro X Y Z rfl rfl _ _; simp
  sub_master := by rintro X rfl; exact subset_rfl

/-- `|?|` is a subsingleton: every element is `False`. -/
theorem unitSys_element_eq (x : unitSys.Element) : x = unitSys.bot := by
  apply Element.ext
  intro Y
  constructor
  * intro hY; rw [mem_bot]; exact x.sub hY
  * intro hY; rw [mem_bot] at hY; subst hY; exact x.master_mem

instance : Unique unitSys.Element where
  default := unitSys.bot
  uniq := unitSys_element_eq
```

```
/-- **Exercise 3.15(iii) (Scott 1981, PRG-19).** Two domain isomorphisms induce one on
     the products:
`|?_0  x  ?_1| ?o |?_0'  x  ?_1'|`. -/
def prodCongrD (e_0 : V_0.Element ?o V_0'.Element) (e_1 : V_1.Element ?o V_1'.Element)
     :
    (prod V_0 V_1).Element ?o (prod V_0' V_1').Element :=
  (prodEquiv V_0 V_1).trans ((prodCongrOrderIso e_0 e_1).trans (prodEquiv V_0' V_1').
    symm)

/-- **Exercise 3.15 (empty product).** `?` is a right unit: `?  x  ?  ~=  ?`. -/
def prodUnitD (V_0 : NeighborhoodSystem a) :
    (prod V_0 unitSys).Element ?o V_0.Element :=
  (prodEquiv V_0 unitSys).trans (prodUniqueOrderIso _ _)

/-- **Exercise 3.15 (empty product).** `?` is a left unit: `?  x  ?  ~=  ?`. -/
def unitProdD (V_0 : NeighborhoodSystem a) :
    (prod unitSys V_0).Element ?o V_0.Element :=
  (prodEquiv unitSys V_0).trans (uniqueProdOrderIso _ _)

/-- **Exercise 3.15(iii).** `?_0  ~=  ?_0'` and `?_1  ~=  ?_1'` imply `?_0  x  ?_1  ~=
     ?_0'  x  ?_1'`. -/
theorem Isomorphic.prod (h_0 : V_0  ~= ? V_0') (h_1 : V_1  ~= ? V_1') : prod V_0 V_1
    ~= ? prod V_0' V_1' :=
  h_0.elim fun e_0 => h_1.elim fun e_1 => <prodCongrD e_0 e_1>

/-- **Exercise 3.15(ii).** `?_0  x  (?_1  x  ?_2)  ~=  (?_0  x  ?_1)  x  ?_2`. -/
theorem prod_assoc_isomorphic : prod V_0 (prod V_1 V_2)  ~= ? prod (prod V_0 V_1) V_2
    :=
  <prodAssocD V_0 V_1 V_2>

/-- **Exercise 3.15(i).** `?_0  x  ?_1  ~=  ?_1  x  ?_0`. -/
theorem prod_comm_isomorphic : prod V_0 V_1  ~= ? prod V_1 V_0 := <prodCommD V_0 V_1>
end

section
set_option linter.unusedVariables false

theorem example_315.{u_1, u_2, u_4, u_5, u_3} : And
  (And
    (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem a}
      {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b},
      @DomainTheory.NeighbourhoodSystems.Isomorphic (Sum a b) (Sum b a)
        (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) (@DomainTheory.
    NeighbourhoodSystems.prod b a V_1 V_0))
    (forall  {a : Type u_1} {b : Type u_2} {a' : Type u_4} {b' : Type u_5}
      {V_0 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
      {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
      {V_0' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a'}
      {V_1' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b'},
      @DomainTheory.NeighbourhoodSystems.Isomorphic a a' V_0 V_0'  ->
        @DomainTheory.NeighbourhoodSystems.Isomorphic b b' V_1 V_1'  ->
          @DomainTheory.NeighbourhoodSystems.Isomorphic (Sum a b) (Sum a' b')
            (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1)
            (@DomainTheory.NeighbourhoodSystems.prod a' b' V_0' V_1')))
```

```
(forall  {a : Type u_1} {b : Type u_2} {? : Type u_3} {V_0 : DomainTheory.
  NeighbourhoodSystems.NeighborhoodSystem a}
  {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
  {V_2 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem ?},
  @DomainTheory.NeighbourhoodSystems.Isomorphic (Sum a (Sum b ?)) (Sum (Sum a b) ?)
    (@DomainTheory.NeighbourhoodSystems.prod a (Sum b ?) V_0 (@DomainTheory.
  NeighbourhoodSystems.prod b ? V_1 V_2))
    (@DomainTheory.NeighbourhoodSystems.prod (Sum a b) ? (@DomainTheory.
  NeighbourhoodSystems.prod a b V_0 V_1) V_2)) :=
@And.intro
(And
  (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem a}
    {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b},
    @DomainTheory.NeighbourhoodSystems.Isomorphic (Sum a b) (Sum b a)
      (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) (@DomainTheory.
  NeighbourhoodSystems.prod b a V_1 V_0))
  (forall  {a : Type u_1} {b : Type u_2} {a' : Type u_4} {b' : Type u_5}
    {V_0 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
    {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
    {V_0' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a'}
    {V_1' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b'},
    @DomainTheory.NeighbourhoodSystems.Isomorphic a a' V_0 V_0'  ->
      @DomainTheory.NeighbourhoodSystems.Isomorphic b b' V_1 V_1'  ->
        @DomainTheory.NeighbourhoodSystems.Isomorphic (Sum a b) (Sum a' b')
          (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1)
          (@DomainTheory.NeighbourhoodSystems.prod a' b' V_0' V_1')))
(forall  {a : Type u_1} {b : Type u_2} {? : Type u_3} {V_0 : DomainTheory.
  NeighbourhoodSystems.NeighborhoodSystem a}
  {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
  {V_2 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem ?},
  @DomainTheory.NeighbourhoodSystems.Isomorphic (Sum a (Sum b ?)) (Sum (Sum a b) ?)
    (@DomainTheory.NeighbourhoodSystems.prod a (Sum b ?) V_0 (@DomainTheory.
  NeighbourhoodSystems.prod b ? V_1 V_2))
    (@DomainTheory.NeighbourhoodSystems.prod (Sum a b) ? (@DomainTheory.
  NeighbourhoodSystems.prod a b V_0 V_1) V_2))
(@And.intro
  (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem a}
    {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b},
    @DomainTheory.NeighbourhoodSystems.Isomorphic (Sum a b) (Sum b a)
      (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) (@DomainTheory.
  NeighbourhoodSystems.prod b a V_1 V_0))
  (forall  {a : Type u_1} {b : Type u_2} {a' : Type u_4} {b' : Type u_5}
    {V_0 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
    {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
    {V_0' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a'}
    {V_1' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b'},
    @DomainTheory.NeighbourhoodSystems.Isomorphic a a' V_0 V_0'  ->
      @DomainTheory.NeighbourhoodSystems.Isomorphic b b' V_1 V_1'  ->
        @DomainTheory.NeighbourhoodSystems.Isomorphic (Sum a b) (Sum a' b')
          (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) (@DomainTheory.
  NeighbourhoodSystems.prod a' b' V_0' V_1'))
  @DomainTheory.NeighbourhoodSystems.prod_comm_isomorphic @DomainTheory.
  NeighbourhoodSystems.Isomorphic.prod)
@DomainTheory.NeighbourhoodSystems.prod_assoc_isomorphic
```

```
end

section
open NeighborhoodSystem
variable {a b ? a' b' : Type*}
variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b} {V_2 :
    NeighborhoodSystem ?}
variable {V_0' : NeighborhoodSystem a'} {V_1' : NeighborhoodSystem b'}

theorem prod_unit_isomorphic : prod V_0 unitSys  ~= ? V_0 := <prodUnitD V_0>

theorem unit_prod_isomorphic : prod unitSys V_0  ~= ? V_0 := <unitProdD V_0>
end
end DomainTheory.NeighbourhoodSystems
end
```

3.4.17 Exercise 3.16 — infinite sequences of elements

Let $\mathcal{D}$ be a given neighbourhood system over $\Delta \subseteq \Sigma^*$. Define

$$\Delta^\infty = \bigcup_{n=0}^{\infty} 1^n 0 \Delta,$$

so that Δ^∞ is split into infinitely many disjoint copies of Δ. Let $\mathcal{D}^\infty$ be the least family of subsets of Σ^* where

1. $\Delta^\infty \in \mathcal{D}^\infty$, and
2. whenever $X \in \mathcal{D}$ and $Y \in \mathcal{D}^\infty$, then $0X \cup 1Y \in \mathcal{D}^\infty$.

Show that $\mathcal{D}^\infty$ is a neighbourhood system over Δ^∞. Prove the isomorphism

$$\mathcal{D}^\infty \cong \mathcal{D} \times \mathcal{D}^\infty.$$

Show, moreover, that the elements of $|\mathcal{D}^\infty|$ are in a one-one correspondence with arbitrary infinite sequences $\langle x_n \rangle_{n=0}^{\infty}$ of elements $x_n \in |\mathcal{D}|$ by using combinations of neighbourhoods

$$0X_0 \cup 10X_1 \cup \cdots \cup 1^n 0X_n \cup \cdots,$$

where from some point on all the X_m are equal to Δ.

Exercise_3_16.lean

```
module
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Exercise_3_15

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_5
@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
```

```
open NeighborhoodSystem
variable {a : Type*}
/- concordance: source L1448 EXERCISE 3.16 infinite sequences of elements -/
/-- The `i`-th *fiber* of a set `W  subseteq  Nat  x  a`: the tokens appearing in copy
    `i`. -/
def fiber (W : Set (Nat  x  a)) (i : Nat) : Set a := {a | (i, a)  in  W}

@[simp] theorem mem_fiber {W : Set (Nat  x  a)} {i : Nat} {a : a} : a  in  fiber W i
    <->  (i, a)  in  W := Iff.rfl

theorem eq_of_fiber_eq {W W' : Set (Nat  x  a)} (h : forall  i, fiber W i = fiber W' i
    ) : W = W' := by
  ext <i, a>
  exact Set.ext_iff.mp (h i) a

theorem fiber_inter (W W' : Set (Nat  x  a)) (i : Nat) :
    fiber (W  I  W') i = fiber W i  I  fiber W' i := rfl

theorem fiber_mono {W W' : Set (Nat  x  a)} (h : W  subseteq  W') (i : Nat) : fiber W
    i  subseteq  fiber W' i :=
  fun _ ha => h ha
end

section
open NeighborhoodSystem
variable {a : Type*}
variable (V : NeighborhoodSystem a)

/-- The `?^ inf `-neighbourhood pinning copy `n` to `X` and leaving all other copies at
     `Delta`: Scott's
`Delta^ inf   I  (1?0X)`, i.e. the combination with `X_n = X` and `X? = Delta` for `m
    !=  n`. -/
def single (n : Nat) (X : Set a) : Set (Nat  x  a) :=
  {p | if p.1 = n then p.2  in  X else p.2  in  V.master}
end

section
open NeighborhoodSystem
variable {a : Type*}
variable (V : NeighborhoodSystem a)
variable {V}

@[simp] theorem fiber_single_self (n : Nat) (X : Set a) : fiber (single V n X) n = X
    := by
  ext a; simp [fiber, single]

theorem fiber_single_ne {n i : Nat} (h : i  !=  n) (X : Set a) : fiber (single V n X)
    i = V.master := by
  ext a; simp [fiber, single, h]

end

section
open NeighborhoodSystem
variable {a : Type*}
variable (V : NeighborhoodSystem a)
```

```
variable {V}

/-- **Exercise 3.16 (Scott 1981, PRG-19).** The infinite iterate `?^ inf ` over `Delta^
     inf  = Nat  x  Delta`:
`W  in  ?^ inf ` iff every fiber is a neighbourhood of `?` and all but finitely many
    fibers equal `Delta`. -/
def iterSys (V : NeighborhoodSystem a) : NeighborhoodSystem (Nat  x  a) where
  mem W := (forall  i, V.mem (fiber W i))  /\  exists  N, forall  i, N  <=  i  ->
    fiber W i = V.master
  master := {p | p.2  in  V.master}
  master_nonempty := by
    obtain <a, ha> := V.master_nonempty
    exact <(0, a), ha>
  master_mem := <fun _ => V.master_mem, 0, fun _ _ => rfl>
  inter_mem := by
    rintro W W' Z <hWf, NW, hNW> <hW'f, NW', hNW'> <hZf, _> hsub
    refine <fun i => ?_, max NW NW', fun i hi => ?_>
    * rw [fiber_inter]
      exact V.inter_mem (hWf i) (hW'f i) (hZf i) (fiber_mono hsub i)
    * rw [fiber_inter, hNW i (le_trans (le_max_left _ _) hi),
        hNW' i (le_trans (le_max_right _ _) hi), Set.inter_self]
  sub_master := by
    rintro W <hWf, _> <i, a> ha
    exact V.sub_master (hWf i) ha

@[simp] theorem iterSys_master : (iterSys V).master = {p : Nat  x  a | p.2  in  V.
    master} := rfl

theorem fiber_iterSys_master (i : Nat) : fiber ((iterSys V).master) i = V.master :=
    rfl

@[simp] theorem mem_iterSys {W : Set (Nat  x  a)} :
    (iterSys V).mem W  <->  (forall  i, V.mem (fiber W i))  /\  exists  N, forall  i,
    N  <=  i  ->  fiber W i = V.master := Iff.rfl

theorem interUpTo_subset_master (F : Nat  ->  Set (Nat  x  a)) (N : Nat) :
    (iterSys V).interUpTo F N  subseteq  (iterSys V).master := by
  induction N with
  | zero => exact subset_rfl
  | succ n ih => rw [NeighborhoodSystem.interUpTo_succ]; exact Set.inter_subset_left.
    trans ih

/-- The finite intersection `?_{i<N} single i (fiber W i)` reconstructs `W` from below,
     once `N`
exceeds the cofinite-`Delta` bound of `W`. -/
theorem reconstruct_subset {W : Set (Nat  x  a)} (_hW : (iterSys V).mem W) {N : Nat}
    (hN : forall  i, N  <=  i  ->  fiber W i = V.master) :
    (iterSys V).interUpTo (fun i => single V i (fiber W i)) N  subseteq  W := by
  rintro <j, a> ha
  by_cases h : j < N
  * have hsub := (iterSys V).interUpTo_subset (fun i => single V i (fiber W i)) h
    have hmem : (j, a)  in  single V j (fiber W j) := hsub ha
    simpa [single] using hmem
  * have haM : a  in  V.master := interUpTo_subset_master (fun i => single V i (fiber
    W i)) N ha
    have : a  in  fiber W j := by rw [hN j (not_lt.mp h)]; exact haM
```

```
    exact this

theorem single_inter {n : Nat} (X X' : Set a) :
    single V n X  I  single V n X' = single V n (X  I  X') := by
  apply eq_of_fiber_eq
  intro i
  by_cases h : i = n
  * subst h; rw [fiber_inter, fiber_single_self, fiber_single_self, fiber_single_self]
  * rw [fiber_inter, fiber_single_ne h, fiber_single_ne h, fiber_single_ne h, Set.
    inter_self]

theorem single_master (n : Nat) : single V n V.master = (iterSys V).master := by
  apply eq_of_fiber_eq
  intro i
  rw [fiber_iterSys_master]
  by_cases h : i = n
  * subst h; rw [fiber_single_self]
  * rw [fiber_single_ne h]

/-- `single V n X` is a `?^ inf `-neighbourhood whenever `X  in  ?`. -/
theorem single_mem {n : Nat} {X : Set a} (hX : V.mem X) : (iterSys V).mem (single V n
    X) := by
  refine <fun i => ?_, n + 1, fun i hi => ?_>
  * by_cases h : i = n
    * subst h; rw [fiber_single_self]; exact hX
    * rw [fiber_single_ne h]; exact V.master_mem
  * exact fiber_single_ne (by omega : i  !=  n) X

theorem single_mono {n : Nat} {X X' : Set a} (h : X  subseteq  X') : single V n X
    subseteq  single V n X' := by
  intro p hp
  simp only [single, Set.mem_ofPred_eq] at hp  |-
  by_cases hc : p.1 = n
  * rw [ite_eq_left hc] at hp  |- ; exact h hp
  * rw [ite_eq_right hc] at hp  |- ; exact hp

/-- The `n`-th component `x_n  in  |?|` of a `?^ inf `-element `z` (Scott's coordinate
    at copy `n`). -/
def component (z : (iterSys V).Element) (n : Nat) : V.Element where
  mem X := V.mem X  /\  z.mem (single V n X)
  sub h := h.1
  master_mem := <V.master_mem, by rw [single_master]; exact z.master_mem>
  inter_mem := by
    rintro X X' <_, hzX> <_, hzX'>
    have hz : z.mem (single V n (X  I  X')) := by rw [ <-  single_inter]; exact z.
    inter_mem hzX hzX'
    have hmem : V.mem (X  I  X') := by
      have := (z.sub hz).1 n; rwa [fiber_single_self] at this
    exact <hmem, hz>
  up_mem := by
    rintro X X' <_, hzX> hX' hXX'
    exact <hX', z.up_mem hzX (single_mem hX') (single_mono hXX')>

@[simp] theorem mem_component {z : (iterSys V).Element} {n : Nat} {X : Set a} :
    (component z n).mem X  <->  V.mem X  /\  z.mem (single V n X) := Iff.rfl
```

```
/-- The `?^ inf `-element determined by an infinite sequence `<x_n>` of `?`-elements:
    the neighbourhoods
`W` whose every fiber lies in the corresponding `x_i`. -/
def ofSeq (seq : Nat  ->  V.Element) : (iterSys V).Element where
  mem W := (iterSys V).mem W  /\  forall  i, (seq i).mem (fiber W i)
  sub h := h.1
  master_mem := <(iterSys V).master_mem, fun i => by
    rw [fiber_iterSys_master]; exact (seq i).master_mem>
  inter_mem := by
    rintro W W' <hW, hWf> <hW', hW'f>
    refine <<fun i => ?_, ?_>, fun i => ?_>
    * rw [fiber_inter]; exact (seq i).sub ((seq i).inter_mem (hWf i) (hW'f i))
    * obtain <NW, hNW> := hW.2
      obtain <NW', hNW'> := hW'.2
      refine <max NW NW', fun i hi => ?_>
      rw [fiber_inter, hNW i (le_trans (le_max_left _ _) hi),
        hNW' i (le_trans (le_max_right _ _) hi), Set.inter_self]
    * rw [fiber_inter]; exact (seq i).inter_mem (hWf i) (hW'f i)
  up_mem := by
    rintro W W' <_, hWf> hW' hWW'
    exact <hW', fun i => (seq i).up_mem (hWf i) (hW'.1 i) (fiber_mono hWW' i)>

@[simp] theorem mem_ofSeq {seq : Nat  ->  V.Element} {W : Set (Nat  x  a)} :
    (ofSeq seq).mem W  <->  (iterSys V).mem W  /\  forall  i, (seq i).mem (fiber W i)
    := Iff.rfl

@[simp] theorem component_ofSeq (seq : Nat  ->  V.Element) (n : Nat) :
    component (ofSeq seq) n = seq n := by
  apply Element.ext
  intro X
  constructor
  * rintro <_, _, hfib>
    have := hfib n; rwa [fiber_single_self] at this
  * intro hX
    refine <(seq n).sub hX, single_mem ((seq n).sub hX), fun i => ?_>
    by_cases h : i = n
    * subst h; rw [fiber_single_self]; exact hX
    * rw [fiber_single_ne h]; exact (seq i).master_mem

theorem ofSeq_mono {seq seq' : Nat  ->  V.Element} (h : forall  n, seq n  <=  seq' n)
    :
    ofSeq seq  <=  ofSeq seq' := by
  rintro W <hW, hf>
  exact <hW, fun i => h i _ (hf i)>

/-- Every `?^ inf `-neighbourhood is contained in the pinning of its own `i`-th fiber.
    -/
theorem subset_single {W : Set (Nat  x  a)} (hW : (iterSys V).mem W) (i : Nat) :
    W  subseteq  single V i (fiber W i) := by
  rintro <j, a> ha
  simp only [single, Set.mem_ofPred_eq]
  by_cases h : j = i
  * rw [ite_eq_left h]; subst h; exact ha
  * rw [ite_eq_right h]; exact V.sub_master (hW.1 j) ha

@[simp] theorem ofSeq_component (z : (iterSys V).Element) :
```

```
    ofSeq (fun n => component z n) = z := by
  apply Element.ext
  intro W
  constructor
  * rintro <hW, hfib>
    obtain <N, hN> := hW.2
    exact z.up_mem (z.mem_interUpTo _ (n := N) (fun i _ => (hfib i).2)) hW
      (reconstruct_subset hW hN)
  * intro hzW
    refine <z.sub hzW, fun i => ?_>
    exact <(z.sub hzW).1 i, z.up_mem hzW (single_mem ((z.sub hzW).1 i)) (subset_single
     (z.sub hzW) i)>

/-- `z  sqsub  z'` is detected component-wise. -/
theorem le_of_component_le {z z' : (iterSys V).Element}
    (h : forall  n, component z n  <=  component z' n) : z  <=  z' := by
  have hmono := ofSeq_mono h
  rwa [ofSeq_component, ofSeq_component] at hmono

/-- **Exercise 3.16 (Scott 1981, PRG-19).** The elements of `|?^ inf |` are in one-one,
order-preserving correspondence with infinite sequences `<x_n>` of elements of `|?|`.
    -/
def iterSeqEquiv (V : NeighborhoodSystem a) : (iterSys V).Element ?o (forall  _ : Nat,
     V.Element) where
  toFun z := fun n => component z n
  invFun seq := ofSeq seq
  left_inv z := ofSeq_component z
  right_inv seq := by funext n; exact component_ofSeq seq n
  map_rel_iff' := by
    intro z z'
    constructor
    * intro h
      exact le_of_component_le (fun n => h n)
    * intro h n X hX
      exact <hX.1, h (single V n X) hX.2>

/-- The shift order-isomorphism `(Nat  ->  E) ?o E  x  (Nat  ->  E)`, `f  |->  (f 0, f
     comp  succ)`. -/
def natShiftEquiv (E : Type*) [Preorder E] : (Nat  ->  E) ?o E  x  (Nat  ->  E) where
  toFun f := (f 0, fun n => f (n + 1))
  invFun p := fun n => Nat.casesOn n p.1 (fun m => p.2 m)
  left_inv f := by funext n; cases n <;> rfl
  right_inv p := rfl
  map_rel_iff' := by
    intro f g
    constructor
    * rintro <h0, hs>
      intro n
      cases n with
      | zero => exact h0
      | succ m => exact hs m
    * intro h
      exact <h 0, fun m => h (m + 1)>

/-- **Exercise 3.16 (Scott 1981, PRG-19).** The isomorphism `|?^ inf | ?o |?  x  ?^ inf
     |`, obtained from the
```

```
sequence correspondence and the shift. -/
def iterProdIso (V : NeighborhoodSystem a) :
    (iterSys V).Element ?o (prod V (iterSys V)).Element :=
  (iterSeqEquiv V).trans <|
    (natShiftEquiv V.Element).trans <|
      (prodCongrOrderIso (OrderIso.refl V.Element) (iterSeqEquiv V).symm).trans
        (prodEquiv V (iterSys V)).symm
end

section
open NeighborhoodSystem
variable {a : Type*}
variable (V : NeighborhoodSystem a)
variable {V}
open ApproximableMap

/-- The `n`-th coordinate projection `projN n : ?^ inf   ->  ?`, `W (projN n) X  <->
    fiber W n  subseteq  X`. -/
def projN (V : NeighborhoodSystem a) (n : Nat) : ApproximableMap (iterSys V) V where
  rel W X := (iterSys V).mem W  /\  V.mem X  /\  fiber W n  subseteq  X
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := <(iterSys V).master_mem, V.master_mem, by rw [fiber_iterSys_master]>
  inter_right := by
    rintro W X X' <hW, hX, hsub> <_, hX', hsub'>
    exact <hW, V.inter_mem hX hX' (hW.1 n) (Set.subset_inter hsub hsub'),
      Set.subset_inter hsub hsub'>
  mono := by
    rintro W W_2 X X_2 <_, _, hsub> hW_2W hXX_2 hW_2 hX_2
    exact <hW_2, hX_2, (fiber_mono hW_2W n).trans (hsub.trans hXX_2)>
end

section
open NeighborhoodSystem
variable {a : Type*}

theorem subset_of_fiber_subset {W W' : Set (Nat  x  a)} (h : forall  i, fiber W i
    subseteq  fiber W' i) :
    W  subseteq  W' := fun <i, _> ha => h i ha

variable (V : NeighborhoodSystem a)

variable {V}

open ApproximableMap

@[simp] theorem projN_rel {n : Nat} {W : Set (Nat  x  a)} {X : Set a} :
    (projN V n).rel W X  <->  (iterSys V).mem W  /\  V.mem X  /\  fiber W n  subseteq
     X := Iff.rfl

@[simp] theorem toElementMap_projN (z : (iterSys V).Element) (n : Nat) :
    (projN V n).toElementMap z = component z n := by
  apply Element.ext
  intro X
  constructor
  * rintro <W, hzW, hW, hX, hsub>
```

```
    refine <hX, z.up_mem hzW (single_mem hX) ?_>
    refine subset_of_fiber_subset (fun i => ?_)
    by_cases h : i = n
    * subst h; rw [fiber_single_self]; exact hsub
    * rw [fiber_single_ne h]; exact V.sub_master (hW.1 i)
  * rintro <hX, hz>
    exact <single V n X, hz, single_mem hX, hX, by rw [fiber_single_self]>
end

section
open NeighborhoodSystem
variable {a : Type*}
variable (V : NeighborhoodSystem a)
variable {V}

/-- **Exercise 3.16 (Scott 1981, PRG-19).** `?^ inf   ~=  ?  x  ?^ inf `. -/
theorem iter_isomorphic (V : NeighborhoodSystem a) : iterSys V  ~= ? prod V (iterSys V
    ) :=
  <iterProdIso V>
end

section
set_option linter.unusedVariables false

theorem example_316.{u_1} : forall  {a : Type u_1} (V : DomainTheory.
    NeighbourhoodSystems.NeighborhoodSystem a),
  @DomainTheory.NeighbourhoodSystems.Isomorphic (Prod Nat a) (Sum a (Prod Nat a))
    (@DomainTheory.NeighbourhoodSystems.iterSys a V)
    (@DomainTheory.NeighbourhoodSystems.prod a (Prod Nat a) V (@DomainTheory.
    NeighbourhoodSystems.iterSys a V)) :=
  @DomainTheory.NeighbourhoodSystems.iter_isomorphic
end
end DomainTheory.NeighbourhoodSystems
end
```

3.4.18 Exercise 3.17 — embedding binary sequences

Using the $\mathcal{B}$ and $\mathcal{T}$ of Example 2.3 show there is a one-one approximable mapping

$$f : \mathcal{B} \to \mathcal{T}^\infty$$

and another approximable mapping

$$g : \mathcal{T}^\infty \to \mathcal{B}$$

such that

$$g \circ f = I_{\mathcal{B}} \quad \text{and} \quad f \circ g \subseteq I_{\mathcal{T}^\infty}.$$

Are $\mathcal{B}$ and $\mathcal{T}^\infty$ isomorphic? Are $\mathcal{B}$ and $\mathcal{T} \times \mathcal{B}$ isomorphic?

Exercise_3_17.lean

```
module
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Exercise_3_16
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Definition_3_8
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Examples_1_2_1_5_revisited
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems.Exercise317
section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ExampleB ApproximableMap

/-- The truth domain `T` of Example 1.2. -/
abbrev T : NeighborhoodSystem Example12.Token := Example12.neighborhoodSystem

/-- Tokens of `T`: `{0, 1}`. -/
abbrev Token := Example12.Token

/-- `T`-neighbourhoods are non-empty (`{}  notin  T`). -/
theorem T_nbhd_nonempty {F : Set Token} (hF : T.mem F) : F.Nonempty := by
  rcases (Example12.mem_iff F).mp hF with rfl | rfl | rfl
  * exact <0, by simp [Example12.master]>
  * exact <0, by simp [Example12.zero]>
  * exact <1, by simp [Example12.one]>

/-- The `T`-neighbourhood selected by a bit: `false  |->  {0}`, `true  |->  {1}`. -/
def bitNbhd : Bool  ->  Set Token
  | false => Example12.zero
  | true => Example12.one

theorem bitNbhd_mem (b : Bool) : T.mem (bitNbhd b) := by
  cases b
  * exact Example12.mem_zero
  * exact Example12.mem_one

/-- `encSet sigma  subseteq  Nat  x  {0,1}`: pin copy `i` to `bitNbhd sigma[i]` for `i
    < |sigma|`, free otherwise. -/
def encSet (sigma : Str) : Set (Nat  x  Token) := {p | forall  b, sigma[p.1]? = some b
      ->  p.2  in  bitNbhd b}

@[simp] theorem mem_encSet {sigma : Str} {p : Nat  x  Token} :
    p  in  encSet sigma  <->  forall  b, sigma[p.1]? = some b  ->  p.2  in  bitNbhd b
    := Iff.rfl

@[simp] theorem encSet_nil : encSet [] = Set.univ := by
  ext p; simp [encSet]

theorem comparable_of_agree : forall  {tau tau' : Str},
    (forall  (i : Nat) (b b' : Bool), tau[i]? = some b  ->  tau'[i]? = some b'  ->  b
    = b')  ->  tau <+: tau'  \/  tau' <+: tau
  | [], _, _ => Or.inl (List.nil_prefix)
  | _ :: _, [], _ => Or.inr (List.nil_prefix)
  | a :: s, a' :: s', hag => by
      have ha : a = a' := hag 0 a a' (by simp) (by simp)
```

```
      subst ha
      have hrec : forall  (i : Nat) (b b' : Bool), s[i]? = some b  ->  s'[i]? = some b
    '  ->  b = b' := by
        intro i b b' hb hb'
        exact hag (i + 1) b b' (by simpa using hb) (by simpa using hb')
      rcases comparable_of_agree hrec with h | h
      * exact Or.inl (List.cons_prefix_cons.mpr <rfl, h>)
      * exact Or.inr (List.cons_prefix_cons.mpr <rfl, h>)

/-- Two prefixes that both bound a common non-empty `T^ inf `-neighbourhood agree where
     both defined. -/
theorem encSet_agree {tau tau' : Str} {W : Set (Nat  x  Token)} (hW : (iterSys T).mem
    W)
    (h : W  subseteq  encSet tau) (h' : W  subseteq  encSet tau') {i : Nat} {b b' :
    Bool}
    (hb : tau[i]? = some b) (hb' : tau'[i]? = some b') : b = b' := by
  obtain <t, ht> := T_nbhd_nonempty (hW.1 i)
  have h1 : t  in  bitNbhd b := (h ht) b hb
  have h2 : t  in  bitNbhd b' := (h' ht) b' hb'
  by_contra hne
  cases b <;> cases b' <;> simp_all [bitNbhd, Example12.zero, Example12.one]

/-- A longer prefix encodes to a *smaller* set: `sigma ? sigma' ? encSet sigma'
    subseteq  encSet sigma`. -/
theorem encSet_anti {sigma sigma' : Str} (h : sigma <+: sigma') : encSet sigma'
    subseteq  encSet sigma := by
  obtain <t, rfl> := h
  rintro <i, x> hp b hb
  have hi : i < sigma.length := by
    obtain <hlt, -> := List.getElem?_eq_some_iff.mp hb; exact hlt
  exact hp b (by rw [List.getElem?_append_left hi]; exact hb)

theorem fiber_encSet_none {sigma : Str} {i : Nat} (h : sigma[i]? = none) :
    fiber (encSet sigma) i = Set.univ := by
  ext t
  simp only [mem_fiber, mem_encSet, Set.mem_univ, iff_true]
  intro b hb; rw [h] at hb; exact absurd hb (by simp)

theorem fiber_encSet_some {sigma : Str} {i : Nat} {b : Bool} (h : sigma[i]? = some b)
    :
    fiber (encSet sigma) i = bitNbhd b := by
  ext t
  simp only [mem_fiber, mem_encSet]
  constructor
  * intro hh; exact hh b h
  * intro ht b' hb'
    rw [h] at hb'
    obtain rfl := Option.some.inj hb'
    exact ht

/-- `encSet sigma` is a `T^ inf `-neighbourhood. -/
theorem encSet_mem (sigma : Str) : (iterSys T).mem (encSet sigma) := by
  refine <fun i => ?_, sigma.length, fun i hi => ?_>
  * cases h : sigma[i]? with
    | none => rw [fiber_encSet_none h]; exact T.master_mem
    | some b => rw [fiber_encSet_some h]; exact bitNbhd_mem b
```

```
  * rw [fiber_encSet_none (List.getElem?_eq_none_iff.mpr hi)]; rfl

/-- **The encoding map `f : B  ->  T^ inf `.** Relates the cone `sigmaSigma*` to every
    `T^ inf `-neighbourhood weaker
than `encSet sigma`. -/
def f : ApproximableMap B (iterSys T) where
  rel X W := exists  sigma, X = cone sigma  /\  (iterSys T).mem W  /\  encSet sigma
    subseteq  W
  rel_dom := fun <sigma, hX, _> => hX  |>  memB_cone sigma
  rel_cod := fun <_, _, hW, _> => hW
  master_rel := <[], cone_nil.symm, (iterSys T).master_mem, by rw [encSet_nil]; exact
    Set.subset_univ _>
  inter_right := by
    rintro X W W' <sigma, hX, hWmem, hWsub> <sigma', hX', hW'mem, hW'sub>
    obtain rfl : sigma = sigma' := cone_injective (hX.symm.trans hX')
    exact <sigma, hX, (iterSys T).inter_mem hWmem hW'mem (encSet_mem sigma)
      (Set.subset_inter hWsub hW'sub), Set.subset_inter hWsub hW'sub>
  mono := by
    rintro X X' W W' <sigma, hX, _, hWsub> hX'X hWW' hX' hW'mem
    obtain <sigma', rfl> := hX'
    subst hX
    have hpre : sigma <+: sigma' := cone_subset_cone.mp hX'X
    exact <sigma', rfl, hW'mem, (encSet_anti hpre).trans (hWsub.trans hWW')>

/-- **The decoding map `g : T^ inf   ->  B`.** Relates `W` to the cone `tauSigma*`
    exactly when `W  subseteq  encSet tau`,
i.e. `W` pins the first `|tau|` copies to `tau`. -/
def g : ApproximableMap (iterSys T) B where
  rel W X := (iterSys T).mem W  /\  exists  tau, X = cone tau  /\  W  subseteq  encSet
     tau
  rel_dom h := h.1
  rel_cod := fun <_, tau, hX, _> => hX  |>  memB_cone tau
  master_rel := <(iterSys T).master_mem, [], cone_nil.symm, by rw [encSet_nil]; exact
    Set.subset_univ _>
  inter_right := by
    rintro W X X' <hW, tau, hX, hWtau> <_, tau', hX', hWtau'>
    subst hX; subst hX'
    rcases comparable_of_agree (fun i b b' => encSet_agree hW hWtau hWtau') with hpre
    | hpre
    * exact <hW, tau', by rw [Set.inter_eq_right.mpr (cone_subset_cone.mpr hpre)],
    hWtau'>
    * exact <hW, tau, by rw [Set.inter_eq_left.mpr (cone_subset_cone.mpr hpre)], hWtau
    >
  mono := by
    rintro W W_2 X X_2 <_, tau, hX, hWtau> hW_2W hXX_2 hW_2 hX_2
    obtain <tau_2, rfl> := hX_2
    subst hX
    have hpre : tau_2 <+: tau := cone_subset_cone.mp hXX_2
    exact <hW_2, tau_2, rfl, hW_2W.trans (hWtau.trans (encSet_anti hpre))>

theorem exists_not_mem_bitNbhd (b : Bool) : exists  t : Token, t  notin  bitNbhd b :=
    by
  cases b
  * exact <1, by simp [bitNbhd, Example12.zero]>
  * exact <0, by simp [bitNbhd, Example12.one]>
```

```
/- concordance: source L1473 EXERCISE 3.17 embedding binary sequences -/
/-- **Exercise 3.17 -- `f` is a section of `g`.** `f  comp  g  sqsub  I_{T^ inf }`. -/
theorem fg_le_id : f.comp g  <=  idMap (iterSys T) := by
  intro W Z
  rw [comp_rel, idMap_rel]
  rintro <X, <hW, tau, hX, hWtau>, sigma, hXsigma, hZmem, hsigmaZ>
  obtain rfl : tau = sigma := cone_injective (hX.symm.trans hXsigma)
  exact <hW, hZmem, hWtau.trans hsigmaZ>

/-- **Prefix recovery.** `encSet sigma  subseteq  encSet tau` forces `tau` to be an
    initial segment of `sigma`. -/
theorem prefix_of_encSet_subset {sigma tau : Str} (h : encSet sigma  subseteq  encSet
    tau) : tau <+: sigma := by
  have hag : forall  (i : Nat) (b b' : Bool), sigma[i]? = some b  ->  tau[i]? = some b
    '  ->  b = b' :=
    fun i b b' => encSet_agree (encSet_mem sigma) subset_rfl h
  rcases comparable_of_agree hag with hsigmatau | htausigma
  * by_cases hlen : sigma.length = tau.length
    * exact (hsigmatau.eq_of_length hlen)  |>  List.prefix_rfl
    * exfalso
      have hlt : sigma.length < tau.length := lt_of_le_of_ne hsigmatau.length_le hlen
      have hb' : tau[sigma.length]? = some tau[sigma.length] := List.getElem?
    _eq_getElem hlt
      obtain <t, ht> := exists_not_mem_bitNbhd tau[sigma.length]
      have hmemsigma : (sigma.length, t)  in  encSet sigma := by
        intro b hb
        rw [List.getElem?_eq_none_iff.mpr (le_refl _)] at hb
        exact absurd hb (by simp)
      exact ht ((h hmemsigma) _ hb')
  * exact htausigma

/-- **Exercise 3.17 -- `g` is a retraction of `f`.** `g  comp  f = I_B`. -/
theorem gf_eq_id : g.comp f = idMap B := by
  apply ApproximableMap.ext
  intro X Y
  rw [comp_rel, idMap_rel]
  constructor
  * rintro <W, <sigma, hX, _, hsigmaW>, _, tau, hY, hWtau>
    refine <hX  |>  memB_cone sigma, hY  |>  memB_cone tau, ?_>
    have hsub : encSet sigma  subseteq  encSet tau := hsigmaW.trans hWtau
    rw [hX, hY]
    exact cone_subset_cone.mpr (prefix_of_encSet_subset hsub)
  * rintro <hX, hY, hXY>
    obtain <sigma, rfl> := hX
    obtain <tau, rfl> := hY
    have hpre : tau <+: sigma := cone_subset_cone.mp hXY
    exact <encSet sigma, <sigma, rfl, encSet_mem sigma, subset_rfl>,
      encSet_mem sigma, tau, rfl, encSet_anti hpre>

section
set_option linter.unusedVariables false

theorem example_317 : And
  (@LE.le
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Prod Nat DomainTheory.
    NeighbourhoodSystems.Example12.Token)
```

```
    (Prod Nat DomainTheory.NeighbourhoodSystems.Example12.Token)
    (@DomainTheory.NeighbourhoodSystems.iterSys DomainTheory.NeighbourhoodSystems.
  Example12.Token
      DomainTheory.NeighbourhoodSystems.Exercise317.T)
    (@DomainTheory.NeighbourhoodSystems.iterSys DomainTheory.NeighbourhoodSystems.
  Example12.Token
      DomainTheory.NeighbourhoodSystems.Exercise317.T))
   (@Preorder.toLE
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Prod Nat DomainTheory.
  NeighbourhoodSystems.Example12.Token)
      (Prod Nat DomainTheory.NeighbourhoodSystems.Example12.Token)
      (@DomainTheory.NeighbourhoodSystems.iterSys DomainTheory.NeighbourhoodSystems.
  Example12.Token
        DomainTheory.NeighbourhoodSystems.Exercise317.T)
      (@DomainTheory.NeighbourhoodSystems.iterSys DomainTheory.NeighbourhoodSystems.
  Example12.Token
        DomainTheory.NeighbourhoodSystems.Exercise317.T))
    (@PartialOrder.toPreorder
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Prod Nat DomainTheory.
  NeighbourhoodSystems.Example12.Token)
        (Prod Nat DomainTheory.NeighbourhoodSystems.Example12.Token)
        (@DomainTheory.NeighbourhoodSystems.iterSys DomainTheory.
  NeighbourhoodSystems.Example12.Token
          DomainTheory.NeighbourhoodSystems.Exercise317.T)
        (@DomainTheory.NeighbourhoodSystems.iterSys DomainTheory.
  NeighbourhoodSystems.Example12.Token
          DomainTheory.NeighbourhoodSystems.Exercise317.T))
      (@DomainTheory.NeighbourhoodSystems.instPartialOrderApproximableMap
        (Prod Nat DomainTheory.NeighbourhoodSystems.Example12.Token)
        (Prod Nat DomainTheory.NeighbourhoodSystems.Example12.Token)
        (@DomainTheory.NeighbourhoodSystems.iterSys DomainTheory.
  NeighbourhoodSystems.Example12.Token
          DomainTheory.NeighbourhoodSystems.Exercise317.T)
        (@DomainTheory.NeighbourhoodSystems.iterSys DomainTheory.
  NeighbourhoodSystems.Example12.Token
          DomainTheory.NeighbourhoodSystems.Exercise317.T))))
   (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp
    (Prod Nat DomainTheory.NeighbourhoodSystems.Example12.Token) DomainTheory.
  NeighbourhoodSystems.ExampleB.Str
    (Prod Nat DomainTheory.NeighbourhoodSystems.Example12.Token)
    (@DomainTheory.NeighbourhoodSystems.iterSys DomainTheory.NeighbourhoodSystems.
  Example12.Token
      DomainTheory.NeighbourhoodSystems.Exercise317.T)
    DomainTheory.NeighbourhoodSystems.ExampleB.B
    (@DomainTheory.NeighbourhoodSystems.iterSys DomainTheory.NeighbourhoodSystems.
  Example12.Token
      DomainTheory.NeighbourhoodSystems.Exercise317.T)
    DomainTheory.NeighbourhoodSystems.Exercise317.f DomainTheory.
  NeighbourhoodSystems.Exercise317.g)
   (@DomainTheory.NeighbourhoodSystems.ApproximableMap.idMap
    (Prod Nat DomainTheory.NeighbourhoodSystems.Example12.Token)
    (@DomainTheory.NeighbourhoodSystems.iterSys DomainTheory.NeighbourhoodSystems.
  Example12.Token
      DomainTheory.NeighbourhoodSystems.Exercise317.T)))
(@Eq
  (@DomainTheory.NeighbourhoodSystems.ApproximableMap DomainTheory.
```

```
  NeighbourhoodSystems.ExampleB.Str
    DomainTheory.NeighbourhoodSystems.ExampleB.Str DomainTheory.NeighbourhoodSystems
  .ExampleB.B
    DomainTheory.NeighbourhoodSystems.ExampleB.B)
   (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp DomainTheory.
  NeighbourhoodSystems.ExampleB.Str
     (Prod Nat DomainTheory.NeighbourhoodSystems.Example12.Token) DomainTheory.
  NeighbourhoodSystems.ExampleB.Str
    DomainTheory.NeighbourhoodSystems.ExampleB.B
    (@DomainTheory.NeighbourhoodSystems.iterSys DomainTheory.NeighbourhoodSystems.
  Example12.Token
      DomainTheory.NeighbourhoodSystems.Exercise317.T)
    DomainTheory.NeighbourhoodSystems.ExampleB.B DomainTheory.NeighbourhoodSystems.
  Exercise317.g
    DomainTheory.NeighbourhoodSystems.Exercise317.f)
   (@DomainTheory.NeighbourhoodSystems.ApproximableMap.idMap DomainTheory.
  NeighbourhoodSystems.ExampleB.Str
    DomainTheory.NeighbourhoodSystems.ExampleB.B)) :=
@And.intro
(@LE.le
  (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Prod Nat DomainTheory.
  NeighbourhoodSystems.Example12.Token)
    (Prod Nat DomainTheory.NeighbourhoodSystems.Example12.Token)
    (@DomainTheory.NeighbourhoodSystems.iterSys DomainTheory.NeighbourhoodSystems.
  Example12.Token
      DomainTheory.NeighbourhoodSystems.Exercise317.T)
    (@DomainTheory.NeighbourhoodSystems.iterSys DomainTheory.NeighbourhoodSystems.
  Example12.Token
      DomainTheory.NeighbourhoodSystems.Exercise317.T))
   (@Preorder.toLE
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Prod Nat DomainTheory.
  NeighbourhoodSystems.Example12.Token)
      (Prod Nat DomainTheory.NeighbourhoodSystems.Example12.Token)
      (@DomainTheory.NeighbourhoodSystems.iterSys DomainTheory.NeighbourhoodSystems.
  Example12.Token
        DomainTheory.NeighbourhoodSystems.Exercise317.T)
      (@DomainTheory.NeighbourhoodSystems.iterSys DomainTheory.NeighbourhoodSystems.
  Example12.Token
        DomainTheory.NeighbourhoodSystems.Exercise317.T))
    (@PartialOrder.toPreorder
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Prod Nat DomainTheory.
  NeighbourhoodSystems.Example12.Token)
        (Prod Nat DomainTheory.NeighbourhoodSystems.Example12.Token)
        (@DomainTheory.NeighbourhoodSystems.iterSys DomainTheory.
  NeighbourhoodSystems.Example12.Token
          DomainTheory.NeighbourhoodSystems.Exercise317.T)
        (@DomainTheory.NeighbourhoodSystems.iterSys DomainTheory.
  NeighbourhoodSystems.Example12.Token
          DomainTheory.NeighbourhoodSystems.Exercise317.T))
      (@DomainTheory.NeighbourhoodSystems.instPartialOrderApproximableMap
        (Prod Nat DomainTheory.NeighbourhoodSystems.Example12.Token)
        (Prod Nat DomainTheory.NeighbourhoodSystems.Example12.Token)
        (@DomainTheory.NeighbourhoodSystems.iterSys DomainTheory.
  NeighbourhoodSystems.Example12.Token
          DomainTheory.NeighbourhoodSystems.Exercise317.T)
        (@DomainTheory.NeighbourhoodSystems.iterSys DomainTheory.
```

```
    NeighbourhoodSystems.Example12.Token
            DomainTheory.NeighbourhoodSystems.Exercise317.T))))
     (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp
       (Prod Nat DomainTheory.NeighbourhoodSystems.Example12.Token) DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
       (Prod Nat DomainTheory.NeighbourhoodSystems.Example12.Token)
       (@DomainTheory.NeighbourhoodSystems.iterSys DomainTheory.NeighbourhoodSystems.
    Example12.Token
         DomainTheory.NeighbourhoodSystems.Exercise317.T)
       DomainTheory.NeighbourhoodSystems.ExampleB.B
       (@DomainTheory.NeighbourhoodSystems.iterSys DomainTheory.NeighbourhoodSystems.
    Example12.Token
         DomainTheory.NeighbourhoodSystems.Exercise317.T)
       DomainTheory.NeighbourhoodSystems.Exercise317.f DomainTheory.
    NeighbourhoodSystems.Exercise317.g)
     (@DomainTheory.NeighbourhoodSystems.ApproximableMap.idMap
       (Prod Nat DomainTheory.NeighbourhoodSystems.Example12.Token)
       (@DomainTheory.NeighbourhoodSystems.iterSys DomainTheory.NeighbourhoodSystems.
    Example12.Token
         DomainTheory.NeighbourhoodSystems.Exercise317.T)))
  (@Eq
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
      DomainTheory.NeighbourhoodSystems.ExampleB.Str DomainTheory.NeighbourhoodSystems
    .ExampleB.B
      DomainTheory.NeighbourhoodSystems.ExampleB.B)
     (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
       (Prod Nat DomainTheory.NeighbourhoodSystems.Example12.Token) DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
       DomainTheory.NeighbourhoodSystems.ExampleB.B
       (@DomainTheory.NeighbourhoodSystems.iterSys DomainTheory.NeighbourhoodSystems.
    Example12.Token
         DomainTheory.NeighbourhoodSystems.Exercise317.T)
       DomainTheory.NeighbourhoodSystems.ExampleB.B DomainTheory.NeighbourhoodSystems.
    Exercise317.g
       DomainTheory.NeighbourhoodSystems.Exercise317.f)
     (@DomainTheory.NeighbourhoodSystems.ApproximableMap.idMap DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
       DomainTheory.NeighbourhoodSystems.ExampleB.B))
  DomainTheory.NeighbourhoodSystems.Exercise317.fg_le_id DomainTheory.
    NeighbourhoodSystems.Exercise317.gf_eq_id
end

/-- **`f` is one-one.** Since `g  comp  f = I_B`, the elementwise map of `f` is
    injective. -/
theorem f_injective : Function.Injective f.toElementMap := by
  intro x y hxy
  have : (g.comp f).toElementMap x = (g.comp f).toElementMap y := by
    rw [toElementMap_comp, toElementMap_comp, hxy]
  rwa [gf_eq_id, toElementMap_idMap, toElementMap_idMap] at this
end

end DomainTheory.NeighbourhoodSystems.Exercise317
end
```

3.4.19 Exercise 3.18 — separated sum

Let $\mathcal{D}_0$ and $\mathcal{D}_1$ be neighbourhood systems over Δ_0 and Δ_1, where we again assume that these are subsets of Σ^*. We assume that in addition *no neighbourhood is empty*. Why is this possible without loss of generality?

Define the *sum system* by:

$$\mathcal{D}_0 + \mathcal{D}_1 = \{\{\Lambda\} \cup 0\Delta_0 \cup 1\Delta_1\} \cup \{0X \mid X \in \mathcal{D}_0\} \cup \{1Y \mid Y \in \mathcal{D}_1\}.$$

Prove that this is a neighbourhood system over $\{\Lambda\} \cup 0\Delta_0 \cup 1\Delta_1$. (Throwing in $\{\Lambda\}$ was not all that necessary, but note that $\mathcal{B} = \mathcal{B} + \mathcal{B}$, and this is an equality of sets not just an isomorphism of systems.)

Prove that in general there are mappings

$$in_i : \mathcal{D}_i \to \mathcal{D}_0 + \mathcal{D}_1 \quad \text{and} \quad out_i : \mathcal{D}_0 + \mathcal{D}_1 \to \mathcal{D}_i,$$

where $out_i \circ in_i = I_{\mathcal{D}_i}$. Where does the assumption $\emptyset \notin \mathcal{D}_i$ come in? How can these sums be generalized to n-terms? (Hint: As for products use sets $1^i 0\Delta_i$.) Draw some pictures.

Exercise_3_18.lean

```
module
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Theorem_2_5

public import Mathlib.Data.Set.Operations
@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem ApproximableMap
variable {a b : Type*}

/-- Left tag `0a = some (inl a)`. -/
def il (a : a) : Option (a ? b) := some (Sum.inl a)

/-- The tagged left copy `0X = {some (inl a) ? a  in  X}`. -/
def inj_0 (X : Set a) : Set (Option (a ? b)) := il '' X

@[simp] theorem il_mem_inj_0 {X : Set a} {a : a} : (il a : Option (a ? b))  in  inj_0
    X  <->  a  in  X := by
  simp only [inj_0, Set.mem_image, il]
  constructor
  * rintro <a', ha', hb>; simp only [Option.some.injEq, Sum.inl.injEq] at hb; exact hb
      |>  ha'
  * intro ha; exact <a, ha, rfl>

/-- Right tag `1b = some (inr b)`. -/
def ir (b : b) : Option (a ? b) := some (Sum.inr b)

/-- The tagged right copy `1Y = {some (inr b) ? b  in  Y}`. -/
def inj_1 (Y : Set b) : Set (Option (a ? b)) := ir '' Y

@[simp] theorem ir_mem_inj_0 {X : Set a} {b : b} : (ir b : Option (a ? b))  notin
    inj_0 X := by
  rintro <a, _, hb>; exact absurd hb (by simp [il, ir])
```

```
@[simp] theorem none_mem_inj_0 {X : Set a} : (none : Option (a ? b))  notin  inj_0 X
    := by
  rintro <a, _, hb>; exact absurd hb (by simp [il])

@[simp] theorem ir_mem_inj_1 {Y : Set b} {b : b} : (ir b : Option (a ? b))  in  inj_1
    Y  <->  b  in  Y := by
  simp only [inj_1, Set.mem_image, ir]
  constructor
  * rintro <b', hb', hb>; simp only [Option.some.injEq, Sum.inr.injEq] at hb; exact hb
      |>  hb'
  * intro hb; exact <b, hb, rfl>

@[simp] theorem il_mem_inj_1 {Y : Set b} {a : a} : (il a : Option (a ? b))  notin
    inj_1 Y := by
  rintro <b, _, hb>; exact absurd hb (by simp [il, ir])

@[simp] theorem none_mem_inj_1 {Y : Set b} : (none : Option (a ? b))  notin  inj_1 Y
    := by
  rintro <b, _, hb>; exact absurd hb (by simp [ir])

theorem inj_0_inter (X X' : Set a) :
    (inj_0 X  I  inj_0 X' : Set (Option (a ? b))) = inj_0 (X  I  X') := by
  ext t; rcases t with _ | (a | b) <;>
    simp [Set.mem_inter_iff, il, inj_0]

theorem inj_0_inter_inj_1 (X : Set a) (Y : Set b) :
    (inj_0 X  I  inj_1 Y : Set (Option (a ? b))) = {} := by
  rw [Set.eq_empty_iff_forall_notMem]
  rintro t <ht0, ht1>
  rcases t with _ | (a | b)
  * exact none_mem_inj_0 ht0
  * exact il_mem_inj_1 ht1
  * exact ir_mem_inj_0 ht0

theorem inj_0_nonempty {X : Set a} (hX : X.Nonempty) : (inj_0 X : Set (Option (a ? b))
    ).Nonempty :=
  Set.Nonempty.image il hX

theorem inj_0_subset_inj_0 {X X' : Set a} :
    (inj_0 X : Set (Option (a ? b)))  subseteq  inj_0 X'  <->  X  subseteq  X' := by
  constructor
  * intro h a ha; exact il_mem_inj_0.mp (h (il_mem_inj_0.mpr ha))
  * intro h t ht
    rw [inj_0, Set.mem_image] at ht
    obtain <a, ha, rfl> := ht
    exact il_mem_inj_0.mpr (h ha)

theorem inj_1_inter (Y Y' : Set b) :
    (inj_1 Y  I  inj_1 Y' : Set (Option (a ? b))) = inj_1 (Y  I  Y') := by
  ext t; rcases t with _ | (a | b) <;>
    simp [Set.mem_inter_iff, ir, inj_1]

theorem inj_1_nonempty {Y : Set b} (hY : Y.Nonempty) : (inj_1 Y : Set (Option (a ? b))
    ).Nonempty :=
  Set.Nonempty.image ir hY
```

```
theorem inj_1_subset_inj_1 {Y Y' : Set b} :
    (inj_1 Y : Set (Option (a ? b)))  subseteq  inj_1 Y'  <->  Y  subseteq  Y' := by
  constructor
  * intro h b hb; exact ir_mem_inj_1.mp (h (ir_mem_inj_1.mpr hb))
  * intro h t ht
    rw [inj_1, Set.mem_image] at ht
    obtain <b, hb, rfl> := ht
    exact ir_mem_inj_1.mpr (h hb)
end

section
open NeighborhoodSystem ApproximableMap
variable {a b : Type*}
variable (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b)

/-- The master neighbourhood of the sum: `{?}  U  0Delta_0  U  1Delta_1`. -/
def sumMaster : Set (Option (a ? b)) := insert none (inj_0 V_0.master  U  inj_1 V_1.
    master)
end

section
open NeighborhoodSystem ApproximableMap
variable {a b : Type*}
variable (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b)
variable {V_0 V_1}

@[simp] theorem none_mem_sumMaster : (none : Option (a ? b))  in  sumMaster V_0 V_1 :=
  Set.mem_insert _ _

theorem il_injective : Function.Injective (il : a  ->  Option (a ? b)) := by
  intro a a' h; simpa [il] using h

@[simp] theorem il_preimage_inj_0 (X : Set a) :
    ((il : a  ->  Option (a ? b)) ^-1' inj_0 X) = X :=
  Set.preimage_image_eq X il_injective

theorem inj_0_subset_sumMaster {X : Set a} (hX : V_0.mem X) :
    (inj_0 X : Set (Option (a ? b)))  subseteq  sumMaster V_0 V_1 := by
  intro t ht
  refine Set.mem_insert_iff.mpr (Or.inr (Set.mem_union_left _ ?_))
  exact (inj_0_subset_inj_0.mpr (V_0.sub_master hX)) ht

theorem inj_1_subset_sumMaster {Y : Set b} (hY : V_1.mem Y) :
    (inj_1 Y : Set (Option (a ? b)))  subseteq  sumMaster V_0 V_1 := by
  intro t ht
  refine Set.mem_insert_iff.mpr (Or.inr (Set.mem_union_right _ ?_))
  exact (inj_1_subset_inj_1.mpr (V_1.sub_master hY)) ht

theorem ir_injective : Function.Injective (ir : b  ->  Option (a ? b)) := by
  intro b b' h; simpa [ir] using h

@[simp] theorem ir_preimage_inj_1 (Y : Set b) :
    ((ir : b  ->  Option (a ? b)) ^-1' inj_1 Y) = Y :=
  Set.preimage_image_eq Y ir_injective

/-- The left content `leftPart W  subseteq  Delta_0` of a sum-neighbourhood `W`: the
```

```
    `0`-tagged tokens of `W`,
*plus* all of `Delta_0` whenever `W` reaches into the right copy or the basepoint (so
    non-left
neighbourhoods contribute only `Delta_0`, i.e. project to `False`). This is a genuine (
    choice-free) function
of `W`. -/
def leftPart (V_0 : NeighborhoodSystem a) (W : Set (Option (a ? b))) : Set a :=
  il ^-1' W  U  {a | a  in  V_0.master  /\  ((exists  b : b, ir b  in  W)  \/  (none :
     Option (a ? b))  in  W)}

@[simp] theorem mem_leftPart {V_0 : NeighborhoodSystem a} {W : Set (Option (a ? b))} {
    a : a} :
    a  in  leftPart V_0 W  <->  il a  in  W  \/  (a  in  V_0.master  /\  ((exists  b :
     b, ir b  in  W)  \/  none  in  W)) := by
  simp only [leftPart, Set.mem_union, Set.mem_preimage, Set.mem_ofPred_eq]

/-- The right content `rightPart W  subseteq  Delta_1`, symmetric to `leftPart`. -/
def rightPart (V_1 : NeighborhoodSystem b) (W : Set (Option (a ? b))) : Set b :=
  ir ^-1' W  U  {b | b  in  V_1.master  /\  ((exists  a : a, il a  in  W)  \/  (none :
     Option (a ? b))  in  W)}

@[simp] theorem mem_rightPart {V_1 : NeighborhoodSystem b} {W : Set (Option (a ? b))}
    {b : b} :
    b  in  rightPart V_1 W  <->  ir b  in  W  \/  (b  in  V_1.master  /\  ((exists  a
    : a, il a  in  W)  \/  none  in  W)) := by
  simp only [rightPart, Set.mem_union, Set.mem_preimage, Set.mem_ofPred_eq]

@[simp] theorem leftPart_inj_0 (V_0 : NeighborhoodSystem a) (X : Set a) :
    leftPart V_0 (inj_0 X : Set (Option (a ? b))) = X := by
  ext a
  simp only [mem_leftPart, il_mem_inj_0, ir_mem_inj_0, none_mem_inj_0, exists_false,
    or_self,
    and_false]
  exact <fun h => h.resolve_right (by simp), fun h => Or.inl h>

theorem leftPart_mono (V_0 : NeighborhoodSystem a) {W W' : Set (Option (a ? b))} (h :
    W  subseteq  W') :
    leftPart V_0 W  subseteq  leftPart V_0 W' := by
  intro a ha
  rw [mem_leftPart] at ha  |-
  exact ha.imp (fun h' => h h') (fun <hm, hc> => <hm, hc.imp (fun <b, hb> => <b, h hb
    >) (fun hn => h hn)>)

@[simp] theorem rightPart_inj_1 (V_1 : NeighborhoodSystem b) (Y : Set b) :
    rightPart V_1 (inj_1 Y : Set (Option (a ? b))) = Y := by
  ext b
  simp only [mem_rightPart, ir_mem_inj_1, il_mem_inj_1, none_mem_inj_1, exists_false,
    or_self,
    and_false]
  exact <fun h => h.resolve_right (by simp), fun h => Or.inl h>

theorem leftPart_inj_1 (V_0 : NeighborhoodSystem a) {Y : Set b} (hY : Y.Nonempty) :
    leftPart V_0 (inj_1 Y : Set (Option (a ? b))) = V_0.master := by
  ext a
  simp only [mem_leftPart, il_mem_inj_1, none_mem_inj_1, or_false, false_or]
  constructor
```

```
  * rintro <ha, _>; exact ha
  * intro ha; obtain <b, hb> := hY; exact <ha, <b, ir_mem_inj_1.mpr hb>>

theorem leftPart_sumMaster (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) :
    leftPart V_0 (sumMaster V_0 V_1) = V_0.master := by
  ext a
  simp only [mem_leftPart, none_mem_sumMaster, or_true, and_true]
  constructor
  * rintro (h | h)
    * have : (il a : Option (a ? b))  in  sumMaster V_0 V_1 := h
      rcases Set.mem_insert_iff.mp this with h' | h'
      * exact absurd h' (by simp [il])
      * rcases h' with h'' | h''
        * exact (il_mem_inj_0).mp h''
        * exact absurd h'' (by simp)
    * exact h
  * intro ha; exact Or.inr ha

theorem rightPart_inj_0 (V_1 : NeighborhoodSystem b) {X : Set a} (hX : X.Nonempty) :
    rightPart V_1 (inj_0 X : Set (Option (a ? b))) = V_1.master := by
  ext b
  simp only [mem_rightPart, ir_mem_inj_0, none_mem_inj_0, or_false, false_or]
  constructor
  * rintro <hb, _>; exact hb
  * intro hb; obtain <a, ha> := hX; exact <hb, <a, il_mem_inj_0.mpr ha>>

theorem rightPart_mono (V_1 : NeighborhoodSystem b) {W W' : Set (Option (a ? b))} (h :
     W  subseteq  W') :
    rightPart V_1 W  subseteq  rightPart V_1 W' := by
  intro b hb
  rw [mem_rightPart] at hb  |-
  exact hb.imp (fun h' => h h') (fun <hm, hc> => <hm, hc.imp (fun <a, ha> => <a, h ha
    >) (fun hn => h hn)>)

theorem rightPart_sumMaster (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b)
    :
    rightPart V_1 (sumMaster V_0 V_1) = V_1.master := by
  ext b
  simp only [mem_rightPart, none_mem_sumMaster, or_true, and_true]
  constructor
  * rintro (h | h)
    * have : (ir b : Option (a ? b))  in  sumMaster V_0 V_1 := h
      rcases Set.mem_insert_iff.mp this with h' | h'
      * exact absurd h' (by simp [ir])
      * rcases h' with h'' | h''
        * exact absurd h'' (by simp)
        * exact (ir_mem_inj_1).mp h''
    * exact h
  * intro hb; exact Or.inr hb

theorem sumMaster_inter_inj_0 {X : Set a} (hX : V_0.mem X) :
    (sumMaster V_0 V_1  I  inj_0 X : Set (Option (a ? b))) = inj_0 X :=
  Set.inter_eq_right.mpr (inj_0_subset_sumMaster hX)

theorem sumMaster_inter_inj_1 {Y : Set b} (hY : V_1.mem Y) :
    (sumMaster V_0 V_1  I  inj_1 Y : Set (Option (a ? b))) = inj_1 Y :=
```

```
  Set.inter_eq_right.mpr (inj_1_subset_sumMaster hY)

/- concordance: source L1495 EXERCISE 3.18 separated sum -/
/-- **Exercise 3.18 (Scott 1981, PRG-19).** The *sum system* `?_0 + ?_1` over `{?}  U
    0Delta_0  U  1Delta_1`,
under the standing assumption that no neighbourhood of `?_0` or `?_1` is empty. -/
def sum (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b)
    (h_0 : forall  X, V_0.mem X  ->  X.Nonempty) (h_1 : forall  Y, V_1.mem Y  ->  Y.
    Nonempty) :
    NeighborhoodSystem (Option (a ? b)) where
  mem W := W = sumMaster V_0 V_1  \/  (exists  X, V_0.mem X  /\  W = inj_0 X)  \/  (
    exists  Y, V_1.mem Y  /\  W = inj_1 Y)
  master := sumMaster V_0 V_1
  master_nonempty := <none, none_mem_sumMaster>
  master_mem := Or.inl rfl
  sub_master := by
    rintro W (rfl | <X, hX, rfl> | <Y, hY, rfl>)
    * exact subset_rfl
    * exact inj_0_subset_sumMaster hX
    * exact inj_1_subset_sumMaster hY
  inter_mem := by
    -- every neighbourhood is non-empty, hence so is any consistency witness `Z`
    have hne : forall  W, (W = sumMaster V_0 V_1  \/  (exists  X, V_0.mem X  /\  W =
    inj_0 X)  \/
        (exists  Y, V_1.mem Y  /\  W = inj_1 Y))  ->  (W : Set (Option (a ? b))).
    Nonempty := by
      rintro W (rfl | <X, hX, rfl> | <Y, hY, rfl>)
      * exact <none, none_mem_sumMaster>
      * exact inj_0_nonempty (h_0 X hX)
      * exact inj_1_nonempty (h_1 Y hY)
    rintro W W' Z hW hW' hZ hZsub
    rcases hW with rfl | <X, hX, rfl> | <Y, hY, rfl>
    * rcases hW' with rfl | <X', hX', rfl> | <Y', hY', rfl>
      * rw [Set.inter_self]; exact Or.inl rfl
      * rw [sumMaster_inter_inj_0 hX']; exact Or.inr (Or.inl <X', hX', rfl>)
      * rw [sumMaster_inter_inj_1 hY']; exact Or.inr (Or.inr <Y', hY', rfl>)
    * rcases hW' with rfl | <X', hX', rfl> | <Y', hY', rfl>
      * rw [Set.inter_comm, sumMaster_inter_inj_0 hX]; exact Or.inr (Or.inl <X, hX,
    rfl>)
      * rw [inj_0_inter] at hZsub  |-
        -- witness `Z  subseteq  inj_0 (X  I  X')`; `Z` non-empty forces it to be a
    left copy `inj_0 Z_0`
        rcases hZ with rfl | <Z_0, hZ_0, rfl> | <Z_1, hZ_1, rfl>
        * exact absurd (hZsub none_mem_sumMaster) none_mem_inj_0
        * refine Or.inr (Or.inl <X  I  X', V_0.inter_mem hX hX' hZ_0 (
    inj_0_subset_inj_0.mp hZsub), rfl>)
        * obtain <b, hb> := h_1 Z_1 hZ_1
          exact absurd (hZsub (ir_mem_inj_1.mpr hb)) ir_mem_inj_0
      * rw [inj_0_inter_inj_1] at hZsub  |-
        obtain <t, ht> := hne Z hZ; exact absurd (hZsub ht) (Set.notMem_empty t)
    * rcases hW' with rfl | <X', hX', rfl> | <Y', hY', rfl>
      * rw [Set.inter_comm, sumMaster_inter_inj_1 hY]; exact Or.inr (Or.inr <Y, hY,
    rfl>)
      * rw [Set.inter_comm, inj_0_inter_inj_1] at hZsub  |-
        obtain <t, ht> := hne Z hZ; exact absurd (hZsub ht) (Set.notMem_empty t)
      * rw [inj_1_inter] at hZsub  |-
```

```
        rcases hZ with rfl | <Z_0, hZ_0, rfl> | <Z_1, hZ_1, rfl>
        * exact absurd (hZsub none_mem_sumMaster) none_mem_inj_1
        * obtain <a, ha> := h_0 Z_0 hZ_0
          exact absurd (hZsub (il_mem_inj_0.mpr ha)) il_mem_inj_1
        * refine Or.inr (Or.inr <Y  I  Y', V_1.inter_mem hY hY' hZ_1 (
    inj_1_subset_inj_1.mp hZsub), rfl>)
end

section
open NeighborhoodSystem ApproximableMap
variable {a b : Type*}
variable (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b)
variable {V_0 V_1}
variable {h_0 : forall  X, V_0.mem X  ->  X.Nonempty} {h_1 : forall  Y, V_1.mem Y  ->
     Y.Nonempty}

/-- **Exercise 3.18 (Scott 1981, PRG-19).** The left injection `in_0 : ?_0  ->  ?_0 + ?
    _1`,
`X (in_0) W  <->  0X  subseteq  W`. -/
def inMap_0 : ApproximableMap V_0 (sum V_0 V_1 h_0 h_1) where
  rel X W := V_0.mem X  /\  (sum V_0 V_1 h_0 h_1).mem W  /\  inj_0 X  subseteq  W
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := <V_0.master_mem, (sum V_0 V_1 h_0 h_1).master_mem,
    inj_0_subset_sumMaster V_0.master_mem>
  inter_right := by
    rintro X W W' <hX, hW, hsub> <_, hW', hsub'>
    exact <hX, (sum V_0 V_1 h_0 h_1).inter_mem hW hW' (Or.inr (Or.inl <X, hX, rfl>))
      (Set.subset_inter hsub hsub'), Set.subset_inter hsub hsub'>
  mono := by
    rintro X X' W W' <_, _, hsub> hX'X hWW' hX' hW'
    exact <hX', hW', (inj_0_subset_inj_0.mpr hX'X).trans (hsub.trans hWW')>

/-- **Exercise 3.18 (Scott 1981, PRG-19).** The right injection `in_1 : ?_1  ->  ?_0 +
    ?_1`. -/
def inMap_1 : ApproximableMap V_1 (sum V_0 V_1 h_0 h_1) where
  rel Y W := V_1.mem Y  /\  (sum V_0 V_1 h_0 h_1).mem W  /\  inj_1 Y  subseteq  W
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := <V_1.master_mem, (sum V_0 V_1 h_0 h_1).master_mem,
    inj_1_subset_sumMaster V_1.master_mem>
  inter_right := by
    rintro Y W W' <hY, hW, hsub> <_, hW', hsub'>
    exact <hY, (sum V_0 V_1 h_0 h_1).inter_mem hW hW' (Or.inr (Or.inr <Y, hY, rfl>))
      (Set.subset_inter hsub hsub'), Set.subset_inter hsub hsub'>
  mono := by
    rintro Y Y' W W' <_, _, hsub> hY'Y hWW' hY' hW'
    exact <hY', hW', (inj_1_subset_inj_1.mpr hY'Y).trans (hsub.trans hWW')>

theorem leftPart_mem {W : Set (Option (a ? b))} (hW : (sum V_0 V_1 h_0 h_1).mem W) :
    V_0.mem (leftPart V_0 W) := by
  rcases hW with rfl | <X, hX, rfl> | <Y, hY, rfl>
  * rw [leftPart_sumMaster]; exact V_0.master_mem
  * rw [leftPart_inj_0]; exact hX
  * rw [leftPart_inj_1 V_0 (h_1 Y hY)]; exact V_0.master_mem
```

```
/-- **Exercise 3.18 (Scott 1981, PRG-19).** The left projection `out_0 : ?_0 + ?_1  ->
     ?_0`,
`W (out_0) X  <->  leftPart W  subseteq  X` (right/basepoint neighbourhoods relate only
     to `Delta_0`). -/
def outMap_0 : ApproximableMap (sum V_0 V_1 h_0 h_1) V_0 where
  rel W X := (sum V_0 V_1 h_0 h_1).mem W  /\  V_0.mem X  /\  leftPart V_0 W  subseteq
     X
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := <(sum V_0 V_1 h_0 h_1).master_mem, V_0.master_mem, (leftPart_sumMaster
     V_0 V_1).subset>
  inter_right := by
    rintro W X X' <hW, hX, hsub> <_, hX', hsub'>
    exact <hW, V_0.inter_mem hX hX' (leftPart_mem hW) (Set.subset_inter hsub hsub'),
      Set.subset_inter hsub hsub'>
  mono := by
    rintro W W' X X' <_, _, hsub> hW'W hXX' hW' hX'
    exact <hW', hX', (leftPart_mono V_0 hW'W).trans (hsub.trans hXX')>

theorem rightPart_mem {W : Set (Option (a ? b))} (hW : (sum V_0 V_1 h_0 h_1).mem W) :
    V_1.mem (rightPart V_1 W) := by
  rcases hW with rfl | <X, hX, rfl> | <Y, hY, rfl>
  * rw [rightPart_sumMaster]; exact V_1.master_mem
  * rw [rightPart_inj_0 V_1 (h_0 X hX)]; exact V_1.master_mem
  * rw [rightPart_inj_1]; exact hY

/-- **Exercise 3.18 (Scott 1981, PRG-19).** The right projection `out_1 : ?_0 + ?_1  ->
      ?_1`. -/
def outMap_1 : ApproximableMap (sum V_0 V_1 h_0 h_1) V_1 where
  rel W Y := (sum V_0 V_1 h_0 h_1).mem W  /\  V_1.mem Y  /\  rightPart V_1 W  subseteq
      Y
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := <(sum V_0 V_1 h_0 h_1).master_mem, V_1.master_mem, (
    rightPart_sumMaster V_0 V_1).subset>
  inter_right := by
    rintro W Y Y' <hW, hY, hsub> <_, hY', hsub'>
    exact <hW, V_1.inter_mem hY hY' (rightPart_mem hW) (Set.subset_inter hsub hsub'),
      Set.subset_inter hsub hsub'>
  mono := by
    rintro W W' Y Y' <_, _, hsub> hW'W hYY' hW' hY'
    exact <hW', hY', (rightPart_mono V_1 hW'W).trans (hsub.trans hYY')>
end

section
open NeighborhoodSystem ApproximableMap
variable {a b : Type*}

theorem inj_0_injective {X X' : Set a} (h : (inj_0 X : Set (Option (a ? b))) = inj_0 X
    ') : X = X' :=
  Set.Subset.antisymm (inj_0_subset_inj_0.mp h.subset) (inj_0_subset_inj_0.mp h.symm.
    subset)

theorem inj_1_injective {Y Y' : Set b} (h : (inj_1 Y : Set (Option (a ? b))) = inj_1 Y
    ') : Y = Y' :=
  Set.Subset.antisymm (inj_1_subset_inj_1.mp h.subset) (inj_1_subset_inj_1.mp h.symm.
```

```
    subset)
end

section
open NeighborhoodSystem ApproximableMap
variable {a b : Type*}
variable (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b)
variable {V_0 V_1}
variable {h_0 : forall  X, V_0.mem X  ->  X.Nonempty} {h_1 : forall  Y, V_1.mem Y  ->
     Y.Nonempty}

/-- **Exercise 3.18 (Scott 1981, PRG-19).** `out_0  comp  in_0 = I_{?_0}`. -/
theorem outMap_0_comp_inMap_0 :
    (outMap_0 (V_0 := V_0) (V_1 := V_1) (h_0 := h_0) (h_1 := h_1)).comp inMap_0 =
    idMap V_0 := by
  apply ApproximableMap.ext
  intro X Z
  constructor
  * rintro <W, <hX, _, hinj>, _, hZ, hsub>
    refine <hX, hZ, ?_>
    have hXW : X  subseteq  leftPart V_0 W := by
      intro a ha
      rw [mem_leftPart]; exact Or.inl (hinj (il_mem_inj_0.mpr ha))
    exact hXW.trans hsub
  * rintro <hX, hZ, hXZ>
    refine <inj_0 X, <hX, Or.inr (Or.inl <X, hX, rfl>), subset_rfl>,
      Or.inr (Or.inl <X, hX, rfl>), hZ, ?_>

    rw [leftPart_inj_0]; exact hXZ

/-- **Exercise 3.18 (Scott 1981, PRG-19).** `out_1  comp  in_1 = I_{?_1}`. -/
theorem outMap_1_comp_inMap_1 :
    (outMap_1 (V_0 := V_0) (V_1 := V_1) (h_0 := h_0) (h_1 := h_1)).comp inMap_1 =
    idMap V_1 := by
  apply ApproximableMap.ext
  intro Y Z
  constructor
  * rintro <W, <hY, _, hinj>, _, hZ, hsub>
    refine <hY, hZ, ?_>
    have hYW : Y  subseteq  rightPart V_1 W := by
      intro b hb
      rw [mem_rightPart]; exact Or.inl (hinj (ir_mem_inj_1.mpr hb))
    exact hYW.trans hsub
  * rintro <hY, hZ, hYZ>
    refine <inj_1 Y, <hY, Or.inr (Or.inr <Y, hY, rfl>), subset_rfl>,
      Or.inr (Or.inr <Y, hY, rfl>), hZ, ?_>
    rw [rightPart_inj_1]; exact hYZ
end

section
set_option linter.unusedVariables false

theorem example_318.{u_1, u_2} : And
  (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem a}
    {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
```

```
  {h_0 : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  mem a V_0 X  ->  @Set.Nonempty a X}
  {h_1 : forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  mem b V_1 Y  ->  @Set.Nonempty b Y},
  @Eq (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V_0 V_0)
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp a (Option (Sum a b)) a
  V_0
      (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1) V_0
      (@DomainTheory.NeighbourhoodSystems.outMap_0 a b V_0 V_1 h_0 h_1)
      (@DomainTheory.NeighbourhoodSystems.inMap_0 a b V_0 V_1 h_0 h_1))
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.idMap a V_0))
(forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem a}
  {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
  {h_0 : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  mem a V_0 X  ->  @Set.Nonempty a X}
  {h_1 : forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  mem b V_1 Y  ->  @Set.Nonempty b Y},
  @Eq (@DomainTheory.NeighbourhoodSystems.ApproximableMap b b V_1 V_1)
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp b (Option (Sum a b)) b
  V_1
      (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1) V_1
      (@DomainTheory.NeighbourhoodSystems.outMap_1 a b V_0 V_1 h_0 h_1)
      (@DomainTheory.NeighbourhoodSystems.inMap_1 a b V_0 V_1 h_0 h_1))
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.idMap b V_1)) :=
@And.intro
(forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem a}
  {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
  {h_0 : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  mem a V_0 X  ->  @Set.Nonempty a X}
  {h_1 : forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  mem b V_1 Y  ->  @Set.Nonempty b Y},
  @Eq (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V_0 V_0)
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp a (Option (Sum a b)) a
  V_0
      (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1) V_0
      (@DomainTheory.NeighbourhoodSystems.outMap_0 a b V_0 V_1 h_0 h_1)
      (@DomainTheory.NeighbourhoodSystems.inMap_0 a b V_0 V_1 h_0 h_1))
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.idMap a V_0))
(forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem a}
  {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
  {h_0 : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  mem a V_0 X  ->  @Set.Nonempty a X}
  {h_1 : forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  mem b V_1 Y  ->  @Set.Nonempty b Y},
  @Eq (@DomainTheory.NeighbourhoodSystems.ApproximableMap b b V_1 V_1)
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp b (Option (Sum a b)) b
  V_1
      (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1) V_1
      (@DomainTheory.NeighbourhoodSystems.outMap_1 a b V_0 V_1 h_0 h_1)
      (@DomainTheory.NeighbourhoodSystems.inMap_1 a b V_0 V_1 h_0 h_1))
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.idMap b V_1))
@DomainTheory.NeighbourhoodSystems.outMap_0_comp_inMap_0 @DomainTheory.
  NeighbourhoodSystems.outMap_1_comp_inMap_1
```

```
end

section
open NeighborhoodSystem ApproximableMap
variable {a b a' b' : Type*}
variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
variable {V_0' : NeighborhoodSystem a'} {V_1' : NeighborhoodSystem b'}
variable {h_0 : forall  X, V_0.mem X  ->  X.Nonempty} {h_1 : forall  Y, V_1.mem Y  ->
     Y.Nonempty}
variable {h_0' : forall  X, V_0'.mem X  ->  X.Nonempty} {h_1' : forall  Y, V_1'.mem Y
     ->  Y.Nonempty}

/-- A sum-neighbourhood that *contains* the master is the master. -/
theorem eq_sumMaster_of_subset {W : Set (Option (a ? b))}
    (hW : (sum V_0 V_1 h_0 h_1).mem W) (hsub : sumMaster V_0 V_1  subseteq  W) :
    W = sumMaster V_0 V_1 :=
  Set.Subset.antisymm ((sum V_0 V_1 h_0 h_1).sub_master hW) hsub

/-- A sum-neighbourhood contained in a *left* copy `0X` is itself a left copy `0X_2` (
    the basepoint and
the right copy are excluded, using non-emptiness). -/
theorem mem_subset_inj_0 {W : Set (Option (a ? b))} {X : Set a}
    (hW : (sum V_0 V_1 h_0 h_1).mem W) (hsub : W  subseteq  inj_0 X) :
    exists  X_2, V_0.mem X_2  /\  W = inj_0 X_2 := by
  rcases hW with rfl | <X_2, hX_2, rfl> | <Y_2, hY_2, rfl>
  * exact absurd (hsub none_mem_sumMaster) none_mem_inj_0
  * exact <X_2, hX_2, rfl>
  * obtain <b, hb> := h_1 Y_2 hY_2
    exact absurd (hsub (ir_mem_inj_1.mpr hb)) ir_mem_inj_0

/-- A sum-neighbourhood contained in a *right* copy `1Y` is itself a right copy `1Y_2`.
     -/
theorem mem_subset_inj_1 {W : Set (Option (a ? b))} {Y : Set b}
    (hW : (sum V_0 V_1 h_0 h_1).mem W) (hsub : W  subseteq  inj_1 Y) :
    exists  Y_2, V_1.mem Y_2  /\  W = inj_1 Y_2 := by
  rcases hW with rfl | <X_2, hX_2, rfl> | <Y_2, hY_2, rfl>
  * exact absurd (hsub none_mem_sumMaster) none_mem_inj_1
  * obtain <a, ha> := h_0 X_2 hX_2
    exact absurd (hsub (il_mem_inj_0.mpr ha)) il_mem_inj_1
  * exact <Y_2, hY_2, rfl>

/-- A nonempty left copy is never contained in a right copy. -/
theorem not_inj_0_subset_inj_1 {X : Set a} {Y : Set b} (hX : X.Nonempty)
    (hsub : (inj_0 X : Set (Option (a ? b)))  subseteq  inj_1 Y) : False := by
  obtain <a, ha> := hX
  exact il_mem_inj_1 (hsub (il_mem_inj_0.mpr ha))

/-- A nonempty right copy is never contained in a left copy. -/
theorem not_inj_1_subset_inj_0 {X : Set a} {Y : Set b} (hY : Y.Nonempty)
    (hsub : (inj_1 Y : Set (Option (a ? b)))  subseteq  inj_0 X) : False := by
  obtain <b, hb> := hY
  exact ir_mem_inj_0 (hsub (ir_mem_inj_1.mpr hb))

end
end DomainTheory.NeighbourhoodSystems
end
```

3.4.20 Exercise 3.19 — induced maps on constructs

Suppose we are given systems and approximable mappings

$$f : \mathcal{D}_0 \to \mathcal{D}'_0 \quad \text{and} \quad g : \mathcal{D}_1 \to \mathcal{D}'_1.$$

Prove there are approximable mappings

$$f \times g : \mathcal{D}_0 \times \mathcal{D}_1 \to \mathcal{D}'_0 \times \mathcal{D}'_1 \quad \text{and} \quad f + g : \mathcal{D}_0 + \mathcal{D}_1 \to \mathcal{D}'_0 + \mathcal{D}'_1$$

such that

(i) $(f \times g)(x, y) = \langle f(x), g(y) \rangle$ for all $x \in |\mathcal{D}_0|$ and $y \in |\mathcal{D}_1|$, and rewrite this as:

(ii) $f \times g = \langle f \circ p_0, g \circ p_1 \rangle$.

In addition prove that

(iii) $out_0 \circ (f + g) \circ in_0 = f$, and

(iv) $out_1 \circ (f + g) \circ in_1 = g$.

Do equations (iii) and (iv) uniquely determine $f + g$?

Exercise_3_19.lean

```
module
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Exercise_3_18
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Definition_3_3
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_4
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem ApproximableMap
variable {a b a' b' : Type*}
variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
variable {V_0' : NeighborhoodSystem a'} {V_1' : NeighborhoodSystem b'}
variable {h_0 : forall  X, V_0.mem X  ->  X.Nonempty} {h_1 : forall  Y, V_1.mem Y  ->
     Y.Nonempty}
variable {h_0' : forall  X, V_0'.mem X  ->  X.Nonempty} {h_1' : forall  Y, V_1'.mem Y
     ->  Y.Nonempty}
/- concordance: source L1513 EXERCISE 3.19 induced maps on constructs -/
/-- **Exercise 3.19 (Scott 1981, PRG-19).** The *sum mapping* `f + g : ?_0 + ?_1  ->  ?
    _0' + ?_1'`. As a
relation between sum-neighbourhoods, `W (f+g) W'` holds iff `W'` is the codomain master
    , or `W = 0X`
with `W' = 0Y'` and `X f Y'`, or `W = 1Y` with `W' = 1Y'` and `Y g Y'`. -/
def sumMap (f : ApproximableMap V_0 V_0') (g : ApproximableMap V_1 V_1') :
    ApproximableMap (sum V_0 V_1 h_0 h_1) (sum V_0' V_1' h_0' h_1') where
  rel W W' := (sum V_0 V_1 h_0 h_1).mem W  /\  (sum V_0' V_1' h_0' h_1').mem W'  /\
    (W' = sumMaster V_0' V_1'  \/
```

```
    (exists  X Y', W = inj_0 X  /\  W' = inj_0 Y'  /\  f.rel X Y')  \/
    (exists  Y Y', W = inj_1 Y  /\  W' = inj_1 Y'  /\  g.rel Y Y'))
rel_dom h := h.1
rel_cod h := h.2.1
master_rel := <(sum V_0 V_1 h_0 h_1).master_mem, (sum V_0' V_1' h_0' h_1').
  master_mem, Or.inl rfl>
inter_right := by
  rintro W W'_1 W'_2 <hW, hW'_1, hd_1> <_, hW'_2, hd_2>
  -- membership of any disjunction-satisfying set
  have hmem : forall  W'' : Set (Option (a' ? b')),
      (W'' = sumMaster V_0' V_1'  \/
        (exists  X Y', W = inj_0 X  /\  W'' = inj_0 Y'  /\  f.rel X Y')  \/
        (exists  Y Y', W = inj_1 Y  /\  W'' = inj_1 Y'  /\  g.rel Y Y'))  ->
        (sum V_0' V_1' h_0' h_1').mem W'' := by
    rintro W'' (rfl | <_, Y', _, rfl, hf> | <_, Y', _, rfl, hg>)
    * exact (sum V_0' V_1' h_0' h_1').master_mem
    * exact Or.inr (Or.inl <Y', f.rel_cod hf, rfl>)
    * exact Or.inr (Or.inr <Y', g.rel_cod hg, rfl>)
  -- the disjunction is preserved by intersection
  have key : (W'_1  I  W'_2 = sumMaster V_0' V_1'  \/
      (exists  X Y', W = inj_0 X  /\  W'_1  I  W'_2 = inj_0 Y'  /\  f.rel X Y')  \/
      (exists  Y Y', W = inj_1 Y  /\  W'_1  I  W'_2 = inj_1 Y'  /\  g.rel Y Y')) :=
  by
    rcases hd_1 with rfl | <X, Y'_1, hWX_1, rfl, hf_1> | <Y, Y'_1, hWY_1, rfl, hg_1>
    * rw [Set.inter_eq_right.mpr
        (show W'_2  subseteq  sumMaster V_0' V_1' from (sum V_0' V_1' h_0' h_1').
  sub_master hW'_2)]
      exact hd_2
    * rcases hd_2 with rfl | <X', Y'_2, hWX_2, rfl, hf_2> | <Y', Y'_2, hWY_2, rfl,
  hg_2>
      * rw [Set.inter_eq_left.mpr (inj_0_subset_sumMaster (f.rel_cod hf_1))]
        exact Or.inr (Or.inl <X, Y'_1, hWX_1, rfl, hf_1>)
      * obtain rfl : X = X' := inj_0_injective (hWX_1.symm.trans hWX_2)
        rw [inj_0_inter]
        exact Or.inr (Or.inl <X, Y'_1  I  Y'_2, hWX_1, rfl, f.inter_right hf_1 hf_2
  >)
      * exact absurd ((hWX_1.symm.trans hWY_2)) (fun h =>
          not_inj_0_subset_inj_1 (h_0 X (f.rel_dom hf_1)) h.subset)
    * rcases hd_2 with rfl | <X', Y'_2, hWX_2, rfl, hf_2> | <Y', Y'_2, hWY_2, rfl,
  hg_2>
      * rw [Set.inter_eq_left.mpr (inj_1_subset_sumMaster (g.rel_cod hg_1))]
        exact Or.inr (Or.inr <Y, Y'_1, hWY_1, rfl, hg_1>)
      * exact absurd ((hWY_1.symm.trans hWX_2)) (fun h =>
          not_inj_1_subset_inj_0 (h_1 Y (g.rel_dom hg_1)) h.subset)
      * obtain rfl : Y = Y' := inj_1_injective (hWY_1.symm.trans hWY_2)
        rw [inj_1_inter]
        exact Or.inr (Or.inr <Y, Y'_1  I  Y'_2, hWY_1, rfl, g.inter_right hg_1 hg_2
  >)
  exact <hW, hmem _ key, key>
mono := by
  rintro W W_2 W' W'_2 <hW, hW', hd> hW_2W hW'W'_2 hW_2mem hW'_2mem
  refine <hW_2mem, hW'_2mem, ?_>
  rcases hd with rfl | <X, Y', rfl, rfl, hf> | <Y, Y', rfl, rfl, hg>
  * -- W' = master; W'_2  supseteq  master so W'_2 = master
    left; exact eq_sumMaster_of_subset hW'_2mem hW'W'_2
  * -- left copy: W_2  subseteq  0X, W'_2  supseteq  0Y'
```

```
      obtain <X_2, hX_2, rfl> := mem_subset_inj_0 hW_2mem hW_2W
      have hXX_2 : X_2  subseteq  X := inj_0_subset_inj_0.mp hW_2W
      rcases hW'_2mem with rfl | <Y'_2, hY'_2, rfl> | <Z'_2, hZ'_2, rfl>
      * left; rfl
      * have hY'Y'_2 : Y'  subseteq  Y'_2 := inj_0_subset_inj_0.mp hW'W'_2
        exact Or.inr (Or.inl <X_2, Y'_2, rfl, rfl,
          f.mono hf hXX_2 hY'Y'_2 hX_2 hY'_2>)
      * exact (not_inj_0_subset_inj_1 (h_0' Y' (f.rel_cod hf)) hW'W'_2).elim
    * -- right copy: W_2  subseteq  1Y, W'_2  supseteq  1Y'
      obtain <Y_2, hY_2, rfl> := mem_subset_inj_1 hW_2mem hW_2W
      have hYY_2 : Y_2  subseteq  Y := inj_1_subset_inj_1.mp hW_2W
      rcases hW'_2mem with rfl | <X'_2, hX'_2, rfl> | <Y'_2, hY'_2, rfl>
      * left; rfl
      * exact (not_inj_1_subset_inj_0 (h_1' Y' (g.rel_cod hg)) hW'W'_2).elim
      * have hY'Y'_2 : Y'  subseteq  Y'_2 := inj_1_subset_inj_1.mp hW'W'_2
        exact Or.inr (Or.inr <Y_2, Y'_2, rfl, rfl,
          g.mono hg hYY_2 hY'Y'_2 hY_2 hY'_2>)

/-- **Exercise 3.19(iii) (Scott 1981, PRG-19).** `out_0  comp  (f + g)  comp  in_0 = f
    `. -/
theorem outMap_0_comp_sumMap_comp_inMap_0 (f : ApproximableMap V_0 V_0') (g :
    ApproximableMap V_1 V_1') :
    (outMap_0 (h_0 := h_0') (h_1 := h_1')).comp
      ((sumMap (h_0 := h_0) (h_1 := h_1) (h_0' := h_0') (h_1' := h_1') f g).comp
        (inMap_0 (h_0 := h_0) (h_1 := h_1))) = f := by
  apply ApproximableMap.ext
  intro X Z
  constructor
  * rintro <W', <W, <hX, _, hinj>, hWmem, _, hd>, _, hZ, hleft>
    rcases hd with rfl | <X_0, Y', hWX_0, rfl, hf> | <Y_0, Y', hWY_0, rfl, hg>
    * -- output master: leftPart = Delta_0', and Z  supseteq  Delta_0' so Z = Delta_0'
      rw [leftPart_sumMaster] at hleft
      have : Z = V_0'.master := Set.Subset.antisymm (V_0'.sub_master hZ) hleft
      subst this
      exact f.mono f.master_rel (V_0.sub_master hX) subset_rfl hX V_0'.master_mem
    * -- output 0Y': X  subseteq  X_0 and Y'  subseteq  Z
      rw [leftPart_inj_0] at hleft
      have hXX_0 : X  subseteq  X_0 := inj_0_subset_inj_0.mp (hWX_0  |>  hinj)
      exact f.mono hf hXX_0 hleft hX hZ
    * -- impossible: in_0 forces 0X  subseteq  W = 1Y_0
      exact (not_inj_0_subset_inj_1 (h_0 X hX) (hWY_0  |>  hinj)).elim
  * intro hf
    refine <inj_0 Z, <inj_0 X, <f.rel_dom hf, ?_, subset_rfl>, ?_, ?_, ?_>, ?_, f.
    rel_cod hf, ?_>
    * exact Or.inr (Or.inl <X, f.rel_dom hf, rfl>)
    * exact Or.inr (Or.inl <X, f.rel_dom hf, rfl>)
    * exact Or.inr (Or.inl <Z, f.rel_cod hf, rfl>)
    * exact Or.inr (Or.inl <X, Z, rfl, rfl, hf>)
    * exact Or.inr (Or.inl <Z, f.rel_cod hf, rfl>)
    * exact (leftPart_inj_0 V_0' Z).subset

/-- **Exercise 3.19(iv) (Scott 1981, PRG-19).** `out_1  comp  (f + g)  comp  in_1 = g`.
     -/
theorem outMap_1_comp_sumMap_comp_inMap_1 (f : ApproximableMap V_0 V_0') (g :
    ApproximableMap V_1 V_1') :
    (outMap_1 (h_0 := h_0') (h_1 := h_1')).comp
```

```
      ((sumMap (h_0 := h_0) (h_1 := h_1) (h_0' := h_0') (h_1' := h_1') f g).comp
        (inMap_1 (h_0 := h_0) (h_1 := h_1))) = g := by
  apply ApproximableMap.ext
  intro Y Z
  constructor
  * rintro <W', <W, <hY, _, hinj>, hWmem, _, hd>, _, hZ, hright>
    rcases hd with rfl | <X_0, Y', hWX_0, rfl, hf> | <Y_0, Y', hWY_0, rfl, hg>
    * rw [rightPart_sumMaster] at hright
      have : Z = V_1'.master := Set.Subset.antisymm (V_1'.sub_master hZ) hright
      subst this
      exact g.mono g.master_rel (V_1.sub_master hY) subset_rfl hY V_1'.master_mem
    * exact (not_inj_1_subset_inj_0 (h_1 Y hY) (hWX_0  |>  hinj)).elim
    * rw [rightPart_inj_1] at hright
      have hYY_0 : Y  subseteq  Y_0 := inj_1_subset_inj_1.mp (hWY_0  |>  hinj)
      exact g.mono hg hYY_0 hright hY hZ
  * intro hg
    refine <inj_1 Z, <inj_1 Y, <g.rel_dom hg, ?_, subset_rfl>, ?_, ?_, ?_>, ?_, g.
    rel_cod hg, ?_>
    * exact Or.inr (Or.inr <Y, g.rel_dom hg, rfl>)
    * exact Or.inr (Or.inr <Y, g.rel_dom hg, rfl>)
    * exact Or.inr (Or.inr <Z, g.rel_cod hg, rfl>)
    * exact Or.inr (Or.inr <Y, Z, rfl, rfl, hg>)
    * exact Or.inr (Or.inr <Z, g.rel_cod hg, rfl>)
    * exact (rightPart_inj_1 V_1' Z).subset
end

section
set_option linter.unusedVariables false

theorem example_319_sum.{u_1, u_2, u_3, u_4} : And
  (forall  {a : Type u_1} {b : Type u_2} {a' : Type u_3} {b' : Type u_4}
    {V_0 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
    {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
    {V_0' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a'}
    {V_1' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b'}
    {h_0 : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    mem a V_0 X  ->  @Set.Nonempty a X}
    {h_1 : forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    mem b V_1 Y  ->  @Set.Nonempty b Y}
    {h_0' : forall  (X : Set a'), @DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.mem a' V_0' X  ->  @Set.Nonempty a' X}
    {h_1' : forall  (Y : Set b'), @DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.mem b' V_1' Y  ->  @Set.Nonempty b' Y}
    (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a' V_0 V_0')
    (g : @DomainTheory.NeighbourhoodSystems.ApproximableMap b b' V_1 V_1'),
    @Eq (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a' V_0 V_0')
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp a (Option (Sum a' b'))
    a' V_0
        (@DomainTheory.NeighbourhoodSystems.sum a' b' V_0' V_1' h_0' h_1') V_0'
        (@DomainTheory.NeighbourhoodSystems.outMap_0 a' b' V_0' V_1' h_0' h_1')
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp a (Option (Sum a b))
    (Option (Sum a' b')) V_0
          (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
          (@DomainTheory.NeighbourhoodSystems.sum a' b' V_0' V_1' h_0' h_1')
          (@DomainTheory.NeighbourhoodSystems.sumMap a b a' b' V_0 V_1 V_0' V_1' h_0
    h_1 h_0' h_1' f g)
```

```
        (@DomainTheory.NeighbourhoodSystems.inMap_0 a b V_0 V_1 h_0 h_1)))
    f)
(forall  {a : Type u_1} {b : Type u_2} {a' : Type u_3} {b' : Type u_4}
  {V_0 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
  {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
  {V_0' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a'}
  {V_1' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b'}
  {h_0 : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  mem a V_0 X  ->  @Set.Nonempty a X}
  {h_1 : forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  mem b V_1 Y  ->  @Set.Nonempty b Y}
  {h_0' : forall  (X : Set a'), @DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.mem a' V_0' X  ->  @Set.Nonempty a' X}
  {h_1' : forall  (Y : Set b'), @DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.mem b' V_1' Y  ->  @Set.Nonempty b' Y}
  (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a' V_0 V_0')
  (g : @DomainTheory.NeighbourhoodSystems.ApproximableMap b b' V_1 V_1'),
  @Eq (@DomainTheory.NeighbourhoodSystems.ApproximableMap b b' V_1 V_1')
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp b (Option (Sum a' b'))
  b' V_1
      (@DomainTheory.NeighbourhoodSystems.sum a' b' V_0' V_1' h_0' h_1') V_1'
      (@DomainTheory.NeighbourhoodSystems.outMap_1 a' b' V_0' V_1' h_0' h_1')
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp b (Option (Sum a b))
  (Option (Sum a' b')) V_1
        (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
        (@DomainTheory.NeighbourhoodSystems.sum a' b' V_0' V_1' h_0' h_1')
        (@DomainTheory.NeighbourhoodSystems.sumMap a b a' b' V_0 V_1 V_0' V_1' h_0
  h_1 h_0' h_1' f g)
        (@DomainTheory.NeighbourhoodSystems.inMap_1 a b V_0 V_1 h_0 h_1)))
    g) :=
@And.intro
(forall  {a : Type u_1} {b : Type u_2} {a' : Type u_3} {b' : Type u_4}
  {V_0 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
  {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
  {V_0' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a'}
  {V_1' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b'}
  {h_0 : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  mem a V_0 X  ->  @Set.Nonempty a X}
  {h_1 : forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  mem b V_1 Y  ->  @Set.Nonempty b Y}
  {h_0' : forall  (X : Set a'), @DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.mem a' V_0' X  ->  @Set.Nonempty a' X}
  {h_1' : forall  (Y : Set b'), @DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.mem b' V_1' Y  ->  @Set.Nonempty b' Y}
  (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a' V_0 V_0')
  (g : @DomainTheory.NeighbourhoodSystems.ApproximableMap b b' V_1 V_1'),
  @Eq (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a' V_0 V_0')
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp a (Option (Sum a' b'))
  a' V_0
      (@DomainTheory.NeighbourhoodSystems.sum a' b' V_0' V_1' h_0' h_1') V_0'
      (@DomainTheory.NeighbourhoodSystems.outMap_0 a' b' V_0' V_1' h_0' h_1')
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp a (Option (Sum a b))
  (Option (Sum a' b')) V_0
        (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
        (@DomainTheory.NeighbourhoodSystems.sum a' b' V_0' V_1' h_0' h_1')
        (@DomainTheory.NeighbourhoodSystems.sumMap a b a' b' V_0 V_1 V_0' V_1' h_0
```

```
    h_1 h_0' h_1' f g)
          (@DomainTheory.NeighbourhoodSystems.inMap_0 a b V_0 V_1 h_0 h_1)))
      f)
  (forall  {a : Type u_1} {b : Type u_2} {a' : Type u_3} {b' : Type u_4}
    {V_0 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
    {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
    {V_0' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a'}
    {V_1' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b'}
    {h_0 : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
   mem a V_0 X  ->  @Set.Nonempty a X}
    {h_1 : forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
   mem b V_1 Y  ->  @Set.Nonempty b Y}
    {h_0' : forall  (X : Set a'), @DomainTheory.NeighbourhoodSystems.
   NeighborhoodSystem.mem a' V_0' X  ->  @Set.Nonempty a' X}
    {h_1' : forall  (Y : Set b'), @DomainTheory.NeighbourhoodSystems.
   NeighborhoodSystem.mem b' V_1' Y  ->  @Set.Nonempty b' Y}
    (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a' V_0 V_0')
    (g : @DomainTheory.NeighbourhoodSystems.ApproximableMap b b' V_1 V_1'),
    @Eq (@DomainTheory.NeighbourhoodSystems.ApproximableMap b b' V_1 V_1')
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp b (Option (Sum a' b'))
   b' V_1
        (@DomainTheory.NeighbourhoodSystems.sum a' b' V_0' V_1' h_0' h_1') V_1'
        (@DomainTheory.NeighbourhoodSystems.outMap_1 a' b' V_0' V_1' h_0' h_1')
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp b (Option (Sum a b))
   (Option (Sum a' b')) V_1
          (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
          (@DomainTheory.NeighbourhoodSystems.sum a' b' V_0' V_1' h_0' h_1')
          (@DomainTheory.NeighbourhoodSystems.sumMap a b a' b' V_0 V_1 V_0' V_1' h_0
   h_1 h_0' h_1' f g)
          (@DomainTheory.NeighbourhoodSystems.inMap_1 a b V_0 V_1 h_0 h_1)))
      g)
  @DomainTheory.NeighbourhoodSystems.outMap_0_comp_sumMap_comp_inMap_0
  @DomainTheory.NeighbourhoodSystems.outMap_1_comp_sumMap_comp_inMap_1
end

section
open NeighborhoodSystem ApproximableMap
variable {a b ? a' b' : Type*}
variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b} {V_2 :
   NeighborhoodSystem ?}
variable {V_0' : NeighborhoodSystem a'} {V_1' : NeighborhoodSystem b'}

/-- **Exercise 3.19(ii) (Scott 1981, PRG-19).** The product mapping `f  x  g = <f  comp
     p_0, g  comp  p_1>`. -/
def prodMap (f : ApproximableMap V_0 V_0') (g : ApproximableMap V_1 V_1') :
    ApproximableMap (prod V_0 V_1) (prod V_0' V_1') :=
  paired (f.comp (proj_0 V_0 V_1)) (g.comp (proj_1 V_0 V_1))

/-- **Exercise 3.19 (Scott 1981, PRG-19).** `(f  x  g)(w) = <f(w_0), g(w_1)>` for every
    product element
`w` (so in particular `(f  x  g)(<x, y>) = <f(x), g(y)>`, equation (i)). -/
theorem toElementMap_prodMap (f : ApproximableMap V_0 V_0') (g : ApproximableMap V_1
   V_1')
    (w : (prod V_0 V_1).Element) :
    (prodMap f g).toElementMap w = pair (f.toElementMap w.fst) (g.toElementMap w.snd)
    := by
```

```
  rw [prodMap, toElementMap_paired, toElementMap_comp, toElementMap_comp,
    toElementMap_proj_0,
    toElementMap_proj_1]

/-- **Exercise 3.19(i) (Scott 1981, PRG-19).** `(f  x  g)(<x, y>) = <f(x), g(y)>`. -/
theorem toElementMap_prodMap_pair (f : ApproximableMap V_0 V_0') (g : ApproximableMap
    V_1 V_1')
    (x : V_0.Element) (y : V_1.Element) :
    (prodMap f g).toElementMap (pair x y) = pair (f.toElementMap x) (g.toElementMap y)
     := by
  rw [toElementMap_prodMap, fst_pair, snd_pair]
end

section
set_option linter.unusedVariables false

theorem example_319.{u_1, u_2, u_4, u_5} : And
  (forall  {a : Type u_1} {b : Type u_2} {a' : Type u_4} {b' : Type u_5}
    {V_0 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
    {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
    {V_0' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a'}
    {V_1' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b'}
    (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a' V_0 V_0')
    (g : @DomainTheory.NeighbourhoodSystems.ApproximableMap b b' V_1 V_1')
    (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V_0)
    (y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element b V_1),
    @Eq
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Sum a' b')
        (@DomainTheory.NeighbourhoodSystems.prod a' b' V_0' V_1'))
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap (Sum a b) (Sum
    a' b')
        (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) (@DomainTheory.
    NeighbourhoodSystems.prod a' b' V_0' V_1')
        (@DomainTheory.NeighbourhoodSystems.prodMap a b a' b' V_0 V_1 V_0' V_1' f g)
        (@DomainTheory.NeighbourhoodSystems.pair a b V_0 V_1 x y))
      (@DomainTheory.NeighbourhoodSystems.pair a' b' V_0' V_1'
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a' V_0 V_0'
     f x)
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap b b' V_1 V_1'
     g y)))
  (forall  {a : Type u_1} {b : Type u_2} {a' : Type u_4} {b' : Type u_5}
    {V_0 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
    {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
    {V_0' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a'}
    {V_1' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b'}
    (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a' V_0 V_0')
    (g : @DomainTheory.NeighbourhoodSystems.ApproximableMap b b' V_1 V_1')
    (w :
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Sum a b)
        (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1)),
    @Eq
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Sum a' b')
        (@DomainTheory.NeighbourhoodSystems.prod a' b' V_0' V_1'))
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap (Sum a b) (Sum
    a' b')
        (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) (@DomainTheory.
```

```
  NeighbourhoodSystems.prod a' b' V_0' V_1')
       (@DomainTheory.NeighbourhoodSystems.prodMap a b a' b' V_0 V_1 V_0' V_1' f g) w
  )
     (@DomainTheory.NeighbourhoodSystems.pair a' b' V_0' V_1'
       (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a' V_0 V_0'
    f
         (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.fst a b V_0
  V_1 w))
       (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap b b' V_1 V_1'
    g
         (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.snd a b V_0
  V_1 w)))) :=
@And.intro
(forall  {a : Type u_1} {b : Type u_2} {a' : Type u_4} {b' : Type u_5}
  {V_0 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
  {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
  {V_0' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a'}
  {V_1' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b'}
  (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a' V_0 V_0')
  (g : @DomainTheory.NeighbourhoodSystems.ApproximableMap b b' V_1 V_1')
  (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V_0)
  (y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element b V_1),
  @Eq
    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Sum a' b')
      (@DomainTheory.NeighbourhoodSystems.prod a' b' V_0' V_1'))
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap (Sum a b) (Sum
  a' b')
       (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) (@DomainTheory.
  NeighbourhoodSystems.prod a' b' V_0' V_1')
       (@DomainTheory.NeighbourhoodSystems.prodMap a b a' b' V_0 V_1 V_0' V_1' f g)
       (@DomainTheory.NeighbourhoodSystems.pair a b V_0 V_1 x y))
    (@DomainTheory.NeighbourhoodSystems.pair a' b' V_0' V_1'
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a' V_0 V_0'
    f x)
       (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap b b' V_1 V_1'
    g y)))
(forall  {a : Type u_1} {b : Type u_2} {a' : Type u_4} {b' : Type u_5}
  {V_0 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
  {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
  {V_0' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a'}
  {V_1' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b'}
  (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a' V_0 V_0')
  (g : @DomainTheory.NeighbourhoodSystems.ApproximableMap b b' V_1 V_1')
  (w :
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Sum a b)
      (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1)),
  @Eq
    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Sum a' b')
      (@DomainTheory.NeighbourhoodSystems.prod a' b' V_0' V_1'))
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap (Sum a b) (Sum
  a' b')
       (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) (@DomainTheory.
  NeighbourhoodSystems.prod a' b' V_0' V_1')
       (@DomainTheory.NeighbourhoodSystems.prodMap a b a' b' V_0 V_1 V_0' V_1' f g) w
  )
     (@DomainTheory.NeighbourhoodSystems.pair a' b' V_0' V_1'
```

```
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a' V_0 V_0'
     f
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.fst a b V_0
    V_1 w))
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap b b' V_1 V_1'
     g
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.snd a b V_0
    V_1 w))))
  @DomainTheory.NeighbourhoodSystems.toElementMap_prodMap_pair @DomainTheory.
    NeighbourhoodSystems.toElementMap_prodMap
end
end DomainTheory.NeighbourhoodSystems
end
```

— Pref-folded `sumMap` companions

sumMap.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Definition_8_9

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap Exercise326

/-!
Pref-folded companions for `sumMap` (def in DomainConstructs `Exercise_3_19`).
Lives under Retracts because several lemmas need Definition 8.9 helpers
(`inMap_0_toElementMap_reaches_left`, `toElementMap_of_comp_eq_idMap`).
-/

namespace sumMap

universe u

variable {? ? ?' ?' ?'' ?'' : Type u}

variable {V0 : NeighborhoodSystem ?} {V1 : NeighborhoodSystem ?}

variable {V0' : NeighborhoodSystem ?'} {V1' : NeighborhoodSystem ?'}

variable {V0'' : NeighborhoodSystem ?''} {V1'' : NeighborhoodSystem ?''}

variable {h0 : forall  X, V0.mem X  ->  X.Nonempty} {h1 : forall  Y, V1.mem Y  ->  Y.
    Nonempty}

variable {h0' : forall  X, V0'.mem X  ->  X.Nonempty} {h1' : forall  Y, V1'.mem Y  ->
     Y.Nonempty}

variable {h0'' : forall  X, V0''.mem X  ->  X.Nonempty} {h1'' : forall  Y, V1''.mem Y
     ->  Y.Nonempty}
```

```
/-- A sum-element reaches "neither" copy iff it is the bottom element. -/
theorem reaches_neither_iff_eq_bot {y : (sum V0 V1 h0 h1).Element} :
    (forall  W, y.mem W  ->  W = (sum V0 V1 h0 h1).master)  <->  y = (sum V0 V1 h0 h1)
    .bot := by
  constructor
  * intro hN
    apply Element.ext
    intro W
    rw [mem_bot]
    exact <hN W, fun h => h  |>  y.master_mem>
  * rintro rfl W hW
    rw [mem_bot] at hW
    exact hW

/-- `sumMap` sends the bottom element to the bottom element. -/
theorem bot (f : ApproximableMap V0 V0') (g : ApproximableMap V1 V1') :
    (sumMap (h_0 := h0) (h_1 := h1) (h_0' := h0') (h_1' := h1') f g).toElementMap
        (sum V0 V1 h0 h1).bot = (sum V0' V1' h0' h1').bot := by
  apply Element.ext
  intro Z
  rw [mem_bot]
  constructor
  * rintro <W, hW, _, hZmem, hd>
    rw [mem_bot] at hW
    subst hW
    rcases hd with rfl | <X, Y', hWX, -, -> | <Y, Y', hWY, -, ->
    * rfl
    * have hWX' : sumMaster V0 V1 = inj_0 X := hWX
      exact absurd (hWX'  |>  none_mem_sumMaster) none_mem_inj_0
    * have hWY' : sumMaster V0 V1 = inj_1 Y := hWY
      exact absurd (hWY'  |>  none_mem_sumMaster) none_mem_inj_1
  * rintro rfl
    exact <sumMaster V0 V1, (sum V0 V1 h0 h1).mem_bot.mpr rfl, (sum V0 V1 h0 h1).
    master_mem,
      (sum V0' V1' h0' h1').master_mem, Or.inl rfl>

/-- If `y` reaches the left copy, `(sumMap f g)(y)` also reaches the left copy. -/
theorem reaches_left {f : ApproximableMap V0 V0'} {g : ApproximableMap V1 V1'}
    {y : (sum V0 V1 h0 h1).Element} (hL : exists  X, V0.mem X  /\  y.mem (inj_0 X)) :
    exists  X', V0'.mem X'  /\  ((sumMap (h_0 := h0) (h_1 := h1) (h_0' := h0') (h_1'
    := h1') f g).toElementMap
      y).mem (inj_0 X') := by
  obtain <X, hX, hXy> := hL
  have hf : f.rel X V0'.master :=
    f.mono f.master_rel (V0.sub_master hX) subset_rfl hX V0'.master_mem
  exact <V0'.master, V0'.master_mem, inj_0 X, hXy,
    Or.inr (Or.inl <X, hX, rfl>), Or.inr (Or.inl <V0'.master, V0'.master_mem, rfl>),
    Or.inr (Or.inl <X, V0'.master, rfl, rfl, hf>)>

/-- If `y` reaches the right copy, `(sumMap f g)(y)` also reaches the right copy. -/
theorem reaches_right {f : ApproximableMap V0 V0'} {g : ApproximableMap V1 V1'}
    {y : (sum V0 V1 h0 h1).Element} (hR : exists  Y, V1.mem Y  /\  y.mem (inj_1 Y)) :
    exists  Y', V1'.mem Y'  /\  ((sumMap (h_0 := h0) (h_1 := h1) (h_0' := h0') (h_1'
    := h1') f g).toElementMap
      y).mem (inj_1 Y') := by
  obtain <Y, hY, hYy> := hR
```

```
  have hg : g.rel Y V1'.master :=
    g.mono g.master_rel (V1.sub_master hY) subset_rfl hY V1'.master_mem
  exact <V1'.master, V1'.master_mem, inj_1 Y, hYy,
    Or.inr (Or.inr <Y, hY, rfl>), Or.inr (Or.inr <V1'.master, V1'.master_mem, rfl>),
    Or.inr (Or.inr <Y, V1'.master, rfl, rfl, hg>)>

variable {? ? ?' ?' ?'' ?'' : Type u}

variable {V0 : NeighborhoodSystem ?} {V1 : NeighborhoodSystem ?}

variable {V0' : NeighborhoodSystem ?'} {V1' : NeighborhoodSystem ?'}

variable {V0'' : NeighborhoodSystem ?''} {V1'' : NeighborhoodSystem ?''}

variable {h0 : forall  X, V0.mem X  ->  X.Nonempty} {h1 : forall  Y, V1.mem Y  ->  Y.
    Nonempty}

variable {h0' : forall  X, V0'.mem X  ->  X.Nonempty} {h1' : forall  Y, V1'.mem Y  ->
     Y.Nonempty}

variable {h0'' : forall  X, V0''.mem X  ->  X.Nonempty} {h1'' : forall  Y, V1''.mem Y
     ->  Y.Nonempty}

theorem toElementMap_inMap_0 (f : ApproximableMap V0 V0') (g : ApproximableMap V1 V1')
    (v : V0.Element) :
    (sumMap (h_0 := h0) (h_1 := h1) (h_0' := h0') (h_1' := h1') f g).toElementMap
        ((inMap_0 (h_0 := h0) (h_1 := h1)).toElementMap v) =
      (inMap_0 (h_0 := h0') (h_1 := h1')).toElementMap (f.toElementMap v) := by
  set z := (sumMap (h_0 := h0) (h_1 := h1) (h_0' := h0') (h_1' := h1') f g).
    toElementMap
    ((inMap_0 (h_0 := h0) (h_1 := h1)).toElementMap v) with hz
  have hLz : exists  X', V0'.mem X'  /\  z.mem (inj_0 X') :=
    reaches_left (inMap_0_toElementMap_reaches_left v)
  rw [ <-  inMap_0_outMap_0_eq_of_left hLz]
  congr 1
  have heq : (outMap_0 (h_0 := h0') (h_1 := h1')).comp
      ((sumMap (h_0 := h0) (h_1 := h1) (h_0' := h0') (h_1' := h1') f g).comp
        (inMap_0 (h_0 := h0) (h_1 := h1))) = f :=
    outMap_0_comp_sumMap_comp_inMap_0 f g
  calc (outMap_0 (h_0 := h0') (h_1 := h1')).toElementMap z
      = ((outMap_0 (h_0 := h0') (h_1 := h1')).comp
          ((sumMap (h_0 := h0) (h_1 := h1) (h_0' := h0') (h_1' := h1') f g).comp
            (inMap_0 (h_0 := h0) (h_1 := h1)))).toElementMap v := by
        rw [toElementMap_comp, toElementMap_comp,  <-  hz]
    _ = f.toElementMap v := by rw [heq]

theorem toElementMap_inMap_1 (f : ApproximableMap V0 V0') (g : ApproximableMap V1 V1')
    (w : V1.Element) :
    (sumMap (h_0 := h0) (h_1 := h1) (h_0' := h0') (h_1' := h1') f g).toElementMap
        ((inMap_1 (h_0 := h0) (h_1 := h1)).toElementMap w) =
      (inMap_1 (h_0 := h0') (h_1 := h1')).toElementMap (g.toElementMap w) := by
  set z := (sumMap (h_0 := h0) (h_1 := h1) (h_0' := h0') (h_1' := h1') f g).
    toElementMap
    ((inMap_1 (h_0 := h0) (h_1 := h1)).toElementMap w) with hz
  have hRz : exists  Y', V1'.mem Y'  /\  z.mem (inj_1 Y') :=
    reaches_right (inMap_1_toElementMap_reaches_right w)
```

```
  rw [ <-  inMap_1_outMap_1_eq_of_right hRz]
  congr 1
  have heq : (outMap_1 (h_0 := h0') (h_1 := h1')).comp
      ((sumMap (h_0 := h0) (h_1 := h1) (h_0' := h0') (h_1' := h1') f g).comp
        (inMap_1 (h_0 := h0) (h_1 := h1))) = g :=
    outMap_1_comp_sumMap_comp_inMap_1 f g
  calc (outMap_1 (h_0 := h0') (h_1 := h1')).toElementMap z
      = ((outMap_1 (h_0 := h0') (h_1 := h1')).comp
          ((sumMap (h_0 := h0) (h_1 := h1) (h_0' := h0') (h_1' := h1') f g).comp
            (inMap_1 (h_0 := h0) (h_1 := h1)))).toElementMap w := by
        rw [toElementMap_comp, toElementMap_comp,  <-  hz]
    _ = g.toElementMap w := by rw [heq]

theorem toElementMap_of_left {f : ApproximableMap V0 V0'} {g : ApproximableMap V1 V1'}
    {y : (sum V0 V1 h0 h1).Element} (hL : exists  X, V0.mem X  /\  y.mem (inj_0 X)) :
    (sumMap (h_0 := h0) (h_1 := h1) (h_0' := h0') (h_1' := h1') f g).toElementMap y =
      (inMap_0 (h_0 := h0') (h_1 := h1')).toElementMap
        (f.toElementMap ((outMap_0 (h_0 := h0) (h_1 := h1)).toElementMap y)) := by
  conv_lhs => rw [ <-  inMap_0_outMap_0_eq_of_left hL]
  exact toElementMap_inMap_0 f g _

theorem toElementMap_of_right {f : ApproximableMap V0 V0'} {g : ApproximableMap V1 V1
    '}
    {y : (sum V0 V1 h0 h1).Element} (hR : exists  Y, V1.mem Y  /\  y.mem (inj_1 Y)) :
    (sumMap (h_0 := h0) (h_1 := h1) (h_0' := h0') (h_1' := h1') f g).toElementMap y =
      (inMap_1 (h_0 := h0') (h_1 := h1')).toElementMap
        (g.toElementMap ((outMap_1 (h_0 := h0) (h_1 := h1)).toElementMap y)) := by
  conv_lhs => rw [ <-  inMap_1_outMap_1_eq_of_right hR]
  exact toElementMap_inMap_1 f g _

variable {? ? ?' ?' ?'' ?'' : Type u}

variable {V0 : NeighborhoodSystem ?} {V1 : NeighborhoodSystem ?}

variable {V0' : NeighborhoodSystem ?'} {V1' : NeighborhoodSystem ?'}

variable {V0'' : NeighborhoodSystem ?''} {V1'' : NeighborhoodSystem ?''}

variable {h0 : forall  X, V0.mem X  ->  X.Nonempty} {h1 : forall  Y, V1.mem Y  ->  Y.
    Nonempty}

variable {h0' : forall  X, V0'.mem X  ->  X.Nonempty} {h1' : forall  Y, V1'.mem Y  ->
     Y.Nonempty}

variable {h0'' : forall  X, V0''.mem X  ->  X.Nonempty} {h1'' : forall  Y, V1''.mem Y
     ->  Y.Nonempty}

/-- **`sumMap` preserves composition: `(f' comp f) + (g' comp g) = (f'+g')  comp  (f+g)
    `.** -/
theorem comp (f' : ApproximableMap V0' V0'') (f : ApproximableMap V0 V0')
    (g' : ApproximableMap V1' V1'') (g : ApproximableMap V1 V1') :
    sumMap (h_0 := h0) (h_1 := h1) (h_0' := h0'') (h_1' := h1'') (f'.comp f) (g'.comp
    g) =
      (sumMap (h_0 := h0') (h_1 := h1') (h_0' := h0'') (h_1' := h1'') f' g').comp
        (sumMap (h_0 := h0) (h_1 := h1) (h_0' := h0') (h_1' := h1') f g) := by
  apply ext_of_toElementMap
```

```
  intro y
  rw [toElementMap_comp]
  rcases sum_element_trichotomy y with hL | hR | hN
  * rw [toElementMap_of_left hL, toElementMap_comp, toElementMap_of_left hL,
      toElementMap_of_left (inMap_0_toElementMap_reaches_left _),
      toElementMap_of_comp_eq_idMap outMap_0_comp_inMap_0]
  * rw [toElementMap_of_right hR, toElementMap_comp, toElementMap_of_right hR,
      toElementMap_of_right (inMap_1_toElementMap_reaches_right _),
      toElementMap_of_comp_eq_idMap outMap_1_comp_inMap_1]
  * rw [reaches_neither_iff_eq_bot] at hN
    rw [hN, bot, bot, bot]

variable {? ? ?' ?' ?'' ?'' : Type u}

variable {V0 : NeighborhoodSystem ?} {V1 : NeighborhoodSystem ?}

variable {V0' : NeighborhoodSystem ?'} {V1' : NeighborhoodSystem ?'}

variable {V0'' : NeighborhoodSystem ?''} {V1'' : NeighborhoodSystem ?''}

variable {h0 : forall  X, V0.mem X  ->  X.Nonempty} {h1 : forall  Y, V1.mem Y  ->  Y.
    Nonempty}

variable {h0' : forall  X, V0'.mem X  ->  X.Nonempty} {h1' : forall  Y, V1'.mem Y  ->
     Y.Nonempty}

variable {h0'' : forall  X, V0''.mem X  ->  X.Nonempty} {h1'' : forall  Y, V1''.mem Y
     ->  Y.Nonempty}

/-- **`sumMap` preserves identities: `id + id = id`.** -/
theorem id : sumMap (h_0 := h0) (h_1 := h1) (h_0' := h0) (h_1' := h1)
    (idMap V0) (idMap V1) = idMap (sum V0 V1 h0 h1) := by
  apply ext_of_toElementMap
  intro y
  rw [toElementMap_idMap]
  rcases sum_element_trichotomy y with hL | hR | hN
  * rw [toElementMap_of_left hL, toElementMap_idMap, inMap_0_outMap_0_eq_of_left hL]
  * rw [toElementMap_of_right hR, toElementMap_idMap, inMap_1_outMap_1_eq_of_right hR]
  * rw [reaches_neither_iff_eq_bot] at hN
    rw [hN, bot]

end sumMap

end DomainTheory.NeighbourhoodSystems
```

3.4.21 Exercise 3.20 — sum and product functors

(For category theorists). Show that the result of 3.19 can be used to prove that $+$ and $\times$ on the category of domains and approximable maps are indeed functors. Show further that $\times$ is the categorical product for this category.

Exercise_3_20.lean

```
module
```

```
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Exercise_3_19
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_4
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem ApproximableMap
variable {a b ? a' b' : Type*}
variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b} {V_2 :
    NeighborhoodSystem ?}
variable {V_0' : NeighborhoodSystem a'} {V_1' : NeighborhoodSystem b'}
/- concordance: source L1541 EXERCISE 3.20 sum and product functors -/
/-- **Exercise 3.20 (Scott 1981, PRG-19).** The universal property of the product (
    uniqueness):
any `k` with `p_0  comp  k = h_0` and `p_1  comp  k = h_1` equals the pairing `<h_0,
    h_1>`. Hence `prod` with
`proj_0`, `proj_1` is the categorical product of `?_0` and `?_1`. -/
theorem paired_unique (h_0 : ApproximableMap V_2 V_0) (h_1 : ApproximableMap V_2 V_1)
    (k : ApproximableMap V_2 (prod V_0 V_1)) (hk_0 : (proj_0 V_0 V_1).comp k = h_0)
    (hk_1 : (proj_1 V_0 V_1).comp k = h_1) : k = paired h_0 h_1 := by
  rw [ <-  hk_0,  <-  hk_1, paired_proj]

/-- **Exercise 3.20 (Scott 1981, PRG-19).** The universal property of the product (
    existence):
`p_0  comp  <h_0, h_1> = h_0` and `p_1  comp  <h_0, h_1> = h_1` (these are Proposition
    3.4(i)). -/
theorem proj_paired (h_0 : ApproximableMap V_2 V_0) (h_1 : ApproximableMap V_2 V_1) :
    (proj_0 V_0 V_1).comp (paired h_0 h_1) = h_0  /\  (proj_1 V_0 V_1).comp (paired
    h_0 h_1) = h_1 :=
  <proj_0_comp_paired h_0 h_1, proj_1_comp_paired h_0 h_1>

/-- **Exercise 3.20 (Scott 1981, PRG-19).** ` x ` preserves composition:
`(f'  comp  f)  x  (g'  comp  g) = (f'  x  g')  comp  (f  x  g)`. -/
theorem prodMap_comp {a'' b'' : Type*} {V_0'' : NeighborhoodSystem a''} {V_1'' :
    NeighborhoodSystem b''}
    (f' : ApproximableMap V_0' V_0'') (f : ApproximableMap V_0 V_0')
    (g' : ApproximableMap V_1' V_1'') (g : ApproximableMap V_1 V_1') :
    prodMap (f'.comp f) (g'.comp g) = (prodMap f' g').comp (prodMap f g) := by
  apply ext_of_toElementMap
  intro w
  rw [toElementMap_prodMap, toElementMap_comp, toElementMap_comp, toElementMap_comp,
    toElementMap_prodMap, toElementMap_prodMap, fst_pair, snd_pair]

/-- **Exercise 3.20 (Scott 1981, PRG-19).** ` x ` preserves identities: `I  x  I = I`.
    -/
theorem prodMap_id : prodMap (idMap V_0) (idMap V_1) = idMap (prod V_0 V_1) := by
  apply ext_of_toElementMap
  intro w
  rw [toElementMap_prodMap, toElementMap_idMap, toElementMap_idMap, toElementMap_idMap
    ,
    pair_fst_snd]
end
end DomainTheory.NeighbourhoodSystems
end
```

3.4.22 Exercise 3.21 — identity of step neighbourhoods

In the proofs of 3.12 in the definition of curry (g) it is rather cavalierly assumed that the neighbourhood $[Y, Z]$ uniquely determines Y and Z. Show that this is true if $Z \neq \Delta_2$. (Hint: Find explicitly the least of $f \in [Y, Z]$.) Show that if $Z = \Delta_2$ the biconditional stated at the start of the proof is still valid even though Y is not uniquely determined. (Hint: Remember that $\Delta_1\ g\ \Delta_2$ must hold.) For arbitrary pairs of neighbourhoods of $(D_1 \to D_2)$ is there a simple criterion for identity?

Exercise_3_21.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem ApproximableMap
section
variable {a b : Type*} {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}

/-- For a single step `[Y, Z]` and an input `X ? Y`, the intersection is the whole
    master `Delta_2`. -/
theorem interYs_single_not_subset_eq {YX : Set a} {Z : Set b} {X : Set a}
    (hXY : not  X  subseteq  YX) : interYs V_1.master [(YX, Z)] X = V_1.master := by
  ext z
  rw [mem_interYs]
  constructor
  * rintro <hm, _>; exact hm
  * intro hz
    exact <hz, fun p hp hXp => by rw [List.mem_singleton] at hp; subst hp; exact
    absurd hXp hXY>

/-- For a single step `[Y, Z]` and a *sharp enough* input `X  subseteq  Y`, the
    intersection is just `Z`. -/
theorem interYs_single_subset_eq {YX : Set a} {Z : Set b} {X : Set a} (hZ : V_1.mem Z)
    (hXY : X  subseteq  YX) : interYs V_1.master [(YX, Z)] X = Z := by
  ext z
  rw [mem_interYs]
  constructor
  * rintro <_, hall>; exact hall (YX, Z) (List.mem_singleton.mpr rfl) hXY
  * intro hz
    exact <V_1.sub_master hZ hz, fun p hp _ => by rw [List.mem_singleton] at hp; subst
     hp; exact hz>

/-- Consistency (`hcons`) for a single step `[Y, Z]`: the intersection over the
    relevant outputs is
either `Z` (sharp input) or `Delta_2` (blunt input), both neighbourhoods. -/
theorem stepCons {Y : Set a} {Z : Set b} (hZ : V_1.mem Z) :
    forall  {X}, V_0.mem X  ->  V_1.mem (interYs V_1.master [(Y, Z)] X) := by
  intro X _
  by_cases hXY : X  subseteq  Y
  * rw [interYs_single_subset_eq hZ hXY]; exact hZ
  * rw [interYs_single_not_subset_eq hXY]; exact V_1.master_mem
```

```
/-- The single-step least map `f_0  in  [Y, Z]`: `leastMap [(Y, Z)]`, the minimal
    approximable mapping
relating `Y` to `Z`. -/
def leastStep (Y : Set a) (Z : Set b) (hY : V_0.mem Y) (hZ : V_1.mem Z) :
    ApproximableMap V_0 V_1 :=
  leastMap [(Y, Z)] (by rintro p hp; rw [List.mem_singleton] at hp; subst hp; exact <
    hY, hZ>)
    (stepCons hZ)

theorem leastStep_le {Y : Set a} {Z : Set b} (hY : V_0.mem Y) (hZ : V_1.mem Z)
    {f : ApproximableMap V_0 V_1} (hf : f  in  step Y Z) : leastStep Y Z hY hZ  <=  f
    := by
  apply leastMap_le
  rw [stepFun_singleton]; exact hf

theorem leastStep_mem {Y : Set a} {Z : Set b} (hY : V_0.mem Y) (hZ : V_1.mem Z) :
    leastStep Y Z hY hZ  in  step Y Z := by
  have := leastMap_mem_stepFun (V_0 := V_0) (V_1 := V_1)
    (L := [(Y, Z)]) (by rintro p hp; rw [List.mem_singleton] at hp; subst hp; exact <
    hY, hZ>)
    (stepCons hZ)
  rw [stepFun_singleton] at this
  exact this

/-- The least elements of equal step neighbourhoods coincide. -/
theorem leastStep_eq_of_step_eq {Y : Set a} {Z : Set b} (hY : V_0.mem Y) (hZ : V_1.mem
     Z)
    {Y' : Set a} {Z' : Set b} (hY' : V_0.mem Y') (hZ' : V_1.mem Z')
    (h : (step Y Z : Set (ApproximableMap V_0 V_1)) = step Y' Z') :
    leastStep Y Z hY hZ = leastStep Y' Z' hY' hZ' := by
  apply le_antisymm
  * exact leastStep_le hY hZ (h  |>  leastStep_mem hY' hZ')
  * exact leastStep_le hY' hZ' (h.symm  |>  leastStep_mem hY hZ)

/-- `f_0 = leastStep Y Z` relates `Y'` to `Z'` iff (`Y'  subseteq  Y` and `Z  subseteq
    Z'`) or `Z' = Delta_2`. -/
theorem leastStep_rel {Y : Set a} {Z : Set b} (hY : V_0.mem Y) (hZ : V_1.mem Z)
    {Y' : Set a} {Z' : Set b} (hY' : V_0.mem Y') (hZ' : V_1.mem Z') :
    (leastStep Y Z hY hZ).rel Y' Z'  <->  interYs V_1.master [(Y, Z)] Y'  subseteq  Z'
     := by
  rw [leastStep, leastMap_rel]
  exact <fun h => h.2.2, fun h => <hY', hZ', h>>

/- concordance: source L1543 EXERCISE 3.21 identity of step neighbourhoods -/
/-- **Exercise 3.21.** If the output is the master, `[Y, Delta_2]` is the whole
    function space. -/
theorem step_master_right (Y : Set a) (hY : V_0.mem Y) :
    (step Y V_1.master : Set (ApproximableMap V_0 V_1)) = Set.univ := by
  ext f
  simp only [mem_step, Set.mem_univ, iff_true]
  exact f.mono f.master_rel (V_0.sub_master hY) subset_rfl hY V_1.master_mem

theorem step_eq_univ_iff {Y : Set a} {Z : Set b} (hY : V_0.mem Y) (hZ : V_1.mem Z) :
    (step Y Z : Set (ApproximableMap V_0 V_1)) = Set.univ  <->  Z = V_1.master := by
  constructor
  * intro h
```

```
    -- the empty-list least map relates `Y` only to `Delta_2`; it lies in `[Y, Z] =
    univ`
    have hmem : leastMap (V_0 := V_0) (V_1 := V_1) ([] : List (Set a  x  Set b))
        (by simp) (fun {X} _ => by rw [interYs_nil]; exact V_1.master_mem)  in  step Y
     Z := by
      rw [h]; exact Set.mem_univ _
    rw [mem_step, leastMap_rel] at hmem
    have : V_1.master  subseteq  Z := by rw [ <-  interYs_nil V_1.master Y]; exact
    hmem.2.2
    exact Set.Subset.antisymm (V_1.sub_master hZ) this
  * rintro rfl; exact step_master_right Y hY

/-- **Exercise 3.21 (Scott 1981, PRG-19).** If `Z  !=  Delta_2`, the neighbourhood `[Y,
     Z]` determines both
endpoints: `[Y, Z] = [Y', Z']` implies `Y = Y'` and `Z = Z'`. -/
theorem step_eq_of_ne_master {Y : Set a} {Z : Set b} (hY : V_0.mem Y) (hZ : V_1.mem Z)
    {Y' : Set a} {Z' : Set b} (hY' : V_0.mem Y') (hZ' : V_1.mem Z') (hZne : Z  !=  V_1
    .master)
    (h : (step Y Z : Set (ApproximableMap V_0 V_1)) = step Y' Z') : Y = Y'  /\  Z = Z'
     := by
  -- `Z'  !=  Delta_2` too, else `[Y', Z']` is the whole space and so is `[Y, Z]`,
    forcing `Z = Delta_2`
  have hZ'ne : Z'  !=  V_1.master := by
    intro hZ'eq
    rw [hZ'eq, step_master_right Y' hY'] at h
    exact hZne ((step_eq_univ_iff hY hZ).mp h)
  have hfeq := leastStep_eq_of_step_eq hY hZ hY' hZ' h
  -- `f_0.rel Y' Z'` and `f_0'.rel Y Z`, both with non-master outputs
  have h1 : (leastStep Y Z hY hZ).rel Y' Z' := by
    rw [hfeq]; exact (leastStep_mem hY' hZ')
  have h2 : (leastStep Y' Z' hY' hZ').rel Y Z := by
    rw [ <-  hfeq]; exact (leastStep_mem hY hZ)
  rw [leastStep_rel hY hZ hY' hZ'] at h1
  rw [leastStep_rel hY' hZ' hY hZ] at h2
  -- from `h1`: `Y'  subseteq  Y` and `Z  subseteq  Z'`
  have hY'Y : Y'  subseteq  Y := by
    by_contra hns
    rw [interYs_single_not_subset_eq hns] at h1
    exact hZ'ne (Set.Subset.antisymm (V_1.sub_master hZ') h1)
  have hZZ' : Z  subseteq  Z' := by
    rw [interYs_single_subset_eq hZ hY'Y] at h1; exact h1
  -- from `h2`: `Y  subseteq  Y'` and `Z'  subseteq  Z`
  have hYY' : Y  subseteq  Y' := by
    by_contra hns
    rw [interYs_single_not_subset_eq hns] at h2
    exact hZne (Set.Subset.antisymm (V_1.sub_master hZ) h2)
  have hZ'Z : Z'  subseteq  Z := by
    rw [interYs_single_subset_eq hZ' hYY'] at h2; exact h2
  exact <Set.Subset.antisymm hYY' hY'Y, Set.Subset.antisymm hZZ' hZ'Z>

/-- **Exercise 3.21 (Scott 1981, PRG-19).** The simple criterion for identity of
    function-space
neighbourhoods: `[Y, Z] = [Y', Z']` iff both outputs are the master `Delta_2`, or the
    pairs coincide. -/
theorem step_eq_iff {Y : Set a} {Z : Set b} (hY : V_0.mem Y) (hZ : V_1.mem Z)
    {Y' : Set a} {Z' : Set b} (hY' : V_0.mem Y') (hZ' : V_1.mem Z') :
```

```
    (step Y Z : Set (ApproximableMap V_0 V_1)) = step Y' Z'  <->
      (Z = V_1.master  /\  Z' = V_1.master)  \/  (Y = Y'  /\  Z = Z') := by
  constructor
  * intro h
    by_cases hZeq : Z = V_1.master
    * refine Or.inl <hZeq, ?_>
      rw [hZeq, step_master_right Y hY] at h
      exact (step_eq_univ_iff hY' hZ').mp h.symm
    * exact Or.inr (step_eq_of_ne_master hY hZ hY' hZ' hZeq h)
  * rintro (<rfl, rfl> | <rfl, rfl>)
    * rw [step_master_right Y hY, step_master_right Y' hY']
    * rfl
end

section
set_option linter.unusedVariables false

theorem example_321.{u_1, u_2} : And
  (And
    (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem a}
      {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b} {Y : Set a} {Z :
    Set b},
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V_0 Y  ->
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b V_1 Z  ->
          forall  {Y' : Set a} {Z' : Set b},
            @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V_0 Y'  ->
              @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b V_1 Z'  ->
                Iff
                  (@Eq (Set (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b
    V_0 V_1))
                    (@DomainTheory.NeighbourhoodSystems.step a b V_0 V_1 Y Z)
                    (@DomainTheory.NeighbourhoodSystems.step a b V_0 V_1 Y' Z'))
                  (Or
                    (And (@Eq (Set b) Z (@DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.master b V_1))
                      (@Eq (Set b) Z' (@DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.master b V_1)))
                    (And (@Eq (Set a) Y Y') (@Eq (Set b) Z Z'))))
    (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem a}
      {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b} {Y : Set a} {Z :
    Set b},
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V_0 Y  ->
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b V_1 Z  ->
          forall  {Y' : Set a} {Z' : Set b},
            @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V_0 Y'  ->
              @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b V_1 Z'  ->
                @Ne (Set b) Z (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    master b V_1)  ->
                  @Eq (Set (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0
     V_1))
                      (@DomainTheory.NeighbourhoodSystems.step a b V_0 V_1 Y Z)
                      (@DomainTheory.NeighbourhoodSystems.step a b V_0 V_1 Y' Z')  ->
                    And (@Eq (Set a) Y Y') (@Eq (Set b) Z Z')))
  (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
```

```
  NeighborhoodSystem a}
  {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b} (Y : Set a),
  @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V_0 Y  ->
    @Eq (Set (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1))
      (@DomainTheory.NeighbourhoodSystems.step a b V_0 V_1 Y
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.master b V_1))
      (@Set.univ (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1)))
  :=
@And.intro
(And
  (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem a}
    {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b} {Y : Set a} {Z :
  Set b},
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V_0 Y  ->
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b V_1 Z  ->
        forall  {Y' : Set a} {Z' : Set b},
          @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V_0 Y'  ->
            @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b V_1 Z'  ->
              Iff
                (@Eq (Set (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b
  V_0 V_1))
                  (@DomainTheory.NeighbourhoodSystems.step a b V_0 V_1 Y Z)
                  (@DomainTheory.NeighbourhoodSystems.step a b V_0 V_1 Y' Z'))
                (Or
                  (And (@Eq (Set b) Z (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.master b V_1))
                    (@Eq (Set b) Z' (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.master b V_1)))
                  (And (@Eq (Set a) Y Y') (@Eq (Set b) Z Z'))))
  (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem a}
    {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b} {Y : Set a} {Z :
  Set b},
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V_0 Y  ->
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b V_1 Z  ->
        forall  {Y' : Set a} {Z' : Set b},
          @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V_0 Y'  ->
            @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b V_1 Z'  ->
              @Ne (Set b) Z (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  master b V_1)  ->
                @Eq (Set (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0
   V_1))
                    (@DomainTheory.NeighbourhoodSystems.step a b V_0 V_1 Y Z)
                    (@DomainTheory.NeighbourhoodSystems.step a b V_0 V_1 Y' Z')  ->
                  And (@Eq (Set a) Y Y') (@Eq (Set b) Z Z')))
(forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem a}
  {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b} (Y : Set a),
  @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V_0 Y  ->
    @Eq (Set (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1))
      (@DomainTheory.NeighbourhoodSystems.step a b V_0 V_1 Y
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.master b V_1))
      (@Set.univ (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1)))
(@And.intro
  (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
```

```
    NeighborhoodSystem a}
      {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b} {Y : Set a} {Z :
    Set b},
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V_0 Y  ->
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b V_1 Z  ->
          forall  {Y' : Set a} {Z' : Set b},
            @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V_0 Y'  ->
              @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b V_1 Z'  ->
                Iff
                  (@Eq (Set (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b
    V_0 V_1))
                    (@DomainTheory.NeighbourhoodSystems.step a b V_0 V_1 Y Z)
                    (@DomainTheory.NeighbourhoodSystems.step a b V_0 V_1 Y' Z'))
                  (Or
                    (And (@Eq (Set b) Z (@DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.master b V_1))
                      (@Eq (Set b) Z' (@DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.master b V_1)))
                    (And (@Eq (Set a) Y Y') (@Eq (Set b) Z Z'))))
     (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem a}
      {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b} {Y : Set a} {Z :
    Set b},
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V_0 Y  ->
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b V_1 Z  ->
          forall  {Y' : Set a} {Z' : Set b},
            @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V_0 Y'  ->
              @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b V_1 Z'  ->
                @Ne (Set b) Z (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    master b V_1)  ->
                @Eq (Set (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0
     V_1))
                    (@DomainTheory.NeighbourhoodSystems.step a b V_0 V_1 Y Z)
                    (@DomainTheory.NeighbourhoodSystems.step a b V_0 V_1 Y' Z')  ->
                  And (@Eq (Set a) Y Y') (@Eq (Set b) Z Z'))
    @DomainTheory.NeighbourhoodSystems.step_eq_iff @DomainTheory.NeighbourhoodSystems.
    step_eq_of_ne_master)
  @DomainTheory.NeighbourhoodSystems.step_master_right
end
end
end DomainTheory.NeighbourhoodSystems
end
```

3.4.23 Exercise 3.22 — composition combinator

Prove that there is an approximable mapping

$$\text{comp} : (D_1 \to D_2) \times (D_0 \to D_1) \to (D_0 \to D_2)$$

where for all $g : D_1 \to D_2$ and $f : D_0 \to D_1$ we have

$$\text{comp}(g, f) = g \circ f.$$

Show this directly by writing down the neighbourhood relation and by building the mapping up from eval and curry (on suitable domains) using $\circ$ and $\langle , \rangle$. (Hint: Fill in maps in the following sequence of domains:

1. $(D_0 \to D_1) \times D_0 \to D_1$
2. $(D_1 \to D_2) \times ((D_0 \to D_1) \times D_0) \to (D_1 \to D_2) \times D_1$
3. $((D_1 \to D_2) \times (D_0 \to D_1)) \times D_0 \to (D_1 \to D_2) \times D_1$
4. $((D_1 \to D_2) \times (D_0 \to D_1)) \times D_0 \to D_2$
5. $(D_1 \to D_2) \times (D_0 \to D_1) \to (D_0 \to D_2)$.

The maps are of course not uniquely determined, but the shifting of brackets ought to suggest the right choice.)

Exercise_3_22.lean

```
module
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_12
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem ApproximableMap
section
variable {a b ? : Type*}
variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b} {V_2 :
    NeighborhoodSystem ?}

/-- The uncurried composition map `((?_1 -> ?_2)  x  (?_0 -> ?_1))  x  ?_0  ->  ?_2`,
    `<<g, f>, x>  |->  g(f(x))`,
built from the inner evaluation `(?_0 -> ?_1)  x  ?_0  ->  ?_1`, the outer evaluation
    `(?_1 -> ?_2)  x  ?_1  ->  ?_2`,
and the product projections/pairing (Definition 3.3). -/
def compApp (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) (V_2 :
    NeighborhoodSystem ?) :
    ApproximableMap (prod (prod (funSpace V_1 V_2) (funSpace V_0 V_1)) V_0) V_2 :=
  (evalMap V_1 V_2).comp
    (paired
      ((proj_0 (funSpace V_1 V_2) (funSpace V_0 V_1)).comp
        (proj_0 (prod (funSpace V_1 V_2) (funSpace V_0 V_1)) V_0))
      ((evalMap V_0 V_1).comp
        (paired
          ((proj_1 (funSpace V_1 V_2) (funSpace V_0 V_1)).comp
            (proj_0 (prod (funSpace V_1 V_2) (funSpace V_0 V_1)) V_0))
          (proj_1 (prod (funSpace V_1 V_2) (funSpace V_0 V_1)) V_0))))

/- concordance: source L1545 EXERCISE 3.22 composition combinator -/
/-- **Exercise 3.22 (Scott 1981, PRG-19).** The composition mapping
`comp : (?_1  ->  ?_2)  x  (?_0  ->  ?_1)  ->  (?_0  ->  ?_2)`, `comp = curry(compApp)
    `. -/
```

```
def compMap (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) (V_2 :
    NeighborhoodSystem ?) :
    ApproximableMap (prod (funSpace V_1 V_2) (funSpace V_0 V_1)) (funSpace V_0 V_2) :=
  curry (compApp V_0 V_1 V_2)

/-- `compApp(<<g, f>, x>) = g(f(x))`. -/
theorem toElementMap_compApp (Gphi : (funSpace V_1 V_2).Element) (Fphi : (funSpace V_0
     V_1).Element)
    (x : V_0.Element) :
    (compApp V_0 V_1 V_2).toElementMap (pair (pair Gphi Fphi) x)
      = (toApproxMap Gphi).toElementMap ((toApproxMap Fphi).toElementMap x) := by
  rw [compApp]
  simp only [toElementMap_comp, toElementMap_paired, toElementMap_proj_0,
    toElementMap_proj_1,
    fst_pair, snd_pair, evalMap_apply]

/-- **Exercise 3.22 (Scott 1981, PRG-19).** `comp(g, f) = g  comp  f`: applying the
    value
`comp(<g, f>)  in  |?_0  ->  ?_2|` to `x` gives `g(f(x))`. -/
theorem toElementMap_compMap_apply (Gphi : (funSpace V_1 V_2).Element) (Fphi : (
    funSpace V_0 V_1).Element)
    (x : V_0.Element) :
    (toApproxMap ((compMap V_0 V_1 V_2).toElementMap (pair Gphi Fphi))).toElementMap x
      = (toApproxMap Gphi).toElementMap ((toApproxMap Fphi).toElementMap x) := by
  rw [compMap, toElementMap_curry_apply, toElementMap_compApp]

/-- **Exercise 3.22 (Scott 1981, PRG-19).** The relational form `comp(g, f) = g  comp
    f`: the value of
`comp` at `<g, f>`, read back as an approximable map, is the composition of the
    approximable maps. -/
theorem toApproxMap_compMap (Gphi : (funSpace V_1 V_2).Element) (Fphi : (funSpace V_0
    V_1).Element) :
    toApproxMap ((compMap V_0 V_1 V_2).toElementMap (pair Gphi Fphi))
      = (toApproxMap Gphi).comp (toApproxMap Fphi) := by
  apply ext_of_toElementMap
  intro x
  rw [toElementMap_compMap_apply, toElementMap_comp]
end

section
set_option linter.unusedVariables false

theorem example_322.{u_1, u_2, u_3} : And
  (forall  {a : Type u_1} {b : Type u_2} {? : Type u_3} {V_0 : DomainTheory.
    NeighbourhoodSystems.NeighborhoodSystem a}
    {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
    {V_2 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem ?}
    (Gphi :
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2)
        (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
    (Fphi :
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1)
        (@DomainTheory.NeighbourhoodSystems.funSpace a b V_0 V_1)),
    @Eq (@DomainTheory.NeighbourhoodSystems.ApproximableMap a ? V_0 V_2)
```

```
    (@DomainTheory.NeighbourhoodSystems.toApproxMap a ? V_0 V_2
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
        (Sum (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2)
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1))
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap a ? V_0 V_2)
        (@DomainTheory.NeighbourhoodSystems.prod (@DomainTheory.NeighbourhoodSystems
  .ApproximableMap b ? V_1 V_2)
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1)
          (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2)
          (@DomainTheory.NeighbourhoodSystems.funSpace a b V_0 V_1))
        (@DomainTheory.NeighbourhoodSystems.funSpace a ? V_0 V_2)
        (@DomainTheory.NeighbourhoodSystems.compMap a b ? V_0 V_1 V_2)
        (@DomainTheory.NeighbourhoodSystems.pair (@DomainTheory.NeighbourhoodSystems
  .ApproximableMap b ? V_1 V_2)
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1)
          (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2)
          (@DomainTheory.NeighbourhoodSystems.funSpace a b V_0 V_1) Gphi Fphi)))
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp a b ? V_0 V_1 V_2
      (@DomainTheory.NeighbourhoodSystems.toApproxMap b ? V_1 V_2 Gphi)
      (@DomainTheory.NeighbourhoodSystems.toApproxMap a b V_0 V_1 Fphi)))
(forall  {a : Type u_1} {b : Type u_2} {? : Type u_3} {V_0 : DomainTheory.
  NeighbourhoodSystems.NeighborhoodSystem a}
  {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
  {V_2 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem ?}
  (Gphi :
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2)
      (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
  (Fphi :
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1)
      (@DomainTheory.NeighbourhoodSystems.funSpace a b V_0 V_1))
  (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V_0),
  @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element ? V_2)
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a ? V_0 V_2
      (@DomainTheory.NeighbourhoodSystems.toApproxMap a ? V_0 V_2
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
          (Sum (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2)
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1))
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a ? V_0 V_2)
          (@DomainTheory.NeighbourhoodSystems.prod (@DomainTheory.
  NeighbourhoodSystems.ApproximableMap b ? V_1 V_2)
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1)
            (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2)
            (@DomainTheory.NeighbourhoodSystems.funSpace a b V_0 V_1))
          (@DomainTheory.NeighbourhoodSystems.funSpace a ? V_0 V_2)
          (@DomainTheory.NeighbourhoodSystems.compMap a b ? V_0 V_1 V_2)
          (@DomainTheory.NeighbourhoodSystems.pair (@DomainTheory.
  NeighbourhoodSystems.ApproximableMap b ? V_1 V_2)
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1)
            (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2)
            (@DomainTheory.NeighbourhoodSystems.funSpace a b V_0 V_1) Gphi Fphi)))
      x)
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap b ? V_1 V_2
      (@DomainTheory.NeighbourhoodSystems.toApproxMap b ? V_1 V_2 Gphi)
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a b V_0 V_1
```

```
        (@DomainTheory.NeighbourhoodSystems.toApproxMap a b V_0 V_1 Fphi) x))) :=
@And.intro
(forall  {a : Type u_1} {b : Type u_2} {? : Type u_3} {V_0 : DomainTheory.
  NeighbourhoodSystems.NeighborhoodSystem a}
  {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
  {V_2 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem ?}
  (Gphi :
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2)
      (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
  (Fphi :
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1)
      (@DomainTheory.NeighbourhoodSystems.funSpace a b V_0 V_1)),
  @Eq (@DomainTheory.NeighbourhoodSystems.ApproximableMap a ? V_0 V_2)
    (@DomainTheory.NeighbourhoodSystems.toApproxMap a ? V_0 V_2
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
        (Sum (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2)
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1))
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap a ? V_0 V_2)
        (@DomainTheory.NeighbourhoodSystems.prod (@DomainTheory.NeighbourhoodSystems
  .ApproximableMap b ? V_1 V_2)
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1)
          (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2)
          (@DomainTheory.NeighbourhoodSystems.funSpace a b V_0 V_1))
        (@DomainTheory.NeighbourhoodSystems.funSpace a ? V_0 V_2)
        (@DomainTheory.NeighbourhoodSystems.compMap a b ? V_0 V_1 V_2)
        (@DomainTheory.NeighbourhoodSystems.pair (@DomainTheory.NeighbourhoodSystems
  .ApproximableMap b ? V_1 V_2)
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1)
          (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2)
          (@DomainTheory.NeighbourhoodSystems.funSpace a b V_0 V_1) Gphi Fphi)))
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp a b ? V_0 V_1 V_2
      (@DomainTheory.NeighbourhoodSystems.toApproxMap b ? V_1 V_2 Gphi)
      (@DomainTheory.NeighbourhoodSystems.toApproxMap a b V_0 V_1 Fphi)))
(forall  {a : Type u_1} {b : Type u_2} {? : Type u_3} {V_0 : DomainTheory.
  NeighbourhoodSystems.NeighborhoodSystem a}
  {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
  {V_2 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem ?}
  (Gphi :
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2)
      (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
  (Fphi :
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1)
      (@DomainTheory.NeighbourhoodSystems.funSpace a b V_0 V_1))
  (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V_0),
  @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element ? V_2)
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a ? V_0 V_2
      (@DomainTheory.NeighbourhoodSystems.toApproxMap a ? V_0 V_2
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
          (Sum (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2)
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1))
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a ? V_0 V_2)
          (@DomainTheory.NeighbourhoodSystems.prod (@DomainTheory.
```

```
    NeighbourhoodSystems.ApproximableMap b ? V_1 V_2)
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1)
              (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2)
              (@DomainTheory.NeighbourhoodSystems.funSpace a b V_0 V_1))
            (@DomainTheory.NeighbourhoodSystems.funSpace a ? V_0 V_2)
            (@DomainTheory.NeighbourhoodSystems.compMap a b ? V_0 V_1 V_2)
            (@DomainTheory.NeighbourhoodSystems.pair (@DomainTheory.
    NeighbourhoodSystems.ApproximableMap b ? V_1 V_2)
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1)
              (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2)
              (@DomainTheory.NeighbourhoodSystems.funSpace a b V_0 V_1) Gphi Fphi)))
        x)
       (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap b ? V_1 V_2
         (@DomainTheory.NeighbourhoodSystems.toApproxMap b ? V_1 V_2 Gphi)
         (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a b V_0 V_1
           (@DomainTheory.NeighbourhoodSystems.toApproxMap a b V_0 V_1 Fphi) x)))
  @DomainTheory.NeighbourhoodSystems.toApproxMap_compMap @DomainTheory.
    NeighbourhoodSystems.toElementMap_compMap_apply
end
end
end DomainTheory.NeighbourhoodSystems
end
```

3.4.24 Exercise 3.23 — cartesian closed category

(For category theorists.) Show that the results of 3.11 and 3.12 prove that the category of domains and approximable mappings is a *cartesian closed category*. (Mac Lane [1971] pp. 95–96 may be consulted for a very brief introduction.) What is the *terminal domain* in this category? What sort of functor is $(D_0 \to D_1)$?

Exercise_3_23.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Exercise_3_15
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_12
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Lemma_3_6
@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem ApproximableMap
section
variable {a b ? : Type*} (V : NeighborhoodSystem a)
variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b} {V_2 :
    NeighborhoodSystem ?}

/-- There is at most one approximable mapping into the terminal domain `?`: the
    codomain `|?|`
is a subsingleton, so any two maps have the same elementwise action. -/
instance : Subsingleton (ApproximableMap V unitSys) :=
  <fun _ _ => ext_of_toElementMap fun _ => Subsingleton.elim _ _>

/- concordance: source L1569 EXERCISE 3.23 cartesian closed category -/
```

```
/-- **Exercise 3.23 (Scott 1981, PRG-19).** `? = unitSys` is the *terminal object*: for
     every domain
`?` there is a unique approximable mapping `?  ->  ?` (the constant map at `False`). -/
instance : Unique (ApproximableMap V unitSys) where
  default := constMap V (default : unitSys.Element)
  uniq _ := Subsingleton.elim _ _

/-- **Exercise 3.23 (Scott 1981, PRG-19).** The cartesian-closed adjunction
`Hom(?_0  x  ?_1, ?_2) ?o Hom(?_0, (?_1  ->  ?_2))`, exhibiting `(?_1  ->  ?_2)` as the
     exponential object. This
is exactly `curryEquiv` of Theorem 3.12. -/
def homAdjunction (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b)
    (V_2 : NeighborhoodSystem ?) :
    ApproximableMap (prod V_0 V_1) V_2 ?o ApproximableMap V_0 (funSpace V_1 V_2) :=
  curryEquiv V_0 V_1 V_2

/-- **Exercise 3.23 (Scott 1981, PRG-19).** The unique map to the terminal object,
    named. -/
def toUnit : ApproximableMap V unitSys := default

theorem toUnit_unique (f : ApproximableMap V unitSys) : f = toUnit V := Subsingleton.
    elim _ _
end

section
set_option linter.unusedVariables false

theorem example_323.{u_1} : forall  {a : Type u_1} (V : DomainTheory.
    NeighbourhoodSystems.NeighborhoodSystem a)
  (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a Unit V DomainTheory.
    NeighbourhoodSystems.unitSys),
  @Eq (@DomainTheory.NeighbourhoodSystems.ApproximableMap a Unit V DomainTheory.
    NeighbourhoodSystems.unitSys) f
    (@DomainTheory.NeighbourhoodSystems.toUnit a V) :=
  @DomainTheory.NeighbourhoodSystems.toUnit_unique
end
end

end DomainTheory.NeighbourhoodSystems
end
```

3.4.25 Exercise 3.24 — function-space isomorphisms

Establish some more isomorphisms :

(i) $(D_0 \to (D_1 \times D_2)) \cong (D_0 \to D_1) \times (D_0 \to D_2)$

(ii) $(D_0 \to D_1^{\infty}) \cong (D_0 \to D_1)^{\infty}$

(iii) $D_0 \times (D_1 + D_2) \cong (D_0 \times D_1) + (D_0 \times D_2)$

(iv) $(D_0 + D_1) \to D_2 \cong (D_0 \to D_2) \times (D_1 \to D_2) \ldots$

If some of the above are not true, perhaps at least some mapping relationships can be established.

Exercise_3_24.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_13
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_10
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_4
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_5
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Exercise_3_16
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Exercise_3_18
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Exercise_3_19
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_5
@[expose] public section

section
/- concordance: source L1571 EXERCISE 3.24 function-space isomorphisms -/
-- Exercise324/funProdEquiv.lean
namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap

variable {a b ? : Type*}

variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b} {V_2 :
    NeighborhoodSystem ?}

/-- **Exercise 3.24(i) (Scott 1981, PRG-19).** The order-isomorphism on approximable
    maps
`Hom(?_0, ?_1  x  ?_2) ?o Hom(?_0, ?_1)  x  Hom(?_0, ?_2)`, `h  |->  (p_0  comp  h, p_1
      comp  h)`. -/
def funProdEquiv (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b)
    (V_2 : NeighborhoodSystem ?) :
    ApproximableMap V_0 (prod V_1 V_2) ?o (ApproximableMap V_0 V_1  x  ApproximableMap
     V_0 V_2) where
  toFun h := ((proj_0 V_1 V_2).comp h, (proj_1 V_1 V_2).comp h)
  invFun p := paired p.1 p.2
  left_inv h := paired_proj h
  right_inv p := by
    ext1
    * exact proj_0_comp_paired p.1 p.2
    * exact proj_1_comp_paired p.1 p.2
  map_rel_iff' := by
    intro h h'
    constructor
    * rintro <h0, h1>
      rw [ <-  paired_proj h,  <-  paired_proj h']
      intro Z P hrel
      obtain <hP, hfP, hgP> := hrel
      exact <hP, h0 _ _ hfP, h1 _ _ hgP>
    * intro hle
      refine <fun X Y hrel => ?_, fun X Y hrel => ?_>
      * obtain <W, hW, hYW> := hrel
        exact <W, hle X W hW, hYW>
      * obtain <W, hW, hYW> := hrel
```

```
        exact <W, hle X W hW, hYW>

end DomainTheory.NeighbourhoodSystems
end

section
namespace DomainTheory.NeighbourhoodSystems
section

open NeighborhoodSystem ApproximableMap

section
variable {a b ? : Type*}
variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b} {V_2 :
    NeighborhoodSystem ?}

/-- **Exercise 3.24(i) (Scott 1981, PRG-19).** The domain isomorphism
`|?_0  ->  (?_1  x  ?_2)| ?o |(?_0  ->  ?_1)  x  (?_0  ->  ?_2)|`. -/
def funProdIso (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b)
    (V_2 : NeighborhoodSystem ?) :
    (funSpace V_0 (prod V_1 V_2)).Element ?o (prod (funSpace V_0 V_1) (funSpace V_0
    V_2)).Element :=
  (funSpaceEquiv V_0 (prod V_1 V_2)).trans <|
    (funProdEquiv V_0 V_1 V_2).trans <|
      (prodCongrOrderIso (funSpaceEquiv V_0 V_1).symm (funSpaceEquiv V_0 V_2).symm).
    trans
        (prodEquiv (funSpace V_0 V_1) (funSpace V_0 V_2)).symm

/-- **Exercise 3.24(i).** `(?_0  ->  (?_1  x  ?_2))  ~=  (?_0  ->  ?_1)  x  (?_0  ->  ?
    _2)`. -/
theorem funProd_isomorphic :
    funSpace V_0 (prod V_1 V_2)  ~= ? prod (funSpace V_0 V_1) (funSpace V_0 V_2) :=
  <funProdIso V_0 V_1 V_2>
end

section
set_option linter.unusedVariables false

theorem example_324.{u_1, u_2, u_3} : forall  {a : Type u_1} {b : Type u_2} {? : Type
    u_3} {V_0 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
  {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
  {V_2 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem ?},
  @DomainTheory.NeighbourhoodSystems.Isomorphic
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap a (Sum b ?) V_0
      (@DomainTheory.NeighbourhoodSystems.prod b ? V_1 V_2))
    (Sum (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1)
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap a ? V_0 V_2))
    (@DomainTheory.NeighbourhoodSystems.funSpace a (Sum b ?) V_0 (@DomainTheory.
    NeighbourhoodSystems.prod b ? V_1 V_2))
    (@DomainTheory.NeighbourhoodSystems.prod (@DomainTheory.NeighbourhoodSystems.
    ApproximableMap a b V_0 V_1)
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap a ? V_0 V_2)
      (@DomainTheory.NeighbourhoodSystems.funSpace a b V_0 V_1) (@DomainTheory.
    NeighbourhoodSystems.funSpace a ? V_0 V_2)) :=
  @DomainTheory.NeighbourhoodSystems.funProd_isomorphic
end
```

```
section
variable {a b : Type*} {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}

/-- The `exists /forall `-swap behind `mapOfSeq`: a single input neighbourhood `X  in
    x` works for all coordinates
iff each coordinate has its own (intersect the finitely many non-trivial ones). -/
theorem exists_forall_swap {x : V_0.Element} {W : Set (Nat  x  b)}
    {g : Nat  ->  ApproximableMap V_0 V_1} (hW : (iterSys V_1).mem W) :
    (exists  X, x.mem X  /\  forall  i, (g i).rel X (fiber W i))  <->
      (forall  i, exists  X, x.mem X  /\  (g i).rel X (fiber W i)) := by
  constructor
  * rintro <X, hXx, hall> i; exact <X, hXx, hall i>
  * intro hall
    obtain <N, hN> := hW.2
    choose Xs hXx hXrel using hall
    have hbase : x.mem (V_0.interUpTo Xs N) := x.mem_interUpTo Xs (n := N) (fun i _ =>
     hXx i)
    refine <V_0.interUpTo Xs N, hbase, fun i => ?_>
    by_cases h : i < N
    * exact (g i).mono (hXrel i) (V_0.interUpTo_subset Xs h) subset_rfl (x.sub hbase)
        ((g i).rel_cod (hXrel i))
    * rw [hN i (not_lt.mp h)]; exact (g i).rel_master (x.sub hbase)

/-- A map `?_0  ->  ?_1^ inf ` is detected coordinate-wise: `f  sqsub  f'` iff `proj_n
     comp  f  sqsub  proj_n  comp  f'` for all `n`. -/
theorem le_of_projComp_le {f f' : ApproximableMap V_0 (iterSys V_1)}
    (h : forall  n, (projN V_1 n).comp f  <=  (projN V_1 n).comp f') : f  <=  f' := by
  rw [le_iff_toElementMap_le]
  intro x
  apply le_of_component_le
  intro n
  have hx := (le_iff_toElementMap_le.mp (h n)) x
  rwa [toElementMap_comp, toElementMap_comp, toElementMap_projN, toElementMap_projN]
    at hx

/-- The "infinite pairing" map `?_0  ->  ?_1^ inf ` of a sequence `g : Nat  ->  Hom(?_0
    , ?_1)`:
`X (mapOfSeq g) W  <->  X  in  ?_0, W  in  ?_1^ inf , and X g_i (fiber W i) for all i`.
     -/
def mapOfSeq (g : Nat  ->  ApproximableMap V_0 V_1) : ApproximableMap V_0 (iterSys V_1
    ) where
  rel X W := V_0.mem X  /\  (iterSys V_1).mem W  /\  forall  i, (g i).rel X (fiber W i
    )
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := <V_0.master_mem, (iterSys V_1).master_mem, fun i => by
    rw [fiber_iterSys_master]; exact (g i).master_rel>
  inter_right := by
    rintro X W W' <hX, hW, hrel> <_, hW', hrel'>
    refine <hX, <fun i => (g i).rel_cod ((g i).inter_right (hrel i) (hrel' i)), ?_>,
      fun i => (g i).inter_right (hrel i) (hrel' i)>
    obtain <NW, hNW> := hW.2
    obtain <NW', hNW'> := hW'.2
    refine <max NW NW', fun i hi => ?_>
    rw [fiber_inter, hNW i (le_trans (le_max_left _ _) hi),
```

```
      hNW' i (le_trans (le_max_right _ _) hi), Set.inter_self]
  mono := by
    rintro X X' W W' <_, _, hrel> hX'X hWW' hX' hW'
    exact <hX', hW', fun i => (g i).mono (hrel i) hX'X (fiber_mono hWW' i) hX' (hW'.1
    i)>

/-- A pointwise family of order-isomorphisms induces one on the dependent product. -/
def piCongrOrderIso {? : Type*} {A B : ?  ->  Type*} [forall  i, Preorder (A i)] [
    forall  i, Preorder (B i)]
    (e : forall  i, A i ?o B i) : (forall  i, A i) ?o (forall  i, B i) where
  toFun f := fun i => e i (f i)
  invFun g := fun i => (e i).symm (g i)
  left_inv f := by funext i; simp
  right_inv g := by funext i; simp
  map_rel_iff' := by
    intro f g
    exact <fun h i => (e i).le_iff_le.mp (h i), fun h i => (e i).le_iff_le.mpr (h i)>

/-- `proj_n  comp  f` is monotone in `f`. -/
theorem projComp_mono {f f' : ApproximableMap V_0 (iterSys V_1)} (h : f  <=  f') (n :
    Nat) :
    (projN V_1 n).comp f  <=  (projN V_1 n).comp f' := by
  rintro X Y <W, hfXW, hWY>
  exact <W, h X W hfXW, hWY>

/-- The value of `mapOfSeq g` is the sequence of values `<g_i(x)>`. -/
theorem toElementMap_mapOfSeq (g : Nat  ->  ApproximableMap V_0 V_1) (x : V_0.Element)
    :
    (mapOfSeq g).toElementMap x = ofSeq (fun i => (g i).toElementMap x) := by
  apply Element.ext
  intro W
  constructor
  * rintro <X, hXx, _, hW, hrel>
    exact <hW, fun i => <X, hXx, hrel i>>
  * rintro <hW, hfib>
    obtain <X, hXx, hrel> := (exists_forall_swap hW).mpr (fun i => hfib i)
    exact <X, hXx, x.sub hXx, hW, hrel>

/-- **Exercise 3.24(ii) (Scott 1981, PRG-19).** The order-isomorphism on approximable
    maps
`Hom(?_0, ?_1^ inf ) ?o (Nat  ->  Hom(?_0, ?_1))`, `f  |->  (n  |->  proj_n  comp  f)`.
    -/
def funIterEquiv (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) :
    ApproximableMap V_0 (iterSys V_1) ?o (forall  _ : Nat, ApproximableMap V_0 V_1)
    where
  toFun f := fun n => (projN V_1 n).comp f
  invFun g := mapOfSeq g
  left_inv f := by
    apply ext_of_toElementMap
    intro x
    rw [toElementMap_mapOfSeq]
    have hfun : (fun i => ((projN V_1 i).comp f).toElementMap x)
        = (fun i => component (f.toElementMap x) i) := by
      funext i; rw [toElementMap_comp, toElementMap_projN]
    rw [hfun, ofSeq_component]
  right_inv g := by
```

```
    funext n
    apply ext_of_toElementMap
    intro x
    rw [toElementMap_comp, toElementMap_mapOfSeq, toElementMap_projN, component_ofSeq]
  map_rel_iff' := by
    intro f f'
    constructor
    * intro h; exact le_of_projComp_le (fun n => h n)
    * intro h n; exact projComp_mono h n

/-- **Exercise 3.24(ii) (Scott 1981, PRG-19).** The domain isomorphism
`|?_0  ->  ?_1^ inf | ?o |(?_0  ->  ?_1)^ inf |`. -/
def funIterIso (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) :
    (funSpace V_0 (iterSys V_1)).Element ?o (iterSys (funSpace V_0 V_1)).Element :=
  (funSpaceEquiv V_0 (iterSys V_1)).trans <|
    (funIterEquiv V_0 V_1).trans <|
      (piCongrOrderIso (fun _ : Nat => (funSpaceEquiv V_0 V_1).symm)).trans
        (iterSeqEquiv (funSpace V_0 V_1)).symm

/-- **Exercise 3.24(ii).** `(?_0  ->  ?_1^ inf )  ~=  (?_0  ->  ?_1)^ inf `. -/
theorem funIter_isomorphic : funSpace V_0 (iterSys V_1)  ~= ? iterSys (funSpace V_0
    V_1) :=
  <funIterIso V_0 V_1>
end

section
set_option linter.unusedVariables false

theorem example_324_iter.{u_1, u_2} : forall  {a : Type u_1} {b : Type u_2} {V_0 :
    DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
  {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b},
  @DomainTheory.NeighbourhoodSystems.Isomorphic
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap a (Prod Nat b) V_0
      (@DomainTheory.NeighbourhoodSystems.iterSys b V_1))
    (Prod Nat (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1))
    (@DomainTheory.NeighbourhoodSystems.funSpace a (Prod Nat b) V_0 (@DomainTheory.
    NeighbourhoodSystems.iterSys b V_1))
    (@DomainTheory.NeighbourhoodSystems.iterSys (@DomainTheory.NeighbourhoodSystems.
    ApproximableMap a b V_0 V_1)
      (@DomainTheory.NeighbourhoodSystems.funSpace a b V_0 V_1)) :=
  @DomainTheory.NeighbourhoodSystems.funIter_isomorphic
end

section
open NeighborhoodSystem ApproximableMap
variable {a b ? : Type*}
variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b} {V_2 :
    NeighborhoodSystem ?}
variable {h_0 : forall  X, V_0.mem X  ->  X.Nonempty} {h_1 : forall  Y, V_1.mem Y  ->
     Y.Nonempty}

/-- **Exercise 3.24(iv) (Scott 1981, PRG-19).** The *copairing* `[a, b] : ?_0 + ?_1  ->
      ?_2`: a left copy
`0X` is routed through `a`, a right copy `1Y` through `b`, and the basepoint `?` to `
    False` (so `?`
relates only to `Delta_2`). -/
```

```
def copair (a : ApproximableMap V_0 V_2) (b : ApproximableMap V_1 V_2) :
    ApproximableMap (sum V_0 V_1 h_0 h_1) V_2 where
  rel W Z := (sum V_0 V_1 h_0 h_1).mem W  /\  V_2.mem Z  /\
    (Z = V_2.master  \/
      (exists  X, W = inj_0 X  /\  a.rel X Z)  \/
      (exists  Y, W = inj_1 Y  /\  b.rel Y Z))
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := <(sum V_0 V_1 h_0 h_1).master_mem, V_2.master_mem, Or.inl rfl>
  inter_right := by
    rintro W Z Z' <hW, _, hd> <_, _, hd'>
    have hmem : forall  Z'' : Set ?,
        (Z'' = V_2.master  \/  (exists  X, W = inj_0 X  /\  a.rel X Z'')  \/  (exists
     Y, W = inj_1 Y  /\  b.rel Y Z''))  ->
        V_2.mem Z'' := by
      rintro Z'' (rfl | <_, _, hf> | <_, _, hg>)
      * exact V_2.master_mem
      * exact a.rel_cod hf
      * exact b.rel_cod hg
    have key : Z  I  Z' = V_2.master  \/  (exists  X, W = inj_0 X  /\  a.rel X (Z  I
    Z'))  \/
        (exists  Y, W = inj_1 Y  /\  b.rel Y (Z  I  Z')) := by
      rcases hd with rfl | <X, hWX, hf> | <Y, hWY, hg>
      * rw [Set.inter_eq_right.mpr (V_2.sub_master (hmem _ hd'))]; exact hd'
      * rcases hd' with rfl | <X', hWX', hf'> | <Y', hWY', hg'>
        * rw [Set.inter_eq_left.mpr (V_2.sub_master (a.rel_cod hf))]
          exact Or.inr (Or.inl <X, hWX, hf>)
        * obtain rfl : X = X' := inj_0_injective (hWX  |>  hWX')
          exact Or.inr (Or.inl <X, hWX, a.inter_right hf hf'>)
        * exact absurd (hWX  |>  hWY' : (inj_0 X : Set _) = inj_1 Y')
            (fun h => not_inj_0_subset_inj_1 (h_0 X (a.rel_dom hf)) h.subset)
      * rcases hd' with rfl | <X', hWX', hf'> | <Y', hWY', hg'>
        * rw [Set.inter_eq_left.mpr (V_2.sub_master (b.rel_cod hg))]
          exact Or.inr (Or.inr <Y, hWY, hg>)
        * exact absurd (hWY  |>  hWX' : (inj_1 Y : Set _) = inj_0 X')
            (fun h => not_inj_1_subset_inj_0 (h_1 Y (b.rel_dom hg)) h.subset)
        * obtain rfl : Y = Y' := inj_1_injective (hWY  |>  hWY')
          exact Or.inr (Or.inr <Y, hWY, b.inter_right hg hg'>)
    exact <hW, hmem _ key, key>
  mono := by
    rintro W W_2 Z Z' <_, _, hd> hW_2W hZZ' hW_2 hZ'
    refine <hW_2, hZ', ?_>
    rcases hd with rfl | <X, rfl, hf> | <Y, rfl, hg>
    * left; exact Set.Subset.antisymm (V_2.sub_master hZ') hZZ'
    * obtain <X_2, hX_2, rfl> := mem_subset_inj_0 hW_2 hW_2W
      exact Or.inr (Or.inl <X_2, rfl, a.mono hf (inj_0_subset_inj_0.mp hW_2W) hZZ'
    hX_2 hZ'>)
    * obtain <Y_2, hY_2, rfl> := mem_subset_inj_1 hW_2 hW_2W
      exact Or.inr (Or.inr <Y_2, rfl, b.mono hg (inj_1_subset_inj_1.mp hW_2W) hZZ'
    hY_2 hZ'>)

@[simp] theorem copair_rel {a : ApproximableMap V_0 V_2} {b : ApproximableMap V_1 V_2}
    {W : Set (Option (a ? b))} {Z : Set ?} :
    (copair (h_0 := h_0) (h_1 := h_1) a b).rel W Z  <->  (sum V_0 V_1 h_0 h_1).mem W
    /\  V_2.mem Z  /\
      (Z = V_2.master  \/  (exists  X, W = inj_0 X  /\  a.rel X Z)  \/  (exists  Y, W
```

```
    = inj_1 Y  /\  b.rel Y Z)) := Iff.rfl

/-- The canonical comparison `Hom(?_0+?_1, ?_2)  ->  Hom(?_0,?_2)  x  Hom(?_1,?_2)`, `h
      |->  (h comp in_0, h comp in_1)`. -/
def copairProj (h : ApproximableMap (sum V_0 V_1 h_0 h_1) V_2) :
    ApproximableMap V_0 V_2  x  ApproximableMap V_1 V_2 :=
  (h.comp inMap_0, h.comp inMap_1)

/-- A product neighbourhood over non-empty factors is non-empty. -/
theorem prod_mem_nonempty (hn_0 : forall  X, V_0.mem X  ->  X.Nonempty) (_hn_1 :
    forall  Y, V_1.mem Y  ->  Y.Nonempty)
    (W : Set (a ? b)) (hW : (prod V_0 V_1).mem W) : W.Nonempty := by
  obtain <X, Y, hX, _, rfl> := hW
  obtain <a, ha> := hn_0 X hX
  exact <Sum.inl a, mem_prodNbhd_inl.mpr ha>

/-- **Exercise 3.24(iii) (Scott 1981, PRG-19).** The canonical *distribution*
    approximable map
`(?_0  x  ?_1) + (?_0  x  ?_2)  ->  ?_0  x  (?_1 + ?_2)`, `in_i<x, u>  |->  <x, in_i u
    >`. (This direction always exists;
the reverse map / isomorphism does not, since the left side has an element `<x, False>`
     for each `x`.) -/
def distribMap (hn_0 : forall  X, V_0.mem X  ->  X.Nonempty) (hn_1 : forall  Y, V_1.
    mem Y  ->  Y.Nonempty)
    (hn_2 : forall  Z, V_2.mem Z  ->  Z.Nonempty) :
    ApproximableMap
      (sum (prod V_0 V_1) (prod V_0 V_2) (prod_mem_nonempty hn_0 hn_1) (
    prod_mem_nonempty hn_0 hn_2))
      (prod V_0 (sum V_1 V_2 hn_1 hn_2)) :=
  copair
    (paired (proj_0 V_0 V_1) ((inMap_0 (h_0 := hn_1) (h_1 := hn_2)).comp (proj_1 V_0
    V_1)))
    (paired (proj_0 V_0 V_2) ((inMap_1 (h_0 := hn_1) (h_1 := hn_2)).comp (proj_1 V_0
    V_2)))

/-- **Exercise 3.24(iv) (Scott 1981, PRG-19).** `[a, b]  comp  in_0 = a`. -/
theorem copair_comp_inMap_0 (a : ApproximableMap V_0 V_2) (b : ApproximableMap V_1 V_2
    ) :
    (copair (h_0 := h_0) (h_1 := h_1) a b).comp inMap_0 = a := by
  apply ApproximableMap.ext
  intro X Z
  constructor
  * rintro <W, <hX, _, hinj>, _, hZ, hd>
    rcases hd with rfl | <X', hWX', hf> | <Y', hWY', hg>
    * exact a.rel_master hX
    * exact a.mono hf (inj_0_subset_inj_0.mp (hWX'  |>  hinj)) subset_rfl hX hZ
    * exact (not_inj_0_subset_inj_1 (h_0 X hX) (hWY'  |>  hinj)).elim
  * intro hf
    exact <inj_0 X, <a.rel_dom hf, Or.inr (Or.inl <X, a.rel_dom hf, rfl>), subset_rfl
    >,
      Or.inr (Or.inl <X, a.rel_dom hf, rfl>), a.rel_cod hf,
      Or.inr (Or.inl <X, rfl, hf>)>

/-- **Exercise 3.24(iv) (Scott 1981, PRG-19).** `[a, b]  comp  in_1 = b`. -/
theorem copair_comp_inMap_1 (a : ApproximableMap V_0 V_2) (b : ApproximableMap V_1 V_2
    ) :
```

```
    (copair (h_0 := h_0) (h_1 := h_1) a b).comp inMap_1 = b := by
  apply ApproximableMap.ext
  intro Y Z
  constructor
  * rintro <W, <hY, _, hinj>, _, hZ, hd>
    rcases hd with rfl | <X', hWX', hf> | <Y', hWY', hg>
    * exact b.rel_master hY
    * exact (not_inj_1_subset_inj_0 (h_1 Y hY) (hWX'  |>  hinj)).elim
    * exact b.mono hg (inj_1_subset_inj_1.mp (hWY'  |>  hinj)) subset_rfl hY hZ
  * intro hg
    exact <inj_1 Y, <b.rel_dom hg, Or.inr (Or.inr <Y, b.rel_dom hg, rfl>), subset_rfl
    >,
      Or.inr (Or.inr <Y, b.rel_dom hg, rfl>), b.rel_cod hg,
      Or.inr (Or.inr <Y, rfl, hg>)>

/-- **Exercise 3.24(iv) (Scott 1981, PRG-19).** `(?_0 -> ?_2)  x  (?_1 -> ?_2)` is a **
    retract** of
`(?_0+?_1)  ->  ?_2`: the copairing is a section of `h  |->  (h comp in_0, h comp in_1)
    `. (It is *not* an isomorphism: the
value of a map on the basepoint `?` is not recoverable from its restrictions to the two
     copies.) -/
theorem copairProj_copair (a : ApproximableMap V_0 V_2) (b : ApproximableMap V_1 V_2)
    :
    copairProj (h_0 := h_0) (h_1 := h_1) (copair a b) = (a, b) := by
  show ((copair (h_0 := h_0) (h_1 := h_1) a b).comp inMap_0,
        (copair (h_0 := h_0) (h_1 := h_1) a b).comp inMap_1) = (a, b)
  rw [copair_comp_inMap_0, copair_comp_inMap_1]
end

section
set_option linter.unusedVariables false

theorem example_324_distrib.{u_1, u_2, u_3} : And
  (And
    (forall  {a : Type u_1} {b : Type u_2} {? : Type u_3} {V_0 : DomainTheory.
   NeighbourhoodSystems.NeighborhoodSystem a}
      {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
      {V_2 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem ?}
      {h_0 : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.
   NeighborhoodSystem.mem a V_0 X  ->  @Set.Nonempty a X}
      {h_1 : forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.
   NeighborhoodSystem.mem b V_1 Y  ->  @Set.Nonempty b Y}
      (a : @DomainTheory.NeighbourhoodSystems.ApproximableMap a ? V_0 V_2)
      (b : @DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2),
      @Eq
        (Prod (@DomainTheory.NeighbourhoodSystems.ApproximableMap a ? V_0 V_2)
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2))
        (@DomainTheory.NeighbourhoodSystems.copairProj a b ? V_0 V_1 V_2 h_0 h_1
          (@DomainTheory.NeighbourhoodSystems.copair a b ? V_0 V_1 V_2 h_0 h_1 a b))
        (@Prod.mk (@DomainTheory.NeighbourhoodSystems.ApproximableMap a ? V_0 V_2)
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) a b))
   (forall  {a : Type u_1} {b : Type u_2} {? : Type u_3} {V_0 : DomainTheory.
   NeighbourhoodSystems.NeighborhoodSystem a}
      {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
      {V_2 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem ?}
      {h_0 : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.
```

```
  NeighborhoodSystem.mem a V_0 X  ->  @Set.Nonempty a X}
    {h_1 : forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.mem b V_1 Y  ->  @Set.Nonempty b Y}
    (a : @DomainTheory.NeighbourhoodSystems.ApproximableMap a ? V_0 V_2)
    (b : @DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2),
    @Eq (@DomainTheory.NeighbourhoodSystems.ApproximableMap a ? V_0 V_2)
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp a (Option (Sum a b))
  ? V_0
        (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1) V_2
        (@DomainTheory.NeighbourhoodSystems.copair a b ? V_0 V_1 V_2 h_0 h_1 a b)
        (@DomainTheory.NeighbourhoodSystems.inMap_0 a b V_0 V_1 h_0 h_1))
      a))
(forall  {a : Type u_1} {b : Type u_2} {? : Type u_3} {V_0 : DomainTheory.
  NeighbourhoodSystems.NeighborhoodSystem a}
  {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
  {V_2 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem ?}
  {h_0 : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  mem a V_0 X  ->  @Set.Nonempty a X}
  {h_1 : forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  mem b V_1 Y  ->  @Set.Nonempty b Y}
  (a : @DomainTheory.NeighbourhoodSystems.ApproximableMap a ? V_0 V_2)
  (b : @DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2),
  @Eq (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2)


    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp b (Option (Sum a b)) ?
  V_1
      (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1) V_2
      (@DomainTheory.NeighbourhoodSystems.copair a b ? V_0 V_1 V_2 h_0 h_1 a b)
      (@DomainTheory.NeighbourhoodSystems.inMap_1 a b V_0 V_1 h_0 h_1))
    b) :=
@And.intro
(And
  (forall  {a : Type u_1} {b : Type u_2} {? : Type u_3} {V_0 : DomainTheory.
  NeighbourhoodSystems.NeighborhoodSystem a}
    {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
    {V_2 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem ?}
    {h_0 : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.mem a V_0 X  ->  @Set.Nonempty a X}
    {h_1 : forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.mem b V_1 Y  ->  @Set.Nonempty b Y}
    (a : @DomainTheory.NeighbourhoodSystems.ApproximableMap a ? V_0 V_2)
    (b : @DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2),
    @Eq
      (Prod (@DomainTheory.NeighbourhoodSystems.ApproximableMap a ? V_0 V_2)
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2))
      (@DomainTheory.NeighbourhoodSystems.copairProj a b ? V_0 V_1 V_2 h_0 h_1
        (@DomainTheory.NeighbourhoodSystems.copair a b ? V_0 V_1 V_2 h_0 h_1 a b))
      (@Prod.mk (@DomainTheory.NeighbourhoodSystems.ApproximableMap a ? V_0 V_2)
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) a b))
  (forall  {a : Type u_1} {b : Type u_2} {? : Type u_3} {V_0 : DomainTheory.
  NeighbourhoodSystems.NeighborhoodSystem a}
    {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
    {V_2 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem ?}
    {h_0 : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.mem a V_0 X  ->  @Set.Nonempty a X}
    {h_1 : forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.
```

```
  NeighborhoodSystem.mem b V_1 Y  ->  @Set.Nonempty b Y}
    (a : @DomainTheory.NeighbourhoodSystems.ApproximableMap a ? V_0 V_2)
    (b : @DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2),
    @Eq (@DomainTheory.NeighbourhoodSystems.ApproximableMap a ? V_0 V_2)
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp a (Option (Sum a b))
  ? V_0
        (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1) V_2
        (@DomainTheory.NeighbourhoodSystems.copair a b ? V_0 V_1 V_2 h_0 h_1 a b)
        (@DomainTheory.NeighbourhoodSystems.inMap_0 a b V_0 V_1 h_0 h_1))
      a))
(forall  {a : Type u_1} {b : Type u_2} {? : Type u_3} {V_0 : DomainTheory.
  NeighbourhoodSystems.NeighborhoodSystem a}
  {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
  {V_2 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem ?}
  {h_0 : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  mem a V_0 X  ->  @Set.Nonempty a X}
  {h_1 : forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  mem b V_1 Y  ->  @Set.Nonempty b Y}
  (a : @DomainTheory.NeighbourhoodSystems.ApproximableMap a ? V_0 V_2)
  (b : @DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2),
  @Eq (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2)
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp b (Option (Sum a b)) ?
  V_1
      (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1) V_2
      (@DomainTheory.NeighbourhoodSystems.copair a b ? V_0 V_1 V_2 h_0 h_1 a b)
      (@DomainTheory.NeighbourhoodSystems.inMap_1 a b V_0 V_1 h_0 h_1))
    b)
(@And.intro
  (forall  {a : Type u_1} {b : Type u_2} {? : Type u_3} {V_0 : DomainTheory.
  NeighbourhoodSystems.NeighborhoodSystem a}
    {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
    {V_2 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem ?}
    {h_0 : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.mem a V_0 X  ->  @Set.Nonempty a X}
    {h_1 : forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.mem b V_1 Y  ->  @Set.Nonempty b Y}
    (a : @DomainTheory.NeighbourhoodSystems.ApproximableMap a ? V_0 V_2)
    (b : @DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2),
    @Eq
      (Prod (@DomainTheory.NeighbourhoodSystems.ApproximableMap a ? V_0 V_2)
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2))
      (@DomainTheory.NeighbourhoodSystems.copairProj a b ? V_0 V_1 V_2 h_0 h_1
        (@DomainTheory.NeighbourhoodSystems.copair a b ? V_0 V_1 V_2 h_0 h_1 a b))
      (@Prod.mk (@DomainTheory.NeighbourhoodSystems.ApproximableMap a ? V_0 V_2)
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) a b))
  (forall  {a : Type u_1} {b : Type u_2} {? : Type u_3} {V_0 : DomainTheory.
  NeighbourhoodSystems.NeighborhoodSystem a}
    {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
    {V_2 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem ?}
    {h_0 : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.mem a V_0 X  ->  @Set.Nonempty a X}
    {h_1 : forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.mem b V_1 Y  ->  @Set.Nonempty b Y}
    (a : @DomainTheory.NeighbourhoodSystems.ApproximableMap a ? V_0 V_2)
    (b : @DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2),
    @Eq (@DomainTheory.NeighbourhoodSystems.ApproximableMap a ? V_0 V_2)
```

```
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp a (Option (Sum a b))
    ? V_0
          (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1) V_2
          (@DomainTheory.NeighbourhoodSystems.copair a b ? V_0 V_1 V_2 h_0 h_1 a b)
          (@DomainTheory.NeighbourhoodSystems.inMap_0 a b V_0 V_1 h_0 h_1))
        a)
    @DomainTheory.NeighbourhoodSystems.copairProj_copair @DomainTheory.
    NeighbourhoodSystems.copair_comp_inMap_0)
  @DomainTheory.NeighbourhoodSystems.copair_comp_inMap_1
end

end
end DomainTheory.NeighbourhoodSystems
end
```

3.4.26 Exercise 3.25 — opens form a domain

(For topologists.) Recall from Exercises 1.21 and 2.13 on how to regard a domain $|D|$ as a topological space. Using 3.10 show that the family of open subsets of $|D|$ is isomorphic to a domain.

Exercise_3_25.lean

```
module
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_20
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_12
@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems.Exercise325
section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem Set

/-- The **Sierpinski domain** `?`: the system over a one-token type whose
    neighbourhoods are `Delta`
(`univ`) and `{}`. Its elements are `False` (only `Delta`) and ` top ` (also `{}`), i.e.
     the two-point domain. -/
def sierpinski : NeighborhoodSystem Unit where
  mem N := N = Set.univ  \/  N = {}
  master := Set.univ
  master_nonempty := <(), Set.mem_univ _>
  master_mem := Or.inl rfl
  inter_mem := by
    intro X Y Z hX hY hZ _
    rcases hX with rfl | rfl
    * rcases hY with rfl | rfl
      * left; rw [Set.univ_inter]
      * right; rw [Set.univ_inter]
    * right; rw [Set.empty_inter]
  sub_master := fun _ => Set.subset_univ _

section
variable {a : Type*} (V : NeighborhoodSystem a)
```

```
/-- `mapOfOpen U : ?  ->  ?`. It always relates `X` to the blunt neighbourhood `univ`
    of `?`, and
relates `X` to the sharp neighbourhood `{}` exactly when the basic open `[X]` lies
    inside `U`. As an
elementwise map it sends `x` to ` top ` iff `x  in  U`. -/
def mapOfOpen (U : Set V.Element) : ApproximableMap V sierpinski where
  rel X Y := V.mem X  /\  (Y = Set.univ  \/  (Y = {}  /\  V.basicOpen X  subseteq  U))
  rel_dom h := h.1
  rel_cod h := by
    rcases h.2 with rfl | <rfl, _>
    * exact Or.inl rfl
    * exact Or.inr rfl
  master_rel := <V.master_mem, Or.inl rfl>
  inter_right := by
    rintro X Y Y' <hX, hY> <_, hY'>
    refine <hX, ?_>
    rcases hY with rfl | <rfl, hsub>
    * rcases hY' with rfl | <rfl, hsub'>
      * left; rw [Set.univ_inter]
      * right; rw [Set.univ_inter]; exact <rfl, hsub'>
    * rcases hY' with rfl | <rfl, _>
      * right; rw [Set.inter_univ]; exact <rfl, hsub>
      * right; rw [Set.inter_self]; exact <rfl, hsub>
  mono := by
    rintro X X' Y Y' <hX, hY> hX'X hYY' hX' hY'
    refine <hX', ?_>
    rcases hY' with rfl | rfl
    * left; rfl
    * right
      refine <rfl, ?_>
      rcases hY with rfl | <rfl, hsub>
      * exact absurd (hYY' (Set.mem_univ ())) (by simp)
      * have hbo : V.basicOpen X'  subseteq  V.basicOpen X := fun z hz => z.up_mem hz
    hX hX'X
        exact subset_trans hbo hsub

@[simp] theorem mapOfOpen_rel (U : Set V.Element) (X : Set a) (Y : Set Unit) :
    (mapOfOpen V U).rel X Y  <->  V.mem X  /\  (Y = Set.univ  \/  (Y = {}  /\  V.
    basicOpen X  subseteq  U)) := Iff.rfl

/-- The decisive special case: `X (mapOfOpen U) {}  <->  [X]  subseteq  U` (for a
    neighbourhood `X`). -/
theorem mapOfOpen_rel_empty (U : Set V.Element) (X : Set a) :
    (mapOfOpen V U).rel X {}  <->  V.mem X  /\  V.basicOpen X  subseteq  U := by
  rw [mapOfOpen_rel]
  constructor
  * rintro <hX, h>
    refine <hX, ?_>
    rcases h with h | <_, h>
    * exact absurd h Set.empty_ne_univ
    * exact h
  * rintro <hX, h>
    exact <hX, Or.inr <rfl, h>>

/-- `openOfMap f = {x ? f(x) =  top } = ? {[X] ? X f {}}`: the points sent by `f` to
    the top of `?`. -/
```

```
def openOfMap (f : ApproximableMap V sierpinski) : Set V.Element :=
  {x | exists  X, x.mem X  /\  f.rel X {}}

/-- `openOfMap f` is open: it is a union of basic opens `[X]`. -/
theorem openOfMap_isOpen (f : ApproximableMap V sierpinski) : IsOpen (openOfMap V f)
    := by
  constructor
  intro x hx
  obtain <X, hxX, hf> := hx
  exact <X, hxX, fun z hz => <X, hz, hf>>

end

section
set_option linter.unusedVariables false

theorem example_325.{u_1} : forall  {a : Type u_1} (V : DomainTheory.
    NeighbourhoodSystems.NeighborhoodSystem a)
  (f :
    @DomainTheory.NeighbourhoodSystems.ApproximableMap a Unit V
      DomainTheory.NeighbourhoodSystems.Exercise325.sierpinski),
  @IsOpen (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.instTopologicalSpaceElement
     a V)
    (@DomainTheory.NeighbourhoodSystems.Exercise325.openOfMap a V f) :=
  @DomainTheory.NeighbourhoodSystems.Exercise325.openOfMap_isOpen
end

section
variable {a : Type*} (V : NeighborhoodSystem a)

/-- **Exercise 3.25 (core).** The open subsets of `|?|` (ordered by ` subseteq `) are
    order-isomorphic to the
approximable maps `?  ->  ?`. -/
def openIso (V : NeighborhoodSystem a) :
    {U : Set V.Element // IsOpen U} ?o ApproximableMap V sierpinski where
  toFun U := mapOfOpen V U.1
  invFun f := <openOfMap V f, openOfMap_isOpen V f>
  left_inv U := by
    apply Subtype.ext
    ext x
    constructor
    * rintro <X, hxX, hrel>
      rw [mapOfOpen_rel_empty] at hrel
      exact hrel.2 hxX
    * intro hx
      obtain <X, hxX, hsub> := U.2.out x hx
      exact <X, hxX, (mapOfOpen_rel_empty V U.1 X).mpr <x.sub hxX, hsub>>
  right_inv f := by
    apply ApproximableMap.ext
    intro X Y
    rw [mapOfOpen_rel]
    constructor
    * rintro <hX, hd>
      rcases hd with rfl | <rfl, hsub>
      * exact f.mono f.master_rel (V.sub_master hX) (Set.subset_univ _) hX (Or.inl rfl
```

```
    )
      * have hpr : V.principal hX  in  V.basicOpen X := <hX, subset_rfl>
        obtain <X', hX', hf> := hsub hpr
        exact f.mono hf hX'.2 subset_rfl hX (Or.inr rfl)
    * intro hf
      refine <f.rel_dom hf, ?_>
      rcases f.rel_cod hf with rfl | rfl
      * left; rfl
      * exact Or.inr <rfl, fun z hz => <X, hz, hf>>
  map_rel_iff' := by
    intro a b
    show mapOfOpen V a.1  <=  mapOfOpen V b.1  <->  a  <=  b
    rw [ApproximableMap.le_iff]
    constructor
    * intro h x hx
      obtain <X, hxX, hsub> := a.2.out x hx
      have hrelB := h X {} ((mapOfOpen_rel_empty V a.1 X).mpr <x.sub hxX, hsub>)
      rw [mapOfOpen_rel_empty] at hrelB
      exact hrelB.2 hxX
    * intro h X Y hrel
      rw [mapOfOpen_rel] at hrel  |-
      obtain <hX, hd> := hrel
      refine <hX, ?_>
      rcases hd with hu | <he, hsub>
      * exact Or.inl hu
      * exact Or.inr <he, fun z hz => h (hsub hz)>

/- concordance: source L1583 EXERCISE 3.25 opens form a domain -/
/-- **Exercise 3.25.** The family of open subsets of `|?|`, ordered by inclusion, is
order-isomorphic to the domain `|?  ->  ?|` of the Sierpinski function space -- hence *
    is* (isomorphic
to) a domain. This uses Theorem 3.10 (`funSpaceEquiv`) exactly as Scott directs. -/
def opensReprIso (V : NeighborhoodSystem a) :
    {U : Set V.Element // IsOpen U} ?o (funSpace V sierpinski).Element :=
  (openIso V).trans (funSpaceEquiv V sierpinski).symm
end
end

end DomainTheory.NeighbourhoodSystems.Exercise325
end
```

3.4.27 Exercise 3.26 — conditional combinator

Show that for every domain D there is an approximable mapping

$$\mathrm{cond} : T \times D \times D \to D,$$

called the *conditional operator*, satisfying

(i) $\mathrm{cond}(\mathrm{true}, x, y) = x$

(ii) $\mathrm{cond}(\mathrm{false}, x, y) = y$

(iii) $\mathrm{cond}(\bot, x, y) = \bot$.

(Hint: Recalling that $T = \{\{0\}, \{1\}, \{0,1\}\}$, define cond as a relation by

$$\begin{aligned} 0C \cup 10X \cup 110Y \text{ cond } Z \quad \text{iff} \quad & 0 \in C \text{ and } X \subseteq Z \text{ or} \\ & 1 \in C \text{ and } Y \subseteq Z \text{ or} \\ & 0, 1 \in C \text{ and } \Delta \subseteq Z, \end{aligned}$$

where $C \in T$ and $X \in D$ and $Y \in D$ and where we are using the construction of Exercise 3.14.) Find a similar operator in the domain

$$T \times D_0 \times D_1 \to D_0 + D_1.$$

Show also there is an approximable mapping

$$\text{which} : D_0 + D_1 \to T$$

such that for all $x \in |D_0 + D_1|$

$$\text{cond}(\text{which}(x), \text{in}_0(\text{out}_0(x)), \text{in}_1(\text{out}_1(x))) = x.$$

Exercise_3_26.lean

```
module
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Exercise_3_18
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Exercise_3_19
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Definition_3_3
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_4
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Example_2_3
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.Exercise326
section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap

section
variable {a : Type*}

/-- The truth domain `T` of Example 1.2 (neighbourhoods `{Delta, {0}, {1}}`). -/
abbrev TD : NeighborhoodSystem Example12.Token := Example12.neighborhoodSystem

/-- A `T`-neighbourhood contained in `{1}` is `{1}`. -/
theorem Tmem_eq_one {C : Set Example12.Token} (hC : Example12.mem C) (h : C  subseteq
     Example12.one) :
    C = Example12.one := by
  rcases (Example12.mem_iff C).mp hC with rfl | rfl | rfl
  * exact absurd (h (Set.mem_univ 0)) (by simp [Example12.one])
  * exact absurd (h (by simp [Example12.zero] : (0 : Example12.Token)  in  Example12.
    zero))
      (by simp [Example12.one])
  * rfl

/-- A `T`-neighbourhood contained in `{0}` is `{0}` (the other two, `Delta` and `{1}`,
    are not). -/
```

```
theorem Tmem_eq_zero {C : Set Example12.Token} (hC : Example12.mem C) (h : C  subseteq
      Example12.zero) :
    C = Example12.zero := by
  rcases (Example12.mem_iff C).mp hC with rfl | rfl | rfl
  * exact absurd (h (Set.mem_univ 1)) (by simp [Example12.zero])
  * rfl
  * exact absurd (h (by simp [Example12.one] : (1 : Example12.Token)  in  Example12.
    one))
      (by simp [Example12.zero])

/-- Scott's guard for `cond`, phrased with explicit equalities on the truth component
`C = inl^-1 W`. With `X = inl^-1 inr^-1 W` and `Y = inr^-1 inr^-1 W`:

* `C = {0}` (`true`):  `X  subseteq  Z`;
* `C = {1}` (`false`): `Y  subseteq  Z`;
* `C = Delta` (`False`):       `Delta_D  subseteq  Z`. -/
def condGuard (V : NeighborhoodSystem a) (W : Set (Example12.Token ? (a ? a))) (Z :
    Set a) : Prop :=
  (Sum.inl ^-1' W = Example12.zero  /\  Sum.inl ^-1' (Sum.inr ^-1' W)  subseteq  Z)
    \/
  (Sum.inl ^-1' W = Example12.one  /\  Sum.inr ^-1' (Sum.inr ^-1' W)  subseteq  Z)  \/
  (Sum.inl ^-1' W = Example12.master  /\  V.master  subseteq  Z)
end

section
variable {a : Type*}
variable (V : NeighborhoodSystem a)

/-- The three components `C, X, Y` of an input neighbourhood are themselves
    neighbourhoods. -/
theorem cond_components {W : Set (Example12.Token ? (a ? a))}
    (hW : (prod TD (prod V V)).mem W) :
    Example12.mem (Sum.inl ^-1' W)  /\
      V.mem (Sum.inl ^-1' (Sum.inr ^-1' W))  /\  V.mem (Sum.inr ^-1' (Sum.inr ^-1' W))
     := by
  obtain <C, P, hC, hP, rfl> := hW
  obtain <X, Y, hX, hY, rfl> := hP
  simp only [inl_preimage_prodNbhd, inr_preimage_prodNbhd]
  exact <hC, hX, hY>
end

section
variable {a : Type*}

theorem one_ne_master : (Example12.one : Set Example12.Token)  !=  Example12.master :=
     by
  intro h
  have h0 := Set.ext_iff.mp h 0
  simp [Example12.one, Example12.master] at h0

theorem zero_ne_master : (Example12.zero : Set Example12.Token)  !=  Example12.master
    := by
  intro h
  have h1 := Set.ext_iff.mp h 1
  simp [Example12.zero, Example12.master] at h1
end
```

```
section
variable {a : Type*}
variable (V : NeighborhoodSystem a)

theorem condGuard_master {W : Set (Example12.Token ? (a ? a))} {Z : Set a} (hg :
    condGuard V W Z)
    (hC : Sum.inl ^-1' W = Example12.master) : V.master  subseteq  Z := by
  rcases hg with <hC', _> | <hC', _> | <_, h>
  * exact absurd (hC.symm.trans hC') zero_ne_master.symm
  * exact absurd (hC.symm.trans hC') one_ne_master.symm
  * exact h
end

section
variable {a : Type*}

theorem zero_ne_one : (Example12.zero : Set Example12.Token)  !=  Example12.one := by
  intro h
  have h0 := Set.ext_iff.mp h 0
  simp [Example12.zero, Example12.one] at h0
end

section
variable {a : Type*}
variable (V : NeighborhoodSystem a)

theorem condGuard_one {W : Set (Example12.Token ? (a ? a))} {Z : Set a} (hg :
    condGuard V W Z)
    (hC : Sum.inl ^-1' W = Example12.one) : Sum.inr ^-1' (Sum.inr ^-1' W)  subseteq  Z
     := by
  rcases hg with <hC', _> | <_, h> | <hC', _>
  * exact absurd (hC.symm.trans hC') zero_ne_one.symm
  * exact h
  * exact absurd (hC.symm.trans hC') one_ne_master

theorem condGuard_zero {W : Set (Example12.Token ? (a ? a))} {Z : Set a} (hg :
    condGuard V W Z)
    (hC : Sum.inl ^-1' W = Example12.zero) : Sum.inl ^-1' (Sum.inr ^-1' W)  subseteq
    Z := by
  rcases hg with <_, h> | <hC', _> | <hC', _>
  * exact h
  * exact absurd (hC.symm.trans hC') zero_ne_one
  * exact absurd (hC.symm.trans hC') zero_ne_master

/-- **Exercise 3.26 (Scott 1981, PRG-19).** The conditional operator `cond : T  x  D  x
     D  ->  D`. -/
def cond (V : NeighborhoodSystem a) : ApproximableMap (prod TD (prod V V)) V where
  rel W Z := (prod TD (prod V V)).mem W  /\  V.mem Z  /\  condGuard V W Z
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := by
    refine <(prod TD (prod V V)).master_mem, V.master_mem, Or.inr (Or.inr <?_,
    subset_rfl>)>
    show Sum.inl ^-1' (prod TD (prod V V)).master = Example12.master
    rw [prod_master, inl_preimage_prodNbhd]; rfl
```

```
  inter_right := by
    rintro W Z Z' <hW, hZ, hg> <_, hZ', hg'>
    obtain <_, hX, hY> := cond_components V hW
    refine <hW, ?_, ?_>
    * rcases hg with <hCz, hXZ> | <hCo, hYZ> | <hCm, hmZ>
      * exact V.inter_mem hZ hZ' hX (Set.subset_inter hXZ (condGuard_zero V hg' hCz))
      * exact V.inter_mem hZ hZ' hY (Set.subset_inter hYZ (condGuard_one V hg' hCo))
      * exact V.inter_mem hZ hZ' V.master_mem (Set.subset_inter hmZ (condGuard_master
    V hg' hCm))
    * rcases hg with <hCz, hXZ> | <hCo, hYZ> | <hCm, hmZ>
      * exact Or.inl <hCz, Set.subset_inter hXZ (condGuard_zero V hg' hCz)>
      * exact Or.inr (Or.inl <hCo, Set.subset_inter hYZ (condGuard_one V hg' hCo)>)
      * exact Or.inr (Or.inr <hCm, Set.subset_inter hmZ (condGuard_master V hg' hCm)>)
  mono := by
    rintro W W_2 Z Z_2 <hW, hZ, hg> hW_2W hZZ_2 hW_2 hZ_2
    obtain <hC_2, hX_2, hY_2> := cond_components V hW_2
    refine <hW_2, hZ_2, ?_>
    have hCsub : Sum.inl ^-1' W_2  subseteq  Sum.inl ^-1' W := Set.preimage_mono hW_2W
    have hXsub : Sum.inl ^-1' (Sum.inr ^-1' W_2)  subseteq  Sum.inl ^-1' (Sum.inr ^-1'
     W) :=
      Set.preimage_mono (Set.preimage_mono hW_2W)
    have hYsub : Sum.inr ^-1' (Sum.inr ^-1' W_2)  subseteq  Sum.inr ^-1' (Sum.inr ^-1'
     W) :=
      Set.preimage_mono (Set.preimage_mono hW_2W)
    rcases hg with <hCz, hXZ> | <hCo, hYZ> | <hCm, hmZ>
    * exact Or.inl <Tmem_eq_zero hC_2 (hCsub.trans hCz.subset), (hXsub.trans hXZ).
    trans hZZ_2>
    * exact Or.inr (Or.inl <Tmem_eq_one hC_2 (hCsub.trans hCo.subset),
        (hYsub.trans hYZ).trans hZZ_2>)
    * have hmZ_2 : V.master  subseteq  Z_2 := hmZ.trans hZZ_2
      rcases (Example12.mem_iff (Sum.inl ^-1' W_2)).mp hC_2 with hC_2m | hC_2z | hC_2o
      * exact Or.inr (Or.inr <hC_2m, hmZ_2>)
      * exact Or.inl <hC_2z, (V.sub_master hX_2).trans hmZ_2>
      * exact Or.inr (Or.inl <hC_2o, (V.sub_master hY_2).trans hmZ_2>)

@[simp] theorem cond_rel {W : Set (Example12.Token ? (a ? a))} {Z : Set a} :
    (cond V).rel W Z  <->  (prod TD (prod V V)).mem W  /\  V.mem Z  /\  condGuard V W
    Z := Iff.rfl

/-- **Exercise 3.26(iii) (Scott 1981, PRG-19).** `cond(False, x, y) = False`. -/
theorem cond_bot (x y : V.Element) :
    (cond V).toElementMap (pair Example23.botElt (pair x y)) = V.bot := by
  apply Element.ext
  intro Z
  constructor
  * rintro <W, hWmem, _, hZ, hg>
    obtain <C, P, hCbot, hPmem, rfl> := hWmem
    obtain <X, Y, hXx, hYy, rfl> := hPmem
    simp only [condGuard, inl_preimage_prodNbhd, inr_preimage_prodNbhd] at hg
    have hCm : C = Example12.master := hCbot
    rcases hg with <hCz, _> | <hCo, _> | <_, hmZ>
    * exact absurd (hCm.symm.trans hCz) zero_ne_master.symm
    * exact absurd (hCm.symm.trans hCo) one_ne_master.symm
    * rw [mem_bot, Set.Subset.antisymm (V.sub_master hZ) hmZ]
  * intro hbZ
    rw [mem_bot] at hbZ
```

```
    subst hbZ
    refine <prodNbhd Example12.master (prodNbhd V.master V.master),
      <Example12.master, prodNbhd V.master V.master, rfl,
        <V.master, V.master, x.master_mem, y.master_mem, rfl>, rfl>,
      prod_mem_prodNbhd Example12.mem_master (prod_mem_prodNbhd V.master_mem V.
    master_mem),
      V.master_mem, ?_>
    exact Or.inr (Or.inr <inl_preimage_prodNbhd _ _, subset_rfl>)

/-- **Exercise 3.26(ii) (Scott 1981, PRG-19).** `cond(false, x, y) = y`. -/
theorem cond_false (x y : V.Element) :
    (cond V).toElementMap (pair Example23.falseElt (pair x y)) = y := by
  apply Element.ext
  intro Z
  constructor
  * rintro <W, hWmem, _, hZ, hg>
    obtain <C, P, hCfalse, hPmem, rfl> := hWmem
    obtain <X, Y, hXx, hYy, rfl> := hPmem
    simp only [condGuard, inl_preimage_prodNbhd, inr_preimage_prodNbhd] at hg
    rcases hg with <hCz, _> | <_, hYZ> | <_, hmZ>
    * rcases hCfalse with hCm | hCo
      * exact absurd (hCm.symm.trans hCz) zero_ne_master.symm
      * exact absurd (hCo.symm.trans hCz) zero_ne_one.symm
    * exact y.up_mem hYy hZ hYZ
    * rw [Set.Subset.antisymm (V.sub_master hZ) hmZ]; exact y.master_mem
  * intro hyZ
    have hZ : V.mem Z := y.sub hyZ
    refine <prodNbhd Example12.one (prodNbhd V.master Z), <Example12.one, prodNbhd V.
    master Z,
      Or.inr rfl, <V.master, Z, x.master_mem, hyZ, rfl>, rfl>,
      prod_mem_prodNbhd Example12.mem_one (prod_mem_prodNbhd V.master_mem hZ), hZ, ?_>
    refine Or.inr (Or.inl <inl_preimage_prodNbhd _ _, ?_>)
    rw [inr_preimage_prodNbhd, inr_preimage_prodNbhd]

/-- The elementwise action of `cond`, computed at a paired argument `<t, <x, y>>`: a
    neighbourhood
`Z` lies in `cond(t, x, y)` iff `t` selects `true` (`{0}  in  t`) and `Z  in  x`, or `t`
     selects `false`
(`{1}  in  t`) and `Z  in  y`, or `Z = Delta_D` (the always-present master). The three
    defining identities are
immediate corollaries. -/
theorem cond_toElementMap_mem (t : TD.Element) (x y : V.Element) {Z : Set a} :
    ((cond V).toElementMap (pair t (pair x y))).mem Z  <->
      (t.mem Example12.zero  /\  x.mem Z)  \/  (t.mem Example12.one  /\  y.mem Z)  \/
     Z = V.master := by
  constructor
  * rintro <W, hWmem, _, hZ, hg>
    obtain <C, P, hCt, hPmem, rfl> := hWmem
    obtain <X, Y, hXx, hYy, rfl> := hPmem
    simp only [condGuard, inl_preimage_prodNbhd, inr_preimage_prodNbhd] at hg
    rcases hg with <hCz, hXZ> | <hCo, hYZ> | <_, hmZ>
    * exact Or.inl <hCz  |>  hCt, x.up_mem hXx hZ hXZ>
    * exact Or.inr (Or.inl <hCo  |>  hCt, y.up_mem hYy hZ hYZ>)
    * exact Or.inr (Or.inr (Set.Subset.antisymm (V.sub_master hZ) hmZ))
  * intro h
    rcases h with <ht0, hxZ> | <ht1, hyZ> | rfl
```

```
    * have hZ : V.mem Z := x.sub hxZ
      refine <prodNbhd Example12.zero (prodNbhd Z V.master), <Example12.zero, prodNbhd
     Z V.master,
        ht0, <Z, V.master, hxZ, y.master_mem, rfl>, rfl>,
        prod_mem_prodNbhd Example12.mem_zero (prod_mem_prodNbhd hZ V.master_mem), hZ,
    ?_>
      refine Or.inl <inl_preimage_prodNbhd _ _, ?_>
      rw [inr_preimage_prodNbhd, inl_preimage_prodNbhd]
    * have hZ : V.mem Z := y.sub hyZ
      refine <prodNbhd Example12.one (prodNbhd V.master Z), <Example12.one, prodNbhd V
    .master Z,
        ht1, <V.master, Z, x.master_mem, hyZ, rfl>, rfl>,
        prod_mem_prodNbhd Example12.mem_one (prod_mem_prodNbhd V.master_mem hZ), hZ, ?
    _>
      refine Or.inr (Or.inl <inl_preimage_prodNbhd _ _, ?_>)
      rw [inr_preimage_prodNbhd, inr_preimage_prodNbhd]
    * refine <prodNbhd Example12.master (prodNbhd V.master V.master),
        <Example12.master, prodNbhd V.master V.master, t.master_mem,
          <V.master, V.master, x.master_mem, y.master_mem, rfl>, rfl>,
        prod_mem_prodNbhd Example12.mem_master (prod_mem_prodNbhd V.master_mem V.
    master_mem),
        V.master_mem, ?_>
      exact Or.inr (Or.inr <inl_preimage_prodNbhd _ _, subset_rfl>)

/-- **Exercise 3.26(i) (Scott 1981, PRG-19).** `cond(true, x, y) = x`. -/
theorem cond_true (x y : V.Element) :
    (cond V).toElementMap (pair Example23.trueElt (pair x y)) = x := by
  apply Element.ext
  intro Z
  constructor
  * rintro <W, hWmem, _, hZ, hg>
    obtain <C, P, hCtrue, hPmem, rfl> := hWmem
    obtain <X, Y, hXx, hYy, rfl> := hPmem
    simp only [condGuard, inl_preimage_prodNbhd, inr_preimage_prodNbhd] at hg
    rcases hg with <_, hXZ> | <hCo, _> | <_, hmZ>
    * exact x.up_mem hXx hZ hXZ
    * rcases hCtrue with hCm | hCz
      * exact absurd (hCm.symm.trans hCo) one_ne_master.symm
      * exact absurd (hCz.symm.trans hCo) zero_ne_one
    * rw [Set.Subset.antisymm (V.sub_master hZ) hmZ]; exact x.master_mem
  * intro hxZ
    have hZ : V.mem Z := x.sub hxZ
    refine <prodNbhd Example12.zero (prodNbhd Z V.master), <Example12.zero, prodNbhd Z
     V.master,
      Or.inr rfl, <Z, V.master, hxZ, y.master_mem, rfl>, rfl>,
      prod_mem_prodNbhd Example12.mem_zero (prod_mem_prodNbhd hZ V.master_mem), hZ, ?_
    >
    refine Or.inl <inl_preimage_prodNbhd _ _, ?_>
    rw [inr_preimage_prodNbhd, inl_preimage_prodNbhd]
end

section
set_option linter.unusedVariables false

theorem example_326.{u_1} : And
  (And
```

```
  (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
  a)
    (x y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V),
    @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
        (Sum DomainTheory.NeighbourhoodSystems.Example12.Token (Sum a a)) a
        (@DomainTheory.NeighbourhoodSystems.prod DomainTheory.NeighbourhoodSystems.
  Example12.Token (Sum a a)
          DomainTheory.NeighbourhoodSystems.Exercise326.TD (@DomainTheory.
  NeighbourhoodSystems.prod a a V V))
        V (@DomainTheory.NeighbourhoodSystems.Exercise326.cond a V)
        (@DomainTheory.NeighbourhoodSystems.pair DomainTheory.NeighbourhoodSystems.
  Example12.Token (Sum a a)
          DomainTheory.NeighbourhoodSystems.Exercise326.TD (@DomainTheory.
  NeighbourhoodSystems.prod a a V V)
          DomainTheory.NeighbourhoodSystems.Example23.trueElt (@DomainTheory.
  NeighbourhoodSystems.pair a a V V x y)))
      x)
  (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
  a)
    (x y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V),
    @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
        (Sum DomainTheory.NeighbourhoodSystems.Example12.Token (Sum a a)) a
        (@DomainTheory.NeighbourhoodSystems.prod DomainTheory.NeighbourhoodSystems.
  Example12.Token (Sum a a)
          DomainTheory.NeighbourhoodSystems.Exercise326.TD (@DomainTheory.
  NeighbourhoodSystems.prod a a V V))
        V (@DomainTheory.NeighbourhoodSystems.Exercise326.cond a V)
        (@DomainTheory.NeighbourhoodSystems.pair DomainTheory.NeighbourhoodSystems.
  Example12.Token (Sum a a)
          DomainTheory.NeighbourhoodSystems.Exercise326.TD (@DomainTheory.
  NeighbourhoodSystems.prod a a V V)
          DomainTheory.NeighbourhoodSystems.Example23.botElt (@DomainTheory.
  NeighbourhoodSystems.pair a a V V x y)))
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bot a V)))
(forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a)
  (x y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V),
  @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
      (Sum DomainTheory.NeighbourhoodSystems.Example12.Token (Sum a a)) a
      (@DomainTheory.NeighbourhoodSystems.prod DomainTheory.NeighbourhoodSystems.
  Example12.Token (Sum a a)
        DomainTheory.NeighbourhoodSystems.Exercise326.TD (@DomainTheory.
  NeighbourhoodSystems.prod a a V V))
      V (@DomainTheory.NeighbourhoodSystems.Exercise326.cond a V)
      (@DomainTheory.NeighbourhoodSystems.pair DomainTheory.NeighbourhoodSystems.
  Example12.Token (Sum a a)
        DomainTheory.NeighbourhoodSystems.Exercise326.TD (@DomainTheory.
  NeighbourhoodSystems.prod a a V V)
        DomainTheory.NeighbourhoodSystems.Example23.falseElt (@DomainTheory.
  NeighbourhoodSystems.pair a a V V x y)))
    y) :=
@And.intro
(And
  (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
```

```
 a)
    (x y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V),
    @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
        (Sum DomainTheory.NeighbourhoodSystems.Example12.Token (Sum a a)) a
        (@DomainTheory.NeighbourhoodSystems.prod DomainTheory.NeighbourhoodSystems.
 Example12.Token (Sum a a)
          DomainTheory.NeighbourhoodSystems.Exercise326.TD (@DomainTheory.
 NeighbourhoodSystems.prod a a V V))
        V (@DomainTheory.NeighbourhoodSystems.Exercise326.cond a V)
        (@DomainTheory.NeighbourhoodSystems.pair DomainTheory.NeighbourhoodSystems.
 Example12.Token (Sum a a)
          DomainTheory.NeighbourhoodSystems.Exercise326.TD (@DomainTheory.
 NeighbourhoodSystems.prod a a V V)
          DomainTheory.NeighbourhoodSystems.Example23.trueElt (@DomainTheory.
 NeighbourhoodSystems.pair a a V V x y)))
      x)
  (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
 a)
    (x y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V),
    @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
        (Sum DomainTheory.NeighbourhoodSystems.Example12.Token (Sum a a)) a
        (@DomainTheory.NeighbourhoodSystems.prod DomainTheory.NeighbourhoodSystems.
 Example12.Token (Sum a a)
          DomainTheory.NeighbourhoodSystems.Exercise326.TD (@DomainTheory.
 NeighbourhoodSystems.prod a a V V))
        V (@DomainTheory.NeighbourhoodSystems.Exercise326.cond a V)
        (@DomainTheory.NeighbourhoodSystems.pair DomainTheory.NeighbourhoodSystems.
 Example12.Token (Sum a a)
          DomainTheory.NeighbourhoodSystems.Exercise326.TD (@DomainTheory.
 NeighbourhoodSystems.prod a a V V)
          DomainTheory.NeighbourhoodSystems.Example23.botElt (@DomainTheory.
 NeighbourhoodSystems.pair a a V V x y)))
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bot a V)))
(forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a)
  (x y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V),
  @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
      (Sum DomainTheory.NeighbourhoodSystems.Example12.Token (Sum a a)) a
      (@DomainTheory.NeighbourhoodSystems.prod DomainTheory.NeighbourhoodSystems.
 Example12.Token (Sum a a)
        DomainTheory.NeighbourhoodSystems.Exercise326.TD (@DomainTheory.
 NeighbourhoodSystems.prod a a V V))
      V (@DomainTheory.NeighbourhoodSystems.Exercise326.cond a V)
      (@DomainTheory.NeighbourhoodSystems.pair DomainTheory.NeighbourhoodSystems.
 Example12.Token (Sum a a)
        DomainTheory.NeighbourhoodSystems.Exercise326.TD (@DomainTheory.
 NeighbourhoodSystems.prod a a V V)
        DomainTheory.NeighbourhoodSystems.Example23.falseElt (@DomainTheory.
 NeighbourhoodSystems.pair a a V V x y)))
    y)
(@And.intro
  (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
 a)
    (x y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V),
```

```
      @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
          (Sum DomainTheory.NeighbourhoodSystems.Example12.Token (Sum a a)) a
          (@DomainTheory.NeighbourhoodSystems.prod DomainTheory.NeighbourhoodSystems.
    Example12.Token (Sum a a)
            DomainTheory.NeighbourhoodSystems.Exercise326.TD (@DomainTheory.
    NeighbourhoodSystems.prod a a V V))
         V (@DomainTheory.NeighbourhoodSystems.Exercise326.cond a V)
          (@DomainTheory.NeighbourhoodSystems.pair DomainTheory.NeighbourhoodSystems.
    Example12.Token (Sum a a)
            DomainTheory.NeighbourhoodSystems.Exercise326.TD (@DomainTheory.
    NeighbourhoodSystems.prod a a V V)
            DomainTheory.NeighbourhoodSystems.Example23.trueElt (@DomainTheory.
    NeighbourhoodSystems.pair a a V V x y)))
        x)
     (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
    a)
       (x y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V),
       @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
         (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
           (Sum DomainTheory.NeighbourhoodSystems.Example12.Token (Sum a a)) a
           (@DomainTheory.NeighbourhoodSystems.prod DomainTheory.NeighbourhoodSystems.
    Example12.Token (Sum a a)
            DomainTheory.NeighbourhoodSystems.Exercise326.TD (@DomainTheory.
    NeighbourhoodSystems.prod a a V V))
         V (@DomainTheory.NeighbourhoodSystems.Exercise326.cond a V)
          (@DomainTheory.NeighbourhoodSystems.pair DomainTheory.NeighbourhoodSystems.
    Example12.Token (Sum a a)
            DomainTheory.NeighbourhoodSystems.Exercise326.TD (@DomainTheory.
    NeighbourhoodSystems.prod a a V V)
            DomainTheory.NeighbourhoodSystems.Example23.botElt (@DomainTheory.
    NeighbourhoodSystems.pair a a V V x y)))
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bot a V))
     @DomainTheory.NeighbourhoodSystems.Exercise326.cond_true @DomainTheory.
    NeighbourhoodSystems.Exercise326.cond_bot)
  @DomainTheory.NeighbourhoodSystems.Exercise326.cond_false
end

section
variable {a b : Type*}
variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
variable {h_0 : forall  X, V_0.mem X  ->  X.Nonempty} {h_1 : forall  Y, V_1.mem Y  ->
     Y.Nonempty}

/-- A `T`-neighbourhood containing `Delta` is `Delta`. -/
theorem Tmem_ge_master {C : Set Example12.Token} (hC : Example12.mem C)
    (h : Example12.master  subseteq  C) : C = Example12.master :=
  Set.Subset.antisymm (Example12.neighborhoodSystem.sub_master hC) h

/-- A `T`-neighbourhood containing `{1}` is `{1}` or `Delta`. -/
theorem Tmem_ge_one {C : Set Example12.Token} (hC : Example12.mem C)
    (h : Example12.one  subseteq  C) : C = Example12.one  \/  C = Example12.master :=
    by
  rcases (Example12.mem_iff C).mp hC with rfl | rfl | rfl
  * exact Or.inr rfl
  * exact absurd (h (by simp [Example12.one] : (1 : Example12.Token)  in  Example12.
```

```
    one))
      (by simp [Example12.zero])
  * exact Or.inl rfl

/-- A `T`-neighbourhood containing `{0}` is `{0}` or `Delta`. -/

theorem Tmem_ge_zero {C : Set Example12.Token} (hC : Example12.mem C)
    (h : Example12.zero  subseteq  C) : C = Example12.zero  \/  C = Example12.master
    := by
  rcases (Example12.mem_iff C).mp hC with rfl | rfl | rfl
  * exact Or.inr rfl
  * exact Or.inl rfl
  * exact absurd (h (by simp [Example12.zero] : (0 : Example12.Token)  in  Example12.
    zero))
      (by simp [Example12.one])

/-- The embedding `T  x  D_0  x  D_1  ->  T  x  S  x  S` (`S = D_0 + D_1`) that injects
     the two value slots into
the sum via `in_0` and `in_1`, leaving the truth slot fixed. -/
def condSumEmb (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b)
    (h_0 : forall  X, V_0.mem X  ->  X.Nonempty) (h_1 : forall  Y, V_1.mem Y  ->  Y.
    Nonempty) :
    ApproximableMap (prod TD (prod V_0 V_1)) (prod TD (prod (sum V_0 V_1 h_0 h_1) (sum
     V_0 V_1 h_0 h_1))) :=
  paired (proj_0 TD (prod V_0 V_1))
    (paired ((inMap_0 (h_0 := h_0) (h_1 := h_1)).comp ((proj_0 V_0 V_1).comp (proj_1
    TD (prod V_0 V_1))))
      ((inMap_1 (h_0 := h_0) (h_1 := h_1)).comp ((proj_1 V_0 V_1).comp (proj_1 TD (
    prod V_0 V_1)))))

/-- **Exercise 3.26 (Scott 1981, PRG-19).** The sum-valued conditional `T  x  D_0  x
    D_1  ->  D_0 + D_1`. -/
def condSum (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b)
    (h_0 : forall  X, V_0.mem X  ->  X.Nonempty) (h_1 : forall  Y, V_1.mem Y  ->  Y.
    Nonempty) :
    ApproximableMap (prod TD (prod V_0 V_1)) (sum V_0 V_1 h_0 h_1) :=
  (cond (sum V_0 V_1 h_0 h_1)).comp (condSumEmb V_0 V_1 h_0 h_1)

theorem inter_in_oneset {C C' : Set Example12.Token} (hc : C = Example12.master  \/  C
     = Example12.one)
    (hc' : C' = Example12.master  \/  C' = Example12.one) :
    C  I  C' = Example12.master  \/  C  I  C' = Example12.one := by
  rcases hc with rfl | rfl <;> rcases hc' with rfl | rfl
  * exact Or.inl (Set.inter_self _)
  * exact Or.inr (Set.univ_inter _)
  * exact Or.inr (Set.inter_univ _)
  * exact Or.inr (Set.inter_self _)

theorem inter_in_zeroset {C C' : Set Example12.Token} (hc : C = Example12.master  \/
    C = Example12.zero)
    (hc' : C' = Example12.master  \/  C' = Example12.zero) :
    C  I  C' = Example12.master  \/  C  I  C' = Example12.zero := by
  rcases hc with rfl | rfl <;> rcases hc' with rfl | rfl
  * exact Or.inl (Set.inter_self _)
  * exact Or.inr (Set.univ_inter _)
  * exact Or.inr (Set.inter_univ _)
```

```
  * exact Or.inr (Set.inter_self _)

/-- Scott's guard for `which`: the truth output is `true` (`{0}`/`Delta`) on a left
    copy `0X`, `false`
(`{1}`/`Delta`) on a right copy `1Y`, and `False` (`Delta`) on the basepoint master. -/
def whichGuard (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b)
    (W : Set (Option (a ? b))) (C : Set Example12.Token) : Prop :=
  ((exists  X, V_0.mem X  /\  W = inj_0 X)  /\  (C = Example12.master  \/  C =
    Example12.zero))  \/
  ((exists  Y, V_1.mem Y  /\  W = inj_1 Y)  /\  (C = Example12.master  \/  C =
    Example12.one))  \/
  (W = sumMaster V_0 V_1  /\  C = Example12.master)

theorem whichGuard_left {W : Set (Option (a ? b))} {C : Set Example12.Token} {X : Set
    a}
    (hg : whichGuard V_0 V_1 W C) (hW : W = inj_0 X) (hX : X.Nonempty) :
    C = Example12.master  \/  C = Example12.zero := by
  rcases hg with <_, hc> | <<Y, _, hWY>, _> | <hWsm, _>
  * exact hc
  * refine absurd (hW.symm.trans hWY) ?_
    intro h; exact not_inj_0_subset_inj_1 hX h.subset
  * refine absurd (hW.symm.trans hWsm) ?_
    intro h; exact none_mem_inj_0 (h  |>  none_mem_sumMaster)

theorem whichGuard_masterC {W : Set (Option (a ? b))} {C : Set Example12.Token}
    (hg : whichGuard V_0 V_1 W C) (hW : W = sumMaster V_0 V_1) : C = Example12.master
    := by
  rcases hg with <<X, _, hWX>, _> | <<Y, _, hWY>, _> | <_, hc>
  * refine absurd (hW.symm.trans hWX) ?_
    intro h; exact none_mem_inj_0 (h  |>  none_mem_sumMaster)
  * refine absurd (hW.symm.trans hWY) ?_
    intro h; exact none_mem_inj_1 (h  |>  none_mem_sumMaster)
  * exact hc

theorem whichGuard_right {W : Set (Option (a ? b))} {C : Set Example12.Token} {Y : Set
     b}
    (hg : whichGuard V_0 V_1 W C) (hW : W = inj_1 Y) (hY : Y.Nonempty) :
    C = Example12.master  \/  C = Example12.one := by
  rcases hg with <<X, _, hWX>, _> | <_, hc> | <hWsm, _>
  * refine absurd (hW.symm.trans hWX) ?_
    intro h; exact not_inj_1_subset_inj_0 hY h.subset
  * exact hc
  * refine absurd (hW.symm.trans hWsm) ?_
    intro h; exact none_mem_inj_1 (h  |>  none_mem_sumMaster)

/-- **Exercise 3.26 (Scott 1981, PRG-19).** The discriminator `which : D_0 + D_1  ->  T
    `. -/
def whichMap (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b)
    (h_0 : forall  X, V_0.mem X  ->  X.Nonempty) (h_1 : forall  Y, V_1.mem Y  ->  Y.
    Nonempty) :
    ApproximableMap (sum V_0 V_1 h_0 h_1) TD where
  rel W C := (sum V_0 V_1 h_0 h_1).mem W  /\  Example12.mem C  /\  whichGuard V_0 V_1
    W C
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := <(sum V_0 V_1 h_0 h_1).master_mem, Example12.mem_master, Or.inr (Or.
```

```
    inr <rfl, rfl>)>
  inter_right := by
    rintro W C C' <hW, _, hg> <_, _, hg'>
    rcases hg with <<X, hX, rfl>, hc> | <<Y, hY, rfl>, hc> | <rfl, hc>
    * have hor := inter_in_zeroset hc (whichGuard_left hg' rfl (h_0 X hX))
      refine <Or.inr (Or.inl <X, hX, rfl>), ?_, Or.inl <<X, hX, rfl>, hor>>
      rcases hor with h | h
      * rw [h]; exact Example12.mem_master
      * rw [h]; exact Example12.mem_zero
    * have hor := inter_in_oneset hc (whichGuard_right hg' rfl (h_1 Y hY))
      refine <Or.inr (Or.inr <Y, hY, rfl>), ?_, Or.inr (Or.inl <<Y, hY, rfl>, hor>)>
      rcases hor with h | h
      * rw [h]; exact Example12.mem_master
      * rw [h]; exact Example12.mem_one
    * have hc' := whichGuard_masterC hg' rfl
      subst hc; subst hc'
      rw [Set.inter_self]
      exact <Or.inl rfl, Example12.mem_master, Or.inr (Or.inr <rfl, rfl>)>
  mono := by
    rintro W W_2 C C_2 <hW, hC, hg> hW_2W hCC_2 hW_2 hC_2
    refine <hW_2, hC_2, ?_>
    rcases hg with <<X, hX, rfl>, hc> | <<Y, hY, rfl>, hc> | <rfl, hc>
    * obtain <X_2, hX_2, rfl> := mem_subset_inj_0 hW_2 hW_2W
      refine Or.inl <<X_2, hX_2, rfl>, ?_>
      rcases hc with rfl | rfl
      * exact Or.inl (Tmem_ge_master hC_2 hCC_2)
      * exact Or.symm (Tmem_ge_zero hC_2 hCC_2)
    * obtain <Y_2, hY_2, rfl> := mem_subset_inj_1 hW_2 hW_2W
      refine Or.inr (Or.inl <<Y_2, hY_2, rfl>, ?_>)
      rcases hc with rfl | rfl
      * exact Or.inl (Tmem_ge_master hC_2 hCC_2)
      * exact Or.symm (Tmem_ge_one hC_2 hCC_2)
    * subst hc
      have hC_2m : C_2 = Example12.master := Tmem_ge_master hC_2 hCC_2
      rcases hW_2 with rfl | <X_2, hX_2, rfl> | <Y_2, hY_2, rfl>
      * exact Or.inr (Or.inr <rfl, hC_2m>)
      * exact Or.inl <<X_2, hX_2, rfl>, Or.inl hC_2m>
      * exact Or.inr (Or.inl <<Y_2, hY_2, rfl>, Or.inl hC_2m>)

theorem condSumEmb_toElementMap (t : TD.Element) (x : V_0.Element) (y : V_1.Element) :
    (condSumEmb V_0 V_1 h_0 h_1).toElementMap (pair t (pair x y))
      = pair t (pair ((inMap_0 (h_0 := h_0) (h_1 := h_1)).toElementMap x)
          ((inMap_1 (h_0 := h_0) (h_1 := h_1)).toElementMap y)) := by
  simp only [condSumEmb, toElementMap_paired, toElementMap_comp, toElementMap_proj_0,
    toElementMap_proj_1, fst_pair, snd_pair]

end
end

section
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap
variable {a b : Type*}
variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
variable {h_0 : forall  X, V_0.mem X  ->  X.Nonempty} {h_1 : forall  Y, V_1.mem Y  ->
     Y.Nonempty}
/- concordance: source L1585 EXERCISE 3.26 conditional combinator -/
```

```
/-- **Exercise 3.26.** `condSum(False, x, y) = False`. -/
theorem condSum_bot (x : V_0.Element) (y : V_1.Element) :
    (condSum V_0 V_1 h_0 h_1).toElementMap (pair Example23.botElt (pair x y)) = (sum
    V_0 V_1 h_0 h_1).bot := by
  rw [condSum, toElementMap_comp, condSumEmb_toElementMap, cond_bot]

/-- **Exercise 3.26.** `condSum(false, x, y) = in_1(y)`. -/
theorem condSum_false (x : V_0.Element) (y : V_1.Element) :
    (condSum V_0 V_1 h_0 h_1).toElementMap (pair Example23.falseElt (pair x y))
      = (inMap_1 (h_0 := h_0) (h_1 := h_1)).toElementMap y := by
  rw [condSum, toElementMap_comp, condSumEmb_toElementMap, cond_false]

/-- **Exercise 3.26.** `condSum(true, x, y) = in_0(x)`. -/
theorem condSum_true (x : V_0.Element) (y : V_1.Element) :
    (condSum V_0 V_1 h_0 h_1).toElementMap (pair Example23.trueElt (pair x y))
      = (inMap_0 (h_0 := h_0) (h_1 := h_1)).toElementMap x := by
  rw [condSum, toElementMap_comp, condSumEmb_toElementMap, cond_true]

/-- `{}` is never a sum-neighbourhood (every neighbourhood is non-empty). -/
theorem not_sum_mem_empty : not  (sum V_0 V_1 h_0 h_1).mem {} := by
  rintro (h | <X, hX, h> | <Y, hY, h>)
  * exact absurd (h.symm  |>  none_mem_sumMaster) (Set.notMem_empty _)
  * obtain <a, ha> := inj_0_nonempty (h_0 X hX)
    exact absurd (h  |>  ha) (Set.notMem_empty _)
  * obtain <b, hb> := inj_1_nonempty (h_1 Y hY)
    exact absurd (h  |>  hb) (Set.notMem_empty _)

/-- On a "left" element (one reaching into the left copy), the round-trip `in_0  comp
    out_0` is the
identity. -/
theorem inMap_0_outMap_0_eq_of_left {x : (sum V_0 V_1 h_0 h_1).Element}
    (hL : exists  X, V_0.mem X  /\  x.mem (inj_0 X)) :
    (inMap_0 (h_0 := h_0) (h_1 := h_1)).toElementMap ((outMap_0 (h_0 := h_0) (h_1 :=
    h_1)).toElementMap x) = x := by
  obtain <X_0, hX_0, hxX_0> := hL
  apply Element.ext
  intro Z
  constructor
  * rintro <X, <W, hxW, _, _, hLWX>, _, hSZ, hinjXZ>
    -- W  I  inj_0 X_0 is a left copy inj_0 X_1  subseteq  Z, and lies in x
    have hxWX_0 : x.mem (W  I  inj_0 X_0) := x.inter_mem hxW hxX_0
    obtain <X_1, _, hX_1eq> := mem_subset_inj_0 (x.sub hxWX_0) Set.inter_subset_right
    have hX_1X : X_1  subseteq  X := by
      have : leftPart V_0 (W  I  inj_0 X_0)  subseteq  X :=
        (leftPart_mono V_0 Set.inter_subset_left).trans hLWX
      rwa [hX_1eq, leftPart_inj_0] at this
    have : (W  I  inj_0 X_0)  subseteq  Z :=
      hX_1eq  |>  (inj_0_subset_inj_0.mpr hX_1X).trans hinjXZ
    exact x.up_mem hxWX_0 hSZ this
  * intro hxZ
    rcases x.sub hxZ with rfl | <X', hX', rfl> | <Y', hY', rfl>
    * -- Z = sumMaster
      exact <V_0.master, <sumMaster V_0 V_1, x.master_mem, (sum V_0 V_1 h_0 h_1).
    master_mem,
        V_0.master_mem, (leftPart_sumMaster V_0 V_1).subset>, V_0.master_mem,
        (sum V_0 V_1 h_0 h_1).master_mem, inj_0_subset_sumMaster V_0.master_mem>
```

```
    * -- Z = inj_0 X'
      exact <X', <inj_0 X', hxZ, Or.inr (Or.inl <X', hX', rfl>), hX', (leftPart_inj_0
    V_0 X').subset>,
        hX', Or.inr (Or.inl <X', hX', rfl>), subset_rfl>
    * -- Z = inj_1 Y' is impossible for a left element
      have := x.inter_mem hxX_0 hxZ
      rw [inj_0_inter_inj_1] at this
      exact absurd (x.sub this) not_sum_mem_empty

/-- On a "right" element, the round-trip `in_1  comp  out_1` is the identity. -/
theorem inMap_1_outMap_1_eq_of_right {x : (sum V_0 V_1 h_0 h_1).Element}
    (hR : exists  Y, V_1.mem Y  /\  x.mem (inj_1 Y)) :
    (inMap_1 (h_0 := h_0) (h_1 := h_1)).toElementMap ((outMap_1 (h_0 := h_0) (h_1 :=
    h_1)).toElementMap x) = x := by
  obtain <Y_0, hY_0, hxY_0> := hR
  apply Element.ext
  intro Z
  constructor
  * rintro <Y, <W, hxW, _, _, hRWY>, _, hSZ, hinjYZ>
    have hxWY_0 : x.mem (W  I  inj_1 Y_0) := x.inter_mem hxW hxY_0
    obtain <Y_1, _, hY_1eq> := mem_subset_inj_1 (x.sub hxWY_0) Set.inter_subset_right
    have hY_1Y : Y_1  subseteq  Y := by
      have : rightPart V_1 (W  I  inj_1 Y_0)  subseteq  Y :=
        (rightPart_mono V_1 Set.inter_subset_left).trans hRWY
      rwa [hY_1eq, rightPart_inj_1] at this
    have : (W  I  inj_1 Y_0)  subseteq  Z :=
      hY_1eq  |>  (inj_1_subset_inj_1.mpr hY_1Y).trans hinjYZ
    exact x.up_mem hxWY_0 hSZ this
  * intro hxZ
    rcases x.sub hxZ with rfl | <X', hX', rfl> | <Y', hY', rfl>
    * exact <V_1.master, <sumMaster V_0 V_1, x.master_mem, (sum V_0 V_1 h_0 h_1).
    master_mem,
        V_1.master_mem, (rightPart_sumMaster V_0 V_1).subset>, V_1.master_mem,
        (sum V_0 V_1 h_0 h_1).master_mem, inj_1_subset_sumMaster V_1.master_mem>
    * have := x.inter_mem hxY_0 hxZ
      rw [Set.inter_comm, inj_0_inter_inj_1] at this
      exact absurd (x.sub this) not_sum_mem_empty
    * exact <Y', <inj_1 Y', hxZ, Or.inr (Or.inr <Y', hY', rfl>), hY', (rightPart_inj_1
     V_1 Y').subset>,
        hY', Or.inr (Or.inr <Y', hY', rfl>), subset_rfl>

/-- Trichotomy for a sum-element: it reaches the left copy, or the right copy, or is `
    False`. -/
theorem sum_element_trichotomy (x : (sum V_0 V_1 h_0 h_1).Element) :
    (exists  X, V_0.mem X  /\  x.mem (inj_0 X))  \/  (exists  Y, V_1.mem Y  /\  x.mem
    (inj_1 Y))  \/
      (forall  W, x.mem W  ->  W = (sum V_0 V_1 h_0 h_1).master) := by
  by_cases hL : exists  X, V_0.mem X  /\  x.mem (inj_0 X)
  * exact Or.inl hL
  by_cases hR : exists  Y, V_1.mem Y  /\  x.mem (inj_1 Y)
  * exact Or.inr (Or.inl hR)
  refine Or.inr (Or.inr ?_)
  intro W hxW
  rcases x.sub hxW with rfl | <X, hX, rfl> | <Y, hY, rfl>
  * rfl
  * exact absurd <X, hX, hxW> hL
```

```
  * exact absurd <Y, hY, hxW> hR

/-- `which(x)` selects `false` (`{1}  in  which x`) exactly when `x` reaches into the
    right copy. -/
theorem which_mem_one {x : (sum V_0 V_1 h_0 h_1).Element} :
    ((whichMap V_0 V_1 h_0 h_1).toElementMap x).mem Example12.one
       <->  exists  Y, V_1.mem Y  /\  x.mem (inj_1 Y) := by
  constructor
  * rintro <W, hWx, _, _, hg>
    rcases hg with <_, hc> | <<Y, hY, rfl>, _> | <_, hc>
    * rcases hc with h | h
      * exact absurd h one_ne_master
      * exact absurd h zero_ne_one.symm
    * exact <Y, hY, hWx>
    * exact absurd hc one_ne_master
  * rintro <Y, hY, hWx>
    exact <inj_1 Y, hWx, Or.inr (Or.inr <Y, hY, rfl>), Example12.mem_one,
      Or.inr (Or.inl <<Y, hY, rfl>, Or.inr rfl>)>

/-- `which(x)` selects `true` (`{0}  in  which x`) exactly when `x` reaches into the
    left copy. -/
theorem which_mem_zero {x : (sum V_0 V_1 h_0 h_1).Element} :
    ((whichMap V_0 V_1 h_0 h_1).toElementMap x).mem Example12.zero
       <->  exists  X, V_0.mem X  /\  x.mem (inj_0 X) := by
  constructor
  * rintro <W, hWx, _, _, hg>
    rcases hg with <<X, hX, rfl>, _> | <_, hc> | <_, hc>
    * exact <X, hX, hWx>
    * rcases hc with h | h
      * exact absurd h zero_ne_master
      * exact absurd h zero_ne_one
    * exact absurd hc zero_ne_master
  * rintro <X, hX, hWx>
    exact <inj_0 X, hWx, Or.inr (Or.inl <X, hX, rfl>), Example12.mem_zero,
      Or.inl <<X, hX, rfl>, Or.inr rfl>>

/-- **Exercise 3.26 (Scott 1981, PRG-19).** Scott's identity for the discriminator:
`cond(which x, in_0(out_0 x), in_1(out_1 x)) = x` for every `x  in  |D_0 + D_1|`. -/
theorem cond_which (x : (sum V_0 V_1 h_0 h_1).Element) :
    (cond (sum V_0 V_1 h_0 h_1)).toElementMap
        (pair ((whichMap V_0 V_1 h_0 h_1).toElementMap x)
          (pair ((inMap_0 (h_0 := h_0) (h_1 := h_1)).toElementMap
                  ((outMap_0 (h_0 := h_0) (h_1 := h_1)).toElementMap x))
                ((inMap_1 (h_0 := h_0) (h_1 := h_1)).toElementMap
                  ((outMap_1 (h_0 := h_0) (h_1 := h_1)).toElementMap x)))) = x := by
  apply Element.ext
  intro Z
  rw [cond_toElementMap_mem, which_mem_zero, which_mem_one]
  constructor
  * rintro (<hL, ha> | <hR, hb> | rfl)
    * rwa [inMap_0_outMap_0_eq_of_left hL] at ha
    * rwa [inMap_1_outMap_1_eq_of_right hR] at hb
    * exact x.master_mem
  * intro hxZ
    rcases sum_element_trichotomy x with hL | hR | hN
    * exact Or.inl <hL, by rw [inMap_0_outMap_0_eq_of_left hL]; exact hxZ>
```

```
    * exact Or.inr (Or.inl <hR, by rw [inMap_1_outMap_1_eq_of_right hR]; exact hxZ>)
    * exact Or.inr (Or.inr (hN Z hxZ))
end

section
set_option linter.unusedVariables false

theorem example_326_sum.{u_1, u_2} : And
  (And
    (And
      (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem a}
        {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
        {h_0 : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.mem a V_0 X  ->  @Set.Nonempty a X}
        {h_1 : forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.mem b V_1 Y  ->  @Set.Nonempty b Y}
        (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V_0)
        (y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element b V_1),
        @Eq
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Option (Sum
    a b))
            (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1))
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
            (Sum DomainTheory.NeighbourhoodSystems.Example12.Token (Sum a b)) (Option
    (Sum a b))
            (@DomainTheory.NeighbourhoodSystems.prod DomainTheory.NeighbourhoodSystems
    .Example12.Token (Sum a b)
              DomainTheory.NeighbourhoodSystems.Exercise326.TD (@DomainTheory.
    NeighbourhoodSystems.prod a b V_0 V_1))
            (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
            (@DomainTheory.NeighbourhoodSystems.Exercise326.condSum a b V_0 V_1 h_0
    h_1)
            (@DomainTheory.NeighbourhoodSystems.pair DomainTheory.NeighbourhoodSystems
    .Example12.Token (Sum a b)
              DomainTheory.NeighbourhoodSystems.Exercise326.TD (@DomainTheory.
    NeighbourhoodSystems.prod a b V_0 V_1)
              DomainTheory.NeighbourhoodSystems.Example23.botElt
              (@DomainTheory.NeighbourhoodSystems.pair a b V_0 V_1 x y)))
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bot (Option (Sum a b)
    )
            (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)))
      (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem a}
        {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
        {h_0 : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.mem a V_0 X  ->  @Set.Nonempty a X}
        {h_1 : forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.mem b V_1 Y  ->  @Set.Nonempty b Y}
        (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V_0)
        (y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element b V_1),
        @Eq
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Option (Sum
    a b))
            (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1))
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
```

```
         (Sum DomainTheory.NeighbourhoodSystems.Example12.Token (Sum a b)) (Option
 (Sum a b))
         (@DomainTheory.NeighbourhoodSystems.prod DomainTheory.NeighbourhoodSystems
 .Example12.Token (Sum a b)
           DomainTheory.NeighbourhoodSystems.Exercise326.TD (@DomainTheory.
 NeighbourhoodSystems.prod a b V_0 V_1))
         (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
         (@DomainTheory.NeighbourhoodSystems.Exercise326.condSum a b V_0 V_1 h_0
 h_1)
         (@DomainTheory.NeighbourhoodSystems.pair DomainTheory.NeighbourhoodSystems
 .Example12.Token (Sum a b)
           DomainTheory.NeighbourhoodSystems.Exercise326.TD (@DomainTheory.
 NeighbourhoodSystems.prod a b V_0 V_1)
           DomainTheory.NeighbourhoodSystems.Example23.falseElt
           (@DomainTheory.NeighbourhoodSystems.pair a b V_0 V_1 x y)))
       (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap b (Option (
 Sum a b)) V_1
         (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
         (@DomainTheory.NeighbourhoodSystems.inMap_1 a b V_0 V_1 h_0 h_1) y)))
 (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem a}
   {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
   {h_0 : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem.mem a V_0 X  ->  @Set.Nonempty a X}
   {h_1 : forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem.mem b V_1 Y  ->  @Set.Nonempty b Y}
   (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V_0)
   (y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element b V_1),
   @Eq
     (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Option (Sum a
 b))
       (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1))
     (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
       (Sum DomainTheory.NeighbourhoodSystems.Example12.Token (Sum a b)) (Option (
 Sum a b))
       (@DomainTheory.NeighbourhoodSystems.prod DomainTheory.NeighbourhoodSystems.
 Example12.Token (Sum a b)
         DomainTheory.NeighbourhoodSystems.Exercise326.TD (@DomainTheory.
 NeighbourhoodSystems.prod a b V_0 V_1))
       (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
       (@DomainTheory.NeighbourhoodSystems.Exercise326.condSum a b V_0 V_1 h_0 h_1)
       (@DomainTheory.NeighbourhoodSystems.pair DomainTheory.NeighbourhoodSystems.
 Example12.Token (Sum a b)
         DomainTheory.NeighbourhoodSystems.Exercise326.TD (@DomainTheory.
 NeighbourhoodSystems.prod a b V_0 V_1)
         DomainTheory.NeighbourhoodSystems.Example23.trueElt
         (@DomainTheory.NeighbourhoodSystems.pair a b V_0 V_1 x y)))
     (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a (Option (
 Sum a b)) V_0
       (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
       (@DomainTheory.NeighbourhoodSystems.inMap_0 a b V_0 V_1 h_0 h_1) x)))
(forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem a}
 {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
 {h_0 : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 mem a V_0 X  ->  @Set.Nonempty a X}
```

```
{h_1 : forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
mem b V_1 Y  ->  @Set.Nonempty b Y}
(x :
  @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Option (Sum a b))
    (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)),
@Eq
  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Option (Sum a b)
)
    (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1))
  (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
    (Sum DomainTheory.NeighbourhoodSystems.Example12.Token (Sum (Option (Sum a b))
 (Option (Sum a b))))
    (Option (Sum a b))
    (@DomainTheory.NeighbourhoodSystems.prod DomainTheory.NeighbourhoodSystems.
Example12.Token
      (Sum (Option (Sum a b)) (Option (Sum a b))) DomainTheory.
NeighbourhoodSystems.Exercise326.TD
      (@DomainTheory.NeighbourhoodSystems.prod (Option (Sum a b)) (Option (Sum a b
))
        (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
        (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)))
    (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
    (@DomainTheory.NeighbourhoodSystems.Exercise326.cond (Option (Sum a b))
      (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1))
    (@DomainTheory.NeighbourhoodSystems.pair DomainTheory.NeighbourhoodSystems.
Example12.Token
      (Sum (Option (Sum a b)) (Option (Sum a b))) DomainTheory.
NeighbourhoodSystems.Exercise326.TD
      (@DomainTheory.NeighbourhoodSystems.prod (Option (Sum a b)) (Option (Sum a b
))
        (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
        (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1))
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap (Option (
Sum a b))
       DomainTheory.NeighbourhoodSystems.Example12.Token (@DomainTheory.
NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
        DomainTheory.NeighbourhoodSystems.Exercise326.TD


        (@DomainTheory.NeighbourhoodSystems.Exercise326.whichMap a b V_0 V_1 h_0
h_1) x)
      (@DomainTheory.NeighbourhoodSystems.pair (Option (Sum a b)) (Option (Sum a b
))
        (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
        (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a (Option
 (Sum a b)) V_0
          (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
          (@DomainTheory.NeighbourhoodSystems.inMap_0 a b V_0 V_1 h_0 h_1)
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap (Option
 (Sum a b)) a
            (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1) V_0
            (@DomainTheory.NeighbourhoodSystems.outMap_0 a b V_0 V_1 h_0 h_1) x))
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap b (Option
 (Sum a b)) V_1
          (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
          (@DomainTheory.NeighbourhoodSystems.inMap_1 a b V_0 V_1 h_0 h_1)
```

```
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap (Option
   (Sum a b)) b
              (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1) V_1
              (@DomainTheory.NeighbourhoodSystems.outMap_1 a b V_0 V_1 h_0 h_1) x)))
 ))
    x) :=
@And.intro
(And
  (And
    (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem a}
      {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
      {h_0 : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem.mem a V_0 X  ->  @Set.Nonempty a X}
      {h_1 : forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem.mem b V_1 Y  ->  @Set.Nonempty b Y}
      (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V_0)
      (y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element b V_1),
      @Eq
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Option (Sum
 a b))
          (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1))
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
          (Sum DomainTheory.NeighbourhoodSystems.Example12.Token (Sum a b)) (Option
 (Sum a b))
          (@DomainTheory.NeighbourhoodSystems.prod DomainTheory.NeighbourhoodSystems
 .Example12.Token (Sum a b)
            DomainTheory.NeighbourhoodSystems.Exercise326.TD (@DomainTheory.
 NeighbourhoodSystems.prod a b V_0 V_1))
          (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
          (@DomainTheory.NeighbourhoodSystems.Exercise326.condSum a b V_0 V_1 h_0
 h_1)
          (@DomainTheory.NeighbourhoodSystems.pair DomainTheory.NeighbourhoodSystems
 .Example12.Token (Sum a b)
            DomainTheory.NeighbourhoodSystems.Exercise326.TD (@DomainTheory.
 NeighbourhoodSystems.prod a b V_0 V_1)
            DomainTheory.NeighbourhoodSystems.Example23.botElt
            (@DomainTheory.NeighbourhoodSystems.pair a b V_0 V_1 x y)))
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bot (Option (Sum a b)
 )
          (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)))
    (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem a}
      {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
      {h_0 : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem.mem a V_0 X  ->  @Set.Nonempty a X}
      {h_1 : forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem.mem b V_1 Y  ->  @Set.Nonempty b Y}
      (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V_0)
      (y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element b V_1),
      @Eq
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Option (Sum
 a b))
          (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1))
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
          (Sum DomainTheory.NeighbourhoodSystems.Example12.Token (Sum a b)) (Option
```

```
 (Sum a b))
         (@DomainTheory.NeighbourhoodSystems.prod DomainTheory.NeighbourhoodSystems
 .Example12.Token (Sum a b)
           DomainTheory.NeighbourhoodSystems.Exercise326.TD (@DomainTheory.
 NeighbourhoodSystems.prod a b V_0 V_1))
         (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
         (@DomainTheory.NeighbourhoodSystems.Exercise326.condSum a b V_0 V_1 h_0
 h_1)
         (@DomainTheory.NeighbourhoodSystems.pair DomainTheory.NeighbourhoodSystems
 .Example12.Token (Sum a b)
           DomainTheory.NeighbourhoodSystems.Exercise326.TD (@DomainTheory.
 NeighbourhoodSystems.prod a b V_0 V_1)
           DomainTheory.NeighbourhoodSystems.Example23.falseElt
           (@DomainTheory.NeighbourhoodSystems.pair a b V_0 V_1 x y)))
       (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap b (Option (
 Sum a b)) V_1
         (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
         (@DomainTheory.NeighbourhoodSystems.inMap_1 a b V_0 V_1 h_0 h_1) y)))
 (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem a}
   {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
   {h_0 : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem.mem a V_0 X  ->  @Set.Nonempty a X}
   {h_1 : forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem.mem b V_1 Y  ->  @Set.Nonempty b Y}
   (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V_0)
   (y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element b V_1),
   @Eq
     (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Option (Sum a
 b))
       (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1))
     (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
       (Sum DomainTheory.NeighbourhoodSystems.Example12.Token (Sum a b)) (Option (
 Sum a b))
       (@DomainTheory.NeighbourhoodSystems.prod DomainTheory.NeighbourhoodSystems.
 Example12.Token (Sum a b)
         DomainTheory.NeighbourhoodSystems.Exercise326.TD (@DomainTheory.
 NeighbourhoodSystems.prod a b V_0 V_1))
       (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
       (@DomainTheory.NeighbourhoodSystems.Exercise326.condSum a b V_0 V_1 h_0 h_1)
       (@DomainTheory.NeighbourhoodSystems.pair DomainTheory.NeighbourhoodSystems.
 Example12.Token (Sum a b)
         DomainTheory.NeighbourhoodSystems.Exercise326.TD (@DomainTheory.
 NeighbourhoodSystems.prod a b V_0 V_1)
         DomainTheory.NeighbourhoodSystems.Example23.trueElt
         (@DomainTheory.NeighbourhoodSystems.pair a b V_0 V_1 x y)))
     (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a (Option (
 Sum a b)) V_0
       (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
       (@DomainTheory.NeighbourhoodSystems.inMap_0 a b V_0 V_1 h_0 h_1) x)))
(forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem a}
 {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
 {h_0 : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 mem a V_0 X  ->  @Set.Nonempty a X}
 {h_1 : forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
```

```
mem b V_1 Y  ->  @Set.Nonempty b Y}
(x :
  @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Option (Sum a b))
    (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)),
@Eq
  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Option (Sum a b)
)
    (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1))
  (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
    (Sum DomainTheory.NeighbourhoodSystems.Example12.Token (Sum (Option (Sum a b))
 (Option (Sum a b))))
    (Option (Sum a b))
    (@DomainTheory.NeighbourhoodSystems.prod DomainTheory.NeighbourhoodSystems.
Example12.Token
      (Sum (Option (Sum a b)) (Option (Sum a b))) DomainTheory.
NeighbourhoodSystems.Exercise326.TD
      (@DomainTheory.NeighbourhoodSystems.prod (Option (Sum a b)) (Option (Sum a b
))
        (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
        (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)))
    (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
    (@DomainTheory.NeighbourhoodSystems.Exercise326.cond (Option (Sum a b))
      (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1))
    (@DomainTheory.NeighbourhoodSystems.pair DomainTheory.NeighbourhoodSystems.
Example12.Token
      (Sum (Option (Sum a b)) (Option (Sum a b))) DomainTheory.
NeighbourhoodSystems.Exercise326.TD
      (@DomainTheory.NeighbourhoodSystems.prod (Option (Sum a b)) (Option (Sum a b
))
        (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
        (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1))
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap (Option (
Sum a b))
       DomainTheory.NeighbourhoodSystems.Example12.Token (@DomainTheory.
NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
        DomainTheory.NeighbourhoodSystems.Exercise326.TD
        (@DomainTheory.NeighbourhoodSystems.Exercise326.whichMap a b V_0 V_1 h_0
h_1) x)
      (@DomainTheory.NeighbourhoodSystems.pair (Option (Sum a b)) (Option (Sum a b
))
        (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
        (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a (Option
 (Sum a b)) V_0
          (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
          (@DomainTheory.NeighbourhoodSystems.inMap_0 a b V_0 V_1 h_0 h_1)
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap (Option
 (Sum a b)) a
            (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1) V_0
            (@DomainTheory.NeighbourhoodSystems.outMap_0 a b V_0 V_1 h_0 h_1) x))
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap b (Option
 (Sum a b)) V_1
          (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
          (@DomainTheory.NeighbourhoodSystems.inMap_1 a b V_0 V_1 h_0 h_1)
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap (Option
 (Sum a b)) b
```

```
            (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1) V_1
            (@DomainTheory.NeighbourhoodSystems.outMap_1 a b V_0 V_1 h_0 h_1) x)))
 ))
   x)
(@And.intro
 (And
   (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem a}
     {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
     {h_0 : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem.mem a V_0 X  ->  @Set.Nonempty a X}
     {h_1 : forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem.mem b V_1 Y  ->  @Set.Nonempty b Y}
     (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V_0)
     (y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element b V_1),
     @Eq
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Option (Sum
 a b))
         (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1))
       (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
         (Sum DomainTheory.NeighbourhoodSystems.Example12.Token (Sum a b)) (Option
 (Sum a b))
         (@DomainTheory.NeighbourhoodSystems.prod DomainTheory.NeighbourhoodSystems
 .Example12.Token (Sum a b)
           DomainTheory.NeighbourhoodSystems.Exercise326.TD (@DomainTheory.
 NeighbourhoodSystems.prod a b V_0 V_1))
         (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
         (@DomainTheory.NeighbourhoodSystems.Exercise326.condSum a b V_0 V_1 h_0
 h_1)
         (@DomainTheory.NeighbourhoodSystems.pair DomainTheory.NeighbourhoodSystems
 .Example12.Token (Sum a b)
           DomainTheory.NeighbourhoodSystems.Exercise326.TD (@DomainTheory.
 NeighbourhoodSystems.prod a b V_0 V_1)
           DomainTheory.NeighbourhoodSystems.Example23.botElt
           (@DomainTheory.NeighbourhoodSystems.pair a b V_0 V_1 x y)))
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bot (Option (Sum a b)
 )
         (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)))
   (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem a}
     {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
     {h_0 : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem.mem a V_0 X  ->  @Set.Nonempty a X}
     {h_1 : forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem.mem b V_1 Y  ->  @Set.Nonempty b Y}
     (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V_0)
     (y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element b V_1),
     @Eq
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Option (Sum
 a b))
         (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1))
       (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
         (Sum DomainTheory.NeighbourhoodSystems.Example12.Token (Sum a b)) (Option
 (Sum a b))
         (@DomainTheory.NeighbourhoodSystems.prod DomainTheory.NeighbourhoodSystems
 .Example12.Token (Sum a b)
```

```
          DomainTheory.NeighbourhoodSystems.Exercise326.TD (@DomainTheory.
NeighbourhoodSystems.prod a b V_0 V_1))
        (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
        (@DomainTheory.NeighbourhoodSystems.Exercise326.condSum a b V_0 V_1 h_0
h_1)
        (@DomainTheory.NeighbourhoodSystems.pair DomainTheory.NeighbourhoodSystems
.Example12.Token (Sum a b)
          DomainTheory.NeighbourhoodSystems.Exercise326.TD (@DomainTheory.
NeighbourhoodSystems.prod a b V_0 V_1)
          DomainTheory.NeighbourhoodSystems.Example23.falseElt
          (@DomainTheory.NeighbourhoodSystems.pair a b V_0 V_1 x y)))
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap b (Option (
Sum a b)) V_1
        (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
        (@DomainTheory.NeighbourhoodSystems.inMap_1 a b V_0 V_1 h_0 h_1) y)))
(forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
  {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
  {h_0 : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem a V_0 X  ->  @Set.Nonempty a X}
  {h_1 : forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem b V_1 Y  ->  @Set.Nonempty b Y}
  (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V_0)
  (y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element b V_1),
  @Eq
    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Option (Sum a
b))
      (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1))
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
      (Sum DomainTheory.NeighbourhoodSystems.Example12.Token (Sum a b)) (Option (
Sum a b))
      (@DomainTheory.NeighbourhoodSystems.prod DomainTheory.NeighbourhoodSystems.
Example12.Token (Sum a b)
        DomainTheory.NeighbourhoodSystems.Exercise326.TD (@DomainTheory.
NeighbourhoodSystems.prod a b V_0 V_1))
      (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
      (@DomainTheory.NeighbourhoodSystems.Exercise326.condSum a b V_0 V_1 h_0 h_1)
      (@DomainTheory.NeighbourhoodSystems.pair DomainTheory.NeighbourhoodSystems.
Example12.Token (Sum a b)
        DomainTheory.NeighbourhoodSystems.Exercise326.TD (@DomainTheory.
NeighbourhoodSystems.prod a b V_0 V_1)
        DomainTheory.NeighbourhoodSystems.Example23.trueElt
        (@DomainTheory.NeighbourhoodSystems.pair a b V_0 V_1 x y)))
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a (Option (
Sum a b)) V_0
      (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
      (@DomainTheory.NeighbourhoodSystems.inMap_0 a b V_0 V_1 h_0 h_1) x))
(@And.intro
  (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
    {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
    {h_0 : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem a V_0 X  ->  @Set.Nonempty a X}
    {h_1 : forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem b V_1 Y  ->  @Set.Nonempty b Y}
    (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V_0)
```

```
    (y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element b V_1),
    @Eq
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Option (Sum
a b))
        (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1))
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
        (Sum DomainTheory.NeighbourhoodSystems.Example12.Token (Sum a b)) (Option
(Sum a b))
        (@DomainTheory.NeighbourhoodSystems.prod DomainTheory.NeighbourhoodSystems
.Example12.Token (Sum a b)
          DomainTheory.NeighbourhoodSystems.Exercise326.TD (@DomainTheory.
NeighbourhoodSystems.prod a b V_0 V_1))
        (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
        (@DomainTheory.NeighbourhoodSystems.Exercise326.condSum a b V_0 V_1 h_0
h_1)
        (@DomainTheory.NeighbourhoodSystems.pair DomainTheory.NeighbourhoodSystems
.Example12.Token (Sum a b)
          DomainTheory.NeighbourhoodSystems.Exercise326.TD (@DomainTheory.
NeighbourhoodSystems.prod a b V_0 V_1)
          DomainTheory.NeighbourhoodSystems.Example23.botElt
           (@DomainTheory.NeighbourhoodSystems.pair a b V_0 V_1 x y)))
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bot (Option (Sum a b)
)
        (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)))
  (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
    {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
    {h_0 : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem a V_0 X  ->  @Set.Nonempty a X}
    {h_1 : forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem b V_1 Y  ->  @Set.Nonempty b Y}
    (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V_0)
    (y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element b V_1),
    @Eq
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Option (Sum
a b))
        (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1))
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
        (Sum DomainTheory.NeighbourhoodSystems.Example12.Token (Sum a b)) (Option
(Sum a b))
        (@DomainTheory.NeighbourhoodSystems.prod DomainTheory.NeighbourhoodSystems
.Example12.Token (Sum a b)
          DomainTheory.NeighbourhoodSystems.Exercise326.TD (@DomainTheory.
NeighbourhoodSystems.prod a b V_0 V_1))
        (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
        (@DomainTheory.NeighbourhoodSystems.Exercise326.condSum a b V_0 V_1 h_0
h_1)
        (@DomainTheory.NeighbourhoodSystems.pair DomainTheory.NeighbourhoodSystems
.Example12.Token (Sum a b)
          DomainTheory.NeighbourhoodSystems.Exercise326.TD (@DomainTheory.
NeighbourhoodSystems.prod a b V_0 V_1)
          DomainTheory.NeighbourhoodSystems.Example23.falseElt
           (@DomainTheory.NeighbourhoodSystems.pair a b V_0 V_1 x y)))
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap b (Option (
Sum a b)) V_1
        (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
```

```
            (@DomainTheory.NeighbourhoodSystems.inMap_1 a b V_0 V_1 h_0 h_1) y))
      @DomainTheory.NeighbourhoodSystems.Exercise326.condSum_bot
      @DomainTheory.NeighbourhoodSystems.Exercise326.condSum_false)
    @DomainTheory.NeighbourhoodSystems.Exercise326.condSum_true)
  @DomainTheory.NeighbourhoodSystems.Exercise326.cond_which
end
end DomainTheory.NeighbourhoodSystems.Exercise326
end
```

3.4.28 Exercise 3.27 — function space via closure

(For set theorists.) Give another proof that the family of approximable mappings $f : D_0 \to D_1$ is isomorphic to a domain by employing the general argument of Exercise 2.22. How does this compare with the proof method of 3.9 and 3.10? Can the general remarks also be employed to show that

$$\text{eval} : (D_1 \to D_2) \times D_1 \to D_2$$

is approximable without bringing in the neighbourhoods in such an explicit way? (Hint: Use 3.5 and the idea of Exercise 2.12.)

Exercise_3_27.lean

```
module
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Exercise_2_22
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_12
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Lemma_3_6
public import Mathlib.Order.Directed
@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems.Exercise327
section
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap

section
variable {a b : Type*} {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}

/-- The **graph** of an approximable map, as a set of neighbourhood pairs. -/
def graph (f : ApproximableMap V_0 V_1) : Set (Set a  x  Set b) := {p | f.rel p.1 p.2}

@[simp] theorem mem_graph {f : ApproximableMap V_0 V_1} {p : Set a  x  Set b} :
    p  in  graph f  <->  f.rel p.1 p.2 := Iff.rfl

/-- Scott's family `C`: all graphs of approximable maps `?_0  ->  ?_1`. -/
def C (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) : Set (Set (Set a  x
    Set b)) :=
  Set.range (graph (V_0 := V_0) (V_1 := V_1))

theorem C_nonempty : (C V_0 V_1).Nonempty :=
  <graph (constMap V_0 V_1.bot), constMap V_0 V_1.bot, rfl>

theorem graph_injective : Function.Injective (graph (V_0 := V_0) (V_1 := V_1)) :=
  fun _ _ h => ApproximableMap.ext fun X Y => Set.ext_iff.mp h (X, Y)
```

```
/-- `graph` is an order-isomorphism of approximable maps (under ` sqsub `) onto the
    family `C`
(under ` subseteq `). -/
noncomputable def graphEquiv (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b)
     :
    ApproximableMap V_0 V_1 ?o {X : Set (Set a  x  Set b) // X  in  C V_0 V_1} where
  toFun f := <graph f, f, rfl>
  invFun X := X.2.choose
  left_inv f := graph_injective (<graph f, f, rfl> : {X // X  in  C V_0 V_1}).2.
    choose_spec
  right_inv X := Subtype.ext X.2.choose_spec
  map_rel_iff' := by
    intro f g
    show graph f  subseteq  graph g  <->  f  <=  g
    rw [ApproximableMap.le_iff]
    exact <fun h X Y hf => h (a := (X, Y)) hf, fun h p hp => h p.1 p.2 hp>

/-- The **join** ` \/  ?` of a directed family of approximable maps: `X ( \/ ?) Y` iff
    `(X, Y)` lies in
some member. Directedness is what restores the intersectivity condition (ii). -/
def joinMap (? : Set (Set (Set a  x  Set b))) (h? : ?  subseteq  C V_0 V_1) (hne : ?.
    Nonempty)
    (hdir : DirectedOn (*  subseteq  *) ?) : ApproximableMap V_0 V_1 where
  rel X Y := exists  S  in  ?, (X, Y)  in  S
  rel_dom := by rintro X Y <S, hS, hp>; obtain <f, rfl> := h? hS; exact f.rel_dom hp
  rel_cod := by rintro X Y <S, hS, hp>; obtain <f, rfl> := h? hS; exact f.rel_cod hp
  master_rel := by
    obtain <S, hS> := hne; obtain <f, rfl> := h? hS
    exact <graph f, hS, f.master_rel>
  inter_right := by
    rintro X Y Y' <S, hS, hp> <S', hS', hp'>
    obtain <S_3, hS_3, hSS_3, hS'S_3> := hdir S hS S' hS'
    obtain <f, rfl> := h? hS_3
    exact <graph f, hS_3, f.inter_right (hSS_3 hp) (hS'S_3 hp')>
  mono := by
    rintro X X' Y Y' <S, hS, hp> hX'X hYY' hX' hY'
    obtain <f, rfl> := h? hS
    exact <graph f, hS, f.mono hp hX'X hYY' hX' hY'>

/-- The pointwise **meet** ` /\  ?` of a family of approximable maps (drawn from `C`):
    `X ( /\ ?) Y` iff
`(X, Y)` lies in every member of `?`. -/
def meetMap (? : Set (Set (Set a  x  Set b))) (h? : ?  subseteq  C V_0 V_1) (hne : ?.
    Nonempty) :
    ApproximableMap V_0 V_1 where
  rel X Y := forall  S  in  ?, (X, Y)  in  S
  rel_dom := by
    intro X Y h
    obtain <S, hS> := hne; obtain <f, rfl> := h? hS
    exact f.rel_dom (h _ hS)
  rel_cod := by
    intro X Y h
    obtain <S, hS> := hne; obtain <f, rfl> := h? hS
    exact f.rel_cod (h _ hS)
  master_rel := by
    intro S hS; obtain <f, rfl> := h? hS; exact f.master_rel
```

```
  inter_right := by
    intro X Y Y' hY hY' S hS
    obtain <f, rfl> := h? hS
    exact f.inter_right (hY _ hS) (hY' _ hS)
  mono := by
    intro X X' Y Y' h hX'X hYY' hX' hY' S hS
    obtain <f, rfl> := h? hS
    exact f.mono (h _ hS) hX'X hYY' hX' hY'

theorem sInter_eq_graph_meetMap (? : Set (Set (Set a  x  Set b))) (h? : ?  subseteq  C
     V_0 V_1)
    (hne : ?.Nonempty) : ?_0 ? = graph (meetMap ? h? hne) := by
  ext p
  simp only [Set.mem_sInter, mem_graph]
  exact <fun h S hS => h S hS, fun h S hS => h S hS>

/-- **Exercise 2.22 hypothesis (i).** `C` is closed under non-empty intersections. -/
theorem C_inter : forall  ? : Set (Set (Set a  x  Set b)), ?.Nonempty  ->  ?  subseteq
      C V_0 V_1  ->  ?_0 ?  in  C V_0 V_1 := by
  intro ? hne h?
  rw [sInter_eq_graph_meetMap ? h? hne]
  exact <meetMap ? h? hne, rfl>

theorem sUnion_eq_graph_joinMap (? : Set (Set (Set a  x  Set b))) (h? : ?  subseteq  C
     V_0 V_1)
    (hne : ?.Nonempty) (hdir : DirectedOn (*  subseteq  *) ?) :
    ?_0 ? = graph (joinMap ? h? hne hdir) := by
  ext p
  simp only [Set.mem_sUnion, mem_graph]
  exact <fun <S, hS, hp> => <S, hS, hp>, fun <S, hS, hp> => <S, hS, hp>>

/-- **Exercise 2.22 hypothesis (ii).** `C` is closed under directed unions. -/
theorem C_dir : forall  ? : Set (Set (Set a  x  Set b)), ?.Nonempty  ->  ?  subseteq
    C V_0 V_1  ->
    DirectedOn (*  subseteq  *) ?  ->  ?_0 ?  in  C V_0 V_1 := by
  intro ? hne h? hdir
  rw [sUnion_eq_graph_joinMap ? h? hne hdir]
  exact <joinMap ? h? hne hdir, rfl>

end

section
set_option linter.unusedVariables false

theorem example_327.{u_1, u_2} : forall  {a : Type u_1} {b : Type u_2} {V_0 :
    DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
  {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b} (? : Set (Set (Prod (
    Set a) (Set b)))),
  @Set.Nonempty (Set (Prod (Set a) (Set b))) ?  ->
    @LE.le (Set (Set (Prod (Set a) (Set b)))) (@Set.instLE (Set (Prod (Set a) (Set b))
    )) ?
        (@DomainTheory.NeighbourhoodSystems.Exercise327.C a b V_0 V_1)  ->
      @DirectedOn (Set (Prod (Set a) (Set b)))
          (fun (x1 x2 : Set (Prod (Set a) (Set b))) =>
            @LE.le (Set (Prod (Set a) (Set b))) (@Set.instLE (Prod (Set a) (Set b)))
    x1 x2)
```

```
          ?  ->
        @Membership.mem (Set (Prod (Set a) (Set b))) (Set (Set (Prod (Set a) (Set b)))
    )
          (@Set.instMembership (Set (Prod (Set a) (Set b))))
          (@DomainTheory.NeighbourhoodSystems.Exercise327.C a b V_0 V_1) (@Set.sUnion
    (Prod (Set a) (Set b)) ?) :=
  @DomainTheory.NeighbourhoodSystems.Exercise327.C_dir
end

section
variable {a b : Type*} {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
/- concordance: source L1627 EXERCISE 3.27 function space via closure -/
/-- **Exercise 3.27.** The function space `|?_0  ->  ?_1|` is (order-)isomorphic to the
     domain produced by
the *abstract* representation theorem (Exercise 2.22) applied to the family `C` of
    graphs of
approximable maps. This re-proves that approximable mappings form a domain without
    appealing to the
explicit step-set neighbourhood system of Definition 3.8 -- composing Exercise 2.22's `
    reprIso` with
the graph isomorphism (`graphEquiv`) and Theorem 3.10 (`funSpaceEquiv`). -/
noncomputable def funSpaceReprIso (V_0 : NeighborhoodSystem a) (V_1 :
    NeighborhoodSystem b) :
    (Exercise222.reprSystem (C V_0 V_1) C_inter C_nonempty).Element ?o (funSpace V_0
    V_1).Element :=
  (Exercise222.reprIso (C V_0 V_1) C_inter C_nonempty C_dir).trans
    ((graphEquiv V_0 V_1).symm.trans (funSpaceEquiv V_0 V_1).symm)
end
end

end DomainTheory.NeighbourhoodSystems.Exercise327
end
```

3.4.29 Exercise 3.28 — minimal step function

In the function space $(D_0 \to D_1)$ let

$$\bigcap\{[X_i, Y_i] \mid i < n\}$$

be a (non-empty) neighbourhood. In 3.9 the minimal element of this neighbourhood is characterized as a relation f_0. Show that as an elementwise mapping it can be defined by the formula

$$f_0(x) = \bigsqcup\{\uparrow Y_i \mid x \in [X_i]\},$$

for $x \in |D_0|$. Try to draw a picture of $|D_0|$ with neighbourhoods $[X_i]$ and the corresponding values of the function f_0.

LECTURE IV FIXED POINTS AND RECURSION

Having at this point a large supply of examples of domains (and further constructs of new domains), we now have to consider some other ways of defining functions — other than by explicit compositions of the very basic functions already mentioned. One of the most fruitful techniques is an infinitely iterated composition that is at the back of the idea of recursion. We will use the process over and over again in these

lectures, not only to define new functions but also to define new domains. The heart of the matter lies in the so-called “Fixed-point Theorem”:

Exercise_3_28.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_27
@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem ApproximableMap
section
variable {a b : Type*} {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
/- concordance: source L1635 EXERCISE 3.28 minimal step function -/
/-- **Exercise 3.28 -- the active values.** The set `{ Y_i ? x  in  [X_i] }` of
    principal filters whose
input `X_i` is a member of `x`. -/
def leastMapVals (L : List (Set a  x  Set b)) (hL : forall  p  in  L, V_0.mem p.1  /\
     V_1.mem p.2)
    (x : V_0.Element) : Set V_1.Element :=
  {z | exists  (p : Set a  x  Set b) (hp : p  in  L), x.mem p.1  /\  z = V_1.principal
     (hL p hp).2}

/-- A filter `z` contains the finite intersection `?{Y_i ? X  subseteq  X_i}` (inside `
    Delta_1`) as soon as it
contains each `Y_i` whose input `X_i  supseteq  X`. (The element-side analogue of `
    rel_interYs`.) -/
theorem Element.mem_interYs_of (z : V_1.Element) {L : List (Set a  x  Set b)} {X : Set
     a}
    (hall : forall  p  in  L, X  subseteq  p.1  ->  z.mem p.2) : z.mem (interYs V_1.
    master L X) := by
  induction L with
  | nil => exact z.master_mem
  | cons p L ih =>
    have htail : forall  q  in  L, X  subseteq  q.1  ->  z.mem q.2 :=
      fun q hq => hall q (List.mem_cons.mpr (Or.inr hq))
    by_cases hXp : X  subseteq  p.1
    * have heq : interYs V_1.master (p :: L) X = p.2  I  interYs V_1.master L X := by
        rw [interYs_cons]; ext w
        simp only [Set.mem_inter_iff, Set.mem_ofPred_eq]
        exact <fun <h1, h2> => <h1 hXp, h2>, fun <h1, h2> => <fun _ => h1, h2>>
      rw [heq]
      exact z.inter_mem (hall p (List.mem_cons.mpr (Or.inl rfl)) hXp) (ih htail)
    * have heq : interYs V_1.master (p :: L) X = interYs V_1.master L X := by
        rw [interYs_cons]; ext w
        simp only [Set.mem_inter_iff, Set.mem_ofPred_eq]
        exact <fun h => h.2, fun h => <fun hc => absurd hc hXp, h>>
      rw [heq]
      exact ih htail

/-- `f_0(x)` is a *lower* bound of the upper bounds: any upper bound `z` of the active
```

```
    values
dominates `f_0(x)`. (Uses that `z` then contains `?{Y_i ? X  subseteq  X_i}`.) -/
theorem leastMap_toElementMap_le_of_ub {L : List (Set a  x  Set b)}
    (hL : forall  p  in  L, V_0.mem p.1  /\  V_1.mem p.2)
    (hcons : forall  {X}, V_0.mem X  ->  V_1.mem (interYs V_1.master L X)) {x : V_0.
    Element} {z : V_1.Element}
    (hz : forall  w  in  leastMapVals L hL x, w  <=  z) :
    (leastMap L hL hcons).toElementMap x  <=  z := by
  rintro Y' <X', hX'x, hX'mem, hY'mem, hsub>
  have hzint : z.mem (interYs V_1.master L X') := by
    apply Element.mem_interYs_of
    intro p hp hX'p
    have hp1x : x.mem p.1 := x.up_mem hX'x (hL p hp).1 hX'p
    have hle : V_1.principal (hL p hp).2  <=  z := hz _ <p, hp, hp1x, rfl>
    exact hle p.2 <(hL p hp).2, subset_rfl>
  exact z.up_mem hzint hY'mem hsub

/-- Each active value `Y_i` (with `X_i  in  x`) approximates `f_0(x)`, because `Y_i  in
     f_0(x)`. -/
theorem val_le_leastMap_toElementMap {L : List (Set a  x  Set b)}
    (hL : forall  p  in  L, V_0.mem p.1  /\  V_1.mem p.2)
    (hcons : forall  {X}, V_0.mem X  ->  V_1.mem (interYs V_1.master L X)) {x : V_0.
    Element}
    {p : Set a  x  Set b} (hp : p  in  L) (hp1x : x.mem p.1) :
    V_1.principal (hL p hp).2  <=  (leastMap L hL hcons).toElementMap x := by
  have hmem : ((leastMap L hL hcons).toElementMap x).mem p.2 :=
    <p.1, hp1x, leastMap_mem_stepFun hL hcons p hp>
  intro Z hZ
  exact ((leastMap L hL hcons).toElementMap x).up_mem hmem hZ.1 hZ.2

/-- The active values are bounded (by `f_0(x)`). -/
theorem leastMapVals_bounded {L : List (Set a  x  Set b)} (hL : forall  p  in  L, V_0.
    mem p.1  /\  V_1.mem p.2)
    (hcons : forall  {X}, V_0.mem X  ->  V_1.mem (interYs V_1.master L X)) (x : V_0.
    Element) :
    V_1.Bounded (leastMapVals L hL x) := by
  refine <(leastMap L hL hcons).toElementMap x, ?_>
  rintro w <p, hp, hp1x, rfl>
  exact val_le_leastMap_toElementMap hL hcons hp hp1x

/-- **Exercise 3.28 (Scott 1981, PRG-19).** The least map's elementwise action is the
    least upper
bound of the active principal filters: `f_0(x) =  sqsup  { Y_i ? x  in  [X_i] }`. -/
theorem toElementMap_leastMap_eq_sSup (L : List (Set a  x  Set b))
    (hL : forall  p  in  L, V_0.mem p.1  /\  V_1.mem p.2)
    (hcons : forall  {X}, V_0.mem X  ->  V_1.mem (interYs V_1.master L X)) (x : V_0.
    Element) :
    (leastMap L hL hcons).toElementMap x
      = V_1.sSup (leastMapVals L hL x) (leastMapVals_bounded hL hcons x) := by
  apply le_antisymm
  * apply leastMap_toElementMap_le_of_ub hL hcons
    intro w hw
    exact V_1.le_sSup _ (leastMapVals_bounded hL hcons x) hw
  * apply V_1.sSup_le
    rintro w <p, hp, hp1x, rfl>
    exact val_le_leastMap_toElementMap hL hcons hp hp1x
```

```
end

section
set_option linter.unusedVariables false

theorem example_328.{u_1, u_2} : forall  {a : Type u_1} {b : Type u_2} {V_0 :
    DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
  {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b} (L : List (Prod (Set
    a) (Set b)))
  (hL :
    forall  (p : Prod (Set a) (Set b)),
      @Membership.mem (Prod (Set a) (Set b)) (List (Prod (Set a) (Set b))) (@List.
    instMembership (Prod (Set a) (Set b)))
          L p  ->
        And (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V_0 (@Prod.
    fst (Set a) (Set b) p))
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b V_1 (@Prod.snd
    (Set a) (Set b) p)))
  (hcons :
    forall  {X : Set a},
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V_0 X  ->
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b V_1
          (@DomainTheory.NeighbourhoodSystems.interYs a b
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.master b V_1) L X))
  (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V_0),
  @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element b V_1)
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a b V_0 V_1
      (@DomainTheory.NeighbourhoodSystems.leastMap a b V_0 V_1 L hL fun {X : Set a} =>
     @hcons X) x)
    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.sSup b V_1
      (@DomainTheory.NeighbourhoodSystems.leastMapVals a b V_0 V_1 L hL x)
      (@DomainTheory.NeighbourhoodSystems.leastMapVals_bounded a b V_0 V_1 L hL (fun {
    X : Set a} => @hcons X) x)) :=
  @DomainTheory.NeighbourhoodSystems.toElementMap_leastMap_eq_sSup
end
end

end DomainTheory.NeighbourhoodSystems
end
```

3.5 Fixed points and recursion

Dependency diagram. Each box is one item. The arrowhead is on the item that imports. An arrow already implied by a longer path is left out. The introduction has no Lean file, so it is not shown. Blue is a definition, amber an example, red a theorem, and indigo an exercise.

3.5.1 Introduction

LECTURE IV FIXED POINTS AND RECURSION

Having at this point a large supply of examples of domains (and further constructs of new domains), we now have to consider some other ways of defining functions — other than by explicit compositions of the very basic functions already mentioned. One of the most fruitful techniques is an infinitely iterated

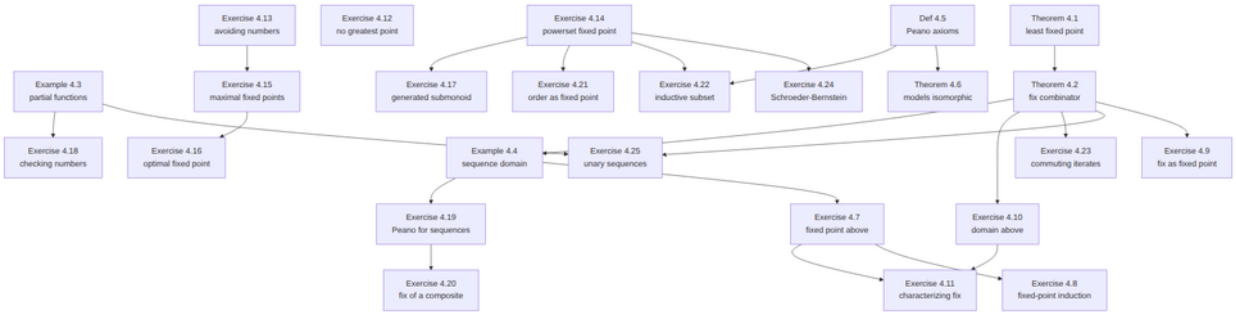


Figure 33: Fixed points and recursion.

composition that is at the back of the idea of recursion. We will use the process over and over again in these lectures, not only to define new functions but also to define new domains. The heart of the matter lies in the so-called "Fixed-point Theorem":

3.5.2 Theorem 4.1 — least fixed point

THEOREM 4.1. For any approximable mapping $f : D \to D$ on any domain, there exists a least element $x \in |D|$ where

$$f(x) = x.$$

Proof: Let f^n for $n \in \mathbf{N}$ stand for the n-fold composition of f with itself. That is,

$$f^0 = I_D, \text{ and}$$

$$f^{n+1} = f \circ f^n.$$

Define

$$x = \{X \in D \mid \Delta\ f^n\ X, \text{ for some } n \in \mathbf{N}\}.$$

We see $X \in x$ iff there is a finite sequence $\Delta = X_0, X_1, \ldots, X_n = X$ where $X_i\ f\ X_{i+1}$ holds for all $i < n$. Now since $\Delta\ f\ \Delta$ automatically holds, a sequence for an $X \in x$ can always be extended to a longer sequence just by adding more Δ's on the front.

We want to prove $x \in |D|$. Clearly $\Delta \in x$; and if $X \subseteq Y$ and $X \in x$, then $Y \in x$. All that remains to be shown is the closure of x under intersection. Note that if

$$U\ f\ V \quad \text{and} \quad U'\ f\ V'$$

hold and U, U' are consistent in D, then V and V' are consistent and
$(U \cap U')\ f\ (V \cap V')$
must hold. Generalizing this to sequences, if

$$\Delta = X_0\ f\ X_1\ f\ \cdots\ f\ X_n = X, \text{ and}$$

$$\Delta = Y_0 \; f \; Y_1 \; f \; \cdots \; f \; Y_n = Y$$

both hold (and note we have arranged the lengths of the two sequences to be equal), then each pair X_i, Y_i is consistent and we have

$$\Delta = (X_0 \cap Y_0) \; f \; (X_1 \cap Y_1) \; f \; \cdots \; f \; (X_n \cap Y_n) = X \cap Y.$$

This establishes the desired closure.

We also note that if $X \in x$ and $X \; f \; Y$ then $Y \in x$. Therefore, $f(x) \subseteq x$ and indeed by its very construction x is the *least* element of $|D|$ with this property. (Why?) But f is monotone, so $f(f(x)) \subseteq f(x)$; hence, $x = f(x)$. By what we have already said it must be the least such element. □

Because the element we have shown to exist in 4.1 is a least element, it is unique. That is, we have associated with each $f : D \to D$ a special element $x_f \in |D|$ determined by the choice of f. A function has therefore been defined mapping the set $|D \to D|$ into $|D|$. The next result shows that this function, or operator on functions, is in fact approximable.

Theorem_4_1.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Exercise_3_26
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Definition_3_8

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap
section
variable {a : Type*} {V : NeighborhoodSystem a}
namespace ApproximableMap

/- concordance: source L1657 THEOREM 4.1 least fixed point -/
/-- **Theorem 4.1 (Scott 1981, PRG-19).** The `n`-fold composition `f?` of an endomap
    with itself:
`f? = I_?` and `f^{n+1} = f  comp  f?`. -/
def iterMap (f : ApproximableMap V V) : Nat  ->  ApproximableMap V V
  | 0 => idMap V
  | (n + 1) => f.comp (f.iterMap n)

@[simp] theorem iterMap_zero (f : ApproximableMap V V) : f.iterMap 0 = idMap V := rfl

@[simp] theorem iterMap_succ (f : ApproximableMap V V) (n : Nat) :
    f.iterMap (n + 1) = f.comp (f.iterMap n) := rfl

/-- Composition is monotone in both arguments. -/
theorem comp_mono {f g a b : ApproximableMap V V} (hfg : f  <=  g) (hab : a  <=  b) :
    f.comp a  <=  g.comp b := by
  intro X Z h
  obtain <Y, hXY, hYZ> := h
  exact <Y, hab X Y hXY, hfg Y Z hYZ>
```

```
/-- The iterate is monotone in the map: `f  sqsub  g ? f?  sqsub  g?` (Scott's "`f?
    subseteq  g?`"). -/
theorem iterMap_mono_map {f g : ApproximableMap V V} (hfg : f  <=  g) (n : Nat) :
    f.iterMap n  <=  g.iterMap n := by
  induction n with
  | zero => show (idMap V)  <=  (idMap V); exact le_refl _
  | succ k ih => exact comp_mono hfg ih

/-- `f` commutes with its own iterate: `f  comp  f? = f?  comp  f`. Proved by induction
     using associativity
and the identity laws. -/
theorem iter_comm (f : ApproximableMap V V) (n : Nat) :
    f.comp (f.iterMap n) = (f.iterMap n).comp f := by
  induction n with
  | zero => rw [show f.iterMap 0 = idMap V from rfl, comp_idMap, idMap_comp]
  | succ n ih =>
    show f.comp (f.comp (f.iterMap n)) = (f.comp (f.iterMap n)).comp f
    rw [comp_assoc,  <-  ih]

/-- Scott's "a sequence for an `X  in  x` can always be extended": if `Delta f? X`,
    then `Delta f^{n+1} X`
(prepend a `Delta`-step, using `Delta f Delta`). -/
theorem rel_master_succ (f : ApproximableMap V V) {n : Nat} {X : Set a}
    (h : (f.iterMap n).rel V.master X) : (f.iterMap (n + 1)).rel V.master X := by
  have hcomm : f.iterMap (n + 1) = (f.iterMap n).comp f := iter_comm f n
  rw [hcomm]
  exact <V.master, f.master_rel, h>

/-- Monotonicity of the reachability relation in the number of steps: `n  <=  m` and `
    Delta f? X` imply
`Delta f? X`. -/
theorem rel_master_mono (f : ApproximableMap V V) {n m : Nat} (hnm : n  <=  m) {X :
    Set a}
    (h : (f.iterMap n).rel V.master X) : (f.iterMap m).rel V.master X := by
  induction hnm with
  | refl => exact h
  | step _ ih => exact rel_master_succ f ih

/-- **Theorem 4.1 (Scott 1981, PRG-19).** The least fixed point of `f`, Scott's
`x = {X  in  ? ? Delta f? X for some n}`. The three filter conditions are exactly Scott
    's: `Delta  in  x` (the
`n = 0` witness `I_?`); closure under intersection follows from intersectivity (`
    inter_right`) of
the single iterate `f^{max n m}` reached by extending the shorter chain; upward closure
     is `mono`. -/
def fixElement (f : ApproximableMap V V) : V.Element where
  mem X := exists  n, (f.iterMap n).rel V.master X
  sub := fun <n, h> => (f.iterMap n).rel_cod h
  master_mem := <0, show (idMap V).rel V.master V.master from (idMap V).master_rel>
  inter_mem := by
    rintro X Y <n, hn> <m, hm>
    refine <max n m, ?_>
    have hX : (f.iterMap (max n m)).rel V.master X := rel_master_mono f (le_max_left n
     m) hn
    have hY : (f.iterMap (max n m)).rel V.master Y := rel_master_mono f (le_max_right
    n m) hm
```

```
    exact (f.iterMap (max n m)).inter_right hX hY
  up_mem := by
    rintro X Y <n, hn> hYmem hXY
    exact <n, (f.iterMap n).mono hn subset_rfl hXY V.master_mem hYmem>

@[simp] theorem mem_fixElement (f : ApproximableMap V V) {X : Set a} :
    f.fixElement.mem X  <->  exists  n, (f.iterMap n).rel V.master X := Iff.rfl

/-- The least fixed point is monotone in the map: `f  sqsub  g ? fix(f)  sqsub  fix(g)`
     (immediate from
`iterMap_mono_map`; underlies the approximability of `fix` in 4.2). -/
theorem fixElement_mono {f g : ApproximableMap V V} (hfg : f  <=  g) :
    f.fixElement  <=  g.fixElement := by
  rintro X <n, hn>
  exact <n, iterMap_mono_map hfg n V.master X hn>

end ApproximableMap
end

end DomainTheory.NeighbourhoodSystems
end
```

3.5.3 Theorem 4.2 — fix combinator

THEOREM 4.2. For any domain D, there is an approximable mapping

$$\mathrm{fix} : (D \to D) \to D$$

such that if $f : D \to D$ is any approximable mapping, then

(i) $\mathrm{fix}(f) = f(\mathrm{fix}(f))$.

Furthermore, if $x \in |D|$, then

(ii) $f(x) \subseteq x$ implies $\mathrm{fix}(f) \subseteq x$.

And this last property implies that fix is unique. Explicitly we can characterize fix by the equation:

(iii)

$$\mathrm{fix}(f) = \bigcup_{n=0}^{\infty} f^n(\bot),$$

for all $f : D \to D$.

Proof: Formula (iii) can be put in a more elementary form:

$$\mathrm{fix}(f) = \{X \mid \Delta f^n X, \text{ for some } n \in \mathbf{N}\}.$$

To show an elementwise mapping approximable we can use the formula of Exercise 2.9, applied to the above as the definition of fix:

$$(*) \quad \mathrm{fix}(f) = \bigcup \{\mathrm{fix}(\uparrow F) \mid f \in [F]\},$$

where F ranges over the neighbourhoods of $(D \to D)$, and where $\uparrow F$ can be considered to be the least element of F as calculated in 3.9.

Now from the definition of fix, it is clear that whenever $f \subseteq g$, then $\mathrm{fix}(f) \subseteq \mathrm{fix}(g)$, because $f^n \subseteq g^n$. (That is, fix is obviously monotone.) Next, if $f \in F$, then $\uparrow F$ is a (finite) approximation to f; so $\uparrow F \subseteq f$ and $\mathrm{fix}(\uparrow F) \subseteq \mathrm{fix}(f)$. This means that half of equation (*) already holds by monotonicity. All that is left is to prove the other half.

So suppose $X \in \mathrm{fix}(f)$. Then, as we have already remarked, there is a finite sequence of neighbourhoods where

$$\Delta = X_0 \; f \; X_1 \; \ldots \; X_{n-1} \; f \; X_n = X.$$

Let the function-space neighbourhood be defined as

$$F = \bigcap \{[X_i, X_{i+1}] \mid i < n\},$$

and note that since $f \in [F]$ we have at once consistency. But, by 3.9, $\uparrow F \in [F]$, so the *same* sequence of X_i is sufficient to show that

$$X \in \mathrm{fix}(\uparrow F).$$

In other words, if X belongs to the left-hand side of (*), it also belongs to the right-hand side. This completes the proof of (*).*

Formula (i) is just a restatement of what we proved in 4.1. And (ii) follows easily, because $f(x) \subseteq x$ implies that $\Delta \in x$ and whenever $X \in x$ and $X \; f \; Y$, then $Y \in x$. Thus, by induction, if $\Delta f^n X$, then $X \in x$. So $\mathrm{fix}(f) \subseteq x$.

Finally, if $\mathrm{fax} : (D \to D) \to D$ were any other operator satisfying (i) and (ii), we would prove at once that

$$\mathrm{fix}(f) \subseteq \mathrm{fax}(f) \quad \text{and}$$

$$\mathrm{fax}(f) \subseteq \mathrm{fix}(f).$$

That is to say, the two operators are identical. □

The reader may have noticed that we used recursion in the proof of 4.1 (we had to define f^n for all $n \in \mathbf{N}$). But 4.1 and 4.2 can be used to justify definitions by recursion on a large number of domains — definitions where the process of iteration is far from being as straightforward. In discussing this point, let us start with some basic examples.

Theorem_4_2.lean

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Exercise_2_11
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Exercise_2_8
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Theorem_4_1
```

```
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Exercise_2_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_10
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_7
public import Mathlib.Tactic.Bound

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap
section
variable {a : Type*} {V : NeighborhoodSystem a}
open ApproximableMap
/- concordance: source L1715 THEOREM 4.2 fix combinator -/
/-- **Theorem 4.2 (Scott 1981, PRG-19).** The fixed-point operator `fix : (?  ->  ?)
    ->  ?` as an
approximable mapping. Built by the extension principle (Exercise 2.8, `ofMono`): on the
     finite
element `F` it returns `fix(F)`, where `F = toApproxMap (principal hF)` is the least
    map of the
neighbourhood `F` (Proposition 3.9). Monotonicity of `F  |->  fix(F)` is `
    fixElement_mono` composed
with the order-iso `funSpaceEquiv`. -/
def fixMap (V : NeighborhoodSystem a) : ApproximableMap (funSpace V V) V :=
  ofMono (fun W hW => (toApproxMap ((funSpace V V).principal hW)).fixElement)
    (fun W W' hW hW' hW'W => by
      apply fixElement_mono
      exact (funSpaceEquiv V V).monotone
        (((funSpace V V).principal_le_iff hW hW').mpr hW'W))
end

section
variable {a : Type*} {V : NeighborhoodSystem a}

namespace ApproximableMap

/-- The `n`-th approximant `f?(False)` of the least fixed point. -/
def iterElem (f : ApproximableMap V V) (n : Nat) : V.Element := (f.iterMap n).
    toElementMap V.bot

end ApproximableMap

open ApproximableMap

/-- **Theorem 4.2 (Scott 1981, PRG-19) -- Scott's equation (?), hard half.** A finite `
    f`-chain
`Delta (toApproxMap phi)? X` factors through a *single* finite approximant `F  in  phi
    `: there is a
neighbourhood `W  in  phi` whose least map already realizes the same chain `Delta (W)?
    X`. The witness `W`
is accumulated as the intersection of the (finitely many) step-neighbourhoods used by
    the chain,
which lies in `phi` because `phi` is a filter. -/
theorem exists_principal_iterMap (V : NeighborhoodSystem a) (phi : (funSpace V V).
    Element) :
```

```
    forall  (n : Nat) (X : Set a), ((toApproxMap phi).iterMap n).rel V.master X  ->
      exists  (W : Set (ApproximableMap V V)) (hw : phi.mem W),
        ((toApproxMap ((funSpace V V).principal (phi.sub hw))).iterMap n).rel V.master
     X := by
  intro n
  induction n with
  | zero =>
    intro X hX
    exact <(funSpace V V).master, phi.master_mem, hX>
  | succ k ih =>
    intro X hX
    obtain <Y, hY, hYX> := hX
    obtain <W_1, hw_1, hW_1> := ih Y hY
    have hVY : V.mem Y := ((toApproxMap phi).iterMap k).rel_cod hY
    have hVX : V.mem X := (toApproxMap phi).rel_cod hYX
    have hw_2 : phi.mem (step Y X) := toApproxMap_rel.mp hYX
    have hwInter : phi.mem (W_1  I  step Y X) := phi.inter_mem hw_1 hw_2
    refine <W_1  I  step Y X, hwInter, ?_>
    have hg_1g : toApproxMap ((funSpace V V).principal (phi.sub hw_1))
         <=  toApproxMap ((funSpace V V).principal (phi.sub hwInter)) :=
      (funSpaceEquiv V V).monotone
        (((funSpace V V).principal_le_iff (phi.sub hw_1) (phi.sub hwInter)).mpr Set.
    inter_subset_left)
    have hYg : ((toApproxMap ((funSpace V V).principal (phi.sub hwInter))).iterMap k).
    rel V.master Y :=
      iterMap_mono_map hg_1g k V.master Y hW_1
    have hgYX : (toApproxMap ((funSpace V V).principal (phi.sub hwInter))).rel Y X :=
    by
      show ((funSpace V V).principal (phi.sub hwInter)).mem (step Y X)
      exact <step_mem hVY hVX, Set.inter_subset_right>
    exact <Y, hYg, hgYX>

end

section
variable {a : Type*} {V : NeighborhoodSystem a}

namespace ApproximableMap

/-- **Theorem 4.1 (Scott 1981, PRG-19).** `fixElement f` is the *least pre-fixed point
    *: if
`f(z)  subseteq  z`, then `x  subseteq  z`. (Scott's induction: `Delta  in  z`, and `X
     in  z`, `X f Y` give `Y  in  f(z)  subseteq  z`,
so `Delta f? X` implies `X  in  z`.) In particular `x` is the least element with `f(x)
    = x`. -/
theorem fixElement_le_of_toElementMap_le (f : ApproximableMap V V) {z : V.Element}
    (hz : f.toElementMap z  <=  z) : f.fixElement  <=  z := by
  have key : forall  n X, (f.iterMap n).rel V.master X  ->  z.mem X := by
    intro n
    induction n with
    | zero =>
      intro X hn
      obtain <_, hXmem, hmX> := hn
      have hXmaster : X = V.master := Set.Subset.antisymm (V.sub_master hXmem) hmX
      subst hXmaster
      exact z.master_mem
```

```
    | succ k ih =>
      intro X hn
      obtain <W, hW, hWX> := hn
      exact hz X <W, ih W hW, hWX>
  rintro X <n, hn>
  exact key n X hn

end ApproximableMap

open ApproximableMap

/-- On a finite element `F`, `fix` returns `fix(F)` (the least fixed point of the least
    map of
`F`). -/
theorem fixMap_toElementMap_principal (V : NeighborhoodSystem a)
    {W : Set (ApproximableMap V V)} (hW : (funSpace V V).mem W) :
    (fixMap V).toElementMap ((funSpace V V).principal hW) =
      (toApproxMap ((funSpace V V).principal hW)).fixElement :=
  toElementMap_ofMono_principal _ _ W hW

/-- **Theorem 4.2 (Scott 1981, PRG-19) -- Scott's equation (?).** The elementwise
    action of `fix` is
the least fixed point of the corresponding map: `fix.toElementMap phi = fix(toApproxMap
    phi)`. The
forward inclusion (` subseteq  x`) is `exists_principal_iterMap`; the reverse is
    monotonicity of `fix` along
`W  sqsub  toApproxMap phi`. -/
theorem fixMap_toElementMap (V : NeighborhoodSystem a) (phi : (funSpace V V).Element)
    :
    (fixMap V).toElementMap phi = (toApproxMap phi).fixElement := by
  apply Element.ext
  intro X
  rw [toElementMap_mem_iff_principal]
  constructor
  * rintro <W, hw, hmem>
    rw [fixMap_toElementMap_principal] at hmem
    have hle : (funSpace V V).principal (phi.sub hw)  <=  phi :=
      fun Z hZ => phi.up_mem hw hZ.1 hZ.2
    exact fixElement_mono ((funSpaceEquiv V V).monotone hle) X hmem
  * rintro <n, hn>
    obtain <W, hw, hWn> := exists_principal_iterMap V phi n X hn
    refine <W, hw, ?_>
    rw [fixMap_toElementMap_principal]
    exact <n, hWn>

/-- **Theorem 4.2(ii) (Scott 1981, PRG-19).** `f(x)  subseteq  x ? fix(f)  subseteq  x
    `: `fix` lands in the least
pre-fixed point. -/
theorem fixMap_least (V : NeighborhoodSystem a) (phi : (funSpace V V).Element) {z : V.
    Element}
    (hz : (toApproxMap phi).toElementMap z  <=  z) : (fixMap V).toElementMap phi  <=
    z := by
  rw [fixMap_toElementMap]
  exact fixElement_le_of_toElementMap_le (toApproxMap phi) hz

/-- `fix` applied to (the filter of) an approximable map `f` returns the least fixed
```

```
    point of `f`.
This is the bridge to the "for any `f : ?  ->  ?`" form of Theorem 4.2, using the
    Theorem 3.10
isomorphism `toApproxMap (toFilter f) = f`. -/
theorem fixMap_toElementMap_toFilter (V : NeighborhoodSystem a) (f : ApproximableMap V
     V) :
    (fixMap V).toElementMap (toFilter f) = f.fixElement := by
  rw [fixMap_toElementMap]
  have h : toApproxMap (toFilter f) = f := by
    have he := (funSpaceEquiv V V).apply_symm_apply f
    rwa [funSpaceEquiv_apply, funSpaceEquiv_symm_apply] at he
  rw [h]

end

section
variable {a : Type*} {V : NeighborhoodSystem a}

namespace ApproximableMap

/-- `f?(False)` agrees with the iterated elementwise function `(f(*))^[n] False` --
    Scott's `f?(False)`. -/
theorem iterElem_eq_iterate (f : ApproximableMap V V) (n : Nat) :
    f.iterElem n = (f.toElementMap)^[n] V.bot := by
  induction n with
  | zero =>
    show (f.iterMap 0).toElementMap V.bot = V.bot
    exact toElementMap_idMap V.bot
  | succ k ih =>
    have hstep : f.iterElem (k + 1) = f.toElementMap (f.iterElem k) := by
      show (f.comp (f.iterMap k)).toElementMap V.bot
          = f.toElementMap ((f.iterMap k).toElementMap V.bot)
      rw [toElementMap_comp]
    rw [hstep, ih, Function.iterate_succ', Function.comp_apply]

/-- `Y  in  f?(False)  <->  Delta f? Y`: the `n`-th approximant is the family of
    neighbourhoods reachable from
`Delta` in exactly the `n` steps recorded by `f?`. -/
theorem mem_iterElem (f : ApproximableMap V V) (n : Nat) {X : Set a} :
    (f.iterElem n).mem X  <->  (f.iterMap n).rel V.master X := by
  constructor
  * rintro <W, hW, hWX>
    rw [mem_bot] at hW; subst hW; exact hWX
  * intro h; exact <V.master, by rw [mem_bot], h>

/-- The approximants form an increasing chain: `n  <=  m ? f?(False)  sqsub  f?(False)
    `. -/
theorem iterElem_mono (f : ApproximableMap V V) {n m : Nat} (hnm : n  <=  m) :
    f.iterElem n  <=  f.iterElem m := by
  intro X hX
  rw [mem_iterElem] at hX  |-
  exact rel_master_mono f hnm hX

/-- **Theorem 4.2(iii) (Scott 1981, PRG-19).** `fix(f) =  sqsup _n f?(False)`, here as
    the directed union of
the increasing chain of approximants. -/
```

```
theorem fixElement_eq_iSupDirected (f : ApproximableMap V V) :
    f.fixElement =
      NeighborhoodSystem.iSupDirected (f.iterElem)
        (fun i j => <max i j, iterElem_mono f (le_max_left i j),
          iterElem_mono f (le_max_right i j)>) := by
  apply Element.ext
  intro X
  rw [NeighborhoodSystem.mem_iSupDirected]
  constructor
  * rintro <n, hn>; exact <n, (mem_iterElem f n).mpr hn>
  * rintro <n, hn>; exact <n, (mem_iterElem f n).mp hn>

end ApproximableMap

open ApproximableMap

/-- **Theorem 4.2(iii) (Scott 1981, PRG-19).** `fix(f) =  sqsup _n f?(False)` (as a
    directed union). -/
theorem fixMap_eq_iSup (V : NeighborhoodSystem a) (phi : (funSpace V V).Element) :
    (fixMap V).toElementMap phi =
      NeighborhoodSystem.iSupDirected ((toApproxMap phi).iterElem)
        (fun i j => <max i j, iterElem_mono _ (le_max_left i j),
          iterElem_mono _ (le_max_right i j)>) := by
  rw [fixMap_toElementMap]
  exact fixElement_eq_iSupDirected (toApproxMap phi)

end

section
variable {a : Type*} {V : NeighborhoodSystem a}

namespace ApproximableMap

/-- **Theorem 4.1 (Scott 1981, PRG-19).** `fixElement f` is a *fixed point*: `f(x) = x
    `.
`f(x)  subseteq  x` appends an `f`-step (`Delta f^{n+1} X` from `Delta f? X' f X`); `x
    subseteq  f(x)` reads off the last
step of the chain (the empty chain forces `X = Delta`, handled by `master_mem`/`
    master_rel`). -/
theorem toElementMap_fixElement (f : ApproximableMap V V) :
    f.toElementMap f.fixElement = f.fixElement := by
  apply Element.ext
  intro Y
  constructor
  * rintro <X, <n, hn>, hXY>
    exact <n + 1, <X, hn, hXY>>
  * rintro <n, hn>
    cases n with
    | zero =>
      obtain <_, hYmem, hmY> := hn
      have hYmaster : Y = V.master := Set.Subset.antisymm (V.sub_master hYmem) hmY
      subst hYmaster
      exact <V.master, f.fixElement.master_mem, f.master_rel>
    | succ k =>
      obtain <Z, hZ, hZY> := hn
      exact <Z, <k, hZ>, hZY>
```

```
end ApproximableMap
end

end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.ApproximableMap
section
set_option linter.unusedVariables false

theorem theorem_41.{u_1} : And
  (forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
    (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V)
    {z : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V},
    @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
        (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
     a V)
          (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.Element a V)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    instPartialOrderElement a V)))
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a V V f z)
    z  ->
      @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
        (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
     a V)
          (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.Element a V)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    instPartialOrderElement a V)))
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixElement a V f) z)
  (forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
    (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V),
    @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a V V f
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixElement a V f))
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixElement a V f)) :=
  @And.intro
  (forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
    (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V)
    {z : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V},
    @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
        (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
     a V)
          (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.Element a V)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    instPartialOrderElement a V)))
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a V V f z)
    z  ->
      @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
        (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
     a V)
          (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.Element a V)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    instPartialOrderElement a V)))
```

```
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixElement a V f) z)
  (forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
    (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V),
    @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a V V f
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixElement a V f))
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixElement a V f))
  @DomainTheory.NeighbourhoodSystems.ApproximableMap.fixElement_le_of_toElementMap_le
  @DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap_fixElement
end

end DomainTheory.NeighbourhoodSystems.ApproximableMap
end
```

3.5.4 Example 4.3 — partial functions on integers

EXAMPLE 4.3. The infinite generalization of our original example 1.2 is the system

$$N = \{\{n\} \mid n \in \mathbf{N}\} \cup \{\mathbf{N}\}.$$

The total elements are clearly in a one-one correspondence with the integers in $\mathbf{N}$. We can apply the construction of Exercise 3.16 to obtain a domain

$$F = N^{\infty}.$$

So we already know quite a bit about this domain — but it has a much more familiar presentation.

Let Φ be the set of all *finite partial functions* $\varphi \subseteq \mathbf{N} \times \mathbf{N}$ (that is, finite sets of ordered pairs of integers where, if $(n, m) \in \varphi$ and $(n, m') \in \varphi$, then $m = m'$). Define

$$\uparrow \varphi = \{\psi \in \Phi \mid \varphi \subseteq \psi\}.$$

Consider the neighbourhood system

$$F' = \{\uparrow \varphi \mid \varphi \in \Phi\}.$$

It is an easy exercise to show that F and F' are isomorphic and that the elements of these domains correspond exactly to the (possibly infinite) *partial functions* $\pi \subseteq \mathbf{N} \times \mathbf{N}$. Moreover, the *total* elements just correspond to the *total* functions $\tau : \mathbf{N} \to \mathbf{N}$ ("function" in the ordinary, set-theoretical sense of the word).

Another easy exercise is to show that the domains

$$F \text{ and } (\mathbf{N} \to \mathbf{N})$$

by our definitions are *NOT* isomorphic; though the two domains are closely related. We can define a mapping

$$\text{val} : F \times N \to N$$

by the relationship

$$\uparrow \phi \cup \{n\} \text{ val } \{m\} \quad \text{iff} \quad (n, m) \in \phi.$$

(Of course val has to relate other neighbourhoods such as:

$$\uparrow \phi \cup N \text{ val } N,$$

but these are all.) It is then simple to prove that if $\pi \in |F|$ is regarded as a partial function $\pi : N \to N$ and if for $n \in N$ we define $\hat{n} \in |N|$ by

$$\hat{n} = \{\{n\}, N\},$$

then we have

$$\text{val}(\pi, \hat{n}) = \widehat{\pi(n)}, \text{ if } \pi \text{ is defined at } n;$$

$$= \{N\}, \text{ otherwise.}$$

(Remember that $\{N\} \in |N|$ is the "undefined" element.) This means that

$$\text{curry}(\text{val}) : F \to (N \to N)$$

is a one-one function on elements. (The rather slight trouble with $(N \to N)$ is that it has *more* elements than F.)

So much for the construction of F, we now wish to consider mappings

$$f : F \to F$$

and their uses. Consider the possibility

$$f(\pi)(n) = 0, \qquad \text{if } n = 0;$$

$$= \pi(n-1) + n - 1, \text{ if } n > 0.$$

If π were a total function, then $f(\pi)$ would be total. But if π is partial, and if it is, say, undefined at k, then $f(\pi)$ becomes undefined at $k+1$. Note that $f(\pi)$ is always defined at 0. Note, too, that f is an approximable mapping because it is completely determined by what it does to finite (partial) functions. Indeed,

$$f(\pi) = \bigcup \{f(\phi) \mid \phi \subseteq \pi\},$$

where ϕ ranges over Φ.

Well, we have proved that every approximable map of a domain into itself has a (*least*) fixed point. What is the least fixed point of this f? Suppose $\sigma = f(\sigma)$. Then $\sigma(0) = 0$, and

$$\sigma(n+1) = f(\sigma)(n+1)$$

$$= \sigma(n) + n.$$

By induction, then

$$\sigma(n) = \sum_{i<n} i$$

and σ is a total function. (Therefore, f has a *unique* fixed point.)

Actually, we can make the procedure more systematic by defining as fixed points elements of $(N \to N)$ rather than F. In the first place we have $\hat{0} \in |N|$, and from now on we will not distinguish between n and $\hat{n}$. Next we have two mappings:

$$\mathrm{succ}, \mathrm{pred} : N \to N$$

where, as approximable mappings we have

$$X \ \mathrm{succ}\ Y \text{ iff } \exists n \in N.\ n \in X \text{ and } n+1 \in Y,$$

$$X \ \mathrm{pred}\ Y \text{ iff } \exists n \in N.\ n+1 \in X \text{ and } n \in Y,$$

for all $X, Y \in N$. This is *correct*, but what we mean in more understandable terms is:

$$\mathrm{succ}(n) = n+1;$$

$$\mathrm{pred}(n) = n-1, \text{ if } n > 0;$$

$$= \perp, \text{ if } n = 0.$$

Here, n has been identified with $\hat{n} \in |N|$ and $\perp = \{N\} \in |N|$. Moreover, we have a mapping

$$\mathrm{zero} : N \to T$$

which is such that

$$\mathrm{zero}(n) = \text{true, if } n = 0;$$

$$= \text{false, if } n > 0.$$

The *structured domain*

$$\langle N, 0, \mathrm{succ}, \mathrm{pred}, \mathrm{zero} \rangle$$

can be called "THE domain of integers" for our present theory. We shall meet many other structured domains in the sequel.

Now the iterated summation function σ can be completely characterized — as a map $\sigma : N \to N$ rather than as an element $\sigma \in |F|$ — by the following equation:

$$\sigma(n) = \mathrm{cond}(\mathrm{zero}(n),\ 0,\ \sigma(\mathrm{pred}(n)) + \mathrm{pred}(n)).$$

The only problem is that we have not defined $+ : N \times N \to N$. (A direct definition is left to the reader; general remarks are given later.) But $+$ could be any function of two variables in order to make the point about the form of the definition of σ. Remember

$$\mathrm{cond} : T \times N \times N \to N,$$

as defined in Exercise 3.26. We do not put cond in as part of the structure of N because (as should be clear from 3.26) it is part of the structure of T.

The above equation for σ is properly called a *functional equation*; it will be written as a fixed-point equation in Lecture V when we have the notation for the λ-calculus. □

Example_4_3.lean

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Example_2_3

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.Example43
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap

section
/-- Membership in Scott's natural-number system: a neighbourhood is the whole space `
    Nat` or a
singleton `{n}`. -/
def memN (X : Set Nat) : Prop := X = Set.univ  \/  exists  n, X = {n}
end

section
theorem memN_singleton (n : Nat) : memN ({n} : Set Nat) := Or.inr <n, rfl>
end

section
theorem memN_univ : memN (Set.univ : Set Nat) := Or.inl rfl
end

section
/-- Any two neighbourhoods of `N` are nested or disjoint: `Nat` contains every
    singleton, and two
distinct singletons are disjoint. -/
theorem nestedOrDisjoint : NestedOrDisjoint memN := by
  rintro X Y (rfl | <n, rfl>) (rfl | <m, rfl>)
  * exact Or.inl (Set.Subset.refl _)
  * exact Or.inr (Or.inl (Set.subset_univ _))
  * exact Or.inl (Set.subset_univ _)
  * by_cases h : n = m
    * subst h; exact Or.inl (Set.Subset.refl _)
    * refine Or.inr (Or.inr ?_)
      ext k
      simp only [Set.mem_inter_iff, Set.mem_singleton_iff, Set.mem_empty_iff_false,
    iff_false,
        not_and]
      rintro rfl hkm
      exact h hkm
end

end DomainTheory.NeighbourhoodSystems.Example43
```

```
section
namespace DomainTheory.NeighbourhoodSystems.Example43
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap
section
/- concordance: source L1795 EXAMPLE 4.3 partial functions on integers -/
/-- **Example 4.3 (Scott 1981, PRG-19).** The natural-number neighbourhood system `N`
    on `Delta = Nat`. -/
abbrev N : NeighborhoodSystem Nat :=
  NeighborhoodSystem.ofNestedOrDisjoint memN Set.univ Set.univ_nonempty memN_univ
    nestedOrDisjoint
    (fun _ => Set.subset_univ _)
end

section
@[simp] theorem N_mem {X : Set Nat} : N.mem X  <->  memN X := Iff.rfl
end

section
@[simp] theorem N_master : N.master = (Set.univ : Set Nat) := rfl
end

section
/-- The two-token truth domain `T` of Example 1.2 (= `Example23.T`). -/
abbrev T : NeighborhoodSystem Example12.Token := Example23.T
end

section
/-- Scott's total element `n = {n} = {{n}, Nat}`, the principal filter of the singleton
     `{n}`. -/
def natElem (n : Nat) : N.Element := N.principal (N_mem.mpr (memN_singleton n))
end

section
/-- The value table for `pred`: `pred(0) = False`, `pred(k+1) = k`. -/
def predVal : Nat  ->  N.Element
  | 0 => N.bot
  | (k + 1) => natElem k
end

section
/-- Singletons of naturals are one-one as sets. -/
theorem singleton_nat_inj {n m : Nat} (h : ({n} : Set Nat) = {m}) : n = m := by
  have h1 : n  in  ({n} : Set Nat) := rfl
  rw [h] at h1
  exact h1
end

section
/-- `Nat` is not a singleton (a witness `n+1  !=  n`). -/
theorem univ_ne_singleton (n : Nat) : (Set.univ : Set Nat)  !=  {n} := by
  intro h
  have h1 : (n + 1)  in  ({n} : Set Nat) := h  |>  Set.mem_univ (n + 1)
  exact Nat.succ_ne_self n h1
end

section
```

```
/-- `{n}  !=  Nat` (the symmetric form). -/
theorem singleton_ne_univ (n : Nat) : ({n} : Set Nat)  !=  Set.univ := fun h =>
    univ_ne_singleton n h.symm
end

section
/-- The *strict* approximable map `N  ->  V` determined by a choice of value `val n  in
      |V|` for each
`n`: it sends the total element `n` to `val n` and the bottom `False` to `False`. The
    relation
`X f Y` holds when either `X = Delta_N` forces the blunt output `Y = Delta_V` (the
    strict `False  |->  False` clause),
or `X = {n}` and `Y` is a neighbourhood of `val n`.

Definition 2.1 checks uniformly: (i) `Delta_N f Delta_V`; (ii) intersect outputs --
    distinct inputs are
impossible (`Nat  !=  {n}`, singletons are one-one), so both outputs come from the *
    same* `val n` and
`inter_mem` of the filter `val n` applies; (iii) sharpening `{n}  subseteq  X` keeps
    the input a singleton
`{n}`, and `up_mem` of `val n` widens the output, while a blunt input `Delta_N` forces `
    Delta_V`. -/
def constLiftN {b : Type*} (V : NeighborhoodSystem b) (val : Nat  ->  V.Element) :
    ApproximableMap N V where
  rel X Y := (X = Set.univ  /\  Y = V.master)  \/  (exists  n, X = {n}  /\  (val n).
    mem Y)
  rel_dom := by
    rintro X Y (<rfl, _> | <n, rfl, _>)
    * exact memN_univ
    * exact memN_singleton n
  rel_cod := by
    rintro X Y (<_, rfl> | <n, _, hY>)
    * exact V.master_mem
    * exact (val n).sub hY
  master_rel := Or.inl <rfl, rfl>
  inter_right := by
    rintro X Y Y' (<rfl, rfl> | <n, rfl, hY>) hc'
    * rcases hc' with <_, rfl> | <n, hX, _>
      * exact Or.inl <rfl, by rw [Set.inter_self]>
      * exact (univ_ne_singleton n hX).elim
    * rcases hc' with <hX, _> | <n', hX', hY'>
      * exact (singleton_ne_univ n hX).elim
      * have hnn' : n = n' := singleton_nat_inj hX'
        subst hnn'
        exact Or.inr <n, rfl, (val n).inter_mem hY hY'>
  mono := by
    rintro X X' Y Y' (<rfl, rfl> | <n, rfl, hY>) hX'X hYY' hX' hY'
    * have hY'master : Y' = V.master := Set.Subset.antisymm (V.sub_master hY') hYY'
      subst hY'master
      rcases hX' with rfl | <k, rfl>
      * exact Or.inl <rfl, rfl>
      * exact Or.inr <k, rfl, (val k).master_mem>
    * rcases hX' with rfl | <k, rfl>
      * exact (singleton_ne_univ n (Set.univ_subset_iff.mp hX'X)).elim
      * have hkn : k = n := Set.mem_singleton_iff.mp (Set.singleton_subset_iff.mp hX'X
    )
```

```
        subst hkn
        exact Or.inr <k, rfl, (val k).up_mem hY hY' hYY'>
end

section
/-- **Example 4.3 -- `pred`.** `pred(0) = False`, `pred(n+1) = n`, strict (`pred(False)
    = False`). -/
def predMap : ApproximableMap N N := constLiftN N predVal
end

section
/-- **Example 4.3 -- `succ`.** `succ(n) = n + 1`, strict (`succ(False) = False`). -/
def succMap : ApproximableMap N N := constLiftN N (fun n => natElem (n + 1))
end

section
/-- Scott's `0  in  |N|`, the distinguished zero of the structured domain. -/
def zeroElt : N.Element := natElem 0
end

section
/-- The value table for `zero`: `zero(0) = true`, `zero(n+1) = false`. -/
def zeroVal : Nat  ->  T.Element
  | 0 => Example23.trueElt
  | (_ + 1) => Example23.falseElt
end

section
/-- **Example 4.3 -- `zero`.** `zero(0) = true`, `zero(n+1) = false`, strict (`zero(
    False) = False`). -/
def zeroMap : ApproximableMap N T := constLiftN T zeroVal
end

section
/-- `False  in  N` reads: a neighbourhood lies in `False` iff it is the whole space `
    Nat`. -/
theorem N_bot_mem {X : Set Nat} : N.bot.mem X  <->  X = Set.univ := NeighborhoodSystem
    .mem_bot N
end

section
/-- **Computation on bottom (strictness).** `f(False) = False`. -/
theorem constLiftN_bot {b : Type*} (V : NeighborhoodSystem b) (val : Nat  ->  V.
    Element) :
    (constLiftN V val).toElementMap N.bot = V.bot := by
  apply Element.ext
  intro Y
  constructor
  * rintro <X, hXmem, hrel>
    rw [N_bot_mem] at hXmem
    subst hXmem
    rcases hrel with <_, rfl> | <k, hXk, _>
    * exact V.mem_bot.mpr rfl
    * exact (univ_ne_singleton k hXk).elim
  * intro hY
    rw [V.mem_bot] at hY
```

```
    subst hY
    exact <Set.univ, N_bot_mem.mpr rfl, Or.inl <rfl, rfl>>
end

section
/-- A neighbourhood belongs to `n` iff it is `Nat` (the master) or the singleton `{n}`.
    -/
theorem mem_natElem_iff {n : Nat} {Y : Set Nat} :
    (natElem n).mem Y  <->  Y = Set.univ  \/  Y = {n} := by
  unfold natElem
  rw [mem_principal]
  constructor
  * rintro <hY, hsub>
    rcases hY with rfl | <m, rfl>
    * exact Or.inl rfl
    * have hnm : n = m := Set.mem_singleton_iff.mp (Set.singleton_subset_iff.mp hsub)
      subst hnm; exact Or.inr rfl
  * rintro (rfl | rfl)
    * exact <memN_univ, Set.subset_univ _>
    * exact <memN_singleton n, subset_rfl>
end

section
/-- **Computation on the total elements.** `f(n) = val n`. -/
theorem constLiftN_natElem {b : Type*} (V : NeighborhoodSystem b) (val : Nat  ->  V.
    Element) (n : Nat) :
    (constLiftN V val).toElementMap (natElem n) = val n := by
  apply Element.ext
  intro Y
  constructor
  * rintro <X, hXmem, hrel>
    rw [mem_natElem_iff] at hXmem
    rcases hrel with <_, rfl> | <k, hXk, hY>
    * exact (val n).master_mem
    * rcases hXmem with rfl | rfl
      * exact (univ_ne_singleton k hXk).elim
      * have hnk : n = k := singleton_nat_inj hXk
        subst hnk; exact hY
  * intro hY
    exact <{n}, mem_natElem_iff.mpr (Or.inr rfl), Or.inr <n, rfl, hY>>
end

section
/-- `pred(False) = False`. -/
theorem predMap_bot : predMap.toElementMap N.bot = N.bot :=
  constLiftN_bot N predVal
end

section
/-- `pred((n+1)^) = n`. -/
theorem predMap_natElem_succ (k : Nat) : predMap.toElementMap (natElem (k + 1)) =
    natElem k :=
  constLiftN_natElem N predVal (k + 1)
end

section
```

```
/-- `pred(0) = False`: predecessor of zero is undefined. -/
theorem predMap_natElem_zero : predMap.toElementMap (natElem 0) = N.bot :=
  constLiftN_natElem N predVal 0
end

section
/-- `succ(False) = False`. -/
theorem succMap_bot : succMap.toElementMap N.bot = N.bot :=
  constLiftN_bot N (fun n => natElem (n + 1))
end

section
/-- `succ(n) = (n+1)^`. -/
theorem succMap_natElem (n : Nat) : succMap.toElementMap (natElem n) = natElem (n + 1)
     :=
  constLiftN_natElem N (fun n => natElem (n + 1)) n
end

section
/-- `pred(succ(n)) = n`: the predecessor undoes the successor on total elements. (A
    sample of the
"recursion" reasoning Scott highlights; cf. the iterated-summation example.) -/
theorem predMap_succMap_natElem (n : Nat) :
    predMap.toElementMap (succMap.toElementMap (natElem n)) = natElem n := by
  rw [succMap_natElem, predMap_natElem_succ]
end

section
/-- `zero(False) = False`. -/
theorem zeroMap_bot : zeroMap.toElementMap N.bot = T.bot :=
  constLiftN_bot T zeroVal
end

section
/-- `zero((n+1)^) = false`. -/
theorem zeroMap_natElem_succ (n : Nat) : zeroMap.toElementMap (natElem (n + 1)) =
    Example23.falseElt :=
  constLiftN_natElem T zeroVal (n + 1)
end

section
/-- `zero(0) = true`. -/
theorem zeroMap_natElem_zero : zeroMap.toElementMap (natElem 0) = Example23.trueElt :=
  constLiftN_natElem T zeroVal 0
end

end DomainTheory.NeighbourhoodSystems.Example43
end
```

3.5.5 Example 4.4 — sequence domain

EXAMPLE 4.4. The domain C of finite or infinite binary sequences mentioned in Exercise 2.21 may be regarded as a generalization of N. This can be made plain by saying how we wish to regard C as a structured domain. To do this we should recall what C is as a neighbourhood system. In the first place

$$B = \{\sigma\Sigma^* \mid \sigma \in \Sigma^*\}$$

where $\Sigma = \{0, 1\}$. To form the system C we have

$$C = B \cup \{\{\sigma\} \mid \sigma \in \Sigma^*\}.$$

The total elements of B correspond to *infinite binary sequences*; while the total elements of C to *finite or infinite sequences*. To simplify notation let us write for $\sigma \in \Sigma^*$

$$\sigma =\uparrow \{\sigma\} \quad \text{(a total element);}$$

$$\sigma \perp =\uparrow \sigma\Sigma^* \quad \text{(a partial element).}$$

In other words we identify σ with the corresponding total element in $|C|$.

We wish now to think of C as a structured domain seen as a kind of generalization of N. The empty sequence Λ will play the rôle of $0 \in |N|$; the map succ has two different analogues for C, however. Just as for B we define for $x \in |C|$ and $\sigma \in \Sigma^*$:

$$\sigma x = \{Y \mid \sigma X \subseteq Y \text{ some } X \in x\},$$

where of course now X and Y range over C. It should be checked that $\sigma\tau$ has the right meaning whether we think of $\tau \in \Sigma^*$ or $\tau \in |C|$. The two "successor" mappings we are looking for are

$$x \mapsto 0x \quad \text{and} \quad x \mapsto 1x.$$

All the maps $x \mapsto \sigma x$ can be obtained as compositions of these iterated as many times as needed.

Here are two questions which we now should ask:

What plays the rôle of pred?

The mapping will be called **tail**, and it is characterized by:

$$\text{tail}(0x) = x,$$

$$\text{tail}(1x) = x, \text{ and}$$

$$\text{tail}(\Lambda) =\perp .$$

It is left to the reader to show that **tail** exists as an approximable mapping.

What plays the rôle of zero?

The answer is not unique, because in C there are several distinctions that have to be made; in fact we will define three maps:

$$\text{empty}, \text{zero}, \text{one} : C \to T$$

where the three maps take on truth-values to distinguish various kinds of elements in $|C|$ as follows:

$$\begin{aligned}
\text{empty}\,(\Lambda) &= \text{true},\\
\text{empty}\,(0x) &= \text{false},\\
\text{empty}\,(1x) &= \text{false},\\
\text{zero}\,(\Lambda) &= \text{false}\\
\text{zero}\,(0x) &= \text{true}\\
\text{zero}\,(1x) &= \text{false}\\
\text{one}\,(\Lambda) &= \text{false}\\
\text{one}\,(0x) &= \text{false}\\
\text{one}\,(1x) &= \text{true}.
\end{aligned}$$

Again, it is an exercise to show these are approximable. The structured domain is therefore

$$\langle C, \Lambda, 0, 1, \text{tail}, \text{empty}, \text{zero}, \text{one}\rangle.$$

Note that we have changed the meaning of some of the symbols in passing from N to C. Note too that there is a confusion between 0 as an element and 0 as the map $x \mapsto 0x$. There are just too few symbols! In any case this is only an example and not a philosophy of life, so the reader can be expected not to suffer too much.

An example of a definition of an *element* of $|C|$ by a fixed-point equation is:

$$a = 0\,1\,a.$$

This equation has one and only one solution in $|C|$, the infinite sequence that alternates 0's and 1's. Note that a is also characterized by:

$$a = 0101a.$$

Another element is

$$b = 010\,b,$$

which is quite different from a.

An example of a *map* in $|C \to C|$ has the characterization

$$\begin{aligned}
d(\Lambda) &= \Lambda\\
d(0x) &= 00d(x), \quad \text{and}\\
d(1x) &= 11d(x).
\end{aligned}$$

We can write:

$$\begin{aligned}
d(x) = \text{cond}(&\text{empty}(x),\ \Lambda,\\
&\text{cond}(\text{zero}(x),\ 00d(\text{tail}(x)),\ 11d(\text{tail}(x)))).
\end{aligned}$$

As we shall see in due course, this can be regarded as a fixed-point definition of d.

An example of a map in $|C \times C \to C|$ was suggested in 2.21. We can write:

$$xy = \mathrm{cond}(\mathrm{empty}(x),\, y,\\ \mathrm{cond}(\mathrm{zero}(x),\, 0(\mathrm{tail}(x)\, y),\, 1(\mathrm{tail}(x)\, y))).$$

It should be checked that this equation exactly characterizes the intended mapping. □

The examples we have given with N and C are examples of definitions of functions by *recursion*. The literal meaning of "recursion" is "running backwards", and a look at the equations for our examples will show that the functions are characterized by giving their values either *outright* (e.g. at 0 or at Λ) or at *earlier* arguments (e.g. at $\mathrm{pred}(x)$ or at $\mathrm{tail}(x)$). The reader should keep in mind that a recursive "definition" is not really a definition in the sense of *explicit definition* but rather is a characterization; a theorem has to be proved to show that such functions exist. Now we have a general definition of domain and a general theorem on fixed points and a general construction of function-space domain; THEREFORE, we know that there are solutions to our equations PROVIDED THAT the variables range over elements of a domain and that the other, given functions that appear in the equations are already known to be approximable (continuous). This proviso is very important, and we shall remark on it time after time.

But, as is well known, recursion also can be done over sets like $\mathbf{N}$, and we should examine now the connection between the familiar kind of recursion and what we are doing over domains. Of course, one simple connection is already provided by the way we regard $\mathbf{N}$ as a subset of N. But there are other useful connections that can be employed in a way that may seem more direct.

Example_4_4.lean

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Exercise_2_11
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Theorem_4_2
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Theorem_4_1

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.Example44
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap ExampleB

section
/-- Membership in `C`: a neighbourhood is a cone `sigmaSigma*` or a singleton `{sigma
    }`. -/
def memC (X : Set Str) : Prop := (exists  sigma, X = cone sigma)  \/  (exists  sigma,
    X = {sigma})
end

section
theorem memC_cone (sigma : Str) : memC (cone sigma) := Or.inl <sigma, rfl>
end

section
theorem memC_singleton (sigma : Str) : memC ({sigma} : Set Str) := Or.inr <sigma, rfl>
end

section
/-- Prepending is monotone in its set argument. -/
```

```
theorem prepend_mono (sigma : Str) {X X' : Set Str} (h : X'  subseteq  X) : prepend
    sigma X'  subseteq  prepend sigma X := by
  rintro w <tau, htau, rfl>
  exact <tau, h htau, rfl>
end

section
/-- `sigma{tau} = {sigmatau}`: prepending `sigma` to a singleton is the singleton of
    the concatenation. -/
theorem prepend_singleton (sigma tau : Str) : prepend sigma {tau} = {sigma ++ tau} :=
    by
  ext w
  simp only [mem_prepend, Set.mem_singleton_iff]
  constructor
  * rintro <t, rfl, rfl>; rfl
  * rintro rfl; exact <tau, rfl, rfl>
end

section
/-- `{tau}  subseteq  sigmaSigma*` iff `sigma` is an initial segment of `tau`. -/
theorem singleton_subset_cone {sigma tau : Str} : ({tau} : Set Str)  subseteq  cone
    sigma  <->  sigma <+: tau := by
  rw [Set.singleton_subset_iff, mem_cone]
end

section
/-- A singleton and a cone are nested-or-disjoint. -/
theorem singleton_cone_nd (sigma tau : Str) :
    ({tau} : Set Str)  subseteq  cone sigma  \/  cone sigma  subseteq  {tau}  \/  ({
    tau} : Set Str)  I  cone sigma = {} := by
  by_cases h : sigma <+: tau
  * exact Or.inl (singleton_subset_cone.mpr h)
  * refine Or.inr (Or.inr ?_)
    ext w
    simp only [Set.mem_inter_iff, Set.mem_singleton_iff, mem_cone, Set.
    mem_empty_iff_false,
      iff_false, not_and]
    rintro rfl htau
    exact h htau
end

section
/-- Any two neighbourhoods of `C` are nested or disjoint. -/
theorem nestedOrDisjoint : NestedOrDisjoint memC := by
  rintro X Y (<sigma, rfl> | <sigma, rfl>) (<tau, rfl> | <tau, rfl>)
  * exact cone_trichotomy sigma tau
  * rcases singleton_cone_nd sigma tau with h | h | h
    * exact Or.inr (Or.inl h)
    * exact Or.inl h
    * exact Or.inr (Or.inr (by rw [Set.inter_comm]; exact h))
  * rcases singleton_cone_nd tau sigma with h | h | h
    * exact Or.inl h
    * exact Or.inr (Or.inl h)
    * exact Or.inr (Or.inr h)
  * by_cases h : sigma = tau
    * subst h; exact Or.inl (Set.Subset.refl _)
```

```
    * refine Or.inr (Or.inr ?_)
      ext w
      simp only [Set.mem_inter_iff, Set.mem_singleton_iff, Set.mem_empty_iff_false,
    iff_false,
        not_and]
      rintro rfl h2
      exact h h2
end

end DomainTheory.NeighbourhoodSystems.Example44

section
namespace DomainTheory.NeighbourhoodSystems.Example44
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap ExampleB
section
/- concordance: source L1989 EXAMPLE 4.4 sequence domain -/
/-- **Example 4.4 (Scott 1981, PRG-19).** The neighbourhood system `C` of finite or
    infinite binary
sequences on `Delta = Sigma*`. -/
abbrev C : NeighborhoodSystem Str :=
  NeighborhoodSystem.ofNestedOrDisjoint memC Set.univ
    <[], Set.mem_univ _> (Or.inl <[], cone_nil.symm>) nestedOrDisjoint
    (fun _ => Set.subset_univ _)
end

section
@[simp] theorem C_mem {X : Set Str} : C.mem X  <->  memC X := Iff.rfl
end

section
@[simp] theorem C_master : C.master = (Set.univ : Set Str) := rfl
end

section
/-- Prepending the bit `b` (or, generally, a prefix `sigma`) to a neighbourhood of `C`
    lands back in
`C`: `sigma(tauSigma*) = (sigmatau)Sigma*` and `sigma{tau} = {sigmatau}`. -/
theorem memC_prepend (sigma : Str) {X : Set Str} (hX : memC X) : memC (prepend sigma X
    ) := by
  rcases hX with <?, rfl> | <?, rfl>
  * exact Or.inl <sigma ++ ?, prepend_cone sigma ?>
  * exact Or.inr <sigma ++ ?, prepend_singleton sigma ?>
end

section
/-- **Example 4.4 -- the successors `x  |->  bx`.** The approximable map prepending the
     bit `b`:
`X (bx) Y  <->  bX  subseteq  Y`. Approximable because `bX` is again a neighbourhood (`
    memC_prepend`) and
prepending is monotone. -/
def consMap (b : Bool) : ApproximableMap C C where
  rel X Y := memC X  /\  memC Y  /\  prepend [b] X  subseteq  Y
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := by
    refine <Or.inl <[], cone_nil.symm>, Or.inl <[], cone_nil.symm>, ?_>
```

```
    exact Set.subset_univ _
  inter_right := by
    rintro X Y Y' <hX, hY, hsub> <_, hY', hsub'>
    have hsubInter : prepend [b] X  subseteq  Y  I  Y' := Set.subset_inter hsub hsub'
    have hZ : memC (prepend [b] X) := memC_prepend [b] hX
    exact <hX, C.inter_mem hY hY' hZ hsubInter, hsubInter>
  mono := by
    rintro X X' Y Y' <hX, hY, hsub> hX'X hYY' hX' hY'
    exact <hX', hY', (prepend_mono [b] hX'X).trans (hsub.trans hYY')>
end

section
/-- **Example 4.4 -- an element defined by a fixed-point equation.** Scott's `a = 01a`,
     the infinite
sequence that alternates `0`s and `1`s. We take the least fixed point of `x  |->  0(1x)
    `
(`= consMap 0  comp  consMap 1`), which exists by the Fixed-point Theorem 4.1. -/
def altElt : C.Element := ((consMap false).comp (consMap true)).fixElement
end

section
/-- Scott's partial element `sigmaFalse = sigmaSigma*` ("the sequence starts with `
    sigma`"). -/
def strBot (sigma : Str) : C.Element := C.principal (C_mem.mpr (memC_cone sigma))
end

section
/-- Scott's total element `sigma = {sigma}` (the finite sequence `sigma`, completed).
    -/
def strElem (sigma : Str) : C.Element := C.principal (C_mem.mpr (memC_singleton sigma)
    )
end

section
/-- `a = 0(1a)`: `altElt` satisfies Scott's defining equation. -/
theorem altElt_eq : (consMap false).toElementMap ((consMap true).toElementMap altElt)
    = altElt := by
  have h := toElementMap_fixElement ((consMap false).comp (consMap true))
  rwa [toElementMap_comp] at h
end

section
/-- `bx` on a partial element: `b(sigmaFalse) = (bsigma)False`. -/
theorem consMap_strBot (b : Bool) (sigma : Str) :
    (consMap b).toElementMap (strBot sigma) = strBot (b :: sigma) := by
  apply Element.ext
  intro Y
  constructor
  * rintro <X', <_, hXX'>, _, hY, hsub>
    refine <hY, ?_>
    have hpre : prepend [b] (cone sigma)  subseteq  Y := (prepend_mono [b] hXX').trans
     hsub
    rwa [prepend_cone] at hpre
  * rintro <hY, hsub>
    refine <cone sigma, <memC_cone sigma, subset_rfl>, memC_cone sigma, hY, ?_>
    rw [prepend_cone]; exact hsub
```

```
end

section
/-- `bx` on a total element: `b(sigma) = (bsigma)` (prepend the bit to a finite
    sequence). -/
theorem consMap_strElem (b : Bool) (sigma : Str) :
    (consMap b).toElementMap (strElem sigma) = strElem (b :: sigma) := by
  apply Element.ext
  intro Y
  constructor
  * rintro <X', <_, hXX'>, _, hY, hsub>
    refine <hY, ?_>
    have hpre : prepend [b] {sigma}  subseteq  Y := (prepend_mono [b] hXX').trans hsub
    rwa [prepend_singleton] at hpre
  * rintro <hY, hsub>
    refine <{sigma}, <memC_singleton sigma, subset_rfl>, memC_singleton sigma, hY, ?_>
    rw [prepend_singleton]; exact hsub
end

section
/-- `sigmaX  subseteq  X` along the prefix order: a prepended neighbourhood is
    contained in any neighbourhood it
refines. (Used only via `prepend_mono`; kept implicit.) -/
theorem prepend_subset_self : True := trivial
end

end DomainTheory.NeighbourhoodSystems.Example44
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

section
variable {a : Type*} {V : NeighborhoodSystem a}

namespace ApproximableMap

/-- The chain of approximants from a base point `a`: `iterFrom f a n = f?(a)`. -/
def iterFrom (f : ApproximableMap V V) (a : V.Element) (n : Nat) : V.Element :=
  (f.toElementMap)^[n] a

@[simp] theorem iterFrom_zero (f : ApproximableMap V V) (a : V.Element) :
    f.iterFrom a 0 = a := rfl

theorem iterFrom_succ (f : ApproximableMap V V) (a : V.Element) (n : Nat) :
    f.iterFrom a (n + 1) = f.toElementMap (f.iterFrom a n) := by
  rw [iterFrom, iterFrom, Function.iterate_succ', Function.comp_apply]

/-- One step of the chain: `f?(a)  sqsub  f?^+1(a)` (proved choice-free by induction).
    -/
theorem iterFrom_step (f : ApproximableMap V V) {a : V.Element} (ha : a  <=  f.
    toElementMap a)
    (n : Nat) : f.iterFrom a n  <=  f.iterFrom a (n + 1) := by
  induction n with
  | zero => rw [iterFrom_succ f a 0, iterFrom_zero]; exact ha
  | succ k ih =>
    rw [iterFrom_succ f a k, iterFrom_succ f a (k + 1)]
    exact f.toElementMap_mono ih
```

```
/-- When `a  sqsub  f(a)`, the chain `f?(a)` is monotone: `n  <=  m ? f?(a)  sqsub  f?(
    a)`. Proved by induction
on `n  <=  m` (so as to stay **choice-free**, unlike `monotone_nat_of_le_succ`). -/
theorem iterFrom_mono (f : ApproximableMap V V) {a : V.Element} (ha : a  <=  f.
    toElementMap a)
    {n m : Nat} (hnm : n  <=  m) : f.iterFrom a n  <=  f.iterFrom a m := by
  induction hnm with
  | refl => exact le_refl _
  | step _ ih => exact le_trans ih (iterFrom_step f ha _)

/-- The fixed point of `f` lying above a pre-fixed-point candidate `a` (with `a  sqsub
    f(a)`),
constructed as the directed union ` sqsup _n f?(a)`. -/
def fixAbove (f : ApproximableMap V V) {a : V.Element} (ha : a  <=  f.toElementMap a)
    : V.Element :=
  NeighborhoodSystem.iSupDirected (f.iterFrom a)
    (fun i j => <max i j, iterFrom_mono f ha (le_max_left i j),
      iterFrom_mono f ha (le_max_right i j)>)

end ApproximableMap
end
end DomainTheory.NeighbourhoodSystems
end
```

3.5.6 Definition 4.5 — Peano axioms

DEFINITION 4.5. A structured set $\langle \mathbf{N}, 0, {}^{+} \rangle$, where $0 \in \mathbf{N}$ and ${}^{+} : \mathbf{N} \to \mathbf{N}$ is a unary function, is said to be a *model for Peano's Axioms* if the following conditions are satisfied:

(i) $0 \neq n^{+}$, for all $n \in \mathbf{N}$;

(ii) $n^{+} = m^{+}$ implies $n = m$, for all $n, m \in \mathbf{N}$;

(iii) whenever $x \subseteq \mathbf{N}$ and $0 \in x$ and $x^{+} \subseteq x$, then $x = \mathbf{N}$.

Here $x^{+} = \{n^{+} \mid n \in x\}$. □

Clause (iii) is recognized as the principle of mathematical induction stated in terms of sets. We usually think of $\mathbf{N}$ as being "God given", and (i)–(iii) as known without question. Suppose God, however, decides to withdraw His set of integers and substitute another. We can ask: "Oh! Why did You take from us our beloved numbers? Why must we now live with these strange new beasts?" God will probably reply "Trust Me!" Perhaps we should in view of the theorem:

Definition_4_5.lean

```
module

public import Mathlib.Tactic.Attr.Core
public import Aesop.BuiltinRules
public import Mathlib.Basic.ExistsUnique
public import Mathlib.Data.Set.Defs

@[expose] public section
```

```
section
universe u
namespace DomainTheory.NeighbourhoodSystems
/- concordance: source L2143 DEFINITION 4.5 Peano axioms -/
/-- **Definition 4.5 (Scott 1981, PRG-19).** A *model for Peano's Axioms*: a type `N`
    with a
distinguished element `zero` and a unary `succ`essor, satisfying

* (i)   `zero  !=  succ n` for all `n`                          (`zero_ne_succ`);
* (ii)  `succ` is injective (`n^+ = m^+ ? n = m`)           (`succ_injective`);
* (iii) **mathematical induction**: any `s  subseteq  N` with `zero  in  s` and closed
    under `succ`
        (`forall  n  in  s, succ n  in  s`, i.e. Scott's `x^+  subseteq  x`) is all of
    `N`
        (`forall  n, n  in  s`, i.e. Scott's `x = N`)               (`induction`). -/
structure PeanoModel (N : Type u) where
  /-- The distinguished zero `0  in  N`. -/
  zero : N
  /-- The unary successor `^+ : N  ->  N`. -/
  succ : N  ->  N
  /-- **4.5(i)** `0  !=  n^+`. -/
  zero_ne_succ : forall  n, zero  !=  succ n
  /-- **4.5(ii)** the successor is one-one. -/
  succ_injective : Function.Injective succ
  /-- **4.5(iii)** the induction principle (point form of `x^+  subseteq  x ? x = N`).
    -/
  induction : forall  (s : Set N), zero  in  s  ->  (forall  n  in  s, succ n  in  s)
     ->  forall  n, n  in  s

end DomainTheory.NeighbourhoodSystems
end

section
namespace DomainTheory.NeighbourhoodSystems
section
universe u v

namespace PeanoModel

variable {M : Type u} {N : Type v}

/-- Scott's least fixed point `r  subseteq  N  x  M`, realized as the inductively
    generated relation: the
least relation containing `(0, [] )` and closed under `(n, m)  |->  (n^+, m#)`. The two
     constructors are
Scott's facts (i) `0 r [] ` and (ii) `n r m ? n^+ r m#`. By Theorem 4.1 this *is* the
    least fixed
point of the approximable operator `u  |->  {(0,[] )}  U  {(n^+,m#) ? (n,m)  in  u}` on
     `P(N  x  M)`. -/
inductive Graph (P : PeanoModel M) (Q : PeanoModel N) : M  ->  N  ->  Prop
  | base : Graph P Q P.zero Q.zero
  | step : forall  {m n}, Graph P Q m n  ->  Graph P Q (P.succ m) (Q.succ n)

/-- The construction is symmetric in the two models: `Graph P Q m n` swaps to `Graph Q
    P n m`.
```

```
(This is why the relation is one-one in *both* directions, as Scott notes: "the roles
    of `N` and
`M` are completely symmetric".) -/
theorem Graph.swap {P : PeanoModel M} {Q : PeanoModel N} {m : M} {n : N}
    (h : Graph P Q m n) : Graph Q P n m := by
  induction h with
  | base => exact Graph.base
  | step _ ih => exact Graph.step ih

/-- **Inversion at a successor (uses 4.5(i)/(ii)).** Anything related to `n^+` on the
    left comes
from a relation `n r n_0` with right value a successor: `n^+ r k ? exists  n_0, n r n_0
     /\  k = n_0#`. -/
theorem graph_succ_right (P : PeanoModel M) (Q : PeanoModel N) {m : M} {k : N}
    (h : Graph P Q (P.succ m) k) : exists  n, Graph P Q m n  /\  k = Q.succ n := by
  generalize hz : P.succ m = z at h
  cases h with
  | base => exact absurd hz.symm (P.zero_ne_succ _)
  | step h' =>
    rename_i m0 n0
    have hm : m = m0 := P.succ_injective hz
    subst hm
    exact <n0, h', rfl>

/-- **Inversion at `0` (uses 4.5(i)).** Anything related to `0` on the left is `[] ` on
     the right:
`0 r k ? k = [] `. (The other generating rule would force `0 = n^+`, impossible by 4.5(
    i).) -/
theorem graph_zero_right (P : PeanoModel M) (Q : PeanoModel N) {k : N}
    (h : Graph P Q P.zero k) : k = Q.zero := by
  generalize hz : P.zero = z at h
  cases h with
  | base => rfl
  | step _ => exact absurd hz (P.zero_ne_succ _)

/-- Each element of the left model is related by `Graph` to exactly one element of the
    right
model. Proved by the induction principle 4.5(iii) on the set `{m ? exists ! n, m r n}`:
     the base case
is the `0`-inversion `graph_zero_right`, the step case the successor-inversion `
    graph_succ_right`
together with 4.5(ii). -/
theorem exists_unique_right (P : PeanoModel M) (Q : PeanoModel N) (m : M) :
    exists ! n, Graph P Q m n := by
  refine P.induction {m | exists ! n, Graph P Q m n} ?_ ?_ m
  * exact <Q.zero, Graph.base, fun k hk => graph_zero_right P Q hk>
  * rintro m' <n', hn', huniq>
    refine <Q.succ n', Graph.step hn', fun k hk => ?_>
    obtain <n'', hn'', rfl> := graph_succ_right P Q hk
    rw [huniq n'' hn'']

/-- Dually (by `Graph.swap`), each element of the right model is related to exactly one
     element of
the left model. -/
theorem exists_unique_left (P : PeanoModel M) (Q : PeanoModel N) (n : N) :
    exists ! m, Graph P Q m n := by
```

```
  obtain <m, hm, huniq> := exists_unique_right Q P n
  exact <m, hm.swap, fun y hy => huniq y hy.swap>

end PeanoModel
end
end DomainTheory.NeighbourhoodSystems
end
```

3.5.7 Theorem 4.6 — Peano models isomorphic

THEOREM 4.6. All models of Peano's Axioms are isomorphic.

Proof: There are several ways to give the proof, but, for the sake of illustration, an application of the fixed-point theorem is appropriate here. Let $\langle \mathbf{N}, 0, {}^+ \rangle$ be one model, and let $\langle \mathbf{M}, \Box, {}^\# \rangle$ be another. Let $\mathbf{N} \times \mathbf{M}$ be the ordinary cartesian product of the two sets and let

$$P(\mathbf{N} \times \mathbf{M})$$

be the powerset (set of all subsets) of $\mathbf{N} \times \mathbf{N}$. As in Exercises 1.15 and 2.20, we regard this set of elements as a domain, whose finite elements are just the finite subsets of the given set $\mathbf{N} \times \mathbf{M}$. The following mapping on $u \subseteq \mathbf{N} \times \mathbf{M}$ is easily proved approximable:

$$u \mapsto \{(0, \Box)\} \cup \{(n^+, m^\#) \mid (n, m) \in u\}.$$

(This assertion should be checked as an exercise.) We thus let r be the (least) fixed point:

$$r = \{(0, \Box)\} \cup \{(n^+, m^\#) \mid (n, m) \in r\}.$$

This $r \subseteq \mathbf{N} \times \mathbf{M}$ as a binary relation will turn out to be a one-one correspondence giving the required isomorphism.

First of all we see by construction that

(i) $0\ r\ \Box$;

(ii) $n\ r\ m$ implies $n^+\ r\ m^\#$.

So, if r proves to be a one-one correspondence, it will then be the desired isomorphism. Now, the two sets shown in the equation

$$\{(0, \Box)\} \cap \{(n^+, m^\#) \mid (n, m) \in r\} = \emptyset$$

are disjoint by virtue of axiom 4.5(i). Therefore, 0 in $\mathbf{N}$ corresponds by r to one and only one element of $\mathbf{M}$, namely the element $\Box$. Let $x \subseteq \mathbf{N}$ be the set of all elements of $\mathbf{N}$ corresponding by r to a unique element of $\mathbf{M}$. We have just shown $0 \in x$. Suppose $n \in x$, and let $m \in \mathbf{M}$ be the unique element with $n\ r\ m$. Now $n^+\ r\ m^\#$ holds, so n^+ corresponds to at least one element of $\mathbf{M}$. If $n^+\ r\ k$ also holds, then since $(n^+, k) \neq (0, \Box)$, the fixed-point equation implies

$$n^+ = n_0^+ \quad \text{and} \quad k = m_0^\#$$

for some $(n_0, m_0) \in r$. By axiom 4.5(ii), $n = n_0$, and, by uniqueness (remember $n \in x$), $m = m_0$; thus, $m^{\#}$ is the unique correspondent for n^{+}. We have proved $n^{+} \in x$. Therefore, $x^{+} \subseteq x$; so by axiom 4.5(iii), $x = \mathbf{N}$ holds. Otherwise said, every element in $\mathbf{N}$ corresponds to a unique element of $\mathbf{M}$.

Note that the rôles of $\mathbf{N}$ and $\mathbf{M}$ are completely symmetric, *and* they satisfy the same axioms as structured sets. It follows, then, that every element of $\mathbf{M}$ corresponds to a unique element of $\mathbf{N}$. The proof that r is a one-one correspondence is now complete. □

Theorem_4_6.lean

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Definition_4_5

public import Mathlib.Algebra.Group.End

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
universe u v
namespace PeanoModel
variable {M : Type u} {N : Type v}
/- concordance: source L2155 THEOREM 4.6 Peano models isomorphic -/
/-- **Theorem 4.6 (Scott 1981, PRG-19).** *All models of Peano's Axioms are isomorphic
    .* Between
any two models there is a bijection `e : N ? M` of the underlying sets that preserves
    the
structure: `e 0 = [] ` and `e (n^+) = (e n)#`. The bijection is the least-fixed-point
    relation
`Graph`, which `exists_unique_right`/`exists_unique_left` show is a one-one
    correspondence; the
two structure equations are the constructors `Graph.base`/`Graph.step`. -/
theorem peano_models_isomorphic (P : PeanoModel M) (Q : PeanoModel N) :
    exists  e : M ? N, e P.zero = Q.zero  /\  forall  m, e (P.succ m) = Q.succ (e m)
    := by
  classical
  refine <<fun m => (exists_unique_right P Q m).choose,
          fun n => (exists_unique_left P Q n).choose, ?_, ?_>, ?_, ?_>
  * -- left inverse `g  comp  f = id`
    intro m
    have hf : Graph P Q m (exists_unique_right P Q m).choose :=
      (exists_unique_right P Q m).choose_spec.1
    exact ((exists_unique_left P Q _).choose_spec.2 m hf).symm
  * -- right inverse `f  comp  g = id`
    intro n
    have hg : Graph P Q (exists_unique_left P Q n).choose n :=
      (exists_unique_left P Q n).choose_spec.1
    exact ((exists_unique_right P Q _).choose_spec.2 n hg).symm
  * -- `f 0 = [] `
    exact ((exists_unique_right P Q P.zero).choose_spec.2 Q.zero Graph.base).symm
  * -- `f (n^+) = (f n)#`
    intro m
    have hf : Graph P Q m (exists_unique_right P Q m).choose :=
      (exists_unique_right P Q m).choose_spec.1
```

```
    exact ((exists_unique_right P Q (P.succ m)).choose_spec.2 _ (Graph.step hf)).symm

end PeanoModel
end

end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.PeanoModel
section
set_option linter.unusedVariables false

theorem theorem_46.{u, v} : forall  {M : Type u} {N : Type v} (P : DomainTheory.
    NeighbourhoodSystems.PeanoModel M)
  (Q : DomainTheory.NeighbourhoodSystems.PeanoModel N),
  exists  (e : Equiv M N),
    And
      (@Eq N
        (@DFunLike.coe (Equiv M N) M (fun (x : M) => N)
          (@EquivLike.toFunLike (Equiv M N) M N (@Equiv.instEquivLike M N)) e
          (@DomainTheory.NeighbourhoodSystems.PeanoModel.zero M P))
        (@DomainTheory.NeighbourhoodSystems.PeanoModel.zero N Q))
      (forall  (m : M),
        @Eq N
          (@DFunLike.coe (Equiv M N) M (fun (x : M) => N)
            (@EquivLike.toFunLike (Equiv M N) M N (@Equiv.instEquivLike M N)) e
            (@DomainTheory.NeighbourhoodSystems.PeanoModel.succ M P m))
          (@DomainTheory.NeighbourhoodSystems.PeanoModel.succ N Q
            (@DFunLike.coe (Equiv M N) M (fun (x : M) => N)
              (@EquivLike.toFunLike (Equiv M N) M N (@Equiv.instEquivLike M N)) e m)))
     :=
  @DomainTheory.NeighbourhoodSystems.PeanoModel.peano_models_isomorphic
end

end DomainTheory.NeighbourhoodSystems.PeanoModel
end
```

3.5.8 Lecture IV exercises

EXERCISES

3.5.9 Exercise 4.7 — fixed point above a seed

Formula 4.2(iii) shows how to find the *least* fixed point of $f : D \to D$. Suppose on the other hand that $a \in |D|$ is such that $a \subseteq f(a)$. Will there be a fixed point $x = f(x)$ with $a \subseteq x$?

(Hint: How do we know $\bigsqcup_{n=0}^{\infty} f^n(\bot) \in |D|$?)

Exercise_4_7.lean

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Example_4_4

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Theorem_4_2
public import Mathlib.Tactic.Ring
```

```
@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap
section
variable {a : Type*} {V : NeighborhoodSystem a}
namespace ApproximableMap

/- concordance: source L2203 EXERCISE 4.7 fixed point above a seed -/
/- concordance: source L2201 EXERCISES -- Lecture IV exercises -/
/-- **Exercise 4.7 (Scott 1981, PRG-19).** `fixAbove f ha` is a fixed point: `f(x) = x
    `. -/
theorem fixAbove_isFixed (f : ApproximableMap V V) {a : V.Element} (ha : a  <=  f.
    toElementMap a) :
    f.toElementMap (f.fixAbove ha) = f.fixAbove ha := by
  set hdir := (fun i j => <max i j, iterFrom_mono f ha (le_max_left i j),
    iterFrom_mono f ha (le_max_right i j)> :
    forall  i j, exists  k, f.iterFrom a i  <=  f.iterFrom a k  /\  f.iterFrom a j  <=
      f.iterFrom a k) with hdir_def
  apply le_antisymm
  * -- `f(x) =  sqsup _n f(f?(a)) =  sqsup _n f^{n+1}(a)  sqsub  x` since each `f^{n
    +1}(a)  sqsub  x`.
    rw [fixAbove, toElementMap_iSupDirected]
    apply NeighborhoodSystem.iSupDirected_le
    intro n
    have : f.toElementMap (f.iterFrom a n) = f.iterFrom a (n + 1) := (iterFrom_succ f
    a n).symm
    rw [this]
    exact NeighborhoodSystem.le_iSupDirected (f.iterFrom a) hdir (n + 1)
  * -- `x  sqsub  f(x)`: each `f?(a)  sqsub  f?^+1(a) = f(f?(a))  sqsub  f(x)`.
    apply NeighborhoodSystem.iSupDirected_le
    intro n
    calc f.iterFrom a n
         <=  f.iterFrom a (n + 1) := iterFrom_mono f ha (Nat.le_succ n)
      _ = f.toElementMap (f.iterFrom a n) := iterFrom_succ f a n
      _  <=  f.toElementMap (f.fixAbove ha) :=
          f.toElementMap_mono (NeighborhoodSystem.le_iSupDirected (f.iterFrom a) hdir
    n)

/-- **Exercise 4.7 (Scott 1981, PRG-19).** `fixAbove f ha` is the *least* fixed point
    above `a`:
any `z` with `a  sqsub  z` and `f(z)  sqsub  z` lies above it. (Relativized form of
    4.2(iii).) -/
theorem fixAbove_least (f : ApproximableMap V V) {a : V.Element} (ha : a  <=  f.
    toElementMap a)
    {z : V.Element} (haz : a  <=  z) (hz : f.toElementMap z  <=  z) : f.fixAbove ha
    <=  z := by
  apply NeighborhoodSystem.iSupDirected_le
  intro n
  induction n with
  | zero => rwa [iterFrom_zero]
  | succ k ih =>
    rw [iterFrom_succ]
    exact le_trans (f.toElementMap_mono ih) hz
```

```
end ApproximableMap
end

end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.ApproximableMap
section
set_option linter.unusedVariables false

theorem example_407.{u_1} : And
  (forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
    (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V)
    {a : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V}
    (ha :
      @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
        (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
    a V)
          (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
   NeighborhoodSystem.Element a V)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
   instPartialOrderElement a V)))
        a (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a V V f a
   )),
    @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a V V f
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixAbove a V f a ha))
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixAbove a V f a ha))
  (forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
    (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V)
    {a : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V}
    (ha :
      @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
        (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
    a V)
          (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
   NeighborhoodSystem.Element a V)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
   instPartialOrderElement a V)))
        a (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a V V f a
   ))
    {z : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V},
    @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
        (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
    a V)
          (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
   NeighborhoodSystem.Element a V)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
   instPartialOrderElement a V)))
        a z  ->
      @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
          (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
   Element a V)
            (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
   NeighborhoodSystem.Element a V)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
   instPartialOrderElement a V)))
```

```
       (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a V V f z
  ) z  ->
     @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
       (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  Element a V)
         (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
           (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement a V)))
       (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixAbove a V f a ha) z)
  :=
@And.intro
(forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
  (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V)
  {a : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V}
  (ha :
    @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
      (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
   a V)
       (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
         (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement a V)))
     a (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a V V f a
  )),
  @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a V V f
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixAbove a V f a ha))
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixAbove a V f a ha))
(forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
  (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V)
  {a : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V}
  (ha :
    @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
      (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
   a V)
       (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
         (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement a V)))
     a (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a V V f a
  ))
  {z : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V},
  @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
     (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
   a V)
       (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
         (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement a V)))
     a z  ->
    @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
       (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  Element a V)
         (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
```

```
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    instPartialOrderElement a V)))
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a V V f z
    ) z  ->
        @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
          (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    Element a V)
           (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.Element a V)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    instPartialOrderElement a V)))
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixAbove a V f a ha) z)
  @DomainTheory.NeighbourhoodSystems.ApproximableMap.fixAbove_isFixed
  @DomainTheory.NeighbourhoodSystems.ApproximableMap.fixAbove_least
end

end DomainTheory.NeighbourhoodSystems.ApproximableMap
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

section
variable {a : Type*} {V : NeighborhoodSystem a}

namespace ApproximableMap

/-- `a  sqsub  fixAbove f ha`: the constructed fixed point lies above `a` (it is the `n
     = 0` term). -/
theorem le_fixAbove (f : ApproximableMap V V) {a : V.Element} (ha : a  <=  f.
    toElementMap a) :
    a  <=  f.fixAbove ha := by
  have := NeighborhoodSystem.le_iSupDirected (f.iterFrom a)
    (fun i j => <max i j, iterFrom_mono f ha (le_max_left i j),
      iterFrom_mono f ha (le_max_right i j)>) 0
  rwa [iterFrom_zero] at this

/-- The sup of a monotone `?`-chain, realized as a directed union (`max`-directedness).
     -/
def supChain (s : Nat  ->  V.Element) (hmono : Monotone s) : V.Element :=
  NeighborhoodSystem.iSupDirected s
    (fun i j => <max i j, hmono (le_max_left i j), hmono (le_max_right i j)>)

/-- The approximants form a monotone `?`-chain (the `Monotone` packaging of `
    iterElem_mono`). -/
theorem iterElem_monotone (f : ApproximableMap V V) : Monotone f.iterElem :=
  fun _ _ hab => iterElem_mono f hab

/-- `f?^+1(False) = f(f?(False))`. -/
theorem iterElem_succ (f : ApproximableMap V V) (n : Nat) :
    f.iterElem (n + 1) = f.toElementMap (f.iterElem n) := by
  show (f.comp (f.iterMap n)).toElementMap V.bot
      = f.toElementMap ((f.iterMap n).toElementMap V.bot)
  rw [toElementMap_comp]

theorem iterElem_zero (f : ApproximableMap V V) : f.iterElem 0 = V.bot :=
  toElementMap_idMap V.bot
```

```
/-- `(f?)?(False) = f^{2n}(False)`. -/
theorem comp_self_iterElem (f : ApproximableMap V V) (n : Nat) :
    (f.comp f).iterElem n = f.iterElem (2 * n) := by
  induction n with
  | zero => rw [iterElem_zero, Nat.mul_zero, iterElem_zero]
  | succ k ih =>
    have h2 : 2 * (k + 1) = 2 * k + 1 + 1 := by ring
    rw [iterElem_succ, ih, toElementMap_comp, h2, iterElem_succ, iterElem_succ]

theorem mem_supChain (s : Nat  ->  V.Element) (hmono : Monotone s) {Z : Set a} :
    (supChain s hmono).mem Z  <->  exists  n, (s n).mem Z :=
  NeighborhoodSystem.mem_iSupDirected s _

/-- The approximants `f?(False)` form a monotone chain whose sup is `fix(f)`. -/
theorem fixElement_eq_supChain (f : ApproximableMap V V) :
    f.fixElement = supChain f.iterElem (iterElem_monotone f) := by
  apply Element.ext
  intro X
  rw [mem_supChain, mem_fixElement]
  constructor
  * rintro <n, hn>; exact <n, (mem_iterElem f n).mpr hn>
  * rintro <n, hn>; exact <n, (mem_iterElem f n).mp hn>

end ApproximableMap
end
end DomainTheory.NeighbourhoodSystems
end
```

3.5.10 Exercise 4.8 — fixed-point induction

Suppose $f : D \to D$ and $S \subseteq |D|$ are such that

(i) $\bot \in S$;

(ii) $x \in S$ always implies $f(x) \in S$;

(iii) whenever $\{x_n\}_{n=0}^{\infty} \subseteq S$ and $x_n \sqsubseteq x_{n+1}$ for all n, then $\bigsqcup_{n=0}^{\infty} x_n \in S$.

Conclude that $\mathrm{fix}(f) \in S$. (This could be called the principle of *fixed-point induction*.) Apply the method to a set of the form

$$S = \{x \in |D| \mid a(x) = b(x)\},$$

where $a, b : D \to D$ are approximable, and where we know $a(\bot) = b(\bot)$, and $f \circ a = a \circ f$ and $f \circ b = b \circ f$.

Exercise_4_8.lean

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Exercise_4_7
```

```
@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap
section
variable {a : Type*} {V : NeighborhoodSystem a}
namespace ApproximableMap

/- concordance: source L2209 EXERCISE 4.8 fixed-point induction -/
/-- **Exercise 4.8 (Scott 1981, PRG-19) -- fixed-point induction.** If a predicate `P`
    holds at `False`,
is preserved by `f`, and is closed under sups of monotone chains, then it holds at `fix
    (f)`. -/
theorem fix_induction (f : ApproximableMap V V) (P : V.Element  ->  Prop)
    (hbot : P V.bot)
    (hstep : forall  x, P x  ->  P (f.toElementMap x))
    (hsup : forall  (s : Nat  ->  V.Element) (hmono : Monotone s), (forall  n, P (s n)
    )  ->  P (supChain s hmono)) :
    P f.fixElement := by
  have hmono : Monotone f.iterElem := iterElem_monotone f
  have hP : forall  n, P (f.iterElem n) := by
    intro n
    induction n with
    | zero => rw [iterElem_zero]; exact hbot
    | succ k ih => rw [iterElem_succ]; exact hstep _ ih
  rw [fixElement_eq_supChain f]
  exact hsup f.iterElem hmono hP

/-- **Exercise 4.8 (Scott 1981, PRG-19) -- application to `S = {x ? a(x) = b(x)}`.** If
     `a(False) = b(False)`
and `f` commutes with both `a` and `b` (`f  comp  a = a  comp  f`, `f  comp  b = b  comp
     f`), then `a` and `b` agree at
the least fixed point: `a(fix f) = b(fix f)`. -/
theorem fix_induction_eq (f a b : ApproximableMap V V)
    (hbot : a.toElementMap V.bot = b.toElementMap V.bot)
    (hfa : f.comp a = a.comp f) (hfb : f.comp b = b.comp f) :
    a.toElementMap f.fixElement = b.toElementMap f.fixElement := by
  -- commutation, elementwise: `a(f x) = f(a x)` and `b(f x) = f(b x)`.
  have hca : forall  x, a.toElementMap (f.toElementMap x) = f.toElementMap (a.
    toElementMap x) := by
    intro x
    have h1 : (a.comp f).toElementMap x = (f.comp a).toElementMap x := by rw [hfa]
    rwa [toElementMap_comp, toElementMap_comp] at h1
  have hcb : forall  x, b.toElementMap (f.toElementMap x) = f.toElementMap (b.
    toElementMap x) := by
    intro x
    have h1 : (b.comp f).toElementMap x = (f.comp b).toElementMap x := by rw [hfb]
    rwa [toElementMap_comp, toElementMap_comp] at h1
  refine fix_induction f (fun x => a.toElementMap x = b.toElementMap x) hbot ?_ ?_
  * intro x hx
    rw [hca, hcb, hx]
  * intro s hmono hs
    -- both sides are directed unions of equal families.
    show a.toElementMap (supChain s hmono) = b.toElementMap (supChain s hmono)
    rw [supChain, toElementMap_iSupDirected, toElementMap_iSupDirected]
```

```
    apply Element.ext
    intro Z
    rw [NeighborhoodSystem.mem_iSupDirected, NeighborhoodSystem.mem_iSupDirected]
    constructor
    * rintro <n, hn>; exact <n, by rw [ <-  hs n]; exact hn>
    * rintro <n, hn>; exact <n, by rw [hs n]; exact hn>

end ApproximableMap
end

end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.ApproximableMap
section
set_option linter.unusedVariables false

theorem example_408.{u_1} : And
  (forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
    (f a b : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V),
    @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a V V a
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bot a V))
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a V V b
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bot a V))  ->
      @Eq (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V)
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp a a a V V V f a)
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp a a a V V V a f)
   ->
       @Eq (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V)
           (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp a a a V V V f b)
           (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp a a a V V V b f)
    ->
         @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
           (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a V V a
             (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixElement a V f))
           (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a V V b
             (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixElement a V f)))
  (forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
    (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V)
    (P : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V  ->  Prop),
    P (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bot a V)  ->
      (forall  (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
    ,
          P x  ->  P (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
    a a V V f x))  ->
        (forall  (s : Nat  ->  @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    Element a V)
           (hmono :
             @Monotone Nat (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    Element a V) Nat.instPreorder
               (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.Element a V)
                 (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    instPartialOrderElement a V))
               s),
           (forall  (n : Nat), P (s n))  ->  P (@DomainTheory.NeighbourhoodSystems.
    ApproximableMap.supChain a V s hmono))  ->
```

```
        P (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixElement a V f)) :=
  @And.intro
  (forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
    (f a b : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V),
    @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a V V a
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bot a V))
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a V V b
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bot a V))  ->
      @Eq (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V)
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp a a a V V V f a)
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp a a a V V V a f)
    ->
        @Eq (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V)
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp a a a V V V f b)
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp a a a V V V b f)
     ->
          @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a V V a
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixElement a V f))
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a V V b
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixElement a V f)))
  (forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
    (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V)
    (P : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V  ->  Prop),
    P (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bot a V)  ->
      (forall  (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
    ,
          P x  ->  P (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
    a a V V f x))  ->
        (forall  (s : Nat  ->  @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    Element a V)
            (hmono :
              @Monotone Nat (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    Element a V) Nat.instPreorder
                (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.Element a V)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    instPartialOrderElement a V))
                s),
            (forall  (n : Nat), P (s n))  ->  P (@DomainTheory.NeighbourhoodSystems.
    ApproximableMap.supChain a V s hmono))  ->
          P (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixElement a V f))
  @DomainTheory.NeighbourhoodSystems.ApproximableMap.fix_induction_eq
  @DomainTheory.NeighbourhoodSystems.ApproximableMap.fix_induction
end

end DomainTheory.NeighbourhoodSystems.ApproximableMap
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

end DomainTheory.NeighbourhoodSystems
end
```

3.5.11 Exercise 4.9 — fix as a fixed point

Show that there is an approximable operator

$$\Psi : ((D \to D) \to D) \to ((D \to D) \to D)$$

such that for $\theta : (D \to D) \to D$ and $f : D \to D$ we have

$$\Psi(\theta)(f) = f(\theta(f)).$$

Prove further that $\mathrm{fix} : (D \to D) \to D$ is the *least* fixed point of Ψ.

Exercise_4_9.lean

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Theorem_4_2
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_13
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_12

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap
section
variable {a : Type*}

/- concordance: source L2225 EXERCISE 4.9 fix as a fixed point -/
/-- **Exercise 4.9 (Scott 1981, PRG-19).** The approximable operator `? = fun F fun f.
    f(F(f))` on the
higher-order domain `E = ((?  ->  ?)  ->  ?)`, built as `curry (eval  comp  <?_G, eval
    >)`. -/
def bigPsi (V : NeighborhoodSystem a) :
    ApproximableMap (funSpace (funSpace V V) V) (funSpace (funSpace V V) V) :=
  curry ((evalMap V V).comp
    (paired (proj_1 (funSpace (funSpace V V) V) (funSpace V V)) (evalMap (funSpace V V
    ) V)))

/-- **Exercise 4.9 (Scott 1981, PRG-19).** The defining equation `?(?)(f) = f(?(f))`.
    -/
theorem bigPsi_apply (V : NeighborhoodSystem a)
    (? : (funSpace (funSpace V V) V).Element) (f : (funSpace V V).Element) :
    (toApproxMap ((bigPsi V).toElementMap ?)).toElementMap f
      = (toApproxMap f).toElementMap ((toApproxMap ?).toElementMap f) := by
  rw [bigPsi, toElementMap_curry_apply, toElementMap_comp, toElementMap_paired,
    toElementMap_proj_1, snd_pair, evalMap_apply, evalMap_apply]

/-- `fix`, as the element of `E = ((?  ->  ?)  ->  ?)` corresponding to the operator `
    fixMap`, unfolds
under `toApproxMap` back to `fixMap` (the `funSpace` round-trip). -/
theorem toApproxMap_toFilter_fixMap (V : NeighborhoodSystem a) :
    toApproxMap (toFilter (fixMap V)) = fixMap V := by
```

```
  have he := (funSpaceEquiv (funSpace V V) V).apply_symm_apply (fixMap V)
  rwa [funSpaceEquiv_apply, funSpaceEquiv_symm_apply] at he

/-- **Exercise 4.9 (Scott 1981, PRG-19).** `fix` is a *fixed point* of `?`: `?(fix) =
    fix`. -/
theorem bigPsi_fix (V : NeighborhoodSystem a) :
    (bigPsi V).toElementMap (toFilter (fixMap V)) = toFilter (fixMap V) := by
  apply (funSpaceEquiv (funSpace V V) V).injective
  rw [funSpaceEquiv_apply, funSpaceEquiv_apply]
  apply ext_of_toElementMap
  intro f
  rw [bigPsi_apply, toApproxMap_toFilter_fixMap, fixMap_toElementMap,
    toElementMap_fixElement]

/-- **Exercise 4.9 (Scott 1981, PRG-19).** `fix` is the *least* pre-fixed point of `?`:
     any `?` with
`?(?)  sqsub  ?` satisfies `fix  sqsub  ?`. -/
theorem bigPsi_least (V : NeighborhoodSystem a) (? : (funSpace (funSpace V V) V).
    Element)
    (h? : (bigPsi V).toElementMap ?  <=  ?) : toFilter (fixMap V)  <=  ? := by
  -- transport `?(?)  sqsub  ?` to the pointwise pre-fixed-point inequality on maps.
  have h?' : toApproxMap ((bigPsi V).toElementMap ?)  <=  toApproxMap ? := by
    rw [ <-  funSpaceEquiv_apply,  <-  funSpaceEquiv_apply]
    exact (funSpaceEquiv (funSpace V V) V).monotone h?
  have hpre : forall  f, (toApproxMap f).toElementMap ((toApproxMap ?).toElementMap f)
       <=  (toApproxMap ?).toElementMap f := by
    intro f
    have h := (le_iff_toElementMap_le.mp h?') f
    rwa [bigPsi_apply] at h
  -- conclude `fix  sqsub  ?` pointwise via Theorem 4.1's minimality.
  apply (funSpaceEquiv (funSpace V V) V).le_iff_le.mp
  rw [funSpaceEquiv_apply, funSpaceEquiv_apply, toApproxMap_toFilter_fixMap,
    le_iff_toElementMap_le]
  intro f
  rw [fixMap_toElementMap]
  exact fixElement_le_of_toElementMap_le (toApproxMap f) (hpre f)

/-- **Exercise 4.9 (Scott 1981, PRG-19).** `fix = fix(?)`: `fix` is the least fixed
    point of `?`,
i.e. coincides with Theorem 4.1's canonical least fixed point `fixElement ?`. -/
theorem fix_eq_fixElement_bigPsi (V : NeighborhoodSystem a) :
    toFilter (fixMap V) = (bigPsi V).fixElement := by
  apply le_antisymm
  * exact bigPsi_least V _ (le_of_eq (toElementMap_fixElement (bigPsi V)))
  * exact fixElement_le_of_toElementMap_le (bigPsi V) (le_of_eq (bigPsi_fix V))

end

section
set_option linter.unusedVariables false

theorem example_409.{u_1} : And
  (And
    (And
      (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem a)
```

```
    (? :
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
        (@DomainTheory.NeighbourhoodSystems.funSpace (@DomainTheory.
NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V))
    (f :
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V)
        (@DomainTheory.NeighbourhoodSystems.funSpace a a V V)),
    @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
        (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V
        (@DomainTheory.NeighbourhoodSystems.toApproxMap (@DomainTheory.
NeighbourhoodSystems.ApproximableMap a a V V)
          a (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
              (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
              (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
            (@DomainTheory.NeighbourhoodSystems.funSpace
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
              (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
            (@DomainTheory.NeighbourhoodSystems.funSpace
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
              (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
            (@DomainTheory.NeighbourhoodSystems.bigPsi a V) ?))
        f)
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a V V
        (@DomainTheory.NeighbourhoodSystems.toApproxMap a a V V f)
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V
          (@DomainTheory.NeighbourhoodSystems.toApproxMap
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
            (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V ?)
          f)))
  (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a),
    @Eq
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
        (@DomainTheory.NeighbourhoodSystems.funSpace (@DomainTheory.
NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V))
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
```

```
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
        (@DomainTheory.NeighbourhoodSystems.funSpace (@DomainTheory.
NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
        (@DomainTheory.NeighbourhoodSystems.funSpace (@DomainTheory.
NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
        (@DomainTheory.NeighbourhoodSystems.bigPsi a V)
        (@DomainTheory.NeighbourhoodSystems.toFilter (@DomainTheory.
NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V (@DomainTheory.
NeighbourhoodSystems.fixMap a V)))
      (@DomainTheory.NeighbourhoodSystems.toFilter (@DomainTheory.
NeighbourhoodSystems.ApproximableMap a a V V) a
        (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V (@DomainTheory.
NeighbourhoodSystems.fixMap a V))))
(forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
a)
  (? :
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
        (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
      (@DomainTheory.NeighbourhoodSystems.funSpace (@DomainTheory.
NeighbourhoodSystems.ApproximableMap a a V V) a
        (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)),
  @LE.le
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
        (@DomainTheory.NeighbourhoodSystems.funSpace (@DomainTheory.
NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V))
      (@Preorder.toLE
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
            (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
          (@DomainTheory.NeighbourhoodSystems.funSpace (@DomainTheory.
NeighbourhoodSystems.ApproximableMap a a V V)
            a (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V))
        (@PartialOrder.toPreorder
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
              (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
            (@DomainTheory.NeighbourhoodSystems.funSpace
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
              (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V))
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
instPartialOrderElement
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap
```

```
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
              (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
            (@DomainTheory.NeighbourhoodSystems.funSpace
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
              (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V))))
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
        (@DomainTheory.NeighbourhoodSystems.funSpace (@DomainTheory.
NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
        (@DomainTheory.NeighbourhoodSystems.funSpace (@DomainTheory.
NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
        (@DomainTheory.NeighbourhoodSystems.bigPsi a V) ?)
      ?  ->
    @LE.le
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
        (@DomainTheory.NeighbourhoodSystems.funSpace (@DomainTheory.
NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V))
      (@Preorder.toLE
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
            (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
          (@DomainTheory.NeighbourhoodSystems.funSpace (@DomainTheory.
NeighbourhoodSystems.ApproximableMap a a V V)
            a (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V))
        (@PartialOrder.toPreorder
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
              (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
            (@DomainTheory.NeighbourhoodSystems.funSpace
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
              (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V))
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
instPartialOrderElement
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
              (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
            (@DomainTheory.NeighbourhoodSystems.funSpace
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
              (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V))))
      (@DomainTheory.NeighbourhoodSystems.toFilter (@DomainTheory.
NeighbourhoodSystems.ApproximableMap a a V V) a
        (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V (@DomainTheory.
NeighbourhoodSystems.fixMap a V))
```

```
       ?))
(forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a)
  ,
  @Eq
    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap (@DomainTheory.
  NeighbourhoodSystems.ApproximableMap a a V V)
        a (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
      (@DomainTheory.NeighbourhoodSystems.funSpace (@DomainTheory.
  NeighbourhoodSystems.ApproximableMap a a V V) a
        (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V))
    (@DomainTheory.NeighbourhoodSystems.toFilter (@DomainTheory.NeighbourhoodSystems
  .ApproximableMap a a V V) a
      (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V (@DomainTheory.
  NeighbourhoodSystems.fixMap a V))
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixElement
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap (@DomainTheory.
  NeighbourhoodSystems.ApproximableMap a a V V)
        a (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
      (@DomainTheory.NeighbourhoodSystems.funSpace (@DomainTheory.
  NeighbourhoodSystems.ApproximableMap a a V V) a
        (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
      (@DomainTheory.NeighbourhoodSystems.bigPsi a V))) :=
@And.intro
(And
  (And
    (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem a)
      (? :
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
            (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
          (@DomainTheory.NeighbourhoodSystems.funSpace (@DomainTheory.
  NeighbourhoodSystems.ApproximableMap a a V V) a
            (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V))
      (f :
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V)
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V)),
      @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V
          (@DomainTheory.NeighbourhoodSystems.toApproxMap (@DomainTheory.
  NeighbourhoodSystems.ApproximableMap a a V V)
            a (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap
                (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
                (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap
                (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
                (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
              (@DomainTheory.NeighbourhoodSystems.funSpace
                (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
```

```
              (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
            (@DomainTheory.NeighbourhoodSystems.funSpace
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
              (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
            (@DomainTheory.NeighbourhoodSystems.bigPsi a V) ?))
        f)
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a V V
        (@DomainTheory.NeighbourhoodSystems.toApproxMap a a V V f)
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V
          (@DomainTheory.NeighbourhoodSystems.toApproxMap
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
            (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V ?)
          f)))
  (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a),
    @Eq
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
        (@DomainTheory.NeighbourhoodSystems.funSpace (@DomainTheory.
NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V))
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
        (@DomainTheory.NeighbourhoodSystems.funSpace (@DomainTheory.
NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
        (@DomainTheory.NeighbourhoodSystems.funSpace (@DomainTheory.
NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
        (@DomainTheory.NeighbourhoodSystems.bigPsi a V)
        (@DomainTheory.NeighbourhoodSystems.toFilter (@DomainTheory.
NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V (@DomainTheory.
NeighbourhoodSystems.fixMap a V)))
      (@DomainTheory.NeighbourhoodSystems.toFilter (@DomainTheory.
NeighbourhoodSystems.ApproximableMap a a V V) a
        (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V (@DomainTheory.
NeighbourhoodSystems.fixMap a V))))
 (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
a)
  (? :
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
        (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
      (@DomainTheory.NeighbourhoodSystems.funSpace (@DomainTheory.
NeighbourhoodSystems.ApproximableMap a a V V) a
```

```
        (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)),
  @LE.le
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
        (@DomainTheory.NeighbourhoodSystems.funSpace (@DomainTheory.
NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V))
      (@Preorder.toLE
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
            (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
          (@DomainTheory.NeighbourhoodSystems.funSpace (@DomainTheory.
NeighbourhoodSystems.ApproximableMap a a V V)
            a (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V))
        (@PartialOrder.toPreorder
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
              (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
            (@DomainTheory.NeighbourhoodSystems.funSpace
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
              (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V))
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
instPartialOrderElement
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
              (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
            (@DomainTheory.NeighbourhoodSystems.funSpace
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
              (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V))))
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
        (@DomainTheory.NeighbourhoodSystems.funSpace (@DomainTheory.
NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
        (@DomainTheory.NeighbourhoodSystems.funSpace (@DomainTheory.
NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
        (@DomainTheory.NeighbourhoodSystems.bigPsi a V) ?)
      ?  ->
    @LE.le
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
        (@DomainTheory.NeighbourhoodSystems.funSpace (@DomainTheory.
NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V))
```

```
       (@Preorder.toLE
         (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
           (@DomainTheory.NeighbourhoodSystems.ApproximableMap
             (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
             (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
           (@DomainTheory.NeighbourhoodSystems.funSpace (@DomainTheory.
 NeighbourhoodSystems.ApproximableMap a a V V)
             a (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V))
         (@PartialOrder.toPreorder
           (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
             (@DomainTheory.NeighbourhoodSystems.ApproximableMap
               (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
               (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
             (@DomainTheory.NeighbourhoodSystems.funSpace
               (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
               (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V))
           (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 instPartialOrderElement
             (@DomainTheory.NeighbourhoodSystems.ApproximableMap
               (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
               (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
             (@DomainTheory.NeighbourhoodSystems.funSpace
               (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
               (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V))))
       (@DomainTheory.NeighbourhoodSystems.toFilter (@DomainTheory.
 NeighbourhoodSystems.ApproximableMap a a V V) a
         (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V (@DomainTheory.
 NeighbourhoodSystems.fixMap a V))
       ?))
(forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a)
 ,

  @Eq
    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap (@DomainTheory.
 NeighbourhoodSystems.ApproximableMap a a V V)
       a (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
      (@DomainTheory.NeighbourhoodSystems.funSpace (@DomainTheory.
 NeighbourhoodSystems.ApproximableMap a a V V) a
       (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V))
    (@DomainTheory.NeighbourhoodSystems.toFilter (@DomainTheory.NeighbourhoodSystems
 .ApproximableMap a a V V) a
      (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V (@DomainTheory.
 NeighbourhoodSystems.fixMap a V))
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixElement
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap (@DomainTheory.
 NeighbourhoodSystems.ApproximableMap a a V V)
       a (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
      (@DomainTheory.NeighbourhoodSystems.funSpace (@DomainTheory.
 NeighbourhoodSystems.ApproximableMap a a V V) a
       (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
      (@DomainTheory.NeighbourhoodSystems.bigPsi a V)))
(@And.intro
  (And
    (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem a)
```

```
    (? :
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
        (@DomainTheory.NeighbourhoodSystems.funSpace (@DomainTheory.
NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V))
    (f :
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V)
        (@DomainTheory.NeighbourhoodSystems.funSpace a a V V)),
    @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
        (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V
        (@DomainTheory.NeighbourhoodSystems.toApproxMap (@DomainTheory.
NeighbourhoodSystems.ApproximableMap a a V V)
          a (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
              (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
              (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
            (@DomainTheory.NeighbourhoodSystems.funSpace
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
              (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
            (@DomainTheory.NeighbourhoodSystems.funSpace
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
              (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
            (@DomainTheory.NeighbourhoodSystems.bigPsi a V) ?))
        f)
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a V V
        (@DomainTheory.NeighbourhoodSystems.toApproxMap a a V V f)
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V
          (@DomainTheory.NeighbourhoodSystems.toApproxMap
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
            (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V ?)
          f)))
  (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a),
    @Eq
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
        (@DomainTheory.NeighbourhoodSystems.funSpace (@DomainTheory.
NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V))
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
```

```
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
        (@DomainTheory.NeighbourhoodSystems.funSpace (@DomainTheory.
NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
        (@DomainTheory.NeighbourhoodSystems.funSpace (@DomainTheory.
NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
        (@DomainTheory.NeighbourhoodSystems.bigPsi a V)
        (@DomainTheory.NeighbourhoodSystems.toFilter (@DomainTheory.
NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V (@DomainTheory.
NeighbourhoodSystems.fixMap a V)))
      (@DomainTheory.NeighbourhoodSystems.toFilter (@DomainTheory.
NeighbourhoodSystems.ApproximableMap a a V V) a
        (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V (@DomainTheory.
NeighbourhoodSystems.fixMap a V))))
(forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
a)
  (? :
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
        (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
      (@DomainTheory.NeighbourhoodSystems.funSpace (@DomainTheory.
NeighbourhoodSystems.ApproximableMap a a V V) a
        (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)),
  @LE.le
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
        (@DomainTheory.NeighbourhoodSystems.funSpace (@DomainTheory.
NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V))
      (@Preorder.toLE
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
            (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
          (@DomainTheory.NeighbourhoodSystems.funSpace (@DomainTheory.
NeighbourhoodSystems.ApproximableMap a a V V)
            a (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V))
        (@PartialOrder.toPreorder
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
              (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
            (@DomainTheory.NeighbourhoodSystems.funSpace
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
              (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V))
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
instPartialOrderElement
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap
```

```
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
              (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
            (@DomainTheory.NeighbourhoodSystems.funSpace
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
              (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V))))
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
        (@DomainTheory.NeighbourhoodSystems.funSpace (@DomainTheory.
NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
        (@DomainTheory.NeighbourhoodSystems.funSpace (@DomainTheory.
NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
        (@DomainTheory.NeighbourhoodSystems.bigPsi a V) ?)
      ?  ->
    @LE.le
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
        (@DomainTheory.NeighbourhoodSystems.funSpace (@DomainTheory.
NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V))
      (@Preorder.toLE
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
            (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
          (@DomainTheory.NeighbourhoodSystems.funSpace (@DomainTheory.
NeighbourhoodSystems.ApproximableMap a a V V)
            a (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V))
        (@PartialOrder.toPreorder
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
              (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
            (@DomainTheory.NeighbourhoodSystems.funSpace
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
              (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V))
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
instPartialOrderElement
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
              (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
            (@DomainTheory.NeighbourhoodSystems.funSpace
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
              (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V))))
      (@DomainTheory.NeighbourhoodSystems.toFilter (@DomainTheory.
NeighbourhoodSystems.ApproximableMap a a V V) a
        (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V (@DomainTheory.
NeighbourhoodSystems.fixMap a V))
```

```
      ?)
(@And.intro
  (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
    (? :
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
        (@DomainTheory.NeighbourhoodSystems.funSpace (@DomainTheory.
NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V))
    (f :
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V)
        (@DomainTheory.NeighbourhoodSystems.funSpace a a V V)),
    @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
        (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V
        (@DomainTheory.NeighbourhoodSystems.toApproxMap (@DomainTheory.
NeighbourhoodSystems.ApproximableMap a a V V)
          a (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
              (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
              (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
            (@DomainTheory.NeighbourhoodSystems.funSpace
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
              (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
            (@DomainTheory.NeighbourhoodSystems.funSpace
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
              (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
            (@DomainTheory.NeighbourhoodSystems.bigPsi a V) ?))
        f)
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a V V
        (@DomainTheory.NeighbourhoodSystems.toApproxMap a a V V f)
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V
          (@DomainTheory.NeighbourhoodSystems.toApproxMap
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
            (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V ?)
          f)))
  (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a),
    @Eq
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
        (@DomainTheory.NeighbourhoodSystems.funSpace (@DomainTheory.
NeighbourhoodSystems.ApproximableMap a a V V) a
```

```
              (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V))
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
              (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V) a
              (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
            (@DomainTheory.NeighbourhoodSystems.funSpace (@DomainTheory.
    NeighbourhoodSystems.ApproximableMap a a V V) a
              (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
            (@DomainTheory.NeighbourhoodSystems.funSpace (@DomainTheory.
    NeighbourhoodSystems.ApproximableMap a a V V) a
              (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V)
            (@DomainTheory.NeighbourhoodSystems.bigPsi a V)
            (@DomainTheory.NeighbourhoodSystems.toFilter (@DomainTheory.
    NeighbourhoodSystems.ApproximableMap a a V V) a
              (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V (@DomainTheory.
    NeighbourhoodSystems.fixMap a V)))
          (@DomainTheory.NeighbourhoodSystems.toFilter (@DomainTheory.
    NeighbourhoodSystems.ApproximableMap a a V V) a
            (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V (@DomainTheory.
    NeighbourhoodSystems.fixMap a V)))
      @DomainTheory.NeighbourhoodSystems.bigPsi_apply @DomainTheory.
    NeighbourhoodSystems.bigPsi_fix)
    @DomainTheory.NeighbourhoodSystems.bigPsi_least)
  @DomainTheory.NeighbourhoodSystems.fix_eq_fixElement_bigPsi
end

end DomainTheory.NeighbourhoodSystems
end
```

3.5.12 Exercise 4.10 — domain above an element

Given a domain D and an element $a \in |D|$, construct a domain D_a where

$$|D_a| = \{x \in |D| \mid x \sqsubseteq a\}.$$

Show that if $f : D \to D$ is approximable, then f can be restricted to an approximable map $f' : D_{\mathrm{fix}(f)} \to D_{\mathrm{fix}(f)}$ where $f'(x) = f(x)$ for all $x \in |D_{\mathrm{fix}(f)}|$. How many fixed points does f' have in $|D_{\mathrm{fix}(f)}|$?

Exercise_4_10.lean

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Theorem_4_2
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Theorem_4_1
public import Mathlib.Tactic.Monotonicity.Lemmas
public import Mathlib.Tactic.Bound

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
```

```
open NeighborhoodSystem ApproximableMap
section
variable {a : Type*} {V : NeighborhoodSystem a}
namespace ApproximableMap

/- concordance: source L2239 EXERCISE 4.10 domain above an element -/
/-- **Exercise 4.10 (Scott 1981, PRG-19).** The relativized neighbourhood system `D?`:
    same tokens
and master, neighbourhoods exactly the members of the filter `a`. -/
def relSystem (a : V.Element) : NeighborhoodSystem a where
  mem X := a.mem X
  master := V.master
  master_nonempty := V.master_nonempty
  master_mem := a.master_mem
  sub_master := fun hX => V.sub_master (a.sub hX)
  inter_mem := fun hX hY _ _ => a.inter_mem hX hY

@[simp] theorem relSystem_mem (a : V.Element) {X : Set a} :
    (relSystem a).mem X  <->  a.mem X := Iff.rfl

@[simp] theorem relSystem_master (a : V.Element) : (relSystem a).master = V.master :=
    rfl

/-- The `?`-element obtained from a `D?`-filter by upward closure in `?`. -/
def embed (a : V.Element) (g : (relSystem a).Element) : V.Element where
  mem X := V.mem X  /\  exists  W, a.mem W  /\  g.mem W  /\  W  subseteq  X
  sub := fun h => h.1
  master_mem := <V.master_mem, V.master, a.master_mem, g.master_mem, subset_rfl>
  inter_mem := by
    rintro X Y <hVX, Wx, haWx, hgWx, hWxX> <hVY, Wy, haWy, hgWy, hWyY>
    have haWxy : a.mem (Wx  I  Wy) := a.inter_mem haWx haWy
    have hgWxy : g.mem (Wx  I  Wy) := g.inter_mem hgWx hgWy
    have hsub : Wx  I  Wy  subseteq  X  I  Y := Set.inter_subset_inter hWxX hWyY
    refine <V.inter_mem hVX hVY (a.sub haWxy) hsub, Wx  I  Wy, haWxy, hgWxy, hsub>
  up_mem := by
    rintro X Y <hVX, W, haW, hgW, hWX> hVY hXY
    exact <hVY, W, haW, hgW, subset_trans hWX hXY>

/-- `embed a g  sqsub  a`: the embedding lands below `a`. -/
theorem embed_le (a : V.Element) (g : (relSystem a).Element) : embed a g  <=  a := by
  rintro X <hVX, W, haW, _, hWX>
  exact a.up_mem haW hVX hWX

theorem embed_mono (a : V.Element) {g_1 g_2 : (relSystem a).Element} (h : g_1  <=  g_2
    ) :
    embed a g_1  <=  embed a g_2 := by
  rintro X <hVX, W, haW, hgW, hWX>
  exact <hVX, W, haW, h _ hgW, hWX>

theorem le_of_embed_le (a : V.Element) {g_1 g_2 : (relSystem a).Element}
    (h : embed a g_1  <=  embed a g_2) : g_1  <=  g_2 := by
  intro X hg_1X
  have hVX : V.mem X := a.sub (g_1.sub hg_1X)
  have : (embed a g_2).mem X := h X <hVX, X, g_1.sub hg_1X, hg_1X, subset_rfl>
  obtain <_, W, haW, hg_2W, hWX> := this
  exact g_2.up_mem hg_2W (g_1.sub hg_1X) hWX
```

```
/-- **Exercise 4.10 (Scott 1981, PRG-19).** When `f(a) = a`, the restriction `f' : D?
    ->  D?`
(`f'(x) = f(x)`). The codomain condition uses `X  sqsub  a ? f(X)  sqsub  f(a) = a`. -/
def relMap (f : ApproximableMap V V) {a : V.Element} (ha : f.toElementMap a = a) :
    ApproximableMap (relSystem a) (relSystem a) where
  rel X Y := a.mem X  /\  f.rel X Y
  rel_dom := fun h => h.1
  rel_cod := by
    rintro X Y <haX, hXY>
    have hVX : V.mem X := a.sub haX
    have hpr : V.principal hVX  <=  a := fun Z <hVZ, hXZ> => a.up_mem haX hVZ hXZ
    have hmemfX : (f.toElementMap (V.principal hVX)).mem Y := (f.rel_iff_mem_principal
     hVX).mp hXY
    have hle : f.toElementMap (V.principal hVX)  <=  f.toElementMap a := f.
    toElementMap_mono hpr
    have : (f.toElementMap a).mem Y := hle Y hmemfX
    rw [ha] at this
    exact this
  master_rel := <a.master_mem, f.master_rel>
  inter_right := by
    rintro X Y Y' <haX, hXY> <_, hXY'>
    exact <haX, f.inter_right hXY hXY'>
  mono := by
    rintro X X' Y Y' <_, hXY> hX'X hYY' haX' haY'
    exact <haX', f.mono hXY hX'X hYY' (a.sub haX') (a.sub haY')>

/-- The `D?`-filter obtained from an element `x  sqsub  a` by restriction (same
    membership). -/
def restrict (a : V.Element) (x : V.Element) (hx : x  <=  a) : (relSystem a).Element
    where
  mem X := x.mem X
  sub := fun h => hx _ h
  master_mem := x.master_mem
  inter_mem := fun hX hY => x.inter_mem hX hY
  up_mem := fun hX hY hXY => x.up_mem hX (a.sub hY) hXY

theorem embed_restrict (a : V.Element) (x : V.Element) (hx : x  <=  a) :
    embed a (restrict a x hx) = x := by
  apply Element.ext
  intro X
  constructor
  * rintro <hVX, W, _, hxW, hWX>
    exact x.up_mem hxW hVX hWX
  * intro hxX
    exact <x.sub hxX, X, hx _ hxX, hxX, subset_rfl>

theorem restrict_embed (a : V.Element) (g : (relSystem a).Element) :
    restrict a (embed a g) (embed_le a g) = g := by
  apply Element.ext
  intro X
  constructor
  * rintro <hVX, W, haW, hgW, hWX>
    exact g.up_mem hgW (a.up_mem haW hVX hWX) hWX
  * intro hgX
    exact <a.sub (g.sub hgX), X, g.sub hgX, hgX, subset_rfl>
```

```
/-- **Exercise 4.10 (Scott 1981, PRG-19).** `|D?| ?o {x  in  |?| ? x  sqsub  a}`: the
    relativized domain has
exactly the elements below `a` as its points, with the inherited order. -/
def relIso (a : V.Element) : (relSystem a).Element ?o {x : V.Element // x  <=  a}
    where
  toFun g := <embed a g, embed_le a g>
  invFun x := restrict a x.1 x.2
  left_inv g := restrict_embed a g
  right_inv x := Subtype.ext (embed_restrict a x.1 x.2)
  map_rel_iff' := by
    intro g_1 g_2
    constructor
    * intro h; exact le_of_embed_le a h
    * intro h; exact embed_mono a h

/-- **Exercise 4.10 (Scott 1981, PRG-19).** Any fixed point of `f` below `fix(f)` *is*
    `fix(f)`:
a fixed point is a pre-fixed point, so leastness forces `fix(f)  sqsub  x`. -/
theorem fixElement_below_unique (f : ApproximableMap V V) {x : V.Element}
    (hxle : x  <=  f.fixElement) (hxfix : f.toElementMap x = x) : x = f.fixElement :=
    by
  have hge : f.fixElement  <=  x := fixElement_le_of_toElementMap_le f (le_of_eq hxfix
    )
  exact le_antisymm hxle hge

/-- The restriction `f'` has the same action as `f`: `embed a (f'(g)) = f(embed a g)`.
    -/
theorem relMap_toElementMap_embed (f : ApproximableMap V V) {a : V.Element}
    (ha : f.toElementMap a = a) (g : (relSystem a).Element) :
    embed a ((relMap f ha).toElementMap g) = f.toElementMap (embed a g) := by
  apply Element.ext
  intro X
  constructor
  * rintro <hVX, W, haW, <U, hgU, haU, hUW>, hWX>
    -- `U  in  g`, `U f W`, `W  subseteq  X`; so `X  in  f(embed g)` via the member `U`
    of `embed g`.
    refine <U, <a.sub haU, U, haU, hgU, subset_rfl>, ?_>
    exact f.mono hUW subset_rfl hWX (a.sub haU) hVX
  * rintro <U, <hVU, W, haW, hgW, hWU>, hUX>
    -- `W  in  g`, `W  subseteq  U`, `U f X`; so `W f X` and `X  in  embed a (f'(g))`.
    have hVX : V.mem X := f.rel_cod hUX
    have hWX : f.rel W X := f.mono hUX hWU subset_rfl (a.sub haW) hVX
    have haX : a.mem X := by
      have hpr : V.principal (a.sub haW)  <=  a := fun Z <hVZ, hXZ> => a.up_mem haW
    hVZ hXZ
      have hmemfW : (f.toElementMap (V.principal (a.sub haW))).mem X :=
        (f.rel_iff_mem_principal (a.sub haW)).mp hWX
      have := (f.toElementMap_mono hpr) X hmemfW
      rw [ha] at this; exact this
    exact <hVX, X, haX, <W, hgW, haW, hWX>, subset_rfl>

/-- **Exercise 4.10 (Scott 1981, PRG-19).** With `a = fix(f)`, the restricted map `f'`
    has *exactly
one* fixed point: the top element of `D_{fix f}`, namely `restrict (fix f)` of `fix f`
    itself. Every
```

```
`D?`-fixed point `g` of `f'` satisfies `embed a g = fix f`, hence is this top element.
    -/
theorem relMap_unique_fixed (f : ApproximableMap V V)
    (ha : f.toElementMap f.fixElement = f.fixElement) (g : (relSystem f.fixElement).
    Element)
    (hg : (relMap f ha).toElementMap g = g) :
    g = restrict f.fixElement f.fixElement (le_refl _) := by
  -- `embed g` is a fixed point of `f` below `fix f`, hence equals `fix f`.
  have hembfix : f.toElementMap (embed f.fixElement g) = embed f.fixElement g := by
    rw [ <-  relMap_toElementMap_embed f ha g, hg]
  have heq : embed f.fixElement g = f.fixElement :=
    fixElement_below_unique f (embed_le f.fixElement g) hembfix
  -- transport along the iso.
  have h1 : restrict f.fixElement (embed f.fixElement g) (embed_le f.fixElement g) = g
     :=
    restrict_embed f.fixElement g
  -- both restricts agree because their underlying elements agree.
  have h2 : restrict f.fixElement (embed f.fixElement g) (embed_le f.fixElement g)
      = restrict f.fixElement f.fixElement (le_refl _) := by
    apply Element.ext
    intro X
    show (embed f.fixElement g).mem X  <->  f.fixElement.mem X
    rw [heq]
  rw [ <-  h1, h2]

end ApproximableMap
end

end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.ApproximableMap
section
set_option linter.unusedVariables false

theorem example_410.{u_1} : And
  (forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
    (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V)
    {x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V},
    @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
        (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
     a V)
          (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.Element a V)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    instPartialOrderElement a V)))
        x (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixElement a V f)  ->
      @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a V V f x
    ) x  ->
        @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V) x
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixElement a V f))
  (forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
    (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V)
    (ha :
      @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a V V f
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixElement a V f))
```

```
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixElement a V f))
  (g :
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.relSystem a V
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixElement a V f))),
  @Eq
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.relSystem a V
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixElement a V f)))
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.relSystem a V
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixElement a V f))
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.relSystem a V
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixElement a V f))
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.relMap a V f
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixElement a V f) ha)
        g)
      g  ->
    @Eq
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.relSystem a V
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixElement a V f)))
      g
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.restrict a V
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixElement a V f)
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixElement a V f)
        (@le_refl (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V
)
          (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement a V))
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixElement a V f))))
  :=
@And.intro
(forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
  (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V)
  {x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V},
  @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
      (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
   a V)
        (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement a V)))
      x (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixElement a V f)  ->
    @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a V V f x
  ) x  ->
      @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V) x
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixElement a V f))
(forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
  (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V)
  (ha :
    @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a V V f
```

```
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixElement a V f))
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixElement a V f))
    (g :
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.relSystem a V
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixElement a V f))),
    @Eq
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.relSystem a V
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixElement a V f)))
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.relSystem a V
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixElement a V f))
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.relSystem a V
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixElement a V f))
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.relMap a V f
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixElement a V f) ha)
          g)
        g  ->
      @Eq
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.relSystem a V
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixElement a V f)))
        g
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.restrict a V
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixElement a V f)
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixElement a V f)
          (@le_refl (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V
    )
            (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.Element a V)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    instPartialOrderElement a V))
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.fixElement a V f))))
  @DomainTheory.NeighbourhoodSystems.ApproximableMap.fixElement_below_unique
  @DomainTheory.NeighbourhoodSystems.ApproximableMap.relMap_unique_fixed
end

end DomainTheory.NeighbourhoodSystems.ApproximableMap
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

section
variable {a : Type*} {V : NeighborhoodSystem a}

namespace ApproximableMap

/-- The inclusion `??  ->  ?` as an approximable map: an `a`-neighbourhood `X` relates
    to any larger
`?`-neighbourhood `Y  supseteq  X`. Its elementwise action is `embed a` (`
    inclMap_toElementMap`). -/
def inclMap (a : V.Element) : ApproximableMap (relSystem a) V where
  rel X Y := a.mem X  /\  V.mem Y  /\  X  subseteq  Y
  rel_dom := fun h => h.1
  rel_cod := fun h => h.2.1
  master_rel := <a.master_mem, V.master_mem, subset_rfl>
```

```
  inter_right := by
    rintro X Y Y' <haX, hVY, hXY> <_, hVY', hXY'>
    exact <haX, V.inter_mem hVY hVY' (a.sub haX) (Set.subset_inter hXY hXY'),
      Set.subset_inter hXY hXY'>
  mono := by
    rintro X X' Y Y' <_, _, hXY> hX'X hYY' haX' hVY'
    exact <haX', hVY', (hX'X.trans hXY).trans hYY'>

/-- The inclusion's elementwise action is `embed a` (Exercise 4.10). -/
theorem inclMap_toElementMap (a : V.Element) (g : (relSystem a).Element) :
    (inclMap a).toElementMap g = embed a g := by
  apply Element.ext
  intro Y
  constructor
  * rintro <X, hgX, haX, hVY, hXY>
    exact <hVY, X, haX, hgX, hXY>
  * rintro <hVY, W, haW, hgW, hWY>
    exact <W, hgW, haW, hVY, hWY>

/-- The inclusion is strict: `?(False) = False`. -/
theorem inclMap_bot (a : V.Element) : (inclMap a).toElementMap (relSystem a).bot = V.
    bot := by
  rw [inclMap_toElementMap]
  apply Element.ext
  intro Y
  rw [mem_bot]
  constructor
  * rintro <hVY, W, _, hbW, hWY>
    rw [mem_bot] at hbW
    rw [hbW] at hWY
    exact Set.Subset.antisymm (V.sub_master hVY) hWY
  * rintro rfl
    exact <V.master_mem, V.master, a.master_mem, (relSystem a).bot.master_mem,
    subset_rfl>

/-- The inclusion intertwines `f'` with `f`: `?  comp  f' = f  comp  ?` (from `
    relMap_toElementMap_embed`). -/
theorem inclMap_intertwine (f : ApproximableMap V V) {a : V.Element} (ha : f.
    toElementMap a = a)
    (g : (relSystem a).Element) :
    (inclMap a).toElementMap ((relMap f ha).toElementMap g) =
      f.toElementMap ((inclMap a).toElementMap g) := by
  rw [inclMap_toElementMap, inclMap_toElementMap, relMap_toElementMap_embed]

end ApproximableMap
end

end DomainTheory.NeighbourhoodSystems
end
```

3.5.13 Exercise 4.11 — characterizing the fix combinator

(Suggested by G. Plotkin). We can regard fix as assigning a fixed-point operator to each domain D. Show that fix is uniquely determined by the following general conditions on an assignment $D \rightsquigarrow F_D$:

(i) $F_D : (D \to D) \to D$;

(ii) $F_D(f) = f(F_D(f))$ for all $f : D \to D$;

(iii) whenever $f_0 : D_0 \to D_0$ and $f_1 : D_1 \to D_1$ are given and $h : D_0 \to D_1$ is such that $h(\bot) = \bot$ and $h \circ f_0 = f_1 \circ h$, then $h(F_{D_0}(f_0)) = F_{D_1}(f_1)$.

(Hint: Apply 4.7 to prove fix satisfies (iii). In the other direction use 4.10.)

Exercise_4_11.lean

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Exercise_4_10
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Exercise_4_7
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Theorem_4_2

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap
section
variable {a : Type*} {V : NeighborhoodSystem a}
open ApproximableMap
universe u
/- concordance: source L2249 EXERCISE 4.11 characterizing the fix combinator -/
/-- **Exercise 4.11 (Scott 1981, PRG-19) -- Plotkin's uniqueness.** Any assignment `F`
    of a
fixed-point operator to every domain satisfying (ii) `F_?(f) = f(F_?(f))` and (iii) the
     uniformity
law coincides with `fix`: `F_?(f) = fix(f)` for every domain `?` and every `f : ?  ->
    ?`. -/
theorem fix_unique_of_uniform
    (F : forall  {b : Type u} (W : NeighborhoodSystem b), ApproximableMap W W  ->  W.
    Element)
    (hfixed : forall  {b : Type u} (W : NeighborhoodSystem b) (g : ApproximableMap W W
    ),
      g.toElementMap (F W g) = F W g)
    (huniform : forall  {b ? : Type u} (W_0 : NeighborhoodSystem b) (W_1 :
    NeighborhoodSystem ?)
      (g_0 : ApproximableMap W_0 W_0) (g_1 : ApproximableMap W_1 W_1) (h :
    ApproximableMap W_0 W_1),
      h.toElementMap W_0.bot = W_1.bot  ->
      (forall  x, h.toElementMap (g_0.toElementMap x) = g_1.toElementMap (h.
    toElementMap x))  ->
      h.toElementMap (F W_0 g_0) = F W_1 g_1)
    {a : Type u} (V : NeighborhoodSystem a) (f : ApproximableMap V V) :
    F V f = f.fixElement := by
  have ha : f.toElementMap f.fixElement = f.fixElement := toElementMap_fixElement f
  -- `F` of the restricted map `f'` is a fixed point of `f'` (by (ii))...
  have hfix' : (relMap f ha).toElementMap (F (relSystem f.fixElement) (relMap f ha)) =
      F (relSystem f.fixElement) (relMap f ha) := hfixed (relSystem f.fixElement) (
    relMap f ha)
  -- ...hence it is the unique fixed point `restrict (fix f) (fix f)` of `f'` (Exercise
     4.10).
```

```
  have huniq : F (relSystem f.fixElement) (relMap f ha) =
      restrict f.fixElement f.fixElement (le_refl _) :=
    relMap_unique_fixed f ha _ hfix'
  -- uniformity along the inclusion `?` transports `F(f')` to `F(f)`.
  have huse := huniform (relSystem f.fixElement) V (relMap f ha) f (inclMap f.
    fixElement)
    (inclMap_bot f.fixElement) (inclMap_intertwine f ha)
  rw [huniq, inclMap_toElementMap, embed_restrict] at huse
  exact huse.symm
end

section
variable {a : Type*} {V : NeighborhoodSystem a}

namespace ApproximableMap

/-- `f?(False)  sqsub  fix(f)`: every approximant lies below the least fixed point. -/
theorem iterElem_le_fixElement (f : ApproximableMap V V) (n : Nat) :
    f.iterElem n  <=  f.fixElement :=
  fun _ hX => <n, (mem_iterElem f n).mp hX>

/-- **Exercise 4.11(iii) (Scott 1981, PRG-19).** `fix` is *uniform*: if `h(False) =
    False` and
`h  comp  f_0 = f_1  comp  h`, then `h(fix f_0) = fix f_1`. -/
theorem fixElement_uniform {b ? : Type*} {W_0 : NeighborhoodSystem b} {W_1 :
    NeighborhoodSystem ?}
    (f_0 : ApproximableMap W_0 W_0) (f_1 : ApproximableMap W_1 W_1) (h :
    ApproximableMap W_0 W_1)
    (hbot : h.toElementMap W_0.bot = W_1.bot)
    (hintw : forall  x, h.toElementMap (f_0.toElementMap x) = f_1.toElementMap (h.
    toElementMap x)) :
    h.toElementMap f_0.fixElement = f_1.fixElement := by
  -- `h` carries the `n`-th approximant of `f_0` to that of `f_1`.
  have hiter : forall  n, h.toElementMap (f_0.iterElem n) = f_1.iterElem n := by
    intro n
    rw [iterElem_eq_iterate f_0 n, iterElem_eq_iterate f_1 n]
    induction n with
    | zero => simpa using hbot
    | succ k ih =>
      rw [Function.iterate_succ', Function.comp_apply, Function.iterate_succ',
        Function.comp_apply, hintw, ih]
  apply le_antisymm
  * -- `h(fix f_0) =  sqsup _n h(f_0?(False)) =  sqsup _n f_1?(False)  sqsub  fix f_1`.
    rw [fixElement_eq_iSupDirected f_0, toElementMap_iSupDirected]
    apply NeighborhoodSystem.iSupDirected_le
    intro n
    rw [hiter n]
    exact iterElem_le_fixElement f_1 n
  * -- `h(fix f_0)` is a fixed point of `f_1`, so `fix f_1  sqsub  h(fix f_0)` by
    minimality.
    have hfp : f_1.toElementMap (h.toElementMap f_0.fixElement) = h.toElementMap f_0.
    fixElement := by
      rw [ <-  hintw f_0.fixElement, toElementMap_fixElement]
    exact fixElement_le_of_toElementMap_le f_1 (le_of_eq hfp)

/-- Commuting approximable maps have a least common fixed point `x = f(x) = g(x)`. -/
```

```
theorem commuting_least_common_fixed {f g : ApproximableMap V V} (hcomm : f.comp g = g
    .comp f) :
    exists  x : V.Element, f.toElementMap x = x  /\  g.toElementMap x = x  /\
      forall  w : V.Element, f.toElementMap w = w  ->  g.toElementMap w = w  ->  x  <=
      w := by
  set a := f.fixElement with ha_def
  have hfa : f.toElementMap a = a := toElementMap_fixElement f
  -- `a  sqsub  g(a)` since `g(a)` is a fixed point of `f`.
  have ha_le : a  <=  g.toElementMap a := by
    apply fixElement_le_of_toElementMap_le f
    rw [commute_apply hcomm, hfa]
  set x := g.fixAbove ha_le with hx_def
  have hgx : g.toElementMap x = x := fixAbove_isFixed g ha_le
  have hax : a  <=  x := le_fixAbove g ha_le
  -- each `g?(a)` is a fixed point of `f`.
  have hfix_each : forall  n, f.toElementMap (g.iterFrom a n) = g.iterFrom a n := by
    intro n
    induction n with
    | zero => rw [iterFrom_zero]; exact hfa
    | succ k ih =>
      rw [iterFrom_succ g a k, commute_apply hcomm, ih]
  -- so `f(x) = x` by continuity.
  have hfx : f.toElementMap x = x := by
    apply le_antisymm
    * rw [hx_def, fixAbove, toElementMap_iSupDirected]
      apply NeighborhoodSystem.iSupDirected_le
      intro n
      rw [hfix_each n]
      exact NeighborhoodSystem.le_iSupDirected (g.iterFrom a) _ n
    * rw [hx_def, fixAbove]
      apply NeighborhoodSystem.iSupDirected_le
      intro n
      rw [ <-  hfix_each n]
      exact f.toElementMap_mono (NeighborhoodSystem.le_iSupDirected (g.iterFrom a) _ n
   )
  refine <x, hfx, hgx, ?_>
  intro w hfw hgw
  have haw : a  <=  w := fixElement_le_of_toElementMap_le f (le_of_eq hfw)
  exact fixAbove_least g ha_le haw (le_of_eq hgw)

/-- If `f` and `g` commute and `f(False) = g(False)`, then `fix(f) = fix(g)`. -/
theorem fixElement_eq_of_commuting_bot {f g : ApproximableMap V V} (hcomm : f.comp g =
     g.comp f)
    (hbot : f.toElementMap V.bot = g.toElementMap V.bot) :
    f.fixElement = g.fixElement := by
  -- `g(f?False) = f?^+1False`.
  have hshift : forall  n, g.toElementMap (f.iterElem n) = f.iterElem (n + 1) := by
    intro n
    induction n with
    | zero =>
      rw [iterElem_zero, iterElem_succ, iterElem_zero,  <-  hbot]
    | succ k ih =>
      rw [iterElem_succ,  <-  commute_apply hcomm, ih]
      exact (iterElem_succ f (k + 1)).symm
  -- `g(fix f) = fix f`.
  have hgfixf : g.toElementMap f.fixElement = f.fixElement := by
```

```
    rw [fixElement_eq_iSupDirected f, toElementMap_iSupDirected]
    apply le_antisymm
    * apply NeighborhoodSystem.iSupDirected_le
      intro n
      rw [hshift n]
      exact NeighborhoodSystem.le_iSupDirected f.iterElem _ (n + 1)
    * apply NeighborhoodSystem.iSupDirected_le
      intro n
      calc f.iterElem n  <=  f.iterElem (n + 1) := iterElem_mono f (Nat.le_succ n)
        _ = g.toElementMap (f.iterElem n) := (hshift n).symm
        _  <=  _ := NeighborhoodSystem.le_iSupDirected (fun m => g.toElementMap (f.
    iterElem m)) _ n
  -- symmetric: `f(fix g) = fix g`.
  have hshift' : forall  n, f.toElementMap (g.iterElem n) = g.iterElem (n + 1) := by
    intro n
    induction n with
    | zero =>
      rw [iterElem_zero, iterElem_succ, iterElem_zero, hbot]
    | succ k ih =>
      rw [iterElem_succ, commute_apply hcomm, ih]
      exact (iterElem_succ g (k + 1)).symm
  have hffixg : f.toElementMap g.fixElement = g.fixElement := by
    rw [fixElement_eq_iSupDirected g, toElementMap_iSupDirected]
    apply le_antisymm
    * apply NeighborhoodSystem.iSupDirected_le
      intro n
      rw [hshift' n]
      exact NeighborhoodSystem.le_iSupDirected g.iterElem _ (n + 1)
    * apply NeighborhoodSystem.iSupDirected_le
      intro n
      calc g.iterElem n  <=  g.iterElem (n + 1) := iterElem_mono g (Nat.le_succ n)
        _ = f.toElementMap (g.iterElem n) := (hshift' n).symm
        _  <=  _ := NeighborhoodSystem.le_iSupDirected (fun m => f.toElementMap (g.
    iterElem m)) _ n
  apply le_antisymm
  * exact fixElement_le_of_toElementMap_le f (le_of_eq hffixg)
  * exact fixElement_le_of_toElementMap_le g (le_of_eq hgfixf)

/-- `fix(f) = fix(f?)`: the `f?`-chain `f^{2n}(False)` is cofinal in the `f`-chain `f?(
    False)`. -/
theorem fixElement_iterTwice (f : ApproximableMap V V) :
    f.fixElement = (f.comp f).fixElement := by
  apply le_antisymm
  * -- `fix f  sqsub  fix f?`: `f?False  sqsub  f^{2n}False = (f?)?False  sqsub  fix f
    ?`.
    rw [fixElement_eq_iSupDirected f]
    apply NeighborhoodSystem.iSupDirected_le
    intro n
    calc f.iterElem n  <=  f.iterElem (2 * n) := iterElem_mono f (by omega)
      _ = (f.comp f).iterElem n := (comp_self_iterElem f n).symm
      _  <=  (f.comp f).fixElement := iterElem_le_fixElement (f.comp f) n
  * -- `fix f?  sqsub  fix f`: `fix f` is a fixed point of `f?`.
    refine fixElement_le_of_toElementMap_le (f.comp f) (le_of_eq ?_)
    rw [toElementMap_comp, toElementMap_fixElement, toElementMap_fixElement]

end ApproximableMap
```

```
end

end DomainTheory.NeighbourhoodSystems
end
```

3.5.14 Exercise 4.12 — no greatest fixed point

Need an approximable $f : D \to D$ have a maximum fixed point? Give an example where there are many fixed points.

Exercise_4_12.lean

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Theorem_2_5

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Example_1_2

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems.Exercise412
open NeighborhoodSystem ApproximableMap Example12 Example12.neighborhoodSystem

section
/-- Abbreviation for Example 1.2's domain `T`. -/
abbrev T : NeighborhoodSystem Example12.Token := Example12.neighborhoodSystem

/-- `{1}  !=  {0}` as tokens. -/
theorem one_ne_zero : (Example12.one)  !=  Example12.zero := by
  intro h
  rw [Example12.one, Example12.zero, Set.ext_iff] at h
  have h1 : (1 : Example12.Token) = 0 := (h 1).mp rfl
  exact absurd h1 (by decide)

/-- `{1}  !=  Delta` as tokens. -/
theorem one_ne_master : (Example12.one)  !=  Example12.master := by
  intro h
  rw [Example12.one, Example12.master, Set.ext_iff] at h
  have h0 : (0 : Example12.Token)  in  ({1} : Set Example12.Token) := (h 0).mpr (Set.
    mem_univ 0)
  simp only [Set.mem_singleton_iff] at h0
  exact absurd h0 (by decide)

/-- `{0}  !=  Delta` as tokens. -/
theorem zero_ne_master : (Example12.zero)  !=  Example12.master := by
  intro h
  rw [Example12.zero, Example12.master, Set.ext_iff] at h
  have h1 : (1 : Example12.Token)  in  ({0} : Set Example12.Token) := (h 1).mpr (Set.
    mem_univ 1)
  simp only [Set.mem_singleton_iff] at h1
  exact absurd h1 (by decide)
end
```

```
section
/-- The two total elements are incomparable: `elemOne ? elemZero`. -/
theorem elemOne_not_le_elemZero : not  elemOne  <=  elemZero := by
  intro h
  rcases h Example12.one (Or.inr rfl) with hm | hz
  * exact one_ne_master hm
  * exact one_ne_zero hz
end

section
/-- The two total elements are incomparable: `elemZero ? elemOne`. -/
theorem elemZero_not_le_elemOne : not  elemZero  <=  elemOne := by
  intro h
  rcases h Example12.zero (Or.inr rfl) with hm | ho
  * exact zero_ne_master hm
  * exact one_ne_zero ho.symm
end

section
/- concordance: source L2259 EXERCISE 4.12 no greatest fixed point -/
/-- **Exercise 4.12 (Scott 1981, PRG-19).** Every element is a fixed point of the
    identity map
(so `I_T` has *many* fixed points: `False`, `elemZero`, `elemOne`). -/
theorem idMap_fixed (x : T.Element) : (idMap T).toElementMap x = x :=
    toElementMap_idMap x
end

section
/-- **Exercise 4.12 (Scott 1981, PRG-19).** `I_T` has no *greatest* (hence no maximum)
    fixed point:
the two total elements `elemZero`, `elemOne` are both fixed points, are incomparable,
    and no element
dominates both. -/
theorem no_greatest_fixedPoint :
    not  exists  z : T.Element, (idMap T).toElementMap z = z  /\
      forall  x, (idMap T).toElementMap x = x  ->  x  <=  z := by
  rintro <z, _, hz>
  have h0 : elemZero  <=  z := hz elemZero (idMap_fixed elemZero)
  have h1 : elemOne  <=  z := hz elemOne (idMap_fixed elemOne)
  rcases element_classification z with rfl | rfl | rfl
  * exact (bot_lt_elemZero).2 h0
  * exact elemOne_not_le_elemZero h1
  * exact elemZero_not_le_elemOne h0
end

section
set_option linter.unusedVariables false

theorem example_412 : And
  (forall
    (x :
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element DomainTheory.
    NeighbourhoodSystems.Example12.Token
        DomainTheory.NeighbourhoodSystems.Exercise412.T),
    @Eq
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element DomainTheory.
```

```
 NeighbourhoodSystems.Example12.Token
     DomainTheory.NeighbourhoodSystems.Exercise412.T)
   (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap DomainTheory.
 NeighbourhoodSystems.Example12.Token
     DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
 NeighbourhoodSystems.Exercise412.T
     DomainTheory.NeighbourhoodSystems.Exercise412.T
     (@DomainTheory.NeighbourhoodSystems.ApproximableMap.idMap DomainTheory.
 NeighbourhoodSystems.Example12.Token
       DomainTheory.NeighbourhoodSystems.Exercise412.T)
     x)
   x)
(Not
  (exists  (z :
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element DomainTheory.
 NeighbourhoodSystems.Example12.Token
     DomainTheory.NeighbourhoodSystems.Exercise412.T),
    And
      (@Eq
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
          DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
 NeighbourhoodSystems.Exercise412.T)
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
          DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
 NeighbourhoodSystems.Example12.Token
          DomainTheory.NeighbourhoodSystems.Exercise412.T DomainTheory.
 NeighbourhoodSystems.Exercise412.T
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.idMap DomainTheory.
 NeighbourhoodSystems.Example12.Token
            DomainTheory.NeighbourhoodSystems.Exercise412.T)
          z)
        z)
      (forall
        (x :
          @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
            DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
 NeighbourhoodSystems.Exercise412.T),
        @Eq
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
              DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
 NeighbourhoodSystems.Exercise412.T)
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
              DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
 NeighbourhoodSystems.Example12.Token
              DomainTheory.NeighbourhoodSystems.Exercise412.T DomainTheory.
 NeighbourhoodSystems.Exercise412.T
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap.idMap
                DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
 NeighbourhoodSystems.Exercise412.T)
              x)
            x  ->
          @LE.le
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
              DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
 NeighbourhoodSystems.Exercise412.T)
            (@Preorder.toLE
```

```
             (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
               DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
  NeighbourhoodSystems.Exercise412.T)
             (@PartialOrder.toPreorder
               (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
                 DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
  NeighbourhoodSystems.Exercise412.T)
               (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement
                 DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
  NeighbourhoodSystems.Exercise412.T)))
           x z))) :=
@And.intro
(forall
  (x :
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element DomainTheory.
  NeighbourhoodSystems.Example12.Token
      DomainTheory.NeighbourhoodSystems.Exercise412.T),
  @Eq
    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element DomainTheory.
  NeighbourhoodSystems.Example12.Token
      DomainTheory.NeighbourhoodSystems.Exercise412.T)
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap DomainTheory.
  NeighbourhoodSystems.Example12.Token
      DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
  NeighbourhoodSystems.Exercise412.T
      DomainTheory.NeighbourhoodSystems.Exercise412.T
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.idMap DomainTheory.
  NeighbourhoodSystems.Example12.Token
        DomainTheory.NeighbourhoodSystems.Exercise412.T)
      x)
    x)
(Not
  (exists  (z :
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element DomainTheory.
  NeighbourhoodSystems.Example12.Token
      DomainTheory.NeighbourhoodSystems.Exercise412.T),
    And
      (@Eq
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
          DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
  NeighbourhoodSystems.Exercise412.T)
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
          DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
  NeighbourhoodSystems.Example12.Token
          DomainTheory.NeighbourhoodSystems.Exercise412.T DomainTheory.
  NeighbourhoodSystems.Exercise412.T
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.idMap DomainTheory.
  NeighbourhoodSystems.Example12.Token
            DomainTheory.NeighbourhoodSystems.Exercise412.T)
          z)
        z)
      (forall
        (x :
          @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
            DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
```

```
    NeighbourhoodSystems.Exercise412.T),
          @Eq
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
                DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
    NeighbourhoodSystems.Exercise412.T)
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
                DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
    NeighbourhoodSystems.Example12.Token
                DomainTheory.NeighbourhoodSystems.Exercise412.T DomainTheory.
    NeighbourhoodSystems.Exercise412.T
                (@DomainTheory.NeighbourhoodSystems.ApproximableMap.idMap
                  DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
    NeighbourhoodSystems.Exercise412.T)
                x)
              x  ->
            @LE.le
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
                DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
    NeighbourhoodSystems.Exercise412.T)
              (@Preorder.toLE
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
                  DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
    NeighbourhoodSystems.Exercise412.T)
                (@PartialOrder.toPreorder
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
                    DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
    NeighbourhoodSystems.Exercise412.T)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    instPartialOrderElement
                    DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
    NeighbourhoodSystems.Exercise412.T)))
              x z)))
  DomainTheory.NeighbourhoodSystems.Exercise412.idMap_fixed
  DomainTheory.NeighbourhoodSystems.Exercise412.no_greatest_fixedPoint
end

end DomainTheory.NeighbourhoodSystems.Exercise412
end
```

3.5.15 Exercise 4.13 — avoiding the natural numbers

The proof of 4.1 uses the integers, whereas the proof of 4.6 uses 4.1. There is a hint of circularity here! It can be eliminated by the following steps:

(1) If a domain D has an element a where, for $f : D \to D$, the relation $f(a) \sqsubseteq a$ holds, then the least fixed point can be defined by

$$\mathrm{fix}(f) = \bigsqcap\{x \in |D| \mid f(x) \sqsubseteq x\}.$$

Note that $\mathrm{fix}(f) \sqsubseteq a$. (Hint: Remark that by 1.17 the formula gives a well-defined element. Call the element b. Prove that $f(b) \sqsubseteq b$ by showing that $f(b) \sqsubseteq x$ whenever $f(x) \sqsubseteq x$. Then note that $f(f(b)) \sqsubseteq f(b)$ so that $b \sqsubseteq f(b)$ also. Conclude $b = \mathrm{fix}(f)$ as least fixed point.)

(2) Remark that this proof uses only the monotonicity property of $f : |D| \to |D|$. Remark, too, that (1) can always be applied to power-set domains PA for any set A.

(3) Review the proof of 4.6 and establish by a fixed-point method that for any structured set $\langle Z, z, \cdot \rangle$ there is a unique function $s : \mathbf{N} \to Z$ such that

(i) $s(0) = z$;

(ii) $s(n^+) = s(n)^{\cdot}$, for $n \in \mathbf{N}$.

(4) Employ (3) for the proof of 4.1 by identifying $\langle \mathbf{N}, 0, {}^+ \rangle$.

Exercise_4_13.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_24

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_18

@[expose] public section

section
/- concordance: source L2261 EXERCISE 4.13 avoiding the natural numbers -/
-- Exercise413/preFix.lean
namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem

variable {a : Type*} {V : NeighborhoodSystem a}

namespace NeighborhoodSystem

/-- The set of *pre-fixed points* `{x ? f(x)  sqsub  x}` of `f`. -/
def preFix (f : V.Element  ->  V.Element) : Set V.Element := {x | f x  <=  x}

end NeighborhoodSystem

end DomainTheory.NeighbourhoodSystems
end

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem

section
variable {a : Type*} {V : NeighborhoodSystem a}

namespace NeighborhoodSystem

@[simp] theorem mem_preFix {f : V.Element  ->  V.Element} {x : V.Element} :
    x  in  preFix f  <->  f x  <=  x := Iff.rfl
```

```
/-- **Exercise 4.13(1) (Scott 1981, PRG-19).** The least fixed point of a *monotone* `f
    ` defined
purely as the greatest lower bound of the pre-fixed points, `? {x ? f(x)  sqsub  x}`.
    Well-defined as an
element by Exercise 1.18 (`sInf`); the non-emptiness witness is any `a` with `f(a)
    sqsub  a`. -/
def monoFix (f : V.Element  ->  V.Element) {a : V.Element} (ha : f a  <=  a) : V.
    Element :=
  V.sInf (preFix f) <a, ha>

/-- **Exercise 4.13(3) (Scott 1981, PRG-19).** For any structured set `<Z, z, *>` (`z :
     Z`,
`op : Z  ->  Z`) there is a *unique* `s : Nat  ->  Z` with `s(0) = z` and `s(n^+) = (s
    n)*`. This is the
choice-free recursion principle (`Nat.rec`); together with 4.13(1) it eliminates the
    circularity in
the proofs of 4.1 and 4.6. -/
theorem exists_unique_nat_rec {Z : Type*} (z : Z) (op : Z  ->  Z) :
    exists ! s : Nat  ->  Z, s 0 = z  /\  forall  n, s (n + 1) = op (s n) := by
  refine <fun n => Nat.rec z (fun _ x => op x) n, <rfl, fun _ => rfl>, ?_>
  rintro s <h0, hsucc>
  funext n
  induction n with
  | zero => exact h0
  | succ k ih => rw [hsucc k, ih]

/-- `monoFix f ha  sqsub  x` for every pre-fixed point `x` (`f(x)  sqsub  x`). -/
theorem monoFix_le_preFix (f : V.Element  ->  V.Element) {a : V.Element} (ha : f a  <=
     a)
    {x : V.Element} (hx : f x  <=  x) : monoFix f ha  <=  x :=
  V.sInf_le (preFix f) <a, ha> hx

/-- `monoFix f ha  sqsub  a`: the least fixed point lies below the given pre-fixed
    point `a`. -/
theorem monoFix_le (f : V.Element  ->  V.Element) {a : V.Element} (ha : f a  <=  a) :
    monoFix f ha  <=  a := monoFix_le_preFix f ha ha

/-- **Exercise 4.13(1) (Scott 1981, PRG-19).** `monoFix f ha` is the *least* fixed
    point: any `y`
with `f(y) = y` satisfies `b  sqsub  y`. -/
theorem monoFix_least (f : V.Element  ->  V.Element) {a : V.Element} (ha : f a  <=  a)
    {y : V.Element} (hy : f y = y) : monoFix f ha  <=  y :=
  monoFix_le_preFix f ha (le_of_eq hy)

/-- The key step: `f(b)  sqsub  b` where `b = monoFix f ha`, by monotonicity alone. -/
theorem monoFix_preFix (f : V.Element  ->  V.Element) (hf : Monotone f) {a : V.Element
    }
    (ha : f a  <=  a) : f (monoFix f ha)  <=  monoFix f ha := by
  apply V.le_sInf (preFix f) <a, ha>
  intro x hx
  exact le_trans (hf (monoFix_le_preFix f ha hx)) hx

/-- **Exercise 4.13(1) (Scott 1981, PRG-19).** `monoFix f ha` is a fixed point: `f(b) =
     b`. (After
`f(b)  sqsub  b`, monotonicity gives `f(f(b))  sqsub  f(b)`, so `f(b)` is itself a pre-
```

```
    fixed point, whence
`b  sqsub  f(b)`.) -/
theorem monoFix_isFixed (f : V.Element  ->  V.Element) (hf : Monotone f) {a : V.
    Element}
    (ha : f a  <=  a) : f (monoFix f ha) = monoFix f ha := by
  have hfb : f (monoFix f ha)  <=  monoFix f ha := monoFix_preFix f hf ha
  have hbf : monoFix f ha  <=  f (monoFix f ha) :=
    monoFix_le_preFix f ha (hf hfb)
  exact le_antisymm hfb hbf

/-- **Exercise 4.13(4) (Scott 1981, PRG-19).** Specializing 4.13(3) to the structured
    set
`<Nat, 0, ^+>`: the *unique* function `s` with `s(0) = 0` and `s(n^+) = (s n)^+` is the
     identity. This
is the iteration `n  |->  f?` underlying Theorem 4.1, now justified without circularity
    . -/
theorem nat_iterate_unique {s : Nat  ->  Nat} (h0 : s 0 = 0) (hsucc : forall  n, s (n
    + 1) = s n + 1) :
    forall  n, s n = n := by
  intro n
  induction n with
  | zero => exact h0
  | succ k ih => rw [hsucc k, ih]

end NeighborhoodSystem
end

end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
section
set_option linter.unusedVariables false

theorem example_413.{u_1, u_2} : And
  (And
    (And
      (forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem a}
        (f :
          @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V  ->
            @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V),
        @Monotone (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
            (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.Element a V)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    instPartialOrderElement a V))
            (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.Element a V)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    instPartialOrderElement a V))
            f  ->
          forall  {a : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
     V}
            (ha :
              @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
    V)
```

```
            (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
 .Element a V)
              (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem.Element a V)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 instPartialOrderElement a V)))
            (f a) a),
          @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
            (f (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.monoFix a V f
 a ha))
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.monoFix a V f a
 ha))
    (forall  {Z : Type u_2} (z : Z) (op : Z  ->  Z),
      @ExistsUnique (Nat  ->  Z) fun (s : Nat  ->  Z) =>
        And (@Eq Z (s (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0)))) z)
          (forall  (n : Nat),
            @Eq Z
              (s
                (@HAdd.hAdd Nat Nat Nat (@instHAdd Nat instAddNat) n
                  (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))))
              (op (s n)))))
 (forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
 a}
    (f :
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V  ->
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
    {a : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V}
    (ha :
      @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
        (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 Element a V)
          (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem.Element a V)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 instPartialOrderElement a V)))
        (f a) a)
    {y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V},
    @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V) (f y) y
  ->
      @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
        (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 Element a V)
          (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem.Element a V)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 instPartialOrderElement a V)))
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.monoFix a V f a ha) y
 ))
(forall  {s : Nat  ->  Nat},
  @Eq Nat (s (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0))))
      (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0)))  ->
    (forall  (n : Nat),
        @Eq Nat
          (s
            (@HAdd.hAdd Nat Nat Nat (@instHAdd Nat instAddNat) n
              (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))))
```

```
          (@HAdd.hAdd Nat Nat Nat (@instHAdd Nat instAddNat) (s n)
            (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))))  ->
      forall  (n : Nat), @Eq Nat (s n) n) :=
@And.intro
(And
  (And
    (forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem a}
      (f :
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V  ->
          @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V),
      @Monotone (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
          (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement a V))
          (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement a V))
          f  ->
        forall  {a : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
   V}
          (ha :
            @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
  V)
              (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
  .Element a V)
                (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement a V)))
              (f a) a),
          @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
            (f (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.monoFix a V f
  a ha))
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.monoFix a V f a
  ha))
    (forall  {Z : Type u_2} (z : Z) (op : Z  ->  Z),
      @ExistsUnique (Nat  ->  Z) fun (s : Nat  ->  Z) =>
        And (@Eq Z (s (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0)))) z)
          (forall  (n : Nat),
            @Eq Z
              (s
                (@HAdd.hAdd Nat Nat Nat (@instHAdd Nat instAddNat) n
                  (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))))
              (op (s n))))))
  (forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
  a}
    (f :
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V  ->
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
    {a : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V}
    (ha :
      @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
```

```
       (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 Element a V)
         (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem.Element a V)
           (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 instPartialOrderElement a V)))
       (f a) a)
   {y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V},
   @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V) (f y) y
  ->
     @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
       (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 Element a V)
         (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem.Element a V)
           (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 instPartialOrderElement a V)))
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.monoFix a V f a ha) y
 ))
(forall  {s : Nat  ->  Nat},
  @Eq Nat (s (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0))))
      (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0)))  ->
    (forall  (n : Nat),
       @Eq Nat
         (s
           (@HAdd.hAdd Nat Nat Nat (@instHAdd Nat instAddNat) n
             (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))))
         (@HAdd.hAdd Nat Nat Nat (@instHAdd Nat instAddNat) (s n)
           (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))))  ->
      forall  (n : Nat), @Eq Nat (s n) n)
(@And.intro
  (And
    (forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem a}
      (f :
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V  ->
          @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V),
      @Monotone (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
          (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem.Element a V)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 instPartialOrderElement a V))
          (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem.Element a V)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 instPartialOrderElement a V))
          f  ->
        forall  {a : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
  V}
          (ha :
            @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
 V)
              (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
 .Element a V)
                (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
```

```
NeighborhoodSystem.Element a V)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
instPartialOrderElement a V)))
            (f a) a),
        @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
          (f (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.monoFix a V f
a ha))
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.monoFix a V f a
ha))
  (forall  {Z : Type u_2} (z : Z) (op : Z  ->  Z),
    @ExistsUnique (Nat  ->  Z) fun (s : Nat  ->  Z) =>
      And (@Eq Z (s (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0)))) z)
        (forall  (n : Nat),
          @Eq Z
            (s
              (@HAdd.hAdd Nat Nat Nat (@instHAdd Nat instAddNat) n
                (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))))
            (op (s n)))))
(forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
a}
  (f :
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V  ->
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
  {a : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V}
  (ha :
    @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
      (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a V)
        (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
instPartialOrderElement a V)))
      (f a) a)
  {y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V},
  @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V) (f y) y
 ->
    @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
      (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a V)
        (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
instPartialOrderElement a V)))
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.monoFix a V f a ha) y
)
(@And.intro
  (forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
    (f :
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V  ->
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V),
    @Monotone (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
        (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
```

```
    instPartialOrderElement a V))
            (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.Element a V)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    instPartialOrderElement a V))
            f  ->
          forall  {a : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
     V}
            (ha :
              @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
    V)
                (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
    .Element a V)
                  (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.Element a V)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    instPartialOrderElement a V)))
                (f a) a),
            @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
              (f (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.monoFix a V f
    a ha))
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.monoFix a V f a
    ha))
       (forall  {Z : Type u_2} (z : Z) (op : Z  ->  Z),
         @ExistsUnique (Nat  ->  Z) fun (s : Nat  ->  Z) =>
           And (@Eq Z (s (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0)))) z)
             (forall  (n : Nat),
               @Eq Z
                 (s
                   (@HAdd.hAdd Nat Nat Nat (@instHAdd Nat instAddNat) n
                     (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))))
                 (op (s n))))
       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.monoFix_isFixed
       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.exists_unique_nat_rec)
     @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.monoFix_least)
   @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.nat_iterate_unique
end

end DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem

section
variable {a : Type*} {V : NeighborhoodSystem a}

namespace NeighborhoodSystem

/-- `chainUnion C` is the least upper bound of the chain: an upper bound `y` of every
    member
dominates it. -/
theorem chainUnion_le (C : Set V.Element) (hne : C.Nonempty) (hchain : IsChain (*  <=
     *) C)
    {y : V.Element} (hy : forall  x  in  C, x  <=  y) : V.chainUnion C hne hchain  <=
     y := by
  rintro Z <x, hxC, hxZ>
  exact hy x hxC Z hxZ
```

```
end NeighborhoodSystem
end
end DomainTheory.NeighbourhoodSystems
end
```

3.5.16 Exercise 4.14 — greatest powerset fixed point

Need a monotone function $f : PA \to PA$ always have a *maximum* fixed point?

Exercise_4_14.lean

```
module

public import Mathlib.Order.BourbakiWitt

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.Exercise414
end DomainTheory.NeighbourhoodSystems.Exercise414

section
namespace DomainTheory.NeighbourhoodSystems.Exercise414
section
variable {A : Type*}

/- concordance: source L2283 EXERCISE 4.14 greatest powerset fixed point -/
/-- **Exercise 4.14 (Scott 1981, PRG-19).** The *greatest* fixed point of a monotone `f
     : P A  ->  P A`,
constructed as the union of all post-fixed points `? {x ? x  subseteq  f(x)}`. -/
def gfpSet (f : Set A  ->  Set A) : Set A := {a | exists  x : Set A, x  subseteq  f x
     /\  a  in  x}

/-- **Exercise 4.13(2) (Scott 1981, PRG-19).** The *least* fixed point of a monotone `f
     : P A  ->  P A`,
as the intersection of all pre-fixed points `? {x ? f(x)  subseteq  x}`. -/
def lfpSet (f : Set A  ->  Set A) : Set A := {a | forall  x : Set A, f x  subseteq  x
     ->  a  in  x}

/-- `lfpSet f  subseteq  x` for every pre-fixed point `f(x)  subseteq  x`. -/
theorem lfpSet_subset (f : Set A  ->  Set A) {x : Set A} (hx : f x  subseteq  x) :
    lfpSet f  subseteq  x :=
  fun _ ha => ha x hx

/-- `f (lfpSet f)  subseteq  lfpSet f`: it is a pre-fixed point. -/
theorem f_subset_lfpSet (f : Set A  ->  Set A) (hf : Monotone f) :
    f (lfpSet f)  subseteq  lfpSet f := by
  intro a ha x hx
  exact hx (hf (lfpSet_subset f hx) ha)

/-- **Exercise 4.13(2) (Scott 1981, PRG-19).** `lfpSet f` is a fixed point: `f(b) = b`.
     -/
theorem lfpSet_isFixed (f : Set A  ->  Set A) (hf : Monotone f) :
    f (lfpSet f) = lfpSet f := by
  have hsub : f (lfpSet f)  subseteq  lfpSet f := f_subset_lfpSet f hf
```

```
  have hsup : lfpSet f  subseteq  f (lfpSet f) := lfpSet_subset f (hf hsub)
  exact Set.Subset.antisymm hsub hsup

/-- **Exercise 4.13(2) (Scott 1981, PRG-19).** `lfpSet f` is the *least* fixed point:
    any fixed
point `y = f(y)` satisfies `b  subseteq  y`. -/
theorem lfpSet_least (f : Set A  ->  Set A) {y : Set A} (hy : f y = y) :
    lfpSet f  subseteq  y := lfpSet_subset f (le_of_eq hy)

/-- Every post-fixed point `x  subseteq  f(x)` is contained in `gfpSet f`. -/
theorem subset_gfpSet (f : Set A  ->  Set A) {x : Set A} (hx : x  subseteq  f x) : x
    subseteq  gfpSet f :=
  fun _ ha => <x, hx, ha>

/-- **Exercise 4.14 (Scott 1981, PRG-19).** `gfpSet f` is the *greatest* fixed point:
    any fixed
point `y = f(y)` satisfies `y  subseteq  g`. Hence `f` *does* have a maximum fixed
    point. -/
theorem gfpSet_greatest (f : Set A  ->  Set A) {y : Set A} (hy : f y = y) :
    y  subseteq  gfpSet f := subset_gfpSet f (le_of_eq hy.symm)

/-- `gfpSet f  subseteq  f (gfpSet f)`: it is a post-fixed point. -/
theorem gfpSet_subset_f (f : Set A  ->  Set A) (hf : Monotone f) :
    gfpSet f  subseteq  f (gfpSet f) := by
  rintro a <x, hx, hax>
  exact hf (subset_gfpSet f hx) (hx hax)

/-- **Exercise 4.14 (Scott 1981, PRG-19).** `gfpSet f` is a fixed point: `f(g) = g`. -/
theorem gfpSet_isFixed (f : Set A  ->  Set A) (hf : Monotone f) :
    f (gfpSet f) = gfpSet f := by
  have hsub : gfpSet f  subseteq  f (gfpSet f) := gfpSet_subset_f f hf
  have hsup : f (gfpSet f)  subseteq  gfpSet f := subset_gfpSet f (hf hsub)
  exact Set.Subset.antisymm hsup hsub

end

section
set_option linter.unusedVariables false

theorem example_414.{u_1} : And
  (forall  {A : Type u_1} (f : Set A  ->  Set A) {y : Set A},
    @Eq (Set A) (f y) y  ->  @LE.le (Set A) (@Set.instLE A) y (@DomainTheory.
    NeighbourhoodSystems.Exercise414.gfpSet A f))
  (forall  {A : Type u_1} (f : Set A  ->  Set A),
    @Monotone (Set A) (Set A)
        (@PartialOrder.toPreorder (Set A)
          (@SemilatticeInf.toPartialOrder (Set A)
            (@Lattice.toSemilatticeInf (Set A) (@DistribLattice.toLattice (Set A) (
    @Set.instDistribLattice A)))))
        (@PartialOrder.toPreorder (Set A)
          (@SemilatticeInf.toPartialOrder (Set A)
            (@Lattice.toSemilatticeInf (Set A) (@DistribLattice.toLattice (Set A) (
    @Set.instDistribLattice A)))))
        f  ->
      @Eq (Set A) (f (@DomainTheory.NeighbourhoodSystems.Exercise414.gfpSet A f))
        (@DomainTheory.NeighbourhoodSystems.Exercise414.gfpSet A f)) :=
```

```
  @And.intro
  (forall  {A : Type u_1} (f : Set A  ->  Set A) {y : Set A},
    @Eq (Set A) (f y) y  ->  @LE.le (Set A) (@Set.instLE A) y (@DomainTheory.
    NeighbourhoodSystems.Exercise414.gfpSet A f))
  (forall  {A : Type u_1} (f : Set A  ->  Set A),
    @Monotone (Set A) (Set A)
        (@PartialOrder.toPreorder (Set A)
          (@SemilatticeInf.toPartialOrder (Set A)
            (@Lattice.toSemilatticeInf (Set A) (@DistribLattice.toLattice (Set A) (
    @Set.instDistribLattice A)))))
        (@PartialOrder.toPreorder (Set A)
          (@SemilatticeInf.toPartialOrder (Set A)
            (@Lattice.toSemilatticeInf (Set A) (@DistribLattice.toLattice (Set A) (
    @Set.instDistribLattice A)))))
        f  ->
      @Eq (Set A) (f (@DomainTheory.NeighbourhoodSystems.Exercise414.gfpSet A f))
        (@DomainTheory.NeighbourhoodSystems.Exercise414.gfpSet A f))
  @DomainTheory.NeighbourhoodSystems.Exercise414.gfpSet_greatest
  @DomainTheory.NeighbourhoodSystems.Exercise414.gfpSet_isFixed
end

end DomainTheory.NeighbourhoodSystems.Exercise414
end
```

3.5.17 Exercise 4.15 — maximal fixed points

(For set theorists.) Let $f : |D| \to |D|$ be a monotone function on (the elements of) a domain. Show that f has a *maximal* fixed point (i.e. a fixed point that cannot be extended to a larger fixed point). (Hint: By Zorn's Lemma consider a maximal chain

$$C \subseteq \{x \in |D| \mid x \sqsubseteq f(x)\},$$

and use 2.11 to remark that $\bigsqcup C \in |D|$.) Now argue that f has a least fixed point.

Exercise_4_15.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_8

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Exercise_4_13
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_18

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem
section
variable {a : Type*} {V : NeighborhoodSystem a}
namespace NeighborhoodSystem
```

```
/- concordance: source L2285 EXERCISE 4.15 maximal fixed points -/
/-- **Exercise 4.15 (Scott 1981, PRG-19).** A monotone `f : |?|  ->  |?|` has a *
    maximal* fixed point:
some `m` with `f(m) = m` that admits no strictly larger fixed point. -/
theorem exists_maximal_fixedPoint (f : V.Element  ->  V.Element) (hf : Monotone f) :
    exists  m, f m = m  /\  forall  y, f y = y  ->  m  <=  y  ->  y  <=  m := by
  -- Zorn on the set of post-fixed points.
  set S : Set V.Element := {x | x  <=  f x} with hS
  have ih : forall  c  subseteq  S, IsChain (*  <=  *) c  ->  exists  ub  in  S,
    forall  z  in  c, z  <=  ub := by
    intro c hcS hchain
    rcases c.eq_empty_or_nonempty with rfl | hne
    * refine <False, ?_, fun z hz => ((Set.mem_empty_iff_false z).mp hz).elim>
      show (False : V.Element)  <=  f False
      exact _root_.bot_le
    * refine <V.chainUnion c hne hchain, ?_, fun z hz => V.le_chainUnion c hne hchain
    hz>
      -- ` sqsup c` is post-fixed: each `x  in  c` has `x  sqsub  f x  sqsub  f( sqsup
    c)`.
      show V.chainUnion c hne hchain  <=  f (V.chainUnion c hne hchain)
      intro Z hZ
      obtain <x, hxc, hxZ> := hZ
      have hxle : x  <=  V.chainUnion c hne hchain := V.le_chainUnion c hne hchain hxc
      exact (le_trans (hcS hxc) (hf hxle)) Z hxZ
  obtain <m, hm> := zorn_le_0 S ih
  have hms : m  <=  f m := hm.1
  have hfm_le_m : f m  <=  m := hm.2 (hf hms) hms
  refine <m, le_antisymm hfm_le_m hms, fun y hy hmy => hm.2 ?_ hmy>
  exact le_of_eq hy.symm

/-- **Exercise 4.15 (Scott 1981, PRG-19).** A monotone `f : |?|  ->  |?|` has a *least*
     fixed point.
(Having a maximal fixed point `m` gives a pre-fixed point `f(m)  sqsub  m`; Exercise
    4.13(1)'s `monoFix`
then constructs the least fixed point `?{x ? f(x)  sqsub  x}`.) -/
theorem exists_least_fixedPoint (f : V.Element  ->  V.Element) (hf : Monotone f) :
    exists  b, f b = b  /\  forall  y, f y = y  ->  b  <=  y := by
  obtain <m, hfm, _> := exists_maximal_fixedPoint f hf
  exact <monoFix f (le_of_eq hfm), monoFix_isFixed f hf (le_of_eq hfm),
    fun y hy => monoFix_least f (le_of_eq hfm) hy>

end NeighborhoodSystem
end

end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
section
set_option linter.unusedVariables false

theorem example_415.{u_1} : And
  (forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
    (f :
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V  ->
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V),
    @Monotone (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
```

```
     (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement a V))
     (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement a V))
     f  ->
    exists  (b : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V),
      And (@Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V) (
  f b) b)
       (forall  (y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
  a V),
         @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V) (f
   y) y  ->
           @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
  V)
             (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
  .Element a V)
               (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
                 (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement a V)))
             b y))
(forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
  (f :
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V  ->
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V),
  @Monotone (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
     (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
     (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement a V))
     (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement a V))
     f  ->
    exists  (m : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V),
      And (@Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V) (
  f m) m)
       (forall  (y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
  a V),
         @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V) (f
   y) y  ->
           @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
  V)
             (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
               (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
                 (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement a V)))
             m y  ->
```

```
             @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
  a V)
               (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
                 (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
                   (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement a V)))
               y m)) :=
@And.intro
(forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
  (f :
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V  ->
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V),
  @Monotone (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
      (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement a V))
      (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement a V))
      f  ->
    exists  (b : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V),
      And (@Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V) (
  f b) b)
        (forall  (y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
  a V),
          @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V) (f
   y) y  ->
            @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
  V)
              (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
  .Element a V)
                (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement a V)))
              b y))
(forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
  (f :
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V  ->
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V),
  @Monotone (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
      (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement a V))
      (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement a V))
      f  ->
```

```
      exists  (m : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V),
        And (@Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V) (
    f m) m)
          (forall  (y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
    a V),
            @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V) (f
     y) y  ->
              @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
    V)
                  (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.Element a V)
                    (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.Element a V)
                      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    instPartialOrderElement a V)))
                m y  ->
              @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
    a V)
                  (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.Element a V)
                    (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.Element a V)
                      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    instPartialOrderElement a V)))
                y m))
  @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.exists_least_fixedPoint
  @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.exists_maximal_fixedPoint
end

end DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem

section
variable {a : Type*} {V : NeighborhoodSystem a}

namespace NeighborhoodSystem

/-- **Exercise 4.16(1) (Scott 1981, PRG-19).** Scott's formula: for a non-empty set `S`
     of fixed
points, `f(?S)  sqsub  ?S`. (`f(?S)  sqsub  f(s) = s` for each `s  in  S`, then take
    the glb.) -/
theorem f_sInf_le (f : V.Element  ->  V.Element) (hf : Monotone f) (S : Set V.Element)
    (hS : S.Nonempty) (hfix : forall  s  in  S, f s = s) : f (V.sInf S hS)  <=  V.sInf
     S hS := by
  apply V.le_sInf
  intro s hsS
  calc f (V.sInf S hS)  <=  f s := hf (V.sInf_le S hS hsS)
    _ = s := hfix s hsS

end NeighborhoodSystem
end
end DomainTheory.NeighbourhoodSystems
end
```

3.5.18 Exercise 4.16 — optimal fixed point

(For fixed-point nuts). Show that a monotone function as in 4.15 has an "optimal" fixed point in the sense that it is the greatest fixed point below all the maximal fixed points and at the same time it is the largest fixed point consistent with all other fixed points. *Consistency* for sets of elements means having a common upper bound. (Hint: Follow these steps:

(1) Show that any non-empty set S of fixed points has a largest fixed point below by using the formula

$$f(\bigsqcap S) \sqsubseteq \bigsqcap S$$

and finding the least fixed point over $\bigsqcap S$.

(2) Letting a be the fixed point of (1) constructed from the set of maximal fixed points, remark that a is consistent with any other fixed point $x = f(x)$, since x can be extended to a maximal one. Suppose b is consistent with all fixed points, then $b \sqsubseteq y$ if y is maximal. (Why?).)

Exercise_4_16.lean

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Exercise_4_15

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Exercise_4_13

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem
section
variable {a : Type*} {V : NeighborhoodSystem a}
namespace NeighborhoodSystem

/- concordance: source L2293 EXERCISE 4.16 optimal fixed point -/
/-- **Exercise 4.16 (Scott 1981, PRG-19).** The *optimal* fixed point associated with
    a non-empty
set `S` of fixed points: the least fixed point sitting below `?S` (Exercise 4.13(1)
    applied to the
pre-fixed point `?S`). -/
def optimalFix (f : V.Element  ->  V.Element) (hf : Monotone f) (S : Set V.Element) (
    hS : S.Nonempty)
    (hfix : forall  s  in  S, f s = s) : V.Element :=
  monoFix f (f_sInf_le f hf S hS hfix)

/-- `optimalFix S` is a fixed point. -/
theorem optimalFix_isFixed (f : V.Element  ->  V.Element) (hf : Monotone f) (S : Set V
    .Element)
    (hS : S.Nonempty) (hfix : forall  s  in  S, f s = s) :
    f (optimalFix f hf S hS hfix) = optimalFix f hf S hS hfix :=
  monoFix_isFixed f hf _
```

```
/-- `optimalFix S  sqsub  ?S`. -/
theorem optimalFix_le_sInf (f : V.Element  ->  V.Element) (hf : Monotone f) (S : Set V
    .Element)
    (hS : S.Nonempty) (hfix : forall  s  in  S, f s = s) :
    optimalFix f hf S hS hfix  <=  V.sInf S hS :=
  monoFix_le f _

/-- **Exercise 4.16 (Scott 1981, PRG-19).** `optimalFix S` lies below every member of `
    S`. -/
theorem optimalFix_le (f : V.Element  ->  V.Element) (hf : Monotone f) (S : Set V.
    Element)
    (hS : S.Nonempty) (hfix : forall  s  in  S, f s = s) {s : V.Element} (hs : s  in
    S) :
    optimalFix f hf S hS hfix  <=  s :=
  le_trans (optimalFix_le_sInf f hf S hS hfix) (V.sInf_le S hS hs)

/-- **Exercise 4.16 (Scott 1981, PRG-19).** `optimalFix S` is *consistent* with every
    member of `S`
(they share a common upper bound, namely `s` itself). -/
theorem optimalFix_consistent (f : V.Element  ->  V.Element) (hf : Monotone f) (S :
    Set V.Element)
    (hS : S.Nonempty) (hfix : forall  s  in  S, f s = s) {s : V.Element} (hs : s  in
    S) :
    exists  ub, optimalFix f hf S hS hfix  <=  ub  /\  s  <=  ub :=
  <s, optimalFix_le f hf S hS hfix hs, le_refl s>

end NeighborhoodSystem
end

end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
section
set_option linter.unusedVariables false

theorem example_416.{u_1} : And
  (And
    (forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
    a}
      (f :
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V  ->
          @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V),
      @Monotone (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
          (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.Element a V)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    instPartialOrderElement a V))
          (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.Element a V)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    instPartialOrderElement a V))
          f  ->
        forall  (S : Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    Element a V))
          (hS : @Set.Nonempty (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    Element a V) S),
```

```
      (forall  (s : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
a V),
          @Membership.mem (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a V)
              (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
a V))
              (@Set.instMembership (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)) S s  ->
            @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V
) (f s) s)  ->
        @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
          (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a V)
            (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
instPartialOrderElement a V)))
          (f (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.sInf a V S hS)
)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.sInf a V S hS))
 (forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
a}
   (f :
     @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V  ->
       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
   (hf :
     @Monotone (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
       (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
         (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
instPartialOrderElement a V))
       (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
         (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
instPartialOrderElement a V))
       f)
   (S : Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V))
   (hS : @Set.Nonempty (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a V) S)
   (hfix :
     forall  (s : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V
),
       @Membership.mem (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a V)
           (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
)
           (@Set.instMembership (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)) S s  ->
         @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V) (f
 s) s)
   {s : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V},
   @Membership.mem (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
 V)
       (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V))
       (@Set.instMembership (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
```

```
  Element a V)) S s  ->
      exists  (ub : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
  V),
        And
          (@LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V
  )
            (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  Element a V)
              (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement a V)))
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.optimalFix a V f
  hf S hS hfix) ub)
          (@LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V
  )
            (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  Element a V)
              (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement a V)))
            s ub)))
(forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
  (f :
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V  ->
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
  (hf :
    @Monotone (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
      (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement a V))
      (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement a V))
      f)
  (S : Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V))
  (hS : @Set.Nonempty (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
   a V) S)
  (hfix :
    forall  (s : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V),
      @Membership.mem (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
   a V)
        (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V))
        (@Set.instMembership (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)) S s  ->
      @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V) (f s
  ) s)
  {s : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V},
  @Membership.mem (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V
  )
    (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V))
    (@Set.instMembership (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
```

```
  Element a V)) S s  ->
    @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
      (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
   a V)
        (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement a V)))
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.optimalFix a V f hf S
  hS hfix) s) :=
@And.intro
(And
  (forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
  a}
    (f :
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V  ->
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V),
    @Monotone (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
        (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement a V))
        (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement a V))
        f  ->
      forall  (S : Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  Element a V))
        (hS : @Set.Nonempty (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  Element a V) S),
        (forall  (s : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
  a V),
           @Membership.mem (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  Element a V)
                (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
  a V))
                (@Set.instMembership (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)) S s  ->
             @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V
  ) (f s) s)  ->
          @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
            (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  Element a V)
              (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement a V)))
            (f (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.sInf a V S hS)
  )
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.sInf a V S hS))
  (forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
  a}
    (f :
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V  ->
```

```
       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
    (hf :
      @Monotone (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
        (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement a V))
        (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement a V))
        f)
    (S : Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V))
    (hS : @Set.Nonempty (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  Element a V) S)
    (hfix :
      forall  (s : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V
  ),
        @Membership.mem (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  Element a V)
          (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
  )
          (@Set.instMembership (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)) S s  ->
         @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V) (f
   s) s)
    {s : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V},
    @Membership.mem (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
   V)
        (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V))
        (@Set.instMembership (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  Element a V)) S s  ->
      exists  (ub : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
  V),
        And
          (@LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V
  )
            (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  Element a V)
              (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement a V)))
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.optimalFix a V f
  hf S hS hfix) ub)
          (@LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V
  )
            (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  Element a V)
              (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement a V)))
            s ub)))
(forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
```

```
 (f :
   @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V  ->
     @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
 (hf :
   @Monotone (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
     (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
     (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
instPartialOrderElement a V))
     (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
instPartialOrderElement a V))
     f)
 (S : Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V))
 (hS : @Set.Nonempty (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
  a V) S)
 (hfix :
   forall  (s : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V),
     @Membership.mem (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
  a V)
         (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V))
         (@Set.instMembership (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)) S s  ->
       @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V) (f s
) s)
 {s : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V},
 @Membership.mem (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V
)
     (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V))
     (@Set.instMembership (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a V)) S s  ->
   @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
     (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
  a V)
       (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
         (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
instPartialOrderElement a V)))
     (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.optimalFix a V f hf S
 hS hfix) s)
(@And.intro
 (forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
 a}
   (f :
     @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V  ->
       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V),
   @Monotone (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
       (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
         (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
instPartialOrderElement a V))
       (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
```

```
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
instPartialOrderElement a V))
      f  ->
    forall  (S : Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a V))
      (hS : @Set.Nonempty (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a V) S),
      (forall  (s : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
a V),
         @Membership.mem (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a V)
           (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
a V))
           (@Set.instMembership (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)) S s  ->
          @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V
) (f s) s)  ->
        @LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
          (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a V)
            (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
instPartialOrderElement a V)))
          (f (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.sInf a V S hS)
)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.sInf a V S hS))
 (forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
a}
   (f :
     @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V  ->
       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
   (hf :
     @Monotone (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
       (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
         (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
instPartialOrderElement a V))
       (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
         (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
instPartialOrderElement a V))
       f)
   (S : Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V))
   (hS : @Set.Nonempty (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a V) S)
   (hfix :
     forall  (s : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V
),
       @Membership.mem (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a V)
         (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
)
         (@Set.instMembership (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)) S s  ->
```

```
          @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V) (f
    s) s)
      {s : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V},
      @Membership.mem (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
    V)
        (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V))
        (@Set.instMembership (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  Element a V)) S s  ->
      exists  (ub : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
  V),
        And
          (@LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V
  )
            (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  Element a V)
              (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement a V)))
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.optimalFix a V f
  hf S hS hfix) ub)
          (@LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V
  )
            (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  Element a V)
              (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement a V)))
            s ub))
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.f_sInf_le
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.optimalFix_consistent)
  @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.optimalFix_le
end

end DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
end
```

3.5.19 Exercise 4.17 — generated submonoid

(For algebraists). Suppose $\langle S, 1, \cdot \rangle$ is a semi-group with unit (sometimes called a monoid). Remark that PS is a domain. For $a, b \in S$, what is the least $x \in PS$ such that

$$x = \{1\} \cup \{a, b\} \cup x \cdot x,$$

where in general for $x, y \subseteq S$

$$x \cdot y = \{t \cdot u \mid t \in x \text{ and } u \in y\}?$$

Need the fixed point be unique?

Exercise_4_17.lean

```
module

public import Mathlib.Algebra.Group.Nat.Defs
public import Mathlib.Algebra.Group.Pointwise.Set.Basic
public import Mathlib.Algebra.Group.Submonoid.Basic
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Exercise_4_14

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.Exercise417
open Pointwise
open DomainTheory.NeighbourhoodSystems.Exercise414

section
variable {S : Type*} [Monoid S]

/-- Scott's operator `F(x) = {1}  U  {a, b}  U  x * x` on `P S`. -/
def F (a b : S) (x : Set S) : Set S := {1}  U  {a, b}  U  x * x

/-- `{1}  U  {a, b}  subseteq  F a b x`. -/
theorem base_subset_F (a b : S) (x : Set S) : {1}  U  {a, b}  subseteq  F a b x :=
  Set.subset_union_left

theorem one_mem_F (a b : S) (x : Set S) : (1 : S)  in  F a b x :=
  base_subset_F a b x (Set.mem_union_left _ rfl)

theorem self_mul_subset_F (a b : S) (x : Set S) : x * x  subseteq  F a b x :=
  Set.subset_union_right

/-- A pre-fixed point of `F` is a submonoid containing `{a, b}`: it contains `1` and is
     closed under
multiplication and contains `a, b`. Packaged as the submonoid with that carrier. -/
def preFixSubmonoid (a b : S) {x : Set S} (hx : F a b x  subseteq  x) : Submonoid S
    where
  carrier := x
  one_mem' := hx (one_mem_F a b x)
  mul_mem' := by
    intro s t hs ht
    exact hx (self_mul_subset_F a b x (Set.mul_mem_mul hs ht))

theorem F_monotone (a b : S) : Monotone (F a b) := by
  intro x y hxy
  exact Set.union_subset_union (le_refl _) (Set.mul_subset_mul hxy hxy)

/-- `{1}` is a fixed point of `F 1 1`: `{1}  U  {1, 1}  U  {1}*{1} = {1}`. -/
theorem F_one_one_singleton : F (1 : Nat) 1 ({1} : Set Nat) = {1} := by
  unfold F
  rw [Set.singleton_mul_singleton]
  simp

/-- In any monoid, `Set.univ` is a fixed point of `F a b`: `s = 1 * s  in  univ * univ
    `. -/
theorem F_univ (a b : S) : F a b (Set.univ : Set S) = Set.univ := by
  apply Set.Subset.antisymm (Set.subset_univ _)
  intro s _
  exact self_mul_subset_F a b _ (Set.mem_mul.mpr <1, trivial, s, trivial, one_mul s>)
```

```
theorem a_mem_F (a b : S) (x : Set S) : a  in  F a b x :=
  base_subset_F a b x (Set.mem_union_right _ (Set.mem_insert _ _))

theorem b_mem_F (a b : S) (x : Set S) : b  in  F a b x :=
  base_subset_F a b x (Set.mem_union_right _ (Set.mem_insert_of_mem _ rfl))
end
end DomainTheory.NeighbourhoodSystems.Exercise417

section
namespace DomainTheory.NeighbourhoodSystems.Exercise417
open Pointwise
open DomainTheory.NeighbourhoodSystems.Exercise414
section
variable {S : Type*} [Monoid S]

/- concordance: source L2305 EXERCISE 4.17 generated submonoid -/
/-- **Exercise 4.17 (Scott 1981, PRG-19).** The fixed point need **not** be unique:
    over `S = Nat`
(under `*`) with `a = b = 1`, both `{1}` (the least solution) and `Set.univ` solve the
    equation, and
they are distinct. -/
theorem fixedPoint_not_unique :
    exists  x y : Set Nat, F 1 1 x = x  /\  F 1 1 y = y  /\  x  !=  y := by
  refine <{1}, Set.univ, F_one_one_singleton, F_univ 1 1, ?_>
  intro h
  have h2 : (2 : Nat)  in  ({1} : Set Nat) := by rw [h]; exact Set.mem_univ 2
  simp at h2

/-- The least solution `lfpSet (F a b)` indeed solves Scott's equation `x = {1}  U  {a,
     b}  U  x * x`. -/
theorem leastSolution_eq (a b : S) :
    F a b (lfpSet (F a b)) = lfpSet (F a b) :=
  lfpSet_isFixed (F a b) (F_monotone a b)

/-- **Exercise 4.17 (Scott 1981, PRG-19).** The least solution of `x = {1}  U  {a, b}
    U  x * x` is the
submonoid generated by `{a, b}` (all finite products of `a`s and `b`s, including `1`).
    -/
theorem lfpSet_eq_closure (a b : S) :
    lfpSet (F a b) = (Submonoid.closure ({a, b} : Set S) : Set S) := by
  apply Set.Subset.antisymm
  * -- `lfpSet  subseteq  closure`: the closure is a (pre-)fixed point of `F`.
    have hclosed : F a b (Submonoid.closure ({a, b} : Set S))  subseteq
        (Submonoid.closure ({a, b} : Set S) : Set S) := by
      apply Set.union_subset
      * apply Set.union_subset
        * rintro s rfl; exact (Submonoid.closure ({a, b} : Set S)).one_mem
        * intro s hs
          rcases hs with rfl | rfl
          * exact Submonoid.subset_closure (Set.mem_insert _ _)
          * exact Submonoid.subset_closure (Set.mem_insert_of_mem _ rfl)
      * rintro s <t, ht, u, hu, rfl>
        exact (Submonoid.closure ({a, b} : Set S)).mul_mem ht hu
    exact lfpSet_subset (F a b) hclosed
  * -- `closure  subseteq  lfpSet`: `lfpSet` is a submonoid containing `{a, b}`.
```

```
    have hfix : F a b (lfpSet (F a b))  subseteq  lfpSet (F a b) :=
      le_of_eq (leastSolution_eq a b)
    have ha : a  in  lfpSet (F a b) := hfix (a_mem_F a b _)
    have hb : b  in  lfpSet (F a b) := hfix (b_mem_F a b _)
    have hmem : ({a, b} : Set S)  subseteq  lfpSet (F a b) := by
      intro s hs
      rcases Set.mem_insert_iff.mp hs with h | h
      * rw [h]; exact ha
      * rw [Set.mem_singleton_iff.mp h]; exact hb
    exact (Submonoid.closure_le (S := preFixSubmonoid a b hfix)).mpr hmem

end

section
set_option linter.unusedVariables false

theorem example_417.{u_1} : And
  (exists  (x : Set Nat) (y : Set Nat),
    And
      (@Eq (Set Nat)
        (@DomainTheory.NeighbourhoodSystems.Exercise417.F Nat Nat.instMonoid
          (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))
          (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))) x)
        x)
      (And
        (@Eq (Set Nat)
          (@DomainTheory.NeighbourhoodSystems.Exercise417.F Nat Nat.instMonoid
            (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))
            (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))) y)
          y)
        (@Ne (Set Nat) x y)))
  (forall  {S : Type u_1} [inst : Monoid S] (a b : S),
    @Eq (Set S)
      (@DomainTheory.NeighbourhoodSystems.Exercise414.lfpSet S
        (@DomainTheory.NeighbourhoodSystems.Exercise417.F S inst a b))
      (@SetLike.coe (@Submonoid S (@Monoid.toMulOneClass S inst)) S
        (@Submonoid.instSetLike S (@Monoid.toMulOneClass S inst))
        (@Submonoid.closure S (@Monoid.toMulOneClass S inst)
          (@Insert.insert S (Set S) (@Set.instInsert S) a
            (@Singleton.singleton S (Set S) (@Set.instSingletonSet S) b))))) :=
  @And.intro
  (exists  (x : Set Nat) (y : Set Nat),
    And
      (@Eq (Set Nat)
        (@DomainTheory.NeighbourhoodSystems.Exercise417.F Nat Nat.instMonoid
          (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))
          (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))) x)
        x)
      (And
        (@Eq (Set Nat)
          (@DomainTheory.NeighbourhoodSystems.Exercise417.F Nat Nat.instMonoid
            (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))
            (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))) y)
          y)
        (@Ne (Set Nat) x y)))
  (forall  {S : Type u_1} [inst : Monoid S] (a b : S),
```

```
    @Eq (Set S)
      (@DomainTheory.NeighbourhoodSystems.Exercise414.lfpSet S
        (@DomainTheory.NeighbourhoodSystems.Exercise417.F S inst a b))
      (@SetLike.coe (@Submonoid S (@Monoid.toMulOneClass S inst)) S
        (@Submonoid.instSetLike S (@Monoid.toMulOneClass S inst))
        (@Submonoid.closure S (@Monoid.toMulOneClass S inst)
          (@Insert.insert S (Set S) (@Set.instInsert S) a
            (@Singleton.singleton S (Set S) (@Set.instSingletonSet S) b)))))
  DomainTheory.NeighbourhoodSystems.Exercise417.fixedPoint_not_unique
  @DomainTheory.NeighbourhoodSystems.Exercise417.lfpSet_eq_closure
end

end DomainTheory.NeighbourhoodSystems.Exercise417
end
```

3.5.20 Exercise 4.18 — checking the number domain

In Example 4.3 there are many unproved assertions about N and F. These should be checked. In particular, the isomorphism theorem of 4.6 could be proved by constructing a simple domain M from $\mathbf{M}$ in the way N is constructed from $\mathbf{N}$.

Exercise_4_18.lean

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Example_4_3

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.Exercise418
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap Example43

section
/-- Distinct singletons are disjoint. -/
theorem singleton_inter_eq_empty {n m : Nat} (hmn : m  !=  n) :
    ({n} : Set Nat)  I  {m} = {} := by
  ext k
  simp only [Set.mem_inter_iff, Set.mem_singleton_iff, Set.mem_empty_iff_false,
    iff_false, not_and]
  rintro rfl h2
  exact hmn h2.symm
end

end DomainTheory.NeighbourhoodSystems.Exercise418

section
namespace DomainTheory.NeighbourhoodSystems.Exercise418
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap Example43
section
/- concordance: source L2321 EXERCISE 4.18 checking the number domain -/
/-- **Exercise 4.18 (Scott 1981, PRG-19).** The numerals are distinct: `n = m ? n = m`.
     -/
theorem natElem_injective {n m : Nat} (h : natElem n = natElem m) : n = m := by
  have hmem : (natElem m).mem {n} := h  |>  (mem_natElem_iff.mpr (Or.inr rfl))
```

```
  rw [mem_natElem_iff] at hmem
  rcases hmem with hu | hs
  * exact absurd hu (singleton_ne_univ n)
  * exact singleton_nat_inj hs
end

section
/-- **Exercise 4.18 (Scott 1981, PRG-19).** `0` is not a numeral successor. -/
theorem natElem_zero_ne_succ (k : Nat) : natElem 0  !=  natElem (k + 1) := fun h =>
  Nat.succ_ne_zero k (natElem_injective h).symm
end

section
/-- `{}` is not a neighbourhood of `N`. -/
theorem not_N_mem_empty : not  N.mem ({} : Set Nat) := by
  rw [N_mem]
  rintro (h | <n, h>)
  * exact Set.empty_ne_univ h
  * exact (Set.singleton_ne_empty n) h.symm
end

section
/-- **Exercise 4.18 (Scott 1981, PRG-19).** Element classification: every element of `|
    N|` is either
the bottom `False` or a total element `n`. So `N` is the flat domain over `Nat`. -/
theorem element_classification (x : N.Element) :
    x = N.bot  \/  exists  n, x = natElem n := by
  by_cases h : exists  n, x.mem {n}
  * obtain <n, hn> := h
    refine Or.inr <n, ?_>
    apply Element.ext
    intro Y
    rw [mem_natElem_iff]
    constructor
    * intro hY
      rcases N_mem.mp (x.sub hY) with rfl | <m, rfl>
      * exact Or.inl rfl
      * by_cases hmn : m = n
        * subst hmn; exact Or.inr rfl
        * exfalso
          have hmem := x.inter_mem hn hY
          rw [singleton_inter_eq_empty hmn] at hmem
          exact not_N_mem_empty (x.sub hmem)
    * rintro (rfl | rfl)
      * exact x.master_mem
      * exact hn
  * refine Or.inl ?_
    apply Element.ext
    intro Y
    rw [N_bot_mem]
    constructor
    * intro hY
      rcases N_mem.mp (x.sub hY) with rfl | <m, rfl>
      * rfl
      * exact absurd hY (fun hm => h <m, hm>)
    * rintro rfl
```

```
      exact x.master_mem
end

section
/-- **Exercise 4.18 (Scott 1981, PRG-19).** `succ` is injective on the total elements.
    -/
theorem succMap_injective {n m : Nat}
    (h : succMap.toElementMap (natElem n) = succMap.toElementMap (natElem m)) : n = m
    := by
  rw [succMap_natElem, succMap_natElem] at h
  have := natElem_injective h
  omega
end

section
/-- **Exercise 4.18 (Scott 1981, PRG-19).** `0` is not in the image of `succ` -- Peano'
    s
"zero is not a successor". -/
theorem zero_ne_succMap (n : Nat) : natElem 0  !=  succMap.toElementMap (natElem n) :=
     by
  rw [succMap_natElem]
  exact natElem_zero_ne_succ n
end

section
set_option linter.unusedVariables false

theorem example_418 : And
  (And
    (And
      (And
        (forall
          (x :
            @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element Nat
              DomainTheory.NeighbourhoodSystems.Example43.N),
          Or
            (@Eq
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element Nat
                DomainTheory.NeighbourhoodSystems.Example43.N)
              x
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bot Nat
                DomainTheory.NeighbourhoodSystems.Example43.N))
            (exists  (n : Nat),
              @Eq
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element Nat
                  DomainTheory.NeighbourhoodSystems.Example43.N)
                x (DomainTheory.NeighbourhoodSystems.Example43.natElem n)))
        (forall  {n m : Nat},
          @Eq
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element Nat
                DomainTheory.NeighbourhoodSystems.Example43.N)
              (DomainTheory.NeighbourhoodSystems.Example43.natElem n)
              (DomainTheory.NeighbourhoodSystems.Example43.natElem m)  ->
            @Eq Nat n m))
      (forall  (k : Nat),
        @Ne
```

```
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element Nat
          DomainTheory.NeighbourhoodSystems.Example43.N)
        (DomainTheory.NeighbourhoodSystems.Example43.natElem
          (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0))))
        (DomainTheory.NeighbourhoodSystems.Example43.natElem
          (@HAdd.hAdd Nat Nat Nat (@instHAdd Nat instAddNat) k
            (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))))))
  (forall  {n m : Nat},
    @Eq
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element Nat
          DomainTheory.NeighbourhoodSystems.Example43.N)
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap Nat Nat
          DomainTheory.NeighbourhoodSystems.Example43.N DomainTheory.
  NeighbourhoodSystems.Example43.N
          DomainTheory.NeighbourhoodSystems.Example43.succMap (DomainTheory.
  NeighbourhoodSystems.Example43.natElem n))
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap Nat Nat
          DomainTheory.NeighbourhoodSystems.Example43.N DomainTheory.
  NeighbourhoodSystems.Example43.N
          DomainTheory.NeighbourhoodSystems.Example43.succMap
          (DomainTheory.NeighbourhoodSystems.Example43.natElem m))  ->
      @Eq Nat n m))
(forall  (n : Nat),
  @Ne
    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element Nat DomainTheory.
  NeighbourhoodSystems.Example43.N)
    (DomainTheory.NeighbourhoodSystems.Example43.natElem (@OfNat.ofNat Nat (nat_lit
  0) (instOfNatNat (nat_lit 0))))
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap Nat Nat
      DomainTheory.NeighbourhoodSystems.Example43.N DomainTheory.
  NeighbourhoodSystems.Example43.N
      DomainTheory.NeighbourhoodSystems.Example43.succMap (DomainTheory.
  NeighbourhoodSystems.Example43.natElem n))) :=
@And.intro
(And
  (And
    (And
      (forall
        (x :
          @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element Nat
            DomainTheory.NeighbourhoodSystems.Example43.N),
        Or
          (@Eq
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element Nat
              DomainTheory.NeighbourhoodSystems.Example43.N)
            x
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bot Nat
              DomainTheory.NeighbourhoodSystems.Example43.N))
          (exists  (n : Nat),
            @Eq
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element Nat
                DomainTheory.NeighbourhoodSystems.Example43.N)
              x (DomainTheory.NeighbourhoodSystems.Example43.natElem n)))
      (forall  {n m : Nat},
        @Eq
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element Nat
```

```
              DomainTheory.NeighbourhoodSystems.Example43.N)
            (DomainTheory.NeighbourhoodSystems.Example43.natElem n)
            (DomainTheory.NeighbourhoodSystems.Example43.natElem m)  ->
          @Eq Nat n m))
    (forall  (k : Nat),
      @Ne
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element Nat
          DomainTheory.NeighbourhoodSystems.Example43.N)
        (DomainTheory.NeighbourhoodSystems.Example43.natElem
          (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0))))
        (DomainTheory.NeighbourhoodSystems.Example43.natElem
          (@HAdd.hAdd Nat Nat Nat (@instHAdd Nat instAddNat) k
            (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))))))
  (forall  {n m : Nat},
    @Eq
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element Nat
          DomainTheory.NeighbourhoodSystems.Example43.N)
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap Nat Nat
          DomainTheory.NeighbourhoodSystems.Example43.N DomainTheory.
 NeighbourhoodSystems.Example43.N
          DomainTheory.NeighbourhoodSystems.Example43.succMap (DomainTheory.
 NeighbourhoodSystems.Example43.natElem n))
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap Nat Nat
          DomainTheory.NeighbourhoodSystems.Example43.N DomainTheory.
 NeighbourhoodSystems.Example43.N
          DomainTheory.NeighbourhoodSystems.Example43.succMap
          (DomainTheory.NeighbourhoodSystems.Example43.natElem m))  ->
      @Eq Nat n m))
(forall  (n : Nat),
  @Ne
    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element Nat DomainTheory.
 NeighbourhoodSystems.Example43.N)
    (DomainTheory.NeighbourhoodSystems.Example43.natElem (@OfNat.ofNat Nat (nat_lit
 0) (instOfNatNat (nat_lit 0))))
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap Nat Nat
      DomainTheory.NeighbourhoodSystems.Example43.N DomainTheory.
 NeighbourhoodSystems.Example43.N
      DomainTheory.NeighbourhoodSystems.Example43.succMap (DomainTheory.
 NeighbourhoodSystems.Example43.natElem n)))
(@And.intro
  (And
    (And
      (forall
        (x :
          @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element Nat
            DomainTheory.NeighbourhoodSystems.Example43.N),
        Or
          (@Eq
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element Nat
              DomainTheory.NeighbourhoodSystems.Example43.N)
            x
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bot Nat
              DomainTheory.NeighbourhoodSystems.Example43.N))
          (exists  (n : Nat),
            @Eq
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element Nat
```

```
              DomainTheory.NeighbourhoodSystems.Example43.N)
            x (DomainTheory.NeighbourhoodSystems.Example43.natElem n)))
    (forall  {n m : Nat},
      @Eq
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element Nat
            DomainTheory.NeighbourhoodSystems.Example43.N)
          (DomainTheory.NeighbourhoodSystems.Example43.natElem n)
          (DomainTheory.NeighbourhoodSystems.Example43.natElem m)  ->
        @Eq Nat n m))
  (forall  (k : Nat),
    @Ne
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element Nat
        DomainTheory.NeighbourhoodSystems.Example43.N)
      (DomainTheory.NeighbourhoodSystems.Example43.natElem
        (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0))))
      (DomainTheory.NeighbourhoodSystems.Example43.natElem
        (@HAdd.hAdd Nat Nat Nat (@instHAdd Nat instAddNat) k
          (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))))))
(forall  {n m : Nat},
  @Eq
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element Nat
        DomainTheory.NeighbourhoodSystems.Example43.N)
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap Nat Nat
        DomainTheory.NeighbourhoodSystems.Example43.N DomainTheory.
NeighbourhoodSystems.Example43.N
        DomainTheory.NeighbourhoodSystems.Example43.succMap (DomainTheory.
NeighbourhoodSystems.Example43.natElem n))
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap Nat Nat
        DomainTheory.NeighbourhoodSystems.Example43.N DomainTheory.
NeighbourhoodSystems.Example43.N
        DomainTheory.NeighbourhoodSystems.Example43.succMap
        (DomainTheory.NeighbourhoodSystems.Example43.natElem m))  ->
    @Eq Nat n m)
(@And.intro
  (And
    (forall
      (x :
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element Nat
          DomainTheory.NeighbourhoodSystems.Example43.N),
      Or
        (@Eq
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element Nat
            DomainTheory.NeighbourhoodSystems.Example43.N)
          x
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bot Nat
            DomainTheory.NeighbourhoodSystems.Example43.N))
        (exists  (n : Nat),
          @Eq
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element Nat
              DomainTheory.NeighbourhoodSystems.Example43.N)
            x (DomainTheory.NeighbourhoodSystems.Example43.natElem n)))
    (forall  {n m : Nat},
      @Eq
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element Nat
            DomainTheory.NeighbourhoodSystems.Example43.N)
          (DomainTheory.NeighbourhoodSystems.Example43.natElem n)
```

```
            (DomainTheory.NeighbourhoodSystems.Example43.natElem m)  ->
          @Eq Nat n m))
      (forall  (k : Nat),
        @Ne
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element Nat
            DomainTheory.NeighbourhoodSystems.Example43.N)
          (DomainTheory.NeighbourhoodSystems.Example43.natElem
            (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0))))
          (DomainTheory.NeighbourhoodSystems.Example43.natElem
            (@HAdd.hAdd Nat Nat Nat (@instHAdd Nat instAddNat) k
              (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))))))
      (@And.intro
        (forall
          (x :
            @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element Nat
              DomainTheory.NeighbourhoodSystems.Example43.N),
          Or
            (@Eq
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element Nat
                DomainTheory.NeighbourhoodSystems.Example43.N)
              x
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bot Nat
                DomainTheory.NeighbourhoodSystems.Example43.N))
            (exists  (n : Nat),
              @Eq
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element Nat
                  DomainTheory.NeighbourhoodSystems.Example43.N)
                x (DomainTheory.NeighbourhoodSystems.Example43.natElem n)))
        (forall  {n m : Nat},
          @Eq
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element Nat
              DomainTheory.NeighbourhoodSystems.Example43.N)
            (DomainTheory.NeighbourhoodSystems.Example43.natElem n)
            (DomainTheory.NeighbourhoodSystems.Example43.natElem m)  ->
          @Eq Nat n m)
        DomainTheory.NeighbourhoodSystems.Exercise418.element_classification
        @DomainTheory.NeighbourhoodSystems.Exercise418.natElem_injective)
      DomainTheory.NeighbourhoodSystems.Exercise418.natElem_zero_ne_succ)
    @DomainTheory.NeighbourhoodSystems.Exercise418.succMap_injective)
  DomainTheory.NeighbourhoodSystems.Exercise418.zero_ne_succMap
end

end DomainTheory.NeighbourhoodSystems.Exercise418
end
```

3.5.21 Exercise 4.19 — Peano axioms for sequences

There are many unproved assertions in Example 4.4! In particular discuss "Peano's Axioms" for $\{0,1\}^*$. Show, moreover, that $\mathrm{one} : C \to T$ can be defined from the rest of the structure by a fixed-point equation.

Exercise_4_19.lean

```
module
```

```
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_8
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Example_4_4
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Theorem_4_2

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.Exercise419
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap ExampleB
    Example44

section
variable {b : Type*}

notation "?" => Example23.trueElt

notation "?" => Example23.falseElt

/-- The codomain value chosen by inspecting the head of a sequence: `[]  |->  z`, `0
    sigma  |->  a_0`, `1sigma  |->  a_1`.
With `z = False` this is the value at the *partial* element `sigmaFalse`; with `z = v?`
     the value at the
*total* element `sigma`. -/
def headValC (V : NeighborhoodSystem b) (z a0 a1 : V.Element) : Str  ->  V.Element
  | [] => z
  | false :: _ => a0
  | true :: _ => a1

@[simp] theorem headValC_nil (V : NeighborhoodSystem b) (z a0 a1 : V.Element) :
    headValC V z a0 a1 [] = z := rfl

@[simp] theorem headValC_false (V : NeighborhoodSystem b) (z a0 a1 : V.Element) (sigma
     : Str) :
    headValC V z a0 a1 (false :: sigma) = a0 := rfl

@[simp] theorem headValC_true (V : NeighborhoodSystem b) (z a0 a1 : V.Element) (sigma
    : Str) :
    headValC V z a0 a1 (true :: sigma) = a1 := rfl

/-- The truth domain `T` of Example 1.2. -/
abbrev T : NeighborhoodSystem Example12.Token := Example23.T

/-- Monotonicity of the head value: along a prefix `sigma <+: tau`, the *partial* value
     (`z = False`) at `sigma`
is below *any* head value at `tau` with the same head constants. Covers both required
    conditions
(cone -> cone with `z' = False`, cone -> singleton with `z' = v?`). -/
theorem headValC_bot_le (V : NeighborhoodSystem b) (z' a0 a1 : V.Element) {sigma tau :
     Str}
    (h : sigma <+: tau) : headValC V V.bot a0 a1 sigma  <=  headValC V z' a0 a1 tau :=
     by
  cases sigma with
  | nil => exact bot_le
  | cons b sigma0 =>
```

```
    obtain <s, rfl> := h
    cases b
    * rw [List.cons_append]; exact le_of_eq rfl
    * rw [List.cons_append]; exact le_of_eq rfl
end

section
/-- A cone is never contained in a singleton (it has at least two elements). -/
theorem not_cone_subset_singleton (tau sigma : Str) : not  cone tau  subseteq  ({sigma
    } : Set Str) := by
  intro h
  have h1 : tau  in  ({sigma} : Set Str) := h (by simp [mem_cone])
  have h2 : (tau ++ [true])  in  ({sigma} : Set Str) := h (by simp [mem_cone])
  rw [Set.mem_singleton_iff] at h1 h2
  have : tau = tau ++ [true] := h1.trans h2.symm
  simp at this
end

section
/-- A cone is never equal to a singleton. -/
theorem cone_ne_singleton (tau sigma : Str) : cone tau  !=  ({sigma} : Set Str) := by
  intro h
  exact not_cone_subset_singleton tau sigma (h  |>  subset_rfl)
end

section
variable {b : Type*}

/-- A map `C  ->  V` determined by its value `coneVal sigma` on each partial element `
    sigmaFalse` and `singVal sigma`
on each total element `sigma`, provided (a) the partial values are monotone along
    prefixes and (b) a
partial value sits below the total value of any extending prefix. The relation says: a
     cone `sigmaSigma*`
relates to the neighbourhoods of `coneVal sigma`, and a singleton `{sigma}` to those of
     `singVal sigma`. -/
def liftC (V : NeighborhoodSystem b) (coneVal singVal : Str  ->  V.Element)
    (hcone : forall  {sigma tau : Str}, sigma <+: tau  ->  coneVal sigma  <=  coneVal
    tau)
    (hsing : forall  {sigma tau : Str}, sigma <+: tau  ->  coneVal sigma  <=  singVal
    tau) :
    ApproximableMap C V where
  rel X Y := (exists  sigma, X = cone sigma  /\  (coneVal sigma).mem Y)  \/  (exists
    sigma, X = {sigma}  /\  (singVal sigma).mem Y)
  rel_dom := by
    rintro X Y (<sigma, rfl, _> | <sigma, rfl, _>)
    * exact memC_cone sigma
    * exact memC_singleton sigma
  rel_cod := by
    rintro X Y (<sigma, _, hY> | <sigma, _, hY>)
    * exact (coneVal sigma).sub hY
    * exact (singVal sigma).sub hY
  master_rel := by
    refine Or.inl <[], ?_, (coneVal []).master_mem>
    rw [C_master]; exact cone_nil.symm
  inter_right := by
```

```
    rintro X Y Y' (<sigma, rfl, hY> | <sigma, rfl, hY>) (<sigma', hX', hY'> | <sigma',
     hX', hY'>)
    * have : sigma = sigma' := cone_injective hX'
      subst this
      exact Or.inl <sigma, rfl, (coneVal sigma).inter_mem hY hY'>
    * exact absurd hX' (cone_ne_singleton sigma sigma')
    * exact absurd hX'.symm (cone_ne_singleton sigma' sigma)
    * have : sigma = sigma' := by rw [Set.singleton_eq_singleton_iff] at hX'; exact hX
    '
      subst this
      exact Or.inr <sigma, rfl, (singVal sigma).inter_mem hY hY'>
  mono := by
    rintro X X' Y Y' (<sigma, rfl, hY> | <sigma, rfl, hY>) hX'X hYY' hX' hY'
    * -- input is the cone `sigmaSigma*`
      rcases hX' with <tau, rfl> | <tau, rfl>
      * -- `X' = cone tau  subseteq  cone sigma`, so `sigma <+: tau`
        have hpre : sigma <+: tau := cone_subset_cone.mp hX'X
        exact Or.inl <tau, rfl, (coneVal tau).up_mem (hcone hpre Y hY) hY' hYY'>
      * -- `X' = {tau}  subseteq  cone sigma`, so `sigma <+: tau`
        have hpre : sigma <+: tau := singleton_subset_cone.mp hX'X
        exact Or.inr <tau, rfl, (singVal tau).up_mem (hsing hpre Y hY) hY' hYY'>
    * -- input is the singleton `{sigma}`
      rcases hX' with <tau, rfl> | <tau, rfl>
      * exact absurd hX'X (not_cone_subset_singleton tau sigma)
      * have htausigma : tau = sigma := by
          have := Set.singleton_subset_iff.mp hX'X
          rwa [Set.mem_singleton_iff] at this
        subst htausigma
        exact Or.inr <tau, rfl, (singVal tau).up_mem hY hY' hYY'>

/-- **Example 4.4 -- `empty`.** `empty(?) = true`, `empty(0x) = empty(1x) = false`,
    strict. -/
def emptyMap : ApproximableMap C T :=
  liftC T (headValC T T.bot ? ?) (headValC T ? ? ?)
    (headValC_bot_le T T.bot ? ?) (headValC_bot_le T ? ? ?)

/-- `liftC` on a partial element: `f(sigmaFalse) = coneVal sigma`. -/
theorem liftC_strBot (V : NeighborhoodSystem b) (coneVal singVal : Str  ->  V.Element)
    (hcone : forall  {sigma tau : Str}, sigma <+: tau  ->  coneVal sigma  <=  coneVal
    tau)
    (hsing : forall  {sigma tau : Str}, sigma <+: tau  ->  coneVal sigma  <=  singVal
    tau) (sigma : Str) :
    (liftC V coneVal singVal hcone hsing).toElementMap (strBot sigma) = coneVal sigma
    := by
  apply Element.ext
  intro Y
  constructor
  * rintro <X, <_, hsub>, hrel>
    rcases hrel with <sigma', hXcone, hY> | <sigma', hXsing, hY>
    * have hpre : sigma' <+: sigma := cone_subset_cone.mp (hXcone  |>  hsub)
      exact hcone hpre Y hY
    * exact absurd (hXsing  |>  hsub) (not_cone_subset_singleton sigma sigma')
  * intro hY
    exact <cone sigma, <memC_cone sigma, subset_rfl>, Or.inl <sigma, rfl, hY>>

/-- `liftC` on a total element: `f(sigma) = singVal sigma`. -/
```

```
theorem liftC_strElem (V : NeighborhoodSystem b) (coneVal singVal : Str  ->  V.Element
    )
    (hcone : forall  {sigma tau : Str}, sigma <+: tau  ->  coneVal sigma  <=  coneVal
    tau)
    (hsing : forall  {sigma tau : Str}, sigma <+: tau  ->  coneVal sigma  <=  singVal
    tau) (sigma : Str) :
    (liftC V coneVal singVal hcone hsing).toElementMap (strElem sigma) = singVal sigma
     := by
  apply Element.ext
  intro Y
  constructor
  * rintro <X, <_, hsub>, hrel>
    rcases hrel with <sigma', hXcone, hY> | <sigma', hXsing, hY>
    * have hpre : sigma' <+: sigma := by
        apply singleton_subset_cone.mp
        rw [ <-  hXcone]; exact hsub
      exact hsing hpre Y hY
    * have hsigmasigma' : sigma = sigma' := by
        have := Set.singleton_subset_iff.mp (hXsing  |>  hsub)
        rwa [Set.mem_singleton_iff] at this
      subst hsigmasigma'; exact hY
  * intro hY
    exact <{sigma}, <memC_singleton sigma, subset_rfl>, Or.inr <sigma, rfl, hY>>

@[simp] theorem emptyMap_strElem (sigma : Str) :
    emptyMap.toElementMap (strElem sigma) = headValC T ? ? ? sigma :=
  liftC_strElem T (headValC T T.bot ? ?) (headValC T ? ? ?)
    (headValC_bot_le T T.bot ? ?) (headValC_bot_le T ? ? ?) sigma

@[simp] theorem emptyMap_strBot (sigma : Str) :
    emptyMap.toElementMap (strBot sigma) = headValC T T.bot ? ? sigma :=
  liftC_strBot T (headValC T T.bot ? ?) (headValC T ? ? ?)
    (headValC_bot_le T T.bot ? ?) (headValC_bot_le T ? ? ?) sigma

/-- **Example 4.4 -- `one`.** `one(?) = false`, `one(0x) = false`, `one(1x) = true`,
    strict. -/
def oneMap : ApproximableMap C T :=
  liftC T (headValC T T.bot ? ?) (headValC T ? ? ?)
    (headValC_bot_le T T.bot ? ?) (headValC_bot_le T ? ? ?)

/-- **Example 4.4 -- `zero`.** `zero(?) = false`, `zero(0x) = true`, `zero(1x) = false
    `, strict. -/
def zeroMap : ApproximableMap C T :=
  liftC T (headValC T T.bot ? ?) (headValC T ? ? ?)
    (headValC_bot_le T T.bot ? ?) (headValC_bot_le T ? ? ?)

@[simp] theorem zeroMap_strElem (sigma : Str) :
    zeroMap.toElementMap (strElem sigma) = headValC T ? ? ? sigma :=
  liftC_strElem T (headValC T T.bot ? ?) (headValC T ? ? ?)
    (headValC_bot_le T T.bot ? ?) (headValC_bot_le T ? ? ?) sigma

@[simp] theorem zeroMap_strBot (sigma : Str) :
    zeroMap.toElementMap (strBot sigma) = headValC T T.bot ? ? sigma :=
  liftC_strBot T (headValC T T.bot ? ?) (headValC T ? ? ?)
    (headValC_bot_le T T.bot ? ?) (headValC_bot_le T ? ? ?) sigma
```

```
@[simp] theorem oneMap_strElem (sigma : Str) :
    oneMap.toElementMap (strElem sigma) = headValC T ? ? ? sigma :=
  liftC_strElem T (headValC T T.bot ? ?) (headValC T ? ? ?)
    (headValC_bot_le T T.bot ? ?) (headValC_bot_le T ? ? ?) sigma

@[simp] theorem oneMap_strBot (sigma : Str) :
    oneMap.toElementMap (strBot sigma) = headValC T T.bot ? ? sigma :=
  liftC_strBot T (headValC T T.bot ? ?) (headValC T ? ? ?)
    (headValC_bot_le T T.bot ? ?) (headValC_bot_le T ? ? ?) sigma

/-- The defining right-hand side: `cond(empty(x), false, cond(zero(x), false, true))`.
    It uses only
`empty`, `zero` and the conditional `cond` (Exercise 3.26) -- not `one`. -/
def oneDef (x : C.Element) : T.Element :=
  (Exercise326.cond T).toElementMap
    (pair (emptyMap.toElementMap x)
      (pair ?
        ((Exercise326.cond T).toElementMap (pair (zeroMap.toElementMap x) (pair ? ?)))
    ))

/-- `cond(False, x, y) = False` in `T`, phrased with `T.bot` (which the head-test `
    empty(False)` produces)
rather than the syntactically distinct `Example23.botElt` of `Exercise326.cond_bot`. -/
theorem condT_bot (x y : T.Element) :
    (Exercise326.cond T).toElementMap (pair T.bot (pair x y)) = T.bot := by
  apply Element.ext
  intro Z
  rw [Exercise326.cond_toElementMap_mem]
  constructor
  * rintro (<h0, _> | <h1, _> | rfl)
    * rw [NeighborhoodSystem.mem_bot] at h0; exact absurd h0 Exercise326.
    zero_ne_master
    * rw [NeighborhoodSystem.mem_bot] at h1; exact absurd h1 Exercise326.one_ne_master
    * exact T.bot.master_mem
  * intro h
    rw [NeighborhoodSystem.mem_bot] at h
    exact Or.inr (Or.inr h)
end
end DomainTheory.NeighbourhoodSystems.Exercise419

section
namespace DomainTheory.NeighbourhoodSystems.Exercise419
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap ExampleB
    Example44
section
variable {b : Type*}

/- concordance: source L2323 EXERCISE 4.19 Peano axioms for sequences -/
/-- **Exercise 4.19 (Scott 1981, PRG-19).** The same defining equation holds on every
    partial
element `sigmaFalse` -- including `False = []False` where `empty(False) = False` forces
     `cond(False, ...) = False`. -/
theorem one_def_strBot (sigma : Str) : oneMap.toElementMap (strBot sigma) = oneDef (
    strBot sigma) := by
  cases sigma with
  | nil =>
```

```
    rw [oneDef, emptyMap_strBot, headValC_nil, condT_bot, oneMap_strBot, headValC_nil]
  | cons b sigma0 =>
    cases b with
    | false =>
      rw [oneDef, emptyMap_strBot, headValC_false, Exercise326.cond_false,
        zeroMap_strBot, headValC_false, Exercise326.cond_true, oneMap_strBot,
    headValC_false]
    | true =>
      rw [oneDef, emptyMap_strBot, headValC_true, Exercise326.cond_false,
        zeroMap_strBot, headValC_true, Exercise326.cond_false, oneMap_strBot,
    headValC_true]

/-- **Exercise 4.19 (Scott 1981, PRG-19).** `one` is definable from `empty`, `zero`, `
    cond`: the
equation `one(x) = cond(empty(x), false, cond(zero(x), false, true))` holds on every
    total element
`sigma`. -/
theorem one_def_strElem (sigma : Str) : oneMap.toElementMap (strElem sigma) = oneDef (
    strElem sigma) := by
  cases sigma with
  | nil =>
    -- empty(?)=true ? cond picks `false`
    rw [oneDef, emptyMap_strElem, headValC_nil, Exercise326.cond_true,
      oneMap_strElem, headValC_nil]
  | cons b sigma0 =>
    cases b with
    | false =>
      -- empty=false ? inner cond; zero(0sigma)=true ? inner cond picks `false`
      rw [oneDef, emptyMap_strElem, headValC_false, Exercise326.cond_false,
        zeroMap_strElem, headValC_false, Exercise326.cond_true, oneMap_strElem,
    headValC_false]
    | true =>
      rw [oneDef, emptyMap_strElem, headValC_true, Exercise326.cond_false,
        zeroMap_strElem, headValC_true, Exercise326.cond_false, oneMap_strElem,
    headValC_true]

end

section
/-- **Exercise 4.19 -- Peano (ii') for `{0,1}*`.** The two successor ranges are
    disjoint:
`0sigma  !=  1tau`. -/
theorem peano_cons_disjoint (sigma tau : Str) : (false :: sigma)  !=  (true :: tau) :=
     by
  simp
end

section
/-- **Exercise 4.19 -- Peano (ii) for `{0,1}*`.** Each successor `b ::*` is injective.
    -/
theorem peano_cons_injective {b : Bool} {sigma tau : Str} (h : b :: sigma = b :: tau)
    : sigma = tau :=
  (List.cons.injEq b sigma b tau).mp h |>.2
end

section
```

```
/-- **Exercise 4.19 -- Peano (iii) for `{0,1}*`.** Induction: a predicate holding at
    `?` and closed
under both successors holds everywhere. (This is `List.rec`; it is the two-successor
    analogue of
Definition 4.5(iii) and the recursion engine behind `tail`/`empty`/`zero`/`one`.) -/
theorem peano_induction (P : Str  ->  Prop) (hnil : P []) (hcons : forall  b sigma, P
    sigma  ->  P (b :: sigma)) :
    forall  sigma, P sigma := by
  intro sigma
  induction sigma with
  | nil => exact hnil
  | cons b sigma ih => exact hcons b sigma ih
end

section
/-- **Exercise 4.19 -- Peano (i) for `{0,1}*`.** `?` is not a successor: `[]  !=  b ::
    sigma`. -/
theorem peano_nil_ne_cons (b : Bool) (sigma : Str) : ([] : Str)  !=  b :: sigma := by
  simp
end

section
set_option linter.unusedVariables false

theorem example_419 : And
  (And
    (And
      (And
        (And
          (forall  (b : Bool) (sigma : DomainTheory.NeighbourhoodSystems.ExampleB.Str)
    ,
            @Ne (List Bool) (@List.nil Bool) (@List.cons Bool b sigma))
          (forall  (sigma : DomainTheory.NeighbourhoodSystems.ExampleB.Str),
            @Eq
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
                DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
    NeighbourhoodSystems.Exercise419.T)
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
                DomainTheory.NeighbourhoodSystems.ExampleB.Str DomainTheory.
    NeighbourhoodSystems.Example12.Token
                DomainTheory.NeighbourhoodSystems.Example44.C DomainTheory.
    NeighbourhoodSystems.Exercise419.T
                DomainTheory.NeighbourhoodSystems.Exercise419.oneMap
                (DomainTheory.NeighbourhoodSystems.Example44.strBot sigma))
              (DomainTheory.NeighbourhoodSystems.Exercise419.oneDef
                (DomainTheory.NeighbourhoodSystems.Example44.strBot sigma))))
        (forall  (sigma : DomainTheory.NeighbourhoodSystems.ExampleB.Str),
          @Eq
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
              DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
    NeighbourhoodSystems.Exercise419.T)
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
              DomainTheory.NeighbourhoodSystems.ExampleB.Str DomainTheory.
    NeighbourhoodSystems.Example12.Token
              DomainTheory.NeighbourhoodSystems.Example44.C DomainTheory.
    NeighbourhoodSystems.Exercise419.T
```

```
           DomainTheory.NeighbourhoodSystems.Exercise419.oneMap
           (DomainTheory.NeighbourhoodSystems.Example44.strElem sigma))
         (DomainTheory.NeighbourhoodSystems.Exercise419.oneDef
           (DomainTheory.NeighbourhoodSystems.Example44.strElem sigma))))
    (forall  (sigma tau : DomainTheory.NeighbourhoodSystems.ExampleB.Str),
      @Ne (List Bool) (@List.cons Bool Bool.false sigma) (@List.cons Bool Bool.true
  tau)))
   (forall  {b : Bool} {sigma tau : DomainTheory.NeighbourhoodSystems.ExampleB.Str},
     @Eq (List Bool) (@List.cons Bool b sigma) (@List.cons Bool b tau)  ->
       @Eq DomainTheory.NeighbourhoodSystems.ExampleB.Str sigma tau))
(forall  (P : DomainTheory.NeighbourhoodSystems.ExampleB.Str  ->  Prop),
  P (@List.nil Bool)  ->
    (forall  (b : Bool) (sigma : DomainTheory.NeighbourhoodSystems.ExampleB.Str), P
  sigma  ->  P (@List.cons Bool b sigma))  ->
      forall  (sigma : DomainTheory.NeighbourhoodSystems.ExampleB.Str), P sigma) :=
@And.intro
(And
  (And
    (And
      (And
        (forall  (b : Bool) (sigma : DomainTheory.NeighbourhoodSystems.ExampleB.Str)
  ,
          @Ne (List Bool) (@List.nil Bool) (@List.cons Bool b sigma))
        (forall  (sigma : DomainTheory.NeighbourhoodSystems.ExampleB.Str),
          @Eq
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
              DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
  NeighbourhoodSystems.Exercise419.T)
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
              DomainTheory.NeighbourhoodSystems.ExampleB.Str DomainTheory.
  NeighbourhoodSystems.Example12.Token
              DomainTheory.NeighbourhoodSystems.Example44.C DomainTheory.
  NeighbourhoodSystems.Exercise419.T
              DomainTheory.NeighbourhoodSystems.Exercise419.oneMap
              (DomainTheory.NeighbourhoodSystems.Example44.strBot sigma))
            (DomainTheory.NeighbourhoodSystems.Exercise419.oneDef
              (DomainTheory.NeighbourhoodSystems.Example44.strBot sigma))))
      (forall  (sigma : DomainTheory.NeighbourhoodSystems.ExampleB.Str),
        @Eq
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
            DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
  NeighbourhoodSystems.Exercise419.T)
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
            DomainTheory.NeighbourhoodSystems.ExampleB.Str DomainTheory.
  NeighbourhoodSystems.Example12.Token
            DomainTheory.NeighbourhoodSystems.Example44.C DomainTheory.
  NeighbourhoodSystems.Exercise419.T
            DomainTheory.NeighbourhoodSystems.Exercise419.oneMap
            (DomainTheory.NeighbourhoodSystems.Example44.strElem sigma))
          (DomainTheory.NeighbourhoodSystems.Exercise419.oneDef
            (DomainTheory.NeighbourhoodSystems.Example44.strElem sigma))))
    (forall  (sigma tau : DomainTheory.NeighbourhoodSystems.ExampleB.Str),
      @Ne (List Bool) (@List.cons Bool Bool.false sigma) (@List.cons Bool Bool.true
  tau)))
   (forall  {b : Bool} {sigma tau : DomainTheory.NeighbourhoodSystems.ExampleB.Str},
     @Eq (List Bool) (@List.cons Bool b sigma) (@List.cons Bool b tau)  ->
```

```
     @Eq DomainTheory.NeighbourhoodSystems.ExampleB.Str sigma tau))
(forall  (P : DomainTheory.NeighbourhoodSystems.ExampleB.Str  ->  Prop),
  P (@List.nil Bool)  ->
    (forall  (b : Bool) (sigma : DomainTheory.NeighbourhoodSystems.ExampleB.Str), P
  sigma  ->  P (@List.cons Bool b sigma))  ->
     forall  (sigma : DomainTheory.NeighbourhoodSystems.ExampleB.Str), P sigma)
(@And.intro
  (And
    (And
      (And
        (forall  (b : Bool) (sigma : DomainTheory.NeighbourhoodSystems.ExampleB.Str)
 ,
          @Ne (List Bool) (@List.nil Bool) (@List.cons Bool b sigma))
        (forall  (sigma : DomainTheory.NeighbourhoodSystems.ExampleB.Str),
          @Eq
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
              DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
 NeighbourhoodSystems.Exercise419.T)
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
              DomainTheory.NeighbourhoodSystems.ExampleB.Str DomainTheory.
 NeighbourhoodSystems.Example12.Token
              DomainTheory.NeighbourhoodSystems.Example44.C DomainTheory.
 NeighbourhoodSystems.Exercise419.T
              DomainTheory.NeighbourhoodSystems.Exercise419.oneMap
              (DomainTheory.NeighbourhoodSystems.Example44.strBot sigma))

            (DomainTheory.NeighbourhoodSystems.Exercise419.oneDef
              (DomainTheory.NeighbourhoodSystems.Example44.strBot sigma))))
      (forall  (sigma : DomainTheory.NeighbourhoodSystems.ExampleB.Str),
        @Eq
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
            DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
 NeighbourhoodSystems.Exercise419.T)
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
            DomainTheory.NeighbourhoodSystems.ExampleB.Str DomainTheory.
 NeighbourhoodSystems.Example12.Token
            DomainTheory.NeighbourhoodSystems.Example44.C DomainTheory.
 NeighbourhoodSystems.Exercise419.T
            DomainTheory.NeighbourhoodSystems.Exercise419.oneMap
            (DomainTheory.NeighbourhoodSystems.Example44.strElem sigma))
          (DomainTheory.NeighbourhoodSystems.Exercise419.oneDef
            (DomainTheory.NeighbourhoodSystems.Example44.strElem sigma))))
    (forall  (sigma tau : DomainTheory.NeighbourhoodSystems.ExampleB.Str),
      @Ne (List Bool) (@List.cons Bool Bool.false sigma) (@List.cons Bool Bool.true
 tau)))
  (forall  {b : Bool} {sigma tau : DomainTheory.NeighbourhoodSystems.ExampleB.Str},
    @Eq (List Bool) (@List.cons Bool b sigma) (@List.cons Bool b tau)  ->
      @Eq DomainTheory.NeighbourhoodSystems.ExampleB.Str sigma tau)
  (@And.intro
    (And
      (And
        (forall  (b : Bool) (sigma : DomainTheory.NeighbourhoodSystems.ExampleB.Str)
 ,
          @Ne (List Bool) (@List.nil Bool) (@List.cons Bool b sigma))
        (forall  (sigma : DomainTheory.NeighbourhoodSystems.ExampleB.Str),
          @Eq
```

```
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
            DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
NeighbourhoodSystems.Exercise419.T)
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
            DomainTheory.NeighbourhoodSystems.ExampleB.Str DomainTheory.
NeighbourhoodSystems.Example12.Token
            DomainTheory.NeighbourhoodSystems.Example44.C DomainTheory.
NeighbourhoodSystems.Exercise419.T
            DomainTheory.NeighbourhoodSystems.Exercise419.oneMap
             (DomainTheory.NeighbourhoodSystems.Example44.strBot sigma))
          (DomainTheory.NeighbourhoodSystems.Exercise419.oneDef
            (DomainTheory.NeighbourhoodSystems.Example44.strBot sigma))))
    (forall  (sigma : DomainTheory.NeighbourhoodSystems.ExampleB.Str),
      @Eq
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
          DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
NeighbourhoodSystems.Exercise419.T)
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
          DomainTheory.NeighbourhoodSystems.ExampleB.Str DomainTheory.
NeighbourhoodSystems.Example12.Token
          DomainTheory.NeighbourhoodSystems.Example44.C DomainTheory.
NeighbourhoodSystems.Exercise419.T
          DomainTheory.NeighbourhoodSystems.Exercise419.oneMap
          (DomainTheory.NeighbourhoodSystems.Example44.strElem sigma))
        (DomainTheory.NeighbourhoodSystems.Exercise419.oneDef
          (DomainTheory.NeighbourhoodSystems.Example44.strElem sigma))))
  (forall  (sigma tau : DomainTheory.NeighbourhoodSystems.ExampleB.Str),
    @Ne (List Bool) (@List.cons Bool Bool.false sigma) (@List.cons Bool Bool.true
tau))
  (@And.intro
    (And
      (forall  (b : Bool) (sigma : DomainTheory.NeighbourhoodSystems.ExampleB.Str)
,
        @Ne (List Bool) (@List.nil Bool) (@List.cons Bool b sigma))
      (forall  (sigma : DomainTheory.NeighbourhoodSystems.ExampleB.Str),
        @Eq
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
            DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
NeighbourhoodSystems.Exercise419.T)
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
            DomainTheory.NeighbourhoodSystems.ExampleB.Str DomainTheory.
NeighbourhoodSystems.Example12.Token
            DomainTheory.NeighbourhoodSystems.Example44.C DomainTheory.
NeighbourhoodSystems.Exercise419.T
            DomainTheory.NeighbourhoodSystems.Exercise419.oneMap
             (DomainTheory.NeighbourhoodSystems.Example44.strBot sigma))
          (DomainTheory.NeighbourhoodSystems.Exercise419.oneDef
            (DomainTheory.NeighbourhoodSystems.Example44.strBot sigma))))
    (forall  (sigma : DomainTheory.NeighbourhoodSystems.ExampleB.Str),
      @Eq
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
          DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
NeighbourhoodSystems.Exercise419.T)
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
          DomainTheory.NeighbourhoodSystems.ExampleB.Str DomainTheory.
NeighbourhoodSystems.Example12.Token
```

```
            DomainTheory.NeighbourhoodSystems.Example44.C DomainTheory.
    NeighbourhoodSystems.Exercise419.T
            DomainTheory.NeighbourhoodSystems.Exercise419.oneMap
            (DomainTheory.NeighbourhoodSystems.Example44.strElem sigma))
          (DomainTheory.NeighbourhoodSystems.Exercise419.oneDef
            (DomainTheory.NeighbourhoodSystems.Example44.strElem sigma)))
        (@And.intro
          (forall  (b : Bool) (sigma : DomainTheory.NeighbourhoodSystems.ExampleB.Str)
    ,
           @Ne (List Bool) (@List.nil Bool) (@List.cons Bool b sigma))
          (forall  (sigma : DomainTheory.NeighbourhoodSystems.ExampleB.Str),
           @Eq
             (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
               DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
    NeighbourhoodSystems.Exercise419.T)
             (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
               DomainTheory.NeighbourhoodSystems.ExampleB.Str DomainTheory.
    NeighbourhoodSystems.Example12.Token
               DomainTheory.NeighbourhoodSystems.Example44.C DomainTheory.
    NeighbourhoodSystems.Exercise419.T
               DomainTheory.NeighbourhoodSystems.Exercise419.oneMap
               (DomainTheory.NeighbourhoodSystems.Example44.strBot sigma))
             (DomainTheory.NeighbourhoodSystems.Exercise419.oneDef
               (DomainTheory.NeighbourhoodSystems.Example44.strBot sigma)))
          DomainTheory.NeighbourhoodSystems.Exercise419.peano_nil_ne_cons
          DomainTheory.NeighbourhoodSystems.Exercise419.one_def_strBot)
        DomainTheory.NeighbourhoodSystems.Exercise419.one_def_strElem)
      DomainTheory.NeighbourhoodSystems.Exercise419.peano_cons_disjoint)
    @DomainTheory.NeighbourhoodSystems.Exercise419.peano_cons_injective)
  DomainTheory.NeighbourhoodSystems.Exercise419.peano_induction
end

end DomainTheory.NeighbourhoodSystems.Exercise419
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

section
variable {a : Type*} {V : NeighborhoodSystem a}

namespace ApproximableMap

/-- `f(fix(g  comp  f))` is a fixed point of `f  comp  g`:
`(f  comp  g)(f(fix(g comp f))) = f((g comp f)(fix(g comp f))) = f(fix(g comp f))`. -/
theorem comp_fixElement_isFixed (f g : ApproximableMap V V) :
    (f.comp g).toElementMap (f.toElementMap (g.comp f).fixElement)
      = f.toElementMap (g.comp f).fixElement := by
  rw [toElementMap_comp]
  -- `g(f(fix(g comp f))) = (g comp f)(fix(g comp f)) = fix(g comp f)`
  have h : g.toElementMap (f.toElementMap (g.comp f).fixElement) = (g.comp f).
    fixElement := by
    rw [ <-  toElementMap_comp]
    exact toElementMap_fixElement (g.comp f)
  rw [h]

end ApproximableMap
end
```

```
end DomainTheory.NeighbourhoodSystems
end
```

3.5.22 Exercise 4.20 — fix of a composite

For approximable $f, g : D \to D$ prove that

$$\mathrm{fix}(f \circ g) = f(\mathrm{fix}(g \circ f)).$$

Exercise_4_20.lean

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Exercise_4_19
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Theorem_4_2
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Theorem_4_1

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap
section
variable {a : Type*} {V : NeighborhoodSystem a}
namespace ApproximableMap
/- concordance: source L2325 EXERCISE 4.20 fix of a composite -/
/-- **Exercise 4.20 (Scott 1981, PRG-19).** `fix(f  comp  g) = f(fix(g  comp  f))`. -/
theorem fixElement_comp_comm (f g : ApproximableMap V V) :
    (f.comp g).fixElement = f.toElementMap (g.comp f).fixElement := by
  apply le_antisymm
  * -- `fix(f comp g)  sqsub  f(fix(g comp f))` since the latter is a (pre-)fixed point
    of `f comp g`.
    exact fixElement_le_of_toElementMap_le (f.comp g)
      (le_of_eq (comp_fixElement_isFixed f g))
  * -- `f(fix(g comp f))  sqsub  fix(f comp g)`. First, `fix(g comp f)  sqsub  g(fix(f
    comp g))`.
    have hga : (g.comp f).fixElement  <=  g.toElementMap (f.comp g).fixElement := by
      apply fixElement_le_of_toElementMap_le (g.comp f)
      have : (g.comp f).toElementMap (g.toElementMap (f.comp g).fixElement)
          = g.toElementMap (f.comp g).fixElement := comp_fixElement_isFixed g f
      exact le_of_eq this
    -- apply `f` and use `f(g(fix(f comp g))) = (f comp g)(fix(f comp g)) = fix(f comp
    g)`.
    calc f.toElementMap (g.comp f).fixElement
        <=  f.toElementMap (g.toElementMap (f.comp g).fixElement) := f.
    toElementMap_mono hga
      _ = (f.comp g).toElementMap (f.comp g).fixElement := by rw [toElementMap_comp]
      _ = (f.comp g).fixElement := toElementMap_fixElement (f.comp g)

end ApproximableMap
end
```

```
end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

end DomainTheory.NeighbourhoodSystems
end
```

3.5.23 Exercise 4.21 — order as a fixed point

Show that the less-than-or-equal-to relation $\ell \subseteq \mathbf{N} \times \mathbf{N}$ is uniquely determined by the fixed point equation

$$\ell = \{(n, n) \mid n \in \mathbf{N}\} \cup \{(n, m^+) \mid (n, m) \in \ell\}.$$

Consider the structured set $\langle P\mathbf{N}, \mathbf{N}, {}^+\rangle$ where, as before, $x^+ = \{n^+ \mid n \in x\}$.

What is the unique function $[\cdot] : \mathbf{N} \to P\mathbf{N}$ given by 4.13(3)? Prove that the structures $\langle \mathbf{N}, 0, {}^+\rangle$ and $\langle [m], m, {}^+\rangle$ are uniquely isomorphic for each $m \in \mathbf{N}$, and connect the isomorphism with ordinary addition of integers. Can the same be done for multiplication? (Hint: Consider the fixed-point equation

$$n \cdot \mathbf{N} = \{0\} \cup \{n + m \mid m \in n \cdot \mathbf{N}\},$$

where $n \in \mathbf{N}$ is fixed.)

Exercise_4_21.lean

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Exercise_4_14
public import Mathlib.Algebra.Group.End
public import Mathlib.Tactic.Ring

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.Exercise421
open DomainTheory.NeighbourhoodSystems.Exercise414

section
/-- Scott's operator `? |-> {(n, n)} U {(n, m^+) ? (n, m) in ?}` on `P(Nat x
    Nat)`. -/
def leOp (u : Set (Nat x Nat)) : Set (Nat x Nat) :=
  {p | p.1 = p.2} U {p | exists m, (p.1, m) in u /\ p.2 = m + 1}
end

section
/-- The relation ` <= ` itself, as a subset of `Nat x Nat`. -/
def leRel : Set (Nat x Nat) := {p | p.1 <= p.2}
end

section
/-- Scott's hint operator `n*Nat = {0} U {n + m ? m in n*Nat}` on `P Nat` (for
    fixed `n`). -/
def mulOp (n : Nat) (u : Set Nat) : Set Nat := {0} U {k | exists m in u, k = n +
    m}
end
```

```
section
/-- Scott's `x^+ = {n^+ ? n  in  x}` on `P Nat`. -/
def succImage (x : Set Nat) : Set Nat := {k | exists  j  in  x, k = j + 1}
end

section
/-- The up-set `[m] = {k ? m  <=  k}` -- the value of Scott's unique function `[*]` of
    4.13(3). -/
def upSet (m : Nat) : Set Nat := {k | m  <=  k}
end

end DomainTheory.NeighbourhoodSystems.Exercise421

section
namespace DomainTheory.NeighbourhoodSystems.Exercise421
open DomainTheory.NeighbourhoodSystems.Exercise414
section
/- concordance: source L2331 EXERCISE 4.21 order as a fixed point -/
/-- **Exercise 4.21 (Scott 1981, PRG-19).** The structure-preserving bijection between
`<Nat, 0, ^+>` and `<[m], m, ^+>` (where `[m] = {k ? m  <=  k}`, with distinguished
    element `m` and the
successor `^+`). It is **ordinary addition** by `m`: `n  |->  m + n`. -/
def addIso (m : Nat) : Nat ? {k : Nat // k  in  upSet m} where
  toFun n := <m + n, Nat.le_add_right m n>
  invFun k := k.1 - m
  left_inv n := by show m + n - m = n; omega
  right_inv := by
    rintro <k, hk>
    exact Subtype.ext (Nat.add_sub_of_le hk)
end

section
/-- The isomorphism is given by addition: `addIso m n = m + n`. -/
theorem addIso_apply (m n : Nat) : (addIso m n : Nat) = m + n := rfl
end

section
/-- The isomorphism preserves the successor: `addIso m (n^+) = (addIso m n)^+`. -/
theorem addIso_succ (m n : Nat) : (addIso m (n + 1) : Nat) = (addIso m n : Nat) + 1 :=
     by
  simp only [addIso_apply]; omega
end

section
/-- The isomorphism sends `0` to the distinguished element `m` of `[m]`. -/
theorem addIso_zero (m : Nat) : (addIso m 0 : Nat) = m := by simp [addIso]
end

section
theorem leOp_monotone : Monotone leOp := by
  intro u v huv p hp
  rcases hp with hp | <m, hm, hpm>
  * exact Or.inl hp
  * exact Or.inr <m, huv hm, hpm>
end
```

```
section
/-- **Exercise 4.21 (Scott 1981, PRG-19).** `≤` solves Scott's fixed-point equation.
    -/
theorem leRel_isFixed : leOp leRel = leRel := by
  apply Set.Subset.antisymm
  · rintro ⟨n, m⟩ (h | ⟨k, hk, rfl⟩)
    · exact le_of_eq (h : n = m)
    · exact Nat.le_succ_of_le (hk : n ≤ k)
  · rintro ⟨n, m⟩ (h : n ≤ m)
    rcases Nat.eq_or_lt_of_le h with h' | h'
    · exact Or.inl h'
    · refine Or.inr ⟨m - 1, ?_, ?_⟩
      · change n ≤ m - 1; omega
      · omega
end

section
/-- The key step toward uniqueness: any fixed point `u` of `leOp` agrees with `≤`
    pointwise.
By induction on the second coordinate `m`; the base case `m = 0` uses that the
    successor clause
cannot produce a `0` on the right. -/
theorem mem_fixedPoint_iff {u : Set (Nat × Nat)} (hu : leOp u = u) (n m : Nat) :
    (n, m) ∈ u ↔ n ≤ m := by
  induction m with
  | zero =>
    rw [← hu]
    constructor
    · rintro (h | ⟨k, _, hk⟩)
      · exact le_of_eq h
      · exact absurd hk (Nat.succ_ne_zero k).symm
    · intro h
      exact Or.inl (Nat.le_zero.mp h)
  | succ k ih =>
    rw [← hu]
    constructor
    · rintro (h | ⟨j, hj, hjk⟩)
      · exact le_of_eq h
      · have hjk' : j = k := by omega
        subst hjk'
        exact Nat.le_succ_of_le (ih.mp hj)
    · intro h
      rcases Nat.eq_or_lt_of_le h with h' | h'
      · exact Or.inl h'
      · exact Or.inr ⟨k, ih.mpr (Nat.lt_succ_iff.mp h'), rfl⟩
end

section
/-- **Exercise 4.21 (Scott 1981, PRG-19).** The order relation `≤` is the **unique**
    fixed point of
Scott's equation: any `u` with `leOp u = u` equals `leRel`. -/
theorem leOp_unique {u : Set (Nat × Nat)} (hu : leOp u = u) : u = leRel := by
  ext ⟨n, m⟩
  rw [mem_fixedPoint_iff hu]
  rfl
end
```

```
section
theorem mulOp_monotone (n : Nat) : Monotone (mulOp n) := by
  rintro u v huv k (hk | <m, hm, rfl>)
  * exact Or.inl hk
  * exact Or.inr <m, huv hm, rfl>
end

section
/-- **Exercise 4.21 (Scott 1981, PRG-19).** The least solution of the multiplication
    equation
`n*Nat = {0}  U  {n + m ? m  in  n*Nat}` is exactly the set of **multiples of `n`**. -/
theorem mulOp_lfp_eq_multiples (n : Nat) :
    lfpSet (mulOp n) = {k | exists  i, k = n * i} := by
  apply Set.Subset.antisymm
  * -- `lfpSet  subseteq  multiples`: the multiples form a pre-fixed point.
    have hpre : mulOp n {k | exists  i, k = n * i}  subseteq  {k | exists  i, k = n *
    i} := by
      rintro k (rfl | <m, <i, rfl>, rfl>)
      * exact <0, by simp>
      * exact <i + 1, by ring>
    exact lfpSet_subset (mulOp n) hpre
  * -- `multiples  subseteq  lfpSet`: each `n * i` is reached by induction on `i`.
    have hfix : mulOp n (lfpSet (mulOp n)) = lfpSet (mulOp n) :=
      lfpSet_isFixed (mulOp n) (mulOp_monotone n)
    rintro k <i, rfl>
    induction i with
    | zero =>
      rw [ <-  hfix]; exact Or.inl (by simp)
    | succ j ih =>
      rw [ <-  hfix]
      exact Or.inr <n * j, ih, by ring>
end

section
/-- `[m^+] = [m]^+`: the recursion step of 4.13(3). -/
theorem upSet_succ (m : Nat) : upSet (m + 1) = succImage (upSet m) := by
  ext k
  simp only [upSet, succImage, Set.mem_ofPred_eq]
  constructor
  * intro h
    exact <k - 1, by omega, by omega>
  * rintro <j, hj, rfl>
    omega
end

section
/-- `[0] = Nat`: the recursion base of 4.13(3). -/
theorem upSet_zero : upSet 0 = Set.univ := by
  ext k; simp [upSet]
end

section
/-- **Exercise 4.21 / 4.13(3) (Scott 1981, PRG-19).** `[*] = upSet` is the *unique*
    function with
`[0] = Nat` and `[m^+] = [m]^+`. -/
```

```
theorem upSet_unique (s : Nat  ->  Set Nat) (h0 : s 0 = Set.univ)
    (hsucc : forall  m, s (m + 1) = succImage (s m)) : s = upSet := by
  funext m
  induction m with
  | zero => rw [h0, upSet_zero]
  | succ k ih => rw [hsucc k, ih, upSet_succ]
end

section
set_option linter.unusedVariables false

theorem example_421 : And
  (And
    (And
      (forall  (s : Nat  ->  Set Nat),
        @Eq (Set Nat) (s (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0)))) (
    @Set.univ Nat)  ->
          (forall  (m : Nat),
              @Eq (Set Nat)
                (s
                  (@HAdd.hAdd Nat Nat Nat (@instHAdd Nat instAddNat) m
                    (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))))
                (DomainTheory.NeighbourhoodSystems.Exercise421.succImage (s m)))  ->
            @Eq (Nat  ->  Set Nat) s DomainTheory.NeighbourhoodSystems.Exercise421.
    upSet)
      (forall  {u : Set (Prod Nat Nat)},
        @Eq (Set (Prod Nat Nat)) (DomainTheory.NeighbourhoodSystems.Exercise421.leOp u
    ) u  ->
          @Eq (Set (Prod Nat Nat)) u DomainTheory.NeighbourhoodSystems.Exercise421.
    leRel))
     (@Eq (Set (Prod Nat Nat))
      (DomainTheory.NeighbourhoodSystems.Exercise421.leOp DomainTheory.
    NeighbourhoodSystems.Exercise421.leRel)
      DomainTheory.NeighbourhoodSystems.Exercise421.leRel))
  (forall  (n : Nat),
    @Eq (Set Nat)
      (@DomainTheory.NeighbourhoodSystems.Exercise414.lfpSet Nat
        (DomainTheory.NeighbourhoodSystems.Exercise421.mulOp n))
      (@Set.ofPred Nat fun (k : Nat) => exists  (i : Nat), @Eq Nat k (@HMul.hMul Nat
    Nat Nat (@instHMul Nat instMulNat) n i))) :=
  @And.intro
  (And
    (And
      (forall  (s : Nat  ->  Set Nat),
        @Eq (Set Nat) (s (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0)))) (
    @Set.univ Nat)  ->
          (forall  (m : Nat),
              @Eq (Set Nat)
                (s
                  (@HAdd.hAdd Nat Nat Nat (@instHAdd Nat instAddNat) m
                    (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))))
                (DomainTheory.NeighbourhoodSystems.Exercise421.succImage (s m)))  ->
            @Eq (Nat  ->  Set Nat) s DomainTheory.NeighbourhoodSystems.Exercise421.
    upSet)
      (forall  {u : Set (Prod Nat Nat)},
        @Eq (Set (Prod Nat Nat)) (DomainTheory.NeighbourhoodSystems.Exercise421.leOp u
```

```
  ) u  ->
        @Eq (Set (Prod Nat Nat)) u DomainTheory.NeighbourhoodSystems.Exercise421.
  leRel))
   (@Eq (Set (Prod Nat Nat))
     (DomainTheory.NeighbourhoodSystems.Exercise421.leOp DomainTheory.
  NeighbourhoodSystems.Exercise421.leRel)
     DomainTheory.NeighbourhoodSystems.Exercise421.leRel))
(forall  (n : Nat),
  @Eq (Set Nat)
    (@DomainTheory.NeighbourhoodSystems.Exercise414.lfpSet Nat
      (DomainTheory.NeighbourhoodSystems.Exercise421.mulOp n))
    (@Set.ofPred Nat fun (k : Nat) => exists  (i : Nat), @Eq Nat k (@HMul.hMul Nat
  Nat Nat (@instHMul Nat instMulNat) n i)))
(@And.intro
  (And
    (forall  (s : Nat  ->  Set Nat),
      @Eq (Set Nat) (s (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0)))) (
  @Set.univ Nat)  ->
        (forall  (m : Nat),
            @Eq (Set Nat)
              (s
                (@HAdd.hAdd Nat Nat Nat (@instHAdd Nat instAddNat) m
                  (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))))
              (DomainTheory.NeighbourhoodSystems.Exercise421.succImage (s m)))  ->
          @Eq (Nat  ->  Set Nat) s DomainTheory.NeighbourhoodSystems.Exercise421.
  upSet)
    (forall  {u : Set (Prod Nat Nat)},
      @Eq (Set (Prod Nat Nat)) (DomainTheory.NeighbourhoodSystems.Exercise421.leOp u
  ) u  ->
        @Eq (Set (Prod Nat Nat)) u DomainTheory.NeighbourhoodSystems.Exercise421.
  leRel))
   (@Eq (Set (Prod Nat Nat))
     (DomainTheory.NeighbourhoodSystems.Exercise421.leOp DomainTheory.
  NeighbourhoodSystems.Exercise421.leRel)
     DomainTheory.NeighbourhoodSystems.Exercise421.leRel)
   (@And.intro
     (forall  (s : Nat  ->  Set Nat),
       @Eq (Set Nat) (s (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0)))) (
  @Set.univ Nat)  ->
         (forall  (m : Nat),
             @Eq (Set Nat)
               (s
                 (@HAdd.hAdd Nat Nat Nat (@instHAdd Nat instAddNat) m
                   (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))))
               (DomainTheory.NeighbourhoodSystems.Exercise421.succImage (s m)))  ->
           @Eq (Nat  ->  Set Nat) s DomainTheory.NeighbourhoodSystems.Exercise421.
  upSet)
     (forall  {u : Set (Prod Nat Nat)},
       @Eq (Set (Prod Nat Nat)) (DomainTheory.NeighbourhoodSystems.Exercise421.leOp u
  ) u  ->
         @Eq (Set (Prod Nat Nat)) u DomainTheory.NeighbourhoodSystems.Exercise421.
  leRel)
     DomainTheory.NeighbourhoodSystems.Exercise421.upSet_unique
     @DomainTheory.NeighbourhoodSystems.Exercise421.leOp_unique)
   DomainTheory.NeighbourhoodSystems.Exercise421.leRel_isFixed)
DomainTheory.NeighbourhoodSystems.Exercise421.mulOp_lfp_eq_multiples
```

```
end

end DomainTheory.NeighbourhoodSystems.Exercise421
end
```

3.5.24 Exercise 4.22 — inductive subset of a model

Suppose $\mathbf{N}^*$ is a structured set satisfying only axioms (i) and (ii) of 4.5. Must there be a subset $\mathbf{N} \subseteq \mathbf{N}^*$ that satisfies (i), (ii), and (iii)? (Hint: Use a least fixed point in $P\mathbf{N}^*$.) (For set theorists): How do we know from the axioms of set theory that there exists such a set $\mathbf{N}^*$?

Exercise_4_22.lean

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Definition_4_5
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Exercise_4_14

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.Exercise422
open DomainTheory.NeighbourhoodSystems DomainTheory.NeighbourhoodSystems.Exercise414

section
variable {M : Type*} (zero : M) (succ : M  ->  M)

/-- Scott's operator `g(x) = {0}  U  x^+` on `P(N*)`, whose least fixed point is the
    smallest subset
containing `0` and closed under the successor. -/
def genOp (x : Set M) : Set M := {zero}  U  succ '' x

/-- *(For set theorists.)* The standard natural numbers `<Nat, 0, ^+>` form a model of
    Peano's Axioms;
their existence is the **axiom of infinity**. (This witnesses that an `N*` satisfying (
    i)/(ii) -- in
fact all three -- exists in the first place.) -/
def natPeano : PeanoModel Nat where
  zero := 0
  succ := Nat.succ
  zero_ne_succ n := (Nat.succ_ne_zero n).symm
  succ_injective := Nat.succ_injective
  induction s h0 hstep := by
    intro n
    induction n with
    | zero => exact h0
    | succ k ih => exact hstep k ih

/-- The carved-out subset `N  subseteq  N*`: the least fixed point of `g` (Exercise
    4.13(2)/4.14). -/
def nats : Set M := lfpSet (genOp zero succ)

/-- The subtype `N = {m // m  in  nats}` carrying the carved-out Peano structure. -/
abbrev Nat' : Type _ := {m : M // m  in  nats zero succ}
```

```
/-- **Induction by minimality.** Any subset `s  subseteq  N*` containing `0` and closed
     under `^+` contains
all of `N`. (This is the heart of axiom (iii): `s` is a pre-fixed point of `g`, hence
    dominates the
least fixed point `nats`.) -/
theorem nats_induction (s : Set M) (h0 : zero  in  s) (hsucc : forall  n  in  s, succ
    n  in  s) :
    nats zero succ  subseteq  s := by
  apply lfpSet_subset (genOp zero succ)
  rintro x (rfl | <n, hn, rfl>)
  * exact h0
  * exact hsucc n hn

/-- `N` is closed under the successor: `n  in  N ? n^+  in  N` -- again directly from `
    lfpSet`. -/
theorem succ_mem_nats {n : M} (hn : n  in  nats zero succ) : succ n  in  nats zero
    succ := by
  intro x hx
  exact hx (Or.inr <n, hn x hx, rfl>)

/-- The successor of the subtype (well-defined by closure `succ_mem_nats`). -/
def subSucc (n : Nat' zero succ) : Nat' zero succ := <succ n.1, succ_mem_nats zero
    succ n.2>

/-- `0  in  N` -- directly from the `lfpSet` definition (no monotonicity needed),
    keeping the
construction choice-free. -/
theorem zero_mem_nats : zero  in  nats zero succ := by
  intro x hx
  exact hx (Or.inl rfl)

/-- The zero of the subtype. -/
def subZero : Nat' zero succ := <zero, zero_mem_nats zero succ>
end
end DomainTheory.NeighbourhoodSystems.Exercise422

section
namespace DomainTheory.NeighbourhoodSystems.Exercise422
open DomainTheory.NeighbourhoodSystems DomainTheory.NeighbourhoodSystems.Exercise414
section
variable {M : Type*} (zero : M) (succ : M  ->  M)

/- concordance: source L2347 EXERCISE 4.22 inductive subset of a model -/
/-- **Exercise 4.22 (Scott 1981, PRG-19).** The carved-out subset `N  subseteq  N*`,
    with the inherited
`0` and `^+`, is a model of **all three** Peano axioms -- even though `N*` was only
    assumed to satisfy
(i) and (ii). Axioms (i)/(ii) are inherited; (iii) holds by minimality of the least
    fixed point. -/
def peanoSub (hzero : forall  n, zero  !=  succ n) (hinj : Function.Injective succ) :
    PeanoModel (Nat' zero succ) where
  zero := subZero zero succ
  succ := subSucc zero succ
  zero_ne_succ n h := hzero n.1 (Subtype.ext_iff.mp h)
  succ_injective a b h := Subtype.ext (hinj (Subtype.ext_iff.mp h))
  induction s h0 hstep := by
```

```
    -- pull `s` back to a subset of `N*`, show it is closed under `0`/`^+`, apply `
    nats_induction`
    intro n
    have hsub : nats zero succ  subseteq
        {m | exists  h : m  in  nats zero succ, (<m, h> : Nat' zero succ)  in  s} :=
    by
      apply nats_induction zero succ
      * exact <zero_mem_nats zero succ, h0>
      * rintro m <hmnat, hms>
        exact <succ_mem_nats zero succ hmnat, hstep <m, hmnat> hms>
    obtain <_, hns> := hsub n.2
    exact hns

/-- **Exercise 4.22 (Scott 1981, PRG-19).** *Yes:* whenever `N*` satisfies (i) and (ii)
    , there is a
subset `N  subseteq  N*` (carrying the inherited structure) satisfying (i), (ii), and (
    iii). -/
theorem exists_peano_submodel (hzero : forall  n, zero  !=  succ n) (hinj : Function.
    Injective succ) :
    Nonempty (PeanoModel (Nat' zero succ)) :=
  <peanoSub zero succ hzero hinj>

end

section
set_option linter.unusedVariables false

theorem example_422.{u_1} : forall  {M : Type u_1} (zero : M) (succ : M  ->  M),
  (forall  (n : M), @Ne M zero (succ n))  ->
    @Function.Injective M M succ  ->
      Nonempty
        (DomainTheory.NeighbourhoodSystems.PeanoModel (@DomainTheory.
    NeighbourhoodSystems.Exercise422.Nat' M zero succ)) :=
  @DomainTheory.NeighbourhoodSystems.Exercise422.exists_peano_submodel
end

section
variable {M : Type*} (zero : M) (succ : M  ->  M)

theorem genOp_monotone : Monotone (genOp zero succ) := by
  intro x y hxy z hz
  rcases hz with hz | <a, ha, rfl>
  * exact Or.inl hz
  * exact Or.inr <a, hxy ha, rfl>

theorem genOp_nats : genOp zero succ (nats zero succ) = nats zero succ :=
  lfpSet_isFixed (genOp zero succ) (genOp_monotone zero succ)

end

end DomainTheory.NeighbourhoodSystems.Exercise422
end
```

3.5.25 Exercise 4.23 — commuting with the iterates

(Suggested by S. Eilenberg). Suppose $f : \mathcal{D} \to \mathcal{D}$ is approximable on a given domain $\mathcal{D}$. Suppose $a_n : \mathcal{D} \to \mathcal{D}$ is a sequence of approximable maps where

(i) $a_0(x) = \bot$, for all $x \in |\mathcal{D}|$;

(ii) $a_n \sqsubseteq a_{n+1}$ in $\mathcal{D} \to \mathcal{D}$, for all $n \in \mathbf{N}$;

(iii) $\bigsqcup_{n=0}^{\infty} a_n = I_{\mathcal{D}}$ in $\mathcal{D} \to \mathcal{D}$;

(iv) $a_{n+1} \circ f = a_{n+1} \circ f \circ a_n$, for all $n \in \mathbf{N}$.

Prove that f has a unique fixed point. (Hint: Show that if $x = f(x)$, then $a_n(x) \sqsubseteq a_n(\mathrm{fix}(f))$ for all $n \in \mathbf{N}$ by induction on n.)

Exercise_4_23.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_8
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_2

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Theorem_4_2
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.bot
public import Mathlib.Order.Bounds.Defs
public import Mathlib.Basic.ExistsUnique

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap
section
variable {a : Type*} {V : NeighborhoodSystem a}
namespace ApproximableMap
/- concordance: source L2351 EXERCISE 4.23 commuting with the iterates -/
/-- **Exercise 4.23 (Scott 1981, PRG-19) -- Eilenberg.** If `f` is approximable and `
    a_n` is an
*approximation scheme* -- (i) `a_0 = False`, (ii) increasing, (iii) ` sqsup _n a_n = I`
     (pointwise `IsLUB`),
(iv) `a_n_+_1  comp  f = a_n_+_1  comp  f  comp  a_n` -- then `f` has a **unique**
    fixed point.

The unique fixed point is `fix f` (Theorem 4.1). -/
theorem f_unique_fixedPoint (f : ApproximableMap V V) (a : Nat  ->  ApproximableMap V
    V)
    (ha0 : forall  x, (a 0).toElementMap x = V.bot)
    (hlub : forall  x, IsLUB {y | exists  n, y = (a n).toElementMap x} x)
    (hcomm : forall  n x, (a (n + 1)).toElementMap (f.toElementMap x) =
      (a (n + 1)).toElementMap (f.toElementMap ((a n).toElementMap x))) :
    exists ! x, f.toElementMap x = x := by
  set fix := f.fixElement with hfixdef
  have hfixed : f.toElementMap fix = fix := toElementMap_fixElement f
```

```
  refine <fix, hfixed, ?_>
  intro x hx
  -- `fix  sqsub  x`: minimality of the least fixed point.
  have hle : fix  <=  x := fixElement_le_of_toElementMap_le f (le_of_eq hx)
  -- `a_n(x)  sqsub  a_n(fix)` by induction on `n` (Scott's hint).
  have approx_le : forall  n, (a n).toElementMap x  <=  (a n).toElementMap fix := by
    intro n
    induction n with
    | zero => rw [ha0 x]; exact bot_le
    | succ k ih =>
      have h1 : (a (k + 1)).toElementMap x =
          (a (k + 1)).toElementMap (f.toElementMap ((a k).toElementMap x)) := by
        rw [ <-  hcomm k x, hx]
      have h2 : (a (k + 1)).toElementMap (f.toElementMap ((a k).toElementMap fix)) =
          (a (k + 1)).toElementMap fix := by
        rw [ <-  hcomm k fix, hfixed]
      rw [h1,  <-  h2]
      exact toElementMap_mono (a (k + 1)) (toElementMap_mono f ih)
  -- each `a_n(fix)  sqsub  fix` (since `fix` is an upper bound of its own scheme).
  have hfix_ub : forall  n, (a n).toElementMap fix  <=  fix := fun n => (hlub fix).1 <
    n, rfl>
  -- so `fix` is an upper bound of `{a_n(x)}`, hence `x =  sqsup _n a_n(x)  sqsub  fix
    `.
  have hge : x  <=  fix := by
    apply (hlub x).2
    rintro y <n, rfl>
    exact le_trans (approx_le n) (hfix_ub n)
  exact le_antisymm hge hle

end ApproximableMap
end

end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.ApproximableMap
section
set_option linter.unusedVariables false

theorem example_423.{u_1} : forall  {a : Type u_1} {V : DomainTheory.
    NeighbourhoodSystems.NeighborhoodSystem a}
  (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V)
  (a : Nat  ->  @DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V),
  (forall  (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V),
      @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a V V
          (a (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0)))) x)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bot a V))  ->
    (forall  (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V),
        @IsLUB (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
          (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    Element a V)
            (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.Element a V)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    instPartialOrderElement a V)))
          (@Set.ofPred (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
    a V)
```

```
          fun (y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V
) =>
          exists  (n : Nat),
            @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
y
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a V
 V (a n) x))
       x)  ->
   (forall  (n : Nat) (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a V),
       @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
         (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a V V
           (a
             (@HAdd.hAdd Nat Nat Nat (@instHAdd Nat instAddNat) n
               (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))))
           (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a V V
 f x))
         (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a V V
           (a
             (@HAdd.hAdd Nat Nat Nat (@instHAdd Nat instAddNat) n
               (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))))
           (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a V V
 f
             (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a V
 V (a n) x))))  ->
    @ExistsUnique (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
 V)
       fun (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
=>
       @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V)
         (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a V V f
 x) x :=
  @DomainTheory.NeighbourhoodSystems.ApproximableMap.f_unique_fixedPoint
end

end DomainTheory.NeighbourhoodSystems.ApproximableMap
end
```

3.5.26 Exercise 4.24 — Schroeder-Bernstein theorem

(For set theorists). Let $f : A \to B$ and $g : B \to A$ be one-one functions (into, not necessarily onto!) Prove the Schroeder - Bernstein theorem to the effect that there exists a one-one correspondence $h : A \leftrightarrow B$. (Hint: (Suggested by A. Tarski). By the fixed-point theorem find $X \subseteq A$ where

$$X = (A - g(B)) \cup g(f(X)),$$

where $f(X) =$ the image of the set X under the function f. Define $h \subseteq A \times B$ as a union of two restrictions:

$$h = f \restriction X \cup g^{-1} \restriction (A - X).$$

A picture helps.)

Exercise_4_24.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NeighborhoodSystem
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Exercise_4_14
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Definition_2_1
public import Mathlib.Algebra.Group.End

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.Exercise424
open DomainTheory.NeighbourhoodSystems.Exercise414 Function

section
variable {A B : Type*} {f : A  ->  B} {g : B  ->  A}

/-- Tarski's operator `T(X) = (A ? g(B))  U  g(f(X))` on `P A`. -/
def sbOp (f : A  ->  B) (g : B  ->  A) (X : Set A) : Set A := (Set.range g)?  U  g ''
    (f '' X)

theorem sbOp_monotone (f : A  ->  B) (g : B  ->  A) : Monotone (sbOp f g) := by
  intro X Y hXY a ha
  rcases ha with ha | <b, <x, hx, rfl>, rfl>
  * exact Or.inl ha
  * exact Or.inr <f x, <x, hXY hx, rfl>, rfl>

/-- The Tarski fixed point `X` with `X = (A ? g(B))  U  g(f(X))` (`lfpSet`, Exercise
    4.14). -/
def sbSet (f : A  ->  B) (g : B  ->  A) : Set A := lfpSet (sbOp f g)

theorem sbSet_isFixed (f : A  ->  B) (g : B  ->  A) : sbOp f g (sbSet f g) = sbSet f g
     :=
  lfpSet_isFixed (sbOp f g) (sbOp_monotone f g)

/-- `g(f(a))  in  X` whenever `a  in  X` (the `g(f(X))  subseteq  T(X) = X` half). -/
theorem g_f_mem_of_mem {a : A} (ha : a  in  sbSet f g) : g (f a)  in  sbSet f g := by
  rw [ <-  sbSet_isFixed]
  exact Or.inr <f a, <a, ha, rfl>, rfl>

/-- Anything outside the fixed set `X` lies in the range of `g` (so it has a `g`-
    preimage). -/
theorem mem_range_of_not_mem {a : A} (ha : a  notin  sbSet f g) : a  in  Set.range g
    := by
  have hne : a  notin  sbOp f g (sbSet f g) := by rw [sbSet_isFixed]; exact ha
  have : not  a  notin  Set.range g := fun h => hne (Or.inl h)
  exact not_not.mp this

open Classical in
end
end DomainTheory.NeighbourhoodSystems.Exercise424

section
namespace DomainTheory.NeighbourhoodSystems.Exercise424
open DomainTheory.NeighbourhoodSystems.Exercise414 Function
section
variable {A B : Type*} {f : A  ->  B} {g : B  ->  A}
```

```
open Classical in
/- concordance: source L2363 EXERCISE 4.24 Schroeder-Bernstein theorem -/
/-- **Exercise 4.24 (Scott 1981, PRG-19).** Tarski's bijection `h : A  ->  B`: `f` on
    the fixed set
`X`, and the `g`-inverse off it. -/
noncomputable def sbFun (f : A  ->  B) (g : B  ->  A) (a : A) : B :=
  if ha : a  in  sbSet f g then f a else Classical.choose (mem_range_of_not_mem ha)

/-- Off the fixed set, `h(a)` is a genuine `g`-preimage of `a`: `g(h(a)) = a`. -/
theorem g_sbFun_not_mem {a : A} (ha : a  notin  sbSet f g) : g (sbFun f g a) = a := by
  rw [sbFun, dite_eq_right ha]
  exact Classical.choose_spec (mem_range_of_not_mem ha)

theorem sbFun_mem {a : A} (ha : a  in  sbSet f g) : sbFun f g a = f a := dite_eq_left
    ha

theorem sbFun_injective (hf : Injective f) : Injective (sbFun f g) := by
  intro a_1 a_2 heq
  by_cases h1 : a_1  in  sbSet f g <;> by_cases h2 : a_2  in  sbSet f g
  * rw [sbFun_mem h1, sbFun_mem h2] at heq
    exact hf heq
  * -- `a_1  in  X`, `a_2  notin  X`: `f a_1 = g^-1 a_2` ? `a_2 = g(f a_1)  in  X`,
    contradiction.
    exfalso
    rw [sbFun_mem h1] at heq
    have : a_2 = g (f a_1) := by rw [heq, g_sbFun_not_mem h2]
    exact h2 (this  |>  g_f_mem_of_mem h1)
  * exfalso
    rw [sbFun_mem h2] at heq
    have : a_1 = g (f a_2) := by rw [ <-  heq, g_sbFun_not_mem h1]
    exact h1 (this  |>  g_f_mem_of_mem h2)
  * -- both off `X`: apply `g` and use injectivity of `g`.
    have : g (sbFun f g a_1) = g (sbFun f g a_2) := by rw [heq]
    rw [g_sbFun_not_mem h1, g_sbFun_not_mem h2] at this
    exact this

theorem sbFun_surjective (hg : Injective g) : Surjective (sbFun f g) := by
  intro b
  by_cases hb : g b  in  sbSet f g
  * -- `g b  in  X = T X`; not in `(range g)?`, so `g b  in  g(f(X))`.
    rw [ <-  sbSet_isFixed] at hb
    rcases hb with hb | <y, <a, ha, rfl>, hgy>
    * exact absurd <b, rfl> hb
    * refine <a, ?_>
      rw [sbFun_mem ha, hg hgy]
  * -- `g b  notin  X`: then `h (g b) = b`.
    refine <g b, ?_>
    exact hg (g_sbFun_not_mem hb)

/-- **Exercise 4.24 (Scott 1981, PRG-19).** The one-one correspondence as an `Equiv A ?
     B`. -/
noncomputable def schroeder_bernstein_equiv (hf : Injective f) (hg : Injective g) : A
    ? B :=
  Equiv.ofBijective (sbFun f g) <sbFun_injective hf, sbFun_surjective hg>
```

```
open Classical in

/-- **Exercise 4.24 (Scott 1981, PRG-19) -- Schroder-Bernstein.** Injections `f : A  ->
      B` and
`g : B  ->  A` yield a bijection `A  ->  B`. -/
theorem schroeder_bernstein (hf : Injective f) (hg : Injective g) :
    exists  h : A  ->  B, Bijective h :=
  <sbFun f g, sbFun_injective hf, sbFun_surjective hg>

end

section
set_option linter.unusedVariables false

theorem example_424.{u_1, u_2} : forall  {A : Type u_1} {B : Type u_2} {f : A  ->  B}
    {g : B  ->  A},
  @Function.Injective A B f  ->  @Function.Injective B A g  ->  exists  (h : A  ->  B)
    , @Function.Bijective A B h :=
  @DomainTheory.NeighbourhoodSystems.Exercise424.schroeder_bernstein
end

end DomainTheory.NeighbourhoodSystems.Exercise424
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

end DomainTheory.NeighbourhoodSystems
end
```

3.5.27 Exercise 4.25 — unary sequence domain

Perhaps the domains $\mathbf{N}$ and C are not exactly analogous? C was based on $\{0, 1\}$ as the underlying set of tokens. Construct a system C_1 based on $\{1\}^*$ (= finite strings of 1's) with neighbourhoods:

$$C_1 = \{\, \{1^m \mid m > n\} \mid n \in \mathbf{N} \,\} \cup \{\, \{1^n\} \mid n \in \mathbf{N} \,\}.$$

What structure should be put on C_1 strictly analogous to that on $C (= C_2)$? What kinds of approximable maps relate $\mathbf{N}$, C_1, and C_2? Draw some pictures.

Exercise_4_25.lean

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Example_4_3
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Theorem_4_2

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.Exercise425
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap

section
/-- Shifting a set up by one length: `shift X = {m + 1 ? m  in  X}` (the token action
    of prepending a
`1`). -/
```

```
def shift (X : Set Nat) : Set Nat := {k | exists  m  in  X, k = m + 1}
end

section
/-- The tail `tail n = {1? ? m  >=  n} = {m ? n  <=  m}` (the partial information "at
    least `n` ones"). -/
def tail (n : Nat) : Set Nat := {m | n  <=  m}
end

section
@[simp] theorem mem_tail {n k : Nat} : k  in  tail n  <->  n  <=  k := Iff.rfl
end

section
/-- Membership in `C_1`: a neighbourhood is a tail `tail n` or a singleton `{n}`. -/
def memC1 (X : Set Nat) : Prop := (exists  n, X = tail n)  \/  (exists  n, X = {n})
end

section
theorem memC1_singleton (n : Nat) : memC1 ({n} : Set Nat) := Or.inr <n, rfl>
end

section
theorem memC1_tail (n : Nat) : memC1 (tail n) := Or.inl <n, rfl>
end

section
theorem shift_mono {X X' : Set Nat} (h : X'  subseteq  X) : shift X'  subseteq  shift
    X := by
  rintro k <m, hm, rfl>; exact <m, h hm, rfl>
end

section
/-- `1*{1?} = {1?^+1}`: shifting a singleton. -/
theorem shift_singleton (n : Nat) : shift ({n} : Set Nat) = {n + 1} := by
  ext k
  simp only [shift, Set.mem_singleton_iff, Set.mem_ofPred_eq]
  constructor
  * rintro <m, rfl, rfl>; rfl
  * rintro rfl; exact <n, rfl, rfl>
end

section
/-- `1*(1?False) = 1?^+1False`: shifting a tail. -/
theorem shift_tail (n : Nat) : shift (tail n) = tail (n + 1) := by
  ext k
  simp only [shift, tail, Set.mem_ofPred_eq]
  constructor
  * rintro <m, hm, rfl>; omega
  * intro h; exact <k - 1, by omega, by omega>
end

section
/-- Shifting keeps us inside `C_1` (`shift (tail n) = tail (n+1)`, `shift {n} = {n+1}`)
    . -/
theorem memC1_shift {X : Set Nat} (hX : memC1 X) : memC1 (shift X) := by
```

```
  rcases hX with <n, rfl> | <n, rfl>
  * exact Or.inl <n + 1, shift_tail n>
  * exact Or.inr <n + 1, shift_singleton n>
end

section
/-- A singleton and a tail are nested or disjoint. -/
theorem singleton_tail_nd (n k : Nat) :
    ({k} : Set Nat)  subseteq  tail n  \/  tail n  subseteq  {k}  \/  ({k} : Set Nat)
     I  tail n = {} := by
  by_cases h : n  <=  k
  * exact Or.inl (by intro x hx; rw [Set.mem_singleton_iff] at hx; subst hx; exact h)
  * refine Or.inr (Or.inr ?_)
    ext w
    simp only [Set.mem_inter_iff, Set.mem_singleton_iff, mem_tail, Set.
   mem_empty_iff_false,
      iff_false, not_and]
    rintro rfl hw; exact h hw
end

section
/-- Any two neighbourhoods of `C_1` are nested or disjoint. -/
theorem nestedOrDisjoint : NestedOrDisjoint memC1 := by
  rintro X Y (<n, rfl> | <n, rfl>) (<m, rfl> | <m, rfl>)
  * rcases le_total n m with h | h
    * exact Or.inr (Or.inl (fun k hk => le_trans h hk))
    * exact Or.inl (fun k hk => le_trans h hk)
  * rcases singleton_tail_nd n m with h | h | h
    * exact Or.inr (Or.inl h)
    * exact Or.inl h
    * exact Or.inr (Or.inr (by rw [Set.inter_comm]; exact h))
  * rcases singleton_tail_nd m n with h | h | h
    * exact Or.inl h
    * exact Or.inr (Or.inl h)
    * exact Or.inr (Or.inr h)
  * by_cases h : n = m
    * subst h; exact Or.inl (Set.Subset.refl _)
    * refine Or.inr (Or.inr ?_)
      ext w
      simp only [Set.mem_inter_iff, Set.mem_singleton_iff, Set.mem_empty_iff_false,
   iff_false,
       not_and]
      rintro rfl h2; exact h h2
end

section
theorem tail_zero : tail 0 = Set.univ := by ext k; simp [tail]
end

section
theorem memC1_univ : memC1 (Set.univ : Set Nat) := Or.inl <0, tail_zero.symm>
end

end DomainTheory.NeighbourhoodSystems.Exercise425

section
```

```
namespace DomainTheory.NeighbourhoodSystems.Exercise425
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap
section
/- concordance: source L2377 EXERCISE 4.25 unary sequence domain -/
/-- **Exercise 4.25 (Scott 1981, PRG-19).** The unary sequence system `C_1` on `Delta =
     {1}*  ~=  Nat`. -/
def C1 : NeighborhoodSystem Nat :=
  NeighborhoodSystem.ofNestedOrDisjoint memC1 Set.univ Set.univ_nonempty
    memC1_univ nestedOrDisjoint
    (fun _ => Set.subset_univ _)
end

section
@[simp] theorem C1_mem {X : Set Nat} : C1.mem X  <->  memC1 X := Iff.rfl
end

section
@[simp] theorem C1_master : C1.master = (Set.univ : Set Nat) := rfl
end

section
/-- **Exercise 4.25 -- the successor `x  |->  1x`** (analogous to the two successors of
     `C`). The
approximable map prepending a `1`, i.e. shifting the length: `X (1x) Y  <->  shift X
    subseteq  Y`. -/
def consMap : ApproximableMap C1 C1 where
  rel X Y := memC1 X  /\  memC1 Y  /\  shift X  subseteq  Y
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := <memC1_univ, memC1_univ, Set.subset_univ _>
  inter_right := by
    rintro X Y Y' <hX, hY, hsub> <_, hY', hsub'>
    have hsubInter : shift X  subseteq  Y  I  Y' := Set.subset_inter hsub hsub'
    exact <hX, C1.inter_mem hY hY' (memC1_shift hX) hsubInter, hsubInter>
  mono := by
    rintro X X' Y Y' <hX, hY, hsub> hX'X hYY' hX' hY'
    exact <hX', hY', (shift_mono hX'X).trans (hsub.trans hYY')>
end

section
/-- **Exercise 4.25 -- the infinite unary sequence `1^ inf `.** Unlike the flat domain
    `N` (whose only
elements are `False` and the numerals `n`), `C_1` has a genuine *infinite* element: the
     least fixed point
of the successor `x  |->  1x`, satisfying `1^ inf  = 1*1^ inf ` (`infElt_eq`). This is
    what distinguishes the
*non-flat* `C_1` (the true analogue of `C`) from `N`. -/
def infElt : C1.Element := consMap.fixElement
end

section
/-- The partial element `1?False = (tail n)` ("at least `n` ones"). -/
def oneBot (n : Nat) : C1.Element := C1.principal (memC1_tail n)
end

section
```

```
/-- The total element `1? = {n}` (the finite string of exactly `n` ones). -/
def oneElem (n : Nat) : C1.Element := C1.principal (memC1_singleton n)
end

section
/-- **Exercise 4.25 -- relating `N` and `C_1`.** The "length  |->  unary expansion" map
     `N  ->  C_1` sending
the numeral `n` to the finite string `1?` and `False` to `False` (strict). Built from
    the strict-lift
combinator `constLiftN` of Example 4.3. -/
def relateNToC1 : ApproximableMap Example43.N C1 := Example43.constLiftN C1 oneElem
end

section
/-- `1*(1?False) = 1?^+1False`. -/
theorem consMap_oneBot (n : Nat) : consMap.toElementMap (oneBot n) = oneBot (n + 1) :=
     by
  apply Element.ext
  intro Y
  constructor
  * rintro <X', <_, hXX'>, _, hY, hsub>
    refine <hY, ?_>
    have hpre : shift (tail n)  subseteq  Y := (shift_mono hXX').trans hsub
    rwa [shift_tail] at hpre
  * rintro <hY, hsub>
    refine <tail n, <memC1_tail n, subset_rfl>, memC1_tail n, hY, ?_>
    rw [shift_tail]; exact hsub
end

section
/-- `1*(1?) = 1?^+1`. -/
theorem consMap_oneElem (n : Nat) : consMap.toElementMap (oneElem n) = oneElem (n + 1)
     := by
  apply Element.ext
  intro Y
  constructor
  * rintro <X', <_, hXX'>, _, hY, hsub>
    refine <hY, ?_>
    have hpre : shift {n}  subseteq  Y := (shift_mono hXX').trans hsub
    rwa [shift_singleton] at hpre
  * rintro <hY, hsub>
    refine <{n}, <memC1_singleton n, subset_rfl>, memC1_singleton n, hY, ?_>
    rw [shift_singleton]; exact hsub
end

section
/-- `1^ inf  = 1*1^ inf `: the infinite sequence is fixed by the successor. -/
theorem infElt_eq : consMap.toElementMap infElt = infElt :=
  toElementMap_fixElement consMap
end

section
/-- `relateNToC1(False) = False`. -/
theorem relateNToC1_bot : relateNToC1.toElementMap Example43.N.bot = C1.bot :=
  Example43.constLiftN_bot C1 oneElem
end
```

```
section
set_option linter.unusedVariables false

theorem example_425 : @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    Element Nat DomainTheory.NeighbourhoodSystems.Exercise425.C1)
  (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap Nat Nat
    DomainTheory.NeighbourhoodSystems.Example43.N
    DomainTheory.NeighbourhoodSystems.Exercise425.C1 DomainTheory.NeighbourhoodSystems
    .Exercise425.relateNToC1
    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bot Nat DomainTheory.
    NeighbourhoodSystems.Example43.N))
  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.bot Nat DomainTheory.
    NeighbourhoodSystems.Exercise425.C1) :=
  DomainTheory.NeighbourhoodSystems.Exercise425.relateNToC1_bot
end

section
/-- `relateNToC1(n) = 1?`. -/
theorem relateNToC1_natElem (n : Nat) :
    relateNToC1.toElementMap (Example43.natElem n) = oneElem n :=
  Example43.constLiftN_natElem C1 oneElem n
end

end DomainTheory.NeighbourhoodSystems.Exercise425
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap
open ApproximableMap

section
variable {a : Type*} {V : NeighborhoodSystem a}

/-- **Theorem 4.2(i) (Scott 1981, PRG-19).** `fix(f) = f(fix(f))`: the value of `fix`
    is a fixed
point of the argument. (Equivalently `eval(f, fix(f)) = fix(f)` by `evalMap_apply`.) -/
theorem fixMap_fixed (V : NeighborhoodSystem a) (phi : (funSpace V V).Element) :
    (toApproxMap phi).toElementMap ((fixMap V).toElementMap phi) = (fixMap V).
    toElementMap phi := by
  rw [fixMap_toElementMap]
  exact toElementMap_fixElement (toApproxMap phi)

/-- **Theorem 4.2 (Scott 1981, PRG-19) -- uniqueness.** Any approximable operator `fax`
     satisfying
(i) and (ii) coincides with `fix`. (Scott: from (i)(ii) one proves `fix(f)  subseteq
    fax(f)` and
`fax(f)  subseteq  fix(f)`.) -/
theorem fixMap_unique (V : NeighborhoodSystem a) (fax : ApproximableMap (funSpace V V)
     V)
    (h_fix : forall  phi, (toApproxMap phi).toElementMap (fax.toElementMap phi) = fax.
    toElementMap phi)
    (h_least : forall  (phi : (funSpace V V).Element) (z : V.Element),
      (toApproxMap phi).toElementMap z  <=  z  ->  fax.toElementMap phi  <=  z) :
    fax = fixMap V := by
  apply ext_of_toElementMap
  intro phi
  apply le_antisymm
```

```
  * exact h_least phi _ (le_of_eq (fixMap_fixed V phi))
  * exact fixMap_least V phi (le_of_eq (h_fix phi))

end

end DomainTheory.NeighbourhoodSystems
end
```

3.6 Typed λ-calculus

Dependency diagram. Each box is one item. The arrowhead is on the item that imports. An arrow already implied by a longer path is left out. The introduction has no Lean file, so it is not shown. Blue is a definition, green a proposition, amber a table, red a theorem, and indigo an exercise.



Figure 34: Typed λ-calculus.

3.6.1 Introduction

LECTURE V TYPED λ - CALCULUS

In Examples 4.3 and 4.4, after suitable domains have been constructed, functions are characterized by recursion equations whose form of expression is — basically — a composition or substitution of known functions together with the function to be defined. This method can be made more precise and more easily usable by expanding our notation for functions — particularly by inventing a "temporary" notation for a function as a thing in itself without having to have special letters for functions. The device is called λ-*abstraction*. It is related to ordinary set abstraction (the $\{x \mid \ldots\}$ — notation already much used in these lectures), but we gear the approach to domains and their elements, and especially to function spaces.

At this stage it would not be so helpful to produce a rigorously formal definition of the syntax of the typed λ-calculus; we shall try to suggest what is needed by example. There are so many examples at hand, the less formal discussion ought to be sufficient.

In the first place we should set aside, in the notational store room as it were, a stock of variables

$$x, y, z, w, \ldots$$

These variables will be required in different "sizes" or "types". Roughly speaking there should be an infinite number of variables to range over the elements of each domain $\mathcal{D}$. We could perhaps write

$$x_0^{\mathcal{D}}, x_1^{\mathcal{D}}, x_2^{\mathcal{D}}, \ldots,$$

but the subscripts to insure an infinity of variables and the superscripts to record the typing of the variables lead to a notation as

tiresome to write as it is to read. We simply agree that we can have as many variables as we need and that they come in all the types.

Strictly speaking we should also introduce *type symbols* and not confuse types with domains. But if the reader will simply keep in mind that *form* in language has always to be kept distinct from *content*, the

confusion at the type level will not matter so very much. A point at which the confusion might cause a real confusion concerns *compound types*. Given $\mathcal{D}_0$ and $\mathcal{D}_1$ we can form such compounds as

$$\mathcal{D}_0 + \mathcal{D}_1, \quad \mathcal{D}_0 \times \mathcal{D}_1, \quad \mathcal{D}_0 \to \mathcal{D}_1$$

What has to be remembered is that a compound domain (neighbourhood system), $\mathcal{D}_0 \times \mathcal{D}_1$ say, does not uniquely determine the "parts" $\mathcal{D}_0$ and $\mathcal{D}_1$. (We could make it do so, but it would cost some effort.) Of course, the symbol "$\mathcal{D}_0 \times \mathcal{D}_1$" has well defined parts. The point is that *different* ways of forming a compound domain could lead to the *same* result, meaning that a domain does not let us retrace its exact history of construction. Compound symbols, however, always carry their histories around with them, since otherwise they would not be readable. What we want, of course, are *both* domain symbols *and* domains, the latter being the *meanings* of the former. Most of the time we can happily pretend that it is only the domains themselves we have to think about.

Besides variables, we will also need certain *constants*. For instance, the symbol 0 (perhaps, better $0^{\mathbf{N}}$) denotes a certain element of $|\mathbf{N}|$. Similarly, in view of Theorem 4.2, for each domain $\mathcal{D}$ there is a well-determined element $\mathrm{fix}^{\mathcal{D}}$ of the compound type $((\mathcal{D} \to \mathcal{D}) \to \mathcal{D})$ denoting the least fixed-point operator. We have considered any number of similar constants of a great variety of types already (cf. 4.3 and 4.4; cond is an especially good one). We can say that the variables and constants are *atomic terms*, where "atomic" here means non-compound.

To form compound terms, there are several means: for example, if $\tau, \ldots, \sigma$ is a list of already obtained terms (including variables or constants), then we can form an ordered *tuple*

We have already done so in 3.1. If the types of $\tau, \ldots, \sigma$ are $D, \ldots, D'$, respectively, then the type of the tuple is the product domain

$$D \times \cdots \times D',$$

because we intend that the tuple denote an element of this domain. (The tuple notation for *functions* as in 3.3 is being forgotten for the time being.)

Next suppose that τ has type $(D_0 \to D_1)$ and σ has type D_0, then the usual *function-value* notation

$$\tau(\sigma)$$

is a compound term of type D_1. We also use

$$\tau(\sigma_0, \ldots, \sigma_{n-1})$$

as an abbreviation of

$$\tau(\langle \sigma_0, \ldots, \sigma_{n-1} \rangle),$$

where, if the types of $\sigma_0, \ldots, \sigma_{n-1}$ are $D_0, \ldots, D_{n-1}$, then the type of τ has to be of the form

$$((D_0 \times \cdots \times D_{n-1}) \to D_n)$$

where D_n is the type of the compound. In this manner, with functions applied to tuples, we have the full facility of substitution into functions of many variables just by iterating the notation.

Having taken into account *function value*, it remains to provide for *function definition*. Suppose that $x_0, \ldots, x_{n-1}$ is a list of distinct variables of types $D_0, \ldots, D_{n-1}$. Suppose further that τ is a term — no matter how complicated — of type D_n. Then we can regard τ as defining a function of n-variables of type

$$((D_0 \times \cdots \times D_{n-1}) \to D_n).$$

What we have not done is to reward our regard by, as yet, providing a quick-to-write "name" for that function. This we now do; it is called

$$\lambda x_0, \ldots, x_{n-1} \cdot \tau,$$

where we stress that the x_i must be *distinct* variables and that this
expression denotes the *whole function*. That is why we provide it with a special symbol.
Here is an example of the λ - notation

$$\lambda x, y.x,$$

which is read "lambda ex wye ... (pause) ... ex". If the types of x and y are D_0 and D_1, then the type of the above is

$$((D_0 \times D_1) \to D_0).$$

Indeed, we know this function very well: it is the *first projection function* p_0 of 3.3 and the equation

$$p_0 = \lambda x, y.x$$

is true, as is the equation

$$p_1 = \lambda x, y.y.$$

In the notation of 3.3, we also find the true equation

$$\langle f, g \rangle = \lambda w.\langle f(w), g(w) \rangle,$$

where on the right-hand side we are using "official" λ - notation for a function of type

$$(D_2 \to (D_0 \times D_1)).$$

The notation on the left is just an *abbreviation* and it should not be confused with the pair (2-tuple) of type

$$((D_2 \to D_0) \times (D_2 \to D_1)).$$

(Since the two domains just mentioned are *isomorphic*, the possible confusion is not all that serious. On the other hand, one confusion we will completely overlook is that between 1-tuples $\langle x \rangle$ and elements x. Strictly speaking they are different, but we shall not bother to make the distinction.)

Here are some other examples of true equations:

$$\text{eval} = \lambda f, x.f(x) \qquad \text{(cf. 3.11)}$$

$$\text{curry} = \lambda g \lambda x \lambda y.g(x, y) \qquad \text{(cf. 3.12)}$$

The first should be immediately clear; while the second is particularly *instructive*. What is being illustrated is that the λ - notation can

be *iterated*. The distinction being drawn is between

$$\lambda x_0, x_1, \ldots, x_{n-1}.\tau \quad \text{and} \quad \lambda x_0 \lambda x_1 \ldots \lambda x_{n-1}.\tau.$$

The first has type

$$((D_0 \times D_1 \times \cdots \times D_{n-1}) \to D_n);$$

while the second has type

$$(D_0 \to (D_1 \to (\ldots (D_{n-1} \to D_n) \ldots))).$$

This is related also to the true equation

$$\mathrm{curry}(\lambda x, y.\tau) = \lambda x \lambda y.\tau,$$

which shows that there are operators relating to the two notations. The first is the *multivariate* form; the second is the *curried* form.

Here is another true equation

$$\mathrm{fix} = \mathrm{fix}(\lambda F \lambda f.f(F(f))),$$

where the fix on the left has type $((D \to D) \to D)$ and that on the right type

$$((((D \to D) \to D) \to ((D \to D) \to D)) \to ((D \to D) \to D)).$$

This is the content of Exercise 4.9. (This also shows why type superscripts are tiresome.)

The combination

$$\mathrm{fix}(\lambda x.\tau)$$

occurs so often, that from time to time we abbreviate it as

$$!x.\tau,$$

but remember it only makes sense if x and τ have the *same* type. For example in 4.3 we could have written

$$\sigma = !f \lambda n.\mathrm{cond}(\mathrm{zero}(n), 0, f(\mathrm{pred}(n)) + \mathrm{pred}(n))$$

and read this as

"σ is the least (recursively defined) function f whose value at n is cond $(\ldots)$."

We note that in the so-called "body" of the expression inside the

cond-part the variable f occurs again. That is just the point! This is a *recursive definition*; it is made into an *explicit* definition by invoking the least fixed-point operator.

In a λ-expression, $\lambda x, y, z.\tau$, say, the variables x, y, z are being *bound* in τ; but τ may have other variables that are nowhere bound in τ and these remain *free variables* of the whole expression. Bound variables are *dummy variables* and may be rewritten by other variables; thus

$$\lambda x.\tau = \lambda y.\tau[y/x]$$

is a true equation *PROVIDED* the variable y does not occur in τ. In the equation the notation $\tau[y/x]$ means the result of *substituting* (*rewriting*) the variable y for the variable x throughout the term τ. We can also write $\tau[\sigma/x]$ for substituting a whole term σ for a variable in the other term.

We have already spoken of "true equations", but how do we know that these curious equations are meaningful at all? They are, but this is something that has to be proved.

3.6.2 Theorem 5.1 — terms denote approximable maps

THEOREM 5.1. Every typed λ-term τ defines an approximable function of its free variables.

Proof: We argue by an induction on the complexity of τ; there will only be a few cases to consider since the "syntax" of λ-terms is limited — even though terms can be of any length.

If τ is a variable or a constant there is nothing to prove. We already know that

$$x \mapsto x \qquad \text{and} \qquad x \mapsto k$$

are approximable functions.

Suppose τ has the form

$$\langle \sigma_0, \ldots, \sigma_{n-1} \rangle.$$

Then the σ_i are less complex terms, and so we can assume — as our induction hypothesis — that they define approximable functions of the free variables. Having said this, we just apply the already

proved 3.4 to conclude (after a suitable generalization to the multivariate case) that τ, which takes on tuples as values, also defines an approximable function.

Next, suppose τ has the form

$$\sigma_0(\sigma_1),$$

where we are sure that the types of all the terms match properly. Again we can assume the σ_i to be well behaved. But the values we seek can also be written as

$$\mathrm{eval}(\sigma_0, \sigma_1).$$

Since eval is approximable by 3.11, we just have to invoke an instance of 3.7 to gain the desired conclusion.

Finally, suppose that τ has the form

$$\lambda x.\sigma.$$

By a judicious choice of the order of the variables in σ (including x), we can assume that σ defines an approximable function

$$g : D_0 \times \cdots \times D_{n-1} \times D_n \to D'$$

where D' is the type of σ, D_n is the type of x, and $D_0, \ldots, D_{n-1}$ are the types of the remaining free variables of σ. We apply 3.12 and obtain an approximable function

$$\text{curry}(g) : D_0 \times \cdots \times D_{n-1} \to (D_n \to D').$$

But, this is just exactly the function defined by τ.

We leave as an exercise the more general case of a term τ of the form

$$\lambda x_0, \ldots, x_{k-1}.\sigma,$$

which has a string of bound variables. □

We can now say more precisely what it means to call $\sigma = \tau$ a "true equation". This means that, if we employ the method of the proof of 5.1, the two terms define the *same function* of the free variables. For example,

$$\lambda x.\tau = \lambda y.\tau[y/x]$$

is true, provided y does not occur free in the term τ, since the systematic generation of the function defined by $\lambda x.\tau$ does not depend on what the variable x *looks like* but only on its *position* in the term τ. Some other obviously desirable rules for generating true equations are stated in the exercises. But one rule is so basic that we state it here in full generality.

Theorem_5_1.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_12
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Definition_3_3
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Lemma_3_6

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

section
variable {a b ? : Type*}
  {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b} {V_2 : NeighborhoodSystem
    ?}

/- concordance: source L2599 THEOREM 5.1 terms denote approximable maps -/
/-- **Theorem 5.1 (Scott 1981, PRG-19).** Every typed fun -term denotes an
    approximable map of its
free variables. Abstraction is `curry`, application is `eval  comp  <_, _>`, a constant
     is the constant
map, a tuple is `paired`, and a variable is a projection. -/
theorem theorem_51 :
    (forall  (g : ApproximableMap (prod V_0 V_1) V_2) (v : V_0.Element) (x : V_1.
    Element),
      (toApproxMap ((curry g).toElementMap v)).toElementMap x = g.toElementMap (pair v
     x))  /\
    (forall  (F : ApproximableMap V_2 (funSpace V_0 V_1)) (G : ApproximableMap V_2 V_0
    ) (w : V_2.Element),
      ((evalMap V_0 V_1).comp (paired F G)).toElementMap w =
        (toApproxMap (F.toElementMap w)).toElementMap (G.toElementMap w))  /\
```

```
    (forall  (k : V_1.Element) (x : V_0.Element), (constMap V_0 k).toElementMap x = k)
     /\
    (forall  (f : ApproximableMap V_2 V_0) (g : ApproximableMap V_2 V_1) (w : V_2.
   Element),
      (paired f g).toElementMap w = pair (f.toElementMap w) (g.toElementMap w))  /\
    (forall  (x : V_0.Element) (y : V_1.Element), (proj_0 V_0 V_1).toElementMap (pair
   x y) = x) :=
  <fun g v x => toElementMap_curry_apply g v x,
   fun F G w => toElementMap_evalMap_comp_paired F G w,
   fun k x => toElementMap_constMap k x,
   fun f g w => toElementMap_paired f g w,
   fun x y => toElementMap_proj_0_pair x y>

end
end DomainTheory.NeighbourhoodSystems
```

3.6.3 Theorem 5.2 — beta conversion

THEOREM 5.2. For suitably typed λ-terms the following equation is true:

$$(\lambda x_0, \ldots, x_{n-1} \cdot \tau)(\sigma_0, \ldots, \sigma_{n-1}) = \tau[\sigma_0/x_0, \ldots, \sigma_{n-1}/x_{n-1}].$$

Proof: It will be sufficient to carry out the proof for $n = 1$. The proof proceeds by induction on the complexity of the term τ.

In case τ is a *constant* k, the result reads

$$(\lambda x.k)(\sigma) = k,$$

and this is a true equation.

In case τ is a *variable* (in particular, the variable x), the result reads

$$(\lambda x.x)(\sigma) = \sigma,$$

and again this is a true equation.

In case τ is a *tuple* (say, $\langle \tau_0, \tau_1 \rangle$) the result reads

$$(\lambda x.\langle \tau_0, \tau_1 \rangle)(\sigma) = \langle \tau_0[\sigma/x], \tau_1[\sigma/x] \rangle.$$

This is true, because the left-hand side can be transformed by the true equation

$$(\lambda x.\langle \tau_0, \tau_1 \rangle)(\sigma) = \langle (\lambda x.\tau_0)(\sigma), (\lambda x.\tau_1)(\sigma) \rangle;$$

and then we apply the inductive assumption for τ_0 and for τ_1.

In case τ is an *application*, we want (supposing the term is $\tau_0(\tau_1)$),

$$(\lambda x.\tau_0(\tau_1))(\sigma) = \tau_0[\sigma/x](\tau_1[\sigma/x]).$$

We can proceed as in the last case, noting that the left-hand side equals

$$\text{eval}\ ((\lambda x.\langle \tau_0, \tau_1 \rangle)\ (\sigma))\ .$$

In case τ is an *abstract* (say, $\lambda y.\tau_0$), we want

$$(\lambda x.\lambda y.\tau_0)\ (\sigma) = \lambda y.\tau_0[\sigma/x]$$

PROVIDED the variable y is not free in σ. For this we require the true equation

$$(\lambda x.\lambda y.\tau)\ (\sigma) = \lambda y.(\lambda x.\tau)\ (\sigma)\ .$$

We argue for this by letting g be the function of $n+2$ free variables defined by τ. Then, by 5.1, the λ-term $\lambda x.\lambda y.\tau$ defines the function $\mathrm{curry}\ (\mathrm{curry}\ (g))$ of n arguments. We can call this function h for the moment. We can write

$$h(v)\ (\sigma)\ (y) = g(v, \sigma, y)\ ,$$

where v is a list of arguments. But, with an appropriate combinator inv, which applied to g inverts the order of the last two arguments, we can write

$$h(v)\ (\sigma)\ (y) = \mathrm{curry}\ (\mathrm{inv}\ (g))\ (v, y)\ (\sigma)\ .$$

But, $\mathrm{curry}\ (\mathrm{inv}\ (g))$ is just the function defined by $(\lambda x.\tau)$. So what we have proved as true is

$$(\lambda x.\lambda y.\tau)\ (\sigma)\ (y) = (\lambda x.\tau)\ (\sigma)\ .$$

But if y is not free in α and

$$\alpha(y) = \beta$$

is true, then so is

$$\alpha = \lambda y.\beta\ .$$

This completes the proof. □

We note that if τ' is the term $\lambda x, y.\tau$, then $\tau'(x, y)$ means the same as τ. This gives a convenient way of indicating free variables: we just write $\sigma(x, y)$ — where x, y are not free in σ — and this will have the same values as any term τ which does involve the extra free variables x and y. We use this notational device in the next theorem.

Theorem_5_2.lean

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_12
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Lemma_3_6

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap
section
```

```
variable {a b ? ? : Type*}
  {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b} {V_2 : NeighborhoodSystem
    ?}
namespace Theorem52

/- concordance: source L2657 THEOREM 5.2 beta conversion -/
/-- **Theorem 5.2 (Scott 1981, PRG-19) -- b as a map equation.** Substituting the
    constant `sigma` for
the bound variable: `(fun x.tau)(sigma) = tau[sigma/x]`, where the left side first
    abstracts (`curry g`) then
applies to the constant `sigma`, and the right side substitutes `sigma` directly. Both
    are approximable
maps `Gamma  ->  ?'` and they are equal. -/
theorem beta (g : ApproximableMap (prod V_0 V_1) V_2) (sigma : V_1.Element) :
    (evalMap V_1 V_2).comp (paired (curry g) (constMap V_0 sigma))
      = g.comp (paired (idMap V_0) (constMap V_0 sigma)) := by
  apply ext_of_toElementMap
  intro v
  rw [toElementMap_comp, toElementMap_paired, evalMap_apply, toElementMap_curry_apply,
    toElementMap_constMap, toElementMap_comp, toElementMap_paired, toElementMap_idMap,
    toElementMap_constMap]

/-- **Theorem 5.2 (Scott 1981, PRG-19) -- abstraction case (`inv` step).**
`(fun x.fun y.tau)(sigma)(y) = tau[sigma/x]` evaluated at `y`: the double-curry
    application law. The bound `y` is
carried inert past the substitution of `sigma` for `x`, exactly Scott's
`(fun x.fun y.tau)(sigma)(y) = (fun x.tau)(sigma)`. -/
theorem beta_abs (g : ApproximableMap (prod (prod V_0 V_1) V_2) V_3) (v : V_0.Element)
     (sigma : V_1.Element)
    (y : V_2.Element) :
    (toApproxMap ((toApproxMap ((curry (curry g)).toElementMap v)).toElementMap sigma)
    ).toElementMap y
      = g.toElementMap (pair (pair v sigma) y) := by
  rw [toElementMap_curry_apply, toElementMap_curry_apply]

/-- **Theorem 5.2 (Scott 1981, PRG-19) -- tuple case.**
`(fun x.<tau_0, tau_1>)(sigma) = <(fun x.tau_0)(sigma), (fun x.tau_1)(sigma)>`:
    abstraction distributes over a tuple. -/
theorem beta_tuple (g_0 : ApproximableMap (prod V_0 V_1) V_2) (g_1 : ApproximableMap (
    prod V_0 V_1) V_3)
    (v : V_0.Element) (x : V_1.Element) :
    (toApproxMap ((curry (paired g_0 g_1)).toElementMap v)).toElementMap x
      = pair ((toApproxMap ((curry g_0).toElementMap v)).toElementMap x)
          ((toApproxMap ((curry g_1).toElementMap v)).toElementMap x) := by
  rw [toElementMap_curry_apply, toElementMap_paired, toElementMap_curry_apply,
    toElementMap_curry_apply]

/-- **Theorem 5.2, base value law (Scott 1981, PRG-19).** `(fun x.tau)(sigma) = tau[
    sigma/x]` evaluated: applying
the abstraction `curry g` (in free-variable context `v`) to `x` substitutes, giving `g(
    v, x)`. -/
theorem subst_value (g : ApproximableMap (prod V_0 V_1) V_2) (v : V_0.Element) (x :
    V_1.Element) :
    (toApproxMap ((curry g).toElementMap v)).toElementMap x = g.toElementMap (pair v x
    ) :=
  toElementMap_curry_apply g v x
```

```
end Theorem52
end

end DomainTheory.NeighbourhoodSystems
end
```

3.6.4 Proposition 5.3 — simultaneous fixed points

PROPOSITION 5.3. The least fixed point of $\lambda x, y.\ \langle \tau(x,y), \sigma(x,y)\rangle$ is the pair with coordinates

$!x.\ \tau(x,\ !y.\ \sigma(x,y))$ and

$!y.\ \sigma(!x.\ \tau(x,y), y)$.

Proof: (We are assuming that x and y are not free in τ and σ.) The purpose of the fixed-point search is to find the least solution of the pair of equations $x = \tau(x,y)$ and $y = \sigma(x,y)$. In other words, we are generalizing the fixed-point equation from one to two variables—and, of course, we could go much further to any number of variables. To this end, let

$y_* = !y.\ \sigma(!x.\ \tau(x,y), y)$, and

$x_* = !x.\ \tau(x, y_*)$.

Then

$x_* = \tau(x_*, y_*)$,

and

$y_* = \sigma(!x.\ \tau(x, y_*), y_*)$

$= \sigma(x_*, y_*)$.

This proves that $\langle x_*, y_*\rangle$ is one fixed-point pair.

Suppose, then, that $\langle x_0, y_0\rangle$ is the least solution. (Why does a least solution have to exist? Hint: Consider a suitable mapping of type $D_0 \times D_1 \to D_0 \times D_1$, where D_0 is the type of x and D_1 the type of y.) Then we know $x_0 = \tau(x_0, y_0)$ and $y_0 = \sigma(x_0, y_0)$, and also $x_0 \sqsubseteq x_*$ and $y_0 \sqsubseteq y_*$. But from

$\tau(x_0, y_0) \sqsubseteq x_0$,

it follows that

$!x.\ \tau(x, y_0) \sqsubseteq x_0$.

Consequently,

$$\sigma(!x.\tau(x, y_0), y_0) \sqsubseteq \sigma(x_0, y_0) \sqsubseteq y_0.$$

By the fixed-point definition of y_*, we have $y_* \sqsubseteq y_0$, so $y_* = y_0$; whence,

$$x_* = !x.\tau(x, y_*) = !x.\tau(x, y_0) \sqsubseteq x_0.$$

So also $x_* = x_0$. □ We have the right formula for y_0, and a similar argument gives x_0.

The purpose of giving the above proof was to illustrate the use of the least fixed-point operator in proofs. We have such true principles as:

$$!x.\tau(x) = \tau(!x.\tau(x));$$

and

$$\tau(y) \sqsubseteq y \text{ implies } !x.\tau(x) \sqsubseteq y,$$

provided, of course, that x is not free in τ. These, together with the monotonicity of all the functions, were just the methods used in the above proof. Here is another example.

Proposition_5_3.lean

```
module

public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.Table_5_5

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap
section
variable {a b : Type*} {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
namespace ApproximableMap
/- concordance: source L2745 PROPOSITION 5.3 simultaneous fixed points -/
/-- **Proposition 5.3 (Scott 1981, PRG-19).** Bekic's theorem: the least fixed point
    of
`F = <tau, sigma>` is the pair `<!x.tau(x, !y.sigma(x,y)-coordinate), !y.sigma(!x.tau(x
    ,y), y)>`. Here the second
coordinate is `ystar = (outerOp tau sigma).fixElement` and the first is `xstar =
    secFixX tau (ystar)`. -/
theorem fixElement_paired_eq (tau : ApproximableMap (prod V_0 V_1) V_0)
    (sigma : ApproximableMap (prod V_0 V_1) V_1) :
    (paired tau sigma).fixElement =
      pair ((secFixX tau).toElementMap (outerOp tau sigma).fixElement) (outerOp tau
    sigma).fixElement := by
  set ystar := (outerOp tau sigma).fixElement with hystar
  set xstar := (secFixX tau).toElementMap ystar with hxstar
  -- `sigma(xstar, ystar) = ystar`: `ystar` is the fixed point of `outerOp`.
  have hsigma : sigma.toElementMap (pair xstar ystar) = ystar := by
    have := toElementMap_fixElement (outerOp tau sigma)
    rw [ <-  hystar] at this
    rw [outerOp_apply] at this
    rw [hxstar]; exact this
  -- `tau(xstar, ystar) = xstar`: `xstar = !x.tau(x, ystar)` is a fixed point of its
    section.
  have htau : tau.toElementMap (pair xstar ystar) = xstar := by
    rw [hxstar]; exact secFixX_isFixed tau ystar
  apply le_antisymm
  * -- `F.fixElement  sqsub  <xstar, ystar>`: the pair is a fixed point of `F`.
    apply fixElement_le_of_toElementMap_le
    rw [toElementMap_paired]
    rw [htau, hsigma]
  * -- `<xstar, ystar>  sqsub  F.fixElement`.
    set fp := (paired tau sigma).fixElement with hfp
    set a := fp.fst with ha
    set b := fp.snd with hb
    -- `tau(a, b) = a` and `sigma(a, b) = b` (decompose the fixed point of `F`).
    have hfpeq : pair (tau.toElementMap fp) (sigma.toElementMap fp) = fp := by
      have := toElementMap_fixElement (paired tau sigma)
      rw [ <-  hfp, toElementMap_paired] at this
      exact this
    have hfp_pair : fp = pair a b := (pair_fst_snd fp).symm
```

```
    have htaua : tau.toElementMap (pair a b) = a := by
      have : (pair (tau.toElementMap fp) (sigma.toElementMap fp)).fst = fp.fst := by
    rw [hfpeq]
      rw [fst_pair] at this
      rw [ <-  hfp_pair]; rw [ha]; exact this
    have hsigmab : sigma.toElementMap (pair a b) = b := by
      have : (pair (tau.toElementMap fp) (sigma.toElementMap fp)).snd = fp.snd := by
    rw [hfpeq]
      rw [snd_pair] at this
      rw [ <-  hfp_pair]; rw [hb]; exact this
    -- `!x.tau(x, b)  sqsub  a` since `tau(a, b) = a`.
    have hxfix_le : (secFixX tau).toElementMap b  <=  a := by
      rw [secFixX_apply]
      apply fixElement_le_of_toElementMap_le
      rw [sectionX_apply]
      exact le_of_eq htaua
    -- `outerOp(b) = sigma(!x.tau(x,b), b)  sqsub  sigma(a, b) = b`.
    have houter_le : (outerOp tau sigma).toElementMap b  <=  b := by
      rw [outerOp_apply]
      calc sigma.toElementMap (pair ((secFixX tau).toElementMap b) b)
           <=  sigma.toElementMap (pair a b) :=
            sigma.toElementMap_mono (pair_le_pair_iff.mpr <hxfix_le, le_refl b>)
        _ = b := hsigmab
    -- so `ystar  sqsub  b`, and `xstar = !x.tau(x, ystar)  sqsub  !x.tau(x, b)  sqsub
     a`.
    have hystar_le : ystar  <=  b := by
      rw [hystar]
      exact fixElement_le_of_toElementMap_le (outerOp tau sigma) houter_le
    have hxstar_le : xstar  <=  a := by
      rw [hxstar]
      calc (secFixX tau).toElementMap ystar
           <=  (secFixX tau).toElementMap b := (secFixX tau).toElementMap_mono
    hystar_le
        _  <=  a := hxfix_le
    rw [hfp_pair]
    exact pair_le_pair_iff.mpr <hxstar_le, hystar_le>

end ApproximableMap
end

end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

section
variable {a b ? ? : Type*}
  {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b} {V_2 : NeighborhoodSystem
    ?}

namespace Theorem51

end Theorem51
end
end DomainTheory.NeighbourhoodSystems
end
```

3.6.5 Proposition 5.4 — fixed point under a parameter

PROPOSITION 5.4. Let x, y, and $\tau(x, y)$ be of the same type D and let g be of type $(D \to D)$, then the equation

$$\lambda x.!y.\tau(x, y) =!g.\lambda x.\tau(x, g(x))$$

is true.

Proof : Let f be the function on the left-hand side. We can write

$$f(x) =!y.\tau(x, y) = \tau(x, f(x)).$$

Therefore

$$f = \lambda x.\tau(x, f(x)),$$

and it follows that

$$g_0 =!g.\lambda x.\tau(x, g(x)) \sqsubseteq f.$$

Then we have at once, by definition of g_0,

$$g_0(x) = \tau(x, g_0(x)),$$

for any given x. But by definition of f we find

$$f(x) =!y.\tau(x, y) \sqsubseteq g_0(x).$$

As this holds for all x, then $f \sqsubseteq g_0$ follows. So the equation is true. □

The last proof is instructive as it uses equations and inclusions between functions. In particular we have just made use of the principle:

> if $\tau \sqsubseteq \sigma$ holds for all values of x,
> then $\lambda x.\tau \sqsubseteq \lambda x.\sigma$ holds.

This is another form of Theorem 3.13(i).

Proposition_5_4.lean

```
module

public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.Exercise_5_16

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_13

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap
section
variable {a : Type*} {V : NeighborhoodSystem a}
namespace ApproximableMap
```

```
/- concordance: source L2799 PROPOSITION 5.4 fixed point under a parameter -/
/-- **Proposition 5.4 (Scott 1981, PRG-19).** `fun x. !y. tau(x, y) = !g. fun x. tau(x,
     g(x))`. -/
theorem pfix_eq_fixElement_recOp (tau : ApproximableMap (prod V V) V) :
    pfix tau = toApproxMap (recOp tau).fixElement := by
  set g_0 := (recOp tau).fixElement with hg_0
  set G := toApproxMap g_0 with hG
  -- `G(x) = tau(x, G(x))`: `g_0` is a fixed point of `recOp tau`.
  have hG_fixed : forall  x, G.toElementMap x = tau.toElementMap (pair x (G.
    toElementMap x)) := by
    intro x
    have hfix : (recOp tau).toElementMap g_0 = g_0 := toElementMap_fixElement (recOp
    tau)
    have := recOp_apply tau g_0 x
    rw [hfix] at this
    rw [hG]; exact this
  apply le_antisymm
  * -- `pfix tau  sqsub  G`: `pfix tau (x)` is the least fixed point of the section, `G
    (x)` is some fixed point.
    rw [le_iff_toElementMap_le]
    intro x
    rw [pfix_apply]
    apply fixElement_le_of_toElementMap_le
    rw [section_2_apply]
    exact le_of_eq (hG_fixed x).symm
  * -- `G  sqsub  pfix tau`: `toFilter (pfix tau)` is a fixed point of `recOp tau`, so
    `g_0  sqsub  toFilter (pfix tau)`.
    have hpre : (recOp tau).toElementMap (toFilter (pfix tau))  <=  toFilter (pfix tau
    ) := by
      apply le_of_eq
      have hmap : toApproxMap ((recOp tau).toElementMap (toFilter (pfix tau))) = pfix
    tau := by
        apply ext_of_toElementMap
        intro x
        rw [recOp_apply]
        have hround : toApproxMap (toFilter (pfix tau)) = pfix tau := by
          have he := (funSpaceEquiv V V).apply_symm_apply (pfix tau)
          rwa [funSpaceEquiv_apply, funSpaceEquiv_symm_apply] at he
        rw [hround, pfix_isFixed]
      -- transport `hmap` through the iso `toApproxMap`/`toFilter`.
      have h1 : toFilter (toApproxMap ((recOp tau).toElementMap (toFilter (pfix tau)))
    ) =
          toFilter (pfix tau) := by rw [hmap]
      have hround2 : toFilter (toApproxMap ((recOp tau).toElementMap (toFilter (pfix
    tau)))) =
          (recOp tau).toElementMap (toFilter (pfix tau)) := by
        have he := (funSpaceEquiv V V).symm_apply_apply ((recOp tau).toElementMap (
    toFilter (pfix tau)))
        rwa [funSpaceEquiv_apply, funSpaceEquiv_symm_apply] at he
      rw [hround2] at h1
      exact h1
    have hle : g_0  <=  toFilter (pfix tau) := fixElement_le_of_toElementMap_le (recOp
     tau) hpre
    have := (funSpaceEquiv V V).monotone hle
    rw [funSpaceEquiv_apply, funSpaceEquiv_apply] at this
    have hround : toApproxMap (toFilter (pfix tau)) = pfix tau := by
```

```
      have he := (funSpaceEquiv V V).apply_symm_apply (pfix tau)
      rwa [funSpaceEquiv_apply, funSpaceEquiv_symm_apply] at he
    rw [hG,  <-  hround]
    exact this

end ApproximableMap
end

end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

section
variable {a b ? : Type*}
  {V : NeighborhoodSystem a} {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
  {V_2 : NeighborhoodSystem ?}

end
end DomainTheory.NeighbourhoodSystems
end
```

3.6.6 Table 5.5 — table of combinators

TABLE 5.5. In the displayed table we give a summary of uses of the λ-notation to define various combinators. We have mentioned some of these equations before, and there are some combinators here we have not mentioned before — their meanings, however, should be clear.

$$
\begin{aligned}
P_0 &= \lambda x, y.x \\
P_1 &= \lambda x, y.y \\
\text{pair} &= \lambda x \lambda y.\langle x, y\rangle \\
\text{n-tuple} &= \lambda x_0 \lambda x_1 \ldots \lambda x_{n-1}.\langle x_0, x_1, \ldots, x_{n-1}\rangle \\
\text{diag} &= \lambda x.\langle x, x\rangle \\
\text{funpair} &= \lambda f \lambda g \lambda x.\langle f(x), g(x)\rangle \\
\text{proj}_i^n &= \lambda x_0, x_1, \ldots, x_{n-1}.x_i \\
\text{inv}_{i,j}^n &= \lambda x_0, \ldots, x_i, \ldots, x_j, \ldots, x_{n-1}.\langle x_0, \ldots, x_j, \ldots, x_i, \ldots, x_{n-1}\rangle \\
\text{eval} &= \lambda f, x.f(x) \\
\text{curry} &= \lambda g \lambda x \lambda y.g(x, y) \\
\text{comp} &= \lambda g, f \lambda x.g(f(x)) \\
\text{const} &= \lambda k \lambda x.k \\
\text{fix} &= \lambda f! x.f(x)
\end{aligned}
$$

A TABLE OF COMBINATORS

It is important to note that since we have not typed the variables, these equations are ambiguous: they only become precise when the types are specified. It follows, therefore, that what we find in the table are *schemes* for combinators; there are actually infinitely many distinct combinators corresponding to any one equation depending on how the variables have types chosen for them. Clearly it is better to imagine this variety of combinators than it is to try to notate them with type superscripts.

One interest of combinators is that it is often possible to write expressions without variables — if enough combinators are used. This is sometimes useful, but it can become clumsy. On the other hand, if the same combination occurs over and over, it is sometimes useful to give it a name. This is what we do with, say, *composition* where

$$\mathrm{comp}\,(g, f) = g \circ f.$$

On the one side we have the prefix notation, and on the other, the more common infix notation. With either notation the variable seen in $\lambda x.\, g(f(x))$ has been got rid of. The choice between equivalent notations ought to be based on a desire for readability. □

The reader will have noted that there are some combinators not appearing in Table 5.5. The reason is that combinators like cond, succ, pred, zero, 0 cannot be defined in the pure λ-notation but are specific to domains like T and $\mathbf{N}$; we, thus, have to regard them as primitive. But once they are in hand, a very large number of other functions can be defined from these combined with λ-expressions. The next theorem gives an indication of the possibilities.

Table_5_5.lean

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Exercise_4_25
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Theorem_4_2
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Theorem_2_7
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_12
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Lemma_3_6

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap
section
variable {a b ? : Type*}
  {V : NeighborhoodSystem a} {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
  {V_2 : NeighborhoodSystem ?}

/- concordance: source L2836 TABLE 5.5 table of combinators -/
/-- **Table 5.5 (Scott 1981, PRG-19).** `P_0 = fun x,y.x`, the first projection `?_0  x
      ?_1  ->  ?_0`. -/
def P_0 (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) : ApproximableMap (
    prod V_0 V_1) V_0 :=
  proj_0 V_0 V_1

@[simp] theorem P_0_apply (x : V_0.Element) (y : V_1.Element) :
    (P_0 V_0 V_1).toElementMap (pair x y) = x := by
  rw [P_0, toElementMap_proj_0, fst_pair]

/-- **Table 5.5 (Scott 1981, PRG-19).** `P_1 = fun x,y.y`, the second projection `?_0
    x  ?_1  ->  ?_1`. -/
def P_1 (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) : ApproximableMap (
    prod V_0 V_1) V_1 :=
```

```
  proj_1 V_0 V_1

@[simp] theorem P_1_apply (x : V_0.Element) (y : V_1.Element) :
    (P_1 V_0 V_1).toElementMap (pair x y) = y := by
  rw [P_1, toElementMap_proj_1, snd_pair]

/-- **Table 5.5 (Scott 1981, PRG-19).** `diag = fun x.<x,x>`, the diagonal `?  ->  ?  x
     ?`. -/
def diagC (V : NeighborhoodSystem a) : ApproximableMap V (prod V V) := paired (idMap V
    ) (idMap V)

@[simp] theorem diagC_apply (x : V.Element) : (diagC V).toElementMap x = pair x x :=
    by
  rw [diagC, toElementMap_paired, toElementMap_idMap]

/-- **Table 5.5 (Scott 1981, PRG-19).** `pair = fun x fun y.<x,y>`, as the curried map
`?_0  ->  (?_1  ->  ?_0  x  ?_1)`. -/
def pairC (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) :
    ApproximableMap V_0 (funSpace V_1 (prod V_0 V_1)) := curry (idMap (prod V_0 V_1))

/-- **Table 5.5 (Scott 1981, PRG-19).** The base case `inv_{0,1}^2 = fun x,y.<y,x>` of
    the
argument-swap scheme: `?_0  x  ?_1  ->  ?_1  x  ?_0`. -/
def swapC (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) :
    ApproximableMap (prod V_0 V_1) (prod V_1 V_0) := paired (proj_1 V_0 V_1) (proj_0
    V_0 V_1)

@[simp] theorem swapC_apply (x : V_0.Element) (y : V_1.Element) :
    (swapC V_0 V_1).toElementMap (pair x y) = pair y x := by
  rw [swapC, toElementMap_paired, toElementMap_proj_1, toElementMap_proj_0, snd_pair,
    fst_pair]

/-- The uncurried `(g, f), x  |->  g(f(x))` over `((?_1 -> ?_2)  x  (?_0 -> ?_1))  x  ?
    _0  ->  ?_2`, built purely from
projections, pairing and `eval` (this is the variable-free expression Scott alludes to)
    . -/
def compMapTbl (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) (V_2 :
    NeighborhoodSystem ?) :
    ApproximableMap (prod (prod (funSpace V_1 V_2) (funSpace V_0 V_1)) V_0) V_2 :=
  (evalMap V_1 V_2).comp
    (paired
      ((proj_0 (funSpace V_1 V_2) (funSpace V_0 V_1)).comp
        (proj_0 (prod (funSpace V_1 V_2) (funSpace V_0 V_1)) V_0))
      ((evalMap V_0 V_1).comp
        (paired
          ((proj_1 (funSpace V_1 V_2) (funSpace V_0 V_1)).comp
            (proj_0 (prod (funSpace V_1 V_2) (funSpace V_0 V_1)) V_0))
          (proj_1 (prod (funSpace V_1 V_2) (funSpace V_0 V_1)) V_0))))

/-- **Table 5.5 (Scott 1981, PRG-19).** `comp = fun g,f fun x.g(f(x))` as an
    approximable map
`((?_1 -> ?_2)  x  (?_0 -> ?_1))  ->  (?_0  ->  ?_2)`. -/
def compC (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) (V_2 :
    NeighborhoodSystem ?) :
    ApproximableMap (prod (funSpace V_1 V_2) (funSpace V_0 V_1)) (funSpace V_0 V_2) :=
  curry (compMapTbl V_0 V_1 V_2)
```

```
/-- **Table 5.5 (Scott 1981, PRG-19).** `const = fun k fun x.k`, sending `k  in  |?_1|`
     to the constant map
`?_0  ->  ?_1`. Realized as `curry(p_0)`. -/
def constC (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) :
    ApproximableMap V_1 (funSpace V_0 V_1) := curry (proj_0 V_1 V_0)

/-- The order-isomorphism `|?_0  x  ?_1  ->  ?_2| ?o |?_0  ->  (?_1  ->  ?_2)|` between
     the *function-space
domains*, obtained from Theorem 3.10 (`funSpaceEquiv`) and Theorem 3.12 (`curryEquiv`).
     -/
def curryIso (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) (V_2 :
    NeighborhoodSystem ?) :
    (funSpace (prod V_0 V_1) V_2).Element ?o (funSpace V_0 (funSpace V_1 V_2)).Element
     :=
  (funSpaceEquiv (prod V_0 V_1) V_2).trans
    ((curryEquiv V_0 V_1 V_2).trans (funSpaceEquiv V_0 (funSpace V_1 V_2)).symm)

/-- **Table 5.5 (Scott 1981, PRG-19).** `curry = fun g fun x fun y.g(x,y)` as an
    approximable map
`(?_0  x  ?_1  ->  ?_2)  ->  (?_0  ->  (?_1  ->  ?_2))`. -/
def curryC (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) (V_2 :
    NeighborhoodSystem ?) :
    ApproximableMap (funSpace (prod V_0 V_1) V_2) (funSpace V_0 (funSpace V_1 V_2)) :=
  ofIso (curryIso V_0 V_1 V_2)

/-- **Table 5.5 (Scott 1981, PRG-19).** `eval = fun f,x.f(x)`, evaluation `(?_0  ->  ?
    _1)  x  ?_0  ->  ?_1`
(this is Theorem 3.11's `evalMap`). -/
def evalC (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) :
    ApproximableMap (prod (funSpace V_0 V_1) V_0) V_1 := evalMap V_0 V_1

/-- **Table 5.5 (Scott 1981, PRG-19).** `fix = fun f.!x.f(x)`, the least fixed-point
    operator
`(?  ->  ?)  ->  ?` (Theorem 4.2's `fixMap`). -/
def fixC (V : NeighborhoodSystem a) : ApproximableMap (funSpace V V) V := fixMap V

/-- The uncurried `(f, g), x  |->  <f(x), g(x)>` over `((?_2 -> ?_0)  x  (?_2 -> ?_1))
     x  ?_2  ->  ?_0  x  ?_1`. -/
def funpairMapTbl (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b)
    (V_2 : NeighborhoodSystem ?) :
    ApproximableMap (prod (prod (funSpace V_2 V_0) (funSpace V_2 V_1)) V_2) (prod V_0
    V_1) :=
  paired
    ((evalMap V_2 V_0).comp
      (paired
        ((proj_0 (funSpace V_2 V_0) (funSpace V_2 V_1)).comp
          (proj_0 (prod (funSpace V_2 V_0) (funSpace V_2 V_1)) V_2))
        (proj_1 (prod (funSpace V_2 V_0) (funSpace V_2 V_1)) V_2)))
    ((evalMap V_2 V_1).comp
      (paired
        ((proj_1 (funSpace V_2 V_0) (funSpace V_2 V_1)).comp
          (proj_0 (prod (funSpace V_2 V_0) (funSpace V_2 V_1)) V_2))
        (proj_1 (prod (funSpace V_2 V_0) (funSpace V_2 V_1)) V_2)))

/-- **Table 5.5 (Scott 1981, PRG-19).** `funpair = fun f fun g fun x.<f(x), g(x)>`, the
```

```
     curried operation
`(?_2 -> ?_0)  ->  ((?_2 -> ?_1)  ->  (?_2  ->  ?_0  x  ?_1))`. -/
def funpairC (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) (V_2 :
    NeighborhoodSystem ?) :
    ApproximableMap (funSpace V_2 V_0)
      (funSpace (funSpace V_2 V_1) (funSpace V_2 (prod V_0 V_1))) :=
  curry (curry (funpairMapTbl V_0 V_1 V_2))

end

section
variable {a b ? : Type*}
  {V : NeighborhoodSystem a} {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}

theorem compC_apply (phi : (funSpace V_1 V_2).Element) (psi : (funSpace V_0 V_1).
    Element) (x : V_0.Element) :
    (toApproxMap ((compC V_0 V_1 V_2).toElementMap (pair phi psi))).toElementMap x
      = (toApproxMap phi).toElementMap ((toApproxMap psi).toElementMap x) := by
  rw [compC, toElementMap_curry_apply, compMapTbl]
  simp only [toElementMap_comp, toElementMap_paired, toElementMap_proj_0,
    toElementMap_proj_1,
    fst_pair, snd_pair, evalMap_apply]

/-- `comp(g, f) = g  comp  f` (Scott's infix `g  comp  f`). -/
theorem compC_eq_comp (phi : (funSpace V_1 V_2).Element) (psi : (funSpace V_0 V_1).
    Element) :
    toApproxMap ((compC V_0 V_1 V_2).toElementMap (pair phi psi)) = (toApproxMap phi).
    comp (toApproxMap psi) := by
  apply ext_of_toElementMap
  intro x
  rw [toElementMap_comp, compC_apply]

theorem constC_apply (k : V_1.Element) (x : V_0.Element) :
    (toApproxMap ((constC V_0 V_1).toElementMap k)).toElementMap x = k := by
  rw [constC, toElementMap_curry_apply, toElementMap_proj_0, fst_pair]

/-- `const(k)` is the constant map `constMap` of Lemma 3.6. -/
theorem constC_eq_constMap (k : V_1.Element) :
    toApproxMap ((constC V_0 V_1).toElementMap k) = constMap V_0 k := by
  apply ext_of_toElementMap
  intro x
  rw [constC_apply, toElementMap_constMap]

/-- `curry(g)` is the curried map of Theorem 3.12. -/
theorem curryC_toApproxMap (phi : (funSpace (prod V_0 V_1) V_2).Element) :
    toApproxMap ((curryC V_0 V_1 V_2).toElementMap phi) = curry (toApproxMap phi) :=
    by
  rw [curryC, toElementMap_ofIso]
  show toApproxMap (toFilter (curry (toApproxMap phi))) = curry (toApproxMap phi)
  have he := (funSpaceEquiv V_0 (funSpace V_1 V_2)).apply_symm_apply (curry (
    toApproxMap phi))
  rwa [funSpaceEquiv_apply, funSpaceEquiv_symm_apply] at he

theorem curryC_apply (phi : (funSpace (prod V_0 V_1) V_2).Element) (x : V_0.Element) (
    y : V_1.Element) :
    (toApproxMap ((toApproxMap ((curryC V_0 V_1 V_2).toElementMap phi)).toElementMap x
```

```
    )).toElementMap y
      = (toApproxMap phi).toElementMap (pair x y) := by
  rw [curryC_toApproxMap, toElementMap_curry_apply]

theorem evalC_apply (phi : (funSpace V_0 V_1).Element) (x : V_0.Element) :
    (evalC V_0 V_1).toElementMap (pair phi x) = (toApproxMap phi).toElementMap x := by
  rw [evalC, evalMap_apply]

theorem fixC_apply (phi : (funSpace V V).Element) :
    (fixC V).toElementMap phi = (toApproxMap phi).fixElement := by
  rw [fixC, fixMap_toElementMap]

/-- `fix(f) = f(fix(f))`. -/
theorem fixC_fixed (phi : (funSpace V V).Element) :
    (toApproxMap phi).toElementMap ((fixC V).toElementMap phi) = (fixC V).toElementMap
     phi := by
  rw [fixC]; exact fixMap_fixed V phi

theorem funpairC_apply (phi : (funSpace V_2 V_0).Element) (psi : (funSpace V_2 V_1).
    Element)
    (x : V_2.Element) :
    (toApproxMap ((toApproxMap ((funpairC V_0 V_1 V_2).toElementMap phi)).toElementMap
     psi)).toElementMap x
      = pair ((toApproxMap phi).toElementMap x) ((toApproxMap psi).toElementMap x) :=
    by
  rw [funpairC, toElementMap_curry_apply, toElementMap_curry_apply, funpairMapTbl]
  simp only [toElementMap_paired, toElementMap_comp, toElementMap_proj_0,
    toElementMap_proj_1,
    fst_pair, snd_pair, evalMap_apply]

/-- `funpair(f)(g)` is the paired map `<f, g>` of Definition 3.3. -/
theorem funpairC_eq_paired (phi : (funSpace V_2 V_0).Element) (psi : (funSpace V_2 V_1
    ).Element) :
    toApproxMap ((toApproxMap ((funpairC V_0 V_1 V_2).toElementMap phi)).toElementMap
    psi)
      = paired (toApproxMap phi) (toApproxMap psi) := by
  apply ext_of_toElementMap
  intro x
  rw [funpairC_apply, toElementMap_paired]

theorem pairC_apply (x : V_0.Element) (y : V_1.Element) :
    (toApproxMap ((pairC V_0 V_1).toElementMap x)).toElementMap y = pair x y := by
  rw [pairC, toElementMap_curry_apply, toElementMap_idMap]

end

section
variable {a b : Type*} {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}

namespace ApproximableMap

/-- The approximable map `y  |->  !x. tau(x, y)`: `fix  comp  curry(tau  comp  swap)`.
    -/
def secFixX (tau : ApproximableMap (prod V_0 V_1) V_0) : ApproximableMap V_1 V_0 :=
  (fixMap V_0).comp (curry (tau.comp (swapC V_1 V_0)))
```

```
/-- The outer operator `y  |->  sigma(!x. tau(x, y), y)`. -/
def outerOp (tau : ApproximableMap (prod V_0 V_1) V_0) (sigma : ApproximableMap (prod
    V_0 V_1) V_1) :
    ApproximableMap V_1 V_1 :=
  sigma.comp (paired (secFixX tau) (idMap V_1))

/-- The section `x  |->  tau(x, y)` of `tau : ?_0  x  ?_1  ->  ?_0` at a fixed `y`, as
    an approximable endomap of
`?_0` (the curried `tau  comp  swap` applied to `y`). -/
def sectionX (tau : ApproximableMap (prod V_0 V_1) V_0) (y : V_1.Element) :
    ApproximableMap V_0 V_0 :=
  toApproxMap ((curry (tau.comp (swapC V_1 V_0))).toElementMap y)

theorem outerOp_apply (tau : ApproximableMap (prod V_0 V_1) V_0) (sigma :
    ApproximableMap (prod V_0 V_1) V_1)
    (y : V_1.Element) :
    (outerOp tau sigma).toElementMap y = sigma.toElementMap (pair ((secFixX tau).
    toElementMap y) y) := by
  rw [outerOp, toElementMap_comp, toElementMap_paired, toElementMap_idMap]

theorem secFixX_apply (tau : ApproximableMap (prod V_0 V_1) V_0) (y : V_1.Element) :
    (secFixX tau).toElementMap y = (sectionX tau y).fixElement := by
  rw [secFixX, toElementMap_comp, fixMap_toElementMap, sectionX]

theorem sectionX_apply (tau : ApproximableMap (prod V_0 V_1) V_0) (y : V_1.Element) (x
    : V_0.Element) :
    (sectionX tau y).toElementMap x = tau.toElementMap (pair x y) := by
  rw [sectionX, toElementMap_curry_apply, toElementMap_comp, swapC_apply]

/-- `!x. tau(x, y)` is a fixed point of the section: `tau(!x.tau(x,y), y) = !x.tau(x,y)
    `. -/
theorem secFixX_isFixed (tau : ApproximableMap (prod V_0 V_1) V_0) (y : V_1.Element) :
    tau.toElementMap (pair ((secFixX tau).toElementMap y) y) = (secFixX tau).
    toElementMap y := by
  rw [secFixX_apply,  <-  sectionX_apply, toElementMap_fixElement]

end ApproximableMap
end
end DomainTheory.NeighbourhoodSystems
end
```

3.6.7 Theorem 5.6 — partial recursive functions

THEOREM 5.6. For every partial recursive function $h : \mathbf{N} \to \mathbf{N}$, there is a λ-term τ of type $(\mathbf{N} \to \mathbf{N})$ such that the only constants occurring in τ are

$$\text{cond},\ \text{succ},\ \text{pred},\ \text{zero},\ 0$$

and where if $h(n) = m$, then

$$\tau(n) = m$$

is true; and if $h(n)$ is undefined, then

$$\tau(n) = \perp$$

is true. The equation $\tau(\perp) = \perp$ is also true.

Proof : We have only formulated the theorem for functions of one variable — but to give the proof, it is convenient to pass through functions of any number of (integer) variables. We shall also have to recall the precise definition of the notion of partial recursive function.

It is also convenient to work with (*very*)*strict* functions

$$f : \mathbf{N}^k \to \mathbf{N}.$$

These are functions such that if $n_0, \ldots, n_{k-1} \in \mathbf{N}$ and $n_i = \perp$ for at least one $i < k$, then

$$f(n_0, \ldots, n_{k-1}) = \perp.$$

It is easy to check that compositions of strict functions are strict. It is also easy to see that any *partial* function

$$g : \mathbf{N}^k \to \mathbf{N}$$

extends to a strict (*approximable*) function

$$\bar{g} : \mathbf{N}^k \to \mathbf{N},$$

which takes the same values as g as long as g is defined; otherwise $\bar{g}$ takes the value $\perp$. What we want to show for *partial recursive* g is that the corresponding $\bar{g}$ is defined by a λ - expression.

In the first place we have to check that *primitive recursive* functions have λ - definitions in this sense. We recall that *primitive recursive* functions are generated from certain elementary *starting* functions by multi-variate composition and the scheme of primitive recrusion. The *starting* functions are the constant function with value zero and the "identity" or "projection" functions. For example, $g(n_0, n_1, n_2) = n_1$ for all $n_0, n_1, n_2 \in \mathbf{N}$ is one of the *starting* functions. Now we cannot just use the λ-term

$$\lambda x_0, x_1, x_2.x_1$$

to represent $\bar{g}$, because the function so defined is not strict. But any function in $\mathbf{N}^k \to \mathbf{N}$ can be cut down to a strict function by a simple device. Consider

$$\lambda x.\ \mathrm{cond}(\mathrm{zero}(x), x, x)$$

with x of type $\mathbf{N}$. This is the strict version of the identity function of one argument. The strict projection function of two arguments can be defined by

$$\lambda x_0, x_1.\ \mathrm{cond}(\mathrm{zero}(x_1), x_0, x_0).$$

The one of three arguments by:

$$\lambda x_0, x_1, x_2.\ \mathrm{cond}(\mathrm{zero}(x_0), \mathrm{cond}(\mathrm{zero}(x_2), x_1, x_1), \mathrm{cond}(\mathrm{zero}(x_2), x_1, x_1)).$$

This is not done very elegantly, and the reader can find for himself a general solution based on perhaps a better notation for the required compositions of functions.

As we remarked, strict functions are closed under substitution, and any substitution of a batch of functions into another function can be given by a λ-term, if the various functions can themselves be so defined. It only remains to λ-define functions obtained by primitive recursion. Thus, suppose, for the sake of argument, that

$$\bar{f} : \mathbf{N} \to \mathbf{N} \qquad \text{and} \qquad \bar{g} : \mathbf{N}^3 \to \mathbf{N}$$

are given as total functions with $\bar{f}$ and $\bar{g}$ being λ-definable. From them, we obtain by primitive recursion $\bar{h} : \mathbf{N}^2 \to \mathbf{N}$ where

$$\bar{h}(0, m) = \bar{f}(m),$$

$$\bar{h}(n+1, m) = \bar{g}(n, m, \bar{h}(n, m))$$

for all $n, m \in \mathbf{N}$. The λ-term defining $\bar{h}$ is

$$!k\ \lambda x, y.\ \mathrm{cond}(\mathrm{zero}(x), \bar{f}(y), \bar{g}(\mathrm{pred}(x), y, k(\mathrm{pred}(x), y))).$$

Here we have had to use the fixed-point operator on a variable k of type $(\mathbf{N}^2 \to \mathbf{N})$. The variables x, y are of type $\mathbf{N}$ and the cond-construction puts the two traditional equations into two clauses of one expression. It is easy to see that the fixed-point function *is* strict and is nothing more than $\bar{h}$.

That completes the representation of *primitive recursive* functions. To obtain the *partial recursive* functions, the idea is to use the so-called μ-scheme (least number operator) and, further, to close up under substitution. We need only treat the μ-scheme. Suppose, by way of example, $f(n, m)$ is given as a

primitive recursive function. We then define h (generally, a partial function) by

$$h(m) = \text{the least } n \text{ where } f(n, m) = 0.$$

This is often written

$$h(m) = \mu n.\ f(n, m) = 0.$$

Supposing, as we may, $\bar{f}$ is λ-definable, we introduce first

$$\bar{g} = !g\ \lambda x, y.\ \mathrm{cond}(\mathrm{zero}(\bar{f}(x, y)), x, g(\mathrm{succ}(x), y)).$$

Then $\bar{h} = \lambda y.\ \bar{g}(0, y)$. This is easily seen to be strict. Also easy to see is that if $h(m)$ is defined, then $\bar{g}(0, m) = h(m)$. But, if $h(m)$ is not defined, it takes some argument to make sure that the least fixed-point construction forces $\bar{g}(0, m) = \bot$. However, the argument is not very difficult. $\square$

What is *not* said in 5.6 is that every λ-term defines a partial recursive function. This is true (with suitable control over the constants and types in the expression), but the proof requires a full analysis of computability properties of domain constructions. This is the topic of Lecture VII.

It should be remarked that the types of variables needed for the proof of 5.6 never get very high. In fact, types like N, N^k, and $(N^k \to N)$ were the only ones needed (with perhaps T thrown in also).

Recursion on N was the topic of 5.6; further examples of recursion on other domains are included in the exercises.

Theorem_5_6.lean

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Example_4_3
public import Mathlib.Computability.PartrecBasis
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.Exercise_5_11
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_8
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_13
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Lemma_3_6

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.Theorem56
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap
open DomainTheory.NeighbourhoodSystems.Example43 (N natElem zeroElt succMap predMap
    zeroMap
  succMap_natElem succMap_bot predMap_natElem_succ predMap_natElem_zero predMap_bot
  zeroMap_natElem_zero zeroMap_natElem_succ zeroMap_bot constLiftN constLiftN_natElem)
open DomainTheory.NeighbourhoodSystems.Exercise326 (cond)

section
/-- The unknown function `k` (`= proj_0`). -/
def kSel : ApproximableMap (prod (funSpace (prod N N) N) (prod N N)) (funSpace (prod N
     N) N) :=
  proj_0 (funSpace (prod N N) N) (prod N N)
end

section
/-- The argument pair `<x, y>` (`= proj_1`). -/
def qSel : ApproximableMap (prod (funSpace (prod N N) N) (prod N N)) (prod N N) :=
  proj_1 (funSpace (prod N N) N) (prod N N)
end

section
/-- The first argument `x`. -/
def argX : ApproximableMap (prod (funSpace (prod N N) N) (prod N N)) N := (proj_0 N N)
    .comp qSel
end

section
/-- The second argument `y`. -/
def argY : ApproximableMap (prod (funSpace (prod N N) N) (prod N N)) N := (proj_1 N N)
    .comp qSel
end

section
section Mu

variable (f : ApproximableMap (prod N N) N)

/-- The body `M(g, <x, y>) = cond(zero(f(x, y)), x, g(succ x, y))`. -/
def muBody : ApproximableMap (prod (funSpace (prod N N) N) (prod N N)) N :=
  (cond N).comp
```

```
    (paired (zeroMap.comp (f.comp qSel))
      (paired argX
        ((evalMap (prod N N) N).comp
          (paired kSel (paired (succMap.comp argX) argY)))))

end Mu
end

section
section Mu

variable (f : ApproximableMap (prod N N) N)

/-- The ?-operator `R(g) = fun x,y. cond(zero(f(x, y)), x, g(succ x, y))`. -/
def muOp : ApproximableMap (funSpace (prod N N) N) (funSpace (prod N N) N) :=
  curry (muBody f)

end Mu
end

section
section Mu

variable (f : ApproximableMap (prod N N) N)

/-- `muRec f : N  x  N  ->  N`, the binary search `g`, least fixed point of `muOp`. -/
def muRec : ApproximableMap (prod N N) N := toApproxMap (muOp f).fixElement

end Mu
end

section
section Mu

variable (f : ApproximableMap (prod N N) N)

/-- The minimization `? : N  ->  N`, `h(m) = g(0, m)` (the strict `0`-seeded search).
    -/
def muMap : ApproximableMap N N := muRec f |>.comp (paired (constLiftN N (fun _ =>
    zeroElt)) (idMap N))

end Mu
end

section
section PrimRec

variable (f : ApproximableMap N N) (g : ApproximableMap (prod N (prod N N)) N)

/-- The two-argument body `M(k, <x, y>) = cond(zero(x), f(y), g(pred x, y, k(pred x, y)
    ))`. -/
def primBody : ApproximableMap (prod (funSpace (prod N N) N) (prod N N)) N :=
  (cond N).comp
    (paired (zeroMap.comp argX)
      (paired (f.comp argY)
        (g.comp
```

```
          (paired (predMap.comp argX)
            (paired argY
              ((evalMap (prod N N) N).comp
                (paired kSel (paired (predMap.comp argX) argY)))))))))

end PrimRec
end

section
section PrimRec

variable (f : ApproximableMap N N) (g : ApproximableMap (prod N (prod N N)) N)

/-- The primitive-recursion operator `R(k) = fun x,y. cond(zero x, f y, g(pred x, y, k(
    pred x, y)))`. -/
def primOp : ApproximableMap (funSpace (prod N N) N) (funSpace (prod N N) N) :=
  curry (primBody f g)

end PrimRec
end

section
section PrimRec

variable (f : ApproximableMap N N) (g : ApproximableMap (prod N (prod N N)) N)

/-- `primRec f g : N  x  N  ->  N`, the least fixed point of `primOp`. -/
def primRec : ApproximableMap (prod N N) N := toApproxMap (primOp f g).fixElement

end PrimRec
end

section
/-- Scott's **strict identity** `fun x. cond(zero(x), x, x)`: the identity on totals, `
    False` on `False`. -/
def strictId : ApproximableMap N N :=
  (cond N).comp (paired zeroMap (paired (idMap N) (idMap N)))
end

section
/-- Scott's **strict first projection** `fun x_0, x_1. cond(zero(x_1), x_0, x_0)` on `N
      x  N`. -/
def strictProj_0 : ApproximableMap (prod N N) N :=
  (cond N).comp
    (paired (zeroMap.comp (proj_1 N N))
      (paired (proj_0 N N) (proj_0 N N)))
end

section
/-- The truth-domain bottom of `T` agrees with `Example23.botElt` (both are the least
    element).
Needed because `zeroMap_bot` lands in `T.bot` while `cond_bot` is phrased with `
    Example23.botElt`,
and `bot` is not reducible. -/
theorem T_bot_eq : (Example43.T).bot = Example23.botElt :=
  le_antisymm ((Example43.T).bot_le _) (Example23.botElt_le _)
```

```
end

section
section Mu

variable (f : ApproximableMap (prod N N) N)

theorem muBody_apply (k : (funSpace (prod N N) N).Element) (x y : N.Element) :
    (muBody f).toElementMap (pair k (pair x y)) =
      (cond N).toElementMap
        (pair (zeroMap.toElementMap (f.toElementMap (pair x y)))
          (pair x ((toApproxMap k).toElementMap (pair (succMap.toElementMap x) y))))
    := by
  rw [muBody, toElementMap_comp]
  simp only [argX, argY, kSel, qSel, toElementMap_paired, toElementMap_comp,
    toElementMap_proj_0,
    toElementMap_proj_1, fst_pair, snd_pair, evalMap_apply]

end Mu
end

section
section Mu

variable (f : ApproximableMap (prod N N) N)

/-- `?(m) = g(0, m)`. -/
theorem muMap_natElem (m : Nat) :
    (muMap f).toElementMap (natElem m) = (muRec f).toElementMap (pair zeroElt (natElem
    m)) := by
  rw [muMap, toElementMap_comp, toElementMap_paired, toElementMap_idMap,
    constLiftN_natElem]

end Mu
end

section
section Mu

variable (f : ApproximableMap (prod N N) N)

/-- The defining recursion of `muRec`. -/
theorem muRec_rec (x y : N.Element) :
    (muRec f).toElementMap (pair x y) =
      (cond N).toElementMap
        (pair (zeroMap.toElementMap (f.toElementMap (pair x y)))
          (pair x ((muRec f).toElementMap (pair (succMap.toElementMap x) y)))) := by
  have hfix : (muOp f).toElementMap (muOp f).fixElement = (muOp f).fixElement :=
    toElementMap_fixElement (muOp f)
  have hval := toElementMap_curry_apply (muBody f) (muOp f).fixElement (pair x y)
  rw [ <-  muOp, hfix] at hval
  rw [muRec, hval, muBody_apply]

end Mu
end
```

```
section
section Mu

variable (f : ApproximableMap (prod N N) N)

/-- **? is strict in the test.** If `f(n, m) = False`, the search diverges. -/
theorem muRec_bot {n m : Nat}
    (h : f.toElementMap (pair (natElem n) (natElem m)) = N.bot) :
    (muRec f).toElementMap (pair (natElem n) (natElem m)) = N.bot := by
  rw [muRec_rec, h, zeroMap_bot, T_bot_eq, Exercise326.cond_bot]

end Mu
end

section
section Mu

variable (f : ApproximableMap (prod N N) N)

/-- **?, found.** If `f(n, m) = 0`, the search stops and returns `n`. -/
theorem muRec_found {n m : Nat}
    (h : f.toElementMap (pair (natElem n) (natElem m)) = natElem 0) :
    (muRec f).toElementMap (pair (natElem n) (natElem m)) = natElem n := by
  rw [muRec_rec, h, zeroMap_natElem_zero, Exercise326.cond_true]

end Mu
end

section
section Mu

variable (f : ApproximableMap (prod N N) N)

/-- **?, step.** If `f(n, m)` is a positive total `(j+1)^`, the search advances to `n
    +1`. -/
theorem muRec_step {n m j : Nat}
    (h : f.toElementMap (pair (natElem n) (natElem m)) = natElem (j + 1)) :
    (muRec f).toElementMap (pair (natElem n) (natElem m)) =
      (muRec f).toElementMap (pair (natElem (n + 1)) (natElem m)) := by
  rw [muRec_rec, h, zeroMap_natElem_succ, Exercise326.cond_false, succMap_natElem]

end Mu
end

section
section Mu

variable (f : ApproximableMap (prod N N) N)

/-- The search advances over a run of positive totals: if `f((n+i)^, m)` is a positive
    total for
all `i < k`, then `g(n, m) = g((n+k)^, m)`. -/
theorem muRec_climb (m : Nat) :
    forall  (k n : Nat), (forall  i < k, exists  j, f.toElementMap (pair (natElem (n +
     i)) (natElem m)) = natElem (j + 1))  ->
      (muRec f).toElementMap (pair (natElem n) (natElem m)) =
```

```
        (muRec f).toElementMap (pair (natElem (n + k)) (natElem m)) := by
  intro k
  induction k with
  | zero => intro n _; rw [Nat.add_zero]
  | succ k ih =>
    intro n h
    obtain <j, hj> := h 0 (Nat.succ_pos k)
    rw [Nat.add_zero] at hj
    have key := ih (n + 1) (fun i hi => by
      obtain <j', hj'> := h (i + 1) (by omega)
      exact <j', by rw [show n + 1 + i = n + (i + 1) by omega]; exact hj'>)
    rw [muRec_step f hj, key, show n + 1 + k = n + (k + 1) by omega]

end Mu
end

section
section Mu

variable (f : ApproximableMap (prod N N) N)

/-- **Capstone (?-scheme correctness).** If `n_0` is the *least* zero of `f(*, m)` --
    that is,
`f(n_0, m) = 0` and `f(i, m)` is a positive total for every `i < n_0` -- then `?(m) =
    n_0`. -/
theorem muMap_eq_least {m n_0 : Nat}
    (hzero : f.toElementMap (pair (natElem n_0) (natElem m)) = natElem 0)
    (hpos : forall  i < n_0, exists  j, f.toElementMap (pair (natElem i) (natElem m))
    = natElem (j + 1)) :
    (muMap f).toElementMap (natElem m) = natElem n_0 := by
  rw [muMap_natElem]
  have hclimb := muRec_climb f m n_0 0 (by simpa using hpos)
  rw [show (zeroElt : N.Element) = natElem 0 from rfl, hclimb, Nat.zero_add]
  exact muRec_found f hzero

end Mu
end

section
section PrimRec

variable (f : ApproximableMap N N) (g : ApproximableMap (prod N (prod N N)) N)

theorem primBody_apply (k : (funSpace (prod N N) N).Element) (x y : N.Element) :
    (primBody f g).toElementMap (pair k (pair x y)) =
      (cond N).toElementMap
        (pair (zeroMap.toElementMap x)
          (pair (f.toElementMap y)
            (g.toElementMap
              (pair (predMap.toElementMap x)
                (pair y ((toApproxMap k).toElementMap
                  (pair (predMap.toElementMap x) y))))))) := by
  rw [primBody, toElementMap_comp]
  simp only [argX, argY, kSel, qSel, toElementMap_paired, toElementMap_comp,
    toElementMap_proj_0,
    toElementMap_proj_1, fst_pair, snd_pair, evalMap_apply]
```

```
end PrimRec
end

section
section PrimRec

variable (f : ApproximableMap N N) (g : ApproximableMap (prod N (prod N N)) N)

/-- The defining recursion of `primRec`. -/
theorem primRec_rec (x y : N.Element) :
    (primRec f g).toElementMap (pair x y) =
      (cond N).toElementMap
        (pair (zeroMap.toElementMap x)
          (pair (f.toElementMap y)
            (g.toElementMap
              (pair (predMap.toElementMap x)
                (pair y ((primRec f g).toElementMap
                  (pair (predMap.toElementMap x) y)))))))) := by
  have hfix : (primOp f g).toElementMap (primOp f g).fixElement = (primOp f g).
    fixElement :=
    toElementMap_fixElement (primOp f g)
  have hval := toElementMap_curry_apply (primBody f g) (primOp f g).fixElement (pair x
     y)
  rw [ <-  primOp, hfix] at hval
  rw [primRec, hval, primBody_apply]

end PrimRec
end

section
section PrimRec

variable (f : ApproximableMap N N) (g : ApproximableMap (prod N (prod N N)) N)

/-- **Primitive recursion is strict in the recursion argument.** `h(False, m) = False`.
     -/
theorem primRec_bot (m : Nat) :
    (primRec f g).toElementMap (pair N.bot (natElem m)) = N.bot := by
  rw [primRec_rec, zeroMap_bot, T_bot_eq, Exercise326.cond_bot]

end PrimRec
end

section
section PrimRec

variable (f : ApproximableMap N N) (g : ApproximableMap (prod N (prod N N)) N)

/-- **Primitive recursion, step case.** `h((n+1)^, m) = g(n, m, h(n, m))`. -/
theorem primRec_succ (n m : Nat) :
    (primRec f g).toElementMap (pair (natElem (n + 1)) (natElem m)) =
      g.toElementMap
        (pair (natElem n)
          (pair (natElem m) ((primRec f g).toElementMap (pair (natElem n) (natElem m))
    ))) := by
```

```
  rw [primRec_rec, zeroMap_natElem_succ, Exercise326.cond_false, predMap_natElem_succ]

end PrimRec
end

section
section PrimRec

variable (f : ApproximableMap N N) (g : ApproximableMap (prod N (prod N N)) N)

/-- **Primitive recursion, base case.** `h(0, m) = f(m)`. -/
theorem primRec_zero (m : Nat) :
    (primRec f g).toElementMap (pair (natElem 0) (natElem m)) = f.toElementMap (
    natElem m) := by
  rw [primRec_rec, zeroMap_natElem_zero, Exercise326.cond_true]

end PrimRec
end

section
theorem strictId_bot : strictId.toElementMap N.bot = N.bot := by
  rw [strictId, toElementMap_comp]
  simp only [toElementMap_paired, toElementMap_idMap]
  rw [zeroMap_bot, T_bot_eq, Exercise326.cond_bot]
end

section
theorem strictId_natElem (n : Nat) : strictId.toElementMap (natElem n) = natElem n :=
    by
  rw [strictId, toElementMap_comp]
  simp only [toElementMap_paired, toElementMap_idMap]
  cases n with
  | zero => rw [zeroMap_natElem_zero, Exercise326.cond_true]
  | succ k => rw [zeroMap_natElem_succ, Exercise326.cond_false]
end

section


theorem strictProj_0_apply (x y : N.Element) :
    strictProj_0.toElementMap (pair x y) =
      (cond N).toElementMap (pair (zeroMap.toElementMap y) (pair x x)) := by
  rw [strictProj_0, toElementMap_comp]
  simp only [toElementMap_paired, toElementMap_comp, toElementMap_proj_0,
    toElementMap_proj_1,
    fst_pair, snd_pair]
end

section
/-- Strict in the first argument: `proj_0(False, m) = False` (the output *is* `x_0 =
    False`). -/
theorem strictProj_0_bot_left (m : Nat) :
    strictProj_0.toElementMap (pair N.bot (natElem m)) = N.bot := by
  rw [strictProj_0_apply]
  cases m with
  | zero => rw [zeroMap_natElem_zero, Exercise326.cond_true]
  | succ k => rw [zeroMap_natElem_succ, Exercise326.cond_false]
```

```
end

section
/-- Strict in the second argument: `proj_0(n, False) = False`. -/
theorem strictProj_0_bot_right (n : Nat) :
    strictProj_0.toElementMap (pair (natElem n) N.bot) = N.bot := by
  rw [strictProj_0_apply, zeroMap_bot, T_bot_eq, Exercise326.cond_bot]
end

section
/-- `proj_0(n, m) = n`. -/
theorem strictProj_0_natElem (n m : Nat) :
    strictProj_0.toElementMap (pair (natElem n) (natElem m)) = natElem n := by
  rw [strictProj_0_apply]
  cases m with
  | zero => rw [zeroMap_natElem_zero, Exercise326.cond_true]
  | succ k => rw [zeroMap_natElem_succ, Exercise326.cond_false]
end

end DomainTheory.NeighbourhoodSystems.Theorem56

section

namespace DomainTheory.NeighbourhoodSystems.Theorem56Full
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap
open DomainTheory.NeighbourhoodSystems.Example43 (N natElem zeroElt succMap predMap
    zeroMap
  succMap_natElem succMap_bot predMap_natElem_succ predMap_natElem_zero predMap_bot
  zeroMap_natElem_zero zeroMap_natElem_succ zeroMap_bot constLiftN)
open DomainTheory.NeighbourhoodSystems.Exercise326 (cond)
open DomainTheory.NeighbourhoodSystems.Exercise511 (head tail push)

section
/-- `none  |->  False`, `some n  |->  n`. -/
def optElem : Option Nat  ->  N.Element
  | none => N.bot
  | some n => natElem n
end

section
/-- The universal argument domain `? = N^ inf ` (Exercise 3.16). All `k`-ary number
    functions are
realised as approximable maps `?  ->  N` depending only on coordinates `0..k-1`. -/
abbrev ? : NeighborhoodSystem (Nat  x  Nat) := iterSys N
end

section
/-- A `?`-element is *arg-like* if every coordinate is total or `False` (true of all `
    argElem`). -/
def ArgLike (z : ?.Element) : Prop := forall  j, component z j = N.bot  \/  exists  k,
     component z j = natElem k
end

section
/-- The argument element of `?` whose first `n` coordinates are `optElem (a i)` and the
     rest `False`. -/
```

```
def argElem {n : Nat} (a : Fin n  ->  Option Nat) : ?.Element :=
  ofSeq (fun i => if h : i < n then optElem (a <i, h>) else N.bot)
end

section
/-- The argument element of `?` carrying a `List.Vector Nat n` in its first `n`
    coordinates. -/
def vecElem {n : Nat} (v : List.Vector Nat n) : ?.Element :=
  argElem (fun i => some (v.get i))
end

section
/-- `phi : ?  ->  N` realises the `n`-ary partial function `f` (a *very strict* fun -
    definition):
`phi` returns `f`'s value on total arguments, `False` where `f` is undefined, and `False
    ` on *any* arg-like
input with a `False` among its first `n` coordinates. -/
structure LamDef {n : Nat} (phi : ApproximableMap ? N) (f : List.Vector Nat n  -> .
    Nat) : Prop where
  defined : forall  (v : List.Vector Nat n) (c : Nat), c  in  f v  ->  phi.
    toElementMap (vecElem v) = natElem c
  undef : forall  (v : List.Vector Nat n), not  (f v).Dom  ->  phi.toElementMap (
    vecElem v) = N.bot
  strict : forall  (z : ?.Element), ArgLike z  ->  forall  i, i < n  ->  component z i
     = N.bot  ->
    phi.toElementMap z = N.bot
end

section
@[simp] theorem component_argElem {n : Nat} (a : Fin n  ->  Option Nat) (i : Nat) :
    component (argElem a) i = if h : i < n then optElem (a <i, h>) else N.bot := by
  rw [argElem, component_ofSeq]
end

section
@[simp] theorem component_vecElem {n : Nat} (v : List.Vector Nat n) (i : Nat) :
    component (vecElem v) i = if h : i < n then natElem (v.get <i, h>) else N.bot :=
    by
  rw [vecElem, component_argElem]; split <;> rfl
end

section
section Rfind

variable (phif : ApproximableMap ? N)

/-- `evalAt z` evaluates a function-space element at the fixed argument `z`. -/
def evalAt (z : ?.Element) : ApproximableMap (funSpace ? N) N :=
  (evalMap ? N).comp (paired (idMap (funSpace ? N)) (constMap (funSpace ? N) z))

end Rfind
end

section
section Rfind
```

```
variable (phif : ApproximableMap ? N)

/-- The search body `S(k, z) = cond(zero(f z), head z, k(push(succ(head z), tail z)))`.
    -/
def findBody : ApproximableMap (prod (funSpace ? N) ?) N :=
  (cond N).comp
    (paired (zeroMap.comp (phif.comp (proj_1 (funSpace ? N) ?)))
      (paired ((head N).comp (proj_1 (funSpace ? N) ?))
        ((evalMap ? N).comp
          (paired (proj_0 (funSpace ? N) ?)
            ((push N).comp
              (paired (succMap.comp ((head N).comp (proj_1 (funSpace ? N) ?)))
                ((tail N).comp (proj_1 (funSpace ? N) ?)))))))))

end Rfind
end

section
section Rfind

variable (phif : ApproximableMap ? N)

/-- The search operator `S(k) = fun z. ...`, whose least fixed point is the running
    search. -/
def findOp : ApproximableMap (funSpace ? N) (funSpace ? N) := curry (findBody phif)

end Rfind
end

section
/-- Test coordinate `j`: `cond(zero(z?), phi z, phi z)`. -/
def guard1 (j : Nat) (phi : ApproximableMap ? N) : ApproximableMap ? N :=
  (cond N).comp (paired (zeroMap.comp (projN N j)) (paired phi phi))
end

section
/-- Inject `N  ->  ?` as the stack whose only nonzero coordinate is the head. -/
def oneArg : ApproximableMap N ? := (push N).comp (paired (idMap N) (constMap N ?.bot)
    )
end

section
section Prec

variable (phif phig : ApproximableMap ? N)

/-- The primitive-recursion body. -/
def precBody : ApproximableMap (prod (funSpace ? N) ?) N :=
  (cond N).comp
    (paired (zeroMap.comp ((head N).comp (proj_1 (funSpace ? N) ?)))
      (paired (phif.comp ((tail N).comp (proj_1 (funSpace ? N) ?)))
        (phig.comp
          ((push N).comp
            (paired (predMap.comp ((head N).comp (proj_1 (funSpace ? N) ?)))
              ((push N).comp
                (paired
```

```
                  ((evalMap ? N).comp
                    (paired (proj_0 (funSpace ? N) ?)
                      ((push N).comp
                        (paired (predMap.comp ((head N).comp (proj_1 (funSpace ? N) ?)
    ))
                          ((tail N).comp (proj_1 (funSpace ? N) ?))))))
                  ((tail N).comp (proj_1 (funSpace ? N) ?)))))))))

end Prec
end

section
section Prec

variable (phif phig : ApproximableMap ? N)

/-- The primitive-recursion operator `R(k) = fun z. ...`. -/
def recOp : ApproximableMap (funSpace ? N) (funSpace ? N) := curry (precBody phif phig
    )

end Prec
end

section
section Prec

variable (phif phig : ApproximableMap ? N)

/-- `recMap : ?  ->  N`, the least fixed point of `recOp` (before strictifying). -/
def recMap : ApproximableMap ? N := toApproxMap (recOp phif phig).fixElement

end Prec
end

section
section Rfind

variable (phif : ApproximableMap ? N)

/-- The `Nat  -> . Bool` predicate `k  |->  f(k ::? v) = 0`, packaged so `Nat.rfind` is
     well-typed at
implicit transparency (Lean 4.33: `Nat  -> . Bool` no longer unfolds to `Nat  ->  Part
    Bool` in `rw`). -/
def rfindZeroPred {n : Nat} (f : List.Vector Nat (n + 1)  ->  Nat) (v : List.Vector
    Nat n) : Nat  -> . Bool :=
  fun k => Part.some (decide (f (k ::? v) = 0))

end Rfind
end

section
section Rfind

variable (phif : ApproximableMap ? N)

/-- `searchMap : ?  ->  N`, the least fixed point of `findOp` (the counter lives in
```

```
    coordinate `0`). -/
def searchMap : ApproximableMap ? N := toApproxMap (findOp phif).fixElement

end Rfind
end

section
section Rfind

variable (phif : ApproximableMap ? N)

/-- `findMap : ?  ->  N` starts the search with counter `0`. -/
def findMap : ApproximableMap ? N :=
  (searchMap phif).comp ((push N).comp (paired (constMap ? (natElem 0)) (idMap ?)))

end Rfind
end

section
/-- Test coordinates `0..n-1`. -/
def strictGuardN : Nat  ->  ApproximableMap ? N  ->  ApproximableMap ? N
  | 0, phi => phi
  | n + 1, phi => guard1 n (strictGuardN n phi)
end

section
/-- `<g_0, ..., g_{n-1}> : ?  ->  ?` placing `g_i` at coordinate `i` (`< n`) and `False`
     beyond. -/
def tupleMap : (n : Nat)  ->  (Fin n  ->  ApproximableMap ? N)  ->  ApproximableMap ?
    ?
  | 0, _ => constMap ? ?.bot
  | n + 1, gs => (push N).comp (paired (gs 0) (tupleMap n (fun i => gs i.succ)))
end

section
/-- `T.bot = Example23.botElt` (both least); bridges `zeroMap_bot` (lands in `T.bot`)
    with `cond_bot`
(phrased with `Example23.botElt`). -/
theorem T_bot_eq : (Example43.T).bot = Example23.botElt :=
  le_antisymm ((Example43.T).bot_le _) (Example23.botElt_le _)
end

section
theorem argElem_argLike {n : Nat} (a : Fin n  ->  Option Nat) : ArgLike (argElem a) :=
     by
  intro j
  rw [component_argElem]
  split
  * cases h : a _ with
    | none => exact Or.inl rfl
    | some k => exact Or.inr <k, rfl>
  * exact Or.inl rfl
end

section
/-- A coordinate that an `argElem` sets to `none` is `False`. -/
```

```
theorem component_argElem_none {n : Nat} (a : Fin n  ->  Option Nat) (i : Fin n) (hi :
     a i = none) :
    component (argElem a) (i : Nat) = N.bot := by
  rw [component_argElem, dite_eq_left i.isLt, Fin.eta, hi]; rfl
end

section
/-- Coordinates of the bottom stack are `False`. -/
theorem component_bot (i : Nat) : component (?.bot) i = N.bot := by
  have hbot : (?.bot) = ofSeq (fun _ => N.bot) :=
    le_antisymm (?.bot_le _)
      (le_of_component_le (fun i => by rw [component_ofSeq]; exact N.bot_le _))
  rw [hbot, component_ofSeq]
end

section
theorem component_push_succ (x : N.Element) (s : ?.Element) (j : Nat) :
    component ((push N).toElementMap (pair x s)) (j + 1) = component s j := by
  rw [ <-  Exercise511.component_tail, Exercise511.tail_push]
end

section
theorem component_push_zero (x : N.Element) (s : ?.Element) :
    component ((push N).toElementMap (pair x s)) 0 = x := by
  rw [ <-  Exercise511.head_apply, Exercise511.head_push]
end

section
/-- Every coordinate `j < n` of `vecElem v` is the total `v.get j`. -/
theorem component_vecElem_lt {n : Nat} (v : List.Vector Nat n) {j : Nat} (hj : j < n)
    :
    component (vecElem v) j = natElem (v.get <j, hj>) := by
  rw [component_vecElem, dite_eq_left hj]
end

section
/-- Two `?`-elements are equal iff all components agree. -/
theorem eq_of_component_eq {z z' : ?.Element}
    (h : forall  n, component z n = component z' n) : z = z' :=
  le_antisymm (le_of_component_le (fun n => (h n).le))
    (le_of_component_le (fun n => (h n).ge))
end

section
section Rfind

variable (phif : ApproximableMap ? N)

theorem evalAt_apply (z : ?.Element) (phi : (funSpace ? N).Element) :
    (evalAt z).toElementMap phi = (toApproxMap phi).toElementMap z := by
  rw [evalAt, toElementMap_comp, toElementMap_paired, toElementMap_idMap,
    toElementMap_constMap,
    evalMap_apply]

end Rfind
end
```

```
section
section Rfind

variable (phif : ApproximableMap ? N)

theorem findBody_apply (k : (funSpace ? N).Element) (z : ?.Element) :
    (findBody phif).toElementMap (pair k z) =
      (cond N).toElementMap
        (pair (zeroMap.toElementMap (phif.toElementMap z))
          (pair ((head N).toElementMap z)
            ((toApproxMap k).toElementMap
              ((push N).toElementMap
                (pair (succMap.toElementMap ((head N).toElementMap z))
                  ((tail N).toElementMap z)))))) := by
  rw [findBody, toElementMap_comp]
  simp only [toElementMap_paired, toElementMap_comp, toElementMap_proj_0,
    toElementMap_proj_1,
    fst_pair, snd_pair, evalMap_apply]

end Rfind
end

section
section Rfind

variable (phif : ApproximableMap ? N)

/-- The `0`-th approximant `S?(False)` is the bottom map. -/
theorem findOp_iterElem_zero : (findOp phif).iterElem 0 = (funSpace ? N).bot := by
  rw [iterElem_eq_iterate, Function.iterate_zero_apply]

end Rfind
end

section
theorem guard1_apply (j : Nat) (phi : ApproximableMap ? N) (z : ?.Element) :
    (guard1 j phi).toElementMap z =
      (cond N).toElementMap (pair (zeroMap.toElementMap (component z j))
        (pair (phi.toElementMap z) (phi.toElementMap z))) := by
  rw [guard1, toElementMap_comp]
  simp only [toElementMap_paired, toElementMap_comp, toElementMap_projN]
end

section
theorem guard1_bot (j : Nat) (phi : ApproximableMap ? N) (z : ?.Element)
    (h : component z j = N.bot) : (guard1 j phi).toElementMap z = N.bot := by
  rw [guard1_apply, h, zeroMap_bot, T_bot_eq, Exercise326.cond_bot]
end

section
/-- If `phi z = False` and `z?` is total-or-`False`, the guard stays `False`. -/
theorem guard1_inner_bot (j : Nat) (phi : ApproximableMap ? N) (z : ?.Element)
    (hz : component z j = N.bot  \/  exists  k, component z j = natElem k)
    (h : phi.toElementMap z = N.bot) : (guard1 j phi).toElementMap z = N.bot := by
  rcases hz with hb | <k, hk>
```

```
  * exact guard1_bot j phi z hb
  * rw [guard1_apply, hk, h]
    cases k with
    | zero => rw [zeroMap_natElem_zero, Exercise326.cond_true]
    | succ m => rw [zeroMap_natElem_succ, Exercise326.cond_false]
end

section
theorem guard1_total (j : Nat) (phi : ApproximableMap ? N) (z : ?.Element) {k : Nat}
    (h : component z j = natElem k) : (guard1 j phi).toElementMap z = phi.toElementMap
     z := by
  rw [guard1_apply, h]
  cases k with
  | zero => rw [zeroMap_natElem_zero, Exercise326.cond_true]
  | succ m => rw [zeroMap_natElem_succ, Exercise326.cond_false]
end

section
section Rfind

variable (phif : ApproximableMap ? N)

/-- Recursion equation for the approximant values: `S^{m+1}(False)` unfolds one search
    step using


`S?(False)` as the continuation. -/
theorem iterVal_succ (m : Nat) (z : ?.Element) :
    (toApproxMap ((findOp phif).iterElem (m + 1))).toElementMap z =
      (cond N).toElementMap
        (pair (zeroMap.toElementMap (phif.toElementMap z))
          (pair ((head N).toElementMap z)
            ((toApproxMap ((findOp phif).iterElem m)).toElementMap
              ((push N).toElementMap
                (pair (succMap.toElementMap ((head N).toElementMap z))
                  ((tail N).toElementMap z)))))) := by
  have h1 : (findOp phif).iterElem (m + 1) = (findOp phif).toElementMap ((findOp phif)
    .iterElem m) := by
    rw [iterElem_eq_iterate, iterElem_eq_iterate, Function.iterate_succ', Function.
    comp_apply]
  rw [h1]
  have hval := toElementMap_curry_apply (findBody phif) ((findOp phif).iterElem m) z
  rw [ <-  findOp] at hval
  rw [hval, findBody_apply]

end Rfind
end

section
/-- A realiser of a total function evaluates to its value. -/
theorem lamDef_coe_val {n : Nat} {phi : ApproximableMap ? N} {g : List.Vector Nat n
    ->  Nat}
    (h : LamDef phi (g : List.Vector Nat n  -> . Nat)) (v : List.Vector Nat n) :
    phi.toElementMap (vecElem v) = natElem (g v) :=
  h.defined v (g v) (by rw [PFun.coe_val]; exact Part.mem_some _)
end
```

```
section
/-- Transport a fun -definition along pointwise equality of the realised function. -/
theorem lamDef_congr {n : Nat} {phi : ApproximableMap ? N} {f f' : List.Vector Nat n
    -> . Nat}
    (h : forall  v, f v = f' v) (hf : LamDef phi f) : LamDef phi f' where
  defined v c hc := hf.defined v c (by rw [h v]; exact hc)
  undef v hv := hf.undef v (by rw [h v]; exact hv)
  strict := hf.strict
end

section
/-- Membership in `mOfFn` (Part monad): a vector is produced iff each coordinate is a
    value. -/
theorem mem_mOfFn : forall  {n : Nat} (p : Fin n  ->  Part Nat) (w : List.Vector Nat n
    ),
    w  in  List.Vector.mOfFn p  <->  forall  i, w.get i  in  p i
  | 0, p, w => by
      constructor
      * intro _ i; exact i.elim0
      * intro _
        show w  in  (pure List.Vector.nil : Part _)
        rw [Part.pure_eq_some, Part.mem_some_iff]
        exact List.Vector.eq_nil w
  | n + 1, p, w => by
      have hmof : List.Vector.mOfFn p
          = (p 0).bind (fun a => (List.Vector.mOfFn fun i => p i.succ).bind
              (fun u => Part.some (a ::? u))) := by
        simp only [List.Vector.mOfFn, Part.bind_eq_bind, Part.pure_eq_some]
      rw [hmof, Part.mem_bind_iff]
      constructor
      * rintro <a, ha, hrest>
        rw [Part.mem_bind_iff] at hrest
        obtain <u, hu, hw> := hrest
        rw [Part.mem_some_iff] at hw
        subst hw
        have hu' := (mem_mOfFn (fun i => p i.succ) u).mp hu
        intro i
        refine Fin.cases ?_ ?_ i
        * simpa using ha
        * intro k; simpa using hu' k
      * intro hall
        refine <w.head, ?_, ?_>
        * have := hall 0; rwa [List.Vector.get_zero] at this
        * rw [Part.mem_bind_iff]
          refine <w.tail, (mem_mOfFn (fun i => p i.succ) w.tail).mpr (fun k => ?_), ?_
    >
          * have h := hall k.succ
            rwa [ <-  List.Vector.get_tail_succ w k] at h
          * rw [Part.mem_some_iff]; exact (List.Vector.cons_head_tail w).symm
end

section
theorem oneArg_bot : (oneArg).toElementMap N.bot = ?.bot := by
  rw [oneArg, toElementMap_comp, toElementMap_paired, toElementMap_idMap,
    toElementMap_constMap]
  apply eq_of_component_eq
```

```
  intro i
  cases i with
  | zero => rw [component_push_zero, component_bot]
  | succ k => rw [component_push_succ, component_bot, component_bot]
end

section
section Prec

variable (phif phig : ApproximableMap ? N)

theorem precBody_apply (k : (funSpace ? N).Element) (z : ?.Element) :
    (precBody phif phig).toElementMap (pair k z) =
      (cond N).toElementMap
        (pair (zeroMap.toElementMap ((head N).toElementMap z))
          (pair (phif.toElementMap ((tail N).toElementMap z))
            (phig.toElementMap
              ((push N).toElementMap
                (pair (predMap.toElementMap ((head N).toElementMap z))
                  ((push N).toElementMap
                    (pair ((toApproxMap k).toElementMap
                            ((push N).toElementMap
                              (pair (predMap.toElementMap ((head N).toElementMap z))
                                ((tail N).toElementMap z))))
                      ((tail N).toElementMap z)))))))) := by
  rw [precBody, toElementMap_comp]
  simp only [toElementMap_paired, toElementMap_comp, toElementMap_proj_0,
    toElementMap_proj_1,
    fst_pair, snd_pair, evalMap_apply]

end Prec
end

section
theorem push_natElem_vecElem {n : Nat} (a : Nat) (u : List.Vector Nat n) :
    (push N).toElementMap (pair (natElem a) (vecElem u)) = vecElem (a ::? u) := by
  apply eq_of_component_eq
  intro j
  cases j with
  | zero =>
    rw [component_push_zero, component_vecElem, dite_eq_left (Nat.succ_pos n)]
    exact congrArg natElem (List.Vector.get_cons_zero a u).symm
  | succ k =>
    rw [component_push_succ, component_vecElem, component_vecElem]
    by_cases hk : k < n
    * rw [dite_eq_left hk, dite_eq_left (show k + 1 < n + 1 by omega)]
      exact congrArg natElem (List.Vector.get_cons_succ a u <k, hk>).symm
    * rw [dite_eq_right hk, dite_eq_right (show not  k + 1 < n + 1 by omega)]
end

section
section Rfind

variable (phif : ApproximableMap ? N)

theorem findMap_vecElem {n : Nat} (v : List.Vector Nat n) :
```

```
    (findMap phif).toElementMap (vecElem v) = (searchMap phif).toElementMap (vecElem
    (0 ::? v)) := by
  rw [findMap, toElementMap_comp, toElementMap_comp, toElementMap_paired,
    toElementMap_constMap,
    toElementMap_idMap, push_natElem_vecElem]

end Rfind
end

section
theorem oneArg_natElem (a : Nat) :
    (oneArg).toElementMap (natElem a) = vecElem (a ::? (List.Vector.nil : List.Vector
    Nat 0)) := by
  rw [oneArg, toElementMap_comp, toElementMap_paired, toElementMap_idMap,
    toElementMap_constMap]
  rw [show ?.bot = vecElem (List.Vector.nil : List.Vector Nat 0) from ?_,
    push_natElem_vecElem]
  exact eq_of_component_eq (fun i => by rw [component_bot, component_vecElem,
    dite_eq_right (by omega)])
end

section
/-- The value a realiser takes on a total argument is total-or-`False` (arg-like). -/
theorem realizer_value {n : Nat} {phi : ApproximableMap ? N} {f : List.Vector Nat n
    -> . Nat}
    (h : LamDef phi f) (v : List.Vector Nat n) :
    phi.toElementMap (vecElem v) = N.bot  \/  exists  k, phi.toElementMap (vecElem v)
    = natElem k := by
  by_cases hd : (f v).Dom
  * exact Or.inr <(f v).get hd, h.defined v _ (Part.get_mem hd)>
  * exact Or.inl (h.undef v hd)
end

section
section Prec

variable (phif phig : ApproximableMap ? N)

/-- The defining recursion equation. -/
theorem recMap_rec (z : ?.Element) :
    (recMap phif phig).toElementMap z =
      (cond N).toElementMap
        (pair (zeroMap.toElementMap ((head N).toElementMap z))
          (pair (phif.toElementMap ((tail N).toElementMap z))
            (phig.toElementMap
              ((push N).toElementMap
                (pair (predMap.toElementMap ((head N).toElementMap z))
                  ((push N).toElementMap
                    (pair ((recMap phif phig).toElementMap
                            ((push N).toElementMap
                              (pair (predMap.toElementMap ((head N).toElementMap z))
                                ((tail N).toElementMap z))))
                      ((tail N).toElementMap z)))))))) := by
  have hfix : (recOp phif phig).toElementMap (recOp phif phig).fixElement = (recOp
    phif phig).fixElement :=
    toElementMap_fixElement (recOp phif phig)
```

```
  have hval := toElementMap_curry_apply (precBody phif phig) (recOp phif phig).
    fixElement z
  rw [ <-  recOp, hfix] at hval
  rw [recMap, hval, precBody_apply]

end Prec
end

section
section Prec

variable (phif phig : ApproximableMap ? N)

/-- **Primitive recursion computes.** `recMap(<d, u>) = (Nat.rec (f u) (g ...) d)`. -/
theorem recMap_eval {n : Nat} (f_0 : List.Vector Nat n  ->  Nat) (g_0 : List.Vector
    Nat (n + 2)  ->  Nat)
    (hf : LamDef phif (f_0 : List.Vector Nat n  -> . Nat)) (hg : LamDef phig (g_0 :
    List.Vector Nat (n + 2)  -> . Nat))
    (u : List.Vector Nat n) (d : Nat) :
    (recMap phif phig).toElementMap (vecElem (d ::? u))
      = natElem (Nat.rec (motive := fun _ => Nat) (f_0 u)
          (fun y IH => g_0 (y ::? IH ::? u)) d) := by
  induction d with
  | zero =>
    rw [ <-  push_natElem_vecElem, recMap_rec, Exercise511.head_push, Exercise511.
    tail_push,
      zeroMap_natElem_zero, Exercise326.cond_true]
    exact lamDef_coe_val hf u
  | succ d ih =>
    rw [ <-  push_natElem_vecElem, recMap_rec, Exercise511.head_push, Exercise511.
    tail_push,
      zeroMap_natElem_succ, Exercise326.cond_false, predMap_natElem_succ,
    push_natElem_vecElem, ih,
      push_natElem_vecElem, push_natElem_vecElem]
    exact lamDef_coe_val hg _

end Prec
end

section
section Rfind

variable (phif : ApproximableMap ? N)

/-- The defining recursion equation of the search. -/
theorem searchMap_rec (z : ?.Element) :
    (searchMap phif).toElementMap z =
      (cond N).toElementMap
        (pair (zeroMap.toElementMap (phif.toElementMap z))
          (pair ((head N).toElementMap z)
            ((searchMap phif).toElementMap
              ((push N).toElementMap
                (pair (succMap.toElementMap ((head N).toElementMap z))
                  ((tail N).toElementMap z)))))) := by
  have hfix : (findOp phif).toElementMap (findOp phif).fixElement = (findOp phif).
    fixElement :=
```

```
    toElementMap_fixElement (findOp phif)
  have hval := toElementMap_curry_apply (findBody phif) (findOp phif).fixElement z
  rw [ <-  findOp, hfix] at hval
  rw [searchMap, hval, findBody_apply]

end Rfind
end

section
section Rfind

variable (phif : ApproximableMap ? N)

/-- **Search step (found).** If `f(k ::? v) = 0`, the search returns the counter `k`.
    -/
theorem searchMap_step_found {n : Nat} {f : List.Vector Nat (n + 1)  ->  Nat}
    (hf : LamDef phif (f : List.Vector Nat (n + 1)  -> . Nat)) (v : List.Vector Nat n)
     (k : Nat)
    (h0 : f (k ::? v) = 0) :
    (searchMap phif).toElementMap (vecElem (k ::? v)) = natElem k := by
  rw [ <-  push_natElem_vecElem, searchMap_rec, Exercise511.head_push, Exercise511.
    tail_push,
    push_natElem_vecElem, lamDef_coe_val hf (k ::? v), h0, zeroMap_natElem_zero,
    Exercise326.cond_true]

end Rfind
end

section
section Rfind

variable (phif : ApproximableMap ? N)

/-- **Search step (continue).** If `f(k ::? v)  !=  0`, the search advances to counter
    `k + 1`. -/
theorem searchMap_step_next {n : Nat} {f : List.Vector Nat (n + 1)  ->  Nat}
    (hf : LamDef phif (f : List.Vector Nat (n + 1)  -> . Nat)) (v : List.Vector Nat n)
     (k : Nat)
    (hne : f (k ::? v)  !=  0) :
    (searchMap phif).toElementMap (vecElem (k ::? v)) =
      (searchMap phif).toElementMap (vecElem ((k + 1) ::? v)) := by
  obtain <m, hm> : exists  m, f (k ::? v) = m + 1 := <f (k ::? v) - 1, by omega>
  rw [ <-  push_natElem_vecElem, searchMap_rec, Exercise511.head_push, Exercise511.
    tail_push,
    push_natElem_vecElem, lamDef_coe_val hf (k ::? v), hm, zeroMap_natElem_succ,
    Exercise326.cond_false, succMap_natElem, push_natElem_vecElem]

end Rfind
end

section
section Rfind

variable (phif : ApproximableMap ? N)

/-- **Capstone (least zero ? value).** If `m` is a zero of `f(* ::? v)` and nothing
```

```
    below it is, then
starting from any counter `j  <=  m` the search returns `m`. (Downward induction on `m
    - j`.) -/
theorem searchMap_climb {n : Nat} {f : List.Vector Nat (n + 1)  ->  Nat}
    (hf : LamDef phif (f : List.Vector Nat (n + 1)  -> . Nat)) (v : List.Vector Nat n)
     (m : Nat)
    (hm0 : f (m ::? v) = 0) (hmin : forall  i, i < m  ->  f (i ::? v)  !=  0) :
    forall  d j, j + d = m  ->  (searchMap phif).toElementMap (vecElem (j ::? v)) =
    natElem m := by
  intro d
  induction d with
  | zero =>
    intro j hj
    have hjm : j = m := by omega
    rw [hjm]
    exact searchMap_step_found phif hf v m hm0
  | succ d ih =>
    intro j hj
    have hjm : j < m := by omega
    rw [searchMap_step_next phif hf v j (hmin j hjm)]
    exact ih (j + 1) (by omega)

end Rfind
end

section
/-- If some coordinate `i < n` is `False` (and `z` is arg-like), the guard forces `
    False`. -/
theorem strictGuardN_strict (n : Nat) (phi : ApproximableMap ? N) (z : ?.Element)
    (hz : ArgLike z) {i : Nat} (hi : i < n) (hb : component z i = N.bot) :
    (strictGuardN n phi).toElementMap z = N.bot := by
  induction n with
  | zero => exact absurd hi (Nat.not_lt_zero i)
  | succ m ih =>
    rw [strictGuardN]
    rcases Nat.lt_succ_iff_lt_or_eq.mp hi with hlt | heq
    * exact guard1_inner_bot m _ z (hz m) (ih hlt)
    * exact guard1_bot m _ z (heq  |>  hb)
end

section
/-- On arguments whose first `n` coordinates are all total, the guard is transparent.
    -/
theorem strictGuardN_total (n : Nat) (phi : ApproximableMap ? N) (z : ?.Element)
    (h : forall  j, j < n  ->  exists  k, component z j = natElem k) :
    (strictGuardN n phi).toElementMap z = phi.toElementMap z := by
  induction n with
  | zero => rfl
  | succ m ih =>
    obtain <k, hk> := h m (Nat.lt_succ_self m)
    rw [strictGuardN, guard1_total m _ z hk]
    exact ih (fun j hj => h j (Nat.lt_succ_of_lt hj))
end

section
section Rfind
```

```
variable (phif : ApproximableMap ? N)

/-- The least map of the function space evaluates to `False` everywhere. -/
theorem toApproxMap_bot (z : ?.Element) :
    (toApproxMap ((funSpace ? N).bot)).toElementMap z = N.bot := by
  refine le_antisymm ?_ (N.bot_le _)
  have h2 := (funSpaceEquiv ? N).monotone
    ((funSpace ? N).bot_le ((funSpaceEquiv ? N).symm (constMap ? N.bot)))
  rw [OrderIso.apply_symm_apply, funSpaceEquiv_apply] at h2
  have h3 := (le_iff_toElementMap_le.mp h2) z
  rwa [toElementMap_constMap] at h3

end Rfind
end

section
section Rfind

variable (phif : ApproximableMap ? N)

/-- Value of the `0`-th approximant: `False`. -/
theorem iterVal_zero (z : ?.Element) :
    (toApproxMap ((findOp phif).iterElem 0)).toElementMap z = N.bot := by
  rw [findOp_iterElem_zero, toApproxMap_bot]

end Rfind
end

section
section Rfind

variable (phif : ApproximableMap ? N)

/-- Along a *no-zero* trace, every finite approximant returns `False` (induction on the
     approximant
index, with the counter universally quantified since the search advances it). -/
theorem iterVal_bot {n : Nat} {f : List.Vector Nat (n + 1)  ->  Nat}
    (hf : LamDef phif (f : List.Vector Nat (n + 1)  -> . Nat)) (v : List.Vector Nat n)
    (hdiv : forall  k, f (k ::? v)  !=  0) :
    forall  m k, (toApproxMap ((findOp phif).iterElem m)).toElementMap (vecElem (k ::?
     v)) = N.bot := by
  intro m
  induction m with
  | zero => intro k; exact iterVal_zero phif (vecElem (k ::? v))
  | succ m ih =>
    intro k
    have hne := hdiv k
    obtain <j, hj> : exists  j, f (k ::? v) = j + 1 := <f (k ::? v) - 1, by omega>
    rw [ <-  push_natElem_vecElem, iterVal_succ, Exercise511.head_push, Exercise511.
    tail_push,
      push_natElem_vecElem, lamDef_coe_val hf (k ::? v), hj, zeroMap_natElem_succ,
      Exercise326.cond_false, succMap_natElem, push_natElem_vecElem]
    exact ih (k + 1)

end Rfind
```

```
end

section
section Rfind

variable (phif : ApproximableMap ? N)

/-- **Divergence.** If `f(* ::? v)` has no zero, the search is `False` from every
    counter. The fixed point
is the directed union of the approximants `S?(False)` (Theorem 4.2(iii)); pushing the
    evaluation at
`vecElem (k ::? v)` through this sup (continuity, `toElementMap_iSupDirected`) reduces
    the claim to
`iterVal_bot`. -/
theorem searchMap_diverge {n : Nat} {f : List.Vector Nat (n + 1)  ->  Nat}
    (hf : LamDef phif (f : List.Vector Nat (n + 1)  -> . Nat)) (v : List.Vector Nat n)
    (hdiv : forall  k, f (k ::? v)  !=  0) (k : Nat) :
    (searchMap phif).toElementMap (vecElem (k ::? v)) = N.bot := by
  refine le_antisymm ?_ (N.bot_le _)
  rw [searchMap,  <-  evalAt_apply, fixElement_eq_iSupDirected,
    toElementMap_iSupDirected]


  apply NeighborhoodSystem.iSupDirected_le
  intro m
  rw [evalAt_apply]
  exact le_of_eq (iterVal_bot phif hf v hdiv m k)

end Rfind
end

section
theorem tupleMap_succ (n : Nat) (gs : Fin (n + 1)  ->  ApproximableMap ? N) :
    tupleMap (n + 1) gs = (push N).comp (paired (gs 0) (tupleMap n (fun i => gs i.succ
    ))) := rfl
end

section
/-- Coordinate `i < n` of the tuple is exactly `g_i`. -/
theorem component_tupleMap : forall  (n : Nat) (gs : Fin n  ->  ApproximableMap ? N) (
    z : ?.Element)
    (i : Nat) (hi : i < n),
    component ((tupleMap n gs).toElementMap z) i = (gs <i, hi>).toElementMap z
  | 0, _, _, i, hi => absurd hi (Nat.not_lt_zero i)
  | n + 1, gs, z, 0, hi => by
      rw [tupleMap_succ, toElementMap_comp, toElementMap_paired, component_push_zero]
      exact congrArg (fun w => (gs w).toElementMap z) (Fin.ext (by simp))
  | n + 1, gs, z, i + 1, hi => by
      rw [tupleMap_succ, toElementMap_comp, toElementMap_paired, component_push_succ,
        component_tupleMap n (fun j => gs j.succ) z i (Nat.lt_of_succ_lt_succ hi)]
      rfl
end

section
theorem tupleMap_zero (gs : Fin 0  ->  ApproximableMap ? N) :
    tupleMap 0 gs = constMap ? ?.bot := rfl
end
```

```
section
/-- Coordinates ` >=  n` of the tuple are `False`. -/
theorem component_tupleMap_ge : forall  (n : Nat) (gs : Fin n  ->  ApproximableMap ? N
    ) (z : ?.Element)
    (i : Nat), n  <=  i  ->  component ((tupleMap n gs).toElementMap z) i = N.bot
  | 0, gs, z, i, _ => by rw [tupleMap_zero, toElementMap_constMap, component_bot]
  | n + 1, gs, z, i, hi => by
      obtain <j, rfl> : exists  j, i = j + 1 := <i - 1, by omega>
      rw [tupleMap_succ, toElementMap_comp, toElementMap_paired, component_push_succ]
      exact component_tupleMap_ge n (fun k => gs k.succ) z j (by omega)
end

section
/-- The tuple of realisers, applied to a total argument, is arg-like. -/
theorem tupleMap_argLike {m nn : Nat} (phig : Fin nn  ->  ApproximableMap ? N)
    (g : Fin nn  ->  List.Vector Nat m  -> . Nat) (hg : forall  i, LamDef (phig i) (g
    i)) (v : List.Vector Nat m) :
    ArgLike ((tupleMap nn phig).toElementMap (vecElem v)) := by
  intro i
  by_cases hi : i < nn
  * rw [component_tupleMap nn phig (vecElem v) i hi]; exact realizer_value (hg <i, hi
    >) v
  * rw [component_tupleMap_ge nn phig (vecElem v) i (Nat.le_of_not_lt hi)]; exact Or.
    inl rfl
end

section
/-- The tuple of realisers, applied to a total argument, equals `vecElem w` where `w_i
    = g_i(v)`. -/
theorem tupleMap_vecElem_eq {m nn : Nat} (phig : Fin nn  ->  ApproximableMap ? N)
    (g : Fin nn  ->  List.Vector Nat m  -> . Nat) (hg : forall  i, LamDef (phig i) (g
    i))
    (v : List.Vector Nat m) (w : List.Vector Nat nn) (hw : forall  i, w.get i  in  g i
     v) :
    (tupleMap nn phig).toElementMap (vecElem v) = vecElem w := by
  apply eq_of_component_eq
  intro i
  by_cases hi : i < nn
  * rw [component_tupleMap nn phig (vecElem v) i hi, component_vecElem_lt w hi]
    exact (hg <i, hi>).defined v (w.get <i, hi>) (hw <i, hi>)
  * rw [component_tupleMap_ge nn phig (vecElem v) i (Nat.le_of_not_lt hi),
    component_vecElem,
      dite_eq_right hi]
end

section
/-- `vecElem v` has total coordinates throughout its arity. -/
theorem vecElem_totals {n : Nat} (v : List.Vector Nat n) :
    forall  j, j < n  ->  exists  k, component (vecElem v) j = natElem k :=
  fun j hj => <v.get <j, hj>, component_vecElem_lt v hj>
end

section
/-- **Master constructor.** Wrapping an `inner` map (correct on total arguments) in `
    strictGuardN n`
```

```
yields a full, very strict fun -definition. The `strict` clause is then automatic. -/
theorem lamDef_of_inner {n : Nat} (inner : ApproximableMap ? N) (f : List.Vector Nat n
      -> . Nat)
    (hdef : forall  (v : List.Vector Nat n) (c : Nat), c  in  f v  ->  inner.
    toElementMap (vecElem v) = natElem c)
    (hundef : forall  (v : List.Vector Nat n), not  (f v).Dom  ->  inner.toElementMap
    (vecElem v) = N.bot) :
    LamDef (strictGuardN n inner) f where
  defined v c hc := by
    rw [strictGuardN_total n inner (vecElem v) (vecElem_totals v)]; exact hdef v c hc
  undef v hv := by
    rw [strictGuardN_total n inner (vecElem v) (vecElem_totals v)]; exact hundef v hv
  strict z hz i hi hb := strictGuardN_strict n inner z hz hi hb
end

section
/-- **Composition** (Scott's multivariate composition). If `f` and each `g_i` are fun -
    defined, so is
`v  |->  (<g_0 v, ..., g_{nn-1} v>) >>= f`. -/
theorem lamDef_comp {m nn : Nat} (f : List.Vector Nat nn  -> . Nat) (g : Fin nn  ->
    List.Vector Nat m  -> . Nat)
    (phif : ApproximableMap ? N) (hf : LamDef phif f)
    (phig : Fin nn  ->  ApproximableMap ? N) (hg : forall  i, LamDef (phig i) (g i)) :
    LamDef (strictGuardN m (phif.comp (tupleMap nn phig)))
      (fun v => (List.Vector.mOfFn fun i => g i v) >>= f) := by
  refine lamDef_of_inner (phif.comp (tupleMap nn phig)) _ ?_ ?_
  * intro v c hc
    have hc' : c  in  (List.Vector.mOfFn fun i => g i v).bind (fun w => f w) := hc
    rw [Part.mem_bind_iff] at hc'
    obtain <w, hw, hcw> := hc'
    rw [toElementMap_comp, tupleMap_vecElem_eq phig g hg v w ((mem_mOfFn _ w).mp hw)]
    exact hf.defined w c hcw
  * intro v hv
    rw [toElementMap_comp]
    by_cases hall : forall  i, (g i v).Dom
    * set w : List.Vector Nat nn := List.Vector.ofFn (fun i => (g i v).get (hall i))
    with hw_def
      have hwget : forall  i, w.get i  in  g i v := by
        intro i; rw [hw_def, List.Vector.get_ofFn]; exact Part.get_mem (hall i)
      rw [tupleMap_vecElem_eq phig g hg v w hwget]
      refine hf.undef w (fun hfw => hv ?_)
      have hmem : (f w).get hfw  in  (List.Vector.mOfFn fun i => g i v).bind (fun w =>
     f w) := by
        rw [Part.mem_bind_iff]
        exact <w, (mem_mOfFn _ w).mpr hwget, Part.get_mem hfw>
      exact Part.dom_iff_mem.mpr <_, hmem>
    * rw [not_forall] at hall
      obtain <j, hj> := hall
      refine hf.strict ((tupleMap nn phig).toElementMap (vecElem v))
        (tupleMap_argLike phig g hg v) j j.isLt ?_
      rw [component_tupleMap nn phig (vecElem v) j j.isLt, Fin.eta]
      exact (hg j).undef v hj
end

section
section Prec
```

```
variable (phif phig : ApproximableMap ? N)

/-- **Primitive recursion** (`Nat.Primrec'.prec` shape). -/
theorem lamDef_prec {n : Nat} (f_0 : List.Vector Nat n  ->  Nat) (g_0 : List.Vector
    Nat (n + 2)  ->  Nat)
    (hf : LamDef phif (f_0 : List.Vector Nat n  -> . Nat))
    (hg : LamDef phig (g_0 : List.Vector Nat (n + 2)  -> . Nat)) :
    LamDef (strictGuardN (n + 1) (recMap phif phig))
      (((fun v : List.Vector Nat (n + 1) =>
        v.head.rec (f_0 v.tail) fun y IH => g_0 (y ::? IH ::? v.tail)) :
        List.Vector Nat (n + 1)  ->  Nat) : List.Vector Nat (n + 1)  -> . Nat) := by
  refine lamDef_of_inner (recMap phif phig) _ (fun v c hc => ?_)
    (fun v hv => by rw [PFun.coe_val] at hv; exact absurd trivial hv)
  rw [PFun.coe_val, Part.mem_some_iff] at hc
  subst hc
  obtain <d, u, rfl> := List.Vector.exists_eq_cons v
  rw [List.Vector.head_cons, List.Vector.tail_cons]
  exact recMap_eval phif phig f_0 g_0 hf hg u d

end Prec
end

section
/-- **Composition of total functions** (the `Nat.Primrec'.comp` shape). -/
theorem lamDef_primComp {m nn : Nat} (f_0 : List.Vector Nat nn  ->  Nat)
    (g_0 : Fin nn  ->  List.Vector Nat m  ->  Nat)
    (phif : ApproximableMap ? N) (hf : LamDef phif (f_0 : List.Vector Nat nn  -> . Nat
    ))
    (phig : Fin nn  ->  ApproximableMap ? N) (hg : forall  i, LamDef (phig i) (g_0 i :
     List.Vector Nat m  -> . Nat)) :
    LamDef (strictGuardN m (phif.comp (tupleMap nn phig)))
      ((fun a => f_0 (List.Vector.ofFn fun i => g_0 i a) : List.Vector Nat m  ->  Nat)
     :
        List.Vector Nat m  -> . Nat) := by
  refine lamDef_congr (fun v => ?_)
    (lamDef_comp (f_0 : _  -> . Nat) (fun i => (g_0 i : _  -> . Nat)) phif hf phig hg)
  simp only [PFun.coe_val, Vector.mOfFn_part_some]
  change (Part.some (List.Vector.ofFn fun i => g_0 i v)).bind (fun w => Part.some (f_0
     w))
      = Part.some (f_0 (List.Vector.ofFn fun i => g_0 i v))
  exact Part.bind_some _ _
end

section
section Rfind

variable (phif : ApproximableMap ? N)

/-- **Minimisation** (`Nat.Partrec'.rfind` shape). Given a (very strict) realiser of a
    total
`f : Vector Nat (n+1)  ->  Nat`, `strictGuardN n (findMap phif)` fun -defines
`v  |->  ?k. f(k ::? v) = 0`. -/
theorem lamDef_rfind {n : Nat} {f : List.Vector Nat (n + 1)  ->  Nat}
    (hf : LamDef phif (f : List.Vector Nat (n + 1)  -> . Nat)) :
    LamDef (strictGuardN n (findMap phif))
```

```
      (fun v : List.Vector Nat n => Nat.rfind (rfindZeroPred f v)) := by
  refine lamDef_of_inner (findMap phif) _ ?_ ?_
  * intro v c hc
    rw [findMap_vecElem]
    have hc' : c  in  Nat.rfind (rfindZeroPred f v) := hc
    rw [Nat.mem_rfind] at hc'
    obtain <hcs, hcm> := hc'
    have h0 : f (c ::? v) = 0 := of_decide_eq_true (Part.mem_some_iff.mp hcs).symm
    have hmin : forall  i, i < c  ->  f (i ::? v)  !=  0 :=
      fun i hi => of_decide_eq_false (Part.mem_some_iff.mp (hcm hi)).symm
    exact searchMap_climb phif hf v c h0 hmin c 0 (by omega)
  * intro v hv
    rw [findMap_vecElem]
    have hdiv : forall  k, f (k ::? v)  !=  0 := by
      intro k h0
      refine hv ?_
      have hdom : (Nat.rfind (rfindZeroPred f v)).Dom := by
        rw [Nat.rfind_dom]
        refine <k, ?_, fun {m} _ => trivial>
        change true  in  Part.some (decide (f (k ::? v) = 0))
        rw [Part.mem_some_iff]
        exact (decide_eq_true_iff.mpr h0).symm
      exact hdom
    exact searchMap_diverge phif hf v hdiv 0

end Rfind
end

section
/-- The total-function specialisation of `lamDef_of_inner`. -/
theorem lamDef_total {n : Nat} (psi : ApproximableMap ? N) (g : List.Vector Nat n  ->
     Nat)
    (hval : forall  v, psi.toElementMap (vecElem v) = natElem (g v)) :
    LamDef (strictGuardN n psi) (g : List.Vector Nat n  -> . Nat) :=
  lamDef_of_inner psi _
    (fun v c hc => by rw [PFun.coe_val, Part.mem_some_iff] at hc; rw [hval,  <-  hc])
    (fun v hv => by rw [PFun.coe_val] at hv; exact absurd trivial hv)
end

section
/-- `get i : Vector Nat n  ->  Nat`, the `i`-th projection. -/
theorem lamDef_get {n : Nat} (i : Fin n) :
    LamDef (strictGuardN n (projN N i))
      ((fun v => v.get i : List.Vector Nat n  ->  Nat) : List.Vector Nat n  -> . Nat)
    :=
  lamDef_total _ _ (fun v => by
    rw [toElementMap_projN, component_vecElem_lt v i.isLt, Fin.eta])
end

section
/-- `succ : Vector Nat 1  ->  Nat`, `v  |->  v.head + 1`. -/
theorem lamDef_succ : LamDef (strictGuardN 1 (succMap.comp (projN N 0)))
    ((fun v => Nat.succ v.head : List.Vector Nat 1  ->  Nat) : List.Vector Nat 1  -> .
     Nat) :=
  lamDef_total _ _ (fun v => by
    rw [toElementMap_comp, toElementMap_projN, component_vecElem_lt v (by norm_num),
```

```
      succMap_natElem]
    congr 1
    simp [List.Vector.get_zero])
end

section
/-- `zero : Vector Nat 0  ->  Nat`, the constant `0` (nullary). -/
theorem lamDef_zero : LamDef (strictGuardN 0 (constMap ? (natElem 0)))
    ((fun _ => 0 : List.Vector Nat 0  ->  Nat) : List.Vector Nat 0  -> . Nat) :=
  lamDef_total (constMap ? (natElem 0)) _ (fun v => by rw [toElementMap_constMap])
end

section
/-- **Every `Nat.Primrec'` function is fun -definable** by a (very strict) approximable
     map. -/
theorem primrec_lamDef {n : Nat} {f : List.Vector Nat n  ->  Nat} (h : Nat.Primrec' f)
     :
    exists  phi : ApproximableMap ? N, LamDef phi (f : List.Vector Nat n  -> . Nat) :=
     by
  induction h with
  | zero => exact <_, lamDef_zero>
  | succ => exact <_, lamDef_succ>
  | get i => exact <_, lamDef_get i>
  | comp g _ _ ihf ihg =>
      obtain <phif, hphif> := ihf
      choose phig hphig using ihg
      exact <_, lamDef_primComp _ g phif hphif phig hphig>
  | prec _ _ ihf ihg =>
      obtain <phif, hphif> := ihf
      obtain <phig, hphig> := ihg
      exact <_, lamDef_prec phif phig _ _ hphif hphig>
end

end DomainTheory.NeighbourhoodSystems.Theorem56Full

namespace DomainTheory.NeighbourhoodSystems.Theorem56Full
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap
open DomainTheory.NeighbourhoodSystems.Example43 (N natElem zeroElt succMap predMap
    zeroMap
  succMap_natElem succMap_bot predMap_natElem_succ predMap_natElem_zero predMap_bot
  zeroMap_natElem_zero zeroMap_natElem_succ zeroMap_bot constLiftN)
open DomainTheory.NeighbourhoodSystems.Exercise326 (cond)
open DomainTheory.NeighbourhoodSystems.Exercise511 (head tail push)
section
/- concordance: source L2877 THEOREM 5.6 partial recursive functions -/
/-- **Theorem 5.6 (Scott 1981, PRG-19) -- every partial recursive function is fun -
    definable.** For each
`Nat.Partrec'` function `f : Vector Nat n  -> . Nat` there is an approximable map `phi
    : ?  ->  N` that is a
*very strict* fun -definition of `f`: it returns `f`'s value on total arguments, `False
    ` where `f` is
undefined, and `False` on any arg-like input with a `False` among its first `n`
    coordinates. -/
theorem partrec_lamDef {n : Nat} {f : List.Vector Nat n  -> . Nat} (h : Nat.Partrec' f
    ) :
    exists  phi : ApproximableMap ? N, LamDef phi f := by
```

```
  induction h with
  | prim hp => exact primrec_lamDef hp
  | comp g _ _ ihf ihg =>
      obtain <phif, hphif> := ihf
      choose phig hphig using ihg
      exact <_, lamDef_comp _ g phif hphif phig hphig>
  | rfind _ ih =>
      obtain <phif, hphif> := ih
      exact <_, lamDef_rfind phif hphif>
end

section
/-- **Scott's 1-ary meta-theorem.** A partial recursive `h : Nat  -> . Nat` (presented
    as a unary
`Nat.Partrec'`) is denoted by a single approximable map `tau : N  ->  N` that is
    correct on values,
divergent where `h` diverges, and strict. -/
theorem partrec_one {h : Nat  -> . Nat}
    (H : Nat.Partrec' (fun v : List.Vector Nat 1 => h v.head)) :
    exists  tau : ApproximableMap N N,
      (forall  a m, m  in  h a  ->  tau.toElementMap (natElem a) = natElem m)  /\
      (forall  a, not  (h a).Dom  ->  tau.toElementMap (natElem a) = N.bot)  /\
      tau.toElementMap N.bot = N.bot := by
  obtain <phi, hphi> := partrec_lamDef H
  refine <phi.comp oneArg, ?_, ?_, ?_>
  * intro a m hm
    rw [toElementMap_comp, oneArg_natElem]
    refine hphi.defined (a ::? List.Vector.nil) m ?_
    rwa [List.Vector.head_cons]
  * intro a hdom
    rw [toElementMap_comp, oneArg_natElem]
    refine hphi.undef (a ::? List.Vector.nil) ?_
    rwa [List.Vector.head_cons]
  * rw [toElementMap_comp, oneArg_bot]
    refine hphi.strict ?.bot (fun j => Or.inl (component_bot j)) 0 (by omega) (
    component_bot 0)
end

end DomainTheory.NeighbourhoodSystems.Theorem56Full
end
```

3.6.8 Exercise 5.7 — currying the lambda notation

Find definitions of

$$\lambda x, y.\, \tau \quad \text{and} \quad \sigma(x, y)$$

which use only λv with one variable and applications only to one argument at a time. Note that use must be made of the combinators p_0, p_1, pair. Generalize the result to functions of many variables.

Exercise_5_7.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
```

```
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_12
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

section
/- concordance: source L3005 EXERCISE 5.7 currying the lambda notation -/
/- concordance: source L3003 EXERCISES -- Lecture V exercises -/
-- Exercise507/curry_3.lean
namespace DomainTheory.NeighbourhoodSystems.Exercise507

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap

variable {a b ? ? : Type*}
  {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
  {V_2 : NeighborhoodSystem ?} {V_3 : NeighborhoodSystem ?}

/-- **Three-variable abstraction** `fun x, y, z. tau`, via nested currying on `(D_0  x
    D_1)  x  D_2  ->  D_3`.
Each `curry` strips one variable, so a function of `n` variables is reached by `n - 1`
   curryings --
single-variable binders all the way down. -/
def curry_3 (g : ApproximableMap (prod (prod V_0 V_1) V_2) V_3) :
    ApproximableMap V_0 (funSpace V_1 (funSpace V_2 V_3)) :=
  curry (curry g)

end DomainTheory.NeighbourhoodSystems.Exercise507
end

section
namespace DomainTheory.NeighbourhoodSystems.Exercise507
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap

section
variable {a b ? ? : Type*}
  {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}

/-- The combinator `sigma(x, y)` (apply `sigma : D_0  x  D_1  ->  D_2` to two arguments
   ) is expressed through the
*one-variable* curried form: `sigma(<x, y>) = (curry sigma)(x)(y)`. The right-hand side
    uses only
single-variable application. -/
theorem app_two_args (sigma : ApproximableMap (prod V_0 V_1) V_2) (x : V_0.Element) (y
    : V_1.Element) :
    sigma.toElementMap (pair x y) =
      (toApproxMap ((curry sigma).toElementMap x)).toElementMap y := by
  rw [toElementMap_curry_apply]

/-- The defining equation of `curry_3`: `(fun x, y, z. g)(x)(y)(z) = g<<x, y>, z>`. -/
theorem curry_3_apply (g : ApproximableMap (prod (prod V_0 V_1) V_2) V_3)
    (x : V_0.Element) (y : V_1.Element) (z : V_2.Element) :
    (toApproxMap ((toApproxMap ((curry_3 g).toElementMap x)).toElementMap y)).
   toElementMap z =
      g.toElementMap (pair (pair x y) z) := by
  rw [curry_3, toElementMap_curry_apply, toElementMap_curry_apply]
```

```
/-- **Two-variable abstraction.** `fun x, y. tau` is `curry`, characterised by `(curry
    g)(x)(y) = g<x, y>`.
It binds one variable `x` and returns the one-variable function `fun y. g<x, y>`. -/
theorem lam_two_vars (g : ApproximableMap (prod V_0 V_1) V_2) (x : V_0.Element) (y :
    V_1.Element) :
    (toApproxMap ((curry g).toElementMap x)).toElementMap y = g.toElementMap (pair x y
    ) :=
  toElementMap_curry_apply g x y

/-- **Surjective pairing.** Every element `z : D_0  x  D_1` is recovered from its
    projections:
`<p_0(z), p_1(z)> = z`. This is what makes `p_0`, `p_1`, `pair` enough to encode the
    product. -/
theorem surjective_pairing (z : (prod V_0 V_1).Element) :
    pair ((proj_0 V_0 V_1).toElementMap z) ((proj_1 V_0 V_1).toElementMap z) = z := by
  rw [toElementMap_proj_0, toElementMap_proj_1, pair_fst_snd]

/-- **One-argument-at-a-time application.** For `h : D_0  ->  (D_1  ->  D_2)`, applying
    the uncurried map to
a `pair` is the same as applying `h` to `x` and then to `y`:
`(uncurry h)(<x, y>) = h(x)(y)`. -/
theorem uncurry_apply (h : ApproximableMap V_0 (funSpace V_1 V_2))
    (x : V_0.Element) (y : V_1.Element) :
    (uncurry h).toElementMap (pair x y) =
      (toApproxMap (h.toElementMap x)).toElementMap y := by
  rw [uncurry_eq, toElementMap_comp, toElementMap_paired, toElementMap_comp,
    toElementMap_proj_0, toElementMap_proj_1, fst_pair, snd_pair, evalMap_apply]

/-- The two encodings are mutually inverse: uncurrying the two-variable abstraction
    returns the
original two-variable function. -/
theorem uncurry_lam_two_vars (g : ApproximableMap (prod V_0 V_1) V_2) : uncurry (curry
    g) = g :=
  uncurry_curry g

end

end DomainTheory.NeighbourhoodSystems.Exercise507
end
```

3.6.9 Exercise 5.8 — combinatory completeness

(For combinator nuts.) Table 5.5 was meant to show how combinators could be defined in terms of λ-expressions. Can the tables be turned to show that with enough combinators available, every λ-expression can be defined by combining combinators, using $\sigma(\tau)$ as the *only* mode of combination?

Exercise_5_8.lean

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_12
```

```
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.Exercise508
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap

section
variable {a b ? : Type*}
variable {ax aa ab : Type*}
  {Vx : NeighborhoodSystem ax} {Va : NeighborhoodSystem aa} {Vb : NeighborhoodSystem
    ab}

/-- A *domain*: a token type together with its neighbourhood system. Bundling lets us
    build an
intrinsically-typed syntax of combinator terms. We work over `Type` (universe `0`),
    which covers all
the concrete domains of these lectures (`N`, `T`, `C`, ...) and their products/function
     spaces. -/
structure Dom where
  /-- The token type. -/
  carrier : Type
  /-- The neighbourhood system on `carrier`. -/
  sys : NeighborhoodSystem carrier

/-- The function domain `(A  ->  B)`; its carrier is `ApproximableMap A.sys B.sys` and
    its system is the
function space `funSpace A.sys B.sys`. -/
abbrev Dom.arrow (A B : Dom) : Dom := <ApproximableMap A.sys B.sys, funSpace A.sys B.
    sys>

/-- `CL A`: a **variable-free** combinator expression of type `A`. The only
    constructors are
constants and application, so application (`sigma(tau)`) is the sole mode of
    combination. -/
inductive CL : Dom  ->  Type 1 where
  | con {A : Dom} (c : A.sys.Element) : CL A
  | app {A B : Dom} (f : CL (A.arrow B)) (a : CL A) : CL B

/-- The denotation of a variable-free combinator expression: a single element. -/
def CL.denote : {A : Dom}  ->  CL A  ->  A.sys.Element
  | _, .con c => c
  | _, .app f a => (toApproxMap f.denote).toElementMap a.denote

/-- `Poly X A`: an open term of type `A` with a single free variable of type `X`.
    Constructors:
the variable, a closed constant (any element of any domain -- "enough combinators
    available"), and
application. This is the syntax of `fun `-bodies whose abstraction we want to eliminate
    . -/
inductive Poly (X : Dom) : Dom  ->  Type 1 where
  | var : Poly X X
  | con {A : Dom} (c : A.sys.Element) : Poly X A
  | app {A B : Dom} (f : Poly X (A.arrow B)) (a : Poly X A) : Poly X B

/-- The denotation of an open term: the body as a function of its free variable. -/
def Poly.denote {X : Dom} : {A : Dom}  ->  Poly X A  ->  X.sys.Element  ->  A.sys.
```

```
    Element
  | _, .var, x => x
  | _, .con c, _ => c
  | _, .app f a, x => (toApproxMap (f.denote x)).toElementMap (a.denote x)

/-- `I = fun x.x` as an element of `(X  ->  X)`. -/
def Ielem (X : Dom) : (X.arrow X).sys.Element := toFilter (idMap X.sys)

/-- `K = fun x, y.x` as an element of `(A  ->  (X  ->  A))`. -/
def Kelem (A X : Dom) : (A.arrow (X.arrow A)).sys.Element := toFilter (curry (proj_0 A
    .sys X.sys))

/-- The uncurried body of `S`: with `F : ??  ->  (??  ->  ?_b)`, `G : ??  ->  ??`, `x :
     ??`,
`Sbody<<F, G>, x> = (F x)(G x)`, expressed using only `proj`, `pair` and `eval`. -/
def Sbody (Vx : NeighborhoodSystem ax) (Va : NeighborhoodSystem aa) (Vb :
    NeighborhoodSystem ab) :
    ApproximableMap
      (prod (prod (funSpace Vx (funSpace Va Vb)) (funSpace Vx Va)) Vx) Vb :=
  (evalMap Va Vb).comp
    (paired
      ((evalMap Vx (funSpace Va Vb)).comp
        (paired
          ((proj_0 (funSpace Vx (funSpace Va Vb)) (funSpace Vx Va)).comp
            (proj_0 (prod (funSpace Vx (funSpace Va Vb)) (funSpace Vx Va)) Vx))
          (proj_1 (prod (funSpace Vx (funSpace Va Vb)) (funSpace Vx Va)) Vx)))
      ((evalMap Vx Va).comp
        (paired
          ((proj_1 (funSpace Vx (funSpace Va Vb)) (funSpace Vx Va)).comp
            (proj_0 (prod (funSpace Vx (funSpace Va Vb)) (funSpace Vx Va)) Vx))
          (proj_1 (prod (funSpace Vx (funSpace Va Vb)) (funSpace Vx Va)) Vx))))

/-- `S = fun f, g, x. f(x)(g(x))`, the curried form
`(??  ->  (??  ->  ?_b))  ->  ((??  ->  ??)  ->  (??  ->  ?_b))`. -/
def Smap (Vx : NeighborhoodSystem ax) (Va : NeighborhoodSystem aa) (Vb :
    NeighborhoodSystem ab) :
    ApproximableMap (funSpace Vx (funSpace Va Vb))
      (funSpace (funSpace Vx Va) (funSpace Vx Vb)) :=
  curry (curry (Sbody Vx Va Vb))

/-- `S = fun f, g, x. f(x)(g(x))` as an element of
`((X  ->  (A  ->  B))  ->  ((X  ->  A)  ->  (X  ->  B)))`. -/
def Selem (X A B : Dom) :
    ((X.arrow (A.arrow B)).arrow ((X.arrow A).arrow (X.arrow B))).sys.Element :=
  toFilter (Smap X.sys A.sys B.sys)

/-- **Bracket abstraction.** Turn an open body `t` (a `fun `-body with one free
    variable) into a
*variable-free* combinator expression denoting `fun x.t`, using only application and
    the combinators
`I`, `K`, `S` together with the constants already occurring in `t`:

* `[x] x        = I`
* `[x] c        = K(c)`           (`c` a constant / closed term)
* `[x] (f a)    = S([x]f)([x]a)`.
-/
```

```
def bracket {X : Dom} : {A : Dom}  ->  Poly X A  ->  CL (X.arrow A)
  | _, .var => .con (Ielem X)
  | _, .con c => .app (.con (Kelem _ X)) (.con c)
  | _, .app f a => .app (.app (.con (Selem X _ _)) (bracket f)) (bracket a)

/-- The defining equation of `S`: `S(F)(G)(x) = F(x)(G(x))`. -/
theorem Smap_apply (F : (funSpace Vx (funSpace Va Vb)).Element) (G : (funSpace Vx Va).
    Element)
    (x : Vx.Element) :
    (toApproxMap ((toApproxMap ((Smap Vx Va Vb).toElementMap F)).toElementMap G)).
    toElementMap x
      = (toApproxMap ((toApproxMap F).toElementMap x)).toElementMap ((toApproxMap G).
    toElementMap x) := by
  rw [Smap, toElementMap_curry_apply, toElementMap_curry_apply]
  simp only [Sbody, toElementMap_comp, toElementMap_paired, toElementMap_proj_0,
    toElementMap_proj_1, fst_pair, snd_pair, evalMap_apply]
end

section
variable {a b ? : Type*}

/-- `toApproxMap  comp  toFilter = id`: the representation of a function-space *element
    * by an approximable
map is a left inverse of `toFilter` (this is `funSpaceEquiv` unfolded). -/
theorem toApproxMap_toFilter {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
    (f : ApproximableMap V_0 V_1) : toApproxMap (toFilter f) = f := by
  have he := (funSpaceEquiv V_0 V_1).apply_symm_apply f
  rwa [funSpaceEquiv_apply, funSpaceEquiv_symm_apply] at he
end

section
variable {a b ? : Type*}
variable {ax aa ab : Type*}
  {Vx : NeighborhoodSystem ax} {Va : NeighborhoodSystem aa} {Vb : NeighborhoodSystem
    ab}

/-- `I(x) = x`. -/
theorem Ielem_apply (X : Dom) (x : X.sys.Element) :
    (toApproxMap (Ielem X)).toElementMap x = x := by
  rw [Ielem, toApproxMap_toFilter, toElementMap_idMap]

/-- `K(c)(x) = c`. -/
theorem Kelem_apply (A X : Dom) (c : A.sys.Element) (x : X.sys.Element) :
    (toApproxMap ((toApproxMap (Kelem A X)).toElementMap c)).toElementMap x = c := by
  rw [Kelem, toApproxMap_toFilter]
  have h := toElementMap_curry_apply (proj_0 A.sys X.sys) c x
  rw [toElementMap_proj_0, fst_pair] at h
  exact h

/-- `S(F)(G)(x) = F(x)(G(x))`. -/
theorem Selem_apply (X A B : Dom)
    (F : (X.arrow (A.arrow B)).sys.Element) (G : (X.arrow A).sys.Element) (x : X.sys.
    Element) :
    (toApproxMap ((toApproxMap ((toApproxMap (Selem X A B)).toElementMap F)).
    toElementMap G)).toElementMap x
      = (toApproxMap ((toApproxMap F).toElementMap x)).toElementMap ((toApproxMap G).
```

```
    toElementMap x) := by
  rw [Selem, toApproxMap_toFilter]
  exact Smap_apply F G x
end
end DomainTheory.NeighbourhoodSystems.Exercise508

section
namespace DomainTheory.NeighbourhoodSystems.Exercise508
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap
section
variable {a b ? : Type*}
variable {ax aa ab : Type*}
  {Vx : NeighborhoodSystem ax} {Va : NeighborhoodSystem aa} {Vb : NeighborhoodSystem
    ab}
/- concordance: source L3013 EXERCISE 5.8 combinatory completeness -/
/-- **Combinatory completeness (Exercise 5.8, Scott 1981, PRG-19).** The variable-free
     combinator
expression `bracket t` denotes exactly the function `fun x.t`: applied to any `x`, it
    yields `t.denote x`.

Thus, with `I`, `K`, `S` (and the constants of `t`) available, every `fun `-abstraction
    can be defined
by combining combinators using application as the only mode of combination -- Scott's
    "turning of the
tables". -/
theorem bracket_spec {X : Dom} {A : Dom} (t : Poly X A) (x : X.sys.Element) :
    (toApproxMap (bracket t).denote).toElementMap x = t.denote x := by
  induction t with
  | var =>
      simp only [bracket, CL.denote, Poly.denote]
      exact Ielem_apply X x
  | @con A c =>
      simp only [bracket, CL.denote, Poly.denote]
      exact Kelem_apply A X c x
  | @app A B f a ihf iha =>
      simp only [bracket, CL.denote, Poly.denote]
      have hS := Selem_apply X A B (CL.denote (bracket f)) (CL.denote (bracket a)) x
      rw [ihf, iha] at hS
      exact hS
end

end DomainTheory.NeighbourhoodSystems.Exercise508
end
```

3.6.10 Exercise 5.9 — common fixed points

Suppose that $f, g : \mathcal{D} \to \mathcal{D}$ are approximable and $f \circ g = g \circ f$. Show that f and g have a *least common fixed point* $x = f(x) = g(x)$. (Hint: Refer back to Exercise 4.20.) If in addition $f(\bot) = g(\bot)$, show that $\mathrm{fix}(f) = \mathrm{fix}(g)$. In particular, will $\mathrm{fix}(f) = \mathrm{fix}(f^2)$? What if we only assume $f \circ g = g^2 \circ f$?

Exercise_5_9.lean

```
module
```

```
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Exercise_4_11

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

section
variable {a : Type*} {V : NeighborhoodSystem a}

/- concordance: source L3015 EXERCISE 5.9 common fixed points -/
/-- **Exercise 5.9 (Scott 1981, PRG-19).** Commuting approximable maps have a least
    common fixed
point. If they also agree at `False`, their least fixed points coincide, and in
    particular
`fix(f) = fix(f?)`. -/
theorem example_509 :
    (forall  {f g : ApproximableMap V V}, f.comp g = g.comp f  ->
      exists  x : V.Element, f.toElementMap x = x  /\  g.toElementMap x = x  /\
        forall  w : V.Element, f.toElementMap w = w  ->  g.toElementMap w = w  ->  x
    <=  w)  /\
    (forall  {f g : ApproximableMap V V}, f.comp g = g.comp f  ->
      f.toElementMap V.bot = g.toElementMap V.bot  ->
        f.fixElement = g.fixElement)  /\
    (forall  f : ApproximableMap V V, f.fixElement = (f.comp f).fixElement) :=
  <commuting_least_common_fixed, fixElement_eq_of_commuting_bot, fixElement_iterTwice>

end
end DomainTheory.NeighbourhoodSystems
```

3.6.11 Exercise 5.10 — smash product

Suppose $\mathcal{D}_0$ and $\mathcal{D}_1$ are neighbourhood systems over disjoint sets Δ_0 and Δ_1. Define the *smash product* $\mathcal{D}_0 \otimes \mathcal{D}_1$ with neighbourhoods

$$\{\Delta_0 \cup \Delta_1\} \cup \{X \cup Y \mid X \in \mathcal{D}_0 \setminus \{\Delta_0\} \text{ and } Y \in \mathcal{D}_1 \setminus \{\Delta_1\}\}.$$

Show that this *is* a neighbourhood system. Define $(\mathcal{D}_0 \to_\perp \mathcal{D}_1)$ so that $|\mathcal{D}_0 \to_\perp \mathcal{D}_1|$ consists exactly of the *strict functions*. By introducing appropriate combinators, show that

$$(\mathcal{D}_0 \to_\perp (\mathcal{D}_1 \to_\perp \mathcal{D}_2)) \quad \text{and} \quad ((\mathcal{D}_0 \otimes \mathcal{D}_1) \to_\perp \mathcal{D}_2)$$

are isomorphic.

Exercise_5_10.lean

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Exercise_2_8

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_8
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Exercise_3_26
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Definition_3_8
```

```
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_5
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Lemma_3_6
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_7

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.Exercise510
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap

section
variable {a b ? : Type*}

/-- A *proper* product neighbourhood of the smash: `X  U  Y` with `X  in  ?_0 ? {
    Delta_0}` and
`Y  in  ?_1 ? {Delta_1}` (both factors strictly below their masters). -/
def SmashProper (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) (W : Set (a
    ? b)) : Prop :=
  exists  X Y, V_0.mem X  /\  X  !=  V_0.master  /\  V_1.mem Y  /\  Y  !=  V_1.master
     /\  W = prodNbhd X Y
end

section
variable {a b ? : Type*}
variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}

/-- A neighbourhood that is ` subseteq  Delta_0` and equals `Delta_0` exactly when it
    contains `Delta_0`. A proper factor
`X  !=  Delta_0` stays proper after intersecting with anything: `X  I  X'  !=  Delta_0
    `. -/
theorem inter_ne_master_left {X X' : Set a} (hX : V_0.mem X) (hXne : X  !=  V_0.master
    ) :
    X  I  X'  !=  V_0.master := by
  intro h
  apply hXne
  refine Set.Subset.antisymm (V_0.sub_master hX) ?_
  rw [ <-  h]; exact Set.inter_subset_left

theorem inter_ne_master_right {Y Y' : Set b} (hY : V_1.mem Y) (hYne : Y  !=  V_1.
    master) :
    Y  I  Y'  !=  V_1.master :=
  inter_ne_master_left (V_0 := V_1) hY hYne
end
end DomainTheory.NeighbourhoodSystems.Exercise510

section
namespace DomainTheory.NeighbourhoodSystems.Exercise510
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap
section
variable {a b ? : Type*}
variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}

/- concordance: source L3017 EXERCISE 5.10 smash product -/
/-- **Exercise 5.10 (Scott 1981, PRG-19) -- the smash product `?_0 ? ?_1`.**
    Neighbourhoods are the
master `Delta_0  U  Delta_1` together with the *proper* product neighbourhoods `X  U  Y
```

```
    ` (both factors proper).

*This is a neighbourhood system.* Condition (i) is the master clause. Condition (ii)
    is the new
content: given two smash neighbourhoods with a consistency witness `Z`,

* if either is the master, the intersection collapses to the other (since `X  subseteq
     Delta_0`, `Y  subseteq  Delta_1`);
* if both are proper, `Z` cannot be the master (that would force a factor to be `
    Delta_0`/`Delta_1`), so `Z` is
  a *proper* `U  U  V`; `U  subseteq  X  I  X'` and `V  subseteq  Y  I  Y'` then
    witness `X  I  X'  in  ?_0`, `Y  I  Y'  in  ?_1`, both
  still proper (`inter_ne_master_*`). -/
def smash (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) :
    NeighborhoodSystem (a ? b) where
  mem W := W = prodNbhd V_0.master V_1.master  \/  SmashProper V_0 V_1 W
  master := prodNbhd V_0.master V_1.master
  master_nonempty := by
    obtain <a, ha> := V_0.master_nonempty
    exact <Sum.inl a, mem_prodNbhd_inl.mpr ha>
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' Z (rfl | <X, Y, hX, hXne, hY, hYne, rfl>) hW' hZ hZsub
    * -- W = master
      rcases hW' with rfl | <X', Y', hX', hX'ne, hY', hY'ne, rfl>
      * left; rw [Set.inter_self]
      * right
        refine <X', Y', hX', hX'ne, hY', hY'ne, ?_>
        rw [prodNbhd_inter, Set.inter_eq_right.mpr (V_0.sub_master hX'),
          Set.inter_eq_right.mpr (V_1.sub_master hY')]
    * -- W = X  U  Y proper
      rcases hW' with rfl | <X', Y', hX', hX'ne, hY', hY'ne, rfl>
      * right
        refine <X, Y, hX, hXne, hY, hYne, ?_>
        rw [prodNbhd_inter, Set.inter_eq_left.mpr (V_0.sub_master hX),
          Set.inter_eq_left.mpr (V_1.sub_master hY)]
      * -- both proper: use the witness Z
        right
        rw [prodNbhd_inter] at hZsub  |-
        rcases hZ with hZeq | <U, V, hU, _, hV, _, rfl>
        * -- Z = master is impossible
          exfalso
          rw [hZeq] at hZsub
          obtain <hDelta_0, _> := prodNbhd_subset_iff.mp hZsub
          exact hXne (Set.Subset.antisymm (V_0.sub_master hX) (hDelta_0.trans Set.
    inter_subset_left))
        * obtain <hUsub, hVsub> := prodNbhd_subset_iff.mp hZsub
          refine <X  I  X', Y  I  Y', ?_, ?_, ?_, ?_, rfl>
          * exact V_0.inter_mem hX hX' hU hUsub
          * exact inter_ne_master_left hX hXne
          * exact V_1.inter_mem hY hY' hV hVsub
          * exact inter_ne_master_right hY hYne
  sub_master := by
    rintro W (rfl | <X, Y, hX, _, hY, _, rfl>)
    * exact subset_rfl
    * exact prodNbhd_subset_iff.mpr <V_0.sub_master hX, V_1.sub_master hY>
```

```
@[simp] theorem smash_master :
    (smash V_0 V_1).master = prodNbhd V_0.master V_1.master := rfl

@[simp] theorem smash_mem_bot {W : Set (a ? b)} :
    (smash V_0 V_1).bot.mem W  <->  W = prodNbhd V_0.master V_1.master := by
  rw [NeighborhoodSystem.mem_bot, smash_master]

/-- A proper product neighbourhood is a neighbourhood of the smash. -/
theorem smash_mem_proper {X : Set a} {Y : Set b} (hX : V_0.mem X) (hXne : X  !=  V_0.
    master)
    (hY : V_1.mem Y) (hYne : Y  !=  V_1.master) : (smash V_0 V_1).mem (prodNbhd X Y)
    :=
  Or.inr <X, Y, hX, hXne, hY, hYne, rfl>

/-- The *strict pairing* `<x, y>?`: the filter generated by the proper product
    neighbourhoods
`X  U  Y` with `X  in  x ? {Delta_0}`, `Y  in  y ? {Delta_1}` (plus the master). When
    either `x` or `y` is `False` (i.e.
contains only its master), this collapses to `False`. -/
def smashPair (x : V_0.Element) (y : V_1.Element) : (smash V_0 V_1).Element where
  mem W := W = prodNbhd V_0.master V_1.master  \/
    exists  X Y, x.mem X  /\  X  !=  V_0.master  /\  y.mem Y  /\  Y  !=  V_1.master
    /\  W = prodNbhd X Y
  sub := by
    rintro W (rfl | <X, Y, hX, hXne, hY, hYne, rfl>)
    * exact (smash V_0 V_1).master_mem
    * exact smash_mem_proper (x.sub hX) hXne (y.sub hY) hYne
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | <X, Y, hX, hXne, hY, hYne, rfl>) hW'
    * rcases hW' with rfl | <X', Y', hX', hX'ne, hY', hY'ne, rfl>
      * left; rw [Set.inter_self]
      * right
        exact <X', Y', hX', hX'ne, hY', hY'ne, by
          rw [prodNbhd_inter, Set.inter_eq_right.mpr (V_0.sub_master (x.sub hX')),
            Set.inter_eq_right.mpr (V_1.sub_master (y.sub hY'))]>
    * rcases hW' with rfl | <X', Y', hX', hX'ne, hY', hY'ne, rfl>
      * right
        exact <X, Y, hX, hXne, hY, hYne, by
          rw [prodNbhd_inter, Set.inter_eq_left.mpr (V_0.sub_master (x.sub hX)),
            Set.inter_eq_left.mpr (V_1.sub_master (y.sub hY))]>
      * right
        refine <X  I  X', Y  I  Y', x.inter_mem hX hX', inter_ne_master_left (x.sub hX
    ) hXne,
          y.inter_mem hY hY', inter_ne_master_right (y.sub hY) hYne, ?_>
        rw [prodNbhd_inter]
  up_mem := by
    rintro W W' (rfl | <X, Y, hX, hXne, hY, hYne, rfl>) hW' hsub
    * left
      exact Set.Subset.antisymm ((smash V_0 V_1).sub_master hW') hsub
    * rcases hW' with rfl | <X', Y', hX', hX'ne, hY', hY'ne, rfl>
      * left; rfl
      * obtain <hXX', hYY'> := prodNbhd_subset_iff.mp hsub
        right
        exact <X', Y', x.up_mem hX hX' hXX', hX'ne, y.up_mem hY hY' hYY', hY'ne, rfl>
```

```
@[simp] theorem mem_smashPair {x : V_0.Element} {y : V_1.Element} {W : Set (a ? b)} :
    (smashPair x y).mem W  <->  W = prodNbhd V_0.master V_1.master  \/
      exists  X Y, x.mem X  /\  X  !=  V_0.master  /\  y.mem Y  /\  Y  !=  V_1.master
     /\  W = prodNbhd X Y := Iff.rfl

/-- **A map is *strict* when `f(False) = False`.** Relationally: `f` sends the master
    input only to the master
output. -/
def IsStrict (f : ApproximableMap V_0 V_1) : Prop :=
  forall  ?Y?, f.rel V_0.master Y  ->  Y = V_1.master

/-- The strict maps `?_0  -> False ?_1`, as a subtype carrying the inherited
    approximation order. -/
abbrev StrictMap (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) :=
  {f : ApproximableMap V_0 V_1 // IsStrict f}

theorem eq_bot_of_no_proper {x : V_0.Element} (hx : forall  X, x.mem X  ->  X = V_0.
    master) :
    x = V_0.bot :=
  le_antisymm (fun W hW => by rw [NeighborhoodSystem.mem_bot]; exact hx W hW)
    (V_0.bot_le x)

/-- An element is `False` iff every neighbourhood it contains is the master: `x  !=
    False` exactly when `x`
contains a *proper* neighbourhood. -/
theorem exists_proper_of_ne_bot {x : V_0.Element} (hx : x  !=  V_0.bot) :
    exists  X, x.mem X  /\  X  !=  V_0.master := by
  by_contra hc
  refine hx (le_antisymm (fun W hW => ?_) (V_0.bot_le x))
  rw [NeighborhoodSystem.mem_bot]
  by_contra hWne
  exact hc <W, hW, hWne>

/-- The constant map at `False` is strict. -/
theorem isStrict_constBot : IsStrict (constMap V_0 (V_1.bot)) := by
  rintro Y <_, hY>
  rwa [NeighborhoodSystem.mem_bot] at hY

/-- Strictness is exactly `f(False) = False`. -/
theorem isStrict_iff_apply_bot {f : ApproximableMap V_0 V_1} :
    IsStrict f  <->  f.toElementMap V_0.bot = V_1.bot := by
  constructor
  * intro h
    apply Element.ext
    intro Y
    rw [NeighborhoodSystem.mem_bot]
    constructor
    * rintro <X, hX, hrel>
      rw [NeighborhoodSystem.mem_bot] at hX; subst hX
      exact h hrel
    * rintro rfl
      exact <V_0.master, V_0.bot.master_mem, f.master_rel>
  * intro h Y hrel
    have : (f.toElementMap V_0.bot).mem Y := <V_0.master, V_0.bot.master_mem, hrel>
    rw [h, NeighborhoodSystem.mem_bot] at this
```

```
    exact this

/-- Strictness is downward closed: a map below a strict map is strict. -/
theorem IsStrict.mono {f g : ApproximableMap V_0 V_1} (hf : IsStrict f) (hgf : g  <=
    f) : IsStrict g :=
  fun _ hrel => hf (hgf _ _ hrel)

/-- The principal filter of the master is `False`. -/
theorem principal_master_eq_bot {X : Set a} (hX : V_0.mem X) (hXm : X = V_0.master) :
    V_0.principal hX = V_0.bot := by
  subst hXm; rfl

/-- A `False` left factor collapses the strict pairing to `False` (choice-free). -/
theorem smashPair_bot_left (y : V_1.Element) : smashPair V_0.bot y = (smash V_0 V_1).
    bot := by
  apply eq_bot_of_no_proper
  rintro W (rfl | <X, Y, hxX, hXne, hyY, hYne, rfl>)
  * rfl
  * rw [NeighborhoodSystem.mem_bot] at hxX; exact absurd hxX hXne

/-- A `False` right factor collapses the strict pairing to `False` (choice-free). -/
theorem smashPair_bot_right (x : V_0.Element) : smashPair x V_1.bot = (smash V_0 V_1).
    bot := by
  apply eq_bot_of_no_proper
  rintro W (rfl | <X, Y, hxX, hXne, hyY, hYne, rfl>)
  * rfl
  * rw [NeighborhoodSystem.mem_bot] at hyY; exact absurd hyY hYne

/-- The strict pairing is `False` iff one of the components is `False`: the smash glues
     `(False, y)` and `(x, False)`
to a single bottom. -/
theorem smashPair_eq_bot_iff {x : V_0.Element} {y : V_1.Element} :
    smashPair x y = (smash V_0 V_1).bot  <->  x = V_0.bot  \/  y = V_1.bot := by
  constructor
  * intro h
    by_contra hcon
    obtain <hx, hy> := not_or.mp hcon
    obtain <X, hxX, hXne> := exists_proper_of_ne_bot hx
    obtain <Y, hyY, hYne> := exists_proper_of_ne_bot hy
    -- `prodNbhd X Y` is a proper member of `smashPair x y`, but `False` contains only
    the master.
    have hmem : (smashPair x y).mem (prodNbhd X Y) :=
      Or.inr <X, Y, hxX, hXne, hyY, hYne, rfl>
    rw [h, smash_mem_bot] at hmem
    obtain <hX, _> := prodNbhd_injective hmem
    exact hXne hX
  * intro h
    apply eq_bot_of_no_proper
    rintro W (rfl | <X, Y, hxX, hXne, hyY, hYne, rfl>)
    * rfl
    * exfalso
      rcases h with hx | hy
      * rw [hx, NeighborhoodSystem.mem_bot] at hxX; exact hXne hxX
      * rw [hy, NeighborhoodSystem.mem_bot] at hyY; exact hYne hyY

/-- The strict pairing is monotone in both arguments. -/
```

```
theorem smashPair_mono {x x' : V_0.Element} {y y' : V_1.Element} (hx : x  <=  x') (hy
    : y  <=  y') :
    smashPair x y  <=  smashPair x' y' := by
  rintro W (rfl | <X, Y, hxX, hXne, hyY, hYne, rfl>)
  * exact Or.inl rfl
  * exact Or.inr <X, Y, hx X hxX, hXne, hy Y hyY, hYne, rfl>

/-- A step set among strict maps: `[X, Y] = {f strict ? X f Y}`. -/
def sstep (X : Set a) (Y : Set b) : Set (StrictMap V_0 V_1) := {f | f.1.rel X Y}

@[simp] theorem mem_sstep {X : Set a} {Y : Set b} {f : StrictMap V_0 V_1} :
    f  in  sstep X Y  <->  f.1.rel X Y := Iff.rfl

/-- A finite intersection of strict step sets. -/
def sstepFun (L : List (Set a  x  Set b)) : Set (StrictMap V_0 V_1) :=
  {f | forall  p  in  L, f.1.rel p.1 p.2}

@[simp] theorem mem_sstepFun {L : List (Set a  x  Set b)} {f : StrictMap V_0 V_1} :
    f  in  sstepFun L  <->  forall  p  in  L, f.1.rel p.1 p.2 := Iff.rfl

@[simp] theorem sstepFun_nil : (sstepFun [] : Set (StrictMap V_0 V_1)) = Set.univ :=
    by
  ext f; simp

theorem sstepFun_append (L L' : List (Set a  x  Set b)) :
    (sstepFun (L ++ L') : Set (StrictMap V_0 V_1)) = sstepFun L  I  sstepFun L' := by
  ext f
  simp only [mem_sstepFun, List.mem_append, Set.mem_inter_iff]
  constructor
  * intro h; exact <fun p hp => h p (Or.inl hp), fun p hp => h p (Or.inr hp)>
  * rintro <hL, hL'> p (hp | hp)
    * exact hL p hp
    * exact hL' p hp

theorem sstepFun_cons (p : Set a  x  Set b) (L : List (Set a  x  Set b)) :
    (sstepFun (p :: L) : Set (StrictMap V_0 V_1)) = sstep p.1 p.2  I  sstepFun L := by
  ext f
  simp only [mem_sstepFun, List.mem_cons, Set.mem_inter_iff, mem_sstep]
  constructor
  * intro h; exact <h p (Or.inl rfl), fun q hq => h q (Or.inr hq)>
  * rintro <hp, hrest> q (rfl | hq)
    * exact hp
    * exact hrest q hq

theorem sstepFun_singleton (X : Set a) (Y : Set b) :
    (sstepFun [(X, Y)] : Set (StrictMap V_0 V_1)) = sstep X Y := by
  rw [sstepFun_cons, sstepFun_nil, Set.inter_univ]

@[simp] theorem sstep_master_eq :
    (sstep V_0.master V_1.master : Set (StrictMap V_0 V_1)) = Set.univ := by
  ext f; simpa using f.1.master_rel

theorem sstep_inter_right {X : Set a} {Y Y' : Set b} (hY : V_1.mem Y) (hY' : V_1.mem Y
    ') :
    (sstep X Y  I  sstep X Y' : Set (StrictMap V_0 V_1)) = sstep X (Y  I  Y') := by
  ext f
```

```
  simp only [Set.mem_inter_iff, mem_sstep]
  constructor
  * rintro <h, h'>; exact f.1.inter_right h h'
  * intro h
    exact <f.1.mono h subset_rfl Set.inter_subset_left (f.1.rel_dom h) hY,
           f.1.mono h subset_rfl Set.inter_subset_right (f.1.rel_dom h) hY'>

theorem sstep_subset {X X' : Set a} {Y Y' : Set b} (hX' : V_0.mem X') (hY' : V_1.mem Y
    ')
    (hX'X : X'  subseteq  X) (hYY' : Y  subseteq  Y') : (sstep X Y : Set (StrictMap
    V_0 V_1))  subseteq  sstep X' Y' := by
  intro f hf
  exact f.1.mono hf hX'X hYY' hX' hY'

/-- **Exercise 5.10 -- the strict function space `(?_0  -> False ?_1)`.** Tokens are
    the strict approximable
maps; neighbourhoods are non-empty finite intersections of step sets. -/
def strictFun (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) :
    NeighborhoodSystem (StrictMap V_0 V_1) where
  mem W := (exists  L : List (Set a  x  Set b), (forall  p  in  L, V_0.mem p.1  /\
    V_1.mem p.2)  /\  W = sstepFun L)
     /\  W.Nonempty
  master := Set.univ
  master_nonempty := <<constMap V_0 V_1.bot, isStrict_constBot>, Set.mem_univ _>
  master_mem := <<[], by simp, sstepFun_nil.symm>,
    <<constMap V_0 V_1.bot, isStrict_constBot>, Set.mem_univ _>>
  inter_mem := by
    rintro W W' Z <<L, hL, rfl>, _> <<L', hL', rfl>, _> <_, hZne> hZsub
    refine <<L ++ L', ?_, (sstepFun_append _ _).symm>, hZne.mono hZsub>
    intro p hp
    rcases List.mem_append.mp hp with h | h
    * exact hL p h
    * exact hL' p h
  sub_master := fun _ => Set.subset_univ _

@[simp] theorem strictFun_master : (strictFun V_0 V_1).master = Set.univ := rfl

theorem sstepFun_up_closed {L : List (Set a  x  Set b)} {f f' : StrictMap V_0 V_1}
    (hf : f  in  sstepFun L) (hff' : f  <=  f') : f'  in  sstepFun L := by
  intro p hp
  exact hff' p.1 p.2 (hf p hp)

/-- A step set is a neighbourhood as soon as it has a (strict) witness. -/
theorem sstep_mem_of_mem {g : StrictMap V_0 V_1} {X : Set a} {Y : Set b} (h : g.1.rel
    X Y) :
    (strictFun V_0 V_1).mem (sstep X Y) := by
  refine <<[(X, Y)], ?_, (sstepFun_singleton X Y).symm>, <g, h>>
  intro p hp; rw [List.mem_singleton] at hp; subst hp
  exact <g.1.rel_dom h, g.1.rel_cod h>

/-- The generation lemma: a filter contains `sstepFun L` iff it contains each step `[
    X_i, Y_i]`. -/
theorem mem_sstepFun_iff (phi : (strictFun V_0 V_1).Element) {L : List (Set a  x  Set
    b)}
    (hL : forall  p  in  L, V_0.mem p.1  /\  V_1.mem p.2) :
    phi.mem (sstepFun L)  <->  forall  p  in  L, phi.mem (sstep p.1 p.2) := by
```

```
  induction L with
  | nil => simp only [sstepFun_nil, List.not_mem_nil, IsEmpty.forall_iff, implies_true
    , iff_true]
           exact phi.master_mem
  | cons p L ih =>
    rw [sstepFun_cons]
    have hLtail : forall  q  in  L, V_0.mem q.1  /\  V_1.mem q.2 :=
      fun q hq => hL q (List.mem_cons.mpr (Or.inr hq))
    constructor
    * intro hmem
      obtain <g, hg> := (phi.sub hmem).2
      have hstep : phi.mem (sstep p.1 p.2) :=
        phi.up_mem hmem (sstep_mem_of_mem (g := g) (hg.1)) Set.inter_subset_left
      have hne : (sstep p.1 p.2  I  sstepFun L).Nonempty := (phi.sub hmem).2
      have htail : phi.mem (sstepFun L) :=
        phi.up_mem hmem <<L, hLtail, rfl>, hne.mono Set.inter_subset_right> Set.
    inter_subset_right
      intro q hq
      rcases List.mem_cons.mp hq with rfl | hq
      * exact hstep
      * exact (ih hLtail).mp htail q hq
    * intro hall
      have hstep : phi.mem (sstep p.1 p.2) := hall p (List.mem_cons.mpr (Or.inl rfl))
      have htail : phi.mem (sstepFun L) :=
        (ih hLtail).mpr (fun q hq => hall q (List.mem_cons.mpr (Or.inr hq)))
      exact phi.inter_mem hstep htail

theorem strictFun_mem_iff {W : Set (StrictMap V_0 V_1)} :
    (strictFun V_0 V_1).mem W  <->
      (exists  L : List (Set a  x  Set b), (forall  p  in  L, V_0.mem p.1  /\  V_1.mem
     p.2)  /\  W = sstepFun L)
         /\  W.Nonempty := Iff.rfl

/-- Intersection of two neighbourhoods, when non-empty, is again one. -/
theorem strictFun_mem_inter {W W' : Set (StrictMap V_0 V_1)}
    (hW : (strictFun V_0 V_1).mem W) (hW' : (strictFun V_0 V_1).mem W') (hne : (W  I
    W').Nonempty) :
    (strictFun V_0 V_1).mem (W  I  W') := by
  obtain <<L, hL, rfl>, _> := hW
  obtain <<L', hL', rfl>, _> := hW'
  refine <<L ++ L', ?_, (sstepFun_append _ _).symm>, hne>
  intro p hp
  rcases List.mem_append.mp hp with h | h
  * exact hL p h
  * exact hL' p h

theorem strictFun_mem_up_closed {W : Set (StrictMap V_0 V_1)} (hW : (strictFun V_0 V_1
    ).mem W)

    {f f' : StrictMap V_0 V_1} (hf : f  in  W) (hff' : f  <=  f') : f'  in  W := by
  obtain <<L, _, rfl>, _> := hW
  exact sstepFun_up_closed hf hff'

/-- **The strict map represented by a filter.** `X (toStrictMap phi) Y  <->  [X, Y]  in
     phi`. It is *strict*
because the step `[Delta_0, Y]` with `Y  !=  Delta_1` is empty (no strict map relates `
```

```
    Delta_0` to a proper output),
hence not a neighbourhood, so it cannot belong to `phi`. -/
def toStrictMap (phi : (strictFun V_0 V_1).Element) : StrictMap V_0 V_1 :=
  <{ rel := fun X Y => phi.mem (sstep X Y)
     rel_dom := by intro X Y h; obtain <f, hf> := (phi.sub h).2; exact f.1.rel_dom hf
     rel_cod := by intro X Y h; obtain <f, hf> := (phi.sub h).2; exact f.1.rel_cod hf
     master_rel := by
       show phi.mem (sstep V_0.master V_1.master); rw [sstep_master_eq]; exact phi.
    master_mem
     inter_right := by
       intro X Y Y' h h'
       obtain <f, hf> := (phi.sub h).2
       obtain <f', hf'> := (phi.sub h').2
       have hY : V_1.mem Y := f.1.rel_cod hf
       have hY' : V_1.mem Y' := f'.1.rel_cod hf'
       show phi.mem (sstep X (Y  I  Y'))
       rw [ <-  sstep_inter_right hY hY']
       exact phi.inter_mem h h'
     mono := by
       intro X X' Y Y' h hX'X hYY' hX' hY'
       obtain <g, hg> := (phi.sub h).2
       have hg' : g.1.rel X' Y' := g.1.mono hg hX'X hYY' hX' hY'
       show phi.mem (sstep X' Y')
       exact phi.up_mem h (sstep_mem_of_mem (g := g) hg') (sstep_subset hX' hY' hX'X
    hYY') },
   by
     intro Y h
     obtain <g, hg> := (phi.sub h).2
     exact g.2 hg>

@[simp] theorem toStrictMap_rel {phi : (strictFun V_0 V_1).Element} {X : Set a} {Y :
    Set b} :
    (toStrictMap phi).1.rel X Y  <->  phi.mem (sstep X Y) := Iff.rfl

/-- **The filter `f-hat = {F ? f  in  F}` of a strict map.** -/
def toStrictFilter (f : StrictMap V_0 V_1) : (strictFun V_0 V_1).Element where
  mem W := (strictFun V_0 V_1).mem W  /\  f  in  W
  sub h := h.1
  master_mem := <(strictFun V_0 V_1).master_mem, Set.mem_univ f>
  inter_mem := by
    rintro W W' <hW, hfW> <hW', hfW'>
    exact <strictFun_mem_inter hW hW' <f, Set.mem_inter hfW hfW'>, Set.mem_inter hfW
    hfW'>
  up_mem := by rintro W W' <hW, hfW> hW' hWW'; exact <hW', hWW' hfW>

@[simp] theorem mem_toStrictFilter {f : StrictMap V_0 V_1} {W : Set (StrictMap V_0 V_1
    )} :
    (toStrictFilter f).mem W  <->  (strictFun V_0 V_1).mem W  /\  f  in  W := Iff.rfl

end

section
variable {a b ? : Type*}
variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
variable {V_2 : NeighborhoodSystem ?}
```

```
/-- The generation lemma for maps into the strict function space: `X h (?_i[Y_i,Z_i])`
    iff
`X h [Y_i,Z_i]` for all `i`. -/
theorem rel_sstepFun_iff (h : ApproximableMap V_0 (strictFun V_1 V_2)) {X : Set a} (hX
     : V_0.mem X)
    {L : List (Set b  x  Set ?)} (hL : forall  p  in  L, V_1.mem p.1  /\  V_2.mem p.2)
     :
    h.rel X (sstepFun L)  <->  forall  p  in  L, h.rel X (sstep p.1 p.2) := by
  induction L with
  | nil =>
    simp only [sstepFun_nil, List.not_mem_nil, IsEmpty.forall_iff, implies_true,
    iff_true]
    show h.rel X (strictFun V_1 V_2).master
    exact h.rel_master hX
  | cons p L ih =>
    rw [sstepFun_cons]
    have hp := hL p (List.mem_cons.mpr (Or.inl rfl))
    have hLtail : forall  q  in  L, V_1.mem q.1  /\  V_2.mem q.2 :=
      fun q hq => hL q (List.mem_cons.mpr (Or.inr hq))
    constructor
    * intro hmem
      obtain <f, hf> := (strictFun_mem_iff.mp (h.rel_cod hmem)).2
      have hstep : h.rel X (sstep p.1 p.2) :=
        h.mono hmem subset_rfl Set.inter_subset_left hX (sstep_mem_of_mem (g := f) (hf
    .1))
      have hne : (sstep p.1 p.2  I  sstepFun L).Nonempty := (h.rel_cod hmem).2
      have htail : h.rel X (sstepFun L) :=
        h.mono hmem subset_rfl Set.inter_subset_right hX
          <<L, hLtail, rfl>, hne.mono Set.inter_subset_right>
      intro q hq
      rcases List.mem_cons.mp hq with rfl | hq
      * exact hstep
      * exact (ih hLtail).mp htail q hq
    * intro hall
      have hstep : h.rel X (sstep p.1 p.2) := hall p (List.mem_cons.mpr (Or.inl rfl))
      have htail : h.rel X (sstepFun L) :=
        (ih hLtail).mpr (fun q hq => hall q (List.mem_cons.mpr (Or.inr hq)))
      exact h.inter_right hstep htail

/-- **The key computation.** For *proper* `X, Y`, `g(<X, Y>?)` contains `Z` iff `g`
    relates the
proper neighbourhood `X  U  Y` to `Z`. (Coarser members of the strict pairing are
    absorbed by
monotonicity; the master member needs only that `X  U  Y  subseteq  Delta_0  U  Delta_1
    `.) -/
theorem smashPair_principal_apply (g : ApproximableMap (smash V_0 V_1) V_2)
    {X : Set a} {Y : Set b} (hX : V_0.mem X) (hXne : X  !=  V_0.master)
    (hY : V_1.mem Y) (hYne : Y  !=  V_1.master) {Z : Set ?} :
    (g.toElementMap (smashPair (V_0.principal hX) (V_1.principal hY))).mem Z
       <->  g.rel (prodNbhd X Y) Z := by
  constructor
  * rintro <W, hW, hrel>
    rcases hW with rfl | <A, B, <_, hXA>, _, <_, hYB>, _, rfl>
    * exact g.mono hrel (prodNbhd_subset_iff.mpr <V_0.sub_master hX, V_1.sub_master hY
    >) subset_rfl
        (smash_mem_proper hX hXne hY hYne) (g.rel_cod hrel)
```

```
    * exact g.mono hrel (prodNbhd_subset_iff.mpr <hXA, hYB>) subset_rfl
        (smash_mem_proper hX hXne hY hYne) (g.rel_cod hrel)
  * intro hrel
    exact <prodNbhd X Y, Or.inr <X, Y, <hX, subset_rfl>, hXne, <hY, subset_rfl>, hYne,
     rfl>, hrel>

/-- The `X`-section of `g : ?_0 ? ?_1  ->  ?_2`, as a map `?_1  ->  ?_2`: `y  |->  g(<X
    , y>?)`. Built with
Exercise 2.8's `ofMono` from its values on principal inputs. -/
def smashSection (g : ApproximableMap (smash V_0 V_1) V_2) {X : Set a} (hX : V_0.mem X
    ) :
    ApproximableMap V_1 V_2 :=
  ofMono (fun Y hY => g.toElementMap (smashPair (V_0.principal hX) (V_1.principal hY))
    )
    (by
      intro Y Y' hY hY' hY'Y
      exact toElementMap_mono g (smashPair_mono le_rfl ((V_1.principal_le_iff hY hY').
    mpr hY'Y)))

/-- The section of a *strict* `g` is itself strict: `g(<X, False>?) = g(False) = False
    `. -/
theorem isStrict_smashSection {g : ApproximableMap (smash V_0 V_1) V_2} (hg : IsStrict
     g)
    {X : Set a} (hX : V_0.mem X) : IsStrict (smashSection g hX) := by
  rw [isStrict_iff_apply_bot, smashSection,
    show (V_1.bot) = V_1.principal V_1.master_mem from rfl,
    toElementMap_ofMono_principal,
    show smashPair (V_0.principal hX) (V_1.principal V_1.master_mem) = (smash V_0 V_1)
    .bot from
      smashPair_bot_right _]
  exact isStrict_iff_apply_bot.mp hg

/-- The section is monotone in the neighbourhood `X` (a smaller input gives a larger
    section). -/
theorem smashSection_mono {g : ApproximableMap (smash V_0 V_1) V_2} {X X' : Set a}
    (hX : V_0.mem X) (hX' : V_0.mem X') (hX'X : X'  subseteq  X) :
    smashSection g hX  <=  smashSection g hX' := by
  intro Y Z hrel
  obtain <hY, hmem> := hrel
  exact <hY, toElementMap_mono g
    (smashPair_mono ((V_0.principal_le_iff hX hX').mpr hX'X) le_rfl) Z hmem>

theorem smashSection_rel {g : ApproximableMap (smash V_0 V_1) V_2} {X : Set a} (hX :
    V_0.mem X)
    {Y : Set b} {Z : Set ?} :
    (smashSection g hX).rel Y Z  <->
      exists  hY : V_1.mem Y, (g.toElementMap (smashPair (V_0.principal hX) (V_1.
    principal hY))).mem Z :=
  Iff.rfl

/-- **Strict curry combinator.** `curryFalse : ((?_0 ? ?_1)  -> False ?_2)  ->  (?_0
    -> False (?_1  -> False ?_2))`, sending `g`
to `x  |->  (y  |->  g(<x, y>?))`. -/
def smashCurryMap (g : StrictMap (smash V_0 V_1) V_2) : StrictMap V_0 (strictFun V_1
    V_2) :=
  <{ rel := fun X N => exists  hX : V_0.mem X, (strictFun V_1 V_2).mem N  /\
```

```
      (<smashSection g.1 hX, isStrict_smashSection g.2 hX> : StrictMap V_1 V_2)  in
    N
     rel_dom := fun <hX, _> => hX
     rel_cod := fun <_, hN, _> => hN
     master_rel := <V_0.master_mem, (strictFun V_1 V_2).master_mem, Set.mem_univ _>
     inter_right := by
       rintro X N N' <hX, hN, hmem> <_, hN', hmem'>
       exact <hX, strictFun_mem_inter hN hN' <_, hmem, hmem'>, Set.mem_inter hmem hmem
    '>
     mono := by
       rintro X X' N N' <hX, hN, hmem> hX'X hNN' hX' hN'
       refine <hX', hN', strictFun_mem_up_closed hN' (hNN' hmem) ?_>
       exact Subtype.coe_le_coe.mp (smashSection_mono hX hX' hX'X) },
  by
    rintro N <hDelta_0, hN, hmem>
    obtain <<L, hL, rfl>, _> := hN
    have hall : forall  p  in  L, p.2 = V_2.master := by
      intro p hp
      have hpr : (smashSection g.1 hDelta_0).rel p.1 p.2 := hmem p hp
      rw [smashSection_rel] at hpr
      obtain <hp1, hpmem> := hpr
      have hbot : smashPair (V_0.principal hDelta_0) (V_1.principal hp1) = (smash V_0
    V_1).bot := by
        rw [principal_master_eq_bot hDelta_0 rfl]; exact smashPair_bot_left _
      rw [hbot, isStrict_iff_apply_bot.mp g.2, NeighborhoodSystem.mem_bot] at hpmem
      exact hpmem
    apply Set.eq_univ_of_forall
    intro f q hq
    rw [hall q hq]
    exact (f.1).rel_master (hL q hq).1>

/-- Every smash neighbourhood is a product neighbourhood `A  U  B` (the master is `
    Delta_0  U  Delta_1`). -/
theorem smash_mem_prodNbhd_form {W : Set (a ? b)} (hW : (smash V_0 V_1).mem W) :
    exists  A B, V_0.mem A  /\  V_1.mem B  /\  W = prodNbhd A B := by
  rcases hW with rfl | <X, Y, hX, _, hY, _, rfl>
  * exact <_, _, V_0.master_mem, V_1.master_mem, rfl>
  * exact <X, Y, hX, hY, rfl>

/-- **Strict uncurry combinator.** `uncurryFalse : (?_0  -> False (?_1  -> False ?_2))
     ->  ((?_0 ? ?_1)  -> False ?_2)`,
`X  U  Y (uncurryFalse h) Z  <->  X h [Y, Z]`. -/
def smashUncurryMap (h : StrictMap V_0 (strictFun V_1 V_2)) : StrictMap (smash V_0 V_1
    ) V_2 :=
  <{ rel := fun W Z => (smash V_0 V_1).mem W  /\  h.1.rel (Sum.inl ^-1' W) (sstep (Sum
    .inr ^-1' W) Z)
     rel_dom := fun hh => hh.1
     rel_cod := by
       rintro W Z <_, hrel>
       obtain <f, hf> := (strictFun_mem_iff.mp (h.1.rel_cod hrel)).2
       exact f.1.rel_cod hf
     master_rel := by
       refine <(smash V_0 V_1).master_mem, ?_>
       rw [show (smash V_0 V_1).master = prodNbhd V_0.master V_1.master from rfl,
         inl_preimage_prodNbhd, inr_preimage_prodNbhd, sstep_master_eq]
       exact h.1.master_rel
```

```
     inter_right := by
       rintro W Z Z' <hW, hrel> <_, hrel'>
       obtain <f, hf> := (strictFun_mem_iff.mp (h.1.rel_cod hrel)).2
       obtain <f', hf'> := (strictFun_mem_iff.mp (h.1.rel_cod hrel')).2
       refine <hW, ?_>
       rw [ <-  sstep_inter_right (f.1.rel_cod hf) (f'.1.rel_cod hf')]
       exact h.1.inter_right hrel hrel'
     mono := by
       rintro W W_2 Z Z' <_, hrel> hW_2W hZZ' hW_2 hZ'
       have hinl : Sum.inl ^-1' W_2  subseteq  Sum.inl ^-1' W := Set.preimage_mono
    hW_2W
       have hinr : Sum.inr ^-1' W_2  subseteq  Sum.inr ^-1' W := Set.preimage_mono
    hW_2W
       obtain <A, B, hA, hB, rfl> := smash_mem_prodNbhd_form hW_2
       refine <hW_2, ?_>
       rw [inl_preimage_prodNbhd] at hinl
       rw [inr_preimage_prodNbhd] at hinr
       rw [inl_preimage_prodNbhd, inr_preimage_prodNbhd]
       obtain <f, hf> := (strictFun_mem_iff.mp (h.1.rel_cod hrel)).2
       have hfB : f.1.rel B Z' := f.1.mono hf hinr hZZ' hB hZ'
       exact h.1.mono hrel hinl (sstep_subset hB hZ' hinr hZZ') hA
         (sstep_mem_of_mem (g := f) hfB) },
   by
     rintro Z <_, hrel>
     rw [show (smash V_0 V_1).master = prodNbhd V_0.master V_1.master from rfl,
       inl_preimage_prodNbhd, inr_preimage_prodNbhd] at hrel
     have huniv : sstep V_1.master Z = (Set.univ : Set (StrictMap V_1 V_2)) := h.2
    hrel
     have hcb : (<constMap V_1 V_2.bot, isStrict_constBot> : StrictMap V_1 V_2)  in
    sstep V_1.master Z := by
       rw [huniv]; exact Set.mem_univ _
     obtain <_, hZ> := hcb
     rwa [NeighborhoodSystem.mem_bot] at hZ>

/-- **Roundtrip (i): `uncurryFalse  comp  curryFalse = id`.** -/
theorem smashUncurry_curry (g : StrictMap (smash V_0 V_1) V_2) :
    smashUncurryMap (smashCurryMap g) = g := by
  apply Subtype.ext
  apply ApproximableMap.ext
  intro W Z
  constructor
  * rintro <hW, hcurry>
    rcases hW with rfl | <X, Y, hXp, hXne, hYp, hYne, rfl>
    * simp only [inl_preimage_prodNbhd, inr_preimage_prodNbhd] at hcurry
      obtain <hXm, _, hsec> := hcurry
      have hZ : Z = V_2.master := isStrict_smashSection g.2 hXm hsec
      rw [hZ]; exact g.1.master_rel
    * simp only [inl_preimage_prodNbhd, inr_preimage_prodNbhd] at hcurry
      obtain <hX, _, hY, hmem> := hcurry
      exact (smashPair_principal_apply g.1 hXp hXne hYp hYne).mp hmem
  * intro hrel
    have hW : (smash V_0 V_1).mem W := g.1.rel_dom hrel
    rcases hW with rfl | <X, Y, hXp, hXne, hYp, hYne, rfl>
    * have hZ : Z = V_2.master := g.2 hrel
      subst hZ
      refine <Or.inl rfl, ?_>
```

```
      simp only [inl_preimage_prodNbhd, inr_preimage_prodNbhd, sstep_master_eq]
      exact <V_0.master_mem, (strictFun V_1 V_2).master_mem, Set.mem_univ _>
    * refine <smash_mem_proper hXp hXne hYp hYne, ?_>
      simp only [inl_preimage_prodNbhd, inr_preimage_prodNbhd]
      have hsec : (smashSection g.1 hXp).rel Y Z :=
        <hYp, (smashPair_principal_apply g.1 hXp hXne hYp hYne).mpr hrel>
      exact <hXp, sstep_mem_of_mem
        (g := <smashSection g.1 hXp, isStrict_smashSection g.2 hXp>) hsec, hsec>

/-- A step with master codomain is everything: every strict map relates `Y` to `Delta_1
    `. -/
theorem sstep_cod_master {Y : Set a} (hY : V_0.mem Y) :
    (sstep Y V_1.master : Set (StrictMap V_0 V_1)) = Set.univ := by
  ext f
  simp only [mem_sstep, Set.mem_univ, iff_true]
  exact f.1.mono f.1.master_rel (V_0.sub_master hY) subset_rfl hY V_1.master_mem

/-- **The decisive computation for the adjunction.** The `X`-section of `uncurryFalse h
    ` evaluated on a
neighbourhood `Y` is exactly the strict-function value `X h [Y, Z]`. At the boundary (a
     master
factor) both sides collapse via strictness; off the boundary it is `
    smashPair_principal_apply`. -/
theorem section_uncurry_rel (h : StrictMap V_0 (strictFun V_1 V_2))
    {X : Set a} (hX : V_0.mem X) {Y : Set b} (hY : V_1.mem Y) {Z : Set ?} :
    (smashSection (smashUncurryMap h).1 hX).rel Y Z  <->  h.1.rel X (sstep Y Z) := by
  rw [smashSection_rel]
  by_cases hXm : X = V_0.master
  * subst hXm
    constructor
    * rintro <hY', hmem>
      rw [principal_master_eq_bot hX rfl,
        show smashPair V_0.bot (V_1.principal hY') = (smash V_0 V_1).bot from
          smashPair_eq_bot_iff.mpr (Or.inl rfl),
        isStrict_iff_apply_bot.mp (smashUncurryMap h).2, NeighborhoodSystem.mem_bot]
    at hmem
      subst hmem
      rw [sstep_cod_master hY]
      exact h.1.rel_master hX
    * intro hrel
      have huniv : sstep Y Z = (Set.univ : Set (StrictMap V_1 V_2)) := h.2 hrel
      have hZ : Z = V_2.master := by
        have hcb : (<constMap V_1 V_2.bot, isStrict_constBot> : StrictMap V_1 V_2)  in
      sstep Y Z := by
          rw [huniv]; exact Set.mem_univ _
        obtain <_, hbz> := hcb
        rwa [NeighborhoodSystem.mem_bot] at hbz
      subst hZ
      exact <hY, ((smashUncurryMap h).1.toElementMap _).master_mem>
  * by_cases hYm : Y = V_1.master
    * subst hYm
      constructor
      * rintro <hY', hmem>
        rw [principal_master_eq_bot hY' rfl,
          show smashPair (V_0.principal hX) V_1.bot = (smash V_0 V_1).bot from
            smashPair_eq_bot_iff.mpr (Or.inr rfl),
```

```
          isStrict_iff_apply_bot.mp (smashUncurryMap h).2, NeighborhoodSystem.mem_bot]
     at hmem
        subst hmem
        rw [sstep_master_eq]
        exact h.1.rel_master hX
      * intro hrel
        obtain <f, hf> := (strictFun_mem_iff.mp (h.1.rel_cod hrel)).2
        have hZ : Z = V_2.master := f.2 hf
        subst hZ
        exact <hY, ((smashUncurryMap h).1.toElementMap _).master_mem>
    * constructor
      * rintro <hY', hmem>
        obtain <_, hrel> :=
          (smashPair_principal_apply (smashUncurryMap h).1 hX hXm hY hYm).mp hmem
        simpa only [inl_preimage_prodNbhd, inr_preimage_prodNbhd] using hrel
      * intro hrel
        refine <hY, (smashPair_principal_apply (smashUncurryMap h).1 hX hXm hY hYm).
    mpr ?_>
        refine <smash_mem_proper hX hXm hY hYm, ?_>
        simpa only [inl_preimage_prodNbhd, inr_preimage_prodNbhd] using hrel

/-- **Roundtrip (ii): `curryFalse  comp  uncurryFalse = id`.** -/
theorem smashCurry_uncurry (h : StrictMap V_0 (strictFun V_1 V_2)) :
    smashCurryMap (smashUncurryMap h) = h := by
  apply Subtype.ext
  apply ApproximableMap.ext
  intro X N
  constructor
  * rintro <hX, hN, hmem>
    obtain <<L, hL, rfl>, _> := hN
    refine (rel_sstepFun_iff h.1 hX hL).mpr (fun p hp => ?_)
    exact (section_uncurry_rel h hX (hL p hp).1).mp (hmem p hp)
  * intro hrel
    have hX : V_0.mem X := h.1.rel_dom hrel
    have hN : (strictFun V_1 V_2).mem N := h.1.rel_cod hrel
    refine <hX, hN, ?_>
    obtain <<L, hL, rfl>, _> := hN
    intro p hp
    exact (section_uncurry_rel h hX (hL p hp).1).mpr
      ((rel_sstepFun_iff h.1 hX hL).mp hrel p hp)

/-- **Exercise 5.10 (Scott 1981, PRG-19) -- the adjunction.** The strict currying
    combinator is an
order isomorphism `((?_0 ? ?_1)  -> False ?_2) ? (?_0  -> False (?_1  -> False ?_2))`:
    the smash product is left adjoint to
the strict function space. -/
def smashCurryEquiv (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b)
    (V_2 : NeighborhoodSystem ?) :
    StrictMap (smash V_0 V_1) V_2 ?o StrictMap V_0 (strictFun V_1 V_2) where
  toFun := smashCurryMap
  invFun := smashUncurryMap
  left_inv := smashUncurry_curry
  right_inv := smashCurry_uncurry
  map_rel_iff' := by
    intro g g'
    constructor
```

```
    * intro hcurry W Z hrel
      have h1 : (smashCurryMap g).1.rel (Sum.inl ^-1' W) (sstep (Sum.inr ^-1' W) Z) :=
     by
        have hu : (smashUncurryMap (smashCurryMap g)).1.rel W Z := by
          rw [smashUncurry_curry]; exact hrel
        exact hu.2
      have h2 := hcurry _ _ h1
      have hu' : (smashUncurryMap (smashCurryMap g')).1.rel W Z := <g.1.rel_dom hrel,
    h2>
      rw [smashUncurry_curry] at hu'
      exact hu'
    * intro hg X N hrel
      obtain <hX, hN, hmem> := hrel
      refine <hX, hN, ?_>
      obtain <<L, hL, rfl>, _> := hN
      intro p hp
      obtain <hY, hmm> := hmem p hp
      obtain <W, hWmem, hWrel> := hmm
      exact <hY, W, hWmem, hg W p.2 hWrel>

end

section
variable {a b ? : Type*}
variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}

/-- **Exercise 5.10 -- the strict function space is complete.** `|?_0  -> False ?_1|`
    is order-isomorphic to
the strict approximable maps `?_0  ->  ?_1`. -/
def strictFunEquiv (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) :
    (strictFun V_0 V_1).Element ?o StrictMap V_0 V_1 where
  toFun := toStrictMap
  invFun := toStrictFilter
  left_inv phi := by
    apply Element.ext
    intro W
    constructor
    * rintro <hWmem, hfW>
      obtain <<L, hL, rfl>, _> := hWmem
      exact (mem_sstepFun_iff phi hL).mpr (fun p hp => hfW p hp)
    * intro hW
      refine <phi.sub hW, ?_>
      obtain <<L, hL, rfl>, _> := phi.sub hW
      intro p hp
      exact (mem_sstepFun_iff phi hL).mp hW p hp
  right_inv f := by
    apply Subtype.ext
    apply ApproximableMap.ext
    intro X Y
    constructor
    * rintro <_, hf>; exact hf
    * intro hf; exact <sstep_mem_of_mem (g := f) hf, hf>
  map_rel_iff' := by
    intro phi phi'
    constructor
```

```
    * intro h W hW
      obtain <<L, hL, rfl>, _> := phi.sub hW
      refine (mem_sstepFun_iff phi' hL).mpr (fun p hp => ?_)
      exact h p.1 p.2 ((mem_sstepFun_iff phi hL).mp hW p hp)
    * intro h X Y hrel
      exact h _ hrel

/-- The identity is strict. -/
theorem isStrict_idMap : IsStrict (idMap V_0) := by
  rintro Y <_, hY, hsub>
  exact Set.Subset.antisymm (V_0.sub_master hY) hsub

end

section
variable {a b ? : Type*}
variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}

theorem smash_mem_iff {W : Set (a ? b)} :
    (smash V_0 V_1).mem W  <->
      W = prodNbhd V_0.master V_1.master  \/  SmashProper V_0 V_1 W := Iff.rfl
end

end DomainTheory.NeighbourhoodSystems.Exercise510
end
```

3.6.12 Exercise 5.11 — stack combinators

For any domain $\mathcal{D}$ we may regard $\mathcal{D}^\infty$ as consisting of (bottomless) *stacks* of elements of $\mathcal{D}$. With this image in mind, define appropriate combinators with the obvious meanings:

$$\mathrm{head} : \mathcal{D}^\infty \to \mathcal{D}; \quad \mathrm{tail} : \mathcal{D}^\infty \to \mathcal{D}^\infty; \quad \mathrm{push} : \mathcal{D} \times \mathcal{D}^\infty \to \mathcal{D}^\infty.$$

Using the fixed-point theorem argue that there is a combinator

$$\mathrm{diag} : \mathcal{D} \to \mathcal{D}^\infty$$

where for all $x \in |\mathcal{D}|$ we have

$$\mathrm{diag}(x) = \langle x \rangle_{n=0}^{\infty}.$$

(Hint: Try a recursive definition, say

$$\mathrm{diag}(x) = \mathrm{push}(x, \mathrm{diag}(x)),$$

but be sure to prove *all* terms of $\mathrm{diag}(x)$ equal x.) Also introduce by an appropriate recursion a combinator

$$\mathrm{map} : (D \to D)^\infty \times D \to D^\infty$$

where for elements of the suitable types:

$$\mathrm{map}(\langle f_n \rangle_{n=0}^{\infty}, x) = \langle f_n(x) \rangle_{n=0}^{\infty}.$$

Exercise_5_11.lean

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Exercise_4_25
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Theorem_4_2
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Theorem_2_7
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_12
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Exercise_3_16
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Exercise_3_14

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.Exercise511
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap

section
variable {a : Type*} (V : NeighborhoodSystem a)

/-- `fold : D  x  D^ inf   ->  D^ inf `, the inverse direction of `iterProdIso`. -/
def fold : ApproximableMap (prod V (iterSys V)) (iterSys V) := ofIso (iterProdIso V).
    symm

/-- `iterProdIso` reads a stack as its head together with the tail sequence. -/
theorem iterProdIso_apply (z : (iterSys V).Element) :
    iterProdIso V z = pair (component z 0) (ofSeq (fun n => component z (n + 1))) :=
    rfl

/-- `push : D  x  D^ inf   ->  D^ inf `, prepend an element to a stack. -/
def push : ApproximableMap (prod V (iterSys V)) (iterSys V) := fold V

/-- `unfold : D^ inf   ->  D  x  D^ inf `, the forward direction of `iterProdIso`. -/
def unfold : ApproximableMap (iterSys V) (prod V (iterSys V)) := ofIso (iterProdIso V)

/-- `head : D^ inf   ->  D`, the top of the stack. -/
def head : ApproximableMap (iterSys V) V := (proj_0 V (iterSys V)).comp (unfold V)

@[simp] theorem head_apply (z : (iterSys V).Element) :
    (head V).toElementMap z = component z 0 := by
  rw [head, toElementMap_comp, unfold, toElementMap_ofIso, toElementMap_proj_0,
    iterProdIso_apply, fst_pair]

/-- `tail : D^ inf   ->  D^ inf `, the stack with its top removed. -/
def tail : ApproximableMap (iterSys V) (iterSys V) := (proj_1 V (iterSys V)).comp (
    unfold V)

@[simp] theorem tail_apply (z : (iterSys V).Element) :
    (tail V).toElementMap z = ofSeq (fun n => component z (n + 1)) := by
  rw [tail, toElementMap_comp, unfold, toElementMap_ofIso, toElementMap_proj_1,
    iterProdIso_apply, snd_pair]

/-- The argument domain of `map`: a stack of functions paired with a point, `(D  ->  D)
    ^ inf   x  D`. -/
```

```
abbrev Arg (V : NeighborhoodSystem a) := prod (iterSys (funSpace V V)) V

/-- `diag : D  ->  D^ inf `, defined as the least fixed point of `s  |->  push(x, s)`.
    -/
def diag : ApproximableMap V (iterSys V) :=
  (fixMap (iterSys V)).comp (curry (push V))

/-- The body of the `map` recursion as a two-argument map
`M(W, <s, x>) = push((head s)(x), W(tail s, x))`, from which `map = fix(curry M)`. -/
def mapBody : ApproximableMap (prod (funSpace (Arg V) (iterSys V)) (Arg V)) (iterSys V
    ) :=
  (push V).comp
    (paired
      ((evalMap V V).comp
        (paired
          ((head (funSpace V V)).comp ((proj_0 (iterSys (funSpace V V)) V).comp
            (proj_1 (funSpace (Arg V) (iterSys V)) (Arg V))))
          ((proj_1 (iterSys (funSpace V V)) V).comp
            (proj_1 (funSpace (Arg V) (iterSys V)) (Arg V)))))
      ((evalMap (Arg V) (iterSys V)).comp
        (paired (proj_0 (funSpace (Arg V) (iterSys V)) (Arg V))
          (paired
            ((tail (funSpace V V)).comp ((proj_0 (iterSys (funSpace V V)) V).comp
              (proj_1 (funSpace (Arg V) (iterSys V)) (Arg V))))
            ((proj_1 (iterSys (funSpace V V)) V).comp
              (proj_1 (funSpace (Arg V) (iterSys V)) (Arg V)))))))

/-- The `map` recursion operator `R(W) = fun <s, x>. push((head s)(x), W(tail s, x))`.
    -/
def mapOp : ApproximableMap (funSpace (Arg V) (iterSys V)) (funSpace (Arg V) (iterSys
    V)) :=
  curry (mapBody V)

/-- `map : (D  ->  D)^ inf   x  D  ->  D^ inf `, the least fixed point of `mapOp`. -/
def mapMap : ApproximableMap (Arg V) (iterSys V) := toApproxMap (mapOp V).fixElement

/-- `(tail z)_n = z_n_+_1`. -/
theorem component_tail (z : (iterSys V).Element) (n : Nat) :
    component ((tail V).toElementMap z) n = component z (n + 1) := by
  rw [tail_apply, component_ofSeq]
end
end DomainTheory.NeighbourhoodSystems.Exercise511

section
namespace DomainTheory.NeighbourhoodSystems.Exercise511
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap
section
variable {a : Type*} (V : NeighborhoodSystem a)

/- concordance: source L3031 EXERCISE 5.11 stack combinators -/
/-- **Exercise 5.11 (Scott 1981, PRG-19).** The recursion `diag(x) = push(x, diag(x))`.
     -/
theorem diag_rec (x : V.Element) :
    (diag V).toElementMap x =
      (push V).toElementMap (pair x ((diag V).toElementMap x)) := by
  rw [diag, toElementMap_comp]
```

```
  set phi := (curry (push V)).toElementMap x with hphi
  have hfix := fixMap_fixed (iterSys V) phi
  have hval := toElementMap_curry_apply (push V) x ((fixMap (iterSys V)).toElementMap
    phi)
  rw [ <-  hphi] at hval
  rw [hval] at hfix
  exact hfix.symm

theorem fold_unfold (z : (iterSys V).Element) :
    (fold V).toElementMap ((unfold V).toElementMap z) = z := by
  rw [unfold, fold, toElementMap_ofIso, toElementMap_ofIso, OrderIso.symm_apply_apply]

theorem mapBody_apply (W : (funSpace (Arg V) (iterSys V)).Element)
    (s : (iterSys (funSpace V V)).Element) (x : V.Element) :
    (mapBody V).toElementMap (pair W (pair s x)) =
      (push V).toElementMap
        (pair ((toApproxMap ((head (funSpace V V)).toElementMap s)).toElementMap x)
          ((toApproxMap W).toElementMap (pair ((tail (funSpace V V)).toElementMap s) x
   ))) := by
  rw [mapBody, toElementMap_comp]
  simp only [toElementMap_paired, toElementMap_comp, toElementMap_proj_0,
    toElementMap_proj_1,
    fst_pair, snd_pair, evalMap_apply]

/-- **Exercise 5.11 (Scott 1981, PRG-19).** The recursion defining `map`:
`map(s, x) = push((head s)(x), map(tail s, x))`. -/
theorem map_rec (s : (iterSys (funSpace V V)).Element) (x : V.Element) :
    (mapMap V).toElementMap (pair s x) =
      (push V).toElementMap
        (pair ((toApproxMap ((head (funSpace V V)).toElementMap s)).toElementMap x)
          ((mapMap V).toElementMap (pair ((tail (funSpace V V)).toElementMap s) x)))
    := by
  have hfix : (mapOp V).toElementMap (mapOp V).fixElement = (mapOp V).fixElement :=
    toElementMap_fixElement (mapOp V)
  have hval := toElementMap_curry_apply (mapBody V) (mapOp V).fixElement (pair s x)
  rw [ <-  mapOp, hfix] at hval
  rw [mapMap, hval, mapBody_apply]

/-- **Stack law.** `push(head z, tail z) = z` (a stack is its top pushed onto its tail)
    . -/
theorem push_head_tail (z : (iterSys V).Element) :
    (push V).toElementMap (pair ((head V).toElementMap z) ((tail V).toElementMap z)) =
     z := by
  rw [head, tail, toElementMap_comp, toElementMap_comp, toElementMap_proj_0,
    toElementMap_proj_1,
    pair_fst_snd, push, fold_unfold]

theorem unfold_fold (p : (prod V (iterSys V)).Element) :
    (unfold V).toElementMap ((fold V).toElementMap p) = p := by
  rw [unfold, fold, toElementMap_ofIso, toElementMap_ofIso, OrderIso.apply_symm_apply]

/-- **Stack law.** `head(push(x, s)) = x`. -/
theorem head_push (x : V.Element) (s : (iterSys V).Element) :
    (head V).toElementMap ((push V).toElementMap (pair x s)) = x := by
  rw [head, toElementMap_comp, push, unfold_fold, toElementMap_proj_0, fst_pair]
```

```
/-- **Stack law.** `tail(push(x, s)) = s`. -/
theorem tail_push (x : V.Element) (s : (iterSys V).Element) :
    (tail V).toElementMap ((push V).toElementMap (pair x s)) = s := by
  rw [tail, toElementMap_comp, push, unfold_fold, toElementMap_proj_1, snd_pair]

/-- **Exercise 5.11 (Scott 1981, PRG-19).** Scott's emphasised property: *all*
    components of
`diag(x)` equal `x`, i.e. `diag(x) = <x>_n`. -/
theorem component_diag (x : V.Element) (n : Nat) :
    component ((diag V).toElementMap x) n = x := by
  induction n with
  | zero =>
    rw [ <-  head_apply, diag_rec, head_push]
  | succ k ih =>
    have ht : (tail V).toElementMap ((diag V).toElementMap x) = (diag V).toElementMap
    x := by
      conv_lhs => rw [diag_rec]
      rw [tail_push]
    rw [ <-  component_tail, ht, ih]

/-- **Exercise 5.11 (Scott 1981, PRG-19).** The `n`-th component of `map(s, x)` is `(
    component s n)(x)`,
i.e. `map(<f_n>, x) = <f_n(x)>_n`. -/
theorem component_map (x : V.Element) :
    forall  (n : Nat) (s : (iterSys (funSpace V V)).Element),
      component ((mapMap V).toElementMap (pair s x)) n =
        (toApproxMap (component s n)).toElementMap x := by
  intro n
  induction n with
  | zero =>
    intro s
    rw [ <-  head_apply, map_rec, head_push, head_apply]
  | succ k ih =>
    intro s
    have ht : (tail V).toElementMap ((mapMap V).toElementMap (pair s x)) =
        (mapMap V).toElementMap (pair ((tail (funSpace V V)).toElementMap s) x) := by
      conv_lhs => rw [map_rec]
      rw [tail_push]
    rw [ <-  component_tail, ht, ih, component_tail]

end

end DomainTheory.NeighbourhoodSystems.Exercise511
end
```

3.6.13 Exercise 5.12 — while combinator

On any domain D introduce (as a least fixed point) a combinator

$$\text{while} : (D \to T) \times (D \to D) \to (D \to D)$$

by the recursion

$$\text{while}(p, f)(x) = \text{cond}(p(x), \text{while}(p, f)(f(x)), x).$$

Prove that

$$\text{while}(p, \text{while}(p, f)) = \text{while}(p, f).$$

Show how while could have been used to obtain the least number operator mentioned in the proof of 5.6. Generalize the idea to define a combinator

$$\text{find} : D^\infty \times (D \to T) \to D$$

with the meaning "find the first term of the sequence (if any) which satisfies the given precicate."

Exercise_5_12.lean

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Theorem_4_1
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Theorem_4_2
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_12

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.Exercise512
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap
open DomainTheory.NeighbourhoodSystems.Exercise326 (cond TD)

section
variable {a : Type*} (V : NeighborhoodSystem a)
  (p : ApproximableMap V TD) (f : ApproximableMap V V)

/-- The body of the loop as a two-argument map `M : (D  ->  D)  x  D  ->  D`,
`M(w, x) = cond(p(x), w(f(x)), x)`. It is assembled from `cond`, `eval`, the
    projections,
pairing and composition, hence approximable. -/
def bodyMap : ApproximableMap (prod (funSpace V V) V) V :=
  (cond V).comp
    (paired (p.comp (proj_1 (funSpace V V) V))
      (paired
        ((evalMap V V).comp
          (paired (proj_0 (funSpace V V) V) (f.comp (proj_1 (funSpace V V) V))))
        (proj_1 (funSpace V V) V)))

/-- The loop operator `Wop : (D  ->  D)  ->  (D  ->  D)`, `Wop(w) = fun x. cond(p(x), w
    (f(x)), x)`. -/
def Wop : ApproximableMap (funSpace V V) (funSpace V V) := curry (bodyMap V p f)

/-- The `while`-loop element of the function space `D  ->  D`: the least fixed point of
     `Wop`. -/
def whileFix : (funSpace V V).Element := (Wop V p f).fixElement

/-- The approximable function `while p do f : D  ->  D` denoted by `whileFix`. -/
def whileMap : ApproximableMap V V := toApproxMap (whileFix V p f)

/-- Value of the loop body: `M(w, x) = cond(p(x), w(f(x)), x)`. -/
```

```
theorem bodyMap_apply (w : (funSpace V V).Element) (x : V.Element) :
    (bodyMap V p f).toElementMap (pair w x) =
      (cond V).toElementMap
        (pair (p.toElementMap x)
          (pair ((toApproxMap w).toElementMap (f.toElementMap x)) x)) := by
  rw [bodyMap, toElementMap_comp]
  simp only [toElementMap_paired, toElementMap_comp, toElementMap_proj_0,
    toElementMap_proj_1,
    fst_pair, snd_pair, evalMap_apply]

/-- `Wop(w)(x) = cond(p(x), w(f(x)), x)`. -/
theorem Wop_apply (w : (funSpace V V).Element) (x : V.Element) :
    (toApproxMap ((Wop V p f).toElementMap w)).toElementMap x =
      (cond V).toElementMap
        (pair (p.toElementMap x)
          (pair ((toApproxMap w).toElementMap (f.toElementMap x)) x)) := by
  rw [Wop, toElementMap_curry_apply, bodyMap_apply]

/-- **Leastness.** Any `w'` satisfying the `while`-recursion `w'(x) = cond(p(x), w'(f(x
    )), x)`
dominates the least solution `whileMap`. -/
theorem whileMap_least (w' : (funSpace V V).Element)
    (hw' : forall  x : V.Element, (toApproxMap w').toElementMap x =
      (cond V).toElementMap
        (pair (p.toElementMap x)
          (pair ((toApproxMap w').toElementMap (f.toElementMap x)) x))) :
    whileFix V p f  <=  w' := by
  apply fixElement_le_of_toElementMap_le
  apply le_of_eq
  apply (funSpaceEquiv V V).injective
  rw [funSpaceEquiv_apply, funSpaceEquiv_apply]
  apply ext_of_toElementMap
  intro x
  rw [Wop_apply]
  exact (hw' x).symm
end
end DomainTheory.NeighbourhoodSystems.Exercise512

section
namespace DomainTheory.NeighbourhoodSystems.Exercise512
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap
open DomainTheory.NeighbourhoodSystems.Exercise326 (cond TD)
section
variable {a : Type*} (V : NeighborhoodSystem a)
  (p : ApproximableMap V TD) (f : ApproximableMap V V)
/- concordance: source L3071 EXERCISE 5.12 while combinator -/
/-- **Exercise 5.12 (Scott 1981, PRG-19).** The defining recursion of the `while`-loop:
`w(x) = cond(p(x), w(f(x)), x)`. -/
theorem whileMap_rec (x : V.Element) :
    (whileMap V p f).toElementMap x =
      (cond V).toElementMap
        (pair (p.toElementMap x)
          (pair ((whileMap V p f).toElementMap (f.toElementMap x)) x)) := by
  have hfix : (Wop V p f).toElementMap (whileFix V p f) = whileFix V p f :=
    toElementMap_fixElement (Wop V p f)
  have := Wop_apply V p f (whileFix V p f) x
```

```
  rw [hfix] at this
  exact this
end

section
variable {a : Type*} (V : NeighborhoodSystem a)
  (p : ApproximableMap V TD) (f : ApproximableMap V V)

/-- If the test diverges (`p(x) = False`), the loop diverges: `w(x) = False`. -/
theorem whileMap_bot {x : V.Element} (hx : p.toElementMap x = Example23.botElt) :
    (whileMap V p f).toElementMap x = V.bot := by
  rw [whileMap_rec, hx, Exercise326.cond_bot]

/-- If the test fails (`p(x) = false`), the loop exits: `w(x) = x`. -/
theorem whileMap_false {x : V.Element} (hx : p.toElementMap x = Example23.falseElt) :
    (whileMap V p f).toElementMap x = x := by
  rw [whileMap_rec, hx, Exercise326.cond_false]

/-- If the test holds (`p(x) = true`), the loop runs the body: `w(x) = w(f(x))`. -/
theorem whileMap_true {x : V.Element} (hx : p.toElementMap x = Example23.trueElt) :
    (whileMap V p f).toElementMap x = (whileMap V p f).toElementMap (f.toElementMap x)
     := by
  rw [whileMap_rec, hx, Exercise326.cond_true]

end

end DomainTheory.NeighbourhoodSystems.Exercise512
end
```

3.6.14 Exercise 5.13 — pairing function on numbers

Prove the existence of a one-one function $\mathrm{num} : \mathbf{N} \times \mathbf{N} \to \mathbf{N}$ such that

$$\mathrm{num}(0,0) = 0;$$

$$\mathrm{num}(n, m+1) = \mathrm{num}(n+1, m) + 1;$$

$$\mathrm{num}(n+1, 0) = \mathrm{num}(0, n) + 1.$$

Draw a picture (i.e. an infinite matrix) for the function and find a closed form for its values, if possible. Use the function to prove the isomorphism of the domains

$$P\mathbf{N},\ P(\mathbf{N} \times \mathbf{N}),\ P\mathbf{N} \times P\mathbf{N}.$$

Exercise_5_13.lean

```
module

public import Aesop.BuiltinRules
public import Mathlib.Algebra.Ring.Parity
public import Mathlib.Data.Nat.Basic
```

```
public import Mathlib.Tactic.NormNum
public import Mathlib.Tactic.Ring
public import Mathlib.Tactic.ToDual
public import Mathlib.Tactic.ToAdditive

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.Exercise513
section
/-- One step of the diagonal walk: the cell holding `num c + 1`. Moving up-right within
    a diagonal,
or to the start of the next diagonal at the top edge. -/
def nextCell : Nat  x  Nat  ->  Nat  x  Nat
  | (n + 1, m) => (n, m + 1)
  | (0, m) => (m + 1, 0)
end

section
/-- The `k`-th triangular number `T(k) = k(k+1)/2`. -/
def tri (k : Nat) : Nat := k * (k + 1) / 2
end

section
/-- The pairing function `num n m = (n+m)(n+m+1)/2 + m` (Cantor's diagonal enumeration)
    . -/
def num (n m : Nat) : Nat := tri (n + m) + m
end

section
/-- Its uncurried form, the actual `N  x  N  ->  N` of the exercise. -/
def numP (p : Nat  x  Nat) : Nat := num p.1 p.2
end

section
theorem num_succ_right (n m : Nat) : num n (m + 1) = num (n + 1) m + 1 := by
  unfold num
  have h : n + (m + 1) = (n + 1) + m := by omega
  rw [h]; omega
end

section
theorem num_zero_zero : num 0 0 = 0 := rfl
end

section
/-- `k(k+1)` is even (choice-free, by induction). -/
theorem two_dvd_mul_succ (k : Nat) : 2 ? k * (k + 1) := by
  induction k with
  | zero => exact <0, by ring>
  | succ n ih =>
      obtain <c, hc> := ih
      refine <c + (n + 1), ?_>
      have hexp : (n + 1) * (n + 1 + 1) = n * (n + 1) + 2 * (n + 1) := by ring
      rw [hexp, hc]; ring
end
```

```
section
/-- The defining doubling identity `2*T(k) = k(k+1)` -- the division is exact because `
    k(k+1)` is
even. -/
theorem tri_mul_two (k : Nat) : tri k * 2 = k * (k + 1) := by
  unfold tri
  exact Nat.div_mul_cancel (two_dvd_mul_succ k)
end

section
/-- The triangular recurrence `T(k+1) = T(k) + (k+1)`. -/
theorem tri_succ (k : Nat) : tri (k + 1) = tri k + (k + 1) := by
  have e1 : tri k * 2 = k * (k + 1) := tri_mul_two k
  have e2 : tri (k + 1) * 2 = (k + 1) * (k + 1 + 1) := tri_mul_two (k + 1)
  have key : tri (k + 1) * 2 = (tri k + (k + 1)) * 2 := by
    have hexp : (tri k + (k + 1)) * 2 = tri k * 2 + (k + 1) * 2 := by ring
    rw [hexp, e1, e2]; ring
  exact Nat.eq_of_mul_eq_mul_right (by norm_num) key
end

section
/-- The value `num n m` sits in the half-open diagonal block `[T(n+m), T(n+m+1))`: the
    upper bound. -/
theorem num_lt_tri_succ (n m : Nat) : num n m < tri (n + m + 1) := by
  rw [tri_succ]; unfold num; omega
end

section
theorem num_succ_left (n : Nat) : num (n + 1) 0 = num 0 n + 1 := by
  unfold num
  have h0 : (n + 1) + 0 = n + 1 := by omega
  have h1 : 0 + n = n := by omega
  rw [h0, h1, tri_succ]; omega
end

section
theorem numP_nextCell (c : Nat  x  Nat) : numP (nextCell c) = numP c + 1 := by
  obtain <n, m> := c
  cases n with
  | zero => exact num_succ_left m
  | succ k => exact num_succ_right k m
end

section
theorem tri_le_succ (k : Nat) : tri k  <=  tri (k + 1) := by
  rw [tri_succ]; omega
end

section
/-- `T` is monotone (proved by hand on ` <= ` to stay choice-free). -/
theorem tri_mono {a b : Nat} (h : a  <=  b) : tri a  <=  tri b := by
  induction h with
  | refl => exact le_refl _
  | step _ ih => exact le_trans ih (tri_le_succ _)
end
```

```
section
/-- **The function is one-one.** The diagonal `s = n+m` is recovered as the unique `s`
    with
`T(s)  <=  num n m < T(s+1)`; then `m` is the offset and `n = s - m`. -/
theorem num_injective : Function.Injective numP := by
  rintro <n_1, m_1> <n_2, m_2> h
  have h' : num n_1 m_1 = num n_2 m_2 := h
  have hs : n_1 + m_1 = n_2 + m_2 := by
    rcases lt_trichotomy (n_1 + m_1) (n_2 + m_2) with hlt | heq | hgt
    * exfalso
      have h1 : num n_1 m_1 < tri (n_1 + m_1 + 1) := num_lt_tri_succ n_1 m_1
      have h2 : tri (n_1 + m_1 + 1)  <=  tri (n_2 + m_2) := tri_mono (by omega)
      have h3 : tri (n_2 + m_2)  <=  num n_2 m_2 := by unfold num; omega
      omega
    * exact heq
    * exfalso
      have h1 : num n_2 m_2 < tri (n_2 + m_2 + 1) := num_lt_tri_succ n_2 m_2
      have h2 : tri (n_2 + m_2 + 1)  <=  tri (n_1 + m_1) := tri_mono (by omega)
      have h3 : tri (n_1 + m_1)  <=  num n_1 m_1 := by unfold num; omega
      omega
  have hbase : tri (n_1 + m_1) + m_1 = tri (n_2 + m_2) + m_2 := h'
  rw [hs] at hbase
  have hm : m_1 = m_2 := by omega
  have hn : n_1 = n_2 := by omega
  subst hm; subst hn; rfl
end

section
/-- The inverse `unnum v = nextCell? (0, 0)`. -/
def unnum : Nat  ->  Nat  x  Nat
  | 0 => (0, 0)
  | v + 1 => nextCell (unnum v)
end

section
theorem numP_unnum (v : Nat) : numP (unnum v) = v := by
  induction v with
  | zero => exact num_zero_zero
  | succ k ih =>
      show numP (nextCell (unnum k)) = k + 1
      rw [numP_nextCell, ih]
end

section
theorem unnum_numP (c : Nat  x  Nat) : unnum (numP c) = c :=
  num_injective (by rw [numP_unnum])
end

end DomainTheory.NeighbourhoodSystems.Exercise513
```

3.6.15 Exercise 5.13 — powerset isomorphisms

Prove the existence of a one-one function $\mathrm{num} : \mathbf{N} \times \mathbf{N} \to \mathbf{N}$ such that

$$\mathrm{num}(0,0) = 0;$$

$$\mathrm{num}(n, m+1) = \mathrm{num}(n+1, m) + 1;$$

$$\mathrm{num}(n+1, 0) = \mathrm{num}(0, n) + 1.$$

Draw a picture (i.e. an infinite matrix) for the function and find a closed form for its values, if possible. Use the function to prove the isomorphism of the domains

$$P\mathbf{N},\ P(\mathbf{N} \times \mathbf{N}),\ P\mathbf{N} \times P\mathbf{N}.$$

Exercise_5_13b.lean

```
module

public import Mathlib.Logic.Equiv.Nat

public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.Exercise_5_13
public import Mathlib.Algebra.Group.End

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems.Exercise513
section
/- concordance: source L3097 EXERCISE 5.13 pairing function on numbers -/
/-- **Exercise 5.13.** The pairing function packaged as a bijection `Nat  x  Nat ? Nat`
     (choice-free, via the
explicit inverse `unnum`). -/
def numEquiv : Nat  x  Nat ? Nat where
  toFun := numP
  invFun := unnum
  left_inv := unnum_numP
  right_inv := numP_unnum
end

section
@[simp] theorem numEquiv_apply (p : Nat  x  Nat) : numEquiv p = num p.1 p.2 := rfl
end

section
variable {a b : Type*}

/-- A bijection `a ? b` induces an order-isomorphism `P a  ~=  P b` of power-set
    domains, by direct
image. -/
def setCongr (e : a ? b) : Set a ?o Set b where
  toFun S := e '' S
  invFun T := e.symm '' T
  left_inv := e.symm_image_image
  right_inv := e.symm.symm_image_image
  map_rel_iff' := by
```

```
    intro a b
    constructor
    * intro h x hx
      obtain <y, hy, hey> := h <x, hx, rfl>
      rwa [e.injective hey] at hy
    * rintro h _ <x, hx, rfl>
      exact <x, h hx, rfl>

/-- `P N  ~=  P(N  x  N)` -- induced by the pairing bijection `numEquiv`. -/
def PN_orderIso_PNN : Set Nat ?o Set (Nat  x  Nat) := setCongr numEquiv.symm

/-- `P N  ~=  P N  x  P N` -- `N  ~=  N ? N` together with `P(A ? B)  ~=  P A  x  P B`
    (`Set.sumEquiv`). -/
def PN_orderIso_prod : Set Nat ?o Set Nat  x  Set Nat :=
  (setCongr Equiv.natSumNatEquivNat.symm).trans Set.sumEquiv

/-- `P(N  x  N)  ~=  P N  x  P N` -- by composing the two isomorphisms above. -/
def PNN_orderIso_prod : Set (Nat  x  Nat) ?o Set Nat  x  Set Nat :=
  PN_orderIso_PNN.symm.trans PN_orderIso_prod

end

end DomainTheory.NeighbourhoodSystems.Exercise513
end
```

3.6.16 Exercise 5.14 — functions as graphs

Show that there are approximable mappings

$$\mathrm{graph} : (P\mathbf{N} \to P\mathbf{N}) \to P\mathbf{N} \quad \text{and} \quad \mathrm{fun} : P\mathbf{N} \to (P\mathbf{N} \to P\mathbf{N}),$$

where we have

$$\mathrm{fun} \circ \mathrm{graph} = \lambda f.\, f, \quad \text{and} \quad \mathrm{graph} \circ \mathrm{fun} \supseteq \lambda x.\, x.$$

(Hint: Using the notation

$$[n_0, n_1, \ldots, n_k] = \mathrm{num}(n_0, [n_1, \ldots, n_k])$$

two such combinators can be given by formulae

$$\mathrm{fun}(u)(x) = \{m \mid \exists n_0, \ldots, n_{k-1} \in x.\, [n_0 + 1, \ldots, n_{k-1} + 1, 0, m] \in u\}$$

$$\mathrm{graph}(f) = \{[n_0 + 1, \ldots, n_{k-1} + 1, 0, m] \mid m \in f(\{n_0, \ldots, n_{k-1}\})\}$$

where k is variable — meaning all finite sequences are to be considered.)

Exercise_5_14.lean

```
module

public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.Exercise_5_13
```

```
public import Mathlib.Data.Set.Defs

@[expose] public section

section
/- concordance: source L3119 EXERCISE 5.14 functions as graphs -/
-- Exercise514/tag.lean
namespace DomainTheory.NeighbourhoodSystems.Exercise514

open DomainTheory.NeighbourhoodSystems.Exercise513

/-- `tag [n_0, ..., n_{k-1}] m = [n_0+1, ..., n_{k-1}+1, 0, m]`, built from the pairing
     function `num`. -/
def tag : List Nat  ->  Nat  ->  Nat
  | [], m => num 0 m
  | (n :: ns), m => num (n + 1) (tag ns m)

end DomainTheory.NeighbourhoodSystems.Exercise514
end

section
namespace DomainTheory.NeighbourhoodSystems.Exercise514
open DomainTheory.NeighbourhoodSystems.Exercise513

section
@[simp] theorem tag_nil (m : Nat) : tag [] m = num 0 m := rfl
end

section
@[simp] theorem tag_cons (n : Nat) (ns : List Nat) (m : Nat) :
    tag (n :: ns) m = num (n + 1) (tag ns m) := rfl
end

section
/-- The finite set of entries of a list, as a subset of `Nat`. -/
def entries (ns : List Nat) : Set Nat := {n | n  in  ns}
end

section
@[simp] theorem mem_entries {n : Nat} {ns : List Nat} : n  in  entries ns  <->  n  in
     ns := Iff.rfl
end

section
/-- `fun(u)(x)` -- apply the "function coded by `u`" to argument `x`. -/
def Fun (u : Set Nat) (x : Set Nat) : Set Nat :=
  {m | exists  ns, (forall  n  in  ns, n  in  x)  /\  tag ns m  in  u}
end

section
/-- `graph(f)` -- the code of the function `f`. -/
def Graph (f : Set Nat  ->  Set Nat) : Set Nat :=
  {c | exists  ns m, c = tag ns m  /\  m  in  f (entries ns)}
end

section
```

```
/-- A map `f : P?  ->  P?` is *approximable* (continuous) when it is monotone and every
     output is
already produced by a finite subset of the input. The finite subsets of `x` are exactly
     the
`entries ns` with all entries in `x`.

Monotonicity is phrased as an explicit ` subseteq `-implication rather than `Monotone f
    `: on `Set Nat` the
order ` <= ` resolves through the `CompleteLattice` instance whose construction uses `
    Classical.choice`,
so the `Monotone`-based statement would not be choice-free. The two are definitionally
    equal. -/
def IsApprox (f : Set Nat  ->  Set Nat) : Prop :=
  (forall  ?x x' : Set Nat?, x  subseteq  x'  ->  f x  subseteq  f x')  /\
    forall  x m, m  in  f x  ->  exists  ns, (forall  n  in  ns, n  in  x)  /\  m  in
     f (entries ns)
end

section
/-- Every `Fun u` is itself approximable, so `fun` really lands in the continuous maps.
     -/
theorem Fun_isApprox (u : Set Nat) : IsApprox (Fun u) := by
  refine <?_, ?_>
  * intro x x' hxx' m hm
    obtain <ns, hsub, htag> := hm
    exact <ns, fun n hn => hxx' (hsub n hn), htag>
  * intro x m hm
    obtain <ns, hsub, htag> := hm
    exact <ns, hsub, ns, fun n hn => hn, htag>
end

section
/-- **`Fun` is monotone in both arguments jointly.** Needed to check the `mono` axiom
    of the
neighbourhood-level approximable map built from `Fun` (Exercise 7.23). -/
theorem Fun_mono {u u' x x' : Set Nat} (hu : u  subseteq  u') (hx : x  subseteq  x') :
     Fun u x  subseteq  Fun u' x' := by
  rintro m <ns, hns, htag>
  exact <ns, fun n hn => hx (hns n hn), hu htag>
end

section
/-- `num (n+1) b` strictly exceeds its second argument: this is what makes the decode
    recursion
terminate. `num (n+1) b = T(n+1+b) + b  >=  (n+b+1) + b > b`. -/
theorem num_succ_left_gt (n b : Nat) : b < num (n + 1) b := by
  unfold num
  have e : n + 1 + b = (n + b) + 1 := by omega
  rw [e, tri_succ]
  omega
end

section
/-- **The tag is one-one**, jointly in both arguments (induction on the list + `
    num_injective`). -/
theorem tag_injective : forall  {ns_1 ns_2 : List Nat} {m_1 m_2 : Nat},
```

```
    tag ns_1 m_1 = tag ns_2 m_2  ->  ns_1 = ns_2  /\  m_1 = m_2 := by
  intro ns_1
  induction ns_1 with
  | nil =>
      intro ns_2 m_1 m_2 h
      cases ns_2 with
      | nil =>
          have hp := num_injective (show numP (0, m_1) = numP (0, m_2) from h)
          injection hp with _ hm
          exact <rfl, hm>
      | cons n_2 ns_2' =>
          exfalso
          have hp := num_injective (show numP (0, m_1) = numP (n_2 + 1, tag ns_2' m_2)
     from h)
          injection hp with h1 _
          omega
  | cons n_1 ns_1' ih =>
      intro ns_2 m_1 m_2 h
      cases ns_2 with
      | nil =>
          exfalso
          have hp := num_injective (show numP (n_1 + 1, tag ns_1' m_1) = numP (0, m_2)
     from h)
          injection hp with h1 _
          omega
      | cons n_2 ns_2' =>
          have hp := num_injective
            (show numP (n_1 + 1, tag ns_1' m_1) = numP (n_2 + 1, tag ns_2' m_2) from h
    )
          injection hp with h1 h2
          have hn : n_1 = n_2 := by omega
          obtain <hns, hm> := ih h2
          exact <by rw [hn, hns], hm>
end

section
/-- **`fun  comp  graph = fun f.f`** for continuous `f`: `Fun (Graph f) x = f x`. -/
theorem Fun_Graph {f : Set Nat  ->  Set Nat} (hf : IsApprox f) (x : Set Nat) :
    Fun (Graph f) x = f x := by
  apply Set.Subset.antisymm
  * intro m hm
    obtain <ns, hsub, htag> := hm
    obtain <ns', m', heq, hmem> := htag
    obtain <hns, hmm> := tag_injective heq
    subst hns; subst hmm
    have hsubset : entries ns  subseteq  x := fun n hn => hsub n hn
    exact hf.1 hsubset hmem
  * intro m hm
    obtain <ns, hsub, hmem> := hf.2 x m hm
    exact <ns, hsub, ns, m, rfl, hmem>
end

section
/-- **The tag is onto**: every `c : Nat` decodes as some `tag ns m`. Strong induction
    on `c`,
decreasing via `num_succ_left_gt`. -/
```

```
theorem tag_surjective (c : Nat) : exists  ns m, tag ns m = c := by
  induction c using Nat.strong_induction_on with
  | _ c ih =>
      obtain <a, b, hab> : exists  a b, num a b = c :=
        <(unnum c).1, (unnum c).2, numP_unnum c>
      cases a with
      | zero => exact <[], b, hab>
      | succ k =>
          have hlt : b < c := by rw [ <-  hab]; exact num_succ_left_gt k b
          obtain <ns, m, hns> := ih b hlt
          exact <k :: ns, m, by rw [tag_cons, hns]; exact hab>
end

section
/-- **`graph  comp  fun  supseteq  fun x.x`**: `u  subseteq  Graph (Fun u)`. -/
theorem id_le_Graph_Fun (u : Set Nat) : u  subseteq  Graph (Fun u) := by
  intro c hc
  obtain <ns, m, hceq> := tag_surjective c
  refine <ns, m, hceq.symm, ?_>
  exact <ns, fun n hn => hn, by rw [hceq]; exact hc>
end

end DomainTheory.NeighbourhoodSystems.Exercise514
end
```

3.6.17 Exercise 5.15 — star of a word

(For algebraists.) We can regard $\langle \{0,1\}^*, \Lambda, \cdot \rangle$ as the free semigroup on two generators 0 and 1. The powerset $P\{0,1\}^*$ is taken as a domain as in Exercise 4.17. For "words" $e \in \{0,1\}^*$ define

$$e^* = \{\Lambda, e, e^2, e^3, \ldots, e^n, \ldots\}.$$

Show that the least fixed point of

$$z = \{e\} \cdot z \cup \{e'\}$$

in $P\{0,1\}^*$ is $z = e^* \cdot \{e'\}$.

Show further (as suggested by David Park) that the least solution of

$$x = a \cdot x \cup b \cdot y \cup c,$$
$$y = b \cdot x \cup a \cdot y \cup d$$

has

$$x = (a \cup b \cdot a^* \cdot b)^* \cdot (c \cup b \cdot a^* \cdot d),$$

where the $\{\cdot\}$ has been dropped off $\{a\}$, $\{b\}$ etc., and where the *-notation has been extended to the whole domain, so that $z^* = \Lambda \cup z^* \cdot z$.

(Hint: Apply 5.3.)

Exercise_5_15.lean

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Exercise_4_14
public import Mathlib.Algebra.FreeMonoid.Basic
public import Mathlib.Algebra.Group.Pointwise.Set.Basic

@[expose] public section

section
/- concordance: source L3149 EXERCISE 5.15 star of a word -/
-- Exercise515/kpow.lean
namespace DomainTheory.NeighbourhoodSystems.Exercise515

open Pointwise

open DomainTheory.NeighbourhoodSystems.Exercise414

variable {S : Type*} [Monoid S]

/-- `z?` as an iterated pointwise product (left-recursion `z^{n+1} = z*z?`). -/
def kpow (z : Set S) : Nat  ->  Set S
  | 0 => 1
  | n + 1 => z * kpow z n

end DomainTheory.NeighbourhoodSystems.Exercise515
end

section
namespace DomainTheory.NeighbourhoodSystems.Exercise515
open Pointwise
open DomainTheory.NeighbourhoodSystems.Exercise414

section
variable {S : Type*} [Monoid S]

@[simp] theorem kpow_zero (z : Set S) : kpow z 0 = 1 := rfl

@[simp] theorem kpow_succ (z : Set S) (n : Nat) : kpow z (n + 1) = z * kpow z n := rfl

/-- Scott's operator `w  |->  z*w  U  v`, whose least fixed point is the least solution
     of `w = z*w  U  v`. -/
def G (z v : Set S) (w : Set S) : Set S := z * w  U  v

/-- Scott's `z*`: the set of all finite products of elements of `z` (including the
    empty product
`1`). Defined as `{s ? exists  n, s  in  z?}` to stay choice-free (avoiding `?` order
    lemmas). -/
def star (z : Set S) : Set S := {s | exists  n, s  in  kpow z n}

/-- The "feedback" coefficient `A = a  U  b*a**b` of the eliminated system. -/
def parkA (a b : Set S) : Set S := a  U  b * star a * b

/-- The "feedback" constant `C = c  U  b*a**d`. -/
def parkC (a b c d : Set S) : Set S := c  U  b * star a * d

/-- Park's least solution for `x`: `x_0 = (a  U  b*a**b)**(c  U  b*a**d)`. -/
```

```
def parkX (a b c d : Set S) : Set S := star (parkA a b) * parkC a b c d

/-- The accompanying least solution for `y`: `y_0 = a**(b*x_0  U  d)`. -/
def parkY (a b c d : Set S) : Set S := star a * (b * parkX a b c d  U  d)

/-- `{e}? = {e?}` at the membership level (choice-free). -/
theorem mem_kpow_singleton (e s : S) (n : Nat) : s  in  kpow ({e} : Set S) n  <->  s =
     e ^ n := by
  induction n generalizing s with
  | zero => rw [kpow_zero, pow_zero]; exact Set.mem_one
  | succ m ih =>
      rw [kpow_succ, pow_succ', Set.mem_mul]
      constructor
      * rintro <a, ha, t, ht, rfl>
        rw [Set.mem_singleton_iff] at ha
        rw [(ih t).mp ht, ha]
      * rintro rfl
        exact <e, Set.mem_singleton_iff.mpr rfl, e ^ m, (ih (e ^ m)).mpr rfl, rfl>

theorem mem_star {z : Set S} {s : S} : s  in  star z  <->  exists  n, s  in  kpow z n
    := Iff.rfl

/-- `{e}* = e* = {?, e, e?, ...}`: a point lies in `star {e}` iff it is some power of `
    e`. -/
theorem mem_star_singleton (e s : S) : s  in  star ({e} : Set S)  <->  exists  n, s =
    e ^ n := by
  rw [mem_star]
  constructor
  * rintro <n, hn>; exact <n, (mem_kpow_singleton e s n).mp hn>
  * rintro <n, rfl>; exact <n, (mem_kpow_singleton e (e ^ n) n).mpr rfl>

/-- Associativity of the pointwise product (membership proof, choice-free). -/
theorem smul_assoc (a b c : Set S) : a * b * c = a * (b * c) := by
  ext s; simp only [Set.mem_mul]
  constructor
  * rintro <u, <p, hp, q, hq, rfl>, w, hw, rfl>
    exact <p, hp, q * w, <q, hq, w, hw, rfl>, by rw [mul_assoc]>
  * rintro <p, hp, u, <q, hq, w, hw, rfl>, rfl>
    exact <p * q, <p, hp, q, hq, rfl>, w, hw, by rw [mul_assoc]>

/-- Left distributivity `a*(b  U  c) = a*b  U  a*c` (choice-free). -/
theorem smul_union (a b c : Set S) : a * (b  U  c) = a * b  U  a * c := by
  ext s; simp only [Set.mem_mul, Set.mem_union]
  constructor
  * rintro <u, hu, w, (hw | hw), rfl>
    * exact Or.inl <u, hu, w, hw, rfl>
    * exact Or.inr <u, hu, w, hw, rfl>
  * rintro (<u, hu, w, hw, rfl> | <u, hu, w, hw, rfl>)
    * exact <u, hu, w, Or.inl hw, rfl>
    * exact <u, hu, w, Or.inr hw, rfl>

/-- **Scott's unfolding** `z* = ?  U  z*z*`. -/
theorem star_eq (z : Set S) : star z = (1 : Set S)  U  z * star z := by
  apply Set.Subset.antisymm
  * intro s hs
    obtain <n, hn> := mem_star.mp hs
```

```
    cases n with
    | zero =>
        rw [kpow_zero] at hn
        rw [Set.mem_union]; exact Or.inl hn
    | succ m =>
        rw [kpow_succ, Set.mem_mul] at hn
        obtain <a, ha, t, ht, rfl> := hn
        rw [Set.mem_union]; right
        rw [Set.mem_mul]
        exact <a, ha, t, mem_star.mpr <m, ht>, rfl>
  * intro s hs
    rw [Set.mem_union] at hs
    rcases hs with h1 | h2
    * exact mem_star.mpr <0, by rw [kpow_zero]; exact h1>
    * rw [Set.mem_mul] at h2
      obtain <a, ha, t, ht, rfl> := h2
      obtain <m, hm> := mem_star.mp ht
      exact mem_star.mpr <m + 1, by rw [kpow_succ, Set.mem_mul]; exact <a, ha, t, hm,
    rfl>>

/-- For any prefixed point `w_0` of `w  |->  z*w  U  v` (i.e. `z*w_0  U  v  subseteq
    w_0`), the candidate `z**v` is
below `w_0`: the induction `z?*v  subseteq  w_0`. -/
theorem star_mul_subset_prefixed (z v x : Set S) (hx : z * x  U  v  subseteq  x) :
    star z * v  subseteq  x := by
  have hvx : v  subseteq  x := Set.subset_union_right.trans hx
  have hzx : z * x  subseteq  x := Set.subset_union_left.trans hx
  have hpow : forall  n, kpow z n * v  subseteq  x := by
    intro n
    induction n with
    | zero => rw [kpow_zero, one_mul]; exact hvx
    | succ m ih =>
        rw [kpow_succ, smul_assoc]
        exact (Set.mul_subset_mul_left ih).trans hzx
  intro s hs
  rw [Set.mem_mul] at hs
  obtain <a, ha, t, ht, rfl> := hs
  obtain <n, hn> := mem_star.mp ha
  exact hpow n (by rw [Set.mem_mul]; exact <a, hn, t, ht, rfl>)

/-- Right distributivity `(a  U  b)*c = a*c  U  b*c` (choice-free). -/
theorem sunion_mul (a b c : Set S) : (a  U  b) * c = a * c  U  b * c := by
  ext s; simp only [Set.mem_mul, Set.mem_union]
  constructor
  * rintro <u, (hu | hu), w, hw, rfl>
    * exact Or.inl <u, hu, w, hw, rfl>
    * exact Or.inr <u, hu, w, hw, rfl>
  * rintro (<u, hu, w, hw, rfl> | <u, hu, w, hw, rfl>)
    * exact <u, Or.inl hu, w, hw, rfl>
    * exact <u, Or.inr hu, w, hw, rfl>

/-- `z**v` is a fixed point of `w  |->  z*w  U  v`: the star identity `z*(z**v)  U  v =
    z**v`. -/
theorem star_mul_isFixed (z v : Set S) : z * (star z * v)  U  v = star z * v := by
  conv_rhs => rw [star_eq z]
  rw [sunion_mul, one_mul, smul_assoc]
```

```
  exact Set.union_comm _ _

/-- **Arden's lemma.** The least solution of `w = z*w ∪ v` in `P S` is `z**v`. (
    Stated via
`lfpSet`, Exercise 4.14; the proof uses neither `Monotone` nor `?` order lemmas, so it
    is
choice-free.) -/
theorem arden (z v : Set S) : lfpSet (G z v) = star z * v := by
  apply Set.Subset.antisymm
  * exact lfpSet_least (G z v) (star_mul_isFixed z v)
  * intro s hs x hx
    exact star_mul_subset_prefixed z v x hx hs

/-- `x_0` satisfies the first equation `x = a*x ∪ b*y ∪ c`. -/
theorem parkX_eq (a b c d : Set S) :
    parkX a b c d = a * parkX a b c d ∪ b * parkY a b c d ∪ c := by
  have hfix : parkA a b * parkX a b c d ∪ parkC a b c d = parkX a b c d :=
    star_mul_isFixed (parkA a b) (parkC a b c d)
  conv_lhs => rw [ <- hfix]
  simp only [parkA, parkC, parkY, sunion_mul, smul_union, smul_assoc,
    Set.union_comm, Set.union_left_comm]

/-- `y_0` satisfies the second equation `y = b*x ∪ a*y ∪ d`. -/
theorem parkY_eq (a b c d : Set S) :
    parkY a b c d = b * parkX a b c d ∪ a * parkY a b c d ∪ d := by
  have hfix : a * parkY a b c d ∪ (b * parkX a b c d ∪ d) = parkY a b c d :=
    star_mul_isFixed a (b * parkX a b c d ∪ d)
  conv_lhs => rw [ <- hfix]
  simp only [Set.union_assoc, Set.union_comm]

/-- **Exercise 5.15(2), leastness.** Any solution `(x, y)` of the system dominates `(
    x_0, y_0)`. Hence
`(x_0, y_0)` is the *least* solution, and `x_0 = (a ∪ b*a**b)**(c ∪ b*a**d)` as
    claimed. The proof is
Gaussian elimination: solve the second equation for `y` by `arden`, substitute, apply `
    arden`
again. -/
theorem park_least (a b c d : Set S) {x y : Set S}
    (hx : x = a * x ∪ b * y ∪ c) (hy : y = b * x ∪ a * y ∪ d) :
    parkX a b c d ⊆ x ∧ parkY a b c d ⊆ y := by
  have hax : a * x ⊆ x := by
    calc a * x ⊆ a * x ∪ b * y ∪ c := Set.subset_union_left.trans Set.
    subset_union_left
      _ = x := hx.symm
  have hby : b * y ⊆ x := by
    calc b * y ⊆ a * x ∪ b * y ∪ c := Set.subset_union_right.trans Set.
    subset_union_left
      _ = x := hx.symm
  have hcx : c ⊆ x := by
    calc c ⊆ a * x ∪ b * y ∪ c := Set.subset_union_right
      _ = x := hx.symm
  have hay : a * y ⊆ y := by
    calc a * y ⊆ b * x ∪ a * y ∪ d := Set.subset_union_right.trans Set.
    subset_union_left
      _ = y := hy.symm
  have hbx : b * x ⊆ y := by
```

```
    calc b * x  subseteq  b * x  U  a * y  U  d := Set.subset_union_left.trans Set.
    subset_union_left
      _ = y := hy.symm
  have hdy : d  subseteq  y := by
    calc d  subseteq  b * x  U  a * y  U  d := Set.subset_union_right
      _ = y := hy.symm
  -- The eliminated `y`: `a**(b*x  U  d)  subseteq  y`.
  have hpre : a * y  U  (b * x  U  d)  subseteq  y := Set.union_subset hay (Set.
    union_subset hbx hdy)
  have hyy : star a * (b * x  U  d)  subseteq  y := by
    rw [ <-  arden a (b * x  U  d)]
    exact lfpSet_subset (G a (b * x  U  d)) hpre
  -- The decisive identity `(b*a**b)*x  U  b*a**d = b*(a**(b*x  U  d))`.
  have key : b * star a * b * x  U  b * star a * d = b * (star a * (b * x  U  d)) :=
    by
    simp only [smul_assoc, smul_union]
  -- `x` is a prefixed point of `w  |->  A*w  U  C`, so `x_0  subseteq  x`.
  have hprefixX : parkA a b * x  U  parkC a b c d  subseteq  x := by
    have hmid : b * star a * b * x  U  b * star a * d  subseteq  x := by
      rw [key]; exact (Set.mul_subset_mul_left hyy).trans hby
    have hm1 : b * star a * b * x  subseteq  x := Set.subset_union_left.trans hmid
    have hm2 : b * star a * d  subseteq  x := Set.subset_union_right.trans hmid
    unfold parkA parkC
    rw [sunion_mul]
    exact Set.union_subset (Set.union_subset hax hm1) (Set.union_subset hcx hm2)
  have hPX : parkX a b c d  subseteq  x := by
    have harden : parkX a b c d = lfpSet (G (parkA a b) (parkC a b c d)) :=
      (arden (parkA a b) (parkC a b c d)).symm
    rw [harden]
    exact lfpSet_subset (G (parkA a b) (parkC a b c d)) hprefixX
  have hPY : parkY a b c d  subseteq  y := by
    show star a * (b * parkX a b c d  U  d)  subseteq  y
    have h1 : b * parkX a b c d  U  d  subseteq  b * x  U  d :=
      Set.union_subset_union_left d (Set.mul_subset_mul_left hPX)
    exact (Set.mul_subset_mul_left h1).trans hyy
  exact <hPX, hPY>

/-- **Exercise 5.15(2), existence.** `(x_0, y_0)` solves David Park's system. -/
theorem park_solves (a b c d : Set S) :
    parkX a b c d = a * parkX a b c d  U  b * parkY a b c d  U  c  /\
      parkY a b c d = b * parkX a b c d  U  a * parkY a b c d  U  d :=
  <parkX_eq a b c d, parkY_eq a b c d>

/-- **Exercise 5.15(1).** The least fixed point of `z = {e}*z  U  {e'}` is `e**{e'}` (`
    star {e}` is
`e*` by `mem_star_singleton`). -/
theorem part1 (e e' : S) :
    lfpSet (fun w => ({e} : Set S) * w  U  {e'}) = star ({e} : Set S) * {e'} := by
  have hG : (fun w => ({e} : Set S) * w  U  {e'}) = G ({e} : Set S) {e'} := rfl
  rw [hG]; exact arden {e} {e'}

end

section
variable {S : Type*} [Monoid S]
```

```
/-- The free semigroup `{0,1}*` of the exercise. Part (1) holds here as a special case.
     -/
theorem part1_freeMonoid (e e' : FreeMonoid Bool) :
    lfpSet (fun w => ({e} : Set (FreeMonoid Bool)) * w  U  {e'})
      = star ({e} : Set (FreeMonoid Bool)) * {e'} :=
  part1 e e'
end

end DomainTheory.NeighbourhoodSystems.Exercise515
end
```

3.6.18 Exercise 5.16 — negation of sequences

Return to the discussion of Example 4.4 and the construction of the domain of finite and infinite binary sequences. Give a fixed-point definition of $\mathrm{neg} : C \to C$, where

$$\mathrm{neg}(0x) = 1\,\mathrm{neg}(x); \qquad \mathrm{neg}(1x) = 0\,\mathrm{neg}(x).$$

Prove that $\mathrm{neg}(\mathrm{neg}(x)) = x$ for all $x \in |C|$. Also define $\mathrm{merge} : C \times C \to C$, where for $\epsilon, \delta \in \{0, 1\}$ we have:

$$\mathrm{merge}(\epsilon x, \delta y) = \epsilon\,\delta\,\mathrm{merge}(x, y).$$

(Note: There may be a little trouble with $\mathrm{merge}(x, y)$ when x is finite and total and y is infinite—you have to decide what you want in e.g. $\mathrm{merge}(\Lambda, y)$.) Prove that

$$\mathrm{merge}(x, x) = d(x),$$

in the notation of 4.4. Consider also the infinite non-periodic sequence

$$t = 0\,\mathrm{merge}(\mathrm{neg}(t), \mathrm{tail}(t)).$$

Prove that the nth digit of t is the sum mod 2 of the digits of the number n written in the binary scale (a suggestion of J. Lambek). Show also that $t \neq u\,a\,a\,a\,v$ where a is any finite sequence $\neq \Lambda$, and where u is finite.

Exercise_5_16.lean

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Exercise_4_11
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Exercise_4_19
public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Theorem_4_2
public import Mathlib.Data.Nat.Bits
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_12
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Exercise_3_16
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_7
```

```
@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.Exercise516
section
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap ExampleB
    Example44 Exercise419

/-- Complement every bit of a finite string. -/
abbrev flip (sigma : Str) : Str := sigma.map not

@[simp] theorem flip_nil : flip [] = [] := rfl

@[simp] theorem flip_cons (b : Bool) (sigma : Str) : flip (b :: sigma) = (!b) :: flip
    sigma := rfl

@[simp] theorem flip_flip (sigma : Str) : flip (flip sigma) = sigma := by
  induction sigma with
  | nil => rfl
  | cons b sigma ih => simp [ih]

/-- Double every bit of a finite string: `double (b :: sigma) = b :: b :: double sigma
    `. -/
def double : Str  ->  Str
  | [] => []
  | b :: sigma => b :: b :: double sigma

@[simp] theorem double_nil : double [] = [] := rfl

@[simp] theorem double_cons (b : Bool) (sigma : Str) : double (b :: sigma) = b :: b ::
     double sigma := rfl

/-- `double` distributes over append. -/
theorem double_append (sigma tau : Str) : double (sigma ++ tau) = double sigma ++
    double tau := by
  induction sigma with
  | nil => rfl
  | cons b sigma ih => simp [double, ih]

/-- `double` preserves the prefix order. -/
theorem double_prefix {sigma tau : Str} (h : sigma <+: tau) : double sigma <+: double
    tau := by
  obtain <?, rfl> := h
  exact <double ?, (double_append sigma ?).symm>

/-- `flip` preserves the prefix order. -/
theorem flip_prefix {sigma tau : Str} (h : sigma <+: tau) : flip sigma <+: flip tau :=
     h.map _

theorem not_strElem_le_strBot {sigma tau : Str} : not  strElem sigma  <=  strBot tau
    := by
  rw [strElem, strBot, C.principal_le_iff]
  exact not_cone_subset_singleton tau sigma

theorem strBot_le_strBot_iff {sigma tau : Str} : strBot sigma  <=  strBot tau  <->
    sigma <+: tau := by
  rw [strBot, strBot, C.principal_le_iff, cone_subset_cone]
```

```
theorem strBot_le_strElem_iff {sigma tau : Str} : strBot sigma  <=  strElem tau  <->
    sigma <+: tau := by
  rw [strBot, strElem, C.principal_le_iff]
  exact singleton_subset_cone

theorem strElem_le_strElem_iff {sigma tau : Str} : strElem sigma  <=  strElem tau  <->
      sigma = tau := by
  rw [strElem, strElem, C.principal_le_iff, Set.singleton_subset_iff, Set.
    mem_singleton_iff,
    eq_comm]

end

section
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap ExampleB
    Example44 Exercise419
variable {b : Type*}

/-- The value of `tail` on a total element `sigma`: `tail(?) = False`, `tail(bsigma') =
     sigma'`. -/
def tailSing : Str  ->  C.Element
  | [] => strBot []
  | _ :: sigma' => strElem sigma'

theorem tail_hsing {sigma tau : Str} (h : sigma <+: tau) : strBot sigma.tail  <=
    tailSing tau := by
  cases tau with
  | nil => obtain rfl := List.prefix_nil.mp h; exact le_refl _
  | cons a tau' =>
    refine strBot_le_strElem_iff.mpr ?_
    cases sigma with
    | nil => exact List.nil_prefix
    | cons b sigma' =>
      obtain <rfl, h'> := List.cons_prefix_cons.mp h
      exact h'

end

section
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap ExampleB
    Example44 Exercise419

/-- A prefix relation descends to tails. -/
theorem tail_prefix {sigma tau : Str} (h : sigma <+: tau) : sigma.tail <+: tau.tail :=
     by
  obtain <?, rfl> := h
  cases sigma with
  | nil => simp
  | cons a sigma' => exact List.prefix_append sigma' ?
end

section
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap ExampleB
    Example44 Exercise419
variable {b : Type*}
```

```
theorem tail_hcone {sigma tau : Str} (h : sigma <+: tau) : strBot sigma.tail  <=
    strBot tau.tail :=
  strBot_le_strBot_iff.mpr (tail_prefix h)

end
end DomainTheory.NeighbourhoodSystems.Exercise516

section
namespace DomainTheory.NeighbourhoodSystems.Exercise516
section
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap ExampleB
    Example44 Exercise419
variable {b : Type*}

/- concordance: source L3184 EXERCISE 5.16 negation of sequences -/
/-- **Exercise 5.16 / Example 4.4 -- `tail : C  ->  C`.** Built with the head-test
    combinator `liftC`:
on `sigmaFalse` it returns `(tail sigma)False`, on `sigma` the total `tail sigma` (with
     `tail ? = False`). -/
def tailMap : ApproximableMap C C :=
  liftC C (fun sigma => strBot sigma.tail) tailSing tail_hcone tail_hsing

@[simp] theorem tailMap_strBot (sigma : Str) :
    tailMap.toElementMap (strBot sigma) = strBot sigma.tail :=
  liftC_strBot C (fun sigma => strBot sigma.tail) tailSing tail_hcone tail_hsing sigma

@[simp] theorem tailMap_strElem (sigma : Str) :
    tailMap.toElementMap (strElem sigma) = tailSing sigma :=
  liftC_strElem C (fun sigma => strBot sigma.tail) tailSing tail_hcone tail_hsing
    sigma

theorem neg_hcone {sigma tau : Str} (h : sigma <+: tau) : strBot (flip sigma)  <=
    strBot (flip tau) :=
  strBot_le_strBot_iff.mpr (flip_prefix h)

theorem neg_hsing {sigma tau : Str} (h : sigma <+: tau) : strBot (flip sigma)  <=
    strElem (flip tau) :=
  strBot_le_strElem_iff.mpr (flip_prefix h)

/-- **Exercise 5.16 -- `neg : C  ->  C`.** The closed-form solution of Scott's
    recursion, built with
`liftC`: `neg(sigmaFalse) = (flip sigma)False` and `neg(sigma) = flip sigma`. -/
def negMap : ApproximableMap C C :=
  liftC C (fun sigma => strBot (flip sigma)) (fun sigma => strElem (flip sigma))
    neg_hcone neg_hsing

@[simp] theorem negMap_strBot (sigma : Str) :
    negMap.toElementMap (strBot sigma) = strBot (flip sigma) :=
  liftC_strBot C (fun sigma => strBot (flip sigma)) (fun sigma => strElem (flip sigma)
    ) neg_hcone neg_hsing sigma

@[simp] theorem negMap_strElem (sigma : Str) :
    negMap.toElementMap (strElem sigma) = strElem (flip sigma) :=
  liftC_strElem C (fun sigma => strBot (flip sigma)) (fun sigma => strElem (flip sigma
    )) neg_hcone neg_hsing sigma
```

```
theorem d_hcone {sigma tau : Str} (h : sigma <+: tau) : strBot (double sigma)  <=
    strBot (double tau) :=
  strBot_le_strBot_iff.mpr (double_prefix h)

theorem d_hsing {sigma tau : Str} (h : sigma <+: tau) : strBot (double sigma)  <=
    strElem (double tau) :=
  strBot_le_strElem_iff.mpr (double_prefix h)

/-- **Exercise 5.16 / Example 4.4 -- `d : C  ->  C`.** The doubling map, closed form
    via `liftC`:
`d(sigmaFalse) = (double sigma)False`, `d(sigma) = double sigma`. -/
def dMap : ApproximableMap C C :=
  liftC C (fun sigma => strBot (double sigma)) (fun sigma => strElem (double sigma))
    d_hcone d_hsing

@[simp] theorem dMap_strBot (sigma : Str) :
    dMap.toElementMap (strBot sigma) = strBot (double sigma) :=
  liftC_strBot C (fun sigma => strBot (double sigma)) (fun sigma => strElem (double
    sigma)) d_hcone d_hsing sigma

@[simp] theorem dMap_strElem (sigma : Str) :
    dMap.toElementMap (strElem sigma) = strElem (double sigma) :=
  liftC_strElem C (fun sigma => strBot (double sigma)) (fun sigma => strElem (double
    sigma)) d_hcone d_hsing sigma

/-- The neighbourhood of the tagged string `(b, sigma)`: `{sigma}` if total, `cone
    sigma` if partial. -/
def shape : Bool  ->  Str  ->  Set Str
  | true, sigma => {sigma}
  | false, sigma => cone sigma

theorem memC_shape : forall  (b : Bool) (sigma : Str), memC (shape b sigma)
  | true, sigma => memC_singleton sigma
  | false, sigma => memC_cone sigma

/-- The element of the tagged string `(b, sigma)`: total `sigma` if `b`, partial `
    sigmaFalse` otherwise. -/
def shapeElem (b : Bool) (sigma : Str) : C.Element := C.principal (memC_shape b sigma)

@[simp] theorem shapeElem_true (sigma : Str) : shapeElem true sigma = strElem sigma :=
     rfl

@[simp] theorem shapeElem_false (sigma : Str) : shapeElem false sigma = strBot sigma
    := rfl

/-- The approximation order between tagged strings: `(b, sigma)  sqsub  (b', sigma')`.
    A total string is maximal
(only ` sqsub ` itself); a partial string `sigmaFalse` is ` sqsub ` anything extending
    `sigma`. -/
def SLe : Bool  ->  Str  ->  Bool  ->  Str  ->  Prop
  | true, sigma, b', sigma' => b' = true  /\  sigma = sigma'
  | false, sigma, _, sigma' => sigma <+: sigma'

/-- The interleave value function: `mergeVal sigma b_0 tau b_1` returns the
    interleaving of the tagged
```

```
strings `(b_0, sigma)` and `(b_1, tau)` as a tagged string. Boundary convention (the
    only monotone one):
`merge(?, y) = ?`, `merge(False, y) = False`, and `merge(?x, y) = ?False` once `y` runs
     out. -/
def mergeVal : Str  ->  Bool  ->  Str  ->  Bool  ->  Str  x  Bool
  | [], b_0, _, _ => ([], b_0)
  | a :: _, _, [], _ => ([a], false)
  | a :: sigma, b_0, b :: tau, b_1 => (a :: b :: (mergeVal sigma b_0 tau b_1).1, (
    mergeVal sigma b_0 tau b_1).2)

@[simp] theorem mergeVal_nil (b_0 : Bool) (tau : Str) (b_1 : Bool) :
    mergeVal [] b_0 tau b_1 = ([], b_0) := rfl

@[simp] theorem mergeVal_cons_nil (a : Bool) (sigma : Str) (b_0 b_1 : Bool) :
    mergeVal (a :: sigma) b_0 [] b_1 = ([a], false) := rfl

@[simp] theorem mergeVal_cons_cons (a : Bool) (sigma : Str) (b_0 b : Bool) (tau : Str)
     (b_1 : Bool) :
    mergeVal (a :: sigma) b_0 (b :: tau) b_1 =
      (a :: b :: (mergeVal sigma b_0 tau b_1).1, (mergeVal sigma b_0 tau b_1).2) :=
    rfl

/-- Two equal head bits prepended preserve `SLe`. -/
theorem SLe_cons2 {p p' : Bool} {? ?' : Str} (c d : Bool) (h : SLe p ? p' ?') :
    SLe p (c :: d :: ?) p' (c :: d :: ?') := by
  cases p with
  | true => obtain <rfl, rfl> := h; exact <rfl, rfl>
  | false =>
    exact List.cons_prefix_cons.mpr <rfl, List.cons_prefix_cons.mpr <rfl, h>>

/-- Invert `SLe` on a cons in the first string: the second string starts with the same
    head. -/
theorem SLe_cons_inv {b_0 b_0' : Bool} {a : Bool} {sigma_0 sigma' : Str} (h : SLe b_0
    (a :: sigma_0) b_0' sigma') :
    exists  sigma_0', sigma' = a :: sigma_0'  /\  SLe b_0 sigma_0 b_0' sigma_0' := by
  cases b_0 with
  | true =>
    obtain <rfl, rfl> := h
    exact <sigma_0, rfl, rfl, rfl>
  | false =>
    cases sigma' with
    | nil => exact absurd h (by simp [SLe])
    | cons a' sigma_0' =>
      obtain <rfl, h'> := List.cons_prefix_cons.mp h
      exact <sigma_0', rfl, h'>

/-- The element produced by interleaving `(b_0, sigma)` and `(b_1, tau)`. -/
def mergeElem (sigma : Str) (b_0 : Bool) (tau : Str) (b_1 : Bool) : C.Element :=
  shapeElem (mergeVal sigma b_0 tau b_1).2 (mergeVal sigma b_0 tau b_1).1

/-- **The monotonicity of interleaving.** If `(b_0, sigma)  sqsub  (b_0', sigma')` and
    `(b_1, tau)  sqsub  (b_1', tau')` then the
interleavings are ` sqsub `-ordered. The crux that makes `merge` approximable. -/
theorem mergeVal_SLe : forall  (sigma : Str) (b_0 : Bool) (sigma' : Str) (b_0' : Bool)
    (tau : Str) (b_1 : Bool) (tau' : Str) (b_1' : Bool),
    SLe b_0 sigma b_0' sigma'  ->  SLe b_1 tau b_1' tau'  ->
```

```
    SLe (mergeVal sigma b_0 tau b_1).2 (mergeVal sigma b_0 tau b_1).1
      (mergeVal sigma' b_0' tau' b_1').2 (mergeVal sigma' b_0' tau' b_1').1
  | [], b_0, sigma', b_0', tau, b_1, tau', b_1', h0, _ => by
    cases b_0 with
    | true =>
      obtain <rfl, rfl> := h0
      simp only [mergeVal_nil]; exact <rfl, rfl>
    | false =>
      simp only [mergeVal_nil]; exact List.nil_prefix
  | a :: sigma_0, b_0, sigma', b_0', [], b_1, tau', b_1', h0, _ => by
    obtain <sigma_0', rfl, _> := SLe_cons_inv h0
    simp only [mergeVal_cons_nil]
    cases tau' with
    | nil => simp only [mergeVal_cons_nil]; exact List.prefix_rfl
    | cons c tau_0' => simp only [mergeVal_cons_cons]; exact <c :: _, rfl>
  | a :: sigma_0, b_0, sigma', b_0', b :: tau_0, b_1, tau', b_1', h0, h1 => by
    obtain <sigma_0', rfl, h0'> := SLe_cons_inv h0
    obtain <tau_0', rfl, h1'> := SLe_cons_inv h1
    simp only [mergeVal_cons_cons]
    exact SLe_cons2 a b (mergeVal_SLe sigma_0 b_0 sigma_0' b_0' tau_0 b_1 tau_0' b_1'
    h0' h1')

/-- `SLe` characterizes the element order on `shapeElem`. -/
theorem shapeElem_le_iff {b b' : Bool} {sigma sigma' : Str} :
    shapeElem b sigma  <=  shapeElem b' sigma'  <->  SLe b sigma b' sigma' := by
  cases b <;> cases b' <;>
    simp only [shapeElem_true, shapeElem_false, SLe, strElem_le_strElem_iff,
      strBot_le_strBot_iff, strBot_le_strElem_iff, true_and]
  * exact <fun h => absurd h not_strElem_le_strBot, fun h => absurd h (by simp)>

/-- The element-order form of `mergeVal_SLe`. -/
theorem mergeElem_mono {sigma sigma' tau tau' : Str} {b_0 b_0' b_1 b_1' : Bool}
    (h0 : shapeElem b_0 sigma  <=  shapeElem b_0' sigma') (h1 : shapeElem b_1 tau  <=
     shapeElem b_1' tau') :
    mergeElem sigma b_0 tau b_1  <=  mergeElem sigma' b_0' tau' b_1' :=
  shapeElem_le_iff.mpr
    (mergeVal_SLe sigma b_0 sigma' b_0' tau b_1 tau' b_1' (shapeElem_le_iff.mp h0) (
    shapeElem_le_iff.mp h1))

theorem shape_injective : forall  {b b' : Bool} {sigma sigma' : Str}, shape b sigma =
    shape b' sigma'  ->  b = b'  /\  sigma = sigma'
  | true, true, sigma, sigma', h => <rfl, by rwa [shape, shape, Set.
    singleton_eq_singleton_iff] at h>
  | true, false, sigma, sigma', h => absurd h.symm (cone_ne_singleton sigma' sigma)
  | false, true, sigma, sigma', h => absurd h (cone_ne_singleton sigma sigma')
  | false, false, sigma, sigma', h => <rfl, cone_injective h>

theorem shape_refine {b : Bool} {sigma : Str} {P : Set Str} (hP : memC P) (hsub : P
    subseteq  shape b sigma) :
    exists  (b' : Bool) (sigma' : Str), P = shape b' sigma'  /\  shapeElem b sigma  <=
      shapeElem b' sigma' := by
  rcases hP with <?, rfl> | <?, rfl>
  * exact <false, ?, rfl, (C.principal_le_iff (memC_shape b sigma) (memC_shape false
    ?)).mpr hsub>
  * exact <true, ?, rfl, (C.principal_le_iff (memC_shape b sigma) (memC_shape true ?))
    .mpr hsub>
```

```
/-- **Exercise 5.16 (Scott 1981, PRG-19).** The interleaving map `merge : C  x  C  ->
    C` with
`merge(?x, ?y) = ?*?*merge(x, y)`. Built directly as an approximable map: an input
    neighbourhood
`shape b_0 sigma  U  shape b_1 tau` relates to the neighbourhoods of `mergeElem sigma
    b_0 tau b_1`. -/
def mergeMap : ApproximableMap (prod C C) C where
  rel W Z := exists  (b_0 : Bool) (sigma : Str) (b_1 : Bool) (tau : Str),
    W = prodNbhd (shape b_0 sigma) (shape b_1 tau)  /\  (mergeElem sigma b_0 tau b_1).
    mem Z
  rel_dom := by
    rintro W Z <b_0, sigma, b_1, tau, rfl, _>
    exact prod_mem_prodNbhd (memC_shape b_0 sigma) (memC_shape b_1 tau)
  rel_cod := by
    rintro W Z <b_0, sigma, b_1, tau, _, hZ>
    exact (mergeElem sigma b_0 tau b_1).sub hZ
  master_rel := by
    refine <false, [], false, [], ?_, (mergeElem [] false [] false).master_mem>
    show (prod C C).master = prodNbhd (shape false []) (shape false [])
    simp only [prod_master, shape, C_master, cone_nil]
  inter_right := by
    rintro W Z Z' <b_0, sigma, b_1, tau, rfl, hZ> <b_0', sigma', b_1', tau', heq, hZ'>
    obtain <hX, hY> := prodNbhd_injective heq
    obtain <rfl, rfl> := shape_injective hX
    obtain <rfl, rfl> := shape_injective hY
    exact <b_0, sigma, b_1, tau, rfl, (mergeElem sigma b_0 tau b_1).inter_mem hZ hZ'>
  mono := by
    rintro W W_2 Z Z' <b_0, sigma, b_1, tau, rfl, hZ> hW_2W hZZ' hW_2 hZ'
    obtain <P, Q, hP, hQ, rfl> := hW_2
    obtain <hPsub, hQsub> := prodNbhd_subset_iff.mp hW_2W
    obtain <b_0', sigma', hPeq, hle0> := shape_refine hP hPsub
    obtain <b_1', tau', hQeq, hle1> := shape_refine hQ hQsub
    subst hPeq; subst hQeq
    refine <b_0', sigma', b_1', tau', rfl, ?_>
    have hmono := mergeElem_mono hle0 hle1
    exact (mergeElem sigma' b_0' tau' b_1').up_mem (hmono Z hZ) hZ' hZZ'

@[simp] theorem consMap_shapeElem (b c : Bool) (sigma : Str) :
    (consMap b).toElementMap (shapeElem c sigma) = shapeElem c (b :: sigma) := by
  cases c with
  | false => rw [shapeElem_false, consMap_strBot]; rfl
  | true => rw [shapeElem_true, consMap_strElem]; rfl

/-- Two approximable maps out of `C` agree as soon as they agree on every finite
    element `sigmaFalse` and
`sigma` (Exercise 2.8). This is the workhorse for the map equalities below. -/
theorem map_ext_C {V : NeighborhoodSystem b} {f g : ApproximableMap C V}
    (hbot : forall  sigma, f.toElementMap (strBot sigma) = g.toElementMap (strBot
    sigma))
    (helem : forall  sigma, f.toElementMap (strElem sigma) = g.toElementMap (strElem
    sigma)) : f = g := by
  apply eq_of_toElementMap_principal
  intro X hX
  obtain (<sigma, rfl> | <sigma, rfl>) := (C_mem.mp hX)
  * exact hbot sigma
```

```
  * exact helem sigma

theorem memC_eq_shape {X : Set Str} (hX : memC X) : exists  (b : Bool) (sigma : Str),
    X = shape b sigma := by
  rcases hX with <sigma, rfl> | <sigma, rfl>
  * exact <false, sigma, rfl>
  * exact <true, sigma, rfl>

/-- **The value of `merge` on a pair of finite elements.** `merge(<(b_0, sigma), (b_1,
    tau)>) =
mergeElem sigma b_0 tau b_1`. The analogue of `liftC_strBot`/`liftC_strElem` for the
    product. -/
theorem mergeMap_pair (b_0 : Bool) (sigma : Str) (b_1 : Bool) (tau : Str) :
    mergeMap.toElementMap (pair (shapeElem b_0 sigma) (shapeElem b_1 tau)) = mergeElem
     sigma b_0 tau b_1 := by
  apply Element.ext
  intro Z
  constructor
  * rintro <W, hWmem, c_0, ?, c_1, ?, rfl, hZ>
    rw [mem_pair_prodNbhd] at hWmem
    obtain <hmsigma, hmtau> := hWmem
    have hle0 : shapeElem c_0 ?  <=  shapeElem b_0 sigma :=
      (C.principal_le_iff (memC_shape c_0 ?) (memC_shape b_0 sigma)).mpr hmsigma.2
    have hle1 : shapeElem c_1 ?  <=  shapeElem b_1 tau :=
      (C.principal_le_iff (memC_shape c_1 ?) (memC_shape b_1 tau)).mpr hmtau.2
    exact mergeElem_mono hle0 hle1 Z hZ
  * intro hZ
    refine <prodNbhd (shape b_0 sigma) (shape b_1 tau), ?_, b_0, sigma, b_1, tau, rfl,
     hZ>
    exact mem_pair_prodNbhd.mpr <<memC_shape b_0 sigma, subset_rfl>, <memC_shape b_1
    tau, subset_rfl>>

/-- The diagonal value: interleaving `(s, sigma)` with itself doubles. -/
theorem mergeVal_diag (s : Bool) (sigma : Str) : mergeVal sigma s sigma s = (double
    sigma, s) := by
  induction sigma with
  | nil => rfl
  | cons a sigma ih => simp [mergeVal_cons_cons, ih]

/-- On the diagonal `merge(<(s, sigma), (s, sigma)>)` doubles `sigma`. -/
theorem mergeElem_diag (s : Bool) (sigma : Str) : mergeElem sigma s sigma s =
    shapeElem s (double sigma) := by
  simp [mergeElem, mergeVal_diag]

/-- **Exercise 5.16 (Scott 1981, PRG-19).** `merge(x, x) = d(x)` for all `x  in  |C|` (
    the doubling map
of Example 4.4). -/
theorem mergeMap_diag (x : C.Element) :
    mergeMap.toElementMap (pair x x) = dMap.toElementMap x := by
  have key : mergeMap.comp (paired (idMap C) (idMap C)) = dMap := by
    apply map_ext_C
    * intro sigma
      rw [toElementMap_comp, toElementMap_paired, toElementMap_idMap,
        show pair (strBot sigma) (strBot sigma) = pair (shapeElem false sigma) (
    shapeElem false sigma) from rfl,
        mergeMap_pair, mergeElem_diag, shapeElem_false, dMap_strBot]
```

```
    * intro sigma
      rw [toElementMap_comp, toElementMap_paired, toElementMap_idMap,
        show pair (strElem sigma) (strElem sigma) = pair (shapeElem true sigma) (
    shapeElem true sigma) from rfl,
        mergeMap_pair, mergeElem_diag, shapeElem_true, dMap_strElem]
  have hx := congrArg (fun m : ApproximableMap C C => m.toElementMap x) key
  simp only [toElementMap_comp, toElementMap_paired, toElementMap_idMap] at hx
  exact hx

/-- **Exercise 5.16 (Scott 1981, PRG-19).** `neg  comp  neg = id` as approximable maps:
     it suffices to
check on the finite elements `sigmaFalse`, `sigma`, where it is `flip  comp  flip = id
    `. -/
theorem negMap_comp_negMap : negMap.comp negMap = idMap C := by
  apply map_ext_C
  * intro sigma
    rw [toElementMap_comp, negMap_strBot, negMap_strBot, flip_flip, toElementMap_idMap
    ]
  * intro sigma


    rw [toElementMap_comp, negMap_strElem, negMap_strElem, flip_flip,
    toElementMap_idMap]

/-- **Exercise 5.16 (Scott 1981, PRG-19).** `neg(neg(x)) = x` for all `x  in  |C|`. -/
theorem negMap_negMap (x : C.Element) : negMap.toElementMap (negMap.toElementMap x) =
    x := by
  have h := negMap_comp_negMap
  rw [ <-  toElementMap_comp, h, toElementMap_idMap]

/-- **Exercise 5.16 -- the recursion for `neg`, case `0`.** `neg(0*x) = 1*neg(x)` as a
    map identity. -/
theorem neg_cons_false : negMap.comp (consMap false) = (consMap true).comp negMap :=
    by
  apply map_ext_C
  * intro sigma
    rw [toElementMap_comp, consMap_strBot, negMap_strBot, toElementMap_comp,
    negMap_strBot,
      consMap_strBot]
    rfl
  * intro sigma
    rw [toElementMap_comp, consMap_strElem, negMap_strElem, toElementMap_comp,
    negMap_strElem,
      consMap_strElem]
    rfl

/-- **Exercise 5.16 -- the recursion for `neg`, case `1`.** `neg(1*x) = 0*neg(x)` as a
    map identity. -/
theorem neg_cons_true : negMap.comp (consMap true) = (consMap false).comp negMap := by
  apply map_ext_C
  * intro sigma
    rw [toElementMap_comp, consMap_strBot, negMap_strBot, toElementMap_comp,
    negMap_strBot,
      consMap_strBot]
    rfl
  * intro sigma
    rw [toElementMap_comp, consMap_strElem, negMap_strElem, toElementMap_comp,
```

```
    negMap_strElem,
      consMap_strElem]
    rfl

theorem prod_principal_pair (b_0 : Bool) (sigma : Str) (b_1 : Bool) (tau : Str) :
    (prod C C).principal (prod_mem_prodNbhd (memC_shape b_0 sigma) (memC_shape b_1 tau
    ))
      = pair (shapeElem b_0 sigma) (shapeElem b_1 tau) := by
  apply Element.ext
  intro P
  rw [mem_principal]
  constructor
  * rintro <hP, hsub>
    obtain <X', Y', hX', hY', rfl> := hP
    obtain <hsX, hsY> := prodNbhd_subset_iff.mp hsub
    exact <X', Y', <hX', hsX>, <hY', hsY>, rfl>
  * rintro <X', Y', <hX', hsX>, <hY', hsY>, rfl>
    exact <prod_mem_prodNbhd hX' hY', prodNbhd_subset_iff.mpr <hsX, hsY>>

/-- Two maps `C  x  C  ->  C` agree as soon as they agree on every pair of finite
    elements. -/
theorem prodMap_ext {f g : ApproximableMap (prod C C) C}
    (h : forall  b_0 sigma b_1 tau, f.toElementMap (pair (shapeElem b_0 sigma) (
    shapeElem b_1 tau))
      = g.toElementMap (pair (shapeElem b_0 sigma) (shapeElem b_1 tau))) : f = g := by
  apply eq_of_toElementMap_principal
  intro W hW
  obtain <b_0, sigma, b_1, tau, rfl> :
      exists  b_0 sigma b_1 tau, W = prodNbhd (shape b_0 sigma) (shape b_1 tau) := by
    obtain <X, Y, hX, hY, rfl> := hW
    obtain <b_0, sigma, rfl> := memC_eq_shape hX
    obtain <b_1, tau, rfl> := memC_eq_shape hY
    exact <b_0, sigma, b_1, tau, rfl>
  have heq : (prod C C).principal hW = pair (shapeElem b_0 sigma) (shapeElem b_1 tau)
    :=
    prod_principal_pair b_0 sigma b_1 tau
  rw [heq]; exact h b_0 sigma b_1 tau

/-- **Exercise 5.16 (Scott 1981, PRG-19).** The defining recursion of `merge`:
`merge(?x, ?y) = ?*?*merge(x, y)` for all `x, y  in  |C|` and bits `?, ?`. -/
theorem mergeMap_cons (? ? : Bool) (x y : C.Element) :
    mergeMap.toElementMap
        (pair ((consMap ?).toElementMap x) ((consMap ?).toElementMap y))
      = (consMap ?).toElementMap
          ((consMap ?).toElementMap (mergeMap.toElementMap (pair x y))) := by
  have key :
      mergeMap.comp (paired ((consMap ?).comp (proj_0 C C)) ((consMap ?).comp (proj_1
    C C)))
        = ((consMap ?).comp (consMap ?)).comp mergeMap := by
    apply prodMap_ext
    intro b_0 sigma b_1 tau
    simp only [toElementMap_comp, toElementMap_paired, toElementMap_proj_0,
    toElementMap_proj_1,
      fst_pair, snd_pair, consMap_shapeElem, mergeMap_pair, mergeElem,
    mergeVal_cons_cons]
  have hx := congrArg (fun m : ApproximableMap (prod C C) C => m.toElementMap (pair x
```

```
    y)) key
  simp only [toElementMap_comp, toElementMap_paired, toElementMap_proj_0,
    toElementMap_proj_1,
    fst_pair, snd_pair] at hx
  exact hx

end

section
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap ExampleB
    Example44 Exercise419
variable {b : Type*}

/-- `tail(b(sigmaFalse)) = sigmaFalse`. -/
theorem tailMap_consMap_strBot (b : Bool) (sigma : Str) :
    tailMap.toElementMap ((consMap b).toElementMap (strBot sigma)) = strBot sigma :=
    by
  rw [consMap_strBot, tailMap_strBot]; rfl

/-- `tail(b(sigma)) = sigma`. -/
theorem tailMap_consMap_strElem (b : Bool) (sigma : Str) :
    tailMap.toElementMap ((consMap b).toElementMap (strElem sigma)) = strElem sigma :=
     by
  rw [consMap_strElem, tailMap_strElem]; rfl

end

section
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap ExampleB
    Example44 Exercise419

/-- The Thue-Morse substitution applied letterwise: each bit `b` is replaced by `b (not
     b)`. -/
def expand : Str  ->  Str
  | [] => []
  | b :: sigma => b :: (!b) :: expand sigma

@[simp] theorem expand_nil : expand [] = [] := rfl

@[simp] theorem expand_cons (b : Bool) (sigma : Str) : expand (b :: sigma) = b :: (!b)
     :: expand sigma := rfl

/-- The Thue-Morse bit-function: `tm n = ?(binary digits of n)`, the parity of the
    number of `1`s in
the binary expansion of `n` (Lambek's description). -/
def tm (n : Nat) : Bool := (Nat.bits n).foldr xor false

/-- **Exercise 5.16 (the operator `?`).** `?(x) = 0*merge(neg x, tail x)` as an
    approximable map. -/
def tmOp : ApproximableMap C C :=
  (consMap false).comp (mergeMap.comp (paired negMap tailMap))

@[simp] theorem tm_zero : tm 0 = false := by simp [tm, Nat.zero_bits]

/-- The length-`n` Thue-Morse prefix `[tm 0, tm 1, ..., tm (n-1)]`. -/
def tmList (n : Nat) : Str := (List.range n).map tm
```

```
@[simp] theorem tmList_zero : tmList 0 = [] := rfl

/-- **Exercise 5.16 (Scott 1981, PRG-19).** Scott's Thue-Morse element `t = 0*merge(neg
    t, tail t)`,
the least fixed point of `?`. -/
def tElt : C.Element := tmOp.fixElement

/-- `sigmaSigma* = Sigma*` iff `sigma = ?`. -/
theorem cone_univ_iff {sigma : Str} : cone sigma = Set.univ  <->  sigma = [] := by
  constructor
  * intro h
    exact List.prefix_nil.mp (mem_cone.mp (h  |>  Set.mem_univ ([] : Str)))
  * rintro rfl; exact cone_nil

/-- `expand` distributes over append (it is a monoid morphism `Sigma*  ->  Sigma*`). -/
theorem expand_append (sigma tau : Str) : expand (sigma ++ tau) = expand sigma ++
    expand tau := by
  induction sigma with
  | nil => rfl
  | cons b sigma ih => simp [ih]

/-- Every iterate of `expand` from `[0]` starts with `0` (so `?` keeps acting as `expand
    `). -/
theorem iterate_expand_cons (k : Nat) : exists  ?, (expand^[k] [false] : Str) = false
    :: ? := by
  induction k with
  | zero => exact <[], rfl>
  | succ k ih =>
    obtain <?, h?> := ih
    exact <true :: expand ?, by rw [Function.iterate_succ_apply', h?]; rfl>

/-- The second (total/partial-flag) component of an interleaving with a partial first
    input is always
`false`: a partial input keeps the output partial. -/
theorem mergeVal_snd_false : forall  (sigma tau : Str) (b_1 : Bool), (mergeVal sigma
    false tau b_1).2 = false
  | [], _, _ => rfl
  | _ :: _, [], _ => rfl
  | _ :: sigma, _ :: tau, b_1 => by simp only [mergeVal_cons_cons]; exact
    mergeVal_snd_false sigma tau b_1

/-- `(strBot tau)` has the cone of `sigma` as a neighbourhood iff `sigma` is a prefix
    of `tau`. -/
theorem strBot_mem_cone {sigma tau : Str} : (strBot tau).mem (cone sigma)  <->  sigma
    <+: tau := by
  rw [strBot, mem_principal]
  exact <fun h => cone_subset_cone.mp h.2, fun h => <memC_cone sigma, cone_subset_cone
    .mpr h>>

/-- `False = []False` in `C` (the cone of `?` is the master). -/
theorem strBot_nil : strBot [] = C.bot := by
  apply Element.ext
  intro Y
  rw [strBot, mem_principal, mem_bot, cone_nil, C_master]
  exact <fun h => Set.Subset.antisymm (Set.subset_univ _) h.2,
```

```
    fun h => <h  |>  C.master_mem, h  |>  Set.Subset.refl _>>

/-- **Exercise 5.16 -- the defining unfolding.** `t = 0*merge(neg t, tail t)`. -/
theorem tElt_unfold :
    (consMap false).toElementMap
        (mergeMap.toElementMap (pair (negMap.toElementMap tElt) (tailMap.toElementMap
    tElt)))
      = tElt := by
  have h := toElementMap_fixElement tmOp
  rw [tmOp, toElementMap_comp, toElementMap_comp, toElementMap_paired] at h
  exact h

theorem tmList_length (n : Nat) : (tmList n).length = n := by
  rw [tmList, List.length_map, List.length_range]

theorem tmList_succ (n : Nat) : tmList (n + 1) = tmList n ++ [tm n] := by
  simp only [tmList, List.range_succ, List.map_append, List.map_cons, List.map_nil]

/-- Thue-Morse prefixes are nested in the prefix order. -/
theorem tmList_prefix {n m : Nat} (h : n  <=  m) : tmList n <+: tmList m := by
  induction m with
  | zero => rw [Nat.le_zero.mp h]
  | succ m ih =>
    rcases Nat.lt_succ_iff_lt_or_eq.mp (Nat.lt_succ_of_le h) with hlt | rfl
    * exact (ih (Nat.lt_succ_iff.mp hlt)).trans (by rw [tmList_succ]; exact List.
    prefix_append _ _)
    * exact List.prefix_rfl

/-- The value of `?` on a partial element `sigmaFalse` is `(0 * weave sigma)False`, the
     leading `0` prepended to the
interleaving of `(flip sigma)False` and `(tail sigma)False`. -/
theorem tmOp_strBot (sigma : Str) :
    tmOp.toElementMap (strBot sigma)
      = strBot (false :: (mergeVal (flip sigma) false sigma.tail false).1) := by
  unfold tmOp
  rw [toElementMap_comp, toElementMap_comp, toElementMap_paired, negMap_strBot,
    tailMap_strBot,
    show (strBot (flip sigma)) = shapeElem false (flip sigma) from rfl,
    show (strBot sigma.tail) = shapeElem false sigma.tail from rfl, mergeMap_pair,
    mergeElem,
    mergeVal_snd_false, shapeElem_false, consMap_strBot]

/-- The even recurrence `tm (2n) = tm n` (appending a `0`-digit does not change the
    parity). -/
theorem tm_two_mul (n : Nat) : tm (2 * n) = tm n := by
  rcases Nat.eq_zero_or_pos n with rfl | hn
  * rfl
  * simp only [tm, Nat.bit0_bits n (by omega : n  !=  0), List.foldr_cons, Bool.
    false_xor]

/-- The odd recurrence `tm (2n+1) = not  tm n` (appending a `1`-digit flips the parity)
    . -/
theorem tm_two_mul_add_one (n : Nat) : tm (2 * n + 1) = !tm n := by
  simp only [tm, Nat.bit1_bits n, List.foldr_cons, Bool.true_xor]

/-- **The morphism step on prefixes.** `expand (tmList m) = tmList (2m)`: applying the
```

```
    Thue-Morse
morphism to a length-`m` prefix yields the length-`2m` prefix -- this is exactly the
    even/odd parity
recurrence in disguise. -/
theorem expand_tmList (m : Nat) : expand (tmList m) = tmList (2 * m) := by
  induction m with
  | zero => rfl
  | succ m ih =>
    have key : tmList (2 * (m + 1)) = tmList (2 * m) ++ [tm m, !tm m] := by
      rw [show 2 * (m + 1) = 2 * m + 1 + 1 by ring, tmList_succ, tmList_succ,
        tm_two_mul_add_one, tm_two_mul, List.append_assoc]
      rfl
    rw [tmList_succ, expand_append, ih, key, expand_cons, expand_nil]

/-- **The bridge.** The `k`-fold morphism iterate of `[0]` is the Thue-Morse parity
    prefix of length
`2?`: `expand?[0] = tmList (2?)`. -/
theorem expand_iterate_eq (k : Nat) : (expand^[k] [false] : Str) = tmList (2 ^ k) :=
    by
  induction k with
  | zero => rfl
  | succ k ih =>
    rw [Function.iterate_succ_apply', ih, expand_tmList, show 2 * 2 ^ k = 2 ^ (k + 1)
    by ring]

/-- The interleaving of `h :: flip ?` with `?` (both starting after a common head `h`)
    is
`h :: expand ?`. This is the heart of why `?` *is* the Thue-Morse morphism on the
    trajectory. -/
theorem weave_head : forall  (h : Bool) (? : Str), (mergeVal (h :: flip ?) false ?
    false).1 = h :: expand ?
  | h, [] => rfl
  | h, c :: ? => by
    simp only [flip_cons, mergeVal_cons_cons, expand_cons]
    rw [weave_head (!c) ?]

/-- **The Thue-Morse trajectory.** On a partial element whose string starts with `0`,
    `?` acts as the
Thue-Morse morphism `expand`. -/
theorem tmOp_strBot_expand (? : Str) :
    tmOp.toElementMap (strBot (false :: ?)) = strBot (expand (false :: ?)) := by
  rw [tmOp_strBot]
  simp only [flip_cons, List.tail_cons, expand_cons]
  rw [weave_head (!false) ?]

/-- **Exercise 5.16 (the approximants).** The `(k+1)`-st approximant of the least fixed
     point is the
partial element of the `k`-fold Thue-Morse morphism iterate of `[0]`:
`??^+1(False) = (expand?[0])False`. -/
theorem iterElem_succ_eq (k : Nat) :
    tmOp.iterElem (k + 1) = strBot (expand^[k] [false]) := by
  induction k with
  | zero =>
    rw [iterElem_succ, iterElem_zero,  <-  strBot_nil, tmOp_strBot]
    rfl
  | succ k ih =>
```

```
    obtain <?, h?> := iterate_expand_cons k
    rw [iterElem_succ, ih, h?, tmOp_strBot_expand,  <-  h?, Function.
    iterate_succ_apply']

end

section
open DomainTheory.NeighbourhoodSystems

/-- An **overlap** of period `p  >=  1` at offset `i`: a factor of length `2p+1` with
    period `p`. -/
def Overlap (i p : Nat) : Prop := 1  <=  p  /\  forall  k, k  <=  p  ->  tm (i + k) =
    tm (i + p + k)
end

section
open DomainTheory.NeighbourhoodSystems
open Example44 ExampleB NeighborhoodSystem

/-- Positions differing by `|a|` inside `a ++ (a ++ a)` carry the same symbol, for
    indices `< 2|a|`
(the three copies of `a` are identical). -/
theorem append_three_period {a : Type*} (a : List a) (m : Nat) (hm : m < 2 * a.length)
     :
    (a ++ (a ++ a))[m]? = (a ++ (a ++ a))[m + a.length]? := by
  rcases lt_or_ge m a.length with h | h
  * rw [List.getElem?_append_left h,
      List.getElem?_append_right (by omega : a.length  <=  m + a.length),
      show m + a.length - a.length = m by omega, List.getElem?_append_left h]
  * rw [List.getElem?_append_right h,
      List.getElem?_append_right (by omega : a.length  <=  m + a.length),
      show m + a.length - a.length = m by omega,
      List.getElem?_append_left (by omega : m - a.length < a.length),
      List.getElem?_append_right (by omega : a.length  <=  m)]

/-- The `j`-th bit of the length-`n` Thue-Morse prefix is `tm j`. -/
theorem tmList_getElem? {n j : Nat} (h : j < n) : (tmList n)[j]? = some (tm j) := by
  rw [tmList, List.getElem?_map, List.getElem?_range h]
  rfl

end

section
open DomainTheory.NeighbourhoodSystems

theorem tm_of_eq_two_mul {x y : Nat} (h : x = 2 * y) : tm x = tm y := by
  rw [h, tm_two_mul]

theorem tm_of_eq_two_mul_add_one {x y : Nat} (h : x = 2 * y + 1) : tm x = !tm y := by
  rw [h, tm_two_mul_add_one]

/-- An even-indexed symbol differs from its successor: `tm (2x)  !=  tm (2x+1)`. -/
theorem tm_two_mul_ne (x : Nat) : tm (2 * x)  !=  tm (2 * x + 1) := by
  rw [tm_two_mul, tm_two_mul_add_one]
  cases tm x <;> simp
```

```
/-- `tm x = tm (x+1)` forces `x` to be **odd**: equal consecutive symbols only straddle
     a block
boundary `(2m+1, 2m+2)`, never a block `(2m, 2m+1)`. -/
theorem odd_of_consec_eq {x : Nat} (h : tm x = tm (x + 1)) : Odd x := by
  rcases Nat.even_or_odd x with <m, hm> | hodd
  * exfalso; rw [hm,  <-  two_mul] at h; exact tm_two_mul_ne m h
  * exact hodd

/-- **No three consecutive equal symbols** (the period-`1` case): `not  (tm j = tm (j
    +1)  /\
tm (j+1) = tm (j+2))`. If both held, `j` and `j+1` would both be odd. -/
theorem no_three_consec (j : Nat) : not  (tm j = tm (j + 1)  /\  tm (j + 1) = tm (j +
    2)) := by
  rintro <h1, h2>
  have hj : Odd j := odd_of_consec_eq h1
  have hj1 : Odd (j + 1) := odd_of_consec_eq h2
  rcases hj with <a, rfl>
  rcases hj1 with <b, hb>
  omega

/-- **Overlap-freeness of the Thue-Morse sequence** (Scott 1981, PRG-19; Thue 1912).
    No overlap
exists at any offset and period. -/
theorem no_overlap (p : Nat) : forall  i, not  Overlap i p := by
  induction p using Nat.strong_induction_on with
  | _ p IH =>
    intro i hov
    obtain <hp1, hrel> := hov
    obtain <q, hq | hq> := Nat.even_or_odd' p
    * -- Even period `p = 2q`: contract to a period-`q` overlap.
      subst hq
      have hq1 : 1  <=  q := by omega
      obtain <a, hi | hi> := Nat.even_or_odd' i
      * -- `i = 2a`: even positions give `tm (a+c) = tm (a+q+c)`.
        refine IH q (by omega) a <hq1, fun c hc => ?_>
        have hk : 2 * c  <=  2 * q := by omega
        have := hrel (2 * c) hk
        rw [tm_of_eq_two_mul (show i + 2 * c = 2 * (a + c) by omega),
          tm_of_eq_two_mul (show i + 2 * q + 2 * c = 2 * (a + q + c) by omega)] at
    this
        exact this
      * -- `i = 2a+1`: odd positions give `tm (a+c) = tm (a+q+c)` after stripping the `
    not `.
        refine IH q (by omega) a <hq1, fun c hc => ?_>
        have hk : 2 * c  <=  2 * q := by omega
        have := hrel (2 * c) hk
        rw [tm_of_eq_two_mul_add_one (show i + 2 * c = 2 * (a + c) + 1 by omega),
          tm_of_eq_two_mul_add_one (show i + 2 * q + 2 * c = 2 * (a + q + c) + 1 by
    omega)] at this
        exact Bool.not_inj this
    * -- Odd period `p = 2q+1`.
      subst hq
      rcases Nat.eq_zero_or_pos q with rfl | hq1
      * -- `p = 1`: three consecutive equal symbols.
        refine no_three_consec i <?_, ?_>
        * have := hrel 0 (by omega); simpa using this
```

```
    * have := hrel 1 (by omega); simpa [show i + 1 + 1 = i + 2 by omega] using
this
  * -- `p = 2q+1  >=  3`.
    obtain <a, hi | hi> := Nat.even_or_odd' i
    * -- `i = 2a`: relations at `k = 0,1,2,3` give a run of three equal symbols at
`a+q`.
      subst hi
      have r0 := hrel 0 (by omega)
      have r1 := hrel 1 (by omega)
      have r2 := hrel 2 (by omega)
      have r3 := hrel 3 (by omega)
      rw [tm_of_eq_two_mul (show 2 * a + 0 = 2 * a by omega),
        tm_of_eq_two_mul_add_one (show 2 * a + (2 * q + 1) + 0 = 2 * (a + q) + 1
by omega)] at r0
      rw [tm_of_eq_two_mul_add_one (show 2 * a + 1 = 2 * a + 1 by omega),
        tm_of_eq_two_mul (show 2 * a + (2 * q + 1) + 1 = 2 * (a + q + 1) by omega)
] at r1
      rw [tm_of_eq_two_mul (show 2 * a + 2 = 2 * (a + 1) by omega),
        tm_of_eq_two_mul_add_one
          (show 2 * a + (2 * q + 1) + 2 = 2 * (a + q + 1) + 1 by omega)] at r2
      rw [tm_of_eq_two_mul_add_one (show 2 * a + 3 = 2 * (a + 1) + 1 by omega),
        tm_of_eq_two_mul (show 2 * a + (2 * q + 1) + 3 = 2 * (a + q + 2) by omega)
] at r3
      refine no_three_consec (a + q) <?_, ?_>
      * revert r0 r1; cases tm a <;> cases tm (a + q) <;> cases tm (a + q + 1) <;>
          simp_all
      * revert r1 r2 r3
        cases tm (a + 1) <;> cases tm (a + q + 1) <;> cases tm (a + q + 2) <;>
cases tm a <;>
          simp_all
    * -- `i = 2a+1`.
      subst hi
      rcases Nat.eq_or_lt_of_le hq1 with hq2 | hq2
      * -- `q = 1`, i.e. `p = 3`: a direct contradiction from the four relations.
        subst hq2
        have r0 := hrel 0 (by omega)
        have r1 := hrel 1 (by omega)
        have r2 := hrel 2 (by omega)
        have r3 := hrel 3 (by omega)
        rw [tm_of_eq_two_mul_add_one (show 2 * a + 1 + 0 = 2 * a + 1 by omega),
          tm_of_eq_two_mul (show 2 * a + 1 + (2 * 1 + 1) + 0 = 2 * (a + 2) by
omega)] at r0
        rw [tm_of_eq_two_mul (show 2 * a + 1 + 1 = 2 * (a + 1) by omega),
          tm_of_eq_two_mul_add_one
            (show 2 * a + 1 + (2 * 1 + 1) + 1 = 2 * (a + 2) + 1 by omega)] at r1

        rw [tm_of_eq_two_mul_add_one (show 2 * a + 1 + 2 = 2 * (a + 1) + 1 by
omega),
          tm_of_eq_two_mul (show 2 * a + 1 + (2 * 1 + 1) + 2 = 2 * (a + 3) by
omega)] at r2
        rw [tm_of_eq_two_mul (show 2 * a + 1 + 3 = 2 * (a + 2) by omega),
          tm_of_eq_two_mul_add_one
            (show 2 * a + 1 + (2 * 1 + 1) + 3 = 2 * (a + 3) + 1 by omega)] at r3
        revert r0 r1 r2 r3
        cases tm a <;> cases tm (a + 1) <;> cases tm (a + 2) <;> cases tm (a + 3)
<;> simp_all
```

```
        * -- `q  >=  2`, i.e. `p  >=  5`: relations at `k = 1,2,3,4` give a run of
    three at `a+q+1`.
          have r1 := hrel 1 (by omega)
          have r2 := hrel 2 (by omega)
          have r3 := hrel 3 (by omega)
          have r4 := hrel 4 (by omega)
          rw [tm_of_eq_two_mul (show 2 * a + 1 + 1 = 2 * (a + 1) by omega),
            tm_of_eq_two_mul_add_one
              (show 2 * a + 1 + (2 * q + 1) + 1 = 2 * (a + q + 1) + 1 by omega)] at
    r1
          rw [tm_of_eq_two_mul_add_one (show 2 * a + 1 + 2 = 2 * (a + 1) + 1 by
    omega),
            tm_of_eq_two_mul
              (show 2 * a + 1 + (2 * q + 1) + 2 = 2 * (a + q + 2) by omega)] at r2
          rw [tm_of_eq_two_mul (show 2 * a + 1 + 3 = 2 * (a + 2) by omega),
            tm_of_eq_two_mul_add_one
              (show 2 * a + 1 + (2 * q + 1) + 3 = 2 * (a + q + 2) + 1 by omega)] at
    r3
          rw [tm_of_eq_two_mul_add_one (show 2 * a + 1 + 4 = 2 * (a + 2) + 1 by
    omega),
            tm_of_eq_two_mul
              (show 2 * a + 1 + (2 * q + 1) + 4 = 2 * (a + q + 3) by omega)] at r4
          refine no_three_consec (a + q + 1) <?_, ?_>
          * revert r1 r2; cases tm (a + 1) <;> cases tm (a + q + 1) <;> cases tm (a
    + q + 2) <;>
            simp_all
          * revert r3 r4; cases tm (a + 2) <;> cases tm (a + q + 2) <;> cases tm (a
    + q + 3) <;>
            simp_all

/-- **The Thue-Morse sequence is cube-free.** A cube of period `p  >=  1` (the factor `
    a*a*a` with
`|a| = p`) is in particular an overlap, so none exists. -/
theorem no_cube (i p : Nat) (hp : 1  <=  p) :
    not  forall  k, k < 2 * p  ->  tm (i + k) = tm (i + p + k) := by
  intro hcube
  exact no_overlap p i <hp, fun k hk => hcube k (by omega)>

end

section
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap ExampleB
    Example44 Exercise419

/-- **Property (a), prefix form.** Every Thue-Morse parity prefix is a prefix of `t`:
`(tmList n)False  sqsub  t` for all `n`. -/
theorem tmList_le_tElt (n : Nat) : strBot (tmList n)  <=  tElt := by
  have hpre : tmList n <+: expand^[n] [false] := by
    rw [expand_iterate_eq]; exact tmList_prefix (Nat.le_of_lt (Nat.lt_two_pow_self))
  calc strBot (tmList n)  <=  strBot (expand^[n] [false]) :=
        strBot_le_strBot_iff.mpr hpre
    _ = tmOp.iterElem (n + 1) := (iterElem_succ_eq n).symm
    _  <=  tElt := iterElem_le_fixElement tmOp (n + 1)

/-- **Property (a), digit form.** The `n`-th digit of `t` is `tm n` (the parity of the
    binary digits
```

```
of `n`): the length-`(n+1)` prefix of `t` is `tmList n ++ [tm n]`. -/
theorem tElt_digit (n : Nat) : strBot (tmList n ++ [tm n])  <=  tElt := by
  have h := tmList_le_tElt (n + 1)
  rwa [tmList_succ] at h

/-- **Property (a) (Scott 1981, PRG-19; Lambek).** The finite-prefix characterization
    of `t`: a
string `sigma` is a prefix of `t` **iff** it is the length-`sigma.length` Thue-Morse
    parity prefix. Reading
off position `n`, the `n`-th digit of `t` is `tm n`, the parity of the binary digits of
     `n`. -/
theorem tElt_mem_cone_iff (sigma : Str) : tElt.mem (cone sigma)  <->  sigma = tmList
    sigma.length := by
  constructor
  * intro hsigma
    rw [show tElt = tmOp.fixElement from rfl, tmOp.fixElement_eq_iSupDirected,
      mem_iSupDirected] at hsigma
    obtain <n, hn> := hsigma
    -- `sigma` is a prefix of some Thue-Morse prefix `tmList N`.
    obtain <N, hpre> : exists  N, sigma <+: tmList N := by
      cases n with
      | zero =>
        rw [iterElem_zero, mem_bot, C_master, cone_univ_iff] at hn
        exact <0, by rw [hn]; exact List.nil_prefix>
      | succ k =>
        rw [iterElem_succ_eq, strBot_mem_cone, expand_iterate_eq] at hn
        exact <2 ^ k, hn>
    -- Both `sigma` and `tmList sigma.length` are the length-`sigma.length` prefix of `
    tmList N`.
    have hlen : sigma.length  <=  N := hpre.length_le.trans_eq (tmList_length _)
    have h1 : sigma = (tmList N).take sigma.length := List.prefix_iff_eq_take.mp hpre
    have h2 : tmList sigma.length = (tmList N).take sigma.length := by
      have := List.prefix_iff_eq_take.mp (tmList_prefix hlen)
      rwa [tmList_length] at this
    exact h1.trans h2.symm
  * intro hsigma
    have h := tmList_le_tElt sigma.length
    have hmem : (strBot (tmList sigma.length)).mem (cone sigma) :=
      strBot_mem_cone.mpr (hsigma  |>  List.prefix_rfl)
    exact h _ hmem

end

section
open DomainTheory.NeighbourhoodSystems
open Example44 ExampleB NeighborhoodSystem

/-- **Exercise 5.16 (Scott 1981, PRG-19) -- `t` is cube-free.** Scott's literal
    statement: `t` is not
of the form `u*a*a*a*v` for any finite prefix `u` and nonempty finite block `a`. (`v`
    is whatever
follows; we phrase "`u*a*a*a` is a prefix of `t`" as `(u*a*a*a)False  sqsub  t`.) -/
theorem tElt_cube_free (u a : Str) (ha : a  !=  []) :
    not  strBot (u ++ a ++ a ++ a)  <=  tElt := by
  intro hle
  -- `u*a*a*a` is a prefix of `t`, hence equals the Thue-Morse prefix of its length.
```

```
  have hmem : tElt.mem (cone (u ++ a ++ a ++ a)) :=
    hle _ (strBot_mem_cone.mpr List.prefix_rfl)
  have heq : u ++ a ++ a ++ a = tmList (u ++ a ++ a ++ a).length :=
    (tElt_mem_cone_iff _).mp hmem
  -- Read off a cube in `tm` at offset `|u|`, period `|a|`.
  set N := (u ++ a ++ a ++ a).length with hN
  have hlen : N = u.length + 3 * a.length := by simp only [hN, List.length_append];
    omega
  have hp : 1  <=  a.length := by
    cases a with
    | nil => exact (ha rfl).elim
    | cons _ _ => simp
  refine no_cube u.length a.length hp (fun k hk => ?_)
  -- Both `tm (|u|+k)` and `tm (|u|+|a|+k)` read the same bit of `u*a*a*a`.
  have hk3 : u.length + k < N := by omega
  have hk3' : u.length + a.length + k < N := by omega
  have e1 : (tmList N)[u.length + k]? = some (tm (u.length + k)) := tmList_getElem?
    hk3
  have e2 : (tmList N)[u.length + a.length + k]? = some (tm (u.length + a.length + k))
     :=
    tmList_getElem? hk3'
  -- Rewrite the prefix as `u ++ (a ++ (a ++ a))` and strip `u`.
  have hassoc : u ++ a ++ a ++ a = u ++ (a ++ (a ++ a)) := by
    simp [List.append_assoc]
  rw [ <-  heq, hassoc] at e1 e2
  rw [List.getElem?_append_right (by omega : u.length  <=  u.length + k),
    show u.length + k - u.length = k by omega] at e1
  rw [List.getElem?_append_right (by omega : u.length  <=  u.length + a.length + k),
    show u.length + a.length + k - u.length = k + a.length by omega] at e2
  -- Period: `(a++(a++a))[k]? = (a++(a++a))[k+|a|]?`.
  have hper := append_three_period a k (by omega)
  have : some (tm (u.length + k)) = some (tm (u.length + a.length + k)) := by
    rw [ <-  e1, hper, e2]
  exact Option.some.inj this
end

end DomainTheory.NeighbourhoodSystems.Exercise516
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

section
variable {a : Type*} {V : NeighborhoodSystem a}

namespace ApproximableMap

/-- The **left-hand side** `fun x. !y. tau(x, y)` of Proposition 5.4: `fix  comp  curry
    (tau)`. -/
def pfix (tau : ApproximableMap (prod V V) V) : ApproximableMap V V :=
  (fixMap V).comp (curry tau)

/-- The **operator** `fun g. fun x. tau(x, g(x))` of Proposition 5.4, as an
    approximable endomap of the
function space `(?  ->  ?)`: `recOp tau := curry(tau  comp  <p_1, eval>)`. -/
def recOp (tau : ApproximableMap (prod V V) V) : ApproximableMap (funSpace V V) (
    funSpace V V) :=
  curry (tau.comp (paired (proj_1 (funSpace V V) V) (evalMap V V)))
```

```
/-- The section `y  |->  tau(x, y)` of `tau : ?  x  ?  ->  ?` at a fixed `x`, as an
    approximable endomap of
`?` (the curried `tau` applied to `x`). -/
def section_2 (tau : ApproximableMap (prod V V) V) (x : V.Element) : ApproximableMap V
     V :=
  toApproxMap ((curry tau).toElementMap x)

/-- `pfix tau (x) = !y. tau(x, y)`, the least fixed point of the section at `x`. -/
theorem pfix_apply (tau : ApproximableMap (prod V V) V) (x : V.Element) :
    (pfix tau).toElementMap x = (section_2 tau x).fixElement := by
  rw [pfix, toElementMap_comp, fixMap_toElementMap, section_2]

/-- `recOp tau (g) (x) = tau(x, g(x))`. -/
theorem recOp_apply (tau : ApproximableMap (prod V V) V) (phi : (funSpace V V).Element
    ) (x : V.Element) :
    (toApproxMap ((recOp tau).toElementMap phi)).toElementMap x
      = tau.toElementMap (pair x ((toApproxMap phi).toElementMap x)) := by
  rw [recOp, toElementMap_curry_apply, toElementMap_comp, toElementMap_paired,
    toElementMap_proj_1, snd_pair, evalMap_apply]

theorem section_2_apply (tau : ApproximableMap (prod V V) V) (x y : V.Element) :
    (section_2 tau x).toElementMap y = tau.toElementMap (pair x y) := by
  rw [section_2, toElementMap_curry_apply]

/-- `pfix tau` is a fixed point of `recOp tau` (pointwise: `tau(x, pfix tau x) = pfix
    tau x`). -/
theorem pfix_isFixed (tau : ApproximableMap (prod V V) V) (x : V.Element) :
    tau.toElementMap (pair x ((pfix tau).toElementMap x)) = (pfix tau).toElementMap x
    := by
  rw [pfix_apply,  <-  section_2_apply, toElementMap_fixElement]

end ApproximableMap
end
end DomainTheory.NeighbourhoodSystems
end
```

3.7 Domain equations

Dependency diagram. Each box is one item. The arrowhead is on the item that imports. An arrow already implied by a longer path is left out. The introduction has no Lean file, so it is not shown. Blue is a definition, green a proposition or lemma, amber an example, red a theorem, and indigo an exercise.

3.7.1 Introduction

LECTURE VI INTRODUCTION TO DOMAIN EQUATIONS

The major reason for introducing the theory of domains is to have a notion of *computability* incorporating both finite and infinite elements. In our many examples already explored we have seen how functions (functionals, operators, combinators) can be defined on domains; owing to the property of *approximability (continuity)* of these functions, we have also seen how they can be "calculated" by finite approximation. In this lecture further examples of domains will be constructed – especially domains having infinite elements, which can be introduced in a variety of ways giving rise to interesting structural possibilities. The next

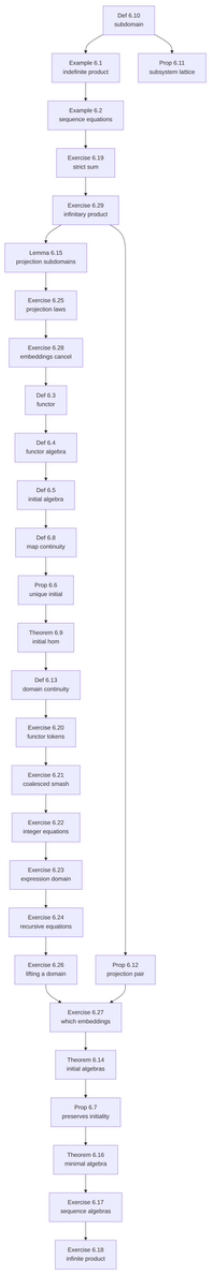


Figure 35: Domain equations.

lecture then treats a precise notion of *computability* appropriate to these domains; while the last lecture opens up new methods of domain construction.

3.7.2 Example 6.1 — indefinite product

EXAMPLE 6.1. Let D be fixed as a given domain. We are now familiar with a useful construct like $D \times D$ whose elements are ordered pairs $\langle x, y\rangle$ of elements x, y of D. The question is: can this construct be iterated? The answer is obviously yes, since $D \times (D \times D)$ and $(D \times D) \times (D \times D)$ and so on can be formed with elements $\langle x, \langle y, z\rangle\rangle$ and $\langle\langle u, v\rangle, \langle x, y\rangle\rangle$ and the like. But the real question is: can the construct be iterated *indefinitely*? AND can the results be collected together into a *single* domain? The answer is yes, but it requires a bit of work to get it right. The method to be introduced will be open to many variations, so more than one answer is possible, giving non-isomorphic domains.

In order to collect all the iterates into one large domain we give ourselves first a very big domain inside of which the desired family of neighbourhoods will be found. There are many ways to make this choice, and we are fixing on one that will keep the notation simple. We have often used binary sequences for examples and constructions, but for this example let us use

ternary sequences. Let $\Sigma = \{0, 1, 2\}$ and let Σ^* be all finite sequences from this three-letter alphabet. We will select subsets of Σ^* for our neighbourhoods. As Σ^* is countably infinite, it is without much loss of generality to assume that $\mathcal{D}$ is a neighbourhood system over Δ, where we take $\Delta \subseteq \Sigma^*$. Also without loss of generality we can assume $\emptyset \notin \mathcal{D}$. (Why?)

We wish to find another set $\Gamma \subseteq \Sigma^*$ to be the set of tokens for the new domain. After we find it, we will still have to say just which $X \subseteq \Gamma$ are appropriate for the structure we want.

The totality $\{X \mid X \subseteq \Sigma^*\}$ is, as a powerset, isomorphic to the set of elements of a domain: a point we have remarked several times. So, by the Fixed-Point Theorem we know there is a set $\Gamma \subseteq \Sigma^*$ where

$$\Gamma = 0\Delta \cup 1\Gamma \cup 2\Gamma.$$

In fact $\Gamma = \{1, 2\}^*0\Delta$, because we can say:

$$\{1, 2\}^* = \{\Lambda\} \cup 1\{1, 2\}^* \cup 2\{1, 2\}^*.$$

The domain we are looking for will be found as a domain $\mathcal{D}^\S$ over Γ. The reason for splitting Γ up, as shown in the equation above, is to ensure that if $X, Y \in \mathcal{D}^\S$ are two neighbourhoods in the system $\mathcal{D}^\S$, then $1X \cup 2Y$ has a chance of being also in $\mathcal{D}^\S$ because

$$1X \cup 2Y \subseteq \Gamma.$$

This will make $\mathcal{D}^\S \times \mathcal{D}^\S$ isomorphic to a part of $\mathcal{D}^\S$. If we make $\mathcal{D}$ also isomorphic to a part of $\mathcal{D}^\S$, then all the iterated products will be contained in $\mathcal{D}^\S$.

What is a neighbourhood system? Just a set of sets. But $\mathcal{PP}\Sigma^*$ is a domain (as a power set) and because $\Gamma \subseteq \Sigma^*$, we find

$$\mathcal{D}^\S \in \mathcal{PP}\Sigma^*$$

as an element. But elements of domains can often be defined by fixed-point equations. Indeed we will introduce $\mathcal{D}^\S$ this way:

$$\mathcal{D}^\S = \{\Gamma\} \cup \{0X \mid X \in \mathcal{D}\} \cup \{1X \cup 2Y \mid X, Y \in \mathcal{D}^\S\}.$$

The reader should stop to think why $\mathcal{D}^\S$ can be immediately seen to exist by writing such an equation. Of course another way to describe $\mathcal{D}^\S$ is to say it is the least family of sets containing (i) the set Γ, (ii) the sets $0X$ for X in the given system $\mathcal{D}$, and (iii) sets $1X \cup 2Y$ whenever it already contains X and Y (closure under a set-forming operation). By saying "least" we mean (iv) nothing else belongs to $\mathcal{D}^\S$ except as allowed by (i)–(iii); this makes the truth of the equation for $\mathcal{D}^\S$ clear. So $\mathcal{D}^\S$ exists as a family of sets, but what good is it?

By our construction of Γ, all the sets we put into $\mathcal{D}^\S$ are subsets of Γ (why?); so $\mathcal{D}^\S$ has a chance of being a system over Γ if we can check the closure under intersection. So suppose $Z \subseteq X \cap Y$ where $Z, X, Y \in \mathcal{D}^\S$; we want to show $X \cap Y \in \mathcal{D}^\S$. We argue by induction on the number of steps required to put X and Y into $\mathcal{D}^\S$ by (i)–(iii). There are several cases.

If $X = \Gamma$ or $Y = \Gamma$, there is nothing to prove, because both sets are subsets of Γ. We note that $\emptyset \notin \mathcal{D}^\S$, because (i)–(iii) cannot introduce $\emptyset$ as a member of $\mathcal{D}^\S$. So, if $X = 0A$ for $A \in \mathcal{D}$, then Y must have this form also (if it is not Γ), because

$$0A \cap (1B \cup 2C) = \emptyset.$$

(That is, if Y had the form (iii), then $Z = \emptyset$ would be a consequence, which is impossible.) Thus, if $X = 0A$ for $A \in \mathcal{D}$, then $Y = 0B$ for some $B \in \mathcal{D}$. But by the same reasoning $Z = 0C$ for some $C \in \mathcal{D}$ also. But the relationship $0C \subseteq 0A \cap 0B$ is equivalent to $C \subseteq A \cap B$. We see, therefore, that $A \cap B \in \mathcal{D}$, and so

$$X \cap Y = 0A \cap 0B = 0(A \cap B)$$

must belong to $\mathcal{D}^\S$.

The final case has X, Y, Z all of the form (iii):

$$X = 1A_1 \cup 2A_2,$$

$$Y = 1B_1 \cup 2B_2, \text{ and}$$

$$Z = 1C_1 \cup 2C_2.$$

We can think of the A_i and B_i put into $\mathcal{D}^\S$ earlier and the intersection result as being already established for them. But the relationship $Z \subseteq X \cap Y$ is equivalent to $C_i \subseteq A_i \cap B_i$ for $i = 1, 2$. Therefore $A_i \cap B_i \in \mathcal{D}^\S$, and so does

$$X \cap Y = (1A_1 \cup 2A_2) \cap (1B_1 \cup 2B_2) = 1(A_1 \cap B_1) \cup 2(A_2 \cap B_2).$$

We have now seen that $D^\S$ is a neighbourhood system, but why was it constructed that way? The reason is simply this isomorphism (or domain equation):

$$D^\S \cong D + (D^\S \times D^\S),$$

as can be seen by reference to the equation for $D^\S$ and the definitions of $+$ and $\times$. What are the elements of $D^\S$? There is always

$$\perp = \{\Gamma\}.$$

Next if $x \in |D|$ we define

$$x^{\S} = \{\Gamma\} \cup \{0X \mid X \in x\}.$$

That gives an isomorphic injection

$$\lambda x.\, x^{\S} : D \to D^{\S}.$$

Then for $x, y \in |D^{\S}|$ we can define

$$\langle x, y\rangle = \{\Gamma\} \cup \{1X \cup 2Y \mid X \in x \text{ and } Y \in y\}.$$

We have another isomorphic injection

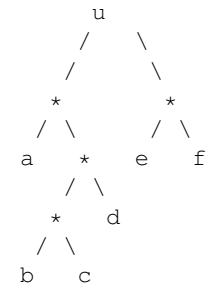
$$\lambda x, y.\, \langle x, y\rangle : D^{\S} \times D^{\S} \to D^{\S}.$$

Indeed by looking at the neighbourhood definition of $D^{\S}$ we conclude that the *finite* elements of $D^{\S}$ are exactly those that are either of the form (i) $\perp$, or (ii) $a^{\S}$, where a is finite in $|D|$, or (iii) $\langle a, b\rangle$, where a and b are previously obtained finite elements of $|D^{\S}|$.

Suppose $a, \ldots, f$ are finite in $|D|$. We can picture the element

$$u = \langle\langle a^{\S}, \langle\langle b^{\S}, c^{\S}\rangle, d^{\S}\rangle\rangle, \langle e^{\S}, f^{\S}\rangle\rangle$$

in $|D^{\S}|$ as a tree:

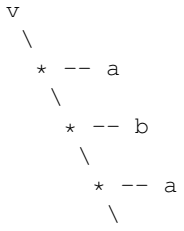


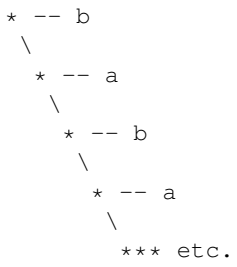
Note that the tree has binary branching with the elements of $|D|$ at the ends of the branches. Any such tree could be given a notation as an element of $|D^{\S}|$. The finite elements of $|D^{\S}|$ correspond exactly to such finite trees.

What of the infinite elements of $|D^{\S}|$? Are there infinite trees? Let $a, b \in |D^{\S}|$ be any elements of $|D^{\S}|$. Since pairing is an approximable mapping, we can solve the fixed-point equation

$$v = \langle a, \langle b, v\rangle\rangle.$$

In pictures we can diagram v roughly as:





The word is "roughly" here, since if a or b were not in the $|D|$ part of $|D^{\S}|$, then in the diagram the letters "a" and "b" should be replaced by the corresponding tree diagrams for a and b.

Suppose that a and b are finite. Then we can easily see that the infinite tree v is the limit of the following sequence of finite trees:

$$v_0 = \bot,$$

$$v_{n+1} = \langle a, \langle b, v_n \rangle \rangle,$$

and

$$v = \bigsqcup_{n=0}^{\infty} v_n.$$

The reader should think how to explain from tree diagrams the approximation relation $v_n \sqsubseteq v$ and more general such relationships.

We could call $D^{\S}$ a tree algebra over D. There may be others. A general one is a structure of the form

$$\langle E, \text{in}, \text{pair} \rangle,$$

where

$$\text{in} : D \to E, \text{ and}$$

$$\text{pair} : E \times E \to E.$$

The algebra

$$\langle D^{\S}, \lambda x.\, x^{\S}, \lambda x, y.\, \langle x, y \rangle \rangle,$$

however, is a very special one: it is "minimal" among all tree algebras over D in a sense we shall have to make precise.

To do this think of how E and $D^{\S}$ can differ. In view of the isomorphism that $D^{\S}$ satisfies, the injection of D and the pairing are one-one, so no "information" is lost by these mappings. The same may not at all be true of E, but it is reasonable to think that at least we can define an approximable mapping $g : D^{\S} \to E$ where

$$(1) \quad g(\bot) = \bot_E,$$

$$(2) \quad g(x^{\S}) = \text{in}(x), \text{ for } x \in |D|, \text{ and}$$

$$(3) \quad g(\langle x, y\rangle) = \text{pair}(g(x), g(y)), \text{ for } x, y \in |D^{\S}|.$$

By what we said earlier, g will be uniquely determined by (1)–(3), because these equations tell us exactly how to calculate g on all finite elements of $|D^{\S}|$. An approximable mapping is always determined by its action on the finite elements. But why does g exist?

It would not be too hard to give an inductive construction of g as a neighbourhood relation, but a fixed-point equation is easier to write down for the same purpose. We need, though, to have the inverse ("predecessor") functions:

$$\text{out} : D^{\S} \to D$$

$$\text{proj}_i : D^{\S} \to D^{\S}, \text{ for } i = 0, 1,$$

where

$$\text{out}(x^{\S}) = x,$$

$$\text{proj}_0(\langle x, y\rangle) = x, \text{ and}$$

$$\text{proj}_1(\langle x, y\rangle) = y.$$

We also need

$$\text{atom} : D^{\S} \to T,$$

where

$$\text{atom}(x^{\S}) = \text{true}, \text{ and}$$

$$\text{atom}(\langle x, y\rangle) = \text{false}.$$

We can then write

$$g(x) = \text{cond}\big(\text{atom}(x), \text{in}(\text{out}(x)), \text{pair}(g(\text{proj}_0(x)), g(\text{proj}_1(x)))\big).$$

This g exists by fixed-point theory, and it satisfies (1)–(3) by what we know about the structure of $|D^{\S}|$. As we said, g is unique because the values on finite elements are fixed.

In algebraic language g is a *homomorphism* of tree algebras; and $D^{\S}$ is called an *initial algebra*, because for any tree algebra E there is a unique homomorphism $g : D^{\S} \to E$. We note at once that any two initial algebras are isomorphic. For if D^* were another, there would exist homomorphisms in both directions between $D^{\S}$ and D^*. But the compositions of homomorphisms are again homomorphisms, and in the case of $D^{\S}$ if we go from $D^{\S}$ to D^* and back to $D^{\S}$, the result must be the identity. The reason is that the identity can be the only homomorphism of an initial algebra into itself. We thus have a precise meaning of the minimal character of $D^{\S}$. But note it still took a construction to show that the domain $D^{\S}$ exists. □

Example_6_1.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.Definition_6_10

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Exercise_3_18
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Examples_1_2_1_5_revisited

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem
namespace Example61
variable {a : Type*}
/- concordance: source L3218 EXAMPLE 6.1 indefinite product -/
/-- **Example 6.1 (Scott 1981, PRG-19).** The neighbourhoods of `D^S`, defined
    inductively as the
*least* family of subsets of `Gamma` containing (i) `Gamma`, (ii) `0X` for `X  in  ?`,
    and (iii) `1P  U  2Q`
whenever it already contains `P` and `Q`. This is Scott's fixed-point family
`?^S = {Gamma}  U  {0X ? X  in  ?}  U  {1P  U  2Q ? P, Q  in  ?^S}`. -/
inductive MemS (D : NeighborhoodSystem a) : Set (List Bool  x  a)  ->  Prop
  | gamma : MemS D (Gamma D)
  | zero {X : Set a} : D.mem X  ->  MemS D (embZero X)
  | pair {P Q : Set (List Bool  x  a)} : MemS D P  ->  MemS D Q  ->  MemS D (embPair P
     Q)

end Example61
end

section
open NeighborhoodSystem ApproximableMap ExampleB

namespace Example62

/-- `bX = {b :: w' ? w'  in  X}`: the `b`-prefixed copy of a neighbourhood `X` (Scott's
     `0X` for
`b = false` and `1X` for `b = true`). -/
def embBit (b : Bool) (X : Set Str) : Set Str := {w | exists  w', w = b :: w'  /\  w'
     in  X}

@[simp] theorem mem_embBit {b : Bool} {X : Set Str} {w : Str} :
    w  in  embBit b X  <->  exists  w', w = b :: w'  /\  w'  in  X := Iff.rfl

/-- Scott's standing assumption `{}  notin  B`: every neighbourhood of `B` is nonempty
    (cones contain their
generating prefix). -/
theorem B_nonempty : forall  X, B.mem X  ->  X.Nonempty := by
  rintro X <sigma, rfl>; exact <sigma, List.prefix_rfl>

/-- The right-hand side of Scott's domain equation: the sum system `B + B` (Exercise
    3.18). -/
```

```
abbrev BB : NeighborhoodSystem (Option (Str ? Str)) := sum B B B_nonempty B_nonempty

/-- `bX = b*X` is exactly the single-bit prepend of Example 1.B. -/
theorem embBit_eq_prepend (b : Bool) (X : Set Str) : embBit b X = prepend [b] X := by
  ext w
  simp only [mem_embBit, mem_prepend]
  constructor
  * rintro <w', rfl, hX>; exact <w', hX, rfl>
  * rintro <tau, hX, rfl>; exact <tau, rfl, hX>

/-- `b(sigmaSigma*) = (bsigma)Sigma*`: prepending a bit to a cone gives the cone of the
     longer prefix. -/
theorem embBit_cone (b : Bool) (sigma : Str) : embBit b (cone sigma) = cone (b ::
    sigma) := by
  rw [embBit_eq_prepend, prepend_cone]; rfl

theorem embBit_inter (b : Bool) (X X' : Set Str) :
    embBit b X  I  embBit b X' = embBit b (X  I  X') := by
  ext w
  simp only [Set.mem_inter_iff, mem_embBit]
  constructor
  * rintro <<w', rfl, hX>, w'', heq, hX'>
    rw [List.cons.injEq] at heq
    obtain <-, rfl> := heq
    exact <w', rfl, hX, hX'>
  * rintro <w', rfl, hX, hX'>
    exact <<w', rfl, hX>, <w', rfl, hX'>>

theorem embBit_inter_ne {b b' : Bool} (h : b  !=  b') (X Y : Set Str) :
    embBit b X  I  embBit b' Y = {} := by
  ext w
  simp only [Set.mem_inter_iff, mem_embBit, Set.mem_empty_iff_false, iff_false,
    not_and]
  rintro <w', rfl, -> <w'', heq, ->
  rw [List.cons.injEq] at heq
  exact h heq.1

theorem embBit_ne {b b' : Bool} (h : b  !=  b') {X Y : Set Str} (hX : X.Nonempty) :
    embBit b X  !=  embBit b' Y := by
  intro heq
  obtain <w', hw'> := hX
  have hmem : (b :: w')  in  embBit b' Y := heq  |>  (<w', rfl, hw'> : (b :: w')  in
    embBit b X)
  obtain <w'', he, -> := hmem
  rw [List.cons.injEq] at he
  exact h he.1

theorem embBit_subset {b : Bool} {X X' : Set Str} :
    embBit b X  subseteq  embBit b X'  <->  X  subseteq  X' := by
  constructor
  * intro h w' hw'
    obtain <w'', heq, hX'> := h <w', rfl, hw'>
    rw [List.cons.injEq] at heq
    obtain <-, rfl> := heq
    exact hX'
  * rintro h w <w', rfl, hX>
```

```
    exact <w', rfl, h hX>

theorem embBit_injective {b : Bool} {X X' : Set Str} (h : embBit b X = embBit b X') :
    X = X' :=
  Set.Subset.antisymm (embBit_subset.mp h.subset) (embBit_subset.mp h.symm.subset)

end Example62
end
end DomainTheory.NeighbourhoodSystems
end
```

3.7.3 Example 6.2 — sequence domain equations

EXAMPLE 6.2. Our staple examples B and C satisfy "domain equations" in the form of isomorphisms as we have previously seen. Indeed

$$B \cong B + B, \text{ and}$$

$$C \cong \{\{\Lambda\}\} + C + C,$$

where if we liked we could construct both systems over $\{0,1\}^*$ and have:

$$B = \{\{0,1\}^*\} \cup \{0X \mid X \in B\} \cup \{1X \mid X \in B\}, \text{ and}$$

$$C = \{\{0,1\}^*\} \cup \{\{\Lambda\}\} \cup \{0X \mid X \in C\} \cup \{1X \mid X \in C\}.$$

We leave to the exercises the explanations of what kinds of algebras B and C are and why they are initial. Here we want to propose a simple, yet interesting generalization of B.

Consider this domain equation:

$$A \cong A^n + A^n,$$

where A^n stands for the n-fold cartesian power of A. We can, with the aid of some encoding solve this equation as a neighbourhood system over $\{0,1\}^*$ as follows:

$$A = \{\{0,1\}^*\} \cup \bigcup_{i=0,1} \left\{ i \bigcup_{j<n} 1^j 0 X_j \mid X_j \in A \text{ for all } j < n \right\}.$$

For instance, if $n = 3$, then a typical neighbourhood in A is something like

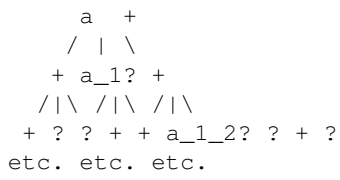
$$00X_0 \cup 010X_1 \cup 0110X_2,$$

where $X_0, X_1, X_2 \in A$. The first '0' could also be a '1' in front of each of the terms.

In words, an element of A (other than $\perp$) is an n-tuple of elements of A: but there are two separate copies of these, the left one and the right one. We can write for $a \in |A|$

$$a = \pm\langle a_0, a_1, \ldots, a_{n-1}\rangle,$$

where $+$ is chosen if a is on the right, and $-$ if on the left. As a tree diagram a might look like this for $n = 3$:



That is, a is an infinite ternary tree with $+$ or $-$ labels at each node. If each node (subtree) is truly infinite, the element is *total*; if $\perp$ is ever encountered, it is only *partial*; if every branch ends with $\perp$, the tree is a *finite element* of $|A|$.

What can be done with such trees? Let $\sigma \in \{0, 1, \ldots, n-1\}^*$ be a finite sequence of "digits" each less than n. We let $\Sigma = \{0, 1, \ldots, n-1\}$. We can define for $a \in |A|$ the operation $\sigma \mapsto a\sigma$ by recursion on σ:

$$a\Lambda = a, \text{ and}$$

$$ai\sigma = (a_i)\sigma.$$

The $a\sigma$ are just the *subtrees* of a with σ as a *selector*. We also have a map

$$\mathrm{pos} : A \to T$$

where

$$\mathrm{pos}(+\langle a_0, a_1, \ldots, a_{n-1}\rangle) = \mathrm{true}, \text{ and}$$

$$\mathrm{pos}(-\langle a_0, a_1, \ldots, a_{n-1}\rangle) = \mathrm{false}.$$

We say that a (total) tree a is *eventually periodic* iff the set $\{a\sigma \mid \sigma \in \Sigma^*\}$ is finite. The result is that the "language"

$$L_a = \{\sigma \in \Sigma^* \mid \mathrm{pos}(a\sigma) = \mathrm{true}\}$$

corresponding to an eventually periodic tree is always a *regular event* of automata theory, and every such language has this form. In fact, a just represents the initial state of an automaton, and $a\sigma$ represents the state after "reading" a tape σ. $\square$

In order to formulate more generally the idea of a domain equation and initial algebra, we must introduce a small amount of the terminology of category theory. To be as specific as possible, think of systems D over sets $\Delta \subseteq \Sigma^*$ with $\Sigma = \{0, 1\}$, say. They form quite an interesting category with respect to the approximable maps $f : D \to D'$. Recall that to be a category of "domains" and "maps" all that is required is an associative composition $g \circ f$ of maps with identity maps $I : D \to D$ for each domain of the category. And this we certainly have for the systems indicated. And there are many other categories waiting around: for instance, restrict systems to those where $\emptyset \notin D$. This is not much of a restriction, as every system is isomorphic to one like this. Or restrict the maps to being the strict maps $f : D \to D'$ where $f(\perp_D) = \perp_{D'}$. This is an essentially different, though related category. We shall find many others.

What examples 6.1 and 6.2 suggest is the notion of a construct which makes new domains out of old. For example, with D fixed, 6.1 suggests for any domain X over $\Gamma \subseteq \Sigma^*$ a domain

$$T(X) = D + (X \times X).$$

More specifically (converting from $\Sigma = \{0,1,2\}$ to $\Sigma = \{0,1\}$) we could write

$$T(X) = \{\Gamma'\} \cup \{0X \mid X \in D\} \cup \{10X \cup 11Y \mid X, Y \in X\},$$

where we have $\Gamma' = 0\Delta \cup 10\Gamma \cup 11\Gamma$. (By the way, here we definitely want to assume $\emptyset \notin D$ and $\emptyset \notin X$ and to get $\emptyset \notin T(X)$.) This construct is an example of a *functor*, a notion that can be defined abstractly on any category.

Example_6_2.lean

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Example_4_4
public import Mathlib.Computability.MyhillNerode
public import DomainTheory.NeighbourhoodSystems.DomainEquations.Example_6_1
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Exercise_3_15

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem ApproximableMap ExampleB
namespace Example62

/- concordance: source L3510 EXAMPLE 6.2 sequence domain equations -/
/-- **Example 6.2 -- the shape of a `B`-neighbourhood.** Every neighbourhood of `B` is
    either the
master `Sigma* = cone []`, a `0`-copy `0X` with `X  in  B`, or a `1`-copy `1X` with `X
    in  B`. -/
theorem memB_cases {W : Set Str} (hW : B.mem W) :
    W = Set.univ  \/  (exists  X, B.mem X  /\  W = embBit false X)  \/  (exists  Y, B.
    mem Y  /\  W = embBit true Y) := by
  obtain <sigma, rfl> := hW
  cases sigma with
  | nil => exact Or.inl cone_nil
  | cons b sigma' =>
    cases b with
    | false => exact Or.inr (Or.inl <cone sigma', memB_cone sigma', (embBit_cone false
     sigma').symm>)
    | true => exact Or.inr (Or.inr <cone sigma', memB_cone sigma', (embBit_cone true
    sigma').symm>)

/-- Prepending a bit lands back in `B`. -/
theorem memB_embBit (b : Bool) {X : Set Str} (hX : B.mem X) : B.mem (embBit b X) := by
  rw [embBit_eq_prepend]; exact memB_prepend [b] hX

/-- If `bW  in  B` then `W  in  B`: a `b`-prefixed neighbourhood that lands in `B` must
     be `b*(cone w')`,
so `W = cone w'`. -/
theorem memB_embBit_inv {b : Bool} {W : Set Str} (h : B.mem (embBit b W)) : B.mem W :=
     by
  obtain <sigma, hsigma> := h
  have hmem : sigma  in  embBit b W := hsigma  |>  (show sigma  in  cone sigma from
    List.prefix_rfl)
  obtain <w', rfl, -> := hmem
```

```
  rw [ <-  embBit_cone] at hsigma
  rw [embBit_injective hsigma]
  exact memB_cone w'

theorem nil_not_mem_embBit {b : Bool} {X : Set Str} : ([] : Str)  notin  embBit b X :=
    by
  rintro <w', heq, ->; exact absurd heq (by simp)

theorem embBit_ne_univ (b : Bool) (X : Set Str) : embBit b X  !=  Set.univ := by
  intro h; exact nil_not_mem_embBit (X := X) (b := b) (by rw [h]; trivial)

theorem sum_mem_inj_0_inv {X : Set Str} (h : BB.mem (inj_0 X)) : B.mem X := by
  rcases h with h0 | <X', hX', heq> | <Y', hY', heq>
  * exact absurd (h0  |>  none_mem_sumMaster) none_mem_inj_0
  * rw [inj_0_injective heq]; exact hX'
  * obtain <b, hb> := B_nonempty Y' hY'
    exact absurd (heq  |>  ir_mem_inj_1.mpr hb) ir_mem_inj_0

theorem sum_mem_inj_1_inv {Y : Set Str} (h : BB.mem (inj_1 Y)) : B.mem Y := by
  rcases h with h0 | <X', hX', heq> | <Y', hY', heq>
  * exact absurd (h0  |>  none_mem_sumMaster) none_mem_inj_1
  * obtain <a, ha> := B_nonempty X' hX'
    exact absurd (heq  |>  il_mem_inj_0.mpr ha) il_mem_inj_1
  * rw [inj_1_injective heq]; exact hY'

theorem sum_mem_nonempty {W : Set (Option (Str ? Str))} (h : BB.mem W) : W.Nonempty :=
    by
  rcases h with rfl | <X, hX, rfl> | <Y, hY, rfl>
  * exact <none, none_mem_sumMaster>
  * exact inj_0_nonempty (B_nonempty X hX)
  * exact inj_1_nonempty (B_nonempty Y hY)

/-- **Example 6.2 -- forward half of `B  ~=  B + B`.** An element `x` of `B` is sent to
    the sum element
recording, for each branch, whether `x` reaches the `0`-copy `0X` (left summand) or the
    `1`-copy `1Y`
(right summand). -/
def toBB (x : B.Element) : BB.Element where
  mem W := W = sumMaster B B
     \/  (exists  X, B.mem X  /\  W = inj_0 X  /\  x.mem (embBit false X))
     \/  (exists  Y, B.mem Y  /\  W = inj_1 Y  /\  x.mem (embBit true Y))
  sub := by
    rintro W (rfl | <X, hX, rfl, -> | <Y, hY, rfl, ->)
    * exact Or.inl rfl
    * exact Or.inr (Or.inl <X, hX, rfl>)
    * exact Or.inr (Or.inr <Y, hY, rfl>)
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | <X, hX, rfl, hzX> | <Y, hY, rfl, hzY>)
      (rfl | <X', hX', rfl, hzX'> | <Y', hY', rfl, hzY'>)
    * exact Or.inl (by rw [Set.inter_self])
    * exact Or.inr (Or.inl <X', hX', by rw [sumMaster_inter_inj_0 hX'], hzX'>)
    * exact Or.inr (Or.inr <Y', hY', by rw [sumMaster_inter_inj_1 hY'], hzY'>)
    * exact Or.inr (Or.inl <X, hX, by rw [Set.inter_comm, sumMaster_inter_inj_0 hX],
    hzX>)
    * refine Or.inr (Or.inl <X  I  X', ?_, by rw [inj_0_inter], ?_>)
```

```
      * have hz := x.inter_mem hzX hzX'; rw [embBit_inter] at hz; exact
    memB_embBit_inv (x.sub hz)
      * have hz := x.inter_mem hzX hzX'; rwa [embBit_inter] at hz
    * exfalso
      have hz := x.inter_mem hzX hzY'
      rw [embBit_inter_ne (show (false : Bool)  !=  true by decide)] at hz
      obtain <t, ht> := B_nonempty _ (x.sub hz); exact Set.notMem_empty t ht
    * exact Or.inr (Or.inr <Y, hY, by rw [Set.inter_comm, sumMaster_inter_inj_1 hY],
    hzY>)
    * exfalso
      have hz := x.inter_mem hzY hzX'
      rw [embBit_inter_ne (show (true : Bool)  !=  false by decide)] at hz
      obtain <t, ht> := B_nonempty _ (x.sub hz); exact Set.notMem_empty t ht
    * refine Or.inr (Or.inr <Y  I  Y', ?_, by rw [inj_1_inter], ?_>)
      * have hz := x.inter_mem hzY hzY'; rw [embBit_inter] at hz; exact
    memB_embBit_inv (x.sub hz)
      * have hz := x.inter_mem hzY hzY'; rwa [embBit_inter] at hz
  up_mem := by
    rintro W W' (rfl | <X, hX, rfl, hzX> | <Y, hY, rfl, hzY>) hW' hsub
    * exact Or.inl (eq_sumMaster_of_subset hW' hsub)
    * rcases hW' with rfl | <X', hX', rfl> | <Y', hY', rfl>
      * exact Or.inl rfl
      * refine Or.inr (Or.inl <X', hX', rfl, ?_>)
        exact x.up_mem hzX (memB_embBit false hX') (embBit_subset.mpr (
    inj_0_subset_inj_0.mp hsub))
      * exfalso
        obtain <a, ha> := B_nonempty X hX
        exact absurd (hsub (il_mem_inj_0.mpr ha)) il_mem_inj_1
    * rcases hW' with rfl | <X', hX', rfl> | <Y', hY', rfl>
      * exact Or.inl rfl
      * exfalso
        obtain <b, hb> := B_nonempty Y hY
        exact absurd (hsub (ir_mem_inj_1.mpr hb)) ir_mem_inj_0
      * refine Or.inr (Or.inr <Y', hY', rfl, ?_>)
        exact x.up_mem hzY (memB_embBit true hY') (embBit_subset.mpr (
    inj_1_subset_inj_1.mp hsub))

@[simp] theorem toBB_mem_inj_0 {x : B.Element} {X : Set Str} (hX : B.mem X) :
    (toBB x).mem (inj_0 X)  <->  x.mem (embBit false X) := by
  constructor
  * rintro (h0 | <X', hX', heq, hz> | <Y', hY', heq, hz>)
    * exact absurd (h0  |>  none_mem_sumMaster) none_mem_inj_0
    * rwa [inj_0_injective heq]
    * obtain <a, ha> := B_nonempty X hX
      exact absurd (heq  |>  il_mem_inj_0.mpr ha) il_mem_inj_1
  * intro hz; exact Or.inr (Or.inl <X, hX, rfl, hz>)

@[simp] theorem toBB_mem_inj_1 {x : B.Element} {Y : Set Str} (hY : B.mem Y) :
    (toBB x).mem (inj_1 Y)  <->  x.mem (embBit true Y) := by
  constructor
  * rintro (h0 | <X', hX', heq, hz> | <Y', hY', heq, hz>)
    * exact absurd (h0  |>  none_mem_sumMaster) none_mem_inj_1
    * obtain <a, ha> := B_nonempty X' hX'
      exact absurd (heq  |>  il_mem_inj_0.mpr ha) il_mem_inj_1
    * rwa [inj_1_injective heq]
  * intro hz; exact Or.inr (Or.inr <Y, hY, rfl, hz>)
```

```
/-- **Example 6.2 -- inverse half of `B  ~=  B + B`.** -/
def fromBB (s : BB.Element) : B.Element where
  mem W := W = Set.univ
     \/  (exists  X, B.mem X  /\  W = embBit false X  /\  s.mem (inj_0 X))
     \/  (exists  Y, B.mem Y  /\  W = embBit true Y  /\  s.mem (inj_1 Y))
  sub := by
    rintro W (rfl | <X, hX, rfl, -> | <Y, hY, rfl, ->)
    * exact <[], cone_nil.symm>
    * exact memB_embBit false hX
    * exact memB_embBit true hY
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | <X, hX, rfl, hsX> | <Y, hY, rfl, hsY>)
      (rfl | <X', hX', rfl, hsX'> | <Y', hY', rfl, hsY'>)
    * exact Or.inl (by rw [Set.inter_self])
    * exact Or.inr (Or.inl <X', hX', by rw [Set.univ_inter], hsX'>)
    * exact Or.inr (Or.inr <Y', hY', by rw [Set.univ_inter], hsY'>)
    * exact Or.inr (Or.inl <X, hX, by rw [Set.inter_univ], hsX>)
    * refine Or.inr (Or.inl <X  I  X', ?_, by rw [embBit_inter], ?_>)
      * have hs := s.inter_mem hsX hsX'; rw [inj_0_inter] at hs
        exact sum_mem_inj_0_inv (s.sub hs)
      * have hs := s.inter_mem hsX hsX'; rwa [inj_0_inter] at hs
    * exfalso
      have hs := s.inter_mem hsX hsY'; rw [inj_0_inter_inj_1] at hs
      obtain <t, ht> := sum_mem_nonempty (s.sub hs); exact Set.notMem_empty t ht
    * exact Or.inr (Or.inr <Y, hY, by rw [Set.inter_univ], hsY>)
    * exfalso
      have hs := s.inter_mem hsY hsX'; rw [Set.inter_comm, inj_0_inter_inj_1] at hs
      obtain <t, ht> := sum_mem_nonempty (s.sub hs); exact Set.notMem_empty t ht
    * refine Or.inr (Or.inr <Y  I  Y', ?_, by rw [embBit_inter], ?_>)
      * have hs := s.inter_mem hsY hsY'; rw [inj_1_inter] at hs
        exact sum_mem_inj_1_inv (s.sub hs)
      * have hs := s.inter_mem hsY hsY'; rwa [inj_1_inter] at hs
  up_mem := by
    rintro W W' (rfl | <X, hX, rfl, hsX> | <Y, hY, rfl, hsY>) hW' hsub
    * exact Or.inl (Set.univ_subset_iff.mp hsub)
    * rcases memB_cases hW' with rfl | <X', hX', rfl> | <Y', hY', rfl>
      * exact Or.inl rfl
      * refine Or.inr (Or.inl <X', hX', rfl, ?_>)
        exact s.up_mem hsX (Or.inr (Or.inl <X', hX', rfl>))
          (inj_0_subset_inj_0.mpr (embBit_subset.mp hsub))
      * exfalso
        obtain <a, ha> := B_nonempty X hX
        obtain <w', he, -> := hsub (<a, rfl, ha> : (false :: a)  in  embBit false X)
        rw [List.cons.injEq] at he; exact absurd he.1 (by decide)
    * rcases memB_cases hW' with rfl | <X', hX', rfl> | <Y', hY', rfl>
      * exact Or.inl rfl
      * exfalso
        obtain <b, hb> := B_nonempty Y hY
        obtain <w', he, -> := hsub (<b, rfl, hb> : (true :: b)  in  embBit true Y)
        rw [List.cons.injEq] at he; exact absurd he.1 (by decide)
      * refine Or.inr (Or.inr <Y', hY', rfl, ?_>)
        exact s.up_mem hsY (Or.inr (Or.inr <Y', hY', rfl>))
          (inj_1_subset_inj_1.mpr (embBit_subset.mp hsub))
```

```
@[simp] theorem fromBB_mem_embF {s : BB.Element} {X : Set Str} (hX : B.mem X) :
    (fromBB s).mem (embBit false X)  <->  s.mem (inj_0 X) := by
  constructor
  * rintro (h0 | <X', hX', heq, hs> | <Y', hY', heq, hs>)
    * exact absurd h0 (embBit_ne_univ false X)
    * rwa [embBit_injective heq]
    * exact absurd heq (embBit_ne (show (false : Bool)  !=  true by decide) (
    B_nonempty X hX))
  * intro hs; exact Or.inr (Or.inl <X, hX, rfl, hs>)

@[simp] theorem fromBB_mem_embT {s : BB.Element} {Y : Set Str} (hY : B.mem Y) :
    (fromBB s).mem (embBit true Y)  <->  s.mem (inj_1 Y) := by
  constructor
  * rintro (h0 | <X', hX', heq, hs> | <Y', hY', heq, hs>)
    * exact absurd h0 (embBit_ne_univ true Y)
    * exact absurd heq.symm (embBit_ne (show (false : Bool)  !=  true by decide) (
    B_nonempty X' hX'))
    * rwa [embBit_injective heq]
  * intro hs; exact Or.inr (Or.inr <Y, hY, rfl, hs>)

theorem fromBB_toBB (x : B.Element) : fromBB (toBB x) = x := by
  apply NeighborhoodSystem.Element.ext
  intro W
  constructor
  * rintro (rfl | <X, hX, rfl, hs> | <Y, hY, rfl, hs>)
    * exact x.master_mem
    * exact (toBB_mem_inj_0 hX).mp hs
    * exact (toBB_mem_inj_1 hY).mp hs
  * intro hW
    rcases memB_cases (x.sub hW) with rfl | <X, hX, rfl> | <Y, hY, rfl>
    * exact Or.inl rfl
    * exact Or.inr (Or.inl <X, hX, rfl, (toBB_mem_inj_0 hX).mpr hW>)
    * exact Or.inr (Or.inr <Y, hY, rfl, (toBB_mem_inj_1 hY).mpr hW>)

theorem toBB_fromBB (s : BB.Element) : toBB (fromBB s) = s := by
  apply NeighborhoodSystem.Element.ext
  intro W
  constructor
  * rintro (rfl | <X, hX, rfl, hs> | <Y, hY, rfl, hs>)
    * exact s.master_mem
    * exact (fromBB_mem_embF hX).mp hs
    * exact (fromBB_mem_embT hY).mp hs
  * intro hW
    rcases s.sub hW with rfl | <X, hX, rfl> | <Y, hY, rfl>
    * exact Or.inl rfl
    * exact Or.inr (Or.inl <X, hX, rfl, (fromBB_mem_embF hX).mpr hW>)
    * exact Or.inr (Or.inr <Y, hY, rfl, (fromBB_mem_embT hY).mpr hW>)

/-- **Example 6.2 (Scott 1981, PRG-19) -- the isomorphism `|B| ?o |B + B|`.** -/
def bbEquiv : B.Element ?o BB.Element where
  toFun := toBB
  invFun := fromBB
  left_inv := fromBB_toBB
  right_inv := toBB_fromBB
  map_rel_iff' := by
    intro x x'
```

```
    constructor
    * intro h X hX
      rcases memB_cases (x.sub hX) with rfl | <A, hA, rfl> | <A, hA, rfl>
      * exact x'.master_mem
      * exact (toBB_mem_inj_0 hA).mp (h _ (Or.inr (Or.inl <A, hA, rfl, hX>)))
      * exact (toBB_mem_inj_1 hA).mp (h _ (Or.inr (Or.inr <A, hA, rfl, hX>)))
    * intro h W hW
      rcases hW with rfl | <X, hX, rfl, hzX> | <Y, hY, rfl, hzY>
      * exact Or.inl rfl
      * exact Or.inr (Or.inl <X, hX, rfl, h _ hzX>)
      * exact Or.inr (Or.inr <Y, hY, rfl, h _ hzY>)

/-- **Example 6.2 (Scott 1981, PRG-19) -- the domain equation `B  ~=  B + B`.** Scott's
    binary-sequence
domain `B` is, as a domain, isomorphic to `B + B`: a sequence is bottom, or begins with
    `0`, or begins
with `1`, the two non-bottom cases giving the two summands. -/
theorem B_domain_equation : B  ~= ? sum B B B_nonempty B_nonempty :=
  <bbEquiv>

theorem embBit_nonempty {b : Bool} {X : Set Str} (hX : X.Nonempty) : (embBit b X).
    Nonempty := by
  obtain <w', hw'> := hX; exact <b :: w', w', rfl, hw'>

end Example62
end

namespace Example62A
open NeighborhoodSystem ApproximableMap ExampleB

section
variable {b : Type*} {n : Nat}
variable {n : Nat}

/-- The slot prefix `i 1? 0`: the leading tag bit `i` (Scott's `+/?`), then `j` ones,
    then a zero.
Reaching component `j` of the tag-`i` tuple means matching this prefix. -/
def slotPre (i : Bool) (j : Nat) : Str := i :: (List.replicate j true ++ [false])

/-- **Scott's tag-`i` tuple neighbourhood** `i ?_{j<n} 1?0 X_j`: the union of the `n`
    slots, slot `j`
holding `X j` behind the prefix `i 1? 0`. (For `i = false`/`true` these are Scott's
    left/right
copies, the `?`/`+` summands.) -/
def embTuple (i : Bool) (X : Fin n  ->  Set Str) : Set Str :=
  {w | exists  j : Fin n, exists  w', w = slotPre i (j : Nat) ++ w'  /\  w'  in  X j}

/-- **Scott's domain `A`** as the *least* family of subsets of `Sigma* = {0,1}*`
    containing (i) the master
`Sigma*` and (ii) every tag-`i` tuple `i ?_{j<n} 1?0 X_j` built from components `X j`
    already in `A`:

`A = {Sigma*}  U  ?_{i in {0,1}} { i ?_{j<n} 1?0 X_j ? X_j  in  A }`. -/
inductive MemA (n : Nat) : Set Str  ->  Prop
  | univ : MemA n Set.univ
  | tuple (i : Bool) {X : Fin n  ->  Set Str} : (forall  j, MemA n (X j))  ->  MemA n
```

```
    (embTuple i X)

end

section
variable {b : Type*} {n : Nat}

/-- The *proper product* neighbourhood `?_{j<n} {j}  x  X_j`: token `(j, b)` lies in it
     iff `b  in  X_j`.
This is the `n`-ary analogue of `prodNbhd` (Definition 3.1). -/
def prodN (X : Fin n  ->  Set b) : Set (Fin n  x  b) := {p | p.2  in  X p.1}

@[simp] theorem mem_prodN {X : Fin n  ->  Set b} {p : Fin n  x  b} : p  in  prodN X
    <->  p.2  in  X p.1 := Iff.rfl

theorem prodN_injective {X X' : Fin n  ->  Set b} (h : prodN X = prodN X') : forall  j
    , X j = X' j := by
  intro j; ext b
  have := Set.ext_iff.mp h (j, b)
  simpa only [mem_prodN] using this

theorem prodN_inter {X X' : Fin n  ->  Set b} :
    prodN X  I  prodN X' = prodN (fun j => X j  I  X' j) := by
  ext p; simp only [Set.mem_inter_iff, mem_prodN]

theorem prodN_subset {X X' : Fin n  ->  Set b} : prodN X  subseteq  prodN X'  <->
    forall  j, X j  subseteq  X' j := by
  constructor
  * intro h j b hb; exact h (show (j, b)  in  prodN X from hb)
  * intro h p hp; exact h p.1 hp

/-- **The positive `n`-fold product `V?`** of a neighbourhood system `V`, over `Fin n
    x  b`.
The assumption `0 < n` is necessary because Scott's token set `Fin n  x  Delta` must be
     non-empty.
Its neighbourhoods are exactly the proper products `prodN X` with each component `X j
    in  V`;
closure under intersection is componentwise. -/
def npow (V : NeighborhoodSystem b) (n : Nat) (hn : 0 < n) : NeighborhoodSystem (Fin n
      x  b) where
  mem W := exists  X : Fin n  ->  Set b, (forall  j, V.mem (X j))  /\  W = prodN X
  master := prodN (fun _ => V.master)
  master_nonempty := by
    obtain <b, hb> := V.master_nonempty
    exact <(<0, hn>, b), hb>
  master_mem := <_, fun _ => V.master_mem, rfl>
  inter_mem := by
    rintro W W' Z <X, hX, rfl> <X', hX', rfl> <Zw, hZw, rfl> hsub
    refine <fun j => X j  I  X' j, ?_, prodN_inter>
    intro j
    rw [prodN_inter] at hsub
    exact V.inter_mem (hX j) (hX' j) (hZw j) (prodN_subset.mp hsub j)
  sub_master := by
    rintro W <X, hX, rfl>
    exact prodN_subset.mpr (fun j => V.sub_master (hX j))
```

```
@[simp] theorem npow_mem {V : NeighborhoodSystem b} {W : Set (Fin n  x  b)} {hn : 0 <
    n} :
     (npow V n hn).mem W  <->  exists  X : Fin n  ->  Set b, (forall  j, V.mem (X j))
    /\  W = prodN X := Iff.rfl

end

section
variable {b : Type*} {n : Nat}
variable {n : Nat}

@[simp] theorem mem_embTuple {i : Bool} {X : Fin n  ->  Set Str} {w : Str} :
    w  in  embTuple i X  <->  exists  j : Fin n, exists  w', w = slotPre i (j : Nat)
    ++ w'  /\  w'  in  X j := Iff.rfl

theorem embTuple_nonempty {i : Bool} {X : Fin n  ->  Set Str} (j : Fin n) (h : (X j).
    Nonempty) :
     (embTuple i X).Nonempty := by
  obtain <a, ha> := h
  exact <slotPre i (j : Nat) ++ a, j, a, rfl, ha>

/-- Under `0 < n` every neighbourhood of `A` is non-empty (Scott's `{}  notin  A`). -/
theorem memA_nonempty (hn : 0 < n) : forall  {W}, MemA n W  ->  W.Nonempty := by
  intro W hW
  induction hW with
  | univ => exact <[], Set.mem_univ []>
  | tuple i hX ih => exact embTuple_nonempty <0, hn> (ih <0, hn>)

theorem nil_not_mem_embTuple {i : Bool} {X : Fin n  ->  Set Str} : ([] : Str)  notin
    embTuple i X := by
  rintro <j, a, heq, ->
  rw [slotPre, List.cons_append] at heq
  exact List.cons_ne_nil _ _ heq.symm

theorem embTuple_ne_univ {i : Bool} (X : Fin n  ->  Set Str) : embTuple i X  !=  Set.
    univ := by
  intro h
  have : ([] : Str)  in  embTuple i X := by rw [h]; exact Set.mem_univ []
  exact nil_not_mem_embTuple this

end

section
variable {b : Type*} {n : Nat}

/-- Under Scott's standing assumption `{}  notin  V`, no neighbourhood of `V?` is empty
     (provided `0 < n`,
so there is a coordinate to witness). -/
theorem npow_nonempty {V : NeighborhoodSystem b} (hn : 0 < n)
    (hV : forall  X, V.mem X  ->  X.Nonempty) : forall  W, (npow V n hn).mem W  ->  W.
    Nonempty := by
  rintro W <X, hX, rfl>
  obtain <b, hb> := hV (X <0, hn>) (hX <0, hn>)
  exact <(<0, hn>, b), hb>

/-- Parsing the `1?0` body: the position of the first `false` pins down `j` and the
```

```
    remainder. -/
theorem slot_list_inj : forall  (j j' : Nat) (u v : Str),
    List.replicate j true ++ false :: u = List.replicate j' true ++ false :: v  ->  j
    = j'  /\  u = v
  | 0, 0, u, v, h => by simpa using h
  | 0, j' + 1, u, v, h => by simp [List.replicate_succ] at h
  | j + 1, 0, u, v, h => by simp [List.replicate_succ] at h
  | j + 1, j' + 1, u, v, h => by
      rw [List.replicate_succ, List.cons_append, List.replicate_succ, List.cons_append
    ,


        List.cons.injEq] at h
      obtain <hj, hu> := slot_list_inj j j' u v h.2
      exact <by rw [hj], hu>

/-- **Uniqueness of slot decomposition.** A token `i 1? 0 u` determines the tag `i`,
    the index `j`,
and the remainder `u`. This is what makes the slots pairwise disjoint. -/
theorem slotPre_inj {i i' : Bool} {j j' : Nat} {u v : Str}
    (h : slotPre i j ++ u = slotPre i' j' ++ v) : i = i'  /\  j = j'  /\  u = v := by
  rw [slotPre, slotPre, List.cons_append, List.cons_append, List.cons.injEq] at h
  obtain <hi, hrest> := h
  rw [List.append_assoc, List.append_assoc, List.singleton_append, List.
    singleton_append] at hrest
  obtain <hj, huv> := slot_list_inj j j' u v hrest
  exact <hi, hj, huv>

end

section
variable {b : Type*} {n : Nat}
variable {n : Nat}

theorem embTuple_inter {i : Bool} (X X' : Fin n  ->  Set Str) :
    embTuple i X  I  embTuple i X' = embTuple i (fun j => X j  I  X' j) := by
  ext w
  simp only [Set.mem_inter_iff, mem_embTuple]
  constructor
  * rintro <<j, a, rfl, ha>, <j', a', heq, ha'>>
    obtain <-, hjj, rfl> := slotPre_inj heq
    obtain rfl : j = j' := Fin.ext hjj
    exact <j, a, rfl, ha, ha'>
  * rintro <j, a, rfl, ha, ha'>
    exact <<j, a, rfl, ha>, <j, a, rfl, ha'>>

theorem embTuple_inter_ne {i i' : Bool} (h : i  !=  i') (X X' : Fin n  ->  Set Str) :
    embTuple i X  I  embTuple i' X' = {} := by
  ext w
  simp only [Set.mem_inter_iff, mem_embTuple, Set.mem_empty_iff_false, iff_false,
    not_and]
  rintro <j, a, rfl, ha> <j', a', heq, ha'>
  exact h (slotPre_inj heq).1

theorem embTuple_ne {i i' : Bool} (h : i  !=  i') {X Y : Fin n  ->  Set Str} (j : Fin
    n)
    (hX : (X j).Nonempty) : embTuple i X  !=  embTuple i' Y := by
```

```
  intro heq
  obtain <a, ha> := hX
  have hmem : slotPre i (j : Nat) ++ a  in  embTuple i' Y := heq  |>  (<j, a, rfl, ha>
     : _  in  embTuple i X)
  obtain <j', a', he, -> := hmem
  exact h (slotPre_inj he).1

theorem embTuple_subset {i : Bool} {X X' : Fin n  ->  Set Str} :
    embTuple i X  subseteq  embTuple i X'  <->  forall  j, X j  subseteq  X' j := by
  constructor
  * intro h j a ha
    obtain <j', a', heq, ha'> := h <j, a, rfl, ha>
    obtain <-, hjj, rfl> := slotPre_inj heq
    obtain rfl : j = j' := Fin.ext hjj
    exact ha'
  * rintro h w <j, a, rfl, ha>
    exact <j, a, rfl, h j ha>

theorem embTuple_injective {i : Bool} {X X' : Fin n  ->  Set Str}
    (h : embTuple i X = embTuple i X') : forall  j, X j = X' j := by
  intro j
  exact Set.Subset.antisymm (embTuple_subset.mp h.subset j) (embTuple_subset.mp h.symm
    .subset j)

/-- Inversion: the components of a tuple-shaped neighbourhood of `A` are themselves in
    `A`. -/
theorem memA_tuple_inv (hn : 0 < n) {i : Bool} {X : Fin n  ->  Set Str}
    (h : MemA n (embTuple i X)) : forall  j, MemA n (X j) := by
  generalize hW : embTuple i X = W at h
  cases h with
  | univ =>
    exfalso
    have : ([] : Str)  in  embTuple i X := by rw [hW]; exact Set.mem_univ []
    exact nil_not_mem_embTuple this
  | @tuple i' X' hX' =>
    have hne : (embTuple i' X').Nonempty :=
      embTuple_nonempty <0, hn> (memA_nonempty hn (hX' <0, hn>))
    rw [ <-  hW] at hne
    obtain <w, j, a, rfl, ha> := hne
    have hmem : slotPre i (j : Nat) ++ a  in  embTuple i' X' := hW  |>  (<j, a, rfl,
    ha> : _  in  embTuple i X)
    obtain <j', a', he, -> := hmem
    have hii : i = i' := (slotPre_inj he).1
    subst hii
    intro k; rw [embTuple_injective hW k]; exact hX' k

/-- A non-empty witness contained in a tag-`i'` tuple must carry the tag `i'`. -/
theorem tag_eq_of_subset (hn : 0 < n) {i i' : Bool} {Z V : Fin n  ->  Set Str}
    (hZ : forall  j, MemA n (Z j)) (hsub : embTuple i Z  subseteq  embTuple i' V) : i
    = i' := by
  obtain <a, ha> := memA_nonempty hn (hZ <0, hn>)
  obtain <j', v, he, -> := hsub <<0, hn>, a, rfl, ha>
  exact (slotPre_inj he).1

/-- **`A` is closed under consistent intersection.** Scott's verification, by induction
     on the way `X`
```

```
entered `A`: the cross case `embTuple false *  I  embTuple true *` collapses to `{}` (
    killed by the
non-empty witness), the same-tag case combines componentwise (recursing on the
    consistency witness's
slots). -/
theorem memA_inter (hn : 0 < n) :
    forall  {X}, MemA n X  ->  forall  {Y}, MemA n Y  ->  forall  {Z}, MemA n Z  ->  Z
      subseteq  X  I  Y  ->  MemA n (X  I  Y) := by
  intro X hX
  induction hX with
  | univ =>
    intro Y hY Z _ _
    rw [Set.univ_inter]; exact hY
  | @tuple i X hX ih =>
    intro Y hY Z hZ hsub
    cases hY with
    | univ =>
      rw [Set.inter_univ]; exact MemA.tuple i hX
    | @tuple i' X' hX' =>
      by_cases hii : i = i'
      * rw [hii, embTuple_inter] at hsub  |-
        cases hZ with
        | univ => exact absurd (hsub (Set.mem_univ [])) nil_not_mem_embTuple
        | @tuple i'' ZW hZW =>
          have htag : i'' = i' := tag_eq_of_subset hn hZW hsub
          rw [htag] at hsub
          have hZsub := embTuple_subset.mp hsub
          exact MemA.tuple i' (fun j => ih j (hX' j) (hZW j) (hZsub j))
      * rw [embTuple_inter_ne hii] at hsub  |-
        exfalso
        obtain <z, hz> := memA_nonempty hn hZ
        exact Set.notMem_empty z (hsub hz)

/-- **Scott's domain `A`** packaged as a neighbourhood system over `{0,1}*` (needs `0 <
    n`). -/
def Asys (n : Nat) (hn : 0 < n) : NeighborhoodSystem Str where
  mem := MemA n
  master := Set.univ
  master_nonempty := <[], Set.mem_univ _>
  master_mem := MemA.univ
  inter_mem := fun hX hY hZ hsub => memA_inter hn hX hY hZ hsub
  sub_master := fun _ => Set.subset_univ _

@[simp] theorem Asys_mem {hn : 0 < n} {W : Set Str} : (Asys n hn).mem W  <->  MemA n W
    := Iff.rfl

@[simp] theorem Asys_master {hn : 0 < n} : (Asys n hn).master = Set.univ := rfl

/-- The `n`-fold product `A? = npow A n`. -/
abbrev Apow (hn : 0 < n) : NeighborhoodSystem (Fin n  x  Str) := npow (Asys n hn) n hn

theorem apowNe (hn : 0 < n) : forall  W, (Apow hn).mem W  ->  W.Nonempty :=
  npow_nonempty hn (fun _ h => memA_nonempty hn h)

/-- The right-hand side of Scott's equation: the separated sum `A? + A?`. -/
abbrev AAsys (hn : 0 < n) : NeighborhoodSystem (Option ((Fin n  x  Str) ? (Fin n  x
```

```
    Str))) :=
  sum (Apow hn) (Apow hn) (apowNe hn) (apowNe hn)

theorem aa_mem_inj_0_inv (hn : 0 < n) {V : Set (Fin n  x  Str)}
    (h : (AAsys hn).mem (inj_0 V)) : (Apow hn).mem V := by
  rcases h with h0 | <X', hX', heq> | <Y', hY', heq>
  * exact absurd (h0  |>  none_mem_sumMaster) none_mem_inj_0
  * rw [inj_0_injective heq]; exact hX'
  * obtain <b, hb> := apowNe hn _ hY'
    exact absurd (heq  |>  ir_mem_inj_1.mpr hb) ir_mem_inj_0

theorem aa_mem_inj_1_inv (hn : 0 < n) {V : Set (Fin n  x  Str)}
    (h : (AAsys hn).mem (inj_1 V)) : (Apow hn).mem V := by
  rcases h with h0 | <X', hX', heq> | <Y', hY', heq>
  * exact absurd (h0  |>  none_mem_sumMaster) none_mem_inj_1
  * obtain <a, ha> := apowNe hn _ hX'
    exact absurd (heq  |>  il_mem_inj_0.mpr ha) il_mem_inj_1
  * rw [inj_1_injective heq]; exact hY'

theorem aa_mem_nonempty (hn : 0 < n) {W : Set (Option ((Fin n  x  Str) ? (Fin n  x
    Str)))}
    (h : (AAsys hn).mem W) : W.Nonempty := by
  rcases h with rfl | <X, hX, rfl> | <Y, hY, rfl>
  * exact <none, none_mem_sumMaster>
  * exact inj_0_nonempty (apowNe hn _ hX)
  * exact inj_1_nonempty (apowNe hn _ hY)

theorem apow_mem_prodN (hn : 0 < n) {X : Fin n  ->  Set Str} (hX : forall  j, MemA n (
    X j)) :
    (Apow hn).mem (prodN X) := <X, hX, rfl>

theorem aa_mem_inj_0 (hn : 0 < n) {X : Fin n  ->  Set Str} (hX : forall  j, MemA n (X
    j)) :
    (AAsys hn).mem (inj_0 (prodN X)) := Or.inr (Or.inl <prodN X, apow_mem_prodN hn hX,
     rfl>)

theorem aa_mem_inj_1 (hn : 0 < n) {Y : Fin n  ->  Set Str} (hY : forall  j, MemA n (Y
    j)) :
    (AAsys hn).mem (inj_1 (prodN Y)) := Or.inr (Or.inr <prodN Y, apow_mem_prodN hn hY,
     rfl>)

theorem apow_mem_prodN_inv (hn : 0 < n) {X : Fin n  ->  Set Str} (h : (Apow hn).mem (
    prodN X)) :
    forall  j, MemA n (X j) := by
  obtain <X', hX', heq> := h
  intro j; rw [prodN_injective heq j]; exact hX' j

/-- **Forward half of `A  ~=  A? + A?`.** An element `z` of `A` is sent to the sum
    element recording, for
each branch, whether `z` reaches the left copy `0X` (left summand `A?`) or the right
    copy `1Y` (right
summand `A?`). -/
def toAA (hn : 0 < n) (z : (Asys n hn).Element) : (AAsys hn).Element where
  mem W := W = sumMaster (Apow hn) (Apow hn)
     \/  (exists  X, (forall  j, MemA n (X j))  /\  W = inj_0 (prodN X)  /\  z.mem (
    embTuple false X))
```

```
   \/  (exists  Y, (forall  j, MemA n (Y j))  /\  W = inj_1 (prodN Y)  /\  z.mem (
  embTuple true Y))
sub := by
  rintro W (rfl | <X, hX, rfl, -> | <Y, hY, rfl, ->)
  * exact (AAsys hn).master_mem
  * exact aa_mem_inj_0 hn hX
  * exact aa_mem_inj_1 hn hY
master_mem := Or.inl rfl
inter_mem := by
  rintro W W' (rfl | <X, hX, rfl, hzX> | <Y, hY, rfl, hzY>)
    (rfl | <X', hX', rfl, hzX'> | <Y', hY', rfl, hzY'>)
  * exact Or.inl (by rw [Set.inter_self])
  * exact Or.inr (Or.inl <X', hX', by rw [sumMaster_inter_inj_0 (apow_mem_prodN hn
  hX')], hzX'>)
  * exact Or.inr (Or.inr <Y', hY', by rw [sumMaster_inter_inj_1 (apow_mem_prodN hn
  hY')], hzY'>)
  * exact Or.inr (Or.inl <X, hX,
      by rw [Set.inter_comm, sumMaster_inter_inj_0 (apow_mem_prodN hn hX)], hzX>)
  * refine Or.inr (Or.inl <fun j => X j  I  X' j, ?_, by rw [inj_0_inter,
  prodN_inter], ?_>)
    * have hz := z.inter_mem hzX hzX'; rw [embTuple_inter] at hz
      exact memA_tuple_inv hn (z.sub hz)
    * have hz := z.inter_mem hzX hzX'; rwa [embTuple_inter] at hz
  * exfalso
    have hz := z.inter_mem hzX hzY'
    rw [embTuple_inter_ne (by decide : (false : Bool)  !=  true)] at hz
    obtain <t, ht> := memA_nonempty hn (z.sub hz); exact Set.notMem_empty t ht
  * exact Or.inr (Or.inr <Y, hY,
      by rw [Set.inter_comm, sumMaster_inter_inj_1 (apow_mem_prodN hn hY)], hzY>)
  * exfalso
    have hz := z.inter_mem hzY hzX'
    rw [embTuple_inter_ne (by decide : (true : Bool)  !=  false)] at hz
    obtain <t, ht> := memA_nonempty hn (z.sub hz); exact Set.notMem_empty t ht
  * refine Or.inr (Or.inr <fun j => Y j  I  Y' j, ?_, by rw [inj_1_inter,
  prodN_inter], ?_>)
    * have hz := z.inter_mem hzY hzY'; rw [embTuple_inter] at hz
      exact memA_tuple_inv hn (z.sub hz)
    * have hz := z.inter_mem hzY hzY'; rwa [embTuple_inter] at hz
up_mem := by
  rintro W W' (rfl | <X, hX, rfl, hzX> | <Y, hY, rfl, hzY>) hW' hsub
  * exact Or.inl (eq_sumMaster_of_subset hW' hsub)
  * rcases hW' with rfl | <V', hV', rfl> | <V', hV', rfl>
    * exact Or.inl rfl
    * obtain <X', hX', rfl> := hV'
      refine Or.inr (Or.inl <X', hX', rfl, ?_>)
      exact z.up_mem hzX (MemA.tuple false hX')
        (embTuple_subset.mpr (prodN_subset.mp (inj_0_subset_inj_0.mp hsub)))
    * obtain <b, hb> := apowNe hn (prodN X) (apow_mem_prodN hn hX)
      exact absurd (hsub (il_mem_inj_0.mpr hb)) il_mem_inj_1
  * rcases hW' with rfl | <V', hV', rfl> | <V', hV', rfl>
    * exact Or.inl rfl
    * obtain <b, hb> := apowNe hn (prodN Y) (apow_mem_prodN hn hY)
      exact absurd (hsub (ir_mem_inj_1.mpr hb)) ir_mem_inj_0
    * obtain <Y', hY', rfl> := hV'
      refine Or.inr (Or.inr <Y', hY', rfl, ?_>)
      exact z.up_mem hzY (MemA.tuple true hY')
```

```
          (embTuple_subset.mpr (prodN_subset.mp (inj_1_subset_inj_1.mp hsub)))

@[simp] theorem toAA_mem_inj_0 (hn : 0 < n) {z : (Asys n hn).Element} {X : Fin n  ->
    Set Str}
    (hX : forall  j, MemA n (X j)) :
    (toAA hn z).mem (inj_0 (prodN X))  <->  z.mem (embTuple false X) := by
  constructor
  * rintro (h0 | <X', hX', heq, hz> | <Y', hY', heq, hz>)
    * exact absurd (h0  |>  none_mem_sumMaster) none_mem_inj_0
    * rw [funext (prodN_injective (inj_0_injective heq))]; exact hz
    * obtain <b, hb> := apowNe hn (prodN X) (apow_mem_prodN hn hX)
      exact absurd (heq  |>  il_mem_inj_0.mpr hb) il_mem_inj_1
  * intro hz; exact Or.inr (Or.inl <X, hX, rfl, hz>)

@[simp] theorem toAA_mem_inj_1 (hn : 0 < n) {z : (Asys n hn).Element} {Y : Fin n  ->
    Set Str}
    (hY : forall  j, MemA n (Y j)) :
    (toAA hn z).mem (inj_1 (prodN Y))  <->  z.mem (embTuple true Y) := by
  constructor
  * rintro (h0 | <X', hX', heq, hz> | <Y', hY', heq, hz>)
    * exact absurd (h0  |>  none_mem_sumMaster) none_mem_inj_1
    * obtain <a, ha> := apowNe hn (prodN X') (apow_mem_prodN hn hX')
      exact absurd (heq  |>  il_mem_inj_0.mpr ha) il_mem_inj_1
    * rw [funext (prodN_injective (inj_1_injective heq))]; exact hz
  * intro hz; exact Or.inr (Or.inr <Y, hY, rfl, hz>)

/-- **Inverse half of `A  ~=  A? + A?`.** -/
def fromAA (hn : 0 < n) (s : (AAsys hn).Element) : (Asys n hn).Element where
  mem W := W = Set.univ
     \/  (exists  X, (forall  j, MemA n (X j))  /\  W = embTuple false X  /\  s.mem (
    inj_0 (prodN X)))
     \/  (exists  Y, (forall  j, MemA n (Y j))  /\  W = embTuple true Y  /\  s.mem (
    inj_1 (prodN Y)))
  sub := by
    rintro W (rfl | <X, hX, rfl, -> | <Y, hY, rfl, ->)
    * exact MemA.univ
    * exact MemA.tuple false hX
    * exact MemA.tuple true hY
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | <X, hX, rfl, hsX> | <Y, hY, rfl, hsY>)
      (rfl | <X', hX', rfl, hsX'> | <Y', hY', rfl, hsY'>)
    * exact Or.inl (by rw [Set.inter_self])
    * exact Or.inr (Or.inl <X', hX', by rw [Set.univ_inter], hsX'>)
    * exact Or.inr (Or.inr <Y', hY', by rw [Set.univ_inter], hsY'>)
    * exact Or.inr (Or.inl <X, hX, by rw [Set.inter_univ], hsX>)
    * have hs := s.inter_mem hsX hsX'
      rw [inj_0_inter, prodN_inter] at hs
      exact Or.inr (Or.inl <fun j => X j  I  X' j,
        apow_mem_prodN_inv hn (aa_mem_inj_0_inv hn (s.sub hs)), by rw [embTuple_inter
    ], hs>)
    * exfalso
      have hs := s.inter_mem hsX hsY'; rw [inj_0_inter_inj_1] at hs
      obtain <t, ht> := aa_mem_nonempty hn (s.sub hs); exact Set.notMem_empty t ht
    * exact Or.inr (Or.inr <Y, hY, by rw [Set.inter_univ], hsY>)
    * exfalso
```

```
      have hs := s.inter_mem hsY hsX'; rw [Set.inter_comm, inj_0_inter_inj_1] at hs
      obtain <t, ht> := aa_mem_nonempty hn (s.sub hs); exact Set.notMem_empty t ht
    * have hs := s.inter_mem hsY hsY'
      rw [inj_1_inter, prodN_inter] at hs
      exact Or.inr (Or.inr <fun j => Y j  I  Y' j,
        apow_mem_prodN_inv hn (aa_mem_inj_1_inv hn (s.sub hs)), by rw [embTuple_inter
    ], hs>)
  up_mem := by
    rintro W W' (rfl | <X, hX, rfl, hsX> | <Y, hY, rfl, hsY>) hW' hsub
    * exact Or.inl (Set.univ_subset_iff.mp hsub)
    * cases hW' with
      | univ => exact Or.inl rfl
      | @tuple i'' V hV =>
        have htag : false = i'' := tag_eq_of_subset hn hX hsub
        subst htag
        refine Or.inr (Or.inl <V, hV, rfl, ?_>)
        exact s.up_mem hsX (aa_mem_inj_0 hn hV)
          (inj_0_subset_inj_0.mpr (prodN_subset.mpr (embTuple_subset.mp hsub)))
    * cases hW' with
      | univ => exact Or.inl rfl
      | @tuple i'' V hV =>
        have htag : true = i'' := tag_eq_of_subset hn hY hsub
        subst htag
        refine Or.inr (Or.inr <V, hV, rfl, ?_>)
        exact s.up_mem hsY (aa_mem_inj_1 hn hV)
          (inj_1_subset_inj_1.mpr (prodN_subset.mpr (embTuple_subset.mp hsub)))

@[simp] theorem fromAA_mem_embF (hn : 0 < n) {s : (AAsys hn).Element} {X : Fin n  ->
    Set Str}
    (hX : forall  j, MemA n (X j)) :
    (fromAA hn s).mem (embTuple false X)  <->  s.mem (inj_0 (prodN X)) := by
  constructor
  * rintro (h0 | <X', hX', heq, hs> | <Y', hY', heq, hs>)
    * exact absurd h0 (embTuple_ne_univ X)
    * rw [funext (embTuple_injective heq)]; exact hs
    * exact absurd heq
        (embTuple_ne (by decide : (false : Bool)  !=  true) <0, hn> (memA_nonempty hn
    (hX <0, hn>)))
  * intro hs; exact Or.inr (Or.inl <X, hX, rfl, hs>)

@[simp] theorem fromAA_mem_embT (hn : 0 < n) {s : (AAsys hn).Element} {Y : Fin n  ->
    Set Str}
    (hY : forall  j, MemA n (Y j)) :
    (fromAA hn s).mem (embTuple true Y)  <->  s.mem (inj_1 (prodN Y)) := by
  constructor
  * rintro (h0 | <X', hX', heq, hs> | <Y', hY', heq, hs>)
    * exact absurd h0 (embTuple_ne_univ Y)
    * exact absurd heq.symm
        (embTuple_ne (by decide : (false : Bool)  !=  true) <0, hn> (memA_nonempty hn
    (hX' <0, hn>)))
    * rw [funext (embTuple_injective heq)]; exact hs
  * intro hs; exact Or.inr (Or.inr <Y, hY, rfl, hs>)

theorem fromAA_toAA (hn : 0 < n) (z : (Asys n hn).Element) : fromAA hn (toAA hn z) = z
     := by
  apply NeighborhoodSystem.Element.ext
```

```
  intro W
  constructor
  * rintro (rfl | <X, hX, rfl, hs> | <Y, hY, rfl, hs>)
    * exact z.master_mem
    * exact (toAA_mem_inj_0 hn hX).mp hs
    * exact (toAA_mem_inj_1 hn hY).mp hs
  * intro hW
    cases z.sub hW with
    | univ => exact Or.inl rfl
    | @tuple i X hX =>
      cases i with
      | false => exact Or.inr (Or.inl <X, hX, rfl, (toAA_mem_inj_0 hn hX).mpr hW>)
      | true => exact Or.inr (Or.inr <X, hX, rfl, (toAA_mem_inj_1 hn hX).mpr hW>)

theorem toAA_fromAA (hn : 0 < n) (s : (AAsys hn).Element) : toAA hn (fromAA hn s) = s
    := by
  apply NeighborhoodSystem.Element.ext
  intro W
  constructor
  * rintro (rfl | <X, hX, rfl, hs> | <Y, hY, rfl, hs>)
    * exact s.master_mem
    * exact (fromAA_mem_embF hn hX).mp hs
    * exact (fromAA_mem_embT hn hY).mp hs
  * intro hW
    rcases s.sub hW with rfl | <V, hV, rfl> | <V, hV, rfl>
    * exact Or.inl rfl
    * obtain <X, hX, rfl> := hV
      exact Or.inr (Or.inl <X, hX, rfl, (fromAA_mem_embF hn hX).mpr hW>)
    * obtain <Y, hY, rfl> := hV
      exact Or.inr (Or.inr <Y, hY, rfl, (fromAA_mem_embT hn hY).mpr hW>)

/-- **The isomorphism `|A| ?o |A? + A?|`.** -/
def aaEquiv (hn : 0 < n) : (Asys n hn).Element ?o (AAsys hn).Element where
  toFun := toAA hn
  invFun := fromAA hn
  left_inv := fromAA_toAA hn
  right_inv := toAA_fromAA hn
  map_rel_iff' := by
    intro z z'
    constructor
    * intro h W hW
      cases z.sub hW with
      | univ => exact z'.master_mem
      | @tuple i X hX =>
        cases i with
        | false => exact (toAA_mem_inj_0 hn hX).mp (h _ (Or.inr (Or.inl <X, hX, rfl,
    hW>)))
        | true => exact (toAA_mem_inj_1 hn hX).mp (h _ (Or.inr (Or.inr <X, hX, rfl, hW
    >)))
    * intro h W hW
      rcases hW with rfl | <X, hX, rfl, hzX> | <Y, hY, rfl, hzY>
      * exact Or.inl rfl
      * exact Or.inr (Or.inl <X, hX, rfl, h _ hzX>)
      * exact Or.inr (Or.inr <Y, hY, rfl, h _ hzY>)

/-- **Example 6.2 (Scott 1981, PRG-19) -- the domain equation `A  ~=  A? + A?`.** Scott
```

```
    's "simple, yet
interesting generalization of `B`": the domain `A` over `{0,1}*` (whose non-`False`
    elements are the two
copies -- left `?` and right `+` -- of an `n`-tuple of elements of `A`) is isomorphic
    to the separated
sum `A? + A?` of two copies of its own `n`-fold cartesian power. -/
theorem A_domain_equation (hn : 0 < n) : Asys n hn  ~= ? AAsys hn := <aaEquiv hn>

end

end Example62A


open NeighborhoodSystem ApproximableMap ExampleB Example44 Example62

section
/-- Left tag `0a = some (inl a)`. -/
def t0 {a b ? : Type*} (a : a) : Option (a ? b ? ?) := some (Sum.inl a)
end

section
/-- The tagged left copy `0X`. -/
def j0 {a b ? : Type*} (X : Set a) : Set (Option (a ? b ? ?)) := {w | exists  a, w =
    t0 a  /\  a  in  X}
end

section
variable {a b ? : Type*}

@[simp] theorem t0_mem_j0 {X : Set a} {a : a} : (t0 a : Option (a ? b ? ?))  in  j0 X
     <->  a  in  X := by
  constructor
  * rintro <a', heq, ha'>; simp only [t0, Option.some.injEq, Sum.inl.injEq] at heq;
    exact heq  |>  ha'
  * intro ha; exact <a, rfl, ha>
end

section
/-- Middle tag `1b = some (inr (inl b))`. -/
def t1 {a b ? : Type*} (b : b) : Option (a ? b ? ?) := some (Sum.inr (Sum.inl b))
end

section
/-- The tagged middle copy `1Y`. -/
def j1 {a b ? : Type*} (Y : Set b) : Set (Option (a ? b ? ?)) := {w | exists  b, w =
    t1 b  /\  b  in  Y}
end

section
variable {a b ? : Type*}

@[simp] theorem t1_mem_j1 {Y : Set b} {b : b} : (t1 b : Option (a ? b ? ?))  in  j1 Y
     <->  b  in  Y := by
  constructor
  * rintro <b', heq, hb'>
    simp only [t1, Option.some.injEq, Sum.inr.injEq, Sum.inl.injEq] at heq; exact heq
     |>  hb'
```

```
  * intro hb; exact <b, rfl, hb>
end

section
/-- Right tag `2c = some (inr (inr c))`. -/
def t2 {a b ? : Type*} (c : ?) : Option (a ? b ? ?) := some (Sum.inr (Sum.inr c))
end

section
/-- The tagged right copy `2Z`. -/
def j2 {a b ? : Type*} (Z : Set ?) : Set (Option (a ? b ? ?)) := {w | exists  c, w =
    t2 c  /\  c  in  Z}
end

section
variable {a b ? : Type*}

@[simp] theorem t2_mem_j2 {Z : Set ?} {c : ?} : (t2 c : Option (a ? b ? ?))  in  j2 Z
     <->  c  in  Z := by
  constructor
  * rintro <c', heq, hc'>
    simp only [t2, Option.some.injEq, Sum.inr.injEq] at heq; exact heq  |>  hc'
  * intro hc; exact <c, rfl, hc>

@[simp] theorem none_not_mem_j0 {X : Set a} : (none : Option (a ? b ? ?))  notin  j0 X
     := by
  rintro <a, heq, ->; exact absurd heq (by simp [t0])

@[simp] theorem none_not_mem_j1 {Y : Set b} : (none : Option (a ? b ? ?))  notin  j1 Y
     := by
  rintro <b, heq, ->; exact absurd heq (by simp [t1])

@[simp] theorem none_not_mem_j2 {Z : Set ?} : (none : Option (a ? b ? ?))  notin  j2 Z
     := by
  rintro <c, heq, ->; exact absurd heq (by simp [t2])

@[simp] theorem t1_not_mem_j0 {X : Set a} {b : b} : (t1 b : Option (a ? b ? ?))  notin
      j0 X := by
  rintro <a, heq, ->; exact absurd heq (by simp [t0, t1])

@[simp] theorem t2_not_mem_j0 {X : Set a} {c : ?} : (t2 c : Option (a ? b ? ?))  notin
      j0 X := by
  rintro <a, heq, ->; exact absurd heq (by simp [t0, t2])

@[simp] theorem t0_not_mem_j1 {Y : Set b} {a : a} : (t0 a : Option (a ? b ? ?))  notin
      j1 Y := by
  rintro <b, heq, ->; exact absurd heq (by simp [t0, t1])

@[simp] theorem t2_not_mem_j1 {Y : Set b} {c : ?} : (t2 c : Option (a ? b ? ?))  notin
      j1 Y := by
  rintro <b, heq, ->; exact absurd heq (by simp [t1, t2])

@[simp] theorem t0_not_mem_j2 {Z : Set ?} {a : a} : (t0 a : Option (a ? b ? ?))  notin
      j2 Z := by
  rintro <c, heq, ->; exact absurd heq (by simp [t0, t2])
```

```
@[simp] theorem t1_not_mem_j2 {Z : Set ?} {b : b} : (t1 b : Option (a ? b ? ?))  notin
      j2 Z := by
  rintro <c, heq, ->; exact absurd heq (by simp [t1, t2])

theorem j0_inter_j0 (X X' : Set a) :
    (j0 X  I  j0 X' : Set (Option (a ? b ? ?))) = j0 (X  I  X') := by
  ext w; rcases w with _ | (a | b | c) <;>
    simp [j0, t0, Set.mem_inter_iff]

theorem j0_inter_j1 (X : Set a) (Y : Set b) :
    (j0 X  I  j1 Y : Set (Option (a ? b ? ?))) = {} := by
  ext w; rcases w with _ | (a | b | c) <;>
    simp [j0, j1, t0, t1, Set.mem_inter_iff]

theorem j0_inter_j2 (X : Set a) (Z : Set ?) :
    (j0 X  I  j2 Z : Set (Option (a ? b ? ?))) = {} := by
  ext w; rcases w with _ | (a | b | c) <;>
    simp [j0, j2, t0, t2, Set.mem_inter_iff]

theorem j0_nonempty {X : Set a} (hX : X.Nonempty) : (j0 X : Set (Option (a ? b ? ?))).
    Nonempty := by
  obtain <a, ha> := hX; exact <t0 a, a, rfl, ha>

theorem j0_subset_j0 {X X' : Set a} :
    (j0 X : Set (Option (a ? b ? ?)))  subseteq  j0 X'  <->  X  subseteq  X' := by
  constructor
  * intro h a ha; exact t0_mem_j0.mp (h (t0_mem_j0.mpr ha))
  * rintro h w <a, rfl, ha>; exact t0_mem_j0.mpr (h ha)

theorem j1_inter_j1 (Y Y' : Set b) :
    (j1 Y  I  j1 Y' : Set (Option (a ? b ? ?))) = j1 (Y  I  Y') := by
  ext w; rcases w with _ | (a | b | c) <;>
    simp [j1, t1, Set.mem_inter_iff]

theorem j1_inter_j2 (Y : Set b) (Z : Set ?) :
    (j1 Y  I  j2 Z : Set (Option (a ? b ? ?))) = {} := by
  ext w; rcases w with _ | (a | b | c) <;>
    simp [j1, j2, t1, t2, Set.mem_inter_iff]

theorem j1_nonempty {Y : Set b} (hY : Y.Nonempty) : (j1 Y : Set (Option (a ? b ? ?))).
    Nonempty := by
  obtain <b, hb> := hY; exact <t1 b, b, rfl, hb>

theorem j1_subset_j1 {Y Y' : Set b} :
    (j1 Y : Set (Option (a ? b ? ?)))  subseteq  j1 Y'  <->  Y  subseteq  Y' := by
  constructor
  * intro h b hb; exact t1_mem_j1.mp (h (t1_mem_j1.mpr hb))
  * rintro h w <b, rfl, hb>; exact t1_mem_j1.mpr (h hb)

theorem j1_injective {Y Y' : Set b}
    (h : (j1 Y : Set (Option (a ? b ? ?))) = j1 Y') : Y = Y' :=
  Set.Subset.antisymm (j1_subset_j1.mp h.subset) (j1_subset_j1.mp h.symm.subset)

theorem j2_inter_j2 (Z Z' : Set ?) :
    (j2 Z  I  j2 Z' : Set (Option (a ? b ? ?))) = j2 (Z  I  Z') := by
  ext w; rcases w with _ | (a | b | c) <;>
```

```
    simp [j2, t2, Set.mem_inter_iff]

theorem j2_nonempty {Z : Set ?} (hZ : Z.Nonempty) : (j2 Z : Set (Option (a ? b ? ?))).
    Nonempty := by
  obtain <c, hc> := hZ; exact <t2 c, c, rfl, hc>

theorem j2_subset_j2 {Z Z' : Set ?} :
    (j2 Z : Set (Option (a ? b ? ?)))  subseteq  j2 Z'  <->  Z  subseteq  Z' := by
  constructor
  * intro h c hc; exact t2_mem_j2.mp (h (t2_mem_j2.mpr hc))
  * rintro h w <c, rfl, hc>; exact t2_mem_j2.mpr (h hc)

theorem j2_injective {Z Z' : Set ?}
    (h : (j2 Z : Set (Option (a ? b ? ?))) = j2 Z') : Z = Z' :=
  Set.Subset.antisymm (j2_subset_j2.mp h.subset) (j2_subset_j2.mp h.symm.subset)

end

section
variable {a b ? : Type*}
variable (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) (V_2 :
    NeighborhoodSystem ?)

/-- The master neighbourhood of the three-way sum: `{?}  U  0Delta_0  U  1Delta_1  U  2
    Delta_2`. -/
def master3 : Set (Option (a ? b ? ?)) :=
  insert none (j0 V_0.master  U  j1 V_1.master  U  j2 V_2.master)
end

section
variable {a b ? : Type*}
variable (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) (V_2 :
    NeighborhoodSystem ?)
variable {V_0 V_1 V_2}

@[simp] theorem none_mem_master3 : (none : Option (a ? b ? ?))  in  master3 V_0 V_1
    V_2 :=
  Set.mem_insert _ _

namespace Example62C

/-- Scott's standing assumption `{}  notin  C`: every neighbourhood of `C` is nonempty.
    -/
theorem C_nonempty : forall  X, C.mem X  ->  X.Nonempty := by
  rintro X (<sigma, rfl> | <sigma, rfl>)
  * exact <sigma, List.prefix_rfl>
  * exact <sigma, rfl>

/-- `b{sigma} = {bsigma}`: prepending a bit to a singleton. -/
theorem embBit_singleton (b : Bool) (sigma : Str) : embBit b ({sigma} : Set Str) = {(b
    :: sigma : Str)} := by
  rw [embBit_eq_prepend, Example44.prepend_singleton]; rfl

end Example62C

theorem eq_master3_of_subset {W : Set (Option (a ? b ? ?))}
```

```
    (hsub : master3 V_0 V_1 V_2  subseteq  W) (hsub' : W  subseteq  master3 V_0 V_1
    V_2) : W = master3 V_0 V_1 V_2 :=
  Set.Subset.antisymm hsub' hsub

theorem j0_subset_master3 {X : Set a} (hX : V_0.mem X) :
    (j0 X : Set (Option (a ? b ? ?)))  subseteq  master3 V_0 V_1 V_2 := by
  rintro w <a, rfl, ha>
  exact Set.mem_insert_iff.mpr (Or.inr (Set.mem_union_left _
    (Set.mem_union_left _ (t0_mem_j0.mpr (V_0.sub_master hX ha)))))

theorem j1_subset_master3 {Y : Set b} (hY : V_1.mem Y) :
    (j1 Y : Set (Option (a ? b ? ?)))  subseteq  master3 V_0 V_1 V_2 := by
  rintro w <b, rfl, hb>
  exact Set.mem_insert_iff.mpr (Or.inr (Set.mem_union_left _
    (Set.mem_union_right _ (t1_mem_j1.mpr (V_1.sub_master hY hb)))))

theorem j2_subset_master3 {Z : Set ?} (hZ : V_2.mem Z) :
    (j2 Z : Set (Option (a ? b ? ?)))  subseteq  master3 V_0 V_1 V_2 := by
  rintro w <c, rfl, hc>
  exact Set.mem_insert_iff.mpr (Or.inr (Set.mem_union_right _ (t2_mem_j2.mpr (V_2.
    sub_master hZ hc))))

theorem master3_inter_j0 {X : Set a} (hX : V_0.mem X) :
    (master3 V_0 V_1 V_2  I  j0 X : Set (Option (a ? b ? ?))) = j0 X :=
  Set.inter_eq_right.mpr (j0_subset_master3 hX)

theorem master3_inter_j1 {Y : Set b} (hY : V_1.mem Y) :
    (master3 V_0 V_1 V_2  I  j1 Y : Set (Option (a ? b ? ?))) = j1 Y :=
  Set.inter_eq_right.mpr (j1_subset_master3 hY)

theorem master3_inter_j2 {Z : Set ?} (hZ : V_2.mem Z) :
    (master3 V_0 V_1 V_2  I  j2 Z : Set (Option (a ? b ? ?))) = j2 Z :=
  Set.inter_eq_right.mpr (j2_subset_master3 hZ)

namespace Example62C

/-- **Example 6.2 -- the shape of a `C`-neighbourhood.** Every neighbourhood of `C` is
    the master
`Sigma* = cone []`, the terminator `{?} = {[]}`, a `0`-copy `0X` with `X  in  C`, or a
    `1`-copy `1X`. -/
theorem memC_cases {W : Set Str} (hW : C.mem W) :
    W = Set.univ  \/  W = ({[]} : Set Str)
       \/  (exists  X, C.mem X  /\  W = embBit false X)  \/  (exists  Y, C.mem Y  /\
    W = embBit true Y) := by
  rcases hW with <sigma, rfl> | <sigma, rfl>
  * cases sigma with
    | nil => exact Or.inl cone_nil
    | cons b sigma' => cases b with
      | false =>
        exact Or.inr (Or.inr (Or.inl <cone sigma', memC_cone sigma', (embBit_cone
    false sigma').symm>))
      | true =>
        exact Or.inr (Or.inr (Or.inr <cone sigma', memC_cone sigma', (embBit_cone true
     sigma').symm>))
  * cases sigma with
    | nil => exact Or.inr (Or.inl rfl)
```

```
    | cons b sigma' => cases b with
      | false =>
        exact Or.inr (Or.inr (Or.inl <{sigma'}, memC_singleton sigma', (
    embBit_singleton false sigma').symm>))
      | true =>
        exact Or.inr (Or.inr (Or.inr <{sigma'}, memC_singleton sigma', (
    embBit_singleton true sigma').symm>))

/-- Prepending a bit lands back in `C`. -/
theorem memC_embBit (b : Bool) {X : Set Str} (hX : C.mem X) : C.mem (embBit b X) := by
  rw [embBit_eq_prepend]; exact Example44.memC_prepend [b] hX

/-- If `bW  in  C` then `W  in  C`. -/
theorem memC_embBit_inv {b : Bool} {W : Set Str} (h : C.mem (embBit b W)) : C.mem W :=
     by
  rcases h with <sigma, hsigma> | <sigma, hsigma>
  * have hmem : sigma  in  embBit b W := hsigma  |>  (show sigma  in  cone sigma from
    List.prefix_rfl)
    obtain <w', rfl, -> := hmem
    rw [ <-  embBit_cone] at hsigma; rw [embBit_injective hsigma]; exact memC_cone w'
  * have hmem : sigma  in  embBit b W := hsigma  |>  (Set.mem_singleton_iff.mpr rfl :
    sigma  in  ({sigma} : Set Str))
    obtain <w', rfl, -> := hmem
    rw [ <-  embBit_singleton] at hsigma; rw [embBit_injective hsigma]; exact
    memC_singleton w'

theorem singleton_nil_inter_embBit (b : Bool) (X : Set Str) :
    (({[]} : Set Str)  I  embBit b X) = {} := by
  ext w
  simp only [Set.mem_inter_iff, Set.mem_singleton_iff, mem_embBit, Set.
    mem_empty_iff_false,
    iff_false, not_and]
  rintro rfl <w', heq, ->
  exact absurd heq (by simp)

theorem singleton_nil_ne_embBit (b : Bool) (X : Set Str) : ({[]} : Set Str)  !=
    embBit b X := by
  intro h
  exact nil_not_mem_embBit (h  |>  (Set.mem_singleton_iff.mpr rfl : ([] : Str)  in
    ({[]} : Set Str)))

theorem singleton_nil_ne_univ : ({[]} : Set Str)  !=  Set.univ := by
  intro h
  have hmem : ([true] : Str)  in  ({[]} : Set Str) := h  |>  Set.mem_univ _
  rw [Set.mem_singleton_iff] at hmem; exact absurd hmem (by simp)

end Example62C

/-- **Example 6.2 -- the three-way separated sum `D_0 + D_1 + D_2`** over `{?}  U  0
    Delta_0  U  1Delta_1  U  2Delta_2`,
under the standing assumption that no neighbourhood of any factor is empty. -/
def sum3 (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) (V_2 :
    NeighborhoodSystem ?)
    (h_0 : forall  X, V_0.mem X  ->  X.Nonempty) (h_1 : forall  Y, V_1.mem Y  ->  Y.
    Nonempty)
    (h_2 : forall  Z, V_2.mem Z  ->  Z.Nonempty) : NeighborhoodSystem (Option (a ? b ?
```

```
   ?)) where
mem W := W = master3 V_0 V_1 V_2  \/  (exists  X, V_0.mem X  /\  W = j0 X)
   \/  (exists  Y, V_1.mem Y  /\  W = j1 Y)  \/  (exists  Z, V_2.mem Z  /\  W = j2 Z
  )
master := master3 V_0 V_1 V_2
master_nonempty := <none, none_mem_master3>
master_mem := Or.inl rfl
sub_master := by
  rintro W (rfl | <X, hX, rfl> | <Y, hY, rfl> | <Z, hZ, rfl>)
  * exact subset_rfl
  * exact j0_subset_master3 hX
  * exact j1_subset_master3 hY
  * exact j2_subset_master3 hZ
inter_mem := by
  have hne : forall  W, (W = master3 V_0 V_1 V_2  \/  (exists  X, V_0.mem X  /\  W =
   j0 X)
       \/  (exists  Y, V_1.mem Y  /\  W = j1 Y)  \/  (exists  Z, V_2.mem Z  /\  W =
  j2 Z))  ->
      (W : Set (Option (a ? b ? ?))).Nonempty := by
    rintro W (rfl | <X, hX, rfl> | <Y, hY, rfl> | <Z, hZ, rfl>)
    * exact <none, none_mem_master3>
    * exact j0_nonempty (h_0 X hX)
    * exact j1_nonempty (h_1 Y hY)
    * exact j2_nonempty (h_2 Z hZ)
  rintro W W' Z hW hW' hZ hZsub
  rcases hW with rfl | <X, hX, rfl> | <Y, hY, rfl> | <Zc, hZc, rfl>
  * rcases hW' with rfl | <X', hX', rfl> | <Y', hY', rfl> | <Z', hZ', rfl>
    * rw [Set.inter_self]; exact Or.inl rfl
    * rw [master3_inter_j0 hX']; exact Or.inr (Or.inl <X', hX', rfl>)
    * rw [master3_inter_j1 hY']; exact Or.inr (Or.inr (Or.inl <Y', hY', rfl>))
    * rw [master3_inter_j2 hZ']; exact Or.inr (Or.inr (Or.inr <Z', hZ', rfl>))
  * rcases hW' with rfl | <X', hX', rfl> | <Y', hY', rfl> | <Z', hZ', rfl>
    * rw [Set.inter_comm, master3_inter_j0 hX]; exact Or.inr (Or.inl <X, hX, rfl>)
    * rw [j0_inter_j0] at hZsub  |-
      rcases hZ with rfl | <Z0, hZ0, rfl> | <Z1, hZ1, rfl> | <Z2, hZ2, rfl>
      * exact absurd (hZsub none_mem_master3) (by simp)
      * exact Or.inr (Or.inl <X  I  X', V_0.inter_mem hX hX' hZ0 (j0_subset_j0.mp
  hZsub), rfl>)
      * obtain <b, hb> := h_1 Z1 hZ1; exact absurd (hZsub (t1_mem_j1.mpr hb)) (by
  simp)
      * obtain <c, hc> := h_2 Z2 hZ2; exact absurd (hZsub (t2_mem_j2.mpr hc)) (by
  simp)
    * rw [j0_inter_j1] at hZsub  |-
      obtain <t, ht> := hne Z hZ; exact absurd (hZsub ht) (Set.notMem_empty t)
    * rw [j0_inter_j2] at hZsub  |-
      obtain <t, ht> := hne Z hZ; exact absurd (hZsub ht) (Set.notMem_empty t)
  * rcases hW' with rfl | <X', hX', rfl> | <Y', hY', rfl> | <Z', hZ', rfl>
    * rw [Set.inter_comm, master3_inter_j1 hY]; exact Or.inr (Or.inr (Or.inl <Y, hY,
   rfl>))
    * rw [Set.inter_comm, j0_inter_j1] at hZsub  |-
      obtain <t, ht> := hne Z hZ; exact absurd (hZsub ht) (Set.notMem_empty t)
    * rw [j1_inter_j1] at hZsub  |-
      rcases hZ with rfl | <Z0, hZ0, rfl> | <Z1, hZ1, rfl> | <Z2, hZ2, rfl>
      * exact absurd (hZsub none_mem_master3) (by simp)
      * obtain <a, ha> := h_0 Z0 hZ0; exact absurd (hZsub (t0_mem_j0.mpr ha)) (by
  simp)
```

```
        * exact Or.inr (Or.inr (Or.inl <Y  I  Y', V_1.inter_mem hY hY' hZ1 (
    j1_subset_j1.mp hZsub), rfl>))
        * obtain <c, hc> := h_2 Z2 hZ2; exact absurd (hZsub (t2_mem_j2.mpr hc)) (by
    simp)
      * rw [j1_inter_j2] at hZsub  |-
        obtain <t, ht> := hne Z hZ; exact absurd (hZsub ht) (Set.notMem_empty t)
    * rcases hW' with rfl | <X', hX', rfl> | <Y', hY', rfl> | <Z', hZ', rfl>
      * rw [Set.inter_comm, master3_inter_j2 hZc]; exact Or.inr (Or.inr (Or.inr <Zc,
    hZc, rfl>))
      * rw [Set.inter_comm, j0_inter_j2] at hZsub  |-
        obtain <t, ht> := hne Z hZ; exact absurd (hZsub ht) (Set.notMem_empty t)
      * rw [Set.inter_comm, j1_inter_j2] at hZsub  |-
        obtain <t, ht> := hne Z hZ; exact absurd (hZsub ht) (Set.notMem_empty t)
      * rw [j2_inter_j2] at hZsub  |-
        rcases hZ with rfl | <Z0, hZ0, rfl> | <Z1, hZ1, rfl> | <Z2, hZ2, rfl>
        * exact absurd (hZsub none_mem_master3) (by simp)
        * obtain <a, ha> := h_0 Z0 hZ0; exact absurd (hZsub (t0_mem_j0.mpr ha)) (by
    simp)
        * obtain <b, hb> := h_1 Z1 hZ1; exact absurd (hZsub (t1_mem_j1.mpr hb)) (by
    simp)
        * exact Or.inr (Or.inr (Or.inr <Zc  I  Z', V_2.inter_mem hZc hZ' hZ2 (
    j2_subset_j2.mp hZsub), rfl>))

namespace Example62C

/-- `?` is positive: its single neighbourhood `univ` (over the inhabited `Unit`) is
    nonempty. -/
theorem unitSys_nonempty : forall  X, unitSys.mem X  ->  X.Nonempty := by
  rintro X rfl; exact Set.univ_nonempty

/-- The right-hand side of the domain equation: the three-way sum `? + C + C`. -/
abbrev CC : NeighborhoodSystem (Option (Unit ? Str ? Str)) :=
  sum3 unitSys C C unitSys_nonempty C_nonempty C_nonempty

theorem sum3_mem_j1_inv {X : Set Str} (h : CC.mem (j1 X)) : C.mem X := by
  rcases h with h0 | <U, hU, heq> | <Y, hY, heq> | <Z, hZ, heq>
  * exact absurd (h0  |>  none_mem_master3) none_not_mem_j1
  * obtain <a, ha> := unitSys_nonempty U hU; exact absurd (heq  |>  (t0_mem_j0.mpr ha)
    ) t0_not_mem_j1
  * rw [j1_injective heq]; exact hY
  * obtain <c, hc> := C_nonempty Z hZ; exact absurd (heq  |>  (t2_mem_j2.mpr hc))
    t2_not_mem_j1

theorem sum3_mem_j2_inv {Y : Set Str} (h : CC.mem (j2 Y)) : C.mem Y := by
  rcases h with h0 | <U, hU, heq> | <Z, hZ, heq> | <Y', hY', heq>
  * exact absurd (h0  |>  none_mem_master3) none_not_mem_j2
  * obtain <a, ha> := unitSys_nonempty U hU; exact absurd (heq  |>  (t0_mem_j0.mpr ha)
    ) t0_not_mem_j2
  * obtain <c, hc> := C_nonempty Z hZ; exact absurd (heq  |>  (t1_mem_j1.mpr hc))
    t1_not_mem_j2
  * rw [j2_injective heq]; exact hY'

theorem sum3_mem_nonempty {W : Set (Option (Unit ? Str ? Str))} (h : CC.mem W) : W.
    Nonempty := by
  rcases h with rfl | <U, hU, rfl> | <Y, hY, rfl> | <Z, hZ, rfl>
  * exact <none, none_mem_master3>
```

```
  * exact j0_nonempty (unitSys_nonempty U hU)
  * exact j1_nonempty (C_nonempty Y hY)
  * exact j2_nonempty (C_nonempty Z hZ)

/-- **Example 6.2 -- forward half of `C  ~=  ? + C + C`.** -/
def toCC (x : C.Element) : CC.Element where
  mem W := W = master3 unitSys C C
     \/  (W = j0 (Set.univ : Set Unit)  /\  x.mem ({[]} : Set Str))
     \/  (exists  X, C.mem X  /\  W = j1 X  /\  x.mem (embBit false X))
     \/  (exists  Y, C.mem Y  /\  W = j2 Y  /\  x.mem (embBit true Y))
  sub := by
    rintro W (rfl | <rfl, -> | <X, hX, rfl, -> | <Y, hY, rfl, ->)
    * exact Or.inl rfl
    * exact Or.inr (Or.inl <Set.univ, rfl, rfl>)
    * exact Or.inr (Or.inr (Or.inl <X, hX, rfl>))
    * exact Or.inr (Or.inr (Or.inr <Y, hY, rfl>))
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | <rfl, hzU> | <X, hX, rfl, hzF> | <Y, hY, rfl, hzT>)
      (rfl | <rfl, hzU'> | <X', hX', rfl, hzF'> | <Y', hY', rfl, hzT'>)
    * exact Or.inl (by rw [Set.inter_self])
    * exact Or.inr (Or.inl <master3_inter_j0 rfl, hzU'>)
    * exact Or.inr (Or.inr (Or.inl <X', hX', master3_inter_j1 hX', hzF'>))
    * exact Or.inr (Or.inr (Or.inr <Y', hY', master3_inter_j2 hY', hzT'>))
    * exact Or.inr (Or.inl <by rw [Set.inter_comm, master3_inter_j0 rfl], hzU>)
    * refine Or.inr (Or.inl <?_, hzU>)
      rw [j0_inter_j0, Set.inter_self]
    * exfalso
      have hx := x.inter_mem hzU hzF'
      rw [singleton_nil_inter_embBit] at hx

      obtain <t, ht> := C_nonempty _ (x.sub hx); exact Set.notMem_empty t ht
    * exfalso
      have hx := x.inter_mem hzU hzT'
      rw [singleton_nil_inter_embBit] at hx
      obtain <t, ht> := C_nonempty _ (x.sub hx); exact Set.notMem_empty t ht
    * exact Or.inr (Or.inr (Or.inl <X, hX, by rw [Set.inter_comm, master3_inter_j1 hX
    ], hzF>))
    * exfalso
      have hx := x.inter_mem hzF hzU'
      rw [Set.inter_comm, singleton_nil_inter_embBit] at hx
      obtain <t, ht> := C_nonempty _ (x.sub hx); exact Set.notMem_empty t ht
    * refine Or.inr (Or.inr (Or.inl <X  I  X', ?_, j1_inter_j1 X X', ?_>))
      * have hx := x.inter_mem hzF hzF'; rw [embBit_inter] at hx; exact
    memC_embBit_inv (x.sub hx)
      * have hx := x.inter_mem hzF hzF'; rwa [embBit_inter] at hx
    * exfalso
      have hx := x.inter_mem hzF hzT'
      rw [embBit_inter_ne (show (false : Bool)  !=  true by decide)] at hx
      obtain <t, ht> := C_nonempty _ (x.sub hx); exact Set.notMem_empty t ht
    * exact Or.inr (Or.inr (Or.inr <Y, hY, by rw [Set.inter_comm, master3_inter_j2 hY
    ], hzT>))
    * exfalso
      have hx := x.inter_mem hzT hzU'
      rw [Set.inter_comm, singleton_nil_inter_embBit] at hx
      obtain <t, ht> := C_nonempty _ (x.sub hx); exact Set.notMem_empty t ht
```

```
    * exfalso
      have hx := x.inter_mem hzT hzF'
      rw [embBit_inter_ne (show (true : Bool)  !=  false by decide)] at hx
      obtain <t, ht> := C_nonempty _ (x.sub hx); exact Set.notMem_empty t ht
    * refine Or.inr (Or.inr (Or.inr <Y  I  Y', ?_, j2_inter_j2 Y Y', ?_>))
      * have hx := x.inter_mem hzT hzT'; rw [embBit_inter] at hx; exact
    memC_embBit_inv (x.sub hx)
      * have hx := x.inter_mem hzT hzT'; rwa [embBit_inter] at hx
  up_mem := by
    rintro W W' (rfl | <rfl, hzU> | <X, hX, rfl, hzF> | <Y, hY, rfl, hzT>) hW' hsub
    * exact Or.inl (eq_master3_of_subset hsub (CC.sub_master hW'))
    * rcases hW' with rfl | <X', hX', rfl> | <Y', hY', rfl> | <Z', hZ', rfl>
      * exact Or.inl rfl
      * obtain rfl := hX'; exact Or.inr (Or.inl <rfl, hzU>)
      * exact absurd (hsub (t0_mem_j0.mpr (Set.mem_univ ()))) t0_not_mem_j1
      * exact absurd (hsub (t0_mem_j0.mpr (Set.mem_univ ()))) t0_not_mem_j2
    * rcases hW' with rfl | <X', hX', rfl> | <Y', hY', rfl> | <Z', hZ', rfl>
      * exact Or.inl rfl
      * obtain <b, hb> := C_nonempty X hX
        exact absurd (hsub (t1_mem_j1.mpr hb)) t1_not_mem_j0
      * refine Or.inr (Or.inr (Or.inl <Y', hY', rfl, ?_>))
        exact x.up_mem hzF (memC_embBit false hY') (embBit_subset.mpr (j1_subset_j1.mp
     hsub))
      * obtain <b, hb> := C_nonempty X hX
        exact absurd (hsub (t1_mem_j1.mpr hb)) t1_not_mem_j2
    * rcases hW' with rfl | <X', hX', rfl> | <Y', hY', rfl> | <Z', hZ', rfl>
      * exact Or.inl rfl
      * obtain <c, hc> := C_nonempty Y hY
        exact absurd (hsub (t2_mem_j2.mpr hc)) t2_not_mem_j0
      * obtain <c, hc> := C_nonempty Y hY
        exact absurd (hsub (t2_mem_j2.mpr hc)) t2_not_mem_j1
      * refine Or.inr (Or.inr (Or.inr <Z', hZ', rfl, ?_>))
        exact x.up_mem hzT (memC_embBit true hZ') (embBit_subset.mpr (j2_subset_j2.mp
    hsub))

@[simp] theorem toCC_mem_j0 {x : C.Element} :
    (toCC x).mem (j0 (Set.univ : Set Unit))  <->  x.mem ({[]} : Set Str) := by
  constructor
  * rintro (h0 | <-, hz> | <X', hX', heq, hz> | <Y', hY', heq, hz>)
    * exact absurd (h0  |>  none_mem_master3) none_not_mem_j0
    * exact hz
    * exact absurd (heq  |>  (t0_mem_j0.mpr (Set.mem_univ ()))) t0_not_mem_j1
    * exact absurd (heq  |>  (t0_mem_j0.mpr (Set.mem_univ ()))) t0_not_mem_j2
  * intro hz; exact Or.inr (Or.inl <rfl, hz>)

@[simp] theorem toCC_mem_j1 {x : C.Element} {X : Set Str} (hX : C.mem X) :
    (toCC x).mem (j1 X)  <->  x.mem (embBit false X) := by
  constructor
  * rintro (h0 | <heq, hz> | <X', hX', heqj, hz> | <Y', hY', heqj, hz>)
    * exact absurd (h0  |>  none_mem_master3) none_not_mem_j1
    * obtain <b, hb> := C_nonempty X hX
      exact absurd (heq  |>  (t1_mem_j1.mpr hb)) t1_not_mem_j0
    * rw [j1_injective heqj]; exact hz
    * obtain <b, hb> := C_nonempty X hX
      exact absurd (heqj  |>  (t1_mem_j1.mpr hb)) t1_not_mem_j2
  * intro hz; exact Or.inr (Or.inr (Or.inl <X, hX, rfl, hz>))
```

```
@[simp] theorem toCC_mem_j2 {x : C.Element} {Y : Set Str} (hY : C.mem Y) :
    (toCC x).mem (j2 Y)  <->  x.mem (embBit true Y) := by
  constructor
  * rintro (h0 | <heq, hz> | <X', hX', heqj, hz> | <Y', hY', heqj, hz>)
    * exact absurd (h0  |>  none_mem_master3) none_not_mem_j2
    * obtain <c, hc> := C_nonempty Y hY
      exact absurd (heq  |>  (t2_mem_j2.mpr hc)) t2_not_mem_j0
    * obtain <c, hc> := C_nonempty Y hY
      exact absurd (heqj  |>  (t2_mem_j2.mpr hc)) t2_not_mem_j1
    * rw [j2_injective heqj]; exact hz
  * intro hz; exact Or.inr (Or.inr (Or.inr <Y, hY, rfl, hz>))

/-- **Example 6.2 -- inverse half of `C  ~=  ? + C + C`.** -/
def fromCC (s : CC.Element) : C.Element where
  mem W := W = Set.univ
     \/  (W = ({[]} : Set Str)  /\  s.mem (j0 (Set.univ : Set Unit)))
     \/  (exists  X, C.mem X  /\  W = embBit false X  /\  s.mem (j1 X))
     \/  (exists  Y, C.mem Y  /\  W = embBit true Y  /\  s.mem (j2 Y))
  sub := by
    rintro W (rfl | <rfl, -> | <X, hX, rfl, -> | <Y, hY, rfl, ->)
    * exact Or.inl <[], cone_nil.symm>
    * exact memC_singleton []
    * exact memC_embBit false hX
    * exact memC_embBit true hY
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | <rfl, hsU> | <X, hX, rfl, hsF> | <Y, hY, rfl, hsT>)
      (rfl | <rfl, hsU'> | <X', hX', rfl, hsF'> | <Y', hY', rfl, hsT'>)
    * exact Or.inl (by rw [Set.inter_self])
    * exact Or.inr (Or.inl <by rw [Set.univ_inter], hsU'>)
    * exact Or.inr (Or.inr (Or.inl <X', hX', by rw [Set.univ_inter], hsF'>))
    * exact Or.inr (Or.inr (Or.inr <Y', hY', by rw [Set.univ_inter], hsT'>))
    * exact Or.inr (Or.inl <by rw [Set.inter_univ], hsU>)
    * exact Or.inr (Or.inl <by rw [Set.inter_self], hsU>)
    * exfalso
      have hs := s.inter_mem hsU hsF'; rw [j0_inter_j1] at hs
      obtain <t, ht> := sum3_mem_nonempty (s.sub hs); exact Set.notMem_empty t ht
    * exfalso
      have hs := s.inter_mem hsU hsT'; rw [j0_inter_j2] at hs
      obtain <t, ht> := sum3_mem_nonempty (s.sub hs); exact Set.notMem_empty t ht
    * exact Or.inr (Or.inr (Or.inl <X, hX, by rw [Set.inter_univ], hsF>))
    * exfalso
      have hs := s.inter_mem hsF hsU'; rw [Set.inter_comm, j0_inter_j1] at hs
      obtain <t, ht> := sum3_mem_nonempty (s.sub hs); exact Set.notMem_empty t ht
    * refine Or.inr (Or.inr (Or.inl <X  I  X', ?_, embBit_inter false X X', ?_>))
      * have hs := s.inter_mem hsF hsF'; rw [j1_inter_j1] at hs
        exact sum3_mem_j1_inv (s.sub hs)
      * have hs := s.inter_mem hsF hsF'; rw [j1_inter_j1] at hs; exact hs
    * exfalso
      have hs := s.inter_mem hsF hsT'; rw [j1_inter_j2] at hs
      obtain <t, ht> := sum3_mem_nonempty (s.sub hs); exact Set.notMem_empty t ht
    * exact Or.inr (Or.inr (Or.inr <Y, hY, by rw [Set.inter_univ], hsT>))
    * exfalso
      have hs := s.inter_mem hsT hsU'; rw [Set.inter_comm, j0_inter_j2] at hs
      obtain <t, ht> := sum3_mem_nonempty (s.sub hs); exact Set.notMem_empty t ht
```

```
    * exfalso
      have hs := s.inter_mem hsT hsF'; rw [Set.inter_comm, j1_inter_j2] at hs
      obtain <t, ht> := sum3_mem_nonempty (s.sub hs); exact Set.notMem_empty t ht
    * refine Or.inr (Or.inr (Or.inr <Y  I  Y', ?_, embBit_inter true Y Y', ?_>))
      * have hs := s.inter_mem hsT hsT'; rw [j2_inter_j2] at hs
        exact sum3_mem_j2_inv (s.sub hs)
      * have hs := s.inter_mem hsT hsT'; rw [j2_inter_j2] at hs; exact hs
  up_mem := by
    rintro W W' (rfl | <rfl, hsU> | <X, hX, rfl, hsF> | <Y, hY, rfl, hsT>) hW' hsub
    * exact Or.inl (Set.univ_subset_iff.mp hsub)
    * rcases memC_cases hW' with rfl | rfl | <X', hX', rfl> | <Y', hY', rfl>
      * exact Or.inl rfl
      * exact Or.inr (Or.inl <rfl, hsU>)
      * exact absurd (hsub (Set.mem_singleton_iff.mpr rfl)) nil_not_mem_embBit
      * exact absurd (hsub (Set.mem_singleton_iff.mpr rfl)) nil_not_mem_embBit
    * rcases memC_cases hW' with rfl | rfl | <X', hX', rfl> | <Y', hY', rfl>
      * exact Or.inl rfl
      * exfalso
        obtain <a, ha> := C_nonempty X hX
        have hm := hsub (<a, rfl, ha> : (false :: a)  in  embBit false X)
        rw [Set.mem_singleton_iff] at hm; exact absurd hm (by simp)
      * refine Or.inr (Or.inr (Or.inl <X', hX', rfl, ?_>))
        exact s.up_mem hsF (Or.inr (Or.inr (Or.inl <X', hX', rfl>)))
          (j1_subset_j1.mpr (embBit_subset.mp hsub))
      * exfalso
        obtain <a, ha> := C_nonempty X hX
        obtain <w', he, -> := hsub (<a, rfl, ha> : (false :: a)  in  embBit false X)
        rw [List.cons.injEq] at he; exact absurd he.1 (by decide)
    * rcases memC_cases hW' with rfl | rfl | <X', hX', rfl> | <Y', hY', rfl>
      * exact Or.inl rfl
      * exfalso
        obtain <a, ha> := C_nonempty Y hY
        have hm := hsub (<a, rfl, ha> : (true :: a)  in  embBit true Y)
        rw [Set.mem_singleton_iff] at hm; exact absurd hm (by simp)
      * exfalso
        obtain <a, ha> := C_nonempty Y hY
        obtain <w', he, -> := hsub (<a, rfl, ha> : (true :: a)  in  embBit true Y)
        rw [List.cons.injEq] at he; exact absurd he.1 (by decide)
      * refine Or.inr (Or.inr (Or.inr <Y', hY', rfl, ?_>))
        exact s.up_mem hsT (Or.inr (Or.inr (Or.inr <Y', hY', rfl>)))
          (j2_subset_j2.mpr (embBit_subset.mp hsub))

@[simp] theorem fromCC_mem_nil {s : CC.Element} :
    (fromCC s).mem ({[]} : Set Str)  <->  s.mem (j0 (Set.univ : Set Unit)) := by
  constructor
  * rintro (h0 | <-, hs> | <X', hX', heq, hs> | <Y', hY', heq, hs>)
    * exact absurd h0 singleton_nil_ne_univ
    * exact hs
    * exact absurd heq (singleton_nil_ne_embBit false X')
    * exact absurd heq (singleton_nil_ne_embBit true Y')
  * intro hs; exact Or.inr (Or.inl <rfl, hs>)

@[simp] theorem fromCC_mem_embF {s : CC.Element} {X : Set Str} (hX : C.mem X) :
    (fromCC s).mem (embBit false X)  <->  s.mem (j1 X) := by
  constructor
  * rintro (h0 | <heq, hs> | <X', hX', heqj, hs> | <Y', hY', heqj, hs>)
```

```
    * exact absurd h0 (embBit_ne_univ false X)
    * exact absurd heq.symm (singleton_nil_ne_embBit false X)
    * rw [embBit_injective heqj]; exact hs
    * exact absurd heqj (embBit_ne (show (false : Bool)  !=  true by decide) (
    C_nonempty X hX))
  * intro hs; exact Or.inr (Or.inr (Or.inl <X, hX, rfl, hs>))

@[simp] theorem fromCC_mem_embT {s : CC.Element} {Y : Set Str} (hY : C.mem Y) :
    (fromCC s).mem (embBit true Y)  <->  s.mem (j2 Y) := by
  constructor
  * rintro (h0 | <heq, hs> | <X', hX', heqj, hs> | <Y', hY', heqj, hs>)
    * exact absurd h0 (embBit_ne_univ true Y)
    * exact absurd heq.symm (singleton_nil_ne_embBit true Y)
    * exact absurd heqj.symm (embBit_ne (show (false : Bool)  !=  true by decide) (
    C_nonempty X' hX'))
    * rw [embBit_injective heqj]; exact hs
  * intro hs; exact Or.inr (Or.inr (Or.inr <Y, hY, rfl, hs>))

theorem fromCC_toCC (x : C.Element) : fromCC (toCC x) = x := by
  apply NeighborhoodSystem.Element.ext
  intro W
  constructor
  * rintro (rfl | <rfl, hs> | <X, hX, rfl, hs> | <Y, hY, rfl, hs>)
    * exact x.master_mem
    * exact toCC_mem_j0.mp hs
    * exact (toCC_mem_j1 hX).mp hs
    * exact (toCC_mem_j2 hY).mp hs
  * intro hW
    rcases memC_cases (x.sub hW) with rfl | rfl | <X, hX, rfl> | <Y, hY, rfl>
    * exact Or.inl rfl
    * exact Or.inr (Or.inl <rfl, toCC_mem_j0.mpr hW>)
    * exact Or.inr (Or.inr (Or.inl <X, hX, rfl, (toCC_mem_j1 hX).mpr hW>))
    * exact Or.inr (Or.inr (Or.inr <Y, hY, rfl, (toCC_mem_j2 hY).mpr hW>))

theorem toCC_fromCC (s : CC.Element) : toCC (fromCC s) = s := by
  apply NeighborhoodSystem.Element.ext
  intro W
  constructor
  * rintro (rfl | <rfl, hs> | <X, hX, rfl, hs> | <Y, hY, rfl, hs>)
    * exact s.master_mem
    * exact fromCC_mem_nil.mp hs
    * exact (fromCC_mem_embF hX).mp hs
    * exact (fromCC_mem_embT hY).mp hs
  * intro hW
    rcases s.sub hW with rfl | <U, hU, rfl> | <X, hX, rfl> | <Y, hY, rfl>
    * exact Or.inl rfl
    * obtain rfl := hU
      exact Or.inr (Or.inl <rfl, fromCC_mem_nil.mpr hW>)
    * exact Or.inr (Or.inr (Or.inl <X, hX, rfl, (fromCC_mem_embF hX).mpr hW>))
    * exact Or.inr (Or.inr (Or.inr <Y, hY, rfl, (fromCC_mem_embT hY).mpr hW>))

/-- **Example 6.2 (Scott 1981, PRG-19) -- the isomorphism `|C| ?o |? + C + C|`.** -/
def ccEquiv : C.Element ?o CC.Element where
  toFun := toCC
  invFun := fromCC
  left_inv := fromCC_toCC
```

```
  right_inv := toCC_fromCC
  map_rel_iff' := by
    intro x x'
    constructor
    * intro h W hW
      rcases memC_cases (x.sub hW) with rfl | rfl | <A, hA, rfl> | <A, hA, rfl>
      * exact x'.master_mem
      * exact toCC_mem_j0.mp (h _ (Or.inr (Or.inl <rfl, hW>)))
      * exact (toCC_mem_j1 hA).mp (h _ (Or.inr (Or.inr (Or.inl <A, hA, rfl, hW>))))
      * exact (toCC_mem_j2 hA).mp (h _ (Or.inr (Or.inr (Or.inr <A, hA, rfl, hW>))))
    * intro h W hW
      rcases hW with rfl | <rfl, hz> | <X, hX, rfl, hz> | <Y, hY, rfl, hz>
      * exact Or.inl rfl
      * exact Or.inr (Or.inl <rfl, h _ hz>)
      * exact Or.inr (Or.inr (Or.inl <X, hX, rfl, h _ hz>))
      * exact Or.inr (Or.inr (Or.inr <Y, hY, rfl, h _ hz>))

/-- **Example 6.2 (Scott 1981, PRG-19) -- the domain equation `C  ~=  {{?}} + C + C`.**
     Scott's domain
`C` of finite-or-infinite binary sequences is, as a domain, isomorphic to the three-way
     separated sum
`? + C + C`: a sequence is bottom, the finished empty sequence `?` (the `?` summand),
    or begins with
`0` or `1` (the two `C` summands). -/
theorem C_domain_equation :
    C  ~= ? sum3 unitSys C C unitSys_nonempty C_nonempty C_nonempty :=
  <ccEquiv>

end Example62C

end

section
variable {a b ? : Type*}

theorem j0_injective {X X' : Set a}
    (h : (j0 X : Set (Option (a ? b ? ?))) = j0 X') : X = X' :=
  Set.Subset.antisymm (j0_subset_j0.mp h.subset) (j0_subset_j0.mp h.symm.subset)
end

namespace Example62Regular
open Language

section
variable {n : Nat}

/-- A **total tree** over the alphabet `Sigma = Fin n` (Scott's `n`-ary `?` tree): an
    assignment of a
`?` label (`Bool`, `true = +`) to every node, addressed by a finite selector `sigma  in
      Sigma*`. A tree is
*total* in Scott's sense (every node carries a genuine label), as opposed to the
    partial elements of
the domain `A` of `Example62A.lean` where labels may be `False`. -/
abbrev Tree (n : Nat) : Type := List (Fin n)  ->  Bool

/-- Scott's `pos : A  ->  T`: the `?` label at the **root** of the tree (`pos(?<...>) =
```

```
     true/false`). -/
def pos (a : Tree n) : Bool := a []

/-- Scott's subtree selector `sigma  |->  asigma`: the subtree of `a` reached by the
    selector `sigma`. Defined so
that `(asigma)tau = a(sigmatau)`; the `pos`-label at node `tau` of `asigma` is the
    label at node `sigmatau` of `a`. -/
def select (a : Tree n) (sigma : List (Fin n)) : Tree n := fun tau => a (sigma ++ tau)

@[simp] theorem select_apply (a : Tree n) (sigma tau : List (Fin n)) : select a sigma
    tau = a (sigma ++ tau) := rfl

/-- The `i`-th immediate subtree `a_i = a i` (Scott's children of the root node). -/
def child (a : Tree n) (i : Fin n) : Tree n := select a [i]

@[simp] theorem select_nil (a : Tree n) : select a [] = a := by
  funext tau; simp

@[simp] theorem pos_select (a : Tree n) (sigma : List (Fin n)) : pos (select a sigma)
    = a sigma := by
  simp [pos]

/-- **Scott's language of a tree.** `L_a = {sigma  in  Sigma* ? pos(asigma) = true}` --
     the selectors that land on a
`+` node. Equivalently (by `pos_select`) the set of `sigma` with `a sigma = true`. A `
    Language (Fin n)` is
just a set of words over the alphabet, exactly Scott's "language". -/
def treeLang (a : Tree n) : Language (Fin n) := {sigma | a sigma = true}

@[simp] theorem mem_treeLang {a : Tree n} {sigma : List (Fin n)} : sigma  in  treeLang
     a  <->  a sigma = true := Iff.rfl

/-- **Eventual periodicity (Scott).** A tree `a` is *eventually periodic* iff the set
    of its subtrees
`{asigma ? sigma  in  Sigma*}` is finite. (Equivalently -- by the picture -- the tree
    is built from finitely many
distinct subtrees, so it is "ultimately periodic" along every branch.) -/
def EventuallyPeriodic (a : Tree n) : Prop := (Set.range (select a)).Finite

/-- Composing selectors: `a(sigmatau) = (asigma)tau`. -/
theorem select_append (a : Tree n) (sigma tau : List (Fin n)) :
    select a (sigma ++ tau) = select (select a sigma) tau := by
  funext ?; simp [List.append_assoc]

/-- **Scott's selector recursion, step `a(isigma) = (a_i)sigma`.** Reading the digit `i
    ` then `sigma` is the same
as descending to the `i`-th child `a_i` and reading `sigma` there. -/
theorem select_cons (a : Tree n) (i : Fin n) (sigma : List (Fin n)) :
    select a (i :: sigma) = select (child a i) sigma := by
  funext tau; simp [child]

/-- `L_a` is genuinely `{sigma ? pos(asigma) = true}` as Scott writes it. -/
theorem treeLang_eq_pos (a : Tree n) : treeLang a = {sigma | pos (select a sigma) =
    true} := by
  simp only [pos_select]; rfl
```

```
/-- The label function recovers the tree from its language, so `treeLang` is **one-one
    **: two trees
with the same language are the same tree (a node's `?` label is recorded by whether its
     selector is
in the language). -/
theorem treeLang_injective : Function.Injective (treeLang (n := n)) := by
  intro a a' h
  funext sigma
  have hmem : (sigma  in  treeLang a)  <->  (sigma  in  treeLang a') := by rw [h]
  simp only [mem_treeLang] at hmem
  cases ha : a sigma <;> cases ha' : a' sigma <;> simp_all

/-- **The subtree is the left quotient.** `L_{asigma} = sigma^-1 L_a`: the language of
    the subtree reached by
reading `sigma` is exactly the *left quotient* of `L_a` by `sigma` (the residual / "
    state after reading
`sigma`"). This is the heart of Scott's "`a` is the initial state, `asigma` the state
    after reading `sigma`". -/
theorem treeLang_select (a : Tree n) (sigma : List (Fin n)) :
    treeLang (select a sigma) = (treeLang a).leftQuotient sigma := by
  ext tau
  simp only [mem_treeLang, select_apply, Language.mem_leftQuotient]

/-- **Scott's closing claim, made precise (Example 6.2).** A tree's language `L_a` is a
     *regular
event* of automata theory **iff** the tree is eventually periodic. This is the Myhill-
    Nerode theorem
in disguise: the subtrees `asigma` are the left quotients `sigma^-1 L_a` (`
    treeLang_select`), so "finitely many
subtrees" is "finitely many left quotients", which Myhill-Nerode equates with
    regularity. -/
theorem eventuallyPeriodic_iff_isRegular (a : Tree n) :
    EventuallyPeriodic a  <->  (treeLang a).IsRegular := by
  have hcomp : (treeLang a).leftQuotient = treeLang  comp  select a :=
    funext fun sigma => (treeLang_select a sigma).symm
  rw [Language.isRegular_iff_finite_range_leftQuotient, hcomp, Set.range_comp]
  exact (Set.finite_image_iff (treeLang_injective.injOn)).symm

/-- The inverse half of Scott's claim: **every regular event arises** as the language
    of an eventually
periodic tree. Concretely, take the tree whose root labels record membership in `L`;
    reading a tape
`sigma` lands on the residual language `sigma^-1 L`, of which there are finitely many
    exactly when `L` is
regular. -/
theorem isRegular_iff_exists_eventuallyPeriodic (L : Language (Fin n)) :
    L.IsRegular  <->  exists  a : Tree n, EventuallyPeriodic a  /\  treeLang a = L :=
    by
  classical
  constructor
  * intro hL
    refine <fun sigma => decide (sigma  in  L), ?_, ?_>
    * have hlang : treeLang (fun sigma => decide (sigma  in  L)) = L := by
        ext sigma; simp only [mem_treeLang, decide_eq_true_eq]
      rw [eventuallyPeriodic_iff_isRegular, hlang]; exact hL
    * ext sigma; simp only [mem_treeLang, decide_eq_true_eq]
```

```
  * rintro <a, hEP, rfl>
    exact (eventuallyPeriodic_iff_isRegular a).mp hEP

end

end Example62Regular
open NeighborhoodSystem ApproximableMap Example62 ExampleB

section
namespace Exercise619

variable {D_0 D_1 : NeighborhoodSystem Str}

/-- The master neighbourhood `{?}  U  0Delta_0  U  1Delta_1` of the concrete sum. -/
def sumTokMaster (D_0 D_1 : NeighborhoodSystem Str) : Set Str :=
  insert [] (embBit false D_0.master  U  embBit true D_1.master)

theorem embF_subset_sumTokMaster {X : Set Str} (hX : D_0.mem X) :
    embBit false X  subseteq  sumTokMaster D_0 D_1 :=
  (embBit_subset.mpr (D_0.sub_master hX)).trans (Set.subset_union_left.trans (Set.
    subset_insert _ _))

theorem embT_subset_sumTokMaster {Y : Set Str} (hY : D_1.mem Y) :

    embBit true Y  subseteq  sumTokMaster D_0 D_1 :=
  (embBit_subset.mpr (D_1.sub_master hY)).trans (Set.subset_union_right.trans (Set.
    subset_insert _ _))

theorem nil_mem_sumTokMaster : ([] : Str)  in  sumTokMaster D_0 D_1 := Set.mem_insert
    _ _

theorem embF_ne_sumTokMaster {X : Set Str} : embBit false X  !=  sumTokMaster D_0 D_1
    := fun h =>
  nil_not_mem_embBit (h.symm  |>  nil_mem_sumTokMaster)

theorem embT_ne_sumTokMaster {Y : Set Str} : embBit true Y  !=  sumTokMaster D_0 D_1
    := fun h =>
  nil_not_mem_embBit (h.symm  |>  nil_mem_sumTokMaster)

theorem sumTokMaster_inter_embF {X : Set Str} (hX : D_0.mem X) :
    sumTokMaster D_0 D_1  I  embBit false X = embBit false X :=
  Set.inter_eq_right.mpr (embF_subset_sumTokMaster hX)

theorem sumTokMaster_inter_embT {Y : Set Str} (hY : D_1.mem Y) :
    sumTokMaster D_0 D_1  I  embBit true Y = embBit true Y :=
  Set.inter_eq_right.mpr (embT_subset_sumTokMaster hY)

end Exercise619
end
end DomainTheory.NeighbourhoodSystems
end
```

3.7.4 Definition 6.3 — functor

DEFINITION 6.3. A *functor* on a category (into itself) associates with every domain X in the category another domain $T(X)$ and to every map

$$f : X \to Y$$

another map

$$T(f) : T(X) \to T(Y)$$

in such a way that identity maps and compositions are preserved:

$$T(I_X) = I_{T(X)}, \text{ and}$$

$$T(g \circ f) = T(g) \circ T(f),$$

whenever $f : X \to Y$ and $g : Y \to Z$. □

In the example from 6.1 we have not checked how the special **T** is a functor. The hint is that whenever $f : X \to Y$, then there is a map

$$f \times f : X \times X \to Y \times Y.$$

But there is also a map

$$I_D + f \times f : D + (X \times X) \to D + (Y \times Y)$$

and this suggests the definition of $T(f)$. The details are left to the exercises. Note that the map $T(f)$ just suggested is always strict, so **T** is a functor also for the category of strict maps.

One good reason for a little of the category-theoretic language is that the next definition becomes very neat indeed.

Definition_6_3.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.Exercise_6_28

@[expose] public section

section
universe u
namespace DomainTheory.NeighbourhoodSystems
variable {Obj : Type u} [Category Obj]
/- concordance: source L3625 DEFINITION 6.3 functor -/
/-- **Definition 6.3 (Scott 1981, PRG-19).** A *functor* on a category into itself (an
*endofunctor*): an assignment `obj` on objects and `map` on morphisms preserving
    identities
(`map_id`) and composition (`map_comp`). -/
structure Endofunctor (Obj : Type u) [Category Obj] where
  /-- The action on objects. -/
  obj : Obj  ->  Obj
  /-- The action on morphisms. -/
```

```
  map : {X Y : Obj}  ->  (X  -->  Y)  ->  ((obj X)  -->  (obj Y))
  /-- `T(I_X) = I_{T(X)}`. -/
  map_id : forall  (X : Obj), map ((? X)) = (? (obj X))
  /-- `T(g  comp  f) = T(g)  comp  T(f)`. -/
  map_comp : forall  {X Y Z : Obj} (g : (Y  -->  Z)) (f : (X  -->  Y)),
    map (g ? f) = (map g) ? (map f)

end DomainTheory.NeighbourhoodSystems
end

section
-- Iso.lean
universe u

namespace DomainTheory.NeighbourhoodSystems

variable {Obj : Type u} [Category Obj]

variable {T : Endofunctor Obj}

/-- An isomorphism in the category: a pair of mutually inverse morphisms. -/
structure Iso (X Y : Obj) where
  /-- The forward morphism. -/
  hom : (X  -->  Y)
  /-- The inverse morphism. -/
  inv : (Y  -->  X)
  /-- `inv  comp  hom = I_X`. -/
  hom_inv_id : inv ? hom = (? X)
  /-- `hom  comp  inv = I_Y`. -/
  inv_hom_id : hom ? inv = (? Y)

end DomainTheory.NeighbourhoodSystems
end
```

3.7.5 Definition 6.4 — algebra for a functor

DEFINITION 6.4. A ***T*-algebra** is a domain E in the category together with a map

$$k : T(E) \to E.$$

If $m : T(F) \to F$ is another ***T*-algebra**, then a *homomorphism* is a map $h : E \to F$ in the category such that the diagram

$$\begin{array}{ccc} T(E) & \xrightarrow{k} & E \\ \downarrow{\scriptstyle T(h)} & & \downarrow{\scriptstyle h} \\ T(F) & \xrightarrow{m} & F \end{array}$$

commutes; that is, the equation

$$h \circ k = m \circ T(h)$$

holds. □

In our example from 6.1 a T-algebra is a *strict* map

$$k : D + (E \times E) \to E.$$

But such strict maps are in a one-one correspondence with pairs of (not necessarily strict) maps

$$n : D \to E \quad \text{and} \quad p : E \times E \to E.$$

And the structure $\langle E, n, p\rangle$ is what we called a tree algebra. Definition 6.4 just makes this abstract. The reader should also work out the details showing that 6.4's definition of homomorphism is just what we ought to expect.

Note that the T-algebras and homomorphisms form a category. (Why?) The following definition is so abstract that it could be given for any category.

Definition_6_4.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.Definition_6_3

@[expose] public section

section
universe u
namespace DomainTheory.NeighbourhoodSystems
variable {Obj : Type u} [Category Obj]
/- concordance: source L3667 DEFINITION 6.4 algebra for a functor -/
/-- **Definition 6.4 (Scott 1981, PRG-19).** A *`T`-algebra*: a domain `carrier`
    together with a
structure map `str : T(carrier)  ->  carrier`. -/
structure TAlgebra (T : Endofunctor Obj) where
  /-- The underlying object `E`. -/
  carrier : Obj
  /-- The structure map `k : T(E)  ->  E`. -/
  str : ((T.obj carrier)  -->  carrier)

end DomainTheory.NeighbourhoodSystems
end

section
-- AlgHom.lean
universe u

namespace DomainTheory.NeighbourhoodSystems

variable {Obj : Type u} [Category Obj]

variable {T : Endofunctor Obj}

/-- **Definition 6.4 (Scott 1981, PRG-19).** A *homomorphism* of `T`-algebras `(E,k)
    ->  (F,m)`: a map
`hom : E  ->  F` making the square commute, i.e. `hom  comp  k = m  comp  T(hom)`. -/
structure AlgHom (A B : TAlgebra T) where
  /-- The underlying morphism `h : E  ->  F`. -/
  hom : (A.carrier  -->  B.carrier)
  /-- The homomorphism square `h  comp  k = m  comp  T(h)`. -/
```

```
  comm : hom ? A.str = B.str ? T.map hom

end DomainTheory.NeighbourhoodSystems
end
```

3.7.6 Definition 6.5 — initial algebra

DEFINITION 6.5. A T-algebra is *initial* if and only if there is a unique homomorphism from it into any other T-algebra. □

The word "other" here is not meant to imply "distinct". For an initial algebra there is one and only one homomorphism into itself: the identity map. As we already indicated in 6.1 it is a general fact that the next proposition holds.

Definition_6_5.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.Definition_6_4
public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.Exercise_5_10

@[expose] public section

section
universe u
namespace DomainTheory.NeighbourhoodSystems
variable {Obj : Type u} [Category Obj]
variable {T : Endofunctor Obj}
/- concordance: source L3705 DEFINITION 6.5 initial algebra -/
/-- **Definition 6.5 (Scott 1981, PRG-19).** A `T`-algebra `A` is *initial* iff there
    is a (unique)
homomorphism from `A` into every `T`-algebra. We package the homomorphism as the data `
    desc` and its
uniqueness as `uniq`. -/
structure IsInitial (A : TAlgebra T) where
  /-- The chosen homomorphism into any algebra `B`. -/
  desc : (B : TAlgebra T)  ->  AlgHom A B
  /-- It is the only homomorphism `A  ->  B`. -/
  uniq : forall  (B : TAlgebra T) (h : AlgHom A B), h = desc B

end DomainTheory.NeighbourhoodSystems
end

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.Exercise510

section
/-- **The category of domains and approximable maps** (Scott's prose preceding
    Definition 6.3):
identities and associative composition come from Theorem 2.5 (`idMap_comp`, `comp_idMap
    `,
`comp_assoc`). -/
instance : Category DomainObj where
  Hom D E := ApproximableMap D.sys E.sys
```

```
  id D := ApproximableMap.idMap D.sys
  comp f g := g.comp f
  id_comp f := ApproximableMap.comp_idMap f
  comp_id f := ApproximableMap.idMap_comp f
  assoc f g h := (ApproximableMap.comp_assoc h g f).symm
end

section
universe u
variable {Obj : Type u} [Category Obj]
variable {T : Endofunctor Obj}

namespace AlgHom

/-- Composition of `T`-algebra homomorphisms (the `T`-algebras and homomorphisms form a
     category --
the remark after Definition 6.4). -/
def comp {A B C : TAlgebra T} (b : AlgHom B C) (a : AlgHom A B) : AlgHom A C where
  hom := b.hom ? a.hom
  comm := by
    have ha := a.comm
    have hb := b.comm
    have hmap := T.map_comp b.hom a.hom
    dsimp only [compAfter] at ha hb hmap  |-
    rw [ <-  CategoryTheory.Category.assoc, ha, CategoryTheory.Category.assoc, hb,
       <-  CategoryTheory.Category.assoc, hmap]

/-- The identity is a homomorphism: `I  comp  k = k  comp  T(I)`. -/
def id (A : TAlgebra T) : AlgHom A A where
  hom := (? A.carrier)
  comm := by
    simp only [compAfter]
    rw [CategoryTheory.Category.comp_id, T.map_id, CategoryTheory.Category.id_comp]

@[simp] theorem id_hom (A : TAlgebra T) : (AlgHom.id A).hom = (? A.carrier) := rfl

@[simp] theorem comp_hom {A B C : TAlgebra T} (b : AlgHom B C) (a : AlgHom A B) :
    (b.comp a).hom = b.hom ? a.hom := rfl

end AlgHom
end

section
universe u

/-- The **identity endofunctor** on any category: it fixes objects and morphisms. (A
    convenient
witness that `Endofunctor` is inhabited; used to show Definition 6.8 is non-vacuous.)
    -/
def idEndofunctor (Obj : Type u) [Category Obj] : Endofunctor Obj where
  obj X := X
  map f := f
  map_id _ := rfl
  map_comp _ _ := rfl

@[simp] theorem idEndofunctor_obj {Obj : Type u} [Category Obj] (X : Obj) :
```

```
    (idEndofunctor Obj).obj X = X := rfl

@[simp] theorem idEndofunctor_map {Obj : Type u} [Category Obj] {X Y : Obj}
    (f : (X  -->  Y)) : (idEndofunctor Obj).map f = f := rfl

end

end DomainTheory.NeighbourhoodSystems
end
```

3.7.7 Proposition 6.6 — uniqueness of initial algebras

PROPOSITION 6.6. Any two initial T-algebras are uniquely isomorphic. □

Slightly more interesting is the behaviour of T on initial algebras.

Proposition_6_6.lean

```
module

public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.Exercise_5_10
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

public import DomainTheory.NeighbourhoodSystems.DomainEquations.Definition_6_8
public import DomainTheory.NeighbourhoodSystems.DomainEquations.Definition_6_5
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_8
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_7

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.Exercise510
section
universe u
variable {Obj : Type u} [Category Obj] {T : Endofunctor Obj} {A B : TAlgebra T}

/- concordance: source L3709 PROPOSITION 6.6 uniqueness of initial algebras -/
/-- **Proposition 6.6 (Scott 1981, PRG-19).** Any two initial `T`-algebras have
    isomorphic carriers;
the isomorphism is built from the unique homomorphisms in both directions. -/
def initialIso (hA : IsInitial A) (hB : IsInitial B) : Iso A.carrier B.carrier where
  hom := (hA.desc B).hom
  inv := (hB.desc A).hom
  hom_inv_id := by
    have h := comp_desc_eq_id hA hB
    have := congrArg AlgHom.hom h
    simpa using this
  inv_hom_id := by
    have h := comp_desc_eq_id hB hA
    have := congrArg AlgHom.hom h
    simpa using this
```

```
/-- The isomorphism of Proposition 6.6 is **unique**: the homomorphism `A  ->  B`
    realising it is the
only homomorphism between the two initial algebras. -/
theorem iso_hom_unique (hA : IsInitial A) (h : AlgHom A B) : h = hA.desc B :=
  hA.uniq B h

end

section
set_option linter.unusedVariables false

theorem theorem_66.{u, u_1} : forall  {Obj : Type u} [inst : CategoryTheory.Category.{
    u_1, u} Obj]
  {T : @DomainTheory.NeighbourhoodSystems.Endofunctor Obj inst}
  {A B : @DomainTheory.NeighbourhoodSystems.TAlgebra Obj inst T}
  (hA : @DomainTheory.NeighbourhoodSystems.IsInitial Obj inst T A)
  (h : @DomainTheory.NeighbourhoodSystems.AlgHom Obj inst T A B),
  @Eq (@DomainTheory.NeighbourhoodSystems.AlgHom Obj inst T A B) h
    (@DomainTheory.NeighbourhoodSystems.IsInitial.desc Obj inst T A hA B) :=
  @DomainTheory.NeighbourhoodSystems.iso_hom_unique
end

section
variable {a b ? : Type*}
  {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b} {V_2 : NeighborhoodSystem
    ?}

/-- Composition is monotone in both arguments (general arities). -/
theorem comp_mono_gen {a a' : ApproximableMap V_1 V_2} {b b' : ApproximableMap V_0 V_1
    }
    (ha : a  <=  a') (hb : b  <=  b') : a.comp b  <=  a'.comp b' := by
  intro X Z h
  obtain <Y, hXY, hYZ> := h
  exact <Y, hb _ _ hXY, ha _ _ hYZ>

/-- The composite of strict maps is strict: `(a  comp  b)(False) = a(b(False)) = a(
    False) = False`. -/
theorem isStrict_comp {a : ApproximableMap V_1 V_2} {b : ApproximableMap V_0 V_1}
    (ha : IsStrict a) (hb : IsStrict b) : IsStrict (a.comp b) := by
  rw [isStrict_iff_apply_bot, toElementMap_comp, isStrict_iff_apply_bot.mp hb,
    isStrict_iff_apply_bot.mp ha]

section HomOp

variable (T : Endofunctor DomainObj) (D E : DomainObj)
  (j : ApproximableMap D.sys (T.obj D).sys)
  (k : ApproximableMap (T.obj E).sys E.sys)
  (hj : IsStrict j) (hk : IsStrict k)

/-- The strict composite `g  |->  k  comp  g  comp  j : (T(D)  -> False T(E))  ->  (D
    -> False E)`. -/
def homOpComp (g : StrictMap (T.obj D).sys (T.obj E).sys) : StrictMap D.sys E.sys :=
  <k.comp (g.1.comp j), isStrict_comp hk (isStrict_comp g.2 hj)>

theorem homOpComp_mono {g g' : StrictMap (T.obj D).sys (T.obj E).sys} (hgg : g  <=  g
    ') :
```

```
    homOpComp T D E j k hj hk g  <=  homOpComp T D E j k hj hk g' := by
  show k.comp (g.1.comp j)  <=  k.comp (g'.1.comp j)
  exact comp_mono_gen le_rfl (comp_mono_gen (Subtype.coe_le_coe.mpr hgg) le_rfl)

end HomOp

/-- `toStrictFilter` is monotone. -/
theorem toStrictFilter_mono {f f' : StrictMap V_0 V_1} (h : f  <=  f') :
    toStrictFilter f  <=  toStrictFilter f' := by
  intro W hW
  rw [mem_toStrictFilter] at hW  |-
  exact <hW.1, strictFun_mem_up_closed hW.1 hW.2 h>

/-- `toStrictMap` is monotone. -/
theorem toStrictMap_mono {phi phi' : (strictFun V_0 V_1).Element} (h : phi  <=  phi')
    :
    toStrictMap phi  <=  toStrictMap phi' := by
  intro X Y hrel
  rw [toStrictMap_rel] at hrel  |-
  exact h _ hrel

section HomOp

variable (T : Endofunctor DomainObj) (D E : DomainObj)
  (j : ApproximableMap D.sys (T.obj D).sys)
  (k : ApproximableMap (T.obj E).sys E.sys)
  (hj : IsStrict j) (hk : IsStrict k)

/-- **The operator `fun g. k  comp  g  comp  j`** as an approximable map between the
    strict function spaces, built
by Exercise 2.8 (`ofMono`) from its values on finite elements. -/
def homOp : ApproximableMap (strictFun (T.obj D).sys (T.obj E).sys) (strictFun D.sys E
    .sys) :=
  ofMono
    (fun N hN => toStrictFilter (homOpComp T D E j k hj hk
      (toStrictMap ((strictFun (T.obj D).sys (T.obj E).sys).principal hN))))
    (by
      intro N N' hN hN' hN'N
      apply toStrictFilter_mono
      apply homOpComp_mono
      apply toStrictMap_mono
      exact ((strictFun (T.obj D).sys (T.obj E).sys).principal_le_iff hN hN').mpr hN'N
    )

end HomOp

section HomOp

variable (T : Endofunctor DomainObj) (D E : DomainObj)
  (j : ApproximableMap D.sys (T.obj D).sys)

theorem homOp_rel {N : Set (StrictMap (T.obj D).sys (T.obj E).sys)}
    {M : Set (StrictMap D.sys E.sys)} :
    (homOp T D E j k hj hk).rel N M  <->
      exists  hN : (strictFun (T.obj D).sys (T.obj E).sys).mem N,
        (toStrictFilter (homOpComp T D E j k hj hk
```

```
          (toStrictMap ((strictFun (T.obj D).sys (T.obj E).sys).principal hN)))).mem M
     :=
  Iff.rfl

end HomOp

/-- If `a  comp  b = I` then `a` is strict: any (split) iso preserves `False`. -/
theorem isStrict_of_comp_eq_id {a : ApproximableMap V_1 V_0} {b : ApproximableMap V_0
    V_1}
    (h : a.comp b = idMap V_0) : IsStrict a := by
  rw [isStrict_iff_apply_bot]
  refine le_antisymm ?_ (V_0.bot_le _)
  calc a.toElementMap V_1.bot
      <=  a.toElementMap (b.toElementMap V_0.bot) := toElementMap_mono a (V_1.bot_le
    _)
    _ = (a.comp b).toElementMap V_0.bot := (toElementMap_comp a b V_0.bot).symm
    _ = V_0.bot := by rw [h, toElementMap_idMap]

/-- `toStrictFilter  comp  toStrictMap = id` (the left inverse of the strict-function-
    space
representation; the mirror of `toStrictMap_toStrictFilter`). -/
theorem toStrictFilter_toStrictMap (phi : (strictFun V_0 V_1).Element) :
    toStrictFilter (toStrictMap phi) = phi := by
  apply Element.ext
  intro W
  constructor
  * rintro <hWmem, hfW>
    obtain <<L, hL, rfl>, _> := hWmem
    exact (mem_sstepFun_iff phi hL).mpr (fun p hp => hfW p hp)
  * intro hW
    refine <phi.sub hW, ?_>
    obtain <<L, hL, rfl>, _> := phi.sub hW
    intro p hp
    exact (mem_sstepFun_iff phi hL).mp hW p hp

section HomOp

variable (T : Endofunctor DomainObj) (D E : DomainObj)
  (j : ApproximableMap D.sys (T.obj D).sys)

/-- **Action lemma.** `homOp` realizes the composite `g  |->  k  comp  g  comp  j` on
    filters of strict maps:
`homOp(g) = (k  comp  g  comp  j)^`. Proved by reducing to single step neighbourhoods
    through the strict
representation. -/
theorem homOp_apply_filter (g : StrictMap (T.obj D).sys (T.obj E).sys) :
    (homOp T D E j k hj hk).toElementMap (toStrictFilter g)
      = toStrictFilter (homOpComp T D E j k hj hk g) := by
  have key : forall  X Z,
      ((homOp T D E j k hj hk).toElementMap (toStrictFilter g)).mem (sstep X Z)
        <->  (homOpComp T D E j k hj hk g).1.rel X Z := by
    intro X Z
    constructor
    * rintro <N, hgN, hrel>
      rw [homOp_rel] at hrel
      obtain <hN, hmem> := hrel
```

```
      rw [mem_toStrictFilter] at hmem
      obtain <_, hsstep> := hmem
      rw [mem_sstep] at hsstep
      obtain <Y2, <Y1, hXY1, hY1Y2>, hY2Z> := hsstep
      rw [toStrictMap_rel, mem_principal] at hY1Y2
      obtain <_, hNsub> := hY1Y2
      exact <Y2, <Y1, hXY1, hNsub (mem_toStrictFilter.mp hgN).2>, hY2Z>
    * intro hrel
      obtain <Y2, <Y1, hXY1, hgY1Y2>, hY2Z> := hrel
      refine <sstep Y1 Y2, ?_, ?_>
      * rw [mem_toStrictFilter]
        exact <sstep_mem_of_mem (g := g) hgY1Y2, hgY1Y2>
      * rw [homOp_rel]
        refine <sstep_mem_of_mem (g := g) hgY1Y2, ?_>
        rw [mem_toStrictFilter]
        refine <sstep_mem_of_mem (g := homOpComp T D E j k hj hk g)
          <Y2, <Y1, hXY1, hgY1Y2>, hY2Z>, ?_>
        rw [mem_sstep]
        refine <Y2, <Y1, hXY1, ?_>, hY2Z>
        rw [toStrictMap_rel, mem_principal]
        exact <sstep_mem_of_mem (g := g) hgY1Y2, subset_rfl>
  have hmap : toStrictMap ((homOp T D E j k hj hk).toElementMap (toStrictFilter g))
      = homOpComp T D E j k hj hk g := by
    apply Subtype.ext
    apply ApproximableMap.ext
    intro X Z
    rw [toStrictMap_rel]
    exact key X Z
  have hL := toStrictFilter_toStrictMap
    ((homOp T D E j k hj hk).toElementMap (toStrictFilter g))
  rw [hmap] at hL
  exact hL.symm

end HomOp

end
end DomainTheory.NeighbourhoodSystems
end
```

3.7.8 Proposition 6.7 — functor preserves initiality

PROPOSITION 6.7. If $i : T(D) \to D$ is an initial T-algebra, then so is $T(i) : T^2(D) \to T(D)$ and i is the isomorphism from $T(D)$ to D.

Proof: Clearly since T is a functor, the map $T(i)$ has the right mapping character to make $T(D)$ a T-algebra. Since D is initial, we have a commuting diagram:

$$\begin{array}{ccc} T(D) & \stackrel{i}{\to} & D \\ \downarrow^{T(j)} & & \downarrow^{j} \\ T^2(D) & \xrightarrow{T(i)} & T(D) \end{array}$$

But we also have the trivial diagram:

$$\begin{array}{ccc} T^2(D) & \xrightarrow{T(i)} & T(D) \\ \downarrow{\scriptstyle T(i)} & & \downarrow{\scriptstyle i} \\ T(D) & \xrightarrow{i} & D \end{array}$$

It follows that $i \circ j$ is a homomorphism, so

$$i \circ j = I_D.$$

But then because T is a functor we find:

$$T(i) \circ T(j) = I_{T(D)},$$

and, since j is a homomorphism, we have

$$j \circ i = I_{T(D)}.$$

This shows that i is an isomorphism. □

From 6.7 we see that if we are going to have initial algebras at all we have to satisfy the domain equation

$$D \cong T(D).$$

But generally that is not enough to assure that D is initial. There is a condition that our functors satisfy, however, which guarantees the existence of homomorphisms.

Proposition_6_7.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.Theorem_6_14

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
universe u
variable {Obj : Type u} [Category Obj] {T : Endofunctor Obj}
/- concordance: source L3713 PROPOSITION 6.7 functor preserves initiality -/
/-- **Proposition 6.7 (Lambek's lemma; Scott 1981, PRG-19).** The structure map `i : T(
    D)  ->  D` of an
initial `T`-algebra is an isomorphism `T(D)  ~=  D`, with inverse the descent
    homomorphism `j`. -/
def lambek (A : TAlgebra T) (hA : IsInitial A) : Iso (T.obj A.carrier) A.carrier where
  hom := A.str
  inv := (hA.desc (tStr A)).hom
  inv_hom_id := str_comp_desc A hA
  hom_inv_id :=
    calc (hA.desc (tStr A)).hom ? A.str
        = (tStr A).str ? T.map (hA.desc (tStr A)).hom := (hA.desc (tStr A)).comm
      _ = T.map A.str ? T.map (hA.desc (tStr A)).hom := rfl
      _ = T.map (A.str ? (hA.desc (tStr A)).hom) := (T.map_comp _ _).symm
      _ = T.map ((? A.carrier)) := by rw [str_comp_desc A hA]
      _ = (? (T.obj A.carrier)) := T.map_id _
```

```
end

end DomainTheory.NeighbourhoodSystems
end
```

3.7.9 Definition 6.8 — continuity on maps

DEFINITION 6.8. On the category of domains and strict approximable maps a functor T is *continuous on maps* if for any systems D and E the induced mapping

$$\lambda f.\, T(f) : (D \to_\perp E) \to (T(D) \to_\perp T(E))$$

is approximable.

Definition_6_8.lean

```
module

public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.Exercise_5_10
public import DomainTheory.NeighbourhoodSystems.DomainEquations.Definition_6_5

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.Exercise510
section
/- concordance: source L3765 DEFINITION 6.8 continuity on maps -/
/-- **Definition 6.8 (Scott 1981, PRG-19).** An endofunctor `T` on the category of
    domains is
*continuous on maps* when, for every pair of domains `D` and `E`, the induced action `
    fun f. T(f)` on the
**strict** function space `(D  -> False E)` is approximable: there is an approximable
    map `?` from
`(D  -> False E)` to `(T(D)  -> False T(E))` whose elementwise action (read through the
     representation
`strictFunEquiv`) sends each strict map `f` to `T(f)`. -/
def ContinuousOnMaps (T : Endofunctor DomainObj) : Prop :=
  forall  D E : DomainObj,
    exists  ? : ApproximableMap (strictFun D.sys E.sys) (strictFun (T.obj D).sys (T.
    obj E).sys),
      forall  f : StrictMap D.sys E.sys,
        (toStrictMap (?.toElementMap (toStrictFilter f))).1 = T.map (X := D) (Y := E)
    f.1
end

section
/-- A functor continuous on maps **preserves strictness** (so it genuinely lives on
    Scott's category
of domains and strict maps): if `f` is strict then so is `T(f)`. This is automatic from
     the witnessing
equation, whose left-hand side is the underlying map of a `StrictMap`. -/
theorem ContinuousOnMaps.isStrict_map {T : Endofunctor DomainObj} (h :
    ContinuousOnMaps T)
```

```
    {D E : DomainObj} (f : StrictMap D.sys E.sys) :
    IsStrict (T.map (X := D) (Y := E) f.1) := by
  obtain <?, h?> := h D E
  rw [ <-  h? f]
  exact (toStrictMap (?.toElementMap (toStrictFilter f))).2
end

section
/-- `toStrictMap  comp  toStrictFilter = id` (the right inverse of the strict-function-
    space
representation, Exercise 5.10). -/
theorem toStrictMap_toStrictFilter {a b : Type*} {V_0 : NeighborhoodSystem a}
    {V_1 : NeighborhoodSystem b} (f : StrictMap V_0 V_1) :
    toStrictMap (toStrictFilter f) = f :=
  (strictFunEquiv V_0 V_1).right_inv f
end

section
/-- **The identity functor is continuous on maps** -- the basic witness that
    Definition 6.8 is
satisfiable. The representing approximable map is the identity on `(D  -> False E)`. -/
theorem continuousOnMaps_id : ContinuousOnMaps (idEndofunctor DomainObj) := by
  intro D E
  refine <idMap (strictFun D.sys E.sys), fun f => ?_>
  show (toStrictMap ((idMap (strictFun D.sys E.sys)).toElementMap (toStrictFilter f)))
    .1 = f.1
  rw [toElementMap_idMap, toStrictMap_toStrictFilter]
end

section
universe u
variable {Obj : Type u} [Category Obj] {T : Endofunctor Obj} {A B : TAlgebra T}

/-- Composing the two unique homomorphisms `A  ->  B  ->  A` gives the identity
    homomorphism on the initial
algebra `A`. -/
theorem comp_desc_eq_id (hA : IsInitial A) (hB : IsInitial B) :
    (hB.desc A).comp (hA.desc B) = AlgHom.id A := by
  rw [hA.uniq A ((hB.desc A).comp (hA.desc B)), hA.uniq A (AlgHom.id A)]

end
end DomainTheory.NeighbourhoodSystems
end
```

3.7.10 Theorem 6.9 — homomorphism from initial algebra

THEOREM 6.9. If the functor T is continuous on maps and if $D \cong T(D)$, so in particular D is a T-algebra, then for any T-algebra $k : T(E) \to E$ there is a homomorphism $h : D \to E$.

Proof: Let $i : T(D) \to D$ make D a T-algebra, where $j : D \to T(D)$ is the inverse so that i is an isomorphism of domains. Suppose that $k : T(E) \to E$ is any T-algebra. A homomorphism $h : D \to E$ would satisfy

$$h \circ i = k \circ T(h).$$

Rewrite this equation as

$$h = k \circ T(h) \circ j.$$

In the domain of strict maps $(D \to E)$ this is a fixed-point equation for an approximable map

$$\lambda h.\, k \circ T(h) \circ j$$

by our assumption on T. Thus, the desired homomorphism exists. □

The final question we have to answer is why in our category the minimal D exist. The reason is that the functors T that we have in mind possess further continuity properties on domains. This is conveniently expressed in terms of a notion of "subdomain".

Theorem_6_9.lean

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Theorem_4_2
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

public import DomainTheory.NeighbourhoodSystems.DomainEquations.Proposition_6_6
public import DomainTheory.NeighbourhoodSystems.DomainEquations.Definition_6_8

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.Exercise510
section
variable {a b ? : Type*}
  {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b} {V_2 : NeighborhoodSystem
    ?}
/- concordance: source L3775 THEOREM 6.9 homomorphism from initial algebra -/
/-- **Theorem 6.9 (Scott 1981, PRG-19).** If `T` is continuous on maps and `D  ~=  T(D)
    ` (so `D` is a
`T`-algebra via `i : T(D)  ->  D`), then for any `T`-algebra `B = (E, k)` with `k`
    strict there is a
homomorphism `D  ->  E`. The homomorphism is the least fixed point of `fun h. k  comp
    T(h)  comp  j`. -/
theorem nonempty_algHom_of_continuousOnMaps
    (T : Endofunctor DomainObj) (hT : ContinuousOnMaps T)
    {D : DomainObj} (iso : Iso (T.obj D) D)
    (B : TAlgebra T) (hk : IsStrict B.str) :
    Nonempty {g : AlgHom (<D, iso.hom> : TAlgebra T) B // IsStrict g.hom} := by
  -- `j = i^-1` is strict (it is an isomorphism of domains).
  have hji : iso.inv.comp iso.hom = idMap (T.obj D).sys := iso.hom_inv_id
  have hj : IsStrict iso.inv := isStrict_of_comp_eq_id hji
  -- Definition 6.8's witness that `fun f. T(f)` is approximable.
  obtain <?, h?> := hT D B.carrier
  -- the operator `Op = (k  comp  *  comp  j)  comp  T` and its least fixed point.
  set Op := (homOp T D B.carrier iso.inv B.str hj hk).comp ? with hOp
  set x := Op.fixElement with hx
  set h := toStrictMap x with hh
  let Tg : StrictMap (T.obj D).sys (T.obj B.carrier).sys :=
    <T.map (X := D) (Y := B.carrier) h.1, hT.isStrict_map (D := D) (E := B.carrier) h>
```

```
  have hfix : Op.toElementMap x = x := toElementMap_fixElement Op
  have hxh : toStrictFilter h = x := toStrictFilter_toStrictMap x
  -- `?` sends `h` to the filter of `T(h)`.
  have hTg : toStrictMap (?.toElementMap (toStrictFilter h)) = Tg := Subtype.ext (h? h
    )
  have hphi : ?.toElementMap (toStrictFilter h) = toStrictFilter Tg := by
    rw [ <-  hTg]; exact (toStrictFilter_toStrictMap _).symm
  -- evaluate the operator at the fixed point.
  have hOpx : Op.toElementMap x
      = toStrictFilter (homOpComp T D B.carrier iso.inv B.str hj hk Tg) := by
    rw [hOp, toElementMap_comp,  <-  hxh, hphi, homOp_apply_filter]
  have hcompeq : toStrictFilter (homOpComp T D B.carrier iso.inv B.str hj hk Tg)
      = toStrictFilter h := by
    rw [ <-  hOpx, hfix, hxh]
  have hh' : homOpComp T D B.carrier iso.inv B.str hj hk Tg = h := by
    have h2 := congrArg toStrictMap hcompeq
    rwa [toStrictMap_toStrictFilter, toStrictMap_toStrictFilter] at h2
  -- the fixed-point equation `h = k  comp  T(h)  comp  j`.
  have hcore : B.str.comp ((T.map (X := D) (Y := B.carrier) h.1).comp iso.inv) = h.1
    :=
    congrArg Subtype.val hh'
  -- rearrange to the homomorphism square `h  comp  i = k  comp  T(h)`.
  refine <<{ hom := h.1, comm := ?_ }, h.2>>
  show h.1.comp iso.hom = B.str.comp (T.map (X := D) (Y := B.carrier) h.1)
  conv_lhs => rw [ <-  hcore]
  rw [comp_assoc, comp_assoc, hji, comp_idMap]
end

section
set_option linter.unusedVariables false

theorem theorem_69.{u_4} : forall
  (T :
    @DomainTheory.NeighbourhoodSystems.Endofunctor.{u_4 + 1, u_4} DomainTheory.
    NeighbourhoodSystems.DomainObj.{u_4}
      DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{u_4}),
  DomainTheory.NeighbourhoodSystems.ContinuousOnMaps.{u_4} T  ->
    forall  {D : DomainTheory.NeighbourhoodSystems.DomainObj.{u_4}}
      (iso :
        @DomainTheory.NeighbourhoodSystems.Iso.{u_4 + 1, u_4} DomainTheory.
    NeighbourhoodSystems.DomainObj.{u_4}
          DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{u_4}
          (@DomainTheory.NeighbourhoodSystems.Endofunctor.obj.{u_4 + 1, u_4}
            DomainTheory.NeighbourhoodSystems.DomainObj.{u_4}
            DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{u_4} T D)
          D)
      (B :
        @DomainTheory.NeighbourhoodSystems.TAlgebra.{u_4 + 1, u_4} DomainTheory.
    NeighbourhoodSystems.DomainObj.{u_4}
          DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{u_4} T),
      @DomainTheory.NeighbourhoodSystems.Exercise510.IsStrict.{u_4, u_4}
          (@DomainTheory.NeighbourhoodSystems.Endofunctor.obj.{u_4 + 1, u_4}
              DomainTheory.NeighbourhoodSystems.DomainObj.{u_4}
              DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{u_4} T
              (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{u_4 + 1, u_4}
                DomainTheory.NeighbourhoodSystems.DomainObj.{u_4}
```

```
            DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{u_4} T B)).
carrier
      (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{u_4 + 1, u_4}
          DomainTheory.NeighbourhoodSystems.DomainObj.{u_4}
          DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{u_4} T B).
carrier
      (@DomainTheory.NeighbourhoodSystems.Endofunctor.obj.{u_4 + 1, u_4}
          DomainTheory.NeighbourhoodSystems.DomainObj.{u_4}
          DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{u_4} T
          (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{u_4 + 1, u_4}
            DomainTheory.NeighbourhoodSystems.DomainObj.{u_4}
            DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{u_4} T B)).
sys
      (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{u_4 + 1, u_4}
          DomainTheory.NeighbourhoodSystems.DomainObj.{u_4}
          DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{u_4} T B).sys
      (@DomainTheory.NeighbourhoodSystems.TAlgebra.str.{u_4 + 1, u_4}
        DomainTheory.NeighbourhoodSystems.DomainObj.{u_4}
        DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{u_4} T B)  ->
    Nonempty.{u_4 + 1}
      (@Subtype.{u_4 + 1}
        (@DomainTheory.NeighbourhoodSystems.AlgHom.{u_4 + 1, u_4} DomainTheory.
NeighbourhoodSystems.DomainObj.{u_4}
          DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{u_4} T
          (@DomainTheory.NeighbourhoodSystems.TAlgebra.mk.{u_4 + 1, u_4}
            DomainTheory.NeighbourhoodSystems.DomainObj.{u_4}
            DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{u_4} T D
            (@DomainTheory.NeighbourhoodSystems.Iso.hom.{u_4 + 1, u_4}
              DomainTheory.NeighbourhoodSystems.DomainObj.{u_4}
              DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{u_4}
              (@DomainTheory.NeighbourhoodSystems.Endofunctor.obj.{u_4 + 1, u_4}
                DomainTheory.NeighbourhoodSystems.DomainObj.{u_4}
                DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{u_4} T D)
              D iso))
          B)
        fun
          (g :
            @DomainTheory.NeighbourhoodSystems.AlgHom.{u_4 + 1, u_4}
              DomainTheory.NeighbourhoodSystems.DomainObj.{u_4}
              DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{u_4} T
              (@DomainTheory.NeighbourhoodSystems.TAlgebra.mk.{u_4 + 1, u_4}
                DomainTheory.NeighbourhoodSystems.DomainObj.{u_4}
                DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{u_4} T D
                (@DomainTheory.NeighbourhoodSystems.Iso.hom.{u_4 + 1, u_4}
                  DomainTheory.NeighbourhoodSystems.DomainObj.{u_4}
                  DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{u_4}
                  (@DomainTheory.NeighbourhoodSystems.Endofunctor.obj.{u_4 + 1,
u_4}
                    DomainTheory.NeighbourhoodSystems.DomainObj.{u_4}
                    DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{u_4}
T D)
                  D iso))
              B) =>
        @DomainTheory.NeighbourhoodSystems.Exercise510.IsStrict.{u_4, u_4}
          (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{u_4 + 1, u_4}
            DomainTheory.NeighbourhoodSystems.DomainObj.{u_4}
```

```
                DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{u_4} T
                (@DomainTheory.NeighbourhoodSystems.TAlgebra.mk.{u_4 + 1, u_4}
                  DomainTheory.NeighbourhoodSystems.DomainObj.{u_4}
                  DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{u_4} T D
                  (@DomainTheory.NeighbourhoodSystems.Iso.hom.{u_4 + 1, u_4}
                    DomainTheory.NeighbourhoodSystems.DomainObj.{u_4}
                    DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{u_4}
                    (@DomainTheory.NeighbourhoodSystems.Endofunctor.obj.{u_4 + 1,
    u_4}
                      DomainTheory.NeighbourhoodSystems.DomainObj.{u_4}
                      DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{u_4}
    T D)
                    D iso))).carrier
              (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{u_4 + 1, u_4}
                DomainTheory.NeighbourhoodSystems.DomainObj.{u_4}
                DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{u_4} T B).
    carrier
              (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{u_4 + 1, u_4}
                DomainTheory.NeighbourhoodSystems.DomainObj.{u_4}
                DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{u_4} T
                (@DomainTheory.NeighbourhoodSystems.TAlgebra.mk.{u_4 + 1, u_4}
                  DomainTheory.NeighbourhoodSystems.DomainObj.{u_4}
                  DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{u_4} T D
                  (@DomainTheory.NeighbourhoodSystems.Iso.hom.{u_4 + 1, u_4}
                    DomainTheory.NeighbourhoodSystems.DomainObj.{u_4}
                    DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{u_4}
                    (@DomainTheory.NeighbourhoodSystems.Endofunctor.obj.{u_4 + 1,
    u_4}
                      DomainTheory.NeighbourhoodSystems.DomainObj.{u_4}
                      DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{u_4}
    T D)
                    D iso))).sys
              (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{u_4 + 1, u_4}
                DomainTheory.NeighbourhoodSystems.DomainObj.{u_4}
                DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{u_4} T B).
    sys
              (@DomainTheory.NeighbourhoodSystems.AlgHom.hom.{u_4 + 1, u_4}
                DomainTheory.NeighbourhoodSystems.DomainObj.{u_4}
                DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{u_4} T
                (@DomainTheory.NeighbourhoodSystems.TAlgebra.mk.{u_4 + 1, u_4}
                  DomainTheory.NeighbourhoodSystems.DomainObj.{u_4}
                  DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{u_4} T D
                  (@DomainTheory.NeighbourhoodSystems.Iso.hom.{u_4 + 1, u_4}
                    DomainTheory.NeighbourhoodSystems.DomainObj.{u_4}
                    DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{u_4}
                    (@DomainTheory.NeighbourhoodSystems.Endofunctor.obj.{u_4 + 1, u_4}
                      DomainTheory.NeighbourhoodSystems.DomainObj.{u_4}
                      DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{u_4} T
    D)
                    D iso))
                B g)) :=
  DomainTheory.NeighbourhoodSystems.nonempty_algHom_of_continuousOnMaps.{u_4}
end

end DomainTheory.NeighbourhoodSystems
end
```

```
section
-- StrictDomainObj.lean
universe w

namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.Exercise510

/-- An object of Scott's category (Exercise 6.19): a token type, a neighbourhood
    system on it, and the
standing assumption `{}  notin  ?` (every neighbourhood is non-empty). -/
structure StrictDomainObj : Type (w + 1) where
  /-- The token type. -/
  carrier : Type w
  /-- The neighbourhood system. -/
  sys : NeighborhoodSystem carrier
  /-- Scott's standing assumption `{}  notin  ?`. -/
  nonempty : forall  X, sys.mem X  ->  X.Nonempty

end DomainTheory.NeighbourhoodSystems
end

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.Exercise510

/-- **The category of `{}`-free domains and strict maps.** Morphisms are strict
    approximable maps
(`StrictMap`, Exercise 5.10); identities and associative composition come from Theorem
    2.5, and
strictness is preserved by `isStrict_idMap` / `isStrict_comp`. -/
instance : Category StrictDomainObj where
  Hom D E := StrictMap D.sys E.sys
  id D := <ApproximableMap.idMap D.sys, isStrict_idMap>
  comp f g := <g.1.comp f.1, isStrict_comp g.2 f.2>
  id_comp f := Subtype.ext (ApproximableMap.comp_idMap f.1)
  comp_id f := Subtype.ext (ApproximableMap.idMap_comp f.1)
  assoc f g h := Subtype.ext (ApproximableMap.comp_assoc h.1 g.1 f.1).symm

@[simp] theorem StrictDomainObj.id_val (D : StrictDomainObj) :
    ((? D) : StrictMap D.sys D.sys).1 = ApproximableMap.idMap D.sys := rfl

@[simp] theorem StrictDomainObj.comp_val {D E F : StrictDomainObj}
    (g : (E  -->  F)) (f : (D  -->  E)) :
    ((g ? f : StrictMap D.sys F.sys)).1 = g.1.comp f.1 := rfl

end

section
open NeighborhoodSystem

namespace Example61

variable {a : Type*}
```

```
@[simp] theorem mem_embZero {X : Set a} {p : List Bool} {a : a} :
    (p, a)  in  embZero X  <->  p = []  /\  a  in  X := Iff.rfl

@[simp] theorem mem_embL {P : Set (List Bool  x  a)} {p : List Bool} {a : a} :
    (p, a)  in  embL P  <->  exists  p', p = true :: p'  /\  (p', a)  in  P := Iff.rfl

@[simp] theorem mem_embR {Q : Set (List Bool  x  a)} {p : List Bool} {a : a} :
    (p, a)  in  embR Q  <->  exists  q', p = false :: q'  /\  (q', a)  in  Q := Iff.
    rfl

@[simp] theorem mem_embPair {P Q : Set (List Bool  x  a)} {p : List Bool} {a : a} :
    (p, a)  in  embPair P Q  <->
      (exists  p', p = true :: p'  /\  (p', a)  in  P)  \/  (exists  q', p = false ::
    q'  /\  (q', a)  in  Q) := Iff.rfl

@[simp] theorem mem_Gamma {D : NeighborhoodSystem a} {p : List Bool} {a : a} :
    (p, a)  in  Gamma D  <->  a  in  D.master := Iff.rfl

theorem Gamma_nonempty {D : NeighborhoodSystem a} (hD : forall  X, D.mem X  ->  X.
    Nonempty) :
    (Gamma D).Nonempty := by
  obtain <a, ha> := hD D.master D.master_mem; exact <([], a), ha>

theorem embPair_inter (P Q P' Q' : Set (List Bool  x  a)) :
    embPair P Q  I  embPair P' Q' = embPair (P  I  P') (Q  I  Q') := by
  ext <p, a>
  simp only [Set.mem_inter_iff, mem_embPair]
  rcases p with _ | <b, ps>
  * simp
  * cases b <;> simp [List.cons.injEq]

theorem embPair_nonempty {P Q : Set (List Bool  x  a)} (hP : P.Nonempty) :
    (embPair P Q).Nonempty := by
  obtain <<p', a>, hP> := hP; exact <(true :: p', a), Or.inl <p', rfl, hP>>

theorem embPair_subset {P Q P' Q' : Set (List Bool  x  a)} :
    embPair P Q  subseteq  embPair P' Q'  <->  P  subseteq  P'  /\  Q  subseteq  Q' :=
     by
  constructor
  * intro h
    refine <fun t ht => ?_, fun t ht => ?_>
    * obtain <p', a> := t
      have hmem : ((true :: p', a) : List Bool  x  a)  in  embPair P' Q' :=
        h (Or.inl <p', rfl, ht>)
      rcases hmem with <p'', hp'', hP'> | <q'', hq'', ->
      * simp only [List.cons.injEq, true_and] at hp''; exact hp''  |>  hP'
      * exact absurd hq'' (by simp)
    * obtain <q', a> := t
      have hmem : ((false :: q', a) : List Bool  x  a)  in  embPair P' Q' :=
        h (Or.inr <q', rfl, ht>)
      rcases hmem with <p'', hp'', -> | <q'', hq'', hQ'>
      * exact absurd hp'' (by simp)
      * simp only [List.cons.injEq, true_and] at hq''; exact hq''  |>  hQ'
  * rintro <hP, hQ> t ht
    rcases ht with <p', hp', hPmem> | <q', hq', hQmem>
```

```
    * exact Or.inl <p', hp', hP hPmem>
    * exact Or.inr <q', hq', hQ hQmem>

theorem embPair_injective {P Q P' Q' : Set (List Bool  x  a)}
    (h : embPair P Q = embPair P' Q') : P = P'  /\  Q = Q' := by
  obtain <hP, hQ> := embPair_subset.mp h.subset
  obtain <hP', hQ'> := embPair_subset.mp h.symm.subset
  exact <Set.Subset.antisymm hP hP', Set.Subset.antisymm hQ hQ'>

theorem embPair_subset_Gamma {D : NeighborhoodSystem a} {P Q : Set (List Bool  x  a)}
    (hP : P  subseteq  Gamma D) (hQ : Q  subseteq  Gamma D) : embPair P Q  subseteq
    Gamma D := by
  rintro <p, a> (<p', -, hPmem> | <q', -, hQmem>)
  * have h := hP hPmem; exact h
  * have h := hQ hQmem; exact h

theorem embZero_inter (X X' : Set a) :
    embZero X  I  embZero X' = embZero (X  I  X' : Set a) := by
  ext <p, a>
  simp only [Set.mem_inter_iff, mem_embZero]
  tauto

theorem embZero_inter_embPair (X : Set a) (P Q : Set (List Bool  x  a)) :
    embZero X  I  embPair P Q = {} := by
  ext <p, a>
  simp only [Set.mem_inter_iff, mem_embZero, mem_embPair, Set.mem_empty_iff_false,
    iff_false,
    not_and, not_or]
  rintro <rfl, ->
  exact <by rintro <p', hp', ->; exact absurd hp' (by simp), by rintro <q', hq', ->;
    exact absurd hq' (by simp)>

theorem embZero_nonempty {X : Set a} (hX : X.Nonempty) : (embZero X).Nonempty := by
  obtain <a, ha> := hX; exact <([], a), rfl, ha>

theorem embZero_subset {X X' : Set a} : embZero X  subseteq  embZero X'  <->  X
    subseteq  X' := by
  constructor
  * intro h a ha
    have : (([], a) : List Bool  x  a)  in  embZero X' := h <rfl, ha>
    exact this.2
  * intro h t ht; exact <ht.1, h ht.2>

theorem embZero_injective {X X' : Set a} (h : embZero X = embZero X') : X = X' :=
  Set.Subset.antisymm (embZero_subset.mp h.subset) (embZero_subset.mp h.symm.subset)

theorem embZero_subset_Gamma {D : NeighborhoodSystem a} {X : Set a} (hX : D.mem X) :
    embZero X  subseteq  Gamma D := by
  rintro <p, a> <-, ha>; exact D.sub_master hX ha

/-- Under Scott's standing assumption `{}  notin  ?`, no neighbourhood of `D^S` is
    empty. -/
theorem memS_nonempty {D : NeighborhoodSystem a} (hD : forall  X, D.mem X  ->  X.
    Nonempty)
    {W : Set (List Bool  x  a)} (hW : MemS D W) : W.Nonempty := by
  induction hW with
```

```
  | gamma => exact Gamma_nonempty hD
  | zero hX => exact embZero_nonempty (hD _ hX)
  | pair _ _ ihP _ => exact embPair_nonempty ihP

/-- Every neighbourhood of `D^S` is a subset of the master `Gamma` (Scott's `?^S
    subseteq  ?(Gamma)`). -/
theorem memS_subset_gamma {D : NeighborhoodSystem a} {W : Set (List Bool  x  a)}
    (hW : MemS D W) : W  subseteq  Gamma D := by
  induction hW with
  | gamma => exact subset_rfl
  | zero hX => exact embZero_subset_Gamma hX
  | pair _ _ ihP ihQ => exact embPair_subset_Gamma ihP ihQ

/-- **Example 6.1 -- closure under consistent intersection.** Scott's central
    verification, "by
induction on the number of steps required to put `X` and `Y` into `?^S`". The cross
    cases
(`0A  I  (1P U 2Q) = {}`, etc.) are discharged because the consistency witness `Z` is
    non-empty
(`memS_nonempty`); the `0A  I  0B` case uses `?`'s own closure; the `(1P U 2Q)  I  (1P'
    U 2Q')` case recurses
on `P, P'` and `Q, Q'`. -/
theorem memS_inter {D : NeighborhoodSystem a} (hD : forall  X, D.mem X  ->  X.Nonempty
    ) :
    forall  {X : Set (List Bool  x  a)}, MemS D X  ->  forall  {Y : Set (List Bool  x
    a)}, MemS D Y  ->
      forall  {Z : Set (List Bool  x  a)}, MemS D Z  ->  Z  subseteq  X  I  Y  ->
    MemS D (X  I  Y) := by
  intro X hX
  induction hX with
  | gamma =>
    intro Y hY Z _ _
    rw [Set.inter_eq_right.mpr (memS_subset_gamma hY)]; exact hY
  | @zero A hA =>
    intro Y hY Z hZ hsub
    cases hY with
    | gamma =>
      rw [Set.inter_eq_left.mpr (embZero_subset_Gamma hA)]; exact MemS.zero hA
    | @zero B hB =>
      rw [embZero_inter]
      -- the witness `Z` must be a `0C`; read off `C  subseteq  A  I  B`, then `D`'s
    closure gives `A  I  B  in  ?`.
      rw [embZero_inter] at hsub
      cases hZ with
      | gamma =>
        obtain <a, ha> := hD D.master D.master_mem
        exact absurd (hsub (show ((true :: [], a) : List Bool  x  a)  in  Gamma D from
    ha)) (by simp)
      | @zero C hC =>
        exact MemS.zero (D.inter_mem hA hB hC (embZero_subset.mp hsub))
      | @pair P Q hP hQ =>
        obtain <t, hmem> := embPair_nonempty (memS_nonempty hD hP) (Q := Q)
        have hempty : t.1 = [] := (hsub hmem).1
        rcases hmem with <p', hp', -> | <q', hq', ->
        * rw [hp'] at hempty; exact absurd hempty (by simp)
        * rw [hq'] at hempty; exact absurd hempty (by simp)
```

```
    | @pair P Q _ _ =>
      rw [embZero_inter_embPair] at hsub
      obtain <z, hz> := memS_nonempty hD hZ
      exact absurd (hsub hz) (Set.notMem_empty z)
  | @pair P Q hP hQ ihP ihQ =>
    intro Y hY Z hZ hsub
    cases hY with
    | gamma =>
      rw [Set.inter_eq_left.mpr (embPair_subset_Gamma (memS_subset_gamma hP) (
    memS_subset_gamma hQ))]
      exact MemS.pair hP hQ
    | @zero B hB =>
      rw [Set.inter_comm, embZero_inter_embPair] at hsub
      obtain <z, hz> := memS_nonempty hD hZ


      exact absurd (hsub hz) (Set.notMem_empty z)
    | @pair P' Q' hP' hQ' =>
      rw [embPair_inter]
      rw [embPair_inter] at hsub
      cases hZ with
      | gamma =>
        obtain <a, ha> := hD D.master D.master_mem
        exact absurd (hsub (show (([], a) : List Bool  x  a)  in  Gamma D from ha)) (
    by simp)
      | @zero C hC =>
        obtain <a, ha> := hD C hC
        exact absurd (hsub (show (([], a) : List Bool  x  a)  in  embZero C from <rfl,
     ha>)) (by simp)
      | @pair Z_0 Z_1 hZ_0 hZ_1 =>
        obtain <hsub_0, hsub_1> := embPair_subset.mp hsub
        exact MemS.pair (ihP hP' hZ_0 hsub_0) (ihQ hQ' hZ_1 hsub_1)

/-- **Example 6.1 (Scott 1981, PRG-19).** The *tree algebra* `D^S`: a neighbourhood
    system over
`Gamma = {1,2}* 0 Delta`, under the standing assumption `{}  notin  ?` (`hD`). -/
abbrev Dsharp (D : NeighborhoodSystem a) (hD : forall  X, D.mem X  ->  X.Nonempty) :
    NeighborhoodSystem (List Bool  x  a) where
  mem := MemS D
  master := Gamma D
  master_nonempty := by
    obtain <a, ha> := D.master_nonempty
    exact <([], a), ha>
  master_mem := MemS.gamma
  inter_mem := fun hX hY hZ hsub => memS_inter hD hX hY hZ hsub
  sub_master := memS_subset_gamma

@[simp] theorem Dsharp_mem {D : NeighborhoodSystem a} {hD : forall  X, D.mem X  ->  X.
    Nonempty}
    {W : Set (List Bool  x  a)} : (Dsharp D hD).mem W  <->  MemS D W := Iff.rfl

@[simp] theorem Dsharp_master {D : NeighborhoodSystem a} {hD : forall  X, D.mem X  ->
     X.Nonempty} :
    (Dsharp D hD).master = Gamma D := rfl

theorem memS_embPair_inv {D : NeighborhoodSystem a} (hD : forall  X, D.mem X  ->  X.
    Nonempty)
```

```
    {P Q : Set (List Bool  x  a)} (h : MemS D (embPair P Q)) : MemS D P  /\  MemS D Q
    := by
  generalize hW : embPair P Q = W at h
  cases h with
  | gamma =>
    exfalso
    obtain <a, ha> := hD D.master D.master_mem
    have h1 : (([], a) : List Bool  x  a)  in  Gamma D := ha
    rw [ <-  hW] at h1
    rcases h1 with <p', hp', -> | <q', hq', ->
    * simp at hp'
    * simp at hq'
  | @zero X hX =>
    exfalso
    obtain <t, ht> := embZero_nonempty (hD X hX)
    have hempty : t.1 = [] := ht.1
    rw [ <-  hW] at ht
    rcases ht with <p', hp', -> | <q', hq', ->
    * rw [hp'] at hempty; simp at hempty
    * rw [hq'] at hempty; simp at hempty
  | @pair P' Q' hP' hQ' =>
    obtain <hPP, hQQ> := embPair_injective hW
    rw [hPP, hQQ]; exact <hP', hQ'>

theorem memS_embZero_inv {D : NeighborhoodSystem a} (hD : forall  X, D.mem X  ->  X.
    Nonempty) {X : Set a}
    (h : MemS D (embZero X)) : D.mem X := by
  generalize hW : embZero X = W at h
  cases h with
  | gamma =>
    exfalso
    obtain <a, ha> := hD D.master D.master_mem
    have h1 : ((true :: [], a) : List Bool  x  a)  in  Gamma D := ha
    rw [ <-  hW] at h1; exact absurd h1.1 (by simp)
  | @zero X' hX' => rw [embZero_injective hW]; exact hX'
  | @pair P Q hP hQ =>
    exfalso
    obtain <t, ht> := embPair_nonempty (memS_nonempty hD hP) (Q := Q)
    have hcons : t.1  !=  [] := by
      rcases ht with <p', hp', -> | <q', hq', ->
      * simp [hp']
      * simp [hq']
    have hz : t  in  embZero X := by rw [ <-  hW] at ht; exact ht
    exact hcons hz.1

section Equation

variable (D : NeighborhoodSystem a) (hD : forall  X, D.mem X  ->  X.Nonempty)

include hD

/-- No neighbourhood of `D^S  x  D^S` is empty (needed to form the sum `D + (D^S  x  D^
    S)`). -/
theorem prod_dsharp_nonempty :
    forall  V, (prod (Dsharp D hD) (Dsharp D hD)).mem V  ->  V.Nonempty := by
  rintro V <P, Q, hP, hQ, rfl>
```

```
  obtain <t, ht> := memS_nonempty hD hP
  exact <Sum.inl t, mem_prodNbhd_inl.mpr ht>

end Equation

end Example61
end

end DomainTheory.NeighbourhoodSystems
end

section
-- Definition613/MonotoneAt.lean
universe w

namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap

/-- **Definition 6.13, monotone part (pointwise).** Given a subdomain relation `h : D ?
     E`, the data
witnessing that the functor `T` carries it to `T(D) ? T(E)` *and* carries the
    projection pair `i, j`
of 6.12 to the projection pair of `T(D) ? T(E)`.

Because `T` may change the token type, the two image systems live a priori over
    different carriers;
`carrier_eq` records that they coincide, `sub` is the resulting subdomain relation (
    with `T(E)`'s
system transported into `T(D)`'s carrier), and `inj_heq`/`proj_heq` say the canonical
    6.12 maps of
`sub` are `T(i)` and `T(j)` (up to that transport, hence `HEq`). -/
structure MonotoneAt (T : Endofunctor DomainObj.{w}) {a : Type w}
    {D E : NeighborhoodSystem a} (h : D ? E) : Prop where
  /-- The two image carriers coincide, so `T(D) ? T(E)` can be stated. -/
  carrier_eq : (T.obj <a, E>).carrier = (T.obj <a, D>).carrier
  /-- The image subdomain relation `T(D) ? T(E)` (with `T(E)`'s system carried into `T(
    D)`'s
  carrier). -/
  sub : (T.obj <a, D>).sys ?
    (carrier_eq  |>  (T.obj <a, E>).sys : NeighborhoodSystem (T.obj <a, D>).carrier)
  /-- The injection of `T(D) ? T(E)` is `T(i)` (Scott: the pair is mapped to `T(i), T(j
    )`). -/
  inj_heq : HEq (T.map (X := <a, D>) (Y := <a, E>) h.inj) sub.inj
  /-- The projection of `T(D) ? T(E)` is `T(j)`. -/
  proj_heq : HEq (T.map (X := <a, E>) (Y := <a, D>) h.proj) sub.proj

end DomainTheory.NeighbourhoodSystems
end
```

3.7.11 Definition 6.10 — subdomain

DEFINITION 6.10. For two neighbourhood systems D and E we write

$$D \triangleleft E$$

to mean that these are neighbourhood systems over the same set of tokens Δ and not only is $D \subseteq E$ but whenever $X, Y \in D$ and $X \cap Y \in E$, then $X \cap Y \in D$. $\square$

For the subdomain relation $D \lhd E$ to hold, D has to be a smaller family of neighbourhoods, but the notion of consistency in D also has to be the same as in E. Note that if $D_0 \lhd E$ and $D_1 \lhd E$ then

$$D_0 \lhd D_1 \text{ iff } D_0 \subseteq D_1.$$

It is also easy to prove that the union of a directed family of subdomains of E is again a subdomain. As a consequence of this remark we have:

Definition_6_10.lean

```
module

public import Mathlib.CategoryTheory.Category.Basic

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NeighborhoodSystem
public import Mathlib.Tactic.ToAdditive

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
export CategoryTheory (Category)

open scoped CategoryTheory

scoped notation "?" => CategoryTheory.CategoryStruct.id
end

section
variable {a : Type*}

/- concordance: source L3799 DEFINITION 6.10 subdomain -/
/-- **Definition 6.10 (Scott 1981, PRG-19).** The *subsystem* (subdomain) relation `D ?
     E` for two
neighbourhood systems over the same token type. It records that `D` and `E` are systems
     over the
same `Delta` (`master_eq`), that `D` is a subfamily of `E` (`sub`), and -- the essential
     clause -- that
consistency is inherited from `E`: an intersection of two `D`-neighbourhoods that
    happens to be an
`E`-neighbourhood is already a `D`-neighbourhood (`inter_closed`). -/
structure Subsystem (D E : NeighborhoodSystem a) : Prop where
  /-- `D` and `E` are systems over the same set of tokens `Delta`. -/
  master_eq : D.master = E.master
  /-- `D  subseteq  E`: every neighbourhood of `D` is a neighbourhood of `E`. -/
  sub : forall  {X : Set a}, D.mem X  ->  E.mem X
  /-- Consistency in `D` is the same as in `E`: if `X, Y  in  D` and `X  I  Y  in  E`,
    then `X  I  Y  in  D`. -/
  inter_closed : forall  {X Y : Set a}, D.mem X  ->  D.mem Y  ->  E.mem (X  I  Y)  ->
     D.mem (X  I  Y)
```

```
@[inherit_doc] infix:50 " ? " => Subsystem

namespace Subsystem

/-- Two neighbourhood systems with the same `mem` and the same `master` are equal (the
    remaining
fields of `NeighborhoodSystem` are `Prop`s). -/
theorem _root_.DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.ext {D E :
    NeighborhoodSystem a}
    (hmem : forall  X, D.mem X  <->  E.mem X) (hmaster : D.master = E.master) : D = E
    := by
  rcases D with <Dmem, Dmaster, _, _, _>
  rcases E with <Emem, Emaster, _, _, _>
  have hm : Dmem = Emem := funext fun X => propext (hmem X)
  subst hm
  subst hmaster
  rfl

/-- The subsystem relation is **antisymmetric**: `D ? E` and `E ? D` force `D = E`. (
    Mutual `sub`
gives equal `mem`, and `master_eq` gives equal masters.) -/
theorem antisymm {D E : NeighborhoodSystem a} (h_1 : D ? E) (h_2 : E ? D) : D = E :=
  NeighborhoodSystem.ext (fun _ => <fun h => h_1.sub h, fun h => h_2.sub h>) h_1.
    master_eq

/-- The subsystem relation is **reflexive**: `D ? D`. (The `inter_closed` clause is
    trivial -- the
hypothesis is already the conclusion.) -/
theorem refl (D : NeighborhoodSystem a) : D ? D where
  master_eq := rfl
  sub h := h
  inter_closed _ _ h := h

/-- The subsystem relation is **transitive**: `D ? E` and `E ? F` give `D ? F`.

The `inter_closed` clause threads through `E`: from `X, Y  in  D  subseteq  E` and `X
    I  Y  in  F`, the relation
`E ? F` puts `X  I  Y  in  E`, and then `D ? E` puts `X  I  Y  in  D`. -/
theorem trans {D E F : NeighborhoodSystem a} (h_1 : D ? E) (h_2 : E ? F) : D ? F where
  master_eq := h_1.master_eq.trans h_2.master_eq
  sub h := h_2.sub (h_1.sub h)
  inter_closed hX hY hXY :=
    h_1.inter_closed hX hY (h_2.inter_closed (h_1.sub hX) (h_1.sub hY) hXY)

/-- **Scott's remark (the prose after Definition 6.10).** Once `D_0` and `D_1` both sit
     inside a
common system `E` as subdomains, the subdomain relation between them is just inclusion
    of
neighbourhood families: `D_0 ? D_1  <->  D_0  subseteq  D_1`.

* ` -> ` is the `sub` clause of `D_0 ? D_1`.
* ` <- ` builds `D_0 ? D_1` from `D_0  subseteq  D_1`: the masters agree because both
    equal `E`'s master, and
  the `inter_closed` clause routes through `E` -- an intersection `X  I  Y` of `D_0`-
    neighbourhoods
  lying in `D_1` lies in `E` (since `D_1  subseteq  E`), and `D_0 ? E` then returns it
```

```
    to `D_0`. -/
theorem subsystem_iff_subset_of_common {D_0 D_1 E : NeighborhoodSystem a}
    (h_0 : D_0 ? E) (h_1 : D_1 ? E) :
    D_0 ? D_1  <->  forall  {X : Set a}, D_0.mem X  ->  D_1.mem X := by
  constructor
  * intro h _ hX; exact h.sub hX
  * intro hsub
    refine <h_0.master_eq.trans h_1.master_eq.symm, hsub, ?_>
    intro X Y hX hY hXY
    exact h_0.inter_closed hX hY (h_1.sub hXY)

end Subsystem

end

end DomainTheory.NeighbourhoodSystems
end

section
-- DomainObj.lean
universe w

namespace DomainTheory.NeighbourhoodSystems

/-- An object of the category of domains: a token type together with a neighbourhood
    system on it. -/
structure DomainObj : Type (w + 1) where
  /-- The token type. -/
  carrier : Type w
  /-- The neighbourhood system (the "domain"). -/
  sys : NeighborhoodSystem carrier

end DomainTheory.NeighbourhoodSystems
end

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem

namespace Example61

variable {a : Type*}

/-- The master neighbourhood `Gamma = {1,2}* 0 Delta` of `D^S`: any path, `D`-token in
    `Delta`. -/
def Gamma (D : NeighborhoodSystem a) : Set (List Bool  x  a) := {t | t.2  in  D.master
    }

/-- `1P = {(1::p, a) ? (p, a)  in  P}`: the `1`-prefixed copy of a `D^S`-neighbourhood
    `P`. -/
def embL (P : Set (List Bool  x  a)) : Set (List Bool  x  a) := {t | exists  p', t.1 =
     true :: p'  /\  (p', t.2)  in  P}

/-- `2Q = {(2::q, a) ? (q, a)  in  Q}`: the `2`-prefixed copy of a `D^S`-neighbourhood
    `Q`. -/
```

```
def embR (Q : Set (List Bool  x  a)) : Set (List Bool  x  a) := {t | exists  q', t.1 =
     false :: q'  /\  (q', t.2)  in  Q}

/-- `1P  U  2Q`: the product-style neighbourhood of `D^S`. -/
def embPair (P Q : Set (List Bool  x  a)) : Set (List Bool  x  a) := embL P  U  embR Q

/-- `0X = {([], a) ? a  in  X}`: the `0`-tagged copy of a `D`-neighbourhood `X` (empty
    `{1,2}`-path). -/
def embZero (X : Set a) : Set (List Bool  x  a) := {t | t.1 = []  /\  t.2  in  X}

end Example61
end
end DomainTheory.NeighbourhoodSystems
end
```

3.7.12 Proposition 6.11 — lattice of subsystems

PROPOSITION 6.11. For a given neighbourhood system E, the set of subsystems

$$\{D \mid D \triangleleft E\}$$

forms a domain in its own right. □

The subdomain relationship implies a mapping relationship between the domains.

Proposition_6_11.lean

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Exercise_2_22

public import DomainTheory.NeighbourhoodSystems.DomainEquations.Definition_6_10

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.Proposition611
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem Set DomainTheory.
    NeighbourhoodSystems.Exercise222

section
variable {a : Type*}

/-- The family of **neighbourhood-sets of subdomains of `E`**: a set of subsets of `
    Delta` lies in
`subFam E` exactly when it is `{X ? D.mem X}` for some subsystem `D ? E`. This is the
    concrete
family of sets that, by Exercise 2.22, represents `{D ? D ? E}` as a domain. -/
def subFam (E : NeighborhoodSystem a) : Set (Set (Set a)) :=
  {? | exists  D : NeighborhoodSystem a, D ? E  /\  ? = {X | D.mem X}}

/-- Consistency is inherited from `E` (Definition 6.10's essential clause): if `X, Y`
    lie in a
subdomain's neighbourhood-set and `X  I  Y  in  E`, then `X  I  Y` lies in it too. -/
theorem subFam_inter_closed (E : NeighborhoodSystem a) {? : Set (Set a)} {X Y : Set a}
    (h : ?  in  subFam E) (hX : X  in  ?) (hY : Y  in  ?) (hXY : E.mem (X  I  Y)) : X
     I  Y  in  ? := by
```

```
  obtain <D, hD, rfl> := h
  exact hD.inter_closed hX hY hXY

/-- The master `Delta = E.master` belongs to the neighbourhood-set of any subdomain (a
    subsystem shares
`E`'s master and contains it). -/
theorem subFam_master_mem (E : NeighborhoodSystem a) {? : Set (Set a)}
    (h : ?  in  subFam E) : E.master  in  ? := by
  obtain <D, hD, rfl> := h
  rw [ <-  hD.master_eq]
  exact D.master_mem

/-- Every member of a subdomain's neighbourhood-set is an `E`-neighbourhood (`D
    subseteq  E`). -/
theorem subFam_mem_E (E : NeighborhoodSystem a) {? : Set (Set a)} {X : Set a}
    (h : ?  in  subFam E) (hX : X  in  ?) : E.mem X := by
  obtain <D, hD, rfl> := h
  exact hD.sub hX

/-- The **intersection subdomain** of a non-empty family `?` of subdomain neighbourhood
    -sets: its
neighbourhoods are the sets common to *every* member of `?`. -/
def interSys (E : NeighborhoodSystem a) (? : Set (Set (Set a)))
    (hne : ?.Nonempty) (h? : ?  subseteq  subFam E) : NeighborhoodSystem a where
  mem X := forall  ?  in  ?, X  in  ?
  master := E.master
  master_nonempty := E.master_nonempty
  master_mem := fun ? h? => subFam_master_mem E (h? h?)
  inter_mem := by
    intro X Y Z hX hY hZ hsub ? h?
    have hEX : E.mem X := subFam_mem_E E (h? h?) (hX ? h?)
    have hEY : E.mem Y := subFam_mem_E E (h? h?) (hY ? h?)
    have hEZ : E.mem Z := subFam_mem_E E (h? h?) (hZ ? h?)
    exact subFam_inter_closed E (h? h?) (hX ? h?) (hY ? h?) (E.inter_mem hEX hEY hEZ
    hsub)
  sub_master := by
    intro X hX
    obtain <?, h?> := hne
    exact E.sub_master (subFam_mem_E E (h? h?) (hX ? h?))

/-- The neighbourhood-set of the intersection subdomain is exactly `?_0 ?`. -/
theorem interSys_nbset (E : NeighborhoodSystem a) (? : Set (Set (Set a)))
    (hne : ?.Nonempty) (h? : ?  subseteq  subFam E) :
    {X | (interSys E ? hne h?).mem X} = ?_0 ? := by
  ext X
  exact Set.mem_sInter.symm

/-- The intersection subdomain is a subsystem of `E`. -/
theorem interSys_subsystem (E : NeighborhoodSystem a) (? : Set (Set (Set a)))
    (hne : ?.Nonempty) (h? : ?  subseteq  subFam E) : interSys E ? hne h? ? E where
  master_eq := rfl
  sub := by
    intro X hX
    obtain <?, h?> := hne
    exact subFam_mem_E E (h? h?) (hX ? h?)
  inter_closed := by
```

```
    intro X Y hX hY hEXY ? h?
    exact subFam_inter_closed E (h? h?) (hX ? h?) (hY ? h?) hEXY

/-- Rebuild a subsystem of `E` from its neighbourhood-set `?  in  subFam E`. The data (
    the `mem`
predicate `⋆  in  ?` and the master `E.master`) depends only on `?`; the proof
    obligations are
discharged from `subFam` membership. -/
def ofMem (E : NeighborhoodSystem a) (? : Set (Set a)) (h : ?  in  subFam E) :
    NeighborhoodSystem a where
  mem X := X  in  ?
  master := E.master
  master_nonempty := E.master_nonempty
  master_mem := subFam_master_mem E h
  inter_mem := by
    intro X Y Z hX hY hZ hsub
    exact subFam_inter_closed E h hX hY
      (E.inter_mem (subFam_mem_E E h hX) (subFam_mem_E E h hY) (subFam_mem_E E h hZ)
    hsub)
  sub_master := fun hX => E.sub_master (subFam_mem_E E h hX)

/-- `ofMem` lands in the subsystems of `E`. -/
theorem ofMem_subsystem (E : NeighborhoodSystem a) (? : Set (Set a)) (h : ?  in
    subFam E) :
    ofMem E ? h ? E where
  master_eq := rfl
  sub := subFam_mem_E E h
  inter_closed := subFam_inter_closed E h
end

section
set_option linter.unusedVariables false

theorem theorem_611.{u_1} : forall  {a : Type u_1} (E : DomainTheory.
    NeighbourhoodSystems.NeighborhoodSystem a) (? : Set (Set a))
  (h :
    @Membership.mem (Set (Set a)) (Set (Set (Set a))) (@Set.instMembership (Set (Set a
    )))
      (@DomainTheory.NeighbourhoodSystems.Proposition611.subFam a E) ?),
  @DomainTheory.NeighbourhoodSystems.Subsystem a (@DomainTheory.NeighbourhoodSystems.
    Proposition611.ofMem a E ? h) E :=
  @DomainTheory.NeighbourhoodSystems.Proposition611.ofMem_subsystem
end

end DomainTheory.NeighbourhoodSystems.Proposition611

namespace DomainTheory.NeighbourhoodSystems.Proposition611
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem Set DomainTheory.
    NeighbourhoodSystems.Exercise222

section
variable {a : Type*}

/-- The subsystems of `E`, ordered by the **subdomain relation** `?`, form a partial
    order
(reflexive, transitive, antisymmetric -- Definition 6.10's API). -/
```

```
instance subPartialOrder (E : NeighborhoodSystem a) :
    PartialOrder {D : NeighborhoodSystem a // D ? E} where
  le D_0 D_1 := D_0.1 ? D_1.1
  le_refl D := Subsystem.refl D.1
  le_trans _ _ _ h_1 h_2 := h_1.trans h_2
  le_antisymm _ _ h_1 h_2 := Subtype.ext (h_1.antisymm h_2)
end

end DomainTheory.NeighbourhoodSystems.Proposition611

namespace DomainTheory.NeighbourhoodSystems.Proposition611
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem Set DomainTheory.
    NeighbourhoodSystems.Exercise222

section
variable {a : Type*}

/-- `subFam E` is non-empty: `E` itself is a subsystem (`Subsystem.refl`), so its own
neighbourhood-set `{X ? E.mem X}` is a member. -/
theorem subFam_nonempty (E : NeighborhoodSystem a) : (subFam E).Nonempty :=
  <{X | E.mem X}, E, Subsystem.refl E, rfl>

/-- **Closure under non-empty intersection** (Exercise 2.22's hypothesis (i)). -/
theorem subFam_sInter_mem (E : NeighborhoodSystem a) (? : Set (Set (Set a)))
    (hne : ?.Nonempty) (h? : ?  subseteq  subFam E) : ?_0 ?  in  subFam E :=
  <interSys E ? hne h?, interSys_subsystem E ? hne h?, (interSys_nbset E ? hne h?).
    symm>

/-- **The poset of subsystems is the family `subFam E`.** The subsystems of `E`,
    ordered by `?`,
are order-isomorphic to `subFam E` ordered by ` subseteq `. A subsystem is sent to its
    family of
neighbourhoods `{X ? D.mem X}`, and recovered by `ofMem`; order is preserved and
    reflected by
Scott's remark `Subsystem.subsystem_iff_subset_of_common`. -/
def subIso (E : NeighborhoodSystem a) :
    {D : NeighborhoodSystem a // D ? E} ?o {? : Set (Set a) // ?  in  subFam E} where
  toFun D := <{X | D.1.mem X}, <D.1, D.2, rfl>>
  invFun ? := <ofMem E ?.1 ?.2, ofMem_subsystem E ?.1 ?.2>
  left_inv := by
    intro D
    apply Subtype.ext
    apply NeighborhoodSystem.ext
    * intro X; exact Iff.rfl
    * exact D.2.master_eq.symm
  right_inv := by
    intro ?
    apply Subtype.ext
    ext X
    exact Iff.rfl
  map_rel_iff' := by
    intro a b
    show ({X | a.1.mem X} : Set (Set a))  subseteq  {X | b.1.mem X}  <->  a.1 ? b.1
    constructor
    * intro hsub
      refine (Subsystem.subsystem_iff_subset_of_common a.2 b.2).mpr ?_
```

```
      intro X hX
      exact hsub hX
    * intro hsub X hX
      exact hsub.sub hX
end

end DomainTheory.NeighbourhoodSystems.Proposition611

namespace DomainTheory.NeighbourhoodSystems.Proposition611
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem Set DomainTheory.
    NeighbourhoodSystems.Exercise222

section
variable {a : Type*}

/-- The **union subdomain** of a directed family `?` of subdomain neighbourhood-sets:
    its
neighbourhoods are those lying in *some* member of `?`. Directedness is what makes this
     closed
under consistent intersection. -/
def unionSys (E : NeighborhoodSystem a) (? : Set (Set (Set a)))
    (hne : ?.Nonempty) (h? : ?  subseteq  subFam E) (hdir : DirectedOn (*  subseteq
    *) ?) :
    NeighborhoodSystem a where
  mem X := exists  ?  in  ?, X  in  ?
  master := E.master
  master_nonempty := E.master_nonempty
  master_mem := by
    obtain <?, h?> := hne
    exact <?, h?, subFam_master_mem E (h? h?)>
  inter_mem := by
    intro X Y Z hX hY hZ hsub
    obtain <?x, h?x, hXx> := hX
    obtain <?y, h?y, hYy> := hY
    obtain <?z, h?z, hZz> := hZ
    obtain <?s, h?s, hxs, hys> := hdir ?x h?x ?y h?y
    have hXs : X  in  ?s := hxs hXx
    have hYs : Y  in  ?s := hys hYy
    have hEX : E.mem X := subFam_mem_E E (h? h?s) hXs
    have hEY : E.mem Y := subFam_mem_E E (h? h?s) hYs
    have hEZ : E.mem Z := subFam_mem_E E (h? h?z) hZz
    exact <?s, h?s, subFam_inter_closed E (h? h?s) hXs hYs (E.inter_mem hEX hEY hEZ
    hsub)>
  sub_master := by
    intro X hX
    obtain <?, h?, hX?> := hX
    exact E.sub_master (subFam_mem_E E (h? h?) hX?)

/-- The neighbourhood-set of the union subdomain is exactly `?_0 ?`. -/
theorem unionSys_nbset (E : NeighborhoodSystem a) (? : Set (Set (Set a)))
    (hne : ?.Nonempty) (h? : ?  subseteq  subFam E) (hdir : DirectedOn (*  subseteq
    *) ?) :
    {X | (unionSys E ? hne h? hdir).mem X} = ?_0 ? := by
  ext X
  exact Set.mem_sUnion.symm
```

```
/-- The union subdomain is a subsystem of `E` (Scott's remark: the directed union of
    subdomains is
again a subdomain). -/
theorem unionSys_subsystem (E : NeighborhoodSystem a) (? : Set (Set (Set a)))
    (hne : ?.Nonempty) (h? : ?  subseteq  subFam E) (hdir : DirectedOn (*  subseteq
    *) ?) :
    unionSys E ? hne h? hdir ? E where
  master_eq := rfl
  sub := by
    intro X hX
    obtain <?, h?, hX?> := hX
    exact subFam_mem_E E (h? h?) hX?
  inter_closed := by
    intro X Y hX hY hEXY
    obtain <?x, h?x, hXx> := hX
    obtain <?y, h?y, hYy> := hY
    obtain <?s, h?s, hxs, hys> := hdir ?x h?x ?y h?y
    exact <?s, h?s, subFam_inter_closed E (h? h?s) (hxs hXx) (hys hYy) hEXY>

/-- **Closure under directed union** (Exercise 2.22's hypothesis (ii)) -- Scott's
    remark. -/
theorem subFam_sUnion_mem (E : NeighborhoodSystem a) (? : Set (Set (Set a)))
    (hne : ?.Nonempty) (h? : ?  subseteq  subFam E) (hdir : DirectedOn (*  subseteq
    *) ?) :
    ?_0 ?  in  subFam E :=
  <unionSys E ? hne h? hdir, unionSys_subsystem E ? hne h? hdir,
    (unionSys_nbset E ? hne h? hdir).symm>
end
end DomainTheory.NeighbourhoodSystems.Proposition611

section
namespace DomainTheory.NeighbourhoodSystems.Proposition611
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem Set DomainTheory.
    NeighbourhoodSystems.Exercise222
section
variable {a : Type*}
/- concordance: source L3817 PROPOSITION 6.11 lattice of subsystems -/
/-- **Proposition 6.11 (Scott 1981, PRG-19).** For a neighbourhood system `E`, the set
    of
subsystems `{D ? D ? E}`, ordered by the subdomain relation `?`, *forms a domain in its
     own
right*: it is order-isomorphic to the domain `|reprSystem (subFam E) ...|` produced by
    the abstract
representation theorem (Exercise 2.22), using that `subFam E` is closed under non-empty
intersections (`subFam_sInter_mem`) and directed unions (Scott's remark, `
    subFam_sUnion_mem`). -/
def subsystemReprIso (E : NeighborhoodSystem a) :
    {D : NeighborhoodSystem a // D ? E} ?o
      (reprSystem (subFam E) (subFam_sInter_mem E) (subFam_nonempty E)).Element :=
  (subIso E).trans
    (reprIso (subFam E) (subFam_sInter_mem E) (subFam_nonempty E) (subFam_sUnion_mem E
    )).symm
end

end DomainTheory.NeighbourhoodSystems.Proposition611
end
```

3.7.13 Proposition 6.12 — projection pair

PROPOSITION 6.12. If $D \triangleleft E$, then there exists a projection pair of approximable mappings:

$$i : D \to E \quad \text{and} \quad j : E \to D$$

where $j \circ i = I_D$ and $i \circ j \subseteq I_E$, which are determined as element-wise functions by these equations:

$$i(x) = \{Y \in E \mid \exists X \in x.\, X \subseteq Y\},$$

and

$$j(y) = y \cap D,$$

for all $x \in |D|$ and $y \in |E|$. □

The proof is left for the exercises.

Proposition_6_12.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.Exercise_6_29

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap
section
variable {a b : Type*} {D : NeighborhoodSystem a} {E : NeighborhoodSystem b}

/- concordance: source L3827 PROPOSITION 6.12 projection pair -/
/-- **Proposition 6.12 + Lemma 6.15 packaged.** A subdomain relation `D ? E` is in
    particular a
witness of `D ? E` (take `D' = D`). Together with `trianglelefteq_of_projectionPair`,
    this shows
`D ? E` holds **iff** there is a projection pair `D  <=>  E`. -/
theorem Subsystem.trianglelefteq {D E : NeighborhoodSystem a} (h : D ? E) : D ? E :=
  <D, h, Isomorphic.refl D>

end
end DomainTheory.NeighbourhoodSystems
end
```

3.7.14 Definition 6.13 — continuity on domains

DEFINITION 6.13. A functor T is *monotone on domains* iff whenever $D \triangleleft E$, then not only do we have $T(D) \triangleleft T(E)$ but the projection pair i, j of 6.12 is mapped to the same kind of projection pair $T(i), T(j)$. A monotone functor is *continuous on domains* iff whenever E is a domain, then the mapping

$$\lambda D.\, T(D) : \{D \mid D \triangleleft E\} \to \{D' \mid D' \triangleleft T(E)\}$$

is approximable. □

We can now state an existence theorem that covers in fairly wide generality the examples of this lecture.

Definition_6_13.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.Theorem_6_9

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap
section
universe w

/- concordance: source L3849 DEFINITION 6.13 continuity on domains -/
/-- **Definition 6.13 (Scott 1981, PRG-19), monotone on domains.** A functor `T` is *
    monotone on
domains* iff every subdomain relation `D ? E` is carried to a subdomain relation `T(D)
    ? T(E)` whose
projection pair is `(T(i), T(j))` -- see `MonotoneAt`. -/
def MonotoneOnDomains (T : Endofunctor DomainObj.{w}) : Prop :=
  forall  {a : Type w} {D E : NeighborhoodSystem a} (h : D ? E), MonotoneAt T h

/-- The **target-side neighbourhood family** of the image `T(D)` of a subsystem `h : D
    ? E`, viewed
over `T(E)`'s carrier (using `MonotoneAt.carrier_eq` to transport neighbourhoods of `T(
    D)` to that
carrier). This is the data on which "`fun D. T(D)` is approximable" is expressed. -/
def targetFam (T : Endofunctor DomainObj.{w}) (hmono : MonotoneOnDomains T)
    {a : Type w} {D E : NeighborhoodSystem a} (h : D ? E) :
    Set (Set (T.obj <a, E>).carrier) :=
  {Y | (T.obj <a, D>).sys.mem ((hmono h).carrier_eq  |>  Y)}

/-- **Definition 6.13 (Scott 1981, PRG-19), continuous on domains.** A monotone
    functor `T` is
*continuous on domains* iff `fun D. T(D) : {D ? D ? E}  ->  {D' ? D' ? T(E)}` is
    approximable. In the
neighbourhood framework this is *preservation of directed unions of subsystems*: for
    any non-empty
directed family `?` of subsystems of `E` whose union is the subsystem `U` (`hU`), the
    target-side
neighbourhood family of `T(U)` is the union of those of the `T(D)` for `D  in  ?`. -/
def ContinuousOnDomains (T : Endofunctor DomainObj.{w}) : Prop :=
  exists  hmono : MonotoneOnDomains T,
    forall  {a : Type w} {E : NeighborhoodSystem a}
      (? : Set (NeighborhoodSystem a)) (h? : forall  ?D?, D  in  ?  ->  D ? E)
      (_hne : ?.Nonempty) (_hdir : DirectedOn (* ? *) ?)
      {U : NeighborhoodSystem a} (hUE : U ? E)
      (_hU : forall  X, U.mem X  <->  exists  D  in  ?, D.mem X),
      targetFam T hmono hUE = ? D, ? hD : D  in  ?, targetFam T hmono (h? hD)

end

section
```

```
/-- The **identity functor is monotone on domains**: it fixes objects and maps, so `T(D
    ) ? T(E)` is
just `D ? E` and the projection pair is unchanged. -/
theorem monotoneOnDomains_id : MonotoneOnDomains (idEndofunctor DomainObj.{w}) := by
  intro a D E h
  exact <rfl, h, HEq.rfl, HEq.rfl>
end

section
/-- The **identity functor is continuous on domains**: `targetFam` reduces to the plain
     neighbourhood
family, so directed-union preservation is exactly the hypothesis `hU` that `U` is the
    union. -/
theorem continuousOnDomains_id : ContinuousOnDomains (idEndofunctor DomainObj.{w}) :=
    by
  refine <monotoneOnDomains_id, ?_>
  intro a E ? h? _hne _hdir U hUE hU
  apply Set.ext
  intro Y
  simp only [targetFam, Set.mem_iUnion, exists_prop]
  exact hU Y
end

namespace Exercise619
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.Exercise619
open DomainTheory.NeighbourhoodSystems.Example62 DomainTheory.NeighbourhoodSystems.
    ExampleB
open DomainTheory.NeighbourhoodSystems.Exercise510

section
/-- **An object of the Exercise 6.19 category.** An `{}`-free neighbourhood system over
`Str = {0,1}*` (`ne`: every neighbourhood is non-empty, Scott's `{}  notin  ?`). -/
structure ScottSys where
  /-- The underlying neighbourhood system on `{0,1}*`. -/
  sys : NeighborhoodSystem Str
  /-- `{}  notin  ?`: every neighbourhood is non-empty. -/
  ne : forall  X, sys.mem X  ->  X.Nonempty
end

end Exercise619
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.Exercise619
open DomainTheory.NeighbourhoodSystems.Example62 DomainTheory.NeighbourhoodSystems.
    ExampleB
open DomainTheory.NeighbourhoodSystems.Exercise510

section
namespace Exercise619

variable {A_0 A_1 B_0 B_1 C_0 C_1 : ScottSys}

/-- **The functor-expression algebra.** A `T(X)` built from constants, the identity,
    and binary sums
and products -- Scott's closed family of constructs. -/
inductive FExpr where
  | const : ScottSys  ->  FExpr
  | var : FExpr
```

```
  | sum : FExpr  ->  FExpr  ->  FExpr
  | prod : FExpr  ->  FExpr  ->  FExpr

/-- A non-empty `b`-tagged copy can never sit inside a `b'`-tagged copy for `b  !=  b
    '`. -/
theorem embBit_not_subset_cross {b b' : Bool} (h : b  !=  b') {X Y : Set Str} (hX : X.
    Nonempty)
    (hsub : embBit b X  subseteq  embBit b' Y) : False := by
  obtain <t, ht> := hX
  obtain <w', he, -> := hsub <t, rfl, ht>
  simp only [List.cons.injEq] at he
  exact h he.1

end Exercise619
end

section
namespace Exercise619

/-- The **product object** `?_0  x  ?_1` of Part A, repackaged as an object of the
    category. -/
def ScottSys.prod (A_0 A_1 : ScottSys) : ScottSys :=
  <prodTok A_0.sys A_1.sys, prodTok_nonempty>

end Exercise619
end

section
namespace Exercise619

variable {A_0 A_1 B_0 B_1 C_0 C_1 : ScottSys}

/-- **`f_0  x  f_1`, the action of the product functor on (approximable) maps.** A
    product
neighbourhood `{?}  U  0X  U  1Y` is sent to `{?}  U  0X'  U  1Y'` whenever `X f_0 X'`
    and `Y f_1 Y'`. -/
def prodMapTok (f_0 : ApproximableMap A_0.sys B_0.sys) (f_1 : ApproximableMap A_1.sys
    B_1.sys) :
    ApproximableMap (A_0.prod A_1).sys (B_0.prod B_1).sys where
  rel W W' := exists  X Y X' Y', f_0.rel X X'  /\  f_1.rel Y Y'  /\
    W = prodTokNbhd X Y  /\  W' = prodTokNbhd X' Y'
  rel_dom := by
    rintro W W' <X, Y, X', Y', h0, h1, rfl, ->
    exact prodTok_mem_prodTokNbhd (f_0.rel_dom h0) (f_1.rel_dom h1)
  rel_cod := by
    rintro W W' <X, Y, X', Y', h0, h1, -, rfl>
    exact prodTok_mem_prodTokNbhd (f_0.rel_cod h0) (f_1.rel_cod h1)
  master_rel :=
    <A_0.sys.master, A_1.sys.master, B_0.sys.master, B_1.sys.master,
      f_0.master_rel, f_1.master_rel, rfl, rfl>
  inter_right := by
    rintro W W'_1 W'_2 <X, Y, X', Y', h0, h1, rfl, rfl> <X_2, Y_2, X'_2, Y'_2, h0_2,
    h1_2, hWeq, rfl>
    obtain <rfl, rfl> := prodTokNbhd_injective hWeq
    exact <X, Y, X'  I  X'_2, Y'  I  Y'_2, f_0.inter_right h0 h0_2, f_1.inter_right h1
     h1_2, rfl,
```

```
      prodTokNbhd_inter X' X'_2 Y' Y'_2>
  mono := by
    rintro W W'' Z Z' <X, Y, X', Y', h0, h1, rfl, rfl> hWW hZZ' hW'' hZ'
    obtain <X_2, Y_2, hX_2, hY_2, rfl> := hW''
    obtain <X'_3, Y'_3, hX'_3, hY'_3, rfl> := hZ'
    obtain <hsX, hsY> := prodTokNbhd_subset_iff.mp hWW
    obtain <hsX', hsY'> := prodTokNbhd_subset_iff.mp hZZ'
    exact <X_2, Y_2, X'_3, Y'_3, f_0.mono h0 hsX hsX' hX_2 hX'_3, f_1.mono h1 hsY hsY'
     hY_2 hY'_3, rfl, rfl>

/-- **The one-neighbourhood system `{Gamma}`** over `{0,1}*`: its only neighbourhood is
    `Gamma` itself, and
its master (token set) is `Gamma`. It is `{}`-free precisely because `Gamma` is non-
    empty. -/
def singletonSys (Gamma : Set Str) (h : Gamma.Nonempty) : ScottSys where
  sys :=
    { mem := fun X => X = Gamma
      master := Gamma
      master_nonempty := h
      master_mem := rfl
      inter_mem := by
        intro X Y Z hX hY _ _
        show X  I  Y = Gamma
        rw [hX, hY, Set.inter_self]
      sub_master := by intro X hX; rw [show X = Gamma from hX] }
  ne := by intro X hX; rw [show X = Gamma from hX]; exact h

end Exercise619
end

section
namespace Exercise619

/-- The **sum object** `?_0 + ?_1` of Part A, repackaged as an object of the category.
    -/
def ScottSys.sum (A_0 A_1 : ScottSys) : ScottSys :=
  <sumTok A_0.sys A_1.sys A_0.ne A_1.ne, sumTok_nonempty>

end Exercise619
end

section
namespace Exercise619

variable {A_0 A_1 B_0 B_1 C_0 C_1 : ScottSys}

/-- **`?  in  tok(C)` for every constant `C` occurring in `T`.** This is what keeps the
     bottom `{?}` and
the whole Kleene chain inside the domain `{Gamma ? ?  in  Gamma}`; sums and products
    satisfy it for free. -/
def FExpr.RootedConst : FExpr  ->  Prop
  | .const C => ([] : Str)  in  C.sys.master
  | .var => True
  | .sum a b => a.RootedConst  /\  b.RootedConst
  | .prod a b => a.RootedConst  /\  b.RootedConst
```

```
/-- **The token-level master recursion.** `mFun T Gamma` computes `tok(T({Gamma}))` purely from `Gamma`, without
touching the neighbourhood data of `{Gamma}` (`mFun_eq_master`): constants are constant, the identity
returns `Gamma`, and both sum and product wrap the two component token sets with the tags `0,1` under a
common root `?` (`sumTokMaster = prodTokNbhd` agree on masters). -/
def mFun : FExpr  ->  Set Str  ->  Set Str
  | .const C, _ => C.sys.master
  | .var, Gamma => Gamma
  | .sum T_0 T_1, Gamma => insert ([] : Str) (embBit false (mFun T_0 Gamma)  U  embBit true (mFun T_1 Gamma))
  | .prod T_0 T_1, Gamma => insert ([] : Str) (embBit false (mFun T_0 Gamma)  U  embBit true (mFun T_1 Gamma))

/-- The **Kleene iteration** `mFun?({?})` whose union is the least fixed point above `{?}`. -/
def mIter (T : FExpr) : Nat  ->  Set Str
  | 0 => {([] : Str)}
  | n + 1 => mFun T (mIter T n)

/-- **The action of `T` on objects.** -/
def FExpr.obj : FExpr  ->  ScottSys  ->  ScottSys
  | .const D, _ => D
  | .var, X => X
  | .sum T_0 T_1, X => (T_0.obj X).sum (T_1.obj X)
  | .prod T_0 T_1, X => (T_0.obj X).prod (T_1.obj X)

/-- **`f_0 + f_1`, the action of the sum functor on (approximable) maps.** It carries the master to the
master (so it is strict), a left copy `0X` to `0X'` whenever `X f_0 X'`, and a right copy `1Y` to
`1Y'` whenever `Y f_1 Y'`. -/
def sumMapTok (f_0 : ApproximableMap A_0.sys B_0.sys) (f_1 : ApproximableMap A_1.sys B_1.sys) :
    ApproximableMap (A_0.sum A_1).sys (B_0.sum B_1).sys where
  rel W W' :=
    ((sumTok A_0.sys A_1.sys A_0.ne A_1.ne).mem W  /\  W' = sumTokMaster B_0.sys B_1.sys)  \/
    (exists  X X', f_0.rel X X'  /\  W = embBit false X  /\  W' = embBit false X')  \/
    (exists  Y Y', f_1.rel Y Y'  /\  W = embBit true Y  /\  W' = embBit true Y')
  rel_dom := by
    rintro W W' (<hW, -> | <X, X', hrel, rfl, -> | <Y, Y', hrel, rfl, ->)
    * exact hW
    * exact Or.inr (Or.inl <X, f_0.rel_dom hrel, rfl>)
    * exact Or.inr (Or.inr <Y, f_1.rel_dom hrel, rfl>)
  rel_cod := by
    rintro W W' (<-, rfl> | <X, X', hrel, -, rfl> | <Y, Y', hrel, -, rfl>)
    * exact Or.inl rfl
    * exact Or.inr (Or.inl <X', f_0.rel_cod hrel, rfl>)
    * exact Or.inr (Or.inr <Y', f_1.rel_cod hrel, rfl>)
  master_rel := Or.inl <(A_0.sum A_1).sys.master_mem, rfl>
  inter_right := by
    rintro W W'_1 W'_2 h1 h2
    rcases h1 with <hW, rfl> | <X, X', hrel, rfl, rfl> | <Y, Y', hrel, rfl, rfl>
    * rcases h2 with <-, rfl> | <X, X', hrel, hWeq, rfl> | <Y, Y', hrel, hWeq, rfl>
```

```
      * exact Or.inl <hW, by rw [Set.inter_self]>
      * exact Or.inr (Or.inl <X, X', hrel, hWeq, by rw [sumTokMaster_inter_embF (f_0.
    rel_cod hrel)]>)
      * exact Or.inr (Or.inr <Y, Y', hrel, hWeq, by rw [sumTokMaster_inter_embT (f_1.
    rel_cod hrel)]>)
    * rcases h2 with <-, rfl> | <X_2, X'_2, hrel_2, hWeq, rfl> | <Y_2, Y'_2, hrel_2,
    hWeq, rfl>
      * refine Or.inr (Or.inl <X, X', hrel, rfl, ?_>)
        rw [Set.inter_comm, sumTokMaster_inter_embF (f_0.rel_cod hrel)]
      * obtain rfl := embBit_injective hWeq
        exact Or.inr (Or.inl <X, X'  I  X'_2, f_0.inter_right hrel hrel_2, rfl,
    embBit_inter false X' X'_2>)
      * exact absurd hWeq (embBit_ne (by decide) (A_0.ne X (f_0.rel_dom hrel)))
    * rcases h2 with <-, rfl> | <X_2, X'_2, hrel_2, hWeq, rfl> | <Y_2, Y'_2, hrel_2,
    hWeq, rfl>
      * refine Or.inr (Or.inr <Y, Y', hrel, rfl, ?_>)
        rw [Set.inter_comm, sumTokMaster_inter_embT (f_1.rel_cod hrel)]
      * exact absurd hWeq (embBit_ne (by decide) (A_1.ne Y (f_1.rel_dom hrel)))
      * obtain rfl := embBit_injective hWeq
        exact Or.inr (Or.inr <Y, Y'  I  Y'_2, f_1.inter_right hrel hrel_2, rfl,
    embBit_inter true Y' Y'_2>)
  mono := by
    rintro W W'' Z Z' h hWW hZZ' hW'' hZ'
    rcases h with <-, rfl> | <X, X', hrel, rfl, rfl> | <Y, Y', hrel, rfl, rfl>
    * -- output was the master; a blunter neighbourhood must again be the master.
      exact Or.inl <hW'', Set.Subset.antisymm ((B_0.sum B_1).sys.sub_master hZ') hZZ'>
    * -- input `0X`, output `0X'`. The sharper input `W''` and blunter output `Z''`.
      rcases hZ' with rfl | <X_3, hX_3, rfl> | <Y_3, hY_3, rfl>
      * exact Or.inl <hW'', rfl>
      * rcases hW'' with rfl | <X_2, hX_2, rfl> | <Y_2, hY_2, rfl>
        * exact absurd (hWW nil_mem_sumTokMaster) nil_not_mem_embBit
        * exact Or.inr (Or.inl <X_2, X_3,
            f_0.mono hrel (embBit_subset.mp hWW) (embBit_subset.mp hZZ') hX_2 hX_3,
    rfl, rfl>)
        * exact absurd hWW (fun hsub => embBit_not_subset_cross (by decide) (A_1.ne
    Y_2 hY_2) hsub)
      * exact absurd hZZ' (fun hsub =>
          embBit_not_subset_cross (by decide) (B_0.ne X' (f_0.rel_cod hrel)) hsub)
    * -- input `1Y`, output `1Y'`.
      rcases hZ' with rfl | <X_3, hX_3, rfl> | <Y_3, hY_3, rfl>
      * exact Or.inl <hW'', rfl>
      * exact absurd hZZ' (fun hsub =>
          embBit_not_subset_cross (by decide) (B_1.ne Y' (f_1.rel_cod hrel)) hsub)
      * rcases hW'' with rfl | <X_2, hX_2, rfl> | <Y_2, hY_2, rfl>
        * exact absurd (hWW nil_mem_sumTokMaster) nil_not_mem_embBit
        * exact absurd hWW (fun hsub => embBit_not_subset_cross (by decide) (A_0.ne
    X_2 hX_2) hsub)
        * exact Or.inr (Or.inr <Y_2, Y_3,
            f_1.mono hrel (embBit_subset.mp hWW) (embBit_subset.mp hZZ') hY_2 hY_3,
    rfl, rfl>)

/-- **The action of `T` on (approximable) maps.** Constants act by the identity, the
    identity functor
acts by `f` itself, and sums/products act by the bifunctorial combinators `sumMapTok`/`
    prodMapTok`. -/
def FExpr.map : (T : FExpr)  ->  {X Y : ScottSys}  ->  ApproximableMap X.sys Y.sys  ->
```

```
    ApproximableMap (T.obj X).sys (T.obj Y).sys
  | .const D, _, _, _ => idMap D.sys
  | .var, _, _, f => f
  | .sum T_0 T_1, _, _, f => sumMapTok (T_0.map f) (T_1.map f)
  | .prod T_0 T_1, _, _, f => prodMapTok (T_0.map f) (T_1.map f)

/-- **`tok(?)`** -- the underlying set of tokens of a system, i.e. its master
    neighbourhood `Delta`. -/
def ScottSys.tok (D : ScottSys) : Set Str := D.sys.master

/-- `mFun T Gamma` is exactly the token set `tok(T({Gamma})) = (T.obj {Gamma}).sys.
    master`. -/
theorem mFun_eq_master : (T : FExpr)  ->  {Gamma : Set Str}  ->  (h : Gamma.Nonempty)
     ->
    mFun T Gamma = (T.obj (singletonSys Gamma h)).sys.master
  | .const _, _, _ => rfl
  | .var, _, _ => rfl
  | .sum T_0 T_1, Gamma, h => by
      show insert ([] : Str) (embBit false (mFun T_0 Gamma)  U  embBit true (mFun T_1
    Gamma))
        = insert ([] : Str) (embBit false ((T_0.obj (singletonSys Gamma h)).sys.master
    )
            U  embBit true ((T_1.obj (singletonSys Gamma h)).sys.master))
      rw [mFun_eq_master T_0 h, mFun_eq_master T_1 h]
  | .prod T_0 T_1, Gamma, h => by
      show insert ([] : Str) (embBit false (mFun T_0 Gamma)  U  embBit true (mFun T_1
    Gamma))
        = insert ([] : Str) (embBit false ((T_0.obj (singletonSys Gamma h)).sys.master
    )
            U  embBit true ((T_1.obj (singletonSys Gamma h)).sys.master))
      rw [mFun_eq_master T_0 h, mFun_eq_master T_1 h]

/-- Monotonicity of the tagged-union shape shared by sum and product. -/
theorem insertTag_mono {p q p' q' : Set Str} (hp : p  subseteq  p') (hq : q  subseteq
     q') :
    insert ([] : Str) (embBit false p  U  embBit true q)
       subseteq  insert ([] : Str) (embBit false p'  U  embBit true q') := by
  rintro w (rfl | hw | hw)
  * exact Or.inl rfl
  * obtain <w', rfl, hw'> := hw
    exact Or.inr (Or.inl <w', rfl, hp hw'>)
  * obtain <w', rfl, hw'> := hw
    exact Or.inr (Or.inr <w', rfl, hq hw'>)

/-- **`fun Gamma. tok(T({Gamma}))` is monotone on the domain.** -/
theorem mFun_mono (T : FExpr) {Gamma Gamma' : Set Str} (h : Gamma  subseteq  Gamma') :
     mFun T Gamma  subseteq  mFun T Gamma' := by
  induction T with
  | const C => exact subset_rfl
  | var => exact h
  | sum T_0 T_1 ih_0 ih_1 => exact insertTag_mono ih_0 ih_1
  | prod T_0 T_1 ih_0 ih_1 => exact insertTag_mono ih_0 ih_1

/-- Continuity (full additivity) of the tagged-union shape shared by sum and product.
    -/
theorem insertTag_continuous {? : Set (Set Str)} {U : Set Str} (hne : ?.Nonempty)
```

```
    {p q : Set Str  ->  Set Str}
    (hp : forall  w, w  in  p U  <->  exists  Gamma  in  ?, w  in  p Gamma)
    (hq : forall  w, w  in  q U  <->  exists  Gamma  in  ?, w  in  q Gamma) (w : Str)
    :
    (w  in  insert ([] : Str) (embBit false (p U)  U  embBit true (q U)))
       <->  exists  Gamma  in  ?, w  in  insert ([] : Str) (embBit false (p Gamma)  U
     embBit true (q Gamma)) := by
  simp only [Set.mem_insert_iff, Set.mem_union]
  constructor
  * rintro (rfl | hw | hw)
    * obtain <Gamma, hGamma> := hne; exact <Gamma, hGamma, Or.inl rfl>
    * obtain <w', rfl, hw'> := hw
      obtain <Gamma, hGamma, hpGamma> := (hp w').mp hw'
      exact <Gamma, hGamma, Or.inr (Or.inl <w', rfl, hpGamma>)>
    * obtain <w', rfl, hw'> := hw
      obtain <Gamma, hGamma, hqGamma> := (hq w').mp hw'
      exact <Gamma, hGamma, Or.inr (Or.inr <w', rfl, hqGamma>)>
  * rintro <Gamma, hGamma, (rfl | hw | hw)>
    * exact Or.inl rfl
    * obtain <w', rfl, hw'> := hw
      exact Or.inr (Or.inl <w', rfl, (hp w').mpr <Gamma, hGamma, hw'>>)
    * obtain <w', rfl, hw'> := hw
      exact Or.inr (Or.inr <w', rfl, (hq w').mpr <Gamma, hGamma, hw'>>)

/-- **`fun Gamma. tok(T({Gamma}))` is continuous on the domain `{Gamma ? ?  in  Gamma
    }`.** For a non-empty ` subseteq `-directed
family `?` with union `U`, the token set of `T({U})` is the union of those of the `T({
    Gamma})`. (The
proof in fact establishes full additivity -- directedness is not needed for the master
     level -- but the
statement is the directed-sup form Scott calls *continuous*.) -/
theorem mFun_continuous (T : FExpr) {? : Set (Set Str)} {U : Set Str}
    (_hdir : DirectedOn (*  subseteq  *) ?) (hne : ?.Nonempty)
    (hU : forall  w, w  in  U  <->  exists  Gamma  in  ?, w  in  Gamma) :
    forall  w, w  in  mFun T U  <->  exists  Gamma  in  ?, w  in  mFun T Gamma := by
  induction T with
  | const C =>
      intro w
      exact <fun hw => let <Gamma, hGamma> := hne; <Gamma, hGamma, hw>, fun <_, _, hw>
     => hw>
  | var => intro w; exact hU w
  | sum T_0 T_1 ih_0 ih_1 => intro w; exact insertTag_continuous hne ih_0 ih_1 w
  | prod T_0 T_1 ih_0 ih_1 => intro w; exact insertTag_continuous hne ih_0 ih_1 w

/-- If `?  in  Gamma` then `?  in  tok(T({Gamma}))` -- so `fun Gamma. tok(T({Gamma}))`
    is an endofunction of the domain. -/
theorem mFun_nil_mem : forall  (T : FExpr), T.RootedConst  ->  {Gamma : Set Str}  ->
    ([] : Str)  in  Gamma  ->  ([] : Str)  in  mFun T Gamma
  | .const _, hC, _, _ => hC
  | .var, _, _, hGamma => hGamma
  | .sum _ _, _, _, _ => Set.mem_insert _ _
  | .prod _ _, _, _, _ => Set.mem_insert _ _

theorem nil_mem_mIter (T : FExpr) (hT : T.RootedConst) : forall  n, ([] : Str)  in
    mIter T n
  | 0 => rfl
```

```
  | n + 1 => mFun_nil_mem T hT (nil_mem_mIter T hT n)

theorem mIter_mono_step (T : FExpr) (hT : T.RootedConst) :
    forall  n, mIter T n  subseteq  mIter T (n + 1)
  | 0 => by
      intro w hw
      have hw' : w = [] := hw
      subst hw'
      exact mFun_nil_mem T hT rfl
  | n + 1 => mFun_mono T (mIter_mono_step T hT n)

theorem mIter_mono (T : FExpr) (hT : T.RootedConst) {m n : Nat} (hmn : m  <=  n) :
    mIter T m  subseteq  mIter T n := by
  induction hmn with
  | refl => exact subset_rfl
  | step _ ih => intro x hx; exact mIter_mono_step T hT _ (ih hx)

/-- The Kleene union is a **fixed point** of `fun Gamma. tok(T({Gamma}))`. -/
theorem mFun_iter_fixed (T : FExpr) (hT : T.RootedConst) :
    mFun T (? n, mIter T n) = ? n, mIter T n := by
  have hstep := mIter_mono_step T hT
  have hne : (Set.range (mIter T)).Nonempty := <mIter T 0, 0, rfl>
  have hdir : DirectedOn (*  subseteq  *) (Set.range (mIter T)) := by
    rintro _ <i, rfl> _ <j, rfl>
    exact <mIter T (max i j), <max i j, rfl>,

      mIter_mono T hT (le_max_left i j), mIter_mono T hT (le_max_right i j)>
  have hU : forall  v, v  in  (? n, mIter T n)  <->  exists  S  in  Set.range (mIter T
    ), v  in  S := by
    intro v
    constructor
    * intro hv; rw [Set.mem_iUnion] at hv; obtain <n, hn> := hv
      exact <mIter T n, <n, rfl>, hn>
    * rintro <S, <n, rfl>, hv>; exact Set.mem_iUnion.mpr <n, hv>
  apply Set.ext; intro w
  rw [mFun_continuous T hdir hne hU w]
  constructor
  * rintro <S, <n, rfl>, hwS>; exact Set.mem_iUnion.mpr <n + 1, hwS>
  * intro hw
    rw [Set.mem_iUnion] at hw; obtain <n, hn> := hw
    exact <mIter T n, <n, rfl>, hstep n hn>

end Exercise619
end
end DomainTheory.NeighbourhoodSystems
end
```

3.7.15 Theorem 6.14 — existence of initial algebras

THEOREM 6.14. If the functor T is continuous on maps and monotone and continuous on domains, and if there is a set Γ such that

$$\{\Gamma\} \lhd T(\{\Gamma\}),$$

then there exists an initial T-algebra.

Proof: We proceed as in the proof of the fixed-point theorem by iterating the functor. The assumption about Γ means that, as a neighbourhood system, $T(\{\Gamma\})$ is a system over the same set Γ. Thus, if we iterate T to form $T^n(\{\Gamma\})$, all these systems are over Γ and indeed

$$T^n(\{\Gamma\}) \lhd T^{n+1}(\{\Gamma\})$$

for all n. We can thus introduce

$$\mathcal{D} = \bigcup_{n=0}^{\infty} T^n(\{\Gamma\}),$$

and it is easy to check that $\mathcal{D}$ is a system over Γ and

$$T^n(\{\Gamma\}) \lhd \mathcal{D}$$

holds for all n. But then we have for all n:

$$T^n(\{\Gamma\}) \lhd T^{n+1}(\{\Gamma\}) \lhd T(\mathcal{D}),$$

which implies $\mathcal{D} \lhd T(\mathcal{D})$. But T is continuous on domains, so

$$\begin{aligned} T(\mathcal{D}) &= T\left(\bigcup_{n=0}^{\infty} T^n(\{\Gamma\})\right) \\ &= \bigcup_{n=0}^{\infty} T^{n+1}(\{\Gamma\}) \\ &= \mathcal{D}. \end{aligned}$$

Thus, not only is $\mathcal{D}$ a T-algebra, but the isomorphism we get for $\mathcal{D}$ and $T(\mathcal{D})$ is just the identity mapping. We know by 6.9 that homomorphisms exist; what remains to show is that homomorphism from $\mathcal{D}$ are unique. As in the examples, we will show in effect they are determined uniquely on the finite elements of $\mathcal{D}$.

Since each $T^n(\{\Gamma\}) \lhd \mathcal{D}$, there are projection mappings $i_n : T^n(\{\Gamma\}) \to \mathcal{D}$ and $j_n : \mathcal{D} \to T^n(\{\Gamma\})$.

Define $\rho_n : \mathcal{D} \to \mathcal{D}$ by $\rho_n = i_n \circ j_n$. Projection pairs are always pairs of strict mappings (Why?), and so are in the category. By assumption and 6.13, the functor T preserves these maps, so we have

$$T(\rho_n) = T(i_n) \circ T(j_n) = i_{n+1} \circ j_{n+1} = \rho_{n+1}.$$

As a neighbourhood relation ρ_n can be characterized by :

$$X \rho_n Y \quad \text{iff} \quad \exists z \in T^n(\{\Gamma\}).\ X \subseteq z \subseteq Y.$$

We thus see that $\rho_n \subseteq \rho_{n+1}$ and

$$\bigcup_{n=0}^{\infty} \rho_n = I_{\mathcal{D}}.$$

Now suppose $k : T(E) \to E$ is any T-algebra and $h : \mathcal{D} \to E$ is a homomorphism. The mapping will satisfy the fixed-point equation

$$h = k \circ T(h),$$

where no other mappings need be written in because $\mathcal{D} = T(\mathcal{D})$ and so

$$T(h) : \mathcal{D} \to T(E).$$

We wish to show that h really is the least fixed point of this equation.

Define $h_n = h \circ \rho_n : \mathcal{D} \to E$. For $n = 0$, the map ρ_0 is the trivial map where $\rho_0(x) = \bot_{\mathcal{D}}$ for all $x \in |\mathcal{D}|$. But h must be strict, so $h_0(x) = \bot_E$ for all $x \in |\mathcal{D}|$; that is, h_0 is the least element of $|\mathcal{D} \to_\bot E|$. Now calculate :

$$\begin{aligned} k \circ T(h_n) &= k \circ T(h) \circ T(\rho_n) \\ &= h \circ \rho_{n+1} \\ &= h_{n+1} \, . \end{aligned}$$

This shows that the union of the h_n is the least fixed point of $\lambda h.\, k \circ T(h)$. But

$$\begin{aligned} \bigcup_{n=0}^{\infty} h_n &= \bigcup_{n=0}^{\infty} h \circ \rho_n \\ &= h \circ \bigcup_{n=0}^{\infty} \rho_n \\ &= h \circ I_D = h, \end{aligned}$$

so the given h is in fact the least fixed point. The homomorphism is uniquely determined, and D is the initial T-algebra. □

Having the existence of initial T-algebras, we can prove one more result that shows just how minimal they are. We need a lemma about projection pairs, first, that shows where sub-domains fit in. We write $D \trianglelefteq E$ as short for $D \cong D'$ for some $D' \triangleleft E$ in the following. The lemma gives a converse to 6.12.

Theorem_6_14.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.Exercise_6_27

public import DomainTheory.NeighbourhoodSystems.DomainEquations.Proposition_6_6
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.Exercise510
section
universe w
namespace Theorem614
variable {a : Type w}

/- concordance: source L3861 THEOREM 6.14 existence of initial algebras -/
/-- **Theorem 6.14 (Scott 1981, PRG-19) -- the canonical fixed point.** Under the
    hypotheses
(continuous on maps, monotone and continuous on domains, with a generating set `{Gamma}
     ? T({Gamma})`), the
```

```
iterated colimit `? = ?_n T?({Gamma})` is a `T`-algebra whose structure map is an
    isomorphism
`T(?)  ~=  ?` (the identity, since `T(?) = ?`), and there is a homomorphism from `?`
    into every
`T`-algebra with a strict structure map (Theorem 6.9). This is Scott's *existence* of
    the initial
`T`-algebra. -/
theorem exists_algebra_with_hom (s : Setup.{w}) :
    exists  A : TAlgebra s.T, Nonempty (Iso (s.T.obj A.carrier) A.carrier)  /\
      forall  B : TAlgebra s.T, IsStrict B.str  ->  Nonempty (AlgHom A B) :=
  <colimAlg s, <colimIso s>, fun B hk => nonempty_algHom s B hk>

/-- **`?_0 = False`** when `{Gamma}` is the trivial one-point system: `?_0` relates `X`
     only to the master.
This is where Scott's `{Gamma}` (a *one-point* domain) is used. -/
theorem rho_zero_rel (s : Setup.{w}) (hGamma : forall  X, s.Gamma.mem X  ->  X = s.
    Gamma.master)
    {X Y : Set s.Tok} :
    (rho s 0).rel X Y  <->  (colim s).mem X  /\  Y = (colim s).master := by
  rw [rho_rel]
  constructor
  * rintro <hcX, hcY, z, hz, _, hzY>
    have hzm : z = s.Gamma.master := hGamma z hz
    subst hzm
    refine <hcX, Set.Subset.antisymm ((colim s).sub_master hcY) ?_>
    rw [colim_master]; exact hzY
  * rintro <hcX, rfl>
    refine <hcX, (colim s).master_mem, s.Gamma.master, s.Gamma.master_mem, ?_, ?_>
    * have h := (colim s).sub_master hcX; rwa [colim_master] at h
    * rw [colim_master]

/-- For a strict map `g`, `g(False) = False` relationally: `g` sends `Delta` only to `
    Delta`. -/
theorem strict_rel_master {b_0 b_1 : Type w} {V_0 : NeighborhoodSystem b_0}
    {V_1 : NeighborhoodSystem b_1} {g : ApproximableMap V_0 V_1} (hg : IsStrict g) {Z
    : Set b_1} :
    g.rel V_0.master Z  <->  Z = V_1.master :=
  <fun h => hg h, fun h => h  |>  g.master_rel>

/-- For a strict homomorphism `g`, the base `g  comp  ?_0` is the least map: it relates
     `X` only to the
master of `E`, independent of `g`. -/
theorem gcomp_rho_zero_rel (s : Setup.{w}) (hGamma : forall  X, s.Gamma.mem X  ->  X =
     s.Gamma.master)
    (B : TAlgebra s.T) {g : ApproximableMap (colim s) B.carrier.sys} (hg : IsStrict g)
    {X : Set s.Tok} {Z : Set B.carrier.carrier} :
    (g.comp (rho s 0)).rel X Z  <->  (colim s).mem X  /\  Z = B.carrier.sys.master :=
    by
  rw [comp_rel]
  constructor
  * rintro <Y, hrho, hgYZ>
    rw [rho_zero_rel s hGamma] at hrho
    obtain <hcX, rfl> := hrho
    exact <hcX, (strict_rel_master hg).mp hgYZ>
  * rintro <hcX, rfl>
    exact <(colim s).master, (rho_zero_rel s hGamma).mpr <hcX, rfl>, g.master_rel>
```

```
/-- The base case of `g`-independence: any two strict maps agree after ` comp  ?_0`. -/
theorem gcomp_rho_zero_indep (s : Setup.{w}) (hGamma : forall  X, s.Gamma.mem X  ->  X
     = s.Gamma.master)
    (B : TAlgebra s.T) {g g' : ApproximableMap (colim s) B.carrier.sys}
    (hg : IsStrict g) (hg' : IsStrict g') :
    g.comp (rho s 0) = g'.comp (rho s 0) := by
  apply ApproximableMap.ext
  intro X Z
  rw [gcomp_rho_zero_rel s hGamma B hg, gcomp_rho_zero_rel s hGamma B hg']

/-- Transport of a `Hom X X` along an object equality is heterogeneously equal to
    itself. -/
theorem transport_heq {Obj : Type*} [Category Obj] {X Y : Obj} (e : X = Y)
    (f : (X  -->  X)) : HEq (e  |>  f : (Y  -->  Y)) f := by
  cases e; rfl

/-- **`?_n_+_1 = i  comp  T(?_n)  comp  j`** (Scott's `T(?_n) = ?_n_+_1`, conjugated by
     the structure iso). Since the
iso `?  ~=  T(?)` is the identity, this is the carrier transport of `T(?_n)`; combined
    with
`map_rho_heq` it pins `?_n_+_1`. -/
theorem key_rho (s : Setup.{w}) (n : Nat) :
    rho s (n + 1) = (colimIso s).hom ? Tmap_rho s n ? (colimIso s).inv := by
  rw [show (colimIso s).hom = (isoOfEq (colimObj_eq s)).hom from rfl,
      show (colimIso s).inv = (isoOfEq (colimObj_eq s)).inv from rfl,
      isoOfEq_conj (colimObj_eq s) (Tmap_rho s n)]
  apply eq_of_heq
  exact HEq.trans (map_rho_heq s n).symm
    (transport_heq (colimObj_eq s) (Tmap_rho s n)).symm

/-- **The fixed-point recursion `g_n_+_1 = k  comp  T(g_n)  comp  j`.** Using `key_rho`
     (`?_n_+_1 = i comp T(?_n) comp j`) and the
homomorphism square `g  comp  i = k  comp  T(g)`. -/
theorem gcomp_rho_succ (s : Setup.{w}) (B : TAlgebra s.T) (g : AlgHom (colimAlg s) B)
    (n : Nat) :
    g.hom.comp (rho s (n + 1))
      = B.str.comp ((s.T.map (X := objColim s) (Y := B.carrier)
          (g.hom.comp (rho s n))).comp (colimIso s).inv) := by
  have hcomm : g.hom ? (colimIso s).hom
      = B.str ? s.T.map (X := objColim s) (Y := B.carrier) g.hom := g.comm
  show g.hom ? rho s (n + 1)
      = B.str ? (s.T.map (X := objColim s) (Y := B.carrier) (g.hom ? rho s n)) ? (
    colimIso s).inv
  calc g.hom ? rho s (n + 1)
      = g.hom ? ((colimIso s).hom ? (Tmap_rho s n ? (colimIso s).inv)) :=
          congrArg (fun x => g.hom ? x) (key_rho s n)
    _ = (g.hom ? (colimIso s).hom) ? (Tmap_rho s n ? (colimIso s).inv) :=
          CategoryTheory.Category.assoc (Tmap_rho s n ? (colimIso s).inv) (colimIso s)
    .hom g.hom
    _ = (B.str ? s.T.map (X := objColim s) (Y := B.carrier) g.hom)
          ? (Tmap_rho s n ? (colimIso s).inv) := by rw [hcomm]
    _ = B.str ? (s.T.map (X := objColim s) (Y := B.carrier) g.hom
          ? (Tmap_rho s n ? (colimIso s).inv)) :=
        (CategoryTheory.Category.assoc (Tmap_rho s n ? (colimIso s).inv)
          (s.T.map (X := objColim s) (Y := B.carrier) g.hom) B.str).symm
```

```
    _ = B.str ? ((s.T.map (X := objColim s) (Y := B.carrier) g.hom ? Tmap_rho s n)
            ? (colimIso s).inv) :=
          congrArg (B.str ? *)
            (CategoryTheory.Category.assoc (colimIso s).inv (Tmap_rho s n)
              (s.T.map (X := objColim s) (Y := B.carrier) g.hom))
    _ = B.str ? (s.T.map (X := objColim s) (Y := B.carrier) (g.hom ? rho s n)
            ? (colimIso s).inv) :=
          congrArg (fun m => B.str ? (m ? (colimIso s).inv))
            (s.T.map_comp (X := objColim s) (Y := objColim s) (Z := B.carrier) g.hom
              (rho s n)).symm

/-- **`g`-independence of `g_n = g  comp  ?_n`.** For any two strict homomorphisms into
     the same algebra,
`g  comp  ?_n = g'  comp  ?_n` for all `n` -- the sequence is determined by the
    recursion, not by `g`. -/
theorem gcomp_rho_indep (s : Setup.{w}) (hGamma : forall  X, s.Gamma.mem X  ->  X = s.
    Gamma.master)
    (B : TAlgebra s.T) (g g' : AlgHom (colimAlg s) B)
    (hg : IsStrict g.hom) (hg' : IsStrict g'.hom) (n : Nat) :
    g.hom.comp (rho s n) = g'.hom.comp (rho s n) := by
  induction n with
  | zero => exact gcomp_rho_zero_indep s hGamma B hg hg'
  | succ k ih => rw [gcomp_rho_succ s B g k, gcomp_rho_succ s B g' k, ih]

/-- **The underlying maps of two strict homomorphisms coincide**: `g = g  comp  I = g
    comp  ?_n ?_n =
?_n (g  comp  ?_n)`, and the latter is `g`-independent. -/
theorem gcomp_eq (s : Setup.{w}) (hGamma : forall  X, s.Gamma.mem X  ->  X = s.Gamma.
    master)
    (B : TAlgebra s.T) (g g' : AlgHom (colimAlg s) B)
    (hg : IsStrict g.hom) (hg' : IsStrict g'.hom) :
    g.hom = g'.hom := by
  have key : g.hom.comp (iSupRho s) = g'.hom.comp (iSupRho s) := by
    apply ApproximableMap.ext
    intro X Z
    rw [comp_rel, comp_rel]
    constructor
    * rintro <Y, <n, hrho>, hgYZ>
      have hin : (g.hom.comp (rho s n)).rel X Z := <Y, hrho, hgYZ>
      rw [gcomp_rho_indep s hGamma B g g' hg hg' n] at hin
      obtain <Y', hrho', hgYZ'> := hin
      exact <Y', <n, hrho'>, hgYZ'>
    * rintro <Y, <n, hrho>, hgYZ>
      have hin : (g'.hom.comp (rho s n)).rel X Z := <Y, hrho, hgYZ>
      rw [ <-  gcomp_rho_indep s hGamma B g g' hg hg' n] at hin
      obtain <Y', hrho', hgYZ'> := hin
      exact <Y', <n, hrho'>, hgYZ'>
  have e : g.hom.comp (idMap (colim s)) = g.hom := comp_idMap g.hom
  have e' : g'.hom.comp (idMap (colim s)) = g'.hom := comp_idMap g'.hom
  rw [ <-  e,  <-  e',  <-  iSupRho_eq_id]
  exact key

/-- **Uniqueness of strict homomorphisms out of `?`.** Any two strict `T`-algebra
    homomorphisms from
the canonical solution into the same algebra are equal. -/
theorem algHom_unique (s : Setup.{w}) (hGamma : forall  X, s.Gamma.mem X  ->  X = s.
```

```
    Gamma.master)
     (B : TAlgebra s.T) (g g' : AlgHom (colimAlg s) B)
     (hg : IsStrict g.hom) (hg' : IsStrict g'.hom) : g = g' :=
  algHom_ext (gcomp_eq s hGamma B g g' hg hg')

/-- **Theorem 6.14 (Scott 1981, PRG-19) -- initial `T`-algebra.** When `{Gamma}` is the
     one-point
generating system, the canonical solution `? = ?_n T?({Gamma})` is the **initial** `T`-
    algebra among the
strict algebras: for every `T`-algebra `B` with a strict structure map there is a *
    unique* strict
homomorphism `?  ->  B`. (Existence is Theorem 6.9; uniqueness is the `?_n` projection-
    chain argument.) -/
theorem exists_unique_strict_algHom (s : Setup.{w}) (hGamma : forall  X, s.Gamma.mem X
      ->  X = s.Gamma.master)
    (B : TAlgebra s.T) (hk : IsStrict B.str) :
    exists  g : AlgHom (colimAlg s) B, IsStrict g.hom  /\
      forall  g' : AlgHom (colimAlg s) B, IsStrict g'.hom  ->  g' = g := by
  obtain <<g, hg>> := nonempty_strict_algHom s B hk
  exact <g, hg, fun g' hg' => algHom_unique s hGamma B g' g hg' hg>

end Theorem614
end

end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.Theorem614
section
set_option linter.unusedVariables false

theorem theorem_614.{w} : And
  (forall  (s : DomainTheory.NeighbourhoodSystems.Theorem614.Setup.{w}),
    exists  (A :
      @DomainTheory.NeighbourhoodSystems.TAlgebra.{w + 1, w} DomainTheory.
    NeighbourhoodSystems.DomainObj.{w}
        DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T),
      And
        (Nonempty.{w + 1}
          (@DomainTheory.NeighbourhoodSystems.Iso.{w + 1, w} DomainTheory.
    NeighbourhoodSystems.DomainObj.{w}
            DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w}
            (@DomainTheory.NeighbourhoodSystems.Endofunctor.obj.{w + 1, w}
              DomainTheory.NeighbourhoodSystems.DomainObj.{w}
              DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T
              (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{w + 1, w}
                DomainTheory.NeighbourhoodSystems.DomainObj.{w}
                DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T A))
            (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{w + 1, w}
              DomainTheory.NeighbourhoodSystems.DomainObj.{w}
              DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T A)))
        (forall
          (B :
            @DomainTheory.NeighbourhoodSystems.TAlgebra.{w + 1, w} DomainTheory.
    NeighbourhoodSystems.DomainObj.{w}
              DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T),
          @DomainTheory.NeighbourhoodSystems.Exercise510.IsStrict.{w, w}
              (@DomainTheory.NeighbourhoodSystems.Endofunctor.obj.{w + 1, w}
```

```
                DomainTheory.NeighbourhoodSystems.DomainObj.{w}
                DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T
                (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{w + 1, w}
                  DomainTheory.NeighbourhoodSystems.DomainObj.{w}
                  DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T B)
  ).carrier
             (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{w + 1, w}
                DomainTheory.NeighbourhoodSystems.DomainObj.{w}
                DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T B).
  carrier
             (@DomainTheory.NeighbourhoodSystems.Endofunctor.obj.{w + 1, w}
                DomainTheory.NeighbourhoodSystems.DomainObj.{w}
                DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T
                (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{w + 1, w}
                  DomainTheory.NeighbourhoodSystems.DomainObj.{w}
                  DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T B)
  ).sys
             (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{w + 1, w}
                DomainTheory.NeighbourhoodSystems.DomainObj.{w}
                DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T B).
  sys
             (@DomainTheory.NeighbourhoodSystems.TAlgebra.str.{w + 1, w}
               DomainTheory.NeighbourhoodSystems.DomainObj.{w}
               DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T B)  ->
           Nonempty.{w + 1}
             (@DomainTheory.NeighbourhoodSystems.AlgHom.{w + 1, w} DomainTheory.
  NeighbourhoodSystems.DomainObj.{w}
               DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T A B)))
(forall  (s : DomainTheory.NeighbourhoodSystems.Theorem614.Setup.{w}),
  (forall  (X : Set.{w} s.Tok),
     @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem.{w} s.Tok s.Gamma X
   ->
       @Eq.{w + 1} (Set.{w} s.Tok) X (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.master.{w} s.Tok s.Gamma))  ->
    forall
      (B :
        @DomainTheory.NeighbourhoodSystems.TAlgebra.{w + 1, w} DomainTheory.
  NeighbourhoodSystems.DomainObj.{w}
          DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T),
      @DomainTheory.NeighbourhoodSystems.Exercise510.IsStrict.{w, w}
          (@DomainTheory.NeighbourhoodSystems.Endofunctor.obj.{w + 1, w}
              DomainTheory.NeighbourhoodSystems.DomainObj.{w}
              DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T
              (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{w + 1, w}
                DomainTheory.NeighbourhoodSystems.DomainObj.{w}
                DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T B)).
  carrier
          (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{w + 1, w}
              DomainTheory.NeighbourhoodSystems.DomainObj.{w}
              DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T B).
  carrier
          (@DomainTheory.NeighbourhoodSystems.Endofunctor.obj.{w + 1, w}
              DomainTheory.NeighbourhoodSystems.DomainObj.{w}
              DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T
              (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{w + 1, w}
                DomainTheory.NeighbourhoodSystems.DomainObj.{w}
```

```
              DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T B)).
sys
         (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{w + 1, w}
             DomainTheory.NeighbourhoodSystems.DomainObj.{w}
             DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T B).sys
         (@DomainTheory.NeighbourhoodSystems.TAlgebra.str.{w + 1, w} DomainTheory.
NeighbourhoodSystems.DomainObj.{w}
           DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T B)  ->
       exists  (g :
         @DomainTheory.NeighbourhoodSystems.AlgHom.{w + 1, w} DomainTheory.
NeighbourhoodSystems.DomainObj.{w}
           DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T
           (DomainTheory.NeighbourhoodSystems.Theorem614.colimAlg.{w} s) B),
         And
           (@DomainTheory.NeighbourhoodSystems.Exercise510.IsStrict.{w, w}
             (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{w + 1, w}
                 DomainTheory.NeighbourhoodSystems.DomainObj.{w}
                 DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T
                 (DomainTheory.NeighbourhoodSystems.Theorem614.colimAlg.{w} s)).
carrier
             (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{w + 1, w}
                 DomainTheory.NeighbourhoodSystems.DomainObj.{w}
                 DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T B)
.carrier
             (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{w + 1, w}
                 DomainTheory.NeighbourhoodSystems.DomainObj.{w}
                 DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T
                 (DomainTheory.NeighbourhoodSystems.Theorem614.colimAlg.{w} s)).sys
             (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{w + 1, w}
                 DomainTheory.NeighbourhoodSystems.DomainObj.{w}
                 DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T B)
.sys
             (@DomainTheory.NeighbourhoodSystems.AlgHom.hom.{w + 1, w}
               DomainTheory.NeighbourhoodSystems.DomainObj.{w}
               DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T
               (DomainTheory.NeighbourhoodSystems.Theorem614.colimAlg.{w} s) B g))
           (forall
             (g' :
               @DomainTheory.NeighbourhoodSystems.AlgHom.{w + 1, w} DomainTheory.
NeighbourhoodSystems.DomainObj.{w}
                 DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T
                 (DomainTheory.NeighbourhoodSystems.Theorem614.colimAlg.{w} s) B),
             @DomainTheory.NeighbourhoodSystems.Exercise510.IsStrict.{w, w}
                 (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{w + 1, w}
                     DomainTheory.NeighbourhoodSystems.DomainObj.{w}
                     DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.
T
                     (DomainTheory.NeighbourhoodSystems.Theorem614.colimAlg.{w} s))
.carrier
                 (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{w + 1, w}
                     DomainTheory.NeighbourhoodSystems.DomainObj.{w}
                     DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.
T B).carrier
                 (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{w + 1, w}
                     DomainTheory.NeighbourhoodSystems.DomainObj.{w}
                     DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.
```

```
  T
                       (DomainTheory.NeighbourhoodSystems.Theorem614.colimAlg.{w} s))
  .sys
                  (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{w + 1, w}
                      DomainTheory.NeighbourhoodSystems.DomainObj.{w}
                      DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.
  T B).sys
                  (@DomainTheory.NeighbourhoodSystems.AlgHom.hom.{w + 1, w}
                    DomainTheory.NeighbourhoodSystems.DomainObj.{w}
                    DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T
                    (DomainTheory.NeighbourhoodSystems.Theorem614.colimAlg.{w} s) B
  g')  ->
                @Eq.{w + 1}
                  (@DomainTheory.NeighbourhoodSystems.AlgHom.{w + 1, w}
                    DomainTheory.NeighbourhoodSystems.DomainObj.{w}
                    DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T
                    (DomainTheory.NeighbourhoodSystems.Theorem614.colimAlg.{w} s) B)
                  g' g)) :=
@And.intro
(forall  (s : DomainTheory.NeighbourhoodSystems.Theorem614.Setup.{w}),
  exists  (A :
    @DomainTheory.NeighbourhoodSystems.TAlgebra.{w + 1, w} DomainTheory.
  NeighbourhoodSystems.DomainObj.{w}
      DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T),
    And
      (Nonempty.{w + 1}
        (@DomainTheory.NeighbourhoodSystems.Iso.{w + 1, w} DomainTheory.
  NeighbourhoodSystems.DomainObj.{w}
          DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w}
          (@DomainTheory.NeighbourhoodSystems.Endofunctor.obj.{w + 1, w}
            DomainTheory.NeighbourhoodSystems.DomainObj.{w}
            DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T
            (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{w + 1, w}
              DomainTheory.NeighbourhoodSystems.DomainObj.{w}
              DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T A))
          (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{w + 1, w}
            DomainTheory.NeighbourhoodSystems.DomainObj.{w}
            DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T A)))
      (forall
        (B :
          @DomainTheory.NeighbourhoodSystems.TAlgebra.{w + 1, w} DomainTheory.
  NeighbourhoodSystems.DomainObj.{w}
            DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T),
        @DomainTheory.NeighbourhoodSystems.Exercise510.IsStrict.{w, w}
            (@DomainTheory.NeighbourhoodSystems.Endofunctor.obj.{w + 1, w}
                DomainTheory.NeighbourhoodSystems.DomainObj.{w}
                DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T
                (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{w + 1, w}
                  DomainTheory.NeighbourhoodSystems.DomainObj.{w}
                  DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T B)
  ).carrier
            (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{w + 1, w}
                DomainTheory.NeighbourhoodSystems.DomainObj.{w}
                DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T B).
  carrier
            (@DomainTheory.NeighbourhoodSystems.Endofunctor.obj.{w + 1, w}
```

```
               DomainTheory.NeighbourhoodSystems.DomainObj.{w}
               DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T
               (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{w + 1, w}
                 DomainTheory.NeighbourhoodSystems.DomainObj.{w}
                 DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T B)
 ).sys
           (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{w + 1, w}
               DomainTheory.NeighbourhoodSystems.DomainObj.{w}
               DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T B).
 sys
           (@DomainTheory.NeighbourhoodSystems.TAlgebra.str.{w + 1, w}
             DomainTheory.NeighbourhoodSystems.DomainObj.{w}
             DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T B)  ->
         Nonempty.{w + 1}
           (@DomainTheory.NeighbourhoodSystems.AlgHom.{w + 1, w} DomainTheory.
 NeighbourhoodSystems.DomainObj.{w}
             DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T A B)))
(forall  (s : DomainTheory.NeighbourhoodSystems.Theorem614.Setup.{w}),
  (forall  (X : Set.{w} s.Tok),
     @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem.{w} s.Tok s.Gamma X
   ->
       @Eq.{w + 1} (Set.{w} s.Tok) X (@DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem.master.{w} s.Tok s.Gamma))  ->
    forall
      (B :
        @DomainTheory.NeighbourhoodSystems.TAlgebra.{w + 1, w} DomainTheory.
 NeighbourhoodSystems.DomainObj.{w}
          DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T),
      @DomainTheory.NeighbourhoodSystems.Exercise510.IsStrict.{w, w}
          (@DomainTheory.NeighbourhoodSystems.Endofunctor.obj.{w + 1, w}
              DomainTheory.NeighbourhoodSystems.DomainObj.{w}
              DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T
              (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{w + 1, w}
                DomainTheory.NeighbourhoodSystems.DomainObj.{w}
                DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T B)).
 carrier
          (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{w + 1, w}
              DomainTheory.NeighbourhoodSystems.DomainObj.{w}
              DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T B).
 carrier
          (@DomainTheory.NeighbourhoodSystems.Endofunctor.obj.{w + 1, w}
              DomainTheory.NeighbourhoodSystems.DomainObj.{w}
              DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T
              (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{w + 1, w}
                DomainTheory.NeighbourhoodSystems.DomainObj.{w}
                DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T B)).
 sys
          (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{w + 1, w}
              DomainTheory.NeighbourhoodSystems.DomainObj.{w}
              DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T B).sys
          (@DomainTheory.NeighbourhoodSystems.TAlgebra.str.{w + 1, w} DomainTheory.
 NeighbourhoodSystems.DomainObj.{w}
            DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T B)  ->
        exists  (g :
          @DomainTheory.NeighbourhoodSystems.AlgHom.{w + 1, w} DomainTheory.
 NeighbourhoodSystems.DomainObj.{w}
```

```
          DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T
          (DomainTheory.NeighbourhoodSystems.Theorem614.colimAlg.{w} s) B),
        And
          (@DomainTheory.NeighbourhoodSystems.Exercise510.IsStrict.{w, w}
            (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{w + 1, w}

                DomainTheory.NeighbourhoodSystems.DomainObj.{w}
                DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T
                (DomainTheory.NeighbourhoodSystems.Theorem614.colimAlg.{w} s)).
carrier
            (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{w + 1, w}
                DomainTheory.NeighbourhoodSystems.DomainObj.{w}
                DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T B)
.carrier
            (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{w + 1, w}
                DomainTheory.NeighbourhoodSystems.DomainObj.{w}
                DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T
                (DomainTheory.NeighbourhoodSystems.Theorem614.colimAlg.{w} s)).sys
            (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{w + 1, w}
                DomainTheory.NeighbourhoodSystems.DomainObj.{w}
                DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T B)
.sys
            (@DomainTheory.NeighbourhoodSystems.AlgHom.hom.{w + 1, w}
              DomainTheory.NeighbourhoodSystems.DomainObj.{w}
              DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T
              (DomainTheory.NeighbourhoodSystems.Theorem614.colimAlg.{w} s) B g))
          (forall
            (g' :
              @DomainTheory.NeighbourhoodSystems.AlgHom.{w + 1, w} DomainTheory.
NeighbourhoodSystems.DomainObj.{w}
                DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T
                (DomainTheory.NeighbourhoodSystems.Theorem614.colimAlg.{w} s) B),
            @DomainTheory.NeighbourhoodSystems.Exercise510.IsStrict.{w, w}
                (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{w + 1, w}
                    DomainTheory.NeighbourhoodSystems.DomainObj.{w}
                    DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.
T
                    (DomainTheory.NeighbourhoodSystems.Theorem614.colimAlg.{w} s))
.carrier
                (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{w + 1, w}
                    DomainTheory.NeighbourhoodSystems.DomainObj.{w}
                    DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.
T B).carrier
                (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{w + 1, w}
                    DomainTheory.NeighbourhoodSystems.DomainObj.{w}
                    DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.
T
                    (DomainTheory.NeighbourhoodSystems.Theorem614.colimAlg.{w} s))
.sys
                (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{w + 1, w}
                    DomainTheory.NeighbourhoodSystems.DomainObj.{w}
                    DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.
T B).sys
                (@DomainTheory.NeighbourhoodSystems.AlgHom.hom.{w + 1, w}
                  DomainTheory.NeighbourhoodSystems.DomainObj.{w}
                  DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T
```

```
                    (DomainTheory.NeighbourhoodSystems.Theorem614.colimAlg.{w} s) B
    g')  ->
                @Eq.{w + 1}
                  (@DomainTheory.NeighbourhoodSystems.AlgHom.{w + 1, w}
                    DomainTheory.NeighbourhoodSystems.DomainObj.{w}
                    DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} s.T
                    (DomainTheory.NeighbourhoodSystems.Theorem614.colimAlg.{w} s) B)
                  g' g))
  DomainTheory.NeighbourhoodSystems.Theorem614.exists_algebra_with_hom.{w}
  DomainTheory.NeighbourhoodSystems.Theorem614.exists_unique_strict_algHom.{w}
end

end DomainTheory.NeighbourhoodSystems.Theorem614
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.Exercise510

open Example62C in
/-- Every neighbourhood of the three-way separated sum `sum3` is non-empty (so `sum3`
    is again an
object of the `{}`-free category). -/
theorem sum3_nonempty {a b ? : Type*} {V_0 : NeighborhoodSystem a} {V_1 :
    NeighborhoodSystem b}
    {V_2 : NeighborhoodSystem ?} {h_0 : forall  X, V_0.mem X  ->  X.Nonempty}
    {h_1 : forall  Y, V_1.mem Y  ->  Y.Nonempty} {h_2 : forall  Z, V_2.mem Z  ->  Z.
    Nonempty} :
    forall  W, (sum3 V_0 V_1 V_2 h_0 h_1 h_2).mem W  ->  W.Nonempty := by
  rintro W (rfl | <X, hX, rfl> | <Y, hY, rfl> | <Z, hZ, rfl>)
  * exact <none, none_mem_master3>
  * exact j0_nonempty (h_0 X hX)
  * exact j1_nonempty (h_1 Y hY)
  * exact j2_nonempty (h_2 Z hZ)
end

section
open NeighborhoodSystem

namespace Example61

variable {a : Type*}

section Equation

variable (D : NeighborhoodSystem a) (hD : forall  X, D.mem X  ->  X.Nonempty)

include hD

/-- The right-hand side of Scott's domain equation: the sum system `D + (D^S  x  D^S)`.
     -/
def sumSys : NeighborhoodSystem (Option (a ? ((List Bool  x  a) ? (List Bool  x  a))))
     :=
  sum D (prod (Dsharp D hD) (Dsharp D hD)) hD (prod_dsharp_nonempty D hD)

/-- No neighbourhood of the sum `D + (D^S  x  D^S)` is empty. -/
theorem sumSys_mem_nonempty {W : Set (Option (a ? ((List Bool  x  a) ? (List Bool  x
    a))))}
```

```
    (h : (sumSys D hD).mem W) : W.Nonempty := by
  rcases h with rfl | <X, hX, rfl> | <Y, hY, rfl>
  * exact <none, none_mem_sumMaster>
  * exact inj_0_nonempty (hD X hX)
  * exact inj_1_nonempty (prod_dsharp_nonempty D hD Y hY)

/-- A sum-neighbourhood of the form `0X` arises only from the left factor, so `X  in
    ?`. -/
theorem sum_mem_inj_0_inv {X : Set a} (h : (sumSys D hD).mem (inj_0 X)) : D.mem X :=
    by
  rcases h with h0 | <X', hX', heq> | <Y', hY', heq>
  * exact absurd (h0  |>  none_mem_sumMaster) none_mem_inj_0
  * rw [inj_0_injective heq]; exact hX'
  * obtain <b, hb> := prod_dsharp_nonempty D hD Y' hY'
    exact absurd (heq  |>  ir_mem_inj_1.mpr hb) ir_mem_inj_0

/-- A sum-neighbourhood of the form `1V` arises only from the right factor, so `V  in
    ?^S  x  ?^S`. -/
theorem sum_mem_inj_1_inv {V : Set ((List Bool  x  a) ? (List Bool  x  a))}
    (h : (sumSys D hD).mem (inj_1 V)) : (prod (Dsharp D hD) (Dsharp D hD)).mem V := by
  rcases h with h0 | <X', hX', heq> | <Y', hY', heq>
  * exact absurd (h0  |>  none_mem_sumMaster) none_mem_inj_1
  * obtain <a, ha> := hD X' hX'
    exact absurd (heq  |>  il_mem_inj_0.mpr ha) il_mem_inj_1
  * rw [inj_1_injective heq]; exact hY'

end Equation

end Example61
end

section
universe u
variable {Obj : Type u} [Category Obj] {T : Endofunctor Obj}

/-- For an algebra `A = (D, i)`, the functor turns the structure map into a new `T`-
    algebra
`(T(D), T(i))`. -/
abbrev tStr (A : TAlgebra T) : TAlgebra T where
  carrier := T.obj A.carrier
  str := T.map A.str

/-- The structure map `i : T(D)  ->  D` is itself a homomorphism `(T(D), T(i))  ->  (D,
     i)`: the square
`i  comp  T(i) = i  comp  T(i)` commutes trivially. -/
def strHom (A : TAlgebra T) : AlgHom (tStr A) A where
  hom := A.str
  comm := rfl

/-- The composite `i  comp  j` of `i` with the descent homomorphism `j : (D,i)  ->  (T(
    D),T(i))` is the
identity on `D`. -/
theorem str_comp_desc (A : TAlgebra T) (hA : IsInitial A) :
    A.str ? (hA.desc (tStr A)).hom = (? A.carrier) := by
  have h : (strHom A).comp (hA.desc (tStr A)) = AlgHom.id A := by
    rw [hA.uniq A ((strHom A).comp (hA.desc (tStr A))), hA.uniq A (AlgHom.id A)]
```

```
  exact congrArg AlgHom.hom h

end

section
set_option linter.unusedVariables false

theorem theorem_67.{u, u_1} : forall  {Obj : Type u} [inst : CategoryTheory.Category.{
    u_1, u} Obj]
  {T : @DomainTheory.NeighbourhoodSystems.Endofunctor Obj inst}
  (A : @DomainTheory.NeighbourhoodSystems.TAlgebra Obj inst T)
  (hA : @DomainTheory.NeighbourhoodSystems.IsInitial Obj inst T A),
  @Eq
    (@Quiver.Hom Obj (@CategoryTheory.CategoryStruct.toQuiver Obj (@CategoryTheory.
    Category.toCategoryStruct Obj inst))
      (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier Obj inst T A)
      (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier Obj inst T A))
    (@DomainTheory.NeighbourhoodSystems.compAfter Obj inst
      (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier Obj inst T A)
      (@DomainTheory.NeighbourhoodSystems.Endofunctor.obj Obj inst T
        (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier Obj inst T A))
      (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier Obj inst T A)
      (@DomainTheory.NeighbourhoodSystems.TAlgebra.str Obj inst T A)
      (@DomainTheory.NeighbourhoodSystems.AlgHom.hom Obj inst T A (@DomainTheory.
    NeighbourhoodSystems.tStr Obj inst T A)
        (@DomainTheory.NeighbourhoodSystems.IsInitial.desc Obj inst T A hA
          (@DomainTheory.NeighbourhoodSystems.tStr Obj inst T A))))
    (@CategoryTheory.CategoryStruct.id Obj (@CategoryTheory.Category.toCategoryStruct
    Obj inst)
      (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier Obj inst T A)) :=
  @DomainTheory.NeighbourhoodSystems.str_comp_desc
end

section
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.Exercise510

/-- The strict map represented by `False` of the strict function space relates `X` to `
    Y` exactly when
`X` is a neighbourhood and `Y` is the master output: it is the constant-`False` (least)
     strict map. -/
theorem botStrict_rel {a b : Type*} {V_0 : NeighborhoodSystem a} {V_1 :
    NeighborhoodSystem b}
    {X : Set a} {Y : Set b} :
    (toStrictMap (strictFun V_0 V_1).bot).1.rel X Y  <->  V_0.mem X  /\  Y = V_1.
    master := by
  rw [toStrictMap_rel, NeighborhoodSystem.mem_bot, strictFun_master]
  constructor
  * intro h
    have hcb : (<constMap V_0 V_1.bot, isStrict_constBot> : StrictMap V_0 V_1)  in
    sstep X Y := by
      rw [h]; exact Set.mem_univ _
    rw [mem_sstep, constMap_rel, NeighborhoodSystem.mem_bot] at hcb
    exact hcb
  * rintro <hX, rfl>
    exact sstep_cod_master hX
```

```
/-- **The per-step computation of Theorem 6.9's operator.** For the operator `Op = (k
    comp  *  comp  j)  comp  ?`
on the strict function space, `Op(x)` is the strict map `k  comp  T(toStrictMap x)
    comp  j`. This is the
content of `homOp_apply_filter` plus the defining property `h?` of the Definition 6.8
    witness `?`. -/
theorem opStep (T : Endofunctor DomainObj.{w}) (D E : DomainObj.{w})
    (j : ApproximableMap D.sys (T.obj D).sys) (k : ApproximableMap (T.obj E).sys E.sys
    )
    (hj : IsStrict j) (hk : IsStrict k)
    (? : ApproximableMap (strictFun D.sys E.sys) (strictFun (T.obj D).sys (T.obj E).
    sys))
    (h? : forall  f : StrictMap D.sys E.sys,
      (toStrictMap (?.toElementMap (toStrictFilter f))).1 = T.map (X := D) (Y := E) f
    .1)
    (x : (strictFun D.sys E.sys).Element) :
    (toStrictMap (((homOp T D E j k hj hk).comp ?).toElementMap x)).1
      = k.comp ((T.map (X := D) (Y := E) (toStrictMap x).1).comp j) := by
  set h := toStrictMap x with hh
  have hx : toStrictFilter h = x := toStrictFilter_toStrictMap x
  have hphi : ?.toElementMap x
      = toStrictFilter (toStrictMap (?.toElementMap (toStrictFilter h))) := by
    rw [ <-  hx]; exact (toStrictFilter_toStrictMap _).symm
  rw [toElementMap_comp, hphi, homOp_apply_filter, toStrictMap_toStrictFilter]
  show k.comp ((toStrictMap (?.toElementMap (toStrictFilter h))).1.comp j)
      = k.comp ((T.map (X := D) (Y := E) h.1).comp j)
  rw [h? h]

end

end DomainTheory.NeighbourhoodSystems
end
```

3.7.16 Lemma 6.15 — subdomains from projection pairs

LEMMA 6.15. For two neighbourhood systems D and E, if there exist a projection pair

$$i : D \to E \quad \text{and} \quad j : E \to D$$

with $j \circ i = I_D$ and $i \circ j \subseteq I_E$, then $D \trianglelefteq E$.

Proof. What we want to show is that i maps finite elements to finite elements, and that the desired D' is the image of D in E.

Suppose $X \in D$. We can write:

$$i(\uparrow X) = \bigcup \{\uparrow Y \mid Y \in i(\uparrow X)\} .$$

Applying j to both sides we have:

$$\uparrow X = j \circ i(\uparrow X) = \bigcup \{j(\uparrow Y) \mid Y \in i(\uparrow X)\}.$$

But then, since $X \in \uparrow X$, we find $X \in j(\uparrow Y)$ for some $Y \in i(\uparrow X)$. This implies

$$\uparrow X \subseteq j(\uparrow Y); \text{ and so } i(\uparrow X) \subseteq i \circ j(\uparrow Y) \subseteq \uparrow Y.$$

Since $\uparrow Y \subseteq i(\uparrow X)$ in any case, we conclude $i(\uparrow X) = \uparrow Y$. This proves finite elements are mapped to finite elements.

What of Δ; that is, what is $i(\uparrow \Delta)$? We find, supposing E to be a neighbourhood system over a set Δ', that since $\uparrow \Delta \subseteq j(\uparrow \Delta')$, then $i(\uparrow \Delta) \subseteq \uparrow \Delta'$ and so $i(\uparrow \Delta) = \uparrow \Delta'$. This means that Δ corresponds to Δ'. So we have established that $\mathcal{D}$ is in an inclusion preserving one-one correspondence with a subset $\mathcal{D}'$ of E where $\Delta' \in \mathcal{D}'$. But it remains to show that $\mathcal{D}'$ is a neighbourhood system and that $\mathcal{D}' \lhd E$ holds. All we really have to show is that $\mathcal{D}'$ is closed under intersection whenever the intersection belongs to E.

Suppose $Y', Z' \in \mathcal{D}'$ and $Y' \cap Z' \in E$. Let $X' = Y' \cap Z'$. We have, for suitable $Y, Z \in \mathcal{D}$,

$$i(\uparrow Y) = \uparrow Y', \text{ and so } \uparrow Y = j(\uparrow Y'); \text{ and}$$

$$i(\uparrow Z) = \uparrow Z', \text{ and so } \uparrow Z = j(\uparrow Z').$$

But $\uparrow Y' \subseteq \uparrow X'$ and $j(\uparrow Y') \subseteq j(\uparrow X')$; thus $Y \in j(\uparrow X')$. For similar reasons $Z \in j(\uparrow X')$. But then $X = Y \cap Z \in j(\uparrow X')$, and therefore $Y \cap Z \in \mathcal{D}$. (The element $j(\uparrow X')$ must be a filter.) Notice, however, that

$$\uparrow Y \subseteq \uparrow X, \text{ and so } \uparrow Y' \subseteq i(\uparrow X); \text{ and}$$

$$\uparrow Z \subseteq \uparrow X, \text{ and so } \uparrow Z' \subseteq i(\uparrow X).$$

It follows that $Y' \cap Z' = X' \in i(\uparrow X)$. On the other hand we already knew $X \in j(\uparrow X')$, which implies $i(\uparrow X) \subseteq \uparrow X'$. We may thus conclude that $i(\uparrow X) = \uparrow X'$. In other words $X' \in \mathcal{D}'$. □

Lemma_6_15.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.Exercise_6_29

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap
section
variable {a b : Type*} {D : NeighborhoodSystem a} {E : NeighborhoodSystem b}
section ProjectionPair
variable (i : ApproximableMap D E) (j : ApproximableMap E D)
/- concordance: source L3957 LEMMA 6.15 subdomains from projection pairs -/
/-- **Lemma 6.15 (Scott 1981, PRG-19).** A projection pair `i : D  ->  E`, `j : E  ->
    D` with
`j  comp  i = I_D` and `i  comp  j  subseteq  I_E` exhibits `D` as a subdomain of `E`: `
    D ? E`. This is the converse
of Proposition 6.12 (`Subsystem.projectionPair`). -/
theorem trianglelefteq_of_projectionPair (hji : j.comp i = idMap D)
    (hij : i.comp j  <=  idMap E) : D ? E :=
  <Dprime i j, Dprime_subsystem i j, <dprimeEquiv i j hji hij>>

end ProjectionPair
end
```

```
section
set_option linter.unusedVariables false

theorem theorem_615.{u_1, u_2} : forall  {a : Type u_1} {b : Type u_2} {D :
    DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
  {E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
  (i : @DomainTheory.NeighbourhoodSystems.ApproximableMap a b D E)
  (j : @DomainTheory.NeighbourhoodSystems.ApproximableMap b a E D),
  @Eq (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a D D)
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp a b a D E D j i)
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.idMap a D)  ->
    @LE.le (@DomainTheory.NeighbourhoodSystems.ApproximableMap b b E E)
        (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.ApproximableMap b b E E)
          (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
    ApproximableMap b b E E)
            (@DomainTheory.NeighbourhoodSystems.instPartialOrderApproximableMap b b E
    E)))
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp b a b E D E i j)
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.idMap b E)  ->
      @DomainTheory.NeighbourhoodSystems.Trianglelefteq a b D E :=
  @DomainTheory.NeighbourhoodSystems.trianglelefteq_of_projectionPair
end

end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems
namespace Subsystem
open NeighborhoodSystem ApproximableMap

section
variable {a : Type*}
variable {D E : NeighborhoodSystem a}

/-- **A projection pair (Definition 6.13 vocabulary).** Bundles Scott's `i, j` together
     with the two
laws, ready for reuse in monotone/continuous-on-domains functors and the existence
    Theorem 6.14. -/
structure ProjectionPair (D E : NeighborhoodSystem a) where
  /-- The injection `i : D  ->  E`. -/
  inj : ApproximableMap D E
  /-- The projection `j : E  ->  D`. -/
  proj : ApproximableMap E D
  /-- `j  comp  i = I_D`. -/
  proj_comp_inj : proj.comp inj = idMap D
  /-- `i  comp  j  subseteq  I_E`. -/
  inj_comp_proj_le : inj.comp proj  <=  idMap E
end

end Subsystem
open NeighborhoodSystem ApproximableMap

section
variable {a : Type*}

namespace Subsystem
```

```
variable {D E : NeighborhoodSystem a}

/-- **The injection `i : D  ->  E` of Proposition 6.12.** As a neighbourhood relation,
`X i Y  <->  X  in  D  /\  Y  in  E  /\  X  subseteq  Y`. Element-wise (see `
    toElementMap_inj`) it is Scott's
`i(x) = {Y  in  E ? exists  X  in  x, X  subseteq  Y}`. -/
def inj (h : D ? E) : ApproximableMap D E where
  rel X Y := D.mem X  /\  E.mem Y  /\  X  subseteq  Y
  rel_dom hr := hr.1
  rel_cod hr := hr.2.1
  master_rel := <D.master_mem, E.master_mem, h.master_eq.subset>
  inter_right := by
    rintro X Y Y' <hX, hY, hXY> <_, hY', hXY'>
    exact <hX, E.inter_mem hY hY' (h.sub hX) (Set.subset_inter hXY hXY'),
      Set.subset_inter hXY hXY'>
  mono := by
    rintro X X' Y Y' <_, _, hXY> hX'X hYY' hX' hY'
    exact <hX', hY', (hX'X.trans hXY).trans hYY'>

@[simp] theorem inj_rel (h : D ? E) {X Y : Set a} :
    (h.inj).rel X Y  <->  D.mem X  /\  E.mem Y  /\  X  subseteq  Y := Iff.rfl

/-- **The projection `j : E  ->  D` of Proposition 6.12.** As a neighbourhood relation,
`Y j X  <->  Y  in  E  /\  X  in  D  /\  Y  subseteq  X`. Element-wise (see `
    toElementMap_proj`) it is Scott's
`j(y) = y  I  D`. The `inter_right` law is exactly where Definition 6.10's consistency
    clause
(`inter_closed`) is used. -/
def proj (h : D ? E) : ApproximableMap E D where
  rel Y X := E.mem Y  /\  D.mem X  /\  Y  subseteq  X
  rel_dom hr := hr.1
  rel_cod hr := hr.2.1
  master_rel := <E.master_mem, D.master_mem, h.master_eq.symm.subset>
  inter_right := by
    rintro Y X X' <hY, hX, hYX> <_, hX', hYX'>
    have hEinter : E.mem (X  I  X') :=
      E.inter_mem (h.sub hX) (h.sub hX') hY (Set.subset_inter hYX hYX')
    exact <hY, h.inter_closed hX hX' hEinter, Set.subset_inter hYX hYX'>
  mono := by
    rintro Y Y' X X' <_, _, hYX> hY'Y hXX' hY' hX'
    exact <hY', hX', (hY'Y.trans hYX).trans hXX'>

@[simp] theorem proj_rel (h : D ? E) {Y X : Set a} :
    (h.proj).rel Y X  <->  E.mem Y  /\  D.mem X  /\  Y  subseteq  X := Iff.rfl

/-- **Proposition 6.12, second law: `i  comp  j  subseteq  I_E`.** A round trip `Y
    subseteq  X  subseteq  Y'` through a common
`D`-neighbourhood `X` is in particular `Y  subseteq  Y'` on `E`. The reverse inclusion
    fails (not every
consistent `E`-pair factors through `D`), so this is an inclusion of relations, not an
    equality. -/
theorem inj_comp_proj_le (h : D ? E) : h.inj.comp h.proj  <=  idMap E := by
  intro Y Y' hr
  obtain <X, <hEY, _, hYX>, _, hEY', hXY'> := hr
  exact <hEY, hEY', hYX.trans hXY'>
```

```
/-- **Proposition 6.12, first law: `j  comp  i = I_D`.** Each side relates `X` and `Z`
    exactly when
`X, Z  in  D` and `X  subseteq  Z`: a round trip `X  subseteq  Y  subseteq  Z` through
    an `E`-neighbourhood `Y` collapses to
`X  subseteq  Z` (forward), and `X  subseteq  Z` factors through `Y = Z` (backward).
    Proved relationally, so
**choice-free**. -/
theorem proj_comp_inj (h : D ? E) : h.proj.comp h.inj = idMap D := by
  apply ApproximableMap.ext
  intro X Z
  rw [comp_rel, idMap_rel]
  constructor
  * rintro <Y, <hX, _, hXY>, _, hZ, hYZ>
    exact <hX, hZ, hXY.trans hYZ>
  * rintro <hX, hZ, hXZ>
    exact <Z, <hX, h.sub hZ, hXZ>, h.sub hZ, hZ, subset_rfl>

end Subsystem
end

end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.Subsystem
section
set_option linter.unusedVariables false

theorem theorem_612.{u_1} : And
  (forall  {a : Type u_1} {D E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
    a}
    (h : @DomainTheory.NeighbourhoodSystems.Subsystem a D E),
    @LE.le (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a E E)
      (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a E E)
        (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.ApproximableMap
    a a E E)
          (@DomainTheory.NeighbourhoodSystems.instPartialOrderApproximableMap a a E E)
    ))
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp a a a E D E
        (@DomainTheory.NeighbourhoodSystems.Subsystem.inj a D E h)
        (@DomainTheory.NeighbourhoodSystems.Subsystem.proj a D E h))
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.idMap a E))
  (forall  {a : Type u_1} {D E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
    a}
    (h : @DomainTheory.NeighbourhoodSystems.Subsystem a D E),
    @Eq (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a D D)
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp a a a D E D
        (@DomainTheory.NeighbourhoodSystems.Subsystem.proj a D E h)
        (@DomainTheory.NeighbourhoodSystems.Subsystem.inj a D E h))
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.idMap a D)) :=
  @And.intro
  (forall  {a : Type u_1} {D E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
    a}
    (h : @DomainTheory.NeighbourhoodSystems.Subsystem a D E),
    @LE.le (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a E E)
      (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a E E)
        (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.ApproximableMap
    a a E E)
          (@DomainTheory.NeighbourhoodSystems.instPartialOrderApproximableMap a a E E)
```

```
    ))
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp a a a E D E
        (@DomainTheory.NeighbourhoodSystems.Subsystem.inj a D E h)
        (@DomainTheory.NeighbourhoodSystems.Subsystem.proj a D E h))
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.idMap a E))
  (forall  {a : Type u_1} {D E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
    a}
    (h : @DomainTheory.NeighbourhoodSystems.Subsystem a D E),
    @Eq (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a D D)
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp a a a D E D
        (@DomainTheory.NeighbourhoodSystems.Subsystem.proj a D E h)
        (@DomainTheory.NeighbourhoodSystems.Subsystem.inj a D E h))
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.idMap a D))
  @DomainTheory.NeighbourhoodSystems.Subsystem.inj_comp_proj_le
  @DomainTheory.NeighbourhoodSystems.Subsystem.proj_comp_inj
end

end DomainTheory.NeighbourhoodSystems.Subsystem
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap Subsystem

section
variable {a : Type*} {D E : NeighborhoodSystem a}

namespace Subsystem.ProjectionPair

/-- The set `{x  in  |?| ? g(x)  sqsub  y}` of the first formula. By the Galois
    connection it is the
down-set `{x ? x  sqsub  h(y)}` of `h(y)`. -/
def lowerSet (P : ProjectionPair D E) (y : E.Element) : Set D.Element :=
  {x | P.inj.toElementMap x  <=  y}

@[simp] theorem mem_lowerSet (P : ProjectionPair D E) {y : E.Element} {x : D.Element}
    :
    x  in  P.lowerSet y  <->  P.inj.toElementMap x  <=  y := Iff.rfl

/-- The set `{y  in  |?| ? x  sqsub  h(y)}` of the second formula. By the Galois
    connection it is the
up-set `{y ? g(x)  sqsub  y}` of `g(x)`. -/
def upperSet (P : ProjectionPair D E) (x : D.Element) : Set E.Element :=
  {y | x  <=  P.proj.toElementMap y}

@[simp] theorem mem_upperSet (P : ProjectionPair D E) {x : D.Element} {y : E.Element}
    :
    y  in  P.upperSet x  <->  x  <=  P.proj.toElementMap y := Iff.rfl

end Subsystem.ProjectionPair
end
end DomainTheory.NeighbourhoodSystems
end
```

3.7.17 Theorem 6.16 — initial algebra is minimal

THEOREM 6.16. If on the category of domains and strict approximable maps the functor T is continuous on maps, and if D is an initial T-algebra, then for any system $E \cong T(E)$ we have $D \trianglelefteq E$.

Proof: There is a homomorphism $h : D \to E$. By 6.9 there is a homomorphism $g : E \to D$. Now $g \circ h : D \to D$ is also a homomorphism, so $g \circ h = I_D$ because D is initial. In view of 6.15, all we have to prove now is that $h \circ g \sqsubseteq I_E$.

Let the maps $i : T(D) \to D$ and $j : D \to T(D)$ give the isomorphism for D, and let $u : T(E) \to E$ and $v : E \to T(E)$ do the same for E. By the proof of 6.9 we know

$$g = i \circ T(g) \circ v \quad \text{and} \quad h = u \circ T(h) \circ j$$

and each of these maps is the least fixed point of its respective equation. Let

$$g_0 = \bot_{E \to D} \quad \text{and} \quad h_0 = \bot_{D \to E}$$

and define by recursion

$$g_{n+1} = i \circ T(g_n) \circ v \quad \text{and} \quad h_{n+1} = u \circ T(h_n) \circ j.$$

By the fixed-point calculation

$$g = \bigsqcup_{n=0}^{\infty} g_n \quad \text{and} \quad h = \bigsqcup_{n=0}^{\infty} h_n.$$

Now we see that

$$h_0 \circ g_0 = \bot_{E \to E},$$

and for each n that

$$\begin{aligned} h_{n+1} \circ g_{n+1} &= u \circ T(h_n) \circ j \circ i \circ T(g_n) \circ v \\ &= u \circ T(h_n) \circ T(g_n) \circ v \\ &= u \circ T(h_n \circ g_n) \circ v. \end{aligned}$$

But this means that

$$h \circ g = \bigsqcup_{n=0}^{\infty} (h_n \circ g_n)$$

is the least fixed point for the equation

$$k = u \circ T(k) \circ v.$$

But I_E is one of the fixed points; whence $h \circ g \sqsubseteq I_E$ must follow. □

Theorem_6_16.lean

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Exercise_4_7

public import DomainTheory.NeighbourhoodSystems.DomainEquations.Proposition_6_7
public import DomainTheory.NeighbourhoodSystems.DomainEquations.Theorem_6_9

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.Exercise510
universe w
/- concordance: source L4015 THEOREM 6.16 initial algebra is minimal -/
/-- **Theorem 6.16 (Scott 1981, PRG-19).** If `T` is continuous on maps and `D` is an
    initial
`T`-algebra, then for any system `E  ~=  T(E)` we have `D ? E`: the initial algebra
    embeds as a
subdomain of every solution of the domain equation. -/
theorem trianglelefteq_of_isInitial
    (T : Endofunctor DomainObj.{w}) (hT : ContinuousOnMaps T)
    (Dalg : TAlgebra T) (hinit : IsInitial Dalg)
    (E : DomainObj.{w}) (isoE : Iso (T.obj E) E) :
    Dalg.carrier.sys ? E.sys := by
  -- Lambek (Proposition 6.7): the structure map of `D` is an isomorphism `i : T(D)  ~=
     D`.
  let isoD : Iso (T.obj Dalg.carrier) Dalg.carrier := lambek Dalg hinit
  -- strictness of the four structure maps (each is a split iso, hence preserves `False
    `).
  have hi : IsStrict isoD.hom := isStrict_of_comp_eq_id isoD.inv_hom_id
  have hj : IsStrict isoD.inv := isStrict_of_comp_eq_id isoD.hom_inv_id
  have hu : IsStrict isoE.hom := isStrict_of_comp_eq_id isoE.inv_hom_id
  have hv : IsStrict isoE.inv := isStrict_of_comp_eq_id isoE.hom_inv_id
  -- the iso laws in `.comp` form.
  have hji : isoD.inv.comp isoD.hom = idMap (T.obj Dalg.carrier).sys := isoD.
    hom_inv_id
  have hvu : isoE.inv.comp isoE.hom = idMap (T.obj E).sys := isoE.hom_inv_id
  have huv : isoE.hom.comp isoE.inv = idMap E.sys := isoE.inv_hom_id
  -- Definition 6.8 witnesses that `fun f. T(f)` is approximable on each strict
    function space.
  obtain <?DE, h?DE> := hT Dalg.carrier E
  obtain <?ED, h?ED> := hT E Dalg.carrier
  obtain <?EE, h?EE> := hT E E
  -- the three operators (Theorem 6.9's `fun f. k  comp  T(f)  comp  j`) for `h`, `g`,
    `k`.
  let Oph := (homOp T Dalg.carrier E isoD.inv isoE.hom hj hu).comp ?DE
  let Opg := (homOp T E Dalg.carrier isoE.inv isoD.hom hv hi).comp ?ED
  let Opk := (homOp T E E isoE.inv isoE.hom hv hu).comp ?EE
  -- the approximant chains.
  let H : Nat  ->  ApproximableMap Dalg.carrier.sys E.sys := fun n => (toStrictMap (
    Oph.iterElem n)).1
  let G : Nat  ->  ApproximableMap E.sys Dalg.carrier.sys := fun n => (toStrictMap (
    Opg.iterElem n)).1
  let K : Nat  ->  ApproximableMap E.sys E.sys := fun n => (toStrictMap (Opk.iterElem
    n)).1
```

```
-- `iterElem 0 = False`.
have iterElem_zero : forall  {? : Type w} {V : NeighborhoodSystem ?} (f :
  ApproximableMap V V),
    f.iterElem 0 = V.bot := by
  intro ? V f
  show (f.iterMap 0).toElementMap V.bot = V.bot
  rw [iterMap_zero, toElementMap_idMap]
-- the recursion equations `h_n_+_1 = u  comp  T(h_n)  comp  j`, etc.
have H_succ : forall  n, H (n + 1)
    = isoE.hom.comp ((T.map (X := Dalg.carrier) (Y := E) (H n)).comp isoD.inv) := by
  intro n
  show (toStrictMap (Oph.iterElem (n + 1))).1 = _
  rw [iterElem_succ]
  exact opStep T Dalg.carrier E isoD.inv isoE.hom hj hu ?DE h?DE (Oph.iterElem n)
have G_succ : forall  n, G (n + 1)
    = isoD.hom.comp ((T.map (X := E) (Y := Dalg.carrier) (G n)).comp isoE.inv) := by
  intro n
  show (toStrictMap (Opg.iterElem (n + 1))).1 = _
  rw [iterElem_succ]
  exact opStep T E Dalg.carrier isoE.inv isoD.hom hv hi ?ED h?ED (Opg.iterElem n)
have K_succ : forall  n, K (n + 1)
    = isoE.hom.comp ((T.map (X := E) (Y := E) (K n)).comp isoE.inv) := by
  intro n
  show (toStrictMap (Opk.iterElem (n + 1))).1 = _
  rw [iterElem_succ]
  exact opStep T E E isoE.inv isoE.hom hv hu ?EE h?EE (Opk.iterElem n)
-- monotonicity of the chains.
have H_mono : forall  {n m : Nat}, n  <=  m  ->  H n  <=  H m := fun hnm =>
  toStrictMap_mono (iterElem_mono Oph hnm)
have G_mono : forall  {n m : Nat}, n  <=  m  ->  G n  <=  G m := fun hnm =>
  toStrictMap_mono (iterElem_mono Opg hnm)
-- the algebraic core: `(u  comp  a  comp  j)  comp  (i  comp  b  comp  v) = u  comp
   (a  comp  b)  comp  v` (uses `j  comp  i = I`).
have key : forall  (a : ApproximableMap (T.obj Dalg.carrier).sys (T.obj E).sys)
    (b : ApproximableMap (T.obj E).sys (T.obj Dalg.carrier).sys),
    (isoE.hom.comp (a.comp isoD.inv)).comp (isoD.hom.comp (b.comp isoE.inv))
      = isoE.hom.comp ((a.comp b).comp isoE.inv) := by
  intro a b
  rw [comp_assoc isoE.hom (a.comp isoD.inv) (isoD.hom.comp (b.comp isoE.inv)),
      comp_assoc a isoD.inv (isoD.hom.comp (b.comp isoE.inv)),
       <-  comp_assoc isoD.inv isoD.hom (b.comp isoE.inv), hji, idMap_comp,
       <-  comp_assoc a b isoE.inv]
-- functoriality `T(p)  comp  T(q) = T(p  comp  q)` in `.comp` form.
have hTcomp : forall  (p : ApproximableMap Dalg.carrier.sys E.sys)
    (q : ApproximableMap E.sys Dalg.carrier.sys),
    (T.map (X := Dalg.carrier) (Y := E) p).comp (T.map (X := E) (Y := Dalg.carrier)
  q)
      = T.map (X := E) (Y := E) (p.comp q) :=
  fun p q => (T.map_comp (X := E) (Y := Dalg.carrier) (Z := E) p q).symm
-- **the ladder**: `h_n  comp  g_n = k_n`.
have ladder : forall  n, (H n).comp (G n) = K n := by
  intro n
  induction n with
  | zero =>
    apply ApproximableMap.ext
    intro X Z
```

```
    have hH0 : forall  {P Q}, (H 0).rel P Q  <->  Dalg.carrier.sys.mem P  /\  Q = E.
  sys.master := by
      intro P Q
      show (toStrictMap (Oph.iterElem 0)).1.rel P Q  <->  _
      rw [iterElem_zero]; exact botStrict_rel
    have hG0 : forall  {P Q}, (G 0).rel P Q  <->  E.sys.mem P  /\  Q = Dalg.carrier.
  sys.master := by
      intro P Q
      show (toStrictMap (Opg.iterElem 0)).1.rel P Q  <->  _
      rw [iterElem_zero]; exact botStrict_rel
    have hK0 : forall  {P Q}, (K 0).rel P Q  <->  E.sys.mem P  /\  Q = E.sys.master
  := by
      intro P Q
      show (toStrictMap (Opk.iterElem 0)).1.rel P Q  <->  _
      rw [iterElem_zero]; exact botStrict_rel
    constructor
    * rintro <Y, hG, hHr>
      rw [hG0] at hG; rw [hH0] at hHr
      obtain <hEX, rfl> := hG
      obtain <_, rfl> := hHr
      rw [hK0]; exact <hEX, rfl>
    * intro hK
      rw [hK0] at hK
      obtain <hEX, rfl> := hK
      exact <Dalg.carrier.sys.master,
        (hG0).mpr <hEX, rfl>, (hH0).mpr <Dalg.carrier.sys.master_mem, rfl>>
  | succ n ih =>
    rw [H_succ n, G_succ n, key, K_succ n, hTcomp (H n) (G n), ih]
-- the fixed-point maps and their ` sqsup `-decomposition.
let hh := (toStrictMap Oph.fixElement).1
let gg := (toStrictMap Opg.fixElement).1
let kk := (toStrictMap Opk.fixElement).1
have H_fix_rel : forall  X Y, hh.rel X Y  <->  exists  n, (H n).rel X Y := by
  intro X Y
  show (toStrictMap Oph.fixElement).1.rel X Y  <->  _
  rw [toStrictMap_rel, Oph.fixElement_eq_iSupDirected, NeighborhoodSystem.
  mem_iSupDirected]
  constructor
  * rintro <n, hn>; exact <n, hn>
  * rintro <n, hn>; exact <n, hn>
have G_fix_rel : forall  X Y, gg.rel X Y  <->  exists  n, (G n).rel X Y := by
  intro X Y
  show (toStrictMap Opg.fixElement).1.rel X Y  <->  _
  rw [toStrictMap_rel, Opg.fixElement_eq_iSupDirected, NeighborhoodSystem.
  mem_iSupDirected]
  constructor
  * rintro <n, hn>; exact <n, hn>
  * rintro <n, hn>; exact <n, hn>
have K_fix_rel : forall  X Y, kk.rel X Y  <->  exists  n, (K n).rel X Y := by
  intro X Y
  show (toStrictMap Opk.fixElement).1.rel X Y  <->  _
  rw [toStrictMap_rel, Opk.fixElement_eq_iSupDirected, NeighborhoodSystem.
  mem_iSupDirected]
  constructor
  * rintro <n, hn>; exact <n, hn>
  * rintro <n, hn>; exact <n, hn>
```

```
-- `h  comp  g = k` (the diagonal of the doubly-indexed directed family, via the
  ladder).
have hgk : hh.comp gg = kk := by
  apply ApproximableMap.ext
  intro X Z
  constructor
  * rintro <Y, hgXY, hhYZ>
    rw [G_fix_rel] at hgXY
    rw [H_fix_rel] at hhYZ
    obtain <m, hm> := hgXY
    obtain <n, hn> := hhYZ
    rw [K_fix_rel]
    refine <max m n, ?_>
    rw [ <-  ladder (max m n)]
    exact <Y, G_mono (le_max_left m n) X Y hm, H_mono (le_max_right m n) Y Z hn>
  * intro hk
    rw [K_fix_rel] at hk
    obtain <p, hp> := hk
    rw [ <-  ladder p] at hp
    obtain <Y, hG, hHr> := hp
    exact <Y, (G_fix_rel X Y).mpr <p, hG>, (H_fix_rel Y Z).mpr <p, hHr>>
-- `k  sqsub  I_E`, because `I_E` is a fixed point of `k  |->  u  comp  T(k)  comp  v
  `.
have hk_le : kk  <=  idMap E.sys := by
  have hstepeq : (toStrictMap (Opk.toElementMap
      (toStrictFilter (<idMap E.sys, isStrict_idMap> : StrictMap E.sys E.sys)))).1
        = idMap E.sys := by
    have hs := opStep T E E isoE.inv isoE.hom hv hu ?EE h?EE
      (toStrictFilter (<idMap E.sys, isStrict_idMap> : StrictMap E.sys E.sys))
    have hmapid : T.map (X := E) (Y := E) (idMap E.sys) = idMap (T.obj E).sys := T.
  map_id E
    rw [hs, show (toStrictMap (toStrictFilter
        (<idMap E.sys, isStrict_idMap> : StrictMap E.sys E.sys))).1 = idMap E.sys
  from
        congrArg Subtype.val (toStrictMap_toStrictFilter _),
      hmapid, idMap_comp, huv]
  have hfp : Opk.toElementMap (toStrictFilter (<idMap E.sys, isStrict_idMap> :
  StrictMap E.sys E.sys))
      = toStrictFilter (<idMap E.sys, isStrict_idMap> : StrictMap E.sys E.sys) := by
    calc Opk.toElementMap (toStrictFilter <idMap E.sys, isStrict_idMap>)
        = toStrictFilter (toStrictMap (Opk.toElementMap
            (toStrictFilter <idMap E.sys, isStrict_idMap>))) :=
          (toStrictFilter_toStrictMap _).symm
      _ = toStrictFilter (<idMap E.sys, isStrict_idMap> : StrictMap E.sys E.sys) :=
  by
          congr 1
          apply Subtype.ext
          rw [hstepeq]
      _ = toStrictFilter <idMap E.sys, isStrict_idMap> := rfl
  have hle : Opk.fixElement  <=  toStrictFilter (<idMap E.sys, isStrict_idMap> :
  StrictMap E.sys E.sys) :=
    fixElement_le_of_toElementMap_le Opk (le_of_eq hfp)
  have hmono := toStrictMap_mono hle
  show (toStrictMap Opk.fixElement).1  <=  idMap E.sys
  refine le_of_le_of_eq hmono ?_
  exact congrArg Subtype.val (toStrictMap_toStrictFilter _)
```

```
  -- `h` and `g` are algebra homomorphisms.
  let Balg : TAlgebra T := <E, isoE.hom>
  have h_fixeq : hh = isoE.hom.comp ((T.map (X := Dalg.carrier) (Y := E) hh).comp isoD
    .inv) := by
    have hs := opStep T Dalg.carrier E isoD.inv isoE.hom hj hu ?DE h?DE Oph.fixElement
    rw [toElementMap_fixElement] at hs
    exact hs
  have h_comm : hh.comp isoD.hom = isoE.hom.comp (T.map (X := Dalg.carrier) (Y := E)
    hh) := by
    conv_lhs => rw [h_fixeq]
    rw [comp_assoc, comp_assoc, hji, comp_idMap]
  let h_alg : AlgHom Dalg Balg := { hom := hh, comm := h_comm }
  have g_fixeq : gg = isoD.hom.comp ((T.map (X := E) (Y := Dalg.carrier) gg).comp isoE
    .inv) := by
    have hs := opStep T E Dalg.carrier isoE.inv isoD.hom hv hi ?ED h?ED Opg.fixElement
    rw [toElementMap_fixElement] at hs
    exact hs
  have g_comm : gg.comp isoE.hom = isoD.hom.comp (T.map (X := E) (Y := Dalg.carrier)
    gg) := by
    conv_lhs => rw [g_fixeq]
    rw [comp_assoc, comp_assoc, hvu, comp_idMap]
  let g_alg : AlgHom Balg Dalg := { hom := gg, comm := g_comm }
  -- `g  comp  h = I_D` by initiality of `D`.
  have hgh_id : gg.comp hh = idMap Dalg.carrier.sys := by
    have huniq : g_alg.comp h_alg = AlgHom.id Dalg := by
      rw [hinit.uniq Dalg (g_alg.comp h_alg), hinit.uniq Dalg (AlgHom.id Dalg)]
    exact congrArg AlgHom.hom huniq
  -- conclude via Lemma 6.15.
  exact trianglelefteq_of_projectionPair hh gg hgh_id (le_of_eq_of_le hgk hk_le)
end

section
set_option linter.unusedVariables false

theorem theorem_616.{w} : forall
  (T :
    @DomainTheory.NeighbourhoodSystems.Endofunctor.{w + 1, w} DomainTheory.
    NeighbourhoodSystems.DomainObj.{w}
      DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w}),
  DomainTheory.NeighbourhoodSystems.ContinuousOnMaps.{w} T  ->
    forall
      (Dalg :
        @DomainTheory.NeighbourhoodSystems.TAlgebra.{w + 1, w} DomainTheory.
    NeighbourhoodSystems.DomainObj.{w}
          DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} T)
      (hinit :
        @DomainTheory.NeighbourhoodSystems.IsInitial.{w + 1, w} DomainTheory.
    NeighbourhoodSystems.DomainObj.{w}
          DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} T Dalg)
      (E : DomainTheory.NeighbourhoodSystems.DomainObj.{w})
      (isoE :
        @DomainTheory.NeighbourhoodSystems.Iso.{w + 1, w} DomainTheory.
    NeighbourhoodSystems.DomainObj.{w}
          DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w}
          (@DomainTheory.NeighbourhoodSystems.Endofunctor.obj.{w + 1, w} DomainTheory.
    NeighbourhoodSystems.DomainObj.{w}
```

```
            DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} T E)
          E),
      @DomainTheory.NeighbourhoodSystems.Trianglelefteq.{w, w}
        (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{w + 1, w} DomainTheory.
    NeighbourhoodSystems.DomainObj.{w}
            DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} T Dalg).
    carrier
        E.carrier
        (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier.{w + 1, w} DomainTheory.
    NeighbourhoodSystems.DomainObj.{w}
            DomainTheory.NeighbourhoodSystems.instCategoryDomainObj.{w} T Dalg).sys
        E.sys :=
  DomainTheory.NeighbourhoodSystems.trianglelefteq_of_isInitial.{w}
end

section
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.Exercise510

/-- **The functor `T(X) = ? + X + X` on objects.** Over `D`, the system is the genuine
    three-way
separated sum `? + D + D` (Example 6.2's `sum3`, with `? = unitSys`), again `{}`-free
    by
`sum3_nonempty`. -/
abbrev tcObj (D : StrictDomainObj.{w}) : StrictDomainObj.{w} where
  carrier := Option (Unit ? D.carrier ? D.carrier)
  sys := sum3 unitSys D.sys D.sys Example62C.unitSys_nonempty D.nonempty D.nonempty
  nonempty := sum3_nonempty

@[simp] theorem tcObj_sys (D : StrictDomainObj.{w}) :
    (tcObj D).sys = sum3 unitSys D.sys D.sys Example62C.unitSys_nonempty D.nonempty D.
    nonempty := rfl

section ShapeLemmas

open Example62C

variable {a b ? : Type*} {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
  {V_2 : NeighborhoodSystem ?} {h_0 : forall  X, V_0.mem X  ->  X.Nonempty}
  {h_1 : forall  Y, V_1.mem Y  ->  Y.Nonempty} {h_2 : forall  Z, V_2.mem Z  ->  Z.
    Nonempty}

/-- A `sum3`-neighbourhood contained in a `0`-copy `0X` is itself a `0`-copy. -/
theorem mem_subset_j0_inv {W : Set (Option (a ? b ? ?))} {X : Set a}
    (hW : (sum3 V_0 V_1 V_2 h_0 h_1 h_2).mem W) (hsub : W  subseteq  j0 X) :
    exists  X_2, V_0.mem X_2  /\  W = j0 X_2 := by
  rcases hW with rfl | <X_2, hX_2, rfl> | <Y_2, hY_2, rfl> | <Z_2, hZ_2, rfl>
  * exact absurd (hsub none_mem_master3) none_not_mem_j0
  * exact <X_2, hX_2, rfl>
  * obtain <b, hb> := h_1 Y_2 hY_2; exact absurd (hsub (t1_mem_j1.mpr hb))
    t1_not_mem_j0
  * obtain <c, hc> := h_2 Z_2 hZ_2; exact absurd (hsub (t2_mem_j2.mpr hc))
    t2_not_mem_j0

/-- A `sum3`-neighbourhood contained in a `1`-copy `1Y` is itself a `1`-copy. -/
theorem mem_subset_j1_inv {W : Set (Option (a ? b ? ?))} {Y : Set b}
    (hW : (sum3 V_0 V_1 V_2 h_0 h_1 h_2).mem W) (hsub : W  subseteq  j1 Y) :
```

```
    exists  Y_2, V_1.mem Y_2  /\  W = j1 Y_2 := by
  rcases hW with rfl | <X_2, hX_2, rfl> | <Y_2, hY_2, rfl> | <Z_2, hZ_2, rfl>
  * exact absurd (hsub none_mem_master3) none_not_mem_j1
  * obtain <a, ha> := h_0 X_2 hX_2; exact absurd (hsub (t0_mem_j0.mpr ha))
    t0_not_mem_j1
  * exact <Y_2, hY_2, rfl>
  * obtain <c, hc> := h_2 Z_2 hZ_2; exact absurd (hsub (t2_mem_j2.mpr hc))
    t2_not_mem_j1

/-- A `sum3`-neighbourhood contained in a `2`-copy `2Z` is itself a `2`-copy. -/
theorem mem_subset_j2_inv {W : Set (Option (a ? b ? ?))} {Z : Set ?}
    (hW : (sum3 V_0 V_1 V_2 h_0 h_1 h_2).mem W) (hsub : W  subseteq  j2 Z) :
    exists  Z_2, V_2.mem Z_2  /\  W = j2 Z_2 := by
  rcases hW with rfl | <X_2, hX_2, rfl> | <Y_2, hY_2, rfl> | <Z_2, hZ_2, rfl>
  * exact absurd (hsub none_mem_master3) none_not_mem_j2
  * obtain <a, ha> := h_0 X_2 hX_2; exact absurd (hsub (t0_mem_j0.mpr ha))
    t0_not_mem_j2
  * obtain <b, hb> := h_1 Y_2 hY_2; exact absurd (hsub (t1_mem_j1.mpr hb))
    t1_not_mem_j2
  * exact <Z_2, hZ_2, rfl>

end ShapeLemmas

section SumInj

open Example62C

variable {a b ? : Type*}
  {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b} {V_2 : NeighborhoodSystem
    ?}
  {h_0 : forall  X, V_0.mem X  ->  X.Nonempty} {h_1 : forall  Y, V_1.mem Y  ->  Y.
    Nonempty}
  {h_2 : forall  Z, V_2.mem Z  ->  Z.Nonempty}

/-- The `0`-injection `D_0 ? D_0+D_1+D_2`: send `x_0 in |D_0|` to the sum element whose
     only proper
neighbourhoods are the `0`-copies `0X` with `X in x_0`. -/
def sinj0 (x_0 : V_0.Element) : (sum3 V_0 V_1 V_2 h_0 h_1 h_2).Element where
  mem W := W = master3 V_0 V_1 V_2  \/  exists  X, V_0.mem X  /\  W = j0 X  /\  x_0.
    mem X
  sub := by
    rintro W (rfl | <X, hX, rfl, ->)
    * exact Or.inl rfl
    * exact Or.inr (Or.inl <X, hX, rfl>)
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | <X, hX, rfl, hx>) (rfl | <X', hX', rfl, hx'>)
    * exact Or.inl (by rw [Set.inter_self])
    * exact Or.inr <X', hX', by rw [master3_inter_j0 hX'], hx'>
    * exact Or.inr <X, hX, by rw [Set.inter_comm, master3_inter_j0 hX], hx>
    * exact Or.inr <X  I  X', x_0.sub (x_0.inter_mem hx hx'), j0_inter_j0 X X', x_0.
    inter_mem hx hx'>
  up_mem := by
    rintro W W' (rfl | <X, hX, rfl, hx>) hW' hsub
    * exact Or.inl (eq_master3_of_subset hsub ((sum3 V_0 V_1 V_2 h_0 h_1 h_2).
    sub_master hW'))
```

```
    * rcases hW' with rfl | <X', hX', rfl> | <Y', hY', rfl> | <Z', hZ', rfl>
      * exact Or.inl rfl
      * exact Or.inr <X', hX', rfl, x_0.up_mem hx hX' (j0_subset_j0.mp hsub)>
      * obtain <a, ha> := h_0 X (x_0.sub hx); exact absurd (hsub (t0_mem_j0.mpr ha))
    t0_not_mem_j1
      * obtain <a, ha> := h_0 X (x_0.sub hx); exact absurd (hsub (t0_mem_j0.mpr ha))
    t0_not_mem_j2

@[simp] theorem sinj0_mem_j0 {x_0 : V_0.Element} {X : Set a} (hX : V_0.mem X) :
    (sinj0 (V_1 := V_1) (V_2 := V_2) (h_0 := h_0) (h_1 := h_1) (h_2 := h_2) x_0).mem (
    j0 X)  <->  x_0.mem X := by
  constructor
  * rintro (h0 | <X', hX', heq, hx>)
    * exact absurd (h0  |>  none_mem_master3) none_not_mem_j0
    * rw [j0_injective heq]; exact hx
  * intro hx; exact Or.inr <X, hX, rfl, hx>

/-- The `1`-injection `D_1 ? D_0+D_1+D_2`. -/
def sinj1 (x_1 : V_1.Element) : (sum3 V_0 V_1 V_2 h_0 h_1 h_2).Element where
  mem W := W = master3 V_0 V_1 V_2  \/  exists  Y, V_1.mem Y  /\  W = j1 Y  /\  x_1.
    mem Y
  sub := by
    rintro W (rfl | <Y, hY, rfl, ->)
    * exact Or.inl rfl
    * exact Or.inr (Or.inr (Or.inl <Y, hY, rfl>))
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | <Y, hY, rfl, hx>) (rfl | <Y', hY', rfl, hx'>)
    * exact Or.inl (by rw [Set.inter_self])
    * exact Or.inr <Y', hY', by rw [master3_inter_j1 hY'], hx'>
    * exact Or.inr <Y, hY, by rw [Set.inter_comm, master3_inter_j1 hY], hx>
    * exact Or.inr <Y  I  Y', x_1.sub (x_1.inter_mem hx hx'), j1_inter_j1 Y Y', x_1.
    inter_mem hx hx'>
  up_mem := by
    rintro W W' (rfl | <Y, hY, rfl, hx>) hW' hsub
    * exact Or.inl (eq_master3_of_subset hsub ((sum3 V_0 V_1 V_2 h_0 h_1 h_2).
    sub_master hW'))
    * rcases hW' with rfl | <X', hX', rfl> | <Y', hY', rfl> | <Z', hZ', rfl>
      * exact Or.inl rfl
      * obtain <b, hb> := h_1 Y (x_1.sub hx); exact absurd (hsub (t1_mem_j1.mpr hb))
    t1_not_mem_j0
      * exact Or.inr <Y', hY', rfl, x_1.up_mem hx hY' (j1_subset_j1.mp hsub)>
      * obtain <b, hb> := h_1 Y (x_1.sub hx); exact absurd (hsub (t1_mem_j1.mpr hb))
    t1_not_mem_j2

@[simp] theorem sinj1_mem_j1 {x_1 : V_1.Element} {Y : Set b} (hY : V_1.mem Y) :
    (sinj1 (V_0 := V_0) (V_2 := V_2) (h_0 := h_0) (h_1 := h_1) (h_2 := h_2) x_1).mem (
    j1 Y)  <->  x_1.mem Y := by
  constructor
  * rintro (h0 | <Y', hY', heq, hx>)
    * exact absurd (h0  |>  none_mem_master3) none_not_mem_j1
    * rw [j1_injective heq]; exact hx
  * intro hx; exact Or.inr <Y, hY, rfl, hx>

/-- The `2`-injection `D_2 ? D_0+D_1+D_2`. -/
def sinj2 (x_2 : V_2.Element) : (sum3 V_0 V_1 V_2 h_0 h_1 h_2).Element where
```

```
  mem W := W = master3 V_0 V_1 V_2  \/  exists  Z, V_2.mem Z  /\  W = j2 Z  /\  x_2.
    mem Z
  sub := by
    rintro W (rfl | <Z, hZ, rfl, ->)
    * exact Or.inl rfl
    * exact Or.inr (Or.inr (Or.inr <Z, hZ, rfl>))
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | <Z, hZ, rfl, hx>) (rfl | <Z', hZ', rfl, hx'>)
    * exact Or.inl (by rw [Set.inter_self])
    * exact Or.inr <Z', hZ', by rw [master3_inter_j2 hZ'], hx'>
    * exact Or.inr <Z, hZ, by rw [Set.inter_comm, master3_inter_j2 hZ], hx>
    * exact Or.inr <Z  I  Z', x_2.sub (x_2.inter_mem hx hx'), j2_inter_j2 Z Z', x_2.
    inter_mem hx hx'>
  up_mem := by
    rintro W W' (rfl | <Z, hZ, rfl, hx>) hW' hsub
    * exact Or.inl (eq_master3_of_subset hsub ((sum3 V_0 V_1 V_2 h_0 h_1 h_2).
    sub_master hW'))


    * rcases hW' with rfl | <X', hX', rfl> | <Y', hY', rfl> | <Z', hZ', rfl>
      * exact Or.inl rfl
      * obtain <c, hc> := h_2 Z (x_2.sub hx); exact absurd (hsub (t2_mem_j2.mpr hc))
    t2_not_mem_j0
      * obtain <c, hc> := h_2 Z (x_2.sub hx); exact absurd (hsub (t2_mem_j2.mpr hc))
    t2_not_mem_j1
      * exact Or.inr <Z', hZ', rfl, x_2.up_mem hx hZ' (j2_subset_j2.mp hsub)>

@[simp] theorem sinj2_mem_j2 {x_2 : V_2.Element} {Z : Set ?} (hZ : V_2.mem Z) :
    (sinj2 (V_0 := V_0) (V_1 := V_1) (h_0 := h_0) (h_1 := h_1) (h_2 := h_2) x_2).mem (
    j2 Z)  <->  x_2.mem Z := by
  constructor
  * rintro (h0 | <Z', hZ', heq, hx>)
    * exact absurd (h0  |>  none_mem_master3) none_not_mem_j2
    * rw [j2_injective heq]; exact hx
  * intro hx; exact Or.inr <Z, hZ, rfl, hx>

end SumInj

section SumInjMap

open Example62C

variable {a b ? a' b' ?' : Type*}
  {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b} {V_2 : NeighborhoodSystem
    ?}
  {V_0' : NeighborhoodSystem a'} {V_1' : NeighborhoodSystem b'} {V_2' :
    NeighborhoodSystem ?'}
  {h_0 : forall  X, V_0.mem X  ->  X.Nonempty} {h_1 : forall  Y, V_1.mem Y  ->  Y.
    Nonempty}
  {h_2 : forall  Z, V_2.mem Z  ->  Z.Nonempty}
  {h_0' : forall  X, V_0'.mem X  ->  X.Nonempty} {h_1' : forall  Y, V_1'.mem Y  ->  Y.
    Nonempty}
  {h_2' : forall  Z, V_2'.mem Z  ->  Z.Nonempty}

theorem sinj1_mono {x x' : V_1.Element} (hx : x  <=  x') :
    sinj1 (V_0 := V_0) (V_2 := V_2) (h_0 := h_0) (h_1 := h_1) (h_2 := h_2) x  <=
```

```
    sinj1 x' := by
  rintro W (rfl | <Y, hY, rfl, hm>)
  * exact Or.inl rfl
  * exact Or.inr <Y, hY, rfl, hx Y hm>

theorem sinj2_mono {x x' : V_2.Element} (hx : x  <=  x') :
    sinj2 (V_0 := V_0) (V_1 := V_1) (h_0 := h_0) (h_1 := h_1) (h_2 := h_2) x  <=
    sinj2 x' := by
  rintro W (rfl | <Z, hZ, rfl, hm>)
  * exact Or.inl rfl
  * exact Or.inr <Z, hZ, rfl, hx Z hm>

end SumInjMap

section SumMap3

open Example62C

variable {a b ? a' b' ?' : Type*}
  {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b} {V_2 : NeighborhoodSystem
    ?}
  {V_0' : NeighborhoodSystem a'} {V_1' : NeighborhoodSystem b'} {V_2' :
    NeighborhoodSystem ?'}
  {h_0 : forall  X, V_0.mem X  ->  X.Nonempty} {h_1 : forall  Y, V_1.mem Y  ->  Y.
    Nonempty}
  {h_2 : forall  Z, V_2.mem Z  ->  Z.Nonempty}
  {h_0' : forall  X, V_0'.mem X  ->  X.Nonempty} {h_1' : forall  Y, V_1'.mem Y  ->  Y.
    Nonempty}
  {h_2' : forall  Z, V_2'.mem Z  ->  Z.Nonempty}

/-- **The three-way sum map `f_0 + f_1 + f_2 : ?_0+?_1+?_2  ->  ?_0'+?_1'+?_2'`.**
    Routes each tagged copy `iX`
through `f_i` (to `iY_i'`), and sends everything to the codomain master. (The three-way
    analogue of
Exercise 3.19's `sumMap`.) -/
def sumMap3 (f_0 : ApproximableMap V_0 V_0') (f_1 : ApproximableMap V_1 V_1')
    (f_2 : ApproximableMap V_2 V_2') :
    ApproximableMap (sum3 V_0 V_1 V_2 h_0 h_1 h_2) (sum3 V_0' V_1' V_2' h_0' h_1' h_2
    ') where
  rel W W' := (sum3 V_0 V_1 V_2 h_0 h_1 h_2).mem W  /\  (sum3 V_0' V_1' V_2' h_0' h_1'
    h_2').mem W'  /\
    (W' = master3 V_0' V_1' V_2'  \/
      (exists  X Y', W = j0 X  /\  W' = j0 Y'  /\  f_0.rel X Y')  \/
      (exists  X Y', W = j1 X  /\  W' = j1 Y'  /\  f_1.rel X Y')  \/
      (exists  X Y', W = j2 X  /\  W' = j2 Y'  /\  f_2.rel X Y'))
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := <(sum3 V_0 V_1 V_2 h_0 h_1 h_2).master_mem, (sum3 V_0' V_1' V_2' h_0'
    h_1' h_2').master_mem,
    Or.inl rfl>
  inter_right := by
    rintro W W'_1 W'_2 <hW, hW'_1, hd_1> <_, hW'_2, hd_2>
    have hmem : forall  W'' : Set (Option (a' ? b' ? ?')),
        (W'' = master3 V_0' V_1' V_2'  \/
          (exists  X Y', W = j0 X  /\  W'' = j0 Y'  /\  f_0.rel X Y')  \/
          (exists  X Y', W = j1 X  /\  W'' = j1 Y'  /\  f_1.rel X Y')  \/
```

```
      (exists  X Y', W = j2 X  /\  W'' = j2 Y'  /\  f_2.rel X Y'))  ->
      (sum3 V_0' V_1' V_2' h_0' h_1' h_2').mem W'' := by
  rintro W'' (rfl | <_, Y', _, rfl, hf> | <_, Y', _, rfl, hf> | <_, Y', _, rfl, hf
>)
  * exact (sum3 V_0' V_1' V_2' h_0' h_1' h_2').master_mem
  * exact Or.inr (Or.inl <Y', f_0.rel_cod hf, rfl>)
  * exact Or.inr (Or.inr (Or.inl <Y', f_1.rel_cod hf, rfl>))
  * exact Or.inr (Or.inr (Or.inr <Y', f_2.rel_cod hf, rfl>))
have key : (W'_1  I  W'_2 = master3 V_0' V_1' V_2'  \/
    (exists  X Y', W = j0 X  /\  W'_1  I  W'_2 = j0 Y'  /\  f_0.rel X Y')  \/
    (exists  X Y', W = j1 X  /\  W'_1  I  W'_2 = j1 Y'  /\  f_1.rel X Y')  \/
    (exists  X Y', W = j2 X  /\  W'_1  I  W'_2 = j2 Y'  /\  f_2.rel X Y')) := by
  rcases hd_1 with rfl | <X, Y'_1, hWX_1, rfl, hf_1> | <Y, Y'_1, hWY_1, rfl, hf_1>
    | <Z, Y'_1, hWZ_1, rfl, hf_1>
  * rw [Set.inter_eq_right.mpr (show W'_2  subseteq  master3 V_0' V_1' V_2' from
      (sum3 V_0' V_1' V_2' h_0' h_1' h_2').sub_master hW'_2)]; exact hd_2
  * rcases hd_2 with rfl | <X', Y'_2, hWX_2, rfl, hf_2> | <Y', Y'_2, hWY_2, rfl,
hf_2>
      | <Z', Y'_2, hWZ_2, rfl, hf_2>
    * rw [Set.inter_eq_left.mpr (j0_subset_master3 (f_0.rel_cod hf_1))]
      exact Or.inr (Or.inl <X, Y'_1, hWX_1, rfl, hf_1>)
    * obtain rfl : X = X' := j0_injective (hWX_1.symm.trans hWX_2)
      rw [j0_inter_j0]
      exact Or.inr (Or.inl <X, Y'_1  I  Y'_2, hWX_1, rfl, f_0.inter_right hf_1
hf_2>)
    * obtain <a, ha> := h_0 X (f_0.rel_dom hf_1)
      exact absurd ((hWX_1.symm.trans hWY_2)  |>  t0_mem_j0.mpr ha) t0_not_mem_j1
    * obtain <a, ha> := h_0 X (f_0.rel_dom hf_1)
      exact absurd ((hWX_1.symm.trans hWZ_2)  |>  t0_mem_j0.mpr ha) t0_not_mem_j2
  * rcases hd_2 with rfl | <X', Y'_2, hWX_2, rfl, hf_2> | <Y', Y'_2, hWY_2, rfl,
hf_2>
      | <Z', Y'_2, hWZ_2, rfl, hf_2>
    * rw [Set.inter_eq_left.mpr (j1_subset_master3 (f_1.rel_cod hf_1))]
      exact Or.inr (Or.inr (Or.inl <Y, Y'_1, hWY_1, rfl, hf_1>))
    * obtain <b, hb> := h_1 Y (f_1.rel_dom hf_1)
      exact absurd ((hWY_1.symm.trans hWX_2)  |>  t1_mem_j1.mpr hb) t1_not_mem_j0
    * obtain rfl : Y = Y' := j1_injective (hWY_1.symm.trans hWY_2)
      rw [j1_inter_j1]
      exact Or.inr (Or.inr (Or.inl <Y, Y'_1  I  Y'_2, hWY_1, rfl, f_1.inter_right
hf_1 hf_2>))
    * obtain <b, hb> := h_1 Y (f_1.rel_dom hf_1)
      exact absurd ((hWY_1.symm.trans hWZ_2)  |>  t1_mem_j1.mpr hb) t1_not_mem_j2
  * rcases hd_2 with rfl | <X', Y'_2, hWX_2, rfl, hf_2> | <Y', Y'_2, hWY_2, rfl,
hf_2>
      | <Z', Y'_2, hWZ_2, rfl, hf_2>
    * rw [Set.inter_eq_left.mpr (j2_subset_master3 (f_2.rel_cod hf_1))]
      exact Or.inr (Or.inr (Or.inr <Z, Y'_1, hWZ_1, rfl, hf_1>))
    * obtain <c, hc> := h_2 Z (f_2.rel_dom hf_1)
      exact absurd ((hWZ_1.symm.trans hWX_2)  |>  t2_mem_j2.mpr hc) t2_not_mem_j0
    * obtain <c, hc> := h_2 Z (f_2.rel_dom hf_1)
      exact absurd ((hWZ_1.symm.trans hWY_2)  |>  t2_mem_j2.mpr hc) t2_not_mem_j1
    * obtain rfl : Z = Z' := j2_injective (hWZ_1.symm.trans hWZ_2)
      rw [j2_inter_j2]
      exact Or.inr (Or.inr (Or.inr <Z, Y'_1  I  Y'_2, hWZ_1, rfl, f_2.inter_right
hf_1 hf_2>))
exact <hW, hmem _ key, key>
```

```
  mono := by
    rintro W W_2 W' W'_2 <hW, hW', hd> hW_2W hW'W'_2 hW_2mem hW'_2mem
    refine <hW_2mem, hW'_2mem, ?_>
    rcases hd with rfl | <X, Y', rfl, rfl, hf> | <Y, Y', rfl, rfl, hf> | <Z, Y', rfl,
    rfl, hf>
    * exact Or.inl (eq_master3_of_subset hW'W'_2 ((sum3 V_0' V_1' V_2' h_0' h_1' h_2')
    .sub_master hW'_2mem))
    * obtain <X_2, hX_2, rfl> := mem_subset_j0_inv hW_2mem hW_2W
      have hX_2X : X_2  subseteq  X := j0_subset_j0.mp hW_2W
      rcases hW'_2mem with rfl | <Y'_2, hY'_2, rfl> | <Y'_2, hY'_2, rfl> | <Y'_2, hY'
    _2, rfl>
      * exact Or.inl rfl
      * exact Or.inr (Or.inl <X_2, Y'_2, rfl, rfl,
          f_0.mono hf hX_2X (j0_subset_j0.mp hW'W'_2) hX_2 hY'_2>)
      * obtain <a, ha> := h_0' Y' (f_0.rel_cod hf)
        exact absurd (hW'W'_2 (t0_mem_j0.mpr ha)) t0_not_mem_j1
      * obtain <a, ha> := h_0' Y' (f_0.rel_cod hf)
        exact absurd (hW'W'_2 (t0_mem_j0.mpr ha)) t0_not_mem_j2
    * obtain <Y_2, hY_2, rfl> := mem_subset_j1_inv hW_2mem hW_2W
      have hY_2Y : Y_2  subseteq  Y := j1_subset_j1.mp hW_2W
      rcases hW'_2mem with rfl | <Y'_2, hY'_2, rfl> | <Y'_2, hY'_2, rfl> | <Y'_2, hY'
    _2, rfl>
      * exact Or.inl rfl
      * obtain <b, hb> := h_1' Y' (f_1.rel_cod hf)
        exact absurd (hW'W'_2 (t1_mem_j1.mpr hb)) t1_not_mem_j0
      * exact Or.inr (Or.inr (Or.inl <Y_2, Y'_2, rfl, rfl,
          f_1.mono hf hY_2Y (j1_subset_j1.mp hW'W'_2) hY_2 hY'_2>))
      * obtain <b, hb> := h_1' Y' (f_1.rel_cod hf)
        exact absurd (hW'W'_2 (t1_mem_j1.mpr hb)) t1_not_mem_j2
    * obtain <Z_2, hZ_2, rfl> := mem_subset_j2_inv hW_2mem hW_2W
      have hZ_2Z : Z_2  subseteq  Z := j2_subset_j2.mp hW_2W
      rcases hW'_2mem with rfl | <Y'_2, hY'_2, rfl> | <Y'_2, hY'_2, rfl> | <Y'_2, hY'
    _2, rfl>
      * exact Or.inl rfl
      * obtain <c, hc> := h_2' Y' (f_2.rel_cod hf)
        exact absurd (hW'W'_2 (t2_mem_j2.mpr hc)) t2_not_mem_j0
      * obtain <c, hc> := h_2' Y' (f_2.rel_cod hf)
        exact absurd (hW'W'_2 (t2_mem_j2.mpr hc)) t2_not_mem_j1
      * exact Or.inr (Or.inr (Or.inr <Z_2, Y'_2, rfl, rfl,
          f_2.mono hf hZ_2Z (j2_subset_j2.mp hW'W'_2) hZ_2 hY'_2>))

/-- The three-way sum map is always strict: `(f_0+f_1+f_2)(False) = False`. (The master
     only relates to the
master, since `master3` is not any tagged copy.) -/
theorem isStrict_sumMap3 (f_0 : ApproximableMap V_0 V_0') (f_1 : ApproximableMap V_1
    V_1')
    (f_2 : ApproximableMap V_2 V_2') :
    IsStrict (sumMap3 (h_0 := h_0) (h_1 := h_1) (h_2 := h_2) (h_0' := h_0') (h_1' :=
    h_1') (h_2' := h_2')
      f_0 f_1 f_2) := by
  rintro Y <-, -, hd>
  have h0 : (none : Option (a ? b ? ?))  in  (sum3 V_0 V_1 V_2 h_0 h_1 h_2).master :=
    none_mem_master3
  rcases hd with rfl | <X, Y', hWX, -, -> | <X, Y', hWX, -, -> | <X, Y', hWX, -, ->
  * rfl
  * exact absurd (hWX  |>  h0) none_not_mem_j0
```

```
  * exact absurd (hWX  |>  h0) none_not_mem_j1
  * exact absurd (hWX  |>  h0) none_not_mem_j2

/-- **Functoriality (composition): `(g_0 comp f_0) + (g_1 comp f_1) + (g_2 comp f_2) =
    (g_0+g_1+g_2)  comp  (f_0+f_1+f_2)`.** -/
theorem sumMap3_comp {a'' b'' ?'' : Type*} {V_0'' : NeighborhoodSystem a''}
    {V_1'' : NeighborhoodSystem b''} {V_2'' : NeighborhoodSystem ?''}
    {h_0'' : forall  X, V_0''.mem X  ->  X.Nonempty} {h_1'' : forall  Y, V_1''.mem Y
    ->  Y.Nonempty}
    {h_2'' : forall  Z, V_2''.mem Z  ->  Z.Nonempty}
    (g_0 : ApproximableMap V_0' V_0'') (g_1 : ApproximableMap V_1' V_1'') (g_2 :
    ApproximableMap V_2' V_2'')
    (f_0 : ApproximableMap V_0 V_0') (f_1 : ApproximableMap V_1 V_1') (f_2 :
    ApproximableMap V_2 V_2') :
    sumMap3 (h_0 := h_0) (h_1 := h_1) (h_2 := h_2) (h_0' := h_0'') (h_1' := h_1'') (
    h_2' := h_2'')
        (g_0.comp f_0) (g_1.comp f_1) (g_2.comp f_2)
      = (sumMap3 (h_0 := h_0') (h_1 := h_1') (h_2 := h_2') (h_0' := h_0'') (h_1' :=
    h_1'') (h_2' := h_2'')
          g_0 g_1 g_2).comp
        (sumMap3 (h_0 := h_0) (h_1 := h_1) (h_2 := h_2) (h_0' := h_0') (h_1' := h_1')
    (h_2' := h_2')
          f_0 f_1 f_2) := by
  apply ApproximableMap.ext
  intro W W''
  constructor
  * rintro <hW, hW'', hd>
    rcases hd with rfl | <X, Z'', rfl, rfl, Y', hf, hg> | <Y, Z'', rfl, rfl, Y', hf,
    hg>
      | <Z, Z'', rfl, rfl, Y', hf, hg>
    * exact <master3 V_0' V_1' V_2', <hW, (sum3 V_0' V_1' V_2' h_0' h_1' h_2').
    master_mem, Or.inl rfl>,
        (sum3 V_0' V_1' V_2' h_0' h_1' h_2').master_mem, hW'', Or.inl rfl>
    * exact <j0 Y', <hW, Or.inr (Or.inl <Y', f_0.rel_cod hf, rfl>),
        Or.inr (Or.inl <X, Y', rfl, rfl, hf>)>,
        Or.inr (Or.inl <Y', f_0.rel_cod hf, rfl>), hW'', Or.inr (Or.inl <Y', Z'', rfl,
     rfl, hg>)>
    * exact <j1 Y', <hW, Or.inr (Or.inr (Or.inl <Y', f_1.rel_cod hf, rfl>)),
        Or.inr (Or.inr (Or.inl <Y, Y', rfl, rfl, hf>))>,
        Or.inr (Or.inr (Or.inl <Y', f_1.rel_cod hf, rfl>)), hW'',
        Or.inr (Or.inr (Or.inl <Y', Z'', rfl, rfl, hg>))>
    * exact <j2 Y', <hW, Or.inr (Or.inr (Or.inr <Y', f_2.rel_cod hf, rfl>)),
        Or.inr (Or.inr (Or.inr <Z, Y', rfl, rfl, hf>))>,
        Or.inr (Or.inr (Or.inr <Y', f_2.rel_cod hf, rfl>)), hW'',
        Or.inr (Or.inr (Or.inr <Y', Z'', rfl, rfl, hg>))>
  * rintro <W', <hW, hW', hdf>, _, hW'', hdg>
    refine <hW, hW'', ?_>
    rcases hdg with rfl | <X', Z'', hW'X', rfl, hg> | <Y', Z'', hW'Y', rfl, hg>
      | <Z', Z'', hW'Z', rfl, hg>
    * exact Or.inl rfl
    * rcases hdf with rfl | <X, Y'_0, rfl, hW'eq, hf> | <Y, Y'_0, rfl, hW'eq, hf>
        | <Z, Y'_0, rfl, hW'eq, hf>
      * exact absurd ((hW'X'.symm)  |>  none_mem_master3) none_not_mem_j0
      * obtain rfl : Y'_0 = X' := j0_injective (hW'eq.symm.trans hW'X')
        exact Or.inr (Or.inl <X, Z'', rfl, rfl, <Y'_0, hf, hg>>)
      * obtain <b, hb> := h_1' Y'_0 (f_1.rel_cod hf)
```

```
        exact absurd ((hW'eq.symm.trans hW'X')  |>  t1_mem_j1.mpr hb) t1_not_mem_j0
      * obtain <c, hc> := h_2' Y'_0 (f_2.rel_cod hf)
        exact absurd ((hW'eq.symm.trans hW'X')  |>  t2_mem_j2.mpr hc) t2_not_mem_j0
    * rcases hdf with rfl | <X, Y'_0, rfl, hW'eq, hf> | <Y, Y'_0, rfl, hW'eq, hf>
        | <Z, Y'_0, rfl, hW'eq, hf>
      * exact absurd ((hW'Y'.symm)  |>  none_mem_master3) none_not_mem_j1
      * obtain <a, ha> := h_0' Y'_0 (f_0.rel_cod hf)
        exact absurd ((hW'eq.symm.trans hW'Y')  |>  t0_mem_j0.mpr ha) t0_not_mem_j1
      * obtain rfl : Y'_0 = Y' := j1_injective (hW'eq.symm.trans hW'Y')
        exact Or.inr (Or.inr (Or.inl <Y, Z'', rfl, rfl, <Y'_0, hf, hg>>))
      * obtain <c, hc> := h_2' Y'_0 (f_2.rel_cod hf)
        exact absurd ((hW'eq.symm.trans hW'Y')  |>  t2_mem_j2.mpr hc) t2_not_mem_j1
    * rcases hdf with rfl | <X, Y'_0, rfl, hW'eq, hf> | <Y, Y'_0, rfl, hW'eq, hf>
        | <Z, Y'_0, rfl, hW'eq, hf>
      * exact absurd ((hW'Z'.symm)  |>  none_mem_master3) none_not_mem_j2
      * obtain <a, ha> := h_0' Y'_0 (f_0.rel_cod hf)
        exact absurd ((hW'eq.symm.trans hW'Z')  |>  t0_mem_j0.mpr ha) t0_not_mem_j2
      * obtain <b, hb> := h_1' Y'_0 (f_1.rel_cod hf)
        exact absurd ((hW'eq.symm.trans hW'Z')  |>  t1_mem_j1.mpr hb) t1_not_mem_j2
      * obtain rfl : Y'_0 = Z' := j2_injective (hW'eq.symm.trans hW'Z')
        exact Or.inr (Or.inr (Or.inr <Z, Z'', rfl, rfl, <Y'_0, hf, hg>>))

/-- **Functoriality (identities): `I + I + I = I`.** -/
theorem sumMap3_id :
    sumMap3 (h_0 := h_0) (h_1 := h_1) (h_2 := h_2) (h_0' := h_0) (h_1' := h_1) (h_2'
    := h_2)
      (idMap V_0) (idMap V_1) (idMap V_2) = idMap (sum3 V_0 V_1 V_2 h_0 h_1 h_2) := by
  apply ApproximableMap.ext
  intro W W'
  constructor
  * rintro <hW, hW', hd>
    refine <hW, hW', ?_>
    rcases hd with rfl | <X, Y', rfl, rfl, _, _, hXY> | <Y, Y', rfl, rfl, _, _, hXY>
      | <Z, Y', rfl, rfl, _, _, hXY>
    * exact (sum3 V_0 V_1 V_2 h_0 h_1 h_2).sub_master hW
    * exact j0_subset_j0.mpr hXY
    * exact j1_subset_j1.mpr hXY
    * exact j2_subset_j2.mpr hXY
  * rintro <hW, hW', hsub>
    refine <hW, hW', ?_>
    rcases hW with rfl | <X, hX, rfl> | <Y, hY, rfl> | <Z, hZ, rfl>
    * left; exact eq_master3_of_subset hsub ((sum3 V_0 V_1 V_2 h_0 h_1 h_2).sub_master
     hW')
    * rcases hW' with rfl | <X', hX', rfl> | <Y', hY', rfl> | <Z', hZ', rfl>
      * exact Or.inl rfl
      * exact Or.inr (Or.inl <X, X', rfl, rfl, hX, hX', j0_subset_j0.mp hsub>)
      * obtain <a, ha> := h_0 X hX; exact absurd (hsub (t0_mem_j0.mpr ha))
    t0_not_mem_j1
      * obtain <a, ha> := h_0 X hX; exact absurd (hsub (t0_mem_j0.mpr ha))
    t0_not_mem_j2
    * rcases hW' with rfl | <X', hX', rfl> | <Y', hY', rfl> | <Z', hZ', rfl>
      * exact Or.inl rfl
      * obtain <b, hb> := h_1 Y hY; exact absurd (hsub (t1_mem_j1.mpr hb))
    t1_not_mem_j0
      * exact Or.inr (Or.inr (Or.inl <Y, Y', rfl, rfl, hY, hY', j1_subset_j1.mp hsub>)
    )
```

```
      * obtain <b, hb> := h_1 Y hY; exact absurd (hsub (t1_mem_j1.mpr hb))
    t1_not_mem_j2
    * rcases hW' with rfl | <X', hX', rfl> | <Y', hY', rfl> | <Z', hZ', rfl>
      * exact Or.inl rfl
      * obtain <c, hc> := h_2 Z hZ; exact absurd (hsub (t2_mem_j2.mpr hc))
    t2_not_mem_j0
      * obtain <c, hc> := h_2 Z hZ; exact absurd (hsub (t2_mem_j2.mpr hc))
    t2_not_mem_j1
      * exact Or.inr (Or.inr (Or.inr <Z, Z', rfl, rfl, hZ, hZ', j2_subset_j2.mp hsub>)
    )

end SumMap3

section SumInjMap

open Example62C

variable {a b ? a' b' ?' : Type*}
  {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b} {V_2 : NeighborhoodSystem
    ?}
  {V_0' : NeighborhoodSystem a'} {V_1' : NeighborhoodSystem b'} {V_2' :
    NeighborhoodSystem ?'}
  {h_0 : forall  X, V_0.mem X  ->  X.Nonempty} {h_1 : forall  Y, V_1.mem Y  ->  Y.
    Nonempty}
  {h_2 : forall  Z, V_2.mem Z  ->  Z.Nonempty}
  {h_0' : forall  X, V_0'.mem X  ->  X.Nonempty} {h_1' : forall  Y, V_1'.mem Y  ->  Y.
    Nonempty}
  {h_2' : forall  Z, V_2'.mem Z  ->  Z.Nonempty}

/-- `(f_0+f_1+f_2)(inj_0 x) = inj_0(f_0 x)`. -/
theorem sumMap3_sinj0 (f_0 : ApproximableMap V_0 V_0') (f_1 : ApproximableMap V_1 V_1
    ')
    (f_2 : ApproximableMap V_2 V_2') (x_0 : V_0.Element) :
    (sumMap3 (h_0 := h_0) (h_1 := h_1) (h_2 := h_2) (h_0' := h_0') (h_1' := h_1') (h_2
    ' := h_2') f_0 f_1 f_2).toElementMap
        (sinj0 (V_1 := V_1) (V_2 := V_2) (h_0 := h_0) (h_1 := h_1) (h_2 := h_2) x_0)
      = sinj0 (V_1 := V_1') (V_2 := V_2') (h_0 := h_0') (h_1 := h_1') (h_2 := h_2') (
    f_0.toElementMap x_0) := by
  apply Element.ext
  intro W'
  constructor
  * rintro <U, hU, hUmem, hU'mem, hd>
    rcases hd with rfl | <X, Y', hUj, rfl, hf> | <X, Y', hUj, rfl, hf> | <X, Y', hUj,
    rfl, hf>
    * exact Or.inl rfl
    * rcases hU with hUm | <X_0, hX_0, hUeq, hx>
      * exact absurd ((hUm.symm.trans hUj)  |>  none_mem_master3) none_not_mem_j0
      * have hXX : X = X_0 := j0_injective (hUj.symm.trans hUeq)
        exact Or.inr <Y', f_0.rel_cod hf, rfl, <X_0, hx, hXX  |>  hf>>
    * rcases hU with hUm | <X_0, hX_0, hUeq, hx>
      * exact absurd ((hUm.symm.trans hUj)  |>  none_mem_master3) none_not_mem_j1
      * obtain <a, ha> := h_0 X_0 hX_0; exact absurd ((hUeq.symm.trans hUj)  |>
    t0_mem_j0.mpr ha) t0_not_mem_j1
    * rcases hU with hUm | <X_0, hX_0, hUeq, hx>
      * exact absurd ((hUm.symm.trans hUj)  |>  none_mem_master3) none_not_mem_j2
      * obtain <a, ha> := h_0 X_0 hX_0; exact absurd ((hUeq.symm.trans hUj)  |>
```

```
    t0_mem_j0.mpr ha) t0_not_mem_j2
  * rintro (rfl | <Y', hY', rfl, hm>)
    * exact <master3 V_0 V_1 V_2, Or.inl rfl, (sum3 V_0 V_1 V_2 h_0 h_1 h_2).
    master_mem,
        (sum3 V_0' V_1' V_2' h_0' h_1' h_2').master_mem, Or.inl rfl>
    * obtain <X, hx, hf> := hm
      exact <j0 X, Or.inr <X, x_0.sub hx, rfl, hx>, Or.inr (Or.inl <X, x_0.sub hx, rfl
    >),
        Or.inr (Or.inl <Y', f_0.rel_cod hf, rfl>), Or.inr (Or.inl <X, Y', rfl, rfl, hf
    >)>

/-- `(f_0+f_1+f_2)(inj_1 x) = inj_1(f_1 x)`. -/
theorem sumMap3_sinj1 (f_0 : ApproximableMap V_0 V_0') (f_1 : ApproximableMap V_1 V_1
    ')
    (f_2 : ApproximableMap V_2 V_2') (x_1 : V_1.Element) :
    (sumMap3 (h_0 := h_0) (h_1 := h_1) (h_2 := h_2) (h_0' := h_0') (h_1' := h_1') (h_2
    ' := h_2') f_0 f_1 f_2).toElementMap
        (sinj1 (V_0 := V_0) (V_2 := V_2) (h_0 := h_0) (h_1 := h_1) (h_2 := h_2) x_1)
      = sinj1 (V_0 := V_0') (V_2 := V_2') (h_0 := h_0') (h_1 := h_1') (h_2 := h_2') (
    f_1.toElementMap x_1) := by
  apply Element.ext
  intro W'
  constructor
  * rintro <U, hU, hUmem, hU'mem, hd>
    rcases hd with rfl | <X, Y', hUj, rfl, hf> | <X, Y', hUj, rfl, hf> | <X, Y', hUj,
    rfl, hf>
    * exact Or.inl rfl
    * rcases hU with hUm | <Y_0, hY_0, hUeq, hx>
      * exact absurd ((hUm.symm.trans hUj)  |>  none_mem_master3) none_not_mem_j0
      * obtain <b, hb> := h_1 Y_0 hY_0; exact absurd ((hUeq.symm.trans hUj)  |>
    t1_mem_j1.mpr hb) t1_not_mem_j0
    * rcases hU with hUm | <Y_0, hY_0, hUeq, hx>
      * exact absurd ((hUm.symm.trans hUj)  |>  none_mem_master3) none_not_mem_j1
      * have hXX : X = Y_0 := j1_injective (hUj.symm.trans hUeq)
        exact Or.inr <Y', f_1.rel_cod hf, rfl, <Y_0, hx, hXX  |>  hf>>
    * rcases hU with hUm | <Y_0, hY_0, hUeq, hx>
      * exact absurd ((hUm.symm.trans hUj)  |>  none_mem_master3) none_not_mem_j2
      * obtain <b, hb> := h_1 Y_0 hY_0; exact absurd ((hUeq.symm.trans hUj)  |>
    t1_mem_j1.mpr hb) t1_not_mem_j2
  * rintro (rfl | <Y', hY', rfl, hm>)
    * exact <master3 V_0 V_1 V_2, Or.inl rfl, (sum3 V_0 V_1 V_2 h_0 h_1 h_2).
    master_mem,
        (sum3 V_0' V_1' V_2' h_0' h_1' h_2').master_mem, Or.inl rfl>
    * obtain <X, hx, hf> := hm
      exact <j1 X, Or.inr <X, x_1.sub hx, rfl, hx>, Or.inr (Or.inr (Or.inl <X, x_1.sub
     hx, rfl>)),
        Or.inr (Or.inr (Or.inl <Y', f_1.rel_cod hf, rfl>)), Or.inr (Or.inr (Or.inl <X,
     Y', rfl, rfl, hf>))>

/-- `(f_0+f_1+f_2)(inj_2 x) = inj_2(f_2 x)`. -/
theorem sumMap3_sinj2 (f_0 : ApproximableMap V_0 V_0') (f_1 : ApproximableMap V_1 V_1
    ')
    (f_2 : ApproximableMap V_2 V_2') (x_2 : V_2.Element) :
    (sumMap3 (h_0 := h_0) (h_1 := h_1) (h_2 := h_2) (h_0' := h_0') (h_1' := h_1') (h_2
    ' := h_2') f_0 f_1 f_2).toElementMap
        (sinj2 (V_0 := V_0) (V_1 := V_1) (h_0 := h_0) (h_1 := h_1) (h_2 := h_2) x_2)
```

```
      = sinj2 (V_0 := V_0') (V_1 := V_1') (h_0 := h_0') (h_1 := h_1') (h_2 := h_2') (
    f_2.toElementMap x_2) := by
  apply Element.ext
  intro W'
  constructor
  * rintro <U, hU, hUmem, hU'mem, hd>
    rcases hd with rfl | <X, Y', hUj, rfl, hf> | <X, Y', hUj, rfl, hf> | <X, Y', hUj,
    rfl, hf>
    * exact Or.inl rfl
    * rcases hU with hUm | <Z_0, hZ_0, hUeq, hx>
      * exact absurd ((hUm.symm.trans hUj)  |>  none_mem_master3) none_not_mem_j0
      * obtain <c, hc> := h_2 Z_0 hZ_0; exact absurd ((hUeq.symm.trans hUj)  |>
    t2_mem_j2.mpr hc) t2_not_mem_j0
    * rcases hU with hUm | <Z_0, hZ_0, hUeq, hx>
      * exact absurd ((hUm.symm.trans hUj)  |>  none_mem_master3) none_not_mem_j1
      * obtain <c, hc> := h_2 Z_0 hZ_0; exact absurd ((hUeq.symm.trans hUj)  |>
    t2_mem_j2.mpr hc) t2_not_mem_j1
    * rcases hU with hUm | <Z_0, hZ_0, hUeq, hx>
      * exact absurd ((hUm.symm.trans hUj)  |>  none_mem_master3) none_not_mem_j2
      * have hXX : X = Z_0 := j2_injective (hUj.symm.trans hUeq)
        exact Or.inr <Y', f_2.rel_cod hf, rfl, <Z_0, hx, hXX  |>  hf>>
  * rintro (rfl | <Y', hY', rfl, hm>)
    * exact <master3 V_0 V_1 V_2, Or.inl rfl, (sum3 V_0 V_1 V_2 h_0 h_1 h_2).
    master_mem,
        (sum3 V_0' V_1' V_2' h_0' h_1' h_2').master_mem, Or.inl rfl>
    * obtain <X, hx, hf> := hm
      exact <j2 X, Or.inr <X, x_2.sub hx, rfl, hx>, Or.inr (Or.inr (Or.inr <X, x_2.sub
     hx, rfl>)),
        Or.inr (Or.inr (Or.inr <Y', f_2.rel_cod hf, rfl>)), Or.inr (Or.inr (Or.inr <X,
     Y', rfl, rfl, hf>))>

end SumInjMap

open Example62C in
/-- The morphism action of `T`: `T(f) = I_? + f + f` (identity on the terminator, `f`
    on each
successor copy). Always strict (`isStrict_sumMap3`). -/
def tcMapHom {D E : StrictDomainObj.{w}} (f : (D  -->  E)) :
    ((tcObj D)  -->  (tcObj E)) :=
  <sumMap3 (h_0 := Example62C.unitSys_nonempty) (h_1 := D.nonempty) (h_2 := D.nonempty
    )
      (h_0' := Example62C.unitSys_nonempty) (h_1' := E.nonempty) (h_2' := E.nonempty)
      (idMap unitSys) f.1 f.1, isStrict_sumMap3 _ _ _>

end
end DomainTheory.NeighbourhoodSystems
end
```

3.7.18 Lecture VI exercises

EXERCISES

3.7.19 Exercise 6.17 — algebras for sequence domains

What are the algebras for which C is initial? If A of 6.2 is a generalization of B, what is the corresponding generalization of C? Prove that it exists and explain what are the algebras involved.

Exercise_6_17.lean

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Exercise_4_19
public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.Exercise_5_16
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Exercise_2_12
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Theorem_2_7
public import DomainTheory.NeighbourhoodSystems.DomainEquations.Theorem_6_16

public import DomainTheory.NeighbourhoodSystems.DomainEquations.Proposition_6_6
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_8
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Exercise_3_16
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Exercise_3_20

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.Exercise510

open Example62C in
/- concordance: source L4077 EXERCISE 6.17 algebras for sequence domains -/
/-- **Exercise 6.17 -- the functor `T(X) = ? + X + X`** on the category of `{}`-free
    domains and strict
maps. On objects, `T(D) = ? + D + D` (Example 6.2's three-way sum); on maps, `T(f) = I_
    ? + f + f`. -/
abbrev Tc : Endofunctor StrictDomainObj.{w} where
  obj := tcObj
  map := tcMapHom
  map_id D := Subtype.ext (by
    show sumMap3 (idMap unitSys) (idMap D.sys) (idMap D.sys) = idMap (tcObj D).sys
    exact sumMap3_id)
  map_comp {D E F} g f := Subtype.ext (by
    show sumMap3 (idMap unitSys) (g.1.comp f.1) (g.1.comp f.1)
      = (sumMap3 (idMap unitSys) g.1 g.1).comp (sumMap3 (idMap unitSys) f.1 f.1)
    have h := sumMap3_comp (h_0 := Example62C.unitSys_nonempty) (h_1 := D.nonempty) (
    h_2 := D.nonempty)
      (h_0' := Example62C.unitSys_nonempty) (h_1' := E.nonempty) (h_2' := E.nonempty)
      (h_0'' := Example62C.unitSys_nonempty) (h_1'' := F.nonempty) (h_2'' := F.
    nonempty)
      (idMap unitSys) g.1 g.1 (idMap unitSys) f.1 f.1
    rw [idMap_comp] at h
    exact h)

@[simp] theorem Tc_obj (D : StrictDomainObj.{w}) : Tc.obj D = tcObj D := rfl

@[simp] theorem Tc_map_val {D E : StrictDomainObj.{w}} (f : (D  -->  E)) :
```

```
    (Tc.map f).1 = sumMap3 (h_0 := Example62C.unitSys_nonempty) (h_1 := D.nonempty) (
    h_2 := D.nonempty)
      (h_0' := Example62C.unitSys_nonempty) (h_1' := E.nonempty) (h_2' := E.nonempty)
      (idMap unitSys) f.1 f.1 := rfl

open Example44 Example62C ExampleB in
/-- `C` (Example 4.4: finite-or-infinite binary sequences) as an object of the `{}`-
    free category. -/
abbrev Cobj : StrictDomainObj.{0} := <Str, C, C_nonempty>

end

section
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.Exercise510

namespace Example62C

open Example44 ExampleB Example62

@[simp] theorem ccEquiv_apply (x : C.Element) : ccEquiv x = toCC x := rfl

/-- `(b*z).mem (bX)  <->  z.mem X`: the `b`-successor's filter restricted to the `b`-
    copy is `z`. -/
theorem consMap_mem_embBit {b : Bool} {z : C.Element} {X : Set Str} (hX : C.mem X) :
    ((consMap b).toElementMap z).mem (embBit b X)  <->  z.mem X := by
  constructor
  * rintro <X', hzX', _, _, hsub>
    rw [ <-  embBit_eq_prepend] at hsub
    exact z.up_mem hzX' hX (embBit_subset.mp hsub)
  * intro hz
    refine <X, hz, z.sub hz, memC_embBit b hX, ?_>
    rw [ <-  embBit_eq_prepend]

/-- `(b*z)` never meets the `(not b)`-copy: `0z` avoids the `1`-copies and vice versa (
    used to discharge
the cross-tag cases in `toCC_consMap`). -/
theorem consMap_not_mem_embBit_ne {b c : Bool} (hbc : b  !=  c) {z : C.Element} {X :
    Set Str} :
    not  ((consMap b).toElementMap z).mem (embBit c X) := by
  rintro <X', hzX', hX'mem, _, hsub>
  obtain <a, ha> := C_nonempty X' hX'mem
  rw [ <-  embBit_eq_prepend] at hsub
  obtain <w, hw, heq> := hsub <a, rfl, ha>
  rw [List.cons.injEq] at hw; exact hbc hw.1

/-- `(b*z)` avoids the terminator `{?}` (since `bsigma  !=  ?`). -/
theorem consMap_not_mem_nil {b : Bool} {z : C.Element} :
    not  ((consMap b).toElementMap z).mem ({[]} : Set Str) := by
  rintro <X', hzX', hX'mem, _, hsub>
  obtain <a, ha> := C_nonempty X' hX'mem
  rw [ <-  embBit_eq_prepend] at hsub
  have := hsub <a, rfl, ha>
  rw [Set.mem_singleton_iff] at this; exact absurd this (by simp)

end Example62C
end
```

```
section
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.Exercise510

section Initial

open Example44 Example62C ExampleB Exercise419 Exercise516

variable (B : TAlgebra Tc)

/-- The distinguished point `e = k(?)`: the image under `k` of the terminator (`?`-
    injection). -/
def descE : B.carrier.sys.Element :=
  B.str.1.toElementMap (sinj0 (h_0 := Example62C.unitSys_nonempty) (h_1 := B.carrier.
    nonempty)
    (h_2 := B.carrier.nonempty) unitSys.bot)

/-- The `b`-th successor operation `f_b = k  comp  inj_b`: `f_0` via the `0`-copy (`
    inj_1`), `f_1` via the
`1`-copy (`inj_2`). -/
def descF (b : Bool) (y : B.carrier.sys.Element) : B.carrier.sys.Element :=
  B.str.1.toElementMap (cond b
    (sinj2 (h_0 := Example62C.unitSys_nonempty) (h_1 := B.carrier.nonempty) (h_2 := B.
    carrier.nonempty) y)
    (sinj1 (h_0 := Example62C.unitSys_nonempty) (h_1 := B.carrier.nonempty) (h_2 := B.
    carrier.nonempty) y))

theorem descF_mono (b : Bool) {y y' : B.carrier.sys.Element} (h : y  <=  y') :
    descF B b y  <=  descF B b y' := by
  cases b
  * exact B.str.1.toElementMap_mono (sinj1_mono h)
  * exact B.str.1.toElementMap_mono (sinj2_mono h)

/-- The recursion `phi(?)=z`, `phi(b*sigma)=f_b(phi(sigma))` on a finite string, with
    base value `z`. -/
def descVal (z : B.carrier.sys.Element) : Str  ->  B.carrier.sys.Element
  | [] => z
  | b :: sigma => descF B b (descVal z sigma)

theorem descVal_append (z : B.carrier.sys.Element) (sigma ? : Str) :
    descVal B z (sigma ++ ?) = descVal B (descVal B z ?) sigma := by
  induction sigma with
  | nil => rfl
  | cons b sigma ih => exact congrArg (descF B b) ih

theorem descVal_mono_z {z z' : B.carrier.sys.Element} (h : z  <=  z') :
    forall  sigma, descVal B z sigma  <=  descVal B z' sigma
  | [] => h
  | _ :: sigma => descF_mono B _ (descVal_mono_z h sigma)

theorem descMap_hcone {sigma tau : Str} (h : sigma <+: tau) :
    descVal B B.carrier.sys.bot sigma  <=  descVal B B.carrier.sys.bot tau := by
  obtain <?, rfl> := h
  rw [descVal_append]
  exact descVal_mono_z B (B.carrier.sys.bot_le _) sigma
```

```
theorem descMap_hsing {sigma tau : Str} (h : sigma <+: tau) :
    descVal B B.carrier.sys.bot sigma  <=  descVal B (descE B) tau := by
  obtain <?, rfl> := h
  rw [descVal_append]
  exact descVal_mono_z B (B.carrier.sys.bot_le _) sigma

end Initial

end

end DomainTheory.NeighbourhoodSystems
end

section
/- concordance: source L4075 EXERCISES -- Lecture VI exercises -/
-- Exercise617/descMap.lean
namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.Exercise510

section Initial

open Example44 Example62C ExampleB Exercise419 Exercise516

variable (B : TAlgebra Tc)

/-- **The homomorphism `C  ->  E`.** Built by `liftC` from the head-recursion: `phi(
    sigmaFalse) = f_{sigma}(False)` and
`phi(sigma) = f_{sigma}(e)`, interpreting `b_0b_1...  |->  f_{b_0}(f_{b_1}(...))`. -/
def descMap : ApproximableMap C B.carrier.sys :=
  liftC B.carrier.sys (descVal B B.carrier.sys.bot) (descVal B (descE B))
    (fun {_ _} => descMap_hcone B) (fun {_ _} => descMap_hsing B)

end Initial

end DomainTheory.NeighbourhoodSystems
end

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.Exercise510

section
section Initial

open Example44 Example62C ExampleB Exercise419 Exercise516

variable (B : TAlgebra Tc)

@[simp] theorem descMap_strBot (sigma : Str) :
    (descMap B).toElementMap (strBot sigma) = descVal B B.carrier.sys.bot sigma :=
  liftC_strBot _ _ _ _ _ sigma

end Initial
end
```

```
section
section Initial

open Example44 Example62C ExampleB Exercise419 Exercise516

variable (B : TAlgebra Tc)

@[simp] theorem descMap_strElem (sigma : Str) :
    (descMap B).toElementMap (strElem sigma) = descVal B (descE B) sigma :=
  liftC_strElem _ _ _ _ _ sigma

end Initial
end

section
section Initial

open Example44 Example62C ExampleB Exercise419 Exercise516

variable (B : TAlgebra Tc)

theorem C_bot_eq_strBot_nil : C.bot = strBot [] := by
  apply NeighborhoodSystem.Element.ext
  intro Y
  show (C.mem Y  /\  C.master  subseteq  Y)  <->  (C.mem Y  /\  cone []  subseteq  Y)
  rw [C_master, cone_nil]

end Initial
end

section
section Initial

open Example44 Example62C ExampleB Exercise419 Exercise516

variable (B : TAlgebra Tc)

theorem ccEquiv_comp_symm :
    (ofIso ccEquiv).comp (ofIso ccEquiv.symm) = idMap (sum3 unitSys C C Example62C.
    unitSys_nonempty
      Example62C.C_nonempty Example62C.C_nonempty) := by
  apply ext_of_toElementMap
  intro s
  rw [toElementMap_comp, toElementMap_ofIso, toElementMap_ofIso, toElementMap_idMap]
  exact ccEquiv.apply_symm_apply s

end Initial
end

section
section Initial

open Example44 Example62C ExampleB Exercise419 Exercise516

variable (B : TAlgebra Tc)
```

```
theorem ccEquiv_symm_comp : (ofIso ccEquiv.symm).comp (ofIso ccEquiv) = idMap C := by
  apply ext_of_toElementMap
  intro x
  rw [toElementMap_comp, toElementMap_ofIso, toElementMap_ofIso, toElementMap_idMap]
  exact ccEquiv.symm_apply_apply x

end Initial
end

section
section Initial

open Example44 Example62C ExampleB Exercise419 Exercise516

variable (B : TAlgebra Tc)

theorem descMap_strict : IsStrict (descMap B) := by
  rw [isStrict_iff_apply_bot, C_bot_eq_strBot_nil, descMap_strBot]
  rfl

end Initial
end

section
section Initial

open Example44 Example62C ExampleB Exercise419 Exercise516

variable (B : TAlgebra Tc)

/-- The bundled strict homomorphism `C  ->  E`. -/
def descStrict : (Cobj  -->  B.carrier) := <descMap B, descMap_strict B>

end Initial
end

section
section Initial

open Example44 Example62C ExampleB Exercise419 Exercise516

variable (B : TAlgebra Tc)

/-- `T(g)` on `False` is `False`; precomposed with `k` it is `False` (both maps are
    strict). -/
theorem genKeyBot (g : ApproximableMap C B.carrier.sys) :
    B.str.1.toElementMap ((sumMap3 (h_0 := Example62C.unitSys_nonempty) (h_1 :=
    Example62C.C_nonempty)
        (h_2 := Example62C.C_nonempty) (h_0' := Example62C.unitSys_nonempty) (h_1' :=
    B.carrier.nonempty)
        (h_2' := B.carrier.nonempty) (idMap unitSys) g g).toElementMap (toCC (strBot
    [])))
      = B.carrier.sys.bot := by
  have hb : toCC (strBot []) = (sum3 unitSys C C Example62C.unitSys_nonempty
    Example62C.C_nonempty
      Example62C.C_nonempty).bot := by
```

```
    rw [ <-  C_bot_eq_strBot_nil,  <-  Example62C.ccEquiv_apply]; exact ccEquiv.
    map_bot
  rw [hb, isStrict_iff_apply_bot.mp (isStrict_sumMap3 (h_0 := Example62C.
    unitSys_nonempty)
    (h_1 := Example62C.C_nonempty) (h_2 := Example62C.C_nonempty) (idMap unitSys) g g)
    ]
  exact isStrict_iff_apply_bot.mp B.str.2

end Initial
end

section
/-- The map of an order-isomorphism is strict (an iso of domains preserves `False`). -/
theorem isStrict_ofIso {a b : Type*} {V_0 : NeighborhoodSystem a} {V_1 :
    NeighborhoodSystem b}
    (e : V_0.Element ?o V_1.Element) : IsStrict (ofIso e) := by
  rw [isStrict_iff_apply_bot, toElementMap_ofIso]
  exact e.map_bot
end

section
open Example44 Example62C in
/-- **The `T`-algebra structure on `C`.** `(tcObj Cobj).sys = ? + C + C` (
    definitionally Example 6.2's
`CC`), and the structure map `i : ? + C + C  ->  C` is the inverse of the domain-
    equation isomorphism
`ccEquiv` (Example 6.2), realised as an approximable map by `ofIso`; it is strict by `
    isStrict_ofIso`.
Concretely `i` sends the terminator to `Lambda-hat` and each `b`-copy of `x` to `b*x`.
    -/
abbrev cStr : ((Tc.obj Cobj)  -->  Cobj) :=
  <ofIso (by exact ccEquiv.symm), isStrict_ofIso _>
end

section
open Example44 Example62C in
/-- **`C` is a `T`-algebra**, `(C, i)` with `T(X) = ? + X + X`. -/
abbrev Calg : TAlgebra Tc := <Cobj, cStr>
end

section
namespace Example62C

open Example44 ExampleB Example62

/-- **`toCC  comp  (0*) = inj_1` and `toCC  comp  (1*) = inj_2`.** Prepending the bit `
    b` to `z` is, across the
isomorphism `C  ~=  ?+C+C`, the injection of `z` into the `b`-th `C`-summand. -/
theorem toCC_consMap (b : Bool) (z : C.Element) :
    toCC ((consMap b).toElementMap z)
      = cond b
          (sinj2 (V_0 := unitSys) (V_1 := C) (h_0 := unitSys_nonempty) (h_1 :=
    C_nonempty)
            (h_2 := C_nonempty) z)
          (sinj1 (V_0 := unitSys) (V_2 := C) (h_0 := unitSys_nonempty) (h_1 :=
    C_nonempty)
```

```
            (h_2 := C_nonempty) z) := by
  apply NeighborhoodSystem.Element.ext
  intro W
  cases b
  * simp only [Bool.cond_false]
    constructor
    * rintro (rfl | <rfl, hz> | <X, hX, rfl, hz> | <Y, hY, rfl, hz>)
      * exact Or.inl rfl
      * exact absurd hz consMap_not_mem_nil
      * exact Or.inr <X, hX, rfl, (consMap_mem_embBit hX).mp hz>
      * exact absurd hz (consMap_not_mem_embBit_ne (by decide))
    * rintro (rfl | <Y, hY, rfl, hz>)
      * exact Or.inl rfl
      * exact Or.inr (Or.inr (Or.inl <Y, hY, rfl, (consMap_mem_embBit hY).mpr hz>))
  * simp only [Bool.cond_true]
    constructor
    * rintro (rfl | <rfl, hz> | <X, hX, rfl, hz> | <Y, hY, rfl, hz>)
      * exact Or.inl rfl
      * exact absurd hz consMap_not_mem_nil
      * exact absurd hz (consMap_not_mem_embBit_ne (by decide))
      * exact Or.inr <Y, hY, rfl, (consMap_mem_embBit hY).mp hz>
    * rintro (rfl | <Y, hY, rfl, hz>)
      * exact Or.inl rfl
      * exact Or.inr (Or.inr (Or.inr <Y, hY, rfl, (consMap_mem_embBit hY).mpr hz>))

end Example62C
end

section
section Initial

open Example44 Example62C ExampleB Exercise419 Exercise516

variable (B : TAlgebra Tc)

/-- The composite `inj_0 comp (...)` of `T(g)` applied to a successor reduces to the
    operation `f_b`. The
single computational step behind both existence and uniqueness, for an *arbitrary* `g`.
     -/
theorem genKey (g : ApproximableMap C B.carrier.sys) (b : Bool) (w : C.Element) :
    B.str.1.toElementMap ((sumMap3 (h_0 := Example62C.unitSys_nonempty) (h_1 :=
    Example62C.C_nonempty)
        (h_2 := Example62C.C_nonempty) (h_0' := Example62C.unitSys_nonempty) (h_1' :=
    B.carrier.nonempty)
        (h_2' := B.carrier.nonempty) (idMap unitSys) g g).toElementMap
      (toCC ((consMap b).toElementMap w)))
      = descF B b (g.toElementMap w) := by
  rw [toCC_consMap]
  cases b
  * simp only [Bool.cond_false]; rw [sumMap3_sinj1]; rfl
  * simp only [Bool.cond_true]; rw [sumMap3_sinj2]; rfl

end Initial

end
```

```
section
namespace Example62C

open Example44 ExampleB Example62

/-- **`toCC Lambda-hat = inj_0`.** The finished empty sequence is the terminator (the
    `?`-summand). -/
theorem toCC_strElem_nil :
    toCC (strElem []) = sinj0 (V_1 := C) (V_2 := C) (h_0 := unitSys_nonempty) (h_1 :=
    C_nonempty)
      (h_2 := C_nonempty) unitSys.bot := by
  apply NeighborhoodSystem.Element.ext
  intro W
  constructor
  * rintro (rfl | <rfl, hz> | <X, hX, rfl, hz> | <Y, hY, rfl, hz>)
    * exact Or.inl rfl
    * exact Or.inr <Set.univ, rfl, rfl, unitSys.bot.master_mem>
    * exact absurd (hz.2 (Set.mem_singleton_iff.mpr rfl)) nil_not_mem_embBit
    * exact absurd (hz.2 (Set.mem_singleton_iff.mpr rfl)) nil_not_mem_embBit
  * rintro (rfl | <X, hX, rfl, hz>)
    * exact Or.inl rfl
    * obtain rfl : X = Set.univ := hX
      exact Or.inr (Or.inl <rfl, memC_singleton [], subset_rfl>)

end Example62C
end

section
section Initial

open Example44 Example62C ExampleB Exercise419 Exercise516

variable (B : TAlgebra Tc)

/-- `T(g)` on the terminator is the terminator; precomposed with `k` it is `e`. -/
theorem genKey0 (g : ApproximableMap C B.carrier.sys) :
    B.str.1.toElementMap ((sumMap3 (h_0 := Example62C.unitSys_nonempty) (h_1 :=
    Example62C.C_nonempty)
        (h_2 := Example62C.C_nonempty) (h_0' := Example62C.unitSys_nonempty) (h_1' :=
    B.carrier.nonempty)
        (h_2' := B.carrier.nonempty) (idMap unitSys) g g).toElementMap (toCC (strElem
    [])))
      = descE B := by
  rw [toCC_strElem_nil, sumMap3_sinj0, toElementMap_idMap]
  rfl

end Initial
end

section
section Initial

open Example44 Example62C ExampleB Exercise419 Exercise516

variable (B : TAlgebra Tc)
```

```
/-- `C`'s algebra map satisfies the recursion. -/
theorem descMap_satisfiesRec :
    descMap B = (B.str.1.comp (sumMap3 (h_0 := Example62C.unitSys_nonempty)
        (h_1 := Example62C.C_nonempty) (h_2 := Example62C.C_nonempty) (h_0' :=
    Example62C.unitSys_nonempty)
        (h_1' := B.carrier.nonempty) (h_2' := B.carrier.nonempty) (idMap unitSys) (
    descMap B)
        (descMap B))).comp (ofIso ccEquiv) := by
  apply map_ext_C
  * intro sigma
    rw [toElementMap_comp, toElementMap_comp, toElementMap_ofIso, descMap_strBot]
    cases sigma with
    | nil => exact (genKeyBot B (descMap B)).symm
    | cons b sigma =>
      rw [Example62C.ccEquiv_apply,  <-  consMap_strBot]
      have h := genKey B (descMap B) b (strBot sigma)
      rw [descMap_strBot] at h
      exact h.symm
  * intro sigma
    rw [toElementMap_comp, toElementMap_comp, toElementMap_ofIso, descMap_strElem]
    cases sigma with
    | nil => exact (genKey0 B (descMap B)).symm
    | cons b sigma =>
      rw [Example62C.ccEquiv_apply,  <-  consMap_strElem]
      have h := genKey B (descMap B) b (strElem sigma)
      rw [descMap_strElem] at h
      exact h.symm

end Initial
end

section
section Initial

open Example44 Example62C ExampleB Exercise419 Exercise516

variable (B : TAlgebra Tc)

/-- **The homomorphism square**, read off at the level of underlying approximable maps
    :
`desc  comp  i = k  comp  T(desc)`. -/
theorem descComm : (descMap B).comp (ofIso ccEquiv.symm)
    = B.str.1.comp (sumMap3 (h_0 := Example62C.unitSys_nonempty) (h_1 := Example62C.
    C_nonempty)
        (h_2 := Example62C.C_nonempty) (h_0' := Example62C.unitSys_nonempty) (h_1' :=
    B.carrier.nonempty)
        (h_2' := B.carrier.nonempty) (idMap unitSys) (descMap B) (descMap B)) := by
  conv_lhs => rw [descMap_satisfiesRec B]
  rw [comp_assoc, ccEquiv_comp_symm, comp_idMap]

end Initial
end

section
section Initial
```

```
open Example44 Example62C ExampleB Exercise419 Exercise516

variable (B : TAlgebra Tc)

/-- **The descent homomorphism `(C, i)  ->  (E, k)`** as a `T`-algebra homomorphism. -/
def descAlgHom : AlgHom Calg B where
  hom := descStrict B
  comm := by
    apply Subtype.ext
    simp only [StrictDomainObj.comp_val]
    exact descComm B

end Initial
end

section
section Initial

open Example44 Example62C ExampleB Exercise419 Exercise516

variable (B : TAlgebra Tc)

/-- **Any map satisfying the homomorphism recursion equals `descMap`.** This is *both*
    the existence
witness (`descMap` satisfies it) and the uniqueness driver. -/
theorem rec_determines (g : ApproximableMap C B.carrier.sys)
    (hg : g = (B.str.1.comp (sumMap3 (h_0 := Example62C.unitSys_nonempty) (h_1 :=
    Example62C.C_nonempty)
        (h_2 := Example62C.C_nonempty) (h_0' := Example62C.unitSys_nonempty) (h_1' :=
    B.carrier.nonempty)
        (h_2' := B.carrier.nonempty) (idMap unitSys) g g)).comp (ofIso ccEquiv)) :
    g = descMap B := by
  have hbot : forall  sigma, g.toElementMap (strBot sigma) = descVal B B.carrier.sys.
    bot sigma := by
    intro sigma
    induction sigma with
    | nil =>
      conv_lhs => rw [hg]
      rw [toElementMap_comp, toElementMap_comp, toElementMap_ofIso, Example62C.
    ccEquiv_apply]
      exact genKeyBot B g
    | cons b sigma ih =>
      conv_lhs => rw [hg]
      rw [toElementMap_comp, toElementMap_comp, toElementMap_ofIso, Example62C.
    ccEquiv_apply,
         <-  consMap_strBot]
      have h := genKey B g b (strBot sigma)
      rw [ih] at h
      exact h
  have helem : forall  sigma, g.toElementMap (strElem sigma) = descVal B (descE B)
    sigma := by
    intro sigma
    induction sigma with
    | nil =>
      conv_lhs => rw [hg]
      rw [toElementMap_comp, toElementMap_comp, toElementMap_ofIso, Example62C.
```

```
    ccEquiv_apply]
      exact genKey0 B g
    | cons b sigma ih =>
      conv_lhs => rw [hg]
      rw [toElementMap_comp, toElementMap_comp, toElementMap_ofIso, Example62C.
    ccEquiv_apply,
         <-  consMap_strElem]
      have h := genKey B g b (strElem sigma)
      rw [ih] at h
      exact h
  apply map_ext_C
  * intro sigma; rw [hbot, descMap_strBot]
  * intro sigma; rw [helem, descMap_strElem]

end Initial
end

section
section Initial

open Example44 Example62C ExampleB Exercise419 Exercise516

variable (B : TAlgebra Tc)

/-- **Uniqueness.** Any `T`-algebra homomorphism out of `(C, i)` equals `descAlgHom`.
    -/
theorem descAlgHom_uniq (h' : AlgHom Calg B) : h' = descAlgHom B := by
  obtain <hom, comm> := h'
  have hg : hom.1 = descMap B := by
    refine rec_determines B hom.1 ?_
    have hc : hom.1.comp (ofIso ccEquiv.symm)
        = B.str.1.comp (sumMap3 (h_0 := Example62C.unitSys_nonempty) (h_1 :=
    Example62C.C_nonempty)
          (h_2 := Example62C.C_nonempty) (h_0' := Example62C.unitSys_nonempty) (h_1'
    := B.carrier.nonempty)
          (h_2' := B.carrier.nonempty) (idMap unitSys) hom.1 hom.1) :=
      congrArg Subtype.val comm
    have h2 := congrArg (fun m => m.comp (ofIso ccEquiv)) hc
    simp only at h2
    rw [comp_assoc] at h2
    erw [ccEquiv_symm_comp, comp_idMap] at h2
    exact h2
  have hhom : hom = descStrict B := Subtype.ext hg
  subst hhom
  rfl

end Initial
end

section
/-- **Exercise 6.17 (existence half) -- `(C, i)` is an initial `T`-algebra for `T(X) =
    ? + X + X`.**
The descent map `phi : C  ->  E` is the closed-form head-recursion `phi(?) = e`, `phi(b
    *x) = f_b(phi x)`
(`f_b = k  comp  inj_b`), built choice-free via `liftC`; it is the unique `T`-algebra
    homomorphism, so `C`
```

```
is determined (up to iso, Proposition 6.6) as the initial algebra of `X  |->  ? + X + X
    `. -/
def CisInitial : IsInitial Calg where
  desc := descAlgHom
  uniq := fun B h => descAlgHom_uniq B h
end

section
set_option linter.unusedVariables false

theorem example_617 : forall
  (B :
    @DomainTheory.NeighbourhoodSystems.TAlgebra DomainTheory.NeighbourhoodSystems.
    StrictDomainObj
      DomainTheory.NeighbourhoodSystems.instCategoryStrictDomainObj DomainTheory.
    NeighbourhoodSystems.Tc)
  (h' :
    @DomainTheory.NeighbourhoodSystems.AlgHom DomainTheory.NeighbourhoodSystems.
    StrictDomainObj
      DomainTheory.NeighbourhoodSystems.instCategoryStrictDomainObj DomainTheory.
    NeighbourhoodSystems.Tc
      DomainTheory.NeighbourhoodSystems.Calg B),
  @Eq
    (@DomainTheory.NeighbourhoodSystems.AlgHom DomainTheory.NeighbourhoodSystems.
    StrictDomainObj
      DomainTheory.NeighbourhoodSystems.instCategoryStrictDomainObj DomainTheory.
    NeighbourhoodSystems.Tc
      DomainTheory.NeighbourhoodSystems.Calg B)
    h' (DomainTheory.NeighbourhoodSystems.descAlgHom B) :=
  DomainTheory.NeighbourhoodSystems.descAlgHom_uniq
end

end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.Exercise617Gen
open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap DomainTheory
    .NeighbourhoodSystems.Exercise510

section
variable {A : Type} [DecidableEq A]

/-- The token type `A*` of finite `A`-strings. -/
abbrev Strn (A : Type) := List A

/-- The cone `sigmaA*`: all extensions of `sigma`. -/
def coneN (sigma : Strn A) : Set (Strn A) := {w | sigma <+: w}

omit [DecidableEq A] in

@[simp] theorem mem_coneN {sigma w : Strn A} : w  in  coneN sigma  <->  sigma <+: w :=
     Iff.rfl

omit [DecidableEq A] in

theorem coneN_nil : coneN ([] : Strn A) = Set.univ := by ext w; simp [coneN]

omit [DecidableEq A] in
```

```
theorem coneN_subset_coneN {sigma tau : Strn A} : coneN sigma  subseteq  coneN tau
    <->  tau <+: sigma := by
  constructor
  * intro h; exact h (show sigma  in  coneN sigma from List.prefix_rfl)
  * intro htausigma w hw; exact htausigma.trans hw

omit [DecidableEq A] in

theorem coneN_injective {sigma tau : Strn A} (h : coneN sigma = coneN tau) : sigma =
    tau := by
  have h1 : tau <+: sigma := coneN_subset_coneN.mp (le_of_eq h)
  have h2 : sigma <+: tau := coneN_subset_coneN.mp (le_of_eq h.symm)
  exact h2.eq_of_length (h2.length_le.antisymm h1.length_le)

theorem coneN_trichotomy (sigma tau : Strn A) :
    coneN sigma  subseteq  coneN tau  \/  coneN tau  subseteq  coneN sigma  \/  coneN
    sigma  I  coneN tau = {} :=
  if hsigmatau : sigma <+: tau then Or.inr (Or.inl (coneN_subset_coneN.mpr hsigmatau))
  else if htausigma : tau <+: sigma then Or.inl (coneN_subset_coneN.mpr htausigma)
  else Or.inr (Or.inr (by
    ext w
    simp only [Set.mem_inter_iff, mem_coneN, Set.mem_empty_iff_false, iff_false,
    not_and]
    intro h1 h2
    rcases List.prefix_or_prefix_of_prefix h1 h2 with h | h
    * exact hsigmatau h
    * exact htausigma h))

/-- Membership in `C?`: a cone `sigmaA*` or a singleton `{sigma}`. -/
def memCn (X : Set (Strn A)) : Prop := (exists  sigma, X = coneN sigma)  \/  (exists
    sigma, X = {sigma})

omit [DecidableEq A] in

theorem memCn_coneN (sigma : Strn A) : memCn (coneN sigma) := Or.inl <sigma, rfl>

omit [DecidableEq A] in

theorem memCn_singleton (sigma : Strn A) : memCn ({sigma} : Set (Strn A)) := Or.inr <
    sigma, rfl>

omit [DecidableEq A] in

theorem singleton_subset_coneN {sigma tau : Strn A} : ({tau} : Set (Strn A))  subseteq
      coneN sigma  <->  sigma <+: tau := by
  rw [Set.singleton_subset_iff, mem_coneN]

theorem singleton_coneN_nd (sigma tau : Strn A) :
    ({tau} : Set (Strn A))  subseteq  coneN sigma  \/  coneN sigma  subseteq  {tau}
    \/  ({tau} : Set (Strn A))  I  coneN sigma = {} := by
  by_cases h : sigma <+: tau
  * exact Or.inl (singleton_subset_coneN.mpr h)
  * refine Or.inr (Or.inr ?_)
    ext w
    simp only [Set.mem_inter_iff, Set.mem_singleton_iff, mem_coneN, Set.
```

```
    mem_empty_iff_false,
      iff_false, not_and]
    rintro rfl htau
    exact h htau

theorem nestedOrDisjointN : NestedOrDisjoint (memCn (A := A)) := by
  rintro X Y (<sigma, rfl> | <sigma, rfl>) (<tau, rfl> | <tau, rfl>)
  * exact coneN_trichotomy sigma tau
  * rcases singleton_coneN_nd sigma tau with h | h | h
    * exact Or.inr (Or.inl h)
    * exact Or.inl h
    * exact Or.inr (Or.inr (by rw [Set.inter_comm]; exact h))
  * rcases singleton_coneN_nd tau sigma with h | h | h
    * exact Or.inl h
    * exact Or.inr (Or.inl h)
    * exact Or.inr (Or.inr h)
  * by_cases h : sigma = tau
    * subst h; exact Or.inl (Set.Subset.refl _)
    * refine Or.inr (Or.inr ?_)
      ext w
      simp only [Set.mem_inter_iff, Set.mem_singleton_iff, Set.mem_empty_iff_false,
    iff_false,
        not_and]
      rintro rfl h2
      exact h h2

/-- **The generic domain `C?`** of finite-or-infinite `A`-sequences. -/
abbrev Cn (A : Type) [DecidableEq A] : NeighborhoodSystem (Strn A) :=
  NeighborhoodSystem.ofNestedOrDisjoint memCn Set.univ
    <[], Set.mem_univ _> (Or.inl <[], coneN_nil.symm>) nestedOrDisjointN
    (fun _ => Set.subset_univ _)

@[simp] theorem Cn_mem {X : Set (Strn A)} : (Cn A).mem X  <->  memCn X := Iff.rfl

@[simp] theorem Cn_master : (Cn A).master = (Set.univ : Set (Strn A)) := rfl

/-- `C?` is `{}`-free: every neighbourhood is non-empty (cones and singletons are
    inhabited). -/
theorem Cn_nonempty : forall  X, (Cn A).mem X  ->  X.Nonempty := by
  rintro X (<sigma, rfl> | <sigma, rfl>)
  * exact <sigma, List.prefix_rfl>
  * exact <sigma, rfl>

/-- `sigmaX = {sigmatau ? tau  in  X}`. -/
def prependN (sigma : Strn A) (X : Set (Strn A)) : Set (Strn A) := {w | exists  tau,
    tau  in  X  /\  w = sigma ++ tau}

omit [DecidableEq A] in

@[simp] theorem mem_prependN {sigma : Strn A} {X : Set (Strn A)} {w : Strn A} :
    w  in  prependN sigma X  <->  exists  tau, tau  in  X  /\  w = sigma ++ tau := Iff
    .rfl

omit [DecidableEq A] in

theorem prependN_coneN (sigma ? : Strn A) : prependN sigma (coneN ?) = coneN (sigma ++
```

```
    ?) := by
  ext w
  simp only [mem_prependN, mem_coneN]
  constructor
  * rintro <tau, htau, rfl>; exact (List.prefix_append_right_inj sigma).mpr htau
  * rintro <t, ht>
    exact <? ++ t, List.prefix_append ? t, by rw [ <-  ht, List.append_assoc]>

omit [DecidableEq A] in

theorem prependN_mono (sigma : Strn A) {X X' : Set (Strn A)} (h : X'  subseteq  X) :
    prependN sigma X'  subseteq  prependN sigma X := by
  rintro w <tau, htau, rfl>; exact <tau, h htau, rfl>

omit [DecidableEq A] in

theorem prependN_singleton (sigma tau : Strn A) : prependN sigma {tau} = {sigma ++ tau
    } := by
  ext w
  simp only [mem_prependN, Set.mem_singleton_iff]
  constructor
  * rintro <t, rfl, rfl>; rfl
  * rintro rfl; exact <tau, rfl, rfl>

omit [DecidableEq A] in

theorem memCn_prependN (sigma : Strn A) {X : Set (Strn A)} (hX : memCn X) : memCn (
    prependN sigma X) := by
  rcases hX with <?, rfl> | <?, rfl>
  * exact Or.inl <sigma ++ ?, prependN_coneN sigma ?>
  * exact Or.inr <sigma ++ ?, prependN_singleton sigma ?>

/-- **The successor `x  |->  a*x`** prepending the letter `a`: `X (a*x) Y  <->  aX
    subseteq  Y`. -/
def consMapN (a : A) : ApproximableMap (Cn A) (Cn A) where
  rel X Y := memCn X  /\  memCn Y  /\  prependN [a] X  subseteq  Y
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := <Or.inl <[], coneN_nil.symm>, Or.inl <[], coneN_nil.symm>,
    fun _ _ => trivial>
  inter_right := by
    rintro X Y Y' <hX, hY, hXY> <_, hY', hXY'>
    have hsubInter : prependN [a] X  subseteq  Y  I  Y' := Set.subset_inter hXY hXY'
    have hZ : memCn (prependN [a] X) := memCn_prependN [a] hX
    exact <hX, (Cn A).inter_mem hY hY' hZ hsubInter, hsubInter>
  mono := by
    rintro X X' Y Y' <_, _, hXY> hX'X hYY' hX' hY'
    exact <hX', hY', (prependN_mono [a] hX'X).trans (hXY.trans hYY')>

@[simp] theorem consMapN_rel {a : A} {X Y : Set (Strn A)} :
    (consMapN a).rel X Y  <->  memCn X  /\  memCn Y  /\  prependN [a] X  subseteq  Y
    := Iff.rfl

section SumSig

universe v
```

```
variable {b : Type v} {? : Type v}


/-- Tokens of the indexed sum `? + Sigma_a b`. -/
abbrev SigTok (A : Type) (b : Type v) := Option (Unit ? (A  x  b))

end SumSig

/-- The partial element `sigmaFalse = sigmaA*`. -/
def strBotN (sigma : Strn A) : (Cn A).Element := (Cn A).principal (Cn_mem.mpr (
    memCn_coneN sigma))

theorem consMapN_strBot (a : A) (sigma : Strn A) :
    (consMapN a).toElementMap (strBotN sigma) = strBotN (a :: sigma) := by
  apply NeighborhoodSystem.Element.ext
  intro Y
  constructor
  * rintro <X, <_, hXcone>, _, hY, hsub>
    refine <hY, ?_>
    calc coneN (a :: sigma) = prependN [a] (coneN sigma) := by rw [prependN_coneN];
    rfl
      _  subseteq  prependN [a] X := prependN_mono [a] hXcone
      _  subseteq  Y := hsub
  * rintro <hY, hsub>
    refine <coneN sigma, <memCn_coneN sigma, subset_rfl>, memCn_coneN sigma, hY, ?_>
    rw [show prependN [a] (coneN sigma) = coneN (a :: sigma) by rw [prependN_coneN];
    rfl]
    exact hsub

/-- The total element `sigma = {sigma}`. -/
def strElemN (sigma : Strn A) : (Cn A).Element := (Cn A).principal (Cn_mem.mpr (
    memCn_singleton sigma))

theorem consMapN_strElem (a : A) (sigma : Strn A) :
    (consMapN a).toElementMap (strElemN sigma) = strElemN (a :: sigma) := by
  apply NeighborhoodSystem.Element.ext
  intro Y
  constructor
  * rintro <X, <_, hXsing>, _, hY, hsub>
    refine <hY, ?_>
    calc ({a :: sigma} : Set (Strn A)) = prependN [a] {sigma} := by rw [
    prependN_singleton]; rfl
      _  subseteq  prependN [a] X := prependN_mono [a] hXsing
      _  subseteq  Y := hsub
  * rintro <hY, hsub>
    refine <{sigma}, <memCn_singleton sigma, subset_rfl>, memCn_singleton sigma, hY, ?
    _>
    rw [show prependN [a] ({sigma} : Set (Strn A)) = {a :: sigma} by rw [
    prependN_singleton]; rfl]
    exact hsub

section SumSig

universe v
```

```
variable {b : Type v} {? : Type v}

/-- The token `t` in the `a`-indexed copy. -/
def tc (a : A) (t : b) : SigTok A b := some (Sum.inr (a, t))

/-- The `a`-indexed tagged copy `aX`. -/
def jc (a : A) (X : Set b) : Set (SigTok A b) := {w | exists  t, w = tc a t  /\  t  in
      X}

omit [DecidableEq A] in

theorem tc_injective {a a' : A} {t t' : b} (h : (tc a t : SigTok A b) = tc a' t') :
    a = a'  /\  t = t' := by
  simp only [tc, Option.some.injEq, Sum.inr.injEq, Prod.mk.injEq] at h; exact h

omit [DecidableEq A] in

@[simp] theorem tc_mem_jc {a : A} {X : Set b} {t : b} :
    (tc a t : SigTok A b)  in  jc a X  <->  t  in  X := by
  constructor
  * rintro <t', heq, ht'>; obtain <-, rfl> := tc_injective heq; exact ht'
  * intro ht; exact <t, rfl, ht>

/-- The lone `?`-token. -/
def tu : SigTok A b := some (Sum.inl ())

/-- The `?`-copy neighbourhood (the image of `univ : Set Unit`). -/
def jU : Set (SigTok A b) := {tu}

omit [DecidableEq A] in

@[simp] theorem mem_jU {w : SigTok A b} : w  in  (jU : Set (SigTok A b))  <->  w = tu
    := Iff.rfl

omit [DecidableEq A] in

@[simp] theorem none_not_mem_jU : (none : SigTok A b)  notin  (jU : Set (SigTok A b))
    := by
  simp [jU, tu]

omit [DecidableEq A] in

@[simp] theorem none_not_mem_jc {a : A} {X : Set b} : (none : SigTok A b)  notin  jc a
     X := by
  rintro <t, heq, ->; exact absurd heq (by simp [tc])

omit [DecidableEq A] in

@[simp] theorem tu_not_mem_jc {a : A} {X : Set b} : (tu : SigTok A b)  notin  jc a X
    := by
  rintro <t, heq, ->; exact absurd heq (by simp [tu, tc])

omit [DecidableEq A] in

@[simp] theorem tc_not_mem_jU {a : A} {t : b} : (tc a t : SigTok A b)  notin  (jU :
    Set (SigTok A b)) := by
```

```
  simp [jU, tu, tc]

omit [DecidableEq A] in

theorem jU_inter_jc (a : A) (X : Set b) : (jU  I  jc a X : Set (SigTok A b)) = {} :=
    by
  ext w
  simp only [Set.mem_inter_iff, mem_jU, Set.mem_empty_iff_false, iff_false, not_and]
  rintro rfl; exact tu_not_mem_jc

omit [DecidableEq A] in

theorem jU_nonempty : (jU : Set (SigTok A b)).Nonempty := <tu, rfl>

omit [DecidableEq A] in

theorem jc_inter_jc_ne {a a' : A} (h : a  !=  a') (X X' : Set b) :
    (jc a X  I  jc a' X' : Set (SigTok A b)) = {} := by
  ext w
  simp only [Set.mem_inter_iff, Set.mem_empty_iff_false, iff_false, not_and]
  rintro <t, rfl, -> <t', heq, ->
  exact h (tc_injective heq).1

omit [DecidableEq A] in

theorem jc_inter_jc_same (a : A) (X X' : Set b) :
    (jc a X  I  jc a X' : Set (SigTok A b)) = jc a (X  I  X') := by
  ext w
  constructor
  * rintro <<t, rfl, ht>, <t', heq, ht'>>
    obtain <-, rfl> := tc_injective heq; exact <t, rfl, ht, ht'>
  * rintro <t, rfl, ht, ht'>; exact <<t, rfl, ht>, <t, rfl, ht'>>

omit [DecidableEq A] in

theorem jc_nonempty {a : A} {X : Set b} (hX : X.Nonempty) : (jc a X : Set (SigTok A b)
    ).Nonempty := by
  obtain <t, ht> := hX; exact <tc a t, t, rfl, ht>

omit [DecidableEq A] in

theorem jc_subset_jc {a : A} {X X' : Set b} :
    (jc a X : Set (SigTok A b))  subseteq  jc a X'  <->  X  subseteq  X' := by
  constructor
  * intro h t ht; exact tc_mem_jc.mp (h (tc_mem_jc.mpr ht))
  * rintro h w <t, rfl, ht>; exact tc_mem_jc.mpr (h ht)

omit [DecidableEq A] in

theorem jc_injective {a : A} {X X' : Set b} (h : (jc a X : Set (SigTok A b)) = jc a X
    ') : X = X' :=
  Set.Subset.antisymm (jc_subset_jc.mp h.subset) (jc_subset_jc.mp h.symm.subset)

omit [DecidableEq A] in
/-- Tagged copies determine both index and set: `aX = a'X'` (with `X` non-empty) forces
    `a=a'`,
```

```
`X=X'`. -/
theorem jc_eq_jc {a a' : A} {X X' : Set b} (hXne : X.Nonempty)
    (heq : (jc a X : Set (SigTok A b)) = jc a' X') : a = a'  /\  X = X' := by
  obtain <t, ht> := hXne
  obtain <t', he, -> := heq  |>  (tc_mem_jc.mpr ht)
  obtain <rfl, -> := tc_injective he
  exact <rfl, jc_injective heq>

end SumSig

section SumSig

universe v

variable {b : Type v} {? : Type v}

variable (V : NeighborhoodSystem b)

/-- The master neighbourhood `{?}  U  {tu}  U  ?_a aDelta`. -/
def masterSig : Set (SigTok A b) :=
  {w | w = none  \/  w = tu  \/  exists  a t, t  in  V.master  /\  w = tc a t}

end SumSig

section SumSig

universe v

variable {b : Type v} {? : Type v}

variable (V : NeighborhoodSystem b)

variable {V}

omit [DecidableEq A] in

@[simp] theorem none_mem_masterSig : (none : SigTok A b)  in  masterSig V := Or.inl
    rfl

end SumSig

section SumSig

universe v

variable {b : Type v} {? : Type v}

omit [DecidableEq A] in

theorem tc_mem_jc_ne {a a' : A} (h : a  !=  a') {X : Set b} {t : b} :
    (tc a t : SigTok A b)  notin  jc a' X := by
  rintro <t', heq, ->; exact h (tc_injective heq).1

end SumSig

section SumSig
```

```
universe v

variable {b : Type v} {? : Type v}

variable (V : NeighborhoodSystem b)

variable {V}

omit [DecidableEq A] in

@[simp] theorem tu_mem_masterSig : (tu : SigTok A b)  in  masterSig V := Or.inr (Or.
    inl rfl)

omit [DecidableEq A] in

theorem eq_masterSig_of_subset {W : Set (SigTok A b)}
    (hsub : masterSig V  subseteq  W) (hsub' : W  subseteq  masterSig V) : W =
    masterSig V :=
  Set.Subset.antisymm hsub' hsub

omit [DecidableEq A] in

theorem jU_subset_masterSig : (jU : Set (SigTok A b))  subseteq  masterSig V := by
  rintro w rfl; exact tu_mem_masterSig

omit [DecidableEq A] in

theorem jc_subset_masterSig {a : A} {X : Set b} (hX : V.mem X) :
    (jc a X : Set (SigTok A b))  subseteq  masterSig V := by
  rintro w <t, rfl, ht>; exact Or.inr (Or.inr <a, t, V.sub_master hX ht, rfl>)

omit [DecidableEq A] in

theorem masterSig_inter_jU : (masterSig V  I  jU : Set (SigTok A b)) = jU :=
  Set.inter_eq_right.mpr jU_subset_masterSig

omit [DecidableEq A] in

theorem masterSig_inter_jc {a : A} {X : Set b} (hX : V.mem X) :
    (masterSig V  I  jc a X : Set (SigTok A b)) = jc a X :=
  Set.inter_eq_right.mpr (jc_subset_masterSig hX)

/-- **The `A`-indexed separated sum `? + Sigma_a V`** over `{?}  U  {tu}  U  ?_a aDelta
    `, under the standing
assumption that no neighbourhood of `V` is empty. The alphabet-generic analogue of `
    sum3 unitSys V V`
(Example 6.2). -/
def sumSig (A : Type) [DecidableEq A] (V : NeighborhoodSystem b)
    (h : forall  X, V.mem X  ->  X.Nonempty) :
    NeighborhoodSystem (SigTok A b) where
  mem W := W = masterSig V  \/  W = jU  \/  exists  a X, V.mem X  /\  W = jc a X
  master := masterSig V
  master_nonempty := <none, none_mem_masterSig>
  master_mem := Or.inl rfl
  sub_master := by
```

```
    rintro W (rfl | rfl | <a, X, hX, rfl>)
    * exact subset_rfl
    * exact jU_subset_masterSig
    * exact jc_subset_masterSig hX
  inter_mem := by
    have hne : forall  W, (W = masterSig V  \/  W = jU  \/  exists  a X, V.mem X  /\
    W = jc a X)  ->
        (W : Set (SigTok A b)).Nonempty := by
      rintro W (rfl | rfl | <a, X, hX, rfl>)
      * exact <none, none_mem_masterSig>
      * exact jU_nonempty
      * exact jc_nonempty (h X hX)
    rintro W W' Z hW hW' hZ hZsub
    rcases hW with rfl | rfl | <a, X, hX, rfl>
    * rcases hW' with rfl | rfl | <a', X', hX', rfl>
      * rw [Set.inter_self]; exact Or.inl rfl
      * rw [masterSig_inter_jU]; exact Or.inr (Or.inl rfl)
      * rw [masterSig_inter_jc hX']; exact Or.inr (Or.inr <a', X', hX', rfl>)
    * rcases hW' with rfl | rfl | <a', X', hX', rfl>
      * rw [Set.inter_comm, masterSig_inter_jU]; exact Or.inr (Or.inl rfl)
      * rw [Set.inter_self]; exact Or.inr (Or.inl rfl)
      * rw [jU_inter_jc] at hZsub  |-
        obtain <t, ht> := hne Z hZ; exact absurd (hZsub ht) (Set.notMem_empty t)
    * rcases hW' with rfl | rfl | <a', X', hX', rfl>
      * rw [Set.inter_comm, masterSig_inter_jc hX]; exact Or.inr (Or.inr <a, X, hX,
    rfl>)
      * rw [Set.inter_comm, jU_inter_jc] at hZsub  |-
        obtain <t, ht> := hne Z hZ; exact absurd (hZsub ht) (Set.notMem_empty t)
      * by_cases haa : a = a'
        * subst haa
          rw [jc_inter_jc_same] at hZsub  |-
          rcases hZ with rfl | rfl | <a_2, Z_2, hZ_2, rfl>
          * exact absurd (hZsub none_mem_masterSig) (by simp)
          * exact absurd (hZsub (show tu  in  jU from rfl)) tu_not_mem_jc
          * by_cases ha_2 : a = a_2
            * subst ha_2
              exact Or.inr (Or.inr <a, X  I  X', V.inter_mem hX hX' hZ_2 (jc_subset_jc
    .mp hZsub), rfl>)
            * obtain <t, ht> := h Z_2 hZ_2
              exact absurd (hZsub (tc_mem_jc.mpr ht)) (tc_mem_jc_ne (Ne.symm ha_2))
        * rw [jc_inter_jc_ne haa] at hZsub  |-
          obtain <t, ht> := hne Z hZ; exact absurd (hZsub ht) (Set.notMem_empty t)

@[simp] theorem sumSig_master {h : forall  X, V.mem X  ->  X.Nonempty} :
    (sumSig A V h).master = masterSig V := rfl

end SumSig

section SumSig

universe v

variable {b : Type v} {? : Type v}

variable (V : NeighborhoodSystem b)
```

```
variable {V}

variable {h : forall  X, V.mem X  ->  X.Nonempty}

/-- A `sumSig`-neighbourhood contained in the `?`-copy `jU` is `jU`. -/
theorem mem_subset_jU_inv {W : Set (SigTok A b)}
    (hW : (sumSig A V h).mem W) (hsub : W  subseteq  jU) : W = jU := by
  rcases hW with rfl | rfl | <a', X_2, hX_2, rfl>
  * exact absurd (hsub none_mem_masterSig) none_not_mem_jU
  * rfl
  * obtain <t, ht> := h X_2 hX_2
    exact absurd (hsub (tc_mem_jc.mpr ht)) tc_not_mem_jU

/-- A `sumSig`-neighbourhood contained in an `a`-copy `aX` is itself an `a`-copy. -/
theorem mem_subset_jc_inv {W : Set (SigTok A b)} {a : A} {X : Set b}
    (hW : (sumSig A V h).mem W) (hsub : W  subseteq  jc a X) : exists  X_2, V.mem X_2
     /\  W = jc a X_2 := by
  rcases hW with rfl | rfl | <a', X_2, hX_2, rfl>
  * exact absurd (hsub none_mem_masterSig) none_not_mem_jc
  * exact absurd (hsub (show tu  in  jU from rfl)) tu_not_mem_jc
  * obtain <t, ht> := h X_2 hX_2
    obtain <t', heq, -> := hsub (tc_mem_jc.mpr ht)
    obtain <rfl, -> := tc_injective heq
    exact <X_2, hX_2, rfl>

/-- The `a`-th copy injection `V ? ?+Sigma_a V`: send `x in |V|` to the sum element
    whose proper
neighbourhoods are the `a`-copies `aX` with `X in x`. -/
def sinjC (a : A) (x : V.Element) : (sumSig A V h).Element where
  mem W := W = masterSig V  \/  exists  X, V.mem X  /\  W = jc a X  /\  x.mem X
  sub := by
    rintro W (rfl | <X, hX, rfl, ->)
    * exact Or.inl rfl
    * exact Or.inr (Or.inr <a, X, hX, rfl>)
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | <X, hX, rfl, hx>) (rfl | <X', hX', rfl, hx'>)
    * exact Or.inl (by rw [Set.inter_self])
    * exact Or.inr <X', hX', by rw [masterSig_inter_jc hX'], hx'>
    * exact Or.inr <X, hX, by rw [Set.inter_comm, masterSig_inter_jc hX], hx>
    * exact Or.inr <X  I  X', x.sub (x.inter_mem hx hx'), jc_inter_jc_same a X X', x.
    inter_mem hx hx'>
  up_mem := by
    rintro W W' (rfl | <X, hX, rfl, hx>) hW' hsub
    * exact Or.inl (eq_masterSig_of_subset hsub ((sumSig A V h).sub_master hW'))
    * rcases hW' with rfl | rfl | <a', X', hX', rfl>
      * exact Or.inl rfl
      * obtain <t, ht> := h X (x.sub hx)
        exact absurd (hsub (tc_mem_jc.mpr ht)) tc_not_mem_jU
      * obtain <t, ht> := h X (x.sub hx)
        obtain <t', heq, -> := hsub (tc_mem_jc.mpr ht)
        obtain <rfl, -> := tc_injective heq
        exact Or.inr <X', hX', rfl, x.up_mem hx hX' (jc_subset_jc.mp hsub)>

/-- The basepoint/`?`-injection: the image of the unique point of `?`. Its proper
    neighbourhood is
```

```
the `?`-copy `jU`. -/
def sinjU : (sumSig A V h).Element where
  mem W := W = masterSig V  \/  W = jU
  sub := by
    rintro W (rfl | rfl)
    * exact Or.inl rfl
    * exact Or.inr (Or.inl rfl)
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | rfl) (rfl | rfl)
    * exact Or.inl (by rw [Set.inter_self])
    * exact Or.inr (by rw [masterSig_inter_jU])
    * exact Or.inr (by rw [Set.inter_comm, masterSig_inter_jU])
    * exact Or.inr (by rw [Set.inter_self])
  up_mem := by
    rintro W W' (rfl | rfl) hW' hsub
    * exact Or.inl (eq_masterSig_of_subset hsub ((sumSig A V h).sub_master hW'))
    * rcases hW' with rfl | rfl | <a', X', hX', rfl>
      * exact Or.inl rfl
      * exact Or.inr rfl
      * exact absurd (hsub (show tu  in  jU from rfl)) tu_not_mem_jc

@[simp] theorem sinjU_mem_jU : (sinjU (h := h)).mem (jU : Set (SigTok A b)) := Or.inr
    rfl

@[simp] theorem sinjC_mem_jc {a : A} {x : V.Element} {X : Set b} (hX : V.mem X) :
    (sinjC (h := h) a x).mem (jc a X)  <->  x.mem X := by
  constructor
  * rintro (h0 | <X', hX', heq, hx>)
    * exact absurd (h0  |>  none_mem_masterSig) none_not_mem_jc
    * rw [jc_injective heq]; exact hx
  * intro hx; exact Or.inr <X, hX, rfl, hx>

theorem sinjC_mono {a : A} {x x' : V.Element} (hxle : x  <=  x') :
    sinjC (h := h) a x  <=  sinjC a x' := by
  rintro W (rfl | <X, hX, rfl, hm>)
  * exact Or.inl rfl
  * exact Or.inr <X, hX, rfl, hxle X hm>

end SumSig

section SumMapSig

universe v

variable {b_0 b_1 b_2 : Type v}
  {V_0 : NeighborhoodSystem b_0} {V_1 : NeighborhoodSystem b_1} {V_2 :
    NeighborhoodSystem b_2}
  {h_0 : forall  X, V_0.mem X  ->  X.Nonempty} {h_1 : forall  Y, V_1.mem Y  ->  Y.
    Nonempty}
  {h_2 : forall  Z, V_2.mem Z  ->  Z.Nonempty}

/-- **The indexed sum map `Sigmaf = I_? + Sigma_a f`** acting as the identity on the
    `?`-summand and as `f`
on each `a`-copy. The generic analogue of `sumMap3 (idMap unitSys) f f`. -/
def sumMapSig (f : ApproximableMap V_0 V_1) :
```

```
  ApproximableMap (sumSig A V_0 h_0) (sumSig A V_1 h_1) where
rel W W' := (sumSig A V_0 h_0).mem W  /\  (sumSig A V_1 h_1).mem W'  /\
  (W' = masterSig V_1  \/  (W = jU  /\  W' = jU)  \/
    exists  a X Y', W = jc a X  /\  W' = jc a Y'  /\  f.rel X Y')
rel_dom hr := hr.1
rel_cod hr := hr.2.1
master_rel := <(sumSig A V_0 h_0).master_mem, (sumSig A V_1 h_1).master_mem, Or.inl
  rfl>
inter_right := by
  rintro W W'_1 W'_2 <hW, hW'_1, hd_1> <-, hW'_2, hd_2>
  have hmem : forall  W'' : Set (SigTok A b_1),
      (W'' = masterSig V_1  \/  (W = jU  /\  W'' = jU)  \/
        exists  a X Y', W = jc a X  /\  W'' = jc a Y'  /\  f.rel X Y')  ->  (sumSig
  A V_1 h_1).mem W'' := by
    rintro W'' (rfl | <-, rfl> | <a, X, Y', -, rfl, hf>)
    * exact (sumSig A V_1 h_1).master_mem
    * exact Or.inr (Or.inl rfl)
    * exact Or.inr (Or.inr <a, Y', f.rel_cod hf, rfl>)
  have key : (W'_1  I  W'_2 = masterSig V_1  \/  (W = jU  /\  W'_1  I  W'_2 = jU)
  \/
      exists  a X Y', W = jc a X  /\  W'_1  I  W'_2 = jc a Y'  /\  f.rel X Y') := by
    rcases hd_1 with rfl | <hWU, rfl> | <a, X, Y'_1, hWX, rfl, hf_1>
    * rw [Set.inter_eq_right.mpr (show W'_2  subseteq  masterSig V_1 from
        (sumSig A V_1 h_1).sub_master hW'_2)]; exact hd_2
    * rcases hd_2 with rfl | <-, rfl> | <a, X, Y', hWX, rfl, hf>
      * rw [Set.inter_eq_left.mpr jU_subset_masterSig]; exact Or.inr (Or.inl <hWU,
  rfl>)
      * rw [Set.inter_self]; exact Or.inr (Or.inl <hWU, rfl>)
      * obtain <t, ht> := h_0 X (f.rel_dom hf)
        exact absurd ((hWX.symm.trans hWU)  |>  tc_mem_jc.mpr ht) tc_not_mem_jU
    * rcases hd_2 with rfl | <hWU2, rfl> | <a', X', Y'_2, hWX', rfl, hf_2>
      * rw [Set.inter_eq_left.mpr (jc_subset_masterSig (f.rel_cod hf_1))]
        exact Or.inr (Or.inr <a, X, Y'_1, hWX, rfl, hf_1>)
      * obtain <t, ht> := h_0 X (f.rel_dom hf_1)
        exact absurd ((hWX.symm.trans hWU2)  |>  tc_mem_jc.mpr ht) tc_not_mem_jU
      * obtain <rfl, rfl> := jc_eq_jc (h_0 X (f.rel_dom hf_1)) (hWX.symm.trans hWX')
        rw [jc_inter_jc_same]
        exact Or.inr (Or.inr <a, X, Y'_1  I  Y'_2, hWX, rfl, f.inter_right hf_1 hf_2
  >)
  exact <hW, hmem _ key, key>
mono := by
  rintro W W_2 W' W'_2 <hW, hW', hd> hW_2W hW'W'_2 hW_2mem hW'_2mem
  refine <hW_2mem, hW'_2mem, ?_>
  rcases hd with rfl | <rfl, rfl> | <a, X, Y', rfl, rfl, hf>
  * exact Or.inl (eq_masterSig_of_subset hW'W'_2 ((sumSig A V_1 h_1).sub_master hW'
  _2mem))
  * have hW_2jU : W_2 = jU := mem_subset_jU_inv hW_2mem hW_2W
    rcases hW'_2mem with rfl | rfl | <a', Y'_2, hY'_2, rfl>
    * exact Or.inl rfl
    * exact Or.inr (Or.inl <hW_2jU, rfl>)
    * exact absurd (hW'W'_2 (show tu  in  jU from rfl)) tu_not_mem_jc
  * obtain <X_2, hX_2, rfl> := mem_subset_jc_inv hW_2mem hW_2W
    have hX_2X : X_2  subseteq  X := jc_subset_jc.mp hW_2W
    rcases hW'_2mem with rfl | rfl | <a', Y'_2, hY'_2, rfl>
    * exact Or.inl rfl
    * obtain <t, ht> := h_1 Y' (f.rel_cod hf)
```

```
        exact absurd (hW'W'_2 (tc_mem_jc.mpr ht)) tc_not_mem_jU
      * obtain <t, ht> := h_1 Y' (f.rel_cod hf)
        obtain <t', he, -> := hW'W'_2 (tc_mem_jc.mpr ht)
        obtain <rfl, -> := tc_injective he
        exact Or.inr (Or.inr <a, X_2, Y'_2, rfl, rfl,
          f.mono hf hX_2X (jc_subset_jc.mp hW'W'_2) hX_2 hY'_2>)

/-- The sum map is strict: it sends `False = master` only to `master`. -/
theorem isStrict_sumMapSig (f : ApproximableMap V_0 V_1) :
    IsStrict (sumMapSig (A := A) (h_0 := h_0) (h_1 := h_1) f) := by
  rintro W' <-, -, hd>
  rcases hd with rfl | <hWU, -> | <a, X, Y', hWX, -, ->
  * rfl
  * exact absurd (hWU  |>  (show (none : SigTok A b_0)  in  (sumSig A V_0 h_0).master
    from
      none_mem_masterSig)) none_not_mem_jU
  * exact absurd (hWX  |>  (show (none : SigTok A b_0)  in  (sumSig A V_0 h_0).master
    from
      none_mem_masterSig)) none_not_mem_jc

@[simp] theorem sumMapSig_rel {f : ApproximableMap V_0 V_1} {W W'} :
    (sumMapSig (A := A) (h_0 := h_0) (h_1 := h_1) f).rel W W'  <->
      (sumSig A V_0 h_0).mem W  /\  (sumSig A V_1 h_1).mem W'  /\
        (W' = masterSig V_1  \/  (W = jU  /\  W' = jU)  \/
          exists  a X Y', W = jc a X  /\  W' = jc a Y'  /\  f.rel X Y') := Iff.rfl

/-- **Functoriality (composition): `Sigma(g comp f) = Sigmag  comp  Sigmaf`.** -/
theorem sumMapSig_comp (g : ApproximableMap V_1 V_2) (f : ApproximableMap V_0 V_1) :
    sumMapSig (A := A) (h_0 := h_0) (h_1 := h_2) (g.comp f)
      = (sumMapSig (A := A) (h_0 := h_1) (h_1 := h_2) g).comp
          (sumMapSig (A := A) (h_0 := h_0) (h_1 := h_1) f) := by
  apply ApproximableMap.ext
  intro W W''
  constructor
  * rintro <hW, hW'', hd>
    rcases hd with rfl | <hWU, rfl> | <a, X, Z'', rfl, rfl, Y', hf, hg>
    * exact <masterSig V_1, <hW, (sumSig A V_1 h_1).master_mem, Or.inl rfl>,
        (sumSig A V_1 h_1).master_mem, hW'', Or.inl rfl>
    * exact <jU, <hW, Or.inr (Or.inl rfl), Or.inr (Or.inl <hWU, rfl>)>,
        Or.inr (Or.inl rfl), hW'', Or.inr (Or.inl <rfl, rfl>)>
    * exact <jc a Y', <hW, Or.inr (Or.inr <a, Y', f.rel_cod hf, rfl>),
        Or.inr (Or.inr <a, X, Y', rfl, rfl, hf>)>,
        Or.inr (Or.inr <a, Y', f.rel_cod hf, rfl>), hW'',
        Or.inr (Or.inr <a, Y', Z'', rfl, rfl, hg>)>
  * rintro <W', <hW, hW', hdf>, -, hW'', hdg>
    refine <hW, hW'', ?_>
    rcases hdg with rfl | <hW'U, rfl> | <a, Y', Z'', hW'Y', rfl, hg>
    * exact Or.inl rfl
    * rcases hdf with rfl | <hWU, -> | <a, X, Y'_0, -, hW'eq, hf>
      * exact absurd ((hW'U.symm)  |>  none_mem_masterSig) none_not_mem_jU
      * exact Or.inr (Or.inl <hWU, rfl>)
      * obtain <t, ht> := h_1 Y'_0 (f.rel_cod hf)
        exact absurd ((hW'eq.symm.trans hW'U)  |>  tc_mem_jc.mpr ht) tc_not_mem_jU
    * rcases hdf with rfl | <-, hW'U> | <a', X, Y'_0, rfl, hW'eq, hf>
      * exact absurd ((hW'Y'.symm)  |>  none_mem_masterSig) none_not_mem_jc
      * obtain <t, ht> := h_1 Y' (g.rel_dom hg)
```

```
        exact absurd ((hW'U.symm.trans hW'Y')  |>  tc_mem_jc.mpr ht) tc_not_mem_jU
      * obtain <rfl, rfl> := jc_eq_jc (h_1 Y'_0 (f.rel_cod hf)) (hW'eq.symm.trans hW'Y
    ')
        exact Or.inr (Or.inr <a', X, Z'', rfl, rfl, <Y'_0, hf, hg>>)

/-- **Functoriality (identities): `Sigma(I) = I`.** -/
theorem sumMapSig_id :
    sumMapSig (A := A) (V_0 := V_0) (V_1 := V_0) (h_0 := h_0) (h_1 := h_0) (idMap V_0)
      = idMap (sumSig A V_0 h_0) := by
  apply ApproximableMap.ext
  intro W W'
  constructor
  * rintro <hW, hW', hd>
    refine <hW, hW', ?_>
    rcases hd with rfl | <rfl, rfl> | <a, X, Y', rfl, rfl, -, -, hXY>
    * exact (sumSig A V_0 h_0).sub_master hW
    * exact subset_rfl
    * exact jc_subset_jc.mpr hXY
  * rintro <hW, hW', hsub>
    refine <hW, hW', ?_>
    rcases hW with rfl | rfl | <a, X, hX, rfl>
    * exact Or.inl (eq_masterSig_of_subset hsub ((sumSig A V_0 h_0).sub_master hW'))
    * rcases hW' with rfl | rfl | <a', X', hX', rfl>
      * exact Or.inl rfl
      * exact Or.inr (Or.inl <rfl, rfl>)
      * exact absurd (hsub (show tu  in  jU from rfl)) tu_not_mem_jc
    * rcases hW' with rfl | rfl | <a', X', hX', rfl>
      * exact Or.inl rfl
      * obtain <t, ht> := h_0 X hX; exact absurd (hsub (tc_mem_jc.mpr ht))
    tc_not_mem_jU
      * obtain <t, ht> := h_0 X hX
        obtain <t', he, -> := hsub (tc_mem_jc.mpr ht)
        obtain <rfl, -> := tc_injective he
        exact Or.inr (Or.inr <a, X, X', rfl, rfl, hX, hX', jc_subset_jc.mp hsub>)

/-- `Sigmaf` on the `a`-copy injection: `(Sigmaf)(inj_a x) = inj_a (f x)`. -/
theorem sumMapSig_sinjC (f : ApproximableMap V_0 V_1) (a : A) (x : V_0.Element) :
    (sumMapSig (A := A) (h_0 := h_0) (h_1 := h_1) f).toElementMap (sinjC (h := h_0) a
    x)
      = sinjC (h := h_1) a (f.toElementMap x) := by
  apply NeighborhoodSystem.Element.ext
  intro W'
  constructor
  * rintro <U, hU, -, -, hd>
    rcases hd with rfl | <hUjU, rfl> | <a', X, Y', hUj, rfl, hf>
    * exact Or.inl rfl
    * rcases hU with hUm | <X_0, hX_0, hUeq, hx>
      * exact absurd ((hUm.symm.trans hUjU)  |>  none_mem_masterSig) none_not_mem_jU
      * obtain <t, ht> := h_0 X_0 hX_0
        exact absurd ((hUeq.symm.trans hUjU)  |>  tc_mem_jc.mpr ht) tc_not_mem_jU
    * rcases hU with hUm | <X_0, hX_0, hUeq, hx>
      * exact absurd ((hUm.symm.trans hUj)  |>  none_mem_masterSig) none_not_mem_jc
      * obtain <rfl, rfl> := jc_eq_jc (h_0 X_0 hX_0) (hUeq.symm.trans hUj)
        exact Or.inr <Y', f.rel_cod hf, rfl, <X_0, hx, hf>>
  * rintro (rfl | <Y', hY', rfl, hm>)
    * exact <masterSig V_0, Or.inl rfl, (sumSig A V_0 h_0).master_mem,
```

```
        (sumSig A V_1 h_1).master_mem, Or.inl rfl>
    * obtain <X, hx, hf> := hm
      exact <jc a X, Or.inr <X, x.sub hx, rfl, hx>, Or.inr (Or.inr <a, X, x.sub hx,
    rfl>),
        Or.inr (Or.inr <a, Y', f.rel_cod hf, rfl>), Or.inr (Or.inr <a, X, Y', rfl, rfl
    , hf>)>

/-- `Sigmaf` fixes the basepoint injection. -/
theorem sumMapSig_sinjU (f : ApproximableMap V_0 V_1) :
    (sumMapSig (A := A) (h_0 := h_0) (h_1 := h_1) f).toElementMap (sinjU (h := h_0))
      = sinjU (h := h_1) := by
  apply NeighborhoodSystem.Element.ext
  intro W'
  constructor
  * rintro <U, hU, -, -, hd>
    rcases hd with rfl | <-, rfl> | <a, X, Y', hUj, rfl, hf>
    * exact Or.inl rfl
    * exact Or.inr rfl
    * rcases hU with hUm | rfl
      * exact absurd ((hUm.symm.trans hUj)  |>  none_mem_masterSig) none_not_mem_jc
      * exact absurd (hUj  |>  (show tu  in  jU from rfl)) tu_not_mem_jc
  * rintro (rfl | rfl)
    * exact <masterSig V_0, Or.inl rfl, (sumSig A V_0 h_0).master_mem,
        (sumSig A V_1 h_1).master_mem, Or.inl rfl>
    * exact <jU, Or.inr rfl, Or.inr (Or.inl rfl), Or.inr (Or.inl rfl),
        Or.inr (Or.inl <rfl, rfl>)>

end SumMapSig

section SumSig

universe v

variable {b : Type v} {? : Type v}

variable (V : NeighborhoodSystem b)

variable {V}

theorem sumSig_nonempty {h : forall  X, V.mem X  ->  X.Nonempty} :
    forall  W, (sumSig A V h).mem W  ->  W.Nonempty := by
  rintro W (rfl | rfl | <a, X, hX, rfl>)
  * exact <none, none_mem_masterSig>
  * exact jU_nonempty
  * exact jc_nonempty (h X hX)

end SumSig

/-- `Tsig` on objects: `Tsig(D) = ? + Sigma_a D`, again `{}`-free (`sumSig_nonempty`).
    -/
abbrev tsigObj (A : Type) [DecidableEq A] (D : StrictDomainObj.{0}) : StrictDomainObj
    .{0} where
  carrier := SigTok A D.carrier
  sys := sumSig A D.sys D.nonempty
  nonempty := sumSig_nonempty
```

```
@[simp] theorem tsigObj_sys (A : Type) [DecidableEq A] (D : StrictDomainObj.{0}) :
    (tsigObj A D).sys = sumSig A D.sys D.nonempty := rfl

/-- `Tsig` on maps: `Tsig(f) = I_? + Sigma_a f`, strict by `isStrict_sumMapSig`. -/
def tsigMapHom (A : Type) [DecidableEq A] {D E : StrictDomainObj.{0}} (f : (D  -->  E)
    ) :
    ((tsigObj A D)  -->  (tsigObj A E)) :=
  <sumMapSig (A := A) (h_0 := D.nonempty) (h_1 := E.nonempty) f.1, isStrict_sumMapSig
    _>

@[simp] theorem tsigMapHom_val (A : Type) [DecidableEq A] {D E : StrictDomainObj.{0}}
    (f : (D  -->  E)) :
    (tsigMapHom A f).1 = sumMapSig (A := A) (h_0 := D.nonempty) (h_1 := E.nonempty) f
    .1 := rfl

/-- **The functor `Tsig(X) = ? + Sigma_{a:A} X`** on the category of `{}`-free domains
    and strict maps. -/
abbrev Tsig (A : Type) [DecidableEq A] : Endofunctor StrictDomainObj.{0} where
  obj := tsigObj A
  map := tsigMapHom A
  map_id _ := Subtype.ext sumMapSig_id
  map_comp {_ _ _} g f := Subtype.ext (sumMapSig_comp g.1 f.1)

@[simp] theorem Tsig_obj (A : Type) [DecidableEq A] (D : StrictDomainObj.{0}) :
    (Tsig A).obj D = tsigObj A D := rfl

@[simp] theorem Tsig_map_val (A : Type) [DecidableEq A] {D E : StrictDomainObj.{0}}
    (f : (D  -->  E)) :
    ((Tsig A).map f).1 = sumMapSig (A := A) (h_0 := D.nonempty) (h_1 := E.nonempty) f
    .1 := rfl

section Iso

variable {A : Type} [DecidableEq A] [Inhabited A]

/-- `aX = {a :: w' ? w'  in  X}`: the `a`-prefixed copy of a neighbourhood. -/
def embA (a : A) (X : Set (Strn A)) : Set (Strn A) := {w | exists  w', w = a :: w'  /\
     w'  in  X}

omit [DecidableEq A] [Inhabited A] in

@[simp] theorem mem_embA {a : A} {X : Set (Strn A)} {w : Strn A} :
    w  in  embA a X  <->  exists  w', w = a :: w'  /\  w'  in  X := Iff.rfl

/-- The right-hand side of the domain equation: the `A`-indexed sum `? + Sigma_a C?`.
    -/
abbrev CCn (A : Type) [DecidableEq A] : NeighborhoodSystem (SigTok A (Strn A)) :=
  sumSig A (Cn A) Cn_nonempty

omit [DecidableEq A] [Inhabited A] in

theorem embA_eq_prependN (a : A) (X : Set (Strn A)) : embA a X = prependN [a] X := by
  ext w
  simp only [mem_embA, mem_prependN]
  constructor
  * rintro <w', rfl, hX>; exact <w', hX, rfl>
```

```
  * rintro <t, hX, rfl>; exact <t, rfl, hX>

omit [DecidableEq A] [Inhabited A] in

theorem embA_coneN (a : A) (sigma : Strn A) : embA a (coneN sigma) = coneN (a :: sigma
    ) := by
  rw [embA_eq_prependN, prependN_coneN]; rfl

omit [DecidableEq A] [Inhabited A] in

theorem embA_inter (a : A) (X X' : Set (Strn A)) : embA a X  I  embA a X' = embA a (X
     I  X') := by
  ext w
  simp only [Set.mem_inter_iff, mem_embA]
  constructor
  * rintro <<w', rfl, hX>, w'', heq, hX'>
    rw [List.cons.injEq] at heq; obtain <-, rfl> := heq; exact <w', rfl, hX, hX'>
  * rintro <w', rfl, hX, hX'>; exact <<w', rfl, hX>, <w', rfl, hX'>>

omit [DecidableEq A] [Inhabited A] in

theorem embA_inter_ne {a a' : A} (h : a  !=  a') (X Y : Set (Strn A)) :
    embA a X  I  embA a' Y = {} := by
  ext w
  simp only [Set.mem_inter_iff, mem_embA, Set.mem_empty_iff_false, iff_false, not_and]
  rintro <w', rfl, -> <w'', heq, ->
  rw [List.cons.injEq] at heq; exact h heq.1

omit [DecidableEq A] [Inhabited A] in

theorem embA_singleton (a : A) (sigma : Strn A) : embA a ({sigma} : Set (Strn A)) = {a
     :: sigma} := by
  rw [embA_eq_prependN, prependN_singleton]; rfl

omit [DecidableEq A] [Inhabited A] in

theorem embA_subset {a : A} {X X' : Set (Strn A)} : embA a X  subseteq  embA a X'  <->
      X  subseteq  X' := by
  constructor
  * intro h w' hw'
    obtain <w'', heq, hX'> := h <w', rfl, hw'>
    rw [List.cons.injEq] at heq; obtain <-, rfl> := heq; exact hX'
  * rintro h w <w', rfl, hX>; exact <w', rfl, h hX>

omit [DecidableEq A] [Inhabited A] in

theorem embA_injective {a : A} {X X' : Set (Strn A)} (h : embA a X = embA a X') : X =
    X' :=
  Set.Subset.antisymm (embA_subset.mp h.subset) (embA_subset.mp h.symm.subset)

omit [DecidableEq A] [Inhabited A] in
/-- **The shape of a `C?`-neighbourhood.** Every neighbourhood is the master `Sigma*`,
    the terminator
`{?}`, or an `a`-copy `aX` with `X  in  C?`. -/
theorem memCn_cases {W : Set (Strn A)} (hW : memCn W) :
    W = Set.univ  \/  W = ({[]} : Set (Strn A))  \/  exists  a X, memCn X  /\  W =
```

```
    embA a X := by
  rcases hW with <sigma, rfl> | <sigma, rfl>
  * cases sigma with
    | nil => exact Or.inl coneN_nil
    | cons a sigma' => exact Or.inr (Or.inr <a, coneN sigma', memCn_coneN sigma', (
    embA_coneN a sigma').symm>)
  * cases sigma with
    | nil => exact Or.inr (Or.inl rfl)
    | cons a sigma' => exact Or.inr (Or.inr <a, {sigma'}, memCn_singleton sigma', (
    embA_singleton a sigma').symm>)

omit [Inhabited A] in

omit [DecidableEq A] in

theorem memCn_embA (a : A) {X : Set (Strn A)} (hX : memCn X) : memCn (embA a X) := by
  rw [embA_eq_prependN]; exact memCn_prependN [a] hX

omit [DecidableEq A] [Inhabited A] in

theorem memCn_embA_inv {a : A} {W : Set (Strn A)} (h : memCn (embA a W)) : memCn W :=
    by
  rcases h with <sigma, hsigma> | <sigma, hsigma>
  * have hmem : sigma  in  embA a W := hsigma  |>  (show sigma  in  coneN sigma from
    List.prefix_rfl)
    obtain <w', rfl, -> := hmem
    rw [ <-  embA_coneN] at hsigma; rw [embA_injective hsigma]; exact memCn_coneN w'
  * have hmem : sigma  in  embA a W := hsigma  |>  (Set.mem_singleton_iff.mpr rfl :
    sigma  in  ({sigma} : Set (Strn A)))
    obtain <w', rfl, -> := hmem
    rw [ <-  embA_singleton] at hsigma; rw [embA_injective hsigma]; exact
    memCn_singleton w'

omit [DecidableEq A] [Inhabited A] in

theorem nil_not_mem_embA {a : A} {X : Set (Strn A)} : ([] : Strn A)  notin  embA a X
    := by
  rintro <w', heq, ->; exact absurd heq (by simp)

omit [DecidableEq A] [Inhabited A] in

theorem embA_ne_univ (a : A) (X : Set (Strn A)) : embA a X  !=  Set.univ := by
  intro h; exact nil_not_mem_embA (X := X) (a := a) (by rw [h]; trivial)

omit [DecidableEq A] [Inhabited A] in

theorem singleton_nil_inter_embA (a : A) (X : Set (Strn A)) :
    (({[]} : Set (Strn A))  I  embA a X) = {} := by
  ext w
  simp only [Set.mem_inter_iff, Set.mem_singleton_iff, mem_embA, Set.
    mem_empty_iff_false,
    iff_false, not_and]
  rintro rfl <w', heq, ->; exact absurd heq (by simp)

omit [DecidableEq A] [Inhabited A] in
```

```
theorem singleton_nil_ne_embA (a : A) (X : Set (Strn A)) :
    ({[]} : Set (Strn A))  !=  embA a X := by
  intro h
  exact nil_not_mem_embA (h  |>  (Set.mem_singleton_iff.mpr rfl : ([] : Strn A)  in
    ({[]} : Set (Strn A))))

omit [DecidableEq A] in

theorem singleton_nil_ne_univ : ({[]} : Set (Strn A))  !=  Set.univ := by
  intro h
  have hmem : ([default] : Strn A)  in  ({[]} : Set (Strn A)) := by rw [h]; trivial
  rw [Set.mem_singleton_iff] at hmem; exact absurd hmem (by simp)

omit [Inhabited A] in

theorem sumSig_mem_jc_inv {a : A} {X : Set (Strn A)} (h : (CCn A).mem (jc a X)) :
    (Cn A).mem X := by
  rcases h with h0 | hU | <a', X', hX', heq>
  * exact absurd (h0  |>  none_mem_masterSig) none_not_mem_jc
  * have : (tu : SigTok A (Strn A))  in  jc a X := by rw [hU]; rfl
    exact absurd this tu_not_mem_jc
  * by_cases haa : a = a'
    * subst haa; rw [jc_injective heq]; exact hX'
    * obtain <t, ht> := Cn_nonempty X' hX'
      exact absurd (heq.symm  |>  (tc_mem_jc.mpr ht)) (tc_mem_jc_ne (Ne.symm haa))

/-- **Forward half of `C?  ~=  ? + Sigma_a C?`.** Records, for each branch, whether `x`
    finishes at `?`
(the `?`-summand) or reaches the `a`-copy `aX` (the `a`-th summand). -/
def toCC (x : (Cn A).Element) : (CCn A).Element where
  mem W := W = masterSig (Cn A)
     \/  (W = jU  /\  x.mem ({[]} : Set (Strn A)))
     \/  (exists  a X, (Cn A).mem X  /\  W = jc a X  /\  x.mem (embA a X))
  sub := by
    rintro W (rfl | <rfl, -> | <a, X, hX, rfl, ->)


    * exact Or.inl rfl
    * exact Or.inr (Or.inl rfl)
    * exact Or.inr (Or.inr <a, X, hX, rfl>)
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | <rfl, hzU> | <a, X, hX, rfl, hzF>)
      (rfl | <rfl, hzU'> | <a', X', hX', rfl, hzF'>)
    * exact Or.inl (by rw [Set.inter_self])
    * exact Or.inr (Or.inl <by rw [masterSig_inter_jU], hzU'>)
    * exact Or.inr (Or.inr <a', X', hX', by rw [masterSig_inter_jc hX'], hzF'>)
    * exact Or.inr (Or.inl <by rw [Set.inter_comm, masterSig_inter_jU], hzU>)
    * exact Or.inr (Or.inl <by rw [Set.inter_self], hzU>)
    * exfalso
      have hx := x.inter_mem hzU hzF'; rw [singleton_nil_inter_embA] at hx
      obtain <t, ht> := Cn_nonempty _ (x.sub hx); exact Set.notMem_empty t ht
    * exact Or.inr (Or.inr <a, X, hX, by rw [Set.inter_comm, masterSig_inter_jc hX],
    hzF>)
    * exfalso
      have hx := x.inter_mem hzF hzU'; rw [Set.inter_comm, singleton_nil_inter_embA]
    at hx
```

```
      obtain <t, ht> := Cn_nonempty _ (x.sub hx); exact Set.notMem_empty t ht
    * by_cases haa : a = a'
      * subst haa
        have hx := x.inter_mem hzF hzF'; rw [embA_inter] at hx
        exact Or.inr (Or.inr <a, X  I  X', memCn_embA_inv (x.sub hx), jc_inter_jc_same
     a X X', hx>)
      * exfalso
        have hx := x.inter_mem hzF hzF'; rw [embA_inter_ne haa] at hx
        obtain <t, ht> := Cn_nonempty _ (x.sub hx); exact Set.notMem_empty t ht
  up_mem := by
    rintro W W' (rfl | <rfl, hzU> | <a, X, hX, rfl, hzF>) hW' hsub
    * exact Or.inl (eq_masterSig_of_subset hsub ((CCn A).sub_master hW'))
    * rcases hW' with rfl | rfl | <a', X', hX', rfl>
      * exact Or.inl rfl
      * exact Or.inr (Or.inl <rfl, hzU>)
      * exact absurd (hsub (show tu  in  jU from rfl)) tu_not_mem_jc
    * rcases hW' with rfl | rfl | <a', X', hX', rfl>
      * exact Or.inl rfl
      * obtain <t, ht> := Cn_nonempty X hX
        exact absurd (hsub (tc_mem_jc.mpr ht)) tc_not_mem_jU
      * obtain <t, ht> := Cn_nonempty X hX
        obtain <t', he, -> := hsub (tc_mem_jc.mpr ht)
        obtain <rfl, -> := tc_injective he
        exact Or.inr (Or.inr <a, X', hX', rfl,
          x.up_mem hzF (memCn_embA a hX') (embA_subset.mpr (jc_subset_jc.mp hsub))>)

omit [Inhabited A] in

@[simp] theorem toCC_mem_jU {x : (Cn A).Element} :
    (toCC x).mem (jU : Set (SigTok A (Strn A)))  <->  x.mem ({[]} : Set (Strn A)) :=
    by
  constructor
  * rintro (h0 | <-, hz> | <a', X', hX', heq, hz>)
    * exact absurd (h0.symm  |>  none_mem_masterSig) none_not_mem_jU
    * exact hz
    * exact absurd (heq  |>  (show tu  in  jU from rfl)) tu_not_mem_jc
  * intro hz; exact Or.inr (Or.inl <rfl, hz>)

omit [Inhabited A] in

@[simp] theorem toCC_mem_jc {x : (Cn A).Element} {a : A} {X : Set (Strn A)} (hX : (Cn
    A).mem X) :
    (toCC x).mem (jc a X)  <->  x.mem (embA a X) := by
  constructor
  * rintro (h0 | <heq, hz> | <a', X', hX', heqj, hz>)
    * exact absurd (h0  |>  none_mem_masterSig) none_not_mem_jc
    * obtain <t, ht> := Cn_nonempty X hX
      exact absurd (heq  |>  (tc_mem_jc.mpr ht)) tc_not_mem_jU
    * obtain <rfl, rfl> := jc_eq_jc (Cn_nonempty X hX) heqj
      exact hz
  * intro hz; exact Or.inr (Or.inr <a, X, hX, rfl, hz>)

omit [DecidableEq A] [Inhabited A] in
/-- Prefixed copies determine index and set: `aX = a'X'` (with `X` non-empty) forces `a
    =a'`, `X=X'`. -/
theorem embA_eq_embA {a a' : A} {X X' : Set (Strn A)} (hXne : X.Nonempty)
```

```
    (h : embA a X = embA a' X') : a = a'  /\  X = X' := by
  obtain <w', hw'> := hXne
  have hmem : (a :: w')  in  embA a X := <w', rfl, hw'>
  rw [h] at hmem
  obtain <u, he, -> := hmem
  rw [List.cons.injEq] at he; obtain <rfl, -> := he
  exact <rfl, embA_injective h>

/-- **Inverse half of `C?  ~=  ? + Sigma_a C?`.** -/
def fromCC (s : (CCn A).Element) : (Cn A).Element where
  mem W := W = Set.univ
     \/  (W = ({[]} : Set (Strn A))  /\  s.mem jU)
     \/  (exists  a X, (Cn A).mem X  /\  W = embA a X  /\  s.mem (jc a X))
  sub := by
    rintro W (rfl | <rfl, -> | <a, X, hX, rfl, ->)
    * exact Or.inl <[], coneN_nil.symm>
    * exact memCn_singleton []
    * exact memCn_embA a hX
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | <rfl, hsU> | <a, X, hX, rfl, hsF>)
      (rfl | <rfl, hsU'> | <a', X', hX', rfl, hsF'>)
    * exact Or.inl (by rw [Set.inter_self])
    * exact Or.inr (Or.inl <by rw [Set.univ_inter], hsU'>)
    * exact Or.inr (Or.inr <a', X', hX', by rw [Set.univ_inter], hsF'>)
    * exact Or.inr (Or.inl <by rw [Set.inter_univ], hsU>)
    * exact Or.inr (Or.inl <by rw [Set.inter_self], hsU>)
    * exfalso
      have hs := s.inter_mem hsU hsF'; rw [jU_inter_jc] at hs
      obtain <t, ht> := sumSig_nonempty _ (s.sub hs); exact Set.notMem_empty t ht
    * exact Or.inr (Or.inr <a, X, hX, by rw [Set.inter_univ], hsF>)
    * exfalso
      have hs := s.inter_mem hsF hsU'; rw [Set.inter_comm, jU_inter_jc] at hs
      obtain <t, ht> := sumSig_nonempty _ (s.sub hs); exact Set.notMem_empty t ht
    * by_cases haa : a = a'
      * subst haa
        have hs := s.inter_mem hsF hsF'; rw [jc_inter_jc_same] at hs
        exact Or.inr (Or.inr <a, X  I  X', sumSig_mem_jc_inv (s.sub hs), embA_inter a
   X X', hs>)
      * exfalso
        have hs := s.inter_mem hsF hsF'; rw [jc_inter_jc_ne haa] at hs
        obtain <t, ht> := sumSig_nonempty _ (s.sub hs); exact Set.notMem_empty t ht
  up_mem := by
    rintro W W' (rfl | <rfl, hsU> | <a, X, hX, rfl, hsF>) hW' hsub
    * exact Or.inl (Set.univ_subset_iff.mp hsub)
    * rcases memCn_cases hW' with rfl | rfl | <a', X', hX', rfl>
      * exact Or.inl rfl
      * exact Or.inr (Or.inl <rfl, hsU>)
      * exact absurd (hsub (Set.mem_singleton_iff.mpr rfl)) nil_not_mem_embA
    * rcases memCn_cases hW' with rfl | rfl | <a', X', hX', rfl>
      * exact Or.inl rfl
      * obtain <t, ht> := Cn_nonempty X hX
        have hm := hsub (<t, rfl, ht> : (a :: t)  in  embA a X)
        rw [Set.mem_singleton_iff] at hm; exact absurd hm (by simp)
      * obtain <t, ht> := Cn_nonempty X hX
        obtain <w', he, -> := hsub (<t, rfl, ht> : (a :: t)  in  embA a X)
```

```
        rw [List.cons.injEq] at he; obtain <rfl, -> := he
        refine Or.inr (Or.inr <a, X', hX', rfl, ?_>)
        exact s.up_mem hsF (Or.inr (Or.inr <a, X', hX', rfl>))
          (jc_subset_jc.mpr (embA_subset.mp hsub))

@[simp] theorem fromCC_mem_nil {s : (CCn A).Element} :
    (fromCC s).mem ({[]} : Set (Strn A))  <->  s.mem (jU : Set (SigTok A (Strn A))) :=
     by
  constructor
  * rintro (h0 | <-, hs> | <a', X', hX', heq, hs>)
    * exact absurd h0 singleton_nil_ne_univ
    * exact hs
    * exact absurd heq (singleton_nil_ne_embA a' X')
  * intro hs; exact Or.inr (Or.inl <rfl, hs>)

omit [Inhabited A] in

@[simp] theorem fromCC_mem_embA {s : (CCn A).Element} {a : A} {X : Set (Strn A)} (hX :
     (Cn A).mem X) :
    (fromCC s).mem (embA a X)  <->  s.mem (jc a X) := by
  constructor
  * rintro (h0 | <heq, hs> | <a', X', hX', heqj, hs>)
    * exact absurd h0 (embA_ne_univ a X)
    * exact absurd heq.symm (singleton_nil_ne_embA a X)
    * obtain <rfl, rfl> := embA_eq_embA (Cn_nonempty X hX) heqj
      exact hs
  * intro hs; exact Or.inr (Or.inr <a, X, hX, rfl, hs>)

omit [Inhabited A] in

theorem fromCC_toCC (x : (Cn A).Element) : fromCC (toCC x) = x := by
  apply NeighborhoodSystem.Element.ext
  intro W
  constructor
  * rintro (rfl | <rfl, hs> | <a, X, hX, rfl, hs>)
    * exact x.master_mem
    * exact toCC_mem_jU.mp hs
    * exact (toCC_mem_jc hX).mp hs
  * intro hW
    rcases memCn_cases (x.sub hW) with rfl | rfl | <a, X, hX, rfl>
    * exact Or.inl rfl
    * exact Or.inr (Or.inl <rfl, toCC_mem_jU.mpr hW>)
    * exact Or.inr (Or.inr <a, X, hX, rfl, (toCC_mem_jc hX).mpr hW>)

theorem toCC_fromCC (s : (CCn A).Element) : toCC (fromCC s) = s := by
  apply NeighborhoodSystem.Element.ext
  intro W
  constructor
  * rintro (rfl | <rfl, hs> | <a, X, hX, rfl, hs>)
    * exact s.master_mem
    * exact fromCC_mem_nil.mp hs
    * exact (fromCC_mem_embA hX).mp hs
  * intro hW
    rcases s.sub hW with rfl | rfl | <a, X, hX, rfl>
    * exact Or.inl rfl
    * exact Or.inr (Or.inl <rfl, fromCC_mem_nil.mpr hW>)
```

```
    * exact Or.inr (Or.inr <a, X, hX, rfl, (fromCC_mem_embA hX).mpr hW>)

/-- **The isomorphism `|C?| ?o |? + Sigma_a C?|`.** -/
def ccEquiv : (Cn A).Element ?o (CCn A).Element where
  toFun := toCC
  invFun := fromCC
  left_inv := fromCC_toCC
  right_inv := toCC_fromCC
  map_rel_iff' := by
    intro x x'
    constructor
    * intro hle W hW
      rcases memCn_cases (x.sub hW) with rfl | rfl | <a, X, hX, rfl>
      * exact x'.master_mem
      * exact toCC_mem_jU.mp (hle _ (Or.inr (Or.inl <rfl, hW>)))
      * exact (toCC_mem_jc hX).mp (hle _ (Or.inr (Or.inr <a, X, hX, rfl, hW>)))
    * intro hle W hW
      rcases hW with rfl | <rfl, hz> | <a, X, hX, rfl, hz>
      * exact Or.inl rfl
      * exact Or.inr (Or.inl <rfl, hle _ hz>)
      * exact Or.inr (Or.inr <a, X, hX, rfl, hle _ hz>)

@[simp] theorem ccEquiv_apply (x : (Cn A).Element) : ccEquiv x = toCC x := rfl

end Iso

section Algebra

variable {A : Type} [DecidableEq A] [Inhabited A]

/-- `C?` as an object of the `{}`-free category. -/
abbrev Cnobj (A : Type) [DecidableEq A] : StrictDomainObj.{0} := <Strn A, Cn A,
    Cn_nonempty>

@[simp] theorem Cnobj_sys (A : Type) [DecidableEq A] : (Cnobj A).sys = Cn A := rfl

/-- **The `Tsig`-algebra structure on `C?`.** The structure map `i : ?+Sigma_a C?  ->
    C?` is the inverse of
the domain-equation isomorphism `ccEquiv`, strict by `isStrict_ofIso`. -/
abbrev cnStr : (((Tsig A).obj (Cnobj A))  -->  (Cnobj A)) :=
  <ofIso ccEquiv.symm, isStrict_ofIso _>

/-- **`C?` is a `Tsig`-algebra** for `Tsig(X) = ? + Sigma_a X`. -/
abbrev Cnalg (A : Type) [DecidableEq A] [Inhabited A] : TAlgebra (Tsig A) := <Cnobj A,
     cnStr>

end Algebra

section Iso

variable {A : Type} [DecidableEq A] [Inhabited A]

omit [Inhabited A] in

theorem consMapN_mem_embA {a : A} {z : (Cn A).Element} {X : Set (Strn A)} (hX : (Cn A)
    .mem X) :
```

```
    ((consMapN a).toElementMap z).mem (embA a X)  <->  z.mem X := by
  constructor
  * rintro <X', hzX', -, -, hsub>
    rw [ <-  embA_eq_prependN] at hsub
    exact z.up_mem hzX' hX (embA_subset.mp hsub)
  * intro hz
    refine <X, hz, z.sub hz, memCn_embA a hX, ?_>
    rw [ <-  embA_eq_prependN]

omit [Inhabited A] in

theorem consMapN_not_mem_embA_ne {a c : A} (hac : a  !=  c) {z : (Cn A).Element} {X :
    Set (Strn A)} :
    not  ((consMapN a).toElementMap z).mem (embA c X) := by
  rintro <X', hzX', hX'mem, -, hsub>
  obtain <t, ht> := Cn_nonempty X' hX'mem
  rw [ <-  embA_eq_prependN] at hsub
  obtain <w, hw, -> := hsub <t, rfl, ht>
  rw [List.cons.injEq] at hw; exact hac hw.1

omit [Inhabited A] in

theorem consMapN_not_mem_nil {a : A} {z : (Cn A).Element} :
    not  ((consMapN a).toElementMap z).mem ({[]} : Set (Strn A)) := by
  rintro <X', hzX', hX'mem, -, hsub>
  obtain <t, ht> := Cn_nonempty X' hX'mem
  rw [ <-  embA_eq_prependN] at hsub
  have hmem := hsub <t, rfl, ht>
  rw [Set.mem_singleton_iff] at hmem; exact absurd hmem (by simp)

end Iso

section Initial

variable {A : Type} [DecidableEq A] [Inhabited A] (B : TAlgebra (Tsig A))

/-- The distinguished point `e = k(inj_?)`: the image under `k` of the terminator. -/
def descE : B.carrier.sys.Element :=
  B.str.1.toElementMap (sinjU (h := B.carrier.nonempty))

/-- The `a`-th successor operation `f_a = k  comp  inj_a`. -/
def descF (a : A) (y : B.carrier.sys.Element) : B.carrier.sys.Element :=
  B.str.1.toElementMap (sinjC (h := B.carrier.nonempty) a y)

omit [Inhabited A] in

theorem descF_mono (a : A) {y y' : B.carrier.sys.Element} (h : y  <=  y') :
    descF B a y  <=  descF B a y' :=
  B.str.1.toElementMap_mono (sinjC_mono h)

/-- The recursion `phi(?)=z`, `phi(a*sigma)=f_a(phi(sigma))` on a finite string, with
    base value `z`. -/
def descVal (z : B.carrier.sys.Element) : Strn A  ->  B.carrier.sys.Element
  | [] => z
  | a :: sigma => descF B a (descVal z sigma)
```

```
omit [Inhabited A] in

theorem descVal_append (z : B.carrier.sys.Element) (sigma ? : Strn A) :
    descVal B z (sigma ++ ?) = descVal B (descVal B z ?) sigma := by
  induction sigma with
  | nil => rfl
  | cons a sigma ih => exact congrArg (descF B a) ih

omit [Inhabited A] in

theorem descVal_mono_z {z z' : B.carrier.sys.Element} (h : z  <=  z') :
    forall  sigma, descVal B z sigma  <=  descVal B z' sigma
  | [] => h
  | _ :: sigma => descF_mono B _ (descVal_mono_z h sigma)

omit [Inhabited A] in

theorem descMap_hcone {sigma tau : Strn A} (h : sigma <+: tau) :
    descVal B B.carrier.sys.bot sigma  <=  descVal B B.carrier.sys.bot tau := by
  obtain <?, rfl> := h
  rw [descVal_append]
  exact descVal_mono_z B (B.carrier.sys.bot_le _) sigma

omit [Inhabited A] in

theorem descMap_hsing {sigma tau : Strn A} (h : sigma <+: tau) :
    descVal B B.carrier.sys.bot sigma  <=  descVal B (descE B) tau := by
  obtain <?, rfl> := h
  rw [descVal_append]
  exact descVal_mono_z B (B.carrier.sys.bot_le _) sigma

end Initial

section Lift

variable {A : Type} [DecidableEq A] [Inhabited A] {b : Type}

omit [Inhabited A] in
/-- Two maps out of `C?` agree once they agree on every `sigmaFalse` and `sigma` (
    generic `Exercise516.map_ext_C`). -/
theorem map_ext_Cn {V : NeighborhoodSystem b} {f g : ApproximableMap (Cn A) V}
    (hbot : forall  sigma, f.toElementMap (strBotN sigma) = g.toElementMap (strBotN
    sigma))
    (helem : forall  sigma, f.toElementMap (strElemN sigma) = g.toElementMap (strElemN
     sigma)) : f = g := by
  apply eq_of_toElementMap_principal
  intro X hX
  obtain (<sigma, rfl> | <sigma, rfl>) := (Cn_mem.mp hX)
  * exact hbot sigma
  * exact helem sigma

omit [DecidableEq A] in
/-- A cone is never contained in a singleton: it has the two distinct elements `tau`
    and `tau*default`. -/
theorem not_coneN_subset_singleton (tau sigma : Strn A) : not  coneN tau  subseteq  ({
    sigma} : Set (Strn A)) := by
```

```
  intro h
  have h1 : tau  in  ({sigma} : Set (Strn A)) := h (show tau  in  coneN tau from List.
    prefix_rfl)
  have h2 : (tau ++ [default])  in  ({sigma} : Set (Strn A)) := h (List.prefix_append
    tau [default])
  rw [Set.mem_singleton_iff] at h1 h2
  have : tau = tau ++ [default] := h1.trans h2.symm
  simp at this

omit [DecidableEq A] in

theorem coneN_ne_singleton (tau sigma : Strn A) : coneN tau  !=  ({sigma} : Set (Strn
    A)) := fun h =>
  not_coneN_subset_singleton tau sigma (h  |>  subset_rfl)

/-- A map `C?  ->  V` determined by its value `coneVal sigma` on each partial element `
    sigmaFalse` and `singVal sigma`
on each total element `sigma`. (The alphabet-generic copy of `Exercise419.liftC`.) -/
def liftCn (V : NeighborhoodSystem b) (coneVal singVal : Strn A  ->  V.Element)
    (hcone : forall  {sigma tau : Strn A}, sigma <+: tau  ->  coneVal sigma  <=
    coneVal tau)
    (hsing : forall  {sigma tau : Strn A}, sigma <+: tau  ->  coneVal sigma  <=
    singVal tau) :
    ApproximableMap (Cn A) V where
  rel X Y := (exists  sigma, X = coneN sigma  /\  (coneVal sigma).mem Y)  \/  (exists
     sigma, X = {sigma}  /\  (singVal sigma).mem Y)
  rel_dom := by
    rintro X Y (<sigma, rfl, _> | <sigma, rfl, _>)
    * exact memCn_coneN sigma
    * exact memCn_singleton sigma
  rel_cod := by
    rintro X Y (<sigma, _, hY> | <sigma, _, hY>)
    * exact (coneVal sigma).sub hY
    * exact (singVal sigma).sub hY
  master_rel := by
    refine Or.inl <[], ?_, (coneVal []).master_mem>
    rw [Cn_master]; exact coneN_nil.symm
  inter_right := by
    rintro X Y Y' (<sigma, rfl, hY> | <sigma, rfl, hY>) (<sigma', hX', hY'> | <sigma',
     hX', hY'>)
    * have hsigmasigma : sigma = sigma' := coneN_injective hX'
      subst hsigmasigma
      exact Or.inl <sigma, rfl, (coneVal sigma).inter_mem hY hY'>
    * exact absurd hX' (coneN_ne_singleton sigma sigma')
    * exact absurd hX'.symm (coneN_ne_singleton sigma' sigma)
    * have hsigmasigma : sigma = sigma' := by rw [Set.singleton_eq_singleton_iff] at
    hX'; exact hX'
      subst hsigmasigma
      exact Or.inr <sigma, rfl, (singVal sigma).inter_mem hY hY'>
  mono := by
    rintro X X' Y Y' (<sigma, rfl, hY> | <sigma, rfl, hY>) hX'X hYY' hX' hY'
    * rcases hX' with <tau, rfl> | <tau, rfl>
      * have hpre : sigma <+: tau := coneN_subset_coneN.mp hX'X
        exact Or.inl <tau, rfl, (coneVal tau).up_mem (hcone hpre Y hY) hY' hYY'>
      * have hpre : sigma <+: tau := singleton_subset_coneN.mp hX'X
        exact Or.inr <tau, rfl, (singVal tau).up_mem (hsing hpre Y hY) hY' hYY'>
```

```
    * rcases hX' with <tau, rfl> | <tau, rfl>
      * exact absurd hX'X (not_coneN_subset_singleton tau sigma)
      * have htausigma : tau = sigma := by
          have hmem := Set.singleton_subset_iff.mp hX'X
          rwa [Set.mem_singleton_iff] at hmem
        subst htausigma
        exact Or.inr <tau, rfl, (singVal tau).up_mem hY hY' hYY'>

end Lift

section Initial

variable {A : Type} [DecidableEq A] [Inhabited A] (B : TAlgebra (Tsig A))

/-- **The homomorphism `C?  ->  E`**, built by `liftCn` from the head-recursion. -/
def descMap : ApproximableMap (Cn A) B.carrier.sys :=
  liftCn B.carrier.sys (descVal B B.carrier.sys.bot) (descVal B (descE B))


    (fun {_ _} => descMap_hcone B) (fun {_ _} => descMap_hsing B)

end Initial

section Lift

variable {A : Type} [DecidableEq A] [Inhabited A] {b : Type}

theorem liftCn_strBot (V : NeighborhoodSystem b) (coneVal singVal : Strn A  ->  V.
    Element)
    (hcone : forall  {sigma tau : Strn A}, sigma <+: tau  ->  coneVal sigma  <=
    coneVal tau)
    (hsing : forall  {sigma tau : Strn A}, sigma <+: tau  ->  coneVal sigma  <=
    singVal tau) (sigma : Strn A) :
    (liftCn V coneVal singVal hcone hsing).toElementMap (strBotN sigma) = coneVal
    sigma := by
  apply NeighborhoodSystem.Element.ext
  intro Y
  constructor
  * rintro <X, <_, hsub>, hrel>
    rcases hrel with <sigma', hXcone, hY> | <sigma', hXsing, hY>
    * have hpre : sigma' <+: sigma := coneN_subset_coneN.mp (hXcone  |>  hsub)
      exact hcone hpre Y hY
    * exact absurd (hXsing  |>  hsub) (not_coneN_subset_singleton sigma sigma')
  * intro hY
    exact <coneN sigma, <memCn_coneN sigma, subset_rfl>, Or.inl <sigma, rfl, hY>>

end Lift

section Initial

variable {A : Type} [DecidableEq A] [Inhabited A] (B : TAlgebra (Tsig A))

@[simp] theorem descMap_strBot (sigma : Strn A) :
    (descMap B).toElementMap (strBotN sigma) = descVal B B.carrier.sys.bot sigma :=
  liftCn_strBot _ _ _ _ _ sigma

end Initial
```

```
section Lift

variable {A : Type} [DecidableEq A] [Inhabited A] {b : Type}

theorem liftCn_strElem (V : NeighborhoodSystem b) (coneVal singVal : Strn A  ->  V.
    Element)
    (hcone : forall  {sigma tau : Strn A}, sigma <+: tau  ->  coneVal sigma  <=
    coneVal tau)
    (hsing : forall  {sigma tau : Strn A}, sigma <+: tau  ->  coneVal sigma  <=
    singVal tau) (sigma : Strn A) :
    (liftCn V coneVal singVal hcone hsing).toElementMap (strElemN sigma) = singVal
    sigma := by
  apply NeighborhoodSystem.Element.ext
  intro Y
  constructor
  * rintro <X, <_, hsub>, hrel>
    rcases hrel with <sigma', hXcone, hY> | <sigma', hXsing, hY>
    * have hpre : sigma' <+: sigma := by
        apply singleton_subset_coneN.mp; rw [ <-  hXcone]; exact hsub
      exact hsing hpre Y hY
    * have hsigmasigma' : sigma = sigma' := by
        have hmem := Set.singleton_subset_iff.mp (hXsing  |>  hsub)
        rwa [Set.mem_singleton_iff] at hmem
      subst hsigmasigma'; exact hY
  * intro hY
    exact <{sigma}, <memCn_singleton sigma, subset_rfl>, Or.inr <sigma, rfl, hY>>

end Lift

section Initial

variable {A : Type} [DecidableEq A] [Inhabited A] (B : TAlgebra (Tsig A))

@[simp] theorem descMap_strElem (sigma : Strn A) :
    (descMap B).toElementMap (strElemN sigma) = descVal B (descE B) sigma :=
  liftCn_strElem _ _ _ _ _ sigma

omit [Inhabited A] in

theorem Cn_bot_eq_strBotN_nil : (Cn A).bot = strBotN ([] : Strn A) := by
  apply NeighborhoodSystem.Element.ext
  intro Y
  show ((Cn A).mem Y  /\  (Cn A).master  subseteq  Y)  <->  ((Cn A).mem Y  /\  coneN
    []  subseteq  Y)
  rw [Cn_master, coneN_nil]

theorem ccEquiv_comp_symm :
    (ofIso (ccEquiv (A := A))).comp (ofIso ccEquiv.symm) = idMap (CCn A) := by
  apply ext_of_toElementMap
  intro s
  rw [toElementMap_comp, toElementMap_ofIso, toElementMap_ofIso, toElementMap_idMap]
  exact ccEquiv.apply_symm_apply s

theorem ccEquiv_symm_comp :
    (ofIso (ccEquiv (A := A)).symm).comp (ofIso ccEquiv) = idMap (Cn A) := by
```

```
  apply ext_of_toElementMap
  intro x
  rw [toElementMap_comp, toElementMap_ofIso, toElementMap_ofIso, toElementMap_idMap]
  exact ccEquiv.symm_apply_apply x

theorem descMap_strict : IsStrict (descMap B) := by
  rw [isStrict_iff_apply_bot, Cn_bot_eq_strBotN_nil, descMap_strBot]
  rfl

/-- The bundled strict homomorphism `C?  ->  E`. -/
def descStrict : ((Cnobj A)  -->  B.carrier) := <descMap B, descMap_strict B>

theorem genKeyBot (g : ApproximableMap (Cn A) B.carrier.sys) :
    B.str.1.toElementMap ((sumMapSig (A := A) (h_0 := Cn_nonempty) (h_1 := B.carrier.
    nonempty)
        g).toElementMap (toCC (strBotN ([] : Strn A))))
      = B.carrier.sys.bot := by
  have hb : toCC (strBotN ([] : Strn A)) = (CCn A).bot := by
    rw [ <-  Cn_bot_eq_strBotN_nil,  <-  ccEquiv_apply]; exact ccEquiv.map_bot
  rw [hb, isStrict_iff_apply_bot.mp (isStrict_sumMapSig (A := A) (h_0 := Cn_nonempty)
    (h_1 := B.carrier.nonempty) g)]
  exact isStrict_iff_apply_bot.mp B.str.2

end Initial

section Iso

variable {A : Type} [DecidableEq A] [Inhabited A]
omit [Inhabited A] in

/-- **`toCC  comp  (a*) = inj_a`.** Prepending the letter `a` to `z` is, across `C?  ~=
      ?+Sigma_a C?`, the
injection of `z` into the `a`-th summand. -/
theorem toCC_consMapN (a : A) (z : (Cn A).Element) :
    toCC ((consMapN a).toElementMap z) = sinjC (h := Cn_nonempty) a z := by
  apply NeighborhoodSystem.Element.ext
  intro W
  constructor
  * rintro (rfl | <rfl, hz> | <c, X, hX, rfl, hz>)
    * exact Or.inl rfl
    * exact absurd hz consMapN_not_mem_nil
    * by_cases hac : a = c
      * subst hac; exact Or.inr <X, hX, rfl, (consMapN_mem_embA hX).mp hz>
      * exact absurd hz (consMapN_not_mem_embA_ne hac)
  * rintro (rfl | <X, hX, rfl, hm>)
    * exact Or.inl rfl
    * exact Or.inr (Or.inr <a, X, hX, rfl, (consMapN_mem_embA hX).mpr hm>)

end Iso

section Initial

variable {A : Type} [DecidableEq A] [Inhabited A] (B : TAlgebra (Tsig A))

omit [Inhabited A] in
```

```
theorem genKey (g : ApproximableMap (Cn A) B.carrier.sys) (a : A) (w : (Cn A).Element)
     :
    B.str.1.toElementMap ((sumMapSig (A := A) (h_0 := Cn_nonempty) (h_1 := B.carrier.
    nonempty)
        g).toElementMap (toCC ((consMapN a).toElementMap w)))
      = descF B a (g.toElementMap w) := by
  rw [toCC_consMapN, sumMapSig_sinjC]; rfl

end Initial

section Iso

variable {A : Type} [DecidableEq A] [Inhabited A]
omit [Inhabited A] in

/-- **`toCC Lambda-hat = inj_?`.** The finished empty sequence is the terminator (the
    `?`-summand). -/
theorem toCC_strElemN_nil :
    toCC (strElemN ([] : Strn A)) = sinjU (h := Cn_nonempty) := by
  apply NeighborhoodSystem.Element.ext
  intro W
  constructor
  * rintro (rfl | <rfl, hz> | <a, X, hX, rfl, hz>)
    * exact Or.inl rfl
    * exact Or.inr rfl
    * exact absurd (hz.2 (Set.mem_singleton_iff.mpr rfl)) nil_not_mem_embA
  * rintro (rfl | rfl)
    * exact Or.inl rfl
    * exact Or.inr (Or.inl <rfl, memCn_singleton [], subset_rfl>)

end Iso

section Initial

variable {A : Type} [DecidableEq A] [Inhabited A] (B : TAlgebra (Tsig A))

omit [Inhabited A] in

theorem genKey0 (g : ApproximableMap (Cn A) B.carrier.sys) :
    B.str.1.toElementMap ((sumMapSig (A := A) (h_0 := Cn_nonempty) (h_1 := B.carrier.
    nonempty)
        g).toElementMap (toCC (strElemN ([] : Strn A))))
      = descE B := by
  rw [toCC_strElemN_nil, sumMapSig_sinjU]; rfl

/-- `C?`'s algebra map satisfies the recursion. -/
theorem descMap_satisfiesRec :
    descMap B = (B.str.1.comp (sumMapSig (A := A) (h_0 := Cn_nonempty) (h_1 := B.
    carrier.nonempty)
        (descMap B))).comp (ofIso ccEquiv) := by
  apply map_ext_Cn
  * intro sigma
    rw [toElementMap_comp, toElementMap_comp, toElementMap_ofIso, descMap_strBot]
    cases sigma with
    | nil => exact (genKeyBot B (descMap B)).symm
    | cons a sigma =>
```

```
      rw [ccEquiv_apply,  <-  consMapN_strBot]
      have h := genKey B (descMap B) a (strBotN sigma)
      rw [descMap_strBot] at h
      exact h.symm
  * intro sigma
    rw [toElementMap_comp, toElementMap_comp, toElementMap_ofIso, descMap_strElem]
    cases sigma with
    | nil => exact (genKey0 B (descMap B)).symm
    | cons a sigma =>
      rw [ccEquiv_apply,  <-  consMapN_strElem]
      have h := genKey B (descMap B) a (strElemN sigma)
      rw [descMap_strElem] at h
      exact h.symm

/-- **The homomorphism square** `desc  comp  i = k  comp  T(desc)`. -/
theorem descComm : (descMap B).comp (ofIso ccEquiv.symm)
    = B.str.1.comp (sumMapSig (A := A) (h_0 := Cn_nonempty) (h_1 := B.carrier.nonempty
    )
        (descMap B)) := by
  conv_lhs => rw [descMap_satisfiesRec B]
  rw [comp_assoc, ccEquiv_comp_symm, comp_idMap]

/-- **The descent homomorphism `(C?, i)  ->  (E, k)`** as a `Tsig`-algebra homomorphism
    . -/
def descAlgHom : AlgHom (Cnalg A) B where
  hom := descStrict B
  comm := by
    apply Subtype.ext
    simp only [StrictDomainObj.comp_val]
    exact descComm B

/-- **Any map satisfying the homomorphism recursion equals `descMap`.** -/
theorem rec_determines (g : ApproximableMap (Cn A) B.carrier.sys)
    (hg : g = (B.str.1.comp (sumMapSig (A := A) (h_0 := Cn_nonempty) (h_1 := B.carrier
    .nonempty)
        g)).comp (ofIso ccEquiv)) :
    g = descMap B := by
  have hbot : forall  sigma, g.toElementMap (strBotN sigma) = descVal B B.carrier.sys.
    bot sigma := by
    intro sigma
    induction sigma with
    | nil =>
      conv_lhs => rw [hg]
      rw [toElementMap_comp, toElementMap_comp, toElementMap_ofIso, ccEquiv_apply]
      exact genKeyBot B g
    | cons a sigma ih =>
      conv_lhs => rw [hg]
      rw [toElementMap_comp, toElementMap_comp, toElementMap_ofIso, ccEquiv_apply,
         <-  consMapN_strBot]
      have h := genKey B g a (strBotN sigma)
      rw [ih] at h
      exact h
  have helem : forall  sigma, g.toElementMap (strElemN sigma) = descVal B (descE B)
    sigma := by
    intro sigma
    induction sigma with
```

```
    | nil =>
      conv_lhs => rw [hg]
      rw [toElementMap_comp, toElementMap_comp, toElementMap_ofIso, ccEquiv_apply]
      exact genKey0 B g
    | cons a sigma ih =>
      conv_lhs => rw [hg]
      rw [toElementMap_comp, toElementMap_comp, toElementMap_ofIso, ccEquiv_apply,
         <-  consMapN_strElem]
      have h := genKey B g a (strElemN sigma)
      rw [ih] at h
      exact h
  apply map_ext_Cn
  * intro sigma; rw [hbot, descMap_strBot]
  * intro sigma; rw [helem, descMap_strElem]

/-- **Uniqueness.** Any `Tsig`-algebra homomorphism out of `(C?, i)` equals `descAlgHom
    `. -/
theorem descAlgHom_uniq (h' : AlgHom (Cnalg A) B) : h' = descAlgHom B := by
  obtain <hom, comm> := h'
  have hg : hom.1 = descMap B := by
    refine rec_determines B hom.1 ?_
    have hc : hom.1.comp (ofIso ccEquiv.symm)
        = B.str.1.comp (sumMapSig (A := A) (h_0 := Cn_nonempty) (h_1 := B.carrier.
    nonempty) hom.1) := by
      exact congrArg Subtype.val comm
    have h2 := congrArg (fun m => m.comp (ofIso ccEquiv)) hc
    simp only at h2
    rw [comp_assoc] at h2
    erw [ccEquiv_symm_comp, comp_idMap] at h2
    exact h2
  have hhom : hom = descStrict B := Subtype.ext hg
  subst hhom
  rfl

end Initial

/-- **Exercise 6.17 part 2 -- `(C?, i)` is an initial `Tsig`-algebra for `Tsig(X) = ? +
     Sigma_a X`.**
The descent map `phi : C?  ->  E` is the closed-form head-recursion `phi(?) = e`, `phi(
    a*x) = f_a(phi x)`
(`f_a = k  comp  inj_a`), built choice-free via `liftCn`; it is the unique `Tsig`-
    algebra homomorphism, so
`C?` is the initial algebra of `X  |->  ? + Sigma_a X`. -/
def CnisInitial (A : Type) [DecidableEq A] [Inhabited A] : IsInitial (Cnalg A) where
  desc := descAlgHom
  uniq := fun B h => descAlgHom_uniq B h

/-- **`C_n` is the initial algebra** of `X  |->  ? + Sigma_{Fin (n+1)} X`. -/
def CfinIsInitial (n : Nat) : IsInitial (Cnalg (Fin (n + 1))) := CnisInitial (Fin (n +
     1))

/-- **Exercise 6.17 part 2 -- the domain equation `C?  ~=  ? + Sigma_a C?`.** -/
theorem Cn_domain_equation (A : Type) [DecidableEq A] [Inhabited A] :
    Cn A  ~= ? CCn A := <ccEquiv>

end
```

```
section
set_option linter.unusedVariables false

theorem example_617_gen : forall  (A : Type) [inst : DecidableEq A] [Inhabited A],
  @DomainTheory.NeighbourhoodSystems.Isomorphic (DomainTheory.NeighbourhoodSystems.
    Exercise617Gen.Strn A)
    (DomainTheory.NeighbourhoodSystems.Exercise617Gen.SigTok A
      (DomainTheory.NeighbourhoodSystems.Exercise617Gen.Strn A))
    (@DomainTheory.NeighbourhoodSystems.Exercise617Gen.Cn A inst)
    (@DomainTheory.NeighbourhoodSystems.Exercise617Gen.CCn A inst) :=
  DomainTheory.NeighbourhoodSystems.Exercise617Gen.Cn_domain_equation
end

section
variable {A : Type} [DecidableEq A]

/-- **`C_n  ~=  ? + (n+1)*C_n`** over the alphabet `Fin (n+1)`. -/
theorem Cfin_domain_equation (n : Nat) : Cn (Fin (n + 1))  ~= ? CCn (Fin (n + 1)) :=
  Cn_domain_equation (Fin (n + 1))
end

section
variable {A : Type}

section Iso

theorem embA_ne {a a' : A} (h : a  !=  a') {X Y : Set (Strn A)} (hX : X.Nonempty) :
    embA a X  !=  embA a' Y := by
  intro heq
  obtain <w', hw'> := hX
  have hmem : (a :: w')  in  embA a' Y := heq  |>  (<w', rfl, hw'> : (a :: w')  in
    embA a X)
  obtain <w'', he, -> := hmem
  rw [List.cons.injEq] at he; exact h he.1

end Iso

section Iso

theorem embA_nonempty {a : A} {X : Set (Strn A)} (hX : X.Nonempty) : (embA a X).
    Nonempty := by
  obtain <w', hw'> := hX; exact <a :: w', w', rfl, hw'>

end Iso

end

end DomainTheory.NeighbourhoodSystems.Exercise617Gen
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.Exercise510

section
namespace Exercise618

variable {a : Type*} {V : NeighborhoodSystem a}
```

```
/-- The "cons" of a head `a : |?|` and a tail `b : |?^ inf |`, as a sequence `<a, b_0,
    b_1, ...>`. -/
def consSeq (a : V.Element) (b : (iterSys V).Element) : Nat  ->  V.Element :=
  fun i => Nat.casesOn i a (fun k => component b k)

@[simp] theorem consSeq_zero (a : V.Element) (b : (iterSys V).Element) : consSeq a b 0
     = a := rfl

@[simp] theorem consSeq_succ (a : V.Element) (b : (iterSys V).Element) (k : Nat) :
    consSeq a b (k + 1) = component b k := rfl

/-- `False` of `?^ inf ` is the all-`False` sequence. -/
theorem iterBot_eq : (iterSys V).bot = ofSeq (fun _ : Nat => V.bot) := by
  apply Element.ext
  intro W
  rw [mem_bot, mem_ofSeq]
  constructor
  * rintro rfl
    refine <(iterSys V).master_mem, fun i => ?_>
    rw [fiber_iterSys_master, mem_bot]
  * rintro <_, hfib>
    apply eq_of_fiber_eq
    intro i
    have hi := hfib i
    rw [fiber_iterSys_master]
    rwa [mem_bot] at hi

/-- The `n`-th coordinate of `False` is `False`. -/
@[simp] theorem component_bot (n : Nat) : component (iterSys V).bot n = V.bot := by
  rw [iterBot_eq, component_ofSeq]

/-- A successor-increasing chain is increasing. -/
theorem chain_le_of_succ {b : Type*} {W : NeighborhoodSystem b} {a : Nat  ->  W.
    Element}
    (h : forall  n, a n  <=  a (n + 1)) {i j : Nat} (hij : i  <=  j) : a i  <=  a j :=
     by
  induction hij with
  | refl => exact le_rfl
  | step _ ih => exact ih.trans (h _)

section Target

variable {? : Type*} {E : NeighborhoodSystem ?}

/-- The bottom map is below everything. -/
theorem constBot_le (f : ApproximableMap (iterSys V) E) :
    constMap (iterSys V) E.bot  <=  f := by
  intro X Y hrel
  obtain <hX, hY> := hrel
  rw [mem_bot] at hY; subst hY
  exact f.rel_master hX

end Target

/-- `i : ?  x  ?^ inf   ->  ?^ inf `, the "cons" iso (`iterProdIso^-1`); the `T`-
```

```
    algebra structure map. -/
def imap (V : NeighborhoodSystem a) : ApproximableMap (prod V (iterSys V)) (iterSys V)
     :=
  ofIso (iterProdIso V).symm

theorem isStrict_imap : IsStrict (imap V) := isStrict_ofIso _


/-- **The forward iso reads off head and tail.** `iterProdIso z = <z_0, <z_1,z_2
    ,...>>`: the first
component is the `0`-coordinate `component z 0`, the second is the shifted sequence. -/
theorem iterProdIso_apply (z : (iterSys V).Element) :
    iterProdIso V z = pair (component z 0) (ofSeq (fun n => component z (n + 1))) :=
    rfl

/-- **The inverse iso is "cons".** `iterProdIso^-1 <a, b> = <a, b_0, b_1, ...>`. -/
theorem iterProdIso_symm_pair (a : V.Element) (b : (iterSys V).Element) :
    (iterProdIso V).symm (pair a b) = ofSeq (consSeq a b) := by
  have hkey : iterProdIso V (ofSeq (consSeq a b)) = pair a b := by
    rw [iterProdIso_apply]
    congr 1
    * rw [component_ofSeq, consSeq_zero]
    * have : (fun n => component (ofSeq (consSeq a b)) (n + 1)) = fun n => component b
     n := by
        funext n; rw [component_ofSeq, consSeq_succ]
      rw [this, ofSeq_component]
  rw [ <-  hkey, OrderIso.symm_apply_apply]

end Exercise618
end

section
namespace Exercise618

/-- The infinite iterate of an `{}`-free system is `{}`-free (each fibre is a non-empty
     neighbourhood). -/
theorem iterSys_nonempty {a : Type*} {V : NeighborhoodSystem a}
    (h : forall  X, V.mem X  ->  X.Nonempty) :
    forall  W, (iterSys V).mem W  ->  W.Nonempty := by
  rintro W <hfib, _>
  obtain <a, ha> := h (fiber W 0) (hfib 0)
  exact <(0, a), ha>

end Exercise618
end

section
namespace Exercise618

variable {a : Type*} {V : NeighborhoodSystem a}

/-- `?^ inf ` (Exercise 3.16's `iterSys`) as an `{}`-free object. -/
def iterObj (Dom : StrictDomainObj.{w}) : StrictDomainObj.{w} where
  carrier := Nat  x  Dom.carrier
  sys := iterSys Dom.sys
  nonempty := iterSys_nonempty Dom.nonempty
```

```
/-- `j : ?^ inf   ->  ?  x  ?^ inf `, the splitting iso (`iterProdIso`). -/
def jmap (V : NeighborhoodSystem a) : ApproximableMap (iterSys V) (prod V (iterSys V))
     :=
  ofIso (iterProdIso V)

section Target

variable {? : Type*} {E : NeighborhoodSystem ?}

/-- The homomorphism operator `Op(f) = k  comp  T(f)  comp  j`. -/
def descOp (k : ApproximableMap (prod V E) E) (f : ApproximableMap (iterSys V) E) :
    ApproximableMap (iterSys V) E :=
  k.comp ((prodMap (idMap V) f).comp (jmap V))

/-- The defining action of the operator: `Op(f)(z) = k(z_0, f(<z_1,z_2,...>))`. -/
theorem descOp_apply (k : ApproximableMap (prod V E) E) (f : ApproximableMap (iterSys
    V) E)
    (z : (iterSys V).Element) :
    (descOp k f).toElementMap z
      = k.toElementMap (pair (component z 0)
          (f.toElementMap (ofSeq (fun n => component z (n + 1))))) := by
  simp only [descOp, jmap]
  rw [toElementMap_comp, toElementMap_comp, toElementMap_ofIso, iterProdIso_apply,
    toElementMap_prodMap_pair, toElementMap_idMap]

/-- The descent chain `h_0 = False`, `h_n_+_1 = Op(h_n)`. -/
def descSeq (k : ApproximableMap (prod V E) E) : Nat  ->  ApproximableMap (iterSys V)
    E
  | 0 => constMap (iterSys V) E.bot
  | (n + 1) => descOp k (descSeq k n)

end Target

section Uniq

variable {? : Type*} {E : NeighborhoodSystem ?}

/-- **`g`-independence step.** If `g` is a homomorphism (`g  comp  i = k  comp  T(g)`)
    then
`g  comp  ?_n_+_1 = k  comp  T(g  comp  ?_n)  comp  j = Op_k(g  comp  ?_n)`: the
    composite depends only on `g  comp  ?_n`. -/
theorem gcomp_rho_succ (k : ApproximableMap (prod V E) E) (g : ApproximableMap (
    iterSys V) E)
    (hc : g.comp (imap V) = k.comp (prodMap (idMap V) g)) (n : Nat) :
    g.comp (descSeq (imap V) (n + 1)) = descOp k (g.comp (descSeq (imap V) n)) := by
  show g.comp ((imap V).comp ((prodMap (idMap V) (descSeq (imap V) n)).comp (jmap V)))
     = k.comp ((prodMap (idMap V) (g.comp (descSeq (imap V) n))).comp (jmap V))
  rw [ <-  comp_assoc, hc, comp_assoc]
  congr 1
  rw [ <-  comp_assoc,  <-  prodMap_comp, idMap_comp]

/-- The descent chain for any strict `g` starts at the constant `False`: `g  comp  ?_0
    = False`. -/
theorem gcomp_rho_zero (g : ApproximableMap (iterSys V) E) (hg : IsStrict g) :
    g.comp (descSeq (imap V) 0) = constMap (iterSys V) E.bot := by
```

```
  apply ext_of_toElementMap
  intro x
  rw [toElementMap_comp, toElementMap_constMap,
    show descSeq (imap V) 0 = constMap (iterSys V) (iterSys V).bot from rfl,
    toElementMap_constMap, isStrict_iff_apply_bot.mp hg]

end Uniq

/-- `j  comp  i = I` on `?  x  ?^ inf `. -/
theorem jmap_comp_imap : (jmap V).comp (imap V) = idMap (prod V (iterSys V)) := by
  apply ext_of_toElementMap
  intro w
  simp only [jmap, imap]
  rw [toElementMap_comp, toElementMap_ofIso, toElementMap_ofIso, toElementMap_idMap,
    OrderIso.apply_symm_apply]

section Target

variable {? : Type*} {E : NeighborhoodSystem ?}

/-- The continuity helper `k(a, *) : E  ->  E` as an approximable map. -/
def kHead (k : ApproximableMap (prod V E) E) (a : V.Element) :
    ApproximableMap E E :=
  k.comp (paired (constMap E a) (idMap E))

theorem kHead_apply (k : ApproximableMap (prod V E) E) (a : V.Element) (u : E.Element)
     :
    (kHead k a).toElementMap u = k.toElementMap (pair a u) := by
  rw [kHead, toElementMap_comp, toElementMap_paired, toElementMap_constMap,
    toElementMap_idMap]

end Target

/-- `False` of a product is the pair of `False`s. -/
theorem pair_bot {a b : Type*} {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem
    b} :
    pair V_0.bot V_1.bot = (prod V_0 V_1).bot := by
  apply Element.ext
  intro W
  rw [mem_bot, prod_master]
  constructor
  * rintro <X, Y, hX, hY, rfl>
    rw [mem_bot] at hX hY; subst hX; subst hY; rfl
  * rintro rfl
    exact <V_0.master, V_1.master, V_0.bot.master_mem, V_1.bot.master_mem, rfl>

section Target

variable {? : Type*} {E : NeighborhoodSystem ?}

theorem descSeq_strict (k : ApproximableMap (prod V E) E) (hk : IsStrict k) :
    forall  n, IsStrict (descSeq k n)
  | 0 => isStrict_constBot
  | (n + 1) => by
      have ih := descSeq_strict k hk n
      rw [isStrict_iff_apply_bot] at ih  |-
```

```
      rw [show descSeq k (n + 1) = descOp k (descSeq k n) from rfl, descOp_apply]
      have htl : ofSeq (fun m => component (iterSys V).bot (m + 1)) = (iterSys V).bot
    := by
        have hconst : (fun m => component (iterSys V).bot (m + 1)) = (fun _ => V.bot)
    := by
          funext m; rw [component_bot]
        rw [hconst,  <-  iterBot_eq]
      rw [component_bot, htl, ih, pair_bot]
      exact isStrict_iff_apply_bot.mp hk

end Target

/-- `T(f_0  x  f_1)` is strict when both factors are: `(f_0  x  f_1)(False,False) = (
    f_0 False, f_1 False) = (False,False)`. -/
theorem isStrict_prodMap {a b a' b' : Type*} {V_0 : NeighborhoodSystem a} {V_1 :
    NeighborhoodSystem b}
    {V_0' : NeighborhoodSystem a'} {V_1' : NeighborhoodSystem b'}
    {f_0 : ApproximableMap V_0 V_0'} {f_1 : ApproximableMap V_1 V_1'}
    (hf_0 : IsStrict f_0) (hf_1 : IsStrict f_1) : IsStrict (prodMap f_0 f_1) := by
  rw [isStrict_iff_apply_bot, show (prod V_0 V_1).bot = pair V_0.bot V_1.bot from
    pair_bot.symm,
    toElementMap_prodMap_pair, isStrict_iff_apply_bot.mp hf_0, isStrict_iff_apply_bot.
    mp hf_1]
  exact pair_bot

end Exercise618
end

end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.Exercise618
section
set_option linter.unusedVariables false

theorem example_618.{u_2, u_3, u_4, u_5} : forall  {a : Type u_2} {b : Type u_3} {a' :
     Type u_4} {b' : Type u_5}
  {V_0 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
  {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
  {V_0' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a'}
  {V_1' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b'}
  {f_0 : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a' V_0 V_0'}
  {f_1 : @DomainTheory.NeighbourhoodSystems.ApproximableMap b b' V_1 V_1'},
  @DomainTheory.NeighbourhoodSystems.Exercise510.IsStrict a a' V_0 V_0' f_0  ->
    @DomainTheory.NeighbourhoodSystems.Exercise510.IsStrict b b' V_1 V_1' f_1  ->
      @DomainTheory.NeighbourhoodSystems.Exercise510.IsStrict (Sum a b) (Sum a' b')
        (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) (@DomainTheory.
    NeighbourhoodSystems.prod a' b' V_0' V_1')
        (@DomainTheory.NeighbourhoodSystems.prodMap a b a' b' V_0 V_1 V_0' V_1' f_0
    f_1) :=
  @DomainTheory.NeighbourhoodSystems.Exercise618.isStrict_prodMap
end

end DomainTheory.NeighbourhoodSystems.Exercise618
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.Exercise510
```

```
namespace Exercise618

variable {a : Type*} {V : NeighborhoodSystem a}

/-- `T(*) = (id_?  x  *)` is monotone. -/
theorem prodMap_idMap_mono {? : Type*} {E : NeighborhoodSystem ?}
    {f f' : ApproximableMap (iterSys V) E} (h : f  <=  f') :
    prodMap (idMap V) f  <=  prodMap (idMap V) f' := by
  intro W P hrel
  simp only [prodMap, paired_rel] at hrel  |-
  exact <hrel.1, hrel.2.1, comp_mono_gen h le_rfl _ _ hrel.2.2>

section Target

variable {? : Type*} {E : NeighborhoodSystem ?}

theorem descOp_mono (k : ApproximableMap (prod V E) E) {f f' : ApproximableMap (
    iterSys V) E}
    (h : f  <=  f') : descOp k f  <=  descOp k f' :=
  comp_mono_gen le_rfl (comp_mono_gen (prodMap_idMap_mono h) le_rfl)

theorem descSeq_mono_succ (k : ApproximableMap (prod V E) E) (n : Nat) :
    descSeq k n  <=  descSeq k (n + 1) := by
  induction n with
  | zero => exact constBot_le _
  | succ m ih => exact descOp_mono k ih

theorem descSeq_mono (k : ApproximableMap (prod V E) E) {n m : Nat} (h : n  <=  m) :
    descSeq k n  <=  descSeq k m := by
  induction h with
  | refl => exact le_rfl
  | step _ ih => exact ih.trans (descSeq_mono_succ k _)

theorem descSeq_dir (k : ApproximableMap (prod V E) E) :
    forall  i j, exists  l, (forall  X Y, (descSeq k i).rel X Y  ->  (descSeq k l).rel
     X Y)  /\
      (forall  X Y, (descSeq k j).rel X Y  ->  (descSeq k l).rel X Y) :=
  fun i j => <max i j, fun _ _ => descSeq_mono k (le_max_left i j) _ _,
    fun _ _ => descSeq_mono k (le_max_right i j) _ _>

/-- **The descent map `h = ?_n h_n : ?^ inf   ->  E`.** -/
def descMap (k : ApproximableMap (prod V E) E) : ApproximableMap (iterSys V) E :=
  iSupMap (descSeq k) (descSeq_dir k)

theorem descMap_toElementMap (k : ApproximableMap (prod V E) E) (z : (iterSys V).
    Element)
    {Y : Set ?} :
    ((descMap k).toElementMap z).mem Y  <->  exists  n, ((descSeq k n).toElementMap z)
    .mem Y :=
  mem_toElementMap_iSupMap (descSeq k) (descSeq_dir k) z

theorem descMap_strict (k : ApproximableMap (prod V E) E) (hk : IsStrict k) :
    IsStrict (descMap k) := by
  rw [isStrict_iff_apply_bot]
  apply Element.ext
  intro Y
```

```
  rw [mem_bot, descMap_toElementMap]
  constructor
  * rintro <n, hn>
    have hs := descSeq_strict k hk n
    rw [isStrict_iff_apply_bot] at hs
    rw [hs, mem_bot] at hn
    exact hn
  * rintro rfl
    have hs := descSeq_strict k hk 0
    rw [isStrict_iff_apply_bot] at hs
    exact <0, by rw [hs]; exact E.bot.master_mem>

end Target

end Exercise618
end

section
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.Exercise510

namespace Exercise618

/-- The product of two `{}`-free systems is `{}`-free. -/
theorem prod_nonempty {a b : Type*} {V_0 : NeighborhoodSystem a} {V_1 :
    NeighborhoodSystem b}
    (h_0 : forall  X, V_0.mem X  ->  X.Nonempty) (_h_1 : forall  Y, V_1.mem Y  ->  Y.
    Nonempty) :
    forall  W, (prod V_0 V_1).mem W  ->  W.Nonempty := by
  rintro W <X, Y, hX, _, rfl>
  obtain <a, ha> := h_0 X hX
  exact <Sum.inl a, mem_prodNbhd_inl.mpr ha>

end Exercise618
end

section
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.Exercise510

namespace Exercise618

variable {a : Type*} {V : NeighborhoodSystem a}

/-- The fixed domain `?` times an object `X`, again an `{}`-free domain. -/
def prodObj (Dom X : StrictDomainObj.{w}) : StrictDomainObj.{w} where
  carrier := Dom.carrier ? X.carrier
  sys := prod Dom.sys X.sys
  nonempty := prod_nonempty Dom.nonempty X.nonempty

/-- The morphism action `T(f) = id_?  x  f`, strict by `isStrict_prodMap`. -/
def prodMapHom (Dom : StrictDomainObj.{w}) {X Y : StrictDomainObj.{w}} (f : (X  -->  Y
    )) :
    ((prodObj Dom X)  -->  (prodObj Dom Y)) :=
  <prodMap (idMap Dom.sys) f.1, isStrict_prodMap isStrict_idMap f.2>

/-- **The product endofunctor `T(X) = ?  x  X`** on `{}`-free domains and strict maps,
    for a fixed
```

```
domain `?`. On objects `T(X) = ?  x  X`; on maps `T(f) = id_?  x  f`. -/
def prodFunctor (Dom : StrictDomainObj.{w}) : Endofunctor StrictDomainObj.{w} where
  obj := prodObj Dom
  map := prodMapHom Dom
  map_id X := Subtype.ext (by
    show prodMap (idMap Dom.sys) (idMap X.sys) = idMap (prod Dom.sys X.sys)
    exact prodMap_id)
  map_comp {X Y Z} g f := Subtype.ext (by
    show prodMap (idMap Dom.sys) (g.1.comp f.1)
       = (prodMap (idMap Dom.sys) g.1).comp (prodMap (idMap Dom.sys) f.1)
    have h := prodMap_comp (idMap Dom.sys) (idMap Dom.sys) g.1 f.1
    rw [idMap_comp] at h
    exact h)

/-- **`?^ inf ` as a `T`-algebra**, `(?^ inf , i)` with `i : ?  x  ?^ inf   ->  ?^ inf
    ` the "cons" iso (`imap`,
Exercise 3.16's `iterProdIso^-1`), strict by `isStrict_imap`. -/
def iterAlg (Dom : StrictDomainObj.{w}) : TAlgebra (prodFunctor Dom) where
  carrier := iterObj Dom
  str := <imap Dom.sys, isStrict_imap>

/-- **The truncation formula** `?_n(<x_0,x_1,...>) = <x_0,...,x_{n-1},False,False
    ,...>`. -/
theorem rho_apply (n : Nat) (z : (iterSys V).Element) :
    (descSeq (imap V) n).toElementMap z
      = ofSeq (fun i => if i < n then component z i else V.bot) := by
  induction n generalizing z with
  | zero =>
    show (constMap (iterSys V) (iterSys V).bot).toElementMap z = _
    rw [toElementMap_constMap, iterBot_eq]
    congr 1
  | succ n ih =>
    rw [show descSeq (imap V) (n + 1) = descOp (imap V) (descSeq (imap V) n) from rfl,
      descOp_apply, ih]
    simp only [imap]
    rw [toElementMap_ofIso, iterProdIso_symm_pair]
    congr 1
    funext j
    cases j with
    | zero =>
      show component z 0 = if (0 : Nat) < n + 1 then component z 0 else V.bot
      rw [ite_eq_left (Nat.zero_lt_succ n)]
    | succ k =>
      rw [consSeq_succ, component_ofSeq, component_ofSeq]
      show (if k < n then component z (k + 1) else V.bot)
         = if k + 1 < n + 1 then component z (k + 1) else V.bot
      by_cases h : k < n
      * rw [ite_eq_left h, ite_eq_left (Nat.succ_lt_succ h)]
      * rw [ite_eq_right h, ite_eq_right (fun hc => h (Nat.lt_of_succ_lt_succ hc))]

/-- **`?_n ?_n = I`.** Every `z` is the directed union of its truncations: the cofinite
    -`Delta` structure
of `?^ inf ` means each neighbourhood of `z` is already realised by a finite truncation
    . -/
theorem iSupRho_eq_id : descMap (imap V) = idMap (iterSys V) := by
  apply ext_of_toElementMap
```

```
  intro z
  rw [toElementMap_idMap]
  apply Element.ext
  intro Y
  rw [descMap_toElementMap]
  constructor
  * rintro <n, hn>
    have hle : (descSeq (imap V) n).toElementMap z  <=  z := by
      rw [rho_apply]
      have hz : z = ofSeq (fun i => component z i) := (ofSeq_component z).symm
      conv_rhs => rw [hz]
      apply ofSeq_mono
      intro i
      show (if i < n then component z i else V.bot)  <=  component z i
      split
      * exact le_rfl
      * exact V.bot_le _
    exact hle Y hn
  * intro hzY
    have hY : (iterSys V).mem Y := z.sub hzY
    obtain <N, hN> := hY.2
    have hcomp : forall  i, (component z i).mem (fiber Y i) := by
      have h := hzY
      rw [ <-  ofSeq_component z] at h
      rw [mem_ofSeq] at h
      exact h.2
    refine <N, ?_>
    rw [rho_apply, mem_ofSeq]
    refine <hY, fun i => ?_>
    show (if i < N then component z i else V.bot).mem (fiber Y i)
    by_cases h : i < N
    * rw [ite_eq_left h]; exact hcomp i
    * rw [ite_eq_right h, hN i (not_lt.mp h)]; exact V.bot.master_mem

section Uniq

variable {? : Type*} {E : NeighborhoodSystem ?}

/-- **Uniqueness.** Any two strict homomorphisms `g, g' : ?^ inf   ->  E` into a `T`-
    algebra `(E,k)` are
equal. By `g`-independence they agree on every truncation (`g  comp  ?_n = g'  comp  ?
    _n`), and `?_n ?_n = I`
forces `g = g'`. -/
theorem comm_unique (k : ApproximableMap (prod V E) E)
    {g g' : ApproximableMap (iterSys V) E} (hg : IsStrict g) (hg' : IsStrict g')
    (hc : g.comp (imap V) = k.comp (prodMap (idMap V) g))
    (hc' : g'.comp (imap V) = k.comp (prodMap (idMap V) g')) : g = g' := by
  have hindep : forall  n, g.comp (descSeq (imap V) n) = g'.comp (descSeq (imap V) n)
    := by
    intro n
    induction n with
    | zero => rw [gcomp_rho_zero g hg, gcomp_rho_zero g' hg']
    | succ m ih => rw [gcomp_rho_succ k g hc m, gcomp_rho_succ k g' hc' m, ih]


  have key : g.comp (descMap (imap V)) = g'.comp (descMap (imap V)) := by
    apply ApproximableMap.ext
```

```
    intro X Z
    simp only [descMap, comp_rel]
    constructor
    * rintro <Y, <i, hXY>, hYZ>
      have hg : (g.comp (descSeq (imap V) i)).rel X Z := <Y, hXY, hYZ>
      rw [hindep i] at hg
      obtain <Y', hXY', hYZ'> := hg
      exact <Y', <i, hXY'>, hYZ'>
    * rintro <Y, <i, hXY>, hYZ>
      have hg' : (g'.comp (descSeq (imap V) i)).rel X Z := <Y, hXY, hYZ>
      rw [ <-  hindep i] at hg'
      obtain <Y', hXY', hYZ'> := hg'
      exact <Y', <i, hXY'>, hYZ'>
  rw [iSupRho_eq_id] at key
  rwa [comp_idMap, comp_idMap] at key

end Uniq

/-- Directedness of a successor-increasing chain. -/
theorem succChainDir {b : Type*} {W : NeighborhoodSystem b} (a : Nat  ->  W.Element)
    (h : forall  n, a n  <=  a (n + 1)) : forall  i j, exists  l, a i  <=  a l  /\  a
    j  <=  a l :=
  fun i j => <max i j, chain_le_of_succ h (le_max_left i j), chain_le_of_succ h (
    le_max_right i j)>

/-- Approximable maps are monotone in the map argument: `f  <=  g ? f(x)  <=  g(x)`. -/
theorem toElementMap_le_of_le {b_0 b_1 : Type*} {W_0 : NeighborhoodSystem b_0}
    {W_1 : NeighborhoodSystem b_1} {f g : ApproximableMap W_0 W_1} (h : f  <=  g) (x :
     W_0.Element) :
    f.toElementMap x  <=  g.toElementMap x := by
  rintro Y <X, hX, hrel>
  exact <X, hX, h _ _ hrel>

section Target

variable {? : Type*} {E : NeighborhoodSystem ?}

/-- The descent chain is increasing element-wise. -/
theorem descSeqEltMono (k : ApproximableMap (prod V E) E) (x : (iterSys V).Element) :
    forall  n, (descSeq k n).toElementMap x  <=  (descSeq k (n + 1)).toElementMap x :=
  fun n => toElementMap_le_of_le (descSeq_mono_succ k n) x

/-- **The descent map as a directed union.** `h(x) = ?_n h_n(x)`. -/
theorem descMap_eq (k : ApproximableMap (prod V E) E) (x : (iterSys V).Element) :
    (descMap k).toElementMap x
      = iSupDirected (fun n => (descSeq k n).toElementMap x)
          (succChainDir _ (descSeqEltMono k x)) := by
  apply Element.ext
  intro Y
  rw [descMap_toElementMap, mem_iSupDirected]

/-- **The descent map is a fixed point** of `Op = k  comp  T(*)  comp  j`. -/
theorem descMap_fix (k : ApproximableMap (prod V E) E) : descMap k = descOp k (descMap
    k) := by
  apply ext_of_toElementMap
  intro z
```

```
  rw [descOp_apply, descMap_eq k (ofSeq (fun n => component z (n + 1))),
     <-  kHead_apply k (component z 0), toElementMap_iSupDirected, descMap_eq k z]
  -- both sides are directed unions; compare the (reindexed) families term-wise
  apply Element.ext
  intro Y
  rw [mem_iSupDirected, mem_iSupDirected]
  have hstep : forall  m, (kHead k (component z 0)).toElementMap
        ((descSeq k m).toElementMap (ofSeq (fun n => component z (n + 1))))
      = (descSeq k (m + 1)).toElementMap z := by
    intro m
    rw [kHead_apply, show descSeq k (m + 1) = descOp k (descSeq k m) from rfl,
    descOp_apply]
  constructor
  * rintro <n, hn>
    cases n with
    | zero => exact <0, by rw [hstep 0]; exact descSeqEltMono k z 0 _ hn>
    | succ m => exact <m, by rw [hstep m]; exact hn>
  * rintro <m, hm>
    rw [hstep m] at hm
    exact <m + 1, hm>

/-- **Existence of the algebra homomorphism.** The descent map makes the square
    commute:
`descMap  comp  i = k  comp  T(descMap)` (using `j  comp  i = I` and the fixed-point
    equation). -/
theorem descMap_comm (k : ApproximableMap (prod V E) E) :
    (descMap k).comp (imap V) = k.comp (prodMap (idMap V) (descMap k)) := by
  conv_lhs => rw [descMap_fix k]
  show (k.comp ((prodMap (idMap V) (descMap k)).comp (jmap V))).comp (imap V)
     = k.comp (prodMap (idMap V) (descMap k))
  rw [comp_assoc, comp_assoc, jmap_comp_imap, comp_idMap]

end Target

/-- **The descent homomorphism `(?^ inf , i)  ->  (E, k)`**: the strict map `descMap k`
     (existence half),
with the homomorphism square supplied by `descMap_comm`. -/
def descAlgHom (Dom : StrictDomainObj.{w}) (B : TAlgebra (prodFunctor Dom)) :
    AlgHom (iterAlg Dom) B where
  hom := <descMap B.str.1, descMap_strict B.str.1 B.str.2>
  comm := by
    apply Subtype.ext
    show (descMap B.str.1).comp (imap Dom.sys)
       = B.str.1.comp (prodMap (idMap Dom.sys) (descMap B.str.1))
    exact descMap_comm B.str.1

end Exercise618
end
end DomainTheory.NeighbourhoodSystems
end
```

3.7.20 Exercise 6.18 — infinite product as initial

With reference back to Exercise 3.16 discuss the construction of $\mathcal{D}^\infty$ as an initial algebra and as a solution to the domain equation

$$\mathcal{D}^\infty \cong \mathcal{D} \times \mathcal{D}^\infty.$$

(I do not know whether all solutions must be of the form $\mathcal{D}^\infty \times E$.)

Exercise_6_18.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.Exercise_6_17

public import DomainTheory.NeighbourhoodSystems.DomainEquations.Theorem_6_14

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.Exercise510
namespace Exercise618
variable {a : Type*} {V : NeighborhoodSystem a}
/- concordance: source L4079 EXERCISE 6.18 infinite product as initial -/
/-- **Exercise 6.18 (initial-algebra half) -- `?^ inf ` is the initial `T`-algebra for
    `T(X) = ?  x  X`.**
For every `T`-algebra `(E, k)` the descent map `h(<x_0,x_1,...>) = k(x_0, k(x_1, ...))`
     is the *unique*
strict homomorphism `?^ inf   ->  E`. Together with Exercise 3.16's `?^ inf   ~=  ?  x
     ?^ inf ` (the domain-equation
half), this exhibits `?^ inf ` both as the canonical solution of the domain equation
    and as the initial
algebra (determined up to iso by Proposition 6.6). -/
def iterIsInitial (Dom : StrictDomainObj.{w}) : IsInitial (iterAlg Dom) where
  desc := descAlgHom Dom
  uniq B h := by
    obtain <hom, comm> := h
    have hcomm : hom.1.comp (imap Dom.sys)
        = B.str.1.comp (prodMap (idMap Dom.sys) hom.1) := congrArg Subtype.val comm
    have hg : hom.1 = descMap B.str.1 :=
      comm_unique B.str.1 hom.2 (descMap_strict B.str.1 B.str.2) hcomm (descMap_comm B
    .str.1)
    have hhom : hom = <descMap B.str.1, descMap_strict B.str.1 B.str.2> := Subtype.ext
     hg
    subst hhom
    rfl

end Exercise618
end

section
open NeighborhoodSystem

namespace Example61

variable {a : Type*}

section Equation
```

```
variable (D : NeighborhoodSystem a) (hD : forall  X, D.mem X  ->  X.Nonempty)

include hD

/-- **Example 6.1 -- the forward half of the domain equation.** Sends an element `z` of
    `D^S` to an
element of `D + (D^S  x  D^S)`, recording for each branch which `0X`/`1V`
   neighbourhoods `z` reaches:
`0X  in  toS z  <->  embZero X  in  z` and `1(P U Q)  in  toS z  <->  (1P U 2Q)  in  z
   `. -/
def toS (z : (Dsharp D hD).Element) : (sumSys D hD).Element where
  mem W := W = sumMaster D (prod (Dsharp D hD) (Dsharp D hD))
     \/  (exists  X, D.mem X  /\  W = inj_0 X  /\  z.mem (embZero X))
     \/  (exists  P Q, MemS D P  /\  MemS D Q  /\  W = inj_1 (prodNbhd P Q)  /\  z.mem
     (embPair P Q))
  sub := by
    rintro W (rfl | <X, hX, rfl, -> | <P, Q, hP, hQ, rfl, ->)
    * exact Or.inl rfl
    * exact Or.inr (Or.inl <X, hX, rfl>)
    * exact Or.inr (Or.inr <prodNbhd P Q, prod_mem_prodNbhd hP hQ, rfl>)
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | <X, hX, rfl, hzX> | <P, Q, hP, hQ, rfl, hzPQ>)
      (rfl | <X', hX', rfl, hzX'> | <P', Q', hP', hQ', rfl, hzPQ'>)
    * exact Or.inl (by rw [Set.inter_self])
    * exact Or.inr (Or.inl <X', hX', by rw [sumMaster_inter_inj_0 hX'], hzX'>)
    * exact Or.inr (Or.inr <P', Q', hP', hQ',
        by rw [sumMaster_inter_inj_1 (prod_mem_prodNbhd hP' hQ')], hzPQ'>)
    * exact Or.inr (Or.inl <X, hX, by rw [Set.inter_comm, sumMaster_inter_inj_0 hX],
    hzX>)
    * refine Or.inr (Or.inl <X  I  X', ?_, by rw [inj_0_inter], ?_>)
      * have hz := z.inter_mem hzX hzX'; rw [embZero_inter] at hz
        exact memS_embZero_inv hD (z.sub hz)
      * have hz := z.inter_mem hzX hzX'; rwa [embZero_inter] at hz
    * exfalso
      have hz := z.inter_mem hzX hzPQ'; rw [embZero_inter_embPair] at hz
      obtain <t, ht> := memS_nonempty hD (z.sub hz); exact Set.notMem_empty t ht
    * exact Or.inr (Or.inr <P, Q, hP, hQ,
        by rw [Set.inter_comm, sumMaster_inter_inj_1 (prod_mem_prodNbhd hP hQ)], hzPQ
    >)
    * exfalso
      have hz := z.inter_mem hzPQ hzX'; rw [Set.inter_comm, embZero_inter_embPair] at
    hz
      obtain <t, ht> := memS_nonempty hD (z.sub hz); exact Set.notMem_empty t ht
    * refine Or.inr (Or.inr <P  I  P', Q  I  Q', ?_, ?_, by rw [inj_1_inter,
    prodNbhd_inter], ?_>)
      * have hz := z.inter_mem hzPQ hzPQ'; rw [embPair_inter] at hz
        exact (memS_embPair_inv hD (z.sub hz)).1
      * have hz := z.inter_mem hzPQ hzPQ'; rw [embPair_inter] at hz
        exact (memS_embPair_inv hD (z.sub hz)).2
      * have hz := z.inter_mem hzPQ hzPQ'; rwa [embPair_inter] at hz
  up_mem := by
    rintro W W' (rfl | <X, hX, rfl, hzX> | <P, Q, hP, hQ, rfl, hzPQ>) hW' hsub
    * left
      exact eq_sumMaster_of_subset hW' hsub
```

```
    * rcases hW' with rfl | <X', hX', rfl> | <V', hV', rfl>
      * exact Or.inl rfl
      * refine Or.inr (Or.inl <X', hX', rfl, ?_>)
        exact z.up_mem hzX (MemS.zero hX') (embZero_subset.mpr (inj_0_subset_inj_0.mp
    hsub))
      * obtain <a, ha> := hD X hX
        exact absurd (hsub (il_mem_inj_0.mpr ha)) il_mem_inj_1
    * rcases hW' with rfl | <X', hX', rfl> | <V', hV', rfl>
      * exact Or.inl rfl
      * obtain <b, hb> := prod_dsharp_nonempty D hD (prodNbhd P Q) (prod_mem_prodNbhd
    hP hQ)
        exact absurd (hsub (ir_mem_inj_1.mpr hb)) ir_mem_inj_0
      * obtain <P', Q', hP', hQ', rfl> := hV'
        refine Or.inr (Or.inr <P', Q', hP', hQ', rfl, ?_>)
        obtain <hPP, hQQ> := prodNbhd_subset_iff.mp (inj_1_subset_inj_1.mp hsub)
        exact z.up_mem hzPQ (MemS.pair hP' hQ') (embPair_subset.mpr <hPP, hQQ>)

@[simp] theorem toS_mem_inj_0 {z : (Dsharp D hD).Element} {X : Set a} (hX : D.mem X) :
    (toS D hD z).mem (inj_0 X)  <->  z.mem (embZero X) := by
  constructor
  * rintro (h0 | <X', hX', heq, hz> | <P, Q, hP, hQ, heq, hz>)
    * exact absurd (h0  |>  none_mem_sumMaster) none_mem_inj_0
    * rwa [inj_0_injective heq]
    * obtain <a, ha> := hD X hX
      exact absurd (heq  |>  il_mem_inj_0.mpr ha) il_mem_inj_1
  * intro hz; exact Or.inr (Or.inl <X, hX, rfl, hz>)

@[simp] theorem toS_mem_inj_1 {z : (Dsharp D hD).Element} {P Q : Set (List Bool  x  a)
    }
    (hP : MemS D P) (hQ : MemS D Q) :
    (toS D hD z).mem (inj_1 (prodNbhd P Q))  <->  z.mem (embPair P Q) := by
  constructor
  * rintro (h0 | <X', hX', heq, hz> | <P', Q', hP', hQ', heq, hz>)
    * exact absurd (h0  |>  none_mem_sumMaster) none_mem_inj_1
    * obtain <a, ha> := hD X' hX'
      exact absurd (heq  |>  il_mem_inj_0.mpr ha) il_mem_inj_1
    * obtain <hPP, hQQ> := prodNbhd_injective (inj_1_injective heq)
      rw [hPP, hQQ]; exact hz
  * intro hz; exact Or.inr (Or.inr <P, Q, hP, hQ, rfl, hz>)

theorem embPair_ne_Gamma (P Q : Set (List Bool  x  a)) : embPair P Q  !=  Gamma D :=
    by
  intro h
  obtain <a, ha> := hD D.master D.master_mem
  have hmem : (([], a) : List Bool  x  a)  in  Gamma D := ha
  rw [ <-  h] at hmem
  rcases hmem with <p', hp', -> | <q', hq', ->
  * simp at hp'
  * simp at hq'

theorem embZero_ne_Gamma (X : Set a) : embZero X  !=  Gamma D := by
  intro h
  obtain <a, ha> := hD D.master D.master_mem
  have hmem : ((true :: [], a) : List Bool  x  a)  in  Gamma D := ha
  rw [ <-  h] at hmem; exact absurd hmem.1 (by simp)
```

```
theorem embZero_ne_embPair {X : Set a} (hX : D.mem X) (P Q : Set (List Bool  x  a)) :
    embZero X  !=  embPair P Q := by
  intro h
  obtain <a, ha> := hD X hX
  have hmem : (([], a) : List Bool  x  a)  in  embPair P Q := h  |>  (<rfl, ha> : ([],
    a)  in  embZero X)
  rcases hmem with <p', hp', -> | <q', hq', ->
  * simp at hp'
  * simp at hq'

/-- **Example 6.1 -- the inverse half of the domain equation.** Sends an element `s` of
`D + (D^S  x  D^S)` back to an element of `D^S`. -/
def fromS (s : (sumSys D hD).Element) : (Dsharp D hD).Element where
  mem W := W = Gamma D
     \/  (exists  X, D.mem X  /\  W = embZero X  /\  s.mem (inj_0 X))
     \/  (exists  P Q, MemS D P  /\  MemS D Q  /\  W = embPair P Q  /\  s.mem (inj_1 (
    prodNbhd P Q)))
  sub := by
    rintro W (rfl | <X, hX, rfl, -> | <P, Q, hP, hQ, rfl, ->)
    * exact MemS.gamma
    * exact MemS.zero hX
    * exact MemS.pair hP hQ
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | <X, hX, rfl, hsX> | <P, Q, hP, hQ, rfl, hsPQ>)
      (rfl | <X', hX', rfl, hsX'> | <P', Q', hP', hQ', rfl, hsPQ'>)
    * exact Or.inl (by rw [Set.inter_self])
    * exact Or.inr (Or.inl <X', hX',
        by rw [Set.inter_eq_right.mpr (embZero_subset_Gamma hX')], hsX'>)
    * exact Or.inr (Or.inr <P', Q', hP', hQ',
        by rw [Set.inter_eq_right.mpr
          (embPair_subset_Gamma (memS_subset_gamma hP') (memS_subset_gamma hQ'))],
    hsPQ'>)
    * exact Or.inr (Or.inl <X, hX,
        by rw [Set.inter_eq_left.mpr (embZero_subset_Gamma hX)], hsX>)
    * refine Or.inr (Or.inl <X  I  X', ?_, by rw [embZero_inter], ?_>)
      * have hs := s.inter_mem hsX hsX'; rw [inj_0_inter] at hs
        exact sum_mem_inj_0_inv D hD (s.sub hs)
      * have hs := s.inter_mem hsX hsX'; rwa [inj_0_inter] at hs
    * exfalso
      have hs := s.inter_mem hsX hsPQ'; rw [inj_0_inter_inj_1] at hs
      obtain <t, ht> := sumSys_mem_nonempty D hD (s.sub hs); exact Set.notMem_empty t
    ht
    * exact Or.inr (Or.inr <P, Q, hP, hQ,
        by rw [Set.inter_eq_left.mpr
          (embPair_subset_Gamma (memS_subset_gamma hP) (memS_subset_gamma hQ))], hsPQ
    >)
    * exfalso
      have hs := s.inter_mem hsPQ hsX'; rw [Set.inter_comm, inj_0_inter_inj_1] at hs
      obtain <t, ht> := sumSys_mem_nonempty D hD (s.sub hs); exact Set.notMem_empty t
    ht
    * refine Or.inr (Or.inr <P  I  P', Q  I  Q', ?_, ?_, by rw [embPair_inter], ?_>)
      * have hs := s.inter_mem hsPQ hsPQ'; rw [inj_1_inter, prodNbhd_inter] at hs
        exact (prod_mem_prodNbhd_iff.mp (sum_mem_inj_1_inv D hD (s.sub hs))).1
      * have hs := s.inter_mem hsPQ hsPQ'; rw [inj_1_inter, prodNbhd_inter] at hs
        exact (prod_mem_prodNbhd_iff.mp (sum_mem_inj_1_inv D hD (s.sub hs))).2
```

```
      * have hs := s.inter_mem hsPQ hsPQ'; rw [inj_1_inter, prodNbhd_inter] at hs;
    exact hs
  up_mem := by
    rintro W W' (rfl | <X, hX, rfl, hsX> | <P, Q, hP, hQ, rfl, hsPQ>) hW' hsub
    * exact Or.inl (Set.Subset.antisymm (memS_subset_gamma hW') hsub)
    * cases hW' with
      | gamma => exact Or.inl rfl
      | @zero X' hX' =>
        refine Or.inr (Or.inl <X', hX', rfl, ?_>)
        exact s.up_mem hsX (Or.inr (Or.inl <X', hX', rfl>))
          (inj_0_subset_inj_0.mpr (embZero_subset.mp hsub))
      | @pair P' Q' hP' hQ' =>
        exfalso
        obtain <a, ha> := hD X hX
        have hmem : (([], a) : List Bool  x  a)  in  embPair P' Q' := hsub <rfl, ha>
        rcases hmem with <p', hp', -> | <q', hq', ->
        * simp at hp'
        * simp at hq'
    * cases hW' with
      | gamma => exact Or.inl rfl
      | @zero X' hX' =>
        exfalso
        obtain <t, ht> := embPair_nonempty (memS_nonempty hD hP) (Q := Q)
        have hcons : t.1  !=  [] := by
          rcases ht with <p', hp', -> | <q', hq', ->
          * simp [hp']
          * simp [hq']
        have hz : t  in  embZero X' := hsub ht
        exact hcons hz.1
      | @pair P' Q' hP' hQ' =>
        refine Or.inr (Or.inr <P', Q', hP', hQ', rfl, ?_>)
        obtain <hPP, hQQ> := embPair_subset.mp hsub
        exact s.up_mem hsPQ (Or.inr (Or.inr <prodNbhd P' Q', prod_mem_prodNbhd hP' hQ
    ', rfl>))
          (inj_1_subset_inj_1.mpr (prodNbhd_subset_iff.mpr <hPP, hQQ>))

@[simp] theorem fromS_mem_embZero {s : (sumSys D hD).Element} {X : Set a} (hX : D.mem
    X) :
    (fromS D hD s).mem (embZero X)  <->  s.mem (inj_0 X) := by
  constructor
  * rintro (h0 | <X', hX', heq, hs> | <P, Q, hP, hQ, heq, hs>)
    * exact absurd h0 (embZero_ne_Gamma D hD X)
    * rwa [embZero_injective heq]
    * exact absurd heq (embZero_ne_embPair D hD hX P Q)
  * intro hs; exact Or.inr (Or.inl <X, hX, rfl, hs>)

@[simp] theorem fromS_mem_embPair {s : (sumSys D hD).Element} {P Q : Set (List Bool  x
      a)}
    (hP : MemS D P) (hQ : MemS D Q) :
    (fromS D hD s).mem (embPair P Q)  <->  s.mem (inj_1 (prodNbhd P Q)) := by
  constructor
  * rintro (h0 | <X', hX', heq, hs> | <P', Q', hP', hQ', heq, hs>)
    * exact absurd h0 (embPair_ne_Gamma D hD P Q)
    * exact absurd heq.symm (embZero_ne_embPair D hD hX' P Q)
    * obtain <hPP, hQQ> := embPair_injective heq
      rw [hPP, hQQ]; exact hs
```

```
  * intro hs; exact Or.inr (Or.inr <P, Q, hP, hQ, rfl, hs>)

/-- `fromS  comp  toS = id`. -/
theorem fromS_toS (z : (Dsharp D hD).Element) : fromS D hD (toS D hD z) = z := by
  apply NeighborhoodSystem.Element.ext
  intro W
  constructor
  * rintro (rfl | <X, hX, rfl, hs> | <P, Q, hP, hQ, rfl, hs>)
    * exact z.master_mem
    * exact (toS_mem_inj_0 D hD hX).mp hs
    * exact (toS_mem_inj_1 D hD hP hQ).mp hs
  * intro hW
    cases z.sub hW with
    | gamma => exact Or.inl rfl
    | @zero X hX => exact Or.inr (Or.inl <X, hX, rfl, (toS_mem_inj_0 D hD hX).mpr hW>)
    | @pair P Q hP hQ => exact Or.inr (Or.inr <P, Q, hP, hQ, rfl, (toS_mem_inj_1 D hD
    hP hQ).mpr hW>)

/-- Scott's injection `x^S = {Gamma}  U  {0X ? X  in  x}` of a `D`-element into `D^S`.
    -/
def inSharp (x : D.Element) : (Dsharp D hD).Element where
  mem W := W = Gamma D  \/  exists  X, x.mem X  /\  W = embZero X
  sub := by
    rintro W (rfl | <X, hX, rfl>)
    * exact MemS.gamma
    * exact MemS.zero (x.sub hX)
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | <X, hX, rfl>) (rfl | <X', hX', rfl>)
    * exact Or.inl (by rw [Set.inter_self])
    * exact Or.inr <X', hX', by rw [Set.inter_eq_right.mpr (embZero_subset_Gamma (x.
    sub hX'))]>
    * exact Or.inr <X, hX, by rw [Set.inter_eq_left.mpr (embZero_subset_Gamma (x.sub
    hX))]>
    * exact Or.inr <X  I  X', x.inter_mem hX hX', by rw [embZero_inter]>
  up_mem := by
    rintro W W' (rfl | <X, hX, rfl>) hW' hsub
    * exact Or.inl (Set.Subset.antisymm (memS_subset_gamma hW') hsub)
    * cases hW' with
      | gamma => exact Or.inl rfl
      | @zero X' hX' => exact Or.inr <X', x.up_mem hX hX' (embZero_subset.mp hsub),
    rfl>
      | @pair P Q hP hQ =>
        exfalso
        obtain <a, ha> := hD X (x.sub hX)
        have hmem : (([], a) : List Bool  x  a)  in  embPair P Q := hsub <rfl, ha>
        rcases hmem with <p', hp', -> | <q', hq', ->
        * simp at hp'
        * simp at hq'

/-- Scott's pairing `<x, y> = {Gamma}  U  {1P  U  2Q ? P  in  x, Q  in  y}` of two `D^S
    `-elements. -/
def pairSharp (x y : (Dsharp D hD).Element) : (Dsharp D hD).Element where
  mem W := W = Gamma D  \/  exists  P Q, x.mem P  /\  y.mem Q  /\  W = embPair P Q
  sub := by
    rintro W (rfl | <P, Q, hP, hQ, rfl>)
```

```
    * exact MemS.gamma
    * exact MemS.pair (x.sub hP) (y.sub hQ)
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | <P, Q, hP, hQ, rfl>) (rfl | <P', Q', hP', hQ', rfl>)
    * exact Or.inl (by rw [Set.inter_self])
    * exact Or.inr <P', Q', hP', hQ', by
        rw [Set.inter_eq_right.mpr (embPair_subset_Gamma (memS_subset_gamma (x.sub hP
    '))
          (memS_subset_gamma (y.sub hQ')))]>
    * exact Or.inr <P, Q, hP, hQ, by
        rw [Set.inter_eq_left.mpr (embPair_subset_Gamma (memS_subset_gamma (x.sub hP))
          (memS_subset_gamma (y.sub hQ)))]>
    * exact Or.inr <P  I  P', Q  I  Q', x.inter_mem hP hP', y.inter_mem hQ hQ', by rw
    [embPair_inter]>
  up_mem := by
    rintro W W' (rfl | <P, Q, hP, hQ, rfl>) hW' hsub
    * exact Or.inl (Set.Subset.antisymm (memS_subset_gamma hW') hsub)
    * cases hW' with
      | gamma => exact Or.inl rfl
      | @zero X' hX' =>
        exfalso
        obtain <t, ht> := embPair_nonempty (memS_nonempty hD (x.sub hP)) (Q := Q)
        have hcons : t.1  !=  [] := by
          rcases ht with <p', hp', -> | <q', hq', ->
          * simp [hp']
          * simp [hq']
        exact hcons (hsub ht).1
      | @pair P' Q' hP' hQ' =>
        obtain <hPP, hQQ> := embPair_subset.mp hsub
        exact Or.inr <P', Q', x.up_mem hP hP' hPP, y.up_mem hQ hQ' hQQ, rfl>

/-- `toS  comp  fromS = id`. -/
theorem toS_fromS (s : (sumSys D hD).Element) : toS D hD (fromS D hD s) = s := by
  apply NeighborhoodSystem.Element.ext
  intro W
  constructor
  * rintro (rfl | <X, hX, rfl, hs> | <P, Q, hP, hQ, rfl, hs>)
    * exact s.master_mem
    * exact (fromS_mem_embZero D hD hX).mp hs
    * exact (fromS_mem_embPair D hD hP hQ).mp hs
  * intro hW
    rcases s.sub hW with rfl | <X, hX, rfl> | <V, hV, rfl>
    * exact Or.inl rfl
    * exact Or.inr (Or.inl <X, hX, rfl, (fromS_mem_embZero D hD hX).mpr hW>)
    * obtain <P, Q, hP, hQ, rfl> := hV
      exact Or.inr (Or.inr <P, Q, hP, hQ, rfl, (fromS_mem_embPair D hD hP hQ).mpr hW>)

/-- **Example 6.1 (Scott 1981, PRG-19) -- the domain equation, as an order-isomorphism
    .**
`|D^S| ?o |D + (D^S  x  D^S)|`. -/
def dsharpEquiv : (Dsharp D hD).Element ?o (sumSys D hD).Element where
  toFun := toS D hD
  invFun := fromS D hD
  left_inv := fromS_toS D hD
  right_inv := toS_fromS D hD
```

```
  map_rel_iff' := by
    intro z z'
    constructor
    * intro h X hX
      cases z.sub hX with
      | gamma => exact z'.master_mem
      | @zero A hA =>
        exact (toS_mem_inj_0 D hD hA).mp (h _ (Or.inr (Or.inl <A, hA, rfl, hX>)))
      | @pair P Q hP hQ =>
        exact (toS_mem_inj_1 D hD hP hQ).mp (h _ (Or.inr (Or.inr <P, Q, hP, hQ, rfl,
    hX>)))
    * intro h W hW
      rcases hW with rfl | <X, hX, rfl, hzX> | <P, Q, hP, hQ, rfl, hzPQ>
      * exact Or.inl rfl
      * exact Or.inr (Or.inl <X, hX, rfl, h _ hzX>)
      * exact Or.inr (Or.inr <P, Q, hP, hQ, rfl, h _ hzPQ>)

/-- **Example 6.1 (Scott 1981, PRG-19) -- the domain equation `D^S  ~=  D + (D^S  x  D^
    S)`.** The tree
algebra `D^S` is, as a domain, isomorphic to `D + (D^S  x  D^S)` -- Scott's defining
    isomorphism, "as can
be seen by reference to the equation for `D^S` and the definitions of `+` and ` x `".
    -/
theorem dsharp_domain_equation :
    Dsharp D hD  ~= ? sum D (prod (Dsharp D hD) (Dsharp D hD)) hD (
    prod_dsharp_nonempty D hD) :=
  <dsharpEquiv D hD>

/-- **Example 6.1 (Scott 1981, PRG-19).** `fun x. x^S` is an *isomorphic injection*: `x
    ^S  sqsub  x'^S  <->  x  sqsub  x'`
(in particular one-one). -/
theorem inSharp_le_iff {x x' : D.Element} : inSharp D hD x  <=  inSharp D hD x'  <->
    x  <=  x' := by
  constructor
  * intro h X hX
    rcases h (embZero X) (Or.inr <X, hX, rfl>) with h0 | <X', hX', heq>
    * exact absurd h0 (embZero_ne_Gamma D hD X)
    * rw [ <-  embZero_injective heq] at hX'; exact hX'
  * intro h W hW
    rcases hW with rfl | <X, hX, rfl>
    * exact Or.inl rfl
    * exact Or.inr <X, h _ hX, rfl>

/-- **Example 6.1 (Scott 1981, PRG-19).** `fun x, y. <x, y>` is an *isomorphic
    injection*:
`<x, y>  sqsub  <x', y'>  <->  x  sqsub  x'  /\  y  sqsub  y'`. -/
theorem pairSharp_le_iff {x x' y y' : (Dsharp D hD).Element} :
    pairSharp D hD x y  <=  pairSharp D hD x' y'  <->  x  <=  x'  /\  y  <=  y' := by
  constructor
  * intro h
    refine <fun P hP => ?_, fun Q hQ => ?_>
    * rcases h (embPair P (Gamma D)) (Or.inr <P, Gamma D, hP, y.master_mem, rfl>) with
        h0 | <P', Q', hP', hQ', heq>
      * exact absurd h0 (embPair_ne_Gamma D hD P (Gamma D))
      * obtain <hPP, -> := embPair_injective heq; rw [ <-  hPP] at hP'; exact hP'
    * rcases h (embPair (Gamma D) Q) (Or.inr <Gamma D, Q, x.master_mem, hQ, rfl>) with
```

```
        h0 | <P', Q', hP', hQ', heq>
      * exact absurd h0 (embPair_ne_Gamma D hD (Gamma D) Q)
      * obtain <-, hQQ> := embPair_injective heq; rw [ <-  hQQ] at hQ'; exact hQ'
  * rintro <hx, hy> W hW

    rcases hW with rfl | <P, Q, hP, hQ, rfl>
    * exact Or.inl rfl
    * exact Or.inr <P, Q, hx _ hP, hy _ hQ, rfl>

end Equation

end Example61
end

end DomainTheory.NeighbourhoodSystems
end
```

3.7.21 Exercise 6.19 — strict sum and product

For the sake of uniformity restrict attention to systems $\mathcal{D}$ on sets $\Delta \subseteq \{0,1\}^*$, where $\Lambda \in \Delta$ and $\emptyset \notin \mathcal{D}$, and to the category of strict maps. Define sum and product by:

$$\mathcal{D}_0 + \mathcal{D}_1 = \{\{\Lambda\} \cup 0\Delta_0 \cup 1\Delta_1\} \cup \{0X \mid X \in \mathcal{D}_0\} \cup \{1Y \mid Y \in \mathcal{D}_1\},$$

$$\mathcal{D}_0 \times \mathcal{D}_1 = \{\{\Lambda\} \cup 0X \cup 1Y \mid X \in \mathcal{D}_0 \text{ and } Y \in \mathcal{D}_1\}.$$

Are these correct up to isomorphism? Now generate all constructs $T(X)$ formed by the constants (that is, $T(X) = \mathcal{D}$ for a fixed $\mathcal{D}$), by the identity ($T(X) = X$), and by sums and products ($T_0(X) + T_1(X)$, etc.). Show that these are all functors, continuous on maps, and monotone and continuous on domains.

Exercise_6_19.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.Example_6_2

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap ExampleB Example44 Example62
open NeighborhoodSystem ApproximableMap Example62 ExampleB
section
namespace Exercise619
variable {D_0 D_1 : NeighborhoodSystem Str}
/- concordance: source L4087 EXERCISE 6.19 strict sum and product -/
/-- **Exercise 6.19 -- the concrete sum system `?_0 + ?_1` over `{0,1}*`.** A
    neighbourhood is the
master `{?}  U  0Delta_0  U  1Delta_1`, a left copy `0X` (`X  in  ?_0`), or a right
    copy `1Y` (`Y  in  ?_1`). The standing
assumption `{}  notin  ?_i` (`h_0`, `h_1`) makes the two tagged copies disjoint, so the
     system is closed under
consistent intersection. -/
```

```
def sumTok (D_0 D_1 : NeighborhoodSystem Str)
    (h_0 : forall  X, D_0.mem X  ->  X.Nonempty) (h_1 : forall  Y, D_1.mem Y  ->  Y.
    Nonempty) :
    NeighborhoodSystem Str where
  mem W := W = sumTokMaster D_0 D_1  \/  (exists  X, D_0.mem X  /\  W = embBit false X
    )  \/
    (exists  Y, D_1.mem Y  /\  W = embBit true Y)
  master := sumTokMaster D_0 D_1
  master_nonempty := <[], nil_mem_sumTokMaster>
  master_mem := Or.inl rfl
  sub_master := by
    rintro W (rfl | <X, hX, rfl> | <Y, hY, rfl>)
    * exact subset_rfl
    * exact embF_subset_sumTokMaster hX
    * exact embT_subset_sumTokMaster hY
  inter_mem := by
    have hne : forall  W, (W = sumTokMaster D_0 D_1  \/  (exists  X, D_0.mem X  /\  W
    = embBit false X)  \/
        (exists  Y, D_1.mem Y  /\  W = embBit true Y))  ->  (W : Set Str).Nonempty :=
    by
      rintro W (rfl | <X, hX, rfl> | <Y, hY, rfl>)
      * exact <[], nil_mem_sumTokMaster>
      * exact embBit_nonempty (h_0 X hX)
      * exact embBit_nonempty (h_1 Y hY)
    rintro W W' Z hW hW' hZ hZsub
    rcases hW with rfl | <X, hX, rfl> | <Y, hY, rfl>
    * rcases hW' with rfl | <X', hX', rfl> | <Y', hY', rfl>
      * rw [Set.inter_self]; exact Or.inl rfl
      * rw [sumTokMaster_inter_embF hX']; exact Or.inr (Or.inl <X', hX', rfl>)
      * rw [sumTokMaster_inter_embT hY']; exact Or.inr (Or.inr <Y', hY', rfl>)
    * rcases hW' with rfl | <X', hX', rfl> | <Y', hY', rfl>
      * rw [Set.inter_comm, sumTokMaster_inter_embF hX]; exact Or.inr (Or.inl <X, hX,
    rfl>)
      * rw [embBit_inter] at hZsub  |-
        rcases hZ with rfl | <Z_0, hZ_0, rfl> | <Z_1, hZ_1, rfl>
        * exact absurd (hZsub nil_mem_sumTokMaster) nil_not_mem_embBit
        * exact Or.inr (Or.inl <X  I  X', D_0.inter_mem hX hX' hZ_0 (embBit_subset.mp
    hZsub), rfl>)
        * obtain <b, hb> := h_1 Z_1 hZ_1
          obtain <w', he, -> := hZsub (<b, rfl, hb> : (true :: b)  in  embBit true Z_1
    )
          simp only [List.cons.injEq] at he; exact absurd he.1 (by decide)
      * rw [embBit_inter_ne (show (false : Bool)  !=  true by decide)] at hZsub  |-
        obtain <t, ht> := hne Z hZ; exact absurd (hZsub ht) (Set.notMem_empty t)
    * rcases hW' with rfl | <X', hX', rfl> | <Y', hY', rfl>
      * rw [Set.inter_comm, sumTokMaster_inter_embT hY]; exact Or.inr (Or.inr <Y, hY,
    rfl>)
      * rw [embBit_inter_ne (show (true : Bool)  !=  false by decide)] at hZsub  |-
        obtain <t, ht> := hne Z hZ; exact absurd (hZsub ht) (Set.notMem_empty t)
      * rw [embBit_inter] at hZsub  |-
        rcases hZ with rfl | <Z_0, hZ_0, rfl> | <Z_1, hZ_1, rfl>
        * exact absurd (hZsub nil_mem_sumTokMaster) nil_not_mem_embBit
        * obtain <a, ha> := h_0 Z_0 hZ_0
          obtain <w', he, -> := hZsub (<a, rfl, ha> : (false :: a)  in  embBit false
    Z_0)
          simp only [List.cons.injEq] at he; exact absurd he.1 (by decide)
```

```
        * exact Or.inr (Or.inr <Y  I  Y', D_1.inter_mem hY hY' hZ_1 (embBit_subset.mp
    hZsub), rfl>)

end Exercise619
end

section
namespace Exercise619

variable {D_0 D_1 : NeighborhoodSystem Str}

variable {h_0 : forall  X, D_0.mem X  ->  X.Nonempty} {h_1 : forall  Y, D_1.mem Y  ->
     Y.Nonempty}

theorem sumTok_mem_embF {X : Set Str} (hX : D_0.mem X) :
    (sumTok D_0 D_1 h_0 h_1).mem (embBit false X) := Or.inr (Or.inl <X, hX, rfl>)

theorem sumTok_mem_embF_inv {W : Set Str} (h : (sumTok D_0 D_1 h_0 h_1).mem (embBit
    false W)) :
    D_0.mem W := by
  rcases h with h0 | <X, hX, heq> | <Y, hY, heq>
  * exact absurd (h0.symm  |>  nil_mem_sumTokMaster) nil_not_mem_embBit
  * rw [embBit_injective heq]; exact hX
  * exact absurd heq.symm (embBit_ne (show (true : Bool)  !=  false by decide) (h_1 Y
    hY))

theorem sumTok_mem_embT {Y : Set Str} (hY : D_1.mem Y) :
    (sumTok D_0 D_1 h_0 h_1).mem (embBit true Y) := Or.inr (Or.inr <Y, hY, rfl>)

theorem sumTok_mem_embT_inv {W : Set Str} (h : (sumTok D_0 D_1 h_0 h_1).mem (embBit
    true W)) :
    D_1.mem W := by
  rcases h with h0 | <X, hX, heq> | <Y, hY, heq>
  * exact absurd (h0.symm  |>  nil_mem_sumTokMaster) nil_not_mem_embBit
  * exact absurd heq.symm (embBit_ne (show (false : Bool)  !=  true by decide) (h_0 X
    hX))
  * rw [embBit_injective heq]; exact hY

theorem sumTok_mem_nonempty {W : Set Str} (h : (sumTok D_0 D_1 h_0 h_1).mem W) : W.
    Nonempty := by
  rcases h with rfl | <X, hX, rfl> | <Y, hY, rfl>
  * exact <[], nil_mem_sumTokMaster>
  * exact embBit_nonempty (h_0 X hX)
  * exact embBit_nonempty (h_1 Y hY)

theorem sum_mem_inj_0_inv {X : Set Str} (h : (sum D_0 D_1 h_0 h_1).mem (inj_0 X)) :
    D_0.mem X := by
  rcases h with h0 | <X', hX', heq> | <Y', hY', heq>
  * exact absurd (h0  |>  none_mem_sumMaster) none_mem_inj_0
  * rw [inj_0_injective heq]; exact hX'
  * obtain <b, hb> := h_1 Y' hY'
    exact absurd (heq  |>  ir_mem_inj_1.mpr hb) ir_mem_inj_0

theorem sum_mem_inj_1_inv {Y : Set Str} (h : (sum D_0 D_1 h_0 h_1).mem (inj_1 Y)) :
    D_1.mem Y := by
  rcases h with h0 | <X', hX', heq> | <Y', hY', heq>
```

```
  * exact absurd (h0  |>  none_mem_sumMaster) none_mem_inj_1
  * obtain <a, ha> := h_0 X' hX'
    exact absurd (heq  |>  il_mem_inj_0.mpr ha) il_mem_inj_1
  * rw [inj_1_injective heq]; exact hY'

theorem sum_mem_nonempty {W : Set (Option (Str ? Str))} (h : (sum D_0 D_1 h_0 h_1).mem
     W) :
    W.Nonempty := by
  rcases h with rfl | <X, hX, rfl> | <Y, hY, rfl>
  * exact <none, none_mem_sumMaster>
  * exact inj_0_nonempty (h_0 X hX)
  * exact inj_1_nonempty (h_1 Y hY)

/-- The forward half of the isomorphism, a token-level analogue of `Example62.toBB`. -/
def toSum (x : (sumTok D_0 D_1 h_0 h_1).Element) : (sum D_0 D_1 h_0 h_1).Element where
  mem W := W = sumMaster D_0 D_1
     \/  (exists  X, D_0.mem X  /\  W = inj_0 X  /\  x.mem (embBit false X))
     \/  (exists  Y, D_1.mem Y  /\  W = inj_1 Y  /\  x.mem (embBit true Y))
  sub := by
    rintro W (rfl | <X, hX, rfl, -> | <Y, hY, rfl, ->)
    * exact Or.inl rfl
    * exact Or.inr (Or.inl <X, hX, rfl>)
    * exact Or.inr (Or.inr <Y, hY, rfl>)
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | <X, hX, rfl, hzX> | <Y, hY, rfl, hzY>)
      (rfl | <X', hX', rfl, hzX'> | <Y', hY', rfl, hzY'>)
    * exact Or.inl (by rw [Set.inter_self])
    * exact Or.inr (Or.inl <X', hX', by rw [sumMaster_inter_inj_0 hX'], hzX'>)
    * exact Or.inr (Or.inr <Y', hY', by rw [sumMaster_inter_inj_1 hY'], hzY'>)
    * exact Or.inr (Or.inl <X, hX, by rw [Set.inter_comm, sumMaster_inter_inj_0 hX],
    hzX>)
    * refine Or.inr (Or.inl <X  I  X', ?_, by rw [inj_0_inter], ?_>)
      * have hz := x.inter_mem hzX hzX'; rw [embBit_inter] at hz; exact
    sumTok_mem_embF_inv (x.sub hz)
      * have hz := x.inter_mem hzX hzX'; rwa [embBit_inter] at hz
    * exfalso
      have hz := x.inter_mem hzX hzY'
      rw [embBit_inter_ne (show (false : Bool)  !=  true by decide)] at hz
      obtain <t, ht> := sumTok_mem_nonempty (x.sub hz); exact Set.notMem_empty t ht
    * exact Or.inr (Or.inr <Y, hY, by rw [Set.inter_comm, sumMaster_inter_inj_1 hY],
    hzY>)
    * exfalso
      have hz := x.inter_mem hzY hzX'
      rw [embBit_inter_ne (show (true : Bool)  !=  false by decide)] at hz
      obtain <t, ht> := sumTok_mem_nonempty (x.sub hz); exact Set.notMem_empty t ht
    * refine Or.inr (Or.inr <Y  I  Y', ?_, by rw [inj_1_inter], ?_>)
      * have hz := x.inter_mem hzY hzY'; rw [embBit_inter] at hz; exact
    sumTok_mem_embT_inv (x.sub hz)
      * have hz := x.inter_mem hzY hzY'; rwa [embBit_inter] at hz
  up_mem := by
    rintro W W' (rfl | <X, hX, rfl, hzX> | <Y, hY, rfl, hzY>) hW' hsub
    * exact Or.inl (eq_sumMaster_of_subset hW' hsub)
    * rcases hW' with rfl | <X', hX', rfl> | <Y', hY', rfl>
      * exact Or.inl rfl
      * refine Or.inr (Or.inl <X', hX', rfl, ?_>)
```

```
        exact x.up_mem hzX (sumTok_mem_embF hX') (embBit_subset.mpr (
    inj_0_subset_inj_0.mp hsub))
      * exfalso
        obtain <a, ha> := h_0 X hX
        exact absurd (hsub (il_mem_inj_0.mpr ha)) il_mem_inj_1
    * rcases hW' with rfl | <X', hX', rfl> | <Y', hY', rfl>
      * exact Or.inl rfl
      * exfalso
        obtain <b, hb> := h_1 Y hY
        exact absurd (hsub (ir_mem_inj_1.mpr hb)) ir_mem_inj_0
      * refine Or.inr (Or.inr <Y', hY', rfl, ?_>)
        exact x.up_mem hzY (sumTok_mem_embT hY') (embBit_subset.mpr (
    inj_1_subset_inj_1.mp hsub))

@[simp] theorem toSum_mem_inj_0 {x : (sumTok D_0 D_1 h_0 h_1).Element} {X : Set Str} (
    hX : D_0.mem X) :
    (toSum x).mem (inj_0 X)  <->  x.mem (embBit false X) := by
  constructor
  * rintro (h0 | <X', hX', heq, hz> | <Y', hY', heq, hz>)
    * exact absurd (h0  |>  none_mem_sumMaster) none_mem_inj_0
    * rwa [inj_0_injective heq]
    * obtain <a, ha> := h_0 X hX
      exact absurd (heq  |>  il_mem_inj_0.mpr ha) il_mem_inj_1
  * intro hz; exact Or.inr (Or.inl <X, hX, rfl, hz>)

@[simp] theorem toSum_mem_inj_1 {x : (sumTok D_0 D_1 h_0 h_1).Element} {Y : Set Str} (
    hY : D_1.mem Y) :
    (toSum x).mem (inj_1 Y)  <->  x.mem (embBit true Y) := by
  constructor
  * rintro (h0 | <X', hX', heq, hz> | <Y', hY', heq, hz>)
    * exact absurd (h0  |>  none_mem_sumMaster) none_mem_inj_1
    * obtain <a, ha> := h_0 X' hX'
      exact absurd (heq  |>  il_mem_inj_0.mpr ha) il_mem_inj_1
    * rwa [inj_1_injective heq]
  * intro hz; exact Or.inr (Or.inr <Y, hY, rfl, hz>)

/-- The inverse half of the isomorphism, a token-level analogue of `Example62.fromBB`.
    -/
def fromSum (s : (sum D_0 D_1 h_0 h_1).Element) : (sumTok D_0 D_1 h_0 h_1).Element
    where
  mem W := W = sumTokMaster D_0 D_1
     \/  (exists  X, D_0.mem X  /\  W = embBit false X  /\  s.mem (inj_0 X))
     \/  (exists  Y, D_1.mem Y  /\  W = embBit true Y  /\  s.mem (inj_1 Y))
  sub := by
    rintro W (rfl | <X, hX, rfl, -> | <Y, hY, rfl, ->)
    * exact Or.inl rfl
    * exact Or.inr (Or.inl <X, hX, rfl>)
    * exact Or.inr (Or.inr <Y, hY, rfl>)
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | <X, hX, rfl, hsX> | <Y, hY, rfl, hsY>)
      (rfl | <X', hX', rfl, hsX'> | <Y', hY', rfl, hsY'>)
    * exact Or.inl (by rw [Set.inter_self])
    * exact Or.inr (Or.inl <X', hX', by rw [sumTokMaster_inter_embF hX'], hsX'>)
    * exact Or.inr (Or.inr <Y', hY', by rw [sumTokMaster_inter_embT hY'], hsY'>)
    * exact Or.inr (Or.inl <X, hX, by rw [Set.inter_comm, sumTokMaster_inter_embF hX],
```

```
    hsX>)
    * refine Or.inr (Or.inl <X  I  X', ?_, by rw [embBit_inter], ?_>)
      * have hs := s.inter_mem hsX hsX'; rw [inj_0_inter] at hs
        exact sum_mem_inj_0_inv (s.sub hs)
      * have hs := s.inter_mem hsX hsX'; rwa [inj_0_inter] at hs
    * exfalso
      have hs := s.inter_mem hsX hsY'; rw [inj_0_inter_inj_1] at hs
      obtain <t, ht> := sum_mem_nonempty (s.sub hs); exact Set.notMem_empty t ht
    * exact Or.inr (Or.inr <Y, hY, by rw [Set.inter_comm, sumTokMaster_inter_embT hY],
     hsY>)
    * exfalso
      have hs := s.inter_mem hsY hsX'; rw [Set.inter_comm, inj_0_inter_inj_1] at hs
      obtain <t, ht> := sum_mem_nonempty (s.sub hs); exact Set.notMem_empty t ht
    * refine Or.inr (Or.inr <Y  I  Y', ?_, by rw [embBit_inter], ?_>)
      * have hs := s.inter_mem hsY hsY'; rw [inj_1_inter] at hs
        exact sum_mem_inj_1_inv (s.sub hs)
      * have hs := s.inter_mem hsY hsY'; rwa [inj_1_inter] at hs
  up_mem := by
    rintro W W' (rfl | <X, hX, rfl, hsX> | <Y, hY, rfl, hsY>) hW' hsub
    * exact Or.inl (Set.Subset.antisymm ((sumTok D_0 D_1 h_0 h_1).sub_master hW') hsub
    )
    * rcases hW' with rfl | <X', hX', rfl> | <Y', hY', rfl>
      * exact Or.inl rfl
      * refine Or.inr (Or.inl <X', hX', rfl, ?_>)
        exact s.up_mem hsX (Or.inr (Or.inl <X', hX', rfl>))
          (inj_0_subset_inj_0.mpr (embBit_subset.mp hsub))
      * exfalso
        obtain <a, ha> := h_0 X hX
        obtain <w', he, -> := hsub (<a, rfl, ha> : (false :: a)  in  embBit false X)
        rw [List.cons.injEq] at he; exact absurd he.1 (by decide)
    * rcases hW' with rfl | <X', hX', rfl> | <Y', hY', rfl>
      * exact Or.inl rfl
      * exfalso
        obtain <b, hb> := h_1 Y hY
        obtain <w', he, -> := hsub (<b, rfl, hb> : (true :: b)  in  embBit true Y)
        rw [List.cons.injEq] at he; exact absurd he.1 (by decide)
      * refine Or.inr (Or.inr <Y', hY', rfl, ?_>)
        exact s.up_mem hsY (Or.inr (Or.inr <Y', hY', rfl>))
          (inj_1_subset_inj_1.mpr (embBit_subset.mp hsub))

@[simp] theorem fromSum_mem_embF {s : (sum D_0 D_1 h_0 h_1).Element} {X : Set Str} (hX
    : D_0.mem X) :
    (fromSum s).mem (embBit false X)  <->  s.mem (inj_0 X) := by
  constructor
  * rintro (h0 | <X', hX', heq, hs> | <Y', hY', heq, hs>)
    * exact absurd h0 embF_ne_sumTokMaster
    * rwa [embBit_injective heq]
    * exact absurd heq (embBit_ne (show (false : Bool)  !=  true by decide) (h_0 X hX)
    )
  * intro hs; exact Or.inr (Or.inl <X, hX, rfl, hs>)

@[simp] theorem fromSum_mem_embT {s : (sum D_0 D_1 h_0 h_1).Element} {Y : Set Str} (hY
    : D_1.mem Y) :
    (fromSum s).mem (embBit true Y)  <->  s.mem (inj_1 Y) := by
  constructor
  * rintro (h0 | <X', hX', heq, hs> | <Y', hY', heq, hs>)
```

```
    * exact absurd h0 embT_ne_sumTokMaster
    * exact absurd heq.symm (embBit_ne (show (false : Bool)  !=  true by decide) (h_0
    X' hX'))
    * rwa [embBit_injective heq]
  * intro hs; exact Or.inr (Or.inr <Y, hY, rfl, hs>)

theorem fromSum_toSum (x : (sumTok D_0 D_1 h_0 h_1).Element) : fromSum (toSum x) = x
    := by
  apply NeighborhoodSystem.Element.ext
  intro W
  constructor
  * rintro (rfl | <X, hX, rfl, hs> | <Y, hY, rfl, hs>)
    * exact x.master_mem
    * exact (toSum_mem_inj_0 hX).mp hs
    * exact (toSum_mem_inj_1 hY).mp hs
  * intro hW
    rcases x.sub hW with rfl | <X, hX, rfl> | <Y, hY, rfl>
    * exact Or.inl rfl
    * exact Or.inr (Or.inl <X, hX, rfl, (toSum_mem_inj_0 hX).mpr hW>)
    * exact Or.inr (Or.inr <Y, hY, rfl, (toSum_mem_inj_1 hY).mpr hW>)

/-- A product neighbourhood `{?}  U  0X  U  1Y` over `{0,1}*`. -/
def prodTokNbhd (X Y : Set Str) : Set Str := insert [] (embBit false X  U  embBit true
     Y)

@[simp] theorem mem_prodTokNbhd_nil {X Y : Set Str} : ([] : Str)  in  prodTokNbhd X Y
    :=
  Set.mem_insert _ _

@[simp] theorem mem_prodTokNbhd_false {X Y : Set Str} {w' : Str} :
    (false :: w')  in  prodTokNbhd X Y  <->  w'  in  X := by
  simp only [prodTokNbhd, Set.mem_insert_iff, Set.mem_union, mem_embBit]
  constructor
  * rintro (h | <u, hu, hX> | <u, hu, hY>)
    * exact absurd h (by simp)
    * rw [List.cons.injEq] at hu; rw [hu.2]; exact hX
    * rw [List.cons.injEq] at hu; exact absurd hu.1 (by decide)
  * intro h; exact Or.inr (Or.inl <w', rfl, h>)

@[simp] theorem mem_prodTokNbhd_true {X Y : Set Str} {w' : Str} :
    (true :: w')  in  prodTokNbhd X Y  <->  w'  in  Y := by
  simp only [prodTokNbhd, Set.mem_insert_iff, Set.mem_union, mem_embBit]
  constructor
  * rintro (h | <u, hu, hX> | <u, hu, hY>)
    * exact absurd h (by simp)
    * rw [List.cons.injEq] at hu; exact absurd hu.1 (by decide)
    * rw [List.cons.injEq] at hu; rw [hu.2]; exact hY
  * intro h; exact Or.inr (Or.inr <w', rfl, h>)

/-- Scott's (2) for the product: product neighbourhoods intersect componentwise. -/
theorem prodTokNbhd_inter (X X' Y Y' : Set Str) :
    prodTokNbhd X Y  I  prodTokNbhd X' Y' = prodTokNbhd (X  I  X') (Y  I  Y') := by
  ext w
  cases w with
  | nil => simp [Set.mem_inter_iff]
  | cons b w' => cases b <;> simp [Set.mem_inter_iff]
```

```
/-- Scott's (1) for the product: inclusion of product neighbourhoods is componentwise.
    -/
theorem prodTokNbhd_subset_iff {X X' Y Y' : Set Str} :
    prodTokNbhd X Y  subseteq  prodTokNbhd X' Y'  <->  X  subseteq  X'  /\  Y
    subseteq  Y' := by
  constructor
  * intro h
    refine <fun a ha => ?_, fun b hb => ?_>
    * exact mem_prodTokNbhd_false.mp (h (mem_prodTokNbhd_false.mpr ha))
    * exact mem_prodTokNbhd_true.mp (h (mem_prodTokNbhd_true.mpr hb))
  * rintro <hX, hY> w hw
    cases w with
    | nil => exact mem_prodTokNbhd_nil
    | cons b w' =>
      cases b with
      | false => exact mem_prodTokNbhd_false.mpr (hX (mem_prodTokNbhd_false.mp hw))
      | true => exact mem_prodTokNbhd_true.mpr (hY (mem_prodTokNbhd_true.mp hw))

/-- **Exercise 6.19 -- the concrete product system `?_0  x  ?_1` over `{0,1}*`.**
    Neighbourhoods are
`{?}  U  0X  U  1Y` with `X  in  ?_0`, `Y  in  ?_1`. Closed under consistent
    intersection by Scott's (1)/(2)
together with the factors' closure. -/
def prodTok (D_0 D_1 : NeighborhoodSystem Str) : NeighborhoodSystem Str where
  mem W := exists  X Y, D_0.mem X  /\  D_1.mem Y  /\  W = prodTokNbhd X Y
  master := prodTokNbhd D_0.master D_1.master
  master_nonempty := <[], mem_prodTokNbhd_nil>
  master_mem := <D_0.master, D_1.master, D_0.master_mem, D_1.master_mem, rfl>
  inter_mem := by
    rintro W W' Z <X, Y, hX, hY, rfl> <X', Y', hX', hY', rfl> <Z_0, Z_1, hZ_0, hZ_1,
    rfl> hsub
    rw [prodTokNbhd_inter] at hsub  |-
    obtain <hsub_0, hsub_1> := prodTokNbhd_subset_iff.mp hsub
    exact <X  I  X', Y  I  Y', D_0.inter_mem hX hX' hZ_0 hsub_0, D_1.inter_mem hY hY'
    hZ_1 hsub_1, rfl>
  sub_master := by
    rintro W <X, Y, hX, hY, rfl>
    exact prodTokNbhd_subset_iff.mpr <D_0.sub_master hX, D_1.sub_master hY>

theorem prodTokNbhd_injective {X X' Y Y' : Set Str} (h : prodTokNbhd X Y = prodTokNbhd
    X' Y') :
    X = X'  /\  Y = Y' :=
  <Set.Subset.antisymm (prodTokNbhd_subset_iff.mp h.subset).1
      (prodTokNbhd_subset_iff.mp h.symm.subset).1,
   Set.Subset.antisymm (prodTokNbhd_subset_iff.mp h.subset).2
      (prodTokNbhd_subset_iff.mp h.symm.subset).2>

@[simp] theorem prodTok_mem_iff {W : Set Str} :
    (prodTok D_0 D_1).mem W  <->  exists  X Y, D_0.mem X  /\  D_1.mem Y  /\  W =
    prodTokNbhd X Y := Iff.rfl

@[simp] theorem prodTok_master : (prodTok D_0 D_1).master = prodTokNbhd D_0.master D_1
    .master := rfl

theorem prodTok_mem_prodTokNbhd {X Y : Set Str} (hX : D_0.mem X) (hY : D_1.mem Y) :
```

```
    (prodTok D_0 D_1).mem (prodTokNbhd X Y) := <X, Y, hX, hY, rfl>

/-- Scott's `z_0`: the first component of a `prodTok`-element. -/
def fstTok (z : (prodTok D_0 D_1).Element) : D_0.Element where
  mem X := D_0.mem X  /\  z.mem (prodTokNbhd X D_1.master)
  sub h := h.1
  master_mem := <D_0.master_mem, z.master_mem>
  inter_mem := by
    rintro X X' <_, hzX> <_, hzX'>
    have hz := z.inter_mem hzX hzX'
    rw [prodTokNbhd_inter, Set.inter_self] at hz
    obtain <A, B, hA, _, heq> := z.sub hz
    obtain <rfl, rfl> := prodTokNbhd_injective heq
    exact <hA, hz>
  up_mem := by
    rintro X X' <_, hzX> hX' hXX'
    exact <hX', z.up_mem hzX (prodTok_mem_prodTokNbhd hX' D_1.master_mem)
      (prodTokNbhd_subset_iff.mpr <hXX', subset_rfl>)>

@[simp] theorem mem_fstTok {z : (prodTok D_0 D_1).Element} {X : Set Str} :
    (fstTok z).mem X  <->  D_0.mem X  /\  z.mem (prodTokNbhd X D_1.master) := Iff.rfl

/-- Scott's `z_1`: the second component of a `prodTok`-element. -/
def sndTok (z : (prodTok D_0 D_1).Element) : D_1.Element where
  mem Y := D_1.mem Y  /\  z.mem (prodTokNbhd D_0.master Y)
  sub h := h.1
  master_mem := <D_1.master_mem, z.master_mem>
  inter_mem := by
    rintro Y Y' <_, hzY> <_, hzY'>
    have hz := z.inter_mem hzY hzY'
    rw [prodTokNbhd_inter, Set.inter_self] at hz
    obtain <A, B, _, hB, heq> := z.sub hz
    obtain <rfl, rfl> := prodTokNbhd_injective heq
    exact <hB, hz>
  up_mem := by
    rintro Y Y' <_, hzY> hY' hYY'
    exact <hY', z.up_mem hzY (prodTok_mem_prodTokNbhd D_0.master_mem hY')
      (prodTokNbhd_subset_iff.mpr <subset_rfl, hYY'>)>


@[simp] theorem mem_sndTok {z : (prodTok D_0 D_1).Element} {Y : Set Str} :
    (sndTok z).mem Y  <->  D_1.mem Y  /\  z.mem (prodTokNbhd D_0.master Y) := Iff.rfl

/-- The element pairing `<x, y>` for the concrete product. -/
def pairTok (x : D_0.Element) (y : D_1.Element) : (prodTok D_0 D_1).Element where
  mem W := exists  X Y, x.mem X  /\  y.mem Y  /\  W = prodTokNbhd X Y
  sub := by rintro W <X, Y, hX, hY, rfl>; exact prodTok_mem_prodTokNbhd (x.sub hX) (y.
    sub hY)
  master_mem := <D_0.master, D_1.master, x.master_mem, y.master_mem, rfl>
  inter_mem := by
    rintro W W' <X, Y, hX, hY, rfl> <X', Y', hX', hY', rfl>
    exact <X  I  X', Y  I  Y', x.inter_mem hX hX', y.inter_mem hY hY',
    prodTokNbhd_inter X X' Y Y'>
  up_mem := by
    rintro W W' <X, Y, hX, hY, rfl> <X', Y', hX', hY', rfl> hsub
    obtain <hXX', hYY'> := prodTokNbhd_subset_iff.mp hsub
```

```
    exact <X', Y', x.up_mem hX hX' hXX', y.up_mem hY hY' hYY', rfl>

theorem mem_pairTok_prodTokNbhd {x : D_0.Element} {y : D_1.Element} {X Y : Set Str} :
    (pairTok x y).mem (prodTokNbhd X Y)  <->  x.mem X  /\  y.mem Y := by
  constructor
  * rintro <X', Y', hX', hY', heq>
    obtain <rfl, rfl> := prodTokNbhd_injective heq
    exact <hX', hY'>
  * rintro <hx, hy>; exact <X, Y, hx, hy, rfl>

@[simp] theorem fstTok_pairTok (x : D_0.Element) (y : D_1.Element) : fstTok (pairTok x
     y) = x := by
  apply NeighborhoodSystem.Element.ext
  intro X
  constructor
  * rintro <hX, hmem>; exact (mem_pairTok_prodTokNbhd.mp hmem).1
  * intro hX; exact <x.sub hX, mem_pairTok_prodTokNbhd.mpr <hX, y.master_mem>>

/-- Scott's (3) for the product: membership of `X  U  Y` splits into its two slices. -/
theorem prodTok_mem_split {z : (prodTok D_0 D_1).Element} {X Y : Set Str}
    (hX : D_0.mem X) (hY : D_1.mem Y) :
    z.mem (prodTokNbhd X Y)  <->
      z.mem (prodTokNbhd X D_1.master)  /\  z.mem (prodTokNbhd D_0.master Y) := by
  constructor
  * intro h
    refine <z.up_mem h (prodTok_mem_prodTokNbhd hX D_1.master_mem) ?_,
            z.up_mem h (prodTok_mem_prodTokNbhd D_0.master_mem hY) ?_>
    * exact prodTokNbhd_subset_iff.mpr <subset_rfl, D_1.sub_master hY>
    * exact prodTokNbhd_subset_iff.mpr <D_0.sub_master hX, subset_rfl>
  * rintro <h1, h2>
    have := z.inter_mem h1 h2
    rwa [prodTokNbhd_inter, Set.inter_eq_left.mpr (D_0.sub_master hX),
      Set.inter_eq_right.mpr (D_1.sub_master hY)] at this

theorem pairTok_fstTok_sndTok (z : (prodTok D_0 D_1).Element) : pairTok (fstTok z) (
    sndTok z) = z := by
  apply NeighborhoodSystem.Element.ext
  intro W
  constructor
  * rintro <X, Y, <hX, hzX>, <hY, hzY>, rfl>
    exact (prodTok_mem_split hX hY).mpr <hzX, hzY>
  * intro hW
    obtain <X, Y, hX, hY, rfl> := z.sub hW
    obtain <h1, h2> := (prodTok_mem_split hX hY).mp hW
    exact <X, Y, <hX, h1>, <hY, h2>, rfl>

@[simp] theorem sndTok_pairTok (x : D_0.Element) (y : D_1.Element) : sndTok (pairTok x
     y) = y := by
  apply NeighborhoodSystem.Element.ext
  intro Y
  constructor
  * rintro <hY, hmem>; exact (mem_pairTok_prodTokNbhd.mp hmem).2
  * intro hY; exact <y.sub hY, mem_pairTok_prodTokNbhd.mpr <x.master_mem, hY>>

/-- The order-isomorphism `|D_0  x  D_1|??? ?o |D_0|  x  |D_1|` (Scott's Proposition
    3.2, token-level). -/
```

```
def prodTokEquiv : (prodTok D_0 D_1).Element ?o D_0.Element  x  D_1.Element where
  toFun z := (fstTok z, sndTok z)
  invFun p := pairTok p.1 p.2
  left_inv z := pairTok_fstTok_sndTok z
  right_inv p := by simp
  map_rel_iff' := by
    intro z z'
    constructor
    * rintro <h1, h2> W hW
      obtain <X, Y, hX, hY, rfl> := z.sub hW
      obtain <ha, hb> := (prodTok_mem_split hX hY).mp hW
      have hX' : (fstTok z').mem X := h1 X <hX, ha>
      have hY' : (sndTok z').mem Y := h2 Y <hY, hb>
      exact (prodTok_mem_split hX hY).mpr <hX'.2, hY'.2>
    * intro h
      exact <fun X <hX, hzX> => <hX, h _ hzX>, fun Y <hY, hzY> => <hY, h _ hzY>>

theorem toSum_fromSum (s : (sum D_0 D_1 h_0 h_1).Element) : toSum (fromSum s) = s :=
    by
  apply NeighborhoodSystem.Element.ext
  intro W
  constructor
  * rintro (rfl | <X, hX, rfl, hs> | <Y, hY, rfl, hs>)
    * exact s.master_mem
    * exact (fromSum_mem_embF hX).mp hs
    * exact (fromSum_mem_embT hY).mp hs
  * intro hW
    rcases s.sub hW with rfl | <X, hX, rfl> | <Y, hY, rfl>
    * exact Or.inl rfl
    * exact Or.inr (Or.inl <X, hX, rfl, (fromSum_mem_embF hX).mpr hW>)
    * exact Or.inr (Or.inr <Y, hY, rfl, (fromSum_mem_embT hY).mpr hW>)

/-- **Exercise 6.19 -- the concrete sum is correct up to isomorphism.** The order-
    isomorphism
`|D_0 + D_1|??? ?o |D_0 + D_1|` between Scott's `{0,1}*`-presentation and the abstract
    separated sum. -/
def sumTokEquiv : (sumTok D_0 D_1 h_0 h_1).Element ?o (sum D_0 D_1 h_0 h_1).Element
    where
  toFun := toSum
  invFun := fromSum
  left_inv := fromSum_toSum
  right_inv := toSum_fromSum
  map_rel_iff' := by
    intro x x'
    constructor
    * intro h X hX
      rcases x.sub hX with rfl | <A, hA, rfl> | <A, hA, rfl>
      * exact x'.master_mem
      * exact (toSum_mem_inj_0 hA).mp (h _ (Or.inr (Or.inl <A, hA, rfl, hX>)))
      * exact (toSum_mem_inj_1 hA).mp (h _ (Or.inr (Or.inr <A, hA, rfl, hX>)))
    * intro h W hW
      rcases hW with rfl | <X, hX, rfl, hzX> | <Y, hY, rfl, hzY>
      * exact Or.inl rfl
      * exact Or.inr (Or.inl <X, hX, rfl, h _ hzX>)
      * exact Or.inr (Or.inr <Y, hY, rfl, h _ hzY>)
```

```
/-- `prodTokNbhd D_0.master D_1.master` is exactly the sum master `{?}  U  0Delta_0  U
     1Delta_1`. -/
theorem prodTokNbhd_master_eq : prodTokNbhd D_0.master D_1.master = sumTokMaster D_0
    D_1 := rfl

/-- **Exercise 6.19 -- *"Are these correct up to isomorphism?"* (product).** Scott's
    uniform
`{0,1}*`-presentation `?_0  x  ?_1` is isomorphic, as a domain, to the abstract product
     of Definition 3.1.
Both compute the same domain `|D_0|  x  |D_1|` (via `prodTokEquiv` and Scott's `
    prodEquiv`). -/
theorem prodTok_iso_prod : prodTok D_0 D_1  ~= ? prod D_0 D_1 :=
  <prodTokEquiv.trans (prodEquiv D_0 D_1).symm>

/-- The concrete product is again `{}`-free (every neighbourhood contains `?`). -/
theorem prodTok_nonempty : forall  W, (prodTok D_0 D_1).mem W  ->  W.Nonempty := by
  rintro W <X, Y, _, _, rfl>; exact <[], mem_prodTokNbhd_nil>

/-- **Exercise 6.19 -- *"Are these correct up to isomorphism?"* (sum).** Scott's
    uniform
`{0,1}*`-presentation `?_0 + ?_1` is isomorphic, as a domain, to the abstract separated
     sum of
Exercise 3.18. -/
theorem sumTok_iso_sum : sumTok D_0 D_1 h_0 h_1  ~= ? sum D_0 D_1 h_0 h_1 := <
    sumTokEquiv>

end Exercise619
end

end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.Exercise619
section
set_option linter.unusedVariables false

theorem example_619 : And
  (forall  {D_0 D_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
    DomainTheory.NeighbourhoodSystems.ExampleB.Str},
    @DomainTheory.NeighbourhoodSystems.Isomorphic DomainTheory.NeighbourhoodSystems.
    ExampleB.Str
      (Sum DomainTheory.NeighbourhoodSystems.ExampleB.Str DomainTheory.
    NeighbourhoodSystems.ExampleB.Str)
      (DomainTheory.NeighbourhoodSystems.Exercise619.prodTok D_0 D_1)
      (@DomainTheory.NeighbourhoodSystems.prod DomainTheory.NeighbourhoodSystems.
    ExampleB.Str
        DomainTheory.NeighbourhoodSystems.ExampleB.Str D_0 D_1))
  (forall  {D_0 D_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
    DomainTheory.NeighbourhoodSystems.ExampleB.Str}
    {h_0 :
      forall  (X : Set DomainTheory.NeighbourhoodSystems.ExampleB.Str),
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem DomainTheory.
    NeighbourhoodSystems.ExampleB.Str D_0 X  ->
          @Set.Nonempty DomainTheory.NeighbourhoodSystems.ExampleB.Str X}
    {h_1 :
      forall  (Y : Set DomainTheory.NeighbourhoodSystems.ExampleB.Str),
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem DomainTheory.
    NeighbourhoodSystems.ExampleB.Str D_1 Y  ->
```

```
          @Set.Nonempty DomainTheory.NeighbourhoodSystems.ExampleB.Str Y},
    @DomainTheory.NeighbourhoodSystems.Isomorphic DomainTheory.NeighbourhoodSystems.
    ExampleB.Str
      (Option (Sum DomainTheory.NeighbourhoodSystems.ExampleB.Str DomainTheory.
    NeighbourhoodSystems.ExampleB.Str))
      (DomainTheory.NeighbourhoodSystems.Exercise619.sumTok D_0 D_1 h_0 h_1)
      (@DomainTheory.NeighbourhoodSystems.sum DomainTheory.NeighbourhoodSystems.
    ExampleB.Str
        DomainTheory.NeighbourhoodSystems.ExampleB.Str D_0 D_1 h_0 h_1)) :=
  @And.intro
  (forall  {D_0 D_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
    DomainTheory.NeighbourhoodSystems.ExampleB.Str},
    @DomainTheory.NeighbourhoodSystems.Isomorphic DomainTheory.NeighbourhoodSystems.
    ExampleB.Str
      (Sum DomainTheory.NeighbourhoodSystems.ExampleB.Str DomainTheory.
    NeighbourhoodSystems.ExampleB.Str)
      (DomainTheory.NeighbourhoodSystems.Exercise619.prodTok D_0 D_1)
      (@DomainTheory.NeighbourhoodSystems.prod DomainTheory.NeighbourhoodSystems.
    ExampleB.Str
        DomainTheory.NeighbourhoodSystems.ExampleB.Str D_0 D_1))
  (forall  {D_0 D_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
    DomainTheory.NeighbourhoodSystems.ExampleB.Str}
    {h_0 :
      forall  (X : Set DomainTheory.NeighbourhoodSystems.ExampleB.Str),
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem DomainTheory.
    NeighbourhoodSystems.ExampleB.Str D_0 X  ->
          @Set.Nonempty DomainTheory.NeighbourhoodSystems.ExampleB.Str X}
    {h_1 :
      forall  (Y : Set DomainTheory.NeighbourhoodSystems.ExampleB.Str),
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem DomainTheory.
    NeighbourhoodSystems.ExampleB.Str D_1 Y  ->
          @Set.Nonempty DomainTheory.NeighbourhoodSystems.ExampleB.Str Y},
    @DomainTheory.NeighbourhoodSystems.Isomorphic DomainTheory.NeighbourhoodSystems.
    ExampleB.Str
      (Option (Sum DomainTheory.NeighbourhoodSystems.ExampleB.Str DomainTheory.
    NeighbourhoodSystems.ExampleB.Str))
      (DomainTheory.NeighbourhoodSystems.Exercise619.sumTok D_0 D_1 h_0 h_1)
      (@DomainTheory.NeighbourhoodSystems.sum DomainTheory.NeighbourhoodSystems.
    ExampleB.Str
        DomainTheory.NeighbourhoodSystems.ExampleB.Str D_0 D_1 h_0 h_1))
  @DomainTheory.NeighbourhoodSystems.Exercise619.prodTok_iso_prod
  @DomainTheory.NeighbourhoodSystems.Exercise619.sumTok_iso_sum
end

end DomainTheory.NeighbourhoodSystems.Exercise619
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem ApproximableMap Example62 ExampleB

namespace Exercise619

variable {D_0 D_1 : NeighborhoodSystem Str}

variable {h_0 : forall  X, D_0.mem X  ->  X.Nonempty} {h_1 : forall  Y, D_1.mem Y  ->
     Y.Nonempty}
```

```
theorem sumTok_mem_master : (sumTok D_0 D_1 h_0 h_1).mem (sumTokMaster D_0 D_1) := Or.
    inl rfl

/-- The concrete sum is again `{}`-free (an object of Scott's category). -/
theorem sumTok_nonempty : forall  W, (sumTok D_0 D_1 h_0 h_1).mem W  ->  W.Nonempty :=
  fun _ h => sumTok_mem_nonempty h

end Exercise619
end

section
open NeighborhoodSystem

namespace Exercise629

universe u v

variable {? : Type u} {a : ?  ->  Type v}

/-- A **product neighbourhood** `iprodNbhd X = {<i, a> ? a  in  X i}` over `Sigma i, a
    i`. -/
def iprodNbhd (X : forall  i, Set (a i)) : Set (Sigma i, a i) := {p | p.2  in  X p.1}

@[simp] theorem mem_iprodNbhd {X : forall  i, Set (a i)} {i : ?} {a : a i} :
    (<i, a> : Sigma i, a i)  in  iprodNbhd X  <->  a  in  X i := Iff.rfl

end Exercise629
end

section
open NeighborhoodSystem

namespace Exercise629

universe u v

variable {? : Type u} {a : ?  ->  Type v}

variable {D : forall  i, NeighborhoodSystem (a i)}

/-- **Choice-free finite support**: a *list* enumerating all coordinates where `X` is
    non-master.
(Mathlib's `Set.Finite` is built on `Fintype` and pulls in `Classical.choice`; a `List`
     witness keeps
the construction constructive, which is exactly what is needed for the cylinder.) -/
def FinSupp (D : forall  i, NeighborhoodSystem (a i)) (X : forall  i, Set (a i)) :
    Prop :=
  exists  l : List ?, forall  i, i  notin  l  ->  X i = (D i).master

theorem FinSupp.inter {X X' : forall  i, Set (a i)}
    (hX : FinSupp D X) (hX' : FinSupp D X') : FinSupp D (fun i => X i  I  X' i) := by
  obtain <l, hl> := hX; obtain <l', hl'> := hX'
  -- Outside `l ++ l'` both factors are the master, so the intersection is too -- fully
     constructive,
  -- no case split on the undecidable proposition `X i = master`.
  refine <l ++ l', fun i hi => ?_>
```

```
  rw [List.mem_append, not_or] at hi
  simp only [hl i hi.1, hl' i hi.2, Set.inter_self]

end Exercise629
end

section
open NeighborhoodSystem

namespace Exercise629

universe u v

variable {? : Type u} {a : ?  ->  Type v}

/-- The tuple representation of a product neighbourhood is unique. -/
theorem iprodNbhd_injective {X X' : forall  i, Set (a i)} (h : iprodNbhd X = iprodNbhd
    X') : X = X' := by
  funext i; apply Set.ext; intro a
  simpa only [mem_iprodNbhd] using Set.ext_iff.mp h <i, a>

/-- Product neighbourhoods intersect coordinatewise (Scott's (2)). -/
theorem iprodNbhd_inter (X X' : forall  i, Set (a i)) :
    iprodNbhd X  I  iprodNbhd X' = iprodNbhd (fun i => X i  I  X' i) := by
  apply Set.ext; rintro <i, a>; simp [iprodNbhd, Set.mem_inter_iff]

/-- Inclusion of product neighbourhoods is coordinatewise (Scott's (1)). -/
theorem iprodNbhd_subset_iff {X X' : forall  i, Set (a i)} :
    iprodNbhd X  subseteq  iprodNbhd X'  <->  forall  i, X i  subseteq  X' i := by
  constructor
  * intro h i a ha; exact h (show (<i, a> : Sigma i, a i)  in  iprodNbhd X from ha)
  * rintro h <i, a> ha; exact h i ha

end Exercise629
end

section
open NeighborhoodSystem

namespace Exercise629

universe u v

variable {? : Type u} {a : ?  ->  Type v}

variable {D : forall  i, NeighborhoodSystem (a i)}

theorem FinSupp.master : FinSupp D (fun i => (D i).master) := <[], fun _ _ => rfl>

end Exercise629
end
end DomainTheory.NeighbourhoodSystems
end
```

3.7.22 Exercise 6.20 — tokens of a functor

For any system $\mathcal{D}$ let $\mathrm{tok}(\mathcal{D})$ be the underlying set of tokens, so that $\mathcal{D}$ is a system over $\mathrm{tok}(\mathcal{D})$. For the category of Exercise 6.19 show that the function

$$\lambda\Gamma.\,\mathrm{tok}(T(\{\Gamma\}))$$

is continuous on the domain $\{\Gamma \subseteq \{0,1\}^* \mid \Lambda \in \Gamma\}$, where T is any of the functors generated in 6.19. Conclude that there must exist a set

$$\Gamma = \mathrm{tok}(T(\{\Gamma\})),$$

so that $\{\Gamma\} \triangleleft T(\{\Gamma\})$, and so 6.14 applies.

Exercise_6_20.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.Definition_6_13

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_12

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.Exercise619
open DomainTheory.NeighbourhoodSystems.Example62 DomainTheory.NeighbourhoodSystems.
    ExampleB
open DomainTheory.NeighbourhoodSystems.Exercise510
section
namespace Exercise619
variable {A_0 A_1 B_0 B_1 C_0 C_1 : ScottSys}

/- concordance: source L4099 EXERCISE 6.20 tokens of a functor -/
/-- **The conclusion of Exercise 6.20 (token level).** For any construct `T` whose
    constants contain
`?`, there is a set `Gamma` with `?  in  Gamma` and `Gamma = tok(T({Gamma}))`. -/
theorem exists_tok_fixedPoint (T : FExpr) (hT : T.RootedConst) :
    exists  Gamma : Set Str, ([] : Str)  in  Gamma  /\  mFun T Gamma = Gamma :=
  <? n, mIter T n, Set.mem_iUnion.mpr <0, nil_mem_mIter T hT 0>, mFun_iter_fixed T hT>

/-- **The conclusion of Exercise 6.20 (object level): `{Gamma} ? T({Gamma})`, so
    Theorem 6.14 applies.**
From the fixed point `Gamma = tok(T({Gamma}))`, the one-neighbourhood system `{Gamma}`
    is a subsystem of
`T({Gamma})`: they share the master `Gamma`, and `Gamma` is a (the master)
    neighbourhood of `T({Gamma})`. -/
theorem exists_singleton_subsystem (T : FExpr) (hT : T.RootedConst) :
    exists  (Gamma : Set Str) (h : Gamma.Nonempty),
      (singletonSys Gamma h).sys ? (T.obj (singletonSys Gamma h)).sys := by
  obtain <Gamma, hnil, hfix> := exists_tok_fixedPoint T hT
  have hne : Gamma.Nonempty := <[], hnil>
  -- `tok(T({Gamma})) = Gamma` (note `tok` is definitionally `.sys.master`).
  have hmaster : (T.obj (singletonSys Gamma hne)).sys.master = Gamma :=
```

```
    (mFun_eq_master T hne).symm.trans hfix
  refine <Gamma, hne, ?_, ?_, ?_>
  * -- master_eq: `Gamma = tok(T({Gamma}))`
    exact hmaster.symm
  * -- sub: the only neighbourhood `Gamma` of `{Gamma}` is the master of `T({Gamma})`
    intro X hX
    have heq : X = (T.obj (singletonSys Gamma hne)).sys.master := (hX : X = Gamma).
    trans hmaster.symm
    rw [heq]
    exact (T.obj (singletonSys Gamma hne)).sys.master_mem
  * -- inter_closed: trivial, both neighbourhoods are `Gamma`
    intro X Y hX hY _
    show X  I  Y = Gamma
    rw [show X = Gamma from hX, show Y = Gamma from hY, Set.inter_self]

/-- **`(g_0  comp  f_0)  x  (g_1  comp  f_1) = (g_0  x  g_1)  comp  (f_0  x  f_1)`.**
    -/
theorem prodMapTok_comp (f_0 : ApproximableMap A_0.sys B_0.sys) (f_1 : ApproximableMap
     A_1.sys B_1.sys)
    (g_0 : ApproximableMap B_0.sys C_0.sys) (g_1 : ApproximableMap B_1.sys C_1.sys) :
    prodMapTok (g_0.comp f_0) (g_1.comp f_1) = (prodMapTok g_0 g_1).comp (prodMapTok
    f_0 f_1) := by
  apply ApproximableMap.ext
  intro W W''
  constructor
  * rintro <X, Y, X'', Y'', <X', hf0, hg0>, <Y', hf1, hg1>, rfl, rfl>
    exact <prodTokNbhd X' Y', <X, Y, X', Y', hf0, hf1, rfl, rfl>,
      <X', Y', X'', Y'', hg0, hg1, rfl, rfl>>
  * rintro <W', <X, Y, X', Y', hf0, hf1, rfl, rfl>, <X_2, Y_2, X'', Y'', hg0, hg1,
    hWeq, rfl>>
    obtain <rfl, rfl> := prodTokNbhd_injective hWeq
    exact <X, Y, X'', Y'', <X', hf0, hg0>, <Y', hf1, hg1>, rfl, rfl>

/-- **`(I_{?_0}  x  I_{?_1}) = I_{?_0 x ?_1}`.** -/
theorem prodMapTok_id :
    prodMapTok (idMap A_0.sys) (idMap A_1.sys) = idMap (A_0.prod A_1).sys := by
  apply ApproximableMap.ext
  intro W W'
  constructor
  * rintro <X, Y, X', Y', <hX, hX', hsX>, <hY, hY', hsY>, rfl, rfl>
    exact <prodTok_mem_prodTokNbhd hX hY, prodTok_mem_prodTokNbhd hX' hY',
      prodTokNbhd_subset_iff.mpr <hsX, hsY>>
  * rintro <<X, Y, hX, hY, rfl>, <X', Y', hX', hY', rfl>, hsub>
    obtain <hsX, hsY> := prodTokNbhd_subset_iff.mp hsub
    exact <X, Y, X', Y', <hX, hX', hsX>, <hY, hY', hsY>, rfl, rfl>

/-- **`prodMapTok` is strict** exactly when both components are strict. -/
theorem prodMapTok_isStrict {f_0 : ApproximableMap A_0.sys B_0.sys}
    {f_1 : ApproximableMap A_1.sys B_1.sys} (hf_0 : IsStrict f_0) (hf_1 : IsStrict f_1
    ) :
    IsStrict (prodMapTok f_0 f_1) := by
  rintro Y <X, Y0, X', Y', h0, h1, hWeq, rfl>
  obtain <rfl, rfl> := prodTokNbhd_injective hWeq
  rw [hf_0 h0, hf_1 h1]
  rfl
```

```
/-- `prodMapTok` is monotone in both arguments. -/
theorem prodMapTok_mono {f_0 f_0' : ApproximableMap A_0.sys B_0.sys}
    {f_1 f_1' : ApproximableMap A_1.sys B_1.sys} (h0 : f_0  <=  f_0') (h1 : f_1  <=
    f_1') :
    prodMapTok f_0 f_1  <=  prodMapTok f_0' f_1' := by
  rw [ApproximableMap.le_iff]
  rintro W W' <X, Y, X', Y', hr0, hr1, rfl, rfl>
  exact <X, Y, X', Y', h0 X X' hr0, h1 Y Y' hr1, rfl, rfl>

/-- The product carries the subsystem relation componentwise. -/
theorem prodTok_subsystem (h0 : A_0.sys ? B_0.sys) (h1 : A_1.sys ? B_1.sys) :
    (A_0.prod A_1).sys ? (B_0.prod B_1).sys := by
  have heqm : prodTokNbhd A_0.sys.master A_1.sys.master
      = prodTokNbhd B_0.sys.master B_1.sys.master := by rw [h0.master_eq, h1.master_eq
    ]
  refine <heqm, ?_, ?_>
  * rintro W <X, Y, hX, hY, rfl>
    exact <X, Y, h0.sub hX, h1.sub hY, rfl>
  * rintro W W' <X, Y, hX, hY, rfl> <X', Y', hX', hY', rfl> hInt
    rw [prodTokNbhd_inter] at hInt  |-
    obtain <X'', Y'', hX'', hY'', heq> := hInt
    obtain <rfl, rfl> := prodTokNbhd_injective heq
    exact <X  I  X', Y  I  Y', h0.inter_closed hX hX' hX'', h1.inter_closed hY hY' hY
    '', rfl>

end Exercise619
end

end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.Exercise619
section
set_option linter.unusedVariables false

theorem example_619_part_b : forall  {A_0 A_1 B_0 B_1 : DomainTheory.
    NeighbourhoodSystems.Exercise619.ScottSys},
  @DomainTheory.NeighbourhoodSystems.Subsystem DomainTheory.NeighbourhoodSystems.
    ExampleB.Str A_0.sys B_0.sys  ->
    @DomainTheory.NeighbourhoodSystems.Subsystem DomainTheory.NeighbourhoodSystems.
    ExampleB.Str A_1.sys B_1.sys  ->
      @DomainTheory.NeighbourhoodSystems.Subsystem DomainTheory.NeighbourhoodSystems.
    ExampleB.Str (A_0.prod A_1).sys
        (B_0.prod B_1).sys :=
  @DomainTheory.NeighbourhoodSystems.Exercise619.prodTok_subsystem
end

end DomainTheory.NeighbourhoodSystems.Exercise619
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.Exercise619
open DomainTheory.NeighbourhoodSystems.Example62 DomainTheory.NeighbourhoodSystems.
    ExampleB
open DomainTheory.NeighbourhoodSystems.Exercise510

section
namespace Exercise619

variable {A_0 A_1 B_0 B_1 C_0 C_1 : ScottSys}
```

```
/-- **`(g_0  comp  f_0) + (g_1  comp  f_1) = (g_0 + g_1)  comp  (f_0 + f_1)`.** -/
theorem sumMapTok_comp (f_0 : ApproximableMap A_0.sys B_0.sys) (f_1 : ApproximableMap
    A_1.sys B_1.sys)
    (g_0 : ApproximableMap B_0.sys C_0.sys) (g_1 : ApproximableMap B_1.sys C_1.sys) :
    sumMapTok (g_0.comp f_0) (g_1.comp f_1) = (sumMapTok g_0 g_1).comp (sumMapTok f_0
    f_1) := by
  apply ApproximableMap.ext
  intro W W''
  constructor
  * rintro (<hW, rfl> | <X, X'', <X', hf, hg>, rfl, rfl> | <Y, Y'', <Y', hf, hg>, rfl,
     rfl>)
    * exact <sumTokMaster B_0.sys B_1.sys, Or.inl <hW, rfl>,
        Or.inl <(B_0.sum B_1).sys.master_mem, rfl>>
    * exact <embBit false X', Or.inr (Or.inl <X, X', hf, rfl, rfl>),
        Or.inr (Or.inl <X', X'', hg, rfl, rfl>)>
    * exact <embBit true Y', Or.inr (Or.inr <Y, Y', hf, rfl, rfl>),
        Or.inr (Or.inr <Y', Y'', hg, rfl, rfl>)>
  * rintro <W', hWW', hW'W''>
    rcases hWW' with <hW, rfl> | <X, X', hf, rfl, rfl> | <Y, Y', hf, rfl, rfl>
    * rcases hW'W'' with <-, rfl> | <X, X', -, heq, -> | <Y, Y', -, heq, ->
      * exact Or.inl <hW, rfl>
      * exact absurd (heq  |>  nil_mem_sumTokMaster) nil_not_mem_embBit
      * exact absurd (heq  |>  nil_mem_sumTokMaster) nil_not_mem_embBit
    * rcases hW'W'' with <-, rfl> | <X_2, X'', hg, heq, rfl> | <Y_2, Y'', hg, heq, ->
      * exact Or.inl <Or.inr (Or.inl <X, f_0.rel_dom hf, rfl>), rfl>
      * obtain rfl := embBit_injective heq
        exact Or.inr (Or.inl <X, X'', <X', hf, hg>, rfl, rfl>)
      * exact absurd heq (embBit_ne (by decide) (B_0.ne X' (f_0.rel_cod hf)))
    * rcases hW'W'' with <-, rfl> | <X_2, X'', hg, heq, -> | <Y_2, Y'', hg, heq, rfl>
      * exact Or.inl <Or.inr (Or.inr <Y, f_1.rel_dom hf, rfl>), rfl>
      * exact absurd heq (embBit_ne (by decide) (B_1.ne Y' (f_1.rel_cod hf)))
      * obtain rfl := embBit_injective heq
        exact Or.inr (Or.inr <Y, Y'', <Y', hf, hg>, rfl, rfl>)

/-- **Functor law 2 -- `T(g  comp  f) = T(g)  comp  T(f)`.** Every construct `T`
    preserves composition; together
with `map_id` this shows *these are all functors*. -/
theorem FExpr.map_comp : (T : FExpr)  ->  {X Y Z : ScottSys}  ->  (f : ApproximableMap
     X.sys Y.sys)  ->
    (g : ApproximableMap Y.sys Z.sys)  ->  T.map (g.comp f) = (T.map g).comp (T.map f)
  | .const D, _, _, _, _, _ => (idMap_comp (idMap D.sys)).symm
  | .var, _, _, _, _, _ => rfl
  | .sum T_0 T_1, _, _, _, f, g => by
      show sumMapTok (T_0.map (g.comp f)) (T_1.map (g.comp f))
          = (sumMapTok (T_0.map g) (T_1.map g)).comp (sumMapTok (T_0.map f) (T_1.map f
    ))
      rw [T_0.map_comp f g, T_1.map_comp f g, sumMapTok_comp]
  | .prod T_0 T_1, _, _, _, f, g => by
      show prodMapTok (T_0.map (g.comp f)) (T_1.map (g.comp f))
          = (prodMapTok (T_0.map g) (T_1.map g)).comp (prodMapTok (T_0.map f) (T_1.map
     f))
      rw [T_0.map_comp f g, T_1.map_comp f g, prodMapTok_comp]

/-- **`(I_{?_0} + I_{?_1}) = I_{?_0+?_1}`.** -/
theorem sumMapTok_id :
```

```
    sumMapTok (idMap A_0.sys) (idMap A_1.sys) = idMap (A_0.sum A_1).sys := by
  apply ApproximableMap.ext
  intro W W'
  constructor
  * rintro (<hW, rfl> | <X, X', <hX, hX', hsub>, rfl, rfl> | <Y, Y', <hY, hY', hsub>,
    rfl, rfl>)
    * exact <hW, (A_0.sum A_1).sys.master_mem, (A_0.sum A_1).sys.sub_master hW>
    * exact <Or.inr (Or.inl <X, hX, rfl>), Or.inr (Or.inl <X', hX', rfl>),
    embBit_subset.mpr hsub>
    * exact <Or.inr (Or.inr <Y, hY, rfl>), Or.inr (Or.inr <Y', hY', rfl>),
    embBit_subset.mpr hsub>
  * rintro <hW, hW', hsub>
    rcases hW' with rfl | <X', hX', rfl> | <Y', hY', rfl>
    * exact Or.inl <hW, rfl>
    * rcases hW with rfl | <X, hX, rfl> | <Y, hY, rfl>
      * exact absurd (hsub nil_mem_sumTokMaster) nil_not_mem_embBit
      * exact Or.inr (Or.inl <X, X', <hX, hX', embBit_subset.mp hsub>, rfl, rfl>)
      * exact absurd hsub (fun h => embBit_not_subset_cross (by decide) (A_1.ne Y hY)
    h)
    * rcases hW with rfl | <X, hX, rfl> | <Y, hY, rfl>
      * exact absurd (hsub nil_mem_sumTokMaster) nil_not_mem_embBit
      * exact absurd hsub (fun h => embBit_not_subset_cross (by decide) (A_0.ne X hX)
    h)
      * exact Or.inr (Or.inr <Y, Y', <hY, hY', embBit_subset.mp hsub>, rfl, rfl>)

/-- **Functor law 1 -- `T(I_X) = I_{T(X)}`.** Every construct `T` preserves identities.
    -/
theorem FExpr.map_id : (T : FExpr)  ->  (X : ScottSys)  ->  T.map (idMap X.sys) =
    idMap (T.obj X).sys
  | .const D, _ => rfl
  | .var, _ => rfl
  | .sum T_0 T_1, X => by
      show sumMapTok (T_0.map (idMap X.sys)) (T_1.map (idMap X.sys))
          = idMap ((T_0.obj X).sum (T_1.obj X)).sys
      rw [T_0.map_id X, T_1.map_id X, sumMapTok_id]
  | .prod T_0 T_1, X => by
      show prodMapTok (T_0.map (idMap X.sys)) (T_1.map (idMap X.sys))
          = idMap ((T_0.obj X).prod (T_1.obj X)).sys
      rw [T_0.map_id X, T_1.map_id X, prodMapTok_id]

/-- **`sumMapTok` is strict** for *any* component maps: the master input `?` (the only
    neighbourhood
containing the empty string) relates only to the output master. -/
theorem sumMapTok_isStrict (f_0 : ApproximableMap A_0.sys B_0.sys)
    (f_1 : ApproximableMap A_1.sys B_1.sys) : IsStrict (sumMapTok f_0 f_1) := by
  rintro Y (<-, rfl> | <X, X', -, heq, -> | <Y0, Y', -, heq, ->)
  * rfl
  * have heq' : sumTokMaster A_0.sys A_1.sys = embBit false X := heq
    exact absurd (heq'  |>  nil_mem_sumTokMaster) nil_not_mem_embBit
  * have heq' : sumTokMaster A_0.sys A_1.sys = embBit true Y0 := heq
    exact absurd (heq'  |>  nil_mem_sumTokMaster) nil_not_mem_embBit

/-- **Every `T` preserves strictness** (so it restricts to Scott's category of strict
    maps): `T(f)`
is strict whenever `f` is (and constants/sums are strict unconditionally). -/
theorem FExpr.map_isStrict : (T : FExpr)  ->  {X Y : ScottSys}  ->  (f :
```

```
    ApproximableMap X.sys Y.sys)  ->
    IsStrict f  ->  IsStrict (T.map f)
  | .const _, _, _, _, _ => isStrict_idMap
  | .var, _, _, _, hf => hf
  | .sum T_0 T_1, _, _, f, _ => sumMapTok_isStrict (T_0.map f) (T_1.map f)
  | .prod T_0 T_1, _, _, f, hf =>
      prodMapTok_isStrict (T_0.map_isStrict f hf) (T_1.map_isStrict f hf)

/-- `sumMapTok` is monotone in both arguments. -/
theorem sumMapTok_mono {f_0 f_0' : ApproximableMap A_0.sys B_0.sys}
    {f_1 f_1' : ApproximableMap A_1.sys B_1.sys} (h0 : f_0  <=  f_0') (h1 : f_1  <=
    f_1') :
    sumMapTok f_0 f_1  <=  sumMapTok f_0' f_1' := by
  rw [ApproximableMap.le_iff]
  rintro W W' (<hW, rfl> | <X, X', hrel, rfl, rfl> | <Y, Y', hrel, rfl, rfl>)
  * exact Or.inl <hW, rfl>
  * exact Or.inr (Or.inl <X, X', h0 X X' hrel, rfl, rfl>)
  * exact Or.inr (Or.inr <Y, Y', h1 Y Y' hrel, rfl, rfl>)

/-- **`fun f. T(f)` is monotone.** A sharper map `f  <=  f'` is sent to a sharper image
     `T(f)  <=  T(f')` --
the monotonicity half of *continuous on maps*. -/
theorem FExpr.map_mono : (T : FExpr)  ->  {X Y : ScottSys}  ->  {f f' :
    ApproximableMap X.sys Y.sys}  ->
    f  <=  f'  ->  T.map f  <=  T.map f'
  | .const _, _, _, _, _, _ => le_rfl
  | .var, _, _, _, _, h => h
  | .sum T_0 T_1, _, _, _, _, h => sumMapTok_mono (T_0.map_mono h) (T_1.map_mono h)
  | .prod T_0 T_1, _, _, _, _, h => prodMapTok_mono (T_0.map_mono h) (T_1.map_mono h)

/-- Forward direction of continuity on maps: a related pair of `T(f)` is already
    related by some
`T(g_i)`. -/
theorem FExpr.map_continuous_mp : (T : FExpr)  ->  {I : Type}  ->  {X Y : ScottSys}
    ->
    {g : I  ->  ApproximableMap X.sys Y.sys}  ->  {f : ApproximableMap X.sys Y.sys}
    ->
    [Nonempty I]  ->  (forall  i j, exists  k, g i  <=  g k  /\  g j  <=  g k)  ->
    (forall  A B, f.rel A B  <->  exists  i, (g i).rel A B)  ->
    {A B : Set Str}  ->  (T.map f).rel A B  ->  exists  i, (T.map (g i)).rel A B
  | .const _, _, _, _, _, _, _, _, _, _, _, hrel => by
      obtain <i> := ?Nonempty _?; exact <i, hrel>
  | .var, _, _, _, _, _, _, _, hf, A, B, hrel => (hf A B).mp hrel
  | .sum T_0 T_1, _, _, _, _, _, _, hdir, hf, _, _, hrel => by
      rcases hrel with <hA, rfl> | <P, P', hr, rfl, rfl> | <Q, Q', hr, rfl, rfl>
      * obtain <i> := ?Nonempty _?; exact <i, Or.inl <hA, rfl>>
      * obtain <i, hi> := T_0.map_continuous_mp hdir hf hr
        exact <i, Or.inr (Or.inl <P, P', hi, rfl, rfl>)>
      * obtain <i, hi> := T_1.map_continuous_mp hdir hf hr
        exact <i, Or.inr (Or.inr <Q, Q', hi, rfl, rfl>)>
  | .prod T_0 T_1, _, _, _, _, _, _, hdir, hf, _, _, hrel => by
      obtain <P, Q, P', Q', hr0, hr1, rfl, rfl> := hrel
      obtain <i, hi> := T_0.map_continuous_mp hdir hf hr0
      obtain <j, hj> := T_1.map_continuous_mp hdir hf hr1
      obtain <k, hik, hjk> := hdir i j
      exact <k, P, Q, P', Q', (T_0.map_mono hik) P P' hi, (T_1.map_mono hjk) Q Q' hj,
```

```
    rfl, rfl>

/-- **`fun f. T(f)` is continuous on maps.** For a non-empty directed family `g_i` with
     pointwise union
`f`, the relation of `T(f)` is the union of those of the `T(g_i)` -- `T(f)` is
    approximable. -/
theorem FExpr.map_continuous (T : FExpr) {I : Type} [Nonempty I] {X Y : ScottSys}
    (g : I  ->  ApproximableMap X.sys Y.sys) (f : ApproximableMap X.sys Y.sys)
    (hdir : forall  i j, exists  k, g i  <=  g k  /\  g j  <=  g k)
    (hf : forall  A B, f.rel A B  <->  exists  i, (g i).rel A B) (A B : Set Str) :
    (T.map f).rel A B  <->  exists  i, (T.map (g i)).rel A B := by
  refine <T.map_continuous_mp hdir hf, ?_>
  rintro <i, hi>
  have hgif : g i  <=  f := by
    rw [ApproximableMap.le_iff]; intro A' B' h; exact (hf A' B').mpr <i, h>
  exact (T.map_mono hgif) A B hi

/-- The sum carries the subsystem relation componentwise: `?_0 ? ?_0` and `?_1 ? ?_1`
    give
`?_0+?_1 ? ?_0+?_1`. -/
theorem sumTok_subsystem (h0 : A_0.sys ? B_0.sys) (h1 : A_1.sys ? B_1.sys) :
    (A_0.sum A_1).sys ? (B_0.sum B_1).sys := by
  have heqm : sumTokMaster A_0.sys A_1.sys = sumTokMaster B_0.sys B_1.sys := by
    unfold sumTokMaster; rw [h0.master_eq, h1.master_eq]
  refine <heqm, ?_, ?_>
  * rintro W (rfl | <X, hX, rfl> | <Y, hY, rfl>)
    * exact Or.inl heqm
    * exact Or.inr (Or.inl <X, h0.sub hX, rfl>)
    * exact Or.inr (Or.inr <Y, h1.sub hY, rfl>)
  * rintro W W' (rfl | <X, hX, rfl> | <Y, hY, rfl>) (rfl | <X', hX', rfl> | <Y', hY',
    rfl>) hInt
    * rw [Set.inter_self]; exact Or.inl rfl
    * rw [sumTokMaster_inter_embF hX']; exact Or.inr (Or.inl <X', hX', rfl>)
    * rw [sumTokMaster_inter_embT hY']; exact Or.inr (Or.inr <Y', hY', rfl>)
    * rw [Set.inter_comm, sumTokMaster_inter_embF hX]; exact Or.inr (Or.inl <X, hX,
    rfl>)
    * rw [embBit_inter] at hInt  |-
      exact Or.inr (Or.inl <X  I  X',
        h0.inter_closed hX hX' (sumTok_mem_embF_inv (h_0 := B_0.ne) (h_1 := B_1.ne)
    hInt), rfl>)
    * rw [embBit_inter_ne (show (false : Bool)  !=  true by decide)] at hInt
      exact absurd ((B_0.sum B_1).ne _ hInt) Set.not_nonempty_empty
    * rw [Set.inter_comm, sumTokMaster_inter_embT hY]; exact Or.inr (Or.inr <Y, hY,
    rfl>)
    * rw [embBit_inter_ne (show (true : Bool)  !=  false by decide)] at hInt
      exact absurd ((B_0.sum B_1).ne _ hInt) Set.not_nonempty_empty
    * rw [embBit_inter] at hInt  |-
      exact Or.inr (Or.inr <Y  I  Y',
        h1.inter_closed hY hY' (sumTok_mem_embT_inv (h_0 := B_0.ne) (h_1 := B_1.ne)
    hInt), rfl>)

/-- **`fun X. T(X)` is monotone on domains.** Whenever `X ? Y` we have `T(X) ? T(Y)`.
    -/
theorem FExpr.obj_subsystem : (T : FExpr)  ->  {X Y : ScottSys}  ->  X.sys ? Y.sys  ->
    (T.obj X).sys ? (T.obj Y).sys
  | .const D, _, _, _ => Subsystem.refl D.sys
```

```
  | .var, _, _, h => h
  | .sum T_0 T_1, _, _, h => sumTok_subsystem (T_0.obj_subsystem h) (T_1.obj_subsystem
    h)
  | .prod T_0 T_1, _, _, h => prodTok_subsystem (T_0.obj_subsystem h) (T_1.
    obj_subsystem h)

/-- Forward direction of continuity on domains: a neighbourhood of `T(U)` is a
    neighbourhood of some
`T(D)` with `D  in  ?`. -/
theorem FExpr.obj_continuous_mp : (T : FExpr)  ->  {? : Set ScottSys}  ->  {U :
    ScottSys}  ->
    DirectedOn (fun a b => a.sys ? b.sys) ?  ->  ?.Nonempty  ->
    (forall  D  in  ?, D.sys ? U.sys)  ->  (forall  X, U.sys.mem X  <->  exists  D  in
      ?, D.sys.mem X)  ->
    {W : Set Str}  ->  (T.obj U).sys.mem W  ->  exists  D  in  ?, (T.obj D).sys.mem W
  | .const _, _, _, _, hne, _, _, _, hmem => by
      obtain <D, hD> := hne; exact <D, hD, hmem>
  | .var, _, _, _, _, _, hU, W, hmem => (hU W).mp hmem
  | .sum T_0 T_1, _, _, hdir, hne, hsub, hU, _, hmem => by
      rcases hmem with rfl | <X, hX, rfl> | <Y, hY, rfl>
      * obtain <D, hD> := hne
        exact <D, hD, Or.inl ((FExpr.sum T_0 T_1).obj_subsystem (hsub D hD)).master_eq
    .symm>
      * obtain <D, hD, hXD> := T_0.obj_continuous_mp hdir hne hsub hU hX
        exact <D, hD, Or.inr (Or.inl <X, hXD, rfl>)>
      * obtain <D, hD, hYD> := T_1.obj_continuous_mp hdir hne hsub hU hY
        exact <D, hD, Or.inr (Or.inr <Y, hYD, rfl>)>
  | .prod T_0 T_1, _, _, hdir, hne, hsub, hU, _, hmem => by
      obtain <X, Y, hX, hY, rfl> := hmem
      obtain <D_1, hD_1, hXD> := T_0.obj_continuous_mp hdir hne hsub hU hX
      obtain <D_2, hD_2, hYD> := T_1.obj_continuous_mp hdir hne hsub hU hY
      obtain <D_3, hD_3, hr1, hr2> := hdir D_1 hD_1 D_2 hD_2
      exact <D_3, hD_3, X, Y, (T_0.obj_subsystem hr1).sub hXD, (T_1.obj_subsystem hr2)
    .sub hYD, rfl>

/-- **`fun D. T(D)` is continuous on domains.** For a non-empty `?`-directed family `?`
     of subsystems of
`U` whose union is `U`, the neighbourhood family of `T(U)` is the union of those of the
     `T(D)`. -/
theorem FExpr.obj_continuous (T : FExpr) {? : Set ScottSys} {U : ScottSys}
    (hdir : DirectedOn (fun a b => a.sys ? b.sys) ?) (hne : ?.Nonempty)
    (hsub : forall  D  in  ?, D.sys ? U.sys) (hU : forall  X, U.sys.mem X  <->  exists
      D  in  ?, D.sys.mem X)
    (W : Set Str) : (T.obj U).sys.mem W  <->  exists  D  in  ?, (T.obj D).sys.mem W :=
     by
  refine <T.obj_continuous_mp hdir hne hsub hU, ?_>
  rintro <D, hD, hmem>
  exact (T.obj_subsystem (hsub D hD)).sub hmem

end Exercise619
end

section
namespace Exercise619

variable {D_0 D_1 : NeighborhoodSystem Str}
```

```
variable {h_0 : forall  X, D_0.mem X  ->  X.Nonempty} {h_1 : forall  Y, D_1.mem Y  ->
     Y.Nonempty}
variable {A_0 A_1 B_0 B_1 : ScottSys}
variable {C_0 C_1 : ScottSys}

/-- **The extended functor-expression algebra** (constants, identity, `+`, ` x `, `?`,
    `?`). -/
inductive GExpr where
  | const : ScottSys  ->  GExpr
  | var : GExpr
  | sum : GExpr  ->  GExpr  ->  GExpr
  | prod : GExpr  ->  GExpr  ->  GExpr
  | oplus : GExpr  ->  GExpr  ->  GExpr
  | otimes : GExpr  ->  GExpr  ->  GExpr

end Exercise619
end

section
namespace Exercise619

variable {D_0 D_1 : NeighborhoodSystem Str}

/-- If `X  subseteq  Delta` and `X  !=  Delta`, then any intersection `X  I  X'` is
    still ` !=  Delta` (it is ` subseteq  X ? Delta`). -/
theorem inter_ne_of_ne_left {X X' Delta : Set Str} (hX : X  subseteq  Delta) (hne : X
     !=  Delta) : X  I  X'  !=  Delta := by
  intro h
  exact hne (Set.Subset.antisymm hX (by rw [ <-  h]; exact Set.inter_subset_left))

end Exercise619
end

section
namespace Exercise619


variable {D_0 D_1 : NeighborhoodSystem Str}

variable {h_0 : forall  X, D_0.mem X  ->  X.Nonempty} {h_1 : forall  Y, D_1.mem Y  ->
     Y.Nonempty}

variable {A_0 A_1 B_0 B_1 : ScottSys}

variable {C_0 C_1 : ScottSys}

/-- **Right-nested n-ary fold** of a binary construct-operation `op` over a non-empty
    list `a, l...`.
With `op = .sum`/`.prod`/`.oplus`/`.otimes` this is the n-ary `+`/` x `/`?`/`?`. -/
def GExpr.naryOp (op : GExpr  ->  GExpr  ->  GExpr) (a : GExpr) : List GExpr  ->
    GExpr
  | [] => a
  | b :: l => op a (GExpr.naryOp op b l)

/-- n-ary separated product `T_0  x  T_1  x  ?  x  T_n`. -/
def GExpr.naryProd : GExpr  ->  List GExpr  ->  GExpr := GExpr.naryOp GExpr.prod
```

```
/-- n-ary separated sum `T_0 + T_1 + ? + T_n`. -/
def GExpr.narySum : GExpr  ->  List GExpr  ->  GExpr := GExpr.naryOp GExpr.sum

end Exercise619
end

end DomainTheory.NeighbourhoodSystems
end
```

3.7.23 Exercise 6.21 — coalesced sum and smash

Do the same as 6.19 and 6.20 when the functors are also allowed to be generated by the operations:

$$D_0 \oplus D_1 = \{\{\Lambda\} \cup 0\Delta_0 \cup 1\Delta_1\} \cup \{0X \mid X \in D_0 \setminus \{\Delta_0\}\} \cup \{1Y \mid Y \in D_1 \setminus \{\Delta_1\}\},$$

$$D_0 \otimes D_1 = \{\{\Lambda\} \cup 0\Delta_0 \cup 1\Delta_1\} \cup \{\{\Lambda\} \cup 0X \cup 1Y \mid X \in D_0 \setminus \{\Delta_0\} \text{ and } Y \in D_1 \setminus \{\Delta_1\}\}.$$

Generalize all of $+$, $\times$, $\oplus$, $\otimes$ to combinations of several terms, not just the binary sums and products.

Exercise_6_21.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.Exercise_6_20

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_12

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.Exercise619
open DomainTheory.NeighbourhoodSystems.Example62 DomainTheory.NeighbourhoodSystems.
    ExampleB
open DomainTheory.NeighbourhoodSystems.Exercise510
section
namespace Exercise619
variable {D_0 D_1 : NeighborhoodSystem Str}
/- concordance: source L4115 EXERCISE 6.21 coalesced sum and smash -/
/-- **Exercise 6.21 -- the coalesced sum system `?_0 ? ?_1` over `{0,1}*`.** As `sumTok
    `, but the
improper copies `0Delta_0`, `1Delta_1` are removed (`X  !=  Delta_0`, `Y  !=  Delta_1`)
    , so the two bottoms are identified. -/
def oplusTok (D_0 D_1 : NeighborhoodSystem Str)
    (h_0 : forall  X, D_0.mem X  ->  X.Nonempty) (h_1 : forall  Y, D_1.mem Y  ->  Y.
    Nonempty) :
    NeighborhoodSystem Str where
  mem W := W = sumTokMaster D_0 D_1  \/  (exists  X, D_0.mem X  /\  X  !=  D_0.master
     /\  W = embBit false X)  \/
    (exists  Y, D_1.mem Y  /\  Y  !=  D_1.master  /\  W = embBit true Y)
  master := sumTokMaster D_0 D_1
  master_nonempty := <[], nil_mem_sumTokMaster>
```

```
  master_mem := Or.inl rfl
  sub_master := by
    rintro W (rfl | <X, hX, -, rfl> | <Y, hY, -, rfl>)
    * exact subset_rfl
    * exact embF_subset_sumTokMaster hX
    * exact embT_subset_sumTokMaster hY
  inter_mem := by
    have hne : forall  W, (W = sumTokMaster D_0 D_1  \/  (exists  X, D_0.mem X  /\  X
     !=  D_0.master  /\  W = embBit false X)  \/
         (exists  Y, D_1.mem Y  /\  Y  !=  D_1.master  /\  W = embBit true Y))  ->  (W
    : Set Str).Nonempty := by
      rintro W (rfl | <X, hX, -, rfl> | <Y, hY, -, rfl>)
      * exact <[], nil_mem_sumTokMaster>
      * exact embBit_nonempty (h_0 X hX)
      * exact embBit_nonempty (h_1 Y hY)
    rintro W W' Z hW hW' hZ hZsub
    rcases hW with rfl | <X, hX, hXne, rfl> | <Y, hY, hYne, rfl>
    * rcases hW' with rfl | <X', hX', hX'ne, rfl> | <Y', hY', hY'ne, rfl>
      * rw [Set.inter_self]; exact Or.inl rfl
      * rw [sumTokMaster_inter_embF hX']; exact Or.inr (Or.inl <X', hX', hX'ne, rfl>)
      * rw [sumTokMaster_inter_embT hY']; exact Or.inr (Or.inr <Y', hY', hY'ne, rfl>)
    * rcases hW' with rfl | <X', hX', hX'ne, rfl> | <Y', hY', hY'ne, rfl>
      * rw [Set.inter_comm, sumTokMaster_inter_embF hX]
        exact Or.inr (Or.inl <X, hX, hXne, rfl>)
      * rw [embBit_inter] at hZsub  |-
        rcases hZ with rfl | <Z_0, hZ_0, -, rfl> | <Z_1, hZ_1, -, rfl>
        * exact absurd (hZsub nil_mem_sumTokMaster) nil_not_mem_embBit
        * exact Or.inr (Or.inl <X  I  X', D_0.inter_mem hX hX' hZ_0 (embBit_subset.mp
    hZsub),
            inter_ne_of_ne_left (D_0.sub_master hX) hXne, rfl>)
        * obtain <b, hb> := h_1 Z_1 hZ_1
          obtain <w', he, -> := hZsub (<b, rfl, hb> : (true :: b)  in  embBit true Z_1
    )
          simp only [List.cons.injEq] at he; exact absurd he.1 (by decide)
      * rw [embBit_inter_ne (show (false : Bool)  !=  true by decide)] at hZsub  |-
        obtain <t, ht> := hne Z hZ; exact absurd (hZsub ht) (Set.notMem_empty t)
    * rcases hW' with rfl | <X', hX', hX'ne, rfl> | <Y', hY', hY'ne, rfl>
      * rw [Set.inter_comm, sumTokMaster_inter_embT hY]
        exact Or.inr (Or.inr <Y, hY, hYne, rfl>)
      * rw [embBit_inter_ne (show (true : Bool)  !=  false by decide)] at hZsub  |-
        obtain <t, ht> := hne Z hZ; exact absurd (hZsub ht) (Set.notMem_empty t)
      * rw [embBit_inter] at hZsub  |-
        rcases hZ with rfl | <Z_0, hZ_0, -, rfl> | <Z_1, hZ_1, -, rfl>
        * exact absurd (hZsub nil_mem_sumTokMaster) nil_not_mem_embBit
        * obtain <a, ha> := h_0 Z_0 hZ_0
          obtain <w', he, -> := hZsub (<a, rfl, ha> : (false :: a)  in  embBit false
    Z_0)
          simp only [List.cons.injEq] at he; exact absurd he.1 (by decide)
        * exact Or.inr (Or.inr <Y  I  Y', D_1.inter_mem hY hY' hZ_1 (embBit_subset.mp
    hZsub),
            inter_ne_of_ne_left (D_1.sub_master hY) hYne, rfl>)

end Exercise619
end

section
```

```
namespace Exercise619

variable {D_0 D_1 : NeighborhoodSystem Str}

variable {h_0 : forall  X, D_0.mem X  ->  X.Nonempty} {h_1 : forall  Y, D_1.mem Y  ->
     Y.Nonempty}

theorem oplusTok_mem_embF {X : Set Str} (hX : D_0.mem X) (hXne : X  !=  D_0.master) :
    (oplusTok D_0 D_1 h_0 h_1).mem (embBit false X) := Or.inr (Or.inl <X, hX, hXne,
    rfl>)

theorem oplusTok_mem_embT {Y : Set Str} (hY : D_1.mem Y) (hYne : Y  !=  D_1.master) :
    (oplusTok D_0 D_1 h_0 h_1).mem (embBit true Y) := Or.inr (Or.inr <Y, hY, hYne, rfl
    >)

theorem oplusTok_nonempty : forall  W, (oplusTok D_0 D_1 h_0 h_1).mem W  ->  W.
    Nonempty := by
  rintro W (rfl | <X, hX, -, rfl> | <Y, hY, -, rfl>)
  * exact <[], nil_mem_sumTokMaster>
  * exact embBit_nonempty (h_0 X hX)
  * exact embBit_nonempty (h_1 Y hY)

/-- The **coalesced sum object** `?_0 ? ?_1`. -/
def ScottSys.oplus (A_0 A_1 : ScottSys) : ScottSys :=
  <oplusTok A_0.sys A_1.sys A_0.ne A_1.ne, oplusTok_nonempty>

end Exercise619
end

section
namespace Exercise619

variable {D_0 D_1 : NeighborhoodSystem Str}

variable {h_0 : forall  X, D_0.mem X  ->  X.Nonempty} {h_1 : forall  Y, D_1.mem Y  ->
     Y.Nonempty}

variable {A_0 A_1 B_0 B_1 : ScottSys}

variable {C_0 C_1 : ScottSys}

/-- n-ary coalesced sum `T_0 ? T_1 ? ? ? T_n`. -/
def GExpr.naryOplus : GExpr  ->  List GExpr  ->  GExpr := GExpr.naryOp GExpr.oplus

/-- **`f_0 ? f_1`, the action of the coalesced sum on maps.** -/
def oplusMapTok (f_0 : ApproximableMap A_0.sys B_0.sys) (f_1 : ApproximableMap A_1.sys
     B_1.sys) :
    ApproximableMap (A_0.oplus A_1).sys (B_0.oplus B_1).sys where
  rel W W' :=
    ((A_0.oplus A_1).sys.mem W  /\  W' = sumTokMaster B_0.sys B_1.sys)  \/
    (exists  X X', f_0.rel X X'  /\  X  !=  A_0.sys.master  /\  X'  !=  B_0.sys.master
      /\
      W = embBit false X  /\  W' = embBit false X')  \/
    (exists  Y Y', f_1.rel Y Y'  /\  Y  !=  A_1.sys.master  /\  Y'  !=  B_1.sys.master
      /\
      W = embBit true Y  /\  W' = embBit true Y')
```

```
rel_dom := by
  rintro W W' (<hW, -> | <X, X', hrel, hXne, -, rfl, -> | <Y, Y', hrel, hYne, -, rfl
  , ->)
  * exact hW
  * exact oplusTok_mem_embF (h_0 := A_0.ne) (h_1 := A_1.ne) (f_0.rel_dom hrel) hXne
  * exact oplusTok_mem_embT (h_0 := A_0.ne) (h_1 := A_1.ne) (f_1.rel_dom hrel) hYne
rel_cod := by
  rintro W W' (<-, rfl> | <X, X', hrel, -, hX'ne, -, rfl> | <Y, Y', hrel, -, hY'ne,
  -, rfl>)
  * exact Or.inl rfl
  * exact oplusTok_mem_embF (h_0 := B_0.ne) (h_1 := B_1.ne) (f_0.rel_cod hrel) hX'ne
  * exact oplusTok_mem_embT (h_0 := B_0.ne) (h_1 := B_1.ne) (f_1.rel_cod hrel) hY'ne
master_rel := Or.inl <(A_0.oplus A_1).sys.master_mem, rfl>
inter_right := by
  rintro W W'_1 W'_2 h1 h2
  rcases h1 with <hW, rfl> | <X, X', hrel, hXne, hX'ne, rfl, rfl> | <Y, Y', hrel,
  hYne, hY'ne, rfl, rfl>
  * rcases h2 with <-, rfl> | <X, X', hrel, hXne, hX'ne, hWeq, rfl> | <Y, Y', hrel,
  hYne, hY'ne, hWeq, rfl>
    * exact Or.inl <hW, by rw [Set.inter_self]>
    * exact Or.inr (Or.inl <X, X', hrel, hXne, hX'ne, hWeq,
        by rw [sumTokMaster_inter_embF (f_0.rel_cod hrel)]>)
    * exact Or.inr (Or.inr <Y, Y', hrel, hYne, hY'ne, hWeq,
        by rw [sumTokMaster_inter_embT (f_1.rel_cod hrel)]>)
  * rcases h2 with <-, rfl> | <X_2, X'_2, hrel_2, hX_2ne, hX'_2ne, hWeq, rfl> | <Y_2
  , Y'_2, hrel_2, -, -, hWeq, rfl>
    * refine Or.inr (Or.inl <X, X', hrel, hXne, hX'ne, rfl, ?_>)
      rw [Set.inter_comm, sumTokMaster_inter_embF (f_0.rel_cod hrel)]
    * obtain rfl := embBit_injective hWeq
      exact Or.inr (Or.inl <X, X'  I  X'_2, f_0.inter_right hrel hrel_2, hXne,
        inter_ne_of_ne_left (B_0.sys.sub_master (f_0.rel_cod hrel)) hX'ne, rfl,
        embBit_inter false X' X'_2>)
    * exact absurd hWeq (embBit_ne (by decide) (A_0.ne X (f_0.rel_dom hrel)))
  * rcases h2 with <-, rfl> | <X_2, X'_2, hrel_2, -, -, hWeq, rfl> | <Y_2, Y'_2,
  hrel_2, hY_2ne, hY'_2ne, hWeq, rfl>
    * refine Or.inr (Or.inr <Y, Y', hrel, hYne, hY'ne, rfl, ?_>)
      rw [Set.inter_comm, sumTokMaster_inter_embT (f_1.rel_cod hrel)]
    * exact absurd hWeq (embBit_ne (by decide) (A_1.ne Y (f_1.rel_dom hrel)))
    * obtain rfl := embBit_injective hWeq
      exact Or.inr (Or.inr <Y, Y'  I  Y'_2, f_1.inter_right hrel hrel_2, hYne,
        inter_ne_of_ne_left (B_1.sys.sub_master (f_1.rel_cod hrel)) hY'ne, rfl,
        embBit_inter true Y' Y'_2>)
mono := by
  rintro W W'' Z Z' h hWW hZZ' hW'' hZ'
  rcases h with <-, rfl> | <X, X', hrel, hXne, hX'ne, rfl, rfl> | <Y, Y', hrel, hYne
  , hY'ne, rfl, rfl>
  * exact Or.inl <hW'', Set.Subset.antisymm ((B_0.oplus B_1).sys.sub_master hZ') hZZ
  '>
  * rcases hZ' with rfl | <X_3, hX_3, hX_3ne, rfl> | <Y_3, hY_3, hY_3ne, rfl>
    * exact Or.inl <hW'', rfl>
    * rcases hW'' with rfl | <X_2, hX_2, hX_2ne, rfl> | <Y_2, hY_2, hY_2ne, rfl>
      * exact absurd (hWW nil_mem_sumTokMaster) nil_not_mem_embBit
      * exact Or.inr (Or.inl <X_2, X_3,
          f_0.mono hrel (embBit_subset.mp hWW) (embBit_subset.mp hZZ') hX_2 hX_3,
          hX_2ne, hX_3ne, rfl, rfl>)
      * exact absurd hWW (fun hsub => embBit_not_subset_cross (by decide) (A_1.ne
```

```
    Y_2 hY_2) hsub)
      * exact absurd hZZ' (fun hsub =>
          embBit_not_subset_cross (by decide) (B_0.ne X' (f_0.rel_cod hrel)) hsub)
    * rcases hZ' with rfl | <X_3, hX_3, hX_3ne, rfl> | <Y_3, hY_3, hY_3ne, rfl>
      * exact Or.inl <hW'', rfl>
      * exact absurd hZZ' (fun hsub =>
          embBit_not_subset_cross (by decide) (B_1.ne Y' (f_1.rel_cod hrel)) hsub)
      * rcases hW'' with rfl | <X_2, hX_2, hX_2ne, rfl> | <Y_2, hY_2, hY_2ne, rfl>
        * exact absurd (hWW nil_mem_sumTokMaster) nil_not_mem_embBit
        * exact absurd hWW (fun hsub => embBit_not_subset_cross (by decide) (A_0.ne
    X_2 hX_2) hsub)
        * exact Or.inr (Or.inr <Y_2, Y_3,
            f_1.mono hrel (embBit_subset.mp hWW) (embBit_subset.mp hZZ') hY_2 hY_3,
            hY_2ne, hY_3ne, rfl, rfl>)

end Exercise619
end

section
namespace Exercise619

variable {D_0 D_1 : NeighborhoodSystem Str}

variable {h_0 : forall  X, D_0.mem X  ->  X.Nonempty} {h_1 : forall  Y, D_1.mem Y  ->
     Y.Nonempty}

/-- **Exercise 6.21 -- the smash product system `?_0 ? ?_1` over `{0,1}*`.** As `
    prodTok`, but proper
rectangles must avoid both top coordinates (`X  !=  Delta_0`, `Y  !=  Delta_1`); the
    full top `M = prodTokNbhd Delta_0 Delta_1`
is kept as the master. -/
def otimesTok (D_0 D_1 : NeighborhoodSystem Str) : NeighborhoodSystem Str where
  mem W := W = prodTokNbhd D_0.master D_1.master  \/
    (exists  X Y, D_0.mem X  /\  D_1.mem Y  /\  X  !=  D_0.master  /\  Y  !=  D_1.
    master  /\  W = prodTokNbhd X Y)
  master := prodTokNbhd D_0.master D_1.master
  master_nonempty := <[], mem_prodTokNbhd_nil>
  master_mem := Or.inl rfl
  sub_master := by
    rintro W (rfl | <X, Y, hX, hY, -, -, rfl>)
    * exact subset_rfl
    * exact prodTokNbhd_subset_iff.mpr <D_0.sub_master hX, D_1.sub_master hY>
  inter_mem := by
    rintro W W' Z hW hW' hZ hZsub
    rcases hW with rfl | <X, Y, hX, hY, hXne, hYne, rfl>
    * rcases hW' with rfl | <X', Y', hX', hY', hX'ne, hY'ne, rfl>
      * rw [Set.inter_self]; exact Or.inl rfl
      * rw [prodTokNbhd_inter, Set.inter_eq_right.mpr (D_0.sub_master hX'),
          Set.inter_eq_right.mpr (D_1.sub_master hY')]
        exact Or.inr <X', Y', hX', hY', hX'ne, hY'ne, rfl>
    * rcases hW' with rfl | <X', Y', hX', hY', hX'ne, hY'ne, rfl>
      * rw [Set.inter_comm, prodTokNbhd_inter, Set.inter_eq_right.mpr (D_0.sub_master
    hX),
          Set.inter_eq_right.mpr (D_1.sub_master hY)]
        exact Or.inr <X, Y, hX, hY, hXne, hYne, rfl>
      * rw [prodTokNbhd_inter] at hZsub  |-
```

```
        rcases hZ with rfl | <Z_0, Z_1, hZ_0, hZ_1, -, -, rfl>
        * obtain <hsub_0, -> := prodTokNbhd_subset_iff.mp hZsub
          exact absurd (Set.Subset.antisymm (D_0.sub_master hX)
            (hsub_0.trans Set.inter_subset_left)) hXne
        * obtain <hsub_0, hsub_1> := prodTokNbhd_subset_iff.mp hZsub
          exact Or.inr <X  I  X', Y  I  Y', D_0.inter_mem hX hX' hZ_0 hsub_0,
            D_1.inter_mem hY hY' hZ_1 hsub_1, inter_ne_of_ne_left (D_0.sub_master hX)
    hXne,
            inter_ne_of_ne_left (D_1.sub_master hY) hYne, rfl>

theorem otimesTok_mem_master : (otimesTok D_0 D_1).mem (prodTokNbhd D_0.master D_1.
    master) := Or.inl rfl

theorem otimesTok_mem_prod {X Y : Set Str} (hX : D_0.mem X) (hY : D_1.mem Y)
    (hXne : X  !=  D_0.master) (hYne : Y  !=  D_1.master) :
    (otimesTok D_0 D_1).mem (prodTokNbhd X Y) := Or.inr <X, Y, hX, hY, hXne, hYne, rfl
    >

theorem otimesTok_nonempty : forall  W, (otimesTok D_0 D_1).mem W  ->  W.Nonempty :=
    by
  rintro W (rfl | <X, Y, -, -, -, -, rfl>) <;> exact <[], mem_prodTokNbhd_nil>

/-- The **smash product object** `?_0 ? ?_1`. -/
def ScottSys.otimes (A_0 A_1 : ScottSys) : ScottSys :=
  <otimesTok A_0.sys A_1.sys, otimesTok_nonempty>

end Exercise619
end

section
namespace Exercise619

variable {D_0 D_1 : NeighborhoodSystem Str}

variable {h_0 : forall  X, D_0.mem X  ->  X.Nonempty} {h_1 : forall  Y, D_1.mem Y  ->
     Y.Nonempty}

variable {A_0 A_1 B_0 B_1 : ScottSys}

variable {C_0 C_1 : ScottSys}

/-- **`?  in  tok(C)` for every constant `C` occurring in `T`.** -/
def GExpr.RootedConst : GExpr  ->  Prop
  | .const C => ([] : Str)  in  C.sys.master
  | .var => True
  | .sum a b => a.RootedConst  /\  b.RootedConst
  | .prod a b => a.RootedConst  /\  b.RootedConst
  | .oplus a b => a.RootedConst  /\  b.RootedConst
  | .otimes a b => a.RootedConst  /\  b.RootedConst

/-- **The token-level master recursion for `GExpr`.** All four binary operations share
    the same body
(`sumTokMaster = prodTokNbhd` on masters). -/
def gFun : GExpr  ->  Set Str  ->  Set Str
  | .const C, _ => C.sys.master
  | .var, Gamma => Gamma
```

```
  | .sum a b, Gamma => insert ([] : Str) (embBit false (gFun a Gamma)  U  embBit true
    (gFun b Gamma))
  | .prod a b, Gamma => insert ([] : Str) (embBit false (gFun a Gamma)  U  embBit true
     (gFun b Gamma))
  | .oplus a b, Gamma => insert ([] : Str) (embBit false (gFun a Gamma)  U  embBit
    true (gFun b Gamma))
  | .otimes a b, Gamma => insert ([] : Str) (embBit false (gFun a Gamma)  U  embBit
    true (gFun b Gamma))

/-- The **Kleene iteration** `gFun?({?})`. -/
def gIter (T : GExpr) : Nat  ->  Set Str
  | 0 => {([] : Str)}
  | n + 1 => gFun T (gIter T n)

/-- n-ary smash product `T_0 ? T_1 ? ? ? T_n`. -/
def GExpr.naryOtimes : GExpr  ->  List GExpr  ->  GExpr := GExpr.naryOp GExpr.otimes

/-- **The action of `T` on objects.** -/
def GExpr.obj : GExpr  ->  ScottSys  ->  ScottSys
  | .const D, _ => D
  | .var, X => X
  | .sum a b, X => (a.obj X).sum (b.obj X)
  | .prod a b, X => (a.obj X).prod (b.obj X)
  | .oplus a b, X => (a.obj X).oplus (b.obj X)
  | .otimes a b, X => (a.obj X).otimes (b.obj X)

/-- **`f_0 ? f_1`, the action of the smash product on maps.** -/
def otimesMapTok (f_0 : ApproximableMap A_0.sys B_0.sys) (f_1 : ApproximableMap A_1.
    sys B_1.sys) :
    ApproximableMap (A_0.otimes A_1).sys (B_0.otimes B_1).sys where
  rel W W' :=
    ((A_0.otimes A_1).sys.mem W  /\  W' = prodTokNbhd B_0.sys.master B_1.sys.master)
    \/
    (exists  X Y X' Y', f_0.rel X X'  /\  f_1.rel Y Y'  /\  X  !=  A_0.sys.master  /\
     Y  !=  A_1.sys.master  /\
      X'  !=  B_0.sys.master  /\  Y'  !=  B_1.sys.master  /\  W = prodTokNbhd X Y  /\
     W' = prodTokNbhd X' Y')
  rel_dom := by
    rintro W W' (<hW, -> | <X, Y, X', Y', h0, h1, hXne, hYne, -, -, rfl, ->)
    * exact hW
    * exact otimesTok_mem_prod (f_0.rel_dom h0) (f_1.rel_dom h1) hXne hYne
  rel_cod := by
    rintro W W' (<-, rfl> | <X, Y, X', Y', h0, h1, -, -, hX'ne, hY'ne, -, rfl>)
    * exact otimesTok_mem_master
    * exact otimesTok_mem_prod (f_0.rel_cod h0) (f_1.rel_cod h1) hX'ne hY'ne
  master_rel := Or.inl <(A_0.otimes A_1).sys.master_mem, rfl>
  inter_right := by
    rintro W W'_1 W'_2 h1 h2
    rcases h1 with <hW, rfl> | <X, Y, X', Y', hr0, hr1, hXne, hYne, hX'ne, hY'ne, rfl,
     rfl>
    * rcases h2 with <-, rfl> | <X, Y, X', Y', hr0, hr1, hXne, hYne, hX'ne, hY'ne,
    hWeq, rfl>
      * exact Or.inl <hW, by rw [Set.inter_self]>
      * refine Or.inr <X, Y, X', Y', hr0, hr1, hXne, hYne, hX'ne, hY'ne, hWeq, ?_>
        rw [prodTokNbhd_inter, Set.inter_eq_right.mpr (B_0.sys.sub_master (f_0.rel_cod
     hr0)),
```

```
          Set.inter_eq_right.mpr (B_1.sys.sub_master (f_1.rel_cod hr1))]
    * rcases h2 with <-, rfl> | <X_2, Y_2, X'_2, Y'_2, hr0_2, hr1_2, hX_2ne, hY_2ne,
    hX'_2ne, hY'_2ne, hWeq, rfl>
      * refine Or.inr <X, Y, X', Y', hr0, hr1, hXne, hYne, hX'ne, hY'ne, rfl, ?_>
        rw [Set.inter_comm, prodTokNbhd_inter,
          Set.inter_eq_right.mpr (B_0.sys.sub_master (f_0.rel_cod hr0)),
          Set.inter_eq_right.mpr (B_1.sys.sub_master (f_1.rel_cod hr1))]
      * obtain <rfl, rfl> := prodTokNbhd_injective hWeq
        refine Or.inr <X, Y, X'  I  X'_2, Y'  I  Y'_2, f_0.inter_right hr0 hr0_2,
          f_1.inter_right hr1 hr1_2, hXne, hYne,
          inter_ne_of_ne_left (B_0.sys.sub_master (f_0.rel_cod hr0)) hX'ne,
          inter_ne_of_ne_left (B_1.sys.sub_master (f_1.rel_cod hr1)) hY'ne, rfl, ?_>
        rw [prodTokNbhd_inter]
  mono := by
    rintro W W'' Z Z' h hWW hZZ' hZmem hZ'mem
    rcases h with <-, rfl> | <X, Y, X', Y', hr0, hr1, hXne, hYne, hX'ne, hY'ne, rfl,
    rfl>
    * exact Or.inl <hZmem, Set.Subset.antisymm ((B_0.otimes B_1).sys.sub_master hZ'mem
    ) hZZ'>
    * rcases hZmem with rfl | <Xz, Yz, hXz, hYz, hXzne, hYzne, rfl>
      * obtain <hsubX, -> := prodTokNbhd_subset_iff.mp hWW
        exact absurd (Set.Subset.antisymm (A_0.sys.sub_master (f_0.rel_dom hr0)) hsubX
    ) hXne
      * rcases hZ'mem with rfl | <Xz', Yz', hXz', hYz', hXz'ne, hYz'ne, rfl>
        * exact Or.inl <otimesTok_mem_prod hXz hYz hXzne hYzne, rfl>
        * obtain <hXzX, hYzY> := prodTokNbhd_subset_iff.mp hWW
          obtain <hX'Xz', hY'Yz'> := prodTokNbhd_subset_iff.mp hZZ'
          exact Or.inr <Xz, Yz, Xz', Yz', f_0.mono hr0 hXzX hX'Xz' hXz hXz',
            f_1.mono hr1 hYzY hY'Yz' hYz hYz', hXzne, hYzne, hXz'ne, hYz'ne, rfl, rfl>

/-- **The action of `T` on maps.** -/
def GExpr.map : (T : GExpr)  ->  {X Y : ScottSys}  ->  ApproximableMap X.sys Y.sys  ->
    ApproximableMap (T.obj X).sys (T.obj Y).sys
  | .const D, _, _, _ => idMap D.sys
  | .var, _, _, f => f
  | .sum a b, _, _, f => sumMapTok (a.map f) (b.map f)
  | .prod a b, _, _, f => prodMapTok (a.map f) (b.map f)
  | .oplus a b, _, _, f => oplusMapTok (a.map f) (b.map f)
  | .otimes a b, _, _, f => otimesMapTok (a.map f) (b.map f)

/-- `gFun T Gamma = tok(T({Gamma}))`. -/
theorem gFun_eq_master : (T : GExpr)  ->  {Gamma : Set Str}  ->  (h : Gamma.Nonempty)
     ->
    gFun T Gamma = (T.obj (singletonSys Gamma h)).sys.master
  | .const _, _, _ => rfl
  | .var, _, _ => rfl
  | .sum a b, Gamma, h => by
      show insert ([] : Str) (embBit false (gFun a Gamma)  U  embBit true (gFun b
    Gamma))
        = insert ([] : Str) (embBit false ((a.obj (singletonSys Gamma h)).sys.master)
            U  embBit true ((b.obj (singletonSys Gamma h)).sys.master))
      rw [gFun_eq_master a h, gFun_eq_master b h]
  | .prod a b, Gamma, h => by
      show insert ([] : Str) (embBit false (gFun a Gamma)  U  embBit true (gFun b
    Gamma))
        = insert ([] : Str) (embBit false ((a.obj (singletonSys Gamma h)).sys.master)
```

```
            U  embBit true ((b.obj (singletonSys Gamma h)).sys.master))
      rw [gFun_eq_master a h, gFun_eq_master b h]
  | .oplus a b, Gamma, h => by
      show insert ([] : Str) (embBit false (gFun a Gamma)  U  embBit true (gFun b
    Gamma))
          = insert ([] : Str) (embBit false ((a.obj (singletonSys Gamma h)).sys.master)
            U  embBit true ((b.obj (singletonSys Gamma h)).sys.master))
      rw [gFun_eq_master a h, gFun_eq_master b h]
  | .otimes a b, Gamma, h => by
      show insert ([] : Str) (embBit false (gFun a Gamma)  U  embBit true (gFun b
    Gamma))
          = insert ([] : Str) (embBit false ((a.obj (singletonSys Gamma h)).sys.master)
            U  embBit true ((b.obj (singletonSys Gamma h)).sys.master))
      rw [gFun_eq_master a h, gFun_eq_master b h]

/-- **`fun Gamma. tok(T({Gamma}))` is monotone.** -/
theorem gFun_mono (T : GExpr) {Gamma Gamma' : Set Str} (h : Gamma  subseteq  Gamma') :
     gFun T Gamma  subseteq  gFun T Gamma' := by
  induction T with
  | const C => exact subset_rfl
  | var => exact h
  | sum a b ih_0 ih_1 => exact insertTag_mono ih_0 ih_1
  | prod a b ih_0 ih_1 => exact insertTag_mono ih_0 ih_1
  | oplus a b ih_0 ih_1 => exact insertTag_mono ih_0 ih_1
  | otimes a b ih_0 ih_1 => exact insertTag_mono ih_0 ih_1

/-- **`fun Gamma. tok(T({Gamma}))` is continuous on `{Gamma ? ?  in  Gamma}`.** -/
theorem gFun_continuous (T : GExpr) {? : Set (Set Str)} {U : Set Str}
    (_hdir : DirectedOn (*  subseteq  *) ?) (hne : ?.Nonempty)
    (hU : forall  w, w  in  U  <->  exists  Gamma  in  ?, w  in  Gamma) :
    forall  w, w  in  gFun T U  <->  exists  Gamma  in  ?, w  in  gFun T Gamma := by
  induction T with
  | const C =>
      intro w
      exact <fun hw => let <Gamma, hGamma> := hne; <Gamma, hGamma, hw>, fun <_, _, hw>
     => hw>
  | var => intro w; exact hU w
  | sum a b ih_0 ih_1 => intro w; exact insertTag_continuous hne ih_0 ih_1 w
  | prod a b ih_0 ih_1 => intro w; exact insertTag_continuous hne ih_0 ih_1 w


  | oplus a b ih_0 ih_1 => intro w; exact insertTag_continuous hne ih_0 ih_1 w
  | otimes a b ih_0 ih_1 => intro w; exact insertTag_continuous hne ih_0 ih_1 w

theorem gFun_nil_mem : forall  (T : GExpr), T.RootedConst  ->  {Gamma : Set Str}  ->
    ([] : Str)  in  Gamma  ->  ([] : Str)  in  gFun T Gamma
  | .const _, hC, _, _ => hC
  | .var, _, _, hGamma => hGamma
  | .sum _ _, _, _, _ => Set.mem_insert _ _
  | .prod _ _, _, _, _ => Set.mem_insert _ _
  | .oplus _ _, _, _, _ => Set.mem_insert _ _
  | .otimes _ _, _, _, _ => Set.mem_insert _ _

theorem nil_mem_gIter (T : GExpr) (hT : T.RootedConst) : forall  n, ([] : Str)  in
    gIter T n
  | 0 => rfl
  | n + 1 => gFun_nil_mem T hT (nil_mem_gIter T hT n)
```

```
theorem gIter_mono_step (T : GExpr) (hT : T.RootedConst) :
    forall  n, gIter T n  subseteq  gIter T (n + 1)
  | 0 => by
      intro w hw
      have hw' : w = [] := hw
      subst hw'
      exact gFun_nil_mem T hT rfl
  | n + 1 => gFun_mono T (gIter_mono_step T hT n)

theorem gIter_mono (T : GExpr) (hT : T.RootedConst) {m n : Nat} (hmn : m  <=  n) :
    gIter T m  subseteq  gIter T n := by
  induction hmn with
  | refl => exact subset_rfl
  | step _ ih => intro x hx; exact gIter_mono_step T hT _ (ih hx)

/-- The Kleene union is a **fixed point** of `fun Gamma. tok(T({Gamma}))`. -/
theorem gFun_iter_fixed (T : GExpr) (hT : T.RootedConst) :
    gFun T (? n, gIter T n) = ? n, gIter T n := by
  have hstep := gIter_mono_step T hT
  have hne : (Set.range (gIter T)).Nonempty := <gIter T 0, 0, rfl>
  have hdir : DirectedOn (*  subseteq  *) (Set.range (gIter T)) := by
    rintro _ <i, rfl> _ <j, rfl>
    exact <gIter T (max i j), <max i j, rfl>,
      gIter_mono T hT (le_max_left i j), gIter_mono T hT (le_max_right i j)>
  have hU : forall  v, v  in  (? n, gIter T n)  <->  exists  S  in  Set.range (gIter T
    ), v  in  S := by
    intro v
    constructor
    * intro hv; rw [Set.mem_iUnion] at hv; obtain <n, hn> := hv
      exact <gIter T n, <n, rfl>, hn>
    * rintro <S, <n, rfl>, hv>; exact Set.mem_iUnion.mpr <n, hv>
  apply Set.ext; intro w
  rw [gFun_continuous T hdir hne hU w]
  constructor
  * rintro <S, <n, rfl>, hwS>; exact Set.mem_iUnion.mpr <n + 1, hwS>
  * intro hw
    rw [Set.mem_iUnion] at hw; obtain <n, hn> := hw
    exact <gIter T n, <n, rfl>, hstep n hn>

/-- **Exercise 6.21/6.20 (token level).** For any `GExpr` `T` whose constants contain
    `?`, there is a
set `Gamma` with `?  in  Gamma` and `Gamma = tok(T({Gamma}))`. -/
theorem gExists_tok_fixedPoint (T : GExpr) (hT : T.RootedConst) :
    exists  Gamma : Set Str, ([] : Str)  in  Gamma  /\  gFun T Gamma = Gamma :=
  <? n, gIter T n, Set.mem_iUnion.mpr <0, nil_mem_gIter T hT 0>, gFun_iter_fixed T hT>

/-- **Exercise 6.21/6.20 (object level): `{Gamma} ? T({Gamma})`, so Theorem 6.14
    applies** for any construct
`T` built from constants, identity, `+`, ` x `, `?`, `?`. -/
theorem gExists_singleton_subsystem (T : GExpr) (hT : T.RootedConst) :
    exists  (Gamma : Set Str) (h : Gamma.Nonempty),
      (singletonSys Gamma h).sys ? (T.obj (singletonSys Gamma h)).sys := by
  obtain <Gamma, hnil, hfix> := gExists_tok_fixedPoint T hT
  have hne : Gamma.Nonempty := <[], hnil>
  have hmaster : (T.obj (singletonSys Gamma hne)).sys.master = Gamma :=
```

```
    (gFun_eq_master T hne).symm.trans hfix
  refine <Gamma, hne, ?_, ?_, ?_>
  * exact hmaster.symm
  * intro X hX
    have heq : X = (T.obj (singletonSys Gamma hne)).sys.master := (hX : X = Gamma).
    trans hmaster.symm
    rw [heq]
    exact (T.obj (singletonSys Gamma hne)).sys.master_mem
  * intro X Y hX hY _
    show X  I  Y = Gamma
    rw [show X = Gamma from hX, show Y = Gamma from hY, Set.inter_self]

end Exercise619
end

end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.Exercise619
section
set_option linter.unusedVariables false

theorem example_621 : And
  (forall  (T : DomainTheory.NeighbourhoodSystems.Exercise619.GExpr),
    T.RootedConst  ->
      exists  (Gamma : Set DomainTheory.NeighbourhoodSystems.ExampleB.Str) (h :
        @Set.Nonempty DomainTheory.NeighbourhoodSystems.ExampleB.Str Gamma),
        @DomainTheory.NeighbourhoodSystems.Subsystem DomainTheory.NeighbourhoodSystems
    .ExampleB.Str
          (DomainTheory.NeighbourhoodSystems.Exercise619.singletonSys Gamma h).sys
          (T.obj (DomainTheory.NeighbourhoodSystems.Exercise619.singletonSys Gamma h))
    .sys)
  (forall  (T : DomainTheory.NeighbourhoodSystems.Exercise619.GExpr),
    T.RootedConst  ->
      exists  (Gamma : Set DomainTheory.NeighbourhoodSystems.ExampleB.Str),
        And
          (@Membership.mem (List Bool) (Set DomainTheory.NeighbourhoodSystems.ExampleB
    .Str)
            (@Set.instMembership DomainTheory.NeighbourhoodSystems.ExampleB.Str) Gamma
     (@List.nil Bool))
          (@Eq (Set DomainTheory.NeighbourhoodSystems.ExampleB.Str)
            (DomainTheory.NeighbourhoodSystems.Exercise619.gFun T Gamma) Gamma)) :=
  @And.intro
  (forall  (T : DomainTheory.NeighbourhoodSystems.Exercise619.GExpr),
    T.RootedConst  ->
      exists  (Gamma : Set DomainTheory.NeighbourhoodSystems.ExampleB.Str) (h :
        @Set.Nonempty DomainTheory.NeighbourhoodSystems.ExampleB.Str Gamma),
        @DomainTheory.NeighbourhoodSystems.Subsystem DomainTheory.NeighbourhoodSystems
    .ExampleB.Str
          (DomainTheory.NeighbourhoodSystems.Exercise619.singletonSys Gamma h).sys
          (T.obj (DomainTheory.NeighbourhoodSystems.Exercise619.singletonSys Gamma h))
    .sys)
  (forall  (T : DomainTheory.NeighbourhoodSystems.Exercise619.GExpr),
    T.RootedConst  ->
      exists  (Gamma : Set DomainTheory.NeighbourhoodSystems.ExampleB.Str),
        And
          (@Membership.mem (List Bool) (Set DomainTheory.NeighbourhoodSystems.ExampleB
    .Str)
```

```
            (@Set.instMembership DomainTheory.NeighbourhoodSystems.ExampleB.Str) Gamma
     (@List.nil Bool))
          (@Eq (Set DomainTheory.NeighbourhoodSystems.ExampleB.Str)
            (DomainTheory.NeighbourhoodSystems.Exercise619.gFun T Gamma) Gamma))
  DomainTheory.NeighbourhoodSystems.Exercise619.gExists_singleton_subsystem
  DomainTheory.NeighbourhoodSystems.Exercise619.gExists_tok_fixedPoint
end

end DomainTheory.NeighbourhoodSystems.Exercise619
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.Exercise619
open DomainTheory.NeighbourhoodSystems.Example62 DomainTheory.NeighbourhoodSystems.
    ExampleB
open DomainTheory.NeighbourhoodSystems.Exercise510

section
namespace Exercise619

variable {D_0 D_1 : NeighborhoodSystem Str}

variable {h_0 : forall  X, D_0.mem X  ->  X.Nonempty} {h_1 : forall  Y, D_1.mem Y  ->
     Y.Nonempty}

variable {A_0 A_1 B_0 B_1 : ScottSys}

variable {C_0 C_1 : ScottSys}

/-- The `?  in  tok` side-condition is preserved by any n-ary fold whose binary
    operation preserves it
(all four of `+`, ` x `, `?`, `?` do, definitionally). -/
theorem GExpr.naryOp_rootedConst {op : GExpr  ->  GExpr  ->  GExpr}
    (hop : forall  x y, (op x y).RootedConst  <->  x.RootedConst  /\  y.RootedConst) :
    forall  (a : GExpr) (l : List GExpr), a.RootedConst  ->  (forall  b  in  l, b.
    RootedConst)  ->
      (GExpr.naryOp op a l).RootedConst
  | a, [], ha, _ => ha
  | a, b :: l, ha, hl => by
      rw [GExpr.naryOp, hop]
      exact <ha, GExpr.naryOp_rootedConst hop b l (hl b (List.mem_cons_self ..))
        (fun c hc => hl c (List.mem_cons_of_mem _ hc))>

end Exercise619
end

section
namespace Exercise619

variable {D_0 D_1 : NeighborhoodSystem Str}

theorem inter_ne_of_ne_right {X X' Delta : Set Str} (hX' : X'  subseteq  Delta) (hne :
     X'  !=  Delta) : X  I  X'  !=  Delta := by
  intro h
  exact hne (Set.Subset.antisymm hX' (by rw [ <-  h]; exact Set.inter_subset_right))

end Exercise619
end
```

```
section
namespace Exercise619

variable {D_0 D_1 : NeighborhoodSystem Str}

variable {h_0 : forall  X, D_0.mem X  ->  X.Nonempty} {h_1 : forall  Y, D_1.mem Y  ->
     Y.Nonempty}

variable {A_0 A_1 B_0 B_1 : ScottSys}

variable {C_0 C_1 : ScottSys}

theorem naryOplus_singleton_subsystem (a : GExpr) (l : List GExpr)
    (ha : a.RootedConst) (hl : forall  b  in  l, b.RootedConst) :
    exists  (Gamma : Set Str) (h : Gamma.Nonempty),
      (singletonSys Gamma h).sys ? ((GExpr.naryOplus a l).obj (singletonSys Gamma h)).
    sys :=
  gExists_singleton_subsystem _ (GExpr.naryOp_rootedConst (fun _ _ => Iff.rfl) a l ha
    hl)

theorem naryOtimes_singleton_subsystem (a : GExpr) (l : List GExpr)
    (ha : a.RootedConst) (hl : forall  b  in  l, b.RootedConst) :
    exists  (Gamma : Set Str) (h : Gamma.Nonempty),
      (singletonSys Gamma h).sys ? ((GExpr.naryOtimes a l).obj (singletonSys Gamma h))
    .sys :=
  gExists_singleton_subsystem _ (GExpr.naryOp_rootedConst (fun _ _ => Iff.rfl) a l ha
    hl)

theorem naryProd_singleton_subsystem (a : GExpr) (l : List GExpr)
    (ha : a.RootedConst) (hl : forall  b  in  l, b.RootedConst) :
    exists  (Gamma : Set Str) (h : Gamma.Nonempty),
      (singletonSys Gamma h).sys ? ((GExpr.naryProd a l).obj (singletonSys Gamma h)).
    sys :=
  gExists_singleton_subsystem _ (GExpr.naryOp_rootedConst (fun _ _ => Iff.rfl) a l ha
    hl)

/-- **Every n-ary construct has a solution `Gamma = tok(T({Gamma}))`** (so `{Gamma} ? T
    ({Gamma})` and 6.14 applies),
illustrated for the n-ary separated sum; identical for `naryProd`/`naryOplus`/`
    naryOtimes`. -/
theorem narySum_singleton_subsystem (a : GExpr) (l : List GExpr)
    (ha : a.RootedConst) (hl : forall  b  in  l, b.RootedConst) :
    exists  (Gamma : Set Str) (h : Gamma.Nonempty),
      (singletonSys Gamma h).sys ? ((GExpr.narySum a l).obj (singletonSys Gamma h)).
    sys :=
  gExists_singleton_subsystem _ (GExpr.naryOp_rootedConst (fun _ _ => Iff.rfl) a l ha
    hl)

/-- **`(g_0  comp  f_0) ? (g_1  comp  f_1) = (g_0 ? g_1)  comp  (f_0 ? f_1)`** for **
    strict** `g_0, g_1`. (Strictness of the
outer maps is exactly what prevents an intermediate top `f_0(X) = Delta_0'` from being
    re-expanded -- that
is the categorical reason `?` is a functor only on the strict-map category Scott
    restricts to.) -/
theorem oplusMapTok_comp (f_0 : ApproximableMap A_0.sys B_0.sys) (f_1 :
```

```
    ApproximableMap A_1.sys B_1.sys)
    {g_0 : ApproximableMap B_0.sys C_0.sys} {g_1 : ApproximableMap B_1.sys C_1.sys}
    (hg_0 : IsStrict g_0) (hg_1 : IsStrict g_1) :
    oplusMapTok (g_0.comp f_0) (g_1.comp f_1) = (oplusMapTok g_0 g_1).comp (
    oplusMapTok f_0 f_1) := by
  apply ApproximableMap.ext
  intro W W''
  constructor
  * rintro (<hW, rfl> | <X, X'', <X', hf, hg>, hXne, hX''ne, rfl, rfl> |
      <Y, Y'', <Y', hf, hg>, hYne, hY''ne, rfl, rfl>)
    * exact <sumTokMaster B_0.sys B_1.sys, Or.inl <hW, rfl>,
        Or.inl <(B_0.oplus B_1).sys.master_mem, rfl>>
    * have hX'ne : X'  !=  B_0.sys.master := fun h => hX''ne (hg_0 (h  |>  hg))
      exact <embBit false X', Or.inr (Or.inl <X, X', hf, hXne, hX'ne, rfl, rfl>),
        Or.inr (Or.inl <X', X'', hg, hX'ne, hX''ne, rfl, rfl>)>
    * have hY'ne : Y'  !=  B_1.sys.master := fun h => hY''ne (hg_1 (h  |>  hg))
      exact <embBit true Y', Or.inr (Or.inr <Y, Y', hf, hYne, hY'ne, rfl, rfl>),
        Or.inr (Or.inr <Y', Y'', hg, hY'ne, hY''ne, rfl, rfl>)>
  * rintro <W', hWW', hW'W''>
    rcases hWW' with <hW, rfl> | <X, X', hf, hXne, hX'ne, rfl, rfl> | <Y, Y', hf, hYne
    , hY'ne, rfl, rfl>
    * rcases hW'W'' with <-, rfl> | <X, X', -, -, -, heq, -> | <Y, Y', -, -, -, heq,
    ->
      * exact Or.inl <hW, rfl>
      * exact absurd (heq  |>  nil_mem_sumTokMaster) nil_not_mem_embBit
      * exact absurd (heq  |>  nil_mem_sumTokMaster) nil_not_mem_embBit
    * rcases hW'W'' with <-, rfl> | <X_2, X'', hg, -, hX''ne, heq, rfl> | <Y_2, Y'',
    hg, -, -, heq, ->
      * exact Or.inl <oplusTok_mem_embF (h_0 := A_0.ne) (h_1 := A_1.ne) (f_0.rel_dom
    hf) hXne, rfl>
      * obtain rfl := embBit_injective heq
        exact Or.inr (Or.inl <X, X'', <X', hf, hg>, hXne, hX''ne, rfl, rfl>)
      * exact absurd heq (embBit_ne (by decide) (B_0.ne X' (f_0.rel_cod hf)))
    * rcases hW'W'' with <-, rfl> | <X_2, X'', hg, -, -, heq, -> | <Y_2, Y'', hg, -,
    hY''ne, heq, rfl>
      * exact Or.inl <oplusTok_mem_embT (h_0 := A_0.ne) (h_1 := A_1.ne) (f_1.rel_dom
    hf) hYne, rfl>
      * exact absurd heq (embBit_ne (by decide) (B_1.ne Y' (f_1.rel_cod hf)))
      * obtain rfl := embBit_injective heq
        exact Or.inr (Or.inr <Y, Y'', <Y', hf, hg>, hYne, hY''ne, rfl, rfl>)

/-- **`(I ? I) = I`.** -/
theorem oplusMapTok_id :
    oplusMapTok (idMap A_0.sys) (idMap A_1.sys) = idMap (A_0.oplus A_1).sys := by
  apply ApproximableMap.ext
  intro W W'
  constructor
  * rintro (<hW, rfl> | <X, X', <hX, hX', hsub>, hXne, hX'ne, rfl, rfl> |
      <Y, Y', <hY, hY', hsub>, hYne, hY'ne, rfl, rfl>)
    * exact <hW, (A_0.oplus A_1).sys.master_mem, (A_0.oplus A_1).sys.sub_master hW>
    * exact <oplusTok_mem_embF (h_0 := A_0.ne) (h_1 := A_1.ne) hX hXne,
        oplusTok_mem_embF (h_0 := A_0.ne) (h_1 := A_1.ne) hX' hX'ne, embBit_subset.mpr
     hsub>
    * exact <oplusTok_mem_embT (h_0 := A_0.ne) (h_1 := A_1.ne) hY hYne,
        oplusTok_mem_embT (h_0 := A_0.ne) (h_1 := A_1.ne) hY' hY'ne, embBit_subset.mpr
     hsub>
```

```
  * rintro <hW, hW', hsub>
    rcases hW' with rfl | <X', hX', hX'ne, rfl> | <Y', hY', hY'ne, rfl>
    * exact Or.inl <hW, rfl>
    * rcases hW with rfl | <X, hX, hXne, rfl> | <Y, hY, hYne, rfl>
      * exact absurd (hsub nil_mem_sumTokMaster) nil_not_mem_embBit
      * exact Or.inr (Or.inl <X, X', <hX, hX', embBit_subset.mp hsub>, hXne, hX'ne,
    rfl, rfl>)
      * exact absurd hsub (fun h => embBit_not_subset_cross (by decide) (A_1.ne Y hY)
    h)
    * rcases hW with rfl | <X, hX, hXne, rfl> | <Y, hY, hYne, rfl>
      * exact absurd (hsub nil_mem_sumTokMaster) nil_not_mem_embBit
      * exact absurd hsub (fun h => embBit_not_subset_cross (by decide) (A_0.ne X hX)
    h)
      * exact Or.inr (Or.inr <Y, Y', <hY, hY', embBit_subset.mp hsub>, hYne, hY'ne,
    rfl, rfl>)

/-- **`oplusMapTok` is always strict.** -/
theorem oplusMapTok_isStrict (f_0 : ApproximableMap A_0.sys B_0.sys)
    (f_1 : ApproximableMap A_1.sys B_1.sys) : IsStrict (oplusMapTok f_0 f_1) := by
  rintro Y (<-, rfl> | <X, X', -, -, -, heq, -> | <Y0, Y', -, -, -, heq, ->)
  * rfl
  * have heq' : sumTokMaster A_0.sys A_1.sys = embBit false X := heq
    exact absurd (heq'  |>  nil_mem_sumTokMaster) nil_not_mem_embBit
  * have heq' : sumTokMaster A_0.sys A_1.sys = embBit true Y0 := heq
    exact absurd (heq'  |>  nil_mem_sumTokMaster) nil_not_mem_embBit

/-- `oplusMapTok` is monotone in both arguments. -/
theorem oplusMapTok_mono {f_0 f_0' : ApproximableMap A_0.sys B_0.sys}
    {f_1 f_1' : ApproximableMap A_1.sys B_1.sys} (h0 : f_0  <=  f_0') (h1 : f_1  <=
    f_1') :
    oplusMapTok f_0 f_1  <=  oplusMapTok f_0' f_1' := by
  rw [ApproximableMap.le_iff]
  rintro W W' (<hW, rfl> | <X, X', hrel, hXne, hX'ne, rfl, rfl> |
    <Y, Y', hrel, hYne, hY'ne, rfl, rfl>)
  * exact Or.inl <hW, rfl>
  * exact Or.inr (Or.inl <X, X', h0 X X' hrel, hXne, hX'ne, rfl, rfl>)
  * exact Or.inr (Or.inr <Y, Y', h1 Y Y' hrel, hYne, hY'ne, rfl, rfl>)

end Exercise619
end

section
namespace Exercise619

variable {D_0 D_1 : NeighborhoodSystem Str}

variable {h_0 : forall  X, D_0.mem X  ->  X.Nonempty} {h_1 : forall  Y, D_1.mem Y  ->
     Y.Nonempty}

theorem oplusTok_mem_embF_inv {W : Set Str} (h : (oplusTok D_0 D_1 h_0 h_1).mem (
    embBit false W)) :
    D_0.mem W := by
  rcases h with h0 | <X, hX, -, heq> | <Y, hY, -, heq>
  * exact absurd (h0.symm  |>  nil_mem_sumTokMaster) nil_not_mem_embBit
  * rw [embBit_injective heq]; exact hX
  * exact absurd heq.symm (embBit_ne (show (true : Bool)  !=  false by decide) (h_1 Y
```

```
    hY))

theorem oplusTok_mem_embT_inv {W : Set Str} (h : (oplusTok D_0 D_1 h_0 h_1).mem (
    embBit true W)) :
    D_1.mem W := by
  rcases h with h0 | <X, hX, -, heq> | <Y, hY, -, heq>
  * exact absurd (h0.symm  |>  nil_mem_sumTokMaster) nil_not_mem_embBit
  * exact absurd heq.symm (embBit_ne (show (false : Bool)  !=  true by decide) (h_0 X
    hX))
  * rw [embBit_injective heq]; exact hY

theorem oplusTok_mem_master : (oplusTok D_0 D_1 h_0 h_1).mem (sumTokMaster D_0 D_1) :=
     Or.inl rfl

end Exercise619
end

section
namespace Exercise619

variable {D_0 D_1 : NeighborhoodSystem Str}

variable {h_0 : forall  X, D_0.mem X  ->  X.Nonempty} {h_1 : forall  Y, D_1.mem Y  ->
     Y.Nonempty}

variable {A_0 A_1 B_0 B_1 : ScottSys}

/-- The coalesced sum carries the subsystem relation componentwise. -/
theorem oplusTok_subsystem (h0 : A_0.sys ? B_0.sys) (h1 : A_1.sys ? B_1.sys) :
    (A_0.oplus A_1).sys ? (B_0.oplus B_1).sys := by
  have heqm : sumTokMaster A_0.sys A_1.sys = sumTokMaster B_0.sys B_1.sys := by
    unfold sumTokMaster; rw [h0.master_eq, h1.master_eq]
  refine <heqm, ?_, ?_>
  * rintro W (rfl | <X, hX, hXne, rfl> | <Y, hY, hYne, rfl>)
    * exact Or.inl heqm
    * exact Or.inr (Or.inl <X, h0.sub hX, h0.master_eq  |>  hXne, rfl>)
    * exact Or.inr (Or.inr <Y, h1.sub hY, h1.master_eq  |>  hYne, rfl>)
  * rintro W W' (rfl | <X, hX, hXne, rfl> | <Y, hY, hYne, rfl>)
      (rfl | <X', hX', hX'ne, rfl> | <Y', hY', hY'ne, rfl>) hInt
    * rw [Set.inter_self]; exact Or.inl rfl
    * rw [sumTokMaster_inter_embF hX']; exact Or.inr (Or.inl <X', hX', hX'ne, rfl>)
    * rw [sumTokMaster_inter_embT hY']; exact Or.inr (Or.inr <Y', hY', hY'ne, rfl>)
    * rw [Set.inter_comm, sumTokMaster_inter_embF hX]; exact Or.inr (Or.inl <X, hX,
    hXne, rfl>)
    * rw [embBit_inter] at hInt  |-
      exact Or.inr (Or.inl <X  I  X',
        h0.inter_closed hX hX' (oplusTok_mem_embF_inv (h_0 := B_0.ne) (h_1 := B_1.ne)
    hInt),
        inter_ne_of_ne_left (A_0.sys.sub_master hX) hXne, rfl>)
    * rw [embBit_inter_ne (show (false : Bool)  !=  true by decide)] at hInt
      exact absurd ((B_0.oplus B_1).ne _ hInt) Set.not_nonempty_empty
    * rw [Set.inter_comm, sumTokMaster_inter_embT hY]; exact Or.inr (Or.inr <Y, hY,
    hYne, rfl>)
    * rw [embBit_inter_ne (show (true : Bool)  !=  false by decide)] at hInt
      exact absurd ((B_0.oplus B_1).ne _ hInt) Set.not_nonempty_empty
    * rw [embBit_inter] at hInt  |-
```

```
      exact Or.inr (Or.inr <Y  I  Y',
        h1.inter_closed hY hY' (oplusTok_mem_embT_inv (h_0 := B_0.ne) (h_1 := B_1.ne)
    hInt),
        inter_ne_of_ne_left (A_1.sys.sub_master hY) hYne, rfl>)

end Exercise619
end

section
namespace Exercise619

variable {D_0 D_1 : NeighborhoodSystem Str}

variable {h_0 : forall  X, D_0.mem X  ->  X.Nonempty} {h_1 : forall  Y, D_1.mem Y  ->
     Y.Nonempty}

variable {A_0 A_1 B_0 B_1 : ScottSys}

variable {C_0 C_1 : ScottSys}

/-- **`(g_0  comp  f_0) ? (g_1  comp  f_1) = (g_0 ? g_1)  comp  (f_0 ? f_1)`** for **
    strict** `g_0, g_1`. -/
theorem otimesMapTok_comp (f_0 : ApproximableMap A_0.sys B_0.sys) (f_1 :
    ApproximableMap A_1.sys B_1.sys)
    {g_0 : ApproximableMap B_0.sys C_0.sys} {g_1 : ApproximableMap B_1.sys C_1.sys}
    (hg_0 : IsStrict g_0) (hg_1 : IsStrict g_1) :
    otimesMapTok (g_0.comp f_0) (g_1.comp f_1) = (otimesMapTok g_0 g_1).comp (
    otimesMapTok f_0 f_1) := by
  apply ApproximableMap.ext
  intro W W''
  constructor
  * rintro (<hW, rfl> | <X, Y, X'', Y'', <X', hf0, hg0>, <Y', hf1, hg1>,
      hXne, hYne, hX''ne, hY''ne, rfl, rfl>)
    * exact <prodTokNbhd B_0.sys.master B_1.sys.master, Or.inl <hW, rfl>,
        Or.inl <(B_0.otimes B_1).sys.master_mem, rfl>>
    * have hX'ne : X'  !=  B_0.sys.master := fun h => hX''ne (hg_0 (h  |>  hg0))
      have hY'ne : Y'  !=  B_1.sys.master := fun h => hY''ne (hg_1 (h  |>  hg1))
      exact <prodTokNbhd X' Y',
        Or.inr <X, Y, X', Y', hf0, hf1, hXne, hYne, hX'ne, hY'ne, rfl, rfl>,
        Or.inr <X', Y', X'', Y'', hg0, hg1, hX'ne, hY'ne, hX''ne, hY''ne, rfl, rfl>>
  * rintro <W', hWW', hW'W''>
    rcases hWW' with <hW, rfl> | <X, Y, X', Y', hf0, hf1, hXne, hYne, hX'ne, hY'ne,
    rfl, rfl>
    * rcases hW'W'' with <-, rfl> | <Xg, Yg, X'', Y'', -, -, hXgne, -, -, -, heq, ->
      * exact Or.inl <hW, rfl>
      * have heq' : prodTokNbhd B_0.sys.master B_1.sys.master = prodTokNbhd Xg Yg :=
    heq
        obtain <h, -> := prodTokNbhd_injective heq'
        exact absurd h.symm hXgne
    * rcases hW'W'' with <-, rfl> |
        <Xg, Yg, X'', Y'', hg0, hg1, hXgne, hYgne, hX''ne, hY''ne, heq, rfl>
      * exact Or.inl <otimesTok_mem_prod (f_0.rel_dom hf0) (f_1.rel_dom hf1) hXne hYne
    , rfl>
      * obtain <rfl, rfl> := prodTokNbhd_injective heq
        exact Or.inr <X, Y, X'', Y'', <X', hf0, hg0>, <Y', hf1, hg1>,
          hXne, hYne, hX''ne, hY''ne, rfl, rfl>
```

```
/-- **`(I ? I) = I`.** -/
theorem otimesMapTok_id :
    otimesMapTok (idMap A_0.sys) (idMap A_1.sys) = idMap (A_0.otimes A_1).sys := by
  apply ApproximableMap.ext
  intro W W'
  constructor
  * rintro (<hW, rfl> | <X, Y, X', Y', <hX, hX', hsubX>, <hY, hY', hsubY>,
      hXne, hYne, hX'ne, hY'ne, rfl, rfl>)
    * exact <hW, (A_0.otimes A_1).sys.master_mem, (A_0.otimes A_1).sys.sub_master hW>
    * exact <otimesTok_mem_prod hX hY hXne hYne, otimesTok_mem_prod hX' hY' hX'ne hY'
    ne,
        prodTokNbhd_subset_iff.mpr <hsubX, hsubY>>
  * rintro <hW, hW', hsub>
    rcases hW' with rfl | <X', Y', hX', hY', hX'ne, hY'ne, rfl>
    * exact Or.inl <hW, rfl>
    * rcases hW with rfl | <X, Y, hX, hY, hXne, hYne, rfl>
      * obtain <hsubX, -> := prodTokNbhd_subset_iff.mp hsub
        exact absurd (Set.Subset.antisymm (A_0.sys.sub_master hX') hsubX) hX'ne
      * obtain <hsX, hsY> := prodTokNbhd_subset_iff.mp hsub
        exact Or.inr <X, Y, X', Y', <hX, hX', hsX>, <hY, hY', hsY>,
          hXne, hYne, hX'ne, hY'ne, rfl, rfl>

/-- **Functor law 1 -- `T(I_X) = I_{T(X)}`.** -/
theorem GExpr.map_id : (T : GExpr)  ->  (X : ScottSys)  ->  T.map (idMap X.sys) =
    idMap (T.obj X).sys
  | .const _, _ => rfl
  | .var, _ => rfl
  | .sum a b, X => by
      show sumMapTok (a.map (idMap X.sys)) (b.map (idMap X.sys)) = idMap ((a.obj X).
    sum (b.obj X)).sys
      rw [a.map_id X, b.map_id X, sumMapTok_id]
  | .prod a b, X => by
      show prodMapTok (a.map (idMap X.sys)) (b.map (idMap X.sys)) = idMap ((a.obj X).
    prod (b.obj X)).sys
      rw [a.map_id X, b.map_id X, prodMapTok_id]
  | .oplus a b, X => by
      show oplusMapTok (a.map (idMap X.sys)) (b.map (idMap X.sys))
          = idMap ((a.obj X).oplus (b.obj X)).sys
      rw [a.map_id X, b.map_id X, oplusMapTok_id]
  | .otimes a b, X => by
      show otimesMapTok (a.map (idMap X.sys)) (b.map (idMap X.sys))
          = idMap ((a.obj X).otimes (b.obj X)).sys
      rw [a.map_id X, b.map_id X, otimesMapTok_id]

/-- **`otimesMapTok` is always strict.** -/
theorem otimesMapTok_isStrict (f_0 : ApproximableMap A_0.sys B_0.sys)
    (f_1 : ApproximableMap A_1.sys B_1.sys) : IsStrict (otimesMapTok f_0 f_1) := by
  rintro Y (<-, rfl> | <X, Y0, X', Y', -, -, hXne, -, -, -, heq, ->)
  * rfl
  * have heq' : prodTokNbhd A_0.sys.master A_1.sys.master = prodTokNbhd X Y0 := heq
    obtain <h, -> := prodTokNbhd_injective heq'
    exact absurd h.symm hXne

/-- **Every `T` preserves strictness.** -/
theorem GExpr.map_isStrict : (T : GExpr)  ->  {X Y : ScottSys}  ->  (f :
    ApproximableMap X.sys Y.sys)  ->
```

```
    IsStrict f  ->  IsStrict (T.map f)
  | .const _, _, _, _, _ => isStrict_idMap
  | .var, _, _, _, hf => hf
  | .sum a b, _, _, f, _ => sumMapTok_isStrict (a.map f) (b.map f)
  | .prod a b, _, _, f, hf => prodMapTok_isStrict (a.map_isStrict f hf) (b.
    map_isStrict f hf)
  | .oplus a b, _, _, f, _ => oplusMapTok_isStrict (a.map f) (b.map f)
  | .otimes a b, _, _, f, _ => otimesMapTok_isStrict (a.map f) (b.map f)

/-- **Functor law 2 -- `T(g  comp  f) = T(g)  comp  T(f)` for strict `g`.** Together
    with `map_id`, *these are
all functors* of the strict-map category. The strictness of `g` is needed (and only)
    for the
coalesced `?`/`?`, whose composition law `oplusMapTok_comp`/`otimesMapTok_comp`
    requires it. -/
theorem GExpr.map_comp : (T : GExpr)  ->  {X Y Z : ScottSys}  ->  (f : ApproximableMap
     X.sys Y.sys)  ->
    {g : ApproximableMap Y.sys Z.sys}  ->  IsStrict g  ->  T.map (g.comp f) = (T.map g
    ).comp (T.map f)
  | .const D, _, _, _, _, _, _ => (idMap_comp (idMap D.sys)).symm
  | .var, _, _, _, _, _, _ => rfl
  | .sum a b, _, _, _, f, g, hg => by
      show sumMapTok (a.map (g.comp f)) (b.map (g.comp f))
          = (sumMapTok (a.map g) (b.map g)).comp (sumMapTok (a.map f) (b.map f))
      rw [a.map_comp f hg, b.map_comp f hg]
      * exact sumMapTok_comp _ _ _ _
  | .prod a b, _, _, _, f, g, hg => by
      show prodMapTok (a.map (g.comp f)) (b.map (g.comp f))
          = (prodMapTok (a.map g) (b.map g)).comp (prodMapTok (a.map f) (b.map f))
      rw [a.map_comp f hg, b.map_comp f hg]
      * exact prodMapTok_comp _ _ _ _
  | .oplus a b, _, _, _, f, g, hg => by
      show oplusMapTok (a.map (g.comp f)) (b.map (g.comp f))
          = (oplusMapTok (a.map g) (b.map g)).comp (oplusMapTok (a.map f) (b.map f))
      rw [a.map_comp f hg, b.map_comp f hg]
      * exact oplusMapTok_comp _ _ (a.map_isStrict g hg) (b.map_isStrict g hg)
  | .otimes a b, _, _, _, f, g, hg => by
      show otimesMapTok (a.map (g.comp f)) (b.map (g.comp f))
          = (otimesMapTok (a.map g) (b.map g)).comp (otimesMapTok (a.map f) (b.map f))
      rw [a.map_comp f hg, b.map_comp f hg]
      * exact otimesMapTok_comp _ _ (a.map_isStrict g hg) (b.map_isStrict g hg)

/-- `otimesMapTok` is monotone in both arguments. -/
theorem otimesMapTok_mono {f_0 f_0' : ApproximableMap A_0.sys B_0.sys}
    {f_1 f_1' : ApproximableMap A_1.sys B_1.sys} (h0 : f_0  <=  f_0') (h1 : f_1  <=
    f_1') :
    otimesMapTok f_0 f_1  <=  otimesMapTok f_0' f_1' := by
  rw [ApproximableMap.le_iff]
  rintro W W' (<hW, rfl> | <X, Y, X', Y', hr0, hr1, hXne, hYne, hX'ne, hY'ne, rfl, rfl
    >)
  * exact Or.inl <hW, rfl>
  * exact Or.inr <X, Y, X', Y', h0 X X' hr0, h1 Y Y' hr1, hXne, hYne, hX'ne, hY'ne,
    rfl, rfl>

/-- **`fun f. T(f)` is monotone** (the order half of *continuous on maps*). -/
theorem GExpr.map_mono : (T : GExpr)  ->  {X Y : ScottSys}  ->  {f f' :
```

```
    ApproximableMap X.sys Y.sys}  ->
    f  <=  f'  ->  T.map f  <=  T.map f'
  | .const _, _, _, _, _, _ => le_rfl
  | .var, _, _, _, _, h => h
  | .sum a b, _, _, _, _, h => sumMapTok_mono (a.map_mono h) (b.map_mono h)
  | .prod a b, _, _, _, _, h => prodMapTok_mono (a.map_mono h) (b.map_mono h)
  | .oplus a b, _, _, _, _, h => oplusMapTok_mono (a.map_mono h) (b.map_mono h)
  | .otimes a b, _, _, _, _, h => otimesMapTok_mono (a.map_mono h) (b.map_mono h)

/-- Forward direction of continuity on maps for `GExpr`. -/
theorem GExpr.map_continuous_mp : (T : GExpr)  ->  {I : Type}  ->  {X Y : ScottSys}
    ->
    {g : I  ->  ApproximableMap X.sys Y.sys}  ->  {f : ApproximableMap X.sys Y.sys}
    ->
    [Nonempty I]  ->  (forall  i j, exists  k, g i  <=  g k  /\  g j  <=  g k)  ->
    (forall  A B, f.rel A B  <->  exists  i, (g i).rel A B)  ->
    {A B : Set Str}  ->  (T.map f).rel A B  ->  exists  i, (T.map (g i)).rel A B
  | .const _, _, _, _, _, _, _, _, _, _, _, hrel => by
      obtain <i> := ?Nonempty _?; exact <i, hrel>
  | .var, _, _, _, _, _, _, hf, A, B, hrel => (hf A B).mp hrel
  | .sum a b, _, _, _, _, _, _, hdir, hf, _, _, hrel => by
      rcases hrel with <hA, rfl> | <P, P', hr, rfl, rfl> | <Q, Q', hr, rfl, rfl>
      * obtain <i> := ?Nonempty _?; exact <i, Or.inl <hA, rfl>>
      * obtain <i, hi> := a.map_continuous_mp hdir hf hr
        exact <i, Or.inr (Or.inl <P, P', hi, rfl, rfl>)>
      * obtain <i, hi> := b.map_continuous_mp hdir hf hr
        exact <i, Or.inr (Or.inr <Q, Q', hi, rfl, rfl>)>
  | .prod a b, _, _, _, _, _, _, hdir, hf, _, _, hrel => by
      obtain <P, Q, P', Q', hr0, hr1, rfl, rfl> := hrel
      obtain <i, hi> := a.map_continuous_mp hdir hf hr0
      obtain <j, hj> := b.map_continuous_mp hdir hf hr1
      obtain <k, hik, hjk> := hdir i j
      exact <k, P, Q, P', Q', (a.map_mono hik) P P' hi, (b.map_mono hjk) Q Q' hj, rfl,
     rfl>
  | .oplus a b, _, _, _, _, _, _, hdir, hf, _, _, hrel => by
      rcases hrel with <hA, rfl> | <P, P', hr, hPne, hP'ne, rfl, rfl> |
        <Q, Q', hr, hQne, hQ'ne, rfl, rfl>
      * obtain <i> := ?Nonempty _?; exact <i, Or.inl <hA, rfl>>
      * obtain <i, hi> := a.map_continuous_mp hdir hf hr
        exact <i, Or.inr (Or.inl <P, P', hi, hPne, hP'ne, rfl, rfl>)>
      * obtain <i, hi> := b.map_continuous_mp hdir hf hr
        exact <i, Or.inr (Or.inr <Q, Q', hi, hQne, hQ'ne, rfl, rfl>)>
  | .otimes a b, _, _, _, _, _, _, hdir, hf, _, _, hrel => by
      rcases hrel with <hA, rfl> | <P, Q, P', Q', hr0, hr1, hPne, hQne, hP'ne, hQ'ne,
    rfl, rfl>
      * obtain <i> := ?Nonempty _?; exact <i, Or.inl <hA, rfl>>
      * obtain <i, hi> := a.map_continuous_mp hdir hf hr0
        obtain <j, hj> := b.map_continuous_mp hdir hf hr1
        obtain <k, hik, hjk> := hdir i j
        exact <k, Or.inr <P, Q, P', Q', (a.map_mono hik) P P' hi, (b.map_mono hjk) Q Q
    ' hj,
          hPne, hQne, hP'ne, hQ'ne, rfl, rfl>>

/-- **`fun f. T(f)` is continuous on maps.** -/
theorem GExpr.map_continuous (T : GExpr) {I : Type} [Nonempty I] {X Y : ScottSys}
    (g : I  ->  ApproximableMap X.sys Y.sys) (f : ApproximableMap X.sys Y.sys)
```

```
    (hdir : forall  i j, exists  k, g i  <=  g k  /\  g j  <=  g k)
    (hf : forall  A B, f.rel A B  <->  exists  i, (g i).rel A B) (A B : Set Str) :
    (T.map f).rel A B  <->  exists  i, (T.map (g i)).rel A B := by
  refine <T.map_continuous_mp hdir hf, ?_>
  rintro <i, hi>
  have hgif : g i  <=  f := by
    rw [ApproximableMap.le_iff]; intro A' B' h; exact (hf A' B').mpr <i, h>
  exact (T.map_mono hgif) A B hi

end Exercise619
end

section
namespace Exercise619

variable {D_0 D_1 : NeighborhoodSystem Str}

variable {h_0 : forall  X, D_0.mem X  ->  X.Nonempty} {h_1 : forall  Y, D_1.mem Y  ->
    Y.Nonempty}

theorem otimesTok_mem_prod_inv {X Y : Set Str} (h : (otimesTok D_0 D_1).mem (
   prodTokNbhd X Y))
    (hX : X  !=  D_0.master) : D_0.mem X  /\  D_1.mem Y := by
  rcases h with heq | <X', Y', hX', hY', -, -, heq>
  * obtain <rfl, -> := prodTokNbhd_injective heq; exact absurd rfl hX
  * obtain <rfl, rfl> := prodTokNbhd_injective heq; exact <hX', hY'>

end Exercise619
end

section
namespace Exercise619

variable {D_0 D_1 : NeighborhoodSystem Str}

variable {h_0 : forall  X, D_0.mem X  ->  X.Nonempty} {h_1 : forall  Y, D_1.mem Y  ->
    Y.Nonempty}

variable {A_0 A_1 B_0 B_1 : ScottSys}

/-- The smash product carries the subsystem relation componentwise. -/
theorem otimesTok_subsystem (h0 : A_0.sys ? B_0.sys) (h1 : A_1.sys ? B_1.sys) :
    (A_0.otimes A_1).sys ? (B_0.otimes B_1).sys := by
  have heqm : prodTokNbhd A_0.sys.master A_1.sys.master
      = prodTokNbhd B_0.sys.master B_1.sys.master := by rw [h0.master_eq, h1.master_eq
    ]
  refine <heqm, ?_, ?_>
  * rintro W (rfl | <X, Y, hX, hY, hXne, hYne, rfl>)
    * exact Or.inl heqm
    * exact Or.inr <X, Y, h0.sub hX, h1.sub hY, h0.master_eq  |>  hXne, h1.master_eq
    |>  hYne, rfl>
  * rintro W W' (rfl | <X, Y, hX, hY, hXne, hYne, rfl>)
      (rfl | <X', Y', hX', hY', hX'ne, hY'ne, rfl>) hInt
    * rw [Set.inter_self]; exact Or.inl rfl
    * rw [prodTokNbhd_inter, Set.inter_eq_right.mpr (A_0.sys.sub_master hX'),
        Set.inter_eq_right.mpr (A_1.sys.sub_master hY')]
```

```
      exact Or.inr <X', Y', hX', hY', hX'ne, hY'ne, rfl>
    * rw [Set.inter_comm, prodTokNbhd_inter, Set.inter_eq_right.mpr (A_0.sys.
    sub_master hX),
        Set.inter_eq_right.mpr (A_1.sys.sub_master hY)]
      exact Or.inr <X, Y, hX, hY, hXne, hYne, rfl>
    * rw [prodTokNbhd_inter] at hInt  |-
      have hXne' : X  I  X'  !=  B_0.sys.master := by
        rw [ <-  h0.master_eq]; exact inter_ne_of_ne_left (A_0.sys.sub_master hX) hXne
      obtain <hmemX, hmemY> := otimesTok_mem_prod_inv hInt hXne'
      exact Or.inr <X  I  X', Y  I  Y', h0.inter_closed hX hX' hmemX, h1.inter_closed
    hY hY' hmemY,
        inter_ne_of_ne_left (A_0.sys.sub_master hX) hXne,
        inter_ne_of_ne_left (A_1.sys.sub_master hY) hYne, rfl>

end Exercise619
end

section
namespace Exercise619

variable {D_0 D_1 : NeighborhoodSystem Str}

variable {h_0 : forall  X, D_0.mem X  ->  X.Nonempty} {h_1 : forall  Y, D_1.mem Y  ->
     Y.Nonempty}

variable {A_0 A_1 B_0 B_1 : ScottSys}

variable {C_0 C_1 : ScottSys}

/-- **`fun X. T(X)` is monotone on domains.** -/
theorem GExpr.obj_subsystem : (T : GExpr)  ->  {X Y : ScottSys}  ->  X.sys ? Y.sys  ->
    (T.obj X).sys ? (T.obj Y).sys
  | .const D, _, _, _ => Subsystem.refl D.sys
  | .var, _, _, h => h
  | .sum a b, _, _, h => sumTok_subsystem (a.obj_subsystem h) (b.obj_subsystem h)
  | .prod a b, _, _, h => prodTok_subsystem (a.obj_subsystem h) (b.obj_subsystem h)
  | .oplus a b, _, _, h => oplusTok_subsystem (a.obj_subsystem h) (b.obj_subsystem h)
  | .otimes a b, _, _, h => otimesTok_subsystem (a.obj_subsystem h) (b.obj_subsystem h
    )

/-- Forward direction of continuity on domains for `GExpr`. -/
theorem GExpr.obj_continuous_mp : (T : GExpr)  ->  {? : Set ScottSys}  ->  {U :
    ScottSys}  ->
    DirectedOn (fun a b => a.sys ? b.sys) ?  ->  ?.Nonempty  ->
    (forall  D  in  ?, D.sys ? U.sys)  ->  (forall  X, U.sys.mem X  <->  exists  D  in
      ?, D.sys.mem X)  ->
    {W : Set Str}  ->  (T.obj U).sys.mem W  ->  exists  D  in  ?, (T.obj D).sys.mem W
  | .const _, _, _, _, hne, _, _, _, hmem => by
      obtain <D, hD> := hne; exact <D, hD, hmem>
  | .var, _, _, _, _, _, hU, W, hmem => (hU W).mp hmem
  | .sum a b, _, _, hdir, hne, hsub, hU, _, hmem => by
      rcases hmem with rfl | <X, hX, rfl> | <Y, hY, rfl>
      * obtain <D, hD> := hne
        exact <D, hD, Or.inl ((GExpr.sum a b).obj_subsystem (hsub D hD)).master_eq.
    symm>
      * obtain <D, hD, hXD> := a.obj_continuous_mp hdir hne hsub hU hX
```

```
        exact <D, hD, Or.inr (Or.inl <X, hXD, rfl>)>
      * obtain <D, hD, hYD> := b.obj_continuous_mp hdir hne hsub hU hY
        exact <D, hD, Or.inr (Or.inr <Y, hYD, rfl>)>
  | .prod a b, _, _, hdir, hne, hsub, hU, _, hmem => by
      obtain <X, Y, hX, hY, rfl> := hmem
      obtain <D_1, hD_1, hXD> := a.obj_continuous_mp hdir hne hsub hU hX
      obtain <D_2, hD_2, hYD> := b.obj_continuous_mp hdir hne hsub hU hY
      obtain <D_3, hD_3, hr1, hr2> := hdir D_1 hD_1 D_2 hD_2
      exact <D_3, hD_3, X, Y, (a.obj_subsystem hr1).sub hXD, (b.obj_subsystem hr2).sub
     hYD, rfl>
  | .oplus a b, _, _, hdir, hne, hsub, hU, _, hmem => by
      rcases hmem with rfl | <X, hX, hXne, rfl> | <Y, hY, hYne, rfl>
      * obtain <D, hD> := hne
        exact <D, hD, Or.inl ((GExpr.oplus a b).obj_subsystem (hsub D hD)).master_eq.
    symm>
      * obtain <D, hD, hXD> := a.obj_continuous_mp hdir hne hsub hU hX
        refine <D, hD, Or.inr (Or.inl <X, hXD, ?_, rfl>)>
        exact fun heq => hXne (heq.trans (a.obj_subsystem (hsub D hD)).master_eq)
      * obtain <D, hD, hYD> := b.obj_continuous_mp hdir hne hsub hU hY
        refine <D, hD, Or.inr (Or.inr <Y, hYD, ?_, rfl>)>
        exact fun heq => hYne (heq.trans (b.obj_subsystem (hsub D hD)).master_eq)
  | .otimes a b, _, _, hdir, hne, hsub, hU, _, hmem => by
      rcases hmem with rfl | <X, Y, hX, hY, hXne, hYne, rfl>
      * obtain <D, hD> := hne
        exact <D, hD, Or.inl ((GExpr.otimes a b).obj_subsystem (hsub D hD)).master_eq.
    symm>
      * obtain <D_1, hD_1, hXD> := a.obj_continuous_mp hdir hne hsub hU hX
        obtain <D_2, hD_2, hYD> := b.obj_continuous_mp hdir hne hsub hU hY
        obtain <D_3, hD_3, hr1, hr2> := hdir D_1 hD_1 D_2 hD_2
        refine <D_3, hD_3, Or.inr <X, Y, (a.obj_subsystem hr1).sub hXD,
          (b.obj_subsystem hr2).sub hYD, ?_, ?_, rfl>>
        * exact fun heq => hXne (heq.trans (a.obj_subsystem (hsub D_3 hD_3)).master_eq
    )
        * exact fun heq => hYne (heq.trans (b.obj_subsystem (hsub D_3 hD_3)).master_eq
    )

/-- **`fun D. T(D)` is continuous on domains.** -/
theorem GExpr.obj_continuous (T : GExpr) {? : Set ScottSys} {U : ScottSys}
    (hdir : DirectedOn (fun a b => a.sys ? b.sys) ?) (hne : ?.Nonempty)
    (hsub : forall  D  in  ?, D.sys ? U.sys) (hU : forall  X, U.sys.mem X  <->  exists
      D  in  ?, D.sys.mem X)
    (W : Set Str) : (T.obj U).sys.mem W  <->  exists  D  in  ?, (T.obj D).sys.mem W :=
     by
  refine <T.obj_continuous_mp hdir hne hsub hU, ?_>
  rintro <D, hD, hmem>
  exact (T.obj_subsystem (hsub D hD)).sub hmem

end Exercise619
end

end DomainTheory.NeighbourhoodSystems
end
```

3.7.24 Exercise 6.22 — equations for the integers

Comment on these domain equations:

$$N \cong \{\{0\}, \{0, \Lambda\}\} \oplus N,$$

$$M \cong \{\{\Lambda\}\} + M,$$

$$N^* \cong N \oplus (N \otimes N^*).$$

Exercise_6_22.lean

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Exercise_2_12
public import DomainTheory.NeighbourhoodSystems.DomainEquations.Exercise_6_21

public import DomainTheory.NeighbourhoodSystems.DomainEquations.Proposition_6_6

@[expose] public section

section
/- concordance: source L4127 EXERCISE 6.22 equations for the integers -/
-- Exercise622/Cnat.lean
namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.Exercise619

open DomainTheory.NeighbourhoodSystems.Example62 DomainTheory.NeighbourhoodSystems.
    ExampleB

namespace Exercise619

/-- **The two-point generator `{{0},{0,?}}`.** Its tokens are `Delta = {0,?}`, with the
     one proper
neighbourhood `{0}` sitting strictly below the master `{0,?} = Delta`. As a domain this
     is the chain
`{0} ? Delta` (one point above the bottom). Here `0 = [false]` and `? = []`. -/
def Cnat : ScottSys where
  sys :=
    { mem := fun X => X = {([false] : Str)}  \/  X = {([false] : Str), ([] : Str)}
      master := {([false] : Str), ([] : Str)}
      master_nonempty := <[false], Set.mem_insert _ _>
      master_mem := Or.inr rfl
      inter_mem := by
        have hAB : ({([false] : Str)} : Set Str)  subseteq  {([false] : Str), ([] :
    Str)} :=
          Set.singleton_subset_iff.mpr (Set.mem_insert _ _)
        rintro X Y Z (rfl | rfl) (rfl | rfl) _ _
        * exact Or.inl (Set.inter_self _)
        * exact Or.inl (Set.inter_eq_self_of_subset_left hAB)
        * exact Or.inl (Set.inter_eq_self_of_subset_right hAB)
        * exact Or.inr (Set.inter_self _)
```

```
      sub_master := by
        have hAB : ({([false] : Str)} : Set Str)  subseteq  {([false] : Str), ([] :
    Str)} :=
          Set.singleton_subset_iff.mpr (Set.mem_insert _ _)
        rintro X (rfl | rfl)
        * exact hAB
        * exact subset_rfl }
  ne := by
    rintro X (rfl | rfl)
    * exact <[false], rfl>
    * exact <[false], Set.mem_insert _ _>

end Exercise619

end DomainTheory.NeighbourhoodSystems
end

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.Exercise619
open DomainTheory.NeighbourhoodSystems.Example62 DomainTheory.NeighbourhoodSystems.
    ExampleB

section
namespace Exercise619

/-- **The one-point domain `{{?}} = ?`** (the terminal object of Scott's category). -/
def Cone : ScottSys := singletonSys ({([] : Str)} : Set Str) <[], rfl>

/-- `T(X) = {{?}} + X` -- the right-hand side of `M  ~=  {{?}} + M`. -/
def MExpr : GExpr := .sum (.const Cone) .var

/-- `T(X) = {{0},{0,?}} ? X` -- the right-hand side of `N  ~=  {{0},{0,?}} ? N`. -/
def NExpr : GExpr := .oplus (.const Cnat) .var

/-- `T(X) = N ? (N ? X)` -- the right-hand side of `N*  ~=  N ? (N ? N*)`, parametrised
     by the (rooted)
datum domain `N`. -/
def NStarExpr (N : ScottSys) : GExpr := .oplus (.const N) (.otimes (.const N) .var)

theorem NStarExpr_rooted {N : ScottSys} (hN : ([] : Str)  in  N.sys.master) :
    (NStarExpr N).RootedConst := <hN, hN, trivial>

/-- **`N*  ~=  N ? (N ? N*)` has a solution** for any rooted datum domain `N` (the
    strict streams over
`N`). -/
theorem NStar_eq_solution (N : ScottSys) (hN : ([] : Str)  in  N.sys.master) :
    exists  (Gamma : Set Str) (h : Gamma.Nonempty),
      (singletonSys Gamma h).sys ? ((NStarExpr N).obj (singletonSys Gamma h)).sys :=
  gExists_singleton_subsystem (NStarExpr N) (NStarExpr_rooted hN)

/-- `?  in  tok(Cnat)`, so `Cnat` is a rooted constant. -/
theorem nil_mem_Cnat : ([] : Str)  in  Cnat.sys.master := Set.mem_insert_iff.mpr (Or.
    inr rfl)

theorem NExpr_rooted : NExpr.RootedConst := <nil_mem_Cnat, trivial>
```

```
/-- **Chaining the equations.** The solution `N` to the first equation is itself a
    rooted domain
(`?  in  tok(N)`, since its token set is the fixed point `Gamma_1 ? ?`), so it is a
    legitimate datum domain
for the third: `N*` exists *over the very `N` produced by* `N  ~=  {{0},{0,?}} ? N`. -/
theorem NStar_over_N_exists :
    exists  N : ScottSys, ([] : Str)  in  N.sys.master  /\
      exists  (Gamma : Set Str) (h : Gamma.Nonempty),
        (singletonSys Gamma h).sys ? ((NStarExpr N).obj (singletonSys Gamma h)).sys :=
     by
  obtain <Gamma_1, hnil_1, _> := gExists_tok_fixedPoint NExpr NExpr_rooted
  exact <singletonSys Gamma_1 <[], hnil_1>, hnil_1, NStar_eq_solution _ hnil_1>

end Exercise619
end

end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.Exercise619
section
set_option linter.unusedVariables false

theorem example_622 : exists  (N : DomainTheory.NeighbourhoodSystems.Exercise619.
    ScottSys),
  And
    (@Membership.mem (List Bool) (Set DomainTheory.NeighbourhoodSystems.ExampleB.Str)
      (@Set.instMembership DomainTheory.NeighbourhoodSystems.ExampleB.Str)
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.master DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
        N.sys)
      (@List.nil Bool))
    (exists  (Gamma : Set DomainTheory.NeighbourhoodSystems.ExampleB.Str) (h :
      @Set.Nonempty DomainTheory.NeighbourhoodSystems.ExampleB.Str Gamma),
      @DomainTheory.NeighbourhoodSystems.Subsystem DomainTheory.NeighbourhoodSystems.
    ExampleB.Str
        (DomainTheory.NeighbourhoodSystems.Exercise619.singletonSys Gamma h).sys
        ((DomainTheory.NeighbourhoodSystems.Exercise619.NStarExpr N).obj
            (DomainTheory.NeighbourhoodSystems.Exercise619.singletonSys Gamma h)).sys)
     :=
  DomainTheory.NeighbourhoodSystems.Exercise619.NStar_over_N_exists
end

end DomainTheory.NeighbourhoodSystems.Exercise619
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.Exercise619

section
open DomainTheory.NeighbourhoodSystems.Example62 DomainTheory.NeighbourhoodSystems.
    ExampleB

namespace Exercise619

/-- **`N  ~=  {{0},{0,?}} ? N` has a solution.** There is `Gamma` (the vertical
    naturals' token set) with
`Gamma = tok(NExpr({Gamma}))`, so `{Gamma} ? NExpr({Gamma})` and Theorem 6.14 applies.
    -/
```

```
theorem N_eq_solution :
    exists  (Gamma : Set Str) (h : Gamma.Nonempty),
      (singletonSys Gamma h).sys ? (NExpr.obj (singletonSys Gamma h)).sys :=
  gExists_singleton_subsystem NExpr NExpr_rooted

/-- `?  in  tok(Cone)`. -/
theorem nil_mem_Cone : ([] : Str)  in  Cone.sys.master := rfl

theorem MExpr_rooted : MExpr.RootedConst := <nil_mem_Cone, trivial>

/-- **`M  ~=  {{?}} + M` has a solution** (the lazy naturals). -/
theorem M_eq_solution :
    exists  (Gamma : Set Str) (h : Gamma.Nonempty),
      (singletonSys Gamma h).sys ? (MExpr.obj (singletonSys Gamma h)).sys :=
  gExists_singleton_subsystem MExpr MExpr_rooted

end Exercise619
end

section
open DomainTheory.NeighbourhoodSystems.Example62 DomainTheory.NeighbourhoodSystems.
    ExampleB DomainTheory.NeighbourhoodSystems.Exercise510

namespace Exercise619

/-- **The fixed-point token set `Gamma = tok(T({Gamma}))`**, as the explicit Kleene
    union `?_n gIter T n`
(no `Classical.choice`). -/
def gFix (T : GExpr) : Set Str := ? n, gIter T n

/-- `Gamma = tok(T({Gamma}))` at the token level: `gFun T Gamma = Gamma`. -/
theorem gFix_fixed (T : GExpr) (hT : T.RootedConst) : gFun T (gFix T) = gFix T :=
  gFun_iter_fixed T hT

theorem gFix_nil_mem (T : GExpr) : ([] : Str)  in  gFix T :=
  Set.mem_iUnion.mpr <0, rfl>

theorem gFix_nonempty (T : GExpr) : (gFix T).Nonempty := <[], gFix_nil_mem T>

/-- **The one-point generator `{Gamma}`** as an object of the category. -/
def gGen (T : GExpr) : ScottSys := singletonSys (gFix T) (gFix_nonempty T)

@[simp] theorem gGen_master (T : GExpr) : (gGen T).sys.master = gFix T := rfl

/-- **`{Gamma} ? T({Gamma})`** -- Scott's hypothesis for Theorem 6.14, the base of the
    tower. (This is the
content of `gExists_singleton_subsystem`, here at the *explicit* generator `Gamma =
    gFix T`.) -/
theorem gBase (T : GExpr) (hT : T.RootedConst) : (gGen T).sys ? (T.obj (gGen T)).sys
    := by
  have hmaster : (T.obj (gGen T)).sys.master = gFix T :=
    (gFun_eq_master T (gFix_nonempty T)).symm.trans (gFix_fixed T hT)
  refine <hmaster.symm, ?_, ?_>
  * intro X hX
    have heq : X = (T.obj (gGen T)).sys.master := (hX : X = gFix T).trans hmaster.symm
    rw [heq]; exact (T.obj (gGen T)).sys.master_mem
```

```
  * intro X Y hX hY _
    show X  I  Y = gFix T
    rw [show X = gFix T from hX, show Y = gFix T from hY, Set.inter_self]

/-- **The tower `T?({Gamma})`** of `{}`-free systems over `Str`: `T?({Gamma}) = {Gamma
    }`, `T?^+1({Gamma}) =
T(T?({Gamma}))`. -/
def gTower (T : GExpr) : Nat  ->  ScottSys
  | 0 => gGen T
  | n + 1 => T.obj (gTower T n)

@[simp] theorem gTower_zero (T : GExpr) : gTower T 0 = gGen T := rfl

@[simp] theorem gTower_succ (T : GExpr) (n : Nat) : gTower T (n + 1) = T.obj (gTower T
     n) := rfl

/-- **The basic chain step `T?({Gamma}) ? T?^+1({Gamma})`.** Base: `gBase`. Step: `T`
    is monotone on domains
(`obj_subsystem`). -/
theorem gChain (T : GExpr) (hT : T.RootedConst) :
    forall  n, (gTower T n).sys ? (gTower T (n + 1)).sys
  | 0 => gBase T hT
  | n + 1 => T.obj_subsystem (gChain T hT n)

/-- The tower is a `?`-chain: `T?({Gamma}) ? T?({Gamma})` whenever `n  <=  m`. -/
theorem gTower_le (T : GExpr) (hT : T.RootedConst) {n m : Nat} (h : n  <=  m) :
    (gTower T n).sys ? (gTower T m).sys := by
  induction h with
  | refl => exact Subsystem.refl _
  | step _ ih => exact ih.trans (gChain T hT _)

/-- Every level of the tower has the same master `Delta = Gamma`. -/
theorem gTower_master (T : GExpr) (hT : T.RootedConst) :
    forall  n, (gTower T n).sys.master = gFix T
  | 0 => rfl
  | n + 1 => ((gChain T hT n).master_eq).symm.trans (gTower_master T hT n)

/-- **The colimit system `? = ?_n T?({Gamma})`** as an `{}`-free system over `Str`. A
    set is a
neighbourhood exactly when it is a neighbourhood of some level; closure under
    consistent intersection
uses that the tower is a chain (any finite collection sits inside one level). -/
def gColim (T : GExpr) (hT : T.RootedConst) : ScottSys where
  sys :=
    { mem := fun X => exists  n, (gTower T n).sys.mem X
      master := gFix T
      master_nonempty := (gTower T 0).sys.master_nonempty
      master_mem := <0, (gTower T 0).sys.master_mem>
      inter_mem := by
        rintro X Y Z <n, hX> <m, hY> <p, hZ> hsub
        set N := max n (max m p) with hN
        have hXN : (gTower T N).sys.mem X := (gTower_le T hT (le_max_left n _)).sub hX
        have hYN : (gTower T N).sys.mem Y :=
          (gTower_le T hT ((le_max_left m p).trans (le_max_right n _))).sub hY
        have hZN : (gTower T N).sys.mem Z :=
          (gTower_le T hT ((le_max_right m p).trans (le_max_right n _))).sub hZ
```

```
        exact <N, (gTower T N).sys.inter_mem hXN hYN hZN hsub>
      sub_master := by
        rintro X <n, hX>
        rw [ <-  gTower_master T hT n]
        exact (gTower T n).sys.sub_master hX }
  ne := by rintro X <n, hX>; exact (gTower T n).ne X hX

@[simp] theorem mem_gColim (T : GExpr) (hT : T.RootedConst) {X : Set Str} :
    (gColim T hT).sys.mem X  <->  exists  n, (gTower T n).sys.mem X := Iff.rfl

@[simp] theorem gColim_master (T : GExpr) (hT : T.RootedConst) :
    (gColim T hT).sys.master = gFix T := rfl

/-- **The functor `T(X) = N ? ((X x X) + (X x X))`** of Exercise 6.23, as a `GExpr`
    over the variable
domain `N`. The `? N` carries the variables, and the two `(X x X)` summands (combined
    by `+`) carry the
two binary operation symbols. -/
def Texp (N : ScottSys) : GExpr :=
  .oplus (.const N) (.sum (.prod .var .var) (.prod .var .var))

/-- `Texp N` is rooted iff the variable domain `N` is (`?  in  tok(N)`, automatic for
    the fixed-point
solutions of 6.19-6.22). -/
theorem Texp_rooted {N : ScottSys} (hN : ([] : Str)  in  N.sys.master) : (Texp N).
    RootedConst :=
  <hN, <trivial, trivial>, <trivial, trivial>>

/-- **The syntactic domain of expressions** `Exp = ?_n Texp?({Gamma})`, the initial
    solution of
`Exp  ~=  N ? ((Exp x Exp)+(Exp x Exp))`. -/
def Exp (N : ScottSys) (hN : ([] : Str)  in  N.sys.master) : ScottSys :=
  gColim (Texp N) (Texp_rooted hN)

/-- Two objects of the category with the same underlying system are equal (the `{}`-
    freeness field is
a `Prop`). -/
theorem ScottSys.ext {A B : ScottSys} (h : A.sys = B.sys) : A = B := by
  cases A; cases B; cases h; rfl

/-- **The category of `{}`-free domains over `Str` and strict maps.** Morphisms are
    strict approximable
maps (`StrictMap`); identities and associative composition are Theorem 2.5, with
    strictness preserved
by `isStrict_idMap`/`isStrict_comp`. The fixed carrier `Str` is what removes all the
    carrier-transport
`HEq` that burdens the abstract `Endofunctor DomainObj`. -/
instance : Category ScottSys where
  Hom A B := StrictMap A.sys B.sys
  id A := <idMap A.sys, isStrict_idMap>
  comp f g := <g.1.comp f.1, isStrict_comp g.2 f.2>
  id_comp f := Subtype.ext (comp_idMap f.1)
  comp_id f := Subtype.ext (idMap_comp f.1)
  assoc f g h := Subtype.ext (comp_assoc h.1 g.1 f.1).symm

/-- Each level of the tower is a subdomain of the colimit: `T?({Gamma}) ? ?`. -/
```

```
theorem gTower_sub_colim (T : GExpr) (hT : T.RootedConst) (n : Nat) :
    (gTower T n).sys ? (gColim T hT).sys where
  master_eq := by rw [gColim_master, gTower_master T hT]
  sub hX := <n, hX>
  inter_closed := by
    rintro X Y hX hY <m, hXY>
    have hN : (gTower T (max n m)).sys.mem (X  I  Y) :=
      (gTower_le T hT (le_max_right n m)).sub hXY
    exact (gTower_le T hT (le_max_left n m)).inter_closed hX hY hN

/-- **`T(?) = ?` at the level of neighbourhood systems.** Membership: continuity on
    domains
(`obj_continuous`) along the directed tower turns `T(?_n T?({Gamma}))` into `?_n T
    ?^+1({Gamma})`, which has the
same neighbourhoods as `?_n T?({Gamma})` (the extra `n=0` level `T?({Gamma})` is
    absorbed by the chain step).
Master: both are `Gamma` (`gTower_master` through `obj_subsystem` of `T?({Gamma}) ? ?`)
    . -/
theorem gColim_obj_sys_eq (T : GExpr) (hT : T.RootedConst) :
    (T.obj (gColim T hT)).sys = (gColim T hT).sys := by
  set ? : Set ScottSys := Set.range (gTower T) with h?
  have hne : ?.Nonempty := <gTower T 0, 0, rfl>
  have hsub : forall  D  in  ?, D.sys ? (gColim T hT).sys := by
    rintro D <n, rfl>; exact gTower_sub_colim T hT n
  have hdir : DirectedOn (fun a b => a.sys ? b.sys) ? := by
    rintro _ <n, rfl> _ <m, rfl>
    exact <gTower T (max n m), <max n m, rfl>,
      gTower_le T hT (le_max_left n m), gTower_le T hT (le_max_right n m)>
  have hU : forall  X, (gColim T hT).sys.mem X  <->  exists  D  in  ?, D.sys.mem X :=
    by
    intro X; constructor
    * rintro <n, hn>; exact <gTower T n, <n, rfl>, hn>
    * rintro <D, <n, rfl>, hn>; exact <n, hn>
  apply NeighborhoodSystem.ext
  * intro W
    rw [T.obj_continuous hdir hne hsub hU W]
    constructor
    * rintro <D, <n, rfl>, hn>
      -- `T(T?({Gamma})) = T?^+1({Gamma})`, a level of the colimit.
      exact <n + 1, hn>
    * rintro <n, hn>
      -- a colimit neighbourhood at level `n` is, after one chain step, at `T(T?({Gamma
    }))`.
      exact <gTower T n, <n, rfl>, (gChain T hT n).sub hn>
  * -- masters: `(T ?).master = (T?^+1({Gamma})).master = Gamma = ?.master`, via `
    obj_subsystem` at `n=0`.
    have h := (T.obj_subsystem (gTower_sub_colim T hT 0)).master_eq
    rw [gColim_master]
    rw [show (T.obj (gTower T 0)) = gTower T 1 from rfl] at h
    rw [ <-  h, gTower_master T hT]

/-- **The structure isomorphism `T(?)  ~=  ?` is the identity** (the two objects are
    literally equal). -/
theorem gColim_obj_eq (T : GExpr) (hT : T.RootedConst) : T.obj (gColim T hT) = gColim
    T hT :=
  ScottSys.ext (gColim_obj_sys_eq T hT)
```

```
/-- **The domain equation `Exp  ~=  N ? ((Exp x Exp)+(Exp x Exp))`**, realised as an
    equality of systems
(the structure map is the identity). This is the "construe the initial solution" half
    of
Exercise 6.23. -/
theorem Exp_structure_eq (N : ScottSys) (hN : ([] : Str)  in  N.sys.master) :
    (Texp N).obj (Exp N hN) = Exp N hN :=
  gColim_obj_eq (Texp N) (Texp_rooted hN)

@[simp] theorem ScottSys.id_val (A : ScottSys) :
    ((? A) : StrictMap A.sys A.sys).1 = idMap A.sys := rfl

@[simp] theorem ScottSys.comp_val {A B C : ScottSys} (g : (B  -->  C)) (f : (A  -->  B
    )) :
    ((g ? f : StrictMap A.sys C.sys)).1 = g.1.comp f.1 := rfl

/-- The morphism action of `gFunctor T`: a strict `f` is sent to the strict map `T(f)`.
     (Typed via
`StrictMap`, which is defeq to the category's `Hom`; this avoids the class-projection
    that blocks the
anonymous `.1` on `Category.Hom`.) -/
def gFunctorMap (T : GExpr) {X Y : ScottSys} (f : StrictMap X.sys Y.sys) :
    StrictMap (T.obj X).sys (T.obj Y).sys :=
  <T.map f.1, T.map_isStrict f.1 f.2>

/-- **Every `GExpr` is an `Endofunctor` of the strict-map category.** On objects it is
    `GExpr.obj`;
on a strict map `f` it is the strict map `T(f)` (`GExpr.map_isStrict`). Functoriality
    is
`GExpr.map_id` and `GExpr.map_comp` (the latter needs `g` strict -- automatic here,
    since every
morphism of this category is strict). -/
abbrev gFunctor (T : GExpr) : Endofunctor ScottSys where
  obj := T.obj
  map := gFunctorMap T
  map_id X := Subtype.ext (T.map_id X)
  map_comp {_ _ _} g f :=
    Subtype.ext (T.map_comp (f : StrictMap _ _).1 (g : StrictMap _ _).2)

@[simp] theorem gFunctor_obj (T : GExpr) (X : ScottSys) : (gFunctor T).obj X = T.obj X
     := rfl

@[simp] theorem gFunctorMap_val (T : GExpr) {X Y : ScottSys} (f : StrictMap X.sys Y.
    sys) :
    (gFunctorMap T f).1 = T.map f.1 := rfl

/-- **The endofunctor `T(X) = N ? ((X x X) + (X x X))`** of Exercise 6.23. -/
abbrev TexpF (N : ScottSys) : Endofunctor ScottSys := gFunctor (Texp N)

/-- The structure map of an algebra `B`, as a raw approximable map (its strictness is `
    algStr_strict`).
The ascription to `StrictMap` forces the categorical `Hom` to its underlying subtype.
    -/
def algStr (B : TAlgebra (TexpF N)) :
    ApproximableMap ((TexpF N).obj B.carrier).sys B.carrier.sys :=
```

```
  (B.str : StrictMap ((TexpF N).obj B.carrier).sys B.carrier.sys).1

theorem algStr_strict (B : TAlgebra (TexpF N)) : IsStrict (algStr B) :=
  (B.str : StrictMap ((TexpF N).obj B.carrier).sys B.carrier.sys).2

/-- The identity isomorphism in any category induced by an object equality. -/
def isoOfObjEq {Obj : Type*} [Category Obj] {X Y : Obj} (h : X = Y) : Iso X Y := by
  cases h
  exact <(? X), (? X), CategoryTheory.Category.id_comp _, CategoryTheory.Category.
    id_comp _>

/-- **The structure isomorphism `T(Exp)  ~=  Exp`.** Since Phase 1 proved `T(Exp) = Exp
    ` as objects
(`Exp_structure_eq`), this is the identity isomorphism. -/
def ExpIso (N : ScottSys) (hN : ([] : Str)  in  N.sys.master) :
    Iso ((TexpF N).obj (Exp N hN)) (Exp N hN) :=
  isoOfObjEq (Exp_structure_eq N hN)

/-- **`Exp` as a `T`-algebra** with structure map the isomorphism `T(Exp)  ~=  Exp` (
    the identity, since
`T(Exp) = Exp`). This realises Scott's "construe the initial solution as a syntactic
    domain of
expressions": `Exp` is an algebra of `T(X) = N ? ((X x X)+(X x X))`. -/
abbrev ExpAlg (N : ScottSys) (hN : ([] : Str)  in  N.sys.master) : TAlgebra (TexpF N)
    where
  carrier := Exp N hN
  str := (ExpIso N hN).hom

/-- The structure map `i : T(Exp)  ->  Exp` as a raw map (the identity, since `T(Exp) =
     Exp`). -/
def expHom (N : ScottSys) (hN : ([] : Str)  in  N.sys.master) :
    ApproximableMap ((TexpF N).obj (Exp N hN)).sys (Exp N hN).sys :=

  ((ExpIso N hN).hom : StrictMap ((TexpF N).obj (Exp N hN)).sys (Exp N hN).sys).1

/-- The inverse `j = i^-1 : Exp  ->  T(Exp)` of the structure isomorphism, as a raw map
    . -/
def expInv (N : ScottSys) (hN : ([] : Str)  in  N.sys.master) :
    ApproximableMap (Exp N hN).sys ((TexpF N).obj (Exp N hN)).sys :=
  ((ExpIso N hN).inv : StrictMap (Exp N hN).sys ((TexpF N).obj (Exp N hN)).sys).1

section Existence

variable {N : ScottSys} (hN : ([] : Str)  in  N.sys.master) (B : TAlgebra (TexpF N))

/-- **The Kleene iterates `val_n : Exp  ->  D`** of the operator `fun h. k  comp  T(h)
     comp  j`. `val_0 = False`,
`val_n_+_1 = k  comp  T(val_n)  comp  j`. -/
def descRel : Nat  ->  ApproximableMap (Exp N hN).sys B.carrier.sys
  | 0 => constMap (Exp N hN).sys B.carrier.sys.bot
  | n + 1 => (algStr B).comp (((Texp N).map (descRel n)).comp (expInv N hN))

@[simp] theorem descRel_succ (n : Nat) :
    descRel hN B (n + 1) = (algStr B).comp (((Texp N).map (descRel hN B n)).comp (
    expInv N hN)) :=
  rfl
```

```
end Existence

end Exercise619
end

section
open DomainTheory.NeighbourhoodSystems.Example62 DomainTheory.NeighbourhoodSystems.
    ExampleB DomainTheory.NeighbourhoodSystems.Exercise510

namespace Exercise619

variable {A_0 A_1 B_0 B_1 : ScottSys}

section Uniqueness

variable {N : ScottSys} (hN : ([] : Str)  in  N.sys.master)

variable (B : TAlgebra (TexpF N))

/-- Two algebra homomorphisms with equal underlying maps are equal. -/
theorem algHom_ext {A C : TAlgebra (TexpF N)} {g g' : AlgHom A C} (h : g.hom = g'.hom)
     : g = g' := by
  cases g; cases g'; cases h; rfl

end Uniqueness

end Exercise619
end

section
open DomainTheory.NeighbourhoodSystems.Example62 DomainTheory.NeighbourhoodSystems.
    ExampleB DomainTheory.NeighbourhoodSystems.Exercise510

namespace Exercise619

section Existence

variable {N : ScottSys} (hN : ([] : Str)  in  N.sys.master) (B : TAlgebra (TexpF N))

/-- The constant `False` map is below every approximable map (it relates each domain
    neighbourhood only
to the codomain master, which every map produces by monotonicity from `master_rel`). -/
theorem constBot_le {a b : Type*} {V_0 : NeighborhoodSystem a} {V_1 :
    NeighborhoodSystem b}
    (g : ApproximableMap V_0 V_1) : constMap V_0 V_1.bot  <=  g := by
  intro X Y hr
  obtain <hX, hY> := hr
  rw [NeighborhoodSystem.mem_bot] at hY
  subst hY
  exact g.mono g.master_rel (V_0.sub_master hX) subset_rfl hX V_1.master_mem

/-- The iterates increase: `val_n  <=  val_n_+_1`. -/
theorem descRel_le_succ : forall  n, descRel hN B n  <=  descRel hN B (n + 1)
  | 0 => constBot_le _
  | n + 1 => by
```

```
      rw [descRel_succ, descRel_succ]
      exact comp_mono_gen le_rfl
        (comp_mono_gen ((Texp N).map_mono (descRel_le_succ n)) le_rfl)

/-- The iterates form a ` <= `-chain. -/
theorem descRel_mono {i j : Nat} (h : i  <=  j) : descRel hN B i  <=  descRel hN B j
    := by
  induction h with
  | refl => exact le_rfl
  | step _ ih => exact ih.trans (descRel_le_succ hN B _)

/-- Directedness witness for the union (any two iterates are dominated by the later
    one). -/
theorem descDir (i j : Nat) : exists  k, (forall  X Y, (descRel hN B i).rel X Y  ->  (
    descRel hN B k).rel X Y)  /\
    (forall  X Y, (descRel hN B j).rel X Y  ->  (descRel hN B k).rel X Y) :=
  <max i j, descRel_mono hN B (le_max_left i j), descRel_mono hN B (le_max_right i j)>

/-- Directedness of the iterates in ` <= `-form (for `map_continuous`). -/
theorem descDirLe (i j : Nat) :
    exists  k, descRel hN B i  <=  descRel hN B k  /\  descRel hN B j  <=  descRel hN
    B k :=
  <max i j, descRel_mono hN B (le_max_left i j), descRel_mono hN B (le_max_right i j)>

/-- **The evaluation map `val = ?_n val_n`** as an approximable map. -/
def descMap : ApproximableMap (Exp N hN).sys B.carrier.sys :=
  iSupMap (descRel hN B) (descDir hN B)

/-- `val` is the relational union of the iterates (the hypothesis for `map_continuous`)
    . -/
theorem descMap_is_sup (A E : Set Str) :
    (descMap hN B).rel A E  <->  exists  n, (descRel hN B n).rel A E := Iff.rfl

/-- **The fixed-point equation `val = k  comp  T(val)  comp  j`.** Forward: an iterate
    `val_n` is, after the
recursion, `k  comp  T(val_n?_1)  comp  j`, and `T(val_n?_1)  subseteq  T(val)` by
    continuity on maps. Backward: a witness
factoring through `T(val_n)` lands in `val_n_+_1`. -/
theorem descMap_fix :
    descMap hN B = (algStr B).comp (((Texp N).map (descMap hN B)).comp (expInv N hN))
    := by
  have hmc : forall  Y C, ((Texp N).map (descMap hN B)).rel Y C
       <->  exists  n, ((Texp N).map (descRel hN B n)).rel Y C :=
    fun Y C => (Texp N).map_continuous (descRel hN B) (descMap hN B) (descDirLe hN B)
      (descMap_is_sup hN B) Y C
  apply ApproximableMap.ext
  intro A E
  rw [comp_rel]
  constructor
  * rintro <n, hn>
    have hn1 : (descRel hN B (n + 1)).rel A E := descRel_le_succ hN B n A E hn
    rw [descRel_succ, comp_rel] at hn1
    obtain <C, hAC, hCE> := hn1
    rw [comp_rel] at hAC
    obtain <Y, hAY, hYC> := hAC
    exact <C, <Y, hAY, (hmc Y C).mpr <n, hYC>>, hCE>
```

```
  * rintro <C, hAC, hCE>
    rw [comp_rel] at hAC
    obtain <Y, hAY, hYC> := hAC
    obtain <n, hn> := (hmc Y C).mp hYC
    refine <n + 1, ?_>
    rw [descRel_succ, comp_rel]
    exact <C, by rw [comp_rel]; exact <Y, hAY, hn>, hCE>

theorem descMap_rel {A E : Set Str} :
    (descMap hN B).rel A E  <->  exists  n, (descRel hN B n).rel A E := Iff.rfl

end Existence

/-- `j  comp  i = I_{T(Exp)}` at the raw level (from the iso's `hom_inv_id`). -/
theorem expInv_comp_expHom (N : ScottSys) (hN : ([] : Str)  in  N.sys.master) :
    (expInv N hN).comp (expHom N hN) = idMap ((TexpF N).obj (Exp N hN)).sys := by
  have h := congrArg (Subtype.val) (ExpIso N hN).hom_inv_id
  exact h

section Existence

variable {N : ScottSys} (hN : ([] : Str)  in  N.sys.master) (B : TAlgebra (TexpF N))

/-- **The homomorphism square `val  comp  i = k  comp  T(val)`** at the raw level (
    conjugating the fixed-point
equation by `i`, using `j  comp  i = I`). -/
theorem descComm :
    (descMap hN B).comp (expHom N hN) = (algStr B).comp ((Texp N).map (descMap hN B))
    := by
  calc (descMap hN B).comp (expHom N hN)
      = ((algStr B).comp (((Texp N).map (descMap hN B)).comp (expInv N hN))).comp (
    expHom N hN) := by
        rw [ <-  descMap_fix]
    _ = (algStr B).comp ((((Texp N).map (descMap hN B)).comp (expInv N hN)).comp (
    expHom N hN)) := by
        rw [comp_assoc]
    _ = (algStr B).comp (((Texp N).map (descMap hN B)).comp ((expInv N hN).comp (
    expHom N hN))) := by
        rw [comp_assoc]
    _ = (algStr B).comp (((Texp N).map (descMap hN B)).comp (idMap _)) := by
        rw [expInv_comp_expHom]
    _ = (algStr B).comp ((Texp N).map (descMap hN B)) := by rw [comp_idMap]

end Existence

theorem expInv_strict (N : ScottSys) (hN : ([] : Str)  in  N.sys.master) : IsStrict (
    expInv N hN) :=
  ((ExpIso N hN).inv : StrictMap (Exp N hN).sys ((TexpF N).obj (Exp N hN)).sys).2

section Existence

variable {N : ScottSys} (hN : ([] : Str)  in  N.sys.master) (B : TAlgebra (TexpF N))

/-- Every iterate is strict. -/
theorem descRel_isStrict : forall  n, IsStrict (descRel hN B n)
  | 0 => isStrict_constBot
```

```
  | n + 1 => by
      rw [descRel_succ]
      exact isStrict_comp (algStr_strict B)
        (isStrict_comp ((Texp N).map_isStrict _ (descRel_isStrict n)) (expInv_strict N
    hN))

/-- `val` is strict (a union of strict maps). -/
theorem descMap_isStrict : IsStrict (descMap hN B) := by
  rintro Y <n, hn>
  exact descRel_isStrict hN B n hn

/-- **The evaluation homomorphism `val(s) : Exp  ->  D`** as a `T`-algebra homomorphism
    -- Scott's
existence of the evaluation map. -/
def descAlgHom : AlgHom (ExpAlg N hN) B where
  hom := <descMap hN B, descMap_isStrict hN B>
  comm := by
    apply Subtype.ext
    show (descMap hN B).comp (expHom N hN) = (algStr B).comp ((Texp N).map (descMap hN
    B))
    exact descComm hN B

end Existence

end Exercise619
end

section
open DomainTheory.NeighbourhoodSystems.Example62 DomainTheory.NeighbourhoodSystems.
   ExampleB DomainTheory.NeighbourhoodSystems.Exercise510

namespace Exercise619

variable {A_0 A_1 B_0 B_1 : ScottSys}

section Uniqueness

variable {N : ScottSys} (hN : ([] : Str)  in  N.sys.master)

/-- The subdomain `Texp?({Gamma}) ? Exp` (Proposition 6.12's pair lives here). -/
theorem expSub (n : Nat) : (gTower (Texp N) n).sys ? (Exp N hN).sys :=
  gTower_sub_colim (Texp N) (Texp_rooted hN) n

end Uniqueness

end Exercise619
end

section
open DomainTheory.NeighbourhoodSystems.Example62 DomainTheory.NeighbourhoodSystems.
   ExampleB DomainTheory.NeighbourhoodSystems.Exercise510

namespace Exercise619

/-- The injection `i : D  ->  E` of a subsystem is **strict**: `i` sends `Delta_D` only
    to `Delta_E`. -/
```

```
theorem Subsystem.inj_isStrict {a : Type*} {D E : NeighborhoodSystem a} (h : D ? E) :
    IsStrict h.inj := by
  intro Y hrel
  rw [Subsystem.inj_rel] at hrel
  obtain <_, hYE, hsub> := hrel
  exact Set.Subset.antisymm (E.sub_master hYE) (by rw [ <-  h.master_eq]; exact hsub)

end Exercise619
end

section
open DomainTheory.NeighbourhoodSystems.Example62 DomainTheory.NeighbourhoodSystems.
    ExampleB DomainTheory.NeighbourhoodSystems.Exercise510

namespace Exercise619

variable {A_0 A_1 B_0 B_1 : ScottSys}

/-- The forward map of the identity iso `isoOfObjEq e` is the inclusion `X ? Y` (= `
    idMap` across the
object equality `e`). -/
theorem isoOfObjEq_hom_rel {X Y : ScottSys} (e : X = Y) {A E : Set Str} :
    ((isoOfObjEq e).hom).1.rel A E  <->  X.sys.mem A  /\  Y.sys.mem E  /\  A  subseteq
      E := by
  cases e; exact idMap_rel

/-- **Relational description of the structure map `i = expHom`** (the identity `T(Exp)
    = Exp`). -/
theorem expHom_rel {N : ScottSys} (hN : ([] : Str)  in  N.sys.master) {A E : Set Str}
    :
    (expHom N hN).rel A E  <->
      ((Texp N).obj (Exp N hN)).sys.mem A  /\  (Exp N hN).sys.mem E  /\  A  subseteq
    E :=
  isoOfObjEq_hom_rel (Exp_structure_eq N hN)

/-- The inverse map of the identity iso `isoOfObjEq e`. -/
theorem isoOfObjEq_inv_rel {X Y : ScottSys} (e : X = Y) {A E : Set Str} :
    ((isoOfObjEq e).inv).1.rel A E  <->  Y.sys.mem A  /\  X.sys.mem E  /\  A  subseteq
      E := by
  cases e; exact idMap_rel

/-- **Relational description of the inverse structure map `j = expInv`**. -/
theorem expInv_rel {N : ScottSys} (hN : ([] : Str)  in  N.sys.master) {A E : Set Str}
    :
    (expInv N hN).rel A E  <->
      (Exp N hN).sys.mem A  /\  ((Texp N).obj (Exp N hN)).sys.mem E  /\  A  subseteq
    E :=
  isoOfObjEq_inv_rel (Exp_structure_eq N hN)

/-- **Coalesced sum carries the injection.** -/
theorem oplusMapTok_inj (h0 : A_0.sys ? B_0.sys) (h1 : A_1.sys ? B_1.sys) :
    oplusMapTok h0.inj h1.inj = (oplusTok_subsystem h0 h1).inj := by
  have hsubM : forall  {W : Set Str}, (A_0.oplus A_1).sys.mem W  ->  W  subseteq
    sumTokMaster B_0.sys B_1.sys := by
    rintro W (rfl | <X, hX, hXne, rfl> | <Y, hY, hYne, rfl>)
    * exact (show sumTokMaster A_0.sys A_1.sys = sumTokMaster B_0.sys B_1.sys by
```

```
        unfold sumTokMaster; rw [h0.master_eq, h1.master_eq]).subset
    * exact embF_subset_sumTokMaster (h0.sub hX)
    * exact embT_subset_sumTokMaster (h1.sub hY)
  apply ApproximableMap.ext
  intro W W'
  rw [Subsystem.inj_rel]
  constructor
  * rintro (<hW, rfl> | <X, X', <hX, hX', hXs>, hXne, hX'ne, rfl, rfl> |
      <Y, Y', <hY, hY', hYs>, hYne, hY'ne, rfl, rfl>)
    * exact <hW, (B_0.oplus B_1).sys.master_mem, hsubM hW>
    * exact <Or.inr (Or.inl <X, hX, hXne, rfl>), Or.inr (Or.inl <X', hX', hX'ne, rfl>)
    ,
        embBit_subset.mpr hXs>
    * exact <Or.inr (Or.inr <Y, hY, hYne, rfl>), Or.inr (Or.inr <Y', hY', hY'ne, rfl>)
    ,
        embBit_subset.mpr hYs>
  * rintro <hW, hW', hsub>
    rcases hW' with rfl | <X', hX', hX'ne, rfl> | <Y', hY', hY'ne, rfl>
    * exact Or.inl <hW, rfl>
    * rcases hW with rfl | <X, hX, hXne, rfl> | <Y, hY, hYne, rfl>
      * exact absurd (hsub nil_mem_sumTokMaster) nil_not_mem_embBit
      * exact Or.inr (Or.inl <X, X', <hX, hX', embBit_subset.mp hsub>, hXne, hX'ne,
    rfl, rfl>)
      * exact absurd hsub (fun h => embBit_not_subset_cross (by decide) (A_1.ne Y hY)
    h)
    * rcases hW with rfl | <X, hX, hXne, rfl> | <Y, hY, hYne, rfl>
      * exact absurd (hsub nil_mem_sumTokMaster) nil_not_mem_embBit
      * exact absurd hsub (fun h => embBit_not_subset_cross (by decide) (A_0.ne X hX)
    h)
      * exact Or.inr (Or.inr <Y, Y', <hY, hY', embBit_subset.mp hsub>, hYne, hY'ne,
    rfl, rfl>)

/-- **Coalesced sum carries the projection.** -/
theorem oplusMapTok_proj (h0 : A_0.sys ? B_0.sys) (h1 : A_1.sys ? B_1.sys) :
    oplusMapTok h0.proj h1.proj = (oplusTok_subsystem h0 h1).proj := by
  have hsubM : forall  {W : Set Str}, (B_0.oplus B_1).sys.mem W  ->  W  subseteq
    sumTokMaster A_0.sys A_1.sys := by
    rintro W (rfl | <X, hX, hXne, rfl> | <Y, hY, hYne, rfl>)
    * exact (show sumTokMaster B_0.sys B_1.sys = sumTokMaster A_0.sys A_1.sys by
        unfold sumTokMaster; rw [h0.master_eq, h1.master_eq]).subset
    * exact (embBit_subset.mpr (by rw [h0.master_eq]; exact B_0.sys.sub_master hX)).
    trans
        (embF_subset_sumTokMaster A_0.sys.master_mem)
    * exact (embBit_subset.mpr (by rw [h1.master_eq]; exact B_1.sys.sub_master hY)).
    trans
        (embT_subset_sumTokMaster A_1.sys.master_mem)
  apply ApproximableMap.ext
  intro W W'
  rw [Subsystem.proj_rel]
  constructor
  * rintro (<hW, rfl> | <X, X', <hX, hX', hXs>, hXne, hX'ne, rfl, rfl> |
      <Y, Y', <hY, hY', hYs>, hYne, hY'ne, rfl, rfl>)
    * exact <hW, (A_0.oplus A_1).sys.master_mem, hsubM hW>
    * exact <Or.inr (Or.inl <X, hX, hXne, rfl>), Or.inr (Or.inl <X', hX', hX'ne, rfl>)
    ,
        embBit_subset.mpr hXs>
```

```
    * exact <Or.inr (Or.inr <Y, hY, hYne, rfl>), Or.inr (Or.inr <Y', hY', hY'ne, rfl>)
    ,
        embBit_subset.mpr hYs>
  * rintro <hW, hW', hsub>
    rcases hW' with rfl | <X', hX', hX'ne, rfl> | <Y', hY', hY'ne, rfl>
    * exact Or.inl <hW, rfl>
    * rcases hW with rfl | <X, hX, hXne, rfl> | <Y, hY, hYne, rfl>
      * exact absurd (hsub nil_mem_sumTokMaster) nil_not_mem_embBit
      * exact Or.inr (Or.inl <X, X', <hX, hX', embBit_subset.mp hsub>, hXne, hX'ne,
    rfl, rfl>)
      * exact absurd hsub (fun h => embBit_not_subset_cross (by decide) (B_1.ne Y hY)
    h)
    * rcases hW with rfl | <X, hX, hXne, rfl> | <Y, hY, hYne, rfl>
      * exact absurd (hsub nil_mem_sumTokMaster) nil_not_mem_embBit
      * exact absurd hsub (fun h => embBit_not_subset_cross (by decide) (B_0.ne X hX)
    h)
      * exact Or.inr (Or.inr <Y, Y', <hY, hY', embBit_subset.mp hsub>, hYne, hY'ne,
    rfl, rfl>)

/-- **Smash product carries the injection.** -/
theorem otimesMapTok_inj (h0 : A_0.sys ? B_0.sys) (h1 : A_1.sys ? B_1.sys) :
    otimesMapTok h0.inj h1.inj = (otimesTok_subsystem h0 h1).inj := by
  apply ApproximableMap.ext
  intro W W'
  rw [Subsystem.inj_rel]
  constructor
  * rintro (<hW, rfl> |
      <X, Y, X', Y', <hX, hX', hXs>, <hY, hY', hYs>, hXne, hYne, hX'ne, hY'ne, rfl,
    rfl>)
    * refine <hW, (B_0.otimes B_1).sys.master_mem, ?_>
      rcases hW with rfl | <P, Q, hP, hQ, hPne, hQne, rfl>
      * exact (show prodTokNbhd A_0.sys.master A_1.sys.master
            = prodTokNbhd B_0.sys.master B_1.sys.master by rw [h0.master_eq, h1.
    master_eq]).subset
        * exact prodTokNbhd_subset_iff.mpr <B_0.sys.sub_master (h0.sub hP),
          B_1.sys.sub_master (h1.sub hQ)>
    * exact <Or.inr <X, Y, hX, hY, hXne, hYne, rfl>, Or.inr <X', Y', hX', hY', hX'ne,
    hY'ne, rfl>,
        prodTokNbhd_subset_iff.mpr <hXs, hYs>>
  * rintro <hW, hW', hsub>
    rcases hW' with rfl | <X', Y', hX', hY', hX'ne, hY'ne, rfl>
    * exact Or.inl <hW, rfl>
    * rcases hW with rfl | <X, Y, hX, hY, hXne, hYne, rfl>
      * obtain <hsX, _> := prodTokNbhd_subset_iff.mp hsub
        exact absurd (Set.Subset.antisymm (B_0.sys.sub_master hX')
          (by rw [ <-  h0.master_eq]; exact hsX)) hX'ne
      * obtain <hXs, hYs> := prodTokNbhd_subset_iff.mp hsub
        exact Or.inr <X, Y, X', Y', <hX, hX', hXs>, <hY, hY', hYs>,
          hXne, hYne, hX'ne, hY'ne, rfl, rfl>

/-- **Smash product carries the projection.** -/
theorem otimesMapTok_proj (h0 : A_0.sys ? B_0.sys) (h1 : A_1.sys ? B_1.sys) :
    otimesMapTok h0.proj h1.proj = (otimesTok_subsystem h0 h1).proj := by
  apply ApproximableMap.ext
  intro W W'
  rw [Subsystem.proj_rel]
```

```
  constructor
  * rintro (<hW, rfl> |
      <X, Y, X', Y', <hX, hX', hXs>, <hY, hY', hYs>, hXne, hYne, hX'ne, hY'ne, rfl,
    rfl>)
    * refine <hW, (A_0.otimes A_1).sys.master_mem, ?_>
      rcases hW with rfl | <P, Q, hP, hQ, hPne, hQne, rfl>
      * exact (show prodTokNbhd B_0.sys.master B_1.sys.master
            = prodTokNbhd A_0.sys.master A_1.sys.master by rw [h0.master_eq, h1.
    master_eq]).subset
      * exact prodTokNbhd_subset_iff.mpr <by rw [h0.master_eq]; exact B_0.sys.
    sub_master hP,
          by rw [h1.master_eq]; exact B_1.sys.sub_master hQ>
    * exact <Or.inr <X, Y, hX, hY, hXne, hYne, rfl>, Or.inr <X', Y', hX', hY', hX'ne,
    hY'ne, rfl>,
        prodTokNbhd_subset_iff.mpr <hXs, hYs>>
  * rintro <hW, hW', hsub>
    rcases hW' with rfl | <X', Y', hX', hY', hX'ne, hY'ne, rfl>
    * exact Or.inl <hW, rfl>
    * rcases hW with rfl | <X, Y, hX, hY, hXne, hYne, rfl>
      * obtain <hsX, _> := prodTokNbhd_subset_iff.mp hsub
        exact absurd (Set.Subset.antisymm (A_0.sys.sub_master hX')
          (by rw [h0.master_eq]; exact hsX)) hX'ne
      * obtain <hXs, hYs> := prodTokNbhd_subset_iff.mp hsub
        exact Or.inr <X, Y, X', Y', <hX, hX', hXs>, <hY, hY', hYs>,
          hXne, hYne, hX'ne, hY'ne, rfl, rfl>

/-- **Product carries the injection.** -/
theorem prodMapTok_inj (h0 : A_0.sys ? B_0.sys) (h1 : A_1.sys ? B_1.sys) :
    prodMapTok h0.inj h1.inj = (prodTok_subsystem h0 h1).inj := by
  apply ApproximableMap.ext
  intro W W'
  rw [Subsystem.inj_rel]
  constructor
  * rintro <X, Y, X', Y', <hX, hX', hXs>, <hY, hY', hYs>, rfl, rfl>
    exact <prodTok_mem_prodTokNbhd hX hY, prodTok_mem_prodTokNbhd hX' hY',
      prodTokNbhd_subset_iff.mpr <hXs, hYs>>
  * rintro <<X, Y, hX, hY, rfl>, <X', Y', hX', hY', rfl>, hsub>
    obtain <hXs, hYs> := prodTokNbhd_subset_iff.mp hsub
    exact <X, Y, X', Y', <hX, hX', hXs>, <hY, hY', hYs>, rfl, rfl>

/-- **Product carries the projection.** -/
theorem prodMapTok_proj (h0 : A_0.sys ? B_0.sys) (h1 : A_1.sys ? B_1.sys) :
    prodMapTok h0.proj h1.proj = (prodTok_subsystem h0 h1).proj := by
  apply ApproximableMap.ext
  intro W W'


  rw [Subsystem.proj_rel]
  constructor
  * rintro <X, Y, X', Y', <hX, hX', hXs>, <hY, hY', hYs>, rfl, rfl>
    exact <prodTok_mem_prodTokNbhd hX hY, prodTok_mem_prodTokNbhd hX' hY',
      prodTokNbhd_subset_iff.mpr <hXs, hYs>>
  * rintro <<X, Y, hX, hY, rfl>, <X', Y', hX', hY', rfl>, hsub>
    obtain <hXs, hYs> := prodTokNbhd_subset_iff.mp hsub
    exact <X, Y, X', Y', <hX, hX', hXs>, <hY, hY', hYs>, rfl, rfl>

section Uniqueness
```

```
variable {N : ScottSys} (hN : ([] : Str)  in  N.sys.master)

/-- **`?_n = i_n  comp  j_n : Exp  ->  Exp`**, the retraction onto `Texp?({Gamma})`. -/
def rho (n : Nat) : ApproximableMap (Exp N hN).sys (Exp N hN).sys :=
  (expSub hN n).inj.comp (expSub hN n).proj

/-- Scott's relational description `A ?_n E  <->  exists  z  in  Texp?({Gamma}), A
    subseteq  z  subseteq  E`. -/
theorem rho_rel (n : Nat) {A E : Set Str} :
    (rho hN n).rel A E  <->  (Exp N hN).sys.mem A  /\  (Exp N hN).sys.mem E  /\
      exists  z, (gTower (Texp N) n).sys.mem z  /\  A  subseteq  z  /\  z  subseteq  E
     := by
  unfold rho
  rw [comp_rel]
  constructor
  * rintro <z, hproj, hinj>
    rw [Subsystem.proj_rel] at hproj
    rw [Subsystem.inj_rel] at hinj
    obtain <hcA, hTz, hAz> := hproj
    obtain <_, hcE, hzE> := hinj
    exact <hcA, hcE, z, hTz, hAz, hzE>
  * rintro <hcA, hcE, z, hTz, hAz, hzE>
    exact <z, by rw [Subsystem.proj_rel]; exact <hcA, hTz, hAz>,
      by rw [Subsystem.inj_rel]; exact <hTz, hcE, hzE>>

/-- `?_n  subseteq  ??` for `n  <=  m`. -/
theorem rho_mono {n m : Nat} (h : n  <=  m) {A E : Set Str} (hr : (rho hN n).rel A E)
    :
    (rho hN m).rel A E := by
  rw [rho_rel] at hr  |-
  obtain <hcA, hcE, z, hTz, hAz, hzE> := hr
  exact <hcA, hcE, z, (gTower_le (Texp N) (Texp_rooted hN) h).sub hTz, hAz, hzE>

/-- The pointwise union `?_n ?_n`. -/
def iSupRho : ApproximableMap (Exp N hN).sys (Exp N hN).sys :=
  iSupMap (rho hN) (fun i j => <max i j,
    fun _ _ h => rho_mono hN (le_max_left i j) h,
    fun _ _ h => rho_mono hN (le_max_right i j) h>)

/-- **`?_n ?_n = I_Exp`** (Scott's key identity). -/
theorem iSupRho_eq_id : iSupRho hN = idMap (Exp N hN).sys := by
  apply ApproximableMap.ext
  intro A E
  rw [idMap_rel]
  constructor
  * rintro <n, hr>
    rw [rho_rel] at hr
    obtain <hcA, hcE, z, _, hAz, hzE> := hr
    exact <hcA, hcE, hAz.trans hzE>
  * rintro <hcA, hcE, hAE>
    obtain <n, hA> := hcA
    exact <n, (rho_rel hN n).mpr <<n, hA>, hcE, A, hA, subset_rfl, hAE>>

/-- **`?_0 = False`** (the generator `{Gamma}` is one-point): `?_0` relates `A` only to
     the master. -/
```

```
theorem rho_zero_rel {A E : Set Str} :
    (rho hN 0).rel A E  <->  (Exp N hN).sys.mem A  /\  E = (Exp N hN).sys.master := by
  rw [rho_rel]
  constructor
  * rintro <hcA, hcE, z, hz, _, hzE>
    have hzm : z = (Exp N hN).sys.master :=
      (hz : z = gFix (Texp N)).trans (gColim_master (Texp N) (Texp_rooted hN)).symm
    subst hzm
    exact <hcA, Set.Subset.antisymm ((Exp N hN).sys.sub_master hcE) hzE>
  * rintro <hcA, rfl>
    exact <hcA, (Exp N hN).sys.master_mem, (Exp N hN).sys.master,
      gColim_master (Texp N) (Texp_rooted hN), (Exp N hN).sys.sub_master hcA,
    subset_rfl>

end Uniqueness

section Uniqueness

variable {N : ScottSys} (hN : ([] : Str)  in  N.sys.master)

variable (B : TAlgebra (TexpF N))

/-- The base of the recursion: `g  comp  ?_0 = False = val_0`, independent of `g`. -/
theorem gcomp_rho_zero (g : AlgHom (ExpAlg N hN) B) :
    g.hom.1.comp (rho hN 0) = descRel hN B 0 := by
  apply ApproximableMap.ext
  intro A Z
  rw [comp_rel]
  constructor
  * rintro <E, hrho, hg>
    rw [rho_zero_rel] at hrho
    obtain <hcA, rfl> := hrho
    have hZ : Z = B.carrier.sys.master := g.hom.2 hg
    exact <hcA, by rw [NeighborhoodSystem.mem_bot]; exact hZ>
  * rintro <hcA, hZ>
    rw [NeighborhoodSystem.mem_bot] at hZ
    subst hZ
    exact <(Exp N hN).sys.master, (rho_zero_rel hN).mpr <hcA, rfl>, g.hom.1.master_rel
    >

end Uniqueness

end Exercise619
end

section
open DomainTheory.NeighbourhoodSystems.Example62 DomainTheory.NeighbourhoodSystems.
    ExampleB DomainTheory.NeighbourhoodSystems.Exercise510

namespace Exercise619

/-- On `D ? D` (e.g. `Subsystem.refl`), the injection is the identity (both relations
    are
`X  in  D  /\  Y  in  D  /\  X  subseteq  Y`). -/
theorem Subsystem.self_inj {a : Type*} {D : NeighborhoodSystem a} (h : D ? D) :
    h.inj = idMap D := by
```

```
  apply ApproximableMap.ext
  intro X Y
  rw [Subsystem.inj_rel, idMap_rel]

/-- On `D ? D`, the projection is the identity. -/
theorem Subsystem.self_proj {a : Type*} {D : NeighborhoodSystem a} (h : D ? D) :
    h.proj = idMap D := by
  apply ApproximableMap.ext
  intro X Y
  rw [Subsystem.proj_rel, idMap_rel]

end Exercise619
end

section
open DomainTheory.NeighbourhoodSystems.Example62 DomainTheory.NeighbourhoodSystems.
    ExampleB DomainTheory.NeighbourhoodSystems.Exercise510

namespace Exercise619

variable {A_0 A_1 B_0 B_1 : ScottSys}

/-- **Sum carries the injection.** `(i_0 + i_1) = i` for the sum subsystem: both relate
     `W  |->  W'` iff
`W  in  ?_0+?_1`, `W'  in  ?_0+?_1`, `W  subseteq  W'`. -/
theorem sumMapTok_inj (h0 : A_0.sys ? B_0.sys) (h1 : A_1.sys ? B_1.sys) :
    sumMapTok h0.inj h1.inj = (sumTok_subsystem h0 h1).inj := by
  have hsubM : forall  {W : Set Str}, (A_0.sum A_1).sys.mem W  ->  W  subseteq
    sumTokMaster B_0.sys B_1.sys := by
    rintro W (rfl | <X, hX, rfl> | <Y, hY, rfl>)
    * exact (show sumTokMaster A_0.sys A_1.sys = sumTokMaster B_0.sys B_1.sys by
        unfold sumTokMaster; rw [h0.master_eq, h1.master_eq]).subset
    * exact embF_subset_sumTokMaster (h0.sub hX)
    * exact embT_subset_sumTokMaster (h1.sub hY)
  apply ApproximableMap.ext
  intro W W'
  rw [Subsystem.inj_rel]
  constructor
  * rintro (<hW, rfl> | <X, X', <hX, hX', hXsub>, rfl, rfl> | <Y, Y', <hY, hY', hYsub
    >, rfl, rfl>)
    * exact <hW, (B_0.sum B_1).sys.master_mem, hsubM hW>
    * exact <Or.inr (Or.inl <X, hX, rfl>), Or.inr (Or.inl <X', hX', rfl>),
    embBit_subset.mpr hXsub>
    * exact <Or.inr (Or.inr <Y, hY, rfl>), Or.inr (Or.inr <Y', hY', rfl>),
    embBit_subset.mpr hYsub>
  * rintro <hW, hW', hsub>
    rcases hW' with rfl | <X', hX', rfl> | <Y', hY', rfl>
    * exact Or.inl <hW, rfl>
    * rcases hW with rfl | <X, hX, rfl> | <Y, hY, rfl>
      * exact absurd (hsub nil_mem_sumTokMaster) nil_not_mem_embBit
      * exact Or.inr (Or.inl <X, X', <hX, hX', embBit_subset.mp hsub>, rfl, rfl>)
      * exact absurd hsub (fun h => embBit_not_subset_cross (by decide) (A_1.ne Y hY)
    h)
    * rcases hW with rfl | <X, hX, rfl> | <Y, hY, rfl>
      * exact absurd (hsub nil_mem_sumTokMaster) nil_not_mem_embBit
      * exact absurd hsub (fun h => embBit_not_subset_cross (by decide) (A_0.ne X hX)
```

```
    h)
      * exact Or.inr (Or.inr <Y, Y', <hY, hY', embBit_subset.mp hsub>, rfl, rfl>)

/-- **`T(i) = i'`** -- the functor carries the injection of `D ? E` to that of `T(D) ?
    T(E)`. -/
theorem GExpr.map_inj : (T : GExpr)  ->  {X Y : ScottSys}  ->  (h : X.sys ? Y.sys)  ->
    T.map h.inj = (T.obj_subsystem h).inj
  | .const D, _, _, _ => (Subsystem.self_inj (Subsystem.refl D.sys)).symm
  | .var, _, _, _ => rfl
  | .sum a b, _, _, h => by
      show sumMapTok (a.map h.inj) (b.map h.inj)
          = (sumTok_subsystem (a.obj_subsystem h) (b.obj_subsystem h)).inj
      rw [a.map_inj h, b.map_inj h, sumMapTok_inj]
  | .prod a b, _, _, h => by
      show prodMapTok (a.map h.inj) (b.map h.inj)
          = (prodTok_subsystem (a.obj_subsystem h) (b.obj_subsystem h)).inj
      rw [a.map_inj h, b.map_inj h, prodMapTok_inj]
  | .oplus a b, _, _, h => by
      show oplusMapTok (a.map h.inj) (b.map h.inj)
          = (oplusTok_subsystem (a.obj_subsystem h) (b.obj_subsystem h)).inj
      rw [a.map_inj h, b.map_inj h, oplusMapTok_inj]
  | .otimes a b, _, _, h => by
      show otimesMapTok (a.map h.inj) (b.map h.inj)
          = (otimesTok_subsystem (a.obj_subsystem h) (b.obj_subsystem h)).inj
      rw [a.map_inj h, b.map_inj h, otimesMapTok_inj]

/-- **Sum carries the projection.** -/
theorem sumMapTok_proj (h0 : A_0.sys ? B_0.sys) (h1 : A_1.sys ? B_1.sys) :
    sumMapTok h0.proj h1.proj = (sumTok_subsystem h0 h1).proj := by
  have hsubM : forall  {W : Set Str}, (B_0.sum B_1).sys.mem W  ->  W  subseteq
    sumTokMaster A_0.sys A_1.sys := by
    rintro W (rfl | <X, hX, rfl> | <Y, hY, rfl>)
    * exact (show sumTokMaster B_0.sys B_1.sys = sumTokMaster A_0.sys A_1.sys by
        unfold sumTokMaster; rw [h0.master_eq, h1.master_eq]).subset
    * exact (embBit_subset.mpr (by rw [h0.master_eq]; exact B_0.sys.sub_master hX)).
    trans
        (embF_subset_sumTokMaster A_0.sys.master_mem)
    * exact (embBit_subset.mpr (by rw [h1.master_eq]; exact B_1.sys.sub_master hY)).
    trans
        (embT_subset_sumTokMaster A_1.sys.master_mem)
  apply ApproximableMap.ext
  intro W W'
  rw [Subsystem.proj_rel]
  constructor
  * rintro (<hW, rfl> | <X, X', <hX, hX', hXsub>, rfl, rfl> | <Y, Y', <hY, hY', hYsub
    >, rfl, rfl>)
    * exact <hW, (A_0.sum A_1).sys.master_mem, hsubM hW>
    * exact <Or.inr (Or.inl <X, hX, rfl>), Or.inr (Or.inl <X', hX', rfl>),
    embBit_subset.mpr hXsub>
    * exact <Or.inr (Or.inr <Y, hY, rfl>), Or.inr (Or.inr <Y', hY', rfl>),
    embBit_subset.mpr hYsub>
  * rintro <hW, hW', hsub>
    rcases hW' with rfl | <X', hX', rfl> | <Y', hY', rfl>
    * exact Or.inl <hW, rfl>
    * rcases hW with rfl | <X, hX, rfl> | <Y, hY, rfl>
      * exact absurd (hsub nil_mem_sumTokMaster) nil_not_mem_embBit
```

```
      * exact Or.inr (Or.inl <X, X', <hX, hX', embBit_subset.mp hsub>, rfl, rfl>)
      * exact absurd hsub (fun h => embBit_not_subset_cross (by decide) (B_1.ne Y hY)
    h)
    * rcases hW with rfl | <X, hX, rfl> | <Y, hY, rfl>
      * exact absurd (hsub nil_mem_sumTokMaster) nil_not_mem_embBit
      * exact absurd hsub (fun h => embBit_not_subset_cross (by decide) (B_0.ne X hX)
    h)
      * exact Or.inr (Or.inr <Y, Y', <hY, hY', embBit_subset.mp hsub>, rfl, rfl>)

/-- **`T(j) = j'`** -- the functor carries the projection of `D ? E` to that of `T(D) ?
    T(E)`. -/
theorem GExpr.map_proj : (T : GExpr)  ->  {X Y : ScottSys}  ->  (h : X.sys ? Y.sys)
    ->
    T.map h.proj = (T.obj_subsystem h).proj
  | .const D, _, _, _ => (Subsystem.self_proj (Subsystem.refl D.sys)).symm
  | .var, _, _, _ => rfl
  | .sum a b, _, _, h => by
      show sumMapTok (a.map h.proj) (b.map h.proj)
          = (sumTok_subsystem (a.obj_subsystem h) (b.obj_subsystem h)).proj
      rw [a.map_proj h, b.map_proj h, sumMapTok_proj]
  | .prod a b, _, _, h => by
      show prodMapTok (a.map h.proj) (b.map h.proj)
          = (prodTok_subsystem (a.obj_subsystem h) (b.obj_subsystem h)).proj
      rw [a.map_proj h, b.map_proj h, prodMapTok_proj]
  | .oplus a b, _, _, h => by
      show oplusMapTok (a.map h.proj) (b.map h.proj)
          = (oplusTok_subsystem (a.obj_subsystem h) (b.obj_subsystem h)).proj
      rw [a.map_proj h, b.map_proj h, oplusMapTok_proj]
  | .otimes a b, _, _, h => by
      show otimesMapTok (a.map h.proj) (b.map h.proj)
          = (otimesTok_subsystem (a.obj_subsystem h) (b.obj_subsystem h)).proj
      rw [a.map_proj h, b.map_proj h, otimesMapTok_proj]

section Uniqueness

variable {N : ScottSys} (hN : ([] : Str)  in  N.sys.master)

/-- `T(?_n) = T(i_n)  comp  T(j_n) = i'_n  comp  j'_n`, the projection pair of `T(Texp
    ?{Gamma}) ? T(Exp)`. -/
theorem map_rho_eq (n : Nat) :
    (Texp N).map (rho hN n)
      = ((Texp N).obj_subsystem (expSub hN n)).inj.comp
        ((Texp N).obj_subsystem (expSub hN n)).proj := by
  unfold rho
  rw [(Texp N).map_comp (expSub hN n).proj (Subsystem.inj_isStrict (expSub hN n)),
      (Texp N).map_inj, (Texp N).map_proj]

/-- **`?_n_+_1 = i  comp  T(?_n)  comp  j`** (Scott's `T(?_n) = ?_n_+_1`, conjugated by
    the structure iso). -/
theorem key_rho (n : Nat) :
    rho hN (n + 1)
      = (expHom N hN).comp (((Texp N).map (rho hN n)).comp (expInv N hN)) := by
  have hsyseq : ((Texp N).obj (Exp N hN)).sys = (Exp N hN).sys :=
    gColim_obj_sys_eq (Texp N) (Texp_rooted hN)
  apply ApproximableMap.ext
  intro A E
```

```
  rw [map_rho_eq]
  simp only [comp_rel, rho_rel, expInv_rel, expHom_rel, Subsystem.proj_rel,
    Subsystem.inj_rel, hsyseq]
  constructor
  * rintro <hcA, hcE, z, hTz, hAz, hzE>
    exact <E, <A, <hcA, hcA, subset_rfl>, z, <hcA, hTz, hAz>, hTz, hcE, hzE>,
      hcE, hcE, subset_rfl>
  * rintro <Y, <C, <hcA, _, hAC>, z, <_, hTz, hCz>, _, _, hzY>, _, hcE, hYE>
    exact <hcA, hcE, z, hTz, hAC.trans hCz, hzY.trans hYE>

end Uniqueness

section Uniqueness

variable {N : ScottSys} (hN : ([] : Str)  in  N.sys.master)

variable (B : TAlgebra (TexpF N))

/-- **The fixed-point recursion `g_n_+_1 = k  comp  T(g_n)  comp  j`** (`key_rho` + the
     homomorphism square). -/
theorem gcomp_rho_succ (g : AlgHom (ExpAlg N hN) B) (n : Nat) :
    g.hom.1.comp (rho hN (n + 1))
      = (algStr B).comp (((Texp N).map (g.hom.1.comp (rho hN n))).comp (expInv N hN))
    := by
  have hcomm : (g.hom.1).comp (expHom N hN) = (algStr B).comp ((Texp N).map g.hom.1)
    :=
    congrArg Subtype.val g.comm
  calc g.hom.1.comp (rho hN (n + 1))
      = g.hom.1.comp ((expHom N hN).comp
          (((Texp N).map (rho hN n)).comp (expInv N hN))) := by rw [key_rho]
    _ = (g.hom.1.comp (expHom N hN)).comp
          (((Texp N).map (rho hN n)).comp (expInv N hN)) :=
        (comp_assoc _ _ _).symm
    _ = ((algStr B).comp ((Texp N).map g.hom.1)).comp
          (((Texp N).map (rho hN n)).comp (expInv N hN)) :=
        congrArg (fun m => m.comp (((Texp N).map (rho hN n)).comp (expInv N hN)))
    hcomm
    _ = (algStr B).comp (((Texp N).map g.hom.1).comp
          (((Texp N).map (rho hN n)).comp (expInv N hN))) := comp_assoc _ _ _
    _ = (algStr B).comp ((((Texp N).map g.hom.1).comp ((Texp N).map (rho hN n))).comp
          (expInv N hN)) :=
        congrArg ((algStr B).comp *)
          (comp_assoc ((Texp N).map g.hom.1) ((Texp N).map (rho hN n)) (expInv N hN)).
    symm
    _ = (algStr B).comp (((Texp N).map (g.hom.1.comp (rho hN n))).comp (expInv N hN))
    :=
        congrArg (fun m => (algStr B).comp (m.comp (expInv N hN)))
          ((Texp N).map_comp (rho hN n) g.hom.2).symm

/-- **`g  comp  ?_n = val_0_n`**: every homomorphism `g` agrees with the canonical
    Kleene iterate after the
`n`-th projection -- the sequence is forced by the recursion, independent of `g`. -/
theorem gcomp_rho_eq (g : AlgHom (ExpAlg N hN) B) :
    forall  n, g.hom.1.comp (rho hN n) = descRel hN B n
  | 0 => gcomp_rho_zero hN B g
  | n + 1 => by rw [gcomp_rho_succ hN B g n, gcomp_rho_eq g n,  <-  descRel_succ]
```

```
/-- **The underlying map of any homomorphism `g : Exp  ->  D` is `val = descMap`.**
    Hence `descAlgHom`
is the *unique* homomorphism. -/
theorem descMap_eq_algHom (g : AlgHom (ExpAlg N hN) B) : g.hom.1 = descMap hN B := by
  have hcomp : g.hom.1.comp (iSupRho hN) = descMap hN B := by
    apply ApproximableMap.ext
    intro A E
    rw [comp_rel, descMap_rel]
    constructor
    * rintro <Y, <n, hrho>, hg>
      refine <n, ?_>
      rw [ <-  gcomp_rho_eq hN B g n, comp_rel]
      exact <Y, hrho, hg>
    * rintro <n, hn>
      rw [ <-  gcomp_rho_eq hN B g n, comp_rel] at hn
      obtain <Y, hrho, hg> := hn
      exact <Y, <n, hrho>, hg>
  calc g.hom.1 = g.hom.1.comp (iSupRho hN) := by
        rw [iSupRho_eq_id hN]; exact (comp_idMap g.hom.1).symm
    _ = descMap hN B := hcomp

end Uniqueness

end Exercise619
end
end DomainTheory.NeighbourhoodSystems
end
```

3.7.25 Exercise 6.23 — syntactic expression domain

Construe the initial solution to

$$Exp \cong N \oplus ((Exp \times Exp) + (Exp \times Exp))$$

as a "syntactical domain" of expressions generated from infinitely many "variables" by means of two binary "operation symbols". Given an algebra D with two operations

$$u : D \times D \to D \quad \text{and} \quad v : D \times D \to D,$$

show how any strict map $s : N \to D$ determines a unique map

$$val(s) : Exp \to D$$

that can be regarded as the "evaluation of an expression".

Exercise_6_23.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.Exercise_6_22

public import DomainTheory.NeighbourhoodSystems.DomainEquations.Proposition_6_6
```

```
@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.Exercise619
section
open DomainTheory.NeighbourhoodSystems.Example62 DomainTheory.NeighbourhoodSystems.
    ExampleB DomainTheory.NeighbourhoodSystems.Exercise510
namespace Exercise619
variable {A_0 A_1 B_0 B_1 : ScottSys}
section Uniqueness
variable {N : ScottSys} (hN : ([] : Str)  in  N.sys.master)
variable (B : TAlgebra (TexpF N))
/- concordance: source L4141 EXERCISE 6.23 syntactic expression domain -/
/-- **Exercise 6.23 (Scott 1981, PRG-19) -- `Exp` is the initial `T`-algebra.** For
    every algebra
`B = (D, s, u, v)` there is a *unique* homomorphism `val(s) : Exp  ->  D` -- Scott's
    evaluation of an
expression. Existence is `descAlgHom` (Phase 3); uniqueness is the projection-chain
    argument. -/
def ExpInitial : IsInitial (ExpAlg N hN) where
  desc B := descAlgHom hN B
  uniq B g := algHom_ext (Subtype.ext (descMap_eq_algHom hN B g))

end Uniqueness

end Exercise619
end

section
open DomainTheory.NeighbourhoodSystems.Example62 DomainTheory.NeighbourhoodSystems.
    ExampleB DomainTheory.NeighbourhoodSystems.Exercise510

namespace Exercise619

/-- The projection `j : E  ->  D` of a subsystem is **strict**. -/
theorem Subsystem.proj_isStrict {a : Type*} {D E : NeighborhoodSystem a} (h : D ? E) :
    IsStrict h.proj := by
  intro X hrel
  rw [Subsystem.proj_rel] at hrel
  obtain <_, hXD, hsub> := hrel
  exact Set.Subset.antisymm (D.sub_master hXD) (by rw [h.master_eq]; exact hsub)

/-- `i  comp  j = I_Exp` at the raw level (from the iso's `inv_hom_id`). -/
theorem expHom_comp_expInv (N : ScottSys) (hN : ([] : Str)  in  N.sys.master) :
    (expHom N hN).comp (expInv N hN) = idMap (Exp N hN).sys := by
  have h := congrArg (Subtype.val) (ExpIso N hN).inv_hom_id
  exact h

section Existence

variable {N : ScottSys} (hN : ([] : Str)  in  N.sys.master) (B : TAlgebra (TexpF N))

/-- **Every homomorphism `g : Exp  ->  D` is a fixed point** of the operator `fun h. k
     comp  T(h)  comp  j`. This is
the homomorphism square `g  comp  i = k  comp  T(g)` (`g.comm`) rearranged by `i  comp
```

```
     j = I`. -/
theorem algHom_fix (g : AlgHom (ExpAlg N hN) B) :
    g.hom.1 = (algStr B).comp (((Texp N).map g.hom.1).comp (expInv N hN)) := by
  have hcomm : (g.hom.1).comp (expHom N hN) = (algStr B).comp ((Texp N).map g.hom.1)
    :=
    congrArg Subtype.val g.comm
  calc g.hom.1
      = (g.hom.1).comp (idMap (Exp N hN).sys) := (comp_idMap _).symm
    _ = (g.hom.1).comp ((expHom N hN).comp (expInv N hN)) := by rw [expHom_comp_expInv
    ]
    _ = ((g.hom.1).comp (expHom N hN)).comp (expInv N hN) := (comp_assoc _ _ _).symm
    _ = ((algStr B).comp ((Texp N).map g.hom.1)).comp (expInv N hN) :=
          congrArg (fun m => m.comp (expInv N hN)) hcomm
    _ = (algStr B).comp (((Texp N).map g.hom.1).comp (expInv N hN)) := comp_assoc _ _
    _

/-- **`descAlgHom` is the least homomorphism**: `val  <=  g` for every homomorphism `g
    : Exp  ->  D`. The
Kleene iterates `val_n` lie below any fixed point `g` (induction: `val_0 = False  <=  g
    `, and the operator is
monotone with `g` its own fixed point), so their union `val` does too. This is the easy
     half of
initiality; the matching `g  <=  val` (so `g = val`) is the projection-chain argument
    of Phase 4. -/
theorem descRel_le_algHom (g : AlgHom (ExpAlg N hN) B) : forall  n, descRel hN B n  <=
      g.hom.1
  | 0 => constBot_le _
  | n + 1 => by
      rw [descRel_succ, algHom_fix hN B g]
      exact comp_mono_gen le_rfl
        (comp_mono_gen ((Texp N).map_mono (descRel_le_algHom g n)) le_rfl)

theorem descMap_le_algHom (g : AlgHom (ExpAlg N hN) B) : descMap hN B  <=  g.hom.1 :=
    by
  intro X Y hr
  obtain <n, hn> := hr
  exact descRel_le_algHom hN B g n X Y hn

end Existence

end Exercise619
end

end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.Exercise619
section
set_option linter.unusedVariables false

theorem example_623 : forall  {N : DomainTheory.NeighbourhoodSystems.Exercise619.
    ScottSys}
  (hN :
    @Membership.mem (List Bool) (Set DomainTheory.NeighbourhoodSystems.ExampleB.Str)
      (@Set.instMembership DomainTheory.NeighbourhoodSystems.ExampleB.Str)
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.master DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
        N.sys)
```

```
    (@List.nil Bool))
(B :
  @DomainTheory.NeighbourhoodSystems.TAlgebra DomainTheory.NeighbourhoodSystems.
  Exercise619.ScottSys
    DomainTheory.NeighbourhoodSystems.Exercise619.instCategoryScottSys
    (DomainTheory.NeighbourhoodSystems.Exercise619.TexpF N))
(g :
  @DomainTheory.NeighbourhoodSystems.AlgHom DomainTheory.NeighbourhoodSystems.
  Exercise619.ScottSys
    DomainTheory.NeighbourhoodSystems.Exercise619.instCategoryScottSys
    (DomainTheory.NeighbourhoodSystems.Exercise619.TexpF N)
    (DomainTheory.NeighbourhoodSystems.Exercise619.ExpAlg N hN) B),
@LE.le
  (@DomainTheory.NeighbourhoodSystems.ApproximableMap DomainTheory.
  NeighbourhoodSystems.ExampleB.Str
    DomainTheory.NeighbourhoodSystems.ExampleB.Str (DomainTheory.
  NeighbourhoodSystems.Exercise619.Exp N hN).sys
    (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier DomainTheory.
  NeighbourhoodSystems.Exercise619.ScottSys
       DomainTheory.NeighbourhoodSystems.Exercise619.instCategoryScottSys
       (DomainTheory.NeighbourhoodSystems.Exercise619.TexpF N) B).sys)
  (@Preorder.toLE
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap DomainTheory.
  NeighbourhoodSystems.ExampleB.Str
     DomainTheory.NeighbourhoodSystems.ExampleB.Str (DomainTheory.
  NeighbourhoodSystems.Exercise619.Exp N hN).sys
     (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier DomainTheory.
  NeighbourhoodSystems.Exercise619.ScottSys
         DomainTheory.NeighbourhoodSystems.Exercise619.instCategoryScottSys
         (DomainTheory.NeighbourhoodSystems.Exercise619.TexpF N) B).sys)
    (@PartialOrder.toPreorder
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap DomainTheory.
  NeighbourhoodSystems.ExampleB.Str
       DomainTheory.NeighbourhoodSystems.ExampleB.Str (DomainTheory.
  NeighbourhoodSystems.Exercise619.Exp N hN).sys
       (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier DomainTheory.
  NeighbourhoodSystems.Exercise619.ScottSys
           DomainTheory.NeighbourhoodSystems.Exercise619.instCategoryScottSys
           (DomainTheory.NeighbourhoodSystems.Exercise619.TexpF N) B).sys)
      (@DomainTheory.NeighbourhoodSystems.instPartialOrderApproximableMap
       DomainTheory.NeighbourhoodSystems.ExampleB.Str DomainTheory.
  NeighbourhoodSystems.ExampleB.Str
       (DomainTheory.NeighbourhoodSystems.Exercise619.Exp N hN).sys
       (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier DomainTheory.
  NeighbourhoodSystems.Exercise619.ScottSys
           DomainTheory.NeighbourhoodSystems.Exercise619.instCategoryScottSys
           (DomainTheory.NeighbourhoodSystems.Exercise619.TexpF N) B).sys)))
  (@DomainTheory.NeighbourhoodSystems.Exercise619.descMap N hN B)
  (@Subtype.val
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap DomainTheory.
  NeighbourhoodSystems.ExampleB.Str
     DomainTheory.NeighbourhoodSystems.ExampleB.Str
     (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier DomainTheory.
  NeighbourhoodSystems.Exercise619.ScottSys
         DomainTheory.NeighbourhoodSystems.Exercise619.instCategoryScottSys
         (DomainTheory.NeighbourhoodSystems.Exercise619.TexpF N)
```

```
          (DomainTheory.NeighbourhoodSystems.Exercise619.ExpAlg N hN)).sys
      (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier DomainTheory.
    NeighbourhoodSystems.Exercise619.ScottSys
         DomainTheory.NeighbourhoodSystems.Exercise619.instCategoryScottSys
         (DomainTheory.NeighbourhoodSystems.Exercise619.TexpF N) B).sys)
    (fun
        (f :
          @DomainTheory.NeighbourhoodSystems.ApproximableMap DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
           DomainTheory.NeighbourhoodSystems.ExampleB.Str
           (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier
               DomainTheory.NeighbourhoodSystems.Exercise619.ScottSys
               DomainTheory.NeighbourhoodSystems.Exercise619.instCategoryScottSys
               (DomainTheory.NeighbourhoodSystems.Exercise619.TexpF N)
               (DomainTheory.NeighbourhoodSystems.Exercise619.ExpAlg N hN)).sys
           (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier
               DomainTheory.NeighbourhoodSystems.Exercise619.ScottSys
               DomainTheory.NeighbourhoodSystems.Exercise619.instCategoryScottSys
               (DomainTheory.NeighbourhoodSystems.Exercise619.TexpF N) B).sys) =>
      @DomainTheory.NeighbourhoodSystems.Exercise510.IsStrict DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
        DomainTheory.NeighbourhoodSystems.ExampleB.Str
        (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier DomainTheory.
    NeighbourhoodSystems.Exercise619.ScottSys
           DomainTheory.NeighbourhoodSystems.Exercise619.instCategoryScottSys
           (DomainTheory.NeighbourhoodSystems.Exercise619.TexpF N)
           (DomainTheory.NeighbourhoodSystems.Exercise619.ExpAlg N hN)).sys
        (@DomainTheory.NeighbourhoodSystems.TAlgebra.carrier DomainTheory.
    NeighbourhoodSystems.Exercise619.ScottSys
           DomainTheory.NeighbourhoodSystems.Exercise619.instCategoryScottSys
           (DomainTheory.NeighbourhoodSystems.Exercise619.TexpF N) B).sys
        f)
    (@DomainTheory.NeighbourhoodSystems.AlgHom.hom DomainTheory.NeighbourhoodSystems
    .Exercise619.ScottSys
       DomainTheory.NeighbourhoodSystems.Exercise619.instCategoryScottSys
       (DomainTheory.NeighbourhoodSystems.Exercise619.TexpF N)
       (DomainTheory.NeighbourhoodSystems.Exercise619.ExpAlg N hN) B g)) :=
  @DomainTheory.NeighbourhoodSystems.Exercise619.descMap_le_algHom
end

end DomainTheory.NeighbourhoodSystems.Exercise619
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem DomainTheory.NeighbourhoodSystems.Exercise619
open DomainTheory.NeighbourhoodSystems.Example62 DomainTheory.NeighbourhoodSystems.
    ExampleB

section
namespace Exercise624

/-- The first right-hand side `D + (D  x  E)`. -/
def Fsol (D E : ScottSys) : ScottSys := D.sum (D.prod E)

/-- The second right-hand side `D + E`. -/
def Gsol (D E : ScottSys) : ScottSys := D.sum E

/-- `gTok p q = tok((D + E))` for `D = {p}`, `E = {q}`: the tagged union `{?}  U  0p  U
```

```
      1q`. It is also
the master of the product `{p}  x  {q}` (sum and product share the master shape over
    `{0,1}*`). -/
def gTok (p q : Set Str) : Set Str := insert ([] : Str) (embBit false p  U  embBit
    true q)

/-- `fTok p q = tok((D + (D x E)))` for `D = {p}`, `E = {q}`: `{?}  U  0p  U  1(gTok p
    q)`, equivalently
`gTok p (gTok p q)`. -/
def fTok (p q : Set Str) : Set Str := gTok p (gTok p q)

/-- The pair iteration `?(p, q) = (fTok p q, gTok p q)` from the bottom pair `({?},
    {?})`. -/
def pIter : Nat  ->  Set Str  x  Set Str
  | 0 => ({([] : Str)}, {([] : Str)})
  | n + 1 => (fTok (pIter n).1 (pIter n).2, gTok (pIter n).1 (pIter n).2)

/-- The first component of the double fixed point, `Gamma_D = ?_n (??)._1`. -/
def GammaD : Set Str := ? n, (pIter n).1

/-- The second component of the double fixed point, `Gamma_E = ?_n (??)._2`. -/
def GammaE : Set Str := ? n, (pIter n).2

theorem nil_mem_GammaD : ([] : Str)  in  GammaD := Set.mem_iUnion.mpr <0, rfl>

/-- The solution system `D = {Gamma_D}`. -/
def Dsol : ScottSys := singletonSys GammaD <[], nil_mem_GammaD>

theorem nil_mem_GammaE : ([] : Str)  in  GammaE := Set.mem_iUnion.mpr <0, rfl>

/-- The solution system `E = {Gamma_E}`. -/
def Esol : ScottSys := singletonSys GammaE <[], nil_mem_GammaE>

/-- `gTok` is monotone in both arguments. -/
theorem gTok_mono {p p' q q' : Set Str} (hp : p  subseteq  p') (hq : q  subseteq  q')
    : gTok p q  subseteq  gTok p' q' := by
  rintro w (rfl | <w', rfl, hw'> | <w', rfl, hw'>)
  * exact Set.mem_insert _ _
  * exact Or.inr (Or.inl <w', rfl, hp hw'>)
  * exact Or.inr (Or.inr <w', rfl, hq hw'>)

/-- `fTok` is monotone in both arguments. -/
theorem fTok_mono {p p' q q' : Set Str} (hp : p  subseteq  p') (hq : q  subseteq  q')
    : fTok p q  subseteq  fTok p' q' :=
  gTok_mono hp (gTok_mono hp hq)

/-- The master (token set) of `D + (D x E)` is `fTok Gamma_D Gamma_E` -- definitionally
    , the sum/product
masters expand to the tagged-union shape `gTok`/`fTok`. -/
theorem master_Fsol : (Fsol Dsol Esol).sys.master = fTok GammaD GammaE := rfl

/-- The master (token set) of `D + E` is `gTok Gamma_D Gamma_E`. -/
theorem master_Gsol : (Gsol Dsol Esol).sys.master = gTok GammaD GammaE := rfl

theorem nil_mem_fTok (p q : Set Str) : ([] : Str)  in  fTok p q := Set.mem_insert _ _
```

```
theorem nil_mem_gTok (p q : Set Str) : ([] : Str)  in  gTok p q := Set.mem_insert _ _

/-- **Additivity of `fTok`.** Every token of `fTok (?_n a_n) (?_n b_n)` already lies in
     some
`fTok a_n b_n`. The nested `true`-branch (the product copy `0p` inside `1(gTok p q)`)
    still references
at most one coordinate, so a single finite stage suffices. -/
theorem mem_fTok_iUnion {a b : Nat  ->  Set Str} {w : Str}
    (hw : w  in  fTok (? n, a n) (? n, b n)) : exists  n, w  in  fTok (a n) (b n) :=
    by
  rcases hw with rfl | <w', rfl, hw'> | <u, rfl, hu>
  * exact <0, nil_mem_fTok _ _>
  * obtain <n, hn> := Set.mem_iUnion.mp hw'
    exact <n, Or.inr (Or.inl <w', rfl, hn>)>
  * rcases hu with rfl | <u', rfl, hu'> | <u', rfl, hu'>
    * exact <0, Or.inr (Or.inr <[], rfl, nil_mem_gTok _ _>)>
    * obtain <n, hn> := Set.mem_iUnion.mp hu'
      exact <n, Or.inr (Or.inr <false :: u', rfl, Or.inr (Or.inl <u', rfl, hn>)>)>
    * obtain <n, hn> := Set.mem_iUnion.mp hu'
      exact <n, Or.inr (Or.inr <true :: u', rfl, Or.inr (Or.inr <u', rfl, hn>)>)>

/-- **Additivity of `gTok`.** Every token of `gTok (?_n a_n) (?_n b_n)` already lies in
     some
`gTok a_n b_n` -- each token references at most one coordinate. -/
theorem mem_gTok_iUnion {a b : Nat  ->  Set Str} {w : Str}
    (hw : w  in  gTok (? n, a n) (? n, b n)) : exists  n, w  in  gTok (a n) (b n) :=
    by
  rcases hw with rfl | <w', rfl, hw'> | <w', rfl, hw'>
  * exact <0, nil_mem_gTok _ _>
  * obtain <n, hn> := Set.mem_iUnion.mp hw'
    exact <n, Or.inr (Or.inl <w', rfl, hn>)>
  * obtain <n, hn> := Set.mem_iUnion.mp hw'
    exact <n, Or.inr (Or.inr <w', rfl, hn>)>

/-- Each stage sits inside the colimit `Gamma_D` (choice-free; avoids `Set.
    subset_iUnion`). -/
theorem pIter_fst_subset_GammaD (m : Nat) : (pIter m).1  subseteq  GammaD :=
  fun _ hx => Set.mem_iUnion.mpr <m, hx>

/-- Each stage sits inside the colimit `Gamma_E` (choice-free). -/
theorem pIter_snd_subset_GammaE (m : Nat) : (pIter m).2  subseteq  GammaE :=
  fun _ hx => Set.mem_iUnion.mpr <m, hx>

/-- **First equation of the double fixed point:** `fTok Gamma_D Gamma_E = Gamma_D`, i.e
    . `tok(D + (D x E)) = Gamma_D`
for `D = {Gamma_D}`, `E = {Gamma_E}`. -/
theorem fTok_GammaD_GammaE : fTok GammaD GammaE = GammaD := by
  apply Set.Subset.antisymm
  * intro w hw
    obtain <n, hn> :=
      mem_fTok_iUnion (a := fun n => (pIter n).1) (b := fun n => (pIter n).2) hw
    exact Set.mem_iUnion.mpr <n + 1, hn>
  * intro w hw
    obtain <n, hn> := Set.mem_iUnion.mp hw
    cases n with
    | zero =>
```

```
        have hw0 : w = [] := hn; subst hw0; exact nil_mem_fTok _ _
    | succ m =>
        exact fTok_mono (pIter_fst_subset_GammaD m) (pIter_snd_subset_GammaE m) hn

/-- **`{Gamma_D} ? D + (D  x  E)`.** The singleton system `D = {Gamma_D}` is a
    subsystem of `D + (D x E)`: by the
fixed point its only neighbourhood `Gamma_D` is the master of `D + (D x E)`. -/
theorem Dsol_subsystem : Dsol.sys ? (Fsol Dsol Esol).sys := by
  have hmaster : (Fsol Dsol Esol).sys.master = GammaD := master_Fsol.trans
    fTok_GammaD_GammaE
  refine <hmaster.symm, ?_, ?_>
  * intro X hX
    have hXeq : X = GammaD := hX
    rw [hXeq,  <-  hmaster]
    exact (Fsol Dsol Esol).sys.master_mem
  * intro X Y hX hY _
    show X  I  Y = GammaD
    rw [show X = GammaD from hX, show Y = GammaD from hY, Set.inter_self]

/-- **Second equation of the double fixed point:** `gTok Gamma_D Gamma_E = Gamma_E`, i.
    e. `tok(D + E) = Gamma_E`. -/
theorem gTok_GammaD_GammaE : gTok GammaD GammaE = GammaE := by
  apply Set.Subset.antisymm
  * intro w hw
    obtain <n, hn> :=
      mem_gTok_iUnion (a := fun n => (pIter n).1) (b := fun n => (pIter n).2) hw
    exact Set.mem_iUnion.mpr <n + 1, hn>
  * intro w hw
    obtain <n, hn> := Set.mem_iUnion.mp hw
    cases n with
    | zero =>
        have hw0 : w = [] := hn; subst hw0; exact nil_mem_gTok _ _
    | succ m =>
        exact gTok_mono (pIter_fst_subset_GammaD m) (pIter_snd_subset_GammaE m) hn

/-- **`{Gamma_E} ? D + E`.** The singleton system `E = {Gamma_E}` is a subsystem of `D
    + E`: by the fixed
point its only neighbourhood `Gamma_E` is the master of `D + E`. -/
theorem Esol_subsystem : Esol.sys ? (Gsol Dsol Esol).sys := by
  have hmaster : (Gsol Dsol Esol).sys.master = GammaE := master_Gsol.trans
    gTok_GammaD_GammaE
  refine <hmaster.symm, ?_, ?_>
  * intro X hX
    have hXeq : X = GammaE := hX
    rw [hXeq,  <-  hmaster]
    exact (Gsol Dsol Esol).sys.master_mem
  * intro X Y hX hY _
    show X  I  Y = GammaE
    rw [show X = GammaE from hX, show Y = GammaE from hY, Set.inter_self]

/-- **The double fixed point (token level).** There is a pair of token sets `(Gamma_D,
    Gamma_E)`, both
containing `?`, that solve the two coupled token equations `tok(D + (D x E)) = Gamma_D`
     and
`tok(D + E) = Gamma_E` simultaneously. -/
theorem exists_double_fixedPoint :
```

```
    exists  Gammad Gammae : Set Str, ([] : Str)  in  Gammad  /\  ([] : Str)  in
    Gammae  /\
      fTok Gammad Gammae = Gammad  /\  gTok Gammad Gammae = Gammae :=
  <GammaD, GammaE, nil_mem_GammaD, nil_mem_GammaE, fTok_GammaD_GammaE,
    gTok_GammaD_GammaE>

end Exercise624
end
end DomainTheory.NeighbourhoodSystems
end
```

3.7.26 Exercise 6.24 — recursive domain equations

Show that there must exist domains satisfying:

$$D \cong D + (D \times E), \quad \text{and}$$

$$E \cong D + E,$$

by using a double fixed-point method. First decide what the underlying set of tokens should be, and then define D and E by simultaneous fixed points. (Syntactical domains as in 6.23 may very well require several simultaneous equations.)

Exercise_6_24.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.Exercise_6_23
public import DomainTheory.NeighbourhoodSystems.DomainEquations.Definition_6_13

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem DomainTheory.NeighbourhoodSystems.Exercise619
open DomainTheory.NeighbourhoodSystems.Example62 DomainTheory.NeighbourhoodSystems.
    ExampleB
section
namespace Exercise624
/- concordance: source L4161 EXERCISE 6.24 recursive domain equations -/
/-- **Exercise 6.24 (object level).** By the double fixed-point method there exist
    `{}`-free systems
`D`, `E` over `{0,1}*` such that `{D} ? D + (D x E)` and `{E} ? D + E` **simultaneously
    **. These two
facts are exactly the joint hypothesis of the simultaneous Theorem 6.14, which then
    yields the
required isomorphisms `D  ~=  D + (D x E)` and `E  ~=  D + E`. -/
theorem exists_simultaneous_subsystems :
    exists  D E : ScottSys, D.sys ? (D.sum (D.prod E)).sys  /\  E.sys ? (D.sum E).sys
    :=
  <Dsol, Esol, Dsol_subsystem, Esol_subsystem>

end Exercise624
```

```
end

end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.Exercise624
section
set_option linter.unusedVariables false

theorem example_624 : exists  (D : DomainTheory.NeighbourhoodSystems.Exercise619.
    ScottSys) (E :
  DomainTheory.NeighbourhoodSystems.Exercise619.ScottSys),
  And
    (@DomainTheory.NeighbourhoodSystems.Subsystem DomainTheory.NeighbourhoodSystems.
    ExampleB.Str D.sys
      (D.sum (D.prod E)).sys)
    (@DomainTheory.NeighbourhoodSystems.Subsystem DomainTheory.NeighbourhoodSystems.
    ExampleB.Str E.sys (D.sum E).sys) :=
  DomainTheory.NeighbourhoodSystems.Exercise624.exists_simultaneous_subsystems
end

end DomainTheory.NeighbourhoodSystems.Exercise624
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.Exercise619
open DomainTheory.NeighbourhoodSystems.Example62 DomainTheory.NeighbourhoodSystems.
    ExampleB
open DomainTheory.NeighbourhoodSystems.Exercise510

section
namespace Exercise619

variable {D : NeighborhoodSystem Str}

/-- The master neighbourhood `{?}  U  0Delta` of the lift. -/
def liftTokMaster (D : NeighborhoodSystem Str) : Set Str := insert [] (embBit false D.
    master)

theorem embF_subset_liftTokMaster {X : Set Str} (hX : D.mem X) :
    embBit false X  subseteq  liftTokMaster D :=
  (embBit_subset.mpr (D.sub_master hX)).trans (Set.subset_insert _ _)

theorem liftTokMaster_inter_embF {X : Set Str} (hX : D.mem X) :
    liftTokMaster D  I  embBit false X = embBit false X :=
  Set.inter_eq_right.mpr (embF_subset_liftTokMaster hX)

theorem nil_mem_liftTokMaster : ([] : Str)  in  liftTokMaster D := Set.mem_insert _ _

theorem embF_ne_liftTokMaster {X : Set Str} : embBit false X  !=  liftTokMaster D :=
    fun h =>
  nil_not_mem_embBit (h.symm  |>  nil_mem_liftTokMaster)

end Exercise619
end
end DomainTheory.NeighbourhoodSystems
end
```

3.7.27 Exercise 6.25 — projection pair laws

For a projection pair $g : \mathcal{D} \to \mathcal{E}$ and $h : \mathcal{E} \to \mathcal{D}$ show that for $x \in |\mathcal{D}|$ and $y \in |\mathcal{E}|$ we have:

$$g(x) \sqsubseteq y \text{ iff } x \sqsubseteq h(y).$$

Thus, conclude that:

$$h(y) = \bigsqcup \{x \in |\mathcal{D}| \mid g(x) \sqsubseteq y\}, \quad \text{and}$$

$$g(x) = \bigsqcap \{y \in |\mathcal{E}| \mid x \sqsubseteq h(y)\},$$

for all $x \in |\mathcal{D}|$ and $y \in |\mathcal{E}|$. So each of the functions determines the other. In the first equation check that the set on the right is directed, and in the second equation that the set on the right is non empty. Prove also that g maps consistent sets to consistent sets and preserves $\bigsqcup$ (not just directed unions).

Exercise_6_25.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.Lemma_6_15

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_13
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_27

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap Subsystem
section
variable {a : Type*} {D E : NeighborhoodSystem a}
namespace Subsystem.ProjectionPair

/- concordance: source L4175 EXERCISE 6.25 projection pair laws -/
/-- **Exercise 6.25 -- `g` maps consistent sets to consistent sets.** "Consistent" for
    sets of
elements is "bounded" (Exercise 1.27). If `S` is bounded by `b`, its image `{g(s) ? s
    in  S}` is
bounded by `g(b)` (monotonicity). -/
theorem inj_bounded (P : ProjectionPair D E) {S : Set D.Element} (hS : D.Bounded S) :
    E.Bounded (P.inj.toElementMap '' S) := by
  obtain <b, hb> := hS
  refine <P.inj.toElementMap b, ?_>
  rintro _ <s, hs, rfl>
  exact P.inj.toElementMap_mono (hb s hs)

/-- **`g(h(y))  sqsub  y`.** The second projection-pair law `g  comp  h  subseteq  I_
    ?`, read on elements. -/
theorem inj_proj_apply_le (P : ProjectionPair D E) (y : E.Element) :
    P.inj.toElementMap (P.proj.toElementMap y)  <=  y := by
  have h := (le_iff_toElementMap_le.mp P.inj_comp_proj_le) y
```

```
  rwa [toElementMap_comp, toElementMap_idMap] at h

/-- **`h(g(x)) = x`.** The first projection-pair law `h  comp  g = I_?`, read on
    elements. -/
theorem proj_inj_apply (P : ProjectionPair D E) (x : D.Element) :
    P.proj.toElementMap (P.inj.toElementMap x) = x := by
  rw [ <-  toElementMap_comp, P.proj_comp_inj, toElementMap_idMap]

/-- **Exercise 6.25, the Galois connection.** For `x  in  |?|`, `y  in  |?|`:
`g(x)  sqsub  y  <->  x  sqsub  h(y)`. So `g` (lower adjoint) and `h` (upper adjoint)
    determine each other.

` -> `: apply the monotone `h` to `g(x)  sqsub  y` and use `h(g(x)) = x`.
` <- `: apply the monotone `g` to `x  sqsub  h(y)` and use `g(h(y))  sqsub  y`. -/
theorem galois (P : ProjectionPair D E) (x : D.Element) (y : E.Element) :
    P.inj.toElementMap x  <=  y  <->  x  <=  P.proj.toElementMap y := by
  constructor
  * intro hxy
    have := P.proj.toElementMap_mono hxy
    rwa [P.proj_inj_apply x] at this
  * intro hxy
    exact (P.inj.toElementMap_mono hxy).trans (P.inj_proj_apply_le y)

/-- **Exercise 6.25 -- `g` preserves ` sqsup ` (not just *directed* unions).** For any
    bounded set `S`,
`g( sqsup S) =  sqsup  {g(s) ? s  in  S}`. This is the characteristic property of a
    lower adjoint.

* `?`: `g( sqsup S)` is an upper bound of `{g(s)}` since `s  sqsub   sqsup S` and `g` is
     monotone.
* ` sqsub `: by the Galois connection it suffices that ` sqsup S  sqsub  h( sqsup {g(s)
    })`, i.e. that each `s  sqsub  h( sqsup {g(s)})`;
  by the Galois connection again this is `g(s)  sqsub   sqsup {g(s)}`, true by `le_sSup
    `. -/
theorem inj_sSup (P : ProjectionPair D E) {S : Set D.Element} (hS : D.Bounded S) :
    P.inj.toElementMap (D.sSup S hS) =
      E.sSup (P.inj.toElementMap '' S) (P.inj_bounded hS) := by
  apply le_antisymm
  * refine (P.galois _ _).mpr (D.sSup_le S hS fun s hs => ?_)
    refine (P.galois s _).mp ?_
    exact E.le_sSup _ (P.inj_bounded hS) <s, hs, rfl>
  * refine E.sSup_le _ (P.inj_bounded hS) ?_
    rintro _ <s, hs, rfl>
    exact P.inj.toElementMap_mono (D.le_sSup S hS hs)

/-- The set `{x ? g(x)  sqsub  y}` is bounded: `h(y)` is an upper bound (it is in fact
    the top). -/
theorem lowerSet_bounded (P : ProjectionPair D E) (y : E.Element) :
    D.Bounded (P.lowerSet y) :=
  <P.proj.toElementMap y, fun _ hx => (P.galois _ y).mp hx>

/-- **Exercise 6.25 -- "check that the set on the right is directed".** Any two
    members of
`{x ? g(x)  sqsub  y}` have a common upper bound *inside the set*, namely `h(y)` itself
     (the down-set of
`h(y)` has `h(y)` as a top, so it is trivially directed). -/
```

```
theorem lowerSet_directed (P : ProjectionPair D E) (y : E.Element) :
    forall  a  in  P.lowerSet y, forall  b  in  P.lowerSet y,
      exists  c  in  P.lowerSet y, a  <=  c  /\  b  <=  c := by
  intro a ha b hb
  exact <P.proj.toElementMap y, P.inj_proj_apply_le y,
    (P.galois a y).mp ha, (P.galois b y).mp hb>

/-- **Exercise 6.25, first extremal formula.** `h(y) =  sqsup  {x  in  |?| ? g(x)
    sqsub  y}`.

`h(y)` is the *greatest* member of the set (`g(h(y))  sqsub  y`, so `h(y)` is in it;
    every member is
` sqsub  h(y)` by the Galois connection), hence it is the least upper bound. -/
theorem proj_eq_sSup (P : ProjectionPair D E) (y : E.Element) :
    P.proj.toElementMap y = D.sSup (P.lowerSet y) (P.lowerSet_bounded y) := by
  apply le_antisymm
  * exact D.le_sSup _ (P.lowerSet_bounded y) (P.inj_proj_apply_le y)
  * exact D.sSup_le _ (P.lowerSet_bounded y) fun x hx => (P.galois x y).mp hx

/-- **Exercise 6.25 -- "check that the set on the right is non-empty".** `g(x)` itself
    lies in
`{y ? x  sqsub  h(y)}`, since `x  sqsub  h(g(x)) = x`. -/
theorem upperSet_nonempty (P : ProjectionPair D E) (x : D.Element) :
    (P.upperSet x).Nonempty :=
  <P.inj.toElementMap x, by rw [mem_upperSet, P.proj_inj_apply x]>

/-- **Exercise 6.25, second extremal formula.** `g(x) =  sqcap  {y  in  |?| ? x  sqsub
     h(y)}`.

`g(x)` is the *least* member of the set (`x  sqsub  h(g(x)) = x`, so `g(x)` is in it;
    every member is
`? g(x)` by the Galois connection), hence it is the greatest lower bound. -/
theorem inj_eq_sInf (P : ProjectionPair D E) (x : D.Element) :
    P.inj.toElementMap x = E.sInf (P.upperSet x) (P.upperSet_nonempty x) := by
  apply le_antisymm
  * exact E.le_sInf _ (P.upperSet_nonempty x) _ fun y hy => (P.galois x y).mpr hy
  * refine E.sInf_le _ (P.upperSet_nonempty x) ?_
    rw [mem_upperSet, P.proj_inj_apply x]

end Subsystem.ProjectionPair
end

end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair
section
set_option linter.unusedVariables false

theorem example_625.{u_1} : And
  (And
    (And
      (And
        (And
          (And
            (forall  {a : Type u_1} {D E : DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem a}
              (P : @DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair a D E)
```

```
          (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a D)
          (y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E),
          Iff
            (@LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
 a E)
              (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a E)
                (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a E)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
instPartialOrderElement a E)))
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a
 D E
                (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.inj a
 D E P) x)
              y)
            (@LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
 a D)
              (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a D)
                (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a D)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
instPartialOrderElement a D)))
              x
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a
 E D
                (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.proj
a D E P) y)))
        (forall  {a : Type u_1} {D E : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
          (P : @DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair a D E)
          {S : Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
a D)},
          @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Bounded a D S  ->
            @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Bounded a E
              (@Set.image (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a D)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E
)
                (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a
 a D E
                  (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.inj
 a D E P))
                S)))
      (forall  {a : Type u_1} {D E : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
        (P : @DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair a D E)
        (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a D),
        @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E)
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a D E
            (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.inj a D E
 P) x)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.sInf a E
            (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.upperSet
a D E P x)
```

```
            (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.
upperSet_nonempty a D E P x))))
    (forall  {a : Type u_1} {D E : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
      (P : @DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair a D E)
      {S : Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a D)
}
      (hS : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Bounded a D S),
      @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E)
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a D E
          (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.inj a D E P
)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.sSup a D S hS))
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.sSup a E
          (@Set.image (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a D)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E)
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a D
 E
              (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.inj a D
 E P))
            S)
          (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.inj_bounded
 a D E P S hS))))
  (forall  {a : Type u_1} {D E : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
    (P : @DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair a D E)
    (y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E)
    (a : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a D),
    @Membership.mem (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
 a D)
      (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a D))
      (@Set.instMembership (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a D))
      (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.lowerSet a D
E P y) a  ->
    forall  (b : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
 D),
      @Membership.mem (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a D)
        (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
D))
        (@Set.instMembership (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a D))
        (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.lowerSet
a D E P y) b  ->
        exists  (c : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a D),
          And
            (@Membership.mem (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a D)
              (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a D))
              (@Set.instMembership (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a D))
              (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.
```

```
  lowerSet a D E P y) c)
                (And
                  (@LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  Element a D)
                    (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a D)
                      (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a D)
                        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement a D)))
                    a c)
                  (@LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  Element a D)
                    (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a D)
                      (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a D)
                        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement a D)))
                    b c))))
   (forall  {a : Type u_1} {D E : DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem a}
     (P : @DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair a D E)
     (y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E),
     @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a D)
       (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a E D
         (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.proj a D E P) y
  )
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.sSup a D
         (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.lowerSet a D E
  P y)
         (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.
  lowerSet_bounded a D E P y))))
(forall  {a : Type u_1} {D E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
  a}
   (P : @DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair a D E)
   (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a D),
   @Set.Nonempty (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E)
     (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.upperSet a D E P x)
  ) :=
@And.intro
(And
  (And
    (And
      (And
        (And
          (forall  {a : Type u_1} {D E : DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem a}
            (P : @DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair a D E)
            (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a D)
            (y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E),
            Iff
              (@LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
   a E)
                (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a E)
```

```
                (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a E)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
instPartialOrderElement a E)))
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a
 D E
                (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.inj a
 D E P) x)
              y)
            (@LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
 a D)
              (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a D)
                (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a D)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
instPartialOrderElement a D)))
              x
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a
 E D
                (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.proj
a D E P) y)))
        (forall  {a : Type u_1} {D E : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
          (P : @DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair a D E)
          {S : Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
a D)},
          @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Bounded a D S  ->
            @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Bounded a E
              (@Set.image (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a D)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E
)
                (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a
 a D E
                  (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.inj
 a D E P))
                S)))
      (forall  {a : Type u_1} {D E : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
        (P : @DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair a D E)
        (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a D),
        @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E)
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a D E
            (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.inj a D E
 P) x)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.sInf a E
            (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.upperSet
a D E P x)
            (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.
upperSet_nonempty a D E P x))))
    (forall  {a : Type u_1} {D E : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
      (P : @DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair a D E)
      {S : Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a D)
}
```

```
      (hS : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Bounded a D S),
      @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E)
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a D E
          (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.inj a D E P
)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.sSup a D S hS))
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.sSup a E
          (@Set.image (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a D)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E)
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a D
 E
              (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.inj a D
 E P))
            S)
          (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.inj_bounded
 a D E P S hS))))
  (forall  {a : Type u_1} {D E : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
    (P : @DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair a D E)
    (y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E)
    (a : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a D),
    @Membership.mem (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
 a D)
      (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a D))
      (@Set.instMembership (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a D))
      (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.lowerSet a D
E P y) a  ->
    forall  (b : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
 D),
      @Membership.mem (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a D)
        (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
D))
        (@Set.instMembership (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a D))
        (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.lowerSet
a D E P y) b  ->
        exists  (c : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a D),
          And
            (@Membership.mem (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a D)
              (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a D))
              (@Set.instMembership (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a D))
              (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.
lowerSet a D E P y) c)
            (And
              (@LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a D)
                (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a D)
                  (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
```

```
 NeighborhoodSystem.Element a D)
                        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 instPartialOrderElement a D)))
                   a c)
                 (@LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 Element a D)
                   (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem.Element a D)
                     (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem.Element a D)
                       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 instPartialOrderElement a D)))
                   b c))))
  (forall  {a : Type u_1} {D E : DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem a}
    (P : @DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair a D E)
    (y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E),
    @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a D)
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a E D
        (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.proj a D E P) y
 )
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.sSup a D
        (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.lowerSet a D E
 P y)
        (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.
 lowerSet_bounded a D E P y))))
(forall  {a : Type u_1} {D E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
 a}
  (P : @DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair a D E)
  (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a D),
  @Set.Nonempty (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E)
    (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.upperSet a D E P x)
 )
(@And.intro
  (And
    (And
      (And
        (And
          (forall  {a : Type u_1} {D E : DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem a}
            (P : @DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair a D E)
            (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a D)
            (y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E),
            Iff
              (@LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
  a E)
                (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem.Element a E)
                  (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem.Element a E)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 instPartialOrderElement a E)))
                (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a
  D E
                  (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.inj a
  D E P) x)
```

```
             y)
            (@LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
 a D)
              (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a D)
                (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a D)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
instPartialOrderElement a D)))
              x
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a
 E D
                (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.proj
a D E P) y)))
        (forall  {a : Type u_1} {D E : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
          (P : @DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair a D E)
          {S : Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
a D)},
          @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Bounded a D S  ->
            @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Bounded a E
              (@Set.image (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a D)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E
)
                (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a
 a D E
                  (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.inj
 a D E P))
                S)))
      (forall  {a : Type u_1} {D E : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
        (P : @DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair a D E)
        (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a D),
        @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E)
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a D E
            (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.inj a D E
 P) x)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.sInf a E
            (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.upperSet
a D E P x)
            (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.
upperSet_nonempty a D E P x))))
    (forall  {a : Type u_1} {D E : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
      (P : @DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair a D E)
      {S : Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a D)
}
      (hS : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Bounded a D S),
      @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E)
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a D E
          (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.inj a D E P
)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.sSup a D S hS))
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.sSup a E
          (@Set.image (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
```

```
Element a D)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E)
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a D
 E
              (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.inj a D
 E P))
            S)
          (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.inj_bounded
 a D E P S hS))))
  (forall  {a : Type u_1} {D E : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
    (P : @DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair a D E)
    (y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E)
    (a : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a D),
    @Membership.mem (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
 a D)
        (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a D))
        (@Set.instMembership (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a D))
        (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.lowerSet a D
E P y) a  ->
      forall  (b : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
 D),
        @Membership.mem (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a D)
            (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
D))
            (@Set.instMembership (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a D))
            (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.lowerSet
a D E P y) b  ->
          exists  (c : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a D),
            And
              (@Membership.mem (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a D)


                (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a D))
                (@Set.instMembership (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a D))
                (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.
lowerSet a D E P y) c)
              (And
                (@LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a D)
                  (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a D)
                    (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a D)
                      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
instPartialOrderElement a D)))
                  a c)
                (@LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a D)
                  (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.
```

```
NeighborhoodSystem.Element a D)
                    (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a D)
                      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
instPartialOrderElement a D)))
                  b c))))
 (forall  {a : Type u_1} {D E : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
   (P : @DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair a D E)
   (y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E),
   @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a D)
     (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a E D
       (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.proj a D E P) y
)
     (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.sSup a D
       (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.lowerSet a D E
P y)
       (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.
lowerSet_bounded a D E P y)))
 (@And.intro
   (And
     (And
       (And
         (forall  {a : Type u_1} {D E : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
           (P : @DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair a D E)
           (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a D)
           (y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E),
           Iff
             (@LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
 a E)
               (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a E)
                 (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a E)
                   (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
instPartialOrderElement a E)))
               (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a
 D E
                 (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.inj a
 D E P) x)
               y)
             (@LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
 a D)
               (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a D)
                 (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a D)
                   (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
instPartialOrderElement a D)))
               x
               (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a
 E D
                 (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.proj
a D E P) y)))
         (forall  {a : Type u_1} {D E : DomainTheory.NeighbourhoodSystems.
```

```
NeighborhoodSystem a}
          (P : @DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair a D E)
          {S : Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
a D)},
          @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Bounded a D S  ->
            @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Bounded a E
              (@Set.image (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a D)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E
)
                (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a
 a D E
                  (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.inj
 a D E P))
                S)))
      (forall  {a : Type u_1} {D E : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
        (P : @DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair a D E)
        (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a D),
        @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E)
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a D E
            (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.inj a D E
 P) x)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.sInf a E
            (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.upperSet
a D E P x)
            (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.
upperSet_nonempty a D E P x))))
    (forall  {a : Type u_1} {D E : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
      (P : @DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair a D E)
      {S : Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a D)
}
      (hS : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Bounded a D S),
      @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E)
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a D E
          (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.inj a D E P
)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.sSup a D S hS))
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.sSup a E
          (@Set.image (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a D)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E)
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a D
 E
              (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.inj a D
 E P))
            S)
          (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.inj_bounded
 a D E P S hS))))
  (forall  {a : Type u_1} {D E : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
    (P : @DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair a D E)
    (y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E)
    (a : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a D),
    @Membership.mem (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
```

```
 a D)
        (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a D))
        (@Set.instMembership (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a D))
        (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.lowerSet a D
E P y) a  ->
      forall  (b : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
 D),
        @Membership.mem (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a D)
            (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
D))
            (@Set.instMembership (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a D))
            (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.lowerSet
a D E P y) b  ->
          exists  (c : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a D),
            And
              (@Membership.mem (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a D)
                (Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a D))
                (@Set.instMembership (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a D))
                (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.
lowerSet a D E P y) c)
              (And
                (@LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a D)
                  (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a D)
                    (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a D)
                      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
instPartialOrderElement a D)))
                  a c)
                (@LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a D)
                  (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a D)
                    (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a D)
                      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
instPartialOrderElement a D)))
                  b c)))
  (@And.intro
    (And
      (And
        (forall  {a : Type u_1} {D E : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
          (P : @DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair a D E)
          (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a D)
          (y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E),
          Iff
            (@LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
```

```
 a E)
              (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a E)
                (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a E)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
instPartialOrderElement a E)))
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a
 D E
                (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.inj a
 D E P) x)
              y)
            (@LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
 a D)
              (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a D)
                (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a D)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
instPartialOrderElement a D)))
              x
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a
 E D
                (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.proj
a D E P) y)))
        (forall  {a : Type u_1} {D E : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
          (P : @DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair a D E)
          {S : Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
a D)},
          @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Bounded a D S  ->
            @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Bounded a E
              (@Set.image (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a D)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E
)
                (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a
 a D E
                  (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.inj
 a D E P))
                S)))
      (forall  {a : Type u_1} {D E : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
        (P : @DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair a D E)
        (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a D),
        @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E)
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a D E
            (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.inj a D E
 P) x)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.sInf a E
            (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.upperSet
a D E P x)
            (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.
upperSet_nonempty a D E P x))))
    (forall  {a : Type u_1} {D E : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
```

```
      (P : @DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair a D E)
      {S : Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a D)
}
      (hS : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Bounded a D S),
      @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E)
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a D E
          (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.inj a D E P
)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.sSup a D S hS))
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.sSup a E
          (@Set.image (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a D)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E)
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a D
 E
              (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.inj a D
 E P))
            S)
          (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.inj_bounded
 a D E P S hS)))
    (@And.intro
      (And
        (forall  {a : Type u_1} {D E : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
          (P : @DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair a D E)
          (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a D)
          (y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E),
          Iff
            (@LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
 a E)
              (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a E)
                (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a E)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
instPartialOrderElement a E)))
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a
 D E
                (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.inj a
 D E P) x)
              y)
            (@LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
 a D)
              (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a D)
                (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a D)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
instPartialOrderElement a D)))
              x
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a
 E D
                (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.proj
a D E P) y)))
        (forall  {a : Type u_1} {D E : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
```

```
          (P : @DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair a D E)
          {S : Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
a D)},
          @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Bounded a D S  ->
            @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Bounded a E
              (@Set.image (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a D)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E
)
                (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a
 a D E
                  (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.inj
 a D E P))
                S)))
      (forall  {a : Type u_1} {D E : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
        (P : @DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair a D E)
        (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a D),
        @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E)
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a D E
            (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.inj a D E
 P) x)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.sInf a E
            (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.upperSet
a D E P x)
            (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.
upperSet_nonempty a D E P x)))
      (@And.intro
        (forall  {a : Type u_1} {D E : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
          (P : @DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair a D E)
          (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a D)
          (y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E),
          Iff
            (@LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
 a E)
              (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a E)
                (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a E)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
instPartialOrderElement a E)))
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a
 D E
                (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.inj a
 D E P) x)
              y)
            (@LE.le (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
 a D)
              (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a D)
                (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a D)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
instPartialOrderElement a D)))
              x
```

```
                (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a
     E D
                  (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.proj
    a D E P) y)))
            (forall  {a : Type u_1} {D E : DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem a}
              (P : @DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair a D E)
              {S : Set (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
    a D)},
              @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Bounded a D S  ->
                @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Bounded a E
                  (@Set.image (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    Element a D)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E
    )
                    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a
     a D E
                      (@DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.inj
     a D E P))
                    S))
            @DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.galois
            @DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.inj_bounded)
          @DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.inj_eq_sInf)
        @DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.inj_sSup)
      @DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.lowerSet_directed)
    @DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.proj_eq_sSup)
  @DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair.upperSet_nonempty
end

end DomainTheory.NeighbourhoodSystems.Subsystem.ProjectionPair
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

section
variable {a b : Type*} {D : NeighborhoodSystem a} {E : NeighborhoodSystem b}

/-- **A neighbourhood system is *finite*** when it has only finitely many
    neighbourhoods. -/
def NeighborhoodSystem.IsFinite {a : Type*} (D : NeighborhoodSystem a) : Prop :=
  Finite {X : Set a // D.mem X}

/-- **The element-wise map of an injection `i` is an order embedding.** Given
    approximable maps
`i : D  ->  E`, `j : E  ->  D` with `j  comp  i = I_D`, the map `x  |->  i(x)` on
    elements is an order embedding
`|D| ?o |E|`: monotone by `toElementMap_mono`, and order-reflecting because `j(*)` is a
     monotone
left inverse (`j(i(x)) = x`). Choice-free. -/
def projElementEmbedding {a b : Type*} {D : NeighborhoodSystem a} {E :
    NeighborhoodSystem b}
    (i : ApproximableMap D E) (j : ApproximableMap E D) (hji : j.comp i = idMap D) :
    D.Element ?o E.Element :=
  OrderEmbedding.ofMapLEIff i.toElementMap (by
    have hround : forall  x : D.Element, j.toElementMap (i.toElementMap x) = x := by
      intro x; rw [ <-  toElementMap_comp, hji, toElementMap_idMap]
    intro a b
```

```
    constructor
    * intro h
      have hjm := toElementMap_mono j h
      rwa [hround a, hround b] at hjm
    * intro h
      exact toElementMap_mono i h)

/-- **`D ? E` yields an order embedding `|D| ?o |E|`.** Unfold `?` to an iso `|D|  ~= o
     |D'|` onto a
subdomain `D' ? E`, turn `D' ? E` into Proposition 6.12's projection pair, and compose
    its
injection embedding with the iso. Choice-free. -/
theorem Trianglelefteq.elementEmbedding (h : D ? E) :
    Nonempty (D.Element ?o E.Element) := by
  obtain <D', hsub, <e>> := h
  exact <e.toOrderEmbedding.trans
    (projElementEmbedding hsub.inj hsub.proj hsub.proj_comp_inj)>

/-- **A finite system has a finite element domain.** A filter `x : |D|` is determined
    by which of
the (finitely many) neighbourhoods it contains, so `x  |->  {p | x.mem p.1}` injects `|
    D|` into the
finite powerset `Set {X // D.mem X}`. -/
theorem finite_element_of_isFinite {a : Type*} {D : NeighborhoodSystem a} (h : D.
    IsFinite) :
    Finite D.Element := by
  have : Finite {X : Set a // D.mem X} := h
  apply Finite.of_injective
    (b := Set {X : Set a // D.mem X}) (fun x => {p | x.mem p.1})
  intro x y hxy
  apply Element.ext
  intro X
  by_cases hX : D.mem X
  * have := Set.ext_iff.mp hxy <X, hX>
    simpa using this
  * exact <fun hmem => absurd (x.sub hmem) hX, fun hmem => absurd (y.sub hmem) hX>

/-- **Finite Cantor-Schroder-Bernstein (order version).** Mutual order embeddings
    between two
*finite* ordered types yield an order isomorphism. The embeddings are injective, so
    equal finite
cardinality forces `f` to be surjective; a surjective strictly-monotone map is an order
     iso. Uses
`Classical.choice` (via `Fintype`/surjection-section). -/
theorem orderIso_of_embeddings {P Q : Type*} [PartialOrder P] [PartialOrder Q]
    [Finite P] [Finite Q] (f : P ?o Q) (g : Q ?o P) : Nonempty (P ?o Q) := by
  classical
  obtain <_> := nonempty_fintype P
  obtain <_> := nonempty_fintype Q
  have hcard : Fintype.card P = Fintype.card Q :=
    le_antisymm (Fintype.card_le_of_injective f f.injective)
      (Fintype.card_le_of_injective g g.injective)
  have hbij : Function.Bijective f :=
    (Fintype.bijective_iff_injective_and_card f).mpr <f.injective, hcard>
  exact <{ toEquiv := Equiv.ofBijective f hbij, map_rel_iff' := f.map_rel_iff' }>
```

```
end
end DomainTheory.NeighbourhoodSystems
end
```

3.7.28 Exercise 6.26 — lifting a domain

For systems $\mathcal{D}$ as in 6.19 define

$$\mathcal{D}_\perp = \{\{\Lambda\} \cup 0\Delta\} \cup \{0X \mid X \in \mathcal{D}\}.$$

Describe the construct in terms of elements. Is this a suitable functor? Prove that

$$\mathcal{D}_\perp \oplus \mathcal{E}_\perp \cong \mathcal{D} + \mathcal{E}.$$

What is

$$\mathcal{D}_\perp \otimes \mathcal{E}_\perp \cong ??$$

Exercise_6_26.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.Exercise_6_24

public import DomainTheory.NeighbourhoodSystems.DomainEquations.Exercise_6_21
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_10
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_7

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.Exercise619
open DomainTheory.NeighbourhoodSystems.Example62 DomainTheory.NeighbourhoodSystems.
    ExampleB
open DomainTheory.NeighbourhoodSystems.Exercise510
section
namespace Exercise619
variable {D : NeighborhoodSystem Str}
/- concordance: source L4187 EXERCISE 6.26 lifting a domain -/
/-- **Exercise 6.26 -- the lifted system `?_False` over `{0,1}*`.** A neighbourhood is
    the master
`{?}  U  0Delta` or a tagged copy `0X` (`X  in  ?`). `{}`-freeness of `?` (`hD`) keeps
    it `{}`-free. -/
def liftTok (D : NeighborhoodSystem Str) (_hD : forall  X, D.mem X  ->  X.Nonempty) :
    NeighborhoodSystem Str where
  mem W := W = liftTokMaster D  \/  exists  X, D.mem X  /\  W = embBit false X
  master := liftTokMaster D
  master_nonempty := <[], nil_mem_liftTokMaster>
  master_mem := Or.inl rfl
  sub_master := by
    rintro W (rfl | <X, hX, rfl>)
    * exact subset_rfl
```

```
    * exact embF_subset_liftTokMaster hX
  inter_mem := by
    rintro W W' Z hW hW' hZ hZsub
    rcases hW with rfl | <X, hX, rfl>
    * rcases hW' with rfl | <X', hX', rfl>
      * rw [Set.inter_self]; exact Or.inl rfl
      * rw [liftTokMaster_inter_embF hX']; exact Or.inr <X', hX', rfl>
    * rcases hW' with rfl | <X', hX', rfl>
      * rw [Set.inter_comm, liftTokMaster_inter_embF hX]; exact Or.inr <X, hX, rfl>
      * rw [embBit_inter] at hZsub  |-
        rcases hZ with rfl | <Z', hZ', rfl>
        * exact absurd (hZsub nil_mem_liftTokMaster) nil_not_mem_embBit
        * exact Or.inr <X  I  X', D.inter_mem hX hX' hZ' (embBit_subset.mp hZsub), rfl
    >

end Exercise619
end

section
namespace Exercise619

variable {D : NeighborhoodSystem Str}

variable {hD : forall  X, D.mem X  ->  X.Nonempty}

/-- The **fresh bottom** of `?_False`: the element whose only neighbourhood is the
    master `{?}  U  0Delta`. -/
def liftBot (D : NeighborhoodSystem Str) (hD : forall  X, D.mem X  ->  X.Nonempty) :
    (liftTok D hD).Element where
  mem W := W = liftTokMaster D
  sub := by rintro W rfl; exact Or.inl rfl
  master_mem := rfl
  inter_mem := by rintro W W' rfl rfl; rw [Set.inter_self]
  up_mem := by
    rintro W W' rfl hW' hsub
    exact Set.Subset.antisymm ((liftTok D hD).sub_master hW') hsub

theorem liftTok_mem_embF {X : Set Str} (hX : D.mem X) :
    (liftTok D hD).mem (embBit false X) := Or.inr <X, hX, rfl>

theorem liftTok_mem_embF_inv {W : Set Str} (h : (liftTok D hD).mem (embBit false W)) :
     D.mem W := by
  rcases h with h0 | <X, hX, heq>
  * exact absurd (h0.symm  |>  nil_mem_liftTokMaster) nil_not_mem_embBit
  * rw [embBit_injective heq]; exact hX

end Exercise619
end

section
namespace Exercise619

variable {D : NeighborhoodSystem Str}

theorem liftTok_nonempty (hD : forall  X, D.mem X  ->  X.Nonempty) :
    forall  W, (liftTok D hD).mem W  ->  W.Nonempty := by
```

```
  rintro W (rfl | <X, hX, rfl>)
  * exact <[], nil_mem_liftTokMaster>
  * exact embBit_nonempty (hD X hX)

/-- The **lift object** `?_False` of Scott's category. -/
def ScottSys.lift (A : ScottSys) : ScottSys := <liftTok A.sys A.ne, liftTok_nonempty A
    .ne>

end Exercise619
end

section
namespace Exercise619

variable {D : NeighborhoodSystem Str}

variable {hD : forall  X, D.mem X  ->  X.Nonempty}

/-- The **embedding** `|?| ? |?_False|`: `liftUp x = {{?}  U  0Delta}  U  {0X ? X  in
    x}`, the image of `x`
sitting above the fresh bottom. -/
def liftUp {D : NeighborhoodSystem Str} {hD : forall  X, D.mem X  ->  X.Nonempty} (x :
     D.Element) :
    (liftTok D hD).Element where
  mem W := W = liftTokMaster D  \/  exists  X, x.mem X  /\  W = embBit false X
  sub := by
    rintro W (rfl | <X, hX, rfl>)
    * exact Or.inl rfl
    * exact Or.inr <X, x.sub hX, rfl>
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | <X, hX, rfl>) (rfl | <X', hX', rfl>)
    * exact Or.inl (by rw [Set.inter_self])
    * exact Or.inr <X', hX', by rw [liftTokMaster_inter_embF (x.sub hX')]>
    * exact Or.inr <X, hX, by rw [Set.inter_comm, liftTokMaster_inter_embF (x.sub hX)
    ]>
    * exact Or.inr <X  I  X', x.inter_mem hX hX', by rw [embBit_inter]>
  up_mem := by
    rintro W W' (rfl | <X, hX, rfl>) hW' hsub
    * exact Or.inl (Set.Subset.antisymm ((liftTok D hD).sub_master hW') hsub)
    * rcases hW' with rfl | <X', hX', rfl>
      * exact Or.inl rfl
      * exact Or.inr <X', x.up_mem hX hX' (embBit_subset.mp hsub), rfl>

@[simp] theorem mem_liftBot {W : Set Str} : (liftBot D hD).mem W  <->  W =
    liftTokMaster D := Iff.rfl

@[simp] theorem mem_liftUp {x : D.Element} {W : Set Str} :
    (liftUp (hD := hD) x).mem W  <->  W = liftTokMaster D  \/  exists  X, x.mem X  /\
     W = embBit false X := Iff.rfl

end Exercise619
end

section
namespace Exercise619
```

```
variable {D : NeighborhoodSystem Str}

variable {hD : forall  X, D.mem X  ->  X.Nonempty}

variable {A B C : ScottSys}

/-- **`f_False`, the action of lifting on (approximable) maps.** It carries the master
    to the master (so
it is strict) and a copy `0X` to `0X'` whenever `X f X'`. -/
def liftMapTok (f : ApproximableMap A.sys B.sys) :
    ApproximableMap (ScottSys.lift A).sys (ScottSys.lift B).sys where
  rel W W' :=
    ((liftTok A.sys A.ne).mem W  /\  W' = liftTokMaster B.sys)  \/
    (exists  X X', f.rel X X'  /\  W = embBit false X  /\  W' = embBit false X')
  rel_dom := by
    rintro W W' (<hW, -> | <X, X', hrel, rfl, ->)
    * exact hW
    * exact liftTok_mem_embF (hD := A.ne) (f.rel_dom hrel)
  rel_cod := by
    rintro W W' (<-, rfl> | <X, X', hrel, -, rfl>)
    * exact Or.inl rfl
    * exact liftTok_mem_embF (hD := B.ne) (f.rel_cod hrel)
  master_rel := Or.inl <(ScottSys.lift A).sys.master_mem, rfl>
  inter_right := by
    rintro W W'_1 W'_2 h1 h2
    rcases h1 with <hW, rfl> | <X, X', hrel, rfl, rfl>
    * rcases h2 with <-, rfl> | <X, X', hrel, hWeq, rfl>
      * exact Or.inl <hW, by rw [Set.inter_self]>
      * exact Or.inr <X, X', hrel, hWeq, by rw [liftTokMaster_inter_embF (f.rel_cod
    hrel)]>
    * rcases h2 with <-, rfl> | <X_2, X'_2, hrel_2, hWeq, rfl>
      * refine Or.inr <X, X', hrel, rfl, ?_>
        rw [Set.inter_comm, liftTokMaster_inter_embF (f.rel_cod hrel)]
      * obtain rfl := embBit_injective hWeq
        exact Or.inr <X, X'  I  X'_2, f.inter_right hrel hrel_2, rfl, embBit_inter
    false X' X'_2>
  mono := by
    rintro W W'' Z Z' h hWW hZZ' hZmem hZ'mem
    rcases h with <-, rfl> | <X, X', hrel, rfl, rfl>
    * exact Or.inl <hZmem, Set.Subset.antisymm ((ScottSys.lift B).sys.sub_master hZ'
    mem) hZZ'>
    * rcases hZ'mem with rfl | <X_3, hX_3, rfl>
      * exact Or.inl <hZmem, rfl>
      * rcases hZmem with rfl | <X_2, hX_2, rfl>
        * exact absurd (hWW nil_mem_liftTokMaster) nil_not_mem_embBit
        * exact Or.inr <X_2, X_3,
            f.mono hrel (embBit_subset.mp hWW) (embBit_subset.mp hZZ') hX_2 hX_3, rfl,
     rfl>

end Exercise619
end

section
namespace Exercise619
```

```
variable {D : NeighborhoodSystem Str}

variable {hD : forall  X, D.mem X  ->  X.Nonempty}

variable {A B C : ScottSys}

variable {D E : ScottSys}

theorem o_embFF_inv {W : Set Str}
    (h : (D.lift.oplus E.lift).sys.mem (embBit false (embBit false W))) : D.sys.mem W
    :=
  liftTok_mem_embF_inv (hD := D.ne)
    (oplusTok_mem_embF_inv (D_0 := D.lift.sys) (D_1 := E.lift.sys)
      (h_0 := D.lift.ne) (h_1 := E.lift.ne) h)

theorem o_embTF_inv {W : Set Str}
    (h : (D.lift.oplus E.lift).sys.mem (embBit true (embBit false W))) : E.sys.mem W
    :=
  liftTok_mem_embF_inv (hD := E.ne)
    (oplusTok_mem_embT_inv (D_0 := D.lift.sys) (D_1 := E.lift.sys)
      (h_0 := D.lift.ne) (h_1 := E.lift.ne) h)

theorem o_mem_embFF {X' : Set Str} (hX' : D.sys.mem X') :
    (D.lift.oplus E.lift).sys.mem (embBit false (embBit false X')) :=
  oplusTok_mem_embF (h_0 := D.lift.ne) (h_1 := E.lift.ne)
    (liftTok_mem_embF (hD := D.ne) hX') (embF_ne_liftTokMaster (D := D.sys))

theorem o_mem_embTF {Y' : Set Str} (hY' : E.sys.mem Y') :
    (D.lift.oplus E.lift).sys.mem (embBit true (embBit false Y')) :=
  oplusTok_mem_embT (h_0 := D.lift.ne) (h_1 := E.lift.ne)
    (liftTok_mem_embF (hD := E.ne) hY') (embF_ne_liftTokMaster (D := E.sys))

/-- The forward half `|?_False ? ?_False|  ->  |? + ?|`: delete the inner `0`. -/
def toSumLift (z : (D.lift.oplus E.lift).sys.Element) : (D.sum E).sys.Element where
  mem W := W = sumTokMaster D.sys E.sys
     \/  (exists  X, D.sys.mem X  /\  W = embBit false X  /\  z.mem (embBit false (
    embBit false X)))
     \/  (exists  Y, E.sys.mem Y  /\  W = embBit true Y  /\  z.mem (embBit true (
    embBit false Y)))
  sub := by
    rintro W (rfl | <X, hX, rfl, -> | <Y, hY, rfl, ->)
    * exact Or.inl rfl
    * exact sumTok_mem_embF (h_0 := D.ne) (h_1 := E.ne) hX
    * exact sumTok_mem_embT (h_0 := D.ne) (h_1 := E.ne) hY
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | <X, hX, rfl, hzX> | <Y, hY, rfl, hzY>)
      (rfl | <X', hX', rfl, hzX'> | <Y', hY', rfl, hzY'>)
    * exact Or.inl (by rw [Set.inter_self])
    * exact Or.inr (Or.inl <X', hX', by rw [sumTokMaster_inter_embF hX'], hzX'>)
    * exact Or.inr (Or.inr <Y', hY', by rw [sumTokMaster_inter_embT hY'], hzY'>)
    * exact Or.inr (Or.inl <X, hX, by rw [Set.inter_comm, sumTokMaster_inter_embF hX],
     hzX>)
    * refine Or.inr (Or.inl <X  I  X', ?_, by rw [embBit_inter], ?_>)
      * have hz := z.inter_mem hzX hzX'; rw [embBit_inter, embBit_inter] at hz
        exact o_embFF_inv (z.sub hz)
```

```
      * have hz := z.inter_mem hzX hzX'; rwa [embBit_inter, embBit_inter] at hz
    * exfalso
      have hz := z.inter_mem hzX hzY'
      rw [embBit_inter_ne (show (false : Bool)  !=  true by decide)] at hz
      obtain <t, ht> := (D.lift.oplus E.lift).ne _ (z.sub hz); exact Set.notMem_empty
    t ht
    * exact Or.inr (Or.inr <Y, hY, by rw [Set.inter_comm, sumTokMaster_inter_embT hY],
     hzY>)
    * exfalso
      have hz := z.inter_mem hzY hzX'
      rw [embBit_inter_ne (show (true : Bool)  !=  false by decide)] at hz
      obtain <t, ht> := (D.lift.oplus E.lift).ne _ (z.sub hz); exact Set.notMem_empty
    t ht
    * refine Or.inr (Or.inr <Y  I  Y', ?_, by rw [embBit_inter], ?_>)
      * have hz := z.inter_mem hzY hzY'; rw [embBit_inter, embBit_inter] at hz
        exact o_embTF_inv (z.sub hz)
      * have hz := z.inter_mem hzY hzY'; rwa [embBit_inter, embBit_inter] at hz
  up_mem := by
    rintro W W' (rfl | <X, hX, rfl, hzX> | <Y, hY, rfl, hzY>) hW' hsub
    * exact Or.inl (Set.Subset.antisymm ((D.sum E).sys.sub_master hW') hsub)
    * rcases hW' with rfl | <X'', hX'', rfl> | <Y'', hY'', rfl>
      * exact Or.inl rfl
      * refine Or.inr (Or.inl <X'', hX'', rfl, ?_>)
        exact z.up_mem hzX (o_mem_embFF hX'')
          (embBit_subset.mpr (embBit_subset.mpr (embBit_subset.mp hsub)))
      * exact absurd hsub
          (fun hs => embBit_not_subset_cross (show (false : Bool)  !=  true by decide)
     (D.ne X hX) hs)
    * rcases hW' with rfl | <X'', hX'', rfl> | <Y'', hY'', rfl>
      * exact Or.inl rfl
      * exact absurd hsub
          (fun hs => embBit_not_subset_cross (show (true : Bool)  !=  false by decide)
     (E.ne Y hY) hs)
      * refine Or.inr (Or.inr <Y'', hY'', rfl, ?_>)
        exact z.up_mem hzY (o_mem_embTF hY'')
          (embBit_subset.mpr (embBit_subset.mpr (embBit_subset.mp hsub)))

@[simp] theorem toSumLift_mem_embF {z : (D.lift.oplus E.lift).sys.Element} {X : Set
    Str}
    (hX : D.sys.mem X) :
    (toSumLift z).mem (embBit false X)  <->  z.mem (embBit false (embBit false X)) :=
    by
  constructor
  * rintro (h0 | <X', hX', heq, hz> | <Y', hY', heq, hz>)
    * exact absurd h0 embF_ne_sumTokMaster
    * rwa [embBit_injective heq]
    * exact absurd heq (embBit_ne (show (false : Bool)  !=  true by decide) (D.ne X hX
    ))
  * intro hz; exact Or.inr (Or.inl <X, hX, rfl, hz>)

@[simp] theorem toSumLift_mem_embT {z : (D.lift.oplus E.lift).sys.Element} {Y : Set
    Str}
    (hY : E.sys.mem Y) :
    (toSumLift z).mem (embBit true Y)  <->  z.mem (embBit true (embBit false Y)) := by
  constructor
  * rintro (h0 | <X', hX', heq, hz> | <Y', hY', heq, hz>)
```

```
    * exact absurd h0 embT_ne_sumTokMaster
    * exact absurd heq (embBit_ne (show (true : Bool)  !=  false by decide) (E.ne Y hY
    ))
    * rwa [embBit_injective heq]
  * intro hz; exact Or.inr (Or.inr <Y, hY, rfl, hz>)

/-- The inverse half `|? + ?|  ->  |?_False ? ?_False|`: reinstate the inner `0`. -/
def fromSumLift (s : (D.sum E).sys.Element) : (D.lift.oplus E.lift).sys.Element where
  mem W := W = sumTokMaster D.lift.sys E.lift.sys
     \/  (exists  X, D.sys.mem X  /\  W = embBit false (embBit false X)  /\  s.mem (
    embBit false X))
     \/  (exists  Y, E.sys.mem Y  /\  W = embBit true (embBit false Y)  /\  s.mem (
    embBit true Y))
  sub := by
    rintro W (rfl | <X, hX, rfl, -> | <Y, hY, rfl, ->)
    * exact Or.inl rfl
    * exact o_mem_embFF hX
    * exact o_mem_embTF hY
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | <X, hX, rfl, hsX> | <Y, hY, rfl, hsY>)
      (rfl | <X', hX', rfl, hsX'> | <Y', hY', rfl, hsY'>)
    * exact Or.inl (by rw [Set.inter_self])
    * refine Or.inr (Or.inl <X', hX', ?_, hsX'>)
      rw [sumTokMaster_inter_embF (D_0 := D.lift.sys) (D_1 := E.lift.sys)
        (liftTok_mem_embF (hD := D.ne) hX')]
    * refine Or.inr (Or.inr <Y', hY', ?_, hsY'>)
      rw [sumTokMaster_inter_embT (D_0 := D.lift.sys) (D_1 := E.lift.sys)
        (liftTok_mem_embF (hD := E.ne) hY')]
    * refine Or.inr (Or.inl <X, hX, ?_, hsX>)
      rw [Set.inter_comm, sumTokMaster_inter_embF (D_0 := D.lift.sys) (D_1 := E.lift.
    sys)
        (liftTok_mem_embF (hD := D.ne) hX)]
    * refine Or.inr (Or.inl <X  I  X', ?_, by rw [embBit_inter, embBit_inter], ?_>)
      * have hs := s.inter_mem hsX hsX'; rw [embBit_inter] at hs
        exact sumTok_mem_embF_inv (h_0 := D.ne) (h_1 := E.ne) (s.sub hs)
      * have hs := s.inter_mem hsX hsX'; rwa [embBit_inter] at hs
    * exfalso
      have hs := s.inter_mem hsX hsY'
      rw [embBit_inter_ne (show (false : Bool)  !=  true by decide)] at hs
      obtain <t, ht> := sumTok_mem_nonempty (h_0 := D.ne) (h_1 := E.ne) (s.sub hs)
      exact Set.notMem_empty t ht
    * refine Or.inr (Or.inr <Y, hY, ?_, hsY>)
      rw [Set.inter_comm, sumTokMaster_inter_embT (D_0 := D.lift.sys) (D_1 := E.lift.
    sys)
        (liftTok_mem_embF (hD := E.ne) hY)]
    * exfalso
      have hs := s.inter_mem hsY hsX'
      rw [embBit_inter_ne (show (true : Bool)  !=  false by decide)] at hs
      obtain <t, ht> := sumTok_mem_nonempty (h_0 := D.ne) (h_1 := E.ne) (s.sub hs)
      exact Set.notMem_empty t ht
    * refine Or.inr (Or.inr <Y  I  Y', ?_, by rw [embBit_inter, embBit_inter], ?_>)
      * have hs := s.inter_mem hsY hsY'; rw [embBit_inter] at hs
        exact sumTok_mem_embT_inv (h_0 := D.ne) (h_1 := E.ne) (s.sub hs)
      * have hs := s.inter_mem hsY hsY'; rwa [embBit_inter] at hs
  up_mem := by
```

```
    rintro W W' (rfl | <X, hX, rfl, hsX> | <Y, hY, rfl, hsY>) hW' hsub
    * exact Or.inl (Set.Subset.antisymm ((D.lift.oplus E.lift).sys.sub_master hW')
    hsub)
    * rcases hW' with rfl | <V, hV, hVne, rfl> | <V, hV, hVne, rfl>
      * exact Or.inl rfl
      * rcases hV with rfl | <X'', hX''D, rfl>
        * exact absurd rfl hVne
        * refine Or.inr (Or.inl <X'', hX''D, rfl, ?_>)
          exact s.up_mem hsX (sumTok_mem_embF (h_0 := D.ne) (h_1 := E.ne) hX''D)
            (embBit_subset.mpr (embBit_subset.mp (embBit_subset.mp hsub)))
      * exact absurd hsub
          (fun hs => embBit_not_subset_cross (show (false : Bool)  !=  true by decide)
            (embBit_nonempty (D.ne X hX)) hs)
    * rcases hW' with rfl | <V, hV, hVne, rfl> | <V, hV, hVne, rfl>
      * exact Or.inl rfl
      * exact absurd hsub
          (fun hs => embBit_not_subset_cross (show (true : Bool)  !=  false by decide)
            (embBit_nonempty (E.ne Y hY)) hs)
      * rcases hV with rfl | <Y'', hY''E, rfl>
        * exact absurd rfl hVne
        * refine Or.inr (Or.inr <Y'', hY''E, rfl, ?_>)
          exact s.up_mem hsY (sumTok_mem_embT (h_0 := D.ne) (h_1 := E.ne) hY''E)
            (embBit_subset.mpr (embBit_subset.mp (embBit_subset.mp hsub)))

@[simp] theorem fromSumLift_mem_embFF {s : (D.sum E).sys.Element} {X : Set Str} (hX :
    D.sys.mem X) :
    (fromSumLift s).mem (embBit false (embBit false X))  <->  s.mem (embBit false X)
    := by
  constructor
  * rintro (h0 | <X', hX', heq, hs> | <Y', hY', heq, hs>)
    * exact absurd h0 embF_ne_sumTokMaster
    * rwa [embBit_injective (embBit_injective heq)]
    * exact absurd heq (embBit_ne (show (false : Bool)  !=  true by decide)
        (embBit_nonempty (D.ne X hX)))
  * intro hs; exact Or.inr (Or.inl <X, hX, rfl, hs>)

@[simp] theorem fromSumLift_mem_embTF {s : (D.sum E).sys.Element} {Y : Set Str} (hY :
    E.sys.mem Y) :
    (fromSumLift s).mem (embBit true (embBit false Y))  <->  s.mem (embBit true Y) :=
    by
  constructor
  * rintro (h0 | <X', hX', heq, hs> | <Y', hY', heq, hs>)
    * exact absurd h0 embT_ne_sumTokMaster
    * exact absurd heq (embBit_ne (show (true : Bool)  !=  false by decide)
        (embBit_nonempty (E.ne Y hY)))
    * rwa [embBit_injective (embBit_injective heq)]
  * intro hs; exact Or.inr (Or.inr <Y, hY, rfl, hs>)

theorem fromSumLift_toSumLift (z : (D.lift.oplus E.lift).sys.Element) :
    fromSumLift (toSumLift z) = z := by
  apply NeighborhoodSystem.Element.ext
  intro W
  constructor
  * rintro (rfl | <X, hX, rfl, hs> | <Y, hY, rfl, hs>)
    * exact z.master_mem
    * exact (toSumLift_mem_embF hX).mp hs
```

```
    * exact (toSumLift_mem_embT hY).mp hs
  * intro hW
    rcases z.sub hW with rfl | <X, hX, hXne, rfl> | <Y, hY, hYne, rfl>
    * exact Or.inl rfl
    * rcases hX with rfl | <X', hX'D, rfl>
      * exact absurd rfl hXne
      * exact Or.inr (Or.inl <X', hX'D, rfl, (toSumLift_mem_embF hX'D).mpr hW>)
    * rcases hY with rfl | <Y', hY'E, rfl>
      * exact absurd rfl hYne
      * exact Or.inr (Or.inr <Y', hY'E, rfl, (toSumLift_mem_embT hY'E).mpr hW>)

theorem lp_mem_embF {X' Y' : Set Str} (hX' : D.sys.mem X') (hY' : E.sys.mem Y') :
    (D.prod E).lift.sys.mem (embBit false (prodTokNbhd X' Y')) :=
  liftTok_mem_embF (hD := (D.prod E).ne) (prodTok_mem_prodTokNbhd hX' hY')

theorem lp_prod_inv {X' Y' : Set Str}


    (h : (D.prod E).lift.sys.mem (embBit false (prodTokNbhd X' Y'))) :
    D.sys.mem X'  /\  E.sys.mem Y' := by
  obtain <A, B, hA, hB, heq> := liftTok_mem_embF_inv (hD := (D.prod E).ne) h
  obtain <rfl, rfl> := prodTokNbhd_injective heq
  exact <hA, hB>

theorem ot_mem_prod {X' Y' : Set Str} (hX' : D.sys.mem X') (hY' : E.sys.mem Y') :
    (D.lift.otimes E.lift).sys.mem (prodTokNbhd (embBit false X') (embBit false Y'))
    :=
  otimesTok_mem_prod (liftTok_mem_embF (hD := D.ne) hX') (liftTok_mem_embF (hD := E.ne
    ) hY')
    (embF_ne_liftTokMaster (D := D.sys)) (embF_ne_liftTokMaster (D := E.sys))

theorem ot_mem_prod_inv {X' Y' : Set Str}
    (h : (D.lift.otimes E.lift).sys.mem (prodTokNbhd (embBit false X') (embBit false Y
    ''))) :
    D.sys.mem X'  /\  E.sys.mem Y' := by
  obtain <h1, h2> := otimesTok_mem_prod_inv (D_0 := D.lift.sys) (D_1 := E.lift.sys) h
    (embF_ne_liftTokMaster (D := D.sys))
  exact <liftTok_mem_embF_inv (hD := D.ne) h1, liftTok_mem_embF_inv (hD := E.ne) h2>

/-- The forward half `|?_False ? ?_False|  ->  |(?  x  ?)_False|`. -/
def toLiftProd (z : (D.lift.otimes E.lift).sys.Element) : (D.prod E).lift.sys.Element
    where
  mem W := W = liftTokMaster (prodTok D.sys E.sys)
     \/  (exists  X Y, D.sys.mem X  /\  E.sys.mem Y  /\  W = embBit false (prodTokNbhd
    X Y)  /\
        z.mem (prodTokNbhd (embBit false X) (embBit false Y)))
  sub := by
    rintro W (rfl | <X, Y, hX, hY, rfl, ->)
    * exact Or.inl rfl
    * exact lp_mem_embF hX hY
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | <X, Y, hX, hY, rfl, hzXY>) (rfl | <X', Y', hX', hY', rfl, hzXY
    '>)
    * exact Or.inl (by rw [Set.inter_self])
    * refine Or.inr <X', Y', hX', hY', ?_, hzXY'>
      rw [liftTokMaster_inter_embF (prodTok_mem_prodTokNbhd hX' hY')]
```

```
    * refine Or.inr <X, Y, hX, hY, ?_, hzXY>
      rw [Set.inter_comm, liftTokMaster_inter_embF (prodTok_mem_prodTokNbhd hX hY)]
    * have hz := z.inter_mem hzXY hzXY'
      rw [prodTokNbhd_inter, embBit_inter, embBit_inter] at hz
      obtain <hXi, hYi> := ot_mem_prod_inv (z.sub hz)
      refine Or.inr <X  I  X', Y  I  Y', hXi, hYi, ?_, hz>
      rw [embBit_inter, prodTokNbhd_inter]
  up_mem := by
    rintro W W' (rfl | <X, Y, hX, hY, rfl, hzXY>) hW' hsub
    * exact Or.inl (Set.Subset.antisymm ((D.prod E).lift.sys.sub_master hW') hsub)
    * rcases hW' with rfl | <Z, hZ, rfl>
      * exact Or.inl rfl
      * obtain <X'', Y'', hX'', hY'', rfl> := hZ
        refine Or.inr <X'', Y'', hX'', hY'', rfl, ?_>
        obtain <hsX, hsY> := prodTokNbhd_subset_iff.mp (embBit_subset.mp hsub)
        exact z.up_mem hzXY (ot_mem_prod hX'' hY'')
          (prodTokNbhd_subset_iff.mpr <embBit_subset.mpr hsX, embBit_subset.mpr hsY>)

@[simp] theorem toLiftProd_mem_embF {z : (D.lift.otimes E.lift).sys.Element} {X Y :
    Set Str}
    (hX : D.sys.mem X) (hY : E.sys.mem Y) :
    (toLiftProd z).mem (embBit false (prodTokNbhd X Y))  <->
      z.mem (prodTokNbhd (embBit false X) (embBit false Y)) := by
  constructor
  * rintro (h0 | <X', Y', hX', hY', heq, hz>)
    * exact absurd h0 (embF_ne_liftTokMaster (D := prodTok D.sys E.sys))
    * obtain <rfl, rfl> := prodTokNbhd_injective (embBit_injective heq); exact hz
  * intro hz; exact Or.inr <X, Y, hX, hY, rfl, hz>

/-- The inverse half `|(?  x  ?)_False|  ->  |?_False ? ?_False|`. -/
def fromLiftProd (s : (D.prod E).lift.sys.Element) : (D.lift.otimes E.lift).sys.
    Element where
  mem W := W = prodTokNbhd (liftTokMaster D.sys) (liftTokMaster E.sys)
     \/  (exists  X Y, D.sys.mem X  /\  E.sys.mem Y  /\  W = prodTokNbhd (embBit false
     X) (embBit false Y)  /\
        s.mem (embBit false (prodTokNbhd X Y)))
  sub := by
    rintro W (rfl | <X, Y, hX, hY, rfl, ->)
    * exact Or.inl rfl
    * exact ot_mem_prod hX hY
  master_mem := Or.inl rfl
  inter_mem := by
    rintro W W' (rfl | <X, Y, hX, hY, rfl, hsXY>) (rfl | <X', Y', hX', hY', rfl, hsXY
    '>)
    * exact Or.inl (by rw [Set.inter_self])
    * refine Or.inr <X', Y', hX', hY', ?_, hsXY'>
      rw [prodTokNbhd_inter, liftTokMaster_inter_embF hX', liftTokMaster_inter_embF hY
    ']
    * refine Or.inr <X, Y, hX, hY, ?_, hsXY>
      rw [Set.inter_comm, prodTokNbhd_inter, liftTokMaster_inter_embF hX,
        liftTokMaster_inter_embF hY]
    * have hs := s.inter_mem hsXY hsXY'
      rw [embBit_inter, prodTokNbhd_inter] at hs
      obtain <hXi, hYi> := lp_prod_inv (s.sub hs)
      refine Or.inr <X  I  X', Y  I  Y', hXi, hYi, ?_, hs>
      rw [prodTokNbhd_inter, embBit_inter, embBit_inter]
```

```
  up_mem := by
    rintro W W' (rfl | <X, Y, hX, hY, rfl, hsXY>) hW' hsub
    * exact Or.inl (Set.Subset.antisymm ((D.lift.otimes E.lift).sys.sub_master hW')
    hsub)
    * rcases hW' with rfl | <U, V, hU, hV, hUne, hVne, rfl>
      * exact Or.inl rfl
      * rcases hU with rfl | <X'', hX''D, rfl>
        * exact absurd rfl hUne
        * rcases hV with rfl | <Y'', hY''E, rfl>
          * exact absurd rfl hVne
          * refine Or.inr <X'', Y'', hX''D, hY''E, rfl, ?_>
            obtain <hsX, hsY> := prodTokNbhd_subset_iff.mp hsub
            exact s.up_mem hsXY (lp_mem_embF hX''D hY''E)
              (embBit_subset.mpr (prodTokNbhd_subset_iff.mpr
                <embBit_subset.mp hsX, embBit_subset.mp hsY>))

@[simp] theorem fromLiftProd_mem_prod {s : (D.prod E).lift.sys.Element} {X Y : Set Str
    }
    (hX : D.sys.mem X) (hY : E.sys.mem Y) :
    (fromLiftProd s).mem (prodTokNbhd (embBit false X) (embBit false Y))  <->
      s.mem (embBit false (prodTokNbhd X Y)) := by
  constructor
  * rintro (h0 | <X', Y', hX', hY', heq, hs>)
    * obtain <hX0, -> := prodTokNbhd_injective h0
      exact absurd hX0 (embF_ne_liftTokMaster (D := D.sys))
    * obtain <hXe, hYe> := prodTokNbhd_injective heq
      rw [embBit_injective hXe, embBit_injective hYe]; exact hs
  * intro hs; exact Or.inr <X, Y, hX, hY, rfl, hs>

theorem fromLiftProd_toLiftProd (z : (D.lift.otimes E.lift).sys.Element) :
    fromLiftProd (toLiftProd z) = z := by
  apply NeighborhoodSystem.Element.ext
  intro W
  constructor
  * rintro (rfl | <X, Y, hX, hY, rfl, hs>)
    * exact z.master_mem
    * exact (toLiftProd_mem_embF hX hY).mp hs
  * intro hW
    rcases z.sub hW with rfl | <X, Y, hX, hY, hXne, hYne, rfl>
    * exact Or.inl rfl
    * rcases hX with rfl | <X'', hX''D, rfl>
      * exact absurd rfl hXne
      * rcases hY with rfl | <Y'', hY''E, rfl>
        * exact absurd rfl hYne
        * exact Or.inr <X'', Y'', hX''D, hY''E, rfl, (toLiftProd_mem_embF hX''D hY''E)
    .mpr hW>

theorem toLiftProd_fromLiftProd (s : (D.prod E).lift.sys.Element) :
    toLiftProd (fromLiftProd s) = s := by
  apply NeighborhoodSystem.Element.ext
  intro W
  constructor
  * rintro (rfl | <X, Y, hX, hY, rfl, hs>)
    * exact s.master_mem
    * exact (fromLiftProd_mem_prod hX hY).mp hs
  * intro hW
```

```
    rcases s.sub hW with rfl | <Z, hZ, rfl>
    * exact Or.inl rfl
    * obtain <X, Y, hX, hY, rfl> := hZ
      exact Or.inr <X, Y, hX, hY, rfl, (fromLiftProd_mem_prod hX hY).mpr hW>

/-- The order-isomorphism `|?_False ? ?_False| ?o |(?  x  ?)_False|`. -/
def liftProdEquiv : (D.lift.otimes E.lift).sys.Element ?o (D.prod E).lift.sys.Element
    where
  toFun := toLiftProd
  invFun := fromLiftProd
  left_inv := fromLiftProd_toLiftProd
  right_inv := toLiftProd_fromLiftProd
  map_rel_iff' := by
    intro z z'
    constructor
    * intro h W hW
      rcases z.sub hW with rfl | <X, Y, hX, hY, hXne, hYne, rfl>
      * exact z'.master_mem
      * rcases hX with rfl | <X'', hX''D, rfl>
        * exact absurd rfl hXne
        * rcases hY with rfl | <Y'', hY''E, rfl>
          * exact absurd rfl hYne
          * exact (toLiftProd_mem_embF hX''D hY''E).mp
              (h _ ((toLiftProd_mem_embF hX''D hY''E).mpr hW))
    * intro h W hW
      rcases hW with rfl | <X, Y, hX, hY, rfl, hzXY>
      * exact Or.inl rfl
      * exact Or.inr <X, Y, hX, hY, rfl, h _ hzXY>

theorem toSumLift_fromSumLift (s : (D.sum E).sys.Element) :
    toSumLift (fromSumLift s) = s := by
  apply NeighborhoodSystem.Element.ext
  intro W
  constructor
  * rintro (rfl | <X, hX, rfl, hs> | <Y, hY, rfl, hs>)
    * exact s.master_mem
    * exact (fromSumLift_mem_embFF hX).mp hs
    * exact (fromSumLift_mem_embTF hY).mp hs
  * intro hW
    rcases s.sub hW with rfl | <X, hX, rfl> | <Y, hY, rfl>
    * exact Or.inl rfl
    * exact Or.inr (Or.inl <X, hX, rfl, (fromSumLift_mem_embFF hX).mpr hW>)
    * exact Or.inr (Or.inr <Y, hY, rfl, (fromSumLift_mem_embTF hY).mpr hW>)

/-- The order-isomorphism `|?_False ? ?_False| ?o |? + ?|`. -/
def sumLiftEquiv : (D.lift.oplus E.lift).sys.Element ?o (D.sum E).sys.Element where
  toFun := toSumLift
  invFun := fromSumLift
  left_inv := fromSumLift_toSumLift
  right_inv := toSumLift_fromSumLift
  map_rel_iff' := by
    intro z z'
    constructor
    * intro h W hW
      rcases z.sub hW with rfl | <X, hX, hXne, rfl> | <Y, hY, hYne, rfl>
      * exact z'.master_mem
```

```
      * rcases hX with rfl | <X', hX'D, rfl>
        * exact absurd rfl hXne
        * exact (toSumLift_mem_embF hX'D).mp (h _ ((toSumLift_mem_embF hX'D).mpr hW))
      * rcases hY with rfl | <Y', hY'E, rfl>
        * exact absurd rfl hYne
        * exact (toSumLift_mem_embT hY'E).mp (h _ ((toSumLift_mem_embT hY'E).mpr hW))
    * intro h W hW
      rcases hW with rfl | <X, hX, rfl, hzX> | <Y, hY, rfl, hzY>
      * exact Or.inl rfl
      * exact Or.inr (Or.inl <X, hX, rfl, h _ hzX>)
      * exact Or.inr (Or.inr <Y, hY, rfl, h _ hzY>)

end Exercise619
end

section
namespace Exercise619

variable {D : NeighborhoodSystem Str}

variable {hD : forall  X, D.mem X  ->  X.Nonempty}

/-- The **unlift** of an element that lies above the fresh bottom (i.e. contains `0
    Delta`): the
`?`-element `{X ? 0X  in  z}`. -/
def unlift (z : (liftTok D hD).Element) (hz : z.mem (embBit false D.master)) : D.
    Element where
  mem X := z.mem (embBit false X)
  sub := fun hX => liftTok_mem_embF_inv (z.sub hX)
  master_mem := hz
  inter_mem := by
    intro X X' hX hX'
    have hz' := z.inter_mem hX hX'
    rwa [embBit_inter] at hz'
  up_mem := by
    intro X Y hX hY hXY
    exact z.up_mem hX (liftTok_mem_embF hY) (embBit_subset.mpr hXY)

/-- `liftBot` is the least element of `?_False`. -/
theorem liftBot_le (z : (liftTok D hD).Element) : liftBot D hD  <=  z := by
  rintro W rfl; exact z.master_mem

/-- The fresh bottom is *strictly* below every lifted element. -/
theorem liftBot_lt_liftUp (x : D.Element) : liftBot D hD < liftUp (hD := hD) x := by
  refine lt_of_le_of_ne (liftBot_le _) (fun heq => ?_)
  have hmem : (liftBot D hD).mem (embBit false D.master) := by
    rw [heq]; exact Or.inr <D.master, x.master_mem, rfl>
  exact embF_ne_liftTokMaster hmem

end Exercise619
end

section
namespace Exercise619

variable {D : NeighborhoodSystem Str}
```

```
variable {hD : forall  X, D.mem X  ->  X.Nonempty}

variable {A B C : ScottSys}

/-- **`(g  comp  f)_False = g_False  comp  f_False`.** -/
theorem liftMapTok_comp (f : ApproximableMap A.sys B.sys) (g : ApproximableMap B.sys C
    .sys) :
    liftMapTok (g.comp f) = (liftMapTok g).comp (liftMapTok f) := by
  apply ApproximableMap.ext
  intro W W''
  constructor
  * rintro (<hW, rfl> | <X, X'', <X', hf, hg>, rfl, rfl>)
    * exact <liftTokMaster B.sys, Or.inl <hW, rfl>,
        Or.inl <(ScottSys.lift B).sys.master_mem, rfl>>
    * exact <embBit false X', Or.inr <X, X', hf, rfl, rfl>, Or.inr <X', X'', hg, rfl,
    rfl>>
  * rintro <W', hWW', hW'W''>
    rcases hWW' with <hW, rfl> | <X, X', hf, rfl, rfl>
    * rcases hW'W'' with <-, rfl> | <X, X', -, heq, ->
      * exact Or.inl <hW, rfl>
      * exact absurd (heq  |>  nil_mem_liftTokMaster) nil_not_mem_embBit
    * rcases hW'W'' with <-, rfl> | <X_2, X'', hg, heq, rfl>
      * exact Or.inl <liftTok_mem_embF (hD := A.ne) (f.rel_dom hf), rfl>
      * obtain rfl := embBit_injective heq
        exact Or.inr <X, X'', <X', hf, hg>, rfl, rfl>

/-- **`(I_?)_False = I_{?_False}`.** -/
theorem liftMapTok_id : liftMapTok (idMap A.sys) = idMap (ScottSys.lift A).sys := by
  apply ApproximableMap.ext
  intro W W'
  constructor
  * rintro (<hW, rfl> | <X, X', <hX, hX', hsub>, rfl, rfl>)
    * exact <hW, (ScottSys.lift A).sys.master_mem, (ScottSys.lift A).sys.sub_master hW
    >
    * exact <liftTok_mem_embF (hD := A.ne) hX, liftTok_mem_embF (hD := A.ne) hX',
        embBit_subset.mpr hsub>
  * rintro <hW, hW', hsub>
    rcases hW' with rfl | <X', hX', rfl>
    * exact Or.inl <hW, rfl>
    * rcases hW with rfl | <X, hX, rfl>
      * exact absurd (hsub nil_mem_liftTokMaster) nil_not_mem_embBit
      * exact Or.inr <X, X', <hX, hX', embBit_subset.mp hsub>, rfl, rfl>

/-- **`f_False` is strict** for *any* `f`: the master `?`-bearing input relates only to
     the master. -/
theorem liftMapTok_isStrict (f : ApproximableMap A.sys B.sys) : IsStrict (liftMapTok f
    ) := by
  rintro Y (<-, rfl> | <X, X', -, heq, ->)
  * rfl
  * have hnil : ([] : Str)  in  embBit false X := by
      rw [ <-  heq]; exact nil_mem_liftTokMaster
    exact absurd hnil nil_not_mem_embBit

end Exercise619
end
```

```
section
namespace Exercise619

variable {D : NeighborhoodSystem Str}

variable {hD : forall  X, D.mem X  ->  X.Nonempty}

theorem liftTok_mem_master : (liftTok D hD).mem (liftTokMaster D) := Or.inl rfl

/-- `liftUp` is an order embedding: `liftUp x  sqsub  liftUp y  <->  x  sqsub  y`. -/
theorem liftUp_le_liftUp_iff {x y : D.Element} :
    liftUp (hD := hD) x  <=  liftUp (hD := hD) y  <->  x  <=  y := by
  constructor
  * intro h X hX
    have hmem := h (embBit false X) (Or.inr <X, hX, rfl>)
    rcases hmem with h0 | <X', hX', heq>
    * exact absurd (h0.symm  |>  nil_mem_liftTokMaster) nil_not_mem_embBit
    * rw [embBit_injective heq]; exact hX'
  * rintro h W (rfl | <X, hX, rfl>)
    * exact Or.inl rfl
    * exact Or.inr <X, h X hX, rfl>

theorem liftUp_unlift (z : (liftTok D hD).Element) (hz : z.mem (embBit false D.master)
    ) :
    liftUp (hD := hD) (unlift z hz) = z := by
  apply NeighborhoodSystem.Element.ext
  intro W
  constructor
  * rintro (rfl | <X, hX, rfl>)
    * exact z.master_mem
    * exact hX
  * intro hW
    rcases z.sub hW with rfl | <X, hX, rfl>
    * exact Or.inl rfl
    * exact Or.inr <X, hW, rfl>

/-- **Exercise 6.26 -- "describe in terms of elements".** Every element of `?_False` is
     either the fresh
bottom or a lifted `?`-element: `|?_False|  ~=  |?|_False`. (`Prop`-level; the case
    split on "does `z` contain
`0Delta`?" uses excluded middle -- the only non-constructive step in this module.) -/
theorem eq_liftBot_or_exists_liftUp (z : (liftTok D hD).Element) :
    z = liftBot D hD  \/  exists  x : D.Element, z = liftUp (hD := hD) x := by
  by_cases hz : z.mem (embBit false D.master)
  * exact Or.inr <unlift z hz, (liftUp_unlift z hz).symm>
  * refine Or.inl ?_
    apply NeighborhoodSystem.Element.ext
    intro W
    constructor
    * intro hW
      rcases z.sub hW with rfl | <X, hX, rfl>
      * rfl
      * exact absurd
          (z.up_mem hW (liftTok_mem_embF D.master_mem) (embBit_subset.mpr (D.
    sub_master hX))) hz
```

```
    * rintro rfl; exact z.master_mem

end Exercise619
end

section
namespace Exercise619

variable {D : NeighborhoodSystem Str}

variable {hD : forall  X, D.mem X  ->  X.Nonempty}

variable {A B C : ScottSys}

variable {D E : ScottSys}

/-- **Exercise 6.26 -- `?_False ? ?_False  ~=  ? + ?`.** Coalescing the fresh bottoms
    of the two lifts
reproduces the separated sum. -/
theorem lift_oplus_lift_iso_sum :
    (D.lift.oplus E.lift).sys  ~= ? (D.sum E).sys := <sumLiftEquiv>

/-- **Exercise 6.26 -- `?_False ? ?_False  ~=  (?  x  ?)_False`** (the answer to Scott'
    s `??`). The smash product of
two lifts is the lift of the product. -/
theorem lift_otimes_lift_iso_lift_prod :
    (D.lift.otimes E.lift).sys  ~= ? (D.prod E).lift.sys := <liftProdEquiv>

end Exercise619
end

end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.Exercise619
section
set_option linter.unusedVariables false

theorem example_626 : And
  (And
    (forall  {D : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem DomainTheory.
    NeighbourhoodSystems.ExampleB.Str}
      {hD :
        forall  (X : Set DomainTheory.NeighbourhoodSystems.ExampleB.Str),
          @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem DomainTheory.
    NeighbourhoodSystems.ExampleB.Str D X  ->
            @Set.Nonempty DomainTheory.NeighbourhoodSystems.ExampleB.Str X}
      (z :
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
          (DomainTheory.NeighbourhoodSystems.Exercise619.liftTok D hD)),
      Or
        (@Eq
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
            (DomainTheory.NeighbourhoodSystems.Exercise619.liftTok D hD))
          z (DomainTheory.NeighbourhoodSystems.Exercise619.liftBot D hD))
        (exists  (x :
```

```
       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element DomainTheory.
  NeighbourhoodSystems.ExampleB.Str

         D),
       @Eq
         (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
           DomainTheory.NeighbourhoodSystems.ExampleB.Str
           (DomainTheory.NeighbourhoodSystems.Exercise619.liftTok D hD))
         z (@DomainTheory.NeighbourhoodSystems.Exercise619.liftUp D hD x)))
  (forall  {D E : DomainTheory.NeighbourhoodSystems.Exercise619.ScottSys},
    @DomainTheory.NeighbourhoodSystems.Isomorphic DomainTheory.NeighbourhoodSystems.
  ExampleB.Str
      DomainTheory.NeighbourhoodSystems.ExampleB.Str (D.lift.oplus E.lift).sys (D.
  sum E).sys))
(forall  {D E : DomainTheory.NeighbourhoodSystems.Exercise619.ScottSys},
  @DomainTheory.NeighbourhoodSystems.Isomorphic DomainTheory.NeighbourhoodSystems.
  ExampleB.Str
    DomainTheory.NeighbourhoodSystems.ExampleB.Str (D.lift.otimes E.lift).sys (D.
  prod E).lift.sys) :=
@And.intro
(And
  (forall  {D : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem DomainTheory.
  NeighbourhoodSystems.ExampleB.Str}
    {hD :
      forall  (X : Set DomainTheory.NeighbourhoodSystems.ExampleB.Str),
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem DomainTheory.
  NeighbourhoodSystems.ExampleB.Str D X  ->
          @Set.Nonempty DomainTheory.NeighbourhoodSystems.ExampleB.Str X}
    (z :
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element DomainTheory.
  NeighbourhoodSystems.ExampleB.Str
        (DomainTheory.NeighbourhoodSystems.Exercise619.liftTok D hD)),
    Or
      (@Eq
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element DomainTheory.
  NeighbourhoodSystems.ExampleB.Str
          (DomainTheory.NeighbourhoodSystems.Exercise619.liftTok D hD))
        z (DomainTheory.NeighbourhoodSystems.Exercise619.liftBot D hD))
      (exists  (x :
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element DomainTheory.
  NeighbourhoodSystems.ExampleB.Str
          D),
        @Eq
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
            DomainTheory.NeighbourhoodSystems.ExampleB.Str
            (DomainTheory.NeighbourhoodSystems.Exercise619.liftTok D hD))
          z (@DomainTheory.NeighbourhoodSystems.Exercise619.liftUp D hD x)))
  (forall  {D E : DomainTheory.NeighbourhoodSystems.Exercise619.ScottSys},
    @DomainTheory.NeighbourhoodSystems.Isomorphic DomainTheory.NeighbourhoodSystems.
  ExampleB.Str
      DomainTheory.NeighbourhoodSystems.ExampleB.Str (D.lift.oplus E.lift).sys (D.
  sum E).sys))
(forall  {D E : DomainTheory.NeighbourhoodSystems.Exercise619.ScottSys},
  @DomainTheory.NeighbourhoodSystems.Isomorphic DomainTheory.NeighbourhoodSystems.
  ExampleB.Str
    DomainTheory.NeighbourhoodSystems.ExampleB.Str (D.lift.otimes E.lift).sys (D.
```

```
    prod E).lift.sys)
  (@And.intro
    (forall  {D : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem DomainTheory.
    NeighbourhoodSystems.ExampleB.Str}
      {hD :
        forall  (X : Set DomainTheory.NeighbourhoodSystems.ExampleB.Str),
          @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem DomainTheory.
    NeighbourhoodSystems.ExampleB.Str D X  ->
            @Set.Nonempty DomainTheory.NeighbourhoodSystems.ExampleB.Str X}
      (z :
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
          (DomainTheory.NeighbourhoodSystems.Exercise619.liftTok D hD)),
      Or
        (@Eq
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
            (DomainTheory.NeighbourhoodSystems.Exercise619.liftTok D hD))
          z (DomainTheory.NeighbourhoodSystems.Exercise619.liftBot D hD))
        (exists  (x :
          @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
            D),
          @Eq
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
              DomainTheory.NeighbourhoodSystems.ExampleB.Str
              (DomainTheory.NeighbourhoodSystems.Exercise619.liftTok D hD))
            z (@DomainTheory.NeighbourhoodSystems.Exercise619.liftUp D hD x)))
    (forall  {D E : DomainTheory.NeighbourhoodSystems.Exercise619.ScottSys},
      @DomainTheory.NeighbourhoodSystems.Isomorphic DomainTheory.NeighbourhoodSystems.
    ExampleB.Str
        DomainTheory.NeighbourhoodSystems.ExampleB.Str (D.lift.oplus E.lift).sys (D.
    sum E).sys)
    @DomainTheory.NeighbourhoodSystems.Exercise619.eq_liftBot_or_exists_liftUp
    @DomainTheory.NeighbourhoodSystems.Exercise619.lift_oplus_lift_iso_sum)
  @DomainTheory.NeighbourhoodSystems.Exercise619.lift_otimes_lift_iso_lift_prod
end

end DomainTheory.NeighbourhoodSystems.Exercise619
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.Exercise619
open DomainTheory.NeighbourhoodSystems.Example62 DomainTheory.NeighbourhoodSystems.
    ExampleB
open DomainTheory.NeighbourhoodSystems.Exercise510

section
namespace Exercise627

variable {D_0 D_1 : NeighborhoodSystem Str}

/-- The injection `i : ?  ->  ?  x  ?`: `X i W  <->  X  in  ?  /\  W  in  ? x ?  /\  (X
    ,Delta_1)  subseteq  W`. -/
def fstInj (D_0 D_1 : NeighborhoodSystem Str) : ApproximableMap D_0 (prodTok D_0 D_1)
    where
  rel X W := D_0.mem X  /\  (prodTok D_0 D_1).mem W  /\  prodTokNbhd X D_1.master
    subseteq  W
```

```
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := <D_0.master_mem, (prodTok D_0 D_1).master_mem, subset_rfl>
  inter_right := by
    rintro X W W' <hX, hW, hsub> <_, hW', hsub'>
    exact <hX, (prodTok D_0 D_1).inter_mem hW hW' (prodTok_mem_prodTokNbhd hX D_1.
    master_mem)
      (Set.subset_inter hsub hsub'), Set.subset_inter hsub hsub'>
  mono := by
    rintro X X' W W' <_, _, hsub> hX'X hWW' hX' hW'
    exact <hX', hW', ((prodTokNbhd_subset_iff.mpr <hX'X, subset_rfl>).trans hsub).
    trans hWW'>

/-- The projection `j : ?  x  ?  ->  ?`: `W j X  <->  W  in  ? x ?  /\  X  in  ?  /\  W
      subseteq  (X,Delta_1)`. -/
def fstProj (D_0 D_1 : NeighborhoodSystem Str) : ApproximableMap (prodTok D_0 D_1) D_0
     where
  rel W X := (prodTok D_0 D_1).mem W  /\  D_0.mem X  /\  W  subseteq  prodTokNbhd X
    D_1.master
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := <(prodTok D_0 D_1).master_mem, D_0.master_mem, subset_rfl>
  inter_right := by
    rintro W X X' <hW, hX, hsub> <_, hX', hsub'>
    obtain <A, B, hA, hB, rfl> := hW
    have hAX : A  subseteq  X := (prodTokNbhd_subset_iff.mp hsub).1
    have hAX' : A  subseteq  X' := (prodTokNbhd_subset_iff.mp hsub').1
    exact <prodTok_mem_prodTokNbhd hA hB, D_0.inter_mem hX hX' hA (Set.subset_inter
    hAX hAX'),
      prodTokNbhd_subset_iff.mpr <Set.subset_inter hAX hAX', D_1.sub_master hB>>
  mono := by
    rintro W W' X X' <_, _, hsub> hW'W hXX' hW' hX'
    exact <hW', hX', hW'W.trans (hsub.trans (prodTokNbhd_subset_iff.mpr <hXX',
    subset_rfl>))>

end Exercise627
end

section
namespace Exercise627

variable {D_0 D_1 : NeighborhoodSystem Str}

variable {h_0 : forall  X, D_0.mem X  ->  X.Nonempty} {h_1 : forall  Y, D_1.mem Y  ->
     Y.Nonempty}

section Part5

variable {a b : Type*} {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}

/-- Element-level inclusion `|?  -> False ?|  ->  |?  ->  ?|`. -/
def incl (phi : (strictFun V_0 V_1).Element) : (funSpace V_0 V_1).Element :=
  toFilter (toStrictMap phi).1

theorem incl_mono : Monotone (incl (V_0 := V_0) (V_1 := V_1)) := by
  intro a b hab M hM
```

```
  obtain <hMmem, haM> := hM
  obtain <<L, hL, rfl>, hne> := hMmem
  refine <<<L, hL, rfl>, hne>, ?_>
  intro p hp
  have hle : (toStrictMap a).1  <=  (toStrictMap b).1 := (strictFunEquiv V_0 V_1).
    monotone hab
  exact hle p.1 p.2 (haM p hp)

/-- The inclusion `i : ?  -> False ? ? ?  ->  ?`. -/
def inclMap : ApproximableMap (strictFun V_0 V_1) (funSpace V_0 V_1) :=
  ofMono (fun _ hW => incl ((strictFun V_0 V_1).principal hW))
    (fun _ _ hW hW' hW'W => incl_mono (((strictFun V_0 V_1).principal_le_iff hW hW').
    mpr hW'W))

end Part5

/-- The left-copy generator: `0X` for proper `X`, the shared bottom `sumTokMaster` for
    `X = Delta_0`. The
set-builder `{w ? w  in  sumTokMaster  /\  X = Delta_0}` keeps the *definition* choice-
    free. -/
def leftN (D_0 D_1 : NeighborhoodSystem Str) (X : Set Str) : Set Str :=
  embBit false X  U  {w | w  in  sumTokMaster D_0 D_1  /\  X = D_0.master}

theorem X_eq_master_of_embT_subset_leftN {B X : Set Str} (hB : B.Nonempty)
    (h : embBit true B  subseteq  leftN D_0 D_1 X) : X = D_0.master := by
  obtain <sigma, hsigma> := hB
  rcases h <sigma, rfl, hsigma> with <w', hw', -> | <-, hXm>
  * rw [List.cons.injEq] at hw'; exact absurd hw'.1 (by decide)
  * exact hXm

theorem X_eq_master_of_nil_mem_leftN {X : Set Str} (h : ([] : Str)  in  leftN D_0 D_1
    X) :
    X = D_0.master := by
  rcases h with h | <-, h>
  * exact absurd h nil_not_mem_embBit
  * exact h

theorem embF_subset_leftN {X : Set Str} : embBit false X  subseteq  leftN D_0 D_1 X :=
  Set.subset_union_left

theorem leftN_master : leftN D_0 D_1 D_0.master = sumTokMaster D_0 D_1 := by
  apply Set.eq_of_subset_of_subset
  * rintro w (hw | <hw, ->)
    * exact embF_subset_sumTokMaster D_0.master_mem hw
    * exact hw
  * exact fun w hw => Or.inr <hw, rfl>

theorem leftN_proper {X : Set Str} (hne : X  !=  D_0.master) : leftN D_0 D_1 X =
    embBit false X := by
  apply Set.eq_of_subset_of_subset
  * rintro w (hw | <-, hXm>)
    * exact hw
    * exact absurd hXm hne
  * exact embF_subset_leftN

/-- The **only** classical step in Exercise 6.27: `leftN X` is an `? ? ?`-neighbourhood
```

```
    . The proof
splits on the (undecidable for arbitrary `X`) test `X = Delta_0`. -/
theorem oplus_mem_leftN {X : Set Str} (hX : D_0.mem X) :
    (oplusTok D_0 D_1 h_0 h_1).mem (leftN D_0 D_1 X) := by
  by_cases h : X = D_0.master
  * rw [h, leftN_master]; exact (oplusTok D_0 D_1 h_0 h_1).master_mem
  * rw [leftN_proper h]; exact oplusTok_mem_embF hX h

section Part5

variable {a b : Type*} {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}

/-- **Strictification of a map** (on relations): keep `f` but force the master input to
     the master
output. The resulting map is strict, lies below `f`, and is the identity on strict maps
    . -/
def strictifyMap (g : ApproximableMap V_0 V_1) : ApproximableMap V_0 V_1 where
  rel X Y := g.rel X Y  /\  (X = V_0.master  ->  Y = V_1.master)
  rel_dom h := g.rel_dom h.1
  rel_cod h := g.rel_cod h.1
  master_rel := <g.master_rel, fun _ => rfl>
  inter_right := by
    rintro X Y Y' <hg, hs> <hg', hs'>
    exact <g.inter_right hg hg', fun hX => by rw [hs hX, hs' hX, Set.inter_self]>
  mono := by
    rintro X X' Y Y' <hg, hs> hX'X hYY' hX' hY'
    refine <g.mono hg hX'X hYY' hX' hY', fun hX'm => ?_>
    have hX : V_0.mem X := g.rel_dom hg
    have hXm : X = V_0.master := Set.Subset.antisymm (V_0.sub_master hX) (hX'm  |>  hX
    'X)
    exact Set.Subset.antisymm (V_1.sub_master hY') ((hs hXm)  |>  hYY')

theorem strictifyMap_isStrict (g : ApproximableMap V_0 V_1) : IsStrict (strictifyMap g
    ) :=
  fun _ h => h.2 rfl

/-- The strictification as an element of the *strict* function space's token type. -/
def strictify (g : ApproximableMap V_0 V_1) : StrictMap V_0 V_1 := <strictifyMap g,
    strictifyMap_isStrict g>

/-- Element-level strictification `|?  ->  ?|  ->  |?  -> False ?|`. -/
def strct (psi : (funSpace V_0 V_1).Element) : (strictFun V_0 V_1).Element :=
  toStrictFilter (strictify (toApproxMap psi))

theorem strct_mono : Monotone (strct (V_0 := V_0) (V_1 := V_1)) := by
  intro a b hab W hW
  obtain <hWmem, haW> := hW
  obtain <<L, hL, rfl>, hne> := hWmem
  refine <<<L, hL, rfl>, hne>, ?_>
  intro p hp
  have hle : toApproxMap a  <=  toApproxMap b := (funSpaceEquiv V_0 V_1).monotone hab
  exact <hle p.1 p.2 (haW p hp).1, (haW p hp).2>

/-- The strictification retraction `j : ?  ->  ?  ->  ?  -> False ?`. -/
def strctMap : ApproximableMap (funSpace V_0 V_1) (strictFun V_0 V_1) :=
  ofMono (fun _ hM => strct ((funSpace V_0 V_1).principal hM))
```

```
    (fun _ _ hM hM' hM'M => strct_mono (((funSpace V_0 V_1).principal_le_iff hM hM').
    mpr hM'M))

end Part5

theorem subset_of_embF_subset_leftN {A X : Set Str} (hA : A  subseteq  D_0.master)
    (h : embBit false A  subseteq  leftN D_0 D_1 X) : A  subseteq  X := by
  intro sigma hsigma
  rcases h <sigma, rfl, hsigma> with <w', hw', hw'X> | <-, hXm>
  * rw [List.cons.injEq] at hw'; rw [hw'.2]; exact hw'X
  * rw [hXm]; exact hA hsigma

theorem leftN_subset_iff {X X' : Set Str} (hX : D_0.mem X) (hX' : D_0.mem X') :
    leftN D_0 D_1 X  subseteq  leftN D_0 D_1 X'  <->  X  subseteq  X' := by
  constructor
  * intro h
    exact subset_of_embF_subset_leftN (D_0.sub_master hX) (embF_subset_leftN.trans h)
  * intro hXX'
    rintro w (<w', rfl, hw'> | <hw, hXm>)
    * exact embF_subset_leftN <w', rfl, hXX' hw'>
    * exact Or.inr <hw, Set.Subset.antisymm (D_0.sub_master hX') (hXm  |>  hXX')>

/-- The injection `i : ?  ->  ? ? ?`: `X i W  <->  X  in  ?  /\  W  in  ???  /\  leftN
    X  subseteq  W`. -/
def inlInj : ApproximableMap D_0 (oplusTok D_0 D_1 h_0 h_1) where
  rel X W := D_0.mem X  /\  (oplusTok D_0 D_1 h_0 h_1).mem W  /\  leftN D_0 D_1 X
    subseteq  W
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := <D_0.master_mem, (oplusTok D_0 D_1 h_0 h_1).master_mem, leftN_master.
    le>
  inter_right := by
    rintro X W W' <hX, hW, hsub> <_, hW', hsub'>
    exact <hX, (oplusTok D_0 D_1 h_0 h_1).inter_mem hW hW' (oplus_mem_leftN hX)
      (Set.subset_inter hsub hsub'), Set.subset_inter hsub hsub'>
  mono := by
    rintro X X' W W' <hX, _, hsub> hX'X hWW' hX' hW'
    exact <hX', hW', ((leftN_subset_iff hX' hX).mpr hX'X).trans (hsub.trans hWW')>

/-- The projection `j : ? ? ?  ->  ?`: `W j X  <->  W  in  ???  /\  X  in  ?  /\  W
    subseteq  leftN X`. -/
def inlProj : ApproximableMap (oplusTok D_0 D_1 h_0 h_1) D_0 where
  rel W X := (oplusTok D_0 D_1 h_0 h_1).mem W  /\  D_0.mem X  /\  W  subseteq  leftN
    D_0 D_1 X
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := <(oplusTok D_0 D_1 h_0 h_1).master_mem, D_0.master_mem, leftN_master.
    ge>
  inter_right := by
    rintro W X X' <hW, hX, hsub> <_, hX', hsub'>
    rcases hW with rfl | <A, hA, hAne, rfl> | <B, hB, hBne, rfl>
    * have hXm := X_eq_master_of_nil_mem_leftN (hsub nil_mem_sumTokMaster)
      have hX'm := X_eq_master_of_nil_mem_leftN (hsub' nil_mem_sumTokMaster)
      subst hXm; subst hX'm; rw [Set.inter_self]
      exact <Or.inl rfl, D_0.master_mem, leftN_master.ge>
    * have hAX : A  subseteq  X := subset_of_embF_subset_leftN (D_0.sub_master hA)
```

```
    hsub
      have hAX' : A  subseteq  X' := subset_of_embF_subset_leftN (D_0.sub_master hA)
    hsub'
      exact <oplusTok_mem_embF hA hAne, D_0.inter_mem hX hX' hA (Set.subset_inter hAX
    hAX'),
         (embBit_subset.mpr (Set.subset_inter hAX hAX')).trans embF_subset_leftN>
    * have hXm := X_eq_master_of_embT_subset_leftN (h_1 B hB) hsub
      have hX'm := X_eq_master_of_embT_subset_leftN (h_1 B hB) hsub'
      subst hXm; subst hX'm; rw [Set.inter_self]
      exact <oplusTok_mem_embT hB hBne, D_0.master_mem, by
        rw [leftN_master]; exact embT_subset_sumTokMaster hB>
  mono := by
    rintro W W' X X' <_, hX, hsub> hW'W hXX' hW' hX'
    exact <hW', hX', hW'W.trans (hsub.trans ((leftN_subset_iff hX hX').mpr hXX'))>

/-- A concrete two-point domain over `{0,1}*`: neighbourhoods `Set.univ` (`= False`)
    and `{[]}`. -/
def twoPt : NeighborhoodSystem Str where
  mem X := X = Set.univ  \/  X = {[]}
  master := Set.univ
  master_nonempty := <[], Set.mem_univ _>
  master_mem := Or.inl rfl
  sub_master := by
    rintro X (rfl | rfl)
    * exact subset_rfl
    * exact Set.subset_univ _
  inter_mem := by
    rintro X Y Z (rfl | rfl) (rfl | rfl) _ _
    * rw [Set.inter_self]; exact Or.inl rfl
    * rw [Set.univ_inter]; exact Or.inr rfl
    * rw [Set.inter_univ]; exact Or.inr rfl
    * rw [Set.inter_self]; exact Or.inr rfl

/-- The one-point unit domain `?` over `{0,1}*` (only neighbourhood `{[]}`). -/
def unitPt : NeighborhoodSystem Str := (singletonSys ({[]} : Set Str) <[], rfl>).sys

theorem inlProj_comp_inlInj :
    (inlProj (D_0 := D_0) (D_1 := D_1) (h_0 := h_0) (h_1 := h_1)).comp inlInj = idMap
    D_0 := by
  apply ApproximableMap.ext
  intro X Z
  rw [comp_rel, idMap_rel]
  constructor
  * rintro <W, <hX, _, hsub>, _, hZ, hsub'>
    exact <hX, hZ, (leftN_subset_iff hX hZ).mp (hsub.trans hsub')>
  * rintro <hX, hZ, hXZ>
    exact <leftN D_0 D_1 X, <hX, oplus_mem_leftN hX, subset_rfl>,
      oplus_mem_leftN hX, hZ, (leftN_subset_iff hX hZ).mpr hXZ>

theorem inlInj_comp_inlProj_le :
    (inlInj (D_0 := D_0) (D_1 := D_1) (h_0 := h_0) (h_1 := h_1)).comp inlProj
       <=  idMap (oplusTok D_0 D_1 h_0 h_1) := by
  intro W W' hr
  obtain <X, <hW, _, hsub>, _, hW', hsub'> := hr
  exact <hW, hW', hsub.trans hsub'>
```

```
end Exercise627
end
end DomainTheory.NeighbourhoodSystems
end
```

3.7.29 Exercise 6.27 — which embeddings hold

Which of the following relationships are true:

$$(\mathcal{D} \otimes \mathcal{E}) \trianglelefteq (\mathcal{D} \times \mathcal{E})\,; \quad \mathcal{D} \trianglelefteq \mathcal{D} \times \mathcal{E}\,;$$

$$(\mathcal{D} \oplus \mathcal{E}) \trianglelefteq (\mathcal{D} + \mathcal{E})\,; \quad \mathcal{D} \trianglelefteq \mathcal{D} \oplus \mathcal{E}\,;$$

$$(\mathcal{D} \to_{\perp} \mathcal{E}) \trianglelefteq (\mathcal{D} \to \mathcal{E})\,; \quad \mathcal{D} \trianglelefteq \mathcal{D} \otimes \mathcal{E}\,?$$

Exercise_6_27.lean

```
module

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Exercise_2_12
public import DomainTheory.NeighbourhoodSystems.DomainEquations.Exercise_6_26

public import DomainTheory.NeighbourhoodSystems.DomainEquations.Proposition_6_12
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_13
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.Exercise619
open DomainTheory.NeighbourhoodSystems.Example62 DomainTheory.NeighbourhoodSystems.
    ExampleB
open DomainTheory.NeighbourhoodSystems.Exercise510
section
namespace Exercise627
variable {D_0 D_1 : NeighborhoodSystem Str}
variable {h_0 : forall  X, D_0.mem X  ->  X.Nonempty} {h_1 : forall  Y, D_1.mem Y  ->
     Y.Nonempty}

/- concordance: source L4199 EXERCISE 6.27 which embeddings hold -/
/-- **Exercise 6.27 (4).** `? ? ? ? ?`. (Uses `Classical.choice` via `oplus_mem_leftN
    `.) -/
theorem inl_trianglelefteq_oplus :
    D_0 ? oplusTok D_0 D_1 h_0 h_1 :=
  trianglelefteq_of_projectionPair inlInj inlProj inlProj_comp_inlInj
    inlInj_comp_inlProj_le

section Part5

variable {a b : Type*} {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
```

```
/-- Choice-free extensionality from agreement on principal inputs. Unlike `
    ext_of_toElementMap`
(which splits on the undecidable `V_0.mem X`), both sides of the relation carry domain
    membership via
`rel_dom`, so no case analysis -- and hence no `Classical.choice` -- is needed. -/
theorem ext_of_principal {? ? : Type*} {W_0 : NeighborhoodSystem ?} {W_1 :
    NeighborhoodSystem ?}
    {f g : ApproximableMap W_0 W_1}
    (h : forall  (X : Set ?) (hX : W_0.mem X),
      f.toElementMap (W_0.principal hX) = g.toElementMap (W_0.principal hX)) : f = g
    := by
  apply ApproximableMap.ext
  intro X Y
  constructor
  * intro hr
    have hX := f.rel_dom hr
    rw [g.rel_iff_mem_principal hX,  <-  h X hX,  <-  f.rel_iff_mem_principal hX];
    exact hr
  * intro hr
    have hX := g.rel_dom hr
    rw [f.rel_iff_mem_principal hX, h X hX,  <-  g.rel_iff_mem_principal hX]; exact hr

end Part5

end Exercise627
end

section
namespace Exercise627

variable {D_0 D_1 : NeighborhoodSystem Str}

theorem fstInj_comp_fstProj_le (D_0 D_1 : NeighborhoodSystem Str) :
    (fstInj D_0 D_1).comp (fstProj D_0 D_1)  <=  idMap (prodTok D_0 D_1) := by
  intro W W' hr
  obtain <X, <hW, _, hsub>, _, hW', hsub'> := hr
  exact <hW, hW', hsub.trans hsub'>

theorem fstProj_comp_fstInj (D_0 D_1 : NeighborhoodSystem Str) :
    (fstProj D_0 D_1).comp (fstInj D_0 D_1) = idMap D_0 := by
  apply ApproximableMap.ext
  intro X Z
  rw [comp_rel, idMap_rel]
  constructor
  * rintro <W, <hX, _, hsub>, _, hZ, hsub'>
    exact <hX, hZ, (prodTokNbhd_subset_iff.mp (hsub.trans hsub')).1>
  * rintro <hX, hZ, hXZ>
    exact <prodTokNbhd Z D_1.master,
      <hX, prodTok_mem_prodTokNbhd hZ D_1.master_mem,
        prodTokNbhd_subset_iff.mpr <hXZ, subset_rfl>>,
      prodTok_mem_prodTokNbhd hZ D_1.master_mem, hZ, subset_rfl>

/-- **Exercise 6.27 (2).** `? ? ?  x  ?`. -/
theorem fst_trianglelefteq_prod (D_0 D_1 : NeighborhoodSystem Str) :
    D_0 ? prodTok D_0 D_1 :=
  trianglelefteq_of_projectionPair (fstInj D_0 D_1) (fstProj D_0 D_1)
```

```
    (fstProj_comp_fstInj D_0 D_1) (fstInj_comp_fstProj_le D_0 D_1)

end Exercise627
end

section
namespace Exercise627

variable {D_0 D_1 : NeighborhoodSystem Str}

variable {h_0 : forall  X, D_0.mem X  ->  X.Nonempty} {h_1 : forall  Y, D_1.mem Y  ->
     Y.Nonempty}

theorem oplusTok_subsystem_sumTok (h_0 : forall  X, D_0.mem X  ->  X.Nonempty)
    (h_1 : forall  Y, D_1.mem Y  ->  Y.Nonempty) :
    oplusTok D_0 D_1 h_0 h_1 ? sumTok D_0 D_1 h_0 h_1 where
  master_eq := rfl
  sub := by
    rintro W (rfl | <X, hX, -, rfl> | <Y, hY, -, rfl>)
    * exact Or.inl rfl
    * exact sumTok_mem_embF hX
    * exact sumTok_mem_embT hY
  inter_closed := by
    rintro W W' (rfl | <X, hX, hXne, rfl> | <Y, hY, hYne, rfl>)
      (rfl | <X', hX', hX'ne, rfl> | <Y', hY', hY'ne, rfl>) hInt
    * rw [Set.inter_self]; exact Or.inl rfl
    * rw [sumTokMaster_inter_embF hX']; exact oplusTok_mem_embF hX' hX'ne
    * rw [sumTokMaster_inter_embT hY']; exact oplusTok_mem_embT hY' hY'ne
    * rw [Set.inter_comm, sumTokMaster_inter_embF hX]; exact oplusTok_mem_embF hX hXne
    * rw [embBit_inter] at hInt  |-
      exact oplusTok_mem_embF (sumTok_mem_embF_inv hInt)
        (inter_ne_of_ne_left (D_0.sub_master hX) hXne)
    * rw [embBit_inter_ne (show (false : Bool)  !=  true by decide)] at hInt
      exact absurd (sumTok_mem_nonempty hInt) Set.not_nonempty_empty
    * rw [Set.inter_comm, sumTokMaster_inter_embT hY]; exact oplusTok_mem_embT hY hYne
    * rw [embBit_inter_ne (show (true : Bool)  !=  false by decide)] at hInt
      exact absurd (sumTok_mem_nonempty hInt) Set.not_nonempty_empty
    * rw [embBit_inter] at hInt  |-
      exact oplusTok_mem_embT (sumTok_mem_embT_inv hInt)
        (inter_ne_of_ne_left (D_1.sub_master hY) hYne)

/-- **Exercise 6.27 (3).** `(? ? ?) ? (? + ?)`. -/
theorem oplus_trianglelefteq_sum (h_0 : forall  X, D_0.mem X  ->  X.Nonempty)
    (h_1 : forall  Y, D_1.mem Y  ->  Y.Nonempty) :
    oplusTok D_0 D_1 h_0 h_1 ? sumTok D_0 D_1 h_0 h_1 :=
  (oplusTok_subsystem_sumTok h_0 h_1).trianglelefteq

end Exercise627
end

section
namespace Exercise627

variable {D_0 D_1 : NeighborhoodSystem Str}

theorem otimesTok_subsystem_prodTok (D_0 D_1 : NeighborhoodSystem Str) :
```

```
    otimesTok D_0 D_1 ? prodTok D_0 D_1 where
  master_eq := rfl
  sub := by
    rintro W (rfl | <X, Y, hX, hY, -, -, rfl>)
    * exact <D_0.master, D_1.master, D_0.master_mem, D_1.master_mem, rfl>
    * exact <X, Y, hX, hY, rfl>
  inter_closed := by
    rintro W W' (rfl | <X, Y, hX, hY, hXne, hYne, rfl>)
      (rfl | <X', Y', hX', hY', hX'ne, hY'ne, rfl>) hInt
    * rw [Set.inter_self]; exact Or.inl rfl
    * rw [prodTokNbhd_inter, Set.inter_eq_right.mpr (D_0.sub_master hX'),
        Set.inter_eq_right.mpr (D_1.sub_master hY')]
      exact Or.inr <X', Y', hX', hY', hX'ne, hY'ne, rfl>
    * rw [Set.inter_comm, prodTokNbhd_inter, Set.inter_eq_right.mpr (D_0.sub_master hX
    ),
        Set.inter_eq_right.mpr (D_1.sub_master hY)]
      exact Or.inr <X, Y, hX, hY, hXne, hYne, rfl>
    * rw [prodTokNbhd_inter] at hInt  |-
      obtain <A, B, hA, hB, heq> := hInt
      obtain <rfl, rfl> := prodTokNbhd_injective heq
      exact Or.inr <X  I  X', Y  I  Y', hA, hB,
        inter_ne_of_ne_left (D_0.sub_master hX) hXne,
        inter_ne_of_ne_left (D_1.sub_master hY) hYne, rfl>

/-- **Exercise 6.27 (1).** `(? ? ?) ? (?  x  ?)`. -/
theorem otimes_trianglelefteq_prod (D_0 D_1 : NeighborhoodSystem Str) :
    otimesTok D_0 D_1 ? prodTok D_0 D_1 :=
  (otimesTok_subsystem_prodTok D_0 D_1).trianglelefteq

end Exercise627
end

section
namespace Exercise627

variable {D_0 D_1 : NeighborhoodSystem Str}

variable {h_0 : forall  X, D_0.mem X  ->  X.Nonempty} {h_1 : forall  Y, D_1.mem Y  ->
    Y.Nonempty}

/-- With the unit second factor, the smash `twoPt ? ?` collapses to its master. -/
theorem otimes_unitPt_collapse :
    forall  W, (otimesTok twoPt unitPt).mem W  ->  W = (otimesTok twoPt unitPt).master
    := by
  rintro W (rfl | <X, Y, _, hY, _, hYne, rfl>)
  * rfl
  * exact absurd hY hYne

section Part5

variable {a b : Type*} {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}

theorem strictifyMap_le (g : ApproximableMap V_0 V_1) : strictifyMap g  <=  g := fun _
    _ h => h.1

theorem strictifyMap_of_isStrict {f : ApproximableMap V_0 V_1} (hf : IsStrict f) :
```

```
    strictifyMap f = f := by
  apply ApproximableMap.ext
  intro X Y
  exact <fun h => h.1, fun h => <h, fun hXm => hf (hXm  |>  h)>>

end Part5

/-- If a system's only neighbourhood is its master, its element lattice is a single
    point (`False`). -/
theorem subsingleton_element_of_only_master {? : Type*} {V : NeighborhoodSystem ?}
    (h : forall  X, V.mem X  ->  X = V.master) : Subsingleton V.Element := by
  refine <fun x y => ?_>
  apply Element.ext
  intro X
  constructor
  * intro hx; have hXm := h X (x.sub hx); rw [hXm]; exact y.master_mem
  * intro hy; have hXm := h X (y.sub hy); rw [hXm]; exact x.master_mem

/-- **Exercise 6.27 (6).** `? ? ? ? ?` does **not** hold for all `?, ?`: take `?` two-
    point and
`? = ?`. Then `? ? ?` has a one-point element lattice while `?` has two, so no
    isomorphism onto a
subsystem can exist. -/
theorem not_trianglelefteq_otimes :
    not  (twoPt ? otimesTok twoPt unitPt) := by
  rintro <D', hsub, <e>>
  have hD'only : forall  X, D'.mem X  ->  X = D'.master := by
    intro X hX
    rw [otimes_unitPt_collapse X (hsub.sub hX)]; exact hsub.master_eq.symm
  have hss : Subsingleton D'.Element := subsingleton_element_of_only_master hD'only
  have hPmem : twoPt.mem ({[]} : Set Str) := Or.inr rfl
  have hne : twoPt.principal hPmem  !=  twoPt.principal twoPt.master_mem := by
    intro heq
    have h1 : (twoPt.principal hPmem).mem ({[]} : Set Str) := <hPmem, subset_rfl>
    rw [heq] at h1
    obtain <-, h2> := h1
    exact absurd (Set.mem_singleton_iff.mp (h2 (Set.mem_univ ([false] : Str))) :
      ([false] : Str) = []) (by decide)
  exact hne (e.injective (hss.allEq (e (twoPt.principal hPmem))
    (e (twoPt.principal twoPt.master_mem))))

section Part5

variable {a b : Type*} {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}

/-- The four inverse identities of the two representation equivalences (defeq
    unfoldings). -/
theorem toApproxMap_toFilter (g : ApproximableMap V_0 V_1) : toApproxMap (toFilter g)
    = g :=
  (funSpaceEquiv V_0 V_1).apply_symm_apply g

theorem toElementMap_inclMap (phi : (strictFun V_0 V_1).Element) :
    inclMap.toElementMap phi = incl phi := by
  apply Element.ext
  intro M
  constructor
```

```
  * rintro <W, hphiW, _, hincl>
    obtain <hMmem, hg> := hincl
    obtain <<L, hL, rfl>, hne> := hMmem
    refine <<<L, hL, rfl>, hne>, ?_>
    intro p hp
    obtain <hsstep, hWsub> := toStrictMap_rel.mp (hg p hp)
    exact phi.up_mem hphiW hsstep hWsub
  * rintro <hMmem, hgphi>
    obtain <<L, hL, rfl>, hne> := hMmem
    have hphiL : phi.mem (sstepFun L) :=
      (mem_sstepFun_iff phi hL).mpr (fun p hp => toStrictMap_rel.mp (hgphi p hp))
    refine <sstepFun L, hphiL, phi.sub hphiL, <<L, hL, rfl>, hne>, ?_>
    intro p hp
    exact <phi.sub (toStrictMap_rel.mp (hgphi p hp)), fun f hf => hf p hp>

theorem toElementMap_strctMap (psi : (funSpace V_0 V_1).Element) :
    strctMap.toElementMap psi = strct psi := by
  apply Element.ext
  intro W
  constructor
  * rintro <M, hpsiM, _, hstrct>
    obtain <hWmem, hg> := hstrct
    obtain <<L, hL, rfl>, hne> := hWmem
    refine <<<L, hL, rfl>, hne>, ?_>
    intro p hp
    obtain <hstepmem, hMsub> := (hg p hp).1
    exact <psi.up_mem hpsiM hstepmem hMsub, (hg p hp).2>
  * rintro <hWmem, hgpsi>
    obtain <<L, hL, rfl>, hne> := hWmem
    have hpsiL : psi.mem (stepFun L) :=
      (mem_stepFun_iff psi hL).mpr (fun p hp => (hgpsi p hp).1)
    refine <stepFun L, hpsiL, psi.sub hpsiL, <<L, hL, rfl>, hne>, ?_>
    intro p hp
    exact <<psi.sub (hgpsi p hp).1, fun f hf => hf p hp>, (hgpsi p hp).2>

theorem toFilter_toApproxMap (psi : (funSpace V_0 V_1).Element) : toFilter (
    toApproxMap psi) = psi :=
  (funSpaceEquiv V_0 V_1).symm_apply_apply psi

theorem toStrictFilter_toStrictMap (phi : (strictFun V_0 V_1).Element) :
    toStrictFilter (toStrictMap phi) = phi :=
  (strictFunEquiv V_0 V_1).symm_apply_apply phi

theorem strct_incl (phi : (strictFun V_0 V_1).Element) : strct (incl phi) = phi := by
  show toStrictFilter (strictify (toApproxMap (toFilter (toStrictMap phi).1))) = phi
  rw [toApproxMap_toFilter,
    show strictify (toStrictMap phi).1 = toStrictMap phi from
      Subtype.ext (strictifyMap_of_isStrict (toStrictMap phi).2)]
  exact toStrictFilter_toStrictMap phi

theorem strctMap_comp_inclMap :
    (strctMap (V_0 := V_0) (V_1 := V_1)).comp inclMap = idMap (strictFun V_0 V_1) :=
    by
  apply ext_of_principal
  intro X hX
  rw [toElementMap_comp, toElementMap_inclMap, toElementMap_strctMap, strct_incl,
```

```
    toElementMap_idMap]

theorem toStrictMap_toStrictFilter (g : StrictMap V_0 V_1) : toStrictMap (
    toStrictFilter g) = g :=
  (strictFunEquiv V_0 V_1).apply_symm_apply g

theorem incl_strct_le (psi : (funSpace V_0 V_1).Element) : incl (strct psi)  <=  psi
    := by
  have key : incl (strct psi) = toFilter (strictifyMap (toApproxMap psi)) := by
    show toFilter (toStrictMap (strct psi)).1 = toFilter (strictifyMap (toApproxMap
    psi))
    rw [show toStrictMap (strct psi) = strictify (toApproxMap psi) from
    toStrictMap_toStrictFilter _]
    rfl
  rw [key]
  conv_rhs => rw [ <-  toFilter_toApproxMap psi]
  exact (funSpaceEquiv V_0 V_1).symm.monotone (strictifyMap_le (toApproxMap psi))

theorem inclMap_comp_strctMap_le :
    (inclMap (V_0 := V_0) (V_1 := V_1)).comp strctMap  <=  idMap (funSpace V_0 V_1) :=
     by
  rw [le_iff_toElementMap_le]
  intro psi
  rw [toElementMap_comp, toElementMap_strctMap, toElementMap_inclMap,
    toElementMap_idMap]
  exact incl_strct_le psi

/-- **Exercise 6.27 (5).** `(?  -> False ?) ? (?  ->  ?)` for all `?, ?` (choice-free).
    -/
theorem strictFun_trianglelefteq_funSpace (V_0 : NeighborhoodSystem a) (V_1 :
    NeighborhoodSystem b) :
    strictFun V_0 V_1 ? funSpace V_0 V_1 :=
  trianglelefteq_of_projectionPair inclMap strctMap strctMap_comp_inclMap
    inclMap_comp_strctMap_le

end Part5

end Exercise627
end

end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.Exercise627
section
set_option linter.unusedVariables false

theorem example_627.{u_1, u_2} : And
  (And
    (And
      (And
        (And
          (forall
            (D_0 D_1 :
              DomainTheory.NeighbourhoodSystems.NeighborhoodSystem DomainTheory.
    NeighbourhoodSystems.ExampleB.Str),
            @DomainTheory.NeighbourhoodSystems.Trianglelefteq DomainTheory.
    NeighbourhoodSystems.ExampleB.Str
```

```
          DomainTheory.NeighbourhoodSystems.ExampleB.Str
          (DomainTheory.NeighbourhoodSystems.Exercise619.otimesTok D_0 D_1)
          (DomainTheory.NeighbourhoodSystems.Exercise619.prodTok D_0 D_1))
       (forall
         {D_0 D_1 :
           DomainTheory.NeighbourhoodSystems.NeighborhoodSystem DomainTheory.
NeighbourhoodSystems.ExampleB.Str}
         {h_0 :
           forall  (X : Set DomainTheory.NeighbourhoodSystems.ExampleB.Str),
             @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem DomainTheory
.NeighbourhoodSystems.ExampleB.Str
               D_0 X  ->
             @Set.Nonempty DomainTheory.NeighbourhoodSystems.ExampleB.Str X}
         {h_1 :
           forall  (Y : Set DomainTheory.NeighbourhoodSystems.ExampleB.Str),
             @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem DomainTheory
.NeighbourhoodSystems.ExampleB.Str
               D_1 Y  ->
             @Set.Nonempty DomainTheory.NeighbourhoodSystems.ExampleB.Str Y},
        @DomainTheory.NeighbourhoodSystems.Trianglelefteq DomainTheory.
NeighbourhoodSystems.ExampleB.Str
          DomainTheory.NeighbourhoodSystems.ExampleB.Str D_0
          (DomainTheory.NeighbourhoodSystems.Exercise619.oplusTok D_0 D_1 h_0 h_1)
))
     (forall
       (D_0 D_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem DomainTheory
.NeighbourhoodSystems.ExampleB.Str),
       @DomainTheory.NeighbourhoodSystems.Trianglelefteq DomainTheory.
NeighbourhoodSystems.ExampleB.Str
         DomainTheory.NeighbourhoodSystems.ExampleB.Str D_0
         (DomainTheory.NeighbourhoodSystems.Exercise619.prodTok D_0 D_1)))
   (Not
     (@DomainTheory.NeighbourhoodSystems.Trianglelefteq DomainTheory.
NeighbourhoodSystems.ExampleB.Str
       DomainTheory.NeighbourhoodSystems.ExampleB.Str DomainTheory.
NeighbourhoodSystems.Exercise627.twoPt
       (DomainTheory.NeighbourhoodSystems.Exercise619.otimesTok DomainTheory.
NeighbourhoodSystems.Exercise627.twoPt
         DomainTheory.NeighbourhoodSystems.Exercise627.unitPt))))
 (forall  {D_0 D_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
DomainTheory.NeighbourhoodSystems.ExampleB.Str}
   (h_0 :
     forall  (X : Set DomainTheory.NeighbourhoodSystems.ExampleB.Str),
       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem DomainTheory.
NeighbourhoodSystems.ExampleB.Str D_0
           X  ->
         @Set.Nonempty DomainTheory.NeighbourhoodSystems.ExampleB.Str X)
   (h_1 :
     forall  (Y : Set DomainTheory.NeighbourhoodSystems.ExampleB.Str),
       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem DomainTheory.
NeighbourhoodSystems.ExampleB.Str D_1
           Y  ->
         @Set.Nonempty DomainTheory.NeighbourhoodSystems.ExampleB.Str Y),
   @DomainTheory.NeighbourhoodSystems.Trianglelefteq DomainTheory.
NeighbourhoodSystems.ExampleB.Str
     DomainTheory.NeighbourhoodSystems.ExampleB.Str
```

```
      (DomainTheory.NeighbourhoodSystems.Exercise619.oplusTok D_0 D_1 h_0 h_1)
      (DomainTheory.NeighbourhoodSystems.Exercise619.sumTok D_0 D_1 h_0 h_1)))
(forall  {a : Type u_1} {b : Type u_2} (V_0 : DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem a)
  (V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b),
  @DomainTheory.NeighbourhoodSystems.Trianglelefteq
    (@DomainTheory.NeighbourhoodSystems.Exercise510.StrictMap a b V_0 V_1)
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1)
    (@DomainTheory.NeighbourhoodSystems.Exercise510.strictFun a b V_0 V_1)
    (@DomainTheory.NeighbourhoodSystems.funSpace a b V_0 V_1)) :=
@And.intro
(And
  (And
    (And
      (And
        (forall
          (D_0 D_1 :
            DomainTheory.NeighbourhoodSystems.NeighborhoodSystem DomainTheory.
  NeighbourhoodSystems.ExampleB.Str),
          @DomainTheory.NeighbourhoodSystems.Trianglelefteq DomainTheory.
  NeighbourhoodSystems.ExampleB.Str
            DomainTheory.NeighbourhoodSystems.ExampleB.Str
            (DomainTheory.NeighbourhoodSystems.Exercise619.otimesTok D_0 D_1)


            (DomainTheory.NeighbourhoodSystems.Exercise619.prodTok D_0 D_1))
        (forall
          {D_0 D_1 :
            DomainTheory.NeighbourhoodSystems.NeighborhoodSystem DomainTheory.
  NeighbourhoodSystems.ExampleB.Str}
          {h_0 :
            forall  (X : Set DomainTheory.NeighbourhoodSystems.ExampleB.Str),
              @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem DomainTheory
  .NeighbourhoodSystems.ExampleB.Str
                D_0 X  ->
              @Set.Nonempty DomainTheory.NeighbourhoodSystems.ExampleB.Str X}
          {h_1 :
            forall  (Y : Set DomainTheory.NeighbourhoodSystems.ExampleB.Str),
              @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem DomainTheory
  .NeighbourhoodSystems.ExampleB.Str
                D_1 Y  ->
              @Set.Nonempty DomainTheory.NeighbourhoodSystems.ExampleB.Str Y},
          @DomainTheory.NeighbourhoodSystems.Trianglelefteq DomainTheory.
  NeighbourhoodSystems.ExampleB.Str
            DomainTheory.NeighbourhoodSystems.ExampleB.Str D_0
            (DomainTheory.NeighbourhoodSystems.Exercise619.oplusTok D_0 D_1 h_0 h_1)
  ))
      (forall
        (D_0 D_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem DomainTheory
  .NeighbourhoodSystems.ExampleB.Str),
        @DomainTheory.NeighbourhoodSystems.Trianglelefteq DomainTheory.
  NeighbourhoodSystems.ExampleB.Str
          DomainTheory.NeighbourhoodSystems.ExampleB.Str D_0
          (DomainTheory.NeighbourhoodSystems.Exercise619.prodTok D_0 D_1)))
    (Not
      (@DomainTheory.NeighbourhoodSystems.Trianglelefteq DomainTheory.
  NeighbourhoodSystems.ExampleB.Str
```

```
       DomainTheory.NeighbourhoodSystems.ExampleB.Str DomainTheory.
  NeighbourhoodSystems.Exercise627.twoPt
       (DomainTheory.NeighbourhoodSystems.Exercise619.otimesTok DomainTheory.
  NeighbourhoodSystems.Exercise627.twoPt
         DomainTheory.NeighbourhoodSystems.Exercise627.unitPt))))
  (forall  {D_0 D_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
  DomainTheory.NeighbourhoodSystems.ExampleB.Str}
    (h_0 :
      forall  (X : Set DomainTheory.NeighbourhoodSystems.ExampleB.Str),
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem DomainTheory.
  NeighbourhoodSystems.ExampleB.Str D_0
          X  ->
        @Set.Nonempty DomainTheory.NeighbourhoodSystems.ExampleB.Str X)
    (h_1 :
      forall  (Y : Set DomainTheory.NeighbourhoodSystems.ExampleB.Str),
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem DomainTheory.
  NeighbourhoodSystems.ExampleB.Str D_1
          Y  ->
        @Set.Nonempty DomainTheory.NeighbourhoodSystems.ExampleB.Str Y),
    @DomainTheory.NeighbourhoodSystems.Trianglelefteq DomainTheory.
  NeighbourhoodSystems.ExampleB.Str
      DomainTheory.NeighbourhoodSystems.ExampleB.Str
      (DomainTheory.NeighbourhoodSystems.Exercise619.oplusTok D_0 D_1 h_0 h_1)
      (DomainTheory.NeighbourhoodSystems.Exercise619.sumTok D_0 D_1 h_0 h_1)))
(forall  {a : Type u_1} {b : Type u_2} (V_0 : DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem a)
  (V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b),
  @DomainTheory.NeighbourhoodSystems.Trianglelefteq
    (@DomainTheory.NeighbourhoodSystems.Exercise510.StrictMap a b V_0 V_1)
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1)
    (@DomainTheory.NeighbourhoodSystems.Exercise510.strictFun a b V_0 V_1)
    (@DomainTheory.NeighbourhoodSystems.funSpace a b V_0 V_1))
(@And.intro
  (And
    (And
      (And
        (forall
          (D_0 D_1 :
            DomainTheory.NeighbourhoodSystems.NeighborhoodSystem DomainTheory.
  NeighbourhoodSystems.ExampleB.Str),
          @DomainTheory.NeighbourhoodSystems.Trianglelefteq DomainTheory.
  NeighbourhoodSystems.ExampleB.Str
            DomainTheory.NeighbourhoodSystems.ExampleB.Str
            (DomainTheory.NeighbourhoodSystems.Exercise619.otimesTok D_0 D_1)
            (DomainTheory.NeighbourhoodSystems.Exercise619.prodTok D_0 D_1))
        (forall
          {D_0 D_1 :
            DomainTheory.NeighbourhoodSystems.NeighborhoodSystem DomainTheory.
  NeighbourhoodSystems.ExampleB.Str}
          {h_0 :
            forall  (X : Set DomainTheory.NeighbourhoodSystems.ExampleB.Str),
              @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem DomainTheory
  .NeighbourhoodSystems.ExampleB.Str
                D_0 X  ->
              @Set.Nonempty DomainTheory.NeighbourhoodSystems.ExampleB.Str X}
          {h_1 :
```

```
          forall  (Y : Set DomainTheory.NeighbourhoodSystems.ExampleB.Str),
            @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem DomainTheory
.NeighbourhoodSystems.ExampleB.Str
                D_1 Y  ->
              @Set.Nonempty DomainTheory.NeighbourhoodSystems.ExampleB.Str Y},
        @DomainTheory.NeighbourhoodSystems.Trianglelefteq DomainTheory.
NeighbourhoodSystems.ExampleB.Str
          DomainTheory.NeighbourhoodSystems.ExampleB.Str D_0
          (DomainTheory.NeighbourhoodSystems.Exercise619.oplusTok D_0 D_1 h_0 h_1)
))
    (forall
      (D_0 D_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem DomainTheory
.NeighbourhoodSystems.ExampleB.Str),
      @DomainTheory.NeighbourhoodSystems.Trianglelefteq DomainTheory.
NeighbourhoodSystems.ExampleB.Str
        DomainTheory.NeighbourhoodSystems.ExampleB.Str D_0
        (DomainTheory.NeighbourhoodSystems.Exercise619.prodTok D_0 D_1)))
  (Not
    (@DomainTheory.NeighbourhoodSystems.Trianglelefteq DomainTheory.
NeighbourhoodSystems.ExampleB.Str
      DomainTheory.NeighbourhoodSystems.ExampleB.Str DomainTheory.
NeighbourhoodSystems.Exercise627.twoPt
      (DomainTheory.NeighbourhoodSystems.Exercise619.otimesTok DomainTheory.
NeighbourhoodSystems.Exercise627.twoPt
        DomainTheory.NeighbourhoodSystems.Exercise627.unitPt))))
(forall  {D_0 D_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
DomainTheory.NeighbourhoodSystems.ExampleB.Str}
  (h_0 :
    forall  (X : Set DomainTheory.NeighbourhoodSystems.ExampleB.Str),
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem DomainTheory.
NeighbourhoodSystems.ExampleB.Str D_0
          X  ->
        @Set.Nonempty DomainTheory.NeighbourhoodSystems.ExampleB.Str X)
  (h_1 :
    forall  (Y : Set DomainTheory.NeighbourhoodSystems.ExampleB.Str),
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem DomainTheory.
NeighbourhoodSystems.ExampleB.Str D_1
          Y  ->
        @Set.Nonempty DomainTheory.NeighbourhoodSystems.ExampleB.Str Y),
  @DomainTheory.NeighbourhoodSystems.Trianglelefteq DomainTheory.
NeighbourhoodSystems.ExampleB.Str
    DomainTheory.NeighbourhoodSystems.ExampleB.Str
    (DomainTheory.NeighbourhoodSystems.Exercise619.oplusTok D_0 D_1 h_0 h_1)
    (DomainTheory.NeighbourhoodSystems.Exercise619.sumTok D_0 D_1 h_0 h_1))
(@And.intro
  (And
    (And
      (forall
        (D_0 D_1 :
          DomainTheory.NeighbourhoodSystems.NeighborhoodSystem DomainTheory.
NeighbourhoodSystems.ExampleB.Str),
        @DomainTheory.NeighbourhoodSystems.Trianglelefteq DomainTheory.
NeighbourhoodSystems.ExampleB.Str
          DomainTheory.NeighbourhoodSystems.ExampleB.Str
          (DomainTheory.NeighbourhoodSystems.Exercise619.otimesTok D_0 D_1)
          (DomainTheory.NeighbourhoodSystems.Exercise619.prodTok D_0 D_1))
```

```
       (forall
         {D_0 D_1 :
           DomainTheory.NeighbourhoodSystems.NeighborhoodSystem DomainTheory.
NeighbourhoodSystems.ExampleB.Str}
         {h_0 :
           forall  (X : Set DomainTheory.NeighbourhoodSystems.ExampleB.Str),
             @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem DomainTheory
.NeighbourhoodSystems.ExampleB.Str
                 D_0 X  ->
               @Set.Nonempty DomainTheory.NeighbourhoodSystems.ExampleB.Str X}
         {h_1 :
           forall  (Y : Set DomainTheory.NeighbourhoodSystems.ExampleB.Str),
             @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem DomainTheory
.NeighbourhoodSystems.ExampleB.Str
                 D_1 Y  ->
               @Set.Nonempty DomainTheory.NeighbourhoodSystems.ExampleB.Str Y},
        @DomainTheory.NeighbourhoodSystems.Trianglelefteq DomainTheory.
NeighbourhoodSystems.ExampleB.Str
           DomainTheory.NeighbourhoodSystems.ExampleB.Str D_0
           (DomainTheory.NeighbourhoodSystems.Exercise619.oplusTok D_0 D_1 h_0 h_1)
))
     (forall
       (D_0 D_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem DomainTheory
.NeighbourhoodSystems.ExampleB.Str),
       @DomainTheory.NeighbourhoodSystems.Trianglelefteq DomainTheory.
NeighbourhoodSystems.ExampleB.Str
         DomainTheory.NeighbourhoodSystems.ExampleB.Str D_0
         (DomainTheory.NeighbourhoodSystems.Exercise619.prodTok D_0 D_1)))
   (Not
     (@DomainTheory.NeighbourhoodSystems.Trianglelefteq DomainTheory.
NeighbourhoodSystems.ExampleB.Str
       DomainTheory.NeighbourhoodSystems.ExampleB.Str DomainTheory.
NeighbourhoodSystems.Exercise627.twoPt
       (DomainTheory.NeighbourhoodSystems.Exercise619.otimesTok DomainTheory.
NeighbourhoodSystems.Exercise627.twoPt
         DomainTheory.NeighbourhoodSystems.Exercise627.unitPt)))
   (@And.intro
     (And
       (forall
         (D_0 D_1 :
           DomainTheory.NeighbourhoodSystems.NeighborhoodSystem DomainTheory.
NeighbourhoodSystems.ExampleB.Str),
         @DomainTheory.NeighbourhoodSystems.Trianglelefteq DomainTheory.
NeighbourhoodSystems.ExampleB.Str
           DomainTheory.NeighbourhoodSystems.ExampleB.Str
           (DomainTheory.NeighbourhoodSystems.Exercise619.otimesTok D_0 D_1)
           (DomainTheory.NeighbourhoodSystems.Exercise619.prodTok D_0 D_1))
       (forall
         {D_0 D_1 :
           DomainTheory.NeighbourhoodSystems.NeighborhoodSystem DomainTheory.
NeighbourhoodSystems.ExampleB.Str}
         {h_0 :
           forall  (X : Set DomainTheory.NeighbourhoodSystems.ExampleB.Str),
             @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem DomainTheory
.NeighbourhoodSystems.ExampleB.Str
                 D_0 X  ->
```

```
              @Set.Nonempty DomainTheory.NeighbourhoodSystems.ExampleB.Str X}
         {h_1 :
           forall  (Y : Set DomainTheory.NeighbourhoodSystems.ExampleB.Str),
             @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem DomainTheory
  .NeighbourhoodSystems.ExampleB.Str
               D_1 Y  ->
             @Set.Nonempty DomainTheory.NeighbourhoodSystems.ExampleB.Str Y},
        @DomainTheory.NeighbourhoodSystems.Trianglelefteq DomainTheory.
  NeighbourhoodSystems.ExampleB.Str
          DomainTheory.NeighbourhoodSystems.ExampleB.Str D_0
          (DomainTheory.NeighbourhoodSystems.Exercise619.oplusTok D_0 D_1 h_0 h_1)
  ))
     (forall
       (D_0 D_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem DomainTheory
  .NeighbourhoodSystems.ExampleB.Str),
       @DomainTheory.NeighbourhoodSystems.Trianglelefteq DomainTheory.
  NeighbourhoodSystems.ExampleB.Str
         DomainTheory.NeighbourhoodSystems.ExampleB.Str D_0
         (DomainTheory.NeighbourhoodSystems.Exercise619.prodTok D_0 D_1))
     (@And.intro
       (forall
         (D_0 D_1 :
           DomainTheory.NeighbourhoodSystems.NeighborhoodSystem DomainTheory.
  NeighbourhoodSystems.ExampleB.Str),
         @DomainTheory.NeighbourhoodSystems.Trianglelefteq DomainTheory.
  NeighbourhoodSystems.ExampleB.Str
           DomainTheory.NeighbourhoodSystems.ExampleB.Str
           (DomainTheory.NeighbourhoodSystems.Exercise619.otimesTok D_0 D_1)
           (DomainTheory.NeighbourhoodSystems.Exercise619.prodTok D_0 D_1))
       (forall
         {D_0 D_1 :
           DomainTheory.NeighbourhoodSystems.NeighborhoodSystem DomainTheory.
  NeighbourhoodSystems.ExampleB.Str}
         {h_0 :
           forall  (X : Set DomainTheory.NeighbourhoodSystems.ExampleB.Str),
             @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem DomainTheory
  .NeighbourhoodSystems.ExampleB.Str
               D_0 X  ->
             @Set.Nonempty DomainTheory.NeighbourhoodSystems.ExampleB.Str X}
         {h_1 :
           forall  (Y : Set DomainTheory.NeighbourhoodSystems.ExampleB.Str),
             @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem DomainTheory
  .NeighbourhoodSystems.ExampleB.Str
               D_1 Y  ->
             @Set.Nonempty DomainTheory.NeighbourhoodSystems.ExampleB.Str Y},
        @DomainTheory.NeighbourhoodSystems.Trianglelefteq DomainTheory.
  NeighbourhoodSystems.ExampleB.Str
          DomainTheory.NeighbourhoodSystems.ExampleB.Str D_0
          (DomainTheory.NeighbourhoodSystems.Exercise619.oplusTok D_0 D_1 h_0 h_1)
  )
       DomainTheory.NeighbourhoodSystems.Exercise627.otimes_trianglelefteq_prod
       @DomainTheory.NeighbourhoodSystems.Exercise627.inl_trianglelefteq_oplus)
     DomainTheory.NeighbourhoodSystems.Exercise627.fst_trianglelefteq_prod)
   DomainTheory.NeighbourhoodSystems.Exercise627.not_trianglelefteq_otimes)
  @DomainTheory.NeighbourhoodSystems.Exercise627.oplus_trianglelefteq_sum)
@DomainTheory.NeighbourhoodSystems.Exercise627.strictFun_trianglelefteq_funSpace
```

```
end

end DomainTheory.NeighbourhoodSystems.Exercise627
end

section
-- Theorem614/Setup.lean
universe w

namespace DomainTheory.NeighbourhoodSystems
namespace Theorem614

open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.Exercise510

variable {a : Type w}

/-- The hypotheses of Theorem 6.14, bundled: a functor `T` that is continuous on maps,
    monotone and
continuous on domains, together with a generating system `Gamma` over a token type `Tok
    ` such that
`{Gamma} ? T({Gamma})` (the carrier of `T({Gamma})` is identified with `Tok` by `ceq`,
    and `hsub` is Scott's
`{Gamma} ? T({Gamma})`). -/
structure Setup where
  /-- The functor. -/
  T : Endofunctor DomainObj.{w}
  /-- `T` is continuous on maps (Definition 6.8). -/
  hmaps : ContinuousOnMaps T
  /-- `T` is monotone on domains (Definition 6.13). -/
  hmono : MonotoneOnDomains T
  /-- `T` is continuous on domains (Definition 6.13). -/
  hcont : ContinuousOnDomains T
  {Tok : Type w}
  /-- The generating system `{Gamma}`. -/
  Gamma : NeighborhoodSystem Tok
  /-- `T({Gamma})` is a system over the same token type. -/
  ceq : (T.obj <Tok, Gamma>).carrier = Tok
  /-- Scott's hypothesis `{Gamma} ? T({Gamma})`. -/
  hsub : Gamma ? (ceq  |>  (T.obj <Tok, Gamma>).sys : NeighborhoodSystem Tok)

end Theorem614
end DomainTheory.NeighbourhoodSystems
end

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.Exercise510

section
universe w

namespace Theorem614

variable {a : Type w}

/-- Membership in a transported system: `(e  |>  V).mem X  <->  V.mem (e.symm  |>  X)`.
```

```
    -/
theorem mem_cast {b : Type w} (e : b = a) (V : NeighborhoodSystem b) (X : Set a) :
    (e  |>  V : NeighborhoodSystem a).mem X  <->  V.mem (e.symm  |>  X : Set b) := by
  cases e; rfl

/-- Transport composition for neighbourhood systems: `e'  |>  (e  |>  x) = (e.trans e')
     |>  x`. -/
theorem rec_trans {b ? : Type w} (e : b = ?) (e' : ? = a) (x : NeighborhoodSystem b) :
    (e'  |>  (e  |>  x : NeighborhoodSystem ?) : NeighborhoodSystem a) = (e.trans e')
     |>  x := by
  cases e; cases e'; rfl

/-- Transport composition for sets: `e'  |>  (e  |>  X) = (e.trans e')  |>  X`. -/
theorem set_rec_trans {b ? : Type w} (e : b = ?) (e' : ? = a) (X : Set b) :
    (e'  |>  (e  |>  X : Set ?) : Set a) = (e.trans e')  |>  X := by
  cases e; cases e'; rfl

/-- Transport a subsystem relation `D ? E` along a carrier-type equality `b = a`.
    Choice-free. -/
theorem subsystem_cast {b : Type w} (e : b = a) {D E : NeighborhoodSystem b} (h : D ?
    E) :
    (e  |>  D : NeighborhoodSystem a) ? (e  |>  E : NeighborhoodSystem a) := by
  cases e; exact h

/-- The iterated tower, as data: at level `n`, the system `T?({Gamma})` over `Tok`, the
     carrier
identification `(T.obj T?({Gamma})).carrier = Tok`, and the subdomain relation `T?({
    Gamma}) ? T?^+1({Gamma})`
(where `T?^+1({Gamma})` is the carrier-transport of `T(T?({Gamma}))`). The successor
    step uses
*monotone on domains* (`MonotoneAt`) to obtain the next carrier identification and
    subdomain
relation. Choice-free. -/
def iter (s : Setup.{w}) : (n : Nat)  ->
    Sigma' (S : NeighborhoodSystem s.Tok), Sigma' (ceq : (s.T.obj <s.Tok, S>).carrier
    = s.Tok),
      S ? (ceq  |>  (s.T.obj <s.Tok, S>).sys : NeighborhoodSystem s.Tok)
  | 0 => <s.Gamma, s.ceq, s.hsub>
  | (n + 1) =>
      let p := iter s n
      <p.2.1  |>  (s.T.obj <s.Tok, p.1>).sys,
        (s.hmono p.2.2).carrier_eq.trans p.2.1,
        by
          have hsub := subsystem_cast p.2.1 (s.hmono p.2.2).sub
          rwa [rec_trans] at hsub>

/-- `T?({Gamma})`, the `n`-th system in the tower (over `Tok`). -/
def Dsys (s : Setup.{w}) (n : Nat) : NeighborhoodSystem s.Tok := (iter s n).1

/-- The carrier identification `(T.obj T?({Gamma})).carrier = Tok`. -/
theorem Dceq (s : Setup.{w}) (n : Nat) : (s.T.obj <s.Tok, Dsys s n>).carrier = s.Tok
    := (iter s n).2.1

/-- The basic subdomain step `T?({Gamma}) ? T?^+1({Gamma})`. -/
theorem Dchain (s : Setup.{w}) (n : Nat) : Dsys s n ? Dsys s (n + 1) := (iter s n).2.2
```

```
/-- Every system in the tower has the same master `Delta = Gamma`. -/
theorem Dsys_master (s : Setup.{w}) (n : Nat) : (Dsys s n).master = s.Gamma.master :=
    by
  induction n with
  | zero => rfl
  | succ k ih => rw [ <-  (Dchain s k).master_eq]; exact ih

/-- `T?^+1({Gamma})` is the carrier-transport of `T(T?({Gamma}))`. -/
theorem Dsys_succ (s : Setup.{w}) (n : Nat) :
    Dsys s (n + 1) = (Dceq s n  |>  (s.T.obj <s.Tok, Dsys s n>).sys :
    NeighborhoodSystem s.Tok) :=
  rfl

/-- The tower is a `?`-chain: `T?({Gamma}) ? T?({Gamma})` whenever `n  <=  m`. -/
theorem chain_le (s : Setup.{w}) {n m : Nat} (h : n  <=  m) : Dsys s n ? Dsys s m :=
    by
  induction h with
  | refl => exact Subsystem.refl _
  | step _ ih => exact ih.trans (Dchain s _)

/-- **The colimit `? = ?_n T?({Gamma})`** as a neighbourhood system over `Tok`: a set
    is a neighbourhood
of `?` exactly when it is a neighbourhood of some `T?({Gamma})`. Closure under
    consistent intersection
uses that the tower is a chain (`chain_le`): any finite collection of neighbourhoods
    sits inside one
level `T?({Gamma})`, whose own `inter_mem` finishes the job. -/
def colim (s : Setup.{w}) : NeighborhoodSystem s.Tok where
  mem X := exists  n, (Dsys s n).mem X
  master := s.Gamma.master
  master_nonempty := s.Gamma.master_nonempty
  master_mem := <0, s.Gamma.master_mem>
  inter_mem := by
    rintro X Y Z <n, hX> <m, hY> <p, hZ> hsub
    set N := max n (max m p) with hN
    have hXN : (Dsys s N).mem X := (chain_le s (le_max_left n _)).sub hX
    have hYN : (Dsys s N).mem Y :=
      (chain_le s ((le_max_left m p).trans (le_max_right n _))).sub hY
    have hZN : (Dsys s N).mem Z :=
      (chain_le s ((le_max_right m p).trans (le_max_right n _))).sub hZ
    exact <N, (Dsys s N).inter_mem hXN hYN hZN hsub>
  sub_master := by
    rintro X <n, hX>
    rw [ <-  Dsys_master s n]
    exact (Dsys s n).sub_master hX

@[simp] theorem mem_colim (s : Setup.{w}) {X : Set s.Tok} :
    (colim s).mem X  <->  exists  n, (Dsys s n).mem X := Iff.rfl

@[simp] theorem colim_master (s : Setup.{w}) : (colim s).master = s.Gamma.master :=
    rfl

/-- Each level of the tower is a subdomain of the colimit: `T?({Gamma}) ? ?`. -/
theorem Dsys_sub_colim (s : Setup.{w}) (n : Nat) : Dsys s n ? colim s where
  master_eq := by rw [colim_master, Dsys_master]
  sub hX := <n, hX>
```

```
  inter_closed := by
    rintro X Y hX hY <m, hXY>
    have hN : (Dsys s (max n m)).mem (X  I  Y) := by
      have hle : (Dsys s m) ? (Dsys s (max n m)) := chain_le s (le_max_right n m)
      exact hle.sub hXY
    -- pull `X  I  Y` back into `T?({Gamma})` using consistency-in-the-bigger-level
    exact (chain_le s (le_max_left n m)).inter_closed hX hY hN

/-- The carrier identification `(T.obj ?).carrier = Tok`, from `MonotoneAt` of `T?({
    Gamma}) ? ?`. -/
theorem colimCeq (s : Setup.{w}) : (s.T.obj <s.Tok, colim s>).carrier = s.Tok :=
  (s.hmono (Dsys_sub_colim s 0)).carrier_eq.trans (Dceq s 0)

/-- `T(?)`, the image of the colimit, as a system over `Tok` (via `colimCeq`). -/
def Tcolim (s : Setup.{w}) : NeighborhoodSystem s.Tok :=
  colimCeq s  |>  (s.T.obj <s.Tok, colim s>).sys

/-- `T?^+1({Gamma}) ? T(?)`: applying *monotone on domains* to `T?({Gamma}) ? ?` and
    transporting. -/
theorem Dsys_sub_Tcolim (s : Setup.{w}) (n : Nat) : Dsys s (n + 1) ? Tcolim s := by
  have h := subsystem_cast (Dceq s n) (s.hmono (Dsys_sub_colim s n)).sub
  rw [rec_trans] at h
  exact h

/-- `T(?)` and `?` share the master `Delta = Gamma`. -/
theorem Tcolim_master (s : Setup.{w}) : (Tcolim s).master = s.Gamma.master := by
  rw [ <-  (Dsys_sub_Tcolim s 0).master_eq, Dsys_master]

/-- **The continuity step (the hard half of `T(?) = ?`).** Every neighbourhood of `T(?)
    ` is a
neighbourhood of `?`. This is exactly Scott's `T(?) = T(?_n T?({Gamma})) = ?_n T?^+1({
    Gamma}) = ?`,
obtained from *continuity on domains* applied to the directed family `{T?({Gamma})}`.
    -/
theorem Tcolim_sub_colim (s : Setup.{w}) {X : Set s.Tok} (hX : (Tcolim s).mem X) :
    (colim s).mem X := by
  obtain <hmono', hC> := s.hcont
  set ? : Set (NeighborhoodSystem s.Tok) := Set.range (Dsys s) with h?def
  have h? : forall  ?D?, D  in  ?  ->  D ? colim s := by rintro D <n, rfl>; exact
    Dsys_sub_colim s n
  have hne : ?.Nonempty := <Dsys s 0, <0, rfl>>
  have hdir : DirectedOn (* ? *) ? := by
    rintro _ <n, rfl> _ <m, rfl>
    exact <Dsys s (max n m), <max n m, rfl>,
      chain_le s (le_max_left n m), chain_le s (le_max_right n m)>
  have hU : forall  Y, (colim s).mem Y  <->  exists  D  in  ?, D.mem Y := by
    intro Y; constructor
    * rintro <n, hn>; exact <Dsys s n, <n, rfl>, hn>
    * rintro <D, <n, rfl>, hn>; exact <n, hn>
  have heq := hC ? h? hne hdir (Subsystem.refl (colim s)) hU
  set Y_0 : Set (s.T.obj <s.Tok, colim s>).carrier := (colimCeq s).symm  |>  X with
    hY_0
  -- `X  in  T(?)` says `Y_0  in  targetFam (refl ?)` = the neighbourhood family of `T
    (?)`.
  have hmem : Y_0  in  targetFam s.T hmono' (Subsystem.refl (colim s)) :=
    (mem_cast (colimCeq s) _ X).mp hX
```

```
  rw [heq, Set.mem_iUnion] at hmem
  obtain <D, hmem> := hmem
  rw [Set.mem_iUnion] at hmem
  obtain <hD, hmemD> := hmem
  obtain <n, rfl> := hD
  simp only [targetFam, Set.mem_ofPred_eq] at hmemD
  -- conclude `X  in  T?^+1({Gamma})  subseteq  ?`.
  refine <n + 1, ?_>


  rw [Dsys_succ s n, mem_cast (Dceq s n)]
  have key : ((Dceq s n).symm  |>  X : Set (s.T.obj <s.Tok, Dsys s n>).carrier)
      = (s.hmono (h? <n, rfl>)).carrier_eq  |>  Y_0 := by
    rw [hY_0, set_rec_trans]
  rw [key]
  exact hmemD

/-- The easy half of `T(?) = ?`: every neighbourhood of `?` is a neighbourhood of `T(?)
    `
(`?  subseteq  T(?)`), since `T?({Gamma}) ? T?^+1({Gamma}) ? T(?)`. -/
theorem colim_sub_Tcolim (s : Setup.{w}) {X : Set s.Tok} (hX : (colim s).mem X) :
    (Tcolim s).mem X := by
  obtain <n, hn> := hX
  exact (Dsys_sub_Tcolim s n).sub ((Dchain s n).sub hn)

/-- **`T(?) = ?`** (Scott's `? = T(?)`): the two systems have the same neighbourhoods (
    mutual
inclusion via `colim_sub_Tcolim`/`Tcolim_sub_colim`) and the same master. -/
theorem Tcolim_eq_colim (s : Setup.{w}) : Tcolim s = colim s :=
  NeighborhoodSystem.ext
    (fun _ => <fun h => Tcolim_sub_colim s h, fun h => colim_sub_Tcolim s h>)
    (by rw [Tcolim_master, colim_master])

/-- A `DomainObj` equality from a carrier equality and a transported-system equality.
    -/
theorem domainObj_ext {c : Type w} (sigma : NeighborhoodSystem c) (e : c = a)
    {V : NeighborhoodSystem a} (h : (e  |>  sigma : NeighborhoodSystem a) = V) :
    (<c, sigma> : DomainObj) = <a, V> := by
  cases e; cases h; rfl

/-- **`T(?)  ~=  ?` is the identity**, packaged as a `DomainObj` equality `T(?) = ?`.
    -/
theorem colimObj_eq (s : Setup.{w}) :
    s.T.obj <s.Tok, colim s> = (<s.Tok, colim s> : DomainObj) :=
  domainObj_ext (s.T.obj <s.Tok, colim s>).sys (colimCeq s) (Tcolim_eq_colim s)

/-- The identity isomorphism induced by an object equality in any category. -/
def isoOfEq {Obj : Type*} [Category Obj] {X Y : Obj} (h : X = Y) : Iso X Y := by
  cases h
  exact <(? X), (? X), CategoryTheory.Category.id_comp _, CategoryTheory.Category.
    id_comp _>

/-- The isomorphism `T(?)  ~=  ?` making `?` a `T`-algebra (the identity, since `T(?) =
     ?`). -/
def colimIso (s : Setup.{w}) : Iso (s.T.obj <s.Tok, colim s>) (<s.Tok, colim s> :
    DomainObj) :=
  isoOfEq (colimObj_eq s)
```

```
/-- `?_n = i_n  comp  j_n : ?  ->  ?`, the retraction onto `T?({Gamma})` (Proposition
    6.12's projection pair for
`T?({Gamma}) ? ?`). -/
def rho (s : Setup.{w}) (n : Nat) : ApproximableMap (colim s) (colim s) :=
  (Dsys_sub_colim s n).inj.comp (Dsys_sub_colim s n).proj

/-- Scott's relational description `X ?_n Y  <->  exists  z  in  T?({Gamma}), X
    subseteq  z  subseteq  Y`. -/
theorem rho_rel (s : Setup.{w}) (n : Nat) {X Y : Set s.Tok} :
    (rho s n).rel X Y  <->
      (colim s).mem X  /\  (colim s).mem Y  /\  exists  z, (Dsys s n).mem z  /\  X
    subseteq  z  /\  z  subseteq  Y := by
  unfold rho
  rw [comp_rel]
  constructor
  * rintro <z, <hcX, hDz, hXz>, _, hcY, hzY>
    exact <hcX, hcY, z, hDz, hXz, hzY>
  * rintro <hcX, hcY, z, hDz, hXz, hzY>
    exact <z, <hcX, hDz, hXz>, hDz, hcY, hzY>

/-- `?_n  subseteq  ??` for `n  <=  m` (the projection chain is increasing). -/
theorem rho_mono (s : Setup.{w}) {n m : Nat} (h : n  <=  m) {X Y : Set s.Tok}
    (hr : (rho s n).rel X Y) : (rho s m).rel X Y := by
  rw [rho_rel] at hr  |-
  obtain <hcX, hcY, z, hDz, hXz, hzY> := hr
  exact <hcX, hcY, z, (chain_le s h).sub hDz, hXz, hzY>

/-- The pointwise union `?_n ?_n` (directed, since the chain is increasing). -/
def iSupRho (s : Setup.{w}) : ApproximableMap (colim s) (colim s) :=
  iSupMap (rho s) (fun i j => <max i j,
    fun _ _ h => rho_mono s (le_max_left i j) h,
    fun _ _ h => rho_mono s (le_max_right i j) h>)

/-- The colimit `?` as a `T`-algebra, with structure map the iso `T(?)  ->  ?`. -/
abbrev colimAlg (s : Setup.{w}) : TAlgebra s.T :=
  <<s.Tok, colim s>, (colimIso s).hom>

/-- The colimit `?` as a category object `<Tok, ?>`. -/
abbrev objColim (s : Setup.{w}) : DomainObj := <s.Tok, colim s>

/-- The `n`-th tower system `T?({Gamma})` as a category object `<Tok, T?({Gamma})>`. -/
abbrev objDsys (s : Setup.{w}) (n : Nat) : DomainObj := <s.Tok, Dsys s n>

/-- `T(?_n)` as an endomorphism of `T(?)`, with the category objects pinned (they
    cannot be inferred
from `rho s n`'s `ApproximableMap` type alone). -/
abbrev Tmap_rho (s : Setup.{w}) (n : Nat) :
    ApproximableMap (s.T.obj (objColim s)).sys (s.T.obj (objColim s)).sys :=
  s.T.map (X := objColim s) (Y := objColim s) (rho s n)

@[simp] theorem Dsys_zero (s : Setup.{w}) : Dsys s 0 = s.Gamma := rfl

/-- Two algebra homomorphisms with equal underlying maps are equal (the commuting
    square is a
`Prop`). -/
```

```
theorem algHom_ext {Obj : Type*} [Category Obj] {T : Endofunctor Obj} {A B : TAlgebra
    T}
    {g g' : AlgHom A B} (h : g.hom = g'.hom) : g = g' := by
  cases g; cases g'; cases h; rfl

/-- In the category of domains, `?` (categorical composition) is `ApproximableMap.comp
    `. -/
theorem cat_comp_eq {X Y Z : DomainObj} (g : (Y  -->  Z)) (f : (X  -->  Y)) :
    g ? f = g.comp f := rfl

/-- **`?_n ?_n = I_?`** (Scott's key identity for uniqueness). The forward inclusion
    uses
`X  subseteq  z  subseteq  Y ? X  subseteq  Y`; the reverse factors the identity step `
    X  subseteq  X  subseteq  Y` through the level
witnessing `X  in  ?`. -/
theorem iSupRho_eq_id (s : Setup.{w}) : iSupRho s = idMap (colim s) := by
  apply ApproximableMap.ext
  intro X Y
  rw [idMap_rel]
  constructor
  * rintro <n, hr>
    rw [rho_rel] at hr
    obtain <hcX, hcY, z, _, hXz, hzY> := hr
    exact <hcX, hcY, hXz.trans hzY>
  * rintro <hcX, hcY, hXY>
    obtain <n, hX> := hcX
    exact <n, (rho_rel s n).mpr <<n, hX>, hcY, X, hX, subset_rfl, hXY>>

/-- Conjugation by the identity isomorphism `isoOfEq e` is the object-transport along `
    e`. -/
theorem isoOfEq_conj {Obj : Type*} [Category Obj] {X Y : Obj} (e : X = Y)
    (f : (X  -->  X)) :
    (isoOfEq e).hom ? f ? (isoOfEq e).inv = (e  |>  f : (Y  -->  Y)) := by
  cases e
  change (? X) ? f ? (? X) = f
  dsimp only [compAfter]
  rw [CategoryTheory.Category.comp_id, CategoryTheory.Category.id_comp]

/-- **The carrier-transport core of `T(?_n) = ?_n_+_1`.** Given the *monotone-on-
    domains* data for a
subsystem (its injection `Tmi`/projection `Tmj` are heterogeneously equal to the
    canonical 6.12 pair
`sub.inj`/`sub.proj` of the image subsystem `sub : Ps ? ce  |>  Qs`), the composite `
    Tmi  comp  Tmj` is --
after carrying the functor-image carriers `Pc, Qc` down to `Tok` -- exactly the
    projection
`i_n_+_1  comp  j_n_+_1` of the next subsystem `hsub' : Dn1 ? Col`. Proved by `subst`
    ing the carrier equalities,
after which proof-irrelevance identifies the two subsystem proofs. -/
theorem map_comp_proj_heq {Tok : Type w} {Pc Qc : Type w} (cn : Pc = Tok) (cc : Qc =
    Tok)
    {Ps : NeighborhoodSystem Pc} {Qs : NeighborhoodSystem Qc} (ce : Qc = Pc)
    (sub : Ps ? (ce  |>  Qs : NeighborhoodSystem Pc))
    {Dn1 Col : NeighborhoodSystem Tok}
    (hDn1 : (cn  |>  Ps : NeighborhoodSystem Tok) = Dn1)
    (hCol : (cc  |>  Qs : NeighborhoodSystem Tok) = Col)
```

```
    (hsub' : Dn1 ? Col)
    (Tmi : ApproximableMap Ps Qs) (Tmj : ApproximableMap Qs Ps)
    (hi : HEq Tmi sub.inj) (hj : HEq Tmj sub.proj) :
    HEq (Tmi.comp Tmj) (hsub'.inj.comp hsub'.proj) := by
  subst cn
  subst cc
  obtain rfl := hDn1
  obtain rfl := hCol
  have e1 : Tmi = sub.inj := eq_of_heq hi
  have e2 : Tmj = sub.proj := eq_of_heq hj
  rw [e1, e2]

/-- **`T(?_n) = ?_n_+_1`, heterogeneously.** The image `T(?_n)` of the `n`-th
    projection, living over
`T(?)`'s carrier, is heterogeneously equal to the `(n+1)`-st projection `?_n_+_1` over
    `Tok`. -/
theorem map_rho_heq (s : Setup.{w}) (n : Nat) :
    HEq (Tmap_rho s n) (rho s (n + 1)) := by
  have hcomp : Tmap_rho s n
      = (s.T.map (X := objDsys s n) (Y := objColim s) (Dsys_sub_colim s n).inj).comp
          (s.T.map (X := objColim s) (Y := objDsys s n) (Dsys_sub_colim s n).proj) :=
    s.T.map_comp (X := objColim s) (Y := objDsys s n) (Z := objColim s)
      (Dsys_sub_colim s n).inj (Dsys_sub_colim s n).proj
  rw [hcomp]
  exact map_comp_proj_heq (Dceq s n) (colimCeq s) (s.hmono (Dsys_sub_colim s n)).
    carrier_eq
    (s.hmono (Dsys_sub_colim s n)).sub (Dsys_succ s n).symm (Tcolim_eq_colim s)
    (Dsys_sub_colim s (n + 1)) (s.T.map (X := objDsys s n) (Y := objColim s) (
    Dsys_sub_colim s n).inj)
    (s.T.map (X := objColim s) (Y := objDsys s n) (Dsys_sub_colim s n).proj)
    (s.hmono (Dsys_sub_colim s n)).inj_heq (s.hmono (Dsys_sub_colim s n)).proj_heq

theorem nonempty_strict_algHom (s : Setup.{w}) (B : TAlgebra s.T) (hk : IsStrict B.str
    ) :
    Nonempty {g : AlgHom (colimAlg s) B // IsStrict g.hom} :=
  nonempty_algHom_of_continuousOnMaps s.T s.hmaps (colimIso s) B hk

theorem nonempty_algHom (s : Setup.{w}) (B : TAlgebra s.T) (hk : IsStrict B.str) :
    Nonempty (AlgHom (colimAlg s) B) :=
  (nonempty_strict_algHom s B hk).map (*.1)

end Theorem614
end
end DomainTheory.NeighbourhoodSystems
end
```

3.7.30 Exercise 6.28 — finite embeddings cancel

(Suggested by G. Plotkin). Show that if $\mathcal{D}$ and $\mathcal{E}$ are *finite* systems and

$$\mathcal{D} \trianglelefteq \mathcal{E} \trianglelefteq \mathcal{D}\,,$$

then $\mathcal{D} \cong \mathcal{E}$. Need the same be true of infinite systems?

Exercise_6_28.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.Exercise_6_25

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap
section
variable {a b : Type*} {D : NeighborhoodSystem a} {E : NeighborhoodSystem b}

/- concordance: source L4207 EXERCISE 6.28 finite embeddings cancel -/
/-- **Exercise 6.28 (core).** If `|D|` and `|E|` are finite and `D ? E ? D`, then `D
    ~= ? E`.
The two `?`s give mutual order embeddings of the (finite) element domains, and
`orderIso_of_embeddings` upgrades them to an isomorphism. -/
theorem isomorphic_of_trianglelefteq_both [Finite D.Element] [Finite E.Element]
    (h_1 : D ? E) (h_2 : E ? D) : D  ~= ? E := by
  obtain <f> := h_1.elementEmbedding
  obtain <g> := h_2.elementEmbedding
  exact orderIso_of_embeddings f g

/-- **Exercise 6.28 (Scott 1981, PRG-19; suggested by G. Plotkin).** If `?` and `?` are
     *finite*
neighbourhood systems with `? ? ?` and `? ? ?`, then `?  ~=  ?`. -/
theorem isomorphic_of_finite_system (hD : D.IsFinite) (hE : E.IsFinite)
    (h_1 : D ? E) (h_2 : E ? D) : D  ~= ? E := by
  have := finite_element_of_isFinite hD
  have := finite_element_of_isFinite hE
  exact isomorphic_of_trianglelefteq_both h_1 h_2

end

section
set_option linter.unusedVariables false

theorem example_628.{u_1, u_2} : And
  (forall  {a : Type u_1} {b : Type u_2} {D : DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem a}
    {E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b},
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsFinite a D  ->
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsFinite b E  ->
        @DomainTheory.NeighbourhoodSystems.Trianglelefteq a b D E  ->
          @DomainTheory.NeighbourhoodSystems.Trianglelefteq b a E D  ->
            @DomainTheory.NeighbourhoodSystems.Isomorphic a b D E)
  (forall  {a : Type u_1} {b : Type u_2} {D : DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem a}
    {E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
    [Finite (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a D)]
    [Finite (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element b E)],
    @DomainTheory.NeighbourhoodSystems.Trianglelefteq a b D E  ->
      @DomainTheory.NeighbourhoodSystems.Trianglelefteq b a E D  ->  @DomainTheory.
    NeighbourhoodSystems.Isomorphic a b D E) :=
  @And.intro
  (forall  {a : Type u_1} {b : Type u_2} {D : DomainTheory.NeighbourhoodSystems.
```

```
    NeighborhoodSystem a}
    {E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b},
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsFinite a D  ->
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsFinite b E  ->
        @DomainTheory.NeighbourhoodSystems.Trianglelefteq a b D E  ->
          @DomainTheory.NeighbourhoodSystems.Trianglelefteq b a E D  ->
            @DomainTheory.NeighbourhoodSystems.Isomorphic a b D E)
  (forall  {a : Type u_1} {b : Type u_2} {D : DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem a}
    {E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
    [Finite (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a D)]
    [Finite (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element b E)],
    @DomainTheory.NeighbourhoodSystems.Trianglelefteq a b D E  ->
      @DomainTheory.NeighbourhoodSystems.Trianglelefteq b a E D  ->  @DomainTheory.
    NeighbourhoodSystems.Isomorphic a b D E)
  @DomainTheory.NeighbourhoodSystems.isomorphic_of_finite_system
  @DomainTheory.NeighbourhoodSystems.isomorphic_of_trianglelefteq_both
end

section
variable {a : Type*}

namespace Subsystem

variable {D E : NeighborhoodSystem a}

/-- **Proposition 6.12 (Scott 1981, PRG-19).** Every subdomain relation `D ? E` gives
    rise to a
projection pair `i : D  ->  E`, `j : E  ->  D` with `j  comp  i = I_D` and `i  comp  j
     subseteq  I_E`. -/
def projectionPair (h : D ? E) : ProjectionPair D E where
  inj := h.inj
  proj := h.proj
  proj_comp_inj := h.proj_comp_inj
  inj_comp_proj_le := h.inj_comp_proj_le

/-- **Element-wise description of `i` (Scott's equation).** `i(x) = {Y  in  E ? exists
     X  in  x, X  subseteq  Y}`. -/
theorem toElementMap_inj (h : D ? E) (x : D.Element) {Y : Set a} :
    (h.inj.toElementMap x).mem Y  <->  E.mem Y  /\  exists  X, x.mem X  /\  X
    subseteq  Y := by
  constructor
  * rintro <X, hX, _, hY, hXY>
    exact <hY, X, hX, hXY>
  * rintro <hY, X, hX, hXY>
    exact <X, hX, x.sub hX, hY, hXY>

/-- **Element-wise description of `j` (Scott's equation).** `j(y) = y  I  D`: the
    neighbourhoods of
`j(y)` are exactly the `D`-neighbourhoods that already belong to `y`. -/
theorem toElementMap_proj (h : D ? E) (y : E.Element) {X : Set a} :
    (h.proj.toElementMap y).mem X  <->  y.mem X  /\  D.mem X := by
  constructor
  * rintro <Y, hY, hEY, hX, hYX>
    exact <y.up_mem hY (h.sub hX) hYX, hX>
  * rintro <hX, hDX>
```

```
    exact <X, hX, y.sub hX, hDX, subset_rfl>

end Subsystem
end

section
export CategoryTheory (Category)

open scoped CategoryTheory

universe u

abbrev compAfter {Obj : Type u} [Category.{v} Obj] {X Y Z : Obj} (g : Y  -->  Z) (f :
    X  -->  Y) : X  -->  Z :=
  f ? g
end

section
export CategoryTheory (Category)

open scoped CategoryTheory

scoped infixr:80 " ? " => compAfter
end

section
export CategoryTheory (Category)
end

end DomainTheory.NeighbourhoodSystems
end
```

3.7.31 Exercise 6.29 — infinitary sum and product

Generalize $+$ and $\times$ to infinitary operations on domains:

$$\sum_{n=0}^{\infty} D_n \quad \text{and} \quad \prod_{n=0}^{\infty} D_n.$$

Would a similar generalization be possible for $\oplus$ and $\otimes$?

Exercise_6_29.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.Exercise_6_19

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem
namespace Exercise629
```

```
universe u v
variable {? : Type u} {a : ?  ->  Type v}
variable {D : forall  i, NeighborhoodSystem (a i)}

section IndexedProduct
variable [Nonempty ?]
variable (D) in
/- concordance: source L4215 EXERCISE 6.29 infinitary sum and product -/
/-- **Exercise 6.29 -- the indexed product `?_i D_i`.** The index type is non-empty so
    the
dependent-sum token set contains a token. Neighbourhoods are cylinders: tuples `X i  in
      ?_i`
that are the master in all but finitely many coordinates. -/
def iprod : NeighborhoodSystem (Sigma i, a i) where
  mem W := exists  X : forall  i, Set (a i), (forall  i, (D i).mem (X i))  /\  FinSupp
     D X  /\  W = iprodNbhd X
  master := iprodNbhd (fun i => (D i).master)
  master_nonempty := by
    cases ?Nonempty ?? with
    | intro i =>
      obtain <a, ha> := (D i).master_nonempty
      exact <<i, a>, ha>
  master_mem := <fun i => (D i).master, fun i => (D i).master_mem, FinSupp.master, rfl
    >
  sub_master := by
    rintro W <X, hX, _, rfl>
    exact iprodNbhd_subset_iff.mpr fun i => (D i).sub_master (hX i)
  inter_mem := by
    rintro W W' Z <X, hX, hXf, rfl> <X', hX', hX'f, rfl> <ZZ, hZZ, _, rfl> hsub
    rw [iprodNbhd_inter] at hsub  |-
    refine <fun i => X i  I  X' i, fun i => ?_, hXf.inter hX'f, rfl>
    have : ZZ i  subseteq  X i  I  X' i := iprodNbhd_subset_iff.mp hsub i
    exact (D i).inter_mem (hX i) (hX' i) (hZZ i) this

end IndexedProduct

section IndexedProduct

variable [Nonempty ?]

section Iso

variable [DecidableEq ?]

/-- The tuple that is `U` at coordinate `i` and the master elsewhere. -/
def updTuple (D : forall  i, NeighborhoodSystem (a i)) (i : ?) (U : Set (a i)) :
    forall  j, Set (a j) :=
  Function.update (fun j => (D j).master) i U

end Iso

end IndexedProduct

section IndexedProduct

variable [Nonempty ?]
```

```
section Iso

variable [DecidableEq ?]

omit [Nonempty ?] in
@[simp] theorem updTuple_apply_self (i : ?) (U : Set (a i)) : updTuple D i U i = U :=
    by
  simp [updTuple]

end Iso

end IndexedProduct

section IndexedProduct

variable [Nonempty ?]

section Iso

variable [DecidableEq ?]

/-- **The element of `?_i D_i` assembled from a tuple of components.** -/
def fromPi (D : forall  i, NeighborhoodSystem (a i)) (x : forall  i, (D i).Element) :
    (iprod D).Element where
  mem W := exists  X : forall  i, Set (a i), (forall  i, (x i).mem (X i))  /\  FinSupp
     D X  /\  W = iprodNbhd X
  sub := by rintro W <X, hX, hXf, rfl>; exact <X, fun i => (x i).sub (hX i), hXf, rfl>
  master_mem := <fun i => (D i).master, fun i => (x i).master_mem, FinSupp.master, rfl
    >
  inter_mem := by
    rintro W W' <X, hX, hXf, rfl> <X', hX', hX'f, rfl>
    exact <fun i => X i  I  X' i, fun i => (x i).inter_mem (hX i) (hX' i), hXf.inter
    hX'f,
      iprodNbhd_inter X X'>
  up_mem := by
    rintro W W' <X, hX, _, rfl> hW' hsub
    obtain <X', hX'mem, hX'f, rfl> := hW'
    rw [iprodNbhd_subset_iff] at hsub
    exact <X', fun i => (x i).up_mem (hX i) (hX'mem i) (hsub i), hX'f, rfl>

end Iso

end IndexedProduct

/-- A tagged copy `inj i X = {some <i, a> ? a  in  X}` of summand `i`. -/
def injI (i : ?) (X : Set (a i)) : Set (Option (Sigma i, a i)) :=
  (fun a => (some <i, a> : Option (Sigma i, a i))) '' X

theorem none_not_mem_injI {i : ?} {X : Set (a i)} : (none : Option (Sigma i, a i))
    notin  injI i X := by
  rintro <a, _, heq>; simp at heq

section IndexedProduct

variable [Nonempty ?]
```

```
section Iso

variable [DecidableEq ?]

/-- The cylinder restricted to a list of coordinates `l` (master outside `l`). -/
def restrictTo (D : forall  i, NeighborhoodSystem (a i)) (l : List ?) (X : forall  i,
    Set (a i)) :
    forall  j, Set (a j) := fun j => if j  in  l then X j else (D j).master

end Iso

end IndexedProduct

section IndexedProduct

variable [Nonempty ?]

section Iso

variable [DecidableEq ?]

/-- The slice cylinder: `U` at coordinate `i`, master elsewhere. -/
def slice (D : forall  i, NeighborhoodSystem (a i)) (i : ?) (U : Set (a i)) : Set (
    Sigma i, a i) :=
  iprodNbhd (updTuple D i U)

end Iso

end IndexedProduct

section IndexedProduct

variable [Nonempty ?]

section Iso

variable [DecidableEq ?]

/-- Recovering the coordinate from a slice neighbourhood. -/
theorem iprod_mem_slice_inv {i : ?} {U : Set (a i)} (h : (iprod D).mem (slice D i U))
    :
    (D i).mem U := by
  obtain <X, hX, _, heq> := h
  have hXeq : updTuple D i U = X := iprodNbhd_injective heq
  have hi := hX i
  rwa [ <-  hXeq, updTuple_apply_self] at hi

end Iso

end IndexedProduct

section IndexedProduct

variable [Nonempty ?]
```

```
section Iso

variable [DecidableEq ?]

omit [Nonempty ?] in
theorem slice_eq (i : ?) (U : Set (a i)) : slice D i U = iprodNbhd (updTuple D i U) :=
     rfl

end Iso

end IndexedProduct

@[simp] theorem some_mem_injI {i : ?} {a : a i} {X : Set (a i)} :
    (some <i, a> : Option (Sigma i, a i))  in  injI i X  <->  a  in  X := by
  constructor
  * rintro <b, hb, heq>
    simp only [Option.some.injEq, Sigma.mk.injEq, heq_eq_eq, true_and] at heq
    rwa [heq] at hb
  * intro ha; exact <a, ha, rfl>

/-- A tagged token determines its summand index. -/
theorem index_of_some_mem_injI {i k : ?} {a : a k} {X : Set (a i)}
    (hmem : (some <k, a> : Option (Sigma i, a i))  in  injI i X) : k = i := by
  obtain <b, _, heq> := hmem
  simp only [Option.some.injEq] at heq
  exact (congrArg Sigma.fst heq).symm

theorem injI_inter_same (i : ?) (X Y : Set (a i)) : injI i X  I  injI i Y = injI i (X
     I  Y) := by
  apply Set.ext
  rintro (_ | <k, a>)
  * constructor
    * rintro <h, _>; exact absurd h none_not_mem_injI
    * intro h; exact absurd h none_not_mem_injI
  * constructor
    * rintro <hx, hy>
      obtain rfl := index_of_some_mem_injI hx
      exact some_mem_injI.mpr <some_mem_injI.mp hx, some_mem_injI.mp hy>
    * intro h
      obtain rfl := index_of_some_mem_injI h
      have h' := some_mem_injI.mp h
      exact <some_mem_injI.mpr h'.1, some_mem_injI.mpr h'.2>

theorem injI_subset_iff {i : ?} {X Y : Set (a i)} : injI i X  subseteq  injI i Y  <->
     X  subseteq  Y := by
  constructor
  * intro h a ha; exact some_mem_injI.mp (h (some_mem_injI.mpr ha))
  * intro h; rintro w <a, ha, rfl>; exact <a, h ha, rfl>

/-- **Exercise 6.29 -- the indexed smash product `?_i D_i`.** The index type is non-
    empty so its
dependent-sum token set is non-empty. A proper neighbourhood is a cylinder proper in *
    every*
coordinate; closure under intersection is as for `iprod` plus the observation that
`X i  I  X' i  subseteq  X i  != ` master stays proper. -/
def iotimes [Nonempty ?] (D : forall  i, NeighborhoodSystem (a i)) :
```

```
    NeighborhoodSystem (Sigma i, a i) where
  mem W := W = iprodNbhd (fun i => (D i).master)  \/
    exists  X : forall  i, Set (a i), (forall  i, (D i).mem (X i))  /\  (forall  i, X
    i  !=  (D i).master)  /\
      FinSupp D X  /\  W = iprodNbhd X
  master := iprodNbhd (fun i => (D i).master)
  master_nonempty := by
    cases ?Nonempty ?? with
    | intro i =>
      obtain <a, ha> := (D i).master_nonempty
      exact <<i, a>, ha>
  master_mem := Or.inl rfl
  sub_master := by
    rintro W (rfl | <X, hX, _, _, rfl>)
    * exact subset_rfl
    * exact iprodNbhd_subset_iff.mpr fun i => (D i).sub_master (hX i)
  inter_mem := by
    rintro W W' Z hW hW' hZ hsub
    rcases hW with rfl | <X, hX, hXne, hXf, rfl>
    * have hW'sub : W'  subseteq  iprodNbhd (fun i => (D i).master) := by
        rcases hW' with rfl | <X', hX', _, _, rfl>
        * exact subset_rfl
        * exact iprodNbhd_subset_iff.mpr fun i => (D i).sub_master (hX' i)
      rw [Set.inter_eq_right.mpr hW'sub]; exact hW'
    * rcases hW' with rfl | <X', hX', hX'ne, hX'f, rfl>
      * rw [Set.inter_eq_left.mpr (iprodNbhd_subset_iff.mpr fun i => (D i).sub_master
    (hX i))]
        exact Or.inr <X, hX, hXne, hXf, rfl>
      * rw [iprodNbhd_inter] at hsub  |-
        obtain <ZZ, hZZmem, rfl> : exists  ZZ, (forall  i, (D i).mem (ZZ i))  /\  Z =
    iprodNbhd ZZ := by
          rcases hZ with rfl | <ZZ, hZZ, _, _, rfl>
          * exact <fun i => (D i).master, fun i => (D i).master_mem, rfl>
          * exact <ZZ, hZZ, rfl>
        refine Or.inr <fun i => X i  I  X' i, fun i => ?_, fun i => ?_, ?_, rfl>
        * exact (D i).inter_mem (hX i) (hX' i) (hZZmem i) (iprodNbhd_subset_iff.mp
    hsub i)
        * intro heq
          exact hXne i (Set.Subset.antisymm ((D i).sub_master (hX i))
            (by rw [ <-  heq]; exact Set.inter_subset_left))
        * exact hXf.inter hX'f

/-- The basepoint master of the indexed sum: `{none}  U  {some <i, a> ? a  in  Delta_i
    }`. -/
def sumMasterI (D : forall  i, NeighborhoodSystem (a i)) : Set (Option (Sigma i, a i))
     :=
  insert none ((fun p => (some p : Option (Sigma i, a i))) '' iprodNbhd (fun i => (D i
    ).master))

theorem injI_subset_sumMasterI {i : ?} {X : Set (a i)} (hX : (D i).mem X) :
    injI i X  subseteq  sumMasterI D := by
  rintro w <a, ha, rfl>
  exact Set.mem_insert_iff.mpr (Or.inr <<i, a>, (D i).sub_master hX ha, rfl>)

theorem none_mem_sumMasterI : (none : Option (Sigma i, a i))  in  sumMasterI D := Set.
    mem_insert _ _
```

```
/-- **Exercise 6.29 -- the indexed coalesced sum `?_i D_i`.** -/
def ioplus (D : forall  i, NeighborhoodSystem (a i))
    (hne : forall  i, forall  X : Set (a i), (D i).mem X  ->  X.Nonempty) :
    NeighborhoodSystem (Option (Sigma i, a i)) where
  mem W := W = sumMasterI D  \/  exists  i, exists  X : Set (a i), (D i).mem X  /\  X
     !=  (D i).master  /\  W = injI i X
  master := sumMasterI D
  master_nonempty := <none, none_mem_sumMasterI>
  master_mem := Or.inl rfl
  sub_master := by
    rintro W (rfl | <i, X, hX, _, rfl>)
    * exact subset_rfl
    * exact injI_subset_sumMasterI hX
  inter_mem := by
    rintro W W' Z hW hW' hZ hsub
    rcases hW with rfl | <i, X, hX, hXne, rfl>
    * have hsub' : W'  subseteq  sumMasterI D := by
        rcases hW' with rfl | <j, Y, hY, _, rfl>
        * exact subset_rfl
        * exact injI_subset_sumMasterI hY
      rw [Set.inter_eq_right.mpr hsub']; exact hW'
    * rcases hW' with rfl | <j, Y, hY, hYne, rfl>
      * rw [Set.inter_eq_left.mpr (injI_subset_sumMasterI hX)]
        exact Or.inr <i, X, hX, hXne, rfl>
      * rcases hZ with rfl | <k, Wk, hWk, _, rfl>
        * exact absurd (hsub none_mem_sumMasterI).1 none_not_mem_injI
        * obtain <w, hw> := hne k Wk hWk
          have hmemZ := hsub (some_mem_injI.mpr hw)
          obtain rfl := index_of_some_mem_injI hmemZ.1
          obtain rfl := index_of_some_mem_injI hmemZ.2
          rw [injI_inter_same] at hsub  |-
          exact Or.inr <k, X  I  Y, (D k).inter_mem hX hY hWk (injI_subset_iff.mp hsub
   ),
            fun heq => hXne (Set.Subset.antisymm ((D k).sub_master hX)
              (heq  |>  Set.inter_subset_left)), rfl>

/-- **Exercise 6.29 -- the indexed separated sum `?_i D_i`.** -/
def isum (D : forall  i, NeighborhoodSystem (a i))
    (hne : forall  i, forall  X : Set (a i), (D i).mem X  ->  X.Nonempty) :
    NeighborhoodSystem (Option (Sigma i, a i)) where
  mem W := W = sumMasterI D  \/  exists  i, exists  X : Set (a i), (D i).mem X  /\  W
    = injI i X
  master := sumMasterI D
  master_nonempty := <none, none_mem_sumMasterI>
  master_mem := Or.inl rfl
  sub_master := by
    rintro W (rfl | <i, X, hX, rfl>)
    * exact subset_rfl
    * exact injI_subset_sumMasterI hX
  inter_mem := by
    rintro W W' Z hW hW' hZ hsub
    rcases hW with rfl | <i, X, hX, rfl>
    * have hsub' : W'  subseteq  sumMasterI D := by
        rcases hW' with rfl | <j, Y, hY, rfl>
        * exact subset_rfl
```

```
        * exact injI_subset_sumMasterI hY
      rw [Set.inter_eq_right.mpr hsub']; exact hW'
    * rcases hW' with rfl | <j, Y, hY, rfl>
      * rw [Set.inter_eq_left.mpr (injI_subset_sumMasterI hX)]
        exact Or.inr <i, X, hX, rfl>
      * -- both proper: the consistency witness `Z` forces `i = j` constructively.
        rcases hZ with rfl | <k, Wk, hWk, rfl>
        * exact absurd (hsub none_mem_sumMasterI).1 none_not_mem_injI
        * obtain <w, hw> := hne k Wk hWk
          have hmemZ := hsub (some_mem_injI.mpr hw)
          obtain rfl := index_of_some_mem_injI hmemZ.1
          obtain rfl := index_of_some_mem_injI hmemZ.2
          rw [injI_inter_same] at hsub  |-
          exact Or.inr <k, X  I  Y, (D k).inter_mem hX hY hWk (injI_subset_iff.mp hsub
    ), rfl>

section IndexedProduct

variable [Nonempty ?]

section Iso

variable [DecidableEq ?]

omit [Nonempty ?] in
theorem updTuple_apply_ne {i j : ?} (U : Set (a i)) (h : j  !=  i) :
    updTuple D i U j = (D j).master := by
  simp [updTuple, Function.update_of_ne h]

end Iso

end IndexedProduct

section IndexedProduct

variable [Nonempty ?]

section Iso

variable [DecidableEq ?]

theorem fromPi_mem_slice (x : forall  i, (D i).Element) (i : ?) (U : Set (a i)) :
    (fromPi D x).mem (slice D i U)  <->  (x i).mem U := by
  constructor
  * rintro <X, hX, _, heq>
    have hXeq : updTuple D i U = X := iprodNbhd_injective heq
    have hi := hX i
    rwa [ <-  hXeq, updTuple_apply_self] at hi
  * intro hU
    refine <updTuple D i U, fun j => ?_, <[i], fun j hj => ?_>, rfl>
    * by_cases h : j = i
      * subst h; rw [updTuple_apply_self]; exact hU
      * rw [updTuple_apply_ne U h]; exact (x j).master_mem
    * exact updTuple_apply_ne U fun h => hj (List.mem_singleton.mpr h)

end Iso
```

```
end IndexedProduct

section IndexedProduct

variable [Nonempty ?]

section Iso

variable [DecidableEq ?]

omit [Nonempty ?] in
theorem iprodNbhd_restrictTo_cons {X : forall  i, Set (a i)} (hXsub : forall  i, X i
    subseteq  (D i).master)
    (a : ?) (l : List ?) :
    iprodNbhd (restrictTo D (a :: l) X) = slice D a (X a)  I  iprodNbhd (restrictTo D
    l X) := by
  rw [slice_eq, iprodNbhd_inter]
  congr 1; funext j
  show (if j  in  a :: l then X j else (D j).master)
      = updTuple D a (X a) j  I  (if j  in  l then X j else (D j).master)
  by_cases hja : j = a
  * subst hja
    rw [updTuple_apply_self]
    by_cases hjl : j  in  l
    * rw [ite_eq_left List.mem_cons_self, ite_eq_left hjl, Set.inter_self]
    * rw [ite_eq_left List.mem_cons_self, ite_eq_right hjl, Set.inter_eq_left.mpr (
    hXsub j)]
  * rw [updTuple_apply_ne (X a) hja]
    by_cases hjl : j  in  l
    * rw [ite_eq_left (List.mem_cons_of_mem a hjl), ite_eq_left hjl, Set.
    inter_eq_right.mpr (hXsub j)]
    * rw [ite_eq_right fun h => (List.mem_cons.mp h).elim hja hjl, ite_eq_right hjl,
    Set.inter_self]


end Iso

end IndexedProduct

section IndexedProduct

variable [Nonempty ?]

section Iso

variable [DecidableEq ?]

omit [Nonempty ?] in
/-- A cylinder is contained in each of its own slices. -/
theorem iprodNbhd_subset_slice {X : forall  i, Set (a i)} (hX : forall  i, (D i).mem (
    X i)) (i : ?) :
    iprodNbhd X  subseteq  slice D i (X i) := by
  rw [slice_eq, iprodNbhd_subset_iff]
  intro j
  by_cases h : j = i
```

```
  * subst h; rw [updTuple_apply_self]
  * rw [updTuple_apply_ne (X i) h]; exact (D j).sub_master (hX j)

end Iso

end IndexedProduct

section IndexedProduct

variable [Nonempty ?]

section Iso

variable [DecidableEq ?]

/-- A slice has support ` subseteq  {i}`, hence is a neighbourhood of the product when
    `U  in  ?_i`. -/
theorem iprod_mem_slice {i : ?} {U : Set (a i)} (hU : (D i).mem U) :
    (iprod D).mem (slice D i U) := by
  refine <updTuple D i U, fun j => ?_, <[i], fun j hj => ?_>, rfl>
  * by_cases h : j = i
    * subst h; rw [updTuple_apply_self]; exact hU
    * rw [updTuple_apply_ne U h]; exact (D j).master_mem
  * exact updTuple_apply_ne U fun h => hj (List.mem_singleton.mpr h)

end Iso

end IndexedProduct

section IndexedProduct

variable [Nonempty ?]

section Iso

variable [DecidableEq ?]

omit [Nonempty ?] in
/-- Slices at the same coordinate intersect by intersecting their data. -/
theorem slice_inter (i : ?) (U U' : Set (a i)) :
    slice D i U  I  slice D i U' = slice D i (U  I  U') := by
  rw [slice_eq, slice_eq, slice_eq, iprodNbhd_inter]
  congr 1; funext j
  by_cases h : j = i
  * subst h; rw [updTuple_apply_self, updTuple_apply_self, updTuple_apply_self]
  * rw [updTuple_apply_ne U h, updTuple_apply_ne U' h, updTuple_apply_ne (U  I  U') h,
      Set.inter_self]

end Iso

end IndexedProduct

section IndexedProduct

variable [Nonempty ?]
```

```
section Iso

variable [DecidableEq ?]

omit [Nonempty ?] in
/-- Slices are monotone in their data. -/
theorem slice_subset (i : ?) {U U' : Set (a i)} (hUU' : U  subseteq  U') : slice D i U
      subseteq  slice D i U' := by
  rw [slice_eq, slice_eq, iprodNbhd_subset_iff]
  intro j
  by_cases h : j = i
  * subst h; rw [updTuple_apply_self, updTuple_apply_self]; exact hUU'
  * rw [updTuple_apply_ne U h, updTuple_apply_ne U' h]

end Iso

end IndexedProduct

section IndexedProduct

variable [Nonempty ?]

section Iso

variable [DecidableEq ?]

/-- **The `i`-th component of a product element** (Scott's `z_i`). -/
def proj (z : (iprod D).Element) (i : ?) : (D i).Element where
  mem U := (D i).mem U  /\  z.mem (slice D i U)
  sub h := h.1
  master_mem := <(D i).master_mem, by
    have : slice D i (D i).master = (iprod D).master := by
      show iprodNbhd (updTuple D i (D i).master) = iprodNbhd (fun j => (D j).master)
      congr 1; funext j
      by_cases h : j = i
      * subst h; rw [updTuple_apply_self]
      * rw [updTuple_apply_ne (D i).master h]
    rw [this]; exact z.master_mem>
  inter_mem := by
    rintro U U' <_, hzU> <_, hzU'>
    have hz := z.inter_mem hzU hzU'
    rw [slice_inter] at hz
    exact <iprod_mem_slice_inv (z.sub hz), hz>
  up_mem := by
    rintro U U' <_, hzU> hU' hUU'
    exact <hU', z.up_mem hzU (iprod_mem_slice hU') (slice_subset i hUU')>

end Iso

end IndexedProduct

section IndexedProduct

variable [Nonempty ?]

section Iso
```

```
variable [DecidableEq ?]

theorem proj_fromPi (x : forall  i, (D i).Element) (i : ?) : proj (fromPi D x) i = x i
     := by
  apply Element.ext
  intro U
  constructor
  * rintro <_, hz>; exact (fromPi_mem_slice x i U).mp hz
  * intro hU; exact <(x i).sub hU, (fromPi_mem_slice x i U).mpr hU>

end Iso

end IndexedProduct

section IndexedProduct

variable [Nonempty ?]

section Iso

variable [DecidableEq ?]

/-- An element contains the restricted cylinder once it contains each listed slice. -/
theorem z_mem_iprodNbhd_restrictTo (z : (iprod D).Element) {X : forall  i, Set (a i)}
    (hXsub : forall  i, X i  subseteq  (D i).master) (hslice : forall  i, z.mem (slice
     D i (X i))) (l : List ?) :
    z.mem (iprodNbhd (restrictTo D l X)) := by
  induction l with
  | nil =>
    have : restrictTo D [] X = fun j => (D j).master := by
      funext j; rw [restrictTo]; exact ite_eq_right List.not_mem_nil
    rw [this]; exact z.master_mem
  | cons a l ih =>
    rw [iprodNbhd_restrictTo_cons hXsub a l]
    exact z.inter_mem (hslice a) ih

end Iso

end IndexedProduct

section IndexedProduct

variable [Nonempty ?]

section Iso

variable [DecidableEq ?]

/-- **Reconstruction.** A product element containing each of a cylinder's slices
    contains the
cylinder. The finite support (a *list* of coordinates) lets the only finitely many non
    -trivial slices
be intersected inside the element -- entirely choice-free. -/
theorem z_mem_of_slices (z : (iprod D).Element) {X : forall  i, Set (a i)}
    (hXmem : forall  i, (D i).mem (X i)) (hXf : FinSupp D X)
```

```
    (hslice : forall  i, z.mem (slice D i (X i))) : z.mem (iprodNbhd X) := by
  obtain <l, hl> := hXf
  have hsub : forall  i, X i  subseteq  (D i).master := fun i => (D i).sub_master (
    hXmem i)
  have key := z_mem_iprodNbhd_restrictTo z hsub hslice l
  have heq : restrictTo D l X = X := by
    funext j
    rw [restrictTo]
    by_cases h : j  in  l
    * rw [ite_eq_left h]
    * rw [ite_eq_right h]; exact (hl j h).symm
  rwa [heq] at key

end Iso

end IndexedProduct

section IndexedProduct

variable [Nonempty ?]

section Iso

variable [DecidableEq ?]

theorem fromPi_toPi (z : (iprod D).Element) : fromPi D (fun i => proj z i) = z := by
  apply Element.ext
  intro W
  constructor
  * rintro <X, hX, hXf, rfl>
    exact z_mem_of_slices z (fun i => (hX i).1) hXf (fun i => (hX i).2)
  * intro hW
    obtain <X, hXmem, hXf, rfl> := z.sub hW
    exact <X, fun i => <hXmem i, z.up_mem hW (iprod_mem_slice (hXmem i))
      (iprodNbhd_subset_slice hXmem i)>, hXf, rfl>

end Iso

end IndexedProduct

section IndexedProduct

variable [Nonempty ?]

section Iso

variable [DecidableEq ?]

/-- **Exercise 6.29 -- ` x ` generalizes to `?`, correct up to isomorphism.** The
    domain of the
indexed product is the pointwise product of the factor domains: `|?_i D_i| ?o forall  i
    , |D_i|`. This is
the infinitary **Proposition 3.2** (`prodEquiv`). -/
def iprodEquiv (D : forall  i, NeighborhoodSystem (a i)) :
    (iprod D).Element ?o (forall  i, (D i).Element) where
  toFun z := fun i => proj z i
```

```
  invFun x := fromPi D x
  left_inv z := fromPi_toPi z
  right_inv x := funext fun i => proj_fromPi x i
  map_rel_iff' := by
    intro z z'
    constructor
    * intro h W hzW
      obtain <X, hXmem, hXf, rfl> := z.sub hzW
      refine z_mem_of_slices z' hXmem hXf fun i => ?_
      have hzi : (proj z i).mem (X i) :=
        <hXmem i, z.up_mem hzW (iprod_mem_slice (hXmem i)) (iprodNbhd_subset_slice
    hXmem i)>
      exact ((h i) (X i) hzi).2
    * intro h i U hU
      exact <hU.1, h _ hU.2>

end Iso

end IndexedProduct

theorem injI_inter_ne {i j : ?} (h : i  !=  j) (X : Set (a i)) (Y : Set (a j)) :
    injI i X  I  injI j Y = {} := by
  apply Set.eq_empty_iff_forall_notMem.mpr
  rintro (_ | <k, a>) <hw1, hw2>
  * exact none_not_mem_injI hw1
  * exact h ((index_of_some_mem_injI hw1).symm.trans (index_of_some_mem_injI hw2))

theorem injI_nonempty {i : ?} {X : Set (a i)} (hX : X.Nonempty) : (injI i X).Nonempty
    := by
  obtain <a, ha> := hX; exact <some <i, a>, some_mem_injI.mpr ha>

/-- **Exercise 6.29 -- `?` does not generalize.** Over an infinite index, the smash
    product has only
its basepoint: an all-coordinates-proper cylinder has support `Set.univ`, which cannot
    be finite. -/
theorem iotimes_only_master [Infinite ?] (D : forall  i, NeighborhoodSystem (a i)) :
    forall  W, (iotimes D).mem W  ->  W = (iotimes D).master := by
  rintro W (rfl | <X, _, hXne, <l, hl>, rfl>)
  * rfl
  * -- Every coordinate is proper, so the finite support list `l` would have to contain
     *all* of `?`,
    -- impossible for an infinite index. This is a cardinality argument: Prop-level
    and unavoidably
    -- classical (Mathlib's `Set.Finite`/`Fintype` API is built on `Classical.choice`).
    refine absurd ((List.finite_toSet l).subset (fun i _ => ?_)) Set.infinite_univ
    exact not_not.mp fun hni => hXne i (hl i hni)

/-- **Exercise 6.29 -- the infinite smash collapses to one point.** Its element domain
     is a
singleton (`False` only), so `?` has no sensible infinitary generalization. -/
theorem iotimes_subsingleton [Infinite ?] (D : forall  i, NeighborhoodSystem (a i)) :
    Subsingleton (iotimes D).Element := by
  refine <fun x y => ?_>
  apply Element.ext
  intro W
  constructor
```

```
  * intro hx; rw [iotimes_only_master D W (x.sub hx)]; exact y.master_mem
  * intro hy; rw [iotimes_only_master D W (y.sub hy)]; exact x.master_mem

section IndexedProduct

variable [Nonempty ?]

theorem iprod_mem_iprodNbhd {X : forall  i, Set (a i)} (hX : forall  i, (D i).mem (X i
    ))
    (hXf : FinSupp D X) : (iprod D).mem (iprodNbhd X) :=
  <X, hX, hXf, rfl>

end IndexedProduct

/-- **Exercise 6.29 -- a sum element reaches at most one summand.** -/
theorem isum_summand_unique {hne : forall  i, forall  X : Set (a i), (D i).mem X  ->
    X.Nonempty}
    (z : (isum D hne).Element) {i j : ?} {X : Set (a i)} {Y : Set (a j)}
    (hX : z.mem (injI i X)) (hY : z.mem (injI j Y)) : i = j := by
  -- The filter element `z` contains the consistent intersection; its representative
    carries a
  -- token whose index is forced to be both `i` and `j` -- no excluded middle on `i = j
    `.
  have hz := z.inter_mem hX hY
  rcases z.sub hz with h0 | <k, W, hWmem, hWeq>
  * rw [Set.ext_iff] at h0
    exact absurd ((h0 none).mpr none_mem_sumMasterI).1 none_not_mem_injI
  * obtain <w, hw> := hne k W hWmem
    rw [Set.ext_iff] at hWeq
    have hmem := (hWeq (some <k, w>)).mpr (some_mem_injI.mpr hw)
    obtain rfl := index_of_some_mem_injI hmem.1
    obtain rfl := index_of_some_mem_injI hmem.2
    rfl

/-- **Exercise 6.29 -- sum trichotomy.** Every element of `?_i D_i` either reaches into
     some summand
or is the basepoint `False`. (Prop-level and genuinely classical: the case split is
    excluded middle on
whether `z` reaches a summand, so this depends on `Classical.choice`.) -/
theorem isum_trichotomy {hne : forall  i, forall  X : Set (a i), (D i).mem X  ->  X.
    Nonempty}
    (z : (isum D hne).Element) :
    (exists  i, exists  X : Set (a i), (D i).mem X  /\  z.mem (injI i X))  \/
      (forall  W, z.mem W  ->  W = (isum D hne).master) := by
  by_cases h : exists  i, exists  X : Set (a i), (D i).mem X  /\  z.mem (injI i X)
  * exact Or.inl h
  * refine Or.inr fun W hW => ?_
    rcases z.sub hW with rfl | <i, X, hX, rfl>
    * rfl
    * exact absurd <i, X, hX, hW> h

end Exercise629
end

end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.Exercise629
```

```
section
set_option linter.unusedVariables false

theorem example_629.{u, v} : And
  (And
    (And
      (forall  {? : Type u} {a : ?  ->  Type v} [inst : Infinite ?]
        (D : (i : ?)  ->  DomainTheory.NeighbourhoodSystems.NeighborhoodSystem (a i))
        (W : Set (@Sigma ? fun (i : ?) => a i)),
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem (@Sigma ? fun (i :
    ?) => a i)
            (@DomainTheory.NeighbourhoodSystems.Exercise629.iotimes ? a
              (@Nontrivial.to_nonempty ? (@Infinite.instNontrivial ? inst)) D)
            W  ->
          @Eq (Set (@Sigma ? fun (i : ?) => a i)) W
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.master (@Sigma ?
    fun (i : ?) => a i)
              (@DomainTheory.NeighbourhoodSystems.Exercise629.iotimes ? a
                (@Nontrivial.to_nonempty ? (@Infinite.instNontrivial ? inst)) D)))
      (forall  {? : Type u} {a : ?  ->  Type v} [inst : Infinite ?]
        (D : (i : ?)  ->  DomainTheory.NeighbourhoodSystems.NeighborhoodSystem (a i)),
        Subsingleton
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (@Sigma ? fun
     (i : ?) => a i)
            (@DomainTheory.NeighbourhoodSystems.Exercise629.iotimes ? a
              (@Nontrivial.to_nonempty ? (@Infinite.instNontrivial ? inst)) D))))
    (forall  {? : Type u} {a : ?  ->  Type v} {D : (i : ?)  ->  DomainTheory.
    NeighbourhoodSystems.NeighborhoodSystem (a i)}
      {hne :
        forall  (i : ?) (X : Set (a i)),
          @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem (a i) (D i) X  ->
     @Set.Nonempty (a i) X}
      (z :
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Option (@Sigma
    ? fun (i : ?) => a i))
          (@DomainTheory.NeighbourhoodSystems.Exercise629.isum ? a D hne))
      {i j : ?} {X : Set (a i)} {Y : Set (a j)},
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem (Option (
    @Sigma ? fun (i : ?) => a i))
          (@DomainTheory.NeighbourhoodSystems.Exercise629.isum ? a D hne) z
          (@DomainTheory.NeighbourhoodSystems.Exercise629.injI ? a i X)  ->
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem (Option (
    @Sigma ? fun (i : ?) => a i))
            (@DomainTheory.NeighbourhoodSystems.Exercise629.isum ? a D hne) z
            (@DomainTheory.NeighbourhoodSystems.Exercise629.injI ? a j Y)  ->
          @Eq ? i j))
  (forall  {? : Type u} {a : ?  ->  Type v} {D : (i : ?)  ->  DomainTheory.
    NeighbourhoodSystems.NeighborhoodSystem (a i)}
    {hne :
      forall  (i : ?) (X : Set (a i)),
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem (a i) (D i) X  ->
    @Set.Nonempty (a i) X}
    (z :
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Option (@Sigma ?
    fun (i : ?) => a i))
        (@DomainTheory.NeighbourhoodSystems.Exercise629.isum ? a D hne)),
```

```
  Or
    (exists  (i : ?) (X : Set (a i)),
      And (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem (a i) (D i) X)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem (Option (
  @Sigma ? fun (i : ?) => a i))
          (@DomainTheory.NeighbourhoodSystems.Exercise629.isum ? a D hne) z
          (@DomainTheory.NeighbourhoodSystems.Exercise629.injI ? a i X)))
    (forall  (W : Set (Option (@Sigma ? fun (i : ?) => a i))),
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem (Option (
  @Sigma ? fun (i : ?) => a i))
          (@DomainTheory.NeighbourhoodSystems.Exercise629.isum ? a D hne) z W  ->
        @Eq (Set (Option (@Sigma ? fun (i : ?) => a i))) W
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.master (Option (
  @Sigma ? fun (i : ?) => a i))
            (@DomainTheory.NeighbourhoodSystems.Exercise629.isum ? a D hne)))) :=
@And.intro
(And
  (And
    (forall  {? : Type u} {a : ?  ->  Type v} [inst : Infinite ?]
      (D : (i : ?)  ->  DomainTheory.NeighbourhoodSystems.NeighborhoodSystem (a i))
      (W : Set (@Sigma ? fun (i : ?) => a i)),
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem (@Sigma ? fun (i :
  ?) => a i)
          (@DomainTheory.NeighbourhoodSystems.Exercise629.iotimes ? a
            (@Nontrivial.to_nonempty ? (@Infinite.instNontrivial ? inst)) D)
          W  ->


        @Eq (Set (@Sigma ? fun (i : ?) => a i)) W
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.master (@Sigma ?
  fun (i : ?) => a i)
            (@DomainTheory.NeighbourhoodSystems.Exercise629.iotimes ? a
              (@Nontrivial.to_nonempty ? (@Infinite.instNontrivial ? inst)) D)))
    (forall  {? : Type u} {a : ?  ->  Type v} [inst : Infinite ?]
      (D : (i : ?)  ->  DomainTheory.NeighbourhoodSystems.NeighborhoodSystem (a i)),
      Subsingleton
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (@Sigma ? fun
   (i : ?) => a i)
          (@DomainTheory.NeighbourhoodSystems.Exercise629.iotimes ? a
            (@Nontrivial.to_nonempty ? (@Infinite.instNontrivial ? inst)) D))))
  (forall  {? : Type u} {a : ?  ->  Type v} {D : (i : ?)  ->  DomainTheory.
  NeighbourhoodSystems.NeighborhoodSystem (a i)}
    {hne :
      forall  (i : ?) (X : Set (a i)),
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem (a i) (D i) X  ->
   @Set.Nonempty (a i) X}
    (z :
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Option (@Sigma
  ? fun (i : ?) => a i))
        (@DomainTheory.NeighbourhoodSystems.Exercise629.isum ? a D hne))
    {i j : ?} {X : Set (a i)} {Y : Set (a j)},
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem (Option (
  @Sigma ? fun (i : ?) => a i))
        (@DomainTheory.NeighbourhoodSystems.Exercise629.isum ? a D hne) z
        (@DomainTheory.NeighbourhoodSystems.Exercise629.injI ? a i X)  ->
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem (Option (
  @Sigma ? fun (i : ?) => a i))
```

```
          (@DomainTheory.NeighbourhoodSystems.Exercise629.isum ? a D hne) z
          (@DomainTheory.NeighbourhoodSystems.Exercise629.injI ? a j Y)  ->
        @Eq ? i j))
(forall  {? : Type u} {a : ?  ->  Type v} {D : (i : ?)  ->  DomainTheory.
  NeighbourhoodSystems.NeighborhoodSystem (a i)}
  {hne :
    forall  (i : ?) (X : Set (a i)),
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem (a i) (D i) X  ->
  @Set.Nonempty (a i) X}
  (z :
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Option (@Sigma ?
  fun (i : ?) => a i))
      (@DomainTheory.NeighbourhoodSystems.Exercise629.isum ? a D hne)),
  Or
    (exists  (i : ?) (X : Set (a i)),
      And (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem (a i) (D i) X)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem (Option (
  @Sigma ? fun (i : ?) => a i))
          (@DomainTheory.NeighbourhoodSystems.Exercise629.isum ? a D hne) z
          (@DomainTheory.NeighbourhoodSystems.Exercise629.injI ? a i X)))
    (forall  (W : Set (Option (@Sigma ? fun (i : ?) => a i))),
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem (Option (
  @Sigma ? fun (i : ?) => a i))
          (@DomainTheory.NeighbourhoodSystems.Exercise629.isum ? a D hne) z W  ->
        @Eq (Set (Option (@Sigma ? fun (i : ?) => a i))) W
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.master (Option (
  @Sigma ? fun (i : ?) => a i))
            (@DomainTheory.NeighbourhoodSystems.Exercise629.isum ? a D hne))))
(@And.intro
  (And
    (forall  {? : Type u} {a : ?  ->  Type v} [inst : Infinite ?]
      (D : (i : ?)  ->  DomainTheory.NeighbourhoodSystems.NeighborhoodSystem (a i))
      (W : Set (@Sigma ? fun (i : ?) => a i)),
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem (@Sigma ? fun (i :
  ?) => a i)
          (@DomainTheory.NeighbourhoodSystems.Exercise629.iotimes ? a
            (@Nontrivial.to_nonempty ? (@Infinite.instNontrivial ? inst)) D)
          W  ->
        @Eq (Set (@Sigma ? fun (i : ?) => a i)) W
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.master (@Sigma ?
  fun (i : ?) => a i)
            (@DomainTheory.NeighbourhoodSystems.Exercise629.iotimes ? a
              (@Nontrivial.to_nonempty ? (@Infinite.instNontrivial ? inst)) D)))
    (forall  {? : Type u} {a : ?  ->  Type v} [inst : Infinite ?]
      (D : (i : ?)  ->  DomainTheory.NeighbourhoodSystems.NeighborhoodSystem (a i)),
      Subsingleton
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (@Sigma ? fun
   (i : ?) => a i)
          (@DomainTheory.NeighbourhoodSystems.Exercise629.iotimes ? a
            (@Nontrivial.to_nonempty ? (@Infinite.instNontrivial ? inst)) D))))
  (forall  {? : Type u} {a : ?  ->  Type v} {D : (i : ?)  ->  DomainTheory.
  NeighbourhoodSystems.NeighborhoodSystem (a i)}
    {hne :
      forall  (i : ?) (X : Set (a i)),
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem (a i) (D i) X  ->
   @Set.Nonempty (a i) X}
```

```
      (z :
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Option (@Sigma
    ? fun (i : ?) => a i))
          (@DomainTheory.NeighbourhoodSystems.Exercise629.isum ? a D hne))
      {i j : ?} {X : Set (a i)} {Y : Set (a j)},
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem (Option (
    @Sigma ? fun (i : ?) => a i))
          (@DomainTheory.NeighbourhoodSystems.Exercise629.isum ? a D hne) z
          (@DomainTheory.NeighbourhoodSystems.Exercise629.injI ? a i X)  ->
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem (Option (
    @Sigma ? fun (i : ?) => a i))
            (@DomainTheory.NeighbourhoodSystems.Exercise629.isum ? a D hne) z
            (@DomainTheory.NeighbourhoodSystems.Exercise629.injI ? a j Y)  ->
          @Eq ? i j)
     (@And.intro
       (forall  {? : Type u} {a : ?  ->  Type v} [inst : Infinite ?]
         (D : (i : ?)  ->  DomainTheory.NeighbourhoodSystems.NeighborhoodSystem (a i))
         (W : Set (@Sigma ? fun (i : ?) => a i)),
         @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem (@Sigma ? fun (i :
    ?) => a i)
            (@DomainTheory.NeighbourhoodSystems.Exercise629.iotimes ? a
              (@Nontrivial.to_nonempty ? (@Infinite.instNontrivial ? inst)) D)
            W  ->
          @Eq (Set (@Sigma ? fun (i : ?) => a i)) W
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.master (@Sigma ?
    fun (i : ?) => a i)
              (@DomainTheory.NeighbourhoodSystems.Exercise629.iotimes ? a
                (@Nontrivial.to_nonempty ? (@Infinite.instNontrivial ? inst)) D)))
       (forall  {? : Type u} {a : ?  ->  Type v} [inst : Infinite ?]
         (D : (i : ?)  ->  DomainTheory.NeighbourhoodSystems.NeighborhoodSystem (a i)),
         Subsingleton
           (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (@Sigma ? fun
      (i : ?) => a i)
              (@DomainTheory.NeighbourhoodSystems.Exercise629.iotimes ? a
                (@Nontrivial.to_nonempty ? (@Infinite.instNontrivial ? inst)) D)))
       @DomainTheory.NeighbourhoodSystems.Exercise629.iotimes_only_master
       @DomainTheory.NeighbourhoodSystems.Exercise629.iotimes_subsingleton)
     @DomainTheory.NeighbourhoodSystems.Exercise629.isum_summand_unique)
   @DomainTheory.NeighbourhoodSystems.Exercise629.isum_trichotomy
end

end DomainTheory.NeighbourhoodSystems.Exercise629
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

section
variable {a b : Type*} {D : NeighborhoodSystem a} {E : NeighborhoodSystem b}

/-- **Scott's `?` (the prose before Lemma 6.15).** `D ? E` means `D  ~=  D'` for some
    subdomain
`D' ? E`: `D` *embeds as a subdomain* of `E`. -/
def Trianglelefteq (D : NeighborhoodSystem a) (E : NeighborhoodSystem b) : Prop :=
  exists  D' : NeighborhoodSystem b, D' ? E  /\  (D  ~= ? D')

@[inherit_doc] infix:50 " ? " => Trianglelefteq
```

```
section ProjectionPair

variable (i : ApproximableMap D E) (j : ApproximableMap E D)

/-- The generator predicate: `Y` generates `i(X)`. Relationally, `X i Y` and `Y j X`.
    -/
def IsGen (X : Set a) (Y : Set b) : Prop := i.rel X Y  /\  j.rel Y X

/-- **Generators exist** (uses `j  comp  i = I_D`). Every `D`-neighbourhood `X` has a
    generator: apply
`j  comp  i = I_D` to the identity relation `X I_D X`. -/
theorem isGen_exists (hji : j.comp i = idMap D) {X : Set a} (hX : D.mem X) :
    exists  Y, IsGen i j X Y := by
  have hrel : (j.comp i).rel X X := by rw [hji]; exact <hX, hX, subset_rfl>
  obtain <Y, hiXY, hjYX> := hrel
  exact <Y, hiXY, hjYX>

/-- **Generators are closed under intersection.** If `Y, Y'` generate `X, X'` and `Y  I
      Y'  in  E`, then
`Y  I  Y'` generates `X  I  X'`. Needs only `mono`/`inter_right` of `i` and `j` (the
    hypothesis
`E.mem (Y  I  Y')` is what licenses the `j.mono` steps). -/
theorem isGen_inter {X X' : Set a} {Y Y' : Set b}
    (h : IsGen i j X Y) (h' : IsGen i j X' Y') (hE : E.mem (Y  I  Y')) :
    IsGen i j (X  I  X') (Y  I  Y') := by
  obtain <hiXY, hjYX> := h
  obtain <hiX'Y', hjY'X'> := h'
  have hj1 : j.rel (Y  I  Y') X :=
    j.mono hjYX Set.inter_subset_left subset_rfl hE (j.rel_cod hjYX)
  have hj2 : j.rel (Y  I  Y') X' :=
    j.mono hjY'X' Set.inter_subset_right subset_rfl hE (j.rel_cod hjY'X')
  have hjInter : j.rel (Y  I  Y') (X  I  X') := j.inter_right hj1 hj2
  have hDXX' : D.mem (X  I  X') := j.rel_cod hjInter
  have hi1 : i.rel (X  I  X') Y :=
    i.mono hiXY Set.inter_subset_left subset_rfl hDXX' (i.rel_cod hiXY)
  have hi2 : i.rel (X  I  X') Y' :=
    i.mono hiX'Y' Set.inter_subset_right subset_rfl hDXX' (i.rel_cod hiX'Y')
  exact <i.inter_right hi1 hi2, hjInter>

/-- The masters generate each other: `IsGen Delta_D Delta_E` (from `i.master_rel`, `j.
    master_rel`). -/
theorem isGen_master : IsGen i j D.master E.master :=
  <i.master_rel, j.master_rel>

/-- **The image subdomain `D'`.** A `b`-set `Y` is a neighbourhood iff it generates
    some
`D`-neighbourhood; the master is `E`'s master. `isGen_inter` supplies condition (ii).
    -/
def Dprime : NeighborhoodSystem b where
  mem Y := exists  X, IsGen i j X Y
  master := E.master
  master_nonempty := E.master_nonempty
  master_mem := <D.master, isGen_master i j>
  inter_mem := by
    rintro Y_1 Y_2 Z <X_1, hg_1> <X_2, hg_2> <_, hgz> hZsub
    have hEY_1 : E.mem Y_1 := i.rel_cod hg_1.1
```

```
    have hEY_2 : E.mem Y_2 := i.rel_cod hg_2.1
    have hEZ : E.mem Z := i.rel_cod hgz.1
    have hEinter : E.mem (Y_1  I  Y_2) := E.inter_mem hEY_1 hEY_2 hEZ hZsub
    exact <X_1  I  X_2, isGen_inter i j hg_1 hg_2 hEinter>
  sub_master := by
    rintro Y <X, hg>
    exact E.sub_master (i.rel_cod hg.1)

/-- **The generator correspondence is monotone** (uses `j  comp  i = I_D`): if `Y, Y'`
    generate `X, X'`
and `Y  subseteq  Y'`, then `X  subseteq  X'`. (Widen `X i Y` to `X i Y'` by `mono`,
    compose with `Y' j X'`, and read
off `X  subseteq  X'` from `j  comp  i = I_D`.) -/
theorem isGen_mono (hji : j.comp i = idMap D) {X X' : Set a} {Z W : Set b}
    (h : IsGen i j X Z) (h' : IsGen i j X' W) (hZW : Z  subseteq  W) : X  subseteq  X'
     := by
  obtain <hiXZ, _> := h
  obtain <_, hjWX'> := h'
  have hiXW : i.rel X W :=
    i.mono hiXZ subset_rfl hZW (i.rel_dom hiXZ) (j.rel_dom hjWX')
  have hrel : (j.comp i).rel X X' := <W, hiXW, hjWX'>
  rw [hji] at hrel
  exact hrel.2.2

/-- Generators are unique in the first argument (`isGen_mono` both ways). -/
theorem isGen_fst_unique (hji : j.comp i = idMap D) {X X' : Set a} {Y : Set b}
    (h : IsGen i j X Y) (h' : IsGen i j X' Y) : X = X' :=
  Set.Subset.antisymm (isGen_mono i j hji h h' subset_rfl)
    (isGen_mono i j hji h' h subset_rfl)

/-- **The generator correspondence is monotone, other direction** (uses `i  comp  j
    subseteq  I_E`): if `Z, W`
generate `X, X'` and `X  subseteq  X'`, then `Z  subseteq  W`. (Widen `Z j X` to `Z j X
    '` by `mono`, compose with
`X' i W`, and read off `Z  subseteq  W` from `i  comp  j  subseteq  I_E`.) -/
theorem isGen_mono' (hij : i.comp j  <=  idMap E) {X X' : Set a} {Z W : Set b}
    (h : IsGen i j X Z) (h' : IsGen i j X' W) (hXX' : X  subseteq  X') : Z  subseteq
    W := by
  obtain <_, hjZX> := h
  obtain <hiX'W, _> := h'
  have hjZX' : j.rel Z X' :=
    j.mono hjZX subset_rfl hXX' (j.rel_dom hjZX) (i.rel_dom hiX'W)
  have hrel : (i.comp j).rel Z W := <X', hjZX', hiX'W>
  exact (hij Z W hrel).2.2

/-- Generators are unique in the second argument (`isGen_mono'` both ways). -/
theorem isGen_snd_unique (hij : i.comp j  <=  idMap E) {X : Set a} {Y Y' : Set b}
    (h : IsGen i j X Y) (h' : IsGen i j X Y') : Y = Y' :=
  Set.Subset.antisymm (isGen_mono' i j hij h h' subset_rfl)
    (isGen_mono' i j hij h' h subset_rfl)

@[simp] theorem mem_Dprime {Y : Set b} : (Dprime i j).mem Y  <->  exists  X, IsGen i j
     X Y := Iff.rfl

/-- **`D' ? E`.** Same master (`rfl`); `D'  subseteq  E` since a generator's `Y` is an `
    E`-neighbourhood; and
```

```
the consistency clause `inter_closed` is exactly `isGen_inter`. -/
theorem Dprime_subsystem : Dprime i j ? E where
  master_eq := rfl
  sub := by rintro Y <X, hg>; exact i.rel_cod hg.1
  inter_closed := by
    rintro Y_1 Y_2 <X_1, hg_1> <X_2, hg_2> hE
    exact <X_1  I  X_2, isGen_inter i j hg_1 hg_2 hE>

/-- **Inverse map of the isomorphism `D  ~=  D'`.** `y  |->  {X ? exists  Y  in  y,
    IsGen i j X Y}`. (Needs both
laws: `j  comp  i = I_D` for generator existence and `i  comp  j  subseteq  I_E` for `
    isGen_mono'`.) -/
def ofEl (hji : j.comp i = idMap D) (hij : i.comp j  <=  idMap E)
    (y : (Dprime i j).Element) : D.Element where
  mem X := exists  Y, y.mem Y  /\  IsGen i j X Y
  sub := by rintro X <Y, _, hg>; exact i.rel_dom hg.1
  master_mem := <E.master, y.master_mem, isGen_master i j>
  inter_mem := by
    rintro X_1 X_2 <Y_1, hY_1y, hg_1> <Y_2, hY_2y, hg_2>
    have hyInter : y.mem (Y_1  I  Y_2) := y.inter_mem hY_1y hY_2y
    have hEInter : E.mem (Y_1  I  Y_2) := (Dprime_subsystem i j).sub (y.sub hyInter)
    exact <Y_1  I  Y_2, hyInter, isGen_inter i j hg_1 hg_2 hEInter>
  up_mem := by
    rintro X X' <Y, hYy, hg> hDX' hXX'
    obtain <Y', hg'> := isGen_exists i j hji hDX'
    have hYY' : Y  subseteq  Y' := isGen_mono' i j hij hg hg' hXX'
    exact <Y', y.up_mem hYy <X', hg'> hYY', hg'>

/-- **Forward map of the isomorphism `D  ~=  D'`.** `x  |->  {Y ? exists  X  in  x,
    IsGen i j X Y}` -- the
generators of the members of `x`. (Needs `j  comp  i = I_D` for upward closure, via `
    isGen_mono`.) -/
def toEl (hji : j.comp i = idMap D) (x : D.Element) : (Dprime i j).Element where
  mem Y := exists  X, x.mem X  /\  IsGen i j X Y
  sub := by rintro Y <X, _, hg>; exact <X, hg>
  master_mem := <D.master, x.master_mem, isGen_master i j>
  inter_mem := by
    rintro Y_1 Y_2 <X_1, hX_1x, hg_1> <X_2, hX_2x, hg_2>
    have hxInter : x.mem (X_1  I  X_2) := x.inter_mem hX_1x hX_2x
    have hDInter : D.mem (X_1  I  X_2) := x.sub hxInter
    have hi1 : i.rel (X_1  I  X_2) Y_1 :=
      i.mono hg_1.1 Set.inter_subset_left subset_rfl hDInter (i.rel_cod hg_1.1)
    have hi2 : i.rel (X_1  I  X_2) Y_2 :=
      i.mono hg_2.1 Set.inter_subset_right subset_rfl hDInter (i.rel_cod hg_2.1)
    have hEinter : E.mem (Y_1  I  Y_2) := i.rel_cod (i.inter_right hi1 hi2)
    exact <X_1  I  X_2, hxInter, isGen_inter i j hg_1 hg_2 hEinter>
  up_mem := by
    rintro Y Y' <X, hXx, hg> <X', hg'> hYY'
    have hXX' : X  subseteq  X' := isGen_mono i j hji hg hg' hYY'
    exact <X', x.up_mem hXx (i.rel_dom hg'.1) hXX', hg'>

/-- **The domain isomorphism `D  ~=  D'`** (Scott's "inclusion-preserving one-one
    correspondence").
Built from `toEl`/`ofEl`; the inverse laws and order-reflection come from generator
    uniqueness. -/
def dprimeEquiv (hji : j.comp i = idMap D) (hij : i.comp j  <=  idMap E) :
```

```
    D.Element ?o (Dprime i j).Element where
  toFun := toEl i j hji
  invFun := ofEl i j hji hij
  left_inv := by
    intro x
    apply Element.ext
    intro X
    constructor
    * rintro <Y, <X_1, hX_1x, hg_1>, hg>
      rw [isGen_fst_unique i j hji hg hg_1]; exact hX_1x
    * intro hXx
      obtain <Y, hg> := isGen_exists i j hji (x.sub hXx)
      exact <Y, <X, hXx, hg>, hg>
  right_inv := by
    intro y
    apply Element.ext
    intro Y
    constructor
    * rintro <X, <Y_1, hY_1y, hg_1>, hg>
      rw [isGen_snd_unique i j hij hg hg_1]; exact hY_1y
    * intro hYy
      obtain <X, hg> := y.sub hYy
      exact <X, <Y, hYy, hg>, hg>
  map_rel_iff' := by
    intro x x'
    constructor
    * intro h X hXx
      obtain <Y, hg> := isGen_exists i j hji (x.sub hXx)
      obtain <X_1, hX_1x', hg_1> := h Y <X, hXx, hg>
      rw [isGen_fst_unique i j hji hg hg_1]; exact hX_1x'
    * rintro h Y <X, hXx, hg>
      exact <X, h X hXx, hg>

end ProjectionPair

end
end DomainTheory.NeighbourhoodSystems
end
```

3.8 Computability in effectively given domains

Dependency diagram. Each box is one item. The arrowhead is on the item that imports. An arrow already implied by a longer path is left out. The introduction has no Lean file, so it is not shown. Blue is a definition, green a proposition, amber an example, red a theorem, and indigo an exercise.

3.8.1 Introduction

LECTURE VII COMPUTABILITY IN EFFECTIVELY GIVEN DOMAINS

For the domain N the strict functions from N into N, the strict maps $f : N \to N$, correspond exactly to the partial functions $g : N \to N$ (as we wrote in 5.6 we had $f = \bar{g}$). For such functions there is a standard theory of computability: g is called computable if it can be defined as a partial recursive function with its "program" written down in a certain standard form. The non-strict maps $h : N \to N$ are all constant, and

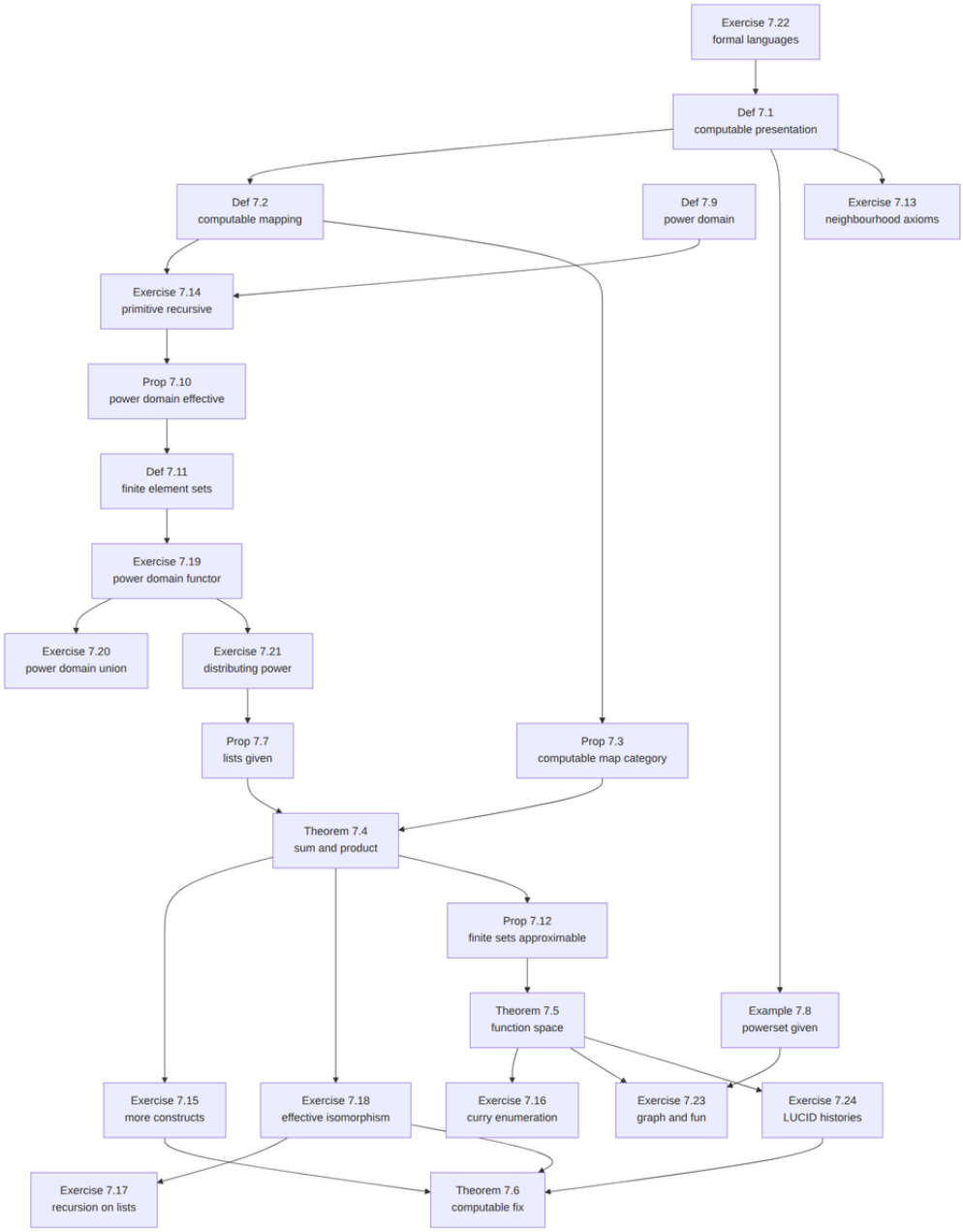


Figure 36: Computability in effectively given domains.

so are intuitively computable; so we know all about computable maps in $|N \to N|$ in general. The question is: what are the computable maps on (elements of) other domains?

The answer will of course depend on how the domain is presented to us. Even with N, there are continuum many isomorphisms $\pi : N \to N$ of N onto itself, not all of which can be computable. That is, if we permute N and, so to speak, present the integers in a different order, then a well-behaved computable function $f : N \to N$ may well be transformed into a non-computable function,

$$\pi \circ f \circ \pi^{-1} : N \to N.$$

(Hint: Consider the characteristic function e of the even numbers. Take $f = \bar{e}$ and let π be very horrid.) The reason we imagined we knew which were the computable $f : N \to N$ is that N is always thought of in a standard presentation. We must thus define "in general" a concept of an *effectively given domain*, that is to say, one with a sufficiently computable presentation to represent the additional knowledge about the domain.

The main idea will be that the *finite elements* of $|D|$ should be regarded as the ones initially known. Abstractly, to know a finite element is to know how it is *related* to other finite elements.

Of course, this will mean that we will allow at most a countable infinity of finite elements — but this restriction well accords with intuition. To make precise the terminology "related to" it proves most convenient to go back to the neighbourhoods (in any case they are in a one-one correspondence with the finite elements).

3.8.2 Definition 7.1 — computable presentation

DEFINITION 7.1. A neighbourhood system D has a *computable presentation* provided we can write

$$D = \{X_n \mid n \in N\},$$

where the following two relations

(i) $X_n \cap X_m = X_k$; and

(ii) $\exists k \in N.\, X_k \subseteq X_n$ and $X_k \subseteq X_m$

are recursively decidable (in integer indices n, m, k and in n, m, respectively). □

More strictly the sequence,

$$\langle X_n \rangle_{n=0}^{\infty},$$

is the presentation. Even more strictly, when it is required to cope with infinitely many domains at a time, it would be necessary to give the actual Gödel numbers of the recursive relations (i) and (ii) (rather than just saying there exists some way of showing them to be recursively decidable).

The intuitive idea of 7.1 is that the system is effectively given if you know how to do elementary "calculations" with neighbourhoods. The basic calculations are the forming of intersections. The neighbourhoods have to be laid out in a systematic way; and, if we are asked for an intersection of two given neighbourhoods, we have to be able to locate it in the standard sequence. Relation (ii) is the *consistency condition* , which is the necessary and sufficient condition for the intersection to exist in D. When (ii) is true, therefore, we have only to try $k = 0, 1, 2, \ldots$ until we discover that we have found the intersection. We are

assuming that these basic decisions can be carried out in "finite time". Note that the obvious biconditional,

$$X_n \subseteq X_m \text{ iff } X_n \cap X_m = X_n,$$

assures us that the inclusion relation between neighbourhoods is itself decidable in terms of the indices. So in (ii) if k exists, *then* it (or the first one) can indeed be found in finite time. The rub is that if it *does not exist*, no finite number of inclusion checks will determine that fact. That is why we have to *assume* that (ii) is always decidable. The information contained in (ii) is a fundamental part of the neighbourhood structure. (An axiomatic characterization of neighbourhood structures is given in Exercise 7.13, which may make clearer what we are assuming and what a presentation is.)

Definition_7_1.lean

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Exercise_7_22
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Exercise_3_15

public import Mathlib.Computability.Primrec.Basic

public import Mathlib.Data.Nat.Bitwise

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
namespace Exercise722
open scoped Computability
open Sum Set
section
variable {sigma_1 sigma_2 : Type}
variable {sigma : Type}
/- concordance: source L4243 DEFINITION 7.1 computable presentation -/
/-- **Definition 7.1 (ii) for encoded neighbourhoods:** when `denote a`, `denote b  in
     S`,
`consistentB` agrees with membership of the intersection in `S`. -/
theorem consistentB_iff_Ssys (a b : SExpr) (ha : InS (denote a)) (hb : InS (denote b))
     :
    consistentB a b = true  <->  InS (denote a  I  denote b) := by
  rw [ <-  denote_cap, consistentB_iff]
  exact Iff.symm (Ssys_isPositive (Ssys_mem.mpr ha) (Ssys_mem.mpr hb))
end

end Exercise722
end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.Exercise722
section
set_option linter.unusedVariables false

theorem example_722_decide : forall  (a b : DomainTheory.NeighbourhoodSystems.
    Exercise722.SExpr),
  DomainTheory.NeighbourhoodSystems.Exercise722.InS (DomainTheory.NeighbourhoodSystems
    .Exercise722.denote a)  ->
```

```
    DomainTheory.NeighbourhoodSystems.Exercise722.InS (DomainTheory.
    NeighbourhoodSystems.Exercise722.denote b)  ->
      Iff (@Eq Bool (DomainTheory.NeighbourhoodSystems.Exercise722.consistentB a b)
    Bool.true)
         (DomainTheory.NeighbourhoodSystems.Exercise722.InS
           (@Inter.inter (Set (List Bool)) (@Set.instInter (List Bool))
             (DomainTheory.NeighbourhoodSystems.Exercise722.denote a)
             (DomainTheory.NeighbourhoodSystems.Exercise722.denote b))) :=
  DomainTheory.NeighbourhoodSystems.Exercise722.consistentB_iff_Ssys
end

end DomainTheory.NeighbourhoodSystems.Exercise722
namespace DomainTheory.NeighbourhoodSystems
namespace Exercise722
open scoped Computability
open Sum Set

section
variable {sigma_1 sigma_2 : Type}
variable {sigma : Type}

/-- **Definition 7.1 relation (ii) as a reachability problem.** `denote e` is empty iff
     no accept
state of its `Fintype` automaton is reachable. The right-hand side ranges over the
    finite state set
`autState e`; the only non-finite-looking quantifier (`forall  x`) is what a
    reachability search eliminates. -/
theorem denote_eq_empty_iff (e : SExpr) :
    denote e = {}  <->  forall  s  in  (toNFA e).accept, forall  x, s  notin  (toNFA e
    ).eval x := by
  rw [Set.eq_empty_iff_forall_notMem]
  constructor
  * intro h s hs x hse
    exact h x (by rw [ <-  toNFA_accepts e]; exact <s, hs, hse>)
  * intro h x hx
    rw [ <-  toNFA_accepts e] at hx
    obtain <s, hs, hse> := hx
    exact h s hs x hse

/-- A nonempty accepted language ? some accept state is reachable by reading some word
    . -/
theorem nfa_accepts_nonempty_iff (M : NFA Bool sigma) :
    M.accepts.Nonempty  <->  exists  s  in  M.accept, exists  x, s  in  M.eval x := by
  constructor
  * rintro <x, S, hS, hSe>; exact <S, hS, x, hSe>
  * rintro <S, hS, x, hSe>; exact <x, S, hS, hSe>

section ShortWord

variable {sigma : Type} [Fintype sigma] {M : NFA Bool sigma}

theorem nfa_accepts_nonempty_iff_short :
    M.accepts.Nonempty  <->  exists  w, w  in  wordsUpTo (Fintype.card sigma)  /\  w
    in  M.accepts := by
  constructor
  * intro h
```

```
    obtain <w, hwlt, hw> := exists_accepted_word_short h
    refine <w, ?_, hw>
    rw [mem_wordsUpTo]
    omega
  * intro <w, _, hw>
    exact <w, hw>

omit [Fintype sigma] in
theorem pathStateAt_last {s t : sigma} {x : List Bool} (p : M.Path s t x) :
    pathStateAt p x.length (Nat.le_refl _) = t := by
  induction p with
  | nil s => simp [pathStateAt, pathStateAtAux]
  | cons sMid s' u a xs hstep p' ih =>
    simp [pathStateAt, pathStateAtAux, List.length_cons]
    exact ih

end ShortWord

end

section
/-- The accept states of `toNFA e`, as a `Finset`. -/
def acceptFin : (e : SExpr)  ->  Finset (autState e)
  | .sigma => Finset.univ
  | .single sigma => {(some <sigma.length, Nat.lt_succ_self _> : SingleState sigma)}
  | .cap a b => (acceptFin a)  x ? (acceptFin b)
  | .cat a b => (acceptFin b).image (inr : autState b  ->  autState a ? autState b)
end

section
/-- The start states of `toNFA e`, as a `Finset`. The `.cat` case handles the one-hop
?-closure directly (`catEps`'s only ?-edges go from an `M_1`-accept state to `M_2`-
    start states,
`catEps_mem_?Closure_iff`): if some start state of `a` is already accepting, `b`'s
    start states
are also immediately live. -/
def startFin : (e : SExpr)  ->  Finset (autState e)
  | .sigma => Finset.univ
  | .single sigma => {(some <0, Nat.succ_pos _> : SingleState sigma)}
  | .cap a b => (startFin a)  x ? (startFin b)
  | .cat a b =>
      if ((startFin a)  I  (acceptFin a)).Nonempty
      then (startFin a).image (inl : autState a  ->  autState a ? autState b)  U
        (startFin b).image (inr : autState b  ->  autState a ? autState b)
      else (startFin a).image (inl : autState a  ->  autState a ? autState b)
end

section
/-- The step function of `toNFA e` from a single state, as a `Finset`. The `.cat` case'
    s `inl`
branch mirrors `startFin`'s epsilon-closure handling exactly (one hop, on reaching an `
    a`-accept
state after the step). -/
def stepFinSingle : (e : SExpr)  ->  autState e  ->  Bool  ->  Finset (autState e)
  | .sigma => fun _ _ => Finset.univ
  | .single sigma => fun s c => {(singleDFA sigma).step s c}
```

```
  | .cap a b => fun s c => (stepFinSingle a s.1 c)  x ? (stepFinSingle b s.2 c)
  | .cat a b => fun s c =>
      match s with
      | inl s' =>
          if ((stepFinSingle a s' c)  I  (acceptFin a)).Nonempty
          then (stepFinSingle a s' c).image (inl : autState a  ->  autState a ?
    autState b)  U
            (startFin b).image (inr : autState b  ->  autState a ? autState b)
          else (stepFinSingle a s' c).image (inl : autState a  ->  autState a ?
    autState b)
      | inr s' => (stepFinSingle b s' c).image (inr : autState b  ->  autState a ?
    autState b)
end

section
/-- The `Finset`-lifted step (mirrors `NFA.stepSet`). -/
def stepFin (e : SExpr) (S : Finset (autState e)) (c : Bool) : Finset (autState e) :=
  S.biUnion (fun s => stepFinSingle e s c)
end

section
/-- Tracks `(evalFin e_1 w, evalFin e_2 w)` deterministically; accepts `w` iff `e_1` is
     live-accepting
and `e_2` is not -- i.e. `w  in  denote e_1 \ denote e_2`. -/
def diffNFA (e1 e2 : SExpr) : NFA Bool (Finset (autState e1)  x  Finset (autState e2))
     where
  step S c := {(stepFin e1 S.1 c, stepFin e2 S.2 c)}
  start := {(startFin e1, startFin e2)}
  accept := {S | (S.1  I  acceptFin e1).Nonempty  /\  S.2  I  acceptFin e2 = {}}
end

section
/-- The live-state `Finset` after reading `w` (mirrors `NFA.eval`, folding `stepFin`
    left to
right from `startFin`). -/
def evalFin (e : SExpr) (w : List Bool) : Finset (autState e) :=
  w.foldl (stepFin e) (startFin e)
end

section
/-- `denote e_1  subseteq  denote e_2`, decided by a bounded search: if no word up to
    the `diffNFA` state
bound witnesses `e_1 \ e_2`, none exists at all (`exists_diff_word_short`). -/
def subsetB (e1 e2 : SExpr) : Bool :=
  (wordsUpTo (Fintype.card (Finset (autState e1)  x  Finset (autState e2)))).all
    (fun w => !matchesB e1 w || matchesB e2 w)
end

section
/-- **Exercise 7.22k.** Relation (i) on `SExpr` syntax -- language equivalence --
    decided by
`subsetB` in both directions. -/
def interEqB (e1 e2 : SExpr) : Bool := subsetB e1 e2 && subsetB e2 e1
end

section
```

```
theorem coe_acceptFin : forall  e : SExpr, ((acceptFin e) : Set (autState e)) = (toNFA
     e).accept
  | .sigma => by
      show ((Finset.univ : Finset Unit) : Set Unit) = sigmaDFA.toNFA.accept
      show ((Finset.univ : Finset Unit) : Set Unit) = Set.univ
      ext x; cases x; simp
  | .single sigma => by
      change (({(some <sigma.length, Nat.lt_succ_self _> : SingleState sigma)} :
    Finset _) : Set _)
        = (singleDFA sigma).toNFA.accept
      simp [DFA.toNFA, singleDFA]
  | .cap a b => by
      change (((acceptFin a)  x ? (acceptFin b)) : Set (autState a  x  autState b))
        = (NFAinter (toNFA a) (toNFA b)).accept
      rw [Finset.coe_product, coe_acceptFin a, coe_acceptFin b]
      rfl
  | .cat a b => by
      show (((acceptFin b).image (inr : autState b  ->  autState a ? autState b)) :
        Set (autState a ? autState b)) = (catEps (toNFA a) (toNFA b)).toNFA.accept
      show (((acceptFin b).image (inr : autState b  ->  autState a ? autState b)) :
        Set (autState a ? autState b)) = inr '' (toNFA b).accept
      rw [Finset.coe_image, coe_acceptFin b]
end

section
theorem coe_startFin : forall  e : SExpr, ((startFin e) : Set (autState e)) = (toNFA e
    ).start
  | .sigma => by
      show ((Finset.univ : Finset Unit) : Set Unit) = sigmaDFA.toNFA.start
      show ((Finset.univ : Finset Unit) : Set Unit) = {sigmaDFA.start}
      ext x; cases x; simp
  | .single sigma => by
      show (({(some <0, Nat.succ_pos _> : SingleState sigma)} : Finset (SingleState
    sigma)) : Set _)
        = (singleDFA sigma).toNFA.start
      simp [DFA.toNFA, singleDFA]
  | .cap a b => by
      show (((startFin a)  x ? (startFin b)) : Set (autState a  x  autState b))
        = (NFAinter (toNFA a) (toNFA b)).start
      rw [Finset.coe_product, coe_startFin a, coe_startFin b]
      rfl
  | .cat a b => by
      show ((if ((startFin a)  I  (acceptFin a)).Nonempty
          then (startFin a).image (inl : autState a  ->  autState a ? autState b)  U
            (startFin b).image (inr : autState b  ->  autState a ? autState b)
          else (startFin a).image (inl : autState a  ->  autState a ? autState b)) :
          Set (autState a ? autState b)) = (catEps (toNFA a) (toNFA b)).toNFA.start
      show _ = (catEps (toNFA a) (toNFA b)).?Closure (catEps (toNFA a) (toNFA b)).
    start
      rw [catEps_start]
      by_cases hne : ((startFin a)  I  (acceptFin a)).Nonempty
      * rw [ite_eq_left hne]
        ext x
        rw [catEps_mem_?Closure_iff,  <-  coe_startFin a,  <-  coe_acceptFin a,  <-
    coe_startFin b]
        simp only [Finset.coe_union, Finset.coe_image, mem_union, mem_image, Finset.
```

```
    mem_coe]
        constructor
        * rintro (<q, hq, rfl> | <q, hq, rfl>)
          * exact Or.inl <q, hq, rfl>
          * refine Or.inr <?_, q, hq, rfl>
            obtain <q0, hq0> := hne
            rw [Finset.mem_inter] at hq0
            exact <q0, hq0.2, q0, hq0.1, rfl>
        * rintro (<q, hq, rfl> | <_, q, hq, rfl>)
          * exact Or.inl <q, hq, rfl>
          * exact Or.inr <q, hq, rfl>
      * rw [ite_eq_right hne]
        ext x
        rw [catEps_mem_?Closure_iff,  <-  coe_startFin a,  <-  coe_acceptFin a,  <-
    coe_startFin b]
        simp only [Finset.coe_image, mem_image, Finset.mem_coe]
        constructor
        * rintro <q, hq, rfl>; exact Or.inl <q, hq, rfl>
        * rintro (<q, hq, rfl> | <<q, hq1, q', hq2, heq>, _>)
          * exact <q, hq, rfl>
          * cases heq
            exact absurd <q, Finset.mem_inter.mpr <hq2, hq1>> hne
end

section
theorem coe_stepFinSingle : forall  (e : SExpr) (s : autState e) (c : Bool),
    ((stepFinSingle e s c) : Set (autState e)) = (toNFA e).step s c
  | .sigma, s, c => by
      show ((Finset.univ : Finset Unit) : Set Unit) = sigmaDFA.toNFA.step s c
      show ((Finset.univ : Finset Unit) : Set Unit) = {sigmaDFA.step s c}
      ext x; cases x; simp
  | .single sigma, s, c => by
      show (({(singleDFA sigma).step s c} : Finset (SingleState sigma)) : Set _)
        = (singleDFA sigma).toNFA.step s c
      simp [DFA.toNFA]
  | .cap a b, (s1, s2), c => by
      show (((stepFinSingle a s1 c)  x ? (stepFinSingle b s2 c)) : Set (autState a  x
     autState b))
        = (NFAinter (toNFA a) (toNFA b)).step (s1, s2) c
      rw [Finset.coe_product, coe_stepFinSingle a s1 c, coe_stepFinSingle b s2 c]
      rfl
  | .cat a b, inl s, c => by
      show ((if ((stepFinSingle a s c)  I  (acceptFin a)).Nonempty
          then (stepFinSingle a s c).image (inl : autState a  ->  autState a ?
    autState b)  U
            (startFin b).image (inr : autState b  ->  autState a ? autState b)
          else (stepFinSingle a s c).image (inl : autState a  ->  autState a ?
    autState b)) :
          Set (autState a ? autState b)) = (catEps (toNFA a) (toNFA b)).toNFA.step (
    inl s) c
      show _ = (catEps (toNFA a) (toNFA b)).?Closure
        ((catEps (toNFA a) (toNFA b)).step (inl s) (some c))
      rw [catEps_step_inl_some]
      by_cases hne : ((stepFinSingle a s c)  I  (acceptFin a)).Nonempty
      * rw [ite_eq_left hne]
        ext x
```

```
        rw [catEps_mem_?Closure_iff,  <-  coe_stepFinSingle a s c,  <-  coe_acceptFin
    a,
           <-  coe_startFin b]
        simp only [Finset.coe_union, Finset.coe_image, mem_union, mem_image, Finset.
    mem_coe]
        constructor
        * rintro (<q, hq, rfl> | <q, hq, rfl>)
          * exact Or.inl <q, hq, rfl>
          * refine Or.inr <?_, q, hq, rfl>
            obtain <q0, hq0> := hne
            rw [Finset.mem_inter] at hq0
            exact <q0, hq0.2, q0, hq0.1, rfl>
        * rintro (<q, hq, rfl> | <_, q, hq, rfl>)
          * exact Or.inl <q, hq, rfl>
          * exact Or.inr <q, hq, rfl>
      * rw [ite_eq_right hne]
        ext x
        rw [catEps_mem_?Closure_iff,  <-  coe_stepFinSingle a s c,  <-  coe_acceptFin
    a,
           <-  coe_startFin b]
        simp only [Finset.coe_image, mem_image, Finset.mem_coe]
        constructor
        * rintro <q, hq, rfl>; exact Or.inl <q, hq, rfl>
        * rintro (<q, hq, rfl> | <<q, hq1, q', hq2, heq>, _>)
          * exact <q, hq, rfl>
          * cases heq
            exact absurd <q, Finset.mem_inter.mpr <hq2, hq1>> hne
  | .cat a b, inr s, c => by
      show (((stepFinSingle b s c).image (inr : autState b  ->  autState a ? autState
    b)) :
        Set (autState a ? autState b)) = (catEps (toNFA a) (toNFA b)).toNFA.step (inr
    s) c
      show _ = (catEps (toNFA a) (toNFA b)).?Closure
        ((catEps (toNFA a) (toNFA b)).step (inr s) (some c))
      rw [catEps_step_inr_some]
      ext x
      rw [catEps_mem_?Closure_iff,  <-  coe_stepFinSingle b s c]
      simp only [Finset.coe_image, mem_image, Finset.mem_coe]
      constructor
      * rintro <q, hq, rfl>; exact Or.inl <q, hq, rfl>
      * rintro (<q, hq, rfl> | <<q, _, hqmem>, _>)
        * exact <q, hq, rfl>
        * exact absurd hqmem (by simp)
end

section
theorem coe_stepFin (e : SExpr) (S : Finset (autState e)) (c : Bool) :
    ((stepFin e S c) : Set (autState e)) = (toNFA e).stepSet S c := by
  ext x
  rw [Finset.mem_coe]
  unfold stepFin
  rw [Finset.mem_biUnion, NFA.mem_stepSet]
  constructor
  * rintro <s, hs, hx>
    exact <s, hs, by rw [ <-  coe_stepFinSingle e s c]; exact Finset.mem_coe.mpr hx>
  * rintro <s, hs, hx>
```

```
    exact <s, hs, by rw [ <-  coe_stepFinSingle e s c] at hx; exact Finset.mem_coe.mp
    hx>
end

section
theorem coe_foldl_stepFin (e : SExpr) (w : List Bool) (S : Finset (autState e)) :
    ((w.foldl (stepFin e) S) : Set (autState e)) = (toNFA e).evalFrom S w := by
  induction w generalizing S with
  | nil => rfl
  | cons c w ih =>
    show ((w.foldl (stepFin e) (stepFin e S c)) : Set (autState e)) = _
    rw [ih (stepFin e S c), NFA.evalFrom_cons, coe_stepFin e S c]
end

section
theorem coe_evalFin (e : SExpr) (w : List Bool) :
    ((evalFin e w) : Set (autState e)) = (toNFA e).eval w := by
  show ((w.foldl (stepFin e) (startFin e)) : Set (autState e))
    = (toNFA e).evalFrom (toNFA e).start w
  rw [coe_foldl_stepFin e w (startFin e), coe_startFin e]
end

section
/-- **`denote e` via the `Finset` simulation.** `w` matches `e` iff some live state
    after reading
`w` is accepting -- a decidable `Finset` intersection test. -/
theorem denote_iff_evalFin (e : SExpr) (w : List Bool) :
    w  in  denote e  <->  (evalFin e w  I  acceptFin e).Nonempty := by
  rw [ <-  toNFA_accepts e]
  constructor
  * rintro <s, hs, hse>
    refine <s, Finset.mem_inter.mpr <?_, ?_>>
    * rw [ <-  Finset.mem_coe, coe_evalFin]; exact hse
    * rw [ <-  Finset.mem_coe, coe_acceptFin]; exact hs
  * rintro <s, hs>
    rw [Finset.mem_inter] at hs
    obtain <hs1, hs2> := hs
    refine <s, ?_, ?_>
    * rw [ <-  Finset.mem_coe, coe_acceptFin] at hs2; exact hs2
    * rw [ <-  Finset.mem_coe, coe_evalFin] at hs1; exact hs1
end

section
theorem diffNFA_evalFrom (e1 e2 : SExpr) (w : List Bool)
    (S1 : Finset (autState e1)) (S2 : Finset (autState e2)) :
    (diffNFA e1 e2).evalFrom {(S1, S2)} w =
      {(w.foldl (stepFin e1) S1, w.foldl (stepFin e2) S2)} := by
  induction w generalizing S1 S2 with
  | nil => rfl
  | cons c w ih =>
    show (diffNFA e1 e2).evalFrom ((diffNFA e1 e2).stepSet {(S1, S2)} c) w = _
    rw [NFA.stepSet_singleton]
    show (diffNFA e1 e2).evalFrom {(stepFin e1 S1 c, stepFin e2 S2 c)} w = _
    exact ih (stepFin e1 S1 c) (stepFin e2 S2 c)
end
```

```
section
theorem diffNFA_eval (e1 e2 : SExpr) (w : List Bool) :
    (diffNFA e1 e2).eval w = {(evalFin e1 w, evalFin e2 w)} := by
  show (diffNFA e1 e2).evalFrom (diffNFA e1 e2).start w = _
  show (diffNFA e1 e2).evalFrom {(startFin e1, startFin e2)} w = _
  exact diffNFA_evalFrom e1 e2 w (startFin e1) (startFin e2)
end

section
/-- **`diffNFA` accepts exactly `denote e_1 \ denote e_2`.** -/
theorem diffNFA_mem_accepts_iff (e1 e2 : SExpr) (w : List Bool) :

    w  in  (diffNFA e1 e2).accepts  <->  w  in  denote e1  /\  w  notin  denote e2 :=
    by
  constructor
  * rintro <s, hs, hse>
    rw [diffNFA_eval, Set.mem_singleton_iff] at hse
    subst hse
    obtain <hacc1, hacc2> := hs
    refine <(denote_iff_evalFin e1 w).mpr hacc1, fun hmem => ?_>
    rw [denote_iff_evalFin] at hmem
    rw [hacc2] at hmem
    exact Finset.not_nonempty_empty hmem
  * rintro <h1, h2>
    refine <(evalFin e1 w, evalFin e2 w), <(denote_iff_evalFin e1 w).mp h1, ?_>, ?_>
    * show evalFin e2 w  I  acceptFin e2 = {}
      by_contra hcontra
      exact h2 ((denote_iff_evalFin e2 w).mpr (Finset.nonempty_iff_ne_empty.mpr
    hcontra))
    * rw [diffNFA_eval]; rfl
end

section
/-- **The shortest witness of `denote e_1 ? denote e_2`, if any, is short.** Generic
`exists_accepted_word_short` applied to `diffNFA`. -/
theorem exists_diff_word_short {e1 e2 : SExpr} (h : not  denote e1  subseteq  denote
    e2) :
    exists  w, w.length < Fintype.card (Finset (autState e1)  x  Finset (autState e2))
      /\
      w  in  denote e1  /\  w  notin  denote e2 := by
  rw [Set.not_subset] at h
  have hne : (diffNFA e1 e2).accepts.Nonempty := by
    obtain <x, hx1, hx2> := h
    exact <x, (diffNFA_mem_accepts_iff e1 e2 x).mpr <hx1, hx2>>
  obtain <w, hlen, hmem> := exists_accepted_word_short hne
  exact <w, hlen, (diffNFA_mem_accepts_iff e1 e2 w).mp hmem>
end

section
theorem subsetB_iff (e1 e2 : SExpr) : subsetB e1 e2 = true  <->  denote e1  subseteq
    denote e2 := by
  unfold subsetB
  rw [List.all_eq_true]
  constructor
  * intro h
    by_contra hsub
```

```
    obtain <w, hlen, hw1, hw2> := exists_diff_word_short hsub
    have hwmem : w  in  wordsUpTo (Fintype.card (Finset (autState e1)  x  Finset (
    autState e2))) :=
      (mem_wordsUpTo _).mpr (le_of_lt hlen)
    have hor := h w hwmem
    have he1 : matchesB e1 w = true := (matchesB_iff e1 w).mpr hw1
    rw [he1] at hor
    simp only [Bool.not_true, Bool.false_or] at hor
    exact hw2 ((matchesB_iff e2 w).mp hor)
  * intro hsub w _
    by_cases hw1 : w  in  denote e1
    * have he2 : matchesB e2 w = true := (matchesB_iff e2 w).mpr (hsub hw1)
      simp [he2]
    * have he1 : matchesB e1 w = false := by
        rw [Bool.eq_false_iff]
        intro hc
        exact hw1 ((matchesB_iff e1 w).mp hc)
      simp [he1]
end

section
theorem interEqB_iff (e1 e2 : SExpr) : interEqB e1 e2 = true  <->  denote e1 = denote
    e2 := by
  unfold interEqB
  rw [Bool.and_eq_true, subsetB_iff, subsetB_iff,  <-  Set.Subset.antisymm_iff]
end

end Exercise722
end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.Exercise722
section
set_option linter.unusedVariables false

theorem example_722_equiv : forall  (e1 e2 : DomainTheory.NeighbourhoodSystems.
    Exercise722.SExpr),
  Iff (@Eq Bool (DomainTheory.NeighbourhoodSystems.Exercise722.interEqB e1 e2) Bool.
    true)
    (@Eq (Set (List Bool)) (DomainTheory.NeighbourhoodSystems.Exercise722.denote e1)
      (DomainTheory.NeighbourhoodSystems.Exercise722.denote e2)) :=
  DomainTheory.NeighbourhoodSystems.Exercise722.interEqB_iff
end

end DomainTheory.NeighbourhoodSystems.Exercise722
namespace Domain.Recursive
open DomainTheory.NeighbourhoodSystems Exercise722
open DomainTheory.NeighbourhoodSystems.Exercise722

section
/-- **AM-GM for naturals (choice-free).** `4 a b  <=  (a+b)?`. Faithful port of mathlib
    's `private`
`Nat.AM_GM` (no classical tactics). -/
theorem amGM : forall  {a b : Nat}, 4 * a * b  <=  (a + b) * (a + b)
  | 0, _ => by rw [Nat.mul_zero, Nat.zero_mul]; exact Nat.zero_le _
  | _, 0 => by rw [Nat.mul_zero]; exact Nat.zero_le _
  | a + 1, b + 1 => by
    simpa only [Nat.mul_add, Nat.add_mul, show (4 : Nat) = 1 + 1 + 1 + 1 from rfl, Nat
```

```
    .one_mul,
      Nat.mul_one, Nat.add_assoc, Nat.add_left_comm, Nat.add_le_add_iff_left]
      using Nat.add_le_add_right (@amGM a b) 4
end

section
/-- **Choice-free `sqrt.iter` lower bound.** `iter n guess ?  <=  n`. Faithful port of
`Nat.sqrt.iter_sq_le`, with `grind` replaced by `omega`. -/
theorem iter_sq_le (n guess : Nat) : Nat.sqrt.iter n guess * Nat.sqrt.iter n guess  <=
      n := by
  unfold Nat.sqrt.iter
  if h : (guess + n / guess) / 2 < guess then
    rw [dite_eq_left h]
    exact iter_sq_le n ((guess + n / guess) / 2)
  else
    rw [dite_eq_right h]
    apply Nat.mul_le_of_le_div
    omega
  termination_by guess
  decreasing_by exact h
end

section
/-- Choice-free left-cancellation for `<` under `*` (mathlib's `Nat.
    lt_of_mul_lt_mul_left` is
classical). From `a*b < a*c` deduce `b < c`, by the contrapositive `c  <=  b  ->  a*c
    <=  a*b`. -/
theorem lt_of_mul_lt_mul_left' {a b c : Nat} (h : a * b < a * c) : b < c := by
  rcases Nat.lt_or_ge b c with hbc | hbc
  * exact hbc
  * exact absurd h (Nat.not_lt.mpr (Nat.mul_le_mul_left a hbc))
end

section
/-- **Choice-free `sqrt.iter` upper bound.** If `n < (guess+1)?` then `n < (iter n
    guess + 1)?`.
Faithful port of `Nat.sqrt.lt_iter_succ_sq`, with `lia` replaced by `omega`. -/
theorem lt_iter_succ_sq (n guess : Nat) (hn : n < (guess + 1) * (guess + 1)) :
    n < (Nat.sqrt.iter n guess + 1) * (Nat.sqrt.iter n guess + 1) := by
  unfold Nat.sqrt.iter
  if h : (guess + n / guess) / 2 < guess then
    rw [dite_eq_left h]
    suffices hsuff : n < ((guess + n / guess) / 2 + 1) * ((guess + n / guess) / 2 + 1)
     from
      lt_iter_succ_sq n ((guess + n / guess) / 2) hsuff
    refine lt_of_mul_lt_mul_left' (a := 4 * (guess * guess)) ?_
    apply Nat.lt_of_le_of_lt amGM
    rw [show (4 : Nat) = 2 * 2 from rfl]
    rw [Nat.mul_mul_mul_comm 2, Nat.mul_mul_mul_comm (2 * guess)]
    refine Nat.mul_self_lt_mul_self (?_ : _ < _ * ((_ / 2) + 1))
    rw [ <-  Nat.add_div_right _ (by decide), Nat.mul_comm 2, Nat.mul_assoc,
      show guess + n / guess + 2 = (guess + n / guess + 1) + 1 from rfl]
    have aux : (guess + n / guess + 1)  <=  2 * ((guess + n / guess + 1 + 1) / 2) :=
    by omega
    refine lt_of_lt_of_le ?_ (Nat.mul_le_mul_left _ aux)
    rw [Nat.add_assoc, Nat.mul_add]
```

```
    exact Nat.add_lt_add_left
      (Nat.lt_mul_div_succ _ (lt_of_le_of_lt (Nat.zero_le _) h)) _
  else
    rw [dite_eq_right h]
    exact hn
  termination_by guess
  decreasing_by exact h
end

section
/-- `n < (sqrt n + 1)?` (choice-free; mathlib's `Nat.lt_succ_sqrt` is classical). The
    initial guess
`2 ^ (log_2 n / 2 + 1)` over-shoots `?n`, which feeds `lt_iter_succ_sq`. -/
theorem lt_succ_sqrt (n : Nat) : n < (Nat.sqrt n + 1) * (Nat.sqrt n + 1) := by
  rcases Nat.lt_or_ge 1 n with h | h
  * rw [Nat.sqrt, ite_eq_right (Nat.not_le.mpr h)]
    refine lt_iter_succ_sq _ _ ?_
    set g := 1 <<< (n.log2 / 2 + 1) with hg
    have hshift : g = 2 ^ (n.log2 / 2 + 1) := by rw [hg, Nat.shiftLeft_eq, Nat.one_mul
    ]
    have hgg : g * g = 2 ^ (2 * (n.log2 / 2 + 1)) := by
      rw [hshift,  <-  pow_add]; congr 1; omega
    calc n < 2 ^ (n.log2 + 1) := Nat.lt_log2_self
      _  <=  2 ^ (2 * (n.log2 / 2 + 1)) := Nat.pow_le_pow_right (by decide) (by omega)
      _ = g * g := hgg.symm
      _  <=  (g + 1) * (g + 1) := Nat.mul_le_mul (Nat.le_succ g) (Nat.le_succ g)
  * rw [Nat.sqrt, ite_eq_left h]
    exact Nat.lt_of_lt_of_le (Nat.lt_succ_self n)
      (Nat.le_mul_of_pos_right (n + 1) (Nat.succ_pos n))
end

section
/-- `sqrt n` squared is ` <=  n` (choice-free; mathlib's `Nat.sqrt_le` is classical).
    -/
theorem sqrt_le (n : Nat) : Nat.sqrt n * Nat.sqrt n  <=  n := by
  rcases Nat.lt_or_ge 1 n with h | h
  * rw [Nat.sqrt, ite_eq_right (Nat.not_le.mpr h)]
    exact iter_sq_le _ _
  * rw [Nat.sqrt, ite_eq_left h]
    calc n * n  <=  1 * n := Nat.mul_le_mul_right n h
      _ = n := Nat.one_mul n
end

section
/-- **The square-root characterization (choice-free).** If `q?  <=  m < (q+1)?` then `
    sqrt m = q`. The
uniqueness step is `omega` from the two bounds plus `sqrt_le`/`lt_succ_sqrt`. -/
theorem sqrt_eq_of {m q : Nat} (h1 : q * q  <=  m) (h2 : m < (q + 1) * (q + 1)) : Nat.
    sqrt m = q := by
  have hle := sqrt_le m
  have hlt := lt_succ_sqrt m
  have hq_le : q  <=  Nat.sqrt m := by
    rcases Nat.lt_or_ge (Nat.sqrt m) q with hh | hh
    * exfalso
      have ha : Nat.sqrt m + 1  <=  q := hh
      have hmul : (Nat.sqrt m + 1) * (Nat.sqrt m + 1)  <=  q * q := Nat.mul_le_mul ha
```

```
    ha
      omega
    * exact hh
  have hle_q : Nat.sqrt m  <=  q := by
    rcases Nat.lt_or_ge q (Nat.sqrt m) with hh | hh
    * exfalso
      have ha : q + 1  <=  Nat.sqrt m := hh
      have hmul : (q + 1) * (q + 1)  <=  Nat.sqrt m * Nat.sqrt m := Nat.mul_le_mul ha
    ha
      omega
    * exact hh
  omega
end

section
/-- `sqrt (q? + a) = q` whenever `a  <=  2q` (choice-free; mathlib's `Nat.sqrt_add_eq`
    is classical). -/
theorem sqrt_add_eq (q a : Nat) (h : a  <=  q + q) : Nat.sqrt (q * q + a) = q := by
  refine sqrt_eq_of (Nat.le_add_right _ _) ?_
  have expand : (q + 1) * (q + 1) = q * q + q + q + 1 := Nat.succ_mul_succ q q
  omega
end

section
/-- **`unpair  comp  pair = id` (choice-free).** Mirrors `Nat.unpair_pair`. -/
theorem unpair_pair (a b : Nat) : Nat.unpair (Nat.pair a b) = (a, b) := by
  rw [Nat.pair]
  if h : a < b then
    rw [ite_eq_left h]
    show Nat.unpair (b * b + a) = (a, b)
    have be : Nat.sqrt (b * b + a) = b := sqrt_add_eq b a (by omega)
    simp only [Nat.unpair, be, Nat.add_sub_cancel_left, ite_eq_left h]
  else
    rw [ite_eq_right h]
    show Nat.unpair (a * a + a + b) = (a, b)
    have ae : Nat.sqrt (a * a + a + b) = a := by
      have e : a * a + a + b = a * a + (a + b) := by omega
      rw [e]; exact sqrt_add_eq a (a + b) (by omega)
    have e1 : a * a + a + b - a * a = a + b := by omega
    have e2 : not  a + b < a := by omega
    have e3 : a + b - a = b := by omega
    simp only [Nat.unpair, ae, e1, ite_eq_right e2, e3]
end

section
@[simp] theorem unpair_pair_fst (a b : Nat) : (Nat.pair a b).unpair.1 = a := by rw [
    unpair_pair]
end

section
@[simp] theorem unpair_pair_snd (a b : Nat) : (Nat.pair a b).unpair.2 = b := by rw [
    unpair_pair]
end

section
/-- Choice-free primitive-recursive **selection**: `selectFn c a b = a` if `c = 1`, `=
```

```
    b` if `c = 0`
(for a `{0,1}`-valued `c`), via `c * a + (1 - c) * b`. -/
def selectFn (c a b : Nat) : Nat := c * a + (1 - c) * b
end

section
@[simp] theorem selectFn_one (a b : Nat) : selectFn 1 a b = a := by simp [selectFn]
end

section
@[simp] theorem selectFn_zero (a b : Nat) : selectFn 0 a b = b := by simp [selectFn]
end

section
@[simp] theorem selectFn_ite {c : Prop} [Decidable c] (a b : Nat) :
    selectFn (if c then 1 else 0) a b = if c then a else b := by
  split <;> simp [selectFn]
end

section
/-- A unary predicate `p : Nat  ->  Prop` is **recursively decidable**. -/
def RecDecidable (p : Nat  ->  Prop) : Prop :=
  exists  f : Nat  ->  Nat, Nat.Primrec f  /\  forall  n, p n  <->  f n = 1
end

section
/-- A unary predicate `p : Nat  ->  Prop` is **recursively enumerable**: it is the
    projection of a
recursively decidable relation, `p n  <->  exists  i, q (Nat.pair i n)`. -/
def REPred (p : Nat  ->  Prop) : Prop :=
  exists  q : Nat  ->  Prop, RecDecidable q  /\  forall  n, p n  <->  exists  i, q (
    Nat.pair i n)
end

section
/-- A binary relation is recursively enumerable when its `Nat.pair`-coding is. -/
def REPred_2 (r : Nat  ->  Nat  ->  Prop) : Prop :=
  REPred fun t => r t.unpair.1 t.unpair.2
end

section
/-- A binary relation is recursively decidable when its `Nat.pair`-coding is. -/
def RecDecidable_2 (r : Nat  ->  Nat  ->  Prop) : Prop :=
  RecDecidable fun t => r t.unpair.1 t.unpair.2
end

section
/-- A ternary relation is recursively decidable when its `Nat.pair`-coding (`pair n (
    pair m k)`) is. -/
def RecDecidable_3 (r : Nat  ->  Nat  ->  Nat  ->  Prop) : Prop :=
  RecDecidable fun t => r t.unpair.1 t.unpair.2.unpair.1 t.unpair.2.unpair.2
end

section
/-- Encode a list of naturals as a single natural: `[]  |->  0`, `a :: l  |->  pair a (
    encodeList l) + 1`.
```

```
The `+1` keeps the empty list (code `0`) distinguishable from any nonempty list. -/
def encodeList : List Nat  ->  Nat
  | [] => 0
  | a :: l => Nat.pair a (encodeList l) + 1
end

section
/-- One step of the code-walking fold. The state `s = pair rc acc` carries the
    remaining code `rc`
and accumulator `acc`. If `rc = 0` the state is a fixed point; otherwise `rc - 1 = pair
     head tail`,
and the step pops `head`, recurses on `tail`, and updates `acc := stp (pair head (pair
    acc params))`. -/
def foldStep (stp : Nat  ->  Nat) (params s : Nat) : Nat :=
  selectFn (1 - s.unpair.1) s
    (Nat.pair (s.unpair.1 - 1).unpair.2
      (stp (Nat.pair (s.unpair.1 - 1).unpair.1 (Nat.pair s.unpair.2 params))))
end

section
/-- Fold the list coded by `c`, threading accumulator `z` and parameter `params`.
    Implemented as
`c`-fold iteration of `foldStep` from the initial state `pair c z`, projecting out the
    accumulator. -/
def foldCode (stp : Nat  ->  Nat) (params z c : Nat) : Nat :=
  ((foldStep stp params)^[c] (Nat.pair c z)).unpair.2
end

section
/-- `halfParity n = pair (n / 2) (n % 2)`, built by structural recursion: from `(h, p)`
     for `n`, the
value for `n+1` is `(h + p, 1 - p)` (carry on odd -> even). -/
def halfParity (n : Nat) : Nat :=
  Nat.rec (motive := fun _ => Nat) 0
    (fun _ ih => Nat.pair (ih.unpair.1 + ih.unpair.2) (1 - ih.unpair.2)) n
end

section
/-- Indicator of `v = 1`, as a `{0,1}`-valued primitive-recursive function. -/
def isOne (v : Nat) : Nat := 1 - ((v - 1) + (1 - v))
end

section
theorem isOne_eq_one_iff (v : Nat) : isOne v = 1  <->  v = 1 := by
  unfold isOne; constructor <;> (intro h; omega)
end

section
@[simp] theorem isOne_one : isOne 1 = 1 := (isOne_eq_one_iff 1).2 rfl
end

section
@[simp] theorem isOne_zero : isOne 0 = 0 := by unfold isOne; omega
end

section
```

```
/-- AND-fold step (`foldCode` shape): `acc  |->  acc  /\  isOne (g <head, params>)`. -/
def allListStp (g : Nat  ->  Nat) (w : Nat) : Nat :=
  selectFn w.unpair.2.unpair.1 (isOne (g (Nat.pair w.unpair.1 w.unpair.2.unpair.2))) 0
end

section
/-- The `{0,1}` flag of the bounded `forall  e  in  decodeList c, g <e, p> = 1`. -/
def allListChar (g : Nat  ->  Nat) (p c : Nat) : Nat := foldCode (allListStp g) p 1 c
end

section
/-- The `{0,1}`-valued bounded-`exists ` indicator: `1` iff `exists  i < N, g (pair i n
    ) = 1`. Folded
right-to-left with `selectFn` so the result stays in `{0,1}`. -/
def bExistsFn (g : Nat  ->  Nat) (n N : Nat) : Nat :=
  Nat.rec (motive := fun _ => Nat) 0 (fun i ih => selectFn ih 1 (isOne (g (Nat.pair i
    n)))) N
end

section
/-- The `{0,1}`-valued bounded-`forall ` indicator: `1` iff `g (pair i n) = 1` for all `
    i < N`. Folded
right-to-left with `selectFn` so the result stays in `{0,1}`. -/
def bForallFn (g : Nat  ->  Nat) (n N : Nat) : Nat :=
  Nat.rec (motive := fun _ => Nat) 1 (fun i ih => selectFn ih (isOne (g (Nat.pair i n)
    )) 0) N
end

section
/-- OR-fold step (`foldCode` shape): `acc  |->  acc  \/  isOne (g <head, params>)`. -/
def existsListStp (g : Nat  ->  Nat) (w : Nat) : Nat :=
  selectFn w.unpair.2.unpair.1 1 (isOne (g (Nat.pair w.unpair.1 w.unpair.2.unpair.2)))
end

section
/-- The `{0,1}` flag of the bounded `exists  e  in  decodeList c, g <e, p> = 1`. -/
def existsListChar (g : Nat  ->  Nat) (p c : Nat) : Nat := foldCode (existsListStp g)
    p 0 c
end

section
/-- `{0,1}` flag: `1` iff `n` encodes a binary digit (`0` or `1`). -/
def isBinDigit (n : Nat) : Nat :=
  selectFn (isOne (1 - n)) 1 (selectFn (isOne (2 - n)) 1 0)
end

section
/-- `{0,1}` flag: every entry of `decodeList c` is a binary digit. -/
def allBinDigitsChar (c : Nat) : Nat :=
  allListChar (fun t => isBinDigit t.unpair.1) 0 c
end

section
/-- `{0,1}` test for `n = 0`. -/
def isZero (n : Nat) : Nat := 1 - min n 1
end
```

```
section
/-- `b  <=  pair a b` (choice-free; avoids mathlib's `Nat.right_le_pair` to keep the
    axiom set clean). -/
theorem le_pair_right (a b : Nat) : b  <=  Nat.pair a b := by
  have hbb : b  <=  b * b := by
    rcases Nat.eq_zero_or_pos b with h | h
    * simp [h]

    * exact Nat.le_mul_of_pos_left b h
  unfold Nat.pair
  split <;> omega
end

section
set_option maxHeartbeats 800000 in

/-- Length fold step: ignore head/params, increment accumulator. -/
def listLenStp (w : Nat) : Nat := w.unpair.2.unpair.1 + 1
end

section
set_option maxHeartbeats 800000 in

/-- Coded list length: `(decodeList c).length`. -/
def listLenChar (c : Nat) : Nat := foldCode listLenStp 0 0 c
end

section
/-- The low-bit OR `(x ||| y) % 2`, in arithmetic `{0,1}`-valued form (so it is
    primitive recursive
without referring to `Nat.lor`). Equal to `(x ||| y) % 2` by `lowOr_eq_mod`. -/
def lowOr (x y : Nat) : Nat := 1 - (1 - (x % 2 + y % 2))
end

section
/-- Packed iteration state `pair (pair curA curB) (pair weight acc)` for the bitwise-OR
     fold. -/
def lorStep (s : Nat) : Nat :=
  Nat.pair (Nat.pair (s.unpair.1.unpair.1 / 2) (s.unpair.1.unpair.2 / 2))
    (Nat.pair (2 * s.unpair.2.unpair.1)
      (s.unpair.2.unpair.2 + s.unpair.2.unpair.1 * lowOr s.unpair.1.unpair.1 s.unpair
    .1.unpair.2))
end

section
/-- Prepend the *pair* `<n, x>` onto the list coded by `acc`. -/
def mapPairStep (n acc x : Nat) : Nat := Nat.pair (Nat.pair n x) acc + 1
end

section
/-- `foldCode`-shaped wrapper of `mapPairStep` (parameter `n` threaded via the `params`
     slot). -/
def mapPairStp (w : Nat) : Nat :=
  mapPairStep w.unpair.2.unpair.2 w.unpair.2.unpair.1 w.unpair.1
end
```

```
section
/-- **`mapPairCode n c`** codes the list `(decodeList c).map <n, *>` (reversed). -/
def mapPairCode (n c : Nat) : Nat := foldCode mapPairStp n 0 c
end

section
set_option maxHeartbeats 800000 in

/-- `{0,1}` AND on flags (both must be `1`). -/
def mulBit (a b : Nat) : Nat := a * b
end

section
set_option maxHeartbeats 800000 in

/-- One step of fuel-bounded decode-ok (previous fuel level supplied as `prev`). -/
def decodeFuelOkCharBody (prev : Nat  ->  Nat) (c : Nat) : Nat :=
  let sub := mulBit (prev c.unpair.2.unpair.1) (prev c.unpair.2.unpair.2)
  selectFn (isOne (1 - c.unpair.1))
    (selectFn (isOne (1 - c.unpair.2)) 1 0)
    (selectFn (isOne (2 - c.unpair.1)) (allBinDigitsChar c.unpair.2)
      (selectFn (isOne (3 - c.unpair.1)) sub
        (selectFn (isOne (4 - c.unpair.1)) sub 0)))
end

section
set_option maxHeartbeats 800000 in

/-- `{0,1}` flag: `1` iff fuel-bounded `SExpr` decode succeeds on code `c`. -/
def decodeFuelOkChar : Nat  ->  Nat  ->  Nat
  | 0, _ => 0
  | fuel + 1, c => decodeFuelOkCharBody (decodeFuelOkChar fuel) c
end

section
/-- The iterative bitwise OR: iterate `lorStep` `a + b` times (enough to consume every
    set bit of `a`
and `b`) from the initial state, and read off the accumulator. -/
def myLor (a b : Nat) : Nat :=
  (lorStep^[a + b] (Nat.pair (Nat.pair a b) (Nat.pair 1 0))).unpair.2.unpair.2
end

section
set_option maxHeartbeats 800000 in

/-- `{0,1}` nat-equality char (reused in C9b tag/element comparisons). -/
def natEqChar (a b : Nat) : Nat := 1 - ((a - b) + (b - a))
end

section
set_option maxHeartbeats 800000 in

@[simp] theorem natEqChar_eq_one_iff (a b : Nat) : natEqChar a b = 1  <->  a = b := by
  unfold natEqChar; omega
end
```

```
section
set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

open DomainTheory.NeighbourhoodSystems Exercise722
open DomainTheory.NeighbourhoodSystems.Exercise722

/-- One fuel step of state-card counting (previous fuel level as `prev`). -/
def autStateCardFuelCharBody (prev : Nat  ->  Nat) (c : Nat) : Nat :=
  let subAdd := prev c.unpair.2.unpair.1 + prev c.unpair.2.unpair.2
  let subMul := prev c.unpair.2.unpair.1 * prev c.unpair.2.unpair.2
  selectFn (isOne (1 - c.unpair.1))
    1
    (selectFn (isOne (2 - c.unpair.1))
      (listLenChar c.unpair.2 + 2)
      (selectFn (isOne (3 - c.unpair.1))
        subAdd
        (selectFn (isOne (4 - c.unpair.1)) subMul 0)))
end

section
set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

open DomainTheory.NeighbourhoodSystems Exercise722
open DomainTheory.NeighbourhoodSystems.Exercise722

/-- Fuel-bounded mirror of `autStateCard` on Godel codes. -/
def autStateCardFuelChar : Nat  ->  Nat  ->  Nat
  | 0, _ => 0
  | fuel + 1, c => autStateCardFuelCharBody (autStateCardFuelChar fuel) c
end

section
set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

open DomainTheory.NeighbourhoodSystems Exercise722
open DomainTheory.NeighbourhoodSystems.Exercise722

/-- **Not** `private`: `Exercise722Presentation.lean` (downstream of this file via `
    Definition71`)
bridges this to its own `boolNat` in Session **C9b7**, since `Recursive.lean` cannot
    import
`Exercise722Presentation.lean` (would cycle) and so cannot state that bridge itself. -/
def c9b5_boolNat (b : Bool) : Nat := if b then 1 else 0
```

```
end

section
set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

open DomainTheory.NeighbourhoodSystems Exercise722
open DomainTheory.NeighbourhoodSystems.Exercise722

def c9b5_encodeListBool (sigma : List Bool) : Nat := encodeList (sigma.map
    c9b5_boolNat)
end

section
set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

open DomainTheory.NeighbourhoodSystems Exercise722
open DomainTheory.NeighbourhoodSystems.Exercise722

def c9b5_sexprDepth : SExpr  ->  Nat
  | .sigma => 1
  | .single _ => 1
  | .cat a b => 1 + max (c9b5_sexprDepth a) (c9b5_sexprDepth b)
  | .cap a b => 1 + max (c9b5_sexprDepth a) (c9b5_sexprDepth b)
end

section
set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

open DomainTheory.NeighbourhoodSystems Exercise722
open DomainTheory.NeighbourhoodSystems.Exercise722

/-- Godel code mirroring `SExpr.encode` in `Exercise722Presentation.lean`. -/
def c9b5_sexprGodelEncode : SExpr  ->  Nat
  | .sigma => Nat.pair 0 0
  | .single sigma => Nat.pair 1 (c9b5_encodeListBool sigma)
  | .cat a b => Nat.pair 2 (Nat.pair (c9b5_sexprGodelEncode a) (c9b5_sexprGodelEncode
    b))
  | .cap a b => Nat.pair 3 (Nat.pair (c9b5_sexprGodelEncode a) (c9b5_sexprGodelEncode
    b))
end

section
set_option maxHeartbeats 800000 in
```

```
set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

open DomainTheory.NeighbourhoodSystems Exercise722
open DomainTheory.NeighbourhoodSystems.Exercise722

set_option maxHeartbeats 1600000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

/-- Tag-3 cap code matching `SExpr.encode` / `c9b5_sexprGodelEncode`. -/
def capCode (a b : Nat) : Nat := Nat.pair 3 (Nat.pair a b)
end

section
set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

open DomainTheory.NeighbourhoodSystems Exercise722
open DomainTheory.NeighbourhoodSystems.Exercise722

set_option maxHeartbeats 1600000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

/-- Upper bound on Godel codes of `{0,1}`-lists of length at most `n`. -/
def codeBound : Nat  ->  Nat
  | 0 => 1
  | n + 1 => Nat.pair 1 (codeBound n) + 1
end

section
set_option maxHeartbeats 800000 in

/-- Non-empty-`remC2` branch of `listEqStp`. -/
def listEqStpNonzero (w : Nat) : Nat :=
  Nat.pair
    (selectFn w.unpair.2.unpair.1.unpair.1
      (natEqChar w.unpair.1 ((w.unpair.2.unpair.1.unpair.2 - 1).unpair.1)) 0)
    ((w.unpair.2.unpair.1.unpair.2 - 1).unpair.2)
end

section
set_option maxHeartbeats 800000 in

/-- Fold step: state `pair flag remC2`; consume one head of `c1`, peel one cons off `
    remC2`. -/
def listEqStp (w : Nat) : Nat :=
```

```
  selectFn (isZero w.unpair.2.unpair.1.unpair.2) (Nat.pair 0 0) (listEqStpNonzero w)
end

section
set_option maxHeartbeats 800000 in

/-- `listEqChar c1 c2 = 1  <->  decodeList c1 = decodeList c2`. -/
def listEqChar (c1 c2 : Nat) : Nat :=
  let r := foldCode listEqStp 0 (Nat.pair 1 c2) c1
  mulBit r.unpair.1 (isZero r.unpair.2)
end

section
set_option maxHeartbeats 800000 in

def listEqStep (st x : Nat) : Nat := listEqStp (Nat.pair x (Nat.pair st 0))
end

section
set_option maxHeartbeats 800000 in

/-- `foldCode`-shaped indexing step. State `pair counter result`, parameter the target
    index `i`;
bump the counter and overwrite the result exactly when `counter = i` (via the `{0,1}`
    equality bit
`1 - ((counter - i) + (i - counter))`). -/
def nthStp (w : Nat) : Nat :=
  Nat.pair (w.unpair.2.unpair.1.unpair.1 + 1)
    (selectFn (1 - ((w.unpair.2.unpair.1.unpair.1 - w.unpair.2.unpair.2)
        + (w.unpair.2.unpair.2 - w.unpair.2.unpair.1.unpair.1)))
      w.unpair.1 w.unpair.2.unpair.1.unpair.2)
end

section
set_option maxHeartbeats 800000 in

/-- **Primitive-recursive indexing into a coded list.** `nthCode c i d = (decodeList c)
    .getD i d`. -/
def nthCode (c i d : Nat) : Nat := (foldCode nthStp i (Nat.pair 0 d) c).unpair.2
end

section
set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

/-- Tabulation lookup for `appendListCode`: params `pair c1 (pair c2 len1)`. -/
def appendListTabFn (w : Nat) : Nat :=
  let i := w.unpair.1
  let p := w.unpair.2
  let len1 := p.unpair.2.unpair.2
  selectFn (isZero ((i + 1) - len1))
    (nthCode p.unpair.1 i 0)
    (nthCode p.unpair.2.unpair.1 (i - len1) 0)
```

```
end

section
set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

open DomainTheory.NeighbourhoodSystems Exercise722
open DomainTheory.NeighbourhoodSystems.Exercise722

/-- `autStateCardFuelCharBody`'s step, with `prev` replaced by table lookup. -/
def autStateCardFuelCharLookup (w : Nat) : Nat :=
  autStateCardFuelCharBody (fun c' => nthCode w.unpair.1 c' 0) w.unpair.2
end

section
set_option maxHeartbeats 800000 in

/-- `decodeFuelOkCharBody`'s step, with `prev` replaced by table lookup: `w.unpair.1` is the
previous fuel level's table, `w.unpair.2` is the code being queried. -/
def decodeFuelOkCharLookup (w : Nat) : Nat :=
  decodeFuelOkCharBody (fun c' => nthCode w.unpair.1 c' 0) w.unpair.2
end

section
set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

/-- Tabulation lookup for `dropCode`: params `pair n c`. -/
def dropListTabFn (w : Nat) : Nat :=
  nthCode (w.unpair.2.unpair.2) (w.unpair.1 + w.unpair.2.unpair.1) 0
end

section
/-- The pure (set-theoretic) witness-threading fold step. The accumulator is `pair remWitness flag`:
at the list head `x` we pop a witness `i` (the `decodeList` head of `remWitness`, i.e.
`(remWitness - 1).unpair.1`) with new remaining code `(remWitness - 1).unpair.2`, and `AND` the flag
with `isOne (qc <i, x>)`. -/
def reForallF (qc : Nat  ->  Nat) (acc x : Nat) : Nat :=
  Nat.pair (acc.unpair.1 - 1).unpair.2
    (selectFn acc.unpair.2 (isOne (qc (Nat.pair (acc.unpair.1 - 1).unpair.1 x))) 0)
end

section
/-- The `foldCode`-form step (parameter unused), used to package `reForallF` primitive-recursively. -/
def reForallStp (qc : Nat  ->  Nat) (w : Nat) : Nat :=
  Nat.pair (w.unpair.2.unpair.1.unpair.1 - 1).unpair.2
```

```
    (selectFn w.unpair.2.unpair.1.unpair.2
      (isOne (qc (Nat.pair (w.unpair.2.unpair.1.unpair.1 - 1).unpair.1 w.unpair.1)))
    0)
end

section
/-- The `{0,1}`-flag computed by threading witness code `w` through the list coded by `
    c`. -/
def reForallChar (qc : Nat  ->  Nat) (w c : Nat) : Nat :=
  (foldCode (reForallStp qc) 0 (Nat.pair w 1) c).unpair.2
end

section
/-- `n  <=  (sqrt n)? + 2*sqrt n` (choice-free; needed for `pair_unpair`). -/
theorem sqrt_le_add (n : Nat) : n  <=  Nat.sqrt n * Nat.sqrt n + Nat.sqrt n + Nat.sqrt
     n := by
  have h := lt_succ_sqrt n
  have e : (Nat.sqrt n + 1) * (Nat.sqrt n + 1)
      = Nat.sqrt n * Nat.sqrt n + Nat.sqrt n + Nat.sqrt n + 1 := Nat.succ_mul_succ _ _
  omega
end

section
/-- **`pair  comp  unpair = id` (choice-free).** Mirrors `Nat.pair_unpair`. -/
theorem pair_unpair (n : Nat) : Nat.pair (Nat.unpair n).1 (Nat.unpair n).2 = n := by
  have sm : Nat.sqrt n * Nat.sqrt n + (n - Nat.sqrt n * Nat.sqrt n) = n :=
    Nat.add_sub_cancel' (sqrt_le _)
  have hadd := sqrt_le_add n
  rw [Nat.unpair]
  if h : n - Nat.sqrt n * Nat.sqrt n < Nat.sqrt n then
    rw [ite_eq_left h]
    show Nat.pair (n - Nat.sqrt n * Nat.sqrt n) (Nat.sqrt n) = n
    rw [Nat.pair, ite_eq_left h]
    omega
  else
    rw [ite_eq_right h]
    have hge : Nat.sqrt n  <=  n - Nat.sqrt n * Nat.sqrt n := Nat.le_of_not_lt h
    have hlt : not  Nat.sqrt n < n - Nat.sqrt n * Nat.sqrt n - Nat.sqrt n := by omega

    show Nat.pair (Nat.sqrt n) (n - Nat.sqrt n * Nat.sqrt n - Nat.sqrt n) = n
    rw [Nat.pair, ite_eq_right hlt]
    omega
end

section
set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

open DomainTheory.NeighbourhoodSystems Exercise722
open DomainTheory.NeighbourhoodSystems.Exercise722

set_option maxHeartbeats 1600000 in
```

```
set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

/-- Numeric mirror of `Fintype.card (Finset (autState e_1)  x  Finset (autState e_2))`
(`exists_diff_word_short`'s state-count bound), via `autStateCard_eq_card` +
`Fintype.card_finset`/`card_prod`. -/
def subsetBoundChar (fuel c1 c2 : Nat) : Nat :=
  2 ^ (autStateCardFuelChar fuel c1) * 2 ^ (autStateCardFuelChar fuel c2)
end

section
set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

open DomainTheory.NeighbourhoodSystems Exercise722
open DomainTheory.NeighbourhoodSystems.Exercise722

set_option maxHeartbeats 1600000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

/-- `{0,1}` gate: code `i` is a valid bit-string of length ` <=  subsetBoundChar fuel
    c1 c2`. Screens
out both non-bit-string codes (`allBinDigitsChar`) and over-long ones (`listLenChar` vs
     the bound),
so the bounded-forall search below never needs the (false) converse of `codeBound_ge`.
    -/
def subsetGuardChar (fuel c1 c2 i : Nat) : Nat :=
  mulBit (allBinDigitsChar i) (isOne (1 - (listLenChar i - subsetBoundChar fuel c1 c2)
    ))
end

section
set_option maxHeartbeats 800000 in

/-- `tabCode`'s recursion step, in the `Nat.Primrec.prec` packaging: the state is
`pair a (pair y IH)` with `a = pair B p`; it prepends `g<B-1-y, p>` onto the tail `IH`.
     -/
def tabStep (g : Nat  ->  Nat) (w : Nat) : Nat :=
  Nat.pair (g (Nat.pair (w.unpair.1.unpair.1 - 1 - w.unpair.2.unpair.1) w.unpair.1.
    unpair.2))
    w.unpair.2.unpair.2 + 1
end

section
set_option maxHeartbeats 800000 in

/-- **Primitive-recursive tabulation.** With `a = pair B p`, `tabCode g a B` codes the
    list
`[g<0,p>, g<1,p>, ..., g<B-1,p>]` (length `B`). Built directly from the `prec` step `
```

```
    tabStep`. -/
def tabCode (g : Nat  ->  Nat) (a B : Nat) : Nat :=
  Nat.rec (motive := fun _ => Nat) 0 (fun y IH => tabStep g (Nat.pair a (Nat.pair y IH
    ))) B
end

section
set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

/-- Coded list append: `decodeList (appendListCode c1 c2) = decodeList c1 ++ decodeList
     c2`. -/
def appendListCode (c1 c2 : Nat) : Nat :=
  let len1 := listLenChar c1
  let len2 := listLenChar c2
  tabCode appendListTabFn (Nat.pair (len1 + len2) (Nat.pair c1 (Nat.pair c2 len1))) (
    len1 + len2)
end

section
set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

/-- Coded list drop: `decodeList (dropCode n c) = (decodeList c).drop n`. -/
def dropCode (n c : Nat) : Nat :=
  let len := listLenChar c
  tabCode dropListTabFn (Nat.pair (len - n) (Nat.pair n c)) (len - n)
end

section
set_option maxHeartbeats 800000 in

/-- One fuel-recursion step, applied to every code in `[0, bound]` at once. `bodyLookup
    `
represents `Body`'s step with `prev` replaced by table lookup: semantically `bodyLookup
     (pair
table c) = Body (fun c' => nthCode table c' 0) c` (see `fuelTable_eq_of_recursion`). -/
def fuelTableStep (bodyLookup : Nat  ->  Nat) (bound prevTable : Nat) : Nat :=
  tabCode (fun w => bodyLookup (Nat.pair w.unpair.2 w.unpair.1))
    (Nat.pair (bound + 1) prevTable) (bound + 1)
end

section
set_option maxHeartbeats 800000 in

/-- Tabulates the fuel-recursive function with base value `0` and step `bodyLookup` at
    every code
in `[0, bound]`, as a coded list, for `fuel` iterations. -/
def fuelTable (bodyLookup : Nat  ->  Nat) (bound fuel : Nat) : Nat :=
  Nat.rec (motive := fun _ => Nat) (tabCode (fun _ => 0) (Nat.pair (bound + 1) 0) (
```

```
    bound + 1))
     (fun _ prevTable => fuelTableStep bodyLookup bound prevTable) fuel
end

section
set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

/-- Tabulation lookup for `takeCode`: params `pair n c`. -/
def takeListTabFn (w : Nat) : Nat := nthCode (w.unpair.2.unpair.2) w.unpair.1 0
end

section
set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

/-- Coded list take: `decodeList (takeCode n c) = (decodeList c).take n`. -/
def takeCode (n c : Nat) : Nat :=
  let len := listLenChar c
  let B := Nat.min n len
  tabCode takeListTabFn (Nat.pair B (Nat.pair n c)) B
end

section
set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

open DomainTheory.NeighbourhoodSystems Exercise722
open DomainTheory.NeighbourhoodSystems.Exercise722

/-- Cat-branch step for `matchesBCharBody` (`p` = `pair cutPoint (pair c cw)`). -/
def matchesBCatG (prev : Nat  ->  Nat) (p : Nat) : Nat :=
  let i := p.unpair.1
  let c := p.unpair.2.unpair.1
  let cw := p.unpair.2.unpair.2
  mulBit (prev (Nat.pair c.unpair.2.unpair.1 (takeCode i cw)))
         (prev (Nat.pair c.unpair.2.unpair.2 (dropCode i cw)))
end

section
set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

open DomainTheory.NeighbourhoodSystems Exercise722
```

```
open DomainTheory.NeighbourhoodSystems.Exercise722

set_option maxHeartbeats 1600000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

/-- `matchesBCatG`'s step, with `prev` replaced by table lookup: `w.unpair.1` is the
    table,
`w.unpair.2` is the packed `pair i (pair c cw)`. -/
def matchesBCatGLookup (w : Nat) : Nat :=
  matchesBCatG (fun p' => nthCode w.unpair.1 p' 0) w.unpair.2
end

section
set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

open DomainTheory.NeighbourhoodSystems Exercise722
open DomainTheory.NeighbourhoodSystems.Exercise722

/-- One fuel step of `matchesB` (packed subcode+word at `prev`). -/
def matchesBCharBody (prev : Nat  ->  Nat) (c cw : Nat) : Nat :=
  let catExist := bExistsFn (matchesBCatG prev) (Nat.pair c cw) (listLenChar cw + 1)
  let capAnd := mulBit (prev (Nat.pair c.unpair.2.unpair.1 cw))
                    (prev (Nat.pair c.unpair.2.unpair.2 cw))
  selectFn (isOne (1 - c.unpair.1))
    1
    (selectFn (isOne (2 - c.unpair.1))
      (listEqChar cw c.unpair.2)
      (selectFn (isOne (3 - c.unpair.1))
        catExist
        (selectFn (isOne (4 - c.unpair.1)) capAnd 0)))
end

section
set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

open DomainTheory.NeighbourhoodSystems Exercise722
open DomainTheory.NeighbourhoodSystems.Exercise722

/-- Fuel-bounded `{0,1}` mirror of `matchesB`. -/
def matchesBChar : Nat  ->  Nat  ->  Nat  ->  Nat
  | 0, _, _ => 0
  | fuel + 1, c, cw =>
      matchesBCharBody (fun p => matchesBChar fuel p.unpair.1 p.unpair.2) c cw
end
```

```
section
set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

open DomainTheory.NeighbourhoodSystems Exercise722
open DomainTheory.NeighbourhoodSystems.Exercise722

set_option maxHeartbeats 1600000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

/-- `{0,1}` non-emptiness decider: bounded search over valid bit-list codes. The
    candidate
word's Godel code `i` and the target `c_e` are threaded through `bExistsFn`'s own
    parameter slot
(`n := c_e`) rather than a fixed `0`, so `primrec_decideNonemptyBChar` composes
    directly out of
`primrec_bExistsFn`. -/
def decideNonemptyBChar (fuel : Nat) (c_e : Nat) : Nat :=
  bExistsFn (fun p =>
      mulBit (allBinDigitsChar p.unpair.1) (matchesBChar fuel p.unpair.2 p.unpair.1))
    c_e (codeBound (autStateCardFuelChar fuel c_e))
end

section
set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

open DomainTheory.NeighbourhoodSystems Exercise722
open DomainTheory.NeighbourhoodSystems.Exercise722

set_option maxHeartbeats 1600000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

/-- `{0,1}` cap-consistency decider on Godel codes. -/
def consistentBChar (fuel : Nat) (c1 c2 : Nat) : Nat :=
  decideNonemptyBChar fuel (capCode c1 c2)
end

section
set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in
```

```
open DomainTheory.NeighbourhoodSystems Exercise722
open DomainTheory.NeighbourhoodSystems.Exercise722

set_option maxHeartbeats 1600000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

/-- `matchesBCharBody`'s step, with `prev` replaced by table lookup. -/
def matchesBCharBodyLookup (w : Nat) : Nat :=
  matchesBCharBody (fun p' => nthCode w.unpair.1 p' 0) w.unpair.2.unpair.1 w.unpair.2.
    unpair.2
end

section
set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

open DomainTheory.NeighbourhoodSystems Exercise722
open DomainTheory.NeighbourhoodSystems.Exercise722

set_option maxHeartbeats 1600000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

@[irreducible] def subsetBChar (fuel c1 c2 : Nat) : Nat :=
  bForallFn (fun p =>
      selectFn (subsetGuardChar fuel p.unpair.2.unpair.1 p.unpair.2.unpair.2 p.unpair
    .1)
        (selectFn (matchesBChar fuel p.unpair.2.unpair.1 p.unpair.1)
          (matchesBChar fuel p.unpair.2.unpair.2 p.unpair.1) 1)
        1)
    (Nat.pair c1 c2) (codeBound (subsetBoundChar fuel c1 c2))
end

section
set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

open DomainTheory.NeighbourhoodSystems Exercise722
open DomainTheory.NeighbourhoodSystems.Exercise722

set_option maxHeartbeats 1600000 in

set_option maxHeartbeats 800000 in
```

```
set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

@[irreducible] def interEqChar (fuel c1 c2 : Nat) : Nat :=
  mulBit (subsetBChar fuel c1 c2) (subsetBChar fuel c2 c1)
end

section
/-- `n.unpair.2  <=  n` (choice-free); the decreasing measure for `decodeList`. -/
theorem unpair_snd_le (n : Nat) : n.unpair.2  <=  n := by
  have h := le_pair_right n.unpair.1 n.unpair.2
  rwa [pair_unpair] at h
end

section
/-- Decode a natural back into a list of naturals, inverting `encodeList`. Well-founded
     on the
remaining code (`c.unpair.2  <=  c < c + 1`). -/
def decodeList : Nat  ->  List Nat
  | 0 => []
  | (c + 1) => c.unpair.1 :: decodeList c.unpair.2
decreasing_by exact Nat.lt_succ_of_le (unpair_snd_le c)
end

section
set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

open DomainTheory.NeighbourhoodSystems Exercise722
open DomainTheory.NeighbourhoodSystems.Exercise722

set_option maxHeartbeats 1600000 in

set_option maxHeartbeats 800000 in

set_option maxHeartbeats 800000 in

def c9b6_decodeListBool (c : Nat) : List Bool :=
  (decodeList c).map fun n => decide (n = 1)
end

section
/-- `id` is primitive recursive (choice-free). -/
theorem primrec_id : Nat.Primrec id :=
  (Nat.Primrec.left.pair Nat.Primrec.right).of_eq fun n => pair_unpair n

/-- The `prec` recursor for addition computes the sum. -/
theorem rec_add (a b : Nat) : Nat.rec a (fun _ IH => IH + 1) b = a + b := by
  induction b with
  | zero => rfl
  | succ k ih => show Nat.rec a (fun _ IH => IH + 1) k + 1 = a + (k + 1); omega
```

```
/-- The `prec` recursor for multiplication computes the product. -/
private theorem rec_mul (a b : Nat) : Nat.rec 0 (fun _ IH => a + IH) b = a * b := by
  induction b with
  | zero => rfl
  | succ k ih =>
    show a + Nat.rec 0 (fun _ IH => a + IH) k = a * (k + 1)
    rw [Nat.mul_succ]; omega

/-- Addition is primitive recursive (choice-free). -/
theorem primrec_add : Nat.Primrec (Nat.unpaired (* + *)) :=
  (Nat.Primrec.prec primrec_id
      ((Nat.Primrec.succ.comp Nat.Primrec.right).comp Nat.Primrec.right)).of_eq fun p
    => by
    simp only [unpair_pair_snd, id_eq]


    exact rec_add _ _

/-- Multiplication is primitive recursive (choice-free). -/
theorem primrec_mul : Nat.Primrec (Nat.unpaired (* * *)) :=
  (Nat.Primrec.prec Nat.Primrec.zero
      (primrec_add.comp (Nat.Primrec.pair Nat.Primrec.left
        (Nat.Primrec.right.comp Nat.Primrec.right)))).of_eq fun p => by
    simp only [unpair_pair_fst, unpair_pair_snd, Nat.unpaired]
    exact rec_mul _ _

/-- The `prec` recursor for predecessor computes `n - 1`. -/
private theorem rec_pred (n : Nat) : Nat.rec 0 (fun y _ => y) n = n - 1 := by
  cases n with
  | zero => rfl
  | succ k => rfl

/-- Predecessor `n  |->  n - 1` is primitive recursive (choice-free; mathlib's `Nat.
    Primrec.pred` is
classical). -/
theorem primrec_pred : Nat.Primrec (fun n => n - 1) :=
  ((Nat.Primrec.prec Nat.Primrec.zero (Nat.Primrec.left.comp Nat.Primrec.right)).comp
    (primrec_id.pair primrec_id)).of_eq fun n => by
    simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd, id_eq]
    exact rec_pred n

/-- The `prec` recursor for truncated subtraction computes `a - b`. -/
private theorem rec_sub (a b : Nat) : Nat.rec a (fun _ IH => IH - 1) b = a - b := by
  induction b with
  | zero => rfl
  | succ k ih => show Nat.rec a (fun _ IH => IH - 1) k - 1 = a - (k + 1); rw [ih];
    omega

/-- Truncated subtraction is primitive recursive (choice-free; mathlib's `Nat.Primrec.
    sub` is
classical). -/
theorem primrec_sub : Nat.Primrec (Nat.unpaired (* - *)) :=
  (Nat.Primrec.prec primrec_id
    ((primrec_pred.comp Nat.Primrec.right).comp Nat.Primrec.right)).of_eq fun p => by
    simp only [unpair_pair_snd, id_eq]
    exact rec_sub _ _
```

```
/-- `a * b = 1  <->  a = 1  /\  b = 1` over `Nat` (choice-free; avoids the classical
    generic `mul_eq_one`). -/
theorem nat_mul_eq_one {a b : Nat} : a * b = 1  <->  a = 1  /\  b = 1 := by
  refine <fun h => <Nat.dvd_one.mp <b, h.symm>, Nat.dvd_one.mp <a, ?_>>, fun h => by
    rw [h.1, h.2]>
  rw [Nat.mul_comm] at h; exact h.symm

/-- `fun n => f n + g n` is primitive recursive. -/
theorem primrec_add_2 {f g : Nat  ->  Nat} (hf : Nat.Primrec f) (hg : Nat.Primrec g) :
    Nat.Primrec (fun n => f n + g n) :=
  (primrec_add.comp (hf.pair hg)).of_eq fun n => by
    simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd]

/-- `fun n => f n * g n` is primitive recursive. -/
theorem primrec_mul_2 {f g : Nat  ->  Nat} (hf : Nat.Primrec f) (hg : Nat.Primrec g) :
    Nat.Primrec (fun n => f n * g n) :=
  (primrec_mul.comp (hf.pair hg)).of_eq fun n => by
    simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd]

/-- `fun n => f n - g n` (truncated) is primitive recursive. -/
theorem primrec_sub_2 {f g : Nat  ->  Nat} (hf : Nat.Primrec f) (hg : Nat.Primrec g) :
    Nat.Primrec (fun n => f n - g n) :=
  (primrec_sub.comp (hf.pair hg)).of_eq fun n => by
    simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd]

/-- `selectFn` of primitive-recursive `c, a, b` is primitive recursive. -/
theorem primrec_selectFn {c a b : Nat  ->  Nat} (hc : Nat.Primrec c) (ha : Nat.Primrec
    a)
    (hb : Nat.Primrec b) : Nat.Primrec (fun n => selectFn (c n) (a n) (b n)) :=
  primrec_add_2 (primrec_mul_2 hc ha) (primrec_mul_2 (primrec_sub_2 (Nat.Primrec.const
    1) hc) hb)

/-- Recursive decidability transfers across a pointwise logical equivalence. -/
theorem RecDecidable.of_iff {p q : Nat  ->  Prop} (h : forall  n, q n  <->  p n) (hp :
    RecDecidable p) :
    RecDecidable q := by
  obtain <f, hf, hfe> := hp
  exact <f, hf, fun n => (h n).trans (hfe n)>

/-- **Reindexing.** If `p` is recursively decidable and `g` is primitive recursive,
    then
`fun n => p (g n)` is recursively decidable. -/
theorem RecDecidable.comp {p : Nat  ->  Prop} (hp : RecDecidable p) {g : Nat  ->  Nat}
    (hg : Nat.Primrec g) : RecDecidable (fun n => p (g n)) := by
  obtain <f, hf, hfe> := hp
  exact <fun n => f (g n), hf.comp hg, fun n => hfe (g n)>

/-- **Conjunction.** Recursive decidability is closed under ` /\ ` (multiply the
    `{0,1}` deciders). -/
theorem RecDecidable.and {p q : Nat  ->  Prop} (hp : RecDecidable p) (hq :
    RecDecidable q) :
    RecDecidable (fun n => p n  /\  q n) := by
  obtain <f, hf, hfe> := hp
  obtain <g, hg, hge> := hq
  refine <fun n => f n * g n, ?_, fun n => ?_>
  * exact (primrec_mul.comp (hf.pair hg)).of_eq fun n => by
```

```
      simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd]
  * rw [nat_mul_eq_one]; exact and_congr (hfe n) (hge n)

/-- **Equality of two primitive-recursive functions is recursively decidable.** The
    `{0,1}`-valued
characteristic function is `1 - ((a t - b t) + (b t - a t))` (truncated subtraction),
    which is `1`
exactly when `a t = b t`; primitive recursive via `primrec_sub`/`primrec_add`, and the
    biconditional
is `omega` (which understands truncated subtraction). -/
theorem RecDecidable.natEq {a b : Nat  ->  Nat} (ha : Nat.Primrec a) (hb : Nat.Primrec
     b) :
    RecDecidable (fun t => a t = b t) := by
  refine <fun t => Nat.unpaired (* - *) (Nat.pair 1 (Nat.unpaired (* + *)
      (Nat.pair (Nat.unpaired (* - *) (Nat.pair (a t) (b t)))
        (Nat.unpaired (* - *) (Nat.pair (b t) (a t))))))), ?_, fun t => ?_>
  * exact primrec_sub.comp ((Nat.Primrec.const 1).pair (primrec_add.comp
      ((primrec_sub.comp (ha.pair hb)).pair (primrec_sub.comp (hb.pair ha)))))
  * simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd]
    constructor <;> intro h <;> omega

/-- **Negation.** Recursive decidability is closed under `not ` (negate the `{0,1}`
    decider). -/
theorem RecDecidable.not {p : Nat  ->  Prop} (hp : RecDecidable p) : RecDecidable (fun
     n => not  p n) := by
  obtain <f, hf, hfe> := hp
  refine <fun n => Nat.unpaired (* - *) (Nat.pair 1 (Nat.unpaired (* - *) (Nat.pair 1
      (Nat.unpaired (* + *) (Nat.pair (Nat.unpaired (* - *) (Nat.pair (f n) 1))
        (Nat.unpaired (* - *) (Nat.pair 1 (f n))))))), ?_, fun n => ?_>
  * exact primrec_sub.comp ((Nat.Primrec.const 1).pair (primrec_sub.comp
      ((Nat.Primrec.const 1).pair (primrec_add.comp
        ((primrec_sub.comp (hf.pair (Nat.Primrec.const 1))).pair
          (primrec_sub.comp ((Nat.Primrec.const 1).pair hf)))))))
  * show not  p n  <->  _
    rw [hfe n]
    simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd]
    constructor <;> intro h <;> omega

/-- Every recursively decidable predicate is **decidable** (the decider is `f n = 1`,
    decidable
equality on `Nat`); useful for choice-free De Morgan. -/
theorem RecDecidable.em {p : Nat  ->  Prop} (hp : RecDecidable p) (n : Nat) : p n  \/
     not  p n := by
  obtain <f, _, hfe> := hp
  by_cases h : f n = 1
  * exact Or.inl ((hfe n).mpr h)
  * exact Or.inr (fun hp => h ((hfe n).mp hp))

/-- **Disjunction.** Recursive decidability is closed under ` \/ `, via choice-free De
    Morgan
`p  \/  q  <->  not (not p  /\  not q)` (the non-constructive direction uses `
    RecDecidable.em`). -/
theorem RecDecidable.or {p q : Nat  ->  Prop} (hp : RecDecidable p) (hq : RecDecidable
     q) :
    RecDecidable (fun n => p n  \/  q n) := by
  refine RecDecidable.of_iff (fun n => ?_) (hp.not.and hq.not).not
```

```
  constructor
  * rintro (h | h) <hnp, hnq>
    * exact hnp h
    * exact hnq h
  * intro h
    rcases hp.em n with hp' | hp'
    * exact Or.inl hp'
    * rcases hq.em n with hq' | hq'
      * exact Or.inr hq'
      * exact absurd <hp', hq'> h

/-- **Every recursively decidable predicate is recursively enumerable.** Use the
    decider as the
relation `q <i, n> := p n` (a reindex of `p` along `unpair.2`, dropping the search
    variable `i`); the
witness `i = 0` makes the projection trivial. -/
theorem RecDecidable.re {p : Nat  ->  Prop} (hp : RecDecidable p) : REPred p := by
  refine <fun t => p t.unpair.2, hp.comp Nat.Primrec.right, fun n => ?_>
  simp only [unpair_pair_snd]
  exact <fun h => <0, h>, fun <_, h> => h>

/-- A recursively decidable binary relation is recursively enumerable. -/
theorem RecDecidable_2.re {r : Nat  ->  Nat  ->  Prop} (hr : RecDecidable_2 r) :
    REPred_2 r :=
  RecDecidable.re hr

/-- Recursive enumerability transfers across a pointwise logical equivalence. -/
theorem REPred.of_iff {p q : Nat  ->  Prop} (h : forall  n, q n  <->  p n) (hp :
    REPred p) : REPred q := by
  obtain <r, hr, hre> := hp
  exact <r, hr, fun n => (h n).trans (hre n)>

/-- **Reindexing.** If `p` is r.e. and `g` is primitive recursive, then `fun n => p (g
    n)` is r.e.
(absorb `g` into the decidable relation along `unpair.2`). -/
theorem REPred.comp {p : Nat  ->  Prop} (hp : REPred p) {g : Nat  ->  Nat} (hg : Nat.
    Primrec g) :
    REPred (fun n => p (g n)) := by
  obtain <q, hq, hqe> := hp
  refine <fun t => q (Nat.pair t.unpair.1 (g t.unpair.2)),
    hq.comp (Nat.Primrec.left.pair (hg.comp Nat.Primrec.right)), fun n => ?_>
  simp only [unpair_pair_fst, unpair_pair_snd]
  exact hqe (g n)

/-- **Conjunction.** Recursive enumerability is closed under ` /\ `: combine the two
    decidable relations
and run the two searches in parallel (pairing the search variables `i, j` into a single
     `w`). -/
theorem REPred.and {p q : Nat  ->  Prop} (hp : REPred p) (hq : REPred q) :
    REPred (fun n => p n  /\  q n) := by
  obtain <a, ha, hae> := hp
  obtain <b, hb, hbe> := hq
  refine <fun u => a (Nat.pair u.unpair.1.unpair.1 u.unpair.2)
       /\  b (Nat.pair u.unpair.1.unpair.2 u.unpair.2),
    (ha.comp ((Nat.Primrec.left.comp Nat.Primrec.left).pair Nat.Primrec.right)).and
      (hb.comp ((Nat.Primrec.right.comp Nat.Primrec.left).pair Nat.Primrec.right)),
```

```
    fun n => ?_>
  simp only [unpair_pair_fst, unpair_pair_snd]
  rw [hae n, hbe n]
  constructor
  * rintro <<i, hi>, <j, hj>>
    exact <Nat.pair i j, by simp only [unpair_pair_fst, unpair_pair_snd]; exact <hi,
    hj>>
  * rintro <w, hw1, hw2>
    exact <<w.unpair.1, hw1>, <w.unpair.2, hw2>>

/-- **Existential projection.** If `p` is r.e. then so is `fun n => exists  i, p <i, n
    >`: fold the new
existential variable `i` into the search variable (pairing it with the decidable
    relation's own
search variable `j`). -/
theorem REPred.proj {p : Nat  ->  Prop} (hp : REPred p) :
    REPred (fun n => exists  i, p (Nat.pair i n)) := by
  obtain <q, hq, hqe> := hp
  refine <fun u => q (Nat.pair u.unpair.1.unpair.2 (Nat.pair u.unpair.1.unpair.1 u.
    unpair.2)),
    hq.comp ((Nat.Primrec.right.comp Nat.Primrec.left).pair
      ((Nat.Primrec.left.comp Nat.Primrec.left).pair Nat.Primrec.right)), fun n => ?_>
  simp only [unpair_pair_fst, unpair_pair_snd]
  constructor
  * rintro <i, hi>
    rw [hqe (Nat.pair i n)] at hi
    obtain <j, hj> := hi
    exact <Nat.pair i j, by simpa only [unpair_pair_fst, unpair_pair_snd] using hj>
  * rintro <w, hw>
    refine <w.unpair.1, ?_>
    rw [hqe (Nat.pair w.unpair.1 n)]
    exact <w.unpair.2, hw>

/-- **Disjunction of r.e. predicates is r.e.** A witness `w` carries a tag `w.1  in
    {0,1}` selecting
which disjunct's search index `w.2` to use; the underlying relation is recursively
    decidable by
`RecDecidable.or`/`.and`/`.natEq`. (Used for the sum-mapping `f + g` of Theorem 7.4.)
    -/
theorem REPred.or {p q : Nat  ->  Prop} (hp : REPred p) (hq : REPred q) :
    REPred (fun n => p n  \/  q n) := by
  obtain <a, ha, hae> := hp
  obtain <b, hb, hbe> := hq
  refine <fun u => (a (Nat.pair u.unpair.1.unpair.2 u.unpair.2)  /\  u.unpair.1.unpair
    .1 = 0)
      \/  (b (Nat.pair u.unpair.1.unpair.2 u.unpair.2)  /\  u.unpair.1.unpair.1 = 1),
     ?_, fun n => ?_>
  * refine RecDecidable.or (RecDecidable.and ?_ ?_) (RecDecidable.and ?_ ?_)
    * exact ha.comp ((Nat.Primrec.right.comp Nat.Primrec.left).pair Nat.Primrec.right)
    * exact RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.left) (Nat.Primrec.
    const 0)
    * exact hb.comp ((Nat.Primrec.right.comp Nat.Primrec.left).pair Nat.Primrec.right)
    * exact RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.left) (Nat.Primrec.
    const 1)
  * show p n  \/  q n  <->  _
    rw [hae n, hbe n]
```

```
    constructor
    · rintro (<i, hi> | <i, hi>)
      · refine <Nat.pair 0 i, Or.inl <?_, ?_>>
        · simpa only [unpair_pair_fst, unpair_pair_snd] using hi
        · simp only [unpair_pair_fst]
      · refine <Nat.pair 1 i, Or.inr <?_, ?_>>
        · simpa only [unpair_pair_fst, unpair_pair_snd] using hi
        · simp only [unpair_pair_fst]
    · rintro <w, hw>
      simp only [unpair_pair_fst, unpair_pair_snd] at hw
      rcases hw with <hi, _> | <hi, _>
      · exact Or.inl <w.unpair.2, hi>
      · exact Or.inr <w.unpair.2, hi>

/-- `a  <=  pair a b` (choice-free). -/
theorem le_pair_left (a b : Nat) : a  <=  Nat.pair a b := by
  unfold Nat.pair
  split <;> omega

/-- The length of a list is bounded by its code; this is the fuel bound that lets a `c
    `-fold iterate
enough times to consume the whole list coded by `c`. -/
theorem encodeList_length_le : forall  l : List Nat, l.length  <=  encodeList l
  | [] => Nat.le_refl 0
  | a :: l => by
    simp only [encodeList, List.length_cons]
    have hrec := encodeList_length_le l
    have hle : encodeList l  <=  Nat.pair a (encodeList l) := le_pair_right a (
    encodeList l)
    omega

/-- Empty remaining code: the fold state is unchanged. -/
theorem foldStep_zero (stp : Nat  ->  Nat) (params acc : Nat) :
    foldStep stp params (Nat.pair 0 acc) = Nat.pair 0 acc := by
  unfold foldStep
  simp only [unpair_pair_fst, unpair_pair_snd, Nat.sub_zero, selectFn_one]

/-- Nonempty remaining code `pair a t + 1`: pop `a`, recurse on `t`, update the
    accumulator. -/
theorem foldStep_pos (stp : Nat  ->  Nat) (params a t acc : Nat) :
    foldStep stp params (Nat.pair (Nat.pair a t + 1) acc)
      = Nat.pair t (stp (Nat.pair a (Nat.pair acc params))) := by
  unfold foldStep
  simp only [unpair_pair_fst, unpair_pair_snd]
  have h1 : 1 - (Nat.pair a t + 1) = 0 := by omega
  have h2 : Nat.pair a t + 1 - 1 = Nat.pair a t := by omega
  rw [h1, h2, selectFn_zero, unpair_pair_fst, unpair_pair_snd]

/-- `Nat.rec` with a counter-independent step is just function iteration (choice-free).
     Needed to
bridge the `Nat.Primrec.prec` form (a `Nat.rec`) with `foldCode`'s `Function.iterate`
    form. -/
theorem rec_const_iterate (f : Nat  ->  Nat) (s : Nat) :
    forall  k : Nat, Nat.rec (motive := fun _ => Nat) s (fun _ ih => f ih) k = f^[k] s
  | 0 => rfl
  | (k + 1) => by
```

```
      rw [Function.iterate_succ_apply']
      exact congrArg f (rec_const_iterate f s k)

/-- Core correctness of the iteration: starting from `pair (encodeList l) acc`, after
    at least
`l.length` steps the accumulator equals the `List.foldl` of the step over `l`. -/
theorem foldStep_iterate (stp : Nat  ->  Nat) (params : Nat) :
    forall  (k : Nat) (l : List Nat) (acc : Nat), l.length  <=  k  ->
      ((foldStep stp params)^[k] (Nat.pair (encodeList l) acc)).unpair.2
        = List.foldl (fun acc x => stp (Nat.pair x (Nat.pair acc params))) acc l := by
  intro k
  induction k with
  | zero =>
    intro l acc hlen
    cases l with
    | nil => simp only [Function.iterate_zero_apply, encodeList, unpair_pair_snd, List
    .foldl_nil]
    | cons a l' => simp only [List.length_cons] at hlen; omega
  | succ k ih =>
    intro l acc hlen
    rw [Function.iterate_succ_apply]
    cases l with
    | nil =>
      rw [show encodeList ([] : List Nat) = 0 from rfl, foldStep_zero]
      exact ih [] acc (Nat.zero_le k)
    | cons a l' =>
      have hlen' : l'.length  <=  k := by simp only [List.length_cons] at hlen; omega
      rw [show encodeList (a :: l') = Nat.pair a (encodeList l') + 1 from rfl,
    foldStep_pos,
        ih l' (stp (Nat.pair a (Nat.pair acc params))) hlen', List.foldl_cons]

/-- **Correctness of `foldCode`.** Folding the code of `l` equals the corresponding `
    List.foldl`. -/
theorem foldCode_eq (stp : Nat  ->  Nat) (params z : Nat) (l : List Nat) :
    foldCode stp params z (encodeList l)
      = List.foldl (fun acc x => stp (Nat.pair x (Nat.pair acc params))) z l := by
  unfold foldCode
  exact foldStep_iterate stp params (encodeList l) l z (encodeList_length_le l)

theorem unpair_left_le (n : Nat) : n.unpair.1  <=  n := by
  have h := le_pair_left n.unpair.1 n.unpair.2
  rwa [pair_unpair] at h

/-- Weak version of mathlib's `Nat.pair_lt_pair_left`. -/
theorem pair_le_pair_left (b : Nat) {a a' : Nat} (h : a  <=  a') : Nat.pair a b  <=
    Nat.pair a' b := by
  rcases eq_or_lt_of_le h with rfl | hlt
  * exact le_refl _
  * exact le_of_lt (Nat.pair_lt_pair_left b hlt)

/-- Weak version of mathlib's `Nat.pair_lt_pair_right`. -/
theorem pair_le_pair_right' (a : Nat) {b b' : Nat} (h : b  <=  b') : Nat.pair a b  <=
     Nat.pair a b' := by
  rcases eq_or_lt_of_le h with rfl | hlt
  * exact le_refl _
  * exact le_of_lt (Nat.pair_lt_pair_right a hlt)
```

```
/-- `Nat.pair` is weakly monotone in both arguments jointly. -/
theorem pair_le_pair {a b a' b' : Nat} (ha : a  <=  a') (hb : b  <=  b') :
    Nat.pair a b  <=  Nat.pair a' b' :=
  le_trans (pair_le_pair_left b ha) (pair_le_pair_right' a' hb)

theorem decodeList_zero : decodeList 0 = [] := by rw [decodeList]

theorem decodeList_succ (c : Nat) :
    decodeList (c + 1) = c.unpair.1 :: decodeList c.unpair.2 := by
  rw [decodeList]

/-- `encodeList  comp  decodeList = id`: every natural is the code of its decoded list.
    -/
theorem encodeList_decodeList (c : Nat) : encodeList (decodeList c) = c := by
  induction c using Nat.strong_induction_on with
  | _ c ih =>
    cases c with
    | zero => simp only [decodeList_zero, encodeList]
    | succ d =>
      rw [decodeList_succ, encodeList, ih d.unpair.2 (Nat.lt_succ_of_le (unpair_snd_le
    d)),
        pair_unpair]

/-- **Correctness of `foldCode` on an arbitrary code.** `foldCode` over any natural `c`
    equals the
`List.foldl` over the list `c` decodes to. -/
theorem foldCode_eq' (stp : Nat  ->  Nat) (params z c : Nat) :
    foldCode stp params z c
      = List.foldl (fun acc x => stp (Nat.pair x (Nat.pair acc params))) z (decodeList
    c) := by
  conv_lhs => rw [ <-  encodeList_decodeList c]
  rw [foldCode_eq]

/-- `foldStep` (with the parameter packed into the state as `pair params state`) is
    primitive
recursive whenever the step `stp` is. This is the workhorse for `primrec_foldCode`. -/
theorem primrec_foldStepPacked {stp : Nat  ->  Nat} (hstp : Nat.Primrec stp) :
    Nat.Primrec (fun w => foldStep stp w.unpair.1 w.unpair.2) := by
  have hparams : Nat.Primrec (fun w => w.unpair.1) := Nat.Primrec.left
  have hstate : Nat.Primrec (fun w => w.unpair.2) := Nat.Primrec.right
  have hrc : Nat.Primrec (fun w => w.unpair.2.unpair.1) := Nat.Primrec.left.comp Nat.
    Primrec.right
  have hacc : Nat.Primrec (fun w => w.unpair.2.unpair.2) := Nat.Primrec.right.comp Nat
    .Primrec.right
  have hcond : Nat.Primrec (fun w => 1 - w.unpair.2.unpair.1) :=
    primrec_sub_2 (Nat.Primrec.const 1) hrc
  have hrcm1 : Nat.Primrec (fun w => w.unpair.2.unpair.1 - 1) :=
    primrec_sub_2 hrc (Nat.Primrec.const 1)
  have hhead : Nat.Primrec (fun w => (w.unpair.2.unpair.1 - 1).unpair.1) :=
    Nat.Primrec.left.comp hrcm1
  have htail : Nat.Primrec (fun w => (w.unpair.2.unpair.1 - 1).unpair.2) :=
    Nat.Primrec.right.comp hrcm1
  have hinner : Nat.Primrec (fun w => Nat.pair w.unpair.2.unpair.2 w.unpair.1) := hacc
    .pair hparams
  have harg : Nat.Primrec
```

```
      (fun w => Nat.pair (w.unpair.2.unpair.1 - 1).unpair.1
        (Nat.pair w.unpair.2.unpair.2 w.unpair.1)) := hhead.pair hinner
  have hstpv : Nat.Primrec
      (fun w => stp (Nat.pair (w.unpair.2.unpair.1 - 1).unpair.1
        (Nat.pair w.unpair.2.unpair.2 w.unpair.1))) := hstp.comp harg
  have helse : Nat.Primrec
      (fun w => Nat.pair (w.unpair.2.unpair.1 - 1).unpair.2
        (stp (Nat.pair (w.unpair.2.unpair.1 - 1).unpair.1
          (Nat.pair w.unpair.2.unpair.2 w.unpair.1)))) := htail.pair hstpv
  exact (primrec_selectFn hcond hstate helse).of_eq fun _ => rfl

/-- **`foldCode` is primitive recursive** in all of its (primitive-recursive) inputs.
    -/
theorem primrec_foldCode {stp : Nat  ->  Nat} (hstp : Nat.Primrec stp)
    {params z c : Nat  ->  Nat} (hp : Nat.Primrec params) (hz : Nat.Primrec z) (hc :
    Nat.Primrec c) :
    Nat.Primrec (fun n => foldCode stp (params n) (z n) (c n)) := by
  have hfoldw : Nat.Primrec (fun w => foldStep stp w.unpair.1 w.unpair.2) :=
    primrec_foldStepPacked hstp
  have hg : Nat.Primrec (fun x => foldStep stp x.unpair.1.unpair.1 x.unpair.2.unpair
    .2) :=
    (hfoldw.comp ((Nat.Primrec.left.comp Nat.Primrec.left).pair
      (Nat.Primrec.right.comp Nat.Primrec.right))).of_eq fun _ => by
        simp only [unpair_pair_fst, unpair_pair_snd]
  have hprec := Nat.Primrec.prec Nat.Primrec.right hg
  refine (Nat.Primrec.right.comp
    (hprec.comp ((hp.pair (hc.pair hz)).pair hc))).of_eq fun n => ?_
  simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd]
  rw [rec_const_iterate]
  rfl

theorem isOne_le_one (v : Nat) : isOne v  <=  1 := by unfold isOne; omega

theorem isOne_of_ne_one {v : Nat} (h : v  !=  1) : isOne v = 0 := by
  unfold isOne
  omega

theorem primrec_isOne : Nat.Primrec isOne :=
  primrec_sub_2 (Nat.Primrec.const 1)
    (primrec_add_2 (primrec_sub_2 primrec_id (Nat.Primrec.const 1))
      (primrec_sub_2 (Nat.Primrec.const 1) primrec_id))

theorem isZero_eq_one_iff (n : Nat) : isZero n = 1  <->  n = 0 := by
  unfold isZero; omega

theorem isZero_le_one (n : Nat) : isZero n  <=  1 := by unfold isZero; omega

theorem primrec_isZero : Nat.Primrec isZero := by
  have hmin : Nat.Primrec (fun n => min n 1) :=
    (primrec_sub_2 primrec_id (primrec_sub_2 primrec_id (Nat.Primrec.const 1))).of_eq
    fun n => by
      rcases n with _ | n
      * rfl
      * show n + 1 - (n + 1 - 1) = min (n + 1) 1
        rw [Nat.min_eq_right (Nat.le_add_left 1 n)]
        omega
```

```
  exact primrec_sub_2 (Nat.Primrec.const 1) hmin

/-- `{0,1}` test for `f n  <=  g n` (truncated subtraction). -/
theorem primrec_le {f g : Nat  ->  Nat} (hf : Nat.Primrec f) (hg : Nat.Primrec g) :
    Nat.Primrec (fun n => isZero (f n - g n)) :=
  primrec_isZero.comp (primrec_sub_2 hf hg)

theorem bForallFn_le_one (g : Nat  ->  Nat) (n N : Nat) : bForallFn g n N  <=  1 := by
  induction N with
  | zero => exact Nat.le_refl 1
  | succ N ih =>
    show selectFn (bForallFn g n N) (isOne (g (Nat.pair N n))) 0  <=  1
    rcases (show bForallFn g n N = 0  \/  bForallFn g n N = 1 by omega) with h | h
    * rw [h, selectFn_zero]; exact Nat.zero_le 1
    * rw [h, selectFn_one]; exact isOne_le_one _

theorem bForallFn_eq_one_iff (g : Nat  ->  Nat) (n N : Nat) :
    bForallFn g n N = 1  <->  forall  i, i < N  ->  g (Nat.pair i n) = 1 := by
  induction N with
  | zero =>
    constructor
    * intro _ i hi; exact absurd hi (Nat.not_lt_zero i)
    * intro _; rfl
  | succ N ih =>
    have hstep : bForallFn g n (N + 1)
        = selectFn (bForallFn g n N) (isOne (g (Nat.pair N n))) 0 := rfl
    have hle := bForallFn_le_one g n N
    rw [hstep]
    rcases (show bForallFn g n N = 0  \/  bForallFn g n N = 1 by omega) with h0 | h1
    * rw [h0, selectFn_zero]
      constructor
      * intro hcontra; exact absurd hcontra (by decide)
      * intro hall
        have hb : bForallFn g n N = 1 := ih.mpr (fun i hi => hall i (Nat.lt_succ_of_lt
     hi))
        rw [h0] at hb; exact hb
    * rw [h1, selectFn_one, isOne_eq_one_iff]
      constructor
      * intro hgN i hi
        rcases (show i < N  \/  i = N by omega) with hlt | heq
        * exact (ih.mp h1) i hlt
        * subst heq; exact hgN
      * intro hall; exact hall N (Nat.lt_succ_self N)

theorem bExistsFn_le_one (g : Nat  ->  Nat) (n N : Nat) : bExistsFn g n N  <=  1 := by
  induction N with
  | zero => simp [bExistsFn]
  | succ N ih =>
    show selectFn (bExistsFn g n N) 1 (isOne (g (Nat.pair N n)))  <=  1
    rcases (show bExistsFn g n N = 0  \/  bExistsFn g n N = 1 by omega) with h | h
    * rw [h, selectFn_zero]
      exact isOne_le_one _
    * rw [h, selectFn_one]

/-- `bExistsFn` only depends on `g` through its values on the search range `{pair i n |
     i < N}`. -/
```

```
theorem bExistsFn_congr {g1 g2 : Nat  ->  Nat} {n N : Nat}
    (h : forall  i < N, g1 (Nat.pair i n) = g2 (Nat.pair i n)) : bExistsFn g1 n N =
    bExistsFn g2 n N := by
  induction N with
  | zero => rfl
  | succ N ih =>
    show selectFn (bExistsFn g1 n N) 1 (isOne (g1 (Nat.pair N n)))
      = selectFn (bExistsFn g2 n N) 1 (isOne (g2 (Nat.pair N n)))
    rw [ih (fun i hi => h i (Nat.lt_succ_of_lt hi)), h N (Nat.lt_succ_self N)]

theorem bExistsFn_eq_one_iff (g : Nat  ->  Nat) (n N : Nat) :
    bExistsFn g n N = 1  <->  exists  i, i < N  /\  g (Nat.pair i n) = 1 := by
  induction N with
  | zero =>
    simp [bExistsFn]
  | succ N ih =>
    have hstep : bExistsFn g n (N + 1)
        = selectFn (bExistsFn g n N) 1 (isOne (g (Nat.pair N n))) := rfl
    rw [hstep]
    have hle := bExistsFn_le_one g n N
    rcases (show bExistsFn g n N = 0  \/  bExistsFn g n N = 1 by omega) with h0 | h1
    * rw [h0, selectFn_zero, isOne_eq_one_iff]
      constructor
      * intro hgN; exact <N, Nat.lt_succ_self N, hgN>
      * intro <i, hi, hg>
        rcases (show i < N  \/  i = N by omega) with hlt | heq
        * have hne : bExistsFn g n N = 1 := ih.mpr <i, hlt, hg>
          omega
        * subst heq; exact hg
    * rw [h1, selectFn_one]
      constructor
      * intro _
        obtain <i, hi, hg> := ih.mp h1
        exact <i, Nat.lt_succ_of_lt hi, hg>
      * intro _; rfl

theorem primrec_bExistsFn {g : Nat  ->  Nat} (hg : Nat.Primrec g) :
    Nat.Primrec (fun t => bExistsFn g t.unpair.1 t.unpair.2) := by
  have hGfn : Nat.Primrec (fun w =>
      selectFn w.unpair.2.unpair.2 1 (isOne (g (Nat.pair w.unpair.2.unpair.1 w.unpair
    .1)))) :=
    primrec_selectFn (Nat.Primrec.right.comp Nat.Primrec.right)
      (Nat.Primrec.const 1)
      (primrec_isOne.comp (hg.comp
        ((Nat.Primrec.left.comp Nat.Primrec.right).pair Nat.Primrec.left)))
  have hprec := Nat.Primrec.prec (Nat.Primrec.const 0) hGfn
  refine (hprec.comp (Nat.Primrec.left.pair Nat.Primrec.right)).of_eq fun t => ?_
  simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd]
  rfl

/-- Parametrized variant of `primrec_bExistsFn`: `g` may also depend on an external `
    param`
(e.g. `fuel`) held fixed throughout the search, packed as `t.unpair.1.unpair.1`. Needed
     once `g`
itself is only jointly (not per-fixed-`param`) primitive recursive. -/
theorem primrec_bExistsFn_param {g : Nat  ->  Nat} (hg : Nat.Primrec g) :
```

```
    Nat.Primrec (fun t => bExistsFn (fun p => g (Nat.pair t.unpair.1.unpair.1 p))
      t.unpair.1.unpair.2 t.unpair.2) := by
  have hGfn : Nat.Primrec (fun w =>
      selectFn w.unpair.2.unpair.2 1
        (isOne (g (Nat.pair w.unpair.1.unpair.1
          (Nat.pair w.unpair.2.unpair.1 w.unpair.1.unpair.2))))) :=
    primrec_selectFn (Nat.Primrec.right.comp Nat.Primrec.right)
      (Nat.Primrec.const 1)
      (primrec_isOne.comp (hg.comp
        (Nat.Primrec.pair (Nat.Primrec.left.comp Nat.Primrec.left)
          (Nat.Primrec.pair (Nat.Primrec.left.comp Nat.Primrec.right)
            (Nat.Primrec.right.comp Nat.Primrec.left)))))
  have hprec := Nat.Primrec.prec (Nat.Primrec.const 0) hGfn
  refine (hprec.comp (Nat.Primrec.left.pair Nat.Primrec.right)).of_eq fun t => ?_
  simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd]
  rfl

/-- `decodeList  comp  encodeList = id` (the round-trip the other way from `
    encodeList_decodeList`). -/
theorem decodeList_encodeList : forall  l : List Nat, decodeList (encodeList l) = l
  | [] => by rw [encodeList, decodeList_zero]
  | a :: l => by
    rw [encodeList, decodeList_succ, unpair_pair_fst, unpair_pair_snd,
    decodeList_encodeList l]

theorem reForallStp_eq (qc : Nat  ->  Nat) (acc x : Nat) :
    reForallStp qc (Nat.pair x (Nat.pair acc 0)) = reForallF qc acc x := by
  unfold reForallStp reForallF
  simp only [unpair_pair_fst, unpair_pair_snd]

theorem reForallChar_eq (qc : Nat  ->  Nat) (w c : Nat) :
    reForallChar qc w c
      = (List.foldl (reForallF qc) (Nat.pair w 1) (decodeList c)).unpair.2 := by
  have hfun : (fun (acc x : Nat) => reForallStp qc (Nat.pair x (Nat.pair acc 0))) =
    reForallF qc := by
    funext acc x; exact reForallStp_eq qc acc x
  unfold reForallChar
  rw [foldCode_eq', hfun]

/-- **Core induction.** Threading witness code `w` through `l`, the final flag is `1`
    iff the start
flag was `1` and, position by position, the `k`-th witness of `decodeList w` satisfies
    `qc`. -/
theorem reForallF_foldl_eq_one_iff (qc : Nat  ->  Nat) :
    forall  (l : List Nat) (w flag : Nat), flag  <=  1  ->
      ((List.foldl (reForallF qc) (Nat.pair w flag) l).unpair.2 = 1  <->
        flag = 1  /\  forall  k, k < l.length  ->
          qc (Nat.pair ((decodeList w).getD k 0) (l.getD k 0)) = 1) := by
  intro l
  induction l with
  | nil =>
    intro w flag _
    simp only [List.foldl_nil, unpair_pair_snd, List.length_nil, Nat.not_lt_zero,
      false_implies, implies_true, and_true]
  | cons x l ih =>
    intro w flag hflag
```

```
    have hi : (decodeList w).getD 0 0 = (w - 1).unpair.1 := by
      rcases w with _ | c
      * have hz : ((0 : Nat) - 1).unpair.1 = 0 := by
          rw [Nat.zero_sub]; exact congrArg Prod.fst Nat.unpair_zero
        rw [decodeList_zero, List.getD_nil, hz]
      * rw [decodeList_succ, List.getD_cons_zero, Nat.add_sub_cancel]
    have htail : forall  k, (decodeList w).getD (k + 1) 0 = (decodeList (w - 1).unpair
    .2).getD k 0 := by
      intro k
      rcases w with _ | c
      * have hz : ((0 : Nat) - 1).unpair.2 = 0 := by
          rw [Nat.zero_sub]; exact congrArg Prod.snd Nat.unpair_zero
        rw [decodeList_zero, hz, decodeList_zero, List.getD_nil, List.getD_nil]
      * rw [decodeList_succ, List.getD_cons_succ, Nat.add_sub_cancel]
    have hstep : List.foldl (reForallF qc) (Nat.pair w flag) (x :: l)
        = List.foldl (reForallF qc) (Nat.pair (w - 1).unpair.2
            (selectFn flag (isOne (qc (Nat.pair (w - 1).unpair.1 x))) 0)) l := by
      rw [List.foldl_cons]
      congr 1
      show reForallF qc (Nat.pair w flag) x = _
      unfold reForallF
      rw [unpair_pair_fst, unpair_pair_snd]
    rw [hstep]
    set flag' := selectFn flag (isOne (qc (Nat.pair (w - 1).unpair.1 x))) 0 with hflag
    'def
    have hflag'le : flag'  <=  1 := by
      rw [hflag'def]
      rcases (show flag = 0  \/  flag = 1 by omega) with h | h
      * rw [h, selectFn_zero]; exact Nat.zero_le 1
      * rw [h, selectFn_one]; exact isOne_le_one _
    rw [ih (w - 1).unpair.2 flag' hflag'le]
    constructor
    * rintro <hf', hrest>
      have hsplit : flag = 1  /\  qc (Nat.pair (w - 1).unpair.1 x) = 1 := by
        rw [hflag'def] at hf'
        rcases (show flag = 0  \/  flag = 1 by omega) with h | h
        * rw [h, selectFn_zero] at hf'; exact absurd hf' (by decide)
        * rw [h, selectFn_one, isOne_eq_one_iff] at hf'; exact <h, hf'>
      refine <hsplit.1, fun k hk => ?_>
      rcases k with _ | k'
      * rw [List.getD_cons_zero, hi]; exact hsplit.2
      * rw [List.getD_cons_succ, htail]
        exact hrest k' (by simp only [List.length_cons] at hk; omega)
    * rintro <hflag1, hall>
      have hhead : qc (Nat.pair (w - 1).unpair.1 x) = 1 := by
        have := hall 0 (by simp only [List.length_cons]; omega)
        rwa [List.getD_cons_zero, hi] at this
      refine <?_, fun k hk => ?_>
      * rw [hflag'def, hflag1, selectFn_one, isOne_eq_one_iff]; exact hhead
      * have := hall (k + 1) (by simp only [List.length_cons]; omega)
        rwa [List.getD_cons_succ, htail] at this

theorem reForallChar_eq_one_iff (qc : Nat  ->  Nat) (w c : Nat) :
    reForallChar qc w c = 1  <->
      forall  k, k < (decodeList c).length  ->
        qc (Nat.pair ((decodeList w).getD k 0) ((decodeList c).getD k 0)) = 1 := by
```

```
  rw [reForallChar_eq, reForallF_foldl_eq_one_iff qc (decodeList c) w 1 (Nat.le_refl
    1)]
  simp only [true_and]

theorem primrec_reForallStp {qc : Nat  ->  Nat} (hqc : Nat.Primrec qc) :
    Nat.Primrec (reForallStp qc) := by
  have hx : Nat.Primrec (fun w : Nat => w.unpair.1) := Nat.Primrec.left
  have hacc : Nat.Primrec (fun w : Nat => w.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.right
  have hrw : Nat.Primrec (fun w : Nat => w.unpair.2.unpair.1.unpair.1) :=
    Nat.Primrec.left.comp hacc
  have hflag : Nat.Primrec (fun w : Nat => w.unpair.2.unpair.1.unpair.2) :=
    Nat.Primrec.right.comp hacc
  have hpm : Nat.Primrec (fun w : Nat => w.unpair.2.unpair.1.unpair.1 - 1) :=
    primrec_sub_2 hrw (Nat.Primrec.const 1)
  have hi : Nat.Primrec (fun w : Nat => (w.unpair.2.unpair.1.unpair.1 - 1).unpair.1)
    :=
    Nat.Primrec.left.comp hpm
  have hrw' : Nat.Primrec (fun w : Nat => (w.unpair.2.unpair.1.unpair.1 - 1).unpair.2)
     :=
    Nat.Primrec.right.comp hpm
  have hcall : Nat.Primrec (fun w : Nat =>
      qc (Nat.pair (w.unpair.2.unpair.1.unpair.1 - 1).unpair.1 w.unpair.1)) :=
    hqc.comp (hi.pair hx)
  have hB : Nat.Primrec (fun w : Nat =>
      selectFn w.unpair.2.unpair.1.unpair.2
        (isOne (qc (Nat.pair (w.unpair.2.unpair.1.unpair.1 - 1).unpair.1 w.unpair.1)))
     0) :=
    primrec_selectFn hflag (primrec_isOne.comp hcall) (Nat.Primrec.const 0)
  exact (hrw'.pair hB).of_eq (fun _ => rfl)

theorem primrec_reForallChar {qc : Nat  ->  Nat} (hqc : Nat.Primrec qc) :
    Nat.Primrec (fun t => reForallChar qc t.unpair.1 t.unpair.2) := by
  have hfold := primrec_foldCode (primrec_reForallStp hqc) (Nat.Primrec.const 0)
    (Nat.Primrec.left.pair (Nat.Primrec.const 1)) Nat.Primrec.right
  exact (Nat.Primrec.right.comp hfold).of_eq (fun _ => rfl)

/-- **Bounded `forall ` over a coded list preserves recursive enumerability.** If `p`
    is r.e. then so is
`fun c => forall  e  in  decodeList c, p e`: the finite tuple of per-entry witnesses is
     packed into a single
search code `w`, and the `{0,1}` flag `reForallChar` makes the body recursively
    decidable. -/
theorem REPred.forall_mem_decodeList {p : Nat  ->  Prop} (hp : REPred p) :
    REPred (fun c => forall  e  in  decodeList c, p e) := by
  obtain <q, hq, hqe> := hp
  obtain <qc, hqcp, hqcs> := hq
  -- per-entry: `p e  <->  exists  j, qc <j, e> = 1`
  have hpe : forall  e, p e  <->  exists  j, qc (Nat.pair j e) = 1 := by
    intro e; rw [hqe e]; exact exists_congr (fun j => hqcs (Nat.pair j e))
  -- membership gives an index whose `getD` recovers the element
  have hmemgetD : forall  (l : List Nat) (e : Nat), e  in  l  ->  exists  k, k < l.
    length  /\  l.getD k 0 = e := by
    intro l
    induction l with
    | nil => intro e he; cases he
```

```
    | cons a l ih =>
      intro e he
      rcases List.mem_cons.mp he with rfl | he'
      * exact <0, by simp only [List.length_cons]; omega, by rw [List.getD_cons_zero]>
      * obtain <k, hk, hek> := ih e he'
        exact <k + 1, by simp only [List.length_cons]; omega, by rw [List.
    getD_cons_succ]; exact hek>
  refine <fun t => reForallChar qc t.unpair.1 t.unpair.2 = 1,
    <fun t => reForallChar qc t.unpair.1 t.unpair.2, primrec_reForallChar hqcp,
      fun _ => Iff.rfl>, fun c => ?_>
  simp only [unpair_pair_fst, unpair_pair_snd]
  constructor
  * intro hall
    -- build a witness list for `decodeList c`
    have hwit : forall  (l : List Nat), (forall  e  in  l, exists  j, qc (Nat.pair j e
    ) = 1)  ->
        exists  iws : List Nat, forall  k, k < l.length  ->
          qc (Nat.pair (iws.getD k 0) (l.getD k 0)) = 1 := by
      intro l
      induction l with
      | nil => intro _; exact <[], fun k hk => absurd hk (Nat.not_lt_zero k)>
      | cons e l ih =>
        intro hh
        obtain <j, hj> := hh e (List.mem_cons.mpr (Or.inl rfl))
        obtain <iws, hiws> := ih (fun e' he' => hh e' (List.mem_cons.mpr (Or.inr he'))
    )
        refine <j :: iws, fun k hk => ?_>
        rcases k with _ | k'
        * rw [List.getD_cons_zero, List.getD_cons_zero]; exact hj
        * rw [List.getD_cons_succ, List.getD_cons_succ]
          exact hiws k' (by simp only [List.length_cons] at hk; omega)
    obtain <iws, hiws> := hwit (decodeList c) (fun e he => (hpe e).mp (hall e he))
    refine <encodeList iws, (reForallChar_eq_one_iff qc _ c).mpr (fun k hk => ?_)>
    rw [decodeList_encodeList]; exact hiws k hk
  * rintro <w, hw>
    rw [reForallChar_eq_one_iff] at hw
    intro e he
    obtain <k, hk, hek> := hmemgetD (decodeList c) e he
    rw [hpe e]
    refine <(decodeList w).getD k 0, ?_>
    have := hw k hk
    rwa [hek] at this

/-- `lowOr x y = (x ||| y) % 2`: both sides depend only on the low bits `x % 2`, `y %
    2`. -/
theorem lowOr_eq_mod (x y : Nat) : lowOr x y = (x ||| y) % 2 := by
  have key : ((x ||| y) % 2 = 1)  <->  (x % 2 = 1  \/  y % 2 = 1) := by
    have hb := Nat.testBit_lor x y 0
    rw [Nat.testBit_zero, Nat.testBit_zero, Nat.testBit_zero] at hb
    rw [ <-  decide_eq_decide, hb, Bool.decide_or]
  have e1 : (x ||| y) % 2 < 2 := Nat.mod_lt _ (by decide)
  unfold lowOr
  -- explicit case split avoids feeding `omega` an ` <-> `/disjunction (which pulls `
    Classical.choice`)
  rcases Nat.mod_two_eq_zero_or_one x with hx | hx <;>
    rcases Nat.mod_two_eq_zero_or_one y with hy | hy <;> rw [hx, hy]
```

```
  * -- x%2=0, y%2=0 : `(x|||y)%2  !=  1`
    have hne : (x ||| y) % 2  !=  1 := fun h => by rcases key.mp h with h' | h' <;>
    omega
    omega
  * -- x%2=0, y%2=1
    rw [key.mpr (Or.inr hy)]
  * -- x%2=1, y%2=0
    rw [key.mpr (Or.inl hx)]
  * -- x%2=1, y%2=1
    rw [key.mpr (Or.inl hx)]

/-- One recursion step for `Nat.lor` on the low bit: `x ||| y = 2 (x/2 ||| y/2) + lowOr
    x y`. -/
theorem lor_low_rec (x y : Nat) : x ||| y = 2 * (x / 2 ||| y / 2) + lowOr x y := by
  have hdiv : (x ||| y) / 2 = x / 2 ||| y / 2 := by
    apply Nat.eq_of_testBit_eq
    intro i
    rw [ <-  Nat.testBit_add_one, Nat.testBit_lor, Nat.testBit_lor, Nat.
    testBit_add_one,
      Nat.testBit_add_one]
  have hmod := lowOr_eq_mod x y
  conv_lhs => rw [ <-  Nat.div_add_mod (x ||| y) 2]
  rw [hdiv, hmod]

/-- **Invariant of the bitwise-OR iteration.** After `k` steps the two running
    arguments are
`a / 2^k`, `b / 2^k`, the weight is `2^k`, and `acc + 2^k * (a/2^k ||| b/2^k) = a ||| b
    `. -/
theorem lorStep_iter_spec (a b : Nat) : forall  k,
    (lorStep^[k] (Nat.pair (Nat.pair a b) (Nat.pair 1 0))).unpair.1.unpair.1 = a / 2 ^
     k  /\
    (lorStep^[k] (Nat.pair (Nat.pair a b) (Nat.pair 1 0))).unpair.1.unpair.2 = b / 2 ^
     k  /\
    (lorStep^[k] (Nat.pair (Nat.pair a b) (Nat.pair 1 0))).unpair.2.unpair.1 = 2 ^ k
    /\
    (lorStep^[k] (Nat.pair (Nat.pair a b) (Nat.pair 1 0))).unpair.2.unpair.2 +
        2 ^ k * (a / 2 ^ k ||| b / 2 ^ k) = a ||| b := by
  intro k
  induction k with
  | zero =>
    simp only [Function.iterate_zero_apply, unpair_pair_fst, unpair_pair_snd, pow_zero
    ,
      Nat.div_one, Nat.one_mul, Nat.zero_add, true_and]
  | succ k ih =>
    obtain <hA, hB, hW, hAcc> := ih
    rw [Function.iterate_succ_apply']
    have hdd : a / 2 ^ (k + 1) = (a / 2 ^ k) / 2 := by rw [Nat.div_div_eq_div_mul,  <-
      pow_succ]
    have hdd' : b / 2 ^ (k + 1) = (b / 2 ^ k) / 2 := by rw [Nat.div_div_eq_div_mul,
    <-  pow_succ]
    -- compute the four projections of one `lorStep`
    have p11 : (lorStep (lorStep^[k] (Nat.pair (Nat.pair a b) (Nat.pair 1 0)))).unpair
    .1.unpair.1
        = (lorStep^[k] (Nat.pair (Nat.pair a b) (Nat.pair 1 0))).unpair.1.unpair.1 / 2
     := by
      unfold lorStep; rw [unpair_pair_fst, unpair_pair_fst]
```

```
    have p12 : (lorStep (lorStep^[k] (Nat.pair (Nat.pair a b) (Nat.pair 1 0)))).unpair
    .1.unpair.2
        = (lorStep^[k] (Nat.pair (Nat.pair a b) (Nat.pair 1 0))).unpair.1.unpair.2 / 2
     := by
      unfold lorStep; rw [unpair_pair_fst, unpair_pair_snd]
    have p21 : (lorStep (lorStep^[k] (Nat.pair (Nat.pair a b) (Nat.pair 1 0)))).unpair
    .2.unpair.1
        = 2 * (lorStep^[k] (Nat.pair (Nat.pair a b) (Nat.pair 1 0))).unpair.2.unpair.1
     := by
      unfold lorStep; rw [unpair_pair_snd, unpair_pair_fst]
    have p22 : (lorStep (lorStep^[k] (Nat.pair (Nat.pair a b) (Nat.pair 1 0)))).unpair
    .2.unpair.2
        = (lorStep^[k] (Nat.pair (Nat.pair a b) (Nat.pair 1 0))).unpair.2.unpair.2
          + (lorStep^[k] (Nat.pair (Nat.pair a b) (Nat.pair 1 0))).unpair.2.unpair.1
            * lowOr (lorStep^[k] (Nat.pair (Nat.pair a b) (Nat.pair 1 0))).unpair.1.
    unpair.1
                (lorStep^[k] (Nat.pair (Nat.pair a b) (Nat.pair 1 0))).unpair.1.unpair
    .2 := by
      unfold lorStep; rw [unpair_pair_snd, unpair_pair_snd]
    refine <?_, ?_, ?_, ?_>
    * rw [p11, hA, Nat.div_div_eq_div_mul,  <-  pow_succ]
    * rw [p12, hB, Nat.div_div_eq_div_mul,  <-  pow_succ]
    * rw [p21, hW,  <-  pow_succ']
    * rw [p22, hA, hB, hW, hdd, hdd', pow_succ,  <-  hAcc, lor_low_rec (a / 2 ^ k) (b
    / 2 ^ k)]
      ring

/-- **Correctness of the iterative bitwise OR.** `myLor a b = a ||| b`. -/
theorem myLor_eq_lor (a b : Nat) : myLor a b = a ||| b := by
  unfold myLor
  obtain <_, _, _, hAcc> := lorStep_iter_spec a b (a + b)


  have ha0 : a / 2 ^ (a + b) = 0 :=
    Nat.div_eq_of_lt (Nat.lt_of_lt_of_le Nat.lt_two_pow_self (Nat.pow_le_pow_right (by
     decide)
      (Nat.le_add_right a b)))
  have hb0 : b / 2 ^ (a + b) = 0 :=
    Nat.div_eq_of_lt (Nat.lt_of_lt_of_le Nat.lt_two_pow_self (Nat.pow_le_pow_right (by
     decide)
      (Nat.le_add_left b a)))
  rw [ha0, hb0] at hAcc
  simpa using hAcc

theorem allListStp_eq (g : Nat  ->  Nat) (acc x p : Nat) :
    allListStp g (Nat.pair x (Nat.pair acc p)) = selectFn acc (isOne (g (Nat.pair x p)
    )) 0 := by
  unfold allListStp; simp only [unpair_pair_fst, unpair_pair_snd]

theorem allListChar_eq (g : Nat  ->  Nat) (p c : Nat) :
    allListChar g p c
      = List.foldl (fun acc x => selectFn acc (isOne (g (Nat.pair x p))) 0) 1 (
    decodeList c) := by
  have hfun : (fun (acc x : Nat) => allListStp g (Nat.pair x (Nat.pair acc p)))
      = (fun acc x => selectFn acc (isOne (g (Nat.pair x p))) 0) := by
    funext acc x; exact allListStp_eq g acc x p
  unfold allListChar; rw [foldCode_eq', hfun]
```

```
/-- Core induction for the AND-fold: starting from `acc  <=  1`, the fold is `1` iff `
    acc = 1` and every
list entry passes the decider `g`. -/
theorem allList_foldl_eq_one_iff (g : Nat  ->  Nat) (p : Nat) :
    forall  (l : List Nat) (acc : Nat), acc  <=  1  ->
      (List.foldl (fun acc x => selectFn acc (isOne (g (Nat.pair x p))) 0) acc l = 1
    <->
        acc = 1  /\  forall  x  in  l, g (Nat.pair x p) = 1) := by
  intro l
  induction l with
  | nil => intro acc _; simp only [List.foldl_nil, List.not_mem_nil, false_implies,
    implies_true,
      and_true]
  | cons x l ih =>
    intro acc hacc
    rw [List.foldl_cons]
    set acc' := selectFn acc (isOne (g (Nat.pair x p))) 0 with hacc'
    have hacc'le : acc'  <=  1 := by
      rw [hacc']
      rcases (show acc = 0  \/  acc = 1 by omega) with h | h
      * rw [h, selectFn_zero]; exact Nat.zero_le 1
      * rw [h, selectFn_one]; exact isOne_le_one _
    rw [ih acc' hacc'le]
    constructor
    * rintro <hacc'1, hrest>
      have hsplit : acc = 1  /\  g (Nat.pair x p) = 1 := by
        rw [hacc'] at hacc'1
        rcases (show acc = 0  \/  acc = 1 by omega) with h | h
        * rw [h, selectFn_zero] at hacc'1; exact absurd hacc'1 (by decide)
        * rw [h, selectFn_one, isOne_eq_one_iff] at hacc'1; exact <h, hacc'1>
      refine <hsplit.1, fun y hy => ?_>
      rcases List.mem_cons.mp hy with rfl | hy'
      * exact hsplit.2
      * exact hrest y hy'
    * rintro <hacc1, hall>
      have hhead : g (Nat.pair x p) = 1 := hall x (List.mem_cons.mpr (Or.inl rfl))
      refine <?_, fun y hy => hall y (List.mem_cons.mpr (Or.inr hy))>
      rw [hacc', hacc1, selectFn_one, isOne_eq_one_iff]; exact hhead

theorem allListChar_eq_one_iff (g : Nat  ->  Nat) (p c : Nat) :
    allListChar g p c = 1  <->  forall  x  in  decodeList c, g (Nat.pair x p) = 1 :=
    by
  rw [allListChar_eq, allList_foldl_eq_one_iff g p (decodeList c) 1 (Nat.le_refl 1)]
  simp only [true_and]

theorem allListChar_le_one (g : Nat  ->  Nat) (p c : Nat) : allListChar g p c  <=  1
    := by
  have h : forall  (l : List Nat) (z : Nat), z  <=  1  ->
      List.foldl (fun acc x => allListStp g (Nat.pair x (Nat.pair acc p))) z l  <=  1
    := by
    intro l
    induction l with
    | nil => intro z hz; simpa using hz
    | cons x xs ih =>
      intro z hz
```

```
      simp only [List.foldl_cons]
      apply ih
      rw [allListStp_eq]
      rcases (show z = 0  \/  z = 1 by omega) with h0 | h1
      * rw [h0, selectFn_zero]; exact Nat.zero_le 1
      * rw [h1, selectFn_one]; exact isOne_le_one _
  unfold allListChar
  rw [foldCode_eq']
  exact h (decodeList c) 1 (le_refl 1)

theorem primrec_allListStp {g : Nat  ->  Nat} (hg : Nat.Primrec g) : Nat.Primrec (
    allListStp g) := by
  have hacc : Nat.Primrec (fun w : Nat => w.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.right
  have hx : Nat.Primrec (fun w : Nat => w.unpair.1) := Nat.Primrec.left
  have hp : Nat.Primrec (fun w : Nat => w.unpair.2.unpair.2) :=
    Nat.Primrec.right.comp Nat.Primrec.right
  have hcall : Nat.Primrec (fun w : Nat => isOne (g (Nat.pair w.unpair.1 w.unpair.2.
    unpair.2))) :=
    primrec_isOne.comp (hg.comp (hx.pair hp))
  exact (primrec_selectFn hacc hcall (Nat.Primrec.const 0)).of_eq fun _ => rfl

theorem isBinDigit_eq_one_iff (n : Nat) : isBinDigit n = 1  <->  n = 0  \/  n = 1 :=
    by
  unfold isBinDigit selectFn isOne
  match n with | 0 | 1 => simp | n + 2 => simp

theorem primrec_isBinDigit : Nat.Primrec isBinDigit := by
  have h01 := primrec_isOne.comp (primrec_sub_2 (Nat.Primrec.const 1) primrec_id)
  have h12 := primrec_isOne.comp (primrec_sub_2 (Nat.Primrec.const 2) primrec_id)
  refine (primrec_selectFn h01 (Nat.Primrec.const 1)
    (primrec_selectFn h12 (Nat.Primrec.const 1) (Nat.Primrec.const 0))).of_eq fun _ =>
     rfl

theorem allBinDigitsChar_eq_one_iff (c : Nat) :
    allBinDigitsChar c = 1  <->  forall  x  in  decodeList c, x = 0  \/  x = 1 := by
  unfold allBinDigitsChar
  rw [allListChar_eq_one_iff]
  simp [isBinDigit_eq_one_iff]

theorem allBinDigitsChar_le_one (c : Nat) : allBinDigitsChar c  <=  1 := by
  unfold allBinDigitsChar; exact allListChar_le_one _ _ _

set_option maxHeartbeats 800000 in
theorem primrec_allBinDigitsChar : Nat.Primrec allBinDigitsChar := by
  unfold allBinDigitsChar
  exact (primrec_foldCode (primrec_allListStp (primrec_isBinDigit.comp Nat.Primrec.
    left))
    (Nat.Primrec.const 0) (Nat.Primrec.const 1) primrec_id).of_eq fun _ => rfl

theorem nthStp_val (i x s r : Nat) :
    nthStp (Nat.pair x (Nat.pair (Nat.pair s r) i))
      = Nat.pair (s + 1) (selectFn (1 - ((s - i) + (i - s))) x r) := by
  unfold nthStp; simp only [unpair_pair_fst, unpair_pair_snd]

theorem nthCode_foldl (i : Nat) : forall  (l : List Nat) (s r : Nat),
```

```
    (List.foldl (fun acc x => nthStp (Nat.pair x (Nat.pair acc i))) (Nat.pair s r) l).
    unpair.2
      = if i < s then r else l.getD (i - s) r := by
  intro l
  induction l with
  | nil => intro s r; simp only [List.foldl_nil, unpair_pair_snd, List.getD_nil];
    split <;> rfl
  | cons x l ih =>
    intro s r
    rw [List.foldl_cons, nthStp_val, ih (s + 1) (selectFn (1 - ((s - i) + (i - s))) x
    r)]
    by_cases h : i = s
    * subst h
      have he : (1 : Nat) - ((i - i) + (i - i)) = 1 := by omega
      rw [he, selectFn_one, ite_eq_right (lt_irrefl i), ite_eq_left (Nat.lt_succ_self
    i), Nat.sub_self,
        List.getD_cons_zero]
    * by_cases h2 : i < s
      * have he : (1 : Nat) - ((s - i) + (i - s)) = 0 := by omega
        rw [he, selectFn_zero, ite_eq_left h2, ite_eq_left (Nat.lt_succ_of_lt h2)]
      * have he : (1 : Nat) - ((s - i) + (i - s)) = 0 := by omega
        rw [he, selectFn_zero, ite_eq_right h2, ite_eq_right (show not  i < s + 1 by
    omega),
          show i - s = (i - (s + 1)) + 1 by omega, List.getD_cons_succ]

theorem nthCode_eq (c i d : Nat) : nthCode c i d = (decodeList c).getD i d := by
  unfold nthCode
  rw [foldCode_eq']
  have h := nthCode_foldl i (decodeList c) 0 d
  rw [ite_eq_right (Nat.not_lt_zero i), Nat.sub_zero] at h
  exact h

theorem primrec_nthStp : Nat.Primrec nthStp := by
  have hcount : Nat.Primrec (fun w : Nat => w.unpair.2.unpair.1.unpair.1) :=
    Nat.Primrec.left.comp (Nat.Primrec.left.comp Nat.Primrec.right)
  have hi : Nat.Primrec (fun w : Nat => w.unpair.2.unpair.2) :=
    Nat.Primrec.right.comp Nat.Primrec.right
  have hx : Nat.Primrec (fun w : Nat => w.unpair.1) := Nat.Primrec.left
  have hres : Nat.Primrec (fun w : Nat => w.unpair.2.unpair.1.unpair.2) :=
    Nat.Primrec.right.comp (Nat.Primrec.left.comp Nat.Primrec.right)
  have hbit : Nat.Primrec (fun w : Nat => 1 - ((w.unpair.2.unpair.1.unpair.1 - w.
    unpair.2.unpair.2)
      + (w.unpair.2.unpair.2 - w.unpair.2.unpair.1.unpair.1))) :=
    primrec_sub_2 (Nat.Primrec.const 1)
      (primrec_add_2 (primrec_sub_2 hcount hi) (primrec_sub_2 hi hcount))
  exact ((Nat.Primrec.succ.comp hcount).pair (primrec_selectFn hbit hx hres)).of_eq
    fun _ => rfl

/-- `nthCode` is primitive recursive in `(c, i, d)` (coded as `pair c (pair i d)`). -/
theorem primrec_nthCode : Nat.Primrec
    (fun t => nthCode t.unpair.1 t.unpair.2.unpair.1 t.unpair.2.unpair.2) := by
  have hfold := primrec_foldCode primrec_nthStp (Nat.Primrec.left.comp Nat.Primrec.
    right)
    ((Nat.Primrec.const 0).pair (Nat.Primrec.right.comp Nat.Primrec.right)) Nat.
    Primrec.left
  exact (Nat.Primrec.right.comp hfold).of_eq fun _ => rfl
```

```
theorem tabStep_val (g : Nat  ->  Nat) (a y IH : Nat) :
    tabStep g (Nat.pair a (Nat.pair y IH))
      = Nat.pair (g (Nat.pair (a.unpair.1 - 1 - y) a.unpair.2)) IH + 1 := by
  unfold tabStep; simp only [unpair_pair_fst, unpair_pair_snd]

/-- After `K  <=  B` steps the accumulated code decodes to the indices `B-K, ..., B-1`.
    -/
theorem decodeList_tabCode_aux (g : Nat  ->  Nat) (B p : Nat) :
    forall  K, K  <=  B  ->  decodeList (Nat.rec (motive := fun _ => Nat) 0
        (fun y IH => tabStep g (Nat.pair (Nat.pair B p) (Nat.pair y IH))) K)
      = (List.range' (B - K) K).map (fun j => g (Nat.pair j p)) := by
  intro K
  induction K with
  | zero => intro _; simp [decodeList_zero]
  | succ K ih =>
    intro hK
    have hKB : K  <=  B := Nat.le_of_succ_le hK
    show decodeList (tabStep g (Nat.pair (Nat.pair B p) (Nat.pair K
      (Nat.rec (motive := fun _ => Nat) 0
        (fun y IH => tabStep g (Nat.pair (Nat.pair B p) (Nat.pair y IH))) K)))) = _
    rw [tabStep_val]
    simp only [unpair_pair_fst, unpair_pair_snd]
    rw [decodeList_succ, unpair_pair_fst, unpair_pair_snd, ih hKB,
      show B - (K + 1) = B - 1 - K by omega, List.range'_succ, List.map_cons,
      show B - 1 - K + 1 = B - K by omega]

theorem decodeList_tabCode (g : Nat  ->  Nat) (B p : Nat) :
    decodeList (tabCode g (Nat.pair B p) B) = (List.range B).map (fun j => g (Nat.pair
     j p)) := by
  have h := decodeList_tabCode_aux g B p B (le_refl B)
  rw [Nat.sub_self,  <-  List.range_eq_range'] at h
  exact h

/-- Choice-free `getD`-as-`getElem` in range. -/
theorem getD_eq_getElem_cf {b : Type*} (l : List b) (d : b) :
    forall  {n : Nat} (h : n < l.length), l.getD n d = l[n] := by
  induction l with
  | nil => intro n h; exact absurd h (Nat.not_lt_zero n)
  | cons a t ih =>
    intro n h
    cases n with
    | zero => rw [List.getD_cons_zero, List.getElem_cons_zero]
    | succ m => rw [List.getD_cons_succ, List.getElem_cons_succ]; exact ih _

/-- Choice-free default-out-of-range. -/
theorem getD_eq_default_cf {b : Type*} (l : List b) (d : b) :
    forall  {n : Nat}, l.length  <=  n  ->  l.getD n d = d := by
  induction l with
  | nil => intro n _; exact List.getD_nil
  | cons a t ih =>
    intro n h
    cases n with
    | zero => exact absurd h (by rw [List.length_cons]; exact Nat.not_succ_le_zero _)
    | succ m => rw [List.getD_cons_succ]; exact ih (by rw [List.length_cons] at h;
    omega)
```

```
/-- Choice-free `getD` of a mapped `range`. -/
theorem getD_map_range_cf {b : Type*} (h : Nat  ->  b) (d : b) {B i : Nat} (hi : i < B
    ) :
     ((List.range B).map h).getD i d = h i := by
  have hlen : ((List.range B).map h).length = B := by rw [List.length_map, List.
    length_range]
  rw [getD_eq_getElem_cf _ d (by rw [hlen]; exact hi), List.getElem_map, List.
    getElem_range]

theorem tabCode_nth_lt {g : Nat  ->  Nat} {B p i : Nat} (hi : i < B) (d : Nat) :
    nthCode (tabCode g (Nat.pair B p) B) i d = g (Nat.pair i p) := by
  rw [nthCode_eq, decodeList_tabCode, getD_map_range_cf]; exact hi

theorem tabCode_nth_ge {g : Nat  ->  Nat} {B p i : Nat} (hi : B  <=  i) (d : Nat) :
    nthCode (tabCode g (Nat.pair B p) B) i d = d := by
  rw [nthCode_eq, decodeList_tabCode, getD_eq_default_cf]
  rw [List.length_map, List.length_range]; exact hi

theorem primrec_tabStep {g : Nat  ->  Nat} (hg : Nat.Primrec g) : Nat.Primrec (tabStep
     g) := by
  have hB : Nat.Primrec (fun w : Nat => w.unpair.1.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.left
  have hp : Nat.Primrec (fun w : Nat => w.unpair.1.unpair.2) :=
    Nat.Primrec.right.comp Nat.Primrec.left
  have hk : Nat.Primrec (fun w : Nat => w.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.right
  have hih : Nat.Primrec (fun w : Nat => w.unpair.2.unpair.2) :=
    Nat.Primrec.right.comp Nat.Primrec.right
  have hidx : Nat.Primrec (fun w : Nat => w.unpair.1.unpair.1 - 1 - w.unpair.2.unpair
    .1) :=
    primrec_sub_2 (primrec_sub_2 hB (Nat.Primrec.const 1)) hk
  exact (Nat.Primrec.succ.comp ((hg.comp (hidx.pair hp)).pair hih)).of_eq fun _ => rfl

/-- `tabCode g * *` is primitive recursive in `(a, B)` (coded `pair a B`), whenever `g`
     is. -/
theorem primrec_tabCode {g : Nat  ->  Nat} (hg : Nat.Primrec g) :
    Nat.Primrec (fun t => tabCode g t.unpair.1 t.unpair.2) :=
  (Nat.Primrec.prec (Nat.Primrec.const 0) (primrec_tabStep hg)).of_eq fun t => by
    simp only [Nat.unpaired]; rfl

/-- Tag-dispatch on `e.unpair.1  in  {0,1,2,3}` (else `fdef e`). -/
theorem primrec_tagCase4 {f0 f1 f2 f3 fdef : Nat  ->  Nat}
    (hf0 : Nat.Primrec f0) (hf1 : Nat.Primrec f1) (hf2 : Nat.Primrec f2) (hf3 : Nat.
    Primrec f3)
    (hfdef : Nat.Primrec fdef) :
    Nat.Primrec (fun e =>
      let t := e.unpair.1
      selectFn (isOne (1 - t)) (f0 e)
        (selectFn (isOne (2 - t)) (f1 e)
          (selectFn (isOne (3 - t)) (f2 e)
            (selectFn (isOne (4 - t)) (f3 e) (fdef e))))) := by
  have ht : Nat.Primrec (fun e => e.unpair.1) := Nat.Primrec.left
  have h01 : Nat.Primrec (fun e => isOne (1 - e.unpair.1)) :=
    primrec_isOne.comp (primrec_sub_2 (Nat.Primrec.const 1) ht)
  have h12 : Nat.Primrec (fun e => isOne (2 - e.unpair.1)) :=
```

```
    primrec_isOne.comp (primrec_sub_2 (Nat.Primrec.const 2) ht)
  have h23 : Nat.Primrec (fun e => isOne (3 - e.unpair.1)) :=
    primrec_isOne.comp (primrec_sub_2 (Nat.Primrec.const 3) ht)
  have h34 : Nat.Primrec (fun e => isOne (4 - e.unpair.1)) :=
    primrec_isOne.comp (primrec_sub_2 (Nat.Primrec.const 4) ht)
  refine (primrec_selectFn h01 hf0
    (primrec_selectFn h12 hf1
      (primrec_selectFn h23 hf2
        (primrec_selectFn h34 hf3 hfdef)))).of_eq fun e => rfl

theorem mulBit_eq_one_iff (a b : Nat) : mulBit a b = 1  <->  a = 1  /\  b = 1 := by
  simp [mulBit]

theorem mulBit_le_one {a b : Nat} (ha : a  <=  1) (hb : b  <=  1) : mulBit a b  <=  1
    := by
  unfold mulBit
  rcases (show a = 0  \/  a = 1 by omega) with rfl | rfl <;>
    rcases (show b = 0  \/  b = 1 by omega) with rfl | rfl <;> simp

theorem primrec_mulBit : Nat.Primrec (fun t => mulBit t.unpair.1 t.unpair.2) :=
  primrec_mul_2 Nat.Primrec.left Nat.Primrec.right

/-- Tag-dispatch form of `decodeFuelOkCharBody` (**7.22i(b)1(c)**). -/
theorem decodeFuelOkCharBody_eq (prev : Nat  ->  Nat) (c : Nat) :
    decodeFuelOkCharBody prev c =
      match c.unpair.1 with
      | 0 => selectFn (isOne (1 - c.unpair.2)) 1 0
      | 1 => allBinDigitsChar c.unpair.2
      | 2 | 3 => mulBit (prev c.unpair.2.unpair.1) (prev c.unpair.2.unpair.2)
      | _ => 0 := by
  unfold decodeFuelOkCharBody mulBit
  match tag : c.unpair.1 with
  | 0 =>
    have h10 : (1 - 0 : Nat) = 1 := rfl
    simp only [h10, isOne_one, selectFn_one]
  | 1 =>
    have h01 : (1 - 1 : Nat) = 0 := rfl
    have h12 : (2 - 1 : Nat) = 1 := rfl
    simp only [h01, h12, isOne_zero, isOne_one, selectFn_zero, selectFn_one]
  | 2 =>
    have h01 : (1 - 2 : Nat) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h12 : (2 - 2 : Nat) = 0 := rfl
    have h23 : (3 - 2 : Nat) = 1 := rfl
    simp only [h01, h23, isOne_zero, isOne_one, selectFn_zero, selectFn_one]
  | 3 =>
    have h01 : (1 - 3 : Nat) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h12 : (2 - 3 : Nat) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h23 : (3 - 3 : Nat) = 0 := rfl
    have h34 : (4 - 3 : Nat) = 1 := rfl
    simp only [h01, h34, isOne_zero, isOne_one, selectFn_zero, selectFn_one]
  | t + 4 =>
    have hne1 : isOne (1 - (t + 4)) = 0 := isOne_of_ne_one (by omega)
    have hne2 : isOne (2 - (t + 4)) = 0 := isOne_of_ne_one (by omega)
    have hne3 : isOne (3 - (t + 4)) = 0 := isOne_of_ne_one (by omega)
    have hne4 : isOne (4 - (t + 4)) = 0 := isOne_of_ne_one (by omega)
    simp only [hne1, hne2, hne3, hne4, selectFn_zero]
```

```
theorem primrec_decodeFuelOkCharBody {prev : Nat  ->  Nat} (hprev : Nat.Primrec prev)
    :
    Nat.Primrec (fun c => decodeFuelOkCharBody prev c) := by
  unfold decodeFuelOkCharBody
  have hleft : Nat.Primrec (fun c => prev c.unpair.2.unpair.1) :=
    (hprev.comp (Nat.Primrec.left.comp Nat.Primrec.right)).of_eq fun c => by
      simp only
  have hright : Nat.Primrec (fun c => prev c.unpair.2.unpair.2) :=
    (hprev.comp (Nat.Primrec.right.comp Nat.Primrec.right)).of_eq fun c => by
      simp only
  have hsub : Nat.Primrec (fun c => mulBit (prev c.unpair.2.unpair.1) (prev c.unpair
    .2.unpair.2)) :=
    primrec_mul_2 hleft hright
  have hf0 : Nat.Primrec (fun c => selectFn (isOne (1 - c.unpair.2)) 1 0) :=
    primrec_selectFn
      (primrec_isOne.comp (primrec_sub_2 (Nat.Primrec.const 1) Nat.Primrec.right))
      (Nat.Primrec.const 1) (Nat.Primrec.const 0)
  have hf1 : Nat.Primrec (fun c => allBinDigitsChar c.unpair.2) :=
    primrec_allBinDigitsChar.comp Nat.Primrec.right
  exact primrec_tagCase4 hf0 hf1 hsub hsub (Nat.Primrec.const 0)

theorem decodeFuelOkCharBody_local (f g : Nat  ->  Nat) (c : Nat) (h : forall  c'  <=
     c, f c' = g c') :
    decodeFuelOkCharBody f c = decodeFuelOkCharBody g c := by
  rw [decodeFuelOkCharBody_eq, decodeFuelOkCharBody_eq]
  have h1 : c.unpair.2.unpair.1  <=  c := le_trans (unpair_left_le c.unpair.2) (
    unpair_snd_le c)
  have h2 : c.unpair.2.unpair.2  <=  c := le_trans (unpair_snd_le c.unpair.2) (
    unpair_snd_le c)
  rw [h _ h1, h _ h2]

theorem listLenStp_eq (acc x p : Nat) :
    listLenStp (Nat.pair x (Nat.pair acc p)) = acc + 1 := by
  unfold listLenStp; simp only [unpair_pair_fst, unpair_pair_snd]

theorem listLenChar_eq (c : Nat) : listLenChar c = (decodeList c).length := by
  have hfun : (fun acc x => listLenStp (Nat.pair x (Nat.pair acc 0))) = fun acc _ =>
    acc + 1 := by
    funext acc x; simp [listLenStp_eq]
  have hfold : forall  (k : Nat) (l : List Nat),
      List.foldl (fun acc _ => acc + 1) k l = k + l.length := by
    intro k l
    induction l generalizing k with
    | nil => simp
    | cons _ xs ih => simp [List.foldl_cons, ih, List.length_cons]; omega
  unfold listLenChar
  rw [foldCode_eq', hfun]
  simp [hfold]

theorem primrec_listLenStp : Nat.Primrec listLenStp :=
  primrec_add_2 (Nat.Primrec.left.comp Nat.Primrec.right) (Nat.Primrec.const 1)

theorem primrec_listLenChar : Nat.Primrec listLenChar := by
  unfold listLenChar
  exact (primrec_foldCode primrec_listLenStp (Nat.Primrec.const 0) (Nat.Primrec.const
```

```
    0)
    primrec_id).of_eq fun _ => rfl

theorem natEqChar_le_one (a b : Nat) : natEqChar a b  <=  1 := by unfold natEqChar;
    omega

theorem natEqChar_eq_zero_or_one (a b : Nat) : natEqChar a b = 0  \/  natEqChar a b =
    1 := by
  unfold natEqChar; omega

theorem primrec_natEqChar : Nat.Primrec (fun t => natEqChar t.unpair.1 t.unpair.2) :=
  (primrec_sub_2 (Nat.Primrec.const 1)
    (primrec_add_2 (primrec_sub_2 Nat.Primrec.left Nat.Primrec.right)
      (primrec_sub_2 Nat.Primrec.right Nat.Primrec.left))).of_eq fun _ => rfl

theorem listEqStp_acc (flag rem x : Nat) :
    listEqStp (Nat.pair x (Nat.pair (Nat.pair flag rem) 0)) =
      selectFn (isZero rem) (Nat.pair 0 0)
        (Nat.pair (selectFn flag (natEqChar x ((rem - 1).unpair.1)) 0) ((rem - 1).
    unpair.2)) := by
  unfold listEqStp listEqStpNonzero
  simp only [unpair_pair_fst, unpair_pair_snd]

theorem decodeList_eq_nil_iff (c : Nat) : decodeList c = []  <->  c = 0 := by
  constructor
  * intro h
    have := congrArg encodeList h
    rwa [encodeList, encodeList_decodeList] at this
  * intro hc; simp [hc, decodeList_zero]



private theorem selectFn_eq_one_iff_zero {a b : Nat} (ha : a = 0  \/  a = 1) :
    selectFn a b 0 = 1  <->  a = 1  /\  b = 1 := by
  rcases ha with ha | ha <;> simp [selectFn, ha]

private theorem listEqStep_zero (flag x : Nat) :
    listEqStep (Nat.pair flag 0) x = Nat.pair 0 0 := by
  simp [listEqStep, listEqStp_acc, isZero, min]

private theorem listEqStep_succ (flag x m : Nat) :
    listEqStep (Nat.pair flag (m + 1)) x =
      Nat.pair (selectFn flag (natEqChar x m.unpair.1) 0) m.unpair.2 := by
  unfold listEqStep listEqStp listEqStpNonzero
  simp only [unpair_pair_fst, unpair_pair_snd]
  have hzero : isZero (m + 1) = 0 := by
    unfold isZero; simp
  rw [hzero, selectFn_zero, Nat.add_sub_cancel]

private theorem listEq_foldl_zero (flag x : Nat) (xs : List Nat) :
    (List.foldl listEqStep (Nat.pair flag 0) (x :: xs)).unpair.1 = 0 := by
  rw [List.foldl_cons, listEqStep_zero]
  induction xs with
  | nil => simp [List.foldl_nil]
  | cons y ys ih =>
    rw [List.foldl_cons, listEqStep_zero, ih]
```

```
private theorem listEq_foldl_end_iff (l1 : List Nat) (flag rem : Nat) (hflag : flag =
    0  \/  flag = 1) :
    (let r := List.foldl listEqStep (Nat.pair flag rem) l1
     r.unpair.1 = 1  /\  isZero r.unpair.2 = 1)  <->
      flag = 1  /\  l1 = decodeList rem := by
  induction l1 generalizing flag rem with
  | nil =>
    simp only [List.foldl_nil, unpair_pair_fst, unpair_pair_snd]
    constructor
    * intro <hflag', hrem0>
      have hrem0' : rem = 0 := (isZero_eq_one_iff rem).1 hrem0
      exact <hflag', ((decodeList_eq_nil_iff rem).2 hrem0').symm>
    * intro <hflag', heq>
      exact <hflag', (isZero_eq_one_iff rem).2 ((decodeList_eq_nil_iff rem).1 heq.symm
    )>
  | cons x xs ih =>
    by_cases hrem : rem = 0
    * subst hrem
      simp only [decodeList_zero]
      constructor
      * intro <h1, _>
        have h0 : (let r := List.foldl listEqStep (Nat.pair flag 0) (x :: xs); r.
    unpair.1) = 0 := by
          show (List.foldl listEqStep (Nat.pair flag 0) (x :: xs)).unpair.1 = 0
          exact listEq_foldl_zero flag x xs
        rw [h0] at h1
        exact absurd h1 (by decide)
      * intro h
        cases h.2
    * rcases rem with _ | m
      * omega
      * set acc' := selectFn flag (natEqChar x m.unpair.1) 0 with hacc'
        have hacc01 : acc' = 0  \/  acc' = 1 := by
          rcases hflag with h | h
          * left; rw [hacc', h, selectFn_zero]
          * have hz := natEqChar_eq_zero_or_one x m.unpair.1
            rw [hacc', h, selectFn_one]
            rcases hz with hz0 | hz1
            * exact Or.inl hz0
            * exact Or.inr hz1
        rw [List.foldl_cons, listEqStep_succ, ih acc' m.unpair.2 hacc01,
    decodeList_succ]
        constructor
        * intro <hacc'1, hxs>
          have hsel : selectFn flag (natEqChar x m.unpair.1) 0 = 1 := hacc'.trans hacc
    '1
          rcases (selectFn_eq_one_iff_zero hflag).1 hsel with <hflag1, hxeq>
          rw [natEqChar_eq_one_iff] at hxeq
          exact <hflag1, by rw [hxeq, hxs]>
        * intro <hflag1, heq>
          rcases List.cons.inj heq with <hx, hxs>
          refine <hacc'.trans ((selectFn_eq_one_iff_zero hflag).2 <hflag1, by
            rw [natEqChar_eq_one_iff, hx]>), hxs>

theorem listEqChar_eq_one_iff (c1 c2 : Nat) :
    listEqChar c1 c2 = 1  <->  decodeList c1 = decodeList c2 := by
```

```
  unfold listEqChar
  rw [foldCode_eq', show (fun acc x => listEqStp (Nat.pair x (Nat.pair acc 0))) =
    listEqStep from rfl,
    mulBit_eq_one_iff]
  exact (listEq_foldl_end_iff (decodeList c1) 1 c2 (Or.inr rfl)).trans (by simp)

private theorem listEq_foldl_flag_le_one : forall  (l : List Nat) (z : Nat), z.unpair
    .1  <=  1  ->
    (List.foldl listEqStep z l).unpair.1  <=  1 := by
  intro l
  induction l with
  | nil => intro z hz; exact hz
  | cons x xs ih =>
    intro z hz
    simp only [List.foldl_cons]
    apply ih
    show (listEqStp (Nat.pair x (Nat.pair z 0))).unpair.1  <=  1
    have hkey := listEqStp_acc z.unpair.1 z.unpair.2 x
    rw [Nat.pair_unpair] at hkey
    rw [hkey]
    rcases (show isZero z.unpair.2 = 0  \/  isZero z.unpair.2 = 1 from by
      have := isZero_le_one z.unpair.2; omega) with h0 | h1
    * rw [h0, selectFn_zero, unpair_pair_fst]
      rcases (show z.unpair.1 = 0  \/  z.unpair.1 = 1 from by omega) with hf0 | hf1
      * rw [hf0, selectFn_zero]; exact Nat.zero_le 1
      * rw [hf1, selectFn_one]
        have := natEqChar_le_one x (z.unpair.2 - 1).unpair.1
        omega
    * rw [h1, selectFn_one, unpair_pair_fst]; exact Nat.zero_le 1

theorem listEqChar_le_one (c1 c2 : Nat) : listEqChar c1 c2  <=  1 := by
  unfold listEqChar
  rw [foldCode_eq', show (fun acc x => listEqStp (Nat.pair x (Nat.pair acc 0))) =
    listEqStep from rfl]
  exact mulBit_le_one
    (listEq_foldl_flag_le_one (decodeList c1) (Nat.pair 1 c2) (by simp))
    (isZero_le_one _)

set_option maxHeartbeats 800000 in
theorem primrec_listEqStpNonzero : Nat.Primrec listEqStpNonzero := by
  have hflag := Nat.Primrec.left.comp (Nat.Primrec.left.comp Nat.Primrec.right)
  have hrem := Nat.Primrec.right.comp (Nat.Primrec.left.comp Nat.Primrec.right)
  have hhead := Nat.Primrec.left
  have hrem1 := primrec_sub_2 hrem (Nat.Primrec.const 1)
  have hcmp := primrec_natEqChar.comp (hhead.pair (Nat.Primrec.left.comp hrem1))
  have htail := Nat.Primrec.right.comp hrem1
  exact (Nat.Primrec.pair (primrec_selectFn hflag hcmp (Nat.Primrec.const 0)) htail).
    of_eq fun w => by
    simp [listEqStpNonzero, selectFn]

set_option maxHeartbeats 800000 in
theorem primrec_listEqStp : Nat.Primrec listEqStp := by
  have hrem := Nat.Primrec.right.comp (Nat.Primrec.left.comp Nat.Primrec.right)
  have hzero := Nat.Primrec.pair (Nat.Primrec.const 0) (Nat.Primrec.const 0)
  exact (primrec_selectFn (primrec_isZero.comp hrem) hzero primrec_listEqStpNonzero).
    of_eq fun w => by
```

```
    simp [listEqStp, selectFn, isZero, min]

theorem primrec_listEqChar : Nat.Primrec (fun t => listEqChar t.unpair.1 t.unpair.2)
    := by
  have hfold := primrec_foldCode primrec_listEqStp (Nat.Primrec.const 0)
    (Nat.Primrec.pair (Nat.Primrec.const 1) Nat.Primrec.right) Nat.Primrec.left
  have hflag : Nat.Primrec (fun t =>
      (foldCode listEqStp 0 (Nat.pair 1 t.unpair.2) t.unpair.1).unpair.1) :=
    Nat.Primrec.left.comp hfold
  have hrem : Nat.Primrec (fun t =>
      (foldCode listEqStp 0 (Nat.pair 1 t.unpair.2) t.unpair.1).unpair.2) :=
    Nat.Primrec.right.comp hfold
  exact (primrec_mulBit.comp (hflag.pair (primrec_isZero.comp hrem))).of_eq fun t =>
    by
    simp [listEqChar]

/-- Two lists agree when every `getD` index does. (Choice-free: `by_contra`/`Classical.
    choice` is
avoided by case-splitting on `Nat.lt_trichotomy` -- a decidable fact -- instead.) -/
theorem list_eq_of_getD {l1 l2 : List Nat} (h : forall  i d, l1.getD i d = l2.getD i d
    ) : l1 = l2 := by
  have hlen : l1.length = l2.length := by
    rcases Nat.lt_trichotomy l1.length l2.length with hlt | heq | hgt
    * have h1 := h l1.length 1
      have h2 := h l1.length 2
      rw [getD_eq_default_cf l1 _ (Nat.le_refl _), getD_eq_getElem_cf l2 _ (by omega)]
     at h1 h2
      omega
    * exact heq
    * have h1 := h l2.length 1
      have h2 := h l2.length 2
      rw [getD_eq_getElem_cf l1 _ (by omega), getD_eq_default_cf l2 _ (Nat.le_refl _)]
     at h1 h2
      omega
  apply List.ext_getElem hlen
  intro i hi _
  have hi' : i < l2.length := by rw [ <-  hlen]; exact hi
  rw [ <-  getD_eq_getElem_cf l1 0 hi,  <-  getD_eq_getElem_cf l2 0 hi', h i 0]

theorem getD_take_cf (l : List Nat) (n i d : Nat) (hi : i < (l.take n).length) :
    (l.take n).getD i d = l.getD i d := by
  have hlen : i < l.length := by
    rw [List.length_take] at hi
    exact Nat.lt_of_lt_of_le hi (Nat.min_le_right n l.length)
  rw [getD_eq_getElem_cf (l.take n) d hi, getD_eq_getElem_cf l d hlen, List.
    getElem_take]

theorem getD_drop_cf (l : List Nat) (n i d : Nat) (hi : i < (l.drop n).length) :
    (l.drop n).getD i d = l.getD (i + n) d := by
  induction n generalizing l i d with
  | zero =>
    simp [List.drop_zero] at hi  |-
  | succ n ih =>
    cases l with
    | nil => simp [List.drop, List.length_nil] at hi
    | cons a tl =>
```

```
      cases i with
      | zero =>
        simp [List.drop]
      | succ i =>
        simp only [List.drop]
        exact ih tl (i + 1) d (by simpa [List.length_drop] using hi)

private theorem isZero_succ_sub_len1 (i len1 : Nat) :
    isZero ((i + 1) - len1) = 1  <->  i < len1 := by
  unfold isZero
  constructor
  * intro h
    by_contra hge
    have hpos : 0 < (i + 1) - len1 := Nat.sub_pos_of_lt (Nat.lt_succ_of_le (Nat.
    le_of_not_gt hge))
    have hmin : min ((i + 1) - len1) 1 = 1 := Nat.min_eq_right (Nat.succ_le_iff.mpr
    hpos)
    omega
  * intro hlt
    have : (i + 1) - len1 = 0 := Nat.sub_eq_zero_iff_le.mpr (Nat.succ_le_iff.mpr hlt)
    simp [this, min]

private theorem isZero_succ_sub_len1_zero (i len1 : Nat) (hge : len1  <=  i) :
    isZero ((i + 1) - len1) = 0 := by
  unfold isZero
  have hpos : 0 < (i + 1) - len1 := Nat.sub_pos_of_lt (Nat.lt_succ_of_le hge)
  have hmin : min ((i + 1) - len1) 1 = 1 := Nat.min_eq_right (Nat.succ_le_iff.mpr hpos
    )
  simp [hmin]

theorem appendListTabFn_eq (c1 c2 len1 i : Nat) :
    appendListTabFn (Nat.pair i (Nat.pair c1 (Nat.pair c2 len1))) =
      if i < len1 then
        (decodeList c1).getD i 0
      else
        (decodeList c2).getD (i - len1) 0 := by
  unfold appendListTabFn
  simp only [unpair_pair_fst, unpair_pair_snd]
  by_cases hlt : i < len1
  * rw [ite_eq_left hlt, (isZero_succ_sub_len1 i len1).2 hlt, selectFn_one, nthCode_eq
    ]
  * rw [ite_eq_right hlt, isZero_succ_sub_len1_zero i len1 (Nat.le_of_not_gt hlt),
    selectFn_zero,
      nthCode_eq]

theorem takeCode_eq (n c : Nat) :
    decodeList (takeCode n c) = (decodeList c).take n := by
  unfold takeCode
  rw [decodeList_tabCode, listLenChar_eq c]
  apply list_eq_of_getD
  intro i d
  by_cases hi : i < Nat.min n (decodeList c).length
  * have hic : i < (decodeList c).length := Nat.lt_of_lt_of_le hi (Nat.min_le_right n
    _)
    rw [getD_map_range_cf (fun j => takeListTabFn (Nat.pair j (Nat.pair n c))) d hi,
      show takeListTabFn (Nat.pair i (Nat.pair n c)) = (decodeList c).getD i d by
```

```
        have hc : (Nat.pair i (Nat.pair n c)).unpair.2.unpair.2 = c := by simp
        have hi' : (Nat.pair i (Nat.pair n c)).unpair.1 = i := by simp
        rw [takeListTabFn, hc, hi', nthCode_eq,
          show (decodeList c).getD i 0 = (decodeList c).getD i d from by
            rw [getD_eq_getElem_cf _ 0 hic, getD_eq_getElem_cf _ d hic]],
      getD_take_cf (decodeList c) n i d (by rwa [List.length_take])]
  * have hge : Nat.min n (decodeList c).length  <=  i := Nat.le_of_not_gt hi
    rw [getD_eq_default_cf _ d (by rw [List.length_map, List.length_range]; exact hge)
    ,
      getD_eq_default_cf _ d (by rw [List.length_take]; exact hge)]

theorem dropCode_eq (n c : Nat) :
    decodeList (dropCode n c) = (decodeList c).drop n := by
  unfold dropCode
  rw [decodeList_tabCode, listLenChar_eq c]
  apply list_eq_of_getD
  intro i d
  by_cases hi : i < (decodeList c).length - n
  * have hic : i + n < (decodeList c).length := Nat.add_lt_of_lt_sub hi
    rw [getD_map_range_cf (fun j => dropListTabFn (Nat.pair j (Nat.pair n c))) d hi,
      show dropListTabFn (Nat.pair i (Nat.pair n c)) = (decodeList c).getD (i + n) d
    by
        have hc : (Nat.pair i (Nat.pair n c)).unpair.2.unpair.2 = c := by simp
        have hn : (Nat.pair i (Nat.pair n c)).unpair.2.unpair.1 = n := by simp
        have hi' : (Nat.pair i (Nat.pair n c)).unpair.1 = i := by simp
        rw [dropListTabFn, hi', hn, hc, nthCode_eq,
          show (decodeList c).getD (i + n) 0 = (decodeList c).getD (i + n) d from by
            rw [getD_eq_getElem_cf _ 0 hic, getD_eq_getElem_cf _ d hic]],
      getD_drop_cf (decodeList c) n i d (by rw [List.length_drop]; exact hi)]
  * have hge : (decodeList c).length - n  <=  i := Nat.le_of_not_gt hi
    rw [getD_eq_default_cf _ d (by rw [List.length_map, List.length_range]; exact hge)
    ,
      getD_eq_default_cf _ d (by rw [List.length_drop]; exact hge)]

/-- Taking a prefix never increases the Godel code (`Nat.pair`'s monotonicity in the
    second
argument, chained through `encodeList`'s recursive `pair`). -/
theorem encodeList_take_le (l : List Nat) : forall  n, encodeList (l.take n)  <=
    encodeList l := by
  induction l with
  | nil => intro n; simp [encodeList]
  | cons a l ih =>
    intro n
    cases n with
    | zero => simp [encodeList]
    | succ n =>
      show encodeList (a :: l.take n)  <=  encodeList (a :: l)
      simp only [encodeList]
      exact Nat.add_le_add_right (pair_le_pair_right' a (ih n)) 1

/-- Dropping a suffix never increases the Godel code. -/
theorem encodeList_drop_le (l : List Nat) : forall  n, encodeList (l.drop n)  <=
    encodeList l := by
  induction l with
  | nil => intro n; simp [encodeList]
  | cons a l ih =>
```

```
    intro n
    cases n with
    | zero => simp [encodeList]
    | succ n =>
      show encodeList (l.drop n)  <=  encodeList (a :: l)
      calc encodeList (l.drop n)  <=  encodeList l := ih n
        _  <=  Nat.pair a (encodeList l) := le_pair_right a (encodeList l)
        _  <=  Nat.pair a (encodeList l) + 1 := Nat.le_succ _

theorem takeCode_le (n c : Nat) : takeCode n c  <=  c := by
  have h1 : takeCode n c = encodeList ((decodeList c).take n) := by
    rw [ <-  takeCode_eq, encodeList_decodeList]
  rw [h1]
  exact le_trans (encodeList_take_le _ n) (le_of_eq (encodeList_decodeList c))

theorem dropCode_le (n c : Nat) : dropCode n c  <=  c := by
  have h1 : dropCode n c = encodeList ((decodeList c).drop n) := by
    rw [ <-  dropCode_eq, encodeList_decodeList]
  rw [h1]
  exact le_trans (encodeList_drop_le _ n) (le_of_eq (encodeList_decodeList c))

theorem primrec_takeListTabFn : Nat.Primrec takeListTabFn :=
  (primrec_nthCode.comp
    ((Nat.Primrec.right.comp Nat.Primrec.right).pair
      (Nat.Primrec.left.pair (Nat.Primrec.const 0)))).of_eq fun w => by
    simp [takeListTabFn]

theorem primrec_min {f g : Nat  ->  Nat} (hf : Nat.Primrec f) (hg : Nat.Primrec g) :
    Nat.Primrec (fun n => Nat.min (f n) (g n)) :=
  (primrec_selectFn (primrec_le hf hg) hf hg).of_eq fun n => by
    unfold selectFn isZero Nat.min
    by_cases h : f n  <=  g n
    * simp [Nat.sub_eq_zero_iff_le.mpr h, Nat.min_eq_left h]
    * have hlt : g n < f n := Nat.lt_of_not_ge h
      have hpos : 0 < f n - g n := Nat.sub_pos_of_lt hlt
      have hmin : min (f n - g n) 1 = 1 := Nat.min_eq_right (Nat.succ_le_iff.mpr hpos)
      simp [hmin, Nat.min_eq_right (Nat.le_of_lt hlt)]

theorem primrec_takeCode : Nat.Primrec (fun t => takeCode t.unpair.1 t.unpair.2) := by
  have hn := Nat.Primrec.left
  have hc := Nat.Primrec.right
  have hlen := primrec_listLenChar.comp hc
  have hB := primrec_min hn hlen
  have hp := hn.pair hc
  have hr := (hB.pair hp).pair hB
  refine ((primrec_tabCode primrec_takeListTabFn).comp hr).of_eq fun t => by
    simp [takeCode]

theorem primrec_dropListTabFn : Nat.Primrec dropListTabFn :=
  (primrec_nthCode.comp
    ((Nat.Primrec.right.comp Nat.Primrec.right).pair
      (Nat.Primrec.pair (primrec_add_2 Nat.Primrec.left (Nat.Primrec.left.comp Nat.
   Primrec.right))
        (Nat.Primrec.const 0)))).of_eq fun w => by
    simp [dropListTabFn]
```

```
theorem primrec_dropCode : Nat.Primrec (fun t => dropCode t.unpair.1 t.unpair.2) := by
  have hn := Nat.Primrec.left
  have hc := Nat.Primrec.right
  have hlen := primrec_listLenChar.comp hc
  have hB := primrec_sub_2 hlen hn
  have hp := hn.pair hc
  have hr := (hB.pair hp).pair hB
  refine ((primrec_tabCode primrec_dropListTabFn).comp hr).of_eq fun t => by
    simp [dropCode]

open DomainTheory.NeighbourhoodSystems Exercise722
open DomainTheory.NeighbourhoodSystems.Exercise722

private theorem c9b5_decodeList_encodeListBool (sigma : List Bool) :
    decodeList (c9b5_encodeListBool sigma) = sigma.map c9b5_boolNat := by
  simp [c9b5_encodeListBool, decodeList_encodeList]

private theorem c9b5_boolNat_injective : Function.Injective c9b5_boolNat := by
  intro b c h
  cases b <;> cases c <;> first | rfl | simp [c9b5_boolNat] at h

private theorem c9b5_decodeList_injective {c1 c2 : Nat} (h : decodeList c1 =
    decodeList c2) : c1 = c2 := by
  have := congrArg encodeList h
  rwa [encodeList_decodeList, encodeList_decodeList] at this

private theorem c9b5_map_boolNat_eq_iff (w sigma : List Bool) :
    w.map c9b5_boolNat = sigma.map c9b5_boolNat  <->  w = sigma := by
  constructor
  * intro h
    revert sigma
    induction w with
    | nil =>
      intro sigma h
      cases sigma with
      | nil => rfl
      | cons _ _ => simp at h
    | cons b w ih =>
      intro sigma h
      cases sigma with
      | nil => simp at h
      | cons c sigma =>
        simp only [List.map_cons] at h
        obtain <hb, ht> := List.cons.inj h
        have hb' : b = c := by
          cases b <;> cases c <;> first | rfl | simp [c9b5_boolNat] at hb
        exact hb'  |>  congrArg (List.cons c) (ih sigma ht)
  * intro h; rw [h]

private theorem c9b5_takeCode_encodeListBool (w : List Bool) (i : Nat) :
    takeCode i (c9b5_encodeListBool w) = c9b5_encodeListBool (w.take i) := by
  apply c9b5_decodeList_injective
  rw [takeCode_eq, c9b5_decodeList_encodeListBool,  <-  List.map_take]
  rw [c9b5_decodeList_encodeListBool (sigma := w.take i)]

private theorem c9b5_dropCode_encodeListBool (w : List Bool) (i : Nat) :
```

```
    dropCode i (c9b5_encodeListBool w) = c9b5_encodeListBool (w.drop i) := by
  apply c9b5_decodeList_injective
  rw [dropCode_eq, c9b5_decodeList_encodeListBool,  <-  List.map_drop]
  rw [c9b5_decodeList_encodeListBool (sigma := w.drop i)]

private theorem c9b5_listEqChar_encodeListBool (w sigma : List Bool) :
    listEqChar (c9b5_encodeListBool w) (c9b5_encodeListBool sigma) = 1  <->  w = sigma
     := by
  simp only [listEqChar_eq_one_iff, c9b5_decodeList_encodeListBool,
    c9b5_map_boolNat_eq_iff]

theorem autStateCardFuelCharBody_eq (prev : Nat  ->  Nat) (c : Nat) :
    autStateCardFuelCharBody prev c =
      match c.unpair.1 with
      | 0 => 1


      | 1 => listLenChar c.unpair.2 + 2
      | 2 => prev c.unpair.2.unpair.1 + prev c.unpair.2.unpair.2
      | 3 => prev c.unpair.2.unpair.1 * prev c.unpair.2.unpair.2
      | _ => 0 := by
  unfold autStateCardFuelCharBody
  match tag : c.unpair.1 with
  | 0 =>
    have h10 : (1 - 0 : Nat) = 1 := rfl
    simp only [h10, isOne_one, selectFn_one]
  | 1 =>
    have h01 : (1 - 1 : Nat) = 0 := rfl
    have h12 : (2 - 1 : Nat) = 1 := rfl
    simp only [h01, h12, isOne_zero, isOne_one, selectFn_zero, selectFn_one]
  | 2 =>
    have h01 : (1 - 2 : Nat) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h12 : (2 - 2 : Nat) = 0 := rfl
    have h23 : (3 - 2 : Nat) = 1 := rfl
    simp only [h01, h23, isOne_zero, isOne_one, selectFn_zero, selectFn_one]
  | 3 =>
    have h01 : (1 - 3 : Nat) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h12 : (2 - 3 : Nat) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h23 : (3 - 3 : Nat) = 0 := rfl
    have h34 : (4 - 3 : Nat) = 1 := rfl
    simp only [h01, h34, isOne_zero, isOne_one, selectFn_zero, selectFn_one]
  | t + 4 =>
    have hne1 : isOne (1 - (t + 4)) = 0 := isOne_of_ne_one (by omega)
    have hne2 : isOne (2 - (t + 4)) = 0 := isOne_of_ne_one (by omega)
    have hne3 : isOne (3 - (t + 4)) = 0 := isOne_of_ne_one (by omega)
    have hne4 : isOne (4 - (t + 4)) = 0 := isOne_of_ne_one (by omega)
    simp only [hne1, hne2, hne3, hne4, selectFn_zero]

theorem autStateCardFuelChar_eq_autStateCard {fuel : Nat} {e : SExpr}
    (h : c9b5_sexprDepth e  <=  fuel) :
    autStateCardFuelChar fuel (c9b5_sexprGodelEncode e) = autStateCard e := by
  induction e generalizing fuel with
  | sigma =>
    cases fuel with
    | zero => simp [c9b5_sexprDepth] at h
    | succ fuel =>
      simp [autStateCardFuelChar, autStateCardFuelCharBody_eq, c9b5_sexprGodelEncode,
```

```
    autStateCard]
  | single sigma =>
    cases fuel with
    | zero => simp [c9b5_sexprDepth] at h
    | succ fuel =>
      simp [autStateCardFuelChar, autStateCardFuelCharBody_eq, c9b5_sexprGodelEncode,
    autStateCard,
        listLenChar_eq, c9b5_decodeList_encodeListBool, List.length_map]
  | cat a b iha ihb =>
    cases fuel with
    | zero => simp [c9b5_sexprDepth] at h
    | succ fuel =>
      have ha : c9b5_sexprDepth a  <=  fuel := by
        simp only [c9b5_sexprDepth] at h  |- ; omega
      have hb : c9b5_sexprDepth b  <=  fuel := by
        simp only [c9b5_sexprDepth] at h  |- ; omega
      simp [autStateCardFuelChar, autStateCardFuelCharBody_eq, c9b5_sexprGodelEncode,
    autStateCard,
        iha ha, ihb hb]
  | cap a b iha ihb =>
    cases fuel with
    | zero => simp [c9b5_sexprDepth] at h
    | succ fuel =>
      have ha : c9b5_sexprDepth a  <=  fuel := by
        simp only [c9b5_sexprDepth] at h  |- ; omega
      have hb : c9b5_sexprDepth b  <=  fuel := by
        simp only [c9b5_sexprDepth] at h  |- ; omega
      simp [autStateCardFuelChar, autStateCardFuelCharBody_eq, c9b5_sexprGodelEncode,
    autStateCard,
        iha ha, ihb hb]

theorem primrec_autStateCardFuelCharBody {prev : Nat  ->  Nat} (hprev : Nat.Primrec
    prev) :
    Nat.Primrec (fun c => autStateCardFuelCharBody prev c) := by
  unfold autStateCardFuelCharBody
  have hleft : Nat.Primrec (fun c => prev c.unpair.2.unpair.1) :=
    (hprev.comp (Nat.Primrec.left.comp Nat.Primrec.right)).of_eq fun c => by
      simp only
  have hright : Nat.Primrec (fun c => prev c.unpair.2.unpair.2) :=
    (hprev.comp (Nat.Primrec.right.comp Nat.Primrec.right)).of_eq fun c => by
      simp only
  have hadd := primrec_add_2 hleft hright
  have hmul := primrec_mul_2 hleft hright
  have hf1 := primrec_add_2 (primrec_listLenChar.comp Nat.Primrec.right) (Nat.Primrec.
    const 2)
  exact primrec_tagCase4 (Nat.Primrec.const 1) hf1 hadd hmul (Nat.Primrec.const 0)

theorem autStateCardFuelCharBody_local (f g : Nat  ->  Nat) (c : Nat) (h : forall  c'
     <=  c, f c' = g c') :
    autStateCardFuelCharBody f c = autStateCardFuelCharBody g c := by
  rw [autStateCardFuelCharBody_eq, autStateCardFuelCharBody_eq]
  have h1 : c.unpair.2.unpair.1  <=  c := le_trans (unpair_left_le c.unpair.2) (
    unpair_snd_le c)
  have h2 : c.unpair.2.unpair.2  <=  c := le_trans (unpair_snd_le c.unpair.2) (
    unpair_snd_le c)
  rw [h _ h1, h _ h2]
```

```
theorem matchesBCharBody_eq (prev : Nat  ->  Nat) (c cw : Nat) :
    matchesBCharBody prev c cw =
      match c.unpair.1 with
      | 0 => 1
      | 1 => listEqChar cw c.unpair.2
      | 2 =>
        bExistsFn (matchesBCatG prev) (Nat.pair c cw) (listLenChar cw + 1)
      | 3 =>
        mulBit (prev (Nat.pair c.unpair.2.unpair.1 cw))
               (prev (Nat.pair c.unpair.2.unpair.2 cw))
      | _ => 0 := by
  unfold matchesBCharBody matchesBCatG
  match tag : c.unpair.1 with
  | 0 =>
    have h10 : (1 - 0 : Nat) = 1 := rfl
    simp only [h10, isOne_one, selectFn_one]
  | 1 =>
    have h01 : (1 - 1 : Nat) = 0 := rfl
    have h12 : (2 - 1 : Nat) = 1 := rfl
    simp only [h01, h12, isOne_zero, isOne_one, selectFn_zero, selectFn_one]
  | 2 =>
    have h01 : (1 - 2 : Nat) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h12 : (2 - 2 : Nat) = 0 := rfl
    have h23 : (3 - 2 : Nat) = 1 := rfl
    simp only [h01, h23, isOne_zero, isOne_one, selectFn_zero, selectFn_one]
  | 3 =>
    have h01 : (1 - 3 : Nat) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h12 : (2 - 3 : Nat) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h23 : (3 - 3 : Nat) = 0 := rfl
    have h34 : (4 - 3 : Nat) = 1 := rfl
    simp only [h01, h34, isOne_zero, isOne_one, selectFn_zero, selectFn_one]
  | t + 4 =>
    have hne1 : isOne (1 - (t + 4)) = 0 := isOne_of_ne_one (by omega)
    have hne2 : isOne (2 - (t + 4)) = 0 := isOne_of_ne_one (by omega)
    have hne3 : isOne (3 - (t + 4)) = 0 := isOne_of_ne_one (by omega)
    have hne4 : isOne (4 - (t + 4)) = 0 := isOne_of_ne_one (by omega)
    simp only [hne1, hne2, hne3, hne4, selectFn_zero]

private theorem c9b5_matchesBCatG_char (fuel : Nat) (a b : SExpr) (w : List Bool) (i :
     Nat) :
    matchesBCatG (fun p => matchesBChar fuel p.unpair.1 p.unpair.2)
        (Nat.pair i
          (Nat.pair (Nat.pair 2 (Nat.pair (c9b5_sexprGodelEncode a) (
    c9b5_sexprGodelEncode b)))
            (c9b5_encodeListBool w))) =
      mulBit (matchesBChar fuel (c9b5_sexprGodelEncode a) (takeCode i (
    c9b5_encodeListBool w)))
        (matchesBChar fuel (c9b5_sexprGodelEncode b) (dropCode i (c9b5_encodeListBool
    w))) := by
  simp [matchesBCatG]

set_option maxHeartbeats 1600000 in
theorem matchesBChar_eq_one_iff {fuel : Nat} {e : SExpr} {w : List Bool}
    (h : c9b5_sexprDepth e  <=  fuel) :
    matchesBChar fuel (c9b5_sexprGodelEncode e) (c9b5_encodeListBool w) = 1  <->
```

```
    matchesB e w = true := by
induction e generalizing fuel w with
| sigma =>
  cases fuel with
  | zero => simp [c9b5_sexprDepth] at h
  | succ fuel => simp [matchesBChar, matchesBCharBody_eq, matchesB,
  c9b5_sexprGodelEncode]
| single sigma =>
  cases fuel with
  | zero => simp [c9b5_sexprDepth] at h
  | succ fuel =>
    simp [matchesBChar, matchesBCharBody_eq, c9b5_sexprGodelEncode, matchesB,
      c9b5_listEqChar_encodeListBool w sigma, decide_eq_true_eq]
| cat a b iha ihb =>
  cases fuel with
  | zero => simp [c9b5_sexprDepth] at h
  | succ fuel =>
    have ha : c9b5_sexprDepth a  <=  fuel := by
      simp only [c9b5_sexprDepth] at h  |- ; omega
    have hb : c9b5_sexprDepth b  <=  fuel := by
      simp only [c9b5_sexprDepth] at h  |- ; omega
    have hbody := matchesBCharBody_eq (fun p => matchesBChar fuel p.unpair.1 p.
  unpair.2)
        (c9b5_sexprGodelEncode (.cat a b)) (c9b5_encodeListBool w)
    rw [matchesBChar, hbody]
    simp only [c9b5_sexprGodelEncode, unpair_pair_fst, matchesB, List.any_eq_true]
    rw [bExistsFn_eq_one_iff]
    have hlen : listLenChar (c9b5_encodeListBool w) + 1 = w.length + 1 := by
      simp [listLenChar_eq, c9b5_decodeList_encodeListBool, List.length_map]
    constructor
    * intro <i, hi, hg>
      rw [c9b5_matchesBCatG_char, mulBit_eq_one_iff] at hg
      obtain <h1, h2> := hg
      have hi' : i < w.length + 1 := by rw [ <-  hlen]; exact hi
      refine <i, (List.mem_range).2 hi', ?_>
      simp only [Bool.and_eq_true]
      exact <(iha ha).mp (by rw [ <-  c9b5_takeCode_encodeListBool w i]; exact h1),
        (ihb hb).mp (by rw [ <-  c9b5_dropCode_encodeListBool w i]; exact h2)>
    * intro <i, hi, hfg>
      have hi' : i < listLenChar (c9b5_encodeListBool w) + 1 := by
        rw [hlen]; exact (List.mem_range).1 hi
      refine <i, hi', ?_>
      rw [c9b5_matchesBCatG_char, mulBit_eq_one_iff]
      simp [Bool.and_eq_true] at hfg
      refine And.intro ?h1 ?h2
      * simpa [c9b5_takeCode_encodeListBool w i] using (iha ha).mpr hfg.1
      * simpa [c9b5_dropCode_encodeListBool w i] using (ihb hb).mpr hfg.2
| cap a b iha ihb =>
  cases fuel with
  | zero => simp [c9b5_sexprDepth] at h
  | succ fuel =>
    have ha : c9b5_sexprDepth a  <=  fuel := by
      simp only [c9b5_sexprDepth] at h  |- ; omega
    have hb : c9b5_sexprDepth b  <=  fuel := by
      simp only [c9b5_sexprDepth] at h  |- ; omega
    have hbody := matchesBCharBody_eq (fun p => matchesBChar fuel p.unpair.1 p.
```

```
    unpair.2)
          (c9b5_sexprGodelEncode (.cap a b)) (c9b5_encodeListBool w)
      rw [matchesBChar, hbody]
      simp only [c9b5_sexprGodelEncode, unpair_pair_fst, unpair_pair_snd, matchesB,
    mulBit_eq_one_iff]
      constructor
      * intro <h1, h2>
        rw [Bool.and_eq_true]
        exact <(iha ha).mp h1, (ihb hb).mp h2>
      * intro hfg
        rw [Bool.and_eq_true] at hfg
        exact <(iha ha).mpr hfg.1, (ihb hb).mpr hfg.2>

set_option maxHeartbeats 800000 in
private theorem primrec_matchesBCatG {prev : Nat  ->  Nat} (hprev : Nat.Primrec prev)
    :
    Nat.Primrec (fun p => matchesBCatG prev p) := by
  have hc := Nat.Primrec.left.comp Nat.Primrec.right
  have ha := Nat.Primrec.left.comp (Nat.Primrec.right.comp hc)
  have hb := Nat.Primrec.right.comp (Nat.Primrec.right.comp hc)
  have hcw := Nat.Primrec.right.comp Nat.Primrec.right
  have hi := Nat.Primrec.left
  have h1 := hprev.comp (Nat.Primrec.pair ha (primrec_takeCode.comp (hi.pair hcw)))
  have h2 := hprev.comp (Nat.Primrec.pair hb (primrec_dropCode.comp (hi.pair hcw)))
  exact (primrec_mul_2 h1 h2).of_eq fun p => by
    dsimp [matchesBCatG, mulBit]
    simp only [unpair_pair_fst, unpair_pair_snd]

set_option maxHeartbeats 800000 in
theorem primrec_matchesBCharBody {prev : Nat  ->  Nat} (hprev : Nat.Primrec prev) :
    Nat.Primrec (fun t => matchesBCharBody prev t.unpair.1 t.unpair.2) := by
  have htag : Nat.Primrec (fun t => (t.unpair.1).unpair.1) := Nat.Primrec.left.comp
    Nat.Primrec.left
  have h01 := primrec_isOne.comp (primrec_sub_2 (Nat.Primrec.const 1) htag)
  have h12 := primrec_isOne.comp (primrec_sub_2 (Nat.Primrec.const 2) htag)
  have h23 := primrec_isOne.comp (primrec_sub_2 (Nat.Primrec.const 3) htag)
  have h34 := primrec_isOne.comp (primrec_sub_2 (Nat.Primrec.const 4) htag)
  have hf1 := primrec_listEqChar.comp (Nat.Primrec.pair Nat.Primrec.right
    (Nat.Primrec.right.comp Nat.Primrec.left))
  have hpack : Nat.Primrec (fun t => Nat.pair t (listLenChar t.unpair.2 + 1)) :=
    Nat.Primrec.pair primrec_id
      (primrec_add_2 (primrec_listLenChar.comp Nat.Primrec.right) (Nat.Primrec.const
    1))
  have hcat := (primrec_bExistsFn (primrec_matchesBCatG hprev)).comp hpack
  have hc := Nat.Primrec.left
  have ha := Nat.Primrec.left.comp (Nat.Primrec.right.comp hc)
  have hb := Nat.Primrec.right.comp (Nat.Primrec.right.comp hc)
  have hcw := Nat.Primrec.right
  have hcap := primrec_mulBit.comp (Nat.Primrec.pair
    (hprev.comp (Nat.Primrec.pair ha hcw)) (hprev.comp (Nat.Primrec.pair hb hcw)))
  exact (primrec_selectFn h01 (Nat.Primrec.const 1)
    (primrec_selectFn h12 hf1
      (primrec_selectFn h23 hcat
        (primrec_selectFn h34 hcap (Nat.Primrec.const 0))))).of_eq fun t => by
    rw [matchesBCharBody_eq]
    match tag : (t.unpair.1).unpair.1 with
```

```
    | 0 | 1 | 2 | 3 =>
      simp [selectFn, isOne, mulBit]
    | _ + 4 =>
      simp [selectFn, isOne, mulBit]

theorem matchesBChar_le_one : forall  fuel c cw, matchesBChar fuel c cw  <=  1
  | 0, _, _ => by simp [matchesBChar]
  | fuel + 1, c, cw => by
    rw [matchesBChar, matchesBCharBody_eq]
    match tag : c.unpair.1 with
    | 0 => simp
    | 1 => exact listEqChar_le_one _ _
    | 2 => exact bExistsFn_le_one _ _ _
    | 3 =>
      exact mulBit_le_one (matchesBChar_le_one fuel _ _) (matchesBChar_le_one fuel _ _
    )
    | _ + 4 => simp

theorem matchesBCharBody_local (f g : Nat  ->  Nat) (c cw : Nat)
    (h : forall  p  <=  Nat.pair c cw, f p = g p) : matchesBCharBody f c cw =
    matchesBCharBody g c cw := by
  rw [matchesBCharBody_eq, matchesBCharBody_eq]
  match tag : c.unpair.1 with
  | 0 => rfl
  | 1 => rfl
  | 2 =>
    simp
    apply bExistsFn_congr
    intro i _
    unfold matchesBCatG
    simp only [unpair_pair_fst, unpair_pair_snd]
    have h1 : Nat.pair c.unpair.2.unpair.1 (takeCode i cw)  <=  Nat.pair c cw :=
      pair_le_pair (le_trans (unpair_left_le c.unpair.2) (unpair_snd_le c)) (
    takeCode_le i cw)
    have h2 : Nat.pair c.unpair.2.unpair.2 (dropCode i cw)  <=  Nat.pair c cw :=
      pair_le_pair (le_trans (unpair_snd_le c.unpair.2) (unpair_snd_le c)) (
    dropCode_le i cw)
    rw [h _ h1, h _ h2]
  | 3 =>
    simp
    have h1 : Nat.pair c.unpair.2.unpair.1 cw  <=  Nat.pair c cw :=
      pair_le_pair (le_trans (unpair_left_le c.unpair.2) (unpair_snd_le c)) (le_refl
    cw)
    have h2 : Nat.pair c.unpair.2.unpair.2 cw  <=  Nat.pair c cw :=
      pair_le_pair (le_trans (unpair_snd_le c.unpair.2) (unpair_snd_le c)) (le_refl cw
    )
    rw [h _ h1, h _ h2]
  | _ + 4 => rfl

/-- `Nat.pair 0 t < Nat.pair 1 b` whenever `t < b`, via mathlib's `Nat.pair`
    monotonicity
in each argument (`pair_lt_pair_right` then `pair_lt_pair_left`). -/
private theorem pair_lt_pair_zero_right {t b : Nat} (ht : t < b) :
    Nat.pair 0 t < Nat.pair 1 b :=
  lt_trans (Nat.pair_lt_pair_right 0 ht) (Nat.pair_lt_pair_left b Nat.zero_lt_one)
```

```
theorem codeBound_ge (n c : Nat) (hlen : (decodeList c).length  <=  n)
    (hbin : forall  x  in  decodeList c, x  <=  1) : c < codeBound n := by
  induction n generalizing c with
  | zero =>
    have hc : c = 0 := by
      cases c with
      | zero => rfl
      | succ c' => simp [decodeList_succ, List.length_cons] at hlen
    simp [hc, codeBound]
  | succ n ih =>
    cases c with
    | zero => simp [codeBound]
    | succ d =>
      simp only [decodeList_succ, List.length_cons] at hlen
      simp only [decodeList_succ] at hbin
      have htlen : (decodeList d.unpair.2).length  <=  n := by omega
      have htbin : forall  x  in  decodeList d.unpair.2, x  <=  1 := fun x hx =>
        hbin x (List.mem_cons.mpr (Or.inr hx))
      have ht := ih d.unpair.2 htlen htbin
      have hh : d.unpair.1  <=  1 := hbin d.unpair.1 (List.mem_cons.mpr (Or.inl rfl))
      have hpair : Nat.pair d.unpair.1 d.unpair.2 < Nat.pair 1 (codeBound n) := by
        rcases (show d.unpair.1 = 0  \/  d.unpair.1 = 1 by omega) with h0 | h1
        * rw [h0]; exact pair_lt_pair_zero_right ht
        * rw [h1]; exact Nat.pair_lt_pair_right 1 ht
      rw [Nat.pair_unpair] at hpair
      simp only [codeBound]
      omega

private theorem c9b6_allBinDigitsChar_encodeListBool (w : List Bool) :
    allBinDigitsChar (c9b5_encodeListBool w) = 1 := by
  rw [allBinDigitsChar_eq_one_iff]
  intro x hx
  rw [c9b5_decodeList_encodeListBool] at hx
  simp only [List.mem_map] at hx
  obtain <b, _, hb> := hx
  cases b <;> simp [c9b5_boolNat] at hb <;> omega

private theorem c9b6_encodeListBool_lt_codeBound {n : Nat} {w : List Bool} (hw : w.
    length  <=  n) :
    c9b5_encodeListBool w < codeBound n := by
  apply codeBound_ge n (c9b5_encodeListBool w)
  * rw [c9b5_decodeList_encodeListBool, List.length_map]; exact hw
  * intro x hx
    rw [c9b5_decodeList_encodeListBool] at hx
    simp only [List.mem_map] at hx
    obtain <b, _, hb> := hx
    cases b <;> simp [c9b5_boolNat] at hb <;> omega

private theorem c9b6_boolNat_decide_eq (x : Nat) (hx : x = 0  \/  x = 1) :
    c9b5_boolNat (decide (x = 1)) = x := by
  rcases hx with rfl | rfl <;> simp [c9b5_boolNat]

/-- Round-trip through `c9b5_encodeListBool`/`c9b6_decodeListBool` for a code known to
    consist
only of binary digits (i.e. `allBinDigitsChar c = 1`). -/
private theorem c9b6_encodeListBool_decodeListBool_of_allBin {c : Nat}
```

```
    (hall : allBinDigitsChar c = 1) :
    c9b5_encodeListBool (c9b6_decodeListBool c) = c := by
  rw [allBinDigitsChar_eq_one_iff] at hall
  apply c9b5_decodeList_injective
  rw [c9b5_decodeList_encodeListBool, c9b6_decodeListBool, List.map_map]
  conv_rhs => rw [ <-  List.map_id (decodeList c)]
  exact List.map_congr_left (fun x hx => c9b6_boolNat_decide_eq x (hall x hx))

private theorem c9b6_matchesBChar_of_allBin {fuel : Nat} {e : SExpr} {c : Nat}
    (h : c9b5_sexprDepth e  <=  fuel) (hall : allBinDigitsChar c = 1)
    (hmatch : matchesBChar fuel (c9b5_sexprGodelEncode e) c = 1) :
    matchesB e (c9b6_decodeListBool c) = true := by
  apply (matchesBChar_eq_one_iff (w := c9b6_decodeListBool c) h).1
  rwa [c9b6_encodeListBool_decodeListBool_of_allBin hall]

theorem decideNonemptyBChar_eq_one_iff {fuel : Nat} {e : SExpr}
    (h : c9b5_sexprDepth e  <=  fuel) :
    decideNonemptyBChar fuel (c9b5_sexprGodelEncode e) = 1  <->  (denote e).Nonempty
    := by
  simp only [decideNonemptyBChar, bExistsFn_eq_one_iff, unpair_pair_fst,
    unpair_pair_snd,
    mulBit_eq_one_iff]
  constructor
  * rintro <i, -, hall, hmatch>
    exact <c9b6_decodeListBool i, (matchesB_iff e _).mp (c9b6_matchesBChar_of_allBin h
     hall hmatch)>
  * intro hne
    obtain <w, hwmem, hmatch> := (denote_nonempty_iff_short e).mp hne
    refine <c9b5_encodeListBool w, ?_, c9b6_allBinDigitsChar_encodeListBool w,
      (matchesBChar_eq_one_iff h).2 hmatch>
    rw [autStateCardFuelChar_eq_autStateCard h]
    exact c9b6_encodeListBool_lt_codeBound (by rwa [mem_wordsUpTo] at hwmem)

theorem c9b6_sexprGodelEncode_cap (a b : SExpr) :
    c9b5_sexprGodelEncode (.cap a b) = capCode (c9b5_sexprGodelEncode a) (
    c9b5_sexprGodelEncode b) :=
  rfl

theorem subsetGuardChar_le_one (fuel c1 c2 i : Nat) : subsetGuardChar fuel c1 c2 i  <=
     1 :=
  mulBit_le_one (allBinDigitsChar_le_one _) (isOne_le_one _)

set_option maxHeartbeats 800000 in
theorem subsetBChar_eq_one_iff {fuel : Nat} {e1 e2 : SExpr}
    (h1 : c9b5_sexprDepth e1  <=  fuel) (h2 : c9b5_sexprDepth e2  <=  fuel) :
    subsetBChar fuel (c9b5_sexprGodelEncode e1) (c9b5_sexprGodelEncode e2) = 1  <->
      denote e1  subseteq  denote e2 := by
  rw [ <-  subsetB_iff]
  have hbound : subsetBoundChar fuel (c9b5_sexprGodelEncode e1) (c9b5_sexprGodelEncode
    e2) =
      Fintype.card (Finset (autState e1)  x  Finset (autState e2)) := by
    unfold subsetBoundChar
    rw [autStateCardFuelChar_eq_autStateCard h1, autStateCardFuelChar_eq_autStateCard
    h2,
      autStateCard_eq_card, autStateCard_eq_card, Fintype.card_prod, Fintype.
    card_finset,
```

```
    Fintype.card_finset]
unfold subsetB subsetBChar
rw [List.all_eq_true, bForallFn_eq_one_iff]
simp only [unpair_pair_fst, unpair_pair_snd, hbound]
constructor

* intro h w hwmem
  rw [mem_wordsUpTo] at hwmem
  have hi := h (c9b5_encodeListBool w) (c9b6_encodeListBool_lt_codeBound hwmem)
  have hguard1 : subsetGuardChar fuel (c9b5_sexprGodelEncode e1) (
  c9b5_sexprGodelEncode e2)
      (c9b5_encodeListBool w) = 1 := by
    unfold subsetGuardChar
    rw [c9b6_allBinDigitsChar_encodeListBool, mulBit, one_mul, listLenChar_eq,
      c9b5_decodeList_encodeListBool, List.length_map, hbound,
      Nat.sub_eq_zero_of_le hwmem]
    exact isOne_one
  rw [hguard1, selectFn_one] at hi
  by_cases hc1 : matchesBChar fuel (c9b5_sexprGodelEncode e1) (c9b5_encodeListBool w
  ) = 1
  * rw [hc1, selectFn_one] at hi
    simp only [Bool.or_eq_true]
    exact Or.inr ((matchesBChar_eq_one_iff h2).mp hi)
  * have hc1' : matchesBChar fuel (c9b5_sexprGodelEncode e1) (c9b5_encodeListBool w)
   = 0 := by
      have := matchesBChar_le_one fuel (c9b5_sexprGodelEncode e1) (
  c9b5_encodeListBool w)
      omega
    simp only [Bool.or_eq_true]
    refine Or.inl ?_
    by_contra hcontra
    exact hc1 ((matchesBChar_eq_one_iff h1).mpr (by simpa using hcontra))
* intro h i _
  rcases (show subsetGuardChar fuel (c9b5_sexprGodelEncode e1) (
  c9b5_sexprGodelEncode e2) i = 0  \/
      subsetGuardChar fuel (c9b5_sexprGodelEncode e1) (c9b5_sexprGodelEncode e2) i =
   1 from by
    have := subsetGuardChar_le_one fuel (c9b5_sexprGodelEncode e1) (
  c9b5_sexprGodelEncode e2) i
    omega) with hguard | hguard
  * rw [hguard, selectFn_zero]
  * rw [hguard, selectFn_one]
    unfold subsetGuardChar at hguard
    rw [mulBit_eq_one_iff, isOne_eq_one_iff] at hguard
    obtain <hallbin, hlen0> := hguard
    have hlenle : listLenChar i  <=
        Fintype.card (Finset (autState e1)  x  Finset (autState e2)) := by omega
    have hieq : c9b5_encodeListBool (c9b6_decodeListBool i) = i :=
      c9b6_encodeListBool_decodeListBool_of_allBin hallbin
    have hwlen : (c9b6_decodeListBool i).length  <=
        Fintype.card (Finset (autState e1)  x  Finset (autState e2)) := by
      unfold c9b6_decodeListBool
      rw [List.length_map]
      rwa [listLenChar_eq] at hlenle
    have hwmem := (mem_wordsUpTo _).mpr hwlen
    have hspec := h (c9b6_decodeListBool i) hwmem
```

```
      rw [ <-  hieq]
      by_cases hc1 : matchesBChar fuel (c9b5_sexprGodelEncode e1) (c9b5_encodeListBool
          (c9b6_decodeListBool i)) = 1
      * rw [hc1, selectFn_one]
        have hc1matches : matchesB e1 (c9b6_decodeListBool i) = true :=
          (matchesBChar_eq_one_iff h1).mp hc1
        simp only [Bool.or_eq_true] at hspec
        rcases hspec with hf | ht
        * rw [hc1matches] at hf; simp at hf
        * exact (matchesBChar_eq_one_iff h2).mpr ht
      * have hc1' : matchesBChar fuel (c9b5_sexprGodelEncode e1)
            (c9b5_encodeListBool (c9b6_decodeListBool i)) = 0 := by
          have := matchesBChar_le_one fuel (c9b5_sexprGodelEncode e1)
            (c9b5_encodeListBool (c9b6_decodeListBool i))
          omega
        rw [hc1', selectFn_zero]
end

section
/-- **Swap the two arguments** of a recursively decidable binary relation. -/
theorem RecDecidable_2.swap {r : Nat  ->  Nat  ->  Prop} (hr : RecDecidable_2 r) :
    RecDecidable_2 (fun a b => r b a) := by
  have hswap : Nat.Primrec (fun t => Nat.pair t.unpair.2 t.unpair.1) :=
    Nat.Primrec.right.pair Nat.Primrec.left
  refine RecDecidable.of_iff (fun t => ?_) (hr.comp hswap)
  simp only [unpair_pair_fst, unpair_pair_snd]
end

section
/-- **Bounded existential quantifier preserves recursive decidability.** Mirrors
`RecDecidable.bForall`, swapping `bForallFn` for `bExistsFn`. -/
theorem RecDecidable.bExists {p : Nat  ->  Prop} (hp : RecDecidable p) {bound : Nat
    ->  Nat}
    (hb : Nat.Primrec bound) :
    RecDecidable (fun n => exists  i, i < bound n  /\  p (Nat.pair i n)) := by
  obtain <f, hf, hfspec> := hp
  refine <fun n => bExistsFn f n (bound n), ?_, ?_>
  * have hGfn : Nat.Primrec (fun w => selectFn w.unpair.2.unpair.2 1
        (isOne (f (Nat.pair w.unpair.2.unpair.1 w.unpair.1)))) :=
      primrec_selectFn (Nat.Primrec.right.comp Nat.Primrec.right) (Nat.Primrec.const
    1)
        (primrec_isOne.comp (hf.comp
          ((Nat.Primrec.left.comp Nat.Primrec.right).pair Nat.Primrec.left)))
    have hprec := Nat.Primrec.prec (Nat.Primrec.const 0) hGfn
    refine (hprec.comp (primrec_id.pair hb)).of_eq (fun n => ?_)
    simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd, id_eq]
    rfl
  * intro n
    show (exists  i, i < bound n  /\  p (Nat.pair i n))  <->  bExistsFn f n (bound n)
    = 1
    rw [bExistsFn_eq_one_iff]
    exact <fun <i, hi, h> => <i, hi, (hfspec _).mp h>, fun <i, hi, h> => <i, hi, (
    hfspec _).mpr h>>
end

section
```

```
/-- **Bounded universal quantifier preserves recursive decidability.** If `p` is
    recursively decidable
and `bound` is primitive recursive, then `fun n => forall  i < bound n, p (pair i n)`
    is recursively
decidable (choice-free). -/
theorem RecDecidable.bForall {p : Nat  ->  Prop} (hp : RecDecidable p) {bound : Nat
    ->  Nat}
    (hb : Nat.Primrec bound) :
    RecDecidable (fun n => forall  i, i < bound n  ->  p (Nat.pair i n)) := by
  obtain <f, hf, hfspec> := hp
  refine <fun n => bForallFn f n (bound n), ?_, ?_>
  * have hGfn : Nat.Primrec (fun w => selectFn w.unpair.2.unpair.2
        (isOne (f (Nat.pair w.unpair.2.unpair.1 w.unpair.1))) 0) :=
      primrec_selectFn (Nat.Primrec.right.comp Nat.Primrec.right)
        (primrec_isOne.comp (hf.comp
          ((Nat.Primrec.left.comp Nat.Primrec.right).pair Nat.Primrec.left)))
        (Nat.Primrec.const 0)
    have hprec := Nat.Primrec.prec (Nat.Primrec.const 1) hGfn
    refine (hprec.comp (primrec_id.pair hb)).of_eq (fun n => ?_)
    simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd, id_eq]
    rfl
  * intro n
    show (forall  i, i < bound n  ->  p (Nat.pair i n))  <->  bForallFn f n (bound n)
    = 1
    rw [bForallFn_eq_one_iff]
    exact <fun h i hi => (hfspec _).mp (h i hi), fun h i hi => (hfspec _).mpr (h i hi)
    >
end

section
/-- **From a primitive-recursive `{0,1}` characteristic to `RecDecidable`.** This is
    the standard
bridge when an executable Bool decider has already been realized as a numeric `{0,1}`
    function; the
logical equivalence is supplied separately (do not reprove the mathematics). -/
theorem RecDecidable.of_zero_one_char {p : Nat  ->  Prop} {f : Nat  ->  Nat} (hf : Nat
    .Primrec f)
    (_h01 : forall  n, f n = 0  \/  f n = 1) (hfe : forall  n, p n  <->  f n = 1) :
    RecDecidable p :=
  <f, hf, hfe>
end

section
/-- **Paired `{0,1}` characteristic.** When the Bool decider has been packaged as a
    unary
primitive-recursive function on `Nat.pair n m`, reindex to `RecDecidable_2`. -/
theorem RecDecidable_2.of_paired_zero_one_char {r : Nat  ->  Nat  ->  Prop} {f : Nat
    ->  Nat} (hf : Nat.Primrec f)
    (h01 : forall  t, f t = 0  \/  f t = 1) (hfe : forall  n m, r n m  <->  f (Nat.
    pair n m) = 1) :
    RecDecidable_2 r := by
  unfold RecDecidable_2
  refine RecDecidable.of_zero_one_char hf h01 (fun t => ?_)
  rw [hfe, pair_unpair]
end
```

```
section
/-- **Triple `{0,1}` characteristic.** When the Bool decider has been packaged as a
    unary
primitive-recursive function on `Nat.pair n (Nat.pair m k)`, reindex to `RecDecidable_3
    `. -/
theorem RecDecidable_3.of_triple_zero_one_char {r : Nat  ->  Nat  ->  Nat  ->  Prop} {
    f : Nat  ->  Nat}
    (hf : Nat.Primrec f) (h01 : forall  t, f t = 0  \/  f t = 1)
    (hfe : forall  n m k, r n m k  <->  f (Nat.pair n (Nat.pair m k)) = 1) :
    RecDecidable_3 r := by
  unfold RecDecidable_3
  refine RecDecidable.of_zero_one_char hf h01 (fun t => ?_)
  rw [hfe, pair_unpair, pair_unpair]
end

section
theorem codeBound_succ (n : Nat) : codeBound (n + 1) = Nat.pair 1 (codeBound n) + 1 :=
    rfl
end

section
theorem codeBound_zero : codeBound 0 = 1 := rfl
end

section
theorem consistentBChar_eq_one_iff {fuel : Nat} {a b : SExpr}
    (h : c9b5_sexprDepth (.cap a b)  <=  fuel) :
    consistentBChar fuel (c9b5_sexprGodelEncode a) (c9b5_sexprGodelEncode b) = 1  <->
      (denote (.cap a b)).Nonempty := by
  simp only [consistentBChar,  <-  c9b6_sexprGodelEncode_cap]
  exact decideNonemptyBChar_eq_one_iff h
end

section
theorem decideNonemptyBChar_le_one (fuel c_e : Nat) : decideNonemptyBChar fuel c_e  <=
     1 :=
  bExistsFn_le_one _ _ _
end

section
/-- Fuel is sized for the *outer* `.cap a b` expression (matching the convention of
`decideNonemptyBChar_eq_one_iff` / `matchesBChar_eq_one_iff`), one more than either
    child alone
needs, since `c9b5_sexprDepth (.cap a b) = 1 + max (c9b5_sexprDepth a) (c9b5_sexprDepth
    b)`. -/
theorem consistentBChar_le_one (fuel c1 c2 : Nat) : consistentBChar fuel c1 c2  <=  1
    :=
  decideNonemptyBChar_le_one _ _
end

section
theorem decodeFuelOkChar_le_one : forall  fuel c, decodeFuelOkChar fuel c  <=  1
  | 0, _ => by simp [decodeFuelOkChar]
  | fuel + 1, c => by
    rw [decodeFuelOkChar, decodeFuelOkCharBody_eq]
    match tag : c.unpair.1 with
```

```
    | 0 =>
      simp
      rcases (show isOne (1 - c.unpair.2) = 0  \/  isOne (1 - c.unpair.2) = 1 by
        have := isOne_le_one (1 - c.unpair.2); omega) with h0 | h1
      * rw [h0, selectFn_zero]; exact Nat.zero_le 1
      * rw [h1, selectFn_one]
    | 1 =>
      simp
      exact allBinDigitsChar_le_one _
    | 2 | 3 =>
      simp
      exact mulBit_le_one (decodeFuelOkChar_le_one fuel c.unpair.2.unpair.1)
        (decodeFuelOkChar_le_one fuel c.unpair.2.unpair.2)
    | _ + 4 =>
      simp
end

section
/-- The decoded list is no longer than its code. -/
theorem decodeList_length_le (c : Nat) : (decodeList c).length  <=  c := by
  induction c using Nat.strong_induction_on with
  | _ c ih =>
    cases c with
    | zero => simp [decodeList_zero]
    | succ d =>
      rw [decodeList_succ, List.length_cons]
      have hle := unpair_snd_le d
      have := ih d.unpair.2 (Nat.lt_succ_of_le hle)
      omega
end

section
theorem decodeList_mapPairStep (n acc x : Nat) :
    decodeList (mapPairStep n acc x) = Nat.pair n x :: decodeList acc := by
  unfold mapPairStep; rw [decodeList_succ, unpair_pair_fst, unpair_pair_snd]
end

section
/-- The `{0,1}` indicator of `a = 0`, written via truncated subtraction. -/
theorem eqZero_bit (a : Nat) : 1 - a = if a = 0 then 1 else 0 := by split <;> omega
end

section
/-- Two `{0,1}`-bounded naturals agreeing on "`= 1`" are equal. -/
theorem eq_of_le_one_iff_one {x y : Nat} (hx : x  <=  1) (hy : y  <=  1) (h : x = 1
    <->  y = 1) : x = y := by
  by_cases hx1 : x = 1
  * have hy1 := h.mp hx1; omega
  * have hy1 : y  !=  1 := fun hy1 => hx1 (h.mpr hy1)
    omega
end

section
theorem existsListStp_eq (g : Nat  ->  Nat) (acc x p : Nat) :
    existsListStp g (Nat.pair x (Nat.pair acc p)) = selectFn acc 1 (isOne (g (Nat.pair
     x p))) := by
```

```
  unfold existsListStp; simp only [unpair_pair_fst, unpair_pair_snd]
end

section
theorem existsListChar_eq (g : Nat  ->  Nat) (p c : Nat) :
    existsListChar g p c
      = List.foldl (fun acc x => selectFn acc 1 (isOne (g (Nat.pair x p)))) 0 (
    decodeList c) := by
  have hfun : (fun (acc x : Nat) => existsListStp g (Nat.pair x (Nat.pair acc p)))
      = (fun acc x => selectFn acc 1 (isOne (g (Nat.pair x p)))) := by
    funext acc x; exact existsListStp_eq g acc x p
  unfold existsListChar; rw [foldCode_eq', hfun]
end

section
theorem existsListChar_le_one (g : Nat  ->  Nat) (p c : Nat) : existsListChar g p c
    <=  1 := by
  have h : forall  (l : List Nat) (z : Nat), z  <=  1  ->
      List.foldl (fun acc x => existsListStp g (Nat.pair x (Nat.pair acc p))) z l  <=
     1 := by
    intro l
    induction l with
    | nil => intro z hz; simpa using hz
    | cons x xs ih =>
      intro z hz
      simp only [List.foldl_cons]
      apply ih
      rw [existsListStp_eq]
      rcases (show z = 0  \/  z = 1 by omega) with h0 | h1
      * rw [h0, selectFn_zero]; exact isOne_le_one _
      * rw [h1, selectFn_one]
  unfold existsListChar
  rw [foldCode_eq']
  exact h (decodeList c) 0 (Nat.zero_le 1)
end

section
/-- Core induction for the OR-fold: starting from `acc  <=  1`, the fold is `1` iff `
    acc = 1` or some
list entry passes the decider `g`. -/
theorem existsList_foldl_eq_one_iff (g : Nat  ->  Nat) (p : Nat) :
    forall  (l : List Nat) (acc : Nat), acc  <=  1  ->
      (List.foldl (fun acc x => selectFn acc 1 (isOne (g (Nat.pair x p)))) acc l = 1
    <->
        acc = 1  \/  exists  x  in  l, g (Nat.pair x p) = 1) := by
  intro l
  induction l with
  | nil => intro acc _; simp only [List.foldl_nil, List.not_mem_nil, false_and,
    exists_false,
      or_false]
  | cons x l ih =>
    intro acc hacc
    rw [List.foldl_cons]
    set acc' := selectFn acc 1 (isOne (g (Nat.pair x p))) with hacc'
    have hacc'le : acc'  <=  1 := by
      rw [hacc']
```

```
      rcases (show acc = 0  \/  acc = 1 by omega) with h | h
      * rw [h, selectFn_zero]; exact isOne_le_one _
      * rw [h, selectFn_one]
    rw [ih acc' hacc'le]
    constructor
    * rintro (hacc'1 | <y, hy, hyg>)
      * rw [hacc'] at hacc'1
        rcases (show acc = 0  \/  acc = 1 by omega) with h | h
        * rw [h, selectFn_zero, isOne_eq_one_iff] at hacc'1
          exact Or.inr <x, List.mem_cons.mpr (Or.inl rfl), hacc'1>
        * exact Or.inl h
      * exact Or.inr <y, List.mem_cons.mpr (Or.inr hy), hyg>
    * rintro (hacc1 | <y, hy, hyg>)
      * exact Or.inl (by rw [hacc', hacc1, selectFn_one])
      * rcases List.mem_cons.mp hy with rfl | hy'
        * refine Or.inl ?_
          rw [hacc']
          rcases (show acc = 0  \/  acc = 1 by omega) with h | h
          * rw [h, selectFn_zero, isOne_eq_one_iff]; exact hyg
          * rw [h, selectFn_one]
        * exact Or.inr <y, hy', hyg>
end

section
theorem existsListChar_eq_one_iff (g : Nat  ->  Nat) (p c : Nat) :
    existsListChar g p c = 1  <->  exists  x  in  decodeList c, g (Nat.pair x p) = 1
    := by
  rw [existsListChar_eq, existsList_foldl_eq_one_iff g p (decodeList c) 0 (Nat.zero_le
    1)]
  simp only [Nat.zero_ne_one, false_or]
end

section
theorem fuelTableStep_nth {bodyLookup : Nat  ->  Nat} {bound prevTable i : Nat} (hi :
    i  <=  bound) (d : Nat) :
    nthCode (fuelTableStep bodyLookup bound prevTable) i d = bodyLookup (Nat.pair
    prevTable i) := by
  unfold fuelTableStep
  rw [tabCode_nth_lt (show i < bound + 1 by omega)]
  simp only [unpair_pair_fst, unpair_pair_snd]
end

section
theorem fuelTable_succ_nth {bodyLookup : Nat  ->  Nat} {bound fuel i : Nat} (hi : i
    <=  bound) (d : Nat) :
    nthCode (fuelTable bodyLookup bound (fuel + 1)) i d =
      bodyLookup (Nat.pair (fuelTable bodyLookup bound fuel) i) :=
  fuelTableStep_nth hi d
end

section
theorem fuelTable_zero_nth {bodyLookup : Nat  ->  Nat} {bound i : Nat} (hi : i  <=
    bound) (d : Nat) :
    nthCode (fuelTable bodyLookup bound 0) i d = 0 := by
  show nthCode (tabCode (fun _ => 0) (Nat.pair (bound + 1) 0) (bound + 1)) i d = 0
  rw [tabCode_nth_lt (show i < bound + 1 by omega)]
```

```
end

section
/-- **Course-of-values correctness.** If `bodyLookup` faithfully implements `Body` via
    table
lookup (`hlookup`), and `Body`'s result at `c` only depends on `prev` restricted to `[0,
     c]`
(`hlocal` -- true whenever `Body`'s own recursive calls are all ` <=  c`, e.g. via `Nat
    .unpair`
projections), then `fuelTable` correctly tabulates the `0`/`Body` fuel recursion on
    `[0, bound]`. -/
theorem fuelTable_eq_of_recursion {F : Nat  ->  Nat  ->  Nat} {Body : (Nat  ->  Nat)
    ->  Nat  ->  Nat} {bodyLookup : Nat  ->  Nat}
    (hF0 : forall  c, F 0 c = 0) (hFsucc : forall  fuel c, F (fuel + 1) c = Body (F
    fuel) c)
    (hlookup : forall  table c, bodyLookup (Nat.pair table c) = Body (fun c' =>
    nthCode table c' 0) c)
    (hlocal : forall  (f g : Nat  ->  Nat) c, (forall  c'  <=  c, f c' = g c')  ->
    Body f c = Body g c) :
    forall  fuel bound c, c  <=  bound  ->  nthCode (fuelTable bodyLookup bound fuel)
    c 0 = F fuel c := by
  intro fuel
  induction fuel with
  | zero => intro bound c hc; rw [fuelTable_zero_nth hc, hF0]
  | succ fuel ih =>
    intro bound c hc
    rw [fuelTable_succ_nth hc, hlookup, hFsucc]
    exact hlocal _ _ c fun c' hc' => ih bound c' (le_trans hc' hc)
end

section
theorem autStateCardFuelChar_fuelTable_eq :
    forall  fuel bound c, c  <=  bound  ->
      nthCode (fuelTable autStateCardFuelCharLookup bound fuel) c 0 =
    autStateCardFuelChar fuel c :=
  fuelTable_eq_of_recursion (F := autStateCardFuelChar) (Body :=
    autStateCardFuelCharBody)
    (fun _ => rfl) (fun _ _ => rfl)
    (fun table c => by unfold autStateCardFuelCharLookup; simp)
    autStateCardFuelCharBody_local
end

section
theorem decodeFuelOkChar_fuelTable_eq :
    forall  fuel bound c, c  <=  bound  ->
      nthCode (fuelTable decodeFuelOkCharLookup bound fuel) c 0 = decodeFuelOkChar
    fuel c :=
  fuelTable_eq_of_recursion (F := decodeFuelOkChar) (Body := decodeFuelOkCharBody)
    (fun _ => rfl) (fun _ _ => rfl)
    (fun table c => by unfold decodeFuelOkCharLookup; simp)
    decodeFuelOkCharBody_local
end

section
/-- The `{0,1}` indicator of `2  <=  a`, written via truncated subtraction. -/
theorem geTwo_bit (a : Nat) : 1 - (2 - a) = if 2  <=  a then 1 else 0 := by split <;>
```

```
    omega
end

section
/-- Choice-free `getD` of a left append. -/
theorem getD_append_cf {b : Type*} (l l' : List b) (d : b) :
    forall  {n : Nat}, n < l.length  ->  (l ++ l').getD n d = l.getD n d := by
  induction l with
  | nil => intro n h; exact absurd h (Nat.not_lt_zero n)
  | cons a t ih =>
    intro n h
    cases n with
    | zero => rw [List.cons_append, List.getD_cons_zero, List.getD_cons_zero]
    | succ m =>
      rw [List.cons_append, List.getD_cons_succ, List.getD_cons_succ]
      exact ih (by rw [List.length_cons] at h; omega)

end

section
/-- Choice-free `getD` of a right append. -/
theorem getD_append_right_cf {b : Type*} (l l' : List b) (d : b) :
    forall  {n : Nat}, l.length  <=  n  ->  (l ++ l').getD n d = l'.getD (n - l.length
    ) d := by
  induction l with
  | nil => intro n _; rw [List.nil_append, List.length_nil, Nat.sub_zero]
  | cons a t ih =>
    intro n h
    cases n with
    | zero => exact absurd h (by rw [List.length_cons]; exact Nat.not_succ_le_zero _)
    | succ m =>
      rw [List.cons_append, List.getD_cons_succ, List.length_cons, Nat.succ_sub_succ]
      exact ih (by rw [List.length_cons] at h; omega)
end

section
theorem appendListCode_eq (c1 c2 : Nat) :
    decodeList (appendListCode c1 c2) = decodeList c1 ++ decodeList c2 := by
  unfold appendListCode
  rw [decodeList_tabCode, listLenChar_eq c1, listLenChar_eq c2]
  apply list_eq_of_getD
  intro i d
  by_cases hi : i < (decodeList c1).length + (decodeList c2).length
  * have hf := appendListTabFn_eq c1 c2 (decodeList c1).length i
    rw [getD_map_range_cf
        (fun j => appendListTabFn (Nat.pair j (Nat.pair c1 (Nat.pair c2 (decodeList c1
    ).length)))) d hi,
      hf]
    by_cases hlt : i < (decodeList c1).length
    * rw [ite_eq_left hlt,
        show (decodeList c1).getD i 0 = (decodeList c1).getD i d from by
          rw [getD_eq_getElem_cf _ 0 hlt, getD_eq_getElem_cf _ d hlt],
        (getD_append_cf (decodeList c1) (decodeList c2) d hlt).symm]
    * rw [ite_eq_right hlt,
        show (decodeList c2).getD (i - (decodeList c1).length) 0 =
            (decodeList c2).getD (i - (decodeList c1).length) d from by
```

```
          have hi2 : i - (decodeList c1).length < (decodeList c2).length := by
            have hle : (decodeList c1).length  <=  i := Nat.le_of_not_gt hlt
            omega
          rw [getD_eq_getElem_cf _ 0 hi2, getD_eq_getElem_cf _ d hi2],
        (getD_append_right_cf (decodeList c1) (decodeList c2) d (Nat.le_of_not_gt hlt)
    ).symm]
  * have hge : (decodeList c1).length + (decodeList c2).length  <=  i := Nat.
    le_of_not_gt hi
    rw [getD_eq_default_cf _ d (by rw [List.length_map, List.length_range]; exact hge)
    ,
      getD_eq_default_cf _ d (by rw [List.length_append]; exact hge)]
end

section
theorem halfParity_spec (n : Nat) : halfParity n = Nat.pair (n / 2) (n % 2) := by
  induction n with
  | zero => rfl
  | succ n ih =>
    show Nat.pair ((halfParity n).unpair.1 + (halfParity n).unpair.2) (1 - (halfParity
     n).unpair.2)
        = Nat.pair ((n + 1) / 2) ((n + 1) % 2)
    rw [ih, unpair_pair_fst, unpair_pair_snd]
    congr 1 <;> omega
end

section
theorem interEqChar_eq_one_iff {fuel : Nat} {e1 e2 : SExpr}
    (h1 : c9b5_sexprDepth e1  <=  fuel) (h2 : c9b5_sexprDepth e2  <=  fuel) :
    interEqChar fuel (c9b5_sexprGodelEncode e1) (c9b5_sexprGodelEncode e2) = 1  <->
      denote e1 = denote e2 := by
  unfold interEqChar
  rw [mulBit_eq_one_iff, subsetBChar_eq_one_iff h1 h2, subsetBChar_eq_one_iff h2 h1,
     <-  Set.Subset.antisymm_iff]
end

section
theorem mapPairStp_eq (n acc x : Nat) :
    mapPairStp (Nat.pair x (Nat.pair acc n)) = mapPairStep n acc x := by
  unfold mapPairStp; simp only [unpair_pair_fst, unpair_pair_snd]
end

section
theorem decodeList_foldl_mapPairStp (n : Nat) (el : List Nat) (acc : Nat) :
    decodeList (List.foldl (fun acc x => mapPairStp (Nat.pair x (Nat.pair acc n))) acc
     el)
      = (el.map (Nat.pair n *)).reverse ++ decodeList acc := by
  induction el generalizing acc with
  | nil => simp
  | cons e el ih =>
    rw [List.foldl_cons, ih, mapPairStp_eq, decodeList_mapPairStep, List.map_cons,
      List.reverse_cons, List.append_assoc, List.singleton_append]
end

section
theorem decodeList_mapPairCode (n c : Nat) :
    decodeList (mapPairCode n c) = ((decodeList c).map (Nat.pair n *)).reverse := by
```

```
  unfold mapPairCode
  rw [foldCode_eq']
  have h := decodeList_foldl_mapPairStp n (decodeList c) 0
  rwa [decodeList_zero, List.append_nil] at h
end

section
theorem matchesBChar_fuelTable_eq :
    forall  fuel bound code, code  <=  bound  ->
      nthCode (fuelTable matchesBCharBodyLookup bound fuel) code 0 =
        matchesBChar fuel code.unpair.1 code.unpair.2 :=
  fuelTable_eq_of_recursion
    (F := fun fuel code => matchesBChar fuel code.unpair.1 code.unpair.2)
    (Body := fun prev code => matchesBCharBody prev code.unpair.1 code.unpair.2)
    (fun _ => rfl) (fun _ _ => rfl)
    (fun table code => by unfold matchesBCharBodyLookup; simp)
    (fun f g code h => matchesBCharBody_local f g code.unpair.1 code.unpair.2 (by
      simpa [pair_unpair] using h))
end

section
theorem primrec_allListChar {g : Nat  ->  Nat} (hg : Nat.Primrec g) :
    Nat.Primrec (fun t => allListChar g t.unpair.2 t.unpair.1) :=
  (primrec_foldCode (primrec_allListStp hg) Nat.Primrec.right (Nat.Primrec.const 1)
    Nat.Primrec.left).of_eq fun _ => rfl
end

section
/-- **Bounded `forall ` over a coded list preserves recursive decidability.** If `q` is
     a recursively
decidable binary relation then `fun c p => forall  e  in  decodeList c, q e p` is too (
    the list is coded by
the first argument, `p` is a free parameter). Choice-free. -/
theorem RecDecidable_2.bForallList {q : Nat  ->  Nat  ->  Prop} (hq : RecDecidable_2 q
    ) :
    RecDecidable_2 (fun c p => forall  e  in  decodeList c, q e p) := by
  obtain <g, hgp, hgs> := hq
  refine <fun t => allListChar g t.unpair.2 t.unpair.1, primrec_allListChar hgp, fun t
     => ?_>
  show (forall  e  in  decodeList t.unpair.1, q e t.unpair.2)  <->  allListChar g t.
    unpair.2 t.unpair.1 = 1
  rw [allListChar_eq_one_iff]
  refine forall_congr' fun e => forall_congr' fun _ => ?_
  have := hgs (Nat.pair e t.unpair.2)
  simp only [unpair_pair_fst, unpair_pair_snd] at this
  exact this
end

section
theorem primrec_appendListTabFn : Nat.Primrec appendListTabFn := by
  have hi := Nat.Primrec.left
  have hp := Nat.Primrec.right
  have hlen1 := Nat.Primrec.right.comp (Nat.Primrec.right.comp hp)
  have hc1 := Nat.Primrec.left.comp hp
  have hc2 := Nat.Primrec.left.comp (Nat.Primrec.right.comp hp)
  have hlt := primrec_isZero.comp
```

```
    (primrec_sub_2 (primrec_add_2 hi (Nat.Primrec.const 1)) hlen1)
  have hleft := primrec_nthCode.comp (hc1.pair (Nat.Primrec.pair hi (Nat.Primrec.const
     0)))
  have hright := primrec_nthCode.comp
    (hc2.pair (Nat.Primrec.pair (primrec_sub_2 hi hlen1) (Nat.Primrec.const 0)))
  exact (primrec_selectFn hlt hleft hright).of_eq fun w => by
    unfold appendListTabFn
    simp only [unpair_pair_fst, unpair_pair_snd]
end

section
theorem primrec_appendListCode : Nat.Primrec (fun t => appendListCode t.unpair.1 t.
    unpair.2) := by
  have hc1 := Nat.Primrec.left
  have hc2 := Nat.Primrec.right
  have hlen1 := primrec_listLenChar.comp hc1
  have hlen2 := primrec_listLenChar.comp hc2
  have hsum := primrec_add_2 hlen1 hlen2
  have hp := hc1.pair (hc2.pair hlen1)
  have hr := (hsum.pair hp).pair hsum
  refine ((primrec_tabCode primrec_appendListTabFn).comp hr).of_eq fun t => by
    unfold appendListCode
    simp only [unpair_pair_fst, unpair_pair_snd]
end

section
theorem primrec_autStateCardFuelChar : forall  fuel, Nat.Primrec (fun c =>
    autStateCardFuelChar fuel c)
  | 0 => Nat.Primrec.const 0
  | fuel + 1 => by
    simpa [autStateCardFuelChar] using
      primrec_autStateCardFuelCharBody (primrec_autStateCardFuelChar fuel)
end

section
theorem primrec_autStateCardFuelCharLookup : Nat.Primrec autStateCardFuelCharLookup :=
     by
  have hleft : Nat.Primrec (fun w => nthCode w.unpair.1 w.unpair.2.unpair.2.unpair.1
    0) :=
    (primrec_nthCode.comp (Nat.Primrec.pair Nat.Primrec.left
      (Nat.Primrec.pair (Nat.Primrec.left.comp (Nat.Primrec.right.comp Nat.Primrec.
    right))
        (Nat.Primrec.const 0)))).of_eq fun w => by simp
  have hright : Nat.Primrec (fun w => nthCode w.unpair.1 w.unpair.2.unpair.2.unpair.2
    0) :=
    (primrec_nthCode.comp (Nat.Primrec.pair Nat.Primrec.left
      (Nat.Primrec.pair (Nat.Primrec.right.comp (Nat.Primrec.right.comp Nat.Primrec.
    right))
        (Nat.Primrec.const 0)))).of_eq fun w => by simp
  have hadd : Nat.Primrec (fun w =>
      nthCode w.unpair.1 w.unpair.2.unpair.2.unpair.1 0 +
        nthCode w.unpair.1 w.unpair.2.unpair.2.unpair.2 0) :=
    primrec_add_2 hleft hright
  have hmul : Nat.Primrec (fun w =>
      nthCode w.unpair.1 w.unpair.2.unpair.2.unpair.1 0 *
        nthCode w.unpair.1 w.unpair.2.unpair.2.unpair.2 0) :=
```

```
    primrec_mul_2 hleft hright
  have hf0 : Nat.Primrec (fun _ : Nat => (1 : Nat)) := Nat.Primrec.const 1
  have hf1 : Nat.Primrec (fun w => listLenChar w.unpair.2.unpair.2 + 2) :=
    primrec_add_2 (primrec_listLenChar.comp (Nat.Primrec.right.comp Nat.Primrec.right)
    )
      (Nat.Primrec.const 2)
  have ht : Nat.Primrec (fun w => w.unpair.2.unpair.1) := Nat.Primrec.left.comp Nat.
    Primrec.right
  have h01 : Nat.Primrec (fun w => isOne (1 - w.unpair.2.unpair.1)) :=
    primrec_isOne.comp (primrec_sub_2 (Nat.Primrec.const 1) ht)
  have h12 : Nat.Primrec (fun w => isOne (2 - w.unpair.2.unpair.1)) :=
    primrec_isOne.comp (primrec_sub_2 (Nat.Primrec.const 2) ht)
  have h23 : Nat.Primrec (fun w => isOne (3 - w.unpair.2.unpair.1)) :=
    primrec_isOne.comp (primrec_sub_2 (Nat.Primrec.const 3) ht)
  have h34 : Nat.Primrec (fun w => isOne (4 - w.unpair.2.unpair.1)) :=
    primrec_isOne.comp (primrec_sub_2 (Nat.Primrec.const 4) ht)
  refine (primrec_selectFn h01 hf0
    (primrec_selectFn h12 hf1
      (primrec_selectFn h23 hadd
        (primrec_selectFn h34 hmul (Nat.Primrec.const 0))))).of_eq fun w => ?_
  show _ = autStateCardFuelCharBody (fun c' => nthCode w.unpair.1 c' 0) w.unpair.2
  rw [autStateCardFuelCharBody_eq]
  match tag : w.unpair.2.unpair.1 with
  | 0 =>
    have h10 : (1 - 0 : Nat) = 1 := rfl
    simp only [h10, isOne_one, selectFn_one]
  | 1 =>
    have h01' : (1 - 1 : Nat) = 0 := rfl
    have h12' : (2 - 1 : Nat) = 1 := rfl
    simp only [h01', h12', isOne_zero, isOne_one, selectFn_zero, selectFn_one]
  | 2 =>
    have h01' : (1 - 2 : Nat) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h12' : (2 - 2 : Nat) = 0 := rfl
    have h23' : (3 - 2 : Nat) = 1 := rfl
    simp only [h01', h23', isOne_zero, isOne_one, selectFn_zero, selectFn_one]
  | 3 =>
    have h01' : (1 - 3 : Nat) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h12' : (2 - 3 : Nat) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h23' : (3 - 3 : Nat) = 0 := rfl
    have h34' : (4 - 3 : Nat) = 1 := rfl
    simp only [h01', h34', isOne_zero, isOne_one, selectFn_zero, selectFn_one]
  | t + 4 =>
    have h01' : (1 - (t + 4) : Nat) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h12' : (2 - (t + 4) : Nat) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h23' : (3 - (t + 4) : Nat) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h34' : (4 - (t + 4) : Nat) = 0 := Nat.sub_eq_zero_of_le (by omega)
    simp only [h01', h12', h23', h34', isOne_zero, selectFn_zero]
end

section
/-- Parametrized variant of `bForallFn`'s joint primrec (mirrors `
    primrec_bExistsFn_param`,
swapping `selectFn`'s branches to match `bForallFn`'s `1`-base / AND-style step). -/
theorem primrec_bForallFn_param {g : Nat  ->  Nat} (hg : Nat.Primrec g) :
    Nat.Primrec (fun t => bForallFn (fun p => g (Nat.pair t.unpair.1.unpair.1 p))
      t.unpair.1.unpair.2 t.unpair.2) := by
```

```
  have hGfn : Nat.Primrec (fun w =>
      selectFn w.unpair.2.unpair.2
        (isOne (g (Nat.pair w.unpair.1.unpair.1
          (Nat.pair w.unpair.2.unpair.1 w.unpair.1.unpair.2)))) 0) :=
    primrec_selectFn (Nat.Primrec.right.comp Nat.Primrec.right)
      (primrec_isOne.comp (hg.comp
        (Nat.Primrec.pair (Nat.Primrec.left.comp Nat.Primrec.left)
          (Nat.Primrec.pair (Nat.Primrec.left.comp Nat.Primrec.right)
            (Nat.Primrec.right.comp Nat.Primrec.left)))))
      (Nat.Primrec.const 0)
  have hprec := Nat.Primrec.prec (Nat.Primrec.const 1) hGfn
  refine (hprec.comp (Nat.Primrec.left.pair Nat.Primrec.right)).of_eq fun t => ?_
  simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd]
  rfl
end

section
theorem primrec_codeBound : Nat.Primrec codeBound := by
  have hstep : Nat.Primrec (fun w => Nat.pair 1 w.unpair.2 + 1) :=
    primrec_add_2 (Nat.Primrec.pair (Nat.Primrec.const 1) Nat.Primrec.right)
      (Nat.Primrec.const 1)
  have haux : forall  n, Nat.rec (motive := fun _ => Nat) 1
      (fun y IH => Nat.pair 1 (Nat.pair y IH).unpair.2 + 1) n = codeBound n := by
    intro n
    induction n with
    | zero => simp [codeBound]
    | succ n ih =>
      simp only [unpair_pair_snd] at ih  |-
      simp [codeBound, ih]
  exact (Nat.Primrec.prec1 1 hstep).of_eq haux
end

section
theorem primrec_decodeFuelOkChar : forall  fuel, Nat.Primrec (fun c =>
    decodeFuelOkChar fuel c)
  | 0 => Nat.Primrec.const 0
  | fuel + 1 => by
    simpa [decodeFuelOkChar] using
      primrec_decodeFuelOkCharBody (primrec_decodeFuelOkChar fuel)
end

section
theorem primrec_decodeFuelOkCharLookup : Nat.Primrec decodeFuelOkCharLookup := by
  have hleft : Nat.Primrec (fun w => nthCode w.unpair.1 w.unpair.2.unpair.2.unpair.1
    0) :=
    (primrec_nthCode.comp (Nat.Primrec.pair Nat.Primrec.left
      (Nat.Primrec.pair (Nat.Primrec.left.comp (Nat.Primrec.right.comp Nat.Primrec.
    right))
        (Nat.Primrec.const 0)))).of_eq fun w => by simp
  have hright : Nat.Primrec (fun w => nthCode w.unpair.1 w.unpair.2.unpair.2.unpair.2
    0) :=
    (primrec_nthCode.comp (Nat.Primrec.pair Nat.Primrec.left
      (Nat.Primrec.pair (Nat.Primrec.right.comp (Nat.Primrec.right.comp Nat.Primrec.
    right))
        (Nat.Primrec.const 0)))).of_eq fun w => by simp
  have hsub : Nat.Primrec (fun w =>
```

```
      mulBit (nthCode w.unpair.1 w.unpair.2.unpair.2.unpair.1 0)
        (nthCode w.unpair.1 w.unpair.2.unpair.2.unpair.2 0)) :=
    primrec_mul_2 hleft hright
  have hf0 : Nat.Primrec (fun w => selectFn (isOne (1 - w.unpair.2.unpair.2)) 1 0) :=
    primrec_selectFn
      (primrec_isOne.comp (primrec_sub_2 (Nat.Primrec.const 1) (Nat.Primrec.right.comp
     Nat.Primrec.right)))
      (Nat.Primrec.const 1) (Nat.Primrec.const 0)
  have hf1 : Nat.Primrec (fun w => allBinDigitsChar w.unpair.2.unpair.2) :=
    primrec_allBinDigitsChar.comp (Nat.Primrec.right.comp Nat.Primrec.right)
  have ht : Nat.Primrec (fun w => w.unpair.2.unpair.1) := Nat.Primrec.left.comp Nat.
    Primrec.right
  have h01 : Nat.Primrec (fun w => isOne (1 - w.unpair.2.unpair.1)) :=
    primrec_isOne.comp (primrec_sub_2 (Nat.Primrec.const 1) ht)
  have h12 : Nat.Primrec (fun w => isOne (2 - w.unpair.2.unpair.1)) :=
    primrec_isOne.comp (primrec_sub_2 (Nat.Primrec.const 2) ht)
  have h23 : Nat.Primrec (fun w => isOne (3 - w.unpair.2.unpair.1)) :=
    primrec_isOne.comp (primrec_sub_2 (Nat.Primrec.const 3) ht)
  have h34 : Nat.Primrec (fun w => isOne (4 - w.unpair.2.unpair.1)) :=
    primrec_isOne.comp (primrec_sub_2 (Nat.Primrec.const 4) ht)
  refine (primrec_selectFn h01 hf0
    (primrec_selectFn h12 hf1
      (primrec_selectFn h23 hsub
        (primrec_selectFn h34 hsub (Nat.Primrec.const 0))))).of_eq fun w => ?_
  show _ = decodeFuelOkCharBody (fun c' => nthCode w.unpair.1 c' 0) w.unpair.2
  rw [decodeFuelOkCharBody_eq]
  match tag : w.unpair.2.unpair.1 with
  | 0 =>
    have h10 : (1 - 0 : Nat) = 1 := rfl
    simp only [h10, isOne_one, selectFn_one]
  | 1 =>
    have h01' : (1 - 1 : Nat) = 0 := rfl
    have h12' : (2 - 1 : Nat) = 1 := rfl
    simp only [h01', h12', isOne_zero, isOne_one, selectFn_zero, selectFn_one]
  | 2 =>
    have h01' : (1 - 2 : Nat) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h12' : (2 - 2 : Nat) = 0 := rfl
    have h23' : (3 - 2 : Nat) = 1 := rfl
    simp only [h01', h23', isOne_zero, isOne_one, selectFn_zero, selectFn_one]
  | 3 =>
    have h01' : (1 - 3 : Nat) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h12' : (2 - 3 : Nat) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h23' : (3 - 3 : Nat) = 0 := rfl
    have h34' : (4 - 3 : Nat) = 1 := rfl
    simp only [h01', h34', isOne_zero, isOne_one, selectFn_zero, selectFn_one]
  | t + 4 =>
    have h01' : (1 - (t + 4) : Nat) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h12' : (2 - (t + 4) : Nat) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h23' : (3 - (t + 4) : Nat) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h34' : (4 - (t + 4) : Nat) = 0 := Nat.sub_eq_zero_of_le (by omega)
    simp only [h01', h12', h23', h34', isOne_zero, selectFn_zero]
end

section
theorem primrec_existsListStp {g : Nat  ->  Nat} (hg : Nat.Primrec g) :
    Nat.Primrec (existsListStp g) := by
```

```
  have hacc : Nat.Primrec (fun w : Nat => w.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.right
  have hx : Nat.Primrec (fun w : Nat => w.unpair.1) := Nat.Primrec.left
  have hp : Nat.Primrec (fun w : Nat => w.unpair.2.unpair.2) :=
    Nat.Primrec.right.comp Nat.Primrec.right
  have hcall : Nat.Primrec (fun w : Nat => isOne (g (Nat.pair w.unpair.1 w.unpair.2.
    unpair.2))) :=
    primrec_isOne.comp (hg.comp (hx.pair hp))
  exact (primrec_selectFn hacc (Nat.Primrec.const 1) hcall).of_eq fun _ => rfl
end

section
theorem primrec_existsListChar {g : Nat  ->  Nat} (hg : Nat.Primrec g) :
    Nat.Primrec (fun t => existsListChar g t.unpair.2 t.unpair.1) :=
  (primrec_foldCode (primrec_existsListStp hg) Nat.Primrec.right (Nat.Primrec.const 0)
    Nat.Primrec.left).of_eq fun _ => rfl
end

section
/-- **Bounded `exists ` over a coded list preserves recursive decidability.** If `q` is
     a recursively
decidable binary relation then `fun c p => exists  e  in  decodeList c, q e p` is too.
    Choice-free. -/
theorem RecDecidable_2.bExistsList {q : Nat  ->  Nat  ->  Prop} (hq : RecDecidable_2 q
    ) :
    RecDecidable_2 (fun c p => exists  e  in  decodeList c, q e p) := by
  obtain <g, hgp, hgs> := hq
  refine <fun t => existsListChar g t.unpair.2 t.unpair.1, primrec_existsListChar hgp,
     fun t => ?_>
  show (exists  e  in  decodeList t.unpair.1, q e t.unpair.2)  <->  existsListChar g t
    .unpair.2 t.unpair.1 = 1
  rw [existsListChar_eq_one_iff]
  refine exists_congr fun e => and_congr_right fun _ => ?_
  have := hgs (Nat.pair e t.unpair.2)
  simp only [unpair_pair_fst, unpair_pair_snd] at this
  exact this
end

section
set_option maxHeartbeats 1000000 in
/-- `fuelTable bodyLookup * *` is primitive recursive in `(bound, fuel)` (coded `pair
    bound
fuel`), whenever `bodyLookup` is. Mirrors `primrec_tabCode`'s `Nat.Primrec.prec`
    packaging. -/
theorem primrec_fuelTable {bodyLookup : Nat  ->  Nat} (hbodyLookup : Nat.Primrec
    bodyLookup) :
    Nat.Primrec (fun t => fuelTable bodyLookup t.unpair.1 t.unpair.2) := by
  have hgLookup : Nat.Primrec (fun v => bodyLookup (Nat.pair v.unpair.2 v.unpair.1))
    :=
    hbodyLookup.comp (Nat.Primrec.pair Nat.Primrec.right Nat.Primrec.left)
  have htab := primrec_tabCode hgLookup
  have hf : Nat.Primrec (fun bound => tabCode (fun _ => 0) (Nat.pair (bound + 1) 0) (
    bound + 1)) :=
    ((primrec_tabCode (Nat.Primrec.const 0)).comp
      (Nat.Primrec.pair
        (Nat.Primrec.pair (primrec_add_2 Nat.Primrec.id (Nat.Primrec.const 1)) (Nat.
```

```
    Primrec.const 0))
        (primrec_add_2 Nat.Primrec.id (Nat.Primrec.const 1)))).of_eq fun bound => by
    simp
  have hpack : Nat.Primrec (fun w =>
      Nat.pair (Nat.pair (w.unpair.1 + 1) w.unpair.2.unpair.2) (w.unpair.1 + 1)) :=
    Nat.Primrec.pair
      (Nat.Primrec.pair (primrec_add_2 Nat.Primrec.left (Nat.Primrec.const 1))
        (Nat.Primrec.right.comp Nat.Primrec.right))
      (primrec_add_2 Nat.Primrec.left (Nat.Primrec.const 1))
  have hg : Nat.Primrec (fun w => fuelTableStep bodyLookup w.unpair.1 w.unpair.2.
    unpair.2) :=
    (htab.comp hpack).of_eq fun w => by simp [fuelTableStep]
  have haux : forall  bound fuel, Nat.rec (motive := fun _ => Nat)


      (tabCode (fun _ => 0) (Nat.pair (bound + 1) 0) (bound + 1))
      (fun _ IH => fuelTableStep bodyLookup bound IH) fuel = fuelTable bodyLookup
    bound fuel := by
    intro bound fuel
    induction fuel with
    | zero => rfl
    | succ fuel ih => show fuelTableStep bodyLookup bound _ = fuelTableStep bodyLookup
     bound _; rw [ih]
  refine (Nat.Primrec.prec hf hg).of_eq fun t => ?_
  simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd]
  exact haux t.unpair.1 t.unpair.2
end

section
/-- **`autStateCardFuelChar` is jointly primitive recursive in `(fuel, code)`.** -/
theorem primrec_autStateCardFuelChar2 :
    Nat.Primrec (fun s => autStateCardFuelChar s.unpair.1 s.unpair.2) := by
  have hft := primrec_fuelTable primrec_autStateCardFuelCharLookup
  have hpack : Nat.Primrec (fun s : Nat => Nat.pair s.unpair.2 s.unpair.1) :=
    Nat.Primrec.pair Nat.Primrec.right Nat.Primrec.left
  have hcomp : Nat.Primrec (fun s => fuelTable autStateCardFuelCharLookup s.unpair.2 s
    .unpair.1) :=
    (hft.comp hpack).of_eq fun s => by simp
  have hnth : Nat.Primrec
      (fun s => nthCode (fuelTable autStateCardFuelCharLookup s.unpair.2 s.unpair.1) s
    .unpair.2 0) :=
    (primrec_nthCode.comp
      (Nat.Primrec.pair hcomp (Nat.Primrec.pair Nat.Primrec.right (Nat.Primrec.const
    0)))).of_eq
      fun s => by simp
  exact hnth.of_eq fun s =>
    autStateCardFuelChar_fuelTable_eq s.unpair.1 s.unpair.2 s.unpair.2 (le_refl _)
end

section
/-- **`decodeFuelOkChar` is jointly primitive recursive in `(fuel, code)`.** -/
theorem primrec_decodeFuelOkChar2 : Nat.Primrec (fun s => decodeFuelOkChar s.unpair.1
    s.unpair.2) := by
  have hft := primrec_fuelTable primrec_decodeFuelOkCharLookup
  have hpack : Nat.Primrec (fun s : Nat => Nat.pair s.unpair.2 s.unpair.1) :=
    Nat.Primrec.pair Nat.Primrec.right Nat.Primrec.left
  have hcomp : Nat.Primrec (fun s => fuelTable decodeFuelOkCharLookup s.unpair.2 s.
```

```
    unpair.1) :=
    (hft.comp hpack).of_eq fun s => by simp
  have hnth : Nat.Primrec
      (fun s => nthCode (fuelTable decodeFuelOkCharLookup s.unpair.2 s.unpair.1) s.
    unpair.2 0) :=
    (primrec_nthCode.comp
      (Nat.Primrec.pair hcomp (Nat.Primrec.pair Nat.Primrec.right (Nat.Primrec.const
    0)))).of_eq
      fun s => by simp
  exact hnth.of_eq fun s => decodeFuelOkChar_fuelTable_eq s.unpair.1 s.unpair.2 s.
    unpair.2 (le_refl _)
end

section
theorem primrec_halfParity : Nat.Primrec halfParity := by
  have hIH : Nat.Primrec (fun w => w.unpair.2.unpair.2) := Nat.Primrec.right.comp Nat.
    Primrec.right
  have hstep : Nat.Primrec (fun w => Nat.pair (w.unpair.2.unpair.2.unpair.1 + w.unpair
    .2.unpair.2.unpair.2)
      (1 - w.unpair.2.unpair.2.unpair.2)) :=
    (primrec_add_2 (Nat.Primrec.left.comp hIH) (Nat.Primrec.right.comp hIH)).pair
      (primrec_sub_2 (Nat.Primrec.const 1) (Nat.Primrec.right.comp hIH))
  refine ((Nat.Primrec.prec (Nat.Primrec.const 0) hstep).comp
    ((Nat.Primrec.const 0).pair primrec_id)).of_eq (fun n => ?_)
  simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd, id_eq]
  rfl
end

section
theorem primrec_div2 : Nat.Primrec (fun n => n / 2) :=
  (Nat.Primrec.left.comp primrec_halfParity).of_eq fun n => by
    rw [halfParity_spec, unpair_pair_fst]
end

section
/-- Primitive-recursive `if` on a `{0,1}` condition. -/
theorem primrec_ite {c t f : Nat  ->  Nat} (hc : Nat.Primrec c) (ht : Nat.Primrec t) (
    hf : Nat.Primrec f) :
    Nat.Primrec (fun n => selectFn (c n) (t n) (f n)) :=
  primrec_selectFn hc ht hf
end

section
theorem primrec_mapPairStp : Nat.Primrec mapPairStp := by
  have h1 : Nat.Primrec (fun w : Nat => w.unpair.1) := Nat.Primrec.left
  have h21 : Nat.Primrec (fun w : Nat => w.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.right
  have h22 : Nat.Primrec (fun w : Nat => w.unpair.2.unpair.2) :=
    Nat.Primrec.right.comp Nat.Primrec.right
  exact (Nat.Primrec.succ.comp ((h22.pair h1).pair h21)).of_eq (fun _ => rfl)
end

section
theorem primrec_mapPairCode : Nat.Primrec (fun t => mapPairCode t.unpair.1 t.unpair.2)
     :=
  (primrec_foldCode primrec_mapPairStp Nat.Primrec.left (Nat.Primrec.const 0)
```

```
    Nat.Primrec.right).of_eq (fun _ => rfl)
end

section
/-- **Parameterised bounded `forall ` over a coded list preserves recursive
    enumerability.** If `p` is an
r.e. binary relation then `fun t => forall  e  in  decodeList t.2, p t.1 e` is r.e.: re
    -code the list into the
pairs `<t.1, e>` (`mapPairCode`) and apply the unparameterised `forall_mem_decodeList`.
     -/
theorem REPred.forall_mem_decodeList_2 {p : Nat  ->  Nat  ->  Prop} (hp : REPred_2 p)
    :
    REPred (fun t => forall  e  in  decodeList t.unpair.2, p t.unpair.1 e) := by
  have hp' : REPred (fun s => p s.unpair.1 s.unpair.2) := hp
  have hbase : REPred (fun c => forall  e'  in  decodeList c, p e'.unpair.1 e'.unpair
    .2) :=
    hp'.forall_mem_decodeList
  refine REPred.of_iff (fun t => ?_) (hbase.comp primrec_mapPairCode)
  rw [decodeList_mapPairCode]
  simp only [List.mem_reverse, List.mem_map]
  constructor
  * rintro hall e' <e, he, rfl>
    simp only [unpair_pair_fst, unpair_pair_snd]
    exact hall e he
  * intro hall e he
    simpa only [unpair_pair_fst, unpair_pair_snd]
      using hall (Nat.pair t.unpair.1 e) <e, he, rfl>
end

section
theorem primrec_matchesBCatGLookup : Nat.Primrec matchesBCatGLookup := by
  have hi : Nat.Primrec (fun w => w.unpair.2.unpair.1) := Nat.Primrec.left.comp Nat.
    Primrec.right
  have hc : Nat.Primrec (fun w => w.unpair.2.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp (Nat.Primrec.right.comp Nat.Primrec.right)
  have hcw : Nat.Primrec (fun w => w.unpair.2.unpair.2.unpair.2) :=
    Nat.Primrec.right.comp (Nat.Primrec.right.comp Nat.Primrec.right)
  have ha : Nat.Primrec (fun w => w.unpair.2.unpair.2.unpair.1.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp (Nat.Primrec.right.comp hc)
  have hb : Nat.Primrec (fun w => w.unpair.2.unpair.2.unpair.1.unpair.2.unpair.2) :=
    Nat.Primrec.right.comp (Nat.Primrec.right.comp hc)
  have htake : Nat.Primrec (fun w => takeCode w.unpair.2.unpair.1 w.unpair.2.unpair.2.
    unpair.2) :=
    (primrec_takeCode.comp (Nat.Primrec.pair hi hcw)).of_eq fun w => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  have hdrop : Nat.Primrec (fun w => dropCode w.unpair.2.unpair.1 w.unpair.2.unpair.2.
    unpair.2) :=
    (primrec_dropCode.comp (Nat.Primrec.pair hi hcw)).of_eq fun w => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  have h1 : Nat.Primrec (fun w => nthCode w.unpair.1
      (Nat.pair w.unpair.2.unpair.2.unpair.1.unpair.2.unpair.1
        (takeCode w.unpair.2.unpair.1 w.unpair.2.unpair.2.unpair.2)) 0) :=
    (primrec_nthCode.comp (Nat.Primrec.pair Nat.Primrec.left
      (Nat.Primrec.pair (Nat.Primrec.pair ha htake) (Nat.Primrec.const 0)))).of_eq fun
     w => by
      simp only [unpair_pair_fst, unpair_pair_snd]
```

```
  have h2 : Nat.Primrec (fun w => nthCode w.unpair.1
      (Nat.pair w.unpair.2.unpair.2.unpair.1.unpair.2.unpair.2
        (dropCode w.unpair.2.unpair.1 w.unpair.2.unpair.2.unpair.2)) 0) :=
    (primrec_nthCode.comp (Nat.Primrec.pair Nat.Primrec.left
      (Nat.Primrec.pair (Nat.Primrec.pair hb hdrop) (Nat.Primrec.const 0)))).of_eq fun
     w => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  exact (primrec_mul_2 h1 h2).of_eq fun w => by
    dsimp [matchesBCatGLookup, matchesBCatG, mulBit]
end

section
theorem primrec_matchesBChar : forall  fuel, Nat.Primrec (fun t => matchesBChar fuel t
    .unpair.1 t.unpair.2)
  | 0 => Nat.Primrec.const 0
  | fuel + 1 => by
    simpa [matchesBChar] using
      primrec_matchesBCharBody (primrec_matchesBChar fuel)
end

section
theorem primrec_decideNonemptyBChar (fuel : Nat) :
    Nat.Primrec (fun c_e => decideNonemptyBChar fuel c_e) := by
  have hg : Nat.Primrec (fun w =>
      mulBit (allBinDigitsChar w.unpair.1) (matchesBChar fuel w.unpair.2 w.unpair.1))
    :=
    (primrec_mulBit.comp (Nat.Primrec.pair (primrec_allBinDigitsChar.comp Nat.Primrec.
    left)
      ((primrec_matchesBChar fuel).comp (Nat.Primrec.pair Nat.Primrec.right Nat.
    Primrec.left)))).of_eq
      fun w => by simp
  have hpack : Nat.Primrec (fun c_e =>
      Nat.pair c_e (codeBound (autStateCardFuelChar fuel c_e))) :=
    Nat.Primrec.pair Nat.Primrec.id
      (primrec_codeBound.comp (primrec_autStateCardFuelChar fuel))
  refine ((primrec_bExistsFn hg).comp hpack).of_eq fun c_e => ?_
  simp [decideNonemptyBChar]
end

section
theorem primrec_consistentBChar (fuel : Nat) :
    Nat.Primrec (fun t => consistentBChar fuel t.unpair.1 t.unpair.2) :=
  (primrec_decideNonemptyBChar fuel).comp
    (Nat.Primrec.pair (Nat.Primrec.const 3)
      (Nat.Primrec.pair Nat.Primrec.left Nat.Primrec.right))
end

section
set_option maxHeartbeats 4000000 in
theorem primrec_matchesBCharBodyLookup : Nat.Primrec matchesBCharBodyLookup := by
  have htable : Nat.Primrec (fun w => w.unpair.1) := Nat.Primrec.left
  have hc : Nat.Primrec (fun w => w.unpair.2.unpair.1) := Nat.Primrec.left.comp Nat.
    Primrec.right
  have hcw : Nat.Primrec (fun w => w.unpair.2.unpair.2) := Nat.Primrec.right.comp Nat.
    Primrec.right
  have htag : Nat.Primrec (fun w => w.unpair.2.unpair.1.unpair.1) := Nat.Primrec.left.
```

```
  comp hc
have h01 := primrec_isOne.comp (primrec_sub_2 (Nat.Primrec.const 1) htag)
have h12 := primrec_isOne.comp (primrec_sub_2 (Nat.Primrec.const 2) htag)
have h23 := primrec_isOne.comp (primrec_sub_2 (Nat.Primrec.const 3) htag)
have h34 := primrec_isOne.comp (primrec_sub_2 (Nat.Primrec.const 4) htag)
have hf1 : Nat.Primrec (fun w => listEqChar w.unpair.2.unpair.2 w.unpair.2.unpair.1.
  unpair.2) :=
   (primrec_listEqChar.comp (Nat.Primrec.pair hcw (Nat.Primrec.right.comp hc))).of_eq
    fun w => by
     simp only [unpair_pair_fst, unpair_pair_snd]
have hg : Nat.Primrec (fun w'' => matchesBCatGLookup
     (Nat.pair w''.unpair.2.unpair.1 (Nat.pair w''.unpair.1 w''.unpair.2.unpair.2)))
  :=
   (primrec_matchesBCatGLookup.comp (Nat.Primrec.pair (Nat.Primrec.left.comp Nat.
  Primrec.right)
     (Nat.Primrec.pair Nat.Primrec.left (Nat.Primrec.right.comp Nat.Primrec.right))))
  .of_eq
     fun _ => rfl
have hcatpack : Nat.Primrec (fun w => Nat.pair
     (Nat.pair w.unpair.1 (Nat.pair w.unpair.2.unpair.1 w.unpair.2.unpair.2))
     (listLenChar w.unpair.2.unpair.2 + 1)) :=
  Nat.Primrec.pair (Nat.Primrec.pair htable (Nat.Primrec.pair hc hcw))
     (primrec_add_2 (primrec_listLenChar.comp hcw) (Nat.Primrec.const 1))
have hcat : Nat.Primrec (fun w => bExistsFn
     (fun w'' => matchesBCatGLookup
       (Nat.pair w''.unpair.2.unpair.1 (Nat.pair w''.unpair.1 w''.unpair.2.unpair.2))
  )
     (Nat.pair w.unpair.1 (Nat.pair w.unpair.2.unpair.1 w.unpair.2.unpair.2))
     (listLenChar w.unpair.2.unpair.2 + 1)) :=
   ((primrec_bExistsFn hg).comp hcatpack).of_eq fun w => by
     simp only [unpair_pair_fst, unpair_pair_snd]
have ha : Nat.Primrec (fun w => w.unpair.2.unpair.1.unpair.2.unpair.1) :=
  Nat.Primrec.left.comp (Nat.Primrec.right.comp hc)
have hb : Nat.Primrec (fun w => w.unpair.2.unpair.1.unpair.2.unpair.2) :=
  Nat.Primrec.right.comp (Nat.Primrec.right.comp hc)
have hlk1 : Nat.Primrec (fun w => nthCode w.unpair.1
     (Nat.pair w.unpair.2.unpair.1.unpair.2.unpair.1 w.unpair.2.unpair.2) 0) :=
   (primrec_nthCode.comp (Nat.Primrec.pair htable
     (Nat.Primrec.pair (Nat.Primrec.pair ha hcw) (Nat.Primrec.const 0)))).of_eq fun w
    => by
     simp only [unpair_pair_fst, unpair_pair_snd]
have hlk2 : Nat.Primrec (fun w => nthCode w.unpair.1
     (Nat.pair w.unpair.2.unpair.1.unpair.2.unpair.2 w.unpair.2.unpair.2) 0) :=
   (primrec_nthCode.comp (Nat.Primrec.pair htable
     (Nat.Primrec.pair (Nat.Primrec.pair hb hcw) (Nat.Primrec.const 0)))).of_eq fun w
    => by
     simp only [unpair_pair_fst, unpair_pair_snd]
have hcap := primrec_mul_2 hlk1 hlk2
refine (primrec_selectFn h01 (Nat.Primrec.const 1)
  (primrec_selectFn h12 hf1
     (primrec_selectFn h23 hcat
       (primrec_selectFn h34 hcap (Nat.Primrec.const 0))))).of_eq fun w => ?_
show _ = matchesBCharBody (fun p' => nthCode w.unpair.1 p' 0) w.unpair.2.unpair.1 w.
  unpair.2.unpair.2
rw [matchesBCharBody_eq]
match tag : w.unpair.2.unpair.1.unpair.1 with
```

```
  | 0 =>
    have h10 : (1 - 0 : Nat) = 1 := rfl
    simp only [h10, isOne_one, selectFn_one]
  | 1 =>
    have h01' : (1 - 1 : Nat) = 0 := rfl
    have h12' : (2 - 1 : Nat) = 1 := rfl
    simp only [h01', h12', isOne_zero, isOne_one, selectFn_zero, selectFn_one]
  | 2 =>
    have h01' : (1 - 2 : Nat) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h12' : (2 - 2 : Nat) = 0 := rfl
    have h23' : (3 - 2 : Nat) = 1 := rfl
    simp only [h01', h23', isOne_zero, isOne_one, selectFn_zero, selectFn_one]
    apply eq_of_le_one_iff_one (bExistsFn_le_one _ _ _) (bExistsFn_le_one _ _ _)
    rw [bExistsFn_eq_one_iff, bExistsFn_eq_one_iff]
    refine exists_congr fun i => and_congr_right fun _ => ?_
    simp only [matchesBCatGLookup, unpair_pair_fst, unpair_pair_snd]
  | 3 =>
    have h01' : (1 - 3 : Nat) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h12' : (2 - 3 : Nat) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h23' : (3 - 3 : Nat) = 0 := rfl
    have h34' : (4 - 3 : Nat) = 1 := rfl
    simp only [h01', h34', isOne_zero, isOne_one, selectFn_zero, selectFn_one,
      mulBit]
  | t + 4 =>
    have h01' : (1 - (t + 4) : Nat) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h12' : (2 - (t + 4) : Nat) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h23' : (3 - (t + 4) : Nat) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h34' : (4 - (t + 4) : Nat) = 0 := Nat.sub_eq_zero_of_le (by omega)
    simp only [h01', h12', h23', h34', isOne_zero, selectFn_zero]
end

section
/-- **`matchesBChar` is jointly primitive recursive in `(fuel, sexprCode, wordCode)`.**
    -/
theorem primrec_matchesBChar2 :
    Nat.Primrec (fun s => matchesBChar s.unpair.1 s.unpair.2.unpair.1 s.unpair.2.
    unpair.2) := by
  have hft := primrec_fuelTable primrec_matchesBCharBodyLookup
  have hpack : Nat.Primrec (fun s : Nat => Nat.pair s.unpair.2 s.unpair.1) :=
    Nat.Primrec.pair Nat.Primrec.right Nat.Primrec.left
  have hcomp : Nat.Primrec (fun s => fuelTable matchesBCharBodyLookup s.unpair.2 s.
    unpair.1) :=
    (hft.comp hpack).of_eq fun s => by simp
  have hnth : Nat.Primrec
      (fun s => nthCode (fuelTable matchesBCharBodyLookup s.unpair.2 s.unpair.1) s.
    unpair.2 0) :=
    (primrec_nthCode.comp
      (Nat.Primrec.pair hcomp (Nat.Primrec.pair Nat.Primrec.right (Nat.Primrec.const
    0)))).of_eq
      fun s => by simp
  exact hnth.of_eq fun s => by
    rw [matchesBChar_fuelTable_eq s.unpair.1 s.unpair.2 s.unpair.2 (le_refl _)]
end

section
theorem primrec_decideNonemptyBChar2 :
```

```
    Nat.Primrec (fun s => decideNonemptyBChar s.unpair.1 s.unpair.2) := by
  have hg : Nat.Primrec (fun u => mulBit (allBinDigitsChar u.unpair.2.unpair.1)
      (matchesBChar u.unpair.1 u.unpair.2.unpair.2 u.unpair.2.unpair.1)) :=
    (primrec_mulBit.comp (Nat.Primrec.pair
      (primrec_allBinDigitsChar.comp (Nat.Primrec.left.comp Nat.Primrec.right))
      (primrec_matchesBChar2.comp (Nat.Primrec.pair Nat.Primrec.left
        (Nat.Primrec.pair (Nat.Primrec.right.comp Nat.Primrec.right)
          (Nat.Primrec.left.comp Nat.Primrec.right)))))).of_eq fun u => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  have hbe := primrec_bExistsFn_param hg
  have hpack : Nat.Primrec (fun s => Nat.pair (Nat.pair s.unpair.1 s.unpair.2)
      (codeBound (autStateCardFuelChar s.unpair.1 s.unpair.2))) :=
    Nat.Primrec.pair (Nat.Primrec.pair Nat.Primrec.left Nat.Primrec.right)
      (primrec_codeBound.comp primrec_autStateCardFuelChar2)
  exact (hbe.comp hpack).of_eq fun s => by
    simp only [unpair_pair_fst, unpair_pair_snd, decideNonemptyBChar]
end

section
/-- **`consistentBChar` is jointly primitive recursive in `(fuel, c1, c2)`.** -/
theorem primrec_consistentBChar2 :
    Nat.Primrec (fun s => consistentBChar s.unpair.1 s.unpair.2.unpair.1 s.unpair.2.
    unpair.2) := by
  have hpack : Nat.Primrec (fun s => Nat.pair s.unpair.1
      (Nat.pair 3 (Nat.pair s.unpair.2.unpair.1 s.unpair.2.unpair.2))) :=
    Nat.Primrec.pair Nat.Primrec.left
      (Nat.Primrec.pair (Nat.Primrec.const 3)
        (Nat.Primrec.pair (Nat.Primrec.left.comp Nat.Primrec.right)
          (Nat.Primrec.right.comp Nat.Primrec.right)))
  exact (primrec_decideNonemptyBChar2.comp hpack).of_eq fun s => by
    simp only [unpair_pair_fst, unpair_pair_snd, consistentBChar, capCode]
end

section
/-- `{0,1}`-valued `max` of two primitive-recursive functions. -/
theorem primrec_max {f g : Nat  ->  Nat} (hf : Nat.Primrec f) (hg : Nat.Primrec g) :
    Nat.Primrec (fun n => Nat.max (f n) (g n)) := by
  refine (primrec_selectFn (primrec_le hf hg) hg hf).of_eq fun n => by
    unfold selectFn isZero Nat.max
    by_cases h : f n  <=  g n
    * simp [Nat.sub_eq_zero_iff_le.mpr h, Nat.max_eq_right h]
    * have hlt : g n < f n := Nat.lt_of_not_ge h
      have hpos : 0 < f n - g n := Nat.sub_pos_of_lt hlt
      have hmin : min (f n - g n) 1 = 1 := Nat.min_eq_right (Nat.succ_le_iff.mpr hpos)
      simp [hmin, Nat.max_eq_left (Nat.le_of_lt hlt)]
end

section
/-- `Nat.max n 1` is primitive recursive. -/
theorem primrec_max_one : Nat.Primrec (fun n => Nat.max n 1) := by
  refine (primrec_selectFn (primrec_isOne.comp (primrec_sub_2 (Nat.Primrec.const 1)
    primrec_id))
    (Nat.Primrec.const 1) primrec_id).of_eq fun n => by
    unfold selectFn isOne Nat.max
    rcases n with _ | n <;> simp
end
```

```
section
theorem primrec_mod2 : Nat.Primrec (fun n => n % 2) :=
  (Nat.Primrec.right.comp primrec_halfParity).of_eq fun n => by
    rw [halfParity_spec, unpair_pair_snd]
end

section
theorem primrec_lowOr : Nat.Primrec (fun t => lowOr t.unpair.1 t.unpair.2) := by
  have hx : Nat.Primrec (fun t : Nat => t.unpair.1 % 2) := primrec_mod2.comp Nat.
    Primrec.left
  have hy : Nat.Primrec (fun t : Nat => t.unpair.2 % 2) := primrec_mod2.comp Nat.
    Primrec.right
  exact (primrec_sub_2 (Nat.Primrec.const 1)
    (primrec_sub_2 (Nat.Primrec.const 1) (primrec_add_2 hx hy))).of_eq fun _ => rfl
end

section
theorem primrec_lorStep : Nat.Primrec lorStep := by
  have hA : Nat.Primrec (fun s : Nat => s.unpair.1.unpair.1) := Nat.Primrec.left.comp
    Nat.Primrec.left
  have hB : Nat.Primrec (fun s : Nat => s.unpair.1.unpair.2) := Nat.Primrec.right.comp
     Nat.Primrec.left
  have hW : Nat.Primrec (fun s : Nat => s.unpair.2.unpair.1) := Nat.Primrec.left.comp
    Nat.Primrec.right
  have hAcc : Nat.Primrec (fun s : Nat => s.unpair.2.unpair.2) := Nat.Primrec.right.
    comp Nat.Primrec.right
  have hlow : Nat.Primrec (fun s : Nat => lowOr s.unpair.1.unpair.1 s.unpair.1.unpair
    .2) :=
    (primrec_lowOr.comp (hA.pair hB)).of_eq fun s => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  exact (((primrec_div2.comp hA).pair (primrec_div2.comp hB)).pair
    ((primrec_mul_2 (Nat.Primrec.const 2) hW).pair
      (primrec_add_2 hAcc (primrec_mul_2 hW hlow)))).of_eq fun _ => rfl
end

section
/-- **Bitwise OR is primitive recursive** (the `Nat.unpaired` form), choice-free. -/
theorem primrec_myLor : Nat.Primrec (fun t => myLor t.unpair.1 t.unpair.2) := by
  have hbase : Nat.Primrec
      (fun z => Nat.pair (Nat.pair z.unpair.1 z.unpair.2) (Nat.pair 1 0)) :=
    (Nat.Primrec.left.pair Nat.Primrec.right).pair
      ((Nat.Primrec.const 1).pair (Nat.Primrec.const 0))
  have hstep : Nat.Primrec (fun w => lorStep w.unpair.2.unpair.2) :=
    primrec_lorStep.comp (Nat.Primrec.right.comp Nat.Primrec.right)
  have hprec := Nat.Primrec.prec hbase hstep
  have hcount : Nat.Primrec (fun t => t.unpair.1 + t.unpair.2) :=
    primrec_add_2 Nat.Primrec.left Nat.Primrec.right
  refine ((Nat.Primrec.right.comp Nat.Primrec.right).comp
    (hprec.comp (primrec_id.pair hcount))).of_eq fun t => ?_
  simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd, id_eq]
  rw [rec_const_iterate]
  rfl
end

section
```

```
/-- An always-true predicate is recursively decidable (constant decider `1`). -/
theorem recDecidable_of_forall {p : Nat  ->  Prop} (h : forall  n, p n) : RecDecidable
     p :=
  <fun _ => 1, Nat.Primrec.const 1, fun n => <fun _ => rfl, fun _ => h n>>
end

section
/-- An always-true predicate is recursively enumerable. -/
theorem rePred_of_forall {p : Nat  ->  Prop} (h : forall  n, p n) : REPred p :=
  (recDecidable_of_forall h).re
end

section
/-- An always-true binary relation is recursively decidable (constant decider `1`). -/
theorem recDecidable_2_of_forall {r : Nat  ->  Nat  ->  Prop} (h : forall  n m, r n m)
     : RecDecidable_2 r :=
  recDecidable_of_forall (fun t => h t.unpair.1 t.unpair.2)
end

section
/-- `Nat.rec 1 (* * b)` computes `b ^ e` (choice-free). -/
theorem recPow_eq (b : Nat) :


    forall  e, Nat.rec (motive := fun _ => Nat) 1 (fun _ ih => ih * b) e = b ^ e
  | 0 => (pow_zero b).symm
  | e + 1 => by rw [pow_succ]; exact congrArg (* * b) (recPow_eq b e)
end

section
/-- **Exponentiation is primitive recursive** (`unpaired (b, e)  |->  b ^ e`), choice-
    free. -/
theorem primrec_pow : Nat.Primrec (Nat.unpaired fun b e => b ^ e) := by
  have hg : Nat.Primrec (fun w => w.unpair.2.unpair.2 * w.unpair.1) :=
    primrec_mul_2 (Nat.Primrec.right.comp Nat.Primrec.right) Nat.Primrec.left
  refine (Nat.Primrec.prec (Nat.Primrec.const 1) hg).of_eq (fun p => ?_)
  simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd]
  exact recPow_eq p.unpair.1 p.unpair.2
end

section
/-- `fun n => f n ^ g n` is primitive recursive. -/
theorem primrec_pow_2 {f g : Nat  ->  Nat} (hf : Nat.Primrec f) (hg : Nat.Primrec g) :
    Nat.Primrec (fun n => f n ^ g n) :=
  (primrec_pow.comp (hf.pair hg)).of_eq fun n => by
    simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd]
end

section
theorem primrec_subsetBChar :
    Nat.Primrec (fun s => subsetBChar s.unpair.1 s.unpair.2.unpair.1 s.unpair.2.unpair
    .2) := by
  have hg : Nat.Primrec (fun u => selectFn
      (subsetGuardChar u.unpair.1 u.unpair.2.unpair.2.unpair.1 u.unpair.2.unpair.2.
    unpair.2
        u.unpair.2.unpair.1)
      (selectFn (matchesBChar u.unpair.1 u.unpair.2.unpair.2.unpair.1 u.unpair.2.
```

```
  unpair.1)
      (matchesBChar u.unpair.1 u.unpair.2.unpair.2.unpair.2 u.unpair.2.unpair.1) 1)
  1) := by
  have hfuel : Nat.Primrec (fun u : Nat => u.unpair.1) := Nat.Primrec.left
  have hi : Nat.Primrec (fun u : Nat => u.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.right
  have hc1 : Nat.Primrec (fun u : Nat => u.unpair.2.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp (Nat.Primrec.right.comp Nat.Primrec.right)
  have hc2 : Nat.Primrec (fun u : Nat => u.unpair.2.unpair.2.unpair.2) :=
    Nat.Primrec.right.comp (Nat.Primrec.right.comp Nat.Primrec.right)
  have hguard : Nat.Primrec (fun u => subsetGuardChar u.unpair.1 u.unpair.2.unpair
  .2.unpair.1
      u.unpair.2.unpair.2.unpair.2 u.unpair.2.unpair.1) := by
    have hbound : Nat.Primrec (fun u => subsetBoundChar u.unpair.1
        u.unpair.2.unpair.2.unpair.1 u.unpair.2.unpair.2.unpair.2) := by
      have h1 : Nat.Primrec (fun u => autStateCardFuelChar u.unpair.1
          u.unpair.2.unpair.2.unpair.1) := by
        have := primrec_autStateCardFuelChar2
        exact (this.comp (Nat.Primrec.pair hfuel hc1)).of_eq fun u => by
          simp only [unpair_pair_fst, unpair_pair_snd]
      have h2 : Nat.Primrec (fun u => autStateCardFuelChar u.unpair.1
          u.unpair.2.unpair.2.unpair.2) := by
        have := primrec_autStateCardFuelChar2
        exact (this.comp (Nat.Primrec.pair hfuel hc2)).of_eq fun u => by
          simp only [unpair_pair_fst, unpair_pair_snd]
      exact (primrec_mul_2 (primrec_pow_2 (Nat.Primrec.const 2) h1)
        (primrec_pow_2 (Nat.Primrec.const 2) h2))
    exact (primrec_mulBit.comp (Nat.Primrec.pair
      (primrec_allBinDigitsChar.comp hi)
      (primrec_isOne.comp (primrec_sub_2 (Nat.Primrec.const 1)
        (primrec_sub_2 (primrec_listLenChar.comp hi) hbound))))).of_eq fun u => by
      simp only [unpair_pair_fst, unpair_pair_snd, subsetGuardChar, mulBit]
  have hm1 : Nat.Primrec (fun u => matchesBChar u.unpair.1 u.unpair.2.unpair.2.
  unpair.1
      u.unpair.2.unpair.1) := by
    have := primrec_matchesBChar2
    exact (this.comp (Nat.Primrec.pair hfuel (Nat.Primrec.pair hc1 hi))).of_eq fun u
   => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  have hm2 : Nat.Primrec (fun u => matchesBChar u.unpair.1 u.unpair.2.unpair.2.
  unpair.2
      u.unpair.2.unpair.1) := by
    have := primrec_matchesBChar2
    exact (this.comp (Nat.Primrec.pair hfuel (Nat.Primrec.pair hc2 hi))).of_eq fun u
   => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  exact primrec_selectFn hguard (primrec_selectFn hm1 hm2 (Nat.Primrec.const 1))
    (Nat.Primrec.const 1)
have hb := primrec_bForallFn_param hg
have hpack : Nat.Primrec (fun s => Nat.pair
    (Nat.pair s.unpair.1 (Nat.pair s.unpair.2.unpair.1 s.unpair.2.unpair.2))
    (codeBound (subsetBoundChar s.unpair.1 s.unpair.2.unpair.1 s.unpair.2.unpair.2))
  ) := by
  have hfuel : Nat.Primrec (fun s : Nat => s.unpair.1) := Nat.Primrec.left
  have hc1 : Nat.Primrec (fun s : Nat => s.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.right
```

```
    have hc2 : Nat.Primrec (fun s : Nat => s.unpair.2.unpair.2) :=
      Nat.Primrec.right.comp Nat.Primrec.right
    have h1 : Nat.Primrec (fun s => autStateCardFuelChar s.unpair.1 s.unpair.2.unpair
    .1) := by
      have := primrec_autStateCardFuelChar2
      exact (this.comp (Nat.Primrec.pair hfuel hc1)).of_eq fun s => by
        simp only [unpair_pair_fst, unpair_pair_snd]
    have h2 : Nat.Primrec (fun s => autStateCardFuelChar s.unpair.1 s.unpair.2.unpair
    .2) := by
      have := primrec_autStateCardFuelChar2
      exact (this.comp (Nat.Primrec.pair hfuel hc2)).of_eq fun s => by
        simp only [unpair_pair_fst, unpair_pair_snd]
    have hbnd : Nat.Primrec (fun s => subsetBoundChar s.unpair.1 s.unpair.2.unpair.1
        s.unpair.2.unpair.2) :=
      primrec_mul_2 (primrec_pow_2 (Nat.Primrec.const 2) h1) (primrec_pow_2 (Nat.
    Primrec.const 2) h2)
    exact Nat.Primrec.pair (Nat.Primrec.pair hfuel (Nat.Primrec.pair hc1 hc2))
      (primrec_codeBound.comp hbnd)
  exact (hb.comp hpack).of_eq fun s => by
    simp only [unpair_pair_fst, unpair_pair_snd, subsetBChar]
end

section
theorem primrec_interEqChar :
    Nat.Primrec (fun s => interEqChar s.unpair.1 s.unpair.2.unpair.1 s.unpair.2.unpair
    .2) := by
  have hswap : Nat.Primrec (fun s : Nat =>
      Nat.pair s.unpair.1 (Nat.pair s.unpair.2.unpair.2 s.unpair.2.unpair.1)) :=
    Nat.Primrec.pair Nat.Primrec.left
      (Nat.Primrec.pair (Nat.Primrec.right.comp Nat.Primrec.right)
        (Nat.Primrec.left.comp Nat.Primrec.right))
  have h1 := primrec_subsetBChar
  have h2 : Nat.Primrec (fun s => subsetBChar s.unpair.1 s.unpair.2.unpair.2 s.unpair
    .2.unpair.1) :=
    (primrec_subsetBChar.comp hswap).of_eq fun s => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  exact (primrec_mulBit.comp (Nat.Primrec.pair h1 h2)).of_eq fun s => by
    simp only [unpair_pair_fst, unpair_pair_snd, interEqChar]
end

section
/-- `n  |->  2 ^ g n` is primitive recursive when `g` is. -/
theorem primrec_two_pow {g : Nat  ->  Nat} (hg : Nat.Primrec g) : Nat.Primrec (fun n
    => 2 ^ g n) :=
  (primrec_pow.comp ((Nat.Primrec.const 2).pair hg)).of_eq fun n => by
    simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd]
end

section
theorem selectFn_isOne_one_sub_sigma (u : Nat) :
    selectFn (isOne (1 - u)) 1 0 = 1  <->  u = 0 := by
  constructor
  * intro h
    by_cases hu : u = 0
    * exact hu
    * have : isOne (1 - u) = 0 := isOne_of_ne_one (by omega)
```

```
      simp [this, selectFn_zero] at h
  * intro hu
    subst hu
    simp [isOne_one, selectFn_one]
end

section
theorem subsetBChar_le_one (fuel c1 c2 : Nat) : subsetBChar fuel c1 c2  <=  1 := by
  unfold subsetBChar; exact bForallFn_le_one _ _ _
end

section
theorem interEqChar_le_one (fuel c1 c2 : Nat) : interEqChar fuel c1 c2  <=  1 := by
  unfold interEqChar; exact mulBit_le_one (subsetBChar_le_one _ _ _) (
    subsetBChar_le_one _ _ _)
end

end Domain.Recursive
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive

section
variable {a : Type*}

/-- **Definition 7.1 (Scott 1981, PRG-19).** A *computable presentation* of a
    neighbourhood system
`V` over a token type `a`: an enumeration `X : Nat  ->  Set a` whose range is exactly
    `?` (`mem_X` and
`surj`), such that Scott's two relations on the integer indices are recursively
    decidable:

* `interEq_computable` is (i): the ternary relation `X_n  I  X? = X_k`;
* `cons_computable` is (ii): the binary consistency relation `exists  k. X_k  subseteq
     X_n  I  X?`
  (Scott's `X_k  subseteq  X_n and X_k  subseteq  X?`).

Only the enumeration `X` is data; the remaining fields are `Prop`s, so a presentation
    is built
choice-free. -/
structure ComputablePresentation (V : NeighborhoodSystem a) where
  /-- The enumeration `? = {X_n ? n  in  Nat}`. -/
  X : Nat  ->  Set a
  /-- Every `X_n` is a neighbourhood. -/
  mem_X : forall  n, V.mem (X n)
  /-- The enumeration is onto `?`: every neighbourhood appears as some `X_n`. -/
  surj : forall  {Y : Set a}, V.mem Y  ->  exists  n, X n = Y
  /-- **7.1(i)** -- `X_n  I  X? = X_k` is recursively decidable in `n, m, k`. -/
  interEq_computable : RecDecidable_3 (fun n m k => X n  I  X m = X k)
  /-- **7.1(ii)** -- consistency `exists  k. X_k  subseteq  X_n  I  X?` is recursively
    decidable in `n, m`. -/
  cons_computable : RecDecidable_2 (fun n m => exists  k, X k  subseteq  X n  I  X m)
  /-- A **primitive-recursive intersection function**: an index of `X_n  I  X?`
    whenever that
  intersection is a neighbourhood (i.e. consistent). This makes explicit the operation
     that 7.1(i)
  is *about*: in Scott's general-recursive reading of "recursively decidable" the
```

```
    index can be
  recovered from `interEq_computable` by an (unbounded) search `?k. X_n  I  X? = X_k`,
    but that search
  is not *primitive* recursive; for the function-space presentation (Theorem 7.5) we
    need to
  *form* component intersections primitively, so we carry the function as part of the
    data of an
  ("acceptable") computable presentation. Off the consistent domain `inter n m` may be
    junk. -/
  inter : Nat  ->  Nat  ->  Nat
  /-- The intersection function is primitive recursive (on the `Nat.pair` coding of `n,
     m`). -/
  inter_primrec : Nat.Primrec (fun t => inter t.unpair.1 t.unpair.2)
  /-- `inter n m` indexes `X_n  I  X?` whenever that intersection is consistent. -/
  inter_spec : forall  {n m : Nat}, (exists  k, X k  subseteq  X n  I  X m)  ->  X (
    inter n m) = X n  I  X m
  /-- A fixed index of the master neighbourhood `Delta` (used to seed finite
    intersections). -/
  masterIdx : Nat
  /-- `X masterIdx = Delta`. -/
  masterIdx_spec : X masterIdx = V.master

/-- **Definition 7.1 (Scott 1981, PRG-19) -- effectively given.** A neighbourhood
    system is
*effectively given* when it admits a computable presentation. -/
def NeighborhoodSystem.IsEffectivelyGiven (V : NeighborhoodSystem a) : Prop :=
  Nonempty (ComputablePresentation V)

namespace ComputablePresentation

variable {V : NeighborhoodSystem a} (P : ComputablePresentation V)

/-- Reindexing `(n, m)  |->  (n, m, n)` on `Nat.pair` codes: `t  |->  pair n (pair m n)
    `. -/
def inclShuffle (t : Nat) : Nat := Nat.pair t.unpair.1 (Nat.pair t.unpair.2 t.unpair
    .1)

/-- **Reindexing a computable presentation along a primitive-recursive involution.**
    Given
`phi : Nat  ->  Nat` with `phi  comp  phi = id` (so `phi` is automatically a bijection,
     with itself as inverse) and
`Nat.Primrec phi`, transport `P`'s enumeration and both deciders along `phi`: `X' n :=
    P.X (phi n)`.
Every one of Scott's structural fields transfers by straightforward reindexing (`
    RecDecidable.comp`
against the pairwise-`phi`-reindexing code), so this is a genuinely choice-free
    construction -- used
to re-point a presentation's `0`-th index at an arbitrary target (Theorem 8.8(b)'s `D_0
      ~=  D` shift,
via `eIdx`). -/
def reindexInvolutive (P : ComputablePresentation V) (phi : Nat  ->  Nat)
    (hphiinv : Function.Involutive phi) (hphip : Nat.Primrec phi) :
    ComputablePresentation V where
  X n := P.X (phi n)
  mem_X n := P.mem_X (phi n)
  surj := fun {Y} hY => by
```

```
    obtain <k, hk> := P.surj hY
    exact <phi k, by rw [hphiinv k]; exact hk>
  interEq_computable := by
    have hreindex : Nat.Primrec (fun t : Nat =>
        Nat.pair (phi t.unpair.1) (Nat.pair (phi t.unpair.2.unpair.1) (phi t.unpair.2.
    unpair.2))) :=
      (hphip.comp Nat.Primrec.left).pair
        ((hphip.comp (Nat.Primrec.left.comp Nat.Primrec.right)).pair
          (hphip.comp (Nat.Primrec.right.comp Nat.Primrec.right)))
    refine RecDecidable.of_iff (fun t => ?_) (P.interEq_computable.comp hreindex)
    simp only [unpair_pair_fst, unpair_pair_snd]
  cons_computable := by
    have hreindex : Nat.Primrec (fun t : Nat =>
        Nat.pair (phi t.unpair.1) (phi t.unpair.2)) :=
      (hphip.comp Nat.Primrec.left).pair (hphip.comp Nat.Primrec.right)
    refine RecDecidable.of_iff (fun t => ?_) (P.cons_computable.comp hreindex)
    simp only [unpair_pair_fst, unpair_pair_snd]
    constructor
    * rintro <k, hk>; exact <phi k, hk>
    * rintro <k, hk>; exact <phi k, by rwa [hphiinv]>
  inter n m := phi (P.inter (phi n) (phi m))
  inter_primrec := by
    have hreindex : Nat.Primrec (fun t : Nat => Nat.pair (phi t.unpair.1) (phi t.
    unpair.2)) :=
      (hphip.comp Nat.Primrec.left).pair (hphip.comp Nat.Primrec.right)
    exact (hphip.comp ((P.inter_primrec.comp hreindex))).of_eq
      (fun t => by simp only [unpair_pair_fst, unpair_pair_snd])
  inter_spec {n m} h := by
    show P.X (phi (phi (P.inter (phi n) (phi m)))) = P.X (phi n)  I  P.X (phi m)
    rw [hphiinv]
    obtain <k, hk> := h
    exact P.inter_spec <phi k, hk>
  masterIdx := phi P.masterIdx
  masterIdx_spec := by show P.X (phi (phi P.masterIdx)) = V.master; rw [hphiinv];
    exact P.masterIdx_spec

/-- Swap the two components of a `Nat.pair` code: `t  |->  pair m n`. -/
def swapPair (t : Nat) : Nat := Nat.pair t.unpair.2 t.unpair.1

end ComputablePresentation

end

section
variable {a : Type*}

/-- The trivial presentation of the one-point system `? = unitSys`: the constant
    enumeration
`X_n = Delta = univ`. Both of Scott's relations are *always true* here (any two
    neighbourhoods are equal
and consistent), hence trivially recursively decidable via the constant `1` decider. -/
def unitPresentation : ComputablePresentation unitSys where
  X _ := Set.univ
  mem_X _ := rfl
  surj := by rintro Y rfl; exact <0, rfl>
  interEq_computable := recDecidable_of_forall fun _ => Set.inter_self Set.univ
```

```
  cons_computable := recDecidable_of_forall fun _ =>
    <0, by rw [Set.inter_self]>
  inter _ _ := 0
  inter_primrec := Nat.Primrec.zero
  inter_spec _ := by rw [Set.inter_self]
  masterIdx := 0
  masterIdx_spec := rfl

namespace ComputablePresentation

variable {V : NeighborhoodSystem a} (P : ComputablePresentation V)

theorem primrec_inclShuffle : Nat.Primrec inclShuffle :=
  Nat.Primrec.pair Nat.Primrec.left (Nat.Primrec.pair Nat.Primrec.right Nat.Primrec.
    left)

private theorem primrec_swapPair : Nat.Primrec swapPair :=
  Nat.Primrec.pair Nat.Primrec.right Nat.Primrec.left

/-- **Scott's biconditional after 7.1.** "The inclusion relation between
    neighbourhoods is itself
decidable in terms of the indices", because `X_n  subseteq  X?  <->  X_n  I  X? = X_n`.
     We obtain the decision by
reindexing `(n, m)  |->  (n, m, n)` into relation (i). -/
theorem incl_computable : RecDecidable_2 (fun n m => P.X n  subseteq  P.X m) := by
  refine RecDecidable.of_iff (fun t => ?_) (P.interEq_computable.comp
    primrec_inclShuffle)
  simp only [inclShuffle, unpair_pair_fst, unpair_pair_snd]
  exact Set.inter_eq_left.symm

/-- **Equality of neighbourhoods is decidable** from the indices: `X_n = X?  <->  X_n
    subseteq  X?  /\  X?  subseteq  X_n`,
so equality is the conjunction of `incl_computable` with its swap. -/
theorem eq_computable : RecDecidable_2 (fun n m => P.X n = P.X m) := by
  refine RecDecidable.of_iff (fun t => ?_)
    (P.incl_computable.and (P.incl_computable.comp primrec_swapPair))
  simp only [swapPair, unpair_pair_fst, unpair_pair_snd]
  exact Set.Subset.antisymm_iff

end ComputablePresentation

end

section
variable {a : Type*}

/-- **The one-point domain `?` is effectively given.** -/
theorem unitSys_isEffectivelyGiven : unitSys.IsEffectivelyGiven :=
  <unitPresentation>
end

namespace Exercise722
open Set Domain.Recursive

section
/-- Definition 7.1 restricted to relation (ii) only (no `interEq_computable`, unlike
```

```
`ComputablePresentation`/`ScottPresentation` in `Definition71.lean`/`Exercise715.lean`,
     whose
pattern this mirrors). -/
structure ConsistencyPresentation (V : NeighborhoodSystem a) where
  /-- The enumeration `? = {X_n ? n  in  Nat}`. -/
  X : Nat  ->  Set a
  /-- Every `X_n` is a neighbourhood. -/
  mem_X : forall  n, V.mem (X n)
  /-- The enumeration is onto `?`. -/
  surj : forall  {Y : Set a}, V.mem Y  ->  exists  n, X n = Y
  /-- **7.1(ii)** -- consistency `exists  k. X_k  subseteq  X_n  I  X?` is recursively
    decidable in `n, m`. -/
  cons_computable : RecDecidable_2 (fun n m => exists  k, X k  subseteq  X n  I  X m)
end

end Exercise722
open NeighborhoodSystem Domain.Recursive ApproximableMap

end DomainTheory.NeighbourhoodSystems
end
```

3.8.3 Definition 7.2 — computable mapping

DEFINITION 7.2. Given two recursively presented domains,

$$\mathcal{D} = \{X_n \mid n \in \mathbf{N}\} \text{ and } E = \{Y_m \mid m \in \mathbf{N}\},$$

an approximable mapping $f : \mathcal{D} \to E$ is said to be *computable* iff the relation

$$X_n f Y_m$$

is recursively enumerable in n and m. □

The question to ask first is why "recursively enumerable" rather than "recursive" (= "recursively decidable")? The answer will become clear when we let $\mathcal{D}$ degenerate to the one-element domain, $\mathcal{D} = \{\Delta\}$. Then what we are considering is merely a single element

$$y = f(\{\Delta\}) \in |E|.$$

Therefore, 7.2 incorporates the notion of a *computable element* of a domain. And the condition reduces to the statement that the filter $y \in |E|$ is such that the set

$$\{m \in \mathbf{N} \mid Y_m \in y\}$$

is a recursively enumerable set of integers. The point is that the elements of $|E|$ are finite or infinite. If y were finite, the set of indices above would indeed be recursive in view of

our assumptions on E. But an infinite element can in general only be approximated "a little at a time". We cannot expect to know the whole story of its approximations in a flash. What it means to be recursively enumerable is that there is a primitive recursive function (hence, a total function), $r : \mathbf{N} \to \mathbf{N}$, such that

$$y = \{Y_{r(i)} \mid i \in \mathbf{N}\}.$$

That is to say, *all* the approximations to y can eventually be listed. In the case of the mapping f we could write

$$f = \{(X_{s(i)}, Y_{r(i)}) \mid i \in \mathbf{N}\},$$

for a suitable pair of primitive recursive functions s and r.

Definitions 7.1 and 7.2 may very well irritate the person hearing them for the first time: instead of explaining computability in direct terms, the whole question is thrown into the lap of recursion theory! There are several answers. "You have to start somewhere" is one thing I always say. Recursion on the integers is a well-understood theory, and we shall not need the refined parts of the development, fortunately. In any case, our definitions apply to *many* domains of quite different structure, not just to the domain $\mathbf{N}$. And the next step we shall take is to show how to build up computable functions (and also effectively given domains) from simpler ones. Thus, often it will not be necessary to go back to the seemingly over-precise definitions involving the indices but to appeal to some broad general principles.

Definition_7_2.lean

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Definition_7_1

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive
open NeighborhoodSystem Domain.Recursive ApproximableMap
section
variable {a b ? : Type*}

/- concordance: source L4271 DEFINITION 7.2 computable mapping -/
/-- **Definition 7.2 (Scott 1981, PRG-19) -- computable element.** Specializing to `f :
     ?  ->  W`, the
condition becomes: the index set `{m ? Y?  in  y}` of the element `y  in  |W|` is
    recursively enumerable.
We take this as the definition of a *computable element*. -/
class ComputableElement {W : NeighborhoodSystem b} (Q : ComputablePresentation W)
    (y : W.Element) : Prop where
  out : REPred (fun m => y.mem (Q.X m))

/-- **Definition 7.2 (Scott 1981, PRG-19) -- computable map.** Relative to computable
    presentations
`P` of `V` and `Q` of `W`, an approximable map `f : V  ->  W` is *computable* iff its
    neighbourhood
relation `X_n f Y?`, transported to the integer indices, is recursively enumerable. -/
class ComputableMap {V : NeighborhoodSystem a} {W : NeighborhoodSystem b}
    (P : ComputablePresentation V) (Q : ComputablePresentation W) (f : ApproximableMap
     V W) : Prop where
  out : REPred_2 (fun n m => f.rel (P.X n) (Q.X m))

end

end DomainTheory.NeighbourhoodSystems
```

```
end
```

3.8.4 Proposition 7.3 — computable maps form a category

PROPOSITION 7.3. The identity map on an effectively given domain is computable; the composition of computable mappings on effectively given domains is again computable. □

The proofs for 7.3 are so trivial they are hardly worth an exercise. Note the immediate and useful consequence: if $f : D \to E$ is computable and $x \in |D|$ is computable, then $f(x) \in |E|$ is also computable. The next result is, however, worth working out even though it is quite easy.

Proposition_7_3.lean

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Definition_7_2

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive
open NeighborhoodSystem Domain.Recursive ApproximableMap
section
variable {a b ? : Type*}
variable {V : NeighborhoodSystem a} {W : NeighborhoodSystem b}

/- concordance: source L4305 PROPOSITION 7.3 computable maps form a category -/
/-- **Proposition 7.3 (consequence) (Scott 1981, PRG-19).** "If `f : ?  ->  ?` is
    computable and
`x  in  |?|` is computable, then `f(x)  in  |?|` is also computable." Here `f(x) = {Y?
    ? exists  X_n  in  x, X_n f Y?}`
(`toElementMap`); surjectivity of `P` lets the witness `X` range over indices `n`, and
    the resulting
`exists  n, X_n  in  x  /\  X_n f Y?` is r.e. by `REPred.and`/`REPred.proj`. -/
theorem apply_isComputableElement {P : ComputablePresentation V} {Q :
    ComputablePresentation W}
    {f : ApproximableMap V W} (hf : ComputableMap P Q f) {x : V.Element}
    (hx : ComputableElement P x) : ComputableElement Q (f.toElementMap x) := by
  have hf' : REPred (fun s => f.rel (P.X s.unpair.1) (Q.X s.unpair.2)) := hf.out
  have hx' : REPred (fun n => x.mem (P.X n)) := hx.out
  constructor
  refine REPred.of_iff (fun m => ?_) ((hx'.comp Nat.Primrec.left).and hf').proj
  simp only [mem_toElementMap, unpair_pair_fst, unpair_pair_snd]
  constructor
  * rintro <X, hX, hfX>
    obtain <n, rfl> := P.surj (x.sub hX)
    exact <n, hX, hfX>
  * rintro <n, hxn, hfn>
    exact <P.X n, hxn, hfn>

/-- **Proposition 7.3 (Scott 1981, PRG-19) -- composition of computable maps is
    computable.** For
```

```
`X (g comp f) Z  <->  exists  Y, X f Y  /\  Y g Z`, surjectivity of the middle
    presentation `Q` lets the witness `Y`
range over indices `l` (`Y = Y?`); the resulting `exists  l, X_n f Y?  /\  Y? g Z?` is
    recursively enumerable
by reindexing (`REPred.comp`), conjunction (`REPred.and`), and existential projection
(`REPred.proj`). -/
theorem comp_isComputable {U : NeighborhoodSystem ?}
    {P : ComputablePresentation V} {Q : ComputablePresentation W} {R :
    ComputablePresentation U}
    {f : ApproximableMap V W} {g : ApproximableMap W U}
    (hf : ComputableMap P Q f) (hg : ComputableMap Q R g) :
    ComputableMap P R (g.comp f) := by
  constructor
  have hf' : REPred (fun s => f.rel (P.X s.unpair.1) (Q.X s.unpair.2)) := hf.out
  have hg' : REPred (fun s => g.rel (Q.X s.unpair.1) (R.X s.unpair.2)) := hg.out
  have hgf : Nat.Primrec (fun u => Nat.pair u.unpair.2.unpair.1 u.unpair.1) :=
    Nat.Primrec.pair (Nat.Primrec.left.comp Nat.Primrec.right) Nat.Primrec.left
  have hgg : Nat.Primrec (fun u => Nat.pair u.unpair.1 u.unpair.2.unpair.2) :=
    Nat.Primrec.pair Nat.Primrec.left (Nat.Primrec.right.comp Nat.Primrec.right)
  refine REPred.of_iff (fun t => ?_) ((hf'.comp hgf).and (hg'.comp hgg)).proj
  simp only [comp_rel, unpair_pair_fst, unpair_pair_snd]
  constructor
  * rintro <Y, hfY, hgY>
    obtain <l, rfl> := Q.surj (g.rel_dom hgY)
    exact <l, hfY, hgY>
  * rintro <l, hfl, hgl>
    exact <Q.X l, hfl, hgl>

/-- **The identity map is computable** (the identity half of Proposition 7.3). The
    relation
`X_n I X?` is `X_n  subseteq  X?` (`incl_computable`), which is recursively *decidable
    *, hence recursively
enumerable. -/
theorem idMap_isComputable (P : ComputablePresentation V) :
    ComputableMap P P (idMap V) :=
  <(RecDecidable.of_iff (fun t => by
    simp only [idMap_rel]
    exact <fun h => h.2.2, fun h => <P.mem_X _, P.mem_X _, h>>)
    P.incl_computable).re>

/-- **Every finite (principal) element is computable** (Scott's remark after 7.2: "If `
    y` were
finite, the set of indices would be recursive"). For the finite element `X_n`, the
    index set
`{m ? X_n  subseteq  X?}` is a recursive slice of `incl_computable` (fix the first
    index to `n` by the
primitive-recursive reindex `m  |->  <n, m>`), hence r.e. -/
theorem principal_isComputableElement (P : ComputablePresentation V) (n : Nat) :
    ComputableElement P (V.principal (P.mem_X n)) := by
  have hg : Nat.Primrec (fun m => Nat.pair n m) :=
    ((Nat.Primrec.const n).pair primrec_id).of_eq (fun _ => rfl)
  have hrec : RecDecidable (fun m => P.X n  subseteq  P.X m) :=
    RecDecidable.of_iff (fun m => by simp only [unpair_pair_fst, unpair_pair_snd])
      (P.incl_computable.comp hg)
  constructor
  refine (RecDecidable.of_iff (fun m => ?_) hrec).re
```

```
  simp only [mem_principal]
  exact <fun h => h.2, fun h => <P.mem_X m, h>>

end

end DomainTheory.NeighbourhoodSystems
end
```

3.8.5 Theorem 7.4 — sum and product effective

THEOREM 7.4. If $\mathcal{D}_0$ and $\mathcal{D}_1$ are effectively given, then so are $(\mathcal{D}_0 + \mathcal{D}_1)$ and $(\mathcal{D}_0 \times \mathcal{D}_1)$.

Moreover the combinators in_i and out_i and proj_i^2 are all computable; further, if f and g are computable maps, then so are $f + g$ and $f \times g$.

Proof: Let the computable presentations be given as:

$$\mathcal{D}_i = \{X_n^i \mid n \in \mathbf{N}\}.$$

We can assume that the sets of tokens Δ_0 and Δ_1 are disjoint and $\emptyset \notin \mathcal{D}_i$. Then the construction of the sum is just

$$\mathcal{D}_0 + \mathcal{D}_1 = \{\Delta_0 \cup \Delta_1\} \cup \mathcal{D}_0 \cup \mathcal{D}_1.$$

As an enumeration we define for $n \in \mathbf{N}$:

$$Z_0 = \Delta_0 \cup \Delta_1 \; ; \; Z_{2n+1} = X_n^0 \; ; \; Z_{2n+2} = X_n^1 \; .$$

We leave as an exercise the check of 7.1(i)–(ii).

For the product we want:

$$\mathcal{D}_0 \times \mathcal{D}_1 = \{X_n^0 \cup X_m^1 \mid n, m \in \mathbf{N}\}.$$

What we then need are recursive functions $p : \mathbf{N} \to \mathbf{N}$, $q : \mathbf{N} \to \mathbf{N}$, and $r : \mathbf{N} \times \mathbf{N} \to \mathbf{N}$ where for $m, n, k \in \mathbf{N}$ we have:

$$p(r(n,m)) = n \text{ and } q(r(n,m)) = m, \text{ and } r(p(k), q(k)) = k.$$

Thus r is a "one-one pairing function"; there are many ways to find such functions (see Exercise 5.13). We can then define for $k \in \mathbf{N}$:

$$W_k = X_{p(k)}^0 \cup X_{q(k)}^1 \; .$$

Again we leave as an exercise the check that this provides a computable presentation of $\mathcal{D}_0 \times \mathcal{D}_1$.

As for the combinators, the neighbourhood relations have to be worked out in terms of the indices. For example

$$X_n^0 \; \mathrm{in}_0 \; Z_m \text{ iff either } m = 0 \text{ or for some } k$$

$$m = 2k + 1 \text{ and } X_n^0 \subseteq X_k^0 \; .$$

and

$$W_k \operatorname{proj}_1^2 X_m^1 \text{ iff } X_{q(k)}^1 \subseteq X_m^1 .$$

The reader needs to check that these are recursively enumerable

Relations in the indices. For this purpose it may be convenient to recall some closure properties of these relations: taking conjunctions, disjunctions, substituting recursive functions, applying an existential quantifier to the front. □

Products give us a way of providing an immediate meaning to the notion of a computable function of several variables. Note that the proof of 3.7 is "effective" and shows that substitution of computable functions of several variables into each other always gives computable functions. We turn next to the function spaces.

Theorem_7_4.lean

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Proposition_7_7

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Proposition_7_3

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive Example61
open NeighborhoodSystem Domain.Recursive ApproximableMap
section
variable {a b ? a' b' : Type*}
variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}

/- concordance: source L4311 THEOREM 7.4 sum and product effective -/
/-- **Theorem 7.4 (Scott 1981, PRG-19) -- `?_0  x  ?_1` is effectively given.** The
    presentation
`W_k = X?_{p k}  U  X1_{q k}`. Scott's 7.1(i) and (ii) each split, via `prodNbhd_inter`
     /
`prodNbhd_subset_iff`, into the *conjunction* of the two factors' relations on the
    projected
indices -- recursively decidable by `RecDecidable.and`/`.comp`/`.of_iff`. -/
def prodPresentation (P_0 : ComputablePresentation V_0) (P_1 : ComputablePresentation
    V_1) :
    ComputablePresentation (prod V_0 V_1) where
  X := prodEnum P_0 P_1
  mem_X t := prod_mem_prodNbhd (P_0.mem_X _) (P_1.mem_X _)
  surj := by
    rintro Z <X, Y, hX, hY, rfl>
    obtain <n, rfl> := P_0.surj hX
    obtain <m, rfl> := P_1.surj hY
    exact <Nat.pair n m, by simp only [prodEnum_apply, unpair_pair_fst,
    unpair_pair_snd]>
  interEq_computable := by
    have hIE0 : RecDecidable (fun s => P_0.X s.unpair.1  I  P_0.X s.unpair.2.unpair.1
        = P_0.X s.unpair.2.unpair.2) := P_0.interEq_computable
    have hIE1 : RecDecidable (fun s => P_1.X s.unpair.1  I  P_1.X s.unpair.2.unpair.1
```

```
      = P_1.X s.unpair.2.unpair.2) := P_1.interEq_computable
  have hg0 : Nat.Primrec (fun t => Nat.pair t.unpair.1.unpair.1
      (Nat.pair t.unpair.2.unpair.1.unpair.1 t.unpair.2.unpair.2.unpair.1)) :=
    (Nat.Primrec.left.comp Nat.Primrec.left).pair
      ((Nat.Primrec.left.comp (Nat.Primrec.left.comp Nat.Primrec.right)).pair
        (Nat.Primrec.left.comp (Nat.Primrec.right.comp Nat.Primrec.right)))
  have hg1 : Nat.Primrec (fun t => Nat.pair t.unpair.1.unpair.2
      (Nat.pair t.unpair.2.unpair.1.unpair.2 t.unpair.2.unpair.2.unpair.2)) :=
    (Nat.Primrec.right.comp Nat.Primrec.left).pair
      ((Nat.Primrec.right.comp (Nat.Primrec.left.comp Nat.Primrec.right)).pair
        (Nat.Primrec.right.comp (Nat.Primrec.right.comp Nat.Primrec.right)))
  refine RecDecidable.of_iff (fun t => ?_) ((hIE0.comp hg0).and (hIE1.comp hg1))
  simp only [prodEnum_apply, unpair_pair_fst, unpair_pair_snd, prodNbhd_inter,
  prodNbhd_eq_iff]
cons_computable := by
  have hC0 : RecDecidable (fun s => exists  k, P_0.X k  subseteq  P_0.X s.unpair.1
  I  P_0.X s.unpair.2) :=
    P_0.cons_computable
  have hC1 : RecDecidable (fun s => exists  k, P_1.X k  subseteq  P_1.X s.unpair.1
  I  P_1.X s.unpair.2) :=
    P_1.cons_computable
  have hh0 : Nat.Primrec (fun t => Nat.pair t.unpair.1.unpair.1 t.unpair.2.unpair.1)
   :=
    (Nat.Primrec.left.comp Nat.Primrec.left).pair (Nat.Primrec.left.comp Nat.Primrec
  .right)
  have hh1 : Nat.Primrec (fun t => Nat.pair t.unpair.1.unpair.2 t.unpair.2.unpair.2)
   :=
    (Nat.Primrec.right.comp Nat.Primrec.left).pair (Nat.Primrec.right.comp Nat.
  Primrec.right)
  refine RecDecidable.of_iff (fun t => ?_) ((hC0.comp hh0).and (hC1.comp hh1))
  simp only [prodEnum_apply, unpair_pair_fst, unpair_pair_snd, prodNbhd_inter,
    prodNbhd_subset_iff]
  constructor
  * rintro <k, hk0, hk1>
    exact <<k.unpair.1, hk0>, <k.unpair.2, hk1>>
  * rintro <<k0, hk0>, <k1, hk1>>
    exact <Nat.pair k0 k1, by simpa only [unpair_pair_fst] using hk0,
      by simpa only [unpair_pair_snd] using hk1>
inter n m := Nat.pair (P_0.inter n.unpair.1 m.unpair.1) (P_1.inter n.unpair.2 m.
  unpair.2)
inter_primrec := by
  have h0 : Nat.Primrec (fun t => P_0.inter t.unpair.1.unpair.1 t.unpair.2.unpair.1)
   :=
    (P_0.inter_primrec.comp ((Nat.Primrec.left.comp Nat.Primrec.left).pair
      (Nat.Primrec.left.comp Nat.Primrec.right))).of_eq fun t => by
        simp only [unpair_pair_fst, unpair_pair_snd]
  have h1 : Nat.Primrec (fun t => P_1.inter t.unpair.1.unpair.2 t.unpair.2.unpair.2)
   :=
    (P_1.inter_primrec.comp ((Nat.Primrec.right.comp Nat.Primrec.left).pair
      (Nat.Primrec.right.comp Nat.Primrec.right))).of_eq fun t => by
        simp only [unpair_pair_fst, unpair_pair_snd]
  exact h0.pair h1
inter_spec := by
  rintro n m <k, hk>
  simp only [prodEnum_apply, unpair_pair_fst, unpair_pair_snd] at hk  |-
  rw [prodNbhd_inter] at hk  |-
```

```
    obtain <h0, h1> := prodNbhd_subset_iff.mp hk
    rw [P_0.inter_spec <_, h0>, P_1.inter_spec <_, h1>]
  masterIdx := Nat.pair P_0.masterIdx P_1.masterIdx
  masterIdx_spec := by
    simp only [prodEnum_apply, unpair_pair_fst, unpair_pair_snd, P_0.masterIdx_spec,
      P_1.masterIdx_spec, prod_master]

@[simp] theorem prodPresentation_X (P_0 : ComputablePresentation V_0) (P_1 :
    ComputablePresentation V_1)
    (t : Nat) :
    (prodPresentation P_0 P_1).X t = prodNbhd (P_0.X t.unpair.1) (P_1.X t.unpair.2) :=
    rfl

section Sum

variable {h_0 : forall  X, V_0.mem X  ->  X.Nonempty} {h_1 : forall  Y, V_1.mem Y  ->
    Y.Nonempty}

/-- Scott's `Z_0 = Delta, Z_{2n+1} = X?_n, Z_{2n+2} = X1_n`, realized with a `Nat.pair`
    tag: `tag = 0  |->  0X?`,
`tag = 1  |->  1X1`, else the master. -/
def sumEnum (P_0 : ComputablePresentation V_0) (P_1 : ComputablePresentation V_1) (t :
    Nat) :
    Set (Option (a ? b)) :=
  if t.unpair.1 = 0 then inj_0 (P_0.X t.unpair.2)
  else if t.unpair.1 = 1 then inj_1 (P_1.X t.unpair.2)
  else sumMaster V_0 V_1

end Sum

section Sum

variable {h_0 : forall  X, V_0.mem X  ->  X.Nonempty} {h_1 : forall  Y, V_1.mem Y  ->
    Y.Nonempty}

variable {P_0 : ComputablePresentation V_0} {P_1 : ComputablePresentation V_1}

@[simp] theorem sumEnum_zero {t : Nat} (h : t.unpair.1 = 0) :
    sumEnum P_0 P_1 t = inj_0 (P_0.X t.unpair.2) := by unfold sumEnum; rw [ite_eq_left
    h]

@[simp] theorem sumEnum_one {t : Nat} (h : t.unpair.1 = 1) :
    sumEnum P_0 P_1 t = inj_1 (P_1.X t.unpair.2) := by
  have h0 : t.unpair.1  !=  0 := by omega
  unfold sumEnum; rw [ite_eq_right h0, ite_eq_left h]

theorem inj_0_eq_iff {A B : Set a} : (inj_0 A : Set (Option (a ? b))) = inj_0 B  <->
    A = B :=
  <inj_0_injective, fun h => h  |>  rfl>

/-- A nonempty left copy cannot equal a right copy. -/
theorem inj_0_eq_inj_1_elim {A : Set a} {B : Set b} (hA : A.Nonempty)
    (h : (inj_0 A : Set (Option (a ? b))) = inj_1 B) : False := by
  obtain <a, ha> := hA; exact il_mem_inj_1 (h  |>  il_mem_inj_0.mpr ha)

theorem inj_0_ne_inj_1_of_nonempty {A : Set a} {B : Set b} (hB : B.Nonempty) :
```

```
    (inj_0 A : Set (Option (a ? b)))  !=  inj_1 B := by
  rintro h; obtain <b, hb> := hB
  exact ir_mem_inj_0 (h.symm  |>  ir_mem_inj_1.mpr hb)

theorem inj_0_ne_sumMaster {A : Set a} : (inj_0 A : Set (Option (a ? b)))  !=
    sumMaster V_0 V_1 :=
  fun h => none_mem_inj_0 (h.symm  |>  none_mem_sumMaster)

theorem inj_1_eq_iff {A B : Set b} : (inj_1 A : Set (Option (a ? b))) = inj_1 B  <->
    A = B :=
  <inj_1_injective, fun h => h  |>  rfl>

theorem inj_1_ne_sumMaster {B : Set b} : (inj_1 B : Set (Option (a ? b)))  !=
    sumMaster V_0 V_1 :=
  fun h => none_mem_inj_1 (h.symm  |>  none_mem_sumMaster)

/-- Equality of two enumerated `?_0`-neighbourhoods is recursively decidable (
    antisymmetry of ` subseteq `,
decided by `incl_computable`). -/
theorem recDec_setEq_0 {f g : Nat  ->  Nat} (hf : Nat.Primrec f) (hg : Nat.Primrec g)
    :
    RecDecidable (fun u => P_0.X (f u) = P_0.X (g u)) := by
  have hincl : RecDecidable (fun s => P_0.X s.unpair.1  subseteq  P_0.X s.unpair.2) :=
     P_0.incl_computable
  refine RecDecidable.of_iff (fun u => ?_) ((hincl.comp (hf.pair hg)).and (hincl.comp
    (hg.pair hf)))
  simp only [unpair_pair_fst, unpair_pair_snd]
  exact <fun h => <h.subset, h.symm.subset>, fun <h1, h2> => Set.Subset.antisymm h1 h2
    >

theorem recDec_setEq_1 {f g : Nat  ->  Nat} (hf : Nat.Primrec f) (hg : Nat.Primrec g)
    :
    RecDecidable (fun u => P_1.X (f u) = P_1.X (g u)) := by
  have hincl : RecDecidable (fun s => P_1.X s.unpair.1  subseteq  P_1.X s.unpair.2) :=
     P_1.incl_computable
  refine RecDecidable.of_iff (fun u => ?_) ((hincl.comp (hf.pair hg)).and (hincl.comp
    (hg.pair hf)))
  simp only [unpair_pair_fst, unpair_pair_snd]
  exact <fun h => <h.subset, h.symm.subset>, fun <h1, h2> => Set.Subset.antisymm h1 h2
    >

theorem sumEnum_master {t : Nat} (h0 : t.unpair.1  !=  0) (h1 : t.unpair.1  !=  1) :
    sumEnum P_0 P_1 t = sumMaster V_0 V_1 := by unfold sumEnum; rw [ite_eq_right h0,
    ite_eq_right h1]

theorem sumEnum_mem (t : Nat) : (sum V_0 V_1 h_0 h_1).mem (sumEnum P_0 P_1 t) := by
  unfold sumEnum
  split
  * exact Or.inr (Or.inl <_, P_0.mem_X _, rfl>)
  * split
    * exact Or.inr (Or.inr <_, P_1.mem_X _, rfl>)
    * exact Or.inl rfl

include h_0 h_1 in
theorem sumEnum_nonempty (t : Nat) : (sumEnum P_0 P_1 t).Nonempty := by
  unfold sumEnum
```

```
  split
  * exact inj_0_nonempty (h_0 _ (P_0.mem_X _))
  * split
    * exact inj_1_nonempty (h_1 _ (P_1.mem_X _))
    * exact <none, none_mem_sumMaster>

/-- Every enumerated sum-neighbourhood lies under the master (no non-emptiness needed)
    . -/
theorem sumEnum_subset_sumMaster (n : Nat) :
    sumEnum P_0 P_1 n  subseteq  sumMaster V_0 V_1 := by
  unfold sumEnum
  split
  * exact inj_0_subset_sumMaster (P_0.mem_X _)
  * split
    * exact inj_1_subset_sumMaster (P_1.mem_X _)
    * exact subset_rfl

/-- `Z  I  M = Z`: the master absorbs on the right. -/
theorem sumEnum_inter_sumMaster (n : Nat) :
    sumEnum P_0 P_1 n  I  sumMaster V_0 V_1 = sumEnum P_0 P_1 n :=
  Set.inter_eq_left.mpr (sumEnum_subset_sumMaster n)

/-- `M  I  Z = Z`: the master absorbs on the left. -/
theorem sumMaster_inter_sumEnum (m : Nat) :
    sumMaster V_0 V_1  I  sumEnum P_0 P_1 m = sumEnum P_0 P_1 m :=
  Set.inter_eq_right.mpr (sumEnum_subset_sumMaster m)

end Sum

/-- Choice-free three-way split of a tag `n  in  {0, 1,  >= 2}` (via `Nat.decEq`, not
    classical `em`). -/
theorem tag_trichotomy (n : Nat) : n = 0  \/  n = 1  \/  (n  !=  0  /\  n  !=  1) :=
    by
  have h_dec : (n  !=  0)  \/  (n = 0) := by
    by_cases h : n = 0
    * exact Or.inr h
    * exact Or.inl h
  rcases h_dec with h | h
  * have h'_dec : (n  !=  1)  \/  (n = 1) := by
      by_cases h' : n = 1
      * exact Or.inr h'
      * exact Or.inl h'
    rcases h'_dec with h' | h'
    * exact Or.inr (Or.inr <h, h'>)
    * exact Or.inr (Or.inl h')
  * exact Or.inl h

section Sum

variable {h_0 : forall  X, V_0.mem X  ->  X.Nonempty} {h_1 : forall  Y, V_1.mem Y  ->
     Y.Nonempty}

variable {P_0 : ComputablePresentation V_0} {P_1 : ComputablePresentation V_1}

include h_1 in
/-- **Equality of two sum-neighbourhoods, in the indices.** Both master, or same tag
```

```
    with equal
component. -/
theorem sumEnum_eq_iff (x y : Nat) :
    sumEnum P_0 P_1 x = sumEnum P_0 P_1 y  <->
      ((x.unpair.1  !=  0  /\  x.unpair.1  !=  1)  /\  (y.unpair.1  !=  0  /\  y.
    unpair.1  !=  1))
         \/  (((x.unpair.1 = 0  /\  y.unpair.1 = 0)  /\  P_0.X x.unpair.2 = P_0.X y.
    unpair.2)
          \/  ((x.unpair.1 = 1  /\  y.unpair.1 = 1)  /\  P_1.X x.unpair.2 = P_1.X y.
    unpair.2)) := by
  rcases tag_trichotomy x.unpair.1 with hx | hx | <hx0, hx1> <;>
    rcases tag_trichotomy y.unpair.1 with hy | hy | <hy0, hy1>
  * -- x0 y0
    rw [sumEnum_zero hx, sumEnum_zero hy, inj_0_eq_iff]
    constructor
    * intro he; exact Or.inr (Or.inl <<hx, hy>, he>)
    * rintro (<<e1, e2>, _> | <_, he> | <<e1, e2>, _>)
      * exfalso; omega
      * exact he
      * exfalso; omega
  * -- x0 y1
    rw [sumEnum_zero hx, sumEnum_one hy]
    constructor
    * intro h; exact absurd h (inj_0_ne_inj_1_of_nonempty (h_1 _ (P_1.mem_X _)))
    * rintro (<<e1, e2>, _> | <<e1, e2>, _> | <<e1, e2>, _>) <;> exfalso <;> omega
  * -- x0 yM
    rw [sumEnum_zero hx, sumEnum_master hy0 hy1]
    constructor
    * intro h; exact absurd h inj_0_ne_sumMaster
    * rintro (<<e1, e2>, _> | <<e1, e2>, _> | <<e1, e2>, _>) <;> exfalso <;> omega
  * -- x1 y0
    rw [sumEnum_one hx, sumEnum_zero hy]
    constructor
    * intro h; exact absurd h.symm (inj_0_ne_inj_1_of_nonempty (h_1 _ (P_1.mem_X _)))
    * rintro (<<e1, e2>, _> | <<e1, e2>, _> | <<e1, e2>, _>) <;> exfalso <;> omega
  * -- x1 y1
    rw [sumEnum_one hx, sumEnum_one hy, inj_1_eq_iff]
    constructor
    * intro he; exact Or.inr (Or.inr <<hx, hy>, he>)
    * rintro (<<e1, e2>, _> | <<e1, e2>, _> | <_, he>)
      * exfalso; omega
      * exfalso; omega
      * exact he
  * -- x1 yM
    rw [sumEnum_one hx, sumEnum_master hy0 hy1]
    constructor
    * intro h; exact absurd h inj_1_ne_sumMaster
    * rintro (<<e1, e2>, _> | <<e1, e2>, _> | <<e1, e2>, _>) <;> exfalso <;> omega
  * -- xM y0
    rw [sumEnum_master hx0 hx1, sumEnum_zero hy]
    constructor
    * intro h; exact absurd h.symm inj_0_ne_sumMaster
    * rintro (<<e1, e2>, <e3, e4>> | <<e1, e2>, _> | <<e1, e2>, _>) <;> exfalso <;>
    omega
  * -- xM y1
    rw [sumEnum_master hx0 hx1, sumEnum_one hy]
```

```
    constructor
    * intro h; exact absurd h.symm inj_1_ne_sumMaster
    * rintro (<<e1, e2>, <e3, e4>> | <<e1, e2>, _> | <<e1, e2>, _>) <;> exfalso <;>
    omega
  * -- xM yM
    rw [sumEnum_master hx0 hx1, sumEnum_master hy0 hy1]
    exact <fun _ => Or.inl <<hx0, hx1>, <hy0, hy1>>, fun _ => rfl>

include h_1 in
/-- **`sumEnum u.1 = sumEnum u.2` is recursively decidable** (the `of_iff` translation
    of
`sumEnum_eq_iff` into the tag/component deciders). -/
theorem eqSEdec :
    RecDecidable (fun u => sumEnum P_0 P_1 u.unpair.1 = sumEnum P_0 P_1 u.unpair.2) :=
     by
  have hX0 : RecDecidable (fun u => u.unpair.1.unpair.1 = 0) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.left) (Nat.Primrec.const 0)
  have hX1 : RecDecidable (fun u => u.unpair.1.unpair.1 = 1) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.left) (Nat.Primrec.const 1)
  have hY0 : RecDecidable (fun u => u.unpair.2.unpair.1 = 0) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.right) (Nat.Primrec.const 0)
  have hY1 : RecDecidable (fun u => u.unpair.2.unpair.1 = 1) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.right) (Nat.Primrec.const 1)
  have hE0 : RecDecidable (fun u => P_0.X u.unpair.1.unpair.2 = P_0.X u.unpair.2.
    unpair.2) :=
    recDec_setEq_0 (Nat.Primrec.right.comp Nat.Primrec.left) (Nat.Primrec.right.comp
    Nat.Primrec.right)
  have hE1 : RecDecidable (fun u => P_1.X u.unpair.1.unpair.2 = P_1.X u.unpair.2.
    unpair.2) :=
    recDec_setEq_1 (Nat.Primrec.right.comp Nat.Primrec.left) (Nat.Primrec.right.comp
    Nat.Primrec.right)
  exact RecDecidable.of_iff (fun u => sumEnum_eq_iff (h_1 := h_1) u.unpair.1 u.unpair
    .2)
    (((hX0.not.and hX1.not).and (hY0.not.and hY1.not)).or
      (((hX0.and hY0).and hE0).or ((hX1.and hY1).and hE1)))

include h_0 h_1

/-- **Theorem 7.4 (Scott 1981, PRG-19) -- `?_0 + ?_1` is effectively given.** The tag
    enumeration
`Z (pair 0 n) = 0X?_n`, `Z (pair 1 n) = 1X1_n`, `Z (pair ( >= 2) _) = M`. Scott's 7.1(i
    )/(ii) follow the
intersection table: master absorbs, `0X  I  0X' = 0(X I X')` reduces to `?_0`, `0X  I
    1Y = {}` is
impossible (non-emptiness). The deciders are assembled from `eqSEdec`, the tag deciders
(`natEq`/`not`/`and`/`or`), and the components' `interEq`/`cons`. -/
def sumPresentation (P_0 : ComputablePresentation V_0) (P_1 : ComputablePresentation
    V_1) :
    ComputablePresentation (sum V_0 V_1 h_0 h_1) where
  X := sumEnum P_0 P_1
  mem_X t := sumEnum_mem t
  surj := by
    rintro W (rfl | <X, hX, rfl> | <Y, hY, rfl>)
    * exact <Nat.pair 2 0, sumEnum_master (by rw [unpair_pair_fst]; decide)
        (by rw [unpair_pair_fst]; decide)>
    * obtain <n, rfl> := P_0.surj hX
```

```
    exact <Nat.pair 0 n, by rw [sumEnum_zero (by rw [unpair_pair_fst]),
  unpair_pair_snd]>
  * obtain <n, rfl> := P_1.surj hY
    exact <Nat.pair 1 n, by rw [sumEnum_one (by rw [unpair_pair_fst]),
  unpair_pair_snd]>
interEq_computable := by
  have hMK : RecDecidable (fun t => sumEnum P_0 P_1 t.unpair.2.unpair.1
      = sumEnum P_0 P_1 t.unpair.2.unpair.2) := by
    refine RecDecidable.of_iff (fun t => ?_) ((eqSEdec (P_0 := P_0) (P_1 := P_1) (
  h_1 := h_1)).comp
      ((Nat.Primrec.left.comp Nat.Primrec.right).pair (Nat.Primrec.right.comp Nat.
  Primrec.right)))
    simp only [unpair_pair_fst, unpair_pair_snd]
  have hNK : RecDecidable (fun t => sumEnum P_0 P_1 t.unpair.1
      = sumEnum P_0 P_1 t.unpair.2.unpair.2) := by
    refine RecDecidable.of_iff (fun t => ?_) ((eqSEdec (P_0 := P_0) (P_1 := P_1) (
  h_1 := h_1)).comp
      (Nat.Primrec.left.pair (Nat.Primrec.right.comp Nat.Primrec.right)))
    simp only [unpair_pair_fst, unpair_pair_snd]
  have hA0 : RecDecidable (fun t => t.unpair.1.unpair.1 = 0) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.left) (Nat.Primrec.const
  0)
  have hA1 : RecDecidable (fun t => t.unpair.1.unpair.1 = 1) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.left) (Nat.Primrec.const
  1)
  have hB0 : RecDecidable (fun t => t.unpair.2.unpair.1.unpair.1 = 0) :=
    RecDecidable.natEq (Nat.Primrec.left.comp (Nat.Primrec.left.comp Nat.Primrec.
  right))
      (Nat.Primrec.const 0)
  have hB1 : RecDecidable (fun t => t.unpair.2.unpair.1.unpair.1 = 1) :=
    RecDecidable.natEq (Nat.Primrec.left.comp (Nat.Primrec.left.comp Nat.Primrec.
  right))
      (Nat.Primrec.const 1)
  have hC0 : RecDecidable (fun t => t.unpair.2.unpair.2.unpair.1 = 0) :=
    RecDecidable.natEq (Nat.Primrec.left.comp (Nat.Primrec.right.comp Nat.Primrec.
  right))
      (Nat.Primrec.const 0)
  have hC1 : RecDecidable (fun t => t.unpair.2.unpair.2.unpair.1 = 1) :=
    RecDecidable.natEq (Nat.Primrec.left.comp (Nat.Primrec.right.comp Nat.Primrec.
  right))
      (Nat.Primrec.const 1)
  have hg : Nat.Primrec (fun t => Nat.pair t.unpair.1.unpair.2
      (Nat.pair t.unpair.2.unpair.1.unpair.2 t.unpair.2.unpair.2.unpair.2)) :=
    (Nat.Primrec.right.comp Nat.Primrec.left).pair
      ((Nat.Primrec.right.comp (Nat.Primrec.left.comp Nat.Primrec.right)).pair
        (Nat.Primrec.right.comp (Nat.Primrec.right.comp Nat.Primrec.right)))
  have hIE0 : RecDecidable (fun t => P_0.X t.unpair.1.unpair.2  I  P_0.X t.unpair.2.
  unpair.1.unpair.2
      = P_0.X t.unpair.2.unpair.2.unpair.2) := by
    refine RecDecidable.of_iff (fun t => ?_) (P_0.interEq_computable.comp hg)
    simp only [unpair_pair_fst, unpair_pair_snd]
  have hIE1 : RecDecidable (fun t => P_1.X t.unpair.1.unpair.2  I  P_1.X t.unpair.2.
  unpair.1.unpair.2
      = P_1.X t.unpair.2.unpair.2.unpair.2) := by
    refine RecDecidable.of_iff (fun t => ?_) (P_1.interEq_computable.comp hg)
    simp only [unpair_pair_fst, unpair_pair_snd]
```

```
refine RecDecidable.of_iff (fun t => ?_)
  (((hA0.not.and hA1.not).and hMK).or
    (((hB0.not.and hB1.not).and hNK).or
      ((((hA0.and hB0).and hC0).and hIE0).or
        (((hA1.and hB1).and hC1).and hIE1))))
dsimp only
rcases tag_trichotomy t.unpair.1.unpair.1 with hna | hna | <hna0, hna1>
* rcases tag_trichotomy t.unpair.2.unpair.1.unpair.1 with hnb | hnb | <hnb0, hnb1>
  * -- na = 0, nb = 0
    rw [sumEnum_zero hna, sumEnum_zero hnb, inj_0_inter]
    rcases tag_trichotomy t.unpair.2.unpair.2.unpair.1 with hnc | hnc | <hnc0,
hnc1>
    * rw [sumEnum_zero hnc, inj_0_eq_iff]
      constructor
      * intro h; exact Or.inr (Or.inr (Or.inl <<<hna, hnb>, hnc>, h>))
      * rintro (<<e1, _>, _> | <<e1, _>, _> | <<<_, _>, _>, h> | <<<e1, _>, _>, _
>)
        * exfalso; omega
        * exfalso; omega
        * exact h
        * exfalso; omega
    * rw [sumEnum_one hnc]
      constructor
      * intro h; exact absurd h (inj_0_ne_inj_1_of_nonempty (h_1 _ (P_1.mem_X _)))
      * rintro (<<e1, _>, _> | <<e1, _>, _> | <<<_, _>, e3>, _> | <<<e1, _>, _>, _
>) <;> exfalso <;> omega
    * rw [sumEnum_master hnc0 hnc1]
      constructor
      * intro h; exact absurd h inj_0_ne_sumMaster
      * rintro (<<e1, _>, _> | <<e1, _>, _> | <<<_, _>, e3>, _> | <<<e1, _>, _>, _
>) <;> exfalso <;> omega
  * -- na = 0, nb = 1: {}
    rw [sumEnum_zero hna, sumEnum_one hnb, inj_0_inter_inj_1]
    constructor
    * intro h
      obtain <x, hx> := sumEnum_nonempty (h_0 := h_0) (h_1 := h_1) t.unpair.2.
unpair.2
      rw [ <-  h] at hx
      exact absurd hx (Set.notMem_empty x)
    * rintro (<<e1, _>, _> | <<_, e2>, _> | <<<_, e2>, _>, _> | <<<e1, _>, _>, _>)
 <;> exfalso <;> omega
  * -- na = 0, nb = M: master on the right
    rw [sumEnum_master hnb0 hnb1, sumEnum_inter_sumMaster]
    constructor
    * intro h; exact Or.inr (Or.inl <<hnb0, hnb1>, h>)


    * rintro (<<e1, _>, _> | <_, h> | <<<_, e2>, _>, _> | <<<e1, _>, _>, _>)
      * exfalso; omega
      * exact h
      * exfalso; omega
      * exfalso; omega
* rcases tag_trichotomy t.unpair.2.unpair.1.unpair.1 with hnb | hnb | <hnb0, hnb1>
  * -- na = 1, nb = 0: {}
    rw [sumEnum_one hna, sumEnum_zero hnb, Set.inter_comm, inj_0_inter_inj_1]
    constructor
    * intro h
```

```
      obtain <x, hx> := sumEnum_nonempty (h_0 := h_0) (h_1 := h_1) t.unpair.2.
  unpair.2
      rw [ <-  h] at hx
      exact absurd hx (Set.notMem_empty x)
    * rintro (<<_, e2>, _> | <<e1, _>, _> | <<<e1, _>, _>, _> | <<<_, e2>, _>, _>)
   <;> exfalso <;> omega
    * -- na = 1, nb = 1
      rw [sumEnum_one hna, sumEnum_one hnb, inj_1_inter]
      rcases tag_trichotomy t.unpair.2.unpair.2.unpair.1 with hnc | hnc | <hnc0,
  hnc1>
      * rw [sumEnum_zero hnc]
        constructor
        * intro h; exact (inj_0_eq_inj_1_elim (h_0 _ (P_0.mem_X _)) h.symm).elim
        * rintro (<<_, e2>, _> | <<_, e2>, _> | <<<e1, _>, _>, _> | <<<_, _>, e3>, _
  >) <;> exfalso <;> omega
      * rw [sumEnum_one hnc, inj_1_eq_iff]
        constructor
        * intro h; exact Or.inr (Or.inr (Or.inr <<<hna, hnb>, hnc>, h>))
        * rintro (<<_, e2>, _> | <<_, e2>, _> | <<<e1, _>, _>, _> | <<<_, _>, _>, h
  >)
          * exfalso; omega
          * exfalso; omega
          * exfalso; omega
          * exact h
      * rw [sumEnum_master hnc0 hnc1]
        constructor
        * intro h; exact absurd h inj_1_ne_sumMaster
        * rintro (<<_, e2>, _> | <<_, e2>, _> | <<<e1, _>, _>, _> | <<<_, _>, e3>, _
  >) <;> exfalso <;> omega
    * -- na = 1, nb = M
      rw [sumEnum_master hnb0 hnb1, sumEnum_inter_sumMaster]
      constructor
      * intro h; exact Or.inr (Or.inl <<hnb0, hnb1>, h>)
      * rintro (<<_, e2>, _> | <_, h> | <<<_, e2>, _>, _> | <<<_, e2>, _>, _>)
        * exfalso; omega
        * exact h
        * exfalso; omega
        * exfalso; omega
  * -- na = M: master on the left (any nb)
    rw [sumEnum_master hna0 hna1, sumMaster_inter_sumEnum]
    constructor
    * intro h; exact Or.inl <<hna0, hna1>, h>
    * rintro (<_, h> | <<e1, e2>, h> | <<<e1, _>, _>, _> | <<<e1, _>, _>, _>)
      * exact h
      * rw [sumEnum_master e1 e2]; exact h
      * exfalso; omega
      * exfalso; omega
cons_computable := by
  have hA0 : RecDecidable (fun s => s.unpair.1.unpair.1 = 0) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.left) (Nat.Primrec.const
  0)
  have hA1 : RecDecidable (fun s => s.unpair.1.unpair.1 = 1) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.left) (Nat.Primrec.const
  1)
  have hB0 : RecDecidable (fun s => s.unpair.2.unpair.1 = 0) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.right) (Nat.Primrec.const
```

```
0)
have hB1 : RecDecidable (fun s => s.unpair.2.unpair.1 = 1) :=
  RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.right) (Nat.Primrec.const
1)
have hgc : Nat.Primrec (fun s => Nat.pair s.unpair.1.unpair.2 s.unpair.2.unpair.2)
 :=
  (Nat.Primrec.right.comp Nat.Primrec.left).pair (Nat.Primrec.right.comp Nat.
Primrec.right)
have hC0 : RecDecidable (fun s => exists  k, P_0.X k  subseteq  P_0.X s.unpair.1.
unpair.2  I  P_0.X s.unpair.2.unpair.2)
    := by
  refine RecDecidable.of_iff (fun s => ?_) (P_0.cons_computable.comp hgc)
  simp only [unpair_pair_fst, unpair_pair_snd]
have hC1 : RecDecidable (fun s => exists  k, P_1.X k  subseteq  P_1.X s.unpair.1.
unpair.2  I  P_1.X s.unpair.2.unpair.2)
    := by
  refine RecDecidable.of_iff (fun s => ?_) (P_1.cons_computable.comp hgc)
  simp only [unpair_pair_fst, unpair_pair_snd]
refine RecDecidable.of_iff (fun s => ?_)
  ((hA0.not.and hA1.not).or
    ((hB0.not.and hB1.not).or
      (((hA0.and hB0).and hC0).or ((hA1.and hB1).and hC1))))
dsimp only
rcases tag_trichotomy s.unpair.1.unpair.1 with hna | hna | <hna0, hna1>
* rcases tag_trichotomy s.unpair.2.unpair.1 with hnb | hnb | <hnb0, hnb1>
  * -- na = 0, nb = 0
    rw [sumEnum_zero hna, sumEnum_zero hnb, inj_0_inter]
    constructor
    * rintro <k, hk>
      obtain <X_2, hX_2mem, hX_2eq> := mem_subset_inj_0 (sumEnum_mem (h_0 := h_0)
(h_1 := h_1) k) hk
      obtain <j, hj> := P_0.surj hX_2mem
      refine Or.inr (Or.inr (Or.inl <<hna, hnb>, j, ?_>))
      rw [hj]
      rw [hX_2eq] at hk
      exact inj_0_subset_inj_0.mp hk
    * rintro (<e1, _> | <e1, _> | <<_, _>, j, hj> | <<e1, _>, _>)
      * exfalso; omega
      * exfalso; omega
      * refine <Nat.pair 0 j, ?_>
        rw [sumEnum_zero (by rw [unpair_pair_fst]), unpair_pair_snd]
        exact inj_0_subset_inj_0.mpr hj
      * exfalso; omega
  * -- na = 0, nb = 1: {}
    rw [sumEnum_zero hna, sumEnum_one hnb, inj_0_inter_inj_1]
    constructor
    * rintro <k, hk>
      obtain <x, hx> := sumEnum_nonempty (h_0 := h_0) (h_1 := h_1) k
      exact absurd (hk hx) (Set.notMem_empty x)
    * rintro (<e1, _> | <_, e2> | <<_, e2>, _> | <<e1, _>, _>) <;> exfalso <;>
omega
  * -- na = 0, nb = M: right master
    rw [sumEnum_master hnb0 hnb1, sumEnum_inter_sumMaster]
    exact <fun _ => Or.inr (Or.inl <hnb0, hnb1>), fun _ => <s.unpair.1, subset_rfl
>>
* rcases tag_trichotomy s.unpair.2.unpair.1 with hnb | hnb | <hnb0, hnb1>
```

```
    * -- na = 1, nb = 0: {}
      rw [sumEnum_one hna, sumEnum_zero hnb, Set.inter_comm, inj_0_inter_inj_1]
      constructor
      * rintro <k, hk>
        obtain <x, hx> := sumEnum_nonempty (h_0 := h_0) (h_1 := h_1) k
        exact absurd (hk hx) (Set.notMem_empty x)
      * rintro (<_, e2> | <e1, _> | <<e1, _>, _> | <<_, e2>, _>) <;> exfalso <;>
  omega
    * -- na = 1, nb = 1
      rw [sumEnum_one hna, sumEnum_one hnb, inj_1_inter]
      constructor
      * rintro <k, hk>
        obtain <Y_2, hY_2mem, hY_2eq> := mem_subset_inj_1 (sumEnum_mem (h_0 := h_0)
  (h_1 := h_1) k) hk
        obtain <j, hj> := P_1.surj hY_2mem
        refine Or.inr (Or.inr (Or.inr <<hna, hnb>, j, ?_>))
        rw [hj]
        rw [hY_2eq] at hk
        exact inj_1_subset_inj_1.mp hk
      * rintro (<_, e2> | <_, e2> | <<e1, _>, _> | <<_, _>, j, hj>)
        * exfalso; omega
        * exfalso; omega
        * exfalso; omega
        * refine <Nat.pair 1 j, ?_>
          rw [sumEnum_one (by rw [unpair_pair_fst]), unpair_pair_snd]
          exact inj_1_subset_inj_1.mpr hj
    * -- na = 1, nb = M: right master
      rw [sumEnum_master hnb0 hnb1, sumEnum_inter_sumMaster]
      exact <fun _ => Or.inr (Or.inl <hnb0, hnb1>), fun _ => <s.unpair.1, subset_rfl
  >>
  * -- na = M: left master (any nb)
    rw [sumEnum_master hna0 hna1, sumMaster_inter_sumEnum]
    exact <fun _ => Or.inl <hna0, hna1>, fun _ => <s.unpair.2, subset_rfl>>
inter n m :=
  selectFn (1 - (2 - n.unpair.1)) m
    (selectFn (1 - (2 - m.unpair.1)) n
      (selectFn (1 - n.unpair.1)
        (Nat.pair 0 (P_0.inter n.unpair.2 m.unpair.2))
        (Nat.pair 1 (P_1.inter n.unpair.2 m.unpair.2))))
inter_primrec := by
  have hnTag : Nat.Primrec (fun t => t.unpair.1.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.left
  have hmTag : Nat.Primrec (fun t => t.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.right
  have hnIdx : Nat.Primrec (fun t => t.unpair.1.unpair.2) :=
    Nat.Primrec.right.comp Nat.Primrec.left
  have hmIdx : Nat.Primrec (fun t => t.unpair.2.unpair.2) :=
    Nat.Primrec.right.comp Nat.Primrec.right
  have hP0 : Nat.Primrec (fun t => Nat.pair 0 (P_0.inter t.unpair.1.unpair.2 t.
  unpair.2.unpair.2)) :=
    (Nat.Primrec.const 0).pair ((P_0.inter_primrec.comp (hnIdx.pair hmIdx)).of_eq
  fun t => by
      simp only [unpair_pair_fst, unpair_pair_snd])
  have hP1 : Nat.Primrec (fun t => Nat.pair 1 (P_1.inter t.unpair.1.unpair.2 t.
  unpair.2.unpair.2)) :=
    (Nat.Primrec.const 1).pair ((P_1.inter_primrec.comp (hnIdx.pair hmIdx)).of_eq
```

```
  fun t => by
      simp only [unpair_pair_fst, unpair_pair_snd])
  have hcnMaster : Nat.Primrec (fun t => 1 - (2 - t.unpair.1.unpair.1)) :=
    primrec_sub_2 (Nat.Primrec.const 1) (primrec_sub_2 (Nat.Primrec.const 2) hnTag)
  have hcmMaster : Nat.Primrec (fun t => 1 - (2 - t.unpair.2.unpair.1)) :=
    primrec_sub_2 (Nat.Primrec.const 1) (primrec_sub_2 (Nat.Primrec.const 2) hmTag)
  have hcnZero : Nat.Primrec (fun t => 1 - t.unpair.1.unpair.1) :=
    primrec_sub_2 (Nat.Primrec.const 1) hnTag
  exact primrec_selectFn hcnMaster Nat.Primrec.right
    (primrec_selectFn hcmMaster Nat.Primrec.left
      (primrec_selectFn hcnZero hP0 hP1))
inter_spec := by
  rintro n m <k, hk>
  rcases tag_trichotomy n.unpair.1 with hna | hna | <hna0, hna1>
  * -- n is a left copy (tag 0)
    rw [show 1 - (2 - n.unpair.1) = 0 by omega, selectFn_zero]
    rcases tag_trichotomy m.unpair.1 with hmb | hmb | <hmb0, hmb1>
    * -- m left copy: consistent left  I  left
      rw [show 1 - (2 - m.unpair.1) = 0 by omega, selectFn_zero,
        show 1 - n.unpair.1 = 1 by omega, selectFn_one,
        sumEnum_zero (by rw [unpair_pair_fst]), unpair_pair_snd,
        sumEnum_zero hna, sumEnum_zero hmb, inj_0_inter]
      rw [sumEnum_zero hna, sumEnum_zero hmb, inj_0_inter] at hk
      obtain <X_2, hX_2mem, hX_2eq> := mem_subset_inj_0 (sumEnum_mem (h_0 := h_0) (
  h_1 := h_1) k) hk
      obtain <j, hj> := P_0.surj hX_2mem
      rw [hX_2eq] at hk
      have hjsub : P_0.X j  subseteq  P_0.X n.unpair.2  I  P_0.X m.unpair.2 := by
        rw [hj]; exact inj_0_subset_inj_0.mp hk
      rw [P_0.inter_spec <j, hjsub>]
    * -- m right copy: inconsistent ({}), refute consistency
      exfalso
      rw [sumEnum_zero hna, sumEnum_one hmb, inj_0_inter_inj_1] at hk
      obtain <x, hx> := sumEnum_nonempty (h_0 := h_0) (h_1 := h_1) k
      exact absurd (hk hx) (Set.notMem_empty x)
    * -- m master
      rw [show 1 - (2 - m.unpair.1) = 1 by omega, selectFn_one,
        sumEnum_master hmb0 hmb1, sumEnum_inter_sumMaster]
  * -- n is a right copy (tag 1)
    rw [show 1 - (2 - n.unpair.1) = 0 by omega, selectFn_zero]
    rcases tag_trichotomy m.unpair.1 with hmb | hmb | <hmb0, hmb1>
    * -- m left copy: inconsistent
      exfalso
      rw [sumEnum_one hna, sumEnum_zero hmb, Set.inter_comm, inj_0_inter_inj_1] at
  hk
      obtain <x, hx> := sumEnum_nonempty (h_0 := h_0) (h_1 := h_1) k
      exact absurd (hk hx) (Set.notMem_empty x)
    * -- m right copy: consistent right  I  right
      rw [show 1 - (2 - m.unpair.1) = 0 by omega, selectFn_zero,
        show 1 - n.unpair.1 = 0 by omega, selectFn_zero,
        sumEnum_one (by rw [unpair_pair_fst]), unpair_pair_snd,
        sumEnum_one hna, sumEnum_one hmb, inj_1_inter]
      rw [sumEnum_one hna, sumEnum_one hmb, inj_1_inter] at hk
      obtain <Y_2, hY_2mem, hY_2eq> := mem_subset_inj_1 (sumEnum_mem (h_0 := h_0) (
  h_1 := h_1) k) hk
      obtain <j, hj> := P_1.surj hY_2mem
```

```
        rw [hY_2eq] at hk
        have hjsub : P_1.X j  subseteq  P_1.X n.unpair.2  I  P_1.X m.unpair.2 := by
          rw [hj]; exact inj_1_subset_inj_1.mp hk
        rw [P_1.inter_spec <j, hjsub>]
      * -- m master
        rw [show 1 - (2 - m.unpair.1) = 1 by omega, selectFn_one,
          sumEnum_master hmb0 hmb1, sumEnum_inter_sumMaster]
    * -- n master
      rw [show 1 - (2 - n.unpair.1) = 1 by omega, selectFn_one,
        sumEnum_master hna0 hna1, sumMaster_inter_sumEnum]
  masterIdx := Nat.pair 2 0
  masterIdx_spec := by
    rw [sumEnum_master (by rw [unpair_pair_fst]; decide) (by rw [unpair_pair_fst];
    decide)]
    rfl

/-- **Theorem 7.4 (Scott 1981, PRG-19).** The sum of effectively given domains is
    effectively given. -/
theorem sum_isEffectivelyGiven (g_0 : V_0.IsEffectivelyGiven) (g_1 : V_1.
    IsEffectivelyGiven) :
    (sum V_0 V_1 h_0 h_1).IsEffectivelyGiven := by
  obtain <P_0> := g_0; obtain <P_1> := g_1
  exact <sumPresentation P_0 P_1>

@[simp] theorem sumPresentation_X (P_0 : ComputablePresentation V_0) (P_1 :
    ComputablePresentation V_1)
    (t : Nat) : (sumPresentation P_0 P_1 (h_0 := h_0) (h_1 := h_1)).X t = sumEnum P_0
    P_1 t := rfl

/-- **Theorem 7.4 (Scott 1981, PRG-19) -- `in_0` is computable.** `X?_n (in_0) Z_m  <->
      0X?_n  subseteq  Z_m`, which
tag-decodes to `(m = 0X?  /\  X?_n  subseteq  X?_{m.2})  \/  (m = master)` (`m = 1X1`
    is impossible as `X?_n  !=  {}`). -/
theorem inMap_0_isComputable (P_0 : ComputablePresentation V_0) (P_1 :
    ComputablePresentation V_1) :
    ComputableMap P_0 (sumPresentation P_0 P_1) (inMap_0 (h_0 := h_0) (h_1 := h_1)) :=
     by
  constructor
  have hincl : RecDecidable (fun s => P_0.X s.unpair.1  subseteq  P_0.X s.unpair.2) :=
     P_0.incl_computable
  have hmtag0 : RecDecidable (fun s => s.unpair.2.unpair.1 = 0) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.right) (Nat.Primrec.const 0)
  have hmtag1 : RecDecidable (fun s => s.unpair.2.unpair.1 = 1) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.right) (Nat.Primrec.const 1)
  have hincl0 : RecDecidable (fun s => P_0.X s.unpair.1  subseteq  P_0.X s.unpair.2.
    unpair.2) :=
    RecDecidable.of_iff (fun s => by simp only [unpair_pair_fst, unpair_pair_snd])
      (hincl.comp (Nat.Primrec.left.pair (Nat.Primrec.right.comp Nat.Primrec.right)))
  refine (RecDecidable.of_iff (fun s => ?_)
    ((hmtag0.and hincl0).or (hmtag0.not.and hmtag1.not))).re
  simp only [sumPresentation_X]
  rcases tag_trichotomy s.unpair.2.unpair.1 with hm | hm | <hm0, hm1>
  * constructor
    * rintro <_, _, hsub>
      rw [sumEnum_zero hm] at hsub
      exact Or.inl <hm, inj_0_subset_inj_0.mp hsub>
```

```
    * rintro (<_, hsub> | <e1, _>)
      * refine <P_0.mem_X _, sumEnum_mem _, ?_>
        rw [sumEnum_zero hm]; exact inj_0_subset_inj_0.mpr hsub
      * exfalso; omega
  * constructor
    * rintro <_, _, hsub>
      rw [sumEnum_one hm] at hsub
      exact (not_inj_0_subset_inj_1 (h_0 _ (P_0.mem_X _)) hsub).elim
    * rintro (<e1, _> | <_, e2>) <;> exfalso <;> omega
  * constructor
    * intro _; exact Or.inr <hm0, hm1>
    * intro _
      refine <P_0.mem_X _, sumEnum_mem _, ?_>
      rw [sumEnum_master hm0 hm1]; exact inj_0_subset_sumMaster (P_0.mem_X _)

/-- **Theorem 7.4 (Scott 1981, PRG-19) -- `in_1` is computable.** Symmetric to `in_0`.
    -/
theorem inMap_1_isComputable (P_0 : ComputablePresentation V_0) (P_1 :
    ComputablePresentation V_1) :
    ComputableMap P_1 (sumPresentation P_0 P_1) (inMap_1 (h_0 := h_0) (h_1 := h_1)) :=
     by
  constructor
  have hincl : RecDecidable (fun s => P_1.X s.unpair.1  subseteq  P_1.X s.unpair.2) :=
     P_1.incl_computable
  have hmtag0 : RecDecidable (fun s => s.unpair.2.unpair.1 = 0) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.right) (Nat.Primrec.const 0)
  have hmtag1 : RecDecidable (fun s => s.unpair.2.unpair.1 = 1) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.right) (Nat.Primrec.const 1)
  have hincl1 : RecDecidable (fun s => P_1.X s.unpair.1  subseteq  P_1.X s.unpair.2.
    unpair.2) :=
    RecDecidable.of_iff (fun s => by simp only [unpair_pair_fst, unpair_pair_snd])
      (hincl.comp (Nat.Primrec.left.pair (Nat.Primrec.right.comp Nat.Primrec.right)))
  refine (RecDecidable.of_iff (fun s => ?_)
    ((hmtag1.and hincl1).or (hmtag0.not.and hmtag1.not))).re
  simp only [sumPresentation_X]
  rcases tag_trichotomy s.unpair.2.unpair.1 with hm | hm | <hm0, hm1>
  * constructor
    * rintro <_, _, hsub>
      rw [sumEnum_zero hm] at hsub
      exact (not_inj_1_subset_inj_0 (h_1 _ (P_1.mem_X _)) hsub).elim
    * rintro (<e1, _> | <_, e2>) <;> exfalso <;> omega
  * constructor
    * rintro <_, _, hsub>
      rw [sumEnum_one hm] at hsub
      exact Or.inl <hm, inj_1_subset_inj_1.mp hsub>
    * rintro (<_, hsub> | <e1, _>)
      * refine <P_1.mem_X _, sumEnum_mem _, ?_>
        rw [sumEnum_one hm]; exact inj_1_subset_inj_1.mpr hsub
      * exfalso; omega
  * constructor
    * intro _; exact Or.inr <hm0, hm1>
    * intro _
      refine <P_1.mem_X _, sumEnum_mem _, ?_>
      rw [sumEnum_master hm0 hm1]; exact inj_1_subset_sumMaster (P_1.mem_X _)

/-- **Theorem 7.4 (Scott 1981, PRG-19) -- `out_0` is computable.** `Z_n (out_0) X?_m
```

```
    <->  leftPart Z_n  subseteq
X?_m`; tag-decoded, `leftPart` is `X?_{n.2}` on a left copy and `Delta_0` on a right
    copy/master, so the
relation is `incl` against either `n.2` or the master index `k_0` (`X?_{k_0} = Delta_0
    `). -/
theorem outMap_0_isComputable (P_0 : ComputablePresentation V_0) (P_1 :
    ComputablePresentation V_1) :
    ComputableMap (sumPresentation P_0 P_1) P_0 (outMap_0 (h_0 := h_0) (h_1 := h_1))
    := by
  constructor
  obtain <k0, hk0> := P_0.surj V_0.master_mem
  have hincl : RecDecidable (fun s => P_0.X s.unpair.1  subseteq  P_0.X s.unpair.2) :=
     P_0.incl_computable
  have hntag0 : RecDecidable (fun s => s.unpair.1.unpair.1 = 0) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.left) (Nat.Primrec.const 0)
  have hincl_left : RecDecidable (fun s => P_0.X s.unpair.1.unpair.2  subseteq  P_0.X
    s.unpair.2) :=
    RecDecidable.of_iff (fun s => by simp only [unpair_pair_fst, unpair_pair_snd])
      (hincl.comp ((Nat.Primrec.right.comp Nat.Primrec.left).pair Nat.Primrec.right))
  have hincl_master : RecDecidable (fun s => P_0.X k0  subseteq  P_0.X s.unpair.2) :=
    RecDecidable.of_iff (fun s => by simp only [unpair_pair_fst, unpair_pair_snd])
      (hincl.comp ((Nat.Primrec.const k0).pair Nat.Primrec.right))
  refine (RecDecidable.of_iff (fun s => ?_)
    ((hntag0.and hincl_left).or (hntag0.not.and hincl_master))).re
  simp only [sumPresentation_X]
  rcases tag_trichotomy s.unpair.1.unpair.1 with hn | hn | <hn0, hn1>
  * constructor
    * rintro <_, _, hsub>
      rw [sumEnum_zero hn, leftPart_inj_0] at hsub
      exact Or.inl <hn, hsub>
    * rintro (<_, hsub> | <e1, _>)
      * refine <sumEnum_mem _, P_0.mem_X _, ?_>
        rw [sumEnum_zero hn, leftPart_inj_0]; exact hsub
      * exfalso; omega
  * constructor
    * rintro <_, _, hsub>
      rw [sumEnum_one hn, leftPart_inj_1 V_0 (h_1 _ (P_1.mem_X _))] at hsub
      refine Or.inr <by omega, ?_>
      rw [hk0]; exact hsub
    * rintro (<e1, _> | <_, hsub>)
      * exfalso; omega
      * refine <sumEnum_mem _, P_0.mem_X _, ?_>
        rw [sumEnum_one hn, leftPart_inj_1 V_0 (h_1 _ (P_1.mem_X _)),  <-  hk0]; exact
     hsub
  * constructor
    * rintro <_, _, hsub>
      rw [sumEnum_master hn0 hn1, leftPart_sumMaster] at hsub
      refine Or.inr <hn0, ?_>
      rw [hk0]; exact hsub
    * rintro (<e1, _> | <_, hsub>)
      * exfalso; omega
      * refine <sumEnum_mem _, P_0.mem_X _, ?_>
        rw [sumEnum_master hn0 hn1, leftPart_sumMaster,  <-  hk0]; exact hsub

/-- **Theorem 7.4 (Scott 1981, PRG-19) -- `out_1` is computable.** Symmetric to `out_0`
     via `rightPart`. -/
```

```
theorem outMap_1_isComputable (P_0 : ComputablePresentation V_0) (P_1 :
    ComputablePresentation V_1) :
    ComputableMap (sumPresentation P_0 P_1) P_1 (outMap_1 (h_0 := h_0) (h_1 := h_1))
    := by
  constructor
  obtain <k1, hk1> := P_1.surj V_1.master_mem
  have hincl : RecDecidable (fun s => P_1.X s.unpair.1  subseteq  P_1.X s.unpair.2) :=
     P_1.incl_computable
  have hntag1 : RecDecidable (fun s => s.unpair.1.unpair.1 = 1) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.left) (Nat.Primrec.const 1)
  have hincl_right : RecDecidable (fun s => P_1.X s.unpair.1.unpair.2  subseteq  P_1.X
     s.unpair.2) :=
    RecDecidable.of_iff (fun s => by simp only [unpair_pair_fst, unpair_pair_snd])
      (hincl.comp ((Nat.Primrec.right.comp Nat.Primrec.left).pair Nat.Primrec.right))
  have hincl_master : RecDecidable (fun s => P_1.X k1  subseteq  P_1.X s.unpair.2) :=
    RecDecidable.of_iff (fun s => by simp only [unpair_pair_fst, unpair_pair_snd])
      (hincl.comp ((Nat.Primrec.const k1).pair Nat.Primrec.right))
  refine (RecDecidable.of_iff (fun s => ?_)
    ((hntag1.and hincl_right).or (hntag1.not.and hincl_master))).re
  simp only [sumPresentation_X]
  rcases tag_trichotomy s.unpair.1.unpair.1 with hn | hn | <hn0, hn1>
  * constructor
    * rintro <_, _, hsub>
      rw [sumEnum_zero hn, rightPart_inj_0 V_1 (h_0 _ (P_0.mem_X _))] at hsub
      refine Or.inr <by omega, ?_>
      rw [hk1]; exact hsub
    * rintro (<e1, _> | <_, hsub>)
      * exfalso; omega
      * refine <sumEnum_mem _, P_1.mem_X _, ?_>
        rw [sumEnum_zero hn, rightPart_inj_0 V_1 (h_0 _ (P_0.mem_X _)),  <-  hk1];
    exact hsub
  * constructor
    * rintro <_, _, hsub>
      rw [sumEnum_one hn, rightPart_inj_1] at hsub
      exact Or.inl <hn, hsub>
    * rintro (<_, hsub> | <e1, _>)
      * refine <sumEnum_mem _, P_1.mem_X _, ?_>
        rw [sumEnum_one hn, rightPart_inj_1]; exact hsub
      * exfalso; omega
  * constructor
    * rintro <_, _, hsub>
      rw [sumEnum_master hn0 hn1, rightPart_sumMaster] at hsub
      refine Or.inr <hn1, ?_>
      rw [hk1]; exact hsub
    * rintro (<e1, _> | <_, hsub>)
      * exfalso; omega
      * refine <sumEnum_mem _, P_1.mem_X _, ?_>
        rw [sumEnum_master hn0 hn1, rightPart_sumMaster,  <-  hk1]; exact hsub

/-- **Theorem 7.4 (Scott 1981, PRG-19) -- `f + g` is computable.** Tag-decoding the sum
     relation,
`Z_n (f+g) W_m` holds iff `W_m` is the codomain master (decidable in `m`'s tag), or
    both are left
copies (tag `0`) with `X?_{n.2} f Y?_{m.2}`, or both are right copies (tag `1`) with
`X1_{n.2} g Y1_{m.2}`. The three disjuncts are r.e. (`REPred.or`): each copy case
    conjoins a decidable
```

```
tag test with the reindexed r.e. relation of `f`/`g`. -/
theorem sumMap_isComputable {V_0' : NeighborhoodSystem a'} {V_1' : NeighborhoodSystem
    b'}

    {h_0' : forall  X, V_0'.mem X  ->  X.Nonempty} {h_1' : forall  Y, V_1'.mem Y  ->
    Y.Nonempty}
    (Q_0 : ComputablePresentation V_0') (Q_1 : ComputablePresentation V_1')
    {f : ApproximableMap V_0 V_0'} {g : ApproximableMap V_1 V_1'}
    (hf : ComputableMap P_0 Q_0 f) (hg : ComputableMap P_1 Q_1 g) :
    ComputableMap (sumPresentation P_0 P_1 (h_0 := h_0) (h_1 := h_1))
      (sumPresentation Q_0 Q_1 (h_0 := h_0') (h_1 := h_1'))
      (sumMap (h_0 := h_0) (h_1 := h_1) (h_0' := h_0') (h_1' := h_1') f g) := by
  constructor
  have hf' : REPred (fun u => f.rel (P_0.X u.unpair.1) (Q_0.X u.unpair.2)) := hf.out
  have hg' : REPred (fun u => g.rel (P_1.X u.unpair.1) (Q_1.X u.unpair.2)) := hg.out
  have hNtag0 : RecDecidable (fun s => s.unpair.1.unpair.1 = 0) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.left) (Nat.Primrec.const 0)
  have hNtag1 : RecDecidable (fun s => s.unpair.1.unpair.1 = 1) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.left) (Nat.Primrec.const 1)
  have hMtag0 : RecDecidable (fun s => s.unpair.2.unpair.1 = 0) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.right) (Nat.Primrec.const 0)
  have hMtag1 : RecDecidable (fun s => s.unpair.2.unpair.1 = 1) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.right) (Nat.Primrec.const 1)
  have hfr : REPred (fun s => f.rel (P_0.X s.unpair.1.unpair.2) (Q_0.X s.unpair.2.
    unpair.2)) :=
    REPred.of_iff (fun s => by simp only [unpair_pair_fst, unpair_pair_snd])
      (hf'.comp ((Nat.Primrec.right.comp Nat.Primrec.left).pair
        (Nat.Primrec.right.comp Nat.Primrec.right)))
  have hgr : REPred (fun s => g.rel (P_1.X s.unpair.1.unpair.2) (Q_1.X s.unpair.2.
    unpair.2)) :=
    REPred.of_iff (fun s => by simp only [unpair_pair_fst, unpair_pair_snd])
      (hg'.comp ((Nat.Primrec.right.comp Nat.Primrec.left).pair
        (Nat.Primrec.right.comp Nat.Primrec.right)))
  have hA : REPred (fun s => s.unpair.2.unpair.1  !=  0  /\  s.unpair.2.unpair.1  !=
    1) :=
    (hMtag0.not.and hMtag1.not).re
  have hB : REPred (fun s => (s.unpair.1.unpair.1 = 0  /\  s.unpair.2.unpair.1 = 0)
       /\  f.rel (P_0.X s.unpair.1.unpair.2) (Q_0.X s.unpair.2.unpair.2)) :=
    ((hNtag0.and hMtag0).re).and hfr
  have hC : REPred (fun s => (s.unpair.1.unpair.1 = 1  /\  s.unpair.2.unpair.1 = 1)
       /\  g.rel (P_1.X s.unpair.1.unpair.2) (Q_1.X s.unpair.2.unpair.2)) :=
    ((hNtag1.and hMtag1).re).and hgr
  refine REPred.of_iff (fun s => ?_) (hA.or (hB.or hC))
  simp only [sumPresentation_X]
  constructor
  * rintro <_, _, hdisj>
    rcases hdisj with hM | <X, Y', hWX, hWY, hfXY> | <Y, Y', hWY, hWY', hgYY>
    * left
      rcases tag_trichotomy s.unpair.2.unpair.1 with hm | hm | hm
      * rw [sumEnum_zero hm] at hM; exact absurd hM inj_0_ne_sumMaster
      * rw [sumEnum_one hm] at hM; exact absurd hM inj_1_ne_sumMaster
      * exact hm
    * right; left
      have ha : s.unpair.1.unpair.1 = 0  /\  X = P_0.X s.unpair.1.unpair.2 := by
        rcases tag_trichotomy s.unpair.1.unpair.1 with hn | hn | <h0, h1>
        * rw [sumEnum_zero hn] at hWX; exact <hn, (inj_0_eq_iff.mp hWX).symm>
```

```
        * rw [sumEnum_one hn] at hWX
          exact absurd hWX.symm (inj_0_ne_inj_1_of_nonempty (h_1 _ (P_1.mem_X _)))
        * rw [sumEnum_master h0 h1] at hWX; exact absurd hWX.symm inj_0_ne_sumMaster
      have hb : s.unpair.2.unpair.1 = 0  /\  Y' = Q_0.X s.unpair.2.unpair.2 := by
        rcases tag_trichotomy s.unpair.2.unpair.1 with hm | hm | <h0, h1>
        * rw [sumEnum_zero hm] at hWY; exact <hm, (inj_0_eq_iff.mp hWY).symm>
        * rw [sumEnum_one hm] at hWY
          exact absurd hWY.symm (inj_0_ne_inj_1_of_nonempty (h_1' _ (Q_1.mem_X _)))
        * rw [sumEnum_master h0 h1] at hWY; exact absurd hWY.symm inj_0_ne_sumMaster
      obtain <ha0, hX> := ha; obtain <hb0, hY> := hb
      subst hX; subst hY
      exact <<ha0, hb0>, hfXY>
    * right; right
      have ha : s.unpair.1.unpair.1 = 1  /\  Y = P_1.X s.unpair.1.unpair.2 := by
        rcases tag_trichotomy s.unpair.1.unpair.1 with hn | hn | <h0, h1>
        * rw [sumEnum_zero hn] at hWY
          exact (inj_0_eq_inj_1_elim (h_0 _ (P_0.mem_X _)) hWY).elim
        * rw [sumEnum_one hn] at hWY; exact <hn, (inj_1_eq_iff.mp hWY).symm>
        * rw [sumEnum_master h0 h1] at hWY; exact absurd hWY.symm inj_1_ne_sumMaster
      have hb : s.unpair.2.unpair.1 = 1  /\  Y' = Q_1.X s.unpair.2.unpair.2 := by
        rcases tag_trichotomy s.unpair.2.unpair.1 with hm | hm | <h0, h1>
        * rw [sumEnum_zero hm] at hWY'
          exact (inj_0_eq_inj_1_elim (h_0' _ (Q_0.mem_X _)) hWY').elim
        * rw [sumEnum_one hm] at hWY'; exact <hm, (inj_1_eq_iff.mp hWY').symm>
        * rw [sumEnum_master h0 h1] at hWY'; exact absurd hWY'.symm inj_1_ne_sumMaster
      obtain <ha1, hYeq> := ha; obtain <hb1, hY'eq> := hb
      subst hYeq; subst hY'eq
      exact <<ha1, hb1>, hgYY>
  * rintro (<hm0, hm1> | <<hn0, hm0>, hfXY> | <<hn1, hm1>, hgYY>)
    * exact <sumEnum_mem _, sumEnum_mem _, Or.inl (sumEnum_master hm0 hm1)>
    * exact <sumEnum_mem _, sumEnum_mem _,
        Or.inr (Or.inl <_, _, sumEnum_zero hn0, sumEnum_zero hm0, hfXY>)>
    * exact <sumEnum_mem _, sumEnum_mem _,
        Or.inr (Or.inr <_, _, sumEnum_one hn1, sumEnum_one hm1, hgYY>)>

end Sum

/-- **Theorem 7.4 (Scott 1981, PRG-19) -- the paired map `<f, g>` is computable.** `Z_n
     <f, g> (X?_k  U
X1_l)  <->  Z_n f X?_k  /\  Z_n g X1_l`, the conjunction of two r.e. relations. -/
theorem paired_isComputable {V_2 : NeighborhoodSystem ?}
    {P_2 : ComputablePresentation V_2} {P_0 : ComputablePresentation V_0}
    {P_1 : ComputablePresentation V_1} {a : ApproximableMap V_2 V_0} {b :
   ApproximableMap V_2 V_1}
    (ha : ComputableMap P_2 P_0 a) (hb : ComputableMap P_2 P_1 b) :
   ComputableMap P_2 (prodPresentation P_0 P_1) (paired a b) := by
  constructor
  have ha' : REPred (fun s => a.rel (P_2.X s.unpair.1) (P_0.X s.unpair.2)) := ha.out
  have hb' : REPred (fun s => b.rel (P_2.X s.unpair.1) (P_1.X s.unpair.2)) := hb.out
  have hra : Nat.Primrec (fun t => Nat.pair t.unpair.1 t.unpair.2.unpair.1) :=
    Nat.Primrec.left.pair (Nat.Primrec.left.comp Nat.Primrec.right)
  have hrb : Nat.Primrec (fun t => Nat.pair t.unpair.1 t.unpair.2.unpair.2) :=
    Nat.Primrec.left.pair (Nat.Primrec.right.comp Nat.Primrec.right)
  refine REPred.of_iff (fun t => ?_) ((ha'.comp hra).and (hb'.comp hrb))
  simp only [prodPresentation_X, paired_rel, inl_preimage_prodNbhd,
    inr_preimage_prodNbhd,
```

```
    unpair_pair_fst, unpair_pair_snd]
  exact <fun h => <h.2.1, h.2.2>,
    fun h => <prod_mem_prodNbhd (P_0.mem_X _) (P_1.mem_X _), h.1, h.2>>

/-- **Theorem 7.4 (Scott 1981, PRG-19).** The product of effectively given domains is
    effectively
given. -/
theorem prod_isEffectivelyGiven (h_0 : V_0.IsEffectivelyGiven) (h_1 : V_1.
    IsEffectivelyGiven) :
    (prod V_0 V_1).IsEffectivelyGiven := by
  obtain <P_0> := h_0; obtain <P_1> := h_1
  exact <prodPresentation P_0 P_1>

/-- **Theorem 7.4 (Scott 1981, PRG-19) -- `proj_0` is computable.** `(X_n?  U  X?1) p_0
     X?_k  <->  X_n?  subseteq  X?_k`,
a recursive slice of `incl_computable`, hence r.e. -/
theorem proj_0_isComputable (P_0 : ComputablePresentation V_0) (P_1 :
    ComputablePresentation V_1) :
    ComputableMap (prodPresentation P_0 P_1) P_0 (proj_0 V_0 V_1) := by
  constructor
  have hincl : RecDecidable (fun s => P_0.X s.unpair.1  subseteq  P_0.X s.unpair.2) :=
     P_0.incl_computable
  have hr : Nat.Primrec (fun t => Nat.pair t.unpair.1.unpair.1 t.unpair.2) :=
    (Nat.Primrec.left.comp Nat.Primrec.left).pair Nat.Primrec.right
  refine (RecDecidable.of_iff (fun t => ?_) (hincl.comp hr)).re
  simp only [prodPresentation_X, proj_0_rel, inl_preimage_prodNbhd, unpair_pair_fst,
    unpair_pair_snd]
  exact <fun h => h.2.2,
    fun h => <prod_mem_prodNbhd (P_0.mem_X _) (P_1.mem_X _), P_0.mem_X _, h>>

/-- **Theorem 7.4 (Scott 1981, PRG-19) -- `proj_1` is computable.** `(X_n?  U  X?1) p_1
     X1_k  <->  X?1  subseteq  X1_k`
(Scott's worked example), a recursive slice of `incl_computable`. -/
theorem proj_1_isComputable (P_0 : ComputablePresentation V_0) (P_1 :
    ComputablePresentation V_1) :
    ComputableMap (prodPresentation P_0 P_1) P_1 (proj_1 V_0 V_1) := by
  constructor
  have hincl : RecDecidable (fun s => P_1.X s.unpair.1  subseteq  P_1.X s.unpair.2) :=
     P_1.incl_computable
  have hr : Nat.Primrec (fun t => Nat.pair t.unpair.1.unpair.2 t.unpair.2) :=
    (Nat.Primrec.right.comp Nat.Primrec.left).pair Nat.Primrec.right
  refine (RecDecidable.of_iff (fun t => ?_) (hincl.comp hr)).re
  simp only [prodPresentation_X, proj_1_rel, inr_preimage_prodNbhd, unpair_pair_fst,
    unpair_pair_snd]
  exact <fun h => h.2.2,
    fun h => <prod_mem_prodNbhd (P_0.mem_X _) (P_1.mem_X _), P_1.mem_X _, h>>

/-- **Theorem 7.4 (Scott 1981, PRG-19) -- the product functor `f  x  g` is computable
    .** Using
`f  x  g = <f  comp  p_0, g  comp  p_1>` (Exercise 3.19) together with computability of
     `proj_i` and `comp`
(Proposition 7.3). -/
theorem prodMap_isComputable {V_0' : NeighborhoodSystem a'} {V_1' : NeighborhoodSystem
     b'}
    {P_0 : ComputablePresentation V_0} {P_1 : ComputablePresentation V_1}
    {P_0' : ComputablePresentation V_0'} {P_1' : ComputablePresentation V_1'}
```

```
    {f : ApproximableMap V_0 V_0'} {g : ApproximableMap V_1 V_1'}
    (hf : ComputableMap P_0 P_0' f) (hg : ComputableMap P_1 P_1' g) :
    ComputableMap (prodPresentation P_0 P_1) (prodPresentation P_0' P_1') (prodMap f g
    ) := by
  rw [prodMap]
  exact paired_isComputable (comp_isComputable (proj_0_isComputable P_0 P_1) hf)
    (comp_isComputable (proj_1_isComputable P_0 P_1) hg)

end

section
set_option linter.unusedVariables false

theorem theorem_74.{u_1, u_2, u_4, u_5, u_3} : And
  (And
    (And
      (And
        (And
          (And
            (And
              (And
                (And
                  (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
    NeighbourhoodSystems.NeighborhoodSystem a}
                    {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
                    {h_0 :
                      forall  (X : Set a),
                        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a
    V_0 X  ->  @Set.Nonempty a X}
                    {h_1 :
                      forall  (Y : Set b),
                        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b
    V_1 Y  ->  @Set.Nonempty b Y},
                  @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    IsEffectivelyGiven a V_0  ->
                    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    IsEffectivelyGiven b V_1  ->
                      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    IsEffectivelyGiven (Option (Sum a b))
                        (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0
    h_1))
                (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
    NeighbourhoodSystems.NeighborhoodSystem a}
                  {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
                  {h_0 :
                    forall  (X : Set a),
                      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a
    V_0 X  ->  @Set.Nonempty a X}
                  {h_1 :
                    forall  (Y : Set b),
                      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b
    V_1 Y  ->  @Set.Nonempty b Y}
                  (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation
     a V_0)
                  (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation
```

```
 b V_1),
                  @DomainTheory.NeighbourhoodSystems.ComputableMap a (Option (Sum
a b)) V_0
                    (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
P_0
                    (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0
V_1 h_0 h_1 P_0 P_1)
                    (@DomainTheory.NeighbourhoodSystems.inMap_0 a b V_0 V_1 h_0
h_1)))
               (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
                 {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
                 {h_0 :
                   forall  (X : Set a),
                     @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a
V_0 X  ->  @Set.Nonempty a X}
                 {h_1 :
                   forall  (Y : Set b),
                     @DomainTheory.NeighbourhoodSystem.mem b
V_1 Y  ->  @Set.Nonempty b Y}
                 (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a
 V_0)
                 (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b
 V_1),
                 @DomainTheory.NeighbourhoodSystems.ComputableMap b (Option (Sum a
b)) V_1
                   (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1) P_1
                   (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0 V_1
h_0 h_1 P_0 P_1)
                   (@DomainTheory.NeighbourhoodSystems.inMap_1 a b V_0 V_1 h_0 h_1)
))
             (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
               {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
               {h_0 :
                 forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem a V_0 X  ->  @Set.Nonempty a X}
               {h_1 :
                 forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem b V_1 Y  ->  @Set.Nonempty b Y}
               (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a
V_0)
               (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b
V_1),
               @DomainTheory.NeighbourhoodSystems.ComputableMap (Option (Sum a b))
a
                 (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1) V_0
                 (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0 V_1
h_0 h_1 P_0 P_1) P_0
                 (@DomainTheory.NeighbourhoodSystems.outMap_0 a b V_0 V_1 h_0 h_1))
)
          (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
            {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
            {h_0 :
              forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.
```

```
NeighborhoodSystem.mem a V_0 X  ->  @Set.Nonempty a X}
            {h_1 :
              forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem b V_1 Y  ->  @Set.Nonempty b Y}
            (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0
)
            (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1
),
            @DomainTheory.NeighbourhoodSystems.ComputableMap (Option (Sum a b)) b
              (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1) V_1
              (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0 V_1 h_0
h_1 P_0 P_1) P_1
              (@DomainTheory.NeighbourhoodSystems.outMap_1 a b V_0 V_1 h_0 h_1)))
        (forall  {a : Type u_1} {b : Type u_2} {a' : Type u_4} {b' : Type u_5}
          {V_0 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
          {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
          {h_0 : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem a V_0 X  ->  @Set.Nonempty a X}
          {h_1 : forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem b V_1 Y  ->  @Set.Nonempty b Y}
          {P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0}
          {P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1}
          {V_0' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a'}
          {V_1' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b'}
          {h_0' :
            forall  (X : Set a'), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem a' V_0' X  ->  @Set.Nonempty a' X}
          {h_1' :
            forall  (Y : Set b'), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem b' V_1' Y  ->  @Set.Nonempty b' Y}
          (Q_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a' V_0
')
          (Q_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b' V_1
')
          {f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a' V_0 V_0'}
          {g : @DomainTheory.NeighbourhoodSystems.ApproximableMap b b' V_1 V_1'},
          @DomainTheory.NeighbourhoodSystems.ComputableMap a a' V_0 V_0' P_0 Q_0 f
  ->
            @DomainTheory.NeighbourhoodSystems.ComputableMap b b' V_1 V_1' P_1 Q_1
 g  ->
              @DomainTheory.NeighbourhoodSystems.ComputableMap (Option (Sum a b))
(Option (Sum a' b'))
                (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
                (@DomainTheory.NeighbourhoodSystems.sum a' b' V_0' V_1' h_0' h_1')
                (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0 V_1
h_0 h_1 P_0 P_1)
                (@DomainTheory.NeighbourhoodSystems.sumPresentation a' b' V_0' V_1
' h_0' h_1' Q_0 Q_1)
                (@DomainTheory.NeighbourhoodSystems.sumMap a b a' b' V_0 V_1 V_0'
V_1' h_0 h_1 h_0' h_1' f g)))
      (forall  {a : Type u_1} {b : Type u_2} {? : Type u_3} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
        {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
        {V_2 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem ?}
        {P_2 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation ? V_2}
        {P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0}
```

```
          {P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1}
          {a : @DomainTheory.NeighbourhoodSystems.ApproximableMap ? a V_2 V_0}
          {b : @DomainTheory.NeighbourhoodSystems.ApproximableMap ? b V_2 V_1},
          @DomainTheory.NeighbourhoodSystems.ComputableMap ? a V_2 V_0 P_2 P_0 a  ->
            @DomainTheory.NeighbourhoodSystems.ComputableMap ? b V_2 V_1 P_2 P_1 b
 ->
              @DomainTheory.NeighbourhoodSystems.ComputableMap ? (Sum a b) V_2
                (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) P_2
                (@DomainTheory.NeighbourhoodSystems.prodPresentation a b V_0 V_1 P_0
  P_1)
                (@DomainTheory.NeighbourhoodSystems.paired a b ? V_0 V_1 V_2 a b)))
      (forall  {a : Type u_1} {b : Type u_2} {a' : Type u_4} {b' : Type u_5}
        {V_0 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
        {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
        {V_0' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a'}
        {V_1' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b'}
        {P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0}
        {P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1}
        {P_0' : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a' V_0'}
        {P_1' : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b' V_1'}
        {f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a' V_0 V_0'}
        {g : @DomainTheory.NeighbourhoodSystems.ApproximableMap b b' V_1 V_1'},
        @DomainTheory.NeighbourhoodSystems.ComputableMap a a' V_0 V_0' P_0 P_0' f
 ->
          @DomainTheory.NeighbourhoodSystems.ComputableMap b b' V_1 V_1' P_1 P_1' g
  ->
            @DomainTheory.NeighbourhoodSystems.ComputableMap (Sum a b) (Sum a' b')
              (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1)
              (@DomainTheory.NeighbourhoodSystems.prod a' b' V_0' V_1')
              (@DomainTheory.NeighbourhoodSystems.prodPresentation a b V_0 V_1 P_0
 P_1)
              (@DomainTheory.NeighbourhoodSystems.prodPresentation a' b' V_0' V_1'
 P_0' P_1')
              (@DomainTheory.NeighbourhoodSystems.prodMap a b a' b' V_0 V_1 V_0' V_1
 ' f g)))
     (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem a}
       {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b},
       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven a V_0
    ->
         @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven b
 V_1  ->
           @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven (
 Sum a b)
             (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1)))
   (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem a}
     {V_1 : DomainTheory.NeighbourhoodSystem b}
     (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0)
     (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1),
     @DomainTheory.NeighbourhoodSystems.ComputableMap (Sum a b) a (@DomainTheory.
 NeighbourhoodSystems.prod a b V_0 V_1)
       V_0 (@DomainTheory.NeighbourhoodSystems.prodPresentation a b V_0 V_1 P_0 P_1)
 P_0
       (@DomainTheory.NeighbourhoodSystems.proj_0 a b V_0 V_1)))
(forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
```

```
  NeighborhoodSystem a}
  {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
  (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0)
  (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1),
  @DomainTheory.NeighbourhoodSystems.ComputableMap (Sum a b) b (@DomainTheory.
  NeighbourhoodSystems.prod a b V_0 V_1)
    V_1 (@DomainTheory.NeighbourhoodSystems.prodPresentation a b V_0 V_1 P_0 P_1)
  P_1
    (@DomainTheory.NeighbourhoodSystems.proj_1 a b V_0 V_1)) :=
@And.intro
(And
  (And
    (And
      (And
        (And
          (And
            (And
              (And
                (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
  NeighbourhoodSystems.NeighborhoodSystem a}
                  {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
                  {h_0 :
                    forall  (X : Set a),
                      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a
  V_0 X  ->  @Set.Nonempty a X}
                  {h_1 :
                    forall  (Y : Set b),
                      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b
  V_1 Y  ->  @Set.Nonempty b Y},
                 @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  IsEffectivelyGiven a V_0  ->
                   @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  IsEffectivelyGiven b V_1  ->
                     @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  IsEffectivelyGiven (Option (Sum a b))
                       (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0
  h_1))
                (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
  NeighbourhoodSystems.NeighborhoodSystem a}
                  {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
                  {h_0 :
                    forall  (X : Set a),
                      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a
  V_0 X  ->  @Set.Nonempty a X}
                  {h_1 :
                    forall  (Y : Set b),
                      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b
  V_1 Y  ->  @Set.Nonempty b Y}
                  (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation
   a V_0)
                  (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation
   b V_1),
                  @DomainTheory.NeighbourhoodSystems.ComputableMap a (Option (Sum
  a b)) V_0
                    (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
```

```
P_0
                    (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0
V_1 h_0 h_1 P_0 P_1)
                    (@DomainTheory.NeighbourhoodSystems.inMap_0 a b V_0 V_1 h_0
h_1)))
               (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
                 {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
                 {h_0 :
                   forall  (X : Set a),
                     @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a
V_0 X  ->  @Set.Nonempty a X}
                 {h_1 :
                   forall  (Y : Set b),
                     @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b
V_1 Y  ->  @Set.Nonempty b Y}
                 (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a
 V_0)
                 (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b
 V_1),
                 @DomainTheory.NeighbourhoodSystems.ComputableMap b (Option (Sum a
b)) V_1
                   (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1) P_1
                   (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0 V_1
h_0 h_1 P_0 P_1)
                   (@DomainTheory.NeighbourhoodSystems.inMap_1 a b V_0 V_1 h_0 h_1)
))
             (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
               {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
               {h_0 :
                 forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem a V_0 X  ->  @Set.Nonempty a X}
               {h_1 :
                 forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem b V_1 Y  ->  @Set.Nonempty b Y}
               (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a
V_0)
               (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b
V_1),
               @DomainTheory.NeighbourhoodSystems.ComputableMap (Option (Sum a b))
a
                 (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1) V_0
                 (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0 V_1
h_0 h_1 P_0 P_1) P_0
                 (@DomainTheory.NeighbourhoodSystems.outMap_0 a b V_0 V_1 h_0 h_1))
)
          (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
            {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
            {h_0 :
              forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem a V_0 X  ->  @Set.Nonempty a X}
            {h_1 :
              forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem b V_1 Y  ->  @Set.Nonempty b Y}
```

```
            (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0
)
            (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1
),
            @DomainTheory.NeighbourhoodSystems.ComputableMap (Option (Sum a b)) b
              (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1) V_1
              (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0 V_1 h_0
h_1 P_0 P_1) P_1
              (@DomainTheory.NeighbourhoodSystems.outMap_1 a b V_0 V_1 h_0 h_1)))
        (forall  {a : Type u_1} {b : Type u_2} {a' : Type u_4} {b' : Type u_5}
          {V_0 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
          {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
          {h_0 : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem a V_0 X  ->  @Set.Nonempty a X}
          {h_1 : forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem b V_1 Y  ->  @Set.Nonempty b Y}
          {P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0}
          {P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1}
          {V_0' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a'}
          {V_1' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b'}
          {h_0' :
            forall  (X : Set a'), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem a' V_0' X  ->  @Set.Nonempty a' X}
          {h_1' :
            forall  (Y : Set b'), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem b' V_1' Y  ->  @Set.Nonempty b' Y}
          (Q_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a' V_0
')
          (Q_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b' V_1
')
          {f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a' V_0 V_0'}
          {g : @DomainTheory.NeighbourhoodSystems.ApproximableMap b b' V_1 V_1'},
          @DomainTheory.NeighbourhoodSystems.ComputableMap a a' V_0 V_0' P_0 Q_0 f
  ->

            @DomainTheory.NeighbourhoodSystems.ComputableMap b b' V_1 V_1' P_1 Q_1
 g  ->
              @DomainTheory.NeighbourhoodSystems.ComputableMap (Option (Sum a b))
(Option (Sum a' b'))
                (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
                (@DomainTheory.NeighbourhoodSystems.sum a' b' V_0' V_1' h_0' h_1')
                (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0 V_1
h_0 h_1 P_0 P_1)
                (@DomainTheory.NeighbourhoodSystems.sumPresentation a' b' V_0' V_1
' h_0' h_1' Q_0 Q_1)
                (@DomainTheory.NeighbourhoodSystems.sumMap a b a' b' V_0 V_1 V_0'
V_1' h_0 h_1 h_0' h_1' f g)))
      (forall  {a : Type u_1} {b : Type u_2} {? : Type u_3} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
        {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
        {V_2 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem ?}
        {P_2 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation ? V_2}
        {P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0}
        {P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1}
        {a : @DomainTheory.NeighbourhoodSystems.ApproximableMap ? a V_2 V_0}
        {b : @DomainTheory.NeighbourhoodSystems.ApproximableMap ? b V_2 V_1},
```

```
         @DomainTheory.NeighbourhoodSystems.ComputableMap ? a V_2 V_0 P_2 P_0 a  ->
           @DomainTheory.NeighbourhoodSystems.ComputableMap ? b V_2 V_1 P_2 P_1 b
 ->
             @DomainTheory.NeighbourhoodSystems.ComputableMap ? (Sum a b) V_2
               (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) P_2
               (@DomainTheory.NeighbourhoodSystems.prodPresentation a b V_0 V_1 P_0
  P_1)
               (@DomainTheory.NeighbourhoodSystems.paired a b ? V_0 V_1 V_2 a b)))
     (forall  {a : Type u_1} {b : Type u_2} {a' : Type u_4} {b' : Type u_5}
       {V_0 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
       {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
       {V_0' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a'}
       {V_1' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b'}
       {P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0}
       {P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1}
       {P_0' : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a' V_0'}
       {P_1' : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b' V_1'}
       {f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a' V_0 V_0'}
       {g : @DomainTheory.NeighbourhoodSystems.ApproximableMap b b' V_1 V_1'},
       @DomainTheory.NeighbourhoodSystems.ComputableMap a a' V_0 V_0' P_0 P_0' f
 ->
         @DomainTheory.NeighbourhoodSystems.ComputableMap b b' V_1 V_1' P_1 P_1' g
  ->
           @DomainTheory.NeighbourhoodSystems.ComputableMap (Sum a b) (Sum a' b')
             (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1)
             (@DomainTheory.NeighbourhoodSystems.prod a' b' V_0' V_1')
             (@DomainTheory.NeighbourhoodSystems.prodPresentation a b V_0 V_1 P_0
 P_1)
             (@DomainTheory.NeighbourhoodSystems.prodPresentation a' b' V_0' V_1'
 P_0' P_1')
             (@DomainTheory.NeighbourhoodSystems.prodMap a b a' b' V_0 V_1 V_0' V_1
 ' f g)))
    (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem a}
      {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b},
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven a V_0
   ->
       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven b
 V_1  ->
         @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven (
 Sum a b)
           (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1)))
  (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem a}
    {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
    (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0)
    (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1),
    @DomainTheory.NeighbourhoodSystems.ComputableMap (Sum a b) a (@DomainTheory.
 NeighbourhoodSystems.prod a b V_0 V_1)
      V_0 (@DomainTheory.NeighbourhoodSystems.prodPresentation a b V_0 V_1 P_0 P_1)
 P_0
      (@DomainTheory.NeighbourhoodSystems.proj_0 a b V_0 V_1)))
(forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem a}
  {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
  (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0)
```

```
  (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1),
  @DomainTheory.NeighbourhoodSystems.ComputableMap (Sum a b) b (@DomainTheory.
  NeighbourhoodSystems.prod a b V_0 V_1)
    V_1 (@DomainTheory.NeighbourhoodSystems.prodPresentation a b V_0 V_1 P_0 P_1)
  P_1
    (@DomainTheory.NeighbourhoodSystems.proj_1 a b V_0 V_1))
(@And.intro
  (And
    (And
      (And
        (And
          (And
            (And
              (And
                (And
                  (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
  NeighbourhoodSystems.NeighborhoodSystem a}
                    {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
                    {h_0 :
                      forall  (X : Set a),
                        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a
  V_0 X  ->  @Set.Nonempty a X}
                    {h_1 :
                      forall  (Y : Set b),
                        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b
  V_1 Y  ->  @Set.Nonempty b Y},
                   @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  IsEffectivelyGiven a V_0  ->
                     @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  IsEffectivelyGiven b V_1  ->
                       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  IsEffectivelyGiven (Option (Sum a b))
                         (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0
  h_1))
                  (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
  NeighbourhoodSystems.NeighborhoodSystem a}
                    {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
                    {h_0 :
                      forall  (X : Set a),
                        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a
  V_0 X  ->  @Set.Nonempty a X}
                    {h_1 :
                      forall  (Y : Set b),
                        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b
  V_1 Y  ->  @Set.Nonempty b Y}
                    (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation
   a V_0)
                    (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation
   b V_1),
                   @DomainTheory.NeighbourhoodSystems.ComputableMap a (Option (Sum
  a b)) V_0
                     (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
  P_0
                     (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0
  V_1 h_0 h_1 P_0 P_1)
                     (@DomainTheory.NeighbourhoodSystems.inMap_0 a b V_0 V_1 h_0
```

```
h_1)))
               (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
                 {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
                 {h_0 :
                   forall  (X : Set a),
                     @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a
V_0 X  ->  @Set.Nonempty a X}
                 {h_1 :
                   forall  (Y : Set b),
                     @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b
V_1 Y  ->  @Set.Nonempty b Y}
                 (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a
 V_0)
                 (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b
 V_1),
                 @DomainTheory.NeighbourhoodSystems.ComputableMap b (Option (Sum a
b)) V_1
                   (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1) P_1
                   (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0 V_1
h_0 h_1 P_0 P_1)
                   (@DomainTheory.NeighbourhoodSystems.inMap_1 a b V_0 V_1 h_0 h_1)
))
             (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
               {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
               {h_0 :
                 forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem a V_0 X  ->  @Set.Nonempty a X}
               {h_1 :
                 forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem b V_1 Y  ->  @Set.Nonempty b Y}
               (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a
V_0)
               (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b
V_1),
               @DomainTheory.NeighbourhoodSystems.ComputableMap (Option (Sum a b))
a
                 (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1) V_0
                 (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0 V_1
h_0 h_1 P_0 P_1) P_0
                 (@DomainTheory.NeighbourhoodSystems.outMap_0 a b V_0 V_1 h_0 h_1))
)
          (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
            {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
            {h_0 :
              forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem a V_0 X  ->  @Set.Nonempty a X}
            {h_1 :
              forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem b V_1 Y  ->  @Set.Nonempty b Y}
            (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0
)
            (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1
),
```

```
            @DomainTheory.NeighbourhoodSystems.ComputableMap (Option (Sum a b)) b
              (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1) V_1
              (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0 V_1 h_0
h_1 P_0 P_1) P_1
              (@DomainTheory.NeighbourhoodSystems.outMap_1 a b V_0 V_1 h_0 h_1)))
         (forall  {a : Type u_1} {b : Type u_2} {a' : Type u_4} {b' : Type u_5}
           {V_0 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
           {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
           {h_0 : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem a V_0 X  ->  @Set.Nonempty a X}
           {h_1 : forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem b V_1 Y  ->  @Set.Nonempty b Y}
           {P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0}
           {P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1}
           {V_0' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a'}
           {V_1' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b'}
           {h_0' :
             forall  (X : Set a'), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem a' V_0' X  ->  @Set.Nonempty a' X}
           {h_1' :
             forall  (Y : Set b'), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem b' V_1' Y  ->  @Set.Nonempty b' Y}
           (Q_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a' V_0
')
           (Q_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b' V_1
')
           {f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a' V_0 V_0'}
           {g : @DomainTheory.NeighbourhoodSystems.ApproximableMap b b' V_1 V_1'},
           @DomainTheory.NeighbourhoodSystems.ComputableMap a a' V_0 V_0' P_0 Q_0 f
  ->
             @DomainTheory.NeighbourhoodSystems.ComputableMap b b' V_1 V_1' P_1 Q_1
 g  ->
               @DomainTheory.NeighbourhoodSystems.ComputableMap (Option (Sum a b))
(Option (Sum a' b'))
                 (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
                 (@DomainTheory.NeighbourhoodSystems.sum a' b' V_0' V_1' h_0' h_1')
                 (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0 V_1
h_0 h_1 P_0 P_1)
                 (@DomainTheory.NeighbourhoodSystems.sumPresentation a' b' V_0' V_1
' h_0' h_1' Q_0 Q_1)
                 (@DomainTheory.NeighbourhoodSystems.sumMap a b a' b' V_0 V_1 V_0'
V_1' h_0 h_1 h_0' h_1' f g)))
       (forall  {a : Type u_1} {b : Type u_2} {? : Type u_3} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
         {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
         {V_2 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem ?}
         {P_2 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation ? V_2}
         {P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0}
         {P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1}
         {a : @DomainTheory.NeighbourhoodSystems.ApproximableMap ? a V_2 V_0}
         {b : @DomainTheory.NeighbourhoodSystems.ApproximableMap ? b V_2 V_1},
         @DomainTheory.NeighbourhoodSystems.ComputableMap ? a V_2 V_0 P_2 P_0 a  ->
           @DomainTheory.NeighbourhoodSystems.ComputableMap ? b V_2 V_1 P_2 P_1 b
->
             @DomainTheory.NeighbourhoodSystems.ComputableMap ? (Sum a b) V_2
               (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) P_2
```

```
              (@DomainTheory.NeighbourhoodSystems.prodPresentation a b V_0 V_1 P_0
 P_1)
              (@DomainTheory.NeighbourhoodSystems.paired a b ? V_0 V_1 V_2 a b)))
    (forall  {a : Type u_1} {b : Type u_2} {a' : Type u_4} {b' : Type u_5}
      {V_0 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
      {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
      {V_0' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a'}
      {V_1' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b'}
      {P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0}
      {P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1}
      {P_0' : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a' V_0'}
      {P_1' : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b' V_1'}
      {f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a' V_0 V_0'}
      {g : @DomainTheory.NeighbourhoodSystems.ApproximableMap b b' V_1 V_1'},
      @DomainTheory.NeighbourhoodSystems.ComputableMap a a' V_0 V_0' P_0 P_0' f
->
        @DomainTheory.NeighbourhoodSystems.ComputableMap b b' V_1 V_1' P_1 P_1' g
 ->
          @DomainTheory.NeighbourhoodSystems.ComputableMap (Sum a b) (Sum a' b')
            (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1)
            (@DomainTheory.NeighbourhoodSystems.prod a' b' V_0' V_1')
            (@DomainTheory.NeighbourhoodSystems.prodPresentation a b V_0 V_1 P_0
P_1)
            (@DomainTheory.NeighbourhoodSystems.prodPresentation a' b' V_0' V_1'
P_0' P_1')
            (@DomainTheory.NeighbourhoodSystems.prodMap a b a' b' V_0 V_1 V_0' V_1
' f g)))
  (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
    {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b},
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven a V_0
  ->
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven b
V_1  ->
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven (
Sum a b)
          (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1)))
(forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
  {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
  (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0)
  (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1),
  @DomainTheory.NeighbourhoodSystems.ComputableMap (Sum a b) a (@DomainTheory.
NeighbourhoodSystems.prod a b V_0 V_1)
    V_0 (@DomainTheory.NeighbourhoodSystems.prodPresentation a b V_0 V_1 P_0 P_1)
P_0
    (@DomainTheory.NeighbourhoodSystems.proj_0 a b V_0 V_1))
(@And.intro
  (And
    (And
      (And
        (And
          (And
            (And
              (And
                (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
```

```
NeighbourhoodSystems.NeighborhoodSystem a}
                  {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
                  {h_0 :
                    forall  (X : Set a),
                      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a
V_0 X  ->  @Set.Nonempty a X}
                  {h_1 :
                    forall  (Y : Set b),
                      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b
V_1 Y  ->  @Set.Nonempty b Y},
                  @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
IsEffectivelyGiven a V_0  ->
                    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
IsEffectivelyGiven b V_1  ->
                      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
IsEffectivelyGiven (Option (Sum a b))
                        (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0
h_1))
                (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
                  {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
                  {h_0 :
                    forall  (X : Set a),
                      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a
V_0 X  ->  @Set.Nonempty a X}
                  {h_1 :
                    forall  (Y : Set b),
                      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b
V_1 Y  ->  @Set.Nonempty b Y}
                  (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation
 a V_0)
                  (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation
 b V_1),
                  @DomainTheory.NeighbourhoodSystems.ComputableMap a (Option (Sum
a b)) V_0
                    (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
P_0
                    (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0
V_1 h_0 h_1 P_0 P_1)
                    (@DomainTheory.NeighbourhoodSystems.inMap_0 a b V_0 V_1 h_0
h_1)))
               (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
                 {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
                 {h_0 :
                   forall  (X : Set a),
                     @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a
V_0 X  ->  @Set.Nonempty a X}
                 {h_1 :
                   forall  (Y : Set b),
                     @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b
V_1 Y  ->  @Set.Nonempty b Y}
                 (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a
 V_0)
                 (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b
 V_1),
```

```
                @DomainTheory.NeighbourhoodSystems.ComputableMap b (Option (Sum a
b)) V_1
                  (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1) P_1
                  (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0 V_1
h_0 h_1 P_0 P_1)
                  (@DomainTheory.NeighbourhoodSystems.inMap_1 a b V_0 V_1 h_0 h_1)
))
            (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
              {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
              {h_0 :
                forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem a V_0 X  ->  @Set.Nonempty a X}
              {h_1 :
                forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem b V_1 Y  ->  @Set.Nonempty b Y}
              (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a
V_0)
              (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b
V_1),
              @DomainTheory.NeighbourhoodSystems.ComputableMap (Option (Sum a b))
a
                (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1) V_0
                (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0 V_1
h_0 h_1 P_0 P_1) P_0
                (@DomainTheory.NeighbourhoodSystems.outMap_0 a b V_0 V_1 h_0 h_1))
)
          (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
            {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
            {h_0 :
              forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem a V_0 X  ->  @Set.Nonempty a X}
            {h_1 :
              forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem b V_1 Y  ->  @Set.Nonempty b Y}
            (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0
)
            (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1
),
            @DomainTheory.NeighbourhoodSystems.ComputableMap (Option (Sum a b)) b
              (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1) V_1
              (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0 V_1 h_0
h_1 P_0 P_1) P_1
              (@DomainTheory.NeighbourhoodSystems.outMap_1 a b V_0 V_1 h_0 h_1)))
        (forall  {a : Type u_1} {b : Type u_2} {a' : Type u_4} {b' : Type u_5}
          {V_0 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
          {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
          {h_0 : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem a V_0 X  ->  @Set.Nonempty a X}
          {h_1 : forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem b V_1 Y  ->  @Set.Nonempty b Y}
          {P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0}
          {P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1}
          {V_0' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a'}
          {V_1' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b'}
```

```
          {h_0' :
            forall  (X : Set a'), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem a' V_0' X  ->  @Set.Nonempty a' X}
          {h_1' :
            forall  (Y : Set b'), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem b' V_1' Y  ->  @Set.Nonempty b' Y}
          (Q_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a' V_0
')
          (Q_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b' V_1
')
          {f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a' V_0 V_0'}
          {g : @DomainTheory.NeighbourhoodSystems.ApproximableMap b b' V_1 V_1'},
          @DomainTheory.NeighbourhoodSystems.ComputableMap a a' V_0 V_0' P_0 Q_0 f
  ->
            @DomainTheory.NeighbourhoodSystems.ComputableMap b b' V_1 V_1' P_1 Q_1
 g  ->
              @DomainTheory.NeighbourhoodSystems.ComputableMap (Option (Sum a b))
(Option (Sum a' b'))
                (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
                (@DomainTheory.NeighbourhoodSystems.sum a' b' V_0' V_1' h_0' h_1')
                (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0 V_1
h_0 h_1 P_0 P_1)
                (@DomainTheory.NeighbourhoodSystems.sumPresentation a' b' V_0' V_1
' h_0' h_1' Q_0 Q_1)
                (@DomainTheory.NeighbourhoodSystems.sumMap a b a' b' V_0 V_1 V_0'
V_1' h_0 h_1 h_0' h_1' f g)))
      (forall  {a : Type u_1} {b : Type u_2} {? : Type u_3} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
        {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
        {V_2 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem ?}
        {P_2 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation ? V_2}
        {P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0}
        {P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1}
        {a : @DomainTheory.NeighbourhoodSystems.ApproximableMap ? a V_2 V_0}
        {b : @DomainTheory.NeighbourhoodSystems.ApproximableMap ? b V_2 V_1},
        @DomainTheory.NeighbourhoodSystems.ComputableMap ? a V_2 V_0 P_2 P_0 a  ->
          @DomainTheory.NeighbourhoodSystems.ComputableMap ? b V_2 V_1 P_2 P_1 b
->
            @DomainTheory.NeighbourhoodSystems.ComputableMap ? (Sum a b) V_2
              (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) P_2
              (@DomainTheory.NeighbourhoodSystems.prodPresentation a b V_0 V_1 P_0
 P_1)
              (@DomainTheory.NeighbourhoodSystems.paired a b ? V_0 V_1 V_2 a b)))
    (forall  {a : Type u_1} {b : Type u_2} {a' : Type u_4} {b' : Type u_5}
      {V_0 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
      {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
      {V_0' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a'}
      {V_1' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b'}
      {P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0}
      {P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1}
      {P_0' : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a' V_0'}
      {P_1' : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b' V_1'}
      {f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a' V_0 V_0'}
      {g : @DomainTheory.NeighbourhoodSystems.ApproximableMap b b' V_1 V_1'},
      @DomainTheory.NeighbourhoodSystems.ComputableMap a a' V_0 V_0' P_0 P_0' f
->
```

```
        @DomainTheory.NeighbourhoodSystems.ComputableMap b b' V_1 V_1' P_1 P_1' g
 ->
          @DomainTheory.NeighbourhoodSystems.ComputableMap (Sum a b) (Sum a' b')
            (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1)
            (@DomainTheory.NeighbourhoodSystems.prod a' b' V_0' V_1')
            (@DomainTheory.NeighbourhoodSystems.prodPresentation a b V_0 V_1 P_0
P_1)
            (@DomainTheory.NeighbourhoodSystems.prodPresentation a' b' V_0' V_1'
P_0' P_1')
            (@DomainTheory.NeighbourhoodSystems.prodMap a b a' b' V_0 V_1 V_0' V_1
' f g)))
   (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
     {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b},
     @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven a V_0
  ->
       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven b
V_1  ->
         @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven (
Sum a b)
           (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1))
   (@And.intro
     (And
       (And
         (And
           (And
             (And
               (And
                 (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
                   {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
                   {h_0 :
                     forall  (X : Set a),
                       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a
V_0 X  ->  @Set.Nonempty a X}
                   {h_1 :
                     forall  (Y : Set b),
                       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b
V_1 Y  ->  @Set.Nonempty b Y},
                   @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
IsEffectivelyGiven a V_0  ->
                     @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
IsEffectivelyGiven b V_1  ->
                       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
IsEffectivelyGiven (Option (Sum a b))
                         (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0
h_1))
                 (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
                   {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
                   {h_0 :
                     forall  (X : Set a),
                       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a
V_0 X  ->  @Set.Nonempty a X}
                   {h_1 :
                     forall  (Y : Set b),
```

```
                    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b
V_1 Y  ->  @Set.Nonempty b Y}
                  (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation
 a V_0)
                  (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation
 b V_1),
                  @DomainTheory.NeighbourhoodSystems.ComputableMap a (Option (Sum
a b)) V_0
                    (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
P_0
                    (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0
V_1 h_0 h_1 P_0 P_1)
                    (@DomainTheory.NeighbourhoodSystems.inMap_0 a b V_0 V_1 h_0
h_1)))
               (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
                 {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
                 {h_0 :
                   forall  (X : Set a),
                     @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a
V_0 X  ->  @Set.Nonempty a X}
                 {h_1 :
                   forall  (Y : Set b),
                     @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b
V_1 Y  ->  @Set.Nonempty b Y}
                 (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a
 V_0)
                 (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b
 V_1),
                 @DomainTheory.NeighbourhoodSystems.ComputableMap b (Option (Sum a
b)) V_1
                   (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1) P_1
                   (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0 V_1
h_0 h_1 P_0 P_1)
                   (@DomainTheory.NeighbourhoodSystems.inMap_1 a b V_0 V_1 h_0 h_1)
))
              (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
                {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
                {h_0 :
                  forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem a V_0 X  ->  @Set.Nonempty a X}
                {h_1 :
                  forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem b V_1 Y  ->  @Set.Nonempty b Y}
                (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a
V_0)
                (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b
V_1),
                @DomainTheory.NeighbourhoodSystems.ComputableMap (Option (Sum a b))
a
                  (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1) V_0
                  (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0 V_1
h_0 h_1 P_0 P_1) P_0
                  (@DomainTheory.NeighbourhoodSystems.outMap_0 a b V_0 V_1 h_0 h_1))
)
```

```
          (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}

            {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
            {h_0 :
              forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem a V_0 X  ->  @Set.Nonempty a X}
            {h_1 :
              forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem b V_1 Y  ->  @Set.Nonempty b Y}
            (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0
)
            (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1
),
            @DomainTheory.NeighbourhoodSystems.ComputableMap (Option (Sum a b)) b
              (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1) V_1
              (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0 V_1 h_0
h_1 P_0 P_1) P_1
              (@DomainTheory.NeighbourhoodSystems.outMap_1 a b V_0 V_1 h_0 h_1)))
        (forall  {a : Type u_1} {b : Type u_2} {a' : Type u_4} {b' : Type u_5}
          {V_0 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
          {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
          {h_0 : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem a V_0 X  ->  @Set.Nonempty a X}
          {h_1 : forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem b V_1 Y  ->  @Set.Nonempty b Y}
          {P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0}
          {P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1}
          {V_0' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a'}
          {V_1' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b'}
          {h_0' :
            forall  (X : Set a'), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem a' V_0' X  ->  @Set.Nonempty a' X}
          {h_1' :
            forall  (Y : Set b'), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem b' V_1' Y  ->  @Set.Nonempty b' Y}
          (Q_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a' V_0
')
          (Q_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b' V_1
')
          {f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a' V_0 V_0'}
          {g : @DomainTheory.NeighbourhoodSystems.ApproximableMap b b' V_1 V_1'},
          @DomainTheory.NeighbourhoodSystems.ComputableMap a a' V_0 V_0' P_0 Q_0 f
  ->
            @DomainTheory.NeighbourhoodSystems.ComputableMap b b' V_1 V_1' P_1 Q_1
 g  ->
              @DomainTheory.NeighbourhoodSystems.ComputableMap (Option (Sum a b))
(Option (Sum a' b'))
                (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
                (@DomainTheory.NeighbourhoodSystems.sum a' b' V_0' V_1' h_0' h_1')
                (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0 V_1
h_0 h_1 P_0 P_1)
                (@DomainTheory.NeighbourhoodSystems.sumPresentation a' b' V_0' V_1
' h_0' h_1' Q_0 Q_1)
                (@DomainTheory.NeighbourhoodSystems.sumMap a b a' b' V_0 V_1 V_0'
V_1' h_0 h_1 h_0' h_1' f g)))
```

```
      (forall  {a : Type u_1} {b : Type u_2} {? : Type u_3} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
        {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
        {V_2 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem ?}
        {P_2 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation ? V_2}
        {P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0}
        {P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1}
        {a : @DomainTheory.NeighbourhoodSystems.ApproximableMap ? a V_2 V_0}
        {b : @DomainTheory.NeighbourhoodSystems.ApproximableMap ? b V_2 V_1},
        @DomainTheory.NeighbourhoodSystems.ComputableMap ? a V_2 V_0 P_2 P_0 a  ->
          @DomainTheory.NeighbourhoodSystems.ComputableMap ? b V_2 V_1 P_2 P_1 b
->
            @DomainTheory.NeighbourhoodSystems.ComputableMap ? (Sum a b) V_2
              (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) P_2
              (@DomainTheory.NeighbourhoodSystems.prodPresentation a b V_0 V_1 P_0
 P_1)
              (@DomainTheory.NeighbourhoodSystems.paired a b ? V_0 V_1 V_2 a b)))
    (forall  {a : Type u_1} {b : Type u_2} {a' : Type u_4} {b' : Type u_5}
      {V_0 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
      {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
      {V_0' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a'}
      {V_1' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b'}
      {P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0}
      {P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1}
      {P_0' : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a' V_0'}
      {P_1' : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b' V_1'}
      {f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a' V_0 V_0'}
      {g : @DomainTheory.NeighbourhoodSystems.ApproximableMap b b' V_1 V_1'},
      @DomainTheory.NeighbourhoodSystems.ComputableMap a a' V_0 V_0' P_0 P_0' f
->
        @DomainTheory.NeighbourhoodSystems.ComputableMap b b' V_1 V_1' P_1 P_1' g
 ->
          @DomainTheory.NeighbourhoodSystems.ComputableMap (Sum a b) (Sum a' b')
            (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1)
            (@DomainTheory.NeighbourhoodSystems.prod a' b' V_0' V_1')
            (@DomainTheory.NeighbourhoodSystems.prodPresentation a b V_0 V_1 P_0
P_1)
            (@DomainTheory.NeighbourhoodSystems.prodPresentation a' b' V_0' V_1'
P_0' P_1')
            (@DomainTheory.NeighbourhoodSystems.prodMap a b a' b' V_0 V_1 V_0' V_1
' f g))
    (@And.intro
      (And
        (And
          (And
            (And
              (And
                (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
                  {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
                  {h_0 :
                    forall  (X : Set a),
                      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a
V_0 X  ->  @Set.Nonempty a X}
                  {h_1 :
                    forall  (Y : Set b),
```

```
                    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b
V_1 Y  ->  @Set.Nonempty b Y},
                  @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
IsEffectivelyGiven a V_0  ->
                    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
IsEffectivelyGiven b V_1  ->
                    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
IsEffectivelyGiven (Option (Sum a b))
                      (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0
h_1))
                (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
                  {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
                  {h_0 :
                    forall  (X : Set a),
                      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a
V_0 X  ->  @Set.Nonempty a X}
                  {h_1 :
                    forall  (Y : Set b),
                      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b
V_1 Y  ->  @Set.Nonempty b Y}
                  (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation
 a V_0)
                  (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation
 b V_1),
                  @DomainTheory.NeighbourhoodSystems.ComputableMap a (Option (Sum
a b)) V_0
                    (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
P_0
                    (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0
V_1 h_0 h_1 P_0 P_1)
                    (@DomainTheory.NeighbourhoodSystems.inMap_0 a b V_0 V_1 h_0
h_1)))
              (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
                {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
                {h_0 :
                  forall  (X : Set a),
                    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a
V_0 X  ->  @Set.Nonempty a X}
                {h_1 :
                  forall  (Y : Set b),
                    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b
V_1 Y  ->  @Set.Nonempty b Y}
                (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a
 V_0)
                (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b
 V_1),
                @DomainTheory.NeighbourhoodSystems.ComputableMap b (Option (Sum a
b)) V_1
                  (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1) P_1
                  (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0 V_1
h_0 h_1 P_0 P_1)
                  (@DomainTheory.NeighbourhoodSystems.inMap_1 a b V_0 V_1 h_0 h_1)
))
            (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
```

```
NeighbourhoodSystems.NeighborhoodSystem a}
              {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
              {h_0 :
                forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem a V_0 X  ->  @Set.Nonempty a X}
              {h_1 :
                forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem b V_1 Y  ->  @Set.Nonempty b Y}
              (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a
V_0)
              (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b
V_1),
              @DomainTheory.NeighbourhoodSystems.ComputableMap (Option (Sum a b))
a
                (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1) V_0
                (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0 V_1
h_0 h_1 P_0 P_1) P_0
                (@DomainTheory.NeighbourhoodSystems.outMap_0 a b V_0 V_1 h_0 h_1))
)
          (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
            {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
            {h_0 :
              forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem a V_0 X  ->  @Set.Nonempty a X}
            {h_1 :
              forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem b V_1 Y  ->  @Set.Nonempty b Y}
            (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0
)
            (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1
),
            @DomainTheory.NeighbourhoodSystems.ComputableMap (Option (Sum a b)) b
              (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1) V_1
              (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0 V_1 h_0
h_1 P_0 P_1) P_1
              (@DomainTheory.NeighbourhoodSystems.outMap_1 a b V_0 V_1 h_0 h_1)))
        (forall  {a : Type u_1} {b : Type u_2} {a' : Type u_4} {b' : Type u_5}
          {V_0 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
          {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
          {h_0 : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem a V_0 X  ->  @Set.Nonempty a X}
          {h_1 : forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem b V_1 Y  ->  @Set.Nonempty b Y}
          {P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0}
          {P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1}
          {V_0' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a'}
          {V_1' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b'}
          {h_0' :
            forall  (X : Set a'), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem a' V_0' X  ->  @Set.Nonempty a' X}
          {h_1' :
            forall  (Y : Set b'), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem b' V_1' Y  ->  @Set.Nonempty b' Y}
          (Q_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a' V_0
')
```

```
          (Q_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b' V_1
')
          {f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a' V_0 V_0'}
          {g : @DomainTheory.NeighbourhoodSystems.ApproximableMap b b' V_1 V_1'},
          @DomainTheory.NeighbourhoodSystems.ComputableMap a a' V_0 V_0' P_0 Q_0 f
  ->
            @DomainTheory.NeighbourhoodSystems.ComputableMap b b' V_1 V_1' P_1 Q_1
 g  ->
              @DomainTheory.NeighbourhoodSystems.ComputableMap (Option (Sum a b))
(Option (Sum a' b'))
                (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
                (@DomainTheory.NeighbourhoodSystems.sum a' b' V_0' V_1' h_0' h_1')
                (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0 V_1
h_0 h_1 P_0 P_1)
                (@DomainTheory.NeighbourhoodSystems.sumPresentation a' b' V_0' V_1
' h_0' h_1' Q_0 Q_1)
                (@DomainTheory.NeighbourhoodSystems.sumMap a b a' b' V_0 V_1 V_0'
V_1' h_0 h_1 h_0' h_1' f g)))
      (forall  {a : Type u_1} {b : Type u_2} {? : Type u_3} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
        {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
        {V_2 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem ?}
        {P_2 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation ? V_2}
        {P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0}
        {P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1}
        {a : @DomainTheory.NeighbourhoodSystems.ApproximableMap ? a V_2 V_0}
        {b : @DomainTheory.NeighbourhoodSystems.ApproximableMap ? b V_2 V_1},
        @DomainTheory.NeighbourhoodSystems.ComputableMap ? a V_2 V_0 P_2 P_0 a  ->
          @DomainTheory.NeighbourhoodSystems.ComputableMap ? b V_2 V_1 P_2 P_1 b
->
            @DomainTheory.NeighbourhoodSystems.ComputableMap ? (Sum a b) V_2
              (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) P_2
              (@DomainTheory.NeighbourhoodSystems.prodPresentation a b V_0 V_1 P_0
 P_1)
              (@DomainTheory.NeighbourhoodSystems.paired a b ? V_0 V_1 V_2 a b))
      (@And.intro
        (And
          (And
            (And
              (And
                (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
                  {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
                  {h_0 :
                    forall  (X : Set a),
                      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a
V_0 X  ->  @Set.Nonempty a X}
                  {h_1 :
                    forall  (Y : Set b),
                      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b
V_1 Y  ->  @Set.Nonempty b Y},
                  @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
IsEffectivelyGiven a V_0  ->
                    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
IsEffectivelyGiven b V_1  ->
                      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
```

```
IsEffectivelyGiven (Option (Sum a b))
                        (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0
h_1))
                (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
                 {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
                 {h_0 :
                   forall  (X : Set a),
                     @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a
V_0 X  ->  @Set.Nonempty a X}
                 {h_1 :
                   forall  (Y : Set b),
                     @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b
V_1 Y  ->  @Set.Nonempty b Y}
                 (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation
 a V_0)
                 (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation
 b V_1),
                 @DomainTheory.NeighbourhoodSystems.ComputableMap a (Option (Sum
a b)) V_0
                   (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
P_0
                   (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0
V_1 h_0 h_1 P_0 P_1)
                   (@DomainTheory.NeighbourhoodSystems.inMap_0 a b V_0 V_1 h_0
h_1)))
              (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
                {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
                {h_0 :
                  forall  (X : Set a),
                    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a
V_0 X  ->  @Set.Nonempty a X}
                {h_1 :
                  forall  (Y : Set b),
                    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b
V_1 Y  ->  @Set.Nonempty b Y}
                (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a
 V_0)
                (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b
 V_1),
                @DomainTheory.NeighbourhoodSystems.ComputableMap b (Option (Sum a
b)) V_1
                  (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1) P_1
                  (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0 V_1
h_0 h_1 P_0 P_1)
                  (@DomainTheory.NeighbourhoodSystems.inMap_1 a b V_0 V_1 h_0 h_1)
))
            (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
              {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
              {h_0 :
                forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem a V_0 X  ->  @Set.Nonempty a X}
              {h_1 :
                forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.
```

```
NeighborhoodSystem.mem b V_1 Y  ->  @Set.Nonempty b Y}
              (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a
V_0)
              (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b
V_1),
              @DomainTheory.NeighbourhoodSystems.ComputableMap (Option (Sum a b))
a
                (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1) V_0
                (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0 V_1
h_0 h_1 P_0 P_1) P_0
                (@DomainTheory.NeighbourhoodSystems.outMap_0 a b V_0 V_1 h_0 h_1))
)
          (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
            {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
            {h_0 :
              forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem a V_0 X  ->  @Set.Nonempty a X}
            {h_1 :
              forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem b V_1 Y  ->  @Set.Nonempty b Y}
            (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0
)
            (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1
),
            @DomainTheory.NeighbourhoodSystems.ComputableMap (Option (Sum a b)) b
              (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1) V_1
              (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0 V_1 h_0
h_1 P_0 P_1) P_1
              (@DomainTheory.NeighbourhoodSystems.outMap_1 a b V_0 V_1 h_0 h_1)))
        (forall  {a : Type u_1} {b : Type u_2} {a' : Type u_4} {b' : Type u_5}
          {V_0 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
          {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
          {h_0 : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem a V_0 X  ->  @Set.Nonempty a X}
          {h_1 : forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem b V_1 Y  ->  @Set.Nonempty b Y}
          {P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0}
          {P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1}
          {V_0' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a'}
          {V_1' : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b'}
          {h_0' :
            forall  (X : Set a'), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem a' V_0' X  ->  @Set.Nonempty a' X}
          {h_1' :
            forall  (Y : Set b'), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem b' V_1' Y  ->  @Set.Nonempty b' Y}
          (Q_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a' V_0
')
          (Q_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b' V_1
')
          {f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a' V_0 V_0'}
          {g : @DomainTheory.NeighbourhoodSystems.ApproximableMap b b' V_1 V_1'},
          @DomainTheory.NeighbourhoodSystems.ComputableMap a a' V_0 V_0' P_0 Q_0 f
  ->
            @DomainTheory.NeighbourhoodSystems.ComputableMap b b' V_1 V_1' P_1 Q_1
```

```
 g  ->
              @DomainTheory.NeighbourhoodSystems.ComputableMap (Option (Sum a b))
(Option (Sum a' b'))
                (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
                (@DomainTheory.NeighbourhoodSystems.sum a' b' V_0' V_1' h_0' h_1')
                (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0 V_1
h_0 h_1 P_0 P_1)
                (@DomainTheory.NeighbourhoodSystems.sumPresentation a' b' V_0' V_1
' h_0' h_1' Q_0 Q_1)
                (@DomainTheory.NeighbourhoodSystems.sumMap a b a' b' V_0 V_1 V_0'
V_1' h_0 h_1 h_0' h_1' f g))
         (@And.intro
           (And
             (And
               (And
                 (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
                   {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
                   {h_0 :
                     forall  (X : Set a),
                       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a
V_0 X  ->  @Set.Nonempty a X}
                   {h_1 :
                     forall  (Y : Set b),
                       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b
V_1 Y  ->  @Set.Nonempty b Y},
                   @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
IsEffectivelyGiven a V_0  ->
                     @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
IsEffectivelyGiven b V_1  ->
                       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
IsEffectivelyGiven (Option (Sum a b))
                         (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0
h_1))
                 (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
                   {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
                   {h_0 :
                     forall  (X : Set a),
                       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a
V_0 X  ->  @Set.Nonempty a X}
                   {h_1 :
                     forall  (Y : Set b),
                       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b
V_1 Y  ->  @Set.Nonempty b Y}
                   (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation
 a V_0)
                   (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation
 b V_1),
                   @DomainTheory.NeighbourhoodSystems.ComputableMap a (Option (Sum
a b)) V_0
                     (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
P_0
                     (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0
V_1 h_0 h_1 P_0 P_1)
                     (@DomainTheory.NeighbourhoodSystems.inMap_0 a b V_0 V_1 h_0
```

```
h_1)))
               (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
                 {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
                 {h_0 :
                   forall  (X : Set a),
                     @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a
V_0 X  ->  @Set.Nonempty a X}
                 {h_1 :
                   forall  (Y : Set b),
                     @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b
V_1 Y  ->  @Set.Nonempty b Y}
                 (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a
 V_0)
                 (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b
 V_1),
                 @DomainTheory.NeighbourhoodSystems.ComputableMap b (Option (Sum a
b)) V_1
                   (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1) P_1
                   (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0 V_1
h_0 h_1 P_0 P_1)
                   (@DomainTheory.NeighbourhoodSystems.inMap_1 a b V_0 V_1 h_0 h_1)
))
             (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
               {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
               {h_0 :
                 forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem a V_0 X  ->  @Set.Nonempty a X}
               {h_1 :
                 forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem b V_1 Y  ->  @Set.Nonempty b Y}
               (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a
V_0)
               (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b
V_1),
               @DomainTheory.NeighbourhoodSystems.ComputableMap (Option (Sum a b))
a
                 (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1) V_0
                 (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0 V_1
h_0 h_1 P_0 P_1) P_0
                 (@DomainTheory.NeighbourhoodSystems.outMap_0 a b V_0 V_1 h_0 h_1))
)
          (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
            {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
            {h_0 :
              forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem a V_0 X  ->  @Set.Nonempty a X}
            {h_1 :
              forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem b V_1 Y  ->  @Set.Nonempty b Y}
            (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0
)
            (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1
),
```

```
            @DomainTheory.NeighbourhoodSystems.ComputableMap (Option (Sum a b)) b
              (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1) V_1
              (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0 V_1 h_0
h_1 P_0 P_1) P_1
              (@DomainTheory.NeighbourhoodSystems.outMap_1 a b V_0 V_1 h_0 h_1))
           (@And.intro
             (And
               (And
                 (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
                   {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
                   {h_0 :
                     forall  (X : Set a),
                       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a
V_0 X  ->  @Set.Nonempty a X}
                   {h_1 :
                     forall  (Y : Set b),
                       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b
V_1 Y  ->  @Set.Nonempty b Y},
                  @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
IsEffectivelyGiven a V_0  ->
                    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
IsEffectivelyGiven b V_1  ->
                      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
IsEffectivelyGiven (Option (Sum a b))
                        (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0
h_1))
                (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
                   {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
                   {h_0 :
                     forall  (X : Set a),
                       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a
V_0 X  ->  @Set.Nonempty a X}
                   {h_1 :
                     forall  (Y : Set b),
                       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b
V_1 Y  ->  @Set.Nonempty b Y}
                   (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation
 a V_0)
                   (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation
 b V_1),
                  @DomainTheory.NeighbourhoodSystems.ComputableMap a (Option (Sum
a b)) V_0
                    (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
P_0
                    (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0
V_1 h_0 h_1 P_0 P_1)
                    (@DomainTheory.NeighbourhoodSystems.inMap_0 a b V_0 V_1 h_0
h_1)))
               (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
                 {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
                 {h_0 :
                   forall  (X : Set a),
                     @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a
```

```
V_0 X  ->  @Set.Nonempty a X}
                {h_1 :
                  forall  (Y : Set b),
                    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b
V_1 Y  ->  @Set.Nonempty b Y}
                (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a
 V_0)
                (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b
 V_1),
                @DomainTheory.NeighbourhoodSystems.ComputableMap b (Option (Sum a
b)) V_1
                  (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1) P_1
                  (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0 V_1
h_0 h_1 P_0 P_1)
                  (@DomainTheory.NeighbourhoodSystems.inMap_1 a b V_0 V_1 h_0 h_1)
))
             (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
               {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
               {h_0 :
                 forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem a V_0 X  ->  @Set.Nonempty a X}
               {h_1 :
                 forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.mem b V_1 Y  ->  @Set.Nonempty b Y}
               (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a
V_0)
               (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b
V_1),
               @DomainTheory.NeighbourhoodSystems.ComputableMap (Option (Sum a b))
a
                 (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1) V_0
                 (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0 V_1
h_0 h_1 P_0 P_1) P_0
                 (@DomainTheory.NeighbourhoodSystems.outMap_0 a b V_0 V_1 h_0 h_1))
             (@And.intro
               (And
                 (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
                   {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
                   {h_0 :

                     forall  (X : Set a),
                       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a
V_0 X  ->  @Set.Nonempty a X}
                   {h_1 :
                     forall  (Y : Set b),
                       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b
V_1 Y  ->  @Set.Nonempty b Y},
                  @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
IsEffectivelyGiven a V_0  ->
                    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
IsEffectivelyGiven b V_1  ->
                      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
IsEffectivelyGiven (Option (Sum a b))
                        (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0
```

```
h_1))
                  (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
                   {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
                   {h_0 :
                     forall  (X : Set a),
                       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a
V_0 X  ->  @Set.Nonempty a X}
                   {h_1 :
                     forall  (Y : Set b),
                       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b
V_1 Y  ->  @Set.Nonempty b Y}
                   (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation
 a V_0)
                   (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation
 b V_1),
                  @DomainTheory.NeighbourhoodSystems.ComputableMap a (Option (Sum
a b)) V_0
                    (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
P_0
                    (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0
V_1 h_0 h_1 P_0 P_1)
                    (@DomainTheory.NeighbourhoodSystems.inMap_0 a b V_0 V_1 h_0
h_1)))
                (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
                 {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
                 {h_0 :
                   forall  (X : Set a),
                     @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a
V_0 X  ->  @Set.Nonempty a X}
                 {h_1 :
                   forall  (Y : Set b),
                     @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b
V_1 Y  ->  @Set.Nonempty b Y}
                 (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a
 V_0)
                 (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b
 V_1),
                @DomainTheory.NeighbourhoodSystems.ComputableMap b (Option (Sum a
b)) V_1
                  (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1) P_1
                  (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0 V_1
h_0 h_1 P_0 P_1)
                  (@DomainTheory.NeighbourhoodSystems.inMap_1 a b V_0 V_1 h_0 h_1)
)
                (@And.intro
                  (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
                   {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
                   {h_0 :
                     forall  (X : Set a),
                       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a
V_0 X  ->  @Set.Nonempty a X}
                   {h_1 :
                     forall  (Y : Set b),
```

```
                        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b
    V_1 Y  ->  @Set.Nonempty b Y},
                     @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    IsEffectivelyGiven a V_0  ->
                       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    IsEffectivelyGiven b V_1  ->
                        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    IsEffectivelyGiven (Option (Sum a b))
                          (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0
    h_1))
                   (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
    NeighbourhoodSystems.NeighborhoodSystem a}
                    {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
                    {h_0 :
                      forall  (X : Set a),
                        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a
    V_0 X  ->  @Set.Nonempty a X}
                    {h_1 :
                      forall  (Y : Set b),
                        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b
    V_1 Y  ->  @Set.Nonempty b Y}
                    (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation
     a V_0)
                    (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation
     b V_1),
                    @DomainTheory.NeighbourhoodSystems.ComputableMap a (Option (Sum
    a b)) V_0
                      (@DomainTheory.NeighbourhoodSystems.sum a b V_0 V_1 h_0 h_1)
    P_0
                      (@DomainTheory.NeighbourhoodSystems.sumPresentation a b V_0
    V_1 h_0 h_1 P_0 P_1)
                      (@DomainTheory.NeighbourhoodSystems.inMap_0 a b V_0 V_1 h_0
    h_1))
                  @DomainTheory.NeighbourhoodSystems.sum_isEffectivelyGiven
                  @DomainTheory.NeighbourhoodSystems.inMap_0_isComputable)
                @DomainTheory.NeighbourhoodSystems.inMap_1_isComputable)
              @DomainTheory.NeighbourhoodSystems.outMap_0_isComputable)
            @DomainTheory.NeighbourhoodSystems.outMap_1_isComputable)
          @DomainTheory.NeighbourhoodSystems.sumMap_isComputable)
        @DomainTheory.NeighbourhoodSystems.paired_isComputable)
      @DomainTheory.NeighbourhoodSystems.prodMap_isComputable)
    @DomainTheory.NeighbourhoodSystems.prod_isEffectivelyGiven)
  @DomainTheory.NeighbourhoodSystems.proj_0_isComputable)
@DomainTheory.NeighbourhoodSystems.proj_1_isComputable
end

section
variable {a b ? : Type*}

/-- **Scott's Definition 7.1 (literal).** A *bare* computable presentation: an
    enumeration
`X : Nat  ->  Set a` onto `?`, with relations (i) `X_n  I  X? = X_k` and (ii)
    consistency recursively
decidable. No primitive-recursive intersection function is carried (cf. the docstring
    above). -/
structure ScottPresentation (V : NeighborhoodSystem a) where
```

```
  /-- The enumeration `? = {X_n ? n  in  Nat}`. -/
  X : Nat  ->  Set a
  /-- Every `X_n` is a neighbourhood. -/
  mem_X : forall  n, V.mem (X n)
  /-- The enumeration is onto `?`. -/
  surj : forall  {Y : Set a}, V.mem Y  ->  exists  n, X n = Y
  /-- **7.1(i)** `X_n  I  X? = X_k` recursively decidable. -/
  interEq_computable : RecDecidable_3 (fun n m k => X n  I  X m = X k)
  /-- **7.1(ii)** consistency `exists k. X_k  subseteq  X_n  I  X?` recursively
    decidable. -/
  cons_computable : RecDecidable_2 (fun n m => exists  k, X k  subseteq  X n  I  X m)

/-- **Definition 7.2 relative to bare presentations.** A computable map between bare
    presentations is
one whose neighbourhood relation `X_n f Y?` is r.e. (identical to `ComputableMap`,
    which only uses
the enumerations). -/
class ComputableMapS {V : NeighborhoodSystem a} {W : NeighborhoodSystem b}
    (P : ScottPresentation V) (Q : ScottPresentation W) (f : ApproximableMap V W) :
    Prop where
  out : REPred_2 (fun n m => f.rel (P.X n) (Q.X m))

/-- **Effectively given (Scott's literal Definition 7.1).** -/
def NeighborhoodSystem.IsEffectivelyGivenS (V : NeighborhoodSystem a) : Prop :=
  Nonempty (ScottPresentation V)

namespace ScottPresentation

variable {V : NeighborhoodSystem a} (P : ScottPresentation V)

/-- Reindexing `(n, m)  |->  (n, m, n)`. -/
def inclShuffle (t : Nat) : Nat := Nat.pair t.unpair.1 (Nat.pair t.unpair.2 t.unpair
    .1)

theorem primrec_inclShuffle : Nat.Primrec inclShuffle :=
  Nat.Primrec.pair Nat.Primrec.left (Nat.Primrec.pair Nat.Primrec.right Nat.Primrec.
    left)

/-- The inclusion relation is decidable: `X_n  subseteq  X?  <->  X_n  I  X? = X_n`. -/
theorem incl_computable : RecDecidable_2 (fun n m => P.X n  subseteq  P.X m) := by
  refine RecDecidable.of_iff (fun t => ?_) (P.interEq_computable.comp
    primrec_inclShuffle)
  simp only [inclShuffle, unpair_pair_fst, unpair_pair_snd]
  exact Set.inter_eq_left.symm

end ScottPresentation

end

section
variable {a b ? : Type*}
open Exercise510

section Smash

variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
```

```
  (P_0 : ComputablePresentation V_0) (P_1 : ComputablePresentation V_1)

open scoped Classical in
/-- **Smash enumeration.** Index `t  |-> ` the proper product `prodNbhd X?_{t.1} X1_{t
    .2}` when both
factors are proper, otherwise the master. The classical `if` (deciding `X_n = Delta_0`)
     is the *only*
classical input of the smash presentation. -/
noncomputable def smashEnum (t : Nat) : Set (a ? b) :=
  if P_0.X t.unpair.1  !=  V_0.master  /\  P_1.X t.unpair.2  !=  V_1.master then
    prodNbhd (P_0.X t.unpair.1) (P_1.X t.unpair.2)
  else prodNbhd V_0.master V_1.master

end Smash

section Smash

variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
  (P_0 : ComputablePresentation V_0) (P_1 : ComputablePresentation V_1)

variable {P_0 P_1}

open scoped Classical in
theorem smashEnum_master {t : Nat}
    (h : not (P_0.X t.unpair.1  !=  V_0.master  /\  P_1.X t.unpair.2  !=  V_1.master))
     :
    smashEnum P_0 P_1 t = prodNbhd V_0.master V_1.master := ite_eq_right h

open scoped Classical in
theorem smashEnum_proper {t : Nat}
    (h : P_0.X t.unpair.1  !=  V_0.master  /\  P_1.X t.unpair.2  !=  V_1.master) :
    smashEnum P_0 P_1 t = prodNbhd (P_0.X t.unpair.1) (P_1.X t.unpair.2) :=
    ite_eq_left h

/-- **The smash enumeration as a product neighbourhood with *effective* factors.**
    Given `{0,1}`
deciders `c_0`, `c_1` of properness, `smashEnum t = prodNbhd X?_{effL} X1_{effR}` where
     the effective
index `effL = (proper? t.1 : masterIdx_0)` is computed primitively by `selectFn`. This
    is the bridge
that turns the smash relations into the components' relations on the effective indices
    . -/
theorem smashEnum_eq_eff {c_0 c_1 : Nat  ->  Nat} (hc_0le : forall  n, c_0 n  <=  1) (
    hc_1le : forall  n, c_1 n  <=  1)
    (hc_0 : forall  n, P_0.X n  !=  V_0.master  <->  c_0 n = 1) (hc_1 : forall  n, P_1
    .X n  !=  V_1.master  <->  c_1 n = 1) (t : Nat) :
    smashEnum P_0 P_1 t =
      prodNbhd (P_0.X (selectFn (c_0 t.unpair.1 * c_1 t.unpair.2) t.unpair.1 P_0.
    masterIdx))
        (P_1.X (selectFn (c_0 t.unpair.1 * c_1 t.unpair.2) t.unpair.2 P_1.masterIdx))
    := by
  by_cases h : P_0.X t.unpair.1  !=  V_0.master  /\  P_1.X t.unpair.2  !=  V_1.master
  * have e0 : c_0 t.unpair.1 = 1 := (hc_0 _).mp h.1
    have e1 : c_1 t.unpair.2 = 1 := (hc_1 _).mp h.2
    rw [smashEnum_proper h, e0, e1, mul_one, selectFn_one, selectFn_one]
  * rw [smashEnum_master h]
```

```
    have hpf : c_0 t.unpair.1 * c_1 t.unpair.2 = 0 := by
      rcases not_and_or.mp h with h0 | h1
      * have hz : c_0 t.unpair.1 = 0 := by
          have hle := hc_0le t.unpair.1
          rcases (show c_0 t.unpair.1 = 0  \/  c_0 t.unpair.1 = 1 by omega) with h | h
          * exact h
          * exact absurd ((hc_0 _).mpr h) h0
        rw [hz, Nat.zero_mul]
      * have hz : c_1 t.unpair.2 = 0 := by
          have hle := hc_1le t.unpair.2
          rcases (show c_1 t.unpair.2 = 0  \/  c_1 t.unpair.2 = 1 by omega) with h | h
          * exact h
          * exact absurd ((hc_1 _).mpr h) h1
        rw [hz, Nat.mul_zero]
    rw [hpf, selectFn_zero, selectFn_zero, P_0.masterIdx_spec, P_1.masterIdx_spec]

/-- Every enumerated smash neighbourhood is a product `prodNbhd A B` of component
    neighbourhoods
(the proper form, or the master `prodNbhd Delta_0 Delta_1`). -/
theorem smashEnum_eq_prodNbhd_mem (k : Nat) :
    exists  A B, V_0.mem A  /\  V_1.mem B  /\  smashEnum P_0 P_1 k = prodNbhd A B :=
    by
  by_cases h : P_0.X k.unpair.1  !=  V_0.master  /\  P_1.X k.unpair.2  !=  V_1.master
  * exact <_, _, P_0.mem_X _, P_1.mem_X _, smashEnum_proper h>
  * exact <_, _, V_0.master_mem, V_1.master_mem, smashEnum_master h>

theorem smashEnum_mem (t : Nat) : (smash V_0 V_1).mem (smashEnum P_0 P_1 t) := by
  by_cases h : P_0.X t.unpair.1  !=  V_0.master  /\  P_1.X t.unpair.2  !=  V_1.master
  * rw [smashEnum_proper h]; exact smash_mem_proper (P_0.mem_X _) h.1 (P_1.mem_X _) h
    .2
  * rw [smashEnum_master h]; exact (smash V_0 V_1).master_mem

theorem smashEnum_subset_master (k : Nat) :
    smashEnum P_0 P_1 k  subseteq  prodNbhd V_0.master V_1.master := by
  have h := (smash V_0 V_1).sub_master (smashEnum_mem (P_0 := P_0) (P_1 := P_1) k)
  rwa [smash_master] at h

end Smash

end

section
variable {a b ? : Type*}

namespace ScottPresentation

variable {V : NeighborhoodSystem a} (P : ScottPresentation V)

def swapPair (t : Nat) : Nat := Nat.pair t.unpair.2 t.unpair.1

theorem primrec_swapPair : Nat.Primrec swapPair :=
  Nat.Primrec.pair Nat.Primrec.right Nat.Primrec.left

/-- Equality of neighbourhoods is decidable. -/
theorem eq_computable : RecDecidable_2 (fun n m => P.X n = P.X m) := by
  refine RecDecidable.of_iff (fun t => ?_)
```

```
    (P.incl_computable.and (P.incl_computable.comp primrec_swapPair))
  simp only [swapPair, unpair_pair_fst, unpair_pair_snd]
  exact Set.Subset.antisymm_iff

end ScottPresentation
end

section
variable {a b ? : Type*}
open Exercise510

section Smash

variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
  (P_0 : ComputablePresentation V_0) (P_1 : ComputablePresentation V_1)

variable {P_0 P_1}

variable (P_0 P_1)

/-- The properness predicate `X_n  !=  Delta_0` is recursively decidable (negation of `
    eq_computable`
against the master index). -/
theorem proper_0_dec : RecDecidable (fun n => P_0.X n  !=  V_0.master) := by
  have heq : RecDecidable (fun n => P_0.X n = V_0.master) := by
    refine RecDecidable.of_iff (fun n => ?_)
      (P_0.eq_computable.comp (primrec_id.pair (Nat.Primrec.const P_0.masterIdx)))
    simp only [unpair_pair_fst, unpair_pair_snd, P_0.masterIdx_spec, id_eq]
  exact heq.not

theorem proper_1_dec : RecDecidable (fun n => P_1.X n  !=  V_1.master) := by
  have heq : RecDecidable (fun n => P_1.X n = V_1.master) := by
    refine RecDecidable.of_iff (fun n => ?_)
      (P_1.eq_computable.comp (primrec_id.pair (Nat.Primrec.const P_1.masterIdx)))
    simp only [unpair_pair_fst, unpair_pair_snd, P_1.masterIdx_spec, id_eq]
  exact heq.not

end Smash

end
end DomainTheory.NeighbourhoodSystems
end
```

3.8.6 Theorem 7.5 — function space effective

THEOREM 7.5. If $\mathcal{D}_0$ and $\mathcal{D}_1$ are effectively given, then so is $(\mathcal{D}_0 \to \mathcal{D}_1)$. The combinators eval and curry are computable, provided all the domains involved are effectively given. The computable elements $f \in |\mathcal{D}_0 \to \mathcal{D}_1|$ are exactly the computable maps $f : \mathcal{D}_0 \to \mathcal{D}_1$.

Proof: The proofs of 3.9, 3.11, and 3.12 were set up with this theorem in mind. If

$$\mathcal{D}_0 = \{X_n \mid n \in \mathbf{N}\} \quad \text{and} \quad \mathcal{D}_1 = \{Y_m \mid m \in \mathbf{N}\}$$

are two effectively given neighbourhood systems, then the neighbourhoods of $(\mathcal{D}_0 \to \mathcal{D}_1)$, by Definition

3.8, are non-empty intersections like

$$\bigcap_{i<q}[X_{n_i}, Y_{m_i}],$$

where $\langle n_0, n_1, \ldots, n_{q-1}\rangle$ and $\langle m_0, m_1, \ldots, m_{q-1}\rangle$ are two finite sequences of integers determining the choice of the function-space neighbourhood. In 3.9(i) the test for nonemptiness is given. Assuming the decidability of relations in $\mathcal{D}_0$ and $\mathcal{D}_1$, one remarks that the consistency of *finite sequences* of neighbourhoods is also decidable. (Hint: Test the first *two*, then form their intersection. Next test the third given neighbourhood against this one set; if consistent, form the intersection, and carry on.) By 3.9(i) at most $2 \cdot 2^q$ such sequential checks must be carried out to determine whether the function-space neighbourhood is non empty.

It may not be fun, but the checks can be carried out in finite time. Owing to this decidability, we can therefore enumerate in a systematic way *all* the pairs of finite sequences $\langle n_0, \ldots\rangle$ and $\langle m_0, \ldots\rangle$ that determine neighbourhoods: that is the way that $(\mathcal{D}_0 \to \mathcal{D}_1)$ obtains its enumeration.

Concerning the decidability of the required relations on $(\mathcal{D}_0 \to \mathcal{D}_1)$, we remark first off that consistency is more of the same: to test two finite intersections against each other, just form one big intersection and test it for non-emptiness as before. Secondly, the testing for intersection comes down in the end to testing one typical intersection of $[X, Y]$-neighbourhoods for equality with another. But equality amounts to two inclusions; inclusion in an intersection amounts to inclusion in each term. Therefore, what we need to do is to check a finite number of statements of the form:

$$\bigcap_{i<q}[X_{n_i}, Y_{m_i}] \subseteq [X_k, Y_\ell].$$

As we pointed out after the proof of 3.9, this inclusion is equivalent to

$$\bigcap\{Y_{m_i} \mid X_k \subseteq X_{n_i}\} \subseteq Y_\ell.$$

By decidability in $\mathcal{D}_0$, we can effectively find the n_i that are needed. Then in $\mathcal{D}_1$, we form the intersection of the corresponding Y_{m_i}. Finally, we check the inclusion. Again, one check in $(\mathcal{D}_0 \to \mathcal{D}_1)$ requires a whole sequence of checks in $\mathcal{D}_0$ and in $\mathcal{D}_1$, but the process is finite. So we have argued that $(\mathcal{D}_0 \to \mathcal{D}_1)$ is effectively given.

In showing that the combinators are computable, we refer first to the proof of 3.11. The typical pair of neighbourhoods possibly belonging to eval is

$$\bigcap_{i<q}[X_{n_i}, Y_{m_i}],\ X_k \text{ eval } Y_\ell.$$

As we needed not to be so specific, we expressed the holding of this relationship in terms of *all* the functions in the function-

space neighbourhood. But we know that the neighbourhood, by 3.9(ii), has a minimal element; it is then sufficient to test for the holding of $X_k f_0 Y_\ell$ at this minimal function f_0. But this test, we have already seen, is decidable. So the pairs in eval actually form a recursive set, not just a recursively enumerable set; thus, eval is a computable function.

The case of curry involves three domains and is a bit more messy. But again, if the required neighbourhoods are written out in full, it will be seen that curry, too, is computable. We leave this minor struggle to the exercises.

The final statement is an easy consequence of the fundamental connection between approximable $f : \mathcal{D}_0 \to \mathcal{D}_1$ as relations and as elements. Recall, as in the proof of 3.10, that we have

$$f \in [X, Y] \text{ iff } X f Y,$$

for all $X \in \mathcal{D}_0$ and $Y \in \mathcal{D}_1$. Therefore,

$$f \in \bigcap_{i<q} [X_{n_i}, Y_{m_i}] \text{ iff } \forall i < q.\, X_{n_i} f Y_{m_i}.$$

It follows that if f is recursively enumerable as a set of pairs, then, by forming all the non-empty intersections (as shown), we get an enumeration of all the neighbourhoods to which f belongs; and this is the same as the filter corresponding to f as an element of the function space. The converse direction is clear. □

We have nearly all our favourite combinators computable, but perhaps the most important one - since it is the key to recursive definitions - is the fixed-point combinator. It is not left out.

Theorem_7_5.lean

```
module

public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.Table_5_5
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Proposition_7_12

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem Domain.Recursive ApproximableMap
variable {a b ? : Type*} {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
variable (P_0 : ComputablePresentation V_0) (P_1 : ComputablePresentation V_1)

/- concordance: source L4361 THEOREM 7.5 function space effective -/
/-- **Theorem 7.5 (existence part) (Scott 1981, PRG-19).** If `?_0` and `?_1` are
    effectively given,
then so is the function space `(?_0  ->  ?_1)`. Choice-free. -/
theorem funSpace_isEffectivelyGiven (h_0 : V_0.IsEffectivelyGiven) (h_1 : V_1.
    IsEffectivelyGiven) :
    (funSpace V_0 V_1).IsEffectivelyGiven := by
  obtain <P_0> := h_0
  obtain <P_1> := h_1
  obtain <incl0, hincl0p, hincl0s> := P_0.incl_computable
  obtain <incl1, hincl1p, hincl1s> := P_1.incl_computable
  obtain <eq1, heq1p, heq1s> := P_1.eq_computable
  obtain <fc0, hfc0p, hfc0s> := P_0.cons_computable
  obtain <fc1, hfc1p, hfc1s> := P_1.cons_computable
  exact <funPresentation P_0 P_1 (funConsChar P_0 P_1 fc0 fc1) incl0 incl1 eq1
    (funConsChar_spec P_0 P_1 fc0 fc1 (fun s => (hfc0s s).symm) (fun s => (hfc1s s).
    symm))
    (primrec_funConsChar P_0 P_1 fc0 fc1 hfc0p hfc1p)
    (fun s => (hincl0s s).symm) hincl0p
    (fun s => (hincl1s s).symm) hincl1p
    (fun s => (heq1s s).symm) heq1p>
```

```
/-- **The step neighbourhood `[X_0?, Y_1?]` as a one-entry `Xenum`.** The code `<<j, m
    >, 0> + 1`
decodes to the singleton entry-list `[<j, m>]`, which is always consistent (the step
    map itself is a
witness), so `Xenum` of it is `stepFun [(X_0?, Y_1?)] = [X_0?, Y_1?]`. -/
theorem Xenum_singleton (gN : Nat  ->  Nat)
    (hgN : forall  c, gN c = 1  <->  (stepFun (funListOf P_0 P_1 (decodeList c))
      : Set (ApproximableMap V_0 V_1)).Nonempty) (j m : Nat) :
    Xenum P_0 P_1 gN (Nat.pair (Nat.pair j m) 0 + 1) = step (P_0.X j) (P_1.X m) := by
  have hfp : funPair P_0 P_1 (Nat.pair j m) = (P_0.X j, P_1.X m) := by
    show (P_0.X (Nat.pair j m).unpair.1, P_1.X (Nat.pair j m).unpair.2) = (P_0.X j,
    P_1.X m)
    rw [unpair_pair_fst, unpair_pair_snd]
  have hdec : decodeList (Nat.pair (Nat.pair j m) 0 + 1) = [Nat.pair j m] := by
    rw [decodeList_succ, unpair_pair_fst, unpair_pair_snd, decodeList_zero]
  have hstepeq : (stepFun (funListOf P_0 P_1 [Nat.pair j m]) : Set (ApproximableMap
    V_0 V_1))
      = step (P_0.X j) (P_1.X m) := by
    show stepFun [funPair P_0 P_1 (Nat.pair j m)] = step (P_0.X j) (P_1.X m)
    rw [hfp, stepFun_singleton]
  have hcons : gN (Nat.pair (Nat.pair j m) 0 + 1) = 1 := by
    rw [hgN, hdec, hstepeq]; exact (step_mem (P_0.mem_X j) (P_1.mem_X m)).2
  rw [Xenum_pos P_0 P_1 hcons, hdec, hstepeq]

section ConsChain

variable {V : NeighborhoodSystem a} (P : ComputablePresentation V)

/-- **Consistency via the `inter`-chain (choice-free).** A finite index set `js` is
    consistent -- i.e.
`?{X_j ? j  in  js}` is a neighbourhood -- exactly when the always-a-neighbourhood `X_{
    idxchain js}` sits
inside every `X_j` (`j  in  js`). -/
theorem consChain_iff (js : List Nat) :
    (forall  j  in  js, P.X (idxchain P js)  subseteq  P.X j)  <->  V.mem (interFrom P
     V.master js) := by
  constructor
  * intro hle
    have hsub : P.X (idxchain P js)  subseteq  interFrom P V.master js := by
      intro z hz
      rw [mem_interFrom]
      exact <V.sub_master (P.mem_X _) hz, fun j hj => hle j hj hz>
    exact interFrom_mem_of_witness P (P.mem_X _) hsub V.master_mem
  * intro hmem j hj
    rw [idxchain_spec P hmem]
    exact fun z hz => (mem_interFrom P |>.mp hz).2 j hj

end ConsChain

/-- **Evaluation against a product neighbourhood.** `(F, X) eval Y  <->  F  subseteq  [
    X, Y]`, for any
function-space neighbourhood `F` and basic neighbourhoods `X, Y`: every map in `F`
    relates `X` to `Y`
exactly when `F` is contained in the step set `[X, Y] = {f ? f X Y}`. -/
theorem evalMap_rel_prodNbhd_iff {F : Set (ApproximableMap V_0 V_1)} (hF : (funSpace
    V_0 V_1).mem F)
```

```
    {X : Set a} (hX : V_0.mem X) {Y : Set b} (hY : V_1.mem Y) :
    (evalMap V_0 V_1).rel (prodNbhd F X) Y  <->  F  subseteq  step X Y := by
  show (prod (funSpace V_0 V_1) V_0).mem (prodNbhd F X)  /\
    (eval V_0 V_1).rel (Sum.inl ^-1' prodNbhd F X) (Sum.inr ^-1' prodNbhd F X) Y  <->
    _
  rw [inl_preimage_prodNbhd, inr_preimage_prodNbhd]
  constructor
  * rintro <_, _, _, _, hrel> f hf; exact mem_step.mpr (hrel f hf)
  * intro hsub
    exact <prod_mem_prodNbhd hF hX, hF, hX, hY, fun f hf => mem_step.mp (hsub hf)>

/-- **Theorem 7.5 (Scott 1981, PRG-19) -- `eval` is computable (choice-free).**
    Relative to the
function-space presentation `funPresentation` (and the product/codomain presentations),
     the
evaluation map `eval : (?_0  ->  ?_1)  x  ?_0  ->  ?_1` is computable: its
    neighbourhood relation is the
*recursively decidable* function-space inclusion `Xenum c  subseteq  [X_0?, Y_1?]`,
    hence recursively
enumerable. -/
theorem evalMap_isComputable (gN incl0 incl1 eq1 : Nat  ->  Nat)
    (hgN : forall  c, gN c = 1  <->  (stepFun (funListOf P_0 P_1 (decodeList c))
      : Set (ApproximableMap V_0 V_1)).Nonempty) (hgNp : Nat.Primrec gN)
    (hincl0 : forall  s, incl0 s = 1  <->  P_0.X s.unpair.1  subseteq  P_0.X s.unpair
    .2) (hincl0p : Nat.Primrec incl0)
    (hincl1 : forall  s, incl1 s = 1  <->  P_1.X s.unpair.1  subseteq  P_1.X s.unpair
    .2) (hincl1p : Nat.Primrec incl1)
    (heq1 : forall  s, eq1 s = 1  <->  P_1.X s.unpair.1 = P_1.X s.unpair.2) (heq1p :
    Nat.Primrec eq1) :
    ComputableMap
      (prodPresentation (funPresentation P_0 P_1 gN incl0 incl1 eq1 hgN hgNp hincl0
    hincl0p
        hincl1 hincl1p heq1 heq1p) P_0) P_1 (evalMap V_0 V_1) := by
  constructor
  -- The function-space inclusion is recursively decidable (`funPresentation.
    incl_computable`);
  -- `funPresentation.X` is definitionally `Xenum`, so we read it off directly.
  have hsub : RecDecidable (fun s => Xenum P_0 P_1 gN s.unpair.1  subseteq  Xenum P_0
    P_1 gN s.unpair.2) :=
    (funPresentation P_0 P_1 gN incl0 incl1 eq1 hgN hgNp hincl0 hincl0p
      hincl1 hincl1p heq1 heq1p).incl_computable
  -- Re-index `t  |->  <(t.1).1, <<(t.1).2, t.2>, 0> + 1>`: `Xenum_{(t.1).1}  subseteq
     [X_0_{(t.1).2}, Y_1_{t.2}]`.
  have h11 : Nat.Primrec (fun t : Nat => t.unpair.1.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.left
  have h12 : Nat.Primrec (fun t : Nat => t.unpair.1.unpair.2) :=
    Nat.Primrec.right.comp Nat.Primrec.left
  have hg : Nat.Primrec (fun t => Nat.pair t.unpair.1.unpair.1
      (Nat.pair (Nat.pair t.unpair.1.unpair.2 t.unpair.2) 0 + 1)) :=
    h11.pair (Nat.Primrec.succ.comp ((h12.pair Nat.Primrec.right).pair (Nat.Primrec.
    const 0)))
  refine (RecDecidable.of_iff (fun t => ?_) (hsub.comp hg)).re
  simp only [unpair_pair_fst, unpair_pair_snd]
  rw [Xenum_singleton P_0 P_1 gN hgN t.unpair.1.unpair.2 t.unpair.2,
    show (prodPresentation (funPresentation P_0 P_1 gN incl0 incl1 eq1 hgN hgNp hincl0
     hincl0p
```

```
        hincl1 hincl1p heq1 heq1p) P_0).X t.unpair.1
      = prodNbhd (Xenum P_0 P_1 gN t.unpair.1.unpair.1) (P_0.X t.unpair.1.unpair.2)
    from rfl,
    evalMap_rel_prodNbhd_iff (Xenum_mem P_0 P_1 gN hgN _) (P_0.mem_X _) (P_1.mem_X _)]

theorem funListOf_append (la lb : List Nat) :
    funListOf P_0 P_1 (la ++ lb) = funListOf P_0 P_1 la ++ funListOf P_0 P_1 lb := by
  simp [funListOf, List.map_append]

end

section
open NeighborhoodSystem Domain.Recursive ApproximableMap
variable {a b ? : Type*} {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}

theorem interList_subset_base {base : Set b} {M : List (Set b)} : interList base M
    subseteq  base :=
  fun _ hz => (mem_interList.mp hz).1
end

section
open NeighborhoodSystem Domain.Recursive ApproximableMap
variable {a b ? : Type*} {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
variable (P_0 : ComputablePresentation V_0) (P_1 : ComputablePresentation V_1)

/-- **Bridge.** `phi ? Xenum c` iff, *when `c` is consistent*, `phi` contains every
    constituent step. -/
theorem mem_Xenum_iff (gN : Nat  ->  Nat) (phi : (funSpace V_0 V_1).Element) (c : Nat)
     :
    phi.mem (Xenum P_0 P_1 gN c)  <->
      (gN c = 1  ->  forall  e  in  decodeList c, phi.mem (step (P_0.X e.unpair.1) (
    P_1.X e.unpair.2))) := by
  by_cases h : gN c = 1
  * rw [Xenum_pos P_0 P_1 h, mem_stepFun_iff phi (funListOf_valid P_0 P_1 (decodeList
    c))]
    constructor
    * intro hp _ e he
      exact hp (funPair P_0 P_1 e) (List.mem_map.mpr <e, he, rfl>)
    * intro himp p hp
      rw [funListOf, List.mem_map] at hp
      obtain <e, he, rfl> := hp
      exact himp h e he
  * rw [Xenum_neg P_0 P_1 h]
    constructor
    * intro _ hcon; exact absurd hcon h
    * intro _; rw [ <-  funSpace_master]; exact phi.master_mem

/-- **Theorem 7.5 (Scott 1981, PRG-19) -- computable elements = computable maps (
    choice-free).** For
the function-space enumeration `Xenum` built from a consistency char `gN`, an element `
    phi` has an r.e.
index set iff the corresponding approximable map `toApproxMap phi` is computable. -/
theorem Xenum_isComputableElement_iff (gN : Nat  ->  Nat) (hgNp : Nat.Primrec gN)
    (hgN : forall  c, gN c = 1  <->  (stepFun (funListOf P_0 P_1 (decodeList c))
      : Set (ApproximableMap V_0 V_1)).Nonempty) (phi : (funSpace V_0 V_1).Element) :
    REPred (fun c => phi.mem (Xenum P_0 P_1 gN c))  <->  ComputableMap P_0 P_1 (
```

```
    toApproxMap phi) := by
  -- `ComputableMap` unfolds (def-eq) to r.e.-ness of `R' s = phi ? [X_0_{s.1}, X_1_{s
    .2}]`.
  have hbridge := mem_Xenum_iff P_0 P_1 gN phi
  constructor
  * -- element computable ? map computable: reindex by the single-step code `pair s 0 +
     1`.
    intro helem
    have hc0p : Nat.Primrec (fun s => Nat.pair s 0 + 1) :=
      Nat.Primrec.succ.comp (primrec_id.pair (Nat.Primrec.const 0))
    have key : REPred (fun s => phi.mem (Xenum P_0 P_1 gN (Nat.pair s 0 + 1))) :=
    helem.comp hc0p
    constructor
    show REPred (fun s => phi.mem (step (P_0.X s.unpair.1) (P_1.X s.unpair.2)))
    refine REPred.of_iff (fun s => ?_) key
    have hdec : decodeList (Nat.pair s 0 + 1) = [s] := by
      rw [decodeList_succ, unpair_pair_fst, unpair_pair_snd, decodeList_zero]
    have hcons : gN (Nat.pair s 0 + 1) = 1 := by
      rw [hgN, hdec]
      show (stepFun [(P_0.X s.unpair.1, P_1.X s.unpair.2)]
        : Set (ApproximableMap V_0 V_1)).Nonempty
      rw [stepFun_singleton]
      exact (step_mem (P_0.mem_X _) (P_1.mem_X _)).2
    rw [hbridge (Nat.pair s 0 + 1), hdec]
    constructor
    * intro hRs _ e he; rw [List.mem_singleton] at he; subst he; exact hRs
    * intro h; exact h hcons s (List.mem_singleton.mpr rfl)
  * -- map computable ? element computable: guard the bounded-`forall ` by the
    decidable `gN c = 1`.
    intro hmap
    have hR' : REPred (fun s => phi.mem (step (P_0.X s.unpair.1) (P_1.X s.unpair.2)))
    := hmap.out
    have hforall : REPred (fun c => forall  e  in  decodeList c,
        phi.mem (step (P_0.X e.unpair.1) (P_1.X e.unpair.2))) := hR'.
    forall_mem_decodeList
    have hne1 : REPred (fun c => not  gN c = 1) :=
      ((RecDecidable.natEq hgNp (Nat.Primrec.const 1)).not).re
    refine REPred.of_iff (fun c => ?_) (hne1.or hforall)
    rw [hbridge c]; exact Decidable.imp_iff_not_or

/-- **Theorem 7.5 (Scott 1981, PRG-19), packaged for `funPresentation`.** An element of
     the
function-space presentation is a computable element iff it is (the filter of) a
    computable map. -/
theorem isComputableElement_funPresentation_iff (gN incl0 incl1 eq1 : Nat  ->  Nat)
    (hgN : forall  c, gN c = 1  <->  (stepFun (funListOf P_0 P_1 (decodeList c))
      : Set (ApproximableMap V_0 V_1)).Nonempty) (hgNp : Nat.Primrec gN)
    (hincl0 : forall  s, incl0 s = 1  <->  P_0.X s.unpair.1  subseteq  P_0.X s.unpair
    .2) (hincl0p : Nat.Primrec incl0)
    (hincl1 : forall  s, incl1 s = 1  <->  P_1.X s.unpair.1  subseteq  P_1.X s.unpair
    .2) (hincl1p : Nat.Primrec incl1)
    (heq1 : forall  s, eq1 s = 1  <->  P_1.X s.unpair.1 = P_1.X s.unpair.2) (heq1p :
    Nat.Primrec eq1)
    (phi : (funSpace V_0 V_1).Element) :
    ComputableElement (funPresentation P_0 P_1 gN incl0 incl1 eq1 hgN hgNp hincl0
    hincl0p
```

```
        hincl1 hincl1p heq1 heq1p) phi  <->  ComputableMap P_0 P_1 (toApproxMap phi)
    :=
  <fun h => (Xenum_isComputableElement_iff P_0 P_1 gN hgNp hgN phi).mp h.out,
    fun h => <(Xenum_isComputableElement_iff P_0 P_1 gN hgNp hgN phi).mpr h>>

/-- **Membership in `Xenum` for a single map.** A map `f` lies in `Xenum c` iff (when `
    c` is
consistent) it relates every coded step `<X_0_i, X_1?>`; junk codes give `univ`. (The
    single-map analogue
of `mem_Xenum_iff`.) -/
theorem mem_Xenum_iff_map (gN : Nat  ->  Nat) (f : ApproximableMap V_0 V_1) (c : Nat)
    :
    f  in  Xenum P_0 P_1 gN c  <->
      (gN c = 1  ->  forall  e  in  decodeList c, f.rel (P_0.X e.unpair.1) (P_1.X e.
    unpair.2)) := by
  by_cases h : gN c = 1
  * rw [Xenum_pos P_0 P_1 h, mem_stepFun]
    constructor
    * intro hp _ e he
      exact hp (funPair P_0 P_1 e) (List.mem_map.mpr <e, he, rfl>)
    * intro himp p hp
      rw [funListOf, List.mem_map] at hp
      obtain <e, he, rfl> := hp
      exact himp h e he
  * rw [Xenum_neg P_0 P_1 h]
    constructor
    * intro _ hcon; exact absurd hcon h
    * intro _; exact Set.mem_univ f

/-- **Bridge for `curry`.** `(X_0_n) curry(g) (Xenum c)` iff, *when `c` is consistent*,
     `g` relates
`X_0_n  U  X_1_{e.1}` to `X_2_{e.2}` for every coded entry `e`. -/
theorem curry_rel_Xenum_iff {V_2 : NeighborhoodSystem ?} (P_2 : ComputablePresentation
     V_2)
    (gN : Nat  ->  Nat)
    (hgN : forall  c, gN c = 1  <->  (stepFun (funListOf P_1 P_2 (decodeList c))
      : Set (ApproximableMap V_1 V_2)).Nonempty)
    (g : ApproximableMap (prod V_0 V_1) V_2) (n c : Nat) :
    (curry g).rel (P_0.X n) (Xenum P_1 P_2 gN c)  <->
      (gN c = 1  ->  forall  e  in  decodeList c,
        g.rel (prodNbhd (P_0.X n) (P_1.X e.unpair.1)) (P_2.X e.unpair.2)) := by
  rw [curry_rel]
  constructor
  * rintro <hX, _, hmem>
    rw [mem_Xenum_iff_map P_1 P_2 gN (gSection g hX) c] at hmem
    intro hc e he
    have hrel := hmem hc e he
    rwa [gSection_rel] at hrel
  * intro himp
    refine <P_0.mem_X n, Xenum_mem P_1 P_2 gN hgN c, ?_>
    rw [mem_Xenum_iff_map P_1 P_2 gN (gSection g (P_0.mem_X n)) c]
    intro hc e he
    rw [gSection_rel]
    exact himp hc e he

/-- **Theorem 7.5 (Scott 1981, PRG-19) -- `curry` is computable (choice-free).** If `g
```

```
    : ?_0  x  ?_1  ->  ?_2`
is computable then so is `curry g : ?_0  ->  (?_1  ->  ?_2)`: its neighbourhood
    relation is the parameterised
bounded `forall ` (`REPred.forall_mem_decodeList_2`) over the r.e. relation of `g`,
    guarded by the decidable
consistency flag. -/
theorem curry_isComputable {V_2 : NeighborhoodSystem ?} (P_2 : ComputablePresentation
    V_2)
    (gN incl0 incl1 eq1 : Nat  ->  Nat)
    (hgN : forall  c, gN c = 1  <->  (stepFun (funListOf P_1 P_2 (decodeList c))
      : Set (ApproximableMap V_1 V_2)).Nonempty) (hgNp : Nat.Primrec gN)
    (hincl0 : forall  s, incl0 s = 1  <->  P_1.X s.unpair.1  subseteq  P_1.X s.unpair
    .2) (hincl0p : Nat.Primrec incl0)
    (hincl1 : forall  s, incl1 s = 1  <->  P_2.X s.unpair.1  subseteq  P_2.X s.unpair
    .2) (hincl1p : Nat.Primrec incl1)
    (heq1 : forall  s, eq1 s = 1  <->  P_2.X s.unpair.1 = P_2.X s.unpair.2) (heq1p :
    Nat.Primrec eq1)
    {g : ApproximableMap (prod V_0 V_1) V_2}
    (hg : ComputableMap (prodPresentation P_0 P_1) P_2 g) :
    ComputableMap P_0 (funPresentation P_1 P_2 gN incl0 incl1 eq1 hgN hgNp hincl0
    hincl0p
      hincl1 hincl1p heq1 heq1p) (curry g) := by
  constructor
  have hg' : REPred (fun s => g.rel ((prodPresentation P_0 P_1).X s.unpair.1) (P_2.X s
    .unpair.2)) := hg.out
  have hre : Nat.Primrec (fun s => Nat.pair (Nat.pair s.unpair.1 s.unpair.2.unpair.1)
      s.unpair.2.unpair.2) :=
    (Nat.Primrec.left.pair (Nat.Primrec.left.comp Nat.Primrec.right)).pair
      (Nat.Primrec.right.comp Nat.Primrec.right)
  -- `g`'s relation re-indexed to `(n, e)  |->  X_0_n  U  X_1_{e.1} g X_2_{e.2}` is r.e
    . in `(n, e)`.
  have hS : REPred_2 (fun n e =>
      g.rel (prodNbhd (P_0.X n) (P_1.X e.unpair.1)) (P_2.X e.unpair.2)) := by
    refine REPred.of_iff (fun s => ?_) (hg'.comp hre)
    simp only [prodPresentation_X, unpair_pair_fst, unpair_pair_snd]
  -- bounded `forall ` over the coded list, with parameter `n = t.1`
  have hforall : REPred (fun t => forall  e  in  decodeList t.unpair.2,
      g.rel (prodNbhd (P_0.X t.unpair.1) (P_1.X e.unpair.1)) (P_2.X e.unpair.2)) :=
    REPred.forall_mem_decodeList_2 hS
  have hne1 : REPred (fun t => not  gN t.unpair.2 = 1) :=
    ((RecDecidable.natEq (hgNp.comp Nat.Primrec.right) (Nat.Primrec.const 1)).not).re
  refine REPred.of_iff (fun t => ?_) (hne1.or hforall)
  show (curry g).rel (P_0.X t.unpair.1) (Xenum P_1 P_2 gN t.unpair.2)  <->  _
  rw [curry_rel_Xenum_iff P_0 P_1 P_2 gN hgN g t.unpair.1 t.unpair.2]
  exact Decidable.imp_iff_not_or

end

section
set_option linter.unusedVariables false

theorem theorem_75.{u_1, u_2, u_3} : And
  (And
    (And
      (And
        (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
```

```
NeighbourhoodSystems.NeighborhoodSystem a}
      {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
      (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0)
      (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1) (gN
: Nat  ->  Nat),
      Nat.Primrec gN  ->
        (forall  (c : Nat),
           Iff (@Eq Nat (gN c) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (
nat_lit 1))))
             (@Set.Nonempty (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
 b V_0 V_1)
               (@DomainTheory.NeighbourhoodSystems.stepFun a b V_0 V_1
                 (@DomainTheory.NeighbourhoodSystems.funListOf a b V_0 V_1 P_0
P_1
                   (Domain.Recursive.decodeList c)))))  ->
          forall
            (phi :
              @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
                (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1)
                (@DomainTheory.NeighbourhoodSystems.funSpace a b V_0 V_1)),
            Iff
              (Domain.Recursive.REPred fun (c : Nat) =>
                @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem
                  (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1)
                  (@DomainTheory.NeighbourhoodSystems.funSpace a b V_0 V_1) phi
                  (@DomainTheory.NeighbourhoodSystems.Xenum a b V_0 V_1 P_0 P_1 gN
 c))
              (@DomainTheory.NeighbourhoodSystems.ComputableMap a b V_0 V_1 P_0
P_1
                (@DomainTheory.NeighbourhoodSystems.toApproxMap a b V_0 V_1 phi)))
    (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
      {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b},
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven a
V_0  ->
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven b
 V_1  ->
          @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1)
            (@DomainTheory.NeighbourhoodSystems.funSpace a b V_0 V_1)))
  (forall  {a : Type u_1} {b : Type u_2} {? : Type u_3} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
    {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
    (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0)
    (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1)
    {V_2 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem ?}
    (P_2 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation ? V_2) (gN
incl0 incl1 eq1 : Nat  ->  Nat)
    (hgN :
      forall  (c : Nat),
        Iff (@Eq Nat (gN c) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit
1))))
          (@Set.Nonempty (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ?
V_1 V_2)
            (@DomainTheory.NeighbourhoodSystems.stepFun b ? V_1 V_2
              (@DomainTheory.NeighbourhoodSystems.funListOf b ? V_1 V_2 P_1 P_2 (
```

```
Domain.Recursive.decodeList c)))))
     (hgNp : Nat.Primrec gN)
     (hincl0 :
       forall  (s : Nat),
         Iff (@Eq Nat (incl0 s) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (
nat_lit 1))))
           (@LE.le (Set b) (@Set.instLE b)
             (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b V_1 P_1
 (@Prod.fst Nat Nat (Nat.unpair s)))
             (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b V_1 P_1
               (@Prod.snd Nat Nat (Nat.unpair s)))))
     (hincl0p : Nat.Primrec incl0)
     (hincl1 :
       forall  (s : Nat),
         Iff (@Eq Nat (incl1 s) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (
nat_lit 1))))
           (@LE.le (Set ?) (@Set.instLE ?)
             (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X ? V_2 P_2
 (@Prod.fst Nat Nat (Nat.unpair s)))
             (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X ? V_2 P_2
               (@Prod.snd Nat Nat (Nat.unpair s)))))
     (hincl1p : Nat.Primrec incl1)
     (heq1 :
       forall  (s : Nat),
         Iff (@Eq Nat (eq1 s) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit
1))))
           (@Eq (Set ?)
             (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X ? V_2 P_2
 (@Prod.fst Nat Nat (Nat.unpair s)))
             (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X ? V_2 P_2
               (@Prod.snd Nat Nat (Nat.unpair s)))))
     (heq1p : Nat.Primrec eq1)
     {g :
       @DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
         (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2},
     @DomainTheory.NeighbourhoodSystems.ComputableMap (Sum a b) ?
         (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2
         (@DomainTheory.NeighbourhoodSystems.prodPresentation a b V_0 V_1 P_0 P_1)
P_2 g  ->
       @DomainTheory.NeighbourhoodSystems.ComputableMap a
         (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
         (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2) P_0
         (@DomainTheory.NeighbourhoodSystems.funPresentation b ? V_1 V_2 P_1 P_2 gN
 incl0 incl1 eq1 hgN hgNp hincl0
           hincl0p hincl1 hincl1p heq1 heq1p)
         (@DomainTheory.NeighbourhoodSystems.curry a b ? V_0 V_1 V_2 g)))
(forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
   {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
   (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0)
   (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1) (gN
incl0 incl1 eq1 : Nat  ->  Nat)
   (hgN :
     forall  (c : Nat),
       Iff (@Eq Nat (gN c) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))
))
```

```
         (@Set.Nonempty (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0
  V_1)
           (@DomainTheory.NeighbourhoodSystems.stepFun a b V_0 V_1
             (@DomainTheory.NeighbourhoodSystems.funListOf a b V_0 V_1 P_0 P_1 (
  Domain.Recursive.decodeList c)))))
    (hgNp : Nat.Primrec gN)
    (hincl0 :
      forall  (s : Nat),
        Iff (@Eq Nat (incl0 s) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit
  1))))


         (@LE.le (Set a) (@Set.instLE a)
           (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V_0 P_0 (
  @Prod.fst Nat Nat (Nat.unpair s)))
           (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V_0 P_0 (
  @Prod.snd Nat Nat (Nat.unpair s)))))
    (hincl0p : Nat.Primrec incl0)
    (hincl1 :
      forall  (s : Nat),
        Iff (@Eq Nat (incl1 s) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit
  1))))
         (@LE.le (Set b) (@Set.instLE b)
           (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b V_1 P_1 (
  @Prod.fst Nat Nat (Nat.unpair s)))
           (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b V_1 P_1 (
  @Prod.snd Nat Nat (Nat.unpair s)))))
    (hincl1p : Nat.Primrec incl1)
    (heq1 :
      forall  (s : Nat),
        Iff (@Eq Nat (eq1 s) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)
  )))
         (@Eq (Set b)
           (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b V_1 P_1 (
  @Prod.fst Nat Nat (Nat.unpair s)))
           (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b V_1 P_1 (
  @Prod.snd Nat Nat (Nat.unpair s)))))
    (heq1p : Nat.Primrec eq1),
    @DomainTheory.NeighbourhoodSystems.ComputableMap
      (Sum (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1) a) b
      (@DomainTheory.NeighbourhoodSystems.prod (@DomainTheory.NeighbourhoodSystems.
  ApproximableMap a b V_0 V_1) a
        (@DomainTheory.NeighbourhoodSystems.funSpace a b V_0 V_1) V_0)
      V_1
      (@DomainTheory.NeighbourhoodSystems.prodPresentation
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1) a
        (@DomainTheory.NeighbourhoodSystems.funSpace a b V_0 V_1) V_0
        (@DomainTheory.NeighbourhoodSystems.funPresentation a b V_0 V_1 P_0 P_1 gN
  incl0 incl1 eq1 hgN hgNp hincl0 hincl0p
          hincl1 hincl1p heq1 heq1p)
        P_0)
      P_1 (@DomainTheory.NeighbourhoodSystems.evalMap a b V_0 V_1)))
(forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem a}
  {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
  (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0)
  (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1) (gN incl0
```

```
  incl1 eq1 : Nat  ->  Nat)
  (hgN :
    forall  (c : Nat),
      Iff (@Eq Nat (gN c) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))))
        (@Set.Nonempty (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0
  V_1)
          (@DomainTheory.NeighbourhoodSystems.stepFun a b V_0 V_1
            (@DomainTheory.NeighbourhoodSystems.funListOf a b V_0 V_1 P_0 P_1 (
  Domain.Recursive.decodeList c)))))
  (hgNp : Nat.Primrec gN)
  (hincl0 :
    forall  (s : Nat),
      Iff (@Eq Nat (incl0 s) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)
  )))
        (@LE.le (Set a) (@Set.instLE a)
          (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V_0 P_0 (
  @Prod.fst Nat Nat (Nat.unpair s)))
          (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V_0 P_0 (
  @Prod.snd Nat Nat (Nat.unpair s)))))
  (hincl0p : Nat.Primrec incl0)
  (hincl1 :
    forall  (s : Nat),
      Iff (@Eq Nat (incl1 s) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)
  )))
        (@LE.le (Set b) (@Set.instLE b)
          (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b V_1 P_1 (
  @Prod.fst Nat Nat (Nat.unpair s)))
          (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b V_1 P_1 (
  @Prod.snd Nat Nat (Nat.unpair s)))))
  (hincl1p : Nat.Primrec incl1)
  (heq1 :
    forall  (s : Nat),
      Iff (@Eq Nat (eq1 s) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))
  )
        (@Eq (Set b)
          (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b V_1 P_1 (
  @Prod.fst Nat Nat (Nat.unpair s)))
          (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b V_1 P_1 (
  @Prod.snd Nat Nat (Nat.unpair s)))))
  (heq1p : Nat.Primrec eq1)
  (phi :
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1)
      (@DomainTheory.NeighbourhoodSystems.funSpace a b V_0 V_1)),
  Iff
    (@DomainTheory.NeighbourhoodSystems.ComputableElement
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1)
      (@DomainTheory.NeighbourhoodSystems.funSpace a b V_0 V_1)
      (@DomainTheory.NeighbourhoodSystems.funPresentation a b V_0 V_1 P_0 P_1 gN
  incl0 incl1 eq1 hgN hgNp hincl0 hincl0p
        hincl1 hincl1p heq1 heq1p)
      phi)
    (@DomainTheory.NeighbourhoodSystems.ComputableMap a b V_0 V_1 P_0 P_1
      (@DomainTheory.NeighbourhoodSystems.toApproxMap a b V_0 V_1 phi))) :=
@And.intro
(And
```

```
(And
  (And
    (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
      {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
      (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0)
      (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1) (gN
: Nat  ->  Nat),
      Nat.Primrec gN  ->
        (forall  (c : Nat),
           Iff (@Eq Nat (gN c) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (
nat_lit 1))))
             (@Set.Nonempty (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
 b V_0 V_1)
               (@DomainTheory.NeighbourhoodSystems.stepFun a b V_0 V_1
                 (@DomainTheory.NeighbourhoodSystems.funListOf a b V_0 V_1 P_0
P_1
                   (Domain.Recursive.decodeList c)))))  ->
          forall
            (phi :
              @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
                (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1)
                (@DomainTheory.NeighbourhoodSystems.funSpace a b V_0 V_1)),
            Iff
              (Domain.Recursive.REPred fun (c : Nat) =>
                @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem
                  (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1)
                  (@DomainTheory.NeighbourhoodSystems.funSpace a b V_0 V_1) phi
                  (@DomainTheory.NeighbourhoodSystems.Xenum a b V_0 V_1 P_0 P_1 gN
 c))
              (@DomainTheory.NeighbourhoodSystems.ComputableMap a b V_0 V_1 P_0
P_1
                (@DomainTheory.NeighbourhoodSystems.toApproxMap a b V_0 V_1 phi)))
    (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
      {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b},
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven a
V_0  ->
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven b
 V_1  ->
          @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1)
            (@DomainTheory.NeighbourhoodSystems.funSpace a b V_0 V_1)))
  (forall  {a : Type u_1} {b : Type u_2} {? : Type u_3} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
    {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
    (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0)
    (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1)
    {V_2 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem ?}
    (P_2 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation ? V_2) (gN
incl0 incl1 eq1 : Nat  ->  Nat)
    (hgN :
      forall  (c : Nat),
        Iff (@Eq Nat (gN c) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit
1))))
          (@Set.Nonempty (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ?
```

```
V_1 V_2)
             (@DomainTheory.NeighbourhoodSystems.stepFun b ? V_1 V_2
               (@DomainTheory.NeighbourhoodSystems.funListOf b ? V_1 V_2 P_1 P_2 (
Domain.Recursive.decodeList c)))))
     (hgNp : Nat.Primrec gN)
     (hincl0 :
       forall  (s : Nat),
         Iff (@Eq Nat (incl0 s) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (
nat_lit 1))))
           (@LE.le (Set b) (@Set.instLE b)
             (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b V_1 P_1
 (@Prod.fst Nat Nat (Nat.unpair s)))
             (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b V_1 P_1
               (@Prod.snd Nat Nat (Nat.unpair s)))))
     (hincl0p : Nat.Primrec incl0)
     (hincl1 :
       forall  (s : Nat),
         Iff (@Eq Nat (incl1 s) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (
nat_lit 1))))
           (@LE.le (Set ?) (@Set.instLE ?)
             (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X ? V_2 P_2
 (@Prod.fst Nat Nat (Nat.unpair s)))
             (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X ? V_2 P_2
               (@Prod.snd Nat Nat (Nat.unpair s)))))
     (hincl1p : Nat.Primrec incl1)
     (heq1 :
       forall  (s : Nat),
         Iff (@Eq Nat (eq1 s) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit
1))))
           (@Eq (Set ?)
             (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X ? V_2 P_2
 (@Prod.fst Nat Nat (Nat.unpair s)))
             (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X ? V_2 P_2
               (@Prod.snd Nat Nat (Nat.unpair s)))))
     (heq1p : Nat.Primrec eq1)
     {g :
       @DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
         (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2},
     @DomainTheory.NeighbourhoodSystems.ComputableMap (Sum a b) ?
         (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2
         (@DomainTheory.NeighbourhoodSystems.prodPresentation a b V_0 V_1 P_0 P_1)
P_2 g  ->
       @DomainTheory.NeighbourhoodSystems.ComputableMap a
         (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
         (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2) P_0
         (@DomainTheory.NeighbourhoodSystems.funPresentation b ? V_1 V_2 P_1 P_2 gN
 incl0 incl1 eq1 hgN hgNp hincl0
           hincl0p hincl1 hincl1p heq1 heq1p)
         (@DomainTheory.NeighbourhoodSystems.curry a b ? V_0 V_1 V_2 g)))
 (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
   {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
   (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0)
   (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1) (gN
incl0 incl1 eq1 : Nat  ->  Nat)
   (hgN :
```

```
     forall  (c : Nat),
       Iff (@Eq Nat (gN c) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))
 ))
         (@Set.Nonempty (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0
  V_1)
           (@DomainTheory.NeighbourhoodSystems.stepFun a b V_0 V_1
             (@DomainTheory.NeighbourhoodSystems.funListOf a b V_0 V_1 P_0 P_1 (
 Domain.Recursive.decodeList c)))))
   (hgNp : Nat.Primrec gN)
   (hincl0 :
     forall  (s : Nat),
       Iff (@Eq Nat (incl0 s) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit
 1))))
         (@LE.le (Set a) (@Set.instLE a)
           (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V_0 P_0 (
 @Prod.fst Nat Nat (Nat.unpair s)))
           (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V_0 P_0 (
 @Prod.snd Nat Nat (Nat.unpair s)))))
   (hincl0p : Nat.Primrec incl0)
   (hincl1 :
     forall  (s : Nat),
       Iff (@Eq Nat (incl1 s) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit
 1))))
         (@LE.le (Set b) (@Set.instLE b)
           (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b V_1 P_1 (
 @Prod.fst Nat Nat (Nat.unpair s)))
           (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b V_1 P_1 (
 @Prod.snd Nat Nat (Nat.unpair s)))))
   (hincl1p : Nat.Primrec incl1)
   (heq1 :
     forall  (s : Nat),
       Iff (@Eq Nat (eq1 s) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)
 )))
         (@Eq (Set b)
           (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b V_1 P_1 (
 @Prod.fst Nat Nat (Nat.unpair s)))
           (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b V_1 P_1 (
 @Prod.snd Nat Nat (Nat.unpair s)))))
   (heq1p : Nat.Primrec eq1),
   @DomainTheory.NeighbourhoodSystems.ComputableMap
     (Sum (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1) a) b
     (@DomainTheory.NeighbourhoodSystems.prod (@DomainTheory.NeighbourhoodSystems.
 ApproximableMap a b V_0 V_1) a
       (@DomainTheory.NeighbourhoodSystems.funSpace a b V_0 V_1) V_0)
     V_1
     (@DomainTheory.NeighbourhoodSystems.prodPresentation
       (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1) a
       (@DomainTheory.NeighbourhoodSystems.funSpace a b V_0 V_1) V_0
       (@DomainTheory.NeighbourhoodSystems.funPresentation a b V_0 V_1 P_0 P_1 gN
 incl0 incl1 eq1 hgN hgNp hincl0 hincl0p
         hincl1 hincl1p heq1 heq1p)
       P_0)
     P_1 (@DomainTheory.NeighbourhoodSystems.evalMap a b V_0 V_1)))
(forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem a}
 {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
```

```
(P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0)
(P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1) (gN incl0
incl1 eq1 : Nat  ->  Nat)
(hgN :
  forall  (c : Nat),
    Iff (@Eq Nat (gN c) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))))
      (@Set.Nonempty (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0
V_1)
        (@DomainTheory.NeighbourhoodSystems.stepFun a b V_0 V_1
          (@DomainTheory.NeighbourhoodSystems.funListOf a b V_0 V_1 P_0 P_1 (
Domain.Recursive.decodeList c)))))
(hgNp : Nat.Primrec gN)
(hincl0 :
  forall  (s : Nat),
    Iff (@Eq Nat (incl0 s) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)
)))
      (@LE.le (Set a) (@Set.instLE a)
        (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V_0 P_0 (
@Prod.fst Nat Nat (Nat.unpair s)))
        (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V_0 P_0 (
@Prod.snd Nat Nat (Nat.unpair s)))))
(hincl0p : Nat.Primrec incl0)
(hincl1 :
  forall  (s : Nat),
    Iff (@Eq Nat (incl1 s) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)
)))
      (@LE.le (Set b) (@Set.instLE b)
        (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b V_1 P_1 (
@Prod.fst Nat Nat (Nat.unpair s)))
        (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b V_1 P_1 (
@Prod.snd Nat Nat (Nat.unpair s)))))
(hincl1p : Nat.Primrec incl1)
(heq1 :
  forall  (s : Nat),
    Iff (@Eq Nat (eq1 s) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))
)
      (@Eq (Set b)
        (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b V_1 P_1 (
@Prod.fst Nat Nat (Nat.unpair s)))
        (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b V_1 P_1 (
@Prod.snd Nat Nat (Nat.unpair s)))))
(heq1p : Nat.Primrec eq1)
(phi :
  @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1)
    (@DomainTheory.NeighbourhoodSystems.funSpace a b V_0 V_1)),
Iff
  (@DomainTheory.NeighbourhoodSystems.ComputableElement
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1)
    (@DomainTheory.NeighbourhoodSystems.funSpace a b V_0 V_1)
    (@DomainTheory.NeighbourhoodSystems.funPresentation a b V_0 V_1 P_0 P_1 gN
incl0 incl1 eq1 hgN hgNp hincl0 hincl0p
      hincl1 hincl1p heq1 heq1p)
    phi)
  (@DomainTheory.NeighbourhoodSystems.ComputableMap a b V_0 V_1 P_0 P_1
    (@DomainTheory.NeighbourhoodSystems.toApproxMap a b V_0 V_1 phi)))
```

```
(@And.intro
  (And
    (And
      (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
 NeighbourhoodSystems.NeighborhoodSystem a}
       {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
       (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0)
       (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1) (gN
 : Nat  ->  Nat),
       Nat.Primrec gN  ->
         (forall  (c : Nat),
            Iff (@Eq Nat (gN c) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (
 nat_lit 1))))
              (@Set.Nonempty (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
  b V_0 V_1)
                (@DomainTheory.NeighbourhoodSystems.stepFun a b V_0 V_1
                  (@DomainTheory.NeighbourhoodSystems.funListOf a b V_0 V_1 P_0
 P_1
                    (Domain.Recursive.decodeList c)))))  ->
           forall
             (phi :
               @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
                 (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1)
                 (@DomainTheory.NeighbourhoodSystems.funSpace a b V_0 V_1)),
             Iff
               (Domain.Recursive.REPred fun (c : Nat) =>
                 @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem
                   (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1)
                   (@DomainTheory.NeighbourhoodSystems.funSpace a b V_0 V_1) phi
                   (@DomainTheory.NeighbourhoodSystems.Xenum a b V_0 V_1 P_0 P_1 gN
  c))
               (@DomainTheory.NeighbourhoodSystems.ComputableMap a b V_0 V_1 P_0
 P_1
                 (@DomainTheory.NeighbourhoodSystems.toApproxMap a b V_0 V_1 phi)))
     (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
 NeighbourhoodSystems.NeighborhoodSystem a}
       {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b},
       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven a
 V_0  ->
         @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven b
  V_1  ->
           @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven
             (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1)
             (@DomainTheory.NeighbourhoodSystems.funSpace a b V_0 V_1)))
   (forall  {a : Type u_1} {b : Type u_2} {? : Type u_3} {V_0 : DomainTheory.
 NeighbourhoodSystems.NeighborhoodSystem a}
     {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
     (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0)
     (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1)
     {V_2 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem ?}
     (P_2 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation ? V_2) (gN
 incl0 incl1 eq1 : Nat  ->  Nat)
     (hgN :
       forall  (c : Nat),
         Iff (@Eq Nat (gN c) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit
 1))))
```

```
          (@Set.Nonempty (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ?
V_1 V_2)
            (@DomainTheory.NeighbourhoodSystems.stepFun b ? V_1 V_2
              (@DomainTheory.NeighbourhoodSystems.funListOf b ? V_1 V_2 P_1 P_2 (
Domain.Recursive.decodeList c)))))
     (hgNp : Nat.Primrec gN)
     (hincl0 :
       forall  (s : Nat),
         Iff (@Eq Nat (incl0 s) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (
nat_lit 1))))
           (@LE.le (Set b) (@Set.instLE b)
             (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b V_1 P_1
 (@Prod.fst Nat Nat (Nat.unpair s)))
             (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b V_1 P_1
               (@Prod.snd Nat Nat (Nat.unpair s)))))
     (hincl0p : Nat.Primrec incl0)
     (hincl1 :
       forall  (s : Nat),
         Iff (@Eq Nat (incl1 s) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (
nat_lit 1))))
           (@LE.le (Set ?) (@Set.instLE ?)
             (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X ? V_2 P_2
 (@Prod.fst Nat Nat (Nat.unpair s)))
             (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X ? V_2 P_2
               (@Prod.snd Nat Nat (Nat.unpair s)))))
     (hincl1p : Nat.Primrec incl1)
     (heq1 :
       forall  (s : Nat),
         Iff (@Eq Nat (eq1 s) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit
1))))
           (@Eq (Set ?)
             (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X ? V_2 P_2
 (@Prod.fst Nat Nat (Nat.unpair s)))
             (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X ? V_2 P_2
               (@Prod.snd Nat Nat (Nat.unpair s)))))
     (heq1p : Nat.Primrec eq1)
     {g :
       @DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
         (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2},
     @DomainTheory.NeighbourhoodSystems.ComputableMap (Sum a b) ?
         (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2
         (@DomainTheory.NeighbourhoodSystems.prodPresentation a b V_0 V_1 P_0 P_1)
P_2 g  ->
       @DomainTheory.NeighbourhoodSystems.ComputableMap a
         (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
         (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2) P_0
         (@DomainTheory.NeighbourhoodSystems.funPresentation b ? V_1 V_2 P_1 P_2 gN
 incl0 incl1 eq1 hgN hgNp hincl0
           hincl0p hincl1 hincl1p heq1 heq1p)
         (@DomainTheory.NeighbourhoodSystems.curry a b ? V_0 V_1 V_2 g)))
 (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
   {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
   (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0)
   (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1) (gN
incl0 incl1 eq1 : Nat  ->  Nat)
```

```
   (hgN :
     forall  (c : Nat),
       Iff (@Eq Nat (gN c) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))
))
         (@Set.Nonempty (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0
 V_1)
           (@DomainTheory.NeighbourhoodSystems.stepFun a b V_0 V_1
             (@DomainTheory.NeighbourhoodSystems.funListOf a b V_0 V_1 P_0 P_1 (
Domain.Recursive.decodeList c)))))
   (hgNp : Nat.Primrec gN)
   (hincl0 :
     forall  (s : Nat),
       Iff (@Eq Nat (incl0 s) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit
1))))
         (@LE.le (Set a) (@Set.instLE a)
           (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V_0 P_0 (
@Prod.fst Nat Nat (Nat.unpair s)))
           (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V_0 P_0 (
@Prod.snd Nat Nat (Nat.unpair s)))))
   (hincl0p : Nat.Primrec incl0)
   (hincl1 :
     forall  (s : Nat),
       Iff (@Eq Nat (incl1 s) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit
1))))
         (@LE.le (Set b) (@Set.instLE b)
           (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b V_1 P_1 (
@Prod.fst Nat Nat (Nat.unpair s)))
           (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b V_1 P_1 (
@Prod.snd Nat Nat (Nat.unpair s)))))
   (hincl1p : Nat.Primrec incl1)
   (heq1 :
     forall  (s : Nat),
       Iff (@Eq Nat (eq1 s) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)
)))
         (@Eq (Set b)
           (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b V_1 P_1 (
@Prod.fst Nat Nat (Nat.unpair s)))
           (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b V_1 P_1 (
@Prod.snd Nat Nat (Nat.unpair s)))))
   (heq1p : Nat.Primrec eq1),
   @DomainTheory.NeighbourhoodSystems.ComputableMap
     (Sum (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1) a) b
     (@DomainTheory.NeighbourhoodSystems.prod (@DomainTheory.NeighbourhoodSystems.
ApproximableMap a b V_0 V_1) a
       (@DomainTheory.NeighbourhoodSystems.funSpace a b V_0 V_1) V_0)
     V_1
     (@DomainTheory.NeighbourhoodSystems.prodPresentation
       (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1) a
       (@DomainTheory.NeighbourhoodSystems.funSpace a b V_0 V_1) V_0
       (@DomainTheory.NeighbourhoodSystems.funPresentation a b V_0 V_1 P_0 P_1 gN
incl0 incl1 eq1 hgN hgNp hincl0 hincl0p
         hincl1 hincl1p heq1 heq1p)
       P_0)
     P_1 (@DomainTheory.NeighbourhoodSystems.evalMap a b V_0 V_1))
 (@And.intro
   (And
```

```
    (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
      {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
      (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0)
      (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1) (gN
: Nat  ->  Nat),
      Nat.Primrec gN  ->
        (forall  (c : Nat),
           Iff (@Eq Nat (gN c) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (
nat_lit 1))))
             (@Set.Nonempty (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
 b V_0 V_1)
               (@DomainTheory.NeighbourhoodSystems.stepFun a b V_0 V_1
                 (@DomainTheory.NeighbourhoodSystems.funListOf a b V_0 V_1 P_0
P_1
                   (Domain.Recursive.decodeList c)))))  ->
          forall
            (phi :
              @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
                (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1)
                (@DomainTheory.NeighbourhoodSystems.funSpace a b V_0 V_1)),
            Iff
              (Domain.Recursive.REPred fun (c : Nat) =>
                @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem
                  (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1)
                  (@DomainTheory.NeighbourhoodSystems.funSpace a b V_0 V_1) phi
                  (@DomainTheory.NeighbourhoodSystems.Xenum a b V_0 V_1 P_0 P_1 gN
 c))
              (@DomainTheory.NeighbourhoodSystems.ComputableMap a b V_0 V_1 P_0
P_1


                (@DomainTheory.NeighbourhoodSystems.toApproxMap a b V_0 V_1 phi)))
    (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
      {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b},
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven a
V_0  ->
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven b
 V_1  ->
          @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1)
            (@DomainTheory.NeighbourhoodSystems.funSpace a b V_0 V_1)))
  (forall  {a : Type u_1} {b : Type u_2} {? : Type u_3} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
    {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
    (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0)
    (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1)
    {V_2 : DomainTheory.NeighbourhoodSystem ?}
    (P_2 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation ? V_2) (gN
incl0 incl1 eq1 : Nat  ->  Nat)
    (hgN :
      forall  (c : Nat),
        Iff (@Eq Nat (gN c) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit
1))))
          (@Set.Nonempty (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ?
V_1 V_2)
```

```
            (@DomainTheory.NeighbourhoodSystems.stepFun b ? V_1 V_2
              (@DomainTheory.NeighbourhoodSystems.funListOf b ? V_1 V_2 P_1 P_2 (
Domain.Recursive.decodeList c)))))
     (hgNp : Nat.Primrec gN)
     (hincl0 :
       forall  (s : Nat),
         Iff (@Eq Nat (incl0 s) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (
nat_lit 1))))
           (@LE.le (Set b) (@Set.instLE b)
             (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b V_1 P_1
 (@Prod.fst Nat Nat (Nat.unpair s)))
             (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b V_1 P_1
               (@Prod.snd Nat Nat (Nat.unpair s)))))
     (hincl0p : Nat.Primrec incl0)
     (hincl1 :
       forall  (s : Nat),
         Iff (@Eq Nat (incl1 s) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (
nat_lit 1))))
           (@LE.le (Set ?) (@Set.instLE ?)
             (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X ? V_2 P_2
 (@Prod.fst Nat Nat (Nat.unpair s)))
             (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X ? V_2 P_2
               (@Prod.snd Nat Nat (Nat.unpair s)))))
     (hincl1p : Nat.Primrec incl1)
     (heq1 :
       forall  (s : Nat),
         Iff (@Eq Nat (eq1 s) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit
1))))
           (@Eq (Set ?)
             (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X ? V_2 P_2
 (@Prod.fst Nat Nat (Nat.unpair s)))
             (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X ? V_2 P_2
               (@Prod.snd Nat Nat (Nat.unpair s)))))
     (heq1p : Nat.Primrec eq1)
     {g :
       @DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
         (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2},
     @DomainTheory.NeighbourhoodSystems.ComputableMap (Sum a b) ?
         (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2
         (@DomainTheory.NeighbourhoodSystems.prodPresentation a b V_0 V_1 P_0 P_1)
P_2 g  ->
       @DomainTheory.NeighbourhoodSystems.ComputableMap a
         (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
         (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2) P_0
         (@DomainTheory.NeighbourhoodSystems.funPresentation b ? V_1 V_2 P_1 P_2 gN
 incl0 incl1 eq1 hgN hgNp hincl0
           hincl0p hincl1 hincl1p heq1 heq1p)
         (@DomainTheory.NeighbourhoodSystems.curry a b ? V_0 V_1 V_2 g))
  (@And.intro
     (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
       {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
       (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0)
       (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1) (gN
: Nat  ->  Nat),
       Nat.Primrec gN  ->
```

```
          (forall  (c : Nat),
             Iff (@Eq Nat (gN c) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (
    nat_lit 1))))
               (@Set.Nonempty (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
     b V_0 V_1)
                 (@DomainTheory.NeighbourhoodSystems.stepFun a b V_0 V_1
                   (@DomainTheory.NeighbourhoodSystems.funListOf a b V_0 V_1 P_0
    P_1
                     (Domain.Recursive.decodeList c)))))  ->
           forall
             (phi :
               @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
                 (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1)
                 (@DomainTheory.NeighbourhoodSystems.funSpace a b V_0 V_1)),
             Iff
               (Domain.Recursive.REPred fun (c : Nat) =>
                 @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem
                   (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1)
                   (@DomainTheory.NeighbourhoodSystems.funSpace a b V_0 V_1) phi
                   (@DomainTheory.NeighbourhoodSystems.Xenum a b V_0 V_1 P_0 P_1 gN
     c))
               (@DomainTheory.NeighbourhoodSystems.ComputableMap a b V_0 V_1 P_0
    P_1
                 (@DomainTheory.NeighbourhoodSystems.toApproxMap a b V_0 V_1 phi)))
       (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.
    NeighbourhoodSystems.NeighborhoodSystem a}
         {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b},
         @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven a
    V_0  ->
           @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven b
     V_1  ->
             @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven
               (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V_0 V_1)
               (@DomainTheory.NeighbourhoodSystems.funSpace a b V_0 V_1))
       @DomainTheory.NeighbourhoodSystems.Xenum_isComputableElement_iff
       @DomainTheory.NeighbourhoodSystems.funSpace_isEffectivelyGiven)
     @DomainTheory.NeighbourhoodSystems.curry_isComputable)
    @DomainTheory.NeighbourhoodSystems.evalMap_isComputable)
  @DomainTheory.NeighbourhoodSystems.isComputableElement_funPresentation_iff
end

section
open NeighborhoodSystem Domain.Recursive ApproximableMap
variable {a b ? : Type*} {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
  {V_2 : NeighborhoodSystem ?}

/-- The code of the one-entry function-space neighbourhood `[<X_{e.1}, X_{e'.1}>, X_{e
    '.2}]` of
`(?_0 x ?_1 -> ?_2)` (a `Xenum`-singleton over the product presentation). -/
def curryStepCode (a e' : Nat) : Nat :=
  Nat.pair (Nat.pair (Nat.pair a e'.unpair.1) e'.unpair.2) 0 + 1

/-- `curryIso V_0 V_1 V_2` sends a filter `x` to the filter of the curried least map (
    by definition). -/
theorem curryIso_apply (x : (funSpace (prod V_0 V_1) V_2).Element) :
    curryIso V_0 V_1 V_2 x = toFilter (curry (toApproxMap x)) := rfl
```

```
theorem primrec_curryStepCode {a e : Nat  ->  Nat} (ha : Nat.Primrec a) (he : Nat.
    Primrec e) :
    Nat.Primrec (fun w => curryStepCode (a w) (e w)) := by
  have h1 : Nat.Primrec (fun w => (e w).unpair.1) := Nat.Primrec.left.comp he
  have h2 : Nat.Primrec (fun w => (e w).unpair.2) := Nat.Primrec.right.comp he
  refine (Nat.Primrec.succ.comp (((ha.pair h1).pair h2).pair (Nat.Primrec.const 0))).
    of_eq
    (fun w => ?_)
  rfl

end
end DomainTheory.NeighbourhoodSystems
end
```

3.8.7 Theorem 7.6 — fix is computable

THEOREM 7.6. For any effectively given domain $\mathcal{D}$, the combinator fix : $(\mathcal{D} \to \mathcal{D}) \to \mathcal{D}$ is computable.

Proof : Referring back to the proof of Theorem 4.2 and thinking of

$$\mathcal{D} = \{X_n \mid n \in \mathbb{N}\}$$

as effectively given, fix as a relation comes down to

$$\bigcap_{i<q}[X_{n_i}, X_{m_i}] \text{ fix } X_\ell \text{ iff for some finite sequence}$$

$$\Delta = X_{k_0}, \ldots, X_{k_p} = X_\ell,$$

we have, for each $j < p$,

$$\bigcap\{X_{m_i} \mid X_{k_j} \subseteq X_{n_i}\} \subseteq X_{k_{j+1}}.$$

Inside the "for some finite sequence" all the checks are decidable by assumption on D. But the existential quantification of a decidable predicate always gives a recursively enumerable predicate. (And, as there is no implied bound on the size of the finite sequence we are looking for, this really *is* an enumerable set and not generally a recursive set.) □

The major consequence of what we have done up to this point concerns typed λ-calculus. Any expression involving only *effectively given types* and, perhaps, some *basic computable constants* using only the $\lambda, !$-notation defines a computable function of its free variables. And such functions applied to computable arguments give computable values. And such functions have computable least fixed points. Etc., etc. In a definite sense then we have in the "metalanguage", as people say, a quite precise and fully *mathematical programming language* for defining computable operators. It is not a machine implemented language, but it is a mathematically well-defined and easy-to-use language. And when we combine the usual type-definition facility together with *domain equations*, we have an especially powerful language.

Theorem_7_6.lean

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Proposition_7_12
```

```
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Exercise_7_18
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Exercise_7_15

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Exercise_7_24
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive ApproximableMap
section
variable {a : Type*} {V : NeighborhoodSystem a}
variable (P : ComputablePresentation V)
/- concordance: source L4411 THEOREM 7.6 fix is computable -/
/-- **Theorem 7.6 (Scott 1981, PRG-19) -- `fix` is computable (choice-free).** For any
    effectively
given domain `?` (here a computable presentation `P` of `V`, together with the function
    -space
consistency/inclusion/equality chars of Theorem 7.5), the combinator `fix : (?  ->  ?)
     ->  ?` is
computable: its neighbourhood relation is the recursively-enumerable existential `
    exists  chain` over the
decidable least-map step relation. -/
theorem fixMap_isComputable
    (gN incl eq : Nat  ->  Nat)
    (hgN : forall  c, gN c = 1  <->  (stepFun (funListOf P P (decodeList c))
      : Set (ApproximableMap V V)).Nonempty) (hgNp : Nat.Primrec gN)
    (hincl : forall  s, incl s = 1  <->  P.X s.unpair.1  subseteq  P.X s.unpair.2) (
    hinclp : Nat.Primrec incl)
    (heq : forall  s, eq s = 1  <->  P.X s.unpair.1 = P.X s.unpair.2) (heqp : Nat.
    Primrec eq) :
    ComputableMap
      (funPresentation P P gN incl incl eq hgN hgNp hincl hinclp hincl hinclp heq heqp
    ) P
      (fixMap V) := by
  constructor
  obtain <fincl, hfinclp, hfincls> :=
    (funPresentation P P gN incl incl eq hgN hgNp hincl hinclp hincl hinclp heq heqp).
    incl_computable
  have hfincl : forall  s, fincl s = 1  <->  Xenum P P gN s.unpair.1  subseteq  Xenum
    P P gN s.unpair.2 :=
    fun s => (hfincls s).symm
  show REPred_2 (fun c ? => (fixMap V).rel (Xenum P P gN c) (P.X ?))
  refine <fun w =>
      (fixChainChar P fincl w.unpair.2.unpair.1 w.unpair.1).unpair.2 = 1
       /\  incl (Nat.pair (fixChainChar P fincl w.unpair.2.unpair.1 w.unpair.1).unpair
    .1
          w.unpair.2.unpair.2) = 1,
    ?_, ?_>
  * -- the search predicate is recursively decidable.
    have hcc : Nat.Primrec (fun w => fixChainChar P fincl w.unpair.2.unpair.1 w.unpair
    .1) :=
```

```
      ((primrec_fixChainChar P fincl hfinclp).comp
        ((Nat.Primrec.left.comp Nat.Primrec.right).pair Nat.Primrec.left)).of_eq (fun
    w => by
          simp only [unpair_pair_fst, unpair_pair_snd])
    have hA : Nat.Primrec (fun w =>
        (fixChainChar P fincl w.unpair.2.unpair.1 w.unpair.1).unpair.2) :=
      Nat.Primrec.right.comp hcc
    have hB : Nat.Primrec (fun w => incl (Nat.pair
        (fixChainChar P fincl w.unpair.2.unpair.1 w.unpair.1).unpair.1 w.unpair.2.
    unpair.2)) :=
      hinclp.comp ((Nat.Primrec.left.comp hcc).pair (Nat.Primrec.right.comp Nat.
    Primrec.right))
    exact (RecDecidable.natEq hA (Nat.Primrec.const 1)).and
      (RecDecidable.natEq hB (Nat.Primrec.const 1))
  * -- the search predicate enumerates `fix`'s neighbourhood relation.
    intro t
    show (fixMap V).rel (Xenum P P gN t.unpair.1) (P.X t.unpair.2)  <->  _
    rw [fixMap_rel_iff P gN hgN t.unpair.1 t.unpair.2]
    constructor
    * rintro <full, hsteps, hlast>
      refine <encodeList full, ?_>
      simp only [unpair_pair_fst, unpair_pair_snd]
      have hdec : decodeList (encodeList full) = full := decodeList_encodeList full
      obtain <hfst, hsnd> := fixChainChar_spec P fincl t.unpair.1 (encodeList full)
      rw [hdec] at hfst hsnd
      refine <hsnd.mpr
        ((chainDec_iff_gStepsOK P gN hgN fincl hfincl t.unpair.1 full P.masterIdx).mpr
     hsteps), ?_>
      rw [hfst, hincl, unpair_pair_fst, unpair_pair_snd]
      exact hlast
    * rintro <i, hi>
      simp only [unpair_pair_fst, unpair_pair_snd] at hi
      obtain <hi1, hi2> := hi
      obtain <hfst, hsnd> := fixChainChar_spec P fincl t.unpair.1 i
      refine <decodeList i, ?_, ?_>
      * exact (chainDec_iff_gStepsOK P gN hgN fincl hfincl t.unpair.1 (decodeList i)
          P.masterIdx).mp (hsnd.mp hi1)
      * rw [ <-  hfst]
        have hi2' := (hincl (Nat.pair (fixChainChar P fincl t.unpair.1 i).unpair.1
          t.unpair.2)).mp hi2
        rwa [unpair_pair_fst, unpair_pair_snd] at hi2'
end

section
set_option linter.unusedVariables false

theorem theorem_76.{u_1} : forall  {a : Type u_1} {V : DomainTheory.
    NeighbourhoodSystems.NeighborhoodSystem a}
  (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V) (gN incl eq :
    Nat  ->  Nat)
  (hgN :
    forall  (c : Nat),
      Iff (@Eq Nat (gN c) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))))
        (@Set.Nonempty (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a V V)
          (@DomainTheory.NeighbourhoodSystems.stepFun a a V V
            (@DomainTheory.NeighbourhoodSystems.funListOf a a V V P P (Domain.
```

```
    Recursive.decodeList c)))))
  (hgNp : Nat.Primrec gN)
  (hincl :
    forall  (s : Nat),
      Iff (@Eq Nat (incl s) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))))
        (@LE.le (Set a) (@Set.instLE a)
          (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V P (@Prod.
    fst Nat Nat (Nat.unpair s)))
          (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V P (@Prod.
    snd Nat Nat (Nat.unpair s)))))
  (hinclp : Nat.Primrec incl)
  (heq :
    forall  (s : Nat),
      Iff (@Eq Nat (eq s) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))))
        (@Eq (Set a)
          (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V P (@Prod.
    fst Nat Nat (Nat.unpair s)))
          (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V P (@Prod.
    snd Nat Nat (Nat.unpair s)))))
  (heqp : Nat.Primrec eq),
  @DomainTheory.NeighbourhoodSystems.ComputableMap (@DomainTheory.NeighbourhoodSystems
    .ApproximableMap a a V V) a
    (@DomainTheory.NeighbourhoodSystems.funSpace a a V V) V
    (@DomainTheory.NeighbourhoodSystems.funPresentation a a V V P P gN incl incl eq
    hgN hgNp hincl hinclp hincl hinclp
      heq heqp)
    P (@DomainTheory.NeighbourhoodSystems.fixMap a V) :=
  @DomainTheory.NeighbourhoodSystems.fixMap_isComputable
end

namespace Exercise722
open Set Domain.Recursive

section
def boolNat (b : Bool) : Nat := if b then 1 else 0
end

section
theorem boolNat_eq_one_iff (b : Bool) : boolNat b = 1  <->  b := by
  cases b <;> simp [boolNat]
end

section
theorem boolNat_zero_one (b : Bool) : boolNat b = 0  \/  boolNat b = 1 := by
  cases b <;> simp [boolNat]
end

section
def encodeListBool (sigma : List Bool) : Nat := encodeList (sigma.map boolNat)
end

section
/-- Tag-based Godel code for an `SExpr` (tags `0..3` for the four constructors). -/
def SExpr.encode : SExpr  ->  Nat
  | .sigma => Nat.pair 0 0
  | .single sigma => Nat.pair 1 (encodeListBool sigma)
```

```
  | .cat a b => Nat.pair 2 (Nat.pair (encode a) (encode b))
  | .cap a b => Nat.pair 3 (Nat.pair (encode a) (encode b))
end

end Exercise722
namespace Exercise718
open NeighborhoodSystem Domain.Recursive ApproximableMap

section
variable {a b : Type*}
variable {V : NeighborhoodSystem a}

/-- The **double-indexed fiber** of a `(D^ inf )^ inf `-set: `fiber2 S i j = fiber (
    fiber S i) j`, the tokens
`a` with `(i, (j, a))  in  S`. -/
def fiber2 (S : Set (Nat  x  (Nat  x  a))) (i j : Nat) : Set a := fiber (fiber S i) j

@[simp] theorem mem_fiber2 {S : Set (Nat  x  (Nat  x  a))} {i j : Nat} {a : a} :
    a  in  fiber2 S i j  <->  (i, (j, a))  in  S := Iff.rfl

theorem fiber2_inter (S S' : Set (Nat  x  (Nat  x  a))) (i j : Nat) :
    fiber2 (S  I  S') i j = fiber2 S i j  I  fiber2 S' i j := rfl

theorem fiber2_master (i j : Nat) :
    fiber2 ((iterSys (iterSys V)).master) i j = V.master := by
  show fiber (fiber ((iterSys (iterSys V)).master) i) j = V.master
  rw [fiber_iterSys_master, fiber_iterSys_master]

theorem fiber2_mono {S S' : Set (Nat  x  (Nat  x  a))} (h : S  subseteq  S') (i j :
    Nat) :
    fiber2 S i j  subseteq  fiber2 S' i j := fun _ ha => h ha

/-- `V.mem (fiber2 S i j)` for every `(D^ inf )^ inf `-neighbourhood `S`. -/
theorem mem_fiber2_of_mem {S : Set (Nat  x  (Nat  x  a))} (hS : (iterSys (iterSys V)).
    mem S) (i j : Nat) :
    V.mem (fiber2 S i j) := (hS.1 i).1 j

end

end Exercise718
namespace Exercise722
open Set Domain.Recursive

section
def natBool (n : Nat) : Option Bool :=
  match n with | 0 => some false | 1 => some true | _ => none
end

section
def decodeListBool (n : Nat) : Option (List Bool) :=
  (decodeList n).mapM natBool
end

section
/-- Fuelled decoder: `fuel` bounds recursive unfoldings. -/
def decodeFuel (fuel : Nat) (n : Nat) : Option SExpr :=
```

```
  match fuel with
  | 0 => none
  | fuel + 1 =>
    match n.unpair.1 with
    | 0 => if n.unpair.2 = 0 then some .sigma else none
    | 1 => (decodeListBool n.unpair.2).map (.single *)
    | 2 =>
      match decodeFuel fuel n.unpair.2.unpair.1, decodeFuel fuel n.unpair.2.unpair.2
    with
      | some a, some b => some (.cat a b)
      | _, _ => none
    | 3 =>
      match decodeFuel fuel n.unpair.2.unpair.1, decodeFuel fuel n.unpair.2.unpair.2
    with
      | some a, some b => some (.cap a b)
      | _, _ => none
    | _ => none
end

section
/-- Decoder: `Nat.unpair n` supplies `(code, fuel)` with `fuel = sexprDepth e` on valid
     indices. -/
def SExpr.decode (n : Nat) : Option SExpr := decodeFuel (n.unpair.2 + 1) n.unpair.1
end

section
/-- **Definition 7.1 enumeration backbone.** Decode `n` to an `SExpr`; if the
    denotation is
non-empty (`decideNonemptyB`), use it; otherwise (or on junk codes) use `Sigma` so
    every index names a
member of the positive system `S`. -/
def SsysX (n : Nat) : Set (List Bool) :=
  match SExpr.decode n with
  | none => SsysX_default
  | some e =>
    if decideNonemptyB e then denote e else SsysX_default
end

section
theorem SsysX_mem (n : Nat) : InS (SsysX n) := by
  unfold SsysX
  cases hdec : SExpr.decode n with
  | none => simp; exact InS.univ
  | some e =>
    by_cases h : decideNonemptyB e = true
    * simp [h]
      exact InS_denote_of_nonempty ((decideNonemptyB_iff e).1 h)
    * simp [h]
      exact InS.univ
end

section
theorem decodeFuel_pair_cap_isSome_iff (fuel c : Nat) :
    (match decodeFuel fuel c.unpair.2.unpair.1, decodeFuel fuel c.unpair.2.unpair.2
    with
      | some a, some b => some (SExpr.cap a b) | _, _ => none).isSome = true  <->
```

```
      (decodeFuel fuel c.unpair.2.unpair.1).isSome = true  /\
        (decodeFuel fuel c.unpair.2.unpair.2).isSome = true := by
  cases decodeFuel fuel c.unpair.2.unpair.1 <;> cases decodeFuel fuel c.unpair.2.
    unpair.2 <;> simp
end

section
theorem decodeFuel_pair_cat_isSome_iff (fuel c : Nat) :
    (match decodeFuel fuel c.unpair.2.unpair.1, decodeFuel fuel c.unpair.2.unpair.2
    with
      | some a, some b => some (SExpr.cat a b) | _, _ => none).isSome = true  <->
      (decodeFuel fuel c.unpair.2.unpair.1).isSome = true  /\
        (decodeFuel fuel c.unpair.2.unpair.2).isSome = true := by
  cases decodeFuel fuel c.unpair.2.unpair.1 <;> cases decodeFuel fuel c.unpair.2.
    unpair.2 <;> simp
end

section
theorem decodeFuel_succ_cap (fuel c : Nat) (h : c.unpair.1 = 3) :
    decodeFuel (fuel + 1) c =
      match decodeFuel fuel c.unpair.2.unpair.1, decodeFuel fuel c.unpair.2.unpair.2
    with
      | some a, some b => some (.cap a b)
      | _, _ => none := by
  simp [decodeFuel, h]
end

section
theorem decodeFuel_succ_cat (fuel c : Nat) (h : c.unpair.1 = 2) :
    decodeFuel (fuel + 1) c =
      match decodeFuel fuel c.unpair.2.unpair.1, decodeFuel fuel c.unpair.2.unpair.2
    with
      | some a, some b => some (.cat a b)
      | _, _ => none := by
  simp [decodeFuel, h]
end

section
theorem decodeFuel_succ_junk (fuel c : Nat) (h : 4  <=  c.unpair.1) :
    decodeFuel (fuel + 1) c = none := by
  match tag : c.unpair.1 with
  | 0 | 1 | 2 | 3 => omega
  | t + 4 => simp [decodeFuel, tag]
end

section
theorem decodeFuel_succ_not_sigma (fuel c : Nat) (h : c.unpair.1 = 0) (hsigma : c.
    unpair.2  !=  0) :
    decodeFuel (fuel + 1) c = none := by simp [decodeFuel, h, hsigma]
end

section
theorem decodeFuel_succ_sigma (fuel c : Nat) (h : c.unpair.1 = 0) (hsigma : c.unpair.2
     = 0) :
    decodeFuel (fuel + 1) c = some .sigma := by simp [decodeFuel, h, hsigma]
end
```

```
section
theorem decodeFuel_succ_single (fuel c : Nat) (h : c.unpair.1 = 1) :
    decodeFuel (fuel + 1) c = (decodeListBool c.unpair.2).map (.single *) := by
  simp [decodeFuel, h]
end

section
theorem decodeListBool_encodeListBool (sigma : List Bool) :
    decodeListBool (encodeListBool sigma) = some sigma := by
  simp [decodeListBool, encodeListBool, decodeList_encodeList, List.mapM_map]
  induction sigma with
  | nil => rfl
  | cons b bs ih =>
    cases b <;> simp [boolNat, natBool, ih]
end

section
theorem natBinDigit_of_natBool_some (n : Nat) {b : Bool} (h : natBool n = some b) :
    n = 0  \/  n = 1 := by
  match n with
  | 0 => simp [natBool] at h; exact Or.inl rfl
  | 1 => simp [natBool] at h; exact Or.inr rfl
  | n + 2 => simp [natBool] at h
end

section
theorem natBool_isSome_iff (n : Nat) :
    (natBool n).isSome = true  <->  n = 0  \/  n = 1 := by
  match n with
  | 0 => simp [natBool]
  | 1 => simp [natBool]
  | n + 2 => simp [natBool]
end

section
theorem mapM_natBool_isSome_iff (l : List Nat) :
    (l.mapM natBool).isSome = true  <->  forall  x  in  l, x = 0  \/  x = 1 := by
  induction l with
  | nil => simp
  | cons x xs ih =>
    rw [List.mapM_cons]
    constructor
    * intro h
      cases hnx : natBool x with
      | none => simp [hnx] at h
      | some b =>
        cases htx : xs.mapM natBool with
        | none => simp [hnx, htx] at h
        | some ts =>
          have hxs' : (xs.mapM natBool).isSome = true := by simp [htx]
          intro y hy
          rcases List.mem_cons.mp hy with hEq | hyTail
          * rw [hEq]
            exact natBinDigit_of_natBool_some x hnx
          * exact ih.mp hxs' y hyTail
```

```
    * intro h
      have hx : (natBool x).isSome = true :=
        (natBool_isSome_iff x).2 (h x (List.mem_cons_self ..))
      have hxs : (xs.mapM natBool).isSome = true :=
        ih.mpr fun y hy => h y (List.mem_cons_of_mem x hy)
      cases hnx : natBool x with
      | none =>
        rcases h x (List.mem_cons_self ..) with h0 | h1
        * subst h0; simp [natBool] at hnx
        * subst h1; simp [natBool] at hnx
      | some b =>
        cases htx : xs.mapM natBool with
        | none => simp [htx] at hxs
        | some ts => simp
end

section
/-- **7.22i(b)1(d):** list decode succeeds iff every coded entry is a binary digit. -/
theorem decodeListBool_isSome_iff (n : Nat) :
    (decodeListBool n).isSome = true  <->  allBinDigitsChar n = 1 := by
  simp only [decodeListBool, mapM_natBool_isSome_iff, allBinDigitsChar_eq_one_iff]
end

section
/-- **7.22i(b)1(e):** shallow link between fuel-bounded char decode and `decodeFuel`.
    -/
theorem decodeFuelOkChar_eq_one_iff (fuel c : Nat) :
    decodeFuelOkChar fuel c = 1  <->  (decodeFuel fuel c).isSome = true := by
  induction fuel generalizing c with
  | zero =>
    simp [decodeFuelOkChar, decodeFuel]
  | succ fuel ih =>
    simp only [decodeFuelOkChar]
    rw [decodeFuelOkCharBody_eq]
    match tag : c.unpair.1 with
    | 0 =>
      rw [show decodeFuel (fuel + 1) c = if c.unpair.2 = 0 then some .sigma else none
    from by
        by_cases hsigma : c.unpair.2 = 0 <;> simp [decodeFuel, tag, hsigma]]
      simp [selectFn_isOne_one_sub_sigma]
    | 1 =>
      rw [decodeFuel_succ_single fuel c tag]
      simp [decodeListBool_isSome_iff, Option.isSome_map]
    | 2 =>
      simp
      rw [decodeFuel_succ_cat fuel c tag, mulBit_eq_one_iff]
      have ih1 := ih c.unpair.2.unpair.1
      have ih2 := ih c.unpair.2.unpair.2
      cases hL : decodeFuel fuel c.unpair.2.unpair.1 <;>
        cases hR : decodeFuel fuel c.unpair.2.unpair.2 <;>

        simp [hL, hR, ih1, ih2]
    | 3 =>
      simp
      rw [decodeFuel_succ_cap fuel c tag, mulBit_eq_one_iff]
      have ih1 := ih c.unpair.2.unpair.1
```

```
      have ih2 := ih c.unpair.2.unpair.2
      cases hL : decodeFuel fuel c.unpair.2.unpair.1 <;>
        cases hR : decodeFuel fuel c.unpair.2.unpair.2 <;>
        simp [hL, hR, ih1, ih2]
    | t + 4 =>
      rw [decodeFuel_succ_junk fuel c (by omega)]
      simp
end

end Exercise722
namespace Exercise718
open NeighborhoodSystem Domain.Recursive ApproximableMap

section
variable {a b : Type*}
variable {V : NeighborhoodSystem a}

/-- The `(D^ inf )^ inf `-set reindexing a `D^ inf `-neighbourhood `W`: `fiber2 (
    reindexF W) i j = fiber W (pair i j)`. -/
def reindexF (W : Set (Nat  x  a)) : Set (Nat  x  (Nat  x  a)) :=
  {p | p.2.2  in  fiber W (Nat.pair p.1 p.2.1)}

@[simp] theorem fiber2_reindexF (W : Set (Nat  x  a)) (i j : Nat) :
    fiber2 (reindexF W) i j = fiber W (Nat.pair i j) := rfl

/-- The `D^ inf `-set reindexing a `(D^ inf )^ inf `-neighbourhood `S`:
`fiber (reindexG S) k = fiber2 S (unpair k).1 (unpair k).2`. -/
def reindexG (S : Set (Nat  x  (Nat  x  a))) : Set (Nat  x  a) :=
  {p | p.2  in  fiber2 S (Nat.unpair p.1).1 (Nat.unpair p.1).2}

@[simp] theorem fiber_reindexG (S : Set (Nat  x  (Nat  x  a))) (k : Nat) :
    fiber (reindexG S) k = fiber2 S (Nat.unpair k).1 (Nat.unpair k).2 := rfl

theorem exists_inner_bound {S : Set (Nat  x  (Nat  x  a))} (hS : (iterSys (iterSys V))
    .mem S) :
    forall  n, exists  M, forall  i, i < n  ->  forall  j, M  <=  j  ->  fiber2 S i j
    = V.master := by
  intro n
  induction n with
  | zero => exact <0, fun i hi => absurd hi (Nat.not_lt_zero i)>
  | succ n ih =>
    obtain <M, hM> := ih
    obtain <Mn, hMn> := (hS.1 n).2
    refine <max M Mn, fun i hi j hj => ?_>
    rcases Nat.lt_succ_iff_lt_or_eq.mp hi with hlt | heq
    * exact hM i hlt j (le_trans (le_max_left _ _) hj)
    * subst heq
      exact hMn j (le_trans (le_max_right _ _) hj)

/-- `Nat.pair` is strictly monotone on strict componentwise order (local copy of the
`Proposition77` lemma to avoid the heavy import). -/
theorem pair_lt_pair_of_lt {x y i j : Nat} (hx : x < i) (hy : y < j) :
    Nat.pair x y < Nat.pair i j := by
  have hms : forall  z : Nat, (z + 1) * (z + 1) = z * z + 2 * z + 1 := fun z => by
    rw [Nat.succ_mul, Nat.mul_succ]; omega
  have hlt : Nat.pair x y < (max x y + 1) * (max x y + 1) := by
```

```
    unfold Nat.pair
    rcases lt_or_ge x y with h | h
    * rw [ite_eq_left h, show max x y = y by omega, hms]; omega
    * rw [ite_eq_right (by omega), show max x y = x by omega, hms]; omega
  have hle : max i j * max i j  <=  Nat.pair i j := by
    unfold Nat.pair
    rcases lt_or_ge i j with h | h
    * rw [ite_eq_left h, show max i j = j by omega]; omega
    * rw [ite_eq_right (by omega), show max i j = i by omega]; omega
  have h3 : max x y + 1  <=  max i j := by
    rcases le_total x y with h | h
    * rw [max_eq_right h]; have := le_max_right i j; omega
    * rw [max_eq_left h]; have := le_max_left i j; omega
  have h4 : (max x y + 1) * (max x y + 1)  <=  max i j * max i j := Nat.mul_le_mul h3
    h3
  omega

theorem reindexF_mem {W : Set (Nat  x  a)} (hW : (iterSys V).mem W) :
    (iterSys (iterSys V)).mem (reindexF W) := by
  obtain <hWf, Nw, hNw> := hW
  refine <fun i => <fun j => ?_, Nw, fun j hj => ?_>, Nw, fun i hi => ?_>
  * -- inner fibers are `V`-neighbourhoods
    show V.mem (fiber2 (reindexF W) i j)
    rw [fiber2_reindexF]; exact hWf (Nat.pair i j)
  * -- inner cofinite-`Delta` (per `i`)
    show fiber2 (reindexF W) i j = V.master
    rw [fiber2_reindexF]
    exact hNw (Nat.pair i j) (le_trans hj (le_pair_right i j))
  * -- outer cofinite-`Delta`
    apply eq_of_fiber_eq
    intro j
    show fiber2 (reindexF W) i j = fiber ((iterSys V).master) j
    rw [fiber2_reindexF, fiber_iterSys_master]
    exact hNw (Nat.pair i j) (le_trans hi (le_pair_left i j))

theorem reindexF_subset_iff {W : Set (Nat  x  a)} {S : Set (Nat  x  (Nat  x  a))} :
    reindexF W  subseteq  S  <->  forall  i j, fiber W (Nat.pair i j)  subseteq
    fiber2 S i j := by
  constructor
  * intro h i j
    have := fiber2_mono h i j
    rwa [fiber2_reindexF] at this
  * intro h
    refine subset_of_fiber_subset (fun i => subset_of_fiber_subset (fun j => ?_))
    show fiber2 (reindexF W) i j  subseteq  fiber2 S i j
    rw [fiber2_reindexF]; exact h i j

/-- **The forward map `F : D^ inf   ->  (D^ inf )^ inf `** of `D^ inf   ~=  (D^ inf )^
    inf ` (reindexing fibers along `Nat.pair`):
`W F S  <->  forall  i j, fiber W (pair i j)  subseteq  fiber2 S i j`. -/
def Fmap (V : NeighborhoodSystem a) : ApproximableMap (iterSys V) (iterSys (iterSys V)
    ) where
  rel W S := (iterSys V).mem W  /\  (iterSys (iterSys V)).mem S  /\
    forall  i j, fiber W (Nat.pair i j)  subseteq  fiber2 S i j
  rel_dom h := h.1
  rel_cod h := h.2.1
```

```
  master_rel := <(iterSys V).master_mem, (iterSys (iterSys V)).master_mem, fun i j =>
    by
    rw [fiber_iterSys_master, fiber2_master]>
  inter_right := by
    rintro W S S' <hW, hS, hf> <-, hS', hf'>
    refine <hW, (iterSys (iterSys V)).inter_mem hS hS' (reindexF_mem hW) ?_, fun i j
    => ?_>
    * exact Set.subset_inter (reindexF_subset_iff.mpr hf) (reindexF_subset_iff.mpr hf
    ')
    * rw [fiber2_inter]; exact Set.subset_inter (hf i j) (hf' i j)
  mono := by
    rintro W W' S S' <hW, hS, hf> hW'W hSS' hW' hS'
    exact <hW', hS', fun i j => ((fiber_mono hW'W _).trans (hf i j)).trans (
    fiber2_mono hSS' i j)>

@[simp] theorem Fmap_rel {W : Set (Nat  x  a)} {S : Set (Nat  x  (Nat  x  a))} :
    (Fmap V).rel W S  <->  (iterSys V).mem W  /\  (iterSys (iterSys V)).mem S  /\
      forall  i j, fiber W (Nat.pair i j)  subseteq  fiber2 S i j := Iff.rfl

theorem reindexG_mem {S : Set (Nat  x  (Nat  x  a))} (hS : (iterSys (iterSys V)).mem S
    ) :
    (iterSys V).mem (reindexG S) := by
  obtain <No, hNo> := hS.2
  obtain <M, hM> := exists_inner_bound hS No
  refine <fun k => ?_, Nat.pair No M, fun k hk => ?_>
  * -- fibers are `V`-neighbourhoods
    rw [fiber_reindexG]; exact mem_fiber2_of_mem hS _ _
  * -- cofinite-`Delta`
    rw [fiber_reindexG]
    set i := (Nat.unpair k).1 with hi
    set j := (Nat.unpair k).2 with hj
    by_cases hiN : i < No
    * by_cases hjM : j < M
      * exfalso
        have hklt : Nat.pair i j < Nat.pair No M := pair_lt_pair_of_lt hiN hjM
        rw [pair_unpair] at hklt
        omega
      * exact hM i hiN j (not_lt.mp hjM)
    * have h1 : fiber S i = (iterSys V).master := hNo i (not_lt.mp hiN)
      show fiber2 S i j = V.master
      rw [fiber2, h1, fiber_iterSys_master]

theorem reindexG_subset_iff {S : Set (Nat  x  (Nat  x  a))} {W : Set (Nat  x  a)} :
    reindexG S  subseteq  W  <->  forall  k, fiber2 S (Nat.unpair k).1 (Nat.unpair k)
    .2  subseteq  fiber W k := by
  constructor
  * intro h k
    have := fiber_mono h k
    rwa [fiber_reindexG] at this
  * intro h
    refine subset_of_fiber_subset (fun k => ?_)
    rw [fiber_reindexG]; exact h k

/-- **The inverse map `G : (D^ inf )^ inf   ->  D^ inf `** (reindexing fibers along `
    Nat.unpair`):
`S G W  <->  forall  k, fiber2 S (unpair k).1 (unpair k).2  subseteq  fiber W k`. -/
```

```
def Gmap (V : NeighborhoodSystem a) : ApproximableMap (iterSys (iterSys V)) (iterSys V
    ) where
  rel S W := (iterSys (iterSys V)).mem S  /\  (iterSys V).mem W  /\
    forall  k, fiber2 S (Nat.unpair k).1 (Nat.unpair k).2  subseteq  fiber W k
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := <(iterSys (iterSys V)).master_mem, (iterSys V).master_mem, fun k => by
    rw [fiber2_master, fiber_iterSys_master]>
  inter_right := by
    rintro S W W' <hS, hW, hg> <-, hW', hg'>
    refine <hS, (iterSys V).inter_mem hW hW' (reindexG_mem hS) ?_, fun k => ?_>
    * exact Set.subset_inter (reindexG_subset_iff.mpr hg) (reindexG_subset_iff.mpr hg
    ')
    * rw [fiber_inter]; exact Set.subset_inter (hg k) (hg' k)
  mono := by
    rintro S S' W W' <hS, hW, hg> hS'S hWW' hS' hW'
    exact <hS', hW', fun k => ((fiber2_mono hS'S _ _).trans (hg k)).trans (fiber_mono
    hWW' k)>

@[simp] theorem Gmap_rel {S : Set (Nat  x  (Nat  x  a))} {W : Set (Nat  x  a)} :
    (Gmap V).rel S W  <->  (iterSys (iterSys V)).mem S  /\  (iterSys V).mem W  /\
      forall  k, fiber2 S (Nat.unpair k).1 (Nat.unpair k).2  subseteq  fiber W k :=
    Iff.rfl

/-- `F  comp  G = I_{(D^ inf )^ inf }`. -/
theorem Fmap_comp_Gmap : (Fmap V).comp (Gmap V) = idMap (iterSys (iterSys V)) := by
  apply ApproximableMap.ext
  intro S S'
  rw [comp_rel, idMap_rel]
  constructor
  * rintro <W, <hS, hW, hg>, <-, hS', hf>>
    refine <hS, hS', subset_of_fiber_subset (fun i => subset_of_fiber_subset (fun j =>
     ?_))>
    show fiber2 S i j  subseteq  fiber2 S' i j
    have h1 : fiber2 S i j  subseteq  fiber W (Nat.pair i j) := by
      have := hg (Nat.pair i j)
      rwa [unpair_pair_fst, unpair_pair_snd] at this
    exact h1.trans (hf i j)
  * rintro <hS, hS', hSS'>
    refine <reindexG S, <hS, reindexG_mem hS, ?_>, <reindexG_mem hS, hS', ?_>>
    * intro k; rw [fiber_reindexG]
    * intro i j
      rw [fiber_reindexG, unpair_pair_fst, unpair_pair_snd]
      exact fiber2_mono hSS' i j

/-- `G  comp  F = I_{D^ inf }`. -/
theorem Gmap_comp_Fmap : (Gmap V).comp (Fmap V) = idMap (iterSys V) := by
  apply ApproximableMap.ext
  intro W W'
  rw [comp_rel, idMap_rel]
  constructor
  * rintro <S, <hW, hS, hf>, <-, hW', hg>>
    refine <hW, hW', subset_of_fiber_subset (fun k => ?_)>
    have h1 : fiber W k  subseteq  fiber2 S (Nat.unpair k).1 (Nat.unpair k).2 := by
      have := hf (Nat.unpair k).1 (Nat.unpair k).2
      rwa [pair_unpair] at this
```

```
    exact h1.trans (hg k)
  * rintro <hW, hW', hWW'>
    refine <reindexF W, <hW, reindexF_mem hW, ?_>, <reindexF_mem hW, hW', ?_>>
    * intro i j; rw [fiber2_reindexF]
    * intro k
      rw [fiber2_reindexF, pair_unpair]
      exact fiber_mono hWW' k

end

section
variable {a b : Type*}
variable {V : NeighborhoodSystem a}
variable (P : ComputablePresentation V)

theorem fiber2_iterEnum_iter (m i j : Nat) :
    fiber2 (iterEnum (iterPresentation P) m) i j
      = P.X (iterIdx P (iterIdx (iterPresentation P) m i) j) := by
  show fiber (fiber (iterEnum (iterPresentation P) m) i) j = _
  rw [fiber_iterEnum, iterPresentation_X, fiber_iterEnum]
end

section
variable {a b : Type*}
variable {V : NeighborhoodSystem a}
variable (P : ComputablePresentation V)
variable {P}

/-- **Characterisation of `G`'s relation on codes.** A single bounded inclusion: only `
    k < n`
matters (the codomain fiber is `Delta` for `k  >=  n`). -/
theorem Gmap_rel_enum_iff (s n : Nat) :
    (Gmap V).rel (iterEnum (iterPresentation P) s) (iterEnum P n)  <->
      forall  k, k < n  ->
        P.X (iterIdx P (iterIdx (iterPresentation P) s (Nat.unpair k).1) (Nat.unpair k
    ).2)
          subseteq  P.X (iterIdx P n k) := by
  rw [Gmap_rel]
  have hmem_1 := iterEnum_mem (P := iterPresentation P) s
  have hmem_2 := iterEnum_mem (P := P) n
  constructor
  * rintro <-, -, hg> k _
    have := hg k
    rwa [fiber2_iterEnum_iter, fiber_iterEnum] at this
  * intro hbd
    refine <hmem_1, hmem_2, fun k => ?_>
    rw [fiber2_iterEnum_iter, fiber_iterEnum]
    by_cases hk : k < n
    * exact hbd k hk
    * rw [iterIdx_ge (not_lt.mp hk), P.masterIdx_spec]
      exact V.sub_master (P.mem_X _)

/-- **`G` is computable** (in fact recursively decidable): a single bounded `forall `
    of `incl_computable`. -/
theorem Gmap_isComputable :
    ComputableMap (iterPresentation (iterPresentation P)) (iterPresentation P) (Gmap V
```

```
    ) := by
  constructor
  -- inner predicate `p_2 u`, with `u = <k, <s, n>>`
  have hLHS : Nat.Primrec (fun u : Nat =>
      iterIdx P (iterIdx (iterPresentation P) u.unpair.2.unpair.1 u.unpair.1.unpair.1)
        u.unpair.1.unpair.2) := by
    have hinner : Nat.Primrec (fun u : Nat =>
        iterIdx (iterPresentation P) u.unpair.2.unpair.1 u.unpair.1.unpair.1) := by
      refine (primrec_nthCode.comp
        ((Nat.Primrec.left.comp Nat.Primrec.right).pair
          ((Nat.Primrec.left.comp Nat.Primrec.left).pair
            (Nat.Primrec.const (iterPresentation P).masterIdx)))).of_eq (fun u => ?_)
      simp only [iterIdx, unpair_pair_fst, unpair_pair_snd]
    refine (primrec_nthCode.comp
      (hinner.pair ((Nat.Primrec.right.comp Nat.Primrec.left).pair
        (Nat.Primrec.const P.masterIdx)))).of_eq (fun u => ?_)
    simp only [iterIdx, unpair_pair_fst, unpair_pair_snd]
  have hRHS : Nat.Primrec (fun u : Nat =>
      iterIdx P u.unpair.2.unpair.2 u.unpair.1) := by
    refine (primrec_nthCode.comp
      ((Nat.Primrec.right.comp Nat.Primrec.right).pair
        (Nat.Primrec.left.pair (Nat.Primrec.const P.masterIdx)))).of_eq (fun u => ?_)
    simp only [iterIdx, unpair_pair_fst, unpair_pair_snd]
  have hp_2 : RecDecidable (fun u : Nat =>
      P.X (iterIdx P (iterIdx (iterPresentation P) u.unpair.2.unpair.1 u.unpair.1.
    unpair.1)
        u.unpair.1.unpair.2)  subseteq  P.X (iterIdx P u.unpair.2.unpair.2 u.unpair.1)
    ) := by
    refine RecDecidable.of_iff (fun u => ?_) (P.incl_computable.comp (hLHS.pair hRHS))
    simp only [unpair_pair_fst, unpair_pair_snd]
  have hbound_1 : Nat.Primrec (fun t : Nat => t.unpair.2) := Nat.Primrec.right
  have hp_0 := hp_2.bForall hbound_1
  refine (RecDecidable.of_iff (fun t => ?_) hp_0).re
  simp only [iterPresentation_X, unpair_pair_fst, unpair_pair_snd]
  exact Gmap_rel_enum_iff t.unpair.1 t.unpair.2

end

section
variable {a b : Type*}
variable {V : NeighborhoodSystem a}
variable (P : ComputablePresentation V)

theorem masterIdx_iterPresentation : (iterPresentation P).masterIdx = 0 := rfl
end

section
variable {a b : Type*}
variable {V : NeighborhoodSystem a}
variable (P : ComputablePresentation V)
variable {P}

/-- **Characterisation of `F`'s relation on codes.** Relative to `iterPresentation P`
    and
`iterPresentation (iterPresentation P)`, the relation reduces to a *bounded* double
    inclusion:
```

```
beyond the coded fiber lengths every fiber is `Delta`, so only `i < m` and `j < iterIdx
     (iter?) m i`
matter. -/
theorem Fmap_rel_enum_iff (n m : Nat) :
    (Fmap V).rel (iterEnum P n) (iterEnum (iterPresentation P) m)  <->
      forall  i, i < m  ->  forall  j, j < iterIdx (iterPresentation P) m i  ->
        P.X (iterIdx P n (Nat.pair i j))  subseteq  P.X (iterIdx P (iterIdx (
    iterPresentation P) m i) j) := by
  rw [Fmap_rel]
  have hmem_1 := iterEnum_mem (P := P) n
  have hmem_2 := iterEnum_mem (P := iterPresentation P) m
  constructor
  * rintro <-, -, hf> i _ j _
    have := hf i j
    rwa [fiber_iterEnum, fiber2_iterEnum_iter] at this
  * intro hbd
    refine <hmem_1, hmem_2, fun i j => ?_>
    rw [fiber_iterEnum, fiber2_iterEnum_iter]
    by_cases hi : i < m
    * by_cases hj : j < iterIdx (iterPresentation P) m i
      * exact hbd i hi j hj
      * -- `j` beyond the inner length ? RHS `= Delta`
        rw [iterIdx_ge (not_lt.mp hj), P.masterIdx_spec]
        exact V.sub_master (P.mem_X _)
    * -- `i` beyond the outer length ? inner code `= 0` ? RHS `= Delta`
      have h0 : iterIdx (iterPresentation P) m i = 0 := by
        rw [iterIdx_ge (not_lt.mp hi), masterIdx_iterPresentation]
      rw [h0, iterIdx_ge (Nat.zero_le j), P.masterIdx_spec]
      exact V.sub_master (P.mem_X _)

/-- **`F` is computable** (in fact recursively decidable): its index relation reduces
    to a nested
bounded `forall ` of `incl_computable`. -/
theorem Fmap_isComputable :
    ComputableMap (iterPresentation P) (iterPresentation (iterPresentation P)) (Fmap V
    ) := by
  constructor
  -- inner predicate `p_2 u`, with `u = <j, <i, <n, m>>>`
  have hLHS : Nat.Primrec (fun u : Nat =>
      iterIdx P u.unpair.2.unpair.2.unpair.1 (Nat.pair u.unpair.2.unpair.1 u.unpair.1)
    ) := by
    refine (primrec_nthCode.comp
      ((Nat.Primrec.left.comp (Nat.Primrec.right.comp Nat.Primrec.right)).pair
        (((Nat.Primrec.left.comp Nat.Primrec.right).pair Nat.Primrec.left).pair
          (Nat.Primrec.const P.masterIdx)))).of_eq (fun u => ?_)
    simp only [iterIdx, unpair_pair_fst, unpair_pair_snd]
  have hRHS : Nat.Primrec (fun u : Nat =>
      iterIdx P (iterIdx (iterPresentation P) u.unpair.2.unpair.2.unpair.2 u.unpair.2.
    unpair.1)
        u.unpair.1) := by
    have hinner : Nat.Primrec (fun u : Nat =>
        iterIdx (iterPresentation P) u.unpair.2.unpair.2.unpair.2 u.unpair.2.unpair.1)
     := by
      refine (primrec_nthCode.comp
        ((Nat.Primrec.right.comp (Nat.Primrec.right.comp Nat.Primrec.right)).pair
          ((Nat.Primrec.left.comp Nat.Primrec.right).pair
```

```
            (Nat.Primrec.const (iterPresentation P).masterIdx)))).of_eq (fun u => ?_)
      simp only [iterIdx, unpair_pair_fst, unpair_pair_snd]
    refine (primrec_nthCode.comp
      (hinner.pair (Nat.Primrec.left.pair (Nat.Primrec.const P.masterIdx)))).of_eq (
    fun u => ?_)
    simp only [iterIdx, unpair_pair_fst, unpair_pair_snd]
  have hp_2 : RecDecidable (fun u : Nat =>
      P.X (iterIdx P u.unpair.2.unpair.2.unpair.1 (Nat.pair u.unpair.2.unpair.1 u.
    unpair.1))
        subseteq  P.X (iterIdx P (iterIdx (iterPresentation P) u.unpair.2.unpair.2.
    unpair.2
          u.unpair.2.unpair.1) u.unpair.1)) := by
    refine RecDecidable.of_iff (fun u => ?_) (P.incl_computable.comp (hLHS.pair hRHS))
    simp only [unpair_pair_fst, unpair_pair_snd]
  -- inner bound `bound_2 v = iterIdx (iter?) m i`, with `v = <i, <n, m>>`
  have hbound_2 : Nat.Primrec (fun v : Nat =>
      iterIdx (iterPresentation P) v.unpair.2.unpair.2 v.unpair.1) := by
    refine (primrec_nthCode.comp
      ((Nat.Primrec.right.comp Nat.Primrec.right).pair
        (Nat.Primrec.left.pair (Nat.Primrec.const (iterPresentation P).masterIdx)))).
    of_eq
      (fun v => ?_)
    simp only [iterIdx, unpair_pair_fst, unpair_pair_snd]
  have hbound_1 : Nat.Primrec (fun t : Nat => t.unpair.2) := Nat.Primrec.right
  have hp_1 := hp_2.bForall hbound_2
  have hp_0 := hp_1.bForall hbound_1
  refine (RecDecidable.of_iff (fun t => ?_) hp_0).re
  -- match the relation with the nested bounded `forall `
  simp only [iterPresentation_X, unpair_pair_fst, unpair_pair_snd]
  exact Fmap_rel_enum_iff t.unpair.1 t.unpair.2

/-- **Exercise 7.18 (Scott 1981, PRG-19) -- `D^ inf   ~=  (D^ inf )^ inf ` is an *
    effective* isomorphism.** Relative
to `iterPresentation P` and `iterPresentation (iterPresentation P)`, the reindexing
    maps `F`, `G` are
mutually inverse computable approximable maps. -/
def iterIterEffectiveIso (P : ComputablePresentation V) :
    EffectiveIso (iterPresentation P) (iterPresentation (iterPresentation P)) where
  toMap := Fmap V
  invMap := Gmap V
  toMap_computable := Fmap_isComputable
  invMap_computable := Gmap_isComputable
  left_inv := Gmap_comp_Fmap


  right_inv := Fmap_comp_Gmap

end

end Exercise718
namespace Exercise722
open Set Domain.Recursive

section
/-- Canonical representative for `SsysX n` as a fragment expression (junk  ->  `Sigma`)
    . -/
def safeDecodeActive (n : Nat) : SExpr :=
```

```
  match SExpr.decode n with
  | none => .sigma
  | some e => if decideNonemptyB e then e else .sigma
end

section
theorem SsysX_eq_denote_safe (n : Nat) : SsysX n = denote (safeDecodeActive n) := by
  unfold SsysX safeDecodeActive
  cases hdec : SExpr.decode n with
  | none => simp [denote_sigma, SsysX_default]
  | some e =>
    by_cases hb : decideNonemptyB e = true
    * simp [hb]
    * simp [hb, denote_sigma, SsysX_default]
end

section
theorem safeDecodeActive_nonempty (n : Nat) : (denote (safeDecodeActive n)).Nonempty
    := by
  unfold safeDecodeActive
  cases hdec : SExpr.decode n with
  | none => exact Set.univ_nonempty
  | some e =>
    by_cases hb : decideNonemptyB e = true
    * simp [hb]; exact (decideNonemptyB_iff e).mp hb
    * simp [hb, denote_sigma]
end

section
theorem consistentB_safe_sigma (n : Nat) : consistentB (safeDecodeActive n) .sigma =
    true := by
  rw [consistentB_iff, denote_cap, denote_sigma, Set.inter_univ]
  exact safeDecodeActive_nonempty n
end

section
theorem consistentB_sigma_safe (m : Nat) : consistentB .sigma (safeDecodeActive m) =
    true := by
  rw [consistentB_iff, denote_cap, denote_sigma, Set.univ_inter]
  exact safeDecodeActive_nonempty m
end

section
/-- Subexpression depth (for fuelled decoding). -/
def sexprDepth : SExpr  ->  Nat
  | .sigma => 1
  | .single _ => 1
  | .cat a b => 1 + max (sexprDepth a) (sexprDepth b)
  | .cap a b => 1 + max (sexprDepth a) (sexprDepth b)
end

section
theorem decodeFuel_encode {fuel : Nat} {e : SExpr} (h : sexprDepth e  <=  fuel) :
    decodeFuel fuel (SExpr.encode e) = some e := by
  induction e generalizing fuel with
  | sigma =>
```

```
    cases fuel with
    | zero => simp [sexprDepth] at h
    | succ fuel =>
      simp [decodeFuel, SExpr.encode]
  | single sigma =>
    cases fuel with
    | zero => simp [sexprDepth] at h
    | succ fuel =>
      simp [decodeFuel, SExpr.encode, decodeListBool_encodeListBool sigma]
  | cat a b iha ihb =>
    cases fuel with
    | zero => simp [sexprDepth] at h
    | succ fuel =>
      have ha : sexprDepth a  <=  fuel := by
        simp only [sexprDepth] at h  |-
        omega
      have hb : sexprDepth b  <=  fuel := by
        simp only [sexprDepth] at h  |-
        omega
      simp [decodeFuel, SExpr.encode, iha ha, ihb hb]
  | cap a b iha ihb =>
    cases fuel with
    | zero => simp [sexprDepth] at h
    | succ fuel =>
      have ha : sexprDepth a  <=  fuel := by
        simp only [sexprDepth] at h  |-
        omega
      have hb : sexprDepth b  <=  fuel := by
        simp only [sexprDepth] at h  |-
        omega
      simp [decodeFuel, SExpr.encode, iha ha, ihb hb]
end

section
/-- Index for the enumeration: code plus depth (fuel for decoding). -/
def SExpr.index (e : SExpr) : Nat := Nat.pair (encode e) (sexprDepth e)
end

section
theorem SExpr.decode_index (e : SExpr) : decode (index e) = some e := by
  simp [decode, index]
  exact decodeFuel_encode (Nat.le_succ (sexprDepth e))
end

section
theorem SsysX_surj {Y : Set (List Bool)} (hY : InS Y) : exists  n, SsysX n = Y := by
  obtain <e, he, hne> := inS_iff_exists_denote.mp hY
  refine <SExpr.index e, ?_>
  unfold SsysX
  rw [SExpr.decode_index e]
  have hne' : (denote e).Nonempty := he  |>  hne
  have hb : decideNonemptyB e = true := (decideNonemptyB_iff e).2 hne'
  simp [hb, he]
end

section
```

```
/-- Inactive index: decode fails or denotes the empty language (mapped to `Sigma` in `
    SsysX`). -/
def ssysActive (n : Nat) : Bool :=
  match SExpr.decode n with
  | none => false
  | some e => decideNonemptyB e
end

section
theorem safeDecodeActive_inactive (n : Nat) (h : ssysActive n = false) :
    safeDecodeActive n = .sigma := by
  dsimp only [ssysActive, safeDecodeActive] at h  |-
  cases hdec : SExpr.decode n with
  | none => rfl
  | some e =>
    by_cases hb : decideNonemptyB e
    * exfalso
      simp [hdec, hb] at h
    * simp [hb]
end

section
/-- `{0,1}` consistency decider on index pairs (active-active uses `consistentB`). -/
def ssysConsistentB (n m : Nat) : Bool :=
  if !ssysActive n || !ssysActive m then true
  else
    match SExpr.decode n, SExpr.decode m with
    | some a, some b => consistentB a b
    | _, _ => true
end

section
theorem ssysConsistentB_iff (n m : Nat) :
    ssysConsistentB n m = true  <->
      consistentB (safeDecodeActive n) (safeDecodeActive m) = true := by
  by_cases hn : ssysActive n
  * by_cases hm : ssysActive m
    * cases hdecn : SExpr.decode n with
      | none => exfalso; simp [ssysActive, hdecn] at hn
      | some a =>
        cases hdecm : SExpr.decode m with
        | none => exfalso; simp [ssysActive, hdecm] at hm
        | some b =>
          have ha : decideNonemptyB a := by simpa [ssysActive, hdecn] using hn
          have hb : decideNonemptyB b := by simpa [ssysActive, hdecm] using hm
          have hsa : safeDecodeActive n = a := by simp [safeDecodeActive, hdecn, ha]
          have hsb : safeDecodeActive m = b := by simp [safeDecodeActive, hdecm, hb]
          constructor
          * intro h
            rw [hsa, hsb]
            have heq : ssysConsistentB n m = consistentB a b := by
              unfold ssysConsistentB
              simp [hn, hm, hdecn, hdecm]
            rw [heq] at h
            exact h
          * intro hcons
```

```
            rw [hsa, hsb] at hcons
            have heq : ssysConsistentB n m = consistentB a b := by
              unfold ssysConsistentB
              simp [hn, hm, hdecn, hdecm]
            rw [heq]
            exact hcons
    * have hm' : ssysActive m = false := by simpa using hm
      constructor
      * intro _; rw [safeDecodeActive_inactive m hm', consistentB_safe_sigma n]
      * intro _; unfold ssysConsistentB; simp [hn, hm']
  * have hn' : ssysActive n = false := by simpa using hn
    constructor
    * intro _; rw [safeDecodeActive_inactive n hn', consistentB_sigma_safe m]
    * intro _; unfold ssysConsistentB; simp [hn']
end

section
theorem ssys_cons_positivity (n m : Nat) :
    (exists  k, SsysX k  subseteq  SsysX n  I  SsysX m)  <->  (SsysX n  I  SsysX m).
    Nonempty := by
  constructor
  * intro <k, hk>
    exact Set.Nonempty.mono hk (InS.nonempty (SsysX_mem k))
  * intro hne
    have hIn : InS (SsysX n  I  SsysX m) :=
      (Ssys_isPositive (Ssys_mem.mpr (SsysX_mem n)) (Ssys_mem.mpr (SsysX_mem m))).2
    hne
    obtain <k, hk> := SsysX_surj hIn
    exact <k, hk.subset>
end

section
theorem ssys_inter_nonempty_iff_consistent (n m : Nat) :
    (SsysX n  I  SsysX m).Nonempty  <->  consistentB (safeDecodeActive n) (
    safeDecodeActive m) = true := by
  rw [SsysX_eq_denote_safe, SsysX_eq_denote_safe,  <-  denote_cap, consistentB_iff]
end

section
theorem ssys_cons_iff (n m : Nat) :
    (exists  k, SsysX k  subseteq  SsysX n  I  SsysX m)  <->
      consistentB (safeDecodeActive n) (safeDecodeActive m) = true := by
  rw [ssys_cons_positivity, ssys_inter_nonempty_iff_consistent]
end

section
theorem ssys_cons_char_iff (n m : Nat) :
    ssysConsistentB n m = true  <->  exists  k, SsysX k  subseteq  SsysX n  I  SsysX m
     := by
  rw [ssysConsistentB_iff, ssys_cons_iff]
end

section
/-- Scott's consistency relation (Definition 7.1 (ii)) on the `SsysX` indices is
    decidable via
`ssysConsistentB` (choice-free on the `Bool` side; see axiom audit of `
```

```
    decideNonemptyB_iff`). -/
instance decidableSsysCons (n m : Nat) :
    Decidable (exists  k, SsysX k  subseteq  SsysX n  I  SsysX m) :=
  decidable_of_iff (ssysConsistentB n m = true) (ssys_cons_char_iff n m)
end

end Exercise722
end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.Exercise722
section
set_option linter.unusedVariables false

theorem example_722_presentation_inst47 : forall  (n m : Nat),
  Iff (@Eq Bool (DomainTheory.NeighbourhoodSystems.Exercise722.ssysConsistentB n m)
    Bool.true)
    (exists  (k : Nat),
      @LE.le (Set (List Bool)) (@Set.instLE (List Bool)) (DomainTheory.
    NeighbourhoodSystems.Exercise722.SsysX k)
        (@Inter.inter (Set (List Bool)) (@Set.instInter (List Bool))
          (DomainTheory.NeighbourhoodSystems.Exercise722.SsysX n)
          (DomainTheory.NeighbourhoodSystems.Exercise722.SsysX m))) :=
  DomainTheory.NeighbourhoodSystems.Exercise722.ssys_cons_char_iff
end

end DomainTheory.NeighbourhoodSystems.Exercise722
namespace DomainTheory.NeighbourhoodSystems
namespace Exercise722
open Set Domain.Recursive

section
set_option maxHeartbeats 5000000

/-- A neighbourhood system is *partially* effectively given when it admits a
    presentation
carrying relation (ii) (but not necessarily (i)). Named at top level (not `
    NeighborhoodSystem....`)
since `Definition71.lean` -- where `NeighborhoodSystem`/`ComputablePresentation` live
    -- is outside
this session's edit scope; a future session may hoist `ConsistencyPresentation` there
    alongside
them for dot-notation parity with `IsEffectivelyGiven`. -/
class PartiallyEffectivelyGiven (V : NeighborhoodSystem a) : Prop where
  out : Nonempty (ConsistencyPresentation V)

theorem c9b5_encodeListBool_eq (sigma : List Bool) :
    Domain.Recursive.c9b5_encodeListBool sigma = encodeListBool sigma := by
  unfold Domain.Recursive.c9b5_encodeListBool encodeListBool
  congr 1

theorem c9b5_sexprDepth_eq (e : SExpr) : Domain.Recursive.c9b5_sexprDepth e =
    sexprDepth e := by
  induction e with
  | sigma => rfl
  | single _ => rfl
  | cat a b iha ihb => simp [Domain.Recursive.c9b5_sexprDepth, sexprDepth, iha, ihb]
  | cap a b iha ihb => simp [Domain.Recursive.c9b5_sexprDepth, sexprDepth, iha, ihb]
```

```
theorem c9b5_sexprGodelEncode_eq (e : SExpr) :
    Domain.Recursive.c9b5_sexprGodelEncode e = SExpr.encode e := by
  induction e with
  | sigma => rfl
  | single sigma => simp [Domain.Recursive.c9b5_sexprGodelEncode, SExpr.encode,
    c9b5_encodeListBool_eq]
  | cat a b iha ihb => simp [Domain.Recursive.c9b5_sexprGodelEncode, SExpr.encode, iha
    , ihb]
  | cap a b iha ihb => simp [Domain.Recursive.c9b5_sexprGodelEncode, SExpr.encode, iha
    , ihb]

/-- **Fuel sufficiency:** the recovered expression never needs more depth than the
    fuel it was
decoded with. -/
theorem decodeFuel_depth_le {fuel c : Nat} {e : SExpr} (h : decodeFuel fuel c = some e
    ) :
    sexprDepth e  <=  fuel := by
  induction fuel generalizing c e with
  | zero => simp [decodeFuel] at h
  | succ fuel ih =>
    match tag : c.unpair.1 with
    | 0 =>
      by_cases hsigma : c.unpair.2 = 0
      * rw [decodeFuel_succ_sigma fuel c tag hsigma] at h
        simp only [Option.some.injEq] at h
        simp [ <-  h, sexprDepth]
      * rw [decodeFuel_succ_not_sigma fuel c tag hsigma] at h; simp at h
    | 1 =>
      rw [decodeFuel_succ_single fuel c tag] at h
      cases hb : decodeListBool c.unpair.2 with
      | none => simp [hb] at h
      | some sigma =>
        rw [hb] at h
        simp only [Option.map_some, Option.some.injEq] at h
        simp [ <-  h, sexprDepth]
    | 2 =>
      rw [decodeFuel_succ_cat fuel c tag] at h
      cases ha : decodeFuel fuel c.unpair.2.unpair.1 with
      | none => simp [ha] at h
      | some a =>
        cases hb : decodeFuel fuel c.unpair.2.unpair.2 with
        | none => simp [ha, hb] at h
        | some b =>
          simp only [ha, hb, Option.some.injEq] at h
          have hda := ih ha
          have hdb := ih hb
          simp only [ <-  h, sexprDepth]
          omega
    | 3 =>
      rw [decodeFuel_succ_cap fuel c tag] at h
      cases ha : decodeFuel fuel c.unpair.2.unpair.1 with
      | none => simp [ha] at h
      | some a =>
        cases hb : decodeFuel fuel c.unpair.2.unpair.2 with
        | none => simp [ha, hb] at h
```

```
        | some b =>
          simp only [ha, hb, Option.some.injEq] at h
          have hda := ih ha
          have hdb := ih hb
          simp only [ <-  h, sexprDepth]
          omega
    | _ + 4 =>
      rw [decodeFuel_succ_junk fuel c (by omega)] at h; simp at h

theorem natBool_eq_some_iff (n : Nat) (b : Bool) : natBool n = some b  <->  n =
    boolNat b := by
  match n with
  | 0 => cases b <;> simp [natBool, boolNat]
  | 1 => cases b <;> simp [natBool, boolNat]
  | n + 2 => cases b <;> simp [natBool, boolNat]

theorem mapM_natBool_eq_some (l : List Nat) (sigma : List Bool)
    (h : l.mapM natBool = some sigma) : l = sigma.map boolNat := by
  induction l generalizing sigma with
  | nil => rw [List.mapM_nil] at h; simpa using h.symm
  | cons x xs ih =>
    rw [List.mapM_cons] at h
    cases hnx : natBool x with
    | none => simp [hnx] at h
    | some b =>
      cases htx : xs.mapM natBool with
      | none => simp [hnx, htx] at h
      | some ys =>
        simp only [hnx, htx] at h
        obtain rfl : sigma = b :: ys := by simpa using h.symm
        simp [ih ys htx, (natBool_eq_some_iff x b).mp hnx]

theorem decodeListBool_eq_some (n : Nat) (sigma : List Bool) (h : decodeListBool n =
    some sigma) :
    n = encodeListBool sigma := by
  have hl : decodeList n = sigma.map boolNat := mapM_natBool_eq_some (decodeList n)
    sigma h
  calc n = encodeList (decodeList n) := (encodeList_decodeList n).symm
    _ = encodeList (sigma.map boolNat) := by rw [hl]
    _ = encodeListBool sigma := rfl

/-- **Soundness:** any code `c` that `decodeFuel` accepts is the canonical `SExpr.
    encode` of the
recovered expression (`decodeList`/`decodeListBool`/`Nat.pair` are all injective in the
     relevant
direction, so nothing "junk" can decode). -/
theorem decodeFuel_sound {fuel c : Nat} {e : SExpr} (h : decodeFuel fuel c = some e) :
    c = SExpr.encode e := by
  induction fuel generalizing c e with
  | zero => simp [decodeFuel] at h
  | succ fuel ih =>
    match tag : c.unpair.1 with
    | 0 =>
      by_cases hsigma : c.unpair.2 = 0
      * have hc : c = Nat.pair 0 0 := by
          have hpu := Nat.pair_unpair c; rw [tag, hsigma] at hpu; exact hpu.symm
```

```
      rw [decodeFuel_succ_sigma fuel c tag hsigma] at h
      simp only [Option.some.injEq] at h
      simp [SExpr.encode,  <-  h, hc]
    * rw [decodeFuel_succ_not_sigma fuel c tag hsigma] at h; simp at h
  | 1 =>
    have hc1 : c = Nat.pair 1 c.unpair.2 := by
      have hpu := Nat.pair_unpair c; rw [tag] at hpu; exact hpu.symm
    rw [decodeFuel_succ_single fuel c tag] at h
    cases hb : decodeListBool c.unpair.2 with
    | none => simp [hb] at h
    | some sigma =>
      rw [hb] at h
      simp only [Option.map_some, Option.some.injEq] at h
      have hsigmac : c.unpair.2 = encodeListBool sigma := decodeListBool_eq_some c.
unpair.2 sigma hb
      rw [ <-  h, SExpr.encode, hc1, hsigmac]
  | 2 =>
    have hc2 : c = Nat.pair 2 c.unpair.2 := by
      have hpu := Nat.pair_unpair c; rw [tag] at hpu; exact hpu.symm
    rw [decodeFuel_succ_cat fuel c tag] at h
    cases ha : decodeFuel fuel c.unpair.2.unpair.1 with
    | none => simp [ha] at h
    | some a =>
      cases hb : decodeFuel fuel c.unpair.2.unpair.2 with
      | none => simp [ha, hb] at h
      | some b =>

        simp only [ha, hb, Option.some.injEq] at h
        have hac : c.unpair.2.unpair.1 = SExpr.encode a := ih ha
        have hbc : c.unpair.2.unpair.2 = SExpr.encode b := ih hb
        have hpc : c.unpair.2 = Nat.pair (SExpr.encode a) (SExpr.encode b) := by
          have hpu := Nat.pair_unpair c.unpair.2; rw [hac, hbc] at hpu; exact hpu.
symm
        rw [ <-  h, SExpr.encode, hc2, hpc]
  | 3 =>
    have hc3 : c = Nat.pair 3 c.unpair.2 := by
      have hpu := Nat.pair_unpair c; rw [tag] at hpu; exact hpu.symm
    rw [decodeFuel_succ_cap fuel c tag] at h
    cases ha : decodeFuel fuel c.unpair.2.unpair.1 with
    | none => simp [ha] at h
    | some a =>
      cases hb : decodeFuel fuel c.unpair.2.unpair.2 with
      | none => simp [ha, hb] at h
      | some b =>
        simp only [ha, hb, Option.some.injEq] at h
        have hac : c.unpair.2.unpair.1 = SExpr.encode a := ih ha
        have hbc : c.unpair.2.unpair.2 = SExpr.encode b := ih hb
        have hpc : c.unpair.2 = Nat.pair (SExpr.encode a) (SExpr.encode b) := by
          have hpu := Nat.pair_unpair c.unpair.2; rw [hac, hbc] at hpu; exact hpu.
symm
        rw [ <-  h, SExpr.encode, hc3, hpc]
  | _ + 4 =>
    rw [decodeFuel_succ_junk fuel c (by omega)] at h; simp at h

/-- `{0,1}` packaging of `ssysActive` via `decodeFuelOkChar` (C9b1) + `
decideNonemptyBChar`
```

```
(C9b6), operating purely on the Godel code `n.unpair.1` with fuel `n.unpair.2 + 1` (
    mirroring
`SExpr.decode`). -/
def ssysActiveChar (n : Nat) : Nat :=
  mulBit (decodeFuelOkChar (n.unpair.2 + 1) n.unpair.1)
    (decideNonemptyBChar (n.unpair.2 + 1) n.unpair.1)

theorem primrec_ssysActiveChar : Nat.Primrec ssysActiveChar := by
  have hpack : Nat.Primrec (fun n : Nat => Nat.pair (n.unpair.2 + 1) n.unpair.1) :=
    Nat.Primrec.pair (primrec_add_2 Nat.Primrec.right (Nat.Primrec.const 1)) Nat.
    Primrec.left
  have h1 : Nat.Primrec (fun n => decodeFuelOkChar (n.unpair.2 + 1) n.unpair.1) :=
    (primrec_decodeFuelOkChar2.comp hpack).of_eq fun n => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  have h2 : Nat.Primrec (fun n => decideNonemptyBChar (n.unpair.2 + 1) n.unpair.1) :=
    (primrec_decideNonemptyBChar2.comp hpack).of_eq fun n => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  exact (primrec_mulBit.comp (Nat.Primrec.pair h1 h2)).of_eq fun n => by
    simp only [unpair_pair_fst, unpair_pair_snd, ssysActiveChar]

theorem ssysActiveChar_eq_one_iff (n : Nat) : ssysActiveChar n = 1  <->  ssysActive n
    = true := by
  simp only [ssysActiveChar, ssysActive, SExpr.decode, mulBit_eq_one_iff,
    decodeFuelOkChar_eq_one_iff]
  cases hdec : decodeFuel (n.unpair.2 + 1) n.unpair.1 with
  | none => simp
  | some e =>
    have hc : n.unpair.1 = SExpr.encode e := decodeFuel_sound hdec
    have hdepth : Domain.Recursive.c9b5_sexprDepth e  <=  n.unpair.2 + 1 := by
      rw [c9b5_sexprDepth_eq]; exact decodeFuel_depth_le hdec
    simp only [Option.isSome_some, true_and]
    rw [hc,  <-  c9b5_sexprGodelEncode_eq, decideNonemptyBChar_eq_one_iff hdepth,
    decideNonemptyB_iff]

theorem ssysActiveChar_le_one (n : Nat) : ssysActiveChar n  <=  1 :=
  mulBit_le_one (decodeFuelOkChar_le_one _ _) (bExistsFn_le_one _ _ _)

theorem ssysActive_exists_decode {n : Nat} (hn : ssysActive n = true) :
    exists  e, SExpr.decode n = some e := by
  unfold ssysActive at hn
  cases hdec : SExpr.decode n with
  | none => simp [hdec] at hn
  | some e => exact <e, rfl>

/-- The Godel code of `safeDecodeActive n` (`ssysCanonicalCode_eq`): `n`'s own decoded
    code if
active, else the code of `.sigma` (matching `safeDecodeActive`'s junk fallback). -/
@[irreducible] def ssysCanonicalCode (n : Nat) : Nat :=
  Domain.Recursive.selectFn (ssysActiveChar n) n.unpair.1 (SExpr.encode .sigma)

/-- `{0,1}` packaging of `ssysConsistentB` on `Nat.pair`-coded index pairs. -/
def ssysConsChar (t : Nat) : Nat := boolNat (ssysConsistentB t.unpair.1 t.unpair.2)

theorem ssysConsChar_eq_one_iff (t : Nat) :
    ssysConsChar t = 1  <->  ssysConsistentB t.unpair.1 t.unpair.2 = true := by
  simp [ssysConsChar, boolNat_eq_one_iff]
```

```
theorem ssysConsChar_le_one (t : Nat) : ssysConsChar t  <=  1 := by
  have := boolNat_zero_one (ssysConsistentB t.unpair.1 t.unpair.2)
  unfold ssysConsChar
  omega

theorem ssysConsChar_zero_one (t : Nat) : ssysConsChar t = 0  \/  ssysConsChar t = 1
    :=
  boolNat_zero_one _

/-- `{0,1}` packaging of `ssysConsistentB`: inactive-either defaults to `1` (consistent
    ), while
active-active reduces to `consistentBChar` (C9b6). Fuel `n.unpair.2 + m.unpair.2 + 2`
    is generous
enough for the `.cap` of both decoded expressions regardless of which side is deeper.
    -/
def ssysConsistentBChar (n m : Nat) : Nat :=
  selectFn (mulBit (ssysActiveChar n) (ssysActiveChar m))
    (consistentBChar (n.unpair.2 + m.unpair.2 + 2) n.unpair.1 m.unpair.1)
    1

theorem primrec_ssysConsistentBChar :
    Nat.Primrec (fun t => ssysConsistentBChar t.unpair.1 t.unpair.2) := by
  have hcond : Nat.Primrec
      (fun t => mulBit (ssysActiveChar t.unpair.1) (ssysActiveChar t.unpair.2)) :=
    (primrec_mulBit.comp (Nat.Primrec.pair (primrec_ssysActiveChar.comp Nat.Primrec.
    left)
      (primrec_ssysActiveChar.comp Nat.Primrec.right))).of_eq fun t => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  have hfuel : Nat.Primrec (fun t : Nat => t.unpair.1.unpair.2 + t.unpair.2.unpair.2 +
     2) :=
    primrec_add_2 (primrec_add_2 (Nat.Primrec.right.comp Nat.Primrec.left)
      (Nat.Primrec.right.comp Nat.Primrec.right)) (Nat.Primrec.const 2)
  have hpack : Nat.Primrec (fun t : Nat => Nat.pair (t.unpair.1.unpair.2 + t.unpair.2.
    unpair.2 + 2)
      (Nat.pair t.unpair.1.unpair.1 t.unpair.2.unpair.1)) :=
    Nat.Primrec.pair hfuel (Nat.Primrec.pair (Nat.Primrec.left.comp Nat.Primrec.left)
      (Nat.Primrec.left.comp Nat.Primrec.right))
  have hcons : Nat.Primrec (fun t => consistentBChar
      (t.unpair.1.unpair.2 + t.unpair.2.unpair.2 + 2) t.unpair.1.unpair.1 t.unpair.2.
    unpair.1) :=
    (primrec_consistentBChar2.comp hpack).of_eq fun t => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  exact (primrec_selectFn hcond hcons (Nat.Primrec.const 1)).of_eq fun t => by
    simp only [ssysConsistentBChar]

theorem ssysConsistentBChar_eq_one_iff (n m : Nat) :
    ssysConsistentBChar n m = 1  <->  ssysConsistentB n m = true := by
  unfold ssysConsistentBChar ssysConsistentB
  by_cases hn : ssysActive n = true
  * by_cases hm : ssysActive m = true
    * have hnc : ssysActiveChar n = 1 := (ssysActiveChar_eq_one_iff n).mpr hn
      have hmc : ssysActiveChar m = 1 := (ssysActiveChar_eq_one_iff m).mpr hm
      obtain <a, hdecn> := ssysActive_exists_decode hn
      obtain <b, hdecm> := ssysActive_exists_decode hm
      have hac : n.unpair.1 = SExpr.encode a := decodeFuel_sound hdecn
```

```
      have hbc : m.unpair.1 = SExpr.encode b := decodeFuel_sound hdecm
      have hda : sexprDepth a  <=  n.unpair.2 + 1 := decodeFuel_depth_le hdecn
      have hdb : sexprDepth b  <=  m.unpair.2 + 1 := decodeFuel_depth_le hdecm
      have hfuel : Domain.Recursive.c9b5_sexprDepth (.cap a b)  <=  n.unpair.2 + m.
    unpair.2 + 2 := by
        rw [c9b5_sexprDepth_eq]
        simp only [sexprDepth]
        omega
      simp only [hnc, hmc, mulBit, mul_one, selectFn_one]
      rw [hac, hbc,  <-  c9b5_sexprGodelEncode_eq,  <-  c9b5_sexprGodelEncode_eq,
        consistentBChar_eq_one_iff hfuel]
      have hnotinactive : (!ssysActive n || !ssysActive m) = false := by simp [hn, hm]
      rw [hnotinactive, ite_eq_right (by decide)]
      simp only [hdecn, hdecm]
      exact (consistentB_iff a b).symm
    * have hm0 : ssysActiveChar m = 0 := by
        have hle := ssysActiveChar_le_one m
        have hne : ssysActiveChar m  !=  1 := fun h => hm ((ssysActiveChar_eq_one_iff
    m).mp h)
        omega
      have hmf : ssysActive m = false := by
        cases hAm : ssysActive m with
        | false => rfl
        | true => exact absurd hAm hm
      simp only [hm0, mulBit, mul_zero, selectFn_zero, hmf, Bool.not_false, Bool.
    or_true, ite_true]
  * have hn0 : ssysActiveChar n = 0 := by
      have hle := ssysActiveChar_le_one n
      have hne : ssysActiveChar n  !=  1 := fun h => hn ((ssysActiveChar_eq_one_iff n)
    .mp h)
      omega
    have hnf : ssysActive n = false := by
      cases hAn : ssysActive n with
      | false => rfl
      | true => exact absurd hAn hn
    simp only [hn0, mulBit, zero_mul, selectFn_zero, hnf, Bool.not_false, Bool.true_or
    , ite_true]

theorem ssysConsistentBChar_le_one (n m : Nat) : ssysConsistentBChar n m  <=  1 := by
  unfold ssysConsistentBChar
  rcases (show mulBit (ssysActiveChar n) (ssysActiveChar m) = 0  \/
      mulBit (ssysActiveChar n) (ssysActiveChar m) = 1 by
    have h1 := ssysActiveChar_le_one n
    have h2 := ssysActiveChar_le_one m
    unfold mulBit
    rcases (show ssysActiveChar n = 0  \/  ssysActiveChar n = 1 by omega) with hn | hn
     <;>
      rcases (show ssysActiveChar m = 0  \/  ssysActiveChar m = 1 by omega) with hm |
    hm <;>
      simp [hn, hm]) with h0 | h1
  * rw [h0, selectFn_zero]
  * rw [h1, selectFn_one]; exact consistentBChar_le_one _ _ _

theorem ssysConsChar_eq_ssysConsistentBChar (t : Nat) :
    ssysConsChar t = ssysConsistentBChar t.unpair.1 t.unpair.2 := by
  apply eq_of_le_one_iff_one (ssysConsChar_le_one t) (ssysConsistentBChar_le_one _ _)
```

```
  rw [ssysConsChar_eq_one_iff, ssysConsistentBChar_eq_one_iff]

/-- **7.22i(b)8 / closes the C9b umbrella.** -/
theorem primrec_ssysConsChar : Nat.Primrec ssysConsChar :=
  (primrec_ssysConsistentBChar).of_eq fun t => (ssysConsChar_eq_ssysConsistentBChar t)
    .symm

/-- **Exercise 7.22k.** `{0,1}` decider for Definition 7.1 relation (i) on `SsysX`
    indices,
`Nat.pair`-coded as `pair n (pair m k)`. -/
def ssysInterEqChar (t : Nat) : Nat :=
  let n := t.unpair.1
  let m := t.unpair.2.unpair.1
  let k := t.unpair.2.unpair.2
  Domain.Recursive.interEqChar (n.unpair.2 + m.unpair.2 + k.unpair.2 + 3)
    (Domain.Recursive.capCode (ssysCanonicalCode n) (ssysCanonicalCode m)) (
    ssysCanonicalCode k)

theorem ssys_cons_char_eq_one_iff (n m : Nat) :
    (exists  k, SsysX k  subseteq  SsysX n  I  SsysX m)  <->  ssysConsChar (Nat.pair n
     m) = 1 := by
  rw [ssysConsChar_eq_one_iff, ssys_cons_char_iff, unpair_pair_fst, unpair_pair_snd]

/-- **Conditional C9 closure.** Instantiated once the sole missing primitive-recursive
     link below
is proved. -/
theorem Ssys_cons_computable_of_primrec_ssysConsChar (hf : Nat.Primrec ssysConsChar) :
    RecDecidable_2 (fun n m => exists  k, SsysX k  subseteq  SsysX n  I  SsysX m) :=
  RecDecidable_2.of_paired_zero_one_char hf ssysConsChar_zero_one
    ssys_cons_char_eq_one_iff

/-- **Definition 7.1 (ii) is recursively decidable** (Exercise 7.22, closing C9). -/
theorem Ssys_cons_computable : RecDecidable_2 (fun n m => exists  k, SsysX k  subseteq
      SsysX n  I  SsysX m) :=
  Ssys_cons_computable_of_primrec_ssysConsChar primrec_ssysConsChar

/-- The partial presentation of `Ssys`: enumeration `SsysX` onto `S`, consistency via
`Ssys_cons_computable`. -/
def SsysPres : ConsistencyPresentation Ssys where
  X := SsysX
  mem_X n := Ssys_mem.mpr (SsysX_mem n)
  surj hY := SsysX_surj (Ssys_mem.mp hY)
  cons_computable := Ssys_cons_computable

theorem c9b5_boolNat_eq (b : Bool) : Domain.Recursive.c9b5_boolNat b = boolNat b := by
  cases b <;> rfl

theorem ssysCanonicalCode_eq (n : Nat) : ssysCanonicalCode n = SExpr.encode (
    safeDecodeActive n) := by
  unfold ssysCanonicalCode
  by_cases hn : ssysActive n = true
  * rw [(ssysActiveChar_eq_one_iff n).mpr hn, Domain.Recursive.selectFn_one]
    obtain <e, hdec> := ssysActive_exists_decode hn
    have hac : n.unpair.1 = SExpr.encode e := decodeFuel_sound hdec
    rw [hac]
    congr 1
```

```
    unfold safeDecodeActive
    rw [hdec]
    have : decideNonemptyB e = true := by
      unfold ssysActive at hn; rw [hdec] at hn; exact hn
    simp [this]
  * have hn0 : ssysActiveChar n = 0 := by
      have hle := ssysActiveChar_le_one n
      have hne : ssysActiveChar n  !=  1 := fun h => hn ((ssysActiveChar_eq_one_iff n)
    .mp h)
      omega
    have hnf : ssysActive n = false := by
      cases hAn : ssysActive n with
      | false => rfl
      | true => exact absurd hAn hn
    rw [hn0, Domain.Recursive.selectFn_zero, safeDecodeActive_inactive n hnf]

theorem sexprDepth_safeDecodeActive_le (n : Nat) : sexprDepth (safeDecodeActive n)  <=
     n.unpair.2 + 1 := by
  by_cases hn : ssysActive n = true
  * obtain <e, hdec> := ssysActive_exists_decode hn
    have hda : sexprDepth e  <=  n.unpair.2 + 1 := decodeFuel_depth_le hdec
    have heq : safeDecodeActive n = e := by
      unfold safeDecodeActive
      rw [hdec]
      have : decideNonemptyB e = true := by
        unfold ssysActive at hn; rw [hdec] at hn; exact hn
      simp [this]
    rw [heq]; exact hda
  * have hnf : ssysActive n = false := by
      cases hAn : ssysActive n with
      | false => rfl
      | true => exact absurd hAn hn
    rw [safeDecodeActive_inactive n hnf]
    simp [sexprDepth]

theorem ssys_interEq_iff (n m k : Nat) :
    SsysX n  I  SsysX m = SsysX k  <->
      denote (.cap (safeDecodeActive n) (safeDecodeActive m)) = denote (
    safeDecodeActive k) := by
  rw [SsysX_eq_denote_safe, SsysX_eq_denote_safe, SsysX_eq_denote_safe, denote_cap]

theorem ssysInterEqChar_le_one (t : Nat) : ssysInterEqChar t  <=  1 := by
  unfold ssysInterEqChar
  exact Domain.Recursive.interEqChar_le_one _ _ _

theorem ssysInterEqChar_eq_one_iff (n m k : Nat) :
    ssysInterEqChar (Nat.pair n (Nat.pair m k)) = 1  <->  SsysX n  I  SsysX m = SsysX
    k := by
  unfold ssysInterEqChar
  simp only [unpair_pair_fst, unpair_pair_snd]
  rw [ssys_interEq_iff]
  have hfuel : n.unpair.2 + m.unpair.2 + k.unpair.2 + 3  >=
      c9b5_sexprDepth (.cap (safeDecodeActive n) (safeDecodeActive m))  /\
      n.unpair.2 + m.unpair.2 + k.unpair.2 + 3  >=  c9b5_sexprDepth (safeDecodeActive
    k) := by
    rw [c9b5_sexprDepth_eq, c9b5_sexprDepth_eq]
```

```
    have hn := sexprDepth_safeDecodeActive_le n
    have hm := sexprDepth_safeDecodeActive_le m
    have hk := sexprDepth_safeDecodeActive_le k
    simp only [sexprDepth]
    omega
  rw [show Domain.Recursive.capCode (ssysCanonicalCode n) (ssysCanonicalCode m) =
      c9b5_sexprGodelEncode (.cap (safeDecodeActive n) (safeDecodeActive m)) from ?_,
    show ssysCanonicalCode k = c9b5_sexprGodelEncode (safeDecodeActive k) from ?_,
    Domain.Recursive.interEqChar_eq_one_iff hfuel.1 hfuel.2]
  * rw [c9b5_sexprGodelEncode_eq]; exact ssysCanonicalCode_eq k
  * unfold Domain.Recursive.capCode
    rw [ssysCanonicalCode_eq, ssysCanonicalCode_eq,  <-  c9b5_sexprGodelEncode_eq,
       <-  c9b5_sexprGodelEncode_eq]
    rfl

theorem primrec_ssysInterEqChar : Nat.Primrec ssysInterEqChar := by
  have hn : Nat.Primrec (fun t : Nat => t.unpair.1) := Nat.Primrec.left
  have hm : Nat.Primrec (fun t : Nat => t.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.right
  have hk : Nat.Primrec (fun t : Nat => t.unpair.2.unpair.2) :=
    Nat.Primrec.right.comp Nat.Primrec.right
  have hcanon : Nat.Primrec (fun n : Nat => ssysCanonicalCode n) := by
    have := primrec_ssysActiveChar
    refine (Domain.Recursive.primrec_selectFn this Nat.Primrec.left
      (Nat.Primrec.const (SExpr.encode .sigma))).of_eq fun n => ?_
    unfold ssysCanonicalCode; rfl
  have hcn : Nat.Primrec (fun t : Nat => ssysCanonicalCode t.unpair.1) := hcanon.comp
    hn
  have hcm : Nat.Primrec (fun t : Nat => ssysCanonicalCode t.unpair.2.unpair.1) :=
    hcanon.comp hm
  have hck : Nat.Primrec (fun t : Nat => ssysCanonicalCode t.unpair.2.unpair.2) :=
    hcanon.comp hk
  have hfuel : Nat.Primrec (fun t : Nat =>
      t.unpair.1.unpair.2 + t.unpair.2.unpair.1.unpair.2 + t.unpair.2.unpair.2.unpair
    .2 + 3) :=
    Domain.Recursive.primrec_add_2
      (Domain.Recursive.primrec_add_2
        (Domain.Recursive.primrec_add_2
          (Nat.Primrec.right.comp hn) (Nat.Primrec.right.comp hm))
        (Nat.Primrec.right.comp hk))
      (Nat.Primrec.const 3)
  have hcap : Nat.Primrec (fun t : Nat =>
      Domain.Recursive.capCode (ssysCanonicalCode t.unpair.1)
        (ssysCanonicalCode t.unpair.2.unpair.1)) := by
    have hp : Nat.Primrec (fun t : Nat => Nat.pair 3 (Nat.pair (ssysCanonicalCode t.
    unpair.1)
        (ssysCanonicalCode t.unpair.2.unpair.1))) :=
      Nat.Primrec.pair (Nat.Primrec.const 3) (Nat.Primrec.pair hcn hcm)
    refine hp.of_eq fun t => ?_
    unfold Domain.Recursive.capCode; rfl
  have hpack : Nat.Primrec (fun t : Nat => Nat.pair
      (t.unpair.1.unpair.2 + t.unpair.2.unpair.1.unpair.2 + t.unpair.2.unpair.2.unpair
    .2 + 3)
      (Nat.pair (Domain.Recursive.capCode (ssysCanonicalCode t.unpair.1)
          (ssysCanonicalCode t.unpair.2.unpair.1))
        (ssysCanonicalCode t.unpair.2.unpair.2))) :=
```

```
    Nat.Primrec.pair hfuel (Nat.Primrec.pair hcap hck)
  refine (Domain.Recursive.primrec_interEqChar.comp hpack).of_eq fun t => ?_
  unfold ssysInterEqChar
  simp only [unpair_pair_fst, unpair_pair_snd]

/-- **Exercise 7.22k.** Definition 7.1 relation (i) is recursively decidable for `Ssys
    `. -/
theorem Ssys_interEq_computable :
    RecDecidable_3 (fun n m k => SsysX n  I  SsysX m = SsysX k) :=
  RecDecidable_3.of_triple_zero_one_char primrec_ssysInterEqChar
    (fun t => by
      rcases (show ssysInterEqChar t = 0  \/  ssysInterEqChar t = 1 from by
        have := ssysInterEqChar_le_one t; omega) with h | h <;> simp [h])
    (fun n m k => (ssysInterEqChar_eq_one_iff n m k).symm)

/-- **Exercise 7.22j.** `Ssys` -- Scott's positive system generated by singleton
    languages under
concatenation and consistent intersection -- is partially effectively given:
    Definition 7.1
relation (ii) is recursively decidable (`Ssys_cons_computable`, closing C9 / **7.22i(b)
    **).

The name "partially" now undersells the full picture: relation (i) is *also* proven,
    separately,
as `Ssys_interEq_computable` (Exercise 7.22k, `ConsistencyPresentation` was never
    extended with an
`interEq_computable` field to absorb it). Together, `SsysPres` and `
    Ssys_interEq_computable` show
`Ssys` satisfies Definition 7.1 **exactly as Scott states it** -- enumeration, plus
    both relations
recursively decidable -- with nothing deferred. What remains unbuilt is only this
    codebase's
*stronger* `ComputablePresentation` (`Definition71.lean`), which the rest of Lecture
    VII's
formalisation relies on but which is not part of Definition 7.1's text; see the
    section docstring
above. -/
theorem Ssys_partially_effectively_given : PartiallyEffectivelyGiven Ssys := <<
    SsysPres>>

end

end Exercise722
end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.Exercise722
section
set_option linter.unusedVariables false

theorem example_722_presentation : Domain.Recursive.RecDecidable_3 fun (n m k : Nat)
    =>
  @Eq (Set (List Bool))
    (@Inter.inter (Set (List Bool)) (@Set.instInter (List Bool)) (DomainTheory.
    NeighbourhoodSystems.Exercise722.SsysX n)
      (DomainTheory.NeighbourhoodSystems.Exercise722.SsysX m))
    (DomainTheory.NeighbourhoodSystems.Exercise722.SsysX k) :=
  DomainTheory.NeighbourhoodSystems.Exercise722.Ssys_interEq_computable
```

```
end

end DomainTheory.NeighbourhoodSystems.Exercise722
namespace DomainTheory.NeighbourhoodSystems
namespace Exercise718
open NeighborhoodSystem Domain.Recursive ApproximableMap

section
variable {a b : Type*}

namespace EffectiveIso

variable {V : NeighborhoodSystem a} {W : NeighborhoodSystem b}
  {P : ComputablePresentation V} {Q : ComputablePresentation W}

/-- An effective isomorphism is in particular a domain isomorphism (`|D| ?o |E|`): the
    elementwise
maps are mutually inverse (from `left_inv`/`right_inv` via `toElementMap_comp`/`
    toElementMap_idMap`)
and each is monotone (`toElementMap_mono`), so they reflect as well as preserve ` sqsub
     `. -/
def toDomainIso (e : EffectiveIso P Q) : DomainIso V W where
  toFun := e.toMap.toElementMap
  invFun := e.invMap.toElementMap
  left_inv x := by
    rw [ <-  toElementMap_comp, e.left_inv, toElementMap_idMap]
  right_inv y := by
    rw [ <-  toElementMap_comp, e.right_inv, toElementMap_idMap]
  map_rel_iff' := by
    intro x y
    show e.toMap.toElementMap x  <=  e.toMap.toElementMap y  <->  x  <=  y
    refine <fun h => ?_, fun h => e.toMap.toElementMap_mono h>
    have hmono := e.invMap.toElementMap_mono h
    rwa [ <-  toElementMap_comp,  <-  toElementMap_comp, e.left_inv,
    toElementMap_idMap,
      toElementMap_idMap] at hmono

end EffectiveIso
end



section
variable {a b : Type*}

namespace EffectiveIso

variable {V : NeighborhoodSystem a} {W : NeighborhoodSystem b}
  {P : ComputablePresentation V} {Q : ComputablePresentation W}

end EffectiveIso

/-- **Exercise 7.18 -- effective isomorphism implies domain isomorphism.** -/
theorem EffectivelyIsomorphic.isomorphic {V : NeighborhoodSystem a} {W :
    NeighborhoodSystem b}
    {P : ComputablePresentation V} {Q : ComputablePresentation W}
    (h : EffectivelyIsomorphic P Q) : V  ~= ? W :=
```

```
  h.elim fun e => <e.toDomainIso>

variable {V : NeighborhoodSystem a}

variable (P : ComputablePresentation V)

variable {P}

/-- **Exercise 7.18 (Scott 1981, PRG-19).** If `D` is effectively given then `D^ inf
    ~=  (D^ inf )^ inf ` is
effective. -/
theorem iterSys_effectivelyIsomorphic_iterIter (P : ComputablePresentation V) :
    EffectivelyIsomorphic (iterPresentation P) (iterPresentation (iterPresentation P))
     :=
  <iterIterEffectiveIso P>

/-- As a corollary, `D^ inf   ~=  (D^ inf )^ inf ` as domains (Scott's underlying
    isomorphism). -/
theorem iterSys_isomorphic_iterIter (P : ComputablePresentation V) :
    iterSys V  ~= ? iterSys (iterSys V) :=
  (iterSys_effectivelyIsomorphic_iterIter P).isomorphic
end

end Exercise718
end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.Exercise718
section
set_option linter.unusedVariables false

theorem example_718.{u_1} : forall  {a : Type u_1} {V : DomainTheory.
    NeighbourhoodSystems.NeighborhoodSystem a}
  (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V),
  @DomainTheory.NeighbourhoodSystems.Exercise718.EffectivelyIsomorphic (Prod Nat a) (
    Prod Nat (Prod Nat a))
     (@DomainTheory.NeighbourhoodSystems.iterSys a V)
     (@DomainTheory.NeighbourhoodSystems.iterSys (Prod Nat a) (@DomainTheory.
    NeighbourhoodSystems.iterSys a V))
     (@DomainTheory.NeighbourhoodSystems.iterPresentation a V P)
     (@DomainTheory.NeighbourhoodSystems.iterPresentation (Prod Nat a) (@DomainTheory.
    NeighbourhoodSystems.iterSys a V)
       (@DomainTheory.NeighbourhoodSystems.iterPresentation a V P)) :=
  @DomainTheory.NeighbourhoodSystems.Exercise718.
    iterSys_effectivelyIsomorphic_iterIter
end

end DomainTheory.NeighbourhoodSystems.Exercise718
end
```

3.8.8 Proposition 7.7 — lists effectively given

PROPOSITION 7.7. For any effectively given domain D, the domain $D^{\S}$ is also effectively given, and all the combinators of Example 6.1 prove to be computable.

Proof: This proof is essentially an exercise, but it is useful to have an easy-to-grasp example. Indeed, to make things easy to reason about, we can assume that D is a system over $\Delta = \mathbf{N}$, and that in the presentation

where

$$D = \{X_n \mid n \in \mathbf{N}\},$$

the relation $k \in X_n$ is *recursive* in k and n. (It is worth thinking why this is so.) Of course, a lot of other things are recursive also.

Now what kind of a system is $D^{\S}$? The construction of 6.1 made it a system over a certain set of strings Γ. For the sake of checking various assertions about computability, we are transposing everything back to $\mathbf{N}$. (These are all denumerable sets in any case.) The set Γ is divided into three equally big parts, and we can do the same for $\mathbf{N}$. Let us write for any $m, k \in \mathbf{N}$ and subset $X \subseteq \mathbf{N}$:

$$mX + k = \{m \cdot n + k \mid n \in X\}.$$

Then by splitting the integers modulo 3 we have:

$$\mathbf{N} = 3\mathbf{N} \cup (3\mathbf{N} + 1) \cup (3\mathbf{N} + 2),$$

and this equation is quite analogous to that for Γ. We then propose this definition for $D^{\S}$:

$$D^{\S} = \{\mathbf{N}\} \cup \{3X \mid X \in D\} \cup \{(3X + 1) \cup (3Y + 2) \mid X, Y \in D^{\S}\},$$

but this does not make the enumeration of $D^{\S}$ all that obvious. This is one way to do it:

$$V_0 = \mathbf{N}\ ;\ V_{2n+1} = 3X_n\ ;\ V_{2n+2} = (3V_{p(n)} + 1) \cup (3V_{q(n)} + 2).$$

Here p and q are the inverse of the pairing functions mentioned in 7.4. They must be chosen so that $p(n) < n$ and $q(n) < n$ for all $n \in \mathbf{N}$. Thus, in calculating V_k where $k = 2n + 2$ we will be using $V_{p(n)}$ and $V_{q(n)}$ where both subscripts are strictly less than k. This observation is required so that $m \in V_k$ is going to be a recursive relation. What we claim is that

$$D^{\S} = \{V_k \mid k \in \mathbf{N}\}.$$

It should be clear that everything on the right belongs to $D^{\S}$. What needs an inductive argument is that everything in $D^{\S}$ is eventually of the form V_k. But this should be fairly obvious owing to the properties of $r : \mathbf{N} \times \mathbf{N} \leftrightarrow \mathbf{N}$.

The reader also has to check that 7.1(i)–(ii) hold for the V_k. The idea is that any such check is either (1) trivial, or (2) something already assumed about D and the X_n, or (3) can be thrown back to some sets V_m with strictly smaller subscripts. Therefore, the checks will give an answer in finite time according to an effective reduction.

Next for the combinators, we have to translate neighbourhood relations into relations among integer indices. A selection of examples must suffice,

$$X_n\ (\lambda x.\, x^{\S})\ V_k \text{ iff } V_{2n+1} \subseteq V_k$$

$$V_m\ \mathrm{proj}_0\ V_k \text{ iff } k = 0 \text{ or } \exists n \in \mathbf{N}.\, m = 2n + 2 \text{ and } V_{p(n)} \subseteq V_k.$$

The reader should write out other cases. □

Proposition_7_7.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.Exercise_6_18
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Exercise_7_21

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem Domain.Recursive Example61
variable {a : Type*}
namespace Proposition77
/- concordance: source L4433 PROPOSITION 7.7 lists effectively given -/
/-- **Proposition 7.7 (Scott 1981, PRG-19).** Scott's enumeration of `D^S`:
`V_0 = Gamma`, `V_{2n+1} = 0*X_n`, `V_{2n+2} = 1*V_{p n}  U  2*V_{q n}` (`p = unpair
    .1`, `q = unpair.2`). -/
def Vsharp (D : NeighborhoodSystem a) (P : ComputablePresentation D) :
    Nat  ->  Set (List Bool  x  a)
  | 0 => Gamma D
  | (k + 1) =>
      if (k + 1) % 2 = 1 then
        embZero (P.X (k / 2))
      else
        embPair (Vsharp D P ((k - 1) / 2).unpair.1) (Vsharp D P ((k - 1) / 2).unpair
    .2)
  termination_by k => k
  decreasing_by
    * have := unpair_fst_le ((k - 1) / 2); omega
    * have := unpair_snd_le ((k - 1) / 2); omega

end Proposition77
end

section
open NeighborhoodSystem Domain.Recursive Example61
variable {a : Type*}

namespace Proposition77

variable {D : NeighborhoodSystem a} (P : ComputablePresentation D)

/-- One-step unfolding of `Vsharp` at a successor index. -/
theorem Vsharp_succ (k : Nat) :
    Vsharp D P (k + 1) =
      if (k + 1) % 2 = 1 then embZero (P.X (k / 2))
      else embPair (Vsharp D P ((k - 1) / 2).unpair.1) (Vsharp D P ((k - 1) / 2).
    unpair.2) := by
  rw [Vsharp]

theorem Vsharp_even (n : Nat) :
    Vsharp D P (2 * n + 2) = embPair (Vsharp D P n.unpair.1) (Vsharp D P n.unpair.2)
    := by
  rw [show 2 * n + 2 = (2 * n + 1) + 1 from rfl, Vsharp_succ, ite_eq_right (by omega),
    show (2 * n + 1 - 1) / 2 = n from by omega]
```

```
theorem Vsharp_even_inter_even (a b : Nat) :
    Vsharp D P (2 * a + 2)  I  Vsharp D P (2 * b + 2)
      = embPair (Vsharp D P a.unpair.1  I  Vsharp D P b.unpair.1)
                (Vsharp D P a.unpair.2  I  Vsharp D P b.unpair.2) := by
  rw [Vsharp_even, Vsharp_even, embPair_inter]

theorem Vsharp_odd (n : Nat) : Vsharp D P (2 * n + 1) = embZero (P.X n) := by
  rw [Vsharp_succ, ite_eq_left (by omega), show 2 * n / 2 = n from by omega]

theorem Vsharp_odd_inter_even (a b : Nat) :
    Vsharp D P (2 * a + 1)  I  Vsharp D P (2 * b + 2) = {} := by
  rw [Vsharp_odd, Vsharp_even, embZero_inter_embPair]

theorem Vsharp_even_inter_odd (a b : Nat) :
    Vsharp D P (2 * a + 2)  I  Vsharp D P (2 * b + 1) = {} := by
  rw [Set.inter_comm]; exact Vsharp_odd_inter_even P b a

theorem Vsharp_odd_inter_odd (a b : Nat) :
    Vsharp D P (2 * a + 1)  I  Vsharp D P (2 * b + 1) = embZero (P.X a  I  P.X b) :=
    by
  rw [Vsharp_odd, Vsharp_odd, embZero_inter]

theorem Vsharp_zero : Vsharp D P 0 = Gamma D := by rw [Vsharp]

theorem Vsharp_eq_Gamma_iff (hD : forall  X, D.mem X  ->  X.Nonempty) (j : Nat) :
    Vsharp D P j = Gamma D  <->  j = 0 := by
  constructor
  * intro h
    rcases nat_shape j with rfl | <a, rfl> | <a, rfl>
    * rfl
    * rw [Vsharp_odd] at h; exact absurd h (embZero_ne_Gamma D hD (P.X a))
    * rw [Vsharp_even] at h; exact absurd h (embPair_ne_Gamma D hD _ _)
  * rintro rfl; exact Vsharp_zero P

/-- Every `V k` is a neighbourhood of `D^S`. -/
theorem Vsharp_mem (k : Nat) : MemS D (Vsharp D P k) := by
  induction k using Nat.strong_induction_on with
  | _ k ih =>
    rcases Nat.eq_zero_or_pos k with rfl | hk
    * rw [Vsharp_zero]; exact MemS.gamma
    * -- `k  >=  1` is odd `2n+1` or even `2n+2`, decided choice-free by `k % 2`.
      rcases (by
        rcases Nat.lt_or_ge (k % 2) 1 with h | h
        * exact Or.inr <k / 2 - 1, by omega>
        * exact Or.inl <k / 2, by omega> :
          (exists  n, k = 2 * n + 1)  \/  (exists  n, k = 2 * n + 2)) with <n, rfl> |
    <n, rfl>
      * rw [Vsharp_odd]; exact MemS.zero (P.mem_X n)
      * rw [Vsharp_even]
        exact MemS.pair (ih _ (by have := unpair_fst_le n; omega))
          (ih _ (by have := unpair_snd_le n; omega))

theorem Vsharp_inter_zero (n : Nat) : Vsharp D P n  I  Vsharp D P 0 = Vsharp D P n :=
    by
  rw [Vsharp_zero, Set.inter_eq_left]; exact memS_subset_gamma (Vsharp_mem P n)
```

```
/-- Under `{}  notin  ?`, no `V k` is empty. -/
theorem Vsharp_nonempty (hD : forall  X, D.mem X  ->  X.Nonempty) (k : Nat) : (Vsharp
    D P k).Nonempty :=
  memS_nonempty hD (Vsharp_mem P k)

/-- Every neighbourhood of `D^S` is enumerated as some `V k` (surjectivity of `V`). -/
theorem Vsharp_surj {W : Set (List Bool  x  a)} (hW : MemS D W) : exists  k, Vsharp D
    P k = W := by
  induction hW with
  | gamma => exact <0, Vsharp_zero P>
  | @zero X hX => obtain <n, hn> := P.surj hX; exact <2 * n + 1, by rw [Vsharp_odd, hn
    ]>
  | @pair Pp Qq _ _ ihP ihQ =>
      obtain <a, ha> := ihP
      obtain <b, hb> := ihQ
      refine <2 * Nat.pair a b + 2, ?_>
      have h1 : (Nat.pair a b).unpair.1 = a := by rw [unpair_pair]
      have h2 : (Nat.pair a b).unpair.2 = b := by rw [unpair_pair]
      rw [Vsharp_even, h1, h2, ha, hb]

theorem Vsharp_zero_inter (m : Nat) : Vsharp D P 0  I  Vsharp D P m = Vsharp D P m :=
    by
  rw [Vsharp_zero, Set.inter_eq_right]; exact memS_subset_gamma (Vsharp_mem P m)

theorem exists_sub_embPair_iff (hD : forall  X, D.mem X  ->  X.Nonempty)
    (A B : Set (List Bool  x  a)) :
    (exists  l, Vsharp D P l  subseteq  embPair A B)  <->
      (exists  l, Vsharp D P l  subseteq  A)  /\  (exists  l, Vsharp D P l  subseteq
    B) := by
  constructor
  * rintro <l, hl>
    obtain <Pp, Qq, hP, hQ, hWeq> := memS_sub_embPair hD (Vsharp_mem P l) hl
    rw [hWeq] at hl
    obtain <hPA, hQB> := embPair_subset.mp hl
    obtain <l1, h1> := Vsharp_surj P hP
    obtain <l2, h2> := Vsharp_surj P hQ
    exact <<l1, h1  |>  hPA>, <l2, h2  |>  hQB>>
  * rintro <<l1, h1>, <l2, h2>>
    obtain <l, hl> := Vsharp_surj P (MemS.pair (Vsharp_mem P l1) (Vsharp_mem P l2))
    exact <l, by rw [hl]; exact embPair_subset.mpr <h1, h2>>

theorem exists_sub_embZero_iff (hD : forall  X, D.mem X  ->  X.Nonempty) (Y : Set a) :
    (exists  l, Vsharp D P l  subseteq  embZero Y)  <->  exists  k, P.X k  subseteq  Y
     := by
  constructor
  * rintro <l, hl>
    obtain <Z, hZ, hWZ> := memS_sub_embZero hD (Vsharp_mem P l) hl
    rw [hWZ] at hl
    obtain <k, hk> := P.surj hZ
    exact <k, hk  |>  embZero_subset.mp hl>
  * rintro <k, hk>
    exact <2 * k + 1, by rw [Vsharp_odd]; exact embZero_subset.mpr hk>

end Proposition77
end
```

```
section
open NeighborhoodSystem Domain.Recursive Example61
variable {a : Type*}

namespace Proposition77

variable {D : NeighborhoodSystem a} (P : ComputablePresentation D)

variable (fcons feq finter : Nat  ->  Nat)

theorem dsharp_decider_spec (hD : forall  X, D.mem X  ->  X.Nonempty)
    (hcons : forall  a b, fcons (Nat.pair a b) = 1  <->  exists  l, P.X l  subseteq  P
    .X a  I  P.X b)
    (heq : forall  a b, feq (Nat.pair a b) = 1  <->  P.X a = P.X b)
    (hinter : forall  a b, (exists  l, P.X l  subseteq  P.X a  I  P.X b)  ->  P.X (
    finter (Nat.pair a b)) = P.X a  I  P.X b) :
    forall  i j : Nat,
      (consB (gOf (dsharpStep fcons feq finter) (Nat.pair i j)) = 1
           <->  exists  l, Vsharp D P l  subseteq  Vsharp D P i  I  Vsharp D P j)  /\
        ((exists  l, Vsharp D P l  subseteq  Vsharp D P i  I  Vsharp D P j)  ->
          Vsharp D P (intI (gOf (dsharpStep fcons feq finter) (Nat.pair i j)))
            = Vsharp D P i  I  Vsharp D P j)  /\
        (eqB (gOf (dsharpStep fcons feq finter) (Nat.pair i j)) = 1
          <->  Vsharp D P i = Vsharp D P j) := by
  have key : forall  w : Nat, forall  i j : Nat, Nat.pair i j = w  ->
      (consB (gOf (dsharpStep fcons feq finter) (Nat.pair i j)) = 1
           <->  exists  l, Vsharp D P l  subseteq  Vsharp D P i  I  Vsharp D P j)  /\
        ((exists  l, Vsharp D P l  subseteq  Vsharp D P i  I  Vsharp D P j)  ->
          Vsharp D P (intI (gOf (dsharpStep fcons feq finter) (Nat.pair i j)))
            = Vsharp D P i  I  Vsharp D P j)  /\
        (eqB (gOf (dsharpStep fcons feq finter) (Nat.pair i j)) = 1
          <->  Vsharp D P i = Vsharp D P j) := by
    intro w
    induction w using Nat.strong_induction_on with
    | _ w ih =>
      intro i j hw
      rcases nat_shape i with rfl | <a, rfl> | <a, rfl>
      * -- i = 0 (Gamma  I  V_j = V_j)
        simp only [gOf_def]
        rw [dsharpStep_i0 fcons feq finter j]
        simp only [eqB_packT, consB_packT, intI_packT]
        refine <?_, ?_, ?_>
        * exact <fun _ => <j, (Vsharp_zero_inter P j).ge>, fun _ => trivial>
        * intro _; rw [Vsharp_zero_inter P]
        * constructor
          * intro h
            have hj : j = 0 := by omega
            rw [hj]
          * intro h
            have hj : j = 0 := (Vsharp_eq_Gamma_iff P hD j).mp (h.symm.trans (
  Vsharp_zero P))
            omega
      * -- i = 2a+1 (leaf)
        rcases nat_shape j with rfl | <b, rfl> | <b, rfl>
        * -- j = 0
```

```
      simp only [gOf_def]
      rw [dsharpStep_j0 fcons feq finter _ (by omega : 2 * a + 1  !=  0)]
      simp only [eqB_packT, consB_packT, intI_packT]
      refine <?_, ?_, ?_>
      * exact <fun _ => <2 * a + 1, (Vsharp_inter_zero P (2 * a + 1)).ge>, fun _
=> trivial>
      * intro _; exact (Vsharp_inter_zero P (2 * a + 1)).symm
      * rw [Vsharp_zero P, Vsharp_eq_Gamma_iff P hD]; omega
    * -- j = 2b+1 (leaf  I  leaf)
      simp only [gOf_def]
      rw [dsharpStep_ll fcons feq finter a b]
      simp only [eqB_packT, consB_packT, intI_packT]
      refine <?_, ?_, ?_>
      * rw [Vsharp_odd_inter_odd P, exists_sub_embZero_iff P hD]; exact hcons a b
      * intro hp
        rw [Vsharp_odd_inter_odd P] at hp  |-
        rw [exists_sub_embZero_iff P hD] at hp
        rw [Vsharp_odd P, hinter a b hp]
      * rw [Vsharp_odd P, Vsharp_odd P, heq a b]
        exact <fun h => by rw [h], fun h => embZero_injective h>
    * -- j = 2b+2 (leaf  I  node : inconsistent)
      simp only [gOf_def]
      rw [dsharpStep_ln fcons feq finter a b]
      simp only [eqB_packT, consB_packT, intI_packT]
      refine <?_, ?_, ?_>
      * rw [Vsharp_odd_inter_even P]
        constructor
        * intro h; exact absurd h (by decide)
        * rintro <l, hl>
          obtain <x, hx> := Vsharp_nonempty P hD l
          exact absurd (hl hx) (by simp)
      * rw [Vsharp_odd_inter_even P]
        rintro <l, hl>
        obtain <x, hx> := Vsharp_nonempty P hD l
        exact absurd (hl hx) (by simp)
      * rw [Vsharp_odd P, Vsharp_even P]
        constructor
        * intro h; exact absurd h (by decide)
        * intro h; exact absurd h (embZero_ne_embPair D hD (P.mem_X a) _ _)
  * -- i = 2a+2 (node)
    rcases nat_shape j with rfl | <b, rfl> | <b, rfl>
    * -- j = 0
      simp only [gOf_def]
      rw [dsharpStep_j0 fcons feq finter _ (by omega : 2 * a + 2  !=  0)]
      simp only [eqB_packT, consB_packT, intI_packT]
      refine <?_, ?_, ?_>
      * exact <fun _ => <2 * a + 2, (Vsharp_inter_zero P (2 * a + 2)).ge>, fun _
=> trivial>
      * intro _; exact (Vsharp_inter_zero P (2 * a + 2)).symm
      * rw [Vsharp_zero P, Vsharp_eq_Gamma_iff P hD]; omega
    * -- j = 2b+1 (node  I  leaf : inconsistent)
      simp only [gOf_def]
      rw [dsharpStep_nl fcons feq finter a b]
      simp only [eqB_packT, consB_packT, intI_packT]
      refine <?_, ?_, ?_>
      * rw [Vsharp_even_inter_odd P]
```

```
            constructor
            * intro h; exact absurd h (by decide)
            * rintro <l, hl>
              obtain <x, hx> := Vsharp_nonempty P hD l
              exact absurd (hl hx) (by simp)
          * rw [Vsharp_even_inter_odd P]
            rintro <l, hl>
            obtain <x, hx> := Vsharp_nonempty P hD l
            exact absurd (hl hx) (by simp)
          * rw [Vsharp_even P, Vsharp_odd P]
            constructor
            * intro h; exact absurd h (by decide)
            * intro h; exact absurd h.symm (embZero_ne_embPair D hD (P.mem_X b) _ _)
        * -- j = 2b+2 (node  I  node : recurse)
          have hlt1 : Nat.pair a.unpair.1 b.unpair.1 < Nat.pair (2 * a + 2) (2 * b +
    2) :=
            pair_lt_pair_of_lt (by have := unpair_fst_le a; omega) (by have :=
    unpair_fst_le b; omega)
          have hlt2 : Nat.pair a.unpair.2 b.unpair.2 < Nat.pair (2 * a + 2) (2 * b +
    2) :=
            pair_lt_pair_of_lt (by have := unpair_snd_le a; omega) (by have :=
    unpair_snd_le b; omega)
          have hr1 := listGet_rtbl (dsharpStep fcons feq finter) hlt1
          have hr2 := listGet_rtbl (dsharpStep fcons feq finter) hlt2
          obtain <ihc1, ihi1, ihe1> :=
            ih (Nat.pair a.unpair.1 b.unpair.1) (hw  |>  hlt1) a.unpair.1 b.unpair.1
    rfl
          obtain <ihc2, ihi2, ihe2> :=
            ih (Nat.pair a.unpair.2 b.unpair.2) (hw  |>  hlt2) a.unpair.2 b.unpair.2
    rfl
          simp only [gOf_def]
          rw [dsharpStep_nn fcons feq finter a b]
          simp only [eqB_packT, consB_packT, intI_packT]
          rw [hr1, hr2]
          refine <?_, ?_, ?_>
          * rw [Vsharp_even_inter_even P, exists_sub_embPair_iff P hD, nat_mul_eq_one]
            exact and_congr ihc1 ihc2
          * intro hp
            rw [Vsharp_even_inter_even P] at hp  |-
            rw [exists_sub_embPair_iff P hD] at hp
            obtain <hp1, hp2> := hp
            rw [Vsharp_even P]
            simp only [unpair_pair_fst, unpair_pair_snd]
            rw [ihi1 hp1, ihi2 hp2]
          * rw [Vsharp_even P, Vsharp_even P, nat_mul_eq_one, ihe1, ihe2]
            constructor
            * rintro <h1, h2>; rw [h1, h2]
            * intro h; exact embPair_injective h
  intro i j
  exact key (Nat.pair i j) i j rfl

/-- **Intersection-index correctness in isolation.** The `intI` component of `gOf (
    dsharpStep ...)` is
independent of the `eq`/`cons` bits (`fcons`/`feq`), so it is correct assuming only
    that `finter`
correctly intersects in `D`. This lets the `inter` *data* field be built with dummy `
```

```
    fcons`/`feq`
while staying choice-free. -/
theorem dsharp_intI_correct (hD : forall  X, D.mem X  ->  X.Nonempty)
    (hinter : forall  a b, (exists  l, P.X l  subseteq  P.X a  I  P.X b)  ->  P.X (
    finter (Nat.pair a b)) = P.X a  I  P.X b) :
    forall  i j : Nat, (exists  l, Vsharp D P l  subseteq  Vsharp D P i  I  Vsharp D P
     j)  ->
      Vsharp D P (intI (gOf (dsharpStep fcons feq finter) (Nat.pair i j)))
        = Vsharp D P i  I  Vsharp D P j := by
  have key : forall  w : Nat, forall  i j : Nat, Nat.pair i j = w  ->
      (exists  l, Vsharp D P l  subseteq  Vsharp D P i  I  Vsharp D P j)  ->
      Vsharp D P (intI (gOf (dsharpStep fcons feq finter) (Nat.pair i j)))
        = Vsharp D P i  I  Vsharp D P j := by
    intro w
    induction w using Nat.strong_induction_on with
    | _ w ih =>
      intro i j hw
      rcases nat_shape i with rfl | <a, rfl> | <a, rfl>
      * intro _
        simp only [gOf_def]; rw [dsharpStep_i0 fcons feq finter j]
        simp only [intI_packT]; rw [Vsharp_zero_inter P]
      * rcases nat_shape j with rfl | <b, rfl> | <b, rfl>
        * intro _
          simp only [gOf_def]; rw [dsharpStep_j0 fcons feq finter _ (by omega : 2 * a
    + 1  !=  0)]
          simp only [intI_packT]; exact (Vsharp_inter_zero P (2 * a + 1)).symm
        * intro hp
          simp only [gOf_def]; rw [dsharpStep_ll fcons feq finter a b]
          simp only [intI_packT]
          rw [Vsharp_odd_inter_odd P] at hp  |-
          rw [exists_sub_embZero_iff P hD] at hp
          rw [Vsharp_odd P, hinter a b hp]
        * intro hp
          exfalso
          rw [Vsharp_odd_inter_even P] at hp
          obtain <l, hl> := hp
          obtain <x, hx> := Vsharp_nonempty P hD l
          exact absurd (hl hx) (by simp)
      * rcases nat_shape j with rfl | <b, rfl> | <b, rfl>
        * intro _
          simp only [gOf_def]; rw [dsharpStep_j0 fcons feq finter _ (by omega : 2 * a
    + 2  !=  0)]
          simp only [intI_packT]; exact (Vsharp_inter_zero P (2 * a + 2)).symm
        * intro hp
          exfalso
          rw [Vsharp_even_inter_odd P] at hp
          obtain <l, hl> := hp
          obtain <x, hx> := Vsharp_nonempty P hD l
          exact absurd (hl hx) (by simp)
        * intro hp
          have hlt1 : Nat.pair a.unpair.1 b.unpair.1 < Nat.pair (2 * a + 2) (2 * b +
    2) :=
            pair_lt_pair_of_lt (by have := unpair_fst_le a; omega) (by have :=
    unpair_fst_le b; omega)
          have hlt2 : Nat.pair a.unpair.2 b.unpair.2 < Nat.pair (2 * a + 2) (2 * b +
    2) :=
```

```
            pair_lt_pair_of_lt (by have := unpair_snd_le a; omega) (by have :=
    unpair_snd_le b; omega)
          have hr1 := listGet_rtbl (dsharpStep fcons feq finter) hlt1
          have hr2 := listGet_rtbl (dsharpStep fcons feq finter) hlt2
          rw [Vsharp_even_inter_even P] at hp  |-
          rw [exists_sub_embPair_iff P hD] at hp
          obtain <hp1, hp2> := hp
          simp only [gOf_def]; rw [dsharpStep_nn fcons feq finter a b]
          simp only [intI_packT]; rw [hr1, hr2, Vsharp_even P]
          simp only [unpair_pair_fst, unpair_pair_snd]
          rw [ih _ (hw  |>  hlt1) a.unpair.1 b.unpair.1 rfl hp1,
              ih _ (hw  |>  hlt2) a.unpair.2 b.unpair.2 rfl hp2]
  intro i j; exact key (Nat.pair i j) i j rfl

/-- **Decidability of `7.1(i)` for `D^S`.** `V_n  I  V? = V_k` iff `(n,m)` is
    consistent and the
computed intersection index agrees with `k` under `D^S`-equality. Inconsistency forces
    the product
to `0` (and then `V_n  I  V? = {}  !=  V_k`, since `V_k` is nonempty), so the single
    product decider is
correct on the nose. -/
theorem dsharp_interEq_iff (hD : forall  X, D.mem X  ->  X.Nonempty)
    (hcons : forall  a b, fcons (Nat.pair a b) = 1  <->  exists  l, P.X l  subseteq  P
    .X a  I  P.X b)
    (heq : forall  a b, feq (Nat.pair a b) = 1  <->  P.X a = P.X b)
    (hinter : forall  a b, (exists  l, P.X l  subseteq  P.X a  I  P.X b)  ->  P.X (
    finter (Nat.pair a b)) = P.X a  I  P.X b)
    (n m k : Nat) :
    (Vsharp D P n  I  Vsharp D P m = Vsharp D P k)  <->
      consB (gOf (dsharpStep fcons feq finter) (Nat.pair n m))
        * eqB (gOf (dsharpStep fcons feq finter)
            (Nat.pair (intI (gOf (dsharpStep fcons feq finter) (Nat.pair n m))) k)) =
    1 := by
  obtain <hc0, hi0, _> := dsharp_decider_spec P fcons feq finter hD hcons heq hinter n
     m
  obtain <_, _, he> := dsharp_decider_spec P fcons feq finter hD hcons heq hinter
    (intI (gOf (dsharpStep fcons feq finter) (Nat.pair n m))) k
  rw [nat_mul_eq_one]
  constructor
  * intro hEq
    have hch : exists  l, Vsharp D P l  subseteq  Vsharp D P n  I  Vsharp D P m := <k,
     hEq.ge>
    exact <hc0.mpr hch, he.mpr ((hi0 hch).trans hEq)>
  * rintro <hcb, heb>
    rw [ <-  hi0 (hc0.mp hcb)]; exact he.mp heb

/-- **Proposition 7.7 (Scott 1981, PRG-19).** A computable presentation of `D^S` from
    one of `D`. -/
def dsharpPresentation (hD : forall  X, D.mem X  ->  X.Nonempty) :
    ComputablePresentation (Dsharp D hD) where
  X := Vsharp D P
  mem_X := fun n => Vsharp_mem P n
  surj := fun hY => Vsharp_surj P hY

  interEq_computable := by
    obtain <fc, hfc_pr, hfc> := P.cons_computable
```

```
  obtain <fe, hfe_pr, hfe> := P.eq_computable
  have hcons : forall  a b, fc (Nat.pair a b) = 1  <->  exists  l, P.X l  subseteq
  P.X a  I  P.X b := fun a b => by
    have := hfc (Nat.pair a b); simp only [unpair_pair_fst, unpair_pair_snd] at this
  ; exact this.symm
  have heq : forall  a b, fe (Nat.pair a b) = 1  <->  P.X a = P.X b := fun a b => by
    have := hfe (Nat.pair a b); simp only [unpair_pair_fst, unpair_pair_snd] at this
  ; exact this.symm
  have hinter : forall  a b, (exists  l, P.X l  subseteq  P.X a  I  P.X b)  ->
      P.X ((fun s => P.inter s.unpair.1 s.unpair.2) (Nat.pair a b)) = P.X a  I  P.X
  b := fun a b h => by
    simp only [unpair_pair_fst, unpair_pair_snd]; exact P.inter_spec h
  refine <fun t => consB (gOf (dsharpStep fc fe (fun s => P.inter s.unpair.1 s.
  unpair.2))
      (Nat.pair t.unpair.1 t.unpair.2.unpair.1))
      * eqB (gOf (dsharpStep fc fe (fun s => P.inter s.unpair.1 s.unpair.2))
          (Nat.pair (intI (gOf (dsharpStep fc fe (fun s => P.inter s.unpair.1 s.
  unpair.2))
            (Nat.pair t.unpair.1 t.unpair.2.unpair.1))) t.unpair.2.unpair.2)), ?_,
  fun t => ?_>
  * have hg := primrec_gOf (primrec_dsharpStep fc fe (fun s => P.inter s.unpair.1 s.
  unpair.2)
      hfc_pr hfe_pr P.inter_primrec)
    have hc0 := hg.comp (Nat.Primrec.left.pair (Nat.Primrec.left.comp Nat.Primrec.
  right))
    exact primrec_mul_2 ((Nat.Primrec.left.comp Nat.Primrec.right).comp hc0)
      (Nat.Primrec.left.comp (hg.comp
        (((Nat.Primrec.right.comp Nat.Primrec.right).comp hc0).pair
          (Nat.Primrec.right.comp Nat.Primrec.right))))
  * exact dsharp_interEq_iff P fc fe (fun s => P.inter s.unpair.1 s.unpair.2) hD
  hcons heq hinter
      t.unpair.1 t.unpair.2.unpair.1 t.unpair.2.unpair.2
cons_computable := by
  obtain <fc, hfc_pr, hfc> := P.cons_computable
  obtain <fe, hfe_pr, hfe> := P.eq_computable
  have hcons : forall  a b, fc (Nat.pair a b) = 1  <->  exists  l, P.X l  subseteq
  P.X a  I  P.X b := fun a b => by
    have := hfc (Nat.pair a b); simp only [unpair_pair_fst, unpair_pair_snd] at this
  ; exact this.symm
  have heq : forall  a b, fe (Nat.pair a b) = 1  <->  P.X a = P.X b := fun a b => by
    have := hfe (Nat.pair a b); simp only [unpair_pair_fst, unpair_pair_snd] at this
  ; exact this.symm
  have hinter : forall  a b, (exists  l, P.X l  subseteq  P.X a  I  P.X b)  ->
      P.X ((fun s => P.inter s.unpair.1 s.unpair.2) (Nat.pair a b)) = P.X a  I  P.X
  b := fun a b h => by
    simp only [unpair_pair_fst, unpair_pair_snd]; exact P.inter_spec h
  refine <fun t => consB (gOf (dsharpStep fc fe (fun s => P.inter s.unpair.1 s.
  unpair.2))
      (Nat.pair t.unpair.1 t.unpair.2)), ?_, fun t => ?_>
  * have hg := primrec_gOf (primrec_dsharpStep fc fe (fun s => P.inter s.unpair.1 s.
  unpair.2)
      hfc_pr hfe_pr P.inter_primrec)
    exact (Nat.Primrec.left.comp Nat.Primrec.right).comp
      (hg.comp (Nat.Primrec.left.pair Nat.Primrec.right))
  * exact (dsharp_decider_spec P fc fe (fun s => P.inter s.unpair.1 s.unpair.2) hD
  hcons heq hinter
```

```
        t.unpair.1 t.unpair.2).1.symm
  inter := fun n m => intI (gOf (dsharpStep (fun _ => 0) (fun _ => 0)
    (fun s => P.inter s.unpair.1 s.unpair.2)) (Nat.pair n m))
  inter_primrec := by
    have hg := primrec_gOf (primrec_dsharpStep (fun _ => 0) (fun _ => 0)
      (fun s => P.inter s.unpair.1 s.unpair.2) (Nat.Primrec.const 0) (Nat.Primrec.
    const 0)
      P.inter_primrec)
    have hpair : Nat.Primrec (fun t : Nat => Nat.pair t.unpair.1 t.unpair.2) :=
      Nat.Primrec.left.pair Nat.Primrec.right
    exact (Nat.Primrec.right.comp Nat.Primrec.right).comp (hg.comp hpair)
  inter_spec := by
    intro n m h
    exact dsharp_intI_correct P (fun _ => 0) (fun _ => 0) (fun s => P.inter s.unpair.1
     s.unpair.2) hD
      (fun a b hh => by simp only [unpair_pair_fst, unpair_pair_snd]; exact P.
    inter_spec hh) n m h
  masterIdx := 0
  masterIdx_spec := Vsharp_zero P

/-- **Proposition 7.7 (Scott 1981, PRG-19).** If `D` is effectively given, so is `D^S`.
    -/
theorem dsharp_isEffectivelyGiven (P : ComputablePresentation D)
    (hD : forall  X, D.mem X  ->  X.Nonempty) :
    (Dsharp D hD).IsEffectivelyGiven :=
  <dsharpPresentation P hD>

end Proposition77
end

end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.Proposition77
section
set_option linter.unusedVariables false

theorem theorem_77.{u_1} : forall  {a : Type u_1} {D : DomainTheory.
    NeighbourhoodSystems.NeighborhoodSystem a}
  (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a D)
  (hD : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem
     a D X  ->  @Set.Nonempty a X),
  @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven (Prod (List
     Bool) a)
    (@DomainTheory.NeighbourhoodSystems.Example61.Dsharp a D hD) :=
  @DomainTheory.NeighbourhoodSystems.Proposition77.dsharp_isEffectivelyGiven
end

end DomainTheory.NeighbourhoodSystems.Proposition77
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive Example61

section
variable {a : Type*}

namespace Proposition77

variable {D : NeighborhoodSystem a} (hD : forall  X, D.mem X  ->  X.Nonempty)
```

```
@[simp] theorem dsharpPresentation_X (P : ComputablePresentation D) (n : Nat) :
    (dsharpPresentation P hD).X n = Vsharp D P n := rfl

/-- **Example 6.1's injection `fun x. x^S` as an approximable map.** Scott's
    neighbourhood relation
`X (fun x.x^S) W  <->  0*X  subseteq  W` (`embZero X  subseteq  W`). The filter laws
    are immediate: `inter_right` uses
`memS_inter` (with witness `0*X`), and `mono` uses monotonicity of `embZero`. -/
def inSharpMap : ApproximableMap D (Dsharp D hD) where
  rel X W := D.mem X  /\  MemS D W  /\  embZero X  subseteq  W
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := <D.master_mem, MemS.gamma, embZero_subset_Gamma D.master_mem>
  inter_right := by
    rintro X W W' <hX, hW, hsub> <-, hW', hsub'>
    exact <hX, memS_inter hD hW hW' (MemS.zero hX) (Set.subset_inter hsub hsub'),
      Set.subset_inter hsub hsub'>
  mono := by
    rintro X X' W W' <hX, hW, hsub> hX'X hWW' hX'mem hW'mem
    exact <hX'mem, hW'mem, (embZero_subset.mpr hX'X).trans (hsub.trans hWW')>

@[simp] theorem inSharpMap_rel {X : Set a} {W : Set (List Bool  x  a)} :
    (inSharpMap hD).rel X W  <->  D.mem X  /\  MemS D W  /\  embZero X  subseteq  W :=
     Iff.rfl

/-- **The first projection of the pair part of `D^S`, as an approximable map.** On the
    pair summand
`1*P  U  2*Q` it returns `P`; everywhere else it returns `False = {Gamma}`. The
    neighbourhood relation is
`W proj_0 Z  <->  Z = Gamma  \/  exists  P Q, W = 1*P  U  2*Q  /\  P  subseteq  Z`. -/
def proj0Map : ApproximableMap (Dsharp D hD) (Dsharp D hD) where
  rel W Z := MemS D W  /\  MemS D Z  /\  (Z = Gamma D  \/  exists  Pp Qq, W = embPair
    Pp Qq  /\  Pp  subseteq  Z)
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := <MemS.gamma, MemS.gamma, Or.inl rfl>
  inter_right := by
    rintro W Z Z' <hW, hZ, hd> <-, hZ', hd'>
    have hgZ : Z  subseteq  Gamma D := memS_subset_gamma hZ
    have hgZ' : Z'  subseteq  Gamma D := memS_subset_gamma hZ'
    rcases hd with rfl | <Pp, Qq, hWeq, hPZ>
    * rw [Set.inter_eq_right.mpr hgZ']; exact <hW, hZ', hd'>
    * rcases hd' with rfl | <Pp', Qq', hWeq', hPZ'>
      * rw [Set.inter_eq_left.mpr hgZ]; exact <hW, hZ, Or.inr <Pp, Qq, hWeq, hPZ>>
      * have hPP : Pp = Pp' := (embPair_injective (hWeq.symm.trans hWeq')).1
        have hPsub : Pp  subseteq  Z  I  Z' := Set.subset_inter hPZ (by rw [hPP];
    exact hPZ')
        have hMemP : MemS D Pp := (memS_embPair_inv hD (hWeq  |>  hW)).1
        exact <hW, memS_inter hD hZ hZ' hMemP hPsub, Or.inr <Pp, Qq, hWeq, hPsub>>
  mono := by
    rintro W W' Z Z' <hW, hZ, hd> hW'W hZZ' hW'mem hZ'mem
    refine <hW'mem, hZ'mem, ?_>
    rcases hd with rfl | <Pp, Qq, hWeq, hPZ>
    * exact Or.inl (Set.Subset.antisymm (memS_subset_gamma hZ'mem) hZZ')
    * subst hWeq
```

```
      cases hW'mem with
      | gamma =>
        exact absurd (Set.Subset.antisymm (memS_subset_gamma hW) hW'W)
          (embPair_ne_Gamma D hD Pp Qq)
      | @zero X hX =>
        exfalso
        obtain <a, ha> := hD X hX
        rcases hW'W (show (([], a) : List Bool  x  a)  in  embZero X from <rfl, ha>)
    with
          <p', hp', -> | <q', hq', ->
        * simp at hp'
        * simp at hq'
      | @pair Pp' Qq' hP' hQ' =>
        obtain <hP'P, -> := embPair_subset.mp hW'W
        exact Or.inr <Pp', Qq', rfl, fun z hz => hZZ' (hPZ (hP'P hz))>

@[simp] theorem proj0Map_rel {W Z : Set (List Bool  x  a)} :
    (proj0Map hD).rel W Z  <->
      MemS D W  /\  MemS D Z  /\  (Z = Gamma D  \/  exists  Pp Qq, W = embPair Pp Qq
    /\  Pp  subseteq  Z) := Iff.rfl

/-- **`inSharpMap` really computes `inSharp`.** Its elementwise action is Scott's `fun
    x. x^S`. -/
theorem inSharpMap_toElementMap (x : D.Element) :
    (inSharpMap hD).toElementMap x = inSharp D hD x := by
  apply Element.ext
  intro W
  simp only [ApproximableMap.mem_toElementMap, inSharpMap_rel]
  constructor
  * rintro <X, hXx, hX, hW, hsub>
    cases hW with
    | gamma => exact Or.inl rfl
    | @zero Y hY => exact Or.inr <Y, x.up_mem hXx hY (embZero_subset.mp hsub), rfl>
    | @pair Pp Qq hP hQ =>
      exfalso
      obtain <a, ha> := hD X hX
      rcases hsub (show (([], a) : List Bool  x  a)  in  embZero X from <rfl, ha>)
    with
        <p', hp', -> | <q', hq', ->
      * simp at hp'
      * simp at hq'
  * rintro (rfl | <Y, hYx, rfl>)
    * exact <D.master, x.master_mem, D.master_mem, MemS.gamma, embZero_subset_Gamma D.
    master_mem>
    * exact <Y, hYx, x.sub hYx, MemS.zero (x.sub hYx), subset_rfl>

/-- **Proposition 7.7 (Scott 1981, PRG-19) -- `fun x. x^S` is computable.**
`X_n (fun x.x^S) V_k  <->  V_{2n+1}  subseteq  V_k` (`embZero X_n  subseteq  V_k`), a
    recursive slice of the inclusion
decider for `D^S`, hence r.e. -/
theorem inSharp_isComputable (P : ComputablePresentation D) :
    ComputableMap P (dsharpPresentation P hD) (inSharpMap hD) := by
  constructor
  show REPred (fun s => (inSharpMap hD).rel (P.X s.unpair.1) (Vsharp D P s.unpair.2))
  have hincl : RecDecidable (fun s => Vsharp D P s.unpair.1  subseteq  Vsharp D P s.
    unpair.2) :=
```

```
    (dsharpPresentation P hD).incl_computable
  have hr : Nat.Primrec (fun t => Nat.pair (2 * t.unpair.1 + 1) t.unpair.2) :=
    (primrec_add_2 (primrec_mul_2 (Nat.Primrec.const 2) Nat.Primrec.left)
      (Nat.Primrec.const 1)).pair Nat.Primrec.right
  refine (RecDecidable.of_iff (fun t => ?_) (hincl.comp hr)).re
  simp only [inSharpMap_rel, unpair_pair_fst, unpair_pair_snd, Vsharp_odd]
  exact <fun h => h.2.2, fun h => <P.mem_X _, Vsharp_mem P _, h>>

/-- The index characterization of `proj0Map` against the `Vsharp` enumeration. `V_m
    proj_0 V_k` iff
`k = 0` (`V_k = Gamma`) or `m` is an even node index `2a+2` with `V_{p a}  subseteq  V_k
    ` (the left child). -/
theorem proj0_rel_Vsharp_iff (P : ComputablePresentation D) (m k : Nat) :
    (proj0Map hD).rel (Vsharp D P m) (Vsharp D P k)  <->
      k = 0  \/  (m % 2 = 0  /\  m  !=  0  /\  Vsharp D P (m / 2 - 1).unpair.1
    subseteq  Vsharp D P k) := by
  rw [proj0Map_rel, and_iff_right (Vsharp_mem P m), and_iff_right (Vsharp_mem P k)]
  refine or_congr (Vsharp_eq_Gamma_iff P hD k) ?_
  rcases nat_shape m with rfl | <a, rfl> | <a, rfl>
  * rw [Vsharp_zero]
    constructor
    * rintro <Pp, Qq, heq, ->; exact absurd heq.symm (embPair_ne_Gamma D hD Pp Qq)
    * rintro <-, hne, ->; exact absurd rfl hne
  * rw [Vsharp_odd]
    constructor
    * rintro <Pp, Qq, heq, ->; exact absurd heq (embZero_ne_embPair D hD (P.mem_X a)
    Pp Qq)
    * rintro <hmod, -, ->; exact absurd hmod (by omega)
  * rw [Vsharp_even, show (2 * a + 2) / 2 - 1 = a from by omega]
    constructor
    * rintro <Pp, Qq, heq, hPk>
      obtain <hP, -> := embPair_injective heq
      exact <by omega, by omega, by rw [hP]; exact hPk>
    * rintro <-, -, hsub>
      exact <Vsharp D P a.unpair.1, Vsharp D P a.unpair.2, rfl, hsub>

/-- **Proposition 7.7 (Scott 1981, PRG-19) -- `proj_0` is computable.**
`V_m proj_0 V_k  <->  k = 0  \/  (m even, m  !=  0, V_{p(m/2-1)}  subseteq  V_k)`: a
    disjunction of the equality and
inclusion deciders with primitive-recursive parity/child reindexings, hence r.e. -/
theorem proj0_isComputable (P : ComputablePresentation D) :
    ComputableMap (dsharpPresentation P hD) (dsharpPresentation P hD) (proj0Map hD) :=
     by
  constructor
  show REPred (fun s => (proj0Map hD).rel (Vsharp D P s.unpair.1) (Vsharp D P s.unpair
    .2))
  have hk0 : RecDecidable (fun s => s.unpair.2 = 0) :=
    RecDecidable.natEq Nat.Primrec.right (Nat.Primrec.const 0)
  have hmod : RecDecidable (fun s => s.unpair.1 % 2 = 0) :=
    RecDecidable.natEq (primrec_mod2.comp Nat.Primrec.left) (Nat.Primrec.const 0)
  have hne : RecDecidable (fun s => s.unpair.1  !=  0) :=
    (RecDecidable.natEq Nat.Primrec.left (Nat.Primrec.const 0)).not
  have hg : Nat.Primrec (fun s => Nat.pair (s.unpair.1 / 2 - 1).unpair.1 s.unpair.2)
    :=
    (Nat.Primrec.left.comp (primrec_sub_2 (primrec_div2.comp Nat.Primrec.left)
      (Nat.Primrec.const 1))).pair Nat.Primrec.right
```

```
  have hincl : RecDecidable (fun s => Vsharp D P (s.unpair.1 / 2 - 1).unpair.1
    subseteq
      Vsharp D P s.unpair.2) := by
    refine RecDecidable.of_iff (fun s => ?_) ((dsharpPresentation P hD).
    incl_computable.comp hg)
    simp only [dsharpPresentation_X, unpair_pair_fst, unpair_pair_snd]
  exact (RecDecidable.of_iff (fun s => proj0_rel_Vsharp_iff hD P s.unpair.1 s.unpair
    .2)
    (hk0.or (hmod.and (hne.and hincl)))).re

/-- **`proj0Map` really is the first projection of the pair part:** `proj_0(<x, y>^S) =
    x`. -/
theorem proj0_toElementMap_pairSharp (x y : (Dsharp D hD).Element) :
    (proj0Map hD).toElementMap (pairSharp D hD x y) = x := by
  apply Element.ext
  intro Z
  simp only [ApproximableMap.mem_toElementMap, proj0Map_rel]
  constructor
  * rintro <W, hWmem, hMW, hMZ, hdisj>
    rcases hWmem with rfl | <Pp, Qq, hxP, hyQ, rfl>
    * rcases hdisj with rfl | <Pp', Qq', hPe, ->
      * exact x.master_mem
      * exact absurd hPe.symm (embPair_ne_Gamma D hD Pp' Qq')
    * rcases hdisj with rfl | <Pp', Qq', hPe, hP'Z>
      * exact x.master_mem
      * obtain <hPP, -> := embPair_injective hPe
        exact x.up_mem hxP hMZ (by rw [hPP]; exact hP'Z)
  * intro hxZ
    exact <embPair Z (Gamma D), Or.inr <Z, Gamma D, hxZ, y.master_mem, rfl>,
      MemS.pair (x.sub hxZ) MemS.gamma, x.sub hxZ, Or.inr <Z, Gamma D, rfl, subset_rfl
    >>

end Proposition77
end

section
variable {a : Type*}

namespace Proposition77

variable {D : NeighborhoodSystem a} (P : ComputablePresentation D)

theorem gList_length (step : Nat  ->  Nat) (w : Nat) : (gList step w).length = w := by
  induction w with
  | zero => rfl
  | succ w ih => rw [gList, List.length_cons, ih]

end Proposition77
end

namespace Exercise722
open Set Domain.Recursive

section
/-- Default neighbourhood for junk / empty-syntax indices: the master `Sigma = Set.univ
    `. -/
```

```
def SsysX_default : Set (List Bool) := Set.univ
end

end Exercise722
open NeighborhoodSystem Domain.Recursive ApproximableMap

section
variable {a b ? a' b' : Type*}
variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}

/-- Scott's `W_k = X?_{p(k)}  U  X1_{q(k)}` with `p = *.unpair.1`, `q = *.unpair.2`. -/
def prodEnum (P_0 : ComputablePresentation V_0) (P_1 : ComputablePresentation V_1) (t
    : Nat) :
    Set (a ? b) :=
  prodNbhd (P_0.X t.unpair.1) (P_1.X t.unpair.2)

@[simp] theorem prodEnum_apply (P_0 : ComputablePresentation V_0) (P_1 :
    ComputablePresentation V_1)
    (t : Nat) : prodEnum P_0 P_1 t = prodNbhd (P_0.X t.unpair.1) (P_1.X t.unpair.2) :=
     rfl

end

section
variable {a b ? a' b' : Type*}

/-- The product representation is determined by its two components (the ` <-> ` form of
`prodNbhd_injective`). -/
theorem prodNbhd_eq_iff {X X' : Set a} {Y Y' : Set b} :
    prodNbhd X Y = prodNbhd X' Y'  <->  X = X'  /\  Y = Y' :=
  <prodNbhd_injective, by rintro <rfl, rfl>; rfl>
end

end DomainTheory.NeighbourhoodSystems
end
```

3.8.9 Example 7.8 — powerset effectively given

EXAMPLE 7.8. We have often made reference to the powerset $P\mathbf{N}$ as a domain and we should check here that it is effectively given. One easy way to see this is to note

$$P\mathbf{N} \cong |T^{\infty}|.$$

The (slight) trouble with $P\mathbf{N}$ is that we usually think of it in terms of *elements* rather than *neighbourhoods*. Going back to Exercise 1.16, we can argue that the neighbourhoods of $P\mathbf{N}$ are ordered not like the finite sets of integers but in the partial ordering *converse* to that. But this is of no trouble, since all will be decidable. What we need first is an enumeration of all finite sets of integers. We can do this by:

$$E_n = \{k \mid \exists i, j.\, i < 2^k \text{ and } n = i + 2^k + j \cdot 2^{k+1}\}.$$

The idea is that $k \in E_n$ means that the exponent k does occur in the binary expansion of n as a sum of powers of 2. All finite subsets of $\mathbf{N}$ are of the form E_n. We then find that as a neighbourhood system

$$(P\mathbf{N}) = \{\mathbf{N} \setminus E_n \mid n \in \mathbf{N}\}.$$

As the relationship $E_n \cup E_m = E_k$ is recursive, there is no trouble in proving that this is a computable presentation. In this system, of course, any two neighbourhoods are consistent. Various combinators on $P\mathbf{N}$ are suggested in Exercise 7.23. □

We end this chapter with an example of another kind of domain construct. This construct is known as the *Smyth Power Domain*. It is defined for any neighbourhood system $\mathcal{D}$ and results in a new system we shall call here $\mathbb{P}\mathcal{D}$. The elements of $\mathbb{P}\mathcal{D}$ behave rather like *sets of elements* of $\mathcal{D}$, but since our elements can be either partial or total, there are certain dangers to pushing the analogy too far. For some purposes a rival construct called the *Plotkin Power Domain* is better, but it leads outside the category of neighbourhood systems as defined in these lectures. Do not confuse $P\mathbf{N}$ with $\mathbb{P}\mathcal{D}$.

Example_7_8.lean

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Definition_7_1

public import Mathlib.Data.Nat.Bitwise

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.Example78
open Domain.Recursive NeighborhoodSystem

section
/-- The neighbourhood `Nat ? E_n` of the powerset domain: the cofinite set of bit
    positions **not**
set in `n`. Scott's `E_n` is `{k ? n.testBit k}`, so `nbhd n = {k ? n.testBit k = false
    }`. -/
def nbhd (n : Nat) : Set Nat := {k | n.testBit k = false}
end

section
@[simp] theorem mem_nbhd {n k : Nat} : k  in  nbhd n  <->  n.testBit k = false := Iff.
    rfl
end

section
/-- The enumeration `n  |->  nbhd n` is **one-one** (Scott's converse-ordered
    neighbourhoods are in
bijection with the indices): from `nbhd n = nbhd m` we recover `n = m` bit by bit. -/
theorem nbhd_injective {n m : Nat} (h : nbhd n = nbhd m) : n = m := by
  apply Nat.eq_of_testBit_eq
  intro k
  have hk : (n.testBit k = false)  <->  (m.testBit k = false) := by
    have := Set.ext_iff.mp h k; simpa only [mem_nbhd] using this
  cases hn : n.testBit k <;> cases hm : m.testBit k <;> simp_all
end

section
/-- **Scott's `E_n  U  E? = E_{n ||| m}`.** The intersection of two cofinite
    neighbourhoods is again
```

```
one: `nbhd n  I  nbhd m = nbhd (n ||| m)` (bitwise OR of the excluded finite sets). -/
theorem nbhd_inter (n m : Nat) : nbhd n  I  nbhd m = nbhd (myLor n m) := by
  ext k
  simp only [nbhd, Set.mem_inter_iff, Set.mem_ofPred_eq, myLor_eq_lor, Nat.testBit_lor
    ,
    Bool.or_eq_false_iff]
end

section
/-- `nbhd 0 = Nat = Delta`: `0` has no set bits, so its excluded set `E_0` is empty. -/
theorem nbhd_zero : nbhd 0 = Set.univ := by
  ext k; simp [nbhd, Nat.zero_testBit]
end

section
/-- **The powerset domain `PN`** as a neighbourhood system over the token type `Nat`:
    the
neighbourhoods are the cofinite sets `nbhd n`, with master `Delta = Nat`. Closure under
     intersection is
`nbhd_inter` (the intersection is `nbhd (n ||| m)`), so no consistency hypothesis is
    even needed. -/
def PN : NeighborhoodSystem Nat where
  mem Y := exists  n, Y = nbhd n
  master := Set.univ
  master_nonempty := Set.univ_nonempty
  master_mem := <0, nbhd_zero.symm>
  inter_mem := by
    rintro X Y Z <a, rfl> <b, rfl> _ _
    exact <myLor a b, nbhd_inter a b>
  sub_master := by rintro X _; exact Set.subset_univ X
end

end DomainTheory.NeighbourhoodSystems.Example78

section
namespace DomainTheory.NeighbourhoodSystems.Example78
open Domain.Recursive NeighborhoodSystem
section
/- concordance: source L4477 EXAMPLE 7.8 powerset effectively given -/
/-- **Example 7.8 -- the computable presentation of `PN`.** The enumeration is `nbhd`;
    the
intersection function is the choice-free primitive-recursive bitwise OR `myLor`; Scott'
    s relation
7.1(i) is `nbhd n  I  nbhd m = nbhd k  <->  (n ||| m) = k`, decided by `RecDecidable.
    natEq`; relation
7.1(ii) (consistency) is always true. -/
def PNpres : ComputablePresentation PN where
  X := nbhd
  mem_X n := <n, rfl>
  surj := by rintro Y <n, rfl>; exact <n, rfl>
  interEq_computable := by
    -- `nbhd n  I  nbhd m = nbhd k  <->  myLor n m = k`, an equality of two primitive-
    recursive functions
    refine RecDecidable.of_iff (q := fun t =>
        nbhd t.unpair.1  I  nbhd t.unpair.2.unpair.1 = nbhd t.unpair.2.unpair.2)
      (fun t => ?_)
```

```
      (RecDecidable.natEq
        (a := fun t => myLor t.unpair.1 t.unpair.2.unpair.1)
        (b := fun t => t.unpair.2.unpair.2)
        ((primrec_myLor.comp
          (Nat.Primrec.left.pair (Nat.Primrec.left.comp Nat.Primrec.right))).of_eq fun
    s => by
           simp only [unpair_pair_fst, unpair_pair_snd])
        (Nat.Primrec.right.comp Nat.Primrec.right))
    -- the pointwise equivalence
    constructor
    * intro h; exact nbhd_injective ((nbhd_inter _ _).symm.trans h)
    * intro h; rw [nbhd_inter]; exact congrArg nbhd h
  cons_computable :=
    recDecidable_of_forall fun t => <myLor t.unpair.1 t.unpair.2, (nbhd_inter _ _).
    symm.subset>
  inter := myLor
  inter_primrec := primrec_myLor
  inter_spec := fun {n m} _ => (nbhd_inter n m).symm
  masterIdx := 0
  masterIdx_spec := nbhd_zero
end

section
/-- **Any two neighbourhoods of `PN` are consistent** (Scott: "any two neighbourhoods
    are
consistent"): their intersection `nbhd (n ||| m)` is itself a neighbourhood contained
    in both. -/
theorem PN_consistent (n m : Nat) : exists  k, nbhd k  subseteq  nbhd n  I  nbhd m :=
  <myLor n m, (nbhd_inter n m).symm.subset>
end

section
/-- **Example 7.8 (Scott 1981, PRG-19).** The powerset domain `PN` is effectively given
    . -/
theorem PN_isEffectivelyGiven : PN.IsEffectivelyGiven := <PNpres>
end

end DomainTheory.NeighbourhoodSystems.Example78
end
```

3.8.10 Definition 7.9 — power domain

DEFINITION 7.9. Let $\mathcal{D}$ be any neighbourhood system and define

$$\mathbb{P}\mathcal{D} = \left\{ \bigcup_{i<n} (\downarrow X_i) \mid \forall i < n.\, X_i \in \mathcal{D} \right\}.$$

We recall that for any $X \in \mathcal{D}$

$$\downarrow X = \{Y \in \mathcal{D} \mid Y \subseteq X\}.$$

The finite unions in $\mathbb{P}\mathcal{D}$ can be empty (i.e. if $n = 0$). $\square$

Formally, the system $\mathbb{P}\mathcal{D}$ is just more or less the closure of $\mathcal{D}$ under finite unions; however, this would not be an isomorphism-invariant construct unless $\mathcal{D}$ is "prepared". The preparation consists of replacing $\mathcal{D}$ by the isomorphic domain

$$\mathcal{D}^\dagger = \{\downarrow X \mid X \in \mathcal{D}\}.$$

(In this connection refer back to Exercise 1.20.) We remark that

$$\downarrow X \cap \downarrow Y \neq \emptyset \text{ iff } \{X, Y\} \text{ is consistent in } \mathcal{D},$$

and in that case

$$\downarrow X \cap \downarrow Y = \downarrow (X \cap Y).$$

Definition_7_9.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_20

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem
section
variable {a : Type*} (V : NeighborhoodSystem a)

/- concordance: source L4495 DEFINITION 7.9 power domain -/
/-- **Definition 7.9 -- the Smyth power domain `??`.** A set `W : Set (Set a)` is a
neighbourhood of `??` iff it is a *finite union of down-sets* `?_{X  in  L} X` for some
     finite
list `L` of `?`-neighbourhoods. The empty list `L = []` realizes Scott's `n = 0` (the
    empty
union `{}`). -/
def PDmem (W : Set (Set a)) : Prop :=
  exists  L : List (Set a), (forall  X  in  L, V.mem X)  /\  W = ? X  in  L, V.upSet X

/-- **Definition 7.9 -- the preparation `??`.** Scott prepares `?` for the power-domain
construct by replacing it with the isomorphic system `?? = {X ? X  in  ?}`, which is
    exactly
Exercise 1.20's `powerSystem` (over the token type `Delta? = ?`, i.e. `Set a`). It is a
     *positive*
neighbourhood system (`powerSystem_isPositive`). -/
abbrev dagger : NeighborhoodSystem (Set a) := V.powerSystem

/-- **Definition 7.9 -- closure under finite unions.** `??` is closed under binary
    union
(concatenate the index lists); with `PDmem_empty` this gives closure under all finite
    unions,
which is exactly how Scott defines the family. -/
theorem PDmem_union {W_1 W_2 : Set (Set a)} (h_1 : V.PDmem W_1) (h_2 : V.PDmem W_2) :
    V.PDmem (W_1  U  W_2) := by
```

```
  obtain <L_1, hL_1, rfl> := h_1
  obtain <L_2, hL_2, rfl> := h_2
  refine <L_1 ++ L_2, ?_, ?_>
  * intro X hX
    rcases List.mem_append.mp hX with h | h
    * exact hL_1 X h
    * exact hL_2 X h
  * ext z
    simp only [Set.mem_union, Set.mem_iUnion, exists_prop, List.mem_append]
    constructor
    * rintro (<X, hX, hz> | <X, hX, hz>)
      * exact <X, Or.inl hX, hz>
      * exact <X, Or.inr hX, hz>
    * rintro <X, hX | hX, hz>
      * exact Or.inl <X, hX, hz>
      * exact Or.inr <X, hX, hz>

/-- **Definition 7.9 -- `?  ~=  ??` (Exercise 1.20).** The preparation does not change
    the domain:
`?` and `??` are isomorphic. (This is what makes `??` isomorphism-invariant.) -/
theorem dagger_isomorphic : V  ~= ? V.dagger := V.isomorphic_powerSystem

/-- **Definition 7.9 (remark 2).** "In that case `X  I  Y = (X  I  Y)`." When `{X, Y}`
    is
consistent (witness `Z  subseteq  X  I  Y`, `Z  in  ?`), the intersection `X  I  Y` is
    again a neighbourhood,
so `X  I  Y = (X  I  Y)` is a neighbourhood of the prepared system `??`. (The
    unconditional set
identity `X  I  Y = (X  I  Y)` is Exercise 1.20's `upSet_inter`; the content here is
    membership
in `??`.) -/
theorem dagger_upSet_inter {X Y Z : Set a} (hX : V.mem X) (hY : V.mem Y) (hZ : V.mem Z
    )
    (hsub : Z  subseteq  X  I  Y) : V.dagger.mem (V.upSet X  I  V.upSet Y) := by
  rw [V.upSet_inter]
  exact <X  I  Y, V.inter_mem hX hY hZ hsub, rfl>

/-- Membership in a finite union of down-sets: `z  in  ?_{X in L} X  <->  exists  X  in
     L, z  in  X`. -/
theorem mem_PDunion {L : List (Set a)} {z : Set a} :
    z  in  (? X  in  L, V.upSet X)  <->  exists  X  in  L, z  in  V.upSet X := by
  simp only [Set.mem_iUnion, exists_prop]

/-- **Definition 7.9 -- the empty union.** "The finite unions in `??` can be empty (i.e
    . if
`n = 0`)." The empty neighbourhood `{}` is a neighbourhood of `??` (witnessed by `L =
    []`). -/
theorem PDmem_empty : V.PDmem {} := by
  refine <[], ?_, ?_>
  * intro X hX; cases hX
  * ext z
    rw [mem_PDunion]
    constructor
    * intro h; exact absurd h (Set.notMem_empty z)
    * rintro <X, hX, _>; cases hX
```

```
/-- **Definition 7.9 -- Scott's `?_{i<n}` form.** The neighbourhood family of `??`,
    written with
a finite sequence `X : Fin n  ->  ?` exactly as Scott displays it: `W = ?_{i<n} X_i`
    with each
`X_i  in  ?`. Equivalent to the `List` form via `List.ofFn` / `List.get`. -/
theorem PDmem_iff_fin {W : Set (Set a)} :
    V.PDmem W  <->  exists  (n : Nat) (X : Fin n  ->  Set a),
      (forall  i, V.mem (X i))  /\  W = ? i, V.upSet (X i) := by
  constructor
  * rintro <L, hL, rfl>
    refine <L.length, L.get, fun i => hL _ (L.get_mem i), ?_>
    ext z
    rw [mem_PDunion, Set.mem_iUnion]
    constructor
    * rintro <X, hX, hz>
      obtain <i, rfl> := List.get_of_mem hX
      exact <i, hz>
    * rintro <i, hz>
      exact <L.get i, L.get_mem i, hz>
  * rintro <n, X, hX, rfl>
    refine <List.ofFn X, ?_, ?_>
    * intro Y hY
      obtain <i, rfl> := Set.mem_range.mp (List.mem_ofFn.mp hY)
      exact hX i
    * ext z
      rw [Set.mem_iUnion, mem_PDunion]
      constructor
      * rintro <i, hz>
        exact <X i, List.mem_ofFn.mpr (Set.mem_range.mpr <i, rfl>), hz>
      * rintro <Y, hY, hz>
        obtain <i, rfl> := Set.mem_range.mp (List.mem_ofFn.mp hY)
        exact <i, hz>

/-- **Definition 7.9 -- singletons.** Every down-set `X` (`X  in  ?`) is a
    neighbourhood of `??`
(the case `n = 1`); these are exactly the neighbourhoods of the prepared system `??`.
    -/
theorem PDmem_upSet {X : Set a} (hX : V.mem X) : V.PDmem (V.upSet X) := by
  refine <[X], ?_, ?_>
  * intro Y hY; rw [List.mem_singleton] at hY; subst hY; exact hX
  * ext z
    rw [mem_PDunion]
    constructor
    * intro hz; exact <X, List.mem_singleton.mpr rfl, hz>
    * rintro <Y, hY, hz>; rw [List.mem_singleton] at hY; subst hY; exact hz

/-- **Definition 7.9 -- the master neighbourhood `Delta`.** The largest neighbourhood
    of `??` is
`Delta = ?` (all `?`-neighbourhoods), itself a finite union (the case `X_0 = Delta`).
    -/
theorem PDmem_master : V.PDmem (V.upSet V.master) := V.PDmem_upSet V.master_mem

/-- **Definition 7.9 (remark 1).** "`X  I  Y  !=  {}` iff `{X, Y}` is consistent in
    `?`." The
down-sets `X`, `Y` meet exactly when `X, Y` have a common lower neighbourhood `Z
    subseteq  X  I  Y`. -/
```

```
theorem upSet_inter_nonempty_iff {X Y : Set a} :
    (V.upSet X  I  V.upSet Y).Nonempty  <->  exists  Z, V.mem Z  /\  Z  subseteq  X  I
      Y := by
  rw [V.upSet_inter]
  constructor
  * rintro <Z, hZmem, hZsub>; exact <Z, hZmem, hZsub>
  * rintro <Z, hZmem, hZsub>; exact <Z, hZmem, hZsub>

end

end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
end
```

3.8.11 Proposition 7.10 — power domain effective

PROPOSITION 7.10. The power domain $\mathbb{P}\mathcal{D}$ is a neighbourhood system if $\mathcal{D}$ is, and it is effectively given if $\mathcal{D}$ is.

Proof : The system $\mathcal{D}^\dagger$ is a neighbourhood system as we just remarked; indeed it is a positive neighbourhood system. It is easy to prove that the closure of any positive system under finite unions is a neighbourhood system, because the resulting family of sets is closed under *all* finite intersections. (If we left out the empty union, the result would be a positive system.) The proof is obvious since intersection of sets distributes over finite union. So $\mathbb{P}\mathcal{D}$ is a neighbourhood system.

For the second half of the proposition, we just have to constructivize the previous argument. Thus, if

$$\mathcal{D} = \{X_n \mid n \in \mathbb{N}\},$$

then the elements of $\mathbb{P}\mathcal{D}$ can be written as:

$$\bigcup_{i<q} (\downarrow X_{n_i}),$$

and hence are indexed by the finite sequences $\langle n_0, \ldots, n_{q-1}\rangle$ of integers. Now one of the standard devices of recursion theory is to put the finite sequences of integers into a recursive one-one correspondence with the integers themselves. This is the start of the recursive presentation of $\mathbb{P}\mathcal{D}$, since it means we can list effectively all the required neighbourhoods.

Next consider an intersection

$$\bigcup_{i<q} (\downarrow X_{n_i}) \cap \bigcup_{j<r} (\downarrow X_{m_j}) = \bigcup_{\substack{i<q \\ j<r}} \downarrow (X_{n_i} \cap X_{m_j})\,.$$

Some of the terms which are $\emptyset$ have to be thrown out — but this requires only a finite number of decisions all computable by assumption. Now we have to rewrite

$$X_{n_i} \cap X_{m_j} = X_{k_{ij}}\,,$$

but the finding of k_{ij} is also computable. *Finally*, we have to re-order the doubly indexed sequence into a singly indexed sequence of length $q \cdot r$, but this is easily seen to be computable also. Therefore, intersections can be "calculated".

It remains to be shown that equality between neighbourhoods in $\mathbb{P}\mathcal{D}$ is decidable. The question really comes down to deciding something like:

$$\downarrow X_k \subseteq \bigcup_{i<q} \downarrow X_{n_i} .$$

Now since $X_k \in \downarrow X_k$, we find that the above is just equivalent to:

$$\exists i < q.\, X_k \subseteq X_{n_i} .$$

By our assumptions on $\mathcal{D}$, this is decidable. (It is this part of the argument that required the passage to $\mathcal{D}^\dagger$. It does not seem to be generally true that the closure under finite unions of an effectively given system is again effectively given.) □

One of the main reasons that $\mathbb{P}\mathcal{D}$ is like a power domain is the possibility of forming "finite sets".

Proposition_7_10.lean

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Exercise_7_14

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive
section
namespace NeighborhoodSystem
variable {a : Type*} (V : NeighborhoodSystem a)

/- concordance: source L4517 PROPOSITION 7.10 power domain effective -/
/-- **Proposition 7.10 (Part A) -- the Smyth power domain `??` as a neighbourhood
    system.** Master
`Delta`; closure under intersection is `PDmem_inter`; `X  subseteq  Delta` since `X
    subseteq  Delta`. -/
abbrev PowerDomain : NeighborhoodSystem (Set a) where
  mem := V.PDmem
  master := V.upSet V.master
  master_nonempty := <V.master, V.master_mem, subset_rfl>
  master_mem := V.PDmem_master
  inter_mem := fun h_1 h_2 _ _ => V.PDmem_inter h_1 h_2
  sub_master := by
    intro W hW
    obtain <L, hL, rfl> := hW
    intro Z hZ
    rw [V.mem_PDunion] at hZ
    obtain <X, hX, hZmem, hZsub> := hZ
    exact <hZmem, hZsub.trans (V.sub_master (hL X hX))>

@[simp] theorem PowerDomain_mem {W : Set (Set a)} : V.PowerDomain.mem W  <->  V.PDmem
    W := Iff.rfl

@[simp] theorem PowerDomain_master : V.PowerDomain.master = V.upSet V.master := rfl
```

```
end NeighborhoodSystem
end

section
namespace NeighborhoodSystem

variable {a : Type*} (V : NeighborhoodSystem a)

variable (P : ComputablePresentation V)

/-- The finite union of down-sets indexed by a `List` of `?`-indices: `?_{b  in  l} X_b
    `. -/
def UPX (l : List Nat) : Set (Set a) := ? b  in  l, V.upSet (P.X b)

theorem UPX_def (l : List Nat) : V.UPX P l = ? b  in  l, V.upSet (P.X b) := rfl

/-- **The enumeration of `??`.** `Ypd c = ?_{a  in  decodeList c} X_a` -- a finite
    union of down-sets,
ranging over all of `??` as `c` ranges over `Nat` (the empty code `0` gives `{}`). -/
def Ypd (c : Nat) : Set (Set a) := V.UPX P (decodeList c)

theorem Ypd_def (c : Nat) : V.Ypd P c = V.UPX P (decodeList c) := rfl

/-- One step of the **inner** fold (over `decodeList m`, parameter the outer index `a`)
    : prepend
`P.inter a b` to the accumulator code iff `(a, b)` is consistent (`cons <a,b> = 1`),
    else leave it.
`foldCode` shape: state `w = <b, <acc, a>>`. -/
def innerInterStp (cons : Nat  ->  Nat) (w : Nat) : Nat :=
  selectFn (isOne (cons (Nat.pair w.unpair.2.unpair.2 w.unpair.1)))
    (Nat.pair (P.inter w.unpair.2.unpair.2 w.unpair.1) w.unpair.2.unpair.1 + 1)
    w.unpair.2.unpair.1

/-- The inner fold: process `decodeList m`, accumulating the consistent intersection
    indices. -/
def innerInterCode (cons : Nat  ->  Nat) (a acc m : Nat) : Nat := foldCode (V.
    innerInterStp P cons) a acc m

theorem innerInterStp_eq (cons : Nat  ->  Nat) (a acc b : Nat) :
    V.innerInterStp P cons (Nat.pair b (Nat.pair acc a))
      = selectFn (isOne (cons (Nat.pair a b))) (Nat.pair (P.inter a b) acc + 1) acc :=
     by
  unfold innerInterStp; simp only [unpair_pair_fst, unpair_pair_snd]

theorem innerInterCode_eq (cons : Nat  ->  Nat) (a acc m : Nat) :
    V.innerInterCode P cons a acc m
      = List.foldl (fun acc b => selectFn (isOne (cons (Nat.pair a b)))
          (Nat.pair (P.inter a b) acc + 1) acc) acc (decodeList m) := by
  have hf : (fun (acc x : Nat) => V.innerInterStp P cons (Nat.pair x (Nat.pair acc a))
    )
      = (fun acc b => selectFn (isOne (cons (Nat.pair a b))) (Nat.pair (P.inter a b)
    acc + 1) acc) := by
    funext acc b; exact V.innerInterStp_eq P cons a acc b
  unfold innerInterCode; rw [foldCode_eq', hf]

theorem mem_UPX {l : List Nat} {z : Set a} :
```

```
    z  in  V.UPX P l  <->  exists  b  in  l, z  in  V.upSet (P.X b) := by
  simp only [UPX, Set.mem_iUnion, exists_prop]

theorem UPX_cons (b : Nat) (l : List Nat) :
    V.UPX P (b :: l) = V.upSet (P.X b)  U  V.UPX P l := by
  ext z
  rw [Set.mem_union, V.mem_UPX P, V.mem_UPX P]
  constructor
  * rintro <c, hc, hz>
    rcases List.mem_cons.mp hc with rfl | hc'
    * exact Or.inl hz
    * exact Or.inr <c, hc', hz>
  * rintro (hz | <c, hc, hz>)
    * exact <b, List.mem_cons.mpr (Or.inl rfl), hz>
    * exact <c, List.mem_cons.mpr (Or.inr hc), hz>

theorem UPX_nil : V.UPX P [] = {} := by
  ext z
  rw [V.mem_UPX P]
  simp only [List.not_mem_nil, false_and, exists_false, Set.mem_empty_iff_false]

theorem Ypd_cons_code (v acc : Nat) :
    V.Ypd P (Nat.pair v acc + 1) = V.upSet (P.X v)  U  V.Ypd P acc := by
  rw [Ypd_def, decodeList_succ, unpair_pair_fst, unpair_pair_snd, V.UPX_cons P,  <-  V
    .Ypd_def P]

theorem Ypd_zero : V.Ypd P 0 = {} := by rw [Ypd_def, decodeList_zero, V.UPX_nil P]

/-- The enumeration is onto `??` (build the index list entry by entry from `P.surj`;
    choice-free). -/
theorem PDmem_exists_Ypd_aux :
    forall  (L : List (Set a)), (forall  X  in  L, V.mem X)  ->  exists  c, V.Ypd P c
    = ? X  in  L, V.upSet X := by
  intro L
  induction L with
  | nil => intro _; exact <0, by rw [V.Ypd_zero P, V.upSetUnion_nil]>
  | cons X L ih =>
    intro hL
    obtain <n, hn> := P.surj (hL X (List.mem_cons.mpr (Or.inl rfl)))
    obtain <c', hc'> := ih (fun Y hY => hL Y (List.mem_cons.mpr (Or.inr hY)))
    exact <Nat.pair n c' + 1, by rw [V.Ypd_cons_code P, hn, hc', V.upSetUnion_cons]>

theorem PDmem_exists_Ypd {W : Set (Set a)} (hW : V.PDmem W) : exists  c, V.Ypd P c = W
     := by
  obtain <L, hL, rfl> := hW
  exact V.PDmem_exists_Ypd_aux P L hL

theorem mem_Ypd {c : Nat} {z : Set a} :
    z  in  V.Ypd P c  <->  exists  a  in  decodeList c, z  in  V.upSet (P.X a) := V.
    mem_UPX P

/-- Every `Ypd c` is a `??`-neighbourhood (list `(decodeList c).map P.X`). -/
theorem Ypd_isPDmem (c : Nat) : V.PDmem (V.Ypd P c) := by
  refine <(decodeList c).map P.X, ?_, ?_>
  * intro X hX
    obtain <a, _, rfl> := List.mem_map.mp hX
```

```
    exact P.mem_X a
  * ext z
    rw [V.mem_Ypd P, V.mem_PDunion]
    constructor
    * rintro <a, ha, hz>; exact <P.X a, List.mem_map.mpr <a, ha, rfl>, hz>
    * rintro <X, hX, hz>
      obtain <a, ha, rfl> := List.mem_map.mp hX
      exact <a, ha, hz>

/-- One step of the **outer** fold (over `decodeList n`, parameter `m`): run the inner
    fold for the
current outer index `a`. `foldCode` shape: state `w = <a, <acc, m>>`. -/
def outerInterStp (cons : Nat  ->  Nat) (w : Nat) : Nat :=
  V.innerInterCode P cons w.unpair.1 w.unpair.2.unpair.1 w.unpair.2.unpair.2

/-- **The intersection code** `interCode cons n m`: a code of the list of `P.inter a b`
     over the
consistent pairs `(a, b)  in  dl n  x  dl m`. -/
def interCode (cons : Nat  ->  Nat) (n m : Nat) : Nat := foldCode (V.outerInterStp P
    cons) m 0 n

theorem outerInterStp_eq (cons : Nat  ->  Nat) (a acc m : Nat) :
    V.outerInterStp P cons (Nat.pair a (Nat.pair acc m)) = V.innerInterCode P cons a
    acc m := by
  unfold outerInterStp; simp only [unpair_pair_fst, unpair_pair_snd]

theorem interCode_eq (cons : Nat  ->  Nat) (n m : Nat) :
    V.interCode P cons n m
      = List.foldl (fun acc a => V.innerInterCode P cons a acc m) 0 (decodeList n) :=
    by
  have hf : (fun (acc x : Nat) => V.outerInterStp P cons (Nat.pair x (Nat.pair acc m))
    )
      = (fun acc a => V.innerInterCode P cons a acc m) := by
    funext acc a; exact V.outerInterStp_eq P cons a acc m
  unfold interCode; rw [foldCode_eq', hf]

theorem primrec_innerInterStp (cons : Nat  ->  Nat) (hconsp : Nat.Primrec cons) :
    Nat.Primrec (V.innerInterStp P cons) := by
  have h1 : Nat.Primrec (fun w : Nat => w.unpair.1) := Nat.Primrec.left
  have hacc : Nat.Primrec (fun w : Nat => w.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.right
  have ha : Nat.Primrec (fun w : Nat => w.unpair.2.unpair.2) :=
    Nat.Primrec.right.comp Nat.Primrec.right
  have hpair_ab : Nat.Primrec (fun w : Nat => Nat.pair w.unpair.2.unpair.2 w.unpair.1)
     := ha.pair h1
  have hcons_ab : Nat.Primrec (fun w : Nat => cons (Nat.pair w.unpair.2.unpair.2 w.
    unpair.1)) :=
    hconsp.comp hpair_ab
  have hinter_ab : Nat.Primrec (fun w : Nat => P.inter w.unpair.2.unpair.2 w.unpair.1)
     :=
    (P.inter_primrec.comp hpair_ab).of_eq (fun w => by simp only [unpair_pair_fst,
    unpair_pair_snd])
  have hprepend : Nat.Primrec
      (fun w : Nat => Nat.pair (P.inter w.unpair.2.unpair.2 w.unpair.1) w.unpair.2.
    unpair.1 + 1) :=
    Nat.Primrec.succ.comp (hinter_ab.pair hacc)
```

```
  exact (primrec_selectFn (primrec_isOne.comp hcons_ab) hprepend hacc).of_eq (fun _ =>
     rfl)

theorem primrec_outerInterStp (cons : Nat  ->  Nat) (hconsp : Nat.Primrec cons) :
    Nat.Primrec (V.outerInterStp P cons) :=
  (primrec_foldCode (V.primrec_innerInterStp P cons hconsp) Nat.Primrec.left
    (Nat.Primrec.left.comp Nat.Primrec.right) (Nat.Primrec.right.comp Nat.Primrec.
    right)).of_eq
    (fun _ => rfl)

theorem primrec_interCode (cons : Nat  ->  Nat) (hconsp : Nat.Primrec cons) :
    Nat.Primrec (fun t => V.interCode P cons t.unpair.1 t.unpair.2) :=
  (primrec_foldCode (V.primrec_outerInterStp P cons hconsp) Nat.Primrec.right
    (Nat.Primrec.const 0) Nat.Primrec.left).of_eq (fun _ => rfl)

/-- `forall  a  in  dl c, exists  b  in  dl k, X_a  subseteq  X_b` is recursively
    decidable (nested bounded `forall /exists ` over
`P.incl_computable`). -/
theorem subCode_computable :
    RecDecidable_2 (fun c k => forall  a  in  decodeList c, exists  b  in  decodeList
    k, P.X a  subseteq  P.X b) := by
  have hinner : RecDecidable_2 (fun k a => exists  b  in  decodeList k, P.X a
    subseteq  P.X b) :=
    P.incl_computable.swap.bExistsList
  exact hinner.swap.bForallList

/-- If `(a, b)` is *inconsistent* (no `?`-neighbourhood below `X_a  I  X_b`) then `X_a
     I  X_b = {}`. -/
theorem upSet_inter_eq_empty_of_not_cons {a b : Nat}
    (h : not  exists  k, P.X k  subseteq  P.X a  I  P.X b) :
    V.upSet (P.X a)  I  V.upSet (P.X b) = {} := by
  rw [V.upSet_inter]
  ext Z
  simp only [mem_upSet, Set.mem_empty_iff_false, iff_false, not_and]
  intro hZmem hZsub
  obtain <n, hn> := P.surj hZmem
  exact h <n, by rw [hn]; exact hZsub>

/-- One inner step at the `Ypd` level: prepending the consistent intersection index
    adds exactly
`X_a  I  X_b` (which is `{}` for inconsistent pairs). The `by_cases` on `cons <a,b> =
    1` is on a
*decidable* `Nat`-equality -- choice-free. -/
theorem Ypd_innerstep (cons : Nat  ->  Nat) {a b : Nat}
    (hcons : cons (Nat.pair a b) = 1  <->  exists  k, P.X k  subseteq  P.X a  I  P.X b
    ) (acc : Nat) :
    V.Ypd P (selectFn (isOne (cons (Nat.pair a b))) (Nat.pair (P.inter a b) acc + 1)
    acc)
      = V.Ypd P acc  U  (V.upSet (P.X a)  I  V.upSet (P.X b)) := by
  by_cases h : cons (Nat.pair a b) = 1
  * rw [(isOne_eq_one_iff _).mpr h, selectFn_one, V.Ypd_cons_code P]
    have hXeq : P.X (P.inter a b) = P.X a  I  P.X b := P.inter_spec (hcons.mp h)
    rw [hXeq,  <-  V.upSet_inter, Set.union_comm]
  * have hisz : isOne (cons (Nat.pair a b)) = 0 := by
      have hle := isOne_le_one (cons (Nat.pair a b))
      rcases (show isOne (cons (Nat.pair a b)) = 0  \/  isOne (cons (Nat.pair a b)) =
```

```
    1 by omega)
        with h0 | h1
      * exact h0
      * exact absurd ((isOne_eq_one_iff _).mp h1) h
    rw [hisz, selectFn_zero,
      V.upSet_inter_eq_empty_of_not_cons P (fun hk => h (hcons.mpr hk)), Set.
    union_empty]

/-- The inner fold computes `X_a  I  (?_{b  in  bs} X_b)` on top of the starting
    accumulator. -/
theorem Ypd_innerfoldl (cons : Nat  ->  Nat) (a : Nat)
    (hcons : forall  b, cons (Nat.pair a b) = 1  <->  exists  k, P.X k  subseteq  P.X
    a  I  P.X b) :
    forall  (bs : List Nat) (acc : Nat),
      V.Ypd P (List.foldl (fun acc b => selectFn (isOne (cons (Nat.pair a b)))
          (Nat.pair (P.inter a b) acc + 1) acc) acc bs)
        = V.Ypd P acc  U  (V.upSet (P.X a)  I  V.UPX P bs) := by
  intro bs
  induction bs with
  | nil => intro acc; rw [List.foldl_nil, V.UPX_nil P, Set.inter_empty, Set.
    union_empty]
  | cons b bs ih =>
    intro acc
    rw [List.foldl_cons, ih, V.Ypd_innerstep P cons (hcons b), V.UPX_cons P,
      Set.inter_union_distrib_left, Set.union_assoc]

/-- `Ypd (innerInterCode cons a acc m) = Ypd acc  U  (X_a  I  Y_m)`. -/
theorem Ypd_innerInterCode (cons : Nat  ->  Nat) (a acc m : Nat)
    (hcons : forall  b, cons (Nat.pair a b) = 1  <->  exists  k, P.X k  subseteq  P.X
    a  I  P.X b) :
    V.Ypd P (V.innerInterCode P cons a acc m)
      = V.Ypd P acc  U  (V.upSet (P.X a)  I  V.Ypd P m) := by
  rw [V.innerInterCode_eq P, V.Ypd_innerfoldl P cons a hcons (decodeList m) acc,  <-
    V.Ypd_def P]

/-- The outer fold computes `(?_{a  in  as} X_a)  I  Y_m` on top of the starting
    accumulator. -/
theorem Ypd_outerfoldl (cons : Nat  ->  Nat) (m : Nat)
    (hcons : forall  a b, cons (Nat.pair a b) = 1  <->  exists  k, P.X k  subseteq  P.
    X a  I  P.X b) :
    forall  (as : List Nat) (acc : Nat),
      V.Ypd P (List.foldl (fun acc a => V.innerInterCode P cons a acc m) acc as)
        = V.Ypd P acc  U  (V.UPX P as  I  V.Ypd P m) := by
  intro as
  induction as with
  | nil => intro acc; rw [List.foldl_nil, V.UPX_nil P, Set.empty_inter, Set.
    union_empty]
  | cons a as ih =>
    intro acc
    rw [List.foldl_cons, ih, V.Ypd_innerInterCode P cons a acc m (hcons a), V.UPX_cons
     P,
      Set.union_inter_distrib_right, Set.union_assoc]

/-- **Correctness of the intersection code:** `Y_{interCode n m} = Y_n  I  Y_m`. -/
theorem Ypd_interCode (cons : Nat  ->  Nat) (n m : Nat)
    (hcons : forall  a b, cons (Nat.pair a b) = 1  <->  exists  k, P.X k  subseteq  P.
```

```
    X a  I  P.X b) :
    V.Ypd P (V.interCode P cons n m) = V.Ypd P n  I  V.Ypd P m := by
  rw [V.interCode_eq P, V.Ypd_outerfoldl P cons m hcons (decodeList n) 0, V.Ypd_zero P
    ,
    Set.empty_union,  <-  V.Ypd_def P]

/-- `X_a  subseteq  Y_k  <->  exists  b  in  dl k, X_a  subseteq  X_b` (a down-set lies
     in a finite union of down-sets iff its top
`X_a` is below one of them). -/
theorem upSet_subset_Ypd_iff {a k : Nat} :
    V.upSet (P.X a)  subseteq  V.Ypd P k  <->  exists  b  in  decodeList k, P.X a
    subseteq  P.X b := by
  constructor
  * intro h
    have hmem : P.X a  in  V.Ypd P k := h <P.mem_X a, subset_rfl>
    obtain <b, hb, hPab> := (V.mem_Ypd P).mp hmem
    exact <b, hb, hPab.2>
  * rintro <b, hb, hab> z hz
    exact (V.mem_Ypd P).mpr <b, hb, hz.1, hz.2.trans hab>

/-- `Y_c  subseteq  Y_k  <->  forall  a  in  dl c, exists  b  in  dl k, X_a  subseteq
    X_b`. -/
theorem Ypd_subset_iff {c k : Nat} :
    V.Ypd P c  subseteq  V.Ypd P k  <->  forall  a  in  decodeList c, exists  b  in
    decodeList k, P.X a  subseteq  P.X b := by
  rw [Ypd_def, UPX_def, Set.iUnion_2_subset_iff]
  constructor
  * intro h a ha; exact (V.upSet_subset_Ypd_iff P).mp (h a ha)
  * intro h a ha; exact (V.upSet_subset_Ypd_iff P).mpr (h a ha)

/-- **Equality of two finite unions of down-sets**, unfolded as a nested bounded `
    forall /exists `. -/
theorem Ypd_eq_iff {c k : Nat} :
    V.Ypd P c = V.Ypd P k  <->
      (forall  a  in  decodeList c, exists  b  in  decodeList k, P.X a  subseteq  P.X
    b)  /\
      (forall  b  in  decodeList k, exists  a  in  decodeList c, P.X b  subseteq  P.X
    a) := by
  rw [Set.Subset.antisymm_iff, V.Ypd_subset_iff P, V.Ypd_subset_iff P]

/-- **Scott's relation (i) for `??`, at the level of codes:** equality of two
    enumerated
neighbourhoods is recursively decidable. -/
theorem eqCode_computable : RecDecidable_2 (fun c k => V.Ypd P c = V.Ypd P k) := by
  have h := (V.subCode_computable P).and (V.subCode_computable P).swap
  refine RecDecidable.of_iff (fun t => ?_) h
  exact V.Ypd_eq_iff P

/-- **Scott's relation (i):** `Y_n  I  Y_m = Y_k` is recursively decidable, by reducing
     to the equality
decider via the (primitive-recursive) intersection code. -/
theorem Ypd_interEq_computable (cons : Nat  ->  Nat) (hconsp : Nat.Primrec cons)
    (hcons : forall  a b, cons (Nat.pair a b) = 1  <->  exists  k, P.X k  subseteq  P.
    X a  I  P.X b) :
    RecDecidable_3 (fun n m k => V.Ypd P n  I  V.Ypd P m = V.Ypd P k) := by
  have hic : Nat.Primrec (fun t => V.interCode P cons t.unpair.1 t.unpair.2.unpair.1)
```

```
    :=
    ((V.primrec_interCode P cons hconsp).comp
      (Nat.Primrec.left.pair (Nat.Primrec.left.comp Nat.Primrec.right))).of_eq
      (fun t => by simp only [unpair_pair_fst, unpair_pair_snd])
  have hg : Nat.Primrec
      (fun t => Nat.pair (V.interCode P cons t.unpair.1 t.unpair.2.unpair.1) t.unpair
    .2.unpair.2) :=
    hic.pair (Nat.Primrec.right.comp Nat.Primrec.right)
  refine RecDecidable.of_iff (fun t => ?_) ((V.eqCode_computable P).comp hg)
  simp only [unpair_pair_fst, unpair_pair_snd]
  rw [V.Ypd_interCode P cons t.unpair.1 t.unpair.2.unpair.1 hcons]

/-- **Proposition 7.10 (Part B), parametrised.** The `??` presentation built from an
    explicit
primitive-recursive consistency decider `cons` for `?` (cf. `funPresentation`, Theorem
    7.5). The data
(`X = Ypd`, `inter = interCode cons`, `masterIdx`) is choice-free given `cons`. -/
def PDPresentation (cons : Nat  ->  Nat) (hconsp : Nat.Primrec cons)
    (hcons : forall  a b, cons (Nat.pair a b) = 1  <->  exists  k, P.X k  subseteq  P.
    X a  I  P.X b) :
    ComputablePresentation V.PowerDomain where
  X := V.Ypd P
  mem_X := fun c => V.Ypd_isPDmem P c
  surj := fun hW => V.PDmem_exists_Ypd P hW
  interEq_computable := V.Ypd_interEq_computable P cons hconsp hcons
  cons_computable :=
    recDecidable_of_forall (fun t => <0, by rw [V.Ypd_zero P]; exact Set.empty_subset
    _>)
  inter := V.interCode P cons
  inter_primrec := V.primrec_interCode P cons hconsp
  inter_spec := fun _ => V.Ypd_interCode P cons _ _ hcons
  masterIdx := Nat.pair P.masterIdx 0 + 1
  masterIdx_spec := by
    show V.Ypd P (Nat.pair P.masterIdx 0 + 1) = V.PowerDomain.master
    rw [V.Ypd_cons_code P, V.Ypd_zero P, Set.union_empty, P.masterIdx_spec,
    PowerDomain_master]

end NeighborhoodSystem
end

section
/-- **Proposition 7.10 (Scott 1981, PRG-19).** The Smyth power domain `??` is a
    neighbourhood system
(`PowerDomain`), and it is **effectively given** whenever `?` is. The consistency
    decider is obtained
from `?`'s presentation inside the `Nonempty` proof (extraction into a `Prop` needs no
    choice). -/
theorem NeighborhoodSystem.PowerDomain_isEffectivelyGiven {a : Type*} {V :
    NeighborhoodSystem a}
    (h : V.IsEffectivelyGiven) : V.PowerDomain.IsEffectivelyGiven := by
  obtain <P> := h
  obtain <cons, hconsp, hconss> := P.cons_computable
  refine <V.PDPresentation P cons hconsp (fun a b => ?_)>
  have hb := hconss (Nat.pair a b)
  simp only [unpair_pair_fst, unpair_pair_snd] at hb
  exact hb.symm
```

```
end

end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
section
set_option linter.unusedVariables false

theorem theorem_710.{u_1} : forall  {a : Type u_1} {V : DomainTheory.
    NeighbourhoodSystems.NeighborhoodSystem a},
  @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven a V  ->
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven (Set a)
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V) :=
  @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain_isEffectivelyGiven
end

end DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem
section
variable {a : Type*} (V : NeighborhoodSystem a)

/-- **Definition 7.11 -- the empty join.** With no arguments, `{ } = False` (the least
    element of
`|??|`). -/
def PDfinJoinZero : V.PowerDomain.Element := V.PowerDomain.bot

/-- **Definition 7.11 -- membership.** A `??`-neighbourhood `W` lies in the join of
`x_0, ..., x_{n-1}` iff some choice of representatives `X_i  in  x_i` has every down-
    set
`X_i` contained in `W` (Scott's `?_{i<n} (X_i)  subseteq  z`, read as down-sets in
    `??`). -/
def PDmemFinJoin {n : Nat} (xs : Fin n  ->  V.Element) (W : Set (Set a)) : Prop :=
  exists  (X : Fin n  ->  Set a) (_hX : forall  i, (xs i).mem (X i)),
    V.PowerDomain.mem W  /\  forall  i, V.upSet (X i)  subseteq  W

/-- **Definition 7.11 -- the non-empty join.** For `n  >=  1`, `{x_0,...,x_{n-1}}` is
    the filter
generated by Scott's union condition. -/
def PDfinJoinSucc {m : Nat} (xs : Fin (m + 1)  ->  V.Element) : V.PowerDomain.Element
    where
  mem := V.PDmemFinJoin xs
  sub hW := by obtain <_, _, hPD, _> := hW; exact hPD
  master_mem := by
    refine <fun i => V.master, fun i => (xs i).master_mem, V.PDmem_master, fun i Z hz
    => hz>
  inter_mem h1 h2 := by
    obtain <X1, hX1, hPD1, hsub1> := h1
    obtain <X2, hX2, hPD2, hsub2> := h2
    refine <fun i => X1 i  I  X2 i, fun i => (xs i).inter_mem (hX1 i) (hX2 i),
      V.PDmem_inter hPD1 hPD2, fun i Y hY => ?_>

    rw [ <-  V.upSet_inter] at hY
    rcases hY with <hY1, hY2>
    exact <hsub1 i hY1, hsub2 i hY2>
  up_mem h hW hsub := by
    obtain <X, hX, _, hsub'> := h
```

```
    exact <X, hX, hW, fun i => (hsub' i).trans hsub>

end
end NeighborhoodSystem
end DomainTheory.NeighbourhoodSystems
end
```

3.8.12 Definition 7.11 — finite sets of elements

DEFINITION 7.11. For elements $x_0, \ldots, x_{n-1} \in |D|$ we define

$$\{x_0, \ldots, x_{n-1}\} = \{z \in \mathbb{P}D \mid \exists X_0 \in x_0 \ldots \exists X_{n-1} \in x_{n-1}.\ \bigcup_{i<n} (\uparrow X_i) \subseteq z\}.$$

(Note, we could also write $\forall i < n.\ X_i \in z$.) □

Definition_7_11.lean

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Proposition_7_10

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
namespace NeighborhoodSystem
section
variable {a : Type*} (V : NeighborhoodSystem a)

/- concordance: source L4557 DEFINITION 7.11 finite sets of elements -/
/-- **Definition 7.11.** The finite-element join `{x_0,...,x_{n-1}}` in `|??|`. The
    empty
family yields `False`; otherwise use Scott's union-of-down-sets condition. -/
def PDfinJoin (n : Nat) (xs : Fin n  ->  V.Element) : V.PowerDomain.Element :=
  match n with
  | 0 => V.PDfinJoinZero
  | _m + 1 => V.PDfinJoinSucc xs

@[simp] theorem PDmem_finJoinZero {W : Set (Set a)} :
    (V.PDfinJoinZero).mem W  <->  W = V.PowerDomain.master := by
  rw [PDfinJoinZero, mem_bot, PowerDomain_master]

/-- **Definition 7.11 -- singleton join.** Write `{x}` for the unary join. -/
def PDsingleton (x : V.Element) : V.PowerDomain.Element := V.PDfinJoin 1 (fun (_ : Fin
     1) => x)

theorem PDmem_finJoinSucc {m : Nat} (xs : Fin (m + 1)  ->  V.Element) {W : Set (Set a)
    } :
    (V.PDfinJoinSucc xs).mem W  <->  V.PDmemFinJoin xs W := Iff.rfl

@[simp] theorem PDmem_singleton {x : V.Element} {W : Set (Set a)} :
    (V.PDsingleton x).mem W  <->
      exists  X, x.mem X  /\  V.PowerDomain.mem W  /\  V.upSet X  subseteq  W := by
```

```
  rw [PDsingleton, PDfinJoin, PDmem_finJoinSucc, PDmemFinJoin]
  constructor
  * rintro <X, hX, hPD, hsub>
    exact <X 0, hX 0, hPD, hsub 0>
  * rintro <X, hX, hPD, hsub>
    refine <fun _ => X, fun _ => hX, hPD, fun _ => hsub>

theorem PDmem_finJoin {n : Nat} (xs : Fin n  ->  V.Element) {W : Set (Set a)} :
    (V.PDfinJoin n xs).mem W  <->
      if n = 0 then W = V.PowerDomain.master else V.PDmemFinJoin xs W := by
  match n with
  | 0 => simp [PDfinJoin, PDfinJoinZero, mem_bot]
  | m + 1 => simp [PDfinJoin, PDfinJoinSucc]

/-- **Definition 7.11 -- Scott's displayed union form.** For `n  >=  1`, membership is
    equivalent to
some representatives `X_i  in  x_i` with `?_{i<n} X_i  subseteq  W`. -/
theorem PDmem_finJoin_iUnion {m : Nat} (xs : Fin (m + 1)  ->  V.Element) {W : Set (Set
     a)} :
    (V.PDfinJoinSucc xs).mem W  <->
      exists  (X : Fin (m + 1)  ->  Set a) (_hX : forall  i, (xs i).mem (X i)),
        V.PowerDomain.mem W  /\
          (? i, V.upSet (X i))  subseteq  W := by
  rw [PDmem_finJoinSucc]
  constructor
  * rintro <X, hX, hPD, hsub>
    exact <X, hX, hPD, Set.iUnion_subset hsub>
  * rintro <X, hX, hPD, hsub>
    exact <X, hX, hPD, (Set.iUnion_subset_iff).mp hsub>

end

end NeighborhoodSystem
section
open NeighborhoodSystem Domain.Recursive ApproximableMap
variable {a b ? : Type*}

namespace NeighborhoodSystem

variable (V : NeighborhoodSystem a)

/-- **`??`-neighbourhoods are downward closed.** If `Z  in  A` and `X  subseteq  Z`
    with `X  in  ?`, then `X  in  A`
(each `A = ? X_i` is a union of down-sets). -/
theorem PDmem_down {A : Set (Set a)} (hA : V.PDmem A) {Z X : Set a}
    (hZ : Z  in  A) (hXZ : X  subseteq  Z) (hX : V.mem X) : X  in  A := by
  obtain <L, hL, rfl> := hA
  obtain <X', hX', _, hZsub> := (V.mem_PDunion).mp hZ
  exact (V.mem_PDunion).mpr <X', hX', hX, hXZ.trans hZsub>

end NeighborhoodSystem
end

end DomainTheory.NeighbourhoodSystems
end
```

3.8.13 Proposition 7.12 — finite sets are approximable

PROPOSITION 7.12. The mapping

$$\lambda x_0, \ldots, x_{n-1}. \{x_0, \ldots, x_{n-1}\} : D^n \to \mathbb{P}D$$

is approximable and is computable if D is effectively given. Moreover, the map $\lambda x. \{x\}$ shows that $D \trianglelefteq \mathbb{P}D$, and we also have the law:

$$\{x_0, \ldots, x_{n-1}\} = \{x_0\} \cap \cdots \cap \{x_{n-1}\}$$

as an intersection of filters.

Proof : The second part shows that everything reduces to $\lambda x. \{x\}$. We see that

$$X_k \ (\lambda x. \{x\}) \bigcup_{i<q} (\uparrow X_{n_i}) \text{ iff } \exists i < q. X_k \sqsubseteq X_{n_i}.$$

Thus, $\lambda x. \{x\}$ is an approximable mapping and is computable in the effectively given case.

The proof of the law can be reduced to the special case

$$\{x\} \cap \{y\} = \{x, y\}$$

for the sake of illustration. In terms of finite elements of the two domains D and $\mathbb{P}D$ we find

$$\{\uparrow X\} = \uparrow\uparrow X,$$

and so,

$$\begin{aligned} \{\uparrow X\} \cap \{\uparrow Y\} &= \uparrow\uparrow X \cap \uparrow\uparrow Y \\ &= \uparrow (\uparrow X \cup \uparrow Y) \\ &= \{\uparrow X, \uparrow Y\}. \end{aligned}$$

An equation between approximable functions that checks for finite elements also holds for all elements.

Finally, we note that

$$D \cong D^\dagger \trianglelefteq \mathbb{P}D$$

and that the isomorphism involved is just $\lambda x. \{x\}$ by what we saw on the finite elements. □

Further combinators on the power domain are given in the exercises.

Proposition_7_12.lean

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Theorem_7_4
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Definition_7_11
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_4
public import Mathlib.Data.Fin.VecNotation

@[expose] public section
```

```
section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem ApproximableMap ApproximableMap_2 Domain.Recursive
namespace NeighborhoodSystem
variable {a : Type*} (V : NeighborhoodSystem a)

/- concordance: source L4563 PROPOSITION 7.12 finite sets are approximable -/
/-- **Proposition 7.12 -- monotonicity of `{*}` on principal inputs.** When `X'
    subseteq  X`, the join
`{X'}` is blunter than `{X}` in `|??|`. -/
theorem PDsingleton_mono {X X' : Set a} (hX : V.mem X) (hX' : V.mem X') (hX'X : X'
    subseteq  X) :
    V.PDsingleton (V.principal hX)  <=  V.PDsingleton (V.principal hX') := by
  intro W hW
  rw [PDmem_singleton] at hW  |-
  obtain <Y, hYpr, hPD, hsub> := hW
  rw [mem_principal] at hYpr
  exact <Y, <hYpr.1, hX'X.trans hYpr.2>, hPD, hsub>

/-- **Proposition 7.12 -- the singleton approximable map `fun  x. {x}`.** Built from
    Exercise 2.8's
extension principle: on the finite element `X` the value is `{X}`. -/
def PDsingletonApproxMap : ApproximableMap V V.PowerDomain :=
  ofMono (fun _X hX => V.PDsingleton (V.principal hX))
    (fun X X' hX hX' hX'X => @PDsingleton_mono a V X X' hX hX' hX'X)

@[simp] theorem PDsingletonApproxMap_rel {X : Set a} {W : Set (Set a)} (hX : V.mem X)
    :
    (PDsingletonApproxMap (V := V)).rel X W  <->  (V.PDsingleton (V.principal hX)).mem
     W := by
  dsimp [PDsingletonApproxMap, ofMono]
  constructor
  * intro <_, h>; exact h
  * intro h; exact <hX, h>

/-- Meet of two `{*}`-values in `|??|`. -/
def PDsingletonMeet (a b : V.PowerDomain.Element) : V.PowerDomain.Element where
  mem W := a.mem W  /\  b.mem W
  sub h := a.sub h.1
  master_mem := <a.master_mem, b.master_mem>
  inter_mem h1 h2 := <a.inter_mem h1.1 h2.1, b.inter_mem h1.2 h2.2>
  up_mem h hW hsub := <a.up_mem h.1 hW hsub, b.up_mem h.2 hW hsub>

@[simp] theorem PDsingletonMeet_mem {a b : V.PowerDomain.Element} {W : Set (Set a)} :
    (V.PDsingletonMeet a b).mem W  <->  a.mem W  /\  b.mem W := Iff.rfl

/-- **Proposition 7.12 -- the binary join as a two-variable approximable map.** -/
def PDfinJoinApproxMap_2 : ApproximableMap_2 V V V.PowerDomain where
  rel X Y W := (PDsingletonApproxMap (V := V)).rel X W  /\  (PDsingletonApproxMap (V
    := V)).rel Y W
  rel_dom_0 h := (PDsingletonApproxMap (V := V)).rel_dom h.1
  rel_dom_1 h := (PDsingletonApproxMap (V := V)).rel_dom h.2
  rel_cod h := (PDsingletonApproxMap (V := V)).rel_cod h.1
  master_rel := <(PDsingletonApproxMap (V := V)).master_rel, (PDsingletonApproxMap (V
```

```
    := V)).master_rel>
  inter_right := fun <h1, h2> <h1', h2'> =>
    <(PDsingletonApproxMap (V := V)).inter_right h1 h1',
      (PDsingletonApproxMap (V := V)).inter_right h2 h2'>
  mono := fun <h1, h2> hXX' hYY' hWW' hX' hY' hW' =>
    <(PDsingletonApproxMap (V := V)).mono h1 hXX' hWW' hX' hW',
      (PDsingletonApproxMap (V := V)).mono h2 hYY' hWW' hY' hW'>

/-- **Proposition 7.12 -- the binary join map on `D  x  D`.** -/
def finJoinMap_prod : ApproximableMap (prod V V) V.PowerDomain := ofMap_2 (
    PDfinJoinApproxMap_2 (V := V))

@[simp] theorem finJoinMap_prod_rel {X Y : Set a} (hX : V.mem X) (hY : V.mem Y) {W :
    Set (Set a)} :
    (finJoinMap_prod (V := V)).rel (prodNbhd X Y) W  <->
      (PDfinJoinApproxMap_2 (V := V)).rel X Y W := by
  dsimp [finJoinMap_prod, ofMap_2]
  simp only [inl_preimage_prodNbhd, inr_preimage_prodNbhd]
  exact <fun <_, h> => h, fun h => <prod_mem_prodNbhd hX hY, h>>

end NeighborhoodSystem
end

section
open NeighborhoodSystem ApproximableMap ApproximableMap_2 Domain.Recursive

namespace Counterexample712C

open NeighborhoodSystem

variable {? ? : Type*}

/-- A domain *has a greatest element* if some element dominates every other. Order-
    isomorphisms and
the inclusion order on filters transport this property. -/
def HasTop (E : NeighborhoodSystem ?) : Prop := exists  t : E.Element, forall  x : E.
    Element, x  <=  t

/-- `{false}  I  {true} = {}` in `Set Bool`. -/
theorem ft_inter : ({false} : Set Bool)  I  {true} = {} := by
  ext b
  simp only [Set.mem_inter_iff, Set.mem_singleton_iff, Set.mem_empty_iff_false,
    iff_false, not_and]
  rintro rfl h; exact absurd h (by decide)

/-- **The improper filter.** When a system `E` is *unconditionally* closed under
    intersection, the
family of *all* its neighbourhoods is a genuine filter -- the greatest element of `|E
    |`. -/
def improperTop (E : NeighborhoodSystem ?)
    (hcl : forall  {X Y : Set ?}, E.mem X  ->  E.mem Y  ->  E.mem (X  I  Y)) : E.
    Element where
  mem := E.mem
  sub h := h
  master_mem := E.master_mem
  inter_mem h1 h2 := hcl h1 h2
```

```
  up_mem _ hY _ := hY

/-- `{true}  I  {false} = {}` in `Set Bool`. -/
theorem tf_inter : ({true} : Set Bool)  I  {false} = {} := by
  ext b
  simp only [Set.mem_inter_iff, Set.mem_singleton_iff, Set.mem_empty_iff_false,
    iff_false, not_and]
  rintro rfl h; exact absurd h (by decide)

/-- The flat two-point domain over `Bool`: the neighbourhoods are the master `Delta = {
    true, false}`
and the two singletons `{true}`, `{false}`. Since `{true}  I  {false} = {}` and `{}` is
     not a
neighbourhood (and has no neighbourhood subset), condition (ii) holds only vacuously
    there -- this is
a genuine Scott neighbourhood system whose two singletons are *inconsistent*. -/
def Vshape : NeighborhoodSystem Bool where
  mem X := X = Set.univ  \/  X = {true}  \/  X = {false}
  master := Set.univ
  master_nonempty := <true, Set.mem_univ _>
  master_mem := Or.inl rfl
  sub_master _ := Set.subset_univ _
  inter_mem := by
    rintro X Y Z hX hY hZ hZsub
    -- every neighbourhood is nonempty, so a witness `Z  subseteq  X  I  Y = {}` is
    impossible
    have hZne : Z.Nonempty := by
      rcases hZ with rfl | rfl | rfl
      * exact <true, Set.mem_univ true>
      * exact <true, rfl>
      * exact <false, rfl>
    obtain <z, hz> := hZne
    rcases hX with rfl | rfl | rfl <;> rcases hY with rfl | rfl | rfl
    * exact Or.inl (by rw [Set.inter_self])
    * exact Or.inr (Or.inl (by rw [Set.univ_inter]))
    * exact Or.inr (Or.inr (by rw [Set.univ_inter]))
    * exact Or.inr (Or.inl (by rw [Set.inter_univ]))
    * exact Or.inr (Or.inl (by rw [Set.inter_self]))
    * rw [tf_inter] at hZsub; exact absurd (hZsub hz) (Set.notMem_empty z)
    * exact Or.inr (Or.inr (by rw [Set.inter_univ]))
    * rw [ft_inter] at hZsub; exact absurd (hZsub hz) (Set.notMem_empty z)
    * exact Or.inr (Or.inr (by rw [Set.inter_self]))

end Counterexample712C
end

section
open NeighborhoodSystem ApproximableMap ApproximableMap_2 Domain.Recursive

namespace NeighborhoodSystem

variable {a : Type*} (V : NeighborhoodSystem a)

/-- Extensionality for `|??|` elements. -/
theorem PDext {a b : V.PowerDomain.Element} (h : forall  W, a.mem W  <->  b.mem W) : a
     = b :=
```

```
  @Element.ext (Set a) V.PowerDomain a b h

end NeighborhoodSystem
end

section
open NeighborhoodSystem ApproximableMap ApproximableMap_2 Domain.Recursive

namespace NeighborhoodSystem

variable {a : Type*} (V : NeighborhoodSystem a)

variable (P : ComputablePresentation V)

/-- **Proposition 7.12 -- binary join reduces to two singleton tests.** -/
theorem PDfinJoinApproxMap_2_rel_iff {X Y : Set a} (_hX : V.mem X) (_hY : V.mem Y) {W
    : Set (Set a)} :
    (PDfinJoinApproxMap_2 (V := V)).rel X Y W  <->
      (PDsingletonApproxMap (V := V)).rel X W  /\  (PDsingletonApproxMap (V := V)).rel
     Y W := Iff.rfl

end NeighborhoodSystem
end

section
open NeighborhoodSystem ApproximableMap ApproximableMap_2 Domain.Recursive

namespace NeighborhoodSystem

variable {a : Type*} (V : NeighborhoodSystem a)

/-- **Proposition 7.12 -- binary intersection law (elementwise).** `{x, y} = {x}  I  {y
    }`. -/
theorem PDfinJoin_pair (x y : V.Element) :
    V.PDfinJoin 2 ![x, y] = V.PDsingletonMeet (V.PDsingleton x) (V.PDsingleton y) :=
    by
  apply PDext
  intro W
  simp only [PDsingletonMeet_mem, PDmem_singleton, PDfinJoin, PDmem_finJoinSucc,
    PDmemFinJoin]
  constructor
  * rintro <X, hX, hPD, hsub>
    exact <<X 0, hX 0, hPD, hsub 0>, <X 1, hX 1, hPD, hsub 1>>
  * rintro <<X0, hx, hPD, hsub0>, <X1, hy, _, hsub1>>
    refine <![X0, X1], ?_, hPD, ?_>
    * intro i; fin_cases i <;> assumption
    * intro i; fin_cases i <;> assumption

/-- **Proposition 7.12 -- binary intersection law (membership form).** `{x, y}` meets `
    W` iff both
singleton joins do. The general `n`-ary fold `{x_0,...,x_{n-1}} = ?_i {x_i}` follows by
     iterating
`PDfinJoin_pair`. -/
theorem PDfinJoin_inter_two (x y : V.Element) {W : Set (Set a)} :
    (V.PDfinJoin 2 ![x, y]).mem W  <->
      (V.PDsingleton x).mem W  /\  (V.PDsingleton y).mem W := by
```

```
  rw [ <-  PDsingletonMeet_mem,  <-  PDfinJoin_pair]

end NeighborhoodSystem
end

section
open NeighborhoodSystem ApproximableMap ApproximableMap_2 Domain.Recursive

namespace NeighborhoodSystem

variable {a : Type*} (V : NeighborhoodSystem a)

variable (P : ComputablePresentation V)

theorem PDfinJoinApproxMap_2_rel_PD {X Y : Set a} (hX : V.mem X) (hY : V.mem Y) {W :
    Set (Set a)} :
    (PDfinJoinApproxMap_2 (V := V)).rel X Y W  <->
      (V.PDfinJoin 2 ![V.principal hX, V.principal hY]).mem W := by
  constructor
  * intro <h1, h2>
    rw [PDsingletonApproxMap_rel (hX := hX)] at h1
    rw [PDsingletonApproxMap_rel (hX := hY)] at h2
    exact (PDfinJoin_inter_two (x := V.principal hX) (y := V.principal hY)).2 <h1, h2>
  * intro h
    rcases (PDfinJoin_inter_two (x := V.principal hX) (y := V.principal hY)).1 h with
    <h1, h2>
    exact <by rw [PDsingletonApproxMap_rel (hX := hX)]; exact h1,
      by rw [PDsingletonApproxMap_rel (hX := hY)]; exact h2>

/-- **Proposition 7.12 -- code form of singleton membership.** `{X_n}` relates to `Y_k`
     iff some
index in `dl k` lies above `X_n`. -/
theorem PDsingletonApproxMap_rel_Ypd_iff {n k : Nat} :
    ((PDsingletonApproxMap (V := V))).rel (P.X n) (V.Ypd P k)  <->
      exists  b  in  decodeList k, P.X n  subseteq  P.X b := by
  rw [PDsingletonApproxMap_rel (hX := P.mem_X n), PDmem_singleton, PowerDomain_mem]
  constructor
  * rintro <Y, hYpr, _, hsub>
    have hn : P.X n  in  V.upSet Y := <P.mem_X n, hYpr.2>
    obtain <b, hb, hPab> := (V.mem_Ypd P).mp (hsub hn)
    exact <b, hb, hPab.2>
  * intro <b, hb, hnb>
    refine <P.X b, <P.mem_X b, hnb>, V.Ypd_isPDmem P k,
      (V.upSet_subset_Ypd_iff P).mpr <b, hb, subset_rfl>>

end NeighborhoodSystem
end

section
open NeighborhoodSystem ApproximableMap ApproximableMap_2 Domain.Recursive

namespace NeighborhoodSystem

variable {a : Type*} (V : NeighborhoodSystem a)

/-- **Proposition 7.12 -- action of `fun  x. {x}`.** `(fun  x. {x})(x) = {x}` for every
```

```
    `x  in  |?|`. -/
theorem PDsingletonApproxMap_toElementMap (x : V.Element) :
    (PDsingletonApproxMap (V := V)).toElementMap x = V.PDsingleton x := by
  apply PDext
  intro W
  rw [ApproximableMap.mem_toElementMap, PDmem_singleton]
  constructor
  * rintro <X, hxX, hrel>
    rw [PDsingletonApproxMap_rel (hX := x.sub hxX)] at hrel
    rw [PDmem_singleton] at hrel
    obtain <Y, <hYmem, hXY>, hPD, hsub> := hrel
    exact <Y, x.up_mem hxX hYmem hXY, hPD, hsub>
  * intro hW
    obtain <Y, hy, hPD, hsub> := hW
    refine <Y, hy, ?_>
    rw [PDsingletonApproxMap_rel (hX := x.sub hy), PDmem_singleton]
    exact <Y, <x.sub hy, subset_rfl>, hPD, hsub>

/-- **Proposition 7.12 -- binary join action.** `PDfinJoinApproxMap_2(x, y) = {x, y}`.
    -/
theorem PDfinJoinApproxMap_2_toElementMap (x y : V.Element) :
    (PDfinJoinApproxMap_2 (V := V)).toElementMap_2 x y = V.PDfinJoin 2 ![x, y] := by
  apply PDext
  intro W
  rw [PDfinJoin_inter_two, ApproximableMap_2.mem_toElementMap_2]
  constructor
  * rintro <X, Y, hxX, hyY, <h1, h2>>
    constructor
    * rw [ <-  PDsingletonApproxMap_toElementMap (V := V)]
      exact (ApproximableMap.mem_toElementMap (PDsingletonApproxMap (V := V)) x).mpr <
    X, hxX, h1>
    * rw [ <-  PDsingletonApproxMap_toElementMap (V := V)]
      exact (ApproximableMap.mem_toElementMap (PDsingletonApproxMap (V := V)) y).mpr <
    Y, hyY, h2>
  * intro <h1, h2>
    rw [ <-  PDsingletonApproxMap_toElementMap (V := V)] at h1 h2
    obtain <X, hxX, hrelX> :=
      (ApproximableMap.mem_toElementMap (PDsingletonApproxMap (V := V)) x).mp h1
    obtain <Y, hyY, hrelY> :=
      (ApproximableMap.mem_toElementMap (PDsingletonApproxMap (V := V)) y).mp h2
    exact <X, Y, hxX, hyY, hrelX, hrelY>

/-- **Proposition 7.12 -- on finite elements `{X} = (X)`.** The singleton join of the
    principal
filter `X  subseteq  |?|` is the principal filter in `|??|` generated by the down-set
    token `X`. -/
theorem PDsingleton_principal {X : Set a} (hX : V.mem X) :
    V.PDsingleton (V.principal hX) =
      V.PowerDomain.principal (V.PDmem_upSet hX) := by
  apply PDext
  intro W
  rw [PDmem_singleton, mem_principal, PowerDomain_mem]
  constructor
  * rintro <Y, <hYmem, hXY>, hPD, hsub>
    exact <hPD, ((V.upSet_subset_iff hX).mpr hXY).trans hsub>
  * rintro <hPD, hsub>
```

```
    exact <X, <hX, subset_rfl>, hPD, hsub>

end NeighborhoodSystem
end

section
open NeighborhoodSystem ApproximableMap ApproximableMap_2 Domain.Recursive

namespace Counterexample712C

open NeighborhoodSystem

variable {? ? : Type*}

theorem Vshape_mem_false : Vshape.mem {false} := Or.inr (Or.inr rfl)

theorem Vshape_mem_true : Vshape.mem {true} := Or.inr (Or.inl rfl)

/-- `{}` is not a `Vshape`-neighbourhood (each neighbourhood is nonempty). -/
theorem Vshape_not_mem_empty : not  Vshape.mem ({} : Set Bool) := by
  rintro (h | h | h)
  * exact Set.empty_ne_univ h
  * exact absurd h.symm (Set.singleton_ne_empty true)
  * exact absurd h.symm (Set.singleton_ne_empty false)

/-- **`|Vshape|` has no greatest element.** A greatest element would contain both `{
    true}` and
`{false}`, hence (by `inter_mem`) their intersection `{}`, which is not a neighbourhood
    . -/
theorem Vshape_not_hasTop : not  HasTop Vshape := by
  rintro <t, ht>
  have htt : t.mem {true} :=
    ht (Vshape.principal Vshape_mem_true) {true} <Vshape_mem_true, subset_rfl>
  have htf : t.mem {false} :=
    ht (Vshape.principal Vshape_mem_false) {false} <Vshape_mem_false, subset_rfl>
  have hint : t.mem (({true} : Set Bool)  I  {false}) := t.inter_mem htt htf
  rw [tf_inter] at hint
  exact Vshape_not_mem_empty (t.sub hint)

end Counterexample712C
end

section
open NeighborhoodSystem ApproximableMap ApproximableMap_2 Domain.Recursive

namespace NeighborhoodSystem

variable {a : Type*} (V : NeighborhoodSystem a)

/-- Choice-free extensionality from agreement on principal inputs (cf. Exercise 6.27).
     -/
theorem ext_of_principal {b : Type*} {W : NeighborhoodSystem b}
    {f g : ApproximableMap V W}
    (h : forall  (X : Set a) (hX : V.mem X),
      f.toElementMap (V.principal hX) = g.toElementMap (V.principal hX)) : f = g := by
  apply ApproximableMap.ext
```

```
  intro X Y
  constructor
  * intro hr
    have hX := f.rel_dom hr
    rw [g.rel_iff_mem_principal hX,  <-  h X hX,  <-  f.rel_iff_mem_principal hX];
    exact hr
  * intro hr
    have hX := g.rel_dom hr
    rw [f.rel_iff_mem_principal hX, h X hX,  <-  g.rel_iff_mem_principal hX]; exact hr

theorem finJoinMap_prod_toElementMap (x y : V.Element) :
    (finJoinMap_prod (V := V)).toElementMap (pair x y) = V.PDfinJoin 2 ![x, y] := by
  have hmap : toMap_2 (finJoinMap_prod (V := V)) = PDfinJoinApproxMap_2 (V := V) :=
    toMap_2_ofMap_2 _
  rw [ <-  toElementMap_2_toMap_2 (finJoinMap_prod (V := V)), hmap,
    PDfinJoinApproxMap_2_toElementMap]

end NeighborhoodSystem
end

section
open NeighborhoodSystem ApproximableMap ApproximableMap_2 Domain.Recursive

namespace Counterexample712C

open NeighborhoodSystem

variable {? ? : Type*}

/-- An unconditionally  I -closed system has a greatest element. -/
theorem hasTop_of_inter_closed (E : NeighborhoodSystem ?)
    (hcl : forall  {X Y : Set ?}, E.mem X  ->  E.mem Y  ->  E.mem (X  I  Y)) : HasTop
    E :=
  <improperTop E hcl, fun x _X hX => x.sub hX>

/-- **`HasTop` transports across domain isomorphisms.** An order-iso `|D| ?o |D'|` 
    carries the
greatest element of `|D'|` to that of `|D|`. -/
theorem hasTop_of_iso {D : NeighborhoodSystem ?} {D' : NeighborhoodSystem ?}
    (e : D.Element ?o D'.Element) (h : HasTop D') : HasTop D := by
  obtain <t, ht> := h
  refine <e.symm t, fun x => ?_>
  rw [ <-  e.le_iff_le, e.apply_symm_apply]
  exact ht (e x)


/-- **`??` is unconditionally closed under intersection** (`PDmem_inter`; the empty 
    union always
supplies the consistency witness). Hence `|??|` always has a greatest element. -/
theorem powerDomain_hasTop (V : NeighborhoodSystem ?) : HasTop V.PowerDomain :=
  hasTop_of_inter_closed V.PowerDomain (fun hX hY => V.PDmem_inter hX hY)

end Counterexample712C
end

section
```

```
open NeighborhoodSystem ApproximableMap ApproximableMap_2 Domain.Recursive

namespace NeighborhoodSystem

variable {a : Type*} (V : NeighborhoodSystem a)

variable (P : ComputablePresentation V)

/-- **Proposition 7.12 -- `fun  x. {x}` is computable when `D` is effectively given.**
    -/
theorem singleton_isComputable (cons : Nat  ->  Nat) (hconsp : Nat.Primrec cons)
    (hcons : forall  a b, cons (Nat.pair a b) = 1  <->  exists  k, P.X k  subseteq  P.
    X a  I  P.X b) :
    ComputableMap P (V.PDPresentation P cons hconsp hcons) ((PDsingletonApproxMap (V
    := V))) := by
  constructor
  have h := P.incl_computable.swap.bExistsList.swap
  refine (RecDecidable.of_iff (fun s => ?_) h).re
  show PDsingletonApproxMap (V := V).rel (P.X s.unpair.1) ((V.PDPresentation P cons
    hconsp hcons).X s.unpair.2)  <->
      exists  e  in  decodeList s.unpair.2, P.X s.unpair.1  subseteq  P.X e
  simp only [PDPresentation]
  apply PDsingletonApproxMap_rel_Ypd_iff (V := V)

/-- **Proposition 7.12 -- the binary join map is computable.** Two independent
    singleton tests. -/
theorem PDfinJoinApproxMap_2_isComputable (cons : Nat  ->  Nat) (hconsp : Nat.Primrec
    cons)
    (hcons : forall  a b, cons (Nat.pair a b) = 1  <->  exists  k, P.X k  subseteq  P.
    X a  I  P.X b) :
    ComputableMap (prodPresentation P P)
      (V.PDPresentation P cons hconsp hcons) (finJoinMap_prod (V := V)) := by
  constructor
  have hs : REPred (fun s => (PDsingletonApproxMap (V := V)).rel (P.X s.unpair.1) (V.
    Ypd P s.unpair.2)) :=
    (singleton_isComputable (V := V) P cons hconsp hcons).out
  have hra : Nat.Primrec (fun t => Nat.pair t.unpair.1.unpair.1 t.unpair.2) :=
    (Nat.Primrec.left.comp Nat.Primrec.left).pair Nat.Primrec.right
  have hrb : Nat.Primrec (fun t => Nat.pair t.unpair.1.unpair.2 t.unpair.2) :=
    (Nat.Primrec.right.comp Nat.Primrec.left).pair Nat.Primrec.right
  refine REPred.of_iff (fun t => by
    dsimp [prodPresentation_X, finJoinMap_prod, ofMap_2]
    simp only [inl_preimage_prodNbhd, inr_preimage_prodNbhd,
      PDPresentation, unpair_pair_fst, unpair_pair_snd]
    constructor
    * rintro <_, <h1, h2>>
      exact <h1, h2>
    * intro <h1, h2>
      refine <prod_mem_prodNbhd (P.mem_X _) (P.mem_X _), h1, h2>) ((hs.comp hra).and (
    hs.comp hrb))

end NeighborhoodSystem
end

section
open NeighborhoodSystem ApproximableMap ApproximableMap_2 Domain.Recursive
```

```
namespace Counterexample712C

open NeighborhoodSystem

variable {? ? : Type*}

/-- A subsystem of an unconditionally  I -closed system is itself unconditionally  I -
    closed: the
`?`-clause `inter_closed` routes the intersection through `E`. -/
theorem subsystem_inter_closed {D E : NeighborhoodSystem ?} (h : D ? E)
    (hE : forall  {X Y : Set ?}, E.mem X  ->  E.mem Y  ->  E.mem (X  I  Y))
    {X Y : Set ?} (hX : D.mem X) (hY : D.mem Y) : D.mem (X  I  Y) :=
  h.inter_closed hX hY (hE (h.sub hX) (h.sub hY))

/-- **`D ? ??` forces `|D|` to have a greatest element.** This is the obstruction
    behind Part C. -/
theorem hasTop_of_trianglelefteq_powerDomain {D : NeighborhoodSystem ?} {E :
    NeighborhoodSystem ?}
    (h : D ? E.PowerDomain) : HasTop D := by
  obtain <D', hsub, <e>> := h
  have hPcl : forall  {X Y : Set (Set ?)}, E.PowerDomain.mem X  ->  E.PowerDomain.mem
    Y  ->
      E.PowerDomain.mem (X  I  Y) := fun hX hY => E.PDmem_inter hX hY
  exact hasTop_of_iso e (hasTop_of_inter_closed D' (subsystem_inter_closed hsub hPcl))

/-- **Proposition 7.12, Part C is FALSE in general.** The flat two-point domain `Vshape
    ` is *not* a
subdomain of its own Smyth power domain: `Vshape ? ?(Vshape)`. (Contrast Parts A, B, D,
     which hold
for every `?`.) -/
theorem vshape_not_trianglelefteq_powerDomain : not  (Vshape ? Vshape.PowerDomain) :=
    fun h =>
  Vshape_not_hasTop (hasTop_of_trianglelefteq_powerDomain h)

end Counterexample712C
end

section
set_option linter.unusedVariables false

theorem theorem_712.{u_1} : And
  (And
    (And
      (And
        (And
          (And
            (And
              (And
                (And
                  (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem a)
                    (x y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    Element a V),
                    @Eq
```

```
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
 (Set a)
                      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V))
                    (@DomainTheory.NeighbourhoodSystems.ApproximableMap_2.
toElementMap_2 a a (Set a) V V
                      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDfinJoinApproxMap_2 a V) x y)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDfinJoin a V
                      (@OfNat.ofNat Nat (nat_lit 2) (instOfNatNat (nat_lit 2)))
                      (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                        (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))
x
                        (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                          (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0))
) y
                          (@Matrix.vecEmpty (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V))))))
                (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
                 (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a
 V) (cons : Nat  ->  Nat)
                 (hconsp : Nat.Primrec cons)
                 (hcons :
                   forall  (a b : Nat),
                     Iff (@Eq Nat (cons (Nat.pair a b)) (@OfNat.ofNat Nat (
nat_lit 1) (instOfNatNat (nat_lit 1))))
                       (exists  (k : Nat),
                         @LE.le (Set a) (@Set.instLE a)
                           (@DomainTheory.NeighbourhoodSystems.
ComputablePresentation.X a V P k)
                           (@Inter.inter (Set a) (@Set.instInter a)
                             (@DomainTheory.NeighbourhoodSystems.
ComputablePresentation.X a V P a)
                             (@DomainTheory.NeighbourhoodSystems.
ComputablePresentation.X a V P b)))),
                 @DomainTheory.NeighbourhoodSystems.ComputableMap (Sum a a) (Set
a)
                   (@DomainTheory.NeighbourhoodSystems.prod a a V V)
                   (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                   (@DomainTheory.NeighbourhoodSystems.prodPresentation a a V V P
 P)
                   (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDPresentation a V P cons hconsp hcons)
                   (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
finJoinMap_prod a V)))
              (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a) {X Y : Set a},
                @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X
->
```

```
                  @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V Y
 ->
                    forall  {W : Set (Set a)},
                      Iff
                        (@DomainTheory.NeighbourhoodSystems.ApproximableMap_2.rel
a a (Set a) V V
                          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDfinJoinApproxMap_2 a V) X Y W)
                        (And
                          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel
a (Set a) V
                            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
.PowerDomain a V)
                            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
.PDsingletonApproxMap a V) X W)
                          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel
a (Set a) V
                            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
.PowerDomain a V)
                            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
.PDsingletonApproxMap a V) Y W))))
            (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
              (x y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
 a V) {W : Set (Set a)},
              Iff
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem
 (Set a)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDfinJoin
 a V
                    (@OfNat.ofNat Nat (nat_lit 2) (instOfNatNat (nat_lit 2)))
                    (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                      (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))) x
                      (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                        (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0)))
y
                        (@Matrix.vecEmpty (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)))))
                  W)
                (And
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.
mem (Set a)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDsingleton a V x) W)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.
mem (Set a)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
```

```
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDsingleton a V y) W))))
          (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
            (x y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
 V),
            @Eq
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Set
a)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V))
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDfinJoin a V
                (@OfNat.ofNat Nat (nat_lit 2) (instOfNatNat (nat_lit 2)))
                (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                  (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))) x
                  (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                    (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0))) y
                    (@Matrix.vecEmpty (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)))))
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDsingletonMeet a V
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDsingleton
 a V x)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDsingleton
 a V y))))
        (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
          (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V) {n k
 : Nat},
          Iff
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel a (Set a) V
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a
 V)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDsingletonApproxMap a V)
              (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V P n
)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd a V P k))
            (exists  (b : Nat),
              And (@Membership.mem Nat (List Nat) (@List.instMembership Nat) (
Domain.Recursive.decodeList k) b)
                (@LE.le (Set a) (@Set.instLE a)
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V
 P n)
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V
 P b)))))
      (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
        (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V),
        @Eq
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Set a)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V
))
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a (Set
```

```
a) V
             (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V
)
             (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDsingletonApproxMap a V) x)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDsingleton a V x
)))
    (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a) {X X' : Set a}
      (hX : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X)
      (hX' : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X'),
      @LE.le (Set a) (@Set.instLE a) X' X  ->
        @LE.le
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Set a)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V
))
          (@Preorder.toLE
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Set a)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a
 V))
            (@PartialOrder.toPreorder
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Set
a)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V))
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
instPartialOrderElement (Set a)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V))))
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDsingleton a V
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.principal a V X
 hX))
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDsingleton a V
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.principal a V X
' hX'))))
  (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a) {X : Set a}
    (hX : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X),
    @Eq
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Set a)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V))
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDsingleton a V
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.principal a V X hX)
)
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.principal (Set a)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.upSet a V X)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDmem_upSet a V X
hX))))
(forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
a)
  (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V) (cons : Nat
 ->  Nat) (hconsp : Nat.Primrec cons)
  (hcons :
    forall  (a b : Nat),
      Iff (@Eq Nat (cons (Nat.pair a b)) (@OfNat.ofNat Nat (nat_lit 1) (
```

```
  instOfNatNat (nat_lit 1))))
           (exists  (k : Nat),
             @LE.le (Set a) (@Set.instLE a) (@DomainTheory.NeighbourhoodSystems.
  ComputablePresentation.X a V P k)
               (@Inter.inter (Set a) (@Set.instInter a)
                 (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V P a
  )
                 (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V P b
  )))),
    @DomainTheory.NeighbourhoodSystems.ComputableMap a (Set a) V
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V) P
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDPresentation a V P
  cons hconsp hcons)
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDsingletonApproxMap a
  V)))
(Not
  (@DomainTheory.NeighbourhoodSystems.Trianglelefteq Bool (Set Bool)
    DomainTheory.NeighbourhoodSystems.Counterexample712C.Vshape
    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain Bool
      DomainTheory.NeighbourhoodSystems.Counterexample712C.Vshape))) :=
@And.intro
(And
  (And
    (And
      (And
        (And
          (And
            (And
              (And
                (And
                  (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem a)
                    (x y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  Element a V),
                    @Eq
                      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
   (Set a)
                        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  PowerDomain a V))
                      (@DomainTheory.NeighbourhoodSystems.ApproximableMap_2.
  toElementMap_2 a a (Set a) V V
                        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  PowerDomain a V)
                        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  PDfinJoinApproxMap_2 a V) x y)
                      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  PDfinJoin a V
                        (@OfNat.ofNat Nat (nat_lit 2) (instOfNatNat (nat_lit 2)))
                        (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
                          (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))
  x
                          (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a V)
                            (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0))
  ) y
```

```
                          (@Matrix.vecEmpty (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V))))))
                (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
                 (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a
 V) (cons : Nat  ->  Nat)
                 (hconsp : Nat.Primrec cons)
                 (hcons :
                   forall  (a b : Nat),
                     Iff (@Eq Nat (cons (Nat.pair a b)) (@OfNat.ofNat Nat (
nat_lit 1) (instOfNatNat (nat_lit 1))))
                         (exists  (k : Nat),
                           @LE.le (Set a) (@Set.instLE a)
                             (@DomainTheory.NeighbourhoodSystems.
ComputablePresentation.X a V P k)
                             (@Inter.inter (Set a) (@Set.instInter a)
                               (@DomainTheory.NeighbourhoodSystems.
ComputablePresentation.X a V P a)
                               (@DomainTheory.NeighbourhoodSystems.
ComputablePresentation.X a V P b)))),
                 @DomainTheory.NeighbourhoodSystems.ComputableMap (Sum a a) (Set
a)
                   (@DomainTheory.NeighbourhoodSystems.prod a a V V)
                   (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                   (@DomainTheory.NeighbourhoodSystems.prodPresentation a a V V P
 P)
                   (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDPresentation a V P cons hconsp hcons)
                   (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
finJoinMap_prod a V)))
              (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a) {X Y : Set a},
                @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X
->
                  @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V Y
 ->
                    forall  {W : Set (Set a)},
                      Iff
                        (@DomainTheory.NeighbourhoodSystems.ApproximableMap_2.rel
a a (Set a) V V
                          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDfinJoinApproxMap_2 a V) X Y W)
                          (And
                            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel
a (Set a) V
                              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
.PowerDomain a V)
                              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
.PDsingletonApproxMap a V) X W)
                            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel
a (Set a) V
                              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
.PowerDomain a V)
```

```
                              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
.PDsingletonApproxMap a V) Y W))))
            (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
              (x y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
 a V) {W : Set (Set a)},
              Iff
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem
 (Set a)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDfinJoin
 a V
                    (@OfNat.ofNat Nat (nat_lit 2) (instOfNatNat (nat_lit 2)))
                    (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                      (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))) x
                      (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                        (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0)))
y
                        (@Matrix.vecEmpty (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)))))
                  W)
                (And
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.
mem (Set a)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDsingleton a V x) W)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.
mem (Set a)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDsingleton a V y) W))))
          (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
            (x y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
 V),
            @Eq
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Set
a)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V))
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDfinJoin a V
                (@OfNat.ofNat Nat (nat_lit 2) (instOfNatNat (nat_lit 2)))
                (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                  (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))) x
                  (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                    (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0))) y
                    (@Matrix.vecEmpty (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)))))
```

```
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDsingletonMeet a V
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDsingleton
 a V x)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDsingleton
 a V y))))
        (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
          (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V) {n k
 : Nat},
          Iff
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel a (Set a) V
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a
 V)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDsingletonApproxMap a V)
              (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V P n
)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd a V P k))
            (exists  (b : Nat),
              And (@Membership.mem Nat (List Nat) (@List.instMembership Nat) (
Domain.Recursive.decodeList k) b)
                (@LE.le (Set a) (@Set.instLE a)
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V
 P n)
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V
 P b)))))
      (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
        (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V),
        @Eq
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Set a)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V
))
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a (Set
a) V
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V
)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDsingletonApproxMap a V) x)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDsingleton a V x
)))
    (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a) {X X' : Set a}
      (hX : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X)
      (hX' : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X'),
      @LE.le (Set a) (@Set.instLE a) X' X  ->
        @LE.le
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Set a)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V
))
          (@Preorder.toLE
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Set a)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a
 V))
            (@PartialOrder.toPreorder
```

```
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Set
 a)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
  a V))
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 instPartialOrderElement (Set a)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
  a V))))
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDsingleton a V
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.principal a V X
  hX))
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDsingleton a V
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.principal a V X
 ' hX'))))
    (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem a) {X : Set a}
      (hX : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X),
      @Eq
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Set a)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V))
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDsingleton a V
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.principal a V X hX)
 )
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.principal (Set a)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.upSet a V X)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDmem_upSet a V X
 hX))))

 (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
 a)
    (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V) (cons : Nat
  ->  Nat) (hconsp : Nat.Primrec cons)
    (hcons :
      forall  (a b : Nat),
        Iff (@Eq Nat (cons (Nat.pair a b)) (@OfNat.ofNat Nat (nat_lit 1) (
 instOfNatNat (nat_lit 1))))
          (exists  (k : Nat),
            @LE.le (Set a) (@Set.instLE a) (@DomainTheory.NeighbourhoodSystems.
 ComputablePresentation.X a V P k)
              (@Inter.inter (Set a) (@Set.instInter a)
                (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V P a
 )
                (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V P b
 )))),
    @DomainTheory.NeighbourhoodSystems.ComputableMap a (Set a) V
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V) P
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDPresentation a V P
 cons hconsp hcons)
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDsingletonApproxMap a
 V)))
(Not
  (@DomainTheory.NeighbourhoodSystems.Trianglelefteq Bool (Set Bool)
    DomainTheory.NeighbourhoodSystems.Counterexample712C.Vshape
    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain Bool
      DomainTheory.NeighbourhoodSystems.Counterexample712C.Vshape)))
```

```
(@And.intro
  (And
    (And
      (And
        (And
          (And
            (And
              (And
                (And
                  (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem a)
                    (x y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 Element a V),
                    @Eq
                      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
  (Set a)
                        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 PowerDomain a V))
                      (@DomainTheory.NeighbourhoodSystems.ApproximableMap_2.
 toElementMap_2 a a (Set a) V V
                        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 PowerDomain a V)
                        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 PDfinJoinApproxMap_2 a V) x y)
                      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
 PDfinJoin a V
                        (@OfNat.ofNat Nat (nat_lit 2) (instOfNatNat (nat_lit 2)))
                        (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem.Element a V)
                          (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))
 x
                          (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem.Element a V)
                            (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0))
 ) y
                            (@Matrix.vecEmpty (@DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem.Element a V))))))
                  (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem a)
                    (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a
  V) (cons : Nat  ->  Nat)
                    (hconsp : Nat.Primrec cons)
                    (hcons :
                      forall  (a b : Nat),
                        Iff (@Eq Nat (cons (Nat.pair a b)) (@OfNat.ofNat Nat (
 nat_lit 1) (instOfNatNat (nat_lit 1))))
                          (exists  (k : Nat),
                            @LE.le (Set a) (@Set.instLE a)
                              (@DomainTheory.NeighbourhoodSystems.
 ComputablePresentation.X a V P k)
                              (@Inter.inter (Set a) (@Set.instInter a)
                                (@DomainTheory.NeighbourhoodSystems.
 ComputablePresentation.X a V P a)
                                (@DomainTheory.NeighbourhoodSystems.
 ComputablePresentation.X a V P b)))),
                    @DomainTheory.NeighbourhoodSystems.ComputableMap (Sum a a) (Set
```

```
a)
                    (@DomainTheory.NeighbourhoodSystems.prod a a V V)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                    (@DomainTheory.NeighbourhoodSystems.prodPresentation a a V V P
 P)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDPresentation a V P cons hconsp hcons)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
finJoinMap_prod a V)))
              (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a) {X Y : Set a},
                @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X
->
                  @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V Y
 ->
                    forall  {W : Set (Set a)},
                      Iff
                        (@DomainTheory.NeighbourhoodSystems.ApproximableMap_2.rel
a a (Set a) V V
                          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDfinJoinApproxMap_2 a V) X Y W)
                        (And
                          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel
a (Set a) V
                            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
.PowerDomain a V)
                            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
.PDsingletonApproxMap a V) X W)
                          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel
a (Set a) V
                            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
.PowerDomain a V)
                            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
.PDsingletonApproxMap a V) Y W))))
            (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
              (x y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
 a V) {W : Set (Set a)},
              Iff
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem
 (Set a)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDfinJoin
 a V
                    (@OfNat.ofNat Nat (nat_lit 2) (instOfNatNat (nat_lit 2)))
                    (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                      (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))) x
                      (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                        (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0)))
y
```

```
                        (@Matrix.vecEmpty (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)))))
                  W)
                (And
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.
mem (Set a)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDsingleton a V x) W)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.
mem (Set a)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDsingleton a V y) W))))
          (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
            (x y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
 V),
            @Eq
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Set
a)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V))
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDfinJoin a V
                (@OfNat.ofNat Nat (nat_lit 2) (instOfNatNat (nat_lit 2)))
                (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                  (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))) x
                  (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                    (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0))) y
                    (@Matrix.vecEmpty (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)))))
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDsingletonMeet a V
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDsingleton
 a V x)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDsingleton
 a V y))))
        (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
          (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V) {n k
 : Nat},
          Iff
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel a (Set a) V
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a
 V)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDsingletonApproxMap a V)
              (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V P n
)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd a V P k))
            (exists  (b : Nat),
              And (@Membership.mem Nat (List Nat) (@List.instMembership Nat) (
```

```
Domain.Recursive.decodeList k) b)
                (@LE.le (Set a) (@Set.instLE a)
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V
 P n)
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V
 P b)))))
      (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
        (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V),
        @Eq
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Set a)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V
))
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a (Set
a) V
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V
)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDsingletonApproxMap a V) x)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDsingleton a V x
)))
    (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a) {X X' : Set a}
      (hX : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X)
      (hX' : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X'),
      @LE.le (Set a) (@Set.instLE a) X' X  ->
        @LE.le
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Set a)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V
))
          (@Preorder.toLE
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Set a)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a
 V))
            (@PartialOrder.toPreorder
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Set
a)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V))
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
instPartialOrderElement (Set a)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V))))
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDsingleton a V
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.principal a V X
 hX))
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDsingleton a V
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.principal a V X
' hX'))))
  (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a) {X : Set a}
    (hX : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X),
    @Eq
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Set a)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V))
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDsingleton a V
```

```
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.principal a V X hX)
)
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.principal (Set a)
         (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V)
         (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.upSet a V X)
         (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDmem_upSet a V X
hX))))
 (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
a)
   (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V) (cons : Nat
 ->  Nat) (hconsp : Nat.Primrec cons)
   (hcons :
     forall  (a b : Nat),
       Iff (@Eq Nat (cons (Nat.pair a b)) (@OfNat.ofNat Nat (nat_lit 1) (
instOfNatNat (nat_lit 1))))
         (exists  (k : Nat),
           @LE.le (Set a) (@Set.instLE a) (@DomainTheory.NeighbourhoodSystems.
ComputablePresentation.X a V P k)
             (@Inter.inter (Set a) (@Set.instInter a)
               (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V P a
)
               (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V P b
)))),
   @DomainTheory.NeighbourhoodSystems.ComputableMap a (Set a) V
     (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V) P
     (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDPresentation a V P
cons hconsp hcons)
     (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDsingletonApproxMap a
V))
 (@And.intro
   (And
     (And
       (And
         (And
           (And
             (And
               (And
                 (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
                   (x y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a V),
                   @Eq
                     (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
 (Set a)
                       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V))
                     (@DomainTheory.NeighbourhoodSystems.ApproximableMap_2.
toElementMap_2 a a (Set a) V V
                       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDfinJoinApproxMap_2 a V) x y)
                     (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDfinJoin a V
                       (@OfNat.ofNat Nat (nat_lit 2) (instOfNatNat (nat_lit 2)))
                       (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
```

```
NeighborhoodSystem.Element a V)
                        (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))
x
                        (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                          (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0))
) y
                          (@Matrix.vecEmpty (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V))))))
                (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
                 (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a
 V) (cons : Nat  ->  Nat)
                 (hconsp : Nat.Primrec cons)
                 (hcons :
                   forall  (a b : Nat),
                     Iff (@Eq Nat (cons (Nat.pair a b)) (@OfNat.ofNat Nat (
nat_lit 1) (instOfNatNat (nat_lit 1))))
                        (exists  (k : Nat),
                          @LE.le (Set a) (@Set.instLE a)
                            (@DomainTheory.NeighbourhoodSystems.
ComputablePresentation.X a V P k)
                            (@Inter.inter (Set a) (@Set.instInter a)
                              (@DomainTheory.NeighbourhoodSystems.
ComputablePresentation.X a V P a)
                              (@DomainTheory.NeighbourhoodSystems.
ComputablePresentation.X a V P b)))),
                 @DomainTheory.NeighbourhoodSystems.ComputableMap (Sum a a) (Set
a)
                   (@DomainTheory.NeighbourhoodSystems.prod a a V V)
                   (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                   (@DomainTheory.NeighbourhoodSystems.prodPresentation a a V V P
 P)
                   (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDPresentation a V P cons hconsp hcons)
                   (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
finJoinMap_prod a V)))
              (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a) {X Y : Set a},
                @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X
->
                  @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V Y
 ->
                    forall  {W : Set (Set a)},
                      Iff
                        (@DomainTheory.NeighbourhoodSystems.ApproximableMap_2.rel
a a (Set a) V V
                          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDfinJoinApproxMap_2 a V) X Y W)
                          (And
                            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel
a (Set a) V
                              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
```

```
.PowerDomain a V)
                              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
.PDsingletonApproxMap a V) X W)
                            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel
a (Set a) V
                              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
.PowerDomain a V)
                              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
.PDsingletonApproxMap a V) Y W))))
            (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
              (x y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
 a V) {W : Set (Set a)},
              Iff
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem
 (Set a)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDfinJoin
 a V
                    (@OfNat.ofNat Nat (nat_lit 2) (instOfNatNat (nat_lit 2)))
                    (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                      (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))) x
                      (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                        (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0)))
y
                        (@Matrix.vecEmpty (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)))))
                  W)
                (And
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.
mem (Set a)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDsingleton a V x) W)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.
mem (Set a)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDsingleton a V y) W))))
          (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
            (x y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
 V),
            @Eq
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Set
a)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V))
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDfinJoin a V
                (@OfNat.ofNat Nat (nat_lit 2) (instOfNatNat (nat_lit 2)))
                (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
```

```
NeighborhoodSystem.Element a V)
                  (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))) x
                  (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                    (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0))) y
                    (@Matrix.vecEmpty (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)))))
               (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDsingletonMeet a V
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDsingleton
 a V x)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDsingleton
 a V y))))
        (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
          (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V) {n k
 : Nat},
          Iff
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel a (Set a) V
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a
 V)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDsingletonApproxMap a V)
              (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V P n
)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd a V P k))
            (exists  (b : Nat),
              And (@Membership.mem Nat (List Nat) (@List.instMembership Nat) (
Domain.Recursive.decodeList k) b)
                (@LE.le (Set a) (@Set.instLE a)
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V
 P n)
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V
 P b)))))
      (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
        (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V),
        @Eq
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Set a)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V
))
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a (Set
a) V
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V
)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDsingletonApproxMap a V) x)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDsingleton a V x
)))
    (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a) {X X' : Set a}
      (hX : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X)
      (hX' : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X'),
      @LE.le (Set a) (@Set.instLE a) X' X  ->
        @LE.le
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Set a)
```

```
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V
))
           (@Preorder.toLE
             (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Set a)
               (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a
 V))
             (@PartialOrder.toPreorder
               (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Set
a)
                 (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V))
               (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
instPartialOrderElement (Set a)
                 (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V))))
           (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDsingleton a V
             (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.principal a V X
 hX))
           (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDsingleton a V
             (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.principal a V X
' hX'))))
   (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a) {X : Set a}
     (hX : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X),
     @Eq
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Set a)
         (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V))
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDsingleton a V
         (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.principal a V X hX)
)
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.principal (Set a)
         (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V)
         (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.upSet a V X)
         (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDmem_upSet a V X
hX)))
   (@And.intro
     (And
       (And
         (And
           (And
             (And
               (And
                 (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
                  (x y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a V),
                  @Eq
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
 (Set a)
                      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V))
                    (@DomainTheory.NeighbourhoodSystems.ApproximableMap_2.
toElementMap_2 a a (Set a) V V
                      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
```

```
PDfinJoinApproxMap_2 a V) x y)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDfinJoin a V
                      (@OfNat.ofNat Nat (nat_lit 2) (instOfNatNat (nat_lit 2)))
                      (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                        (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))
x
                        (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                          (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0))
) y
                          (@Matrix.vecEmpty (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V))))))
                (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
                  (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a
 V) (cons : Nat  ->  Nat)
                  (hconsp : Nat.Primrec cons)
                  (hcons :
                    forall  (a b : Nat),
                      Iff (@Eq Nat (cons (Nat.pair a b)) (@OfNat.ofNat Nat (
nat_lit 1) (instOfNatNat (nat_lit 1))))
                        (exists  (k : Nat),
                          @LE.le (Set a) (@Set.instLE a)
                            (@DomainTheory.NeighbourhoodSystems.
ComputablePresentation.X a V P k)
                            (@Inter.inter (Set a) (@Set.instInter a)
                              (@DomainTheory.NeighbourhoodSystems.
ComputablePresentation.X a V P a)
                              (@DomainTheory.NeighbourhoodSystems.
ComputablePresentation.X a V P b)))),
                  @DomainTheory.NeighbourhoodSystems.ComputableMap (Sum a a) (Set
a)
                    (@DomainTheory.NeighbourhoodSystems.prod a a V V)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                    (@DomainTheory.NeighbourhoodSystems.prodPresentation a a V V P
 P)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDPresentation a V P cons hconsp hcons)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
finJoinMap_prod a V)))
               (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a) {X Y : Set a},
                 @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X
->
                   @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V Y
 ->
                     forall  {W : Set (Set a)},
                       Iff
                         (@DomainTheory.NeighbourhoodSystems.ApproximableMap_2.rel
a a (Set a) V V
                           (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                           (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
```

```
PDfinJoinApproxMap_2 a V) X Y W)
                          (And
                            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel
a (Set a) V
                              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
.PowerDomain a V)
                              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
.PDsingletonApproxMap a V) X W)
                            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel
a (Set a) V
                              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
.PowerDomain a V)
                              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
.PDsingletonApproxMap a V) Y W))))
             (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
               (x y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
 a V) {W : Set (Set a)},
               Iff
                 (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem
 (Set a)
                   (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                   (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDfinJoin
 a V
                     (@OfNat.ofNat Nat (nat_lit 2) (instOfNatNat (nat_lit 2)))
                     (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                       (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))) x
                       (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                         (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0)))
y
                         (@Matrix.vecEmpty (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)))))
                   W)
                 (And
                   (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.
mem (Set a)
                     (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                     (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDsingleton a V x) W)
                   (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.
mem (Set a)
                     (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                     (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDsingleton a V y) W))))
           (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
             (x y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
 V),
             @Eq
               (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Set
a)
```

```
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V))

               (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDfinJoin a V
                 (@OfNat.ofNat Nat (nat_lit 2) (instOfNatNat (nat_lit 2)))
                 (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                   (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))) x
                   (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                     (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0))) y
                     (@Matrix.vecEmpty (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)))))
               (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDsingletonMeet a V
                 (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDsingleton
 a V x)
                 (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDsingleton
 a V y))))
         (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
           (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V) {n k
 : Nat},
           Iff
             (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel a (Set a) V
               (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a
 V)
               (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDsingletonApproxMap a V)
               (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V P n
)
               (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd a V P k))
             (exists  (b : Nat),
               And (@Membership.mem Nat (List Nat) (@List.instMembership Nat) (
Domain.Recursive.decodeList k) b)
                 (@LE.le (Set a) (@Set.instLE a)
                   (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V
 P n)
                   (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V
 P b)))))
       (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
         (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V),
         @Eq
           (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Set a)
             (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V
))
           (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a (Set
a) V
             (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V
)
             (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDsingletonApproxMap a V) x)
           (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDsingleton a V x
)))
     (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
```

```
NeighborhoodSystem a) {X X' : Set a}
       (hX : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X)
       (hX' : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X'),
       @LE.le (Set a) (@Set.instLE a) X' X  ->
         @LE.le
           (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Set a)
             (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V
))
           (@Preorder.toLE
             (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Set a)
               (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a
 V))
             (@PartialOrder.toPreorder
               (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Set
a)
                 (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V))
               (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
instPartialOrderElement (Set a)
                 (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V))))
           (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDsingleton a V
             (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.principal a V X
 hX))
           (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDsingleton a V
             (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.principal a V X
' hX')))
     (@And.intro
       (And
         (And
           (And
             (And
               (And
                 (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
                   (x y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a V),
                   @Eq
                     (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
 (Set a)
                       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V))
                     (@DomainTheory.NeighbourhoodSystems.ApproximableMap_2.
toElementMap_2 a a (Set a) V V
                       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDfinJoinApproxMap_2 a V) x y)
                     (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDfinJoin a V
                       (@OfNat.ofNat Nat (nat_lit 2) (instOfNatNat (nat_lit 2)))
                       (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                         (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))
x
                         (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
```

```
NeighborhoodSystem.Element a V)
                          (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0))
) y
                          (@Matrix.vecEmpty (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V))))))
                (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
                 (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a
 V) (cons : Nat  ->  Nat)
                 (hconsp : Nat.Primrec cons)
                 (hcons :
                   forall  (a b : Nat),
                     Iff (@Eq Nat (cons (Nat.pair a b)) (@OfNat.ofNat Nat (
nat_lit 1) (instOfNatNat (nat_lit 1))))
                       (exists  (k : Nat),
                         @LE.le (Set a) (@Set.instLE a)
                           (@DomainTheory.NeighbourhoodSystems.
ComputablePresentation.X a V P k)
                           (@Inter.inter (Set a) (@Set.instInter a)
                             (@DomainTheory.NeighbourhoodSystems.
ComputablePresentation.X a V P a)
                             (@DomainTheory.NeighbourhoodSystems.
ComputablePresentation.X a V P b)))),
                 @DomainTheory.NeighbourhoodSystems.ComputableMap (Sum a a) (Set
a)
                   (@DomainTheory.NeighbourhoodSystems.prod a a V V)
                   (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                   (@DomainTheory.NeighbourhoodSystems.prodPresentation a a V V P
 P)
                   (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDPresentation a V P cons hconsp hcons)
                   (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
finJoinMap_prod a V)))
              (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a) {X Y : Set a},
               @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X
->
                 @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V Y
 ->
                   forall  {W : Set (Set a)},
                     Iff
                       (@DomainTheory.NeighbourhoodSystems.ApproximableMap_2.rel
a a (Set a) V V
                         (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                         (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDfinJoinApproxMap_2 a V) X Y W)
                       (And
                         (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel
a (Set a) V
                           (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
.PowerDomain a V)
                           (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
.PDsingletonApproxMap a V) X W)
                         (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel
```

```
a (Set a) V
                              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
.PowerDomain a V)
                              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
.PDsingletonApproxMap a V) Y W))))
            (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
              (x y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
 a V) {W : Set (Set a)},
              Iff
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem
 (Set a)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDfinJoin
 a V
                    (@OfNat.ofNat Nat (nat_lit 2) (instOfNatNat (nat_lit 2)))
                    (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                      (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))) x
                      (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                        (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0)))
y
                        (@Matrix.vecEmpty (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)))))
                  W)
                (And
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.
mem (Set a)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDsingleton a V x) W)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.
mem (Set a)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDsingleton a V y) W))))
          (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
            (x y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
 V),
            @Eq
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Set
a)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V))
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDfinJoin a V
                (@OfNat.ofNat Nat (nat_lit 2) (instOfNatNat (nat_lit 2)))
                (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                  (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))) x
                  (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
```

```
                    (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0))) y
                    (@Matrix.vecEmpty (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)))))
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDsingletonMeet a V
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDsingleton
 a V x)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDsingleton
 a V y))))
        (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
          (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V) {n k
 : Nat},
          Iff
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel a (Set a) V
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a
 V)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDsingletonApproxMap a V)
              (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V P n
)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd a V P k))
            (exists  (b : Nat),
              And (@Membership.mem Nat (List Nat) (@List.instMembership Nat) (
Domain.Recursive.decodeList k) b)
                (@LE.le (Set a) (@Set.instLE a)
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V
 P n)
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V
 P b)))))
      (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
        (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a V),
        @Eq
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Set a)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V
))
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a (Set
a) V
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V
)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDsingletonApproxMap a V) x)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDsingleton a V x
))
      (@And.intro
        (And
          (And
            (And
              (And
                (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
                  (x y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a V),
                  @Eq
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
```

```
 (Set a)
                      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V))
                    (@DomainTheory.NeighbourhoodSystems.ApproximableMap_2.
toElementMap_2 a a (Set a) V V
                      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDfinJoinApproxMap_2 a V) x y)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDfinJoin a V
                      (@OfNat.ofNat Nat (nat_lit 2) (instOfNatNat (nat_lit 2)))
                      (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                        (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))
x
                        (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                          (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0))
) y
                          (@Matrix.vecEmpty (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V))))))
                (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
                  (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a
 V) (cons : Nat  ->  Nat)
                  (hconsp : Nat.Primrec cons)
                  (hcons :
                    forall  (a b : Nat),
                      Iff (@Eq Nat (cons (Nat.pair a b)) (@OfNat.ofNat Nat (
nat_lit 1) (instOfNatNat (nat_lit 1))))
                        (exists  (k : Nat),
                          @LE.le (Set a) (@Set.instLE a)
                            (@DomainTheory.NeighbourhoodSystems.
ComputablePresentation.X a V P k)
                            (@Inter.inter (Set a) (@Set.instInter a)
                              (@DomainTheory.NeighbourhoodSystems.
ComputablePresentation.X a V P a)
                              (@DomainTheory.NeighbourhoodSystems.
ComputablePresentation.X a V P b)))),
                  @DomainTheory.NeighbourhoodSystems.ComputableMap (Sum a a) (Set
a)
                    (@DomainTheory.NeighbourhoodSystems.prod a a V V)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                    (@DomainTheory.NeighbourhoodSystems.prodPresentation a a V V P
 P)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDPresentation a V P cons hconsp hcons)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
finJoinMap_prod a V)))
              (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a) {X Y : Set a},
                @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X
->
                  @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V Y
```

```
 ->
                    forall  {W : Set (Set a)},
                      Iff
                        (@DomainTheory.NeighbourhoodSystems.ApproximableMap_2.rel
a a (Set a) V V
                          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDfinJoinApproxMap_2 a V) X Y W)
                        (And
                          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel
a (Set a) V
                            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
.PowerDomain a V)
                            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
.PDsingletonApproxMap a V) X W)
                          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel
a (Set a) V
                            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
.PowerDomain a V)
                            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
.PDsingletonApproxMap a V) Y W))))
            (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
              (x y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
 a V) {W : Set (Set a)},
              Iff
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem
 (Set a)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDfinJoin
 a V
                    (@OfNat.ofNat Nat (nat_lit 2) (instOfNatNat (nat_lit 2)))
                    (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                      (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))) x
                      (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                        (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0)))
y
                        (@Matrix.vecEmpty (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)))))
                  W)
                (And
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.
mem (Set a)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDsingleton a V x) W)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.
mem (Set a)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
```

```
PDsingleton a V y) W))))
          (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
            (x y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
 V),
            @Eq
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Set
a)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V))
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDfinJoin a V
                (@OfNat.ofNat Nat (nat_lit 2) (instOfNatNat (nat_lit 2)))
                (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                  (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))) x
                  (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                    (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0))) y
                    (@Matrix.vecEmpty (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)))))
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDsingletonMeet a V
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDsingleton
 a V x)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDsingleton
 a V y))))
        (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
          (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V) {n k
 : Nat},
          Iff
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel a (Set a) V
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a
 V)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDsingletonApproxMap a V)
              (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V P n
)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd a V P k))
            (exists  (b : Nat),
              And (@Membership.mem Nat (List Nat) (@List.instMembership Nat) (
Domain.Recursive.decodeList k) b)
                (@LE.le (Set a) (@Set.instLE a)
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V
 P n)
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V
 P b))))
        (@And.intro
          (And
            (And
              (And
                (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
                  (x y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a V),
                  @Eq
```

```
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
 (Set a)
                      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V))
                    (@DomainTheory.NeighbourhoodSystems.ApproximableMap_2.
toElementMap_2 a a (Set a) V V
                      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDfinJoinApproxMap_2 a V) x y)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDfinJoin a V
                      (@OfNat.ofNat Nat (nat_lit 2) (instOfNatNat (nat_lit 2)))
                      (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                        (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))
x
                        (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                          (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0))
) y
                          (@Matrix.vecEmpty (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V))))))
                (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
                 (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a
 V) (cons : Nat  ->  Nat)
                 (hconsp : Nat.Primrec cons)
                 (hcons :
                   forall  (a b : Nat),
                     Iff (@Eq Nat (cons (Nat.pair a b)) (@OfNat.ofNat Nat (
nat_lit 1) (instOfNatNat (nat_lit 1))))
                       (exists  (k : Nat),
                         @LE.le (Set a) (@Set.instLE a)
                           (@DomainTheory.NeighbourhoodSystems.
ComputablePresentation.X a V P k)
                           (@Inter.inter (Set a) (@Set.instInter a)
                             (@DomainTheory.NeighbourhoodSystems.
ComputablePresentation.X a V P a)
                             (@DomainTheory.NeighbourhoodSystems.
ComputablePresentation.X a V P b)))),
                 @DomainTheory.NeighbourhoodSystems.ComputableMap (Sum a a) (Set
a)
                   (@DomainTheory.NeighbourhoodSystems.prod a a V V)
                   (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                   (@DomainTheory.NeighbourhoodSystems.prodPresentation a a V V P
 P)
                   (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDPresentation a V P cons hconsp hcons)
                   (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
finJoinMap_prod a V)))
              (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a) {X Y : Set a},
                @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X
->
```

```
                  @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V Y
 ->
                    forall  {W : Set (Set a)},
                      Iff
                        (@DomainTheory.NeighbourhoodSystems.ApproximableMap_2.rel
a a (Set a) V V
                          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDfinJoinApproxMap_2 a V) X Y W)
                        (And
                          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel
a (Set a) V
                            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
.PowerDomain a V)
                            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
.PDsingletonApproxMap a V) X W)
                          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel
a (Set a) V
                            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
.PowerDomain a V)
                            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
.PDsingletonApproxMap a V) Y W))))
             (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
               (x y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
 a V) {W : Set (Set a)},
               Iff
                 (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem
 (Set a)
                   (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                   (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDfinJoin
 a V
                     (@OfNat.ofNat Nat (nat_lit 2) (instOfNatNat (nat_lit 2)))
                     (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                       (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))) x
                       (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                         (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0)))
y
                         (@Matrix.vecEmpty (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)))))
                   W)
                 (And
                   (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.
mem (Set a)
                     (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                     (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDsingleton a V x) W)
                   (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.
mem (Set a)
                     (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
```

```
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDsingleton a V y) W))))
          (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
            (x y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
 V),
            @Eq
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (Set
a)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V))
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDfinJoin a V
                (@OfNat.ofNat Nat (nat_lit 2) (instOfNatNat (nat_lit 2)))
                (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                  (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))) x
                  (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                    (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0))) y
                    (@Matrix.vecEmpty (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)))))
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDsingletonMeet a V
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDsingleton
 a V x)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDsingleton
 a V y)))
          (@And.intro
            (And
              (And
                (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
                  (x y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a V),
                  @Eq
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
 (Set a)
                      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V))
                    (@DomainTheory.NeighbourhoodSystems.ApproximableMap_2.
toElementMap_2 a a (Set a) V V
                      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDfinJoinApproxMap_2 a V) x y)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDfinJoin a V
                      (@OfNat.ofNat Nat (nat_lit 2) (instOfNatNat (nat_lit 2)))
                      (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                        (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))
x
                        (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                          (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0))
) y
```

```
                          (@Matrix.vecEmpty (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V))))))
                (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
                 (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a
 V) (cons : Nat  ->  Nat)
                 (hconsp : Nat.Primrec cons)
                 (hcons :
                   forall  (a b : Nat),
                     Iff (@Eq Nat (cons (Nat.pair a b)) (@OfNat.ofNat Nat (
nat_lit 1) (instOfNatNat (nat_lit 1))))
                         (exists  (k : Nat),
                           @LE.le (Set a) (@Set.instLE a)
                             (@DomainTheory.NeighbourhoodSystems.
ComputablePresentation.X a V P k)
                             (@Inter.inter (Set a) (@Set.instInter a)
                               (@DomainTheory.NeighbourhoodSystems.
ComputablePresentation.X a V P a)
                               (@DomainTheory.NeighbourhoodSystems.
ComputablePresentation.X a V P b)))),
                 @DomainTheory.NeighbourhoodSystems.ComputableMap (Sum a a) (Set
a)
                   (@DomainTheory.NeighbourhoodSystems.prod a a V V)
                   (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                   (@DomainTheory.NeighbourhoodSystems.prodPresentation a a V V P
 P)
                   (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDPresentation a V P cons hconsp hcons)
                   (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
finJoinMap_prod a V)))
              (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a) {X Y : Set a},
                @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X
->
                  @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V Y
 ->
                    forall  {W : Set (Set a)},
                      Iff
                        (@DomainTheory.NeighbourhoodSystems.ApproximableMap_2.rel
a a (Set a) V V
                          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDfinJoinApproxMap_2 a V) X Y W)
                        (And
                          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel
a (Set a) V
                            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
.PowerDomain a V)
                            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
.PDsingletonApproxMap a V) X W)
                          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel
a (Set a) V
                            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
.PowerDomain a V)
```

```
                              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
.PDsingletonApproxMap a V) Y W))))
            (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
              (x y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
 a V) {W : Set (Set a)},
              Iff
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem
 (Set a)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDfinJoin
 a V
                    (@OfNat.ofNat Nat (nat_lit 2) (instOfNatNat (nat_lit 2)))
                    (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                      (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))) x
                      (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                        (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0)))
y
                        (@Matrix.vecEmpty (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)))))
                  W)
                (And
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.
mem (Set a)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDsingleton a V x) W)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.
mem (Set a)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDsingleton a V y) W)))
            (@And.intro
              (And
                (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
                  (x y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a V),
                  @Eq
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
 (Set a)
                      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V))
                    (@DomainTheory.NeighbourhoodSystems.ApproximableMap_2.
toElementMap_2 a a (Set a) V V
                      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDfinJoinApproxMap_2 a V) x y)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDfinJoin a V
```

```
                      (@OfNat.ofNat Nat (nat_lit 2) (instOfNatNat (nat_lit 2)))
                      (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                        (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))
x
                        (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                          (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0))
) y
                          (@Matrix.vecEmpty (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V))))))
                (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
                 (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a
 V) (cons : Nat  ->  Nat)
                 (hconsp : Nat.Primrec cons)
                 (hcons :
                   forall  (a b : Nat),
                     Iff (@Eq Nat (cons (Nat.pair a b)) (@OfNat.ofNat Nat (
nat_lit 1) (instOfNatNat (nat_lit 1))))
                       (exists  (k : Nat),
                         @LE.le (Set a) (@Set.instLE a)
                           (@DomainTheory.NeighbourhoodSystems.
ComputablePresentation.X a V P k)
                           (@Inter.inter (Set a) (@Set.instInter a)
                             (@DomainTheory.NeighbourhoodSystems.
ComputablePresentation.X a V P a)
                             (@DomainTheory.NeighbourhoodSystems.
ComputablePresentation.X a V P b)))),
                 @DomainTheory.NeighbourhoodSystems.ComputableMap (Sum a a) (Set
a)
                   (@DomainTheory.NeighbourhoodSystems.prod a a V V)
                   (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                   (@DomainTheory.NeighbourhoodSystems.prodPresentation a a V V P
 P)
                   (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDPresentation a V P cons hconsp hcons)
                   (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
finJoinMap_prod a V)))
               (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a) {X Y : Set a},
                 @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X
->
                   @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V Y
 ->
                     forall  {W : Set (Set a)},
                       Iff
                         (@DomainTheory.NeighbourhoodSystems.ApproximableMap_2.rel
a a (Set a) V V
                           (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                           (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDfinJoinApproxMap_2 a V) X Y W)
                           (And
                             (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel
```

```
a (Set a) V
                              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
.PowerDomain a V)
                              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
.PDsingletonApproxMap a V) X W)
                            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel
a (Set a) V
                              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
.PowerDomain a V)
                              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
.PDsingletonApproxMap a V) Y W)))
               (@And.intro
                 (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
                   (x y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
Element a V),
                   @Eq
                     (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
 (Set a)
                       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V))
                     (@DomainTheory.NeighbourhoodSystems.ApproximableMap_2.
toElementMap_2 a a (Set a) V V
                       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDfinJoinApproxMap_2 a V) x y)
                     (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDfinJoin a V
                       (@OfNat.ofNat Nat (nat_lit 2) (instOfNatNat (nat_lit 2)))
                       (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                         (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))
x
                         (@Matrix.vecCons (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V)
                           (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0))
) y
                           (@Matrix.vecEmpty (@DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem.Element a V))))))
                 (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a)
                   (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a
 V) (cons : Nat  ->  Nat)
                   (hconsp : Nat.Primrec cons)
                   (hcons :
                     forall  (a b : Nat),
                       Iff (@Eq Nat (cons (Nat.pair a b)) (@OfNat.ofNat Nat (
nat_lit 1) (instOfNatNat (nat_lit 1))))
                         (exists  (k : Nat),
                           @LE.le (Set a) (@Set.instLE a)
                             (@DomainTheory.NeighbourhoodSystems.
ComputablePresentation.X a V P k)
                             (@Inter.inter (Set a) (@Set.instInter a)
                               (@DomainTheory.NeighbourhoodSystems.
ComputablePresentation.X a V P a)
```

```
                                  (@DomainTheory.NeighbourhoodSystems.
    ComputablePresentation.X a V P b)))),
                      @DomainTheory.NeighbourhoodSystems.ComputableMap (Sum a a) (Set
    a)
                        (@DomainTheory.NeighbourhoodSystems.prod a a V V)
                        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    PowerDomain a V)
                        (@DomainTheory.NeighbourhoodSystems.prodPresentation a a V V P
     P)
                        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    PDPresentation a V P cons hconsp hcons)
                        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    finJoinMap_prod a V))
                    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    PDfinJoinApproxMap_2_toElementMap
                    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    PDfinJoinApproxMap_2_isComputable)
                  @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    PDfinJoinApproxMap_2_rel_iff)
                @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    PDfinJoin_inter_two)
              @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDfinJoin_pair)
            @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    PDsingletonApproxMap_rel_Ypd_iff)
          @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    PDsingletonApproxMap_toElementMap)
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDsingleton_mono)
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDsingleton_principal)
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.singleton_isComputable)
  DomainTheory.NeighbourhoodSystems.Counterexample712C.
    vshape_not_trianglelefteq_powerDomain
end

section
open NeighborhoodSystem Domain.Recursive ApproximableMap
variable {a b ? : Type*} {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}

/-- The intersection of the sets in `M`, taken inside the base set `base` (so the empty
     list gives
`base`, matching the convention 1.1a where the empty intersection is `Delta`). -/
def interList (base : Set b) : List (Set b)  ->  Set b
  | [] => base
  | Y :: M => Y  I  interList base M

@[simp] theorem interList_nil (base : Set b) : interList base [] = base := rfl

end

section
open NeighborhoodSystem Domain.Recursive ApproximableMap
variable {a b ? : Type*} {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
variable (P_0 : ComputablePresentation V_0) (P_1 : ComputablePresentation V_1)

/-- The step pair coded by an entry `e`: `(X_{e.unpair.1}, Y_{e.unpair.2})`. -/
def funPair (e : Nat) : Set a  x  Set b := (P_0.X e.unpair.1, P_1.X e.unpair.2)
```

```
@[simp] theorem funPair_fst (e : Nat) : (funPair P_0 P_1 e).1 = P_0.X e.unpair.1 :=
    rfl

@[simp] theorem funPair_snd (e : Nat) : (funPair P_0 P_1 e).2 = P_1.X e.unpair.2 :=
    rfl

/-- The sublist of `L` selected by the bitmask `b` (low bit = head). -/
def bitSelect : List Nat  ->  Nat  ->  List Nat
  | [], _ => []
  | e :: L, b => if b % 2 = 1 then e :: bitSelect L (b / 2) else bitSelect L (b / 2)

@[simp] theorem bitSelect_nil (b : Nat) : bitSelect [] b = [] := rfl

/-- A `{0,1}` left-fold `AND` of `isOne  comp  g` over a list, started from `a  in
    {0,1}`. -/
theorem andFoldl_eq_one_iff (g : Nat  ->  Nat) :
    forall  (el : List Nat) (a : Nat), a  <=  1  ->
      (List.foldl (fun acc x => selectFn acc (isOne (g x)) 0) a el = 1  <->
        a = 1  /\  forall  e  in  el, g e = 1)
  | [], a, _ => by simp
  | e :: el, a, ha => by
    rw [List.foldl_cons]
    have hstep : selectFn a (isOne (g e)) 0  <=  1 := by
      rcases (show a = 0  \/  a = 1 by omega) with h | h <;> subst h
      * rw [selectFn_zero]; exact Nat.zero_le 1
      * rw [selectFn_one]; exact isOne_le_one _
    rw [andFoldl_eq_one_iff g el _ hstep]
    rcases (show a = 0  \/  a = 1 by omega) with h | h <;> subst h
    * rw [selectFn_zero]; simp
    * rw [selectFn_one, List.forall_mem_cons, isOne_eq_one_iff]
      constructor
      * rintro <hge, hall>; exact <rfl, hge, hall>
      * rintro <_, hge, hall>; exact <hge, hall>

theorem andFoldl_one (g : Nat  ->  Nat) (el : List Nat) :
    List.foldl (fun acc x => selectFn acc (isOne (g x)) 0) 1 el = 1  <->  forall  e
    in  el, g e = 1 := by
  rw [andFoldl_eq_one_iff g el 1 (le_refl 1)]; simp

/-- Prepend the entry `x` onto the list coded by `acc`. -/
def appendStep (acc x : Nat) : Nat := Nat.pair x acc + 1

/-- `foldCode`-shaped wrapper of `appendStep`. -/
def appendStp (w : Nat) : Nat := appendStep w.unpair.2.unpair.1 w.unpair.1

/-- **`appendCode a b`** codes `(decodeList b).reverse ++ decodeList a`. -/
def appendCode (a b : Nat) : Nat := foldCode appendStp 0 a b

theorem appendStp_eq (x acc : Nat) : appendStp (Nat.pair x (Nat.pair acc 0)) =
    appendStep acc x := by
  unfold appendStp; simp only [unpair_pair_fst, unpair_pair_snd]

/-- The least map of the empty step-list -- the function-space bottom: it relates
    every `?_0`
neighbourhood only to `Delta_1`. -/
def botMap : ApproximableMap V_0 V_1 :=
```

```
  leastMap [] (by simp) (by intro X _; rw [interYs_nil]; exact V_1.master_mem)

theorem botMap_rel {X : Set a} {Y : Set b} :
    (botMap : ApproximableMap V_0 V_1).rel X Y  <->  V_0.mem X  /\  V_1.mem Y  /\  V_1
    .master  subseteq  Y := Iff.rfl

/-- The running set of the conditional `inter`-chain: `A` intersected with the outputs
    `X_1_{e.2}` of
the entries `e  in  el` whose input dominates `X_0_k`. -/
def condSet (k : Nat) (A : Set b) (el : List Nat) : Set b :=
  {z | z  in  A  /\  forall  e  in  el, P_0.X k  subseteq  P_0.X e.unpair.1  ->  z  in
      P_1.X e.unpair.2}

theorem condSet_nil (k : Nat) (A : Set b) : condSet P_0 P_1 k A [] = A := by
  ext z; simp [condSet]

theorem condSet_subset (k : Nat) (A : Set b) (el : List Nat) : condSet P_0 P_1 k A el
     subseteq  A :=
  fun _ hz => hz.1

/-- The `{0,1}` characteristic function of Scott's consistency relation (ii) for the
    function space:
`exists  k, Xenum k  subseteq  Xenum n  I  Xenum m`. -/
def consPairChar (gN : Nat  ->  Nat) (n m : Nat) : Nat :=
  selectFn (isOne (gN n)) (selectFn (isOne (gN m)) (gN (appendCode n m)) 1) 1

section ConsFold

variable {V : NeighborhoodSystem a} (P : ComputablePresentation V)

/-- One fold step (see the section doc). `projFn` extracts the relevant component index
     from an
entry; `fc` is `P.cons_computable`'s `{0,1}` consistency tester. -/
def consUpd (projFn fc : Nat  ->  Nat) (s x : Nat) : Nat :=
  selectFn (s.unpair.1 % 2)
    (Nat.pair (s.unpair.1 / 2)
      (Nat.pair (P.inter s.unpair.2.unpair.1 (projFn x))
        (selectFn s.unpair.2.unpair.2 (isOne (fc (Nat.pair s.unpair.2.unpair.1 (projFn
     x)))) 0)))
    (Nat.pair (s.unpair.1 / 2) (Nat.pair s.unpair.2.unpair.1 s.unpair.2.unpair.2))

/-- `foldCode`-shaped wrapper of `consUpd` (state lives in `w.unpair.2.unpair.1`, entry
     in
`w.unpair.1`). -/
def consStp (projFn fc : Nat  ->  Nat) (w : Nat) : Nat :=
  consUpd P projFn fc w.unpair.2.unpair.1 w.unpair.1

/-- The `{0,1}` characteristic function of the bitmask-selected-sublist consistency
    test, packaged
through `foldCode` so it is primitive recursive. Input `w = pair b c` (bitmask `b`,
    list code `c`). -/
def consCharAt (projFn fc : Nat  ->  Nat) (w : Nat) : Nat :=
  (foldCode (consStp P projFn fc) 0 (Nat.pair w.unpair.1 (Nat.pair P.masterIdx 1))
    w.unpair.2).unpair.2.unpair.2

end ConsFold
```

```
theorem decodeList_appendStep (acc x : Nat) :
    decodeList (appendStep acc x) = x :: decodeList acc := by
  unfold appendStep; rw [decodeList_succ, unpair_pair_fst, unpair_pair_snd]

/-- `decodeList` inverts `encodeList`: every list is the decoding of its code. -/
theorem decodeList_encodeList (l : List Nat) : decodeList (encodeList l) = l := by
  induction l with
  | nil => rw [show encodeList ([] : List Nat) = 0 from rfl, decodeList_zero]
  | cons a l ih =>
    rw [show encodeList (a :: l) = Nat.pair a (encodeList l) + 1 from rfl,
      decodeList_succ, unpair_pair_fst, unpair_pair_snd, ih]

theorem decodeList_foldl_appendStp (el : List Nat) (acc : Nat) :
    decodeList (List.foldl (fun acc x => appendStp (Nat.pair x (Nat.pair acc 0))) acc
    el)
      = el.reverse ++ decodeList acc := by
  induction el generalizing acc with
  | nil => simp
  | cons e el ih =>
    rw [List.foldl_cons, ih, appendStp_eq, decodeList_appendStep, List.reverse_cons,
      List.append_assoc, List.singleton_append]

theorem decodeList_appendCode (a b : Nat) :
    decodeList (appendCode a b) = (decodeList b).reverse ++ decodeList a := by
  unfold appendCode; rw [foldCode_eq']; exact decodeList_foldl_appendStp (decodeList b
    ) a

/-- The per-bitmask implication char: `cons_0(selected inputs)  ->  cons_1(selected
    outputs)`. -/
def funImpChar (fc0 fc1 : Nat  ->  Nat) (w : Nat) : Nat :=
  selectFn (isOne (consCharAt P_0 (fun e => e.unpair.1) fc0 w))
    (isOne (consCharAt P_1 (fun e => e.unpair.2) fc1 w)) 1

/-- The concrete `{0,1}` characteristic function of function-space consistency. -/
def funConsChar (fc0 fc1 : Nat  ->  Nat) (c : Nat) : Nat :=
  bForallFn (funImpChar P_0 P_1 fc0 fc1) c (2 ^ c)

/-- The list of step pairs coded by an entry-list `el`. -/
def funListOf (el : List Nat) : List (Set a  x  Set b) := el.map (funPair P_0 P_1)

/-- The function-space enumeration: a code `c` lists step-pairs; if they are consistent
     (`gN c = 1`,
where `gN` is the consistency char) the neighbourhood is `stepFun (funListOf (
    decodeList c))`,
otherwise we send the junk code to the master neighbourhood `univ`. Choice-free as *
    data* because the
branch is a `Nat`-equality `if`. -/
def Xenum (gN : Nat  ->  Nat) (c : Nat) : Set (ApproximableMap V_0 V_1) :=
  if gN c = 1 then stepFun (funListOf P_0 P_1 (decodeList c)) else Set.univ

theorem Xenum_neg {gN : Nat  ->  Nat} {c : Nat} (h : gN c  !=  1) : Xenum P_0 P_1 gN c
     = Set.univ := ite_eq_right h

theorem Xenum_pos {gN : Nat  ->  Nat} {c : Nat} (h : gN c = 1) :
    Xenum P_0 P_1 gN c = stepFun (funListOf P_0 P_1 (decodeList c)) := ite_eq_left h
```

```
theorem funListOf_cons (e : Nat) (el : List Nat) :
    funListOf P_0 P_1 (e :: el) = funPair P_0 P_1 e :: funListOf P_0 P_1 el := rfl

/-- Every valid finite step-list is `funListOf` of some entry-list (choice-free: `
    Exists.elim` of
`P_0.surj`/`P_1.surj`, entry by entry). -/
theorem exists_funListOf {L : List (Set a  x  Set b)} (hL : forall  p  in  L, V_0.mem
    p.1  /\  V_1.mem p.2) :
    exists  el, funListOf P_0 P_1 el = L := by
  induction L with
  | nil => exact <[], rfl>
  | cons p L ih =>
    obtain <X, Y> := p
    obtain <i, hi> := P_0.surj (hL (X, Y) (List.mem_cons.mpr (Or.inl rfl))).1
    obtain <j, hj> := P_1.surj (hL (X, Y) (List.mem_cons.mpr (Or.inl rfl))).2
    obtain <el, hel> := ih (fun q hq => hL q (List.mem_cons.mpr (Or.inr hq)))
    refine <Nat.pair i j :: el, ?_>
    rw [funListOf_cons, hel]
    congr 1
    show (P_0.X (Nat.pair i j).unpair.1, P_1.X (Nat.pair i j).unpair.2) = (X, Y)
    rw [unpair_pair_fst, unpair_pair_snd, hi, hj]

theorem funListOf_valid (el : List Nat) :
    forall  p  in  funListOf P_0 P_1 el, V_0.mem p.1  /\  V_1.mem p.2 := by
  intro p hp
  rw [funListOf, List.mem_map] at hp
  obtain <e, _, rfl> := hp
  exact <P_0.mem_X _, P_1.mem_X _>

theorem Xenum_mem (gN : Nat  ->  Nat)
    (hgN : forall  c, gN c = 1  <->  (stepFun (funListOf P_0 P_1 (decodeList c))
      : Set (ApproximableMap V_0 V_1)).Nonempty) (c : Nat) :
    (funSpace V_0 V_1).mem (Xenum P_0 P_1 gN c) := by
  by_cases h : gN c = 1
  * rw [Xenum_pos P_0 P_1 h]
    exact <<funListOf P_0 P_1 (decodeList c), funListOf_valid P_0 P_1 _, rfl>, (hgN c)
    .mp h>
  * rw [Xenum_neg P_0 P_1 h]; exact funSpace_master  |>  (funSpace V_0 V_1).master_mem

theorem Xenum_nonempty (gN : Nat  ->  Nat)
    (hgN : forall  c, gN c = 1  <->  (stepFun (funListOf P_0 P_1 (decodeList c))
      : Set (ApproximableMap V_0 V_1)).Nonempty) (c : Nat) :
    (Xenum P_0 P_1 gN c).Nonempty := (Xenum_mem P_0 P_1 gN hgN c).2

section ConsChain

variable {V : NeighborhoodSystem a} (P : ComputablePresentation V)

/-- The `inter`-chain index of `js`: `inter`-fold from `masterIdx`. -/
def idxchain (js : List Nat) : Nat := js.foldl (fun acc j => P.inter acc j) P.
    masterIdx

/-- The running intersection of `A` with the neighbourhoods `X_j` (`j  in  js`), left-
    accumulated to
match `List.foldl`. -/
```

```
def interFrom (A : Set a) : List Nat  ->  Set a
  | [] => A
  | j :: js => interFrom (A  I  P.X j) js

end ConsChain

/-- The canonical code of the intersection `Xenum n  I  Xenum m`: `appendCode` when
    both are
consistent, otherwise the consistent side (the other being `univ`). -/
def interIdx (gN : Nat  ->  Nat) (n m : Nat) : Nat :=
  selectFn (isOne (gN n)) (selectFn (isOne (gN m)) (appendCode n m) n) m

/-- One step of the `interYs`-index fold. State `pair entry (pair acc k)`: entry `e = w
    .unpair.1`,
running `?_1`-index `acc = w.unpair.2.unpair.1`, input index `k = w.unpair.2.unpair.2`
    (the
`foldCode` parameter). Intersects `acc` with `e`'s output index when the inclusion test
     fires. -/
def interYsStp (incl0 : Nat  ->  Nat) (w : Nat) : Nat :=
  selectFn (isOne (incl0 (Nat.pair w.unpair.2.unpair.2 w.unpair.1.unpair.1)))
    (P_1.inter w.unpair.2.unpair.1 w.unpair.1.unpair.2) w.unpair.2.unpair.1

/-- The index of `interYs Delta_1 (funListOf (decodeList a)) (X_0_k)`: a conditional `
    inter`-fold over
`decodeList a` from `masterIdx`, with `k` carried as the `foldCode` parameter. -/
def interYsIdx (incl0 : Nat  ->  Nat) (a k : Nat) : Nat :=
  foldCode (interYsStp P_1 incl0) k P_1.masterIdx a

theorem mem_condSet {k : Nat} {A : Set b} {el : List Nat} {z : b} :
    z  in  condSet P_0 P_1 k A el  <->  z  in  A  /\  forall  e  in  el, P_0.X k
    subseteq  P_0.X e.unpair.1  ->  z  in  P_1.X e.unpair.2 :=
  Iff.rfl

/-- `condSet` cons-step when the input does not dominate the head entry's input: drop
    the head. -/
theorem condSet_cons_neg {k : Nat} {A : Set b} {e : Nat} {el : List Nat}
    (h : not  P_0.X k  subseteq  P_0.X e.unpair.1) :
    condSet P_0 P_1 k A (e :: el) = condSet P_0 P_1 k A el := by
  ext z
  simp only [mem_condSet, List.mem_cons]
  constructor
  * rintro <hzA, hall>
    exact <hzA, fun e' he' hsub => hall e' (Or.inr he') hsub>
  * rintro <hzA, hall>
    exact <hzA, fun e' he' hsub => he'.elim (fun heq => absurd (heq  |>  hsub) h) (fun
     hmem => hall e' hmem hsub)>

/-- `condSet` cons-step when the input dominates the head entry's input: intersect `A`
    with the
head's output. -/
theorem condSet_cons_pos {k : Nat} {A : Set b} {e : Nat} {el : List Nat}
    (h : P_0.X k  subseteq  P_0.X e.unpair.1) :
    condSet P_0 P_1 k A (e :: el) = condSet P_0 P_1 k (A  I  P_1.X e.unpair.2) el :=
    by
  ext z
  simp only [mem_condSet, List.mem_cons, Set.mem_inter_iff]
```

```
  constructor
  * rintro <hzA, hall>
    exact <<hzA, hall e (Or.inl rfl) h>, fun e' he' hsub => hall e' (Or.inr he') hsub>
  * rintro <<hzA, hzE>, hall>
    exact <hzA, fun e' he' hsub => he'.elim (fun heq => heq  |>  hzE) (fun hmem =>
    hall e' hmem hsub)>

/-- Correctness of the conditional `inter`-fold (`if`-form). When the running set is
    consistent, the
fold computes an index of it, by induction with `inter_spec` (mirrors `
    interFrom_eq_of_foldl`). -/
theorem condFoldl_spec (incl0 : Nat  ->  Nat)
    (hincl0 : forall  s, incl0 s = 1  <->  P_0.X s.unpair.1  subseteq  P_0.X s.unpair
    .2) (k : Nat) :
    forall  (el : List Nat) (A : Set b) (a : Nat), P_1.X a = A  ->  V_1.mem A  ->
      V_1.mem (condSet P_0 P_1 k A el)  ->
      P_1.X (List.foldl (fun acc x => if incl0 (Nat.pair k x.unpair.1) = 1
              then P_1.inter acc x.unpair.2 else acc) a el) = condSet P_0 P_1 k A el
  | [], A, a, hXa, _, _ => by rw [List.foldl_nil, condSet_nil]; exact hXa
  | e :: el, A, a, hXa, hA, hcons => by
    rw [List.foldl_cons]
    by_cases hpred : incl0 (Nat.pair k e.unpair.1) = 1


    * rw [ite_eq_left hpred]
      have hincl : P_0.X k  subseteq  P_0.X e.unpair.1 := by
        have h := (hincl0 (Nat.pair k e.unpair.1)).mp hpred
        rwa [unpair_pair_fst, unpair_pair_snd] at h
      rw [condSet_cons_pos P_0 P_1 hincl] at hcons  |-
      have hwit : V_1.mem (A  I  P_1.X e.unpair.2) :=
        V_1.inter_mem hA (P_1.mem_X _) hcons (condSet_subset P_0 P_1 k (A  I  P_1.X e.
    unpair.2) el)
      have hk : exists  j, P_1.X j  subseteq  P_1.X a  I  P_1.X e.unpair.2 := by
        obtain <j, hj> := P_1.surj hwit; exact <j, by rw [hj, hXa]>
      have hXinter : P_1.X (P_1.inter a e.unpair.2) = A  I  P_1.X e.unpair.2 := by
        rw [P_1.inter_spec hk, hXa]
      exact condFoldl_spec incl0 hincl0 k el (A  I  P_1.X e.unpair.2) (P_1.inter a e.
    unpair.2)
        hXinter hwit hcons
    * rw [ite_eq_right hpred]
      have hincl : not  P_0.X k  subseteq  P_0.X e.unpair.1 := fun hc =>
        hpred ((hincl0 (Nat.pair k e.unpair.1)).mpr (by rwa [unpair_pair_fst,
    unpair_pair_snd]))
      rw [condSet_cons_neg P_0 P_1 hincl] at hcons  |-
      exact condFoldl_spec incl0 hincl0 k el A a hXa hA hcons

/-- `condSet k Delta_1 el` is exactly `interYs Delta_1 (funListOf el) (X_0_k)`. -/
theorem condSet_eq_interYs (k : Nat) (el : List Nat) :
    condSet P_0 P_1 k V_1.master el = interYs V_1.master (funListOf P_0 P_1 el) (P_0.X
     k) := by
  ext z
  rw [mem_interYs, mem_condSet]
  constructor
  * rintro <hz, hall>
    refine <hz, ?_>
    intro p hp hsub
    rw [funListOf, List.mem_map] at hp
```

```
    obtain <e, he, rfl> := hp
    exact hall e he hsub
  * rintro <hz, hall>
    exact <hz, fun e he hsub => hall (funPair P_0 P_1 e) (List.mem_map.mpr <e, he, rfl
    >) hsub>

theorem primrec_appendCode : Nat.Primrec (fun t => appendCode t.unpair.1 t.unpair.2)
    := by
  have hstp : Nat.Primrec appendStp :=
    (primrec_add_2 (Nat.Primrec.left.pair (Nat.Primrec.left.comp Nat.Primrec.right))
      (Nat.Primrec.const 1)).of_eq (fun w => rfl)
  exact (primrec_foldCode hstp (Nat.Primrec.const 0) Nat.Primrec.left Nat.Primrec.
    right).of_eq
    (fun t => rfl)

theorem primrec_consPairChar (gN : Nat  ->  Nat) (hgNp : Nat.Primrec gN) :
    Nat.Primrec (fun t => consPairChar gN t.unpair.1 t.unpair.2) := by
  have hn := hgNp.comp Nat.Primrec.left
  have hm := hgNp.comp Nat.Primrec.right
  have happ := hgNp.comp primrec_appendCode
  have hinner := primrec_selectFn (primrec_isOne.comp hm) happ (Nat.Primrec.const 1)
  exact (primrec_selectFn (primrec_isOne.comp hn) hinner (Nat.Primrec.const 1)).of_eq
    (fun _ => rfl)

theorem primrec_interIdx (gN : Nat  ->  Nat) (hgNp : Nat.Primrec gN) :
    Nat.Primrec (fun t => interIdx gN t.unpair.1 t.unpair.2) := by
  have hn := hgNp.comp Nat.Primrec.left
  have hm := hgNp.comp Nat.Primrec.right
  have hinner := primrec_selectFn (primrec_isOne.comp hm) primrec_appendCode Nat.
    Primrec.left
  exact (primrec_selectFn (primrec_isOne.comp hn) hinner Nat.Primrec.right).of_eq (fun
     _ => rfl)

theorem primrec_interYsStp (incl0 : Nat  ->  Nat) (hincl0p : Nat.Primrec incl0) :
    Nat.Primrec (interYsStp P_1 incl0) := by
  have he1 : Nat.Primrec (fun w : Nat => w.unpair.1.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.left
  have he2 : Nat.Primrec (fun w : Nat => w.unpair.1.unpair.2) :=
    Nat.Primrec.right.comp Nat.Primrec.left
  have hacc : Nat.Primrec (fun w : Nat => w.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.right
  have hk : Nat.Primrec (fun w : Nat => w.unpair.2.unpair.2) :=
    Nat.Primrec.right.comp Nat.Primrec.right
  have hcond := primrec_isOne.comp (hincl0p.comp (hk.pair he1))
  have hinter := P_1.inter_primrec.comp (hacc.pair he2)
  refine (primrec_selectFn hcond hinter hacc).of_eq (fun w => ?_)
  unfold interYsStp
  simp only [unpair_pair_fst, unpair_pair_snd]

theorem primrec_interYsIdx (incl0 : Nat  ->  Nat) (hincl0p : Nat.Primrec incl0) :
    Nat.Primrec (fun t => interYsIdx P_1 incl0 t.unpair.1 t.unpair.2) :=
  (primrec_foldCode (primrec_interYsStp P_1 incl0 hincl0p) Nat.Primrec.right
    (Nat.Primrec.const P_1.masterIdx) Nat.Primrec.left).of_eq (fun _ => rfl)

/-- **Choice-free `rel_interYs` for a presented list.** Any `f  in  stepFun (funListOf
    el)` relates each
```

```
`X_0_{n'}` to the intersection of the relevant outputs `interYs Delta_1 (funListOf el)
    (X_0_{n'})`. The
case split on `X_0_{n'}  subseteq  X_0_{e.1}` is discharged by `P_0.incl_computable.em`
     (no `Classical.choice`). -/
theorem rel_interYs_funList {f : ApproximableMap V_0 V_1} {el : List Nat}
    (hf : f  in  stepFun (funListOf P_0 P_1 el)) (n' : Nat) :
    f.rel (P_0.X n') (interYs V_1.master (funListOf P_0 P_1 el) (P_0.X n')) := by
  induction el with
  | nil =>
    rw [funListOf, List.map_nil, interYs_nil]
    exact f.mono f.master_rel (V_0.sub_master (P_0.mem_X n')) subset_rfl (P_0.mem_X n
    ') V_1.master_mem
  | cons e el ih =>
    rw [funListOf_cons] at hf
    have hftail : f  in  stepFun (funListOf P_0 P_1 el) :=
      fun p hp => hf p (List.mem_cons_of_mem _ hp)
    have htail := ih hftail
    rw [funListOf_cons]
    rcases P_0.incl_computable.em (Nat.pair n' e.unpair.1) with hinc | hinc <;>
      simp only [unpair_pair_fst, unpair_pair_snd] at hinc
    * have hp : f.rel (P_0.X e.unpair.1) (P_1.X e.unpair.2) :=
        hf (funPair P_0 P_1 e) (List.mem_cons.mpr (Or.inl rfl))
      have hXp2 : f.rel (P_0.X n') (P_1.X e.unpair.2) :=
        f.mono hp hinc subset_rfl (P_0.mem_X n') (f.rel_cod hp)
      have heq : interYs V_1.master (funPair P_0 P_1 e :: funListOf P_0 P_1 el) (P_0.X
     n')
          = P_1.X e.unpair.2  I  interYs V_1.master (funListOf P_0 P_1 el) (P_0.X n')
    := by
        rw [interYs_cons]; ext z
        simp only [Set.mem_inter_iff, Set.mem_ofPred_eq, funPair]
        exact <fun <h1, h2> => <h1 hinc, h2>, fun <h1, h2> => <fun _ => h1, h2>>
      rw [heq]; exact f.inter_right hXp2 htail
    * have heq : interYs V_1.master (funPair P_0 P_1 e :: funListOf P_0 P_1 el) (P_0.X
     n')
          = interYs V_1.master (funListOf P_0 P_1 el) (P_0.X n') := by
        rw [interYs_cons]; ext z
        simp only [Set.mem_inter_iff, Set.mem_ofPred_eq, funPair]
        exact <fun h => h.2, fun h => <fun hc => absurd hc hinc, h>>
      rw [heq]; exact htail

/-- If `stepFun (funListOf ea)` is non-empty, then `interYs Delta_1 (funListOf ea) X`
    is a neighbourhood of
`?_1` for every `X  in  ?_0` (the consistency hypothesis needed by `leastMap`). Choice-
    free. -/
theorem interYs_funList_mem_of_nonempty {ea : List Nat}
    (h : (stepFun (funListOf P_0 P_1 ea) : Set (ApproximableMap V_0 V_1)).Nonempty)
    {X : Set a} (hX : V_0.mem X) :
    V_1.mem (interYs V_1.master (funListOf P_0 P_1 ea) X) := by
  obtain <n', hn'> := P_0.surj hX
  obtain <f, hf> := h
  have hr := rel_interYs_funList P_0 P_1 hf n'
  rw [hn'] at hr
  exact f.rel_cod hr

/-- `selectFn` driven by `isOne` is a `Nat`-equality `if` (choice-free; `decEq` on `Nat
    `). -/
```

```
theorem selectFn_isOne (v a b : Nat) : selectFn (isOne v) a b = if v = 1 then a else b
     := by
  by_cases h : v = 1
  * rw [ite_eq_left h, h, (isOne_eq_one_iff 1).mpr rfl, selectFn_one]
  * rw [ite_eq_right h]
    have h0 : isOne v = 0 := by
      rcases (show isOne v = 0  \/  isOne v = 1 from by have := isOne_le_one v; omega)
     with h0 | h1
      * exact h0
      * exact absurd ((isOne_eq_one_iff v).mp h1) h
    rw [h0, selectFn_zero]

/-- **`interYsIdx` indexes `interYs Delta_1 (funListOf (decodeList a)) (X_0_k)`** (when
     that is consistent,
which it is whenever `stepFun (funListOf (decodeList a))` is non-empty). Choice-free.
    -/
theorem interYsIdx_spec (incl0 : Nat  ->  Nat)
    (hincl0 : forall  s, incl0 s = 1  <->  P_0.X s.unpair.1  subseteq  P_0.X s.unpair
    .2) (a k : Nat)
    (hcons : V_1.mem (interYs V_1.master (funListOf P_0 P_1 (decodeList a)) (P_0.X k))
    ) :
    P_1.X (interYsIdx P_1 incl0 a k)
      = interYs V_1.master (funListOf P_0 P_1 (decodeList a)) (P_0.X k) := by
  unfold interYsIdx
  rw [foldCode_eq']
  have hstep : (fun acc x => interYsStp P_1 incl0 (Nat.pair x (Nat.pair acc k)))
      = (fun acc x => if incl0 (Nat.pair k x.unpair.1) = 1
            then P_1.inter acc x.unpair.2 else acc) := by
    funext acc x
    show interYsStp P_1 incl0 (Nat.pair x (Nat.pair acc k)) = _
    unfold interYsStp
    simp only [unpair_pair_fst, unpair_pair_snd]
    exact selectFn_isOne _ _ _
  rw [hstep]
  rw [ <-  condSet_eq_interYs P_0 P_1 k (decodeList a)] at hcons  |-
  exact condFoldl_spec P_0 P_1 incl0 hincl0 k (decodeList a) V_1.master P_1.masterIdx
    P_1.masterIdx_spec V_1.master_mem hcons

/-- `selectFn c d 0 = 1` decides the conjunction `c = 1  /\  d = 1`. -/
theorem selectFn_mul_iff (c d : Nat) : selectFn c d 0 = 1  <->  c = 1  /\  d = 1 := by
  have h : selectFn c d 0 = c * d := by simp only [selectFn, Nat.mul_zero, Nat.
    add_zero]
  rw [h, nat_mul_eq_one]

/-- `stepFun (funListOf (decodeList (appendCode a b)))` is exactly `X a  I  X b` at the
     level of the
underlying step-intersections. -/
theorem stepFun_funListOf_appendCode (a b : Nat) :
    (stepFun (funListOf P_0 P_1 (decodeList (appendCode a b))) : Set (ApproximableMap
    V_0 V_1)) =
      stepFun (funListOf P_0 P_1 (decodeList a))  I  stepFun (funListOf P_0 P_1 (
    decodeList b)) := by
  ext f
  rw [decodeList_appendCode]
  simp only [funListOf, List.map_append, List.map_reverse, mem_stepFun, Set.
    mem_inter_iff,
```

```
    List.mem_append, List.mem_reverse]
  constructor
  * intro h; exact <fun p hp => h p (Or.inr hp), fun p hp => h p (Or.inl hp)>
  * rintro <ha, hb> p (hp | hp)
    * exact hb p hp
    * exact ha p hp

/-- **`interIdx` codes the intersection** whenever it is non-empty (and the result is
    in the
enumeration). -/
theorem Xenum_inter_eq (gN : Nat  ->  Nat)
    (hgN : forall  c, gN c = 1  <->  (stepFun (funListOf P_0 P_1 (decodeList c))
      : Set (ApproximableMap V_0 V_1)).Nonempty) (n m : Nat)
    (hne : (Xenum P_0 P_1 gN n  I  Xenum P_0 P_1 gN m).Nonempty) :
    Xenum P_0 P_1 gN (interIdx gN n m) = Xenum P_0 P_1 gN n  I  Xenum P_0 P_1 gN m :=
    by
  unfold interIdx
  rw [selectFn_isOne]
  by_cases hn : gN n = 1
  * rw [ite_eq_left hn, selectFn_isOne]
    by_cases hm : gN m = 1
    * rw [ite_eq_left hm, Xenum_pos P_0 P_1 hn, Xenum_pos P_0 P_1 hm,  <-
    stepFun_funListOf_appendCode P_0 P_1 n m]
      rw [Xenum_pos P_0 P_1 hn, Xenum_pos P_0 P_1 hm] at hne
      have hgne : gN (appendCode n m) = 1 :=
        (hgN _).mpr (by rw [stepFun_funListOf_appendCode P_0 P_1 n m]; exact hne)
      rw [Xenum_pos P_0 P_1 hgne]
    * rw [ite_eq_right hm, Xenum_neg P_0 P_1 hm, Set.inter_univ]
  * rw [ite_eq_right hn, Xenum_neg P_0 P_1 hn, Set.univ_inter]

theorem consPairChar_spec (gN : Nat  ->  Nat)
    (hgN : forall  c, gN c = 1  <->  (stepFun (funListOf P_0 P_1 (decodeList c))
      : Set (ApproximableMap V_0 V_1)).Nonempty) (n m : Nat) :
    consPairChar gN n m = 1  <->
      exists  k, Xenum P_0 P_1 gN k  subseteq  Xenum P_0 P_1 gN n  I  Xenum P_0 P_1 gN
     m := by
  have hcons_iff : (exists  k, Xenum P_0 P_1 gN k  subseteq  Xenum P_0 P_1 gN n  I
    Xenum P_0 P_1 gN m)  <->
      (Xenum P_0 P_1 gN n  I  Xenum P_0 P_1 gN m).Nonempty := by
    constructor
    * rintro <k, hk>; exact (Xenum_nonempty P_0 P_1 gN hgN k).mono hk
    * intro hne
      exact <interIdx gN n m, (Xenum_inter_eq P_0 P_1 gN hgN n m hne).subset>
  rw [hcons_iff]
  unfold consPairChar
  by_cases hn : gN n = 1
  * rw [selectFn_isOne, ite_eq_left hn, selectFn_isOne]
    by_cases hm : gN m = 1
    * rw [ite_eq_left hm, hgN (appendCode n m), Xenum_pos P_0 P_1 hn, Xenum_pos P_0
    P_1 hm,
        <-  stepFun_funListOf_appendCode P_0 P_1 n m]
    * rw [ite_eq_right hm, Xenum_pos P_0 P_1 hn, Xenum_neg P_0 P_1 hm, Set.inter_univ]
      exact iff_of_true rfl ((hgN n).mp hn)
  * rw [selectFn_isOne, ite_eq_right hn, Xenum_neg P_0 P_1 hn, Set.univ_inter]
    exact iff_of_true rfl (Xenum_nonempty P_0 P_1 gN hgN m)
```

```
/-- **`stepFun (funListOf el)` is everything iff every output neighbourhood is `Delta_1
    `.** Forward:
the bottom map `botMap` lies in `univ`, and `botMap.rel X Y` forces `Y = Delta_1`.
    Backward: `[X, Delta_1]`
is satisfied by every map. Choice-free. -/
theorem stepFun_funListOf_eq_univ_iff (el : List Nat) :
    (stepFun (funListOf P_0 P_1 el) : Set (ApproximableMap V_0 V_1)) = Set.univ  <->
      forall  e  in  el, P_1.X e.unpair.2 = V_1.master := by
  constructor
  * intro h e he
    have hbot : (botMap : ApproximableMap V_0 V_1)  in  stepFun (funListOf P_0 P_1 el)
     := by
      rw [h]; exact Set.mem_univ _
    have hr := hbot (funPair P_0 P_1 e) (List.mem_map.mpr <e, he, rfl>)
    exact Set.Subset.antisymm (V_1.sub_master (P_1.mem_X _)) (botMap_rel.mp hr).2.2
  * intro h
    ext f
    simp only [Set.mem_univ, iff_true, mem_stepFun]
    intro p hp
    rw [funListOf, List.mem_map] at hp
    obtain <e, he, rfl> := hp
    show f.rel (P_0.X e.unpair.1) (P_1.X e.unpair.2)
    rw [h e he]
    exact f.mono f.master_rel (V_0.sub_master (P_0.mem_X _)) subset_rfl (P_0.mem_X _)
    V_1.master_mem

/-- **Function-space inclusion characterisation (choice-free).** With `ea` consistent (
    so the least
map exists), `stepFun (funListOf ea)  subseteq  stepFun (funListOf eb)` iff for every
    entry `e'  in  eb`, the
intersection of the `ea`-outputs relevant to the input `X_0_{e'.1}` is contained in `
    X_1_{e'.2}`. -/
theorem stepFun_funListOf_subset_iff {ea eb : List Nat}
    (hcons : forall  {X : Set a}, V_0.mem X  ->  V_1.mem (interYs V_1.master (
    funListOf P_0 P_1 ea) X)) :
    (stepFun (funListOf P_0 P_1 ea) : Set (ApproximableMap V_0 V_1))  subseteq
    stepFun (funListOf P_0 P_1 eb)  <->
      forall  e'  in  eb, interYs V_1.master (funListOf P_0 P_1 ea) (P_0.X e'.unpair
    .1)  subseteq  P_1.X e'.unpair.2 := by
  have hL : forall  p  in  funListOf P_0 P_1 ea, V_0.mem p.1  /\  V_1.mem p.2 :=
    funListOf_valid P_0 P_1 ea
  constructor
  * intro hsub e' he'
    have hmem := hsub (leastMap_mem_stepFun hL hcons)
    have hrel := hmem (funPair P_0 P_1 e') (List.mem_map.mpr <e', he', rfl>)
    exact (leastMap_rel.mp hrel).2.2
  * intro hcond f hf p hp
    rw [funListOf, List.mem_map] at hp
    obtain <e', he', rfl> := hp
    show f.rel (P_0.X e'.unpair.1) (P_1.X e'.unpair.2)
    exact f.mono (rel_interYs_funList P_0 P_1 hf e'.unpair.1) subset_rfl (hcond e' he
    ')
      (P_0.mem_X _) (P_1.mem_X _)

/-- One step of the `stepFun`-inclusion fold over `decodeList b`: test `interYs-index
    subseteq  output`. -/
```

```
def subStp (incl0 incl1 : Nat  ->  Nat) (w : Nat) : Nat :=
  selectFn w.unpair.2.unpair.1
    (isOne (incl1 (Nat.pair (interYsIdx P_1 incl0 w.unpair.2.unpair.2 w.unpair.1.
    unpair.1)
      w.unpair.1.unpair.2))) 0

theorem primrec_subStp (incl0 incl1 : Nat  ->  Nat) (hincl0p : Nat.Primrec incl0)
    (hincl1p : Nat.Primrec incl1) : Nat.Primrec (subStp P_1 incl0 incl1) := by
  have hflag : Nat.Primrec (fun w : Nat => w.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.right
  have ha : Nat.Primrec (fun w : Nat => w.unpair.2.unpair.2) :=
    Nat.Primrec.right.comp Nat.Primrec.right
  have hk : Nat.Primrec (fun w : Nat => w.unpair.1.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.left
  have he2 : Nat.Primrec (fun w : Nat => w.unpair.1.unpair.2) :=
    Nat.Primrec.right.comp Nat.Primrec.left
  have hidx : Nat.Primrec
      (fun w : Nat => interYsIdx P_1 incl0 w.unpair.2.unpair.2 w.unpair.1.unpair.1) :=
    ((primrec_interYsIdx P_1 incl0 hincl0p).comp (ha.pair hk)).of_eq (fun w => by
      show interYsIdx P_1 incl0 (Nat.pair w.unpair.2.unpair.2 w.unpair.1.unpair.1).
    unpair.1
        (Nat.pair w.unpair.2.unpair.2 w.unpair.1.unpair.1).unpair.2 = _
      rw [unpair_pair_fst, unpair_pair_snd])
  have hincl := hincl1p.comp (hidx.pair he2)
  refine (primrec_selectFn hflag (primrec_isOne.comp hincl) (Nat.Primrec.const 0)).
    of_eq (fun w => ?_)
  unfold subStp
  rfl

/-- The `{0,1}` characteristic function of `stepFun (funListOf (decodeList a))
    subseteq  stepFun (funListOf
(decodeList b))` (correct when `a` is consistent). `a` is the `foldCode` parameter, `b`
    the code. -/
def subChar (incl0 incl1 : Nat  ->  Nat) (a b : Nat) : Nat := foldCode (subStp P_1
    incl0 incl1) a 1 b

theorem primrec_subChar (incl0 incl1 : Nat  ->  Nat) (hincl0p : Nat.Primrec incl0)
    (hincl1p : Nat.Primrec incl1) :
    Nat.Primrec (fun t => subChar P_1 incl0 incl1 t.unpair.1 t.unpair.2) :=
  (primrec_foldCode (primrec_subStp P_1 incl0 incl1 hincl0p hincl1p) Nat.Primrec.left
    (Nat.Primrec.const 1) Nat.Primrec.right).of_eq (fun _ => rfl)

theorem subChar_spec (incl0 incl1 : Nat  ->  Nat)
    (hincl0 : forall  s, incl0 s = 1  <->  P_0.X s.unpair.1  subseteq  P_0.X s.unpair
    .2)
    (hincl1 : forall  s, incl1 s = 1  <->  P_1.X s.unpair.1  subseteq  P_1.X s.unpair
    .2)
    {a : Nat}
    (hane : (stepFun (funListOf P_0 P_1 (decodeList a)) : Set (ApproximableMap V_0 V_1
    )).Nonempty)
    (b : Nat) :
    subChar P_1 incl0 incl1 a b = 1  <->
      (stepFun (funListOf P_0 P_1 (decodeList a)) : Set (ApproximableMap V_0 V_1))
    subseteq
        stepFun (funListOf P_0 P_1 (decodeList b)) := by
  have hcons : forall  {X : Set a},
```

```
      V_0.mem X  ->  V_1.mem (interYs V_1.master (funListOf P_0 P_1 (decodeList a)) X)
     :=
    fun {X} hX => interYs_funList_mem_of_nonempty P_0 P_1 hane hX
  rw [stepFun_funListOf_subset_iff P_0 P_1 hcons]
  unfold subChar
  rw [foldCode_eq']
  have hstep : (fun acc x => subStp P_1 incl0 incl1 (Nat.pair x (Nat.pair acc a)))
      = (fun acc x => selectFn acc
          (isOne (incl1 (Nat.pair (interYsIdx P_1 incl0 a x.unpair.1) x.unpair.2))) 0)
     := by
    funext acc x
    show subStp P_1 incl0 incl1 (Nat.pair x (Nat.pair acc a)) = _
    unfold subStp
    simp only [unpair_pair_fst, unpair_pair_snd]
  rw [hstep, andFoldl_one (fun x => incl1 (Nat.pair (interYsIdx P_1 incl0 a x.unpair
    .1) x.unpair.2))]
  refine forall_congr' (fun e' => ?_)
  refine imp_congr_right (fun he' => ?_)
  rw [hincl1]
  simp only [unpair_pair_fst, unpair_pair_snd]
  rw [interYsIdx_spec P_0 P_1 incl0 hincl0 a e'.unpair.1 (hcons (P_0.mem_X e'.unpair
    .1))]

/-- One step of the trivial-test fold: is this entry's output index equal to `masterIdx
    `? -/
def trivStp (eq1 : Nat  ->  Nat) (w : Nat) : Nat :=
  selectFn w.unpair.2.unpair.1
    (isOne (eq1 (Nat.pair w.unpair.1.unpair.2 P_1.masterIdx))) 0

theorem primrec_trivStp (eq1 : Nat  ->  Nat) (heq1p : Nat.Primrec eq1) :
    Nat.Primrec (trivStp P_1 eq1) := by
  have hflag : Nat.Primrec (fun w : Nat => w.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.right
  have he2 : Nat.Primrec (fun w : Nat => w.unpair.1.unpair.2) :=
    Nat.Primrec.right.comp Nat.Primrec.left
  have harg := heq1p.comp (he2.pair (Nat.Primrec.const P_1.masterIdx))
  refine (primrec_selectFn hflag (primrec_isOne.comp harg) (Nat.Primrec.const 0)).
    of_eq (fun w => ?_)
  unfold trivStp
  rfl

/-- The `{0,1}` characteristic function of `stepFun (funListOf (decodeList a)) = univ`.
     -/
def trivialChar (eq1 : Nat  ->  Nat) (a : Nat) : Nat := foldCode (trivStp P_1 eq1) 0 1
     a

/-- The `{0,1}` characteristic function of `Xenum a = Xenum b`. Both consistent  ->
    inclusion both
ways (`subChar`); one consistent, one junk (`univ`)  ->  the consistent side is the
    trivial
neighbourhood (`trivialChar`); both junk  ->  equal. -/
def eqEnumChar (gN incl0 incl1 eq1 : Nat  ->  Nat) (a b : Nat) : Nat :=
  selectFn (isOne (gN a))
    (selectFn (isOne (gN b))
      (selectFn (subChar P_1 incl0 incl1 a b) (subChar P_1 incl0 incl1 b a) 0)
      (trivialChar P_1 eq1 a))
```

```
    (selectFn (isOne (gN b)) (trivialChar P_1 eq1 b) 1)

/-- The `{0,1}` characteristic function of Scott's relation (i) for the function space:
`Xenum n  I  Xenum m = Xenum k`. -/
def interEqChar (gN incl0 incl1 eq1 : Nat  ->  Nat) (n m k : Nat) : Nat :=
  selectFn (consPairChar gN n m) (eqEnumChar P_1 gN incl0 incl1 eq1 (interIdx gN n m)
    k) 0

theorem primrec_trivialChar (eq1 : Nat  ->  Nat) (heq1p : Nat.Primrec eq1) :
    Nat.Primrec (trivialChar P_1 eq1) :=
  (primrec_foldCode (primrec_trivStp P_1 eq1 heq1p) (Nat.Primrec.const 0)
    (Nat.Primrec.const 1) primrec_id).of_eq (fun _ => rfl)

theorem primrec_eqEnumChar (gN incl0 incl1 eq1 : Nat  ->  Nat) (hgNp : Nat.Primrec gN)
    (hincl0p : Nat.Primrec incl0) (hincl1p : Nat.Primrec incl1) (heq1p : Nat.Primrec
    eq1) :
    Nat.Primrec (fun t => eqEnumChar P_1 gN incl0 incl1 eq1 t.unpair.1 t.unpair.2) :=
    by
  have ha := primrec_isOne.comp (hgNp.comp Nat.Primrec.left)
  have hb := primrec_isOne.comp (hgNp.comp Nat.Primrec.right)
  have hsubab : Nat.Primrec (fun t => subChar P_1 incl0 incl1 t.unpair.1 t.unpair.2)
    :=
    primrec_subChar P_1 incl0 incl1 hincl0p hincl1p
  have hsubba : Nat.Primrec (fun t => subChar P_1 incl0 incl1 t.unpair.2 t.unpair.1)
    :=
    ((primrec_subChar P_1 incl0 incl1 hincl0p hincl1p).comp
      (Nat.Primrec.right.pair Nat.Primrec.left)).of_eq (fun t => by
        show subChar P_1 incl0 incl1 (Nat.pair t.unpair.2 t.unpair.1).unpair.1
          (Nat.pair t.unpair.2 t.unpair.1).unpair.2 = _
        rw [unpair_pair_fst, unpair_pair_snd])
  have htriva := (primrec_trivialChar P_1 eq1 heq1p).comp Nat.Primrec.left
  have htrivb := (primrec_trivialChar P_1 eq1 heq1p).comp Nat.Primrec.right
  have hbothcons := primrec_selectFn hsubab hsubba (Nat.Primrec.const 0)
  have hacons := primrec_selectFn hb hbothcons htriva
  have hanot := primrec_selectFn hb htrivb (Nat.Primrec.const 1)
  exact (primrec_selectFn ha hacons hanot).of_eq (fun _ => rfl)

theorem primrec_interEqChar (gN incl0 incl1 eq1 : Nat  ->  Nat) (hgNp : Nat.Primrec gN
    )
    (hincl0p : Nat.Primrec incl0) (hincl1p : Nat.Primrec incl1) (heq1p : Nat.Primrec
    eq1) :
    Nat.Primrec (fun t => interEqChar P_1 gN incl0 incl1 eq1 t.unpair.1
      t.unpair.2.unpair.1 t.unpair.2.unpair.2) := by
  have hn : Nat.Primrec (fun t : Nat => t.unpair.1) := Nat.Primrec.left
  have hm : Nat.Primrec (fun t : Nat => t.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.right
  have hk : Nat.Primrec (fun t : Nat => t.unpair.2.unpair.2) :=
    Nat.Primrec.right.comp Nat.Primrec.right
  have hcons := (primrec_consPairChar gN hgNp).comp (hn.pair hm)
  have hidx := (primrec_interIdx gN hgNp).comp (hn.pair hm)
  have heqc := (primrec_eqEnumChar P_1 gN incl0 incl1 eq1 hgNp hincl0p hincl1p heq1p).
    comp
    (hidx.pair hk)

  refine (primrec_selectFn hcons heqc (Nat.Primrec.const 0)).of_eq (fun t => ?_)
  show _ = interEqChar P_1 gN incl0 incl1 eq1 t.unpair.1 t.unpair.2.unpair.1 t.unpair
```

```
    .2.unpair.2
  unfold interEqChar
  simp only [unpair_pair_fst, unpair_pair_snd]

theorem trivialChar_spec (eq1 : Nat  ->  Nat)
    (heq1 : forall  s, eq1 s = 1  <->  P_1.X s.unpair.1 = P_1.X s.unpair.2) (a : Nat)
    :
    trivialChar P_1 eq1 a = 1  <->
      (stepFun (funListOf P_0 P_1 (decodeList a)) : Set (ApproximableMap V_0 V_1)) =
    Set.univ := by
  rw [stepFun_funListOf_eq_univ_iff P_0 P_1]
  unfold trivialChar
  rw [foldCode_eq']
  have hstep : (fun acc x => trivStp P_1 eq1 (Nat.pair x (Nat.pair acc 0)))
      = (fun acc x => selectFn acc (isOne (eq1 (Nat.pair x.unpair.2 P_1.masterIdx)))
    0) := by
    funext acc x
    show trivStp P_1 eq1 (Nat.pair x (Nat.pair acc 0)) = _
    unfold trivStp
    simp only [unpair_pair_fst, unpair_pair_snd]
  rw [hstep, andFoldl_one (fun x => eq1 (Nat.pair x.unpair.2 P_1.masterIdx))]
  refine forall_congr' (fun e => ?_)
  refine imp_congr_right (fun he => ?_)
  rw [heq1]
  simp only [unpair_pair_fst, unpair_pair_snd, P_1.masterIdx_spec]

theorem eqEnumChar_spec (gN incl0 incl1 eq1 : Nat  ->  Nat)
    (hgN : forall  c, gN c = 1  <->  (stepFun (funListOf P_0 P_1 (decodeList c))
      : Set (ApproximableMap V_0 V_1)).Nonempty)
    (hincl0 : forall  s, incl0 s = 1  <->  P_0.X s.unpair.1  subseteq  P_0.X s.unpair
    .2)
    (hincl1 : forall  s, incl1 s = 1  <->  P_1.X s.unpair.1  subseteq  P_1.X s.unpair
    .2)
    (heq1 : forall  s, eq1 s = 1  <->  P_1.X s.unpair.1 = P_1.X s.unpair.2) (a b : Nat
    ) :
    eqEnumChar P_1 gN incl0 incl1 eq1 a b = 1  <->  Xenum P_0 P_1 gN a = Xenum P_0 P_1
     gN b := by
  unfold eqEnumChar
  by_cases ha : gN a = 1
  * rw [selectFn_isOne, ite_eq_left ha, selectFn_isOne]
    have hanea := (hgN a).mp ha
    by_cases hb : gN b = 1
    * rw [ite_eq_left hb, Xenum_pos P_0 P_1 ha, Xenum_pos P_0 P_1 hb, selectFn_mul_iff
    ,
        subChar_spec P_0 P_1 incl0 incl1 hincl0 hincl1 hanea b,
        subChar_spec P_0 P_1 incl0 incl1 hincl0 hincl1 ((hgN b).mp hb) a]
      exact <fun <h1, h2> => Set.Subset.antisymm h1 h2, fun h => <h.subset, h.superset
    >>
    * rw [ite_eq_right hb, Xenum_pos P_0 P_1 ha, Xenum_neg P_0 P_1 hb,
    trivialChar_spec P_0 P_1 eq1 heq1 a]
  * rw [selectFn_isOne, ite_eq_right ha, selectFn_isOne]
    by_cases hb : gN b = 1
    * rw [ite_eq_left hb, Xenum_neg P_0 P_1 ha, Xenum_pos P_0 P_1 hb, trivialChar_spec
     P_0 P_1 eq1 heq1 b]
      exact eq_comm
    * rw [ite_eq_right hb, Xenum_neg P_0 P_1 ha, Xenum_neg P_0 P_1 hb]
```

```
      exact iff_of_true rfl rfl

theorem interEqChar_spec (gN incl0 incl1 eq1 : Nat  ->  Nat)
    (hgN : forall  c, gN c = 1  <->  (stepFun (funListOf P_0 P_1 (decodeList c))
      : Set (ApproximableMap V_0 V_1)).Nonempty)
    (hincl0 : forall  s, incl0 s = 1  <->  P_0.X s.unpair.1  subseteq  P_0.X s.unpair
    .2)
    (hincl1 : forall  s, incl1 s = 1  <->  P_1.X s.unpair.1  subseteq  P_1.X s.unpair
    .2)
    (heq1 : forall  s, eq1 s = 1  <->  P_1.X s.unpair.1 = P_1.X s.unpair.2) (n m k :
    Nat) :
    interEqChar P_1 gN incl0 incl1 eq1 n m k = 1  <->
      Xenum P_0 P_1 gN n  I  Xenum P_0 P_1 gN m = Xenum P_0 P_1 gN k := by
  unfold interEqChar
  rw [selectFn_mul_iff, consPairChar_spec P_0 P_1 gN hgN n m,
    eqEnumChar_spec P_0 P_1 gN incl0 incl1 eq1 hgN hincl0 hincl1 heq1 (interIdx gN n m
    ) k]
  constructor
  * rintro <<k', hk'>, heq>
    have hne : (Xenum P_0 P_1 gN n  I  Xenum P_0 P_1 gN m).Nonempty :=
      (Xenum_nonempty P_0 P_1 gN hgN k').mono hk'
    rw [ <-  Xenum_inter_eq P_0 P_1 gN hgN n m hne]; exact heq
  * intro heq
    have hne : (Xenum P_0 P_1 gN n  I  Xenum P_0 P_1 gN m).Nonempty := by
      rw [heq]; exact Xenum_nonempty P_0 P_1 gN hgN k
    exact <<k, heq.superset>, by rw [Xenum_inter_eq P_0 P_1 gN hgN n m hne]; exact heq
    >

/-- **The function-space presentation.** Built from explicit characteristic functions
    for the
component presentations' relations (`gN` = function-space consistency, `incl0`/`incl1`
    = inclusion,
`eq1` = equality), so it is choice-free given those concrete functions. -/
def funPresentation (gN incl0 incl1 eq1 : Nat  ->  Nat)
    (hgN : forall  c, gN c = 1  <->  (stepFun (funListOf P_0 P_1 (decodeList c))
      : Set (ApproximableMap V_0 V_1)).Nonempty) (hgNp : Nat.Primrec gN)
    (hincl0 : forall  s, incl0 s = 1  <->  P_0.X s.unpair.1  subseteq  P_0.X s.unpair
    .2) (hincl0p : Nat.Primrec incl0)
    (hincl1 : forall  s, incl1 s = 1  <->  P_1.X s.unpair.1  subseteq  P_1.X s.unpair
    .2) (hincl1p : Nat.Primrec incl1)
    (heq1 : forall  s, eq1 s = 1  <->  P_1.X s.unpair.1 = P_1.X s.unpair.2) (heq1p :
    Nat.Primrec eq1) :
    ComputablePresentation (funSpace V_0 V_1) where
  X := Xenum P_0 P_1 gN
  mem_X := Xenum_mem P_0 P_1 gN hgN
  surj := by
    rintro W <<L, hL, rfl>, hne>
    obtain <el, hel> := exists_funListOf P_0 P_1 hL
    have hc : gN (encodeList el) = 1 := by rw [hgN, decodeList_encodeList, hel]; exact
     hne
    exact <encodeList el, by rw [Xenum_pos P_0 P_1 hc, decodeList_encodeList, hel]>
  interEq_computable :=
    <fun t => interEqChar P_1 gN incl0 incl1 eq1 t.unpair.1 t.unpair.2.unpair.1 t.
    unpair.2.unpair.2,
      primrec_interEqChar P_1 gN incl0 incl1 eq1 hgNp hincl0p hincl1p heq1p,
      fun t => (interEqChar_spec P_0 P_1 gN incl0 incl1 eq1 hgN hincl0 hincl1 heq1
```

```
        t.unpair.1 t.unpair.2.unpair.1 t.unpair.2.unpair.2).symm>
  cons_computable :=
    <fun t => consPairChar gN t.unpair.1 t.unpair.2, primrec_consPairChar gN hgNp,
      fun t => (consPairChar_spec P_0 P_1 gN hgN t.unpair.1 t.unpair.2).symm>
  inter := interIdx gN
  inter_primrec := primrec_interIdx gN hgNp
  inter_spec := by
    intro n m h
    obtain <k, hk> := h
    exact Xenum_inter_eq P_0 P_1 gN hgN n m ((Xenum_nonempty P_0 P_1 gN hgN k).mono hk
    )
  masterIdx := 0
  masterIdx_spec := by
    show Xenum P_0 P_1 gN 0 = Set.univ
    unfold Xenum
    rw [decodeList_zero]
    simp only [funListOf, List.map_nil, stepFun_nil, ite_self]

end

section
open NeighborhoodSystem Domain.Recursive ApproximableMap
variable {a b ? : Type*} {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}

theorem interList_cons (base : Set b) (Y : Set b) (M : List (Set b)) :
    interList base (Y :: M) = Y  I  interList base M := rfl

theorem mem_interList {base : Set b} {M : List (Set b)} {z : b} :
    z  in  interList base M  <->  z  in  base  /\  forall  Y  in  M, z  in  Y := by
  induction M with
  | nil => simp
  | cons Y M ih =>
    rw [interList_cons]
    simp only [Set.mem_inter_iff, ih, List.mem_cons]
    constructor
    * rintro <hY, hbase, hM>
      exact <hbase, fun W hW => hW.elim (fun h => h  |>  hY) (hM W)>
    * rintro <hbase, hall>
      exact <hall Y (Or.inl rfl), hbase, fun W hW => hall W (Or.inr hW)>

/-- A witness map `f  in  stepFun L` relates a common lower neighbourhood `Z` of the *
    selected* inputs
to the intersection of the *selected* outputs. The selection `sub` is processed entry-
    by-entry with
`inter_right`, so the proof is choice-free (no inclusion case-split). -/
theorem rel_interList_of_selection {f : ApproximableMap V_0 V_1}
    {Z : Set a} (hZ : V_0.mem Z) {sub : List (Set a  x  Set b)}
    (hmem : forall  p  in  sub, f.rel p.1 p.2) (hZle : forall  p  in  sub, Z  subseteq
      p.1) :
    f.rel Z (interList V_1.master (sub.map Prod.snd)) := by
  induction sub with
  | nil =>
    simp only [List.map_nil, interList_nil]
    exact f.mono f.master_rel (V_0.sub_master hZ) subset_rfl hZ V_1.master_mem
  | cons p sub ih =>
    have hp : f.rel p.1 p.2 := hmem p (List.mem_cons.mpr (Or.inl rfl))
```

```
    have hZp : f.rel Z p.2 :=
      f.mono hp (hZle p (List.mem_cons.mpr (Or.inl rfl))) subset_rfl hZ (f.rel_cod hp)
    have htail : f.rel Z (interList V_1.master (sub.map Prod.snd)) :=
      ih (fun q hq => hmem q (List.mem_cons.mpr (Or.inr hq)))
        (fun q hq => hZle q (List.mem_cons.mpr (Or.inr hq)))
    rw [List.map_cons, interList_cons]
    exact f.inter_right hZp htail

/-- **Proposition 3.9(i) (Scott 1981, PRG-19), forward direction -- choice-free.** If
    the
function-space neighbourhood `stepFun L` is non-empty, then for every selection `sub
    sqsub  L` whose
inputs admit a common lower neighbourhood `Z  in  ?_0`, the intersection of the
    selected outputs is a
neighbourhood of `?_1`. -/
theorem interList_mem_of_stepFun_nonempty {L : List (Set a  x  Set b)}
    (h : (stepFun L : Set (ApproximableMap V_0 V_1)).Nonempty) {sub : List (Set a  x
    Set b)}
    (hsub : sub.Sublist L) {Z : Set a} (hZ : V_0.mem Z) (hZle : forall  p  in  sub, Z
     subseteq  p.1) :
    V_1.mem (interList V_1.master (sub.map Prod.snd)) := by
  obtain <f, hf> := h
  have hmem : forall  p  in  sub, f.rel p.1 p.2 := fun p hp => hf p (hsub.subset hp)
  exact f.rel_cod (rel_interList_of_selection hZ hmem hZle)

variable (P_0 : ComputablePresentation V_0) (P_1 : ComputablePresentation V_1)

/-- **Proposition 3.9(i) (Scott 1981, PRG-19), over coded entry-lists -- choice-free
    .** The
function-space neighbourhood `?[X_{e_i}, Y_{e_i}]` coded by `el` is non-empty iff for
    every
sub-selection `sub  sqsub  el` whose inputs `{X_{e} ? e  in  sub}` admit a common lower
     neighbourhood
`Z  in  ?_0`, the intersection of the selected outputs `{Y_{e} ? e  in  sub}` is a
    neighbourhood of `?_1`.

* (?) is `interList_mem_of_stepFun_nonempty` (Milestone 1), pushed through `funPair`.
* (?) builds `leastMap`; its consistency hypothesis `hcons` is discharged for each
    input `X'` by
  **filtering `el` with the decidable `?_0`-inclusion `X'  subseteq  X_e`** (supplied
    choice-free by `P_0`), so
  that `interYs Delta_1 (funListOf el) X'` is exactly the intersection of the selected
    outputs. -/
theorem stepFun_funListOf_nonempty_iff (el : List Nat) :
    (stepFun (funListOf P_0 P_1 el) : Set (ApproximableMap V_0 V_1)).Nonempty  <->
      forall  sub : List Nat, sub.Sublist el  ->
        (exists  Z, V_0.mem Z  /\  forall  e  in  sub, Z  subseteq  P_0.X e.unpair.1)
     ->
          V_1.mem (interList V_1.master (sub.map (fun e => P_1.X e.unpair.2))) := by
  constructor
  * -- forward
    intro h sub hsub hcons
    obtain <Z, hZ, hZle> := hcons
    have hsub' : (sub.map (funPair P_0 P_1)).Sublist (funListOf P_0 P_1 el) := hsub.
    map _
    have hZle' : forall  p  in  sub.map (funPair P_0 P_1), Z  subseteq  p.1 := by
```

```
    intro p hp
    rw [List.mem_map] at hp
    obtain <e, he, rfl> := hp
    exact hZle e he
  have hres := interList_mem_of_stepFun_nonempty h hsub' hZ hZle'
  have hlist : (sub.map (funPair P_0 P_1)).map Prod.snd = sub.map (fun e => P_1.X e.
 unpair.2) := by
    rw [List.map_map]; rfl
  rwa [hlist] at hres
* -- backward
  intro hcond
  have hL := funListOf_valid P_0 P_1 el
  have hcons : forall  {X' : Set a}, V_0.mem X'  ->
      V_1.mem (interYs V_1.master (funListOf P_0 P_1 el) X') := by
    intro X' hX'
    obtain <nx, hnx> := P_0.surj hX'
    obtain <finc, _, hfinc> := P_0.incl_computable
    let : DecidablePred (fun e : Nat => X'  subseteq  P_0.X e.unpair.1) := fun e =>
      decidable_of_iff (finc (Nat.pair nx e.unpair.1) = 1) (by
        have h := hfinc (Nat.pair nx e.unpair.1)
        simp only [unpair_pair_fst, unpair_pair_snd] at h
        rw [hnx] at h
        exact h.symm)
    set sub : List Nat := el.filter (fun e => decide (X'  subseteq  P_0.X e.unpair
 .1)) with hsubdef
    have hsub : sub.Sublist el := List.filter_sublist
    have hmem_iff : forall  e, e  in  sub  <->  e  in  el  /\  X'  subseteq  P_0.X e
 .unpair.1 := by
      intro e
      rw [hsubdef, List.mem_filter, decide_eq_true_iff]
    have heq : interYs V_1.master (funListOf P_0 P_1 el) X'
        = interList V_1.master (sub.map (fun e => P_1.X e.unpair.2)) := by
      apply Set.ext; intro z
      rw [mem_interYs, mem_interList]
      constructor
      * rintro <hzb, hall>
        refine <hzb, ?_>
        intro Y hY
        rw [List.mem_map] at hY
        obtain <e, hesub, rfl> := hY
        have he := (hmem_iff e).mp hesub
        have hpair : funPair P_0 P_1 e  in  funListOf P_0 P_1 el := List.mem_map.mpr
  <e, he.1, rfl>
        exact hall (funPair P_0 P_1 e) hpair he.2
      * rintro <hzb, hall>
        refine <hzb, ?_>
        intro p hp hXp
        rw [funListOf, List.mem_map] at hp
        obtain <e, hee, rfl> := hp
        have hesub : e  in  sub := (hmem_iff e).mpr <hee, hXp>
        exact hall (P_1.X e.unpair.2) (List.mem_map.mpr <e, hesub, rfl>)
    rw [heq]
    refine hcond sub hsub <X', hX', ?_>
    intro e he
    exact ((hmem_iff e).mp he).2
  exact <leastMap (funListOf P_0 P_1 el) hL hcons, leastMap_mem_stepFun hL hcons>
```

```
section ConsChain

variable {V : NeighborhoodSystem a} (P : ComputablePresentation V)

theorem mem_interFrom {A : Set a} {js : List Nat} {z : a} :
    z  in  interFrom P A js  <->  z  in  A  /\  forall  j  in  js, z  in  P.X j := by
  induction js generalizing A with
  | nil => simp [interFrom]
  | cons j js ih =>
    rw [interFrom, ih]
    simp only [Set.mem_inter_iff, List.mem_cons]
    constructor
    * rintro <<hA, hj>, hrest>
      exact <hA, fun j' hj' => hj'.elim (fun h => h  |>  hj) (hrest j')>
    * rintro <hA, hall>
      exact <<hA, hall j (Or.inl rfl)>, fun j' hj' => hall j' (Or.inr hj')>

theorem interFrom_subset {A : Set a} {js : List Nat} : interFrom P A js  subseteq  A
    :=
  fun _ hz => (mem_interFrom P |>.mp hz).1

/-- A finite running intersection with a neighbourhood inside it is itself a
    neighbourhood. -/
theorem interFrom_mem_of_witness {A : Set a} {js : List Nat} {Z : Set a}
    (hZ : V.mem Z) (hZsub : Z  subseteq  interFrom P A js) (hA : V.mem A) :
    V.mem (interFrom P A js) := by
  induction js generalizing A with
  | nil => simpa [interFrom] using hA
  | cons j js ih =>
    rw [interFrom] at hZsub  |-
    have hAXj : V.mem (A  I  P.X j) :=
      V.inter_mem hA (P.mem_X j) hZ (hZsub.trans (interFrom_subset P))
    exact ih hZsub hAXj

/-- The fold of `inter` along `js` (starting from `a`, with `X a = A`) computes the
    running
intersection `interFrom A js` -- *provided* that intersection is consistent (so each
    prefix is too, by
`inter_spec`). -/
theorem interFrom_eq_of_foldl : forall  (js : List Nat) (A : Set a) (a : Nat),
    P.X a = A  ->  V.mem A  ->  V.mem (interFrom P A js)  ->
    P.X (js.foldl (fun acc j => P.inter acc j) a) = interFrom P A js
  | [], A, a, hXa, _, _ => by simpa [interFrom] using hXa
  | j :: js, A, a, hXa, hA, hmem => by
    have hmem' : V.mem (interFrom P (A  I  P.X j) js) := by rwa [interFrom] at hmem
    have hAXj : V.mem (A  I  P.X j) :=
      V.inter_mem hA (P.mem_X j) hmem' (interFrom_subset P)
    have hk : exists  k, P.X k  subseteq  P.X a  I  P.X j := by
      obtain <k, hk> := P.surj hAXj
      exact <k, by rw [hk, hXa]>
    have hXinter : P.X (P.inter a j) = A  I  P.X j := by rw [P.inter_spec hk, hXa]
    rw [List.foldl_cons, interFrom]
    exact interFrom_eq_of_foldl js (A  I  P.X j) (P.inter a j) hXinter hAXj hmem'

/-- When `js` is consistent, the `inter`-chain index genuinely indexes `?{X_j ? j  in
```

```
    js}`. -/
theorem idxchain_spec {js : List Nat} (h : V.mem (interFrom P V.master js)) :
    P.X (idxchain P js) = interFrom P V.master js :=
  interFrom_eq_of_foldl P js V.master P.masterIdx P.masterIdx_spec V.master_mem h

end ConsChain

theorem bitSelect_cons (e : Nat) (L : List Nat) (b : Nat) :
    bitSelect (e :: L) b =
      if b % 2 = 1 then e :: bitSelect L (b / 2) else bitSelect L (b / 2) := rfl

theorem bitSelect_sublist (L : List Nat) (b : Nat) : (bitSelect L b).Sublist L := by
  induction L generalizing b with
  | nil => simp
  | cons e L ih =>
    rw [bitSelect_cons]
    split
    * exact (ih (b / 2)).cons_cons e
    * exact (ih (b / 2)).cons e

theorem exists_bitSelect_lt {L sub : List Nat} (hsub : sub.Sublist L) :
    exists  b, b < 2 ^ L.length  /\  bitSelect L b = sub := by
  induction hsub with
  | slnil => exact <0, by rw [List.length_nil, pow_zero]; exact Nat.one_pos, rfl>
  | @cons l_1 l_2 e _h ih =>
    obtain <b, hb, heq> := ih
    refine <2 * b, ?_, ?_>
    * have hpow : 2 ^ (l_2.length + 1) = 2 ^ l_2.length * 2 := pow_succ 2 l_2.length
      rw [List.length_cons]; omega
    * rw [bitSelect_cons, ite_eq_right (by omega), show 2 * b / 2 = b by omega]; exact
     heq
  | @cons_cons l_1 l_2 e _h ih =>
    obtain <b, hb, heq> := ih
    refine <2 * b + 1, ?_, ?_>
    * have hpow : 2 ^ (l_2.length + 1) = 2 ^ l_2.length * 2 := pow_succ 2 l_2.length
      rw [List.length_cons]; omega
    * rw [bitSelect_cons, ite_eq_left (by omega), show (2 * b + 1) / 2 = b by omega];
    rw [heq]

/-- A universal statement over all sublists of `decodeList c` is a *bounded* universal
    over bitmasks
`b < 2 ^ c` (using `(decodeList c).length  <=  c`). -/
theorem forall_sublist_iff_forall_bitmask (c : Nat) (p : List Nat  ->  Prop) :
    (forall  sub, sub.Sublist (decodeList c)  ->  p sub)  <->
      forall  b, b < 2 ^ c  ->  p (bitSelect (decodeList c) b) := by
  constructor
  * intro hall b _; exact hall _ (bitSelect_sublist _ _)
  * intro hall sub hsub
    obtain <b, hb, heq> := exists_bitSelect_lt hsub
    have hb' : b < 2 ^ c :=
      Nat.lt_of_lt_of_le hb (Nat.pow_le_pow_right (Nat.le_succ 1) (
    decodeList_length_le c))
    rw [ <-  heq]; exact hall b hb'

section ConsFold
```

```
variable {V : NeighborhoodSystem a} (P : ComputablePresentation V)

theorem consUpd_eval (projFn fc : Nat  ->  Nat) (b idx flag x : Nat) :
    consUpd P projFn fc (Nat.pair b (Nat.pair idx flag)) x =
      selectFn (b % 2)
        (Nat.pair (b / 2)
          (Nat.pair (P.inter idx (projFn x))
            (selectFn flag (isOne (fc (Nat.pair idx (projFn x)))) 0)))
        (Nat.pair (b / 2) (Nat.pair idx flag)) := by
  unfold consUpd
  simp only [unpair_pair_fst, unpair_pair_snd]

/-- Once the flag is `0`, the fold keeps it `0` (inconsistency persists). -/
theorem consUpd_foldl_flag_zero (projFn fc : Nat  ->  Nat) (el : List Nat) (b a : Nat)
     :
    (List.foldl (fun s x => consUpd P projFn fc s x)
      (Nat.pair b (Nat.pair a 0)) el).unpair.2.unpair.2 = 0 := by
  induction el generalizing b a with
  | nil => simp only [List.foldl_nil, unpair_pair_snd]
  | cons e el ih =>
    rw [List.foldl_cons, consUpd_eval]
    by_cases hb : b % 2 = 1
    * rw [hb, selectFn_one, selectFn_zero]
      exact ih (b / 2) (P.inter a (projFn e))
    * rw [show b % 2 = 0 by omega, selectFn_zero]
      exact ih (b / 2) a

/-- **Single-pass consistency fold -- correctness.** Starting from index `a` (an index
    of `X a`) with
flag `1`, the final flag is `1` iff the running intersection of `X a` with the selected
     components is
a neighbourhood. Choice-free. -/
theorem consUpd_foldl_spec (projFn fc : Nat  ->  Nat)
    (hfc : forall  s, fc s = 1  <->  exists  k, P.X k  subseteq  P.X s.unpair.1  I  P.
    X s.unpair.2)
    (el : List Nat) (b a : Nat) :
    (List.foldl (fun s x => consUpd P projFn fc s x)
        (Nat.pair b (Nat.pair a 1)) el).unpair.2.unpair.2 = 1  <->
      V.mem (interFrom P (P.X a) (List.map projFn (bitSelect el b))) := by
  induction el generalizing b a with
  | nil =>
    have hL : (List.foldl (fun s x => consUpd P projFn fc s x)
        (Nat.pair b (Nat.pair a 1)) []).unpair.2.unpair.2 = 1 := by
      simp only [List.foldl_nil, unpair_pair_snd]
    rw [bitSelect_nil, List.map_nil, show interFrom P (P.X a) [] = P.X a from rfl]
    exact iff_of_true hL (P.mem_X a)
  | cons e el ih =>
    rw [List.foldl_cons, consUpd_eval, bitSelect_cons]
    by_cases hb : b % 2 = 1
    * rw [ite_eq_left hb, List.map_cons, hb, selectFn_one, selectFn_one,
        show interFrom P (P.X a) (projFn e :: List.map projFn (bitSelect el (b / 2)))
          = interFrom P (P.X a  I  P.X (projFn e)) (List.map projFn (bitSelect el (b /
     2))) from rfl]
      by_cases hfce : fc (Nat.pair a (projFn e)) = 1
      * have hcons : exists  k, P.X k  subseteq  P.X a  I  P.X (projFn e) := by
          have h := (hfc (Nat.pair a (projFn e))).mp hfce
```

```
          rwa [unpair_pair_fst, unpair_pair_snd] at h
        have hXa' : P.X (P.inter a (projFn e)) = P.X a  I  P.X (projFn e) := P.
    inter_spec hcons
        rw [(isOne_eq_one_iff _).mpr hfce, ih (b / 2) (P.inter a (projFn e)), hXa']
      * have his0 : isOne (fc (Nat.pair a (projFn e))) = 0 := by
          have hle := isOne_le_one (fc (Nat.pair a (projFn e)))
          rcases (show isOne (fc (Nat.pair a (projFn e))) = 0  \/
              isOne (fc (Nat.pair a (projFn e))) = 1 by omega) with h | h
          * exact h
          * exact absurd ((isOne_eq_one_iff _).mp h) hfce
        rw [his0]


        constructor
        * intro h
          rw [consUpd_foldl_flag_zero P projFn fc el (b / 2) (P.inter a (projFn e))]
    at h
          exact absurd h (by decide)
        * intro h
          exact absurd
            ((hfc (Nat.pair a (projFn e))).mpr (by
              obtain <k, hk> := P.surj h
              rw [unpair_pair_fst, unpair_pair_snd]
              exact <k, by rw [hk]; exact interFrom_subset P>))
            hfce
    * rw [ite_eq_right hb, show b % 2 = 0 by omega, selectFn_zero]
      exact ih (b / 2) a

theorem consCharAt_spec (projFn fc : Nat  ->  Nat)
    (hfc : forall  s, fc s = 1  <->  exists  k, P.X k  subseteq  P.X s.unpair.1  I  P.
    X s.unpair.2) (w : Nat) :
    consCharAt P projFn fc w = 1  <->
      V.mem (interFrom P V.master
        (List.map projFn (bitSelect (decodeList w.unpair.2) w.unpair.1))) := by
  unfold consCharAt
  rw [foldCode_eq']
  have hstep : (fun acc x => consStp P projFn fc (Nat.pair x (Nat.pair acc 0)))
      = (fun acc x => consUpd P projFn fc acc x) := by
    funext acc x
    show consStp P projFn fc (Nat.pair x (Nat.pair acc 0)) = _
    unfold consStp
    simp only [unpair_pair_fst, unpair_pair_snd]
  rw [hstep, consUpd_foldl_spec P projFn fc hfc (decodeList w.unpair.2) w.unpair.1 P.
    masterIdx,
    P.masterIdx_spec]

theorem primrec_consStp {projFn fc : Nat  ->  Nat} (hprojp : Nat.Primrec projFn) (hfcp
     : Nat.Primrec fc) :
    Nat.Primrec (consStp P projFn fc) := by
  have hb : Nat.Primrec (fun w : Nat => w.unpair.2.unpair.1.unpair.1) :=
    Nat.Primrec.left.comp (Nat.Primrec.left.comp Nat.Primrec.right)
  have hidx : Nat.Primrec (fun w : Nat => w.unpair.2.unpair.1.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp (Nat.Primrec.right.comp (Nat.Primrec.left.comp Nat.Primrec.
    right))
  have hflag : Nat.Primrec (fun w : Nat => w.unpair.2.unpair.1.unpair.2.unpair.2) :=
    Nat.Primrec.right.comp (Nat.Primrec.right.comp (Nat.Primrec.left.comp Nat.Primrec.
    right))
```

```
  have hpx : Nat.Primrec (fun w : Nat => projFn w.unpair.1) := hprojp.comp Nat.Primrec
    .left
  have hpair := hidx.pair hpx
  have hinter := P.inter_primrec.comp hpair
  have hfcv := primrec_isOne.comp (hfcp.comp hpair)
  have hsel := primrec_selectFn hflag hfcv (Nat.Primrec.const 0)
  have hyes := (primrec_div2.comp hb).pair (hinter.pair hsel)
  have hno := (primrec_div2.comp hb).pair (hidx.pair hflag)
  refine (primrec_selectFn (primrec_mod2.comp hb) hyes hno).of_eq (fun w => ?_)
  unfold consStp consUpd
  simp only [unpair_pair_fst, unpair_pair_snd]

theorem primrec_consCharAt {projFn fc : Nat  ->  Nat} (hprojp : Nat.Primrec projFn)
    (hfcp : Nat.Primrec fc) : Nat.Primrec (consCharAt P projFn fc) := by
  have hfold := primrec_foldCode (primrec_consStp P hprojp hfcp) (Nat.Primrec.const 0)
    (Nat.Primrec.left.pair ((Nat.Primrec.const P.masterIdx).pair (Nat.Primrec.const 1)
    ))
    Nat.Primrec.right
  exact ((Nat.Primrec.right.comp Nat.Primrec.right).comp hfold).of_eq (fun _ => rfl)

/-- **Consistency of the bitmask-selected sublist is recursively decidable.** Choice-
    free. -/
theorem consFold_decidable (projFn fc : Nat  ->  Nat)
    (hfc : forall  s, fc s = 1  <->  exists  k, P.X k  subseteq  P.X s.unpair.1  I  P.
    X s.unpair.2)
    (hprojp : Nat.Primrec projFn) (hfcp : Nat.Primrec fc) :
    RecDecidable (fun w =>
      V.mem (interFrom P V.master
        (List.map projFn (bitSelect (decodeList w.unpair.2) w.unpair.1)))) :=
  <consCharAt P projFn fc, primrec_consCharAt P hprojp hfcp,
    fun w => (consCharAt_spec P projFn fc hfc w).symm>

end ConsFold

/-- `interList` over `js.map X` is the same set as the `interFrom`-chain over `js`. -/
theorem interList_X_eq_interFrom {V : NeighborhoodSystem a} (P :
    ComputablePresentation V)
    (js : List Nat) : interList V.master (js.map (fun j => P.X j)) = interFrom P V.
    master js := by
  apply Set.ext; intro z
  rw [mem_interList, mem_interFrom]
  constructor
  * rintro <hb, hall>
    exact <hb, fun j hj => hall (P.X j) (List.mem_map.mpr <j, hj, rfl>)>
  * rintro <hb, hall>
    refine <hb, fun Y hY => ?_>
    rw [List.mem_map] at hY
    obtain <j, hj, rfl> := hY
    exact hall j hj

/-- The antecedent of Prop 3.9(i) -- `{X_{e.1} ? e  in  sub}` admits a common lower
    neighbourhood -- is
exactly consistency of `{e.1 ? e  in  sub}` in `?_0` (in `interFrom` form). Choice-free
    . -/
theorem antecedent_cons_iff (sub : List Nat) :
    (exists  Z, V_0.mem Z  /\  forall  e  in  sub, Z  subseteq  P_0.X e.unpair.1)  <->
```

```
      V_0.mem (interFrom P_0 V_0.master (sub.map (fun e => e.unpair.1))) := by
  constructor
  * rintro <Z, hZ, hZle>
    refine interFrom_mem_of_witness P_0 hZ (fun z hz => ?_) V_0.master_mem
    rw [mem_interFrom]
    refine <V_0.sub_master hZ hz, fun j hj => ?_>
    rw [List.mem_map] at hj
    obtain <e, he, rfl> := hj
    exact hZle e he hz
  * intro hmem
    refine <P_0.X (idxchain P_0 (sub.map (fun e => e.unpair.1))), P_0.mem_X _, fun e
    he z hz => ?_>
    have hj : e.unpair.1  in  sub.map (fun e => e.unpair.1) := List.mem_map.mpr <e, he
    , rfl>
    rw [idxchain_spec P_0 hmem] at hz
    exact (mem_interFrom P_0 |>.mp hz).2 _ hj

/-- The consequent of Prop 3.9(i), rewritten from `interList` to `interFrom` form. -/
theorem funConsequent_eq (sub : List Nat) :
    interList V_1.master (sub.map (fun e => P_1.X e.unpair.2))
      = interFrom P_1 V_1.master (sub.map (fun e => e.unpair.2)) := by
  rw [ <-  interList_X_eq_interFrom P_1, List.map_map]; rfl

/-- **Proposition 3.9(i) as a bounded bitmask quantifier.** The function-space
    neighbourhood coded by
`decodeList c` is non-empty iff for every bitmask `b < 2^c`, consistency of the
    selected inputs in
`?_0` forces consistency of the selected outputs in `?_1`. Choice-free. -/
theorem funCons_iff (c : Nat) :
    (stepFun (funListOf P_0 P_1 (decodeList c)) : Set (ApproximableMap V_0 V_1)).
    Nonempty  <->
      forall  b, b < 2 ^ c  ->
        V_0.mem (interFrom P_0 V_0.master ((bitSelect (decodeList c) b).map (fun e =>
    e.unpair.1)))  ->
          V_1.mem (interFrom P_1 V_1.master ((bitSelect (decodeList c) b).map (fun e
    => e.unpair.2))) := by
  rw [stepFun_funListOf_nonempty_iff]
  constructor
  * intro h b hb hant0
    have hsub : (bitSelect (decodeList c) b).Sublist (decodeList c) :=
    bitSelect_sublist _ _
    have hcons := h _ hsub ((antecedent_cons_iff P_0 (bitSelect (decodeList c) b)).mpr
     hant0)
    rwa [funConsequent_eq P_1] at hcons
  * intro h sub hsub hant
    obtain <b, hb, heq> := exists_bitSelect_lt hsub
    have hb' : b < 2 ^ c :=
      Nat.lt_of_lt_of_le hb (Nat.pow_le_pow_right (Nat.le_succ 1) (
    decodeList_length_le c))
    have hcons := h b hb' (by rw [heq]; exact (antecedent_cons_iff P_0 sub).mp hant)
    rw [heq,  <-  funConsequent_eq P_1] at hcons
    exact hcons

/-- **The function-space consistency relation is recursively decidable.** Given the
    binary
consistency deciders `fc0`/`fc1` of the two presentations, `(stepFun (funListOf (
```

```
    decodeList c)))` is
non-empty is recursively decidable in `c`. Choice-free. -/
theorem funCons_decidable (fc0 fc1 : Nat  ->  Nat)
    (hfc0 : forall  s, fc0 s = 1  <->  exists  k, P_0.X k  subseteq  P_0.X s.unpair.1
     I  P_0.X s.unpair.2)
    (hfc1 : forall  s, fc1 s = 1  <->  exists  k, P_1.X k  subseteq  P_1.X s.unpair.1
     I  P_1.X s.unpair.2)
    (hfc0p : Nat.Primrec fc0) (hfc1p : Nat.Primrec fc1) :
    RecDecidable (fun c =>
      (stepFun (funListOf P_0 P_1 (decodeList c)) : Set (ApproximableMap V_0 V_1)).
    Nonempty) := by
  have hc0 := consFold_decidable P_0 (fun e => e.unpair.1) fc0 hfc0 Nat.Primrec.left
    hfc0p
  have hc1 := consFold_decidable P_1 (fun e => e.unpair.2) fc1 hfc1 Nat.Primrec.right
    hfc1p
  have himp : RecDecidable (fun w =>
      V_0.mem (interFrom P_0 V_0.master
          (List.map (fun e => e.unpair.1) (bitSelect (decodeList w.unpair.2) w.unpair
    .1)))  ->
        V_1.mem (interFrom P_1 V_1.master
          (List.map (fun e => e.unpair.2) (bitSelect (decodeList w.unpair.2) w.unpair
    .1)))) := by
    refine RecDecidable.of_iff (fun w => ?_) (hc0.not.or hc1)
    constructor
    * intro himp
      rcases hc0.em w with h0 | h0
      * exact Or.inr (himp h0)
      * exact Or.inl h0
    * rintro (h0 | h1) hp0
      * exact absurd hp0 h0
      * exact h1
  refine RecDecidable.of_iff (fun c => ?_)
    (himp.bForall (bound := fun n => 2 ^ n) (primrec_two_pow primrec_id))
  rw [funCons_iff P_0 P_1 c]
  simp only [unpair_pair_fst, unpair_pair_snd]

/-- `selectFn A B 1 = 1` decides the implication `A = 1  ->  B = 1` for `A  in  {0,1}`.
     -/
theorem selectFn_one_iff_imp {A B : Nat} (hA : A  <=  1) :
    selectFn A B 1 = 1  <->  (A = 1  ->  B = 1) := by
  rcases (show A = 0  \/  A = 1 by omega) with h | h <;> subst h
  * rw [selectFn_zero]; simp
  * rw [selectFn_one]; simp

theorem funConsChar_spec (fc0 fc1 : Nat  ->  Nat)
    (hfc0 : forall  s, fc0 s = 1  <->  exists  k, P_0.X k  subseteq  P_0.X s.unpair.1
     I  P_0.X s.unpair.2)
    (hfc1 : forall  s, fc1 s = 1  <->  exists  k, P_1.X k  subseteq  P_1.X s.unpair.1
     I  P_1.X s.unpair.2) (c : Nat) :
    funConsChar P_0 P_1 fc0 fc1 c = 1  <->
      (stepFun (funListOf P_0 P_1 (decodeList c)) : Set (ApproximableMap V_0 V_1)).
    Nonempty := by
  rw [funCons_iff P_0 P_1 c]
  unfold funConsChar
  rw [bForallFn_eq_one_iff]
  refine forall_congr' (fun b => ?_)
```

```
  refine imp_congr_right (fun hb => ?_)
  unfold funImpChar
  rw [selectFn_one_iff_imp (isOne_le_one _), isOne_eq_one_iff, isOne_eq_one_iff,
    consCharAt_spec P_0 (fun e => e.unpair.1) fc0 hfc0,
    consCharAt_spec P_1 (fun e => e.unpair.2) fc1 hfc1]
  simp only [unpair_pair_fst, unpair_pair_snd]

theorem primrec_funImpChar (fc0 fc1 : Nat  ->  Nat) (hfc0p : Nat.Primrec fc0) (hfc1p :
     Nat.Primrec fc1) :
    Nat.Primrec (funImpChar P_0 P_1 fc0 fc1) :=
  primrec_selectFn (primrec_isOne.comp (primrec_consCharAt P_0 Nat.Primrec.left hfc0p)
    )
    (primrec_isOne.comp (primrec_consCharAt P_1 Nat.Primrec.right hfc1p)) (Nat.Primrec
    .const 1)

theorem primrec_funConsChar (fc0 fc1 : Nat  ->  Nat) (hfc0p : Nat.Primrec fc0) (hfc1p
    : Nat.Primrec fc1) :
    Nat.Primrec (funConsChar P_0 P_1 fc0 fc1) := by
  have hgp := primrec_funImpChar P_0 P_1 fc0 fc1 hfc0p hfc1p
  have hGfn : Nat.Primrec (fun w => selectFn w.unpair.2.unpair.2
      (isOne (funImpChar P_0 P_1 fc0 fc1 (Nat.pair w.unpair.2.unpair.1 w.unpair.1)))
    0) :=
    primrec_selectFn (Nat.Primrec.right.comp Nat.Primrec.right)
      (primrec_isOne.comp (hgp.comp
        ((Nat.Primrec.left.comp Nat.Primrec.right).pair Nat.Primrec.left)))
      (Nat.Primrec.const 0)
  have hprec := Nat.Primrec.prec (Nat.Primrec.const 1) hGfn
  refine (hprec.comp (primrec_id.pair (primrec_two_pow primrec_id))).of_eq (fun c => ?
    _)
  show _ = funConsChar P_0 P_1 fc0 fc1 c
  unfold funConsChar bForallFn
  simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd, id_eq]

end
end DomainTheory.NeighbourhoodSystems
end
```

3.8.14 Lecture VII exercises

EXERCISES

3.8.15 Exercise 7.13 — axiomatic neighbourhood structure

Show that an effectively given domain can always be identified with a relation $\mathrm{INCL}(n, m)$ on integers, where the two derived relations

$$\mathrm{CONS}(n, m) \text{ iff } \exists k.\, \mathrm{INCL}(k, n) \text{ and } \mathrm{INCL}(k, m);$$

$$\mathrm{MEET}(n, m, k) \text{ iff } \forall j\, [\mathrm{INCL}(j, k) \text{ iff } \mathrm{INCL}(j, n) \text{ and } \mathrm{INCL}(j, m)]$$

are both recursively decidable, and where the following axioms hold:

(i) $\forall n.\, \mathrm{INCL}(n, n)$;

(ii) $\forall n, m, k.\, \mathrm{INCL}(n, m)$ and $\mathrm{INCL}(m, k)$ imply $\mathrm{INCL}(n, k)$;

(iii) $\exists m\, \forall n.\, \mathrm{INCL}(n, m)$

(iv) $\forall n, m.\, \mathrm{CONS}(n, m)$ implies $\exists k.\, \mathrm{MEET}(n, m, k)$.

(Hint: Consider the neighbourhood system

$$\mathcal{D} = \big\{\{m \in \mathbf{N} \mid \mathrm{INCL}(m, n)\} \mid n \in \mathbf{N}\big\}.$$

Is this essentially any effectively given system?)

Exercise_7_13.lean

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Definition_7_1
public import Mathlib.Order.BourbakiWitt

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems.Exercise713
open NeighborhoodSystem Domain.Recursive
/- concordance: source L4609 EXERCISE 7.13 axiomatic neighbourhood structure -/
/-- **Exercise 7.13 -- the abstract data.** A relation `INCL : Nat  ->  Nat  ->  Prop`
    together with the
witnesses and decidability/axiom data Scott requires. The derived relations are
`CONS`/`MEET` (below); the literal existential axioms (iii)/(iv) are `axiom_iii`/`
    axiom_iv`.

We carry `topIdx` (a master code) and a primitive-recursive `meetIdx` as *data* --
    exactly mirroring
`ComputablePresentation`, which carries its `inter` function as primitive-recursive
    data rather than
recovering it by an unbounded search. This is the faithful "effective" reading of
    axioms (iii)/(iv).
-/
structure InclStructure where
  /-- Scott's integer inclusion relation `INCL(n, m)` (the image of `X_n  subseteq  X
    ?`). -/
  INCL : Nat  ->  Nat  ->  Prop
  /-- A primitive-recursive meet function: an index of the binary meet of consistent
    codes. -/
  meetIdx : Nat  ->  Nat  ->  Nat
  /-- A master code (witness of axiom (iii)). -/
  topIdx : Nat
  /-- `INCL` is recursively decidable. -/
  incl_dec : RecDecidable_2 INCL
  /-- `CONS(n, m)  <->  exists  k. INCL(k, n)  /\  INCL(k, m)` is recursively decidable
    . -/
  cons_dec : RecDecidable_2 (fun n m => exists  k, INCL k n  /\  INCL k m)
  /-- `MEET(n, m, k)  <->  forall  j. (INCL(j, k)  <->  INCL(j, n)  /\  INCL(j, m))` is
     recursively decidable. -/
```

```
  meet_dec : RecDecidable_3 (fun n m k => forall  j, INCL j k  <->  (INCL j n  /\
    INCL j m))
  /-- The meet function is primitive recursive (on the `Nat.pair` coding of `n, m`). -/
  meetIdx_primrec : Nat.Primrec (fun t => meetIdx t.unpair.1 t.unpair.2)
  /-- **(i)** `INCL` is reflexive. -/
  incl_refl : forall  n, INCL n n
  /-- **(ii)** `INCL` is transitive. -/
  incl_trans : forall  {n m k}, INCL n m  ->  INCL m k  ->  INCL n k
  /-- **(iii)** the master code `topIdx` is `INCL`-greatest. -/
  topIdx_spec : forall  n, INCL n topIdx
  /-- **(iv)** `meetIdx n m` is a `MEET` of `n, m` whenever `(n, m)` is consistent. -/
  meetIdx_spec : forall  {n m}, (exists  k, INCL k n  /\  INCL k m)  ->
    forall  j, INCL j (meetIdx n m)  <->  (INCL j n  /\  INCL j m)

end DomainTheory.NeighbourhoodSystems.Exercise713
end

section
namespace DomainTheory.NeighbourhoodSystems.Exercise713
open NeighborhoodSystem Domain.Recursive

section
namespace InclStructure

variable (I : InclStructure)

/-- Scott's consistency relation `CONS(n, m) : <->  exists  k. INCL(k, n)  /\  INCL(k,
    m)`. -/
def CONS (n m : Nat) : Prop := exists  k, I.INCL k n  /\  I.INCL k m

/-- Scott's binary-meet relation `MEET(n, m, k) : <->  forall  j. (INCL(j, k)  <->
    INCL(j, n)  /\  INCL(j, m))`. -/
def MEET (n m k : Nat) : Prop := forall  j, I.INCL j k  <->  (I.INCL j n  /\  I.INCL j
     m)

/-- The hint neighbourhood `S_n = {m ? INCL(m, n)}` over the token type `Nat`. -/
def toNbhd (n : Nat) : Set Nat := {m | I.INCL m n}

@[simp] theorem mem_toNbhd {n m : Nat} : m  in  I.toNbhd n  <->  I.INCL m n := Iff.rfl

/-- `S_n  I  S? = S?  <->  MEET(n, m, k)` -- Scott's relation 7.1(i) *is* `MEET`. -/
theorem toNbhd_inter_eq_iff (n m k : Nat) :
    I.toNbhd n  I  I.toNbhd m = I.toNbhd k  <->  I.MEET n m k := by
  unfold MEET
  rw [Set.ext_iff]
  refine forall_congr' fun j => ?_
  rw [Set.mem_inter_iff, mem_toNbhd, mem_toNbhd, mem_toNbhd]
  exact iff_comm

/-- **The key inclusion law:** `S_n  subseteq  S?  <->  INCL(n, k)`. ` -> ` reads off `
    INCL(n, k)` from `n  in  S_n`
(reflexivity (i)); ` <- ` is transitivity (ii). -/
theorem toNbhd_subset_iff {n k : Nat} : I.toNbhd n  subseteq  I.toNbhd k  <->  I.INCL
    n k := by
  constructor
  * intro h
```

```
    exact h (I.incl_refl n)
  * intro hnk m hm
    exact I.incl_trans hm hnk

/-- `S?  subseteq  S_n  I  S?  <->  INCL(k, n)  /\  INCL(k, m)`. -/
theorem toNbhd_subset_inter_iff (k n m : Nat) :
    I.toNbhd k  subseteq  I.toNbhd n  I  I.toNbhd m  <->  (I.INCL k n  /\  I.INCL k m)
     := by
  rw [Set.subset_inter_iff, I.toNbhd_subset_iff, I.toNbhd_subset_iff]

/-- `S_{topIdx} = Nat`: the master code's neighbourhood is everything (axiom (iii)). -/
theorem toNbhd_top : I.toNbhd I.topIdx = Set.univ := by
  ext m
  simp only [mem_toNbhd, Set.mem_univ, iff_true]
  exact I.topIdx_spec m

/-- **Exercise 7.13 (?) -- the hint system is a neighbourhood system.** `S_n = {m ?
    INCL(m, n)}` over
tokens `Nat`, master `Delta = Nat = S_{topIdx}`; closure under intersection is axiom (
    iv) via `MEET`. -/
def toSystem : NeighborhoodSystem Nat where
  mem Y := exists  n, Y = I.toNbhd n
  master := Set.univ
  master_nonempty := Set.univ_nonempty
  master_mem := <I.topIdx, I.toNbhd_top.symm>
  inter_mem := by
    rintro X Y Z <n, rfl> <m, rfl> <l, rfl> hZsub
    have hln : I.INCL l n := I.toNbhd_subset_iff.mp (hZsub.trans Set.inter_subset_left
    )
    have hlm : I.INCL l m := I.toNbhd_subset_iff.mp (hZsub.trans Set.
    inter_subset_right)
    exact <I.meetIdx n m, (I.toNbhd_inter_eq_iff n m (I.meetIdx n m)).mpr
      (I.meetIdx_spec <l, hln, hlm>)>
  sub_master := by rintro X _; exact Set.subset_univ X

@[simp] theorem toSystem_mem {Y : Set Nat} : I.toSystem.mem Y  <->  exists  n, Y = I.
    toNbhd n := Iff.rfl

end InclStructure
end

section
variable {a : Type*} {V : NeighborhoodSystem a}

/-- **The `MEET  <->  interEq` reduction.** On an effectively given system, Scott's `
    MEET(n, m, k)` is
equivalent to `X_n  I  X? = X?`, so it is decidable by relation 7.1(i).

`?` is `Set.subset_inter_iff`. ` => `: from `MEET` at `j = k` we get `X?  subseteq  X_n
     I  X?`, so `X?` witnesses
consistency of `(n, m)`; `inter_mem` then makes `X_n  I  X?` a neighbourhood, `surj`
    enumerates it as
some `X_p`, and `MEET` at `j = p` gives `X_n  I  X? = X_p  subseteq  X?`. -/
theorem meet_iff_interEq (P : ComputablePresentation V) (n m k : Nat) :
    (forall  j, P.X j  subseteq  P.X k  <->  (P.X j  subseteq  P.X n  /\  P.X j
    subseteq  P.X m))  <->  (P.X n  I  P.X m = P.X k) := by
```

```
  constructor
  * intro h
    have hk : P.X k  subseteq  P.X n  I  P.X m :=
      Set.subset_inter ((h k).mp subset_rfl).1 ((h k).mp subset_rfl).2
    refine Set.Subset.antisymm ?_ hk
    have hmem : V.mem (P.X n  I  P.X m) := V.inter_mem (P.mem_X n) (P.mem_X m) (P.
    mem_X k) hk
    obtain <p, hp> := P.surj hmem
    have hpk : P.X p  subseteq  P.X k :=
      (h p).mpr (by rw [hp]; exact <Set.inter_subset_left, Set.inter_subset_right>)
    exact hp  |>  hpk
  * intro h j
    rw [ <-  h, Set.subset_inter_iff]

/-- **Exercise 7.13 ( => ) -- every effectively given domain is an `INCL` relation.**
    Given a computable
presentation `P` of `V`, set `INCL(n, m) := X_n  subseteq  X?`. Reflexivity/
    transitivity are those of ` subseteq `; the
master code is `P.masterIdx` (`X_n  subseteq  master`); the meet function is `P.inter`.
     `INCL`/`CONS` decidability
are `P.incl_computable`/`P.cons_computable`; `MEET` decidability is `meet_iff_interEq`
     comp
`P.interEq_computable`. -/
def ofPresentation (P : ComputablePresentation V) : InclStructure where
  INCL n m := P.X n  subseteq  P.X m
  meetIdx := P.inter
  topIdx := P.masterIdx
  incl_dec := P.incl_computable
  cons_dec :=
    RecDecidable.of_iff
      (fun t => by
        refine exists_congr fun k => ?_
        exact (Set.subset_inter_iff).symm)
      P.cons_computable
  meet_dec :=
    RecDecidable.of_iff
      (fun t => meet_iff_interEq P t.unpair.1 t.unpair.2.unpair.1 t.unpair.2.unpair.2)
      P.interEq_computable
  meetIdx_primrec := P.inter_primrec
  incl_refl := fun _ => subset_rfl
  incl_trans := fun h1 h2 => h1.trans h2
  topIdx_spec := by
    intro n
    show P.X n  subseteq  P.X P.masterIdx
    rw [P.masterIdx_spec]
    exact V.sub_master (P.mem_X n)
  meetIdx_spec := by
    intro n m hcons j
    obtain <k, h1, h2> := hcons
    have hcons' : exists  k, P.X k  subseteq  P.X n  I  P.X m := <k, Set.subset_inter
    h1 h2>
    show P.X j  subseteq  P.X (P.inter n m)  <->  _
    rw [P.inter_spec hcons', Set.subset_inter_iff]

@[simp] theorem ofPresentation_INCL (P : ComputablePresentation V) (n m : Nat) :
    (ofPresentation P).INCL n m  <->  P.X n  subseteq  P.X m := Iff.rfl
```

```
@[simp] theorem mem_ofPresentation_toNbhd (P : ComputablePresentation V) (j n : Nat) :
    j  in  (ofPresentation P).toNbhd n  <->  P.X j  subseteq  P.X n := Iff.rfl

/-- For `S = ofPresentation P`, two hint neighbourhoods coincide iff the underlying
    neighbourhoods
do: `S_n = S?  <->  X_n = X?`. -/
theorem ofPresentation_toNbhd_eq_iff (P : ComputablePresentation V) (n m : Nat) :
    (ofPresentation P).toNbhd n = (ofPresentation P).toNbhd m  <->  P.X n = P.X m :=
    by
  rw [Set.Subset.antisymm_iff, (ofPresentation P).toNbhd_subset_iff,
    (ofPresentation P).toNbhd_subset_iff]
  show (P.X n  subseteq  P.X m  /\  P.X m  subseteq  P.X n)  <->  P.X n = P.X m
  exact (Set.Subset.antisymm_iff).symm

/-- The reconstructed system `toSystem (ofPresentation P)` over tokens `Nat`. -/
abbrev reconstruct (P : ComputablePresentation V) : NeighborhoodSystem Nat :=
  (ofPresentation P).toSystem

/-- The element of the reconstructed system corresponding to `x  in  |V|`: the filter
`{S_n ? X_n  in  x}`. -/
def reconElem (P : ComputablePresentation V) (x : V.Element) : (reconstruct P).Element
    where
  mem Y := exists  n, Y = (ofPresentation P).toNbhd n  /\  x.mem (P.X n)
  sub := by rintro Y <n, rfl, _>; exact <n, rfl>
  master_mem :=
    <P.masterIdx, (ofPresentation P).toNbhd_top.symm, by rw [P.masterIdx_spec]; exact
    x.master_mem>
  inter_mem := by
    rintro X Y <n, rfl, hxn> <m, rfl, hxm>
    have hmem : V.mem (P.X n  I  P.X m) := x.sub (x.inter_mem hxn hxm)
    obtain <q, hq> := P.surj hmem
    refine <q, ?_, ?_>
    * ext j
      rw [Set.mem_inter_iff, mem_ofPresentation_toNbhd, mem_ofPresentation_toNbhd,
        mem_ofPresentation_toNbhd, hq, Set.subset_inter_iff]
    * have : x.mem (P.X q) := by rw [hq]; exact x.inter_mem hxn hxm
      exact this
  up_mem := by
    rintro X Y <n, rfl, hxn> <m, rfl> hsub
    have hnm : P.X n  subseteq  P.X m := (ofPresentation P).toNbhd_subset_iff.mp hsub
    exact <m, rfl, x.up_mem hxn (P.mem_X m) hnm>

/-- The element of `|V|` corresponding to `y` in the reconstructed system: the filter
`{X_n ? S_n  in  y}`. -/
def reconElemInv (P : ComputablePresentation V) (y : (reconstruct P).Element) : V.
    Element where
  mem W := exists  n, W = P.X n  /\  y.mem ((ofPresentation P).toNbhd n)
  sub := by rintro W <n, rfl, _>; exact P.mem_X n
  master_mem :=
    <P.masterIdx, P.masterIdx_spec.symm, by
      rw [show (ofPresentation P).toNbhd P.masterIdx = Set.univ from (ofPresentation P
    ).toNbhd_top]
      exact y.master_mem>
  inter_mem := by
    rintro X Y <n, rfl, hyn> <m, rfl, hym>
```

```
    obtain <p, hp> := y.sub (y.inter_mem hyn hym)
    have hpmem : p  in  (ofPresentation P).toNbhd n  I  (ofPresentation P).toNbhd m :=
      hp  |>  ((ofPresentation P).mem_toNbhd.mpr ((ofPresentation P).incl_refl p))
    rw [Set.mem_inter_iff, mem_ofPresentation_toNbhd, mem_ofPresentation_toNbhd] at
    hpmem
    have hwit : P.X p  subseteq  P.X n  I  P.X m := Set.subset_inter hpmem.1 hpmem.2
    have hmem : V.mem (P.X n  I  P.X m) := V.inter_mem (P.mem_X n) (P.mem_X m) (P.
    mem_X p) hwit
    obtain <q, hq> := P.surj hmem
    refine <q, hq.symm, ?_>
    have htoNbhd : (ofPresentation P).toNbhd q
        = (ofPresentation P).toNbhd n  I  (ofPresentation P).toNbhd m := by
      ext j
      rw [Set.mem_inter_iff, mem_ofPresentation_toNbhd, mem_ofPresentation_toNbhd,
        mem_ofPresentation_toNbhd, hq, Set.subset_inter_iff]
    rw [htoNbhd]
    exact y.inter_mem hyn hym
  up_mem := by
    rintro W Z <n, rfl, hyn> hZ hWZ
    obtain <p, hp> := P.surj hZ
    refine <p, hp.symm, ?_>
    have hnp : (ofPresentation P).toNbhd n  subseteq  (ofPresentation P).toNbhd p :=
      (ofPresentation P).toNbhd_subset_iff.mpr (by show P.X n  subseteq  P.X p; rw [hp
    ]; exact hWZ)
    exact y.up_mem hyn <p, rfl> hnp

/-- **Round-trip B (the isomorphism).** `reconElem`/`reconElemInv` are mutually inverse
     and preserve
and reflect the approximation order, giving `|V| ?o |toSystem (ofPresentation P)|`. -/
def reconIso (P : ComputablePresentation V) : V.Element ?o (reconstruct P).Element
    where
  toFun := reconElem P
  invFun := reconElemInv P
  left_inv := by
    intro x
    apply Element.ext
    intro W
    constructor
    * rintro <n, rfl, m, hWm, hxm>
      rw [ofPresentation_toNbhd_eq_iff] at hWm
      rw [hWm]; exact hxm
    * intro hxW
      obtain <n, hn> := P.surj (x.sub hxW)
      exact <n, hn.symm, n, rfl, by rw [hn]; exact hxW>
  right_inv := by
    intro y
    apply Element.ext
    intro T
    constructor
    * rintro <n, rfl, m, hXm, hym>
      have : (ofPresentation P).toNbhd n = (ofPresentation P).toNbhd m :=
        (ofPresentation_toNbhd_eq_iff P n m).mpr hXm
      rw [this]; exact hym
    * intro hyT
      obtain <n, hn> := y.sub hyT
      exact <n, hn, n, rfl, by rw [ <-  hn]; exact hyT>
```

```
  map_rel_iff' := by
    intro x x'
    constructor
    * intro h W hxW
      obtain <n, hn> := P.surj (x.sub hxW)
      have hmem : (reconElem P x).mem ((ofPresentation P).toNbhd n) :=
        <n, rfl, by rw [hn]; exact hxW>
      obtain <m, hm, hx'm> := h ((ofPresentation P).toNbhd n) hmem
      rw [ofPresentation_toNbhd_eq_iff] at hm
      rw [ <-  hn, hm]; exact hx'm
    * intro h Y hY
      obtain <n, rfl, hxn> := hY
      exact <n, rfl, h (P.X n) hxn>

end

section
namespace InclStructure

variable (I : InclStructure)

/-- **Exercise 7.13 (?) -- the hint system is effectively given.** The enumeration is `
    S_n`; relation
7.1(i) `S_n  I  S? = S?` is `MEET` (`meet_dec`); relation 7.1(ii) consistency is `CONS`
     (`cons_dec`);
the intersection function is `meetIdx`; the master code is `topIdx`. -/
def toPresentation : ComputablePresentation I.toSystem where
  X := I.toNbhd
  mem_X n := <n, rfl>
  surj := by rintro Y <n, rfl>; exact <n, rfl>
  interEq_computable :=
    RecDecidable.of_iff
      (fun t => (I.toNbhd_inter_eq_iff t.unpair.1 t.unpair.2.unpair.1 t.unpair.2.
    unpair.2))
      I.meet_dec
  cons_computable :=
    RecDecidable.of_iff
      (fun t => by
        refine exists_congr fun k => ?_
        exact I.toNbhd_subset_inter_iff k t.unpair.1 t.unpair.2)
      I.cons_dec
  inter := I.meetIdx
  inter_primrec := I.meetIdx_primrec
  inter_spec := by
    intro n m hcons
    obtain <k, hk> := hcons
    rw [Set.subset_inter_iff] at hk
    have hcons' : I.CONS n m := <k, I.toNbhd_subset_iff.mp hk.1, I.toNbhd_subset_iff.
    mp hk.2>
    exact ((I.toNbhd_inter_eq_iff n m (I.meetIdx n m)).mpr (I.meetIdx_spec hcons')).
    symm
  masterIdx := I.topIdx
  masterIdx_spec := I.toNbhd_top

/-- **Axiom (i).** `INCL` is reflexive. -/
theorem axiom_i (n : Nat) : I.INCL n n := I.incl_refl n
```

```
/-- **Axiom (ii).** `INCL` is transitive. -/
theorem axiom_ii {n m k : Nat} (h1 : I.INCL n m) (h2 : I.INCL m k) : I.INCL n k := I.
    incl_trans h1 h2

/-- **Axiom (iii).** There is an `INCL`-greatest code. -/
theorem axiom_iii : exists  m, forall  n, I.INCL n m := <I.topIdx, I.topIdx_spec>

/-- **Axiom (iv).** Consistent pairs have a meet. -/
theorem axiom_iv (n m : Nat) (h : I.CONS n m) : exists  k, I.MEET n m k :=
  <I.meetIdx n m, I.meetIdx_spec h>

end InclStructure
end

section
variable {a : Type*} {V : NeighborhoodSystem a}

/-- **Round-trip A.** Starting from an abstract `InclStructure I`, building the hint
    system and its
presentation, then reading off the integer inclusion relation recovers `I.INCL` exactly
    . -/
theorem ofPresentation_toPresentation_INCL (I : InclStructure) (n m : Nat) :
    (ofPresentation I.toPresentation).INCL n m  <->  I.INCL n m := by
  show I.toNbhd n  subseteq  I.toNbhd m  <->  I.INCL n m
  exact I.toNbhd_subset_iff

/-- **Exercise 7.13 ( => , "essentially any effectively given system").** The system
    reconstructed from
an effectively given `V` via its `INCL` relation is domain-isomorphic to `V`. -/
theorem reconstruct_isomorphic (P : ComputablePresentation V) : reconstruct P  ~= ? V
    :=
  <(reconIso P).symm>

end

section
namespace InclStructure

variable (I : InclStructure)

/-- **Exercise 7.13 (?).** The hint system is effectively given. -/
theorem toSystem_isEffectivelyGiven : I.toSystem.IsEffectivelyGiven := <I.
    toPresentation>

end InclStructure
end

section
set_option linter.unusedVariables false

theorem example_713.{u_1} : And
  (forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
    (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V),
    @DomainTheory.NeighbourhoodSystems.Isomorphic Nat a
      (@DomainTheory.NeighbourhoodSystems.Exercise713.reconstruct a V P) V)
```

```
  (forall  (I : DomainTheory.NeighbourhoodSystems.Exercise713.InclStructure),
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven Nat I.
    toSystem) :=
  @And.intro
  (forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
    (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V),
    @DomainTheory.NeighbourhoodSystems.Isomorphic Nat a
      (@DomainTheory.NeighbourhoodSystems.Exercise713.reconstruct a V P) V)
  (forall  (I : DomainTheory.NeighbourhoodSystems.Exercise713.InclStructure),
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven Nat I.
    toSystem)
  (@DomainTheory.NeighbourhoodSystems.Exercise713.reconstruct_isomorphic)
  DomainTheory.NeighbourhoodSystems.Exercise713.InclStructure.
    toSystem_isEffectivelyGiven
end

end DomainTheory.NeighbourhoodSystems.Exercise713
end
```

3.8.16 Exercise 7.14 — primitive recursive witnesses

(For recursive-function theorists.) Prove the statements after definition 7.2 about the existence of primitive recursive functions for showing things recursively enumerable. (Recall that a non-empty set is r.e. iff it is the range of a primitive recursive function.) Show also that every computable element $y \in |E|$ can be written

$$y = \bigcup \{\uparrow Y_{t(i)} \mid i \in \mathbf{N}\},$$

where $t : \mathbf{N} \to \mathbf{N}$ is primitive recursive and where we may assume $Y_{t(i+1)} \subseteq Y_{t(i)}$ for all $i \in \mathbf{N}$.

Exercise_7_14.lean

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Definition_7_2
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Definition_7_9

@[expose] public section

section
/- concordance: source L4631 EXERCISE 7.14 primitive recursive witnesses -/
-- Exercise714/tFun.lean
namespace DomainTheory.NeighbourhoodSystems.Exercise714

open NeighborhoodSystem Domain.Recursive DomainTheory.NeighbourhoodSystems

variable {b : Type*}

/-- The **running-intersection enumeration** of a computable element: `tFun Q r 0 = r
    0` and
```

```
`tFun Q r (i+1) = Q.inter (tFun Q r i) (r (i+1))`. As `i` grows it lists the meet of
    the first `i+1`
members `Y_{r 0}, ..., Y_{r i}` of the element, giving a *decreasing* chain of
    neighbourhoods. -/
def tFun {W : NeighborhoodSystem b} (Q : ComputablePresentation W) (r : Nat  ->  Nat)
    (n : Nat) :
    Nat :=
  Nat.rec (motive := fun _ => Nat) (r 0) (fun y IH => Q.inter IH (r (y + 1))) n

end DomainTheory.NeighbourhoodSystems.Exercise714
end

section
namespace DomainTheory.NeighbourhoodSystems.Exercise714
open NeighborhoodSystem Domain.Recursive DomainTheory.NeighbourhoodSystems

section
/-- **A non-empty recursively enumerable set is the range of a primitive recursive
    function.** This
is the precise content of Scott's prose after Definition 7.2 (*"there is a primitive
    recursive `r`
with `y = {Y_{r(i)}}`"*). Given a witness `a  in  p` and the `{0,1}` decider `qc` of
    the relation `q`
underlying `p` (`p n  <->  exists  i, q <i,n>`), the function `r w := selectFn (isOne (
    qc w)) w.2 a` enumerates
`p`: on a code `w = <i,n>` with `q <i,n>` it returns `n`, otherwise it returns the
    fixed `a`. Its
range is exactly `p` (the fall-back `a` covers the codes that do not witness anything
    -- this is where
non-emptiness is used). -/
theorem repred_exists_primrec_range {p : Nat  ->  Prop} (hp : REPred p) {a : Nat} (ha
    : p a) :
    exists  r : Nat  ->  Nat, Nat.Primrec r  /\  forall  n, p n  <->  exists  i, r i =
     n := by
  obtain <q, <qc, hqcp, hqcs>, hqe> := hp
  refine <fun w => selectFn (isOne (qc w)) w.unpair.2 a,
    primrec_selectFn (primrec_isOne.comp hqcp) Nat.Primrec.right (Nat.Primrec.const a)
    , ?_>
  -- Every value of the enumerator lies in `p`.
  have hrw_mem : forall  w, p (selectFn (isOne (qc w)) w.unpair.2 a) := by
    intro w
    rcases (show isOne (qc w) = 0  \/  isOne (qc w) = 1 by
        have := isOne_le_one (qc w); omega) with h0 | h1
    * rw [h0, selectFn_zero]; exact ha
    * rw [h1, selectFn_one]
      have hqw : q w := (hqcs w).mpr ((isOne_eq_one_iff (qc w)).mp h1)
      rw [hqe w.unpair.2]
      exact <w.unpair.1, by rw [pair_unpair]; exact hqw>
  intro n
  constructor
  * intro hpn
    rw [hqe n] at hpn
    obtain <i, hi> := hpn
    refine <Nat.pair i n, ?_>
    show selectFn (isOne (qc (Nat.pair i n))) (Nat.pair i n).unpair.2 a = n
    have hqc1 : isOne (qc (Nat.pair i n)) = 1 := (isOne_eq_one_iff _).mpr ((hqcs _).mp
```

```
     hi)
    rw [hqc1, selectFn_one, unpair_pair_snd]
  * rintro <w, rfl>
    exact hrw_mem w

variable {b : Type*}

theorem tFun_zero {W : NeighborhoodSystem b} (Q : ComputablePresentation W) (r : Nat
    ->  Nat) :
    tFun Q r 0 = r 0 := rfl

private theorem tFun_succ {W : NeighborhoodSystem b} (Q : ComputablePresentation W) (r
     : Nat  ->  Nat)
    (i : Nat) : tFun Q r (i + 1) = Q.inter (tFun Q r i) (r (i + 1)) := rfl

/-- The running-intersection enumeration is primitive recursive whenever `r` is (
    genuine primitive
recursion with a counter-dependent step, via `Nat.Primrec.prec`). -/
private theorem primrec_tFun {W : NeighborhoodSystem b} (Q : ComputablePresentation W)
     {r : Nat  ->  Nat}
    (hr : Nat.Primrec r) : Nat.Primrec (tFun Q r) := by
  have hg : Nat.Primrec (fun w => Q.inter w.unpair.2.unpair.2 (r (w.unpair.2.unpair.1
    + 1))) := by
    have hIH : Nat.Primrec (fun w : Nat => w.unpair.2.unpair.2) :=
      Nat.Primrec.right.comp Nat.Primrec.right
    have hy1 : Nat.Primrec (fun w : Nat => r (w.unpair.2.unpair.1 + 1)) :=
      hr.comp (Nat.Primrec.succ.comp (Nat.Primrec.left.comp Nat.Primrec.right))
    exact (Q.inter_primrec.comp (hIH.pair hy1)).of_eq fun w => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  refine ((Nat.Primrec.prec (Nat.Primrec.const (r 0)) hg).comp
    ((Nat.Primrec.const 0).pair primrec_id)).of_eq (fun n => ?_)
  simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd, id_eq]
  rfl

/-- **Exercise 7.14, Half 2 (Scott 1981, PRG-19).** Every computable element `y  in
    |?|` can be
written `y = ? {Y_{t(i)} ? i  in  Nat}` with `t : Nat  ->  Nat` primitive recursive and
     the approximations
*decreasing*, `Y_{t(i+1)}  subseteq  Y_{t(i)}`.

The index set `{m ? Y?  in  y}` is r.e. (`ComputableElement`) and non-empty (it
    contains the master
`Delta`, by `y.master_mem`), so Half 1's `repred_exists_primrec_range` lists it as the
    range of a
primitive recursive `r_0`. Running intersections (`tFun Q r_0`) turn this into a
    decreasing chain whose
members are still in `y` (a filter is closed under ` I `) and still cofinal in `y` (
    each
`Y_{t(i)}  subseteq  Y_{r_0(i)}`), so the principal-filter union is unchanged. -/
theorem computableElement_eq_decreasing_iUnion_principal {W : NeighborhoodSystem b}
    (Q : ComputablePresentation W) {y : W.Element} (hy : ComputableElement Q y) :
    exists  t : Nat  ->  Nat, Nat.Primrec t  /\
      (forall  i, Q.X (t (i + 1))  subseteq  Q.X (t i))  /\
      (forall  Z, y.mem Z  <->  exists  i, (W.principal (Q.mem_X (t i))).mem Z) := by
  have hy' : REPred (fun m => y.mem (Q.X m)) := hy.out
  -- The index set is non-empty: it contains the master neighbourhood `Delta`.
```

```
  obtain <m_0, hm_0> := Q.surj W.master_mem
  have hwit : y.mem (Q.X m_0) := by rw [hm_0]; exact y.master_mem
  obtain <r_0, hr_0, hr0_spec> := repred_exists_primrec_range hy' hwit
  -- Each enumerated index gives a member of `y`.
  have hr0_mem : forall  i, y.mem (Q.X (r_0 i)) := fun i => (hr0_spec (r_0 i)).mpr <i,
     rfl>
  -- Consistency of a pair of members (witnessed by their meet, itself in `y`).
  have hcons : forall  a c : Nat, y.mem (Q.X a)  ->  y.mem (Q.X c)  ->
      exists  k, Q.X k  subseteq  Q.X a  I  Q.X c := by
    intro a c ha hc
    obtain <k, hk> := Q.surj (y.sub (y.inter_mem ha hc))
    exact <k, hk.subset>
  -- Every `tFun` value is a member of `y`.
  have ht_mem : forall  i, y.mem (Q.X (tFun Q r_0 i)) := by
    intro i
    induction i with
    | zero => rw [tFun_zero]; exact hr0_mem 0
    | succ k ih =>
      have heq : Q.X (tFun Q r_0 (k + 1)) = Q.X (tFun Q r_0 k)  I  Q.X (r_0 (k + 1))
    :=
        Q.inter_spec (hcons (tFun Q r_0 k) (r_0 (k + 1)) ih (hr0_mem (k + 1)))
      rw [heq]; exact y.inter_mem ih (hr0_mem (k + 1))
  -- The defining intersection equation for `tFun`.
  have ht_eq : forall  i, Q.X (tFun Q r_0 (i + 1)) = Q.X (tFun Q r_0 i)  I  Q.X (r_0 (
    i + 1)) := fun i =>
    Q.inter_spec (hcons (tFun Q r_0 i) (r_0 (i + 1)) (ht_mem i) (hr0_mem (i + 1)))
  -- Decreasing: the next approximation is contained in the current one.
  have ht_dec : forall  i, Q.X (tFun Q r_0 (i + 1))  subseteq  Q.X (tFun Q r_0 i) :=
    by
    intro i; rw [ht_eq i]; exact Set.inter_subset_left
  -- Each approximation is contained in the corresponding enumerated member.
  have ht_sub_r : forall  i, Q.X (tFun Q r_0 i)  subseteq  Q.X (r_0 i) := by
    intro i
    cases i with
    | zero => rw [tFun_zero]
    | succ k => rw [ht_eq k]; exact Set.inter_subset_right
  refine <tFun Q r_0, primrec_tFun Q hr_0, ht_dec, fun Z => ?_>
  constructor
  * intro hZ
    obtain <m, hm> := Q.surj (y.sub hZ)
    have hym : y.mem (Q.X m) := by rw [hm]; exact hZ
    obtain <i, hi> := (hr0_spec m).mp hym
    have hZeq : Q.X (r_0 i) = Z := by rw [hi]; exact hm
    exact <i, y.sub hZ, by rw [ <-  hZeq]; exact ht_sub_r i>
  * rintro <i, hWZ, hsub>
    exact y.up_mem (ht_mem i) hWZ hsub
end

section
set_option linter.unusedVariables false

theorem example_714.{u_1} : forall  {b : Type u_1} {W : DomainTheory.
    NeighbourhoodSystems.NeighborhoodSystem b}
  (Q : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b W)
  {y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element b W},
  @DomainTheory.NeighbourhoodSystems.ComputableElement b W Q y  ->
```

```
    exists  (t : Nat  ->  Nat),
      And (Nat.Primrec t)
        (And
          (forall  (i : Nat),
            @LE.le (Set b) (@Set.instLE b)
              (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b W Q
                (t
                  (@HAdd.hAdd Nat Nat Nat (@instHAdd Nat instAddNat) i
                    (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))))))
              (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b W Q (t i)
    ))
          (forall  (Z : Set b),
            Iff (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem b W
     y Z)
              (exists  (i : Nat),
                @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem b W
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.principal b W
                    (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b W Q
     (t i))
                    (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.mem_X b
     W Q (t i)))
                Z))) :=
  @DomainTheory.NeighbourhoodSystems.Exercise714.
    computableElement_eq_decreasing_iUnion_principal
end

section
/-- **The range of a primitive recursive function is recursively enumerable.** The
    relation
`r i = n` is recursively decidable (`RecDecidable.natEq`), and `exists  i, r i = n` is
    its projection, so
it is r.e. by definition of `REPred`. -/
theorem repred_range_primrec {r : Nat  ->  Nat} (hr : Nat.Primrec r) :
    REPred (fun n => exists  i, r i = n) := by
  refine <fun t => r t.unpair.1 = t.unpair.2,
    RecDecidable.natEq (hr.comp Nat.Primrec.left) Nat.Primrec.right, fun n => ?_>
  refine exists_congr (fun i => ?_)
  simp only [unpair_pair_fst, unpair_pair_snd]
end

section
/-- **A non-empty recursively enumerable relation is enumerated by a pair of primitive
     recursive
functions** (Scott's `f = {(X_{s(i)}, Y_{r(i)}) ? i  in  Nat}`). Apply `
    repred_exists_primrec_range` to
the `Nat.pair`-coded relation and split the resulting range function into its two
    coordinates. -/
theorem repred_2_exists_primrec_enum {p : Nat  ->  Nat  ->  Prop} (hp : REPred_2 p) {a
     b : Nat}
    (hab : p a b) :
    exists  s r : Nat  ->  Nat, Nat.Primrec s  /\  Nat.Primrec r  /\
      forall  n m, p n m  <->  exists  i, s i = n  /\  r i = m := by
  have hp' : REPred (fun t => p t.unpair.1 t.unpair.2) := hp
  have hwit : (fun t => p t.unpair.1 t.unpair.2) (Nat.pair a b) := by
    simp only [unpair_pair_fst, unpair_pair_snd]; exact hab
  obtain <pf, hpf, hspec> := repred_exists_primrec_range hp' hwit
```

```
  refine <fun i => (pf i).unpair.1, fun i => (pf i).unpair.2,
    Nat.Primrec.left.comp hpf, Nat.Primrec.right.comp hpf, fun n m => ?_>
  have h := hspec (Nat.pair n m)
  simp only [unpair_pair_fst, unpair_pair_snd] at h
  rw [h]
  constructor
  * rintro <i, hi>
    refine <i, ?_, ?_>
    * show (pf i).unpair.1 = n; rw [hi, unpair_pair_fst]
    * show (pf i).unpair.2 = m; rw [hi, unpair_pair_snd]
  * rintro <i, hs, hr>
    refine <i, ?_>
    change (pf i).unpair.1 = n at hs
    change (pf i).unpair.2 = m at hr
    rw [ <-  hs,  <-  hr]; exact (pair_unpair _).symm
end

end DomainTheory.NeighbourhoodSystems.Exercise714
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive

section
namespace NeighborhoodSystem

variable {a : Type*} (V : NeighborhoodSystem a)

/-- **Each product term `X  I  Y` is a `??`-neighbourhood.** By `upSet_inter` it is `(X
      I  Y)`; if
`{X, Y}` is consistent (`X  I  Y  in  ?`) this is the down-set of a neighbourhood (`
    PDmem_upSet`),
otherwise `(X  I  Y) = {}` (no `Z  in  ?` lies below `X  I  Y`), the empty union (`
    PDmem_empty`). The
`by_cases` on `V.mem (X  I  Y)` is the lone `Classical` step (deciding membership in an
     arbitrary
system). -/
theorem PDmem_upSet_inter {X Y : Set a} (hX : V.mem X) (hY : V.mem Y) :
    V.PDmem (V.upSet X  I  V.upSet Y) := by
  rw [V.upSet_inter]
  by_cases h : V.mem (X  I  Y)
  * exact V.PDmem_upSet h
  * have hempty : V.upSet (X  I  Y) = {} := by
      ext Z
      simp only [mem_upSet, Set.mem_empty_iff_false, iff_false, not_and]
      intro hZmem hZsub
      exact h (V.inter_mem hX hY hZmem hZsub)
    rw [hempty]; exact V.PDmem_empty

/-- Cons law for the finite union of down-sets: `?_{X  in  Y::L} X = Y  U  ?_{X  in  L}
     X`. -/
theorem upSetUnion_cons (Y : Set a) (L : List (Set a)) :
    (? X  in  (Y :: L), V.upSet X) = V.upSet Y  U  ? X  in  L, V.upSet X := by
  ext z
  rw [Set.mem_union, V.mem_PDunion, V.mem_PDunion]
  constructor
  * rintro <X, hX, hz>
    rcases List.mem_cons.mp hX with rfl | hX'
```

```
    * exact Or.inl hz
    * exact Or.inr <X, hX', hz>
  * rintro (hz | <X, hX, hz>)
    * exact <Y, List.mem_cons.mpr (Or.inl rfl), hz>
    * exact <X, List.mem_cons.mpr (Or.inr hX), hz>

/-- The finite union `?_{X  in  []} X` is empty. -/
theorem upSetUnion_nil : (? X  in  ([] : List (Set a)), V.upSet X) = {} := by
  ext z
  rw [V.mem_PDunion]
  simp only [List.not_mem_nil, false_and, exists_false, Set.mem_empty_iff_false]

/-- `X  I  (?_{Y  in  L} Y)  in  ??`: distribute ` I ` over the finite union and use `
    PDmem_upSet_inter`
term by term. -/
theorem PDmem_upSet_inter_biUnion {X : Set a} (hX : V.mem X) :
    forall  (L : List (Set a)), (forall  Y  in  L, V.mem Y)  ->  V.PDmem (V.upSet X  I
      ? Y  in  L, V.upSet Y) := by
  intro L
  induction L with
  | nil =>
    intro _
    rw [V.upSetUnion_nil, Set.inter_empty]
    exact V.PDmem_empty
  | cons Y L ih =>
    intro hL
    rw [V.upSetUnion_cons, Set.inter_union_distrib_left]
    exact V.PDmem_union (V.PDmem_upSet_inter hX (hL Y (List.mem_cons.mpr (Or.inl rfl))
    ))
      (ih (fun Z hZ => hL Z (List.mem_cons.mpr (Or.inr hZ))))

/-- `(?_{X  in  L_1} X)  I  W_2  in  ??` for any `??`-neighbourhood `W_2` (induction on
     `L_1`). -/
theorem PDmem_biUnion_inter :
    forall  (L_1 : List (Set a)), (forall  X  in  L_1, V.mem X)  ->  forall  {W_2 :
    Set (Set a)}, V.PDmem W_2  ->
      V.PDmem ((? X  in  L_1, V.upSet X)  I  W_2) := by
  intro L_1
  induction L_1 with
  | nil =>
    intro _ W_2 _
    rw [V.upSetUnion_nil, Set.empty_inter]
    exact V.PDmem_empty
  | cons X L ih =>
    intro hL W_2 hW_2
    rw [V.upSetUnion_cons, Set.union_inter_distrib_right]
    refine V.PDmem_union ?_ (ih (fun Z hZ => hL Z (List.mem_cons.mpr (Or.inr hZ)))
    hW_2)
    obtain <L_2, hL_2, rfl> := hW_2
    exact V.PDmem_upSet_inter_biUnion (hL X (List.mem_cons.mpr (Or.inl rfl))) L_2 hL_2

/-- **`??` is closed under binary intersection.** (The content of Part A's condition (
    ii).) -/
theorem PDmem_inter {W_1 W_2 : Set (Set a)} (h_1 : V.PDmem W_1) (h_2 : V.PDmem W_2) :
    V.PDmem (W_1  I  W_2) := by
  obtain <L_1, hL_1, rfl> := h_1
```

```
  exact V.PDmem_biUnion_inter L_1 hL_1 h_2

end NeighborhoodSystem
end
end DomainTheory.NeighbourhoodSystems
end
```

3.8.17 Exercise 7.15 — further constructs effective

Finish the proof of 7.4 and establish similar results for the constructs $(D_0 \otimes D_1)$, $(D_0 \oplus D_1)$ and D^∞. Take into account the various appropriate combinators.

Exercise_7_15.lean

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Theorem_7_4

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive Example61
open NeighborhoodSystem Domain.Recursive ApproximableMap
section
variable {a b ? : Type*}
open Exercise510

section Smash
variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
  (P_0 : ComputablePresentation V_0) (P_1 : ComputablePresentation V_1)
variable {P_0 P_1}
variable (P_0 P_1)

/- concordance: source L4643 EXERCISE 7.15 further constructs effective -/
/-- **Exercise 7.15 -- `?_0 ? ?_1` is effectively given (Scott's Definition 7.1).** A
    bare presentation
of the smash product. The enumeration is classical (re-routing improper products to
    the master); the
two relations (i),(ii) are choice-free recursively decidable, reducing to the
    components' relations
on the *effective* indices (relation (i)) and the components' consistency together
    with properness
tests (relation (ii)). -/
noncomputable def smashPresentation : ScottPresentation (smash V_0 V_1) where
  X := smashEnum P_0 P_1
  mem_X := smashEnum_mem
  surj := by
    rintro Z (rfl | <X, Y, hX, hXne, hY, hYne, rfl>)
    * refine <Nat.pair P_0.masterIdx P_1.masterIdx, ?_>
      rw [smashEnum_master (by
        rw [unpair_pair_fst, P_0.masterIdx_spec]; exact fun h => h.1 rfl)]
    * obtain <n, rfl> := P_0.surj hX
      obtain <m, rfl> := P_1.surj hY
```

```
    refine <Nat.pair n m, ?_>
    rw [smashEnum_proper (by rw [unpair_pair_fst, unpair_pair_snd]; exact <hXne,
  hYne>),
      unpair_pair_fst, unpair_pair_snd]
interEq_computable := by
  obtain <f_0, hf_0p, hf_0e> := proper_0_dec P_0
  obtain <f_1, hf_1p, hf_1e> := proper_1_dec P_1
  set c_0 := fun n => isOne (f_0 n) with hc_0def
  set c_1 := fun n => isOne (f_1 n) with hc_1def
  have hc_0le : forall  n, c_0 n  <=  1 := fun n => isOne_le_one _
  have hc_1le : forall  n, c_1 n  <=  1 := fun n => isOne_le_one _
  have hc_0 : forall  n, P_0.X n  !=  V_0.master  <->  c_0 n = 1 := fun n => by
    rw [hc_0def, isOne_eq_one_iff]; exact hf_0e n
  have hc_1 : forall  n, P_1.X n  !=  V_1.master  <->  c_1 n = 1 := fun n => by
    rw [hc_1def, isOne_eq_one_iff]; exact hf_1e n
  have hc_0p : Nat.Primrec c_0 := primrec_isOne.comp hf_0p
  have hc_1p : Nat.Primrec c_1 := primrec_isOne.comp hf_1p
  have hpf : Nat.Primrec (fun n => c_0 n.unpair.1 * c_1 n.unpair.2) :=
    (primrec_mul.comp ((hc_0p.comp Nat.Primrec.left).pair (hc_1p.comp Nat.Primrec.
  right))).of_eq
      (fun n => by simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd])
  -- effective left/right index of a single code `n`
  have heffLp : Nat.Primrec (fun n => selectFn (c_0 n.unpair.1 * c_1 n.unpair.2)
      n.unpair.1 P_0.masterIdx) :=
    primrec_selectFn hpf Nat.Primrec.left (Nat.Primrec.const _)
  have heffRp : Nat.Primrec (fun n => selectFn (c_0 n.unpair.1 * c_1 n.unpair.2)
      n.unpair.2 P_1.masterIdx) :=
    primrec_selectFn hpf Nat.Primrec.right (Nat.Primrec.const _)
  -- reindexings `t  |->  pair (effL t.1) (pair (effL t.2.1) (effL t.2.2))`, ditto
  for the right factor
  have hgL : Nat.Primrec (fun t => Nat.pair
      (selectFn (c_0 t.unpair.1.unpair.1 * c_1 t.unpair.1.unpair.2) t.unpair.1.
  unpair.1 P_0.masterIdx)
      (Nat.pair
        (selectFn (c_0 t.unpair.2.unpair.1.unpair.1 * c_1 t.unpair.2.unpair.1.unpair
  .2)
          t.unpair.2.unpair.1.unpair.1 P_0.masterIdx)
        (selectFn (c_0 t.unpair.2.unpair.2.unpair.1 * c_1 t.unpair.2.unpair.2.unpair
  .2)
          t.unpair.2.unpair.2.unpair.1 P_0.masterIdx))) :=
    (heffLp.comp Nat.Primrec.left).pair
      ((heffLp.comp (Nat.Primrec.left.comp Nat.Primrec.right)).pair
        (heffLp.comp (Nat.Primrec.right.comp Nat.Primrec.right)))
  have hgR : Nat.Primrec (fun t => Nat.pair
      (selectFn (c_0 t.unpair.1.unpair.1 * c_1 t.unpair.1.unpair.2) t.unpair.1.
  unpair.2 P_1.masterIdx)
      (Nat.pair
        (selectFn (c_0 t.unpair.2.unpair.1.unpair.1 * c_1 t.unpair.2.unpair.1.unpair
  .2)
          t.unpair.2.unpair.1.unpair.2 P_1.masterIdx)
        (selectFn (c_0 t.unpair.2.unpair.2.unpair.1 * c_1 t.unpair.2.unpair.2.unpair
  .2)
          t.unpair.2.unpair.2.unpair.2 P_1.masterIdx))) :=
    (heffRp.comp Nat.Primrec.left).pair
      ((heffRp.comp (Nat.Primrec.left.comp Nat.Primrec.right)).pair
        (heffRp.comp (Nat.Primrec.right.comp Nat.Primrec.right)))
```

```
  refine RecDecidable.of_iff (fun t => ?_)
    ((P_0.interEq_computable.comp hgL).and (P_1.interEq_computable.comp hgR))
  dsimp only
  simp only [unpair_pair_fst, unpair_pair_snd]
  rw [smashEnum_eq_eff hc_0le hc_1le hc_0 hc_1, smashEnum_eq_eff hc_0le hc_1le hc_0
  hc_1,
    smashEnum_eq_eff hc_0le hc_1le hc_0 hc_1, prodNbhd_inter, prodNbhd_eq_iff]
cons_computable := by
  obtain <f_0, hf_0p, hf_0e> := proper_0_dec P_0
  obtain <f_1, hf_1p, hf_1e> := proper_1_dec P_1
  -- `cons(a,b)  <->  not proper a  \/  not proper b  \/  (P_0.cons(a.1,b.1)  /\  P_1
  .cons(a.2,b.2))`
  have hpa : RecDecidable (fun t => not (P_0.X t.unpair.1.unpair.1  !=  V_0.master
       /\  P_1.X t.unpair.1.unpair.2  !=  V_1.master)) := by
    have h0 : RecDecidable (fun t => P_0.X t.unpair.1.unpair.1  !=  V_0.master) :=
      (proper_0_dec P_0).comp (Nat.Primrec.left.comp Nat.Primrec.left)
    have h1 : RecDecidable (fun t => P_1.X t.unpair.1.unpair.2  !=  V_1.master) :=
      (proper_1_dec P_1).comp (Nat.Primrec.right.comp Nat.Primrec.left)
    exact (h0.and h1).not
  have hpb : RecDecidable (fun t => not (P_0.X t.unpair.2.unpair.1  !=  V_0.master
       /\  P_1.X t.unpair.2.unpair.2  !=  V_1.master)) := by
    have h0 : RecDecidable (fun t => P_0.X t.unpair.2.unpair.1  !=  V_0.master) :=
      (proper_0_dec P_0).comp (Nat.Primrec.left.comp Nat.Primrec.right)
    have h1 : RecDecidable (fun t => P_1.X t.unpair.2.unpair.2  !=  V_1.master) :=
      (proper_1_dec P_1).comp (Nat.Primrec.right.comp Nat.Primrec.right)
    exact (h0.and h1).not
  have hcons0 : RecDecidable (fun t => exists  k, P_0.X k  subseteq  P_0.X t.unpair
  .1.unpair.1
       I  P_0.X t.unpair.2.unpair.1) := by
    refine RecDecidable.of_iff (fun t => ?_) (P_0.cons_computable.comp
      ((Nat.Primrec.left.comp Nat.Primrec.left).pair (Nat.Primrec.left.comp Nat.
  Primrec.right)))
    simp only [unpair_pair_fst, unpair_pair_snd]
  have hcons1 : RecDecidable (fun t => exists  k, P_1.X k  subseteq  P_1.X t.unpair
  .1.unpair.2
       I  P_1.X t.unpair.2.unpair.2) := by
    refine RecDecidable.of_iff (fun t => ?_) (P_1.cons_computable.comp
      ((Nat.Primrec.right.comp Nat.Primrec.left).pair (Nat.Primrec.right.comp Nat.
  Primrec.right)))
    simp only [unpair_pair_fst, unpair_pair_snd]
  refine RecDecidable.of_iff (fun t => ?_) (hpa.or (hpb.or (hcons0.and hcons1)))
  constructor
  * rintro <k, hk>
    by_cases hpa' : P_0.X t.unpair.1.unpair.1  !=  V_0.master  /\  P_1.X t.unpair.1.
  unpair.2  !=  V_1.master
    * by_cases hpb' : P_0.X t.unpair.2.unpair.1  !=  V_0.master  /\  P_1.X t.unpair
  .2.unpair.2  !=  V_1.master
      * refine Or.inr (Or.inr <?_, ?_>)
        * rw [smashEnum_proper hpa', smashEnum_proper hpb', prodNbhd_inter] at hk
          obtain <A, B, hA, _, hkeq> := smashEnum_eq_prodNbhd_mem (P_0 := P_0) (P_1
  := P_1) k
          rw [hkeq] at hk
          obtain <hAsub, _> := prodNbhd_subset_iff.mp hk
          obtain <j_0, hj_0> := P_0.surj hA
          exact <j_0, by rw [hj_0]; exact hAsub>
        * rw [smashEnum_proper hpa', smashEnum_proper hpb', prodNbhd_inter] at hk
```

```
            obtain <A, B, _, hB, hkeq> := smashEnum_eq_prodNbhd_mem (P_0 := P_0) (P_1
    := P_1) k
            rw [hkeq] at hk
            obtain <_, hBsub> := prodNbhd_subset_iff.mp hk
            obtain <j_1, hj_1> := P_1.surj hB
            exact <j_1, by rw [hj_1]; exact hBsub>
        * exact Or.inr (Or.inl hpb')
      * exact Or.inl hpa'
    * intro hp
      by_cases hpa' : P_0.X t.unpair.1.unpair.1  !=  V_0.master  /\  P_1.X t.unpair.1.
    unpair.2  !=  V_1.master
      * by_cases hpb' : P_0.X t.unpair.2.unpair.1  !=  V_0.master  /\  P_1.X t.unpair
    .2.unpair.2  !=  V_1.master
        * rcases hp with h | h | <<j_0, hj_0>, <j_1, hj_1>>
          * exact absurd hpa' h
          * exact absurd hpb' h
          * have hXint : V_0.mem (P_0.X t.unpair.1.unpair.1  I  P_0.X t.unpair.2.
    unpair.1) :=
              V_0.inter_mem (P_0.mem_X _) (P_0.mem_X _) (P_0.mem_X j_0) hj_0
            have hYint : V_1.mem (P_1.X t.unpair.1.unpair.2  I  P_1.X t.unpair.2.
    unpair.2) :=
              V_1.inter_mem (P_1.mem_X _) (P_1.mem_X _) (P_1.mem_X j_1) hj_1
            have hXne : P_0.X t.unpair.1.unpair.1  I  P_0.X t.unpair.2.unpair.1  !=
    V_0.master :=
              inter_ne_master_left (P_0.mem_X _) hpa'.1
            have hYne : P_1.X t.unpair.1.unpair.2  I  P_1.X t.unpair.2.unpair.2  !=
    V_1.master :=
              inter_ne_master_left (P_1.mem_X _) hpa'.2
            obtain <p_0, hp_0> := P_0.surj hXint
            obtain <p_1, hp_1> := P_1.surj hYint
            refine <Nat.pair p_0 p_1, ?_>
            rw [smashEnum_proper (by
                  rw [unpair_pair_fst, unpair_pair_snd, hp_0, hp_1]; exact <hXne, hYne
    >),
              unpair_pair_fst, unpair_pair_snd, hp_0, hp_1, smashEnum_proper hpa',
              smashEnum_proper hpb', prodNbhd_inter]
        * refine <t.unpair.1, ?_>
          rw [smashEnum_master (t := t.unpair.2) hpb',
            Set.inter_eq_left.mpr (smashEnum_subset_master t.unpair.1)]
      * refine <t.unpair.2, ?_>
        rw [smashEnum_master (t := t.unpair.1) hpa',
          Set.inter_eq_right.mpr (smashEnum_subset_master t.unpair.2)]

@[simp] theorem smashPresentation_X (t : Nat) :
    (smashPresentation P_0 P_1).X t = smashEnum P_0 P_1 t := rfl

/-- **The smash projection `p_0 : ?_0 ? ?_1  ->  ?_0`**, `(X  U  Y) p_0 X'  <->  X
    subseteq  X'`. -/
def smashProj_0 (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) :
    ApproximableMap (smash V_0 V_1) V_0 where
  rel W X' := (smash V_0 V_1).mem W  /\  V_0.mem X'  /\  Sum.inl ^-1' W  subseteq  X'
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := <(smash V_0 V_1).master_mem, V_0.master_mem, by simp>
  inter_right := by
    rintro W X' X'' <hW, hX', hsub> <-, hX'', hsub'>
```

```
    obtain <A, B, hA, _, rfl> := smash_mem_prodNbhd_form hW
    rw [inl_preimage_prodNbhd] at hsub hsub'  |-
    exact <hW, V_0.inter_mem hX' hX'' hA (Set.subset_inter hsub hsub'), Set.
    subset_inter hsub hsub'>
  mono := by
    rintro W W_2 X' X_2' <_, _, hsub> hW_2W hX'X_2' hW_2 hX_2'
    exact <hW_2, hX_2', ((Set.preimage_mono hW_2W).trans hsub).trans hX'X_2'>

@[simp] theorem smashProj_0_rel {W : Set (a ? b)} {X' : Set a} :
    (smashProj_0 V_0 V_1).rel W X'  <->  (smash V_0 V_1).mem W  /\  V_0.mem X'  /\
    Sum.inl ^-1' W  subseteq  X' := Iff.rfl

/-- **The smash projection `p_1 : ?_0 ? ?_1  ->  ?_1`**, `(X  U  Y) p_1 Y'  <->  Y
    subseteq  Y'`. -/
def smashProj_1 (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b) :
    ApproximableMap (smash V_0 V_1) V_1 where
  rel W Y' := (smash V_0 V_1).mem W  /\  V_1.mem Y'  /\  Sum.inr ^-1' W  subseteq  Y'
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := <(smash V_0 V_1).master_mem, V_1.master_mem, by simp>
  inter_right := by
    rintro W Y' Y'' <hW, hY', hsub> <-, hY'', hsub'>
    obtain <A, B, _, hB, rfl> := smash_mem_prodNbhd_form hW
    rw [inr_preimage_prodNbhd] at hsub hsub'  |-
    exact <hW, V_1.inter_mem hY' hY'' hB (Set.subset_inter hsub hsub'), Set.
    subset_inter hsub hsub'>
  mono := by
    rintro W W_2 Y' Y_2' <_, _, hsub> hW_2W hY'Y_2' hW_2 hY_2'
    exact <hW_2, hY_2', ((Set.preimage_mono hW_2W).trans hsub).trans hY'Y_2'>

@[simp] theorem smashProj_1_rel {W : Set (a ? b)} {Y' : Set b} :
    (smashProj_1 V_0 V_1).rel W Y'  <->  (smash V_0 V_1).mem W  /\  V_1.mem Y'  /\
    Sum.inr ^-1' W  subseteq  Y' := Iff.rfl

/-- **The strict pairing `<a, b>? : ?_2  ->  ?_0 ? ?_1`.** Like Theorem 7.4's `paired`,
     but the smash
collapse forces the image factors to be proper (else the result is the master/bottom
    of the smash). -/
def smashPaired {V_2 : NeighborhoodSystem ?} (a : ApproximableMap V_2 V_0)
    (b : ApproximableMap V_2 V_1) : ApproximableMap V_2 (smash V_0 V_1) where
  rel Z W := V_2.mem Z  /\  (smash V_0 V_1).mem W  /\
    (W = prodNbhd V_0.master V_1.master  \/
      exists  A B, W = prodNbhd A B  /\  A  !=  V_0.master  /\  B  !=  V_1.master  /\
     a.rel Z A  /\  b.rel Z B)
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := <V_2.master_mem, (smash V_0 V_1).master_mem, Or.inl rfl>
  inter_right := by
    rintro Z W_1 W_2 <hZ, hW_1, hd_1> <-, hW_2, hd_2>
    have key : W_1  I  W_2 = prodNbhd V_0.master V_1.master  \/
        exists  A B, W_1  I  W_2 = prodNbhd A B  /\  A  !=  V_0.master  /\  B  !=  V_1
    .master  /\
          a.rel Z A  /\  b.rel Z B := by
      rcases hd_1 with rfl | <A_1, B_1, rfl, hA_1, hB_1, ha_1, hb_1>
      * rw [Set.inter_eq_right.mpr (show W_2  subseteq  prodNbhd V_0.master V_1.master
     from
```

```
          (smash V_0 V_1).sub_master hW_2)]
        exact hd_2
      * rcases hd_2 with rfl | <A_2, B_2, rfl, hA_2, hB_2, ha_2, hb_2>
        * rw [Set.inter_eq_left.mpr (show prodNbhd A_1 B_1  subseteq  prodNbhd V_0.
    master V_1.master from
            (smash V_0 V_1).sub_master hW_1)]
          exact Or.inr <A_1, B_1, rfl, hA_1, hB_1, ha_1, hb_1>
        * rw [prodNbhd_inter]
          exact Or.inr <A_1  I  A_2, B_1  I  B_2, rfl,
            inter_ne_master_left (a.rel_cod ha_1) hA_1,
            inter_ne_master_left (b.rel_cod hb_1) hB_1,
            a.inter_right ha_1 ha_2, b.inter_right hb_1 hb_2>
    refine <hZ, ?_, key>
    rcases key with hm | <A, B, hWW, hA, hB, ha, hb>
    * rw [hm]; exact (smash V_0 V_1).master_mem
    * rw [hWW]; exact smash_mem_proper (a.rel_cod ha) hA (b.rel_cod hb) hB
  mono := by
    rintro Z Z_2 W W_2 <_, _, hd> hZ_2Z hWW_2 hZ_2mem hW_2mem
    refine <hZ_2mem, hW_2mem, ?_>
    rcases hd with rfl | <A, B, rfl, hA, hB, ha, hb>
    * left; exact Set.Subset.antisymm ((smash V_0 V_1).sub_master hW_2mem) hWW_2
    * rcases smash_mem_iff.mp hW_2mem with rfl | <A_2, B_2, hA_2, hA_2ne, hB_2, hB_2ne
    , rfl>
      * left; rfl
      * obtain <hAA_2, hBB_2> := prodNbhd_subset_iff.mp hWW_2
        exact Or.inr <A_2, B_2, rfl, hA_2ne, hB_2ne,
          a.mono ha hZ_2Z hAA_2 hZ_2mem hA_2, b.mono hb hZ_2Z hBB_2 hZ_2mem hB_2>

@[simp] theorem smashPaired_rel {V_2 : NeighborhoodSystem ?} {a : ApproximableMap V_2
    V_0}
    {b : ApproximableMap V_2 V_1} {Z : Set ?} {W : Set (a ? b)} :
    (smashPaired a b).rel Z W  <->  V_2.mem Z  /\  (smash V_0 V_1).mem W  /\
      (W = prodNbhd V_0.master V_1.master  \/
        exists  A B, W = prodNbhd A B  /\  A  !=  V_0.master  /\  B  !=  V_1.master
    /\  a.rel Z A  /\  b.rel Z B) :=
  Iff.rfl

end Smash

section Coalesced

variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
  (h_0 : forall  X, V_0.mem X  ->  X.Nonempty) (h_1 : forall  Y, V_1.mem Y  ->  Y.
    Nonempty)

/-- **Exercise 7.15 -- the coalesced sum system `?_0 ? ?_1`** over `Option (a ? b)`:
    the separated sum
with the improper copies `0Delta_0`, `1Delta_1` deleted. (Generalises `Exercise621.
    oplusTok` to an arbitrary
carrier.) -/
def osum : NeighborhoodSystem (Option (a ? b)) where
  mem W := W = sumMaster V_0 V_1  \/  (exists  X, V_0.mem X  /\  X  !=  V_0.master  /\
      W = inj_0 X)  \/
    (exists  Y, V_1.mem Y  /\  Y  !=  V_1.master  /\  W = inj_1 Y)
  master := sumMaster V_0 V_1
  master_nonempty := <none, none_mem_sumMaster>
```

```
  master_mem := Or.inl rfl
  sub_master := by
    rintro W (rfl | <X, hX, -, rfl> | <Y, hY, -, rfl>)
    * exact subset_rfl
    * exact inj_0_subset_sumMaster hX
    * exact inj_1_subset_sumMaster hY
  inter_mem := by
    have hne : forall  W, (W = sumMaster V_0 V_1  \/  (exists  X, V_0.mem X  /\  X  !=
      V_0.master  /\  W = inj_0 X)  \/
        (exists  Y, V_1.mem Y  /\  Y  !=  V_1.master  /\  W = inj_1 Y))  ->  (W : Set
    (Option (a ? b))).Nonempty := by
      rintro W (rfl | <X, hX, -, rfl> | <Y, hY, -, rfl>)
      * exact <none, none_mem_sumMaster>
      * exact inj_0_nonempty (h_0 X hX)
      * exact inj_1_nonempty (h_1 Y hY)
    rintro W W' Z hW hW' hZ hZsub
    rcases hW with rfl | <X, hX, hXne, rfl> | <Y, hY, hYne, rfl>
    * rcases hW' with rfl | <X', hX', hX'ne, rfl> | <Y', hY', hY'ne, rfl>
      * rw [Set.inter_self]; exact Or.inl rfl
      * rw [sumMaster_inter_inj_0 hX']; exact Or.inr (Or.inl <X', hX', hX'ne, rfl>)
      * rw [sumMaster_inter_inj_1 hY']; exact Or.inr (Or.inr <Y', hY', hY'ne, rfl>)
    * rcases hW' with rfl | <X', hX', hX'ne, rfl> | <Y', hY', hY'ne, rfl>
      * rw [Set.inter_comm, sumMaster_inter_inj_0 hX]; exact Or.inr (Or.inl <X, hX,
    hXne, rfl>)
      * rw [inj_0_inter] at hZsub  |-
        rcases hZ with rfl | <Z_0, hZ_0, -, rfl> | <Z_1, hZ_1, -, rfl>
        * exact absurd (hZsub none_mem_sumMaster) none_mem_inj_0
        * exact Or.inr (Or.inl <X  I  X', V_0.inter_mem hX hX' hZ_0 (
    inj_0_subset_inj_0.mp hZsub),
            inter_ne_master_left hX hXne, rfl>)
        * obtain <b, hb> := h_1 Z_1 hZ_1
          exact absurd (hZsub (ir_mem_inj_1.mpr hb)) ir_mem_inj_0
      * rw [inj_0_inter_inj_1] at hZsub
        obtain <t, ht> := hne Z hZ; exact absurd (hZsub ht) (Set.notMem_empty t)
    * rcases hW' with rfl | <X', hX', hX'ne, rfl> | <Y', hY', hY'ne, rfl>
      * rw [Set.inter_comm, sumMaster_inter_inj_1 hY]; exact Or.inr (Or.inr <Y, hY,
    hYne, rfl>)
      * rw [Set.inter_comm, inj_0_inter_inj_1] at hZsub
        obtain <t, ht> := hne Z hZ; exact absurd (hZsub ht) (Set.notMem_empty t)
      * rw [inj_1_inter] at hZsub  |-
        rcases hZ with rfl | <Z_0, hZ_0, -, rfl> | <Z_1, hZ_1, -, rfl>
        * exact absurd (hZsub none_mem_sumMaster) none_mem_inj_1
        * obtain <a, ha> := h_0 Z_0 hZ_0
          exact absurd (hZsub (il_mem_inj_0.mpr ha)) il_mem_inj_1
        * exact Or.inr (Or.inr <Y  I  Y', V_1.inter_mem hY hY' hZ_1 (
    inj_1_subset_inj_1.mp hZsub),
            inter_ne_master_right hY hYne, rfl>)

@[simp] theorem osum_master : (osum h_0 h_1).master = sumMaster V_0 V_1 := rfl

end Coalesced

section Coalesced

variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
  (h_0 : forall  X, V_0.mem X  ->  X.Nonempty) (h_1 : forall  Y, V_1.mem Y  ->  Y.
```

```
    Nonempty)

variable (P_0 : ComputablePresentation V_0) (P_1 : ComputablePresentation V_1)

open scoped Classical in
/-- **Coalesced-sum enumeration.** Tag `0`/`1`/` >= 2`; a tag-0/1 code is its proper
    left/right copy
when the indexed factor is proper, otherwise it is re-routed to the master (the
    classical input). -/
noncomputable def osumEnum (t : Nat) : Set (Option (a ? b)) :=
  if t.unpair.1 = 0  /\  P_0.X t.unpair.2  !=  V_0.master then inj_0 (P_0.X t.unpair
    .2)
  else if t.unpair.1 = 1  /\  P_1.X t.unpair.2  !=  V_1.master then inj_1 (P_1.X t.
    unpair.2)
  else sumMaster V_0 V_1

end Coalesced

section Coalesced

variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
  (h_0 : forall  X, V_0.mem X  ->  X.Nonempty) (h_1 : forall  Y, V_1.mem Y  ->  Y.
    Nonempty)

variable (P_0 : ComputablePresentation V_0) (P_1 : ComputablePresentation V_1)

variable {P_0 P_1}

open scoped Classical in
theorem osumEnum_left {t : Nat} (h : t.unpair.1 = 0  /\  P_0.X t.unpair.2  !=  V_0.
    master) :
    osumEnum P_0 P_1 t = inj_0 (P_0.X t.unpair.2) := ite_eq_left h

open scoped Classical in
theorem osumEnum_master {t : Nat}
    (h0 : not (t.unpair.1 = 0  /\  P_0.X t.unpair.2  !=  V_0.master))
    (h1 : not (t.unpair.1 = 1  /\  P_1.X t.unpair.2  !=  V_1.master)) :
    osumEnum P_0 P_1 t = sumMaster V_0 V_1 := by
  unfold osumEnum; rw [ite_eq_right h0, ite_eq_right h1]

open scoped Classical in
theorem osumEnum_right {t : Nat} (h : t.unpair.1 = 1  /\  P_1.X t.unpair.2  !=  V_1.
    master) :
    osumEnum P_0 P_1 t = inj_1 (P_1.X t.unpair.2) := by
  have h0 : not (t.unpair.1 = 0  /\  P_0.X t.unpair.2  !=  V_0.master) := by
    rintro <hc, ->; obtain <h1, -> := h; omega
  unfold osumEnum; rw [ite_eq_right h0, ite_eq_left h]

theorem osumEnum_mem (t : Nat) : (osum h_0 h_1).mem (osumEnum P_0 P_1 t) := by
  by_cases hL : t.unpair.1 = 0  /\  P_0.X t.unpair.2  !=  V_0.master
  * rw [osumEnum_left hL]; exact Or.inr (Or.inl <_, P_0.mem_X _, hL.2, rfl>)
  * by_cases hR : t.unpair.1 = 1  /\  P_1.X t.unpair.2  !=  V_1.master
    * rw [osumEnum_right hR]; exact Or.inr (Or.inr <_, P_1.mem_X _, hR.2, rfl>)
    * rw [osumEnum_master hL hR]; exact Or.inl rfl

include h_0 h_1 in
```

```
theorem osumEnum_nonempty (t : Nat) : (osumEnum P_0 P_1 t).Nonempty := by
  by_cases hL : t.unpair.1 = 0  /\  P_0.X t.unpair.2  !=  V_0.master
  * rw [osumEnum_left hL]; exact inj_0_nonempty (h_0 _ (P_0.mem_X _))
  * by_cases hR : t.unpair.1 = 1  /\  P_1.X t.unpair.2  !=  V_1.master
    * rw [osumEnum_right hR]; exact inj_1_nonempty (h_1 _ (P_1.mem_X _))
    * rw [osumEnum_master hL hR]; exact <none, none_mem_sumMaster>

include h_0 h_1 in
theorem osumEnum_subset_master (t : Nat) : osumEnum P_0 P_1 t  subseteq  sumMaster V_0
     V_1 :=
  (osum h_0 h_1).sub_master (osumEnum_mem h_0 h_1 t)

end Coalesced

section Coalesced

variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
  (h_0 : forall  X, V_0.mem X  ->  X.Nonempty) (h_1 : forall  Y, V_1.mem Y  ->  Y.
    Nonempty)

variable (P_0 : ComputablePresentation V_0) (P_1 : ComputablePresentation V_1)

variable {P_0 P_1}

variable {h_0 h_1}

/-- **Coalesced left projection `out_0 : ?_0 ? ?_1  ->  ?_0`**, `W (out_0) X  <->
    leftPart W  subseteq  X`. -/
def osumOutMap_0 : ApproximableMap (osum h_0 h_1) V_0 where
  rel W X := (osum h_0 h_1).mem W  /\  V_0.mem X  /\  leftPart V_0 W  subseteq  X
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := <(osum h_0 h_1).master_mem, V_0.master_mem, (leftPart_sumMaster V_0
    V_1).subset>
  inter_right := by

    rintro W X X' <hW, hX, hsub> <-, hX', hsub'>
    refine <hW, V_0.inter_mem hX hX' ?_ (Set.subset_inter hsub hsub'), Set.
    subset_inter hsub hsub'>
    rcases hW with rfl | <A, hA, -, rfl> | <B, hB, -, rfl>
    * rw [leftPart_sumMaster]; exact V_0.master_mem
    * rw [leftPart_inj_0]; exact hA
    * rw [leftPart_inj_1 V_0 (h_1 B hB)]; exact V_0.master_mem
  mono := by
    rintro W W' X X' <_, _, hsub> hW'W hXX' hW' hX'
    exact <hW', hX', (leftPart_mono V_0 hW'W).trans (hsub.trans hXX')>

/-- **Coalesced right projection `out_1 : ?_0 ? ?_1  ->  ?_1`**. -/
def osumOutMap_1 : ApproximableMap (osum h_0 h_1) V_1 where
  rel W Y := (osum h_0 h_1).mem W  /\  V_1.mem Y  /\  rightPart V_1 W  subseteq  Y
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := <(osum h_0 h_1).master_mem, V_1.master_mem, (rightPart_sumMaster V_0
    V_1).subset>
  inter_right := by
    rintro W Y Y' <hW, hY, hsub> <-, hY', hsub'>
```

```
    refine <hW, V_1.inter_mem hY hY' ?_ (Set.subset_inter hsub hsub'), Set.
    subset_inter hsub hsub'>
    rcases hW with rfl | <A, hA, -, rfl> | <B, hB, -, rfl>
    * rw [rightPart_sumMaster]; exact V_1.master_mem
    * rw [rightPart_inj_0 V_1 (h_0 A hA)]; exact V_1.master_mem
    * rw [rightPart_inj_1]; exact hB
  mono := by
    rintro W W' Y Y' <_, _, hsub> hW'W hYY' hW' hY'
    exact <hW', hY', (rightPart_mono V_1 hW'W).trans (hsub.trans hYY')>

end Coalesced

section Coalesced

variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
  (h_0 : forall  X, V_0.mem X  ->  X.Nonempty) (h_1 : forall  Y, V_1.mem Y  ->  Y.
    Nonempty)

variable (P_0 : ComputablePresentation V_0) (P_1 : ComputablePresentation V_1)

variable {P_0 P_1}

/-- **Exercise 7.15 -- `?_0 ? ?_1` is effectively given (Scott's Definition 7.1).** -/
noncomputable def osumPresentation : ScottPresentation (osum h_0 h_1) where
  X := osumEnum P_0 P_1
  mem_X := osumEnum_mem h_0 h_1
  surj := by
    rintro W (rfl | <X, hX, hXne, rfl> | <Y, hY, hYne, rfl>)
    * refine <Nat.pair 2 0, ?_>
      rw [osumEnum_master (by rw [unpair_pair_fst]; rintro <h, ->; omega)
        (by rw [unpair_pair_fst]; rintro <h, ->; omega)]
    * obtain <n, rfl> := P_0.surj hX
      refine <Nat.pair 0 n, ?_>
      rw [osumEnum_left (by rw [unpair_pair_fst, unpair_pair_snd]; exact <rfl, hXne>),
        unpair_pair_snd]
    * obtain <n, rfl> := P_1.surj hY
      refine <Nat.pair 1 n, ?_>
      rw [osumEnum_right (by rw [unpair_pair_fst, unpair_pair_snd]; exact <rfl, hYne>)
    ,
        unpair_pair_snd]
  interEq_computable := by
    obtain <f_0, hf_0p, hf_0e> := proper_0_dec P_0
    obtain <f_1, hf_1p, hf_1e> := proper_1_dec P_1
    have hc_01 : forall  n, isOne (f_0 n) = 1  <->  P_0.X n  !=  V_0.master := fun n
    => by
      rw [isOne_eq_one_iff]; exact (hf_0e n).symm
    have hc_11 : forall  n, isOne (f_1 n) = 1  <->  P_1.X n  !=  V_1.master := fun n
    => by
      rw [isOne_eq_one_iff]; exact (hf_1e n).symm
    classical
    set r : Nat  ->  Nat := fun t =>
      if t.unpair.1 = 0  /\  P_0.X t.unpair.2  !=  V_0.master then Nat.pair 0 t.unpair
    .2
      else if t.unpair.1 = 1  /\  P_1.X t.unpair.2  !=  V_1.master then Nat.pair 1 t.
    unpair.2
      else Nat.pair 2 0 with hrdef
```

```
have hr_left : forall  t, (t.unpair.1 = 0  /\  P_0.X t.unpair.2  !=  V_0.master)
->
    r t = Nat.pair 0 t.unpair.2 := fun t h => by simp only [hrdef]; exact
ite_eq_left h
have hr_right : forall  t, not (t.unpair.1 = 0  /\  P_0.X t.unpair.2  !=  V_0.
master)  ->
    (t.unpair.1 = 1  /\  P_1.X t.unpair.2  !=  V_1.master)  ->  r t = Nat.pair 1 t
.unpair.2 :=
  fun t h0 h1 => by simp only [hrdef]; rw [ite_eq_right h0, ite_eq_left h1]
have hr_master : forall  t, not (t.unpair.1 = 0  /\  P_0.X t.unpair.2  !=  V_0.
master)  ->
    not (t.unpair.1 = 1  /\  P_1.X t.unpair.2  !=  V_1.master)  ->  r t = Nat.pair
 2 0 :=
  fun t h0 h1 => by simp only [hrdef]; rw [ite_eq_right h0, ite_eq_right h1]
have hval : forall  t, osumEnum P_0 P_1 t = sumEnum P_0 P_1 (r t) := by
  intro t
  by_cases hL : t.unpair.1 = 0  /\  P_0.X t.unpair.2  !=  V_0.master
  * rw [osumEnum_left hL, hr_left t hL, sumEnum_zero (by rw [unpair_pair_fst]),
      unpair_pair_snd]
  * by_cases hR : t.unpair.1 = 1  /\  P_1.X t.unpair.2  !=  V_1.master
    * rw [osumEnum_right hR, hr_right t hL hR, sumEnum_one (by rw [unpair_pair_fst
]),
        unpair_pair_snd]
    * rw [osumEnum_master hL hR, hr_master t hL hR,
        sumEnum_master (by rw [unpair_pair_fst]; decide) (by rw [unpair_pair_fst];
 decide)]
have hbit0 : forall  v, isOne v = if v = 1 then 1 else 0 := fun v => by
  by_cases h : v = 1
  * rw [ite_eq_left h, h]; rfl
  * rw [ite_eq_right h]; have h1 := isOne_le_one v; have h2 := isOne_eq_one_iff v;
 omega
have hrp : Nat.Primrec r := by
  have hchr1 : Nat.Primrec (fun t : Nat => (1 - t.unpair.1) * isOne (f_0 t.unpair
.2)) :=
    primrec_mul_2 (primrec_sub_2 (Nat.Primrec.const 1) Nat.Primrec.left)
      (primrec_isOne.comp (hf_0p.comp Nat.Primrec.right))
  have hchr2 : Nat.Primrec (fun t : Nat => isOne t.unpair.1 * isOne (f_1 t.unpair
.2)) :=
    primrec_mul_2 (primrec_isOne.comp Nat.Primrec.left)
      (primrec_isOne.comp (hf_1p.comp Nat.Primrec.right))
  have hA : Nat.Primrec (fun t : Nat => Nat.pair 0 t.unpair.2) :=
    (Nat.Primrec.const 0).pair Nat.Primrec.right
  have hB : Nat.Primrec (fun t : Nat => Nat.pair 1 t.unpair.2) :=
    (Nat.Primrec.const 1).pair Nat.Primrec.right
  have hC : Nat.Primrec (fun _ : Nat => Nat.pair 2 0) := Nat.Primrec.const _
  have hchr1eq : forall  t : Nat, (1 - t.unpair.1) * isOne (f_0 t.unpair.2)
      = if (t.unpair.1 = 0  /\  P_0.X t.unpair.2  !=  V_0.master) then 1 else 0 :=
 by
    intro t
    by_cases hc : t.unpair.1 = 0  /\  P_0.X t.unpair.2  !=  V_0.master
    * rw [ite_eq_left hc]
      have ht0 := hc.1
      have e1 : 1 - t.unpair.1 = 1 := by omega
      rw [e1, (hc_01 _).mpr hc.2]
    * rw [ite_eq_right hc]
      rcases not_and_or.mp hc with h | h
```

```
      * have e1 : 1 - t.unpair.1 = 0 := by omega
        rw [e1, Nat.zero_mul]
      * have e2 : isOne (f_0 t.unpair.2) = 0 := by
          have hle := isOne_le_one (f_0 t.unpair.2)
          by_contra h0; exact h ((hc_01 _).mp (by omega))
        rw [e2, Nat.mul_zero]
  have hchr2eq : forall  t : Nat, isOne t.unpair.1 * isOne (f_1 t.unpair.2)
      = if (t.unpair.1 = 1  /\  P_1.X t.unpair.2  !=  V_1.master) then 1 else 0 :=
 by
    intro t
    by_cases hc : t.unpair.1 = 1  /\  P_1.X t.unpair.2  !=  V_1.master
    * rw [ite_eq_left hc, (by rw [isOne_eq_one_iff]; exact hc.1 : isOne t.unpair.1
 = 1),
        (hc_11 _).mpr hc.2]
    * rw [ite_eq_right hc]
      rcases not_and_or.mp hc with h | h
      * have e1 : isOne t.unpair.1 = 0 := by rw [hbit0]; exact ite_eq_right h
        rw [e1, Nat.zero_mul]
      * have e2 : isOne (f_1 t.unpair.2) = 0 := by
          have hle := isOne_le_one (f_1 t.unpair.2)
          by_contra h0; exact h ((hc_11 _).mp (by omega))
        rw [e2, Nat.mul_zero]
  refine (primrec_selectFn hchr1 hA (primrec_selectFn hchr2 hB hC)).of_eq (fun t
=> ?_)
  simp only [hchr1eq, hchr2eq, selectFn_ite, hrdef]
 have hg : Nat.Primrec (fun t => Nat.pair (r t.unpair.1)
     (Nat.pair (r t.unpair.2.unpair.1) (r t.unpair.2.unpair.2))) :=
   (hrp.comp Nat.Primrec.left).pair
     ((hrp.comp (Nat.Primrec.left.comp Nat.Primrec.right)).pair
       (hrp.comp (Nat.Primrec.right.comp Nat.Primrec.right)))
 refine RecDecidable.of_iff (fun t => ?_)
  ((sumPresentation (h_0 := h_0) (h_1 := h_1) P_0 P_1).interEq_computable.comp hg)
 have hXeq : forall  x, (sumPresentation (h_0 := h_0) (h_1 := h_1) P_0 P_1).X x =
 sumEnum P_0 P_1 x :=
  fun _ => rfl
 simp only [unpair_pair_fst, unpair_pair_snd, hXeq]
 rw [hval t.unpair.1, hval t.unpair.2.unpair.1, hval t.unpair.2.unpair.2]
cons_computable := by
 have hLa : RecDecidable (fun t => t.unpair.1.unpair.1 = 0
     /\  P_0.X t.unpair.1.unpair.2  !=  V_0.master) :=
   (RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.left) (Nat.Primrec.const
 0)).and
     ((proper_0_dec P_0).comp (Nat.Primrec.right.comp Nat.Primrec.left))
 have hRa : RecDecidable (fun t => t.unpair.1.unpair.1 = 1
     /\  P_1.X t.unpair.1.unpair.2  !=  V_1.master) :=
   (RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.left) (Nat.Primrec.const
 1)).and
     ((proper_1_dec P_1).comp (Nat.Primrec.right.comp Nat.Primrec.left))
 have hLb : RecDecidable (fun t => t.unpair.2.unpair.1 = 0
     /\  P_0.X t.unpair.2.unpair.2  !=  V_0.master) :=
   (RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.right) (Nat.Primrec.const
  0)).and
     ((proper_0_dec P_0).comp (Nat.Primrec.right.comp Nat.Primrec.right))
 have hRb : RecDecidable (fun t => t.unpair.2.unpair.1 = 1
     /\  P_1.X t.unpair.2.unpair.2  !=  V_1.master) :=
   (RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.right) (Nat.Primrec.const
```

```
 1)).and
    ((proper_1_dec P_1).comp (Nat.Primrec.right.comp Nat.Primrec.right))
have hMca : RecDecidable (fun t => not (t.unpair.1.unpair.1 = 0
     /\  P_0.X t.unpair.1.unpair.2  !=  V_0.master)
     /\  not (t.unpair.1.unpair.1 = 1  /\  P_1.X t.unpair.1.unpair.2  !=  V_1.
master)) := hLa.not.and hRa.not
have hMcb : RecDecidable (fun t => not (t.unpair.2.unpair.1 = 0
     /\  P_0.X t.unpair.2.unpair.2  !=  V_0.master)
     /\  not (t.unpair.2.unpair.1 = 1  /\  P_1.X t.unpair.2.unpair.2  !=  V_1.
master)) := hLb.not.and hRb.not
have hcons0 : RecDecidable (fun t => exists  k, P_0.X k  subseteq  P_0.X t.unpair
.1.unpair.2
     I  P_0.X t.unpair.2.unpair.2) := by
  refine RecDecidable.of_iff (fun t => ?_) (P_0.cons_computable.comp
    ((Nat.Primrec.right.comp Nat.Primrec.left).pair (Nat.Primrec.right.comp Nat.
Primrec.right)))
  simp only [unpair_pair_fst, unpair_pair_snd]
have hcons1 : RecDecidable (fun t => exists  k, P_1.X k  subseteq  P_1.X t.unpair
.1.unpair.2
     I  P_1.X t.unpair.2.unpair.2) := by
  refine RecDecidable.of_iff (fun t => ?_) (P_1.cons_computable.comp
    ((Nat.Primrec.right.comp Nat.Primrec.left).pair (Nat.Primrec.right.comp Nat.
Primrec.right)))
  simp only [unpair_pair_fst, unpair_pair_snd]
refine RecDecidable.of_iff (fun t => ?_)
  (hMca.or (hMcb.or ((hLa.and (hLb.and hcons0)).or (hRa.and (hRb.and hcons1)))))
constructor
* rintro <k, hk>
  by_cases hLa' : t.unpair.1.unpair.1 = 0  /\  P_0.X t.unpair.1.unpair.2  !=  V_0.
master
  * by_cases hLb' : t.unpair.2.unpair.1 = 0  /\  P_0.X t.unpair.2.unpair.2  !=
V_0.master
    * rw [osumEnum_left hLa', osumEnum_left hLb', inj_0_inter] at hk
      refine Or.inr (Or.inr (Or.inl <hLa', hLb', ?_>))
      rcases osumEnum_mem h_0 h_1 k with hkm | <Xk, hXk, -, hkeq> | <Yk, hYk, -,
hkeq>
      * exact absurd (hk (hkm  |>  none_mem_sumMaster)) none_mem_inj_0
      * rw [hkeq, inj_0_subset_inj_0] at hk
        obtain <j, hj> := P_0.surj hXk
        exact <j, by rw [hj]; exact hk>
      * rw [hkeq] at hk
        obtain <y0, hy0> := h_1 Yk hYk
        exact absurd (hk (ir_mem_inj_1.mpr hy0)) ir_mem_inj_0
    * by_cases hRb' : t.unpair.2.unpair.1 = 1  /\  P_1.X t.unpair.2.unpair.2  !=
V_1.master
      * rw [osumEnum_left hLa', osumEnum_right hRb', inj_0_inter_inj_1] at hk
        obtain <m, hm> := osumEnum_nonempty h_0 h_1 k
        exact absurd (hk hm) (Set.notMem_empty m)
      * exact Or.inr (Or.inl <hLb', hRb'>)
  * by_cases hRa' : t.unpair.1.unpair.1 = 1  /\  P_1.X t.unpair.1.unpair.2  !=
V_1.master
    * by_cases hRb' : t.unpair.2.unpair.1 = 1  /\  P_1.X t.unpair.2.unpair.2  !=
V_1.master
      * rw [osumEnum_right hRa', osumEnum_right hRb', inj_1_inter] at hk
        refine Or.inr (Or.inr (Or.inr <hRa', hRb', ?_>))
        rcases osumEnum_mem h_0 h_1 k with hkm | <Xk, hXk, -, hkeq> | <Yk, hYk, -,
```

```
 hkeq>
        * exact absurd (hk (hkm  |>  none_mem_sumMaster)) none_mem_inj_1
        * rw [hkeq] at hk
          obtain <x0, hx0> := h_0 Xk hXk
          exact absurd (hk (il_mem_inj_0.mpr hx0)) il_mem_inj_1
        * rw [hkeq, inj_1_subset_inj_1] at hk
          obtain <j, hj> := P_1.surj hYk
          exact <j, by rw [hj]; exact hk>
      * by_cases hLb' : t.unpair.2.unpair.1 = 0  /\  P_0.X t.unpair.2.unpair.2  !=
  V_0.master
        * rw [osumEnum_right hRa', osumEnum_left hLb', Set.inter_comm,
            inj_0_inter_inj_1] at hk
          obtain <m, hm> := osumEnum_nonempty h_0 h_1 k
          exact absurd (hk hm) (Set.notMem_empty m)
        * exact Or.inr (Or.inl <hLb', hRb'>)
    * exact Or.inl <hLa', hRa'>
* intro hp
  rcases hp with hMca' | hMcb' | <hLa', hLb', hc0> | <hRa', hRb', hc1>
  * refine <t.unpair.2, ?_>
    rw [osumEnum_master hMca'.1 hMca'.2]
    exact Set.subset_inter (osumEnum_subset_master h_0 h_1 t.unpair.2) subset_rfl
  * refine <t.unpair.1, ?_>
    rw [osumEnum_master hMcb'.1 hMcb'.2]
    exact Set.subset_inter subset_rfl (osumEnum_subset_master h_0 h_1 t.unpair.1)
  * obtain <j, hj> := hc0
    have hRHSmem : V_0.mem (P_0.X t.unpair.1.unpair.2  I  P_0.X t.unpair.2.unpair
.2) :=
      V_0.inter_mem (P_0.mem_X _) (P_0.mem_X _) (P_0.mem_X j) hj
    have hRHSne : P_0.X t.unpair.1.unpair.2  I  P_0.X t.unpair.2.unpair.2  !=  V_0
.master :=
      inter_ne_master_left (P_0.mem_X _) hLa'.2
    have hjne : P_0.X j  !=  V_0.master := fun he =>
      hRHSne (Set.Subset.antisymm (V_0.sub_master hRHSmem) (he  |>  hj))
    refine <Nat.pair 0 j, ?_>
    rw [osumEnum_left (by rw [unpair_pair_fst, unpair_pair_snd]; exact <rfl, hjne
>),
      unpair_pair_snd, osumEnum_left hLa', osumEnum_left hLb', inj_0_inter,
inj_0_subset_inj_0]
    exact hj
  * obtain <j, hj> := hc1
    have hRHSmem : V_1.mem (P_1.X t.unpair.1.unpair.2  I  P_1.X t.unpair.2.unpair
.2) :=
      V_1.inter_mem (P_1.mem_X _) (P_1.mem_X _) (P_1.mem_X j) hj
    have hRHSne : P_1.X t.unpair.1.unpair.2  I  P_1.X t.unpair.2.unpair.2  !=  V_1
.master :=
      inter_ne_master_right (P_1.mem_X _) hRa'.2
    have hjne : P_1.X j  !=  V_1.master := fun he =>
      hRHSne (Set.Subset.antisymm (V_1.sub_master hRHSmem) (he  |>  hj))
    refine <Nat.pair 1 j, ?_>
    rw [osumEnum_right (by rw [unpair_pair_fst, unpair_pair_snd]; exact <rfl, hjne
>),
      unpair_pair_snd, osumEnum_right hRa', osumEnum_right hRb', inj_1_inter,
inj_1_subset_inj_1]
    exact hj

end Coalesced
```

```
section Coalesced

variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
  (h_0 : forall  X, V_0.mem X  ->  X.Nonempty) (h_1 : forall  Y, V_1.mem Y  ->  Y.
    Nonempty)

variable (P_0 : ComputablePresentation V_0) (P_1 : ComputablePresentation V_1)

variable {P_0 P_1}

variable {h_0 h_1}

variable (P_0 P_1)

@[simp] theorem osumPresentation_X (t : Nat) :
    (osumPresentation h_0 h_1 (P_0 := P_0) (P_1 := P_1)).X t = osumEnum P_0 P_1 t :=
    rfl

end Coalesced

section Iterate

open Domain.Recursive

variable {V : NeighborhoodSystem a} (P : ComputablePresentation V)

/-- The `j`-th fiber **index** of the `D^ inf `-code `t`: the `j`-th entry of the coded
     list, defaulting
to `P.masterIdx` (the master index) beyond the coded length. -/
def iterIdx (t j : Nat) : Nat := nthCode t j P.masterIdx

/-- The `D^ inf `-enumeration: the neighbourhood whose `j`-th fiber is `P.X (iterIdx t
    j)`. -/
def iterEnum (t : Nat) : Set (Nat  x  a) := {p | p.2  in  P.X (iterIdx P t p.1)}

end Iterate

section Iterate

open Domain.Recursive

variable {V : NeighborhoodSystem a} (P : ComputablePresentation V)

variable {P}

@[simp] theorem fiber_iterEnum (t j : Nat) : fiber (iterEnum P t) j = P.X (iterIdx P t
     j) := rfl

/-- Build, by induction on `N`, a list of fiber indices reproducing `W`'s first `N`
    fibers. -/
theorem exists_list_fiber {W : Set (Nat  x  a)} (hW : (iterSys V).mem W) :
    forall  N, exists  L : List Nat, L.length = N  /\  forall  j, j < N  ->  P.X (L.
    getD j P.masterIdx) = fiber W j := by
  intro N
  induction N with
```

```
  | zero => exact <[], rfl, fun j hj => absurd hj (Nat.not_lt_zero j)>
  | succ N ih =>
    obtain <L, hlen, hL> := ih
    obtain <n, hn> := P.surj (hW.1 N)
    refine <L ++ [n], by simp [List.length_append, hlen], fun j hj => ?_>
    by_cases h : j < N
    * rw [getD_append_cf L [n] P.masterIdx (by rw [hlen]; exact h)]
      exact hL j h
    * have hjN : j = N := by omega
      subst hjN
      rw [getD_append_right_cf L [n] P.masterIdx hlen.le, hlen, Nat.sub_self,
        List.getD_cons_zero]
      exact hn

/-- The coordinate-wise intersection generator: with `s = <j, <n, m>>`, returns an
    index of
`X (iterIdx n j)  I  X (iterIdx m j)` via `P.inter`. -/
def interG (P : ComputablePresentation V) (s : Nat) : Nat :=
  P.inter (iterIdx P s.unpair.2.unpair.1 s.unpair.1) (iterIdx P s.unpair.2.unpair.2 s.
    unpair.1)

/-- Beyond the coded length the fiber index defaults to the master: if `t  <=  j` then
`iterIdx t j = masterIdx` (the coded list has length ` <=  t`). -/
theorem iterIdx_ge {t j : Nat} (h : t  <=  j) : iterIdx P t j = P.masterIdx := by
  unfold iterIdx
  rw [nthCode_eq, getD_eq_default_cf]
  exact le_trans (decodeList_length_le t) h

/-- **Relation (i) reduces to a bounded coordinate check.** `iterEnum n  I  iterEnum m
    = iterEnum k`
iff fiber-wise equality holds for all `j` below `n+m+k` (beyond which every fiber is `
    Delta`). -/
theorem iterEnum_inter_eq_iff {n m k : Nat} :
    iterEnum P n  I  iterEnum P m = iterEnum P k  <->
      forall  j, j < n + m + k  ->
        P.X (iterIdx P n j)  I  P.X (iterIdx P m j) = P.X (iterIdx P k j) := by
  constructor
  * intro h j _
    have := congrArg (fun W => fiber W j) h
    simpa only [fiber_inter, fiber_iterEnum] using this
  * intro h
    apply eq_of_fiber_eq
    intro j
    rw [fiber_inter, fiber_iterEnum, fiber_iterEnum, fiber_iterEnum]
    by_cases hj : j < n + m + k
    * exact h j hj
    * rw [iterIdx_ge (show n  <=  j by omega), iterIdx_ge (show m  <=  j by omega),
        iterIdx_ge (show k  <=  j by omega), P.masterIdx_spec, Set.inter_self]

theorem iterEnum_mem (t : Nat) : (iterSys V).mem (iterEnum P t) := by
  refine <fun j => ?_, t, fun j hj => ?_>
  * rw [fiber_iterEnum]; exact P.mem_X _
  * rw [fiber_iterEnum, iterIdx_ge hj, P.masterIdx_spec]

/-- The `D^ inf `-intersection function: tabulate `interG` over the coordinates `0 ...
    n+m-1`. -/
```

```
def iterInter (P : ComputablePresentation V) (n m : Nat) : Nat :=
  tabCode (interG P) (Nat.pair (n + m) (Nat.pair n m)) (n + m)

theorem iterInter_idx_ge {n m j : Nat} (hj : n + m  <=  j) :
    iterIdx P (iterInter P n m) j = P.masterIdx := by
  show nthCode (iterInter P n m) j P.masterIdx = _
  rw [iterInter, tabCode_nth_ge hj]

theorem iterInter_idx_lt {n m j : Nat} (hj : j < n + m) :
    iterIdx P (iterInter P n m) j = P.inter (iterIdx P n j) (iterIdx P m j) := by
  show nthCode (iterInter P n m) j P.masterIdx = _
  rw [iterInter, tabCode_nth_lt hj]
  unfold interG
  simp only [unpair_pair_fst, unpair_pair_snd]

/-- **Relation (ii) reduces to a bounded coordinate check.** `iterEnum n  I  iterEnum m
    ` is consistent
iff every coordinate `j < n+m` is consistent in `V`. The witness is `iterInter n m`. -/
theorem iterEnum_cons_iff {n m : Nat} :
    (exists  k, iterEnum P k  subseteq  iterEnum P n  I  iterEnum P m)  <->
      forall  j, j < n + m  ->  exists  i, P.X i  subseteq  P.X (iterIdx P n j)  I  P.
    X (iterIdx P m j) := by
  constructor
  * rintro <k, hk> j _
    refine <iterIdx P k j, ?_>
    have := fiber_mono hk j
    rwa [fiber_inter, fiber_iterEnum, fiber_iterEnum, fiber_iterEnum] at this
  * intro h
    refine <iterInter P n m, ?_>
    apply subset_of_fiber_subset
    intro j
    rw [fiber_inter, fiber_iterEnum, fiber_iterEnum, fiber_iterEnum]
    by_cases hj : j < n + m
    * rw [iterInter_idx_lt hj]
      obtain <i, hi> := h j hj
      rw [P.inter_spec <i, hi>]
    * rw [iterInter_idx_ge (by omega), iterIdx_ge (show n  <=  j by omega),
        iterIdx_ge (show m  <=  j by omega), P.masterIdx_spec, Set.inter_self]

end Iterate

section Coalesced

variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
  (h_0 : forall  X, V_0.mem X  ->  X.Nonempty) (h_1 : forall  Y, V_1.mem Y  ->  Y.
    Nonempty)

variable (P_0 : ComputablePresentation V_0) (P_1 : ComputablePresentation V_1)

variable {P_0 P_1}

variable {h_0 h_1}

/-- An `osum`-member containing `inj_0 Delta_0` is forced to be the master (the bottom-
    collapse: the only
```

```
proper copy ` supseteq  inj_0 Delta_0` would need `Delta_0  subseteq  X` with `X  !=
    Delta_0`). -/
theorem osum_eq_master_of_inj_0master {W : Set (Option (a ? b))} (hW : (osum h_0 h_1).
    mem W)
    (hsub : inj_0 V_0.master  subseteq  W) : W = sumMaster V_0 V_1 := by
  rcases hW with rfl | <X', hX', hX'ne, rfl> | <Y', hY', -, rfl>
  * rfl
  * exact absurd (Set.Subset.antisymm (V_0.sub_master hX') (inj_0_subset_inj_0.mp hsub
    )) hX'ne
  * obtain <a, ha> := h_0 V_0.master V_0.master_mem
    exact absurd (hsub (il_mem_inj_0.mpr ha)) il_mem_inj_1

/-- **Coalesced left injection `in_0 : ?_0  ->  ?_0 ? ?_1`**, `X (in_0) W  <->  inj_0 X
      subseteq  W`. -/
def osumInMap_0 : ApproximableMap V_0 (osum h_0 h_1) where
  rel X W := V_0.mem X  /\  (osum h_0 h_1).mem W  /\  inj_0 X  subseteq  W
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := <V_0.master_mem, (osum h_0 h_1).master_mem, inj_0_subset_sumMaster V_0
    .master_mem>
  inter_right := by
    rintro X W W' <hX, hW, hsub> <-, hW', hsub'>
    refine <hX, ?_, Set.subset_inter hsub hsub'>
    by_cases hXm : X = V_0.master
    * subst hXm
      rw [osum_eq_master_of_inj_0master hW hsub, osum_eq_master_of_inj_0master hW'
    hsub',
        Set.inter_self]
      exact (osum h_0 h_1).master_mem
    * exact (osum h_0 h_1).inter_mem hW hW' (Or.inr (Or.inl <X, hX, hXm, rfl>))
        (Set.subset_inter hsub hsub')
  mono := by
    rintro X X' W W' <_, _, hsub> hX'X hWW' hX' hW'
    exact <hX', hW', (inj_0_subset_inj_0.mpr hX'X).trans (hsub.trans hWW')>

theorem osum_eq_master_of_inj_1master {W : Set (Option (a ? b))} (hW : (osum h_0 h_1).
    mem W)
    (hsub : inj_1 V_1.master  subseteq  W) : W = sumMaster V_0 V_1 := by
  rcases hW with rfl | <X', hX', -, rfl> | <Y', hY', hY'ne, rfl>
  * rfl
  * obtain <b, hb> := h_1 V_1.master V_1.master_mem
    exact absurd (hsub (ir_mem_inj_1.mpr hb)) ir_mem_inj_0
  * exact absurd (Set.Subset.antisymm (V_1.sub_master hY') (inj_1_subset_inj_1.mp hsub
    )) hY'ne

/-- **Coalesced right injection `in_1 : ?_1  ->  ?_0 ? ?_1`**. -/
def osumInMap_1 : ApproximableMap V_1 (osum h_0 h_1) where
  rel Y W := V_1.mem Y  /\  (osum h_0 h_1).mem W  /\  inj_1 Y  subseteq  W
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := <V_1.master_mem, (osum h_0 h_1).master_mem, inj_1_subset_sumMaster V_1
    .master_mem>
  inter_right := by
    rintro Y W W' <hY, hW, hsub> <-, hW', hsub'>
    refine <hY, ?_, Set.subset_inter hsub hsub'>
    by_cases hYm : Y = V_1.master
```

```
    * subst hYm
      rw [osum_eq_master_of_inj_1master hW hsub, osum_eq_master_of_inj_1master hW'
    hsub',
        Set.inter_self]
      exact (osum h_0 h_1).master_mem
    * exact (osum h_0 h_1).inter_mem hW hW' (Or.inr (Or.inr <Y, hY, hYm, rfl>))
        (Set.subset_inter hsub hsub')
  mono := by
    rintro Y Y' W W' <_, _, hsub> hY'Y hWW' hY' hW'
    exact <hY', hW', (inj_1_subset_inj_1.mpr hY'Y).trans (hsub.trans hWW')>

end Coalesced

section Coalesced

variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
  (h_0 : forall  X, V_0.mem X  ->  X.Nonempty) (h_1 : forall  Y, V_1.mem Y  ->  Y.
    Nonempty)

variable (P_0 : ComputablePresentation V_0) (P_1 : ComputablePresentation V_1)

variable {P_0 P_1}

variable {h_0 h_1}

variable (P_0 P_1)

section CoalescedFunctor

variable {a' b' : Type*} {V_0' : NeighborhoodSystem a'} {V_1' : NeighborhoodSystem b'}
  (h_0' : forall  X, V_0'.mem X  ->  X.Nonempty) (h_1' : forall  Y, V_1'.mem Y  ->  Y.
    Nonempty)

/-- An `osum`-member ` subseteq  inj_0 X` is a *proper* left copy. -/
theorem osum_mem_subset_inj_0 {W : Set (Option (a ? b))} {X : Set a}
    (hW : (osum h_0 h_1).mem W) (hsub : W  subseteq  inj_0 X) :
    exists  X_2, V_0.mem X_2  /\  X_2  !=  V_0.master  /\  W = inj_0 X_2 := by
  rcases hW with rfl | <X_2, hX_2, hne, rfl> | <Y_2, hY_2, hne, rfl>
  * exact absurd (hsub none_mem_sumMaster) none_mem_inj_0
  * exact <X_2, hX_2, hne, rfl>
  * obtain <b, hb> := h_1 Y_2 hY_2; exact absurd (hsub (ir_mem_inj_1.mpr hb))
    ir_mem_inj_0

end CoalescedFunctor

end Coalesced

section Coalesced

variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
  (h_0 : forall  X, V_0.mem X  ->  X.Nonempty) (h_1 : forall  Y, V_1.mem Y  ->  Y.
    Nonempty)

variable (P_0 : ComputablePresentation V_0) (P_1 : ComputablePresentation V_1)

variable {P_0 P_1}
```

```
variable {h_0 h_1}

variable (P_0 P_1)

section CoalescedFunctor

variable {a' b' : Type*} {V_0' : NeighborhoodSystem a'} {V_1' : NeighborhoodSystem b'}
  (h_0' : forall  X, V_0'.mem X  ->  X.Nonempty) (h_1' : forall  Y, V_1'.mem Y  ->  Y.
    Nonempty)

theorem osum_mem_subset_inj_1 {W : Set (Option (a ? b))} {Y : Set b}
    (hW : (osum h_0 h_1).mem W) (hsub : W  subseteq  inj_1 Y) :
    exists  Y_2, V_1.mem Y_2  /\  Y_2  !=  V_1.master  /\  W = inj_1 Y_2 := by
  rcases hW with rfl | <X_2, hX_2, hne, rfl> | <Y_2, hY_2, hne, rfl>
  * exact absurd (hsub none_mem_sumMaster) none_mem_inj_1
  * obtain <a, ha> := h_0 X_2 hX_2; exact absurd (hsub (il_mem_inj_0.mpr ha))
    il_mem_inj_1
  * exact <Y_2, hY_2, hne, rfl>

end CoalescedFunctor

end Coalesced

section Coalesced

variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
  (h_0 : forall  X, V_0.mem X  ->  X.Nonempty) (h_1 : forall  Y, V_1.mem Y  ->  Y.
    Nonempty)

variable (P_0 : ComputablePresentation V_0) (P_1 : ComputablePresentation V_1)

variable {P_0 P_1}

variable {h_0 h_1}

variable (P_0 P_1)

section CoalescedFunctor

variable {a' b' : Type*} {V_0' : NeighborhoodSystem a'} {V_1' : NeighborhoodSystem b'}
  (h_0' : forall  X, V_0'.mem X  ->  X.Nonempty) (h_1' : forall  Y, V_1'.mem Y  ->  Y.
    Nonempty)

/-- **The coalesced sum of two maps `f ? g : ?_0 ? ?_1  ->  ?_0' ? ?_1'`.** -/
def osumMap (f : ApproximableMap V_0 V_0') (g : ApproximableMap V_1 V_1') :
    ApproximableMap (osum h_0 h_1) (osum h_0' h_1') where
  rel W W' := (osum h_0 h_1).mem W  /\  (osum h_0' h_1').mem W'  /\
    (W' = sumMaster V_0' V_1'  \/
      (exists  X Y', W = inj_0 X  /\  W' = inj_0 Y'  /\  Y'  !=  V_0'.master  /\  f.
    rel X Y')  \/
      (exists  Y Y', W = inj_1 Y  /\  W' = inj_1 Y'  /\  Y'  !=  V_1'.master  /\  g.
    rel Y Y'))
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := <(osum h_0 h_1).master_mem, (osum h_0' h_1').master_mem, Or.inl rfl>
```

```
inter_right := by
  rintro W W'_1 W'_2 <hW, hW'_1, hd_1> <-, hW'_2, hd_2>
  have hmem : forall  W'' : Set (Option (a' ? b')),
      (W'' = sumMaster V_0' V_1'  \/
        (exists  X Y', W = inj_0 X  /\  W'' = inj_0 Y'  /\  Y'  !=  V_0'.master  /\
   f.rel X Y')  \/
        (exists  Y Y', W = inj_1 Y  /\  W'' = inj_1 Y'  /\  Y'  !=  V_1'.master  /\
   g.rel Y Y'))  ->
        (osum h_0' h_1').mem W'' := by
    rintro W'' (rfl | <_, Y', _, rfl, hY'ne, hf> | <_, Y', _, rfl, hY'ne, hg>)
    * exact (osum h_0' h_1').master_mem
    * exact Or.inr (Or.inl <Y', f.rel_cod hf, hY'ne, rfl>)
    * exact Or.inr (Or.inr <Y', g.rel_cod hg, hY'ne, rfl>)
  have key : W'_1  I  W'_2 = sumMaster V_0' V_1'  \/
      (exists  X Y', W = inj_0 X  /\  W'_1  I  W'_2 = inj_0 Y'  /\  Y'  !=  V_0'.
  master  /\  f.rel X Y')  \/
      (exists  Y Y', W = inj_1 Y  /\  W'_1  I  W'_2 = inj_1 Y'  /\  Y'  !=  V_1'.
  master  /\  g.rel Y Y') := by
    rcases hd_1 with rfl | <X, Y'_1, hWX_1, rfl, hY'_1ne, hf_1> | <Y, Y'_1, hWY_1,
  rfl, hY'_1ne, hg_1>
    * have he : sumMaster V_0' V_1'  I  W'_2 = W'_2 :=
        Set.inter_eq_self_of_subset_right
          ((osum h_0' h_1').sub_master hW'_2 : W'_2  subseteq  sumMaster V_0' V_1')
      rw [he]; exact hd_2
    * rcases hd_2 with rfl | <X', Y'_2, hWX_2, rfl, hY'_2ne, hf_2> | <Y', Y'_2,
  hWY_2, rfl, hY'_2ne, hg_2>
      * rw [Set.inter_eq_left.mpr (inj_0_subset_sumMaster (f.rel_cod hf_1))]
        exact Or.inr (Or.inl <X, Y'_1, hWX_1, rfl, hY'_1ne, hf_1>)
      * obtain rfl : X = X' := inj_0_injective (hWX_1.symm.trans hWX_2)
        rw [inj_0_inter]
        exact Or.inr (Or.inl <X, Y'_1  I  Y'_2, hWX_1, rfl,
          inter_ne_master_left (f.rel_cod hf_1) hY'_1ne, f.inter_right hf_1 hf_2>)
      * exact absurd (hWX_1.symm.trans hWY_2)
          (fun h => not_inj_0_subset_inj_1 (h_0 X (f.rel_dom hf_1)) h.subset)
    * rcases hd_2 with rfl | <X', Y'_2, hWX_2, rfl, hY'_2ne, hf_2> | <Y', Y'_2,
  hWY_2, rfl, hY'_2ne, hg_2>
      * rw [Set.inter_eq_left.mpr (inj_1_subset_sumMaster (g.rel_cod hg_1))]
        exact Or.inr (Or.inr <Y, Y'_1, hWY_1, rfl, hY'_1ne, hg_1>)
      * exact absurd (hWY_1.symm.trans hWX_2)
          (fun h => not_inj_1_subset_inj_0 (h_1 Y (g.rel_dom hg_1)) h.subset)
      * obtain rfl : Y = Y' := inj_1_injective (hWY_1.symm.trans hWY_2)
        rw [inj_1_inter]
        exact Or.inr (Or.inr <Y, Y'_1  I  Y'_2, hWY_1, rfl,
          inter_ne_master_left (g.rel_cod hg_1) hY'_1ne, g.inter_right hg_1 hg_2>)
  exact <hW, hmem _ key, key>
mono := by
  rintro W W_2 W' W'_2 <hW, hW', hd> hW_2W hW'W'_2 hW_2mem hW'_2mem
  refine <hW_2mem, hW'_2mem, ?_>
  rcases hd with rfl | <X, Y', rfl, rfl, hY'ne, hf> | <Y, Y', rfl, rfl, hY'ne, hg>
  * left; exact Set.Subset.antisymm ((osum h_0' h_1').sub_master hW'_2mem) hW'W'_2
  * obtain <X_2, hX_2, hX_2ne, rfl> := osum_mem_subset_inj_0 hW_2mem hW_2W
    have hXX_2 : X_2  subseteq  X := inj_0_subset_inj_0.mp hW_2W
    rcases hW'_2mem with rfl | <Y'_2, hY'_2, hY'_2ne, rfl> | <Z'_2, hZ'_2, hZ'_2ne,
  rfl>
    * left; rfl
    * exact Or.inr (Or.inl <X_2, Y'_2, rfl, rfl, hY'_2ne,
```

```
          f.mono hf hXX_2 (inj_0_subset_inj_0.mp hW'W'_2) hX_2 hY'_2>)
      * exact (not_inj_0_subset_inj_1 (h_0' Y' (f.rel_cod hf)) hW'W'_2).elim
    * obtain <Y_2, hY_2, hY_2ne, rfl> := osum_mem_subset_inj_1 hW_2mem hW_2W
      have hYY_2 : Y_2  subseteq  Y := inj_1_subset_inj_1.mp hW_2W
      rcases hW'_2mem with rfl | <X'_2, hX'_2, hX'_2ne, rfl> | <Y'_2, hY'_2, hY'_2ne,
    rfl>
      * left; rfl
      * exact (not_inj_1_subset_inj_0 (h_1' Y' (g.rel_cod hg)) hW'W'_2).elim
      * exact Or.inr (Or.inr <Y_2, Y'_2, rfl, rfl, hY'_2ne,
          g.mono hg hYY_2 (inj_1_subset_inj_1.mp hW'W'_2) hY_2 hY'_2>)

end CoalescedFunctor

end Coalesced

section Iterate

open Domain.Recursive

variable {V : NeighborhoodSystem a} (P : ComputablePresentation V)

variable {P}

/-- Primitive recursivity of `fun w  |->  iterIdx (codeProj w) (w.unpair.1)` for a
    primrec `codeProj`. -/
theorem primrec_iterIdx_proj {codeProj : Nat  ->  Nat} (hc : Nat.Primrec codeProj) :
    Nat.Primrec (fun w => iterIdx P (codeProj w) w.unpair.1) := by
  refine (primrec_nthCode.comp (hc.pair (Nat.Primrec.left.pair
    (Nat.Primrec.const P.masterIdx)))).of_eq (fun w => ?_)
  simp only [iterIdx, unpair_pair_fst, unpair_pair_snd]

theorem primrec_interG : Nat.Primrec (interG P) := by
  have hA : Nat.Primrec (fun s => iterIdx P s.unpair.2.unpair.1 s.unpair.1) :=
    primrec_iterIdx_proj (Nat.Primrec.left.comp Nat.Primrec.right)
  have hB : Nat.Primrec (fun s => iterIdx P s.unpair.2.unpair.2 s.unpair.1) :=
    primrec_iterIdx_proj (Nat.Primrec.right.comp Nat.Primrec.right)
  refine (P.inter_primrec.comp (hA.pair hB)).of_eq (fun s => ?_)
  unfold interG
  simp only [unpair_pair_fst, unpair_pair_snd]

/-- **Exercise 7.15 (Scott 1981, PRG-19) -- `D^ inf ` carries a full computable
    presentation.** Built
choice-free from a presentation `P` of `V`: the enumeration codes finite fiber-index
    lists, the two
relations reduce to bounded coordinate checks (`RecDecidable.bForall`), and the
    intersection function
tabulates `P.inter` coordinate-wise. -/
noncomputable def iterPresentation (P : ComputablePresentation V) :
    ComputablePresentation (iterSys V) where
  X := iterEnum P
  mem_X := iterEnum_mem
  surj := by
    rintro W hW
    obtain <N, hN> := hW.2
    obtain <L, hlen, hL> := exists_list_fiber (P := P) hW N
    refine <encodeList L, ?_>
```

```
  apply eq_of_fiber_eq
  intro j
  show P.X (nthCode (encodeList L) j P.masterIdx) = fiber W j
  rw [nthCode_eq, decodeList_encodeList]
  by_cases hj : j < N
  * exact hL j hj
  * rw [getD_eq_default_cf L P.masterIdx (by rw [hlen]; omega), P.masterIdx_spec,
      hN j (by omega)]
interEq_computable := by
  have hbound : Nat.Primrec
      (fun t : Nat => t.unpair.1 + t.unpair.2.unpair.1 + t.unpair.2.unpair.2) :=
    primrec_add_2 (primrec_add_2 Nat.Primrec.left (Nat.Primrec.left.comp Nat.Primrec
  .right))
      (Nat.Primrec.right.comp Nat.Primrec.right)
  have hA : Nat.Primrec (fun w => iterIdx P w.unpair.2.unpair.1 w.unpair.1) :=
    primrec_iterIdx_proj (Nat.Primrec.left.comp Nat.Primrec.right)
  have hB : Nat.Primrec (fun w => iterIdx P w.unpair.2.unpair.2.unpair.1 w.unpair.1)
   :=
    primrec_iterIdx_proj (Nat.Primrec.left.comp (Nat.Primrec.right.comp Nat.Primrec.
  right))
  have hC : Nat.Primrec (fun w => iterIdx P w.unpair.2.unpair.2.unpair.2 w.unpair.1)
   :=
    primrec_iterIdx_proj (Nat.Primrec.right.comp (Nat.Primrec.right.comp Nat.Primrec
  .right))
  have hq : RecDecidable (fun w => P.X (iterIdx P w.unpair.2.unpair.1 w.unpair.1)
       I  P.X (iterIdx P w.unpair.2.unpair.2.unpair.1 w.unpair.1)
      = P.X (iterIdx P w.unpair.2.unpair.2.unpair.2 w.unpair.1)) := by
    refine RecDecidable.of_iff (fun w => ?_) (P.interEq_computable.comp (hA.pair (hB
  .pair hC)))
    simp only [unpair_pair_fst, unpair_pair_snd]
  refine RecDecidable.of_iff (fun t => ?_) (hq.bForall hbound)
  simp only [unpair_pair_fst, unpair_pair_snd]
  exact iterEnum_inter_eq_iff
cons_computable := by
  have hbound : Nat.Primrec (fun t : Nat => t.unpair.1 + t.unpair.2) :=
    primrec_add_2 Nat.Primrec.left Nat.Primrec.right
  have hA : Nat.Primrec (fun w => iterIdx P w.unpair.2.unpair.1 w.unpair.1) :=
    primrec_iterIdx_proj (Nat.Primrec.left.comp Nat.Primrec.right)
  have hB : Nat.Primrec (fun w => iterIdx P w.unpair.2.unpair.2 w.unpair.1) :=
    primrec_iterIdx_proj (Nat.Primrec.right.comp Nat.Primrec.right)
  have hq : RecDecidable (fun w => exists  i, P.X i  subseteq  P.X (iterIdx P w.
  unpair.2.unpair.1 w.unpair.1)
       I  P.X (iterIdx P w.unpair.2.unpair.2 w.unpair.1)) := by
    refine RecDecidable.of_iff (fun w => ?_) (P.cons_computable.comp (hA.pair hB))
    simp only [unpair_pair_fst, unpair_pair_snd]
  refine RecDecidable.of_iff (fun t => ?_) (hq.bForall hbound)
  simp only [unpair_pair_fst, unpair_pair_snd]
  exact iterEnum_cons_iff
inter := iterInter P
inter_primrec := by
  have hr : Nat.Primrec (fun t => Nat.pair
      (Nat.pair (t.unpair.1 + t.unpair.2) (Nat.pair t.unpair.1 t.unpair.2))
      (t.unpair.1 + t.unpair.2)) :=
    ((primrec_add_2 Nat.Primrec.left Nat.Primrec.right).pair
      (Nat.Primrec.left.pair Nat.Primrec.right)).pair
      (primrec_add_2 Nat.Primrec.left Nat.Primrec.right)
```

```
    refine ((primrec_tabCode (primrec_interG (P := P))).comp hr).of_eq (fun t => ?_)
    show tabCode (interG P) _ _ = iterInter P t.unpair.1 t.unpair.2
    unfold iterInter
    simp only [unpair_pair_fst, unpair_pair_snd]
  inter_spec := by
    intro n m hcons
    apply eq_of_fiber_eq
    intro j
    rw [fiber_inter, fiber_iterEnum, fiber_iterEnum, fiber_iterEnum]
    by_cases hj : j < n + m
    * rw [iterInter_idx_lt hj, P.inter_spec ((iterEnum_cons_iff (P := P)).mp hcons j
    hj)]
    * rw [iterInter_idx_ge (by omega), iterIdx_ge (show n  <=  j by omega),
        iterIdx_ge (show m  <=  j by omega), P.masterIdx_spec, Set.inter_self]
  masterIdx := 0
  masterIdx_spec := by
    apply eq_of_fiber_eq
    intro j
    rw [fiber_iterEnum, fiber_iterSys_master, iterIdx_ge (Nat.zero_le j), P.
    masterIdx_spec]

end Iterate

section Smash

variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
  (P_0 : ComputablePresentation V_0) (P_1 : ComputablePresentation V_1)

variable {P_0 P_1}

variable (P_0 P_1)

section SmashFunctor

variable {a' b' : Type*} {V_0' : NeighborhoodSystem a'} {V_1' : NeighborhoodSystem b'}

/-- **The smash of two maps `f ? g : ?_0 ? ?_1  ->  ?_0' ? ?_1'`.** -/
def smashMap (f : ApproximableMap V_0 V_0') (g : ApproximableMap V_1 V_1') :
    ApproximableMap (smash V_0 V_1) (smash V_0' V_1') where
  rel W W' := (smash V_0 V_1).mem W  /\  (smash V_0' V_1').mem W'  /\
    (W' = prodNbhd V_0'.master V_1'.master  \/
      exists  A B A' B', W = prodNbhd A B  /\  W' = prodNbhd A' B'  /\
        A'  !=  V_0'.master  /\  B'  !=  V_1'.master  /\  f.rel A A'  /\  g.rel B B')
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := <(smash V_0 V_1).master_mem, (smash V_0' V_1').master_mem, Or.inl rfl>
  inter_right := by
    rintro W W'_1 W'_2 <hW, hW'_1, hd_1> <-, hW'_2, hd_2>
    have key : W'_1  I  W'_2 = prodNbhd V_0'.master V_1'.master  \/
        exists  A B A' B', W = prodNbhd A B  /\  W'_1  I  W'_2 = prodNbhd A' B'  /\
          A'  !=  V_0'.master  /\  B'  !=  V_1'.master  /\  f.rel A A'  /\  g.rel B B'
     := by
      rcases hd_1 with rfl | <A, B, A'_1, B'_1, hWAB, rfl, hA'_1, hB'_1, hf_1, hg_1>
      * rw [Set.inter_eq_right.mpr (show W'_2  subseteq  prodNbhd V_0'.master V_1'.
    master from
          (smash V_0' V_1').sub_master hW'_2)]
```

```
        exact hd_2
      * rcases hd_2 with rfl | <A_2, B_2, A'_2, B'_2, hWAB_2, rfl, hA'_2, hB'_2, hf_2,
     hg_2>
        * rw [Set.inter_eq_left.mpr (show prodNbhd A'_1 B'_1  subseteq  prodNbhd V_0'.
    master V_1'.master from
            (smash V_0' V_1').sub_master hW'_1)]
          exact Or.inr <A, B, A'_1, B'_1, hWAB, rfl, hA'_1, hB'_1, hf_1, hg_1>
        * obtain <rfl, rfl> := prodNbhd_injective (hWAB.symm.trans hWAB_2)
          rw [prodNbhd_inter]
          exact Or.inr <A, B, A'_1  I  A'_2, B'_1  I  B'_2, hWAB, rfl,
            inter_ne_master_left (f.rel_cod hf_1) hA'_1,
            inter_ne_master_left (g.rel_cod hg_1) hB'_1,
            f.inter_right hf_1 hf_2, g.inter_right hg_1 hg_2>
    refine <hW, ?_, key>
    rcases key with hm | <A, B, A', B', -, hWW', hA', hB', hf, hg>
    * rw [hm]; exact (smash V_0' V_1').master_mem
    * rw [hWW']; exact smash_mem_proper (f.rel_cod hf) hA' (g.rel_cod hg) hB'
  mono := by
    rintro W W_2 W' W'_2 <hW, hW', hd> hW_2W hW'W'_2 hW_2mem hW'_2mem
    refine <hW_2mem, hW'_2mem, ?_>
    rcases hd with rfl | <A, B, A', B', rfl, rfl, hA', hB', hf, hg>
    * left; exact Set.Subset.antisymm ((smash V_0' V_1').sub_master hW'_2mem) hW'W'_2
    * obtain <A_2, B_2, hA_2, hB_2, rfl> := smash_mem_prodNbhd_form hW_2mem
      obtain <hA_2A, hB_2B> := prodNbhd_subset_iff.mp hW_2W
      rcases smash_mem_iff.mp hW'_2mem with rfl |
        <A'_2, B'_2, hA'_2, hA'_2ne, hB'_2, hB'_2ne, rfl>
      * left; rfl
      * obtain <hA'A'_2, hB'B'_2> := prodNbhd_subset_iff.mp hW'W'_2
        exact Or.inr <A_2, B_2, A'_2, B'_2, rfl, rfl, hA'_2ne, hB'_2ne,
          f.mono hf hA_2A hA'A'_2 hA_2 hA'_2, g.mono hg hB_2B hB'B'_2 hB_2 hB'_2>

end SmashFunctor

end Smash

end

section
variable {a b ? : Type*}

/-- A full (`inter`-carrying) presentation forgets to a bare one. Hence the project's
`IsEffectivelyGiven` implies Scott's `IsEffectivelyGivenS`. -/
def ComputablePresentation.toScott {V : NeighborhoodSystem a} (P :
    ComputablePresentation V) :
    ScottPresentation V where
  X := P.X
  mem_X := P.mem_X
  surj := P.surj
  interEq_computable := P.interEq_computable
  cons_computable := P.cons_computable

namespace ScottPresentation

variable {V : NeighborhoodSystem a} (P : ScottPresentation V)

end ScottPresentation
```

```
open Exercise510

section Iterate

open Domain.Recursive

variable {V : NeighborhoodSystem a} (P : ComputablePresentation V)

variable {P}

@[simp] theorem iterPresentation_X (t : Nat) : (iterPresentation P).X t = iterEnum P t
     := rfl

end Iterate

end

section
variable {a b ? : Type*}

theorem NeighborhoodSystem.IsEffectivelyGiven.toS {V : NeighborhoodSystem a}
    (h : V.IsEffectivelyGiven) : V.IsEffectivelyGivenS :=
  h.elim fun P => <P.toScott>

open Exercise510

section Iterate

open Domain.Recursive

variable {V : NeighborhoodSystem a} (P : ComputablePresentation V)

variable {P}

/-- **Exercise 7.15 (Scott 1981, PRG-19) -- the infinite iterate `D^ inf ` is
    effectively given.** -/
theorem iterSys_isEffectivelyGiven (e : V.IsEffectivelyGiven) :
    (iterSys V).IsEffectivelyGiven := by
  obtain <Q> := e
  exact <iterPresentation (P := Q)>

end Iterate

section Coalesced

variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
  (h_0 : forall  X, V_0.mem X  ->  X.Nonempty) (h_1 : forall  Y, V_1.mem Y  ->  Y.
    Nonempty)

variable (P_0 : ComputablePresentation V_0) (P_1 : ComputablePresentation V_1)

variable {P_0 P_1}

variable {h_0 h_1}
```

```
variable (P_0 P_1)

/-- **`in_0` is computable.** `X?_n (in_0) (osumEnum m)` decodes to: `m` a proper left
    copy with
`X?_n  subseteq  X?_{m.2}`, or `m` the master (`inj_0` of a proper neighbourhood always
     sits under the master);
the proper-right case is impossible (`inj_0` of a nonempty set is not ` subseteq  inj_1
    `). -/
theorem osumInMap_0_isComputable :
    ComputableMapS P_0.toScott (osumPresentation h_0 h_1 (P_0 := P_0) (P_1 := P_1))
    osumInMap_0 := by
  constructor
  have hT0 : RecDecidable (fun t => t.unpair.2.unpair.1 = 0) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.right) (Nat.Primrec.const 0)
  have hT1 : RecDecidable (fun t => t.unpair.2.unpair.1 = 1) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.right) (Nat.Primrec.const 1)
  have hP0 : RecDecidable (fun t => P_0.X t.unpair.2.unpair.2  !=  V_0.master) :=
    (proper_0_dec P_0).comp (Nat.Primrec.right.comp Nat.Primrec.right)
  have hP1 : RecDecidable (fun t => P_1.X t.unpair.2.unpair.2  !=  V_1.master) :=
    (proper_1_dec P_1).comp (Nat.Primrec.right.comp Nat.Primrec.right)
  have hInc : RecDecidable (fun t => P_0.X t.unpair.1  subseteq  P_0.X t.unpair.2.
    unpair.2) :=
    RecDecidable.of_iff (fun t => by simp only [unpair_pair_fst, unpair_pair_snd])
      (P_0.incl_computable.comp (Nat.Primrec.left.pair (Nat.Primrec.right.comp Nat.
    Primrec.right)))
  refine (RecDecidable.of_iff (fun t => ?_)
    ((hT0.and (hP0.and hInc)).or ((hT0.and hP0).not.and (hT1.and hP1).not))).re
  show (osumInMap_0 (h_0 := h_0) (h_1 := h_1)).rel (P_0.X t.unpair.1) (osumEnum P_0
    P_1 t.unpair.2)  <->  _
  rw [show (osumInMap_0 (h_0 := h_0) (h_1 := h_1)).rel (P_0.X t.unpair.1) (osumEnum
    P_0 P_1 t.unpair.2)
      <->  inj_0 (P_0.X t.unpair.1)  subseteq  osumEnum P_0 P_1 t.unpair.2 from
    <fun h => h.2.2, fun h => <P_0.mem_X _, osumEnum_mem h_0 h_1 _, h>>]
  by_cases hL : t.unpair.2.unpair.1 = 0  /\  P_0.X t.unpair.2.unpair.2  !=  V_0.master
  * rw [osumEnum_left hL, inj_0_subset_inj_0]
    exact <fun h => Or.inl <hL.1, hL.2, h>,
      fun h => h.elim (fun h => h.2.2) (fun h => absurd hL h.1)>
  * by_cases hR : t.unpair.2.unpair.1 = 1  /\  P_1.X t.unpair.2.unpair.2  !=  V_1.
    master
    * rw [osumEnum_right hR]
      refine <fun h => ?_, fun h => h.elim (fun h => absurd <h.1, h.2.1> hL)
        (fun h => absurd hR h.2)>
      obtain <a, ha> := h_0 _ (P_0.mem_X t.unpair.1)
      exact absurd (h (il_mem_inj_0.mpr ha)) il_mem_inj_1
    * rw [osumEnum_master hL hR]
      exact <fun _ => Or.inr <hL, hR>, fun _ => inj_0_subset_sumMaster (P_0.mem_X _)>

/-- **`in_1` is computable.** Symmetric to `in_0`. -/
theorem osumInMap_1_isComputable :
    ComputableMapS P_1.toScott (osumPresentation h_0 h_1 (P_0 := P_0) (P_1 := P_1))
    osumInMap_1 := by
  constructor
  have hT0 : RecDecidable (fun t => t.unpair.2.unpair.1 = 0) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.right) (Nat.Primrec.const 0)
  have hT1 : RecDecidable (fun t => t.unpair.2.unpair.1 = 1) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.right) (Nat.Primrec.const 1)
```

```
  have hP0 : RecDecidable (fun t => P_0.X t.unpair.2.unpair.2  !=  V_0.master) :=
    (proper_0_dec P_0).comp (Nat.Primrec.right.comp Nat.Primrec.right)
  have hP1 : RecDecidable (fun t => P_1.X t.unpair.2.unpair.2  !=  V_1.master) :=
    (proper_1_dec P_1).comp (Nat.Primrec.right.comp Nat.Primrec.right)
  have hInc : RecDecidable (fun t => P_1.X t.unpair.1  subseteq  P_1.X t.unpair.2.
    unpair.2) :=
    RecDecidable.of_iff (fun t => by simp only [unpair_pair_fst, unpair_pair_snd])
      (P_1.incl_computable.comp (Nat.Primrec.left.pair (Nat.Primrec.right.comp Nat.
    Primrec.right)))
  refine (RecDecidable.of_iff (fun t => ?_)
    ((hT1.and (hP1.and hInc)).or ((hT0.and hP0).not.and (hT1.and hP1).not))).re
  show (osumInMap_1 (h_0 := h_0) (h_1 := h_1)).rel (P_1.X t.unpair.1) (osumEnum P_0
    P_1 t.unpair.2)  <->  _
  rw [show (osumInMap_1 (h_0 := h_0) (h_1 := h_1)).rel (P_1.X t.unpair.1) (osumEnum
    P_0 P_1 t.unpair.2)
       <->  inj_1 (P_1.X t.unpair.1)  subseteq  osumEnum P_0 P_1 t.unpair.2 from
    <fun h => h.2.2, fun h => <P_1.mem_X _, osumEnum_mem h_0 h_1 _, h>>]
  by_cases hL : t.unpair.2.unpair.1 = 0  /\  P_0.X t.unpair.2.unpair.2  !=  V_0.master
  * rw [osumEnum_left hL]
    refine <fun h => ?_, fun h => ?_>
    * obtain <b, hb> := h_1 _ (P_1.mem_X t.unpair.1)
      exact absurd (h (ir_mem_inj_1.mpr hb)) ir_mem_inj_0
    * rcases h with <h1, -> | <hn0, ->
      * rw [hL.1] at h1; exact absurd h1 (by decide)
      * exact absurd hL hn0
  * by_cases hR : t.unpair.2.unpair.1 = 1  /\  P_1.X t.unpair.2.unpair.2  !=  V_1.
    master
    * rw [osumEnum_right hR, inj_1_subset_inj_1]
      exact <fun h => Or.inl <hR.1, hR.2, h>,
        fun h => h.elim (fun h => h.2.2) (fun h => absurd hR h.2)>
    * rw [osumEnum_master hL hR]
      exact <fun _ => Or.inr <hL, hR>, fun _ => inj_1_subset_sumMaster (P_1.mem_X _)>

end Coalesced

section Coalesced

variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
  (h_0 : forall  X, V_0.mem X  ->  X.Nonempty) (h_1 : forall  Y, V_1.mem Y  ->  Y.
    Nonempty)

variable (P_0 : ComputablePresentation V_0) (P_1 : ComputablePresentation V_1)

variable {P_0 P_1}

variable {h_0 h_1}

variable (P_0 P_1)

section CoalescedFunctor

variable {a' b' : Type*} {V_0' : NeighborhoodSystem a'} {V_1' : NeighborhoodSystem b'}
  (h_0' : forall  X, V_0'.mem X  ->  X.Nonempty) (h_1' : forall  Y, V_1'.mem Y  ->  Y.
    Nonempty)

/-- **`f ? g` is computable.** The relation `(osumEnum n) (f?g) (osumEnum' m)` decodes
```

```
    by tags: `m`
improper (master absorbs), or `m`/`n` both proper left copies with `f`'s r.e. relation,
     or both
proper right copies with `g`'s. -/
theorem osumMap_isComputable (P_0' : ComputablePresentation V_0') (P_1' :
    ComputablePresentation V_1')
    {f : ApproximableMap V_0 V_0'} {g : ApproximableMap V_1 V_1'}
    (hf : ComputableMapS P_0.toScott P_0'.toScott f)
    (hg : ComputableMapS P_1.toScott P_1'.toScott g) :
    ComputableMapS (osumPresentation h_0 h_1 (P_0 := P_0) (P_1 := P_1))
      (osumPresentation h_0' h_1' (P_0 := P_0') (P_1 := P_1')) (osumMap h_0' h_1' f g)
     := by
  constructor
  have mT0 : RecDecidable (fun t => t.unpair.2.unpair.1 = 0) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.right) (Nat.Primrec.const 0)
  have mT1 : RecDecidable (fun t => t.unpair.2.unpair.1 = 1) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.right) (Nat.Primrec.const 1)
  have mP0 : RecDecidable (fun t => P_0'.X t.unpair.2.unpair.2  !=  V_0'.master) :=
    (proper_0_dec P_0').comp (Nat.Primrec.right.comp Nat.Primrec.right)
  have mP1 : RecDecidable (fun t => P_1'.X t.unpair.2.unpair.2  !=  V_1'.master) :=
    (proper_1_dec P_1').comp (Nat.Primrec.right.comp Nat.Primrec.right)
  have nT0 : RecDecidable (fun t => t.unpair.1.unpair.1 = 0) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.left) (Nat.Primrec.const 0)
  have nT1 : RecDecidable (fun t => t.unpair.1.unpair.1 = 1) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.left) (Nat.Primrec.const 1)
  have nP0 : RecDecidable (fun t => P_0.X t.unpair.1.unpair.2  !=  V_0.master) :=
    (proper_0_dec P_0).comp (Nat.Primrec.right.comp Nat.Primrec.left)
  have nP1 : RecDecidable (fun t => P_1.X t.unpair.1.unpair.2  !=  V_1.master) :=
    (proper_1_dec P_1).comp (Nat.Primrec.right.comp Nat.Primrec.left)
  have Hf : REPred (fun t => f.rel (P_0.toScott.X t.unpair.1.unpair.2)
      (P_0'.toScott.X t.unpair.2.unpair.2)) :=
    REPred.of_iff (fun t => by simp only [unpair_pair_fst, unpair_pair_snd])
      (hf.out.comp ((Nat.Primrec.right.comp Nat.Primrec.left).pair
        (Nat.Primrec.right.comp Nat.Primrec.right)))
  have Hg : REPred (fun t => g.rel (P_1.toScott.X t.unpair.1.unpair.2)
      (P_1'.toScott.X t.unpair.2.unpair.2)) :=
    REPred.of_iff (fun t => by simp only [unpair_pair_fst, unpair_pair_snd])
      (hg.out.comp ((Nat.Primrec.right.comp Nat.Primrec.left).pair
        (Nat.Primrec.right.comp Nat.Primrec.right)))
  have hImp : RecDecidable (fun t => not (t.unpair.2.unpair.1 = 0  /\  P_0'.X t.unpair
    .2.unpair.2  !=  V_0'.master)
       /\  not (t.unpair.2.unpair.1 = 1  /\  P_1'.X t.unpair.2.unpair.2  !=  V_1'.
    master)) :=
    (mT0.and mP0).not.and (mT1.and mP1).not
  refine REPred.of_iff (fun t => ?_)
    ((hImp.re).or ((((mT0.and mP0).and (nT0.and nP0)).re.and Hf).or
      (((mT1.and mP1).and (nT1.and nP1)).re.and Hg)))
  show (osumMap h_0' h_1' f g).rel (osumEnum P_0 P_1 t.unpair.1) (osumEnum P_0' P_1' t
    .unpair.2)  <->  _
  by_cases hm0 : t.unpair.2.unpair.1 = 0  /\  P_0'.X t.unpair.2.unpair.2  !=  V_0'.
    master
  * rw [osumEnum_left hm0]
    constructor
    * rintro <-, -, hd>
      rcases hd with hmaster | <X, Y', hnX, hW'eq, -, hf> | <Y, Y', -, hW'eq, -, ->
      * exact absurd hmaster inj_0_ne_sumMaster
```

```
      * obtain rfl : Y' = P_0'.X t.unpair.2.unpair.2 := (inj_0_injective hW'eq).symm
        by_cases hn0 : t.unpair.1.unpair.1 = 0  /\  P_0.X t.unpair.1.unpair.2  !=  V_0
    .master
        * rw [osumEnum_left hn0] at hnX
          obtain rfl : P_0.X t.unpair.1.unpair.2 = X := inj_0_injective hnX
          exact Or.inr (Or.inl <<hm0, hn0>, hf>)
        * exfalso
          by_cases hn1 : t.unpair.1.unpair.1 = 1  /\  P_1.X t.unpair.1.unpair.2  !=
    V_1.master
          * rw [osumEnum_right hn1] at hnX
            obtain <b, hb> := h_1 _ (P_1.mem_X _)
            exact ir_mem_inj_0 (hnX  |>  ir_mem_inj_1.mpr hb)
          * rw [osumEnum_master hn0 hn1] at hnX
            exact none_mem_inj_0 (hnX  |>  none_mem_sumMaster)
      * obtain <a, ha> := h_0' _ (P_0'.mem_X _)
        exact (il_mem_inj_1 (hW'eq  |>  il_mem_inj_0.mpr ha)).elim
    * rintro (himp | <<-, hn0>, hf> | <<hm1, ->, ->)
      * exact absurd hm0 himp.1
      * exact <osumEnum_mem h_0 h_1 _, Or.inr (Or.inl <_, P_0'.mem_X _, hm0.2, rfl>),
          Or.inr (Or.inl <_, _, osumEnum_left hn0, rfl, hm0.2, hf>)>
      * obtain <he, -> := hm1; rw [hm0.1] at he; exact absurd he (by decide)
  * by_cases hm1 : t.unpair.2.unpair.1 = 1  /\  P_1'.X t.unpair.2.unpair.2  !=  V_1'.
    master
    * rw [osumEnum_right hm1]
      constructor
      * rintro <-, -, hd>
        rcases hd with hmaster | <X, Y', -, hW'eq, -, -> | <Y, Y', hnY, hW'eq, -, hg>
        * exact absurd hmaster inj_1_ne_sumMaster
        * exact absurd hW'eq.symm (inj_0_ne_inj_1_of_nonempty (h_1' _ (P_1'.mem_X _)))
        * obtain rfl : Y' = P_1'.X t.unpair.2.unpair.2 := (inj_1_injective hW'eq).symm
          by_cases hn1 : t.unpair.1.unpair.1 = 1  /\  P_1.X t.unpair.1.unpair.2  !=
    V_1.master
          * rw [osumEnum_right hn1] at hnY
            obtain rfl : P_1.X t.unpair.1.unpair.2 = Y := inj_1_injective hnY
            exact Or.inr (Or.inr <<hm1, hn1>, hg>)
          * exfalso
            by_cases hn0 : t.unpair.1.unpair.1 = 0  /\  P_0.X t.unpair.1.unpair.2  !=
     V_0.master
            * rw [osumEnum_left hn0] at hnY
              obtain <a, ha> := h_0 _ (P_0.mem_X _)
              exact il_mem_inj_1 (hnY  |>  il_mem_inj_0.mpr ha)
            * rw [osumEnum_master hn0 hn1] at hnY
              exact none_mem_inj_1 (hnY  |>  none_mem_sumMaster)
      * rintro (himp | <<hm0', ->, -> | <<-, hn1>, hg>)
        * exact absurd hm1 himp.2
        * obtain <he, -> := hm0'; rw [hm1.1] at he; exact absurd he (by decide)
        * exact <osumEnum_mem h_0 h_1 _, Or.inr (Or.inr <_, P_1'.mem_X _, hm1.2, rfl>)
    ,
          Or.inr (Or.inr <_, _, osumEnum_right hn1, rfl, hm1.2, hg>)>
    * rw [osumEnum_master hm0 hm1]
      constructor
      * intro _; exact Or.inl <hm0, hm1>
      * intro _
        exact <osumEnum_mem h_0 h_1 _, (osum h_0' h_1').master_mem, Or.inl rfl>

end CoalescedFunctor
```

```
/-- **`out_0` is computable.** `(osumEnum n) (out_0) X?_m  <->  leftPart (osumEnum n)
    subseteq  X?_m`; the left
part is `X?_{n.2}` on a proper left copy and `Delta_0` on a right copy / the master, so
     the relation is
`incl` against either `n.2` or the master index `k_0` (`X?_{k_0} = Delta_0`). -/
theorem osumOutMap_0_isComputable :
    ComputableMapS (osumPresentation h_0 h_1 (P_0 := P_0) (P_1 := P_1)) P_0.toScott
    osumOutMap_0 := by
  constructor
  obtain <k0, hk0> := P_0.surj V_0.master_mem
  have hT0 : RecDecidable (fun t => t.unpair.1.unpair.1 = 0) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.left) (Nat.Primrec.const 0)
  have hP0 : RecDecidable (fun t => P_0.X t.unpair.1.unpair.2  !=  V_0.master) :=
    (proper_0_dec P_0).comp (Nat.Primrec.right.comp Nat.Primrec.left)
  have hIncL : RecDecidable (fun t => P_0.X t.unpair.1.unpair.2  subseteq  P_0.X t.
    unpair.2) :=
    RecDecidable.of_iff (fun t => by simp only [unpair_pair_fst, unpair_pair_snd])
      (P_0.incl_computable.comp ((Nat.Primrec.right.comp Nat.Primrec.left).pair Nat.
    Primrec.right))
  have hIncM : RecDecidable (fun t => V_0.master  subseteq  P_0.X t.unpair.2) :=
    RecDecidable.of_iff (fun t => by simp only [unpair_pair_fst, unpair_pair_snd, hk0
    ])
      (P_0.incl_computable.comp ((Nat.Primrec.const k0).pair Nat.Primrec.right))
  refine (RecDecidable.of_iff (fun t => ?_)
    ((hT0.and (hP0.and hIncL)).or ((hT0.and hP0).not.and hIncM))).re
  show (osumOutMap_0 (h_0 := h_0) (h_1 := h_1)).rel (osumEnum P_0 P_1 t.unpair.1) (P_0
    .X t.unpair.2)  <->  _
  rw [show (osumOutMap_0 (h_0 := h_0) (h_1 := h_1)).rel (osumEnum P_0 P_1 t.unpair.1)
    (P_0.X t.unpair.2)
       <->  leftPart V_0 (osumEnum P_0 P_1 t.unpair.1)  subseteq  P_0.X t.unpair.2
    from
    <fun h => h.2.2, fun h => <osumEnum_mem h_0 h_1 _, P_0.mem_X _, h>>]
  by_cases hL : t.unpair.1.unpair.1 = 0  /\  P_0.X t.unpair.1.unpair.2  !=  V_0.master
  * rw [osumEnum_left hL, leftPart_inj_0]
    exact <fun h => Or.inl <hL.1, hL.2, h>,
      fun h => h.elim (fun h => h.2.2) (fun h => absurd hL h.1)>
  * by_cases hR : t.unpair.1.unpair.1 = 1  /\  P_1.X t.unpair.1.unpair.2  !=  V_1.
    master
    * rw [osumEnum_right hR, leftPart_inj_1 V_0 (h_1 _ (P_1.mem_X _))]
      exact <fun h => Or.inr <hL, h>,
        fun h => h.elim (fun h => absurd <h.1, h.2.1> hL) (fun h => h.2)>
    * rw [osumEnum_master hL hR, leftPart_sumMaster]
      exact <fun h => Or.inr <hL, h>,
        fun h => h.elim (fun h => absurd <h.1, h.2.1> hL) (fun h => h.2)>

/-- **`out_1` is computable.** Symmetric to `out_0` via `rightPart`. -/
theorem osumOutMap_1_isComputable :
    ComputableMapS (osumPresentation h_0 h_1 (P_0 := P_0) (P_1 := P_1)) P_1.toScott
    osumOutMap_1 := by
  constructor
  obtain <k1, hk1> := P_1.surj V_1.master_mem
  have hT1 : RecDecidable (fun t => t.unpair.1.unpair.1 = 1) :=
    RecDecidable.natEq (Nat.Primrec.left.comp Nat.Primrec.left) (Nat.Primrec.const 1)
  have hP1 : RecDecidable (fun t => P_1.X t.unpair.1.unpair.2  !=  V_1.master) :=
    (proper_1_dec P_1).comp (Nat.Primrec.right.comp Nat.Primrec.left)
```

```
  have hIncR : RecDecidable (fun t => P_1.X t.unpair.1.unpair.2  subseteq  P_1.X t.
    unpair.2) :=
    RecDecidable.of_iff (fun t => by simp only [unpair_pair_fst, unpair_pair_snd])
      (P_1.incl_computable.comp ((Nat.Primrec.right.comp Nat.Primrec.left).pair Nat.
    Primrec.right))
  have hIncM : RecDecidable (fun t => V_1.master  subseteq  P_1.X t.unpair.2) :=
    RecDecidable.of_iff (fun t => by simp only [unpair_pair_fst, unpair_pair_snd, hk1
    ])
      (P_1.incl_computable.comp ((Nat.Primrec.const k1).pair Nat.Primrec.right))
  refine (RecDecidable.of_iff (fun t => ?_)
    ((hT1.and (hP1.and hIncR)).or ((hT1.and hP1).not.and hIncM))).re
  show (osumOutMap_1 (h_0 := h_0) (h_1 := h_1)).rel (osumEnum P_0 P_1 t.unpair.1) (P_1
    .X t.unpair.2)  <->  _
  rw [show (osumOutMap_1 (h_0 := h_0) (h_1 := h_1)).rel (osumEnum P_0 P_1 t.unpair.1)
    (P_1.X t.unpair.2)
       <->  rightPart V_1 (osumEnum P_0 P_1 t.unpair.1)  subseteq  P_1.X t.unpair.2
    from
    <fun h => h.2.2, fun h => <osumEnum_mem h_0 h_1 _, P_1.mem_X _, h>>]
  by_cases hR : t.unpair.1.unpair.1 = 1  /\  P_1.X t.unpair.1.unpair.2  !=  V_1.master
  * rw [osumEnum_right hR, rightPart_inj_1]
    exact <fun h => Or.inl <hR.1, hR.2, h>,
      fun h => h.elim (fun h => h.2.2) (fun h => absurd hR h.1)>
  * by_cases hL : t.unpair.1.unpair.1 = 0  /\  P_0.X t.unpair.1.unpair.2  !=  V_0.
    master
    * rw [osumEnum_left hL, rightPart_inj_0 V_1 (h_0 _ (P_0.mem_X _))]
      exact <fun h => Or.inr <hR, h>,
        fun h => h.elim (fun h => absurd <h.1, h.2.1> hR) (fun h => h.2)>
    * rw [osumEnum_master hL hR, rightPart_sumMaster]
      exact <fun h => Or.inr <hR, h>,
        fun h => h.elim (fun h => absurd <h.1, h.2.1> hR) (fun h => h.2)>

end Coalesced

section Coalesced

variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
  (h_0 : forall  X, V_0.mem X  ->  X.Nonempty) (h_1 : forall  Y, V_1.mem Y  ->  Y.
    Nonempty)

variable (P_0 : ComputablePresentation V_0) (P_1 : ComputablePresentation V_1)

variable {P_0 P_1}

include h_0 h_1 in
/-- **Exercise 7.15 (Scott 1981, PRG-19) -- the coalesced sum `?_0 ? ?_1` is
    effectively given.**
(Scott's Definition 7.1; the only classical input is the enumeration's properness
    branch.) -/
theorem osum_isEffectivelyGivenS (e_0 : V_0.IsEffectivelyGiven) (e_1 : V_1.
    IsEffectivelyGiven) :
    (osum h_0 h_1).IsEffectivelyGivenS := by
  obtain <Q_0> := e_0; obtain <Q_1> := e_1
  exact <osumPresentation h_0 h_1 (P_0 := Q_0) (P_1 := Q_1)>

end Coalesced
```

```
section Coalesced

variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
  (h_0 : forall  X, V_0.mem X  ->  X.Nonempty) (h_1 : forall  Y, V_1.mem Y  ->  Y.
    Nonempty)

theorem osum_mem_iff {W : Set (Option (a ? b))} :
    (osum h_0 h_1).mem W  <->  W = sumMaster V_0 V_1  \/
      (exists  X, V_0.mem X  /\  X  !=  V_0.master  /\  W = inj_0 X)  \/
      (exists  Y, V_1.mem Y  /\  Y  !=  V_1.master  /\  W = inj_1 Y) := Iff.rfl

end Coalesced

section Iterate

open Domain.Recursive

variable {V : NeighborhoodSystem a} (P : ComputablePresentation V)

variable {P}

theorem projN_isComputable (P : ComputablePresentation V) (n : Nat) :
    ComputableMap (iterPresentation P) P (projN V n) := by
  constructor
  have hincl : RecDecidable (fun s => P.X s.unpair.1  subseteq  P.X s.unpair.2) := P.
    incl_computable
  have hr : Nat.Primrec (fun t => Nat.pair (nthCode t.unpair.1 n P.masterIdx) t.unpair
    .2) :=
    ((primrec_nthCode.comp (Nat.Primrec.left.pair
      ((Nat.Primrec.const n).pair (Nat.Primrec.const P.masterIdx)))).of_eq
        (fun t => by simp only [unpair_pair_fst, unpair_pair_snd])).pair Nat.Primrec.
    right
  refine (RecDecidable.of_iff (fun t => ?_) (hincl.comp hr)).re
  simp only [iterPresentation_X, projN_rel, fiber_iterEnum, iterIdx, unpair_pair_fst,
    unpair_pair_snd]
  exact <fun h => h.2.2, fun h => <iterEnum_mem _, P.mem_X _, h>>

end Iterate

section Smash

variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
  (P_0 : ComputablePresentation V_0) (P_1 : ComputablePresentation V_1)

variable {P_0 P_1}

variable (P_0 P_1)

section SmashFunctor

variable {a' b' : Type*} {V_0' : NeighborhoodSystem a'} {V_1' : NeighborhoodSystem b'}

/-- **`f ? g` is computable.** Writing each smash code via its *effective* product
    index
(`smashEnum_eq_eff`), the relation `(smashEnum n) (f?g) (smashEnum' m)` decodes to: `m`
     improper
```

```
(the codomain master absorbs everything), or `m` proper together with `f`'s and `g`'s r
    .e. relations
on the effective coordinate indices. R.e.-ness then follows from `hf`, `hg`, the
    codomain properness
deciders, and closure of r.e. predicates under ` \/ `/` /\ `/primitive-recursive
    substitution. -/
theorem smashMap_isComputable (P_0' : ComputablePresentation V_0') (P_1' :
    ComputablePresentation V_1')
    {f : ApproximableMap V_0 V_0'} {g : ApproximableMap V_1 V_1'}
    (hf : ComputableMapS P_0.toScott P_0'.toScott f)
    (hg : ComputableMapS P_1.toScott P_1'.toScott g) :
    ComputableMapS (smashPresentation P_0 P_1) (smashPresentation P_0' P_1') (smashMap
     f g) := by
  constructor
  obtain <f_0, hf_0p, hf_0e> := proper_0_dec P_0
  obtain <f_1, hf_1p, hf_1e> := proper_1_dec P_1
  set c_0 := fun n => isOne (f_0 n) with hc_0def
  set c_1 := fun n => isOne (f_1 n) with hc_1def

  have hc_0le : forall  n, c_0 n  <=  1 := fun n => isOne_le_one _
  have hc_1le : forall  n, c_1 n  <=  1 := fun n => isOne_le_one _
  have hc_0 : forall  n, P_0.X n  !=  V_0.master  <->  c_0 n = 1 := fun n => by
    rw [hc_0def, isOne_eq_one_iff]; exact hf_0e n
  have hc_1 : forall  n, P_1.X n  !=  V_1.master  <->  c_1 n = 1 := fun n => by
    rw [hc_1def, isOne_eq_one_iff]; exact hf_1e n
  have hc_0p : Nat.Primrec c_0 := primrec_isOne.comp hf_0p
  have hc_1p : Nat.Primrec c_1 := primrec_isOne.comp hf_1p
  have hpf : Nat.Primrec (fun n => c_0 n.unpair.1 * c_1 n.unpair.2) :=
    (primrec_mul.comp ((hc_0p.comp Nat.Primrec.left).pair (hc_1p.comp Nat.Primrec.
    right))).of_eq
      (fun n => by simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd])
  have heffLp : Nat.Primrec (fun n => selectFn (c_0 n.unpair.1 * c_1 n.unpair.2)
      n.unpair.1 P_0.masterIdx) :=
    primrec_selectFn hpf Nat.Primrec.left (Nat.Primrec.const _)
  have heffRp : Nat.Primrec (fun n => selectFn (c_0 n.unpair.1 * c_1 n.unpair.2)
      n.unpair.2 P_1.masterIdx) :=
    primrec_selectFn hpf Nat.Primrec.right (Nat.Primrec.const _)
  -- `f`'s r.e. relation on `(effL n, m.1)`; `g`'s on `(effR n, m.2)`
  have Hf : REPred (fun t => f.rel (P_0.toScott.X
      (selectFn (c_0 t.unpair.1.unpair.1 * c_1 t.unpair.1.unpair.2)
        t.unpair.1.unpair.1 P_0.masterIdx)) (P_0'.toScott.X t.unpair.2.unpair.1)) :=
    REPred.of_iff (fun t => by simp only [unpair_pair_fst, unpair_pair_snd])
      (hf.out.comp ((heffLp.comp Nat.Primrec.left).pair
        (Nat.Primrec.left.comp Nat.Primrec.right)))
  have Hg : REPred (fun t => g.rel (P_1.toScott.X
      (selectFn (c_0 t.unpair.1.unpair.1 * c_1 t.unpair.1.unpair.2)
        t.unpair.1.unpair.2 P_1.masterIdx)) (P_1'.toScott.X t.unpair.2.unpair.2)) :=
    REPred.of_iff (fun t => by simp only [unpair_pair_fst, unpair_pair_snd])
      (hg.out.comp ((heffRp.comp Nat.Primrec.left).pair
        (Nat.Primrec.right.comp Nat.Primrec.right)))
  have hJP : RecDecidable (fun t => P_0'.X t.unpair.2.unpair.1  !=  V_0'.master  /\
      P_1'.X t.unpair.2.unpair.2  !=  V_1'.master) :=
    ((proper_0_dec P_0').comp (Nat.Primrec.left.comp Nat.Primrec.right)).and
      ((proper_1_dec P_1').comp (Nat.Primrec.right.comp Nat.Primrec.right))
  refine REPred.of_iff (fun t => ?_) ((hJP.not.re).or (hJP.re.and (Hf.and Hg)))
  show (smashMap f g).rel (smashEnum P_0 P_1 t.unpair.1) (smashEnum P_0' P_1' t.unpair
```

```
    .2)  <->  _
  have heq : smashEnum P_0 P_1 t.unpair.1 = prodNbhd
      (P_0.X (selectFn (c_0 t.unpair.1.unpair.1 * c_1 t.unpair.1.unpair.2)
        t.unpair.1.unpair.1 P_0.masterIdx))
      (P_1.X (selectFn (c_0 t.unpair.1.unpair.1 * c_1 t.unpair.1.unpair.2)
        t.unpair.1.unpair.2 P_1.masterIdx)) := smashEnum_eq_eff hc_0le hc_1le hc_0
    hc_1 t.unpair.1
  by_cases hJPm : P_0'.X t.unpair.2.unpair.1  !=  V_0'.master  /\  P_1'.X t.unpair.2.
    unpair.2  !=  V_1'.master
  * constructor
    * rintro <-, -, hd>
      rcases hd with hmaster | <A, B, A', B', hWAB, hW'AB, -, -, hfr, hgr>
      * rw [smashEnum_proper hJPm] at hmaster
        exact absurd (prodNbhd_injective hmaster).1 hJPm.1
      * rw [smashEnum_proper hJPm] at hW'AB
        obtain <rfl, rfl> := prodNbhd_injective (heq.symm.trans hWAB)
        obtain <rfl, rfl> := prodNbhd_injective hW'AB
        exact Or.inr <hJPm, hfr, hgr>
    * rintro (hnp | <-, hfr, hgr>)
      * exact absurd hJPm hnp
      * exact <smashEnum_mem (P_0 := P_0) (P_1 := P_1) t.unpair.1,
          smashEnum_mem (P_0 := P_0') (P_1 := P_1') t.unpair.2,
          Or.inr <_, _, _, _, heq, smashEnum_proper hJPm, hJPm.1, hJPm.2, hfr, hgr>>
  * constructor
    * intro _; exact Or.inl hJPm
    * intro _
      exact <smashEnum_mem (P_0 := P_0) (P_1 := P_1) t.unpair.1,
        smashEnum_mem (P_0 := P_0') (P_1 := P_1') t.unpair.2,
        Or.inl (smashEnum_master hJPm)>

end SmashFunctor

/-- **The strict pairing `<a, b>?` is computable.** `Z_n <a,b>? (smashEnum m)  <-> `
    either `m` improper
(the codomain master absorbs) or `m` proper together with `a`'s and `b`'s r.e.
    relations on the
(raw) codomain coordinate indices -- an ` \/ `/` /\ ` of r.e. predicates and the
    codomain properness tests.
No effective-index bridge is needed: the proper branch reads `smashEnum`'s raw factors
    directly. -/
theorem smashPaired_isComputable {V_2 : NeighborhoodSystem ?} (P_2 :
    ComputablePresentation V_2)
    {a : ApproximableMap V_2 V_0} {b : ApproximableMap V_2 V_1}
    (ha : ComputableMapS P_2.toScott P_0.toScott a)
    (hb : ComputableMapS P_2.toScott P_1.toScott b) :
    ComputableMapS P_2.toScott (smashPresentation P_0 P_1) (smashPaired a b) := by
  constructor
  have Ha : REPred (fun t => a.rel (P_2.toScott.X t.unpair.1) (P_0.toScott.X t.unpair
    .2.unpair.1)) :=
    REPred.of_iff (fun t => by simp only [unpair_pair_fst, unpair_pair_snd])
      (ha.out.comp (Nat.Primrec.left.pair (Nat.Primrec.left.comp Nat.Primrec.right)))
  have Hb : REPred (fun t => b.rel (P_2.toScott.X t.unpair.1) (P_1.toScott.X t.unpair
    .2.unpair.2)) :=
    REPred.of_iff (fun t => by simp only [unpair_pair_fst, unpair_pair_snd])
      (hb.out.comp (Nat.Primrec.left.pair (Nat.Primrec.right.comp Nat.Primrec.right)))
  have hJP : RecDecidable (fun t => P_0.X t.unpair.2.unpair.1  !=  V_0.master  /\
```

```
      P_1.X t.unpair.2.unpair.2  !=  V_1.master) :=
    ((proper_0_dec P_0).comp (Nat.Primrec.left.comp Nat.Primrec.right)).and
      ((proper_1_dec P_1).comp (Nat.Primrec.right.comp Nat.Primrec.right))
  refine REPred.of_iff (fun t => ?_) ((hJP.not.re).or (hJP.re.and (Ha.and Hb)))
  show (smashPaired a b).rel (P_2.X t.unpair.1) (smashEnum P_0 P_1 t.unpair.2)  <->  _
  by_cases hJPm : P_0.X t.unpair.2.unpair.1  !=  V_0.master  /\  P_1.X t.unpair.2.
    unpair.2  !=  V_1.master
  * constructor
    * rintro <-, -, hd>
      rcases hd with hmaster | <A, B, hWAB, -, -, ha', hb'>
      * rw [smashEnum_proper hJPm] at hmaster
        exact absurd (prodNbhd_injective hmaster).1 hJPm.1
      * rw [smashEnum_proper hJPm] at hWAB
        obtain <rfl, rfl> := prodNbhd_injective hWAB
        exact Or.inr <hJPm, ha', hb'>
    * rintro (hnp | <-, ha', hb'>)
      * exact absurd hJPm hnp
      * exact <P_2.mem_X _, smashEnum_mem (P_0 := P_0) (P_1 := P_1) _,
          Or.inr <_, _, smashEnum_proper hJPm, hJPm.1, hJPm.2, ha', hb'>>
  * constructor
    * intro _; exact Or.inl hJPm
    * intro _
      exact <P_2.mem_X _, smashEnum_mem (P_0 := P_0) (P_1 := P_1) _,
        Or.inl (smashEnum_master hJPm)>

/-- **The smash projection `p_0` is computable.** `(smashEnum n) p_0 X?_m  <->  X?_{
    effL n}  subseteq  X?_m`
(`smashEnum_eq_eff` rewrites `Sum.inl ^-1' (smashEnum n)` to the effective left factor)
    , a recursive
slice of `incl_computable`. -/
theorem smashProj_0_isComputable :
    ComputableMapS (smashPresentation P_0 P_1) P_0.toScott (smashProj_0 V_0 V_1) := by
  constructor
  obtain <f_0, hf_0p, hf_0e> := proper_0_dec P_0
  obtain <f_1, hf_1p, hf_1e> := proper_1_dec P_1
  set c_0 := fun n => isOne (f_0 n) with hc_0def
  set c_1 := fun n => isOne (f_1 n) with hc_1def
  have hc_0le : forall  n, c_0 n  <=  1 := fun n => isOne_le_one _
  have hc_1le : forall  n, c_1 n  <=  1 := fun n => isOne_le_one _
  have hc_0 : forall  n, P_0.X n  !=  V_0.master  <->  c_0 n = 1 := fun n => by
    rw [hc_0def, isOne_eq_one_iff]; exact hf_0e n
  have hc_1 : forall  n, P_1.X n  !=  V_1.master  <->  c_1 n = 1 := fun n => by
    rw [hc_1def, isOne_eq_one_iff]; exact hf_1e n
  have hc_0p : Nat.Primrec c_0 := primrec_isOne.comp hf_0p
  have hc_1p : Nat.Primrec c_1 := primrec_isOne.comp hf_1p
  have hpf : Nat.Primrec (fun n => c_0 n.unpair.1 * c_1 n.unpair.2) :=
    (primrec_mul.comp ((hc_0p.comp Nat.Primrec.left).pair (hc_1p.comp Nat.Primrec.
    right))).of_eq
      (fun n => by simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd])
  have heffLp : Nat.Primrec (fun n => selectFn (c_0 n.unpair.1 * c_1 n.unpair.2)
      n.unpair.1 P_0.masterIdx) :=
    primrec_selectFn hpf Nat.Primrec.left (Nat.Primrec.const _)
  have hincl : RecDecidable (fun s => P_0.X s.unpair.1  subseteq  P_0.X s.unpair.2) :=
     P_0.incl_computable
  have hr : Nat.Primrec (fun t => Nat.pair
      (selectFn (c_0 t.unpair.1.unpair.1 * c_1 t.unpair.1.unpair.2) t.unpair.1.unpair
```

```
    .1 P_0.masterIdx)
      t.unpair.2) := (heffLp.comp Nat.Primrec.left).pair Nat.Primrec.right
  refine (RecDecidable.of_iff (fun t => ?_) (hincl.comp hr)).re
  show (smashProj_0 V_0 V_1).rel (smashEnum P_0 P_1 t.unpair.1) (P_0.X t.unpair.2)
    <->  _
  simp only [smashProj_0_rel, unpair_pair_fst, unpair_pair_snd]
  constructor
  * rintro <-, -, hsub>
    rw [smashEnum_eq_eff hc_0le hc_1le hc_0 hc_1, inl_preimage_prodNbhd] at hsub
    exact hsub
  * intro h
    refine <smashEnum_mem (P_0 := P_0) (P_1 := P_1) _, P_0.mem_X _, ?_>
    rw [smashEnum_eq_eff hc_0le hc_1le hc_0 hc_1, inl_preimage_prodNbhd]
    exact h

/-- **The smash projection `p_1` is computable.** Symmetric to `
    smashProj_0_isComputable`, using the
effective right factor `Sum.inr ^-1' (smashEnum n) = X1_{effR n}`. -/
theorem smashProj_1_isComputable :
    ComputableMapS (smashPresentation P_0 P_1) P_1.toScott (smashProj_1 V_0 V_1) := by
  constructor
  obtain <f_0, hf_0p, hf_0e> := proper_0_dec P_0
  obtain <f_1, hf_1p, hf_1e> := proper_1_dec P_1
  set c_0 := fun n => isOne (f_0 n) with hc_0def
  set c_1 := fun n => isOne (f_1 n) with hc_1def
  have hc_0le : forall  n, c_0 n  <=  1 := fun n => isOne_le_one _
  have hc_1le : forall  n, c_1 n  <=  1 := fun n => isOne_le_one _
  have hc_0 : forall  n, P_0.X n  !=  V_0.master  <->  c_0 n = 1 := fun n => by
    rw [hc_0def, isOne_eq_one_iff]; exact hf_0e n
  have hc_1 : forall  n, P_1.X n  !=  V_1.master  <->  c_1 n = 1 := fun n => by
    rw [hc_1def, isOne_eq_one_iff]; exact hf_1e n
  have hc_0p : Nat.Primrec c_0 := primrec_isOne.comp hf_0p
  have hc_1p : Nat.Primrec c_1 := primrec_isOne.comp hf_1p
  have hpf : Nat.Primrec (fun n => c_0 n.unpair.1 * c_1 n.unpair.2) :=
    (primrec_mul.comp ((hc_0p.comp Nat.Primrec.left).pair (hc_1p.comp Nat.Primrec.
    right))).of_eq
      (fun n => by simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd])
  have heffRp : Nat.Primrec (fun n => selectFn (c_0 n.unpair.1 * c_1 n.unpair.2)
      n.unpair.2 P_1.masterIdx) :=
    primrec_selectFn hpf Nat.Primrec.right (Nat.Primrec.const _)
  have hincl : RecDecidable (fun s => P_1.X s.unpair.1  subseteq  P_1.X s.unpair.2) :=
     P_1.incl_computable
  have hr : Nat.Primrec (fun t => Nat.pair
      (selectFn (c_0 t.unpair.1.unpair.1 * c_1 t.unpair.1.unpair.2) t.unpair.1.unpair
    .2 P_1.masterIdx)
      t.unpair.2) := (heffRp.comp Nat.Primrec.left).pair Nat.Primrec.right
  refine (RecDecidable.of_iff (fun t => ?_) (hincl.comp hr)).re
  show (smashProj_1 V_0 V_1).rel (smashEnum P_0 P_1 t.unpair.1) (P_1.X t.unpair.2)
    <->  _
  simp only [smashProj_1_rel, unpair_pair_fst, unpair_pair_snd]
  constructor
  * rintro <-, -, hsub>
    rw [smashEnum_eq_eff hc_0le hc_1le hc_0 hc_1, inr_preimage_prodNbhd] at hsub
    exact hsub
  * intro h
    refine <smashEnum_mem (P_0 := P_0) (P_1 := P_1) _, P_1.mem_X _, ?_>
```

```
    rw [smashEnum_eq_eff hc_0le hc_1le hc_0 hc_1, inr_preimage_prodNbhd]
    exact h

/-- **Exercise 7.15 (Scott 1981, PRG-19) -- the smash product `?_0 ? ?_1` is
    effectively given.**
(Scott's Definition 7.1; the only classical input is the enumeration's properness
    branch.) -/
theorem smash_isEffectivelyGivenS (h_0 : V_0.IsEffectivelyGiven) (h_1 : V_1.
    IsEffectivelyGiven) :
    (smash V_0 V_1).IsEffectivelyGivenS := by
  obtain <Q_0> := h_0; obtain <Q_1> := h_1
  exact <smashPresentation Q_0 Q_1>

end Smash

end

section
set_option linter.unusedVariables false

theorem example_715.{u_1, u_2} : And
  (And
    (forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
    a},
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven a V  ->
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven (Prod
     Nat a)
          (@DomainTheory.NeighbourhoodSystems.iterSys a V))
    (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem a}
      {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
      (h_0 : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.mem a V_0 X  ->  @Set.Nonempty a X)
      (h_1 : forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.mem b V_1 Y  ->  @Set.Nonempty b Y),
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven a V_0
    ->
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven b V_1
      ->
          @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGivenS (
    Option (Sum a b))
            (@DomainTheory.NeighbourhoodSystems.osum a b V_0 V_1 h_0 h_1)))
  (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem a}
    {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b},
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven a V_0  ->
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven b V_1
    ->
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGivenS (Sum
     a b)
          (@DomainTheory.NeighbourhoodSystems.Exercise510.smash a b V_0 V_1)) :=
  @And.intro
  (And
    (forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
    a},
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven a V  ->
```

```
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven (Prod
     Nat a)
           (@DomainTheory.NeighbourhoodSystems.iterSys a V))
     (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem a}
       {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
       (h_0 : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.mem a V_0 X  ->  @Set.Nonempty a X)
       (h_1 : forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.mem b V_1 Y  ->  @Set.Nonempty b Y),
       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven a V_0
    ->
         @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven b V_1
      ->
           @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGivenS (
    Option (Sum a b))
            (@DomainTheory.NeighbourhoodSystems.osum a b V_0 V_1 h_0 h_1)))
  (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem a}
    {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b},
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven a V_0  ->
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven b V_1
    ->
         @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGivenS (Sum
     a b)
           (@DomainTheory.NeighbourhoodSystems.Exercise510.smash a b V_0 V_1))
  (@And.intro
    (forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
    a},
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven a V  ->
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven (Prod
     Nat a)
           (@DomainTheory.NeighbourhoodSystems.iterSys a V))
     (forall  {a : Type u_1} {b : Type u_2} {V_0 : DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem a}
       {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
       (h_0 : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.mem a V_0 X  ->  @Set.Nonempty a X)
       (h_1 : forall  (Y : Set b), @DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.mem b V_1 Y  ->  @Set.Nonempty b Y),
       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven a V_0
    ->
         @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven b V_1
      ->
           @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGivenS (
    Option (Sum a b))
            (@DomainTheory.NeighbourhoodSystems.osum a b V_0 V_1 h_0 h_1))
     @DomainTheory.NeighbourhoodSystems.iterSys_isEffectivelyGiven
     @DomainTheory.NeighbourhoodSystems.osum_isEffectivelyGivenS)
  @DomainTheory.NeighbourhoodSystems.smash_isEffectivelyGivenS
end

namespace Exercise718
open NeighborhoodSystem Domain.Recursive ApproximableMap

end Exercise718
```

```
end DomainTheory.NeighbourhoodSystems
end
```

3.8.18 Exercise 7.16 — curry as an enumeration

Let $D_0 = \{X_n \mid n \in \mathbf{N}\}$, $D_1 = \{Y_m \mid m \in \mathbf{N}\}$ and $D_2 = \{Z_k \mid k \in \mathbf{N}\}$ be three effectively given domains. Complete the proof of 7.5 by writing out curry as a relation between neighbourhoods. Is it a recursive set or only a recursively enumerable set?

Exercise_7_16.lean

```
module

public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.Table_5_5

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Proposition_7_12

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Theorem_7_5

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem Domain.Recursive ApproximableMap
variable {a b ? : Type*} {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
  {V_2 : NeighborhoodSystem ?}

/- concordance: source L4645 EXERCISE 7.16 curry as an enumeration -/
/-- **Exercise 7.16 -- `curry` is a *recursive* set.** Relative to the function-space
    presentations
`PA` of `(?_0 x ?_1 -> ?_2)` and `PB` of `(?_0 -> (?_1 -> ?_2))` (and the inner one `Pc`
     of `(?_1 -> ?_2)`) built by
Theorem 7.5, the `curry` combinator's neighbourhood relation is **recursively decidable
    **:

  `Xenum_PA n curryComb Xenum_PB m  ?  gNb m = 1  ->  forall  e  in  ?m?, gNc e_2 = 1
    ->
                                          forall  e'  in  ?e_2?, X_PA n  subseteq
    X_PA (curryStepCode e_1 e')`,

a bounded double-`forall ` over the coded entry-lists of the recursively *decidable*
    function-space
inclusion. So `curry` is a recursive set -- exactly as for `eval`. -/
theorem curryComb_rel_recDecidable
    (P_0 : ComputablePresentation V_0) (P_1 : ComputablePresentation V_1)
    (P_2 : ComputablePresentation V_2) (gNa gNb gNc : Nat  ->  Nat)
    (PA : ComputablePresentation (funSpace (prod V_0 V_1) V_2))
    (PB : ComputablePresentation (funSpace V_0 (funSpace V_1 V_2)))
    (Pc : ComputablePresentation (funSpace V_1 V_2))
    (hPAX : PA.X = Xenum (prodPresentation P_0 P_1) P_2 gNa)
    (hPBX : PB.X = Xenum P_0 Pc gNb)
    (hPcX : Pc.X = Xenum P_1 P_2 gNc)
```

```
  (hgNa : forall  c, gNa c = 1  <->
    (stepFun (funListOf (prodPresentation P_0 P_1) P_2 (decodeList c))
      : Set (ApproximableMap (prod V_0 V_1) V_2)).Nonempty)
  (hgNc : forall  c, gNc c = 1  <->
    (stepFun (funListOf P_1 P_2 (decodeList c)) : Set (ApproximableMap V_1 V_2)).
  Nonempty)
  (hgNbp : Nat.Primrec gNb) (hgNcp : Nat.Primrec gNc) :
  RecDecidable_2 (fun n m => (curryC V_0 V_1 V_2).rel (PA.X n) (PB.X m)) := by
-- The explicit neighbourhood relation, written out over the coded entries.
have hcomb : forall  n m, (curryC V_0 V_1 V_2).rel (PA.X n) (PB.X m)  <->
    (gNb m = 1  ->  forall  e  in  decodeList m, gNc e.unpair.2 = 1  ->
      forall  e'  in  decodeList e.unpair.2, PA.X n  subseteq  PA.X (curryStepCode e
  .unpair.1 e')) := by
  intro n m
  have hstep1 : (curryC V_0 V_1 V_2).rel (PA.X n) (PB.X m)  <->
      curry (toApproxMap ((funSpace (prod V_0 V_1) V_2).principal (PA.mem_X n)))  in
    PB.X m := by
    constructor
    * rintro <hG, hH>
      rw [curryIso_apply, mem_toFilter] at hH
      exact hH.2
    * intro h
      exact <PA.mem_X n, by rw [curryIso_apply, mem_toFilter]; exact <PB.mem_X m, h
  >>
  rw [hstep1, hPBX, mem_Xenum_iff_map P_0 Pc gNb
    (curry (toApproxMap ((funSpace (prod V_0 V_1) V_2).principal (PA.mem_X n)))) m]
  apply imp_congr_right; intro _
  apply forall_congr'; intro e
  apply imp_congr_right; intro _
  rw [hPcX, curry_rel_Xenum_iff P_0 P_1 P_2 gNc hgNc
    (toApproxMap ((funSpace (prod V_0 V_1) V_2).principal (PA.mem_X n))) e.unpair.1
  e.unpair.2]
  apply imp_congr_right; intro _
  apply forall_congr'; intro e'
  apply imp_congr_right; intro _
  have hstepeq : step (prodNbhd (P_0.X e.unpair.1) (P_1.X e'.unpair.1)) (P_2.X e'.
  unpair.2)
      = PA.X (curryStepCode e.unpair.1 e') := by
    have hs := Xenum_singleton (prodPresentation P_0 P_1) P_2 gNa hgNa
      (Nat.pair e.unpair.1 e'.unpair.1) e'.unpair.2
    rw [prodPresentation_X, unpair_pair_fst, unpair_pair_snd] at hs
    rw [hPAX]; exact hs.symm
  rw [toApproxMap_rel, mem_principal, hstepeq]
  exact <fun h => h.2, fun h => <PA.mem_X _, h>>
-- The right-hand side is recursively decidable.
have q1_dec : RecDecidable_2 (fun e' p =>
    PA.X p.unpair.1  subseteq  PA.X (curryStepCode p.unpair.2 e')) := by
  have hr : Nat.Primrec (fun w => Nat.pair w.unpair.2.unpair.1
      (curryStepCode w.unpair.2.unpair.2 w.unpair.1)) :=
    (Nat.Primrec.left.comp Nat.Primrec.right).pair
      (primrec_curryStepCode (Nat.Primrec.right.comp Nat.Primrec.right) Nat.Primrec.
  left)
  refine RecDecidable.of_iff (fun w => ?_) (RecDecidable.comp PA.incl_computable hr)
  simp only [unpair_pair_fst, unpair_pair_snd]
have dInner := q1_dec.bForallList
have dMid : RecDecidable_2 (fun e n =>
```

```
      gNc e.unpair.2 = 1  ->  forall  e'  in  decodeList e.unpair.2,
        PA.X n  subseteq  PA.X (curryStepCode e.unpair.1 e')) := by
    have hg_mid : Nat.Primrec (fun s => Nat.pair s.unpair.1.unpair.2
        (Nat.pair s.unpair.2 s.unpair.1.unpair.1)) :=
      (Nat.Primrec.right.comp Nat.Primrec.left).pair
        (Nat.Primrec.right.pair (Nat.Primrec.left.comp Nat.Primrec.left))
    have B1 : RecDecidable (fun s => forall  e'  in  decodeList s.unpair.1.unpair.2,
        PA.X s.unpair.2  subseteq  PA.X (curryStepCode s.unpair.1.unpair.1 e')) := by
      refine RecDecidable.of_iff (fun s => ?_) (RecDecidable.comp dInner hg_mid)
      simp only [unpair_pair_fst, unpair_pair_snd]
    have guard1 : RecDecidable (fun s => gNc s.unpair.1.unpair.2 = 1) :=
      RecDecidable.natEq (hgNcp.comp (Nat.Primrec.right.comp Nat.Primrec.left))
        (Nat.Primrec.const 1)
    refine RecDecidable.of_iff (fun s => ?_) (guard1.not.or B1)
    exact Decidable.imp_iff_not_or
  have dOuter := dMid.bForallList
  refine RecDecidable.of_iff (fun t => hcomb t.unpair.1 t.unpair.2) ?_
  have hswap2 : Nat.Primrec (fun t => Nat.pair t.unpair.2 t.unpair.1) :=
    Nat.Primrec.right.pair Nat.Primrec.left
  have B2 : RecDecidable (fun t => forall  e  in  decodeList t.unpair.2, gNc e.unpair
    .2 = 1  ->
      forall  e'  in  decodeList e.unpair.2, PA.X t.unpair.1  subseteq  PA.X (
    curryStepCode e.unpair.1 e')) := by
    refine RecDecidable.of_iff (fun t => ?_) (RecDecidable.comp dOuter hswap2)
    simp only [unpair_pair_fst, unpair_pair_snd]
  have guard2 : RecDecidable (fun t => gNb t.unpair.2 = 1) :=
    RecDecidable.natEq (hgNbp.comp Nat.Primrec.right) (Nat.Primrec.const 1)
  refine RecDecidable.of_iff (fun t => ?_) (guard2.not.or B2)
  exact Decidable.imp_iff_not_or

/-- **Exercise 7.16 -- `curry` is computable, and in fact a recursive set.** If `?_0, ?
    _1, ?_2` are
effectively given, then there are computable presentations of `(?_0 x ?_1 -> ?_2)` and
    `(?_0 -> (?_1 -> ?_2))`
relative to which the `curry` combinator's neighbourhood relation is **recursively
    decidable**
(`RecDecidable_2`) -- hence `curry` is a computable map (`ComputableMap`, the r.e.
    condition of
Definition 7.2). The answer to Scott's question: `curry` is a *recursive* set, not
    merely r.e. -/
theorem curryComb_isComputable
    (P_0 : ComputablePresentation V_0) (P_1 : ComputablePresentation V_1)
    (P_2 : ComputablePresentation V_2) :
    exists  (PA : ComputablePresentation (funSpace (prod V_0 V_1) V_2))
      (PB : ComputablePresentation (funSpace V_0 (funSpace V_1 V_2))),
      RecDecidable_2 (fun n m => (curryC V_0 V_1 V_2).rel (PA.X n) (PB.X m))  /\
        ComputableMap PA PB (curryC V_0 V_1 V_2) := by
  obtain <incl0, hincl0p, hincl0s> := P_0.incl_computable
  obtain <fc0, hfc0p, hfc0s> := P_0.cons_computable
  obtain <incl1, hincl1p, hincl1s> := P_1.incl_computable
  obtain <fc1, hfc1p, hfc1s> := P_1.cons_computable
  obtain <incl2, hincl2p, hincl2s> := P_2.incl_computable
  obtain <eq2, heq2p, heq2s> := P_2.eq_computable
  obtain <fc2, hfc2p, hfc2s> := P_2.cons_computable
  obtain <inclpr, hinclprp, hinclprs> := (prodPresentation P_0 P_1).incl_computable
  obtain <fcpr, hfcprp, hfcprs> := (prodPresentation P_0 P_1).cons_computable
```

```
  -- inner function space `Pc = (?_1  ->  ?_2)`
  let Pc : ComputablePresentation (funSpace V_1 V_2) :=
    funPresentation P_1 P_2 (funConsChar P_1 P_2 fc1 fc2) incl1 incl2 eq2
      (funConsChar_spec P_1 P_2 fc1 fc2 (fun s => (hfc1s s).symm) (fun s => (hfc2s s).
    symm))
      (primrec_funConsChar P_1 P_2 fc1 fc2 hfc1p hfc2p)
      (fun s => (hincl1s s).symm) hincl1p (fun s => (hincl2s s).symm) hincl2p
      (fun s => (heq2s s).symm) heq2p
  obtain <inclc, hinclcp, hinclcs> := Pc.incl_computable
  obtain <eqc, heqcp, heqcs> := Pc.eq_computable
  obtain <fcc, hfccp, hfccs> := Pc.cons_computable
  -- domain function space `PA = (?_0 x ?_1  ->  ?_2)`
  let PA : ComputablePresentation (funSpace (prod V_0 V_1) V_2) :=
    funPresentation (prodPresentation P_0 P_1) P_2
      (funConsChar (prodPresentation P_0 P_1) P_2 fcpr fc2) inclpr incl2 eq2
      (funConsChar_spec (prodPresentation P_0 P_1) P_2 fcpr fc2
        (fun s => (hfcprs s).symm) (fun s => (hfc2s s).symm))
      (primrec_funConsChar (prodPresentation P_0 P_1) P_2 fcpr fc2 hfcprp hfc2p)
      (fun s => (hinclprs s).symm) hinclprp (fun s => (hincl2s s).symm) hincl2p
      (fun s => (heq2s s).symm) heq2p
  -- codomain function space `PB = (?_0  ->  (?_1  ->  ?_2))`
  let PB : ComputablePresentation (funSpace V_0 (funSpace V_1 V_2)) :=
    funPresentation P_0 Pc (funConsChar P_0 Pc fc0 fcc) incl0 inclc eqc
      (funConsChar_spec P_0 Pc fc0 fcc (fun s => (hfc0s s).symm) (fun s => (hfccs s).
    symm))
      (primrec_funConsChar P_0 Pc fc0 fcc hfc0p hfccp)
      (fun s => (hincl0s s).symm) hincl0p (fun s => (hinclcs s).symm) hinclcp
      (fun s => (heqcs s).symm) heqcp
  have hRec : RecDecidable_2 (fun n m => (curryC V_0 V_1 V_2).rel (PA.X n) (PB.X m))
    :=
    curryComb_rel_recDecidable P_0 P_1 P_2
      (funConsChar (prodPresentation P_0 P_1) P_2 fcpr fc2) (funConsChar P_0 Pc fc0
    fcc)
      (funConsChar P_1 P_2 fc1 fc2) PA PB Pc rfl rfl rfl
      (funConsChar_spec (prodPresentation P_0 P_1) P_2 fcpr fc2
        (fun s => (hfcprs s).symm) (fun s => (hfc2s s).symm))
      (funConsChar_spec P_1 P_2 fc1 fc2 (fun s => (hfc1s s).symm) (fun s => (hfc2s s).
    symm))
      (primrec_funConsChar P_0 Pc fc0 fcc hfc0p hfccp)
      (primrec_funConsChar P_1 P_2 fc1 fc2 hfc1p hfc2p)
  exact <PA, PB, hRec, <hRec.re>>

/-- The least map of a function-space neighbourhood `G` -- `toApproxMap G` -- belongs
    to `G`. -/
theorem toApproxMap_principal_mem {G : Set (ApproximableMap V_0 V_1)}
    (hG : (funSpace V_0 V_1).mem G) : toApproxMap ((funSpace V_0 V_1).principal hG)
    in  G := by
  obtain <L, hL, rfl> := hG.1
  intro p hp
  show (toApproxMap ((funSpace V_0 V_1).principal hG)).rel p.1 p.2
  rw [toApproxMap_rel, mem_principal]
  exact <step_mem (hL p hp).1 (hL p hp).2, fun f hf => hf p hp>

/-- **Exercise 7.16 -- `curry` written out as a relation between neighbourhoods.** The
    combinator
relates a neighbourhood `G` of `(?_0 x ?_1 -> ?_2)` to a neighbourhood `H` of `(?_0 ->
```

```
    (?_1 -> ?_2))` exactly when
`curry g  in  H` for every `g  in  G`. -/
theorem curryComb_rel {G : Set (ApproximableMap (prod V_0 V_1) V_2)}
    {H : Set (ApproximableMap V_0 (funSpace V_1 V_2))} :
    (curryC V_0 V_1 V_2).rel G H  <->  (funSpace (prod V_0 V_1) V_2).mem G  /\
      (funSpace V_0 (funSpace V_1 V_2)).mem H  /\  forall  g  in  G, curry g  in  H :=
     by
  constructor
  * rintro <hG, hH>
    rw [curryIso_apply, mem_toFilter] at hH
    obtain <hHmem, hcurry> := hH
    refine <hG, hHmem, fun g hg => ?_>
    have hle : toApproxMap ((funSpace (prod V_0 V_1) V_2).principal hG)  <=  g := by
      rw [ApproximableMap.le_iff]
      intro X Y hrel
      rw [toApproxMap_rel, mem_principal] at hrel
      exact hrel.2 hg
    have hcle : curry (toApproxMap ((funSpace (prod V_0 V_1) V_2).principal hG))  <=
    curry g :=
      (curryEquiv V_0 V_1 V_2).monotone hle
    exact funSpace_mem_up_closed hHmem hcurry hcle
  * rintro <hG, hH, hall>
    refine <hG, ?_>
    rw [curryIso_apply, mem_toFilter]
    exact <hH, hall _ (toApproxMap_principal_mem hG)>

end

section
set_option linter.unusedVariables false

theorem example_716.{u_1, u_2, u_3} : And
  (And
    (forall  {a : Type u_1} {b : Type u_2} {? : Type u_3} {V_0 : DomainTheory.
   NeighbourhoodSystems.NeighborhoodSystem a}
      {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
      {V_2 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem ?}
      (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0)
      (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1)
      (P_2 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation ? V_2),
      exists  (PA :
        @DomainTheory.NeighbourhoodSystems.ComputablePresentation
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
            (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2)
          (@DomainTheory.NeighbourhoodSystems.funSpace (Sum a b) ? (@DomainTheory.
   NeighbourhoodSystems.prod a b V_0 V_1)
            V_2))
        (PB :
        @DomainTheory.NeighbourhoodSystems.ComputablePresentation
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
            (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
          (@DomainTheory.NeighbourhoodSystems.funSpace a (@DomainTheory.
   NeighbourhoodSystems.ApproximableMap b ? V_1 V_2)
            V_0 (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))),
        And
```

```
      (Domain.Recursive.RecDecidable_2 fun (n m : Nat) =>
        @DomainTheory.NeighbourhoodSystems.ApproximableMap.rel
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
            (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2)
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
            (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
          (@DomainTheory.NeighbourhoodSystems.funSpace (Sum a b) ?
            (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2)
          (@DomainTheory.NeighbourhoodSystems.funSpace a
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
            (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
          (@DomainTheory.NeighbourhoodSystems.curryC a b ? V_0 V_1 V_2)
          (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
              (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2)
            (@DomainTheory.NeighbourhoodSystems.funSpace (Sum a b) ?
              (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2)
            PA n)
          (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
              (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
            (@DomainTheory.NeighbourhoodSystems.funSpace a
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
              (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
            PB m))
      (@DomainTheory.NeighbourhoodSystems.ComputableMap
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
          (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2)
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
          (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
        (@DomainTheory.NeighbourhoodSystems.funSpace (Sum a b) ? (@DomainTheory.
NeighbourhoodSystems.prod a b V_0 V_1)
          V_2)
        (@DomainTheory.NeighbourhoodSystems.funSpace a
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
          (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
        PA PB (@DomainTheory.NeighbourhoodSystems.curryC a b ? V_0 V_1 V_2)))
 (forall  {a : Type u_1} {b : Type u_2} {? : Type u_3} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
   {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
   {V_2 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem ?}
   {G :
     Set
       (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
         (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2)}
   {H :
     Set
       (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
         (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
         (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))},
   Iff
     (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel
       (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
```

```
        (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2)
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
        (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
      (@DomainTheory.NeighbourhoodSystems.funSpace (Sum a b) ? (@DomainTheory.
NeighbourhoodSystems.prod a b V_0 V_1)
       V_2)
      (@DomainTheory.NeighbourhoodSystems.funSpace a (@DomainTheory.
NeighbourhoodSystems.ApproximableMap b ? V_1 V_2)
       V_0 (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
      (@DomainTheory.NeighbourhoodSystems.curryC a b ? V_0 V_1 V_2) G H)
    (And
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
          (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2)
        (@DomainTheory.NeighbourhoodSystems.funSpace (Sum a b) ? (@DomainTheory.
NeighbourhoodSystems.prod a b V_0 V_1)
         V_2)
        G)
      (And
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
            (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
          (@DomainTheory.NeighbourhoodSystems.funSpace a
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
            (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
          H)
        (forall
          (g :
            @DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
              (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2),
          @Membership.mem
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
                (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2)
              (Set
                (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
                  (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2))
              (@Set.instMembership
                (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
                  (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2))
              G g  ->
            @Membership.mem
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
                (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2)
V_0
                (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
              (Set
                (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
                  (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2)
 V_0
                  (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2)))
              (@Set.instMembership
                (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
                  (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2)
 V_0
```

```
                    (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2)))
                H (@DomainTheory.NeighbourhoodSystems.curry a b ? V_0 V_1 V_2 g)))))
 )
(forall  {a : Type u_1} {b : Type u_2} {? : Type u_3} {V_0 : DomainTheory.
 NeighbourhoodSystems.NeighborhoodSystem a}
 {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
 {V_2 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem ?}
 (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0)
 (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1)
 (P_2 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation ? V_2) (gNa gNb
 gNc : Nat  ->  Nat)
 (PA :
   @DomainTheory.NeighbourhoodSystems.ComputablePresentation
     (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
       (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2)
     (@DomainTheory.NeighbourhoodSystems.funSpace (Sum a b) ? (@DomainTheory.
 NeighbourhoodSystems.prod a b V_0 V_1)
       V_2))
 (PB :
   @DomainTheory.NeighbourhoodSystems.ComputablePresentation
     (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
       (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
       (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
     (@DomainTheory.NeighbourhoodSystems.funSpace a (@DomainTheory.
 NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
       (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2)))
 (Pc :
   @DomainTheory.NeighbourhoodSystems.ComputablePresentation
     (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2)
     (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2)),
 @Eq
     (Nat  ->
       Set
         (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
           (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2))
     (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X
       (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
         (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2)
       (@DomainTheory.NeighbourhoodSystems.funSpace (Sum a b) ? (@DomainTheory.
 NeighbourhoodSystems.prod a b V_0 V_1)
         V_2)
       PA)
     (@DomainTheory.NeighbourhoodSystems.Xenum (Sum a b) ? (@DomainTheory.
 NeighbourhoodSystems.prod a b V_0 V_1) V_2
       (@DomainTheory.NeighbourhoodSystems.prodPresentation a b V_0 V_1 P_0 P_1)
 P_2 gNa)  ->
   @Eq
       (Nat  ->
         Set
           (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
             (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
             (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2)))
       (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X
         (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
           (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
           (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
```

```
        (@DomainTheory.NeighbourhoodSystems.funSpace a
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
          (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
        PB)
      (@DomainTheory.NeighbourhoodSystems.Xenum a (@DomainTheory.
NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
        (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2) P_0 Pc gNb)  ->
    @Eq (Nat  ->  Set (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1
V_2))
        (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2)
          (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2) Pc)
        (@DomainTheory.NeighbourhoodSystems.Xenum b ? V_1 V_2 P_1 P_2 gNc)  ->
      (forall  (c : Nat),
         Iff (@Eq Nat (gNa c) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (
nat_lit 1))))
             (@Set.Nonempty
               (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
                 (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2)


               (@DomainTheory.NeighbourhoodSystems.stepFun (Sum a b) ?
                 (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2
                 (@DomainTheory.NeighbourhoodSystems.funListOf (Sum a b) ?
                   (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2
                   (@DomainTheory.NeighbourhoodSystems.prodPresentation a b V_0 V_1
 P_0 P_1) P_2
                   (Domain.Recursive.decodeList c)))))  ->
        (forall  (c : Nat),
           Iff (@Eq Nat (gNc c) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (
nat_lit 1))))
               (@Set.Nonempty (@DomainTheory.NeighbourhoodSystems.ApproximableMap b
 ? V_1 V_2)
                 (@DomainTheory.NeighbourhoodSystems.stepFun b ? V_1 V_2
                   (@DomainTheory.NeighbourhoodSystems.funListOf b ? V_1 V_2 P_1
P_2
                     (Domain.Recursive.decodeList c)))))  ->
          Nat.Primrec gNb  ->
            Nat.Primrec gNc  ->
              Domain.Recursive.RecDecidable_2 fun (n m : Nat) =>
                @DomainTheory.NeighbourhoodSystems.ApproximableMap.rel
                  (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
                    (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2)
                  (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
                    (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1
V_2) V_0
                    (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
                  (@DomainTheory.NeighbourhoodSystems.funSpace (Sum a b) ?
                    (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2)
                  (@DomainTheory.NeighbourhoodSystems.funSpace a
                    (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1
V_2) V_0
                    (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
                  (@DomainTheory.NeighbourhoodSystems.curryC a b ? V_0 V_1 V_2)
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X
                    (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b)
?
```

```
                        (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2)
                      (@DomainTheory.NeighbourhoodSystems.funSpace (Sum a b) ?
                        (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2)
                      PA n)
                    (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X
                      (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
                        (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1
  V_2) V_0
                        (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
                      (@DomainTheory.NeighbourhoodSystems.funSpace a
                        (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1
  V_2) V_0
                        (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
                      PB m)) :=
@And.intro
(And
  (forall  {a : Type u_1} {b : Type u_2} {? : Type u_3} {V_0 : DomainTheory.
  NeighbourhoodSystems.NeighborhoodSystem a}
    {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
    {V_2 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem ?}
    (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0)
    (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1)
    (P_2 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation ? V_2),
    exists  (PA :
      @DomainTheory.NeighbourhoodSystems.ComputablePresentation
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
          (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2)
        (@DomainTheory.NeighbourhoodSystems.funSpace (Sum a b) ? (@DomainTheory.
  NeighbourhoodSystems.prod a b V_0 V_1)
          V_2))
      (PB :
      @DomainTheory.NeighbourhoodSystems.ComputablePresentation
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
          (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
        (@DomainTheory.NeighbourhoodSystems.funSpace a (@DomainTheory.
  NeighbourhoodSystems.ApproximableMap b ? V_1 V_2)
          V_0 (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))),
      And
        (Domain.Recursive.RecDecidable_2 fun (n m : Nat) =>
          @DomainTheory.NeighbourhoodSystems.ApproximableMap.rel
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
              (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2)
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
              (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
            (@DomainTheory.NeighbourhoodSystems.funSpace (Sum a b) ?
              (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2)
            (@DomainTheory.NeighbourhoodSystems.funSpace a
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
              (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
            (@DomainTheory.NeighbourhoodSystems.curryC a b ? V_0 V_1 V_2)
            (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
                (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2)
              (@DomainTheory.NeighbourhoodSystems.funSpace (Sum a b) ?
```

```
              (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2)
            PA n)
          (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
              (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
            (@DomainTheory.NeighbourhoodSystems.funSpace a
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
              (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
            PB m))
      (@DomainTheory.NeighbourhoodSystems.ComputableMap
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
          (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2)
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
          (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
        (@DomainTheory.NeighbourhoodSystems.funSpace (Sum a b) ? (@DomainTheory.
NeighbourhoodSystems.prod a b V_0 V_1)
          V_2)
        (@DomainTheory.NeighbourhoodSystems.funSpace a
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
          (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
        PA PB (@DomainTheory.NeighbourhoodSystems.curryC a b ? V_0 V_1 V_2)))
(forall  {a : Type u_1} {b : Type u_2} {? : Type u_3} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
  {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
  {V_2 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem ?}
  {G :
    Set
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
        (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2)}
  {H :
    Set
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
        (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))},
  Iff
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
        (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2)
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
        (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
      (@DomainTheory.NeighbourhoodSystems.funSpace (Sum a b) ? (@DomainTheory.
NeighbourhoodSystems.prod a b V_0 V_1)
        V_2)
      (@DomainTheory.NeighbourhoodSystems.funSpace a (@DomainTheory.
NeighbourhoodSystems.ApproximableMap b ? V_1 V_2)
        V_0 (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
      (@DomainTheory.NeighbourhoodSystems.curryC a b ? V_0 V_1 V_2) G H)
    (And
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
          (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2)
        (@DomainTheory.NeighbourhoodSystems.funSpace (Sum a b) ? (@DomainTheory.
NeighbourhoodSystems.prod a b V_0 V_1)
```

```
          V_2)
        G)
      (And
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
            (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
          (@DomainTheory.NeighbourhoodSystems.funSpace a
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
            (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
          H)
        (forall
          (g :
            @DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
              (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2),
          @Membership.mem
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
                (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2)
              (Set
                (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
                  (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2))
              (@Set.instMembership
                (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
                  (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2))
              G g  ->
            @Membership.mem
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
                (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2)
 V_0
                (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
              (Set
                (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
                  (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2)
  V_0
                  (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2)))
              (@Set.instMembership
                (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
                  (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2)
  V_0
                  (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2)))
             H (@DomainTheory.NeighbourhoodSystems.curry a b ? V_0 V_1 V_2 g)))))
 )
(forall  {a : Type u_1} {b : Type u_2} {? : Type u_3} {V_0 : DomainTheory.
 NeighbourhoodSystems.NeighborhoodSystem a}
 {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
 {V_2 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem ?}
 (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0)
 (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1)
 (P_2 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation ? V_2) (gNa gNb
 gNc : Nat  ->  Nat)
 (PA :
   @DomainTheory.NeighbourhoodSystems.ComputablePresentation
     (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
       (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2)
     (@DomainTheory.NeighbourhoodSystems.funSpace (Sum a b) ? (@DomainTheory.
 NeighbourhoodSystems.prod a b V_0 V_1)
```

```
      V_2))
(PB :
  @DomainTheory.NeighbourhoodSystems.ComputablePresentation
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
      (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
    (@DomainTheory.NeighbourhoodSystems.funSpace a (@DomainTheory.
NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
      (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2)))
(Pc :
  @DomainTheory.NeighbourhoodSystems.ComputablePresentation
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2)
    (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2)),
@Eq
    (Nat  ->
      Set
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
          (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2))
    (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
        (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2)
      (@DomainTheory.NeighbourhoodSystems.funSpace (Sum a b) ? (@DomainTheory.
NeighbourhoodSystems.prod a b V_0 V_1)
        V_2)
      PA)
    (@DomainTheory.NeighbourhoodSystems.Xenum (Sum a b) ? (@DomainTheory.
NeighbourhoodSystems.prod a b V_0 V_1) V_2
      (@DomainTheory.NeighbourhoodSystems.prodPresentation a b V_0 V_1 P_0 P_1)
P_2 gNa)  ->
  @Eq
      (Nat  ->
        Set
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
            (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2)))
      (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
          (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
        (@DomainTheory.NeighbourhoodSystems.funSpace a
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
          (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
        PB)
      (@DomainTheory.NeighbourhoodSystems.Xenum a (@DomainTheory.
NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
        (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2) P_0 Pc gNb)  ->
    @Eq (Nat  ->  Set (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1
V_2))
        (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2)
          (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2) Pc)
        (@DomainTheory.NeighbourhoodSystems.Xenum b ? V_1 V_2 P_1 P_2 gNc)  ->
      (forall  (c : Nat),
         Iff (@Eq Nat (gNa c) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (
nat_lit 1))))
           (@Set.Nonempty
```

```
                (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
                  (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2)
                (@DomainTheory.NeighbourhoodSystems.stepFun (Sum a b) ?
                  (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2
                  (@DomainTheory.NeighbourhoodSystems.funListOf (Sum a b) ?
                    (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2
                    (@DomainTheory.NeighbourhoodSystems.prodPresentation a b V_0 V_1
 P_0 P_1) P_2
                    (Domain.Recursive.decodeList c)))))  ->
          (forall  (c : Nat),
             Iff (@Eq Nat (gNc c) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (
nat_lit 1))))
                (@Set.Nonempty (@DomainTheory.NeighbourhoodSystems.ApproximableMap b
 ? V_1 V_2)
                  (@DomainTheory.NeighbourhoodSystems.stepFun b ? V_1 V_2
                    (@DomainTheory.NeighbourhoodSystems.funListOf b ? V_1 V_2 P_1
P_2
                      (Domain.Recursive.decodeList c)))))  ->
            Nat.Primrec gNb  ->
              Nat.Primrec gNc  ->
                Domain.Recursive.RecDecidable_2 fun (n m : Nat) =>
                  @DomainTheory.NeighbourhoodSystems.ApproximableMap.rel
                    (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
                      (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2)
                    (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
                      (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1
V_2) V_0
                      (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
                    (@DomainTheory.NeighbourhoodSystems.funSpace (Sum a b) ?
                      (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2)
                    (@DomainTheory.NeighbourhoodSystems.funSpace a
                      (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1
V_2) V_0
                      (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
                    (@DomainTheory.NeighbourhoodSystems.curryC a b ? V_0 V_1 V_2)
                    (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X
                      (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b)
?
                        (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2)
                      (@DomainTheory.NeighbourhoodSystems.funSpace (Sum a b) ?
                        (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2)
                      PA n)
                    (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X
                      (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
                        (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1
V_2) V_0
                        (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
                      (@DomainTheory.NeighbourhoodSystems.funSpace a
                        (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1
V_2) V_0
                        (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
                      PB m))
(@And.intro
  (forall  {a : Type u_1} {b : Type u_2} {? : Type u_3} {V_0 : DomainTheory.
  NeighbourhoodSystems.NeighborhoodSystem a}
    {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
```

```
  {V_2 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem ?}
  (P_0 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V_0)
  (P_1 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b V_1)
  (P_2 : @DomainTheory.NeighbourhoodSystems.ComputablePresentation ? V_2),
  exists  (PA :
    @DomainTheory.NeighbourhoodSystems.ComputablePresentation
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
        (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2)
      (@DomainTheory.NeighbourhoodSystems.funSpace (Sum a b) ? (@DomainTheory.
NeighbourhoodSystems.prod a b V_0 V_1)
        V_2))
    (PB :
    @DomainTheory.NeighbourhoodSystems.ComputablePresentation
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
        (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
      (@DomainTheory.NeighbourhoodSystems.funSpace a (@DomainTheory.
NeighbourhoodSystems.ApproximableMap b ? V_1 V_2)
        V_0 (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))),
    And
      (Domain.Recursive.RecDecidable_2 fun (n m : Nat) =>
        @DomainTheory.NeighbourhoodSystems.ApproximableMap.rel
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
            (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2)
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
            (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
          (@DomainTheory.NeighbourhoodSystems.funSpace (Sum a b) ?
            (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2)
          (@DomainTheory.NeighbourhoodSystems.funSpace a
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
            (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
          (@DomainTheory.NeighbourhoodSystems.curryC a b ? V_0 V_1 V_2)
          (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
              (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2)
            (@DomainTheory.NeighbourhoodSystems.funSpace (Sum a b) ?
              (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2)
            PA n)
          (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
              (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
            (@DomainTheory.NeighbourhoodSystems.funSpace a
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
              (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
            PB m))
      (@DomainTheory.NeighbourhoodSystems.ComputableMap
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
          (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2)
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
          (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
        (@DomainTheory.NeighbourhoodSystems.funSpace (Sum a b) ? (@DomainTheory.
NeighbourhoodSystems.prod a b V_0 V_1)
          V_2)
```

```
        (@DomainTheory.NeighbourhoodSystems.funSpace a
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
          (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
        PA PB (@DomainTheory.NeighbourhoodSystems.curryC a b ? V_0 V_1 V_2)))
(forall  {a : Type u_1} {b : Type u_2} {? : Type u_3} {V_0 : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
  {V_1 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
  {V_2 : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem ?}
  {G :
    Set
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
        (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2)}
  {H :
    Set
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
        (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))},
  Iff
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
        (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2)
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
        (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
      (@DomainTheory.NeighbourhoodSystems.funSpace (Sum a b) ? (@DomainTheory.
NeighbourhoodSystems.prod a b V_0 V_1)
        V_2)
      (@DomainTheory.NeighbourhoodSystems.funSpace a (@DomainTheory.
NeighbourhoodSystems.ApproximableMap b ? V_1 V_2)
        V_0 (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
      (@DomainTheory.NeighbourhoodSystems.curryC a b ? V_0 V_1 V_2) G H)
    (And
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
          (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2)
        (@DomainTheory.NeighbourhoodSystems.funSpace (Sum a b) ? (@DomainTheory.
NeighbourhoodSystems.prod a b V_0 V_1)
          V_2)
        G)
      (And
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
            (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
          (@DomainTheory.NeighbourhoodSystems.funSpace a
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2) V_0
            (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
          H)
        (forall
          (g :
            @DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
              (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2),
          @Membership.mem
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
                (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2)
              (Set
```

```
                    (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
                      (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2))
                  (@Set.instMembership
                    (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum a b) ?
                      (@DomainTheory.NeighbourhoodSystems.prod a b V_0 V_1) V_2))
                  G g  ->
                @Membership.mem
                  (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
                    (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2)
    V_0
                    (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2))
                  (Set
                    (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
                      (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2)
     V_0
                      (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2)))
                  (@Set.instMembership
                    (@DomainTheory.NeighbourhoodSystems.ApproximableMap a
                      (@DomainTheory.NeighbourhoodSystems.ApproximableMap b ? V_1 V_2)
     V_0
                      (@DomainTheory.NeighbourhoodSystems.funSpace b ? V_1 V_2)))
                  H (@DomainTheory.NeighbourhoodSystems.curry a b ? V_0 V_1 V_2 g)))))
    @DomainTheory.NeighbourhoodSystems.curryComb_isComputable @DomainTheory.
    NeighbourhoodSystems.curryComb_rel)

  @DomainTheory.NeighbourhoodSystems.curryComb_rel_recDecidable
end

end DomainTheory.NeighbourhoodSystems
end
```

3.8.19 Exercise 7.17 — recursion over lists

Complete the proof of 7.7 for showing that $D^{\S}$ is effectively given if D is. Include all the combinators of 6.2. Prove also that if E is effectively given and $u : D \to E$ and $v : E \times E \to E$ are computable, then the unique strict mapping $g : D^{\S} \to E$, where, for $x \in |D|$ and $y, z \in |E|$,

$$g(\mathrm{in}(x)) = u(x), \quad \text{and}$$

$$g(\mathrm{pair}(y, z)) = v(g(y), g(z)),$$

is a computable mapping.

Exercise_7_17.lean

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Exercise_7_18

@[expose] public section

section
```

```
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive Example61
open NeighborhoodSystem Domain.Recursive ApproximableMap
open NeighborhoodSystem Domain.Recursive ApproximableMap Example61
section
variable {a : Type*}
namespace Proposition77
variable {D : NeighborhoodSystem a} (hD : forall  X, D.mem X  ->  X.Nonempty)

/- concordance: source L4647 EXERCISE 7.17 recursion over lists -/
/-- **Exercise 7.17 (Scott 1981, PRG-19) -- the pairing `pair : D^S  x  D^S  ->  D^S`
    is computable.**
Its index relation is exactly `V_{2*<a,b>+2}  subseteq  V_k`, a recursive slice of `D^S
    `'s inclusion decider
(`dsharpPresentation.incl_computable`) reindexed by the primitive-recursive node map
`s  |->  <2*(p s)+2, q s>`, hence r.e. -/
theorem pairSharp_isComputable (P : ComputablePresentation D) :
    ComputableMap (prodPresentation (dsharpPresentation P hD) (dsharpPresentation P hD
    ))
      (dsharpPresentation P hD) (pairSharpMap hD) := by
  constructor
  show REPred (fun s => (pairSharpMap hD).rel
    ((prodPresentation (dsharpPresentation P hD) (dsharpPresentation P hD)).X s.unpair
    .1)
    ((dsharpPresentation P hD).X s.unpair.2))
  have hincl : RecDecidable (fun s => Vsharp D P s.unpair.1  subseteq  Vsharp D P s.
    unpair.2) :=
    (dsharpPresentation P hD).incl_computable
  have hr : Nat.Primrec (fun s => Nat.pair (2 * s.unpair.1 + 2) s.unpair.2) :=
    (primrec_add_2 (primrec_mul_2 (Nat.Primrec.const 2) Nat.Primrec.left)
      (Nat.Primrec.const 2)).pair Nat.Primrec.right
  refine (RecDecidable.of_iff (fun s => ?_) (hincl.comp hr)).re
  simp only [unpair_pair_fst, unpair_pair_snd]
  rw [pairSharp_rel_Vsharp_iff hD P s.unpair.1 s.unpair.2]

/-- The index characterization of `proj1Map`. `V_m proj_1 V_k` iff `k = 0` (`V_k =
    Gamma`) or `m` is an
even node index `2a+2` with `V_{q a}  subseteq  V_k` (the *right* child, `q a = a.
    unpair.2`). -/
theorem proj1_rel_Vsharp_iff (P : ComputablePresentation D) (m k : Nat) :
    (proj1Map hD).rel (Vsharp D P m) (Vsharp D P k)  <->
      k = 0  \/  (m % 2 = 0  /\  m  !=  0  /\  Vsharp D P (m / 2 - 1).unpair.2
    subseteq  Vsharp D P k) := by
  rw [proj1Map_rel, and_iff_right (Vsharp_mem P m), and_iff_right (Vsharp_mem P k)]
  refine or_congr (Vsharp_eq_Gamma_iff P hD k) ?_
  rcases nat_shape m with rfl | <a, rfl> | <a, rfl>
  * rw [Vsharp_zero]
    constructor
    * rintro <Pp, Qq, heq, ->; exact absurd heq.symm (embPair_ne_Gamma D hD Pp Qq)
    * rintro <-, hne, ->; exact absurd rfl hne
  * rw [Vsharp_odd]
    constructor
    * rintro <Pp, Qq, heq, ->; exact absurd heq (embZero_ne_embPair D hD (P.mem_X a)
    Pp Qq)
    * rintro <hmod, -, ->; exact absurd hmod (by omega)
  * rw [Vsharp_even, show (2 * a + 2) / 2 - 1 = a from by omega]
```

```
    constructor
    * rintro <Pp, Qq, heq, hQk>
      obtain <-, hQ> := embPair_injective heq
      exact <by omega, by omega, by rw [hQ]; exact hQk>
    * rintro <-, -, hsub>
      exact <Vsharp D P a.unpair.1, Vsharp D P a.unpair.2, rfl, hsub>

/-- **Exercise 7.17 (Scott 1981, PRG-19) -- `proj_1` is computable.**
`V_m proj_1 V_k  <->  k = 0  \/  (m even, m  !=  0, V_{q(m/2-1)}  subseteq  V_k)`: a
    disjunction of the equality and
inclusion deciders with primitive-recursive parity/right-child reindexings, hence r.e.
    -/
theorem proj1_isComputable (P : ComputablePresentation D) :
    ComputableMap (dsharpPresentation P hD) (dsharpPresentation P hD) (proj1Map hD) :=
     by
  constructor
  show REPred (fun s => (proj1Map hD).rel (Vsharp D P s.unpair.1) (Vsharp D P s.unpair
    .2))
  have hk0 : RecDecidable (fun s => s.unpair.2 = 0) :=
    RecDecidable.natEq Nat.Primrec.right (Nat.Primrec.const 0)
  have hmod : RecDecidable (fun s => s.unpair.1 % 2 = 0) :=
    RecDecidable.natEq (primrec_mod2.comp Nat.Primrec.left) (Nat.Primrec.const 0)
  have hne : RecDecidable (fun s => s.unpair.1  !=  0) :=
    (RecDecidable.natEq Nat.Primrec.left (Nat.Primrec.const 0)).not
  have hg : Nat.Primrec (fun s => Nat.pair (s.unpair.1 / 2 - 1).unpair.2 s.unpair.2)
    :=
    (Nat.Primrec.right.comp (primrec_sub_2 (primrec_div2.comp Nat.Primrec.left)
      (Nat.Primrec.const 1))).pair Nat.Primrec.right
  have hincl : RecDecidable (fun s => Vsharp D P (s.unpair.1 / 2 - 1).unpair.2
    subseteq
      Vsharp D P s.unpair.2) := by
    refine RecDecidable.of_iff (fun s => ?_) ((dsharpPresentation P hD).
    incl_computable.comp hg)
    simp only [dsharpPresentation_X, unpair_pair_fst, unpair_pair_snd]
  exact (RecDecidable.of_iff (fun s => proj1_rel_Vsharp_iff hD P s.unpair.1 s.unpair
    .2)
    (hk0.or (hmod.and (hne.and hincl)))).re

end Proposition77
end

end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.Proposition77
section
set_option linter.unusedVariables false

theorem example_717.{u_1} : And
  (forall  {a : Type u_1} {D : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
    (hD : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    mem a D X  ->  @Set.Nonempty a X)
    (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a D),
    @DomainTheory.NeighbourhoodSystems.ComputableMap (Sum (Prod (List Bool) a) (Prod (
    List Bool) a))
      (Prod (List Bool) a)
      (@DomainTheory.NeighbourhoodSystems.prod (Prod (List Bool) a) (Prod (List Bool)
    a)
```

```
      (@DomainTheory.NeighbourhoodSystems.Example61.Dsharp a D hD)
      (@DomainTheory.NeighbourhoodSystems.Example61.Dsharp a D hD))
    (@DomainTheory.NeighbourhoodSystems.Example61.Dsharp a D hD)
    (@DomainTheory.NeighbourhoodSystems.prodPresentation (Prod (List Bool) a) (Prod
  (List Bool) a)
      (@DomainTheory.NeighbourhoodSystems.Example61.Dsharp a D hD)
      (@DomainTheory.NeighbourhoodSystems.Example61.Dsharp a D hD)
      (@DomainTheory.NeighbourhoodSystems.Proposition77.dsharpPresentation a D P hD)
      (@DomainTheory.NeighbourhoodSystems.Proposition77.dsharpPresentation a D P hD)
  )
    (@DomainTheory.NeighbourhoodSystems.Proposition77.dsharpPresentation a D P hD)
    (@DomainTheory.NeighbourhoodSystems.Proposition77.pairSharpMap a D hD))
(forall  {a : Type u_1} {D : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
  (hD : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  mem a D X  ->  @Set.Nonempty a X)
  (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a D),
  @DomainTheory.NeighbourhoodSystems.ComputableMap (Prod (List Bool) a) (Prod (List
  Bool) a)
    (@DomainTheory.NeighbourhoodSystems.Example61.Dsharp a D hD)
    (@DomainTheory.NeighbourhoodSystems.Example61.Dsharp a D hD)
    (@DomainTheory.NeighbourhoodSystems.Proposition77.dsharpPresentation a D P hD)
    (@DomainTheory.NeighbourhoodSystems.Proposition77.dsharpPresentation a D P hD)
    (@DomainTheory.NeighbourhoodSystems.Proposition77.proj1Map a D hD)) :=
@And.intro
(forall  {a : Type u_1} {D : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
  (hD : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  mem a D X  ->  @Set.Nonempty a X)
  (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a D),
  @DomainTheory.NeighbourhoodSystems.ComputableMap (Sum (Prod (List Bool) a) (Prod (
  List Bool) a))
    (Prod (List Bool) a)
    (@DomainTheory.NeighbourhoodSystems.prod (Prod (List Bool) a) (Prod (List Bool)
  a)
      (@DomainTheory.NeighbourhoodSystems.Example61.Dsharp a D hD)
      (@DomainTheory.NeighbourhoodSystems.Example61.Dsharp a D hD))
    (@DomainTheory.NeighbourhoodSystems.Example61.Dsharp a D hD)
    (@DomainTheory.NeighbourhoodSystems.prodPresentation (Prod (List Bool) a) (Prod
  (List Bool) a)
      (@DomainTheory.NeighbourhoodSystems.Example61.Dsharp a D hD)
      (@DomainTheory.NeighbourhoodSystems.Example61.Dsharp a D hD)
      (@DomainTheory.NeighbourhoodSystems.Proposition77.dsharpPresentation a D P hD)
      (@DomainTheory.NeighbourhoodSystems.Proposition77.dsharpPresentation a D P hD)
  )
    (@DomainTheory.NeighbourhoodSystems.Proposition77.dsharpPresentation a D P hD)
    (@DomainTheory.NeighbourhoodSystems.Proposition77.pairSharpMap a D hD))
(forall  {a : Type u_1} {D : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
  (hD : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  mem a D X  ->  @Set.Nonempty a X)
  (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a D),
  @DomainTheory.NeighbourhoodSystems.ComputableMap (Prod (List Bool) a) (Prod (List
  Bool) a)
    (@DomainTheory.NeighbourhoodSystems.Example61.Dsharp a D hD)
    (@DomainTheory.NeighbourhoodSystems.Example61.Dsharp a D hD)
    (@DomainTheory.NeighbourhoodSystems.Proposition77.dsharpPresentation a D P hD)
    (@DomainTheory.NeighbourhoodSystems.Proposition77.dsharpPresentation a D P hD)
    (@DomainTheory.NeighbourhoodSystems.Proposition77.proj1Map a D hD))
```

```
  @DomainTheory.NeighbourhoodSystems.Proposition77.pairSharp_isComputable
  @DomainTheory.NeighbourhoodSystems.Proposition77.proj1_isComputable
end

end DomainTheory.NeighbourhoodSystems.Proposition77
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem Domain.Recursive ApproximableMap Example61
variable {a : Type*}

namespace Proposition77

variable {D : NeighborhoodSystem a} (hD : forall  X, D.mem X  ->  X.Nonempty)

/-- **`proj1Map` really is the second projection of the pair part:** `proj_1(<x, y>^S)
    = y`. -/
theorem proj1_toElementMap_pairSharp (x y : (Dsharp D hD).Element) :
    (proj1Map hD).toElementMap (pairSharp D hD x y) = y := by
  apply Element.ext
  intro Z
  simp only [ApproximableMap.mem_toElementMap, proj1Map_rel]
  constructor
  * rintro <W, hWmem, hMW, hMZ, hdisj>
    rcases hWmem with rfl | <Pp, Qq, hxP, hyQ, rfl>
    * rcases hdisj with rfl | <Pp', Qq', hPe, ->
      * exact y.master_mem
      * exact absurd hPe.symm (embPair_ne_Gamma D hD Pp' Qq')
    * rcases hdisj with rfl | <Pp', Qq', hPe, hQ'Z>
      * exact y.master_mem
      * obtain <-, hQQ> := embPair_injective hPe
        exact y.up_mem hyQ hMZ (by rw [hQQ]; exact hQ'Z)
  * intro hyZ
    exact <embPair (Gamma D) Z, Or.inr <Gamma D, Z, x.master_mem, hyZ, rfl>,
      MemS.pair MemS.gamma (y.sub hyZ), y.sub hyZ, Or.inr <Gamma D, Z, rfl, subset_rfl
    >>

end Proposition77
end

section
open NeighborhoodSystem Domain.Recursive ApproximableMap Example61 Exercise510
    Proposition77
variable {a b : Type*} {D : NeighborhoodSystem a} {E : NeighborhoodSystem b}

namespace Exercise717

/-- **The universal strict map `g : D^S  ->  E`, as a neighbourhood relation.** Defined
     by recursion on
the `D^S`-neighbourhood shape: `Gamma  |->  Delta_E`, `0*X  |->  u`'s relation, `1*P U
    2*Q  |->  v  comp  (g  x  g)`. -/
inductive GRel (u : ApproximableMap D E) (v : ApproximableMap (prod E E) E) :
    Set (List Bool  x  a)  ->  Set b  ->  Prop
  | gamma {Z} : Z = E.master  ->  GRel u v (Gamma D) Z
  | leaf {X Z} : u.rel X Z  ->  GRel u v (embZero X) Z
  | node {P Q Z_1 Z_2 Z} : GRel u v P Z_1  ->  GRel u v Q Z_2  ->
      v.rel (prodNbhd Z_1 Z_2) Z  ->  GRel u v (embPair P Q) Z
```

```
end Exercise717
end

section
open NeighborhoodSystem Domain.Recursive ApproximableMap Example61 Exercise510
    Proposition77
variable {a b : Type*} {D : NeighborhoodSystem a} {E : NeighborhoodSystem b}

namespace Exercise717

variable {u : ApproximableMap D E} {v : ApproximableMap (prod E E) E}

/-- `embZero X  !=  embPair P Q` from `P  in  ?^S` (no `D.mem X` needed). -/
theorem embZero_ne_embPair_of_memS (hD : forall  X, D.mem X  ->  X.Nonempty)
    {X : Set a} {P Q : Set (List Bool  x  a)} (hP : MemS D P) :
    embZero X  !=  embPair P Q := by
  intro h
  obtain <z, hz> := embPair_nonempty (memS_nonempty hD hP) (Q := Q)
  have hz0 : z  in  embZero X := h  |>  hz
  have hpath : z.1 = [] := hz0.1
  rcases hz with <p', hp', -> | <q', hq', ->
  * rw [hp'] at hpath; simp at hpath
  * rw [hq'] at hpath; simp at hpath

/-- Related outputs are `E`-neighbourhoods. -/
theorem gRel_cod {W Z} (h : GRel u v W Z) : E.mem Z := by
  induction h with
  | gamma hZ => exact hZ  |>  E.master_mem
  | leaf hrel => exact u.rel_cod hrel
  | node _ _ hvrel _ _ => exact v.rel_cod hvrel

/-- Related inputs are `D^S`-neighbourhoods. -/
theorem gRel_dom {W Z} (h : GRel u v W Z) : MemS D W := by
  induction h with
  | gamma _ => exact MemS.gamma
  | leaf hrel => exact MemS.zero (u.rel_dom hrel)
  | node _ _ _ ihP ihQ => exact MemS.pair ihP ihQ

/-- Inversion at a node `1*P  U  2*Q`: `(1*P U 2*Q) g Z  <->  exists  Z_1 Z_2, P g Z_1
     /\  Q g Z_2  /\  <Z_1,Z_2> v Z`. -/
theorem gRel_embPair_inv (hD : forall  X, D.mem X  ->  X.Nonempty) {P Q : Set (List
    Bool  x  a)} {Z}
    (h : GRel u v (embPair P Q) Z) :
    exists  Z_1 Z_2, GRel u v P Z_1  /\  GRel u v Q Z_2  /\  v.rel (prodNbhd Z_1 Z_2)
    Z := by
  have hPQ := gRel_dom h
  obtain <hPmem, hQmem> := memS_embPair_inv hD hPQ
  generalize hW : embPair P Q = W at h
  cases h with
  | gamma hZ => exact absurd hW (embPair_ne_Gamma D hD P Q)
  | @leaf X Z hrel => exact absurd hW.symm (embZero_ne_embPair_of_memS hD hPmem)
  | @node P' Q' Z_1 Z_2 Z hP' hQ' hvrel =>
      obtain <rfl, rfl> := embPair_injective hW
      exact <Z_1, Z_2, hP', hQ', hvrel>
```

```
/-- Inversion at a leaf `0*X`: `0*X g Z  <->  X u Z`. -/
theorem gRel_embZero_inv (hD : forall  X, D.mem X  ->  X.Nonempty) {X : Set a} {Z}
    (h : GRel u v (embZero X) Z) : u.rel X Z := by
  generalize hW : embZero X = W at h
  cases h with
  | gamma hZ => exact absurd hW (embZero_ne_Gamma D hD X)
  | @leaf X' Z' hrel => rw [embZero_injective hW]; exact hrel
  | @node P Q Z_1 Z_2 Z hP hQ hvrel =>
      exact absurd hW (embZero_ne_embPair_of_memS hD (gRel_dom hP))

/-- Inversion at `Gamma`: `Gamma g Z` forces `Z = Delta_E` (Scott's strictness). -/
theorem gRel_gamma_inv (hD : forall  X, D.mem X  ->  X.Nonempty) {Z}
    (h : GRel u v (Gamma D) Z) : Z = E.master := by
  generalize hW : Gamma D = W at h
  cases h with
  | gamma hZ => exact hZ
  | @leaf X Z hrel => exact absurd hW.symm (embZero_ne_Gamma D hD X)
  | @node P Q Z_1 Z_2 Z hP hQ hvrel => exact absurd hW.symm (embPair_ne_Gamma D hD P Q
    )

/-- Intersectivity on the output (Scott's 2.1(ii)). -/
theorem gRel_interRight (hD : forall  X, D.mem X  ->  X.Nonempty) {W Z} (h : GRel u v
    W Z) :
    forall  {Z'}, GRel u v W Z'  ->  GRel u v W (Z  I  Z') := by
  induction h with
  | @gamma Z hZ =>
      intro Z' h'
      subst hZ
      have hZ' := gRel_gamma_inv hD h'
      subst hZ'
      rw [Set.inter_self]
      exact GRel.gamma rfl
  | @leaf X Z hrel =>
      intro Z' h'
      exact GRel.leaf (u.inter_right hrel (gRel_embZero_inv hD h'))
  | @node P Q Z_1 Z_2 Z hP hQ hvrel ihP ihQ =>
      intro Z' h'
      obtain <Z_1', Z_2', hP', hQ', hv'> := gRel_embPair_inv hD h'
      have hgP : GRel u v P (Z_1  I  Z_1') := ihP hP'
      have hgQ : GRel u v Q (Z_2  I  Z_2') := ihQ hQ'
      refine GRel.node hgP hgQ ?_
      have hmem : (prod E E).mem (prodNbhd (Z_1  I  Z_1') (Z_2  I  Z_2')) :=
        prod_mem_prodNbhd (gRel_cod hgP) (gRel_cod hgQ)
      have hv1 : v.rel (prodNbhd (Z_1  I  Z_1') (Z_2  I  Z_2')) Z :=
        v.mono hvrel (prodNbhd_subset_iff.mpr <Set.inter_subset_left, Set.
    inter_subset_left>)
          subset_rfl hmem (v.rel_cod hvrel)
      have hv2 : v.rel (prodNbhd (Z_1  I  Z_1') (Z_2  I  Z_2')) Z' :=
        v.mono hv' (prodNbhd_subset_iff.mpr <Set.inter_subset_right, Set.
    inter_subset_right>)
          subset_rfl hmem (v.rel_cod hv')
      exact v.inter_right hv1 hv2

/-- **Strictness in the relation: every `D^S`-neighbourhood folds to `Delta_E`.** This
    is why no separate
"top" clause is needed -- `u`/`v` relate `Delta  |->  Delta`, so the master propagates
```

```
    through the tree. -/
theorem gRel_master : forall  {W}, MemS D W  ->  GRel u v W E.master := by
  intro W hW
  induction hW with
  | gamma => exact GRel.gamma rfl
  | @zero X hX =>
      exact GRel.leaf ((u.rel_iff_mem_principal hX).mpr (u.toElementMap _).master_mem)
  | @pair P Q _ _ ihP ihQ =>
      refine GRel.node ihP ihQ ?_
      have h := v.master_rel
      rwa [prod_master] at h

/-- Monotonicity (Scott's 2.1(iii)). -/
theorem gRel_mono (hD : forall  X, D.mem X  ->  X.Nonempty) {W Z} (h : GRel u v W Z) :
    forall  {W' Z'}, W'  subseteq  W  ->  Z  subseteq  Z'  ->  MemS D W'  ->  E.mem Z'
      ->  GRel u v W' Z' := by
  induction h with
  | @gamma Z hZ =>
      intro W' Z' hW'W hZZ' hW'mem hZ'mem
      subst hZ
      have hZ' : Z' = E.master := Set.Subset.antisymm (E.sub_master hZ'mem) hZZ'
      subst hZ'
      exact gRel_master hW'mem
  | @leaf X Z hrel =>
      intro W' Z' hW'W hZZ' hW'mem hZ'mem
      obtain <X', hX'mem, rfl> := memS_sub_embZero hD hW'mem hW'W
      exact GRel.leaf (u.mono hrel (embZero_subset.mp hW'W) hZZ' hX'mem hZ'mem)
  | @node P Q Z_1 Z_2 Z hP hQ hvrel ihP ihQ =>
      intro W' Z' hW'W hZZ' hW'mem hZ'mem
      obtain <P', Q', hP'mem, hQ'mem, rfl> := memS_sub_embPair hD hW'mem hW'W
      obtain <hP'P, hQ'Q> := embPair_subset.mp hW'W
      refine GRel.node (ihP hP'P subset_rfl hP'mem (gRel_cod hP))
        (ihQ hQ'Q subset_rfl hQ'mem (gRel_cod hQ)) ?_
      exact v.mono hvrel subset_rfl hZZ'
        (prod_mem_prodNbhd (gRel_cod hP) (gRel_cod hQ)) hZ'mem

/-- **Exercise 7.17 Part 2 -- the universal strict map `g : D^S  ->  E`.** -/
def gMap (u : ApproximableMap D E) (v : ApproximableMap (prod E E) E)
    (hD : forall  X, D.mem X  ->  X.Nonempty) : ApproximableMap (Dsharp D hD) E where
  rel := GRel u v
  rel_dom h := gRel_dom h
  rel_cod h := gRel_cod h
  master_rel := GRel.gamma rfl
  inter_right h h' := gRel_interRight hD h h'
  mono h hsub hsup hX' hY' := gRel_mono hD h hsub hsup hX' hY'

@[simp] theorem gMap_rel {hD : forall  X, D.mem X  ->  X.Nonempty} {W : Set (List Bool
      x  a)} {Z : Set b} :
    (gMap u v hD).rel W Z  <->  GRel u v W Z := Iff.rfl

section Computability

variable (fe fU fV : Nat  ->  Nat) (mIdx : Nat)

/-- **The certificate step.** Input `p = <w, tbl>` with `w = <n, cert>` and `tbl` the
    memo table.
```

```
`cert` decodes to `<out, wit, lcert, rcert>`. Returns `<okBit, out>`:
* `n = 0`   : `okBit = fe<out, mIdx>` (is `Q_out = Delta_E`?);
* `n = 2a+1`: `okBit = fU<wit, <a, out>>` (does `wit` witness `X_a u Q_out`?);
* `n = 2a+2`: read children `<a.1, lcert>`, `<a.2, rcert>` from `tbl`; `okBit = okL*okR*
  fV<wit, <<outL, outR>, out>>`. -/
def gStep (p : Nat) : Nat :=
  let w := p.unpair.1
  let tbl := p.unpair.2
  let nn := w.unpair.1
  let cert := w.unpair.2
  let out := cert.unpair.1
  let wit := cert.unpair.2.unpair.1
  let lcert := cert.unpair.2.unpair.2.unpair.1
  let rcert := cert.unpair.2.unpair.2.unpair.2
  let b := nn / 2 - 1
  let childL := listGet tbl (w - 1 - Nat.pair b.unpair.1 lcert)
  let childR := listGet tbl (w - 1 - Nat.pair b.unpair.2 rcert)
  selectFn (1 - nn)
    (Nat.pair (fe (Nat.pair out mIdx)) out)
    (selectFn (nn % 2)
      (Nat.pair (fU (Nat.pair wit (Nat.pair (nn / 2) out))) out)
      (Nat.pair (childL.unpair.1 * childR.unpair.1
          * fV (Nat.pair wit (Nat.pair (Nat.pair childL.unpair.2 childR.unpair.2) out)
   )) out))

/-- The full evaluator: the course-of-values value of `gStep` at `w = <n, cert>`. -/
def gEval : Nat  ->  Nat := gOf (gStep fe fU fV mIdx)

theorem gEval_pair (n cert : Nat) :
    gEval fe fU fV mIdx (Nat.pair n cert)
      = gStep fe fU fV mIdx
          (Nat.pair (Nat.pair n cert) (rtbl (gStep fe fU fV mIdx) (Nat.pair n cert)))
    := by
  rw [gEval, gOf_def]

end Computability

/-- **`g(in x) = u(x)`.** The injection summand of `D^S  ~=  D + (D^S x D^S)` folds via
     `u`. -/
theorem gMap_in (hD : forall  X, D.mem X  ->  X.Nonempty) (x : D.Element) :
    (gMap u v hD).toElementMap (inSharp D hD x) = u.toElementMap x := by
  apply Element.ext
  intro Z
  simp only [mem_toElementMap, gMap_rel]
  constructor
  * rintro <W, hWmem, hrel>
    rcases hWmem with rfl | <X, hXx, rfl>
    * have hZ := gRel_gamma_inv hD hrel; subst hZ
      exact <D.master, x.master_mem, u.master_rel>
    * exact <X, hXx, gRel_embZero_inv hD hrel>
  * rintro <X, hXx, hrel>
    exact <embZero X, Or.inr <X, hXx, rfl>, GRel.leaf hrel>

/-- **`g(pair y z) = v(<g y, g z>)`.** The product summand folds via `v` applied to the
     recursive
results -- the defining recursion of the catamorphism. -/
```

```
theorem gMap_pair (hD : forall  X, D.mem X  ->  X.Nonempty) (y z : (Dsharp D hD).
    Element) :
    (gMap u v hD).toElementMap (pairSharp D hD y z)
      = v.toElementMap (pair ((gMap u v hD).toElementMap y) ((gMap u v hD).
    toElementMap z)) := by
  apply Element.ext
  intro Z
  simp only [mem_toElementMap, gMap_rel]
  constructor
  * rintro <W, hWmem, hrel>
    rcases hWmem with rfl | <P, Q, hPy, hQz, rfl>
    * have hZ := gRel_gamma_inv hD hrel; subst hZ
      exact <(prod E E).master, (pair _ _).master_mem, v.master_rel>
    * obtain <Z_1, Z_2, hP1, hQ2, hvrel> := gRel_embPair_inv hD hrel
      exact <prodNbhd Z_1 Z_2,
        mem_pair.mpr <Z_1, Z_2, <P, hPy, hP1>, <Q, hQz, hQ2>, rfl>, hvrel>
  * rintro <V, hVmem, hvrel>
    obtain <Z_1, Z_2, <P, hPy, hP1>, <Q, hQz, hQ2>, rfl> := mem_pair.mp hVmem
    exact <embPair P Q, Or.inr <P, Q, hPy, hQz, rfl>, GRel.node hP1 hQ2 hvrel>

/-- **`g` is strict** (`g(False) = False`): Scott's "the unique *strict* mapping". -/
theorem gMap_strict (hD : forall  X, D.mem X  ->  X.Nonempty) :
    IsStrict (gMap u v hD) := fun _ h => gRel_gamma_inv hD h

section Computability

variable (fe fU fV : Nat  ->  Nat) (mIdx : Nat)

theorem gStep_gamma (cert tbl : Nat) :
    gStep fe fU fV mIdx (Nat.pair (Nat.pair 0 cert) tbl)
      = Nat.pair (fe (Nat.pair cert.unpair.1 mIdx)) cert.unpair.1 := by
  simp only [gStep, unpair_pair_fst, unpair_pair_snd, Nat.sub_zero, selectFn_one]

theorem gStep_leaf (a cert tbl : Nat) :
    gStep fe fU fV mIdx (Nat.pair (Nat.pair (2 * a + 1) cert) tbl)
      = Nat.pair (fU (Nat.pair cert.unpair.2.unpair.1 (Nat.pair a cert.unpair.1)))
          cert.unpair.1 := by
  simp only [gStep, unpair_pair_fst, unpair_pair_snd]
  rw [show (1 : Nat) - (2 * a + 1) = 0 by omega, selectFn_zero,
    show (2 * a + 1) % 2 = 1 by omega, selectFn_one, show (2 * a + 1) / 2 = a by omega
    ]

theorem gStep_node (a cert tbl : Nat) :
    gStep fe fU fV mIdx (Nat.pair (Nat.pair (2 * a + 2) cert) tbl)
      = Nat.pair
          ((listGet tbl (Nat.pair (2 * a + 2) cert - 1
                - Nat.pair a.unpair.1 cert.unpair.2.unpair.2.unpair.1)).unpair.1
            * (listGet tbl (Nat.pair (2 * a + 2) cert - 1
                - Nat.pair a.unpair.2 cert.unpair.2.unpair.2.unpair.2)).unpair.1
            * fV (Nat.pair cert.unpair.2.unpair.1
                (Nat.pair (Nat.pair
                    (listGet tbl (Nat.pair (2 * a + 2) cert - 1
                      - Nat.pair a.unpair.1 cert.unpair.2.unpair.2.unpair.1)).unpair.2
                    (listGet tbl (Nat.pair (2 * a + 2) cert - 1
                      - Nat.pair a.unpair.2 cert.unpair.2.unpair.2.unpair.2)).unpair
    .2)
```

```
                cert.unpair.1)))
          cert.unpair.1 := by
  simp only [gStep, unpair_pair_fst, unpair_pair_snd]
  rw [show (1 : Nat) - (2 * a + 2) = 0 by omega, selectFn_zero,
    show (2 * a + 2) % 2 = 0 by omega, selectFn_zero, show (2 * a + 2) / 2 - 1 = a by
    omega]

/-- The output index of `gEval <n, cert>` is just `cert`'s first component (all three
    branches emit
`out`). -/
theorem gEval_out (n cert : Nat) :
    (gEval fe fU fV mIdx (Nat.pair n cert)).unpair.2 = cert.unpair.1 := by
  rw [gEval_pair]
  rcases nat_shape n with rfl | <a, rfl> | <a, rfl>
  * rw [gStep_gamma, unpair_pair_snd]
  * rw [gStep_leaf, unpair_pair_snd]
  * rw [gStep_node, unpair_pair_snd]

/-- Memo lookup, in `gEval` form: at any earlier code `w' < w`, position `w-1-w'` of
    the table is
`gEval w'`. -/
theorem listGet_rtbl_gEval {w' w : Nat} (h : w' < w) :
    listGet (rtbl (gStep fe fU fV mIdx) w) (w - 1 - w') = gEval fe fU fV mIdx w' :=
  listGet_rtbl (gStep fe fU fV mIdx) h

end Computability

/-- `Nat.pair` is monotone in its second argument (first argument fixed). -/
theorem pair_le_pair_right {a b d : Nat} (h : b  <=  d) : Nat.pair a b  <=  Nat.pair a
     d := by
  unfold Nat.pair
  rcases Nat.lt_or_ge a b with hab | hab
  * rw [ite_eq_left hab, ite_eq_left (lt_of_lt_of_le hab h)]
    have : b * b  <=  d * d := Nat.mul_le_mul h h
    omega
  * rw [ite_eq_right (Nat.not_lt.mpr hab)]
    rcases Nat.lt_or_ge a d with had | had
    * rw [ite_eq_left had]
      have h1 : a * a + a  <=  d * d := by
        calc a * a + a = a * (a + 1) := by ring
          _  <=  a * d := Nat.mul_le_mul (le_refl a) had
          _  <=  d * d := Nat.mul_le_mul had.le (le_refl d)
      omega
    * rw [ite_eq_right (Nat.not_lt.mpr had)]; omega

/-- `Nat.pair` is strictly monotone in its first argument (second argument fixed). -/
theorem pair_lt_pair_left {a c b : Nat} (h : a < c) : Nat.pair a b < Nat.pair c b :=
    by
  unfold Nat.pair
  rcases Nat.lt_or_ge a b with hab | hab
  * rw [ite_eq_left hab]
    rcases Nat.lt_or_ge c b with hcb | hcb
    * rw [ite_eq_left hcb]; omega
    * rw [ite_eq_right (Nat.not_lt.mpr hcb)]
      have hbc : b * b  <=  c * c := Nat.mul_le_mul hcb hcb
      omega
```

```
  * rw [ite_eq_right (Nat.not_lt.mpr hab),
      ite_eq_right (Nat.not_lt.mpr (le_of_lt (lt_of_le_of_lt hab h)))]
    have hcc : (a + 1) * (a + 1)  <=  c * c := Nat.mul_le_mul h h
    have hexp : (a + 1) * (a + 1) = a * a + 2 * a + 1 := by ring
    omega

/-- Combined: strict in the first argument, monotone in the second. -/
theorem pair_lt_pair_of_lt_le {a c b d : Nat} (hac : a < c) (hbd : b  <=  d) :
    Nat.pair a b < Nat.pair c d :=
  lt_of_lt_of_le (pair_lt_pair_left hac) (pair_le_pair_right hbd)

section Computability

variable (fe fU fV : Nat  ->  Nat) (mIdx : Nat)

set_option maxHeartbeats 1000000 in
/-- **Completeness of the certificate.** If `g` relates `V_n` to some `Z`, there is a `
    cert` the memo
certifies (ok-bit `1`) with `Q_{out} = Z`. Strong induction on `n`; the node case
    builds the certificate
from the children's certificates (children codes are `<` by `pair_lt_pair_of_lt_le`).
    -/
theorem gEval_complete (hD : forall  X, D.mem X  ->  X.Nonempty)
    (P : ComputablePresentation D) (Q : ComputablePresentation E)
    (hEq : forall  m, fe (Nat.pair m mIdx) = 1  <->  Q.X m = E.master) (hMidx : Q.X
    mIdx = E.master)
    (hU : forall  a m, (exists  i, fU (Nat.pair i (Nat.pair a m)) = 1)  <->  u.rel (P.
    X a) (Q.X m))
    (hV : forall  k1 k2 m, (exists  i, fV (Nat.pair i (Nat.pair (Nat.pair k1 k2) m)) =
     1)  <->
        v.rel (prodNbhd (Q.X k1) (Q.X k2)) (Q.X m)) :
    forall  n Z, GRel u v (Vsharp D P n) Z  ->
      exists  cert, (gEval fe fU fV mIdx (Nat.pair n cert)).unpair.1 = 1
        /\  Q.X cert.unpair.1 = Z := by
  intro n
  induction n using Nat.strong_induction_on with
  | _ n ih =>
    intro Z hg
    rcases nat_shape n with rfl | <a, rfl> | <a, rfl>
    * -- n = 0
      rw [Vsharp_zero] at hg
      have hZ := gRel_gamma_inv hD hg
      refine <Nat.pair mIdx (Nat.pair 0 0), ?_, ?_>
      * rw [gEval_pair, gStep_gamma, unpair_pair_fst, unpair_pair_fst]
        exact (hEq mIdx).mpr hMidx
      * rw [unpair_pair_fst, hMidx, hZ]
    * -- n = 2a+1
      rw [Vsharp_odd] at hg
      have hu := gRel_embZero_inv hD hg
      obtain <e, he> := Q.surj (u.rel_cod hu)
      rw [ <-  he] at hu
      obtain <wit, hwit> := (hU a e).mpr hu
      refine <Nat.pair e (Nat.pair wit 0), ?_, ?_>
      * rw [gEval_pair, gStep_leaf, unpair_pair_fst, unpair_pair_snd, unpair_pair_fst,
          unpair_pair_fst]
        exact hwit
```

```
      * rw [unpair_pair_fst]; exact he
    * -- n = 2a+2
      rw [Vsharp_even] at hg
      obtain <Z_1, Z_2, hg1, hg2, hvrel> := gRel_embPair_inv hD hg
      obtain <certL, hokL, heqL> := ih a.unpair.1 (by have := unpair_fst_le a; omega)
    Z_1 hg1
      obtain <certR, hokR, heqR> := ih a.unpair.2 (by have := unpair_snd_le a; omega)
    Z_2 hg2
      obtain <e, he> := Q.surj (v.rel_cod hvrel)
      rw [ <-  heqL,  <-  heqR,  <-  he] at hvrel
      obtain <wit, hwit> := (hV certL.unpair.1 certR.unpair.1 e).mpr hvrel
      refine <Nat.pair e (Nat.pair wit (Nat.pair certL certR)), ?_, ?_>
      * rw [gEval_pair, gStep_node, unpair_pair_fst]
        have hcertL : certL  <=  Nat.pair e (Nat.pair wit (Nat.pair certL certR)) :=
          le_trans (Domain.Recursive.le_pair_left _ _)
            (le_trans (Domain.Recursive.le_pair_right wit _) (Domain.Recursive.
    le_pair_right e _))
        have hcertR : certR  <=  Nat.pair e (Nat.pair wit (Nat.pair certL certR)) :=
          le_trans (Domain.Recursive.le_pair_right _ _)
            (le_trans (Domain.Recursive.le_pair_right wit _) (Domain.Recursive.
    le_pair_right e _))
        have hlt1 : Nat.pair a.unpair.1 certL < Nat.pair (2 * a + 2)
            (Nat.pair e (Nat.pair wit (Nat.pair certL certR))) :=
          pair_lt_pair_of_lt_le (by have := unpair_fst_le a; omega) hcertL
        have hlt2 : Nat.pair a.unpair.2 certR < Nat.pair (2 * a + 2)
            (Nat.pair e (Nat.pair wit (Nat.pair certL certR))) :=
          pair_lt_pair_of_lt_le (by have := unpair_snd_le a; omega) hcertR
        simp only [unpair_pair_fst, unpair_pair_snd]
        rw [listGet_rtbl_gEval fe fU fV mIdx hlt1, listGet_rtbl_gEval fe fU fV mIdx
    hlt2,
          gEval_out, gEval_out]
        rw [show (gEval fe fU fV mIdx (Nat.pair a.unpair.1 certL)).unpair.1 = 1 from
    hokL,
          show (gEval fe fU fV mIdx (Nat.pair a.unpair.2 certR)).unpair.1 = 1 from
    hokR]
        simp only [Nat.one_mul]
        exact hwit
      * rw [unpair_pair_fst]; exact he

/-- The certificate step is primitive recursive (when the underlying functions are).
    -/
theorem primrec_gStep (hfe : Nat.Primrec fe) (hfU : Nat.Primrec fU) (hfV : Nat.Primrec
     fV) :
    Nat.Primrec (gStep fe fU fV mIdx) := by
  have hw : Nat.Primrec (fun p : Nat => p.unpair.1) := Nat.Primrec.left
  have htbl : Nat.Primrec (fun p : Nat => p.unpair.2) := Nat.Primrec.right
  have hnn : Nat.Primrec (fun p : Nat => p.unpair.1.unpair.1) := Nat.Primrec.left.comp
     Nat.Primrec.left
  have hcert : Nat.Primrec (fun p : Nat => p.unpair.1.unpair.2) :=
    Nat.Primrec.right.comp Nat.Primrec.left
  have hout : Nat.Primrec (fun p : Nat => p.unpair.1.unpair.2.unpair.1) := Nat.Primrec
    .left.comp hcert
  have hwit : Nat.Primrec (fun p : Nat => p.unpair.1.unpair.2.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp (Nat.Primrec.right.comp hcert)
  have hlc : Nat.Primrec (fun p : Nat => p.unpair.1.unpair.2.unpair.2.unpair.2.unpair
    .1) :=
```

```
  Nat.Primrec.left.comp (Nat.Primrec.right.comp (Nat.Primrec.right.comp hcert))
have hrc : Nat.Primrec (fun p : Nat => p.unpair.1.unpair.2.unpair.2.unpair.2.unpair
  .2) :=
  Nat.Primrec.right.comp (Nat.Primrec.right.comp (Nat.Primrec.right.comp hcert))
have hb : Nat.Primrec (fun p : Nat => p.unpair.1.unpair.1 / 2 - 1) :=
  primrec_sub_2 (primrec_div2.comp hnn) (Nat.Primrec.const 1)
have hlpos : Nat.Primrec (fun p : Nat =>
    p.unpair.1 - 1 - Nat.pair (p.unpair.1.unpair.1 / 2 - 1).unpair.1
      p.unpair.1.unpair.2.unpair.2.unpair.2.unpair.1) :=
  primrec_sub_2 (primrec_sub_2 hw (Nat.Primrec.const 1))
    ((Nat.Primrec.left.comp hb).pair hlc)
have hrpos : Nat.Primrec (fun p : Nat =>
    p.unpair.1 - 1 - Nat.pair (p.unpair.1.unpair.1 / 2 - 1).unpair.2
      p.unpair.1.unpair.2.unpair.2.unpair.2.unpair.2) :=
  primrec_sub_2 (primrec_sub_2 hw (Nat.Primrec.const 1))
    ((Nat.Primrec.right.comp hb).pair hrc)
have hchildL : Nat.Primrec (fun p : Nat => listGet p.unpair.2
    (p.unpair.1 - 1 - Nat.pair (p.unpair.1.unpair.1 / 2 - 1).unpair.1
      p.unpair.1.unpair.2.unpair.2.unpair.2.unpair.1)) :=
  primrec_listGet_2 htbl hlpos
have hchildR : Nat.Primrec (fun p : Nat => listGet p.unpair.2
    (p.unpair.1 - 1 - Nat.pair (p.unpair.1.unpair.1 / 2 - 1).unpair.2
      p.unpair.1.unpair.2.unpair.2.unpair.2.unpair.2)) :=
  primrec_listGet_2 htbl hrpos
have hgamma : Nat.Primrec (fun p : Nat =>
    Nat.pair (fe (Nat.pair p.unpair.1.unpair.2.unpair.1 mIdx)) p.unpair.1.unpair.2.
  unpair.1) :=
  (hfe.comp (hout.pair (Nat.Primrec.const mIdx))).pair hout
have hleaf : Nat.Primrec (fun p : Nat =>
    Nat.pair (fU (Nat.pair p.unpair.1.unpair.2.unpair.2.unpair.1
        (Nat.pair (p.unpair.1.unpair.1 / 2) p.unpair.1.unpair.2.unpair.1)))
      p.unpair.1.unpair.2.unpair.1) :=
  (hfU.comp (hwit.pair ((primrec_div2.comp hnn).pair hout))).pair hout
have hfVcall : Nat.Primrec (fun p : Nat =>
    fV (Nat.pair p.unpair.1.unpair.2.unpair.2.unpair.1
      (Nat.pair (Nat.pair
          (listGet p.unpair.2 (p.unpair.1 - 1 - Nat.pair (p.unpair.1.unpair.1 / 2 -
  1).unpair.1
            p.unpair.1.unpair.2.unpair.2.unpair.2.unpair.1)).unpair.2
          (listGet p.unpair.2 (p.unpair.1 - 1 - Nat.pair (p.unpair.1.unpair.1 / 2 -
  1).unpair.2
            p.unpair.1.unpair.2.unpair.2.unpair.2.unpair.2)).unpair.2)
        p.unpair.1.unpair.2.unpair.1))) :=
  hfV.comp (hwit.pair
    (((Nat.Primrec.right.comp hchildL).pair (Nat.Primrec.right.comp hchildR)).pair
  hout))
have hnode : Nat.Primrec (fun p : Nat =>
    Nat.pair ((listGet p.unpair.2 (p.unpair.1 - 1 - Nat.pair (p.unpair.1.unpair.1 /
  2 - 1).unpair.1
            p.unpair.1.unpair.2.unpair.2.unpair.2.unpair.1)).unpair.1
        * (listGet p.unpair.2 (p.unpair.1 - 1 - Nat.pair (p.unpair.1.unpair.1 / 2 -
  1).unpair.2
            p.unpair.1.unpair.2.unpair.2.unpair.2.unpair.2)).unpair.1
        * fV (Nat.pair p.unpair.1.unpair.2.unpair.2.unpair.1
            (Nat.pair (Nat.pair
                (listGet p.unpair.2 (p.unpair.1 - 1 - Nat.pair (p.unpair.1.unpair.1
```

```
    / 2 - 1).unpair.1
                    p.unpair.1.unpair.2.unpair.2.unpair.2.unpair.1)).unpair.2
                  (listGet p.unpair.2 (p.unpair.1 - 1 - Nat.pair (p.unpair.1.unpair.1
    / 2 - 1).unpair.2
                    p.unpair.1.unpair.2.unpair.2.unpair.2.unpair.2)).unpair.2)
                p.unpair.1.unpair.2.unpair.1)))
        p.unpair.1.unpair.2.unpair.1) :=
     (primrec_mul_2 (primrec_mul_2 (Nat.Primrec.left.comp hchildL) (Nat.Primrec.left.
    comp hchildR))
       hfVcall).pair hout
  refine (primrec_selectFn (primrec_sub_2 (Nat.Primrec.const 1) hnn) hgamma
     (primrec_selectFn (primrec_mod2.comp hnn) hleaf hnode)).of_eq fun p => rfl

end Computability

/-- A decoded sub-component is ` <= ` the code it is decoded from. -/
theorem unpair_snd_snd_fst_le (c : Nat) : c.unpair.2.unpair.2.unpair.1  <=  c := by
  have h1 := unpair_snd_le c
  have h2 := unpair_snd_le c.unpair.2
  have h3 := unpair_fst_le c.unpair.2.unpair.2
  omega

theorem unpair_snd_snd_snd_le (c : Nat) : c.unpair.2.unpair.2.unpair.2  <=  c := by
  have h1 := unpair_snd_le c
  have h2 := unpair_snd_le c.unpair.2
  have h3 := unpair_snd_le c.unpair.2.unpair.2
  omega

section Computability

variable (fe fU fV : Nat  ->  Nat) (mIdx : Nat)

/-- **Soundness of the certificate.** If the memo certifies `<n, cert>` (ok-bit `1`),
    then `g` relates
`V_n` to `Q_{out}` (`out = cert.unpair.1`). Strong induction on the combined code `<n,
    cert>`; the node
case reads the two children from the memo table (`listGet_rtbl`) and applies the IH. -/
theorem gEval_sound
    (P : ComputablePresentation D) (Q : ComputablePresentation E)
    (hEq : forall  m, fe (Nat.pair m mIdx) = 1  <->  Q.X m = E.master)
    (hU : forall  a m, (exists  i, fU (Nat.pair i (Nat.pair a m)) = 1)  <->  u.rel (P.
    X a) (Q.X m))
    (hV : forall  k1 k2 m, (exists  i, fV (Nat.pair i (Nat.pair (Nat.pair k1 k2) m)) =
     1)  <->
        v.rel (prodNbhd (Q.X k1) (Q.X k2)) (Q.X m)) :
    forall  n cert, (gEval fe fU fV mIdx (Nat.pair n cert)).unpair.1 = 1  ->
      GRel u v (Vsharp D P n) (Q.X cert.unpair.1) := by
  have key : forall  w n cert, Nat.pair n cert = w  ->
      (gEval fe fU fV mIdx (Nat.pair n cert)).unpair.1 = 1  ->
        GRel u v (Vsharp D P n) (Q.X cert.unpair.1) := by
    intro w
    induction w using Nat.strong_induction_on with
    | _ w ih =>
      intro n cert hw hok
      rcases nat_shape n with rfl | <a, rfl> | <a, rfl>
      * rw [gEval_pair, gStep_gamma, unpair_pair_fst] at hok
```

```
        rw [Vsharp_zero]
        exact GRel.gamma ((hEq cert.unpair.1).mp hok)
      * rw [gEval_pair, gStep_leaf, unpair_pair_fst] at hok
        rw [Vsharp_odd]
        exact GRel.leaf ((hU a cert.unpair.1).mp <cert.unpair.2.unpair.1, hok>)
      * rw [Vsharp_even]
        rw [gEval_pair, gStep_node, unpair_pair_fst] at hok
        have hlt1 : Nat.pair a.unpair.1 cert.unpair.2.unpair.2.unpair.1 < Nat.pair (2
    * a + 2) cert :=
          pair_lt_pair_of_lt_le (by have := unpair_fst_le a; omega) (
    unpair_snd_snd_fst_le cert)
        have hlt2 : Nat.pair a.unpair.2 cert.unpair.2.unpair.2.unpair.2 < Nat.pair (2
    * a + 2) cert :=
          pair_lt_pair_of_lt_le (by have := unpair_snd_le a; omega) (
    unpair_snd_snd_snd_le cert)
        rw [listGet_rtbl_gEval fe fU fV mIdx hlt1,
          listGet_rtbl_gEval fe fU fV mIdx hlt2] at hok
        rw [nat_mul_eq_one, nat_mul_eq_one] at hok
        obtain <<hokL, hokR>, hokV> := hok
        have hgL := ih _ (hw  |>  hlt1) a.unpair.1 cert.unpair.2.unpair.2.unpair.1 rfl
     hokL
        have hgR := ih _ (hw  |>  hlt2) a.unpair.2 cert.unpair.2.unpair.2.unpair.2 rfl
     hokR
        rw [gEval_out, gEval_out] at hokV
        refine GRel.node hgL hgR ?_
        exact (hV _ _ cert.unpair.1).mp <cert.unpair.2.unpair.1, hokV>
  intro n cert; exact key (Nat.pair n cert) n cert rfl

end Computability

/-- **Exercise 7.17 (Part 2) (Scott 1981, PRG-19).** If `E` is effectively given (
    presentation `Q`) and
`u : D  ->  E`, `v : E  x  E  ->  E` are computable, then the unique strict
    catamorphism `g : D^S  ->  E`
(`gMap`) is computable.

The index relation `X_n g Y? = GRel u v (V_n)(Y?)` is realised as the recursively-
    enumerable existential
`exists  cert, gEval<n,cert>.ok = 1  /\  Y_{cert.out} = Y?` over the primitive-
    recursive certificate evaluator
`gEval`. Soundness (`gEval_sound`) and completeness (`gEval_complete`) of the
    certificate give the
equivalence; the deciders `fe`/`fU`/`fV` are read off `Q.eq_computable` and the r.e.
    relations of
`u`/`v`. -/
theorem gMap_isComputable (hD : forall  X, D.mem X  ->  X.Nonempty)
    (P : ComputablePresentation D) (Q : ComputablePresentation E)
    (hu : ComputableMap P Q u)
    (hv : ComputableMap (prodPresentation Q Q) Q v) :
    ComputableMap (dsharpPresentation P hD) Q (gMap u v hD) := by
  constructor
  obtain <qU, hqU, hqUe> := hu
  obtain <fU, hfUp, hfUe> := hqU
  obtain <qV, hqV, hqVe> := hv
  obtain <fV, hfVp, hfVe> := hqV
  obtain <fe, hfep, hfee> := Q.eq_computable
```

```
have hMidx : Q.X Q.masterIdx = E.master := Q.masterIdx_spec
have hEq : forall  m, fe (Nat.pair m Q.masterIdx) = 1  <->  Q.X m = E.master := by
  intro m
  have h := hfee (Nat.pair m Q.masterIdx)
  simp only [unpair_pair_fst, unpair_pair_snd] at h
  rw [hMidx] at h; exact h.symm
have hU : forall  a m, (exists  i, fU (Nat.pair i (Nat.pair a m)) = 1)  <->  u.rel (
  P.X a) (Q.X m) := by
  intro a m
  have h := hqUe (Nat.pair a m)
  simp only [unpair_pair_fst, unpair_pair_snd] at h
  rw [h]; exact exists_congr fun i => (hfUe (Nat.pair i (Nat.pair a m))).symm
have hV : forall  k1 k2 m, (exists  i, fV (Nat.pair i (Nat.pair (Nat.pair k1 k2) m))
   = 1)  <->
    v.rel (prodNbhd (Q.X k1) (Q.X k2)) (Q.X m) := by
  intro k1 k2 m
  have h := hqVe (Nat.pair (Nat.pair k1 k2) m)
  simp only [prodPresentation_X, unpair_pair_fst, unpair_pair_snd] at h
  rw [h]; exact exists_congr fun i => (hfVe (Nat.pair i (Nat.pair (Nat.pair k1 k2) m
  ))).symm
-- The certificate body is recursively decidable, hence its existential projection
  is r.e.
have hgEval : Nat.Primrec (gEval fe fU fV Q.masterIdx) :=
  primrec_gOf (primrec_gStep fe fU fV Q.masterIdx hfep hfUp hfVp)
have hidx : Nat.Primrec (fun s : Nat => Nat.pair s.unpair.2.unpair.1 s.unpair.1) :=
  (Nat.Primrec.left.comp Nat.Primrec.right).pair Nat.Primrec.left
have hp1 : RecDecidable (fun s : Nat =>
    (gEval fe fU fV Q.masterIdx (Nat.pair s.unpair.2.unpair.1 s.unpair.1)).unpair.1
  = 1) :=
  <_, Nat.Primrec.left.comp (hgEval.comp hidx), fun _ => Iff.rfl>
have hg2 : Nat.Primrec (fun s : Nat => Nat.pair s.unpair.1.unpair.1 s.unpair.2.
  unpair.2) :=
  (Nat.Primrec.left.comp Nat.Primrec.left).pair (Nat.Primrec.right.comp Nat.Primrec.
  right)
have hp2 : RecDecidable (fun s : Nat => Q.X s.unpair.1.unpair.1 = Q.X s.unpair.2.
  unpair.2) :=
  RecDecidable.of_iff (fun s => by simp only [unpair_pair_fst, unpair_pair_snd])
    (Q.eq_computable.comp hg2)
have hbase : REPred (fun s : Nat =>
    (gEval fe fU fV Q.masterIdx (Nat.pair s.unpair.2.unpair.1 s.unpair.1)).unpair.1
  = 1
      /\  Q.X s.unpair.1.unpair.1 = Q.X s.unpair.2.unpair.2) := (hp1.and hp2).re
unfold REPred_2
refine REPred.of_iff (fun t => ?_) (REPred.proj hbase)
simp only [unpair_pair_fst, unpair_pair_snd]
constructor
* intro hg
  obtain <cert, hok, heq> :=
    gEval_complete fe fU fV Q.masterIdx hD P Q hEq hMidx hU hV t.unpair.1 (Q.X t.
  unpair.2) hg
  exact <cert, hok, heq>
* rintro <cert, hok, heq>
  have hsound := gEval_sound fe fU fV Q.masterIdx P Q hEq hU hV t.unpair.1 cert hok
  rw [heq] at hsound
  exact hsound
```

```
end Exercise717
end

end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.Exercise717
section
set_option linter.unusedVariables false

theorem example_717_part2.{u_1, u_2} : forall  {a : Type u_1} {b : Type u_2} {D :
    DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
  {E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
  {u : @DomainTheory.NeighbourhoodSystems.ApproximableMap a b D E}
  {v :
    @DomainTheory.NeighbourhoodSystems.ApproximableMap (Sum b b) b (@DomainTheory.
    NeighbourhoodSystems.prod b b E E) E}

  (hD : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem
     a D X  ->  @Set.Nonempty a X)
  (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a D)
  (Q : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b E),
  @DomainTheory.NeighbourhoodSystems.ComputableMap a b D E P Q u  ->
    @DomainTheory.NeighbourhoodSystems.ComputableMap (Sum b b) b (@DomainTheory.
    NeighbourhoodSystems.prod b b E E) E
        (@DomainTheory.NeighbourhoodSystems.prodPresentation b b E E Q Q) Q v  ->
      @DomainTheory.NeighbourhoodSystems.ComputableMap (Prod (List Bool) a) b
        (@DomainTheory.NeighbourhoodSystems.Example61.Dsharp a D hD) E
        (@DomainTheory.NeighbourhoodSystems.Proposition77.dsharpPresentation a D P hD)
     Q
        (@DomainTheory.NeighbourhoodSystems.Exercise717.gMap a b D E u v hD) :=
  @DomainTheory.NeighbourhoodSystems.Exercise717.gMap_isComputable
end

end DomainTheory.NeighbourhoodSystems.Exercise717
namespace DomainTheory.NeighbourhoodSystems
end DomainTheory.NeighbourhoodSystems
end
```

3.8.20 Exercise 7.18 — effective isomorphism

Two effectively given systems D and E are *effectively isomorphic* iff … (complete the sentence!). Show that if D is effectively given then the isomorphism

$$D^\infty \cong (D^\infty)^\infty$$

is effective.

Exercise_7_18.lean

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Theorem_7_4

@[expose] public section
```

```
section
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive Example61
open NeighborhoodSystem Domain.Recursive ApproximableMap
namespace Exercise718
open NeighborhoodSystem Domain.Recursive ApproximableMap
section
variable {a b : Type*}

/- concordance: source L4655 EXERCISE 7.18 effective isomorphism -/
/-- **Exercise 7.18 (Scott 1981, PRG-19) -- effective isomorphism (completing the
    sentence).** Two
effectively given systems `D` (presentation `P`) and `E` (presentation `Q`) are *
    effectively
isomorphic* iff there is a pair of mutually inverse approximable maps `toMap : D  ->  E
    `,
`invMap : E  ->  D`, **both computable** (Definition 7.2). -/
structure EffectiveIso {V : NeighborhoodSystem a} {W : NeighborhoodSystem b}
    (P : ComputablePresentation V) (Q : ComputablePresentation W) where
  /-- The forward map `D  ->  E`. -/
  toMap : ApproximableMap V W
  /-- The inverse map `E  ->  D`. -/
  invMap : ApproximableMap W V
  /-- The forward map is computable. -/
  toMap_computable : ComputableMap P Q toMap
  /-- The inverse map is computable. -/
  invMap_computable : ComputableMap Q P invMap
  /-- `invMap  comp  toMap = I_D`. -/
  left_inv : invMap.comp toMap = idMap V
  /-- `toMap  comp  invMap = I_E`. -/
  right_inv : toMap.comp invMap = idMap W

/-- **Exercise 7.18 -- effectively isomorphic.** `D` and `E` are effectively isomorphic
    when an
`EffectiveIso` exists. -/
def EffectivelyIsomorphic {V : NeighborhoodSystem a} {W : NeighborhoodSystem b}
    (P : ComputablePresentation V) (Q : ComputablePresentation W) : Prop :=
  Nonempty (EffectiveIso P Q)

end

end Exercise718
open NeighborhoodSystem Domain.Recursive ApproximableMap Example61

section
variable {a : Type*}

namespace Proposition77

variable {D : NeighborhoodSystem a} (hD : forall  X, D.mem X  ->  X.Nonempty)

/-- **The second projection of the pair part of `D^S`, as an approximable map.** On the
    pair summand
`1*P  U  2*Q` it returns `Q`; everywhere else it returns `False = {Gamma}`. The exact
   mirror of `proj0Map`
with the *second* component: `W proj_1 Z  <->  Z = Gamma  \/  exists  P Q, W = 1*P  U
```

```
    2*Q  /\  Q  subseteq  Z`. -/
def proj1Map : ApproximableMap (Dsharp D hD) (Dsharp D hD) where
  rel W Z := MemS D W  /\  MemS D Z  /\  (Z = Gamma D  \/  exists  Pp Qq, W = embPair
    Pp Qq  /\  Qq  subseteq  Z)
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := <MemS.gamma, MemS.gamma, Or.inl rfl>
  inter_right := by
    rintro W Z Z' <hW, hZ, hd> <-, hZ', hd'>
    have hgZ : Z  subseteq  Gamma D := memS_subset_gamma hZ
    have hgZ' : Z'  subseteq  Gamma D := memS_subset_gamma hZ'
    rcases hd with rfl | <Pp, Qq, hWeq, hQZ>
    * rw [Set.inter_eq_right.mpr hgZ']; exact <hW, hZ', hd'>
    * rcases hd' with rfl | <Pp', Qq', hWeq', hQZ'>
      * rw [Set.inter_eq_left.mpr hgZ]; exact <hW, hZ, Or.inr <Pp, Qq, hWeq, hQZ>>
      * have hQQ : Qq = Qq' := (embPair_injective (hWeq.symm.trans hWeq')).2
        have hQsub : Qq  subseteq  Z  I  Z' := Set.subset_inter hQZ (by rw [hQQ];
    exact hQZ')
        have hMemQ : MemS D Qq := (memS_embPair_inv hD (hWeq  |>  hW)).2
        exact <hW, memS_inter hD hZ hZ' hMemQ hQsub, Or.inr <Pp, Qq, hWeq, hQsub>>
  mono := by
    rintro W W' Z Z' <hW, hZ, hd> hW'W hZZ' hW'mem hZ'mem
    refine <hW'mem, hZ'mem, ?_>
    rcases hd with rfl | <Pp, Qq, hWeq, hQZ>
    * exact Or.inl (Set.Subset.antisymm (memS_subset_gamma hZ'mem) hZZ')
    * subst hWeq
      cases hW'mem with
      | gamma =>
        exact absurd (Set.Subset.antisymm (memS_subset_gamma hW) hW'W)
          (embPair_ne_Gamma D hD Pp Qq)
      | @zero X hX =>
        exfalso
        obtain <a, ha> := hD X hX
        rcases hW'W (show (([], a) : List Bool  x  a)  in  embZero X from <rfl, ha>)
    with
          <p', hp', -> | <q', hq', ->
        * simp at hp'
        * simp at hq'
      | @pair Pp' Qq' hP' hQ' =>
        obtain <-, hQ'Q> := embPair_subset.mp hW'W
        exact Or.inr <Pp', Qq', rfl, fun z hz => hZZ' (hQZ (hQ'Q hz))>

@[simp] theorem proj1Map_rel {W Z : Set (List Bool  x  a)} :
    (proj1Map hD).rel W Z  <->
      MemS D W  /\  MemS D Z  /\  (Z = Gamma D  \/  exists  Pp Qq, W = embPair Pp Qq
    /\  Qq  subseteq  Z) := Iff.rfl

/-- **Example 6.1's pairing constructor `pair : D^S  x  D^S  ->  D^S`, as a joint
    approximable map.**
`(A  U  B) pair W  <->  A, B  in  ?^S  /\  W  in  ?^S  /\  1*A  U  2*B  subseteq  W` (
    with `A  U  B = prodNbhd A B`). -/
def pairSharpMap : ApproximableMap (prod (Dsharp D hD) (Dsharp D hD)) (Dsharp D hD)
    where
  rel V W := (prod (Dsharp D hD) (Dsharp D hD)).mem V  /\  MemS D W  /\
    exists  A B, MemS D A  /\  MemS D B  /\  V = prodNbhd A B  /\  embPair A B
    subseteq  W
```

```
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := <(prod (Dsharp D hD) (Dsharp D hD)).master_mem, MemS.gamma,
    <Gamma D, Gamma D, MemS.gamma, MemS.gamma, rfl, embPair_subset_Gamma subset_rfl
    subset_rfl>>
  inter_right := by
    rintro V W W' <hV, hW, A, B, hA, hB, rfl, hsub> <-, hW', A', B', hA', hB', heq,
    hsub'>
    obtain <rfl, rfl> := prodNbhd_injective heq
    exact <hV, memS_inter hD hW hW' (MemS.pair hA hB) (Set.subset_inter hsub hsub'),
      A, B, hA, hB, rfl, Set.subset_inter hsub hsub'>
  mono := by
    rintro V V' W W' <hV, hW, A, B, hA, hB, rfl, hsub> hV'V hWW' hV'mem hW'mem
    obtain <A', B', hA', hB', rfl> := hV'mem
    obtain <hA'A, hB'B> := prodNbhd_subset_iff.mp hV'V
    exact <prod_mem_prodNbhd hA' hB', hW'mem, A', B', hA', hB', rfl,
      (embPair_subset.mpr <hA'A, hB'B>).trans (hsub.trans hWW')>

@[simp] theorem pairSharpMap_rel {V : Set ((List Bool  x  a) ? (List Bool  x  a))}
    {W : Set (List Bool  x  a)} :
    (pairSharpMap hD).rel V W  <->
      (prod (Dsharp D hD) (Dsharp D hD)).mem V  /\  MemS D W  /\
        exists  A B, MemS D A  /\  MemS D B  /\  V = prodNbhd A B  /\  embPair A B
    subseteq  W := Iff.rfl

/-- **`pairSharpMap` really is Example 6.1's pairing:** `pair(x, y)  |->  <x, y>^S` (`
    pairSharp`). -/
theorem pairSharpMap_toElementMap (x y : (Dsharp D hD).Element) :
    (pairSharpMap hD).toElementMap (DomainTheory.NeighbourhoodSystems.pair x y) =
    pairSharp D hD x y := by
  apply Element.ext
  intro W
  simp only [ApproximableMap.mem_toElementMap, pairSharpMap_rel, mem_pair]
  constructor
  * rintro <V, <A0, B0, hxA0, hyB0, rfl>, -, hW, A, B, -, -, heq, hsub>
    obtain <rfl, rfl> := prodNbhd_injective heq
    exact (pairSharp D hD x y).up_mem (Or.inr <A0, B0, hxA0, hyB0, rfl>) hW hsub
  * intro hW
    rcases hW with rfl | <P, Q, hxP, hyQ, rfl>
    * exact <prodNbhd (Gamma D) (Gamma D), <Gamma D, Gamma D, x.master_mem, y.
    master_mem, rfl>,
        <Gamma D, Gamma D, MemS.gamma, MemS.gamma, rfl>, MemS.gamma,
        Gamma D, Gamma D, MemS.gamma, MemS.gamma, rfl,
        embPair_subset_Gamma subset_rfl subset_rfl>
    * exact <prodNbhd P Q, <P, Q, hxP, hyQ, rfl>, prod_mem_prodNbhd (x.sub hxP) (y.sub
     hyQ),
        MemS.pair (x.sub hxP) (y.sub hyQ), P, Q, x.sub hxP, y.sub hyQ, rfl, subset_rfl
    >

/-- The index characterization of `pairSharpMap` against the product/`Vsharp`
    enumerations:
`(V?_{p t}  U  V1_{q t}) pair V_k  <->  V_{2*t+2}  subseteq  V_k` (since `V_{2t+2} = 1*
    V_{p t}  U  2*V_{q t}`). -/
theorem pairSharp_rel_Vsharp_iff (P : ComputablePresentation D) (t k : Nat) :
    (pairSharpMap hD).rel
        ((prodPresentation (dsharpPresentation P hD) (dsharpPresentation P hD)).X t)
```

```
        ((dsharpPresentation P hD).X k)  <->
      Vsharp D P (2 * t + 2)  subseteq  Vsharp D P k := by
  rw [pairSharpMap_rel, prodPresentation_X, Vsharp_even]
  constructor
  * rintro <-, -, A, B, -, -, heq, hsub>
    obtain <rfl, rfl> := prodNbhd_injective heq
    exact hsub
  * intro hsub
    exact <prod_mem_prodNbhd (Vsharp_mem P _) (Vsharp_mem P _), Vsharp_mem P k,
      _, _, Vsharp_mem P _, Vsharp_mem P _, rfl, hsub>

end Proposition77
end
end DomainTheory.NeighbourhoodSystems
end
```

3.8.21 Exercise 7.19 — power domain functor

Prove that $D \mapsto \mathbb{P}D$ is a functor by defining for each $f : D \to E$ a mapping

$$\mathbb{P}f : \mathbb{P}D \to \mathbb{P}E$$

by the formula

$$\bigcup_{i<n} \downarrow X_i \quad \mathbb{P}f \quad \bigcup_{j<m} \downarrow Y_j \quad \text{iff} \quad \forall i < n \exists j < m.\, X_i f Y_j\,.$$

Be sure to check that $\mathbb{P}f$ is approximable and that $\mathbb{P}$ preserves identity maps and composition. If f is computable, is $\mathbb{P}f$? Is there a combinator $\lambda f.\, \mathbb{P}f$? What is

$$\mathbb{P}f(\{x, y\}) = ??$$

Exercise_7_19.lean

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Proposition_7_10

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Definition_7_11

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem Domain.Recursive ApproximableMap
variable {a b ? : Type*}
variable {V : NeighborhoodSystem a} {W : NeighborhoodSystem b} {U : NeighborhoodSystem
     ?}

/- concordance: source L4663 EXERCISE 7.19 power domain functor -/
```

```
/-- **Exercise 7.19 (Scott 1981, PRG-19) -- the functorial action `?f : ?D  ->  ?E`.**
    Given an
approximable map `f : D  ->  E`, the relation between `?D`- and `?E`-neighbourhoods is
    "every member of
the source is `f`-related to some member of the target". This is the representation-
    independent form
of Scott's displayed `forall  i < n exists  j < m. X_i f Y?` (see `PFmap_rel_fin`). -/
def PFmap (f : ApproximableMap V W) : ApproximableMap V.PowerDomain W.PowerDomain
    where
  rel A B := V.PDmem A  /\  W.PDmem B  /\  forall  X  in  A, exists  Y  in  B, f.rel X
     Y
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := by
    refine <V.PDmem_master, W.PDmem_master, ?_>
    rintro X <hXmem, hXsub>
    exact <W.master, <W.master_mem, subset_rfl>,
      f.mono f.master_rel hXsub subset_rfl hXmem W.master_mem>
  inter_right := by
    rintro A B B' <hA, hB, hbody> <-, hB', hbody'>
    refine <hA, W.PDmem_inter hB hB', ?_>
    intro X hX
    obtain <Y, hYB, hrel> := hbody X hX
    obtain <Y', hY'B', hrel'> := hbody' X hX
    have hinmem : W.mem (Y  I  Y') := f.rel_cod (f.inter_right hrel hrel')
    exact <Y  I  Y', Set.mem_inter
      (W.PDmem_down hB hYB Set.inter_subset_left hinmem)
      (W.PDmem_down hB' hY'B' Set.inter_subset_right hinmem),
      f.inter_right hrel hrel'>
  mono := by
    rintro A A' B B' <_, _, hbody> hA'A hBB' hA' hB'
    refine <hA', hB', ?_>
    intro X hX
    obtain <Y, hYB, hrel> := hbody X (hA'A hX)
    exact <Y, hBB' hYB, hrel>

@[simp] theorem PFmap_rel {f : ApproximableMap V W} {A : Set (Set a)} {B : Set (Set b)
    } :
    (PFmap f).rel A B  <->  V.PDmem A  /\  W.PDmem B  /\  forall  X  in  A, exists  Y
     in  B, f.rel X Y := Iff.rfl

/-- **Exercise 7.19 -- the combinator `fun f. ?f : (D  ->  E)  ->  (?D  ->  ?E)`.** The
     flat Smyth-style lift
`(?, ?)  |->  mem ?  /\  mem ?  /\  forall  f  in  ?, ?f  in  ?` of Exercise 7.19's
    functorial action, one level up on the
function spaces (`funSpace`, Theorem 3.10) -- the same recipe as Exercise 7.16's `
    curryComb` and
Exercise 7.21's `papplyEval`. Approximability is immediate: unlike `PFmap` itself, the
    body is a bare
membership check with no witnesses to gather. -/
def PFmapComb (V : NeighborhoodSystem a) (W : NeighborhoodSystem b) :
    ApproximableMap (funSpace V W) (funSpace V.PowerDomain W.PowerDomain) where
  rel ? ? := (funSpace V W).mem ?  /\  (funSpace V.PowerDomain W.PowerDomain).mem ?
    /\
    forall  f  in  ?, PFmap f  in  ?
  rel_dom h := h.1
```

```
  rel_cod h := h.2.1
  master_rel :=
    <(funSpace V W).master_mem, (funSpace V.PowerDomain W.PowerDomain).master_mem,
      fun _ _ => Set.mem_univ _>
  inter_right := by
    rintro ? ? ?' <h?, h?, hbody> <-, h?', hbody'>
    obtain <f_0, hf_0> := h?.2
    obtain <<L_2, hL_2, rfl>, -> := h?
    obtain <<L_2', hL_2', rfl>, -> := h?'
    refine <h?, <<L_2 ++ L_2', ?_, (stepFun_append _ _).symm>,
        <PFmap f_0, Set.mem_inter (hbody f_0 hf_0) (hbody' f_0 hf_0)>>,
      fun f hf => Set.mem_inter (hbody f hf) (hbody' f hf)>
    intro q hq
    rcases List.mem_append.mp hq with hq' | hq'
    * exact hL_2 q hq'
    * exact hL_2' q hq'
  mono := by
    rintro ? ?' ? ?' <_, _, hbody> h?'? h??' h?' h?'
    exact <h?', h?', fun f hf => h??' (hbody f (h?'? hf))>

end

section
open NeighborhoodSystem Domain.Recursive ApproximableMap
variable {a b ? : Type*}

namespace NeighborhoodSystem

variable (V : NeighborhoodSystem a)

end NeighborhoodSystem

variable {V : NeighborhoodSystem a} {W : NeighborhoodSystem b} {U : NeighborhoodSystem
     ?}

@[simp] theorem PFmapComb_rel {? : Set (ApproximableMap V W)}
    {? : Set (ApproximableMap V.PowerDomain W.PowerDomain)} :
    (PFmapComb V W).rel ? ?  <->
      (funSpace V W).mem ?  /\  (funSpace V.PowerDomain W.PowerDomain).mem ?  /\
        forall  f  in  ?, PFmap f  in  ? := Iff.rfl
end

section
open NeighborhoodSystem Domain.Recursive ApproximableMap
variable {a b ? : Type*}

namespace NeighborhoodSystem

variable (V : NeighborhoodSystem a)

/-- **Every member of a `??`-neighbourhood is a `?`-neighbourhood.** -/
theorem PDmem_mem {A : Set (Set a)} (hA : V.PDmem A) {X : Set a} (hX : X  in  A) : V.
    mem X := by
  obtain <L, hL, rfl> := hA
  obtain <X', _, hXmem, _> := (V.mem_PDunion).mp hX
  exact hXmem
```

```
end NeighborhoodSystem
end

section
open NeighborhoodSystem Domain.Recursive ApproximableMap
variable {a b ? : Type*}
variable {V : NeighborhoodSystem a} {W : NeighborhoodSystem b} {U : NeighborhoodSystem
     ?}

/-- **Exercise 7.19 -- `?` preserves identity maps.** `?(I_D) = I_{?D}`. The body
`forall  X  in  A, exists  Y  in  B, X  subseteq  Y` is equivalent to `A  subseteq  B`
    by downward closure of `B`. -/
theorem PFmap_idMap (V : NeighborhoodSystem a) : PFmap (idMap V) = idMap V.PowerDomain
     := by
  apply ApproximableMap.ext
  intro A B
  simp only [PFmap_rel, idMap_rel]
  constructor
  * rintro <hA, hB, hbody>
    refine <hA, hB, ?_>
    intro X hX
    obtain <Y, hYB, hXmem, _, hXY> := hbody X hX
    exact V.PDmem_down hB hYB hXY hXmem
  * rintro <hA, hB, hsub>
    refine <hA, hB, ?_>
    intro X hX
    exact <X, hsub hX, V.PDmem_mem hA hX, V.PDmem_mem hA hX, subset_rfl>

/-- **Exercise 7.19 -- the relation of `?f` at the level of `??`-codes.** Over the
    canonical
presentations of Proposition 7.10, `(?f).rel (Y_c) (Y_d)  <->  forall  a  in  dl c,
    exists  b  in  dl d, X? f Y_b`. -/
theorem PFmap_rel_Ypd_iff (P : ComputablePresentation V) (Q : ComputablePresentation W
    )
    (f : ApproximableMap V W) {c d : Nat} :
    (PFmap f).rel (V.Ypd P c) (W.Ypd Q d)  <->
      forall  a  in  decodeList c, exists  b  in  decodeList d, f.rel (P.X a) (Q.X b)
    := by
  constructor
  * rintro <-, -, hbody> a ha
    have hXin : P.X a  in  V.Ypd P c := (V.mem_Ypd P).mpr <a, ha, P.mem_X a,
    subset_rfl>
    obtain <Y, hYin, hrel> := hbody (P.X a) hXin
    obtain <b, hb, hYmem, hYsub> := (W.mem_Ypd Q).mp hYin
    exact <b, hb, f.mono hrel subset_rfl hYsub (P.mem_X a) (Q.mem_X b)>
  * intro hgen
    refine <V.Ypd_isPDmem P c, W.Ypd_isPDmem Q d, ?_>
    rintro X hXin
    obtain <a, ha, hXmem, hXsub> := (V.mem_Ypd P).mp hXin
    obtain <b, hb, hrel> := hgen a ha
    exact <Q.X b, (W.mem_Ypd Q).mpr <b, hb, Q.mem_X b, subset_rfl>,
      f.mono hrel hXsub subset_rfl hXmem (Q.mem_X b)>

/-- **Exercise 7.19 -- Scott's displayed formula.** On generators (finite lists of
    down-sets)
```

```
`(?f).rel (?_{X in L_1} X) (?_{Y in L_2} Y)  <->  forall  X  in  L_1, exists  Y  in
    L_2, X f Y`, exactly Scott's
`forall  i < n exists  j < m. X_i f Y?`. (` -> ` reads off the generators `X_i`; ` <- `
     extends to all members by `mono`
and downward closure of the target.) -/
theorem PFmap_rel_fin (f : ApproximableMap V W) {L_1 : List (Set a)} {L_2 : List (Set
    b)}
    (hL_1 : forall  X  in  L_1, V.mem X) (hL_2 : forall  Y  in  L_2, W.mem Y) :
    (PFmap f).rel (? X  in  L_1, V.upSet X) (? Y  in  L_2, W.upSet Y)
       <->  forall  X  in  L_1, exists  Y  in  L_2, f.rel X Y := by
  constructor
  * rintro <-, -, hbody> X hX
    have hXin : X  in  ? X'  in  L_1, V.upSet X' :=
      (V.mem_PDunion).mpr <X, hX, hL_1 X hX, subset_rfl>
    obtain <Z, hZin, hrel> := hbody X hXin
    obtain <Y, hY, hZmem, hZsub> := (W.mem_PDunion).mp hZin
    exact <Y, hY, f.mono hrel subset_rfl hZsub (hL_1 X hX) (hL_2 Y hY)>
  * intro hgen
    refine <<L_1, hL_1, rfl>, <L_2, hL_2, rfl>, ?_>
    rintro Z hZin
    obtain <X, hX, hZmem, hZsub> := (V.mem_PDunion).mp hZin
    obtain <Y, hY, hrel> := hgen X hX
    exact <Y, (W.mem_PDunion).mpr <Y, hY, hL_2 Y hY, subset_rfl>,
      f.mono hrel hZsub subset_rfl hZmem (hL_2 Y hY)>

/-- **Bounded `exists ` over a coded list preserves recursive enumerability.** If `R`
    is r.e. then so is
`fun d a => exists  b  in  decodeList d, R a b`: project out `b` (`REPred.proj`) over
    the conjunction of the
*recursively decidable* membership `b  in  decodeList d` with the r.e. body `R a b`. -/
theorem bExists_decodeList_re {R : Nat  ->  Nat  ->  Prop} (hR : REPred_2 R) :
    REPred_2 (fun d a => exists  b  in  decodeList d, R a b) := by
  have hmem : RecDecidable_2 (fun d b => b  in  decodeList d) := by
    have heq : RecDecidable_2 (fun e b => e = b) :=
      RecDecidable.natEq Nat.Primrec.left Nat.Primrec.right
    have hbase : RecDecidable_2 (fun d b => exists  e  in  decodeList d, e = b) := heq
    .bExistsList
    refine RecDecidable.of_iff (fun t => ?_) hbase
    constructor
    * intro h; exact <_, h, rfl>
    * rintro <e, he, rfl>; exact he
  have hpart1 : REPred (fun w : Nat => w.unpair.1  in  decodeList w.unpair.2.unpair.1)
     := by
    have hg : Nat.Primrec (fun w : Nat => Nat.pair w.unpair.2.unpair.1 w.unpair.1) :=
      (Nat.Primrec.left.comp Nat.Primrec.right).pair Nat.Primrec.left
    refine RecDecidable.re (RecDecidable.of_iff (fun w => ?_) (hmem.comp hg))
    simp only [unpair_pair_fst, unpair_pair_snd]
  have hpart2 : REPred (fun w : Nat => R w.unpair.2.unpair.2 w.unpair.1) := by
    have hg : Nat.Primrec (fun w : Nat => Nat.pair w.unpair.2.unpair.2 w.unpair.1) :=
      (Nat.Primrec.right.comp Nat.Primrec.right).pair Nat.Primrec.left
    refine REPred.of_iff (fun w => ?_) (REPred.comp hR hg)
    simp only [unpair_pair_fst, unpair_pair_snd]
  refine REPred.of_iff (fun s => ?_) (hpart1.and hpart2).proj
  simp only [unpair_pair_fst, unpair_pair_snd]

/-- **Exercise 7.19 (Scott's first question) -- `?f` is computable when `f` is.**
```

```
    Stated at the level
of the `??`-codes of Proposition 7.10 (`Y_c = ?_{a in dl c} X?`): the relation `(?f).
    rel (Y_c) (Y_d)`
is recursively enumerable in `(c, d)`. -/
theorem PFmap_isComputable_Ypd (P : ComputablePresentation V) (Q :
    ComputablePresentation W)
    {f : ApproximableMap V W} (hf : ComputableMap P Q f) :
    REPred_2 (fun c d => (PFmap f).rel (V.Ypd P c) (W.Ypd Q d)) := by
  have hR : REPred_2 (fun a b => f.rel (P.X a) (Q.X b)) := hf.out
  have hInner : REPred_2 (fun d a => exists  b  in  decodeList d, f.rel (P.X a) (Q.X b
    )) :=
    bExists_decodeList_re hR
  have hForall := REPred.forall_mem_decodeList_2 hInner
  have hswap : Nat.Primrec (fun s : Nat => Nat.pair s.unpair.2 s.unpair.1) :=
    Nat.Primrec.right.pair Nat.Primrec.left
  refine REPred.of_iff (fun s => ?_) (hForall.comp hswap)
  simp only [unpair_pair_fst, unpair_pair_snd]
  exact PFmap_rel_Ypd_iff P Q f

/-- **Exercise 7.19 (Scott's first question), packaged.** `?f` is a computable map
    between the
canonical effective presentations `??`, `??` (Proposition 7.10) whenever `f` is
    computable. -/
theorem PFmap_isComputable (P : ComputablePresentation V) (Q : ComputablePresentation
    W)
    (consV : Nat  ->  Nat) (hconsVp : Nat.Primrec consV)
    (hconsVs : forall  a b, consV (Nat.pair a b) = 1  <->  exists  k, P.X k  subseteq
     P.X a  I  P.X b)
    (consW : Nat  ->  Nat) (hconsWp : Nat.Primrec consW)
    (hconsWs : forall  a b, consW (Nat.pair a b) = 1  <->  exists  k, Q.X k  subseteq
     Q.X a  I  Q.X b)
    {f : ApproximableMap V W} (hf : ComputableMap P Q f) :
    ComputableMap (V.PDPresentation P consV hconsVp hconsVs)
      (W.PDPresentation Q consW hconsWp hconsWs) (PFmap f) := by
  exact <PFmap_isComputable_Ypd P Q hf>

/-- Choice-free gathering of composition witnesses along the generators of the source:
     from a list
`L` of `?`-neighbourhoods each admitting a `(g comp f)`-witness into `C`, build a
    single list `M` of
`?`-neighbourhoods covering every `X  in  L` from the `f`-side and covered into `C`
    from the `g`-side.
Proved by list recursion `obtain`-ing the witnesses (no choice in a `Prop` goal). -/
theorem comp_witness (f : ApproximableMap V W) (g : ApproximableMap W U) (C : Set (Set
     ?)) :
    forall  (L : List (Set a)), (forall  X  in  L, V.mem X)  ->
      (forall  X  in  L, exists  Z, Z  in  C  /\  exists  Y, f.rel X Y  /\  g.rel Y Z)
      ->
      exists  M : List (Set b), (forall  Y  in  M, W.mem Y)  /\
        (forall  X  in  L, exists  Y  in  M, f.rel X Y)  /\  (forall  Y  in  M, exists
      Z  in  C, g.rel Y Z) := by
  intro L
  induction L with
  | nil =>
    intro _ _
    refine <[], ?_, ?_, ?_>
```

```
    * intro Y hY; cases hY
    * intro X hX; cases hX
    * intro Y hY; cases hY
  | cons X L ih =>
    intro hL hpre
    obtain <Zx, hZxC, Yx, hfx, hgx> := hpre X (List.mem_cons.mpr (Or.inl rfl))
    obtain <M, hMmem, hfM, hgM> :=
      ih (fun X' hX' => hL X' (List.mem_cons.mpr (Or.inr hX')))
         (fun X' hX' => hpre X' (List.mem_cons.mpr (Or.inr hX')))
    refine <Yx :: M, ?_, ?_, ?_>
    * intro Y hY
      rcases List.mem_cons.mp hY with rfl | hY'
      * exact f.rel_cod hfx
      * exact hMmem Y hY'
    * intro X' hX'
      rcases List.mem_cons.mp hX' with rfl | hX''
      * exact <Yx, List.mem_cons.mpr (Or.inl rfl), hfx>
      * obtain <Y, hYM, hrel> := hfM X' hX''
        exact <Y, List.mem_cons.mpr (Or.inr hYM), hrel>
    * intro Y hY
      rcases List.mem_cons.mp hY with rfl | hY'
      * exact <Zx, hZxC, hgx>
      * exact hgM Y hY'

/-- **Exercise 7.19 -- `?` preserves composition.** `?(g  comp  f) = ?g  comp  ?f`. The
     forward direction
constructs the intermediate `?E`-neighbourhood from `comp_witness`; the backward
    direction merely
threads the existential witnesses. -/
theorem PFmap_comp (g : ApproximableMap W U) (f : ApproximableMap V W) :
    PFmap (g.comp f) = (PFmap g).comp (PFmap f) := by
  apply ApproximableMap.ext
  intro A C
  simp only [PFmap_rel, comp_rel]
  constructor
  * rintro <hA, hC, hbody>
    obtain <L, hL, rfl> := hA
    have hpre : forall  X  in  L, exists  Z, Z  in  C  /\  exists  Y, f.rel X Y  /\  g
    .rel Y Z := by
      intro X hX
      obtain <Z, hZC, Y, hfXY, hgYZ> :=
        hbody X ((V.mem_PDunion).mpr <X, hX, hL X hX, subset_rfl>)
      exact <Z, hZC, Y, hfXY, hgYZ>
    obtain <M, hMmem, hfL, hgM> := comp_witness f g C L hL hpre
    refine <? Y  in  M, W.upSet Y,
      <<L, hL, rfl>, <M, hMmem, rfl>, ?_>, <M, hMmem, rfl>, hC, ?_>
    * intro X hX
      obtain <X', hX', hXmem, hXsub> := (V.mem_PDunion).mp hX
      obtain <Y, hYM, hrel> := hfL X' hX'
      exact <Y, (W.mem_PDunion).mpr <Y, hYM, hMmem Y hYM, subset_rfl>,
        f.mono hrel hXsub subset_rfl hXmem (hMmem Y hYM)>
    * intro Y hY
      obtain <Y', hY', hYmem, hYsub> := (W.mem_PDunion).mp hY
      obtain <Z, hZC, hrel> := hgM Y' hY'
      exact <Z, hZC, g.mono hrel hYsub subset_rfl hYmem (g.rel_cod hrel)>
  * rintro <B, <hA, hB, hfAB>, _, hC, hgBC>
```

```
    refine <hA, hC, ?_>
    intro X hX
    obtain <Y, hYB, hfXY> := hfAB X hX
    obtain <Z, hZC, hgYZ> := hgBC Y hYB
    exact <Z, hZC, Y, hfXY, hgYZ>

/-- **Choice-free witness gathering (single list).** From `forall  X  in  L_1, exists
    Y  in  L_2, f.rel X Y`, build one
list `M` of `(X, Y)` pairs, each drawn from `L_1  x  L_2` and `f`-related, that *covers
    * `L_1`: every
`X  in  L_1` has a matching `Y` with `(X, Y)  in  M`. (The one-map analogue of `
    comp_witness`.) -/
theorem gather_witnesses (f : ApproximableMap V W) :
    forall  (L_1 : List (Set a)) (L_2 : List (Set b)), (forall  X  in  L_1, exists  Y
     in  L_2, f.rel X Y)  ->
      exists  M : List (Set a  x  Set b), (forall  p  in  M, p.1  in  L_1  /\  p.2  in
      L_2  /\  f.rel p.1 p.2)  /\
        forall  X  in  L_1, exists  Y  in  L_2, (X, Y)  in  M := by
  intro L_1
  induction L_1 with
  | nil => intro L_2 _; exact <[], by simp, by simp>
  | cons X L_1 ih =>
    intro L_2 h
    obtain <Y, hY, hrel> := h X (List.mem_cons.mpr (Or.inl rfl))
    obtain <M, hMprop, hMwit> := ih L_2 (fun X' hX' => h X' (List.mem_cons.mpr (Or.inr
     hX')))
    refine <(X, Y) :: M, ?_, ?_>
    * rintro p hp
      rcases List.mem_cons.mp hp with rfl | hp'
      * exact <List.mem_cons.mpr (Or.inl rfl), hY, hrel>
      * obtain <hX1, hY1, hrel1> := hMprop p hp'
        exact <List.mem_cons.mpr (Or.inr hX1), hY1, hrel1>
    * intro X' hX'
      rcases List.mem_cons.mp hX' with rfl | hX''
      * exact <Y, hY, List.mem_cons.mpr (Or.inl rfl)>
      * obtain <Y', hY', hmem> := hMwit X' hX''
        exact <Y', hY', List.mem_cons.mpr (Or.inr hmem)>

/-- **Choice-free witness gathering (list of step constraints).** From a finite list `L
    ` of
`?D  x  ?E`-neighbourhood pairs `(A, B)`, each satisfied by `?f` (`(?f).rel A B`),
    build a *single*
finite list `M` of `D  x  E`-pairs, `f`-related and covering every generator of every `
    A` in `L`, such
that *any* `g` agreeing with `f` on `M` again satisfies `(?g).rel A B` for every `(A, B)
      in  L`. This is
exactly "continuity of `f  |->  ?f`, unwound to finite data": `M` is the finite portion
     of `f` that
already forces `?f`'s membership in every step-neighbourhood of `L`. -/
theorem exists_stepFun_forall_PFmap_rel (f : ApproximableMap V W) :
    forall  (L : List (Set (Set a)  x  Set (Set b))),
      (forall  p  in  L, V.PDmem p.1  /\  W.PDmem p.2)  ->  (forall  p  in  L, (PFmap
    f).rel p.1 p.2)  ->
      exists  M : List (Set a  x  Set b), (forall  p  in  M, V.mem p.1  /\  W.mem p.2)
      /\
        (forall  p  in  M, f.rel p.1 p.2)  /\
```

```
        forall  g : ApproximableMap V W, (forall  p  in  M, g.rel p.1 p.2)  ->  forall
      p  in  L, (PFmap g).rel p.1 p.2 := by
  intro L
  induction L with
  | nil => intro _ _; exact <[], by simp, by simp, by simp>
  | cons p L ih =>
    intro hL hf
    obtain <hA, hB> := hL p (List.mem_cons.mpr (Or.inl rfl))
    obtain <L_1, hL_1, hAeq> := hA
    obtain <L_2, hL_2, hBeq> := hB
    have hrelAB : (PFmap f).rel p.1 p.2 := hf p (List.mem_cons.mpr (Or.inl rfl))
    rw [hAeq, hBeq] at hrelAB
    have hgen := (PFmap_rel_fin f hL_1 hL_2).mp hrelAB
    obtain <M_0, hM_0prop, hM_0wit> := gather_witnesses f L_1 L_2 hgen
    obtain <M, hMmem, hMrel, hMgen> :=
      ih (fun p' hp' => hL p' (List.mem_cons.mpr (Or.inr hp')))
         (fun p' hp' => hf p' (List.mem_cons.mpr (Or.inr hp')))
    refine <M_0 ++ M, ?_, ?_, ?_>
    * intro q hq
      rcases List.mem_append.mp hq with hq' | hq'
      * obtain <hq1, hq2, _> := hM_0prop q hq'
        exact <hL_1 q.1 hq1, hL_2 q.2 hq2>
      * exact hMmem q hq'
    * intro q hq
      rcases List.mem_append.mp hq with hq' | hq'
      * exact (hM_0prop q hq').2.2
      * exact hMrel q hq'
    * intro g hg p' hp'
      rcases List.mem_cons.mp hp' with rfl | hp''
      * rw [hAeq, hBeq]
        refine (PFmap_rel_fin g hL_1 hL_2).mpr ?_
        intro X hX
        obtain <Y, hY, hmem> := hM_0wit X hX
        exact <Y, hY, hg (X, Y) (List.mem_append.mpr (Or.inl hmem))>
      * exact hMgen g (fun q hq => hg q (List.mem_append.mpr (Or.inr hq))) p' hp''

/-- **Exercise 7.19 -- faithfulness of `PFmapComb`.** Under the function-space
    correspondence of
Theorem 3.10 (`toFilter f` is the point of `(D  ->  E)` fixed by `f`), `PFmapComb`
    really does send
`f` to `?f`, at **every** `f` -- not merely the finite/principal points. The forward
    direction is
immediate; the backward direction is where the work is: given `?f  in  ?`, `
    exists_stepFun_forall_PFmap_rel`
manufactures a finite neighbourhood `stepFun M ? f` all of whose members `g` already
    satisfy `?g  in  ?`,
i.e. `?` is reached from a genuine (finite) neighbourhood of `f`, exactly as continuity
     demands. -/
theorem PFmapComb_toElementMap (f : ApproximableMap V W) :
    (PFmapComb V W).toElementMap (toFilter f) = toFilter (PFmap f) := by
  apply Element.ext
  intro ?
  simp only [mem_toElementMap, mem_toFilter, PFmapComb_rel]
  constructor
  * rintro <?, <_, h?f>, _, h?mem, hbody>
    exact <h?mem, hbody f h?f>
```

```
  * rintro <h?mem, hf?>
    obtain <<L_2, hL_2, rfl>, -> := h?mem
    obtain <M, hMmem, hMrel, hMgen> :=
      exists_stepFun_forall_PFmap_rel f L_2 hL_2 (mem_stepFun.mp hf?)

    have h?mem : (funSpace V W).mem (stepFun M) := <<M, hMmem, rfl>, f, mem_stepFun.
    mpr hMrel>
    have h?mem' : (funSpace V.PowerDomain W.PowerDomain).mem (stepFun L_2) :=
      <<L_2, hL_2, rfl>, PFmap f, hf?>
    exact <stepFun M, <h?mem, mem_stepFun.mpr hMrel>, h?mem, h?mem',
      fun g hg => mem_stepFun.mpr (hMgen g (mem_stepFun.mp hg))>

end

section
set_option linter.unusedVariables false

theorem example_719.{u_1, u_2, u_3} : And
  (And
    (And
      (And
        (And
          (And
            (forall  {a : Type u_1} {b : Type u_2} {V : DomainTheory.
   NeighbourhoodSystems.NeighborhoodSystem a}
              {W : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
              (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W),
              @Eq
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
                  (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Set a) (Set b)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
    a V)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
    b W))
                  (@DomainTheory.NeighbourhoodSystems.funSpace (Set a) (Set b)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
    a V)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
    b W)))
                (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
                  (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W)
                  (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Set a) (Set b)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
    a V)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
    b W))
                  (@DomainTheory.NeighbourhoodSystems.funSpace a b V W)
                  (@DomainTheory.NeighbourhoodSystems.funSpace (Set a) (Set b)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
    a V)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
    b W))
                  (@DomainTheory.NeighbourhoodSystems.PFmapComb a b V W)
                  (@DomainTheory.NeighbourhoodSystems.toFilter a b V W f))
                (@DomainTheory.NeighbourhoodSystems.toFilter (Set a) (Set b)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a
```

```
 V)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain b
 W)
              (@DomainTheory.NeighbourhoodSystems.PFmap a b V W f)))
        (forall  {a : Type u_1} {b : Type u_2} {? : Type u_3} {V : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
          {W : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
          {U : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem ?}
          (g : @DomainTheory.NeighbourhoodSystems.ApproximableMap b ? W U)
          (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W),
          @Eq
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Set a) (Set ?)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a
 V)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain ?
 U))
            (@DomainTheory.NeighbourhoodSystems.PFmap a ? V U
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp a b ? V W U
 g f))
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp (Set a) (Set
b) (Set ?)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a
 V)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain b
 W)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain ?
 U)
              (@DomainTheory.NeighbourhoodSystems.PFmap b ? W U g)
              (@DomainTheory.NeighbourhoodSystems.PFmap a b V W f))))
      (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a),
        @Eq
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Set a) (Set a)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V
)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V
))
          (@DomainTheory.NeighbourhoodSystems.PFmap a a V V
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.idMap a V))
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.idMap (Set a)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V
))))
    (forall  {a : Type u_1} {b : Type u_2} {V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
      {W : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
      (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V)
      (Q : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b W) (consV :
 Nat  ->  Nat)
      (hconsVp : Nat.Primrec consV)
      (hconsVs :
        forall  (a b : Nat),
          Iff (@Eq Nat (consV (Nat.pair a b)) (@OfNat.ofNat Nat (nat_lit 1) (
instOfNatNat (nat_lit 1))))
            (exists  (k : Nat),
              @LE.le (Set a) (@Set.instLE a) (@DomainTheory.NeighbourhoodSystems.
ComputablePresentation.X a V P k)
```

```
                (@Inter.inter (Set a) (@Set.instInter a)
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V
 P a)
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V
 P b))))
      (consW : Nat  ->  Nat) (hconsWp : Nat.Primrec consW)
      (hconsWs :
        forall  (a b : Nat),
          Iff (@Eq Nat (consW (Nat.pair a b)) (@OfNat.ofNat Nat (nat_lit 1) (
instOfNatNat (nat_lit 1))))
            (exists  (k : Nat),
              @LE.le (Set b) (@Set.instLE b) (@DomainTheory.NeighbourhoodSystems.
ComputablePresentation.X b W Q k)
                (@Inter.inter (Set b) (@Set.instInter b)
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b W
 Q a)
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b W
 Q b))))
      {f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W},
      @DomainTheory.NeighbourhoodSystems.ComputableMap a b V W P Q f  ->
        @DomainTheory.NeighbourhoodSystems.ComputableMap (Set a) (Set b)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain b W)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDPresentation a
V P consV hconsVp hconsVs)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDPresentation b
W Q consW hconsWp hconsWs)
          (@DomainTheory.NeighbourhoodSystems.PFmap a b V W f)))
  (forall  {a : Type u_1} {b : Type u_2} {V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
    {W : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
    (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V)
    (Q : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b W)
    {f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W},
    @DomainTheory.NeighbourhoodSystems.ComputableMap a b V W P Q f  ->
      Domain.Recursive.REPred_2 fun (c d : Nat) =>
        @DomainTheory.NeighbourhoodSystems.ApproximableMap.rel (Set a) (Set b)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain b W)
          (@DomainTheory.NeighbourhoodSystems.PFmap a b V W f)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd a V P c)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd b W Q d)))
(forall  {a : Type u_1} {b : Type u_2} {V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
  {W : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
  (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V)
  (Q : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b W)
  (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W) {c d : Nat},
  Iff
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel (Set a) (Set b)
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V)
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain b W)
      (@DomainTheory.NeighbourhoodSystems.PFmap a b V W f)
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd a V P c)
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd b W Q d))
    (forall  (a : Nat),
```

```
      @Membership.mem Nat (List Nat) (@List.instMembership Nat) (Domain.Recursive.
  decodeList c) a  ->
        exists  (b : Nat),
          And (@Membership.mem Nat (List Nat) (@List.instMembership Nat) (Domain.
  Recursive.decodeList d) b)
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel a b V W f
              (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V P a
  )
              (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b W Q b
  )))))
(forall  {a : Type u_1} {b : Type u_2} {V : DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem a}
  {W : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
  (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W) {L_1 : List (Set
  a)} {L_2 : List (Set b)},
  (forall  (X : Set a),
      @Membership.mem (Set a) (List (Set a)) (@List.instMembership (Set a)) L_1 X
  ->
       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X)  ->
    (forall  (Y : Set b),
       @Membership.mem (Set b) (List (Set b)) (@List.instMembership (Set b)) L_2 Y
   ->
         @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b W Y)  ->
      Iff
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel (Set a) (Set b)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain b W)
          (@DomainTheory.NeighbourhoodSystems.PFmap a b V W f)
          (? (X : Set a),
            ? (_ : @Membership.mem (Set a) (List (Set a)) (@List.instMembership (Set
   a)) L_1 X),
              @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.upSet a V X)
          (? (Y : Set b),
            ? (_ : @Membership.mem (Set b) (List (Set b)) (@List.instMembership (Set
   b)) L_2 Y),
              @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.upSet b W Y))
        (forall  (X : Set a),
          @Membership.mem (Set a) (List (Set a)) (@List.instMembership (Set a)) L_1
  X  ->
           exists  (Y : Set b),
             And (@Membership.mem (Set b) (List (Set b)) (@List.instMembership (Set
   b)) L_2 Y)
               (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel a b V W f X
  Y))) :=
@And.intro
(And
  (And
    (And
      (And
        (And
          (forall  {a : Type u_1} {b : Type u_2} {V : DomainTheory.
  NeighbourhoodSystems.NeighborhoodSystem a}
            {W : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
            (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W),
            @Eq
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
```

```
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Set a) (Set b)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 b W))
              (@DomainTheory.NeighbourhoodSystems.funSpace (Set a) (Set b)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 b W)))
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W)
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Set a) (Set b)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 b W))
              (@DomainTheory.NeighbourhoodSystems.funSpace a b V W)
              (@DomainTheory.NeighbourhoodSystems.funSpace (Set a) (Set b)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 b W))
              (@DomainTheory.NeighbourhoodSystems.PFmapComb a b V W)
              (@DomainTheory.NeighbourhoodSystems.toFilter a b V W f))
            (@DomainTheory.NeighbourhoodSystems.toFilter (Set a) (Set b)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a
 V)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain b
 W)
              (@DomainTheory.NeighbourhoodSystems.PFmap a b V W f)))
        (forall  {a : Type u_1} {b : Type u_2} {? : Type u_3} {V : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
          {W : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
          {U : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem ?}
          (g : @DomainTheory.NeighbourhoodSystems.ApproximableMap b ? W U)
          (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W),
          @Eq
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Set a) (Set ?)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a
 V)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain ?
 U))
            (@DomainTheory.NeighbourhoodSystems.PFmap a ? V U
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp a b ? V W U
 g f))
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp (Set a) (Set
b) (Set ?)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a
 V)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain b
 W)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain ?
 U)
              (@DomainTheory.NeighbourhoodSystems.PFmap b ? W U g)
              (@DomainTheory.NeighbourhoodSystems.PFmap a b V W f))))
```

```
      (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a),
        @Eq
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Set a) (Set a)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V
)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V
))
          (@DomainTheory.NeighbourhoodSystems.PFmap a a V V
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.idMap a V))
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.idMap (Set a)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V
))))
    (forall  {a : Type u_1} {b : Type u_2} {V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
      {W : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
      (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V)
      (Q : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b W) (consV :
 Nat  ->  Nat)
      (hconsVp : Nat.Primrec consV)
      (hconsVs :
        forall  (a b : Nat),
          Iff (@Eq Nat (consV (Nat.pair a b)) (@OfNat.ofNat Nat (nat_lit 1) (
instOfNatNat (nat_lit 1))))
            (exists  (k : Nat),
              @LE.le (Set a) (@Set.instLE a) (@DomainTheory.NeighbourhoodSystems.
ComputablePresentation.X a V P k)
                (@Inter.inter (Set a) (@Set.instInter a)
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V
 P a)
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V
 P b))))
      (consW : Nat  ->  Nat) (hconsWp : Nat.Primrec consW)
      (hconsWs :
        forall  (a b : Nat),
          Iff (@Eq Nat (consW (Nat.pair a b)) (@OfNat.ofNat Nat (nat_lit 1) (
instOfNatNat (nat_lit 1))))
            (exists  (k : Nat),
              @LE.le (Set b) (@Set.instLE b) (@DomainTheory.NeighbourhoodSystems.
ComputablePresentation.X b W Q k)
                (@Inter.inter (Set b) (@Set.instInter b)
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b W
 Q a)
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b W
 Q b))))
      {f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W},
      @DomainTheory.NeighbourhoodSystems.ComputableMap a b V W P Q f  ->
        @DomainTheory.NeighbourhoodSystems.ComputableMap (Set a) (Set b)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain b W)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDPresentation a
V P consV hconsVp hconsVs)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDPresentation b
W Q consW hconsWp hconsWs)
          (@DomainTheory.NeighbourhoodSystems.PFmap a b V W f)))
  (forall  {a : Type u_1} {b : Type u_2} {V : DomainTheory.NeighbourhoodSystems.
```

```
 NeighborhoodSystem a}
     {W : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
     (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V)
     (Q : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b W)
     {f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W},
     @DomainTheory.NeighbourhoodSystems.ComputableMap a b V W P Q f  ->
       Domain.Recursive.REPred_2 fun (c d : Nat) =>
         @DomainTheory.NeighbourhoodSystems.ApproximableMap.rel (Set a) (Set b)
           (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V)
           (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain b W)
           (@DomainTheory.NeighbourhoodSystems.PFmap a b V W f)
           (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd a V P c)
           (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd b W Q d)))
 (forall  {a : Type u_1} {b : Type u_2} {V : DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem a}
   {W : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
   (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V)
   (Q : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b W)
   (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W) {c d : Nat},
   Iff
     (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel (Set a) (Set b)
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V)
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain b W)
       (@DomainTheory.NeighbourhoodSystems.PFmap a b V W f)
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd a V P c)
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd b W Q d))
     (forall  (a : Nat),
       @Membership.mem Nat (List Nat) (@List.instMembership Nat) (Domain.Recursive.
 decodeList c) a  ->
         exists  (b : Nat),
           And (@Membership.mem Nat (List Nat) (@List.instMembership Nat) (Domain.
 Recursive.decodeList d) b)
             (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel a b V W f
               (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V P a
 )
               (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b W Q b
 )))))
(forall  {a : Type u_1} {b : Type u_2} {V : DomainTheory.NeighbourhoodSystems.
 NeighborhoodSystem a}
 {W : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
 (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W) {L_1 : List (Set
 a)} {L_2 : List (Set b)},
 (forall  (X : Set a),
     @Membership.mem (Set a) (List (Set a)) (@List.instMembership (Set a)) L_1 X
 ->
       @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X)  ->
   (forall  (Y : Set b),
       @Membership.mem (Set b) (List (Set b)) (@List.instMembership (Set b)) L_2 Y
  ->
         @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b W Y)  ->
     Iff
       (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel (Set a) (Set b)
         (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V)
         (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain b W)
         (@DomainTheory.NeighbourhoodSystems.PFmap a b V W f)
         (? (X : Set a),
```

```
          ? (_ : @Membership.mem (Set a) (List (Set a)) (@List.instMembership (Set
 a)) L_1 X),
            @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.upSet a V X)
        (? (Y : Set b),
          ? (_ : @Membership.mem (Set b) (List (Set b)) (@List.instMembership (Set
 b)) L_2 Y),
            @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.upSet b W Y))
      (forall  (X : Set a),
        @Membership.mem (Set a) (List (Set a)) (@List.instMembership (Set a)) L_1
 X  ->
          exists  (Y : Set b),
            And (@Membership.mem (Set b) (List (Set b)) (@List.instMembership (Set
 b)) L_2 Y)
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel a b V W f X
 Y)))
(@And.intro
  (And
    (And
      (And
        (And
          (forall  {a : Type u_1} {b : Type u_2} {V : DomainTheory.
 NeighbourhoodSystems.NeighborhoodSystem a}
           {W : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
           (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W),
           @Eq
             (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
               (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Set a) (Set b)
                 (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V)
                 (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 b W))
               (@DomainTheory.NeighbourhoodSystems.funSpace (Set a) (Set b)
                 (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V)
                 (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 b W)))
             (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
               (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W)
               (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Set a) (Set b)
                 (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V)
                 (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 b W))
               (@DomainTheory.NeighbourhoodSystems.funSpace a b V W)
               (@DomainTheory.NeighbourhoodSystems.funSpace (Set a) (Set b)
                 (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V)
                 (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 b W))
               (@DomainTheory.NeighbourhoodSystems.PFmapComb a b V W)
               (@DomainTheory.NeighbourhoodSystems.toFilter a b V W f))
             (@DomainTheory.NeighbourhoodSystems.toFilter (Set a) (Set b)
               (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a
 V)
               (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain b
 W)
```

```
              (@DomainTheory.NeighbourhoodSystems.PFmap a b V W f)))
        (forall  {a : Type u_1} {b : Type u_2} {? : Type u_3} {V : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
          {W : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
          {U : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem ?}
          (g : @DomainTheory.NeighbourhoodSystems.ApproximableMap b ? W U)
          (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W),
          @Eq
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Set a) (Set ?)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a
 V)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain ?
 U))
            (@DomainTheory.NeighbourhoodSystems.PFmap a ? V U
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp a b ? V W U
 g f))
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp (Set a) (Set
b) (Set ?)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a
 V)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain b
 W)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain ?
 U)
              (@DomainTheory.NeighbourhoodSystems.PFmap b ? W U g)
              (@DomainTheory.NeighbourhoodSystems.PFmap a b V W f))))
      (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a),
        @Eq
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Set a) (Set a)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V
)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V
))
          (@DomainTheory.NeighbourhoodSystems.PFmap a a V V
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.idMap a V))
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.idMap (Set a)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V
))))
    (forall  {a : Type u_1} {b : Type u_2} {V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
      {W : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
      (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V)
      (Q : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b W) (consV :
 Nat  ->  Nat)
      (hconsVp : Nat.Primrec consV)
      (hconsVs :
        forall  (a b : Nat),
          Iff (@Eq Nat (consV (Nat.pair a b)) (@OfNat.ofNat Nat (nat_lit 1) (
instOfNatNat (nat_lit 1))))
            (exists  (k : Nat),
              @LE.le (Set a) (@Set.instLE a) (@DomainTheory.NeighbourhoodSystems.
ComputablePresentation.X a V P k)
                (@Inter.inter (Set a) (@Set.instInter a)
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V
 P a)
```

```
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V
 P b))))
      (consW : Nat  ->  Nat) (hconsWp : Nat.Primrec consW)
      (hconsWs :
        forall  (a b : Nat),
          Iff (@Eq Nat (consW (Nat.pair a b)) (@OfNat.ofNat Nat (nat_lit 1) (
instOfNatNat (nat_lit 1))))
            (exists  (k : Nat),
              @LE.le (Set b) (@Set.instLE b) (@DomainTheory.NeighbourhoodSystems.
ComputablePresentation.X b W Q k)
                (@Inter.inter (Set b) (@Set.instInter b)
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b W
 Q a)
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b W
 Q b))))
      {f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W},
      @DomainTheory.NeighbourhoodSystems.ComputableMap a b V W P Q f  ->
        @DomainTheory.NeighbourhoodSystems.ComputableMap (Set a) (Set b)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain b W)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDPresentation a
V P consV hconsVp hconsVs)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDPresentation b
W Q consW hconsWp hconsWs)
          (@DomainTheory.NeighbourhoodSystems.PFmap a b V W f)))
  (forall  {a : Type u_1} {b : Type u_2} {V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
    {W : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
    (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V)
    (Q : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b W)
    {f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W},
    @DomainTheory.NeighbourhoodSystems.ComputableMap a b V W P Q f  ->
      Domain.Recursive.REPred_2 fun (c d : Nat) =>
        @DomainTheory.NeighbourhoodSystems.ApproximableMap.rel (Set a) (Set b)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain b W)
          (@DomainTheory.NeighbourhoodSystems.PFmap a b V W f)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd a V P c)


          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd b W Q d)))
(forall  {a : Type u_1} {b : Type u_2} {V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
  {W : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
  (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V)
  (Q : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b W)
  (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W) {c d : Nat},
  Iff
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel (Set a) (Set b)
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V)
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain b W)
      (@DomainTheory.NeighbourhoodSystems.PFmap a b V W f)
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd a V P c)
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd b W Q d))
    (forall  (a : Nat),
      @Membership.mem Nat (List Nat) (@List.instMembership Nat) (Domain.Recursive.
decodeList c) a  ->
```

```
        exists  (b : Nat),
          And (@Membership.mem Nat (List Nat) (@List.instMembership Nat) (Domain.
Recursive.decodeList d) b)
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel a b V W f
              (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V P a
)
              (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b W Q b
))))
 (@And.intro
   (And
     (And
       (And
         (forall  {a : Type u_1} {b : Type u_2} {V : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
           {W : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
           (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W),
           @Eq
             (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
               (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Set a) (Set b)
                 (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V)
                 (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 b W))
               (@DomainTheory.NeighbourhoodSystems.funSpace (Set a) (Set b)
                 (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V)
                 (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 b W)))
             (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
               (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W)
               (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Set a) (Set b)
                 (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V)
                 (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 b W))
               (@DomainTheory.NeighbourhoodSystems.funSpace a b V W)
               (@DomainTheory.NeighbourhoodSystems.funSpace (Set a) (Set b)
                 (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V)
                 (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 b W))
               (@DomainTheory.NeighbourhoodSystems.PFmapComb a b V W)
               (@DomainTheory.NeighbourhoodSystems.toFilter a b V W f))
             (@DomainTheory.NeighbourhoodSystems.toFilter (Set a) (Set b)
               (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a
 V)
               (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain b
 W)
               (@DomainTheory.NeighbourhoodSystems.PFmap a b V W f)))
         (forall  {a : Type u_1} {b : Type u_2} {? : Type u_3} {V : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
           {W : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
           {U : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem ?}
           (g : @DomainTheory.NeighbourhoodSystems.ApproximableMap b ? W U)
           (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W),
           @Eq
```

```
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Set a) (Set ?)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a
 V)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain ?
 U))
            (@DomainTheory.NeighbourhoodSystems.PFmap a ? V U
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp a b ? V W U
 g f))
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp (Set a) (Set
b) (Set ?)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a
 V)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain b
 W)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain ?
 U)
              (@DomainTheory.NeighbourhoodSystems.PFmap b ? W U g)
              (@DomainTheory.NeighbourhoodSystems.PFmap a b V W f))))
      (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a),
        @Eq
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Set a) (Set a)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V
)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V
))
          (@DomainTheory.NeighbourhoodSystems.PFmap a a V V
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.idMap a V))
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.idMap (Set a)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V
))))
    (forall  {a : Type u_1} {b : Type u_2} {V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
      {W : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
      (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V)
      (Q : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b W) (consV :
 Nat  ->  Nat)
      (hconsVp : Nat.Primrec consV)
      (hconsVs :
        forall  (a b : Nat),
          Iff (@Eq Nat (consV (Nat.pair a b)) (@OfNat.ofNat Nat (nat_lit 1) (
instOfNatNat (nat_lit 1))))
            (exists  (k : Nat),
              @LE.le (Set a) (@Set.instLE a) (@DomainTheory.NeighbourhoodSystems.
ComputablePresentation.X a V P k)
                (@Inter.inter (Set a) (@Set.instInter a)
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V
 P a)
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V
 P b))))
      (consW : Nat  ->  Nat) (hconsWp : Nat.Primrec consW)
      (hconsWs :
        forall  (a b : Nat),
          Iff (@Eq Nat (consW (Nat.pair a b)) (@OfNat.ofNat Nat (nat_lit 1) (
instOfNatNat (nat_lit 1))))
            (exists  (k : Nat),
```

```
              @LE.le (Set b) (@Set.instLE b) (@DomainTheory.NeighbourhoodSystems.
ComputablePresentation.X b W Q k)
                (@Inter.inter (Set b) (@Set.instInter b)
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b W
 Q a)
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b W
 Q b))))
      {f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W},
      @DomainTheory.NeighbourhoodSystems.ComputableMap a b V W P Q f  ->
        @DomainTheory.NeighbourhoodSystems.ComputableMap (Set a) (Set b)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain b W)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDPresentation a
V P consV hconsVp hconsVs)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDPresentation b
W Q consW hconsWp hconsWs)
          (@DomainTheory.NeighbourhoodSystems.PFmap a b V W f)))
  (forall  {a : Type u_1} {b : Type u_2} {V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
    {W : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
    (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V)
    (Q : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b W)
    {f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W},
    @DomainTheory.NeighbourhoodSystems.ComputableMap a b V W P Q f  ->
      Domain.Recursive.REPred_2 fun (c d : Nat) =>
        @DomainTheory.NeighbourhoodSystems.ApproximableMap.rel (Set a) (Set b)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain b W)
          (@DomainTheory.NeighbourhoodSystems.PFmap a b V W f)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd a V P c)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd b W Q d))
  (@And.intro
    (And
      (And
        (forall  {a : Type u_1} {b : Type u_2} {V : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
          {W : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
          (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W),
          @Eq
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Set a) (Set b)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 b W))
              (@DomainTheory.NeighbourhoodSystems.funSpace (Set a) (Set b)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 b W)))
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W)
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Set a) (Set b)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
```

```
 b W))
              (@DomainTheory.NeighbourhoodSystems.funSpace a b V W)
              (@DomainTheory.NeighbourhoodSystems.funSpace (Set a) (Set b)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 b W))
              (@DomainTheory.NeighbourhoodSystems.PFmapComb a b V W)
              (@DomainTheory.NeighbourhoodSystems.toFilter a b V W f))
            (@DomainTheory.NeighbourhoodSystems.toFilter (Set a) (Set b)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a
 V)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain b
 W)
              (@DomainTheory.NeighbourhoodSystems.PFmap a b V W f)))
        (forall  {a : Type u_1} {b : Type u_2} {? : Type u_3} {V : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
          {W : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
          {U : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem ?}
          (g : @DomainTheory.NeighbourhoodSystems.ApproximableMap b ? W U)
          (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W),
          @Eq
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Set a) (Set ?)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a
 V)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain ?
 U))
            (@DomainTheory.NeighbourhoodSystems.PFmap a ? V U
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp a b ? V W U
 g f))
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp (Set a) (Set
b) (Set ?)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a
 V)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain b
 W)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain ?
 U)
              (@DomainTheory.NeighbourhoodSystems.PFmap b ? W U g)
              (@DomainTheory.NeighbourhoodSystems.PFmap a b V W f))))
      (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a),
        @Eq
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Set a) (Set a)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V
)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V
))
          (@DomainTheory.NeighbourhoodSystems.PFmap a a V V
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.idMap a V))
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.idMap (Set a)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V
))))
    (forall  {a : Type u_1} {b : Type u_2} {V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
      {W : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
```

```
      (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V)
      (Q : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b W) (consV :
 Nat  ->  Nat)
      (hconsVp : Nat.Primrec consV)
      (hconsVs :
        forall  (a b : Nat),
          Iff (@Eq Nat (consV (Nat.pair a b)) (@OfNat.ofNat Nat (nat_lit 1) (
instOfNatNat (nat_lit 1))))
            (exists  (k : Nat),
              @LE.le (Set a) (@Set.instLE a) (@DomainTheory.NeighbourhoodSystems.
ComputablePresentation.X a V P k)
                (@Inter.inter (Set a) (@Set.instInter a)
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V
 P a)
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V
 P b))))
      (consW : Nat  ->  Nat) (hconsWp : Nat.Primrec consW)
      (hconsWs :
        forall  (a b : Nat),
          Iff (@Eq Nat (consW (Nat.pair a b)) (@OfNat.ofNat Nat (nat_lit 1) (
instOfNatNat (nat_lit 1))))
            (exists  (k : Nat),
              @LE.le (Set b) (@Set.instLE b) (@DomainTheory.NeighbourhoodSystems.
ComputablePresentation.X b W Q k)
                (@Inter.inter (Set b) (@Set.instInter b)
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b W
 Q a)
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b W
 Q b))))
      {f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W},
      @DomainTheory.NeighbourhoodSystems.ComputableMap a b V W P Q f  ->
        @DomainTheory.NeighbourhoodSystems.ComputableMap (Set a) (Set b)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain b W)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDPresentation a
V P consV hconsVp hconsVs)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDPresentation b
W Q consW hconsWp hconsWs)
          (@DomainTheory.NeighbourhoodSystems.PFmap a b V W f))
    (@And.intro
      (And
        (forall  {a : Type u_1} {b : Type u_2} {V : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
          {W : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
          (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W),
          @Eq
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Set a) (Set b)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 b W))
              (@DomainTheory.NeighbourhoodSystems.funSpace (Set a) (Set b)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
```

```
 b W)))
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W)
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Set a) (Set b)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 b W))
              (@DomainTheory.NeighbourhoodSystems.funSpace a b V W)
              (@DomainTheory.NeighbourhoodSystems.funSpace (Set a) (Set b)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 b W))
              (@DomainTheory.NeighbourhoodSystems.PFmapComb a b V W)
              (@DomainTheory.NeighbourhoodSystems.toFilter a b V W f))
            (@DomainTheory.NeighbourhoodSystems.toFilter (Set a) (Set b)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a
 V)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain b
 W)
              (@DomainTheory.NeighbourhoodSystems.PFmap a b V W f)))
        (forall  {a : Type u_1} {b : Type u_2} {? : Type u_3} {V : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
          {W : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
          {U : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem ?}
          (g : @DomainTheory.NeighbourhoodSystems.ApproximableMap b ? W U)
          (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W),
          @Eq
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Set a) (Set ?)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a
 V)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain ?
 U))
            (@DomainTheory.NeighbourhoodSystems.PFmap a ? V U
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp a b ? V W U
 g f))
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp (Set a) (Set
b) (Set ?)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a
 V)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain b
 W)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain ?
 U)
              (@DomainTheory.NeighbourhoodSystems.PFmap b ? W U g)
              (@DomainTheory.NeighbourhoodSystems.PFmap a b V W f))))
      (forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a),
        @Eq
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Set a) (Set a)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V
)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V
))
          (@DomainTheory.NeighbourhoodSystems.PFmap a a V V
```

```
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.idMap a V))
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.idMap (Set a)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V
)))
       (@And.intro
         (forall  {a : Type u_1} {b : Type u_2} {V : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
           {W : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
           (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W),
           @Eq
             (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
               (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Set a) (Set b)
                 (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V)
                 (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 b W))
               (@DomainTheory.NeighbourhoodSystems.funSpace (Set a) (Set b)
                 (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V)
                 (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 b W)))
             (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap
               (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W)
               (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Set a) (Set b)
                 (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V)
                 (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 b W))
               (@DomainTheory.NeighbourhoodSystems.funSpace a b V W)
               (@DomainTheory.NeighbourhoodSystems.funSpace (Set a) (Set b)
                 (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V)
                 (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 b W))
               (@DomainTheory.NeighbourhoodSystems.PFmapComb a b V W)
               (@DomainTheory.NeighbourhoodSystems.toFilter a b V W f))
             (@DomainTheory.NeighbourhoodSystems.toFilter (Set a) (Set b)
               (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a
 V)
               (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain b
 W)
               (@DomainTheory.NeighbourhoodSystems.PFmap a b V W f)))
         (forall  {a : Type u_1} {b : Type u_2} {? : Type u_3} {V : DomainTheory.
NeighbourhoodSystems.NeighborhoodSystem a}
           {W : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
           {U : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem ?}
           (g : @DomainTheory.NeighbourhoodSystems.ApproximableMap b ? W U)
           (f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W),
           @Eq
             (@DomainTheory.NeighbourhoodSystems.ApproximableMap (Set a) (Set ?)
               (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a
 V)
               (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain ?
 U))
             (@DomainTheory.NeighbourhoodSystems.PFmap a ? V U
               (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp a b ? V W U
```

```
     g f))
                (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp (Set a) (Set
    b) (Set ?)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a
     V)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain b
     W)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain ?
     U)
                  (@DomainTheory.NeighbourhoodSystems.PFmap b ? W U g)
                  (@DomainTheory.NeighbourhoodSystems.PFmap a b V W f)))
           @DomainTheory.NeighbourhoodSystems.PFmapComb_toElementMap @DomainTheory.
    NeighbourhoodSystems.PFmap_comp)
         @DomainTheory.NeighbourhoodSystems.PFmap_idMap)
       @DomainTheory.NeighbourhoodSystems.PFmap_isComputable)
     @DomainTheory.NeighbourhoodSystems.PFmap_isComputable_Ypd)
    @DomainTheory.NeighbourhoodSystems.PFmap_rel_Ypd_iff)
  @DomainTheory.NeighbourhoodSystems.PFmap_rel_fin
end

end DomainTheory.NeighbourhoodSystems
end
```

3.8.22 Exercise 7.20 — union on power domains

Show that there is a combinator

$$\text{union} : \mathbb{P}(\mathbb{P}D) \to \mathbb{P}D$$

where for suitable neighbourhoods

$$\bigcup_{i<n} \downarrow \Big(\bigcup_{j<m_i} \downarrow X_{ij} \Big) \quad \text{union} \quad \bigcup_{k<q} \downarrow Y_k \quad \text{iff} \quad \forall i < n \forall j < m_i \exists k < q.\, X_{ij} \subseteq Y_k\,.$$

Is union computable if D is effectively given? What is

$$\text{union}(\{\{x\},\{y,z\}\}) = ??$$

Are $\mathbb{P}(\mathbb{P}D)$ and $\mathbb{P}D$ generally isomorphic??

Exercise_7_20.lean

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Definition_7_11
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Proposition_7_10

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Exercise_7_19

@[expose] public section

section
```

```
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem Domain.Recursive ApproximableMap
variable {a : Type*}

/- concordance: source L4675 EXERCISE 7.20 union on power domains -/
/-- **Exercise 7.20 (Scott 1981, PRG-19) -- the flattening combinator `union : ?(?D)
    ->  ?D`.** Given a
`??D`-neighbourhood `A` and a `?D`-neighbourhood `B`, the relation holds iff every `D`-
    neighbourhood
`X` occurring in some member-set `S  in  A` is covered by some `Y  in  B`. This is the
representation-independent form of Scott's displayed `forall  i<n forall  j<m_i exists
     k<q. X_{ij}  subseteq  Y_k`
(see `unionMap_rel_fin`). It is the multiplication `?` of the power-domain monad. -/
def unionMap (V : NeighborhoodSystem a) :
    ApproximableMap V.PowerDomain.PowerDomain V.PowerDomain where
  rel A B := V.PowerDomain.PDmem A  /\  V.PDmem B  /\  forall  S  in  A, forall  X  in
      S, exists  Y  in  B, X  subseteq  Y
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := by
    refine <V.PowerDomain.PDmem_master, V.PDmem_master, ?_>
    intro S hS X hX
    simp only [mem_upSet] at hS
    exact <V.master, <V.master_mem, subset_rfl>, V.sub_master (V.PDmem_mem hS.1 hX)>
  inter_right := by
    rintro A B B' <hA, hB, hbody> <-, hB', hbody'>
    refine <hA, V.PDmem_inter hB hB', ?_>
    intro S hS X hX
    obtain <Y, hYB, hXY> := hbody S hS X hX
    obtain <Y', hY'B', hXY'> := hbody' S hS X hX
    have hXmem : V.mem X := V.PDmem_mem (V.PowerDomain.PDmem_mem hA hS) hX
    have hYmem : V.mem Y := V.PDmem_mem hB hYB
    have hY'mem : V.mem Y' := V.PDmem_mem hB' hY'B'
    have hsub : X  subseteq  Y  I  Y' := Set.subset_inter hXY hXY'
    have hinmem : V.mem (Y  I  Y') := V.inter_mem hYmem hY'mem hXmem hsub
    exact <Y  I  Y', Set.mem_inter
      (V.PDmem_down hB hYB Set.inter_subset_left hinmem)
      (V.PDmem_down hB' hY'B' Set.inter_subset_right hinmem), hsub>
  mono := by
    rintro A A' B B' <_, _, hbody> hA'A hBB' hA' hB'
    refine <hA', hB', ?_>
    intro S hS X hX
    obtain <Y, hYB, hXY> := hbody S (hA'A hS) X hX
    exact <Y, hBB' hYB, hXY>

@[simp] theorem unionMap_rel {V : NeighborhoodSystem a} {A : Set (Set (Set a))} {B :
    Set (Set a)} :
    (unionMap V).rel A B  <->  V.PowerDomain.PDmem A  /\  V.PDmem B  /\  forall  S  in
      A, forall  X  in  S, exists  Y  in  B, X  subseteq  Y :=
  Iff.rfl

/-- **Exercise 7.20 -- the relation of `union` at the level of `???`/`??`-codes.** For
    any
presentation `Q` of `??` whose `c`-th neighbourhood is `Y_c = ?_{a in dl c} X?` (e.g.
    the
```

```
`PDPresentation` of Proposition 7.10), and the canonical enumeration `(???).Ypd Q` of
    `???`,
`(union).rel ((???).Ypd Q n) (??.Ypd P m)  <->  forall  c  in  dl n, forall  a  in  dl
    c, exists  b  in  dl m, X?  subseteq  X_b`. -/
theorem unionMap_rel_Ypd_iff {V : NeighborhoodSystem a} (P : ComputablePresentation V)
    (Q : ComputablePresentation V.PowerDomain) (hQ : forall  c, Q.X c = V.Ypd P c) {n
    m : Nat} :
    (unionMap V).rel (V.PowerDomain.Ypd Q n) (V.Ypd P m)
       <->  forall  c  in  decodeList n, forall  a  in  decodeList c, exists  b  in
    decodeList m, P.X a  subseteq  P.X b := by
  constructor
  * rintro <-, -, hbody> c hc a ha
    have hSmem : Q.X c  in  V.PowerDomain.Ypd Q n :=
      (V.PowerDomain.mem_Ypd Q).mpr <c, hc, Q.mem_X c, subset_rfl>
    have hXinS : P.X a  in  Q.X c := by
      rw [hQ c]; exact (V.mem_Ypd P).mpr <a, ha, P.mem_X a, subset_rfl>
    obtain <Y, hYB, hXY> := hbody (Q.X c) hSmem (P.X a) hXinS
    obtain <b, hb, hYin> := (V.mem_Ypd P).mp hYB
    exact <b, hb, hXY.trans hYin.2>
  * intro hgen
    refine <V.PowerDomain.Ypd_isPDmem Q n, V.Ypd_isPDmem P m, ?_>
    intro S hS X hX
    obtain <c, hc, hScod> := (V.PowerDomain.mem_Ypd Q).mp hS
    have hXinQ : X  in  Q.X c := hScod.2 hX
    rw [hQ c] at hXinQ
    obtain <a, ha, hXin> := (V.mem_Ypd P).mp hXinQ
    obtain <b, hb, hab> := hgen c hc a ha
    exact <P.X b, (V.mem_Ypd P).mpr <b, hb, P.mem_X b, subset_rfl>, hXin.2.trans hab>

/-- The empty power-domain neighbourhood lies under every intersection of
    neighbourhoods. -/
theorem ypd_zero_subset_inter {V : NeighborhoodSystem a} (P : ComputablePresentation V
    ) (a b : Nat) :
    V.Ypd P 0  subseteq  V.Ypd P a  I  V.Ypd P b := by
  rw [V.Ypd_zero P]
  exact Set.empty_subset _

/-- **Exercise 7.20 (Scott's first question) -- `union` is computable.** Given `P :
    Computable
Presentation ?` with a primitive-recursive consistency decider `consV`, `union` is a
    computable map
between the canonical presentations of `???` and `??` (Proposition 7.10). The inner
    consistency
decider of `??` (needed to build the `???` presentation) is the constant decider `fun _
     => 1`,
correct because the empty union (`code 0`, `Y_0 = {}`) lies below every `??`-
    neighbourhood. The
relation is in fact recursively *decidable* (a nested bounded `forall forall exists `
    over `P.incl_computable`). -/
theorem unionMap_isComputable {V : NeighborhoodSystem a} (P : ComputablePresentation V
    )
    (consV : Nat  ->  Nat) (hconsVp : Nat.Primrec consV)
    (hconsVs : forall  a b, consV (Nat.pair a b) = 1  <->  exists  k, P.X k  subseteq
     P.X a  I  P.X b) :
    ComputableMap
      (V.PowerDomain.PDPresentation (V.PDPresentation P consV hconsVp hconsVs)
```

```
        (fun _ => 1) (Nat.Primrec.const 1)
        (fun a b => <fun _ => <0, ypd_zero_subset_inter P a b>, fun _ => rfl>))
      (V.PDPresentation P consV hconsVp hconsVs)
      (unionMap V) := by
  constructor
  have hrd : RecDecidable_2 (fun n m => forall  c  in  decodeList n, forall  a  in
    decodeList c,
      exists  b  in  decodeList m, P.X a  subseteq  P.X b) := (V.subCode_computable P)
    .bForallList
  have key : RecDecidable_2 (fun n m => (unionMap V).rel
      (V.PowerDomain.Ypd (V.PDPresentation P consV hconsVp hconsVs) n) (V.Ypd P m)) :=
     by
    refine RecDecidable.of_iff (fun t => ?_) hrd
    exact unionMap_rel_Ypd_iff P (V.PDPresentation P consV hconsVp hconsVs) (fun _ =>
    rfl)
  exact key.re

/-- **Exercise 7.20 -- Scott's displayed formula.** On generators (nested finite lists
     of down-sets)
`(union).rel (?_{l in LS} _{?D}(?_{X in l} X)) (?_{Y in LY} Y)  <->  forall  l  in  LS,
     forall  X  in  l, exists  Y  in  LY, X  subseteq  Y`,
exactly Scott's `forall  i<n forall  j<m_i exists  k<q. X_{ij}  subseteq  Y_k`. (` -> `
     reads off the generators `X_{ij}`; ` <- `
extends to all members by downward closure at both levels and ` subseteq `-monotonicity
     of the target.) -/
theorem unionMap_rel_fin (V : NeighborhoodSystem a) {LS : List (List (Set a))} {LY :
    List (Set a)}
    (hLS : forall  l  in  LS, forall  X  in  l, V.mem X) (hLY : forall  Y  in  LY, V.
    mem Y) :
    (unionMap V).rel (? l  in  LS, V.PowerDomain.upSet (? X  in  l, V.upSet X)) (? Y
    in  LY, V.upSet Y)
       <->  forall  l  in  LS, forall  X  in  l, exists  Y  in  LY, X  subseteq  Y :=
    by
  constructor
  * rintro <-, -, hbody> l hl X hX
    have hXmem : V.mem X := hLS l hl X hX
    have hSinA : (? X  in  l, V.upSet X)  in  ? l  in  LS, V.PowerDomain.upSet (? X
    in  l, V.upSet X) :=
      Set.mem_iUnion_2.mpr <l, hl, (<l, hLS l hl, rfl> : V.PDmem _), subset_rfl>
    have hXinS : X  in  ? X'  in  l, V.upSet X' := (V.mem_PDunion).mpr <X, hX, hXmem,
    subset_rfl>
    obtain <Y, hYB, hXY> := hbody _ hSinA X hXinS
    obtain <Y', hY', hYin> := (V.mem_PDunion).mp hYB
    exact <Y', hY', hXY.trans hYin.2>
  * intro hgen
    refine <?_, <LY, hLY, rfl>, ?_>
    * refine <LS.map (fun l => ? X  in  l, V.upSet X), ?_, ?_>
      * intro S hS
        obtain <l, hl, rfl> := List.mem_map.mp hS
        exact <l, hLS l hl, rfl>
      * ext z
        simp only [Set.mem_iUnion, List.mem_map, exists_prop]
        constructor
        * rintro <l, hl, hz>
          exact <_, <l, hl, rfl>, hz>
        * rintro <S, <l, hl, rfl>, hz>
```

```
        exact <l, hl, hz>
    * intro S hS X hX
      obtain <l, hl, hSin> := Set.mem_iUnion_2.mp hS
      have hXin : X  in  ? X'  in  l, V.upSet X' := hSin.2 hX
      obtain <X', hX', hXmem> := (V.mem_PDunion).mp hXin
      obtain <Y, hY, hX'Y> := hgen l hl X' hX'
      exact <Y, (V.mem_PDunion).mpr <Y, hY, hLY Y hY, subset_rfl>, hXmem.2.trans hX'Y>

end

section
set_option linter.unusedVariables false

theorem example_720.{u_1} : And
  (And
    (forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
    a}
      (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V) (consV : Nat
      ->  Nat)
      (hconsVp : Nat.Primrec consV)
      (hconsVs :
        forall  (a b : Nat),
          Iff (@Eq Nat (consV (Nat.pair a b)) (@OfNat.ofNat Nat (nat_lit 1) (
    instOfNatNat (nat_lit 1))))
            (exists  (k : Nat),
              @LE.le (Set a) (@Set.instLE a) (@DomainTheory.NeighbourhoodSystems.
    ComputablePresentation.X a V P k)
                (@Inter.inter (Set a) (@Set.instInter a)
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V P a
    )
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V P b
    )))),
      @DomainTheory.NeighbourhoodSystems.ComputableMap (Set (Set a)) (Set a)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain (Set a)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V))
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDPresentation (Set a)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDPresentation a V P
    consV hconsVp hconsVs)
          (fun (x : Nat) => @OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))
          (Nat.Primrec.const (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))
    ) fun (a b : Nat) =>
          @Iff.intro
            (@Eq Nat (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))
              (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))))
            (exists  (k : Nat),
              @LE.le (Set (Set a)) (@Set.instLE (Set a))
                (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X (Set a)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a
     V)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    PDPresentation a V P consV hconsVp hconsVs) k)
                (@Inter.inter (Set (Set a)) (@Set.instInter (Set a))
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X (Set a)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
```

```
 a V)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDPresentation a V P consV hconsVp hconsVs)
                a)
              (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X (Set a)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDPresentation a V P consV hconsVp hconsVs)
                b)))
         (fun
             (x :
               @Eq Nat (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))
                 (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))) =>
           @Exists.intro Nat
             (fun (k : Nat) =>
               @LE.le (Set (Set a)) (@Set.instLE (Set a))
                 (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X (Set
a)
                   (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                   (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDPresentation a V P consV hconsVp hconsVs)
                   k)
                 (@Inter.inter (Set (Set a)) (@Set.instInter (Set a))
                   (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X (
Set a)
                     (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                     (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDPresentation a V P consV hconsVp
                       hconsVs)
                     a)
                   (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X (
Set a)
                     (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                     (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDPresentation a V P consV hconsVp
                       hconsVs)
                     b)))
             (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0)))
             (@DomainTheory.NeighbourhoodSystems.ypd_zero_subset_inter a V P a b))
         fun
           (x :
             exists  (k : Nat),
               @LE.le (Set (Set a)) (@Set.instLE (Set a))
                 (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X (Set
a)
                   (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                   (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDPresentation a V P consV hconsVp hconsVs)
                   k)
                 (@Inter.inter (Set (Set a)) (@Set.instInter (Set a))
                   (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X (
```

```
Set a)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDPresentation a V P consV hconsVp
                      hconsVs)
                    a)
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X (
Set a)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDPresentation a V P consV hconsVp
                      hconsVs)
                    b))) =>
        @rfl Nat (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))))
    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDPresentation a V P
consV hconsVp hconsVs)
    (@DomainTheory.NeighbourhoodSystems.unionMap a V))
(forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
a}
  (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V)
  (Q :
    @DomainTheory.NeighbourhoodSystems.ComputablePresentation (Set a)
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V)),
  (forall  (c : Nat),
      @Eq (Set (Set a))
        (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X (Set a)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V)
Q c)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd a V P c))  ->
    forall  {n m : Nat},
      Iff
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel (Set (Set a)) (Set
 a)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain (Set
a)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V
))
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V)
          (@DomainTheory.NeighbourhoodSystems.unionMap a V)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd (Set a)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V
) Q n)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd a V P m))
        (forall  (c : Nat),
          @Membership.mem Nat (List Nat) (@List.instMembership Nat) (Domain.
Recursive.decodeList n) c  ->
            forall  (a : Nat),
              @Membership.mem Nat (List Nat) (@List.instMembership Nat) (Domain.
Recursive.decodeList c) a  ->
                exists  (b : Nat),
                  And (@Membership.mem Nat (List Nat) (@List.instMembership Nat) (
Domain.Recursive.decodeList m) b)
                    (@LE.le (Set a) (@Set.instLE a)
                      (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X
```

```
   a V P a)
                          (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X
   a V P b)))))
(forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a)
   {LS : List (List (Set a))}
  {LY : List (Set a)},
  (forall  (l : List (Set a)),
      @Membership.mem (List (Set a)) (List (List (Set a))) (@List.instMembership (
  List (Set a))) LS l  ->
       forall  (X : Set a),
         @Membership.mem (Set a) (List (Set a)) (@List.instMembership (Set a)) l X
   ->
           @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X)  ->
    (forall  (Y : Set a),
       @Membership.mem (Set a) (List (Set a)) (@List.instMembership (Set a)) LY Y
  ->
         @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V Y)  ->
      Iff
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel (Set (Set a)) (Set a
  )
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain (Set a)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V))
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V)
          (@DomainTheory.NeighbourhoodSystems.unionMap a V)
          (? (l : List (Set a)),
            ? (_ : @Membership.mem (List (Set a)) (List (List (Set a))) (@List.
  instMembership (List (Set a))) LS l),
              @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.upSet (Set a)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a
   V)
                (? (X : Set a),
                  ? (_ : @Membership.mem (Set a) (List (Set a)) (@List.
  instMembership (Set a)) l X),
                     @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.upSet a V
  X))
          (? (Y : Set a),
            ? (_ : @Membership.mem (Set a) (List (Set a)) (@List.instMembership (Set
   a)) LY Y),
              @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.upSet a V Y))
        (forall  (l : List (Set a)),
          @Membership.mem (List (Set a)) (List (List (Set a))) (@List.instMembership
   (List (Set a))) LS l  ->
            forall  (X : Set a),
              @Membership.mem (Set a) (List (Set a)) (@List.instMembership (Set a))
  l X  ->
                exists  (Y : Set a),
                  And (@Membership.mem (Set a) (List (Set a)) (@List.instMembership
  (Set a)) LY Y)
                    (@LE.le (Set a) (@Set.instLE a) X Y))) :=
@And.intro
(And
  (forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
  a}
    (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V) (consV : Nat
    ->  Nat)
    (hconsVp : Nat.Primrec consV)
```

```
   (hconsVs :
     forall  (a b : Nat),
       Iff (@Eq Nat (consV (Nat.pair a b)) (@OfNat.ofNat Nat (nat_lit 1) (
instOfNatNat (nat_lit 1))))
         (exists  (k : Nat),
           @LE.le (Set a) (@Set.instLE a) (@DomainTheory.NeighbourhoodSystems.
ComputablePresentation.X a V P k)
             (@Inter.inter (Set a) (@Set.instInter a)
               (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V P a
)
               (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V P b
)))),
   @DomainTheory.NeighbourhoodSystems.ComputableMap (Set (Set a)) (Set a)
     (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain (Set a)
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V))
     (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V)
     (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDPresentation (Set a)
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V)
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDPresentation a V P
consV hconsVp hconsVs)
       (fun (x : Nat) => @OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))
       (Nat.Primrec.const (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))
) fun (a b : Nat) =>
       @Iff.intro
         (@Eq Nat (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))
           (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))))
         (exists  (k : Nat),
           @LE.le (Set (Set a)) (@Set.instLE (Set a))
             (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X (Set a)
               (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a
 V)
               (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDPresentation a V P consV hconsVp hconsVs) k)
             (@Inter.inter (Set (Set a)) (@Set.instInter (Set a))
               (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X (Set a)
                 (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V)
                 (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDPresentation a V P consV hconsVp hconsVs)
                 a)
               (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X (Set a)
                 (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V)
                 (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDPresentation a V P consV hconsVp hconsVs)
                 b)))
         (fun
             (x :
               @Eq Nat (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))
                 (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))) =>
           @Exists.intro Nat
             (fun (k : Nat) =>
               @LE.le (Set (Set a)) (@Set.instLE (Set a))
                 (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X (Set
a)
                   (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
```

```
PowerDomain a V)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDPresentation a V P consV hconsVp hconsVs)
                  k)
                (@Inter.inter (Set (Set a)) (@Set.instInter (Set a))
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X (
Set a)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDPresentation a V P consV hconsVp
                      hconsVs)
                    a)
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X (
Set a)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDPresentation a V P consV hconsVp
                      hconsVs)
                    b)))
            (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0)))
            (@DomainTheory.NeighbourhoodSystems.ypd_zero_subset_inter a V P a b))
        fun
          (x :
            exists  (k : Nat),
              @LE.le (Set (Set a)) (@Set.instLE (Set a))
                (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X (Set
a)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDPresentation a V P consV hconsVp hconsVs)
                  k)
                (@Inter.inter (Set (Set a)) (@Set.instInter (Set a))
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X (
Set a)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDPresentation a V P consV hconsVp
                      hconsVs)
                    a)
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X (
Set a)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDPresentation a V P consV hconsVp
                      hconsVs)
                    b))) =>
        @rfl Nat (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))))
    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDPresentation a V P
consV hconsVp hconsVs)
    (@DomainTheory.NeighbourhoodSystems.unionMap a V))
(forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
```

```
 a}
    (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V)
    (Q :
      @DomainTheory.NeighbourhoodSystems.ComputablePresentation (Set a)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V)),
    (forall  (c : Nat),
        @Eq (Set (Set a))
          (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X (Set a)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V)
 Q c)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd a V P c))  ->
      forall  {n m : Nat},
        Iff
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel (Set (Set a)) (Set
  a)


            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain (Set
 a)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V
 ))
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V)
            (@DomainTheory.NeighbourhoodSystems.unionMap a V)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd (Set a)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V
 ) Q n)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd a V P m))
          (forall  (c : Nat),
            @Membership.mem Nat (List Nat) (@List.instMembership Nat) (Domain.
 Recursive.decodeList n) c  ->
              forall  (a : Nat),
                @Membership.mem Nat (List Nat) (@List.instMembership Nat) (Domain.
 Recursive.decodeList c) a  ->
                  exists  (b : Nat),
                    And (@Membership.mem Nat (List Nat) (@List.instMembership Nat) (
 Domain.Recursive.decodeList m) b)
                      (@LE.le (Set a) (@Set.instLE a)
                        (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X
  a V P a)
                        (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X
  a V P b)))))
(forall  {a : Type u_1} (V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a)
   {LS : List (List (Set a))}
  {LY : List (Set a)},
  (forall  (l : List (Set a)),
      @Membership.mem (List (Set a)) (List (List (Set a))) (@List.instMembership (
 List (Set a))) LS l  ->
        forall  (X : Set a),
          @Membership.mem (Set a) (List (Set a)) (@List.instMembership (Set a)) l X
  ->
            @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X)  ->
    (forall  (Y : Set a),
        @Membership.mem (Set a) (List (Set a)) (@List.instMembership (Set a)) LY Y
 ->
          @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V Y)  ->
      Iff
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel (Set (Set a)) (Set a
```

```
)
         (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain (Set a)
           (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V))
         (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V)
         (@DomainTheory.NeighbourhoodSystems.unionMap a V)
         (? (l : List (Set a)),
           ? (_ : @Membership.mem (List (Set a)) (List (List (Set a))) (@List.
 instMembership (List (Set a))) LS l),
             @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.upSet (Set a)
               (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a
  V)
               (? (X : Set a),
                 ? (_ : @Membership.mem (Set a) (List (Set a)) (@List.
 instMembership (Set a)) l X),
                    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.upSet a V
 X))
         (? (Y : Set a),
           ? (_ : @Membership.mem (Set a) (List (Set a)) (@List.instMembership (Set
  a)) LY Y),
             @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.upSet a V Y))
       (forall  (l : List (Set a)),
         @Membership.mem (List (Set a)) (List (List (Set a))) (@List.instMembership
  (List (Set a))) LS l  ->
           forall  (X : Set a),
             @Membership.mem (Set a) (List (Set a)) (@List.instMembership (Set a))
 l X  ->
               exists  (Y : Set a),
                 And (@Membership.mem (Set a) (List (Set a)) (@List.instMembership
 (Set a)) LY Y)
                   (@LE.le (Set a) (@Set.instLE a) X Y)))
(@And.intro
  (forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
 a}
    (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V) (consV : Nat
   ->  Nat)
    (hconsVp : Nat.Primrec consV)
    (hconsVs :
      forall  (a b : Nat),
        Iff (@Eq Nat (consV (Nat.pair a b)) (@OfNat.ofNat Nat (nat_lit 1) (
 instOfNatNat (nat_lit 1))))
         (exists  (k : Nat),
           @LE.le (Set a) (@Set.instLE a) (@DomainTheory.NeighbourhoodSystems.
 ComputablePresentation.X a V P k)
             (@Inter.inter (Set a) (@Set.instInter a)
               (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V P a
 )
               (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V P b
 )))),
    @DomainTheory.NeighbourhoodSystems.ComputableMap (Set (Set a)) (Set a)
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain (Set a)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V))
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V)
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDPresentation (Set a)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDPresentation a V P
 consV hconsVp hconsVs)
```

```
      (fun (x : Nat) => @OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))
      (Nat.Primrec.const (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))
) fun (a b : Nat) =>
      @Iff.intro
        (@Eq Nat (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))
          (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))))
        (exists  (k : Nat),
          @LE.le (Set (Set a)) (@Set.instLE (Set a))
            (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X (Set a)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a
 V)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDPresentation a V P consV hconsVp hconsVs) k)
            (@Inter.inter (Set (Set a)) (@Set.instInter (Set a))
              (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X (Set a)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDPresentation a V P consV hconsVp hconsVs)
                a)
              (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X (Set a)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
 a V)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDPresentation a V P consV hconsVp hconsVs)
                b)))
        (fun
            (x :
              @Eq Nat (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))
                (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))) =>
          @Exists.intro Nat
            (fun (k : Nat) =>
              @LE.le (Set (Set a)) (@Set.instLE (Set a))
                (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X (Set
a)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDPresentation a V P consV hconsVp hconsVs)
                  k)
                (@Inter.inter (Set (Set a)) (@Set.instInter (Set a))
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X (
Set a)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDPresentation a V P consV hconsVp
                      hconsVs)
                    a)
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X (
Set a)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDPresentation a V P consV hconsVp
                      hconsVs)
```

```
                  b)))
            (@OfNat.ofNat Nat (nat_lit 0) (instOfNatNat (nat_lit 0)))
            (@DomainTheory.NeighbourhoodSystems.ypd_zero_subset_inter a V P a b))
        fun
          (x :
            exists  (k : Nat),
              @LE.le (Set (Set a)) (@Set.instLE (Set a))
                (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X (Set
a)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PowerDomain a V)
                  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
PDPresentation a V P consV hconsVp hconsVs)
                  k)
                (@Inter.inter (Set (Set a)) (@Set.instInter (Set a))
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X (
Set a)
                    (@DomainTheory.NeighbourhoodSystem.
PowerDomain a V)
                    (@DomainTheory.NeighbourhoodSystem.
PDPresentation a V P consV hconsVp
                      hconsVs)
                    a)
                  (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X (
Set a)
                    (@DomainTheory.NeighbourhoodSystem.
PowerDomain a V)
                    (@DomainTheory.NeighbourhoodSystem.
PDPresentation a V P consV hconsVp
                      hconsVs)
                    b))) =>
        @rfl Nat (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))))
    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PDPresentation a V P
consV hconsVp hconsVs)
    (@DomainTheory.NeighbourhoodSystems.unionMap a V))
(forall  {a : Type u_1} {V : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
a}
  (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V)
  (Q :
    @DomainTheory.NeighbourhoodSystems.ComputablePresentation (Set a)
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V)),
  (forall  (c : Nat),
      @Eq (Set (Set a))
        (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X (Set a)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V)
Q c)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd a V P c))  ->
    forall  {n m : Nat},
      Iff
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel (Set (Set a)) (Set
 a)
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain (Set
a)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V
))
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V)
```

```
            (@DomainTheory.NeighbourhoodSystems.unionMap a V)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd (Set a)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V
    ) Q n)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd a V P m))
          (forall  (c : Nat),
            @Membership.mem Nat (List Nat) (@List.instMembership Nat) (Domain.
    Recursive.decodeList n) c  ->
              forall  (a : Nat),
                @Membership.mem Nat (List Nat) (@List.instMembership Nat) (Domain.
    Recursive.decodeList c) a  ->
                exists  (b : Nat),
                  And (@Membership.mem Nat (List Nat) (@List.instMembership Nat) (
    Domain.Recursive.decodeList m) b)
                      (@LE.le (Set a) (@Set.instLE a)
                        (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X
     a V P a)
                        (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X
     a V P b))))
    @DomainTheory.NeighbourhoodSystems.unionMap_isComputable @DomainTheory.
    NeighbourhoodSystems.unionMap_rel_Ypd_iff)
  @DomainTheory.NeighbourhoodSystems.unionMap_rel_fin
end

end DomainTheory.NeighbourhoodSystems
end
```

3.8.23 Exercise 7.21 — distributing the power domain

Is there a non-trivial combinator of type

$$\mathbb{P}(D \to E) \to (\mathbb{P}D \to \mathbb{P}E)\,?$$

Are there in general any isomorphisms between the systems

$$(D \to \mathbb{P}E),\ \mathbb{P}(D \times E),\ \mathbb{P}D \times \mathbb{P}E\ ??$$

Is there a non-trivial combinator of type

$$\mathbb{P}(D \times E) \times \mathbb{P}(E \times F) \to \mathbb{P}(D \times F)\ ???$$

Is there any connection between

$$\mathbb{P}\mathbf{N} \text{ and } P\mathbf{N}\ ????$$

Exercise_7_21.lean

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Exercise_7_19
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Definition_7_11
```

```
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Proposition_7_10
public import DomainTheory.NeighbourhoodSystems.DomainEquations.Example_6_1
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_12

public import DomainTheory.NeighbourhoodSystems.DomainEquations.Theorem_6_9

@[expose] public section

section
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem Domain.Recursive ApproximableMap
variable {a b : Type*}

/- concordance: source L4689 EXERCISE 7.21 distributing the power domain -/
/-- **Exercise 7.21 -- the two-variable combinator `papplyEval : ?(D -> E)  x  ?D  ->
    ?E`.** The Smyth
power-domain lift of evaluation (Theorem 3.11): a `?(D -> E)`-neighbourhood `?` applied
     to a
`?D`-neighbourhood `A` covers a `?E`-neighbourhood `B` iff every function-neighbourhood
     `G  in  ?`
applied to every argument-neighbourhood `X  in  A` lands in some `Y  in  B` (via `eval
    `, i.e. every
`f  in  G` sends `X` into `Y`). This is the two-variable analogue of Exercise 7.19's `?
    f`. -/
def papplyEval (V : NeighborhoodSystem a) (W : NeighborhoodSystem b) :
    ApproximableMap_2 (funSpace V W).PowerDomain V.PowerDomain W.PowerDomain where
  rel ? A B := (funSpace V W).PDmem ?  /\  V.PDmem A  /\  W.PDmem B  /\
    forall  G  in  ?, forall  X  in  A, exists  Y  in  B, (eval V W).rel G X Y
  rel_dom_0 h := h.1
  rel_dom_1 h := h.2.1
  rel_cod h := h.2.2.1
  master_rel := by
    refine <(funSpace V W).PDmem_master, V.PDmem_master, W.PDmem_master, ?_>
    intro G hG X hX
    simp only [mem_upSet] at hG hX
    refine <W.master, <W.master_mem, subset_rfl>, ?_>
    exact <hG.1, hX.1, W.master_mem,
      fun f hf => f.mono f.master_rel (V.sub_master hX.1) subset_rfl hX.1 W.master_mem
    >
  inter_right := by
    rintro ? A B B' <h?, hA, hB, hbody> <-, -, hB', hbody'>
    refine <h?, hA, W.PDmem_inter hB hB', ?_>
    intro G hG X hX
    obtain <Y, hYB, hrel> := hbody G hG X hX
    obtain <Y', hY'B', hrel'> := hbody' G hG X hX
    have hinmem : W.mem (Y  I  Y') := (eval V W).rel_cod ((eval V W).inter_right hrel
    hrel')
    exact <Y  I  Y', Set.mem_inter
      (W.PDmem_down hB hYB Set.inter_subset_left hinmem)
      (W.PDmem_down hB' hY'B' Set.inter_subset_right hinmem),
      (eval V W).inter_right hrel hrel'>
  mono := by
    rintro ? ?' A A' B B' <_, _, _, hbody> h?'? hA'A hBB' h?' hA' hB'
    refine <h?', hA', hB', ?_>
```

```
    intro G hG X hX
    obtain <Y, hYB, hrel> := hbody G (h?'? hG) X (hA'A hX)
    exact <Y, hBB' hYB, hrel>

@[simp] theorem papplyEval_rel {V : NeighborhoodSystem a} {W : NeighborhoodSystem b}
    {? : Set (Set (ApproximableMap V W))} {A : Set (Set a)} {B : Set (Set b)} :
    (papplyEval V W).rel ? A B  <->  (funSpace V W).PDmem ?  /\  V.PDmem A  /\  W.
    PDmem B  /\
      forall  G  in  ?, forall  X  in  A, exists  Y  in  B, (eval V W).rel G X Y :=
    Iff.rfl

/-- **Exercise 7.21 -- the combinator as a map out of the product `?(D -> E)  x  ?D  ->
      ?E`.** -/
def papplyB (V : NeighborhoodSystem a) (W : NeighborhoodSystem b) :
    ApproximableMap (prod (funSpace V W).PowerDomain V.PowerDomain) W.PowerDomain :=
  ofMap_2 (papplyEval V W)

/-- **Exercise 7.21 -- the headline combinator `papply : ?(D -> E)  ->  (?D  ->  ?E)
    `.** Currying `papplyB`
(Theorem 3.12) gives the combinator of *exactly* the type Scott asks for. -/
def papply (V : NeighborhoodSystem a) (W : NeighborhoodSystem b) :
    ApproximableMap (funSpace V W).PowerDomain (funSpace V.PowerDomain W.PowerDomain)
    :=
  curry (papplyB V W)

/-- **Exercise 7.21 -- `papply`'s relation at the level of `??`-codes.** Over the
    canonical
presentations of Proposition 7.10 (function-space presentation `Pf`, domain `P`,
    codomain `Q`):
`papplyEval.rel (Y_?c) (Y_Ac) (Y_Bc)  <->  forall  g  in  dl ?c, forall  x  in  dl Ac,
    exists  y  in  dl Bc, eval (Pf.X g)(P.X x)(Q.X y)`. -/
theorem papplyEval_rel_Ypd_iff {V : NeighborhoodSystem a} {W : NeighborhoodSystem b}
    (Pf : ComputablePresentation (funSpace V W)) (P : ComputablePresentation V)
    (Q : ComputablePresentation W) {?c Ac Bc : Nat} :
    (papplyEval V W).rel ((funSpace V W).Ypd Pf ?c) (V.Ypd P Ac) (W.Ypd Q Bc)
       <->  forall  g  in  decodeList ?c, forall  x  in  decodeList Ac, exists  y  in
     decodeList Bc,
          (eval V W).rel (Pf.X g) (P.X x) (Q.X y) := by
  constructor
  * rintro <-, -, -, hbody> g hg x hx
    have hGin : Pf.X g  in  (funSpace V W).Ypd Pf ?c :=
      ((funSpace V W).mem_Ypd Pf).mpr <g, hg, Pf.mem_X g, subset_rfl>
    have hXin : P.X x  in  V.Ypd P Ac := (V.mem_Ypd P).mpr <x, hx, P.mem_X x,
    subset_rfl>
    obtain <Y, hYin, hrel> := hbody (Pf.X g) hGin (P.X x) hXin
    obtain <b, hb, hYmem, hYsub> := (W.mem_Ypd Q).mp hYin
    exact <b, hb, (eval V W).mono hrel subset_rfl subset_rfl hYsub
      ((eval V W).rel_dom_0 hrel) ((eval V W).rel_dom_1 hrel) (Q.mem_X b)>
  * intro hgen
    refine <(funSpace V W).Ypd_isPDmem Pf ?c, V.Ypd_isPDmem P Ac, W.Ypd_isPDmem Q Bc,
    ?_>
    intro G hGin X hXin
    obtain <g, hg, hGmem, hGsub> := ((funSpace V W).mem_Ypd Pf).mp hGin
    obtain <x, hx, hXmem, hXsub> := (V.mem_Ypd P).mp hXin
    obtain <y, hy, hrel> := hgen g hg x hx
    exact <Q.X y, (W.mem_Ypd Q).mpr <y, hy, Q.mem_X y, subset_rfl>,
```

```
      (eval V W).mono hrel hGsub hXsub subset_rfl hGmem hXmem (Q.mem_X y)>

/-- **Exercise 7.21 -- non-triviality.** For any neighbourhoods `X_0  in  D`, `Y_0  in
    E`, the set of
functions sending `X_0` into `Y_0` (the principal `?(D -> E)`-element `[X_0,Y_0]`),
   applied to the
argument-set `X_0`, covers `Y_0`. So `papply` does genuine work (it is not the constant
    map). -/
theorem papplyEval_step_witness {V : NeighborhoodSystem a} {W : NeighborhoodSystem b}
    {X_0 : Set a} {Y_0 : Set b} (hX_0 : V.mem X_0) (hY_0 : W.mem Y_0) :
    (papplyEval V W).rel ((funSpace V W).upSet (step X_0 Y_0)) (V.upSet X_0) (W.upSet
    Y_0) := by
  refine <(funSpace V W).PDmem_upSet (step_mem hX_0 hY_0), V.PDmem_upSet hX_0,
    W.PDmem_upSet hY_0, ?_>
  intro G hG X hX
  obtain <hGmem, hGsub> := hG
  obtain <hXmem, hXsub> := hX
  refine <Y_0, <hY_0, subset_rfl>, hGmem, hXmem, hY_0, ?_>
  intro f hf
  have hfstep : f.rel X_0 Y_0 := mem_step.mp (hGsub hf)
  exact f.mono hfstep hXsub subset_rfl hXmem hY_0

/-- **Three nested bounded quantifiers preserve recursive enumerability.** If `R` is r.
   e. (in the
paired form `R g x y`) then so is `forall  g  in  dl w.1, forall  x  in  dl w.2.1,
   exists  y  in  dl w.2.2, R g x y`. Built from
the bounded-`exists ` lemma `bExists_decodeList_re` (Exercise 7.19) and the
   parameterised bounded-`forall ` lemma
`REPred.forall_mem_decodeList_2`, layered with primitive-recursive re-indexings. -/
theorem re_forallG_forallX_existsY {R : Nat  ->  Nat  ->  Nat  ->  Prop}
    (hR : REPred (fun s => R s.unpair.1 s.unpair.2.unpair.1 s.unpair.2.unpair.2)) :
    REPred (fun w => forall  g  in  decodeList w.unpair.1, forall  x  in  decodeList w
    .unpair.2.unpair.1,
      exists  y  in  decodeList w.unpair.2.unpair.2, R g x y) := by
  -- reindex `<g,x,y> = <s.1.1, s.1.2, s.2>`  |->  `<g,<x,y>>`
  have hm1 : Nat.Primrec (fun s : Nat => Nat.pair s.unpair.1.unpair.1
      (Nat.pair s.unpair.1.unpair.2 s.unpair.2)) :=
    (Nat.Primrec.left.comp Nat.Primrec.left).pair
      ((Nat.Primrec.right.comp Nat.Primrec.left).pair Nat.Primrec.right)
  have hR2 : REPred_2 (fun gx y => R gx.unpair.1 gx.unpair.2 y) := by
    show REPred (fun t => R t.unpair.1.unpair.1 t.unpair.1.unpair.2 t.unpair.2)
    refine REPred.of_iff (fun s => ?_) (hR.comp hm1)
    simp only [unpair_pair_fst, unpair_pair_snd]
  -- bounded `exists  y`
  have hE : REPred (fun u => exists  b  in  decodeList u.unpair.1,
      R u.unpair.2.unpair.1 u.unpair.2.unpair.2 b) := bExists_decodeList_re hR2
  -- bounded `forall  x` (parameters `<Bc, g>`)
  have hm2 : Nat.Primrec (fun t : Nat => Nat.pair t.unpair.1.unpair.1
      (Nat.pair t.unpair.1.unpair.2 t.unpair.2)) := hm1
  have hF : REPred_2 (fun param x => exists  y  in  decodeList param.unpair.1,
      R param.unpair.2 x y) := by
    show REPred (fun t => exists  y  in  decodeList t.unpair.1.unpair.1,
      R t.unpair.1.unpair.2 t.unpair.2 y)
    refine REPred.of_iff (fun t => ?_) (hE.comp hm2)
    simp only [unpair_pair_fst, unpair_pair_snd]
  have hFfull : REPred (fun t => forall  x  in  decodeList t.unpair.2,
```

```
      exists  y  in  decodeList t.unpair.1.unpair.1, R t.unpair.1.unpair.2 x y) :=
    REPred.forall_mem_decodeList_2 hF
  -- bounded `forall  g` (parameters `<Bc, Ac>`)
  have hm3 : Nat.Primrec (fun t : Nat => Nat.pair
      (Nat.pair t.unpair.1.unpair.1 t.unpair.2) t.unpair.1.unpair.2) :=
    ((Nat.Primrec.left.comp Nat.Primrec.left).pair Nat.Primrec.right).pair
      (Nat.Primrec.right.comp Nat.Primrec.left)
  have hG : REPred_2 (fun param g => forall  x  in  decodeList param.unpair.2,
      exists  y  in  decodeList param.unpair.1, R g x y) := by
    show REPred (fun t => forall  x  in  decodeList t.unpair.1.unpair.2,
      exists  y  in  decodeList t.unpair.1.unpair.1, R t.unpair.2 x y)
    refine REPred.of_iff (fun t => ?_) (hFfull.comp hm3)
    simp only [unpair_pair_fst, unpair_pair_snd]
  have hGfull : REPred (fun t => forall  g  in  decodeList t.unpair.2,
      forall  x  in  decodeList t.unpair.1.unpair.2,
        exists  y  in  decodeList t.unpair.1.unpair.1, R g x y) :=
    REPred.forall_mem_decodeList_2 hG
  -- final reindex to `w = <?c, <Ac, Bc>>`
  have hm4 : Nat.Primrec (fun w : Nat => Nat.pair
      (Nat.pair w.unpair.2.unpair.2 w.unpair.2.unpair.1) w.unpair.1) :=
    ((Nat.Primrec.right.comp Nat.Primrec.right).pair
      (Nat.Primrec.left.comp Nat.Primrec.right)).pair Nat.Primrec.left
  refine REPred.of_iff (fun w => ?_) (hGfull.comp hm4)
  simp only [unpair_pair_fst, unpair_pair_snd]

/-- **Exercise 7.21 (the computability half) -- `papply` is computable when `eval` is
    .** Stated at the
level of the `??`-codes of Proposition 7.10: given that `eval` is recursively
    enumerable on codes
(`heval` -- this is Theorem 7.5's `evalMap_isComputable` transported through `
    funPresentation`), the
relation `papplyEval.rel (Y_?c)(Y_Ac)(Y_Bc)` is recursively enumerable in `<?c, Ac, Bc
    >`. -/
theorem papplyEval_isComputable {V : NeighborhoodSystem a} {W : NeighborhoodSystem b}
    (Pf : ComputablePresentation (funSpace V W)) (P : ComputablePresentation V)
    (Q : ComputablePresentation W)
    (heval : REPred (fun s => (eval V W).rel (Pf.X s.unpair.1)
      (P.X s.unpair.2.unpair.1) (Q.X s.unpair.2.unpair.2))) :
    REPred (fun w => (papplyEval V W).rel ((funSpace V W).Ypd Pf w.unpair.1)
      (V.Ypd P w.unpair.2.unpair.1) (W.Ypd Q w.unpair.2.unpair.2)) := by
  refine REPred.of_iff (fun w => ?_)
    (re_forallG_forallX_existsY
      (R := fun g x y => (eval V W).rel (Pf.X g) (P.X x) (Q.X y)) heval)
  exact papplyEval_rel_Ypd_iff Pf P Q

end

section
set_option linter.unusedVariables false

theorem example_721.{u_1, u_2} : And
  (And
    (forall  {a : Type u_1} {b : Type u_2} {V : DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem a}
      {W : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
      (Pf :
```

```
    @DomainTheory.NeighbourhoodSystems.ComputablePresentation
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W)
      (@DomainTheory.NeighbourhoodSystems.funSpace a b V W))
  (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V)
  (Q : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b W),
  (Domain.Recursive.REPred fun (s : Nat) =>
      @DomainTheory.NeighbourhoodSystems.ApproximableMap_2.rel
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W) a b
        (@DomainTheory.NeighbourhoodSystems.funSpace a b V W) V W (@DomainTheory.
NeighbourhoodSystems.eval a b V W)
        (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W)
          (@DomainTheory.NeighbourhoodSystems.funSpace a b V W) Pf (@Prod.fst Nat
Nat (Nat.unpair s)))
        (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V P
          (@Prod.fst Nat Nat (Nat.unpair (@Prod.snd Nat Nat (Nat.unpair s)))))
        (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b W Q
          (@Prod.snd Nat Nat (Nat.unpair (@Prod.snd Nat Nat (Nat.unpair s))))))
->
    Domain.Recursive.REPred fun (w : Nat) =>
      @DomainTheory.NeighbourhoodSystems.ApproximableMap_2.rel
        (Set (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W)) (Set a)
 (Set b)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W)
          (@DomainTheory.NeighbourhoodSystems.funSpace a b V W))
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain b W)
        (@DomainTheory.NeighbourhoodSystems.papplyEval a b V W)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W)
          (@DomainTheory.NeighbourhoodSystems.funSpace a b V W) Pf (@Prod.fst Nat
Nat (Nat.unpair w)))
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd a V P
          (@Prod.fst Nat Nat (Nat.unpair (@Prod.snd Nat Nat (Nat.unpair w)))))
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd b W Q
          (@Prod.snd Nat Nat (Nat.unpair (@Prod.snd Nat Nat (Nat.unpair w))))))
 (forall  {a : Type u_1} {b : Type u_2} {V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
  {W : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
  (Pf :
    @DomainTheory.NeighbourhoodSystems.ComputablePresentation
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W)
      (@DomainTheory.NeighbourhoodSystems.funSpace a b V W))
  (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V)
  (Q : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b W) {?c Ac Bc :
Nat},
  Iff
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap_2.rel
      (Set (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W)) (Set a) (
Set b)
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W)
        (@DomainTheory.NeighbourhoodSystems.funSpace a b V W))
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V)
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain b W)
```

```
        (@DomainTheory.NeighbourhoodSystems.papplyEval a b V W)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W)
          (@DomainTheory.NeighbourhoodSystems.funSpace a b V W) Pf ?c)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd a V P Ac)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd b W Q Bc))
      (forall  (g : Nat),
        @Membership.mem Nat (List Nat) (@List.instMembership Nat) (Domain.Recursive.
  decodeList ?c) g  ->
          forall  (x : Nat),
            @Membership.mem Nat (List Nat) (@List.instMembership Nat) (Domain.
  Recursive.decodeList Ac) x  ->
              exists  (y : Nat),
                And (@Membership.mem Nat (List Nat) (@List.instMembership Nat) (
  Domain.Recursive.decodeList Bc) y)
                  (@DomainTheory.NeighbourhoodSystems.ApproximableMap_2.rel
                    (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W) a b
                    (@DomainTheory.NeighbourhoodSystems.funSpace a b V W) V W
                    (@DomainTheory.NeighbourhoodSystems.eval a b V W)
                    (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X
                      (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W)
                      (@DomainTheory.NeighbourhoodSystems.funSpace a b V W) Pf g)
                    (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V
   P x)
                    (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b W
   Q y)))))
(forall  {a : Type u_1} {b : Type u_2} {V : DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem a}
  {W : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b} {X_0 : Set a} {Y_0 :
  Set b},
  @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X_0  ->
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b W Y_0  ->
      @DomainTheory.NeighbourhoodSystems.ApproximableMap_2.rel
        (Set (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W)) (Set a) (
  Set b)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W)
          (@DomainTheory.NeighbourhoodSystems.funSpace a b V W))
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain b W)
        (@DomainTheory.NeighbourhoodSystems.papplyEval a b V W)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.upSet
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W)
          (@DomainTheory.NeighbourhoodSystems.funSpace a b V W)
          (@DomainTheory.NeighbourhoodSystems.step a b V W X_0 Y_0))
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.upSet a V X_0)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.upSet b W Y_0)) :=
@And.intro
(And
  (forall  {a : Type u_1} {b : Type u_2} {V : DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem a}
    {W : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
    (Pf :
      @DomainTheory.NeighbourhoodSystems.ComputablePresentation
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W)
        (@DomainTheory.NeighbourhoodSystems.funSpace a b V W))
```

```
   (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V)
   (Q : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b W),
   (Domain.Recursive.REPred fun (s : Nat) =>
       @DomainTheory.NeighbourhoodSystems.ApproximableMap_2.rel
         (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W) a b
         (@DomainTheory.NeighbourhoodSystems.funSpace a b V W) V W (@DomainTheory.
NeighbourhoodSystems.eval a b V W)
         (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X
           (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W)
           (@DomainTheory.NeighbourhoodSystems.funSpace a b V W) Pf (@Prod.fst Nat
Nat (Nat.unpair s)))
         (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V P
           (@Prod.fst Nat Nat (Nat.unpair (@Prod.snd Nat Nat (Nat.unpair s)))))
         (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b W Q
           (@Prod.snd Nat Nat (Nat.unpair (@Prod.snd Nat Nat (Nat.unpair s))))))
->
    Domain.Recursive.REPred fun (w : Nat) =>
      @DomainTheory.NeighbourhoodSystems.ApproximableMap_2.rel
        (Set (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W)) (Set a)
 (Set b)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W)
          (@DomainTheory.NeighbourhoodSystems.funSpace a b V W))
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain b W)
        (@DomainTheory.NeighbourhoodSystems.papplyEval a b V W)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W)
          (@DomainTheory.NeighbourhoodSystems.funSpace a b V W) Pf (@Prod.fst Nat
Nat (Nat.unpair w)))
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd a V P
          (@Prod.fst Nat Nat (Nat.unpair (@Prod.snd Nat Nat (Nat.unpair w)))))
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd b W Q
          (@Prod.snd Nat Nat (Nat.unpair (@Prod.snd Nat Nat (Nat.unpair w))))))
 (forall  {a : Type u_1} {b : Type u_2} {V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
   {W : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
   (Pf :
     @DomainTheory.NeighbourhoodSystems.ComputablePresentation
       (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W)
       (@DomainTheory.NeighbourhoodSystems.funSpace a b V W))
   (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V)
   (Q : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b W) {?c Ac Bc :
Nat},
   Iff
     (@DomainTheory.NeighbourhoodSystems.ApproximableMap_2.rel
       (Set (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W)) (Set a) (
Set b)
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
         (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W)
         (@DomainTheory.NeighbourhoodSystems.funSpace a b V W))
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V)
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain b W)
       (@DomainTheory.NeighbourhoodSystems.papplyEval a b V W)
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd
         (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W)
```

```
          (@DomainTheory.NeighbourhoodSystems.funSpace a b V W) Pf ?c)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd a V P Ac)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd b W Q Bc))
      (forall  (g : Nat),
        @Membership.mem Nat (List Nat) (@List.instMembership Nat) (Domain.Recursive.
  decodeList ?c) g  ->
          forall  (x : Nat),
            @Membership.mem Nat (List Nat) (@List.instMembership Nat) (Domain.
  Recursive.decodeList Ac) x  ->
              exists  (y : Nat),
                And (@Membership.mem Nat (List Nat) (@List.instMembership Nat) (
  Domain.Recursive.decodeList Bc) y)
                  (@DomainTheory.NeighbourhoodSystems.ApproximableMap_2.rel
                    (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W) a b
                    (@DomainTheory.NeighbourhoodSystems.funSpace a b V W) V W
                    (@DomainTheory.NeighbourhoodSystems.eval a b V W)
                    (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X
                      (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W)
                      (@DomainTheory.NeighbourhoodSystems.funSpace a b V W) Pf g)
                    (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V
   P x)
                    (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b W
   Q y)))))
(forall  {a : Type u_1} {b : Type u_2} {V : DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem a}
  {W : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b} {X_0 : Set a} {Y_0 :
  Set b},
  @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a V X_0  ->
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem b W Y_0  ->
      @DomainTheory.NeighbourhoodSystems.ApproximableMap_2.rel
        (Set (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W)) (Set a) (
  Set b)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W)
          (@DomainTheory.NeighbourhoodSystems.funSpace a b V W))
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain b W)
        (@DomainTheory.NeighbourhoodSystems.papplyEval a b V W)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.upSet
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W)
          (@DomainTheory.NeighbourhoodSystems.funSpace a b V W)
          (@DomainTheory.NeighbourhoodSystems.step a b V W X_0 Y_0))
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.upSet a V X_0)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.upSet b W Y_0))
(@And.intro
  (forall  {a : Type u_1} {b : Type u_2} {V : DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem a}
    {W : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
    (Pf :
      @DomainTheory.NeighbourhoodSystems.ComputablePresentation
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W)
        (@DomainTheory.NeighbourhoodSystems.funSpace a b V W))
    (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V)
    (Q : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b W),
    (Domain.Recursive.REPred fun (s : Nat) =>
        @DomainTheory.NeighbourhoodSystems.ApproximableMap_2.rel
```

```
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W) a b
        (@DomainTheory.NeighbourhoodSystems.funSpace a b V W) V W (@DomainTheory.
NeighbourhoodSystems.eval a b V W)
        (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W)
          (@DomainTheory.NeighbourhoodSystems.funSpace a b V W) Pf (@Prod.fst Nat
Nat (Nat.unpair s)))
        (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V P
          (@Prod.fst Nat Nat (Nat.unpair (@Prod.snd Nat Nat (Nat.unpair s)))))
        (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b W Q
          (@Prod.snd Nat Nat (Nat.unpair (@Prod.snd Nat Nat (Nat.unpair s))))))
->
    Domain.Recursive.REPred fun (w : Nat) =>
      @DomainTheory.NeighbourhoodSystems.ApproximableMap_2.rel
        (Set (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W)) (Set a)
 (Set b)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W)


          (@DomainTheory.NeighbourhoodSystems.funSpace a b V W))
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain b W)
        (@DomainTheory.NeighbourhoodSystems.papplyEval a b V W)
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W)
          (@DomainTheory.NeighbourhoodSystems.funSpace a b V W) Pf (@Prod.fst Nat
Nat (Nat.unpair w)))
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd a V P
          (@Prod.fst Nat Nat (Nat.unpair (@Prod.snd Nat Nat (Nat.unpair w)))))
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd b W Q
          (@Prod.snd Nat Nat (Nat.unpair (@Prod.snd Nat Nat (Nat.unpair w))))))
 (forall  {a : Type u_1} {b : Type u_2} {V : DomainTheory.NeighbourhoodSystems.
NeighborhoodSystem a}
   {W : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem b}
   (Pf :
     @DomainTheory.NeighbourhoodSystems.ComputablePresentation
       (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W)
       (@DomainTheory.NeighbourhoodSystems.funSpace a b V W))
   (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a V)
   (Q : @DomainTheory.NeighbourhoodSystems.ComputablePresentation b W) {?c Ac Bc :
Nat},
   Iff
     (@DomainTheory.NeighbourhoodSystems.ApproximableMap_2.rel
       (Set (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W)) (Set a) (
Set b)
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain
         (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W)
         (@DomainTheory.NeighbourhoodSystems.funSpace a b V W))
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain a V)
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.PowerDomain b W)
       (@DomainTheory.NeighbourhoodSystems.papplyEval a b V W)
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd
         (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W)
         (@DomainTheory.NeighbourhoodSystems.funSpace a b V W) Pf ?c)
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd a V P Ac)
       (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Ypd b W Q Bc))
```

```
        (forall  (g : Nat),
          @Membership.mem Nat (List Nat) (@List.instMembership Nat) (Domain.Recursive.
    decodeList ?c) g  ->
            forall  (x : Nat),
              @Membership.mem Nat (List Nat) (@List.instMembership Nat) (Domain.
    Recursive.decodeList Ac) x  ->
                exists  (y : Nat),
                  And (@Membership.mem Nat (List Nat) (@List.instMembership Nat) (
    Domain.Recursive.decodeList Bc) y)
                    (@DomainTheory.NeighbourhoodSystems.ApproximableMap_2.rel
                      (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W) a b
                      (@DomainTheory.NeighbourhoodSystems.funSpace a b V W) V W
                      (@DomainTheory.NeighbourhoodSystems.eval a b V W)
                      (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X
                        (@DomainTheory.NeighbourhoodSystems.ApproximableMap a b V W)
                        (@DomainTheory.NeighbourhoodSystems.funSpace a b V W) Pf g)
                      (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a V
     P x)
                      (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X b W
     Q y))))
    @DomainTheory.NeighbourhoodSystems.papplyEval_isComputable
    @DomainTheory.NeighbourhoodSystems.papplyEval_rel_Ypd_iff)
  @DomainTheory.NeighbourhoodSystems.papplyEval_step_witness
end

section
open NeighborhoodSystem Domain.Recursive Example61
variable {a : Type*}

namespace Proposition77

variable {D : NeighborhoodSystem a} (P : ComputablePresentation D)

/-- Consistency bit of a packed triple. -/
def consB (r : Nat) : Nat := r.unpair.2.unpair.1

/-- Equality bit of a packed triple. -/
def eqB (r : Nat) : Nat := r.unpair.1

/-- Intersection index of a packed triple. -/
def intI (r : Nat) : Nat := r.unpair.2.unpair.2

end Proposition77
end

section
open NeighborhoodSystem Domain.Recursive Example61
variable {a : Type*}

namespace Proposition77

/-- `a  <=  Nat.pair a b` (choice-free). -/
theorem le_pair_left (a b : Nat) : a  <=  Nat.pair a b := by
  unfold Nat.pair; split <;> omega

end Proposition77
```

```
end

section
open NeighborhoodSystem Domain.Recursive Example61
variable {a : Type*}

namespace Proposition77

variable {D : NeighborhoodSystem a} (P : ComputablePresentation D)

/-- One step of the list-indexing fold: state `pair countdown value`; capture the
    current element
when `countdown = 1`, else carry the value and decrement. -/
def listGetStp (t : Nat) : Nat :=
  Nat.pair (t.unpair.2.unpair.1.unpair.1 - 1)
    (selectFn (isOne t.unpair.2.unpair.1.unpair.1) t.unpair.1 t.unpair.2.unpair.1.
    unpair.2)

/-- `listGet c i = (decodeList c).getD i 0`: the `i`-th entry of the list coded by `c`.
     -/
def listGet (c i : Nat) : Nat :=
  (foldCode listGetStp 0 (Nat.pair (i + 1) 0) c).unpair.2

/-- The fold step on an explicit accumulator `pair cd val`. -/
theorem listGetStp_pair (cd val x : Nat) :
    listGetStp (Nat.pair x (Nat.pair (Nat.pair cd val) 0))
      = Nat.pair (cd - 1) (selectFn (isOne cd) x val) := by
  unfold listGetStp
  simp only [unpair_pair_fst, unpair_pair_snd]

/-- With `cd = 0` the fold is a fixed point on the value. -/
theorem listGet_foldl_zero (l : List Nat) (val : Nat) :
    (List.foldl (fun acc x => listGetStp (Nat.pair x (Nat.pair acc 0))) (Nat.pair 0
    val) l).unpair.2
      = val := by
  induction l generalizing val with
  | nil => simp
  | cons x xs ih =>
      rw [List.foldl_cons, listGetStp_pair, Nat.zero_sub,
        show isOne 0 = 0 from rfl, selectFn_zero, ih]

/-- The fold reads off the `cd-1`-th element (default `val`). -/
theorem listGet_foldl (l : List Nat) (c val : Nat) :
    (List.foldl (fun acc x => listGetStp (Nat.pair x (Nat.pair acc 0)))
        (Nat.pair (c + 1) val) l).unpair.2 = l.getD c val := by
  induction l generalizing c val with
  | nil => simp
  | cons x xs ih =>
      rw [List.foldl_cons, listGetStp_pair, Nat.add_sub_cancel]
      rcases Nat.eq_zero_or_pos c with rfl | hc
      * rw [show isOne (0 + 1) = 1 from rfl, selectFn_one, listGet_foldl_zero,
          List.getD_cons_zero]
      * obtain <c', rfl> : exists  c', c = c' + 1 := <c - 1, by omega>
        rw [show isOne (c' + 1 + 1) = 0 by unfold isOne; omega, selectFn_zero, ih,
          List.getD_cons_succ]
```

```
/-- **Correctness of `listGet`** on a coded list. -/
theorem listGet_encodeList (l : List Nat) (i : Nat) : listGet (encodeList l) i = l.
    getD i 0 := by
  unfold listGet
  rw [foldCode_eq]
  exact listGet_foldl l i 0

end Proposition77
end

section
open NeighborhoodSystem Domain.Recursive Example61
variable {a : Type*}

namespace Proposition77

theorem max_sq_le_pair (i j : Nat) : max i j * max i j  <=  Nat.pair i j := by
  unfold Nat.pair
  rcases lt_or_ge i j with h | h
  * rw [ite_eq_left h, show max i j = j by omega]; omega
  * rw [ite_eq_right (by omega), show max i j = i by omega]; omega

end Proposition77
end

section
open NeighborhoodSystem Domain.Recursive Example61
variable {a : Type*}

namespace Proposition77

variable {D : NeighborhoodSystem a} (P : ComputablePresentation D)

theorem memS_sub_embPair (hD : forall  X, D.mem X  ->  X.Nonempty)
    {W A B : Set (List Bool  x  a)} (hW : MemS D W) (hsub : W  subseteq  embPair A B)
    :
    exists  Pp Qq, MemS D Pp  /\  MemS D Qq  /\  W = embPair Pp Qq := by
  cases hW with
  | gamma =>
      obtain <a, ha> := hD D.master D.master_mem
      rcases hsub (show (([], a) : List Bool  x  a)  in  Gamma D from ha) with <p', hp
    ', -> | <q', hq', ->
      * simp at hp'
      * simp at hq'
  | @zero Z hZ =>
      obtain <a, ha> := hD Z hZ
      rcases hsub (show (([], a) : List Bool  x  a)  in  embZero Z from <rfl, ha>)
    with
        <p', hp', -> | <q', hq', ->
      * simp at hp'
      * simp at hq'
  | @pair Pp Qq hP hQ => exact <Pp, Qq, hP, hQ, rfl>

theorem memS_sub_embZero (hD : forall  X, D.mem X  ->  X.Nonempty) {W : Set (List Bool
      x  a)} {Y : Set a}
    (hW : MemS D W) (hsub : W  subseteq  embZero Y) : exists  Z, D.mem Z  /\  W =
```

```
    embZero Z := by
  cases hW with
  | gamma =>
      obtain <a, ha> := hD D.master D.master_mem
      exact absurd (hsub (show ((true :: [], a) : List Bool  x  a)  in  Gamma D from
    ha)).1 (by simp)
  | @zero Z hZ => exact <Z, hZ, rfl>
  | @pair Pp Qq hP hQ =>
      obtain <<pth, a>, ht> := embPair_nonempty (memS_nonempty hD hP) (Q := Qq)
      have hmem := hsub ht
      rcases ht with <p', hp', -> | <q', hq', ->
      * exact absurd (hp'.symm.trans hmem.1) (by simp)
      * exact absurd (hq'.symm.trans hmem.1) (by simp)

end Proposition77
end

section
open NeighborhoodSystem Domain.Recursive Example61
variable {a : Type*}

namespace Proposition77

/-- The three index shapes: `0` (master `Gamma`), `2a+1` (leaf), `2a+2` (node). Choice-
    free. -/
theorem nat_shape (n : Nat) : n = 0  \/  (exists  a, n = 2 * a + 1)  \/  (exists  a, n
     = 2 * a + 2) := by
  rcases Nat.eq_zero_or_pos n with rfl | hn
  * exact Or.inl rfl
  * rcases Nat.lt_or_ge (n % 2) 1 with h | h
    * exact Or.inr (Or.inr <n / 2 - 1, by omega>)
    * exact Or.inr (Or.inl <n / 2, by omega>)

end Proposition77
end

section
open NeighborhoodSystem Domain.Recursive Example61
variable {a : Type*}

namespace Proposition77

variable {D : NeighborhoodSystem a} (P : ComputablePresentation D)

/-- Pack the triple `(eqBit, consBit, interIdx)` into one natural. -/
def packT (e c ii : Nat) : Nat := Nat.pair e (Nat.pair c ii)

@[simp] theorem eqB_packT (e c ii : Nat) : eqB (packT e c ii) = e := by
  simp only [eqB, packT, unpair_pair_fst]

@[simp] theorem consB_packT (e c ii : Nat) : consB (packT e c ii) = c := by
  simp only [consB, packT, unpair_pair_fst, unpair_pair_snd]

@[simp] theorem intI_packT (e c ii : Nat) : intI (packT e c ii) = ii := by
  simp only [intI, packT, unpair_pair_snd]
```

```
end Proposition77
end

section
open NeighborhoodSystem Domain.Recursive Example61
variable {a : Type*}

namespace Proposition77

variable {D : NeighborhoodSystem a} (P : ComputablePresentation D)

variable (fcons feq finter : Nat  ->  Nat)

/-- **The `D^S` decider step.** `p = pair (pair i j) tbl`. Returns `packT eqBit consBit
     interIdx`,
case-splitting on `i = 0` / `j = 0` (the `Gamma` identity), then on parities (leaf `2a
    +1` vs node
`2a+2`). Leaf I leaf delegates to `D`'s deciders; node I node ANDs/combines the two
    children read from
the memo table `tbl`. -/
def dsharpStep (p : Nat) : Nat :=
  let i := p.unpair.1.unpair.1
  let j := p.unpair.1.unpair.2
  let tbl := p.unpair.2
  let w := p.unpair.1
  let na := i / 2 - 1
  let nb := j / 2 - 1
  let r1 := listGet tbl (w - 1 - Nat.pair na.unpair.1 nb.unpair.1)
  let r2 := listGet tbl (w - 1 - Nat.pair na.unpair.2 nb.unpair.2)
  selectFn (1 - i) (packT (1 - j) 1 j)
    (selectFn (1 - j) (packT (1 - i) 1 i)
      (selectFn (i % 2)
        (selectFn (j % 2)
          (packT (feq (Nat.pair (i / 2) (j / 2))) (fcons (Nat.pair (i / 2) (j / 2)))
            (2 * finter (Nat.pair (i / 2) (j / 2)) + 1))
          (packT 0 0 0))
        (selectFn (j % 2) (packT 0 0 0)
          (packT (eqB r1 * eqB r2) (consB r1 * consB r2)
            (2 * Nat.pair (intI r1) (intI r2) + 2)))))

theorem dsharpStep_i0 (j tbl : Nat) :
    dsharpStep fcons feq finter (Nat.pair (Nat.pair 0 j) tbl) = packT (1 - j) 1 j :=
    by
  simp only [dsharpStep, unpair_pair_fst, unpair_pair_snd, Nat.sub_zero, selectFn_one]

theorem dsharpStep_j0 {i : Nat} (tbl : Nat) (hi : i  !=  0) :
    dsharpStep fcons feq finter (Nat.pair (Nat.pair i 0) tbl) = packT (1 - i) 1 i :=
    by
  simp only [dsharpStep, unpair_pair_fst, unpair_pair_snd]
  rw [show (1 : Nat) - i = 0 by omega, selectFn_zero, Nat.sub_zero, selectFn_one]

theorem dsharpStep_ll (a b tbl : Nat) :
    dsharpStep fcons feq finter (Nat.pair (Nat.pair (2 * a + 1) (2 * b + 1)) tbl)
      = packT (feq (Nat.pair a b)) (fcons (Nat.pair a b)) (2 * finter (Nat.pair a b) +
     1) := by
  simp only [dsharpStep, unpair_pair_fst, unpair_pair_snd]
```

```
  rw [show (1 : Nat) - (2 * a + 1) = 0 by omega, selectFn_zero,
    show (1 : Nat) - (2 * b + 1) = 0 by omega, selectFn_zero,
    show (2 * a + 1) % 2 = 1 by omega, selectFn_one,
    show (2 * b + 1) % 2 = 1 by omega, selectFn_one,
    show (2 * a + 1) / 2 = a by omega, show (2 * b + 1) / 2 = b by omega]

theorem dsharpStep_ln (a b tbl : Nat) :
    dsharpStep fcons feq finter (Nat.pair (Nat.pair (2 * a + 1) (2 * b + 2)) tbl) =
    packT 0 0 0 := by
  simp only [dsharpStep, unpair_pair_fst, unpair_pair_snd]
  rw [show (1 : Nat) - (2 * a + 1) = 0 by omega, selectFn_zero,
    show (1 : Nat) - (2 * b + 2) = 0 by omega, selectFn_zero,
    show (2 * a + 1) % 2 = 1 by omega, selectFn_one,
    show (2 * b + 2) % 2 = 0 by omega, selectFn_zero]

theorem dsharpStep_nl (a b tbl : Nat) :
    dsharpStep fcons feq finter (Nat.pair (Nat.pair (2 * a + 2) (2 * b + 1)) tbl) =
    packT 0 0 0 := by
  simp only [dsharpStep, unpair_pair_fst, unpair_pair_snd]
  rw [show (1 : Nat) - (2 * a + 2) = 0 by omega, selectFn_zero,
    show (1 : Nat) - (2 * b + 1) = 0 by omega, selectFn_zero,
    show (2 * a + 2) % 2 = 0 by omega, selectFn_zero,
    show (2 * b + 1) % 2 = 1 by omega, selectFn_one]

theorem dsharpStep_nn (a b tbl : Nat) :
    dsharpStep fcons feq finter (Nat.pair (Nat.pair (2 * a + 2) (2 * b + 2)) tbl)
      = packT
          (eqB (listGet tbl (Nat.pair (2 * a + 2) (2 * b + 2) - 1 - Nat.pair a.unpair
    .1 b.unpair.1))
           * eqB (listGet tbl (Nat.pair (2 * a + 2) (2 * b + 2) - 1 - Nat.pair a.
    unpair.2 b.unpair.2)))
          (consB (listGet tbl (Nat.pair (2 * a + 2) (2 * b + 2) - 1 - Nat.pair a.
    unpair.1 b.unpair.1))
           * consB (listGet tbl (Nat.pair (2 * a + 2) (2 * b + 2) - 1 - Nat.pair a.
    unpair.2 b.unpair.2)))
          (2 * Nat.pair
            (intI (listGet tbl (Nat.pair (2 * a + 2) (2 * b + 2) - 1 - Nat.pair a.
    unpair.1 b.unpair.1)))
            (intI (listGet tbl (Nat.pair (2 * a + 2) (2 * b + 2) - 1 - Nat.pair a.
    unpair.2 b.unpair.2)))
            + 2) := by
  simp only [dsharpStep, unpair_pair_fst, unpair_pair_snd]
  rw [show (1 : Nat) - (2 * a + 2) = 0 by omega, selectFn_zero,
    show (1 : Nat) - (2 * b + 2) = 0 by omega, selectFn_zero,
    show (2 * a + 2) % 2 = 0 by omega, selectFn_zero,
    show (2 * b + 2) % 2 = 0 by omega, selectFn_zero,
    show (2 * a + 2) / 2 - 1 = a by omega, show (2 * b + 2) / 2 - 1 = b by omega]

end Proposition77
end

section
open NeighborhoodSystem Domain.Recursive Example61
variable {a : Type*}

namespace Proposition77
```

```
theorem pair_lt_succ_sq (x y : Nat) : Nat.pair x y < (max x y + 1) * (max x y + 1) :=
    by
  have hm : forall  z : Nat, (z + 1) * (z + 1) = z * z + 2 * z + 1 := fun z => by
    rw [Nat.succ_mul, Nat.mul_succ]; omega
  unfold Nat.pair
  rcases lt_or_ge x y with h | h
  * rw [ite_eq_left h, show max x y = y by omega, hm]; omega
  * rw [ite_eq_right (by omega), show max x y = x by omega, hm]; omega

theorem pair_lt_pair_of_lt {x y i j : Nat} (hx : x < i) (hy : y < j) :
    Nat.pair x y < Nat.pair i j := by
  have h1 : Nat.pair x y < (max x y + 1) * (max x y + 1) := pair_lt_succ_sq x y
  have h2 : max i j * max i j  <=  Nat.pair i j := max_sq_le_pair i j
  have h3 : max x y + 1  <=  max i j := by
    rcases le_total x y with h | h
    * rw [max_eq_right h]; have := le_max_right i j; omega
    * rw [max_eq_left h]; have := le_max_left i j; omega
  have h4 : (max x y + 1) * (max x y + 1)  <=  max i j * max i j := Nat.mul_le_mul h3
    h3
  omega

end Proposition77
end

section
open NeighborhoodSystem Domain.Recursive Example61
variable {a : Type*}

namespace Proposition77

variable {D : NeighborhoodSystem a} (P : ComputablePresentation D)

/-- `listGet` is primitive recursive (on the `Nat.pair` coding of `c, i`). -/
theorem primrec_listGet : Nat.Primrec (fun p => listGet p.unpair.1 p.unpair.2) := by
  have hstp : Nat.Primrec listGetStp := by
    unfold listGetStp
    have hcd : Nat.Primrec (fun t => t.unpair.2.unpair.1.unpair.1) :=
      Nat.Primrec.left.comp (Nat.Primrec.left.comp Nat.Primrec.right)
    have hval : Nat.Primrec (fun t => t.unpair.2.unpair.1.unpair.2) :=
      Nat.Primrec.right.comp (Nat.Primrec.left.comp Nat.Primrec.right)
    have hx : Nat.Primrec (fun t : Nat => t.unpair.1) := Nat.Primrec.left
    exact (primrec_sub_2 hcd (Nat.Primrec.const 1)).pair
      (primrec_selectFn (primrec_isOne.comp hcd) hx hval)
  have hz : Nat.Primrec (fun p : Nat => Nat.pair (p.unpair.2 + 1) 0) :=
    (Nat.Primrec.succ.comp Nat.Primrec.right).pair (Nat.Primrec.const 0)
  exact (Nat.Primrec.right.comp
    (primrec_foldCode hstp (Nat.Primrec.const 0) hz Nat.Primrec.left)).of_eq fun p =>
    rfl

/-- `listGet` as a binary primrec composite. -/
theorem primrec_listGet_2 {c i : Nat  ->  Nat} (hc : Nat.Primrec c) (hi : Nat.Primrec
    i) :
    Nat.Primrec (fun p => listGet (c p) (i p)) :=
  (primrec_listGet.comp (hc.pair hi)).of_eq fun p => by
    simp only [unpair_pair_fst, unpair_pair_snd]
```

```
end Proposition77
end

section
open NeighborhoodSystem Domain.Recursive Example61
variable {a : Type*}

namespace Proposition77

variable {D : NeighborhoodSystem a} (P : ComputablePresentation D)


variable (fcons feq finter : Nat  ->  Nat)

/-- The decider step is primitive recursive when the `D`-level functions are. -/
theorem primrec_dsharpStep (hfcons : Nat.Primrec fcons) (hfeq : Nat.Primrec feq)
    (hfinter : Nat.Primrec finter) : Nat.Primrec (dsharpStep fcons feq finter) := by
  have hi : Nat.Primrec (fun p : Nat => p.unpair.1.unpair.1) := Nat.Primrec.left.comp
    Nat.Primrec.left
  have hj : Nat.Primrec (fun p : Nat => p.unpair.1.unpair.2) := Nat.Primrec.right.comp
     Nat.Primrec.left
  have htbl : Nat.Primrec (fun p : Nat => p.unpair.2) := Nat.Primrec.right
  have hw : Nat.Primrec (fun p : Nat => p.unpair.1) := Nat.Primrec.left
  have hidiv : Nat.Primrec (fun p : Nat => p.unpair.1.unpair.1 / 2) := primrec_div2.
    comp hi
  have hjdiv : Nat.Primrec (fun p : Nat => p.unpair.1.unpair.2 / 2) := primrec_div2.
    comp hj
  have hna : Nat.Primrec (fun p : Nat => p.unpair.1.unpair.1 / 2 - 1) :=
    primrec_sub_2 hidiv (Nat.Primrec.const 1)
  have hnb : Nat.Primrec (fun p : Nat => p.unpair.1.unpair.2 / 2 - 1) :=
    primrec_sub_2 hjdiv (Nat.Primrec.const 1)
  have hv1 : Nat.Primrec (fun p : Nat =>
      Nat.pair (p.unpair.1.unpair.1 / 2 - 1).unpair.1 (p.unpair.1.unpair.2 / 2 - 1).
    unpair.1) :=
     (Nat.Primrec.left.comp hna).pair (Nat.Primrec.left.comp hnb)
  have hv2 : Nat.Primrec (fun p : Nat =>
      Nat.pair (p.unpair.1.unpair.1 / 2 - 1).unpair.2 (p.unpair.1.unpair.2 / 2 - 1).
    unpair.2) :=
     (Nat.Primrec.right.comp hna).pair (Nat.Primrec.right.comp hnb)
  have hr1 : Nat.Primrec (fun p : Nat => listGet p.unpair.2
      (p.unpair.1 - 1 - Nat.pair (p.unpair.1.unpair.1 / 2 - 1).unpair.1
        (p.unpair.1.unpair.2 / 2 - 1).unpair.1)) :=
    primrec_listGet_2 htbl (primrec_sub_2 (primrec_sub_2 hw (Nat.Primrec.const 1)) hv1
    )
  have hr2 : Nat.Primrec (fun p : Nat => listGet p.unpair.2
      (p.unpair.1 - 1 - Nat.pair (p.unpair.1.unpair.1 / 2 - 1).unpair.2
        (p.unpair.1.unpair.2 / 2 - 1).unpair.2)) :=
    primrec_listGet_2 htbl (primrec_sub_2 (primrec_sub_2 hw (Nat.Primrec.const 1)) hv2
    )
  have hidiv2 : Nat.Primrec (fun p : Nat => Nat.pair (p.unpair.1.unpair.1 / 2) (p.
    unpair.1.unpair.2 / 2)) :=
    hidiv.pair hjdiv
  have hpackT : forall  {e c ii : Nat  ->  Nat}, Nat.Primrec e  ->  Nat.Primrec c  ->
     Nat.Primrec ii  ->
      Nat.Primrec (fun p => packT (e p) (c p) (ii p)) := by
```

```
    intro e c ii he hc hii
    exact (he.pair (hc.pair hii)).of_eq fun p => rfl
  have hleaf : Nat.Primrec (fun p : Nat =>
      packT (feq (Nat.pair (p.unpair.1.unpair.1 / 2) (p.unpair.1.unpair.2 / 2)))
        (fcons (Nat.pair (p.unpair.1.unpair.1 / 2) (p.unpair.1.unpair.2 / 2)))
        (2 * finter (Nat.pair (p.unpair.1.unpair.1 / 2) (p.unpair.1.unpair.2 / 2)) +
    1)) :=
    hpackT (hfeq.comp hidiv2) (hfcons.comp hidiv2)
      (Nat.Primrec.succ.comp (primrec_mul_2 (Nat.Primrec.const 2) (hfinter.comp hidiv2
    )))
  have hzero : Nat.Primrec (fun _ : Nat => packT 0 0 0) := Nat.Primrec.const _
  have heqB1 : Nat.Primrec (fun p => eqB _) := Nat.Primrec.left.comp hr1
  have heqB2 : Nat.Primrec (fun p => eqB _) := Nat.Primrec.left.comp hr2
  have hconsB1 : Nat.Primrec (fun p => consB _) := (Nat.Primrec.left.comp Nat.Primrec.
    right).comp hr1
  have hconsB2 : Nat.Primrec (fun p => consB _) := (Nat.Primrec.left.comp Nat.Primrec.
    right).comp hr2
  have hintI1 : Nat.Primrec (fun p => intI _) := (Nat.Primrec.right.comp Nat.Primrec.
    right).comp hr1
  have hintI2 : Nat.Primrec (fun p => intI _) := (Nat.Primrec.right.comp Nat.Primrec.
    right).comp hr2
  have hnode : Nat.Primrec (fun p : Nat =>
      packT (eqB (listGet p.unpair.2 (p.unpair.1 - 1 - Nat.pair (p.unpair.1.unpair.1 /
     2 - 1).unpair.1
                (p.unpair.1.unpair.2 / 2 - 1).unpair.1))
            * eqB (listGet p.unpair.2 (p.unpair.1 - 1 - Nat.pair (p.unpair.1.unpair.1
    / 2 - 1).unpair.2
                (p.unpair.1.unpair.2 / 2 - 1).unpair.2)))
        (consB (listGet p.unpair.2 (p.unpair.1 - 1 - Nat.pair (p.unpair.1.unpair.1 / 2
     - 1).unpair.1
                (p.unpair.1.unpair.2 / 2 - 1).unpair.1))
            * consB (listGet p.unpair.2 (p.unpair.1 - 1 - Nat.pair (p.unpair.1.unpair
    .1 / 2 - 1).unpair.2
                (p.unpair.1.unpair.2 / 2 - 1).unpair.2)))
        (2 * Nat.pair (intI (listGet p.unpair.2 (p.unpair.1 - 1 - Nat.pair (p.unpair
    .1.unpair.1 / 2 - 1).unpair.1
                (p.unpair.1.unpair.2 / 2 - 1).unpair.1)))
              (intI (listGet p.unpair.2 (p.unpair.1 - 1 - Nat.pair (p.unpair.1.unpair
    .1 / 2 - 1).unpair.2
                (p.unpair.1.unpair.2 / 2 - 1).unpair.2))) + 2)) :=
    hpackT (primrec_mul_2 heqB1 heqB2) (primrec_mul_2 hconsB1 hconsB2)
      ((Nat.Primrec.succ.comp Nat.Primrec.succ).comp (primrec_mul_2 (Nat.Primrec.const
     2)
        (hintI1.pair hintI2)))
  refine (primrec_selectFn (primrec_sub_2 (Nat.Primrec.const 1) hi)
    (hpackT (primrec_sub_2 (Nat.Primrec.const 1) hj) (Nat.Primrec.const 1) hj)
    (primrec_selectFn (primrec_sub_2 (Nat.Primrec.const 1) hj)
      (hpackT (primrec_sub_2 (Nat.Primrec.const 1) hi) (Nat.Primrec.const 1) hi)
      (primrec_selectFn (primrec_mod2.comp hi)
        (primrec_selectFn (primrec_mod2.comp hj) hleaf hzero)
        (primrec_selectFn (primrec_mod2.comp hj) hzero hnode)))).of_eq fun p => rfl

end Proposition77
end

section
```

```
open NeighborhoodSystem Domain.Recursive Example61
variable {a : Type*}

namespace Proposition77

variable {D : NeighborhoodSystem a} (P : ComputablePresentation D)

/-- The reverse memo table: `rtbl step w` codes `[g(w-1), ..., g 0]`. -/
def rtbl (step : Nat → Nat) : Nat → Nat
  | 0 => 0
  | (w + 1) => Nat.pair (step (Nat.pair w (rtbl step w))) (rtbl step w) + 1

/-- The course-of-values value at `w`: `g w = step (pair w (table of g below w))`. -/
def gOf (step : Nat → Nat) (w : Nat) : Nat := step (Nat.pair w (rtbl step w))

/-- The reverse list of memo values `[g(w-1), ..., g 0]`. -/
def gList (step : Nat → Nat) : Nat → List Nat
  | 0 => []
  | (w + 1) => gOf step w :: gList step w

theorem gOf_def (step : Nat → Nat) (w : Nat) : gOf step w = step (Nat.pair w (rtbl
    step w)) := rfl

theorem getD_gList (step : Nat → Nat) : forall (w p : Nat), p < w →
    (gList step w).getD p 0 = gOf step (w - 1 - p) := by
  intro w
  induction w with
  | zero => intro p hp; omega
  | succ w ih =>
      intro p hp
      cases p with
      | zero =>
          rw [gList, List.getD_cons_zero]
          have : w + 1 - 1 - 0 = w := by omega
          rw [this]
      | succ p =>
          rw [gList, List.getD_cons_succ, ih p (by omega)]
          have : w + 1 - 1 - (p + 1) = w - 1 - p := by omega
          rw [this]

theorem rtbl_eq_encode (step : Nat → Nat) (w : Nat) : rtbl step w = encodeList (
    gList step w) := by
  induction w with
  | zero => rfl
  | succ w ih =>
      show Nat.pair (step (Nat.pair w (rtbl step w))) (rtbl step w) + 1
        = encodeList (gList step (w + 1))
      rw [gList, encodeList, gOf, ih]

/-- **The course-of-values lookup.** Inside `step` at `w`, position `w-1-v` of the
    reverse table is
exactly `g v`, for any earlier `v < w`. -/
theorem listGet_rtbl (step : Nat → Nat) {v w : Nat} (hv : v < w) :
    listGet (rtbl step w) (w - 1 - v) = gOf step v := by
  rw [rtbl_eq_encode, listGet_encodeList, getD_gList step w (w - 1 - v) (by omega)]
  congr 1; omega
```

```
theorem rtbl_eq_rec (step : Nat  ->  Nat) (w : Nat) :
    rtbl step w
      = Nat.rec (motive := fun _ => Nat) 0
          (fun y IH => Nat.pair (step (Nat.pair y IH)) IH + 1) w := by
  induction w with
  | zero => rfl
  | succ w ih => rw [rtbl, ih]

/-- `rtbl step` is primitive recursive when `step` is (a single `prec` whose step is a
    cons). -/
theorem primrec_rtbl {step : Nat  ->  Nat} (hstep : Nat.Primrec step) : Nat.Primrec (
    rtbl step) := by
  have hg : Nat.Primrec (fun c => Nat.pair (step c.unpair.2) c.unpair.2.unpair.2 + 1)
    :=
    Nat.Primrec.succ.comp ((hstep.comp Nat.Primrec.right).pair
      (Nat.Primrec.right.comp Nat.Primrec.right))
  refine ((Nat.Primrec.prec Nat.Primrec.zero hg).comp
    (Nat.Primrec.zero.pair primrec_id)).of_eq fun w => ?_
  simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd, id_eq]
  rw [rtbl_eq_rec]

/-- `gOf step` is primitive recursive when `step` is. -/
theorem primrec_gOf {step : Nat  ->  Nat} (hstep : Nat.Primrec step) : Nat.Primrec (
    gOf step) :=
  (hstep.comp (primrec_id.pair (primrec_rtbl hstep))).of_eq fun _ => rfl

end Proposition77
end

section
open NeighborhoodSystem Domain.Recursive Example61
variable {a : Type*}

namespace Proposition77

/-- `n.unpair.1  <=  n` (choice-free); the decreasing measure for `Vsharp`'s left child
    . -/
theorem unpair_fst_le (n : Nat) : n.unpair.1  <=  n := by
  have h := le_pair_left n.unpair.1 n.unpair.2
  rwa [pair_unpair] at h

end Proposition77
end
end DomainTheory.NeighbourhoodSystems
end
```

3.8.24 Exercise 7.22 — domain of formal languages

(For algebraists.) Let $\Sigma = \{0,1\}^*$ be the free semigroup. A new domain is constructed by defining a family of sets by the least fixed point theorem as follows:

$$S = \{\Sigma\} \cup \{\{\sigma\} \mid \sigma \in \Sigma\} \cup \{XY \mid X, Y \in S\} \cup \{X \cap Y \mid X, Y \in S \text{ and } X \cap Y \neq \emptyset\}.$$

Here we write: $XY = \{\sigma\tau \mid \sigma \in X \text{ and } \tau \in Y\}$.

Prove that S is an effectively given, positive neighbourhood system. (Hint: The sets in S are each "regular events" in the terminology of automata theory, and we have a decision method for the set algebra of regular events.)

Define multiplication on $|S|$ by

$$xy = \{Z \in S \mid \exists X \in x\, \exists Y \in y.\, XY \subseteq Z\},$$

and show $|S|$ becomes a semigroup with Σ embedded into $|S|$ by the homomorphism $\sigma \mapsto \{X \in S \mid \sigma \in X\}$.

Investigate some *infinite words* in S, say those defined by least fixed points such as: $\vec{\sigma} = \sigma\vec{\sigma}$ and $\vec{\sigma} = \vec{\sigma}\sigma$. Are these equations true:

$$\vec{\sigma}\vec{\sigma} = \vec{\sigma}, \quad \vec{\sigma}\vec{\sigma}\vec{\sigma} = \vec{\sigma}, \quad \vec{\sigma}\vec{1}\vec{\sigma}\vec{1} = \vec{\sigma}\vec{1},$$

and $0\vec{1}\,0\vec{1}\,0\vec{1}\,0\vec{1} = 0\vec{1}\,0\vec{1}$?

Exercise_7_22.lean

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Theorem_4_2
public import Mathlib.Computability.EpsilonNFA
public import Mathlib.Data.Fintype.Option

public import Mathlib.Logic.Equiv.List
public import Mathlib.Data.Fintype.Prod

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
namespace Exercise722
open NeighborhoodSystem ApproximableMap

section
/-- Scott's `XY = {sigmatau ? sigma  in  X and tau  in  Y}`: the concatenation of two
    languages. -/
def concat (X Y : Set (List Bool)) : Set (List Bool) := {w | exists  a  in  X, exists
     b  in  Y, a ++ b = w}
end

section
/-- **Scott's family `S`**, as the least fixed point (an inductive predicate). A
    language `X` is *in
`S`* iff it is built from the four generators:

* `Sigma = {0,1}*` itself (`Set.univ`);
* a singleton `{sigma}`;
* a concatenation `XY` of two members;
* a *non-empty* intersection `X  I  Y` of two members. -/
inductive InS : Set (List Bool)  ->  Prop
  | univ : InS Set.univ
```

```
  | singleton (sigma : List Bool) : InS {sigma}
  | mul {X Y : Set (List Bool)} : InS X  ->  InS Y  ->  InS (concat X Y)
  | inter {X Y : Set (List Bool)} : InS X  ->  InS Y  ->  (X  I  Y).Nonempty  ->  InS
    (X  I  Y)
end

section
@[simp] theorem mem_concat {X Y : Set (List Bool)} {w : List Bool} :
    w  in  concat X Y  <->  exists  a  in  X, exists  b  in  Y, a ++ b = w := Iff.rfl
end

section
/-- `Sigma` is a neighbourhood of `S`. -/
theorem Ssys_master_mem : InS Set.univ := InS.univ
end

section
/-- The token set `Sigma = {0,1}*` is non-empty. -/
theorem Ssys_master_nonempty : (Set.univ : Set (List Bool)).Nonempty := by
  exact <[], Set.mem_univ []>
end

section
/-- Every member of `S` is a subset of `Sigma`. -/
theorem Ssys_sub_master {X : Set (List Bool)} (_h : InS X) : X  subseteq  Set.univ :=
  Set.subset_univ _
end

section
/-- Concatenation is monotone in both arguments. -/
theorem concat_mono {X X' Y Y' : Set (List Bool)} (hX : X  subseteq  X') (hY : Y
    subseteq  Y') :
    concat X Y  subseteq  concat X' Y' := by
  rintro w <a, ha, b, hb, rfl>; exact <a, hX ha, b, hY hb, rfl>
end

section
/-- The concatenation of two non-empty languages is non-empty. -/
theorem concat_nonempty {X Y : Set (List Bool)} (hX : X.Nonempty) (hY : Y.Nonempty) :
    (concat X Y).Nonempty := by
  obtain <a, ha> := hX
  obtain <b, hb> := hY
  exact <a ++ b, a, ha, b, hb, rfl>
end

section
/-- **Every member of `S` is non-empty.** (`Sigma` and singletons are non-empty;
    concatenation preserves
non-emptiness; intersections are only admitted to `S` when non-empty.) This is what
    makes `S`
*positive*. -/
theorem InS.nonempty {X : Set (List Bool)} (h : InS X) : X.Nonempty := by
  induction h with
  | univ => exact <[], trivial>
  | singleton sigma => exact <sigma, rfl>
  | mul _ _ ihX ihY => exact concat_nonempty ihX ihY
```

```
  | inter _ _ hne _ _ => exact hne
end

section
/-- Intersection of two members of `S` is again in `S` when a consistency witness
    exists. -/
theorem Ssys_inter_mem {X Y Z : Set (List Bool)}
    (hX : InS X) (hY : InS Y) (hZ : InS Z) (hZsub : Z  subseteq  X  I  Y) : InS (X  I
     Y) :=
  InS.inter hX hY (hZ.nonempty.mono hZsub)
end

end Exercise722
end DomainTheory.NeighbourhoodSystems

section
namespace DomainTheory.NeighbourhoodSystems
namespace Exercise722
open NeighborhoodSystem ApproximableMap
section
/- concordance: source L4707 EXERCISE 7.22 domain of formal languages -/
/-- **Exercise 7.22 (neighbourhood-system part).** `S` is a *positive* neighbourhood
    system over the
token type `Sigma = {0,1}*`, with master neighbourhood `Delta = Sigma = Set.univ`.
    Proof fields are named so
Palomar can lock this value. -/
def Ssys : NeighborhoodSystem (List Bool) where
  mem := InS
  master := Set.univ
  master_nonempty := Ssys_master_nonempty
  master_mem := Ssys_master_mem
  inter_mem := Ssys_inter_mem
  sub_master := Ssys_sub_master
end

section
@[simp] theorem Ssys_mem {X : Set (List Bool)} : Ssys.mem X  <->  InS X := Iff.rfl
end

section
/-- **Scott's multiplication on `|S|`.** `xy = {Z  in  S ? exists  X  in  x exists  Y
    in  y. XY  subseteq  Z}`. The filter
conditions:

* `master_mem`: take `X = Y = Sigma` (both in any filter), `Sigma*Sigma  subseteq
    Sigma`;
* `inter_mem`: from witnesses `X_1Y_1  subseteq  Z_1`, `X_2Y_2  subseteq  Z_2`, the
    pair `X_1  I  X_2  in  x`, `Y_1  I  Y_2  in  y` (filter
  closure) gives `(X_1  I  X_2)(Y_1  I  Y_2)  subseteq  Z_1  I  Z_2` by monotonicity of
     `concat`, and `Z_1  I  Z_2  in  S`
  because this non-empty witness sits inside it (positivity);
* `up_mem`: transitivity of ` subseteq `. -/
def mulElem (x y : Ssys.Element) : Ssys.Element where
  mem Z := InS Z  /\  exists  X, x.mem X  /\  exists  Y, y.mem Y  /\  concat X Y
    subseteq  Z
  sub h := h.1
```

```
  master_mem :=
    <InS.univ, Set.univ, x.master_mem, Set.univ, y.master_mem, Set.subset_univ _>
  inter_mem := by
    rintro Z1 Z2 <hZ1, X1, hX1, Y1, hY1, hsub1> <hZ2, X2, hX2, Y2, hY2, hsub2>
    have hXi : x.mem (X1  I  X2) := x.inter_mem hX1 hX2
    have hYi : y.mem (Y1  I  Y2) := y.inter_mem hY1 hY2
    have hcsub : concat (X1  I  X2) (Y1  I  Y2)  subseteq  Z1  I  Z2 := by
      intro w hw
      exact <hsub1 (concat_mono Set.inter_subset_left Set.inter_subset_left hw),
             hsub2 (concat_mono Set.inter_subset_right Set.inter_subset_right hw)>
    have hne : (Z1  I  Z2).Nonempty :=
      (concat_nonempty (x.sub hXi).nonempty (y.sub hYi).nonempty).mono hcsub
    exact <InS.inter hZ1 hZ2 hne, X1  I  X2, hXi, Y1  I  Y2, hYi, hcsub>
  up_mem := by
    rintro Z W <_, X, hX, Y, hY, hsub> hW hZW
    exact <hW, X, hX, Y, hY, hsub.trans hZW>
end

section
@[simp] theorem mem_mulElem {x y : Ssys.Element} {Z : Set (List Bool)} :
    (mulElem x y).mem Z  <->  InS Z  /\  exists  X, x.mem X  /\  exists  Y, y.mem Y
    /\  concat X Y  subseteq  Z := Iff.rfl
end

section
/-- **Scott's embedding** `sigma  |->  {X  in  S ? sigma  in  X}`. This is a filter (an
     element of `|S|`): it contains
`Sigma`, is closed under intersection (the intersection is non-empty since it still
    contains `sigma`, so it
lies in `S` by positivity), and is upward closed. -/
def emb (sigma : List Bool) : Ssys.Element where
  mem X := InS X  /\  sigma  in  X
  sub h := h.1
  master_mem := <InS.univ, Set.mem_univ sigma>
  inter_mem := by
    rintro X Y <hX, hsigmaX> <hY, hsigmaY>
    exact <InS.inter hX hY <sigma, hsigmaX, hsigmaY>, hsigmaX, hsigmaY>
  up_mem := by
    rintro X Y <_, hsigma> hY hsub
    exact <hY, hsub hsigma>
end

section
@[simp] theorem mem_emb {sigma : List Bool} {X : Set (List Bool)} :
    (emb sigma).mem X  <->  InS X  /\  sigma  in  X := Iff.rfl
end

section
/-- **`x  |->  sigma*x`, realised as an approximable self-map on `|S|`.** Mirrors `
    Example44.lean`'s
`consMap`, generalised from a single bit `b` to an arbitrary word `sigma`. -/
def prependMap (sigma : List Bool) : ApproximableMap Ssys Ssys where
  rel Y Z := InS Y  /\  InS Z  /\  concat {sigma} Y  subseteq  Z
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := <InS.univ, InS.univ, Set.subset_univ _>
```

```
  inter_right := by
    rintro Y Z Z' <hY, hZ, hsub> <_, hZ', hsub'>
    have hcsub : concat {sigma} Y  subseteq  Z  I  Z' := Set.subset_inter hsub hsub'
    have hne : (Z  I  Z').Nonempty :=
      (concat_nonempty (Set.singleton_nonempty sigma) hY.nonempty).mono hcsub
    exact <hY, InS.inter hZ hZ' hne, hcsub>
  mono := by
    rintro Y Y' Z Z' <_, hZ, hsub> hYY' hZZ' hY' hZ'
    exact <hY', hZ', (concat_mono (Set.Subset.refl _) hYY').trans (hsub.trans hZZ')>
end

section
/-- `w?`: `w` concatenated with itself `n` times (`w? = []`). -/
def repeatWord (w : List Bool) : Nat  ->  List Bool
  | 0 => []
  | n + 1 => w ++ repeatWord w n
end

section
@[simp] theorem repeatWord_zero (w : List Bool) : repeatWord w 0 = [] := rfl
end

section
/-- `{w? ? n}` -- the language of all finite powers of `w`. -/
def powerLang (w : List Bool) : Set (List Bool) :=
  {u | exists  n, repeatWord w n = u}
end

section
@[simp] theorem mem_powerLang {w u : List Bool} :
    u  in  powerLang w  <->  exists  n, repeatWord w n = u := Iff.rfl
end

section
/-- **Scott's `sigma-vec`, as a genuine least fixed point** (Theorem 4.1) of `x  |->
    sigma*x` in `|S|`. -/
def streamArrow (sigma : List Bool) : Ssys.Element := (prependMap sigma).fixElement
end

section
/-- Membership in Scott's **`w-vec`**: every power `w?` lies in `Z`. Concatenation-
    closure of `Z`
  is not required for membership, but holds for neighbourhoods in `mulElem (streamElem
    w) (streamElem w)`. -/
def streamElemMem (w : List Bool) (Z : Set (List Bool)) : Prop :=
  InS Z  /\  forall  n, repeatWord w n  in  Z
end

section
/-- Scott's **`w-vec`** as a domain element. -/
def streamElem (w : List Bool) : Ssys.Element where
  mem Z := streamElemMem w Z
  sub h := h.1
  master_mem := <InS.univ, fun n => Set.mem_univ (repeatWord w n)>
  inter_mem := by
    intro X Y hX hY
```

```
    obtain <hXIn, hXw> := hX
    obtain <hYIn, hYw> := hY
    exact <InS.inter hXIn hYIn <repeatWord w 0, hXw 0, hYw 0>, fun n => <hXw n, hYw n
    >>
  up_mem := by
    intro X Y hX hInSY hsub
    obtain <_hXIn, hXw> := hX
    exact <hInSY, fun n => hsub (hXw n)>
end

section
/-- `a  in  X`, `b  in  Y ? a ++ b  in  XY`. -/
theorem append_mem_concat {X Y : Set (List Bool)} {a b : List Bool} (ha : a  in  X) (
    hb : b  in  Y) :
    a ++ b  in  concat X Y := <a, ha, b, hb, rfl>

/-- Concatenation is associative (inherited from `List.append_assoc`). -/
theorem concat_assoc (X Y Z : Set (List Bool)) :
    concat (concat X Y) Z = concat X (concat Y Z) := by
  ext w
  constructor
  * rintro <ab, <a, ha, b, hb, rfl>, c, hc, rfl>
    exact <a, ha, b ++ c, <b, hb, c, hc, rfl>, by rw [List.append_assoc]>
  * rintro <a, ha, bc, <b, hb, c, hc, rfl>, rfl>
    exact <a ++ b, <a, ha, b, hb, rfl>, c, hc, by rw [List.append_assoc]>

/-- **Exercise 7.22 (semigroup part): multiplication on `|S|` is associative**, so `|S
    |` is a
semigroup. The forward inclusion rewrites `X(YZ) = (XY)Z` (`concat_assoc`) and uses
    monotonicity of
`concat` to push the witnesses through; the converse is symmetric. -/
theorem mulElem_assoc (x y z : Ssys.Element) :
    mulElem (mulElem x y) z = mulElem x (mulElem y z) := by
  apply NeighborhoodSystem.Element.ext
  intro W
  constructor
  * rintro <hW, P, <_, X, hX, Y, hY, hXY>, Z, hZ, hPZ>
    refine <hW, X, hX, concat Y Z, <InS.mul (y.sub hY) (z.sub hZ), Y, hY, Z, hZ,
      Set.Subset.refl _>, ?_>
    rw [ <-  concat_assoc]
    exact (concat_mono hXY (Set.Subset.refl _)).trans hPZ
  * rintro <hW, X, hX, Q, <_, Y, hY, Z, hZ, hYZ>, hXQ>
    refine <hW, concat X Y, <InS.mul (x.sub hX) (y.sub hY), X, hX, Y, hY,
      Set.Subset.refl _>, Z, hZ, ?_>
    rw [concat_assoc]
    exact (concat_mono (Set.Subset.refl _) hYZ).trans hXQ

theorem repeatWord_add (w : List Bool) (a b : Nat) :
    repeatWord w (a + b) = repeatWord w a ++ repeatWord w b := by
  induction a with
  | zero => simp [repeatWord]
  | succ a ih =>
      simp only [Nat.succ_add, repeatWord]
      rw [ih, List.append_assoc]

theorem repeatWord_eq_empty (w : List Bool) (n : Nat) (hw : w = []) :
```

```
    repeatWord w n = [] := by
  subst hw
  induction n with
  | zero => rfl
  | succ n ih => simp [repeatWord, ih]

theorem InS_powerLang_empty : InS (powerLang []) := by
  have h : powerLang [] = {[]} := by
    ext u
    simp only [mem_powerLang, Set.mem_singleton_iff]
    constructor
    * rintro <n, hn>; exact hn.symm.trans (repeatWord_eq_empty [] n rfl)
    * intro hu; subst hu; exact <0, rfl>
  simpa [h] using InS.singleton []

@[simp] theorem mem_streamElem {w : List Bool} {Z : Set (List Bool)} :
    (streamElem w).mem Z  <->  streamElemMem w Z := Iff.rfl

/-- From **`w-vec * w-vec`**, every power `w?` still lies in `Z`. -/
theorem streamElem_powers_of_mul (w : List Bool) (Z : Set (List Bool))
    (h : (mulElem (streamElem w) (streamElem w)).mem Z) :
    streamElemMem w Z := by
  obtain <hZ, X, hX, Y, hY, hsub> := h
  obtain <_hXIn, hXw> := hX
  obtain <_hYIn, hYw> := hY
  refine <hZ, fun n => ?_>
  have h1 := hsub (append_mem_concat (hXw n) (hYw 0))
  simpa [repeatWord_zero, List.append_nil] using h1

/-- **`w-vec * w-vec` is in `Z` whenever `w-vec` is**, using the witness `powerLang w
    in  S`. -/
theorem streamElem_mul_self_mem (w : List Bool) (Z : Set (List Bool))
    (h : streamElemMem w Z) (hPL : InS (powerLang w)) :
    (mulElem (streamElem w) (streamElem w)).mem Z := by
  refine <h.1, powerLang w, <hPL, fun n => <n, rfl>>, powerLang w, <hPL, fun n => <n,
    rfl>>, ?_>
  intro u hu
  obtain <a, <m, hm>, b, <n, hn>, rfl> := hu
  simpa [hm, hn, repeatWord_add] using h.2 (m + n)

/-- Scott's stream equation **`w-vec * w-vec = w-vec`** (filter equality), when `{w?}`
    lies in `S`. -/
theorem streamElem_idempotent (w : List Bool) (hPL : InS (powerLang w)) :
    mulElem (streamElem w) (streamElem w) = streamElem w := by
  apply NeighborhoodSystem.Element.ext
  intro Z
  exact <streamElem_powers_of_mul w Z, fun h => streamElem_mul_self_mem w Z h hPL>

example : mulElem (streamElem []) (streamElem []) = streamElem [] :=
  streamElem_idempotent [] InS_powerLang_empty

example (sigma : List Bool) (Z : Set (List Bool)) :
    (mulElem (streamElem sigma) (streamElem sigma)).mem Z  ->  streamElemMem sigma Z
    :=
  streamElem_powers_of_mul sigma Z
```

```
example (sigma : List Bool) (h : InS (powerLang sigma)) :
    mulElem (mulElem (streamElem sigma) (streamElem sigma)) (streamElem sigma) =
    streamElem sigma := by
  rw [mulElem_assoc, streamElem_idempotent sigma h, streamElem_idempotent sigma h]

example (sigma : List Bool) (h : InS (powerLang (sigma ++ [true]))) :
    mulElem (streamElem (sigma ++ [true])) (streamElem (sigma ++ [true])) =
      streamElem (sigma ++ [true]) :=
  streamElem_idempotent (sigma ++ [true]) h

example (h : InS (powerLang [false, true])) :
    mulElem
        (mulElem (streamElem [false, true]) (streamElem [false, true]))
        (mulElem (streamElem [false, true]) (streamElem [false, true])) =
      mulElem (streamElem [false, true]) (streamElem [false, true]) := by
  rw [streamElem_idempotent _ h, streamElem_idempotent _ h]
end

section
/-- `S` is indeed positive (Exercise 1.19's `IsPositive`). -/
theorem Ssys_isPositive : Ssys.IsPositive := by
  intro X Y hX hY
  exact <fun h => h.nonempty, fun h => InS.inter hX hY h>
end

section
theorem Ssys_master : Ssys.master = Set.univ := rfl
end

section
/-- `{a}{b} = {a ++ b}`: concatenation of singletons is the singleton of the
    concatenation. -/
theorem concat_singleton (a b : List Bool) : concat {a} {b} = {a ++ b} := by
  ext w
  simp only [mem_concat, Set.mem_singleton_iff]
  constructor
  * rintro <a', rfl, b', rfl, rfl>; rfl
  * rintro rfl; exact <a, rfl, b, rfl, rfl>
end

section
/-- The embedding is **injective**: distinct words give distinct elements of `|S|`. (If
`emb sigma = emb tau` then `emb tau` contains `{sigma}`, forcing `tau = sigma`.) So `
    Sigma` genuinely *embeds* into `|S|`. -/
theorem emb_injective : Function.Injective emb := by
  intro sigma tau h
  have hmem : (emb tau).mem {sigma} := h  |>  (<InS.singleton sigma, rfl> : (emb sigma
    ).mem {sigma})
  exact (Set.mem_singleton_iff.mp hmem.2).symm
end

section
/-- **Exercise 7.22 (homomorphism part): `emb` is a semigroup homomorphism**,
`emb (sigma ++ tau) = emb sigma * emb tau`. Forward: from `sigma ++ tau  in  Z`, the
    witnesses `X = {sigma}`, `Y = {tau}` give
`{sigma}{tau} = {sigma ++ tau}  subseteq  Z`. Converse: if `{sigma}`  in  `emb sigma`,
```

```
    `{tau}`  in  `emb tau` with `XY  subseteq  Z` then
`sigma ++ tau  in  XY  subseteq  Z`. -/
theorem emb_mul (sigma tau : List Bool) : emb (sigma ++ tau) = mulElem (emb sigma) (
    emb tau) := by
  apply NeighborhoodSystem.Element.ext
  intro Z
  constructor
  * rintro <hZ, hsigmatau>
    refine <hZ, {sigma}, <InS.singleton sigma, rfl>, {tau}, <InS.singleton tau, rfl>,
    ?_>
    rw [concat_singleton]
    intro w hw

    rw [Set.mem_singleton_iff] at hw
    subst hw; exact hsigmatau
  * rintro <hZ, X, <_, hsigmaX>, Y, <_, htauY>, hsub>
    exact <hZ, hsub (append_mem_concat hsigmaX htauY)>
end

section
/-- **`False` is a left-annihilator up to ` <= `**: `False*y  <=  y`. (`False`'s only
    neighbourhood is `Delta = Sigma`, and
`Y  subseteq  Sigma*Y` via the empty-word split, so any witness collapses onto `y`
    itself.) -/
theorem mulElem_bot_le (y : Ssys.Element) : mulElem Ssys.bot y  <=  y := by
  rintro Z <hInS, X, hX, Y, hY, hsub>
  rw [NeighborhoodSystem.mem_bot, Ssys_master] at hX
  subst hX
  refine y.up_mem hY hInS fun w hw => hsub ?_
  simpa using append_mem_concat (Set.mem_univ ([] : List Bool)) hw
end

section
/-- `mulElem` is monotone in its right argument. -/
theorem mulElem_mono_right (x : Ssys.Element) : Monotone (mulElem x) := by
  rintro y y' hyy' Z <hInS, X, hX, Y, hY, hsub>
  exact <hInS, X, hX, Y, hyy' Y hY, hsub>
end

section
theorem powerLang_concat (w : List Bool) :
    concat (powerLang w) (powerLang w)  subseteq  powerLang w := by
  rintro u <a, <m, hm>, b, <n, hn>, rfl>
  exact <m + n, by rw [repeatWord_add, hm, hn]>
end

section
/-- `(prependMap sigma).toElementMap y = sigma * y`: the approximable-map action agrees
     with Scott's
multiplication `mulElem`, using `{sigma}  subseteq  X` (any valid witness `X` for `emb
    sigma` contains `sigma`) to
tighten the witness set to `{sigma}` without loss. -/
theorem prependMap_toElementMap (sigma : List Bool) (y : Ssys.Element) :
    (prependMap sigma).toElementMap y = mulElem (emb sigma) y := by
  apply NeighborhoodSystem.Element.ext
  intro Z
```

```
  constructor
  * rintro <Y, hY, hInSY, hInSZ, hsub>
    exact <hInSZ, {sigma}, <InS.singleton sigma, rfl>, Y, hY, hsub>
  * rintro <hInSZ, X, <_, hsigmaX>, Y, hY, hsub>
    refine <Y, hY, y.sub hY, hInSZ, ?_>
    exact (concat_mono (Set.singleton_subset_iff.mpr hsigmaX) (Set.Subset.refl _)).
    trans hsub
end

section
theorem repeatWord_succ (w : List Bool) (n : Nat) :
    repeatWord w (n + 1) = w ++ repeatWord w n := rfl
end

section
/-- **`sigma-vec = sigma*sigma-vec`** (Scott's defining equation), unconditionally. -/
theorem streamArrow_eq (sigma : List Bool) :
    mulElem (emb sigma) (streamArrow sigma) = streamArrow sigma := by
  rw [ <-  prependMap_toElementMap]
  exact toElementMap_fixElement (prependMap sigma)
end

section
/-- **Per-approximant bound**: `f?(False) * sigma-vec  <=  sigma-vec`, by induction
    using `mulElem_bot_le` (base case)
and associativity + monotonicity + `streamArrow_eq` (step). -/
theorem prependMap_iterElem_mul_streamArrow_le (sigma : List Bool) :
    forall  n, mulElem ((prependMap sigma).iterElem n) (streamArrow sigma)  <=
    streamArrow sigma
  | 0 => by
      have h0 : (prependMap sigma).iterElem 0 = Ssys.bot := by
        show (idMap Ssys).toElementMap Ssys.bot = Ssys.bot
        exact toElementMap_idMap Ssys.bot
      rw [h0]
      exact mulElem_bot_le (streamArrow sigma)
  | n + 1 => by
      have hsucc : (prependMap sigma).iterElem (n + 1) =
          mulElem (emb sigma) ((prependMap sigma).iterElem n) := by
        show ((prependMap sigma).comp ((prependMap sigma).iterMap n)).toElementMap
    Ssys.bot = _
        rw [toElementMap_comp]
        exact prependMap_toElementMap sigma _
      rw [hsucc, mulElem_assoc]
      calc mulElem (emb sigma) (mulElem ((prependMap sigma).iterElem n) (streamArrow
    sigma))
          <=  mulElem (emb sigma) (streamArrow sigma) :=
           mulElem_mono_right (emb sigma) (prependMap_iterElem_mul_streamArrow_le
    sigma n)
        _ = streamArrow sigma := streamArrow_eq sigma
end

section
/-- **`sigma-vec  <=  sigma-vec*sigma-vec`**: `sigma-vec*sigma-vec` is itself a fixed
    point of `x  |->  sigma*x` (by associativity and `sigma-vec = sigma*sigma-vec`),
and `sigma-vec` is the *least* such fixed point (`fixElement_le_of_toElementMap_le`).
    -/
```

```
theorem streamArrow_le_mul_self (sigma : List Bool) :
    streamArrow sigma  <=  mulElem (streamArrow sigma) (streamArrow sigma) := by
  apply fixElement_le_of_toElementMap_le
  have heq : (prependMap sigma).toElementMap (mulElem (streamArrow sigma) (streamArrow
     sigma)) =
      mulElem (streamArrow sigma) (streamArrow sigma) := by
    rw [prependMap_toElementMap,  <-  mulElem_assoc, streamArrow_eq]
  exact le_of_eq heq
end

section
/-- **`sigma-vec*sigma-vec  <=  sigma-vec`**: `sigma-vec`'s membership witnesses come
    from *some* finite approximant `f?(False)`
(`mem_fixElement`/`mem_iterElem`), and every approximant satisfies the per-`n` bound
    above. -/
theorem streamArrow_mul_self_le (sigma : List Bool) :
    mulElem (streamArrow sigma) (streamArrow sigma)  <=  streamArrow sigma := by
  rintro Z <hInSZ, X, hX, Y, hY, hsub>
  obtain <n, hn> := (mem_fixElement (prependMap sigma)).mp hX
  have hXn : ((prependMap sigma).iterElem n).mem X := (mem_iterElem (prependMap sigma)
     n).mpr hn
  exact prependMap_iterElem_mul_streamArrow_le sigma n Z <hInSZ, X, hXn, Y, hY, hsub>
end

section
/-- **Exercise 7.22l (Scott's first equation, unconditional).** `sigma-vec*sigma-vec =
    sigma-vec`, for the genuine
least-fixed-point `sigma-vec`, with no side-condition. -/
theorem streamArrow_mul_self (sigma : List Bool) :
    mulElem (streamArrow sigma) (streamArrow sigma) = streamArrow sigma :=
  le_antisymm (streamArrow_mul_self_le sigma) (streamArrow_le_mul_self sigma)
end

end Exercise722
end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.Exercise722
section
set_option linter.unusedVariables false

theorem example_722 : And
  (And
    (forall  (sigma tau : List Bool),
      @Eq
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (List Bool)
          DomainTheory.NeighbourhoodSystems.Exercise722.Ssys)
        (DomainTheory.NeighbourhoodSystems.Exercise722.emb
          (@HAppend.hAppend (List Bool) (List Bool) (List Bool)
            (@instHAppendOfAppend (List Bool) (@List.instAppend Bool)) sigma tau))
        (DomainTheory.NeighbourhoodSystems.Exercise722.mulElem (DomainTheory.
   NeighbourhoodSystems.Exercise722.emb sigma)
          (DomainTheory.NeighbourhoodSystems.Exercise722.emb tau)))
    (forall
      (x y z :
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (List Bool)
          DomainTheory.NeighbourhoodSystems.Exercise722.Ssys),
      @Eq
```

```
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (List Bool)
        DomainTheory.NeighbourhoodSystems.Exercise722.Ssys)
      (DomainTheory.NeighbourhoodSystems.Exercise722.mulElem
        (DomainTheory.NeighbourhoodSystems.Exercise722.mulElem x y) z)
      (DomainTheory.NeighbourhoodSystems.Exercise722.mulElem x
        (DomainTheory.NeighbourhoodSystems.Exercise722.mulElem y z))))
(forall  (sigma : List Bool),
  @Eq
    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (List Bool)
      DomainTheory.NeighbourhoodSystems.Exercise722.Ssys)
    (DomainTheory.NeighbourhoodSystems.Exercise722.mulElem
      (DomainTheory.NeighbourhoodSystems.Exercise722.streamArrow sigma)
      (DomainTheory.NeighbourhoodSystems.Exercise722.streamArrow sigma))
    (DomainTheory.NeighbourhoodSystems.Exercise722.streamArrow sigma)) :=
@And.intro
(And
  (forall  (sigma tau : List Bool),
    @Eq
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (List Bool)
        DomainTheory.NeighbourhoodSystems.Exercise722.Ssys)
      (DomainTheory.NeighbourhoodSystems.Exercise722.emb
        (@HAppend.hAppend (List Bool) (List Bool) (List Bool)
          (@instHAppendOfAppend (List Bool) (@List.instAppend Bool)) sigma tau))
      (DomainTheory.NeighbourhoodSystems.Exercise722.mulElem (DomainTheory.
 NeighbourhoodSystems.Exercise722.emb sigma)
        (DomainTheory.NeighbourhoodSystems.Exercise722.emb tau)))
  (forall
    (x y z :
      @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (List Bool)
        DomainTheory.NeighbourhoodSystems.Exercise722.Ssys),
    @Eq
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (List Bool)
        DomainTheory.NeighbourhoodSystems.Exercise722.Ssys)
      (DomainTheory.NeighbourhoodSystems.Exercise722.mulElem
        (DomainTheory.NeighbourhoodSystems.Exercise722.mulElem x y) z)
      (DomainTheory.NeighbourhoodSystems.Exercise722.mulElem x
        (DomainTheory.NeighbourhoodSystems.Exercise722.mulElem y z))))
(forall  (sigma : List Bool),
  @Eq
    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (List Bool)
      DomainTheory.NeighbourhoodSystems.Exercise722.Ssys)
    (DomainTheory.NeighbourhoodSystems.Exercise722.mulElem
      (DomainTheory.NeighbourhoodSystems.Exercise722.streamArrow sigma)
      (DomainTheory.NeighbourhoodSystems.Exercise722.streamArrow sigma))
    (DomainTheory.NeighbourhoodSystems.Exercise722.streamArrow sigma))
(@And.intro
  (forall  (sigma tau : List Bool),
    @Eq
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (List Bool)
        DomainTheory.NeighbourhoodSystems.Exercise722.Ssys)
      (DomainTheory.NeighbourhoodSystems.Exercise722.emb
        (@HAppend.hAppend (List Bool) (List Bool) (List Bool)
          (@instHAppendOfAppend (List Bool) (@List.instAppend Bool)) sigma tau))
      (DomainTheory.NeighbourhoodSystems.Exercise722.mulElem (DomainTheory.
 NeighbourhoodSystems.Exercise722.emb sigma)
        (DomainTheory.NeighbourhoodSystems.Exercise722.emb tau)))
```

```
    (forall
      (x y z :
        @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (List Bool)
          DomainTheory.NeighbourhoodSystems.Exercise722.Ssys),
      @Eq
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element (List Bool)
          DomainTheory.NeighbourhoodSystems.Exercise722.Ssys)
        (DomainTheory.NeighbourhoodSystems.Exercise722.mulElem
          (DomainTheory.NeighbourhoodSystems.Exercise722.mulElem x y) z)
        (DomainTheory.NeighbourhoodSystems.Exercise722.mulElem x
          (DomainTheory.NeighbourhoodSystems.Exercise722.mulElem y z)))
    DomainTheory.NeighbourhoodSystems.Exercise722.emb_mul DomainTheory.
    NeighbourhoodSystems.Exercise722.mulElem_assoc)
  DomainTheory.NeighbourhoodSystems.Exercise722.streamArrow_mul_self
end

end DomainTheory.NeighbourhoodSystems.Exercise722
namespace DomainTheory.NeighbourhoodSystems
namespace Exercise722
open NeighborhoodSystem ApproximableMap

section
/-- **Scott's fourth equation (the concrete numeric instance).** `01-vec*01-vec*01-vec
    *01-vec = 01-vec*01-vec`, for
`01-vec := streamArrow [false, true]`. Follows from `streamArrow_mul_self` applied
    twice. -/
theorem streamArrow_containsZero_pow_four :
    mulElem
        (mulElem (streamArrow [false, true]) (streamArrow [false, true]))
        (mulElem (streamArrow [false, true]) (streamArrow [false, true]))
      = mulElem (streamArrow [false, true]) (streamArrow [false, true]) := by
  rw [streamArrow_mul_self, streamArrow_mul_self]
end

section
/-- **Scott's third equation.** `sigma1-vec * sigma1-vec = sigma1-vec`, where `sigma1-
    vec := streamArrow (sigma ++ [true])` is the
arrow of the *combined* token `sigma1` (Scott's notation `sigma-vec1-vec` names the
    infinite repetition of the
word `sigma` followed by `1`, not a product of two separate arrows -- matching how the
    file's earlier
`streamElem`-based examples read the same equation). An instance of `
    streamArrow_mul_self`. -/
theorem streamArrow_mul_self_append_true (sigma : List Bool) :
    mulElem (streamArrow (sigma ++ [true])) (streamArrow (sigma ++ [true])) =
      streamArrow (sigma ++ [true]) :=
  streamArrow_mul_self (sigma ++ [true])
end

section
/-- **Scott's second equation.** `sigma-vec*sigma-vec*sigma-vec = sigma-vec`. -/
theorem streamArrow_mul_self_self (sigma : List Bool) :
    mulElem (mulElem (streamArrow sigma) (streamArrow sigma)) (streamArrow sigma) =
    streamArrow sigma := by
  rw [streamArrow_mul_self, streamArrow_mul_self]
end
```

```
section
/-- **The syntax of `S`-terms** (Scott's four generators): `Sigma`, a singleton `{sigma
    }`, concatenation
`X*Y`, and intersection `X  I  Y`. A fragment of regular expressions with `*` and ` I `
     and the
universal language `Sigma`, but no union / complement / star. -/
inductive SExpr : Type
  | sigma : SExpr
  | single : List Bool  ->  SExpr
  | cat : SExpr  ->  SExpr  ->  SExpr
  | cap : SExpr  ->  SExpr  ->  SExpr
  deriving DecidableEq
end

section
/-- Decode with an explicit fuel bound so the recursion is structural. -/
def sexprOfNatFuel : Nat  ->  Nat  ->  Option SExpr
  | 0, _ => none
  | fuel + 1, n =>
    match n.unpair.1 with
    | 0 => some .sigma
    | 1 => (Encodable.decode n.unpair.2).map SExpr.single
    | 2 =>
      match sexprOfNatFuel fuel n.unpair.2.unpair.1, sexprOfNatFuel fuel n.unpair.2.
    unpair.2 with
      | some a, some b => some (.cat a b)
      | _, _ => none
    | 3 =>
      match sexprOfNatFuel fuel n.unpair.2.unpair.1, sexprOfNatFuel fuel n.unpair.2.
    unpair.2 with
      | some a, some b => some (.cap a b)
      | _, _ => none
    | _ => none
end

section
def sexprOfNat (n : Nat) : Option SExpr := sexprOfNatFuel (n + 1) n
end

section
def sexprToNat : SExpr  ->  Nat
  | .sigma => Nat.pair 0 0
  | .single sigma => Nat.pair 1 (Encodable.encode sigma)
  | .cat a b => Nat.pair 2 (Nat.pair (sexprToNat a) (sexprToNat b))
  | .cap a b => Nat.pair 3 (Nat.pair (sexprToNat a) (sexprToNat b))
end

section
theorem snd_lt_pair_of_pos_fst {a b : Nat} (ha : 0 < a) : b < Nat.pair a b := by
  rw [Nat.pair]
  split_ifs with h
  * have hb : 0 < b := Nat.lt_trans ha h
    have hbb : b  <=  b * b := by
      calc
        b = 1 * b := (Nat.one_mul b).symm
```

```
        _  <=  b * b := Nat.mul_le_mul_right b (Nat.succ_le_of_lt hb)
    have : b * b < b * b + a := Nat.lt_add_of_pos_right ha
    omega
  * have : 0 < a * a + a := by
      have : a  <=  a * a + a := Nat.le_add_left _ _
      omega
    omega
end

section
theorem sexprOfNatFuel_sexprToNat (e : SExpr) :
    forall  fuel, sexprToNat e < fuel  ->  sexprOfNatFuel fuel (sexprToNat e) = some e
     := by
  induction e with
  | sigma =>
    intro fuel hfuel
    cases fuel with
    | zero => omega
    | succ fuel => simp [sexprToNat, sexprOfNatFuel, Nat.unpair_pair]
  | single sigma =>
    intro fuel hfuel
    cases fuel with
    | zero => omega
    | succ fuel => simp [sexprToNat, sexprOfNatFuel, Nat.unpair_pair, Encodable.
    encodek]
  | cat a b iha ihb =>
    intro fuel hfuel
    cases fuel with
    | zero => omega
    | succ fuel =>
      have ha : sexprToNat a < fuel := by
        have hinner : sexprToNat a  <=  Nat.pair (sexprToNat a) (sexprToNat b) := Nat.
    left_le_pair _ _
        have houter : Nat.pair (sexprToNat a) (sexprToNat b) < sexprToNat (.cat a b)
    := by
          simpa [sexprToNat] using snd_lt_pair_of_pos_fst (a := 2) (b := Nat.pair (
    sexprToNat a) (sexprToNat b)) (by omega)
        omega
      have hb : sexprToNat b < fuel := by
        have hinner : sexprToNat b  <=  Nat.pair (sexprToNat a) (sexprToNat b) := Nat.
    right_le_pair _ _
        have houter : Nat.pair (sexprToNat a) (sexprToNat b) < sexprToNat (.cat a b)
    := by
          simpa [sexprToNat] using snd_lt_pair_of_pos_fst (a := 2) (b := Nat.pair (
    sexprToNat a) (sexprToNat b)) (by omega)
        omega
      simp [sexprToNat, sexprOfNatFuel, Nat.unpair_pair, iha _ ha, ihb _ hb]
  | cap a b iha ihb =>
    intro fuel hfuel
    cases fuel with
    | zero => omega
    | succ fuel =>
      have ha : sexprToNat a < fuel := by
        have hinner : sexprToNat a  <=  Nat.pair (sexprToNat a) (sexprToNat b) := Nat.
    left_le_pair _ _
        have houter : Nat.pair (sexprToNat a) (sexprToNat b) < sexprToNat (.cap a b)
```

```
    := by
          simpa [sexprToNat] using snd_lt_pair_of_pos_fst (a := 3) (b := Nat.pair (
    sexprToNat a) (sexprToNat b)) (by omega)
        omega
      have hb : sexprToNat b < fuel := by
        have hinner : sexprToNat b  <=  Nat.pair (sexprToNat a) (sexprToNat b) := Nat.
    right_le_pair _ _
        have houter : Nat.pair (sexprToNat a) (sexprToNat b) < sexprToNat (.cap a b)
    := by
          simpa [sexprToNat] using snd_lt_pair_of_pos_fst (a := 3) (b := Nat.pair (
    sexprToNat a) (sexprToNat b)) (by omega)
        omega
      simp [sexprToNat, sexprOfNatFuel, Nat.unpair_pair, iha _ ha, ihb _ hb]
end

section
instance : Encodable SExpr where
  encode := sexprToNat
  decode := sexprOfNat
  encodek e := sexprOfNatFuel_sexprToNat e (sexprToNat e + 1) (Nat.lt_succ_self _)
end

section
/-- The language denoted by an `S`-term. -/
def denote : SExpr  ->  Set (List Bool)
  | .sigma => Set.univ
  | .single sigma => {sigma}
  | .cat a b => concat (denote a) (denote b)
  | .cap a b => denote a  I  denote b
end

section
@[simp] theorem denote_sigma : denote .sigma = Set.univ := rfl
end

section
@[simp] theorem denote_single (sigma : List Bool) : denote (.single sigma) = {sigma}
    := rfl
end

section
@[simp] theorem denote_cat (a b : SExpr) : denote (.cat a b) = concat (denote a) (
    denote b) := rfl
end

section
@[simp] theorem denote_cap (a b : SExpr) : denote (.cap a b) = denote a  I  denote b
    := rfl
end

section
/-- The **`Bool` matcher** for `S`-terms: a structurally-recursive decision of word
    membership. The
concatenation case tries every cut point `i  in  {0,...,|w|}`; the intersection case is
     the conjunction. -/
def matchesB : SExpr  ->  List Bool  ->  Bool
```

```
  | .sigma, _ => true
  | .single sigma, w => decide (w = sigma)
  | .cat a b, w => (List.range (w.length + 1)).any (fun i => matchesB a (w.take i) &&
    matchesB b (w.drop i))
  | .cap a b, w => matchesB a w && matchesB b w
end

section
/-- A word `w` lies in a concatenation `X*Y` iff it splits as `w = w.take i ++ w.drop i
    ` with the
prefix in `X` and the suffix in `Y` for some cut point `i  <=  |w|`. (The cut point is
    the length of
the `X`-part.) -/
theorem mem_concat_iff_split {X Y : Set (List Bool)} {w : List Bool} :
    w  in  concat X Y  <->  exists  i, i  <=  w.length  /\  w.take i  in  X  /\  w.
    drop i  in  Y := by
  constructor
  * rintro <a, ha, b, hb, rfl>
    refine <a.length, by simp, ?_, ?_>
    * rw [List.take_left]; exact ha

    * rw [List.drop_left]; exact hb
  * rintro <i, _, hpre, hsuf>
    exact <w.take i, hpre, w.drop i, hsuf, w.take_append_drop i>
end

section
/-- **The matcher is correct**: `matchesB e w = true  <->  w  in  denote e`. By
    structural induction on `e`;
the concatenation case unfolds `List.any` over the cut points and applies `
    mem_concat_iff_split`. -/
theorem matchesB_iff (e : SExpr) (w : List Bool) : matchesB e w = true  <->  w  in
    denote e := by
  induction e generalizing w with
  | sigma => simp [matchesB]
  | single sigma => simp [matchesB]
  | cat a b iha ihb =>
    rw [denote_cat, mem_concat_iff_split]
    simp only [matchesB, List.any_eq_true, List.mem_range, Bool.and_eq_true]
    constructor
    * rintro <i, hi, ha, hb>
      exact <i, Nat.lt_succ_iff.mp hi, (iha _).mp ha, (ihb _).mp hb>
    * rintro <i, hi, ha, hb>
      exact <i, Nat.lt_succ_iff.mpr hi, (iha _).mpr ha, (ihb _).mpr hb>
  | cap a b iha ihb =>
    simp only [matchesB, Bool.and_eq_true, denote_cap, Set.mem_inter_iff]
    rw [iha, ihb]
end

section
/-- **Membership in a regular event is decidable** (choice-free), via the matcher. -/
instance decidableMemDenote (e : SExpr) : DecidablePred (*  in  denote e) :=
  fun w => decidable_of_iff (matchesB e w = true) (matchesB_iff e w)
end

section
```

```
instance : Countable SExpr := inferInstance
end

section
/-- The empty language is denotable in the fragment (it is **not** a member of `S`,
    which is
positive): `{0}  I  {1} = {}`. This is the canonical "`{}`" expression used to phrase
    emptiness as an
equivalence query. -/
def emptyExpr : SExpr := .cap (.single [false]) (.single [true])
end

section
@[simp] theorem denote_emptyExpr : denote emptyExpr = ({} : Set (List Bool)) := by
  ext w
  simp only [emptyExpr, denote_cap, denote_single, Set.mem_inter_iff, Set.
    mem_singleton_iff,
    Set.mem_empty_iff_false, iff_false, not_and]
  rintro rfl
  decide
end

section
/-- "Contains a `0`" as a fragment expression: `Sigma * {0} * Sigma`. -/
def containsZero : SExpr := .cat .sigma (.cat (.single [false]) .sigma)
end

section
/-- **Soundness.** A *non-empty* denotation is a member of `S`. (Non-emptiness is
    needed only for the
intersection generator, where `InS.inter` demands it; it propagates down to subterms
    because
`concat`/` I ` are non-empty only when their parts are.) -/
theorem InS_denote_of_nonempty : forall  {e : SExpr}, (denote e).Nonempty  ->  InS (
    denote e)
  | .sigma, _ => InS.univ
  | .single sigma, _ => InS.singleton sigma
  | .cat a b, h => by
    rw [denote_cat] at h  |-
    obtain <w, x, hx, y, hy, _> := h
    exact InS.mul (InS_denote_of_nonempty <x, hx>) (InS_denote_of_nonempty <y, hy>)
  | .cap a b, h => by
    rw [denote_cap] at h  |-
    have ha : (denote a).Nonempty := h.mono Set.inter_subset_left
    have hb : (denote b).Nonempty := h.mono Set.inter_subset_right
    exact InS.inter (InS_denote_of_nonempty ha) (InS_denote_of_nonempty hb) h
end

section
/-- **Completeness.** Every member of `S` is the denotation of some `S`-term. -/
theorem InS_exists_denote {X : Set (List Bool)} (h : InS X) : exists  e : SExpr,
    denote e = X := by
  induction h with
  | univ => exact <.sigma, rfl>
  | singleton sigma => exact <.single sigma, rfl>
  | mul _ _ iha ihb =>
```

```
    obtain <ea, rfl> := iha
    obtain <eb, rfl> := ihb
    exact <.cat ea eb, rfl>
  | inter _ _ _ iha ihb =>
    obtain <ea, rfl> := iha
    obtain <eb, rfl> := ihb
    exact <.cap ea eb, rfl>
end

section
/-- **`denote` is far from injective: `Sigma * Sigma = Sigma`.** A nontrivial language
    identity (with distinct
syntax), so relation (i) `X_n I X? = X_k` is *not* syntactic equality of codes -- it is
     genuine
regular-language equivalence, the whole difficulty of the exercise. -/
theorem denote_catSigmaSigma : denote (.cat .sigma .sigma) = denote .sigma := by
  ext w
  simp only [denote_cat, denote_sigma, mem_concat, Set.mem_univ, iff_true]
  exact <[], trivial, w, trivial, rfl>
end

section
/-- **Definition 7.1(ii), syntactically.** Inconsistency of `e_n, e?` -- emptiness of `
    X_n  I  X?` -- is
exactly *language-equivalence of `cap e_n e?` with `emptyExpr`*. Consistency (relation
    (ii)) is its
**negation**, so even (ii) is an instance of deciding language equivalence on `SExpr` (
    here:
equivalence to `{}`). Stated as an emptiness-equivalence to keep it choice-free (
    recovering a witnessing
element from ` !=  {}` is the one genuinely non-constructive step, and is not needed
    for the decision). -/
theorem empty_iff_equiv_emptyExpr (a b : SExpr) :
    denote (.cap a b) = ({} : Set (List Bool))  <->  denote (.cap a b) = denote
    emptyExpr := by
  rw [denote_emptyExpr]
end

section
theorem inS_eq_range_denote : {X | InS X} = denote '' {e | (denote e).Nonempty} := by
  ext X
  simp only [Set.mem_ofPred_eq, Set.mem_image]
  constructor
  * intro h
    obtain <e, he> := InS_exists_denote h
    exact <e, by rw [he]; exact h.nonempty, he>
  * rintro <e, hne, rfl>
    exact InS_denote_of_nonempty hne
end

section
/-- **Characterization of `S` as the regular events of this fragment.** A language `X`
    is in `S`
exactly when it is the *non-empty* denotation of some `Sigma`/`{sigma}`/`*`/` I `
    expression. This is the
precise content of Scott's "the sets in `S` are each regular events". -/
```

```
theorem inS_iff_exists_denote {X : Set (List Bool)} :
    InS X  <->  exists  e : SExpr, denote e = X  /\  X.Nonempty := by
  constructor
  * intro h
    obtain <e, he> := InS_exists_denote h
    exact <e, he, h.nonempty>
  * rintro <e, rfl, hne>
    exact InS_denote_of_nonempty hne
end

section
/-- **Definition 7.1(i), syntactically**, is *exactly language equivalence on `SExpr`*:
`X_n  I  X? = X_k  <->  denote (cap e_n e?) = denote e_k`. -/
theorem interEq_iff (a b k : SExpr) :
    denote a  I  denote b = denote k  <->  denote (.cap a b) = denote k := by
  rw [denote_cap]
end

section
/-- `denote .sigma = Sigma` contains every word. -/
theorem mem_denote_sigma (w : List Bool) : w  in  denote .sigma := Set.mem_univ w
end

section
/-- **The emptiness decider cannot decide relation (i).** `Sigma  !=  "contains 0"`,
    witnessed by `[1]  in  Sigma`
but `[1]  notin  Sigma*{0}*Sigma`. To *decide* this inequality one must detect a word
    in `Sigma  I  ("contains 0")?` --
i.e. in the complement language `{1}*` -- but `{1}*` is **not** expressible in the
    fragment (no
complement, no star other than the full `Sigma`). Hence no fragment-emptiness query
    exposes the
difference: deciding equivalence needs machinery (a product/complement DFA, or a
    derivative
bisimulation) that goes strictly beyond emptiness of fragment expressions. -/
theorem sigma_ne_containsZero : denote .sigma  !=  denote containsZero := by
  intro h
  have h1 : ([true] : List Bool)  in  denote .sigma := mem_denote_sigma _
  rw [h] at h1
  exact absurd h1 (by decide)
end

end Exercise722
end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.Exercise722
section
set_option linter.unusedVariables false

theorem example_722_regular : @Ne (Set (List Bool))
  (DomainTheory.NeighbourhoodSystems.Exercise722.denote DomainTheory.
    NeighbourhoodSystems.Exercise722.SExpr.sigma)
  (DomainTheory.NeighbourhoodSystems.Exercise722.denote DomainTheory.
    NeighbourhoodSystems.Exercise722.containsZero) :=
  DomainTheory.NeighbourhoodSystems.Exercise722.sigma_ne_containsZero
end
```

```
end DomainTheory.NeighbourhoodSystems.Exercise722
namespace DomainTheory.NeighbourhoodSystems
namespace Exercise722
open scoped Computability
open Sum Set

section
variable {sigma_1 sigma_2 : Type}

/-- The concatenation `?NFA`: `M_1` on `inl`, `M_2` on `inr`, ?-edges from `M_1`-accept
    to `M_2`-start. -/
def catEps (M_1 : NFA Bool sigma_1) (M_2 : NFA Bool sigma_2) : ?NFA Bool (sigma_1 ?
    sigma_2) where
  step s a :=
    match s, a with
    | inl s, some c => inl '' M_1.step s c
    | inl s, none   => {t | s  in  M_1.accept  /\  t  in  inr '' M_2.start}
    | inr s, some c => inr '' M_2.step s c
    | inr _, none   => {}
  start := inl '' M_1.start
  accept := inr '' M_2.accept
end

section
variable {sigma_1 sigma_2 : Type}
variable (M_1 : NFA Bool sigma_1) (M_2 : NFA Bool sigma_2)

@[simp] theorem catEps_step_inl_some (s : sigma_1) (c : Bool) :
    (catEps M_1 M_2).step (inl s) (some c) = inl '' M_1.step s c := rfl
@[simp] theorem catEps_step_inl_none (s : sigma_1) :
    (catEps M_1 M_2).step (inl s) none = {t | s  in  M_1.accept  /\  t  in  inr '' M_2
    .start} := rfl
@[simp] theorem catEps_step_inr_some (s : sigma_2) (c : Bool) :
    (catEps M_1 M_2).step (inr s) (some c) = inr '' M_2.step s c := rfl
@[simp] theorem catEps_step_inr_none (s : sigma_2) :
    (catEps M_1 M_2).step (inr s) none = {} := rfl
@[simp] theorem catEps_start : (catEps M_1 M_2).start = inl '' M_1.start := rfl
@[simp] theorem catEps_accept : (catEps M_1 M_2).accept = inr '' M_2.accept := rfl

example [Fintype sigma_1] [Fintype sigma_2] : Fintype (sigma_1 ? sigma_2) :=
    inferInstance

/-- ?-closure: the only ?-edges go from an `inl`-accept state to `inr`-start states, so
     the closure
of `T` adds `inr '' M_2.start` exactly when `T` already contains some `inl`-accept
    state. -/
theorem catEps_mem_?Closure_iff (T : Set (sigma_1 ? sigma_2)) (s : sigma_1 ? sigma_2)
    :
    s  in  (catEps M_1 M_2).?Closure T  <->
      s  in  T  \/  ((exists  q  in  M_1.accept, (inl q : sigma_1 ? sigma_2)  in  T)
    /\  s  in  inr '' M_2.start) := by
  constructor
  * intro h
    induction h using ?NFA.?Closure_induction with
    | base x hx => exact Or.inl hx
    | step x y hyx _hclos ih =>
```

```
      -- a `none`-step `x  ->  y`; nonempty only when `x = inl q`, `q  in  M_1.accept`,
     `y  in  inr '' start`
      cases x with
      | inl q =>
        rcases ih with hT | <_, hmem>
        * simp only [catEps_step_inl_none, mem_ofPred_eq] at hyx
          exact Or.inr <<q, hyx.1, hT>, hyx.2>
        * obtain <w, _, hw> := hmem; exact absurd hw.symm (by simp)
      | inr q => simp only [catEps_step_inr_none] at hyx; exact absurd hyx (by simp)
  * rintro (hs | <<q, hq, hqT>, hs>)
    * exact (catEps M_1 M_2).subset_?Closure T hs
    * obtain <t, ht, rfl> := hs
      have hbase : (inl q : sigma_1 ? sigma_2)  in  (catEps M_1 M_2).?Closure T :=
        (catEps M_1 M_2).subset_?Closure T hqT
      refine (catEps M_1 M_2).mem_?Closure_of_mem_step ?_ hbase
      simp only [catEps_step_inl_none, mem_ofPred_eq]
      exact <hq, t, ht, rfl>

/-- **Closed form for `eval`.** After reading `x`, the reachable `inl`-states mirror `
    M_1.eval x`, and
the reachable `inr`-states are exactly the `M_2`-states reachable from some prefix-
    split `x = u ++ v`
with the `M_1`-prefix `u` accepted. This is the engine behind `catEps_accepts`. -/
theorem catEps_mem_eval_iff (x : List Bool) (s : sigma_1 ? sigma_2) :
    s  in  (catEps M_1 M_2).eval x  <->
      (exists  s_1, s = inl s_1  /\  s_1  in  M_1.eval x)  \/
      (exists  t, s = inr t  /\  exists  u v, x = u ++ v  /\  u  in  M_1.accepts  /\
    t  in  M_2.eval v) := by
  induction x using List.reverseRecOn generalizing s with
  | nil =>
    show s  in  (catEps M_1 M_2).?Closure (catEps M_1 M_2).start  <->  _
    rw [catEps_start, catEps_mem_?Closure_iff]
    constructor
    * rintro (hs | <<q, hq, hqs>, hs>)
      * obtain <s_1, hs_1, rfl> := hs
        exact Or.inl <s_1, rfl, hs_1>
      * obtain <t, ht, rfl> := hs
        refine Or.inr <t, rfl, [], [], rfl, ?_, ht>
        obtain <q', hq', hqq> := hqs
        cases hqq
        exact <q, hq, hq'>
    * rintro (<s_1, rfl, hs_1> | <t, rfl, u, v, hx, hu, hv>)
      * exact Or.inl <s_1, hs_1, rfl>
      * obtain <rfl, rfl> := List.append_eq_nil_iff.mp hx.symm
        obtain <q, hq, hqstart> := hu
        exact Or.inr <<q, hq, q, hqstart, rfl>, t, hv, rfl>
  | append_singleton x a ih =>
    rw [?NFA.eval_append_singleton, ?NFA.mem_stepSet_iff]
    constructor
    * rintro <t, ht, hs>
      rw [ih] at ht
      rcases ht with <s_1, rfl, hs_1> | <t_2, rfl, u, v, hx, hu, hv>
      * -- came from an `inl` state of `M_1`
        rw [catEps_step_inl_some, catEps_mem_?Closure_iff] at hs
        rcases hs with hs | <<q, hq, hqmem>, hs>
        * -- still in `M_1`: extends the `inl` form
```

```
      obtain <s', hs', rfl> := hs
      exact Or.inl <s', rfl, by rw [NFA.eval_append_singleton, NFA.mem_stepSet];
    exact <s_1, hs_1, hs'>>
    * -- `M_1` just reached an accept state on reading `a`: cross to `M_2` (split `
    v = []`)
      obtain <q', hq', hqq> := hqmem
      cases hqq
      obtain <t', ht', rfl> := hs
      refine Or.inr <t', rfl, x ++ [a], [], (List.append_nil _).symm, ?_, ht'>
      rw [NFA.mem_accepts]
      refine <q, hq, ?_>
      change q  in  M_1.eval (x ++ [a])
      rw [NFA.eval_append_singleton, NFA.mem_stepSet]
      exact <s_1, hs_1, hq'>
  * -- came from an `inr` state of `M_2`: extends the `inr` form (split `v  |->  v
    ++ [a]`)
    rw [catEps_step_inr_some, catEps_mem_?Closure_iff] at hs
    rcases hs with hs | <<q, _, hqmem>, _>
    * obtain <t', ht', rfl> := hs
      refine Or.inr <t', rfl, u, v ++ [a], by rw [hx, List.append_assoc], hu, ?_>
      rw [NFA.eval_append_singleton, NFA.mem_stepSet]; exact <t_2, hv, ht'>
    * exact absurd hqmem (by simp)
* rintro (<s', rfl, hs'> | <t', rfl, u, v, hx, hu, hv>)
  * -- `inl` form: step `M_1` one more
    rw [NFA.eval_append_singleton, NFA.mem_stepSet] at hs'
    obtain <s_1, hs_1, hstep> := hs'
    exact <inl s_1, (ih (inl s_1)).mpr (Or.inl <s_1, rfl, hs_1>),
      by rw [catEps_step_inl_some, catEps_mem_?Closure_iff]; exact Or.inl <s',
    hstep, rfl>>
  * -- `inr` form: the split `x ++ [a] = u ++ v`; cases on whether `v` is empty
    rcases List.eq_nil_or_concat v with rfl | <v', c, rfl>
    * -- `v = []`: `u = x ++ [a]  in  L_1`; cross via the ?-edge after reading `a`
      rw [List.append_nil] at hx; subst hx
      rw [NFA.mem_accepts] at hu
      obtain <q, hq, hqe> := hu
      change q  in  M_1.eval (x ++ [a]) at hqe
      rw [NFA.eval_append_singleton, NFA.mem_stepSet] at hqe
      obtain <s_1, hs_1, hstep> := hqe
      refine <inl s_1, (ih (inl s_1)).mpr (Or.inl <s_1, rfl, hs_1>), ?_>
      rw [catEps_step_inl_some, catEps_mem_?Closure_iff]
      exact Or.inr <<q, hq, q, hstep, rfl>, t', hv, rfl>
    * -- `v = v' ++ [c]`: necessarily `c = a`, `x = u ++ v'`; step `M_2` one more
      rw [List.concat_eq_append] at hx hv
      rw [ <-  List.append_assoc] at hx
      obtain <hxe, hac> := List.append_inj' hx rfl
      have hca : c = a := by simpa using hac.symm
      subst hca
      rw [NFA.eval_append_singleton, NFA.mem_stepSet] at hv
      obtain <t_2, ht_2, hstep> := hv
      refine <inr t_2, (ih (inr t_2)).mpr (Or.inr <t_2, rfl, u, v', hxe, hu, ht_2
    >), ?_>
      rw [catEps_step_inr_some, catEps_mem_?Closure_iff]
      exact Or.inl <t', hstep, rfl>

/-- **The concatenation automaton recognises exactly the concatenation language.** -/
theorem catEps_accepts :
```

```
    (catEps M_1 M_2).accepts = concat M_1.accepts M_2.accepts := by
  ext x
  constructor
  * rintro <S, hS, hSeval>
    obtain <t, ht, rfl> := hS
    rw [catEps_mem_eval_iff] at hSeval
    rcases hSeval with <s_1, hc, _> | <t', heq, u, v, hx, hu, hv>
    * exact absurd hc (by simp)
    * cases heq
      exact <u, hu, v, <t, ht, hv>, hx.symm>
  * rintro <u, hu, v, hv, rfl>
    obtain <t, ht, hteval> := hv
    refine <inr t, <t, ht, rfl>, ?_>
    rw [catEps_mem_eval_iff]
    exact Or.inr <t, rfl, u, v, rfl, hu, hteval>

end

end Exercise722
end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.Exercise722
section
set_option linter.unusedVariables false

theorem example_722_cat : forall  {sigma_1 sigma_2 : Type} (M_1 : NFA Bool sigma_1) (
    M_2 : NFA Bool sigma_2),
  @Eq (Language Bool)
    (@?NFA.accepts Bool (Sum sigma_1 sigma_2) (@DomainTheory.NeighbourhoodSystems.
    Exercise722.catEps sigma_1 sigma_2 M_1 M_2))
    (DomainTheory.NeighbourhoodSystems.Exercise722.concat (@NFA.accepts Bool sigma_1
    M_1) (@NFA.accepts Bool sigma_2 M_2)) :=
  @DomainTheory.NeighbourhoodSystems.Exercise722.catEps_accepts
end

end DomainTheory.NeighbourhoodSystems.Exercise722
namespace DomainTheory.NeighbourhoodSystems
namespace Exercise722
open scoped Computability

section
/-- The one-state DFA recognising `Sigma = {0,1}*` (every word accepted). -/
def sigmaDFA : DFA Bool Unit where
  step _ _ := ()
  start := ()
  accept := Set.univ
end

section
@[simp] theorem sigmaDFA_accepts : sigmaDFA.accepts = Set.univ :=
  Set.eq_univ_of_forall fun _ => Set.mem_univ _
end

section
/-- States of the single-word DFA for `sigma`: `some i` means "the input so far is the
    length-`i` prefix
of `sigma`" (`i  <=  |sigma|`); `none` is the dead/sink state (input has diverged from
```

```
    `sigma`). -/
abbrev SingleState (sigma : List Bool) : Type := Option (Fin (sigma.length + 1))
end

section
/-- The single-word DFA recognising `{sigma}`. From `some i` (`i < |sigma|`) reading
    the correct next
character advances to `some (i+1)`; any mismatch -- or any character once `sigma` is
    fully consumed -- goes
to the dead state `none`. -/
def singleDFA (sigma : List Bool) : DFA Bool (SingleState sigma) where
  step s b := s.bind fun i =>
    if h : (i : Nat) < sigma.length then
      if sigma.get <i, h> = b then some <i + 1, by omega> else none
    else none
  start := some <0, by omega>
  accept := {some <sigma.length, by omega>}
end

section
/-- The complement DFA recognises the set-complement of the original language (over
    all of `Sigma*`).
(Choice-free; avoids mathlib's classical `accepts_compl`.) -/
theorem complDFA_accepts {sigma : Type} (M : DFA Bool sigma) (a : SExpr) (h : M.
    accepts = denote a) :
    (M?).accepts = (denote a)? := by


  rw [ <-  h]
  ext w
  exact Iff.rfl

example : Fintype (Unit) := inferInstance
example : DecidableEq (Unit) := inferInstance
example (sigma : List Bool) : Fintype (SingleState sigma) := by unfold SingleState;
    infer_instance
example (sigma : List Bool) : DecidableEq (SingleState sigma) := by unfold SingleState
    ; infer_instance
example {sigma_1 sigma_2 : Type} [Fintype sigma_1] [Fintype sigma_2] : Fintype (
    sigma_1  x  sigma_2) := inferInstance
example {sigma_1 sigma_2 : Type} [DecidableEq sigma_1] [DecidableEq sigma_2] :
    DecidableEq (sigma_1  x  sigma_2) := inferInstance
end

section
/-- The product DFA evaluates componentwise. (Choice-free; avoids mathlib's classical
`accepts_inter`.) -/
theorem inter_eval {sigma_1 sigma_2 : Type} (M_1 : DFA Bool sigma_1) (M_2 : DFA Bool
    sigma_2) (w : List Bool) :
    (M_1.inter M_2).eval w = (M_1.eval w, M_2.eval w) := by
  induction w using List.reverseRecOn with
  | nil => rfl
  | append_singleton w a ih =>
    rw [DFA.eval_append_singleton, DFA.eval_append_singleton, DFA.
    eval_append_singleton, ih]
    rfl
end
```

```
section
/-- If `M_1`, `M_2` recognise `denote a`, `denote b`, their product DFA recognises `
    denote (a  I  b)`. -/
theorem interDFA_accepts {sigma_1 sigma_2 : Type} (M_1 : DFA Bool sigma_1) (M_2 : DFA
    Bool sigma_2)
    (a b : SExpr) (h_1 : M_1.accepts = denote a) (h_2 : M_2.accepts = denote b) :
    (M_1.inter M_2).accepts = denote (.cap a b) := by
  rw [denote_cap,  <-  h_1,  <-  h_2]
  ext w
  constructor
  * intro hw
    have hp : (M_1.inter M_2).eval w  in  (M_1.inter M_2).accept := hw
    rw [inter_eval] at hp
    exact hp
  * intro hw
    show (M_1.inter M_2).eval w  in  (M_1.inter M_2).accept
    rw [inter_eval]
    exact hw
end

end Exercise722
end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.Exercise722
section
set_option linter.unusedVariables false

theorem example_722_dfa : forall  {sigma_1 sigma_2 : Type} (M_1 : DFA Bool sigma_1) (
    M_2 : DFA Bool sigma_2) (a b : DomainTheory.NeighbourhoodSystems.Exercise722.SExpr)
    ,
  @Eq (Language Bool) (@DFA.accepts Bool sigma_1 M_1) (DomainTheory.
    NeighbourhoodSystems.Exercise722.denote a)  ->
    @Eq (Language Bool) (@DFA.accepts Bool sigma_2 M_2) (DomainTheory.
    NeighbourhoodSystems.Exercise722.denote b)  ->
      @Eq (Language Bool) (@DFA.accepts Bool (Prod sigma_1 sigma_2) (@DFA.inter Bool
    sigma_1 sigma_2 M_1 M_2))
        (DomainTheory.NeighbourhoodSystems.Exercise722.denote (a.cap b)) :=
  @DomainTheory.NeighbourhoodSystems.Exercise722.interDFA_accepts
end

end DomainTheory.NeighbourhoodSystems.Exercise722
namespace DomainTheory.NeighbourhoodSystems
namespace Exercise722
open scoped Computability

section
theorem sigmaDFA_accepts_denote : sigmaDFA.accepts = denote .sigma := by
  rw [sigmaDFA_accepts, denote_sigma]
end

section
/-- The dead state `none` is a sink. -/
theorem singleDFA_evalFrom_none (sigma : List Bool) (w : List Bool) :
    (singleDFA sigma).evalFrom none w = none := by
  induction w with
  | nil => rfl
```

```
  | cons b w ih => rw [DFA.evalFrom_cons]; exact ih
end

section
/-- From state `some k`, reading `w` reaches `some (k+|w|)` exactly when `w` is a
    prefix of the
suffix `sigma.drop k` (i.e. `w` continues `sigma` from position `k` without diverging);
     otherwise it dies. -/
theorem singleDFA_evalFrom (sigma : List Bool) (w : List Bool) (k : Fin (sigma.length
    + 1)) :
    (singleDFA sigma).evalFrom (some k) w =
      if h : w <+: List.drop (k : Nat) sigma
      then some <(k : Nat) + w.length, by
        have hlen := h.length_le
        rw [List.length_drop] at hlen
        have := k.isLt
        omega>
      else none := by
  induction w generalizing k with
  | nil =>
    rw [DFA.evalFrom_nil, dite_eq_left List.nil_prefix]
    simp
  | cons b w ih =>
    rw [DFA.evalFrom_cons]
    show (singleDFA sigma).evalFrom ((singleDFA sigma).step (some k) b) w = _
    by_cases hk : (k : Nat) < sigma.length
    * have hdrop : List.drop (k : Nat) sigma = sigma.get <k, hk> :: List.drop ((k :
    Nat) + 1) sigma := by
        rw [List.get_eq_getElem]; exact List.drop_eq_getElem_cons hk
      have hcondiff : ((b :: w) <+: List.drop (k : Nat) sigma)  <->  (b = sigma.get <k
    , hk>  /\  w <+: List.drop ((k : Nat) + 1) sigma) := by
        rw [hdrop, List.cons_prefix_cons]
      by_cases hchar : sigma.get <k, hk> = b
      * have hstep : (singleDFA sigma).step (some k) b = some <(k : Nat) + 1, by omega
    > := by
          simp only [singleDFA, Option.bind_some, dite_eq_left hk, ite_eq_left hchar]
        rw [hstep, ih <(k : Nat) + 1, by omega>]
        by_cases hpre : w <+: List.drop ((k : Nat) + 1) sigma
        * rw [dite_eq_left hpre, dite_eq_left (hcondiff.mpr <hchar.symm, hpre>)]
          congr 1
          apply Fin.ext
          simp only [List.length_cons]
          omega
        * rw [dite_eq_right hpre, dite_eq_right (fun hc => hpre (hcondiff.mp hc).2)]
      * have hstep : (singleDFA sigma).step (some k) b = none := by
          simp only [singleDFA, Option.bind_some, dite_eq_left hk, ite_eq_right hchar]
        rw [hstep, singleDFA_evalFrom_none, dite_eq_right]
        intro hc
        exact hchar (hcondiff.mp hc).1.symm
    * have hstep : (singleDFA sigma).step (some k) b = none := by
        simp only [singleDFA, Option.bind_some, dite_eq_right hk]
      rw [hstep, singleDFA_evalFrom_none, dite_eq_right]
      have hd : List.drop (k : Nat) sigma = [] := by
        rw [List.drop_eq_nil_iff]; have := k.isLt; omega
      rw [hd]; exact (by simp : not  (b :: w) <+: [])
end
```

```
section
/-- **The single-word DFA recognises exactly `{sigma}`.** -/
theorem singleDFA_accepts (sigma : List Bool) : (singleDFA sigma).accepts = {sigma} :=
     by
  ext w
  show (singleDFA sigma).evalFrom (some <0, by omega>) w  in  (singleDFA sigma).accept
      <->  _
  rw [singleDFA_evalFrom]
  by_cases h : w <+: List.drop (0 : Nat) (sigma : List Bool)
  * rw [dite_eq_left h, List.drop_zero] at *
    show some (<0 + w.length, _> : Fin (sigma.length + 1))  in  ({some <sigma.length,
    _>} : Set _)  <->  w  in  ({sigma} : Set _)
    rw [Set.mem_singleton_iff, Set.mem_singleton_iff, Option.some.injEq, Fin.mk.injEq,
     Nat.zero_add]
    constructor
    * intro hlen; exact h.eq_of_length hlen
    * intro hw; subst hw; rfl
  * rw [dite_eq_right h, List.drop_zero] at *
    show (none : SingleState sigma)  in  ({some <sigma.length, _>} : Set _)  <->  w
    in  ({sigma} : Set _)
    rw [Set.mem_singleton_iff, Set.mem_singleton_iff]
    constructor
    * intro hcontra; exact absurd hcontra (by simp)
    * intro hw; subst hw; exact absurd List.prefix_rfl h
end

section
theorem singleDFA_accepts_denote (sigma : List Bool) : (singleDFA sigma).accepts =
    denote (.single sigma) := by
  rw [singleDFA_accepts]; exact (denote_single sigma).symm
end

section
/-- Every word over `{false,true}` of length at most `n`, listed without duplicates. -/
def wordsUpTo : Nat  ->  List (List Bool)
  | 0 => [[]]
  | n + 1 =>
    (wordsUpTo n) ++ (wordsUpTo n).flatMap fun w => [[false] ++ w, [true] ++ w]
end

section
/-- True iff some word in `ws` is accepted by `matchesB e`. -/
def anyMatchesB (e : SExpr) (ws : List (List Bool)) : Bool := ws.any (matchesB e)

#eval anyMatchesB .sigma (wordsUpTo 0) -- should be `true` (the empty word lies in
    Sigma)
end

section
theorem mem_wordsUpTo (n : Nat) {w : List Bool} : w  in  wordsUpTo n  <->  w.length
    <=  n := by
  induction n generalizing w with
  | zero =>
    simp only [wordsUpTo, List.mem_singleton]
    constructor
```

```
    * rintro (rfl : w = [])
      simp
    * intro h
      exact (List.eq_nil_of_length_eq_zero (Nat.eq_zero_of_le_zero h)).symm  |>  rfl
  | succ n ih =>
    simp only [wordsUpTo]
    rw [List.mem_append, List.mem_flatMap]
    constructor
    * intro h
      rcases h with h | <w', hw', hmem>
      * exact Nat.le_succ_of_le (ih.mp h)
      * rw [List.mem_cons, List.mem_singleton] at hmem
        rcases hmem with rfl | rfl
        * simpa [List.length_cons] using Nat.succ_le_succ (ih.mp hw')
        * simpa [List.length_cons] using Nat.succ_le_succ (ih.mp hw')
    * intro hlen
      by_cases hw : w.length  <=  n
      * exact Or.inl (ih.mpr hw)
      * match w with
        | [] => exfalso; exact hw (by simp)
        | hd :: tl =>
          have htl_len : tl.length  <=  n := by
            have : tl.length + 1  <=  n + 1 := by simpa [List.length_cons] using hlen
            exact Nat.le_of_succ_le_succ this
          have htl : tl  in  wordsUpTo n := ih.mpr htl_len
          refine Or.inr <tl, htl, ?_>
          rw [List.mem_cons, List.mem_singleton]
          cases hd with
          | false => exact Or.inl rfl
          | true => exact Or.inr rfl
end

end Exercise722
end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.Exercise722
section
set_option linter.unusedVariables false

theorem example_722_words : forall  (n : Nat) {w : List Bool},
  Iff
    (@Membership.mem (List Bool) (List (List Bool)) (@List.instMembership (List Bool))
      (DomainTheory.NeighbourhoodSystems.Exercise722.wordsUpTo n) w)
    (@LE.le Nat instLENat (@List.length Bool w) n) :=
  DomainTheory.NeighbourhoodSystems.Exercise722.mem_wordsUpTo
end

end DomainTheory.NeighbourhoodSystems.Exercise722
namespace DomainTheory.NeighbourhoodSystems
namespace Exercise722
open scoped Computability
open Sum Set

section
variable {sigma_1 sigma_2 : Type}
variable {sigma : Type}
variable {sigma : Type} [Fintype sigma] {M : NFA Bool sigma}
```

```
/-- State after reading the first `n` symbols along a concrete path (`n  <=  x.length`)
    . -/
def pathStateAtAux {s t : sigma} {x : List Bool} : M.Path s t x  ->  forall  n, n  <=
     x.length  ->  sigma
  | .nil s, 0, _ => s
  | .nil _, n + 1, hn => (Nat.not_succ_le_zero n hn).elim
  | .cons _ s' _ _ _ _ p', 0, _ => s'
  | .cons _ _ _ _ _ _ p', n + 1, hn =>
      pathStateAtAux p' n (Nat.le_of_succ_le_succ hn)

def pathStateAt {s t : sigma} {x : List Bool} (p : M.Path s t x) (n : Nat) (hn : n  <=
      x.length) : sigma :=
  pathStateAtAux p n hn

structure PathSplit {s t : sigma} {x : List Bool} (p : M.Path s t x) (n : Nat) (hn : n
      <=  x.length) where
  u : sigma
  hp : M.Path s u (x.take n)
  ht : M.Path u t (x.drop n)
  hu : pathStateAt p n hn = u

end

section
variable {sigma_1 sigma_2 : Type}

section NFAinterLemmas

/-- Product NFA recognising the intersection: states `sigma_1  x  sigma_2`, stepping/
    starting/accepting
componentwise. -/
def NFAinter (M_1 : NFA Bool sigma_1) (M_2 : NFA Bool sigma_2) : NFA Bool (sigma_1  x
     sigma_2) where
  step s a := {t | t.1  in  M_1.step s.1 a  /\  t.2  in  M_2.step s.2 a}
  start := {s | s.1  in  M_1.start  /\  s.2  in  M_2.start}
  accept := {s | s.1  in  M_1.accept  /\  s.2  in  M_2.accept}

end NFAinterLemmas

section NFAinterLemmas

variable (M_1 : NFA Bool sigma_1) (M_2 : NFA Bool sigma_2)

theorem NFAinter_mem_eval_iff (x : List Bool) (p : sigma_1  x  sigma_2) :
    p  in  (NFAinter M_1 M_2).eval x  <->  p.1  in  M_1.eval x  /\  p.2  in  M_2.eval
    x := by
  induction x using List.reverseRecOn generalizing p with
  | nil => exact Iff.rfl
  | append_singleton x a ih =>
    rw [NFA.eval_append_singleton, NFA.mem_stepSet]
    constructor
    * rintro <q, hq, hstep>
      rw [ih] at hq
      rw [NFA.eval_append_singleton, NFA.mem_stepSet, NFA.eval_append_singleton, NFA.
    mem_stepSet]
```

```
      exact <<q.1, hq.1, hstep.1>, <q.2, hq.2, hstep.2>>
    * rintro <h1, h2>
      rw [NFA.eval_append_singleton, NFA.mem_stepSet] at h1 h2
      obtain <q1, hq1, hs1> := h1
      obtain <q2, hq2, hs2> := h2
      exact <(q1, q2), (ih (q1, q2)).mpr <hq1, hq2>, hs1, hs2>

theorem NFAinter_mem_accepts_iff (x : List Bool) :
    x in (NFAinter M_1 M_2).accepts <-> x in M_1.accepts /\ x in M_2.accepts
     := by
  constructor
  * rintro <p, hp, hpe>
    rw [NFAinter_mem_eval_iff] at hpe
    exact <<p.1, hp.1, hpe.1>, <p.2, hp.2, hpe.2>>
  * rintro <<s, hs, hse>, <t, ht, hte>>
    exact <(s, t), <hs, ht>, (NFAinter_mem_eval_iff M_1 M_2 x (s, t)).mpr <hse, hte>>

end NFAinterLemmas

/-- State type of the automaton for an `SExpr` (a `Fintype`, see `instFintypeAutState`)
    . -/
def autState : SExpr -> Type
  | .sigma => Unit
  | .single sigma => Option (Fin (sigma.length + 1))
  | .cap a b => autState a x autState b
  | .cat a b => autState a ? autState b

/-- The NFA from a DFA tracks the single deterministic state as a singleton. -/
theorem dfaToNFA_eval {sigma : Type} (M : DFA Bool sigma) (x : List Bool) :
    M.toNFA.eval x = {M.eval x} := by
  induction x using List.reverseRecOn with
  | nil => rfl
  | append_singleton x a ih =>
    rw [NFA.eval_append_singleton, ih]
    ext s
    rw [NFA.mem_stepSet]
    constructor
    * rintro <t, ht, hs>
      rw [Set.mem_singleton_iff] at ht
      subst ht
      rw [Set.mem_singleton_iff, DFA.eval_append_singleton]
      exact hs
    * intro hs
      rw [Set.mem_singleton_iff, DFA.eval_append_singleton] at hs
      exact <M.eval x, Set.mem_singleton _, hs>

/-- Choice-free version of `DFA.toNFA_correct`. -/
theorem dfaToNFA_accepts {sigma : Type} (M : DFA Bool sigma) : M.toNFA.accepts = M.
    accepts := by
  ext x
  constructor
  * rintro <S, hS, hSe>
    rw [dfaToNFA_eval, Set.mem_singleton_iff] at hSe
    rw [DFA.mem_accepts, <- hSe]
    exact hS
  * intro hx
```

```
    rw [DFA.mem_accepts] at hx
    exact <M.eval x, hx, by rw [dfaToNFA_eval]; exact Set.mem_singleton _>

instance instFintypeAutState : (e : SExpr)  ->  Fintype (autState e)
  | .sigma => inferInstanceAs (Fintype Unit)
  | .single sigma => inferInstanceAs (Fintype (Option (Fin (sigma.length + 1))))
  | .cap a b => by
      letI := instFintypeAutState a; letI := instFintypeAutState b
      exact inferInstanceAs (Fintype (autState a  x  autState b))
  | .cat a b => by
      letI := instFintypeAutState a; letI := instFintypeAutState b
      exact inferInstanceAs (Fintype (autState a ? autState b))

instance instDecidableEqAutState : (e : SExpr)  ->  DecidableEq (autState e)
  | .sigma => inferInstanceAs (DecidableEq Unit)
  | .single sigma => inferInstanceAs (DecidableEq (Option (Fin (sigma.length + 1))))
  | .cap a b => by
      letI := instDecidableEqAutState a; letI := instDecidableEqAutState b
      exact inferInstanceAs (DecidableEq (autState a  x  autState b))
  | .cat a b => by
      letI := instDecidableEqAutState a; letI := instDecidableEqAutState b
      exact inferInstanceAs (DecidableEq (autState a ? autState b))

/-- A crude upper bound on `Fintype.card (autState e)` (used to bound word search
    length). -/
def autStateCard : SExpr  ->  Nat
  | .sigma => 1
  | .single sigma => sigma.length + 2
  | .cap a b => autStateCard a * autStateCard b
  | .cat a b => autStateCard a + autStateCard b

end

section
variable {sigma_1 sigma_2 : Type}
variable {sigma : Type}

section ShortWord

variable {sigma : Type} [Fintype sigma] {M : NFA Bool sigma}

omit [Fintype sigma] in
@[simp] theorem pathStateAt_zero {s t : sigma} {x : List Bool} (p : M.Path s t x) :
    pathStateAt p 0 (Nat.zero_le _) = s := by
  cases x with
  | nil =>
    cases p; simp [pathStateAt, pathStateAtAux]
  | cons a xs =>
    cases p with
    | cons _ s' _ _ _ _ _ => simp [pathStateAt, pathStateAtAux]

theorem accepts_card_zero (hsigma : Fintype.card sigma = 0) {w : List Bool} (hw : w
    in  M.accepts) : False := by
  rw [NFA.mem_accepts] at hw
  obtain <t, ht, hte> := hw
  rw [NFA.mem_evalFrom_iff_exists (S := M.start) (s := t) (x := w)] at hte
```

```
  obtain <s, hs, _> := hte
  exact (Fintype.card_eq_zero_iff.mp hsigma).elim s

theorem autStateCard_eq_card (e : SExpr) : autStateCard e = Fintype.card (autState e)
    := by
  induction e with
  | sigma =>
    simp [autStateCard, autState]
    exact (Fintype.card_unique (a := Unit)).symm
  | single sigma =>
    simp only [autStateCard, autState]
    have h1 : Fintype.card (Option (Fin (sigma.length + 1))) =
        Fintype.card (Fin (sigma.length + 1)) + 1 :=
      Fintype.card_option (a := Fin (sigma.length + 1))
    have h2 : Fintype.card (Fin (sigma.length + 1)) = sigma.length + 1 :=
      Fintype.card_fin (sigma.length + 1)
    calc autStateCard (.single sigma)
        = sigma.length + 2 := rfl
      _ = (sigma.length + 1) + 1 := rfl
      _ = Fintype.card (Fin (sigma.length + 1)) + 1 := (congrArg (fun n => n + 1) h2).
    symm


      _ = Fintype.card (Option (Fin (sigma.length + 1))) := h1.symm
  | cap a b ih_a ih_b =>
    simp [autStateCard, autState, ih_a, ih_b]
    exact (Fintype.card_prod (autState a) (autState b)).symm
  | cat a b ih_a ih_b =>
    simp [autStateCard, autState, ih_a, ih_b]
    exact (Fintype.card_sum (a := autState a) (b := autState b)).symm

end ShortWord

/-- Search `wordsUpTo (autStateCard e)` for a word accepted by `matchesB e`. -/
def decideNonemptyB (e : SExpr) : Bool :=
  anyMatchesB e (wordsUpTo (autStateCard e))

/-- Decidable emptiness of `denote e` via bounded word search. -/
def decideEmptyB (e : SExpr) : Bool := !decideNonemptyB e

section ShortWord

variable {sigma : Type} [Fintype sigma] {M : NFA Bool sigma}

omit [Fintype sigma] in
theorem mem_accepts_of_path {s t : sigma} {x : List Bool} (hs : s  in  M.start) (ht :
    t  in  M.accept)
    (p : M.Path s t x) : x  in  M.accepts := by
  rw [NFA.accepts_iff_exists_path]
  exact <s, hs, t, ht, <p>>

/-- Append two paths meeting at the middle state. -/
def pathAppend {s u t : sigma} {a b : List Bool} (p : M.Path s u a) (p' : M.Path u t b
    ) :
    M.Path s t (a ++ b) :=
  match p with
  | .nil _ => p'
```

```
  | .cons sMid s' _ a' xs hstep p'' =>
      .cons sMid s' _ a' (xs ++ b) hstep (pathAppend p'' p')

omit [Fintype sigma] in
theorem pathStateAt_succ {sMid s' u : sigma} {a : Bool} {xs : List Bool}
    (hstep : sMid  in  M.step s' a) (p' : M.Path sMid u xs) (n : Nat) (hn : n  <=  xs.
    length) :
    pathStateAt (.cons sMid s' u a xs hstep p') (n + 1) (Nat.succ_le_succ hn) =
      pathStateAt p' n hn := by
  simp [pathStateAt, pathStateAtAux]

noncomputable def pathAppend_take_drop {s t : sigma} {x : List Bool} (p : M.Path s t x
    ) (n : Nat)
    (hn : n  <=  x.length) : PathSplit p n hn := by
  revert n hn
  induction p with
  | nil s =>
    intro n hn
    have hn0 : n = 0 := Nat.eq_zero_of_le_zero hn
    subst hn0
    exact { u := s, hp := .nil s, ht := .nil s, hu := by rfl }
  | cons sMid s' u a xs hstep p' ih =>
    intro n hn
    cases n with
    | zero =>
      exact { u := s', hp := .nil s', ht := .cons sMid s' u a xs hstep p', hu := by
    rfl }
    | succ n' =>
      have hn' : n'  <=  xs.length := by simpa [List.length_cons] using hn
      have split := ih n' hn'
      exact {
        u := split.u
        hp := .cons sMid s' split.u a (xs.take n') hstep split.hp
        ht := split.ht
        hu := by rw [pathStateAt_succ hstep p' n' hn']; exact split.hu
      }

omit [Fintype sigma] in
theorem accepts_skip_loop {s t : sigma} {x : List Bool} {i j : Nat}
    (hi : i  <=  x.length) (hj : j  <=  x.length) (_hij : i < j)
    (hs : s  in  M.start) (ht : t  in  M.accept) (hp : M.Path s t x)
    (heq : pathStateAt hp i hi = pathStateAt hp j hj) :
    x.take i ++ x.drop j  in  M.accepts := by
  let splitI := pathAppend_take_drop hp i hi
  let splitJ := pathAppend_take_drop hp j hj
  have hqq' : splitI.u = splitJ.u := by rw [ <-  splitI.hu, heq, splitJ.hu]
  exact mem_accepts_of_path hs ht (pathAppend splitI.hp (hqq'  |>  splitJ.ht))

theorem accepts_shorten_step {x : List Bool} (hx : x  in  M.accepts)
    (hlen : Fintype.card sigma  <=  x.length) :
    exists  y, y  in  M.accepts  /\  y.length < x.length := by
  rw [NFA.accepts_iff_exists_path] at hx
  obtain <s, hs, t, ht, <hp>> := hx
  obtain <i, j, hne, heq> :=
    Fintype.exists_ne_map_eq_of_card_lt
      (f := fun k : Fin (x.length + 1) => pathStateAt hp k.val (Nat.le_of_lt_succ k.
```

```
    isLt))
      (by rw [Fintype.card_fin]; exact Nat.lt_succ_iff.mpr hlen)
  have hval : i.val  !=  j.val := fun h => hne (Fin.ext h)
  rcases Nat.lt_or_gt_of_ne hval with hij | hij
  * refine <x.take i.val ++ x.drop j.val, ?_, ?_>
    * exact accepts_skip_loop (by omega) (by omega) hij hs ht hp heq
    * have : j.val - i.val > 0 := Nat.sub_pos_of_lt hij
      simp [List.length_append, List.length_take, List.length_drop]
      omega
  * refine <x.take j.val ++ x.drop i.val, ?_, ?_>
    * exact accepts_skip_loop (by omega) (by omega) hij hs ht hp heq.symm
    * have : i.val - j.val > 0 := Nat.sub_pos_of_lt hij
      simp [List.length_append, List.length_take, List.length_drop]
      omega

theorem exists_accepted_word_short (h : M.accepts.Nonempty) :
    exists  w, w.length < Fintype.card sigma  /\  w  in  M.accepts := by
  by_cases hsigma : Fintype.card sigma = 0
  * obtain <x, hx> := h
    exact (accepts_card_zero hsigma hx).elim
  * obtain <x, hx> := h
    exact Nat.strongRecOn x.length
      (motive := fun n =>
        forall  w, w.length = n  ->  w  in  M.accepts  ->  exists  u, u.length <
    Fintype.card sigma  /\  u  in  M.accepts)
      (fun n ih w hwlen hw => by
        by_cases hlt : n < Fintype.card sigma
        * exact <w, hwlen  |>  hlt, hw>
        * have hge : Fintype.card sigma  <=  n := Nat.not_lt.mp hlt
          obtain <y, hy, hylt> := accepts_shorten_step hw (hwlen.symm  |>  hge)
          exact ih y.length (Nat.lt_of_lt_of_eq hylt hwlen) y rfl hy)
      x rfl hx

end ShortWord

end

section
variable {sigma_1 sigma_2 : Type}

/-- The uniform automaton for a fragment expression: leaves from `Exercise722DFA.lean`,
    `cap` by the
product construction, `cat` by the concatenation automaton (`Exercise722Cat.lean`). -/
def toNFA : (e : SExpr)  ->  NFA Bool (autState e)
  | .sigma => sigmaDFA.toNFA
  | .single sigma => (singleDFA sigma).toNFA
  | .cap a b => NFAinter (toNFA a) (toNFA b)
  | .cat a b => (catEps (toNFA a) (toNFA b)).toNFA

/-- **The automaton recognises exactly the language it should.** -/
theorem toNFA_accepts : (e : SExpr)  ->  (toNFA e).accepts = denote e
  | .sigma => by
      change sigmaDFA.toNFA.accepts = denote .sigma
      rw [dfaToNFA_accepts sigmaDFA, sigmaDFA_accepts, denote_sigma]
  | .single sigma => by
      change (singleDFA sigma).toNFA.accepts = denote (.single sigma)
```

```
      rw [dfaToNFA_accepts (singleDFA sigma)]
      exact singleDFA_accepts_denote sigma
  | .cap a b => by
      ext x
      change x  in  (NFAinter (toNFA a) (toNFA b)).accepts  <->  _
      rw [NFAinter_mem_accepts_iff, toNFA_accepts a, toNFA_accepts b, denote_cap]
      exact (Set.mem_inter_iff x (denote a) (denote b)).symm
  | .cat a b => by
      change (catEps (toNFA a) (toNFA b)).toNFA.accepts = denote (.cat a b)
      rw [?NFA.toNFA_correct, catEps_accepts, toNFA_accepts a, toNFA_accepts b,
    denote_cat]

end

section
variable {sigma_1 sigma_2 : Type}
variable {sigma : Type}

section ShortWord

variable {sigma : Type} [Fintype sigma] {M : NFA Bool sigma}

theorem denote_nonempty_iff_short (e : SExpr) :
    (denote e).Nonempty  <->  exists  w  in  wordsUpTo (autStateCard e), matchesB e w
    = true := by
  have hc : autStateCard e = Fintype.card (autState e) := autStateCard_eq_card e
  constructor
  * intro hne
    rw [ <-  toNFA_accepts e] at hne
    obtain <w, hwlt, hw> := exists_accepted_word_short hne
    refine <w, ?_, ?_>
    * rw [mem_wordsUpTo, hc]; omega
    * rw [matchesB_iff,  <-  toNFA_accepts e]; exact hw
  * intro <w, hwmem, hmatch>
    rw [mem_wordsUpTo, hc] at hwmem
    exact <w, (matchesB_iff e w).mp hmatch>

end ShortWord

@[simp] theorem decideNonemptyB_iff (e : SExpr) :
    decideNonemptyB e = true  <->  (denote e).Nonempty := by
  simp [decideNonemptyB, anyMatchesB, List.any_eq_true, denote_nonempty_iff_short]

@[simp] theorem decideEmptyB_iff (e : SExpr) :
    decideEmptyB e = true  <->  denote e = {} := by
  rw [decideEmptyB, Bool.not_eq_true_eq_eq_false,  <-  Bool.eq_false_eq_not_eq_true,
    decideNonemptyB_iff, Set.not_nonempty_iff_eq_empty]

instance decidableEmptyDenote (e : SExpr) : Decidable (denote e = {}) :=
  if h : decideEmptyB e then .isTrue (decideEmptyB_iff e |>.mp h)
  else .isFalse fun he => h (decideEmptyB_iff e |>.mpr he)

#eval decideEmptyB (.cap (.single [false]) (.single [true])) -- should be `true` (
    disjoint singletons)

/-- Decides relation (ii) on a pair of expressions: `true` iff `denote a  I  denote b`
```

```
    is non-empty. -/
def consistentB (a b : SExpr) : Bool := !decideEmptyB (.cap a b)

end

section
variable {sigma_1 sigma_2 : Type}

example : DecidableEq (autState (.sigma : SExpr)) := inferInstance
example : DecidableEq (autState (.single [true, false] : SExpr)) := inferInstance

variable {sigma : Type}

@[simp] theorem consistentB_iff (a b : SExpr) :
    consistentB a b = true  <->  (denote (.cap a b)).Nonempty := by
  have h : consistentB a b = decideNonemptyB (.cap a b) := by
    simp [consistentB, decideEmptyB, decideNonemptyB]
  rw [h, decideNonemptyB_iff]

@[simp] theorem capNonempty_iff_consistent (a b : SExpr) :
    (denote a  I  denote b).Nonempty  <->  consistentB a b = true := by
  rw [ <-  denote_cap, consistentB_iff]
end

section
variable {sigma_1 sigma_2 : Type}

theorem autStateCard_le_card (e : SExpr) : autStateCard e  <=  Fintype.card (autState
    e) := by
  induction e with
  | sigma =>
    simp [autStateCard, autState]
    exact (Fintype.card_unique (a := Unit)).ge
  | single sigma =>
    have heq : autStateCard (.single sigma) = Fintype.card (autState (.single sigma))
    := by
      simp only [autStateCard, autState]
      have h1 : Fintype.card (Option (Fin (sigma.length + 1))) =
          Fintype.card (Fin (sigma.length + 1)) + 1 :=
        Fintype.card_option (a := Fin (sigma.length + 1))
      have h2 : Fintype.card (Fin (sigma.length + 1)) = sigma.length + 1 :=
        Fintype.card_fin (sigma.length + 1)
      calc sigma.length + 2
          = (sigma.length + 1) + 1 := rfl
        _ = Fintype.card (Fin (sigma.length + 1)) + 1 := (congrArg (fun n => n + 1) h2
    ).symm
        _ = Fintype.card (Option (Fin (sigma.length + 1))) := h1.symm
    exact Nat.le_of_eq heq
  | cap a b ih_a ih_b =>
    simp only [autStateCard, autState]
    exact (Nat.mul_le_mul ih_a ih_b).trans (Fintype.card_prod (autState a) (autState b
    )).ge
  | cat a b ih_a ih_b =>
    simp only [autStateCard, autState]
    exact (Nat.add_le_add ih_a ih_b).trans (Fintype.card_sum (a := autState a) (b :=
    autState b)).ge
```

```
end

end Exercise722
end DomainTheory.NeighbourhoodSystems
end
```

3.8.25 Exercise 7.23 — graph and fun computable

Complete the discussion of $P\mathbf{N}$ of Example 7.8. Show that the combinators fun and graph of Exercise 5.14 are computable. Also do the same for

$$\lambda x, y.\, x \cap y, \quad \lambda x, y.\, x \cup y, \quad \text{and } \lambda x, y.\, x + y,$$

where for $x, y \in P\mathbf{N}$ we define

$$x + y = \{n + m \mid n \in x \text{ and } m \in y\}.$$

What are the computable elements of $P\mathbf{N}$?

Exercise_7_23.lean

```
module

public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.Exercise_5_14

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Theorem_7_5
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Proposition_7_12
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Example_7_8

public import Mathlib.Data.Nat.Bitwise

@[expose] public section

section
/- concordance: source L4727 EXERCISE 7.23 graph and fun computable -/
-- Exercise723/capMap.lean
namespace DomainTheory.NeighbourhoodSystems.Exercise723

open DomainTheory.NeighbourhoodSystems Example78 NeighborhoodSystem Domain.Recursive

open DomainTheory.NeighbourhoodSystems.Exercise513 (tri num numP unnum nextCell
    numP_unnum)

open DomainTheory.NeighbourhoodSystems.Exercise514 (tag tag_nil tag_cons Fun Fun_mono
    entries
  num_succ_left_gt tag_injective)

/-- **`fun x,y. x I y` as an approximable map** `PN  x  PN  ->  PN`. On neighbourhoods:
     `X  U  Y` relates to
`Z` iff `X  subseteq  Z` and `Y  subseteq  Z` (the "test at the minimal witness" idiom:
     `X`, `Y` are themselves the
tightest possible arguments consistent with the input information). -/
```

```
def capMap : ApproximableMap (prod PN PN) PN where
  rel W Z := exists  X Y, W = prodNbhd X Y  /\  PN.mem X  /\  PN.mem Y  /\  PN.mem Z
    /\  X  subseteq  Z  /\  Y  subseteq  Z
  rel_dom := by rintro W Z <X, Y, rfl, hX, hY, _, _, _>; exact prod_mem_prodNbhd hX hY
  rel_cod := by rintro W Z <X, Y, rfl, _, _, hZ, _, _>; exact hZ
  master_rel := <Set.univ, Set.univ, rfl, PN.master_mem, PN.master_mem, PN.master_mem,
    Set.subset_univ _, Set.subset_univ _>
  inter_right := by
    rintro W Z Z' <X, Y, rfl, hX, hY, hZ, hXZ, hYZ> <X', Y', heq, _, _, hZ', hXZ', hYZ
    '>
    obtain <rfl, rfl> := prodNbhd_injective heq
    obtain <k, hk> := hZ; obtain <k', hk'> := hZ'; subst hk; subst hk'
    exact <X, Y, rfl, hX, hY, <myLor k k', nbhd_inter k k'>,
      Set.subset_inter hXZ hXZ', Set.subset_inter hYZ hYZ'>
  mono := by
    rintro W W' Z Z' <X, Y, rfl, hX, hY, hZ, hXZ, hYZ> hW'W hZZ' hW' hZ'
    obtain <X', Y', hX', hY', rfl> := hW'
    obtain <hXX', hYY'> := prodNbhd_subset_iff.mp hW'W
    exact <X', Y', rfl, hX', hY', hZ', hXX'.trans (hXZ.trans hZZ'), hYY'.trans (hYZ.
    trans hZZ')>

end DomainTheory.NeighbourhoodSystems.Exercise723
end

section
namespace DomainTheory.NeighbourhoodSystems.Exercise723
open DomainTheory.NeighbourhoodSystems Example78 NeighborhoodSystem Domain.Recursive
open DomainTheory.NeighbourhoodSystems.Exercise513 (tri num numP unnum nextCell
    numP_unnum)
open DomainTheory.NeighbourhoodSystems.Exercise514 (tag tag_nil tag_cons Fun Fun_mono
    entries
  num_succ_left_gt tag_injective)

section
/-- Iterated halving `(*/2)^[k] n`, the primitive-recursive core of `Nat.testBit`. -/
def halfIter (n k : Nat) : Nat := (fun x : Nat => x / 2)^[k] n
end

section
@[simp] theorem halfIter_zero (n : Nat) : halfIter n 0 = n := rfl
end

section
/-- The `{0,1}`-valued primitive-recursive proxy for `Nat.testBit`. -/
def bitAt (n k : Nat) : Nat := halfIter n k % 2
end

section
/-- **`fun x,y. x U y` as an approximable map** `PN  x  PN  ->  PN`. On neighbourhoods:
     `X  U  Y` relates to
`Z` iff `nbhd(myLor n m)  subseteq  Z` -- the excluded set of `Z` must fit inside the *
    union* `E_n  U  E?`, i.e.
`nbhd n  I  nbhd m  subseteq  Z` (`E_n  U  E? = E_{myLor n m}`, `nbhd_inter`). -/
def cupMap : ApproximableMap (prod PN PN) PN where
  rel W Z := exists  X Y, W = prodNbhd X Y  /\  PN.mem X  /\  PN.mem Y  /\  PN.mem Z
    /\  X  I  Y  subseteq  Z
```

```
  rel_dom := by rintro W Z <X, Y, rfl, hX, hY, _, _>; exact prod_mem_prodNbhd hX hY
  rel_cod := by rintro W Z <X, Y, rfl, _, _, hZ, _>; exact hZ
  master_rel := <Set.univ, Set.univ, rfl, PN.master_mem, PN.master_mem, PN.master_mem,
    Set.inter_subset_left.trans (Set.subset_univ _)>
  inter_right := by
    rintro W Z Z' <X, Y, rfl, hX, hY, hZ, hXYZ> <X', Y', heq, _, _, hZ', hXYZ'>
    obtain <rfl, rfl> := prodNbhd_injective heq
    obtain <k, hk> := hZ; obtain <k', hk'> := hZ'; subst hk; subst hk'
    exact <X, Y, rfl, hX, hY, <myLor k k', nbhd_inter k k'>, Set.subset_inter hXYZ
    hXYZ'>
  mono := by
    rintro W W' Z Z' <X, Y, rfl, hX, hY, hZ, hXYZ> hW'W hZZ' hW' hZ'
    obtain <X', Y', hX', hY', rfl> := hW'
    obtain <hXX', hYY'> := prodNbhd_subset_iff.mp hW'W
    exact <X', Y', rfl, hX', hY', hZ',
      (Set.inter_subset_inter hXX' hYY').trans (hXYZ.trans hZZ')>
end

section
/-- Scott's `x + y = {n+m ? n  in  x, m  in  y}` (Minkowski sum) for `x, y  subseteq
    Nat`. -/
def sumSet (X Y : Set Nat) : Set Nat := {k | exists  a  in  X, exists  b  in  Y, a + b
     = k}
end

section
@[inherit_doc] infixl:65 " +? " => sumSet
end

section
/-- **Choice-free De Morgan for `nbhd`'s excluded sets.** The general `Set.compl_inter`
     needs
excluded middle; here, since membership in `nbhd a`/`nbhd b` is decided by `Nat.testBit
    `, a case
split on the two (concrete) bits suffices. -/
theorem compl_inter_nbhd (a b : Nat) : (nbhd a  I  nbhd b)? = (nbhd a)?  U  (nbhd b)?
    := by
  ext x
  simp only [Set.mem_compl_iff, Set.mem_inter_iff, Set.mem_union, mem_nbhd]
  cases a.testBit x <;> cases b.testBit x <;> decide
end

section
/-- **Choice-free antitone `compl`** (only the direction needed here): unlike the
    general
`Set.compl_subset_compl` (a `BooleanAlgebra` lemma whose ` <-> ` proof is classical),
    this direction is
constructive contraposition and needs no excluded middle. -/
theorem compl_subset_compl_of_subset {X Y : Set Nat} (h : X  subseteq  Y) : Y?
    subseteq  X? :=
  fun _ hx hxX => hx (h hxX)
end

section
/-- The subset of `Nat` "positively described" by a `PN`-element `x`: the union of the
    excluded finite
```

```
sets of every neighbourhood it contains. -/
def elemSet (x : PN.Element) : Set Nat := {k | exists  n, x.mem (nbhd n)  /\  n.
    testBit k = true}
end

section
@[simp] theorem mem_elemSet {x : PN.Element} {k : Nat} :
    k  in  elemSet x  <->  exists  n, x.mem (nbhd n)  /\  n.testBit k = true := Iff.
    rfl
end

section
/-- `bitsCode n N` codes the list of `k < N` with `n.testBit k = true`. -/
def bitsCode : Nat  ->  Nat  ->  Nat
  | _, 0 => 0
  | n, N + 1 => selectFn (bitAt n N) (Nat.pair N (bitsCode n N) + 1) (bitsCode n N)
end

section
/-- The (non-primitive-recursive) list of set-bit positions of `n`: every set bit `k`
    of `n`
satisfies `k < n` (`lt_of_testBit_true`), so ranging over `List.range n` catches them
    all. Used only
to state/prove the pure covering fact `nbhd_mem_iff_subset_elemSet`; see `bitsCode`
    below for the
primitive-recursive analogue used in the computability direction. -/
def bitsList (n : Nat) : List Nat := (List.range n).filter (fun k => n.testBit k)
end

section
/-- The primitive-recursive step function for `bitsCode`'s iteration: `(n, N, acc)  |->
(n, N+1, if n.testBit N then pair N acc + 1 else acc)`. -/
def bitsStep (s : Nat) : Nat :=
  Nat.pair s.unpair.1
    (Nat.pair (s.unpair.2.unpair.1 + 1)
      (selectFn (bitAt s.unpair.1 s.unpair.2.unpair.1)
        (Nat.pair s.unpair.2.unpair.1 s.unpair.2.unpair.2 + 1) s.unpair.2.unpair.2))
end

section
/-- **`fun x,y. Fun x y` as an approximable map** `PN  x  PN  ->  PN`, built by the
    same "reversal" idiom as
`plusMap`: `Z?  subseteq  Fun X? Y?`. `funMap` (the actual `fun` combinator of Exercise
     5.14, a map
`PN  ->  (PN  -> -vec PN)`) is `curry gMap` (`Theorem75.curry`). -/
def gMap : ApproximableMap (prod PN PN) PN where
  rel W Z := exists  X Y, W = prodNbhd X Y  /\  PN.mem X  /\  PN.mem Y  /\  PN.mem Z
    /\  Z?  subseteq  Fun X? Y?
  rel_dom := by rintro W Z <X, Y, rfl, hX, hY, _, _>; exact prod_mem_prodNbhd hX hY
  rel_cod := by rintro W Z <X, Y, rfl, _, _, hZ, _>; exact hZ
  master_rel := <Set.univ, Set.univ, rfl, PN.master_mem, PN.master_mem, PN.master_mem,
    fun x hx => absurd (Set.mem_univ x) hx>
  inter_right := by
    rintro W Z Z' <X, Y, rfl, hX, hY, hZ, hsub> <X', Y', heq, _, _, hZ', hsub'>
    obtain <rfl, rfl> := prodNbhd_injective heq
    obtain <k, hk> := hZ; obtain <k', hk'> := hZ'; subst hk; subst hk'
```

```
    refine <X, Y, rfl, hX, hY, <myLor k k', nbhd_inter k k'>, ?_>
    rw [compl_inter_nbhd]
    exact Set.union_subset hsub hsub'
  mono := by
    rintro W W' Z Z' <X, Y, rfl, hX, hY, hZ, hsub> hW'W hZZ' hW' hZ'
    obtain <X', Y', hX', hY', rfl> := hW'
    obtain <hXX', hYY'> := prodNbhd_subset_iff.mp hW'W
    refine <X', Y', rfl, hX', hY', hZ', ?_>
    calc Z'?  subseteq  Z? := compl_subset_compl_of_subset hZZ'
      _  subseteq  Fun X? Y? := hsub
      _  subseteq  Fun X'? Y'? :=
        Fun_mono (compl_subset_compl_of_subset hXX') (compl_subset_compl_of_subset hYY
    ')
end

section
/-- **`fun` as an approximable map** `PN  ->  (PN  ->  PN)`, obtained by currying `gMap
    `. -/
def funMap : ApproximableMap PN (funSpace PN PN) := curry gMap
end

section
/-- Pair-coded version of `nextCell : Nat  x  Nat  ->  Nat  x  Nat` (`(0, m)  |->  (m
    +1, 0)`, `(n+1, m)  |->  (n, m+1)`). -/
def nextCellStep (w : Nat) : Nat :=
  selectFn (isZero w.unpair.1) (Nat.pair (w.unpair.2 + 1) 0)
    (Nat.pair (w.unpair.1 - 1) (w.unpair.2 + 1))
end

section
/-- **Reference spec for `plusIdx`.** The running bitwise-OR of `m <<< a` over set bits
     `a < N` of
`n`, built by ordinary (non-primitive-recursive) structural recursion -- used only to
    state and prove
correctness of the primitive-recursive `plusIdx` below. -/
def orUpTo (n m : Nat) : Nat  ->  Nat
  | 0 => 0
  | k + 1 => myLor (orUpTo n m k) (selectFn (bitAt n k) (m <<< k) 0)
end

section
/-- **`plusIdx n m`: the primitive-recursive index of `E_n + E?`.** -/
def plusIdx (n m : Nat) : Nat := orUpTo n m n
end

section
/-- The primitive-recursive step function for `orUpTo`'s bitwise-OR fold: `(n, m, a,
    acc)  |->
(n, m, a+1, acc ||| (if testBit n a then m<<<a else 0))`. -/
def plusStep (s : Nat) : Nat :=
  Nat.pair (Nat.pair s.unpair.1.unpair.1 s.unpair.1.unpair.2)
    (Nat.pair (s.unpair.2.unpair.1 + 1)
      (myLor s.unpair.2.unpair.2
        (selectFn (bitAt s.unpair.1.unpair.1 s.unpair.2.unpair.1)
          (s.unpair.1.unpair.2 <<< s.unpair.2.unpair.1) 0)))
end
```

```
section
theorem sumSet_mono {X X' Y Y' : Set Nat} (hX : X  subseteq  X') (hY : Y  subseteq  Y
    ') : X +? Y  subseteq  X' +? Y' := by
  rintro k <a, ha, b, hb, rfl>
  exact <a, hX ha, b, hY hb, rfl>
end

section
/-- **`fun x,y. x+y` as an approximable map** `PN  x  PN  ->  PN`. On neighbourhoods: `
    X  U  Y` relates to `Z`
iff `Z?  subseteq  X? +? Y?` -- the excluded set of the output must fit inside the *
    Minkowski sum* of the
inputs' excluded sets, the most that is guaranteed knowing only `E_n  subseteq  Sx` and
     `E?  subseteq  Sy`. -/
def plusMap : ApproximableMap (prod PN PN) PN where
  rel W Z := exists  X Y, W = prodNbhd X Y  /\  PN.mem X  /\  PN.mem Y  /\  PN.mem Z
    /\  Z?  subseteq  X? +? Y?
  rel_dom := by rintro W Z <X, Y, rfl, hX, hY, _, _>; exact prod_mem_prodNbhd hX hY
  rel_cod := by rintro W Z <X, Y, rfl, _, _, hZ, _>; exact hZ
  master_rel := <Set.univ, Set.univ, rfl, PN.master_mem, PN.master_mem, PN.master_mem,
    fun x hx => absurd (Set.mem_univ x) hx>
  inter_right := by
    rintro W Z Z' <X, Y, rfl, hX, hY, hZ, hsub> <X', Y', heq, _, _, hZ', hsub'>
    obtain <rfl, rfl> := prodNbhd_injective heq
    obtain <k, hk> := hZ; obtain <k', hk'> := hZ'; subst hk; subst hk'
    refine <X, Y, rfl, hX, hY, <myLor k k', nbhd_inter k k'>, ?_>
    rw [compl_inter_nbhd]
    exact Set.union_subset hsub hsub'
  mono := by
    rintro W W' Z Z' <X, Y, rfl, hX, hY, hZ, hsub> hW'W hZZ' hW' hZ'
    obtain <X', Y', hX', hY', rfl> := hW'
    obtain <hXX', hYY'> := prodNbhd_subset_iff.mp hW'W
    refine <X', Y', rfl, hX', hY', hZ', ?_>
    calc Z'?  subseteq  Z? := compl_subset_compl_of_subset hZZ'
      _  subseteq  X? +? Y? := hsub
      _  subseteq  X'? +? Y'? :=
        sumSet_mono (compl_subset_compl_of_subset hXX') (compl_subset_compl_of_subset
    hYY')
end

section
/-- `foldCode`-shaped step for `tagCode`: `(acc, x)  |->  num (x+1) acc` (the `params`
    slot is unused). -/
def tagStep (w : Nat) : Nat := num (w.unpair.1 + 1) w.unpair.2.unpair.1
end

section
/-- **`tagCode code m` codes `tag (decodeList code) m`.** Built via `appendCode 0 code`
     (which
reverses `decodeList code`) so that `foldCode`'s left fold computes `tag`'s right fold.
     -/
def tagCode (code m : Nat) : Nat := foldCode tagStep 0 (num 0 m) (appendCode 0 code)
end

section
```

```
/-- `tagOfBits j m` codes `tag (decodeList (bitsCode j j)) m`, primitive recursive in
    `(j, m)`. -/
def tagOfBits (j m : Nat) : Nat := tagCode (bitsCode j j) m
end

section
/-- `GraphIdx W`: the domain-theoretic "positive content" of `graph(W)`, mirroring
    Exercise 5.14's
`Graph f = {tag ns m ? m  in  f (entries ns)}`, but phrased at the level of
    neighbourhood relations and
tested against *every* map `f  in  W` (Scott's "guaranteed regardless of which `f  in
    W`" idiom, as in
`eval`'s `forall  f  in  F, f.rel X Y`). The "argument" is canonically `tagOfBits n m =
     tag (decodeList
(bitsCode n n)) m` (`decodeList (bitsCode n n)` has entries exactly `E_n`, `
    mem_bitsCodeList`), so it
ranges over the countably many finite sets `(nbhd n)?` rather than over arbitrary lists
     -- exactly the
finite-set arguments `Graph` needs, up to the reindexing already used throughout this
    file. Using the
*primitive-recursive* list `bitsCode n n` (rather than the merely well-founded `
    bitsList n`) is what
makes the decode direction (`mem_GraphIdx_iff` below) computable. -/
def GraphIdx (W : Set (ApproximableMap PN PN)) : Set Nat :=
  {c | exists  n m_0 m, c = tagOfBits n m  /\  (forall  f  in  W, f.rel (nbhd n) (nbhd
     m_0))  /\  m_0.testBit m = true}
end

section
/-- **`GraphIdx` is antitone: shrinking `W` (more information about the map) can only
    add
guaranteed content.** -/
theorem GraphIdx_mono {W W' : Set (ApproximableMap PN PN)} (h : W'  subseteq  W) :
    GraphIdx W  subseteq  GraphIdx W' := by
  rintro c <n, m_0, m, rfl, hf, hbit>
  exact <n, m_0, m, rfl, fun f hf' => hf f (h hf'), hbit>
end

section
/-- **`graph` as an approximable map** `(PN  ->  PN)  ->  PN`, dually to `gMap`'s "
    reversal idiom":
`Z?  subseteq  GraphIdx W`. -/
def graphMap : ApproximableMap (funSpace PN PN) PN where
  rel W Z := (funSpace PN PN).mem W  /\  PN.mem Z  /\  Z?  subseteq  GraphIdx W
  rel_dom := fun <h, _, _> => h
  rel_cod := fun <_, h, _> => h
  master_rel := <(funSpace PN PN).master_mem, PN.master_mem, by
    show (Set.univ : Set Nat)?  subseteq  GraphIdx Set.univ
    intro x hx
    exact absurd (Set.mem_univ x) hx>
  inter_right := by
    rintro W Z Z' <hW, hZ, hsub> <_, hZ', hsub'>
    obtain <k, hk> := hZ; obtain <k', hk'> := hZ'; subst hk; subst hk'
    refine <hW, <myLor k k', nbhd_inter k k'>, ?_>
    rw [compl_inter_nbhd]
    exact Set.union_subset hsub hsub'
```

```
  mono := by
    rintro W W' Z Z' <hW, hZ, hsub> hW'W hZZ' hW' hZ'
    obtain <k', rfl> := hZ'
    refine <hW', <k', rfl>, ?_>
    calc (nbhd k')?  subseteq  Z? := compl_subset_compl_of_subset hZZ'
      _  subseteq  GraphIdx W := hsub
      _  subseteq  GraphIdx W' := GraphIdx_mono hW'W
end

section
/-- `nextCellStep` iterated from `(0,0)`, coding `unnum v` as a single `Nat.pair`. -/
def unnumPair (v : Nat) : Nat := nextCellStep^[v] (Nat.pair 0 0)
end

section
/-- The first component of `unnum v`, primitive recursive. -/
def unnum1 (v : Nat) : Nat := (unnumPair v).unpair.1
end

section
/-- The second component of `unnum v`, primitive recursive. -/
def unnum2 (v : Nat) : Nat := (unnumPair v).unpair.2
end

section
/-- `untagRef c = (ns, m)` with `tag ns m = c` (`tag_untagRef`). -/
def untagRef (c : Nat) : List Nat  x  Nat :=
  if _h : (unnum c).1 = 0 then ([], (unnum c).2)
  else
    have hlt : (unnum c).2 < c := by
      have heq : num (unnum c).1 (unnum c).2 = c := by
        have h := numP_unnum c
        simpa [numP] using h
      obtain <k, hk> : exists  k, (unnum c).1 = k + 1 := <(unnum c).1 - 1, by omega>
      rw [hk] at heq
      have hb := num_succ_left_gt k (unnum c).2
      omega
    (((unnum c).1 - 1) :: (untagRef (unnum c).2).1, (untagRef (unnum c).2).2)
termination_by c
decreasing_by all_goals exact hlt
end

section
/-- The state-transition step: no-op once `done`; otherwise peel one `unnum` layer. -/
def untagStep (w : Nat) : Nat :=
  selectFn w.unpair.2.unpair.1 w
    (selectFn (isZero (unnum1 w.unpair.1))
      (Nat.pair (unnum2 w.unpair.1)
        (Nat.pair 1 (Nat.pair w.unpair.2.unpair.2.unpair.1 (unnum2 w.unpair.1))))
      (Nat.pair (unnum2 w.unpair.1)
        (Nat.pair 0
          (Nat.pair (Nat.pair (unnum1 w.unpair.1 - 1) w.unpair.2.unpair.2.unpair.1 +
   1)
            w.unpair.2.unpair.2.unpair.2))))
end
```

```
section
/-- `untagStep` iterated `c + 1` times from the fresh state `(c, 0, 0, 0)`. -/
def untagState (c : Nat) : Nat := untagStep^[c + 1] (Nat.pair c (Nat.pair 0 (Nat.pair
    0 0)))
end

section
/-- **`untagList c`**: the (`appendCode`-reversed) code of `(untagRef c).1`, primitive
    recursive. -/
def untagList (c : Nat) : Nat := appendCode 0 (untagState c).unpair.2.unpair.2.unpair
    .1
end

section
/-- **`untagVal c`**: the value component of `untagRef c`, primitive recursive. -/
def untagVal (c : Nat) : Nat := (untagState c).unpair.2.unpair.2.unpair.2
end

section
/-- **Every set bit lies below the number itself.** If `n.testBit i = true` then `i < n
    ` (since
`n  >=  2^i > i`). -/
theorem lt_of_testBit_true {n i : Nat} (h : n.testBit i = true) : i < n := by
  by_contra hle
  have hle' : n  <=  i := by omega
  have hlt : n < 2 ^ i := lt_of_le_of_lt hle' Nat.lt_two_pow_self
  rw [Nat.testBit_eq_false_of_lt hlt] at h
  exact absurd h (by decide)

theorem halfIter_succ (n k : Nat) : halfIter n (k + 1) = halfIter (n / 2) k := by
  unfold halfIter; rw [Function.iterate_succ_apply]

theorem primrec_halfIter : Nat.Primrec (fun t => halfIter t.unpair.1 t.unpair.2) := by
  have hstep : Nat.Primrec (fun w : Nat => w.unpair.2.unpair.2 / 2) :=
    primrec_div2.comp (Nat.Primrec.right.comp Nat.Primrec.right)
  refine (Nat.Primrec.prec primrec_id hstep).of_eq (fun t => ?_)
  simp only [Nat.unpaired, unpair_pair_snd]
  exact rec_const_iterate (fun x => x / 2) t.unpair.1 t.unpair.2


theorem primrec_bitAt : Nat.Primrec (fun t => bitAt t.unpair.1 t.unpair.2) :=
  primrec_mod2.comp primrec_halfIter

theorem bitAt_le_one (n k : Nat) : bitAt n k  <=  1 := by unfold bitAt; omega

/-- **Correctness of `bitAt`.** `bitAt n k = 1  <->  n.testBit k = true`. -/
theorem bitAt_eq_one_iff (n k : Nat) : bitAt n k = 1  <->  n.testBit k = true := by
  induction k generalizing n with
  | zero =>
    show n % 2 = 1  <->  n.testBit 0 = true
    rw [Nat.testBit_zero, decide_eq_true_iff]
  | succ k ih =>
    show halfIter n (k + 1) % 2 = 1  <->  n.testBit (k + 1) = true
    rw [halfIter_succ, Nat.testBit_add_one]
    exact ih (n / 2)

/-- The excluded set of `nbhd n`, as a set of naturals: `(nbhd n)? = {k ? n.testBit k
```

```
    }`. -/
theorem compl_nbhd (n : Nat) : (nbhd n)? = {k | n.testBit k = true} := by
  ext k; simp [nbhd]

/-- **Finite covering (choice-free, structural induction on the witness list).** If
    every `k` in a
list `L` has *some* neighbourhood `nbhd m` of `x` with `m.testBit k`, a single `nbhd m_0
      in  x` works for
all of `L` at once: combine the per-entry witnesses with `myLor` via `x.inter_mem`, one
     list entry at
a time. -/
theorem exists_combined_witness (x : PN.Element) :
    forall  L : List Nat, (forall  k  in  L, exists  m, x.mem (nbhd m)  /\  m.testBit
    k = true)  ->
      exists  m_0, x.mem (nbhd m_0)  /\  forall  k  in  L, m_0.testBit k = true
  | [] => fun _ => <0, nbhd_zero  |>  x.master_mem, by simp>
  | k :: L => by
    intro hL
    obtain <mk, hmk, hbk> := hL k List.mem_cons_self
    obtain <m_0, hm_0, hall> :=
      exists_combined_witness x L (fun j hj => hL j (List.mem_cons_of_mem k hj))
    refine <myLor mk m_0, ?_, ?_>
    * rw [ <-  nbhd_inter]; exact x.inter_mem hmk hm_0
    * intro j hj
      rw [myLor_eq_lor, Nat.testBit_lor, Bool.or_eq_true]
      rcases List.mem_cons.mp hj with rfl | hj
      * exact Or.inl hbk
      * exact Or.inr (hall j hj)

theorem mem_bitsList {n k : Nat} : k  in  bitsList n  <->  n.testBit k = true := by
  unfold bitsList
  rw [List.mem_filter, List.mem_range]
  exact <fun h => h.2, fun h => <lt_of_testBit_true h, h>>

/-- **`x.mem (nbhd n)  <->  E_n  subseteq  elemSet x`.** Neighbourhood-membership
    reduces to positive-information
containment: `n`'s excluded set must already be covered by `x`'s recorded information.
    -/
theorem nbhd_mem_iff_subset_elemSet (x : PN.Element) (n : Nat) :
    x.mem (nbhd n)  <->  (nbhd n)?  subseteq  elemSet x := by
  rw [compl_nbhd]
  constructor
  * intro hn k hk
    exact <n, hn, hk>
  * intro hsub
    obtain <m_0, hm_0, hall> :=
      exists_combined_witness x (bitsList n) (fun k hk => hsub (mem_bitsList.mp hk))
    have hincl : nbhd m_0  subseteq  nbhd n := by
      intro k hk
      simp only [mem_nbhd] at hk  |-
      cases hn2 : n.testBit k with
      | false => rfl
      | true =>
        have hcontra := hall k (mem_bitsList.mpr hn2)
        rw [hk] at hcontra
        exact absurd hcontra (by decide)
```

```
    exact x.up_mem hm_0 <n, rfl> hincl

/-- **`elemSet x` is r.e. when `x` is a computable element.** Direct: `k  in  elemSet x
      <->  exists  n,
x.mem (nbhd n)  /\  n.testBit k`, an r.e. projection of the conjunction of an r.e.
    predicate and a
(`bitAt`-)decidable one. -/
theorem elemSet_re_of_isComputableElement {x : PN.Element} (hx : ComputableElement
    PNpres x) :
    REPred (fun k => k  in  elemSet x) := by
  have hx' : REPred (fun n => x.mem (nbhd n)) := hx.out
  have hbit : RecDecidable (fun t : Nat => t.unpair.1.testBit t.unpair.2 = true) :=
    <fun t => bitAt t.unpair.1 t.unpair.2, primrec_bitAt, fun t => (bitAt_eq_one_iff _
     _).symm>
  have hand : REPred (fun t : Nat => x.mem (nbhd t.unpair.1)  /\  t.unpair.1.testBit t
    .unpair.2 = true) :=
    (hx'.comp Nat.Primrec.left).and hbit.re
  refine REPred.of_iff (fun k => ?_) hand.proj
  simp only [mem_elemSet, unpair_pair_fst, unpair_pair_snd]

theorem mem_decodeList_bitsCode (n : Nat) :
    forall  N k, k  in  decodeList (bitsCode n N)  <->  k < N  /\  n.testBit k = true
  | 0, k => by simp [bitsCode, decodeList_zero]
  | N + 1, k => by
    show k  in  decodeList (selectFn (bitAt n N) (Nat.pair N (bitsCode n N) + 1) (
    bitsCode n N))  <->  _
    by_cases hbit : n.testBit N = true
    * rw [(bitAt_eq_one_iff n N).mpr hbit, selectFn_one, decodeList_succ,
    unpair_pair_fst,
        unpair_pair_snd, List.mem_cons, mem_decodeList_bitsCode n N k]
      constructor
      * rintro (rfl | <hkN, hk>)
        * exact <by omega, hbit>
        * exact <by omega, hk>
      * rintro <hkN1, hk>
        rcases (show k < N  \/  k = N by omega) with hkN | hkN
        * exact Or.inr <hkN, hk>
        * exact Or.inl hkN
    * have hne1 : bitAt n N  !=  1 := fun h => hbit ((bitAt_eq_one_iff n N).mp h)
      have hle := bitAt_le_one n N
      have hnbit : bitAt n N = 0 := by omega
      rw [hnbit, selectFn_zero, mem_decodeList_bitsCode n N k]
      constructor
      * rintro <hkN, hk>
        exact <Nat.lt_succ_of_lt hkN, hk>
      * rintro <hkN1, hk>
        rcases (show k < N  \/  k = N by omega) with hkN | hkN
        * exact <hkN, hk>
        * subst hkN; exact absurd hk hbit

theorem primrec_bitsStep : Nat.Primrec bitsStep := by
  have hn : Nat.Primrec (fun s : Nat => s.unpair.1) := Nat.Primrec.left
  have hN : Nat.Primrec (fun s : Nat => s.unpair.2.unpair.1) := Nat.Primrec.left.comp
    Nat.Primrec.right
  have hacc : Nat.Primrec (fun s : Nat => s.unpair.2.unpair.2) :=
    Nat.Primrec.right.comp Nat.Primrec.right
```

```
  have hbit : Nat.Primrec (fun s : Nat => bitAt s.unpair.1 s.unpair.2.unpair.1) :=
    (primrec_bitAt.comp (hn.pair hN)).of_eq fun s => by simp only [unpair_pair_fst,
    unpair_pair_snd]
  have hcons : Nat.Primrec (fun s : Nat => Nat.pair s.unpair.2.unpair.1 s.unpair.2.
    unpair.2 + 1) :=
    Nat.Primrec.succ.comp ((hN.pair hacc).of_eq fun _ => rfl)
  have hsel := primrec_selectFn hbit hcons hacc
  exact (hn.pair ((primrec_add_2 hN (Nat.Primrec.const 1)).pair hsel)).of_eq fun _ =>
    rfl

theorem bitsStep_unpair11 (s : Nat) : (bitsStep s).unpair.1 = s.unpair.1 := by
  unfold bitsStep; rw [unpair_pair_fst]

theorem bitsStep_unpair21 (s : Nat) : (bitsStep s).unpair.2.unpair.1 = s.unpair.2.
    unpair.1 + 1 := by
  unfold bitsStep; rw [unpair_pair_snd, unpair_pair_fst]

theorem bitsStep_unpair22 (s : Nat) : (bitsStep s).unpair.2.unpair.2 =
    selectFn (bitAt s.unpair.1 s.unpair.2.unpair.1)
      (Nat.pair s.unpair.2.unpair.1 s.unpair.2.unpair.2 + 1) s.unpair.2.unpair.2 := by
  unfold bitsStep; rw [unpair_pair_snd, unpair_pair_snd]

theorem bitsStep_iter_spec (n : Nat) (k : Nat) :
    (bitsStep^[k] (Nat.pair n (Nat.pair 0 0))).unpair.1 = n  /\
    (bitsStep^[k] (Nat.pair n (Nat.pair 0 0))).unpair.2.unpair.1 = k  /\
    (bitsStep^[k] (Nat.pair n (Nat.pair 0 0))).unpair.2.unpair.2 = bitsCode n k := by
  induction k with
  | zero =>
    refine <?_, ?_, ?_> <;>
      simp only [Function.iterate_zero, id_eq, unpair_pair_fst, unpair_pair_snd,
    bitsCode]
  | succ k ih =>
    obtain <hn, hN, hacc> := ih
    rw [Function.iterate_succ_apply']
    set s := bitsStep^[k] (Nat.pair n (Nat.pair 0 0))
    refine <?_, ?_, ?_>
    * rw [bitsStep_unpair11, hn]
    * rw [bitsStep_unpair21, hN]
    * rw [bitsStep_unpair22, hn, hN, hacc]; rfl

theorem primrec_bitsCode : Nat.Primrec (fun t => bitsCode t.unpair.1 t.unpair.2) := by
  have hbase : Nat.Primrec (fun z : Nat => Nat.pair z.unpair.1 (Nat.pair 0 0)) :=
    Nat.Primrec.left.pair ((Nat.Primrec.const 0).pair (Nat.Primrec.const 0))
  have hstep : Nat.Primrec (fun w : Nat => bitsStep w.unpair.2.unpair.2) :=
    primrec_bitsStep.comp (Nat.Primrec.right.comp Nat.Primrec.right)
  have hprec := Nat.Primrec.prec hbase hstep
  refine ((Nat.Primrec.right.comp Nat.Primrec.right).comp
    (hprec.comp (primrec_id.pair Nat.Primrec.right))).of_eq fun t => ?_
  simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd, id_eq]
  rw [rec_const_iterate]
  exact (bitsStep_iter_spec t.unpair.1 t.unpair.2).2.2

/-- **`x` is a computable element when `elemSet x` is r.e.** The bounded conjunction `
    E_n  subseteq  elemSet x`,
phrased over the coded list `bitsCode n n`, is r.e. by `REPred.forall_mem_decodeList`;
    by
```

```
`nbhd_mem_iff_subset_elemSet` this containment is exactly `x.mem (nbhd n)`. -/
theorem isComputableElement_of_elemSet_re {x : PN.Element} (hA : REPred (fun k => k
    in  elemSet x)) :
    ComputableElement PNpres x := by
  have hforall : REPred (fun c => forall  e  in  decodeList c, e  in  elemSet x) := hA
    .forall_mem_decodeList
  have hcode : Nat.Primrec (fun n : Nat => bitsCode n n) :=
    (primrec_bitsCode.comp (primrec_id.pair primrec_id)).of_eq fun n => by
      simp only [unpair_pair_fst, unpair_pair_snd, id_eq]
  constructor
  refine REPred.of_iff (fun n => ?_) (hforall.comp hcode)
  show x.mem (nbhd n)  <->  _
  rw [nbhd_mem_iff_subset_elemSet, compl_nbhd]
  constructor
  * intro hsub e he
    exact hsub ((mem_decodeList_bitsCode n n e).mp he).2
  * intro hall k hk
    exact hall k ((mem_decodeList_bitsCode n n k).mpr <lt_of_testBit_true hk, hk>)

/-- **What are the computable elements of `PN`? -- Exercise 7.23 (Scott 1981, PRG-19)
    .** Exactly the
recursively enumerable subsets of `Nat`, via the identification `x  |->  elemSet x` of
    `PN` with the
classical powerset domain `(Set Nat,  subseteq )`. -/
theorem isComputableElement_iff_elemSet_re (x : PN.Element) :
    ComputableElement PNpres x  <->  REPred (fun k => k  in  elemSet x) :=
  <elemSet_re_of_isComputableElement, isComputableElement_of_elemSet_re>

/-- **`unnum`'s second component strictly decreases** when the first is a successor --
    the
decreasing measure for both `untagRef` (well-founded recursion) and `untagStep` (
    bounded
iteration) below. -/
theorem unnum_snd_lt_of_fst_succ {c k : Nat} (h : (unnum c).1 = k + 1) : (unnum c).2 <
     c := by
  have heq : num (k + 1) (unnum c).2 = c := by
    have h' := numP_unnum c
    simpa [numP, h] using h'
  have hb := num_succ_left_gt k (unnum c).2
  omega
end

section
/-- `(i,m)  |->  bitAt m i = 1`, i.e. `m.testBit i`, as a `RecDecidable_2` relation (
    argument order
swapped from `bitAt`'s own `(n,k)  |->  bitAt n k`, to feed `RecDecidable_2.bForallList
    `). -/
theorem bitAt_swap_isComputable : RecDecidable_2 (fun i m => bitAt m i = 1) :=
  <fun t => bitAt t.unpair.2 t.unpair.1,
    (primrec_bitAt.comp (Nat.Primrec.right.pair Nat.Primrec.left)).of_eq fun t => by
      simp only [unpair_pair_fst, unpair_pair_snd],
    fun _ => Iff.rfl>
end

section
/-- `capMap`'s neighbourhood relation on indices: `nbhd n  U  nbhd m` relates to `nbhd
```

```
    k` iff both
`nbhd n  subseteq  nbhd k` and `nbhd m  subseteq  nbhd k`. -/
theorem capMap_rel_iff (n m k : Nat) :
    capMap.rel (prodNbhd (nbhd n) (nbhd m)) (nbhd k)  <->  nbhd n  subseteq  nbhd k
    /\  nbhd m  subseteq  nbhd k := by
  constructor
  * rintro <X, Y, heq, _, _, _, hXZ, hYZ>
    obtain <rfl, rfl> := prodNbhd_injective heq
    exact <hXZ, hYZ>
  * rintro <hXZ, hYZ>
    exact <nbhd n, nbhd m, rfl, <n, rfl>, <m, rfl>, <k, rfl>, hXZ, hYZ>
end

section
/-- **`fun x,y. x I y` is computable.** Reduces to `PNpres.incl_computable`, tested
    twice and
conjoined. -/
theorem capMap_isComputable :
    ComputableMap (prodPresentation PNpres PNpres) PNpres capMap := by
  constructor
  have hincl : RecDecidable (fun s => nbhd s.unpair.1  subseteq  nbhd s.unpair.2) :=
    PNpres.incl_computable
  have hr0 : Nat.Primrec (fun t => Nat.pair t.unpair.1.unpair.1 t.unpair.2) :=
    (Nat.Primrec.left.comp Nat.Primrec.left).pair Nat.Primrec.right
  have hr1 : Nat.Primrec (fun t => Nat.pair t.unpair.1.unpair.2 t.unpair.2) :=
    (Nat.Primrec.right.comp Nat.Primrec.left).pair Nat.Primrec.right
  refine (RecDecidable.of_iff (fun t => ?_) ((hincl.comp hr0).and (hincl.comp hr1))).
    re
  simp only [prodPresentation_X, unpair_pair_fst, unpair_pair_snd]
  exact capMap_rel_iff t.unpair.1.unpair.1 t.unpair.1.unpair.2 t.unpair.2
end

section
/-- `cupMap`'s neighbourhood relation on indices: `nbhd n  U  nbhd m` relates to `nbhd
    k` iff
`nbhd (myLor n m)  subseteq  nbhd k`. -/
theorem cupMap_rel_iff (n m k : Nat) :
    cupMap.rel (prodNbhd (nbhd n) (nbhd m)) (nbhd k)  <->  nbhd (myLor n m)  subseteq
     nbhd k := by
  rw [ <-  nbhd_inter n m]
  constructor
  * rintro <X, Y, heq, _, _, _, hXYZ>
    obtain <rfl, rfl> := prodNbhd_injective heq
    exact hXYZ
  * intro h
    exact <nbhd n, nbhd m, rfl, <n, rfl>, <m, rfl>, <k, rfl>, h>
end

section
/-- **`fun x,y. x U y` is computable.** Reduces to `PNpres.incl_computable` against the
     primitive
recursive reindexing `myLor`. -/
theorem cupMap_isComputable :
    ComputableMap (prodPresentation PNpres PNpres) PNpres cupMap := by
  constructor
  have hincl : RecDecidable (fun s => nbhd s.unpair.1  subseteq  nbhd s.unpair.2) :=
```

```
    PNpres.incl_computable
  have hlor : Nat.Primrec (fun t : Nat => myLor t.unpair.1.unpair.1 t.unpair.1.unpair
    .2) :=
    (primrec_myLor.comp ((Nat.Primrec.left.comp Nat.Primrec.left).pair
      (Nat.Primrec.right.comp Nat.Primrec.left))).of_eq fun t => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  have hr : Nat.Primrec (fun t => Nat.pair (myLor t.unpair.1.unpair.1 t.unpair.1.
    unpair.2)
      t.unpair.2) := hlor.pair Nat.Primrec.right
  refine (RecDecidable.of_iff (fun t => ?_) (hincl.comp hr)).re
  simp only [prodPresentation_X, unpair_pair_fst, unpair_pair_snd]
  exact cupMap_rel_iff t.unpair.1.unpair.1 t.unpair.1.unpair.2 t.unpair.2
end

section
/-- **`decodeList` is injective**, immediately from the round-trip `
    encodeList_decodeList`
(`decodeList` has a left inverse). -/
theorem decodeList_injective {a b : Nat} (h : decodeList a = decodeList b) : a = b :=
    by
  have := congrArg encodeList h
  rwa [encodeList_decodeList, encodeList_decodeList] at this
end

section
/-- **Membership in a function-space step set, as inclusion of `Xenum`s.** `forall  f
    in  W, f.rel X Y` iff
`W  subseteq  step X Y` (`mem_step`); specialised to `W = Xenum ... c` and the
    singleton step neighbourhood
(`Xenum_singleton`), this becomes the *decidable* function-space inclusion `
    incl_computable`. -/
theorem forall_rel_iff_subset_step {W : Set (ApproximableMap PN PN)} {n m_0 : Nat} :
    (forall  f  in  W, f.rel (nbhd n) (nbhd m_0))  <->  W  subseteq  step (nbhd n) (
    nbhd m_0) := by
  constructor
  * intro h f hf; exact mem_step.mpr (h f hf)
  * intro h f hf; exact mem_step.mp (h hf)
end

section
/-- `gMap`'s neighbourhood relation on indices: `nbhd n  U  nbhd m` relates to `nbhd k`
     iff
`nbhd k`'s excluded set fits inside `Fun (nbhd n)? (nbhd m)?`. -/
theorem gMap_rel_iff (n m k : Nat) :
    gMap.rel (prodNbhd (nbhd n) (nbhd m)) (nbhd k)  <->  (nbhd k)?  subseteq  Fun (
    nbhd n)? (nbhd m)? := by
  constructor
  * rintro <X, Y, heq, _, _, _, hsub>
    obtain <rfl, rfl> := prodNbhd_injective heq
    exact hsub
  * intro h
    exact <nbhd n, nbhd m, rfl, <n, rfl>, <m, rfl>, <k, rfl>, h>
end

section
/-- `graphMap`'s neighbourhood relation on indices: `Xenum ... c` relates to `nbhd m`
```

```
    iff `nbhd m`'s
excluded set fits inside `GraphIdx (Xenum ... c)`. -/
theorem graphMap_rel_iff (gN : Nat  ->  Nat)
    (hgN : forall  c, gN c = 1  <->  (stepFun (funListOf PNpres PNpres (decodeList c))
      : Set (ApproximableMap PN PN)).Nonempty) (c m : Nat) :
    graphMap.rel (Xenum PNpres PNpres gN c) (nbhd m)  <->
      (nbhd m)?  subseteq  GraphIdx (Xenum PNpres PNpres gN c) := by
  constructor
  * rintro <_, _, hsub>; exact hsub
  * intro h; exact <Xenum_mem PNpres PNpres gN hgN c, <m, rfl>, h>
end

section
/-- **The canonical witness list for `E? = (nbhd j)?`.** `decodeList (bitsCode j j)`
    has exactly the
set bits of `j` as entries (order immaterial for `entries`, but fixed here for `tag`'s
    coding). -/
theorem mem_bitsCodeList {n k : Nat} : k  in  decodeList (bitsCode n n)  <->  n.
    testBit k = true := by
  rw [mem_decodeList_bitsCode]
  exact <fun h => h.2, fun h => <lt_of_testBit_true h, h>>
end

section
/-- Pointwise form of `compl_nbhd`, convenient for `rw`/case-splitting on the (
    decidable) bit. -/
theorem mem_compl_nbhd {n x : Nat} : x  in  (nbhd n)?  <->  n.testBit x = true := by
  rw [compl_nbhd]; rfl
end

section
/-- **`(nbhd k)?  subseteq  Fun (nbhd n)? (nbhd m)?`, bounded-quantifier form.** Every
    excluded point of
`nbhd k` is below `k` (`lt_of_testBit_true`), so the subset test is a bounded `forall j
     < k`. -/
theorem compl_nbhd_subset_Fun_iff (n m k : Nat) : (nbhd k)?  subseteq  Fun (nbhd n)? (
    nbhd m)?  <->
    forall  j, j < k  ->  (k.testBit j = true  ->  j  in  Fun (nbhd n)? (nbhd m)?) :=
    by
  constructor
  * intro h j _ hj
    exact h (mem_compl_nbhd.mpr hj)
  * intro h j hj
    rw [mem_compl_nbhd] at hj
    exact h j (lt_of_testBit_true hj) hj
end

section
/-- **`(nbhd k)?  subseteq  S`, bounded-quantifier form**, for an *arbitrary* target
    set `S`: every excluded
point of `nbhd k` is below `k` (`lt_of_testBit_true`), so the subset test is a bounded
    `forall  j < k`.
(The `S`-generic form of `compl_nbhd_subset_Fun_iff`.) -/
theorem compl_nbhd_subset_iff (S : Set Nat) (k : Nat) : (nbhd k)?  subseteq  S  <->
    forall  j, j < k  ->  (k.testBit j = true  ->  j  in  S) := by
  constructor
```

```
  * intro h j _ hj
    exact h (mem_compl_nbhd.mpr hj)
  * intro h j hj
    rw [mem_compl_nbhd] at hj
    exact h j (lt_of_testBit_true hj) hj
end

section
theorem mem_sumSet {X Y : Set Nat} {k : Nat} : k  in  X +? Y  <->  exists  a  in  X,
    exists  b  in  Y, a + b = k := Iff.rfl
end

section
/-- **Choice-free converse of `compl_subset_compl_of_subset`, specialized to `nbhd`.**
    For cofinite
neighbourhoods the excluded sets have decidable membership (`Nat.testBit`), so unlike
    the general
`Set.compl_subset_compl` this direction needs only a case split on a `Bool`, not
    excluded middle. -/
theorem nbhd_subset_iff_compl_subset_compl (a b : Nat) :
    nbhd a  subseteq  nbhd b  <->  (nbhd b)?  subseteq  (nbhd a)? := by
  constructor
  * exact compl_subset_compl_of_subset
  * intro h x hxa
    rw [mem_nbhd] at hxa  |-
    cases hbit : b.testBit x with
    | false => rfl
    | true => exact absurd (mem_compl_nbhd.mp (h (mem_compl_nbhd.mpr hbit))) (by rw [
    hxa]; decide)
end

section
/-- **The reversal fact.** `nbhd n  subseteq  nbhd k  <->  myLor n k = n`: `nbhd n
    subseteq  nbhd k` iff
`nbhd n  I  nbhd k = nbhd n` iff (by `nbhd_inter` + injectivity) `nbhd (myLor n k) =
    nbhd n`. -/
theorem nbhd_subset_iff_myLor_eq (n k : Nat) : nbhd n  subseteq  nbhd k  <->  myLor n
    k = n := by
  constructor
  * intro h
    apply nbhd_injective
    rw [ <-  nbhd_inter]
    exact (Set.inter_eq_left.mpr h)
  * intro h
    rw [ <-  h,  <-  nbhd_inter]
    exact Set.inter_subset_right
end

section
theorem nextCellStep_eq (n m : Nat) :
    nextCellStep (Nat.pair n m) = Nat.pair (nextCell (n, m)).1 (nextCell (n, m)).2 :=
    by
  unfold nextCellStep
  simp only [unpair_pair_fst, unpair_pair_snd]
  rcases n with _ | k
  * have hz : isZero 0 = 1 := by unfold isZero; omega
```

```
    rw [hz, selectFn_one]; rfl
  * have hz : isZero (k + 1) = 0 := by unfold isZero; omega
    rw [hz, selectFn_zero]; rfl
end

section
theorem plusStep_unpair11 (s : Nat) : (plusStep s).unpair.1.unpair.1 = s.unpair.1.
    unpair.1 := by
  unfold plusStep; rw [unpair_pair_fst, unpair_pair_fst]
end

section
theorem plusStep_unpair12 (s : Nat) : (plusStep s).unpair.1.unpair.2 = s.unpair.1.
    unpair.2 := by
  unfold plusStep; rw [unpair_pair_fst, unpair_pair_snd]
end

section
theorem plusStep_unpair21 (s : Nat) : (plusStep s).unpair.2.unpair.1 = s.unpair.2.
    unpair.1 + 1 := by
  unfold plusStep; rw [unpair_pair_snd, unpair_pair_fst]
end

section
theorem plusStep_unpair22 (s : Nat) : (plusStep s).unpair.2.unpair.2 =
    myLor s.unpair.2.unpair.2 (selectFn (bitAt s.unpair.1.unpair.1 s.unpair.2.unpair
    .1)
      (s.unpair.1.unpair.2 <<< s.unpair.2.unpair.1) 0) := by
  unfold plusStep; rw [unpair_pair_snd, unpair_pair_snd]
end

section
theorem plusStep_iter_spec (n m : Nat) (k : Nat) :
    (plusStep^[k] (Nat.pair (Nat.pair n m) (Nat.pair 0 0))).unpair.1.unpair.1 = n  /\
    (plusStep^[k] (Nat.pair (Nat.pair n m) (Nat.pair 0 0))).unpair.1.unpair.2 = m  /\
    (plusStep^[k] (Nat.pair (Nat.pair n m) (Nat.pair 0 0))).unpair.2.unpair.1 = k  /\
    (plusStep^[k] (Nat.pair (Nat.pair n m) (Nat.pair 0 0))).unpair.2.unpair.2 = orUpTo
     n m k := by
  induction k with
  | zero =>
    refine <?_, ?_, ?_, ?_> <;>
      simp only [Function.iterate_zero, id_eq, unpair_pair_fst, unpair_pair_snd,
    orUpTo]
  | succ k ih =>
    obtain <hn, hm, ha, hacc> := ih
    rw [Function.iterate_succ_apply']
    set s := plusStep^[k] (Nat.pair (Nat.pair n m) (Nat.pair 0 0))
    refine <?_, ?_, ?_, ?_>
    * rw [plusStep_unpair11, hn]
    * rw [plusStep_unpair12, hm]
    * rw [plusStep_unpair21, ha]
    * rw [plusStep_unpair22, hn, hm, ha, hacc]; rfl
end

section
theorem primrec_nextCellStep : Nat.Primrec nextCellStep := by
```

```
  have hn : Nat.Primrec (fun w : Nat => w.unpair.1) := Nat.Primrec.left
  have hm : Nat.Primrec (fun w : Nat => w.unpair.2) := Nat.Primrec.right
  have hz : Nat.Primrec (fun w : Nat => isZero w.unpair.1) := primrec_isZero.comp hn
  have ha : Nat.Primrec (fun w : Nat => Nat.pair (w.unpair.2 + 1) 0) :=
    (primrec_add_2 hm (Nat.Primrec.const 1)).pair (Nat.Primrec.const 0)
  have hb : Nat.Primrec (fun w : Nat => Nat.pair (w.unpair.1 - 1) (w.unpair.2 + 1)) :=
    (primrec_sub_2 hn (Nat.Primrec.const 1)).pair (primrec_add_2 hm (Nat.Primrec.const
    1))
  exact (primrec_selectFn hz ha hb).of_eq fun _ => rfl
end

section
theorem primrec_plusStep : Nat.Primrec plusStep := by
  have hn : Nat.Primrec (fun s : Nat => s.unpair.1.unpair.1) := Nat.Primrec.left.comp
    Nat.Primrec.left
  have hm : Nat.Primrec (fun s : Nat => s.unpair.1.unpair.2) := Nat.Primrec.right.comp
     Nat.Primrec.left
  have ha : Nat.Primrec (fun s : Nat => s.unpair.2.unpair.1) := Nat.Primrec.left.comp
    Nat.Primrec.right
  have hacc : Nat.Primrec (fun s : Nat => s.unpair.2.unpair.2) := Nat.Primrec.right.
    comp Nat.Primrec.right
  have hbit : Nat.Primrec (fun s : Nat => bitAt s.unpair.1.unpair.1 s.unpair.2.unpair
    .1) :=
    (primrec_bitAt.comp (hn.pair ha)).of_eq fun s => by simp only [unpair_pair_fst,
    unpair_pair_snd]
  have hshift : Nat.Primrec (fun s : Nat => s.unpair.1.unpair.2 <<< s.unpair.2.unpair
    .1) := by
    have := primrec_mul_2 hm (primrec_two_pow ha)
    exact this.of_eq fun s => by rw [Nat.shiftLeft_eq]
  have hsel := primrec_selectFn hbit hshift (Nat.Primrec.const 0)
  have hlor : Nat.Primrec (fun s : Nat => myLor s.unpair.2.unpair.2
      (selectFn (bitAt s.unpair.1.unpair.1 s.unpair.2.unpair.1)
        (s.unpair.1.unpair.2 <<< s.unpair.2.unpair.1) 0)) :=
    (primrec_myLor.comp (hacc.pair hsel)).of_eq fun s => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  exact ((hn.pair hm).pair ((primrec_add_2 ha (Nat.Primrec.const 1)).pair hlor)).of_eq
     fun _ => rfl
end

section
theorem primrec_plusIdx : Nat.Primrec (fun t => plusIdx t.unpair.1 t.unpair.2) := by
  have hbase : Nat.Primrec
      (fun z => Nat.pair (Nat.pair z.unpair.1 z.unpair.2) (Nat.pair 0 0)) :=
    (Nat.Primrec.left.pair Nat.Primrec.right).pair
      ((Nat.Primrec.const 0).pair (Nat.Primrec.const 0))
  have hstep : Nat.Primrec (fun w => plusStep w.unpair.2.unpair.2) :=
    primrec_plusStep.comp (Nat.Primrec.right.comp Nat.Primrec.right)
  have hprec := Nat.Primrec.prec hbase hstep
  refine ((Nat.Primrec.right.comp Nat.Primrec.right).comp
    (hprec.comp (primrec_id.pair Nat.Primrec.left))).of_eq fun t => ?_
  simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd, id_eq]
  rw [rec_const_iterate]
  exact (plusStep_iter_spec t.unpair.1 t.unpair.2 t.unpair.1).2.2.2
end

section
```

```
/-- `tri k = k(k+1)/2` is primitive recursive: `tri k * 2 = k(k+1)` is *exact* division
     by the
literal `2`, computed by `primrec_div2`. -/
theorem primrec_tri : Nat.Primrec tri :=
  (primrec_div2.comp (primrec_mul_2 primrec_id Nat.Primrec.succ)).of_eq fun _ => rfl
end

section
/-- `num n m = tri(n+m) + m` is primitive recursive. -/
theorem primrec_num : Nat.Primrec (fun t => num t.unpair.1 t.unpair.2) :=
  (primrec_add_2 (primrec_tri.comp (primrec_add_2 Nat.Primrec.left Nat.Primrec.right))
    Nat.Primrec.right).of_eq fun _ => rfl
end

section
theorem primrec_tagStep : Nat.Primrec tagStep := by
  have h1 : Nat.Primrec (fun w : Nat => w.unpair.1 + 1) :=
    primrec_add_2 Nat.Primrec.left (Nat.Primrec.const 1)
  have h2 : Nat.Primrec (fun w : Nat => w.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.right
  exact (primrec_num.comp (h1.pair h2)).of_eq fun w => by
    simp only [unpair_pair_fst, unpair_pair_snd]
    rfl
end

section
theorem primrec_tagCode : Nat.Primrec (fun t => tagCode t.unpair.1 t.unpair.2) := by
  have hz : Nat.Primrec (fun t : Nat => num 0 t.unpair.2) :=
    (primrec_num.comp ((Nat.Primrec.const 0).pair Nat.Primrec.right)).of_eq fun t =>
    by
      simp only [unpair_pair_fst, unpair_pair_snd]
  have hc : Nat.Primrec (fun t : Nat => appendCode 0 t.unpair.1) :=
    (primrec_appendCode.comp ((Nat.Primrec.const 0).pair Nat.Primrec.left)).of_eq fun
    t => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  exact (primrec_foldCode primrec_tagStep (Nat.Primrec.const 0) hz hc).of_eq fun _ =>
    rfl
end

section
theorem primrec_tagOfBits : Nat.Primrec (fun t => tagOfBits t.unpair.1 t.unpair.2) :=
    by
  have hj : Nat.Primrec (fun t : Nat => bitsCode t.unpair.1 t.unpair.1) :=
    (primrec_bitsCode.comp (Nat.Primrec.left.pair Nat.Primrec.left)).of_eq fun t => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  exact (primrec_tagCode.comp (hj.pair Nat.Primrec.right)).of_eq fun t => by
    simp only [unpair_pair_fst, unpair_pair_snd]
    rfl
end

section
theorem primrec_unnumPair : Nat.Primrec unnumPair := by
  have hbase : Nat.Primrec (fun _ : Nat => Nat.pair (0 : Nat) 0) := Nat.Primrec.const
    (Nat.pair 0 0)
  have hstep : Nat.Primrec (fun w : Nat => nextCellStep w.unpair.2.unpair.2) :=
    primrec_nextCellStep.comp (Nat.Primrec.right.comp Nat.Primrec.right)
```

```
  have hprec := Nat.Primrec.prec hbase hstep
  refine (hprec.comp ((Nat.Primrec.const 0).pair primrec_id)).of_eq fun v => ?_
  simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd, id_eq]
  show Nat.rec (Nat.pair 0 0) (fun _ ih => nextCellStep ih) v = unnumPair v
  rw [rec_const_iterate]; rfl
end

section
theorem primrec_unnum1 : Nat.Primrec unnum1 := Nat.Primrec.left.comp primrec_unnumPair
end

section
theorem primrec_unnum2 : Nat.Primrec unnum2 := Nat.Primrec.right.comp
    primrec_unnumPair
end

section
theorem primrec_untagStep : Nat.Primrec untagStep := by
  have hdone : Nat.Primrec (fun w : Nat => w.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.right
  have hrevAcc : Nat.Primrec (fun w : Nat => w.unpair.2.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp (Nat.Primrec.right.comp Nat.Primrec.right)
  have hval : Nat.Primrec (fun w : Nat => w.unpair.2.unpair.2.unpair.2) :=
    Nat.Primrec.right.comp (Nat.Primrec.right.comp Nat.Primrec.right)
  have ha : Nat.Primrec (fun w : Nat => unnum1 w.unpair.1) := primrec_unnum1.comp Nat.
    Primrec.left
  have hb : Nat.Primrec (fun w : Nat => unnum2 w.unpair.1) := primrec_unnum2.comp Nat.
    Primrec.left
  have hz : Nat.Primrec (fun w : Nat => isZero (unnum1 w.unpair.1)) := primrec_isZero.
    comp ha
  have hhalt : Nat.Primrec (fun w : Nat => Nat.pair (unnum2 w.unpair.1)
      (Nat.pair 1 (Nat.pair w.unpair.2.unpair.2.unpair.1 (unnum2 w.unpair.1)))) :=
    hb.pair ((Nat.Primrec.const 1).pair (hrevAcc.pair hb))
  have hcont : Nat.Primrec (fun w : Nat => Nat.pair (unnum2 w.unpair.1)
      (Nat.pair 0
        (Nat.pair (Nat.pair (unnum1 w.unpair.1 - 1) w.unpair.2.unpair.2.unpair.1 + 1)
          w.unpair.2.unpair.2.unpair.2))) :=
    hb.pair ((Nat.Primrec.const 0).pair
      ((Nat.Primrec.succ.comp ((primrec_sub_2 ha (Nat.Primrec.const 1)).pair hrevAcc))
    .pair hval))
  have hinner := primrec_selectFn hz hhalt hcont
  exact (primrec_selectFn hdone primrec_id hinner).of_eq fun _ => rfl
end

section
theorem primrec_untagState : Nat.Primrec untagState := by
  have hbase : Nat.Primrec (fun c : Nat => Nat.pair c (Nat.pair (0 : Nat) (Nat.pair 0
    0))) :=
    primrec_id.pair ((Nat.Primrec.const 0).pair ((Nat.Primrec.const 0).pair (Nat.
    Primrec.const 0)))
  have hstep : Nat.Primrec (fun w : Nat => untagStep w.unpair.2.unpair.2) :=
    primrec_untagStep.comp (Nat.Primrec.right.comp Nat.Primrec.right)
  have hprec := Nat.Primrec.prec hbase hstep
  refine (hprec.comp (primrec_id.pair Nat.Primrec.succ)).of_eq fun c => ?_
  simp only [Nat.unpaired, unpair_pair_fst, unpair_pair_snd, id_eq]
  rw [rec_const_iterate]
```

```
  unfold untagState
  rw [Function.iterate_succ_apply']
end

section
theorem primrec_untagList : Nat.Primrec untagList :=
  primrec_appendCode.comp
    ((Nat.Primrec.const 0).pair
      (Nat.Primrec.left.comp (Nat.Primrec.right.comp (Nat.Primrec.right.comp
    primrec_untagState))))
    |>.of_eq fun c => by unfold untagList; simp only [unpair_pair_fst, unpair_pair_snd
    ]
end

section
theorem primrec_untagVal : Nat.Primrec untagVal :=
  Nat.Primrec.right.comp (Nat.Primrec.right.comp (Nat.Primrec.right.comp
    primrec_untagState))
end

section
/-- **`tag` reformulated as a left fold over the reversed list.** `tag`'s recursion is
    a `foldr`;
`List.foldl_reverse` bridges it to the `foldl`-shaped `foldCode`. -/
theorem tag_eq_foldr (ns : List Nat) (m : Nat) :
    tag ns m = ns.foldr (fun x y => num (x + 1) y) (num 0 m) := by
  induction ns with
  | nil => rfl
  | cons a ns ih => rw [tag_cons, ih]; rfl
end

section
theorem tag_eq_foldl_reverse (ns : List Nat) (m : Nat) :
    tag ns m = ns.reverse.foldl (fun acc a => num (a + 1) acc) (num 0 m) := by
  rw [List.foldl_reverse]; exact tag_eq_foldr ns m
end

section
theorem tagCode_spec (code m : Nat) : tagCode code m = tag (decodeList code) m := by
  unfold tagCode
  rw [foldCode_eq']
  have hstep : (fun acc x => tagStep (Nat.pair x (Nat.pair acc 0)))
      = (fun acc x => num (x + 1) acc) := by
    funext acc x
    show tagStep (Nat.pair x (Nat.pair acc 0)) = num (x + 1) acc
    unfold tagStep
    simp only [unpair_pair_fst, unpair_pair_snd]
  rw [hstep, decodeList_appendCode, decodeList_zero, List.append_nil]
  exact (tag_eq_foldl_reverse (decodeList code) m).symm
end

section
theorem tagOfBits_spec (j m : Nat) : tagOfBits j m = tag (decodeList (bitsCode j j)) m
     :=
  tagCode_spec (bitsCode j j) m
end
```

```
section
/-- **`orUpTo`'s bit-by-bit characterization.** Bit `k` of `orUpTo n m N` is set
    exactly when some
`a < N` is a set bit of `n` with `a  <=  k` and `k - a` a set bit of `m`. -/
theorem testBit_orUpTo (n m : Nat) : forall  N k,
    (orUpTo n m N).testBit k = true  <->  exists  a, a < N  /\  n.testBit a = true  /\
      a  <=  k  /\  m.testBit (k - a) = true
  | 0, k => by simp [orUpTo]
  | N + 1, k => by
    show (myLor (orUpTo n m N) (selectFn (bitAt n N) (m <<< N) 0)).testBit k = true
    <->  _
    rw [myLor_eq_lor, Nat.testBit_lor, Bool.or_eq_true, testBit_orUpTo n m N]
    by_cases hbit : n.testBit N = true
    * rw [(bitAt_eq_one_iff n N).mpr hbit, selectFn_one, Nat.testBit_shiftLeft]
      constructor
      * rintro (<a, haN, ha, hak, hb> | h)
        * exact <a, Nat.lt_succ_of_lt haN, ha, hak, hb>
        * rw [Bool.and_eq_true, decide_eq_true_iff] at h
          exact <N, Nat.lt_succ_self N, hbit, h.1, h.2>
      * rintro <a, haN1, ha, hak, hb>
        rcases (show a < N  \/  a = N by omega) with haN | haN
        * exact Or.inl <a, haN, ha, hak, hb>
        * subst haN
          exact Or.inr (by rw [Bool.and_eq_true, decide_eq_true_iff]; exact <hak, hb>)
    * have hne1 : bitAt n N  !=  1 := fun h => hbit ((bitAt_eq_one_iff n N).mp h)
      have hle := bitAt_le_one n N
      have hnbit : bitAt n N = 0 := by omega
      rw [hnbit, selectFn_zero]
      simp only [Nat.zero_testBit, Bool.false_eq_true, or_false]
      constructor
      * rintro <a, haN, ha, hak, hb>; exact <a, Nat.lt_succ_of_lt haN, ha, hak, hb>
      * rintro <a, haN1, ha, hak, hb>
        rcases (show a < N  \/  a = N by omega) with haN | haN
        * exact <a, haN, ha, hak, hb>
        * subst haN; exact absurd ha hbit
end

section
/-- **`orUpTo n m n` computes the Minkowski sum `E_n + E?` exactly.** Every set bit `a`
     of `n`
satisfies `a < n` (`lt_of_testBit_true`), so ranging `a` over `N = n` sees every set
    bit of `n`. -/
theorem testBit_orUpTo_self (n m k : Nat) :
    (orUpTo n m n).testBit k = true  <->  exists  a, a  <=  k  /\  n.testBit a = true
     /\  m.testBit (k - a) = true := by
  rw [testBit_orUpTo]
  constructor
  * rintro <a, _, ha, hak, hb>; exact <a, hak, ha, hb>
  * rintro <a, hak, ha, hb>; exact <a, lt_of_testBit_true ha, ha, hak, hb>
end

section
theorem plusIdx_testBit (n m k : Nat) :
    (plusIdx n m).testBit k = true  <->  exists  a, a  <=  k  /\  n.testBit a = true
    /\  m.testBit (k - a) = true :=
```

```
  testBit_orUpTo_self n m k
end

section
/-- **`plusIdx`'s excluded set is the Minkowski sum `E_n + E?`.** -/
theorem compl_nbhd_plusIdx (n m : Nat) : (nbhd (plusIdx n m))? = (nbhd n)? +? (nbhd m)
    ? := by
  rw [compl_nbhd, compl_nbhd, compl_nbhd]
  ext k
  simp only [mem_sumSet, Set.mem_ofPred_eq, plusIdx_testBit]
  constructor
  * rintro <a, hak, ha, hb>; exact <a, ha, k - a, hb, by omega>
  * rintro <a, ha, b, hb, rfl>
    exact <a, by omega, ha, by rw [Nat.add_sub_cancel_left]; exact hb>
end

section
/-- `plusMap`'s neighbourhood relation on indices: `nbhd n  U  nbhd m` relates to `nbhd
    k` iff
`nbhd (plusIdx n m)  subseteq  nbhd k`. -/
theorem plusMap_rel_iff (n m k : Nat) :
    plusMap.rel (prodNbhd (nbhd n) (nbhd m)) (nbhd k)  <->  nbhd (plusIdx n m)
    subseteq  nbhd k := by
  rw [nbhd_subset_iff_compl_subset_compl, compl_nbhd_plusIdx]
  constructor
  * rintro <X, Y, heq, _, _, _, hsub>
    obtain <rfl, rfl> := prodNbhd_injective heq
    exact hsub
  * intro h
    exact <nbhd n, nbhd m, rfl, <n, rfl>, <m, rfl>, <k, rfl>, h>
end

section
/-- **`fun x,y. x+y` is computable.** Reduces to `PNpres.incl_computable` against the
    primitive
recursive reindexing `plusIdx`. -/
theorem plusMap_isComputable :
    ComputableMap (prodPresentation PNpres PNpres) PNpres plusMap := by
  constructor
  have hincl : RecDecidable (fun s => nbhd s.unpair.1  subseteq  nbhd s.unpair.2) :=
    PNpres.incl_computable
  have hpair : Nat.Primrec (fun t : Nat => Nat.pair t.unpair.1.unpair.1 t.unpair.1.
    unpair.2) :=
    (Nat.Primrec.left.comp Nat.Primrec.left).pair (Nat.Primrec.right.comp Nat.Primrec.
    left)
  have hplus : Nat.Primrec (fun t : Nat => plusIdx t.unpair.1.unpair.1 t.unpair.1.
    unpair.2) :=
    (primrec_plusIdx.comp hpair).of_eq fun t => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  have hr : Nat.Primrec (fun t => Nat.pair (plusIdx t.unpair.1.unpair.1 t.unpair.1.
    unpair.2)
      t.unpair.2) := hplus.pair Nat.Primrec.right
  refine (RecDecidable.of_iff (fun t => ?_) (hincl.comp hr)).re
  simp only [prodPresentation_X, unpair_pair_fst, unpair_pair_snd]
  exact plusMap_rel_iff t.unpair.1.unpair.1 t.unpair.1.unpair.2 t.unpair.2
end
```

```
section
theorem unnumPair_spec (v : Nat) : unnumPair v = Nat.pair (unnum v).1 (unnum v).2 :=
    by
  unfold unnumPair
  induction v with
  | zero => rfl
  | succ v ih => rw [Function.iterate_succ_apply', ih, nextCellStep_eq]; rfl
end

section
theorem unnum1_spec (v : Nat) : unnum1 v = (unnum v).1 := by
  unfold unnum1; rw [unnumPair_spec, unpair_pair_fst]
end

section
theorem unnum2_spec (v : Nat) : unnum2 v = (unnum v).2 := by
  unfold unnum2; rw [unnumPair_spec, unpair_pair_snd]
end

section
/-- **The innermost list-check is recursively decidable.** Reindexes `bForallList`
    along
`untagList` to get `forall i  in  decodeList (untagList c), bitAt m i = 1` as a
    function of `(c,m)`, packed
`(c,(j,m))`-style (`j` is a dummy free parameter here, along for the ride to match `
    gMap`'s later
packing). -/
theorem untagList_bitAt_isComputable :
    RecDecidable (fun w : Nat => forall  i  in  decodeList (untagList w.unpair.1),
    bitAt w.unpair.2.unpair.2 i = 1) :=
  RecDecidable.of_iff (fun w => by simp only [unpair_pair_fst, unpair_pair_snd])
    (bitAt_swap_isComputable.bForallList.comp
      ((primrec_untagList.comp Nat.Primrec.left).pair (Nat.Primrec.right.comp Nat.
    Primrec.right)))
end

section
theorem untagRef_succ_case {c k : Nat} (h : (unnum c).1 = k + 1) :
    untagRef c = (k :: (untagRef (unnum c).2).1, (untagRef (unnum c).2).2) := by
  have h0 : not  (unnum c).1 = 0 := by omega
  rw [untagRef, dite_eq_right h0, h]
  have hsub : k + 1 - 1 = k := by omega
  rw [hsub]
end

section
theorem untagRef_zero_case {c : Nat} (h : (unnum c).1 = 0) :
    untagRef c = ([], (unnum c).2) := by
  rw [untagRef]; simp [h]
end

section
/-- **`untagRef` is a genuine right-inverse of `tag`.** Strong induction on `c`,
    mirroring
`tag_surjective`. -/
```

```
theorem tag_untagRef (c : Nat) : tag (untagRef c).1 (untagRef c).2 = c := by
  induction c using Nat.strong_induction_on with
  | _ c ih =>
    rcases hc : (unnum c).1 with a | k
    * rw [untagRef_zero_case hc]
      have heq := numP_unnum c
      simp only [numP, hc] at heq
      rw [tag_nil]
      exact heq
    * rw [untagRef_succ_case hc]
      have heq : num (k + 1) (unnum c).2 = c := by
        have h := numP_unnum c
        simpa [numP, hc] using h

      have hlt : (unnum c).2 < c := by
        have hb := num_succ_left_gt k (unnum c).2
        omega
      rw [tag_cons, ih (unnum c).2 hlt, heq]
end

section
theorem untagStep_cont {w : Nat} (hdone : w.unpair.2.unpair.1 = 0) {k : Nat}
    (ha : unnum1 w.unpair.1 = k + 1) :
    untagStep w = Nat.pair (unnum2 w.unpair.1)
      (Nat.pair 0
        (Nat.pair (Nat.pair k w.unpair.2.unpair.2.unpair.1 + 1) w.unpair.2.unpair.2.
    unpair.2)) := by
  unfold untagStep
  rw [hdone, selectFn_zero, ha]
  have hz : isZero (k + 1) = 0 := by unfold isZero; omega
  have hsub : k + 1 - 1 = k := by omega
  rw [hz, selectFn_zero, hsub]
end

section
theorem untagStep_done {w : Nat} (h : w.unpair.2.unpair.1 = 1) : untagStep w = w := by
  unfold untagStep; rw [h, selectFn_one]
end

section
theorem untagStep_halt {w : Nat} (hdone : w.unpair.2.unpair.1 = 0) (ha : unnum1 w.
    unpair.1 = 0) :
    untagStep w = Nat.pair (unnum2 w.unpair.1)
      (Nat.pair 1 (Nat.pair w.unpair.2.unpair.2.unpair.1 (unnum2 w.unpair.1))) := by
  unfold untagStep
  rw [hdone, selectFn_zero, ha]
  have : isZero 0 = 1 := by unfold isZero; omega
  rw [this, selectFn_one]
end

section
theorem untagStep_of_done_iterate (w : Nat) (h : w.unpair.2.unpair.1 = 1) :
    forall  k, untagStep^[k] w = w
  | 0 => rfl
  | k + 1 => by rw [Function.iterate_succ_apply', untagStep_of_done_iterate w h k,
    untagStep_done h]
```

```
end

section
/-- **Main invariant.** Given enough fuel (`c  <=  fuel`), `fuel + 1` iterations of `
    untagStep` from a
fresh `code = c` state (arbitrary carried `r`, `v`) freeze with `revAcc` holding `(
    untagRef c).1`
consed onto `r` (i.e. `reverse (untagRef c).1 ++ decodeList r`, matching `appendCode`'s
     convention)
and `val` holding `(untagRef c).2`. -/
theorem untagStep_iter_spec : forall  c, forall  fuel r v : Nat, c  <=  fuel  ->
    untagStep^[fuel + 1] (Nat.pair c (Nat.pair 0 (Nat.pair r v))) =
      Nat.pair (untagRef c).2 (Nat.pair 1
        (Nat.pair ((untagRef c).1.foldl (fun acc x => Nat.pair x acc + 1) r) (untagRef
     c).2)) := by
  intro c
  induction c using Nat.strong_induction_on with
  | _ c ih =>
    intro fuel r v hfuel
    set w : Nat := Nat.pair c (Nat.pair 0 (Nat.pair r v)) with hw
    have hw1 : w.unpair.1 = c := by rw [hw, unpair_pair_fst]
    have hw2 : w.unpair.2.unpair.1 = 0 := by rw [hw]; simp only [unpair_pair_snd,
    unpair_pair_fst]
    have hw3 : w.unpair.2.unpair.2.unpair.1 = r := by
      rw [hw]; simp only [unpair_pair_snd, unpair_pair_fst]
    have hw4 : w.unpair.2.unpair.2.unpair.2 = v := by
      rw [hw]; simp only [unpair_pair_snd]
    rw [Function.iterate_succ_apply]
    rcases hc : (unnum c).1 with _ | k
    * have ha0 : unnum1 w.unpair.1 = 0 := by rw [hw1, unnum1_spec, hc]
      have hstep : untagStep w = Nat.pair (unnum c).2 (Nat.pair 1 (Nat.pair r (unnum c
    ).2)) := by
        rw [untagStep_halt hw2 ha0, hw1, hw3, unnum2_spec]
      rw [hstep]
      have hdone1 : (Nat.pair (unnum c).2
          (Nat.pair 1 (Nat.pair r (unnum c).2))).unpair.2.unpair.1 = 1 := by
        simp only [unpair_pair_snd, unpair_pair_fst]
      rw [untagStep_of_done_iterate _ hdone1, untagRef_zero_case hc, List.foldl_nil]
    * have hak : unnum1 w.unpair.1 = k + 1 := by rw [hw1, unnum1_spec, hc]
      set b : Nat := (unnum c).2 with hbdef
      have hblt : b < c := unnum_snd_lt_of_fst_succ hc
      have hc1 : 1  <=  c := by omega
      have hstep : untagStep w = Nat.pair b (Nat.pair 0 (Nat.pair (Nat.pair k r + 1) v
    )) := by
        rw [untagStep_cont hw2 hak, hw1, hw3, hw4, unnum2_spec]
      rw [hstep]
      rcases fuel with _ | fuel'
      * omega
      * have hble : b  <=  fuel' := by omega
        have hihb := ih b hblt fuel' (Nat.pair k r + 1) v hble
        rw [hihb]
        have href := untagRef_succ_case (c := c) (k := k) hc
        rw [ <-  hbdef] at href
        rw [href, List.foldl_cons]
end
```

```
section
theorem untagState_spec (c : Nat) :
    untagState c = Nat.pair (untagRef c).2
      (Nat.pair 1 (Nat.pair ((untagRef c).1.foldl (fun acc x => Nat.pair x acc + 1) 0)
        (untagRef c).2)) :=
  untagStep_iter_spec c c 0 0 (le_refl c)
end

section
/-- **`decodeList (untagList c) = (untagRef c).1`.** The `appendCode`-reversal in `
    untagList`
undoes the reversal built up by `untagStep`'s accumulator. -/
theorem decodeList_untagList (c : Nat) : decodeList (untagList c) = (untagRef c).1 :=
    by
  unfold untagList
  rw [decodeList_appendCode, decodeList_zero, List.append_nil]
  have hspec := untagState_spec c
  have hrevAcc : (untagState c).unpair.2.unpair.2.unpair.1
      = (untagRef c).1.foldl (fun acc x => Nat.pair x acc + 1) 0 := by
    rw [hspec]; simp only [unpair_pair_snd, unpair_pair_fst]
  rw [hrevAcc]
  have hfold : forall  l : List Nat, forall  r : Nat,
      decodeList (l.foldl (fun acc x => Nat.pair x acc + 1) r) = l.reverse ++
    decodeList r := by
    intro l
    induction l with
    | nil => intro r; simp
    | cons a l ih =>
      intro r
      rw [List.foldl_cons, ih, decodeList_succ, unpair_pair_fst, unpair_pair_snd,
        List.reverse_cons, List.append_assoc, List.singleton_append]
  rw [hfold (untagRef c).1 0, decodeList_zero, List.append_nil, List.reverse_reverse]
end

section
/-- **`(c, j, m)  |->  untagVal c = j  /\  forall i  in  decodeList (untagList c), m.
    testBit i`.** Recursively
decidable, packed `(c,(j,m))`-style. -/
theorem untagVal_and_isComputable :
    RecDecidable (fun w : Nat => untagVal w.unpair.1 = w.unpair.2.unpair.1  /\
      forall  i  in  decodeList (untagList w.unpair.1), bitAt w.unpair.2.unpair.2 i =
    1) :=
  (RecDecidable.natEq (primrec_untagVal.comp Nat.Primrec.left)
    (Nat.Primrec.left.comp Nat.Primrec.right)).and untagList_bitAt_isComputable
end

section
/-- **`untagVal c = (untagRef c).2`.** -/
theorem untagVal_spec (c : Nat) : untagVal c = (untagRef c).2 := by
  unfold untagVal; rw [untagState_spec]; simp only [unpair_pair_snd]
end

section
/-- **`untagList`/`untagVal` genuinely decode `tag`.** -/
theorem tag_untagList_untagVal (c : Nat) : tag (decodeList (untagList c)) (untagVal c)
     = c := by
```

```
  rw [decodeList_untagList, untagVal_spec]; exact tag_untagRef c
end

section
/-- **`Fun` membership, decoded.** `Fun`'s defining existential over lists reduces to a
    bounded
existential over the finite excluded set `E_n`: given `c  in  E_n`, `tag`'s injectivity
    forces the
witness list `ns` to be *exactly* `decodeList (untagList c)`, so testing `(forall n in
   ns, n in E?)  /\  tag ns j
 in  E_n` reduces to testing `c` itself against `untagVal`/`untagList`. -/
theorem mem_Fun_compl_nbhd_iff (n m j : Nat) :
    j  in  Fun (nbhd n)? (nbhd m)?  <->
      exists  c  in  decodeList (bitsCode n n), untagVal c = j  /\
        forall  i  in  decodeList (untagList c), i  in  (nbhd m)? := by
  constructor
  * rintro <ns, hns, htag>
    have hc : tag ns j  in  decodeList (bitsCode n n) := by
      rw [mem_bitsCodeList]; exact mem_compl_nbhd.mp htag
    refine <tag ns j, hc, ?_, ?_>
    * exact (tag_injective (tag_untagList_untagVal (tag ns j))).2
    * rw [(tag_injective (tag_untagList_untagVal (tag ns j))).1]
      exact hns
  * rintro <c, hc, hval, hall>
    refine <decodeList (untagList c), hall, ?_>
    rw [mem_compl_nbhd,  <-  hval, tag_untagList_untagVal]
    exact mem_bitsCodeList.mp hc
end

section
/-- **Bounded-quantifier reformulation** of `mem_Fun_compl_nbhd_iff`, avoiding `
   decodeList`/
`bitsCode` for the outer existential (feeding directly into `RecDecidable.bExists`). -/
theorem mem_Fun_compl_nbhd_iff' (n m j : Nat) :
    j  in  Fun (nbhd n)? (nbhd m)?  <->
      exists  c, c < n  /\  n.testBit c = true  /\  untagVal c = j  /\
        forall  i  in  decodeList (untagList c), bitAt m i = 1 := by
  rw [mem_Fun_compl_nbhd_iff]
  simp only [mem_bitsCodeList, mem_compl_nbhd, bitAt_eq_one_iff]
  constructor
  * rintro <c, hbit, hval, hall>
    exact <c, lt_of_testBit_true hbit, hbit, hval, hall>
  * rintro <c, -, hbit, hval, hall>
    exact <c, hbit, hval, hall>
end

section
/-- **`j  in  Fun (nbhd n)? (nbhd m)?` is recursively decidable, jointly in `(n,j,m)
   `.** Combines
`mem_Fun_compl_nbhd_iff'` (a bounded `exists c < n`) with `RecDecidable.bExists`,
   packed `(n,(j,m))`-style. -/
theorem mem_Fun_compl_nbhd_isComputable :
    RecDecidable (fun w : Nat =>
      w.unpair.2.unpair.1  in  Fun (nbhd w.unpair.1)? (nbhd w.unpair.2.unpair.2)?) :=
   by
  have hbitN : RecDecidable (fun w : Nat => w.unpair.2.unpair.1.testBit w.unpair.1 =
```

```
    true) :=
    <fun w => bitAt w.unpair.2.unpair.1 w.unpair.1,
      (primrec_bitAt.comp ((Nat.Primrec.left.comp Nat.Primrec.right).pair Nat.Primrec.
    left)).of_eq
        fun w => by simp only [unpair_pair_fst, unpair_pair_snd],
      fun w => (bitAt_eq_one_iff _ _).symm>
  have hCJ' : RecDecidable (fun w : Nat => untagVal w.unpair.1 = w.unpair.2.unpair.2.
    unpair.1  /\
      forall  i  in  decodeList (untagList w.unpair.1), bitAt w.unpair.2.unpair.2.
    unpair.2 i = 1) :=
    RecDecidable.of_iff (fun w => by simp only [unpair_pair_fst, unpair_pair_snd])
      (untagVal_and_isComputable.comp (Nat.Primrec.left.pair (Nat.Primrec.right.comp
    Nat.Primrec.right)))
  have hp := hbitN.and hCJ'
  refine RecDecidable.of_iff (fun w => ?_) (hp.bExists Nat.Primrec.left)
  rw [mem_Fun_compl_nbhd_iff']
  simp only [unpair_pair_fst, unpair_pair_snd]
end

section
/-- **`GraphIdx` membership, decoded.** `tagOfBits n m = tag (decodeList (bitsCode n n)
    ) m` is a
*specific* choice of encoding for the pair `(E_n, m)`; since `tag` is injective and
    every code `c`
decodes (via `untagList`/`untagVal`) to *some* `tag`-preimage, `c  in  GraphIdx W`
    reduces to the
primitive-recursive check `untagList c = bitsCode n n` (pinning down `n`, by
    injectivity of
`decodeList`) together with `untagVal c` playing the role of `m`. -/
theorem mem_GraphIdx_iff (W : Set (ApproximableMap PN PN)) (c : Nat) :
    c  in  GraphIdx W  <->  exists  n m_0, untagList c = bitsCode n n  /\
      (forall  f  in  W, f.rel (nbhd n) (nbhd m_0))  /\  m_0.testBit (untagVal c) =
    true := by
  constructor
  * rintro <n, m_0, m, rfl, hf, hbit>
    have htag : tag (decodeList (untagList (tagOfBits n m))) (untagVal (tagOfBits n m)
    ) =
        tag (decodeList (bitsCode n n)) m :=
      (tag_untagList_untagVal (tagOfBits n m)).trans (tagOfBits_spec n m)
    obtain <hlist, hval> := tag_injective htag
    refine <n, m_0, decodeList_injective hlist, hf, ?_>
    rw [hval]; exact hbit
  * rintro <n, m_0, heq, hf, hbit>
    refine <n, m_0, untagVal c, ?_, hf, hbit>
    rw [tagOfBits_spec,  <-  heq]
    exact (tag_untagList_untagVal c).symm
end

section
/-- **`j  in  GraphIdx (Xenum ... c)` is recursively enumerable, jointly in `(c, j)`.**
     Unfolds via
`mem_GraphIdx_iff`/`forall_rel_iff_subset_step`/`Xenum_singleton` to `exists  n m_0, <
    primitive-recursive
check>  /\  Xenum ... c  subseteq  Xenum ... (pair (pair n m_0) 0 + 1)  /\  <bit test>`
     -- a decidable body existentially
quantified over the two witnesses `n, m_0` (`REPred.proj`, applied twice). -/
```

```
theorem graphIdx_isComputable (gN incl0 incl1 eq1 : Nat  ->  Nat)
    (hgN : forall  c, gN c = 1  <->  (stepFun (funListOf PNpres PNpres (decodeList c))
      : Set (ApproximableMap PN PN)).Nonempty) (hgNp : Nat.Primrec gN)
    (hincl0 : forall  s, incl0 s = 1  <->  PNpres.X s.unpair.1  subseteq  PNpres.X s.
   unpair.2)
    (hincl0p : Nat.Primrec incl0)
    (hincl1 : forall  s, incl1 s = 1  <->  PNpres.X s.unpair.1  subseteq  PNpres.X s.
   unpair.2)
    (hincl1p : Nat.Primrec incl1)
    (heq1 : forall  s, eq1 s = 1  <->  PNpres.X s.unpair.1 = PNpres.X s.unpair.2) (
   heq1p : Nat.Primrec eq1) :
   REPred_2 (fun c j => j  in  GraphIdx
     (Xenum PNpres PNpres gN c : Set (ApproximableMap PN PN))) := by
  have hinclP : RecDecidable_2 (fun a b =>
     (Xenum PNpres PNpres gN a : Set (ApproximableMap PN PN))  subseteq  Xenum PNpres
    PNpres gN b) :=
   (funPresentation PNpres PNpres gN incl0 incl1 eq1 hgN hgNp hincl0 hincl0p hincl1
  hincl1p heq1
     heq1p).incl_computable
  -- packed `w = <m_0, <n, <c, j>>>`; the base predicate is decidable in `w`.
  have hj : Nat.Primrec (fun w : Nat => w.unpair.2.unpair.2.unpair.2) :=
    Nat.Primrec.right.comp (Nat.Primrec.right.comp Nat.Primrec.right)
  have hn : Nat.Primrec (fun w : Nat => w.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.right
  have hc : Nat.Primrec (fun w : Nat => w.unpair.2.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp (Nat.Primrec.right.comp Nat.Primrec.right)
  have hm0 : Nat.Primrec (fun w : Nat => w.unpair.1) := Nat.Primrec.left
  have hbitsCodeArg : Nat.Primrec (fun w : Nat => bitsCode w.unpair.2.unpair.1 w.
    unpair.2.unpair.1) :=
    (primrec_bitsCode.comp (hn.pair hn)).of_eq fun w => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  have hpairArg : Nat.Primrec
      (fun w : Nat => Nat.pair (Nat.pair w.unpair.2.unpair.1 w.unpair.1) 0 + 1) :=
    Nat.Primrec.succ.comp ((hn.pair hm0).pair (Nat.Primrec.const 0))
  have hA : RecDecidable (fun w : Nat =>
      untagList w.unpair.2.unpair.2.unpair.2 =
        bitsCode w.unpair.2.unpair.1 w.unpair.2.unpair.1) :=
    RecDecidable.natEq (primrec_untagList.comp hj) hbitsCodeArg
  have hB : RecDecidable (fun w : Nat =>
      (Xenum PNpres PNpres gN w.unpair.2.unpair.2.unpair.1 : Set (ApproximableMap PN
    PN))  subseteq
        Xenum PNpres PNpres gN (Nat.pair (Nat.pair w.unpair.2.unpair.1 w.unpair.1) 0 +
     1)) :=
    RecDecidable.of_iff (fun w => by simp only [unpair_pair_fst, unpair_pair_snd])
      (hinclP.comp (hc.pair hpairArg))
  have hC : RecDecidable (fun w : Nat =>
      w.unpair.1.testBit (untagVal w.unpair.2.unpair.2.unpair.2) = true) :=
    <fun w => bitAt w.unpair.1 (untagVal w.unpair.2.unpair.2.unpair.2),
      (primrec_bitAt.comp (hm0.pair (primrec_untagVal.comp hj))).of_eq fun w => by
        simp only [unpair_pair_fst, unpair_pair_snd],
      fun w => (bitAt_eq_one_iff _ _).symm>
  have hre : REPred (fun w : Nat =>
      untagList w.unpair.2.unpair.2.unpair.2 =
        bitsCode w.unpair.2.unpair.1 w.unpair.2.unpair.1  /\
      (Xenum PNpres PNpres gN w.unpair.2.unpair.2.unpair.1 : Set (ApproximableMap PN
    PN))  subseteq
```

```
        Xenum PNpres PNpres gN (Nat.pair (Nat.pair w.unpair.2.unpair.1 w.unpair.1) 0 +
     1)  /\
      w.unpair.1.testBit (untagVal w.unpair.2.unpair.2.unpair.2) = true) :=
    (hA.and (hB.and hC)).re
  have hp2 := hre.proj.proj
  refine REPred.of_iff (fun t => ?_) hp2
  dsimp only
  rw [mem_GraphIdx_iff]
  constructor
  * rintro <n, m_0, heq, hf, hbit>
    refine <n, m_0, ?_>
    show untagList (Nat.pair m_0 (Nat.pair n t)).unpair.2.unpair.2.unpair.2 =
          bitsCode (Nat.pair m_0 (Nat.pair n t)).unpair.2.unpair.1
            (Nat.pair m_0 (Nat.pair n t)).unpair.2.unpair.1  /\
        (Xenum PNpres PNpres gN (Nat.pair m_0 (Nat.pair n t)).unpair.2.unpair.2.unpair
    .1
          : Set (ApproximableMap PN PN))  subseteq
          Xenum PNpres PNpres gN (Nat.pair
            (Nat.pair (Nat.pair m_0 (Nat.pair n t)).unpair.2.unpair.1
              (Nat.pair m_0 (Nat.pair n t)).unpair.1) 0 + 1)  /\
        (Nat.pair m_0 (Nat.pair n t)).unpair.1.testBit
          (untagVal (Nat.pair m_0 (Nat.pair n t)).unpair.2.unpair.2.unpair.2) = true
    simp only [unpair_pair_fst, unpair_pair_snd]
    refine <heq, ?_, hbit>
    rw [Xenum_singleton PNpres PNpres gN hgN n m_0]
    exact forall_rel_iff_subset_step.mp hf
  * rintro <n, m_0, hbase>
    simp only [unpair_pair_fst, unpair_pair_snd] at hbase
    obtain <heq, hsub, hbit> := hbase
    refine <n, m_0, heq, ?_, hbit>
    apply forall_rel_iff_subset_step.mpr
    rwa [Xenum_singleton PNpres PNpres gN hgN n m_0] at hsub
end

section
/-- **`graph` is computable (Exercise 7.23, dually to `gMap`).** Relative to *any*
    valid
function-space presentation `funPresentation PNpres PNpres gN incl0 incl1 eq1 ...`, `
    graphMap`'s
neighbourhood relation reduces (via `graphMap_rel_iff`) to a bounded `forall  j < m`, r
    .e. by
`graphIdx_isComputable`/`REPred.forall_mem_decodeList_2` (the bound enumerated by `
    bitsCode m m`,
`mem_bitsCodeList`). -/
theorem graphMap_isComputable (gN incl0 incl1 eq1 : Nat  ->  Nat)
    (hgN : forall  c, gN c = 1  <->  (stepFun (funListOf PNpres PNpres (decodeList c))
      : Set (ApproximableMap PN PN)).Nonempty) (hgNp : Nat.Primrec gN)
    (hincl0 : forall  s, incl0 s = 1  <->  PNpres.X s.unpair.1  subseteq  PNpres.X s.
    unpair.2)
    (hincl0p : Nat.Primrec incl0)
    (hincl1 : forall  s, incl1 s = 1  <->  PNpres.X s.unpair.1  subseteq  PNpres.X s.
    unpair.2)
    (hincl1p : Nat.Primrec incl1)
    (heq1 : forall  s, eq1 s = 1  <->  PNpres.X s.unpair.1 = PNpres.X s.unpair.2) (
    heq1p : Nat.Primrec eq1) :
    ComputableMap
```

```
      (funPresentation PNpres PNpres gN incl0 incl1 eq1 hgN hgNp hincl0 hincl0p hincl1
     hincl1p
        heq1 heq1p)
      PNpres graphMap := by
  constructor
  have hidx := graphIdx_isComputable gN incl0 incl1 eq1 hgN hgNp hincl0 hincl0p hincl1
     hincl1p
    heq1 heq1p
  have hforall : REPred (fun t : Nat => forall  j  in  decodeList t.unpair.2,
      j  in  GraphIdx (Xenum PNpres PNpres gN t.unpair.1 : Set (ApproximableMap PN PN)
    )) :=
    REPred.forall_mem_decodeList_2 hidx
  have hreidx : Nat.Primrec (fun s : Nat => Nat.pair s.unpair.1 (bitsCode s.unpair.2 s
    .unpair.2)) :=
    Nat.Primrec.left.pair ((primrec_bitsCode.comp (Nat.Primrec.right.pair
      Nat.Primrec.right)).of_eq fun s => by simp only [unpair_pair_fst,
    unpair_pair_snd])
  refine REPred.of_iff (fun s => ?_) (hforall.comp hreidx)
  simp only [unpair_pair_fst, unpair_pair_snd]
  exact (graphMap_rel_iff gN hgN s.unpair.1 s.unpair.2).trans (by
    rw [compl_nbhd_subset_iff]
    exact <fun h j hj => h j (lt_of_testBit_true (mem_bitsCodeList.mp hj))
        (mem_bitsCodeList.mp hj),
      fun h j hjlt hjbit => h j (mem_bitsCodeList.mpr hjbit)>)
end

section
set_option linter.unusedVariables false

theorem example_723 : forall  (gN incl0 incl1 eq1 : Nat  ->  Nat)
  (hgN :
    forall  (c : Nat),
      Iff (@Eq Nat (gN c) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))))
        (@Set.Nonempty
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap Nat Nat DomainTheory.
    NeighbourhoodSystems.Example78.PN
            DomainTheory.NeighbourhoodSystems.Example78.PN)
          (@DomainTheory.NeighbourhoodSystems.stepFun Nat Nat DomainTheory.
    NeighbourhoodSystems.Example78.PN
            DomainTheory.NeighbourhoodSystems.Example78.PN
            (@DomainTheory.NeighbourhoodSystems.funListOf Nat Nat DomainTheory.
    NeighbourhoodSystems.Example78.PN
              DomainTheory.NeighbourhoodSystems.Example78.PN DomainTheory.
    NeighbourhoodSystems.Example78.PNpres
              DomainTheory.NeighbourhoodSystems.Example78.PNpres (Domain.Recursive.
    decodeList c)))))
  (hgNp : Nat.Primrec gN)
  (hincl0 :
    forall  (s : Nat),
      Iff (@Eq Nat (incl0 s) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))
    )
        (@LE.le (Set Nat) (@Set.instLE Nat)
          (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X Nat
            DomainTheory.NeighbourhoodSystems.Example78.PN DomainTheory.
    NeighbourhoodSystems.Example78.PNpres
            (@Prod.fst Nat Nat (Nat.unpair s)))
```

```
          (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X Nat
            DomainTheory.NeighbourhoodSystems.Example78.PN DomainTheory.
    NeighbourhoodSystems.Example78.PNpres
             (@Prod.snd Nat Nat (Nat.unpair s)))))
  (hincl0p : Nat.Primrec incl0)
  (hincl1 :
    forall  (s : Nat),
      Iff (@Eq Nat (incl1 s) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1)))
    )
        (@LE.le (Set Nat) (@Set.instLE Nat)
          (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X Nat
            DomainTheory.NeighbourhoodSystems.Example78.PN DomainTheory.
    NeighbourhoodSystems.Example78.PNpres
             (@Prod.fst Nat Nat (Nat.unpair s)))
          (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X Nat
            DomainTheory.NeighbourhoodSystems.Example78.PN DomainTheory.
    NeighbourhoodSystems.Example78.PNpres
             (@Prod.snd Nat Nat (Nat.unpair s)))))
  (hincl1p : Nat.Primrec incl1)
  (heq1 :
    forall  (s : Nat),
      Iff (@Eq Nat (eq1 s) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))))
        (@Eq (Set Nat)
          (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X Nat
            DomainTheory.NeighbourhoodSystems.Example78.PN DomainTheory.
    NeighbourhoodSystems.Example78.PNpres
             (@Prod.fst Nat Nat (Nat.unpair s)))
          (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X Nat
            DomainTheory.NeighbourhoodSystems.Example78.PN DomainTheory.
    NeighbourhoodSystems.Example78.PNpres
             (@Prod.snd Nat Nat (Nat.unpair s)))))
  (heq1p : Nat.Primrec eq1),
  @DomainTheory.NeighbourhoodSystems.ComputableMap
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap Nat Nat DomainTheory.
    NeighbourhoodSystems.Example78.PN
      DomainTheory.NeighbourhoodSystems.Example78.PN)
    Nat
    (@DomainTheory.NeighbourhoodSystems.funSpace Nat Nat DomainTheory.
    NeighbourhoodSystems.Example78.PN
      DomainTheory.NeighbourhoodSystems.Example78.PN)
    DomainTheory.NeighbourhoodSystems.Example78.PN

    (@DomainTheory.NeighbourhoodSystems.funPresentation Nat Nat DomainTheory.
    NeighbourhoodSystems.Example78.PN
      DomainTheory.NeighbourhoodSystems.Example78.PN DomainTheory.NeighbourhoodSystems
    .Example78.PNpres
      DomainTheory.NeighbourhoodSystems.Example78.PNpres gN incl0 incl1 eq1 hgN hgNp
    hincl0 hincl0p hincl1 hincl1p heq1
      heq1p)
    DomainTheory.NeighbourhoodSystems.Example78.PNpres DomainTheory.
    NeighbourhoodSystems.Exercise723.graphMap :=
  DomainTheory.NeighbourhoodSystems.Exercise723.graphMap_isComputable
end

section
/-- **The implication guarding a single bit of `nbhd k`'s excluded set is recursively
```

```
    decidable,
jointly in `(j,n,m,k)`.** Packed `(j,((n,m),k))`-style. -/
theorem testBit_imp_mem_Fun_isComputable :
    RecDecidable (fun w : Nat => w.unpair.2.unpair.2.testBit w.unpair.1 = true  ->
      w.unpair.1  in  Fun (nbhd w.unpair.2.unpair.1.unpair.1)? (nbhd w.unpair.2.unpair
    .1.unpair.2)?) := by
  have hktest : RecDecidable (fun w : Nat => w.unpair.2.unpair.2.testBit w.unpair.1 =
    true) :=
    <fun w => bitAt w.unpair.2.unpair.2 w.unpair.1,
      (primrec_bitAt.comp ((Nat.Primrec.right.comp Nat.Primrec.right).pair Nat.Primrec
    .left)).of_eq
        fun w => by simp only [unpair_pair_fst, unpair_pair_snd],
      fun w => (bitAt_eq_one_iff _ _).symm>
  have hmem2 : RecDecidable (fun w : Nat =>
      w.unpair.1  in  Fun (nbhd w.unpair.2.unpair.1.unpair.1)? (nbhd w.unpair.2.unpair
    .1.unpair.2)?) :=
    RecDecidable.of_iff (fun w => by simp only [unpair_pair_fst, unpair_pair_snd])
      (mem_Fun_compl_nbhd_isComputable.comp
        ((Nat.Primrec.left.comp (Nat.Primrec.left.comp Nat.Primrec.right)).pair
          (Nat.Primrec.left.pair (Nat.Primrec.right.comp (Nat.Primrec.left.comp Nat.
    Primrec.right)))))
  refine RecDecidable.of_iff (fun w => ?_) (hktest.not.or hmem2)
  constructor
  * intro h
    rcases hktest.em w with hb | hb
    * exact Or.inr (h hb)
    * exact Or.inl hb
  * rintro (hb | hb) hbit
    * exact absurd hbit hb
    * exact hb
end

section
/-- **`gMap` is computable.** The neighbourhood relation `gMap_rel_iff` reduces (via
`compl_nbhd_subset_Fun_iff`) to a bounded `forall j < k`, primitive recursive by
`testBit_imp_mem_Fun_isComputable`/`RecDecidable.bForall`. -/
theorem gMap_isComputable :
    ComputableMap (prodPresentation PNpres PNpres) PNpres gMap := by
  constructor
  have hall : RecDecidable (fun u : Nat => forall  j, j < u.unpair.2  ->
      u.unpair.2.testBit j = true  ->
        j  in  Fun (nbhd u.unpair.1.unpair.1)? (nbhd u.unpair.1.unpair.2)?) :=
    RecDecidable.of_iff (fun u => by simp only [unpair_pair_fst, unpair_pair_snd])
      (testBit_imp_mem_Fun_isComputable.bForall Nat.Primrec.right)
  refine (RecDecidable.of_iff (fun t => ?_) hall).re
  simp only [prodPresentation_X]
  exact (gMap_rel_iff _ _ _).trans (compl_nbhd_subset_Fun_iff _ _ _)
end

section
/-- **`fun` is computable**, relative to *any* valid function-space presentation
`funPresentation PNpres PNpres gN incl0 incl1 eq1 ...` (Theorem 7.5's generic `
    curry_isComputable`,
fed `gMap_isComputable`). Every such presentation exists (`funSpace_isEffectivelyGiven
    `); this
result is stated generically over the witnessing deciders, exactly as `
```

```
    curry_isComputable` itself
is, rather than committing to one concrete choice of `gN`/`incl0`/`incl1`/`eq1` for `PN
    `. -/
theorem funMap_isComputable (gN incl0 incl1 eq1 : Nat  ->  Nat)
    (hgN : forall  c, gN c = 1  <->  (stepFun (funListOf PNpres PNpres (decodeList c))
      : Set (ApproximableMap PN PN)).Nonempty) (hgNp : Nat.Primrec gN)
    (hincl0 : forall  s, incl0 s = 1  <->  PNpres.X s.unpair.1  subseteq  PNpres.X s.
    unpair.2)
    (hincl0p : Nat.Primrec incl0)
    (hincl1 : forall  s, incl1 s = 1  <->  PNpres.X s.unpair.1  subseteq  PNpres.X s.
    unpair.2)
    (hincl1p : Nat.Primrec incl1)
    (heq1 : forall  s, eq1 s = 1  <->  PNpres.X s.unpair.1 = PNpres.X s.unpair.2) (
    heq1p : Nat.Primrec eq1) :
    ComputableMap PNpres
      (funPresentation PNpres PNpres gN incl0 incl1 eq1 hgN hgNp hincl0 hincl0p hincl1
     hincl1p
        heq1 heq1p)
      funMap :=
  curry_isComputable PNpres PNpres PNpres gN incl0 incl1 eq1 hgN hgNp hincl0 hincl0p
    hincl1
    hincl1p heq1 heq1p gMap_isComputable
end

end DomainTheory.NeighbourhoodSystems.Exercise723
end
```

3.8.26 Exercise 7.24 — LUCID history domain

(Suggested by the LUCID language of Ashcroft and Wadge: SIAM Jour. Comp. vol. 5 (1976).)

Define a set Γ by

$$\Gamma = \bigcup_{i=0}^{\infty} (\{i\} \times \Gamma) \cup \{\star\}.$$

Define a system

$$L = \{\Gamma\} \cup \{\{i\} \times X \mid i \in \mathbf{N} \text{ and } X \in L\}.$$

Show that L is effectively given. Show that the elements of $|L|$ can be identified with the finite and infinite sequences of natural numbers. What is the connection between B and L?

Show that the combinators of LUCID can be construed as computable mappings of type

$$(L \to T) \to (L \to T)$$

or of type

$$(L \to T) \times (L \to T) \to (L \to T).$$

Conclude that programs in LUCID define computable maps.

Exercise_7_24.lean

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Theorem_7_5
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Proposition_7_12
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Proposition_7_3
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_8
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_7

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems.Exercise724
open NeighborhoodSystem Domain.Recursive

section
/-- Scott's set `Gamma`, the *terminal* solution of `Gamma = (?_i {i}  x  Gamma)  U
    {*}`: a finite sequence,
terminating at `*` (`Sum.inl`), or an infinite one that never terminates (`Sum.inr`).
    -/
abbrev Gamma : Type := List Nat ? (Nat  ->  Nat)
end

section
/-- Scott's basepoint `*`: the empty (already-terminated) sequence. -/
def star : Gamma := Sum.inl []
end

section
/-- Prepend `i` to an infinite stream (`Stream'.cons`, spelled out by hand). -/
def streamCons (i : Nat) (s : Nat  ->  Nat) : Nat  ->  Nat
  | 0 => i
  | n + 1 => s n
end

section
@[simp] theorem streamCons_zero (i : Nat) (s : Nat  ->  Nat) : streamCons i s 0 = i :=
     rfl
end

section
@[simp] theorem streamCons_succ (i : Nat) (s : Nat  ->  Nat) (n : Nat) : streamCons i
    s (n + 1) = s n := rfl
end

section
/-- Scott's `(i, ?)`: prepend the natural `i` to a finite-or-infinite sequence `?`. -/
def cons (i : Nat) : Gamma  ->  Gamma
  | .inl l => .inl (i :: l)
  | .inr s => .inr (streamCons i s)
end

section
```

```
/-- Scott's `{i}  x  X`: the `i`-prefixed copy of a neighbourhood `X`. -/
def consSet (i : Nat) (X : Set Gamma) : Set Gamma := {z | exists  ?  in  X, z = cons i
     ?}
end

section
/-- **`cons` is injective** (jointly in both arguments): `(i, ?) = (j, ?) ? i = j  /\
    ? = ?`. -/
theorem cons_injective {i j : Nat} {? ? : Gamma} (h : cons i ? = cons j ?) : i = j  /\
      ? = ? := by
  cases ? with
  | inl l =>
    cases ? with
    | inl l' =>
      simp only [cons, Sum.inl.injEq, List.cons.injEq] at h
      exact <h.1, by rw [h.2]>
    | inr s' => simp [cons] at h
  | inr s =>
    cases ? with
    | inl l' => simp [cons] at h
    | inr s' =>
      have heq : streamCons i s = streamCons j s' := by simpa [cons] using h
      have hi : i = j := by simpa using congrFun heq 0
      refine <hi, ?_>
      have hs : s = s' := funext fun n => by simpa using congrFun heq (n + 1)
      rw [hs]
end

section
@[simp] theorem mem_consSet {i : Nat} {X : Set Gamma} {z : Gamma} :
    z  in  consSet i X  <->  exists  ?  in  X, z = cons i ? := Iff.rfl
end

section
@[simp] theorem consSet_empty (i : Nat) : consSet i ({} : Set Gamma) = {} := by
  ext z; simp [mem_consSet]
end

section
theorem consSet_inter (i : Nat) (X X' : Set Gamma) :
    consSet i X  I  consSet i X' = consSet i (X  I  X') := by
  ext z
  simp only [Set.mem_inter_iff, mem_consSet]
  constructor
  * rintro <<?, h?, rfl>, ?', h?', heq>
    obtain <-, h??'> := cons_injective heq
    exact <?, <h?, h??'  |>  h?'>, rfl>
  * rintro <?, <h?, h?'>, rfl>
    exact <<?, h?, rfl>, <?, h?', rfl>>
end

section
theorem consSet_inter_ne {i j : Nat} (h : i  !=  j) (X Y : Set Gamma) :
    consSet i X  I  consSet j Y = {} := by
  ext z
  simp only [Set.mem_inter_iff, mem_consSet, Set.mem_empty_iff_false, iff_false,
```

```
    not_and]
  rintro <?, -, rfl> <?', -, heq>
  exact h (cons_injective heq).1
end

section
theorem consSet_nonempty {i : Nat} {X : Set Gamma} (h : X.Nonempty) : (consSet i X).
    Nonempty := by
  obtain <?, h?> := h; exact <cons i ?, ?, h?, rfl>
end

section
theorem consSet_subset_of_subset {i : Nat} {X X' : Set Gamma} (h : X  subseteq  X') :
    consSet i X  subseteq  consSet i X' := by
  rintro z <?, h?, rfl>; exact <?, h h?, rfl>
end

section
/-- **`{i}  x  X  subseteq  {i}  x  X'  <->  X  subseteq  X'`.** -/
theorem consSet_subset {i : Nat} {X X' : Set Gamma} :
    consSet i X  subseteq  consSet i X'  <->  X  subseteq  X' := by
  refine <fun h ? h? => ?_, consSet_subset_of_subset>
  obtain <?', h?', heq> := h <?, h?, rfl>
  rwa [(cons_injective heq.symm).2] at h?'
end

section
/-- **The neighbourhood coded by a finite list.** `nbhd [] = Gamma` (Scott's master `
    Gamma`);
`nbhd (i :: l) = {i}  x  nbhd l`. Every `Lmem`-neighbourhood is `nbhd l` for a *unique*
     `l`
(`nbhd_injective`), mirroring `ExampleB.cone`/`Example62.embBit` generalized to `Nat`-
    ary
branching. -/
def nbhd : List Nat  ->  Set Gamma
  | [] => Set.univ
  | i :: l => consSet i (nbhd l)
end

section
@[simp] theorem nbhd_nil : nbhd [] = (Set.univ : Set Gamma) := rfl
end

section
@[simp] theorem nbhd_cons (i : Nat) (l : List Nat) : nbhd (i :: l) = consSet i (nbhd l
    ) := rfl
end

section
/-- Membership in Scott's system `L = {Gamma}  U  {{i}  x  X ? i  in  Nat, X  in  L}`:
    `X  in  L` iff `X = nbhd l`
for some finite list `l` (the "address" of the neighbourhood). -/
def Lmem (X : Set Gamma) : Prop := exists  l, X = nbhd l
end

section
```

```
@[simp] theorem mem_nbhd (l : List Nat) : Lmem (nbhd l) := <l, rfl>
end

section
/-- **`L` is nested-or-disjoint** (Scott's "very special circumstance", generalizing
`ExampleB.cone_trichotomy`/`Example62`'s pairwise structure from `Bool` to `Nat`-ary
    branching). -/
theorem L_nestedOrDisjoint : NestedOrDisjoint Lmem := by
  rintro X Y <l, rfl> <l', rfl>
  induction l generalizing l' with
  | nil => exact Or.inr (Or.inl (by simp))
  | cons i s ih =>
    cases l' with
    | nil => exact Or.inl (by simp)
    | cons j t =>
      by_cases hij : i = j
      * subst hij
        rcases ih t with h1 | h2 | h3
        * exact Or.inl (by rw [nbhd_cons, nbhd_cons]; exact consSet_subset_of_subset
    h1)
        * exact Or.inr (Or.inl (by rw [nbhd_cons, nbhd_cons]; exact
    consSet_subset_of_subset h2))
        * exact Or.inr (Or.inr (by rw [nbhd_cons, nbhd_cons, consSet_inter, h3,
    consSet_empty]))
      * exact Or.inr (Or.inr (by rw [nbhd_cons, nbhd_cons]; exact consSet_inter_ne hij
     _ _))
end

end DomainTheory.NeighbourhoodSystems.Exercise724

section
namespace DomainTheory.NeighbourhoodSystems.Exercise724
open NeighborhoodSystem Domain.Recursive
section
/- concordance: source L4739 EXERCISE 7.24 LUCID history domain -/
/-- **Exercise 7.24 -- `L` is a neighbourhood system** on tokens `Gamma`, master `Gamma
     = nbhd []`. -/
abbrev L : NeighborhoodSystem Gamma :=
  NeighborhoodSystem.ofNestedOrDisjoint Lmem Set.univ Set.univ_nonempty
    (mem_nbhd []) L_nestedOrDisjoint
    (fun {X} _ => Set.subset_univ X)
end

section
/-- The enumeration `Lenum n := nbhd (decodeList n)`: every code `n` names the
    neighbourhood
addressed by its decoded list, and (via `encodeList`/`decodeList` round-tripping) this
    hits every
`Lmem` set. -/
def Lenum (n : Nat) : Set Gamma := nbhd (decodeList n)
end

section
@[simp] theorem Lenum_mem (n : Nat) : Lmem (Lenum n) := mem_nbhd (decodeList n)
end
```

```
section
theorem Lenum_surj {Y : Set Gamma} (hY : Lmem Y) : exists  n, Lenum n = Y := by
  obtain <l, rfl> := hY
  exact <encodeList l, by unfold Lenum; rw [decodeList_encodeList]>
end

section
/-- The master neighbourhood is coded by `0` (`decodeList 0 = []`). -/
theorem Lenum_zero : Lenum 0 = Set.univ := by unfold Lenum; rw [decodeList_zero]; rfl
end

section
/-- **The `{0,1}` prefix test.** `isPrefixChar m n = 1  <->  decodeList m <+:
    decodeList n`, built from
`Recursive.lean`'s `listLenChar`/`takeCode`/`listEqChar` (already primitive recursive):
     `l_1 <+: l_2`
iff `l_2.take l_1.length = l_1` (`List.prefix_iff_eq_take`), unconditionally (`List.
    take` auto-caps at
the available length, so no separate length guard is needed). -/
def isPrefixChar (m n : Nat) : Nat := listEqChar (takeCode (listLenChar m) n) m
end

section
/-- **The `{0,1}` consistency characteristic** -- a flat `orBit` of the two `
    isPrefixChar`
directions (kept as a single hand-rolled arithmetic combinator rather than composing
`RecDecidable.or`/`.and`, per the project's course-of-values discipline: nesting the
    generic
`RecDecidable` combinators here elaborates a deeply-repeated composition and blows the
    `whnf`
budget). -/
def LenumConsChar (n m : Nat) : Nat :=
  isPrefixChar m n + isPrefixChar n m - isPrefixChar m n * isPrefixChar n m
end

section
/-- Named-function packaging of `LenumConsChar` on `Nat.pair`-coded pairs (same
    rationale as
`LenumSubsetChar` above: avoids elaborator defeq-unification against `Nat.pair`/`Nat.
    unpair`). -/
def LenumConsCharP (t : Nat) : Nat := LenumConsChar t.unpair.1 t.unpair.2
end

section
def LenumInter (n m : Nat) : Nat := selectFn (isPrefixChar m n) n m
end

section
/-- **The `{0,1}` intersection-equality characteristic** -- same flat-`orBit`
    discipline as
`LenumConsChar`. -/
def LenumInterEqChar (n m k : Nat) : Nat :=
  mulBit (isPrefixChar m n) (listEqChar k n) + mulBit (isPrefixChar n m) (listEqChar k
     m)
    - mulBit (isPrefixChar m n) (listEqChar k n) * mulBit (isPrefixChar n m) (
    listEqChar k m)
```

```
end

section
/-- Named-function packaging of `LenumInterEqChar` on `Nat.pair (Nat.pair)`-coded
    triples (same
rationale as `LenumSubsetChar`/`LenumConsCharP` above). -/
def LenumInterEqCharP (t : Nat) : Nat :=
  LenumInterEqChar t.unpair.1 t.unpair.2.unpair.1 t.unpair.2.unpair.2
end

section
theorem isPrefixChar_eq_one_iff (m n : Nat) :
    isPrefixChar m n = 1  <->  decodeList m <+: decodeList n := by
  unfold isPrefixChar
  rw [listEqChar_eq_one_iff, takeCode_eq, listLenChar_eq, List.prefix_iff_eq_take]
  exact eq_comm
end

section
theorem isPrefixChar_le_one (m n : Nat) : isPrefixChar m n  <=  1 := listEqChar_le_one
     _ _
end

section
theorem isPrefixChar_eq_zero_or_one (m n : Nat) :
    isPrefixChar m n = 0  \/  isPrefixChar m n = 1 := by
  have := isPrefixChar_le_one m n; omega
end

section
theorem LenumConsChar_eq_one_iff (n m : Nat) :
    LenumConsChar n m = 1  <->  isPrefixChar m n = 1  \/  isPrefixChar n m = 1 := by
  unfold LenumConsChar
  rcases isPrefixChar_eq_zero_or_one m n with ha | ha <;>
    rcases isPrefixChar_eq_zero_or_one n m with hb | hb <;> simp [ha, hb]
end

section
theorem LenumConsChar_le_one (n m : Nat) : LenumConsChar n m  <=  1 := by
  unfold LenumConsChar
  rcases isPrefixChar_eq_zero_or_one m n with ha | ha <;>
    rcases isPrefixChar_eq_zero_or_one n m with hb | hb <;> simp [ha, hb]
end

section
theorem LenumConsCharP_le_one (t : Nat) : LenumConsCharP t  <=  1 :=
  LenumConsChar_le_one _ _
end

section
theorem mulBit_eq_zero_or_one {a b : Nat} (ha : a  <=  1) (hb : b  <=  1) :
    mulBit a b = 0  \/  mulBit a b = 1 := by
  have := mulBit_le_one ha hb; omega
end

section
```

```
theorem LenumInterEqChar_eq_one_iff (n m k : Nat) :
    LenumInterEqChar n m k = 1  <->
      (isPrefixChar m n = 1  /\  decodeList k = decodeList n)  \/
      (isPrefixChar n m = 1  /\  decodeList k = decodeList m) := by
  have hstep : LenumInterEqChar n m k = 1  <->
      mulBit (isPrefixChar m n) (listEqChar k n) = 1  \/
      mulBit (isPrefixChar n m) (listEqChar k m) = 1 := by
    unfold LenumInterEqChar
    rcases mulBit_eq_zero_or_one (isPrefixChar_le_one m n) (listEqChar_le_one k n)
    with ha | ha <;>
      rcases mulBit_eq_zero_or_one (isPrefixChar_le_one n m) (listEqChar_le_one k m)
    with hb | hb <;>
        simp [ha, hb]
  rw [hstep, mulBit_eq_one_iff, mulBit_eq_one_iff, listEqChar_eq_one_iff,
    listEqChar_eq_one_iff]
end

section
theorem LenumInterEqChar_le_one (n m k : Nat) : LenumInterEqChar n m k  <=  1 := by
  unfold LenumInterEqChar
  rcases mulBit_eq_zero_or_one (isPrefixChar_le_one m n) (listEqChar_le_one k n) with
    ha | ha <;>
    rcases mulBit_eq_zero_or_one (isPrefixChar_le_one n m) (listEqChar_le_one k m)
    with hb | hb <;>
      simp [ha, hb]
end

section
theorem LenumInterEqCharP_le_one (t : Nat) : LenumInterEqCharP t  <=  1 :=
  LenumInterEqChar_le_one _ _ _
end

section
theorem nbhd_nonempty : forall  l : List Nat, (nbhd l).Nonempty
  | [] => <star, trivial>
  | _ :: l => consSet_nonempty (nbhd_nonempty l)
end

section
theorem primrec_isPrefixChar : Nat.Primrec (fun t => isPrefixChar t.unpair.1 t.unpair
    .2) := by
  have hlen : Nat.Primrec (fun t : Nat => listLenChar t.unpair.1) :=
    primrec_listLenChar.comp Nat.Primrec.left
  have htake : Nat.Primrec (fun t : Nat => takeCode (listLenChar t.unpair.1) t.unpair
    .2) :=
    (primrec_takeCode.comp (Nat.Primrec.pair hlen Nat.Primrec.right)).of_eq fun t =>
    by
      simp only [unpair_pair_fst, unpair_pair_snd]
  exact (primrec_listEqChar.comp (Nat.Primrec.pair htake Nat.Primrec.left)).of_eq fun
    t => by
    simp only [isPrefixChar, unpair_pair_fst, unpair_pair_snd]
end

section
theorem primrec_LenumInterEqChar :
    Nat.Primrec (fun t => LenumInterEqChar t.unpair.1 t.unpair.2.unpair.1 t.unpair.2.
```

```
    unpair.2) := by
  have hn : Nat.Primrec (fun t : Nat => t.unpair.1) := Nat.Primrec.left
  have hm : Nat.Primrec (fun t : Nat => t.unpair.2.unpair.1) := Nat.Primrec.left.comp
    Nat.Primrec.right
  have hk : Nat.Primrec (fun t : Nat => t.unpair.2.unpair.2) := Nat.Primrec.right.comp
     Nat.Primrec.right
  have hpmn : Nat.Primrec (fun t : Nat => isPrefixChar t.unpair.2.unpair.1 t.unpair.1)
     :=
    (primrec_isPrefixChar.comp (Nat.Primrec.pair hm hn)).of_eq fun t => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  have hpnm : Nat.Primrec (fun t : Nat => isPrefixChar t.unpair.1 t.unpair.2.unpair.1)
     :=
    (primrec_isPrefixChar.comp (Nat.Primrec.pair hn hm)).of_eq fun t => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  have hekn : Nat.Primrec (fun t : Nat => listEqChar t.unpair.2.unpair.2 t.unpair.1)
    :=
    (primrec_listEqChar.comp (Nat.Primrec.pair hk hn)).of_eq fun t => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  have hekm : Nat.Primrec (fun t : Nat => listEqChar t.unpair.2.unpair.2 t.unpair.2.
    unpair.1) :=
    (primrec_listEqChar.comp (Nat.Primrec.pair hk hm)).of_eq fun t => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  have ha : Nat.Primrec (fun t : Nat => mulBit (isPrefixChar t.unpair.2.unpair.1 t.
    unpair.1)
      (listEqChar t.unpair.2.unpair.2 t.unpair.1)) :=
    (primrec_mulBit.comp (Nat.Primrec.pair hpmn hekn)).of_eq fun t => by simp only [
    unpair_pair_fst, unpair_pair_snd]
  have hb : Nat.Primrec (fun t : Nat => mulBit (isPrefixChar t.unpair.1 t.unpair.2.
    unpair.1)
      (listEqChar t.unpair.2.unpair.2 t.unpair.2.unpair.1)) :=
    (primrec_mulBit.comp (Nat.Primrec.pair hpnm hekm)).of_eq fun t => by simp only [
    unpair_pair_fst, unpair_pair_snd]
  exact (primrec_sub_2 (primrec_add_2 ha hb) (primrec_mul_2 ha hb)).of_eq fun t => by
    simp only [LenumInterEqChar]
end

section
theorem primrec_LenumInterEqCharP : Nat.Primrec LenumInterEqCharP :=
  primrec_LenumInterEqChar.of_eq fun t => by simp only [LenumInterEqCharP]
end

section
/-- `isPrefixChar` composed with the swap of `unpair`, built via `.of_eq` (rather than
    relying
on elaborator defeq-unification against `Nat.pair`/`Nat.unpair`, which is expensive and
     can blow
the `whnf` budget). -/
theorem primrec_isPrefixChar_swap :
    Nat.Primrec (fun t => isPrefixChar t.unpair.2 t.unpair.1) :=
  (primrec_isPrefixChar.comp (Nat.Primrec.pair Nat.Primrec.right Nat.Primrec.left)).
    of_eq
    fun t => by simp only [unpair_pair_fst, unpair_pair_snd]
end

section
theorem primrec_LenumConsChar :
```

```
    Nat.Primrec (fun t => LenumConsChar t.unpair.1 t.unpair.2) := by
  have ha : Nat.Primrec (fun t : Nat => isPrefixChar t.unpair.2 t.unpair.1) :=
    primrec_isPrefixChar_swap
  have hb : Nat.Primrec (fun t : Nat => isPrefixChar t.unpair.1 t.unpair.2) :=
    primrec_isPrefixChar
  exact (primrec_sub_2 (primrec_add_2 ha hb) (primrec_mul_2 ha hb)).of_eq fun t => by
    simp only [LenumConsChar]
end

section
theorem primrec_LenumConsCharP : Nat.Primrec LenumConsCharP :=
  primrec_LenumConsChar.of_eq fun t => by simp only [LenumConsCharP]
end

section
theorem primrec_LenumInter : Nat.Primrec (fun t => LenumInter t.unpair.1 t.unpair.2)
    :=
  (primrec_selectFn primrec_isPrefixChar_swap Nat.Primrec.left Nat.Primrec.right).
    of_eq


    fun t => by simp only [LenumInter]
end

section
/-- `*` is never of the shape `(i, ?)`. -/
theorem star_ne_cons (i : Nat) (? : Gamma) : star  !=  cons i ? := by
  cases ? <;> simp [star, cons]
end

section
theorem star_not_mem_consSet (i : Nat) (X : Set Gamma) : star  notin  consSet i X :=
    by
  rintro <?, -, heq>; exact star_ne_cons i ? heq
end

section
/-- **The master reduction (mirrors `ExampleB.cone_subset_cone`).** `nbhd l  subseteq
    nbhd l'  <->  l' <+: l`:
cones reverse the prefix order. Proved by induction on `l`, peeling off matching heads.
    -/
theorem nbhd_subset_iff : forall  (l l' : List Nat), nbhd l  subseteq  nbhd l'  <->  l
    ' <+: l := by
  intro l
  induction l with
  | nil =>
    intro l'
    cases l' with
    | nil => simp
    | cons j t =>
      constructor
      * intro h
        exact absurd (h (Set.mem_univ star)) (by rw [nbhd_cons]; exact
    star_not_mem_consSet j (nbhd t))
      * rintro <u, hu>
        simp at hu
  | cons i s ih =>
```

```
    intro l'
    cases l' with
    | nil => simp [nbhd_cons]
    | cons j t =>
      by_cases hij : i = j
      * subst hij
        rw [nbhd_cons, nbhd_cons, consSet_subset, ih t, List.cons_prefix_cons]
        simp
      * constructor
        * intro h
          exfalso
          obtain <?, h?> := nbhd_nonempty s
          have hmem : cons i ?  in  consSet j (nbhd t) := h <?, h?, rfl>
          obtain <?', -, heq> := hmem
          exact hij (cons_injective heq).1
        * intro h
          rw [List.cons_prefix_cons] at h
          exact absurd h.1.symm hij
end

section
/-- **`X_n  subseteq  X?` on the `Lenum` coding, decided by `isPrefixChar`.** -/
theorem Lenum_subset_iff (n m : Nat) : Lenum n  subseteq  Lenum m  <->  isPrefixChar m
     n = 1 := by
  unfold Lenum; rw [nbhd_subset_iff, isPrefixChar_eq_one_iff]
end

section
/-- **7.1(ii) -- consistency is recursively decidable.** `exists  k, Lenum k  subseteq
     Lenum n  I  Lenum m` holds
iff `n, m` are nested (`isPrefixChar m n = 1  \/  isPrefixChar n m = 1`): if nested,
    the more specific
of `n, m` itself witnesses it; conversely a common lower bound forces nestedness by
`L_nestedOrDisjoint` (a properly disjoint pair has no nonempty common lower bound, and
    every
`Lmem` set is nonempty). -/
theorem Lenum_cons_iff (n m : Nat) :
    (exists  k, Lenum k  subseteq  Lenum n  I  Lenum m)  <->  isPrefixChar m n = 1  \/
      isPrefixChar n m = 1 := by
  constructor
  * rintro <k, hk>
    rcases L_nestedOrDisjoint (mem_nbhd (decodeList n)) (mem_nbhd (decodeList m)) with
     h | h | h
    * exact Or.inl ((isPrefixChar_eq_one_iff m n).mpr ((nbhd_subset_iff _ _).mp h))
    * exact Or.inr ((isPrefixChar_eq_one_iff n m).mpr ((nbhd_subset_iff _ _).mp h))
    * exfalso
      unfold Lenum at hk
      obtain <t, ht> := nbhd_nonempty (decodeList k)
      exact Set.notMem_empty t (h  |>  hk ht)
  * rintro (hp | hp)
    * exact <n, Set.subset_inter subset_rfl ((Lenum_subset_iff n m).mpr hp)>
    * exact <m, Set.subset_inter ((Lenum_subset_iff m n).mpr hp) subset_rfl>
end

section
theorem LenumInter_spec {n m : Nat} (h : exists  k, Lenum k  subseteq  Lenum n  I
```

```
    Lenum m) :
    Lenum (LenumInter n m) = Lenum n  I  Lenum m := by
  unfold LenumInter
  rcases isPrefixChar_eq_zero_or_one m n with h0 | h1
  * rw [h0, selectFn_zero]
    have hp : isPrefixChar n m = 1 := by
      rcases (Lenum_cons_iff n m).mp h with hp | hp
      * rw [h0] at hp; exact absurd hp (by decide)
      * exact hp
    exact (Set.inter_eq_right.mpr ((Lenum_subset_iff m n).mpr hp)).symm
  * rw [h1, selectFn_one]
    exact (Set.inter_eq_left.mpr ((Lenum_subset_iff n m).mpr h1)).symm
end

section
theorem Lenum_cons_charP_eq_one_iff (n m : Nat) :
    (exists  k, Lenum k  subseteq  Lenum n  I  Lenum m)  <->  LenumConsCharP (Nat.pair
     n m) = 1 := by
  unfold LenumConsCharP
  rw [unpair_pair_fst, unpair_pair_snd, LenumConsChar_eq_one_iff]
  exact Lenum_cons_iff n m
end

section
/-- **7.1(ii) -- consistency is recursively decidable.** -/
theorem Lenum_cons_computable : RecDecidable_2 (fun n m => exists  k, Lenum k
    subseteq  Lenum n  I  Lenum m) :=
  RecDecidable_2.of_paired_zero_one_char primrec_LenumConsCharP
    (fun t => by have := LenumConsCharP_le_one t; omega)
    Lenum_cons_charP_eq_one_iff
end

section
/-- **`nbhd` is injective.** Antisymmetry of `nbhd_subset_iff`: `l' <+: l` and `l <+: l
    '` force
`l.length = l'.length`, and a prefix of equal length is the whole list. -/
theorem nbhd_injective {l l' : List Nat} (h : nbhd l = nbhd l') : l = l' := by
  have h1 : l' <+: l := (nbhd_subset_iff l l').mp h.subset
  have h2 : l <+: l' := (nbhd_subset_iff l' l).mp h.symm.subset
  exact (h1.eq_of_length (h1.length_le.antisymm h2.length_le)).symm
end

section
/-- **`X_n  I  X? = X_k` on the `Lenum` coding.** The intersection of two `nbhd`s is
    the more specific
of the two whenever they are nested, and no `Lmem` set at all otherwise (`nbhd` is
    always
nonempty, so `{}` never equals an `Lenum k`). -/
theorem Lenum_interEq_iff (n m k : Nat) :
    Lenum n  I  Lenum m = Lenum k  <->
      (isPrefixChar m n = 1  /\  decodeList k = decodeList n)  \/
      (isPrefixChar n m = 1  /\  decodeList k = decodeList m) := by
  unfold Lenum
  rcases L_nestedOrDisjoint (mem_nbhd (decodeList n)) (mem_nbhd (decodeList m)) with h
    | h | h
  * have hp : isPrefixChar m n = 1 := (isPrefixChar_eq_one_iff m n).mpr ((
```

```
    nbhd_subset_iff _ _).mp h)
    rw [Set.inter_eq_left.mpr h]
    constructor
    * intro heq; exact Or.inl <hp, (nbhd_injective heq).symm>
    * rintro (<-, heq> | <hp', heq>)
      * rw [heq]
      * rw [isPrefixChar_eq_one_iff] at hp'
        rw [heq]
        exact Set.Subset.antisymm h ((nbhd_subset_iff (decodeList m) (decodeList n)).
    mpr hp')
  * have hp : isPrefixChar n m = 1 := (isPrefixChar_eq_one_iff n m).mpr ((
    nbhd_subset_iff _ _).mp h)
    rw [Set.inter_eq_right.mpr h]
    constructor
    * intro heq; exact Or.inr <hp, (nbhd_injective heq).symm>
    * rintro (<hp', heq> | <-, heq>)
      * rw [isPrefixChar_eq_one_iff] at hp'
        rw [heq]
        exact Set.Subset.antisymm h ((nbhd_subset_iff (decodeList n) (decodeList m)).
    mpr hp')
      * rw [heq]
  * rw [h]
    have h1 : isPrefixChar m n  !=  1 := by
      intro hp; rw [isPrefixChar_eq_one_iff] at hp
      obtain <t, ht> := (nbhd_nonempty (decodeList n))
      exact Set.notMem_empty t (h  |>  Set.mem_inter ht ((nbhd_subset_iff _ _).mpr hp
    ht))
    have h2 : isPrefixChar n m  !=  1 := by
      intro hp; rw [isPrefixChar_eq_one_iff] at hp
      obtain <t, ht> := (nbhd_nonempty (decodeList m))
      exact Set.notMem_empty t (h  |>  Set.mem_inter ((nbhd_subset_iff _ _).mpr hp ht)
     ht)
    constructor
    * intro heq; exact absurd heq.symm (nbhd_nonempty (decodeList k)).ne_empty
    * rintro (<hp, -> | <hp, ->)
      * exact absurd hp h1
      * exact absurd hp h2
end

section
theorem Lenum_interEq_charP_eq_one_iff (n m k : Nat) :
    Lenum n  I  Lenum m = Lenum k  <->  LenumInterEqCharP (Nat.pair n (Nat.pair m k))
    = 1 := by
  unfold LenumInterEqCharP
  simp only [unpair_pair_fst, unpair_pair_snd]
  rw [LenumInterEqChar_eq_one_iff]
  exact Lenum_interEq_iff n m k
end

section
/-- **7.1(i) -- `X_n  I  X? = X_k` is recursively decidable.** -/
theorem Lenum_interEq_computable : RecDecidable_3 (fun n m k => Lenum n  I  Lenum m =
    Lenum k) :=
  RecDecidable_3.of_triple_zero_one_char primrec_LenumInterEqCharP
    (fun t => by have := LenumInterEqCharP_le_one t; omega)
    Lenum_interEq_charP_eq_one_iff
```

```
end

section
/-- **Exercise 7.24, first claim -- `L` is effectively given.** The enumeration `Lenum`
     (indices
via `encodeList`/`decodeList`), Scott's two relations (`Lenum_interEq_computable`,
`Lenum_cons_computable`), the primitive-recursive intersection (`LenumInter`), and the
    master
index `0` (`Lenum_zero`) assemble into a `ComputablePresentation L`. -/
def Lpres : ComputablePresentation L where
  X := Lenum
  mem_X n := Lenum_mem n
  surj hY := Lenum_surj hY
  interEq_computable := Lenum_interEq_computable
  cons_computable := Lenum_cons_computable
  inter := LenumInter
  inter_primrec := primrec_LenumInter
  inter_spec h := LenumInter_spec h
  masterIdx := 0
  masterIdx_spec := Lenum_zero
end

section
open DomainTheory.NeighbourhoodSystems.Example12 (Token zero one)
variable {b ? ? : Type*}

/-- The characteristic-function bundle needed to write down `funPresentation Lpres P`
    concretely:
`gN` for function-space consistency, `incl0`/`incl1` for `L`/`V`-inclusion, `eq1` for `V
    `-equality. -/
structure LFunData {V : NeighborhoodSystem b} (P : ComputablePresentation V) where
  gN : Nat  ->  Nat
  incl0 : Nat  ->  Nat
  incl1 : Nat  ->  Nat
  eq1 : Nat  ->  Nat
  hgN : forall  c, gN c = 1  <->  (stepFun (funListOf Lpres P (decodeList c))
    : Set (ApproximableMap L V)).Nonempty
  hgNp : Nat.Primrec gN
  hincl0 : forall  s, incl0 s = 1  <->  Lpres.X s.unpair.1  subseteq  Lpres.X s.unpair
    .2
  hincl0p : Nat.Primrec incl0
  hincl1 : forall  s, incl1 s = 1  <->  P.X s.unpair.1  subseteq  P.X s.unpair.2
  hincl1p : Nat.Primrec incl1
  heq1 : forall  s, eq1 s = 1  <->  P.X s.unpair.1 = P.X s.unpair.2
  heq1p : Nat.Primrec eq1
end

section
instance : Nonempty Gamma := <Sum.inl []>
end

section
@[simp] theorem cons_inl (i : Nat) (l : List Nat) : cons i (.inl l) = .inl (i :: l) :=
     rfl
end
```

```
section
@[simp] theorem cons_inr (i : Nat) (s : Nat  ->  Nat) : cons i (.inr s) = .inr (
    streamCons i s) := rfl
end

section
@[simp] theorem L_mem {X : Set Gamma} : L.mem X  <->  Lmem X := Iff.rfl
end

section
@[simp] theorem L_master : L.master = Set.univ := rfl
end

section
/-- **`X_n  subseteq  X?` `{0,1}` characteristic**, packaged as a single *named* unary
    function (rather than
an anonymous `t.unpair`-lambda passed positionally to `RecDecidable_2.
    of_paired_zero_one_char`):
naming it lets the `hfe` obligation be stated and proved as a standalone lemma whose
    conclusion is
syntactically `LenumSubsetChar (Nat.pair n m) = 1`, so the final application only needs
     to match
this name against itself -- no elaborator-side defeq unfolding of `Nat.pair`/`Nat.
    unpair` (which is
expensive and blows the `whnf` budget; see the project's course-of-values discipline).
    -/
def LenumSubsetChar (t : Nat) : Nat := isPrefixChar t.unpair.2 t.unpair.1
end

section
/-- The length-`n` prefix of an infinite stream `f`: `finPrefix f n = [f 0, ..., f (n -
     1)]`. -/
def finPrefix (f : Nat  ->  Nat) (n : Nat) : List Nat := (List.range n).map f
end

section
@[simp] theorem finPrefix_zero (f : Nat  ->  Nat) : finPrefix f 0 = [] := rfl
end

section
/-- A filter `x` **has a minimum neighbourhood** -- the hallmark of Scott's *finite*
    elements. -/
def HasMin (x : L.Element) : Prop := exists  X, x.mem X  /\  forall  Y, x.mem Y  ->  X
      subseteq  Y
end

section
/-- The digit-wise embedding of a bit into `Nat`: `false  |->  0`, `true  |->  1`. -/
def bitToNat (b : Bool) : Nat := if b then 1 else 0
end

section
@[simp] theorem bitToNat_false : bitToNat false = 0 := rfl
end

section
```

```
@[simp] theorem bitToNat_true : bitToNat true = 1 := rfl
end

section
/-- Embed a finite bit-string `Sigma*` into a finite `Nat`-list `Gamma`'s finite part,
    digit by digit. -/
def embStr (sigma : ExampleB.Str) : List Nat := sigma.map bitToNat
end

section
@[simp] theorem embStr_nil : embStr [] = [] := rfl
end

section
open DomainTheory.NeighbourhoodSystems.Example12 (Token zero one)

/-- **`T`'s enumeration.** `0 |-> Delta`, `1 |-> {0}`, `2 |-> {1}`, and every `n
    >= 2` collapses to `{1}` -- we
only need *some* enumeration onto the three neighbourhoods, not a bijection. -/
def Tenum (n : Nat) : Set Token :=
  match n with
  | 0 => Example12.master
  | 1 => zero
  | _ => one

@[simp] theorem Tenum_zero : Tenum 0 = Example12.master := rfl

@[simp] theorem Tenum_one : Tenum 1 = zero := rfl

@[simp] theorem Tenum_two : Tenum 2 = one := rfl

/-- `Tenum` only depends on `n` through `min n 2`. -/
theorem Tenum_eq_canon (n : Nat) : Tenum n = Tenum (min n 2) := by
  rcases n with _ | _ | n
  * rfl
  * rfl
  * show one = Tenum (min (n + 2) 2)
    rw [show min (n + 2) 2 = 2 from by omega]
    rfl

theorem Tenum_mem (n : Nat) : Example23.T.mem (Tenum n) := by
  rcases n with _ | _ | n
  * exact Example12.mem_master
  * exact Example12.mem_zero
  * exact Example12.mem_one

/-- `Tenum` never hits `{}` (it always lands on a genuine `T`-neighbourhood, and `{}`
    is not one). -/
theorem Tenum_ne_empty (n : Nat) : Tenum n  !=  ({} : Set Token) := by
  intro h
  exact Example12.not_mem_empty (h  |>  Tenum_mem n)

theorem Tenum_surj {Y : Set Token} (hY : Example23.T.mem Y) : exists  n, Tenum n = Y
    := by
  rcases (Example12.mem_iff Y).mp hY with rfl | rfl | rfl
  * exact <0, rfl>
```

```
  * exact <1, rfl>
  * exact <2, rfl>

/-- `TeqChar a b = 1` decides `a = b` (truncated-subtraction trick, primitive recursive
    ). -/
def TeqChar (a b : Nat) : Nat := isZero ((a - b) + (b - a))

/-- **The intersection code.** `TinterCode a b` names the intersection `Tenum a  I
    Tenum b` when it
is a `T`-neighbourhood (`a = 0`, `b = 0`, or `a = b`), and is the junk value `3` (never
    ` <=  2`, hence
never named by `Tenum`'s canonical range) exactly when the intersection is inconsistent
(`{0}  I  {1} = {}`). -/
def TinterCode (a b : Nat) : Nat := if a = 0 then b else if b = 0 then a else if a = b
     then a else 3

/-- `andCode a b` names `andFn (Tenum a) (Tenum b)` when `a, b  <=  2`: `andCode 0 b =
    0`,
`andCode 1 b = b`, `andCode 2 b = 2`. -/
def andCode (a b : Nat) : Nat := if a = 1 then b else if a = 2 then 2 else 0

/-- **Sequential conjunction** on `T`'s neighbourhoods: `andFn X Y` is `Y` when `X`
    names `true`
(`zero`), the constant `false` (`one`) when `X` names `false` -- regardless of `Y` --
    and `Delta` (no
information yet) otherwise. This is the standard *left-strict* `and`, matching `zero`/`
    one` naming
`true`/`false` (`Example23.trueElt = elemZero`, `Example23.falseElt = elemOne`). -/
noncomputable def andFn (X Y : Set Token) : Set Token :=
  if X = zero then Y else if X = one then one else Example12.master

theorem andFn_zero (Y : Set Token) : andFn zero Y = Y := by unfold andFn; rw [
    ite_eq_left rfl]

theorem eq_master_of_master_subset {Y : Set Token} (h : Example12.master  subseteq  Y)
     :
    Y = Example12.master := (Set.univ_subset_iff.mp h : Y = Set.univ)

end

section
/-- If `l ++ [i] <+: l'` fails for every `i`, then... (helper): a list strictly longer
    than a
prefix of it extends by *some* single element first. -/
theorem exists_cons_prefix_of_prefix_of_length_lt {l l' : List Nat} (h : l <+: l')
    (hlt : l.length < l'.length) : exists  i, l ++ [i] <+: l' := by
  obtain <r, hr> := h
  cases r with
  | nil => simp [ <-  hr] at hlt
  | cons i r' => exact <i, r', by rw [ <-  hr]; simp>
end

section
/-- **One-step extension exists whenever `x` has no minimum.** If `x.mem (nbhd l)` and
    `x` has no
smallest neighbourhood, some length-`(l.length + 1)` extension `l ++ [i]` is also in `x
```

```
    `: pick any
`Y  in  x.mem` witnessing that `nbhd l` is not the minimum (`hx`), which -- being
    nested with `nbhd l`
(disjointness is impossible, `x.inter_mem` would force `L.mem {}`) and *not* containing
     it -- is a
strictly smaller neighbourhood `nbhd l' ? nbhd l`; the first new list entry of `l'`
    past `l` is the
witness `i`. -/
theorem exists_extend_of_not_hasMin {x : L.Element} (hx : not  HasMin x) {l : List Nat
    }
    (hl : x.mem (nbhd l)) : exists  i, x.mem (nbhd (l ++ [i])) := by
  have hex : exists  Y, x.mem Y  /\  not  nbhd l  subseteq  Y := by
    by_contra hc; push Not at hc; exact hx <nbhd l, hl, hc>
  obtain <Y, hYmem, hnsub> := hex
  obtain <l', rfl> := x.sub hYmem
  rcases L_nestedOrDisjoint (mem_nbhd l) (mem_nbhd l') with hsub | hsub | hdisj
  * exact absurd hsub hnsub
  * have hpre : l <+: l' := (nbhd_subset_iff l' l).mp hsub
    have hne : l  !=  l' := by rintro rfl; exact hnsub subset_rfl
    have hlt : l.length < l'.length :=
      lt_of_le_of_ne hpre.length_le (fun heqlen => hne (hpre.eq_of_length heqlen))
    obtain <i, hi> := exists_cons_prefix_of_prefix_of_length_lt hpre hlt
    exact <i, x.up_mem hYmem (mem_nbhd _) ((nbhd_subset_iff l' (l ++ [i])).mpr hi)>
  * exfalso
    have hxmem : x.mem (nbhd l  I  nbhd l') := x.inter_mem hl hYmem
    rw [hdisj] at hxmem
    obtain <l'', hl''> := x.sub hxmem
    exact (nbhd_nonempty l'').ne_empty hl''.symm
end

section
/-- **Choice-built data**: the length-`n` list `x.mem`s, together with the invariant,
    by iterating
`exists_extend_of_not_hasMin`. This is the one genuinely non-constructive step (`
    Classical.choose`
picking a witness coordinate at every stage), unavoidable for a *general* filter `x` (
    no
computability is assumed of `x` at this point in the exercise). -/

noncomputable def buildData (x : L.Element) (hx : not  HasMin x) :
    forall  n : Nat, {l : List Nat // x.mem (nbhd l)  /\  l.length = n} :=
  fun n =>
    Nat.rec (<[], x.master_mem, rfl> : {l : List Nat // x.mem (nbhd l)  /\  l.length =
     0})
      (fun n prev =>
        <prev.1 ++ [Classical.choose (exists_extend_of_not_hasMin hx prev.2.1)],
          Classical.choose_spec (exists_extend_of_not_hasMin hx prev.2.1),
          by rw [List.length_append, List.length_singleton, prev.2.2]>)
      n
end

section
theorem finPrefix_length (f : Nat  ->  Nat) (n : Nat) : (finPrefix f n).length = n :=
    by
  unfold finPrefix; rw [List.length_map, List.length_range]
end
```

```
section
theorem finPrefix_succ (f : Nat  ->  Nat) (n : Nat) :
    finPrefix f (n + 1) = finPrefix f n ++ [f n] := by
  unfold finPrefix; rw [List.range_succ, List.map_append, List.map_singleton]
end

section
theorem finPrefix_prefix_succ (f : Nat  ->  Nat) (n : Nat) : finPrefix f n <+:
    finPrefix f (n + 1) := by
  rw [finPrefix_succ]; exact List.prefix_append _ _
end

section
theorem finPrefix_prefix_of_le (f : Nat  ->  Nat) {n m : Nat} (h : n  <=  m) :
    finPrefix f n <+: finPrefix f m := by
  induction h with
  | refl => exact List.prefix_refl _
  | step _ ih => exact ih.trans (finPrefix_prefix_succ f _)
end

section
theorem finPrefix_take (f : Nat  ->  Nat) {n m : Nat} (h : n  <=  m) :
    (finPrefix f m).take n = finPrefix f n := by
  unfold finPrefix; rw [ <-  List.map_take, List.take_range, min_eq_left h]
end

section
open DomainTheory.NeighbourhoodSystems.Example12 (Token zero one)

theorem master_ne_one_T : Example12.master  !=  one := by
  intro h
  have h1 : (0 : Token)  in  Example12.master := by simp [Example12.master]
  rw [h] at h1; simp [one] at h1

theorem master_ne_zero_T : Example12.master  !=  zero := by
  intro h
  have h1 : (1 : Token)  in  Example12.master := by simp [Example12.master]
  rw [h] at h1; simp [zero] at h1

theorem andFn_master (Y : Set Token) : andFn Example12.master Y = Example12.master :=
    by
  unfold andFn; rw [ite_eq_right master_ne_zero_T, ite_eq_right master_ne_one_T]

end

section
/-- Longer prefixes give smaller (more specific) neighbourhoods. -/
theorem nbhd_finPrefix_antitone (f : Nat  ->  Nat) {n m : Nat} (h : n  <=  m) :
    nbhd (finPrefix f m)  subseteq  nbhd (finPrefix f n) :=
  (nbhd_subset_iff _ _).mpr (finPrefix_prefix_of_le f h)
end

section
open DomainTheory.NeighbourhoodSystems.Example12 (Token zero one)
```

```
/-- `notCode c` names `notFn (Tenum c)` when `c  <=  2`: `0  |->  0`, `1  |->  2`, `2
    |->  1`. -/
def notCode (c : Nat) : Nat := if c = 1 then 2 else if c = 2 then 1 else 0

/-- The pointwise negation function on `T`'s three neighbourhoods: swaps `{0}`/`{1}`,
    fixes `Delta`. -/
noncomputable def notFn (X : Set Token) : Set Token :=
  if X = zero then one else if X = one then zero else Example12.master

theorem notFn_master : notFn Example12.master = Example12.master := by
  unfold notFn; rw [ite_eq_right master_ne_zero_T, ite_eq_right master_ne_one_T]

theorem notFn_zero : notFn zero = one := by unfold notFn; rw [ite_eq_left rfl]

end

section
open DomainTheory.NeighbourhoodSystems.Example12 (Token zero one)
variable {b ? ? : Type*}

/-- **The one use of `Classical.choice` in this section.** Every `
    ComputablePresentation P` of a
`V` yields *some* `LFunData P`, by choosing concrete deciders out of `Lpres`'s and `P`'
    s existential
`incl_computable`/`cons_computable`/`eq_computable` witnesses. -/
noncomputable def LFunData.ofPresentation {V : NeighborhoodSystem b} (P :
    ComputablePresentation V) :
    LFunData P where
  gN := funConsChar Lpres P Lpres.cons_computable.choose P.cons_computable.choose
  incl0 := Lpres.incl_computable.choose
  incl1 := P.incl_computable.choose
  eq1 := P.eq_computable.choose
  hgN := funConsChar_spec Lpres P Lpres.cons_computable.choose P.cons_computable.
    choose
    (fun s => (Lpres.cons_computable.choose_spec.2 s).symm)
    (fun s => (P.cons_computable.choose_spec.2 s).symm)
  hgNp := primrec_funConsChar Lpres P Lpres.cons_computable.choose P.cons_computable.
    choose
    Lpres.cons_computable.choose_spec.1 P.cons_computable.choose_spec.1
  hincl0 := fun s => (Lpres.incl_computable.choose_spec.2 s).symm
  hincl0p := Lpres.incl_computable.choose_spec.1
  hincl1 := fun s => (P.incl_computable.choose_spec.2 s).symm
  hincl1p := P.incl_computable.choose_spec.1
  heq1 := fun s => (P.eq_computable.choose_spec.2 s).symm
  heq1p := P.eq_computable.choose_spec.1
end

section
open DomainTheory.NeighbourhoodSystems.Example12 (Token zero one)

theorem one_inter_zero_T : one  I  zero = ({} : Set Token) := by
  ext t; fin_cases t <;> simp [zero, one]
end

section
open DomainTheory.NeighbourhoodSystems.Example12 (Token zero one)
```

```
variable {b ? ? : Type*}

/-- **`postcompose h`**: post-composing every `L`-indexed sequence `f : L  ->  V_1`
    with `h`, i.e.
`f  |->  h  comp  f`. Built as `curry (h  comp  eval)`, following Scott. -/
def postcompose {V_1 : NeighborhoodSystem b} {V_2 : NeighborhoodSystem ?}
    (h : ApproximableMap V_1 V_2) : ApproximableMap (funSpace L V_1) (funSpace L V_2)
    :=
  curry (h.comp (evalMap L V_1))

/-- The concrete `ComputablePresentation (L  ->  V)` assembled from an `LFunData`
    bundle. -/
def LFunData.pres {V : NeighborhoodSystem b} {P : ComputablePresentation V} (d :
    LFunData P) :
    ComputablePresentation (funSpace L V) :=
  funPresentation Lpres P d.gN d.incl0 d.incl1 d.eq1 d.hgN d.hgNp d.hincl0 d.hincl0p
    d.hincl1 d.hincl1p d.heq1 d.heq1p

end

section
open DomainTheory.NeighbourhoodSystems.Example12 (Token zero one)

theorem primrec_TeqChar {f g : Nat  ->  Nat} (hf : Nat.Primrec f) (hg : Nat.Primrec g)
     :
    Nat.Primrec (fun n => TeqChar (f n) (g n)) :=
  (primrec_isZero.comp (primrec_add_2 (primrec_sub_2 hf hg) (primrec_sub_2 hg hf))).
    of_eq
    fun _ => rfl

theorem primrec_TinterCode {f g : Nat  ->  Nat} (hf : Nat.Primrec f) (hg : Nat.Primrec
     g) :
    Nat.Primrec (fun n => TinterCode (f n) (g n)) := by
  have heq : Nat.Primrec (fun n => TeqChar (f n) (g n)) := primrec_TeqChar hf hg
  have hz1 := primrec_selectFn heq hf (Nat.Primrec.const 3)
  have hz2 := primrec_selectFn (primrec_isZero.comp hg) hf hz1
  refine (primrec_selectFn (primrec_isZero.comp hf) hg hz2).of_eq fun n => ?_
  unfold TinterCode TeqChar
  by_cases h0 : f n = 0
  * rw [ite_eq_left h0, h0, isZero]; simp
  * rw [ite_eq_right h0]
    have hz0 : isZero (f n) = 0 := by unfold isZero; omega
    rw [hz0, selectFn_zero]
    by_cases h1 : g n = 0
    * rw [ite_eq_left h1, h1, isZero]; simp
    * rw [ite_eq_right h1]
      have hz0' : isZero (g n) = 0 := by unfold isZero; omega
      rw [hz0', selectFn_zero]
      by_cases h2 : f n = g n
      * have : isZero ((f n - g n) + (g n - f n)) = 1 := by unfold isZero; omega
        rw [ite_eq_left h2, this, selectFn_one]
      * have : isZero ((f n - g n) + (g n - f n)) = 0 := by unfold isZero; omega
        rw [ite_eq_right h2, this, selectFn_zero]

end
```

```
section
theorem principal_hasMin (l : List Nat) : HasMin (L.principal (mem_nbhd l)) :=
  <nbhd l, <mem_nbhd l, subset_rfl>, fun _ hY => hY.2>
end

section
open DomainTheory.NeighbourhoodSystems.Example12 (Token zero one)
variable {b ? ? : Type*}

/-- The "extract the shared argument `x`" projection out of the pointwise-binary domain
    . -/
def pwArg (V_0 : NeighborhoodSystem b) (V_1 : NeighborhoodSystem ?) :
    ApproximableMap (prod (prod (funSpace L V_0) (funSpace L V_1)) L) L :=
  proj_1 (prod (funSpace L V_0) (funSpace L V_1)) L

/-- The "extract `f`" projection out of the pointwise-binary domain `(L -> V_0)  x  (L
    -> V_1)  x  L`. -/
def pwFst (V_0 : NeighborhoodSystem b) (V_1 : NeighborhoodSystem ?) :
    ApproximableMap (prod (prod (funSpace L V_0) (funSpace L V_1)) L) (funSpace L V_0)
     :=
  (proj_0 (funSpace L V_0) (funSpace L V_1)).comp (proj_0 (prod (funSpace L V_0) (
    funSpace L V_1)) L)

/-- The "extract `g`" projection out of the pointwise-binary domain. -/
def pwSnd (V_0 : NeighborhoodSystem b) (V_1 : NeighborhoodSystem ?) :
    ApproximableMap (prod (prod (funSpace L V_0) (funSpace L V_1)) L) (funSpace L V_1)
     :=
  (proj_1 (funSpace L V_0) (funSpace L V_1)).comp (proj_0 (prod (funSpace L V_0) (
    funSpace L V_1)) L)

/-- **`pointwiseBin h`**: given `f : L  ->  V_0`, `g : L  ->  V_1`, produce the `L`-
    sequence
`x  |->  h(f x, g x)`. Built by evaluating both factors at the shared argument and
    applying `h`, then
`curry`-ing. -/
def pointwiseBin {V_0 : NeighborhoodSystem b} {V_1 : NeighborhoodSystem ?} {V_2 :
    NeighborhoodSystem ?}
    (h : ApproximableMap (prod V_0 V_1) V_2) :
    ApproximableMap (prod (funSpace L V_0) (funSpace L V_1)) (funSpace L V_2) :=
  curry (h.comp (paired
    ((evalMap L V_0).comp (paired (pwFst V_0 V_1) (pwArg V_0 V_1)))
    ((evalMap L V_1).comp (paired (pwSnd V_0 V_1) (pwArg V_0 V_1)))))

end

section
/-- **The element of `|L|` determined by an infinite sequence `f`**: the filter of
    neighbourhoods
containing some finite prefix of `f` -- Scott's `limitFamily` (Definition 1.6's
    motivating prose),
specialized to `L`'s prefix chain `nbhd (finPrefix f *)`. -/
def streamElement (f : Nat  ->  Nat) : L.Element where
  mem Z := L.mem Z  /\  exists  n, nbhd (finPrefix f n)  subseteq  Z
  sub h := h.1
  master_mem := <mem_nbhd [], 0, subset_rfl>
  inter_mem := by
```

```
    rintro X Y <hXmem, n, hXsub> <hYmem, m, hYsub>
    have hn : nbhd (finPrefix f (max n m))  subseteq  X :=
      (nbhd_finPrefix_antitone f (le_max_left n m)).trans hXsub
    have hm : nbhd (finPrefix f (max n m))  subseteq  Y :=
      (nbhd_finPrefix_antitone f (le_max_right n m)).trans hYsub
    exact <L.inter_mem hXmem hYmem (mem_nbhd _) (Set.subset_inter hn hm),
      max n m, Set.subset_inter hn hm>
  up_mem := by
    rintro X Y <-, n, hn> hYmem hXY
    exact <hYmem, n, hn.trans hXY>
end

section
/-- **Prefixes match on the nose whenever the stream elements agree.** -/
theorem finPrefix_eq_of_streamElement_eq {f_1 f_2 : Nat  ->  Nat} (heq : streamElement
     f_1 = streamElement f_2) :
    forall  n, finPrefix f_1 n = finPrefix f_2 n := by
  intro n
  have hmem1 : (streamElement f_1).mem (nbhd (finPrefix f_1 n)) := <mem_nbhd _, n,
    subset_rfl>
  rw [heq] at hmem1
  obtain <-, m, hm> := hmem1
  have hpre : finPrefix f_1 n <+: finPrefix f_2 m := (nbhd_subset_iff _ _).mp hm
  have hlen : n  <=  m := by
    have hl := hpre.length_le; rwa [finPrefix_length, finPrefix_length] at hl
  have hpre' : finPrefix f_1 n = (finPrefix f_2 m).take (finPrefix f_1 n).length :=
    List.prefix_iff_eq_take.mp hpre
  rw [finPrefix_length] at hpre'
  rw [hpre', finPrefix_take f_2 hlen]
end

section
/-- **`streamElement` is injective**: two streams agreeing on every finite prefix agree
everywhere. -/
theorem streamElement_injective : Function.Injective streamElement := by
  intro f_1 f_2 heq
  have hpre := finPrefix_eq_of_streamElement_eq heq
  funext n
  have h1 := hpre (n + 1)
  rw [finPrefix_succ, finPrefix_succ, hpre n] at h1
  simpa using List.append_cancel_left h1
end

section
/-- **The infinite elements have no minimum neighbourhood.** Given any witness `X
    supseteq  nbhd
(finPrefix f n)` in the filter, the *next* prefix neighbourhood is strictly smaller yet
     still in
the filter, so `X` cannot be the minimum. -/
theorem streamElement_not_hasMin (f : Nat  ->  Nat) : not  HasMin (streamElement f) :=
     by
  rintro <X, <-, n, hXsub>, hmin>
  have hmemsucc : (streamElement f).mem (nbhd (finPrefix f (n + 1))) := <mem_nbhd _, n
     + 1, subset_rfl>
  have hle : X  subseteq  nbhd (finPrefix f (n + 1)) := hmin _ hmemsucc
  have heq : nbhd (finPrefix f n) = nbhd (finPrefix f (n + 1)) :=
```

```
    Set.Subset.antisymm (hXsub.trans hle) (nbhd_finPrefix_antitone f (Nat.le_succ n))
  have hlen := congrArg List.length (nbhd_injective heq)
  rw [finPrefix_length, finPrefix_length] at hlen
  omega
end

section
open DomainTheory.NeighbourhoodSystems.Example12 (Token zero one)

/-- **Among `T`'s three neighbourhoods, `X'  subseteq  X` forces `X' = X` or `X = Delta
    `.** (`Delta` is the only
non-trivial containee since `{0}`, `{1}` are incomparable singletons.) -/
theorem subset_iff_eq_or_eq_master {X' X : Set Token} (hX' : Example23.T.mem X')
    (hX : Example23.T.mem X) (h : X'  subseteq  X) : X' = X  \/  X = Example12.master
    := by
  rcases (Example12.mem_iff X).mp hX with rfl | rfl | rfl
  * exact Or.inr rfl
  * refine Or.inl ?_
    rcases (Example12.mem_iff X').mp hX' with rfl | rfl | rfl
    * exact absurd (show (1 : Token)  in  zero from h (by simp [Example12.master])) (
    by simp [zero])
    * rfl
    * exact absurd (show (1 : Token)  in  zero from h (by simp [one])) (by simp [zero
    ])
  * refine Or.inl ?_
    rcases (Example12.mem_iff X').mp hX' with rfl | rfl | rfl
    * exact absurd (show (0 : Token)  in  one from h (by simp [Example12.master])) (by
     simp [one])
    * exact absurd (show (0 : Token)  in  one from h (by simp [zero])) (by simp [one])
    * rfl
end

section
/-- **`toElement`: `Gamma  ->  |L|`**, realizing Scott's identification. -/
def toElement : Gamma  ->  L.Element
  | .inl l => L.principal (L_mem.mpr (mem_nbhd l))
  | .inr f => streamElement f
end

section
/-- **`toElement` is injective** (Exercise 7.24, second claim, uniqueness half). Finite
     vs. finite
is `nbhd_injective`; the two mixed cases are separated by `HasMin`; infinite vs.
    infinite is
`streamElement_injective`. -/
theorem toElement_injective : Function.Injective toElement := by
  rintro (l_1 | f_1) (l_2 | f_2) heq
  * exact congrArg Sum.inl
      (nbhd_injective (principal_injective (V := L) (mem_nbhd l_1) (mem_nbhd l_2) heq)
    )
  * have heq' : L.principal (mem_nbhd l_1) = streamElement f_2 := heq
    exact absurd (heq'  |>  principal_hasMin l_1) (streamElement_not_hasMin f_2)
  * have heq' : streamElement f_1 = L.principal (mem_nbhd l_2) := heq
    exact absurd (principal_hasMin l_2) (heq'  |>  streamElement_not_hasMin f_1)
  * exact congrArg Sum.inr (streamElement_injective heq)
end
```

```
section
/-- **`toStream x hx`: the witness infinite sequence for a min-free filter `x`.** -/
noncomputable def toStream (x : L.Element) (hx : not  HasMin x) (n : Nat) : Nat :=
  Classical.choose (exists_extend_of_not_hasMin hx (buildData x hx n).2.1)
end

section
theorem buildData_succ (x : L.Element) (hx : not  HasMin x) (n : Nat) :
    (buildData x hx (n + 1)).1 = (buildData x hx n).1 ++ [toStream x hx n] := rfl
end

section
theorem buildData_eq_finPrefix (x : L.Element) (hx : not  HasMin x) :
    forall  n, (buildData x hx n).1 = finPrefix (toStream x hx) n
  | 0 => rfl
  | n + 1 => by rw [buildData_succ, finPrefix_succ, buildData_eq_finPrefix x hx n]
end

section
theorem mem_nbhd_finPrefix_toStream (x : L.Element) (hx : not  HasMin x) (n : Nat) :
    x.mem (nbhd (finPrefix (toStream x hx) n)) :=
  buildData_eq_finPrefix x hx n  |>  (buildData x hx n).2.1
end

section
/-- **Every neighbourhood `Z  in  x` contains some sufficiently deep prefix.** `Z =
    nbhd l` for some
`l` (`x.sub`), and comparing `nbhd l` against the (unboundedly long) chain
`nbhd (finPrefix (toStream x hx) n)` -- nested, not disjoint, by the same
`x.inter_mem`/`L_nestedOrDisjoint` argument as `exists_extend_of_not_hasMin` -- forces
    `l` itself to
be a prefix of the chain once `n > l.length` (the chain entry is then too long to be *
    the* shorter
side). Together with `mem_nbhd_finPrefix_toStream` this gives `streamElement (toStream
    x hx) = x`
(used directly in `toElement_surjective`). -/
theorem exists_finPrefix_subset_of_mem (x : L.Element) (hx : not  HasMin x) {Z : Set
    Gamma}
    (hZ : x.mem Z) : exists  n, nbhd (finPrefix (toStream x hx) n)  subseteq  Z := by
  obtain <l, rfl> := x.sub hZ
  refine <l.length + 1, ?_>
  set n := l.length + 1
  have hn : x.mem (nbhd (finPrefix (toStream x hx) n)) := mem_nbhd_finPrefix_toStream
    x hx n
  have hinter := x.inter_mem hZ hn
  rcases L_nestedOrDisjoint (mem_nbhd l) (mem_nbhd (finPrefix (toStream x hx) n)) with
    hsub | hsub | hdisj
  * -- nbhd l  subseteq  nbhd (finPrefix _ n): impossible by length once n = l.length
    + 1
    have hpre : finPrefix (toStream x hx) n <+: l := (nbhd_subset_iff l _).mp hsub
    have := hpre.length_le
    rw [finPrefix_length] at this
    omega
  * exact hsub
  * exfalso
```

```
    rw [hdisj] at hinter
    obtain <l', hl'> := x.sub hinter
    exact (nbhd_nonempty l').ne_empty hl'.symm
end

section
/-- **`toElement` is surjective** (Exercise 7.24, second claim, existence half; uses
`Classical.choice` via `buildData`/`toStream` for the infinite case -- see `buildData`'
    s docstring).
-/
theorem toElement_surjective : Function.Surjective toElement := by
  intro x
  by_cases hx : HasMin x
  * obtain <X, hXmem, hXmin> := hx
    obtain <l, rfl> := x.sub hXmem
    exact <Sum.inl l, (eq_principal_of_isMin (V := L) x hXmem hXmin).symm>
  * exact <Sum.inr (toStream x hx), by
      apply Element.ext
      intro Z
      constructor
      * rintro <hZmem, n, hn>
        exact x.up_mem (mem_nbhd_finPrefix_toStream x hx n) hZmem hn
      * intro hZ
        exact <x.sub hZ, exists_finPrefix_subset_of_mem x hx hZ>>
end

section
/-- **Exercise 7.24, second claim.** `|L|` (the domain elements `L.Element`) is exactly
     the type of
finite-or-infinite sequences of naturals `Gamma`, via `toElement`. -/
theorem toElement_bijective : Function.Bijective toElement :=
  <toElement_injective, toElement_surjective>
end

section
/-- **`Gamma ? |L|`**, the promised identification (noncomputable: surjectivity used
    choice). -/
noncomputable def gammaEquivElement : Gamma ? L.Element :=
  Equiv.ofBijective toElement toElement_bijective
end

section
open DomainTheory.NeighbourhoodSystems.Example12 (Token zero one)

theorem zero_inter_one_T : zero  I  one = ({} : Set Token) := by
  ext t; fin_cases t <;> simp [zero, one]

theorem Tenum_inter_eq {a b : Nat} (ha : a  <=  2) (hb : b  <=  2) :
    Tenum a  I  Tenum b = if a = 0 then Tenum b else if b = 0 then Tenum a
      else if a = b then Tenum a else ({} : Set Token) := by
  interval_cases a <;> interval_cases b <;>
    simp [Tenum, Example12.master, Set.univ_inter, Set.inter_univ, Set.inter_self,
      zero_inter_one_T, one_inter_zero_T]

/-- `Tenum n  I  Tenum m` is `Tenum (TinterCode *)` when consistent, and `{}` exactly
    when
```

```
`TinterCode` hits the junk value `3`. -/
theorem Tenum_inter_eq_TinterCode' {a b : Nat} (ha : a  <=  2) (hb : b  <=  2) :
    Tenum a  I  Tenum b = if TinterCode a b = 3 then ({} : Set Token) else Tenum (
    TinterCode a b) := by
  interval_cases a <;> interval_cases b <;>
    simp [Tenum, TinterCode, Example12.master, Set.univ_inter, Set.inter_univ, Set.
    inter_self,
      zero_inter_one_T, one_inter_zero_T]

theorem Tenum_inter_eq_TinterCode (n m : Nat) :
    Tenum n  I  Tenum m = if TinterCode (min n 2) (min m 2) = 3 then ({} : Set Token)
      else Tenum (TinterCode (min n 2) (min m 2)) := by
  rw [Tenum_eq_canon n, Tenum_eq_canon m]
  exact Tenum_inter_eq_TinterCode' (Nat.min_le_right n 2) (Nat.min_le_right m 2)

theorem Tpres_cons_iff (n m : Nat) :
    (exists  k, Tenum k  subseteq  Tenum n  I  Tenum m)  <->  TinterCode (min n 2) (
    min m 2)  !=  3 := by
  rw [Tenum_inter_eq_TinterCode n m]
  constructor
  * rintro <k, hk> hcontra
    rw [ite_eq_left hcontra] at hk
    exact Tenum_ne_empty k (Set.subset_empty_iff.mp hk)
  * intro hne
    rw [ite_eq_right hne]
    exact <TinterCode (min n 2) (min m 2), subset_rfl>

/-- **The `{0,1}` consistency characteristic for `T`** (Scott's relation (ii)): `exists
      k, X?  subseteq  X_n  I  X?`
holds iff `X_n  I  X?` is itself a `T`-neighbourhood, i.e. `TinterCode` did not hit the
     junk value `3`. -/
theorem Tpres_cons_computable : RecDecidable_2 (fun n m => exists  k, Tenum k
    subseteq  Tenum n  I  Tenum m) := by
  have ha : Nat.Primrec (fun t : Nat => TinterCode (min t.unpair.1 2) (min t.unpair.2
    2)) :=
    primrec_TinterCode (primrec_min Nat.Primrec.left (Nat.Primrec.const 2))
      (primrec_min Nat.Primrec.right (Nat.Primrec.const 2))
  have hne : RecDecidable (fun t : Nat => TinterCode (min t.unpair.1 2) (min t.unpair
    .2 2)  !=  3) :=
    (RecDecidable.natEq ha (Nat.Primrec.const 3)).not
  exact RecDecidable.of_iff (fun t => Tpres_cons_iff t.unpair.1 t.unpair.2) hne

theorem zero_ne_one_T : zero  !=  one := by
  intro h
  have h1 : (0 : Token)  in  zero := by simp [zero]
  rw [h] at h1; simp [one] at h1

/-- `Tenum` is injective on its "canonical" range `{0, 1, 2}`. -/
theorem Tenum_injOn {a b : Nat} (ha : a  <=  2) (hb : b  <=  2) (h : Tenum a = Tenum b
    ) : a = b := by
  interval_cases a <;> interval_cases b <;>
    simp only [Tenum_zero, Tenum_one, Tenum_two] at h <;>
    first
      | rfl
      | exact absurd h master_ne_zero_T
      | exact absurd h.symm master_ne_zero_T
```

```
      | exact absurd h master_ne_one_T
      | exact absurd h.symm master_ne_one_T
      | exact absurd h zero_ne_one_T
      | exact absurd h.symm zero_ne_one_T

theorem Tenum_interEq_iff {a b c : Nat} (ha : a  <=  2) (hb : b  <=  2) (hc : c  <=
    2) :
    Tenum a  I  Tenum b = Tenum c  <->  TinterCode a b = c := by
  rw [Tenum_inter_eq ha hb]
  unfold TinterCode
  split_ifs with h1 h2 h3
  * exact <fun h => (Tenum_injOn hb hc h).symm  |>  rfl, fun h => h  |>  rfl>
  * exact <fun h => (Tenum_injOn ha hc h).symm  |>  rfl, fun h => h  |>  rfl>
  * exact <fun h => (Tenum_injOn ha hc h).symm  |>  rfl, fun h => h  |>  rfl>
  * exact <fun h => absurd h.symm (Tenum_ne_empty c), fun h => absurd h.symm (by omega
    )>

/-- Same characterization, restated for arbitrary `n, m, k` via the `min * 2`
    canonicalization. -/
theorem Tenum_interEq_iff' (n m k : Nat) :
    Tenum n  I  Tenum m = Tenum k  <->  TinterCode (min n 2) (min m 2) = min k 2 := by
  rw [Tenum_eq_canon n, Tenum_eq_canon m, Tenum_eq_canon k,
    Tenum_interEq_iff (Nat.min_le_right n 2) (Nat.min_le_right m 2) (Nat.min_le_right
    k 2)]

/-- **The `{0,1}` intersection-equality characteristic for `T`** (Scott's relation (i))
    . -/
theorem Tpres_interEq_computable : RecDecidable_3 (fun n m k => Tenum n  I  Tenum m =
    Tenum k) := by
  have ha : Nat.Primrec (fun t : Nat => TinterCode (min t.unpair.1 2) (min t.unpair.2.
    unpair.1 2)) :=
    primrec_TinterCode (primrec_min Nat.Primrec.left (Nat.Primrec.const 2))
      (primrec_min (Nat.Primrec.left.comp Nat.Primrec.right) (Nat.Primrec.const 2))
  have hb : Nat.Primrec (fun t : Nat => min t.unpair.2.unpair.2 2) :=
    primrec_min (Nat.Primrec.right.comp Nat.Primrec.right) (Nat.Primrec.const 2)
  exact RecDecidable.of_iff
    (fun t => Tenum_interEq_iff' t.unpair.1 t.unpair.2.unpair.1 t.unpair.2.unpair.2)
    (RecDecidable.natEq ha hb)

/-- **`T` is effectively given**, via the hand-built enumeration `Tenum`. The
    presentation's
`inter`/`masterIdx` fields reuse `TinterCode`/`0` (both concrete, no search required,
    since `T`'s
finiteness makes the intersection function immediate from the case analysis above). -/
def Tpres : ComputablePresentation Example23.T where
  X := Tenum
  mem_X := Tenum_mem
  surj := Tenum_surj
  interEq_computable := Tpres_interEq_computable
  cons_computable := Tpres_cons_computable
  inter n m := TinterCode (min n 2) (min m 2)
  inter_primrec :=
    (primrec_TinterCode (primrec_min Nat.Primrec.left (Nat.Primrec.const 2))
      (primrec_min Nat.Primrec.right (Nat.Primrec.const 2))).of_eq fun _ => rfl
  inter_spec {n m} h := by
    rw [Tenum_inter_eq_TinterCode n m, ite_eq_right ((Tpres_cons_iff n m).mp h)]
```

```
  masterIdx := 0
  masterIdx_spec := rfl

end

section
open DomainTheory.NeighbourhoodSystems.Example12 (Token zero one)
variable {b ? ? : Type*}

/-- The one concrete presentation of `(L  ->  T)` used to state the LUCID combinators'
    computability. -/
noncomputable def LTdata : LFunData Tpres := LFunData.ofPresentation Tpres

/-- **The computable presentation of `(L  ->  T)`.** -/
noncomputable def LTpres : ComputablePresentation (funSpace L Example23.T) := LTdata.
    pres

end

section
open DomainTheory.NeighbourhoodSystems.Example12 (Token zero one)

theorem andFn_one (Y : Set Token) : andFn one Y = one := by
  unfold andFn; rw [ite_eq_right (Ne.symm zero_ne_one_T), ite_eq_left rfl]

theorem andFn_mem {X Y : Set Token} (hX : Example23.T.mem X) (hY : Example23.T.mem Y)
    :
    Example23.T.mem (andFn X Y) := by
  rcases (Example12.mem_iff X).mp hX with rfl | rfl | rfl
  * rw [andFn_master]; exact Example12.mem_master
  * rw [andFn_zero]; exact hY
  * rw [andFn_one]; exact Example12.mem_one

/-- **Pointwise (sequential) conjunction on `T`**, as a two-variable approximable
    mapping:
`X, Y f Z  <->  mem X  /\  mem Y  /\  mem Z  /\  andFn X Y  subseteq  Z` (Scott's "step
    " pattern again). -/
noncomputable def andMap2 : ApproximableMap_2 Example23.T Example23.T Example23.T
    where
  rel X Y Z := Example23.T.mem X  /\  Example23.T.mem Y  /\  Example23.T.mem Z  /\
    andFn X Y  subseteq  Z
  rel_dom_0 h := h.1
  rel_dom_1 h := h.2.1
  rel_cod h := h.2.2.1
  master_rel := <Example12.mem_master, Example12.mem_master, Example12.mem_master, by
    show andFn Example12.master Example12.master  subseteq  Example12.master
    rw [andFn_master]>
  inter_right := by
    rintro X Y Z Z' <hX, hY, hZ, hsub> <_, _, hZ', hsub'>
    exact <hX, hY, Example23.T.inter_mem hZ hZ' (andFn_mem hX hY) (Set.subset_inter
    hsub hsub'),
      Set.subset_inter hsub hsub'>
  mono := by
    rintro X X' Y Y' Z Z' <hX, hY, hZ, hsub> hX'X hY'Y hZZ' hX' hY' hZ'
    refine <hX', hY', hZ', ?_>
    rcases (Example12.mem_iff X).mp hX with rfl | rfl | rfl
```

```
    * rw [andFn_master] at hsub
      have hZ'master : Z' = Example12.master := eq_master_of_master_subset (hsub.trans
     hZZ')
      rw [hZ'master, Example12.master]; exact Set.subset_univ _
    * rw [andFn_zero] at hsub
      have hX'eq : X' = zero :=
        (subset_iff_eq_or_eq_master hX' Example12.mem_zero hX'X).resolve_right
          master_ne_zero_T.symm
      rw [hX'eq, andFn_zero]
      exact (hY'Y.trans hsub).trans hZZ'
    * rw [andFn_one] at hsub
      have hX'eq : X' = one :=
        (subset_iff_eq_or_eq_master hX' Example12.mem_one hX'X).resolve_right
          master_ne_one_T.symm
      rw [hX'eq, andFn_one]
      exact hsub.trans hZZ'

/-- **`andT`**, the joint approximable mapping `T  x  T  ->  T` corresponding to `
    andMap2`
(Theorem 3.5's `ofMap_2`). -/
noncomputable def andT : ApproximableMap (prod Example23.T Example23.T) Example23.T :=
  ofMap_2 andMap2

theorem notFn_one : notFn one = zero := by
  unfold notFn; rw [ite_eq_right (Ne.symm zero_ne_one_T), ite_eq_left rfl]

theorem notFn_mem {X : Set Token} (hX : Example23.T.mem X) : Example23.T.mem (notFn X)
     := by
  rcases (Example12.mem_iff X).mp hX with rfl | rfl | rfl
  * rw [notFn_master]; exact Example12.mem_master
  * rw [notFn_zero]; exact Example12.mem_one
  * rw [notFn_one]; exact Example12.mem_zero

/-- **Pointwise negation on `T`**, as an approximable map: `X f Y  <->  mem X  /\  mem
    Y  /\  notFn X  subseteq  Y`
(Scott's "step" pattern, turning the pointwise function `notFn` into a neighbourhood
    relation). -/
noncomputable def notT : ApproximableMap Example23.T Example23.T where
  rel X Y := Example23.T.mem X  /\  Example23.T.mem Y  /\  notFn X  subseteq  Y
  rel_dom h := h.1
  rel_cod h := h.2.1
  master_rel := <Example12.mem_master, Example12.mem_master, by
    show notFn Example12.master  subseteq  Example12.master; rw [notFn_master]>
  inter_right := by
    rintro X Y Y' <hX, hY, hsub> <_, hY', hsub'>
    exact <hX, Example23.T.inter_mem hY hY' (notFn_mem hX) (Set.subset_inter hsub hsub
    '),
      Set.subset_inter hsub hsub'>
  mono := by
    rintro X X' Y Y' <hX, hY, hsub> hX'X hYY' hX' hY'
    refine <hX', hY', ?_>
    rcases subset_iff_eq_or_eq_master hX' hX hX'X with heq | heq
    * rw [heq]; exact hsub.trans hYY'
    * rw [heq, notFn_master] at hsub
      have hY'master : Y' = Example12.master := eq_master_of_master_subset (hsub.trans
     hYY')
```

```
      rw [hY'master, Example12.master]; exact Set.subset_univ _

end

section
theorem bitToNat_injective : Function.Injective bitToNat := by
  intro a b h; cases a <;> cases b <;> simp_all [bitToNat]

theorem embStr_cons (b : Bool) (s : ExampleB.Str) : embStr (b :: s) = bitToNat b ::
    embStr s := rfl

/-- **`embStr` is an order-embedding of the prefix posets** (`Sigma*`'s into `List Nat
    `'s): it respects
and reflects the initial-segment relation, since `bitToNat` is injective. -/
theorem embStr_prefix_iff : forall  sigma tau : ExampleB.Str, embStr sigma <+: embStr
    tau  <->  sigma <+: tau := by
  intro sigma
  induction sigma with
  | nil => intro tau; simp
  | cons b s ih =>
    intro tau
    cases tau with
    | nil => simp [embStr_cons]
    | cons c t =>
      rw [embStr_cons, embStr_cons, List.cons_prefix_cons, List.cons_prefix_cons,
        bitToNat_injective.eq_iff, ih t]

/-- **The connection between `B` and `L` (Exercise 7.24, third claim), at the
    neighbourhood
level.** `B`'s cone-inclusion order embeds into `L`'s nbhd-inclusion order via `embStr
    `: `sigma`'s cone
sits inside `tau`'s cone in `B` iff `embStr sigma`'s neighbourhood sits inside `embStr
    tau`'s in `L`. -/
theorem cone_subset_cone_iff_nbhd_embStr (sigma tau : ExampleB.Str) :
    ExampleB.cone sigma  subseteq  ExampleB.cone tau  <->  nbhd (embStr sigma)
    subseteq  nbhd (embStr tau) := by
  rw [ExampleB.cone_subset_cone, nbhd_subset_iff, embStr_prefix_iff]

/-- **The connection between `B` and `L`, at the level of finite elements.** `
    sigmaFalse  sqsub  tauFalse` in `|B|`
iff the corresponding `L`-elements (`toElement` applied to the embedded addresses) are
    ` sqsub `-related
in `|L|` -- `B`'s finite elements embed order-isomorphically into `L`'s. -/
theorem sigmaBot_le_iff_toElement_inl_embStr (sigma tau : ExampleB.Str) :
    ExampleB.sigmaBot sigma  <=  ExampleB.sigmaBot tau  <->
      toElement (Sum.inl (embStr sigma))  <=  toElement (Sum.inl (embStr tau)) := by
  show ExampleB.sigmaBot sigma  <=  ExampleB.sigmaBot tau  <->
      L.principal (L_mem.mpr (mem_nbhd (embStr sigma)))  <=
        L.principal (L_mem.mpr (mem_nbhd (embStr tau)))
  rw [ExampleB.sigmaBot_le_iff, L.principal_le_iff, nbhd_subset_iff, embStr_prefix_iff
    ]

open DomainTheory.NeighbourhoodSystems.Example12 (Token zero one)

theorem primrec_notCode {f : Nat  ->  Nat} (hf : Nat.Primrec f) : Nat.Primrec (fun n
    => notCode (f n)) := by
```

```
  refine (primrec_selectFn (primrec_TeqChar hf (Nat.Primrec.const 1)) (Nat.Primrec.
    const 2)
    (primrec_selectFn (primrec_TeqChar hf (Nat.Primrec.const 2)) (Nat.Primrec.const 1)
      (Nat.Primrec.const 0))).of_eq fun n => ?_
  unfold notCode
  by_cases h1 : f n = 1
  * have h1' : TeqChar (f n) 1 = 1 := by unfold TeqChar isZero; omega
    rw [ite_eq_left h1, h1', selectFn_one]
  * have h1' : TeqChar (f n) 1 = 0 := by unfold TeqChar isZero; omega
    rw [ite_eq_right h1, h1', selectFn_zero]
    by_cases h2 : f n = 2
    * have h2' : TeqChar (f n) 2 = 1 := by unfold TeqChar isZero; omega
      rw [ite_eq_left h2, h2', selectFn_one]
    * have h2' : TeqChar (f n) 2 = 0 := by unfold TeqChar isZero; omega
      rw [ite_eq_right h2, h2', selectFn_zero]

theorem notFn_Tenum (n : Nat) : notFn (Tenum n) = Tenum (notCode (min n 2)) := by
  rw [Tenum_eq_canon n]
  have hc2 : min n 2  <=  2 := Nat.min_le_right n 2
  interval_cases (min n 2) <;> simp [Tenum, notCode, notFn_master, notFn_zero,
    notFn_one]

/-- **`notT` is a computable map** (relative to `Tpres` on both sides): its
    neighbourhood relation
`notFn (X_n)  subseteq  X?` reduces, via `notFn_Tenum`, to the *decidable* `Tpres`-
    inclusion relation
`X_{notCode (min n 2)}  subseteq  X?`, hence is recursively decidable -- a fortiori r.e
    . -/
theorem notT_isComputable : ComputableMap Tpres Tpres notT := by
  constructor
  have hcomp : Nat.Primrec (fun t : Nat => Nat.pair (notCode (min t.unpair.1 2)) t.
    unpair.2) :=
    (primrec_notCode (primrec_min Nat.Primrec.left (Nat.Primrec.const 2))).pair Nat.
    Primrec.right
  have hdec : RecDecidable_2 (fun n m => Tenum (notCode (min n 2))  subseteq  Tenum m)
     :=
    RecDecidable.of_iff (fun t => by simp only [unpair_pair_fst, unpair_pair_snd]; rfl
    )
      (Tpres.incl_computable.comp hcomp)
  have hiff : forall  n m, notT.rel (Tenum n) (Tenum m)  <->  Tenum (notCode (min n 2)
    )  subseteq  Tenum m := by
    intro n m
    show (Example23.T.mem (Tenum n)  /\  Example23.T.mem (Tenum m)  /\  notFn (Tenum n
    )  subseteq  Tenum m)  <->  _
    rw [notFn_Tenum]
    exact <fun h => h.2.2, fun h => <Tenum_mem n, Tenum_mem m, h>>
  have key : RecDecidable_2 (fun n m => notT.rel (Tenum n) (Tenum m)) :=
    RecDecidable.of_iff (fun t => hiff t.unpair.1 t.unpair.2) hdec
  exact key.re

theorem primrec_andCode {f g : Nat  ->  Nat} (hf : Nat.Primrec f) (hg : Nat.Primrec g)
     :
    Nat.Primrec (fun n => andCode (f n) (g n)) := by
  refine (primrec_selectFn (primrec_TeqChar hf (Nat.Primrec.const 1)) hg
    (primrec_selectFn (primrec_TeqChar hf (Nat.Primrec.const 2)) (Nat.Primrec.const 2)
      (Nat.Primrec.const 0))).of_eq fun n => ?_
```

```
  unfold andCode
  by_cases h1 : f n = 1
  * have h1' : TeqChar (f n) 1 = 1 := by unfold TeqChar isZero; omega
    rw [ite_eq_left h1, h1', selectFn_one]
  * have h1' : TeqChar (f n) 1 = 0 := by unfold TeqChar isZero; omega
    rw [ite_eq_right h1, h1', selectFn_zero]
    by_cases h2 : f n = 2
    * have h2' : TeqChar (f n) 2 = 1 := by unfold TeqChar isZero; omega
      rw [ite_eq_left h2, h2', selectFn_one]
    * have h2' : TeqChar (f n) 2 = 0 := by unfold TeqChar isZero; omega
      rw [ite_eq_right h2, h2', selectFn_zero]

theorem andFn_Tenum (n m : Nat) :
    andFn (Tenum n) (Tenum m) = Tenum (andCode (min n 2) (min m 2)) := by
  rw [Tenum_eq_canon n, Tenum_eq_canon m]
  have hn2 : min n 2  <=  2 := Nat.min_le_right n 2
  have hm2 : min m 2  <=  2 := Nat.min_le_right m 2
  interval_cases (min n 2) <;> interval_cases (min m 2) <;>
    simp [Tenum, andCode, andFn_master, andFn_zero, andFn_one]

/-- **`andT` is a computable map** (relative to `prodPresentation Tpres Tpres` on the
    input side and
`Tpres` on the output side): its neighbourhood relation reduces, via `ofMap_2`'s
    unfolding and
`andFn_Tenum`, to the *decidable* `Tpres`-inclusion relation `X_{andCode (min n 2) (min
     m 2)}  subseteq  X?`,
hence is recursively decidable -- a fortiori r.e. -/
theorem andT_isComputable : ComputableMap (prodPresentation Tpres Tpres) Tpres andT :=
     by
  constructor
  have hcomp : Nat.Primrec (fun s : Nat => Nat.pair
      (andCode (min s.unpair.1.unpair.1 2) (min s.unpair.1.unpair.2 2)) s.unpair.2) :=
    (primrec_andCode (primrec_min (Nat.Primrec.left.comp Nat.Primrec.left) (Nat.
    Primrec.const 2))
      (primrec_min (Nat.Primrec.right.comp Nat.Primrec.left) (Nat.Primrec.const 2))).
    pair
      Nat.Primrec.right
  have hdec : RecDecidable_2
      (fun t k => Tenum (andCode (min t.unpair.1 2) (min t.unpair.2 2))  subseteq
    Tenum k) :=
    RecDecidable.of_iff (fun s => by simp only [unpair_pair_fst, unpair_pair_snd]; rfl
    )
      (Tpres.incl_computable.comp hcomp)
  have hiff : forall  n m k, andT.rel (prodNbhd (Tenum n) (Tenum m)) (Tenum k)  <->
      Tenum (andCode (min n 2) (min m 2))  subseteq  Tenum k := by
    intro n m k
    have hstep : andT.rel (prodNbhd (Tenum n) (Tenum m)) (Tenum k)  <->
        andMap2.rel (Tenum n) (Tenum m) (Tenum k) := by
      show ((prod Example23.T Example23.T).mem (prodNbhd (Tenum n) (Tenum m))  /\
        andMap2.rel (Sum.inl ^-1' prodNbhd (Tenum n) (Tenum m))
          (Sum.inr ^-1' prodNbhd (Tenum n) (Tenum m)) (Tenum k))  <->  _
      rw [inl_preimage_prodNbhd, inr_preimage_prodNbhd]
      exact and_iff_right (prod_mem_prodNbhd (Tenum_mem n) (Tenum_mem m))
    rw [hstep]
    show (Example23.T.mem (Tenum n)  /\  Example23.T.mem (Tenum m)  /\  Example23.T.
    mem (Tenum k)  /\
```

```
      andFn (Tenum n) (Tenum m)  subseteq  Tenum k)  <->  _
    rw [andFn_Tenum n m]
    exact <fun h => h.2.2.2, fun h => <Tenum_mem n, Tenum_mem m, Tenum_mem k, h>>
  have key : RecDecidable_2
      (fun t k => andT.rel ((prodPresentation Tpres Tpres).X t) (Tpres.X k)) :=
    RecDecidable.of_iff (fun s => by
      show andT.rel ((prodPresentation Tpres Tpres).X s.unpair.1) (Tpres.X s.unpair.2)
      <->  _
      rw [prodPresentation_X]
      exact hiff s.unpair.1.unpair.1 s.unpair.1.unpair.2 s.unpair.2) hdec
  exact key.re

variable {b ? ? : Type*}

/-- **`postcompose` preserves computability.** If `h : V_1  ->  V_2` is computable,
    then so is
`postcompose h : (L  ->  V_1)  ->  (L  ->  V_2)`, relative to any concrete function-
    space presentations built
from `LFunData` bundles: `eval` is computable (Theorem 7.5), composing with `h` stays
    computable
(Proposition 7.3), and `curry`-ing the result stays computable (Theorem 7.5 again). -/
theorem postcompose_isComputable {V_1 : NeighborhoodSystem b} {V_2 :
    NeighborhoodSystem ?}
    {P_1 : ComputablePresentation V_1} {P_2 : ComputablePresentation V_2}
    (d_1 : LFunData P_1) (d_2 : LFunData P_2) {h : ApproximableMap V_1 V_2}
    (hh : ComputableMap P_1 P_2 h) :
    ComputableMap d_1.pres d_2.pres (postcompose h) := by
  have heval : ComputableMap (prodPresentation d_1.pres Lpres) P_1 (evalMap L V_1) :=
    evalMap_isComputable Lpres P_1 d_1.gN d_1.incl0 d_1.incl1 d_1.eq1
      d_1.hgN d_1.hgNp d_1.hincl0 d_1.hincl0p d_1.hincl1 d_1.hincl1p d_1.heq1 d_1.
    heq1p
  exact curry_isComputable d_1.pres Lpres P_2 d_2.gN d_2.incl0 d_2.incl1 d_2.eq1
    d_2.hgN d_2.hgNp d_2.hincl0 d_2.hincl0p d_2.hincl1 d_2.hincl1p d_2.heq1 d_2.heq1p
    (comp_isComputable heval hh)

/-- **`pointwiseBin` preserves computability.** If `h : V_0  x  V_1  ->  V_2` is
    computable, then so is
`pointwiseBin h : (L  ->  V_0)  x  (L  ->  V_1)  ->  (L  ->  V_2)`, by the same
    composition-closure argument as
`postcompose` (projections, pairing, `eval`, and `curry` are all computable). -/
theorem pointwiseBin_isComputable {V_0 : NeighborhoodSystem b} {V_1 :
    NeighborhoodSystem ?}
    {V_2 : NeighborhoodSystem ?} {P_0 : ComputablePresentation V_0} {P_1 :
    ComputablePresentation V_1}
    {P_2 : ComputablePresentation V_2} (d_0 : LFunData P_0) (d_1 : LFunData P_1) (d_2
    : LFunData P_2)
    {h : ApproximableMap (prod V_0 V_1) V_2} (hh : ComputableMap (prodPresentation P_0
     P_1) P_2 h) :
    ComputableMap (prodPresentation d_0.pres d_1.pres) d_2.pres (pointwiseBin h) := by
  have h?AB : ComputableMap (prodPresentation (prodPresentation d_0.pres d_1.pres)
    Lpres)
      (prodPresentation d_0.pres d_1.pres) (proj_0 (prod (funSpace L V_0) (funSpace L
    V_1)) L) :=
    proj_0_isComputable (prodPresentation d_0.pres d_1.pres) Lpres
  have h?L : ComputableMap (prodPresentation (prodPresentation d_0.pres d_1.pres)
    Lpres) Lpres
```

```
      (proj_1 (prod (funSpace L V_0) (funSpace L V_1)) L) :=
    proj_1_isComputable (prodPresentation d_0.pres d_1.pres) Lpres
  have hp0 : ComputableMap (prodPresentation (prodPresentation d_0.pres d_1.pres)
    Lpres) d_0.pres
      (pwFst V_0 V_1) := comp_isComputable h?AB (proj_0_isComputable d_0.pres d_1.pres
    )
  have hp1 : ComputableMap (prodPresentation (prodPresentation d_0.pres d_1.pres)
    Lpres) d_1.pres
      (pwSnd V_0 V_1) := comp_isComputable h?AB (proj_1_isComputable d_0.pres d_1.pres
    )
  have heval0 : ComputableMap (prodPresentation (prodPresentation d_0.pres d_1.pres)
    Lpres) P_0
      ((evalMap L V_0).comp (paired (pwFst V_0 V_1) (pwArg V_0 V_1))) :=
    comp_isComputable (paired_isComputable hp0 h?L)
      (evalMap_isComputable Lpres P_0 d_0.gN d_0.incl0 d_0.incl1 d_0.eq1
        d_0.hgN d_0.hgNp d_0.hincl0 d_0.hincl0p d_0.hincl1 d_0.hincl1p d_0.heq1 d_0.
    heq1p)
  have heval1 : ComputableMap (prodPresentation (prodPresentation d_0.pres d_1.pres)
    Lpres) P_1
      ((evalMap L V_1).comp (paired (pwSnd V_0 V_1) (pwArg V_0 V_1))) :=
    comp_isComputable (paired_isComputable hp1 h?L)
      (evalMap_isComputable Lpres P_1 d_1.gN d_1.incl0 d_1.incl1 d_1.eq1
        d_1.hgN d_1.hgNp d_1.hincl0 d_1.hincl0p d_1.hincl1 d_1.hincl1p d_1.heq1 d_1.
    heq1p)
  exact curry_isComputable (prodPresentation d_0.pres d_1.pres) Lpres P_2 d_2.gN d_2.
    incl0 d_2.incl1
    d_2.eq1 d_2.hgN d_2.hgNp d_2.hincl0 d_2.hincl0p d_2.hincl1 d_2.hincl1p d_2.heq1
    d_2.heq1p
    (comp_isComputable (paired_isComputable heval0 heval1) hh)

/-- **Exercise 7.24, fourth claim, `notT` case.** LUCID's pointwise negation lifts to a
     computable


map of type `(L -> T)  ->  (L -> T)`. -/
theorem notT_lifted_isComputable : ComputableMap LTpres LTpres (postcompose notT) :=
  postcompose_isComputable LTdata LTdata notT_isComputable

/-- **Exercise 7.24, fourth claim, `andT` case.** LUCID's pointwise (sequential)
    conjunction lifts to
a computable map of type `(L -> T)  x  (L -> T)  ->  (L -> T)`. -/
theorem andT_lifted_isComputable :
    ComputableMap (prodPresentation LTpres LTpres) LTpres (pointwiseBin andT) :=
  pointwiseBin_isComputable LTdata LTdata LTdata andT_isComputable

/-- **A deeper LUCID program is still computable.** `deMorganT f g := not (not f  /\
    not g)` on `(L -> T)`,
built by composing/pairing the two lifted combinators above (De Morgan's law for the
    sequential
`and`); it is computable purely by the closure lemmas `comp_isComputable`/`
    paired_isComputable`,
**with no fresh recursion-theoretic argument needed** -- exactly Scott's "conclude
    that programs in
LUCID define computable maps." -/
theorem deMorganT_isComputable :
    ComputableMap (prodPresentation LTpres LTpres) LTpres
      ((postcompose notT).comp (pointwiseBin andT)
```

```
        |>.comp (paired
          (postcompose notT |>.comp (proj_0 (funSpace L Example23.T) (funSpace L
    Example23.T)))
          (postcompose notT |>.comp (proj_1 (funSpace L Example23.T) (funSpace L
    Example23.T))))) :=
  comp_isComputable
    (paired_isComputable
      (comp_isComputable (proj_0_isComputable LTpres LTpres) notT_lifted_isComputable)
      (comp_isComputable (proj_1_isComputable LTpres LTpres) notT_lifted_isComputable)
    )
    (comp_isComputable andT_lifted_isComputable notT_lifted_isComputable)
end

section
/-- **The defining equation of `Gamma`, read as a theorem.** Every `?  in  Gamma` is
    either `*` or `(i, ?')`
for some `i, ?'`: `Gamma = (?_i {i}  x  Gamma)  U  {*}`. -/
theorem Gamma_cases (? : Gamma) : ? = star  \/  exists  i ?', ? = cons i ?' := by
  cases ? with
  | inl l =>
    cases l with
    | nil => exact Or.inl rfl
    | cons i l' => exact Or.inr <i, .inl l', rfl>
  | inr s =>
    refine Or.inr <s 0, .inr (fun n => s (n + 1)), ?_>
    simp only [cons]
    congr 1
    funext n
    cases n <;> rfl
end

section
/-- **Exercise 7.24 (i) -- `L` is effectively given.** -/
theorem L_isEffectivelyGiven : L.IsEffectivelyGiven := <Lpres>
end

section
set_option linter.unusedVariables false

theorem example_724 : And
  (And
    (And
      (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven
        DomainTheory.NeighbourhoodSystems.Exercise724.Gamma DomainTheory.
    NeighbourhoodSystems.Exercise724.L)
      (@DomainTheory.NeighbourhoodSystems.ComputableMap
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap DomainTheory.
    NeighbourhoodSystems.Exercise724.Gamma
          DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
    NeighbourhoodSystems.Exercise724.L
          DomainTheory.NeighbourhoodSystems.Example23.T)
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap DomainTheory.
    NeighbourhoodSystems.Exercise724.Gamma
          DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
    NeighbourhoodSystems.Exercise724.L
          DomainTheory.NeighbourhoodSystems.Example23.T)
```

```
    (@DomainTheory.NeighbourhoodSystems.funSpace DomainTheory.NeighbourhoodSystems
.Exercise724.Gamma
      DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
NeighbourhoodSystems.Exercise724.L
      DomainTheory.NeighbourhoodSystems.Example23.T)
    (@DomainTheory.NeighbourhoodSystems.funSpace DomainTheory.NeighbourhoodSystems
.Exercise724.Gamma
      DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
NeighbourhoodSystems.Exercise724.L
      DomainTheory.NeighbourhoodSystems.Example23.T)
    DomainTheory.NeighbourhoodSystems.Exercise724.LTpres DomainTheory.
NeighbourhoodSystems.Exercise724.LTpres
    (@DomainTheory.NeighbourhoodSystems.Exercise724.postcompose DomainTheory.
NeighbourhoodSystems.Example12.Token
      DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
NeighbourhoodSystems.Example23.T
      DomainTheory.NeighbourhoodSystems.Example23.T DomainTheory.
NeighbourhoodSystems.Exercise724.notT)))
(@DomainTheory.NeighbourhoodSystems.ComputableMap
  (Sum
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap DomainTheory.
NeighbourhoodSystems.Exercise724.Gamma
      DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
NeighbourhoodSystems.Exercise724.L
      DomainTheory.NeighbourhoodSystems.Example23.T)
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap DomainTheory.
NeighbourhoodSystems.Exercise724.Gamma
      DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
NeighbourhoodSystems.Exercise724.L
      DomainTheory.NeighbourhoodSystems.Example23.T))
  (@DomainTheory.NeighbourhoodSystems.ApproximableMap DomainTheory.
NeighbourhoodSystems.Exercise724.Gamma
    DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
NeighbourhoodSystems.Exercise724.L
    DomainTheory.NeighbourhoodSystems.Example23.T)
  (@DomainTheory.NeighbourhoodSystems.prod
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap DomainTheory.
NeighbourhoodSystems.Exercise724.Gamma
      DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
NeighbourhoodSystems.Exercise724.L
      DomainTheory.NeighbourhoodSystems.Example23.T)
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap DomainTheory.
NeighbourhoodSystems.Exercise724.Gamma
      DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
NeighbourhoodSystems.Exercise724.L
      DomainTheory.NeighbourhoodSystems.Example23.T)
    (@DomainTheory.NeighbourhoodSystems.funSpace DomainTheory.NeighbourhoodSystems
.Exercise724.Gamma
      DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
NeighbourhoodSystems.Exercise724.L
      DomainTheory.NeighbourhoodSystems.Example23.T)
    (@DomainTheory.NeighbourhoodSystems.funSpace DomainTheory.NeighbourhoodSystems
.Exercise724.Gamma
      DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
NeighbourhoodSystems.Exercise724.L
      DomainTheory.NeighbourhoodSystems.Example23.T))
```

```
    (@DomainTheory.NeighbourhoodSystems.funSpace DomainTheory.NeighbourhoodSystems.
  Exercise724.Gamma
      DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
  NeighbourhoodSystems.Exercise724.L
      DomainTheory.NeighbourhoodSystems.Example23.T)
    (@DomainTheory.NeighbourhoodSystems.prodPresentation
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap DomainTheory.
  NeighbourhoodSystems.Exercise724.Gamma
        DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
  NeighbourhoodSystems.Exercise724.L
        DomainTheory.NeighbourhoodSystems.Example23.T)
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap DomainTheory.
  NeighbourhoodSystems.Exercise724.Gamma
        DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
  NeighbourhoodSystems.Exercise724.L
        DomainTheory.NeighbourhoodSystems.Example23.T)
      (@DomainTheory.NeighbourhoodSystems.funSpace DomainTheory.NeighbourhoodSystems
  .Exercise724.Gamma
        DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
  NeighbourhoodSystems.Exercise724.L
        DomainTheory.NeighbourhoodSystems.Example23.T)
      (@DomainTheory.NeighbourhoodSystems.funSpace DomainTheory.NeighbourhoodSystems
  .Exercise724.Gamma
        DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
  NeighbourhoodSystems.Exercise724.L
        DomainTheory.NeighbourhoodSystems.Example23.T)
      DomainTheory.NeighbourhoodSystems.Exercise724.LTpres DomainTheory.
  NeighbourhoodSystems.Exercise724.LTpres)
    DomainTheory.NeighbourhoodSystems.Exercise724.LTpres
    (@DomainTheory.NeighbourhoodSystems.Exercise724.pointwiseBin DomainTheory.
  NeighbourhoodSystems.Example12.Token
      DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
  NeighbourhoodSystems.Example12.Token
      DomainTheory.NeighbourhoodSystems.Example23.T DomainTheory.
  NeighbourhoodSystems.Example23.T
      DomainTheory.NeighbourhoodSystems.Example23.T DomainTheory.
  NeighbourhoodSystems.Exercise724.andT)))
(@Function.Bijective DomainTheory.NeighbourhoodSystems.Exercise724.Gamma
  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element DomainTheory.
  NeighbourhoodSystems.Exercise724.Gamma
    DomainTheory.NeighbourhoodSystems.Exercise724.L)
  DomainTheory.NeighbourhoodSystems.Exercise724.toElement) :=
@And.intro
(And
  (And
    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven
      DomainTheory.NeighbourhoodSystems.Exercise724.Gamma DomainTheory.
  NeighbourhoodSystems.Exercise724.L)
    (@DomainTheory.NeighbourhoodSystems.ComputableMap
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap DomainTheory.
  NeighbourhoodSystems.Exercise724.Gamma
        DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
  NeighbourhoodSystems.Exercise724.L
        DomainTheory.NeighbourhoodSystems.Example23.T)
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap DomainTheory.
  NeighbourhoodSystems.Exercise724.Gamma
```

```
      DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
NeighbourhoodSystems.Exercise724.L
      DomainTheory.NeighbourhoodSystems.Example23.T)
    (@DomainTheory.NeighbourhoodSystems.funSpace DomainTheory.NeighbourhoodSystems
.Exercise724.Gamma
      DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
NeighbourhoodSystems.Exercise724.L
      DomainTheory.NeighbourhoodSystems.Example23.T)
    (@DomainTheory.NeighbourhoodSystems.funSpace DomainTheory.NeighbourhoodSystems
.Exercise724.Gamma
      DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
NeighbourhoodSystems.Exercise724.L
      DomainTheory.NeighbourhoodSystems.Example23.T)
    DomainTheory.NeighbourhoodSystems.Exercise724.LTpres DomainTheory.
NeighbourhoodSystems.Exercise724.LTpres
    (@DomainTheory.NeighbourhoodSystems.Exercise724.postcompose DomainTheory.
NeighbourhoodSystems.Example12.Token
      DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
NeighbourhoodSystems.Example23.T
      DomainTheory.NeighbourhoodSystems.Example23.T DomainTheory.
NeighbourhoodSystems.Exercise724.notT)))
 (@DomainTheory.NeighbourhoodSystems.ComputableMap
   (Sum
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap DomainTheory.
NeighbourhoodSystems.Exercise724.Gamma
      DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
NeighbourhoodSystems.Exercise724.L
      DomainTheory.NeighbourhoodSystems.Example23.T)
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap DomainTheory.
NeighbourhoodSystems.Exercise724.Gamma
      DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
NeighbourhoodSystems.Exercise724.L
      DomainTheory.NeighbourhoodSystems.Example23.T))
   (@DomainTheory.NeighbourhoodSystems.ApproximableMap DomainTheory.
NeighbourhoodSystems.Exercise724.Gamma
    DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
NeighbourhoodSystems.Exercise724.L
    DomainTheory.NeighbourhoodSystems.Example23.T)
   (@DomainTheory.NeighbourhoodSystems.prod
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap DomainTheory.
NeighbourhoodSystems.Exercise724.Gamma
      DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
NeighbourhoodSystems.Exercise724.L
      DomainTheory.NeighbourhoodSystems.Example23.T)
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap DomainTheory.
NeighbourhoodSystems.Exercise724.Gamma
      DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
NeighbourhoodSystems.Exercise724.L
      DomainTheory.NeighbourhoodSystems.Example23.T)
    (@DomainTheory.NeighbourhoodSystems.funSpace DomainTheory.NeighbourhoodSystems
.Exercise724.Gamma
      DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
NeighbourhoodSystems.Exercise724.L
      DomainTheory.NeighbourhoodSystems.Example23.T)
    (@DomainTheory.NeighbourhoodSystems.funSpace DomainTheory.NeighbourhoodSystems
.Exercise724.Gamma
```

```
       DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
  NeighbourhoodSystems.Exercise724.L
       DomainTheory.NeighbourhoodSystems.Example23.T))
    (@DomainTheory.NeighbourhoodSystems.funSpace DomainTheory.NeighbourhoodSystems.
  Exercise724.Gamma
      DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
  NeighbourhoodSystems.Exercise724.L
      DomainTheory.NeighbourhoodSystems.Example23.T)
    (@DomainTheory.NeighbourhoodSystems.prodPresentation
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap DomainTheory.
  NeighbourhoodSystems.Exercise724.Gamma
       DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
  NeighbourhoodSystems.Exercise724.L
       DomainTheory.NeighbourhoodSystems.Example23.T)
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap DomainTheory.
  NeighbourhoodSystems.Exercise724.Gamma
       DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
  NeighbourhoodSystems.Exercise724.L
       DomainTheory.NeighbourhoodSystems.Example23.T)
      (@DomainTheory.NeighbourhoodSystems.funSpace DomainTheory.NeighbourhoodSystems
  .Exercise724.Gamma
       DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
  NeighbourhoodSystems.Exercise724.L
       DomainTheory.NeighbourhoodSystems.Example23.T)
      (@DomainTheory.NeighbourhoodSystems.funSpace DomainTheory.NeighbourhoodSystems
  .Exercise724.Gamma
       DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
  NeighbourhoodSystems.Exercise724.L
       DomainTheory.NeighbourhoodSystems.Example23.T)
      DomainTheory.NeighbourhoodSystems.Exercise724.LTpres DomainTheory.
  NeighbourhoodSystems.Exercise724.LTpres)
    DomainTheory.NeighbourhoodSystems.Exercise724.LTpres
    (@DomainTheory.NeighbourhoodSystems.Exercise724.pointwiseBin DomainTheory.
  NeighbourhoodSystems.Example12.Token
      DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
  NeighbourhoodSystems.Example12.Token
      DomainTheory.NeighbourhoodSystems.Example23.T DomainTheory.
  NeighbourhoodSystems.Example23.T
      DomainTheory.NeighbourhoodSystems.Example23.T DomainTheory.
  NeighbourhoodSystems.Exercise724.andT)))
(@Function.Bijective DomainTheory.NeighbourhoodSystems.Exercise724.Gamma
  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element DomainTheory.
  NeighbourhoodSystems.Exercise724.Gamma
    DomainTheory.NeighbourhoodSystems.Exercise724.L)
  DomainTheory.NeighbourhoodSystems.Exercise724.toElement)
(@And.intro
  (And
    (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven
      DomainTheory.NeighbourhoodSystems.Exercise724.Gamma DomainTheory.
  NeighbourhoodSystems.Exercise724.L)
    (@DomainTheory.NeighbourhoodSystems.ComputableMap
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap DomainTheory.
  NeighbourhoodSystems.Exercise724.Gamma
       DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
  NeighbourhoodSystems.Exercise724.L
       DomainTheory.NeighbourhoodSystems.Example23.T)
```

```
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap DomainTheory.
NeighbourhoodSystems.Exercise724.Gamma
      DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
NeighbourhoodSystems.Exercise724.L
      DomainTheory.NeighbourhoodSystems.Example23.T)
    (@DomainTheory.NeighbourhoodSystems.funSpace DomainTheory.NeighbourhoodSystems
.Exercise724.Gamma
      DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
NeighbourhoodSystems.Exercise724.L
      DomainTheory.NeighbourhoodSystems.Example23.T)
    (@DomainTheory.NeighbourhoodSystems.funSpace DomainTheory.NeighbourhoodSystems
.Exercise724.Gamma
      DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
NeighbourhoodSystems.Exercise724.L
      DomainTheory.NeighbourhoodSystems.Example23.T)
    DomainTheory.NeighbourhoodSystems.Exercise724.LTpres DomainTheory.
NeighbourhoodSystems.Exercise724.LTpres
    (@DomainTheory.NeighbourhoodSystems.Exercise724.postcompose DomainTheory.
NeighbourhoodSystems.Example12.Token
      DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
NeighbourhoodSystems.Example23.T
      DomainTheory.NeighbourhoodSystems.Example23.T DomainTheory.
NeighbourhoodSystems.Exercise724.notT)))
(@DomainTheory.NeighbourhoodSystems.ComputableMap
  (Sum
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap DomainTheory.
NeighbourhoodSystems.Exercise724.Gamma
      DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
NeighbourhoodSystems.Exercise724.L
      DomainTheory.NeighbourhoodSystems.Example23.T)
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap DomainTheory.
NeighbourhoodSystems.Exercise724.Gamma
      DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
NeighbourhoodSystems.Exercise724.L
      DomainTheory.NeighbourhoodSystems.Example23.T))
  (@DomainTheory.NeighbourhoodSystems.ApproximableMap DomainTheory.
NeighbourhoodSystems.Exercise724.Gamma
    DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
NeighbourhoodSystems.Exercise724.L
    DomainTheory.NeighbourhoodSystems.Example23.T)
  (@DomainTheory.NeighbourhoodSystems.prod
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap DomainTheory.
NeighbourhoodSystems.Exercise724.Gamma
      DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
NeighbourhoodSystems.Exercise724.L
      DomainTheory.NeighbourhoodSystems.Example23.T)
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap DomainTheory.
NeighbourhoodSystems.Exercise724.Gamma
      DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
NeighbourhoodSystems.Exercise724.L
      DomainTheory.NeighbourhoodSystems.Example23.T)
    (@DomainTheory.NeighbourhoodSystems.funSpace DomainTheory.NeighbourhoodSystems
.Exercise724.Gamma
      DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
NeighbourhoodSystems.Exercise724.L
      DomainTheory.NeighbourhoodSystems.Example23.T)
```

```
    (@DomainTheory.NeighbourhoodSystems.funSpace DomainTheory.NeighbourhoodSystems
.Exercise724.Gamma
      DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
NeighbourhoodSystems.Exercise724.L
      DomainTheory.NeighbourhoodSystems.Example23.T))
  (@DomainTheory.NeighbourhoodSystems.funSpace DomainTheory.NeighbourhoodSystems.
Exercise724.Gamma
    DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
NeighbourhoodSystems.Exercise724.L
    DomainTheory.NeighbourhoodSystems.Example23.T)
  (@DomainTheory.NeighbourhoodSystems.prodPresentation
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap DomainTheory.
NeighbourhoodSystems.Exercise724.Gamma
      DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
NeighbourhoodSystems.Exercise724.L
      DomainTheory.NeighbourhoodSystems.Example23.T)
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap DomainTheory.
NeighbourhoodSystems.Exercise724.Gamma
      DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
NeighbourhoodSystems.Exercise724.L
      DomainTheory.NeighbourhoodSystems.Example23.T)
    (@DomainTheory.NeighbourhoodSystems.funSpace DomainTheory.NeighbourhoodSystems
.Exercise724.Gamma
      DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
NeighbourhoodSystems.Exercise724.L
      DomainTheory.NeighbourhoodSystems.Example23.T)
    (@DomainTheory.NeighbourhoodSystems.funSpace DomainTheory.NeighbourhoodSystems
.Exercise724.Gamma
      DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
NeighbourhoodSystems.Exercise724.L
      DomainTheory.NeighbourhoodSystems.Example23.T)
    DomainTheory.NeighbourhoodSystems.Exercise724.LTpres DomainTheory.
NeighbourhoodSystems.Exercise724.LTpres)
  DomainTheory.NeighbourhoodSystems.Exercise724.LTpres
  (@DomainTheory.NeighbourhoodSystems.Exercise724.pointwiseBin DomainTheory.
NeighbourhoodSystems.Example12.Token
    DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
NeighbourhoodSystems.Example12.Token
    DomainTheory.NeighbourhoodSystems.Example23.T DomainTheory.
NeighbourhoodSystems.Example23.T
    DomainTheory.NeighbourhoodSystems.Example23.T DomainTheory.
NeighbourhoodSystems.Exercise724.andT))
(@And.intro
  (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven
    DomainTheory.NeighbourhoodSystems.Exercise724.Gamma DomainTheory.
NeighbourhoodSystems.Exercise724.L)
  (@DomainTheory.NeighbourhoodSystems.ComputableMap
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap DomainTheory.
NeighbourhoodSystems.Exercise724.Gamma
      DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
NeighbourhoodSystems.Exercise724.L
      DomainTheory.NeighbourhoodSystems.Example23.T)
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap DomainTheory.
NeighbourhoodSystems.Exercise724.Gamma
      DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
NeighbourhoodSystems.Exercise724.L
```

```
          DomainTheory.NeighbourhoodSystems.Example23.T)
        (@DomainTheory.NeighbourhoodSystems.funSpace DomainTheory.NeighbourhoodSystems
    .Exercise724.Gamma
          DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
    NeighbourhoodSystems.Exercise724.L
          DomainTheory.NeighbourhoodSystems.Example23.T)
        (@DomainTheory.NeighbourhoodSystems.funSpace DomainTheory.NeighbourhoodSystems
    .Exercise724.Gamma
          DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
    NeighbourhoodSystems.Exercise724.L
          DomainTheory.NeighbourhoodSystems.Example23.T)
        DomainTheory.NeighbourhoodSystems.Exercise724.LTpres DomainTheory.
    NeighbourhoodSystems.Exercise724.LTpres
        (@DomainTheory.NeighbourhoodSystems.Exercise724.postcompose DomainTheory.
    NeighbourhoodSystems.Example12.Token
          DomainTheory.NeighbourhoodSystems.Example12.Token DomainTheory.
    NeighbourhoodSystems.Example23.T
          DomainTheory.NeighbourhoodSystems.Example23.T DomainTheory.
    NeighbourhoodSystems.Exercise724.notT))
      DomainTheory.NeighbourhoodSystems.Exercise724.L_isEffectivelyGiven
      DomainTheory.NeighbourhoodSystems.Exercise724.notT_lifted_isComputable)
    DomainTheory.NeighbourhoodSystems.Exercise724.andT_lifted_isComputable)
  DomainTheory.NeighbourhoodSystems.Exercise724.toElement_bijective
end

section
theorem LenumSubsetChar_le_one (t : Nat) : LenumSubsetChar t  <=  1 :=
  isPrefixChar_le_one _ _
end

section
theorem Lenum_subset_char_eq_one_iff (n m : Nat) :
    Lenum n  subseteq  Lenum m  <->  LenumSubsetChar (Nat.pair n m) = 1 := by
  unfold LenumSubsetChar
  rw [unpair_pair_fst, unpair_pair_snd, Lenum_subset_iff]
end

section
open DomainTheory.NeighbourhoodSystems.Example12 (Token zero one)

theorem T_isEffectivelyGiven : Example23.T.IsEffectivelyGiven := <Tpres>

theorem TeqChar_eq_one_iff (a b : Nat) : TeqChar a b = 1  <->  a = b := by
  unfold TeqChar isZero; omega

end

section
/-- `embStr` is injective. -/
theorem embStr_injective : Function.Injective embStr :=
  List.map_injective_iff.mpr bitToNat_injective
end

section
theorem primrec_LenumSubsetChar : Nat.Primrec LenumSubsetChar :=
  primrec_isPrefixChar_swap.of_eq fun t => by simp only [LenumSubsetChar]
```

```
end

section
/-- `isPrefixChar` gives a `RecDecidable_2` decider for `Lenum n  subseteq  Lenum m`.
    -/
theorem Lenum_subset_recDecidable : RecDecidable_2 (fun n m => Lenum n  subseteq
    Lenum m) :=
  RecDecidable_2.of_paired_zero_one_char primrec_LenumSubsetChar
    (fun t => by have := LenumSubsetChar_le_one t; omega)
    Lenum_subset_char_eq_one_iff
end

end DomainTheory.NeighbourhoodSystems.Exercise724
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive ApproximableMap

section
variable {a : Type*} {V : NeighborhoodSystem a}

/-- The one-entry function-space code: `Xenum (codePair a b) = [X_a, X_b]`. -/
def codePair (a b : Nat) : Nat := Nat.pair (Nat.pair a b) 0 + 1
end

section
variable {a : Type*} {V : NeighborhoodSystem a}
variable (P : ComputablePresentation V)

/-- The decidable mirror of `gStepsOK`, phrased with the function-space inclusion char
    `fincl`. -/
def chainDec (fincl : Nat  ->  Nat) (c : Nat) : Nat  ->  List Nat  ->  Prop
  | _, [] => True
  | a, (b :: rest) => fincl (Nat.pair c (codePair a b)) = 1  /\  chainDec fincl c b
    rest

/-- The pure left-fold step corresponding to `fixStp` (parameter `c`, accumulator `acc
    = pair prev
flag`, current element `x`). -/
def fixPStep (fincl : Nat  ->  Nat) (c acc x : Nat) : Nat :=
  Nat.pair x (selectFn acc.unpair.2 (isOne (fincl (Nat.pair c (codePair acc.unpair.1 x
    )))) 0)

/-- One packed `foldCode` step: state `pair acc params` with `acc = pair prev flag`,
    current element
`x`. Updates the flag with the one-step consistency test and advances `prev` to `x`. -/
def fixStp (fincl : Nat  ->  Nat) (w : Nat) : Nat :=
  Nat.pair w.unpair.1
    (selectFn w.unpair.2.unpair.1.unpair.2
      (isOne (fincl (Nat.pair w.unpair.2.unpair.2
        (codePair w.unpair.2.unpair.1.unpair.1 w.unpair.1)))) 0)

/-- The chain `foldCode`: starts from `pair masterIdx 1`, threads the parameter `c`,
    over the list
coded by `i`. Returns `pair lastIdx flag`. -/
def fixChainChar (fincl : Nat  ->  Nat) (c i : Nat) : Nat :=
  foldCode (fixStp fincl) c (Nat.pair P.masterIdx 1) i
```

```
end

section
variable {a : Type*} {V : NeighborhoodSystem a}

/-- The last index of the chain `a, full_0, full_1, ...`. -/
def gLastOf : Nat  ->  List Nat  ->  Nat
  | a, [] => a
  | _, (b :: rest) => gLastOf b rest

/-- The consecutive `g`-steps `g.rel (X_a) (X_{full_0})`, `g.rel (X_{full_0}) (X_{
    full_1})`, ... all hold. -/
def gStepsOK (g : ApproximableMap V V) (P : ComputablePresentation V) : Nat  ->  List
    Nat  ->  Prop
  | _, [] => True
  | a, (b :: rest) => g.rel (P.X a) (P.X b)  /\  gStepsOK g P b rest

end

section
variable {a : Type*} {V : NeighborhoodSystem a}
variable (P : ComputablePresentation V)

/-- `Xenum (codePair a b) = [X_a, X_b]` (a one-entry, always-consistent function-space
neighbourhood); a thin wrapper around `Xenum_singleton`. -/
theorem Xenum_codePair (gN : Nat  ->  Nat)
    (hgN : forall  c, gN c = 1  <->  (stepFun (funListOf P P (decodeList c))
      : Set (ApproximableMap V V)).Nonempty) (a b : Nat) :
    Xenum P P gN (codePair a b) = step (P.X a) (P.X b) :=
  Xenum_singleton P P gN hgN a b

/-- The `foldCode` tracks the last index reached. -/
theorem fixPStep_foldl_fst (fincl : Nat  ->  Nat) (c : Nat) :


    forall  (full : List Nat) (prev fl : Nat),
      (List.foldl (fixPStep fincl c) (Nat.pair prev fl) full).unpair.1 = gLastOf prev
    full := by
  intro full
  induction full with
  | nil =>
    intro prev fl
    show (Nat.pair prev fl).unpair.1 = prev
    exact unpair_pair_fst prev fl
  | cons b rest ih =>
    intro prev fl
    rw [List.foldl_cons]
    have hstep : fixPStep fincl c (Nat.pair prev fl) b
        = Nat.pair b (selectFn fl (isOne (fincl (Nat.pair c (codePair prev b)))) 0) :=
     by
      unfold fixPStep; simp only [unpair_pair_fst, unpair_pair_snd]
    rw [hstep, ih b (selectFn fl (isOne (fincl (Nat.pair c (codePair prev b)))) 0)]
    rfl

/-- The `foldCode` flag is `1` iff the start flag is `1` and the whole chain is
    consistent. -/
theorem fixPStep_foldl_snd (fincl : Nat  ->  Nat) (c : Nat) :
```

```
    forall  (full : List Nat) (prev fl : Nat), fl  <=  1  ->
      ((List.foldl (fixPStep fincl c) (Nat.pair prev fl) full).unpair.2 = 1  <->
        fl = 1  /\  chainDec fincl c prev full) := by
  intro full
  induction full with
  | nil =>
    intro prev fl _
    show (Nat.pair prev fl).unpair.2 = 1  <->  fl = 1  /\  chainDec fincl c prev []
    rw [unpair_pair_snd]
    simp [chainDec]
  | cons b rest ih =>
    intro prev fl _
    rw [List.foldl_cons]
    have hstep : fixPStep fincl c (Nat.pair prev fl) b
        = Nat.pair b (selectFn fl (isOne (fincl (Nat.pair c (codePair prev b)))) 0) :=
     by
      unfold fixPStep; simp only [unpair_pair_fst, unpair_pair_snd]
    rw [hstep]
    set nf := selectFn fl (isOne (fincl (Nat.pair c (codePair prev b)))) 0 with hnf
    have hnfle : nf  <=  1 := by
      rw [hnf]
      rcases (show fl = 0  \/  fl = 1 by omega) with h | h <;> rw [h]
      * rw [selectFn_zero]; exact Nat.zero_le 1
      * rw [selectFn_one]; exact isOne_le_one _
    rw [ih b nf hnfle]
    have hnf1 : nf = 1  <->  (fl = 1  /\  fincl (Nat.pair c (codePair prev b)) = 1) :=
     by
      rw [hnf]
      rcases (show fl = 0  \/  fl = 1 by omega) with h | h <;> rw [h]
      * rw [selectFn_zero]; simp
      * rw [selectFn_one, isOne_eq_one_iff]; simp
    rw [hnf1]
    constructor
    * rintro <<hfl1, hfincl1>, hrest>; exact <hfl1, hfincl1, hrest>
    * rintro <hfl1, hfincl1, hrest>; exact <<hfl1, hfincl1>, hrest>

theorem fixStp_eq (fincl : Nat  ->  Nat) (c acc x : Nat) :
    fixStp fincl (Nat.pair x (Nat.pair acc c)) = fixPStep fincl c acc x := by
  unfold fixStp fixPStep
  simp only [unpair_pair_fst, unpair_pair_snd]

theorem fixChainChar_eq (fincl : Nat  ->  Nat) (c i : Nat) :
    fixChainChar P fincl c i
      = List.foldl (fixPStep fincl c) (Nat.pair P.masterIdx 1) (decodeList i) := by
  unfold fixChainChar
  rw [foldCode_eq']
  have hstep : (fun acc x => fixStp fincl (Nat.pair x (Nat.pair acc c))) = fixPStep
    fincl c := by
    funext acc x; exact fixStp_eq fincl c acc x
  rw [hstep]

/-- Read-off of the `foldCode`: its `.1` is the last index of `decodeList i`, and its
    `.2 = 1` iff
the decoded chain is consistent. -/
theorem fixChainChar_spec (fincl : Nat  ->  Nat) (c i : Nat) :
    (fixChainChar P fincl c i).unpair.1 = gLastOf P.masterIdx (decodeList i)
```

```
   /\  ((fixChainChar P fincl c i).unpair.2 = 1  <->  chainDec fincl c P.masterIdx (
    decodeList i)) := by
  rw [fixChainChar_eq]
  refine <fixPStep_foldl_fst fincl c (decodeList i) P.masterIdx 1, ?_>
  rw [fixPStep_foldl_snd fincl c (decodeList i) P.masterIdx 1 (le_refl 1)]
  simp

end

section
variable {a : Type*} {V : NeighborhoodSystem a}

/-- **Completeness of a chain.** If `(g?).rel (X_a) (X_?)`, then there is a valid `g`-
    chain from `a`
whose final neighbourhood is contained in `X_?`. (The intermediate neighbourhoods are
    named via
`surj`; the relaxed final condition `X_{last}  subseteq  X_?` handles the `n = 0` base
    .) -/
theorem gStepsOK_complete (g : ApproximableMap V V) (P : ComputablePresentation V) :
    forall  (n a ? : Nat), (g.iterMap n).rel (P.X a) (P.X ?)  ->
      exists  full, gStepsOK g P a full  /\  P.X (gLastOf a full)  subseteq  P.X ? :=
    by
  intro n
  induction n with
  | zero =>
    intro a ? h
    rw [iterMap_zero, idMap_rel] at h
    exact <[], trivial, h.2.2>
  | succ n ih =>
    intro a ? h
    have hcomm : g.iterMap (n + 1) = (g.iterMap n).comp g := by
      rw [iterMap_succ]; exact iter_comm g n
    rw [hcomm, comp_rel] at h
    obtain <Y, h1, h2> := h
    obtain <k, hk> := P.surj (g.rel_cod h1)
    rw [ <-  hk] at h1 h2
    obtain <full', hs', hl'> := ih k ? h2
    exact <k :: full', <h1, hs'>, hl'>

/-- **Soundness of a chain.** A valid `g`-chain from `a` realises the iterate relation
`(g?).rel (X_a) (X_{last})` with `k` the chain length. -/
theorem gStepsOK_sound (g : ApproximableMap V V) (P : ComputablePresentation V) :
    forall  (full : List Nat) (a : Nat), gStepsOK g P a full  ->
      (g.iterMap full.length).rel (P.X a) (P.X (gLastOf a full)) := by
  intro full
  induction full with
  | nil =>
    intro a _
    show (g.iterMap ([] : List Nat).length).rel (P.X a) (P.X a)
    rw [List.length_nil, iterMap_zero]
    exact idMap_rel.mpr <P.mem_X a, P.mem_X a, subset_rfl>
  | cons b rest ih =>
    intro a hsteps
    obtain <h1, hrest> := hsteps
    have hcomm : g.iterMap (rest.length + 1) = (g.iterMap rest.length).comp g := by
      rw [iterMap_succ]; exact iter_comm g rest.length
```

```
    show (g.iterMap (b :: rest).length).rel (P.X a) (P.X (gLastOf b rest))
    rw [List.length_cons, hcomm, comp_rel]
    exact <P.X b, h1, ih b hrest>

/-- **Chain characterisation of `g`'s least fixed point.** A neighbourhood `X_?` is in
    `fix(g)` iff a
valid `g`-chain (from a master index `a_0` with `X_{a_0} = Delta`) reaches a final
    neighbourhood inside
`X_?`. -/
theorem fixElement_mem_iff_chain (g : ApproximableMap V V) (P : ComputablePresentation
     V)
    (a0 : Nat) (ha0 : P.X a0 = V.master) (? : Nat) :
    g.fixElement.mem (P.X ?)  <->
      exists  full, gStepsOK g P a0 full  /\  P.X (gLastOf a0 full)  subseteq  P.X ?
    := by
  rw [mem_fixElement]
  constructor
  * rintro <n, hn>
    rw [ <-  ha0] at hn
    exact gStepsOK_complete g P n a0 ? hn
  * rintro <full, hsteps, hlast>
    refine <full.length, ?_>
    have hsound := gStepsOK_sound g P full a0 hsteps
    have hmono := (g.iterMap full.length).mono hsound subset_rfl hlast (P.mem_X a0) (P
    .mem_X ?)
    rwa [ha0] at hmono

end

section
variable {a : Type*} {V : NeighborhoodSystem a}
variable (P : ComputablePresentation V)

/-- **The `fix` neighbourhood relation, chain form.** `(fix V).rel (Xenum c) (X_?)`
    holds iff there is
a valid least-map chain from the master index reaching a neighbourhood inside `X_?`.
    This is Scott's
"`?[X_{n_i}, X_{m_i}] fix X_?` iff for some finite sequence ...". -/
theorem fixMap_rel_iff (gN : Nat  ->  Nat)
    (hgN : forall  c, gN c = 1  <->  (stepFun (funListOf P P (decodeList c))
      : Set (ApproximableMap V V)).Nonempty) (c ? : Nat) :
    (fixMap V).rel (Xenum P P gN c) (P.X ?)  <->
      exists  full : List Nat,
        gStepsOK (toApproxMap ((funSpace V V).principal (Xenum_mem P P gN hgN c))) P P
    .masterIdx full
         /\  P.X (gLastOf P.masterIdx full)  subseteq  P.X ? := by
  rw [(fixMap V).rel_iff_mem_principal (Xenum_mem P P gN hgN c), fixMap_toElementMap]
  exact fixElement_mem_iff_chain _ P P.masterIdx P.masterIdx_spec ?

/-- The least map `g` of the function-space neighbourhood `Xenum c` relates `X_a` to `
    X_b` exactly
when `Xenum c  subseteq  [X_a, X_b]` (Scott's `?{X_{m_i} ? X_a  subseteq  X_{n_i}}
    subseteq  X_b`). -/
theorem leastMap_Xenum_rel (gN : Nat  ->  Nat)
    (hgN : forall  c, gN c = 1  <->  (stepFun (funListOf P P (decodeList c))
      : Set (ApproximableMap V V)).Nonempty) (c a b : Nat) :
```

```
    (toApproxMap ((funSpace V V).principal (Xenum_mem P P gN hgN c))).rel (P.X a) (P.X
     b)
       <->  Xenum P P gN c  subseteq  step (P.X a) (P.X b) := by
  rw [toApproxMap_rel, mem_principal]
  exact <fun h => h.2, fun h => <step_mem (P.mem_X a) (P.mem_X b), h>>

/-- `chainDec` is equivalent to `gStepsOK` of the least map, via the inclusion char's
    spec and
`Xenum (codePair a b) = [X_a, X_b]`. -/
theorem chainDec_iff_gStepsOK (gN : Nat  ->  Nat)
    (hgN : forall  c, gN c = 1  <->  (stepFun (funListOf P P (decodeList c))
      : Set (ApproximableMap V V)).Nonempty) (fincl : Nat  ->  Nat)
    (hfincl : forall  s, fincl s = 1  <->  Xenum P P gN s.unpair.1  subseteq  Xenum P
    P gN s.unpair.2) (c : Nat) :
    forall  (full : List Nat) (a : Nat),
      chainDec fincl c a full  <->
        gStepsOK (toApproxMap ((funSpace V V).principal (Xenum_mem P P gN hgN c))) P a
     full := by
  intro full
  induction full with
  | nil => intro a; exact Iff.rfl
  | cons b rest ih =>
    intro a
    constructor
    * rintro <hstep, hrest>
      refine <?_, (ih b).mp hrest>
      rw [hfincl, unpair_pair_fst, unpair_pair_snd, Xenum_codePair P gN hgN a b] at
    hstep
      exact (leastMap_Xenum_rel P gN hgN c a b).mpr hstep
    * rintro <hstep, hrest>
      refine <?_, (ih b).mpr hrest>
      have hsub : Xenum P P gN c  subseteq  step (P.X a) (P.X b) :=
        (leastMap_Xenum_rel P gN hgN c a b).mp hstep
      rw [hfincl, unpair_pair_fst, unpair_pair_snd, Xenum_codePair P gN hgN a b]
      exact hsub

theorem primrec_fixStp (fincl : Nat  ->  Nat) (hfinclp : Nat.Primrec fincl) :
    Nat.Primrec (fixStp fincl) := by
  have hx : Nat.Primrec (fun w : Nat => w.unpair.1) := Nat.Primrec.left
  have hc : Nat.Primrec (fun w : Nat => w.unpair.2.unpair.2) :=
    Nat.Primrec.right.comp Nat.Primrec.right
  have hprev : Nat.Primrec (fun w : Nat => w.unpair.2.unpair.1.unpair.1) :=
    Nat.Primrec.left.comp (Nat.Primrec.left.comp Nat.Primrec.right)
  have hflag : Nat.Primrec (fun w : Nat => w.unpair.2.unpair.1.unpair.2) :=
    Nat.Primrec.right.comp (Nat.Primrec.left.comp Nat.Primrec.right)
  have hcode : Nat.Primrec (fun w : Nat => codePair w.unpair.2.unpair.1.unpair.1 w.
    unpair.1) :=
    Nat.Primrec.succ.comp ((hprev.pair hx).pair (Nat.Primrec.const 0))
  have hinner : Nat.Primrec (fun w => Nat.pair w.unpair.2.unpair.2
      (codePair w.unpair.2.unpair.1.unpair.1 w.unpair.1)) := hc.pair hcode
  have hsel : Nat.Primrec (fun w => selectFn w.unpair.2.unpair.1.unpair.2
      (isOne (fincl (Nat.pair w.unpair.2.unpair.2
        (codePair w.unpair.2.unpair.1.unpair.1 w.unpair.1)))) 0) :=
    primrec_selectFn hflag (primrec_isOne.comp (hfinclp.comp hinner)) (Nat.Primrec.
    const 0)
  exact (hx.pair hsel).of_eq (fun _ => rfl)
```

```
theorem primrec_fixChainChar (fincl : Nat  ->  Nat) (hfinclp : Nat.Primrec fincl) :
    Nat.Primrec (fun t => fixChainChar P fincl t.unpair.1 t.unpair.2) :=
  (primrec_foldCode (primrec_fixStp fincl hfinclp) Nat.Primrec.left
    (Nat.Primrec.const (Nat.pair P.masterIdx 1)) Nat.Primrec.right).of_eq (fun _ =>
    rfl)

end
end DomainTheory.NeighbourhoodSystems
end
```

3.9 Retracts of the universal domain

Dependency diagram. Each box is one item. The arrowhead is on the item that imports. An arrow already implied by a longer path is left out. The introduction has no Lean file, so it is not shown. Blue is a definition, green a proposition, amber an example, red a theorem, and indigo an exercise.

3.9.1 Introduction

LECTURE VIII RETRACTS OF THE UNIVERSAL DOMAIN

In order to be able to have a fully flexible method of solving domain equations and to be able to see why the domains obtained are effectively given, we shall embed all the desired domains in one "largest" domain. This universal domain will be easily shown to be effectively given, and the mappings needed to extract the other domains will be found to be computable. In order to be able to carry out this programme, we investigate first how certain subdomains correspond to mappings — the so-called *retracts*. An advantage of this analysis is that all the necessary definitions can be written out in λ-calculus notation, thus demonstrating the power of our mathematical programming language.

3.9.2 Definition 8.1 — retraction

DEFINITION 8.1. A retraction of a given domain E is an approximable mapping $a : E \to E$ such that $a \circ a = a$. $\square$

Definition_8_1.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.Lemma_6_15

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem ApproximableMap
variable {a : Type*} {E : NeighborhoodSystem a}

/- concordance: source L4769 DEFINITION 8.1 retraction -/
/-- **Definition 8.1 (Scott 1981, PRG-19).** A *retraction* of `E` is an approximable
    mapping
`a : E  ->  E` with `a  comp  a = a`. -/
```

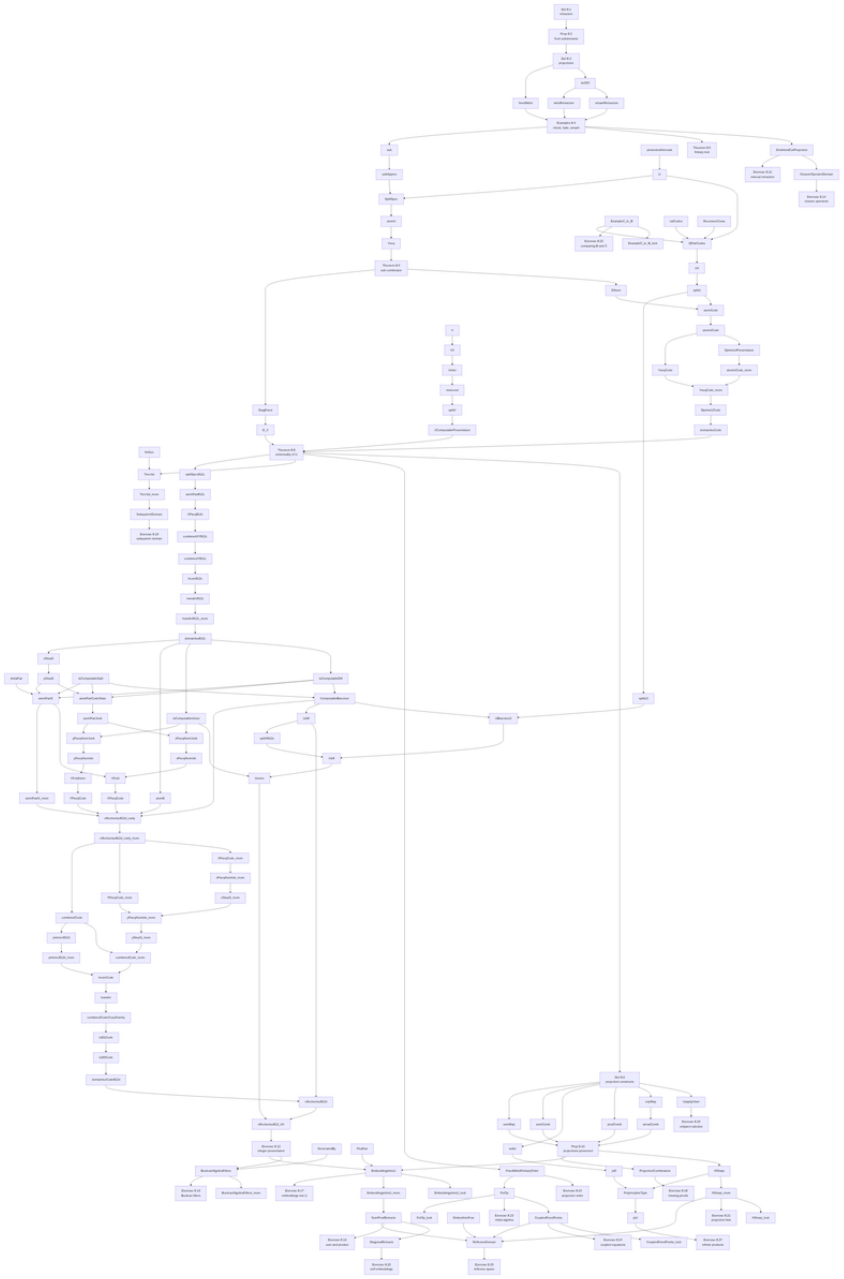


Figure 37: Retracts of the universal domain.

```
class Retraction (a : ApproximableMap E E) : Prop where
  out : a.comp a = a

/-- The identity mapping `I_E` is (trivially) a retraction. -/
@[simp] theorem isRetraction_idMap : Retraction (idMap E) :=
  <idMap_comp (idMap E)>

end

section
open NeighborhoodSystem ApproximableMap
variable {a : Type*} {D E : NeighborhoodSystem a}

namespace Subsystem

/-- **Proposition 8.2's retraction `a = i  comp  j`.** As a neighbourhood relation (
    unfolding `inj`/
`proj`), `X a Z  <->  exists  Y  in  D, X  subseteq  Y  subseteq  Z` -- exactly Scott's
     formula. -/
def retractionOfSubsystem (h : D ? E) : ApproximableMap E E := h.inj.comp h.proj

@[simp] theorem retractionOfSubsystem_rel (h : D ? E) {X Z : Set a} :
    (retractionOfSubsystem h).rel X Z  <->  E.mem X  /\  E.mem Z  /\  exists  Y, D.mem
     Y  /\  X  subseteq  Y  /\  Y  subseteq  Z := by
  simp only [retractionOfSubsystem, comp_rel, inj_rel, proj_rel]
  constructor
  * rintro <Y, <hEX, hDY, hXY>, hDY', hEZ, hYZ>
    exact <hEX, hEZ, Y, hDY, hXY, hYZ>
  * rintro <hEX, hEZ, Y, hDY, hXY, hYZ>
    exact <Y, <hEX, hDY, hXY>, hDY, hEZ, hYZ>

/-- `retractionOfSubsystem h` fixes exactly `i(x)` for `x  in  |D|`: `a(i(x)) = i(x)`.
    -/
theorem retractionOfSubsystem_toElementMap_inj (h : D ? E) (x : D.Element) :
    (retractionOfSubsystem h).toElementMap (h.inj.toElementMap x) = h.inj.toElementMap
     x := by
  have h1 : (retractionOfSubsystem h).toElementMap (h.inj.toElementMap x)
      = h.inj.toElementMap (h.proj.toElementMap (h.inj.toElementMap x)) :=
    toElementMap_comp h.inj h.proj (h.inj.toElementMap x)
  have h2 : h.proj.toElementMap (h.inj.toElementMap x) = (h.proj.comp h.inj).
    toElementMap x :=
    (toElementMap_comp h.proj h.inj x).symm
  rw [h1, h2, h.proj_comp_inj, toElementMap_idMap]

/-- **Proposition 8.2, second half.** `|D|` is order-isomorphic to the fixed-point set
    of
`a = retractionOfSubsystem h`, via `x  |->  i(x)` with inverse `y  |->  j(y)`. -/
def elementIso (h : D ? E) :
    D.Element ?o {y : E.Element // (retractionOfSubsystem h).toElementMap y = y} where
  toFun x := <h.inj.toElementMap x, retractionOfSubsystem_toElementMap_inj h x>
  invFun y := h.proj.toElementMap y.1
  left_inv x := by
    have h1 : h.proj.toElementMap (h.inj.toElementMap x) = (h.proj.comp h.inj).
    toElementMap x :=
      (toElementMap_comp h.proj h.inj x).symm
    show h.proj.toElementMap (h.inj.toElementMap x) = x
```

```
    rw [h1, h.proj_comp_inj, toElementMap_idMap]
  right_inv y := by
    apply Subtype.ext
    have hfix : (h.inj.comp h.proj).toElementMap y.1 = y.1 := y.2
    have h1 : (h.inj.comp h.proj).toElementMap y.1 = h.inj.toElementMap (h.proj.
    toElementMap y.1) :=
      toElementMap_comp h.inj h.proj y.1
    show h.inj.toElementMap (h.proj.toElementMap y.1) = y.1
    rw [ <-  h1]; exact hfix
  map_rel_iff' := by
    intro x x'
    show h.inj.toElementMap x  <=  h.inj.toElementMap x'  <->  x  <=  x'
    constructor
    * intro hle
      have hmono := h.proj.toElementMap_mono hle
      have h1 : h.proj.toElementMap (h.inj.toElementMap x) = (h.proj.comp h.inj).
    toElementMap x :=
        (toElementMap_comp h.proj h.inj x).symm
      have h2 : h.proj.toElementMap (h.inj.toElementMap x') = (h.proj.comp h.inj).
    toElementMap x' :=
        (toElementMap_comp h.proj h.inj x').symm
      rw [h1, h2, h.proj_comp_inj, toElementMap_idMap, toElementMap_idMap] at hmono
      exact hmono
    * intro hle
      exact h.inj.toElementMap_mono hle

/-- **Proposition 8.2, first half.** `retractionOfSubsystem h` is a retraction: `a
    comp  a = a`, via
`a  comp  a = i comp (j comp i) comp j = i comp I_D comp j = i comp j = a`. -/
theorem isRetraction_retractionOfSubsystem (h : D ? E) :
    Retraction (retractionOfSubsystem h) := by
  constructor
  show (h.inj.comp h.proj).comp (h.inj.comp h.proj) = h.inj.comp h.proj
  rw [comp_assoc,  <-  comp_assoc h.proj h.inj h.proj, h.proj_comp_inj, idMap_comp]

end Subsystem
end
end DomainTheory.NeighbourhoodSystems
```

3.9.3 Proposition 8.2 — retractions from subdomains

PROPOSITION 8.2. If $D \triangleleft E$ and if $a : E \to E$ is defined by

$$X\ a\ Z \quad \text{iff} \quad \exists Y \in D.\ X \subseteq Y \subseteq Z$$

for all $X, Z \in E$, then a is a retraction and $|D|$ is isomorphic to the fixed-point set of a, the set $\{y \in |E| \mid a(y) = y\}$, under inclusion.

Proof: That a is an approximable mapping is a direct consequence of Definition 6.10. Indeed, in the notation of Proposition 6.12, we have

$$a = i \circ j,$$

and this is another proof that a is approximable. This remark is also convenient, since we know from 6.10

$$j \circ i = I_D.$$

Therefore, we find:

$$a \circ a = i \circ j \circ i \circ j = i \circ j = a;$$

and so a is a retraction.

We can also employ i and j to give the isomorphism on $|D|$. If $x \in |D|$, then $i(x) \in |E|$ and we calculate:

$$a(i(x)) = i \circ j \circ i(x) = i(x).$$

Thus, $i(x)$ belongs to the fixed-point set of a. In the other direction, if $a(y) = y$, then $i(j(y)) = y$. But $j(y) \in |D|$, so i maps $|D|$ one-one and onto the fixed-point set of a. As i and j are monotone, the map is an isomorphism with respect to $\sqsubseteq$. $\square$

Not every retraction comes from a relationship like $D \triangleleft E$; in fact, we can see from the definition of a above that $a \sqsubseteq 1_E$. But, as is indicated in Exercise 8.11, even this condition is not sufficient to characterize the kind of retractions provided by 8.2. The characterization is as follows.

Proposition_8_2.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Definition_8_1

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem ApproximableMap
variable {a : Type*} {D E : NeighborhoodSystem a}
namespace Subsystem
/- concordance: source L4771 PROPOSITION 8.2 retractions from subdomains -/
/-- **Proposition 8.2 (Scott 1981, PRG-19), packaged.** `D ? E` induces a retraction `a
    ` on `E`
whose fixed-point set is (order-isomorphic to) `|D|`. -/
theorem retraction_of_subsystem (h : D ? E) :
    Retraction (retractionOfSubsystem h)  /\
      Nonempty (D.Element ?o {y : E.Element // (retractionOfSubsystem h).toElementMap
    y = y}) :=
  <isRetraction_retractionOfSubsystem h, <elementIso h>>

end Subsystem
end

end DomainTheory.NeighbourhoodSystems
namespace DomainTheory.NeighbourhoodSystems.Subsystem
set_option linter.unusedVariables false

theorem theorem_82.{u_1} : And
```

```
(forall  {a : Type u_1} {D E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
  a}
   (h : @DomainTheory.NeighbourhoodSystems.Subsystem a D E),
  @DomainTheory.NeighbourhoodSystems.Retraction a E
    (@DomainTheory.NeighbourhoodSystems.Subsystem.retractionOfSubsystem a D E h))
(forall  {a : Type u_1} {D E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
  a}
   (h : @DomainTheory.NeighbourhoodSystems.Subsystem a D E),
  And
    (@DomainTheory.NeighbourhoodSystems.Retraction a E
      (@DomainTheory.NeighbourhoodSystems.Subsystem.retractionOfSubsystem a D E h))
    (Nonempty
      (@OrderIso (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a D)
        (@Subtype (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E
  )
          fun (y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E
  ) =>
          @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E)
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a E E
              (@DomainTheory.NeighbourhoodSystems.Subsystem.retractionOfSubsystem a
  D E h) y)
            y)
        (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  Element a D)
          (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a D)
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement a D)))
        (@Subtype.instLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  Element a E)
          (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  Element a E)
            (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
  NeighborhoodSystem.Element a E)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  instPartialOrderElement a E)))
          fun (y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E
  ) =>
          @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E)
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a E E
              (@DomainTheory.NeighbourhoodSystems.Subsystem.retractionOfSubsystem a
  D E h) y)
            y)))) :=
@And.intro
(forall  {a : Type u_1} {D E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
  a}
   (h : @DomainTheory.NeighbourhoodSystems.Subsystem a D E),
  @DomainTheory.NeighbourhoodSystems.Retraction a E
    (@DomainTheory.NeighbourhoodSystems.Subsystem.retractionOfSubsystem a D E h))
(forall  {a : Type u_1} {D E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
  a}
   (h : @DomainTheory.NeighbourhoodSystems.Subsystem a D E),
  And
    (@DomainTheory.NeighbourhoodSystems.Retraction a E
      (@DomainTheory.NeighbourhoodSystems.Subsystem.retractionOfSubsystem a D E h))
    (Nonempty
```

```
        (@OrderIso (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a D)
          (@Subtype (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E
    )
            fun (y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E
    ) =>
            @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E)
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a E E
                (@DomainTheory.NeighbourhoodSystems.Subsystem.retractionOfSubsystem a
    D E h) y)
              y)
          (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    Element a D)
            (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.Element a D)
              (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    instPartialOrderElement a D)))
          (@Subtype.instLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    Element a E)
            (@Preorder.toLE (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    Element a E)
              (@PartialOrder.toPreorder (@DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem.Element a E)
                (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    instPartialOrderElement a E)))
            fun (y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E
    ) =>
            @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E)
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a E E
                (@DomainTheory.NeighbourhoodSystems.Subsystem.retractionOfSubsystem a
    D E h) y)
              y))))
  @DomainTheory.NeighbourhoodSystems.Subsystem.isRetraction_retractionOfSubsystem
  @DomainTheory.NeighbourhoodSystems.Subsystem.retraction_of_subsystem

end DomainTheory.NeighbourhoodSystems.Subsystem
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem ApproximableMap
universe u
variable {a : Type u} {D E : NeighborhoodSystem a}

/-- **Definition 8.3, second clause (Scott 1981, PRG-19).** A retraction `a : E  ->  E`
     is *finitary*
iff its fixed-point set `{y  in  |E| ? a(y) = y}` -- ordered by the inherited inclusion
     ` sqsub ` -- is
isomorphic to (the element poset of) some neighbourhood system. -/
def IsFinitary (a : ApproximableMap E E) : Prop :=
  exists  (b : Type u) (F : NeighborhoodSystem b),
    Nonempty ({y : E.Element // a.toElementMap y = y} ?o F.Element)

end
end DomainTheory.NeighbourhoodSystems
```

3.9.4 Definition 8.3 — projections

DEFINITION 8.3. A retraction $a : E \to E$ is called a *projection* provided

$$a \sqsubseteq 1_E;$$

it is *finitary* iff its fixed-point set is isomorphic to a domain. □

Definition_8_3.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Proposition_8_2

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem ApproximableMap
universe u
variable {a : Type u} {D E : NeighborhoodSystem a}

/- concordance: source L4801 DEFINITION 8.3 projections -/
/-- **Definition 8.3, first clause (Scott 1981, PRG-19).** A retraction `a : E  ->  E`
    is a
*projection* provided `a  sqsub  I_E`. (We do not separately require `Retraction a` in
    the
definition, matching Scott's usage where "projection" is applied to a retraction
    already in
hand; the conjunction is spelled out where needed, e.g. `FinitaryProjection`.) -/
def IsProjection (a : ApproximableMap E E) : Prop := Retraction a  /\  a  <=  idMap E

/-- A **finitary projection**: both clauses of Definition 8.3 together with `Retraction
    `. -/
class FinitaryProjection (a : ApproximableMap E E) : Prop where
  out : IsProjection a  /\  IsFinitary a

end

section
open NeighborhoodSystem ApproximableMap
universe u
variable {a : Type u} {D E : NeighborhoodSystem a}

namespace Subsystem

variable {h : D ? E}

/-- **Every `retractionOfSubsystem h` is finitary** (Definition 8.3): its fixed-point
    set is
isomorphic to `|D|` itself, by Proposition 8.2's `elementIso`. -/
theorem isFinitary_retractionOfSubsystem (h : D ? E) :
    IsFinitary (retractionOfSubsystem h) :=
  <a, D, <(elementIso h).symm>>
```

```
/-- **Scott's remark before Definition 8.3.** Every retraction `a =
    retractionOfSubsystem h` of
Proposition 8.2 satisfies `a  sqsub  I_E`: unwinding `X a Z  <->  exists  Y  in  D, X
    subseteq  Y  subseteq  Z` gives `X  subseteq  Z`
directly. -/
theorem retractionOfSubsystem_le_idMap (h : D ? E) : retractionOfSubsystem h  <=
    idMap E := by
  intro X Z hr
  obtain <hEX, hEZ, Y, _, hXY, hYZ> := (retractionOfSubsystem_rel h).mp hr
  exact <hEX, hEZ, hXY.trans hYZ>

/-- **Every `retractionOfSubsystem h` is a projection** (Definition 8.3). -/
theorem isProjection_retractionOfSubsystem (h : D ? E) :
    IsProjection (retractionOfSubsystem h) :=
  <isRetraction_retractionOfSubsystem h, retractionOfSubsystem_le_idMap h>

/-- **Every `retractionOfSubsystem h` is a finitary projection** (Definition 8.3, both
    clauses). -/
theorem isFinitaryProjection_retractionOfSubsystem (h : D ? E) :
    FinitaryProjection (retractionOfSubsystem h) :=
  <<isProjection_retractionOfSubsystem h, isFinitary_retractionOfSubsystem h>>

end Subsystem
end

open NeighborhoodSystem ApproximableMap ApproximableMap_2

theorem fin2_zero_ne_zero_one : ({0} : Set (Fin 2))  !=  ({0, 1} : Set (Fin 2)) := by
  intro h
  have h1 : (1 : Fin 2)  in  ({0, 1} : Set (Fin 2)) := by simp
  rw [ <-  h] at h1
  simp at h1

theorem fin2_zero_subset_zero_one : ({0} : Set (Fin 2))  subseteq  ({0, 1} : Set (Fin
    2)) := by
  intro x hx; simp only [Set.mem_singleton_iff] at hx; subst hx; simp

end DomainTheory.NeighbourhoodSystems
```

3.9.5 Examples 8.4 — check, fade, and smash

EXAMPLES 8.4. If a system D over Δ is not trivial, then the two element system $O = \{\{0\}, \{0,1\}\}$ comes from a retraction on D. Specifically, define a combinator

$$\text{check} : D \to O$$

by the relation

$$X \text{ check } Y \quad \text{iff either } Y = \{0,1\} \text{ or } X \neq \Delta.$$

We see $\text{check}(x) = \bot_O$ iff $x = \bot_D$. We leave to the reader the definition of a combinator:

$$\text{fade} : O \times D \to D,$$

where we have for $t \in |O|$ and $x \in |D|$:

$$\begin{aligned} \mathrm{fade}(t, x) &= \bot_D, \text{ if } t = \bot_O; \\ &= x, \text{ if not.} \end{aligned}$$

Now, take any $u \in |D|$ with $u \neq \bot$, and define

$$a(x) = \mathrm{fade}(\mathrm{check}(x), u).$$

Then a is a retraction (not a projection in general) and the range of a is isomorphic to O.

Another way of using these combinators is to find $(D \to_\bot E)$ as a retraction of $(D \to E)$. Specifically, define a combinator

$$\mathrm{strict} : (D \to E) \to (D \to E)$$

by the equation

$$\mathrm{strict}(f) = \lambda x.\ \mathrm{fade}(\mathrm{check}(x), f(x)),$$

where this time

$$\mathrm{fade} : O \times E \to E\ .$$

The range of strict consists exactly of the strict functions and this time strict is a projection whose range is indeed a domain.

Similarly, we can find a projection on $D \times E$ with a range isomorphic to $D \otimes E$ by the combinator such that:

$$\mathrm{smash}(x, y) = \mathrm{fade}(\mathrm{check}(x), \mathrm{fade}(\mathrm{check}(y), \langle x, y \rangle)),$$

for $x \in |D|$ and $y \in |E|$. □

Examples_8_4.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.fixOfO
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    smashRetraction
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    strictRetraction
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.fixedNbhd

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

/- concordance: source L4807 EXAMPLES 8.4 check, fade, and smash -/

end DomainTheory.NeighbourhoodSystems
```

3.9.6 Theorem 8.5 — finitary projection test

THEOREM 8.5. For an approximable mapping $a : E \to E$ the following are equivalent:

(i) a is a finitary projection;

(ii) $a(x) = \{Y \in E \mid \exists X \in x.\, X \subseteq Y \wedge X\ a\ X\}$, for all $x \in |E|$.

Proof: Suppose a satisfies (ii) first. Inasmuch as $X \in x$ and $X \subseteq Y$ always imply $Y \in x$, for all $x \in |E|$, we see $a(x) \subseteq x$ must always hold. Moreover, it is obvious that $X \in x$ and $X\ a\ X$ always imply $X \in a(x)$; therefore, $a(x) \subseteq a(a(x))$ for all $x \in |E|$. This shows that a
is indeed a projection.

Let $D = \{X \in E \mid X\ a\ X\}$, then it is easy to check that $D \triangleleft E$ and that a is determined from D exactly as in 8.2; thus, the fixed-point set of a is isomorphic to a domain, by what we have already proved. So we have shown (ii) implies (i).

In the converse direction, assume that a is a finitary projection. And let the system D be isomorphic to the fixed-point set of a. We have the situation of Theorem 6.15. There is a projection pair,

$$i : D \to E \quad \text{and} \quad j : E \to D,$$

where the connection with a gives:

$$j \circ i = I_D \quad \text{and} \quad i \circ j = a \subseteq I_E.$$

By 6.15, $D \cong D' \triangleleft E$, and we want to identify D' in terms of a as follows:

$$D' = \{X \in E \mid X\ a\ X\}.$$

Now from a reading of the proof of 6.15 the neighbourhoods of D' are just those corresponding to the finite elements of D. But any such element is a fixed point of a. We have

$$X \in D' \quad \text{implies} \quad a(\uparrow X) = \uparrow X \quad \text{implies} \quad X\ a\ X.$$

Conversely, if $X\ a\ X$ holds, then $\uparrow X \subseteq a(\uparrow X)$. But a is a projection, so $\uparrow X$ is a fixed point. But $i(j(\uparrow X)) = \uparrow X$ means $j(\uparrow X)$ is a finite element of $|D|$. So $X \in D'$, and we have D' identified as desired.

Finally, if we calculate $a = i \circ j$ by the formulae of 6.12 (with D' for D, of course), we obtain our formula (ii). □

The criterion for being a finitary projection just obtained provides us with a very interesting new combinator.

Theorem_8_5.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Definition_8_3

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Examples_8_4

@[expose] public section
```

```
namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem ApproximableMap
universe u
variable {a : Type u} {E : NeighborhoodSystem a}
/- concordance: source L4854 THEOREM 8.5 finitary projection test -/
/-- **Theorem 8.5 (Scott 1981, PRG-19), in full.** `a : E  ->  E` is a finitary
    projection iff it
satisfies the step-closure formula `a(x) = {Y  in  E ? exists X in x, X subseteq Y  /\
     XaX}` for every `x  in  |E|`. -/
theorem finitaryProjection_iff_formula (a : ApproximableMap E E) :
    FinitaryProjection a  <->  forall  (x : E.Element) {Y : Set a}, (a.toElementMap x)
    .mem Y  <->
      E.mem Y  /\  exists  X, x.mem X  /\  X  subseteq  Y  /\  a.rel X X :=
  <fun h => formula_of_isFinitaryProjection h, fun hii =>
    isFinitaryProjection_of_formula a hii>
end

set_option linter.unusedVariables false

theorem theorem_85.{u} : And
  (And
    (forall  {a : Type u} {E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
      {a : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a E E},
      @DomainTheory.NeighbourhoodSystems.FinitaryProjection a E a  ->
        forall  (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E
   ) {Y : Set a},
          Iff
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem a E
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a E E
    a x) Y)
            (And (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a E Y)
              (exists  (X : Set a),
                And (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem
    a E x X)
                  (And (@LE.le (Set a) (@Set.instLE a) X Y)
                    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel a a E E a
   X X)))))
    (forall  {a : Type u} {E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
      (a : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a E E),
      Iff (@DomainTheory.NeighbourhoodSystems.FinitaryProjection a E a)
        (forall  (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
   E) {Y : Set a},
          Iff
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem a E
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a E E
    a x) Y)
            (And (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a E Y)
              (exists  (X : Set a),
                And (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem
    a E x X)
                  (And (@LE.le (Set a) (@Set.instLE a) X Y)
                    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel a a E E a
   X X)))))))
  (forall  {a : Type u} {E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
    (a : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a E E),
```

```
  (forall  (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E) {
  Y : Set a},
      Iff
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem a E
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a E E a
   x) Y)
        (And (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a E Y)
          (exists  (X : Set a),
            And (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem a
   E x X)
              (And (@LE.le (Set a) (@Set.instLE a) X Y)
                (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel a a E E a X
  X)))))  ->
    @DomainTheory.NeighbourhoodSystems.FinitaryProjection a E a) :=
@And.intro
(And
  (forall  {a : Type u} {E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
    {a : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a E E},
    @DomainTheory.NeighbourhoodSystems.FinitaryProjection a E a  ->
      forall  (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E
  ) {Y : Set a},
        Iff
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem a E
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a E E
   a x) Y)
          (And (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a E Y)
            (exists  (X : Set a),
              And (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem
   a E x X)
                (And (@LE.le (Set a) (@Set.instLE a) X Y)
                  (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel a a E E a
  X X)))))
  (forall  {a : Type u} {E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
    (a : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a E E),
    Iff (@DomainTheory.NeighbourhoodSystems.FinitaryProjection a E a)
      (forall  (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
  E) {Y : Set a},
        Iff
          (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem a E
            (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a E E
   a x) Y)
          (And (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a E Y)
            (exists  (X : Set a),
              And (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem
   a E x X)
                (And (@LE.le (Set a) (@Set.instLE a) X Y)
                  (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel a a E E a
  X X)))))))
(forall  {a : Type u} {E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
  (a : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a E E),
  (forall  (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E) {
  Y : Set a},
      Iff
        (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem a E
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a E E a
   x) Y)
```

```
          (And (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a E Y)
            (exists  (X : Set a),
              And (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem a
     E x X)
                (And (@LE.le (Set a) (@Set.instLE a) X Y)
                  (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel a a E E a X
    X)))))  ->
      @DomainTheory.NeighbourhoodSystems.FinitaryProjection a E a)
  (@And.intro
    (forall  {a : Type u} {E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
      {a : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a E E},
      @DomainTheory.NeighbourhoodSystems.FinitaryProjection a E a  ->
        forall  (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E
    ) {Y : Set a},
          Iff
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem a E
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a E E
     a x) Y)
            (And (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a E Y)
              (exists  (X : Set a),
                And (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem
     a E x X)
                  (And (@LE.le (Set a) (@Set.instLE a) X Y)
                    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel a a E E a
    X X)))))
    (forall  {a : Type u} {E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
      (a : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a E E),
      Iff (@DomainTheory.NeighbourhoodSystems.FinitaryProjection a E a)
        (forall  (x : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
    E) {Y : Set a},
          Iff
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem a E
              (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a E E
     a x) Y)
            (And (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem a E Y)
              (exists  (X : Set a),
                And (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element.mem
     a E x X)
                  (And (@LE.le (Set a) (@Set.instLE a) X Y)
                    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.rel a a E E a
    X X))))))
    @DomainTheory.NeighbourhoodSystems.formula_of_isFinitaryProjection
    @DomainTheory.NeighbourhoodSystems.finitaryProjection_iff_formula)
  @DomainTheory.NeighbourhoodSystems.isFinitaryProjection_of_formula

end DomainTheory.NeighbourhoodSystems
```

3.9.7 Theorem 8.6 — sub combinator

THEOREM 8.6. For any domain E define

$$\mathrm{sub} : (E \to E) \to (E \to E)$$

by the formula

$$X \,\mathrm{sub}\,(f)\, Z \quad \text{iff} \quad \exists Y \in E.\quad X \subseteq Y \;\; f\, Y \subseteq Z,$$

for all $X, Z \in E$ and all $f : E \to E$. Then the range of sub consists exactly of the finitary projections on E, and moreover sub itself is a finitary projection on $(E \to E)$. If E is effectively given, then sub is computable.

Proof: It is trivial to check that $\mathrm{sub}(f)$ is always approximable. Also, it is obvious from the definition that the correspondence

$$f \mapsto \mathrm{sub}(f)$$

preserves directed unions of f's. Thus, sub is itself approximable. We note that

$$X \subseteq Y \;\; f\, Y \subseteq Z \text{ always implies } X \; f \; Z;$$

hence, $\mathrm{sub}(f) \subseteq f$ holds. Also

$$Y \; f \; Y \text{ always implies } Y \; \mathrm{sub}(f) \; Y,$$

hence, $\mathrm{sub}(f) \subseteq \mathrm{sub}(\mathrm{sub}(f))$ holds. This shows sub to be a projection on $(E \to E)$. The effectiveness of the definition makes it also clear that sub is computable when E has a computable presentation.

Since, sub is a projection, its range is the same as its fixed-point set. If

$$\mathrm{sub}(a) = a,$$

then there is no problem in checking that a satisfies 8.5(ii) *and conversely*. So the range of sub picks out exactly the finitary projections in view of 8.5.

Finally, to prove that sub is a finitary projection of $(E \to E)$, we invoke 6.11 and remark that, in view of 8.2, the fixed point set (range) of sub is in a one-one inclusion-preserving correspondence with the domain $\{D \mid D \lhd E\}$. □

These results have almost completely translated the theory of $\lhd$-subdomains into λ-calculus via the sub-combinator. One last step will complete the passage, and then we shall be able to return to solving domain equations.

Theorem_8_6.lean

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Exercise_7_24
public import DomainTheory.NeighbourhoodSystems.DomainEquations.Proposition_6_11
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Exercise_2_13

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Examples_8_4
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.sub
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.subApprox
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.Yseq
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.U
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_13
```

```
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Theorem_7_5
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open Domain.Recursive NeighborhoodSystem
open Domain.Recursive NeighborhoodSystem
open Domain.Recursive NeighborhoodSystem
open Domain.Recursive NeighborhoodSystem
open Domain.Recursive NeighborhoodSystem
open Domain.Recursive NeighborhoodSystem
open Domain.Recursive NeighborhoodSystem
open Domain.Recursive NeighborhoodSystem
open Domain.Recursive NeighborhoodSystem
open Domain.Recursive NeighborhoodSystem

set_option linter.unusedVariables false

theorem theorem_86.{u} : And
  (And
    (forall  {a : Type u} {E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
      {f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a E E},
      Iff
        (@Eq (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a E E) (
   @DomainTheory.NeighbourhoodSystems.sub a E f)
          f)
        (@DomainTheory.NeighbourhoodSystems.FinitaryProjection a E f))
    (forall  {a : Type u} {E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a
    },
      @DomainTheory.NeighbourhoodSystems.IsFinitary (@DomainTheory.
   NeighbourhoodSystems.ApproximableMap a a E E)
        (@DomainTheory.NeighbourhoodSystems.funSpace a a E E)
        (@DomainTheory.NeighbourhoodSystems.subApprox a E)))
  (forall  {a : Type u} {E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a},
    @DomainTheory.NeighbourhoodSystems.FinitaryProjection (@DomainTheory.
   NeighbourhoodSystems.ApproximableMap a a E E)
      (@DomainTheory.NeighbourhoodSystems.funSpace a a E E) (@DomainTheory.
   NeighbourhoodSystems.subApprox a E)) :=
  @And.intro
  (And
    (forall  {a : Type u} {E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
      {f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a E E},
      Iff
        (@Eq (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a E E) (
   @DomainTheory.NeighbourhoodSystems.sub a E f)
          f)
        (@DomainTheory.NeighbourhoodSystems.FinitaryProjection a E f))
    (forall  {a : Type u} {E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a
    },
      @DomainTheory.NeighbourhoodSystems.IsFinitary (@DomainTheory.
   NeighbourhoodSystems.ApproximableMap a a E E)
        (@DomainTheory.NeighbourhoodSystems.funSpace a a E E)
        (@DomainTheory.NeighbourhoodSystems.subApprox a E)))
```

```
  (forall  {a : Type u} {E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a},
    @DomainTheory.NeighbourhoodSystems.FinitaryProjection (@DomainTheory.
    NeighbourhoodSystems.ApproximableMap a a E E)
      (@DomainTheory.NeighbourhoodSystems.funSpace a a E E) (@DomainTheory.
    NeighbourhoodSystems.subApprox a E))
  (@And.intro
    (forall  {a : Type u} {E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
      {f : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a E E},
      Iff
        (@Eq (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a E E) (
    @DomainTheory.NeighbourhoodSystems.sub a E f)
          f)
        (@DomainTheory.NeighbourhoodSystems.FinitaryProjection a E f))
    (forall  {a : Type u} {E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a
    },
      @DomainTheory.NeighbourhoodSystems.IsFinitary (@DomainTheory.
    NeighbourhoodSystems.ApproximableMap a a E E)
        (@DomainTheory.NeighbourhoodSystems.funSpace a a E E) (@DomainTheory.
    NeighbourhoodSystems.subApprox a E))
    @DomainTheory.NeighbourhoodSystems.sub_eq_self_iff_isFinitaryProjection
    @DomainTheory.NeighbourhoodSystems.subApprox.isFinitary)
  @DomainTheory.NeighbourhoodSystems.Sub8_6.isFinitaryProjection_subApprox

set_option linter.unusedVariables false

theorem theorem_86_c.{u_1} : And
  (forall  {a : Type u_1} {E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
    (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a E) (gN : Nat  ->
    Nat)
    (hgN :
      forall  (c : Nat),
        Iff (@Eq Nat (gN c) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))))
          (@Set.Nonempty (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a E E)
            (@DomainTheory.NeighbourhoodSystems.stepFun a a E E
              (@DomainTheory.NeighbourhoodSystems.funListOf a a E E P P (Domain.
    Recursive.decodeList c)))))
    (incl eq : Nat  ->  Nat) (hgNp : Nat.Primrec gN)
    (hincl :
      forall  (s : Nat),
        Iff (@Eq Nat (incl s) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))
    ))
          (@LE.le (Set a) (@Set.instLE a)
            (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a E P (@Prod.
    fst Nat Nat (Nat.unpair s)))
            (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a E P (@Prod.
    snd Nat Nat (Nat.unpair s)))))
    (hinclp : Nat.Primrec incl)
    (heq :
      forall  (s : Nat),
        Iff (@Eq Nat (eq s) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))))
          (@Eq (Set a)
            (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a E P (@Prod.
    fst Nat Nat (Nat.unpair s)))
            (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a E P (@Prod.
    snd Nat Nat (Nat.unpair s)))))
    (heqp : Nat.Primrec eq),
```

```
  @DomainTheory.NeighbourhoodSystems.ComputableMap (@DomainTheory.
  NeighbourhoodSystems.ApproximableMap a a E E)
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a E E) (@DomainTheory.
  NeighbourhoodSystems.funSpace a a E E)
    (@DomainTheory.NeighbourhoodSystems.funSpace a a E E)
    (@DomainTheory.NeighbourhoodSystems.funPresentation a a E E P P gN incl incl eq
  hgN hgNp hincl hinclp hincl hinclp
      heq heqp)
    (@DomainTheory.NeighbourhoodSystems.funPresentation a a E E P P gN incl incl eq
  hgN hgNp hincl hinclp hincl hinclp
      heq heqp)
    (@DomainTheory.NeighbourhoodSystems.subApprox a E))
(forall  {a : Type u_1} {E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a
  },
  @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven a E  ->
    exists  (Pfun :
      @DomainTheory.NeighbourhoodSystems.ComputablePresentation
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a E E)
        (@DomainTheory.NeighbourhoodSystems.funSpace a a E E)),
      @DomainTheory.NeighbourhoodSystems.ComputableMap (@DomainTheory.
  NeighbourhoodSystems.ApproximableMap a a E E)
        (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a E E)
        (@DomainTheory.NeighbourhoodSystems.funSpace a a E E) (@DomainTheory.
  NeighbourhoodSystems.funSpace a a E E)
        Pfun Pfun (@DomainTheory.NeighbourhoodSystems.subApprox a E)) :=
@And.intro
(forall  {a : Type u_1} {E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
  (P : @DomainTheory.NeighbourhoodSystems.ComputablePresentation a E) (gN : Nat  ->
   Nat)
  (hgN :
    forall  (c : Nat),
      Iff (@Eq Nat (gN c) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))))
        (@Set.Nonempty (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a E E)
          (@DomainTheory.NeighbourhoodSystems.stepFun a a E E
            (@DomainTheory.NeighbourhoodSystems.funListOf a a E E P P (Domain.
  Recursive.decodeList c)))))
  (incl eq : Nat  ->  Nat) (hgNp : Nat.Primrec gN)
  (hincl :
    forall  (s : Nat),
      Iff (@Eq Nat (incl s) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))
  ))
        (@LE.le (Set a) (@Set.instLE a)
          (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a E P (@Prod.
  fst Nat Nat (Nat.unpair s)))
          (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a E P (@Prod.
  snd Nat Nat (Nat.unpair s)))))
  (hinclp : Nat.Primrec incl)
  (heq :
    forall  (s : Nat),
      Iff (@Eq Nat (eq s) (@OfNat.ofNat Nat (nat_lit 1) (instOfNatNat (nat_lit 1))))
        (@Eq (Set a)
          (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a E P (@Prod.
  fst Nat Nat (Nat.unpair s)))
          (@DomainTheory.NeighbourhoodSystems.ComputablePresentation.X a E P (@Prod.
  snd Nat Nat (Nat.unpair s)))))
  (heqp : Nat.Primrec eq),
```

```
    @DomainTheory.NeighbourhoodSystems.ComputableMap (@DomainTheory.
    NeighbourhoodSystems.ApproximableMap a a E E)
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a E E) (@DomainTheory.
    NeighbourhoodSystems.funSpace a a E E)
      (@DomainTheory.NeighbourhoodSystems.funSpace a a E E)
      (@DomainTheory.NeighbourhoodSystems.funPresentation a a E E P P gN incl incl eq
    hgN hgNp hincl hinclp hincl hinclp
        heq heqp)
      (@DomainTheory.NeighbourhoodSystems.funPresentation a a E E P P gN incl incl eq
    hgN hgNp hincl hinclp hincl hinclp
        heq heqp)
      (@DomainTheory.NeighbourhoodSystems.subApprox a E))
  (forall  {a : Type u_1} {E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a
    },
    @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.IsEffectivelyGiven a E  ->
      exists  (Pfun :
        @DomainTheory.NeighbourhoodSystems.ComputablePresentation
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a E E)
          (@DomainTheory.NeighbourhoodSystems.funSpace a a E E)),
        @DomainTheory.NeighbourhoodSystems.ComputableMap (@DomainTheory.
    NeighbourhoodSystems.ApproximableMap a a E E)
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a E E)
          (@DomainTheory.NeighbourhoodSystems.funSpace a a E E) (@DomainTheory.
    NeighbourhoodSystems.funSpace a a E E)
          Pfun Pfun (@DomainTheory.NeighbourhoodSystems.subApprox a E))
  @DomainTheory.NeighbourhoodSystems.subApprox_isComputable
  @DomainTheory.NeighbourhoodSystems.sub_isComputable_of_isEffectivelyGiven

variable {a : Type*}

section Reencode

variable (e : Nat  ->  Set a)

/-- **Scott's `(e n)`, reindexed over `Nat`**: the set of indices `m` whose
    neighbourhood `e m` sits
below `e n`. This is `D?`'s replacement for `e n` itself. -/
def idxSet (n : Nat) : Set Nat := {m | e m  subseteq  e n}

@[simp] theorem mem_idxSet {m n : Nat} : m  in  idxSet e n  <->  e m  subseteq  e n :=
     Iff.rfl

end Reencode

section Setup

variable (D : NeighborhoodSystem a) (e : Nat  ->  Set a)
  (hcover : forall  S, D.mem S  <->  exists  n, S = e n) (he0 : e 0 = D.master)

include hcover in
/-- Every enumerated set is a genuine `D`-neighbourhood. -/
theorem D_mem_e (n : Nat) : D.mem (e n) := (hcover (e n)).mpr <n, rfl>

/-- **Scott's `Y_n`, built from the separated reindexing.** Instantiated with the
    classical
`splitChoice` (this is `Theorem88.lean`'s general, non-effective apparatus). -/
```

```
noncomputable abbrev Yidx (n : Nat) : Set ? := Yseq splitChoice (idxSet e) Set.univ n

include hcover in
theorem e_subset_master (n : Nat) : e n  subseteq  D.master := D.sub_master (D_mem_e D
     e hcover n)

end Setup

section Reencode

variable (e : Nat  ->  Set a)

/-- If `e i  I  e j = e m` (i.e. `{e i, e j}` is witnessed-consistent, with witness
    realizing the
intersection exactly as `e m`), then `idxSet e i  I  idxSet e j = idxSet e m` -- a **
    definitional**
rewriting of the set-builder, needing no transfer at all. -/
theorem idxSet_inter_of_inter_eq {i j m : Nat} (h : e i  I  e j = e m) :
    idxSet e i  I  idxSet e j = idxSet e m := by
  ext k
  show e k  subseteq  e i  /\  e k  subseteq  e j  <->  e k  subseteq  e m
  rw [ <-  Set.subset_inter_iff, h]

theorem idxSet_zero_eq_univ {n0 : Nat} (hsub : forall  m, e m  subseteq  e n0) :
    idxSet e n0 = Set.univ :=
  Set.eq_univ_of_forall hsub

end Reencode

section Setup

variable (D : NeighborhoodSystem a) (e : Nat  ->  Set a)
  (hcover : forall  S, D.mem S  <->  exists  n, S = e n) (he0 : e 0 = D.master)

include hcover he0 in
theorem idxSet_zero : idxSet e 0 = Set.univ :=
  idxSet_zero_eq_univ e (fun m => he0  |>  e_subset_master D e hcover m)

end Setup

section Reencode

variable (e : Nat  ->  Set a)

/-- Every index belongs to its own `idxSet` (`e n  subseteq  e n`) -- so `idxSet e n`
    is always non-empty,
regardless of whether `e n` itself is empty. -/
theorem self_mem_idxSet (n : Nat) : n  in  idxSet e n := show e n  subseteq  e n from
    subset_rfl

/-- The raw inclusion order on `idxSet`s matches `e`'s own inclusion order -- this is
    the
"separation" property `D?` is built to have. -/
theorem idxSet_subset_iff (i j : Nat) : idxSet e i  subseteq  idxSet e j  <->  e i
    subseteq  e j :=
  <fun h => h (self_mem_idxSet e i), fun h _ hm => hm.trans h>
```

```
end Reencode

section Setup

variable (D : NeighborhoodSystem a) (e : Nat  ->  Set a)
  (hcover : forall  S, D.mem S  <->  exists  n, S = e n) (he0 : e 0 = D.master)

/-- `Delta' := Set.univ` is always non-empty, so the general `Yseq`/transfer apparatus
    of
`Theorem88.lean` applies to `(idxSet e, Set.univ)` unconditionally -- no `{}`-freeness
    hypothesis on
`D` is ever needed, since we reindexed onto `Nat` before invoking it (`idxSet_nonempty
    `). -/
theorem univ_nonempty_nat : (Set.univ : Set Nat).Nonempty := <0, trivial>

/-- `Yidx e n` is always `{}` or a genuine `U`-neighbourhood. -/
theorem Yidx_empty_or_mem (n : Nat) : Yidx e n = {}  \/  U.mem (Yidx e n) :=
  Yseq_empty_or_mem splitChoice (idxSet e) Set.univ univ_nonempty_nat
    splitChoice_isSplitSpec n

/-- **`Yidx e n` is always non-empty** (hence, by `Yidx_empty_or_mem`, always a genuine
`U`-neighbourhood): `n` itself witnesses `n  in  idxSet e n`. -/
theorem Yidx_nonempty (n : Nat) : (Yidx e n).Nonempty :=
  Yseq_nonempty_of_mem splitChoice (idxSet e) Set.univ univ_nonempty_nat
    splitChoice_isSplitSpec
     (Set.mem_univ n) (self_mem_idxSet e n)

theorem Yidx_mem (n : Nat) : U.mem (Yidx e n) :=
  (Yidx_empty_or_mem e n).resolve_left (Yidx_nonempty e n).ne_empty

/-- `Yidx e n` is always ` subseteq  U.master`. -/
theorem Yidx_subset_master (n : Nat) : Yidx e n  subseteq  U.master :=
  Yseq_subset_master splitChoice (idxSet e) Set.univ univ_nonempty_nat
    splitChoice_isSplitSpec n

include hcover he0 in
/-- `Yidx e 0 = U.master`, since `idxSet e 0 = Set.univ` (Scott's `X_0 = Delta`). -/
theorem Yidx_zero : Yidx e 0 = U.master :=
  Yseq_zero_eq_master splitChoice (idxSet e) Set.univ univ_nonempty_nat
    splitChoice_isSplitSpec
     (idxSet_zero D e hcover he0)

/-- **The order-embedding at the neighbourhood level.** This is where the whole `idxSet
    `
reindexing pays off: because `idxSet`'s inclusion order matches `e`'s (`
    idxSet_subset_iff`), the
generic `transfer_subset_iff` (with `Delta' = Set.univ`, so the `Delta'  I  *`/`U.
    master  I  *` wrappers are
trivial since `Yidx e i  subseteq  U.master` always) yields exactly Scott's matching
    invariant restricted to
plain inclusion. -/
theorem embed_subset_iff (i j : Nat) : e i  subseteq  e j  <->  Yidx e i  subseteq
    Yidx e j := by
  rw [ <-  idxSet_subset_iff e i j]
  have := transfer_subset_iff splitChoice (idxSet e) Set.univ univ_nonempty_nat
```

```
    splitChoice_isSplitSpec i j
  rwa [Set.univ_inter, Set.inter_eq_self_of_subset_right (Yidx_subset_master e i)] at
    this

theorem embed_eq_iff (i j : Nat) : e i = e j  <->  Yidx e i = Yidx e j :=
  <fun h => Set.Subset.antisymm ((embed_subset_iff e i j).mp h.subset)
      ((embed_subset_iff e j i).mp h.symm.subset),
    fun h => Set.Subset.antisymm ((embed_subset_iff e i j).mpr h.subset)
      ((embed_subset_iff e j i).mpr h.symm.subset)>

end Setup

section Subsystem

variable (D : NeighborhoodSystem a) (e : Nat  ->  Set a)
  (hcover : forall  S, D.mem S  <->  exists  n, S = e n) (he0 : e 0 = D.master)

include hcover in
/-- **The shared "find a matching index" step.** Given `D.mem (e i  I  e j)` (however
    that was
established -- a direct witness, or `D`'s own `inter_mem`), `hcover` names it as `e m`
    for some
`m`, and `transfer_inter_eq_iff` (fed through the `idxSet`-level rewriting
`idxSet_inter_of_inter_eq`) pushes the equation across to `Yidx`. This single lemma
    drives
`DprimeU`'s `inter_mem`, `DprimeU_subsystem`'s `inter_closed`, and both halves of the
    element-level
isomorphism below. -/
theorem exists_inter_index_of_dmem {i j : Nat} (hDij : D.mem (e i  I  e j)) :
    exists  m, e i  I  e j = e m  /\  Yidx e i  I  Yidx e j = Yidx e m := by
  obtain <m, hm> := (hcover (e i  I  e j)).mp hDij
  exact <m, hm, (transfer_inter_eq_iff splitChoice (idxSet e) Set.univ
    univ_nonempty_nat
    splitChoice_isSplitSpec i j m
    (Set.subset_univ _) (Set.subset_univ _) (Set.subset_univ _)).mp (
    idxSet_inter_of_inter_eq e hm)>

/-- **`D'`**: the neighbourhood system generated by `{Yidx e n ? n  in  Nat}`. Its `
    inter_mem` axiom
transfers *directly* from `D`'s own `inter_mem`: a witness `X k  subseteq  X i  I  X j`
     on the `D'`-side pulls
back (`embed_subset_iff`) to a witness `e k  subseteq  e i  I  e j` on the `D`-side, so
     `D`'s own axiom
supplies `e m = e i  I  e j` for some `m`, and `transfer_inter_eq_iff` pushes this
    equation back
across to `Yidx e i  I  Yidx e j = Yidx e m`. -/
noncomputable def DprimeU : NeighborhoodSystem ? where
  mem Y := exists  n, Y = Yidx e n
  master := U.master
  master_nonempty := U.master_nonempty
  master_mem := <0, (Yidx_zero D e hcover he0).symm>
  sub_master := by rintro Y <n, rfl>; exact Yidx_subset_master e n
  inter_mem := by
    rintro X Y Z <i, rfl> <j, rfl> <k, rfl> hZsub
    have h1 : e k  subseteq  e i := (embed_subset_iff e k i).mpr (hZsub.trans Set.
    inter_subset_left)
```

```
    have h2 : e k  subseteq  e j := (embed_subset_iff e k j).mpr (hZsub.trans Set.
    inter_subset_right)
    obtain <m, -, hYeq> := exists_inter_index_of_dmem D e hcover
      (D.inter_mem (D_mem_e D e hcover i) (D_mem_e D e hcover j) (D_mem_e D e hcover k
    )
        (Set.subset_inter h1 h2))
    exact <m, hYeq>

include hcover in
/-- **The "unwitnessed" variant**, used exactly when only the raw non-emptiness of `
    Yidx e i  I
Yidx e j` is known (not an explicit `D`-side witness): non-emptiness transfers down
(`transfer_inter_empty_iff`) to `idxSet e i  I  idxSet e j`, any element `k` of which
    supplies
`e k  subseteq  e i  I  e j`, so `D`'s own `inter_mem` produces `D.mem (e i  I  e j)`
    and
`exists_inter_index_of_dmem` finishes. -/
theorem exists_inter_index_of_nonempty {i j : Nat} (hne : (Yidx e i  I  Yidx e j).
    Nonempty) :
    exists  m, e i  I  e j = e m  /\  Yidx e i  I  Yidx e j = Yidx e m := by
  have hne' : (idxSet e i  I  idxSet e j).Nonempty := by
    by_contra hcon
    rw [Set.not_nonempty_iff_eq_empty] at hcon
    have hkey := transfer_inter_empty_iff splitChoice (idxSet e) Set.univ
    univ_nonempty_nat
      splitChoice_isSplitSpec i j
    rw [Set.univ_inter, Set.inter_eq_self_of_subset_right (Yidx_subset_master e i)] at
     hkey
    exact hne.ne_empty (hkey.mp hcon)
  obtain <k, hki, hkj> := hne'
  exact exists_inter_index_of_dmem D e hcover
    (D.inter_mem (D_mem_e D e hcover i) (D_mem_e D e hcover j) (D_mem_e D e hcover k)
      (Set.subset_inter hki hkj))

include hcover he0 in
/-- **`D' ? U`.** `sub` is `Yidx_mem`; `inter_closed` uses `
    exists_inter_index_of_nonempty` to find
*some* witnessed-consistent pair on the `D`-side from the mere non-emptiness of `Yidx e
     i  I  Yidx e
j` (guaranteed by `U.mem`). -/
theorem DprimeU_subsystem : DprimeU D e hcover he0 ? U where
  master_eq := rfl
  sub := by rintro Y <n, rfl>; exact Yidx_mem e n
  inter_closed := by
    rintro X Y <i, rfl> <j, rfl> hUmem
    obtain <m, -, hYeq> := exists_inter_index_of_nonempty D e hcover hUmem.2.1
    exact <m, hYeq>

end Subsystem

section Iso

variable (D : NeighborhoodSystem a) (e : Nat  ->  Set a)
  (hcover : forall  S, D.mem S  <->  exists  n, S = e n) (he0 : e 0 = D.master)

/-- **Pullback**: the `D`-filter `{e n ? Yidx e n  in  y}` induced by a `D'`-filter `y
```

```
    `. -/
def toD (y : (DprimeU D e hcover he0).Element) : D.Element where
  mem S := exists  n, S = e n  /\  y.mem (Yidx e n)
  sub := fun <n, hn, _> => hn  |>  D_mem_e D e hcover n
  master_mem := <0, he0.symm, by rw [Yidx_zero D e hcover he0]; exact y.master_mem>
  inter_mem := by
    rintro S T <i, rfl, hyi> <j, rfl, hyj>
    have hD'mem : (DprimeU D e hcover he0).mem (Yidx e i  I  Yidx e j) := y.sub (y.
    inter_mem hyi hyj)
    obtain <m, hem, hYeq> := exists_inter_index_of_nonempty D e hcover
      ((DprimeU_subsystem D e hcover he0).sub hD'mem).2.1
    exact <m, hem, hYeq  |>  y.inter_mem hyi hyj>
  up_mem := by
    rintro S T <i, rfl, hyi> hDT hST
    obtain <j, rfl> := (hcover T).mp hDT
    have hYij : Yidx e i  subseteq  Yidx e j := (embed_subset_iff e i j).mp hST
    exact <j, rfl, y.up_mem hyi <j, rfl> hYij>

/-- **Pushforward**: the `D'`-filter `{Yidx e n ? e n  in  x}` induced by a `D`-filter
    `x`. -/
def toDprimeU (x : D.Element) : (DprimeU D e hcover he0).Element where
  mem Y := exists  n, Y = Yidx e n  /\  x.mem (e n)
  sub := fun <n, hn, _> => <n, hn>
  master_mem := <0, (Yidx_zero D e hcover he0).symm, by rw [he0]; exact x.master_mem>
  inter_mem := by
    rintro X Y <i, rfl, hxi> <j, rfl, hxj>
    obtain <m, hem, hYeq> :=
      exists_inter_index_of_dmem D e hcover (x.sub (x.inter_mem hxi hxj))
    exact <m, hYeq, hem  |>  x.inter_mem hxi hxj>


  up_mem := by
    rintro X Y <i, rfl, hxi> <j, rfl> hXY
    have heij : e i  subseteq  e j := (embed_subset_iff e i j).mpr hXY
    exact <j, rfl, x.up_mem hxi (D_mem_e D e hcover j) heij>

/-- **The order isomorphism `D.Element ?o D'.Element`.** `toDprimeU`/`toD` are mutually
     inverse
(via `embed_eq_iff`, which resolves the ambiguity of *which* index represents a given
neighbourhood) and preserve/reflect ` <= ` (via `hcover`, which shows every `D`-
    neighbourhood a
filter can mention is literally some `e n`). -/
noncomputable def domainIso : DomainIso D (DprimeU D e hcover he0) where
  toFun := toDprimeU D e hcover he0
  invFun := toD D e hcover he0
  left_inv x := by
    apply Element.ext
    intro S
    constructor
    * rintro <n, hn, k, hk, hxk>
      rw [hn, (embed_eq_iff e n k).mpr hk]
      exact hxk
    * intro hS
      obtain <n, hn> := (hcover S).mp (x.sub hS)
      refine <n, hn, n, rfl, ?_>
      rwa [ <-  hn]
  right_inv y := by
```

```
    apply Element.ext
    intro Y
    constructor
    * rintro <n, hn, k, hk, hyk>
      rw [hn, (embed_eq_iff e n k).mp hk]
      exact hyk
    * intro hY
      obtain <n, hn> := y.sub hY
      refine <n, hn, n, rfl, ?_>
      rwa [ <-  hn]
  map_rel_iff' := by
    intro x x2
    constructor
    * intro hle S hxS
      obtain <n, hn> := (hcover S).mp (x.sub hxS)
      have hxn : x.mem (e n) := hn  |>  hxS
      obtain <k, hk, hx2k> := hle _ (<n, rfl, hxn> : (toDprimeU D e hcover he0 x).mem
    (Yidx e n))
      rw [hn, (embed_eq_iff e n k).mpr hk]
      exact hx2k
    * intro hle Y hY
      obtain <n, hn, hxn> := hY
      exact <n, hn, hle _ hxn>

end Iso

section Reencode

variable (e : Nat  ->  Set a)

theorem idxSet_eq_iff (i j : Nat) : idxSet e i = idxSet e j  <->  e i = e j :=
  <fun h => Set.Subset.antisymm ((idxSet_subset_iff e i j).mp h.subset)
      ((idxSet_subset_iff e j i).mp h.symm.subset),
    fun h => by rw [idxSet, idxSet, h]>

theorem idxSet_nonempty (n : Nat) : (idxSet e n).Nonempty := <n, self_mem_idxSet e n>

end Reencode

section Iso

variable (D : NeighborhoodSystem a) (e : Nat  ->  Set a)
  (hcover : forall  S, D.mem S  <->  exists  n, S = e n) (he0 : e 0 = D.master)

/-- **First sentence of Theorem 8.8 (isomorphism half).** `D  ~= ? D'`. -/
theorem isomorphic_DprimeU : D  ~= ? DprimeU D e hcover he0 := <domainIso D e hcover
    he0>

end Iso

end DomainTheory.NeighbourhoodSystems
```

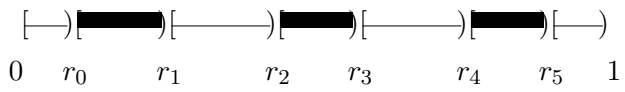
3.9.8 Definition 8.7 — interval presentation of U

DEFINITION 8.7. Let $\mathbb{Q}$ be the set of rational numbers, and let

$$[0,1) = \{q \in \mathbb{Q} \mid 0 \leq q < 1\},$$

and similarly for $[r,s)$ for any $r < s$ in $\mathbb{Q}$. The neighbourhood system $\mathcal{U}$ over $[0,1)$ is the set of all non-empty finite unions of rational intervals $[r,s)$ with $0 \leq r < s \leq 1$. □

A picture of a typical element of $\mathcal{U}$ could be drawn like this:



Note that any union can be taken as a *disjoint* union of the form

$$\bigcup_{i \leq n} [r_{2i}, r_{2i+1})$$

where $0 \leq r_0 < r_1 < r_2 < \cdots < r_{2n} < r_{2n+1} \leq 1$. (Hint: Any overlapping intervals or abutting intervals can always be combined into one long interval.) It is a most elementary exercise to show that, by virtue of this representation, the system $\mathcal{U}$ has a computable presentation. (Some isomorphic versions of $\mathcal{U}$ — equally effective — are recorded in the exercises.) Note that $\mathcal{U}$ has no minimal neighbourhoods: every set in $\mathcal{U}$ can be written as the union of two disjoint sets in $\mathcal{U}$. (Hint: Use the density of the ordering of $\mathbb{Q}$.) The significance of $\mathcal{U}$ can now be explained.

U.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    presentedIntervals
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_19
public import Mathlib.Data.Fintype.Defs

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem
/-- Scott's master neighbourhood `Delta = [0,1)` for `?`. -/
abbrev UMaster : Set ? := Set.Ico (0 : ?) 1

/-- Scott's token set `[0,1)` is non-empty. -/
theorem UMaster_nonempty : UMaster.Nonempty := by
  exact <0, by simp [UMaster]>

/-- Membership in Scott's universal family: non-empty subsets of `[0,1)` presentable as
     a
finite union of rational half-open intervals. -/
def UMem (X : Set ?) : Prop :=
  (exists  L : List (?  x  ?), X = presentedIntervals L)  /\  X.Nonempty  /\  X
    subseteq  Set.Ico (0 : ?) 1

namespace UMem
```

```
/-- Every `UMem` set is contained in `[0,1)`. -/
theorem sub_master {X : Set ?} (hX : UMem X) : X  subseteq  UMaster := hX.2.2

/-- `[0,1)` is presented by the single interval `[0,1)`. -/
theorem master_mem : UMem UMaster :=
  <<[(0, 1)], by rw [presentedIntervals.cons, presentedIntervals.nil, Set.union_empty
    ]>,
    <0, by norm_num>, subset_rfl>

/-- Intersection of two presentable sets is presentable, via `combineIntervals`. -/
theorem inter_mem {X Y Z : Set ?} (hX : UMem X) (hY : UMem Y) (hZ : UMem Z)
    (hZsub : Z  subseteq  X  I  Y) : UMem (X  I  Y) := by
  rcases hX with <<L1, rfl>, -, hXsub>
  rcases hY with <<L2, rfl>, -, ->
  rcases hZ with <-, hZne, ->
  exact <<combineIntervals L1 L2, presentedIntervals.inter L1 L2>,
    hZne.mono hZsub, Set.inter_subset_left.trans hXsub>
end UMem

/- concordance: source L4928 DEFINITION 8.7 interval presentation of U -/
/-- **Definition 8.7 (Scott 1981, PRG-19).** The neighbourhood system `?` over `[0,1)
    subseteq  ?`: its
neighbourhoods are the non-empty subsets of `[0,1)` presentable as a finite union of
    rational
intervals `[r,s)` (see the module docstring for why the raw endpoints need no `0  <=  r
     < s  <=  1`
side-condition, and `mem_iff_scott` for the literal reconciliation). Proof fields are
    named
theorems so Palomar can lock this value without generated `._proof_N` constants. -/
def U : NeighborhoodSystem ? where
  mem := UMem
  master := UMaster
  master_nonempty := UMaster_nonempty
  master_mem := UMem.master_mem
  inter_mem := UMem.inter_mem
  sub_master := UMem.sub_master

namespace U

/-- **`U` really is Scott's literal family**: a set is a `U`-neighbourhood iff it is
    presentable
by a list of intervals `[r,s)` individually satisfying Scott's bounds `0  <=  r < s  <=
     1`, and is
non-empty. -/
theorem mem_iff_scott {X : Set ?} :
    U.mem X  <->  exists  L : List (?  x  ?), (forall  p  in  L, (0 : ?)  <=  p.1  /\
     p.1 < p.2  /\  p.2  <=  1)  /\
      X = presentedIntervals L  /\  X.Nonempty := by
  constructor
  * rintro <<L, rfl>, hne, hsub>
    refine <(L.map clip).filter (fun p => decide (p.1 < p.2)), ?_, ?_, ?_>
    * rintro p hp
      obtain <hp1, hp2> := List.mem_filter.mp hp
      obtain <q, hq, rfl> := List.mem_map.mp hp1
      exact <le_sup_right, decide_eq_true_eq.mp hp2, inf_le_right>
```

```
    * rw [presentedIntervals.filter_lt, presentedIntervals.map_clip,
        Set.inter_eq_left.mpr hsub]
    * exact hne
  * rintro <L, hbound, rfl, hne>
    refine <<L, rfl>, hne, ?_>
    rintro x hx
    obtain <p, hp, hx1, hx2> := presentedIntervals.mem.mp hx
    obtain <hp0, -, hp1> := hbound p hp
    exact <hp0.trans hx1, hx2.trans_le hp1>

/-- **Scott's remark after Definition 8.7**: "`U` has no minimal neighbourhoods." -/
theorem no_minimal {X : Set ?} (hX : U.mem X) :
    exists  Y Z : Set ?, U.mem Y  /\  U.mem Z  /\  Y  I  Z = {}  /\  Y  U  Z = X  /\
    Y  !=  X  /\  Z  !=  X := by
  obtain <<L, rfl>, hne, hsub> := hX
  obtain <x, hx> := hne
  obtain <p, hp, hx1, hx2> := presentedIntervals.mem.mp hx
  have hlt : p.1 < p.2 := lt_of_le_of_lt hx1 hx2
  set m := (p.1 + p.2) / 2 with hm
  have hpm1 : p.1 < m := left_lt_add_div_two.mpr hlt
  have hpm2 : m < p.2 := add_div_two_lt_right.mpr hlt
  have hYZinter : (presentedIntervals L  I  Set.Iio m)  I  (presentedIntervals L  I
    Set.Ici m) =
      ({} : Set ?) := by
    ext y
    simp only [Set.mem_inter_iff, Set.mem_Iio, Set.mem_Ici, Set.mem_empty_iff_false,
    iff_false]
    rintro <<-, h1>, -, h2>
    exact absurd h1 (not_lt.mpr h2)
  have hYZunion : (presentedIntervals L  I  Set.Iio m)  U  (presentedIntervals L  I
    Set.Ici m) =
      presentedIntervals L := by
    ext y
    simp only [Set.mem_union, Set.mem_inter_iff, Set.mem_Iio, Set.mem_Ici]
    constructor
    * rintro (<h, -> | <h, ->) <;> exact h
    * intro h
      rcases lt_or_ge y m with hlt' | hge'
      * exact Or.inl <h, hlt'>
      * exact Or.inr <h, hge'>
  have hYne : (presentedIntervals L  I  Set.Iio m).Nonempty :=
    <p.1, presentedIntervals.mem.mpr <p, hp, le_refl _, hlt>, hpm1>
  have hZne : (presentedIntervals L  I  Set.Ici m).Nonempty :=
    <m, presentedIntervals.mem.mpr <p, hp, hpm1.le, hpm2>, le_refl m>
  refine <_, _, <<_, (presentedIntervals.map_clipLt m L).symm>, hYne,
      Set.inter_subset_left.trans hsub>,
    <<_, (presentedIntervals.map_clipGe m L).symm>, hZne, Set.inter_subset_left.trans
    hsub>,
    hYZinter, hYZunion, ?_, ?_>
  * intro hEq
    have hZsub : presentedIntervals L  I  Set.Ici m  subseteq  presentedIntervals L  I
      Set.Iio m := by
      rw [hEq]; exact Set.inter_subset_left
    exact hZne.ne_empty ((Set.inter_eq_right.mpr hZsub).symm.trans hYZinter)
  * intro hEq
    have hYsub : presentedIntervals L  I  Set.Iio m  subseteq  presentedIntervals L  I
```

```
      Set.Ici m := by
      rw [hEq]; exact Set.inter_subset_left
    exact hYne.ne_empty ((Set.inter_eq_left.mpr hYsub).symm.trans hYZinter)
/-- **`U` is closed under binary union**, as long as the result is non-empty. -/
theorem union_mem {X Y : Set ?} (hX : X = {}  \/  U.mem X) (hY : Y = {}  \/  U.mem Y)
    :
    X  U  Y = {}  \/  U.mem (X  U  Y) := by
  rcases hX with rfl | hX
  * rcases hY with rfl | hY
    * exact Or.inl (by simp)
    * exact Or.inr (by simpa using hY)
  * rcases hY with rfl | hY
    * exact Or.inr (by simpa using hX)
    * obtain <<L1, rfl>, hXne, hXsub> := hX
      obtain <<L2, rfl>, hYne, hYsub> := hY
      refine Or.inr <<L1 ++ L2, (presentedIntervals.append L1 L2).symm>,
        hXne.mono Set.subset_union_left, Set.union_subset hXsub hYsub>
/-- **`U` is closed under finite (`Fintype`-indexed) unions**, as long as the result is
    non-empty. -/
theorem iUnion_mem {? : Type*} [Fintype ?] {f : ?  ->  Set ?} (hf : forall  i, f i =
    {}  \/  U.mem (f i))
    (hne : (? i, f i).Nonempty) : U.mem (? i, f i) := by
  classical
  have hstep : forall  s : Finset ?, (? i  in  s, f i) = {}  \/  U.mem (? i  in  s, f
    i) := by
    intro s
    induction s using Finset.induction_on with
    | empty => exact Or.inl (by simp)
    | insert i s hi ih =>
      rw [Finset.set_biUnion_insert]
      exact union_mem (hf i) ih
  have hall : (? i, f i) = ? i  in  (Finset.univ : Finset ?), f i := by simp
  rw [hall] at hne  |-
  exact (hstep Finset.univ).resolve_left hne.ne_empty

/-- **`U` is closed under set-difference of two of its own neighbourhoods** (or the
    difference is
empty). -/
theorem diff_mem {X Y : Set ?} (hX : U.mem X) (hY : U.mem Y) : X \ Y = {}  \/  U.mem (
    X \ Y) := by
  obtain <<L1, rfl>, -, hXsub> := hX
  obtain <<L2, rfl>, -, -> := hY
  rcases Set.eq_empty_or_nonempty (presentedIntervals L1 \ presentedIntervals L2) with
    h | h
  * exact Or.inl h
  * exact Or.inr <<diffLists L1 L2, (presentedIntervals.diffLists L1 L2).symm>, h,
      Set.sdiff_subset.trans hXsub>

theorem diffClosed : U.DiffClosed := fun hX hY => diff_mem hX hY

/-- **`U` is Positive**: `combineIntervals` presents any raw intersection
    unconditionally; only
non-emptiness can obstruct `U.mem (X  I  Y)`. -/
theorem isPositive : U.IsPositive := by
  rintro X Y <<L1, rfl>, -, hXsub> <<L2, rfl>, -, ->
  refine <fun h => h.2.1, fun hne => <<combineIntervals L1 L2, presentedIntervals.
```

```
    inter L1 L2>,
    hne, Set.inter_subset_left.trans hXsub>>
theorem noMinimal : U.NoMinimal := by
  intro X hX
  obtain <Y, Z, hY, hZ, hYZinter, hYZunion, -, -> := no_minimal hX
  exact <Y, Z, hY, hZ, hY.2.1, hZ.2.1, hYZinter, hYZunion>

end U

end DomainTheory.NeighbourhoodSystems
```

3.9.9 Theorem 8.8 — universality of U

THEOREM 8.8. The system $\mathcal{U}$ is universal in the sense that, for every countable neighbourhood system $\mathcal{D}$, we have

$$\mathcal{D} \trianglelefteq \mathcal{U}.$$

Moreover, if $\mathcal{D}$ is effectively given, then the projection pair making the embedding can be taken as computable. Indeed there is a correspondence between effectively presented domains and the computable, finitary projections of $\mathcal{U}$.

Proof: As $\mathcal{D}$ is countable, we can assume that

$$\mathcal{D} = \{X_n \mid n \in \mathbb{N}\},$$

where $\mathcal{D}$ is a system over a set Δ (say, $X_0 = \Delta$). We shall do the effective and general cases together, where for the latter all remarks on recursiveness are just left out. So, if we want $\mathcal{D}$ effectively given, the above enumeration should be taken as the computable presentation.

Without loss of generality we can assume $\mathcal{D} \cong \mathcal{D}^\dagger$, since otherwise we would just replace $\mathcal{D}$ by $\mathcal{D}^\dagger$. The advantage of this preparation is that unions in $\mathcal{D}^\dagger$ keep things rather *separate* (as we noticed in constructing $\mathbb{P}\mathcal{D}$). In particular, we can be sure of this equivalence:

$$(\blacklozenge) \qquad X_m \subseteq \bigcup_{i<k} X_{n_i} \quad \text{iff} \quad \exists i < k.\, X_m \subseteq X_{n_i}.$$

This property, for example, fails for the system $\mathcal{U}$ as presented in Definition 8.7. However, that observation is of no moment, because we are employing the assumption with respect to $\mathcal{D}$ not $\mathcal{U}$.

The reason for the assumption is this: for $\delta \in \{+,-\}$ define for $X \in \mathcal{D}$:

$$\begin{aligned} \delta X = X \qquad & \text{if } \delta = +; \\ = \Delta \setminus X \quad & \text{if } \delta = -. \end{aligned}$$

(A similar notation will be used for $Y \in \mathcal{U}$.) Then for $\delta \in \{+,-\}^n$ the sets of the form

$$\bigcap_{i<n} \delta_i X_i \quad (= X_\delta, \text{ for short})$$

form a partition of Δ into (at most) 2^n parts. The reason for assumption $(\blacklozenge)$ is that we can effectively decide for each $\delta \in \{+,-\}^n$ whether one of these intersections is empty or not. (Why? — assuming that $\mathcal{D}$ is effectively given, of course). If for some reason we had not wanted to pass to $\mathcal{D}^\dagger$, we could have made this stronger assumption of decidability on the (positive) system $\mathcal{D}$. ($\mathcal{U}$, for example, satisfies it.)

Suppose, corresponding to $X_0, X_1, \ldots, X_{n-1}$, we have selected $Y_0, Y_1, \ldots, Y_{n-1} \in \mathcal{U}$ so that, for all $\delta \in \{+,-\}^n$,

$$(\blacksquare) \qquad \bigcap_{i<n} \delta_i X_i = \emptyset \quad \text{iff} \quad \bigcap_{i<n} \delta_i Y_i = \emptyset.$$

We wish to show - effectively - how to choose Y_n corresponding to X_n, so that ($\blacksquare$) holds with $n+1$ replacing n. Proceeding inductively, we obtain a recursive enumeration of sets $Y_n \in \mathcal{U}$ so that

$$\mathcal{D} \equiv \{Y_n \mid n \in \mathbb{N}\} \lhd \mathcal{U}.$$

Clearly the isomorphism (matching X_i to Y_i) will be computable and the projection is computable. (It will then remain only to consider the arbitrary finitary computable projection to complete the proof of the theorem.)

So, consider X_n; for each $\delta \in \{+,-\}^n$ there are four cases:

$$X_\delta \cap X_n = \emptyset, \quad X_\delta \cap -X_n = \emptyset,$$
$$X_\delta \cap X_n \neq \emptyset, \quad X_\delta \cap -X_n \neq \emptyset.$$

Corresponding to X_δ is a similar intersection Y_δ. If X_δ were $\emptyset$, then Y_δ would be also. If not, $Y_\delta \subseteq [0,1)$ is a union of rational intervals that can be written down explicitly. (Why?) In our four cases on X_n, the first implies the fourth. (Why?) Thus, we need only make some choices in these circumstances:

$$X_\delta \cap X_n = \emptyset : \text{choose } I_{\delta,n} = \emptyset;$$
$$X_\delta \cap -X_n = \emptyset : \text{choose } I_{\delta,n} = Y_\delta;$$
$$\text{otherwise} : \text{choose } I_{\delta,n} \subseteq Y_\delta, \text{ with } \emptyset \neq I_{\delta,n} \neq Y_\delta.$$

All these cases are decidable by assumption on $\mathcal{D}$, and the effective choice of (unions of) intervals is effective by construction of $\mathcal{U}$. Now set

$$Y_n = \bigcup_{\delta \in \{+,-\}^n} I_{\delta,n} \neq \emptyset.$$

The set $Y_n \in \mathcal{U}$, it can be found effectively, and ($\blacksquare$) is obviously satisfied for $n+1$.

Finally, suppose that a is a computable, finitary projection of $\mathcal{U}$. As we have seen in the proof of 8.5, the domain corresponding to the range of a is isomorphic to the neighbourhood system

$$\{Y \in \mathcal{U} \mid Y \; a \; Y\} \lhd \mathcal{U}.$$

Clearly, if a as a set of ordered pairs of neighbourhoods is recursively enumerable, then the above set is also recursively enumerable (because equality between neighbourhoods is decidable). It follows easily that the subsystem is effectively given as a neighbourhood system in its own right. $\square$

We have now proved that $\mathcal{U}$ is a nice and big domain that is nicely behaved with respect to computable mappings. It has some very interesting subdomains; to name a few:

$$\mathcal{U} + \mathcal{U}, \quad \mathcal{U} \oplus \mathcal{U}, \quad \mathcal{U} \times \mathcal{U}, \quad \mathcal{U} \otimes \mathcal{U}$$

$$\mathcal{U}_\perp, \quad \mathcal{U}^\infty, \quad \mathcal{U}^\S, \quad \mathbb{P}\mathcal{U}, \quad \mathcal{U} \to \mathcal{U}$$

That all of these are $\trianglelefteq \mathcal{U}$ follows from knowing that they are all effectively presented. What we wish to check next is that they all combine well with respect to projections. To this end the explicit definitions are given for the constructs $+$, $\times$, and $\to$, and the details of the others are left for the exercises.

Theorem_8_8.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Theorem_8_6
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.DAtom
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.UX
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.splitU
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.atomCode
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.atomUCode
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.YseqCode
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.DiagFixed
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.D_X
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    DprimeUPresentation
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    YseqCode_more
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    domainIsoCode
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    DprimeUCode
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    atomUCode_more

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    QPairCodes
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Exercise_7_18
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    VComputablePresentation
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Theorem_7_5
public import Mathlib.Data.Nat.Bitwise
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Exercise_3_26

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open Domain.Recursive NeighborhoodSystem
section
open NeighborhoodSystem
variable {a : Type*}

/- concordance: source L4943 THEOREM 8.8 universality of U -/
/-- **First sentence of Theorem 8.8 (Scott 1981, PRG-19, Lecture VIII).** `?` is
    universal: every *countable*
neighbourhood system `D` embeds, up to isomorphism, as a subsystem of `?`.

The enumeration `e` is built from any surjection `f : Nat  ->  {S // D.mem S}` (`
    exists_surjective_nat`,
```

```
using `Countable`/`Nonempty` -- the only place `Classical.choice` enters, since neither
     hypothesis is
data), shifted by one and patched at `0` to enforce Scott's convention `X_0 = Delta`.
    Everything else --
`idxSet`'s "separated" reindexing, the `Yidx`/transfer apparatus of `Theorem88.lean`,
    and the
`DprimeU`/`domainIso` machinery above -- is then assembled directly. -/
theorem theorem_8_8_a.{u} {a : Type u} (D : NeighborhoodSystem a)
    [Countable {S : Set a // D.mem S}] :
    exists  D' : NeighborhoodSystem ?, (D  ~= ? D')  /\  (D' ? U) := by
  have : Nonempty {S : Set a // D.mem S} := <<D.master, D.master_mem>>
  obtain <f, hf> := exists_surjective_nat {S : Set a // D.mem S}
  set e : Nat  ->  Set a := fun n => if n = 0 then D.master else (f (n - 1)).1 with
    hedef
  have he0 : e 0 = D.master := ite_eq_left rfl
  have hen : forall  n, e (n + 1) = (f n).1 := by
    intro n
    show (if n + 1 = 0 then D.master else (f (n + 1 - 1)).1) = (f n).1
    rw [ite_eq_right (Nat.succ_ne_zero n), Nat.succ_sub_one]
  have hcover : forall  S, D.mem S  <->  exists  n, S = e n := by
    intro S
    constructor
    * intro hS
      obtain <k, hk> := hf <S, hS>
      exact <k + 1, by rw [hen k]; exact (congrArg Subtype.val hk).symm>
    * rintro <n, rfl>
      cases n with
      | zero => rw [he0]; exact D.master_mem
      | succ n => rw [hen n]; exact (f n).2
  exact <DprimeU D e hcover he0, isomorphic_DprimeU D e hcover he0, DprimeU_subsystem
    D e hcover he0>

/-- **First sentence of Theorem 8.8 (Scott 1981, PRG-19).** Scott's wording:
"The system `?` is universal in the sense that, for every countable neighbourhood
system `?`, we have `? ? ?`." This packages `theorem_8_8_a` as that relation. -/
theorem theorem_8_8.{u} {a : Type u} (D : NeighborhoodSystem a)
    [Countable {S : Set a // D.mem S}] : D ? U := by
  obtain <D', hiso, hsub> := theorem_8_8_a D
  exact <D', hsub, hiso>

end

open NeighborhoodSystem Domain.Recursive

/-- **Theorem 8.8(b) (Scott 1981, PRG-19), final assembly.** If `D` is effectively
    given (via a
`ComputablePresentation P`), the general universality of `U` (Theorem 8.8(a)) can be
    witnessed by a
`D' : NeighborhoodSystem ?` that is *itself* effectively given (`P' :
    ComputablePresentation D'`),
isomorphic to `D`, a subsystem of `U`, **and** whose projection pair `i := (D' ? U).inj
    `,
`j := (D' ? U).proj` (Proposition 6.12) is computable in both directions relative to `P
    '` and `U`'s
own presentation `UComputablePresentation`. -/
theorem theorem_8_8_b {a : Type*} {D : NeighborhoodSystem a} (P :
```

```
    ComputablePresentation D) :
    exists  (D' : NeighborhoodSystem ?) (P' : ComputablePresentation D') (h : D' ? U),
      (D  ~= ? D')  /\
        ComputableMap P' UComputablePresentation h.inj  /\
        ComputableMap UComputablePresentation P' h.proj :=
  <DprimeUCode P, DprimeUCodePresentation P, DprimeUCode_subsystem P,
    isomorphic_DprimeUCode P, DprimeUCode_inj_isComputableMap P,
    DprimeUCode_proj_isComputableMap P>

section
open NeighborhoodSystem Domain.Recursive ApproximableMap
variable {a : Type*} {V : NeighborhoodSystem a}

/-- **Theorem 8.8(c), Part 6 of 6 -- final assembly.** A computable approximable self-
    map `a` of an
effectively given `V` (via `P`) makes `fixedNbhd a` effectively given: package Parts
    1-5's `D_X`/
`D_inter` enumeration, instantiated at concrete witnesses `qChar` (Part 1) and `cons`
    (`P`'s own
`cons_computable`), into a `ComputablePresentation (fixedNbhd a)`. -/
theorem fixedNbhd_isEffectivelyGiven {P : ComputablePresentation V} {a :
    ApproximableMap V V}
    (hcomp : ComputableMap P P a) : (fixedNbhd a).IsEffectivelyGiven := by
  obtain <qChar, hqCharp, hqChar> := diagFixed_exists_qChar hcomp
  obtain <cons, hconsp, hconse> := P.cons_computable
  have hcons : forall  n m, cons (Nat.pair n m) = 1  <->  exists  k, P.X k  subseteq
    P.X n  I  P.X m := by
    intro n m
    have h := hconse (Nat.pair n m)
    simp only [unpair_pair_fst, unpair_pair_snd] at h
    exact h.symm
  refine <{
    X := D_X P qChar cons
    mem_X := D_X_mem hqChar hcons
    surj := fun hY => D_X_surj hqChar hcons hY
    interEq_computable := D_X_interEq_computable P qChar cons hqCharp hconsp
    cons_computable := D_X_cons_computable P qChar cons hqChar hcons hqCharp hconsp
    inter := D_inter
    inter_primrec := D_inter_primrec
    inter_spec := fun h => D_X_inter_spec hcons h
    masterIdx := 0
    masterIdx_spec := ?_ }>
  show D_X P qChar cons 0 = V.master
  show P.X (myFoldCode P qChar cons 0) = V.master
  rw [myFoldCode_eq, decodeList_zero, myFold_nil, P.masterIdx_spec]

/-- **Theorem 8.8(c) (Scott 1981, PRG-19), headline.** If `a : U  ->  U` is a
    computable, finitary
projection of the universal domain, then `{Y  in  U ? Y a Y}` is effectively given (and
    , unconditionally
by Theorem 8.5, a subsystem of `U`). As in Theorem 8.6(c)'s converse, `hfin` is carried
     in the
signature to match Scott's stated hypothesis on `a`, but is **not** actually needed by
    this
implication -- only `hcomp` drives the effectiveness argument; this is called out here
    rather than
```

```
silently dropping the hypothesis. -/
theorem theorem_8_8_c {a : ApproximableMap U U} (_hfin : FinitaryProjection a)
    (hcomp : ComputableMap UComputablePresentation UComputablePresentation a) :
    (fixedNbhd a).IsEffectivelyGiven  /\  fixedNbhd a ? U :=
  <fixedNbhd_isEffectivelyGiven hcomp, fixedNbhd_subsystem a>

end
open NeighborhoodSystem ApproximableMap Exercise326

/-- Every `?`-neighbourhood is non-empty (built into Definition 8.7). -/
theorem U_mem_nonempty : forall  X, U.mem X  ->  X.Nonempty := fun _ hX => hX.2.1

/-- `?  ->  ?` is effectively given (Theorem 7.5, applied to `U`'s own presentation
    twice). -/
theorem funSpaceUU_isEffectivelyGiven : (funSpace U U).IsEffectivelyGiven :=
  funSpace_isEffectivelyGiven U_isEffectivelyGiven U_isEffectivelyGiven

/-- A fixed presentation of `?  ->  ?`. -/
noncomputable def funSpaceUUPresentation : ComputablePresentation (funSpace U U) :=
  funSpaceUU_isEffectivelyGiven.some

/-- **`fun f. b comp f comp a : (? -> ?)  ->  (? -> ?)`**, uncurried as `b  comp  eval
    comp  (id  x ? a)`. -/
noncomputable def lamComb (a b : ApproximableMap U U) :
    ApproximableMap (funSpace U U) (funSpace U U) :=
  curry (b.comp ((evalMap U U).comp (prodMap (idMap (funSpace U U)) a)))

/-- `?  x  ?` is effectively given (Theorem 7.4, applied to `U`'s own presentation
    twice). -/
theorem prodUU_isEffectivelyGiven : (prod U U).IsEffectivelyGiven :=
  prod_isEffectivelyGiven U_isEffectivelyGiven U_isEffectivelyGiven

/-- A fixed presentation of `?  x  ?`. -/
noncomputable def prodUUPresentation : ComputablePresentation (prod U U) :=
  prodUU_isEffectivelyGiven.some

/-- `? + ?` is effectively given (Theorem 7.4, applied to `U`'s own presentation twice)
    . -/
theorem sumUU_isEffectivelyGiven :
    (sum U U U_mem_nonempty U_mem_nonempty).IsEffectivelyGiven :=
  sum_isEffectivelyGiven U_isEffectivelyGiven U_isEffectivelyGiven

/-- A fixed presentation of `? + ?`. -/
noncomputable def sumUUPresentation : ComputablePresentation (sum U U U_mem_nonempty
    U_mem_nonempty) :=
  sumUU_isEffectivelyGiven.some

section
open NeighborhoodSystem Domain.Recursive ApproximableMap
variable {a : Type*} {D : NeighborhoodSystem a} (P : ComputablePresentation D)

/-- **`theorem_8_8_b`, strengthened.** For any effectively given `D` (via `P`), there
    is a
*single* computable projection pair `i : D  ->  U`, `j : U  ->  D` directly (no
    intermediate isomorphic
copy): compose `isoInj`/`isoProj` (the code-level iso, computable) with `DprimeUCode`'s
```

```
     own projection pair
into `U` (Theorem 8.8(b)(vii), computable). -/
theorem theorem_8_8_b_strong :
    exists  (i : ApproximableMap D U) (j : ApproximableMap U D),
      j.comp i = idMap D  /\  i.comp j  <=  idMap U  /\
        ComputableMap P UComputablePresentation i  /\
        ComputableMap UComputablePresentation P j := by
  set h := DprimeUCode_subsystem P with hh
  refine <h.inj.comp (isoInj P), (isoProj P).comp h.proj, ?_, ?_, ?_, ?_>
  * rw [comp_assoc,  <-  comp_assoc h.proj h.inj (isoInj P), h.proj_comp_inj,
    idMap_comp]
    exact isoProj_comp_isoInj P
  * have heq : (h.inj.comp (isoInj P)).comp ((isoProj P).comp h.proj) = h.inj.comp h.
    proj := by
      rw [comp_assoc,  <-  comp_assoc (isoInj P) (isoProj P) h.proj,
    isoInj_comp_isoProj, idMap_comp]
    rw [heq]
    exact h.inj_comp_proj_le
  * exact comp_isComputable (isoInj_isComputableMap P) (
    DprimeUCode_inj_isComputableMap P)
  * exact comp_isComputable (DprimeUCode_proj_isComputableMap P) (
    isoProj_isComputableMap P)

end

end DomainTheory.NeighbourhoodSystems
```

3.9.10 Definition 8.9 — constructs on projections

DEFINITION 8.9. Let the computable projection pairs $i_+ : \mathcal{U} + \mathcal{U} \to \mathcal{U}$ and $j_+ : \mathcal{U} \to \mathcal{U} + \mathcal{U}$ be fixed. Similarly choose $i_\times, j_\times$ and $i_\to, j_\to$ for $\mathcal{U} \times \mathcal{U}$ and $\mathcal{U} \to \mathcal{U}$. Define:

$$a + b = \mathrm{cond} \circ \langle \mathrm{which}, i_+ \circ \mathrm{in}_0 \circ a \circ \mathrm{out}_0, i_+ \circ \mathrm{in}_1 \circ b \circ \mathrm{out}_1 \rangle \circ j_+ \ ;$$

$$a \times b = i_\times \circ \langle a \circ \mathrm{proj}_0, b \circ \mathrm{proj}_1 \rangle \circ j_\times \ ;$$

$$a \to b = i_\to \circ (\lambda f.\ b \circ f \circ a) \circ j_\to \ ,$$

for all $a, b : \mathcal{U} \to \mathcal{U}$. □

These interesting(computable!) combinators on elements of $\mathcal{U} \to \mathcal{U}$ have many, many properties. We shall, however, only see what they do to projections.

— embeddings and `lamComb`

Definition_8_9.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Theorem_8_8
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_13
```

```
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap Exercise326

/- concordance: source L5028 DEFINITION 8.9 constructs on projections -/
/-- **`i_ ->  : (? -> ?)  ->  ?`** (Definition 8.9), fixed by `theorem_8_8_b_strong`
    applied to `? -> ?`. -/
noncomputable def iArrow : ApproximableMap (funSpace U U) U :=
  (theorem_8_8_b_strong funSpaceUUPresentation).choose

/-- **`i_+ : ?+?  ->  ?`** (Definition 8.9), fixed by `theorem_8_8_b_strong` applied to
     `?+?`. -/
noncomputable def iPlus : ApproximableMap (sum U U U_mem_nonempty U_mem_nonempty) U :=
  (theorem_8_8_b_strong sumUUPresentation).choose

/-- **`i_ x  : ? x ?  ->  ?`** (Definition 8.9), fixed by `theorem_8_8_b_strong`
    applied to `? x ?`. -/
noncomputable def iTimes : ApproximableMap (prod U U) U :=
  (theorem_8_8_b_strong prodUUPresentation).choose

/-- **`j_ ->  : ?  ->  (? -> ?)`** (Definition 8.9). -/
noncomputable def jArrow : ApproximableMap U (funSpace U U) :=
  (theorem_8_8_b_strong funSpaceUUPresentation).choose_spec.choose

/-- **`j_+ : ?  ->  ?+?`** (Definition 8.9). -/
noncomputable def jPlus : ApproximableMap U (sum U U U_mem_nonempty U_mem_nonempty) :=
  (theorem_8_8_b_strong sumUUPresentation).choose_spec.choose

/-- **`j_ x  : ?  ->  ? x ?`** (Definition 8.9). -/
noncomputable def jTimes : ApproximableMap U (prod U U) :=
  (theorem_8_8_b_strong prodUUPresentation).choose_spec.choose

theorem iArrow_comp_jArrow_le : iArrow.comp jArrow  <=  idMap U :=
  (theorem_8_8_b_strong funSpaceUUPresentation).choose_spec.choose_spec.2.1

theorem iArrow_isComputableMap :
    ComputableMap funSpaceUUPresentation UComputablePresentation iArrow :=
  (theorem_8_8_b_strong funSpaceUUPresentation).choose_spec.choose_spec.2.2.1

theorem iPlus_comp_jPlus_le : iPlus.comp jPlus  <=  idMap U :=
  (theorem_8_8_b_strong sumUUPresentation).choose_spec.choose_spec.2.1

theorem iPlus_isComputableMap : ComputableMap sumUUPresentation
    UComputablePresentation iPlus :=
  (theorem_8_8_b_strong sumUUPresentation).choose_spec.choose_spec.2.2.1

theorem iTimes_comp_jTimes_le : iTimes.comp jTimes  <=  idMap U :=
  (theorem_8_8_b_strong prodUUPresentation).choose_spec.choose_spec.2.1

theorem iTimes_isComputableMap :
    ComputableMap prodUUPresentation UComputablePresentation iTimes :=
  (theorem_8_8_b_strong prodUUPresentation).choose_spec.choose_spec.2.2.1
```

```
theorem jArrow_comp_iArrow : jArrow.comp iArrow = idMap _ :=
  (theorem_8_8_b_strong funSpaceUUPresentation).choose_spec.choose_spec.1

theorem jArrow_isComputableMap :
    ComputableMap UComputablePresentation funSpaceUUPresentation jArrow :=
  (theorem_8_8_b_strong funSpaceUUPresentation).choose_spec.choose_spec.2.2.2

theorem jPlus_comp_iPlus : jPlus.comp iPlus = idMap _ :=
  (theorem_8_8_b_strong sumUUPresentation).choose_spec.choose_spec.1

theorem jPlus_isComputableMap : ComputableMap UComputablePresentation
    sumUUPresentation jPlus :=
  (theorem_8_8_b_strong sumUUPresentation).choose_spec.choose_spec.2.2.2

theorem jTimes_comp_iTimes : jTimes.comp iTimes = idMap _ :=
  (theorem_8_8_b_strong prodUUPresentation).choose_spec.choose_spec.1

theorem jTimes_isComputableMap :
    ComputableMap UComputablePresentation prodUUPresentation jTimes :=
  (theorem_8_8_b_strong prodUUPresentation).choose_spec.choose_spec.2.2.2

section
open NeighborhoodSystem ApproximableMap Exercise326
variable {a b : Type*} {D : NeighborhoodSystem a} {E : NeighborhoodSystem b}

/-- The image of `in_0` always reaches the left copy (witnessed by `V_0.master`, since
    `v.master_mem`
always holds and `in_0`'s relation only needs `inj_0 X  subseteq  inj_0 V_0.master`).
    -/
theorem inMap_0_toElementMap_reaches_left {? ?} {V_0 : NeighborhoodSystem ?} {V_1 :
    NeighborhoodSystem ?}
    {h_0 : forall  X, V_0.mem X  ->  X.Nonempty} {h_1 : forall  Y, V_1.mem Y  ->  Y.
    Nonempty} (v : V_0.Element) :
    exists  X, V_0.mem X  /\  ((inMap_0 (h_0 := h_0) (h_1 := h_1)).toElementMap v).mem
     (inj_0 X) := by
  refine <V_0.master, V_0.master_mem, V_0.master, v.master_mem, V_0.master_mem, ?_,
    subset_rfl>
  exact Or.inr (Or.inl <V_0.master, V_0.master_mem, rfl>)

/-- The image of `in_1` always reaches the right copy. -/
theorem inMap_1_toElementMap_reaches_right {? ?} {V_0 : NeighborhoodSystem ?} {V_1 :
    NeighborhoodSystem ?}
    {h_0 : forall  X, V_0.mem X  ->  X.Nonempty} {h_1 : forall  Y, V_1.mem Y  ->  Y.
    Nonempty} (v : V_1.Element) :
    exists  Y, V_1.mem Y  /\  ((inMap_1 (h_0 := h_0) (h_1 := h_1)).toElementMap v).mem
     (inj_1 Y) := by
  refine <V_1.master, V_1.master_mem, V_1.master, v.master_mem, V_1.master_mem, ?_,
    subset_rfl>
  exact Or.inr (Or.inr <V_1.master, V_1.master_mem, rfl>)

theorem lamComb_eq (a b : ApproximableMap U U) :
    lamComb a b = curry (b.comp ((evalMap U U).comp (prodMap (idMap (funSpace U U)) a)
    )) := rfl

/-- No element of `?_0+?_1` reaches into both the left and the right copy (Exercise
```

```
    3.18's
disjointness `inj_0_inter_inj_1`, plus `not_sum_mem_empty`). -/
theorem not_sum_reaches_both {? ?} {V_0 : NeighborhoodSystem ?} {V_1 :
    NeighborhoodSystem ?}
    {h_0 : forall  X, V_0.mem X  ->  X.Nonempty} {h_1 : forall  Y, V_1.mem Y  ->  Y.
    Nonempty}
    {w : (sum V_0 V_1 h_0 h_1).Element} (hL : exists  X, V_0.mem X  /\  w.mem (inj_0 X
    ))
    (hR : exists  Y, V_1.mem Y  /\  w.mem (inj_1 Y)) : False := by
  obtain <X, _, hX> := hL
  obtain <Y, _, hY> := hR
  have h := w.inter_mem hX hY
  rw [inj_0_inter_inj_1] at h
  exact not_sum_mem_empty (w.sub h)

/-- **`lamComb a b`, transported through `funSpaceEquiv`, is exactly `fun f. b comp f
    comp a`.** Chains
`toElementMap_curry_apply` (Theorem 3.12(i)), `toElementMap_prodMap_pair`/`
    toElementMap_idMap`,
and `evalMap_apply` (Theorem 3.11(i)). -/
theorem toApproxMap_toElementMap_lamComb (a b : ApproximableMap U U)
    (phi : (funSpace U U).Element) :
    toApproxMap ((lamComb a b).toElementMap phi) = b.comp ((toApproxMap phi).comp a)
    := by
  apply ext_of_toElementMap
  intro z
  rw [lamComb_eq, toElementMap_curry_apply, toElementMap_comp, toElementMap_comp,
    toElementMap_prodMap_pair, toElementMap_idMap, evalMap_apply, toElementMap_comp,
    toElementMap_comp]

/-- **Proposition 8.10, ` -> `-case (the `lamComb` half), retraction.** -/
theorem isRetraction_lamComb {a b : ApproximableMap U U} (ha : Retraction a)
    (hb : Retraction b) : Retraction (lamComb a b) := by
  constructor
  apply ext_of_toElementMap
  intro phi
  apply (funSpaceEquiv U U).injective
  rw [funSpaceEquiv_apply, funSpaceEquiv_apply, toElementMap_comp,
    toApproxMap_toElementMap_lamComb a b ((lamComb a b).toElementMap phi),
    toApproxMap_toElementMap_lamComb a b phi,
    comp_assoc b ((toApproxMap phi).comp a) a,
     <-  comp_assoc b b (((toApproxMap phi).comp a).comp a), hb.out,
    comp_assoc (toApproxMap phi) a a, ha.out]

/-- **Proposition 8.10, ` -> `-case (the `lamComb` half), projection.** -/
theorem le_idMap_lamComb {a b : ApproximableMap U U} (ha : a  <=  idMap U) (hb : b  <=
     idMap U) :
    lamComb a b  <=  idMap (funSpace U U) := by
  rw [le_iff_toElementMap_le]
  intro phi
  rw [toElementMap_idMap,  <-  (funSpaceEquiv U U).le_iff_le, funSpaceEquiv_apply,
    funSpaceEquiv_apply, toApproxMap_toElementMap_lamComb]
  calc b.comp ((toApproxMap phi).comp a)  <=  (idMap U).comp ((toApproxMap phi).comp (
    idMap U)) :=
      comp_mono_gen hb (comp_mono_gen le_rfl ha)
    _ = toApproxMap phi := by rw [comp_idMap, idMap_comp]
```

```
/-- `a  comp  a = a` unwound at the element level: `a(a(x)) = a(x)` for every `x`. -/
theorem toElementMap_idem_of_isRetraction {a : ApproximableMap E E} (ha : Retraction a
    )
    (x : E.Element) : a.toElementMap (a.toElementMap x) = a.toElementMap x := by
  rw [ <-  toElementMap_comp, ha.out]

/-- If `i  comp  j  <=  I_E`, then `i(j(x))  <=  x` for every `x  in  |E|`. -/
theorem toElementMap_le_of_comp_le_idMap {i : ApproximableMap D E} {j :
    ApproximableMap E D}
    (hij : i.comp j  <=  idMap E) (x : E.Element) : i.toElementMap (j.toElementMap x)
     <=  x := by
  have h := (le_iff_toElementMap_le.mp hij) x
  rwa [toElementMap_comp, toElementMap_idMap] at h

/-- `a  <=  idMap E` unwound at the element level: `a(x)  <=  x` for every `x`. -/
theorem toElementMap_le_self_of_le_idMap {a : ApproximableMap E E} (ha : a  <=  idMap
    E)
    (x : E.Element) : a.toElementMap x  <=  x := by
  have h := (le_iff_toElementMap_le.mp ha) x
  rwa [toElementMap_idMap] at h

/-- If `j  comp  i = I_D`, then `j(i(v)) = v` for every `v  in  |D|`. -/
theorem toElementMap_of_comp_eq_idMap {i : ApproximableMap D E} {j : ApproximableMap E
     D}
    (hji : j.comp i = idMap D) (v : D.Element) : j.toElementMap (i.toElementMap v) = v
     := by
  rw [ <-  toElementMap_comp, hji, toElementMap_idMap]

end
end DomainTheory.NeighbourhoodSystems
```

— sumComb and Pref-folded companions

sumComb.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Definition_8_9

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap Exercise326

/-- **`a + b : ?  ->  ?`** (Definition 8.9): `cond  comp  <which, i_+ comp in_0 comp a
    comp out_0, i_+ comp in_1 comp b comp out_1>  comp  j_+`. -/
noncomputable def sumComb (a b : ApproximableMap U U) : ApproximableMap U U :=
  (cond U).comp
    ((paired (whichMap U U U_mem_nonempty U_mem_nonempty)
        (paired (iPlus.comp ((inMap_0 (h_0 := U_mem_nonempty) (h_1 := U_mem_nonempty))
    .comp
                (a.comp (outMap_0 (h_0 := U_mem_nonempty) (h_1 := U_mem_nonempty))))
    )
```

```
                (iPlus.comp ((inMap_1 (h_0 := U_mem_nonempty) (h_1 := U_mem_nonempty))
    .comp
                  (b.comp (outMap_1 (h_0 := U_mem_nonempty) (h_1 := U_mem_nonempty))))
    ))).comp
      jPlus)

/-- If `j  comp  i = I_D`, then `j(E.bot) = D.bot`. -/
theorem toElementMap_bot_eq_bot_of_comp_eq_idMap {a b : Type*} {D : NeighborhoodSystem
     a}
    {E : NeighborhoodSystem b} {i : ApproximableMap D E} {j : ApproximableMap E D}
    (hji : j.comp i = idMap D) : j.toElementMap E.bot = D.bot := by
  apply le_antisymm
  * have h2 := j.toElementMap_mono (E.bot_le (i.toElementMap D.bot))
    rwa [toElementMap_of_comp_eq_idMap hji] at h2
  * exact D.bot_le _

theorem jPlus_bot_eq_bot :
    jPlus.toElementMap U.bot = (sum U U U_mem_nonempty U_mem_nonempty).bot :=
  toElementMap_bot_eq_bot_of_comp_eq_idMap jPlus_comp_iPlus

/-- Every neighbourhood of `D_a := fixedNbhd a` is non-empty (inherited from `?`). -/
theorem fixedNbhd_mem_nonempty (a : ApproximableMap U U) :
    forall  X, (fixedNbhd a).mem X  ->  X.Nonempty :=
  fun X hX => U_mem_nonempty X ((fixedNbhd_subsystem a).sub hX)

namespace sumComb

theorem eq (a b : ApproximableMap U U) :
    sumComb a b = (cond U).comp
      ((paired (whichMap U U U_mem_nonempty U_mem_nonempty)
          (paired (iPlus.comp ((inMap_0 (h_0 := U_mem_nonempty) (h_1 := U_mem_nonempty
    )).comp
                    (a.comp (outMap_0 (h_0 := U_mem_nonempty) (h_1 := U_mem_nonempty))
    )))
                  (iPlus.comp ((inMap_1 (h_0 := U_mem_nonempty) (h_1 := U_mem_nonempty
    )).comp
                    (b.comp (outMap_1 (h_0 := U_mem_nonempty) (h_1 := U_mem_nonempty))
    ))))).comp
        jPlus) := rfl

/-- **Elementwise unfolding of `sumComb`.** -/
theorem toElementMap_mem (a b : ApproximableMap U U) (x : U.Element) {Z : Set ?} :
    ((sumComb a b).toElementMap x).mem Z  <->
      (((whichMap U U U_mem_nonempty U_mem_nonempty).toElementMap (jPlus.toElementMap
    x)).mem
          Example12.zero  /\
        (iPlus.toElementMap ((inMap_0 (h_0 := U_mem_nonempty) (h_1 := U_mem_nonempty))
    .toElementMap
          (a.toElementMap ((outMap_0 (h_0 := U_mem_nonempty) (h_1 := U_mem_nonempty)).
    toElementMap
            (jPlus.toElementMap x))))).mem Z)  \/
      (((whichMap U U U_mem_nonempty U_mem_nonempty).toElementMap (jPlus.toElementMap
    x)).mem
          Example12.one  /\
        (iPlus.toElementMap ((inMap_1 (h_0 := U_mem_nonempty) (h_1 := U_mem_nonempty))
    .toElementMap
```

```
          (b.toElementMap ((outMap_1 (h_0 := U_mem_nonempty) (h_1 := U_mem_nonempty)).
    toElementMap
            (jPlus.toElementMap x))))).mem Z)  \/
      Z = U.master := by
  rw [eq]
  simp only [toElementMap_comp, toElementMap_paired]
  rw [cond_toElementMap_mem]

/-- **On a "left" point**, `sumComb a b` evaluates to Scott's `i_+ comp in_0 comp a
    comp out_0` formula. -/
theorem toElementMap_of_left {a b : ApproximableMap U U} {x : U.Element}
    (hL : exists  X, U.mem X  /\  (jPlus.toElementMap x).mem (inj_0 X)) :
    (sumComb a b).toElementMap x =
      iPlus.toElementMap ((inMap_0 (h_0 := U_mem_nonempty) (h_1 := U_mem_nonempty)).
    toElementMap
        (a.toElementMap ((outMap_0 (h_0 := U_mem_nonempty) (h_1 := U_mem_nonempty)).
    toElementMap
          (jPlus.toElementMap x)))) := by
  have hz : ((whichMap U U U_mem_nonempty U_mem_nonempty).toElementMap
      (jPlus.toElementMap x)).mem Example12.zero := which_mem_zero.mpr hL
  have hno : not  ((whichMap U U U_mem_nonempty U_mem_nonempty).toElementMap
      (jPlus.toElementMap x)).mem Example12.one :=
    fun h => not_sum_reaches_both hL (which_mem_one.mp h)
  apply Element.ext
  intro Z
  rw [toElementMap_mem]
  constructor
  * rintro (<_, h> | <h1, _> | rfl)
    * exact h
    * exact absurd h1 hno
    * exact Element.master_mem _
  * intro h
    exact Or.inl <hz, h>

/-- **On a point reaching neither copy**, `sumComb a b` sends it to `?`'s `False`. -/
theorem toElementMap_of_neither {a b : ApproximableMap U U} {x : U.Element}
    (hN : forall  W, (jPlus.toElementMap x).mem W  ->  W = (sum U U U_mem_nonempty
    U_mem_nonempty).master) :
    (sumComb a b).toElementMap x = U.bot := by
  have hnz : not  ((whichMap U U U_mem_nonempty U_mem_nonempty).toElementMap
      (jPlus.toElementMap x)).mem Example12.zero := by
    rw [which_mem_zero]
    rintro <X, _, hX>
    exact inj_0_ne_sumMaster (hN _ hX)
  have hno : not  ((whichMap U U U_mem_nonempty U_mem_nonempty).toElementMap
      (jPlus.toElementMap x)).mem Example12.one := by
    rw [which_mem_one]
    rintro <Y, _, hY>
    exact inj_1_ne_sumMaster (hN _ hY)
  apply Element.ext
  intro Z
  rw [toElementMap_mem, mem_bot]
  constructor
  * rintro (<h, _> | <h, _> | h)
    * exact absurd h hnz
    * exact absurd h hno
```

```
    * exact h
  * intro h
    exact Or.inr (Or.inr h)

/-- **On a "right" point**, symmetric to `toElementMap_of_left`. -/
theorem toElementMap_of_right {a b : ApproximableMap U U} {x : U.Element}
    (hR : exists  Y, U.mem Y  /\  (jPlus.toElementMap x).mem (inj_1 Y)) :
    (sumComb a b).toElementMap x =
      iPlus.toElementMap ((inMap_1 (h_0 := U_mem_nonempty) (h_1 := U_mem_nonempty)).
    toElementMap
        (b.toElementMap ((outMap_1 (h_0 := U_mem_nonempty) (h_1 := U_mem_nonempty)).
    toElementMap
          (jPlus.toElementMap x)))) := by
  have ho : ((whichMap U U U_mem_nonempty U_mem_nonempty).toElementMap
      (jPlus.toElementMap x)).mem Example12.one := which_mem_one.mpr hR
  have hnz : not  ((whichMap U U U_mem_nonempty U_mem_nonempty).toElementMap
      (jPlus.toElementMap x)).mem Example12.zero :=
    fun h => not_sum_reaches_both (which_mem_zero.mp h) hR
  apply Element.ext
  intro Z
  rw [toElementMap_mem]
  constructor
  * rintro (<h1, _> | <_, h> | rfl)
    * exact absurd h1 hnz
    * exact h
    * exact Element.master_mem _
  * intro h
    exact Or.inr (Or.inl <ho, h>)

/-- **Proposition 8.10, `+`-case, retraction half.** -/
theorem isRetraction {a b : ApproximableMap U U} (ha : Retraction a)
    (hb : Retraction b) : Retraction (sumComb a b) := by
  constructor
  apply ext_of_toElementMap
  intro x
  rw [toElementMap_comp]
  rcases sum_element_trichotomy (jPlus.toElementMap x) with hL | hR | hN
  * rw [toElementMap_of_left hL]
    set v := a.toElementMap (outMap_0.toElementMap (jPlus.toElementMap x)) with hv
    have hjy : jPlus.toElementMap (iPlus.toElementMap (inMap_0.toElementMap v))
        = inMap_0.toElementMap v := toElementMap_of_comp_eq_idMap jPlus_comp_iPlus _
    have hLy : exists  X, U.mem X  /\
        (jPlus.toElementMap (iPlus.toElementMap (inMap_0.toElementMap v))).mem (inj_0
    X) := by
      rw [hjy]; exact inMap_0_toElementMap_reaches_left v
    rw [toElementMap_of_left hLy, hjy,
      toElementMap_of_comp_eq_idMap outMap_0_comp_inMap_0 v,
    toElementMap_idem_of_isRetraction ha]
  * rw [toElementMap_of_right hR]
    set v := b.toElementMap (outMap_1.toElementMap (jPlus.toElementMap x)) with hv
    have hjy : jPlus.toElementMap (iPlus.toElementMap (inMap_1.toElementMap v))
        = inMap_1.toElementMap v := toElementMap_of_comp_eq_idMap jPlus_comp_iPlus _
    have hRy : exists  Y, U.mem Y  /\
        (jPlus.toElementMap (iPlus.toElementMap (inMap_1.toElementMap v))).mem (inj_1
    Y) := by
      rw [hjy]; exact inMap_1_toElementMap_reaches_right v
```

```
    rw [toElementMap_of_right hRy, hjy,
      toElementMap_of_comp_eq_idMap outMap_1_comp_inMap_1 v,
    toElementMap_idem_of_isRetraction hb]
  * rw [toElementMap_of_neither hN]
    apply toElementMap_of_neither
    intro W hW
    rw [jPlus_bot_eq_bot, mem_bot] at hW
    exact hW

/-- **Proposition 8.10, `+`-case, projection half.** -/
theorem le_idMap {a b : ApproximableMap U U} (ha : a  <=  idMap U) (hb : b  <=  idMap
    U) :
    sumComb a b  <=  idMap U := by
  rw [le_iff_toElementMap_le]
  intro x
  rw [toElementMap_idMap]
  rcases sum_element_trichotomy (jPlus.toElementMap x) with hL | hR | hN
  * calc (sumComb a b).toElementMap x
        = iPlus.toElementMap ((inMap_0 (h_0 := U_mem_nonempty) (h_1 := U_mem_nonempty)
    ).toElementMap
            (a.toElementMap ((outMap_0 (h_0 := U_mem_nonempty) (h_1 := U_mem_nonempty)
    ).toElementMap
              (jPlus.toElementMap x)))) := toElementMap_of_left hL
      _  <=  iPlus.toElementMap ((inMap_0 (h_0 := U_mem_nonempty) (h_1 :=
    U_mem_nonempty)).toElementMap
            ((outMap_0 (h_0 := U_mem_nonempty) (h_1 := U_mem_nonempty)).toElementMap
              (jPlus.toElementMap x))) :=
          iPlus.toElementMap_mono (inMap_0.toElementMap_mono (
    toElementMap_le_self_of_le_idMap ha _))
      _ = iPlus.toElementMap (jPlus.toElementMap x) := by rw [
    inMap_0_outMap_0_eq_of_left hL]
      _  <=  x := toElementMap_le_of_comp_le_idMap iPlus_comp_jPlus_le x
  * calc (sumComb a b).toElementMap x
        = iPlus.toElementMap ((inMap_1 (h_0 := U_mem_nonempty) (h_1 := U_mem_nonempty)
    ).toElementMap
            (b.toElementMap ((outMap_1 (h_0 := U_mem_nonempty) (h_1 := U_mem_nonempty)
    ).toElementMap
              (jPlus.toElementMap x)))) := toElementMap_of_right hR
      _  <=  iPlus.toElementMap ((inMap_1 (h_0 := U_mem_nonempty) (h_1 :=
    U_mem_nonempty)).toElementMap
            ((outMap_1 (h_0 := U_mem_nonempty) (h_1 := U_mem_nonempty)).toElementMap
              (jPlus.toElementMap x))) :=
          iPlus.toElementMap_mono (inMap_1.toElementMap_mono (
    toElementMap_le_self_of_le_idMap hb _))
      _ = iPlus.toElementMap (jPlus.toElementMap x) := by rw [
    inMap_1_outMap_1_eq_of_right hR]
      _  <=  x := toElementMap_le_of_comp_le_idMap iPlus_comp_jPlus_le x
  * rw [toElementMap_of_neither hN]
    exact U.bot_le x

/-- **Proposition 8.10 (Scott 1981, PRG-19), `+`-case.** -/
theorem isProjection {a b : ApproximableMap U U}
    (ha : IsProjection a) (hb : IsProjection b) : IsProjection (sumComb a b) :=
  <isRetraction ha.1 hb.1, le_idMap ha.2 hb.2>

/-- **`I_+ : D_a + D_b  ->  ?`**. -/
```

```
noncomputable def I (a b : ApproximableMap U U) :
    ApproximableMap (sum (fixedNbhd a) (fixedNbhd b)
      (fixedNbhd_mem_nonempty a) (fixedNbhd_mem_nonempty b)) U :=
  iPlus.comp (sumMap (h_0' := U_mem_nonempty) (h_1' := U_mem_nonempty)
    (fixedNbhd_subsystem a).inj (fixedNbhd_subsystem b).inj)

/-- **`J_+ : ?  ->  D_a + D_b`**. -/
noncomputable def J (a b : ApproximableMap U U) :
    ApproximableMap U (sum (fixedNbhd a) (fixedNbhd b)
      (fixedNbhd_mem_nonempty a) (fixedNbhd_mem_nonempty b)) :=
  (sumMap (h_0 := U_mem_nonempty) (h_1 := U_mem_nonempty)
    (fixedNbhd_subsystem a).proj (fixedNbhd_subsystem b).proj).comp jPlus

end sumComb

end DomainTheory.NeighbourhoodSystems
```

— `prodComb` and Pref-folded companions

`prodComb.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Definition_8_9

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap Exercise326

/-- **`a  x  b : ?  ->  ?`** (Definition 8.9): `i_ x   comp  <a comp proj_0, b comp
    proj_1>  comp  j_ x `. -/
noncomputable def prodComb (a b : ApproximableMap U U) : ApproximableMap U U :=
  iTimes.comp ((paired (a.comp (proj_0 U U)) (b.comp (proj_1 U U))).comp jTimes)

namespace prodComb

theorem eq (a b : ApproximableMap U U) :
    prodComb a b = iTimes.comp ((prodMap a b).comp jTimes) := rfl

theorem toElementMap (a b : ApproximableMap U U) (x : U.Element) :
    (prodComb a b).toElementMap x =
      iTimes.toElementMap (pair (a.toElementMap (jTimes.toElementMap x).fst)
        (b.toElementMap (jTimes.toElementMap x).snd)) := by
  rw [eq]
  simp only [toElementMap_comp, toElementMap_prodMap]

/-- **Proposition 8.10, ` x `-case, retraction half.** -/
theorem isRetraction {a b : ApproximableMap U U} (ha : Retraction a)
    (hb : Retraction b) : Retraction (prodComb a b) := by
  constructor
  apply ext_of_toElementMap
  intro x
  rw [toElementMap_comp, toElementMap a b ((prodComb a b).toElementMap x),
```

```
    toElementMap a b x, toElementMap_of_comp_eq_idMap jTimes_comp_iTimes, fst_pair,
    snd_pair, toElementMap_idem_of_isRetraction ha, toElementMap_idem_of_isRetraction
    hb]

/-- **Proposition 8.10, ` x `-case, projection half.** -/
theorem le_idMap {a b : ApproximableMap U U} (ha : a  <=  idMap U) (hb : b  <=  idMap
    U) :
    prodComb a b  <=  idMap U := by
  rw [le_iff_toElementMap_le]
  intro x
  rw [toElementMap, toElementMap_idMap]
  have hpair : pair (a.toElementMap (jTimes.toElementMap x).fst)
      (b.toElementMap (jTimes.toElementMap x).snd)
       <=  pair (jTimes.toElementMap x).fst (jTimes.toElementMap x).snd :=
    pair_le_pair_iff.mpr
      <toElementMap_le_self_of_le_idMap ha _, toElementMap_le_self_of_le_idMap hb _>
  rw [pair_fst_snd] at hpair
  calc iTimes.toElementMap (pair (a.toElementMap (jTimes.toElementMap x).fst)
        (b.toElementMap (jTimes.toElementMap x).snd))
       <=  iTimes.toElementMap (jTimes.toElementMap x) := iTimes.toElementMap_mono
    hpair
    _  <=  x := toElementMap_le_of_comp_le_idMap iTimes_comp_jTimes_le x

/-- **Proposition 8.10 (Scott 1981, PRG-19), ` x `-case.** -/
theorem isProjection {a b : ApproximableMap U U}
    (ha : IsProjection a) (hb : IsProjection b) : IsProjection (prodComb a b) :=
  <isRetraction ha.1 hb.1, le_idMap ha.2 hb.2>

/-- **`I_ x  : D_a  x  D_b  ->  ?`**. -/
noncomputable def I (a b : ApproximableMap U U) :
    ApproximableMap (prod (fixedNbhd a) (fixedNbhd b)) U :=
  iTimes.comp (prodMap (fixedNbhd_subsystem a).inj (fixedNbhd_subsystem b).inj)

/-- **`J_ x  : ?  ->  D_a  x  D_b`**. -/
noncomputable def J (a b : ApproximableMap U U) :
    ApproximableMap U (prod (fixedNbhd a) (fixedNbhd b)) :=
  (prodMap (fixedNbhd_subsystem a).proj (fixedNbhd_subsystem b).proj).comp jTimes

theorem J_comp_I (a b : ApproximableMap U U) :
    (J a b).comp (I a b) = idMap (prod (fixedNbhd a) (fixedNbhd b)) := by
  unfold J I
  rw [comp_assoc,  <-  comp_assoc jTimes iTimes (prodMap _ _), jTimes_comp_iTimes,
    idMap_comp,
     <-  prodMap_comp, (fixedNbhd_subsystem a).proj_comp_inj, (fixedNbhd_subsystem b).
    proj_comp_inj,
    prodMap_id]

end prodComb

end DomainTheory.NeighbourhoodSystems
```

— `arrowComb` and Pref-folded companions

`arrowComb.lean`

```
module
```

```
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Definition_8_9
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.expMap

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap Exercise326

/-- **`a  ->  b : ?  ->  ?`** (Definition 8.9): `i_ ->   comp  (fun f. b comp f comp a)
      comp  j_ -> `. -/
noncomputable def arrowComb (a b : ApproximableMap U U) : ApproximableMap U U :=
  iArrow.comp ((lamComb a b).comp jArrow)

namespace arrowComb

theorem eq (a b : ApproximableMap U U) :
    arrowComb a b = iArrow.comp ((lamComb a b).comp jArrow) := rfl

theorem toElementMap (a b : ApproximableMap U U) (x : U.Element) :
    (arrowComb a b).toElementMap x =
      iArrow.toElementMap ((lamComb a b).toElementMap (jArrow.toElementMap x)) := by
  rw [eq]
  simp only [toElementMap_comp]

/-- **Proposition 8.10, ` -> `-case, retraction half.** -/
theorem isRetraction {a b : ApproximableMap U U} (ha : Retraction a)
    (hb : Retraction b) : Retraction (arrowComb a b) := by
  constructor
  apply ext_of_toElementMap
  intro x
  rw [toElementMap_comp, toElementMap a b ((arrowComb a b).toElementMap x),
    toElementMap a b x, toElementMap_of_comp_eq_idMap jArrow_comp_iArrow,
    toElementMap_idem_of_isRetraction (isRetraction_lamComb ha hb)]

/-- **Proposition 8.10, ` -> `-case, projection half.** -/
theorem le_idMap {a b : ApproximableMap U U} (ha : a  <=  idMap U) (hb : b  <=  idMap
    U) :
    arrowComb a b  <=  idMap U := by
  rw [le_iff_toElementMap_le]
  intro x
  rw [toElementMap, toElementMap_idMap]
  calc iArrow.toElementMap ((lamComb a b).toElementMap (jArrow.toElementMap x))
       <=  iArrow.toElementMap (jArrow.toElementMap x) :=
        iArrow.toElementMap_mono
          (toElementMap_le_self_of_le_idMap (le_idMap_lamComb ha hb) _)
    _  <=  x := toElementMap_le_of_comp_le_idMap iArrow_comp_jArrow_le x

/-- **Proposition 8.10 (Scott 1981, PRG-19), ` -> `-case.** -/
theorem isProjection {a b : ApproximableMap U U}
    (ha : IsProjection a) (hb : IsProjection b) : IsProjection (arrowComb a b) :=
  <isRetraction ha.1 hb.1, le_idMap ha.2 hb.2>

/-- **`I_ ->  : (D_a  ->  D_b)  ->  ?`**, built by transporting `i_ -> ` (Definition
```

```
    8.9) through the
functorial action of ` -> ` (`expMap`) on `j_a : ?  ->  D_a` (contravariant slot) and
`i_b : D_b  ->  ?` (covariant slot). -/
noncomputable def I (a b : ApproximableMap U U) :
    ApproximableMap (funSpace (fixedNbhd a) (fixedNbhd b)) U :=
  iArrow.comp (expMap (fixedNbhd_subsystem a).proj (fixedNbhd_subsystem b).inj)

/-- **`J_ ->  : ?  ->  (D_a  ->  D_b)`**, symmetric to `I`, via `i_a` (contravariant)
    and
`j_b` (covariant). -/
noncomputable def J (a b : ApproximableMap U U) :
    ApproximableMap U (funSpace (fixedNbhd a) (fixedNbhd b)) :=
  (expMap (fixedNbhd_subsystem a).inj (fixedNbhd_subsystem b).proj).comp jArrow

theorem J_comp_I (a b : ApproximableMap U U) :
    (J a b).comp (I a b) = idMap (funSpace (fixedNbhd a) (fixedNbhd b)) := by
  unfold J I
  rw [comp_assoc,  <-  comp_assoc jArrow iArrow (expMap _ _), jArrow_comp_iArrow,
    idMap_comp,
     <-  expMap.comp, (fixedNbhd_subsystem a).proj_comp_inj, (fixedNbhd_subsystem b).
    proj_comp_inj,
    expMap.id]

end arrowComb

end DomainTheory.NeighbourhoodSystems
```

— `expMap` and Pref-folded companions

`expMap.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Definition_8_9

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap Exercise326

/-- **The Hom-functor's action on maps**, `f  |->  k comp f comp h`, transported to the
     function-space
neighbourhood systems via `curry`/`evalMap`/`prodMap`, exactly as `lamComb` does for
    the
self-map case. -/
noncomputable def expMap {? ? ?' ?' : Type*} {V0 : NeighborhoodSystem ?} {V1 :
    NeighborhoodSystem ?}
    {V0' : NeighborhoodSystem ?'} {V1' : NeighborhoodSystem ?'}
    (h : ApproximableMap V0' V0) (k : ApproximableMap V1 V1') :
    ApproximableMap (funSpace V0 V1) (funSpace V0' V1') :=
  curry (k.comp ((evalMap V0 V1).comp (prodMap (idMap (funSpace V0 V1)) h)))

namespace expMap
```

```
variable {? ? ?' ?' ?'' ?'' : Type*}
variable {V0 : NeighborhoodSystem ?} {V1 : NeighborhoodSystem ?}
variable {V0' : NeighborhoodSystem ?'} {V1' : NeighborhoodSystem ?'}
variable {V0'' : NeighborhoodSystem ?''} {V1'' : NeighborhoodSystem ?''}

theorem eq (h : ApproximableMap V0' V0) (k : ApproximableMap V1 V1') :
    expMap h k = curry (k.comp ((evalMap V0 V1).comp (prodMap (idMap (funSpace V0 V1))
     h))) := rfl

/-- `expMap` specializes to `lamComb` on self-maps of `?`. -/
theorem eq_lamComb (a b : ApproximableMap U U) : expMap a b = lamComb a b := rfl

/-- **`expMap h k`, transported through `funSpaceEquiv`, is exactly `f  |->  k comp f
    comp h`.** -/
theorem toApproxMap_toElementMap (h : ApproximableMap V0' V0) (k : ApproximableMap V1
    V1')
     (phi : (funSpace V0 V1).Element) :
    toApproxMap ((expMap h k).toElementMap phi) = k.comp ((toApproxMap phi).comp h) :=
     by
  apply ext_of_toElementMap
  intro z
  rw [eq, toElementMap_curry_apply, toElementMap_comp, toElementMap_comp,
    toElementMap_prodMap_pair, toElementMap_idMap, evalMap_apply, toElementMap_comp,
    toElementMap_comp]

/-- **`expMap` preserves composition, contravariantly in `h`.** -/
theorem comp (h : ApproximableMap V0' V0) (k : ApproximableMap V1 V1')
    (h' : ApproximableMap V0'' V0') (k' : ApproximableMap V1' V1'') :
    expMap (h.comp h') (k'.comp k) = (expMap h' k').comp (expMap h k) := by
  apply ext_of_toElementMap
  intro phi
  apply (funSpaceEquiv V0'' V1'').injective
  rw [funSpaceEquiv_apply, funSpaceEquiv_apply, toApproxMap_toElementMap,
    toElementMap_comp,
    toApproxMap_toElementMap, toApproxMap_toElementMap]
  simp only [comp_assoc]

/-- **`expMap` preserves identities: `expMap I I = I`.** -/
theorem id : expMap (idMap V0) (idMap V1) = idMap (funSpace V0 V1) := by
  apply ext_of_toElementMap
  intro phi
  apply (funSpaceEquiv V0 V1).injective
  rw [funSpaceEquiv_apply, funSpaceEquiv_apply, toElementMap_idMap,
    toApproxMap_toElementMap, comp_idMap, idMap_comp]

/-- **`expMap` is monotone in its contravariant argument** (`k` fixed). -/
theorem mono_left {h h' : ApproximableMap V0' V0} (hh : h  <=  h') (k :
    ApproximableMap V1 V1') :
    expMap h k  <=  expMap h' k := by
  rw [le_iff_toElementMap_le]
  intro phi
  apply (funSpaceEquiv V0' V1').map_rel_iff.mp
  rw [funSpaceEquiv_apply, funSpaceEquiv_apply, toApproxMap_toElementMap,
    toApproxMap_toElementMap]
  exact comp_mono_gen le_rfl (comp_mono_gen le_rfl hh)
```

```
end expMap

end DomainTheory.NeighbourhoodSystems
```

3.9.11 Proposition 8.10 — constructs preserve projections

PROPOSITION 8.10. If $a, b : \mathcal{U} \to \mathcal{U}$ are projections, then so are $a + b$, $a \times b$, and $a \to b$. If a and b are finitary, then so are the others; for the fixed-point set of each of them is isomorphic to the corresponding construct applied to the domains determined by a and b.

Proof: Suppose that $a, b \sqsubseteq I_{\mathcal{U}}$ (= I for short). Then

$$a + b \sqsubseteq I + I = i_+ \circ j_+ \sqsubseteq I.$$

The other cases are similar.

Suppose $a = a \circ a$ and $b = b \circ b$, then, for example,

$$\begin{aligned}(a \times b) \circ (a \times b) &= i_\times \circ \langle a \circ \mathrm{proj}_0,\ b \circ \mathrm{proj}_1 \rangle \circ \langle a \circ \mathrm{proj}_0,\ b \circ \mathrm{proj}_1 \rangle \circ j_\times \\ &= i_\times \circ \langle a \circ a \circ \mathrm{proj}_0,\ b \circ b \circ \mathrm{proj}_1 \rangle \circ j_\times \\ &= a \times b.\end{aligned}$$

The other cases are similar.

Now in case the fixed-point sets of a and b are domains, they are respectively isomorphic to

$$D_a = \{X \in \mathcal{U} \mid X\ a\ X\} \quad \text{and}$$
$$D_b = \{Y \in \mathcal{U} \mid Y\ b\ Y\}.$$

We have to show, for example, that

$$D_a \to D_b \cong D_{a \to b}.$$

Now to simplify matters, remark that the fixed-point set of $a \to b$ on $\mathcal{U}$ is isomorphic to the fixed-point set of $\lambda f.\ b \circ f \circ a$ on $(\mathcal{U} \to \mathcal{U})$. (Hint: use $i_\to$ and $j_\to$ to set up the isomorphism.) So we have to think what it is for an $f : \mathcal{U} \to \mathcal{U}$ to satisfy

$$f = b \circ f \circ a.$$

Notice that we might as well say that $a : \mathcal{U} \to D_a$ and that this map is the other half of an obvious projection pair where

$$i_a : D_a \to \mathcal{U},$$

and $i_a \circ a = a$ and $a \circ i_a = i_a$. So if $g : D_a \to D_b$, let

$$f = i_b \circ g \circ a;$$

then $b \circ f \circ a = f$. Conversely, if f is like this, then let

$$g = b \circ f \circ i_a.$$

Thus, $i_b \circ g \circ a = b \circ f \circ a = f$; so there is an order-preserving isomorphism between the $g : D_a \to D_b$ and the $f = b \circ f \circ a$.

The isomorphism proofs for $+$ and $\times$ are similar. $\square$

Well, this was a lot of work, but the pay-off is rather handsome. What we have done is transpose all the

$$D_a \lhd U$$

over to finitary projections $a : U \to U$. This transposition is an isomorphism, because

$$D_a \lhd D_b \text{ iff } a \sqsubseteq b.$$

Moreover, by the method of 8.9 and 8.10, all our favourite constructs have been made into *combinators*, that is, approximable — even computable — maps on the domain of finitary projections. *ALL APPROXIMABLE (COMPUTABLE) MAPS HAVE (COMPUTABLE) FIXED POINTS.* And there you are! The standard fixed-point method is available to obtain computable (i.e. effectively given) solutions to *all* domain equations (even sets of equations) where the constructs can be reworked in *this* way to be defined on projections. Examples are suggested in the exercises.

Another pay-off concerns the λ-calculus itself. Inasmuch as

$$U + U, \quad U \times U, \quad U \to U \lhd U,$$

we might just as well forget the outside world and regard all these useful domains as being part of U. For example, on the left we have the new notation and on the right the old notation:

$$\begin{aligned}
\mathrm{which}(z) &= \mathrm{which}(j_+(z)); \\
\mathrm{in}_i(x) &= i_+(\mathrm{in}_i(x)), \quad i = 0, 1; \\
\mathrm{out}_i(x) &= \mathrm{out}_i(j_+(x)), \quad i = 0, 1; \\
\langle x, y \rangle &= i_\times(\langle x, y \rangle); \\
\mathrm{proj}_i(z) &= \mathrm{proj}_i(j_\times(z)), \quad i = 0, 1; \\
u(x) &= j_\to(u)(x); \\
\lambda x.\, \tau &\cong i_\to(\lambda x.\, \tau).
\end{aligned}$$

And, there is no reason to stop here. The system

$$T \cong \{\{0, 1/2\}, \{1/2, 1\}, \{0, 1\}\} \lhd U,$$

so we might as well think of

$$\text{true, false} \in |U|$$

and think of $\mathrm{cond} : U \times U \times U \to U$. No! that is wrong: under the new regime *EVERYTHING IS AN ELEMENT OF U*. With the new meaning of λ, all functions, all pairs, all combinators, all constructs become elements of U.

It takes a little time to get used to "universal conscription" with all elements doing (at least) double duty in the same domain, but there are many advantages, both notational and conceptual.

`Proposition_8_10.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Definition_8_9
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.sumComb
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.prodComb
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.arrowComb
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.sumMap
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.expMap

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_13

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap Exercise326
universe u
variable {a b : Type u} {D : NeighborhoodSystem a} {E : NeighborhoodSystem b}

/- concordance: source L5040 PROPOSITION 8.10 constructs preserve projections -/
/-- **Proposition 8.10 (Scott 1981, PRG-19), first half.** If `a, b : ?  ->  ?` are
    projections,
then so are `a+b`, `a x b`, and `a -> b`. -/
theorem isProjection_combinators {a b : ApproximableMap U U}
    (ha : IsProjection a) (hb : IsProjection b) :
    IsProjection (sumComb a b)  /\  IsProjection (prodComb a b)  /\  IsProjection (
    arrowComb a b) :=
  <sumComb.isProjection ha hb, prodComb.isProjection ha hb, arrowComb.isProjection ha
    hb>

/-- **Generalized Proposition 8.2 `elementIso`.** Given an approximable projection pair
`i : D  ->  E`, `j : E  ->  D` with `j  comp  i = I_D` (no requirement `i  comp  j  <=
    I_E`, and `D`, `E` need not
share a token type -- unlike `Subsystem.elementIso`, which requires a literal `D ? E`),
    and
`g : E  ->  E` with `g = i  comp  j`, then `D.Element ?o Fix(g)`. The proof is verbatim
`Subsystem.elementIso`'s, generalized. -/
def elementIsoOfProjectionPair (i : ApproximableMap D E) (j : ApproximableMap E D)
    (hji : j.comp i = idMap D) {g : ApproximableMap E E} (hg : g = i.comp j) :
    D.Element ?o {y : E.Element // g.toElementMap y = y} where
  toFun x := <i.toElementMap x, by
    have h1 : j.toElementMap (i.toElementMap x) = (j.comp i).toElementMap x :=
      (toElementMap_comp j i x).symm
    rw [hg, toElementMap_comp, h1, hji, toElementMap_idMap]>
  invFun y := j.toElementMap y.1
  left_inv x := by
    have h1 : j.toElementMap (i.toElementMap x) = (j.comp i).toElementMap x :=
      (toElementMap_comp j i x).symm
    show j.toElementMap (i.toElementMap x) = x
    rw [h1, hji, toElementMap_idMap]
  right_inv y := by
    apply Subtype.ext
    have hfix : g.toElementMap y.1 = y.1 := y.2
```

```
    show i.toElementMap (j.toElementMap y.1) = y.1
    rw [ <-  toElementMap_comp,  <-  hg]
    exact hfix
  map_rel_iff' := by
    intro x x'
    show i.toElementMap x  <=  i.toElementMap x'  <->  x  <=  x'
    constructor
    * intro hle
      have hmono := j.toElementMap_mono hle
      have h1 : j.toElementMap (i.toElementMap x) = (j.comp i).toElementMap x :=
        (toElementMap_comp j i x).symm
      have h2 : j.toElementMap (i.toElementMap x') = (j.comp i).toElementMap x' :=
        (toElementMap_comp j i x').symm
      rw [h1, h2, hji, toElementMap_idMap, toElementMap_idMap] at hmono
      exact hmono
    * intro hle
      exact i.toElementMap_mono hle

/-- **`i_a  comp  j_a = a`, for a finitary projection `a`.** Theorem 8.6's `
    sub_eq_self_of_isFinitaryProjection`,
restated via `retractionOfSubsystem`'s definition. -/
theorem inj_comp_proj_eq_self {a : ApproximableMap U U} (ha : FinitaryProjection a) :
    (fixedNbhd_subsystem a).inj.comp (fixedNbhd_subsystem a).proj = a :=
  sub_eq_self_of_isFinitaryProjection ha

/-- If `i  comp  j  <=  I_E`, then `i` sends `D`'s bottom to `E`'s bottom. -/
theorem toElementMap_bot_eq_bot_of_comp_le_idMap {i : ApproximableMap D E} {j :
    ApproximableMap E D}
    (hij : i.comp j  <=  idMap E) : i.toElementMap D.bot = E.bot := by
  apply le_antisymm
  * calc i.toElementMap D.bot  <=  i.toElementMap (j.toElementMap E.bot) :=
          i.toElementMap_mono (D.bot_le _)
      _  <=  E.bot := toElementMap_le_of_comp_le_idMap hij E.bot
  * exact E.bot_le _

/-- **`IsFinitary` from a concrete projection-pair witness.** -/
theorem isFinitary_of_projectionPair (i : ApproximableMap D E) (j : ApproximableMap E
    D)
    (hji : j.comp i = idMap D) {g : ApproximableMap E E} (hg : g = i.comp j) :
    IsFinitary g :=
  <a, D, <(elementIsoOfProjectionPair i j hji hg).symm>>

namespace prodComb

theorem I_comp_J (a b : ApproximableMap U U) (ha : FinitaryProjection a) (hb :
    FinitaryProjection b) :
    (prodComb.I a b).comp (prodComb.J a b) = prodComb a b := by
  unfold I J
  rw [comp_assoc,  <-  comp_assoc (prodMap _ _) (prodMap _ _) jTimes,  <-
    prodMap_comp,
    inj_comp_proj_eq_self ha, inj_comp_proj_eq_self hb,  <-  comp_assoc, prodComb.eq,
    comp_assoc]

/-- **Proposition 8.10(b), ` x `-case, the isomorphism `D_{a x b}  ~=  D_a  x  D_b`.**
    -/
noncomputable def elementIso (a b : ApproximableMap U U)
```

```
    (ha : FinitaryProjection a) (hb : FinitaryProjection b) :
    (prod (fixedNbhd a) (fixedNbhd b)).Element ?o
      {y : U.Element // (prodComb a b).toElementMap y = y} :=
  elementIsoOfProjectionPair (prodComb.I a b) (prodComb.J a b) (prodComb.J_comp_I a b)
    (prodComb.I_comp_J a b ha hb).symm

theorem finitary (a b : ApproximableMap U U) (ha : FinitaryProjection a) (hb :
    FinitaryProjection b) :
    IsFinitary (prodComb a b) :=
  isFinitary_of_projectionPair (prodComb.I a b) (prodComb.J a b) (prodComb.J_comp_I a
    b)
    (prodComb.I_comp_J a b ha hb).symm

theorem finitaryProjection (a b : ApproximableMap U U)
    (ha : FinitaryProjection a) (hb : FinitaryProjection b) :
    FinitaryProjection (prodComb a b) :=
  <<prodComb.isProjection ha.out.1 hb.out.1, prodComb.finitary a b ha hb>>

end prodComb

namespace arrowComb

theorem I_comp_J (a b : ApproximableMap U U) (ha : FinitaryProjection a) (hb :
    FinitaryProjection b) :
    (arrowComb.I a b).comp (arrowComb.J a b) = arrowComb a b := by
  unfold I J
  rw [comp_assoc,  <-  comp_assoc (expMap _ _) (expMap _ _) jArrow,  <-  expMap.comp,
    inj_comp_proj_eq_self ha, inj_comp_proj_eq_self hb, expMap.eq_lamComb,  <-
    comp_assoc,
    arrowComb.eq, comp_assoc]

/-- **Proposition 8.10(b), ` -> `-case, the isomorphism `D_{a -> b}  ~=  (D_a  ->  D_b)
    `.** -/
noncomputable def elementIso (a b : ApproximableMap U U)
    (ha : FinitaryProjection a) (hb : FinitaryProjection b) :
    (funSpace (fixedNbhd a) (fixedNbhd b)).Element ?o
      {y : U.Element // (arrowComb a b).toElementMap y = y} :=
  elementIsoOfProjectionPair (arrowComb.I a b) (arrowComb.J a b) (arrowComb.J_comp_I a
    b)
    (arrowComb.I_comp_J a b ha hb).symm

theorem finitary (a b : ApproximableMap U U) (ha : FinitaryProjection a) (hb :
    FinitaryProjection b) :
    IsFinitary (arrowComb a b) :=
  isFinitary_of_projectionPair (arrowComb.I a b) (arrowComb.J a b) (arrowComb.J_comp_I
    a b)
    (arrowComb.I_comp_J a b ha hb).symm

theorem finitaryProjection (a b : ApproximableMap U U)
    (ha : FinitaryProjection a) (hb : FinitaryProjection b) :
    FinitaryProjection (arrowComb a b) :=
  <<arrowComb.isProjection ha.out.1 hb.out.1, arrowComb.finitary a b ha hb>>

end arrowComb

namespace sumComb
```

```
/-- **`sumComb a b = i_+  comp  (a+b)  comp  j_+`, at the level of approximable maps.**
     -/
theorem eq_iPlus_sumMap_jPlus (a b : ApproximableMap U U) :
    sumComb a b = iPlus.comp
      ((sumMap (h_0 := U_mem_nonempty) (h_1 := U_mem_nonempty) (h_0' := U_mem_nonempty
    )
          (h_1' := U_mem_nonempty) a b).comp jPlus) := by
  apply ext_of_toElementMap
  intro x
  rw [toElementMap_comp, toElementMap_comp]
  rcases sum_element_trichotomy (jPlus.toElementMap x) with hL | hR | hN
  * rw [sumComb.toElementMap_of_left hL, sumMap.toElementMap_of_left hL]
  * rw [sumComb.toElementMap_of_right hR, sumMap.toElementMap_of_right hR]
  * rw [sumComb.toElementMap_of_neither hN, sumMap.reaches_neither_iff_eq_bot.mp hN,
    sumMap.bot,
      toElementMap_bot_eq_bot_of_comp_le_idMap iPlus_comp_jPlus_le]

theorem I_comp_J (a b : ApproximableMap U U) (ha : FinitaryProjection a) (hb :
    FinitaryProjection b) :
    (sumComb.I a b).comp (sumComb.J a b) = sumComb a b := by
  unfold I J
  rw [comp_assoc,  <-  comp_assoc (sumMap _ _) (sumMap _ _) jPlus,  <-  sumMap.comp,
    inj_comp_proj_eq_self ha, inj_comp_proj_eq_self hb,  <-  comp_assoc,
    eq_iPlus_sumMap_jPlus, comp_assoc]

theorem J_comp_I (a b : ApproximableMap U U) :
    (sumComb.J a b).comp (sumComb.I a b) =
      idMap (sum (fixedNbhd a) (fixedNbhd b) (fixedNbhd_mem_nonempty a)
        (fixedNbhd_mem_nonempty b)) := by
  unfold J I
  rw [comp_assoc,  <-  comp_assoc jPlus iPlus (sumMap _ _), jPlus_comp_iPlus,
    idMap_comp,
     <-  sumMap.comp, (fixedNbhd_subsystem a).proj_comp_inj, (fixedNbhd_subsystem b).
    proj_comp_inj,
    sumMap.id]

/-- **Proposition 8.10(b), `+`-case, the isomorphism `D_{a+b}  ~=  D_a + D_b`.** -/
noncomputable def elementIso (a b : ApproximableMap U U)
    (ha : FinitaryProjection a) (hb : FinitaryProjection b) :
    (sum (fixedNbhd a) (fixedNbhd b) (fixedNbhd_mem_nonempty a) (
    fixedNbhd_mem_nonempty b)).Element
      ?o {y : U.Element // (sumComb a b).toElementMap y = y} :=
  elementIsoOfProjectionPair (sumComb.I a b) (sumComb.J a b) (sumComb.J_comp_I a b)
    (sumComb.I_comp_J a b ha hb).symm

theorem finitary (a b : ApproximableMap U U) (ha : FinitaryProjection a) (hb :
    FinitaryProjection b) :
    IsFinitary (sumComb a b) :=
  isFinitary_of_projectionPair (sumComb.I a b) (sumComb.J a b) (sumComb.J_comp_I a b)
    (sumComb.I_comp_J a b ha hb).symm

theorem finitaryProjection (a b : ApproximableMap U U)
    (ha : FinitaryProjection a) (hb : FinitaryProjection b) :
    FinitaryProjection (sumComb a b) :=
  <<sumComb.isProjection ha.out.1 hb.out.1, sumComb.finitary a b ha hb>>
```

```
end sumComb

/-- **Proposition 8.10(b) (Scott 1981, PRG-19), in full.** -/
theorem finitaryProjection_combinators {a b : ApproximableMap U U}
    (ha : FinitaryProjection a) (hb : FinitaryProjection b) :
    FinitaryProjection (sumComb a b)  /\  FinitaryProjection (prodComb a b)  /\
      FinitaryProjection (arrowComb a b) :=
  <sumComb.finitaryProjection a b ha hb, prodComb.finitaryProjection a b ha hb,
    arrowComb.finitaryProjection a b ha hb>

/-- **Proposition 8.10 (Scott 1981, PRG-19).** If `a, b : ?  ->  ?` are finitary
    projections, then
so are `a+b`, `a x b`, and `a -> b`, and `D_{a+b}  ~=  D_a + D_b`, `D_{a x b}  ~=  D_a
    x  D_b`,
`D_{a -> b}  ~=  (D_a  ->  D_b)`, where `D_a = fixedNbhd a` and `D_b = fixedNbhd b`. -/
theorem proposition_8_10 {a b : ApproximableMap U U}
    (ha : FinitaryProjection a) (hb : FinitaryProjection b) :
    FinitaryProjection (sumComb a b)  /\  FinitaryProjection (prodComb a b)  /\
      FinitaryProjection (arrowComb a b)  /\
      Nonempty
        ((sum (fixedNbhd a) (fixedNbhd b) (fixedNbhd_mem_nonempty a)
            (fixedNbhd_mem_nonempty b)).Element ?o
          {y : U.Element // (sumComb a b).toElementMap y = y})  /\
      Nonempty
        ((prod (fixedNbhd a) (fixedNbhd b)).Element ?o
          {y : U.Element // (prodComb a b).toElementMap y = y})  /\
      Nonempty
        ((funSpace (fixedNbhd a) (fixedNbhd b)).Element ?o
          {y : U.Element // (arrowComb a b).toElementMap y = y}) :=
  <sumComb.finitaryProjection a b ha hb, prodComb.finitaryProjection a b ha hb,
    arrowComb.finitaryProjection a b ha hb, <sumComb.elementIso a b ha hb>,
    <prodComb.elementIso a b ha hb>, <arrowComb.elementIso a b ha hb>>

end DomainTheory.NeighbourhoodSystems
```

3.9.12 Lecture VIII exercises

EXERCISES

3.9.13 Exercise 8.11 — retraction on intervals

Let $\mathbb{Q}$ be the set of rational numbers and define a neighbourhood system by the equation

$$R = \{[0, r) \mid r \in \mathbb{Q} \text{ and } 0 < r \leq 1\}.$$

Show that the following defines an approximable map $a : R \to R$:

$$[0, r)\ a\ [0, s) \quad \text{iff} \quad r < s \text{ or } r = s = 1.$$

Show in addition that a is a projection where the fixed-point set of a is in a one-one correspondence with the *real* numbers between 0 and 1 inclusive. (Hint: Recall Dedekind cuts and show $\subseteq$ matches $\leq$.) Conclude that a is *NOT* finitary. (Hint: Aside from $\perp$ there are no finite elements for $\{x \mid x = a(x)\}$.)

Tip summary The tip records that Scott's map `a` on `R` is a projection and is not finitary.

Development Builds the Dedekind-cut system `R` and proves the projection `a` is not finitary.

DedekindCutProjection.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Examples_8_4
public import Mathlib.Algebra.BigOperators.Group.Finset.Defs
public import Mathlib.Algebra.Order.Archimedean.Real.Basic

public import Mathlib.Combinatorics.Colex

@[expose] public section

/- concordance: source L5135 EXERCISES -- Lecture VIII exercises -/
namespace DomainTheory.NeighbourhoodSystems.Exercise811

open NeighborhoodSystem ApproximableMap

/-- **Exercise 8.11's `R`.** The neighbourhoods are exactly the half-open rational
    intervals
`[0,r)` for `0 < r  <=  1`, nested by `r` (`Ico0_subset_or_supset`), so `
    ofNestedOrDisjoint` applies
directly with no disjoint case ever needed. -/
def R : NeighborhoodSystem ? :=
  NeighborhoodSystem.ofNestedOrDisjoint
    (mem := fun X => exists  r : ?, 0 < r  /\  r  <=  1  /\  X = Set.Ico (0 : ?) r)
    (master := Set.Ico (0 : ?) 1)
    (master_nonempty := <0, by norm_num>)
    (master_mem := <1, one_pos, le_refl 1, rfl>)
    (hnd := by
      rintro X Y <r, hr0, hr1, rfl> <s, hs0, hs1, rfl>
      rcases le_total r s with h | h
      * exact Or.inl (Set.Ico_subset_Ico le_rfl h)
      * exact Or.inr (Or.inl (Set.Ico_subset_Ico le_rfl h)))
    (sub_master := by
      rintro X <r, hr0, hr1, rfl>
      exact Set.Ico_subset_Ico le_rfl hr1)

@[simp] theorem R_mem_iff {X : Set ?} :
    R.mem X  <->  exists  r : ?, 0 < r  /\  r  <=  1  /\  X = Set.Ico (0 : ?) r := Iff
    .rfl

@[simp] theorem R_master : R.master = Set.Ico (0 : ?) 1 := rfl

/-- The parametrization `r  |->  [0,r)` is injective on positive `r`: if `r < s` then `
    r` itself
witnesses the difference (`r  in  [0,s)` but `r  notin  [0,r)`). -/
theorem Ico0_inj {r s : ?} (hr : 0 < r) (hs : 0 < s)
    (h : Set.Ico (0 : ?) r = Set.Ico (0 : ?) s) : r = s := by
  rcases lt_trichotomy r s with hlt | heq | hgt
  * have hmem : r  in  Set.Ico (0 : ?) s := <hr.le, hlt>
    rw [ <-  h] at hmem
```

```
    exact absurd hmem.2 (lt_irrefl r)
  * exact heq
  * have hmem : s  in  Set.Ico (0 : ?) r := <hs.le, hgt>
    rw [h] at hmem
    exact absurd hmem.2 (lt_irrefl s)

/-- Two `[0,*)`-intervals intersect to a third, with the endpoint `min r s`. -/
theorem Ico0_inter (r s : ?) :
    Set.Ico (0 : ?) r  I  Set.Ico (0 : ?) s = Set.Ico (0 : ?) (min r s) := by
  rw [Set.Ico_inter_Ico, sup_idem]

/-- If `[0,p)  subseteq  [0,q)` and `p > 0`, then `p  <=  q`. (The converse direction
    of nestedness; note `q > 0`
is *derived*, from `0  in  [0,p)  subseteq  [0,q)`, not assumed.) -/
theorem Ico0_le_of_subset {p q : ?} (hp : 0 < p)
    (hsub : Set.Ico (0 : ?) p  subseteq  Set.Ico (0 : ?) q) : p  <=  q := by
  have hq : (0 : ?) < q := (hsub <le_refl 0, hp>).2
  by_contra hlt
  rw [not_le] at hlt
  exact absurd (hsub <hq.le, hlt>).2 (lt_irrefl q)

/-- Scott's relation: `[0,r) a [0,s)` iff `r < s` or `r = s = 1`. -/
def aRel (X Y : Set ?) : Prop :=
  exists  r s : ?, 0 < r  /\  r  <=  1  /\  0 < s  /\  s  <=  1  /\
    X = Set.Ico (0 : ?) r  /\  Y = Set.Ico (0 : ?) s  /\  (r < s  \/  (r = 1  /\  s =
    1))

/-- **Scott's combinator law**: if `X a Y` and `X a Y'` then `X a (Y  I  Y')`, unwound
    to endpoints. -/
theorem aRel_combine {r s s' : ?} (hs1 : s  <=  1) (hs1' : s'  <=  1)
    (h1 : r < s  \/  (r = 1  /\  s = 1)) (h2 : r < s'  \/  (r = 1  /\  s' = 1)) :
    r < min s s'  \/  (r = 1  /\  min s s' = 1) := by
  rcases h1 with h1 | <rfl, rfl>
  * rcases h2 with h2 | <rfl, rfl>
    * exact Or.inl (lt_min h1 h2)
    * exact absurd (h1.trans_le hs1) (lt_irrefl _)
  * rcases h2 with h2 | <-, rfl>
    * exact absurd (h2.trans_le hs1') (lt_irrefl _)
    * exact Or.inr <rfl, min_self 1>

/- concordance: source L5137 EXERCISE 8.11 retraction on intervals -/
/-- **Exercise 8.11 (first part).** `aRel` defines an approximable map `a : R  ->  R`.
    -/
def a : ApproximableMap R R where
  rel := aRel
  rel_dom := fun <r, _, hr0, hr1, _, _, hXr, _, _> => <r, hr0, hr1, hXr>
  rel_cod := fun <_, s, _, _, hs0, hs1, _, hYs, _> => <s, hs0, hs1, hYs>
  master_rel := <1, 1, one_pos, le_rfl, one_pos, le_rfl, rfl, rfl, Or.inr <rfl, rfl>>
  inter_right := by
    rintro X Y Y' <r, s, hr0, hr1, hs0, hs1, rfl, rfl, hcond>
      <r', s', hr0', hr1', hs0', hs1', hXr', rfl, hcond'>
    have hrr' : r = r' := Ico0_inj hr0 hr0' hXr'
    subst hrr'
    rw [Set.Ico_inter_Ico, sup_idem]
    exact <r, min s s', hr0, hr1, lt_min hs0 hs0', (min_le_left s s').trans hs1, rfl,
    rfl,
```

```
      aRel_combine hs1 hs1' hcond hcond'>
  mono := by
    rintro X X' Y Y' <r, s, hr0, hr1, hs0, hs1, rfl, rfl, hcond> hX'X hYY'
      <r', hr0', hr1', rfl> <s', hs0', hs1', rfl>
    have hr'r : r'  <=  r := Ico0_le_of_subset hr0' hX'X
    have hss' : s  <=  s' := Ico0_le_of_subset hs0 hYY'
    refine <r', s', hr0', hr1', hs0', hs1', rfl, rfl, ?_>
    rcases hcond with hlt | <rfl, rfl>
    * exact Or.inl (hr'r.trans_lt (hlt.trans_le hss'))
    * rcases hr1'.lt_or_eq with h | h
      * exact Or.inl (h.trans_le hss')
      * exact Or.inr <h, le_antisymm hs1' hss'>

/-- `aRel` unwound at concrete endpoints: exactly Scott's displayed condition. -/
theorem aRel_iff {r s : ?} (hr0 : 0 < r) (hr1 : r  <=  1) (hs0 : 0 < s) (hs1 : s  <=
    1) :
    aRel (Set.Ico (0 : ?) r) (Set.Ico (0 : ?) s)  <->  (r < s  \/  (r = 1  /\  s = 1))
     := by
  constructor
  * rintro <r', s', hr0', hr1', hs0', hs1', hXr, hXs, hcond>
    rw [Ico0_inj hr0 hr0' hXr, Ico0_inj hs0 hs0' hXs]
    exact hcond
  * intro h
    exact <r, s, hr0, hr1, hs0, hs1, rfl, rfl, h>

@[simp] theorem a_rel {X Y : Set ?} : a.rel X Y  <->  aRel X Y := Iff.rfl

/-- `R`'s elements are entirely determined by their membership at the endpoints `[0,r)
    `: two
elements agreeing on every `r` agree everywhere (any other `X` with `x.mem X` is forced
    , by
`x.sub`, to already be some `[0,r)`). -/
theorem Element.ext_U {x y : R.Element} (h : forall  r : ?, x.mem (Set.Ico 0 r)  <->
    y.mem (Set.Ico 0 r)) :
    x = y := by
  apply NeighborhoodSystem.Element.ext
  intro X
  constructor
  * intro hx
    obtain <r, _, _, rfl> := x.sub hx
    exact (h r).mp hx
  * intro hy
    obtain <r, _, _, rfl> := y.sub hy
    exact (h r).mpr hy

/-- **Building an `R.Element` from an "up-set" predicate `U`.** `U` must contain `1`,
    be bounded
to `(0,1]`, and be upward closed within `(0,1]` -- exactly Definition 1.6's filter
    conditions,
transported along the bijection `r  |->  [0,r)`. -/
def mkElement (U : ?  ->  Prop) (hU1 : U 1) (hUpos : forall  r, U r  ->  0 < r) (hUle1
     : forall  r, U r  ->  r  <=  1)
    (hUup : forall  r s, U r  ->  r  <=  s  ->  s  <=  1  ->  U s) : R.Element where
  mem X := exists  r, U r  /\  X = Set.Ico (0 : ?) r
  sub := fun <r, hr, hXr> => <r, hUpos r hr, hUle1 r hr, hXr>
  master_mem := <1, hU1, rfl>
```

```
  inter_mem := by
    rintro X Y <r, hr, rfl> <s, hs, rfl>
    rw [Ico0_inter]
    rcases le_total r s with h | h
    * exact <r, hr, by rw [min_eq_left h]>
    * exact <s, hs, by rw [min_eq_right h]>
  up_mem := by
    rintro X Y <r, hr, rfl> hY hsub
    obtain <s, hs0, hs1, rfl> := hY
    exact <s, hUup r s hr (Ico0_le_of_subset (hUpos r hr) hsub) hs1, rfl>

@[simp] theorem mkElement_mem {U hU1 hUpos hUle1 hUup} {X : Set ?} :
    (mkElement U hU1 hUpos hUle1 hUup).mem X  <->  exists  r, U r  /\  X = Set.Ico (0
    : ?) r := Iff.rfl

/-- Any `R`-neighbourhood `[0,s)` forces `0 < s  <=  1` (even though `R_mem_iff`
    phrases `R.mem X` as
an existential over a *possibly different* endpoint, injectivity pins it down to `s`
    itself). -/
theorem R_mem_Ico0 {s : ?} (h : R.mem (Set.Ico (0 : ?) s)) : 0 < s  /\  s  <=  1 := by
  obtain <r, hr0, hr1, heq> := h
  have hsne : (Set.Ico (0 : ?) s).Nonempty := heq  |>  <0, le_refl 0, hr0>
  obtain <x, hx0, hxs> := hsne
  have hs0 : 0 < s := lt_of_le_of_lt hx0 hxs
  have hsr : s = r := Ico0_inj hs0 hr0 heq
  exact <hs0, hsr  |>  hr1>

/-- **The Dedekind-cut up-set for `t  in  [0,1]`.** `cutU t r` holds at `r  <=  1` with
     `1 - t < r`
(strictly), or at the single endpoint `r = 1`; `1 - t` is Scott's "cut point" `c`,
    chosen so the
represented real `t` increases exactly as the up-set does (Scott's hint: ` subseteq `
    matches ` <= `). -/
def cutU (t : ?) (r : ?) : Prop := r  <=  1  /\  ((1 - t : ?) < (r : ?)  \/  r = 1)

theorem cutU_le1 {t : ?} {r : ?} (h : cutU t r) : r  <=  1 := h.1

theorem cutU_one {t : ?} : cutU t 1 := <le_rfl, Or.inr rfl>

theorem cutU_pos {t : ?} (ht1 : t  <=  1) {r : ?} (h : cutU t r) : 0 < r := by
  rcases h.2 with h2 | rfl
  * have hc : (0 : ?)  <=  1 - t := by linarith
    exact_mod_cast lt_of_le_of_lt hc h2
  * norm_num

theorem cutU_up {r s : ?} {t : ?} (h : cutU t r) (hrs : r  <=  s) (hs1 : s  <=  1) :
    cutU t s := by
  refine <hs1, ?_>
  rcases h.2 with h2 | rfl
  * left
    have hcast : (r : ?)  <=  (s : ?) := by exact_mod_cast hrs
    linarith
  * exact Or.inr (le_antisymm hs1 hrs)

/-- The fixed point of `a` corresponding to `t  in  [0,1]` (Scott's Dedekind cut). -/
def cutElt (t : ?) (ht1 : t  <=  1) : R.Element :=
```

```
  mkElement (cutU t) cutU_one (fun _ h => cutU_pos ht1 h) (fun _ h => cutU_le1 h)
    (fun _ _ h hrs hs1 => cutU_up h hrs hs1)

@[simp] theorem cutElt_mem {t : ?} {ht1 : t  <=  1} {r : ?} :
    (cutElt t ht1).mem (Set.Ico 0 r)  <->  cutU t r := by
  simp only [cutElt, mkElement_mem]
  constructor
  * rintro <r', hr', heq>
    have hr'0 := cutU_pos ht1 hr'
    have hne : (Set.Ico (0 : ?) r).Nonempty := heq  |>  <0, le_refl 0, hr'0>
    obtain <x, hx0, hxr> := hne
    have hr0 : 0 < r := lt_of_le_of_lt hx0 hxr
    rwa [Ico0_inj hr0 hr'0 heq]
  * exact fun h => <r, h, rfl>

/-- The (real-valued) image of `x`'s up-set: `{(r:?) ? x.mem [0,r)}`. -/
def cutS (x : R.Element) : Set ? := (fun r : ? => (r : ?)) '' {r | x.mem (Set.Ico 0 r)
    }

/-- **The canonical Dedekind cut point** `1 - c` of a fixed element `x`, `c := sInf (
    cutS x)`. -/
noncomputable def cutPoint (x : R.Element) : ? := 1 - sInf (cutS x)

theorem cutS_bddBelow (x : R.Element) : BddBelow (cutS x) := by
  refine <0, ?_>
  rintro y <r, hr, rfl>
  dsimp only
  exact_mod_cast (R_mem_Ico0 (x.sub hr)).1.le

theorem cutPoint_nonneg (x : R.Element) : 0  <=  cutPoint x := by
  have hmem1 : x.mem (Set.Ico (0 : ?) 1) := by rw [ <-  R_master]; exact x.master_mem
  have hc1 : sInf (cutS x)  <=  1 := csInf_le (cutS_bddBelow x) <1, hmem1, by norm_num
    >
  unfold cutPoint; linarith

theorem cutS_nonempty (x : R.Element) : (cutS x).Nonempty :=
  <1, 1, show x.mem (Set.Ico (0 : ?) 1) by rw [ <-  R_master]; exact x.master_mem, by
    norm_num>

theorem cutPoint_le_one (x : R.Element) : cutPoint x  <=  1 := by
  have hc0 : (0:?)  <=  sInf (cutS x) := le_csInf (cutS_nonempty x)
    (by rintro y <r, hr, rfl>; dsimp only; exact_mod_cast (R_mem_Ico0 (x.sub hr)).1.le
    )
  unfold cutPoint; linarith

/-- **Surjectivity**: every fixed point of `a` equals `cutElt (cutPoint x) _`; the
    fixed-point
equation supplies exactly the "openness" (no minimum, aside from `r = 1`) that makes
    this work. -/
theorem cutElt_cutPoint {x : R.Element} (hfix : a.toElementMap x = x) :
    cutElt (cutPoint x) (cutPoint_le_one x) = x := by
  have hSne := cutS_nonempty x
  have hSbdd := cutS_bddBelow x
  apply Element.ext_U
  intro r
  rw [cutElt_mem]
```

```
  have hcU : cutU (cutPoint x) r  <->  (r  <=  1  /\  (sInf (cutS x) < (r : ?)  \/  r
    = 1)) := by
    unfold cutPoint
    constructor
    * rintro <h1, h2 | h2>
      * exact <h1, Or.inl (by linarith)>
      * exact <h1, Or.inr h2>
    * rintro <h1, h2 | h2>
      * exact <h1, Or.inl (by linarith)>
      * exact <h1, Or.inr h2>
  rw [hcU, Iff.comm]
  have hopen : forall  Y, x.mem Y  <->  exists  X, x.mem X  /\  a.rel X Y := by
    intro Y
    conv_lhs => rw [ <-  hfix]
    exact mem_toElementMap a x
  constructor
  * intro hxr
    refine <(R_mem_Ico0 (x.sub hxr)).2, ?_>
    by_cases hr1 : r = 1
    * exact Or.inr hr1
    * refine Or.inl ?_
      obtain <X, hXmem, hrel> := (hopen (Set.Ico 0 r)).mp hxr
      obtain <r'', hr''0, hr''1, rfl> := x.sub hXmem
      rw [a_rel, aRel_iff hr''0 hr''1 (R_mem_Ico0 (x.sub hxr)).1 (R_mem_Ico0 (x.sub
    hxr)).2] at hrel
      rcases hrel with hlt | <-, hreq>
      * calc sInf (cutS x)  <=  (r'' : ?) := csInf_le hSbdd <r'', hXmem, rfl>
          _ < (r : ?) := by exact_mod_cast hlt
      * exact absurd hreq hr1
  * rintro <hr1, hc | rfl>
    * obtain <q, <r0, hr0mem, rfl>, hqlt> := exists_lt_of_csInf_lt hSne hc
      dsimp only at hqlt
      have hr0r : r0 < r := by exact_mod_cast hqlt
      have hr0pos : 0 < r0 := (R_mem_Ico0 (x.sub hr0mem)).1
      exact x.up_mem hr0mem <r, hr0pos.trans hr0r, hr1, rfl> (Set.Ico_subset_Ico
    le_rfl hr0r.le)
    * rw [ <-  R_master]; exact x.master_mem

/-- `cutU` is monotone in `t`: raising `t` only weakens the cut point `1 - t`, so it
    only
*enlarges* the up-set. -/
theorem cutU_mono {t t' : ?} (htt' : t  <=  t') {r : ?} (h : cutU t r) : cutU t' r :=
    by
  refine <h.1, ?_>
  rcases h.2 with hc | rfl
  * exact Or.inl (by linarith)
  * exact Or.inr rfl

/-- Domain order reduces to a statement about `mem` at the endpoints `[0,r)` alone. -/
theorem le_iff_U {x y : R.Element} :
    x  <=  y  <->  forall  r : ?, x.mem (Set.Ico 0 r)  ->  y.mem (Set.Ico 0 r) :=
  <fun h r => h (Set.Ico 0 r), fun h X hX => by
    obtain <r, _, _, rfl> := x.sub hX; exact h r hX>

/-- **Scott's hint made precise:** ` subseteq ` (the up-sets, i.e. ` sqsub ` on `R.
    Element`) matches ` <= ` (on the
```

```
represented reals) -- `cutElt` is order-preserving *both ways*. -/
theorem cutElt_le_iff {t t' : ?} (_ht0 : 0  <=  t) (ht1 : t  <=  1) (ht0' : 0  <=  t')
     (ht1' : t'  <=  1) :
    cutElt t ht1  <=  cutElt t' ht1'  <->  t  <=  t' := by
  rw [le_iff_U]
  simp only [cutElt_mem]
  constructor
  * intro h
    by_contra hlt
    rw [not_le] at hlt
    obtain <r, hr1, hr2> := exists_rat_btwn (show (1 - t : ?) < 1 - t' by linarith)
    have hrle1 : r  <=  1 := by
      have : (r : ?) < 1 := by linarith
      exact_mod_cast this.le
    have hcutt' : cutU t' r := h r <hrle1, Or.inl hr1>
    rcases hcutt'.2 with hc' | hreq
    * linarith
    * rw [hreq] at hr2; norm_num at hr2; linarith
  * exact fun htt' r hr => cutU_mono htt' hr

/-- `cutElt` is injective on `[0,1]` (antisymmetry of `cutElt_le_iff`). -/
theorem cutElt_injective {t t' : ?} (ht0 : 0  <=  t) (ht1 : t  <=  1) (ht0' : 0  <=  t
    ') (ht1' : t'  <=  1)
    (h : cutElt t ht1 = cutElt t' ht1') : t = t' :=
  le_antisymm ((cutElt_le_iff ht0 ht1 ht0' ht1').mp h.le) ((cutElt_le_iff ht0' ht1'
    ht0 ht1).mp h.ge)

/-- **`cutElt t` is a fixed point of `a`**, for every `t  in  [0,1]`: this is exactly
    the "open cut"
computation -- `a` shifts `[0,r)` strictly forward, so `s` re-enters the up-set from
    below iff some
smaller `r` was already in it, i.e. iff `1 - t < s` (density of `?` supplies the
    witness `r`). -/
theorem toElementMap_cutElt {t : ?} (ht1 : t  <=  1) :
    a.toElementMap (cutElt t ht1) = cutElt t ht1 := by
  apply Element.ext_U
  intro s
  simp only [mem_toElementMap, a_rel, cutElt_mem]
  constructor
  * rintro <X, hX, hrel>
    obtain <r, hUr, rfl> := hX
    have hs01 := R_mem_Ico0 (a.rel_cod (a_rel.mpr hrel))
    rw [aRel_iff (cutU_pos ht1 hUr) (cutU_le1 hUr) hs01.1 hs01.2] at hrel
    rcases hrel with hlt | <hr1, hs1>
    * rcases hUr.2 with hc | hr1
      * exact <hs01.2, Or.inl (lt_trans hc (by exact_mod_cast hlt))>
      * exact absurd (hr1  |>  hlt : (1:?) < s) (not_lt.mpr hs01.2)
    * exact <hs01.2, Or.inr hs1>
  * intro hs
    rcases hs.2 with hc | rfl
    * obtain <r, hr1, hr2> := exists_rat_btwn hc
      have hr0 : 0 < r := by
        have : (0:?)  <=  1 - t := by linarith
        exact_mod_cast lt_of_le_of_lt this hr1
      have hrle1 : r  <=  1 := by
        have : (r:?) < (s:?) := hr2
```

```
        have hrs : r < s := by exact_mod_cast this
        exact hrs.le.trans hs.1
      exact <Set.Ico 0 r, <r, <hrle1, Or.inl hr1>, rfl>, r, s, hr0, hrle1, cutU_pos
    ht1 hs,
        cutU_le1 hs, rfl, rfl, Or.inl (by exact_mod_cast hr2)>
    * exact <Set.Ico 0 1, <1, cutU_one, rfl>, 1, 1, one_pos, le_rfl, one_pos, le_rfl,
    rfl, rfl,
        Or.inr <rfl, rfl>>

/-- **Exercise 8.11 (Dedekind-cut correspondence).** The fixed-point set of `a` is
    order
isomorphic to `Set.Icc (0 : ?) 1` -- a one-one correspondence with the reals between
    `0` and `1`
inclusive, under which ` subseteq ` (domain order) matches ` <= ` (Scott's hint). -/
noncomputable def fixOrderIso : {y : R.Element // a.toElementMap y = y} ?o Set.Icc (0
    : ?) 1 where
  toFun y := <cutPoint y.1, cutPoint_nonneg y.1, cutPoint_le_one y.1>
  invFun t := <cutElt t.1 t.2.2, toElementMap_cutElt t.2.2>
  left_inv y := Subtype.ext (cutElt_cutPoint y.2)
  right_inv t := Subtype.ext (cutElt_injective (cutPoint_nonneg _) (cutPoint_le_one _)
     t.2.1 t.2.2
    (cutElt_cutPoint (toElementMap_cutElt t.2.2)))
  map_rel_iff' := by
    intro y y'
    show cutPoint y.1  <=  cutPoint y'.1  <->  y.1  <=  y'.1
    constructor
    * intro h
      have h' := (cutElt_le_iff (cutPoint_nonneg y.1) (cutPoint_le_one y.1) (
    cutPoint_nonneg y'.1)
        (cutPoint_le_one y'.1)).mpr h
      rwa [cutElt_cutPoint y.2, cutElt_cutPoint y'.2] at h'
    * intro h
      rw [ <-  cutElt_cutPoint y.2,  <-  cutElt_cutPoint y'.2] at h
      exact (cutElt_le_iff (cutPoint_nonneg y.1) (cutPoint_le_one y.1) (
    cutPoint_nonneg y'.1)
        (cutPoint_le_one y'.1)).mp h

theorem aRel_le {r s : ?} (h : r < s  \/  (r = 1  /\  s = 1)) : r  <=  s := by
  rcases h with h | <rfl, rfl>
  * exact h.le
  * exact le_rfl

/-- **Exercise 8.11 (second part, retraction).** `a  comp  a = a`. -/
theorem isRetraction_a : Retraction a := by
  constructor
  apply ApproximableMap.ext
  intro X Z
  constructor
  * rintro <Y, <r, s, hr0, hr1, hs0, hs1, rfl, rfl, hcond1>,
      <s', t, hs0', hs1', ht0, ht1, hYs', rfl, hcond2>>
    have hss' : s = s' := Ico0_inj hs0 hs0' hYs'
    subst hss'
    refine <r, t, hr0, hr1, ht0, ht1, rfl, rfl, ?_>
    rcases hcond1 with h1 | <rfl, rfl>
    * rcases hcond2 with h2 | <rfl, rfl>
      * exact Or.inl (h1.trans h2)
```

```
      * exact Or.inl h1
    * rcases hcond2 with h2 | <-, rfl>
      * exact absurd (h2.trans_le ht1) (lt_irrefl _)
      * exact Or.inr <rfl, rfl>
  * rintro <r, t, hr0, hr1, ht0, ht1, rfl, rfl, hcond>
    rcases hcond with h | <rfl, rfl>
    * obtain <s, hrs, hst> := exists_between h
      have hs0 : 0 < s := hr0.trans hrs
      have hs1 : s  <=  1 := (hst.trans_le ht1).le
      exact <Set.Ico 0 s, <r, s, hr0, hr1, hs0, hs1, rfl, rfl, Or.inl hrs>,


        <s, t, hs0, hs1, ht0, ht1, rfl, rfl, Or.inl hst>>
    * exact <Set.Ico 0 1, <1, 1, one_pos, le_rfl, one_pos, le_rfl, rfl, rfl, Or.inr <
    rfl, rfl>>,
        <1, 1, one_pos, le_rfl, one_pos, le_rfl, rfl, rfl, Or.inr <rfl, rfl>>>

/-- **Exercise 8.11 (second part, projection).** `a  sqsub  I_R`. -/
theorem isProjection_a : IsProjection a := by
  refine <isRetraction_a, ?_>
  rintro X Y <r, s, hr0, hr1, hs0, hs1, rfl, rfl, hcond>
  exact <<r, hr0, hr1, rfl>, <s, hs0, hs1, rfl>, Set.Ico_subset_Ico le_rfl (aRel_le
    hcond)>

/-- **No point `g <t,_,_>` with `t > 0` is compact**, for *any* neighbourhood system `F
    ` and order
isomorphism `g : Set.Icc (0:?) 1 ?o F.Element`: the directed family `{g <s,_,_> ? 0  <=
     s < t}` has
supremum exactly `g <t,_,_>` (`isLUB_Ico` transported through `g`), yet `g <t,_,_>`
    sits strictly
above every member. This is Scott's hint made precise: aside from `False` (`t = 0`), `
    Set.Icc (0:?) 1`
has no finite/compact elements. -/
theorem not_isCompactElt_pos {b : Type} {F : NeighborhoodSystem b}
    (g : Set.Icc (0 : ?) 1 ?o F.Element) {t : ?} (ht0 : 0  <=  t) (ht1 : t  <=  1) (
    htpos : 0 < t) :
    not  IsCompactElt (g <t, ht0, ht1>) := by
  intro hcompact
  set S : Set (Set.Icc (0 : ?) 1) := {x | x.1  in  Set.Ico (0 : ?) t} with hSdef
  have hmem0 : (<0, le_refl 0, zero_le_one> : Set.Icc (0 : ?) 1)  in  S := <le_refl 0,
     htpos>
  have : Nonempty S := <<_, hmem0>>
  set fam : S  ->  F.Element := fun x => g x.1 with hfamdef
  have hdir : forall  i j : S, exists  k : S, fam i  <=  fam k  /\  fam j  <=  fam k
    := by
    rintro <x, hx> <y, hy>
    refine <<<max x.1 y.1, le_max_of_le_left x.2.1, (max_lt hx.2 hy.2).le.trans ht1>,
      <le_max_of_le_left x.2.1, max_lt hx.2 hy.2>>, ?_, ?_>
    * exact g.monotone (le_max_left x.1 y.1)
    * exact g.monotone (le_max_right x.1 y.1)
  have hSisLUB : IsLUB S (<t, ht0, ht1> : Set.Icc (0 : ?) 1) := by
    constructor
    * rintro x hx
      exact hx.2.le
    * intro ub hub
      exact (isLUB_Ico htpos).2 (fun y hy =>
        hub (show (<y, hy.1, hy.2.le.trans ht1> : Set.Icc (0 : ?) 1)  in  S from hy))
```

```
  have hgSisLUB : IsLUB (g '' S) (g <t, ht0, ht1>) := g.isLUB_image'.mpr hSisLUB
  have hrange : Set.range fam = g '' S := by
    ext z
    constructor
    * rintro <<x, hx>, rfl>; exact <x, hx, rfl>
    * rintro <x, hx, rfl>; exact <<x, hx>, rfl>
  have hisLUBfam : IsLUB (Set.range fam) (g <t, ht0, ht1>) := hrange  |>  hgSisLUB
  have hisLUBiSup : IsLUB (Set.range fam) (iSupDirected fam hdir) :=
    <fun _ <i, hi> => hi  |>  le_iSupDirected fam hdir i,
      fun y hy => iSupDirected_le fam hdir (fun i => hy <i, rfl>)>
  have heq : iSupDirected fam hdir = g <t, ht0, ht1> := hisLUBiSup.unique hisLUBfam
  obtain <i, hi> := hcompact fam hdir (le_of_eq heq.symm)
  have hle : (<t, ht0, ht1> : Set.Icc (0 : ?) 1)  <=  i.1 := g.le_iff_le.mp hi
  exact absurd hle (not_le.mpr i.2.2)

/-- **Exercise 8.11 (conclusion).** `a` is *not* finitary: were it, its fixed-point set
     -- order
isomorphic to `Set.Icc (0:?) 1` via `fixOrderIso` -- would also be order isomorphic to
    some `F.Element`,
which is algebraic (`eq_iSupDirected_principal`); but every principal approximant of `g
     <1,_,_>`
would then be compact, hence (`not_isCompactElt_pos`) equal to `g <0,_,_>`, forcing
`g <1,_,_> = g <0,_,_>` -- contradicting injectivity of `g` at `1  !=  0`. Exactly
    Scott's hint: aside
from `False`, `{x ? x = a(x)}` has no finite elements, so it cannot be algebraic unless
     it is trivial. -/
theorem not_isFinitary_a : not  IsFinitary a := by
  rintro <b, F, <e>>
  set g : Set.Icc (0 : ?) 1 ?o F.Element := fixOrderIso.symm.trans e with hgdef
  set x : F.Element := g <1, zero_le_one, le_refl 1> with hxdef
  have hcompact_bot : forall  i : {X : Set b // x.mem X}, F.principal (x.sub i.2) = g
    <0, le_refl 0, zero_le_one> := by
    intro i
    by_contra hne
    have hc0 : g.symm (F.principal (x.sub i.2))  !=  <0, le_refl 0, zero_le_one> :=
    fun h => hne (by
      rw [ <-  h, OrderIso.apply_symm_apply])
    have ht0 : (0:?)  <=  (g.symm (F.principal (x.sub i.2))).1 := (g.symm (F.principal
     (x.sub i.2))).2.1
    have ht1 : (g.symm (F.principal (x.sub i.2))).1  <=  1 := (g.symm (F.principal (x.
    sub i.2))).2.2
    have htpos : 0 < (g.symm (F.principal (x.sub i.2))).1 := by
      rcases ht0.lt_or_eq with h | h
      * exact h
      * exact absurd (Subtype.ext h.symm) hc0
    have hsub : (<(g.symm (F.principal (x.sub i.2))).1, ht0, ht1> : Set.Icc (0 : ?) 1)
        = g.symm (F.principal (x.sub i.2)) := Subtype.ext rfl
    have heq2 : F.principal (x.sub i.2) = g <(g.symm (F.principal (x.sub i.2))).1, ht0
    , ht1> := by
      rw [hsub, OrderIso.apply_symm_apply]
    exact not_isCompactElt_pos g ht0 ht1 htpos (heq2  |>  principal_isCompactElt (x.
    sub i.2))
  have hxeq : x = g <0, le_refl 0, zero_le_one> := by
    conv_lhs => rw [eq_iSupDirected_principal x]
    apply le_antisymm
    * apply iSupDirected_le
```

```
      intro i
      exact le_of_eq (hcompact_bot i)
    * obtain <i0> := (instNonemptyMemSubtype x)
      have hstep : g <0, le_refl 0, zero_le_one>  <=
          iSupDirected (fun j : {X : Set b // x.mem X} => F.principal (x.sub j.2))
            (principalFamily_directed x) := by
        rw [ <-  hcompact_bot i0]
        exact le_iSupDirected (fun j : {X : Set b // x.mem X} => F.principal (x.sub j
    .2))
          (principalFamily_directed x) i0
      exact hstep
  have : (<1, zero_le_one, le_refl 1> : Set.Icc (0:?) 1) = <0, le_refl 0, zero_le_one>
     :=
    g.injective hxeq
  exact absurd (congrArg Subtype.val this) one_ne_zero

end DomainTheory.NeighbourhoodSystems.Exercise811
```

The tip then assembles those tools into Scott's statement.

— `exercise_8_11`

Exercise_8_11.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    DedekindCutProjection

@[expose] public section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem

/-- **Exercise 8.11 (Scott 1981, PRG-19).** The Dedekind-cut map on `R` is a projection
     that is not finitary. -/
theorem exercise_8_11 :
    IsProjection Exercise811.a  /\  Retraction Exercise811.a  /\  not IsFinitary
    Exercise811.a :=
  <Exercise811.isProjection_a, Exercise811.isRetraction_a, Exercise811.
    not_isFinitary_a>

end
```

3.9.14 Exercise 8.12 — integer presentation of U

Generalize the notation $2X + 1$ for subsets $X \subseteq \mathbb{N}$ to sets of the form

$$2^k X + \ell, \quad \text{where } \ell < 2^k.$$

Let V be the non-empty finite unions of sets $2^k\mathbb{N} + \ell$. Show that $U \cong V$ and that the isomorphism is effective, thus obtaining another presentation of U.

Tip summary Scott's claim is a single conjunction: V is effectively given, and U is effectively isomorphic to V. The Lean tip `exercise_8_12` packages exactly that, assembling `V_isEffectivelyGiven` with the concrete effective isomorphism `effectiveIso812_UV`.

Development Scott gives no written proof—only the construction of V and the demand that the isomorphism be effective. The formalization therefore supplies a structured development under the exercise: present V, build an abstract domain isomorphism, make it effective, equip both sides with computable bisections/splits, and assemble the effective iso.

Presenting V The integer neighbourhood system V is built from affine residue classes $2^k\mathbb{N} + \ell$ and their non-empty finite unions (`affine` / `levelSet` / `upsample`). Pure neighbourhood facts live under `namespace V` (`V.union_mem`, `V.diff_mem`, `V.isPositive`, ...).

— `V` and Pref-folded neighbourhood lemmas

`V.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    NeighborhoodSystem
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_19
public import Mathlib.Data.Fintype.Defs
public import Mathlib.Algebra.BigOperators.Group.Finset.Basic
public import Mathlib.Data.Nat.Bitwise
public import Mathlib.Data.Nat.BitIndices
public import Mathlib.Combinatorics.Colex

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem

/-- **`2^k Nat + ?`** (for `? < 2^k`): the residue class of `?` modulo `2^k`.
    Automatically `{}` when
`?  >=  2^k` (no `n` can leave a remainder that large), matching Scott's side condition
     `? < 2^k`
without needing to bake it into the definition. -/
def affine (k ? : Nat) : Set Nat := {n : Nat | n % 2 ^ k = ?}

@[simp] theorem mem_affine {k ? n : Nat} : n  in  affine k ?  <->  n % 2 ^ k = ? :=
    Iff.rfl

namespace affine
open NeighborhoodSystem

theorem nonempty {k ? : Nat} (h? : ? < 2 ^ k) : (affine k ?).Nonempty :=
  <?, Nat.mod_eq_of_lt h?>

end affine
open NeighborhoodSystem

/-- The union of the residue classes `affine k ?` for `? < 2^k` with `m.testBit ? =
    true`. Every
finite union of `affine`-sets sharing the *same* level `k` is of this form for a unique
     `m`
(`Finset.range (2^k)` residues partition `Nat`, unlike arbitrary rational intervals),
    so a single
```

```
natural number `m` replaces the `List` bookkeeping `Definition87.lean` needs for `?`.
    -/
def levelSet (k m : Nat) : Set Nat := {n : Nat | m.testBit (n % 2 ^ k) = true}

@[simp] theorem mem_levelSet {k m n : Nat} : n  in  levelSet k m  <->  m.testBit (n %
    2 ^ k) = true :=
  Iff.rfl

namespace levelSet
open NeighborhoodSystem

theorem inter_same_level (k m_1 m_2 : Nat) :
    levelSet k m_1  I  levelSet k m_2 = levelSet k (m_1 &&& m_2) := by
  ext n; simp [levelSet, Nat.testBit_and]

theorem nonempty_iff {k m : Nat} :
    (levelSet k m).Nonempty  <->  exists  ? < 2 ^ k, m.testBit ? = true := by
  constructor
  * rintro <n, hn>
    exact <n % 2 ^ k, Nat.mod_lt n (by positivity), hn>
  * rintro <?, h?, hm>
    exact <?, by rw [mem_levelSet, Nat.mod_eq_of_lt h?]; exact hm>

theorem union_same_level (k m_1 m_2 : Nat) :
    levelSet k m_1  U  levelSet k m_2 = levelSet k (m_1 ||| m_2) := by
  ext n; simp [levelSet, Nat.testBit_or]

/-- `levelSet k m` is exactly the union of `affine k ?` over `?` with `m.testBit ?`. -/
theorem eq_biUnion (k m : Nat) :
    levelSet k m = ? ?  in  (Finset.range (2 ^ k)).filter (fun ? => m.testBit ?),
    affine k ? := by
  ext n
  simp only [mem_levelSet, Set.mem_iUnion, Finset.mem_filter, Finset.mem_range,
    mem_affine,
    exists_prop]
  constructor
  * intro h; exact <n % 2 ^ k, <Nat.mod_lt n (by positivity), h>, rfl>
  * rintro <?, <-, hm>, rfl>; exact hm

end levelSet
namespace affine
open NeighborhoodSystem

/-- `affine k ?` itself is the singleton-bitmask `levelSet`. -/
theorem eq_levelSet (k ? : Nat) : affine k ? = levelSet k (2 ^ ?) := by
  ext n
  simp [mem_affine, mem_levelSet, Nat.testBit_two_pow, eq_comm]

end affine
open NeighborhoodSystem

/-- Re-express a level-`k` bitmask `m` at a finer level `k'  >=  k`: residue `r` (mod
    `2^k'`) is
included iff `r`'s coarser residue `r % 2^k` was. Built via `Nat.equivBitIndices` (the
    bijection
`Nat ? Finset Nat` sending `n` to its set bits) rather than raw shifting arithmetic. -/
```

```
def upsample (k k' m : Nat) : Nat :=
  ? r  in  (Finset.range (2 ^ k')).filter (fun r => m.testBit (r % 2 ^ k)), 2 ^ r

namespace testBit
open NeighborhoodSystem

theorem upsample {k k' m r : Nat} (hr : r < 2 ^ k') :
    (_root_.DomainTheory.NeighbourhoodSystems.upsample k k' m).testBit r  <->  m.
    testBit (r % 2 ^ k) := by
  unfold _root_.DomainTheory.NeighbourhoodSystems.upsample
  rw [ <-  Nat.mem_bitIndices,  <-  List.mem_toFinset, Finset.
    toFinset_bitIndices_sum_two_pow,
    Finset.mem_filter, Finset.mem_range]
  simp [hr]

end testBit
namespace levelSet
open NeighborhoodSystem

/-- `levelSet 0 1 = Nat`: at level `0` there is a single residue class (everything is
    `? 0 mod 1`),
and `1` (bit `0` set) selects it. This is `V`'s master neighbourhood. -/
theorem zero_one : levelSet 0 1 = Set.univ := by
  ext n; simp [levelSet, Nat.mod_one]

/-- **`upsample` preserves the set `levelSet` represents.** -/
theorem upsample {k k' m : Nat} (hk : k  <=  k') : levelSet k' (_root_.DomainTheory.
    NeighbourhoodSystems.upsample k k' m) = levelSet k m := by
  ext n
  simp only [mem_levelSet]
  rw [testBit.upsample (Nat.mod_lt n (by positivity)),
    Nat.mod_mod_of_dvd n (pow_dvd_pow 2 hk)]

/-- **`levelSet` is closed under intersection, unconditionally** (unlike `Definition87.
    lean`'s `U`,
which genuinely needs a consistency hypothesis): upsample both masks to the common
    level
`max k_1 k_2` and take the bitwise `AND`. -/
theorem inter (k_1 m_1 k_2 m_2 : Nat) :
    levelSet k_1 m_1  I  levelSet k_2 m_2
      = levelSet (max k_1 k_2) (_root_.DomainTheory.NeighbourhoodSystems.upsample k_1
    (max k_1 k_2) m_1 &&& _root_.DomainTheory.NeighbourhoodSystems.upsample k_2 (max
    k_1 k_2) m_2) := by
  rw [ <-  levelSet.upsample (le_max_left k_1 k_2) (m := m_1),  <-  levelSet.upsample
    (le_max_right k_1 k_2) (m := m_2)]
  ext n
  simp only [mem_levelSet, Set.mem_inter_iff, Nat.testBit_and, Bool.and_eq_true]

end levelSet

namespace testBit
open NeighborhoodSystem

/-- **The bitwise "and-not" identity**: `a ^^^ (a &&& b) = a` on bits where `b`'s bit
    is `0`, and
`= 0` on bits where `b`'s bit is `1` (whether or not `a`'s bit is set) -- i.e. exactly
```

```
    "`a`'s bits
with `b`'s bits cleared". -/
theorem xor_and_self (a b : Nat) (i : Nat) :
    (a ^^^ (a &&& b)).testBit i = (a.testBit i && !b.testBit i) := by
  rw [Nat.testBit_xor, Nat.testBit_and]
  cases a.testBit i <;> cases b.testBit i <;> rfl

end testBit

namespace levelSet
open NeighborhoodSystem

/-- **`levelSet` is closed under set-difference, unconditionally** -- mirrors `levelSet
    .inter`
with `testBit.xor_and_self` in place of plain `&&&`. -/
theorem diff (k_1 m_1 k_2 m_2 : Nat) :
    levelSet k_1 m_1 \ levelSet k_2 m_2
      = levelSet (max k_1 k_2)
          (let a := _root_.DomainTheory.NeighbourhoodSystems.upsample k_1 (max k_1 k_2
    ) m_1
           let b := _root_.DomainTheory.NeighbourhoodSystems.upsample k_2 (max k_1 k_2
    ) m_2
           a ^^^ (a &&& b)) := by
  rw [ <-  levelSet.upsample (le_max_left k_1 k_2) (m := m_1),
     <-  levelSet.upsample (le_max_right k_1 k_2) (m := m_2)]
  ext n
  simp only [mem_levelSet, Set.mem_sdiff, testBit.xor_and_self, Bool.and_eq_true, Bool
    .not_eq_true,
    Bool.not_eq_true']

end levelSet

open NeighborhoodSystem

/-- **Exercise 8.12's `V`**: the neighbourhood system over `Nat` whose neighbourhoods
    are the
non-empty finite unions of residue classes `2^k Nat + ?` (`? < 2^k`). Bookkeeping-free
    via
`levelSet`: every finite union of `affine`-sets -- even at *different* levels `k_i` --
    refines
(`levelSet.upsample`) to a single common level, so `V.mem X := exists  k m, X =
    levelSet k m` already
captures every such union (`V.mem_iff_finiteUnion`). Closure under intersection needs
    no
consistency hypothesis (`levelSet.inter`). -/
def V : NeighborhoodSystem Nat where
  mem X := exists  k m, X = levelSet k m  /\  X.Nonempty
  master := Set.univ
  master_nonempty := Set.univ_nonempty
  master_mem := <0, 1, levelSet.zero_one.symm, Set.univ_nonempty>
  inter_mem := by
    rintro X Y Z <k_1, m_1, rfl, -> <k_2, m_2, rfl, -> <k_3, m_3, rfl, hZne> hZsub
    exact <max k_1 k_2, upsample k_1 (max k_1 k_2) m_1 &&& upsample k_2 (max k_1 k_2)
    m_2,
      levelSet.inter k_1 m_1 k_2 m_2, hZne.mono hZsub>
  sub_master := by rintro X _; exact Set.subset_univ X
```

```
namespace V

/-- **`V` is closed under binary union**, as long as the result is non-empty. -/
theorem union_mem {X Y : Set Nat} (hX : X = {}  \/  V.mem X) (hY : Y = {}  \/  V.mem Y
    ) :
    X  U  Y = {}  \/  V.mem (X  U  Y) := by
  rcases hX with rfl | hX
  * simpa using hY
  rcases hY with rfl | hY
  * simpa using Or.inr hX
  obtain <k_1, m_1, rfl, hXne> := hX
  obtain <k_2, m_2, rfl, -> := hY
  have heq : levelSet k_1 m_1  U  levelSet k_2 m_2
      = levelSet (max k_1 k_2) (upsample k_1 (max k_1 k_2) m_1 ||| upsample k_2 (max
    k_1 k_2) m_2) := by
    rw [ <-  levelSet.union_same_level, levelSet.upsample (le_max_left k_1 k_2),
      levelSet.upsample (le_max_right k_1 k_2)]
  exact Or.inr <max k_1 k_2, _, heq, hXne.mono Set.subset_union_left>

/-- **`V` is closed under finite (`Fintype`-indexed) unions**, as long as the result is
non-empty -- mirrors `Definition87.lean`'s `U.iUnion_mem` verbatim. -/
theorem iUnion_mem {? : Type*} [Fintype ?] {f : ?  ->  Set Nat} (hf : forall  i, f i =
     {}  \/  V.mem (f i))
    (hne : (? i, f i).Nonempty) : V.mem (? i, f i) := by
  classical
  have hstep : forall  s : Finset ?, (? i  in  s, f i) = {}  \/  V.mem (? i  in  s, f
    i) := by
    intro s
    induction s using Finset.induction_on with
    | empty => exact Or.inl (by simp)
    | insert i s hi ih =>
      rw [Finset.set_biUnion_insert]
      exact union_mem (hf i) ih
  have hall : (? i, f i) = ? i  in  (Finset.univ : Finset ?), f i := by simp
  rw [hall] at hne  |-
  exact (hstep Finset.univ).resolve_left hne.ne_empty

/-- **`V.mem` literally matches Scott's "non-empty finite unions of `2^k Nat + ?`, `? <
     2^k`"** --
reconciles the bookkeeping-free `levelSet` encoding with Exercise 8.12's literal
    description,
mirroring `Definition87.lean`'s `U.mem_iff_scott`. -/
theorem mem_iff_finiteUnion {X : Set Nat} :
    V.mem X  <->
      exists  F : Finset (Nat  x  Nat), (forall  p  in  F, p.2 < 2 ^ p.1)  /\  X = ? p
      in  F, affine p.1 p.2  /\  X.Nonempty := by
  constructor
  * rintro <k, m, rfl, hne>
    refine <(Finset.range (2 ^ k)).filter (fun ? => m.testBit ?) |>.image (fun ? => (k
    , ?)), ?_, ?_, hne>
    * rintro <k', ?> hp
      obtain <?', h?', heq> := Finset.mem_image.mp hp
      obtain <hlt, -> := Finset.mem_filter.mp h?'
      simp only [Finset.mem_range] at hlt
      obtain <rfl, rfl> := Prod.mk.injEq ..  |>  heq
```

```
      exact hlt
    * rw [levelSet.eq_biUnion]
      ext n
      simp only [Set.mem_iUnion, Finset.mem_image, Finset.mem_filter, Finset.mem_range
    ,
        exists_prop, exists_exists_and_eq_and]
  * rintro <F, hbound, rfl, hne>
    have hf : forall  p : F, affine p.1.1 p.1.2 = {}  \/  V.mem (affine p.1.1 p.1.2)
    :=
      fun <p, hp> => Or.inr <p.1, 2 ^ p.2, affine.eq_levelSet p.1 p.2,
        affine.eq_levelSet p.1 p.2  |>  affine.nonempty (hbound p hp)>
    have hrw : (? p  in  F, affine p.1 p.2) = ? p : F, affine p.1.1 p.1.2 := by
      ext n; simp only [Set.mem_iUnion, exists_prop, Subtype.exists]
    rw [hrw] at hne  |-
    exact iUnion_mem hf hne

set_option linter.unusedVariables false

theorem example_812 : forall  {X : Set Nat},
  Iff (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.mem Nat DomainTheory.
    NeighbourhoodSystems.V X)
    (exists  (F : Finset (Prod Nat Nat)),
      And
        (forall  (p : Prod Nat Nat),
          @Membership.mem (Prod Nat Nat) (Finset (Prod Nat Nat))
              (@SetLike.instMembership (Finset (Prod Nat Nat)) (Prod Nat Nat) (@Finset
    .instSetLike (Prod Nat Nat))) F
              p  ->
            @LT.lt Nat instLTNat (@Prod.snd Nat Nat p)
              (@HPow.hPow Nat Nat Nat (@instHPow Nat Nat (@NPow.toPow Nat (@Monoid.
    toNPow Nat Nat.instMonoid)))
                (@OfNat.ofNat Nat (nat_lit 2) (instOfNatNat (nat_lit 2))) (@Prod.fst
    Nat Nat p)))
        (And
          (@Eq (Set Nat) X
            (? (p : Prod Nat Nat),
              ? (_ :
                @Membership.mem (Prod Nat Nat) (Finset (Prod Nat Nat))
                  (@SetLike.instMembership (Finset (Prod Nat Nat)) (Prod Nat Nat) (
    @Finset.instSetLike (Prod Nat Nat)))
                  F p),
                DomainTheory.NeighbourhoodSystems.affine (@Prod.fst Nat Nat p) (@Prod.
    snd Nat Nat p)))
          (@Set.Nonempty Nat X))) :=
  mem_iff_finiteUnion

/-- **Scott's remark after Definition 8.7, transplanted to `V`**: every `V`-
    neighbourhood splits
into two disjoint proper `V`-neighbourhoods. Refine to one level finer via `upsample`,
    which
duplicates every set bit `?` of `m` into a matching pair `(?, ? + 2^k)`; peeling off *
    one*
representative bit `?_0` (witnessing `X`'s non-emptiness) leaves its twin `?_0 + 2^k`
    behind, so the
"rest" is guaranteed non-empty regardless of how many other bits `m` had. -/
theorem no_minimal {X : Set Nat} (hX : V.mem X) :
```

```
  exists  Y Z : Set Nat, V.mem Y  /\  V.mem Z  /\  Y  I  Z = {}  /\  Y  U  Z = X  /\
    Y  !=  X  /\  Z  !=  X := by
obtain <k, m, rfl, hne> := hX
obtain <?_0, h?_0, hbit> := levelSet.nonempty_iff.mp hne
have hstep : 2 ^ (k + 1) = 2 ^ k + 2 ^ k := by rw [pow_succ]; ring
have h?_02 : ?_0 < 2 ^ (k + 1) := by omega
have h?_0' : ?_0 + 2 ^ k < 2 ^ (k + 1) := by omega
set M := upsample k (k + 1) m with hM
have hXM : levelSet k m = levelSet (k + 1) M := (levelSet.upsample (Nat.le_succ k)).
  symm
have hM?_0 : M.testBit ?_0 = true := by
  rw [hM, testBit.upsample h?_02, Nat.mod_eq_of_lt h?_0]; exact hbit
have hM?_0' : M.testBit (?_0 + 2 ^ k) = true := by
  rw [hM, testBit.upsample h?_0', Nat.add_mod_right, Nat.mod_eq_of_lt h?_0]; exact
  hbit
set Y := levelSet (k + 1) (2 ^ ?_0) with hYdef
set Z := levelSet (k + 1) (M ^^^ 2 ^ ?_0) with hZdef
have hYmem : V.mem Y := <k + 1, 2 ^ ?_0, rfl, levelSet.nonempty_iff.mpr <?_0, h?_02,
   by simp>>
have hZmem : V.mem Z := <k + 1, M ^^^ 2 ^ ?_0, rfl,
  levelSet.nonempty_iff.mpr <?_0 + 2 ^ k, h?_0', by
    rw [Nat.testBit_xor, hM?_0']; simp>>
have hInter : Y  I  Z = {} := by
  rw [hYdef, hZdef, levelSet.inter_same_level]
  ext n
  simp only [mem_levelSet, Set.mem_empty_iff_false, iff_false, Nat.testBit_and, Bool
  .and_eq_true]
  rintro <h1, h2>
  have hpos : n % 2 ^ (k + 1) = ?_0 := by
    by_contra hne'
    rw [Nat.testBit_two_pow] at h1
    exact hne' (of_decide_eq_true h1).symm
  rw [hpos, Nat.testBit_xor, hM?_0] at h2
  simp at h2
have hUnion : Y  U  Z = levelSet k m := by
  rw [hYdef, hZdef, levelSet.union_same_level, hXM]
  congr 1
  apply Nat.eq_of_testBit_eq
  intro i
  rw [Nat.testBit_or, Nat.testBit_xor]
  rcases eq_or_ne i ?_0 with rfl | hi
  * simp [hM?_0]
  * simp [Ne.symm hi]
have hYne : Y.Nonempty := (levelSet.nonempty_iff (k := k + 1) (m := 2 ^ ?_0)).mpr <?
  _0, h?_02, by simp>
have hZne : Z.Nonempty := (levelSet.nonempty_iff (k := k + 1) (m := M ^^^ 2 ^ ?_0)).
  mpr
  <?_0 + 2 ^ k, h?_0', by
    rw [Nat.testBit_xor, hM?_0']; simp>
refine <Y, Z, hYmem, hZmem, hInter, hUnion, ?_, ?_>
* intro hEq
  apply hZne.ne_empty
  have hZsub : Z  subseteq  Y := by rw [hEq]; exact hUnion  |>  Set.
  subset_union_right
  rw [ <-  hInter, Set.inter_eq_right.mpr hZsub]
* intro hEq
```

```
    apply hYne.ne_empty
    have hYsub : Y  subseteq  Z := by rw [hEq]; exact hUnion  |>  Set.
    subset_union_left
    rw [ <-  hInter, Set.inter_eq_left.mpr hYsub]

/-- **`V` is Positive**: `levelSet.inter` presents *any* raw intersection
    unconditionally, so only
non-emptiness can obstruct `V.mem (X  I  Y)`. -/
theorem isPositive : V.IsPositive := by
  rintro X Y <k_1, m_1, rfl, -> <k_2, m_2, rfl, ->
  refine <fun <k, m, heq, hne> => hne, fun hne => <max k_1 k_2, _, levelSet.inter k_1
    m_1 k_2 m_2, hne>>

theorem mem_nonempty {X : Set Nat} (hX : V.mem X) : X.Nonempty := by
  obtain <k, m, -, hne> := hX; exact hne

theorem noMinimal : V.NoMinimal := by
  intro X hX
  obtain <Y, Z, hY, hZ, hYZinter, hYZunion, -, -> := no_minimal hX
  exact <Y, Z, hY, hZ, mem_nonempty hY, mem_nonempty hZ, hYZinter, hYZunion>

/-- **`V` is closed under set-difference of two of its own neighbourhoods** (or the
    difference is
empty). -/
theorem diff_mem {X Y : Set Nat} (hX : V.mem X) (hY : V.mem Y) : X \ Y = {}  \/  V.mem
     (X \ Y) := by
  obtain <k_1, m_1, rfl, -> := hX
  obtain <k_2, m_2, rfl, -> := hY
  rcases Set.eq_empty_or_nonempty (levelSet k_1 m_1 \ levelSet k_2 m_2) with h | h
  * exact Or.inl h
  * exact Or.inr <max k_1 k_2, _, levelSet.diff k_1 m_1 k_2 m_2, h>

theorem diffClosed : V.DiffClosed := fun hX hY => diff_mem hX hY

end V

end DomainTheory.NeighbourhoodSystems
```

Its computable presentation (indices, decoding, intersection/union/difference codes, and the canonical bisection `splitV*`) lives in a dedicated library module.

— computable presentation of $V, `including`VComputablePresentation`and`V.isEffectivelyGiven‘

VComputablePresentation.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.VX
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.Vinter
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.minLevel
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.splitV

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

end DomainTheory.NeighbourhoodSystems
```

```
end
```

Abstract isomorphism (8.12(c)) Given two positive, difference-closed, atomless neighbourhood systems with enumerations that cover every neighbourhood and start at the master set, one builds paired atom sequences (`XPseq`/`YPseq`) from mutual `SplitSpec′` probes and obtains a domain isomorphism. The headline is `isomorphic_812c`.

— difference-closure / no-minimal apparatus, atom pairing, and `isomorphic_812c`

`domainIso812c.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.V
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Theorem_8_8
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    VComputablePresentation
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Exercise_7_18
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Theorem_7_5
public import Mathlib.Data.Nat.Bitwise
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Exercise_3_26
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    atomPair812c
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    splitSpec812c
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    combinedXY812c
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    combinedY812c
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.XPseq812c
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    transfer812c
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    transfer812c_more
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.hcore812c

@[expose] public section

@[expose] public section

/- concordance: source L5147 EXERCISE 8.12 integer presentation of U -/
-- Exercise812/affine_2.lean
namespace DomainTheory.NeighbourhoodSystems
namespace affine

open NeighborhoodSystem

end affine
end DomainTheory.NeighbourhoodSystems

/-! ### Exercise 8.12(c)-(e) (moved from Exercise_8_15 monolith) -/
```

```
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem
section
section Iso

variable {a b : Type*} (D_0 : NeighborhoodSystem a) (D_1 : NeighborhoodSystem b)
  (hD_0pos : D_0.IsPositive) (hD_0diff : D_0.DiffClosed) (hD_0nomin : D_0.NoMinimal)
  (hD_1pos : D_1.IsPositive) (hD_1diff : D_1.DiffClosed) (hD_1nomin : D_1.NoMinimal)
  (X : Nat  ->  Set a) (Y : Nat  ->  Set b) (hXmem : forall  n, D_0.mem (X n)) (hYmem
    : forall  n, D_1.mem (Y n))
  (hD_0mne : D_0.master.Nonempty) (hD_1mne : D_1.master.Nonempty)
  (hXcover : forall  S, D_0.mem S  <->  exists  n, S = X n) (hYcover : forall  S, D_1.
    mem S  <->  exists  n, S = Y n)
  (hX0 : X 0 = D_0.master) (hY0 : Y 0 = D_1.master)

/-- **Exercise 8.12(c)(vii)(6), final assembly.** The order isomorphism `D_0.Element ?o
     D_1.Element`.
`toD1`/`toD0` are mutually inverse (via (4)'s `XPseq.eq_Y_iff_X_eq_YPseq`, resolving
    which index in
the *other* family represents a given neighbourhood) and preserve/reflect ` <= ` (via
`X_eq_iff_XPseq_eq`) -- direct transcription of `Theorem88a.lean`'s `domainIso`,
    generalized from one
shared index family (`e`/`Yidx`) to genuinely two (`X`/`XPseq` and `Y`/`YPseq`). -/

noncomputable def domainIso812c : DomainIso D_0 D_1 where
  toFun := toD1 D_0 D_1 hD_0pos hD_0diff hD_0nomin hD_1pos hD_1diff hD_1nomin X Y
    hXmem hYmem hD_0mne hD_1mne
    hXcover hYcover hX0
  invFun := toD0 D_0 D_1 hD_0pos hD_0diff hD_0nomin hD_1pos hD_1diff hD_1nomin X Y
    hXmem hYmem hD_0mne
    hD_1mne hXcover hYcover hX0 hY0
  left_inv x := by
    apply Element.ext
    intro S
    constructor
    * rintro <n, hn, k, hk, hxk>
      have hXeq : X k = YPseq D_0 D_1 hD_0nomin hD_1nomin X Y n :=
        (XPseq.eq_Y_iff_X_eq_YPseq D_0 D_1 hD_0pos hD_0diff hD_0nomin hD_1pos hD_1diff
    hD_1nomin X Y hXmem
          hYmem hD_0mne hD_1mne k n).mp hk.symm
      rw [hn,  <-  hXeq]
      exact hxk
    * intro hS
      obtain <m, hm> := (hXcover S).mp (x.sub hS)
      have hxXm : x.mem (X m) := hm  |>  hS
      obtain <n, hn> := (hYcover (XPseq D_0 D_1 hD_0nomin hD_1nomin X Y m)).mp
        (XPseq.mem D_0 D_1 hD_0pos hD_0diff hD_0nomin hD_1pos hD_1diff hD_1nomin X Y
    hXmem hYmem hD_0mne
          hD_1mne m)
      have hSYPn : S = YPseq D_0 D_1 hD_0nomin hD_1nomin X Y n :=
        hm.trans ((XPseq.eq_Y_iff_X_eq_YPseq D_0 D_1 hD_0pos hD_0diff hD_0nomin
    hD_1pos hD_1diff hD_1nomin
          X Y hXmem hYmem hD_0mne hD_1mne m n).mp hn)
      exact <n, hSYPn, m, hn.symm, hxXm>
  right_inv y := by
```

```
    apply Element.ext
    intro T
    constructor
    * rintro <n, hn, k, hk, hyk>
      have hXPeq : XPseq D_0 D_1 hD_0nomin hD_1nomin X Y n = Y k :=
        (XPseq.eq_Y_iff_X_eq_YPseq D_0 D_1 hD_0pos hD_0diff hD_0nomin hD_1pos hD_1diff
     hD_1nomin X Y hXmem
          hYmem hD_0mne hD_1mne n k).mpr hk
      rw [hn, hXPeq]
      exact hyk
    * intro hT
      obtain <m, hm> := (hYcover T).mp (y.sub hT)
      have hyYm : y.mem (Y m) := hm  |>  hT
      obtain <n, hn> := (hXcover (YPseq D_0 D_1 hD_0nomin hD_1nomin X Y m)).mp
        (YPseq.mem D_0 D_1 hD_0pos hD_0diff hD_0nomin hD_1pos hD_1diff hD_1nomin X Y
    hXmem hYmem hD_0mne
          hD_1mne m)
      have hTXPn : T = XPseq D_0 D_1 hD_0nomin hD_1nomin X Y n :=
        hm.trans (((XPseq.eq_Y_iff_X_eq_YPseq D_0 D_1 hD_0pos hD_0diff hD_0nomin
    hD_1pos hD_1diff
          hD_1nomin X Y hXmem hYmem hD_0mne hD_1mne n m).mpr hn.symm).symm)
      exact <n, hTXPn, m, hn.symm, hyYm>
  map_rel_iff' := by
    intro x x2
    constructor
    * intro hle S hxS
      obtain <n, hn> := (hXcover S).mp (x.sub hxS)
      have hxn : x.mem (X n) := hn  |>  hxS
      obtain <k, hk, hx2k> := hle _
        (<n, rfl, hxn> : (toD1 D_0 D_1 hD_0pos hD_0diff hD_0nomin hD_1pos hD_1diff
    hD_1nomin X Y hXmem
          hYmem hD_0mne hD_1mne hXcover hYcover hX0 x).mem (XPseq D_0 D_1 hD_0nomin
    hD_1nomin X Y n))
      rw [hn, (X_eq_iff_XPseq_eq D_0 D_1 hD_0pos hD_0diff hD_0nomin hD_1pos hD_1diff
    hD_1nomin X Y hXmem
        hYmem hD_0mne hD_1mne n k).mpr hk]
      exact hx2k
    * intro hle T hT
      obtain <n, hn, hxn> := hT
      exact <n, hn, hle _ hxn>

end Iso
end

section

/-- **Generalization of `Theorem88.lean`'s `split_fst_subset`**: a splitting operation'
    s first
output is a subset of `B` (from `I  U  J = B`). -/
theorem split_fst_subset' {a ? : Type*} {E : NeighborhoodSystem ?}
    {split : Set a  ->  Set ?  ->  Set a  ->  Set ?  x  Set ?} (hsplit : SplitSpec' E
    split)
    {A : Set a} {B : Set ?} (hAB : A = {}  <->  B = {}) (hBE : B = {}  \/  E.mem B) (
    Xn : Set a) :
    (split A B Xn).1  subseteq  B :=
  Set.subset_union_left.trans_eq (hsplit hAB hBE Xn).2.2.2.2.1
```

```
/-- **Generalization of `Theorem88.lean`'s `split_snd_subset`**: a splitting operation'
    s second
output is a subset of `B` (from `I  U  J = B`). -/
theorem split_snd_subset' {a ? : Type*} {E : NeighborhoodSystem ?}
    {split : Set a  ->  Set ?  ->  Set a  ->  Set ?  x  Set ?} (hsplit : SplitSpec' E
    split)
    {A : Set a} {B : Set ?} (hAB : A = {}  <->  B = {}) (hBE : B = {}  \/  E.mem B) (
    Xn : Set a) :
    (split A B Xn).2  subseteq  B :=
  Set.subset_union_right.trans_eq (hsplit hAB hBE Xn).2.2.2.2.1

section AtomPair

variable {a b : Type*} (D_0 : NeighborhoodSystem a) (D_1 : NeighborhoodSystem b)
  (hD_0pos : D_0.IsPositive) (hD_0diff : D_0.DiffClosed) (hD_0nomin : D_0.NoMinimal)
  (hD_1pos : D_1.IsPositive) (hD_1diff : D_1.DiffClosed) (hD_1nomin : D_1.NoMinimal)
  (X : Nat  ->  Set a) (Y : Nat  ->  Set b) (hXmem : forall  n, D_0.mem (X n)) (hYmem
    : forall  n, D_1.mem (Y n))

variable (hD_0mne : D_0.master.Nonempty) (hD_1mne : D_1.master.Nonempty)
include hD_0pos hD_0diff hD_0nomin hD_1pos hD_1diff hD_1nomin hXmem hYmem hD_0mne
    hD_1mne

end AtomPair

section Iso

variable {a b : Type*} (D_0 : NeighborhoodSystem a) (D_1 : NeighborhoodSystem b)
  (hD_0pos : D_0.IsPositive) (hD_0diff : D_0.DiffClosed) (hD_0nomin : D_0.NoMinimal)
  (hD_1pos : D_1.IsPositive) (hD_1diff : D_1.DiffClosed) (hD_1nomin : D_1.NoMinimal)
  (X : Nat  ->  Set a) (Y : Nat  ->  Set b) (hXmem : forall  n, D_0.mem (X n)) (hYmem
    : forall  n, D_1.mem (Y n))
  (hD_0mne : D_0.master.Nonempty) (hD_1mne : D_1.master.Nonempty)
  (hXcover : forall  S, D_0.mem S  <->  exists  n, S = X n) (hYcover : forall  S, D_1.
    mem S  <->  exists  n, S = Y n)
  (hX0 : X 0 = D_0.master) (hY0 : Y 0 = D_1.master)

include hD_0pos hD_0diff hD_0nomin hD_1pos hD_1diff hD_1nomin X Y hXmem hYmem hD_0mne
    hD_1mne hXcover
  hYcover hX0 hY0 in
/-- **Exercise 8.12(c)(vii)(6), headline.** `D_0  ~= ? D_1`: completes Exercise 8.12(c)
     in full. -/
theorem isomorphic_812c : D_0  ~= ? D_1 :=
  <domainIso812c D_0 D_1 hD_0pos hD_0diff hD_0nomin hD_1pos hD_1diff hD_1nomin X Y
    hXmem hYmem hD_0mne
    hD_1mne hXcover hYcover hX0 hY0>
end Iso
end

end DomainTheory.NeighbourhoodSystems
end
```

Abstract E-side atoms The one-sided atom apparatus against an abstract neighbourhood system `E` (the `atomU` analogue) lives under `atomE`, with Pref-folded companions `atomE.zero`, `atomE.succ`, `atomE.congr`, `atomE.invariant`, ...

— `atomE`, `YseqE`, and Pref-folded `atomE.xxx` lemmas

atomE.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    domainIso812c

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem

/-- The `E`-side atom matching `genAtom X Delta ? n`, built recursively via an abstract
     splitting
operation `split` satisfying `SplitSpec' E split`. Generalizes `Theorem88.lean`'s `
    atomU` from the
hardcoded target `U` to an abstract `E`. -/
noncomputable def atomE {a ? : Type*} (E : NeighborhoodSystem ?)
    (split : Set a  ->  Set ?  ->  Set a  ->  Set ?  x  Set ?)
    (X : Nat  ->  Set a) (Delta : Set a) (? : Nat  ->  Bool) : Nat  ->  Set ?
  | 0 => E.master
  | (n + 1) =>
      if ? n then (split (genAtom X Delta ? n) (atomE E split X Delta ? n) (X n)).1
      else (split (genAtom X Delta ? n) (atomE E split X Delta ? n) (X n)).2

section
variable {a ? : Type*}
variable (E : NeighborhoodSystem ?) (split : Set a  ->  Set ?  ->  Set a  ->  Set ?  x
      Set ?)
variable (X : Nat  ->  Set a) (Delta : Set a)

/-- **The abstract analogue of Scott's `Y_n`**: the union, over all `2?` depth-`n`
    atoms, of the
"+"-piece chosen when splitting against `X n`. Generalizes `Theorem88.lean`'s `Yseq`.
    -/
noncomputable def YseqE (n : Nat) : Set ? :=
  ? ?' : Fin n  ->  Bool, atomE E split X Delta (extendTrue ?') (n + 1)

namespace atomE

@[simp] theorem zero (? : Nat  ->  Bool) : atomE E split X Delta ? 0 = E.master := rfl

theorem succ (? : Nat  ->  Bool) (n : Nat) :
    atomE E split X Delta ? (n + 1) =
      if ? n then (split (genAtom X Delta ? n) (atomE E split X Delta ? n) (X n)).1
      else (split (genAtom X Delta ? n) (atomE E split X Delta ? n) (X n)).2 := rfl

/-- Extending/changing `?` at or beyond position `n` does not change `atomE E split X
    Delta ? n`
(mirrors `atomU_congr`). -/
theorem congr {? ?' : Nat  ->  Bool} {n : Nat} (h : forall  i < n, ? i = ?' i) :
```

```
    atomE E split X Delta ? n = atomE E split X Delta ?' n := by
  induction n with
  | zero => rfl
  | succ n ih =>
    have hprev : atomE E split X Delta ? n = atomE E split X Delta ?' n :=
      ih (fun i hi => h i (Nat.lt_succ_of_lt hi))
    have hA : genAtom X Delta ? n = genAtom X Delta ?' n :=
      genAtom_congr X Delta (fun i hi => h i (Nat.lt_succ_of_lt hi))
    have hn := h n (Nat.lt_succ_self n)
    rw [succ, succ, hA, hprev, hn]

end atomE

variable (hDelta : Delta.Nonempty) (hEmne : E.master.Nonempty) (hsplit : SplitSpec' E
    split)
include hDelta hEmne hsplit

namespace atomE

/-- **The core invariant, generalizing `atomU_invariant`.** At every depth `n` and for
    every sign
sequence `?`: (a) the atom-matching invariant (`genAtom X Delta ? n = {}  <->  atomE E
    split X Delta ? n = {}`),
(b) `atomE E split X Delta ? n` is either empty or a genuine `E`-neighbourhood, and (c)
     pairwise
disjointness for sign sequences disagreeing somewhere below `n`. -/
theorem invariant :
    forall  n, (forall  ?, genAtom X Delta ? n = {}  <->  atomE E split X Delta ? n =
    {})  /\
      (forall  ?, atomE E split X Delta ? n = {}  \/  E.mem (atomE E split X Delta ? n
    ))  /\
      (forall  ? ?', (exists  i < n, ? i  !=  ?' i)  ->
        atomE E split X Delta ? n  I  atomE E split X Delta ?' n = {}) := by
  intro n
  induction n with
  | zero =>
    refine <fun ? => ?_, fun ? => Or.inr E.master_mem,
      fun ? ?' <i, hi, _> => absurd hi (Nat.not_lt_zero i)>
    show Delta = {}  <->  (E.master : Set ?) = {}
    exact <fun h => absurd h hDelta.ne_empty, fun h => absurd h hEmne.ne_empty>
  | succ n ih =>
    obtain <ihmatch, ihmem, ihdisj> := ih
    refine <fun ? => ?_, fun ? => ?_, fun ? ?' <i, hi, hne> => ?_>
    * have hspec := hsplit (ihmatch ?) (ihmem ?) (X n)
      rw [succ]
      by_cases h? : ? n = true
      * rw [show genAtom X Delta ? (n + 1) = genAtom X Delta ? n  I  X n from by simp
    [genAtom, h?]]
        simp only [h?, ite_true]
        exact hspec.2.2.1
      * rw [Bool.not_eq_true] at h?
        have hsub := genAtom_subset X Delta ? n
        rw [show genAtom X Delta ? (n + 1) = genAtom X Delta ? n \ X n from by
          have heq : genAtom X Delta ? (n + 1) = genAtom X Delta ? n  I  (Delta \ X n)
     := by simp [genAtom, h?]
          rw [heq]
```

```
        ext x; constructor
        * rintro <hx1, -, hx2>; exact <hx1, hx2>
        * rintro <hx1, hx2>; exact <hx1, hsub hx1, hx2>]
      simp only [h?]
      exact hspec.2.2.2.1
  * have hspec := hsplit (ihmatch ?) (ihmem ?) (X n)
    rw [succ]
    by_cases h? : ? n = true
    * simp only [h?, ite_true]; exact hspec.1
    * simp only [h?]; exact hspec.2.1
  * by_cases hagree : forall  j < n, ? j = ?' j
    * have h?n : ? n  !=  ?' n := by
        intro heq
        exact hne (by
          rcases Nat.lt_succ_iff_lt_or_eq.mp hi with hi' | rfl
          * exact hagree i hi'
          * exact heq)
      have hA : genAtom X Delta ? n = genAtom X Delta ?' n := genAtom_congr X Delta
  hagree
      have hB : atomE E split X Delta ? n = atomE E split X Delta ?' n := congr E
  split X Delta hagree
      have hspec := hsplit (ihmatch ?) (ihmem ?) (X n)
      rw [succ, succ, hA, hB]
      have hIJ := hspec.2.2.2.2.2
      rcases Bool.eq_false_or_eq_true (? n) with h1 | h1 <;>
        rcases Bool.eq_false_or_eq_true (?' n) with h2 | h2 <;>
        simp_all [Set.inter_comm]
    * push Not at hagree
      obtain <j, hj, hjne> := hagree
      have hd : atomE E split X Delta ? n  I  atomE E split X Delta ?' n = {} :=
  ihdisj ? ?' <j, hj, hjne>
      have h1 : atomE E split X Delta ? (n + 1)  subseteq  atomE E split X Delta ? n
   := by
        rw [succ]
        by_cases h? : ? n = true
        * simp only [h?, ite_true]; exact split_fst_subset' hsplit (ihmatch ?) (
  ihmem ?) (X n)
        * simp only [h?]; exact split_snd_subset' hsplit (ihmatch ?) (ihmem ?) (X n)
      have h2 : atomE E split X Delta ?' (n + 1)  subseteq  atomE E split X Delta ?'
   n := by
        rw [succ]
        by_cases h?' : ?' n = true
        * simp only [h?', ite_true]; exact split_fst_subset' hsplit (ihmatch ?') (
  ihmem ?') (X n)
        * simp only [h?']; exact split_snd_subset' hsplit (ihmatch ?') (ihmem ?') (X
   n)
      exact Set.subset_eq_empty (Set.inter_subset_inter h1 h2) hd

/-- Corollary of `atomE.invariant`, extracted for reuse: `atomE` only shrinks as `n`
    grows
(mirrors `atomU_succ_subset`). -/
theorem succ_subset (? : Nat  ->  Bool) (n : Nat) :
    atomE E split X Delta ? (n + 1)  subseteq  atomE E split X Delta ? n := by
  obtain <hmatch, hmem, -> := invariant E split X Delta hDelta hEmne hsplit n
  rw [succ]
  by_cases h? : ? n = true
```

```
  * simp only [h?, ite_true]; exact split_fst_subset' hsplit (hmatch ?) (hmem ?) (X n)
  * simp only [h?]; exact split_snd_subset' hsplit (hmatch ?) (hmem ?) (X n)

/-- `atomE E split X Delta ? n` is always a subset of `E.master` (mirrors `
    atomU_subset_master`). -/
theorem subset_master (? : Nat  ->  Bool) (n : Nat) : atomE E split X Delta ? n
    subseteq  E.master := by
  induction n with
  | zero => exact subset_rfl
  | succ n ih => exact (succ_subset E split X Delta hDelta hEmne hsplit ? n).trans ih

end atomE

/-- **The "I-formula", generalizing `split_fst_eq_inter_Yseq`**: the "+"-piece chosen
    when
splitting the depth-`n` atom for `?` against `X n` is *exactly* the intersection of
    that atom with
`YseqE E split X Delta n`. -/
theorem split_fst_eq_inter_YseqE (? : Nat  ->  Bool) (n : Nat) :
    (split (genAtom X Delta ? n) (atomE E split X Delta ? n) (X n)).1 =
      atomE E split X Delta ? n  I  YseqE E split X Delta n := by
  obtain <hmatch, hmem, hdisj> := atomE.invariant E split X Delta hDelta hEmne hsplit
    n
  set A := genAtom X Delta ? n with hAdef
  set B := atomE E split X Delta ? n with hBdef
  set I := (split A B (X n)).1 with hIdef
  set ?2 := Function.update ? n true with h?2def
  set ?3 := extendTrue (restrictFin ?2 n) with h?3def
  have hagreefull : forall  i < n + 1, ?3 i = ?2 i := by
    intro i hi
    rcases Nat.lt_succ_iff_lt_or_eq.mp hi with hi' | rfl
    * exact extendTrue_restrictFin_agree ?2 n i hi'
    * simp [h?3def, extendTrue, h?2def]
  have hI_eq : atomE E split X Delta ?2 (n + 1) = I := by
    rw [atomE.succ]
    have hd2n : ?2 n = true := by simp [h?2def]
    have hA2 : genAtom X Delta ?2 n = A := by
      apply genAtom_congr; intro i hi; simp [h?2def, Function.update_of_ne (ne_of_lt
    hi)]
    have hB2 : atomE E split X Delta ?2 n = B := by
      apply atomE.congr; intro i hi; simp [h?2def, Function.update_of_ne (ne_of_lt hi)
    ]
    rw [hd2n, ite_eq_left rfl, hA2, hB2]
  have h?3_eq : atomE E split X Delta ?3 (n + 1) = I := (atomE.congr E split X Delta
    hagreefull).trans hI_eq
  apply Set.Subset.antisymm
  * have hIsubB : I  subseteq  B := split_fst_subset' hsplit (hmatch ?) (hmem ?) (X n)
    have hIsubY : I  subseteq  YseqE E split X Delta n :=
      h?3_eq  |>  Set.subset_iUnion (fun ?' => atomE E split X Delta (extendTrue ?') (
    n + 1))
        (restrictFin ?2 n)
    exact Set.subset_inter hIsubB hIsubY
  * rintro z <hzB, hzY>
    obtain <?', hz'> := Set.mem_iUnion.mp hzY
    by_cases hagree : forall  i < n, extendTrue ?' i = ? i
    * have hAeq : genAtom X Delta (extendTrue ?') n = A := genAtom_congr X Delta
```

```
    hagree
      have hBeq : atomE E split X Delta (extendTrue ?') n = B := atomE.congr E split X
     Delta hagree
      have hlast : extendTrue ?' n = true := by simp [extendTrue]
      have heq : atomE E split X Delta (extendTrue ?') (n + 1) = I := by
        rw [atomE.succ, hAeq, hBeq, hlast, ite_eq_left rfl]
      rwa [heq] at hz'
    * push Not at hagree
      obtain <j, hj, hjne> := hagree
      have hzB' : z  in  atomE E split X Delta (extendTrue ?') n :=
        atomE.succ_subset E split X Delta hDelta hEmne hsplit (extendTrue ?') n hz'
      have hempty := hdisj ? (extendTrue ?') <j, hj, fun h => hjne (h.symm)>
      exact absurd (Set.mem_inter hzB hzB') (by rw [hempty]; simp)

namespace atomE

/-- **Closed form for `atomE`, generalizing `atomU_succ_eq`**: with `YseqE` in hand, `
    atomE`
satisfies exactly the same recursive description as `genAtom (YseqE E split X Delta) E.
    master`. -/
theorem succ_eq (? : Nat  ->  Bool) (n : Nat) :
    atomE E split X Delta ? (n + 1) =
      atomE E split X Delta ? n  I
        (if ? n then YseqE E split X Delta n else E.master \ YseqE E split X Delta n)
    := by
  obtain <hmatch, hmem, -> := invariant E split X Delta hDelta hEmne hsplit n
  have hspec := hsplit (hmatch ?) (hmem ?) (X n)
  have hIeq := split_fst_eq_inter_YseqE E split X Delta hDelta hEmne hsplit ? n
  by_cases h? : ? n = true
  * rw [succ, ite_eq_left h?, ite_eq_left h?]; exact hIeq
  * rw [succ, ite_eq_right h?, ite_eq_right h?]
    have hJeq : (split (genAtom X Delta ? n) (atomE E split X Delta ? n) (X n)).2 =
        atomE E split X Delta ? n \ YseqE E split X Delta n := by
      have hunion := hspec.2.2.2.2.1
      have hinter := hspec.2.2.2.2.2
      ext x
      constructor
      * intro hxJ
        have hxB : x  in  atomE E split X Delta ? n := hunion  |>  Or.inr hxJ
        refine <hxB, fun hxY => ?_>
        have hxI : x  in  (split (genAtom X Delta ? n) (atomE E split X Delta ? n) (X
    n)).1 :=
          hIeq  |>  Set.mem_inter hxB hxY
        exact absurd (Set.mem_inter hxI hxJ) (by rw [hinter]; simp)
      * rintro <hxB, hxnY>
        rw [ <-  hunion] at hxB
        rcases hxB with hxI | hxJ
        * exact absurd (hIeq  |>  hxI : x  in  atomE E split X Delta ? n  I  YseqE E
    split X Delta n).2 hxnY
        * exact hxJ
    rw [hJeq]
    have hsub := subset_master E split X Delta hDelta hEmne hsplit ? n
    ext x
    constructor
    * rintro <hx1, hx2>; exact <hx1, hsub hx1, hx2>
    * rintro <hx1, -, hx2>; exact <hx1, hx2>
```

```
/-- **`atomE` coincides with the generic atom construction on `YseqE`/`E.master`**,
    generalizing
`atomU_eq_genAtom`. Lets every later argument treat `atomE` and `genAtom` uniformly (in
     particular,
`genAtom_forward`/`genAtom_self` -- proved once, generically, in `Theorem88.lean` --
    apply verbatim
to the `E`-side atoms too). -/
theorem eq_genAtom (? : Nat  ->  Bool) :
    forall  n, atomE E split X Delta ? n = genAtom (YseqE E split X Delta) E.master ?
    n
  | 0 => rfl
  | (n + 1) => by
      rw [succ_eq E split X Delta hDelta hEmne hsplit, eq_genAtom ? n]; rfl

end atomE

/-- **The transfer lemma, generalized to `E`.** A finite Boolean combination of the `X
    i` is
non-empty in `D`'s carrier `Delta` iff the *same* Boolean combination of `YseqE E split
     X Delta i` is
non-empty in `E.master`. Mirrors `transfer_empty_iff`. -/
theorem transfer_empty_iffE {cs : List (Nat  x  Bool)} {n : Nat} (hn : forall  p  in
    cs, p.1 < n) :
    {x  in  Delta | forall  p  in  cs, (p.2 = true  <->  x  in  X p.1)}.Nonempty  <->
      {y  in  E.master | forall  p  in  cs, (p.2 = true  <->  y  in  YseqE E split X
    Delta p.1)}.Nonempty := by
  have hcore : forall  ? n, genAtom X Delta ? n = {}  <->  genAtom (YseqE E split X
    Delta) E.master ? n = {} :=
    fun ? n => (atomE.invariant E split X Delta hDelta hEmne hsplit n).1 ?
      |>.trans (by rw [atomE.eq_genAtom E split X Delta hDelta hEmne hsplit])
  have hcore' : forall  ? n, genAtom (YseqE E split X Delta) E.master ? n = {}  <->
    genAtom X Delta ? n = {} :=
    fun ? n => (hcore ? n).symm
  exact <transfer_dir X Delta (YseqE E split X Delta) E.master hcore hn,
    transfer_dir (YseqE E split X Delta) E.master X Delta hcore' hn>

end

end DomainTheory.NeighbourhoodSystems
```

Computable presentation predicates The data-carrying structures for computable difference, split, union, and bisection each live in their own library module, with Pref-folded companions under the structure namespace.

— `IsComputableDiff` and Diff-side deciders

`IsComputableDiff.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    domainIso812c
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Exercise_7_18
```

```
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Theorem_7_5

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive ApproximableMap Exercise718

structure IsComputableDiff {a : Type*} {V : NeighborhoodSystem a} (P :
    ComputablePresentation V) where
  diffIdx : Nat  ->  Nat  ->  Nat
  diffIdx_primrec : Nat.Primrec (fun t => diffIdx t.unpair.1 t.unpair.2)
  diffIdx_spec : forall  {n m : Nat}, (exists  k, P.X k = P.X n \ P.X m)  ->  P.X (
    diffIdx n m) = P.X n \ P.X m
  diff_computable : RecDecidable_2 (fun n m => exists  k, P.X k = P.X n \ P.X m)

namespace IsComputableDiff

variable {a : Type*} {V : NeighborhoodSystem a} (P : ComputablePresentation V)
variable (hDiff : IsComputableDiff P)

theorem exists_iff_ne_empty (hdiff : V.DiffClosed) (hnomin : V.NoMinimal) (n m : Nat)
    :
    (exists  k, P.X k = P.X n \ P.X m)  <->  P.X n \ P.X m  !=  {} := by
  constructor
  * rintro <k, hk> hempty
    exact NoMinimal.mem_ne_empty hnomin (P.mem_X k) (hk.trans hempty)
  * intro hne
    rcases hdiff (P.mem_X n) (P.mem_X m) with hempty | hmem
    * exact absurd hempty hne
    * exact P.surj hmem

noncomputable def existsDiffDec : Nat  ->  Nat := fun t => isOne (hDiff.
    diff_computable.choose t)

namespace existsDiffDec

theorem le_one (t : Nat) : existsDiffDec P hDiff t  <=  1 := isOne_le_one _

theorem spec (n m : Nat) :
    existsDiffDec P hDiff (Nat.pair n m) = 1  <->  exists  k, P.X k = P.X n \ P.X m :=
     by
  unfold existsDiffDec
  rw [isOne_eq_one_iff]
  have h := hDiff.diff_computable.choose_spec.2 (Nat.pair n m)
  dsimp only at h
  rw [unpair_pair_fst, unpair_pair_snd] at h
  exact h.symm

theorem eq_zero_iff (hdiff : V.DiffClosed) (hnomin : V.NoMinimal) (n m : Nat) :
    existsDiffDec P hDiff (Nat.pair n m) = 0  <->  P.X n \ P.X m = {} := by
  have hle := existsDiffDec.le_one P hDiff (Nat.pair n m)
  have h1 := existsDiffDec.spec P hDiff n m
  have h2 := exists_iff_ne_empty P hdiff hnomin n m
  constructor
  * intro h0
```

```
    by_contra hne
    have h1' : existsDiffDec P hDiff (Nat.pair n m) = 1 := h1.mpr (h2.mpr hne)
    omega
  * intro hempty
    by_contra hne0
    have h1' : existsDiffDec P hDiff (Nat.pair n m) = 1 := by omega
    exact (h2.mp (h1.mp h1')) hempty

end existsDiffDec

noncomputable def emptyDiffDec : Nat  ->  Nat := fun t => 1 - existsDiffDec P hDiff t

namespace emptyDiffDec

theorem le_one (t : Nat) : emptyDiffDec P hDiff t  <=  1 := by
  unfold emptyDiffDec; have := existsDiffDec.le_one P hDiff t; omega

theorem eq_one_iff (hdiff : V.DiffClosed) (hnomin : V.NoMinimal) (n m : Nat) :
    emptyDiffDec P hDiff (Nat.pair n m) = 1  <->  P.X n \ P.X m = {} := by
  unfold emptyDiffDec
  have hle := existsDiffDec.le_one P hDiff (Nat.pair n m)
  have h0 := existsDiffDec.eq_zero_iff P hDiff hdiff hnomin n m
  constructor
  * intro h1; apply h0.mp; omega
  * intro hempty; have := h0.mpr hempty; omega

end emptyDiffDec

namespace primrec

theorem existsDiffDec : Nat.Primrec (_root_.DomainTheory.NeighbourhoodSystems.
    IsComputableDiff.existsDiffDec P hDiff) :=
  (primrec_isOne.comp hDiff.diff_computable.choose_spec.1).of_eq fun _ => rfl

theorem emptyDiffDec : Nat.Primrec (_root_.DomainTheory.NeighbourhoodSystems.
    IsComputableDiff.emptyDiffDec P hDiff) :=
  primrec_sub_2 (Nat.Primrec.const 1) (primrec.existsDiffDec P hDiff)

end primrec

end IsComputableDiff

variable {a : Type*} {V : NeighborhoodSystem a} (P : ComputablePresentation V)

noncomputable def existsInterDec : Nat  ->  Nat := fun t => isOne (P.cons_computable.
    choose t)

namespace existsInterDec

theorem le_one (t : Nat) : existsInterDec P t  <=  1 := isOne_le_one _

theorem spec (n m : Nat) :
    existsInterDec P (Nat.pair n m) = 1  <->  exists  k, P.X k  subseteq  P.X n  I  P.
    X m := by
  unfold existsInterDec
  rw [isOne_eq_one_iff]
```

```
  have h := P.cons_computable.choose_spec.2 (Nat.pair n m)
  dsimp only at h
  rw [unpair_pair_fst, unpair_pair_snd] at h
  exact h.symm

theorem eq_zero_iff (hpos : V.IsPositive) (hnomin : V.NoMinimal) (n m : Nat) :
    existsInterDec P (Nat.pair n m) = 0  <->  P.X n  I  P.X m = {} := by
  have hle := existsInterDec.le_one P (Nat.pair n m)
  constructor
  * intro h0
    by_contra hne
    have hmem : V.mem (P.X n  I  P.X m) :=
      (hpos (P.mem_X n) (P.mem_X m)).mpr (Set.nonempty_iff_ne_empty.mpr hne)
    obtain <k, hk> := P.surj hmem
    have h1 : existsInterDec P (Nat.pair n m) = 1 :=
      (existsInterDec.spec P n m).mpr <k, by rw [hk]>
    omega
  * intro hempty
    by_contra hne1
    have h1 : existsInterDec P (Nat.pair n m) = 1 := by omega
    obtain <k, hk> := (existsInterDec.spec P n m).mp h1
    exact absurd (Set.subset_eq_empty hk hempty) (hnomin.mem_ne_empty (P.mem_X k))

end existsInterDec

noncomputable def emptyInterDec : Nat  ->  Nat := fun t => 1 - existsInterDec P t

namespace emptyInterDec

theorem le_one (t : Nat) : emptyInterDec P t  <=  1 := by
  unfold emptyInterDec; have := existsInterDec.le_one P t; omega

theorem eq_one_iff (hpos : V.IsPositive) (hnomin : V.NoMinimal) (n m : Nat) :
    emptyInterDec P (Nat.pair n m) = 1  <->  P.X n  I  P.X m = {} := by
  unfold emptyInterDec
  have hle := existsInterDec.le_one P (Nat.pair n m)
  have h0 := existsInterDec.eq_zero_iff P hpos hnomin n m
  constructor
  * intro h1; apply h0.mp; omega
  * intro hempty; have := h0.mpr hempty; omega

end emptyInterDec

namespace primrec

theorem existsInterDec : Nat.Primrec (_root_.DomainTheory.NeighbourhoodSystems.
    existsInterDec P) :=
  (primrec_isOne.comp P.cons_computable.choose_spec.1).of_eq fun _ => rfl

theorem emptyInterDec : Nat.Primrec (_root_.DomainTheory.NeighbourhoodSystems.
    emptyInterDec P) :=
  primrec_sub_2 (Nat.Primrec.const 1) (primrec.existsInterDec P)

end primrec

end DomainTheory.NeighbourhoodSystems
```

— IsComputableSplit

IsComputableSplit.lean

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Theorem_7_5
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Exercise_7_18

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive ApproximableMap Exercise718

structure IsComputableSplit {a ? : Type*} {V : NeighborhoodSystem a} {W :
    NeighborhoodSystem ?}
    (P : ComputablePresentation V) (Q : ComputablePresentation W)
    (split : Set a  ->  Set ?  ->  Set a  ->  Set ?  x  Set ?) where

  posIdx : Nat  ->  Nat  ->  Nat  ->  Nat

  negIdx : Nat  ->  Nat  ->  Nat  ->  Nat

  posIdx_primrec : Nat.Primrec (fun t => posIdx t.unpair.1 t.unpair.2.unpair.1 t.
    unpair.2.unpair.2)

  negIdx_primrec : Nat.Primrec (fun t => negIdx t.unpair.1 t.unpair.2.unpair.1 t.
    unpair.2.unpair.2)

  posIdx_spec : forall  n m k, (split (P.X n) (Q.X m) (P.X k)).1  !=  {}  ->
    (split (P.X n) (Q.X m) (P.X k)).1 = Q.X (posIdx n m k)

  negIdx_spec : forall  n m k, (split (P.X n) (Q.X m) (P.X k)).2  !=  {}  ->
    (split (P.X n) (Q.X m) (P.X k)).2 = Q.X (negIdx n m k)

end DomainTheory.NeighbourhoodSystems
```

— IsComputableUnion and combineFound2

IsComputableUnion.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    domainIso812c
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Exercise_7_18
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Theorem_7_5

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive ApproximableMap Exercise718
```

```
structure IsComputableUnion {a : Type*} {V : NeighborhoodSystem a} (P :
    ComputablePresentation V)
    where

  unionIdx : Nat  ->  Nat  ->  Nat

  unionIdx_primrec : Nat.Primrec (fun t => unionIdx t.unpair.1 t.unpair.2)

  unionIdx_spec : forall  {n m : Nat}, (exists  k, P.X k = P.X n  U  P.X m)  ->  P.X (
    unionIdx n m) = P.X n  U  P.X m

  union_computable : RecDecidable_2 (fun n m => exists  k, P.X k = P.X n  U  P.X m)

theorem NeighborhoodSystem.mem_union_of_mem {? : Type*} {D : NeighborhoodSystem ?}
    (hpos : D.IsPositive) (hdiff : D.DiffClosed) (hnomin : D.NoMinimal) {X Y : Set ?}
    (hX : D.mem X) (hY : D.mem Y) : D.mem (X  U  Y) :=
  (union.mem_or_empty hpos hdiff (Or.inr hX) (Or.inr hY)).resolve_left fun h =>
    hnomin.mem_ne_empty hX (Set.subset_eq_empty Set.subset_union_left h)

namespace IsComputableUnion

noncomputable def combineFound2 {? : Type*} {W : NeighborhoodSystem ?}
    {Q : ComputablePresentation W} (hUnion : IsComputableUnion Q) (r0 r1 : Nat) : Nat
    :=
  selectFn r0.unpair.1
    (selectFn r1.unpair.1 (Nat.pair 1 (hUnion.unionIdx r0.unpair.2 r1.unpair.2)) r0)
    r1

namespace combineFound2
open NeighborhoodSystem Domain.Recursive ApproximableMap Exercise718

theorem found_iff {? : Type*} {W : NeighborhoodSystem ?}
    {Q : ComputablePresentation W} (hUnion : IsComputableUnion Q) {r0 r1 : Nat}
    (h0 : r0.unpair.1  <=  1) (h1 : r1.unpair.1  <=  1) :
    (combineFound2 hUnion r0 r1).unpair.1 = 1  <->  r0.unpair.1 = 1  \/  r1.unpair.1 =
     1 := by
  unfold combineFound2
  rcases Nat.eq_zero_or_pos r0.unpair.1 with hr0 | hr0
  * rw [hr0, selectFn_zero]; omega
  * rw [show r0.unpair.1 = 1 from by omega, selectFn_one]
    rcases Nat.eq_zero_or_pos r1.unpair.1 with hr1 | hr1
    * rw [hr1, selectFn_zero]; omega
    * rw [show r1.unpair.1 = 1 from by omega, selectFn_one, unpair_pair_fst]; omega

theorem mem_iff {? : Type*} {W : NeighborhoodSystem ?}
    {Q : ComputablePresentation W} (hpos : W.IsPositive) (hdiff : W.DiffClosed)
    (hnomin : W.NoMinimal) (hUnion : IsComputableUnion Q) {r0 r1 : Nat}
    (h0 : r0.unpair.1  <=  1) (h1 : r1.unpair.1  <=  1)
    (hmem0 : r0.unpair.1 = 1  ->  W.mem (Q.X r0.unpair.2))
    (hmem1 : r1.unpair.1 = 1  ->  W.mem (Q.X r1.unpair.2))
    (hfound : (combineFound2 hUnion r0 r1).unpair.1 = 1) (z : ?) :
    z  in  Q.X (combineFound2 hUnion r0 r1).unpair.2  <->
      (r0.unpair.1 = 1  /\  z  in  Q.X r0.unpair.2)  \/  (r1.unpair.1 = 1  /\  z  in
   Q.X r1.unpair.2) := by
  unfold combineFound2 at hfound  |-
```

```
  rcases Nat.eq_zero_or_pos r0.unpair.1 with hr0 | hr0
  * rw [hr0, selectFn_zero] at hfound  |-
    constructor
    * intro hz; exact Or.inr <hfound, hz>
    * rintro (<h, -> | <-, hz>)
      * omega
      * exact hz
  * rw [show r0.unpair.1 = 1 from by omega, selectFn_one] at hfound  |-
    have hmem0' := hmem0 (by omega)
    rcases Nat.eq_zero_or_pos r1.unpair.1 with hr1 | hr1
    * rw [hr1, selectFn_zero] at hfound  |-
      constructor
      * intro hz; exact Or.inl <by omega, hz>
      * rintro (<-, hz> | <h, ->)
        * exact hz
        * omega
    * rw [show r1.unpair.1 = 1 from by omega, selectFn_one, unpair_pair_snd]
      have hmem1' := hmem1 (by omega)
      have hex : exists  k, Q.X k = Q.X r0.unpair.2  U  Q.X r1.unpair.2 :=
        Q.surj (NeighborhoodSystem.mem_union_of_mem hpos hdiff hnomin hmem0' hmem1')
      rw [hUnion.unionIdx_spec hex, Set.mem_union]
      constructor
      * rintro (hz | hz)
        * exact Or.inl <by omega, hz>
        * exact Or.inr <by omega, hz>
      * rintro (<-, hz> | <-, hz>)
        * exact Or.inl hz
        * exact Or.inr hz

theorem mem_of_found {? : Type*} {W : NeighborhoodSystem ?}
    {Q : ComputablePresentation W} (hpos : W.IsPositive) (hdiff : W.DiffClosed)
    (hnomin : W.NoMinimal) (hUnion : IsComputableUnion Q) {r0 r1 : Nat}
    (h0 : r0.unpair.1  <=  1) (h1 : r1.unpair.1  <=  1)
    (hmem0 : r0.unpair.1 = 1  ->  W.mem (Q.X r0.unpair.2))
    (hmem1 : r1.unpair.1 = 1  ->  W.mem (Q.X r1.unpair.2)) :
    (combineFound2 hUnion r0 r1).unpair.1 = 1  ->
      W.mem (Q.X (combineFound2 hUnion r0 r1).unpair.2) := by
  unfold combineFound2
  rcases Nat.eq_zero_or_pos r0.unpair.1 with hr0 | hr0
  * rw [hr0, selectFn_zero]
    exact hmem1
  * rw [show r0.unpair.1 = 1 from by omega, selectFn_one]
    have hmem0' := hmem0 (by omega)
    rcases Nat.eq_zero_or_pos r1.unpair.1 with hr1 | hr1
    * rw [hr1, selectFn_zero]
      exact fun _ => hmem0'
    * rw [show r1.unpair.1 = 1 from by omega, selectFn_one, unpair_pair_snd]
      intro _
      have hmem1' := hmem1 (by omega)
      have hex : exists  k, Q.X k = Q.X r0.unpair.2  U  Q.X r1.unpair.2 :=
        Q.surj (NeighborhoodSystem.mem_union_of_mem hpos hdiff hnomin hmem0' hmem1')
      rw [hUnion.unionIdx_spec hex]
      exact NeighborhoodSystem.mem_union_of_mem hpos hdiff hnomin hmem0' hmem1'

theorem found_le_one {? : Type*} {W : NeighborhoodSystem ?}
    {Q : ComputablePresentation W} (hUnion : IsComputableUnion Q) {r0 r1 : Nat}
```

```
    (h0 : r0.unpair.1  <=  1) (h1 : r1.unpair.1  <=  1) :
    (combineFound2 hUnion r0 r1).unpair.1  <=  1 := by
  unfold combineFound2
  rcases Nat.eq_zero_or_pos r0.unpair.1 with hr0 | hr0
  * rw [hr0, selectFn_zero]; exact h1
  * rw [show r0.unpair.1 = 1 from by omega, selectFn_one]
    rcases Nat.eq_zero_or_pos r1.unpair.1 with hr1 | hr1
    * rw [hr1, selectFn_zero]; exact h0
    * rw [show r1.unpair.1 = 1 from by omega, selectFn_one, unpair_pair_fst]

end combineFound2

end IsComputableUnion

end DomainTheory.NeighbourhoodSystems
```

— `ComputableBisection` and bisection-derived splits

`ComputableBisection.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    IsComputableDiff
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    IsComputableSplit
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    domainIso812c
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Exercise_7_18
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Theorem_7_5

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive ApproximableMap Exercise718

structure ComputableBisection {? : Type*} {W : NeighborhoodSystem ?} (Q :
    ComputablePresentation W) where
  left : Nat  ->  Nat
  right : Nat  ->  Nat
  left_primrec : Nat.Primrec left
  right_primrec : Nat.Primrec right
  disjoint : forall  k, Q.X (left k)  I  Q.X (right k) = {}
  union : forall  k, Q.X (left k)  U  Q.X (right k) = Q.X k
  left_congr : forall  k k', Q.X k = Q.X k'  ->  Q.X (left k) = Q.X (left k')
  right_congr : forall  k k', Q.X k = Q.X k'  ->  Q.X (right k) = Q.X (right k')

namespace ComputableBisection

variable {a ? : Type*} {V : NeighborhoodSystem a} {W : NeighborhoodSystem ?}
  (P : ComputablePresentation V) (hDiff : IsComputableDiff P)
  {Q : ComputablePresentation W} (B : ComputableBisection Q) (hWnomin : W.NoMinimal)

open scoped Classical in
```

```
noncomputable def splitFromBisection (A : Set a) (B' : Set ?) (Xn : Set a) : Set ?  x
     Set ? :=
  if h : exists  n m k, A = P.X n  /\  B' = Q.X m  /\  Xn = P.X k then
    if emptyInterDec P (Nat.pair h.choose h.choose_spec.choose_spec.choose) = 1 then
      ({}, B')
    else if IsComputableDiff.emptyDiffDec P hDiff (Nat.pair h.choose h.choose_spec.
    choose_spec.choose) = 1 then
      (B', {})
    else
      (Q.X (B.left h.choose_spec.choose), Q.X (B.right h.choose_spec.choose))
  else
    splitChoice' W hWnomin A B' Xn

noncomputable def negIdxFromBisection (n m k : Nat) : Nat :=
  selectFn (emptyInterDec P (Nat.pair n k)) m
    (selectFn (IsComputableDiff.emptyDiffDec P hDiff (Nat.pair n k)) m (B.right m))

noncomputable def posIdxFromBisection (n m k : Nat) : Nat :=
  selectFn (emptyInterDec P (Nat.pair n k)) m
    (selectFn (IsComputableDiff.emptyDiffDec P hDiff (Nat.pair n k)) m (B.left m))

namespace emptyDiffDec

theorem congr {n n' k k' : Nat} (hdiffClosed : V.DiffClosed) (hnomin : V.NoMinimal)
    (hn : P.X n = P.X n') (hk : P.X k = P.X k') :
    IsComputableDiff.emptyDiffDec P hDiff (Nat.pair n k) =
      IsComputableDiff.emptyDiffDec P hDiff (Nat.pair n' k') := by
  have h1 := IsComputableDiff.emptyDiffDec.eq_one_iff P hDiff hdiffClosed hnomin n k
  have h2 := IsComputableDiff.emptyDiffDec.eq_one_iff P hDiff hdiffClosed hnomin n' k'
  rw [hn, hk] at h1
  have hiff : IsComputableDiff.emptyDiffDec P hDiff (Nat.pair n k) = 1  <->
      IsComputableDiff.emptyDiffDec P hDiff (Nat.pair n' k') = 1 :=
    h1.trans h2.symm
  have hle1 := IsComputableDiff.emptyDiffDec.le_one P hDiff (Nat.pair n k)
  have hle2 := IsComputableDiff.emptyDiffDec.le_one P hDiff (Nat.pair n' k')
  omega

end emptyDiffDec

namespace emptyInterDec

theorem congr {n n' k k' : Nat} (hpos : V.IsPositive) (hnomin : V.NoMinimal)
    (hn : P.X n = P.X n') (hk : P.X k = P.X k') :
    emptyInterDec P (Nat.pair n k) = emptyInterDec P (Nat.pair n' k') := by
  have h1 := emptyInterDec.eq_one_iff P hpos hnomin n k
  have h2 := emptyInterDec.eq_one_iff P hpos hnomin n' k'
  rw [hn, hk] at h1
  have hiff : emptyInterDec P (Nat.pair n k) = 1  <->  emptyInterDec P (Nat.pair n' k
    ') = 1 :=
    h1.trans h2.symm
  have hle1 := emptyInterDec.le_one P (Nat.pair n k)
  have hle2 := emptyInterDec.le_one P (Nat.pair n' k')
  omega

end emptyInterDec
```

```
namespace splitFromBisection

theorem eq (hpos : V.IsPositive) (hnomin : V.NoMinimal)
    (hdiffClosed : V.DiffClosed) (n m k : Nat) :
    splitFromBisection P hDiff B hWnomin (P.X n) (Q.X m) (P.X k) =
      if emptyInterDec P (Nat.pair n k) = 1 then ({}, Q.X m)
      else if IsComputableDiff.emptyDiffDec P hDiff (Nat.pair n k) = 1 then (Q.X m,
    {})
      else (Q.X (B.left m), Q.X (B.right m)) := by
  have hex : exists  n' m' k', P.X n = P.X n'  /\  Q.X m = Q.X m'  /\  P.X k = P.X k'
    :=
    <n, m, k, rfl, rfl, rfl>
  obtain <hn', hm', hk'> := hex.choose_spec.choose_spec.choose_spec
  have hInterEq : emptyInterDec P (Nat.pair hex.choose hex.choose_spec.choose_spec.
    choose) =
      emptyInterDec P (Nat.pair n k) := (emptyInterDec.congr P hpos hnomin hn' hk').
    symm
  have hDiffEq : IsComputableDiff.emptyDiffDec P hDiff
      (Nat.pair hex.choose hex.choose_spec.choose_spec.choose) =
      IsComputableDiff.emptyDiffDec P hDiff (Nat.pair n k) :=
    (emptyDiffDec.congr P hDiff hdiffClosed hnomin hn' hk').symm
  unfold splitFromBisection
  rw [dite_eq_left hex, hInterEq, hDiffEq]
  split_ifs with h1 h2
  * rfl
  * rfl
  * have hl := (B.left_congr m hex.choose_spec.choose hm').symm
    have hr := (B.right_congr m hex.choose_spec.choose hm').symm
    rw [hl, hr]

end splitFromBisection

namespace primrec

theorem negIdxFromBisection :
    Nat.Primrec (fun t => negIdxFromBisection P hDiff B
      t.unpair.1 t.unpair.2.unpair.1 t.unpair.2.unpair.2) := by
  have hn : Nat.Primrec (fun t : Nat => t.unpair.1) := Nat.Primrec.left
  have hm : Nat.Primrec (fun t : Nat => t.unpair.2.unpair.1) := Nat.Primrec.left.comp
    Nat.Primrec.right
  have hk : Nat.Primrec (fun t : Nat => t.unpair.2.unpair.2) := Nat.Primrec.right.comp
     Nat.Primrec.right
  have hnk : Nat.Primrec (fun t : Nat => Nat.pair t.unpair.1 t.unpair.2.unpair.2) :=
    hn.pair hk
  have hinter : Nat.Primrec
      (fun t : Nat => _root_.DomainTheory.NeighbourhoodSystems.emptyInterDec P
        (Nat.pair t.unpair.1 t.unpair.2.unpair.2)) :=
    (primrec.emptyInterDec P).comp hnk
  have hdiffD : Nat.Primrec
      (fun t : Nat => _root_.DomainTheory.NeighbourhoodSystems.IsComputableDiff.
    emptyDiffDec P hDiff
        (Nat.pair t.unpair.1 t.unpair.2.unpair.2)) :=
    (IsComputableDiff.primrec.emptyDiffDec P hDiff).comp hnk
  have hright : Nat.Primrec (fun t : Nat => B.right t.unpair.2.unpair.1) := B.
    right_primrec.comp hm
  have hinner : Nat.Primrec (fun t : Nat => selectFn
```

```
      (_root_.DomainTheory.NeighbourhoodSystems.IsComputableDiff.emptyDiffDec P hDiff
        (Nat.pair t.unpair.1 t.unpair.2.unpair.2)) t.unpair.2.unpair.1
      (B.right t.unpair.2.unpair.1)) :=
    primrec_selectFn hdiffD hm hright
  exact (primrec_selectFn hinter hm hinner).of_eq fun _ => rfl

theorem posIdxFromBisection :
    Nat.Primrec (fun t => posIdxFromBisection P hDiff B
      t.unpair.1 t.unpair.2.unpair.1 t.unpair.2.unpair.2) := by
  have hn : Nat.Primrec (fun t : Nat => t.unpair.1) := Nat.Primrec.left
  have hm : Nat.Primrec (fun t : Nat => t.unpair.2.unpair.1) := Nat.Primrec.left.comp
    Nat.Primrec.right
  have hk : Nat.Primrec (fun t : Nat => t.unpair.2.unpair.2) := Nat.Primrec.right.comp
     Nat.Primrec.right
  have hnk : Nat.Primrec (fun t : Nat => Nat.pair t.unpair.1 t.unpair.2.unpair.2) :=
    hn.pair hk
  have hinter : Nat.Primrec
      (fun t : Nat => _root_.DomainTheory.NeighbourhoodSystems.emptyInterDec P
        (Nat.pair t.unpair.1 t.unpair.2.unpair.2)) :=
    (primrec.emptyInterDec P).comp hnk
  have hdiffD : Nat.Primrec
      (fun t : Nat => _root_.DomainTheory.NeighbourhoodSystems.IsComputableDiff.
    emptyDiffDec P hDiff
        (Nat.pair t.unpair.1 t.unpair.2.unpair.2)) :=
    (IsComputableDiff.primrec.emptyDiffDec P hDiff).comp hnk
  have hleft : Nat.Primrec (fun t : Nat => B.left t.unpair.2.unpair.1) := B.
    left_primrec.comp hm
  have hinner : Nat.Primrec (fun t : Nat => selectFn
      (_root_.DomainTheory.NeighbourhoodSystems.IsComputableDiff.emptyDiffDec P hDiff
        (Nat.pair t.unpair.1 t.unpair.2.unpair.2)) t.unpair.2.unpair.1
      (B.left t.unpair.2.unpair.1)) :=
    primrec_selectFn hdiffD hm hleft
  exact (primrec_selectFn hinter hm hinner).of_eq fun _ => rfl

end primrec

namespace isComputableSplit

noncomputable def ofBisection (hpos : V.IsPositive) (hnomin : V.NoMinimal)
    (hdiffClosed : V.DiffClosed) :
    IsComputableSplit P Q (splitFromBisection P hDiff B hWnomin) where
  posIdx := posIdxFromBisection P hDiff B
  negIdx := negIdxFromBisection P hDiff B
  posIdx_primrec := primrec.posIdxFromBisection P hDiff B
  negIdx_primrec := primrec.negIdxFromBisection P hDiff B
  posIdx_spec n m k hne := by
    rw [splitFromBisection.eq P hDiff B hWnomin hpos hnomin hdiffClosed n m k] at hne
     |-
    split_ifs at hne  |-  with h1 h2
    * exact absurd rfl hne
    * have h1' : emptyInterDec P (Nat.pair n k) = 0 := by
        have := emptyInterDec.le_one P (Nat.pair n k); omega
      simp [posIdxFromBisection, h1', h2]
    * have h1' : emptyInterDec P (Nat.pair n k) = 0 := by
        have := emptyInterDec.le_one P (Nat.pair n k); omega
      have h2' : IsComputableDiff.emptyDiffDec P hDiff (Nat.pair n k) = 0 := by
```

```
        have := IsComputableDiff.emptyDiffDec.le_one P hDiff (Nat.pair n k); omega
      simp [posIdxFromBisection, h1', h2']
  negIdx_spec n m k hne := by
    rw [splitFromBisection.eq P hDiff B hWnomin hpos hnomin hdiffClosed n m k] at hne
     |-
    split_ifs at hne  |-  with h1 h2
    * simp [negIdxFromBisection, h1]
    * exact absurd rfl hne
    * have h1' : emptyInterDec P (Nat.pair n k) = 0 := by
        have := emptyInterDec.le_one P (Nat.pair n k); omega
      have h2' : IsComputableDiff.emptyDiffDec P hDiff (Nat.pair n k) = 0 := by
        have := IsComputableDiff.emptyDiffDec.le_one P hDiff (Nat.pair n k); omega
      simp [negIdxFromBisection, h1', h2']

end isComputableSplit

end ComputableBisection

end DomainTheory.NeighbourhoodSystems
```

Effective isomorphism (8.12(d)) The same back-and-forth is carried out on computable presentations: coded atom sequences, primitivity of the step maps, and the order isomorphism `domainIsoCode812d` lift to an `EffectiveIso`. The generic assembly is `effectiveIso812d`.

— coded pairing, primrec witnesses, and `effectiveIso812d`

`effectiveIso812d.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    domainIso812c
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.atomE
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.deltaPair
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.xStepG
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.yStepG
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.atomPairG
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    atomPairG_more
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    atomPairCodeState
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    atomPairJunk
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    xPseqAtomJunk
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    xPseqAtomIdx
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.XFold
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    YFoldInner
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.XPseqCode
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.YPseqCode
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    yPseqAtomJunk
```

```
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    yPseqAtomIdx
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    IsComputableDiff
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    IsComputableSplit
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    IsComputableUnion
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    ComputableBisection
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Exercise_7_18
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Theorem_7_5
public import Mathlib.Data.Nat.Bitwise
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Exercise_3_26
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    effectiveIso812d_early
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    effectiveIso812d_early_more
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    combinedCode
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    primrec812d
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    primrec812d_more
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    combinedCode_more
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.Udiff
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    domainIsoCode812d
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.toD0Code
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.toD1Code
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    combinedCodeCrossFamily
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.transfer
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.hcoreCode
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    yStepG_more
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    yPseqAtomIdx_more
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    xStepG_more
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    xPseqAtomIdx_more
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    YPseqCode_more
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    XPseqCode_more

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

section
```

```
open NeighborhoodSystem Domain.Recursive ApproximableMap Exercise718

open NeighborhoodSystem Domain.Recursive ApproximableMap Exercise718

open NeighborhoodSystem Domain.Recursive ApproximableMap Exercise718

open NeighborhoodSystem Domain.Recursive ApproximableMap Exercise718

section
open NeighborhoodSystem Domain.Recursive ApproximableMap Exercise718

def NeighborhoodSystem.UnionClosed {a : Type*} (D : NeighborhoodSystem a) : Prop :=
  forall  {X Y : Set a}, D.mem X  ->  D.mem Y  ->  D.mem (X  U  Y)

section InvMapComp812d

variable {a b : Type*} {D_0 : NeighborhoodSystem a} {D_1 : NeighborhoodSystem b}
  (P_0 : ComputablePresentation D_0) (P_1 : ComputablePresentation D_1)
  (hDiff0 : IsComputableDiff P_0) (hDiff1 : IsComputableDiff P_1)
  (splitX : Set a  ->  Set b  ->  Set a  ->  Set b  x  Set b) (hSplitX :
    IsComputableSplit P_0 P_1 splitX)
  (splitY : Set b  ->  Set a  ->  Set b  ->  Set a  x  Set a) (hSplitY :
    IsComputableSplit P_1 P_0 splitY)
  (hD_0pos : D_0.IsPositive) (hD_0diff : D_0.DiffClosed) (hD_0nomin : D_0.NoMinimal)
  (hxSplit : SplitSpec' D_1 splitX)
  (hD_1pos : D_1.IsPositive) (hD_1diff : D_1.DiffClosed) (hD_1nomin : D_1.NoMinimal)
  (hySplit : SplitSpec' D_0 splitY)
  (hD_0mne : D_0.master.Nonempty) (hD_1mne : D_1.master.Nonempty)
  (hUnion0 : IsComputableUnion P_0) (hUnion1 : IsComputableUnion P_1)
  (hX0 : P_0.X 0 = D_0.master) (hY0 : P_1.X 0 = D_1.master)

include hD_0pos hD_0diff hD_0nomin hxSplit hD_1pos hD_1diff hD_1nomin hySplit hD_0mne
    hD_1mne hUnion0
  hUnion1 hX0 hY0 in

theorem invMap_comp_toMap :
    (ofIso (domainIsoCode812d P_0 P_1 hDiff0 hDiff1 splitX hSplitX splitY hSplitY
    hD_0pos hD_0diff
      hD_0nomin hxSplit hD_1pos hD_1diff hD_1nomin hySplit hD_0mne hD_1mne hUnion0
    hUnion1 hX0
      hY0).symm).comp
    (ofIso (domainIsoCode812d P_0 P_1 hDiff0 hDiff1 splitX hSplitX splitY hSplitY
    hD_0pos hD_0diff
      hD_0nomin hxSplit hD_1pos hD_1diff hD_1nomin hySplit hD_0mne hD_1mne hUnion0
    hUnion1 hX0 hY0)) =
    idMap D_0 := by
  apply ext_of_toElementMap
  intro x
  rw [toElementMap_comp]
  show (ofIso (domainIsoCode812d P_0 P_1 hDiff0 hDiff1 splitX hSplitX splitY hSplitY
    hD_0pos hD_0diff
      hD_0nomin hxSplit hD_1pos hD_1diff hD_1nomin hySplit hD_0mne hD_1mne hUnion0
    hUnion1 hX0
      hY0).symm).toElementMap
    ((ofIso (domainIsoCode812d P_0 P_1 hDiff0 hDiff1 splitX hSplitX splitY hSplitY
    hD_0pos hD_0diff
```

```
      hD_0nomin hxSplit hD_1pos hD_1diff hD_1nomin hySplit hD_0mne hD_1mne hUnion0
    hUnion1 hX0
      hY0)).toElementMap x) = _
  rw [toElementMap_ofIso, toElementMap_ofIso, OrderIso.symm_apply_apply,
    toElementMap_idMap]

end InvMapComp812d

section ToMapComp812d

variable {a b : Type*} {D_0 : NeighborhoodSystem a} {D_1 : NeighborhoodSystem b}
  (P_0 : ComputablePresentation D_0) (P_1 : ComputablePresentation D_1)
  (hDiff0 : IsComputableDiff P_0) (hDiff1 : IsComputableDiff P_1)
  (splitX : Set a  ->  Set b  ->  Set a  ->  Set b  x  Set b) (hSplitX :
    IsComputableSplit P_0 P_1 splitX)
  (splitY : Set b  ->  Set a  ->  Set b  ->  Set a  x  Set a) (hSplitY :
    IsComputableSplit P_1 P_0 splitY)
  (hD_0pos : D_0.IsPositive) (hD_0diff : D_0.DiffClosed) (hD_0nomin : D_0.NoMinimal)
  (hxSplit : SplitSpec' D_1 splitX)
  (hD_1pos : D_1.IsPositive) (hD_1diff : D_1.DiffClosed) (hD_1nomin : D_1.NoMinimal)
  (hySplit : SplitSpec' D_0 splitY)
  (hD_0mne : D_0.master.Nonempty) (hD_1mne : D_1.master.Nonempty)
  (hUnion0 : IsComputableUnion P_0) (hUnion1 : IsComputableUnion P_1)
  (hX0 : P_0.X 0 = D_0.master) (hY0 : P_1.X 0 = D_1.master)

include hD_0pos hD_0diff hD_0nomin hxSplit hD_1pos hD_1diff hD_1nomin hySplit hD_0mne
    hD_1mne hUnion0
  hUnion1 hX0 hY0 in

theorem toMap_comp_invMap :
    (ofIso (domainIsoCode812d P_0 P_1 hDiff0 hDiff1 splitX hSplitX splitY hSplitY
    hD_0pos hD_0diff
      hD_0nomin hxSplit hD_1pos hD_1diff hD_1nomin hySplit hD_0mne hD_1mne hUnion0
    hUnion1 hX0 hY0)).comp
    (ofIso (domainIsoCode812d P_0 P_1 hDiff0 hDiff1 splitX hSplitX splitY hSplitY
    hD_0pos hD_0diff
      hD_0nomin hxSplit hD_1pos hD_1diff hD_1nomin hySplit hD_0mne hD_1mne hUnion0
    hUnion1 hX0
      hY0).symm) =
    idMap D_1 := by
  apply ext_of_toElementMap
  intro y
  rw [toElementMap_comp]
  show (ofIso (domainIsoCode812d P_0 P_1 hDiff0 hDiff1 splitX hSplitX splitY hSplitY
    hD_0pos hD_0diff
      hD_0nomin hxSplit hD_1pos hD_1diff hD_1nomin hySplit hD_0mne hD_1mne hUnion0
    hUnion1 hX0
      hY0)).toElementMap
    ((ofIso (domainIsoCode812d P_0 P_1 hDiff0 hDiff1 splitX hSplitX splitY hSplitY
    hD_0pos hD_0diff
      hD_0nomin hxSplit hD_1pos hD_1diff hD_1nomin hySplit hD_0mne hD_1mne hUnion0
    hUnion1 hX0
      hY0).symm).toElementMap y) = _
  rw [toElementMap_ofIso, toElementMap_ofIso, OrderIso.apply_symm_apply,
    toElementMap_idMap]
```

```
end ToMapComp812d

section EffectiveIso812d

variable {a b : Type*} {D_0 : NeighborhoodSystem a} {D_1 : NeighborhoodSystem b}
  (P_0 : ComputablePresentation D_0) (P_1 : ComputablePresentation D_1)
  (hDiff0 : IsComputableDiff P_0) (hDiff1 : IsComputableDiff P_1)
  (splitX : Set a  ->  Set b  ->  Set a  ->  Set b  x  Set b) (hSplitX :
    IsComputableSplit P_0 P_1 splitX)
  (splitY : Set b  ->  Set a  ->  Set b  ->  Set a  x  Set a) (hSplitY :
    IsComputableSplit P_1 P_0 splitY)
  (hD_0pos : D_0.IsPositive) (hD_0diff : D_0.DiffClosed) (hD_0nomin : D_0.NoMinimal)
  (hxSplit : SplitSpec' D_1 splitX)
  (hD_1pos : D_1.IsPositive) (hD_1diff : D_1.DiffClosed) (hD_1nomin : D_1.NoMinimal)
  (hySplit : SplitSpec' D_0 splitY)
  (hD_0mne : D_0.master.Nonempty) (hD_1mne : D_1.master.Nonempty)
  (hUnion0 : IsComputableUnion P_0) (hUnion1 : IsComputableUnion P_1)
  (hX0 : P_0.X 0 = D_0.master) (hY0 : P_1.X 0 = D_1.master)

include hD_0pos hD_0diff hD_0nomin hxSplit hD_1pos hD_1diff hD_1nomin hySplit hD_0mne
    hD_1mne hUnion0
  hUnion1 hX0 hY0 in

noncomputable def effectiveIso812d : EffectiveIso P_0 P_1 where
  toMap := ofIso (domainIsoCode812d P_0 P_1 hDiff0 hDiff1 splitX hSplitX splitY
    hSplitY hD_0pos
    hD_0diff hD_0nomin hxSplit hD_1pos hD_1diff hD_1nomin hySplit hD_0mne hD_1mne
    hUnion0 hUnion1 hX0 hY0)
  invMap := ofIso (domainIsoCode812d P_0 P_1 hDiff0 hDiff1 splitX hSplitX splitY
    hSplitY hD_0pos
    hD_0diff hD_0nomin hxSplit hD_1pos hD_1diff hD_1nomin hySplit hD_0mne hD_1mne
    hUnion0 hUnion1 hX0
    hY0).symm
  toMap_computable := domainIsoCode812d.isComputableMap P_0 P_1 hDiff0 hDiff1 splitX
    hSplitX splitY
    hSplitY hD_0pos hD_0diff hD_0nomin hxSplit hD_1pos hD_1diff hD_1nomin hySplit
    hD_0mne hD_1mne hUnion0
    hUnion1 hX0 hY0
  invMap_computable := domainIsoCode812d.symm_isComputableMap P_0 P_1 hDiff0 hDiff1
    splitX hSplitX
    splitY hSplitY hD_0pos hD_0diff hD_0nomin hxSplit hD_1pos hD_1diff hD_1nomin
    hySplit hD_0mne hD_1mne
    hUnion0 hUnion1 hX0 hY0
  left_inv := invMap_comp_toMap P_0 P_1 hDiff0 hDiff1 splitX hSplitX splitY hSplitY
    hD_0pos hD_0diff
    hD_0nomin hxSplit hD_1pos hD_1diff hD_1nomin hySplit hD_0mne hD_1mne hUnion0
    hUnion1 hX0 hY0
  right_inv := toMap_comp_invMap P_0 P_1 hDiff0 hDiff1 splitX hSplitX splitY hSplitY
    hD_0pos hD_0diff
    hD_0nomin hxSplit hD_1pos hD_1diff hD_1nomin hySplit hD_0mne hD_1mne hUnion0
    hUnion1 hX0 hY0

end EffectiveIso812d

section DomainIsoCode812d
```

```
variable {a b : Type*} {D_0 : NeighborhoodSystem a} {D_1 : NeighborhoodSystem b}
  (P_0 : ComputablePresentation D_0) (P_1 : ComputablePresentation D_1)
  (hDiff0 : IsComputableDiff P_0) (hDiff1 : IsComputableDiff P_1)
  (splitX : Set a  ->  Set b  ->  Set a  ->  Set b  x  Set b) (hSplitX :
    IsComputableSplit P_0 P_1 splitX)
  (splitY : Set b  ->  Set a  ->  Set b  ->  Set a  x  Set a) (hSplitY :
    IsComputableSplit P_1 P_0 splitY)
  (hD_0pos : D_0.IsPositive) (hD_0diff : D_0.DiffClosed) (hD_0nomin : D_0.NoMinimal)
  (hxSplit : SplitSpec' D_1 splitX)
  (hD_1pos : D_1.IsPositive) (hD_1diff : D_1.DiffClosed) (hD_1nomin : D_1.NoMinimal)
  (hySplit : SplitSpec' D_0 splitY)
  (hD_0mne : D_0.master.Nonempty) (hD_1mne : D_1.master.Nonempty)
  (hUnion0 : IsComputableUnion P_0) (hUnion1 : IsComputableUnion P_1)
  (hX0 : P_0.X 0 = D_0.master) (hY0 : P_1.X 0 = D_1.master)

include hDiff0 hDiff1 hSplitX hSplitY hD_0pos hD_0diff hD_0nomin hxSplit hD_1pos
    hD_1diff hD_1nomin
  hySplit hD_0mne hD_1mne hUnion0 hUnion1 hX0 hY0 in

theorem isomorphic_812d : D_0  ~= ? D_1 :=
  <domainIsoCode812d P_0 P_1 hDiff0 hDiff1 splitX hSplitX splitY hSplitY hD_0pos
    hD_0diff hD_0nomin
    hxSplit hD_1pos hD_1diff hD_1nomin hySplit hD_0mne hD_1mne hUnion0 hUnion1 hX0 hY0
    >

end DomainIsoCode812d

section EffectiveIso812d

variable {a b : Type*} {D_0 : NeighborhoodSystem a} {D_1 : NeighborhoodSystem b}
  (P_0 : ComputablePresentation D_0) (P_1 : ComputablePresentation D_1)
  (hDiff0 : IsComputableDiff P_0) (hDiff1 : IsComputableDiff P_1)
  (splitX : Set a  ->  Set b  ->  Set a  ->  Set b  x  Set b) (hSplitX :
    IsComputableSplit P_0 P_1 splitX)
  (splitY : Set b  ->  Set a  ->  Set b  ->  Set a  x  Set a) (hSplitY :
    IsComputableSplit P_1 P_0 splitY)
  (hD_0pos : D_0.IsPositive) (hD_0diff : D_0.DiffClosed) (hD_0nomin : D_0.NoMinimal)
  (hxSplit : SplitSpec' D_1 splitX)
  (hD_1pos : D_1.IsPositive) (hD_1diff : D_1.DiffClosed) (hD_1nomin : D_1.NoMinimal)
  (hySplit : SplitSpec' D_0 splitY)
  (hD_0mne : D_0.master.Nonempty) (hD_1mne : D_1.master.Nonempty)
  (hUnion0 : IsComputableUnion P_0) (hUnion1 : IsComputableUnion P_1)
  (hX0 : P_0.X 0 = D_0.master) (hY0 : P_1.X 0 = D_1.master)

include hDiff0 hDiff1 hSplitX hSplitY hD_0pos hD_0diff hD_0nomin hxSplit hD_1pos
    hD_1diff hD_1nomin
  hySplit hD_0mne hD_1mne hUnion0 hUnion1 hX0 hY0 in

theorem effectivelyIsomorphic_812d : EffectivelyIsomorphic P_0 P_1 :=
  <effectiveIso812d P_0 P_1 hDiff0 hDiff1 splitX hSplitX splitY hSplitY hD_0pos
    hD_0diff hD_0nomin
    hxSplit hD_1pos hD_1diff hD_1nomin hySplit hD_0mne hD_1mne hUnion0 hUnion1 hX0 hY0
    >
end EffectiveIso812d

end
```

```
open Domain.Recursive NeighborhoodSystem
```

Computable split of U against V (8.12(e)) With `U`'s computable difference and V's bisection in hand, `splitX812e` is the concrete `IsComputableSplit` of U probing V (`isComputableSplit.«812e»`).
— `U_isComputableDiff`, `splitX812e`, and `isComputableSplit.«812e»`

splitX812e.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.Udiff

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

/-- **`splitX812e`: the exercise's literal target split**, `U` as prober against `V`'s
    canonical
bisection. -/
noncomputable def splitX812e : Set ?  ->  Set Nat  ->  Set ?  ->  Set Nat  x  Set Nat
    :=
  ComputableBisection.splitFromBisection UComputablePresentation U_isComputableDiff
    B812e
    V.noMinimal

namespace isComputableSplit
open Domain.Recursive NeighborhoodSystem

/-- **Exercise 8.12(e), completed in full**: `splitX812e` satisfies `IsComputableSplit
    `.
`noncomputable def`, not `theorem` -- `IsComputableSplit` is a data-carrying structure
    (its
`posIdx`/`negIdx` fields are `Nat`-valued functions), mirroring `isComputableSplit.
    ofBisection`
itself (`(e)(c)(ii)`). -/
noncomputable def ?812e? :
    IsComputableSplit UComputablePresentation VComputablePresentation splitX812e :=
  ComputableBisection.isComputableSplit.ofBisection UComputablePresentation
    U_isComputableDiff
    B812e V.noMinimal U.isPositive U.noMinimal U.diffClosed

end isComputableSplit

end DomainTheory.NeighbourhoodSystems
end
```

Symmetric split and UV assembly (8.12(f)–(g)) The mirror split `splitX812f` (V probing U via `UBisection2`), both `SplitSpec'` halves, computable unions, and the final `EffectiveIso U V` are assembled as `effectiveIso812_UV`.
— «812f», `hxSplit812e`/`hySplit812f`, unions, and `effectiveIso812_UV`

effectiveIso812_UV.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.Vunion
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    effectiveIso812d
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Exercise_7_18
public import Mathlib.Combinatorics.Colex
public import Mathlib.Data.Nat.Bitwise
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

open Domain.Recursive NeighborhoodSystem Exercise718

/-- **Exercise 8.12, completed in full**: `U` and `V` are effectively isomorphic
neighbourhood systems. -/
noncomputable def effectiveIso812_UV : EffectiveIso UComputablePresentation
    VComputablePresentation :=
  effectiveIso812d UComputablePresentation VComputablePresentation U_isComputableDiff
    V.isComputableDiff splitX812e isComputableSplit.?812e? splitX812f
    isComputableSplit.?812f?
    U.isPositive U.diffClosed U.noMinimal hxSplit812e V.isPositive V.diffClosed V.
    noMinimal
    hySplit812f U_master_nonempty V.master_nonempty U_isComputableUnion V.
    isComputableUnion UX_zero
    VX_zero

end DomainTheory.NeighbourhoodSystems
```

The tip then simply pairs effective givenness of V with that isomorphism, which is Exercise 8.12 in Scott's sense.

— Scott tip `exercise_8_12`

Exercise_8_12.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    effectiveIso812_UV
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Exercise_7_18

@[expose] public section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem Exercise718

/-- **Exercise 8.12 (Scott 1981, PRG-19).** `V` is effectively given and effectively
    isomorphic
to `U`, hence another presentation of `U`. --/
theorem exercise_8_12 :
    V.IsEffectivelyGiven  /\
```

```
      EffectivelyIsomorphic UComputablePresentation VComputablePresentation :=
  <V.isEffectivelyGiven, <effectiveIso812_UV>>

end
```

3.9.15 Exercise 8.13 — Boolean-algebra filters

(For logicians.) Prove that the universal domain U is isomorphic to the domain of all proper filters of the free Boolean algebra on $\aleph_0$-generators (= the Lindenbaum algebra of propositional calculus). (For topologists.) Connect this

representation of $\mathcal{U}$ with the collection of non-empty open subsets of the product space $2^{\mathbf{N}}$ (= Cantor space).

Tip summary The tip is the isomorphism $U \cong V$, obtained from the effective iso of Exercise 8.12 together with the Boolean-algebra presentation of `V`.

Development Defines the Boolean algebra generated by bit-test generators.

GeneratedBy.lean

```
module

public import Mathlib.Algebra.BigOperators.Group.Finset.Basic
public import Mathlib.Data.Nat.Bitwise
public import Mathlib.Data.Nat.Bits
public import Mathlib.Data.Nat.BitIndices
public import Mathlib.Combinatorics.Colex

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

/-- The Boolean subalgebra of `Set Nat` generated by a family `g` of "generators": the
    smallest
family containing every `g i` and `Set.univ`, closed under ` I `, ` U `, and complement
    . (`{}` is
derivable: `univ.compl`.) -/
inductive GeneratedBy {a : Type*} (g : Nat  ->  Set a) : Set a  ->  Prop
  | of (i : Nat) : GeneratedBy g (g i)
  | univ : GeneratedBy g Set.univ
  | inter {X Y} : GeneratedBy g X  ->  GeneratedBy g Y  ->  GeneratedBy g (X  I  Y)
  | union {X Y} : GeneratedBy g X  ->  GeneratedBy g Y  ->  GeneratedBy g (X  U  Y)
  | compl {X} : GeneratedBy g X  ->  GeneratedBy g X?

/-- Build a natural number with a prescribed (finite) set of bits set -- the "witness"
     constructor
used both for `generator`'s `levelSet` realization and for the complement/independence
    proofs
below. -/
def bitmaskOf (s : Finset Nat) : Nat := ? j  in  s, 2 ^ j

/-- **`(? j  in  s, 2^j).testBit i  <->  i  in  s`.** The general "read off a
    prescribed finite bit pattern"
```

```
fact behind `upsample` (`Exercise812.lean`) -- isolated here for reuse. -/
theorem testBit_sum_two_pow {s : Finset Nat} {i : Nat} : (? j  in  s, 2 ^ j).testBit i
      <->  i  in  s := by
  rw [ <-  Nat.mem_bitIndices,  <-  List.mem_toFinset, Finset.
    toFinset_bitIndices_sum_two_pow]

@[simp] theorem testBit_bitmaskOf {s : Finset Nat} {i : Nat} : (bitmaskOf s).testBit i
      <->  i  in  s :=
  testBit_sum_two_pow

/-- The **`i`-th generator**: `{n | n.testBit i}`, the set of naturals whose `i`-th
    binary digit is
`1`. These are Scott's `?_0` propositional variables, realized concretely. -/
def generator (i : Nat) : Set Nat := {n : Nat | n.testBit i = true}

@[simp] theorem mem_generator {i n : Nat} : n  in  generator i  <->  n.testBit i =
    true := Iff.rfl

end DomainTheory.NeighbourhoodSystems
end
```

Independence and filter facts that identify `V` with filters of that algebra.

BooleanAlgebraFilters.lean

```
module

public import Mathlib.Topology.Constructions
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Exercise_8_12
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    GeneratedBy

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Theorem_7_6
public import Mathlib.Data.Nat.Bitwise

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap
open Domain.Recursive NeighborhoodSystem
open NeighborhoodSystem
/- concordance: source L5153 EXERCISE 8.13 Boolean-algebra filters -/
/-- **Exercise 8.13(a).** `U` is isomorphic to `V`'s domain of elements, and `V`'s
    domain of
elements is (`V.mem_iff_generatedBy`, `generators_independent`) *by construction* the
    domain of
proper filters of the free Boolean algebra on `?_0` generators. No new back-and-forth
    argument is
needed: `effectiveIso812_UV` (`Exercise812g4.lean`, already `Pass`) already supplies
    the
isomorphism (in fact an *effective* one, stronger than what `8.13` asks for). -/
theorem exercise813a : U  ~= ? V := <effectiveIso812_UV.toDomainIso>

set_option linter.unusedVariables false
```

```
theorem example_813_a : @DomainTheory.NeighbourhoodSystems.Isomorphic Rat Nat
    DomainTheory.NeighbourhoodSystems.U
  DomainTheory.NeighbourhoodSystems.V :=
  DomainTheory.NeighbourhoodSystems.exercise813a

open NeighborhoodSystem

theorem generator_nonempty (i : Nat) : (generator i).Nonempty := <2 ^ i, by simp>

/-- **The `generator i` are independent**: every finite Boolean combination -- assign
    each `i  in  s`
to `generator i` (if `? i`) or its complement (otherwise) -- is non-empty. This is the
    standard
combinatorial characterization of "freely generates a Boolean algebra": no relation
    among finitely
many generators is forced beyond what Boolean algebra alone requires. Combined with
`V.mem_iff_generatedBy`, this identifies the algebra generated by `generator` as *the*
    free Boolean
algebra on `?_0` generators (= the Lindenbaum algebra of propositional calculus,
    `8.13`'s own gloss
-- a standard citation, see `8.13(b)` for the optional literal syntactic bridge). -/
theorem generators_independent (s : Finset Nat) (? : Nat  ->  Bool) :
    (? i  in  s, if ? i then generator i else (generator i)?).Nonempty := by
  refine <bitmaskOf (s.filter (fun i => ? i)), ?_>
  simp only [Set.mem_iInter]
  intro i hi
  split_ifs with hb <;> simp [testBit_bitmaskOf, Finset.mem_filter, hi, hb]

/-- **`levelSet` is closed under complement** (within the "`{}`-or-genuine" dichotomy
    every closure
fact in this development uses): the bitmask that selects exactly the residues `< 2^k` *
    not*
selected by `m`. -/
theorem levelSet_compl (k m : Nat) :
    (levelSet k m)? = levelSet k (bitmaskOf ((Finset.range (2 ^ k)).filter
      (fun ? => not  m.testBit ?))) := by
  ext n
  simp only [Set.mem_compl_iff, mem_levelSet, testBit_bitmaskOf, Finset.mem_filter,
    Finset.mem_range]
  exact <fun h => <Nat.mod_lt n (Nat.two_pow_pos _), h>, fun h => h.2>

theorem V.compl_mem {X : Set Nat} (hX : V.mem X) : X? = {}  \/  V.mem X? := by
  obtain <k, m, rfl, -> := hX
  rw [levelSet_compl]
  rcases (levelSet _ _).eq_empty_or_nonempty with h | h
  * exact Or.inl h
  * exact Or.inr <k, _, rfl, h>

/-- **`n % 2^k`'s bit `i`** is `n`'s own bit `i` when `i < k` (the low bits are
    preserved by
reduction mod `2^k`), and `false` otherwise (nothing above bit `k-1` survives). Proved
    via the
division algorithm (`Nat.div_add_mod`) fed into `Nat.testBit_two_pow_mul_add`. -/
theorem testBit_mod_two_pow (n k i : Nat) :
    (n % 2 ^ k).testBit i = (decide (i < k) && n.testBit i) := by
```

```
  have hlt : n % 2 ^ k < 2 ^ k := Nat.mod_lt n (Nat.two_pow_pos k)
  have h := Nat.testBit_two_pow_mul_add (n / 2 ^ k) hlt i
  rw [Nat.div_add_mod] at h
  rcases Nat.lt_or_ge i k with hik | hik
  * simp only [hik, ite_true, decide_true, Bool.true_and] at h  |-
    exact h.symm
  * simp only [Nat.not_lt.mpr hik, decide_false, Bool.false_and]
    exact Nat.testBit_eq_false_of_lt (hlt.trans_le (Nat.pow_le_pow_right (by norm_num)
     hik))

/-- **Completeness**: every `V`-neighbourhood is generated by `generator`. `affine k ?`
     (a single
residue class) is a finite intersection of generators/their complements (one
    constraint per bit
`i < k`); `levelSet k m` is a finite union of those over the residues `?` selected by `
    m`
(`levelSet.eq_biUnion`), assembled by `Finset` induction using `GeneratedBy.union`. -/
theorem affine_eq_biInter (k ? : Nat) (h? : ? < 2 ^ k) :
    affine k ? = ? i  in  Finset.range k, (if ?.testBit i then generator i else (
    generator i)?) := by
  ext n
  simp only [mem_affine, Set.mem_iInter, Finset.mem_range]
  have key : n % 2 ^ k = ?  <->  forall  i < k, n.testBit i = ?.testBit i := by
    constructor
    * rintro rfl i hi
      rw [testBit_mod_two_pow]; simp [hi]
    * intro h
      apply Nat.eq_of_testBit_eq
      intro i
      rw [testBit_mod_two_pow]
      rcases Nat.lt_or_ge i k with hik | hik
      * simp [hik, h i hik]
      * simp only [Nat.not_lt.mpr hik, decide_false, Bool.false_and]
        exact (Nat.testBit_eq_false_of_lt (h?.trans_le (Nat.pow_le_pow_right (by
    norm_num) hik))).symm
  rw [key]
  refine forall_2_congr fun i _ => ?_
  split_ifs with hb <;> simp [hb]

theorem generatedBy_affine (k ? : Nat) (h? : ? < 2 ^ k) : GeneratedBy generator (
    affine k ?) := by
  rw [affine_eq_biInter k ? h?]
  induction Finset.range k using Finset.induction_on with
  | empty => simpa using GeneratedBy.univ
  | insert a s _ ih =>
    rw [Finset.set_biInter_insert]
    refine GeneratedBy.inter ?_ ih
    split_ifs with hb
    * exact GeneratedBy.of _
    * exact GeneratedBy.compl (GeneratedBy.of _)

theorem generatedBy_biUnion_affine (k : Nat) (s : Finset Nat) (hs : forall  ?  in  s,
    ? < 2 ^ k) :
    GeneratedBy generator (? ?  in  s, affine k ?) := by
  induction s using Finset.induction_on with
  | empty => simpa using GeneratedBy.univ.compl
```

```
  | insert a s ha ih =>
    rw [Finset.set_biUnion_insert]
    exact GeneratedBy.union (generatedBy_affine k a (hs a (Finset.mem_insert_self a s)
    ))
      (ih fun ? h? => hs ? (Finset.mem_insert_of_mem h?))

theorem generatedBy_levelSet (k m : Nat) : GeneratedBy generator (levelSet k m) := by
  rw [levelSet.eq_biUnion]
  exact generatedBy_biUnion_affine k _ fun ? h? => (Finset.mem_filter.mp h?).1 |>
    Finset.mem_range.mp

/-- Specialization of `testBit_mod_two_pow` used repeatedly below: reducing mod `2^(i
    +1)` never
disturbs bit `i` itself. -/
theorem testBit_mod_two_pow_succ (n i : Nat) :
    (n % 2 ^ (i + 1)).testBit i = n.testBit i := by
  rw [testBit_mod_two_pow]; simp

/-- Every `generator i` is a `V`-neighbourhood: `generator i = levelSet (i+1) M` for
    the bitmask
`M` selecting exactly the residues `< 2^(i+1)` whose own bit `i` is set. -/
theorem generator_eq_levelSet (i : Nat) :
    generator i = levelSet (i + 1) (bitmaskOf ((Finset.range (2 ^ (i + 1))).filter
      (fun ? => ?.testBit i))) := by
  ext n
  simp only [mem_generator, mem_levelSet, testBit_bitmaskOf, Finset.mem_filter, Finset
    .mem_range]
  rw [ <-  testBit_mod_two_pow_succ]
  exact <fun h => <Nat.mod_lt n (Nat.two_pow_pos _), h>, fun h => h.2>

theorem V.mem_generator (i : Nat) : V.mem (generator i) :=
  generator_eq_levelSet i  |>  <i + 1, _, rfl, generator_eq_levelSet i  |>
    generator_nonempty i>

/-- **Soundness**: everything generated by `generator` is either empty or a genuine
`V`-neighbourhood. -/
theorem generatedBy_generator_imp {X : Set Nat} (h : GeneratedBy generator X) :
    X = {}  \/  V.mem X := by
  induction h with
  | of i => exact Or.inr (V.mem_generator i)
  | univ => exact Or.inr V.master_mem
  | @inter X Y _ _ ih1 ih2 =>
    rcases ih1 with rfl | hX
    * exact Or.inl (Set.empty_inter Y)
    rcases ih2 with rfl | hY
    * exact Or.inl (Set.inter_empty X)
    obtain <k_1, m_1, rfl, -> := hX
    obtain <k_2, m_2, rfl, -> := hY
    rw [levelSet.inter]
    rcases (levelSet _ _).eq_empty_or_nonempty with h | h
    * exact Or.inl h
    * exact Or.inr <_, _, rfl, h>
  | union _ _ ih1 ih2 => exact V.union_mem ih1 ih2
  | @compl X _ ih =>
    rcases ih with rfl | hX
    * simp only [Set.compl_empty]
```

```
      exact Or.inr V.master_mem
    * exact V.compl_mem hX

/-- **`V.mem` is *exactly* "non-empty and generated by `generator`"** -- soundness
(`generatedBy_generator_imp`) plus completeness (`generatedBy_levelSet`). Since `V.
    Element`'s
defining conditions (`Basic.lean`'s `Element`: upward-closed, finite-meet-closed,
    contains the
top, and -- because `V.mem` excludes `{}` -- never contains the bottom) are precisely "
    proper filter
of `{X | GeneratedBy generator X}`", this makes **`V.Element` *by definition* the
    domain of proper
filters of the Boolean algebra generated by `generator`**. -/
theorem V.mem_iff_generatedBy {X : Set Nat} : V.mem X  <->  GeneratedBy generator X
    /\  X.Nonempty := by
  constructor
  * rintro <k, m, rfl, hne>
    exact <generatedBy_levelSet k m, hne>
  * rintro <hgen, hne>
    exact (generatedBy_generator_imp hgen).resolve_left hne.ne_empty

/-- A minimal propositional-logic syntax over `?_0` variables (indexed by `Nat`). -/
inductive Formula : Type
  | var (i : Nat) : Formula
  | bot : Formula
  | top : Formula
  | neg (phi : Formula) : Formula
  | and (phi psi : Formula) : Formula
  | or (phi psi : Formula) : Formula

/-- The *same* recursion, but interpreted in `Set Nat` via `generator` (`Exercise813a.
    lean`) instead
of an abstract Boolean -- i.e. the specific homomorphism `var i  |->  generator i`. -/
def evalSet : Formula  ->  Set Nat
  | .var i => generator i
  | .bot => {}
  | .top => Set.univ
  | .neg phi => (evalSet phi)?
  | .and phi psi => evalSet phi  I  evalSet psi
  | .or phi psi => evalSet phi  U  evalSet psi

/-- The standard truth-valuation semantics: `v : Nat  ->  Bool` assigns each variable a
     truth value. -/
def evalV (v : Nat  ->  Bool) : Formula  ->  Bool
  | .var i => v i
  | .bot => false
  | .top => true
  | .neg phi => !(evalV v phi)
  | .and phi psi => evalV v phi && evalV v psi
  | .or phi psi => evalV v phi || evalV v psi

/-- **Scott's Lindenbaum equivalence**: `phi ~ psi` iff they agree under every truth
    valuation. -/
def SemanticEquiv (phi psi : Formula) : Prop := forall  v, evalV v phi = evalV v psi

/-- Given any (finite) target set of variables and any valuation `v`, some `n : Nat`
```

```
    matches `v` on
exactly that set -- witnessed directly via `bitmaskOf` (`Exercise813a.lean`). -/
theorem exists_bitsOf_agree (S : Finset Nat) (v : Nat  ->  Bool) :
    exists  n : Nat, forall  i  in  S, n.testBit i = v i := by
  refine <bitmaskOf (S.filter (fun i => v i)), fun i hi => ?_>
  have key : (bitmaskOf (S.filter (fun i => v i))).testBit i = true  <->  v i = true
    := by
    rw [testBit_bitmaskOf]; simp [Finset.mem_filter, hi]
  rcases hv : v i with - | - <;> rcases hn : (bitmaskOf (S.filter (fun i => v i))).
    testBit i
    with - | - <;> simp_all

instance formulaSetoid : Setoid Formula where
  r := SemanticEquiv
  iseqv := <fun _ _ => rfl, fun h v => (h v).symm, fun h1 h2 v => (h1 v).trans (h2 v)>

/-- **The Lindenbaum algebra of propositional calculus**: `Formula` quotiented by
    semantic
equivalence. -/
def Lindenbaum : Type := Quotient formulaSetoid

/-- **The bridge between the two evaluations**: `evalSet` at `n : Nat` agrees with `
    evalV` at `n`'s
own bit-sequence `fun i => n.testBit i`. -/
theorem mem_evalSet_iff (n : Nat) (phi : Formula) :
    n  in  evalSet phi  <->  evalV (fun i => n.testBit i) phi = true := by
  induction phi with
  | var i => simp [evalSet, evalV]
  | bot => simp [evalSet, evalV]
  | top => simp [evalSet, evalV]
  | neg phi ih => simp [evalSet, evalV, ih]
  | and phi psi ihphi ihpsi => simp [evalSet, evalV, ihphi, ihpsi]
  | or phi psi ihphi ihpsi => simp [evalSet, evalV, ihphi, ihpsi]

/-- The (finite) set of variables occurring in a formula. -/
def vars : Formula  ->  Finset Nat
  | .var i => {i}
  | .bot => {}
  | .top => {}
  | .neg phi => vars phi
  | .and phi psi => vars phi  U  vars psi
  | .or phi psi => vars phi  U  vars psi

theorem evalV_eq_of_agree {phi : Formula} {v v' : Nat  ->  Bool} (h : forall  i  in
    vars phi, v i = v' i) :
    evalV v phi = evalV v' phi := by
  induction phi with
  | var i => exact h i (Finset.mem_singleton_self i)
  | bot => rfl
  | top => rfl
  | neg phi ih => simp only [evalV, ih h]
  | and phi psi ihphi ihpsi =>
    simp only [evalV, vars] at h  |-
    rw [ihphi (fun i hi => h i (Finset.mem_union_left _ hi)),
      ihpsi (fun i hi => h i (Finset.mem_union_right _ hi))]
  | or phi psi ihphi ihpsi =>
```

```
    simp only [evalV, vars] at h  |-
    rw [ihphi (fun i hi => h i (Finset.mem_union_left _ hi)),
      ihpsi (fun i hi => h i (Finset.mem_union_right _ hi))]

/-- **The one substantive lemma**: semantic equivalence (quantifying over *all*
    valuations) agrees
with `evalSet`-equality (which only ever "tests" the countably many bit-derived
    valuations) --
because a formula only mentions finitely many variables, any valuation can be swapped
    for a
matching bit-derived one on the (finite) variables that actually occur. -/
theorem semanticEquiv_iff_evalSet_eq {phi psi : Formula} :
    SemanticEquiv phi psi  <->  evalSet phi = evalSet psi := by
  constructor
  * intro h
    ext n
    rw [mem_evalSet_iff, mem_evalSet_iff, h]
  * intro h v
    obtain <n, hn> := exists_bitsOf_agree (vars phi  U  vars psi) v
    have h1 : evalV v phi = evalV (fun i => n.testBit i) phi :=
      evalV_eq_of_agree fun i hi => (hn i (Finset.mem_union_left _ hi)).symm
    have h2 : evalV v psi = evalV (fun i => n.testBit i) psi :=
      evalV_eq_of_agree fun i hi => (hn i (Finset.mem_union_right _ hi)).symm
    have hiff : evalV (fun i => n.testBit i) phi = true  <->  evalV (fun i => n.
    testBit i) psi = true := by
      rw [ <-  mem_evalSet_iff,  <-  mem_evalSet_iff, h]
    rw [h1, h2]
    rcases hphi : evalV (fun i => n.testBit i) phi with - | - <;>
      rcases hpsi : evalV (fun i => n.testBit i) psi with - | - <;> simp_all

/-- The canonical map to `Set Nat`, well-defined by `semanticEquiv_iff_evalSet_eq`. -/
def Lindenbaum.toSet : Lindenbaum  ->  Set Nat :=
  Quotient.lift evalSet fun _ _ h => semanticEquiv_iff_evalSet_eq.mp h

@[simp] theorem Lindenbaum.toSet_mk (phi : Formula) :
    Lindenbaum.toSet ?phi? = evalSet phi := rfl

/-- Semantic entailment: `phi  |=  psi` iff every valuation satisfying `phi` satisfies
    `psi`. -/
def Entails (phi psi : Formula) : Prop := forall  v, evalV v phi = true  ->  evalV v
    psi = true

/-- **`Lindenbaum.toSet` is injective.** -/
theorem Lindenbaum.toSet_injective : Function.Injective Lindenbaum.toSet := by
  intro x y
  induction x using Quotient.ind with
  | _ phi =>
    induction y using Quotient.ind with
    | _ psi =>
      intro h
      exact Quotient.sound (semanticEquiv_iff_evalSet_eq.mpr h)

/-- **The bijection is an order correspondence, not just a set bijection**: semantic
    entailment
matches ` subseteq ` of the corresponding `evalSet`s, by the same finitary argument as
`semanticEquiv_iff_evalSet_eq`. -/
```

```
theorem entails_iff_evalSet_subset {phi psi : Formula} :
    Entails phi psi  <->  evalSet phi  subseteq  evalSet psi := by
  constructor
  * intro h n hn
    rw [mem_evalSet_iff] at hn  |-
    exact h _ hn
  * intro h v hv
    obtain <n, hn> := exists_bitsOf_agree (vars phi  U  vars psi) v
    have h1 : evalV v phi = evalV (fun i => n.testBit i) phi :=
      evalV_eq_of_agree fun i hi => (hn i (Finset.mem_union_left _ hi)).symm
    have h2 : evalV v psi = evalV (fun i => n.testBit i) psi :=
      evalV_eq_of_agree fun i hi => (hn i (Finset.mem_union_right _ hi)).symm
    rw [h2,  <-  mem_evalSet_iff]
    exact h (by rw [mem_evalSet_iff,  <-  h1]; exact hv)

set_option linter.unusedVariables false

theorem example_813_b : forall  {phi psi : DomainTheory.NeighbourhoodSystems.Formula},
  Iff (DomainTheory.NeighbourhoodSystems.Entails phi psi)
    (@LE.le (Set Nat) (@Set.instLE Nat) (DomainTheory.NeighbourhoodSystems.evalSet phi
    )
      (DomainTheory.NeighbourhoodSystems.evalSet psi)) :=
  @DomainTheory.NeighbourhoodSystems.entails_iff_evalSet_subset

end DomainTheory.NeighbourhoodSystems
```

The tip then assembles those tools into Scott's statement.
— `exercise_8_13`

Exercise_8_13.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    BooleanAlgebraFilters

@[expose] public section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem

/-- **Exercise 8.13 (Scott 1981, PRG-19).** `U  ~=  V`, realizing filters of the free
    Boolean algebra on ?_0 generators. -/
theorem exercise_8_13 : U  ~= ? V := exercise813a

end
```

3.9.16 Exercise 8.14 — closure operators

A retraction $a : D \to D$ is called a *closure operator* iff $I_D \sqsubseteq a$. On a domain like $P\mathbf{N}$, give some examples of closure operators. (Hint: Close up a set of integers under addition. Is this continuous on $P\mathbf{N}$?) Prove in general for any closure $a : D \to D$ that the fixed-point set of a is always a finitary domain. (Hint: Show that the fixed-point set is closed under intersections and directed unions.) What are the finite elements of the fixed-point set?

Tip summary The tip states that the fixed-point set of any closure operator is a finitary domain.

Development Defines closure operators and shows `Fix(a)` is a finitary domain.

ClosureOperatorDomain.lean

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Definition_7_1

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    DedekindCutProjection
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_8
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_13

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem ApproximableMap
universe u
variable {a : Type u} {E : NeighborhoodSystem a}

/- concordance: source L5159 EXERCISE 8.14 closure operators -/
/-- **Exercise 8.14 (Scott 1981, PRG-19), the definition.** A retraction `a : E  ->  E`
     is a
*closure operator* iff `I_E  sqsub  a`, i.e. `a` is inflationary. -/
class ClosureOperator (a : ApproximableMap E E) : Prop where
  out : Retraction a  /\  idMap E  <=  a

section ClosureOperator

variable {a : ApproximableMap E E}

/-- **Compactness of a fixed point w.r.t. directed unions of `Fix(a)`.** Phrased via
    the ambient
`E.Element` order and `iSupDirected` (since `Fix(a)` is not itself a `
    NeighborhoodSystem.Element`
type), mirroring `Theorem85.lean`'s `IsCompactElt`. -/
def IsCompactFix (_h : ClosureOperator a) (y : FixSet a) : Prop :=
  forall  {I : Type u} [Nonempty I] (d : I  ->  FixSet a)
    (hdir : forall  i j, exists  k, (d i).1  <=  (d k).1  /\  (d j).1  <=  (d k).1),
    y.1  <=  iSupDirected (fun i => (d i).1) hdir  ->  exists  i, y.1  <=  (d i).1

end ClosureOperator

section FiniteDomain

variable {a : ApproximableMap E E} (h : ClosureOperator a)

/-- The compact ("finite") fixed points of `a`, as a subtype of `Fix(a)`. -/
def CompactFix (h : ClosureOperator a) := {y : FixSet a // IsCompactFix h y}

end FiniteDomain
```

```
section ClosureOperator

variable {a : ApproximableMap E E}

/-- **The bottom of `Fix(a)`**: `a(False)`. -/
def botFix (h : ClosureOperator a) : FixSet a :=
  <a.toElementMap False, toElementMap_idem h.out.1 False>

/-- **`a(False)` is the least element of `Fix(a)`**: `a(False)  sqsub  y` for every `y
     in  Fix(a)`. Since `False  sqsub  y`
always, monotonicity gives `a(False)  sqsub  a(y) = y`. -/
theorem botFix_le (h : ClosureOperator a) (y : FixSet a) : (botFix h).1  <=  y.1 := by
  have h1 : a.toElementMap False  <=  a.toElementMap y.1 := a.toElementMap_mono bot_le
  rwa [y.2] at h1

/-- `botFix` is compact: it is below *every* element of `Fix(a)` (`botFix_le`), so
    trivially below
any particular member of a (non-empty) directed family. -/
theorem isCompactFix_botFix (h : ClosureOperator a) : IsCompactFix h (botFix h) := by
  intro I _ d _ _
  obtain <i0> := (inferInstance : Nonempty I)
  exact <i0, botFix_le h (d i0)>

end ClosureOperator

section FiniteDomain

variable {a : ApproximableMap E E} (h : ClosureOperator a)

/-- The bottom compact fixed point `a(False)`. -/
def botK : CompactFix h := <botFix h, isCompactFix_botFix h>

end FiniteDomain

section ClosureOperator

variable {a : ApproximableMap E E}

/-- Unfolding `I_E  sqsub  a` at the element level (Theorem 3.13(i)): `y  sqsub  a(y)`
    for every `y`. -/
theorem le_toElementMap_of_isClosureOperator (h : ClosureOperator a) (y : E.Element) :
    y  <=  a.toElementMap y := by
  have h1 := (le_iff_toElementMap_le.mp h.out.2) y
  rwa [toElementMap_idMap] at h1

/-- **Finite joins of compacts are compact.** If `y_1, y_2  in  Fix(a)` are compact and
     bounded above
(in `E`), their join `a( sqsup {y_1,y_2})` (` sqsup ` = Exercise 1.27's `sSup`) is
    again compact: if it sits below
a directed union, so do `y_1` and `y_2` individually (they are ` sqsub  a( sqsup {y_1,
    y_2})` since ` sqsup   sqsub  a( sqsup )` by
inflation), compactness places each below some member of the family, directedness
    finds a common
one, and the join being *least* upper bound of `{y_1,y_2}` (composed with `a`)
    transfers the bound. -/
```

```
theorem isCompactFix_sup (h : ClosureOperator a) {y1 y2 : FixSet a}
    (hc1 : IsCompactFix h y1) (hc2 : IsCompactFix h y2)
    (hb : E.Bounded ({y1.1, y2.1} : Set E.Element)) :
    IsCompactFix h <a.toElementMap (E.sSup {y1.1, y2.1} hb), toElementMap_idem h.out.1
     _> := by
  intro I _ d hdir hle
  have hs_le : E.sSup {y1.1, y2.1} hb  <=  iSupDirected (fun i => (d i).1) hdir :=
    (le_toElementMap_of_isClosureOperator h _).trans hle
  have h1le : y1.1  <=  iSupDirected (fun i => (d i).1) hdir :=
    (E.le_sSup {y1.1, y2.1} hb (by simp)).trans hs_le
  have h2le : y2.1  <=  iSupDirected (fun i => (d i).1) hdir :=
    (E.le_sSup {y1.1, y2.1} hb (by simp)).trans hs_le
  obtain <i1, hi1> := hc1 d hdir h1le
  obtain <i2, hi2> := hc2 d hdir h2le
  obtain <k, hk1, hk2> := hdir i1 i2
  have hle1 : y1.1  <=  (d k).1 := hi1.trans hk1
  have hle2 : y2.1  <=  (d k).1 := hi2.trans hk2
  have hsup_le : E.sSup {y1.1, y2.1} hb  <=  (d k).1 :=
    E.sSup_le {y1.1, y2.1} hb (by rintro x (rfl | rfl) <;> assumption)
  refine <k, ?_>
  have h3 := a.toElementMap_mono hsup_le
  rwa [(d k).2] at h3

/-- **The key new fact.** `a(X)`, for any `E`-neighbourhood `X`, is compact w.r.t.
    directed unions
*within `Fix(a)`* -- even though it need not be principal in `E` itself. Proof: if
`a(X)  sqsub  ?_i d(i)` then (since `X  sqsub  a(X)` by inflation) `X  sqsub  ?_i d(i)
    `, so by compactness of `X`
in `E` (`principal_isCompactElt`), `X  sqsub  d(i)` for some `i`; applying `a` (monotone
    ) and using that
`d(i)` is already fixed gives `a(X)  sqsub  d(i)`. -/
theorem isCompactFix_toElementMap_principal (h : ClosureOperator a) {X : Set a} (hX :
    E.mem X) :
    IsCompactFix h <a.toElementMap (E.principal hX), toElementMap_idem h.out.1 _> :=
    by
  intro I _ d hdir hle
  have hle' : E.principal hX  <=  iSupDirected (fun i => (d i).1) hdir :=
    (le_toElementMap_of_isClosureOperator h (E.principal hX)).trans hle
  obtain <i, hi> := principal_isCompactElt hX (fun i => (d i).1) hdir hle'
  refine <i, ?_>
  have h2 := a.toElementMap_mono hi
  rwa [(d i).2] at h2

end ClosureOperator

section FiniteDomain

variable {a : ApproximableMap E E} (h : ClosureOperator a)

/-- The generator of a compact fixed point from an `E`-neighbourhood `X`. -/
def genK {X : Set a} (hX : E.mem X) : CompactFix h :=
  <<a.toElementMap (E.principal hX), toElementMap_idem h.out.1 _>,
    isCompactFix_toElementMap_principal h hX>

/-- The join of two compact fixed points bounded above by a common `y  in  Fix(a)`. -/
def joinCompactFix {k1 k2 : CompactFix h} {y : FixSet a}
```

```
    (hk1y : k1.1.1  <=  y.1) (hk2y : k2.1.1  <=  y.1) : CompactFix h :=
  <<a.toElementMap (E.sSup {k1.1.1, k2.1.1} <y.1, by rintro x (rfl | rfl) <;>
    assumption>),
      toElementMap_idem h.out.1 _>,
    isCompactFix_sup h k1.2 k2.2 <y.1, by rintro x (rfl | rfl) <;> assumption>>

theorem joinCompactFix_le {k1 k2 : CompactFix h} {y : FixSet a}
    (hk1y : k1.1.1  <=  y.1) (hk2y : k2.1.1  <=  y.1) : (joinCompactFix h hk1y hk2y)
    .1.1  <=  y.1 := by
  have hb : E.Bounded ({k1.1.1, k2.1.1} : Set E.Element) :=
    <y.1, by rintro x (rfl | rfl) <;> assumption>
  have hsup_le : E.sSup {k1.1.1, k2.1.1} hb  <=  y.1 :=
    E.sSup_le {k1.1.1, k2.1.1} hb (by rintro x (rfl | rfl) <;> assumption)
  have h1 := a.toElementMap_mono hsup_le
  rwa [y.2] at h1

/-- The principal up-set of a compact fixed point `k`: `{q ? k  sqsub  q}`. This is the
     "neighbourhood"
of `k` in the reconstructed system (smaller compacts give *bigger* up-sets, matching
    the
information-order convention `Factoid 1.7a`). -/
def upFix (k : CompactFix h) : Set (CompactFix h) := {q | k.1.1  <=  q.1.1}

@[simp] theorem mem_upFix {k q : CompactFix h} : q  in  upFix h k  <->  k.1.1  <=  q
    .1.1 := Iff.rfl

theorem self_mem_upFix (k : CompactFix h) : k  in  upFix h k := (mem_upFix h).mpr (
    le_refl _)

theorem upFix_antitone {k k' : CompactFix h} (hkk' : k.1.1  <=  k'.1.1) :
    upFix h k'  subseteq  upFix h k :=
  fun _ hq => hkk'.trans hq

/-- The generator of a non-empty up-set is unique. -/
theorem upFix_injOn {k k' : CompactFix h} (heq : upFix h k = upFix h k') : k.1.1 = k
    '.1.1 :=
  le_antisymm ((Set.ext_iff.mp heq k').mpr (self_mem_upFix h k'))
    ((Set.ext_iff.mp heq k).mp (self_mem_upFix h k))

/-- **The key algebraic identity**: the intersection of two principal up-sets, when
    jointly
bounded by some `y`, is again a principal up-set -- of the join. -/
theorem upFix_inter (k1 k2 : CompactFix h) {y : FixSet a}
    (hk1y : k1.1.1  <=  y.1) (hk2y : k2.1.1  <=  y.1) :
    upFix h k1  I  upFix h k2 = upFix h (joinCompactFix h hk1y hk2y) := by
  have hb : E.Bounded ({k1.1.1, k2.1.1} : Set E.Element) :=
    <y.1, by rintro x (rfl | rfl) <;> assumption>
  apply Set.ext
  intro q
  simp only [mem_upFix, Set.mem_inter_iff, joinCompactFix]
  constructor
  * rintro <h1q, h2q>
    have hb' : E.Bounded ({k1.1.1, k2.1.1} : Set E.Element) :=
      <q.1.1, by rintro x (rfl | rfl) <;> assumption>
    have hsup_le : E.sSup {k1.1.1, k2.1.1} hb'  <=  q.1.1 :=
      E.sSup_le {k1.1.1, k2.1.1} hb' (by rintro x (rfl | rfl) <;> assumption)
```

```
    have h3 := a.toElementMap_mono hsup_le
    rwa [q.1.2] at h3
  * intro hjq
    refine <?_, ?_>
    * exact ((E.le_sSup {k1.1.1, k2.1.1} hb (by simp)).trans
        (le_toElementMap_of_isClosureOperator h _)).trans hjq
    * exact ((E.le_sSup {k1.1.1, k2.1.1} hb (by simp)).trans
        (le_toElementMap_of_isClosureOperator h _)).trans hjq

theorem upFix_subset_upFix_botK (k : CompactFix h) : upFix h k  subseteq  upFix h (
    botK h) :=
  fun q _ => botFix_le h q.1

/-- **The reconstructed neighbourhood system.** Tokens are the compact fixed points of
    `a`;
neighbourhoods are their principal up-sets. -/
def domainOfClosure (h : ClosureOperator a) : NeighborhoodSystem (CompactFix h) where
  mem S := exists  k : CompactFix h, S = upFix h k
  master := upFix h (botK h)
  master_nonempty := <botK h, self_mem_upFix h (botK h)>
  master_mem := <botK h, rfl>
  sub_master := by rintro S <k, rfl>; exact upFix_subset_upFix_botK h k
  inter_mem := by
    rintro S1 S2 Z <k1, rfl> <k2, rfl> <k3, rfl> hZsub
    obtain <hk13, hk23> := hZsub (self_mem_upFix h k3)
    exact <joinCompactFix h hk13 hk23, upFix_inter h k1 k2 hk13 hk23>

/-- The compact fixed points confirmed by a filter `g` of `domainOfClosure h`. -/
def confirmedIdx (g : (domainOfClosure h).Element) := {k : CompactFix h // g.mem (
    upFix h k)}

instance instNonemptyConfirmedIdx (g : (domainOfClosure h).Element) :
    Nonempty (confirmedIdx h g) :=
  <<botK h, g.master_mem>>

theorem confirmedIdx_directed (g : (domainOfClosure h).Element) :
    forall  i j : confirmedIdx h g, exists  k : confirmedIdx h g,
      i.1.1.1  <=  k.1.1.1  /\  j.1.1.1  <=  k.1.1.1 := by
  rintro <k1, hk1> <k2, hk2>
  have hint : g.mem (upFix h k1  I  upFix h k2) := g.inter_mem hk1 hk2
  obtain <k3, hk3eq> := g.sub hint
  rw [hk3eq] at hint
  have hmem3 : k3  in  upFix h k1  I  upFix h k2 := hk3eq  |>  self_mem_upFix h k3
  exact <<k3, hint>, hmem3.1, hmem3.2>

/-- **The backward map `|domainOfClosure h|  ->  Fix(a)`**: the directed union of all
    confirmed
compacts. -/
def fromElementFilter (g : (domainOfClosure h).Element) : FixSet a :=
  <iSupDirected (fun i : confirmedIdx h g => i.1.1.1) (confirmedIdx_directed h g),
    toElementMap_iSupDirected_fixed _ (confirmedIdx_directed h g) (fun i => i.1.1.2)>

/-- The family `a(X)`, for `X` ranging over the (principal-approximant) neighbourhoods
    of `y.1`. -/
def genFamily (y : FixSet a) (i : {X : Set a // y.1.mem X}) : FixSet a :=
  (genK h (y.1.sub i.2)).1
```

```
theorem genFamily_directed (y : FixSet a) :
    forall  i j : {X : Set a // y.1.mem X}, exists  k, (genFamily h y i).1  <=  (
    genFamily h y k).1  /\
      (genFamily h y j).1  <=  (genFamily h y k).1 := by
  intro i j
  obtain <k, hik, hjk> := principalFamily_directed y.1 i j
  exact <k, a.toElementMap_mono hik, a.toElementMap_mono hjk>

/-- **The forward map `Fix(a)  ->  |domainOfClosure h|`**: `y  |->  {S ? S = k  /\  k
    sqsub  y}`. -/
def toElementFilter (y : FixSet a) : (domainOfClosure h).Element where
  mem S := exists  k : CompactFix h, S = upFix h k  /\  k.1.1  <=  y.1
  sub := fun <k, hSeq, _> => <k, hSeq>
  master_mem := <botK h, rfl, botFix_le h y>
  inter_mem := by
    rintro S1 S2 <k1, rfl, hk1y> <k2, rfl, hk2y>
    exact <joinCompactFix h hk1y hk2y, upFix_inter h k1 k2 hk1y hk2y,
    joinCompactFix_le h hk1y hk2y>
  up_mem := by
    rintro S1 S2 <k1, rfl, hk1y> hS2mem hsub
    obtain <k2, rfl> := hS2mem
    exact <k2, rfl, (mem_upFix h |>.mp (hsub (self_mem_upFix h k1))).trans hk1y>

theorem le_of_toElementFilter_le {y y' : FixSet a}
    (hle : toElementFilter h y  <=  toElementFilter h y') : y.1  <=  y'.1 := by
  intro X hXy
  have hX : E.mem X := y.1.sub hXy
  have hky : (toElementFilter h y).mem (upFix h (genK h hX)) := by
    refine <genK h hX, rfl, ?_>
    have h1 : E.principal hX  <=  y.1 := principal_le_of_mem hXy
    have h2 := a.toElementMap_mono h1
    rwa [y.2] at h2
  obtain <k', hkk', hk'y'> := hle _ hky
  have hkeq : (genK h hX).1.1 = k'.1.1 := upFix_injOn h hkk'
  have h1 : E.principal hX  <=  a.toElementMap (E.principal hX) :=
    le_toElementMap_of_isClosureOperator h _
  rw [show a.toElementMap (E.principal hX) = k'.1.1 from hkeq] at h1
  exact (h1.trans hk'y') X ((E.mem_principal hX).mpr <hX, subset_rfl>)

theorem toElementFilter_fromElementFilter (g : (domainOfClosure h).Element) :
    toElementFilter h (fromElementFilter h g) = g := by
  apply Element.ext
  intro S
  constructor
  * rintro <k0, rfl, hk0>
    obtain <i, hi> := k0.2 (fun i : confirmedIdx h g => i.1.1) (confirmedIdx_directed
    h g) hk0
    exact g.up_mem i.2 <k0, rfl> (upFix_antitone h hi)
  * intro hS
    obtain <k0, rfl> := g.sub hS
    exact <k0, rfl, le_iSupDirected _ (confirmedIdx_directed h g) <k0, hS>>

theorem toElementFilter_mono {y y' : FixSet a} (hyy' : y.1  <=  y'.1) :
    toElementFilter h y  <=  toElementFilter h y' :=
  fun _ <k, hSeq, hky> => <k, hSeq, hky.trans hyy'>
```

```
/-- **`y` is the directed union of `a(X)` for `X  in  y`.** Apply `a` (continuous, `
    toElementMap_
iSupDirected`) to `E`'s own algebraicity decomposition (`eq_iSupDirected_principal`) of
     `y.1`, then
use `a(y.1) = y.1`. -/
theorem toElementMap_eq_iSupDirected_genFamily (y : FixSet a) :
    y.1 = iSupDirected (fun i => (genFamily h y i).1) (genFamily_directed h y) := by
  have h1 : a.toElementMap y.1 = iSupDirected (fun i => (genFamily h y i).1) (
    genFamily_directed h y) := by
    conv_lhs => rw [eq_iSupDirected_principal y.1]
    exact toElementMap_iSupDirected a _ (principalFamily_directed y.1)
  rw [y.2] at h1
  exact h1

theorem fromElementFilter_toElementFilter (y : FixSet a) :
    fromElementFilter h (toElementFilter h y) = y := by
  apply Subtype.ext
  show iSupDirected (fun i : confirmedIdx h (toElementFilter h y) => i.1.1.1)
      (confirmedIdx_directed h (toElementFilter h y)) = y.1
  apply le_antisymm
  * apply iSupDirected_le
    rintro <k, k', hkk', hk'y>
    exact (upFix_injOn h hkk'  |>  hk'y : k.1.1  <=  y.1)
  * rw [toElementMap_eq_iSupDirected_genFamily h y]
    apply iSupDirected_le
    intro i
    have hk : (toElementFilter h y).mem (upFix h (genK h (y.1.sub i.2))) := by
      refine <genK h (y.1.sub i.2), rfl, ?_>
      have h1 : E.principal (y.1.sub i.2)  <=  y.1 := principal_le_of_mem i.2
      have h2 := a.toElementMap_mono h1
      rwa [y.2] at h2
    exact le_iSupDirected (fun i : confirmedIdx h (toElementFilter h y) => i.1.1.1)
      (confirmedIdx_directed h (toElementFilter h y)) <genK h (y.1.sub i.2), hk>

/-- **Exercise 8.14, the domain isomorphism.** `Fix(a)` is order-isomorphic to
`|domainOfClosure h|`. -/
def fixSetOrderIso (h : ClosureOperator a) : FixSet a ?o (domainOfClosure h).Element
    where
  toFun := toElementFilter h
  invFun := fromElementFilter h
  left_inv := fromElementFilter_toElementFilter h
  right_inv := toElementFilter_fromElementFilter h
  map_rel_iff' := by
    intro y y'
    exact <le_of_toElementFilter_le h, toElementFilter_mono h>

/-- **Exercise 8.14, the final question, answered exactly.** `y  in  Fix(a)` is compact
     (i.e. is a
*finite element* of the fixed-point domain) **iff** `y = a(X)` for a single `E`-
    neighbourhood `X`.
The forward direction is a sandwich argument: `y` is the directed union of the compacts
     `a(X)`
(`X  in  y`, `toElementMap_eq_iSupDirected_genFamily`), so compactness pulls out a
    single witness `X`
with `y  sqsub  a(X)`; the reverse inequality `a(X)  sqsub  y` is automatic since `X
```

```
    in  y`. -/
theorem isCompactFix_iff (y : FixSet a) :
    IsCompactFix h y  <->  exists  (X : Set a) (hX : E.mem X), y.1 = a.toElementMap (E
    .principal hX) := by
  constructor
  * intro hc
    obtain <i, hi> := hc (genFamily h y) (genFamily_directed h y)
      (le_of_eq (toElementMap_eq_iSupDirected_genFamily h y))
    have hX : E.mem i.1 := y.1.sub i.2
    refine <i.1, hX, le_antisymm hi ?_>
    have h1 : E.principal hX  <=  y.1 := principal_le_of_mem i.2
    have h2 := a.toElementMap_mono h1
    rwa [y.2] at h2
  * rintro <X, hX, hy>
    exact hy  |>  isCompactFix_toElementMap_principal h hX

/-- **Exercise 8.14 (Scott 1981, PRG-19), the main theorem.** For any closure operator
`a : E  ->  E`, the fixed-point set of `a` is a finitary domain. -/
theorem isFinitary_of_isClosureOperator (h : ClosureOperator a) : IsFinitary a :=
  <CompactFix h, domainOfClosure h, <fixSetOrderIso h>>

end FiniteDomain

end

set_option linter.unusedVariables false

theorem example_814.{u} : And
  (forall  {a : Type u} {E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
    {a : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a E E}
    (h : @DomainTheory.NeighbourhoodSystems.ClosureOperator a E a)
    (y : @DomainTheory.NeighbourhoodSystems.FixSet a E a),
    Iff (@DomainTheory.NeighbourhoodSystems.IsCompactFix a E a h y)
      (exists  (X : Set a) (hX : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
    .mem a E X),
        @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E)
          (@Subtype.val (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
     a E)
            (fun (y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
    E) =>
              @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E)
                (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a E
     E a y) y)
            y)
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a E E a
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.principal a E X hX)
    )))
  (forall  {a : Type u} {E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
    {a : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a E E},
    @DomainTheory.NeighbourhoodSystems.ClosureOperator a E a  ->  @DomainTheory.
    NeighbourhoodSystems.IsFinitary a E a) :=
  @And.intro
  (forall  {a : Type u} {E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
    {a : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a E E}
    (h : @DomainTheory.NeighbourhoodSystems.ClosureOperator a E a)
    (y : @DomainTheory.NeighbourhoodSystems.FixSet a E a),
```

```
    Iff (@DomainTheory.NeighbourhoodSystems.IsCompactFix a E a h y)
      (exists  (X : Set a) (hX : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
    .mem a E X),
        @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E)
          (@Subtype.val (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element
     a E)
            (fun (y : @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a
    E) =>
              @Eq (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.Element a E)
                (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a E
     E a y) y)
            y)
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap.toElementMap a a E E a
            (@DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.principal a E X hX)
    )))
  (forall  {a : Type u} {E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}

    {a : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a E E},
    @DomainTheory.NeighbourhoodSystems.ClosureOperator a E a  ->  @DomainTheory.
    NeighbourhoodSystems.IsFinitary a E a)
  @DomainTheory.NeighbourhoodSystems.isCompactFix_iff @DomainTheory.
    NeighbourhoodSystems.isFinitary_of_isClosureOperator

end DomainTheory.NeighbourhoodSystems
```

The tip then assembles those tools into Scott's statement.

— exercise_8_14

Exercise_8_14.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    ClosureOperatorDomain

@[expose] public section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap

/-- **Exercise 8.14 (Scott 1981, PRG-19).** The fixed-point set of a closure operator
    is a finitary domain. -/
theorem exercise_8_14 {a : Type*} {E : NeighborhoodSystem a} {a : ApproximableMap E E}
    (h : ClosureOperator a) : IsFinitary a :=
  isFinitary_of_isClosureOperator h

end
```

3.9.17 Exercise 8.15 — domain of subsystems

Give a direct proof that the domain $\{X \mid X \triangleleft D\}$ is effectively presented if D is. (Hint: The finite elements of the domain correspond exactly to the finite systems $X \triangleleft D$.) In the case of $D = \mathcal{U}$, show that the computable elements of the domain correspond exactly to the effectively presented domains (up to effective isomorphism).

Tip summary The tip states that if `D` is effectively given then so is (a presentation of) the domain of its subsystems.

Development Library tools: finitely generated subsystems and the reconstructed system `SubD`.
— `finGen`, `finGenSys`, `SubD`

`finGen.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Theorem_7_5

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem Domain.Recursive

variable {a : Type*} {D : NeighborhoodSystem a}
variable (P : ComputablePresentation D)

/- concordance: source L5161 EXERCISE 8.15 domain of subsystems -- Fin(js) / SubD -/

/-- The sub-selection of `js` consisting of entries belonging to `sub1` or `sub2` --
    the `List`-level
"join" of two sub-selections, used to combine two consistent sub-meets into a third.
    -/
def combineSub (js sub1 sub2 : List Nat) : List Nat :=
  js.filter (fun j => decide (j  in  sub1  \/  j  in  sub2))

theorem combineSub_sublist (js sub1 sub2 : List Nat) :
    (combineSub js sub1 sub2).Sublist js := List.filter_sublist

theorem mem_combineSub {js sub1 sub2 : List Nat} {j : Nat} :
    j  in  combineSub js sub1 sub2  <->  j  in  js  /\  (j  in  sub1  \/  j  in  sub2)
     := by
  unfold combineSub
  rw [List.mem_filter, decide_eq_true_iff]

/-- **Combining two consistent sub-meets.** If `sub1, sub2  sqsub  js`, then `interFrom
     P D.master
(combineSub js sub1 sub2) = interFrom P D.master sub1  I  interFrom P D.master sub2` --
     a pure
set-theoretic fact about `interFrom`'s membership characterization (no consistency
    needed here). -/
theorem interFrom_combineSub_eq {js sub1 sub2 : List Nat}
    (h1 : sub1.Sublist js) (h2 : sub2.Sublist js) :
    interFrom P D.master (combineSub js sub1 sub2) =
      interFrom P D.master sub1  I  interFrom P D.master sub2 := by
  apply Set.ext
  intro z
  rw [Set.mem_inter_iff, mem_interFrom, mem_interFrom, mem_interFrom]
  constructor
  * rintro <hzm, hall>
```

```
    exact <<hzm, fun j hj => hall j (mem_combineSub.mpr <h1.subset hj, Or.inl hj>)>,
      hzm, fun j hj => hall j (mem_combineSub.mpr <h2.subset hj, Or.inr hj>)>
  * rintro <<hzm, hall1>, _, hall2>
    refine <hzm, fun j hj => ?_>
    obtain <_, hj1 | hj2> := mem_combineSub.mp hj
    * exact hall1 j hj1
    * exact hall2 j hj2

/-- **`Fin(js)`**: the finitely generated subsystem of `D` on the index list `js` --
    the set of
`D`-consistent "sub-meets" `interFrom P D.master sub` for sublists `sub  sqsub  js`.
    Every member is
automatically a `D`-neighbourhood (the `D.mem Y` conjunct), so `finGen P js  subseteq
    D.mem` by
construction. -/
def finGen (js : List Nat) : Set (Set a) :=
  {Y | exists  sub : List Nat, sub.Sublist js  /\  D.mem Y  /\  Y = interFrom P D.
    master sub}

/-- **Monotonicity.** A sublist generates a smaller (or equal) finitely generated
    subsystem. -/
theorem finGen_mono {js js' : List Nat} (h : js.Sublist js') : finGen P js  subseteq
    finGen P js' :=
  fun _ <sub, hsub, hD, hY> => <sub, hsub.trans h, hD, hY>

/-- Every member of `Fin(js)` is a `D`-neighbourhood. -/
theorem finGen_sub_mem {js : List Nat} {Y : Set a} : Y  in  finGen P js  ->  D.mem Y
    :=
  fun <_, _, hD, _> => hD

/-- Every member of `Fin(js)` is a subset of `D.master`. -/
theorem finGen_sub_master {js : List Nat} {Y : Set a} (h : Y  in  finGen P js) : Y
    subseteq  D.master :=
  D.sub_master (finGen_sub_mem P h)

/-- `D.master  in  Fin(js)` always (witness: the empty sub-selection). -/
theorem master_mem_finGen (js : List Nat) : D.master  in  finGen P js :=
  <[], List.nil_sublist js, D.master_mem, rfl>

/-- The single-index generator `P.X i` lies in `Fin(js)` whenever `i  in  js` (witness:
     the singleton
sub-selection `[i]`, always `D`-consistent since `interFrom P D.master [i] = D.master
    I  P.X i =
P.X i` using `P.X i  subseteq  D.master`). -/
theorem principal_mem_finGen {js : List Nat} {i : Nat} (hi : i  in  js) : P.X i  in
    finGen P js := by
  refine <[i], (List.singleton_sublist).mpr hi, P.mem_X i, ?_>
  apply Set.ext
  intro z
  rw [mem_interFrom]
  constructor
  * intro hz
    exact <D.sub_master (P.mem_X i) hz,
      fun j hj => by rw [List.mem_singleton] at hj; subst hj; exact hz>
  * rintro <_, hall>
    exact hall i (List.mem_singleton.mpr rfl)
```

```
/-- **`Fin(js)` as a `NeighborhoodSystem`.** Master is `D.master`; closure under
    consistent
intersection combines the two witnessing sub-selections via `combineSub`. -/
def finGenSys (js : List Nat) : NeighborhoodSystem a where
  mem Y := Y  in  finGen P js
  master := D.master
  master_nonempty := D.master_nonempty
  master_mem := master_mem_finGen P js
  inter_mem := by
    rintro X Y Z <sub1, h1, hDX, hX> <sub2, h2, hDY, hY> <sub3, h3, hDZ, hZ> hZsub
    exact <combineSub js sub1 sub2, combineSub_sublist js sub1 sub2,
      D.inter_mem hDX hDY hDZ hZsub, by
        rw [interFrom_combineSub_eq P h1 h2, hX, hY]>
  sub_master := fun <sub, _, _, hY> => hY  |>  interFrom_subset P

@[simp] theorem mem_finGenSys {js : List Nat} {Y : Set a} :
    (finGenSys P js).mem Y  <->  Y  in  finGen P js := Iff.rfl

/-- **`Fin(js)` is a subsystem of `D`** (Definition 6.10). -/
theorem finGenSys_subsystem (js : List Nat) : finGenSys P js ? D where
  master_eq := rfl
  sub := finGen_sub_mem P
  inter_closed := by
    rintro X Y <sub1, h1, hDX, hX> <sub2, h2, hDY, hY> hXY
    exact <combineSub js sub1 sub2, combineSub_sublist js sub1 sub2, hXY, by
      rw [interFrom_combineSub_eq P h1 h2, hX, hY]>

/-- `interFrom P A (j :: sub)` splits off the head as an intersection with `P.X j` -- a
     pure
set-theoretic fact from `mem_interFrom`'s characterization. -/
theorem interFrom_cons_eq (A : Set a) (j : Nat) (sub : List Nat) :
    interFrom P A (j :: sub) = P.X j  I  interFrom P A sub := by
  apply Set.ext
  intro z
  rw [Set.mem_inter_iff, mem_interFrom, mem_interFrom, List.forall_mem_cons]
  tauto

/-- **The workhorse lemma.** For any subsystem `D' ? D`, `Fin(js)  subseteq  D'.mem`
    iff every single
generator `P.X i` (`i  in  js`) already lies in `D'.mem`. Proved by induction on `js`:
    the `cons` step
uses `interFrom_mem_of_witness` (`Theorem75.lean`) to show that the "tail" sub-meet of
    a consistent
`j :: sub`-selection is itself consistent (using the *full* meet as witness), then
    combines via
`D'.inter_closed`. -/
theorem finGen_le_iff_forall_mem {D' : NeighborhoodSystem a} (hD' : D' ? D) (js : List
     Nat) :
    finGen P js  subseteq  D'.mem  <->  forall  i  in  js, D'.mem (P.X i) := by
  induction js with
  | nil =>
    refine <fun _ i hi => absurd hi List.not_mem_nil, fun _ => ?_>
    rintro Y <sub, hsub, _, hYeq>
    rw [List.eq_nil_of_sublist_nil hsub] at hYeq
    rw [hYeq, show interFrom P D.master ([] : List Nat) = D.master from rfl,  <-  hD'.
```

```
    master_eq]
    exact D'.master_mem
  | cons j js' ih =>
    rw [List.forall_mem_cons]
    constructor
    * intro hsub
      refine <hsub (principal_mem_finGen P List.mem_cons_self), ?_>
      exact ih.mp ((finGen_mono P (List.sublist_cons_self j js')).trans hsub)
    * intro hall
      have hj := hall.1
      have hFinjs' : finGen P js'  subseteq  D'.mem := ih.mpr hall.2
      rintro Y <sub, hsub, hDY, hYeq>
      cases hsub with
      | cons _ hsub' => exact hFinjs' <sub, hsub', hDY, hYeq>
      | cons_cons _ hsub' =>
        rename_i sub2
        rw [interFrom_cons_eq P D.master j sub2] at hYeq
        set Y2 := interFrom P D.master sub2 with hY2def
        have hY_sub_Y2 : Y  subseteq  Y2 := hYeq  |>  Set.inter_subset_right
        have hDY2 : D.mem Y2 := interFrom_mem_of_witness P hDY hY_sub_Y2 D.master_mem
        have hY2mem : D'.mem Y2 := hFinjs' <sub2, hsub', hDY2, rfl>
        have hXYmem : D.mem (P.X j  I  Y2) := hYeq  |>  hDY
        exact hYeq  |>  hD'.inter_closed hj hY2mem hXYmem

/-- **The concatenation identity, general form.** For any subsystem `D' ? D`,
`Fin(js_1 ++ js_2)  subseteq  D'.mem  <->  Fin(js_1)  subseteq  D'.mem  /\  Fin(js_2)
    subseteq  D'.mem` -- an immediate corollary of the
workhorse lemma via `List.forall_mem_append`. -/
theorem finGen_append_le_iff {D' : NeighborhoodSystem a} (hD' : D' ? D) (js1 js2 :
    List Nat) :
    finGen P (js1 ++ js2)  subseteq  D'.mem  <->  finGen P js1  subseteq  D'.mem  /\
    finGen P js2  subseteq  D'.mem := by
  rw [finGen_le_iff_forall_mem P hD', finGen_le_iff_forall_mem P hD',
    finGen_le_iff_forall_mem P hD', List.forall_mem_append]

/-- **`nbhd(js)`**: the (anti-monotone) principal up-set of finite generators `js'`
    whose finitely
generated subsystem extends `Fin(js)` -- the neighbourhood of `SubD` generated by `js`.
     -/
def nbhdGen (js : List Nat) : Set (List Nat) := {js' | finGen P js  subseteq  finGen P
     js'}

@[simp] theorem mem_nbhdGen {js js' : List Nat} :
    js'  in  nbhdGen P js  <->  finGen P js  subseteq  finGen P js' := Iff.rfl

/-- **Anti-monotonicity.** A bigger generator (`Fin`-wise) has a smaller up-set. -/
theorem nbhdGen_antitone {js js' : List Nat} (h : finGen P js  subseteq  finGen P js')
     :
    nbhdGen P js'  subseteq  nbhdGen P js := fun _ hk => h.trans hk

theorem self_mem_nbhdGen (js : List Nat) : js  in  nbhdGen P js := (mem_nbhdGen P).mpr
     subset_rfl

/-- **The concatenation identity for `nbhd`.** `nbhd(js_1)  I  nbhd(js_2) = nbhd(js_1
    ++ js_2)` --
`finGen_append_le_iff` instantiated at `D' := finGenSys(js')` for each candidate `js'`.
```

```
    -/
theorem nbhdGen_inter (js1 js2 : List Nat) :
    nbhdGen P js1  I  nbhdGen P js2 = nbhdGen P (js1 ++ js2) := by
  apply Set.ext
  intro js'
  simp only [mem_nbhdGen, Set.mem_inter_iff]
  exact (finGen_append_le_iff P (finGenSys_subsystem P js') js1 js2).symm

/-- **`SubD`: the reconstructed neighbourhood system.** Tokens are finite index lists
    (`List Nat`,
representing finitely generated subsystems of `D`); neighbourhoods are the principal up
    -sets
`nbhd(js)`. -/
def SubD (P : ComputablePresentation D) : NeighborhoodSystem (List Nat) where
  mem N := exists  js, N = nbhdGen P js
  master := nbhdGen P []
  master_nonempty := <[], self_mem_nbhdGen P []>
  master_mem := <[], rfl>
  inter_mem := by
    rintro N1 N2 N3 <js1, rfl> <js2, rfl> <js3, rfl> _
    exact <js1 ++ js2, nbhdGen_inter P js1 js2>
  sub_master := by
    rintro N <js, rfl>
    exact nbhdGen_antitone P (finGen_mono P (List.nil_sublist js))

/-- **Injectivity of `nbhd` on `Fin`.** `nbhd(a) = nbhd(b)  ->  Fin(a) = Fin(b)`:
    evaluate the set
equality at `a` and at `b` (both reflexively self-membership) and combine by
    antisymmetry. -/
theorem finGen_eq_of_nbhdGen_eq {a b : List Nat} (h : nbhdGen P a = nbhdGen P b) :
    finGen P a = finGen P b :=
  Set.Subset.antisymm ((Set.ext_iff.mp h b).mpr (self_mem_nbhdGen P b))
    ((Set.ext_iff.mp h a).mp (self_mem_nbhdGen P a))

/-- **`nbhd` equality is exactly `Fin` equality.** -/
theorem nbhdGen_eq_iff {a b : List Nat} : nbhdGen P a = nbhdGen P b  <->  finGen P a =
     finGen P b := by
  refine <finGen_eq_of_nbhdGen_eq P, fun h => ?_>
  unfold nbhdGen
  rw [h]

@[simp] theorem mem_SubD {N : Set (List Nat)} : (SubD P).mem N  <->  exists  js, N =
    nbhdGen P js := Iff.rfl

@[simp] theorem SubD_master : (SubD P).master = nbhdGen P [] := rfl

/-! ## Subsystem  <->  SubD.Element (order-iso building blocks) -/

/-- **The finite-fold lemma.** If every generator `P.X i` (`i  in  js`) is individually
     confirmed by
`x` via some witness list, the witnesses can be concatenated (fold over the finite list
     `js`,
combining via `x.inter_mem`) into a *single* list `js_0` confirmed by `x` with `Fin(js)
      subseteq
Fin(js_0)`. -/
theorem exists_common_witness (x : (SubD P).Element) {js : List Nat}
```

```
    (h : forall  i  in  js, exists  js', x.mem (nbhdGen P js')  /\  P.X i  in  finGen
    P js') :
    exists  js0, x.mem (nbhdGen P js0)  /\  finGen P js  subseteq  finGen P js0 := by
  induction js with
  | nil => exact <[], SubD_master P  |>  x.master_mem, subset_rfl>
  | cons j js' ih =>
    obtain <jsj, hxj, hjmem> := h j List.mem_cons_self
    obtain <js0', hx0', hsub0'> := ih (fun i hi => h i (List.mem_cons_of_mem j hi))
    refine <jsj ++ js0', nbhdGen_inter P jsj js0'  |>  x.inter_mem hxj hx0',
      (finGen_le_iff_forall_mem P (finGenSys_subsystem P (jsj ++ js0')) (j :: js')).
    mpr
        (List.forall_mem_cons.mpr <?_, ?_>)>
    * exact finGen_mono P (List.sublist_append_left jsj js0') hjmem
    * exact fun i hi =>
        finGen_mono P (List.sublist_append_right jsj js0') (hsub0' (
    principal_mem_finGen P hi))

/-- **`interFrom` splits over list concatenation.** A pure membership-characterization
    fact
(`mem_interFrom` + `List.forall_mem_append`), independent of consistency. -/
theorem interFrom_append_eq (A : Set a) (l1 l2 : List Nat) :
    interFrom P A (l1 ++ l2) = interFrom P A l1  I  interFrom P A l2 := by
  apply Set.ext
  intro z
  rw [Set.mem_inter_iff, mem_interFrom, mem_interFrom, mem_interFrom, List.
    forall_mem_append]
  tauto

/-- **Forward direction: a subsystem induces a filter of `SubD`.** Given `D' ? D`, the
    filter
`ofSubsystem P D' hD'` consists of the neighbourhoods `nbhd(js)` such that `D'` already
     contains
every singleton generator `P.X i` for `i  in  js`. -/
def ofSubsystem (D' : NeighborhoodSystem a) (hD' : D' ? D) : (SubD P).Element where
  mem N := exists  js, N = nbhdGen P js  /\  forall  i  in  js, D'.mem (P.X i)
  sub := fun <js, hN, _> => <js, hN>
  master_mem := <[], rfl, fun i hi => absurd hi List.not_mem_nil>
  inter_mem := by
    rintro N1 N2 <js1, rfl, h1> <js2, rfl, h2>
    exact <js1 ++ js2, nbhdGen_inter P js1 js2, List.forall_mem_append.mpr <h1, h2>>
  up_mem := by
    rintro N1 N2 <js1, rfl, h1> <js2, rfl> hsub
    have hle : finGen P js2  subseteq  finGen P js1 := hsub (self_mem_nbhdGen P js1)
    exact <js2, rfl, (finGen_le_iff_forall_mem P hD' js2).mp
      (hle.trans ((finGen_le_iff_forall_mem P hD' js1).mpr h1))>

/-- **A reusable membership characterization**, generalizing the forward half of
`toSubsystem_ofSubsystem_mem`: for *any* `D' ? D`, `ofSubsystem P D' hD'` confirms `
    nbhd(js)`
exactly when `D'` confirms every generator in `js`. -/
theorem mem_ofSubsystem_nbhdGen {D' : NeighborhoodSystem a} (hD' : D' ? D) (js : List
    Nat) :
    (ofSubsystem P D' hD').mem (nbhdGen P js)  <->  forall  i  in  js, D'.mem (P.X i)
    := by
  constructor
  * rintro <js', hjseq, hall>
```

```
    have hfin : finGen P js = finGen P js' := (nbhdGen_eq_iff P).mp hjseq
    refine (finGen_le_iff_forall_mem P hD' js).mp ?_
    rw [hfin]
    exact (finGen_le_iff_forall_mem P hD' js').mpr hall
  * exact fun hall => <js, rfl, hall>

/-- **Backward direction: a filter of `SubD` induces a subsystem.** The neighbourhoods
    are the
`Fin(js)`-members for lists `js` confirmed by the filter `x`. -/
def toSubsystemSys (x : (SubD P).Element) : NeighborhoodSystem a where
  mem Y := exists  js, x.mem (nbhdGen P js)  /\  Y  in  finGen P js
  master := D.master
  master_nonempty := D.master_nonempty
  master_mem := <[], x.master_mem, master_mem_finGen P []>
  inter_mem := by
    rintro X Y Z <js1, hx1, hX> <js2, hx2, hY> <js3, _, hZ> hZsub
    have hDX := finGen_sub_mem P hX
    have hDY := finGen_sub_mem P hY
    have hDZ := finGen_sub_mem P hZ
    obtain <sub1, hsub1, _, hXeq> := hX
    obtain <sub2, hsub2, _, hYeq> := hY
    refine <js1 ++ js2, nbhdGen_inter P js1 js2  |>  x.inter_mem hx1 hx2,
      sub1 ++ sub2, hsub1.append hsub2, D.inter_mem hDX hDY hDZ hZsub, ?_>
    rw [interFrom_append_eq, hXeq, hYeq]
  sub_master := fun <_, _, hY> => finGen_sub_master P hY

/-- **`toSubsystemSys x` is a genuine subsystem of `D`.** Uses the same `
    interFrom_append_eq`
witness-combination argument as `inter_mem`, without needing the extra `Z`-witness. -/
theorem toSubsystemSys_subsystem (x : (SubD P).Element) : toSubsystemSys P x ? D where
  master_eq := rfl
  sub := fun <_, _, hY> => finGen_sub_mem P hY
  inter_closed := by
    rintro X Y <js1, hx1, hX> <js2, hx2, hY> hXY
    obtain <sub1, hsub1, _, hXeq> := hX
    obtain <sub2, hsub2, _, hYeq> := hY
    refine <js1 ++ js2, nbhdGen_inter P js1 js2  |>  x.inter_mem hx1 hx2,
      sub1 ++ sub2, hsub1.append hsub2, hXY, ?_>
    rw [interFrom_append_eq, hXeq, hYeq]

/-- **Round trip 2.** `ofSubsystem P (toSubsystem P x).1 (toSubsystem P x).2 = x`. -/
theorem ofSubsystem_toSubsystem (x : (SubD P).Element) :
    ofSubsystem P (toSubsystemSys P x) (toSubsystemSys_subsystem P x) = x := by
  apply Element.ext
  intro N
  constructor
  * rintro <js, rfl, hall>
    obtain <js0, hx0, hsub0> := exists_common_witness P x hall
    exact x.up_mem hx0 <js, rfl> (nbhdGen_antitone P hsub0)
  * intro hx
    obtain <js, rfl> := x.sub hx
    exact <js, rfl, fun i hi => <js, hx, principal_mem_finGen P hi>>

/-- **Membership transfer, filter form.** Combines round trip 2 with `
    mem_ofSubsystem_nbhdGen`:
a filter `x` confirms `nbhd(js)` exactly when the reconstructed subsystem `
```

```
    toSubsystemSys P x`
confirms every generator in `js`. The key bridge for `map_rel_iff'` below. -/
theorem mem_nbhdGen_iff (x : (SubD P).Element) (js : List Nat) :
    x.mem (nbhdGen P js)  <->  forall  i  in  js, (toSubsystemSys P x).mem (P.X i) :=
    by
  conv_lhs => rw [ <-  ofSubsystem_toSubsystem P x]
  exact mem_ofSubsystem_nbhdGen P (toSubsystemSys_subsystem P x) js

/-- **The backward map, packaged as a subsystem.** -/
def toSubsystem (x : (SubD P).Element) : {D' : NeighborhoodSystem a // D' ? D} :=
  <toSubsystemSys P x, toSubsystemSys_subsystem P x>

/-- **Round trip, membership form.** `toSubsystemSys (ofSubsystem D')` has exactly the
    same
neighbourhoods as `D'` itself. ` -> `: unwind the double existential via `
    nbhdGen_eq_iff` and the
workhorse lemma. ` <- `: `P.surj` finds an index `n` with `P.X n = Y`; the singleton
    list `[n]`
witnesses membership on both sides. -/
theorem toSubsystem_ofSubsystem_mem (D' : NeighborhoodSystem a) (hD' : D' ? D) (Y :
    Set a) :
    (toSubsystemSys P (ofSubsystem P D' hD')).mem Y  <->  D'.mem Y := by
  constructor
  * rintro <js, <js', hjseq, hall>, hY>
    have hfin : finGen P js = finGen P js' := (nbhdGen_eq_iff P).mp hjseq
    exact (finGen_le_iff_forall_mem P hD' js').mpr hall (hfin  |>  hY)
  * intro hY
    obtain <n, hn> := P.surj (hD'.sub hY)
    refine <[n], <[n], rfl, fun i hi => ?_>, hn  |>  principal_mem_finGen P (List.
    mem_singleton.mpr rfl)>
    rw [List.mem_singleton] at hi
    subst hi
    rw [hn]
    exact hY

/-- **Round trip 1.** `toSubsystem P (ofSubsystem P D' hD') = <D', hD'>`. -/
theorem toSubsystem_ofSubsystem (D' : NeighborhoodSystem a) (hD' : D' ? D) :
    toSubsystem P (ofSubsystem P D' hD') = <D', hD'> :=
  Subtype.ext (NeighborhoodSystem.ext (toSubsystem_ofSubsystem_mem P D' hD') hD'.
    master_eq.symm)

end DomainTheory.NeighbourhoodSystems
```

— QBody, QFuel, TreeVal, Q′X

TreeVal.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.finGen
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Theorem_8_8
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Exercise_7_18
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Theorem_7_5
```

```
public import Mathlib.Data.Nat.Bitwise

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive

/- concordance: source L5161 EXERCISE 8.15 -- QFuel / TreeVal / Q'X -/

/-- One fuel-recursion step for evaluating a term-algebra code: tag `0` (`leaf j`)
    applies `r` to
the payload; tag `1` (`node t1 t2`) combines the two (previous-fuel-level) recursive
    values via
`Uinter`; any other tag (`master`, or junk) falls back to `?`'s master index. Mirrors
`decodeFuelOkCharBody`'s tag-dispatch shape exactly. -/
def QBody (r : Nat  ->  Nat) (prev : Nat  ->  Nat) (c : Nat) : Nat :=
  selectFn (isOne (1 - c.unpair.1)) (r c.unpair.2)
    (selectFn (isOne (2 - c.unpair.1))
      (Uinter (prev c.unpair.2.unpair.1) (prev c.unpair.2.unpair.2))
      UComputablePresentation.masterIdx)

/-- `QBody`'s value at `c` depends on `prev` only through its values at `c`'s two (`Nat.
    unpair`)
children -- the locality fact needed to bridge two different `prev`s that merely *agree
    * there. -/
theorem QBody_local (r : Nat  ->  Nat) (f g : Nat  ->  Nat) (c : Nat)
    (h1 : f c.unpair.2.unpair.1 = g c.unpair.2.unpair.1)
    (h2 : f c.unpair.2.unpair.2 = g c.unpair.2.unpair.2) :
    QBody r f c = QBody r g c := by
  unfold QBody
  rw [h1, h2]

/-- `QFuel r fuel c`: fuel-indexed evaluation of a term-algebra code (base value `0` at
     `fuel = 0`,
irrelevant since every use below supplies `fuel > c`). -/
def QFuel (r : Nat  ->  Nat) : Nat  ->  Nat  ->  Nat
  | 0, _ => 0
  | fuel + 1, c => QBody r (QFuel r fuel) c

/-- `QBody`'s step with `prev` replaced by table lookup (`fuelTable`'s calling
    convention). -/
def QLookup (r : Nat  ->  Nat) (w : Nat) : Nat := QBody r (fun c' => nthCode w.unpair
    .1 c' 0) w.unpair.2

theorem QFuel_fuelTable_eq (r : Nat  ->  Nat) :
    forall  fuel bound c, c  <=  bound  ->
      nthCode (fuelTable (QLookup r) bound fuel) c 0 = QFuel r fuel c :=
  fuelTable_eq_of_recursion (F := QFuel r) (Body := QBody r) (fun _ => rfl) (fun _ _
    => rfl)
    (fun table c => by unfold QLookup QBody; simp only [unpair_pair_fst,
    unpair_pair_snd])
    (fun f g c h => QBody_local r f g c
      (h c.unpair.2.unpair.1 (le_trans (unpair_left_le c.unpair.2) (unpair_snd_le c)))
      (h c.unpair.2.unpair.2 (le_trans (unpair_snd_le c.unpair.2) (unpair_snd_le c))))

/-- `foldCode`'s per-step function for `Q'X`: combine the running accumulator with `r`
```

```
    applied to
the next list entry. Packaged in `foldCode`'s `pair x (pair acc params)` calling
    convention
(`params` unused). -/
def QstepFn (r : Nat  ->  Nat) (t : Nat) : Nat := Uinter t.unpair.2.unpair.1 (r t.
    unpair.1)

/-- The raw `U`-index computed by folding `Uinter  comp  r` over the list coded by `n`,
     starting from
`U`'s master index. -/
def Q'Xcode (r : Nat  ->  Nat) (n : Nat) : Nat :=
  foldCode (QstepFn r) 0 UComputablePresentation.masterIdx n

/-- **`Q'X r`: the candidate enumeration of `D'x`'s neighbourhoods.** -/
def Q'X (r : Nat  ->  Nat) (n : Nat) : Set ? := UX (Q'Xcode r n)

/-- `Q'X r n` is `UX` at the `idxchain`-index of the `r`-image of `n`'s list -- i.e. `Q'
    X r` really is
`Theorem75`'s generic `idxchain` fold, just specialized through `r`. -/
theorem Q'X_eq (r : Nat  ->  Nat) (n : Nat) :
    Q'X r n = UX (idxchain UComputablePresentation (List.map r (decodeList n))) := by
  have hstep : (fun (acc x : Nat) => QstepFn r (Nat.pair x (Nat.pair acc 0)))
      = fun acc x => Uinter acc (r x) := by
    funext acc x; unfold QstepFn; simp only [unpair_pair_fst, unpair_pair_snd]
  unfold Q'X Q'Xcode idxchain
  rw [foldCode_eq', hstep, List.foldl_map]
  rfl

/-- **`TreeVal r c`: the fuel-free evaluator** (fuel `c + 1` is always enough, `c`
    being an upper
bound on any code's own recursion depth). -/
def TreeVal (r : Nat  ->  Nat) (c : Nat) : Nat := QFuel r (c + 1) c

/-- **Tag-`0` unfolding, for an arbitrary code (not just a literal `leafCode`).**
    Needed for the
`mem_X` induction below, which case-splits on an arbitrary code's tag rather than
    reconstructing it
as `leafCode`/`nodeCode`/`masterCode`. -/
theorem TreeVal_of_tag0 (r : Nat  ->  Nat) {c : Nat} (h : c.unpair.1 = 0) : TreeVal r
    c = r c.unpair.2 := by
  have h1 : isOne (1 - c.unpair.1) = 1 := by rw [h]; rfl
  show QBody r (QFuel r c) c = r c.unpair.2
  unfold QBody
  simp only [h1, selectFn_one]

/-- **Tag-` >=  2` unfolding, for an arbitrary code.** -/
theorem TreeVal_of_tag_ge2 (r : Nat  ->  Nat) {c : Nat} (h : 2  <=  c.unpair.1) :
    TreeVal r c = UComputablePresentation.masterIdx := by
  have h1 : isOne (1 - c.unpair.1) = 0 := by
    have : (1 - c.unpair.1 : Nat) = 0 := by omega
    rw [this]; rfl
  have h2 : isOne (2 - c.unpair.1) = 0 := by
    have : (2 - c.unpair.1 : Nat) = 0 := by omega
    rw [this]; rfl
  show QBody r (QFuel r c) c = UComputablePresentation.masterIdx
  unfold QBody
```

```
  simp only [h1, h2, selectFn_zero]

/-- **`Uinter`'s unconditional dichotomy.** Unlike a generic `ComputablePresentation.
    inter` (whose
behaviour off the consistent domain is unspecified "junk"), `?`'s canonicalizing `
    Uinter` always
produces *either* the true intersection *or* falls back to `U.master` -- `canonList`'s
    documented
"fall back to `(0,1)` if nothing survives" behaviour, holding **unconditionally** (no
    consistency
hypothesis needed). This is the key fact an abstract `ComputablePresentation` cannot
    supply. -/
theorem UX_Uinter_dichotomy (n m : Nat) :
    UX (Uinter n m) = U.master  \/  UX (Uinter n m) = UX n  I  UX m := by
  have hUXeq : UX (Uinter n m) = presentedIntervals (canonList (decodeQPairList (
    Uinter n m))) :=
    UX_eq (Uinter n m)
  have hL : presentedIntervals (decodeQPairList (Uinter n m)) = UX n  I  UX m := by
    show presentedIntervals (decodeQPairList (combineCode (canonCode n) (canonCode m))
    )
        = UX n  I  UX m
    rw [presentedIntervals_decodeQPairList_combineCode]
    rfl
  rcases hfilt : ((decodeQPairList (Uinter n m)).map qpClip).filter
      (fun p => decide (p.1 < p.2)) with _ | <a, l>
  * left
    rw [hUXeq, canonList_eq_of_filter_eq_nil hfilt,
    presentedIntervals_singleton_zero_one]
  * right
    rw [hUXeq, canonList_eq_of_filter_eq_cons hfilt,  <-  hfilt,
    presentedIntervals_filter_qpPos,
      presentedIntervals_map_qpClip, hL]
    obtain <-, -, hsubn> := U_mem_UX n
    exact Set.inter_eq_left.mpr (Set.inter_subset_left.trans hsubn)

/-- **List-level choice, choice-free.** If every entry of `l` has an `r`-preimage, `l`
    itself is the
`r`-image of some list -- built by ordinary list induction (`Exists.elim` inside a `
    Prop`-valued goal
at each step), not `Classical.choice`. -/
theorem exists_preimage_of_forall_mem {r : Nat  ->  Nat} {l : List Nat} (h : forall  i
     in  l, exists  n, r n = i) :
    exists  l0 : List Nat, l0.map r = l := by
  induction l with
  | nil => exact <[], rfl>
  | cons i l' ih =>
    obtain <n0, hn0> := h i List.mem_cons_self
    obtain <l0', hl0'> := ih (fun j hj => h j (List.mem_cons_of_mem i hj))
    exact <n0 :: l0', by simp [hn0, hl0']>

/-- `leaf j`: the term-algebra code evaluating to `r j`. -/
def leafCode (j : Nat) : Nat := Nat.pair 0 j

/-- **Unfolding at a leaf.** -/
theorem TreeVal_leaf (r : Nat  ->  Nat) (j : Nat) : TreeVal r (leafCode j) = r j := by
  have htag : (leafCode j).unpair.1 = 0 := by unfold leafCode; rw [unpair_pair_fst]
```

```
  have hpay : (leafCode j).unpair.2 = j := by unfold leafCode; rw [unpair_pair_snd]
  have h1 : isOne (1 - (leafCode j).unpair.1) = 1 := by rw [htag]; rfl
  show QBody r (QFuel r (leafCode j)) (leafCode j) = r j
  unfold QBody
  simp only [h1, hpay, selectFn_one]

/-- `master`: the term-algebra code evaluating to `?`'s master index. -/
def masterCode : Nat := Nat.pair 2 0

/-- **Unfolding at the master constant.** -/
theorem TreeVal_master (r : Nat  ->  Nat) : TreeVal r masterCode =
    UComputablePresentation.masterIdx := by
  have htag : masterCode.unpair.1 = 2 := by unfold masterCode; rw [unpair_pair_fst]
  have h1 : isOne (1 - masterCode.unpair.1) = 0 := by rw [htag]; rfl
  have h2 : isOne (2 - masterCode.unpair.1) = 0 := by rw [htag]; rfl
  show QBody r (QFuel r masterCode) masterCode = UComputablePresentation.masterIdx
  unfold QBody
  simp only [h1, h2, selectFn_zero]

/-- `node t1 t2`: the term-algebra code evaluating to `Uinter (val t1) (val t2)`. -/
def nodeCode (t1 t2 : Nat) : Nat := Nat.pair 1 (Nat.pair t1 t2)

theorem primrec_QLookup (r : Nat  ->  Nat) (hr : Nat.Primrec r) : Nat.Primrec (QLookup
     r) := by
  have hleaf : Nat.Primrec (fun w : Nat => r w.unpair.2.unpair.2) :=
    hr.comp (Nat.Primrec.right.comp Nat.Primrec.right)
  have hleft : Nat.Primrec (fun w : Nat => nthCode w.unpair.1 w.unpair.2.unpair.2.
    unpair.1 0) :=
    (primrec_nthCode.comp (Nat.Primrec.left.pair
      ((Nat.Primrec.left.comp (Nat.Primrec.right.comp Nat.Primrec.right)).pair
        (Nat.Primrec.const 0)))).of_eq fun w => by simp only [unpair_pair_fst,
    unpair_pair_snd]
  have hright : Nat.Primrec (fun w : Nat => nthCode w.unpair.1 w.unpair.2.unpair.2.
    unpair.2 0) :=
    (primrec_nthCode.comp (Nat.Primrec.left.pair
      ((Nat.Primrec.right.comp (Nat.Primrec.right.comp Nat.Primrec.right)).pair
        (Nat.Primrec.const 0)))).of_eq fun w => by simp only [unpair_pair_fst,
    unpair_pair_snd]
  have hnode : Nat.Primrec (fun w : Nat =>
      Uinter (nthCode w.unpair.1 w.unpair.2.unpair.2.unpair.1 0)
        (nthCode w.unpair.1 w.unpair.2.unpair.2.unpair.2 0)) :=
    (Uinter_primrec.comp (hleft.pair hright)).of_eq fun w => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  have htag : Nat.Primrec (fun w : Nat => w.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.right
  have hisone1 : Nat.Primrec (fun w : Nat => isOne (1 - w.unpair.2.unpair.1)) :=
    primrec_isOne.comp (primrec_sub_2 (Nat.Primrec.const 1) htag)
  have hisone2 : Nat.Primrec (fun w : Nat => isOne (2 - w.unpair.2.unpair.1)) :=
    primrec_isOne.comp (primrec_sub_2 (Nat.Primrec.const 2) htag)
  refine (primrec_selectFn hisone1 hleaf
    (primrec_selectFn hisone2 hnode (Nat.Primrec.const UComputablePresentation.
    masterIdx))).of_eq
    fun w => ?_
  show _ = QLookup r w
  unfold QLookup QBody
  match tag : w.unpair.2.unpair.1 with
```

```
  | 0 =>
    have h10 : (1 - 0 : Nat) = 1 := rfl
    simp only [h10, isOne_one, selectFn_one]
  | 1 =>
    have h01' : (1 - 1 : Nat) = 0 := rfl
    have h12' : (2 - 1 : Nat) = 1 := rfl
    simp only [h01', h12', isOne_zero, isOne_one, selectFn_zero, selectFn_one]
  | t + 2 =>
    have h01' : (1 - (t + 2) : Nat) = 0 := Nat.sub_eq_zero_of_le (by omega)
    have h12' : (2 - (t + 2) : Nat) = 0 := Nat.sub_eq_zero_of_le (by omega)
    simp only [h01', h12', isOne_zero, selectFn_zero]

/-- **`QFuel r` is jointly primitive recursive in `(fuel, code)`.** Mirrors
`primrec_decodeFuelOkChar2` exactly. -/
theorem primrec_QFuel2 (r : Nat  ->  Nat) (hr : Nat.Primrec r) :
    Nat.Primrec (fun s : Nat => QFuel r s.unpair.1 s.unpair.2) := by
  have hft := primrec_fuelTable (primrec_QLookup r hr)
  have hpack : Nat.Primrec (fun s : Nat => Nat.pair s.unpair.2 s.unpair.1) :=
    Nat.Primrec.pair Nat.Primrec.right Nat.Primrec.left
  have hcomp : Nat.Primrec (fun s => fuelTable (QLookup r) s.unpair.2 s.unpair.1) :=
    (hft.comp hpack).of_eq fun s => by simp only [unpair_pair_fst, unpair_pair_snd]
  have hnth : Nat.Primrec
      (fun s => nthCode (fuelTable (QLookup r) s.unpair.2 s.unpair.1) s.unpair.2 0) :=
    (primrec_nthCode.comp
      (hcomp.pair ((Nat.Primrec.right).pair (Nat.Primrec.const 0)))).of_eq
      fun s => by simp only [unpair_pair_fst, unpair_pair_snd]
  exact hnth.of_eq fun s => QFuel_fuelTable_eq r s.unpair.1 s.unpair.2 s.unpair.2 (
    le_refl _)

theorem primrec_TreeVal (r : Nat  ->  Nat) (hr : Nat.Primrec r) : Nat.Primrec (TreeVal
    r) := by
  have h2 := primrec_QFuel2 r hr
  have hpack : Nat.Primrec (fun c : Nat => Nat.pair (c + 1) c) :=
    (primrec_add_2 Nat.Primrec.id (Nat.Primrec.const 1)).pair Nat.Primrec.id
  refine (h2.comp hpack).of_eq fun c => ?_
  show QFuel r (Nat.pair (c + 1) c).unpair.1 (Nat.pair (c + 1) c).unpair.2 = TreeVal r
    c
  rw [unpair_pair_fst, unpair_pair_snd]
  rfl

/-- If `c`'s tag (`c.unpair.1`) is nonzero, `c`'s payload (`c.unpair.2`) is *strictly*
    smaller than
`c` itself -- the key well-foundedness fact driving `nodeCode`'s children being
    strictly smaller
than the parent (choice-free: only uses the custom `unpair_pair`/`le_pair_right`/
`Nat.pair_lt_pair_left`, all already audited elsewhere in this file). -/
theorem unpair_snd_lt_of_unpair_fst_pos {c : Nat} (h : 0 < c.unpair.1) : c.unpair.2 <
    c := by
  have h1 : c.unpair.2  <=  Nat.pair 0 c.unpair.2 := le_pair_right 0 c.unpair.2
  have h2 : Nat.pair 0 c.unpair.2 < Nat.pair c.unpair.1 c.unpair.2 := Nat.
    pair_lt_pair_left _ h
  rw [Nat.pair_unpair] at h2
  omega

/-- **Stabilization.** Once the fuel exceeds the code, `QFuel r`'s value no longer
    depends on
```

```
*which* sufficiently-large fuel was supplied -- the key fact letting us read off a
    genuine,
fuel-free evaluator `TreeVal` below. Proved by strong induction on `c`, splitting on
    whether `c`'s
tag is `0` (no recursion, both sides agree immediately) or positive (both children are
    strictly
`< c`, via `unpair_snd_lt_of_unpair_fst_pos`, so the induction hypothesis applies to
    them). -/
theorem QFuel_stable (r : Nat  ->  Nat) :
    forall  c fuel fuel', c < fuel  ->  c < fuel'  ->  QFuel r fuel c = QFuel r fuel'
    c := by
  intro c
  induction c using Nat.strong_induction_on with
  | _ c ih =>
    intro fuel fuel' hfuel hfuel'
    obtain <f, rfl> : exists  f, fuel = f + 1 := <fuel - 1, by omega>
    obtain <f', rfl> : exists  f', fuel' = f' + 1 := <fuel' - 1, by omega>
    show QBody r (QFuel r f) c = QBody r (QFuel r f') c
    rcases Nat.eq_zero_or_pos c.unpair.1 with htag0 | htagpos
    * have hval : isOne (1 - c.unpair.1) = 1 := by rw [htag0]; rfl
      unfold QBody
      simp only [hval, selectFn_one]
    * have hsndlt : c.unpair.2 < c := unpair_snd_lt_of_unpair_fst_pos htagpos
      have h1lt : c.unpair.2.unpair.1 < c := lt_of_le_of_lt (unpair_left_le c.unpair
    .2) hsndlt
      have h2lt : c.unpair.2.unpair.2 < c := lt_of_le_of_lt (unpair_snd_le c.unpair.2)
     hsndlt
      have hfc : c  <=  f := by omega
      have hfc' : c  <=  f' := by omega
      have e1 := ih c.unpair.2.unpair.1 h1lt f f'
        (lt_of_lt_of_le h1lt hfc) (lt_of_lt_of_le h1lt hfc')
      have e2 := ih c.unpair.2.unpair.2 h2lt f f'
        (lt_of_lt_of_le h2lt hfc) (lt_of_lt_of_le h2lt hfc')
      exact QBody_local r (QFuel r f) (QFuel r f') c e1 e2

theorem TreeVal_eq_QFuel (r : Nat  ->  Nat) {c fuel : Nat} (h : c < fuel) : TreeVal r
    c = QFuel r fuel c :=
  QFuel_stable r c (c + 1) fuel (Nat.lt_succ_self c) h

/-- **Tag-`1` unfolding, for an arbitrary code.** -/
theorem TreeVal_of_tag1 (r : Nat  ->  Nat) {c : Nat} (h : c.unpair.1 = 1) :
    TreeVal r c = Uinter (TreeVal r c.unpair.2.unpair.1) (TreeVal r c.unpair.2.unpair
    .2) := by
  have h1 : isOne (1 - c.unpair.1) = 0 := by rw [h]; rfl
  have h2 : isOne (2 - c.unpair.1) = 1 := by rw [h]; rfl
  have hsndlt : c.unpair.2 < c := unpair_snd_lt_of_unpair_fst_pos (by omega)
  have ht1 : c.unpair.2.unpair.1 < c := lt_of_le_of_lt (unpair_left_le c.unpair.2)
    hsndlt
  have ht2 : c.unpair.2.unpair.2 < c := lt_of_le_of_lt (unpair_snd_le c.unpair.2)
    hsndlt
  show QBody r (QFuel r c) c = Uinter (TreeVal r c.unpair.2.unpair.1) (TreeVal r c.
    unpair.2.unpair.2)
  unfold QBody
  simp only [h1, h2, selectFn_zero, selectFn_one]
  rw [TreeVal_eq_QFuel r ht1, TreeVal_eq_QFuel r ht2]
```

```
/-- `nodeCode t1 t2`'s two children are each strictly smaller than the whole code. -/
theorem nodeCode_fst_lt (t1 t2 : Nat) : t1 < nodeCode t1 t2 := by
  have hpos : 0 < (nodeCode t1 t2).unpair.1 := by
    unfold nodeCode; rw [unpair_pair_fst]; omega
  have hsnd : (nodeCode t1 t2).unpair.2 < nodeCode t1 t2 :=
    unpair_snd_lt_of_unpair_fst_pos hpos
  have ht1 : t1  <=  (nodeCode t1 t2).unpair.2 := by
    unfold nodeCode; rw [unpair_pair_snd]; exact le_pair_left t1 t2
  omega

theorem nodeCode_snd_lt (t1 t2 : Nat) : t2 < nodeCode t1 t2 := by
  have hpos : 0 < (nodeCode t1 t2).unpair.1 := by
    unfold nodeCode; rw [unpair_pair_fst]; omega
  have hsnd : (nodeCode t1 t2).unpair.2 < nodeCode t1 t2 :=
    unpair_snd_lt_of_unpair_fst_pos hpos
  have ht2 : t2  <=  (nodeCode t1 t2).unpair.2 := by
    unfold nodeCode; rw [unpair_pair_snd]; exact le_pair_right t1 t2
  omega

/-- **Unfolding at a node.** -/
theorem TreeVal_node (r : Nat  ->  Nat) (t1 t2 : Nat) :
    TreeVal r (nodeCode t1 t2) = Uinter (TreeVal r t1) (TreeVal r t2) := by
  have ht1 : t1 < nodeCode t1 t2 := nodeCode_fst_lt t1 t2
  have ht2 : t2 < nodeCode t1 t2 := nodeCode_snd_lt t1 t2
  have htag : (nodeCode t1 t2).unpair.1 = 1 := by unfold nodeCode; rw [unpair_pair_fst
    ]
  have hc1 : (nodeCode t1 t2).unpair.2.unpair.1 = t1 := by
    unfold nodeCode; rw [unpair_pair_snd, unpair_pair_fst]
  have hc2 : (nodeCode t1 t2).unpair.2.unpair.2 = t2 := by
    unfold nodeCode; rw [unpair_pair_snd, unpair_pair_snd]
  have h1 : isOne (1 - (nodeCode t1 t2).unpair.1) = 0 := by rw [htag]; rfl
  have h2 : isOne (2 - (nodeCode t1 t2).unpair.1) = 1 := by rw [htag]; rfl
  show QBody r (QFuel r (nodeCode t1 t2)) (nodeCode t1 t2) = Uinter (TreeVal r t1) (
    TreeVal r t2)
  unfold QBody
  simp only [h1, h2, hc1, hc2, selectFn_zero, selectFn_one]
  rw [TreeVal_eq_QFuel r ht1, TreeVal_eq_QFuel r ht2]

theorem primrec_QstepFn {r : Nat  ->  Nat} (hr : Nat.Primrec r) : Nat.Primrec (QstepFn
     r) := by
  unfold QstepFn
  have h1 : Nat.Primrec (fun t : Nat => t.unpair.2.unpair.1) := Nat.Primrec.left.comp
    Nat.Primrec.right
  have h2 : Nat.Primrec (fun t : Nat => r t.unpair.1) := hr.comp Nat.Primrec.left
  exact (Uinter_primrec.comp (h1.pair h2)).of_eq fun t => by
    simp only [unpair_pair_fst, unpair_pair_snd]

theorem primrec_Q'Xcode {r : Nat  ->  Nat} (hr : Nat.Primrec r) : Nat.Primrec (Q'Xcode
     r) :=
  primrec_foldCode (primrec_QstepFn hr) (Nat.Primrec.const 0)
    (Nat.Primrec.const UComputablePresentation.masterIdx) Nat.Primrec.id

end DomainTheory.NeighbourhoodSystems
```

— Q2X, Q′xPresentation

TreeVal_more.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.finGen
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.TreeVal
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Theorem_8_8
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Exercise_7_18
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Theorem_7_5
public import Mathlib.Data.Nat.Bitwise

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive

variable (r : Nat  ->  Nat) (x : (SubD UComputablePresentation).Element)

/- concordance: source L5161 EXERCISE 8.15 -- Q2X / Q'xPresentation -/

/-- **`Q2X r`: the term-algebra enumeration of `D'x`'s neighbourhoods.** -/
def Q2X (c : Nat) : Set ? := UX (TreeVal r c)

/-- **`Q2X r`'s intersection function**: literally the `node` constructor. -/
def Q2inter (c1 c2 : Nat) : Nat := nodeCode c1 c2

/-- **`surj` bridge, part 1: embed a `List Nat` into a term-algebra code.** Left-fold
    accumulator
mirroring *exactly* the fold shape of `Theorem75.idxchain`/`Q'X` (`Uinter`-folded from
    the master),
via `nodeCode`/`leafCode` instead of `Uinter`/`r` directly -- so that `TreeVal`
    evaluates it to the
same `U`-index (`TreeVal_listToTreeAux` below). -/
def listToTreeAux : Nat  ->  List Nat  ->  Nat
  | acc, [] => acc
  | acc, j :: l => listToTreeAux (nodeCode acc (leafCode j)) l

theorem primrec_Q2inter : Nat.Primrec (fun t : Nat => Q2inter t.unpair.1 t.unpair.2)
    := by
  have heq : (fun t : Nat => Q2inter t.unpair.1 t.unpair.2) = fun t => Nat.pair 1 t :=
     by
    funext t; unfold Q2inter nodeCode; rw [pair_unpair]
  rw [heq]
  exact Nat.Primrec.pair (Nat.Primrec.const 1) Nat.Primrec.id

/-- **`interEq_computable` for `Q2X r`.** A pure reindexing of `U`'s own `
    U_interEq_computable`
along the primitive-recursive `TreeVal r` -- exactly `ComputablePresentation.
    reindexInvolutive`'s
`interEq_computable` proof pattern, minus the involution (only needed there for the `
    masterIdx`/
`inter` fields, not for `.comp`-reindexing a `RecDecidable`). -/
theorem Q2_interEq_computable (hr : Nat.Primrec r) :
```

```
    RecDecidable_3 (fun n m k => Q2X r n  I  Q2X r m = Q2X r k) := by
  have hTV := primrec_TreeVal r hr
  have hreindex : Nat.Primrec (fun t : Nat => Nat.pair (TreeVal r t.unpair.1)
      (Nat.pair (TreeVal r t.unpair.2.unpair.1) (TreeVal r t.unpair.2.unpair.2))) :=
    (hTV.comp Nat.Primrec.left).pair
      ((hTV.comp (Nat.Primrec.left.comp Nat.Primrec.right)).pair
        (hTV.comp (Nat.Primrec.right.comp Nat.Primrec.right)))
  refine RecDecidable.of_iff (fun t => ?_) (U_interEq_computable.comp hreindex)
  simp only [unpair_pair_fst, unpair_pair_snd]
  rfl

theorem Q2X_masterIdx_spec :
    Q2X r masterCode = (toSubsystemSys UComputablePresentation x).master := by
  show UX (TreeVal r masterCode) = U.master
  rw [TreeVal_master]
  exact UX_UmasterIdx

/-- **`surj` for `Q'X r`.** Every `D'x`-neighbourhood `Y` is some `Q'X r n`: unwind `Y`'
    s `finGen`
witness `sub`, use `hspec` + `exists_preimage_of_forall_mem` to find a preimage list `
    sub0` with
`sub0.map r = sub`, and read off `Y` via the *generic* `Theorem75.idxchain_spec` (
    applicable because
`sub`'s own consistency -- `U.mem (interFrom ... sub)` -- is exactly `hDY` rewritten
    along `hYeq`). -/
theorem Q'x_surj
    (hspec : forall  i, (toSubsystemSys UComputablePresentation x).mem (UX i)  <->
    exists  n, r n = i)
    {Y : Set ?} (hY : (toSubsystemSys UComputablePresentation x).mem Y) :
    exists  n, Q'X r n = Y := by
  obtain <js, hxjs, sub, hsub, hDY, hYeq> := hY
  have hjsmem : forall  i  in  js, (toSubsystemSys UComputablePresentation x).mem (UX
    i) :=
    (mem_nbhdGen_iff UComputablePresentation x js).mp hxjs
  have hsubmem : forall  i  in  sub, (toSubsystemSys UComputablePresentation x).mem (
    UX i) :=
    fun i hi => hjsmem i (hsub.subset hi)
  obtain <sub0, hsub0> := exists_preimage_of_forall_mem (fun i hi => (hspec i).mp (
    hsubmem i hi))
  refine <encodeList sub0, ?_>
  rw [Q'X_eq, decodeList_encodeList, hsub0, hYeq]
  exact idxchain_spec UComputablePresentation (hYeq  |>  hDY)

/-- **`mem_X` for `Q2X r`.** Every `Q2X r c` is a `D'x`-neighbourhood -- by strong
    induction on the
code `c`, splitting on its tag exactly as `TreeVal` itself does: tag `0` (leaf) is
`D'x.mem (UX (r j))`, always true by `hspec` (witness `j` itself); tag `1` (node)
    combines the two
(inductively already-`D'x.mem`) children via `Uinter`'s dichotomy (`UX_Uinter_dichotomy
    `), landing on
`D'x.master_mem` on fallback or on `x`'s own filter closure (`nbhdGen_inter`/`
    interFrom_append_eq`,
exactly as in `toSubsystemSys_mem_foldl_Uinter`) otherwise; tag ` >=  2` (master) is `D
    'x.master_mem`
directly via `masterIdx_spec`. -/
theorem Q2x_mem_X (hspec : forall  i, (toSubsystemSys UComputablePresentation x).mem (
```

```
    UX i)  <->  exists  n, r n = i) :
    forall  c, (toSubsystemSys UComputablePresentation x).mem (Q2X r c) := by
  intro c
  induction c using Nat.strong_induction_on with
  | _ c ih =>
    show (toSubsystemSys UComputablePresentation x).mem (UX (TreeVal r c))
    rcases Nat.lt_or_ge c.unpair.1 2 with htag | htag
    * have htag01 : c.unpair.1 = 0  \/  c.unpair.1 = 1 := by omega
      rcases htag01 with htagv | htagv
      * rw [TreeVal_of_tag0 r htagv]
        exact (hspec (r c.unpair.2)).mpr <c.unpair.2, rfl>
      * rw [TreeVal_of_tag1 r htagv]
        have hsndlt : c.unpair.2 < c := unpair_snd_lt_of_unpair_fst_pos (by omega)
        have ht1 : c.unpair.2.unpair.1 < c := lt_of_le_of_lt (unpair_left_le c.unpair
    .2) hsndlt
        have ht2 : c.unpair.2.unpair.2 < c := lt_of_le_of_lt (unpair_snd_le c.unpair
    .2) hsndlt
        have hm1 := ih c.unpair.2.unpair.1 ht1
        have hm2 := ih c.unpair.2.unpair.2 ht2
        unfold Q2X at hm1 hm2
        rcases UX_Uinter_dichotomy (TreeVal r c.unpair.2.unpair.1) (TreeVal r c.unpair
    .2.unpair.2)
          with hcase | hcase
        * rw [hcase]; exact (toSubsystemSys UComputablePresentation x).master_mem
        * rw [hcase]
          obtain <jsA, hxA, subA, hsubA, hDA, hAeq> := hm1
          obtain <jsB, hxB, subB, hsubB, hDB, hBeq> := hm2
          have hUmem : U.mem (UX (TreeVal r c.unpair.2.unpair.1)  I  UX (TreeVal r c.
    unpair.2.unpair.2)) :=
            hcase  |>  U_mem_UX (Uinter (TreeVal r c.unpair.2.unpair.1) (TreeVal r c.
    unpair.2.unpair.2))
          refine <jsA ++ jsB, nbhdGen_inter UComputablePresentation jsA jsB  |>  x.
    inter_mem hxA hxB,
            subA ++ subB, hsubA.append hsubB, hUmem, ?_>
          rw [hAeq, hBeq, interFrom_append_eq UComputablePresentation U.master subA
    subB]
    * rw [TreeVal_of_tag_ge2 r htag]
      show (toSubsystemSys UComputablePresentation x).mem (UX UmasterIdx)
      rw [UX_UmasterIdx]
      exact (toSubsystemSys UComputablePresentation x).master_mem

/-- **`cons_computable` for `Q2X r`.** The crux resolved by the term algebra: the
    backward
direction's witness is `nodeCode n m`, no search needed to *name* it, and it works
    because
`Uinter_spec` applies exactly when consistency holds. -/
theorem Q2_cons_computable (hr : Nat.Primrec r) :
    RecDecidable_2 (fun n m => exists  k, Q2X r k  subseteq  Q2X r n  I  Q2X r m) :=
    by
  have hTV := primrec_TreeVal r hr
  have hreindex : Nat.Primrec (fun t : Nat => Nat.pair (TreeVal r t.unpair.1) (TreeVal
     r t.unpair.2)) :=
    (hTV.comp Nat.Primrec.left).pair (hTV.comp Nat.Primrec.right)
  refine RecDecidable.of_iff (fun t => ?_) (U_cons_computable.comp hreindex)
  simp only [unpair_pair_fst, unpair_pair_snd]
  constructor
```

```
  * rintro <k, hk>
    exact <TreeVal r k, hk>
  * rintro <k, hk>
    refine <nodeCode t.unpair.1 t.unpair.2, ?_>
    show UX (TreeVal r (nodeCode t.unpair.1 t.unpair.2))  subseteq
      UX (TreeVal r t.unpair.1)  I  UX (TreeVal r t.unpair.2)
    rw [TreeVal_node, Uinter_spec <k, hk>]

/-- **`inter_spec` for `Q2X r`.** The hypothesis is exactly what `Uinter_spec` needs (a
    `UX`-witness
of consistency, transported through `TreeVal`), so `nodeCode n m` genuinely indexes the
    intersection
-- no fallback case to worry about. -/
theorem Q2inter_spec {n m : Nat} (h : exists  k, Q2X r k  subseteq  Q2X r n  I  Q2X r
    m) :
    Q2X r (Q2inter n m) = Q2X r n  I  Q2X r m := by
  obtain <k, hk> := h
  show UX (TreeVal r (nodeCode n m)) = UX (TreeVal r n)  I  UX (TreeVal r m)
  rw [TreeVal_node]
  exact Uinter_spec <TreeVal r k, hk>

theorem TreeVal_listToTreeAux (acc : Nat) (l : List Nat) :
    TreeVal r (listToTreeAux acc l) = l.foldl (fun a j => Uinter a (r j)) (TreeVal r
    acc) := by
  induction l generalizing acc with
  | nil => simp [listToTreeAux]
  | cons j l ih =>
    simp only [listToTreeAux, List.foldl_cons]
    rw [ih, TreeVal_node, TreeVal_leaf]

/-- **`surj` for `Q2X r`.** Every `D'x`-neighbourhood `Y` is some `Q2X r c` -- reduce to
    the already-
proven list-based `Q'x_surj` (`Y = Q'X r n` for some `n`), then push `decodeList n`
    through
`listToTreeAux` and identify the result with `idxchain`'s own fold via `List.foldl_map
    `, so the same
`n` (repackaged as a tree code) also witnesses `Q2X r`. No fresh combinatorial work:
    this is a pure
bridging argument between the list-indexed and term-algebra-indexed enumerations of
    the *same*
underlying `Uinter`-folds. -/
theorem Q2x_surj (hspec : forall  i, (toSubsystemSys UComputablePresentation x).mem (
    UX i)  <->  exists  n, r n = i)
    {Y : Set ?} (hY : (toSubsystemSys UComputablePresentation x).mem Y) :
    exists  c, Q2X r c = Y := by
  obtain <n, hn> := Q'x_surj r x hspec hY
  refine <listToTreeAux masterCode (decodeList n), ?_>
  have hcode : (decodeList n).foldl (fun a j => Uinter a (r j))
    UComputablePresentation.masterIdx
      = idxchain UComputablePresentation (List.map r (decodeList n)) := by
    unfold idxchain
    rw [List.foldl_map]
    rfl
  show UX (TreeVal r (listToTreeAux masterCode (decodeList n))) = Y
  rw [TreeVal_listToTreeAux, TreeVal_master, hcode,  <-  Q'X_eq]
  exact hn
```

```
/-- **Subgoal 6f, capstone.** `D'x := toSubsystemSys UComputablePresentation x` admits
    a genuine
`ComputablePresentation`, assembled from `Q2X`/`Q2x_mem_X`/`Q2x_surj`/`
    Q2_interEq_computable`/
`Q2_cons_computable`/`Q2inter`/`Q2inter_spec`/`masterCode`/`Q2X_masterIdx_spec` --
    resolving the gap
flagged above via the binary term algebra. -/
def Q'xPresentation (hr : Nat.Primrec r)
    (hspec : forall  i, (toSubsystemSys UComputablePresentation x).mem (UX i)  <->
    exists  n, r n = i) :
    ComputablePresentation (toSubsystemSys UComputablePresentation x) where
  X := Q2X r
  mem_X := Q2x_mem_X r x hspec
  surj := Q2x_surj r x hspec
  interEq_computable := Q2_interEq_computable r hr
  cons_computable := Q2_cons_computable r hr
  inter := Q2inter
  inter_primrec := primrec_Q2inter
  inter_spec := fun h => Q2inter_spec r h
  masterIdx := masterCode
  masterIdx_spec := Q2X_masterIdx_spec r x

/-- **The safe-fold invariant.** Folding `Uinter` from a starting point already in `D'x
    .mem` along a
list all of whose `UX`-images are also in `D'x.mem` stays in `D'x.mem`: `x`'s own *
    unconditional*
filter closure (`Element.inter_mem`, via `nbhdGen_inter`) combines the two generators'
    witnessing
index-lists when `Uinter`'s fold genuinely computes an intersection (dichotomy's right
    disjunct,
using `interFrom_append_eq`), and `D'x.master_mem` handles the fallback (left disjunct)
    . -/
theorem toSubsystemSys_mem_foldl_Uinter (acc : Nat)
    (L : List Nat) (hacc : (toSubsystemSys UComputablePresentation x).mem (UX acc))
    (hL : forall  i  in  L, (toSubsystemSys UComputablePresentation x).mem (UX i)) :
    (toSubsystemSys UComputablePresentation x).mem
      (UX (L.foldl (fun a j => Uinter a j) acc)) := by
  induction L generalizing acc with
  | nil => simpa using hacc
  | cons j L' ih =>
    rw [List.forall_mem_cons] at hL
    have hacc' : (toSubsystemSys UComputablePresentation x).mem (UX (Uinter acc j)) :=
     by
      rcases UX_Uinter_dichotomy acc j with hcase | hcase
      * rw [hcase]; exact (toSubsystemSys UComputablePresentation x).master_mem
      * rw [hcase]
        obtain <jsA, hxA, subA, hsubA, hDA, hAeq> := hacc
        obtain <jsJ, hxJ, subJ, hsubJ, hDJ, hJeq> := hL.1
        have hUmem : U.mem (UX acc  I  UX j) := hcase  |>  U_mem_UX (Uinter acc j)
        refine <jsA ++ jsJ,
          nbhdGen_inter UComputablePresentation jsA jsJ  |>  x.inter_mem hxA hxJ,
          subA ++ subJ, hsubA.append hsubJ, hUmem, ?_>
        rw [hAeq, hJeq, interFrom_append_eq UComputablePresentation U.master subA subJ
    ]
    exact ih (Uinter acc j) hacc' hL.2
```

```
/-- **`mem_X` for `Q'X r`.** Every `Q'X r n` is a `D'x`-neighbourhood -- no consistency
     hypothesis on
`n` needed, since (a) every entry `r j` of the folded list is *unconditionally* `D'x.
    mem (UX (r j))`
by `hspec` (taking the trivial witness `j`), and (b) `toSubsystemSys_mem_foldl_Uinter`
    safely folds
any such list, landing on `D'x.master_mem` if `Uinter`'s dichotomy ever falls back. -/
theorem Q'x_mem_X
    (hspec : forall  i, (toSubsystemSys UComputablePresentation x).mem (UX i)  <->
    exists  n, r n = i) (n : Nat) :
    (toSubsystemSys UComputablePresentation x).mem (Q'X r n) := by
  rw [Q'X_eq]
  show (toSubsystemSys UComputablePresentation x).mem
      (UX ((List.map r (decodeList n)).foldl (fun a j => Uinter a j)
        UComputablePresentation.masterIdx))
  apply toSubsystemSys_mem_foldl_Uinter
  * show (toSubsystemSys UComputablePresentation x).mem (UX UmasterIdx)
    rw [UX_UmasterIdx]
    exact (toSubsystemSys UComputablePresentation x).master_mem
  * intro i hi
    obtain <j, -, rfl> := List.mem_map.mp hi
    exact (hspec (r j)).mpr <j, rfl>

end DomainTheory.NeighbourhoodSystems
```

Assembles those tools into the Exercise 8.15 development (order iso, presentations, U-case).

SubsystemDomain.lean

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Exercise_7_18
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.finGen
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.TreeVal
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    TreeVal_more
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Theorem_8_8

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    VComputablePresentation

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Theorem_7_5
public import Mathlib.Data.Nat.Bitwise
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem Domain.Recursive
variable {a : Type*} {D : NeighborhoodSystem a}

/- concordance: source L5161 EXERCISE 8.15 domain of subsystems -/
```

```
section Computability

variable (P : ComputablePresentation D)

/-- **The enumeration.** -/
def SubDX (n : Nat) : Set (List Nat) := nbhdGen P (decodeList n)

/-- **`SubDX(appendListCode n m) = SubDX n  I  SubDX m` exactly** (not just when
    consistent) --
`appendListCode_eq` turns list-code concatenation into `List` `++`, then `nbhdGen_inter
    `. -/
theorem SubDX_inter_eq (n m : Nat) :
    SubDX P (appendListCode n m) = SubDX P n  I  SubDX P m := by
  show nbhdGen P (decodeList (appendListCode n m))
      = nbhdGen P (decodeList n)  I  nbhdGen P (decodeList m)
  rw [appendListCode_eq, nbhdGen_inter]

theorem SubDX_masterIdx_spec : SubDX P (encodeList ([] : List Nat)) = (SubD P).master
    := by
  show nbhdGen P (decodeList (encodeList ([] : List Nat))) = nbhdGen P []
  rw [decodeList_encodeList]

theorem SubDX_mem (n : Nat) : (SubD P).mem (SubDX P n) := <decodeList n, rfl>

theorem SubDX_surj {N : Set (List Nat)} (hN : (SubD P).mem N) : exists  n, SubDX P n =
     N := by
  obtain <js, rfl> := hN
  exact <encodeList js, by unfold SubDX; rw [decodeList_encodeList]>

/-- **Consistency is always true** (`appendListCode n m` is always a witness), hence
    trivially
recursively decidable. -/
theorem SubD_cons_computable :
    RecDecidable_2 (fun n m => exists  k, SubDX P k  subseteq  SubDX P n  I  SubDX P m
    ) := by
  refine recDecidable_of_forall (fun t => <appendListCode t.unpair.1 t.unpair.2, ?_>)
  rw [SubDX_inter_eq]

/-- The index-component of `consUpd`'s fold state, after processing `el`, is
    independent of the
flag threaded alongside it: it only accumulates `P.inter` along the entries *selected*
    by the
bitmask `b`. Pure computation fact, no consistency needed. -/
theorem consUpd_foldl_idx (projFn fc : Nat  ->  Nat) (el : List Nat) (b a flag : Nat)
    :
    (List.foldl (fun s x => consUpd P projFn fc s x) (Nat.pair b (Nat.pair a flag)) el
    ).unpair.2.unpair.1
      = (List.map projFn (bitSelect el b)).foldl (fun acc j => P.inter acc j) a := by
  induction el generalizing b a flag with
  | nil => rw [bitSelect_nil, List.foldl_nil, List.map_nil, List.foldl_nil,
    unpair_pair_snd,
      unpair_pair_fst]
  | cons e el ih =>
    rw [List.foldl_cons, consUpd_eval, bitSelect_cons]
    by_cases hb : b % 2 = 1
```

```
    * rw [ite_eq_left hb, List.map_cons, List.foldl_cons, hb, selectFn_one]
      exact ih (b / 2) (P.inter a (projFn e)) _
    * rw [ite_eq_right hb, show b % 2 = 0 by omega, selectFn_zero]
      exact ih (b / 2) a flag

/-- `idxCharAt fc w` (`w = pair b c`): the same fold as `consCharAt`, projecting the
    running
intersection *index* instead of the consistency flag. -/
def idxCharAt (fc : Nat  ->  Nat) (w : Nat) : Nat :=
  (foldCode (consStp P id fc) 0 (Nat.pair w.unpair.1 (Nat.pair P.masterIdx 1))
    w.unpair.2).unpair.2.unpair.1

/-- **`idxCharAt` computes `idxchain (bitSelect (decodeList c) b)`.** Unfolds `foldCode
    ` to a
`List.foldl` over `decodeList c`, identifies it with `consUpd`'s fold via `consStp`'s
    definition,
and reads off the index component via `consUpd_foldl_idx` (at `projFn := id`, `a :=
    masterIdx`,
matching `idxchain`'s own definition as a fold from `masterIdx`). -/
theorem idxCharAt_eq_idxchain (fc : Nat  ->  Nat) (w : Nat) :
    idxCharAt P fc w = idxchain P (bitSelect (decodeList w.unpair.2) w.unpair.1) := by
  unfold idxCharAt idxchain
  rw [foldCode_eq']
  have hstep : (fun acc x => consStp P id fc (Nat.pair x (Nat.pair acc 0)))
      = (fun acc x => consUpd P id fc acc x) := by
    funext acc x
    show consStp P id fc (Nat.pair x (Nat.pair acc 0)) = _
    unfold consStp
    simp only [unpair_pair_fst, unpair_pair_snd]
  rw [hstep, consUpd_foldl_idx P id fc (decodeList w.unpair.2) w.unpair.1 P.masterIdx
    1,
    List.map_id]

theorem primrec_idxCharAt {fc : Nat  ->  Nat} (hfcp : Nat.Primrec fc) :
    Nat.Primrec (idxCharAt P fc) := by
  have hfold := primrec_foldCode (primrec_consStp P primrec_id hfcp) (Nat.Primrec.
    const 0)
    (Nat.Primrec.left.pair ((Nat.Primrec.const P.masterIdx).pair (Nat.Primrec.const 1)
    ))
    Nat.Primrec.right
  exact Nat.Primrec.left.comp (Nat.Primrec.right.comp hfold)

/-- **A generator `P.X m` belongs to `Fin(decodeList c)` iff some bitmask-selected
    sublist of
`decodeList c` (`b < 2^c`, using `decodeList_length_le`) is `D`-consistent and equals `P
    .X m`** --
consistency via `consFold_decidable`, equality via `idxCharAt_eq_idxchain` + `
    idxchain_spec` +
`P.eq_computable`. The `D.mem (P.X m)` conjunct in `finGen`'s definition is always true
(`P.mem_X`), so it drops out of the decidable content. -/
theorem mem_finGen_computable :
    RecDecidable_2 (fun m c => P.X m  in  finGen P (decodeList c)) := by
  obtain <fc, hfcp, hfcs> := P.cons_computable
  have hfc : forall  s, fc s = 1  <->  exists  k, P.X k  subseteq  P.X s.unpair.1  I
    P.X s.unpair.2 := fun s => (hfcs s).symm
  have hconsFold := consFold_decidable P id fc hfc primrec_id hfcp
```

```
  have hg : Nat.Primrec (fun w : Nat => Nat.pair w.unpair.1 w.unpair.2.unpair.2) :=
    Nat.Primrec.left.pair (Nat.Primrec.right.comp Nat.Primrec.right)
  have hp1 : RecDecidable (fun w : Nat =>
      D.mem (interFrom P D.master (bitSelect (decodeList w.unpair.2.unpair.2) w.unpair
    .1))) := by
    refine RecDecidable.of_iff (fun w => ?_) (hconsFold.comp hg)
    simp only [unpair_pair_fst, unpair_pair_snd, List.map_id]
  have hidx : Nat.Primrec (fun w : Nat => idxCharAt P fc (Nat.pair w.unpair.1 w.unpair
    .2.unpair.2)) :=
    (primrec_idxCharAt P hfcp).comp hg
  have hh : Nat.Primrec (fun w : Nat =>
      Nat.pair w.unpair.2.unpair.1 (idxCharAt P fc (Nat.pair w.unpair.1 w.unpair.2.
    unpair.2))) :=
    (Nat.Primrec.left.comp Nat.Primrec.right).pair hidx
  have hp2 : RecDecidable (fun w : Nat =>
      P.X w.unpair.2.unpair.1 = P.X (idxCharAt P fc (Nat.pair w.unpair.1 w.unpair.2.
    unpair.2))) := by
    refine RecDecidable.of_iff (fun w => ?_) (P.eq_computable.comp hh)
    simp only [unpair_pair_fst, unpair_pair_snd]
  have hp := hp1.and hp2
  have hbound : Nat.Primrec (fun n : Nat => 2 ^ n.unpair.2) := primrec_two_pow Nat.
    Primrec.right
  have hbe := hp.bExists (bound := fun n : Nat => 2 ^ n.unpair.2) hbound
  refine RecDecidable.of_iff (fun t => ?_) hbe
  simp only [unpair_pair_fst, unpair_pair_snd]
  constructor
  * rintro <sub, hsub, -, hYeq>
    obtain <b, hb, hbeq> := exists_bitSelect_lt hsub
    have hb' : b < 2 ^ t.unpair.2 :=
      lt_of_lt_of_le hb (Nat.pow_le_pow_right (by omega) (decodeList_length_le t.
    unpair.2))
    have hDsub : D.mem (interFrom P D.master sub) := hYeq  |>  P.mem_X t.unpair.1
    refine <b, hb', hbeq  |>  hDsub, ?_>
    rw [idxCharAt_eq_idxchain]
    simp only [unpair_pair_fst, unpair_pair_snd]
    rw [hbeq, idxchain_spec P hDsub,  <-  hYeq]
  * rintro <b, -, hDb, hXeq>
    refine <bitSelect (decodeList t.unpair.2) b, bitSelect_sublist _ _, P.mem_X t.
    unpair.1, ?_>
    rw [hXeq, idxCharAt_eq_idxchain]
    simp only [unpair_pair_fst, unpair_pair_snd]
    rw [idxchain_spec P hDb]

/-- **`Fin(decodeList c1)  subseteq  Fin(decodeList c2)` is recursively decidable** --
    rewrite via
`finGen_le_iff_forall_mem` (at `D' := finGenSys P (decodeList c2)`, a genuine subsystem
     of `D` by
`finGenSys_subsystem`) to `forall  i  in  decodeList c1, P.X i  in  finGen P (
    decodeList c2)`, which is exactly
`mem_finGen_computable.bForallList`. -/
theorem finGen_subset_computable :
    RecDecidable_2 (fun c1 c2 => finGen P (decodeList c1)  subseteq  finGen P (
    decodeList c2)) := by
  have hiff : forall  c1 c2, finGen P (decodeList c1)  subseteq  finGen P (decodeList
    c2)  <->
      forall  i  in  decodeList c1, P.X i  in  finGen P (decodeList c2) :=
```

```
    fun c1 c2 => finGen_le_iff_forall_mem P (finGenSys_subsystem P (decodeList c2)) (
    decodeList c1)
  refine RecDecidable.of_iff (fun t => ?_) (mem_finGen_computable P).bForallList
  exact hiff t.unpair.1 t.unpair.2

/-- **`SubDX n  I  SubDX m = SubDX k` is recursively decidable.** Chain of exact
    rewrites reduces
membership-equality of a *pairwise* intersection to a *single* subset-decidable
    statement at the
`appendListCode`-combined index: `nbhdGen(decodeList n)  I  nbhdGen(decodeList m) =
    nbhdGen(decodeList
k)` ` <-> ` (`nbhdGen_inter`) `nbhdGen(decodeList n ++ decodeList m) = nbhdGen(
    decodeList k)` ` <-> `
(`appendListCode_eq`) `nbhdGen(decodeList(appendListCode n m)) = nbhdGen(decodeList k)`
     ` <-> `
(`nbhdGen_eq_iff`) `finGen(decodeList(appendListCode n m)) = finGen(decodeList k)` `
    <-> `
(`Set.Subset.antisymm_iff`) both inclusions, each an instance of `
    finGen_subset_computable`
reindexed along the primitive-recursive `appendListCode`. -/
theorem SubD_interEq_computable :
    RecDecidable_3 (fun n m k => SubDX P n  I  SubDX P m = SubDX P k) := by
  have hiff : forall  n m k, SubDX P n  I  SubDX P m = SubDX P k  <->
      finGen P (decodeList (appendListCode n m))  subseteq  finGen P (decodeList k)
    /\
        finGen P (decodeList k)  subseteq  finGen P (decodeList (appendListCode n m))
    := by
    intro n m k
    show nbhdGen P (decodeList n)  I  nbhdGen P (decodeList m) = nbhdGen P (decodeList
     k)  <->  _
    rw [nbhdGen_inter,  <-  appendListCode_eq, nbhdGen_eq_iff, Set.Subset.antisymm_iff
    ]
  have hg1 : Nat.Primrec (fun t : Nat => appendListCode t.unpair.1 t.unpair.2.unpair
    .1) :=
    (primrec_appendListCode.comp
        (Nat.Primrec.left.pair (Nat.Primrec.left.comp Nat.Primrec.right))).of_eq
      (fun t => by simp only [unpair_pair_fst, unpair_pair_snd])
  have hreindex1 : Nat.Primrec (fun t : Nat =>
      Nat.pair (appendListCode t.unpair.1 t.unpair.2.unpair.1) t.unpair.2.unpair.2) :=
    hg1.pair (Nat.Primrec.right.comp Nat.Primrec.right)
  have hreindex2 : Nat.Primrec (fun t : Nat =>
      Nat.pair t.unpair.2.unpair.2 (appendListCode t.unpair.1 t.unpair.2.unpair.1)) :=
    (Nat.Primrec.right.comp Nat.Primrec.right).pair hg1
  have hforward : RecDecidable (fun t : Nat =>
      finGen P (decodeList (appendListCode t.unpair.1 t.unpair.2.unpair.1))  subseteq
        finGen P (decodeList t.unpair.2.unpair.2)) := by
    refine RecDecidable.of_iff (fun t => ?_) ((finGen_subset_computable P).comp
    hreindex1)
    simp only [unpair_pair_fst, unpair_pair_snd]
  have hbackward : RecDecidable (fun t : Nat =>
      finGen P (decodeList t.unpair.2.unpair.2)  subseteq
        finGen P (decodeList (appendListCode t.unpair.1 t.unpair.2.unpair.1))) := by
    refine RecDecidable.of_iff (fun t => ?_) ((finGen_subset_computable P).comp
    hreindex2)
    simp only [unpair_pair_fst, unpair_pair_snd]
  refine RecDecidable.of_iff (fun t => ?_) (hforward.and hbackward)
```

```
  exact hiff t.unpair.1 t.unpair.2.unpair.1 t.unpair.2.unpair.2

/-- **Subgoal 4, capstone.** `SubD P` admits a computable presentation, assembled from
    4a-4g. -/
def SubDPresentation : ComputablePresentation (SubD P) where
  X := SubDX P
  mem_X := SubDX_mem P
  surj := SubDX_surj P
  interEq_computable := SubD_interEq_computable P
  cons_computable := SubD_cons_computable P
  inter := appendListCode
  inter_primrec := primrec_appendListCode
  inter_spec := fun {n m} _ => SubDX_inter_eq P n m
  masterIdx := encodeList []
  masterIdx_spec := SubDX_masterIdx_spec P

end Computability

section OrderIso

variable (P : ComputablePresentation D)

/-- **Subgoal 3, capstone.** `SubD.Element ?o {D' ? D' ? D}` -- fully choice-free.
    Forward map
`toSubsystem`, inverse `ofSubsystem`; order preservation/reflection reduces via Scott's
     remark
(`Subsystem.subsystem_iff_subset_of_common`) to `mem_nbhdGen_iff` applied pointwise. -/
def subsystemIso : (SubD P).Element ?o {D' : NeighborhoodSystem a // D' ? D} where
  toFun := toSubsystem P
  invFun x := ofSubsystem P x.1 x.2
  left_inv := ofSubsystem_toSubsystem P
  right_inv x := toSubsystem_ofSubsystem P x.1 x.2
  map_rel_iff' := by
    intro a b
    show toSubsystemSys P a ? toSubsystemSys P b  <->  (forall  N, a.mem N  ->  b.mem
    N)
    rw [Subsystem.subsystem_iff_subset_of_common (toSubsystemSys_subsystem P a)
      (toSubsystemSys_subsystem P b)]
    constructor
    * intro hYsub N hN
      obtain <js, rfl> := a.sub hN
      exact (mem_nbhdGen_iff P b js).mpr
        (fun i hi => hYsub ((mem_nbhdGen_iff P a js).mp hN i hi))
    * rintro hab Y <js, haj, hY>
      exact <js, hab (nbhdGen P js) haj, hY>

end OrderIso

section ComputableElements

variable (P : ComputablePresentation D)

/-- The singleton-list code `i  |->  encodeList [i]` is primitive recursive. -/
theorem primrec_encodeList_singleton : Nat.Primrec (fun i => encodeList [i]) := by
  have : (fun i => encodeList [i]) = fun i => Nat.pair i 0 + 1 := by
    funext i; show Nat.pair i (encodeList ([] : List Nat)) + 1 = Nat.pair i 0 + 1
```

```
    rw [encodeList]
  rw [this]
  exact Nat.Primrec.succ.comp (Nat.Primrec.pair primrec_id (Nat.Primrec.const 0))

/-- **Subgoal 6a.** `x` is a computable element of `SubD P` iff its "index set relative
     to `P`" --
the set of `P`-indices `i` with `(toSubsystemSys P x).mem (P.X i)` -- is recursively
    enumerable.
` -> `: reindex along `decodeList` (`mem_nbhdGen_iff`) then along the singleton-list
    code. ` <- `:
`REPred.forall_mem_decodeList` directly. -/
theorem subsystemU_computable_iff_index_set_re (x : (SubD P).Element) :
    ComputableElement (SubDPresentation P) x  <->
      REPred (fun i => (toSubsystemSys P x).mem (P.X i)) := by
  have hiff : forall  n, x.mem (SubDX P n)  <->  forall  i  in  decodeList n, (
    toSubsystemSys P x).mem (P.X i) := by
    intro n; unfold SubDX; exact mem_nbhdGen_iff P x (decodeList n)
  constructor
  * intro hx
    have hx' : REPred (fun n => forall  i  in  decodeList n, (toSubsystemSys P x).mem
    (P.X i)) :=
      REPred.of_iff (fun n => (hiff n).symm) hx.out
    refine REPred.of_iff (fun i => ?_) (hx'.comp primrec_encodeList_singleton)
    rw [decodeList_encodeList]
    simp only [List.mem_singleton]
    constructor
    * intro h j hj; rw [hj]; exact h
    * intro h; exact h i rfl
  * intro hx
    constructor
    exact REPred.of_iff hiff hx.forall_mem_decodeList

/-- **Subgoals 6b-6d, combined.** `x` is a computable element of `SubD P` iff the index
     set of
`D'x := toSubsystemSys P x` (relative to `P`'s own enumeration) is the range of a
    primitive
recursive function -- Scott's "index set r.e." reduced (Exercise 7.14) to an explicit
    enumerator.
` -> ` (6b): `Exercise714.repred_exists_primrec_range`, seeded by the witness `P.
    masterIdx`
(`D'x.mem (P.X P.masterIdx) = D'x.mem D.master`, always true). ` <- ` (6c):
`Exercise714.repred_range_primrec`. -/
theorem subsystemU_element_computable_iff (x : (SubD P).Element) :
    ComputableElement (SubDPresentation P) x  <->
      exists  r : Nat  ->  Nat, Nat.Primrec r  /\
        forall  i, (toSubsystemSys P x).mem (P.X i)  <->  exists  n, r n = i := by
  rw [subsystemU_computable_iff_index_set_re P x]
  constructor
  * intro hre
    have ha : (toSubsystemSys P x).mem (P.X P.masterIdx) := by
      rw [P.masterIdx_spec]; exact (toSubsystemSys P x).master_mem
    exact Exercise714.repred_exists_primrec_range hre ha
  * rintro <r, hr, hspec>
    exact REPred.of_iff hspec (Exercise714.repred_range_primrec hr)

end ComputableElements
```

```
section UniversalSubsystem
section TreeVal2

variable (r : Nat  ->  Nat) (x : (SubD UComputablePresentation).Element)

/-- **Subgoal 6f, main corollary.** If `x` is a computable element of `SubD
    UComputablePresentation`,
then `D'x := toSubsystemSys UComputablePresentation x` is effectively given --
    completing, for
`D = ?`, the second half of Exercise 8.15: computable elements of `SubD
    UComputablePresentation`
correspond to effectively presented domains, presented *relative to* `
    UComputablePresentation`'s own
enumeration (see the 6g note in `arxiv.md` for why the fully general "arbitrary
    presentation"
converse -- genuine "uniqueness of computable numberings" -- is out of scope). -/
theorem subsystemU_effectivelyGiven_of_isComputableElement
    (hx : ComputableElement (SubDPresentation UComputablePresentation) x) :
    (toSubsystemSys UComputablePresentation x).IsEffectivelyGiven := by
  obtain <r, hr, hspec> := (subsystemU_element_computable_iff UComputablePresentation
    x).mp hx
  exact <Q'xPresentation r x hr hspec>

end TreeVal2
end UniversalSubsystem

/-- **Exercise 8.15, first half.** If `D` is effectively given, there is an effectively
    given
neighbourhood system (`SubD P`, on tokens `List Nat`) whose elements are order-
    isomorphic to
`{D' ? D' ? D}` -- a *direct*, choice-free construction (Subgoals 1-4), unlike the
    classical route
through `Exercise222.reprIso` (`Proposition611.subsystemReprIso`). -/
theorem subsystem_isEffectivelyGiven_of_isEffectivelyGiven (hD : D.IsEffectivelyGiven)
    :
    exists  V : NeighborhoodSystem (List Nat), V.IsEffectivelyGiven  /\
      Nonempty (V.Element ?o {D' : NeighborhoodSystem a // D' ? D}) := by
  obtain <P> := hD
  exact <SubD P, <SubDPresentation P>, <subsystemIso P>>

end DomainTheory.NeighbourhoodSystems
```

The tip then assembles those tools into Scott's statement.

— exercise_8_15

Exercise_8_15.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    SubsystemDomain

@[expose] public section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem
```

```
/-- **Exercise 8.15 (Scott 1981, PRG-19).** If `D` is effectively given, the domain of
    subsystems
`{D' ? D' ? D}` is (order-isomorphic to the elements of) an effectively given
    neighbourhood system. -/
theorem exercise_8_15 {a : Type*} {D : NeighborhoodSystem a} (hD : D.
    IsEffectivelyGiven) :
    exists  V : NeighborhoodSystem (List Nat), V.IsEffectivelyGiven  /\
      Nonempty (V.Element ?o {D' : NeighborhoodSystem a // D' ? D}) :=
  subsystem_isEffectivelyGiven_of_isEffectivelyGiven hD

end
```

3.9.18 Exercise 8.16 — order of projections

For finitary projections $a : E \to E$, write

$$D_a = \{X \in E \mid X \sqsubseteq aX\}$$

(cf. 8.5.). Show that for any two such projections $a, b : E \to E$ we have

$$a \sqsubseteq b \quad \text{iff} \quad D_a \lhd D_b.$$

(This fills in the gap at the end of the proof of 8.6.) Also finish off the proof of 8.8 by showing that if E is effectively given and $a : E \to E$ is computable, then D_a is effectively given.

Tip summary The tip packages the order correspondence for finitary projections with the effectivity of their fixed neighbourhoods.

Development Matches the order of finitary projections with subdomain order and records effectivity of `D_a`.

`FixedNbhdFinitaryOrder.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Definition_7_2

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Examples_8_4

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Theorem_8_8

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open Domain.Recursive NeighborhoodSystem
open NeighborhoodSystem ApproximableMap
section
universe u
```

```
variable {a : Type u} {E : NeighborhoodSystem a}
/- concordance: source L5163 EXERCISE 8.16 order of projections -/
/-- **Exercise 8.16, Part 1 (Scott 1981, PRG-19).** For finitary projections `a, b : E
     ->  E`,
`a  <=  b  <->  D_a ? D_b` where `D_a := fixedNbhd a = {X  in  E ? X a X}` (Theorem
    8.5's subdomain). This is
exactly Theorem 8.6(a)'s order-isomorphism `finitaryProjectionSubsystemEquiv`,
    restricted from
`sub`-fixed points to finitary projections via `sub_eq_self_iff_isFinitaryProjection`,
    and fills the
gap Scott leaves at the end of the proof of Theorem 8.6. -/
theorem isFinitaryProjection_le_iff_fixedNbhd_subsystem {a b : ApproximableMap E E}
    (ha : FinitaryProjection a) (hb : FinitaryProjection b) :
    a  <=  b  <->  fixedNbhd a ? fixedNbhd b := by
  have hsa : sub a = a := sub_eq_self_iff_isFinitaryProjection.mpr ha
  have hsb : sub b = b := sub_eq_self_iff_isFinitaryProjection.mpr hb
  set fa : {f : ApproximableMap E E // sub f = f} := <a, hsa> with hfa_def
  set fb : {f : ApproximableMap E E // sub f = f} := <b, hsb> with hfb_def
  constructor
  * intro hab
    have hfab : fa  <=  fb := hab
    exact (finitaryProjectionSubsystemEquiv E).monotone hfab
  * intro hD
    have hle : (finitaryProjectionSubsystemEquiv E) fa  <=  (
    finitaryProjectionSubsystemEquiv E) fb :=
      hD
    have hback := (finitaryProjectionSubsystemEquiv E).symm.monotone hle
    rwa [OrderIso.symm_apply_apply, OrderIso.symm_apply_apply] at hback

/-- **Exercise 8.16 (Scott 1981, PRG-19), in full.**

Part 1: for finitary projections `a, b : E  ->  E`, `a  <=  b  <->  D_a ? D_b` (fills
    the gap at the end of
Theorem 8.6's proof).

Part 2: if `a : E  ->  E` is computable relative to a `ComputablePresentation P` of `E`
     -- i.e. `E` is
effectively given (via `P`) and `a` is computable (relative to `P`) -- then `D_a` is
    effectively
given (finishes off Theorem 8.8's proof). This is `Theorem88m.lean`'s `
    fixedNbhd_isEffectivelyGiven`,
cited here under the exercise's name. -/
theorem exercise_8_16_assembly :
    (forall  {a b : ApproximableMap E E}, FinitaryProjection a  ->  FinitaryProjection
     b  ->
        (a  <=  b  <->  fixedNbhd a ? fixedNbhd b))  /\
      (forall  {P : ComputablePresentation E} {a : ApproximableMap E E},
        ComputableMap P P a  ->  (fixedNbhd a).IsEffectivelyGiven) :=
  <fun ha hb => isFinitaryProjection_le_iff_fixedNbhd_subsystem ha hb,
    fun hcomp => fixedNbhd_isEffectivelyGiven hcomp>
end

end DomainTheory.NeighbourhoodSystems
```

The tip then assembles those tools into Scott's statement.

— exercise_8_16

Exercise_8_16.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    FixedNbhdFinitaryOrder

@[expose] public section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap Domain.
    Recursive

/-- **Exercise 8.16 (Scott 1981, PRG-19).** -/
theorem exercise_8_16 {a : Type*} {E : NeighborhoodSystem a} :
    (forall  {a b : ApproximableMap E E}, FinitaryProjection a  ->  FinitaryProjection
     b  ->
        (a  <=  b  <->  fixedNbhd a ? fixedNbhd b))  /\
      (forall  {P : ComputablePresentation E} {a : ApproximableMap E E},
        ComputableMap P P a  ->  (fixedNbhd a).IsEffectivelyGiven) :=
  @exercise_8_16_assembly a E

end
```

3.9.19 Exercise 8.17 — explicit embeddings into U

Find explicitly (if possible) the projection pairs for $\mathcal{U}+\mathcal{U}$, $\mathcal{U}\times\mathcal{U}$, and $\mathcal{U}\to\mathcal{U}$ needed for 8.9. Are any of these domains isomorphic with $\mathcal{U}$? (The author does not know a really good construction for $\mathcal{U}\to\mathcal{U}$.) Find a universal domain $V\neq\mathcal{U}$.

Tip summary The tip asserts universality of `V` together with the product/sum/function-space embeddings into `U`.

Development Proves `V` is universal and records the standard embeddings into `U`.
— `ProjPair` and composition/iso/subsystem constructors

ProjPair.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_13
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Theorem_7_6

@[expose] public section

section
universe u v w

namespace DomainTheory.NeighbourhoodSystems
```

```
open NeighborhoodSystem ApproximableMap Subsystem

variable {a : Type u} {b : Type v} {? : Type w}

structure ProjPair (D : NeighborhoodSystem a) (E : NeighborhoodSystem b) where
  /-- The injection `i : D  ->  E`. -/
  inj : ApproximableMap D E
  /-- The projection `j : E  ->  D`. -/
  proj : ApproximableMap E D
  /-- `j  comp  i = I_D`. -/
  proj_comp_inj : proj.comp inj = idMap D
  /-- `i  comp  j  subseteq  I_E`. -/
  inj_comp_proj_le : inj.comp proj  <=  idMap E

namespace ProjPair

def comp {D : NeighborhoodSystem a} {E : NeighborhoodSystem b}
    {F : NeighborhoodSystem ?} (p1 : ProjPair D E) (p2 : ProjPair E F) :
    ProjPair D F where
  inj := p2.inj.comp p1.inj
  proj := p1.proj.comp p2.proj
  proj_comp_inj := by
    show (p1.proj.comp p2.proj).comp (p2.inj.comp p1.inj) = idMap D
    rw [comp_assoc,  <-  comp_assoc p2.proj p2.inj p1.inj, p2.proj_comp_inj,
    idMap_comp,
      p1.proj_comp_inj]
  inj_comp_proj_le := by
    show (p2.inj.comp p1.inj).comp (p1.proj.comp p2.proj)  <=  idMap F
    rw [comp_assoc,  <-  comp_assoc p1.inj p1.proj p2.proj]
    calc p2.inj.comp ((p1.inj.comp p1.proj).comp p2.proj)
         <=  p2.inj.comp ((idMap E).comp p2.proj) :=
          comp_mono_gen le_rfl (comp_mono_gen p1.inj_comp_proj_le le_rfl)
      _ = p2.inj.comp p2.proj := by rw [idMap_comp]
      _  <=  idMap F := p2.inj_comp_proj_le

def ofIso {D : NeighborhoodSystem a} {E : NeighborhoodSystem b} (e : DomainIso D E) :
    ProjPair D E where
  inj := ApproximableMap.ofIso e
  proj := ApproximableMap.ofIso e.symm
  proj_comp_inj := by
    have heq : forall  x, ((ApproximableMap.ofIso e.symm).comp (ApproximableMap.ofIso
    e)).toElementMap x
        = (idMap D).toElementMap x := by
      intro x
      rw [toElementMap_comp, toElementMap_ofIso, toElementMap_ofIso, OrderIso.
    symm_apply_apply,
        toElementMap_idMap]
    exact le_antisymm (le_iff_toElementMap_le.mpr fun x => (heq x).le)
      (le_iff_toElementMap_le.mpr fun x => (heq x).ge)
  inj_comp_proj_le := by
    have heq : forall  y, ((ApproximableMap.ofIso e).comp (ApproximableMap.ofIso e.
    symm)).toElementMap y
        = (idMap E).toElementMap y := by
      intro y
      rw [toElementMap_comp, toElementMap_ofIso, toElementMap_ofIso, OrderIso.
```

```
    apply_symm_apply,
        toElementMap_idMap]
    exact le_iff_toElementMap_le.mpr fun y => (heq y).le

def ofSubsystem {D E : NeighborhoodSystem a} (h : D ? E) : ProjPair D E where
  inj := h.inj
  proj := h.proj
  proj_comp_inj := h.proj_comp_inj
  inj_comp_proj_le := h.inj_comp_proj_le

end ProjPair

end DomainTheory.NeighbourhoodSystems
end
```

— universality and the concrete U/V embeddings

EmbeddingsIntoU.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Exercise_8_12
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.ProjPair
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Proposition_8_10
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_13

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Theorem_7_6
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
section
/- concordance: source L5173 EXERCISE 8.17 explicit embeddings into U -/
open NeighborhoodSystem ApproximableMap Subsystem
universe u v w
variable {a : Type u} {b : Type v} {? : Type w}

/-- **Universality, as an abstract predicate**: every countable neighbourhood system
    embeds
(up to isomorphism) as a subsystem of `W` -- Theorem 8.8(a)'s property, extracted from
    the specific
domain `?` to an arbitrary `W`. -/
def IsUniversal {? : Type w} (W : NeighborhoodSystem ?) : Prop :=
  forall  {a : Type u} (D : NeighborhoodSystem a) [Countable {S : Set a // D.mem S}],
    D ? W

/-- **`D ? E` composed with an isomorphism `D  ~= ? D'` on the left is still `D ? E`**
    -- immediate
from unfolding `?` and composing the two `DomainIso`s (`OrderIso.trans`). -/
theorem trianglelefteq_of_isomorphic_left {D : NeighborhoodSystem a} {D' :
    NeighborhoodSystem b}
```

```
    {E : NeighborhoodSystem ?} (hDD' : D  ~= ? D') (hD'E : D' ? E) : D ? E := by
  obtain <F, hFE, <eD'F>> := hD'E
  obtain <eDD'> := hDD'
  exact <F, hFE, <eDD'.trans eD'F>>

/-- **`?` is transitive.** `D ? E` and `E ? F` give `D ? F`: unfold both to a subsystem
    `D' ? E`
(`D  ~= ? D'`) and `E' ? F` (`E  ~= ? E'`); `ProjPair.ofSubsystem` turns `D' ? E`/`E' ?
    F` into projection
pairs, `ProjPair.ofIso` turns `E  ~= ? E'` into one too, and composing all three (`
    ProjPair.comp`) gives
a projection pair `D'  <=>  F`, hence `D' ? F` (Lemma 6.15). Gluing back `D  ~= ? D'`
    on the left
(`trianglelefteq_of_isomorphic_left`) finishes. -/
theorem trianglelefteq_trans {D : NeighborhoodSystem a} {E : NeighborhoodSystem b}
    {F : NeighborhoodSystem ?} (h1 : D ? E) (h2 : E ? F) : D ? F := by
  obtain <D', hD'E, <eDD'>> := h1
  obtain <E', hE'F, <eEE'>> := h2
  have pD'F : ProjPair D' F :=
    ((ProjPair.ofSubsystem hD'E).comp (ProjPair.ofIso eEE')).comp (ProjPair.
    ofSubsystem hE'F)
  have hD'F : D' ? F :=
    trianglelefteq_of_projectionPair pD'F.inj pD'F.proj pD'F.proj_comp_inj pD'F.
    inj_comp_proj_le
  exact trianglelefteq_of_isomorphic_left <eDD'> hD'F

/-- **`?` is universal** (restating Theorem 8.8(a) in `?`-shape). -/
theorem U_isUniversal : IsUniversal.{u} U := by
  intro a D _
  obtain <D', hiso, hsub> := theorem_8_8_a D
  exact <D', hsub, hiso>

/-- **Exercise 8.17, Part 1 (` -> `).** `iArrow/jArrow` already are the pair `? -> ?
    <=>  ?`; bonus `?`. -/
theorem funSpaceUU_trianglelefteq_U : funSpace U U ? U :=
  trianglelefteq_of_projectionPair iArrow jArrow jArrow_comp_iArrow
    iArrow_comp_jArrow_le

/-- **Universality transports along isomorphism.** If `W` is universal and `W  ~= ? W
    '`, then `W'` is
universal too: every countable `D` embeds in `W` (`D ? W`), and `W ? W'` (take `D' := W
    '` in `?`'s
definition, using `Subsystem.refl` and the given isomorphism), so `trianglelefteq_trans
    ` finishes. -/
theorem isUniversal_of_isomorphic {W : NeighborhoodSystem b} {W' : NeighborhoodSystem
    ?}
    (hW : IsUniversal.{u} W) (hWW' : W  ~= ? W') : IsUniversal.{u} W' := by
  have hWW'_tri : W ? W' := hWW'.elim fun e => <W', Subsystem.refl W', <e>>
  intro a D _
  exact trianglelefteq_trans (hW D) hWW'_tri

/-- **Exercise 8.17, Part 3.** Exercise 8.12's `V` (a `NeighborhoodSystem Nat`, built
    from `2^kNat+?` --
literally a *different* neighbourhood system from `U : NeighborhoodSystem ?`, not
    merely a
relabelling) is a universal domain, since it is effectively isomorphic to `?` (`
```

```
    effectiveIso812_UV`,
Exercise 8.12's headline) and universality transports along isomorphism. -/
theorem V.isUniversal : IsUniversal.{u} V :=
  isUniversal_of_isomorphic U_isUniversal <effectiveIso812_UV.toDomainIso>

/-- **Exercise 8.17, Part 1 (` x `).** `iTimes/jTimes` already are the pair `? x ?  <=>
      ?`; bonus `?`. -/
theorem prodUU_trianglelefteq_U : prod U U ? U :=
  trianglelefteq_of_projectionPair iTimes jTimes jTimes_comp_iTimes
    iTimes_comp_jTimes_le

/-- **Exercise 8.17, Part 1 (`+`).** `iPlus/jPlus` (`Definition89.lean`) already *are*
    the computable
projection pair `?+?  <=>  ?` needed for 8.9. Bonus: Lemma 6.15 upgrades it to `?+? ?
    ?`. -/
theorem sumUU_trianglelefteq_U : sum U U U_mem_nonempty U_mem_nonempty ? U :=
  trianglelefteq_of_projectionPair iPlus jPlus jPlus_comp_iPlus iPlus_comp_jPlus_le

end

end DomainTheory.NeighbourhoodSystems
```

The tip then assembles those tools into Scott's statement.
— exercise_8_17

Exercise_8_17.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    EmbeddingsIntoU

@[expose] public section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem

/-- **Exercise 8.17 (Scott 1981, PRG-19).** `V` is universal, and the standard
    embeddings
`? x ?`, `?+?`, `? -> ?` into `?` hold. -/
theorem exercise_8_17 :
    IsUniversal.{0} V  /\
      (prod U U ? U)  /\
      (sum U U U_mem_nonempty U_mem_nonempty ? U)  /\
      (funSpace U U ? U) :=
  <V.isUniversal, prodUU_trianglelefteq_U, sumUU_trianglelefteq_U,
    funSpaceUU_trianglelefteq_U>

end
```

3.9.20 Exercise 8.18 — missing proofs of 8.10

Many of the cases of 8.10 were left unproved. Please establish these assertions explicitly.

Tip summary The tip asserts that the sum/product/arrow combinators preserve projections and finitary projections.

Development Shows sum/product/arrow combinators preserve (finitary) projections.

ProjectionCombinators.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Definition_8_9

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Proposition_8_10

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap
/- concordance: source L5177 EXERCISE 8.18 missing proofs of 8.10 -/
/-- **Exercise 8.18 (Scott 1981, PRG-19), in full: Proposition 8.10, both halves, all
    three
combinators.** If `a, b : ?  ->  ?` are projections, then so are `a+b`, `a x b`, `a ->
    b`
(`isProjection_combinators`); if moreover `a, b` are finitary, then so are `a+b`, `a x
    b`, `a -> b`
(`finitaryProjection_combinators`) -- and (packaged inside `FinitaryProjection`'s `
    IsFinitary`
witness) the fixed-point set of each is isomorphic to the corresponding construct
    applied to `D_a`,
`D_b` (`sumComb_elementIso`/`prodComb_elementIso`/`arrowComb_elementIso` in `
    Proposition810b.lean`).
No case of Proposition 8.10 is left unestablished. -/
theorem exercise_8_18_assembly {a b : ApproximableMap U U} (ha : IsProjection a) (hb :
     IsProjection b) :
    (IsProjection (sumComb a b)  /\  IsProjection (prodComb a b)  /\  IsProjection (
    arrowComb a b))  /\
      (FinitaryProjection a  ->  FinitaryProjection b  ->
        FinitaryProjection (sumComb a b)  /\  FinitaryProjection (prodComb a b)  /\
          FinitaryProjection (arrowComb a b)) :=
  <isProjection_combinators ha hb, fun hfa hfb => finitaryProjection_combinators hfa
    hfb>

set_option linter.unusedVariables false

theorem example_818 : forall
  {a b :
    @DomainTheory.NeighbourhoodSystems.ApproximableMap Rat Rat DomainTheory.
    NeighbourhoodSystems.U
      DomainTheory.NeighbourhoodSystems.U},
  @DomainTheory.NeighbourhoodSystems.IsProjection Rat DomainTheory.
    NeighbourhoodSystems.U a  ->
    @DomainTheory.NeighbourhoodSystems.IsProjection Rat DomainTheory.
    NeighbourhoodSystems.U b  ->
      And
        (And
```

```
          (@DomainTheory.NeighbourhoodSystems.IsProjection Rat DomainTheory.
    NeighbourhoodSystems.U
            (DomainTheory.NeighbourhoodSystems.sumComb a b))
          (And
            (@DomainTheory.NeighbourhoodSystems.IsProjection Rat DomainTheory.
    NeighbourhoodSystems.U
              (DomainTheory.NeighbourhoodSystems.prodComb a b))
            (@DomainTheory.NeighbourhoodSystems.IsProjection Rat DomainTheory.
    NeighbourhoodSystems.U
              (DomainTheory.NeighbourhoodSystems.arrowComb a b))))
        (@DomainTheory.NeighbourhoodSystems.FinitaryProjection Rat DomainTheory.
    NeighbourhoodSystems.U a  ->
          @DomainTheory.NeighbourhoodSystems.FinitaryProjection Rat DomainTheory.
    NeighbourhoodSystems.U b  ->
           And
            (@DomainTheory.NeighbourhoodSystems.FinitaryProjection Rat DomainTheory.
    NeighbourhoodSystems.U
              (DomainTheory.NeighbourhoodSystems.sumComb a b))
            (And
              (@DomainTheory.NeighbourhoodSystems.FinitaryProjection Rat
    DomainTheory.NeighbourhoodSystems.U
                (DomainTheory.NeighbourhoodSystems.prodComb a b))
              (@DomainTheory.NeighbourhoodSystems.FinitaryProjection Rat
    DomainTheory.NeighbourhoodSystems.U
                (DomainTheory.NeighbourhoodSystems.arrowComb a b)))) :=
  @DomainTheory.NeighbourhoodSystems.exercise_8_18_assembly

end DomainTheory.NeighbourhoodSystems
```

The tip then assembles those tools into Scott's statement.

— exercise_8_18

Exercise_8_18.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    ProjectionCombinators

@[expose] public section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap

/-- **Exercise 8.18 (Scott 1981, PRG-19).** Sum/product/arrow combinators preserve
    projections
(and finitary projections). -/
theorem exercise_8_18 {a b : ApproximableMap U U} (ha : IsProjection a) (hb :
    IsProjection b) :
    (IsProjection (sumComb a b)  /\  IsProjection (prodComb a b)  /\  IsProjection (
    arrowComb a b))  /\
      (FinitaryProjection a  ->  FinitaryProjection b  ->
        FinitaryProjection (sumComb a b)  /\  FinitaryProjection (prodComb a b)  /\
          FinitaryProjection (arrowComb a b)) :=
  exercise_8_18_assembly ha hb

end
```

3.9.21 Exercise 8.19 — closure under sum and product

Suppose we know both

$$T \quad \text{and} \quad E \to E \trianglelefteq E\,.$$

Does it follow that $E + E$ and $E \times E \trianglelefteq E$?

Tip summary The tip asserts closure of $\trianglelefteq$ under sum and product under Scott's hypotheses.

Development Closes $\trianglelefteq$ under sum and product given a terminal object and a function-space retract.

SumProdRetracts.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.Exercise_6_29
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_13
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_12
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Lemma_3_6

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    EmbeddingsIntoU
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    EmbeddingsIntoU_more
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem ApproximableMap Exercise326
universe u
/- concordance: source L5179 EXERCISE 8.19 closure under sum and product -/
/-- **Exercise 8.19.** If `T ? E` and `E  ->  E ? E`, then `E + E ? E` and `E  x  E ? E
    `. -/
theorem exercise_8_19_assembly {a : Type u} {E : NeighborhoodSystem a}
    (hE : forall  X, E.mem X  ->  X.Nonempty) (hT : TD ? E) (hArrow : funSpace E E ? E
    ) :
    sum E E hE hE ? E  /\  prod E E ? E :=
  <sum_trianglelefteq_of hE hT hArrow, prod_trianglelefteq_of hT hArrow>
end

set_option linter.unusedVariables false

theorem example_819.{u} : And
  (And
    (forall  {a : Type u} {E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
      (hE : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
    .mem a E X  ->  @Set.Nonempty a X),
      @DomainTheory.NeighbourhoodSystems.Trianglelefteq DomainTheory.
    NeighbourhoodSystems.Example12.Token a
          DomainTheory.NeighbourhoodSystems.Exercise326.TD E  ->
```

```
      @DomainTheory.NeighbourhoodSystems.Trianglelefteq (@DomainTheory.
  NeighbourhoodSystems.ApproximableMap a a E E) a
           (@DomainTheory.NeighbourhoodSystems.funSpace a a E E) E  ->
         And
           (@DomainTheory.NeighbourhoodSystems.Trianglelefteq (Option (Sum a a)) a
             (@DomainTheory.NeighbourhoodSystems.sum a a E E hE hE) E)
           (@DomainTheory.NeighbourhoodSystems.Trianglelefteq (Sum a a) a
             (@DomainTheory.NeighbourhoodSystems.prod a a E E) E))
   (forall  {a : Type u} {E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a
   },
     @DomainTheory.NeighbourhoodSystems.Trianglelefteq DomainTheory.
   NeighbourhoodSystems.Example12.Token a
         DomainTheory.NeighbourhoodSystems.Exercise326.TD E  ->
       @DomainTheory.NeighbourhoodSystems.Trianglelefteq (@DomainTheory.
   NeighbourhoodSystems.ApproximableMap a a E E) a
           (@DomainTheory.NeighbourhoodSystems.funSpace a a E E) E  ->
         @DomainTheory.NeighbourhoodSystems.Trianglelefteq (Sum a a) a
           (@DomainTheory.NeighbourhoodSystems.prod a a E E) E))
(forall  {a : Type u} {E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
  (hE : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
  mem a E X  ->  @Set.Nonempty a X),
  @DomainTheory.NeighbourhoodSystems.Trianglelefteq DomainTheory.
  NeighbourhoodSystems.Example12.Token a
      DomainTheory.NeighbourhoodSystems.Exercise326.TD E  ->
    @DomainTheory.NeighbourhoodSystems.Trianglelefteq (@DomainTheory.
  NeighbourhoodSystems.ApproximableMap a a E E) a
         (@DomainTheory.NeighbourhoodSystems.funSpace a a E E) E  ->
       @DomainTheory.NeighbourhoodSystems.Trianglelefteq (Option (Sum a a)) a
         (@DomainTheory.NeighbourhoodSystems.sum a a E E hE hE) E) :=
@And.intro
(And
  (forall  {a : Type u} {E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
    (hE : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
   .mem a E X  ->  @Set.Nonempty a X),
    @DomainTheory.NeighbourhoodSystems.Trianglelefteq DomainTheory.
   NeighbourhoodSystems.Example12.Token a
         DomainTheory.NeighbourhoodSystems.Exercise326.TD E  ->
       @DomainTheory.NeighbourhoodSystems.Trianglelefteq (@DomainTheory.
   NeighbourhoodSystems.ApproximableMap a a E E) a
           (@DomainTheory.NeighbourhoodSystems.funSpace a a E E) E  ->
         And
           (@DomainTheory.NeighbourhoodSystems.Trianglelefteq (Option (Sum a a)) a
             (@DomainTheory.NeighbourhoodSystems.sum a a E E hE hE) E)
           (@DomainTheory.NeighbourhoodSystems.Trianglelefteq (Sum a a) a
             (@DomainTheory.NeighbourhoodSystems.prod a a E E) E))
   (forall  {a : Type u} {E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a
   },
     @DomainTheory.NeighbourhoodSystems.Trianglelefteq DomainTheory.
   NeighbourhoodSystems.Example12.Token a
         DomainTheory.NeighbourhoodSystems.Exercise326.TD E  ->
       @DomainTheory.NeighbourhoodSystems.Trianglelefteq (@DomainTheory.
   NeighbourhoodSystems.ApproximableMap a a E E) a
           (@DomainTheory.NeighbourhoodSystems.funSpace a a E E) E  ->
         @DomainTheory.NeighbourhoodSystems.Trianglelefteq (Sum a a) a
           (@DomainTheory.NeighbourhoodSystems.prod a a E E) E))
(forall  {a : Type u} {E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
```

```
    (hE : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem.
    mem a E X  ->  @Set.Nonempty a X),
    @DomainTheory.NeighbourhoodSystems.Trianglelefteq DomainTheory.
    NeighbourhoodSystems.Example12.Token a
        DomainTheory.NeighbourhoodSystems.Exercise326.TD E  ->
      @DomainTheory.NeighbourhoodSystems.Trianglelefteq (@DomainTheory.
    NeighbourhoodSystems.ApproximableMap a a E E) a
          (@DomainTheory.NeighbourhoodSystems.funSpace a a E E) E  ->
        @DomainTheory.NeighbourhoodSystems.Trianglelefteq (Option (Sum a a)) a
          (@DomainTheory.NeighbourhoodSystems.sum a a E E hE hE) E)
  (@And.intro
    (forall  {a : Type u} {E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a}
      (hE : forall  (X : Set a), @DomainTheory.NeighbourhoodSystems.NeighborhoodSystem
    .mem a E X  ->  @Set.Nonempty a X),
      @DomainTheory.NeighbourhoodSystems.Trianglelefteq DomainTheory.
    NeighbourhoodSystems.Example12.Token a
          DomainTheory.NeighbourhoodSystems.Exercise326.TD E  ->
        @DomainTheory.NeighbourhoodSystems.Trianglelefteq (@DomainTheory.
    NeighbourhoodSystems.ApproximableMap a a E E) a
            (@DomainTheory.NeighbourhoodSystems.funSpace a a E E) E  ->
          And
            (@DomainTheory.NeighbourhoodSystems.Trianglelefteq (Option (Sum a a)) a
              (@DomainTheory.NeighbourhoodSystems.sum a a E E hE hE) E)
            (@DomainTheory.NeighbourhoodSystems.Trianglelefteq (Sum a a) a
              (@DomainTheory.NeighbourhoodSystems.prod a a E E) E))
    (forall  {a : Type u} {E : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a
    },
      @DomainTheory.NeighbourhoodSystems.Trianglelefteq DomainTheory.
    NeighbourhoodSystems.Example12.Token a
          DomainTheory.NeighbourhoodSystems.Exercise326.TD E  ->
        @DomainTheory.NeighbourhoodSystems.Trianglelefteq (@DomainTheory.
    NeighbourhoodSystems.ApproximableMap a a E E) a
            (@DomainTheory.NeighbourhoodSystems.funSpace a a E E) E  ->
          @DomainTheory.NeighbourhoodSystems.Trianglelefteq (Sum a a) a
            (@DomainTheory.NeighbourhoodSystems.prod a a E E) E)
    @DomainTheory.NeighbourhoodSystems.exercise_8_19_assembly @DomainTheory.
    NeighbourhoodSystems.prod_trianglelefteq_of)
  @DomainTheory.NeighbourhoodSystems.sum_trianglelefteq_of

section
open NeighborhoodSystem ApproximableMap
universe u v

section FunSpace

variable {a : Type u} {b : Type v} (A : NeighborhoodSystem a) (B : NeighborhoodSystem
    b)

/-- **`b  |->  (fun _. b)`, the constant-function embedding.** `curry (proj_0 B A) : B
     ->  (A  ->  B)` sends `b`
to the function that ignores its argument and always returns `b`. -/
noncomputable def constFunMap : ApproximableMap B (funSpace A B) := curry (proj_0 B A)

end FunSpace

section Prod
```

```
variable {a : Type u} {b : Type v} (A : NeighborhoodSystem a) (B : NeighborhoodSystem
    b)

/-- **`a  |->  <a, False>`.** -/
noncomputable def embedFst : ApproximableMap A (prod A B) := paired (idMap A) (
    constMap A B.bot)

/-- **`b  |->  <False, b>`.** -/
noncomputable def embedSnd : ApproximableMap B (prod A B) := paired (constMap B A.bot)
     (idMap B)

end Prod

section FunSpace

variable {a : Type u} {b : Type v} (A : NeighborhoodSystem a) (B : NeighborhoodSystem
    b)

/-- **`phi  |->  phi(False)`, evaluation at the bottom.** -/
noncomputable def evalBotMap : ApproximableMap (funSpace A B) B :=
  (evalMap A B).comp (paired (idMap (funSpace A B)) (constMap (funSpace A B) A.bot))

@[simp] theorem toApproxMap_constFunMap (b : B.Element) (a : A.Element) :
    (toApproxMap ((constFunMap A B).toElementMap b)).toElementMap a = b := by
  rw [constFunMap, toElementMap_curry_apply, toElementMap_proj_0, fst_pair]

end FunSpace

variable {a : Type u} (D : NeighborhoodSystem a)

end

section
open NeighborhoodSystem ApproximableMap
universe u v

section Prod

variable {a : Type u} {b : Type v} (A : NeighborhoodSystem a) (B : NeighborhoodSystem
    b)

theorem embedFst_comp_proj_0_le : (embedFst A B).comp (proj_0 A B)  <=  idMap (prod A
    B) := by
  rw [le_iff_toElementMap_le]
  intro z
  rw [ <-  pair_fst_snd z]
  set x := z.fst; set y := z.snd
  rw [toElementMap_comp, toElementMap_idMap, toElementMap_proj_0, fst_pair, embedFst,
    toElementMap_paired, toElementMap_idMap, toElementMap_constMap]
  exact pair_le_pair_iff.mpr <le_refl x, B.bot_le y>

end Prod
end

section
```

```
open NeighborhoodSystem ApproximableMap
universe u

/-- **Multiplicity, product case.** `embedFst`/`embedSnd` (hence the two projection
    pairs above)
are genuinely *different maps* as soon as `D` has an element `x  !=  False`: evaluating
     both at `x` gives
`<x, False>` vs. `<False, x>`, which differ in their first coordinate (`fst_pair`). -/
theorem embedFst_ne_embedSnd_of_ne_bot {a : Type u} {D : NeighborhoodSystem a} {x : D.
    Element}
    (hx : x  !=  D.bot) : embedFst D D  !=  embedSnd D D := fun h => hx <| by
  have heq := congrArg (fun f => f.toElementMap x) h
  simp only [embedFst, embedSnd, toElementMap_paired, toElementMap_idMap,
    toElementMap_constMap] at heq
  simpa using congrArg (*.fst) heq
end

section
open NeighborhoodSystem ApproximableMap
universe u v

section Prod

variable {a : Type u} {b : Type v} (A : NeighborhoodSystem a) (B : NeighborhoodSystem
    b)

theorem embedSnd_comp_proj_1_le : (embedSnd A B).comp (proj_1 A B)  <=  idMap (prod A
    B) := by
  rw [le_iff_toElementMap_le]
  intro z
  rw [ <-  pair_fst_snd z]
  set x := z.fst; set y := z.snd
  rw [toElementMap_comp, toElementMap_idMap, toElementMap_proj_1, snd_pair, embedSnd,
    toElementMap_paired, toElementMap_idMap, toElementMap_constMap]
  exact pair_le_pair_iff.mpr <A.bot_le x, le_refl y>

theorem proj_0_comp_embedFst : (proj_0 A B).comp (embedFst A B) = idMap A := by
  apply le_antisymm <;> rw [le_iff_toElementMap_le] <;> intro a <;>
    simp [embedFst, toElementMap_comp, toElementMap_idMap, toElementMap_paired,
    toElementMap_constMap,
      toElementMap_proj_0, fst_pair]

theorem proj_1_comp_embedSnd : (proj_1 A B).comp (embedSnd A B) = idMap B := by
  apply le_antisymm <;> rw [le_iff_toElementMap_le] <;> intro b <;>
    simp [embedSnd, toElementMap_comp, toElementMap_idMap, toElementMap_paired,
    toElementMap_constMap,
      toElementMap_proj_1, snd_pair]

end Prod

section FunSpace

variable {a : Type u} {b : Type v} (A : NeighborhoodSystem a) (B : NeighborhoodSystem
    b)

theorem toElementMap_evalBotMap (phi : (funSpace A B).Element) :
```

```
    (evalBotMap A B).toElementMap phi = (toApproxMap phi).toElementMap A.bot := by
  simp [evalBotMap, toElementMap_comp, toElementMap_paired, toElementMap_idMap,
    toElementMap_constMap, evalMap_apply]

/-- `const_{phi(False)}  <=  phi` pointwise, since `phi(False)  <=  phi(x)` for every `
    x` (monotonicity,
`toElementMap_mono`, Proposition 2.2(iii), applied to `False  <=  x`). -/
theorem constFunMap_comp_evalBotMap_le :
    (constFunMap A B).comp (evalBotMap A B)  <=  idMap (funSpace A B) := by
  rw [le_iff_toElementMap_le]
  intro phi
  rw [toElementMap_comp, toElementMap_idMap]
  apply (funSpaceEquiv A B).map_rel_iff.mp
  rw [funSpaceEquiv_apply, funSpaceEquiv_apply, le_iff_toElementMap_le]
  intro a
  rw [toApproxMap_constFunMap, toElementMap_evalBotMap]
  exact toElementMap_mono (toApproxMap phi) (A.bot_le a)

theorem evalBotMap_comp_constFunMap : (evalBotMap A B).comp (constFunMap A B) = idMap
    B := by
  apply le_antisymm <;> rw [le_iff_toElementMap_le] <;> intro b <;>
    rw [toElementMap_comp, toElementMap_idMap, toElementMap_evalBotMap,
    toApproxMap_constFunMap]

/-- **`B ? funSpace A B`**, via `b  |->  (fun _. b)` / evaluation at `False`. -/
theorem trianglelefteq_funSpace_const : B ? funSpace A B :=
  trianglelefteq_of_projectionPair (constFunMap A B) (evalBotMap A B)
    (evalBotMap_comp_constFunMap A B) (constFunMap_comp_evalBotMap_le A B)

end FunSpace

end

section
open NeighborhoodSystem ApproximableMap
universe u
variable {a : Type u} (D : NeighborhoodSystem a)

/-- **Exercise 8.20, `D ? D -> D`.** -/
theorem D_trianglelefteq_funSpace : D ? funSpace D D := trianglelefteq_funSpace_const
    D D
end

section
open NeighborhoodSystem ApproximableMap
universe u v

section Prod

variable {a : Type u} {b : Type v} (A : NeighborhoodSystem a) (B : NeighborhoodSystem
    b)

/-- **`A ? prod A B`**, via `a  |->  <a, False>` / `proj_0`. -/
theorem trianglelefteq_prod_fst : A ? prod A B :=
  trianglelefteq_of_projectionPair (embedFst A B) (proj_0 A B) (proj_0_comp_embedFst A
    B)
```

```
    (embedFst_comp_proj_0_le A B)

end Prod
end

section
open NeighborhoodSystem ApproximableMap
universe u
variable {a : Type u} (D : NeighborhoodSystem a)

/-- **Exercise 8.20, `D ? D x D`.** -/
theorem D_trianglelefteq_prod : D ? prod D D := trianglelefteq_prod_fst D D

end
end DomainTheory.NeighbourhoodSystems
```

The tip then assembles those tools into Scott's statement.

— `exercise_8_19`

Exercise_8_19.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    SumProdRetracts

@[expose] public section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap Exercise326

universe u

/-- **Exercise 8.19 (Scott 1981, PRG-19).** If `T ? E` and `E  ->  E ? E`, then `E+E ?
    E` and `E x E ? E`. -/
theorem exercise_8_19 {a : Type u} {E : NeighborhoodSystem a}
    (hE : forall  X, E.mem X  ->  X.Nonempty) (hT : TD ? E) (hArrow : funSpace E E ? E
    ) :
    sum E E hE hE ? E  /\  prod E E ? E :=
  exercise_8_19_assembly hE hT hArrow

end
```

3.9.22 Exercise 8.20 — self-embeddings of a domain

For any system we know $D \trianglelefteq D + D$, but what about

$$D \trianglelefteq D \times D \quad \text{and} \quad D \trianglelefteq D \to D\,?$$

Would these projections be computable if D is effectively given? Are there more than one projection pair in each case?

Tip summary The tip asserts $D \trianglelefteq D \times D$ and $D \trianglelefteq D \to D$ for every domain D.

Development Proves the unconditional self-embeddings $D \trianglelefteq D \times D$ and $D \trianglelefteq D \to D$.

DiagonalRetracts.lean

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Exercise_7_15
public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.Table_5_5

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    EmbeddingsIntoU
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Proposition_8_10
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    SumProdRetracts
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem ApproximableMap
universe u
variable {a : Type u} (D : NeighborhoodSystem a)
/- concordance: source L5185 EXERCISE 8.20 self-embeddings of a domain -/
/-- **Exercise 8.20.** For any domain `D`, `D ? D x D` and `D ? D -> D` (
    unconditionally -- no hypotheses
needed, unlike Exercise 8.19). See the module docstring for the discussion of
    computability and
multiplicity of projection pairs that Scott also asks about. -/
theorem exercise_8_20_assembly : D ? prod D D  /\  D ? funSpace D D :=
  <D_trianglelefteq_prod D, D_trianglelefteq_funSpace D>
end

set_option linter.unusedVariables false

theorem example_820.{u} : And
  (And
    (forall  {a : Type u} (D : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a)
    ,
      And
        (@DomainTheory.NeighbourhoodSystems.Trianglelefteq a (Sum a a) D
          (@DomainTheory.NeighbourhoodSystems.prod a a D D))
        (@DomainTheory.NeighbourhoodSystems.Trianglelefteq a
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a D D) D
          (@DomainTheory.NeighbourhoodSystems.funSpace a a D D)))
    (forall  {a : Type u} (D : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a)
    ,
      @DomainTheory.NeighbourhoodSystems.Trianglelefteq a (@DomainTheory.
    NeighbourhoodSystems.ApproximableMap a a D D) D
        (@DomainTheory.NeighbourhoodSystems.funSpace a a D D)))
  (forall  {a : Type u} (D : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a),
    @DomainTheory.NeighbourhoodSystems.Trianglelefteq a (Sum a a) D (@DomainTheory.
```

```
    NeighbourhoodSystems.prod a a D D)) :=
  @And.intro
  (And
    (forall  {a : Type u} (D : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a)
    ,
      And
        (@DomainTheory.NeighbourhoodSystems.Trianglelefteq a (Sum a a) D
          (@DomainTheory.NeighbourhoodSystems.prod a a D D))
        (@DomainTheory.NeighbourhoodSystems.Trianglelefteq a
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a D D) D
          (@DomainTheory.NeighbourhoodSystems.funSpace a a D D)))
    (forall  {a : Type u} (D : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a)
    ,
      @DomainTheory.NeighbourhoodSystems.Trianglelefteq a (@DomainTheory.
    NeighbourhoodSystems.ApproximableMap a a D D) D
        (@DomainTheory.NeighbourhoodSystems.funSpace a a D D)))
  (forall  {a : Type u} (D : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a),
    @DomainTheory.NeighbourhoodSystems.Trianglelefteq a (Sum a a) D (@DomainTheory.
    NeighbourhoodSystems.prod a a D D))
  (@And.intro
    (forall  {a : Type u} (D : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a)
    ,
      And
        (@DomainTheory.NeighbourhoodSystems.Trianglelefteq a (Sum a a) D
          (@DomainTheory.NeighbourhoodSystems.prod a a D D))
        (@DomainTheory.NeighbourhoodSystems.Trianglelefteq a
          (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a D D) D
          (@DomainTheory.NeighbourhoodSystems.funSpace a a D D)))
    (forall  {a : Type u} (D : DomainTheory.NeighbourhoodSystems.NeighborhoodSystem a)
    ,
      @DomainTheory.NeighbourhoodSystems.Trianglelefteq a (@DomainTheory.
    NeighbourhoodSystems.ApproximableMap a a D D) D
        (@DomainTheory.NeighbourhoodSystems.funSpace a a D D))
    @DomainTheory.NeighbourhoodSystems.exercise_8_20_assembly @DomainTheory.
    NeighbourhoodSystems.D_trianglelefteq_funSpace)
  @DomainTheory.NeighbourhoodSystems.D_trianglelefteq_prod

section
open NeighborhoodSystem ApproximableMap
universe u v

section Prod

variable {a : Type u} {b : Type v} (A : NeighborhoodSystem a) (B : NeighborhoodSystem
    b)

/-- **`B ? prod A B`**, via `b  |->  <False, b>` / `proj_1`. -/
theorem trianglelefteq_prod_snd : B ? prod A B :=
  trianglelefteq_of_projectionPair (embedSnd A B) (proj_1 A B) (proj_1_comp_embedSnd A
     B)
    (embedSnd_comp_proj_1_le A B)

end Prod
end

end DomainTheory.NeighbourhoodSystems
```

The tip then assembles those tools into Scott's statement.
— `exercise_8_20`

`Exercise_8_20.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    DiagonalRetracts

@[expose] public section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem

/-- **Exercise 8.20 (Scott 1981, PRG-19).** For any domain `D`, `D ? D x D` and `D ? D
    -> D`. -/
theorem exercise_8_20 {a : Type*} (D : NeighborhoodSystem a) :
    D ? prod D D  /\  D ? funSpace D D :=
  exercise_8_20_assembly D

end
```

3.9.23 Exercise 8.21 — lists of a projection

Using the fixed-point construction, show that there is a continuous and computable operator $\lambda a.\, a^{\S}$, such that if a is a finitary projection of U, then

$$D_{a^{\S}} \cong (D_a)^{\S}\,.$$

Tip summary The tip asserts that the sharp of a finitary projection is again a finitary projection.

Development Develops the sharp construction and shows it yields a finitary projection.

`ASharp.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Exercise_7_17

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Proposition_8_10
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Theorem_8_8

public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Exercise_3_24
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Proposition_7_3
public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.Table_5_5
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section
```

```
/- concordance: source L5191 EXERCISE 8.21 lists of a projection -/

namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap Example61 Proposition77 Exercise717
    Exercise510

variable {a b : Type*} {D : NeighborhoodSystem a} {E : NeighborhoodSystem b}

variable {hD : forall  X, D.mem X  ->  X.Nonempty}

variable {u : ApproximableMap D E} {v : ApproximableMap (prod E E) E}

/-- **The "inner" recursive extension of `a : U  ->  U` to `US  ->  US`**, satisfying `
    aS(xS) = (a x)S`
and `aS(<y,z>) = <aS(y), aS(z)>` by `gMap_in`/`gMap_pair` -- Scott's fixed-point
    construction for
`aS`, realized directly by Exercise 7.17 Part 2's catamorphism `gMap`. -/
noncomputable def aSharpInner (a : ApproximableMap U U) :
    ApproximableMap (Dsharp U U_mem_nonempty) (Dsharp U U_mem_nonempty) :=
  gMap ((inSharpMap U_mem_nonempty).comp a) (pairSharpMap U_mem_nonempty)
    U_mem_nonempty

section DsharpFunctor

variable {? ? ? : Type*} {D_0 : NeighborhoodSystem ?} {D_1 : NeighborhoodSystem ?}
  {D_2 : NeighborhoodSystem ?}

variable (hD_0 : forall  X, D_0.mem X  ->  X.Nonempty) (hD_1 : forall  X, D_1.mem X
    ->  X.Nonempty)
  {hD_2 : forall  X, D_2.mem X  ->  X.Nonempty}

/-- **The action of `D  |->  DS` on morphisms.** Recurses `f : D_1  ->  D_0` through
    the leaves via `gMap`,
leaving the pairing structure alone -- the general form of `aSharpInner` (which
    specializes this at
`D_0 = D_1 = U`, `f := a`). -/
noncomputable def dsharpMap (f : ApproximableMap D_1 D_0) :
    ApproximableMap (Dsharp D_1 hD_1) (Dsharp D_0 hD_0) :=
  gMap ((inSharpMap hD_0).comp f) (pairSharpMap hD_0) hD_1

theorem dsharpMap_eq (f : ApproximableMap D_1 D_0) :
    dsharpMap hD_0 hD_1 f = gMap ((inSharpMap hD_0).comp f) (pairSharpMap hD_0) hD_1
    := rfl

/-- **`aSharpInner` is `dsharpMap` at `D_0 = D_1 = U`, `f := a`.** -/
theorem dsharpMap_self_eq_aSharpInner (a : ApproximableMap U U) :
    dsharpMap U_mem_nonempty U_mem_nonempty a = aSharpInner a := rfl

end DsharpFunctor

/-- Forward direction: every `GRel (inSharpMap) (pairSharpMap)`-related pair is an `
    idMap`-related
pair (`MemS`-membership on both sides plus ` subseteq `). -/
theorem grel_inSharp_pairSharp_imp_idMap_rel :
    forall  {W Z}, GRel (inSharpMap hD) (pairSharpMap hD) W Z  ->  MemS D W  /\  MemS
```

```
    D Z  /\  W  subseteq  Z := by
  intro W Z h
  induction h with
  | gamma hZ => exact <MemS.gamma, hZ  |>  MemS.gamma, hZ  |>  subset_rfl>
  | leaf hrel => exact <MemS.zero hrel.1, hrel.2.1, hrel.2.2>
  | @node P Q Z_1 Z_2 Z hP hQ hvrel ihP ihQ =>
      obtain <hPmem, hZ1mem, hPZ1> := ihP
      obtain <hQmem, hZ2mem, hQZ2> := ihQ
      obtain <-, hZmem, A, B, hAmem, hBmem, heq, hsub> := hvrel
      obtain <rfl, rfl> := prodNbhd_injective heq
      exact <MemS.pair hPmem hQmem, hZmem, (embPair_subset.mpr <hPZ1, hQZ2>).trans
    hsub>

/-- **`iS : US  ->  U`** (Definition-8.9-style fixed computable projection pair for `US
    `). -/
noncomputable def iSharp : ApproximableMap (Dsharp U U_mem_nonempty) U :=
  (theorem_8_8_b_strong (dsharpPresentation UComputablePresentation U_mem_nonempty)).
    choose

/-- **`I : (D_a)S  ->  U`**, transporting `iSharp` through `dsharpMap`'s action on the
    subsystem
injection `(fixedNbhd_subsystem a).inj : D_a  ->  U`. -/
noncomputable def ISharpComb (a : ApproximableMap U U) :
    ApproximableMap (Dsharp (fixedNbhd a) (fixedNbhd_mem_nonempty a)) U :=
  iSharp.comp (dsharpMap U_mem_nonempty (fixedNbhd_mem_nonempty a) (
    fixedNbhd_subsystem a).inj)

/-- Backward direction: every `idMap`-related pair is `GRel (inSharpMap) (pairSharpMap)
    `-related,
by structural induction on `W`'s `MemS`-derivation (generalizing over `Z`). -/
theorem idMap_rel_imp_grel_inSharp_pairSharp :
    forall  {W}, MemS D W  ->  forall  {Z}, MemS D Z  ->  W  subseteq  Z  ->  GRel (
    inSharpMap hD) (pairSharpMap hD) W Z := by
  intro W hW
  induction hW with
  | gamma =>
      intro Z hZ hsub
      have hZeq : Z = Gamma D := Set.Subset.antisymm (memS_subset_gamma hZ) hsub
      subst hZeq
      exact GRel.gamma rfl
  | @zero X hX =>
      intro Z hZ hsub
      exact GRel.leaf <hX, hZ, hsub>
  | @pair P Q hP hQ ihP ihQ =>
      intro Z hZ hsub
      exact GRel.node (ihP hP subset_rfl) (ihQ hQ subset_rfl)
        <prod_mem_prodNbhd hP hQ, hZ, P, Q, hP, hQ, rfl, hsub>

/-- **`gMap in pair = I_{DS}`**: the catamorphism instantiated at `u := in`, `v := pair
    ` (the tree
algebra's own structure maps) is the identity on `DS`. -/
theorem gMap_inSharp_pairSharp_eq_idMap :
    gMap (inSharpMap hD) (pairSharpMap hD) hD = idMap (Dsharp D hD) := by
  apply le_antisymm
  * rw [ApproximableMap.le_iff]
    intro W Z h
```

```
    exact grel_inSharp_pairSharp_imp_idMap_rel h
  * rw [ApproximableMap.le_iff]
    intro W Z h
    exact idMap_rel_imp_grel_inSharp_pairSharp h.1 h.2.1 h.2.2

section DsharpFunctor

variable {? ? ? : Type*} {D_0 : NeighborhoodSystem ?} {D_1 : NeighborhoodSystem ?}
  {D_2 : NeighborhoodSystem ?}

variable (hD_0 : forall  X, D_0.mem X  ->  X.Nonempty) (hD_1 : forall  X, D_1.mem X
    ->  X.Nonempty)
  {hD_2 : forall  X, D_2.mem X  ->  X.Nonempty}

/-- **`idS = id`.** -/
theorem dsharpMap_id : dsharpMap hD_0 hD_0 (idMap D_0) = idMap (Dsharp D_0 hD_0) := by
  rw [dsharpMap_eq, comp_idMap]
  exact gMap_inSharp_pairSharp_eq_idMap

end DsharpFunctor

/-- **`x^S|_{X} = (0X)`**: `inSharp` sends the principal element `X` to the principal
    element of
the leaf neighbourhood `0X`. -/
theorem inSharp_principal {X : Set a} (hX : D.mem X) :
    inSharp D hD (D.principal hX) = (Dsharp D hD).principal (MemS.zero hX) := by
  rw [ <-  inSharpMap_toElementMap hD]
  apply Element.ext
  intro W
  simp only [mem_principal,  <-  rel_iff_mem_principal (inSharpMap hD) hX,
    inSharpMap_rel]
  exact <fun h => h.2, fun h => <hX, h>>

/-- **`jS : U  ->  US`**. -/
noncomputable def jSharp : ApproximableMap U (Dsharp U U_mem_nonempty) :=
  (theorem_8_8_b_strong (dsharpPresentation UComputablePresentation U_mem_nonempty)).
    choose_spec.choose

/-- **`J : U  ->  (D_a)S`**, symmetric to `ISharpComb`, via the subsystem projection
`(fixedNbhd_subsystem a).proj : U  ->  D_a`. -/
noncomputable def JSharpComb (a : ApproximableMap U U) :
    ApproximableMap U (Dsharp (fixedNbhd a) (fixedNbhd_mem_nonempty a)) :=
  (dsharpMap (fixedNbhd_mem_nonempty a) U_mem_nonempty (fixedNbhd_subsystem a).proj).
    comp jSharp

/-- **Scott's operator `aS : U  ->  U`** (Exercise 8.21), conjugating `aSharpInner a`
    through the
`US ? U` projection pair, exactly as Definition 8.9 builds `a+b`, `a x b`, `a -> b`. -/
noncomputable def aSharp (a : ApproximableMap U U) : ApproximableMap U U :=
  iSharp.comp ((aSharpInner a).comp jSharp)

namespace aSharp

theorem eq (a : ApproximableMap U U) :
    aSharp a = iSharp.comp ((aSharpInner a).comp jSharp) := rfl
```

```
end aSharp

theorem jSharp_comp_iSharp : jSharp.comp iSharp = idMap _ :=
  (theorem_8_8_b_strong (dsharpPresentation UComputablePresentation U_mem_nonempty)).
    choose_spec.choose_spec.1

/-- **`<P, Q> = (P U Q)`** (general, for any two neighbourhood systems): the pairing of
     two
principal elements is the principal element of the product neighbourhood. -/
theorem pair_principal_eq_principal_prodNbhd {? ? : Type*} {V_0 : NeighborhoodSystem
    ?}
    {V_1 : NeighborhoodSystem ?} {P : Set ?} {Q : Set ?} (hP : V_0.mem P) (hQ : V_1.
    mem Q) :
    pair (V_0.principal hP) (V_1.principal hQ)
      = (prod V_0 V_1).principal (prod_mem_prodNbhd hP hQ) := by
  apply Element.ext
  intro W
  simp only [mem_pair, mem_principal, prod_mem_iff]
  constructor
  * rintro <P', Q', <hP', hPP'>, <hQ', hQQ'>, rfl>
    exact <<P', Q', hP', hQ', rfl>, prodNbhd_subset_iff.mpr <hPP', hQQ'>>
  * rintro <<P', Q', hP', hQ', rfl>, hsub>
    obtain <hPP', hQQ'> := prodNbhd_subset_iff.mp hsub
    exact <P', Q', <hP', hPP'>, <hQ', hQQ'>, rfl>

/-- **`<P, Q>^S = (1P U 2Q)`**: `pairSharp` sends a pair of principal elements to the
    principal
element of the corresponding node neighbourhood. -/
theorem pairSharp_principal {P Q : Set (List Bool  x  a)} (hP : MemS D P) (hQ : MemS D
     Q) :
    pairSharp D hD ((Dsharp D hD).principal hP) ((Dsharp D hD).principal hQ)
      = (Dsharp D hD).principal (MemS.pair hP hQ) := by
  rw [ <-  pairSharpMap_toElementMap hD, pair_principal_eq_principal_prodNbhd hP hQ]
  apply Element.ext
  intro W
  rw [mem_principal,  <-  rel_iff_mem_principal (pairSharpMap hD) (prod_mem_prodNbhd
    hP hQ),
    pairSharpMap_rel]
  constructor
  * rintro <-, hW, A, B, hA, hB, heq, hsub>
    obtain <rfl, rfl> := prodNbhd_injective heq
    exact <hW, hsub>
  * rintro <hW, hsub>
    exact <prod_mem_prodNbhd hP hQ, hW, P, Q, hP, hQ, rfl, hsub>

/-- **Uniqueness of the catamorphism** (Exercise 8.21(b)(1), Scott's own remark "`g` is
     unique
because the values on finite elements are fixed"). Any *strict* `k : D^S  ->  E`
    satisfying `gMap`'s two
defining equations at the *element* level -- `k(x^S) = u(x)` and `k(<y,z>) = v(<k(y),k(
    z)>)` -- is
literally the catamorphism `gMap u v hD`. Proved by induction on the `MemS`-derivation
    of the input
neighbourhood, generalizing over the output: the `Gamma` case is strictness plus `
    master_rel`; the leaf
case reads the hypothesis off at the principal element `X` via `inSharp_principal` and
```

```
`rel_iff_mem_principal`; the node case does the same with `pairSharp_principal`, using
    the induction
hypotheses to identify the two recursive outputs. -/
theorem gMap_eq_of_satisfies {k : ApproximableMap (Dsharp D hD) E} (hk : IsStrict k)
    (hu : forall  x : D.Element, k.toElementMap (inSharp D hD x) = u.toElementMap x)
    (hv : forall  y z : (Dsharp D hD).Element,
      k.toElementMap (pairSharp D hD y z)
        = v.toElementMap (pair (k.toElementMap y) (k.toElementMap z))) :
    k = gMap u v hD := by
  have main : forall  {W}, MemS D W  ->  forall  Z, k.rel W Z  <->  GRel u v W Z := by
    intro W hW
    induction hW with
    | gamma =>
        intro Z
        constructor
        * intro h; exact GRel.gamma (hk h)
        * intro h
          have hZ : Z = E.master := gRel_gamma_inv hD h
          subst hZ
          exact k.master_rel
    | @zero X hX =>
        intro Z
        have key := hu (D.principal hX)
        rw [inSharp_principal hX] at key
        constructor
        * intro h
          have h1 := (rel_iff_mem_principal k (MemS.zero hX)).mp h
          rw [key] at h1
          exact GRel.leaf ((rel_iff_mem_principal u hX).mpr h1)
        * intro h
          have h1 := (rel_iff_mem_principal u hX).mp (gRel_embZero_inv hD h)
          rw [ <-  key] at h1
          exact (rel_iff_mem_principal k (MemS.zero hX)).mpr h1
    | @pair P Q hP hQ ihP ihQ =>
        intro Z
        have key := hv ((Dsharp D hD).principal hP) ((Dsharp D hD).principal hQ)
        rw [pairSharp_principal hP hQ] at key
        constructor
        * intro h
          have h1 := (rel_iff_mem_principal k (MemS.pair hP hQ)).mp h
          rw [key] at h1
          obtain <V, hVmem, hVrel> := h1
          obtain <Z1, Z2, hZ1, hZ2, rfl> := hVmem
          have hkZ1 : k.rel P Z1 := (rel_iff_mem_principal k hP).mpr hZ1
          have hkZ2 : k.rel Q Z2 := (rel_iff_mem_principal k hQ).mpr hZ2
          exact GRel.node ((ihP Z1).mp hkZ1) ((ihQ Z2).mp hkZ2) hVrel
        * intro h
          obtain <Z1, Z2, hg1, hg2, hvrel> := gRel_embPair_inv hD h
          have h1 := (rel_iff_mem_principal k hP).mp ((ihP Z1).mpr hg1)
          have h2 := (rel_iff_mem_principal k hQ).mp ((ihQ Z2).mpr hg2)
          have h3 : (v.toElementMap (pair (k.toElementMap ((Dsharp D hD).principal hP)
    )
              (k.toElementMap ((Dsharp D hD).principal hQ)))).mem Z :=
            <prodNbhd Z1 Z2, <Z1, Z2, h1, h2, rfl>, hvrel>
          rw [ <-  key] at h3
          exact (rel_iff_mem_principal k (MemS.pair hP hQ)).mpr h3
```

```
  apply ApproximableMap.ext
  intro W Z
  by_cases hW : MemS D W
  * exact main hW Z
  * exact <fun h => absurd (k.rel_dom h) hW, fun h => absurd (gRel_dom h) hW>

section DsharpFunctor

variable {? ? ? : Type*} {D_0 : NeighborhoodSystem ?} {D_1 : NeighborhoodSystem ?}
  {D_2 : NeighborhoodSystem ?}

variable (hD_0 : forall  X, D_0.mem X  ->  X.Nonempty) (hD_1 : forall  X, D_1.mem X
    ->  X.Nonempty)
  {hD_2 : forall  X, D_2.mem X  ->  X.Nonempty}

/-- **`(f'  comp  f)S = f'S  comp  fS`.** Applies `gMap_eq_of_satisfies` to `k := (
    dsharpMap f').comp
(dsharpMap f)`: both defining equations of `k` reduce, via `gMap_in`/`gMap_pair`
    applied twice, to
the corresponding equation for `dsharpMap (f'.comp f)` -- verbatim
`gMap_selfComp_eq_of_isRetraction`'s proof, generalized away from `f' = f` (so no
    retraction
hypothesis is needed here). -/
theorem dsharpMap_comp (f : ApproximableMap D_2 D_1) (f' : ApproximableMap D_1 D_0) :
    dsharpMap hD_0 hD_2 (f'.comp f) = (dsharpMap hD_0 hD_1 f').comp (dsharpMap hD_1
    hD_2 f) := by
  set g1 : ApproximableMap (Dsharp D_2 hD_2) (Dsharp D_1 hD_1) := dsharpMap hD_1 hD_2
    f with hg1_def
  set g2 : ApproximableMap (Dsharp D_1 hD_1) (Dsharp D_0 hD_0) := dsharpMap hD_0 hD_1
    f' with hg2_def
  symm
  show g2.comp g1 = dsharpMap hD_0 hD_2 (f'.comp f)
  rw [dsharpMap_eq]
  apply gMap_eq_of_satisfies
  * exact isStrict_comp (by rw [hg2_def, dsharpMap_eq]; exact gMap_strict hD_1)
      (by rw [hg1_def, dsharpMap_eq]; exact gMap_strict hD_2)
  * intro x
    have e1 : g1.toElementMap (inSharp D_2 hD_2 x) = inSharp D_1 hD_1 (f.toElementMap
    x) := by
      rw [hg1_def, dsharpMap_eq, gMap_in hD_2 x, toElementMap_comp,
    inSharpMap_toElementMap]
    have e2 : g2.toElementMap (inSharp D_1 hD_1 (f.toElementMap x))
        = inSharp D_0 hD_0 (f'.toElementMap (f.toElementMap x)) := by
      rw [hg2_def, dsharpMap_eq, gMap_in hD_1 (f.toElementMap x), toElementMap_comp,
        inSharpMap_toElementMap]
    calc (g2.comp g1).toElementMap (inSharp D_2 hD_2 x)
        = g2.toElementMap (g1.toElementMap (inSharp D_2 hD_2 x)) := toElementMap_comp
    _ = _
      _ = g2.toElementMap (inSharp D_1 hD_1 (f.toElementMap x)) := by rw [e1]
      _ = inSharp D_0 hD_0 (f'.toElementMap (f.toElementMap x)) := e2
      _ = ((inSharpMap hD_0).comp (f'.comp f)).toElementMap x := by
          rw [toElementMap_comp, toElementMap_comp, inSharpMap_toElementMap]
  * intro y z
    have e1 : g1.toElementMap (pairSharp D_2 hD_2 y z)
        = pairSharp D_1 hD_1 (g1.toElementMap y) (g1.toElementMap z) := by
      rw [hg1_def, dsharpMap_eq, gMap_pair hD_2 y z, pairSharpMap_toElementMap]
```

```
    have e2 : g2.toElementMap (pairSharp D_1 hD_1 (g1.toElementMap y) (g1.toElementMap
     z))
        = pairSharp D_0 hD_0 (g2.toElementMap (g1.toElementMap y))
          (g2.toElementMap (g1.toElementMap z)) := by
      rw [hg2_def, dsharpMap_eq, gMap_pair hD_1 (g1.toElementMap y) (g1.toElementMap z
    ),
        pairSharpMap_toElementMap]
    calc (g2.comp g1).toElementMap (pairSharp D_2 hD_2 y z)
        = g2.toElementMap (g1.toElementMap (pairSharp D_2 hD_2 y z)) :=
    toElementMap_comp _ _ _
      _ = g2.toElementMap (pairSharp D_1 hD_1 (g1.toElementMap y) (g1.toElementMap z))
     := by
          rw [e1]
      _ = pairSharp D_0 hD_0 (g2.toElementMap (g1.toElementMap y))
          (g2.toElementMap (g1.toElementMap z)) := e2
      _ = (pairSharpMap hD_0).toElementMap
          (pair (g2.toElementMap (g1.toElementMap y)) (g2.toElementMap (g1.
    toElementMap z))) :=
        (pairSharpMap_toElementMap hD_0 _ _).symm
      _ = (pairSharpMap hD_0).toElementMap
          (pair ((g2.comp g1).toElementMap y) ((g2.comp g1).toElementMap z)) := by
        rw [toElementMap_comp, toElementMap_comp]

end DsharpFunctor

/-- **`I  comp  J = aS`**, when `a` is a finitary projection. -/
theorem ISharpComb_comp_JSharpComb {a : ApproximableMap U U} (ha : FinitaryProjection
    a) :
    (ISharpComb a).comp (JSharpComb a) = aSharp a := by
  unfold ISharpComb JSharpComb
  rw [comp_assoc,  <-  comp_assoc (dsharpMap _ _ _) (dsharpMap _ _ _) jSharp,  <-
    dsharpMap_comp,
    inj_comp_proj_eq_self ha, dsharpMap_self_eq_aSharpInner]
  exact (aSharp.eq a).symm

/-- **`J  comp  I = I_{(D_a)S}`**, unconditionally (no finitary/projection hypothesis
    on `a` needed). -/
theorem JSharpComb_comp_ISharpComb (a : ApproximableMap U U) :
    (JSharpComb a).comp (ISharpComb a) =
      idMap (Dsharp (fixedNbhd a) (fixedNbhd_mem_nonempty a)) := by
  unfold JSharpComb ISharpComb
  rw [comp_assoc,  <-  comp_assoc jSharp iSharp (dsharpMap _ _ _), jSharp_comp_iSharp,
    idMap_comp,
    <-  dsharpMap_comp, (fixedNbhd_subsystem a).proj_comp_inj, dsharpMap_id]

/-- **Exercise 8.21(c) (Scott 1981, PRG-19).** For a finitary projection `a` of `U`,
    the tree
algebra `(D_a)S` of `D_a := fixedNbhd a` is order-isomorphic to `aS`'s own fixed-point
    domain --
Scott's headline isomorphism `D_{aS}  ~=  (D_a)S`. -/
noncomputable def dsharp_fixedNbhd_elementIso {a : ApproximableMap U U}
    (ha : FinitaryProjection a) :
    (Dsharp (fixedNbhd a) (fixedNbhd_mem_nonempty a)).Element ?o
      {y : U.Element // (aSharp a).toElementMap y = y} :=
  elementIsoOfProjectionPair (ISharpComb a) (JSharpComb a) (JSharpComb_comp_ISharpComb
    a)
```

```
    (ISharpComb_comp_JSharpComb ha).symm

/-- **Exercise 8.21(b)(2), general form.** If `a : D  ->  D` is a retraction, so is the
     recursive
extension `gMap ((inSharpMap hD).comp a) (pairSharpMap hD) hD` of `a`'s tree algebra (`
    aSharpInner a`
specializes this at `D := U`). Applies `gMap_eq_of_satisfies` to `k := g.comp g` (`g`
    the catamorphism
itself): both defining equations of `k` reduce, via `gMap_in`/`gMap_pair` applied twice
    , to the
corresponding equation for `g` composed with `a`'s own idempotence. -/
theorem gMap_selfComp_eq_of_isRetraction {a : ApproximableMap D D} (ha : Retraction a)
     :
    (gMap ((inSharpMap hD).comp a) (pairSharpMap hD) hD).comp
        (gMap ((inSharpMap hD).comp a) (pairSharpMap hD) hD)
      = gMap ((inSharpMap hD).comp a) (pairSharpMap hD) hD := by
  set u : ApproximableMap D (Dsharp D hD) := (inSharpMap hD).comp a with hu_def
  set v : ApproximableMap (prod (Dsharp D hD) (Dsharp D hD)) (Dsharp D hD) :=
    pairSharpMap hD with hv_def
  set g : ApproximableMap (Dsharp D hD) (Dsharp D hD) := gMap u v hD with hg_def
  have hu_eq : forall  y : D.Element, u.toElementMap y = inSharp D hD (a.toElementMap
    y) := by
    intro y; rw [hu_def, toElementMap_comp, inSharpMap_toElementMap]
  show g.comp g = g
  apply gMap_eq_of_satisfies
  * rw [hg_def]; exact isStrict_comp (gMap_strict hD) (gMap_strict hD)
  * intro x
    have e1 : g.toElementMap (inSharp D hD x) = u.toElementMap x := by
      rw [hg_def]; exact gMap_in hD x
    have e2 : g.toElementMap (u.toElementMap x) = u.toElementMap (a.toElementMap x) :=
     by
      rw [hu_eq x, hg_def]; exact gMap_in hD (a.toElementMap x)
    calc (g.comp g).toElementMap (inSharp D hD x)
        = g.toElementMap (g.toElementMap (inSharp D hD x)) := toElementMap_comp _ _ _
      _ = g.toElementMap (u.toElementMap x) := by rw [e1]
      _ = u.toElementMap (a.toElementMap x) := e2
      _ = inSharp D hD (a.toElementMap (a.toElementMap x)) := hu_eq _
      _ = inSharp D hD (a.toElementMap x) := by rw [toElementMap_idem_of_isRetraction
    ha]
      _ = u.toElementMap x := (hu_eq x).symm
  * intro y z
    have e1 : g.toElementMap (pairSharp D hD y z)
        = v.toElementMap (pair (g.toElementMap y) (g.toElementMap z)) := by
      rw [hg_def]; exact gMap_pair hD y z
    have e2 : v.toElementMap (pair (g.toElementMap y) (g.toElementMap z))
        = pairSharp D hD (g.toElementMap y) (g.toElementMap z) := by
      rw [hv_def]; exact pairSharpMap_toElementMap hD _ _
    have e3 : g.toElementMap (pairSharp D hD (g.toElementMap y) (g.toElementMap z))
        = v.toElementMap
            (pair (g.toElementMap (g.toElementMap y)) (g.toElementMap (g.toElementMap
    z))) := by
      rw [hg_def]; exact gMap_pair hD _ _
    calc (g.comp g).toElementMap (pairSharp D hD y z)
        = g.toElementMap (g.toElementMap (pairSharp D hD y z)) := toElementMap_comp _
    _ _
      _ = g.toElementMap (v.toElementMap (pair (g.toElementMap y) (g.toElementMap z)))
```

```
     := by
          rw [e1]
      _ = g.toElementMap (pairSharp D hD (g.toElementMap y) (g.toElementMap z)) := by
    rw [e2]
      _ = v.toElementMap
            (pair (g.toElementMap (g.toElementMap y)) (g.toElementMap (g.toElementMap
    z))) := e3
      _ = v.toElementMap (pair ((g.comp g).toElementMap y) ((g.comp g).toElementMap z)
    ) := by
          rw [toElementMap_comp, toElementMap_comp]

/-- **`GRel` is monotone in `u`, `v`.** Direct structural induction on the `GRel`
    derivation,


parallel to `gRel_mono`'s induction on shape. -/
theorem GRel.mono_uv (_hD : forall  X, D.mem X  ->  X.Nonempty)
    {u u' : ApproximableMap D E} {v v' : ApproximableMap (prod E E) E}
    (hu : u  <=  u') (hv : v  <=  v') :
    forall  {W Z}, GRel u v W Z  ->  GRel u' v' W Z := by
  intro W Z h
  induction h with
  | gamma hZ => exact GRel.gamma hZ
  | leaf hrel => exact GRel.leaf (hu _ _ hrel)
  | node hP hQ hvrel ihP ihQ => exact GRel.node ihP ihQ (hv _ _ hvrel)

theorem aSharpInner_eq (a : ApproximableMap U U) :
    aSharpInner a = gMap ((inSharpMap U_mem_nonempty).comp a) (pairSharpMap
    U_mem_nonempty)
      U_mem_nonempty := rfl

/-- **Exercise 8.21(b)(2).** If `a` is a retraction (`a comp a = a`), so is its
    recursive extension
`aSharpInner a`: the specialization of `gMap_selfComp_eq_of_isRetraction` to `D := U`.
    -/
theorem aSharpInner_idem_of_isRetraction {a : ApproximableMap U U} (ha : Retraction a)
     :
    Retraction (aSharpInner a) := by
  constructor
  show (aSharpInner a).comp (aSharpInner a) = aSharpInner a
  rw [aSharpInner_eq]
  exact gMap_selfComp_eq_of_isRetraction ha

/-- **`gMap` is monotone in `u`, `v`.** -/
theorem gMap_mono (hD : forall  X, D.mem X  ->  X.Nonempty)
    {u u' : ApproximableMap D E} {v v' : ApproximableMap (prod E E) E}
    (hu : u  <=  u') (hv : v  <=  v') : gMap u v hD  <=  gMap u' v' hD := by
  rw [ApproximableMap.le_iff]
  intro W Z h
  exact GRel.mono_uv hD hu hv h

theorem iSharp_comp_jSharp_le : iSharp.comp jSharp  <=  idMap U :=
  (theorem_8_8_b_strong (dsharpPresentation UComputablePresentation U_mem_nonempty)).
    choose_spec.choose_spec.2.1

/-- **`US ? U`.** -/
theorem dsharpU_trianglelefteq_U : Dsharp U U_mem_nonempty ? U :=
```

```
  trianglelefteq_of_projectionPair iSharp jSharp jSharp_comp_iSharp
    iSharp_comp_jSharp_le

theorem iSharp_isComputableMap :
    ComputableMap (dsharpPresentation UComputablePresentation U_mem_nonempty)
      UComputablePresentation iSharp :=
  (theorem_8_8_b_strong (dsharpPresentation UComputablePresentation U_mem_nonempty)).
    choose_spec.choose_spec.2.2.1

theorem jSharp_isComputableMap :
    ComputableMap UComputablePresentation
      (dsharpPresentation UComputablePresentation U_mem_nonempty) jSharp :=
  (theorem_8_8_b_strong (dsharpPresentation UComputablePresentation U_mem_nonempty)).
    choose_spec.choose_spec.2.2.2

end DomainTheory.NeighbourhoodSystems
```

— `aSharp` projection/computability companions and Ex 8.25 closing

ASharp_more.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.ASharp

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap Example61 Proposition77 Exercise717
    Exercise510

variable {a b : Type*} {D : NeighborhoodSystem a} {E : NeighborhoodSystem b}
variable {hD : forall  X, D.mem X  ->  X.Nonempty}
variable {u : ApproximableMap D E} {v : ApproximableMap (prod E E) E}

namespace aSharp

/-- **`a  <=  I ? aS  <=  I`** (half of "`a` a projection ? `aS` a projection",
    Proposition 8.10's
proof recipe transcribed to `S`). -/
theorem le_idMap_of_le {a : ApproximableMap U U} (ha : a  <=  idMap U) :
    aSharp a  <=  idMap U := by
  have h1 : (inSharpMap U_mem_nonempty).comp a  <=  inSharpMap U_mem_nonempty := by
    calc (inSharpMap U_mem_nonempty).comp a
         <=  (inSharpMap U_mem_nonempty).comp (idMap U) := comp_mono_gen le_rfl ha
      _ = inSharpMap U_mem_nonempty := comp_idMap _
  have h2 : aSharpInner a  <=  idMap (Dsharp U U_mem_nonempty) := by
    calc aSharpInner a
         <=  gMap (inSharpMap U_mem_nonempty) (pairSharpMap U_mem_nonempty)
    U_mem_nonempty :=
          gMap_mono U_mem_nonempty h1 le_rfl
      _ = idMap (Dsharp U U_mem_nonempty) := gMap_inSharp_pairSharp_eq_idMap
  calc aSharp a = iSharp.comp ((aSharpInner a).comp jSharp) := eq a
    _  <=  iSharp.comp ((idMap (Dsharp U U_mem_nonempty)).comp jSharp) :=
        comp_mono_gen le_rfl (comp_mono_gen h2 le_rfl)
    _ = iSharp.comp jSharp := by rw [idMap_comp]
```

```
    _  <=  idMap U := iSharp_comp_jSharp_le

/-- **`aS` is finitary whenever `a` is a finitary projection.** -/
theorem isFinitary {a : ApproximableMap U U} (ha : FinitaryProjection a) :
    IsFinitary (aSharp a) :=
  isFinitary_of_projectionPair (ISharpComb a) (JSharpComb a) (
    JSharpComb_comp_ISharpComb a)
    (ISharpComb_comp_JSharpComb ha).symm

/-- **`aS` is computable whenever `a` is.** -/
theorem isComputable {a : ApproximableMap U U}
    (ha : ComputableMap UComputablePresentation UComputablePresentation a) :
    ComputableMap UComputablePresentation UComputablePresentation (aSharp a) := by
  have hu : ComputableMap UComputablePresentation
      (dsharpPresentation UComputablePresentation U_mem_nonempty)
      ((inSharpMap U_mem_nonempty).comp a) :=
    comp_isComputable ha (inSharp_isComputable U_mem_nonempty UComputablePresentation)
  have hInner : ComputableMap (dsharpPresentation UComputablePresentation
    U_mem_nonempty)
      (dsharpPresentation UComputablePresentation U_mem_nonempty) (aSharpInner a) :=
    gMap_isComputable U_mem_nonempty UComputablePresentation
      (dsharpPresentation UComputablePresentation U_mem_nonempty) hu
      (pairSharp_isComputable U_mem_nonempty UComputablePresentation)
  exact comp_isComputable (comp_isComputable jSharp_isComputableMap hInner)
    iSharp_isComputableMap

end aSharp

/-- **`aS`, unfolded one step through the `US ? U` projection pair.** Mirrors
`Proposition810.lean`'s `toElementMap_prodComb`/`toElementMap_arrowComb` pattern. -/
theorem toElementMap_aSharp (a : ApproximableMap U U) (x : U.Element) :
    (aSharp a).toElementMap x
      = iSharp.toElementMap ((aSharpInner a).toElementMap (jSharp.toElementMap x)) :=
    by
  rw [aSharp.eq]
  simp only [toElementMap_comp]

/-- **Exercise 8.21(b)(3).** Idempotence of `aSharpInner a` transports through the
    projection pair
`iSharp`/`jSharp` to idempotence of `aSharp a` itself -- exactly `Proposition810.lean`'
    s
`isRetraction_prodComb`/`isRetraction_arrowComb` recipe (`toElementMap_of_comp_eq_idMap
    ` +
`toElementMap_idem_of_isRetraction`), conjugated through `jSharp_comp_iSharp` instead
    of
`jTimes_comp_iTimes`/`jArrow_comp_iArrow`. -/
theorem isRetraction_aSharp {a : ApproximableMap U U} (ha2 : Retraction (aSharpInner a
    )) :
    Retraction (aSharp a) := by
  constructor
  apply ext_of_toElementMap
  intro x
  rw [toElementMap_comp, toElementMap_aSharp a ((aSharp a).toElementMap x),
    toElementMap_aSharp a x, toElementMap_of_comp_eq_idMap jSharp_comp_iSharp,
    toElementMap_idem_of_isRetraction ha2]
```

```
/-- **Exercise 8.21(b), assembled.** `a` a projection of `U` `?` `aS` a projection: the
     ` <=  I` half
is 8.21(a)'s `aSharp.le_idMap_of_le`; the retraction half chains `
    aSharpInner_idem_of_isRetraction`
(the `gMap`-level idempotence, via the catamorphism-uniqueness lemma `
    gMap_eq_of_satisfies`) with
`isRetraction_aSharp` (transport through `US ? U`). This completes "`a` a projection ?
    `aS` a
projection" in full, matching `Proposition810.lean`'s `isProjection_sumComb`/`
    isProjection_prodComb`/
`isProjection_arrowComb`. -/
theorem isProjection_aSharp {a : ApproximableMap U U} (ha : IsProjection a) :
    IsProjection (aSharp a) :=
  <isRetraction_aSharp (aSharpInner_idem_of_isRetraction ha.1), aSharp.le_idMap_of_le
    ha.2>

/-- **Exercise 8.21, in full (Scott 1981, PRG-19).** `a` a finitary projection of `U`
    `?` `aS` is
a finitary projection of `U`, with `D_{aS}  ~=  (D_a)S`. -/
theorem isFinitaryProjection_aSharp {a : ApproximableMap U U} (ha : FinitaryProjection
     a) :
    FinitaryProjection (aSharp a) :=
  <<isProjection_aSharp ha.out.1, aSharp.isFinitary ha>>

/- Ex 8.25 closing / funSpace congruence -/
variable {? a' b' : Type*}
variable {V_0 : NeighborhoodSystem a} {V_1 : NeighborhoodSystem b}
variable {V_0' : NeighborhoodSystem a'} {V_1' : NeighborhoodSystem b'}
variable {V : NeighborhoodSystem b}

theorem ofIso_comp_ofIso_symm (e : V_0.Element ?o V_0'.Element) :
    (ofIso e).comp (ofIso e.symm) = idMap V_0' := by
  apply ext_of_toElementMap
  intro x
  rw [toElementMap_comp, toElementMap_ofIso, toElementMap_ofIso, toElementMap_idMap,
    OrderIso.apply_symm_apply]

theorem funSpaceCongr_hij (e0 : V_0.Element ?o V_0'.Element) (e1 : V_1.Element ?o V_1
    '.Element) :
    (expMap (ofIso e0.symm) (ofIso e1)).comp (expMap (ofIso e0) (ofIso e1.symm)) =
      idMap (funSpace V_0' V_1') := by
  rw [ <-  expMap.comp, ofIso_comp_ofIso_symm, ofIso_comp_ofIso_symm, expMap.id]

theorem ofIso_symm_comp_ofIso (e : V_0.Element ?o V_0'.Element) :
    (ofIso e.symm).comp (ofIso e) = idMap V_0 := by
  apply ext_of_toElementMap
  intro x
  rw [toElementMap_comp, toElementMap_ofIso, toElementMap_ofIso, toElementMap_idMap,
    OrderIso.symm_apply_apply]

theorem funSpaceCongr_hji (e0 : V_0.Element ?o V_0'.Element) (e1 : V_1.Element ?o V_1
    '.Element) :
    (expMap (ofIso e0) (ofIso e1.symm)).comp (expMap (ofIso e0.symm) (ofIso e1)) =
      idMap (funSpace V_0 V_1) := by
  rw [ <-  expMap.comp, ofIso_symm_comp_ofIso, ofIso_symm_comp_ofIso, expMap.id]
```

```
/-- **Two approximable maps that are *exact* (not merely ` <= `) mutual inverses give
    an
order-isomorphism of element types.** The generic upgrade behind `
    elementIsoOfProjectionPair`
(`Proposition810b.lean`) when the projection-pair equation holds in *both* directions:
    then the
fixed-point subtype on the codomain side is *everything*, collapsing to a genuine
    bijection. -/
def orderIsoOfMutualInverse {D : NeighborhoodSystem a} {E : NeighborhoodSystem b}
    (i : ApproximableMap D E) (j : ApproximableMap E D)
    (hji : j.comp i = idMap D) (hij : i.comp j = idMap E) : D.Element ?o E.Element
    where
  toFun := i.toElementMap
  invFun := j.toElementMap
  left_inv x := by rw [ <-  toElementMap_comp, hji, toElementMap_idMap]
  right_inv y := by rw [ <-  toElementMap_comp, hij, toElementMap_idMap]
  map_rel_iff' := by
    intro x x'
    show i.toElementMap x  <=  i.toElementMap x'  <->  x  <=  x'
    constructor
    * intro h
      have hm := j.toElementMap_mono h
      rwa [ <-  toElementMap_comp,  <-  toElementMap_comp, hji, toElementMap_idMap,
        toElementMap_idMap] at hm
    * intro h
      exact i.toElementMap_mono h

/-- **`funSpace` is a congruence for `?o`.** `?_0 ?o ?_0'`, `?_1 ?o ?_1'` `?`
`(?_0  ->  ?_1) ?o (?_0'  ->  ?_1')`. -/
noncomputable def funSpaceCongrOrderIso (e0 : V_0.Element ?o V_0'.Element)
    (e1 : V_1.Element ?o V_1'.Element) :
    (funSpace V_0 V_1).Element ?o (funSpace V_0' V_1').Element :=
  orderIsoOfMutualInverse (expMap (ofIso e0.symm) (ofIso e1)) (expMap (ofIso e0) (
    ofIso e1.symm))
    (funSpaceCongr_hji e0 e1) (funSpaceCongr_hij e0 e1)

/-- **Currying (Theorem 3.12), as a domain isomorphism.** `(?_0  x  ?_1  ->  ?_2)  ~=
    (?_0  ->  (?_1  ->  ?_2))`. -/
theorem curry_isomorphic (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b)
    (V_2 : NeighborhoodSystem ?) :
    funSpace (prod V_0 V_1) V_2  ~= ? funSpace V_0 (funSpace V_1 V_2) :=
  <curryIso V_0 V_1 V_2>

/-- **`funSpace` is a congruence for ` ~= ?`.** `?_0  ~= ? ?_0'`, `?_1  ~= ? ?_1'` `?`
`(?_0  ->  ?_1)  ~= ? (?_0'  ->  ?_1')`. -/
theorem funSpace_congr (h0 : V_0  ~= ? V_0') (h1 : V_1  ~= ? V_1') :
    funSpace V_0 V_1  ~= ? funSpace V_0' V_1' :=
  h0.elim fun e0 => h1.elim fun e1 => <funSpaceCongrOrderIso e0 e1>

/-- **`funSpace` congruence, domain slot.** `?_0  ~= ? ?_0'` `?` `(?_0  ->  ?_1)  ~= ?
    (?_0'  ->  ?_1)`. -/
theorem funSpace_congr_left (h0 : V_0  ~= ? V_0') : funSpace V_0 V_1  ~= ? funSpace
    V_0' V_1 :=
  funSpace_congr h0 (Isomorphic.refl V_1)

/-- **`funSpace` congruence, codomain slot.** `?_1  ~= ? ?_1'` `?` `(?_0  ->  ?_1)  ~=
```

```
    ? (?_0  ->  ?_1')`. -/
theorem funSpace_congr_right (h1 : V_1  ~= ? V_1') : funSpace V_0 V_1  ~= ? funSpace
    V_0 V_1' :=
  funSpace_congr (Isomorphic.refl V_0) h1

/-- **Closing argument, step 1.** `D  ~=  D  ->  V` and `V  x  V  ~=  V` imply `D  x  D
      ~=  D`. -/
theorem prod_self_isomorphic (hDV : D  ~= ? funSpace D V) (hVV : prod V V  ~= ? V) :
    prod D D  ~= ? D :=
  ((hDV.prod hDV).trans (funProd_isomorphic).symm).trans
    ((funSpace_congr_right hVV).trans hDV.symm)

/-- **Closing argument, step 2 (Exercise 8.25's main equation).** `D  ~=  D  ->  V` and
     `V  x  V  ~=  V`
imply `D  ~=  D  ->  D`. -/
theorem funSpace_self_isomorphic (hDV : D  ~= ? funSpace D V) (hVV : prod V V  ~= ? V)
     :
    D  ~= ? funSpace D D :=
  have hDD : prod D D  ~= ? D := prod_self_isomorphic hDV hVV
  have step : funSpace D D  ~= ? D :=
    ((funSpace_congr_right hDV).trans (curry_isomorphic D D V).symm).trans
      ((funSpace_congr_left hDD).trans hDV.symm)
  step.symm

end DomainTheory.NeighbourhoodSystems
```

The tip then assembles those tools into Scott's statement.

— `exercise_8_21`

`Exercise_8_21.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.ASharp
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    ASharp_more

@[expose] public section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap

/-- **Exercise 8.21 (Scott 1981, PRG-19).** The sharp construction yields a finitary
    projection. -/
theorem exercise_8_21 {a : ApproximableMap U U} (ha : FinitaryProjection a) :
    FinitaryProjection (aSharp a) :=
  isFinitaryProjection_aSharp ha

end
```

3.9.24 Exercise 8.22 — comparing B and C

Which of the two relations hold:

$$B \trianglelefteq C \quad \text{or} \quad C \trianglelefteq B\,?$$

Or do they both hold? In general if we use domain equations

$$D = T(D) + S(D)\,, \quad \text{and}$$

$$E = T(E)\,,$$

will $E \trianglelefteq D$ hold? What projections do you see in the examples in 6.2?

Tip summary The tip asserts the concrete embedding relation between Scott's examples `C` and `B`.

Development Compares Scott's concrete examples `B` and `C`.

ExampleC_le_B.lean

```
module

public import DomainTheory.NeighbourhoodSystems.DomainEquations.Exercise_6_17
public import DomainTheory.NeighbourhoodSystems.DomainEquations.Proposition_6_12
public import DomainTheory.NeighbourhoodSystems.DomainEquations.Exercise_6_29
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_8
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_13
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Lemma_3_6

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

/- concordance: source L5195 EXERCISE 8.22 comparing B and C -/
namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap ExampleB Example44 Example62

namespace Exercise822

/-- `C` is positive: every `C`-neighbourhood (a cone or a singleton) is nonempty. -/
theorem C_nonempty : forall  X, C.mem X  ->  X.Nonempty := by
  rintro X (<sigma, rfl> | <sigma, rfl>)
  * exact <sigma, List.prefix_rfl>
  * exact <sigma, rfl>

/-- Doubles every bit of a string: `enc [] = []`, `enc (b :: sigma) = b :: b :: enc
    sigma`. -/
def enc : Str  ->  Str
  | [] => []
  | b :: sigma => b :: b :: enc sigma

/-- The "completed" encoding: `enc sigma` followed by the two-bit marker `[true, false
    ]`, which can
never arise as a doubled block `[b, b]`. -/
def encC (sigma : Str) : Str := enc sigma ++ [true, false]

@[simp] theorem enc_nil : enc ([] : Str) = [] := rfl
```

```
@[simp] theorem enc_cons (b : Bool) (sigma : Str) : enc (b :: sigma) = b :: b :: enc
    sigma := rfl

theorem encC_cons (b : Bool) (sigma : Str) : encC (b :: sigma) = b :: b :: encC sigma
    := by
  simp [encC]

theorem encC_nil : encC ([] : Str) = [true, false] := rfl

theorem enc_append (sigma tau : Str) : enc (sigma ++ tau) = enc sigma ++ enc tau := by
  induction sigma with
  | nil => simp
  | cons b sigma' ih => simp [ih]

theorem enc_length (sigma : Str) : (enc sigma).length = 2 * sigma.length := by
  induction sigma with
  | nil => simp
  | cons b sigma' ih => simp [ih]; ring

theorem encC_length (sigma : Str) : (encC sigma).length = 2 * sigma.length + 2 := by
  simp [encC, enc_length]

/-- **Two completions relate only to themselves.** `encC sigma <+: encC tau  <->  sigma
     = tau`: distinct completed
positions of `C` become disjoint (never nested) `B`-cones. -/
theorem encC_prefix_encC_iff {sigma tau : Str} : encC sigma <+: encC tau  <->  sigma =
     tau := by
  constructor
  * induction sigma generalizing tau with
    | nil =>
      intro h
      cases tau with
      | nil => rfl
      | cons c tau' =>
        exfalso
        rw [encC_nil, encC_cons] at h
        obtain <rfl, h2> := List.cons_prefix_cons.mp h
        obtain <h3, -> := List.cons_prefix_cons.mp h2
        exact absurd h3 (by decide)
    | cons b sigma' ih =>
      intro h
      cases tau with
      | nil =>
        exfalso
        rw [encC_cons, encC_nil] at h
        have hlen := h.length_le
        simp only [List.length_cons, List.length_nil, encC_length] at hlen
        omega
      | cons c tau' =>
        rw [encC_cons, encC_cons] at h
        obtain <rfl, h2> := List.cons_prefix_cons.mp h
        obtain <-, h3> := List.cons_prefix_cons.mp h2
        rw [ih h3]
  * rintro rfl; exact List.prefix_rfl

theorem encC_injective {sigma tau : Str} (h : encC sigma = encC tau) : sigma = tau :=
```

```
  encC_prefix_encC_iff.mp (h  |>  List.prefix_rfl)

theorem enc_prefix_of_prefix {sigma tau : Str} (h : sigma <+: tau) : enc sigma <+: enc
    tau := by
  obtain <t, rfl> := h
  rw [enc_append]
  exact <enc t, rfl>

/-- **The marker never collides with a real prefix.** `enc ? <+: encC sigma` exactly
   reproduces `C`'s
own "cone  subseteq  completed" relation `? <+: sigma`: the extra length used by the
   marker can never be
matched by a genuine (doubled) continuation, since `[b, b]  !=  [true, false]`. -/
theorem enc_prefix_encC_iff {? sigma : Str} : enc ? <+: encC sigma  <->  ? <+: sigma
   := by
  constructor
  * induction ? generalizing sigma with
    | nil => intro _; exact List.nil_prefix
    | cons b ?' ih =>
      intro h
      cases sigma with
      | nil =>
        exfalso
        rw [enc_cons, encC_nil] at h
        obtain <rfl, h2> := List.cons_prefix_cons.mp h
        obtain <h3, -> := List.cons_prefix_cons.mp h2
        exact absurd h3 (by decide)
      | cons c sigma' =>
        rw [enc_cons, encC_cons] at h
        obtain <rfl, h2> := List.cons_prefix_cons.mp h
        obtain <-, h3> := List.cons_prefix_cons.mp h2
        rw [List.cons_prefix_cons]
        exact <rfl, ih h3>
  * intro h
    calc enc ? <+: enc sigma := enc_prefix_of_prefix h
      _ <+: encC sigma := <[true, false], rfl>

/-- **`enc` is prefix-reflecting.** Doubling never merges or confuses distinct prefixes
   : a prefix
relation between doubled strings comes from a prefix relation between the originals.
   -/
theorem enc_prefix_iff {sigma tau : Str} : enc sigma <+: enc tau  <->  sigma <+: tau
   := by
  refine <?_, enc_prefix_of_prefix>
  induction sigma generalizing tau with
  | nil => intro _; exact List.nil_prefix
  | cons b sigma' ih =>
    cases tau with
    | nil =>
      intro h
      rw [enc_cons, enc_nil] at h
      obtain <t, ht> := h
      simp at ht
    | cons c tau' =>
      intro h
      rw [enc_cons, enc_cons, List.cons_prefix_cons, List.cons_prefix_cons] at h
```

```
      obtain <rfl, -, h3> := h
      rw [List.cons_prefix_cons]
      exact <rfl, ih h3>

theorem enc_injective {sigma tau : Str} (h : enc sigma = enc tau) : sigma = tau := by
  have h1 : sigma <+: tau := enc_prefix_iff.mp <[], by simpa using h>
  have h2 : tau <+: sigma := enc_prefix_iff.mp <[], by simpa using h.symm>
  exact h1.eq_of_length (h1.length_le.antisymm h2.length_le)

/-- The image family: `B`-cones re-encoding `C`'s cones (`enc`) and completions (`encC
    `). -/
def memD' (X : Set Str) : Prop := (exists  sigma, X = cone (enc sigma))  \/  (exists
    sigma, X = cone (encC sigma))

/-- Every `D'`-set is literally a `B`-cone, so nestedness-or-disjointness is free from
    `B`'s own
`cone_trichotomy`. -/
theorem D'_nestedOrDisjoint : NestedOrDisjoint memD' := by
  rintro X Y (<sigma, rfl> | <sigma, rfl>) (<tau, rfl> | <tau, rfl>)
  * exact cone_trichotomy (enc sigma) (enc tau)
  * exact cone_trichotomy (enc sigma) (encC tau)
  * exact cone_trichotomy (encC sigma) (enc tau)
  * exact cone_trichotomy (encC sigma) (encC tau)

/-- **The image neighbourhood system `D'`.** -/
def D' : NeighborhoodSystem Str :=
  NeighborhoodSystem.ofNestedOrDisjoint memD' Set.univ <[], Set.mem_univ _>
    (Or.inl <[], by rw [enc_nil, cone_nil]>)
    D'_nestedOrDisjoint (by rintro X (<sigma, rfl> | <sigma, rfl>) <;> exact Set.
    subset_univ _)

@[simp] theorem D'_mem {X : Set Str} : D'.mem X  <->  memD' X := Iff.rfl

@[simp] theorem D'_master : D'.master = (Set.univ : Set Str) := rfl

/-- **`D' ? B`.** Same master; `D'.mem  subseteq  B.mem` since every `D'`-set is a `B`-
    cone; and
consistency is inherited via `D'_nestedOrDisjoint` exactly as in `B_subsystem_C`. -/
theorem D'_subsystem_B : D' ? B where
  master_eq := rfl
  sub := by rintro X (<sigma, rfl> | <sigma, rfl>) <;> exact <_, rfl>
  inter_closed := by
    rintro X Y hX hY hBXY
    rcases D'_nestedOrDisjoint hX hY with h | h | h
    * rw [Set.inter_eq_left.mpr h]; exact hX
    * rw [Set.inter_eq_right.mpr h]; exact hY
    * rw [h] at hBXY; exact absurd (B_nonempty {} hBXY) (by simp)

theorem cone_encC_subset_cone_enc_iff {sigma tau : Str} : cone (encC sigma)  subseteq
     cone (enc tau)  <->  tau <+: sigma := by
  rw [cone_subset_cone, enc_prefix_encC_iff]

theorem cone_encC_subset_iff {sigma tau : Str} : cone (encC sigma)  subseteq  cone (
    encC tau)  <->  sigma = tau := by
  rw [cone_subset_cone, encC_prefix_encC_iff, eq_comm]
```

```
theorem cone_enc_subset_iff {sigma tau : Str} : cone (enc sigma)  subseteq  cone (enc
    tau)  <->  tau <+: sigma := by
  rw [cone_subset_cone, enc_prefix_iff]

theorem cone_ne_singleton (sigma tau : Str) : cone sigma  !=  ({tau} : Set Str) := by
  intro h
  have h1 : sigma  in  cone sigma := List.prefix_rfl
  have h2 : sigma ++ [true]  in  cone sigma := <[true], rfl>
  rw [h] at h1 h2
  simp only [Set.mem_singleton_iff] at h1 h2
  exact absurd (h2.trans h1.symm) (by simp)

/-- **No `B`-cone reaches "past" a completion.** `encC sigma` is never a prefix of `enc
    tau`: a
"completed" position of `C` never contains, as a subset, a "still going" cone. -/
theorem not_encC_prefix_enc {sigma tau : Str} : not  encC sigma <+: enc tau := by
  induction sigma generalizing tau with
  | nil =>
    cases tau with
    | nil => simp [encC_nil]
    | cons c tau' =>
      rw [encC_nil, enc_cons]
      intro h
      obtain <rfl, h2> := List.cons_prefix_cons.mp h
      obtain <h3, -> := List.cons_prefix_cons.mp h2
      exact absurd h3 (by decide)
  | cons b sigma' ih =>
    cases tau with
    | nil =>
      rw [encC_cons, enc_nil]
      intro h
      exact absurd h.length_le (by simp)
    | cons c tau' =>
      rw [encC_cons, enc_cons]
      intro h
      obtain <rfl, h2> := List.cons_prefix_cons.mp h
      obtain <-, h3> := List.cons_prefix_cons.mp h2
      exact ih h3

theorem enc_ne_encC (sigma tau : Str) : enc sigma  !=  encC tau := by
  intro h
  exact not_encC_prefix_enc (sigma := tau) (tau := sigma) (h  |>  List.prefix_rfl)

theorem not_cone_enc_subset_cone_encC {sigma tau : Str} : not  cone (enc sigma)
    subseteq  cone (encC tau) := by
  rw [cone_subset_cone]; exact not_encC_prefix_enc

/-- **Inverse half of `C  ~= ? D'`.** -/
def fromD' (y : D'.Element) : C.Element where
  mem X := (exists  sigma, y.mem (cone (enc sigma))  /\  X = cone sigma)  \/  (exists
     sigma, y.mem (cone (encC sigma))  /\  X = ({sigma} : Set Str))
  sub := by
    rintro X (<sigma, -, rfl> | <sigma, -, rfl>)
    * exact memC_cone sigma
    * exact memC_singleton sigma
  master_mem := by
```

```
  refine Or.inl <[], ?_, cone_nil.symm>
  rw [enc_nil, cone_nil,  <-  D'_master]
  exact y.master_mem
inter_mem := by
  rintro X X' (<sigma, hsigma, rfl> | <sigma, hsigma, rfl>) (<tau, htau, rfl> | <tau
  , htau, rfl>)
  * -- cone/cone
    have hy := y.inter_mem hsigma htau
    rcases cone_trichotomy (enc sigma) (enc tau) with h | h | h
    * exact Or.inl <sigma, hsigma, by rw [Set.inter_eq_left.mpr (cone_subset_cone.
  mpr
        (cone_enc_subset_iff.mp h))]>
    * exact Or.inl <tau, htau, by rw [Set.inter_eq_right.mpr (cone_subset_cone.mpr
        (cone_enc_subset_iff.mp h))]>
    * rw [h] at hy; exact absurd (B_nonempty {} (D'_subsystem_B.sub (y.sub hy))) (by
   simp)
  * -- cone/completion
    have hy := y.inter_mem hsigma htau
    rcases cone_trichotomy (enc sigma) (encC tau) with h | h | h
    * exfalso
      exact absurd h not_cone_enc_subset_cone_encC
    * exact Or.inr <tau, htau, by rw [Set.inter_eq_right.mpr (singleton_subset_cone.
  mpr
        (cone_encC_subset_cone_enc_iff.mp h))]>
    * rw [h] at hy; exact absurd (B_nonempty {} (D'_subsystem_B.sub (y.sub hy))) (by
   simp)
  * -- completion/cone
    have hy := y.inter_mem hsigma htau
    rcases cone_trichotomy (encC sigma) (enc tau) with h | h | h
    * exact Or.inr <sigma, hsigma, by rw [Set.inter_eq_left.mpr (
  singleton_subset_cone.mpr
        (cone_encC_subset_cone_enc_iff.mp h))]>
    * exfalso
      exact absurd h not_cone_enc_subset_cone_encC
    * rw [h] at hy; exact absurd (B_nonempty {} (D'_subsystem_B.sub (y.sub hy))) (by
   simp)
  * -- completion/completion
    have hy := y.inter_mem hsigma htau
    by_cases hsigmatau : sigma = tau
    * subst hsigmatau
      exact Or.inr <sigma, hsigma, by rw [Set.inter_self]>
    * exfalso
      have h' : cone (encC sigma)  I  cone (encC tau) = {} := by
        rcases cone_trichotomy (encC sigma) (encC tau) with h | h | h
        * exact absurd (cone_encC_subset_iff.mp h) hsigmatau
        * exact absurd (cone_encC_subset_iff.mp h) (Ne.symm hsigmatau)
        * exact h
      rw [h'] at hy
      exact absurd (B_nonempty {} (D'_subsystem_B.sub (y.sub hy))) (by simp)
up_mem := by
  rintro X X' (<sigma, hsigma, rfl> | <sigma, hsigma, rfl>) hX' hsub
  * rcases hX' with <?, rfl> | <?, rfl>
    * refine Or.inl <?, ?_, rfl>
      have h?sigma : ? <+: sigma := cone_subset_cone.mp hsub
      exact y.up_mem hsigma (Or.inl <?, rfl>) (cone_enc_subset_iff.mpr h?sigma)
    * exfalso
```

```
        have h1 : sigma  in  cone sigma := List.prefix_rfl
        have h2 : sigma ++ [true]  in  cone sigma := <[true], rfl>
        have hsigma?1 : sigma = ? := hsub h1
        have hsigma?2 : sigma ++ [true] = ? := hsub h2
        exact absurd (hsigma?2.trans hsigma?1.symm) (by simp)
    * rcases hX' with <?, rfl> | <?, rfl>
      * refine Or.inl <?, ?_, rfl>
        have h?sigma : ? <+: sigma := singleton_subset_cone.mp hsub
        exact y.up_mem hsigma (Or.inl <?, rfl>) (cone_encC_subset_cone_enc_iff.mpr h?
    sigma)
      * refine Or.inr <?, ?_, rfl>
        have hsigma? : sigma = ? := hsub rfl
        exact y.up_mem hsigma (Or.inr <?, rfl>) (cone_encC_subset_iff.mpr hsigma?)

/-- **Forward half of `C  ~= ? D'`.** Sends a `C`-element `x` to the `D'`-element
    recording, for each
`D'`-shape, whether `x` reaches the corresponding `C`-neighbourhood. -/
def toD' (x : C.Element) : D'.Element where
  mem W := (exists  sigma, x.mem (cone sigma)  /\  W = cone (enc sigma))  \/  (exists
     sigma, x.mem ({sigma} : Set Str)  /\  W = cone (encC sigma))
  sub := by
    rintro W (<sigma, -, rfl> | <sigma, -, rfl>)
    * exact Or.inl <sigma, rfl>
    * exact Or.inr <sigma, rfl>
  master_mem := by
    rw [D'_master]
    refine Or.inl <[], ?_, ?_>
    * rw [cone_nil]; exact x.master_mem
    * rw [enc_nil, cone_nil]
  inter_mem := by
    rintro W W' (<sigma, hsigma, rfl> | <sigma, hsigma, rfl>) (<tau, htau, rfl> | <tau
    , htau, rfl>)
    * -- cone/cone
      have hx := x.inter_mem hsigma htau
      rcases cone_trichotomy sigma tau with h | h | h
      * exact Or.inl <sigma, hsigma, by rw [Set.inter_eq_left.mpr
          (cone_enc_subset_iff.mpr (cone_subset_cone.mp h))]>
      * exact Or.inl <tau, htau, by rw [Set.inter_eq_right.mpr
          (cone_enc_subset_iff.mpr (cone_subset_cone.mp h))]>
      * rw [h] at hx; exact absurd (C_nonempty {} (x.sub hx)) (by simp)
    * -- cone/singleton
      have hx := x.inter_mem hsigma htau
      rcases singleton_cone_nd sigma tau with h | h | h
      * exact Or.inr <tau, htau, by rw [Set.inter_eq_right.mpr
          (cone_encC_subset_cone_enc_iff.mpr (singleton_subset_cone.mp h))]>
      * exfalso
        have h1 : sigma  in  cone sigma := List.prefix_rfl
        have h2 : sigma ++ [true]  in  cone sigma := <[true], rfl>
        have hsigmatau1 : sigma = tau := h h1
        have hsigmatau2 : sigma ++ [true] = tau := h h2
        exact absurd (hsigmatau2.trans hsigmatau1.symm) (by simp)
      * exfalso
        have h' : cone sigma  I  {tau} = {} := by rw [Set.inter_comm]; exact h
        rw [h'] at hx
        exact absurd (C_nonempty {} (x.sub hx)) (by simp)
    * -- singleton/cone
```

```
      have hx := x.inter_mem hsigma htau
      rcases singleton_cone_nd tau sigma with h | h | h
      * exact Or.inr <sigma, hsigma, by rw [Set.inter_eq_left.mpr
          (cone_encC_subset_cone_enc_iff.mpr (singleton_subset_cone.mp h))]>
      * exfalso
        have h1 : tau  in  cone tau := List.prefix_rfl
        have h2 : tau ++ [true]  in  cone tau := <[true], rfl>
        have htausigma1 : tau = sigma := h h1
        have htausigma2 : tau ++ [true] = sigma := h h2
        exact absurd (htausigma2.trans htausigma1.symm) (by simp)
      * exfalso
        rw [h] at hx
        exact absurd (C_nonempty {} (x.sub hx)) (by simp)
    * -- singleton/singleton
      have hx := x.inter_mem hsigma htau
      by_cases hsigmatau : sigma = tau
      * subst hsigmatau
        exact Or.inr <sigma, hsigma, by rw [Set.inter_self]>
      * exfalso
        have h' : ({sigma} : Set Str)  I  {tau} = {} := by
          ext w; simp only [Set.mem_inter_iff, Set.mem_singleton_iff, Set.
    mem_empty_iff_false,
            iff_false, not_and]
          rintro rfl; exact hsigmatau
        rw [h'] at hx
        exact absurd (C_nonempty {} (x.sub hx)) (by simp)
  up_mem := by
    rintro W W' (<sigma, hsigma, rfl> | <sigma, hsigma, rfl>) hW' hsub
    * rcases hW' with <?, rfl> | <?, rfl>
      * refine Or.inl <?, ?_, rfl>
        have h?sigma : ? <+: sigma := cone_enc_subset_iff.mp hsub
        exact x.up_mem hsigma (memC_cone ?) (cone_subset_cone.mpr h?sigma)
      * exact absurd hsub not_cone_enc_subset_cone_encC
    * rcases hW' with <?, rfl> | <?, rfl>
      * refine Or.inl <?, ?_, rfl>
        have h?sigma : ? <+: sigma := cone_encC_subset_cone_enc_iff.mp hsub
        exact x.up_mem hsigma (memC_cone ?) (singleton_subset_cone.mpr h?sigma)
      * refine Or.inr <?, ?_, rfl>
        have hsigmatau : sigma = ? := cone_encC_subset_iff.mp hsub
        exact hsigmatau  |>  hsigma

@[simp] theorem toD'_mem_enc {x : C.Element} (? : Str) :
    (toD' x).mem (cone (enc ?))  <->  x.mem (cone ?) := by
  constructor
  * rintro (<sigma, hsigma, heq> | <sigma, hsigma, heq>)
    * have h?sigma : ? = sigma := enc_injective (cone_injective heq)
      rw [h?sigma]; exact hsigma
    * exact absurd (cone_injective heq) (enc_ne_encC ? sigma)
  * intro hx; exact Or.inl <?, hx, rfl>

@[simp] theorem toD'_mem_encC {x : C.Element} (? : Str) :
    (toD' x).mem (cone (encC ?))  <->  x.mem ({?} : Set Str) := by
  constructor
  * rintro (<sigma, hsigma, heq> | <sigma, hsigma, heq>)
    * exact absurd (cone_injective heq).symm (enc_ne_encC sigma ?)
    * have h?sigma : ? = sigma := encC_injective (cone_injective heq)
```

```
      rw [h?sigma]; exact hsigma
  * intro hx; exact Or.inr <?, hx, rfl>

@[simp] theorem fromD'_mem_cone {y : D'.Element} (? : Str) :
    (fromD' y).mem (cone ?)  <->  y.mem (cone (enc ?)) := by


  constructor
  * rintro (<sigma, hsigma, heq> | <sigma, hsigma, heq>)
    * have h?sigma : ? = sigma := cone_injective heq
      rw [h?sigma]; exact hsigma
    * exact absurd heq (cone_ne_singleton ? sigma)
  * intro hy; exact Or.inl <?, hy, rfl>

@[simp] theorem fromD'_mem_singleton {y : D'.Element} (? : Str) :
    (fromD' y).mem ({?} : Set Str)  <->  y.mem (cone (encC ?)) := by
  constructor
  * rintro (<sigma, hsigma, heq> | <sigma, hsigma, heq>)
    * exact absurd heq.symm (cone_ne_singleton sigma ?)
    * have h?sigma : ? = sigma := by
        have h1 : ?  in  ({sigma} : Set Str) := heq  |>  rfl
        simpa using h1
      rw [h?sigma]; exact hsigma
  * intro hy; exact Or.inr <?, hy, rfl>

theorem fromD'_toD' (x : C.Element) : fromD' (toD' x) = x := by
  apply NeighborhoodSystem.Element.ext
  intro X
  constructor
  * rintro (<sigma, hsigma, rfl> | <sigma, hsigma, rfl>)
    * exact (toD'_mem_enc sigma).mp hsigma
    * exact (toD'_mem_encC sigma).mp hsigma
  * intro hX
    rcases x.sub hX with <sigma, rfl> | <sigma, rfl>
    * exact Or.inl <sigma, (toD'_mem_enc sigma).mpr hX, rfl>
    * exact Or.inr <sigma, (toD'_mem_encC sigma).mpr hX, rfl>

theorem toD'_fromD' (y : D'.Element) : toD' (fromD' y) = y := by
  apply NeighborhoodSystem.Element.ext
  intro W
  constructor
  * rintro (<?, h?, rfl> | <?, h?, rfl>)
    * exact (fromD'_mem_cone ?).mp h?
    * exact (fromD'_mem_singleton ?).mp h?
  * intro hW
    rcases y.sub hW with <?, rfl> | <?, rfl>
    * exact Or.inl <?, (fromD'_mem_cone ?).mpr hW, rfl>
    * exact Or.inr <?, (fromD'_mem_singleton ?).mpr hW, rfl>

/-- **`C  ~= ? D'`.** -/
def cdEquiv : C.Element ?o D'.Element where
  toFun := toD'
  invFun := fromD'
  left_inv := fromD'_toD'
  right_inv := toD'_fromD'
  map_rel_iff' := by
    intro x x'
```

```
    constructor
    * intro h X hX
      rcases x.sub hX with <sigma, rfl> | <sigma, rfl>
      * exact (toD'_mem_enc sigma).mp (h _ ((toD'_mem_enc sigma).mpr hX))
      * exact (toD'_mem_encC sigma).mp (h _ ((toD'_mem_encC sigma).mpr hX))
    * intro h W hW
      rcases hW with <sigma, hsigma, rfl> | <sigma, hsigma, rfl>
      * exact Or.inl <sigma, h _ hsigma, rfl>
      * exact Or.inr <sigma, h _ hsigma, rfl>

/-- **`B` is a literal subsystem of `C`** (Definition 6.10): every `B`-neighbourhood is
     already a
`C`-neighbourhood, and consistency is inherited -- if two cones intersect to a `C`-
    neighbourhood
(hence nonempty), `cone_trichotomy` rules out the disjoint case, so the intersection is
     already the
smaller of the two cones. -/
theorem B_subsystem_C : B ? C where
  master_eq := rfl
  sub := by rintro X <sigma, rfl>; exact memC_cone sigma
  inter_closed := by
    rintro X Y <sigma, rfl> <tau, rfl> hCXY
    rcases cone_trichotomy sigma tau with h | h | h
    * rw [Set.inter_eq_left.mpr h]; exact <sigma, rfl>
    * rw [Set.inter_eq_right.mpr h]; exact <tau, rfl>
    * rw [h] at hCXY; exact absurd (C_nonempty {} hCXY) (by simp)

/-- **`B ? C`** (Exercise 8.22). The projection Scott asks us to notice: the identity
    inclusion
`B ? C`. -/
theorem B_trianglelefteq_C : B ? C :=
  Subsystem.trianglelefteq B_subsystem_C

theorem C_isomorphic_D' : C  ~= ? D' := <cdEquiv>

/-- **`C ? B`.** The nontrivial half of Exercise 8.22: `C` (finite-or-infinite binary
    sequences)
embeds as a subdomain of `B` (infinite binary sequences) via the bit-doubling re-
    encoding
`enc`/`encC` and the image subsystem `D' ? B`. -/
theorem C_trianglelefteq_B : C ? B := <D', D'_subsystem_B, C_isomorphic_D'>

end Exercise822

end DomainTheory.NeighbourhoodSystems
```

The tip then assembles those tools into Scott's statement.

— exercise_8_22

Exercise_8_22.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    ExampleC_le_B

@[expose] public section
```

```
open DomainTheory.NeighbourhoodSystems

/-- **Exercise 8.22 (Scott 1981, PRG-19).** `C ? B` for Scott's examples. -/
theorem exercise_8_22 : Example44.C ? ExampleB.B :=
  Exercise822.C_trianglelefteq_B

end
```

3.9.25 Exercise 8.23 — initial algebra inside U

Suppose a construct T on domains can be made into a computable operator $t : (U \to U) \to (U \to U)$ so that whenever $a : U \to U$ is a finitary projection, then so is $t(a)$ and

$$D_{t(a)} \cong T(D_a) \,.$$

Does it follow that $\|t\| = \mathrm{fix}(t)$ is such that

$$D_{\|t\|} \cong T(D_{\|t\|})$$

really is the initial solution of the domain equation with respect to projections? Since t is computable, will this solution be effectively given?

Tip summary The tip asserts that `fixOp t` is a finitary projection whenever `t` preserves them.

Development Defines `fixOp` and proves it is finitary when the transformer preserves finitary projections.

FixOp.lean

```
module

public import DomainTheory.NeighbourhoodSystems.FixedPointsAndRecursion.Theorem_4_2

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    FixedNbhdFinitaryOrder
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Examples_8_4

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

/- concordance: source L5207 EXERCISE 8.23 initial algebra inside U -/
namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap Sub8_6

namespace Exercise823

variable {a : Type} {E : NeighborhoodSystem a}

/-- The fixed-point set of the constant-bottom map is the singleton `{False}`. -/
```

```
theorem eq_bot_of_fixed_constMap_bot {y : E.Element}
    (hy : (constMap E E.bot).toElementMap y = y) : y = E.bot := by
  rw [toElementMap_constMap] at hy
  exact hy.symm

instance uniqueFixedConstMapBot :
    Unique {y : E.Element // (constMap E E.bot).toElementMap y = y} where
  default := <E.bot, toElementMap_constMap E.bot E.bot>
  uniq := fun <_y, hy> => Subtype.ext (eq_bot_of_fixed_constMap_bot hy)

/-- The singleton fixed-point set of the constant-bottom map is order-isomorphic to
    the (also
singleton) terminal system `unitSys` (`Exercise315.lean`). -/
def finitaryWitnessEquivBot (E : NeighborhoodSystem a) :
    {y : E.Element // (constMap E E.bot).toElementMap y = y} ?o unitSys.Element where
  toFun _ := default
  invFun _ := default
  left_inv a := (Unique.eq_default a).symm
  right_inv b := (Unique.eq_default b).symm
  map_rel_iff' := by
    intro a b
    have hab : a = b := (Unique.eq_default a).trans (Unique.eq_default b).symm
    exact iff_of_true (le_refl default) (le_of_eq hab)

/-- **Scott's `?t? = fix(t)`**, as an actual approximable map `E  ->  E`. -/
def fixOp (t : ApproximableMap (funSpace E E) (funSpace E E)) : ApproximableMap E E :=
  toApproxMap t.fixElement

/-- **Scott's `t(a)`**: the operator `t : (E  ->  E)  ->  (E  ->  E)` induced by an
    approximable self-map
`t` of the function space `funSpace E E`, applied to an actual map `a : E  ->  E`. -/
def tOp (t : ApproximableMap (funSpace E E) (funSpace E E)) (a : ApproximableMap E E)
    :
    ApproximableMap E E :=
  toApproxMap (t.toElementMap (toFilter a))

/-- The fixed-point-set ("`D_a`") of an approximable self-map, matching `IsFinitary`'s
    own
formulation. -/
abbrev Fix (a : ApproximableMap E E) := {y : E.Element // a.toElementMap y = y}

theorem constMap_bot_le_idMap (E : NeighborhoodSystem a) :
    constMap E E.bot  <=  idMap E := by
  rintro X Z hr
  rw [constMap_rel, mem_bot] at hr
  obtain <hX, rfl> := hr
  exact <hX, E.master_mem, E.sub_master hX>

theorem isRetraction_constMap_bot (E : NeighborhoodSystem a) :
    Retraction (constMap E E.bot) := by
  constructor
  show (constMap E E.bot).comp (constMap E E.bot) = constMap E E.bot
  apply ApproximableMap.ext
  intro X Z
  simp only [comp_rel, constMap_rel, mem_bot]
  constructor
```

```
  * rintro <Y, <hX, rfl>, -, hZ>
    exact <hX, hZ>
  * rintro <hX, hZ>
    exact <E.master, <hX, rfl>, E.master_mem, hZ>

theorem isProjection_constMap_bot (E : NeighborhoodSystem a) :
    IsProjection (constMap E E.bot) :=
  <isRetraction_constMap_bot E, constMap_bot_le_idMap E>

theorem isFinitaryProjection_constMap_bot (E : NeighborhoodSystem a) :
    FinitaryProjection (constMap E E.bot) :=
  <<isProjection_constMap_bot E, Unit, unitSys, <finitaryWitnessEquivBot E>>>

theorem iterElem_succ' (t : ApproximableMap (funSpace E E) (funSpace E E)) (n : Nat) :
    t.iterElem (n + 1) = t.toElementMap (t.iterElem n) := by
  show (t.comp (t.iterMap n)).toElementMap (funSpace E E).bot = t.toElementMap (t.
    iterElem n)
  rw [toElementMap_comp]
  rfl

theorem iterElem_zero' (t : ApproximableMap (funSpace E E) (funSpace E E)) :
    t.iterElem 0 = (funSpace E E).bot := by
  show (idMap (funSpace E E)).toElementMap (funSpace E E).bot = (funSpace E E).bot
  rw [toElementMap_idMap]

/-- Every approximable map relates every consistent neighbourhood to the master (widen
     the master
relation's input down to `X` using `mono`, since `X  subseteq  Delta` always). -/
theorem rel_master_of_mem (f : ApproximableMap E E) {X : Set a} (hX : E.mem X) :
    f.rel X E.master :=
  f.mono f.master_rel (E.sub_master hX) subset_rfl hX E.master_mem

/-- **`?t?` is a genuine (not merely approximate) fixed point of `t`.** -/
theorem tOp_fixOp (t : ApproximableMap (funSpace E E) (funSpace E E)) :
    tOp t (fixOp t) = fixOp t := by
  show toApproxMap (t.toElementMap (toFilter (toApproxMap t.fixElement))) =
    toApproxMap t.fixElement
  rw [toFilter_toApproxMap, toElementMap_fixElement]

theorem tOp_toApproxMap (t : ApproximableMap (funSpace E E) (funSpace E E))
    (phi : (funSpace E E).Element) : tOp t (toApproxMap phi) = toApproxMap (t.
    toElementMap phi) := by
  unfold tOp
  rw [toFilter_toApproxMap]

/-- **`toApproxMap` of the function space's own bottom element is the constant-bottom
    map.**
Mirrors `Theorem616.lean`'s `botStrict_rel` for the (non-strict) function space `
    funSpace E E`. -/
theorem toApproxMap_bot_eq_constMap (E : NeighborhoodSystem a) :
    toApproxMap (funSpace E E).bot = constMap E E.bot := by
  apply ApproximableMap.ext
  intro X Y
  simp only [toApproxMap_rel, mem_bot, constMap_rel]
  constructor
  * intro h
```

```
    have hcb : constMap E E.bot  in  step X Y := by rw [h]; trivial
    rwa [mem_step, constMap_rel, mem_bot] at hcb
  * rintro <hX, rfl>
    exact Set.eq_univ_of_forall fun f => rel_master_of_mem f hX

/-- **Exercise 8.23, Claim 1.** Provided `t` maps finitary projections to finitary
    projections,
`?t? = fix(t)` really is a finitary projection. -/
theorem isFinitaryProjection_fixOp (t : ApproximableMap (funSpace E E) (funSpace E E))
    (ht : forall  a, FinitaryProjection a  ->  FinitaryProjection (tOp t a)) :
    FinitaryProjection (fixOp t) := by
  have hchain : forall  n, FinitaryProjection (toApproxMap (t.iterElem n)) := by
    intro n
    induction n with
    | zero =>
      rw [iterElem_zero', toApproxMap_bot_eq_constMap]
      exact isFinitaryProjection_constMap_bot E
    | succ k ih =>
      rw [iterElem_succ',  <-  tOp_toApproxMap t (t.iterElem k)]
      exact ht _ ih
  have hsub : forall  n, subFilter (t.iterElem n) = t.iterElem n := by
    intro n
    apply toApproxMap_injective
    rw [toApproxMap_subFilter]
    exact sub_eq_self_of_isFinitaryProjection (hchain n)
  have hdir : forall  i j : Nat, exists  k, t.iterElem i  <=  t.iterElem k  /\  t.
    iterElem j  <=  t.iterElem k :=
    fun i j => <max i j, iterElem_mono t (le_max_left i j), iterElem_mono t (
    le_max_right i j)>
  have hdirSub : forall  i j : Nat, exists  k, subFilter (t.iterElem i)  <=  subFilter
     (t.iterElem k)  /\
      subFilter (t.iterElem j)  <=  subFilter (t.iterElem k) := by
    intro i j
    obtain <k, hik, hjk> := hdir i j
    refine <k, ?_, ?_>
    * rw [hsub i, hsub k]; exact hik
    * rw [hsub j, hsub k]; exact hjk
  have hEq : t.fixElement = iSupDirected t.iterElem hdir := by
    apply NeighborhoodSystem.Element.ext
    intro Z
    rw [mem_fixElement, mem_iSupDirected]
    simp_rw [mem_iterElem]
  have hkey : subFilter t.fixElement = t.fixElement := by
    rw [hEq, subFilter_iSupDirected t.iterElem hdir hdirSub]
    apply NeighborhoodSystem.Element.ext
    intro Z
    rw [mem_iSupDirected, mem_iSupDirected]
    simp_rw [hsub]
  show FinitaryProjection (toApproxMap t.fixElement)
  apply isFinitaryProjection_of_sub_eq_self
  rw [ <-  toApproxMap_subFilter, hkey]

/-- **Exercise 8.23, Claim 2 (the domain equation).** Given any correspondence `T` (
    bundling, for
each map, a type and a domain over it) with `Fix(t(a))  ~=  T(a)` for every finitary
    projection `a`
```

```
(Scott's hypothesis `D_{t(a)}  ~=  T(D_a)`), substituting the *exact* fixed point `a :=
     ?t?` and using
`tOp_fixOp` gives `Fix(?t?)  ~=  T(?t?)` unconditionally -- no colimit/continuity
    argument needed,
since `?t?` solves the equation on the nose rather than merely in the limit. -/
theorem fixedDomain_fixOp_iso_T (t : ApproximableMap (funSpace E E) (funSpace E E))
    (ht : forall  a, FinitaryProjection a  ->  FinitaryProjection (tOp t a))
    (T : ApproximableMap E E  ->  Sigma b : Type, NeighborhoodSystem b)
    (hT : forall  a, FinitaryProjection a  ->  Nonempty (Fix (tOp t a) ?o (T a).2.
    Element)) :
    Nonempty (Fix (fixOp t) ?o (T (fixOp t)).2.Element) := by
  have h := hT (fixOp t) (isFinitaryProjection_fixOp t ht)
  rwa [tOp_fixOp] at h

/-- **Exercise 8.23, Claim 2 (initiality with respect to projections).** If `a` is *any
    * other
finitary projection that is a pre-fixed point of the same operator (`tOp t a  <=  a`,
    in particular
any exact alternative solution `tOp t a = a`), then `?t?`'s domain embeds as a literal
    subsystem of
`a`'s domain: `D_{?t?} ? D_a` (hence certainly `D_{?t?} ? D_a`, Lemma 6.15) -- Scott's
    "the initial
solution... with respect to projections". The proof is Theorem 4.1's minimality
(`fixElement_le_of_toElementMap_le`) transported through Exercise 8.16's order-
    isomorphism
`a  <=  b  <->  D_a ? D_b`. -/
theorem fixedNbhd_fixOp_subsystem (t : ApproximableMap (funSpace E E) (funSpace E E))
    (ht : forall  a, FinitaryProjection a  ->  FinitaryProjection (tOp t a))
    {a : ApproximableMap E E} (ha : FinitaryProjection a) (hpre : tOp t a  <=  a) :
    fixedNbhd (fixOp t) ? fixedNbhd a := by
  have hpre' : toApproxMap (t.toElementMap (toFilter a))  <=  toApproxMap (toFilter a)
     := by
    show tOp t a  <=  toApproxMap (toFilter a)
    rw [toApproxMap_toFilter]
    exact hpre
  have h1 : t.toElementMap (toFilter a)  <=  toFilter a := by
    have hiff := (funSpaceEquiv E E).le_iff_le
      (x := t.toElementMap (toFilter a)) (y := toFilter a)
    rw [funSpaceEquiv_apply, funSpaceEquiv_apply] at hiff
    exact hiff.mp hpre'
  have h2 : t.fixElement  <=  toFilter a := fixElement_le_of_toElementMap_le t h1
  have hle : fixOp t  <=  a := by
    show toApproxMap t.fixElement  <=  a
    rw [ <-  toApproxMap_toFilter a]
    exact toApproxMap_monotone h2
  exact (isFinitaryProjection_le_iff_fixedNbhd_subsystem
    (isFinitaryProjection_fixOp t ht) ha).mp hle

end Exercise823

end DomainTheory.NeighbourhoodSystems
```

The tip then assembles those tools into Scott's statement.

— exercise_8_23

Exercise_8_23.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.FixOp

@[expose] public section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap Exercise823

/-- **Exercise 8.23 (Scott 1981, PRG-19).** The fixed-point operator `fixOp t` is a
    finitary projection
when `t` preserves finitary projections. -/
theorem exercise_8_23 {a : Type} {E : NeighborhoodSystem a}
    (t : ApproximableMap (funSpace E E) (funSpace E E))
    (ht : forall  a, FinitaryProjection a  ->  FinitaryProjection (tOp t a)) :
    FinitaryProjection (fixOp t) :=
  isFinitaryProjection_fixOp t ht

end
```

3.9.26 Exercise 8.24 — coupled domain equations

Suppose S and T are two (binary-argument) constructs on domains that can be made into computable operators on projections of the universal domain. Show that we can therefore find a pair of effectively presented domains such that

$$D \cong S(D,E) \quad \text{and} \quad E \cong T(D,E).$$

Tip summary The tip asserts that the coupled solutions `aStar`, `bStar` are finitary projections.

Development Solves the coupled domain equations with finitary projections `aStar`, `bStar`.

CoupledFixedPoints.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.FixOp

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Theorem_8_8

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Proposition_7_3
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

/- concordance: source L5219 EXERCISE 8.24 coupled domain equations -/
namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap Sub8_6 Exercise823

namespace Exercise824

variable {a : Type} {E : NeighborhoodSystem a}
```

```
/-- Scott's pair space `(E  ->  E)  x  (E  ->  E)`: pairs of self-maps of `E`, as an
    actual
`NeighborhoodSystem` (Lecture III's product of two copies of the function space). -/
abbrev PairSpace : NeighborhoodSystem (ApproximableMap E E ? ApproximableMap E E) :=
  prod (funSpace E E) (funSpace E E)

/-- The `n`-th approximant's first component, as an actual approximable map `E  ->  E`.
     -/
def aOp (s t : ApproximableMap (PairSpace (E := E)) (funSpace E E)) (n : Nat) :
    ApproximableMap E E :=
  toApproxMap ((paired s t).iterElem n).fst

/-- Scott's `?s? = fix(<s, t>)`'s first component, as an actual approximable map `E  ->
     E`. -/
def aStar (s t : ApproximableMap (PairSpace (E := E)) (funSpace E E)) :
    ApproximableMap E E :=
  toApproxMap (paired s t).fixElement.fst

/-- The `n`-th approximant's second component, as an actual approximable map `E  ->  E
    `. -/
def bOp (s t : ApproximableMap (PairSpace (E := E)) (funSpace E E)) (n : Nat) :
    ApproximableMap E E :=
  toApproxMap ((paired s t).iterElem n).snd

/-- Scott's `?t? = fix(<s, t>)`'s second component, as an actual approximable map `E
    ->  E`. -/
def bStar (s t : ApproximableMap (PairSpace (E := E)) (funSpace E E)) :
    ApproximableMap E E :=
  toApproxMap (paired s t).fixElement.snd

/-- Scott's `s(a, b)`: the operator `(E  ->  E)  x  (E  ->  E)  ->  (E  ->  E)` induced
     by an approximable map
`s` out of the pair space, applied to two actual maps `a b : E  ->  E`. -/
def binOp (f : ApproximableMap (PairSpace (E := E)) (funSpace E E))
    (a b : ApproximableMap E E) : ApproximableMap E E :=
  toApproxMap (f.toElementMap (pair (toFilter a) (toFilter b)))

theorem fst_mono {z z' : (PairSpace (E := E)).Element} (h : z  <=  z') : z.fst  <=  z
    '.fst := by
  intro X hX
  rw [mem_fst] at hX  |-
  exact <hX.1, h _ hX.2>

/-- The `n`-th approximant of `<s, t>` is exactly the pairing of `aOp`'s and `bOp`'s `n
    `-th
approximants (`Product.lean`'s `pair_fst_snd`, transported through the `toFilter`/`
    toApproxMap`
round trip). -/
theorem iterElem_pair_eq_pair (s t : ApproximableMap (PairSpace (E := E)) (funSpace E
    E)) (n : Nat) :
    (paired s t).iterElem n = pair (toFilter (aOp s t n)) (toFilter (bOp s t n)) := by
  show (paired s t).iterElem n
      = pair (toFilter (toApproxMap ((paired s t).iterElem n).fst))
          (toFilter (toApproxMap ((paired s t).iterElem n).snd))
  rw [toFilter_toApproxMap, toFilter_toApproxMap, pair_fst_snd]
```

```
theorem iterElem_succ_pair (s t : ApproximableMap (PairSpace (E := E)) (funSpace E E))
     (n : Nat) :
    (paired s t).iterElem (n + 1) = (paired s t).toElementMap ((paired s t).iterElem n
    ) := by
  show ((paired s t).comp ((paired s t).iterMap n)).toElementMap (PairSpace (E := E)).
    bot
      = (paired s t).toElementMap (((paired s t).iterMap n).toElementMap (PairSpace (E
     := E)).bot)
  rw [toElementMap_comp]

/-- **The mutual recursion on approximants**: `aOp s t (n+1) = binOp s (aOp s t n) (bOp
     s t n)`. -/
theorem aOp_succ (s t : ApproximableMap (PairSpace (E := E)) (funSpace E E)) (n : Nat)
     :
    aOp s t (n + 1) = binOp s (aOp s t n) (bOp s t n) := by
  show toApproxMap ((paired s t).iterElem (n + 1)).fst
      = toApproxMap (s.toElementMap (pair (toFilter (aOp s t n)) (toFilter (bOp s t n)
    )))
  rw [iterElem_succ_pair, toElementMap_paired, fst_pair, iterElem_pair_eq_pair]

/-- **The mutual recursion on approximants**: `bOp s t (n+1) = binOp t (aOp s t n) (bOp
     s t n)`. -/
theorem bOp_succ (s t : ApproximableMap (PairSpace (E := E)) (funSpace E E)) (n : Nat)
     :
    bOp s t (n + 1) = binOp t (aOp s t n) (bOp s t n) := by
  show toApproxMap ((paired s t).iterElem (n + 1)).snd
      = toApproxMap (t.toElementMap (pair (toFilter (aOp s t n)) (toFilter (bOp s t n)
    )))
  rw [iterElem_succ_pair, toElementMap_paired, snd_pair, iterElem_pair_eq_pair]

theorem iterElem_zero_pair (s t : ApproximableMap (PairSpace (E := E)) (funSpace E E))
     :
    (paired s t).iterElem 0 = (PairSpace (E := E)).bot := by
  show (idMap (PairSpace (E := E))).toElementMap (PairSpace (E := E)).bot
      = (PairSpace (E := E)).bot
  rw [toElementMap_idMap]

theorem pair_bot_bot : pair ((funSpace E E).bot) ((funSpace E E).bot) = (PairSpace (E
    := E)).bot := by
  apply Element.ext
  intro W
  rw [mem_pair, mem_bot]
  constructor
  * rintro <X, Y, hX, hY, rfl>
    rw [mem_bot] at hX hY
    rw [hX, hY, prod_master]
  * intro hW
    refine <(funSpace E E).master, (funSpace E E).master, ?_, ?_, ?_>
    * rw [mem_bot]
    * rw [mem_bot]
    * rw [hW, prod_master]

theorem pairSpace_bot_fst : ((PairSpace (E := E)).bot).fst = (funSpace E E).bot := by
  rw [ <-  pair_bot_bot, fst_pair]
```

```
theorem aOp_zero (s t : ApproximableMap (PairSpace (E := E)) (funSpace E E)) :
    aOp s t 0 = constMap E E.bot := by
  show toApproxMap ((paired s t).iterElem 0).fst = constMap E E.bot
  rw [iterElem_zero_pair, pairSpace_bot_fst, Exercise823.toApproxMap_bot_eq_constMap]

theorem pairSpace_bot_snd : ((PairSpace (E := E)).bot).snd = (funSpace E E).bot := by
  rw [ <-  pair_bot_bot, snd_pair]

theorem bOp_zero (s t : ApproximableMap (PairSpace (E := E)) (funSpace E E)) :
    bOp s t 0 = constMap E E.bot := by
  show toApproxMap ((paired s t).iterElem 0).snd = constMap E E.bot
  rw [iterElem_zero_pair, pairSpace_bot_snd, Exercise823.toApproxMap_bot_eq_constMap]

/-- Every approximant pair `(aOp s t n, bOp s t n)` consists of finitary projections,
    provided
`s`/`t` send pairs of finitary projections to finitary projections. -/
theorem isFinitaryProjection_chain (s t : ApproximableMap (PairSpace (E := E)) (
    funSpace E E))
    (hst : forall  a b, FinitaryProjection a  ->  FinitaryProjection b  ->
      FinitaryProjection (binOp s a b)  /\  FinitaryProjection (binOp t a b)) :
    forall  n, FinitaryProjection (aOp s t n)  /\  FinitaryProjection (bOp s t n) :=
    by
  intro n
  induction n with
  | zero =>
    rw [aOp_zero, bOp_zero]
    exact <Exercise823.isFinitaryProjection_constMap_bot E,
      Exercise823.isFinitaryProjection_constMap_bot E>
  | succ k ih =>
    rw [aOp_succ, bOp_succ]
    exact hst _ _ ih.1 ih.2

theorem snd_mono {z z' : (PairSpace (E := E)).Element} (h : z  <=  z') : z.snd  <=  z
    '.snd := by
  intro X hX
  rw [mem_snd] at hX  |-
  exact <hX.1, h _ hX.2>

/-- **Exercise 8.24, Claim 1.** Provided `s`/`t` map pairs of finitary projections to
    finitary
projections, the simultaneous fixed point `(aStar s t, bStar s t)` really consists of
    finitary
projections. -/
theorem isFinitaryProjection_aStar_bStar (s t : ApproximableMap (PairSpace (E := E)) (
    funSpace E E))
    (hst : forall  a b, FinitaryProjection a  ->  FinitaryProjection b  ->
      FinitaryProjection (binOp s a b)  /\  FinitaryProjection (binOp t a b)) :
    FinitaryProjection (aStar s t)  /\  FinitaryProjection (bStar s t) := by
  have hchain := isFinitaryProjection_chain s t hst
  have hsubA : forall  n, subFilter (((paired s t).iterElem n).fst) = ((paired s t).
    iterElem n).fst := by
    intro n
    apply toApproxMap_injective
    rw [toApproxMap_subFilter]
    show sub (aOp s t n) = aOp s t n
    exact sub_eq_self_of_isFinitaryProjection (hchain n).1
```

```
have hsubB : forall  n, subFilter (((paired s t).iterElem n).snd) = ((paired s t).
  iterElem n).snd := by
  intro n
  apply toApproxMap_injective
  rw [toApproxMap_subFilter]
  show sub (bOp s t n) = bOp s t n
  exact sub_eq_self_of_isFinitaryProjection (hchain n).2
have hdirA : forall  i j : Nat, exists  k, ((paired s t).iterElem i).fst  <=  ((
  paired s t).iterElem k).fst  /\
    ((paired s t).iterElem j).fst  <=  ((paired s t).iterElem k).fst :=
  fun i j => <max i j, fst_mono (iterElem_mono (paired s t) (le_max_left i j)),
    fst_mono (iterElem_mono (paired s t) (le_max_right i j))>
have hdirB : forall  i j : Nat, exists  k, ((paired s t).iterElem i).snd  <=  ((
  paired s t).iterElem k).snd  /\
    ((paired s t).iterElem j).snd  <=  ((paired s t).iterElem k).snd :=
  fun i j => <max i j, snd_mono (iterElem_mono (paired s t) (le_max_left i j)),
    snd_mono (iterElem_mono (paired s t) (le_max_right i j))>
have hdirSubA : forall  i j : Nat, exists  k, subFilter (((paired s t).iterElem i).
  fst)  <=
    subFilter (((paired s t).iterElem k).fst)  /\
    subFilter (((paired s t).iterElem j).fst)  <=  subFilter (((paired s t).iterElem
   k).fst) := by
  intro i j
  obtain <k, hik, hjk> := hdirA i j
  exact <k, by rw [hsubA i, hsubA k]; exact hik, by rw [hsubA j, hsubA k]; exact hjk
  >
have hdirSubB : forall  i j : Nat, exists  k, subFilter (((paired s t).iterElem i).
  snd)  <=
    subFilter (((paired s t).iterElem k).snd)  /\
    subFilter (((paired s t).iterElem j).snd)  <=  subFilter (((paired s t).iterElem
   k).snd) := by
  intro i j
  obtain <k, hik, hjk> := hdirB i j
  exact <k, by rw [hsubB i, hsubB k]; exact hik, by rw [hsubB j, hsubB k]; exact hjk
  >
have hEqA : (paired s t).fixElement.fst =
    iSupDirected (fun n => ((paired s t).iterElem n).fst) hdirA := by
  apply Element.ext
  intro X
  rw [mem_fst, mem_fixElement, mem_iSupDirected]
  constructor
  * rintro <hX, n, hn>
    exact <n, mem_fst.mpr <hX, (mem_iterElem (paired s t) n).mpr hn>>
  * rintro <n, hn>
    rw [mem_fst] at hn
    exact <hn.1, n, (mem_iterElem (paired s t) n).mp hn.2>
have hEqB : (paired s t).fixElement.snd =
    iSupDirected (fun n => ((paired s t).iterElem n).snd) hdirB := by
  apply Element.ext
  intro X
  rw [mem_snd, mem_fixElement, mem_iSupDirected]
  constructor
  * rintro <hX, n, hn>
    exact <n, mem_snd.mpr <hX, (mem_iterElem (paired s t) n).mpr hn>>
  * rintro <n, hn>
    rw [mem_snd] at hn
```

```
      exact <hn.1, n, (mem_iterElem (paired s t) n).mp hn.2>
  have hkeyA : subFilter ((paired s t).fixElement.fst) = (paired s t).fixElement.fst
    := by
    rw [hEqA, subFilter_iSupDirected _ hdirA hdirSubA]
    apply Element.ext
    intro Z
    rw [mem_iSupDirected, mem_iSupDirected]
    simp_rw [hsubA]
  have hkeyB : subFilter ((paired s t).fixElement.snd) = (paired s t).fixElement.snd
    := by
    rw [hEqB, subFilter_iSupDirected _ hdirB hdirSubB]
    apply Element.ext
    intro Z
    rw [mem_iSupDirected, mem_iSupDirected]
    simp_rw [hsubB]
  refine <?_, ?_>
  * show FinitaryProjection (toApproxMap (paired s t).fixElement.fst)
    apply isFinitaryProjection_of_sub_eq_self
    rw [ <-  toApproxMap_subFilter, hkeyA]
  * show FinitaryProjection (toApproxMap (paired s t).fixElement.snd)
    apply isFinitaryProjection_of_sub_eq_self
    rw [ <-  toApproxMap_subFilter, hkeyB]

/-- **Exercise 8.24, Claim 2 (initiality with respect to projections).** If `a, b` is *
    any* other
pair of finitary projections that is a pre-fixed point of the same pair of operators
(`binOp s a b  <=  a`, `binOp t a b  <=  b`, in particular any exact alternative
    solution), then
`(aStar s t, bStar s t)`'s domains embed as literal subsystems of `a`'s and `b`'s
    domains:
`D_{aStar s t} ? D_a`, `D_{bStar s t} ? D_b` -- Scott's "the initial solution... with
    respect to
projections", for both components simultaneously. -/
theorem fixedNbhd_aStar_bStar_subsystem (s t : ApproximableMap (PairSpace (E := E)) (
    funSpace E E))
    (hst : forall  a b, FinitaryProjection a  ->  FinitaryProjection b  ->
      FinitaryProjection (binOp s a b)  /\  FinitaryProjection (binOp t a b))
    {a b : ApproximableMap E E} (ha : FinitaryProjection a) (hb : FinitaryProjection b
    )
    (hpreA : binOp s a b  <=  a) (hpreB : binOp t a b  <=  b) :
    fixedNbhd (aStar s t) ? fixedNbhd a  /\  fixedNbhd (bStar s t) ? fixedNbhd b := by
  have h1 : s.toElementMap (pair (toFilter a) (toFilter b))  <=  toFilter a := by
    have hpreA' : toApproxMap (s.toElementMap (pair (toFilter a) (toFilter b)))  <=
        toApproxMap (toFilter a) := by
      show binOp s a b  <=  toApproxMap (toFilter a)
      rw [toApproxMap_toFilter]
      exact hpreA
    have hiff := (funSpaceEquiv E E).le_iff_le
      (x := s.toElementMap (pair (toFilter a) (toFilter b))) (y := toFilter a)
    rw [funSpaceEquiv_apply, funSpaceEquiv_apply] at hiff
    exact hiff.mp hpreA'
  have h2 : t.toElementMap (pair (toFilter a) (toFilter b))  <=  toFilter b := by
    have hpreB' : toApproxMap (t.toElementMap (pair (toFilter a) (toFilter b)))  <=
        toApproxMap (toFilter b) := by
      show binOp t a b  <=  toApproxMap (toFilter b)
      rw [toApproxMap_toFilter]
```

```
      exact hpreB
    have hiff := (funSpaceEquiv E E).le_iff_le
      (x := t.toElementMap (pair (toFilter a) (toFilter b))) (y := toFilter b)
    rw [funSpaceEquiv_apply, funSpaceEquiv_apply] at hiff
    exact hiff.mp hpreB'
  have hle : (paired s t).toElementMap (pair (toFilter a) (toFilter b))  <=
     pair (toFilter a) (toFilter b) := by
    rw [toElementMap_paired]
    exact pair_le_pair_iff.mpr <h1, h2>
  have hfix : (paired s t).fixElement  <=  pair (toFilter a) (toFilter b) :=
    fixElement_le_of_toElementMap_le (paired s t) hle
  have hfstle : (paired s t).fixElement.fst  <=  toFilter a := by
    have := fst_mono hfix; rwa [fst_pair] at this
  have hsndle : (paired s t).fixElement.snd  <=  toFilter b := by
    have := snd_mono hfix; rwa [snd_pair] at this
  have hAle : aStar s t  <=  a := by
    show toApproxMap (paired s t).fixElement.fst  <=  a
    rw [ <-  toApproxMap_toFilter a]
    exact toApproxMap_monotone hfstle
  have hBle : bStar s t  <=  b := by
    show toApproxMap (paired s t).fixElement.snd  <=  b
    rw [ <-  toApproxMap_toFilter b]
    exact toApproxMap_monotone hsndle
  obtain <hFinA, hFinB> := isFinitaryProjection_aStar_bStar s t hst
  exact <(isFinitaryProjection_le_iff_fixedNbhd_subsystem hFinA ha).mp hAle,
    (isFinitaryProjection_le_iff_fixedNbhd_subsystem hFinB hb).mp hBle>

theorem toElementMap_fixElement_fst (s t : ApproximableMap (PairSpace (E := E)) (
    funSpace E E)) :
    s.toElementMap (paired s t).fixElement = (paired s t).fixElement.fst := by
  have h : (paired s t).toElementMap (paired s t).fixElement = (paired s t).fixElement
     :=
    toElementMap_fixElement (paired s t)
  rw [toElementMap_paired] at h
  have h' := congrArg NeighborhoodSystem.Element.fst h
  rwa [fst_pair] at h'

/-- `(aStar s t, bStar s t)` is a *genuine* fixed point of the pair `(s, t)`. -/
theorem binOp_aStar_bStar_left (s t : ApproximableMap (PairSpace (E := E)) (funSpace E
     E)) :
    binOp s (aStar s t) (bStar s t) = aStar s t := by
  unfold binOp aStar bStar
  rw [toFilter_toApproxMap, toFilter_toApproxMap, pair_fst_snd,
    toElementMap_fixElement_fst]

theorem toElementMap_fixElement_snd (s t : ApproximableMap (PairSpace (E := E)) (
    funSpace E E)) :
    t.toElementMap (paired s t).fixElement = (paired s t).fixElement.snd := by
  have h : (paired s t).toElementMap (paired s t).fixElement = (paired s t).fixElement
     :=
    toElementMap_fixElement (paired s t)
  rw [toElementMap_paired] at h
  have h' := congrArg NeighborhoodSystem.Element.snd h
  rwa [snd_pair] at h'

/-- `(aStar s t, bStar s t)` is a *genuine* fixed point of the pair `(s, t)`. -/
```

```
theorem binOp_aStar_bStar_right (s t : ApproximableMap (PairSpace (E := E)) (funSpace
    E E)) :
    binOp t (aStar s t) (bStar s t) = bStar s t := by
  unfold binOp aStar bStar
  rw [toFilter_toApproxMap, toFilter_toApproxMap, pair_fst_snd,
    toElementMap_fixElement_snd]

/-- **Exercise 8.24, Claim 2 (the domain equations).** Given any correspondences `S, T`
     (bundling,
for each pair of maps, a type and a domain over it) with `Fix(binOp s a b)  ~=  S(a,b)`
     and
`Fix(binOp t a b)  ~=  T(a,b)` for every pair of finitary projections `a, b` (Scott's
    hypotheses
`D_{s(a,b)}  ~=  S(D_a,D_b)`, `D_{t(a,b)}  ~=  T(D_a,D_b)`), substituting the *exact*
    simultaneous fixed
point `a := aStar s t`, `b := bStar s t` and using `binOp_aStar_bStar_left/right` gives
`Fix(aStar s t)  ~=  S(aStar s t, bStar s t)` and `Fix(bStar s t)  ~=  T(aStar s t,
    bStar s t)`
unconditionally -- Scott's `D  ~=  S(D,E)`, `E  ~=  T(D,E)`. -/
theorem fixedDomain_aStar_bStar_iso (s t : ApproximableMap (PairSpace (E := E)) (
    funSpace E E))
    (hst : forall  a b, FinitaryProjection a  ->  FinitaryProjection b  ->
      FinitaryProjection (binOp s a b)  /\  FinitaryProjection (binOp t a b))
    (S T : ApproximableMap E E  ->  ApproximableMap E E  ->  Sigma b : Type,
    NeighborhoodSystem b)
    (hS : forall  a b, FinitaryProjection a  ->  FinitaryProjection b  ->
      Nonempty (Fix (binOp s a b) ?o (S a b).2.Element))
    (hT : forall  a b, FinitaryProjection a  ->  FinitaryProjection b  ->
      Nonempty (Fix (binOp t a b) ?o (T a b).2.Element)) :
    Nonempty (Fix (aStar s t) ?o (S (aStar s t) (bStar s t)).2.Element)  /\
      Nonempty (Fix (bStar s t) ?o (T (aStar s t) (bStar s t)).2.Element) := by
  obtain <hFinA, hFinB> := isFinitaryProjection_aStar_bStar s t hst
  have h1 := hS (aStar s t) (bStar s t) hFinA hFinB
  have h2 := hT (aStar s t) (bStar s t) hFinA hFinB
  rw [binOp_aStar_bStar_left] at h1
  rw [binOp_aStar_bStar_right] at h2
  exact <h1, h2>

end Exercise824

end DomainTheory.NeighbourhoodSystems

namespace DomainTheory.NeighbourhoodSystems.Example15

end DomainTheory.NeighbourhoodSystems.Example15
namespace DomainTheory.NeighbourhoodSystems.Exercise722
/-- Countability of the neighbourhoods of `S`, via the `SExpr` presentation.
Picking a preimage uses Mathlib's `Surjective.countable` (hence `Classical.choice`);
the syntax `SExpr` itself is countable choice-free. -/
instance Ssys_countable : Countable {S : Set (List Bool) // Ssys.mem S} := by
  classical
  let f (e : SExpr) : {S : Set (List Bool) // InS S} :=
    if h : (denote e).Nonempty then <denote e, InS_denote_of_nonempty h>
    else <Set.univ, InS.univ>
  have : Countable {S : Set (List Bool) // InS S} :=
    Function.Surjective.countable (f := f) fun <X, hX> => by
```

```
      obtain <e, he> := InS_exists_denote hX
      refine <e, ?_>
      have hne : (denote e).Nonempty := he.symm  |>  hX.nonempty
      dsimp [f]
      rw [dite_eq_left hne]
      exact Subtype.ext he
  exact Countable.of_equiv ({S : Set (List Bool) // InS S})
    (Equiv.subtypeEquivRight fun _ => Ssys_mem.symm)

end DomainTheory.NeighbourhoodSystems.Exercise722
namespace DomainTheory.NeighbourhoodSystems.Example15
/-- **New corollary for Example 1.5.** The finite powerset system is countable, so the
     first
sentence of Theorem 8.8 applies. Scott does not separately state this embedding. -/
theorem P4_embeds : neighborhoodSystem ? U :=
  theorem_8_8 neighborhoodSystem

end DomainTheory.NeighbourhoodSystems.Example15
namespace DomainTheory.NeighbourhoodSystems.Exercise722
/-- **New corollary for Exercise 7.22.** Countability is `Ssys_countable` (via `SExpr`)
    , so the
first sentence of Theorem 8.8 applies. Scott does not separately state this embedding.
     -/
theorem Ssys_embeds : Ssys ? U :=
  theorem_8_8 Ssys

end DomainTheory.NeighbourhoodSystems.Exercise722
```

The tip then assembles those tools into Scott's statement.

— exercise_8_24

Exercise_8_24.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    CoupledFixedPoints

@[expose] public section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap Exercise824

/-- **Exercise 8.24 (Scott 1981, PRG-19).** The coupled fixed-point projections `a?`, `
    b?` are finitary. -/
theorem exercise_8_24 {a : Type} {E : NeighborhoodSystem a}
    (s t : ApproximableMap (PairSpace (E := E)) (funSpace E E))
    (hst : forall  a b, FinitaryProjection a  ->  FinitaryProjection b  ->
      FinitaryProjection (binOp s a b)  /\  FinitaryProjection (binOp t a b)) :
    FinitaryProjection (aStar s t)  /\  FinitaryProjection (bStar s t) :=
  isFinitaryProjection_aStar_bStar s t hst

end
```

3.9.27 Exercise 8.25 — reflexive function space

The problem is to find non-trivial solutions to the domain equation

$$(\spadesuit) \qquad D \cong D \to D.$$

Show that the "obvious" solution by retracts is of no use because

$$1 \to 1 = 1$$

for projections. Change the method as follows. Show first

$$U^\infty \times U^\infty \cong U^\infty.$$

Next solve

$$D \cong D \to U^\infty$$

and remark that $U \lhd D$; so D is universal and non-trivial. Finally prove $(\spadesuit)$ for this D. (Hint: First show

$$D \times D \cong D,$$

and then show D satisfies $(\spadesuit)$.) Is this D effectively given?

Tip summary The tip asserts existence of a non-trivial domain isomorphic to its function space and containing `U`.

Development Rational-interval coding used by effective presentations.

QPairCodes.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.ratCodes
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    RecursiveCross
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.U
public import DomainTheory.NeighbourhoodSystems.DomainEquations.Exercise_6_29
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    ExampleC_le_B
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Proposition_7_12

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

open Domain.Recursive

/-- Decode a natural as a pair of rationals. -/
def decodeRatPair (c : Nat) : ?  x  ? := (decodeRat c.unpair.1, decodeRat c.unpair.2)

/-- Encode a pair of rationals as a single natural. -/
def encodeRatPair (p : ?  x  ?) : Nat := Nat.pair (encodeRat p.1) (encodeRat p.2)
```

```
@[simp] theorem decodeRatPair_encodeRatPair (p : ?  x  ?) : decodeRatPair (
    encodeRatPair p) = p := by
  unfold decodeRatPair encodeRatPair
  simp only [unpair_pair_fst, unpair_pair_snd, decodeRat_encodeRat]

/-- Decode a natural as a list of rational pairs. -/
def decodeQPairList (c : Nat) : List (?  x  ?) := (decodeList c).map decodeRatPair

/-- Encode a list of rational pairs as a single natural, via `encodeList`. -/
def encodeQPairList (L : List (?  x  ?)) : Nat := encodeList (L.map encodeRatPair)

@[simp] theorem decodeQPairList_encodeQPairList (L : List (?  x  ?)) :
    decodeQPairList (encodeQPairList L) = L := by
  unfold decodeQPairList encodeQPairList
  rw [decodeList_encodeList, List.map_map]
  simp only [Function.comp_def, decodeRatPair_encodeRatPair, List.map_id_fun', id]

/-- Code-level `diffOneList`, taking `t = pair qc pc` (subtracted interval `q` **first
    **, matching
`flatMapCode`'s calling convention). -/
def diffOneCode (t : Nat) : Nat :=
  Nat.pair
    (Nat.pair t.unpair.2.unpair.1 (ratMinCode (Nat.pair t.unpair.2.unpair.2 t.unpair
    .1.unpair.1)))
    (Nat.pair (Nat.pair (ratMaxCode (Nat.pair t.unpair.2.unpair.1 t.unpair.1.unpair.2)
    )
      t.unpair.2.unpair.2) 0 + 1) + 1

/-- `diffSingleCode accL q` subtracts the single interval-pair coded by `q` from the
    whole
QPair-list coded by `accL`. -/
def diffSingleCode (accL q : Nat) : Nat := flatMapCode diffOneCode q accL

/-- `foldCode`'s step for `diffAllCode`'s fold over `L2` (params unused): pops `q`,
    subtracts it
from the "remaining pieces" accumulator via `diffSingleCode`. -/
def diffAllStep (w : Nat) : Nat := diffSingleCode w.unpair.2.unpair.1 w.unpair.1

/-- `diffAllCode pc L2` subtracts the whole QPair-list coded by `L2` from the single
    interval-pair
coded by `pc`, folding `diffSingleCode` over `L2` from the seed `[pc]`. -/
def diffAllCode (pc L2 : Nat) : Nat := foldCode diffAllStep 0 (Nat.pair pc 0 + 1) L2

/-- `diffCode c1 c2` subtracts the QPair-list coded by `c2` from the QPair-list coded
    by `c1`. -/
def diffCode (c1 c2 : Nat) : Nat := flatMapCode (fun t => diffAllCode t.unpair.2 t.
    unpair.1) c2 c1

/-- The per-pair-code operation `(pc, qc)  |->  (max, min)` realizing `(p.1  sqsup  q
    .1, p.2  sqcap  q.2)`. -/
def qpCombineBop (t : Nat) : Nat :=
  Nat.pair (ratMaxCode (Nat.pair t.unpair.1.unpair.1 t.unpair.2.unpair.1))
    (ratMinCode (Nat.pair t.unpair.1.unpair.2 t.unpair.2.unpair.2))

/-- `combineCode` realizes `combineIntervals` on codes: `Nat.Primrec` in both arguments
    . -/
```

```
def combineCode (c1 c2 : Nat) : Nat := crossCombine qpCombineBop c1 c2

/-- The `{0,1}` flag: does the rational-pair code `e` decode to a non-degenerate pair `
    p.1 < p.2`? -/
def qpNonemptyBop (e : Nat) : Nat := ratLtCode (Nat.pair e.unpair.1 e.unpair.2)

/-- The `{0,1}` flag: does the QPair-list code `c` present a non-empty set? A bounded `
    exists ` over
`decodeList c` via `Recursive.lean`'s `existsListChar` (the second argument of `
    existsListChar` is
an unused parameter, fixed to `0`). -/
def qpNonemptyChar (c : Nat) : Nat := existsListChar (fun t => qpNonemptyBop t.unpair
    .1) 0 c

theorem decodeQPairList_diffOneCode (qc pc : Nat) :
    decodeQPairList (diffOneCode (Nat.pair qc pc)) = diffOneList (decodeRatPair pc) (
    decodeRatPair qc) := by
  unfold diffOneCode decodeQPairList diffOneList decodeRatPair
  simp only [unpair_pair_fst, unpair_pair_snd, decodeList_succ, decodeList_zero, List.
    map_cons,
    List.map_nil]
  rw [decodeRat_ratMinCode, decodeRat_ratMaxCode]

/-- **`flatMapCode`'s membership correctness, lifted to `decodeQPairList`.** -/
theorem mem_decodeQPairList_flatMapCode (f : Nat  ->  Nat) (x c : Nat) (p : ?  x  ?) :
    p  in  decodeQPairList (flatMapCode f x c)  <->  exists  y  in  decodeList c, p
    in  decodeQPairList (f (Nat.pair x y)) := by
  unfold decodeQPairList
  rw [List.mem_map]
  constructor
  * rintro <z, hz, rfl>
    rw [mem_decodeList_flatMapCode] at hz
    obtain <y, hy, hz'> := hz
    exact <y, hy, List.mem_map_of_mem hz'>
  * rintro <y, hy, hp>
    obtain <e, he, rfl> := List.mem_map.mp hp
    exact <e, (mem_decodeList_flatMapCode (f := f) x c e).mpr <y, hy, he>, rfl>

theorem mem_decodeQPairList_diffSingleCode (accL q : Nat) (p : ?  x  ?) :
    p  in  decodeQPairList (diffSingleCode accL q)  <->  p  in  diffSingleList (
    decodeQPairList accL) (decodeRatPair q) := by
  unfold diffSingleCode
  rw [mem_decodeQPairList_flatMapCode]
  simp only [decodeQPairList_diffOneCode]
  unfold diffSingleList decodeQPairList
  simp only [List.mem_flatMap, List.mem_map]
  constructor
  * rintro <y, hy, hp>; exact <decodeRatPair y, <y, hy, rfl>, hp>
  * rintro <p', <y, hy, rfl>, hp>; exact <y, hy, hp>

theorem presentedIntervals_decodeQPairList_diffSingleCode (accL q : Nat) :
    presentedIntervals (decodeQPairList (diffSingleCode accL q))
      = presentedIntervals (decodeQPairList accL) \ Set.Ico (decodeRatPair q).1 (
    decodeRatPair q).2 := by
  rw [presentedIntervals.congr (mem_decodeQPairList_diffSingleCode accL q),
    presentedIntervals.diffSingleList]
```

```
theorem diffAllStep_eq (acc x : Nat) : diffAllStep (Nat.pair x (Nat.pair acc 0)) =
    diffSingleCode acc x := by
  unfold diffAllStep; simp only [unpair_pair_fst, unpair_pair_snd]

theorem presentedIntervals_decodeQPairList_foldl_diffAllStep (L2list : List Nat) :
    forall  acc : Nat, presentedIntervals (decodeQPairList
        (List.foldl (fun acc x => diffAllStep (Nat.pair x (Nat.pair acc 0))) acc
    L2list))
      = presentedIntervals (decodeQPairList acc) \ presentedIntervals (L2list.map
    decodeRatPair) := by
  induction L2list with
  | nil => intro acc; simp
  | cons a l ih =>
    intro acc
    rw [List.foldl_cons, diffAllStep_eq, ih (diffSingleCode acc a),
      presentedIntervals_decodeQPairList_diffSingleCode, List.map_cons,
    presentedIntervals.cons,
      Set.sdiff_sdiff]

theorem presentedIntervals_decodeQPairList_diffAllCode (pc L2 : Nat) :
    presentedIntervals (decodeQPairList (diffAllCode pc L2))
      = Set.Ico (decodeRatPair pc).1 (decodeRatPair pc).2 \ presentedIntervals (
    decodeQPairList L2) := by
  unfold diffAllCode
  rw [foldCode_eq', presentedIntervals_decodeQPairList_foldl_diffAllStep]
  unfold decodeQPairList
  simp only [decodeList_succ, unpair_pair_fst, unpair_pair_snd, decodeList_zero, List.
    map_cons,
    List.map_nil, presentedIntervals.cons, presentedIntervals.nil, Set.union_empty]

theorem mem_decodeQPairList_diffCode (c1 c2 : Nat) (p : ?  x  ?) :
    p  in  decodeQPairList (diffCode c1 c2)  <->  exists  y  in  decodeList c1, p  in
     decodeQPairList (diffAllCode y c2) := by
  unfold diffCode
  rw [mem_decodeQPairList_flatMapCode]
  refine exists_congr fun y => and_congr_right fun _ => ?_
  simp only [unpair_pair_fst, unpair_pair_snd]

theorem presentedIntervals_decodeQPairList_flatMap_diffAllCode (c1list : List Nat) (c2
     : Nat) :
    presentedIntervals (c1list.flatMap (fun y => decodeQPairList (diffAllCode y c2)))
      = presentedIntervals (c1list.map decodeRatPair) \ presentedIntervals (
    decodeQPairList c2) := by
  induction c1list with
  | nil => simp
  | cons a l ih =>
    rw [List.flatMap_cons, List.map_cons, presentedIntervals.append,
      presentedIntervals_decodeQPairList_diffAllCode, ih, presentedIntervals.cons,
      Set.union_sdiff_distrib]

/-- **`diffCode` realizes `diffLists` (hence set difference) at the set level.** -/
theorem presentedIntervals_decodeQPairList_diffCode (c1 c2 : Nat) :
    presentedIntervals (decodeQPairList (diffCode c1 c2))
      = presentedIntervals (decodeQPairList c1) \ presentedIntervals (decodeQPairList
    c2) := by
```

```
  rw [presentedIntervals.congr
      (fun p => (mem_decodeQPairList_diffCode c1 c2 p).trans (List.mem_flatMap).symm),
    presentedIntervals_decodeQPairList_flatMap_diffAllCode]
  rfl

/-- **A presented union of intervals is non-empty iff some pair in the list is itself
    non-degenerate**
(`p.1 < p.2`): every degenerate pair contributes the empty interval, and a single non-
    degenerate pair
already witnesses non-emptiness (via its left endpoint). -/
theorem presentedIntervals_nonempty_iff (L : List (?  x  ?)) :
    (presentedIntervals L).Nonempty  <->  exists  p  in  L, p.1 < p.2 := by
  constructor
  * rintro <x, hx>
    obtain <p, hp, hx1, hx2> := presentedIntervals.mem.mp hx
    exact <p, hp, lt_of_le_of_lt hx1 hx2>
  * rintro <p, hp, hlt>
    exact <p.1, presentedIntervals.mem.mpr <p, hp, le_refl _, hlt>>

theorem decodeRatPair_qpCombineBop (e1 e2 : Nat) :
    decodeRatPair (qpCombineBop (Nat.pair e1 e2))
      = ((decodeRatPair e1).1  sqsup  (decodeRatPair e2).1, (decodeRatPair e1).2
    sqcap  (decodeRatPair e2).2) := by
  unfold qpCombineBop decodeRatPair
  simp only [unpair_pair_fst, unpair_pair_snd]
  rw [decodeRat_ratMaxCode, decodeRat_ratMinCode]

theorem mem_combineIntervals {L1 L2 : List (?  x  ?)} {p : ?  x  ?} :
    p  in  combineIntervals L1 L2  <->  exists  p1  in  L1, exists  p2  in  L2, p = (
    p1.1  sqsup  p2.1, p1.2  sqcap  p2.2) := by
  unfold combineIntervals
  rw [List.mem_flatMap]
  simp only [List.mem_map, eq_comm]

theorem mem_decodeQPairList (c : Nat) (p : ?  x  ?) :
    p  in  decodeQPairList c  <->  exists  e  in  decodeList c, p = decodeRatPair e :=
     by
  unfold decodeQPairList; rw [List.mem_map]; simp [eq_comm]

theorem mem_decodeQPairList_combineCode (c1 c2 : Nat) (p : ?  x  ?) :
    p  in  decodeQPairList (combineCode c1 c2)  <->
      exists  p1  in  decodeQPairList c1, exists  p2  in  decodeQPairList c2, p = (p1
    .1  sqsup  p2.1, p1.2  sqcap  p2.2) := by
  unfold decodeQPairList combineCode
  rw [List.mem_map]
  constructor
  * rintro <z, hz, hpz>
    rw [mem_decodeList_crossCombine] at hz
    obtain <e1, he1, e2, he2, hze> := hz
    refine <decodeRatPair e1, List.mem_map_of_mem he1, decodeRatPair e2, List.
    mem_map_of_mem he2, ?_>
    rw [ <-  hpz, hze, decodeRatPair_qpCombineBop]
  * rintro <p1, hp1, p2, hp2, hp>
    obtain <e1, he1, he1'> := List.mem_map.mp hp1
    obtain <e2, he2, he2'> := List.mem_map.mp hp2
    refine <qpCombineBop (Nat.pair e1 e2),
```

```
      (mem_decodeList_crossCombine (bop := qpCombineBop) c1 c2 _).mpr <e1, he1, e2,
    he2, rfl>, ?_>
    rw [decodeRatPair_qpCombineBop, he1', he2', hp]

/-- **`combineCode` realizes `combineIntervals` at the set level.** -/
theorem presentedIntervals_decodeQPairList_combineCode (c1 c2 : Nat) :
    presentedIntervals (decodeQPairList (combineCode c1 c2))
      = presentedIntervals (decodeQPairList c1)  I  presentedIntervals (
    decodeQPairList c2) := by
  rw [presentedIntervals.inter]
  apply presentedIntervals.congr
  intro p
  rw [mem_decodeQPairList_combineCode, mem_combineIntervals]

/-- **Subset between `presentedIntervals`-coded sets reduces to non-emptiness of `
    diffCode`.** -/
theorem presentedIntervals_decodeQPairList_subset_iff (c1 c2 : Nat) :
    presentedIntervals (decodeQPairList c1)  subseteq  presentedIntervals (
    decodeQPairList c2)  <->
      not  (presentedIntervals (decodeQPairList (diffCode c1 c2))).Nonempty := by
  rw [presentedIntervals_decodeQPairList_diffCode, Set.not_nonempty_iff_eq_empty, Set.
    sdiff_eq_empty]

theorem primrec_diffOneCode : Nat.Primrec diffOneCode := by
  have ha : Nat.Primrec (fun t : Nat => t.unpair.2.unpair.1) := Nat.Primrec.left.comp
    Nat.Primrec.right
  have hb : Nat.Primrec (fun t : Nat => t.unpair.2.unpair.2) := Nat.Primrec.right.comp
     Nat.Primrec.right
  have hc : Nat.Primrec (fun t : Nat => t.unpair.1.unpair.1) := Nat.Primrec.left.comp
    Nat.Primrec.left
  have hd : Nat.Primrec (fun t : Nat => t.unpair.1.unpair.2) := Nat.Primrec.right.comp
     Nat.Primrec.left
  have hleft : Nat.Primrec (fun t : Nat =>
      Nat.pair t.unpair.2.unpair.1 (ratMinCode (Nat.pair t.unpair.2.unpair.2 t.unpair
    .1.unpair.1))) :=
    ha.pair (primrec_ratMinCode.comp (hb.pair hc))
  have hright : Nat.Primrec (fun t : Nat =>
      Nat.pair (ratMaxCode (Nat.pair t.unpair.2.unpair.1 t.unpair.1.unpair.2)) t.
    unpair.2.unpair.2) :=
    (primrec_ratMaxCode.comp (ha.pair hd)).pair hb
  exact (Nat.Primrec.succ.comp (hleft.pair
    (Nat.Primrec.succ.comp (hright.pair (Nat.Primrec.const 0))))).of_eq fun _ => rfl

theorem primrec_diffSingleCode : Nat.Primrec (fun t : Nat => diffSingleCode t.unpair.1
     t.unpair.2) := by
  unfold diffSingleCode
  exact ((primrec_flatMapCode primrec_diffOneCode).comp (Nat.Primrec.right.pair Nat.
    Primrec.left)).of_eq
    fun t => by simp only [unpair_pair_fst, unpair_pair_snd]

theorem primrec_diffAllStep : Nat.Primrec diffAllStep := by
  unfold diffAllStep
  have hacc : Nat.Primrec (fun w : Nat => w.unpair.2.unpair.1) := Nat.Primrec.left.
    comp Nat.Primrec.right
  have hq : Nat.Primrec (fun w : Nat => w.unpair.1) := Nat.Primrec.left
  exact (primrec_diffSingleCode.comp (hacc.pair hq)).of_eq fun w => by
```

```
    simp only [unpair_pair_fst, unpair_pair_snd]

theorem primrec_diffAllCode : Nat.Primrec (fun t : Nat => diffAllCode t.unpair.1 t.
    unpair.2) :=
  (primrec_foldCode primrec_diffAllStep (Nat.Primrec.const 0)
    (Nat.Primrec.succ.comp (Nat.Primrec.left.pair (Nat.Primrec.const 0)))
    Nat.Primrec.right).of_eq fun _ => rfl

theorem primrec_diffCode : Nat.Primrec (fun t : Nat => diffCode t.unpair.1 t.unpair.2)
     := by
  unfold diffCode
  have hf : Nat.Primrec (fun t : Nat => diffAllCode t.unpair.2 t.unpair.1) :=
    (primrec_diffAllCode.comp (Nat.Primrec.right.pair Nat.Primrec.left)).of_eq fun t
    => by
      simp only [unpair_pair_fst, unpair_pair_snd]
  exact ((primrec_flatMapCode hf).comp (Nat.Primrec.right.pair Nat.Primrec.left)).
    of_eq fun t => by
    simp only [unpair_pair_fst, unpair_pair_snd]

theorem primrec_qpCombineBop : Nat.Primrec qpCombineBop := by
  have h11 : Nat.Primrec (fun t : Nat => t.unpair.1.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.left
  have h12 : Nat.Primrec (fun t : Nat => t.unpair.1.unpair.2) :=
    Nat.Primrec.right.comp Nat.Primrec.left
  have h21 : Nat.Primrec (fun t : Nat => t.unpair.2.unpair.1) :=
    Nat.Primrec.left.comp Nat.Primrec.right
  have h22 : Nat.Primrec (fun t : Nat => t.unpair.2.unpair.2) :=
    Nat.Primrec.right.comp Nat.Primrec.right
  exact ((primrec_ratMaxCode.comp (h11.pair h21)).pair
    (primrec_ratMinCode.comp (h12.pair h22))).of_eq fun _ => rfl

theorem primrec_combineCode : Nat.Primrec (fun t => combineCode t.unpair.1 t.unpair.2)
     :=
  primrec_crossCombine primrec_qpCombineBop

theorem primrec_qpNonemptyBop : Nat.Primrec qpNonemptyBop :=
  primrec_ratLtCode.comp (Nat.Primrec.left.pair Nat.Primrec.right)

theorem primrec_qpNonemptyChar : Nat.Primrec qpNonemptyChar := by
  have hg : Nat.Primrec (fun t : Nat => qpNonemptyBop t.unpair.1) :=
    primrec_qpNonemptyBop.comp Nat.Primrec.left
  exact ((primrec_existsListChar hg).comp (primrec_id.pair (Nat.Primrec.const 0))).
    of_eq
    fun c => by
      unfold qpNonemptyChar
      simp only [id, unpair_pair_fst, unpair_pair_snd]

theorem qpNonemptyBop_eq_one_iff (e : Nat) :
    qpNonemptyBop e = 1  <->  (decodeRatPair e).1 < (decodeRatPair e).2 := by
  unfold qpNonemptyBop decodeRatPair; exact ratLtCode_eq_one_iff e.unpair.1 e.unpair.2

theorem qpNonemptyChar_eq_one_iff (c : Nat) :
    qpNonemptyChar c = 1  <->  exists  e  in  decodeList c, (decodeRatPair e).1 < (
    decodeRatPair e).2 := by
  unfold qpNonemptyChar
  rw [existsListChar_eq_one_iff]
```

```
  refine exists_congr fun e => and_congr_right fun _ => ?_
  rw [unpair_pair_fst, qpNonemptyBop_eq_one_iff]

/-- **`presentedIntervals (decodeQPairList c)` is non-empty iff `qpNonemptyChar c =
    1`.** -/
theorem presentedIntervals_decodeQPairList_nonempty_iff (c : Nat) :
    (presentedIntervals (decodeQPairList c)).Nonempty  <->  qpNonemptyChar c = 1 := by
  rw [presentedIntervals_nonempty_iff, qpNonemptyChar_eq_one_iff]
  constructor
  * rintro <p, hp, hlt>
    obtain <e, he, rfl> := (mem_decodeQPairList c p).mp hp
    exact <e, he, hlt>
  * rintro <e, he, hlt>
    exact <decodeRatPair e, (mem_decodeQPairList c (decodeRatPair e)).mpr <e, he, rfl
    >, hlt>

theorem qpNonemptyChar_le_one (c : Nat) : qpNonemptyChar c  <=  1 :=
    existsListChar_le_one _ 0 c

/-- **Non-emptiness of the set presented by a QPair-list code is recursively decidable
    .** -/
theorem recDecidable_presentedIntervals_nonempty :
    RecDecidable (fun c => (presentedIntervals (decodeQPairList c)).Nonempty) :=
  RecDecidable.of_zero_one_char primrec_qpNonemptyChar
    (fun c => by have := qpNonemptyChar_le_one c; omega)
    presentedIntervals_decodeQPairList_nonempty_iff

/-- **Subset between `presentedIntervals`-coded sets is recursively decidable.** -/
theorem recDecidable_2_presentedIntervals_subset :
    RecDecidable_2 (fun c1 c2 =>
      presentedIntervals (decodeQPairList c1)  subseteq  presentedIntervals (
    decodeQPairList c2)) := by
  unfold RecDecidable_2
  refine RecDecidable.of_iff (fun t => ?_)
    (recDecidable_presentedIntervals_nonempty.comp primrec_diffCode).not
  exact presentedIntervals_decodeQPairList_subset_iff t.unpair.1 t.unpair.2

/-- **Equality between `presentedIntervals`-coded sets is recursively decidable**, from
`recDecidable_2_presentedIntervals_subset` in both directions (`Set.Subset.antisymm_iff
    `). -/
theorem recDecidable_2_presentedIntervals_eq :
    RecDecidable_2 (fun c1 c2 =>
      presentedIntervals (decodeQPairList c1) = presentedIntervals (decodeQPairList c2
    )) := by
  unfold RecDecidable_2
  refine RecDecidable.of_iff (fun t => ?_)
    (recDecidable_2_presentedIntervals_subset.and
    recDecidable_2_presentedIntervals_subset.swap)
  exact Set.Subset.antisymm_iff

theorem mem_decodeQPairList_appendCode (c1 c2 : Nat) (p : ?  x  ?) :
    p  in  decodeQPairList (Domain.Recursive.appendCode c1 c2)  <->  p  in
    decodeQPairList c1  \/  p  in  decodeQPairList c2 := by
  simp only [mem_decodeQPairList, mem_decodeList_appendCode]
  constructor
  * rintro <v, (hv | hv), rfl>
```

```
    * exact Or.inl <v, hv, rfl>
    * exact Or.inr <v, hv, rfl>
  * rintro (<v, hv, rfl> | <v, hv, rfl>)
    * exact <v, Or.inl hv, rfl>
    * exact <v, Or.inr hv, rfl>

theorem presentedIntervals_decodeQPairList_appendCode (c1 c2 : Nat) :
    presentedIntervals (decodeQPairList (Domain.Recursive.appendCode c1 c2))
      = presentedIntervals (decodeQPairList c1)  U  presentedIntervals (
    decodeQPairList c2) := by
  ext x
  simp only [presentedIntervals.mem, Set.mem_union]
  constructor
  * rintro <p, hp, h1, h2>
    rcases (mem_decodeQPairList_appendCode c1 c2 p).mp hp with hp' | hp'
    * exact Or.inl <p, hp', h1, h2>
    * exact Or.inr <p, hp', h1, h2>
  * rintro (<p, hp, h1, h2> | <p, hp, h1, h2>)
    * exact <p, (mem_decodeQPairList_appendCode c1 c2 p).mpr (Or.inl hp), h1, h2>
    * exact <p, (mem_decodeQPairList_appendCode c1 c2 p).mpr (Or.inr hp), h1, h2>

end DomainTheory.NeighbourhoodSystems
```

Builds a non-trivial reflexive domain $D \cong D \to D$ containing `U`.

ReflexiveDomain.lean

```
module

public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Definition_7_1

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.U
public import DomainTheory.NeighbourhoodSystems.DomainEquations.Exercise_6_29

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    EmbedIntoPow
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    ExampleC_le_B
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Proposition_7_12

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.ASharp
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    ASharp_more
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.FixOp
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    CoupledFixedPoints
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    EmbeddingsIntoU
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    SumProdRetracts
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Proposition_8_10
public import DomainTheory.NeighbourhoodSystems.ComputabilityInEffectivelyGivenDomains
    .Exercise_7_15
```

```
public import DomainTheory.NeighbourhoodSystems.TypedLambdaCalculus.Table_5_5
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems
section
open NeighborhoodSystem ApproximableMap DomainTheory.NeighbourhoodSystems.Exercise511
variable {a : Type*} (V : NeighborhoodSystem a)
/- concordance: source L5224 EXERCISE 8.25 reflexive function space -/
/-- **Exercise 8.25 (Scott 1981, PRG-19): `? ? ?^ inf `**, for every `?`. Specialized
    to `? = ?` this is
Scott's remark `? ? ?^ inf `. -/
theorem trianglelefteq_iterSys : V ? iterSys V :=
  trianglelefteq_of_projectionPair (embedIntoPow V) (head V)
    (head_comp_embedIntoPow V) (embedIntoPow_comp_head_le V)
end

set_option linter.unusedVariables false

theorem example_825_embed.{u_1} : forall  {a : Type u_1} (V : DomainTheory.
    NeighbourhoodSystems.NeighborhoodSystem a),
  @DomainTheory.NeighbourhoodSystems.Trianglelefteq a (Prod Nat a) V (@DomainTheory.
    NeighbourhoodSystems.iterSys a V) :=
  @DomainTheory.NeighbourhoodSystems.trianglelefteq_iterSys

section
open NeighborhoodSystem
variable {a : Type*}

/-- Reindexing a non-dependent `?  ->  E` arrow type along a bijection of index types
    is an order
isomorphism, for the pointwise order on both sides. (Named distinctly from `
    Exercise324Iter.lean`'s
`piCongrOrderIso`, which reindexes *dependent* `Pi` types along a fixed index type
    instead.) -/
def arrowReindexOrderIso {? ?' : Type*} (e : ? ? ?') (E : Type*) [Preorder E] :
    (?  ->  E) ?o (?'  ->  E) where
  toFun f i' := f (e.symm i')
  invFun g i := g (e i)
  left_inv f := by funext i; simp
  right_inv g := by funext i'; simp
  map_rel_iff' := by
    intro f g
    constructor
    * intro h i
      simpa using h (e i)
    * intro h i'
      simpa using h (e.symm i')

/-- The pointwise order on `a ? b  ->  E` corresponds, coordinatewise, to the product
    order on
`(a  ->  E)  x  (b  ->  E)`: splitting a sum-indexed family into its two summand-
    indexed restrictions is an
order isomorphism. -/
```

```
def sumArrowOrderIso (a b : Type*) (E : Type*) [Preorder E] :
    (a ? b  ->  E) ?o (a  ->  E)  x  (b  ->  E) where
  toFun f := (fun a => f (Sum.inl a), fun b => f (Sum.inr b))
  invFun p := Sum.elim p.1 p.2
  left_inv f := by funext i; cases i <;> rfl
  right_inv p := rfl
  map_rel_iff' := by
    intro f g
    constructor
    * rintro <h1, h2> i
      cases i with
      | inl a => exact h1 a
      | inr b => exact h2 b
    * intro h
      exact <fun a => h (Sum.inl a), fun b => h (Sum.inr b)>

/-- **Interleaving.** Two `Nat`-indexed sequences of `E` can be merged, order-
    isomorphically, into a
single `Nat`-indexed sequence, via the pairing bijection `Nat ? Nat ? Nat`. -/
def interleaveOrderIso (E : Type*) [Preorder E] : (Nat  ->  E)  x  (Nat  ->  E) ?o (
    Nat  ->  E) :=
  (sumArrowOrderIso Nat Nat E).symm.trans (arrowReindexOrderIso Equiv.
    natSumNatEquivNat E)

/-- The order isomorphism `|?^ inf   x  ?^ inf | ?o |?^ inf |` underlying `
    pow_prod_isomorphic`. -/
def powProdOrderIso (V : NeighborhoodSystem a) :
    (prod (iterSys V) (iterSys V)).Element ?o (iterSys V).Element :=
  (prodEquiv (iterSys V) (iterSys V)).trans <|
    (prodCongrOrderIso (iterSeqEquiv V) (iterSeqEquiv V)).trans <|
      (interleaveOrderIso V.Element).trans (iterSeqEquiv V).symm

/-- **Exercise 8.25 (Scott 1981, PRG-19), first step: `?^ inf   x  ?^ inf   ~=  ?^ inf
    `**, for any `?`. Specialized
to `? = ?` this is Scott's `?^ inf   x  ?^ inf   ~=  ?^ inf `. -/
theorem pow_prod_isomorphic (V : NeighborhoodSystem a) :
    prod (iterSys V) (iterSys V)  ~= ? iterSys V :=
  <powProdOrderIso V>

end

set_option linter.unusedVariables false

theorem example_825_pow.{u_1} : forall  {a : Type u_1} (V : DomainTheory.
    NeighbourhoodSystems.NeighborhoodSystem a),
  @DomainTheory.NeighbourhoodSystems.Isomorphic (Sum (Prod Nat a) (Prod Nat a)) (Prod
    Nat a)
    (@DomainTheory.NeighbourhoodSystems.prod (Prod Nat a) (Prod Nat a) (@DomainTheory.
    NeighbourhoodSystems.iterSys a V)
      (@DomainTheory.NeighbourhoodSystems.iterSys a V))
    (@DomainTheory.NeighbourhoodSystems.iterSys a V) :=
  @DomainTheory.NeighbourhoodSystems.pow_prod_isomorphic

section
open NeighborhoodSystem ApproximableMap
variable {a b : Type*}
```

```
/-- **Any two `NeighborhoodSystem`s with `Unique` element posets are isomorphic.** Both
     sides are
order-isomorphic to the one-point poset `PUnit`, so the isomorphism is forced. -/
def isoOfUnique (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b)
    [Unique V_0.Element] [Unique V_1.Element] : V_0.Element ?o V_1.Element where
  toFun _ := default
  invFun _ := default
  left_inv x := (Unique.eq_default x).symm
  right_inv y := (Unique.eq_default y).symm
  map_rel_iff' := by
    intro a b
    exact iff_of_true (le_refl _) (le_of_eq (Subsingleton.elim a b))

/-- **There is only one approximable self-map of `?`, the identity.** Both maps induce
    the same
(unique) elementwise function, so `ext_of_toElementMap` identifies them. -/
instance uniqueApproximableMapUnitSys : Unique (ApproximableMap unitSys unitSys) where
  default := idMap unitSys
  uniq _f := ApproximableMap.ext_of_toElementMap fun _x => Subsingleton.elim _ _

/-- **`(?  ->  ?)` has a unique element**, transported from the unique approximable
    self-map of `?`
along `funSpaceEquiv`. -/
instance uniqueFunSpaceUnitSys : Unique (funSpace unitSys unitSys).Element where
  default := (funSpaceEquiv unitSys unitSys).symm default
  uniq _x := (funSpaceEquiv unitSys unitSys).injective (Subsingleton.elim _ _)

/-- **Any two `NeighborhoodSystem`s with `Unique` element posets determine isomorphic
    domains.** -/
theorem isomorphic_of_unique (V_0 : NeighborhoodSystem a) (V_1 : NeighborhoodSystem b)
    [Unique V_0.Element] [Unique V_1.Element] : V_0  ~= ? V_1 :=
  <isoOfUnique V_0 V_1>

/-- **Exercise 8.25 (Scott 1981, PRG-19) -- the obstruction `?  ->  ? = ?`.** The
    terminal one-point
domain is a fixed point of ` -> `, carrying no information: this is why "the obvious
    solution by
retracts is of no use". -/
theorem funSpace_unitSys_isomorphic : funSpace unitSys unitSys  ~= ? unitSys :=
  isomorphic_of_unique _ _

end

set_option linter.unusedVariables false

theorem example_825_unit : @DomainTheory.NeighbourhoodSystems.Isomorphic
  (@DomainTheory.NeighbourhoodSystems.ApproximableMap Unit Unit DomainTheory.
    NeighbourhoodSystems.unitSys
    DomainTheory.NeighbourhoodSystems.unitSys)
  Unit
  (@DomainTheory.NeighbourhoodSystems.funSpace Unit Unit DomainTheory.
    NeighbourhoodSystems.unitSys
    DomainTheory.NeighbourhoodSystems.unitSys)
  DomainTheory.NeighbourhoodSystems.unitSys :=
  DomainTheory.NeighbourhoodSystems.funSpace_unitSys_isomorphic
```

```
open NeighborhoodSystem ApproximableMap

/-- The domain `(? -> ?)  x  ((? -> ?)  x  ?)` of the joint evaluator: an element `<psi
    , <phi, x>>` packages
the varying map `psi` (to become `a`), the curry variable `phi`, and the point `x`. -/
abbrev RDom := prod (funSpace U U) (prod (funSpace U U) U)

/-- `R(psi, phi, x) := c(phi(psi(x)))`, purely as a composite of `evalMap`, `proj`s and
    `paired` -- jointly
continuous in `(psi, phi, x)` since `evalMap` is. -/
noncomputable def RMap (c : ApproximableMap U U) : ApproximableMap RDom U :=
  c.comp ((evalMap U U).comp
    (paired
      ((proj_0 (funSpace U U) U).comp (proj_1 (funSpace U U) (prod (funSpace U U) U)))
      ((evalMap U U).comp
        (paired (proj_0 (funSpace U U) (prod (funSpace U U) U))
          ((proj_1 (funSpace U U) U).comp (proj_1 (funSpace U U) (prod (funSpace U U)
    U)))))))

/-- First curry (peeling off `psi`), leaving the `(phi, x)` pair bundled: an element of
`? -> ?  ->  funSpace ((? -> ?)  x  ?) ?`. -/
noncomputable def curryOnceC (c : ApproximableMap U U) :
    ApproximableMap (funSpace U U) (funSpace (prod (funSpace U U) U) U) :=
  curry (RMap c)

/-- **`fixedNbhd a  ~=  Fix(a)`, for any finitary projection `a : ?  ->  ?`.** Theorem
    8.6(a)'s
`inj_comp_proj_eq_self` identifies `retractionOfSubsystem (fixedNbhd_subsystem a)` with
    `a` itself,
transporting Proposition 8.2's `elementIso` along the identification. -/
noncomputable def fixedNbhd_elementIso_fix {a : ApproximableMap U U} (ha :
    FinitaryProjection a) :
    (fixedNbhd a).Element ?o {y : U.Element // a.toElementMap y = y} := by
  have h := Subsystem.elementIso (fixedNbhd_subsystem a)
  unfold Subsystem.retractionOfSubsystem at h
  rwa [inj_comp_proj_eq_self ha] at h

/-- Second curry (via Table 5.5's `curryC`, peeling off `phi`): `psi  |->  (phi  |->  (
    x  |->  R(psi,phi,x)))`, an
element of the fourth-order function space `(? -> ?)  ->  ((? -> ?)  ->  (? -> ?))`. -/
noncomputable def lamOp (c : ApproximableMap U U) :
    ApproximableMap (funSpace U U) (funSpace (funSpace U U) (funSpace U U)) :=
  (curryC (funSpace U U) U U).comp (curryOnceC c)

/-- A fixed computable presentation of `?^ inf  = iterSys ?` (Exercise 7.15). -/
noncomputable def presentationIterU : ComputablePresentation (iterSys U) :=
  Classical.choice (iterSys_isEffectivelyGiven U_isEffectivelyGiven)

/-- **`i : ?^ inf   ->  ?`**, fixed by Theorem 8.8(b) applied to `?^ inf `. -/
noncomputable def cInj : ApproximableMap (iterSys U) U :=
  (theorem_8_8_b_strong presentationIterU).choose

/-- **`j : ?  ->  ?^ inf `**, fixed by Theorem 8.8(b) applied to `?^ inf `. -/
noncomputable def cProj : ApproximableMap U (iterSys U) :=
  (theorem_8_8_b_strong presentationIterU).choose_spec.choose
```

```
/-- **`c := i  comp  j : ?  ->  ?`**, the finitary projection witnessing `?^ inf `
    inside `?`. -/
noncomputable def cCombinator : ApproximableMap U U := cInj.comp cProj

/-- Exercise 8.23's abstract correspondence `T(a) := D_a  ->  D_c` (a domain, for each
    finitary
projection `a`). -/
noncomputable def TArrow (a : ApproximableMap U U) : Sigma b : Type,
    NeighborhoodSystem b :=
  <_, funSpace (fixedNbhd a) (fixedNbhd cCombinator)>

theorem cProj_comp_cInj : cProj.comp cInj = idMap (iterSys U) :=
  (theorem_8_8_b_strong presentationIterU).choose_spec.choose_spec.1

/-- **`?^ inf   ~=  Fix(c)`**, from the generic projection-pair `elementIso` (
    Proposition 8.10(b)'s
`elementIsoOfProjectionPair`). -/
noncomputable def cElementIso :
    (iterSys U).Element ?o {y : U.Element // cCombinator.toElementMap y = y} :=
  elementIsoOfProjectionPair cInj cProj cProj_comp_cInj rfl

/-- **`t(a) := a  ->  c`, made continuous in `a`.** Conjugates `lamOp c`'s output by
    the fixed
projection pair `jArrow : ?  ->  (? -> ?)`, `iArrow : (? -> ?)  ->  ?` (Definition 8.9)
    , via `expMap jArrow
iArrow`. -/
noncomputable def tOpMap (c : ApproximableMap U U) :
    ApproximableMap (funSpace U U) (funSpace U U) :=
  (expMap jArrow iArrow).comp (lamOp c)

theorem cInj_comp_cProj_le : cInj.comp cProj  <=  idMap U :=
  (theorem_8_8_b_strong presentationIterU).choose_spec.choose_spec.2.1

theorem isFinitary_cCombinator : IsFinitary cCombinator :=
  isFinitary_of_projectionPair cInj cProj cProj_comp_cInj rfl

theorem isRetraction_cCombinator : Retraction cCombinator := by
  constructor
  show (cInj.comp cProj).comp (cInj.comp cProj) = cInj.comp cProj
  rw [comp_assoc,  <-  comp_assoc cProj cInj cProj, cProj_comp_cInj, idMap_comp]

theorem isProjection_cCombinator : IsProjection cCombinator :=
  <isRetraction_cCombinator, cInj_comp_cProj_le>

theorem isFinitaryProjection_cCombinator : FinitaryProjection cCombinator :=
  <<isProjection_cCombinator, isFinitary_cCombinator>>

/-- **`fixedNbhd c  ~=  ?^ inf `.** -/
theorem fixedNbhd_cCombinator_isomorphic : fixedNbhd cCombinator  ~= ? iterSys U :=
  <(fixedNbhd_elementIso_fix isFinitaryProjection_cCombinator).trans cElementIso.symm>

/-- **`fixedNbhd c  x  fixedNbhd c  ~=  fixedNbhd c`**, transported from `
    Exercise825Pow.pow_prod_isomorphic`
along `fixedNbhd_cCombinator_isomorphic`. -/
theorem hVV_cCombinator :
```

```
    prod (fixedNbhd cCombinator) (fixedNbhd cCombinator)  ~= ? fixedNbhd cCombinator
    :=
  ((fixedNbhd_cCombinator_isomorphic.prod fixedNbhd_cCombinator_isomorphic).trans
    (pow_prod_isomorphic U)).trans fixedNbhd_cCombinator_isomorphic.symm

theorem toElementMap_RMap (c : ApproximableMap U U) (psi phi : (funSpace U U).Element)
     (x : U.Element) :
    (RMap c).toElementMap (pair psi (pair phi x)) =
      c.toElementMap ((toApproxMap phi).toElementMap ((toApproxMap psi).toElementMap x
    )) := by
  simp only [RMap, toElementMap_comp, toElementMap_paired, toElementMap_proj_0,
    toElementMap_proj_1,
    fst_pair, snd_pair, evalMap_apply]

/-- **`lamOp c` recovers `expMap (-) c = lamComb (-) c` at every `a`.** The heart of
    the
continuity argument: unwinding both curry layers (`curryC_toApproxMap`, `
    toElementMap_curry_apply`
twice) and `RMap`'s value formula (`toElementMap_RMap`) against `expMap`'s own value
    formula
(`expMap.toApproxMap_toElementMap`) shows both sides send `phi` to (the element
    representing)
`x  |->  c(phi(a(x)))`. -/
theorem toApproxMap_lamOp (c a : ApproximableMap U U) :
    toApproxMap ((lamOp c).toElementMap (toFilter a)) = expMap a c := by
  apply ApproximableMap.ext_of_toElementMap
  intro phi
  apply (funSpaceEquiv U U).injective
  rw [funSpaceEquiv_apply, funSpaceEquiv_apply]
  apply ApproximableMap.ext_of_toElementMap
  intro x
  show (toApproxMap ((toApproxMap ((lamOp c).toElementMap (toFilter a))).toElementMap
    phi)).toElementMap x
    = (toApproxMap ((expMap a c).toElementMap phi)).toElementMap x
  rw [lamOp, toElementMap_comp, curryC_toApproxMap, toElementMap_curry_apply,
    curryOnceC,
    toElementMap_curry_apply, toElementMap_RMap, Sub8_6.toApproxMap_toFilter,
    expMap.toApproxMap_toElementMap, toElementMap_comp, toElementMap_comp]

/-- **`tOp (tOpMap c) a = arrowComb a c`**, for every `a`: `t` really does implement
`a  |->  arrowComb a c`, continuously. -/
theorem tOp_tOpMap (c a : ApproximableMap U U) :
    Exercise823.tOp (tOpMap c) a = arrowComb a c := by
  show toApproxMap ((tOpMap c).toElementMap (toFilter a)) = arrowComb a c
  rw [tOpMap, toElementMap_comp, expMap.toApproxMap_toElementMap, toApproxMap_lamOp,
    expMap.eq_lamComb]
  rfl

theorem hT_TArrow (a : ApproximableMap U U) (ha : FinitaryProjection a) :
    Nonempty (Exercise823.Fix (Exercise823.tOp (tOpMap cCombinator) a) ?o
      (TArrow a).2.Element) := by
  rw [tOp_tOpMap]
  exact <(arrowComb.elementIso a cCombinator ha isFinitaryProjection_cCombinator).symm
    >

/-- **`t` sends finitary projections to finitary projections** (Exercise 8.23's
```

```
    hypothesis `ht`),
via `tOp_tOpMap` and Proposition 8.10(b)'s `arrowComb.finitaryProjection`. -/
theorem ht_tOpMap (c : ApproximableMap U U) (hc : FinitaryProjection c) :
    forall  a, FinitaryProjection a  ->  FinitaryProjection (Exercise823.tOp (tOpMap c
    ) a) := by
  intro a ha
  rw [tOp_tOpMap]
  exact arrowComb.finitaryProjection a c ha hc

/-- **`D  ~=  D  ->  ?^ inf ` for `D := fixedNbhd (fixOp t)`.** Exercise 8.23's `
    fixedDomain_fixOp_iso_T`
gives `Fix(?t?)  ~=  D  ->  D_c`; `fixedNbhd_elementIso_fix` identifies `D`'s own
    element type with
`Fix(?t?)`, and `fixedNbhd_cCombinator_isomorphic` identifies `D_c` with `?^ inf `. -/
theorem Dsol_isomorphic_funSpace_cCombinator :
    fixedNbhd (Exercise823.fixOp (tOpMap cCombinator))  ~= ?
      funSpace (fixedNbhd (Exercise823.fixOp (tOpMap cCombinator))) (fixedNbhd
    cCombinator) := by
  have hht := ht_tOpMap cCombinator isFinitaryProjection_cCombinator
  have hFP := Exercise823.isFinitaryProjection_fixOp (tOpMap cCombinator) hht
  have h1 : Nonempty ((fixedNbhd (Exercise823.fixOp (tOpMap cCombinator))).Element ?o
      Exercise823.Fix (Exercise823.fixOp (tOpMap cCombinator))) :=
    <fixedNbhd_elementIso_fix hFP>
  have h2 := Exercise823.fixedDomain_fixOp_iso_T (tOpMap cCombinator) hht TArrow
    hT_TArrow
  exact h1.elim fun e1 => h2.elim fun e2 => <e1.trans e2>

/-- **`? ? D`**: the solution domain contains a copy of the universal domain `?`, so it
     is
non-trivial (Scott's remark). Chains `? ? ?^ inf ` (`Exercise825Embed.
    trianglelefteq_iterSys`),
`?^ inf   ~=  fixedNbhd c`, the general constant-function embedding `? ? (?  ->  ?)`
(`Exercise820.trianglelefteq_funSpace_const`), and `D  ->  (fixedNbhd c)  ~=  D`. -/
theorem U_trianglelefteq_Dsol : U ? fixedNbhd (Exercise823.fixOp (tOpMap cCombinator))
     := by
  set D := fixedNbhd (Exercise823.fixOp (tOpMap cCombinator)) with hD
  have h1 : U ? iterSys U := trianglelefteq_iterSys U
  have h2 : iterSys U ? fixedNbhd cCombinator :=
    <fixedNbhd cCombinator, Subsystem.refl _, fixedNbhd_cCombinator_isomorphic.symm>
  have h3 : fixedNbhd cCombinator ? funSpace D (fixedNbhd cCombinator) :=
    trianglelefteq_funSpace_const D (fixedNbhd cCombinator)
  have h4 : funSpace D (fixedNbhd cCombinator) ? D :=
    <D, Subsystem.refl D, Dsol_isomorphic_funSpace_cCombinator.symm>
  exact trianglelefteq_trans (trianglelefteq_trans (trianglelefteq_trans h1 h2) h3) h4

/-- **Exercise 8.25 (Scott 1981, PRG-19), main theorem.** `D  ~=  D  ->  D`, for
`D := fixedNbhd (fixOp (tOpMap cCombinator))`: a non-trivial domain isomorphic to its
    own function
space (see `U_trianglelefteq_Dsol` below for non-triviality). -/
theorem exercise_8_25_main :
    fixedNbhd (Exercise823.fixOp (tOpMap cCombinator))  ~= ?
      funSpace (fixedNbhd (Exercise823.fixOp (tOpMap cCombinator)))
        (fixedNbhd (Exercise823.fixOp (tOpMap cCombinator))) :=
  funSpace_self_isomorphic Dsol_isomorphic_funSpace_cCombinator hVV_cCombinator

set_option linter.unusedVariables false
```

```
theorem example_825_fixed_point : @DomainTheory.NeighbourhoodSystems.Isomorphic Rat
  (@DomainTheory.NeighbourhoodSystems.ApproximableMap Rat Rat
    (@DomainTheory.NeighbourhoodSystems.fixedNbhd Rat DomainTheory.
    NeighbourhoodSystems.U
      (@DomainTheory.NeighbourhoodSystems.Exercise823.fixOp Rat DomainTheory.
    NeighbourhoodSystems.U
        (DomainTheory.NeighbourhoodSystems.tOpMap DomainTheory.NeighbourhoodSystems.
    cCombinator)))
    (@DomainTheory.NeighbourhoodSystems.fixedNbhd Rat DomainTheory.
    NeighbourhoodSystems.U
      (@DomainTheory.NeighbourhoodSystems.Exercise823.fixOp Rat DomainTheory.
    NeighbourhoodSystems.U
        (DomainTheory.NeighbourhoodSystems.tOpMap DomainTheory.NeighbourhoodSystems.
    cCombinator))))
  (@DomainTheory.NeighbourhoodSystems.fixedNbhd Rat DomainTheory.NeighbourhoodSystems.
    U
    (@DomainTheory.NeighbourhoodSystems.Exercise823.fixOp Rat DomainTheory.
    NeighbourhoodSystems.U
      (DomainTheory.NeighbourhoodSystems.tOpMap DomainTheory.NeighbourhoodSystems.
    cCombinator)))
  (@DomainTheory.NeighbourhoodSystems.funSpace Rat Rat
    (@DomainTheory.NeighbourhoodSystems.fixedNbhd Rat DomainTheory.
    NeighbourhoodSystems.U
      (@DomainTheory.NeighbourhoodSystems.Exercise823.fixOp Rat DomainTheory.
    NeighbourhoodSystems.U
        (DomainTheory.NeighbourhoodSystems.tOpMap DomainTheory.NeighbourhoodSystems.
    cCombinator)))
    (@DomainTheory.NeighbourhoodSystems.fixedNbhd Rat DomainTheory.
    NeighbourhoodSystems.U
      (@DomainTheory.NeighbourhoodSystems.Exercise823.fixOp Rat DomainTheory.
    NeighbourhoodSystems.U
        (DomainTheory.NeighbourhoodSystems.tOpMap DomainTheory.NeighbourhoodSystems.
    cCombinator)))) :=
  DomainTheory.NeighbourhoodSystems.exercise_8_25_main

/-- **Exercise 8.25 (Scott 1981, PRG-19), the answer.** The concrete domain
`D := fixedNbhd (fixOp (tOpMap cCombinator))` (built in `Exercise825FixedPoint.lean` by
    the
fixed-point method of Exercise 8.23, applied to `T(a) := a  ->  ?^ inf `) satisfies `D
    ~=  D  ->  D`. -/
theorem exercise_8_25 :
    fixedNbhd (Exercise823.fixOp (tOpMap cCombinator))  ~= ?
      funSpace (fixedNbhd (Exercise823.fixOp (tOpMap cCombinator)))
        (fixedNbhd (Exercise823.fixOp (tOpMap cCombinator))) :=
  exercise_8_25_main

/-- **Exercise 8.25 (Scott 1981, PRG-19), non-triviality/universality.** The solution
    domain `D`
of `exercise_8_25` contains a genuine embedded copy of the universal domain `?` (`? ? D
    `), so it is
certainly not the one-point domain, and is in fact universal: every countable
    neighbourhood system
embeds (up to isomorphism) into `?`, hence into `D`. -/
theorem exercise_8_25_universal : U ? fixedNbhd (Exercise823.fixOp (tOpMap cCombinator
    )) :=
```

```
  U_trianglelefteq_Dsol

set_option linter.unusedVariables false

theorem example_825 : And
  (@DomainTheory.NeighbourhoodSystems.Isomorphic Rat
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap Rat Rat
      (@DomainTheory.NeighbourhoodSystems.fixedNbhd Rat DomainTheory.
    NeighbourhoodSystems.U
        (@DomainTheory.NeighbourhoodSystems.Exercise823.fixOp Rat DomainTheory.
    NeighbourhoodSystems.U
          (DomainTheory.NeighbourhoodSystems.tOpMap DomainTheory.NeighbourhoodSystems.
    cCombinator)))
      (@DomainTheory.NeighbourhoodSystems.fixedNbhd Rat DomainTheory.
    NeighbourhoodSystems.U
        (@DomainTheory.NeighbourhoodSystems.Exercise823.fixOp Rat DomainTheory.
    NeighbourhoodSystems.U
          (DomainTheory.NeighbourhoodSystems.tOpMap DomainTheory.NeighbourhoodSystems.
    cCombinator))))

    (@DomainTheory.NeighbourhoodSystems.fixedNbhd Rat DomainTheory.
    NeighbourhoodSystems.U
      (@DomainTheory.NeighbourhoodSystems.Exercise823.fixOp Rat DomainTheory.
    NeighbourhoodSystems.U
        (DomainTheory.NeighbourhoodSystems.tOpMap DomainTheory.NeighbourhoodSystems.
    cCombinator)))
    (@DomainTheory.NeighbourhoodSystems.funSpace Rat Rat
      (@DomainTheory.NeighbourhoodSystems.fixedNbhd Rat DomainTheory.
    NeighbourhoodSystems.U
        (@DomainTheory.NeighbourhoodSystems.Exercise823.fixOp Rat DomainTheory.
    NeighbourhoodSystems.U
          (DomainTheory.NeighbourhoodSystems.tOpMap DomainTheory.NeighbourhoodSystems.
    cCombinator)))
      (@DomainTheory.NeighbourhoodSystems.fixedNbhd Rat DomainTheory.
    NeighbourhoodSystems.U
        (@DomainTheory.NeighbourhoodSystems.Exercise823.fixOp Rat DomainTheory.
    NeighbourhoodSystems.U
          (DomainTheory.NeighbourhoodSystems.tOpMap DomainTheory.NeighbourhoodSystems.
    cCombinator)))))
  (@DomainTheory.NeighbourhoodSystems.Trianglelefteq Rat Rat DomainTheory.
    NeighbourhoodSystems.U
    (@DomainTheory.NeighbourhoodSystems.fixedNbhd Rat DomainTheory.
    NeighbourhoodSystems.U
      (@DomainTheory.NeighbourhoodSystems.Exercise823.fixOp Rat DomainTheory.
    NeighbourhoodSystems.U
        (DomainTheory.NeighbourhoodSystems.tOpMap DomainTheory.NeighbourhoodSystems.
    cCombinator)))) :=
  @And.intro
  (@DomainTheory.NeighbourhoodSystems.Isomorphic Rat
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap Rat Rat
      (@DomainTheory.NeighbourhoodSystems.fixedNbhd Rat DomainTheory.
    NeighbourhoodSystems.U
        (@DomainTheory.NeighbourhoodSystems.Exercise823.fixOp Rat DomainTheory.
    NeighbourhoodSystems.U
          (DomainTheory.NeighbourhoodSystems.tOpMap DomainTheory.NeighbourhoodSystems.
    cCombinator)))
```

```
      (@DomainTheory.NeighbourhoodSystems.fixedNbhd Rat DomainTheory.
    NeighbourhoodSystems.U
        (@DomainTheory.NeighbourhoodSystems.Exercise823.fixOp Rat DomainTheory.
    NeighbourhoodSystems.U
          (DomainTheory.NeighbourhoodSystems.tOpMap DomainTheory.NeighbourhoodSystems.
    cCombinator))))
     (@DomainTheory.NeighbourhoodSystems.fixedNbhd Rat DomainTheory.
    NeighbourhoodSystems.U
      (@DomainTheory.NeighbourhoodSystems.Exercise823.fixOp Rat DomainTheory.
    NeighbourhoodSystems.U
        (DomainTheory.NeighbourhoodSystems.tOpMap DomainTheory.NeighbourhoodSystems.
    cCombinator)))
     (@DomainTheory.NeighbourhoodSystems.funSpace Rat Rat
      (@DomainTheory.NeighbourhoodSystems.fixedNbhd Rat DomainTheory.
    NeighbourhoodSystems.U
        (@DomainTheory.NeighbourhoodSystems.Exercise823.fixOp Rat DomainTheory.
    NeighbourhoodSystems.U
          (DomainTheory.NeighbourhoodSystems.tOpMap DomainTheory.NeighbourhoodSystems.
    cCombinator)))
      (@DomainTheory.NeighbourhoodSystems.fixedNbhd Rat DomainTheory.
    NeighbourhoodSystems.U
        (@DomainTheory.NeighbourhoodSystems.Exercise823.fixOp Rat DomainTheory.
    NeighbourhoodSystems.U
          (DomainTheory.NeighbourhoodSystems.tOpMap DomainTheory.NeighbourhoodSystems.
    cCombinator)))))
   (@DomainTheory.NeighbourhoodSystems.Trianglelefteq Rat Rat DomainTheory.
    NeighbourhoodSystems.U
     (@DomainTheory.NeighbourhoodSystems.fixedNbhd Rat DomainTheory.
    NeighbourhoodSystems.U
      (@DomainTheory.NeighbourhoodSystems.Exercise823.fixOp Rat DomainTheory.
    NeighbourhoodSystems.U
        (DomainTheory.NeighbourhoodSystems.tOpMap DomainTheory.NeighbourhoodSystems.
    cCombinator))))
  DomainTheory.NeighbourhoodSystems.exercise_8_25 DomainTheory.NeighbourhoodSystems.
    exercise_8_25_universal

end DomainTheory.NeighbourhoodSystems
```

The tip then assembles those tools into Scott's statement.

— exercise_8_25

Exercise_8_25.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    ReflexiveDomain

@[expose] public section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap Exercise823

/-- **Exercise 8.25 (Scott 1981, PRG-19).** A non-trivial domain `D  ~=  D  ->  D`
    containing `?`. -/
theorem exercise_8_25 :
    (fixedNbhd (fixOp (tOpMap cCombinator))  ~= ?
      funSpace (fixedNbhd (fixOp (tOpMap cCombinator)))
```

```
        (fixedNbhd (fixOp (tOpMap cCombinator))))  /\
      (U ? fixedNbhd (fixOp (tOpMap cCombinator))) :=
  <exercise_8_25_main, exercise_8_25_universal>

end
```

3.9.28 Exercise 8.26 — untyped calculus in U

Discuss in more detail the "pay-off" for U, namely the translation of "untyped" λ-calculus into U as shown by the equations at the end of the lecture after the proof of 8.9. In particular show how the whole of the **typed** λ-calculus can be retranslated back into U with the aid of projections. (Hint: Whenever you want to write

$$f : D_a \to D_b,$$

write instead

$$f = b \circ f \circ a,$$

where a, b are finitary projections. Whenever you want to form a λ-abstraction

$$\lambda x^{D_a}.\, \sigma,$$

where σ is of type D_b, instead form

$$\lambda x.\, b(\sigma'[a(x)/x]),$$

where σ' is the further translation of σ into untyped λ-calculus. Be sure to show that this result "has the right type" in the sense defined above.)

Tip summary The tip asserts β-soundness of the self-hosted encoding: `Uapply (Ulam f) x = f(x)`.

Development Encodes self-application and self-abstraction inside `U` and proves β-soundness.

`UapplyUlam.lean`

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Definition_8_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9

public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

/- concordance: source L5246 EXERCISE 8.26 untyped calculus in U -/
-- Exercise826/Uapply.lean
namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap
```

```
/-- **Self-application inside `?`** (the end-of-lecture equation `u(x) := j_ -> (u)(x)
    `): decode
`u : ?` into an actual function `?  ->  ?` via `j_ -> ` (`jArrow`, Definition 8.9),
    then apply it to
`x`. -/
noncomputable def Uapply (u x : U.Element) : U.Element :=
  (evalMap U U).toElementMap (pair (jArrow.toElementMap u) x)

end DomainTheory.NeighbourhoodSystems

namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

/-- **Self-abstraction inside `?`** (the end-of-lecture equation `fun x.tau  ~=  i_ ->
    (fun x.tau)`): given the
"outside" function `f : ?  ->  ?` (`tau`'s ordinary denotation as an approximable map),
     encode it into
`?` via `i_ -> ` (`iArrow`, Definition 8.9). -/
noncomputable def Ulam (f : ApproximableMap U U) : U.Element :=
  iArrow.toElementMap (toFilter f)

section
universe u
variable {a : Type u} {E : NeighborhoodSystem a}

/-- **Scott's translation recipe for a `fun `-abstraction**, `fun x^{D_a}.sigma  |->
    fun x. b(sigma'[a(x)/x])`: given
the further-translated body `body : E  ->  E` (Scott's `sigma'`, the untyped
    realization of `sigma` as a
function of `x`), form the projection-sandwiched abstraction `b comp body comp a`.
    Stated for an arbitrary
neighbourhood system `E` and arbitrary `a, b : E  ->  E` -- specializes to Scott's `?`/
    finitary
projections, but needs nothing special about `E`. -/
def translateAbs (a b body : ApproximableMap E E) : ApproximableMap E E :=
  b.comp (body.comp a)
end

/-- **Faithfulness (`b`-soundness) of the self-hosted encoding.** Applying the self-
    hosted
abstraction of `f` recovers `f` itself: `Uapply (Ulam f) x = f(x)`. The internal `fun
    `-calculus of
`?` literally computes the same thing as the "outside" one it is modelling -- this is
    the actual
content of "the pay-off," beyond bare notation. Uses only `jArrow_comp_iArrow : j_ ->
    comp i_ ->  = I` (the
retraction half of Definition 8.9's fixed pair) and `evalMap`'s defining equation
(`evalMap_apply`). -/
theorem Uapply_Ulam (f : ApproximableMap U U) (x : U.Element) :
    Uapply (Ulam f) x = f.toElementMap x := by
  have hround : toApproxMap (toFilter f) = f := by
    have he := (funSpaceEquiv U U).apply_symm_apply f
    rwa [funSpaceEquiv_apply, funSpaceEquiv_symm_apply] at he
  unfold Uapply Ulam
  have h1 : jArrow.toElementMap (iArrow.toElementMap (toFilter f))
```

```
      = (jArrow.comp iArrow).toElementMap (toFilter f) :=
    (toElementMap_comp jArrow iArrow (toFilter f)).symm
  rw [h1, jArrow_comp_iArrow, toElementMap_idMap, evalMap_apply, hround]

section
universe u
variable {a : Type u} {E : NeighborhoodSystem a}

/-- **A sanity check: the identity abstraction translates to the identity, and the
    recipe applied
to `body = id` (the trivial "translation" of a bare variable `sigma = x`) already has
    the right type
for free** -- `translateAbs a b (idMap E) = b comp a`, matching that a bare variable of
     type `D_a` fed
through the `a`-then-`b` sandwich needs no further work. Included as a `0`-th case of
    the
induction discussed above (the base case: variables). -/
theorem translateAbs_idMap {a b : ApproximableMap E E} : translateAbs a b (idMap E) =
    b.comp a := by
  unfold translateAbs
  rw [idMap_comp]
end

set_option linter.unusedVariables false

theorem example_826.{u} : forall  {a : Type u} {E : DomainTheory.NeighbourhoodSystems.
    NeighborhoodSystem a}
  {a b : @DomainTheory.NeighbourhoodSystems.ApproximableMap a a E E},
  @Eq (@DomainTheory.NeighbourhoodSystems.ApproximableMap a a E E)
    (@DomainTheory.NeighbourhoodSystems.translateAbs a E a b
      (@DomainTheory.NeighbourhoodSystems.ApproximableMap.idMap a E))
    (@DomainTheory.NeighbourhoodSystems.ApproximableMap.comp a a a E E E b a) :=
  @DomainTheory.NeighbourhoodSystems.translateAbs_idMap

section
universe u
variable {a : Type u} {E : NeighborhoodSystem a}

/-- **"Be sure to show that this result has the right type."** `translateAbs a b body`
    always
satisfies the defining sandwich equation `f = b comp f comp a` of Scott's `D_a  ->  D_b
    ` translation scheme
(the "sense defined above" -- see `Proposition810b.elementIsoOfProjectionPair`/
`arrowComb_elementIso`), **regardless of `body`** -- the *only* thing this uses is
    idempotence of
`a` and `b` (true of every retraction, Definition 8.1, in particular every finitary
    projection,
Definition 8.3). Pure associativity + idempotence, mirroring the book's own one-line
    proof
("`(a x b) comp (a x b) = ... = a x b`," `Proposition 8.10`'s proof) transplanted to
    the abstraction case. -/
theorem translateAbs_sandwich {a b body : ApproximableMap E E}
    (ha : Retraction a) (hb : Retraction b) :
    b.comp ((translateAbs a b body).comp a) = translateAbs a b body := by
  show b.comp ((b.comp (body.comp a)).comp a) = b.comp (body.comp a)
  rw [comp_assoc b (body.comp a) a, comp_assoc body a a, ha.out,  <-  comp_assoc b b (
```

```
    body.comp a), hb.out]

/-- **The matching claim for application**: if `f` already "has the right type" (`f = b
     comp f comp a`) and
`x` already "has type `D_a`" (`a(x) = x`), then `f(x)` already "has type `D_b`" (`b(f(x)
    ) = f(x)`).
Together with `translateAbs_sandwich`, this is the type-soundness invariant that both
    the
`fun `-abstraction and application clauses of the typed-to-untyped translation preserve
    . -/
theorem translateApp_hasType {a b f : ApproximableMap E E} (hsandwich : b.comp (f.comp
     a) = f)
    {x : E.Element} (hx : a.toElementMap x = x) :
    b.toElementMap (f.toElementMap x) = f.toElementMap x := by
  conv_rhs => rw [ <-  hsandwich, toElementMap_comp, toElementMap_comp, hx]

end

end DomainTheory.NeighbourhoodSystems
```

The tip then assembles those tools into Scott's statement.

— exercise_8_26

Exercise_8_26.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    UapplyUlam

@[expose] public section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap

/-- **Exercise 8.26 (Scott 1981, PRG-19).** Self-application of self-abstraction
    recovers the map:
`Uapply (Ulam f) x = f(x)`. -/
theorem exercise_8_26 (f : ApproximableMap U U) (x : U.Element) :
    Uapply (Ulam f) x = f.toElementMap x :=
  Uapply_Ulam f x

end
```

3.9.29 Exercise 8.27 — infinite cartesian products

(Suggested by James Donahue.)

Finite cartesian products of domains are formed by the $D_0 \times D_1$-construct we have used so often. The problem is to define — computably — some *infinite* cartesian products. In particular, as applied to the universal domain U, the combinator sub is to be regarded as a finitary projection of U whose fixed points are exactly *all* the finitary projections. A map

$$d = \mathrm{sub} \circ d \circ \mathrm{sub}$$

can be regarded as a *polymorphic type* (because, whenever t is a finitary projection (= type), then so is $d(t)$). The *continuous product* of *all* these types would be the domain of all approximable functions x such that

$$x(t) = d(t)(x(t))$$

for all types t. (Why does this equation mean that x is in the product?) Define Π as a combinator by

$$\Pi = \lambda d\, \lambda x\, \lambda t.\, \mathrm{sub}(d(\mathrm{sub}(t)))(x(\mathrm{sub}(t))).$$

Show that for d a polymorphic type, $\Pi(d)$ is a type. (Hint: It is easy to check that $\Pi(d)$ is a projection; the problem is to show it is *finitary*.)

Tip summary The tip asserts that for any polymorphic type `d`, $\Pi(d)$ is a finitary projection.

Development Defines polymorphic types and proves the `piD` side of finitariness.

PolymorphicType.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.ASharp
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.subU
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.piD
public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    Definition_8_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Proposition_3_9
public import DomainTheory.NeighbourhoodSystems.DomainConstructs.Theorem_3_13
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Factoid_1_7

public import Mathlib.Combinatorics.Colex
public import DomainTheory.NeighbourhoodSystems.MappingsBetweenDomains.Proposition_2_6

@[expose] public section

/- concordance: source L5267 EXERCISE 8.27 infinite cartesian products -/
namespace DomainTheory.NeighbourhoodSystems
open NeighborhoodSystem ApproximableMap

/-- **A polymorphic type** (Scott 1981, PRG-19, Exercise 8.27): `d = sub comp d comp
    sub`, `sub` here being
`subU` (Step 1's "`sub` regarded as a combinator on `?`"). -/
class PolymorphicType (d : ApproximableMap U U) : Prop where
  out : subU.comp (d.comp subU) = d

/-- The family of `subU`-self-consistent nbhds already in `t`'s filter. -/
def scFamily (t : U.Element) : Type := {T : Set ? // subU.rel T T  /\  t.mem T}

instance instNonemptyScFamily (t : U.Element) : Nonempty (scFamily t) :=
  <<U.master, subU.master_rel, t.master_mem>>

/-- The principal element witnessed by a member of `scFamily t`. -/
def scPrincipal {t : U.Element} (i : scFamily t) : U.Element :=
```

```
  U.principal (t.sub i.2.2)

/-- If `a` is idempotent and `f = a comp g comp a` for *some* `g`, then `a(f(z)) = f(z)
    ` for *every* `z` --
i.e. any map "sandwiched" between two copies of an idempotent `a` always lands in `Fix(
    a)`,
regardless of what `g` is. (General form of the pattern behind Exercise 8.26's
`translateApp_hasType`, but derived automatically rather than assumed as a hypothesis.)
     -/
theorem toElementMap_mem_fix_of_isRetraction_sandwich {a g : ApproximableMap U U}
    (ha : Retraction a) (z : U.Element) :
    a.toElementMap ((a.comp (g.comp a)).toElementMap z) = (a.comp (g.comp a)).
    toElementMap z := by
  rw [toElementMap_comp,  <-  toElementMap_comp a a, ha.out]

/-- **The exercise's own justification, formalized**: "whenever `t` is a finitary
    projection, then
so is `d(t)`" -- in fact, for `d` a polymorphic type, `d(z)  in  Fix(subU)` for *every*
     `z`, not just
for `z` a "type". -/
theorem polymorphicType_apply_mem_fix {d : ApproximableMap U U} (hd : PolymorphicType
    d)
    (z : U.Element) : subU.toElementMap (d.toElementMap z) = d.toElementMap z := by
  have h := toElementMap_mem_fix_of_isRetraction_sandwich subU.isProjection.1 (g := d)
     z
  rwa [hd.out] at h

/-- **Any `j_ -> `-decoded fixed point of `subU` is a finitary projection of `?`.** -/
theorem isFinitaryProjection_toApproxMap_jArrow_of_fix {w : U.Element}
    (hw : subU.toElementMap w = w) :
    FinitaryProjection (toApproxMap (jArrow.toElementMap w)) := by
  apply isFinitaryProjection_of_sub_eq_self
  have hfix := subU.subApprox_fix_of_fix hw
  have hstep := congrArg toApproxMap hfix
  rwa [subApprox.toElementMap_eq, Sub8_6.toApproxMap_subFilter] at hstep

/-- **The key lemma: `j_ -> (subU(z))` is always a finitary projection of `?`, for any
    `z`.** -/
theorem isFinitaryProjection_decode_subU (z : U.Element) :
    FinitaryProjection (toApproxMap (jArrow.toElementMap (subU.toElementMap z))) :=
  isFinitaryProjection_toApproxMap_jArrow_of_fix
    (toElementMap_idem_of_isRetraction subU.isProjection.1 z)

theorem toElementMap_piDUncurried (d : ApproximableMap U U) (phi : (funSpace U U).
    Element)
    (t : U.Element) :
    (piDUncurried d).toElementMap (pair phi t) =
      (toApproxMap (jArrow.toElementMap (subU.toElementMap (d.toElementMap
          (subU.toElementMap t))))).toElementMap
        ((toApproxMap phi).toElementMap (subU.toElementMap t)) := by
  simp only [piDUncurried, toElementMap_comp, toElementMap_paired, toElementMap_proj_0
    ,
    toElementMap_proj_1, fst_pair, snd_pair, evalMap_apply]

theorem toElementMap_piDApply (d f : ApproximableMap U U) (t : U.Element) :
    (piDApply d f).toElementMap t =
```

```
      (toApproxMap (jArrow.toElementMap (subU.toElementMap (d.toElementMap
          (subU.toElementMap t))))).toElementMap (f.toElementMap (subU.toElementMap t)
    ) := by
  simp only [piDApply, toElementMap_comp, toElementMap_paired, evalMap_apply]

/-- **`piD d`, transported through `funSpaceEquiv`, is exactly `piDApply d`.** Mirrors
`toApproxMap_toElementMap_lamComb`/`expMap.toApproxMap_toElementMap`. -/
theorem toApproxMap_toElementMap_piD (d : ApproximableMap U U) (phi : (funSpace U U).
    Element) :
    toApproxMap ((piD d).toElementMap phi) = piDApply d (toApproxMap phi) := by
  apply ApproximableMap.ext_of_toElementMap
  intro t
  rw [piD, toElementMap_curry_apply, toElementMap_piDUncurried, toElementMap_piDApply]

/-- **Exercise 8.27(b)(2)(a).** `(piD d).rel X W` reduces to a literal membership test
    -- the direct
specialization of `curry_rel` (`FunctionSpace.lean`), no new proof needed. -/
theorem piD_rel_iff (d : ApproximableMap U U) {X W : Set (ApproximableMap U U)} :
    (piD d).rel X W  <->  exists  hX : (funSpace U U).mem X, (funSpace U U).mem W  /\
      gSection (piDUncurried d) hX  in  W :=
  curry_rel

/-- **Exercise 8.27(b)(2)(b).** `gSection (piDUncurried d) hX`'s own relation, fully
    unwound to a
closed elementwise formula in terms of the already-proven closed form `piDApply` (built
    , in its
own proof, purely from `subU`'s and `d`'s `toElementMap` actions plus `evalMap_apply`):
     reduce via
`gSection_rel` to `(piDUncurried d).rel (prodNbhd X Y) Z`, then via `
    rel_iff_mem_principal` +
`pair_principal_eq_principal_prodNbhd` to the element pair `pair (principal hX) (
    principal hY)`,
then read off `toElementMap_piDUncurried`/`toElementMap_piDApply`. This is the "unwind
    `piD d`'s
neighbourhood relation" content of Exercise 8.27(b)(2). -/
theorem gSection_piDUncurried_rel_iff (d : ApproximableMap U U)
    {X : Set (ApproximableMap U U)} (hX : (funSpace U U).mem X) {Y Z : Set ?} (hY : U.
    mem Y) :
    (gSection (piDUncurried d) hX).rel Y Z  <->
      ((piDApply d (toApproxMap ((funSpace U U).principal hX))).toElementMap
        (U.principal hY)).mem Z := by
  rw [gSection_rel, rel_iff_mem_principal (piDUncurried d) (prod_mem_prodNbhd hX hY),
     <-  pair_principal_eq_principal_prodNbhd hX hY, toElementMap_piDUncurried,
     <-  toElementMap_piDApply]

/-- **Exercise 8.27(b)(3)(a).** `hii`'s LHS, reduced to a literal membership test: no
    abstract
"funSpace element" reasoning left, purely `piDApply d (toApproxMap x)  in  Y` as an
    actual map/set. -/
theorem piD_toElementMap_mem_iff (d : ApproximableMap U U) (x : (funSpace U U).Element
    )
    {Y : Set (ApproximableMap U U)} :
    ((piD d).toElementMap x).mem Y  <->
      (funSpace U U).mem Y  /\  piDApply d (toApproxMap x)  in  Y := by
  conv_lhs => rw [ <-  Sub8_6.toFilter_toApproxMap ((piD d).toElementMap x)]
  rw [mem_toFilter, toApproxMap_toElementMap_piD]
```

```
/-- **Exercise 8.27(b)(3), the `?` half -- closes unconditionally, for any `d` at all
    .** Direct
specialization of `Theorem85.lean`'s newly-extracted `mem_of_exists_rel_self` (no
    hypothesis on
`a` needed) to `a := piD d`. The remaining, genuinely hard direction is documented
    above and in
`HANDOFF.md`; **not** attempted here. -/
theorem hii_easy_direction (d : ApproximableMap U U) (x : (funSpace U U).Element)
    {Y : Set (ApproximableMap U U)} (hYE : (funSpace U U).mem Y)
    {X : Set (ApproximableMap U U)} (hXx : x.mem X) (hXY : X  subseteq  Y) (hXX : (piD
     d).rel X X) :
    ((piD d).toElementMap x).mem Y :=
  mem_of_exists_rel_self x hYE hXx hXY hXX

/-- **`subU`'s self-relating neighbourhoods are exactly fixed under `principal`.** If `
    subU.rel T T`,
the "witnessing element" `T` is *exactly* fixed by `subU` (not merely approximated): `
    <= ` is
`subU`'s projection property (`subU  <=  I`), ` >= ` is `principal_le_of_mem` applied
    to the membership
`subU.rel T T` gives via `rel_iff_mem_principal`. -/
theorem subU_principal_eq_of_rel_self {T : Set ?} (hT : U.mem T) (hTT : subU.rel T T)
    :
    subU.toElementMap (U.principal hT) = U.principal hT := by
  apply le_antisymm
  * exact toElementMap_le_self_of_le_idMap subU.isProjection.2 _
  * exact principal_le_of_mem ((subU.rel_iff_mem_principal hT).mp hTT)

/-- **Descending to a self-consistent approximant.** For `t` already `subU`-fixed,
    formula (ii) for
`subU` (Exercise 8.27(a)) applied at `t` and any target `T_0  in  t` produces a *self-
    consistent*
`T  subseteq  T_0` still in `t`'s filter -- the cofinality fact that makes the self-
    consistent nbhds a directed
family sup-ing to `t` itself (`eq_iSupDirected_scPrincipal` below). -/
theorem exists_rel_self_subset_of_mem {t : U.Element} (ht : subU.toElementMap t = t)
    {T_0 : Set ?} (hT_0 : t.mem T_0) : exists  T, subU.rel T T  /\  t.mem T  /\  T
    subseteq  T_0 := by
  have hmem : (subU.toElementMap t).mem T_0 := by rw [ht]; exact hT_0
  obtain <_, T, hTt, hTT_0, hTT> :=
    (formula_of_isFinitaryProjection subU.isFinitaryProjection t).mp hmem
  exact <T, hTT, hTt, hTT_0>

/-- **`scFamily t`'s principals form a directed family.** Given `i, j`, apply
`exists_rel_self_subset_of_mem` at the common refinement `i.1  I  j.1` (valid via `t.
    inter_mem`). -/
theorem scFamily_directed {t : U.Element} (ht : subU.toElementMap t = t) :
    forall  i j : scFamily t, exists  k : scFamily t, scPrincipal i  <=  scPrincipal k
      /\  scPrincipal j  <=  scPrincipal k :=
  fun i j => by
    obtain <T, hTT, htT, hTsub> := exists_rel_self_subset_of_mem ht (t.inter_mem i.2.2
     j.2.2)
    refine <<T, hTT, htT>, ?_, ?_>
    * exact (U.principal_le_iff (t.sub i.2.2) (t.sub htT)).mpr (hTsub.trans Set.
    inter_subset_left)
```

```
    * exact (U.principal_le_iff (t.sub j.2.2) (t.sub htT)).mpr (hTsub.trans Set.
    inter_subset_right)

/-- **`t` is the directed sup of its own self-consistent approximants.** Cofinality
(`exists_rel_self_subset_of_mem`) shows the self-consistent sub-family already has the
    same sup as
the full principal-approximant family (`eq_iSupDirected_principal`), namely `t` itself.
     -/
theorem eq_iSupDirected_scPrincipal {t : U.Element} (ht : subU.toElementMap t = t) :
    t = NeighborhoodSystem.iSupDirected scPrincipal (scFamily_directed ht) := by
  apply Element.ext
  intro Z
  rw [mem_iSupDirected]
  constructor
  * intro hZ
    obtain <T, hTT, htT, hTZ> := exists_rel_self_subset_of_mem ht hZ
    exact <<T, hTT, htT>, (U.mem_principal _).mpr <t.sub hZ, hTZ>>
  * rintro <i, hi>
    obtain <hZmem, hTZ> := (U.mem_principal _).mp hi
    exact t.up_mem i.2.2 hZmem hTZ

/-- **Exercise 8.27(b)(1).** If `piD d` satisfies Scott's step-closure formula (ii), it
     is a
finitary projection -- a direct specialization of `Theorem85.lean`'s general
`isFinitaryProjection_of_formula`, no new proof needed beyond the specialization itself
    . -/
theorem isFinitaryProjection_piD_of_formula (d : ApproximableMap U U)
    (hii : forall  (x : (funSpace U U).Element) {Y}, ((piD d).toElementMap x).mem Y
    <->
      (funSpace U U).mem Y  /\  exists  X, x.mem X  /\  X  subseteq  Y  /\  (piD d).
    rel X X) :
    FinitaryProjection (piD d) :=
  isFinitaryProjection_of_formula (piD d) hii

/-- **`?(d)  <=  I`, for any `d`.** Chains: `subU(t)  sqsub  t` (`subU`'s own
    projection property) with
`f`'s monotonicity to get `f(subU(t))  sqsub  f(t)`, then the "type at `t`"'s own ` <=
     I` (from
`isFinitaryProjection_decode_subU`) to get `(type at t)(f(subU(t)))  sqsub  f(subU(t))
    `. -/
theorem le_idMap_piD (d : ApproximableMap U U) : piD d  <=  idMap (funSpace U U) := by
  rw [le_iff_toElementMap_le]
  intro phi
  rw [toElementMap_idMap,  <-  (funSpaceEquiv U U).le_iff_le, funSpaceEquiv_apply,
    funSpaceEquiv_apply, toApproxMap_toElementMap_piD, le_iff_toElementMap_le]
  intro t
  rw [toElementMap_piDApply]
  have hP := (isFinitaryProjection_decode_subU (d.toElementMap (subU.toElementMap t)))
    .out.1.2
  calc (toApproxMap (jArrow.toElementMap (subU.toElementMap
        (d.toElementMap (subU.toElementMap t))))).toElementMap
        ((toApproxMap phi).toElementMap (subU.toElementMap t))
      <=  (toApproxMap phi).toElementMap (subU.toElementMap t) :=
       toElementMap_le_self_of_le_idMap hP _
    _  <=  (toApproxMap phi).toElementMap t :=
        (toApproxMap phi).toElementMap_mono
```

```
          (toElementMap_le_self_of_le_idMap subU.isProjection.2 t)

/-- Evaluating `piDApply d f` at an already-`subU`-projected point simplifies (via `
    subU`'s
idempotency) exactly to `toElementMap_piDApply`'s formula with `t` replaced by `subU.
    toElementMap
t` throughout. The key computational step behind `isRetraction_piD`. -/
theorem piDApply_toElementMap_eq (d f : ApproximableMap U U) (t : U.Element) :
    (piDApply d f).toElementMap (subU.toElementMap t) =
      (toApproxMap (jArrow.toElementMap (subU.toElementMap (d.toElementMap
          (subU.toElementMap t))))).toElementMap (f.toElementMap (subU.toElementMap t)
    ) := by
  rw [toElementMap_piDApply, toElementMap_idem_of_isRetraction subU.isProjection.1 t]

/-- **`?(d)` is a retraction, for any `d`.** -/
theorem isRetraction_piD (d : ApproximableMap U U) : Retraction (piD d) := by
  constructor
  apply ApproximableMap.ext_of_toElementMap
  intro phi
  apply (funSpaceEquiv U U).injective
  rw [funSpaceEquiv_apply, funSpaceEquiv_apply, toElementMap_comp,
    toApproxMap_toElementMap_piD d ((piD d).toElementMap phi),
    toApproxMap_toElementMap_piD d phi]
  apply ApproximableMap.ext_of_toElementMap
  intro t
  rw [toElementMap_piDApply d (piDApply d (toApproxMap phi)) t,
    piDApply_toElementMap_eq d (toApproxMap phi) t,
    toElementMap_piDApply d (toApproxMap phi) t]
  exact toElementMap_idem_of_isRetraction
    (isFinitaryProjection_decode_subU (d.toElementMap (subU.toElementMap t))).out.1.1
    _

/-- **Scott's own hint, in full: `?(d)` is a projection, for every `d`.** -/
theorem isProjection_piD (d : ApproximableMap U U) : IsProjection (piD d) :=
  <isRetraction_piD d, le_idMap_piD d>

/-- **`piDType d` is monotone in `s`.** Composite of monotone `toElementMap`s (`d`, `
    subU`,
`jArrow`) followed by monotone `toApproxMap` (`Sub8_6.toApproxMap_monotone`). -/
theorem piDType_monotone (d : ApproximableMap U U) {s s' : U.Element} (h : s  <=  s')
    :
    piDType d s  <=  piDType d s' :=
  Sub8_6.toApproxMap_monotone ((piDTypeMap d).toElementMap_mono h)

/-- **Exercise 8.27(b)(2), specialized to self-relation** (the shape Theorem 8.5's
    formula (ii)
actually tests, `a.rel X X`): combines (a) and (b) into the single membership-test
    formula that
Exercise 8.27(b)(3) will chain through `subU`'s and each `D'_s`'s own formula (ii). -/
theorem piD_rel_self_iff (d : ApproximableMap U U) {X : Set (ApproximableMap U U)} :
    (piD d).rel X X  <->  exists  hX : (funSpace U U).mem X, gSection (piDUncurried d)
     hX  in  X := by
  rw [piD_rel_iff]
  exact <fun <hX, _, hmem> => <hX, hmem>, fun <hX, hmem> => <hX, hX, hmem>>

theorem toElementMap_piDTypeMap (d : ApproximableMap U U) (s : U.Element) :
```

```
    (piDTypeMap d).toElementMap s = jArrow.toElementMap (subU.toElementMap (d.
    toElementMap s)) := by
  unfold piDTypeMap
  rw [toElementMap_comp, toElementMap_comp]

theorem isFinitaryProjection_piDType (d : ApproximableMap U U) (s : U.Element) :
    FinitaryProjection (piDType d s) := by
  unfold piDType
  rw [toElementMap_piDTypeMap]
  exact isFinitaryProjection_decode_subU (d.toElementMap s)

/-- `piDApply`'s defining formula, restated via `piDType`/`piDTypeMap`. -/
theorem toElementMap_piDApply' (d f : ApproximableMap U U) (t : U.Element) :
    (piDApply d f).toElementMap t =
      (piDType d (subU.toElementMap t)).toElementMap (f.toElementMap (subU.
    toElementMap t)) := by
  rw [toElementMap_piDApply, piDType, toElementMap_piDTypeMap]

/-- **Exercise 8.27(b)(3)(b), single-pair case -- the genuine mathematical content.**
    Given
`f := piDApply d (toApproxMap x)` relates `Y_0` to `Z_0`, produce `X` (a single `step`)
    with
`x.mem X`, `X  subseteq  step Y_0 Z_0`, `(piD d).rel X X`.

The compactness-descent argument, in outline (`t := subU(Y_0)`):
1. Formula (ii) for the finitary projection `piDType d t` (at `t`'s own image of `x`)
    gives `W  subseteq  Z_0`
   with `piDType d t` self-relating `W`.
2. Both `(toApproxMap x).rel _ W`'s witness and `(piDType d t).rel W W` are pushed down
    , via
   `t =  bigcup  scPrincipal` and directed-sup compactness, to *some* self-consistent
    approximants
   `i_0, i_1  in  scFamily t`.
3. A common refinement `k` (directedness of `scFamily t`) makes *both* facts hold
    simultaneously at
   `T := k.1`: `(toApproxMap x).rel T W` (monotonicity) and `(piDType d (scPrincipal k)
    ).rel W W`
   (monotonicity of `piDType d`, since bigger domain element ? bigger map).
4. `scPrincipal k` is *exactly* `subU`-fixed (self-consistency), so `piDType d (
    scPrincipal k))` is
   *exactly* the type-projection `X`'s own self-test uses at `T` -- no mismatch. -/
theorem exists_X_of_mem_step (d : ApproximableMap U U) (x : (funSpace U U).Element)
    {Y_0 Z_0 : Set ?} (hmem : piDApply d (toApproxMap x)  in  step Y_0 Z_0) :
    exists  X, x.mem X  /\  X  subseteq  (step Y_0 Z_0 : Set (ApproximableMap U U))
    /\  (piD d).rel X X := by
  have hrelY0Z0 : (piDApply d (toApproxMap x)).rel Y_0 Z_0 := hmem
  have hY_0 : U.mem Y_0 := (piDApply d (toApproxMap x)).rel_dom hrelY0Z0
  have hZ_0 : U.mem Z_0 := (piDApply d (toApproxMap x)).rel_cod hrelY0Z0
  set t : U.Element := subU.toElementMap (U.principal hY_0) with ht_def
  have ht : subU.toElementMap t = t := toElementMap_idem_of_isRetraction subU.
    isProjection.1 _
  have hmem' : ((piDType d t).toElementMap ((toApproxMap x).toElementMap t)).mem Z_0
    := by
    have h := ((piDApply d (toApproxMap x)).rel_iff_mem_principal hY_0).mp hrelY0Z0
    rwa [toElementMap_piDApply'] at h
  obtain <_, W, hW_mem, hWZ_0, hWW> :=
```

```
  (formula_of_isFinitaryProjection (isFinitaryProjection_piDType d t)
    ((toApproxMap x).toElementMap t)).mp hmem'
have hw_sup := eq_iSupDirected_scPrincipal ht
-- Push the continuity witness `W` for `x` down to a single self-consistent
  approximant `i_0`.
rw [hw_sup, toElementMap_iSupDirected, mem_iSupDirected] at hW_mem
obtain <i_0, hi_0> := hW_mem
have hrelT0 : (toApproxMap x).rel i_0.1 W :=
  (rel_iff_mem_principal (toApproxMap x) (t.sub i_0.2.2)).mpr hi_0
-- Push the self-relation witness `W` for `piDType d t` down to a single self-
  consistent `i_1`.
have hWW2 : exists  i : scFamily t, (piDType d (scPrincipal i)).rel W W := by
  have h2 : (piDType d t).rel W W := hWW
  unfold piDType at h2
  rw [hw_sup, toElementMap_iSupDirected, Sub8_6.toApproxMap_rel_iSupDirected] at h2
  exact h2
obtain <i_1, hi_1> := hWW2
-- Common refinement `k` of `i_0, i_1` makes both facts hold simultaneously.
obtain <k, hik0, hik1> := scFamily_directed ht i_0 i_1
have hksub0 : k.1  subseteq  i_0.1 := (U.principal_le_iff (t.sub i_0.2.2) (t.sub k
  .2.2)).mp hik0
have hTfinal : U.mem k.1 := subU.rel_dom k.2.1
have hWmem : U.mem W := (piDType d t).rel_dom hWW
have hrelTfinal : (toApproxMap x).rel k.1 W :=
  (toApproxMap x).mono hrelT0 hksub0 subset_rfl hTfinal hWmem
have hPfinal : (piDType d (scPrincipal k)).rel W W := piDType_monotone d hik1 W W
  hi_1
-- `k.1` is `subU`-self-consistent, so `scPrincipal k = subU(k.1)` exactly.
have hTfinal_fix : subU.toElementMap (U.principal hTfinal) = U.principal hTfinal :=
  subU_principal_eq_of_rel_self hTfinal k.2.1
set X : Set (ApproximableMap U U) := step k.1 W with hX_def
have hXvalid : (funSpace U U).mem X := step_mem hTfinal hWmem
refine <X, ?_, ?_, ?_>
* -- `x.mem X`
  exact toApproxMap_rel.mp hrelTfinal
* -- `X  subseteq  step Y_0 Z_0`
  have hle : t  <=  U.principal hY_0 := toElementMap_le_self_of_le_idMap subU.
  isProjection.2 _
  have hY0sub : Y_0  subseteq  k.1 := ((U.mem_principal hY_0).mp (hle k.1 k.2.2)).2
  exact step_subset hY_0 hZ_0 hY0sub hWZ_0
* -- `(piD d).rel X X`
  rw [piD_rel_self_iff]
  refine <hXvalid, ?_>
  show (gSection (piDUncurried d) hXvalid).rel k.1 W
  rw [gSection_piDUncurried_rel_iff d hXvalid hTfinal]
  set g : ApproximableMap U U := toApproxMap ((funSpace U U).principal hXvalid) with
   hg_def
  show ((piDApply d g).toElementMap (U.principal hTfinal)).mem W
  rw [toElementMap_piDApply', hTfinal_fix]
  have hg : g.rel k.1 W := by
    rw [hg_def, toApproxMap_rel]
    exact ((funSpace U U).mem_principal hXvalid).mpr <hXvalid, subset_rfl>
  have hgmem : (g.toElementMap (U.principal hTfinal)).mem W :=
    (rel_iff_mem_principal g hTfinal).mp hg
  exact mem_of_exists_rel_self (g.toElementMap (U.principal hTfinal))
    ((piDType d (U.principal hTfinal)).rel_dom hPfinal) hgmem subset_rfl hPfinal
```

```
/-- **Exercise 8.27(b)(3)(b), assembled over a whole `stepFun` list.** Induction on the
     list `L`
using `exists_X_of_mem_step` for the head pair and the inductive hypothesis for the
    tail, combined
via `x.inter_mem` (giving the ambient validity of the intersection "for free", `x.sub`)
     and
`(piD d).mono`/`.inter_right` (self-relation of an intersection of two self-related
    nbhds). -/
theorem exists_X_of_mem_stepFun (d : ApproximableMap U U) (x : (funSpace U U).Element)
    (L : List (Set ?  x  Set ?)) (hmem : piDApply d (toApproxMap x)  in  stepFun L) :
    exists  X, x.mem X  /\  X  subseteq  (stepFun L : Set (ApproximableMap U U))  /\
    (piD d).rel X X := by
  induction L with
  | nil =>
    refine <(funSpace U U).master, x.master_mem, ?_, (piD d).master_rel>
    simp
  | cons p L ih =>
    rw [stepFun_cons] at hmem
    obtain <hmemP, hmemL> := hmem
    obtain <Xp, hXpx, hXpsub, hXpXp> := exists_X_of_mem_step d x hmemP
    obtain <XL, hXLx, hXLsub, hXLXL> := ih hmemL
    refine <Xp  I  XL, x.inter_mem hXpx hXLx, ?_, ?_>
    * rw [stepFun_cons]
      exact Set.inter_subset_inter hXpsub hXLsub
    * have hvalidP : (funSpace U U).mem Xp := x.sub hXpx
      have hvalidL : (funSpace U U).mem XL := x.sub hXLx
      have hvalidInter : (funSpace U U).mem (Xp  I  XL) := x.sub (x.inter_mem hXpx
    hXLx)
      have h1 : (piD d).rel (Xp  I  XL) Xp :=
        (piD d).mono hXpXp Set.inter_subset_left subset_rfl hvalidInter hvalidP
      have h2 : (piD d).rel (Xp  I  XL) XL :=
        (piD d).mono hXLXL Set.inter_subset_right subset_rfl hvalidInter hvalidL
      exact (piD d).inter_right h1 h2

/-- **Exercise 8.27(b)(3), the `?` half, closed for any `d` at all.** Destructure the
    general
`Y : Set (ApproximableMap U U)` via `funSpace_mem_iff` into `Y = stepFun L`, then apply
`exists_X_of_mem_stepFun`. -/
theorem hii_hard_direction (d : ApproximableMap U U) (x : (funSpace U U).Element)
    {Y : Set (ApproximableMap U U)} (hYE : (funSpace U U).mem Y)
    (hmem : piDApply d (toApproxMap x)  in  Y) :
    exists  X, x.mem X  /\  X  subseteq  Y  /\  (piD d).rel X X := by
  obtain <<L, _, rfl>, _> := hYE
  exact exists_X_of_mem_stepFun d x L hmem

/-- **Exercise 8.27(b)(3), in full: `piD d` satisfies Scott's step-closure formula (ii)
    , for *any*
`d` at all** (`d`'s polymorphism is not needed). Combines the reduction (b)(3)(a) with
    the easy
(`hii_easy_direction`) and now-closed hard (`hii_hard_direction`) halves. -/
theorem hii_piD (d : ApproximableMap U U) (x : (funSpace U U).Element)
    {Y : Set (ApproximableMap U U)} :
    ((piD d).toElementMap x).mem Y  <->
      (funSpace U U).mem Y  /\  exists  X, x.mem X  /\  X  subseteq  Y  /\  (piD d).
    rel X X := by
```

```
  rw [piD_toElementMap_mem_iff]
  constructor
  * rintro <hYE, hmem>
    exact <hYE, hii_hard_direction d x hYE hmem>
  * rintro <hYE, X, hXx, hXY, hXX>
    have := hii_easy_direction d x hYE hXx hXY hXX
    rwa [piD_toElementMap_mem_iff] at this

/-- **Exercise 8.27(b)(4): `piD d` is a finitary projection, for every `d`** (Exercise
    8.27(b)(1)


specialized to `hii_piD`). Combined with Exercise 8.27(b)(0) (`isFinitary.
    piU_of_isFinitary_piD`),
this closes "`?(d)` is a type" -- Scott's own hint, in full, and *without even needing
    `d` to be a
polymorphic type* (a strictly stronger statement than the exercise asks for). -/
theorem isFinitaryProjection_piD (d : ApproximableMap U U) : FinitaryProjection (piD d
    ) :=
  isFinitaryProjection_piD_of_formula d (fun x => hii_piD d x)

end DomainTheory.NeighbourhoodSystems
```

Carves `piU` and folds `XXX_piU` companions as `XXX.piU`.
— `piU`, `toElementMap.piU`, `isProjection.piU`, `isFinitaryProjection.piU`, ...

piU.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.
    PolymorphicType

@[expose] public section

namespace DomainTheory.NeighbourhoodSystems

open NeighborhoodSystem ApproximableMap

/-- **`?(d)`, regarded as a combinator living in `?` itself** -- conjugating `piD d`
    through
`i_ -> , j_ -> ` (Definition 8.9), exactly as `subU` was built from `subApprox` in Step
     1. This is what
Scott's "type" means: a self-map *of `?`*, not of `(? -> ?)`. -/
noncomputable def piU (d : ApproximableMap U U) : ApproximableMap U U :=
  iArrow.comp ((piD d).comp jArrow)

namespace toElementMap

/-- `piU d`'s defining formula unfolded at the element level (mirrors `subU.
    toElementMap_eq`). -/
theorem piU (d : ApproximableMap U U) (w : U.Element) :
    (DomainTheory.NeighbourhoodSystems.piU d).toElementMap w =
      iArrow.toElementMap ((piD d).toElementMap (jArrow.toElementMap w)) := by
  unfold DomainTheory.NeighbourhoodSystems.piU
  rw [toElementMap_comp, toElementMap_comp]
```

```
end toElementMap

namespace piD

/-- Unfolds `(piU d).toElementMap w = w` into the statement that `j_ -> (w)` is already
     a fixed
point of `piD d` (mirrors `subU.subApprox_fix_of_fix`). -/
theorem fix_of_piU_fix {d : ApproximableMap U U} {w : U.Element}
    (hw : (piU d).toElementMap w = w) :
    (piD d).toElementMap (jArrow.toElementMap w) = jArrow.toElementMap w := by
  rw [toElementMap.piU] at hw
  have hstep := congrArg jArrow.toElementMap hw
  rwa [jArrow_comp_iArrow_apply] at hstep

end piD

namespace piU

/-- Mirrors `subU.fix_of_subApprox_fix`. -/
theorem fix_of_piD_fix {d : ApproximableMap U U} {phi : (funSpace U U).Element}
    (hphi : (piD d).toElementMap phi = phi) :
    (piU d).toElementMap (iArrow.toElementMap phi) = iArrow.toElementMap phi := by
  rw [toElementMap.piU, jArrow_comp_iArrow_apply, hphi]

/-- **Exercise 8.27(b)(0): `Fix(piU d) ?o Fix(piD d)`.** Forward: `w  |->  j_ -> (w)`;
    backward:
`phi  |->  i_ -> (phi)`. Needs only `j_ ->  comp i_ ->  = I` (no inequality on `i_ ->
    comp j_ -> `) -- mirrors `subU.fixIso`
verbatim, with `piD d`/`piU d` in place of `subApprox`/`subU`. -/
noncomputable def FixIso (d : ApproximableMap U U) :
    {w : U.Element // (piU d).toElementMap w = w} ?o
      {phi : (funSpace U U).Element // (piD d).toElementMap phi = phi} where
  toFun w := <jArrow.toElementMap w.1, piD.fix_of_piU_fix w.2>
  invFun phi := <iArrow.toElementMap phi.1, fix_of_piD_fix phi.2>
  left_inv w := by
    apply Subtype.ext
    show iArrow.toElementMap (jArrow.toElementMap w.1) = w.1
    have hw := toElementMap.piU d w.1
    rw [w.2] at hw
    rw [piD.fix_of_piU_fix w.2] at hw
    exact hw.symm
  right_inv phi := by
    apply Subtype.ext
    exact jArrow_comp_iArrow_apply phi.1
  map_rel_iff' := by
    intro w w'
    show jArrow.toElementMap w.1  <=  jArrow.toElementMap w'.1  <->  w.1  <=  w'.1
    constructor
    * intro hle
      have h2 := iArrow.toElementMap_mono hle
      have e1 : iArrow.toElementMap (jArrow.toElementMap w.1) = w.1 := by
        have hw := toElementMap.piU d w.1
        rw [w.2, piD.fix_of_piU_fix w.2] at hw
        exact hw.symm
      have e2 : iArrow.toElementMap (jArrow.toElementMap w'.1) = w'.1 := by
        have hw' := toElementMap.piU d w'.1
```

```
        rw [w'.2, piD.fix_of_piU_fix w'.2] at hw'
        exact hw'.symm
      rwa [e1, e2] at h2
    * intro hle
      exact jArrow.toElementMap_mono hle

end piU

namespace isFinitary

/-- **Exercise 8.27(b)(0), corollary: if `piD d` is finitary then so is `piU d`.**
    Mirrors
`subU.isFinitary`'s use of `subU.fixIso`. This is the reduction Exercise 8.27(b)(1)-(b)
    (4) will
target: it suffices to prove `IsFinitary (piD d)` for `d` polymorphic. -/
theorem piU_of_isFinitary_piD {d : ApproximableMap U U} (h : IsFinitary (piD d)) :
    IsFinitary (piU d) := by
  obtain <b, F, <iso>> := h
  exact <b, F, <(piU.FixIso d).trans iso>>

/-- **Exercise 8.27(b)(1), assembled with (b)(0):** formula (ii) for `piD d` alone
    gives
`IsFinitary (piU d)`, i.e. (combined with the already-Pass 8.27(a)) the *entire*
    exercise for `d`
polymorphic. This pins down exactly what Exercise 8.27(b)(2)-(b)(3) need to supply: `
    hii`. -/
theorem piU_of_formula {d : ApproximableMap U U}
    (hii : forall  (x : (funSpace U U).Element) {Y}, ((piD d).toElementMap x).mem Y
    <->
      (funSpace U U).mem Y  /\  exists  X, x.mem X  /\  X  subseteq  Y  /\  (piD d).
    rel X X) :
    IsFinitary (piU d) :=
  piU_of_isFinitary_piD (isFinitaryProjection_piD_of_formula d hii).out.2

end isFinitary

namespace isProjection

/-- **`?(d)`, regarded as living in `?` (`piU d`), is a projection, for every `d`.**
    Half of "for
`d` a polymorphic type, `?(d)` is a type" (Definition 8.3's `IsProjection` clause) --
    Scott's own
"it is easy to check" -- obtained "for free" from `isProjection_piD` via the same
    conjugation
gadget (`subU.isProjection_conjArrow`) used for `subU` in Step 1. Note this does not
    even need `d` to
be a polymorphic type. -/
theorem piU (d : ApproximableMap U U) : IsProjection (DomainTheory.
    NeighbourhoodSystems.piU d) :=
  subU.isProjection_conjArrow (isProjection_piD d)

end isProjection

namespace isFinitaryProjection

/-- **Exercise 8.27, in full: for `d` a polymorphic type, `?(d)` is a type.** (In fact
```

```
    for *any*
`d` at all -- see `isFinitaryProjection_piD`.) This is the exercise's own statement,
    the last exercise
in the book, closed. -/
theorem piU (d : ApproximableMap U U) : FinitaryProjection (DomainTheory.
    NeighbourhoodSystems.piU d) :=
  <<isProjection.piU d, isFinitary.piU_of_isFinitary_piD (isFinitaryProjection_piD d).
    out.2>>

end isFinitaryProjection

end DomainTheory.NeighbourhoodSystems
```

The tip then assembles those tools into Scott's statement.

— exercise_8_27

Exercise_8_27.lean

```
module

public import DomainTheory.NeighbourhoodSystems.RetractsOfTheUniversalDomain.piU

@[expose] public section

open DomainTheory.NeighbourhoodSystems NeighborhoodSystem ApproximableMap

/-- **Exercise 8.27 (Scott 1981, PRG-19).** For a polymorphic type `d`, `?(d)` is a
    type
(finitary projection). -/
theorem exercise_8_27 (d : ApproximableMap U U) : FinitaryProjection (piU d) :=
  isFinitaryProjection.piU d

end
```

4 Information Systems (1982)

The purpose of the theory of domains is to give models for spaces on which to define computable functions. The kinds of spaces needed for denotational semantics involve not only spaces of higher type (e.g. function spaces) but also spaces defined recursively (e.g. reflexive domains). Also required are many special domain constructs (or functors) in order to create the desired structures. There are several choices of a suitable category of domains, but the basic one which has the simplest properties is the one sometimes called *consistently complete algebraic* cpo*'s*. This category of domains is studied in this chapter from a new, and it is to be hoped, simpler point of view incorporating the approaches of many authors into a unified presentation. Briefly, the domains of elements are represented set theoretically with the aid of structures called *information systems*. These systems are very familiar from mathematical logic, and their use seems to accord well with intuition. Many things that were done previously axiomatically can now be proved in a straightforward way as theorems. The present chapter discusses many examples in an informal way that should serve as an introduction to the subject.

Dependencies. Each box is one folder. The arrowhead points at the folder that imports, and the number is how many direct imports.

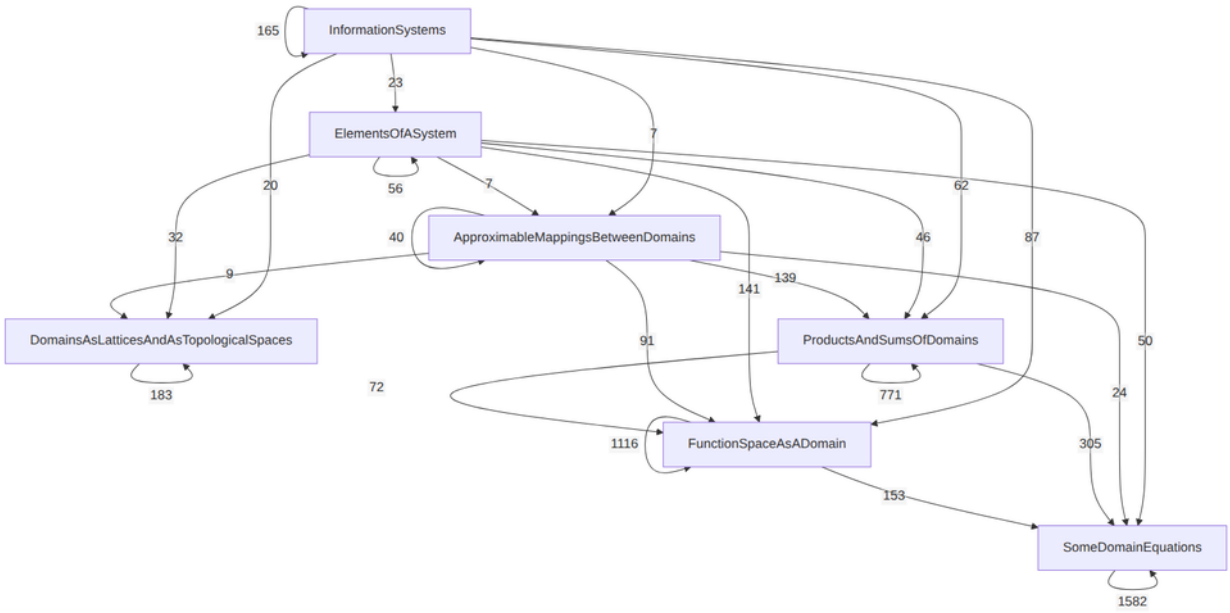


Figure 38: Information Systems (1982).

4.1 Introduction

The difficulty in the presentation of the subject is in justifying the level of abstraction used in comparison with the payoff: too often the effort needed for understanding the abstractions does not seem worth the trouble—especially if the notions are unfamiliar or excessively general.

For example, Category Theory, which is much used in discussions of the Theory of Domains, seems far too abstract to many people. On the other hand, Automata Theory—which has often benefited from some use of Category Theory—is quite abstract and, in many aspects perhaps, quite useless, but the level of abstraction is fairly low and the reasons for the definitions are usually evident. Thus, the subject has become by now a standard topic even for undergraduate courses. Michael Rabin and Scott had much to do with influencing the subsequent development of the theory by formulating some years ago some basic concepts (many previously known) in a way that made sense to anyone who had been exposed to a little Abstract Algebra, Logic and Set Theory. This expository part of their paper did not require originality, but rather a certain frame of mind and a certain style of writing to put the ideas and problems in sharp relief. (The original paper is Rabin-Scott [1959]; see the excellent historical review of the subject in Greibach [1981].)

Lattices are very familiar to logicians and those interested in Universal Algebra. But it takes some time to learn the special terminology and to become comfortable with the necessary examples. Indeed, without a stock of examples, it is impossible to have sufficient intuition for making the required constructions. True, some people took to the approach, and the lattice-theoretic definitions were simple enough to motivate. But a much greater percentage of people Scott came in contact with found the large number of things to learn a definite roadblock to understanding. And the Lattice Theory was only partly standard, since this approach to the theory has to employ complete lattices and topological notions in order to explain *limits* and *continuity* adequately. In fact, Topology of a rather mathematically simple set-theoretical kind could have been used as the whole foundation much as in Scott [1972], but that kind of mathematics seems even less attractive to computer scientists. Nevertheless, the lattice-theoretic approach has great consistency and coherence; it has been carried out in full detail in Gierz, *et al.* [1980], which also gives the topological connections. That book contains a large number of references to the mathematical literature to show the connections of the theory to many other topics. What we had to miss out there is a clear hook-up with computability theory, owing simply to a lack of space and time to be able to cover everything.

In Scott [1981], Scott tried out another, different presentation (which is also explained in Scott [1982].) However, Scott found that the use of *neighborhood systems* causes confusion in people not used to thinking in terms of sets of sets of sets of sets . . . , even though the set theory required is quite simple. This section tries out another plan for making the story easy and natural. The notion to be used we call an *information system*. This is a "static" notion appropriate to bodies of information about coherent groups of elements. The "dynamic" ideas enter in the ways different systems can be related and in the semantical definitions about meanings of programming language constructs. (For discussions of semantical definitions see Stoy [1977], Gordon [1979], and Tennent [1981].) Neighborhood systems are, in a precise sense, equivalent to information systems, but there seems to be an interesting trade-off between what properties of structures are *axiomatized* and what are *implicit* in the form of the structure. Perhaps, despite their simplicity, neighborhoods leave too much implicit, too much to be extracted by definition; information systems, on the other hand, do seem to be more flexible in doing several of the important constructions. It all comes down to just where one imposes the closure conditions on the structures. Information systems give the right mix of axioms *vs.* definitions.

Another word about Category Theory: it is particularly significant and important for the theory and for the whole area of semantics. But it must be approached with great caution, for the sheer number of definitions and axioms can try the most patient reader. It is especially necessary in discussing applications of abstract mathematical ideas to keep the motivation strongly in mind. This is often hard to do if the categories get too thick, but of course it all depends on the writer. Category Theory is especially useful in stating *general* properties of structures and in *characterizing* constructions uniquely; however, there often is a problem actually justifying the *existence* of certain constructions, and a direct approach can be quicker than quoting

lots of theorems. But, man cannot live by construction alone: theorems have to be proved in order to get the proper value out of the work. Domain Theory must also be convenient for demonstrating the soundness of various proof rules for properties of recursively defined objects and recursively defined domains. Category Theory can be helpful here. A step in the right direction has been made in the LCF system (see Gordon-Milner-Wadsworth [1979]), which, however, does not take advantage of general Category Theory.

The main purpose of the Theory of Domains in Denotational Semantics is to give a mathematical model of a system of *types*—including function-space types—and to explain a notion of computability with respect to these types. There are many possible theories of types, but the construction of domains is meant to justify recursive definitions at all types, and—most essentially—to justify recursive definitions of types. Pursuing these goals has certain consequences, as we shall see. The model presented here is only one approach, and some comparisons with other methods will be made. Again some help from Category Theory would be welcome in comparing the different kinds of modelling.

One benefit of Domain Theory is that it is possible to make sense of types containing *infinite* elements. The cost required seems to be that the domains therefore have to contain *partial* elements as well as *total* elements. And this is where the lattice-theoretic definitions entered in the earlier presentations: the partial ordering with relations like $x \sqsubseteq y$ was used to express the fact that the element x was "less defined" (i.e. more partial) than y but "contained in" y. The relation $\sqsubseteq$, however, must be subjected to many axioms to have a theory suitable to the desired applications. Instead of such a big group of axioms, we put forward here a construction where the "skeleton" (or better: "backbone") of a domain is introduced by just a few axioms. Then the domain itself is defined as a certain construct from the backbone in order to define the appropriate notion of element. There are several advantages to this approach:

- Simple definitions of the basic concepts can be given.
- Detailed properties are proved as theorems rather than assumed as axioms.
- Emphasis can be given to the constructive nature of the definitions.
- Domains can be made more visible.
- The theory of domains is made more available for applications, because it is easier to produce the needed complex domains.

4.2 Information systems

Dependencies. Inside InformationSystems, each box is one subfolder, or one Lean file sitting directly in the folder. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. The arrowhead points at the file or folder that imports, and the number is how many direct imports.

Intuitively, an information system is a set of "propositions" that can be made about "possible elements" of the desired domain. We will assume that sufficiently many propositions have been supplied to distinguish between distinct elements; as a consequence, an element can be constructed abstractly as the set of all the propositions that are true of it. Partial elements have "small" sets; while total elements have "large" sets (even: maximal). To make this somewhat rough idea precise, we have to explain—by a suitable, but small choice of axioms—how the collection of all propositions relevant to the domain hangs together, or better, is structured as a set of abstract propositions. Fortunately, the axioms for this structure are very simple and familiar, which is a great help in making up examples.

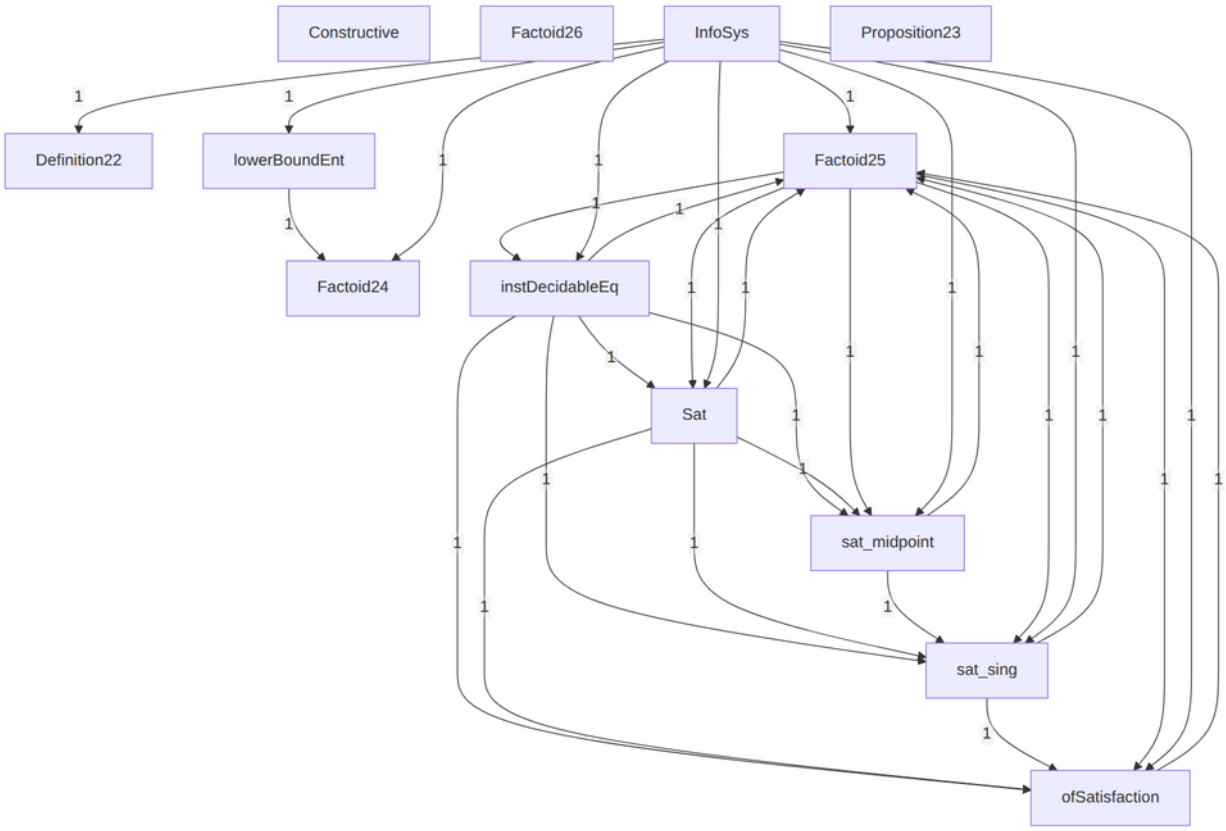


Figure 39: Information systems.

- Definition 2.1
- A first example
- A second example
- A third example
- Definition 2.2
- Proposition 2.3

So much for the definition of what an information system is as a mathematical structure. The next question is: What is it good for? More precisely, if information systems are the backbones of domains, then where are the elements? That is the topic of the next section.

coe_funion.lean

```
module

public import Mathlib.Data.Finset.Basic
public import DomainTheory.InformationSystems.InformationSystems.Constructive.funion
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    decidableEq_finset
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    insert_comm'
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    instLeftCommutativeInsert
public import DomainTheory.InformationSystems.InformationSystems.Constructive.Basic.
    notation5
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    mem_foldr_insert
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    mem_funion
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    funion_empty_right

@[expose] public section

namespace DomainTheory.InformationSystems.Constructive

variable {a : Type*} [DecidableEq a]

/-- The coercion of `u  U ' v` to a `Set` is the (choice-free) set union of the
    coercions. -/
theorem coe_funion (u v : Finset a) :
    ((u  U ' v) : Set a) = (u : Set a)  U  v := by
  ext x
  simp only [Set.mem_union, Finset.mem_coe, mem_funion]

end DomainTheory.InformationSystems.Constructive
```

insert_comm'.lean

```
module

public import Mathlib.Data.Finset.Basic

@[expose] public section

namespace DomainTheory.InformationSystems.Constructive

variable {a : Type*} [DecidableEq a]

/-- Choice-free commutativity of `insert` (mathlib's `Finset.insert_comm` is choice-
    tainted).
Needed to fold `insert` over a `Multiset`. -/
theorem insert_comm' (a b : a) (s : Finset a) :
    insert a (insert b s) = insert b (insert a s) := by
  ext x
  simp only [Finset.mem_insert]
  constructor
  * rintro (h | h | h)
    exacts [Or.inr (Or.inl h), Or.inl h, Or.inr (Or.inr h)]
  * rintro (h | h | h)
    exacts [Or.inr (Or.inl h), Or.inl h, Or.inr (Or.inr h)]

end DomainTheory.InformationSystems.Constructive
```

instLeftCommutativeInsert.lean

```
module

public import Mathlib.Data.Finset.Basic
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    insert_comm'

@[expose] public section

namespace DomainTheory.InformationSystems.Constructive

variable {a : Type*} [DecidableEq a]

instance instLeftCommutativeInsert :
    LeftCommutative (insert : a  ->  Finset a  ->  Finset a) := <insert_comm'>

end DomainTheory.InformationSystems.Constructive
```

funion.lean

```
module

public import Mathlib.Data.Finset.Basic
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    instLeftCommutativeInsert

@[expose] public section

namespace DomainTheory.InformationSystems.Constructive
```

```
variable {a : Type*} [DecidableEq a]

/-- Choice-free binary union of finite sets, obtained by folding `insert` over the
    second
argument's underlying multiset. Definitionally equal in content to `u  U  v`, but --
    unlike
mathlib's `(*  U  *)` -- free of any `Classical.choice` dependency. -/
def funion (u v : Finset a) : Finset a := Multiset.foldr insert u v.1

end DomainTheory.InformationSystems.Constructive
```

notation5.lean

```
module

public import Mathlib.Data.Finset.Basic
public import DomainTheory.InformationSystems.InformationSystems.Constructive.funion

@[expose] public section

namespace DomainTheory.InformationSystems.Constructive

variable {a : Type*} [DecidableEq a]

@[inherit_doc] infixl:65 "  U ' " => funion

end DomainTheory.InformationSystems.Constructive
```

mem_foldr_insert.lean

```
module

public import Mathlib.Data.Finset.Basic
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    instLeftCommutativeInsert
public import DomainTheory.InformationSystems.InformationSystems.Constructive.Basic.
    notation5

@[expose] public section

namespace DomainTheory.InformationSystems.Constructive

variable {a : Type*} [DecidableEq a]

theorem mem_foldr_insert (a : a) (u : Finset a) (s : Multiset a) :
    a  in  Multiset.foldr insert u s  <->  a  in  u  \/  a  in  s := by
  refine Multiset.induction_on s ?_ ?_
  * simp
  * intro b t ih
    simp only [Multiset.foldr_cons, Finset.mem_insert, ih, Multiset.mem_cons]
    constructor
    * rintro (h | h | h)
      exacts [Or.inr (Or.inl h), Or.inl h, Or.inr (Or.inr h)]
    * rintro (h | h | h)
```

```
      exacts [Or.inr (Or.inl h), Or.inl h, Or.inr (Or.inr h)]

end DomainTheory.InformationSystems.Constructive
```

mem_funion.lean

```
module

public import Mathlib.Data.Finset.Basic
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    instLeftCommutativeInsert
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    mem_foldr_insert
public import DomainTheory.InformationSystems.InformationSystems.Constructive.Basic.
    notation5

@[expose] public section

namespace DomainTheory.InformationSystems.Constructive

variable {a : Type*} [DecidableEq a]

@[simp] theorem mem_funion {a : a} {u v : Finset a} :
    a  in  u  U ' v  <->  a  in  u  \/  a  in  v := mem_foldr_insert a u v.1

end DomainTheory.InformationSystems.Constructive
```

funion_empty_right.lean

```
module

public import Mathlib.Data.Finset.Basic
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    instLeftCommutativeInsert
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    mem_funion
public import DomainTheory.InformationSystems.InformationSystems.Constructive.Basic.
    notation5

@[expose] public section

namespace DomainTheory.InformationSystems.Constructive

variable {a : Type*} [DecidableEq a]

@[simp] theorem funion_empty_right (u : Finset a) : u  U ' ({} : Finset a) = u := by
  ext x
  simp only [mem_funion, Finset.notMem_empty, or_false]

end DomainTheory.InformationSystems.Constructive
```

decidableEq_finset.lean

```
module
```

```
public import Mathlib.Data.Finset.Basic
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    instLeftCommutativeInsert
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    mem_funion
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    funion_empty_right
public import DomainTheory.InformationSystems.InformationSystems.Constructive.Basic.
    notation5

@[expose] public section

namespace DomainTheory.InformationSystems.Constructive

variable {a : Type*} [DecidableEq a]

omit [DecidableEq a] in
/-- If mutual subset holds, the finsets are equal. -/
theorem decidableEq_finset.eq_of_subset (s t : Finset a) (h : s  subseteq  t  /\  t
    subseteq  s) : s = t :=
  Finset.Subset.antisymm h.1 h.2

omit [DecidableEq a] in
/-- Mutual subset is required for finset equality in this decidable instance. -/
theorem decidableEq_finset.false_of_ne (s t : Finset a) (h : not (s  subseteq  t  /\
    t  subseteq  s)) (heq : s = t) :
    False := by
  subst heq
  exact h <Finset.Subset.refl _, Finset.Subset.refl _>

/-- Choice-free decidable equality for `Finset`.
mathlib's `Finset.decidableEq` goes through `Multiset` quotients and pulls
`Classical.choice`; this version uses only decidable membership and subset. -/
def decidableEq_finset {a : Type*} [DecidableEq a] : DecidableEq (Finset a) :=
  fun s t =>
    if h : s  subseteq  t  /\  t  subseteq  s then
      isTrue (decidableEq_finset.eq_of_subset s t h)
    else
      isFalse (decidableEq_finset.false_of_ne s t h)

end DomainTheory.InformationSystems.Constructive
```

subset_funion_left.lean

```
module

public import Mathlib.Data.Finset.Basic
public import DomainTheory.InformationSystems.InformationSystems.Constructive.funion
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    decidableEq_finset
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    insert_comm'
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    instLeftCommutativeInsert
```

```
public import DomainTheory.InformationSystems.InformationSystems.Constructive.Basic.
    notation5
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    mem_foldr_insert
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    mem_funion
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    funion_empty_right

@[expose] public section

namespace DomainTheory.InformationSystems.Constructive

variable {a : Type*} [DecidableEq a]

theorem subset_funion_left (u v : Finset a) : u  subseteq  u  U ' v := fun _ hx =>
    mem_funion.2 (Or.inl hx)

end DomainTheory.InformationSystems.Constructive
```

subset_funion_right.lean

```
module

public import Mathlib.Data.Finset.Basic
public import DomainTheory.InformationSystems.InformationSystems.Constructive.funion
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    decidableEq_finset
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    insert_comm'
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    instLeftCommutativeInsert
public import DomainTheory.InformationSystems.InformationSystems.Constructive.Basic.
    notation5
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    mem_foldr_insert
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    mem_funion
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    funion_empty_right

@[expose] public section

namespace DomainTheory.InformationSystems.Constructive

variable {a : Type*} [DecidableEq a]

theorem subset_funion_right (u v : Finset a) : v  subseteq  u  U ' v := fun _ hx =>
    mem_funion.2 (Or.inr hx)

end DomainTheory.InformationSystems.Constructive
```

funion_subset_iff.lean

```
module
```

```
public import Mathlib.Data.Finset.Basic
public import DomainTheory.InformationSystems.InformationSystems.Constructive.funion
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    decidableEq_finset
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    insert_comm'
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    instLeftCommutativeInsert
public import DomainTheory.InformationSystems.InformationSystems.Constructive.Basic.
    notation5
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    mem_foldr_insert
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    mem_funion
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    funion_empty_right
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    subset_funion_left
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    subset_funion_right

@[expose] public section

namespace DomainTheory.InformationSystems.Constructive

variable {a : Type*} [DecidableEq a]

/-- Universal property of the union: `u  U ' v  subseteq  w` iff both `u  subseteq  w`
    and `v  subseteq  w`. -/
theorem funion_subset_iff {u v w : Finset a} : u  U ' v  subseteq  w  <->  u  subseteq
      w  /\  v  subseteq  w := by
  constructor
  * intro h
    exact <fun x hx => h (subset_funion_left u v hx),
           fun x hx => h (subset_funion_right u v hx)>
  * rintro <hu, hv> x hx
    rcases mem_funion.1 hx with h | h
    exacts [hu h, hv h]

end DomainTheory.InformationSystems.Constructive
```

Constructive.lean

```
module

public import Mathlib.Data.Finset.Basic
public import DomainTheory.InformationSystems.InformationSystems.Constructive.funion
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    decidableEq_finset
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    insert_comm'
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    instLeftCommutativeInsert
public import DomainTheory.InformationSystems.InformationSystems.Constructive.Basic.
```

```
    notation5
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    mem_foldr_insert
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    mem_funion
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    funion_empty_right
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    coe_funion
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    subset_funion_left
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    subset_funion_right
public import DomainTheory.InformationSystems.InformationSystems.Constructive.
    funion_subset_iff

@[expose] public section
```

4.3 The elements of a system

Dependencies. Inside ElementsOfASystem, each box is one subfolder, or one Lean file sitting directly in the folder. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. The arrowhead points at the file or folder that imports, and the number is how many direct imports.

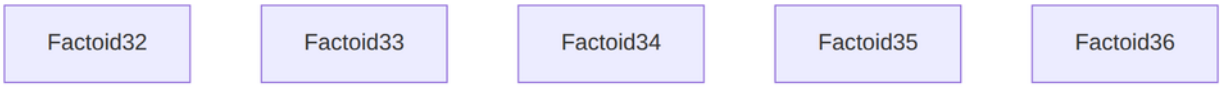


Figure 40: The elements of a system.

Let $\mathbf{A}$ be an information system, and suppose we already had a concept of being an element of $\mathbf{A}$. The data objects are meant to be propositions about elements, so if X is in $D_{\mathbf{A}}$ and if x is an element, then we can be expected to know what it means to say that X is true of x. Since all we are given is the set of data objects $D_{\mathbf{A}}$, we have to assume that it contains enough objects to distinguish between distinct elements. (If there were not enough of them, we would have to change the set $D_{\mathbf{A}}$.) Formally, we can write of two elements x and y:

$$x = y \quad \text{iff all } X \in D_{\mathbf{A}} \text{ which are true of } x \text{ are also true of } y, \text{ and conversely.}$$

If this principle is accepted, then the elements can be *identified* with the sets of propositions true of them; formally we can assert:

$$x = \{X \in D_{\mathbf{A}} \mid X \text{ is true of } x\}.$$

There can be no confusion injected here in identifying elements that ought to be distinct, since we are assuming that there are enough data objects in the system. If for some reason we felt we ought to have more of them to distinguish between more elements, then we would have to pass to a larger and *different* information system. By agreement, then, the above equation is a tautology. So, for the sake of simplicity, elements can be taken to be *sets* of data objects. But, we hasten to ask, which sets? The question is really:

What are the properties of truth? The answers are well known, and, in fact, we have already used them in our examples in the last section. In words, the set of true propositions about a possible element must be (i) consistent in itself, and (ii) closed under entailment (or *deductively closed*, for short). Condition (i) is clear because we are talking about a *possible* element, not an impossible one. Condition (ii) is acceptable, because, by the very meaning of the word, entailment should be truth preserving. In addition, we are also saying the converse: *any* set having properties (i) and (ii) corresponds to a (partial) element. Here is the formal statement:

- Definition 3.1

factoid_3_2.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Proposition23

@[expose] public section

namespace DomainTheory.InformationSystems

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- **Factoid 3.2.** Every element contains `Delta`. -/
theorem factoid_3_2 (x : sys.Element) : sys.bot  in  x.carrier := by
  have hempty : (({} : Finset a) : Set a)  subseteq  x.carrier := by
    intro a ha
    exact False.elim (Finset.notMem_empty a (Finset.mem_coe.1 ha))
  have hEnt : sys.Ent {} sys.bot := sys.ent_bot sys.con_empty
  exact x.closed {} sys.bot hempty hEnt

end InfoSys
end DomainTheory.InformationSystems
```

Factoid32.lean

```
module

public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid32.factoid_3_2

@[expose] public section

namespace DomainTheory.InformationSystems.InfoSys

set_option linter.unusedVariables false

theorem theorem_32.{u_1} : forall  {a : Type u_1} [inst : DecidableEq a] (sys :
    @DomainTheory.InformationSystems.InfoSys a inst)
  (x : @DomainTheory.InformationSystems.InfoSys.Element a inst sys),
  @Membership.mem a (Set a) (@Set.instMembership a)
    (@DomainTheory.InformationSystems.InfoSys.Element.carrier a inst sys x)
```

```
    (@DomainTheory.InformationSystems.InfoSys.bot a inst sys) :=
  @DomainTheory.InformationSystems.InfoSys.factoid_3_2

end DomainTheory.InformationSystems.InfoSys
```

botElement.lean

```
module

public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid32

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- **Factoid 3.3.** The bottom element `False = {X ? {Delta}  |-  X}`. -/
def botElement : sys.Element where
  carrier := {X | sys.Ent {sys.bot} X}
  consistent := by
    intro Y hY
    have hbot : ({sys.bot} : Finset a)  in  sys.Con := sys.con_sing sys.bot
    have hEnt : sys.EntSet {sys.bot} Y := fun X hX => hY (Finset.mem_coe.2 hX)
    have hunion : {sys.bot}  U ' Y  in  sys.Con := proposition_2_3_ii sys hbot hEnt
    exact sys.con_subset hunion (subset_funion_right _ _)
  closed := by
    intro Y a hY hEnt
    have hbot : ({sys.bot} : Finset a)  in  sys.Con := sys.con_sing sys.bot
    have hYcon : Y  in  sys.Con := by
      have hEntY : sys.EntSet {sys.bot} Y := fun X hX => hY (Finset.mem_coe.2 hX)
      exact sys.con_subset (proposition_2_3_ii sys hbot hEntY) (subset_funion_right _
    _)
    exact sys.ent_trans hbot hYcon (fun y hy => hY (Finset.mem_coe.2 hy)) hEnt

/-- `False` is least. -/
theorem botElement.le (x : sys.Element) : sys.botElement  <=  x := by
  intro a ha
  have hsub : (({sys.bot} : Finset a) : Set a)  subseteq  x.carrier := by
    intro b hb
    have hb' : b  in  ({sys.bot} : Finset a) := Finset.mem_coe.1 hb
    rw [Finset.mem_singleton] at hb'
    subst hb'
    exact factoid_3_2 sys x
  exact x.closed {sys.bot} a hsub ha

end InfoSys

end DomainTheory.InformationSystems
```

Factoid33.lean

```
module

public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid33.botElement

@[expose] public section

namespace DomainTheory.InformationSystems.InfoSys

set_option linter.unusedVariables false

theorem theorem_33.{u_1} : forall  {a : Type u_1} [inst : DecidableEq a] (sys :
    @DomainTheory.InformationSystems.InfoSys a inst)
  (x : @DomainTheory.InformationSystems.InfoSys.Element a inst sys),
  @LE.le (@DomainTheory.InformationSystems.InfoSys.Element a inst sys)
    (@Preorder.toLE (@DomainTheory.InformationSystems.InfoSys.Element a inst sys)
      (@PartialOrder.toPreorder (@DomainTheory.InformationSystems.InfoSys.Element a
    inst sys)
        (@DomainTheory.InformationSystems.InfoSys.instPartialOrderElement a inst sys))
    )
    (@DomainTheory.InformationSystems.InfoSys.botElement a inst sys) x :=
  @DomainTheory.InformationSystems.InfoSys.botElement.le

end DomainTheory.InformationSystems.InfoSys
```

AllConsistent.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.InfoSys

@[expose] public section

namespace DomainTheory.InformationSystems

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- Every finite set of tokens is consistent. -/
def AllConsistent : Prop := forall  u : Finset a, u  in  sys.Con

end InfoSys

end DomainTheory.InformationSystems
```

IsTotal.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.InfoSys

@[expose] public section
```

```
namespace DomainTheory.InformationSystems

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- A **total** element is a maximal element of `|A|` under inclusion. -/
def IsTotal (x : sys.Element) : Prop := forall  y, x  <=  y  ->  y  <=  x

end InfoSys

end DomainTheory.InformationSystems
```

eq_of_carrier_univ_of_isTotal.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.InfoSys
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34.IsTotal
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34.
    AllConsistent
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34.topElement

@[expose] public section

namespace DomainTheory.InformationSystems

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- Conversely: any total element equals an element with carrier `D`. -/
theorem eq_of_carrier_univ_of_isTotal {x t : sys.Element}
    (hx : x.carrier = Set.univ) (ht : sys.IsTotal t) : t = x :=
  le_antisymm (fun _ _ => hx  |>  trivial) (ht x (fun _ _ => hx  |>  trivial))

end InfoSys
end DomainTheory.InformationSystems
```

exists_top_iff_allConsistent.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.InfoSys
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34.IsTotal
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34.
    AllConsistent
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34.topElement

@[expose] public section

namespace DomainTheory.InformationSystems
```

```
namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- **Factoid 3.4.** Top ` top  = D` exists as an element iff every finite subset is
    consistent. -/
theorem exists_top_iff_allConsistent :
    (exists  x : sys.Element, x.carrier = Set.univ)  <->  sys.AllConsistent := by
  constructor
  * intro <x, hx> u
    exact x.consistent u (hx  |>  Set.subset_univ _)
  * intro h
    exact <sys.topElement h, rfl>

end InfoSys
end DomainTheory.InformationSystems
```

Factoid34.lean

```
module

public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34.
    exists_top_iff_allConsistent

@[expose] public section

namespace DomainTheory.InformationSystems.InfoSys

set_option linter.unusedVariables false

theorem theorem_34.{u_1} : forall  {a : Type u_1} [inst : DecidableEq a] (sys :
    @DomainTheory.InformationSystems.InfoSys a inst),
  Iff
    (exists  (x : @DomainTheory.InformationSystems.InfoSys.Element a inst sys),
      @Eq (Set a) (@DomainTheory.InformationSystems.InfoSys.Element.carrier a inst sys
     x) (@Set.univ a))
    (@DomainTheory.InformationSystems.InfoSys.AllConsistent a inst sys) :=
  @DomainTheory.InformationSystems.InfoSys.exists_top_iff_allConsistent

end DomainTheory.InformationSystems.InfoSys
```

isTotal_of_carrier_univ.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.InfoSys
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34.IsTotal
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34.
    AllConsistent
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34.topElement

@[expose] public section

namespace DomainTheory.InformationSystems
```

```
namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- An element whose carrier is the whole token set is total (hence the greatest
    element). -/
theorem isTotal_of_carrier_univ {x : sys.Element} (hx : x.carrier = Set.univ) :
    sys.IsTotal x := fun _ _ _ _ => hx  |>  trivial

end InfoSys
end DomainTheory.InformationSystems
```

le_topElement.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.InfoSys
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34.IsTotal
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34.
    AllConsistent
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34.topElement

@[expose] public section

namespace DomainTheory.InformationSystems

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

theorem le_topElement (h : sys.AllConsistent) (x : sys.Element) :
    x  <=  sys.topElement h := fun _ _ => trivial

end InfoSys
end DomainTheory.InformationSystems
```

eq_topElement_of_isTotal.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.InfoSys
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34.IsTotal
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34.
    AllConsistent
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34.topElement
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34.
    le_topElement

@[expose] public section

namespace DomainTheory.InformationSystems
```

```
namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- When ` top ` exists, it is the unique total element. -/
theorem eq_topElement_of_isTotal (h : sys.AllConsistent) {x : sys.Element}
    (hx : sys.IsTotal x) : x = sys.topElement h :=
  le_antisymm (sys.le_topElement h x) (hx _ (sys.le_topElement h x))

end InfoSys
end DomainTheory.InformationSystems
```

topElement.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.InfoSys
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34.
    AllConsistent

@[expose] public section

namespace DomainTheory.InformationSystems

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- **Factoid 3.4.** When every finite set is consistent, the top element is `D` itself
    . -/
def topElement (h : sys.AllConsistent) : sys.Element where
  carrier := Set.univ
  consistent := fun Y _ => h Y
  closed := fun _ _ _ _ => trivial

end InfoSys

end DomainTheory.InformationSystems
```

topElement_isTotal.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.InfoSys
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34.IsTotal
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34.
    AllConsistent
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34.topElement
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34.
    le_topElement

@[expose] public section
```

```
namespace DomainTheory.InformationSystems

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

theorem topElement_isTotal (h : sys.AllConsistent) :
    sys.IsTotal (sys.topElement h) := fun y _ => sys.le_topElement h y

end InfoSys
end DomainTheory.InformationSystems
```

exists_unique_total_of_allConsistent.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.InfoSys
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34.IsTotal
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34.
    AllConsistent
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34.topElement
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34.
    topElement_isTotal
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34.
    eq_topElement_of_isTotal

@[expose] public section

namespace DomainTheory.InformationSystems

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

theorem exists_unique_total_of_allConsistent (h : sys.AllConsistent) :
    exists ! x : sys.Element, sys.IsTotal x :=
  <sys.topElement h, <sys.topElement_isTotal h, fun _ hy =>
    sys.eq_topElement_of_isTotal h hy>>

end InfoSys
end DomainTheory.InformationSystems
```

isTotal_iff_eq_topElement.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.InfoSys
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34.IsTotal
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34.
    AllConsistent
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34.topElement
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34.
    topElement_isTotal
```

```
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34.
    eq_topElement_of_isTotal

@[expose] public section

namespace DomainTheory.InformationSystems

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

theorem isTotal_iff_eq_topElement (h : sys.AllConsistent) {x : sys.Element} :
    sys.IsTotal x  <->  x = sys.topElement h :=
  <sys.eq_topElement_of_isTotal h, fun hx => hx  |>  sys.topElement_isTotal h>

end InfoSys
end DomainTheory.InformationSystems
```

closure.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Proposition23

public import DomainTheory.InformationSystems.InformationSystems.Proposition23.
    proposition_2_3_ii
@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- **Factoid 3.5.** Entailment closure of a consistent set. -/
def closure (u : Finset a) (hu : u  in  sys.Con) : sys.Element where
  carrier := {X | sys.Ent u X}
  consistent := by
    intro Y hY
    have hEnt : sys.EntSet u Y := fun X hX => hY (Finset.mem_coe.2 hX)
    exact sys.con_subset (proposition_2_3_ii sys hu hEnt) (subset_funion_right _ _)
  closed := by
    intro Y a hY hEnt
    have hYcon : Y  in  sys.Con := by
      have hEntY : sys.EntSet u Y := fun X hX => hY (Finset.mem_coe.2 hX)
      exact sys.con_subset (proposition_2_3_ii sys hu hEntY) (subset_funion_right _ _)
    exact sys.ent_trans hu hYcon (fun y hy => hY (Finset.mem_coe.2 hy)) hEnt

end InfoSys

end DomainTheory.InformationSystems
```

mem_closure_iff.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35.closure

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

theorem mem_closure_iff {u : Finset a} (hu : u  in  sys.Con) {X : a} :
    X  in  (sys.closure u hu).carrier  <->  sys.Ent u X := Iff.rfl

end InfoSys
end DomainTheory.InformationSystems
```

subset_closure.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35.closure

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- `u  subseteq  u`. -/
theorem subset_closure {u : Finset a} (hu : u  in  sys.Con) :
    u  subseteq  (sys.closure u hu).carrier :=
  fun _ hX => sys.ent_refl hu (Finset.mem_coe.1 hX)

end InfoSys
end DomainTheory.InformationSystems
```

Factoid35.lean

```
module
```

```
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35.
    subset_closure

@[expose] public section

namespace DomainTheory.InformationSystems.InfoSys

set_option linter.unusedVariables false

theorem theorem_35.{u_1} : forall  {a : Type u_1} [inst : DecidableEq a] (sys :
    @DomainTheory.InformationSystems.InfoSys a inst) {u : Finset a}
  (hu :
    @Membership.mem (Finset a) (Set (Finset a)) (@Set.instMembership (Finset a))
      (@DomainTheory.InformationSystems.InfoSys.Con a inst sys) u),
  @LE.le (Set a) (@Set.instLE a) (@SetLike.coe (Finset a) a (@Finset.instSetLike a) u)
    (@DomainTheory.InformationSystems.InfoSys.Element.carrier a inst sys
      (@DomainTheory.InformationSystems.InfoSys.closure a inst sys u hu)) :=
  @DomainTheory.InformationSystems.InfoSys.subset_closure

end DomainTheory.InformationSystems.InfoSys
```

approxUnion.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- The set-theoretic union of all finite approximations of `x`. -/
def approxUnion (x : sys.Element) : Set a :=
  {a | exists  (u : Finset a) (hu : u  in  sys.Con),
    u  subseteq  x.carrier  /\  a  in  (sys.closure u hu).carrier}

end InfoSys

end DomainTheory.InformationSystems
```

closure_le_element.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
```

```
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid36.approxUnion

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- Finite approximations below `x` really do approximate `x`. -/
theorem closure_le_element (x : sys.Element) {u : Finset a} (hu : u  in  sys.Con)
    (hsub : u  subseteq  x.carrier) : sys.closure u hu  <=  x := by
  intro a ha
  exact x.closed u a hsub ha

end InfoSys
end DomainTheory.InformationSystems
```

closure_le_of_entSet.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid36.approxUnion

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- If `w` entails every member of `u`, then `u  subseteq  w`. -/
theorem closure_le_of_entSet {u w : Finset a} (hu : u  in  sys.Con) (hw : w  in  sys.
    Con)
    (h : sys.EntSet w u) : sys.closure u hu  <=  sys.closure w hw := by
  intro a ha
  exact sys.ent_trans hw hu h ha

end InfoSys
end DomainTheory.InformationSystems
```

closure_le_of_subset.lean

```
module
```

```
public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid36.approxUnion
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid36.
    closure_le_of_entSet

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- Monotonicity of closure under inclusion of consistent sets. -/
theorem closure_le_of_subset {u w : Finset a} (hu : u  in  sys.Con) (hw : w  in  sys.
    Con)
     (hsub : u  subseteq  w) : sys.closure u hu  <=  sys.closure w hw :=
  sys.closure_le_of_entSet hu hw fun _ hy => sys.ent_refl hw (hsub hy)

end InfoSys
end DomainTheory.InformationSystems
```

closures_directed.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid36.approxUnion
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid36.
    closure_le_of_subset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- The family of finite approximations to `x` is directed under ` <= `. -/
theorem closures_directed (x : sys.Element) {u v : Finset a}
    (hu : u  in  sys.Con) (hv : v  in  sys.Con)
    (huX : u  subseteq  x.carrier) (hvX : v  subseteq  x.carrier) :
    exists  (w : Finset a) (hw : w  in  sys.Con),
      w  subseteq  x.carrier  /\
        sys.closure u hu  <=  sys.closure w hw  /\
        sys.closure v hv  <=  sys.closure w hw := by
  let w := u  U ' v
```

```
  have hwX : w  subseteq  x.carrier := by
    intro a ha
    rcases mem_funion.1 (Finset.mem_coe.1 ha) with h | h
    * exact huX (Finset.mem_coe.2 h)
    * exact hvX (Finset.mem_coe.2 h)
  have hw : w  in  sys.Con := x.consistent w hwX
  refine <w, hw, hwX, ?_, ?_>
  * exact sys.closure_le_of_subset hu hw (subset_funion_left u v)
  * exact sys.closure_le_of_subset hv hw (subset_funion_right u v)

end InfoSys
end DomainTheory.InformationSystems
```

mem_element_iff_mem_closure.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid36.approxUnion

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- **Factoid 3.6.** Membership in `x` iff membership in some finite approximation `u
    subseteq  x`. -/
theorem mem_element_iff_mem_closure (x : sys.Element) (a : a) :
    a  in  x.carrier  <->  a  in  sys.approxUnion x := by
  constructor
  * intro ha
    refine <{a}, sys.con_sing a, ?_, ?_>
    * intro b hb
      have hb' : b  in  ({a} : Finset a) := Finset.mem_coe.1 hb
      rw [Finset.mem_singleton] at hb'
      subst hb'
      exact ha
    * exact sys.ent_refl (sys.con_sing a) (Finset.mem_singleton_self a)
  * rintro <u, hu, hsub, ha>
    exact x.closed u a hsub ha

end InfoSys
end DomainTheory.InformationSystems
```

element_eq_approxUnion.lean

```
module
```

```
public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid36.approxUnion
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid36.
    mem_element_iff_mem_closure

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- **Factoid 3.6.** Carrier form: `x = ?{u | u  in  Con, u  subseteq  x}`. -/
theorem element_eq_approxUnion (x : sys.Element) : x.carrier = sys.approxUnion x := by
  ext a
  exact sys.mem_element_iff_mem_closure x a

end InfoSys
end DomainTheory.InformationSystems
```

Factoid36.lean

```
module

public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid36.
    element_eq_approxUnion
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid36.
    mem_element_iff_mem_closure

@[expose] public section

namespace DomainTheory.InformationSystems.InfoSys

set_option linter.unusedVariables false

theorem theorem_36.{u_1} : And
  (forall  {a : Type u_1} [inst : DecidableEq a] (sys : @DomainTheory.
    InformationSystems.InfoSys a inst)
    (x : @DomainTheory.InformationSystems.InfoSys.Element a inst sys),
    @Eq (Set a) (@DomainTheory.InformationSystems.InfoSys.Element.carrier a inst sys x
    )
      (@DomainTheory.InformationSystems.InfoSys.approxUnion a inst sys x))
  (forall  {a : Type u_1} [inst : DecidableEq a] (sys : @DomainTheory.
    InformationSystems.InfoSys a inst)
    (x : @DomainTheory.InformationSystems.InfoSys.Element a inst sys) (a : a),
    Iff
      (@Membership.mem a (Set a) (@Set.instMembership a)
        (@DomainTheory.InformationSystems.InfoSys.Element.carrier a inst sys x) a)
      (@Membership.mem a (Set a) (@Set.instMembership a)
        (@DomainTheory.InformationSystems.InfoSys.approxUnion a inst sys x) a)) :=
```

```
  @And.intro
  (forall  {a : Type u_1} [inst : DecidableEq a] (sys : @DomainTheory.
    InformationSystems.InfoSys a inst)
     (x : @DomainTheory.InformationSystems.InfoSys.Element a inst sys),
     @Eq (Set a) (@DomainTheory.InformationSystems.InfoSys.Element.carrier a inst sys x
     )
       (@DomainTheory.InformationSystems.InfoSys.approxUnion a inst sys x))
  (forall  {a : Type u_1} [inst : DecidableEq a] (sys : @DomainTheory.
    InformationSystems.InfoSys a inst)
     (x : @DomainTheory.InformationSystems.InfoSys.Element a inst sys) (a : a),
     Iff
       (@Membership.mem a (Set a) (@Set.instMembership a)
         (@DomainTheory.InformationSystems.InfoSys.Element.carrier a inst sys x) a)
       (@Membership.mem a (Set a) (@Set.instMembership a)
         (@DomainTheory.InformationSystems.InfoSys.approxUnion a inst sys x) a))
  @DomainTheory.InformationSystems.InfoSys.element_eq_approxUnion
  @DomainTheory.InformationSystems.InfoSys.mem_element_iff_mem_closure

end DomainTheory.InformationSystems.InfoSys
```

Element.lean

```
module

public import Mathlib.Data.Finset.Basic
public import Mathlib.Data.Set.Basic
public import DomainTheory.InformationSystems.InformationSystems.InfoSys.InfoSys

@[expose] public section

universe u

namespace DomainTheory.InformationSystems
namespace InfoSys

variable {a : Type u} [DecidableEq a] (sys : InfoSys a)

/-- An *element* (ideal) of the domain: a set of tokens that is consistent on every
finite subset and closed under entailment. -/
structure Element where
  /-- The underlying set of tokens. -/
  carrier : Set a
  /-- Every finite subset of the element is consistent. -/
  consistent : forall  Y : Finset a, (Y : Set a)  subseteq  carrier  ->  Y  in  sys.
    Con
  /-- The element is closed under entailment. -/
  closed : forall  (Y : Finset a) (a : a), (Y : Set a)  subseteq  carrier  ->  sys.Ent
     Y a  ->  a  in  carrier

end InfoSys
end DomainTheory.InformationSystems
```

Basic.lean

```
module
```

```
public import Mathlib.Data.Finset.Basic
public import Mathlib.Data.Set.Basic
public import DomainTheory.InformationSystems.InformationSystems.InfoSys.InfoSys
public import DomainTheory.InformationSystems.InformationSystems.InfoSys.Element

@[expose] public section

namespace DomainTheory.InformationSystems

universe u

namespace InfoSys

variable {a : Type u} [DecidableEq a] (sys : InfoSys a)

/-- Extensional equality of elements. -/
theorem Element.ext {x y : sys.Element} (h : x.carrier = y.carrier) : x = y := by
  cases x
  cases y
  subst h
  rfl

/-- Reflexivity of the carrier-inclusion order. -/
theorem element_le_refl (x : sys.Element) : x.carrier  subseteq  x.carrier :=
  Set.Subset.refl _

/-- Transitivity of the carrier-inclusion order. -/
theorem element_le_trans (x y z : sys.Element)
    (hxy : x.carrier  subseteq  y.carrier) (hyz : y.carrier  subseteq  z.carrier) :
    x.carrier  subseteq  z.carrier :=
  Set.Subset.trans hxy hyz

/-- Antisymmetry of the carrier-inclusion order. -/
theorem element_le_antisymm (x y : sys.Element)
    (hxy : x.carrier  subseteq  y.carrier) (hyx : y.carrier  subseteq  x.carrier) :
    x = y :=
  Element.ext sys (Set.Subset.antisymm hxy hyx)

/-- Elements are ordered by inclusion of their carriers; this is the Scott ordering.
    -/
instance instPartialOrderElement : PartialOrder sys.Element where
  le x y := x.carrier  subseteq  y.carrier
  le_refl := element_le_refl sys
  le_trans := element_le_trans sys
  le_antisymm := element_le_antisymm sys
end InfoSys
end DomainTheory.InformationSystems
```

con_empty.lean

```
module

public import Mathlib.Data.Finset.Basic
public import Mathlib.Data.Set.Basic
```

```
public import DomainTheory.InformationSystems.InformationSystems.InfoSys.Basic

@[expose] public section

namespace DomainTheory.InformationSystems

universe u

namespace InfoSys

variable {a : Type u} [DecidableEq a] (sys : InfoSys a)

/-- Empty set is consistent (subset of any singleton). -/
theorem con_empty : ({} : Finset a)  in  sys.Con :=
  sys.con_subset (sys.con_sing sys.bot) (Finset.empty_subset _)

end InfoSys
end DomainTheory.InformationSystems
```

InfoSys.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.InfoSys.Basic
public import DomainTheory.InformationSystems.InformationSystems.InfoSys.con_empty

@[expose] public section
```

Basic.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.InfoSys

public import DomainTheory.InformationSystems.InformationSystems.Factoid26.Token
public import DomainTheory.InformationSystems.InformationSystems.Factoid26.Consistent
public import DomainTheory.InformationSystems.InformationSystems.Factoid26.Ent
public import DomainTheory.InformationSystems.InformationSystems.Factoid26.
    partialFunctionSystem
public import DomainTheory.InformationSystems.InformationSystems.Factoid26.
    consistent_singleton
public import DomainTheory.InformationSystems.InformationSystems.Factoid26.
    consistent_subset
public import DomainTheory.InformationSystems.InformationSystems.Factoid26.
    consistent_insert_bot
public import DomainTheory.InformationSystems.InformationSystems.Factoid26.
    consistent_insert_mem

@[expose] public section

namespace DomainTheory.InformationSystems

namespace Factoid26

example : InfoSys (Token Nat Bool) := partialFunctionSystem Nat Bool
```

```
end Factoid26
end DomainTheory.InformationSystems
```

Definition22.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.InfoSys
public import DomainTheory.InformationSystems.InformationSystems.Definition22.EntSet
public import DomainTheory.InformationSystems.InformationSystems.Definition22.
    entSet_empty
public import DomainTheory.InformationSystems.InformationSystems.Definition22.
    entSet_singleton

@[expose] public section
```

4.4 Domains as lattices and as topological spaces

Dependencies. Inside DomainsAsLatticesAndAsTopologicalSpaces, each box is one subfolder, or one Lean file sitting directly in the folder. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. The arrowhead points at the file or folder that imports, and the number is how many direct imports.



Figure 41: Domains as lattices and as topological spaces.

I will attempt to keep this section as informal as possible—especially as Sections 2 and 3 were quite formal (i.e. mathematical) enough. The main purpose of the discussion of this section is to relate the new presentation of the Theory of Domains to previously published ones. (Some alternatives are discussed at the end of the paper.)

Lattice-theoretic considerations. Let $\mathbf{A}$ be an information system. Because the elements of $|\mathbf{A}|$ are introduced as sets, these elements can be given structure from what we know already about ordinary sets. For instance, the set-theoretic inclusion relation between sets can certainly be applied to elements. The question is: What does $x \subseteq y$ mean intuitively? In fact, we have already mentioned this relationship before. It means that every proposition (among the ones given by the information system) true of x is also true of y (though not necessarily conversely). In other words, x is perhaps only partially determined, while y is more fully determined in a way that includes everything true of x. Clearly, then, if we grant the interest of partial elements, then this relation is a natural and basic one. We often read "$x \subseteq y$" as "x approximates y".

Now here is one of the main points of the new exposition: Because $x \subseteq y$ is *defined* in terms of a familiar mathematical notion, then as a relation between elements it inherits all the well-known properties of the set-theoretical relation. It should not be necessary to write them out, for everyone who can read this far knows that $\subseteq$ is in particular reflexive and transitive. We say that the domain $|\mathbf{A}|$ is a *partially ordered set under inclusion*. We note that this (trivial) result is *proved* from the definition rather than assumed as an axiom.

Let us try out another notion. Elements are consistent, deductively closed sets of data objects. Right? Every subset of a consistent set is consistent, but not every subset is closed under entailment, in general. Right? But suppose that x and y are two elements. It can easily be seen that their intersection $x \cap y$ is again consistent and deductively closed (by a 2 nanosecond proof). Therefore, the set of elements of a domain is closed under intersection. This means that, as a partially ordered set, $|\mathbf{A}|$ is an *inf semilattice*.

Well, the exact terminology is quite unimportant, but the properties of $\cap$ when combined with the properties of $\subseteq$ are pleasantly "algebraic" (and run to several lines when written out). We all know them: $\cap$ is idempotent, commutative, and associative. The element $\bot$, being the The element $\bot$, being the smallest element of the domain, is a zero element for $\cap$. The operation $\cap$ is monotonic with respect to $\subseteq$, and we have the connection:

$$x \subseteq y \text{ iff } x \cap y = x.$$

What has just been alluded to in words is the standard mathematical axiomatization of the properties of an infimum (or meet) operation in a partially ordered set. (Remember, however, to have an operation well defined within a set it is necessary also to have closure under that operation.)

What about infs of subsets? No problem. For any *non-empty* subfamily of $|\mathbf{A}|$, it is just as easy to prove that the set-theoretical intersection of all the elements in the subfamily is again an element of the domain. This makes $|\mathbf{A}|$ a (conditionally) complete inf semilattice. Even if you do not have any idea what I am talking about, it does not matter because the properties of the domain are built in by the very definition; you do not have to worry *at the start* about their formulation, for you can prove them when you need them (if ever).

What about suprema (or sups or joins)? Here there is a slight problem, because in the way we set things up they *do not* always have to exist. Consider what happens with just two elements x and y. The set-theoretical union of the two elements as subsets of $D_{\mathbf{A}}$ need be neither consistent nor deductively closed; and even if $x \cup y$ is consistent, it need not be closed under entailment. So if we want sups *within* $|\mathbf{A}|$, we cannot get by as cheaply as we can with a domain of arbitrary sets for which the simple union is the answer.

Let us write the sup (supposing it exists in $|\mathbf{A}|$) as $x \sqcup y$. What do we have when we actually have it? By the lore of partially ordered sets, $x \sqcup y$ must be the *least element* in $|\mathbf{A}|$ which includes (in the sense of $\subseteq$) both x and y. Now we have just spoken about infs of families of elements: they exist if the family is non-empty. This indicates that $x \sqcup y$ exists exactly when there is at least one element z in $|\mathbf{A}|$ such that $x \subseteq z$ and $y \subseteq z$. This turns out to be a way to say—entirely inside $|\mathbf{A}|$—that $x \cup y$ is consistent. In other words (and more generally) a sup of a subfamily exists if, and only if, the union of the family is consistent. (And in that case the sup is just the deductive closure of the union—that is, it is generally larger than the simple set-theoretical union.) This has an axiomatic version, but it is slightly complicated by the fact that sups do not always exist. In case $\top_{\mathbf{A}}$ exists, then we always have all the sups, and $|\mathbf{A}|$ is a *complete lattice*. (This is discussed in full axiomatic splendor in Gierz, et al. [1980].) But, let it hasten to be added, not every complete lattice is isomorphic to a domain. The lattices that correspond to domains are the so-called *algebraic lattices*. (See the discussion loc. cit. for the additional axiomatics required.) When the top is missing, the partially ordered structures corresponding to domains are called *conditionally complete, algebraic cpo's*.

There is, however, an important case in $|\mathbf{A}|$ where the union is consistent and is, in fact, the sup in the domain. Suppose we have a sequence of elements such that

$$x_0 \subseteq x_1 \subseteq x_2 \subseteq \cdots \subseteq x_n \subseteq x_{n+1} \subseteq \cdots$$

As n increases, we can say that the elements x_n are getting "better and better"; thus, they must be approaching something even more desirable *in the limit*. Let

$$y = \bigcup_{n=0}^{\infty} x_n,$$

then there is no question that y is a subset of $D_{\mathbf{A}}$. But is it an element? Consistency, recall, means that every *finite* subset is in $\mathrm{Con}_{\mathbf{A}}$. The trick is that a finite subset of y must be a subset of one of the x_n, because the sequence of elements x_n is increasing. But all the terms of our given sequence are elements, and so, consistent; therefore, every finite subset of y is consistent.

The same argument also shows that the set y of data objects is closed under entailment, because $\vdash_{\mathbf{A}}$ is defined as a relation between finite (consistent) sets and data objects. It follows that since each x_n is a set closed under $\vdash_{\mathbf{A}}$, then so is y. In other words, y is indeed an element. Otherwise said, the domain $|\mathbf{A}|$ is closed under the formation of unions of increasing chains of elements, and the union is, of course, the sup in the sense of the partial ordering. Again we have proved closure rather than assuming it, and the necessary properties of the sup follow from its definition as a union.

This last argument can be generalized—as is well known to mathematicians—to the case of *directed* families of elements. The word "directed," or better "upward-directed," means that every finite number of elements in the family is included in some further element of the family; chains always have this property. A good example of the use of directed sets of elements figures in the discussion of the *finite elements* of the domain.

As we have remarked several times, any consistent set of data objects *generates* an element by closing it up under the entailment relation; thus, in particular, any set $u \in \mathrm{Con}$ generates an element. Let us write

$$\overline{u} = \{X \in D_{\mathbf{A}} \mid u \vdash_{\mathbf{A}} X\}$$

for the *closure* of u under entailment. (This notation could be used for any consistent subset of $D_{\mathbf{A}}$, but the case of finitely generated elements is especially important.) Such a $\overline{u}$ is always an element of $|\mathbf{A}|$; the totality of such elements form, by definition, the *finite elements* of the domain. The notation $\overline{u}$ should of course be decorated with a subscript $\mathbf{A}$, but, as there is no good place to write it in, we leave it out.

We have for all elements x of the domain the basic formula:

$$x = \bigcup \{\overline{u} \mid u \in \mathrm{Con}_{\mathbf{A}} \text{ and } u \subseteq x\}.$$

This can be read intuitively as saying that every element of the domain is the *limit* of its *finite approximations*. And, having used the word "limit" so often, we ought now to say something about how to make the meaning of this word correct mathematically. Note that in the limit representation for x, the union is a directed one. (Why?)

Topological considerations. Geometrically speaking, a topological space is a collection of "points" which group themselves in various "neighborhoods" providing thereby a sense of "nearness." More specifically, a neighborhood of a point in a *metric space* consists of all those points within a certain positive "distance" from the given point; that is, nearness is limited, but it is not pushed to *zero*. The major reason for leaving some "breathing space" around the point comes out in defining continuity of functions.

A function f from one topological space to another is said to be *continuous* provided that, for every point x of the first space, and for every neighborhood in the second space around $f(x)$, it is possible to find—in the first space—a neighborhood around x with the following property. If the function is restricted to this neighborhood, then its values lie entirely in the given neighborhood around $f(x)$. In the metric case we can

say that if we do not want the function to wander any great distance from $f(x)$, then there is a restricted distance around x giving a neighborhood over which the function values stay as close to $f(x)$ as specified. This keeps the function from jumping around wildly, since small variations in x lead to only small variations in $f(x)$.

The intuition about continuity is no doubt very clear in the more geometric examples, and geometric intuition can carry us quite far. But the spaces obtained from domain theory are not really geometric spaces, so we need to make some shifts in our metaphors. The geometric notion of a point usually implies that two points are perfectly distinguishable: if x and y are different, then they have disjoint neighborhoods. This is certainly true in the metric case, where, if two points are different, then they are at a positive distance apart. Early on in the study of topological spaces, however, it was found that not all spaces were metric, and weaker separation properties of pairs of distinct points were uncovered. One of the very weak versions of such a condition is called the "T_0-axiom." The best way to put it is that if two points have the same neighborhoods, then they are the same. The contrapositive statement sounds odd: if two points are distinct, then there is a neighborhood that contains one but not the other. The oddity of this statement lies in the feeling that you cannot make up your mind over which is the better point.

In our domains there is already a notion of "betterness" which proves to be very closely related to the implicit neighborhood structure. The reason why our domains, as spaces of points, are not "geometric" in the familiar sense is that they contain partial elements. A geometric point, on the other hand, is "perfect" or totally determined; it cannot be made better than it already is. The metric in common spaces is measuring something other than betterness, for there are competing notions of approximation afoot here. Thus, we must keep our special goals in defining domains clearly in mind to avoid confusion.

Consider an information system $\mathbf{A}$. For each $u \in \mathrm{Con}_{\mathbf{A}}$, we define a corresponding neighborhood of $|\mathbf{A}|$ by the equation:

$$[u]_{\mathbf{A}} = \{x \in |\mathbf{A}| \mid u \subseteq x\}.$$

The neighborhoods of an element x are all those sets $[u]_{\mathbf{A}}$, where $u \subseteq x$. Note that this is the same as saying $\overline{u} \subseteq x$ or $\overline{u}$ approximates x. Indeed, the neighborhoods are in a one-one correspondence with the finite elements of $|\mathbf{A}|$. A neighborhood collects together all those elements sharing the same finite amount of information.

If both $u \subseteq x$ and $v \subseteq x$, then it is easy to see that the intersection of the two neighborhoods is again a neighborhood of x by virtue of the formula:

$$[u]_{\mathbf{A}} \cap [v]_{\mathbf{A}} = [u \cup v]_{\mathbf{A}}.$$

In case u and v are inconsistent with each other, then the intersection will be the empty set $\emptyset$. Because every element of our domain $|\mathbf{A}|$ is the union of the finite elements it contains, it is trivial to prove that two elements with exactly the same neighborhoods are the same. For these reasons, then, it follows that $|\mathbf{A}|$ is a T_0-topological space. The immediate reaction to this intelligence is: So what?

The answer to this scepticism will appear in the best light when we discuss in the next section the notion of function or mapping appropriate to our domains. The argument will be based on the extremely elementary form of the definition that characterizes the general continuous function together with the fact that the geometric intuition is suggestive of theorems to prove. In Section 6 we will find that the space of all continuous functions between two given domains is also a domain, a most useful result that is interesting topologically. But further interest for Domain Theory comes from the circumstance that there is a connection between geometry and domains. We cannot go into the full details here, but I can make some brief remarks.

Before continuing that discussion, though, it should be remarked that once the topology of the domain has been uncovered, then the unions of chains of elements (or directed sets of elements) are topological limits according to accepted general definitions. Thus, what we felt intuitively was a limiting process (the getting better and better up to the union) is in fact a limit formation in a precise mathematical sense. This helps convince us that we are on the right track.

Total elements as a space. In any domain it can be proved that any element x can be extended to a *total element* t, which is characterized by the property that it is a *maximal* element of the partial ordering. (Perhaps there are many such t, but there is always at least one with $x \subseteq t$.) Recall that the set of all total elements of $|\mathbf{A}|$ is denoted by $\mathrm{Tot}_{\mathbf{A}}$. Because the set $\mathrm{Tot}_{\mathbf{A}} \subseteq |\mathbf{A}|$, it is a topological space itself. Suppose s and t are two distinct total elements, then they must be inconsistent with each other—but we shall prove more. If every finite consistent $u \subseteq s$ also satisfied $u \subseteq t$, then $s \subseteq t$ would follow. But this is impossible, because s is maximal. Let, then, $u \in \mathrm{Con}_{\mathbf{A}}$ with $u \subseteq s$ be chosen so that $u \subseteq t$ fails. Now u must be inconsistent with t, since otherwise there would be an element containing them both. It follows that u must be inconsistent with some $v \in \mathrm{Con}_{\mathbf{A}}$ with $v \subseteq t$. In this way we show that the neighborhoods $[u]_{\mathbf{A}}$ and $[v]_{\mathbf{A}}$ are disjoint.

This argument proves that $\mathrm{Tot}_{\mathbf{A}}$ is a *Hausdorff space* (or T_2-space). We cannot go further into the details here, but it is also a totally disconnected, compact Hausdorff space. (These spaces—well known from studies of Logic and the Theory of Boolean Algebras—are also zero-dimensional.) Assuming the countability of $\mathrm{Con}_{\mathbf{A}}$, which is not such a bold assumption, all such spaces can be conveniently embedded into the real line so as to preserve the topological structure. Perhaps this does not seem so surprising, but it shows that the topological nature of $\mathrm{Tot}_{\mathbf{A}}$, the uppermost level of the T_0 space $|\mathbf{A}|$, is indeed quite geometric. And it can also be shown that any arbitrary continuous function on $\mathrm{Tot}_{\mathbf{A}}$ into *any* topological space can be extended to a continuous function on $|\mathbf{A}|$, so the whole domain proves to be quite a "roomy" space with many good connections to more usual topological spaces.

A note on neighborhood structures. Topological spaces can be defined in several ways, and in general they do not carry a preferred neighborhood structure. (For example, there is no topological significance to the usual rational intervals we use to define neighborhoods for the real line: any similarly dense set of points would do in place of the rational numbers.) Domains, the way we have defined them, however, do have preferred neighborhoods. The set of finite elements of $|\mathbf{A}|$ can be defined topologically, and as we have seen they determine a convenient set of neighborhoods. (The reason for all this is the so-called zero-dimensional character of the domains. There is a higher-dimensional theory, but this leads to continuous lattices and can only be explained using the kind of presentation in Gierz, *et al.* [1980]. It is too long-winded a story for the present paper. The higher-dimensional domains do not have a preferred set of neighborhoods, however.)

Having realized this, it is a short step to giving an algebraic axiomatization of the neighborhood structure. Smyth [1975] contains a presentation in terms of the equivalent notion of

inf.lean

```
module

public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid33

@[expose] public section

namespace DomainTheory.InformationSystems
namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)
```

```
/-- **Factoid 4.1.** Binary meet of elements is set-theoretic intersection. -/
def inf (x y : sys.Element) : sys.Element where
  carrier := x.carrier  I  y.carrier
  consistent := fun Y hY =>
    x.consistent Y (Set.Subset.trans hY Set.inter_subset_left)
  closed := fun Y a hY hEnt =>
    <x.closed Y a (Set.Subset.trans hY Set.inter_subset_left) hEnt,
     y.closed Y a (Set.Subset.trans hY Set.inter_subset_right) hEnt>

theorem inf.le_left (x y : sys.Element) : sys.inf x y  <=  x :=
  Set.inter_subset_left

theorem inf.le_right (x y : sys.Element) : sys.inf x y  <=  y :=
  Set.inter_subset_right

end InfoSys
end DomainTheory.InformationSystems
```

le_iff_inf_eq.lean

```
module

public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid33
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid41.inf

@[expose] public section

namespace DomainTheory.InformationSystems

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- **Factoid 4.1.** `x  subseteq  y` iff `x  I  y = x`. -/
theorem le_iff_inf_eq {x y : sys.Element} : x  <=  y  <->  sys.inf x y = x := by
  constructor
  * intro h
    refine le_antisymm (inf.le_left sys x y) ?_
    intro a ha
    exact <ha, h ha>
  * intro heq
    have hc : x.carrier = x.carrier  I  y.carrier :=
      Eq.symm (congrArg Element.carrier heq)
    intro a ha
    exact ((Set.ext_iff.1 hc a).1 ha).2

end InfoSys
end DomainTheory.InformationSystems
```

Factoid41.lean

```
module
```

```
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid41.le_iff_inf_eq

@[expose] public section

namespace DomainTheory.InformationSystems.InfoSys

set_option linter.unusedVariables false

theorem theorem_41.{u_1} : forall  {a : Type u_1} [inst : DecidableEq a] (sys :
    @DomainTheory.InformationSystems.InfoSys a inst)
  {x y : @DomainTheory.InformationSystems.InfoSys.Element a inst sys},
  Iff
    (@LE.le (@DomainTheory.InformationSystems.InfoSys.Element a inst sys)
      (@Preorder.toLE (@DomainTheory.InformationSystems.InfoSys.Element a inst sys)
        (@PartialOrder.toPreorder (@DomainTheory.InformationSystems.InfoSys.Element a
    inst sys)
          (@DomainTheory.InformationSystems.InfoSys.instPartialOrderElement a inst sys
    )))
      x y)
    (@Eq (@DomainTheory.InformationSystems.InfoSys.Element a inst sys)
      (@DomainTheory.InformationSystems.InfoSys.inf a inst sys x y) x) :=
  @DomainTheory.InformationSystems.InfoSys.le_iff_inf_eq

end DomainTheory.InformationSystems.InfoSys
```

botElement_inf.lean

```
module

public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid33
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid41.inf
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid41.le_iff_inf_eq

@[expose] public section

namespace DomainTheory.InformationSystems

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- `False` is a zero for meet: `False  I  x = False`. -/
theorem botElement_inf (x : sys.Element) : sys.inf sys.botElement x = sys.botElement
    :=
  (sys.le_iff_inf_eq (x := sys.botElement) (y := x)).1 (botElement.le sys x)

end InfoSys
end DomainTheory.InformationSystems
```

inf_idem.lean

```
module
```

```
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid33
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid41.inf
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid41.le_iff_inf_eq

@[expose] public section

namespace DomainTheory.InformationSystems

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

theorem inf_idem (x : sys.Element) : sys.inf x x = x :=
  (sys.le_iff_inf_eq (y := x)).1 (le_refl x)

end InfoSys
end DomainTheory.InformationSystems
```

le_inf.lean

```
module

public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid33
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid41.inf

@[expose] public section

namespace DomainTheory.InformationSystems

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

theorem le_inf {x y z : sys.Element} (hxy : x  <=  y) (hxz : x  <=  z) : x  <=  sys.
    inf y z :=
  fun _ ha => <hxy ha, hxz ha>

end InfoSys
end DomainTheory.InformationSystems
```

inf_assoc.lean

```
module

public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid33
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid41.inf
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid41.le_inf

@[expose] public section
```

```
namespace DomainTheory.InformationSystems

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

theorem inf_assoc (x y z : sys.Element) : sys.inf (sys.inf x y) z = sys.inf x (sys.inf
     y z) := by
  refine le_antisymm ?_ ?_
  * refine sys.le_inf ?_ (sys.le_inf ?_ ?_)
    * exact le_trans (inf.le_left sys _ _) (inf.le_left sys _ _)
    * exact le_trans (inf.le_left sys _ _) (inf.le_right sys _ _)
    * exact inf.le_right sys _ _
  * refine sys.le_inf (sys.le_inf ?_ ?_) ?_
    * exact inf.le_left sys _ _
    * exact le_trans (inf.le_right sys _ _) (inf.le_left sys _ _)
    * exact le_trans (inf.le_right sys _ _) (inf.le_right sys _ _)

end InfoSys
end DomainTheory.InformationSystems
```

inf_comm.lean

```
module

public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid33
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid41.inf
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid41.le_inf

@[expose] public section

namespace DomainTheory.InformationSystems

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

theorem inf_comm (x y : sys.Element) : sys.inf x y = sys.inf y x := by
  refine le_antisymm ?_ ?_
  * exact sys.le_inf (inf.le_right sys x y) (inf.le_left sys x y)
  * exact sys.le_inf (inf.le_right sys y x) (inf.le_left sys y x)

end InfoSys
end DomainTheory.InformationSystems
```

inf_botElement.lean

```
module

public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid33
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid41.inf
```

```
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid41.inf_comm
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid41.botElement_inf

@[expose] public section

namespace DomainTheory.InformationSystems

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

theorem inf_botElement (x : sys.Element) : sys.inf x sys.botElement = sys.botElement
    := by
  rw [sys.inf_comm, sys.botElement_inf]

end InfoSys
end DomainTheory.InformationSystems
```

inf_le_iff.lean

```
module

public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid33
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid41.inf
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid41.le_inf

@[expose] public section

namespace DomainTheory.InformationSystems

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- Meet is the greatest lower bound. -/
theorem inf_le_iff {x y z : sys.Element} : z  <=  sys.inf x y  <->  z  <=  x  /\  z
    <=  y :=
  <fun h => <le_trans h (inf.le_left sys x y), le_trans h (inf.le_right sys x y)>,
   fun <hx, hy> => sys.le_inf hx hy>

end InfoSys
end DomainTheory.InformationSystems
```

inf_mono.lean

```
module

public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid33
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid41.inf
```

```
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid41.le_inf

@[expose] public section

namespace DomainTheory.InformationSystems

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

theorem inf_mono {x_1 x_2 y_1 y_2 : sys.Element} (hx : x_1  <=  x_2) (hy : y_1  <=
    y_2) :
    sys.inf x_1 y_1  <=  sys.inf x_2 y_2 :=
  sys.le_inf (le_trans (inf.le_left sys _ _) hx) (le_trans (inf.le_right sys _ _) hy)

end InfoSys
end DomainTheory.InformationSystems
```

familyInfCarrier.lean

```
module

public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid41

@[expose] public section

namespace DomainTheory.InformationSystems

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- Carrier intersection of a family of elements. -/
def familyInfCarrier (S : Set sys.Element) : Set a :=
  {a | forall  x  in  S, a  in  x.carrier}

end InfoSys

end DomainTheory.InformationSystems
```

familyInf.lean

```
module

public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid41
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid42.familyInfCarrier

@[expose] public section

namespace DomainTheory.InformationSystems
namespace InfoSys
```

```
variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- **Factoid 4.2.** Meet of a nonempty family of elements (set-theoretic intersection
    ). -/
def familyInf (S : Set sys.Element) (hne : S.Nonempty) : sys.Element where
  carrier := sys.familyInfCarrier S
  consistent := by
    intro Y hY
    obtain <x_0, hx_0> := hne
    have hYx : (Y : Set a)  subseteq  x_0.carrier := fun a ha => (hY ha) x_0 hx_0
    exact x_0.consistent Y hYx
  closed := by
    intro Y a hY hEnt x hx
    have hYx : (Y : Set a)  subseteq  x.carrier := fun b hb => (hY hb) x hx
    exact x.closed Y a hYx hEnt

theorem familyInf.le (S : Set sys.Element) (hne : S.Nonempty) {x : sys.Element}
    (hx : x  in  S) : sys.familyInf S hne  <=  x :=
  fun _a ha => ha x hx

end InfoSys
end DomainTheory.InformationSystems
```

le_familyInf.lean

```
module

public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid41
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid42.familyInfCarrier
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid42.familyInf

@[expose] public section

namespace DomainTheory.InformationSystems

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

theorem le_familyInf (S : Set sys.Element) (hne : S.Nonempty) {z : sys.Element}
    (h : forall  x  in  S, z  <=  x) : z  <=  sys.familyInf S hne :=
  fun _a ha x hx => h x hx ha

end InfoSys
end DomainTheory.InformationSystems
```

familyInf_le_iff.lean

```
module
```

```
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid41
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid42.familyInfCarrier
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid42.familyInf
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid42.le_familyInf

@[expose] public section

namespace DomainTheory.InformationSystems

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- `familyInf` is the greatest lower bound of the family. -/
theorem familyInf.le_iff (S : Set sys.Element) (hne : S.Nonempty) {z : sys.Element} :
    z  <=  sys.familyInf S hne  <->  forall  x  in  S, z  <=  x :=
  <fun hz _x hx => le_trans hz (familyInf.le sys S hne hx), sys.le_familyInf S hne>

end InfoSys
end DomainTheory.InformationSystems
```

familyInf_pair.lean

```
module

public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid41
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid42.familyInfCarrier
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid42.familyInf
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid42.le_familyInf

public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid41.inf
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid41.le_inf
@[expose] public section

namespace DomainTheory.InformationSystems

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- Binary meet agrees with the two-element family meet. -/
theorem familyInf_pair (x y : sys.Element) :
    sys.familyInf ({x, y} : Set sys.Element) <x, Set.mem_insert x {y}> = sys.inf x y
    := by
  refine le_antisymm ?_ ?_
```

```
  * exact sys.le_inf
      (familyInf.le sys _ _ (Set.mem_insert x {y}))
      (familyInf.le sys _ _ (Set.mem_insert_of_mem _ rfl))
  * apply sys.le_familyInf
    intro z hz
    rcases Set.mem_insert_iff.mp hz with rfl | hy
    * exact inf.le_left sys _ _
    * have : z = y := hy
      subst this
      exact inf.le_right sys _ _

end InfoSys
end DomainTheory.InformationSystems
```

Factoid42.lean

```
module

public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid42.familyInf_pair

@[expose] public section

namespace DomainTheory.InformationSystems.InfoSys

set_option linter.unusedVariables false

theorem theorem_42.{u_1} : forall  {a : Type u_1} [inst : DecidableEq a] (sys :
    @DomainTheory.InformationSystems.InfoSys a inst)
  (x y : @DomainTheory.InformationSystems.InfoSys.Element a inst sys),
  @Eq (@DomainTheory.InformationSystems.InfoSys.Element a inst sys)
    (@DomainTheory.InformationSystems.InfoSys.familyInf a inst sys
      (@Insert.insert (@DomainTheory.InformationSystems.InfoSys.Element a inst sys)
        (Set (@DomainTheory.InformationSystems.InfoSys.Element a inst sys))
        (@Set.instInsert (@DomainTheory.InformationSystems.InfoSys.Element a inst sys)
    ) x
        (@Singleton.singleton (@DomainTheory.InformationSystems.InfoSys.Element a inst
     sys)
          (Set (@DomainTheory.InformationSystems.InfoSys.Element a inst sys))
          (@Set.instSingletonSet (@DomainTheory.InformationSystems.InfoSys.Element a
    inst sys)) y))
      (@Exists.intro (@DomainTheory.InformationSystems.InfoSys.Element a inst sys)
        (fun (x_1 : @DomainTheory.InformationSystems.InfoSys.Element a inst sys) =>
          @Membership.mem (@DomainTheory.InformationSystems.InfoSys.Element a inst sys
    )
            (Set (@DomainTheory.InformationSystems.InfoSys.Element a inst sys))
            (@Set.instMembership (@DomainTheory.InformationSystems.InfoSys.Element a
    inst sys))
            (@Insert.insert (@DomainTheory.InformationSystems.InfoSys.Element a inst
    sys)
              (Set (@DomainTheory.InformationSystems.InfoSys.Element a inst sys))
              (@Set.instInsert (@DomainTheory.InformationSystems.InfoSys.Element a
    inst sys)) x
              (@Singleton.singleton (@DomainTheory.InformationSystems.InfoSys.Element
    a inst sys)
```

```
              (Set (@DomainTheory.InformationSystems.InfoSys.Element a inst sys))
              (@Set.instSingletonSet (@DomainTheory.InformationSystems.InfoSys.
    Element a inst sys)) y))
            x_1)
        x
        (@Set.mem_insert (@DomainTheory.InformationSystems.InfoSys.Element a inst sys)
     x
          (@Singleton.singleton (@DomainTheory.InformationSystems.InfoSys.Element a
    inst sys)
            (Set (@DomainTheory.InformationSystems.InfoSys.Element a inst sys))
            (@Set.instSingletonSet (@DomainTheory.InformationSystems.InfoSys.Element a
     inst sys)) y))))
    (@DomainTheory.InformationSystems.InfoSys.inf a inst sys x y) :=
  @DomainTheory.InformationSystems.InfoSys.familyInf_pair

end DomainTheory.InformationSystems.InfoSys
```

IsFinitelyConsistent.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid42

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- A (possibly infinite) set of tokens is *finitely consistent* when every finite
    subset
lies in `Con`. -/
def IsFinitelyConsistent (U : Set a) : Prop :=
  forall  Y : Finset a, (Y : Set a)  subseteq  U  ->  Y  in  sys.Con

end InfoSys

end DomainTheory.InformationSystems
```

exists_entSet_of_subset_deductive.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid42
```

```
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.IsFinitelyConsistent

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- Witness: some finite `w  subseteq  U` entails every member of `Y  subseteq
    deductiveClosure U`. -/
theorem exists_entSet_of_subset_deductive
    (U : Set a) (hU : sys.IsFinitelyConsistent U) (Y : Finset a)
    (hY : (Y : Set a)  subseteq  {a | exists  u : Finset a, (u : Set a)  subseteq  U
    /\  sys.Ent u a}) :
    exists  w : Finset a, (w : Set a)  subseteq  U  /\  sys.EntSet w Y := by
  induction Y using Finset.induction_on with
  | empty =>
    exact <{}, fun _ ha => False.elim (Finset.notMem_empty _ (Finset.mem_coe.1 ha)),
      fun _ ha => False.elim (Finset.notMem_empty _ ha)>
  | insert a s _ha ih =>
    have ha' : exists  u : Finset a, (u : Set a)  subseteq  U  /\  sys.Ent u a :=
      hY (Finset.mem_coe.2 (Finset.mem_insert_self a s))
    have hs' : (s : Set a)  subseteq  {b | exists  u : Finset a, (u : Set a)  subseteq
      U  /\  sys.Ent u b} := by
      intro b hb
      exact hY (Finset.mem_coe.2 (Finset.mem_insert_of_mem (Finset.mem_coe.1 hb)))
    obtain <ua, hua, hEnta> := ha'
    obtain <us, hus, hEnts> := ih hs'
    refine <ua  U ' us, ?_, ?_>
    * intro z hz
      rcases mem_funion.mp (Finset.mem_coe.1 hz) with h | h
      * exact hua (Finset.mem_coe.2 h)
      * exact hus (Finset.mem_coe.2 h)
    * intro b hb
      have hwU : (ua  U ' us : Set a)  subseteq  U := by
        intro z hz
        rcases mem_funion.mp (Finset.mem_coe.1 hz) with h | h
        * exact hua (Finset.mem_coe.2 h)
        * exact hus (Finset.mem_coe.2 h)
      have hw : ua  U ' us  in  sys.Con := hU _ hwU
      rcases Finset.mem_insert.mp hb with rfl | hb'
      * exact sys.ent_trans hw (hU ua hua)
          (fun y hy => sys.ent_refl hw (mem_funion.mpr (Or.inl hy))) hEnta
      * exact sys.ent_trans hw (hU us hus)
          (fun y hy => sys.ent_refl hw (mem_funion.mpr (Or.inr hy))) (hEnts b hb')

end InfoSys

end DomainTheory.InformationSystems
```

deductiveClosure.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid42
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.IsFinitelyConsistent
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.exists_entSet_of_subset_deductive

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- Deductive closure of a finitely consistent set. -/
def deductiveClosure (U : Set a) (hU : sys.IsFinitelyConsistent U) : sys.Element where
  carrier := {a | exists  u : Finset a, (u : Set a)  subseteq  U  /\  sys.Ent u a}
  consistent := by
    intro Y hY
    obtain <w, hwU, hEnt> := sys.exists_entSet_of_subset_deductive U hU Y hY
    have hw : w  in  sys.Con := hU w hwU
    exact sys.con_subset (proposition_2_3_ii sys hw hEnt) (subset_funion_right _ _)
  closed := by
    intro Y a hY hEnt
    obtain <w, hwU, hEntY> := sys.exists_entSet_of_subset_deductive U hU Y hY
    have hw : w  in  sys.Con := hU w hwU
    have hYcon : Y  in  sys.Con :=
      sys.con_subset (proposition_2_3_ii sys hw hEntY) (subset_funion_right _ _)
    exact <w, hwU, sys.ent_trans hw hYcon hEntY hEnt>

end InfoSys

end DomainTheory.InformationSystems
```

familySup.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid42
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.IsFinitelyConsistent
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.deductiveClosure
```

```
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.familyUnionCarrier

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- Join of a family whose carrier-union is finitely consistent. -/
def familySup (S : Set sys.Element)
    (hU : sys.IsFinitelyConsistent (sys.familyUnionCarrier S)) : sys.Element :=
  sys.deductiveClosure (sys.familyUnionCarrier S) hU

theorem familySup.le (S : Set sys.Element)
    (hU : sys.IsFinitelyConsistent (sys.familyUnionCarrier S)) {z : sys.Element}
    (h : forall  x  in  S, x  <=  z) : sys.familySup S hU  <=  z := by
  intro a ha
  obtain <u, huU, hEnt> := ha
  have huZ : (u : Set a)  subseteq  z.carrier := by
    intro b hb
    obtain <x, hx, hb'> := huU hb
    exact h x hx hb'
  exact z.closed u a huZ hEnt

end InfoSys

end DomainTheory.InformationSystems
```

familyUnionCarrier.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid42

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- Carrier union of a family of elements. -/
def familyUnionCarrier (S : Set sys.Element) : Set a :=
  {a | exists  x  in  S, a  in  x.carrier}
```

```
end InfoSys

end DomainTheory.InformationSystems
```

finitelyConsistent_of_upperBound.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid42
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.IsFinitelyConsistent
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.deductiveClosure
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.familyUnionCarrier
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.familySup
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.join
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.exists_entSet_of_subset_deductive

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- An upper bound of `S` forces the carrier-union to be finitely consistent. -/
theorem finitelyConsistent_of_upperBound (S : Set sys.Element) {z : sys.Element}
    (h : forall  x  in  S, x  <=  z) : sys.IsFinitelyConsistent (sys.
    familyUnionCarrier S) := by
  intro Y hY
  have hYZ : (Y : Set a)  subseteq  z.carrier := by
    intro a ha
    obtain <x, hx, ha'> := hY ha
    exact h x hx ha'
  exact z.consistent Y hYZ

end InfoSys
end DomainTheory.InformationSystems
```

join.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
```

```
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid42
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.IsFinitelyConsistent
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.deductiveClosure

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- Binary join via deductive closure of `x.carrier  U  y.carrier`, when consistent. -/
def join (x y : sys.Element)
    (h : sys.IsFinitelyConsistent (x.carrier  U  y.carrier)) : sys.Element :=
  sys.deductiveClosure (x.carrier  U  y.carrier) h

theorem join.le (x y z : sys.Element)
    (h : sys.IsFinitelyConsistent (x.carrier  U  y.carrier))
    (hx : x  <=  z) (hy : y  <=  z) : sys.join x y h  <=  z := by
  intro a ha
  obtain <u, huU, hEnt> := ha
  have huZ : (u : Set a)  subseteq  z.carrier := by
    intro b hb
    rcases huU hb with hb' | hb'
    * exact hx hb'
    * exact hy hb'
  exact z.closed u a huZ hEnt

end InfoSys

end DomainTheory.InformationSystems
```

subset_deductiveClosure.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid42
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.IsFinitelyConsistent
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.deductiveClosure
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.familyUnionCarrier
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.familySup
```

```
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.join
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.exists_entSet_of_subset_deductive

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

theorem subset_deductiveClosure (U : Set a) (hU : sys.IsFinitelyConsistent U) :
    U  subseteq  (sys.deductiveClosure U hU).carrier := by
  intro a ha
  refine <{a}, ?_, sys.ent_refl (hU {a} ?_) (Finset.mem_singleton_self a)>
  * intro b hb
    have : b = a := Finset.mem_singleton.mp (Finset.mem_coe.1 hb)
    subst this
    exact ha
  * intro b hb
    have : b = a := Finset.mem_singleton.mp (Finset.mem_coe.1 hb)
    subst this
    exact ha

end InfoSys
end DomainTheory.InformationSystems
```

le_familySup.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid42
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.IsFinitelyConsistent
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.deductiveClosure
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.familyUnionCarrier
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.familySup
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.join
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.exists_entSet_of_subset_deductive
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.subset_deductiveClosure

@[expose] public section
```

```
namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

theorem le_familySup (S : Set sys.Element)
    (hU : sys.IsFinitelyConsistent (sys.familyUnionCarrier S)) {x : sys.Element}
    (hx : x  in  S) : x  <=  sys.familySup S hU := by
  intro a ha
  exact sys.subset_deductiveClosure _ hU <x, hx, ha>

end InfoSys
end DomainTheory.InformationSystems
```

exists_isLUB_iff.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid42
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.IsFinitelyConsistent
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.deductiveClosure
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.familyUnionCarrier
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.familySup
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.join
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.exists_entSet_of_subset_deductive
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.le_familySup
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.finitelyConsistent_of_upperBound

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- Join exists iff the union is finitely consistent. -/
theorem exists_isLUB_iff (S : Set sys.Element) :
```

```
    (exists  z : sys.Element, (forall  x  in  S, x  <=  z)  /\  forall  w, (forall  x
     in  S, x  <=  w)  ->  z  <=  w)  <->
      Nonempty (sys.IsFinitelyConsistent (sys.familyUnionCarrier S)) := by
  constructor
  * rintro <z, hub, _>
    exact <sys.finitelyConsistent_of_upperBound S hub>
  * rintro <hU>
    refine <sys.familySup S hU, fun x hx => sys.le_familySup S hU hx,
      fun w hw => familySup.le sys S hU hw>

end InfoSys
end DomainTheory.InformationSystems
```

le_join_left.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid42
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.IsFinitelyConsistent
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.deductiveClosure
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.familyUnionCarrier
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.familySup
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.join
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.exists_entSet_of_subset_deductive
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.subset_deductiveClosure

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

theorem le_join_left (x y : sys.Element)
    (h : sys.IsFinitelyConsistent (x.carrier  U  y.carrier)) : x  <=  sys.join x y h
    :=
  fun _a ha => sys.subset_deductiveClosure _ h (Or.inl ha)

end InfoSys
end DomainTheory.InformationSystems
```

le_join_right.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid42
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.IsFinitelyConsistent
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.deductiveClosure
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.familyUnionCarrier
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.familySup
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.join
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.exists_entSet_of_subset_deductive
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.subset_deductiveClosure

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

theorem le_join_right (x y : sys.Element)
    (h : sys.IsFinitelyConsistent (x.carrier  U  y.carrier)) : y  <=  sys.join x y h
    :=
  fun _a ha => sys.subset_deductiveClosure _ h (Or.inr ha)

end InfoSys
end DomainTheory.InformationSystems
```

exists_join_iff.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid42
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.IsFinitelyConsistent
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.deductiveClosure
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.familyUnionCarrier
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.familySup
```

```
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.join
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.exists_entSet_of_subset_deductive
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.le_join_left
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.le_join_right

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- Binary join exists iff some element bounds both. -/
theorem exists_join_iff (x y : sys.Element) :
    (exists  z : sys.Element, x  <=  z  /\  y  <=  z)  <->
      Nonempty (sys.IsFinitelyConsistent (x.carrier  U  y.carrier)) := by
  constructor
  * rintro <z, hx, hy>
    exact <fun Y hY => z.consistent Y (fun a ha => by
      rcases hY ha with h | h
      * exact hx h
      * exact hy h)>
  * rintro <h>
    exact <sys.join x y h, sys.le_join_left x y h, sys.le_join_right x y h>

end InfoSys
end DomainTheory.InformationSystems
```

Factoid43.lean

```
module

public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.exists_join_iff

@[expose] public section

namespace DomainTheory.InformationSystems.InfoSys

set_option linter.unusedVariables false

theorem theorem_43.{u_1} : forall  {a : Type u_1} [inst : DecidableEq a] (sys :
    @DomainTheory.InformationSystems.InfoSys a inst)
  (x y : @DomainTheory.InformationSystems.InfoSys.Element a inst sys),
  Iff
    (exists  (z : @DomainTheory.InformationSystems.InfoSys.Element a inst sys),
      And
        (@LE.le (@DomainTheory.InformationSystems.InfoSys.Element a inst sys)
```

```
        (@Preorder.toLE (@DomainTheory.InformationSystems.InfoSys.Element a inst sys
    )
          (@PartialOrder.toPreorder (@DomainTheory.InformationSystems.InfoSys.
    Element a inst sys)
            (@DomainTheory.InformationSystems.InfoSys.instPartialOrderElement a inst
     sys)))
        x z)
      (@LE.le (@DomainTheory.InformationSystems.InfoSys.Element a inst sys)
        (@Preorder.toLE (@DomainTheory.InformationSystems.InfoSys.Element a inst sys
    )
          (@PartialOrder.toPreorder (@DomainTheory.InformationSystems.InfoSys.
    Element a inst sys)
            (@DomainTheory.InformationSystems.InfoSys.instPartialOrderElement a inst
     sys)))
        y z))
    (Nonempty
      (@DomainTheory.InformationSystems.InfoSys.IsFinitelyConsistent a inst sys
        (@Union.union (Set a) (@Set.instUnion a) (@DomainTheory.InformationSystems.
    InfoSys.Element.carrier a inst sys x)
        (@DomainTheory.InformationSystems.InfoSys.Element.carrier a inst sys y))))
    :=
  @DomainTheory.InformationSystems.InfoSys.exists_join_iff

end DomainTheory.InformationSystems.InfoSys
```

Chain.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- A chain: any two elements are comparable. -/
class Chain (S : Set sys.Element) : Prop where
  out : forall  (x y : sys.Element), x  in  S  ->  y  in  S  ->  x  <=  y  \/  y  <=
    x

end InfoSys

end DomainTheory.InformationSystems
```

IsDirected.lean

```
module
```

```
public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid44.Chain

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- Upward-directed family of elements: any two have a common upper bound in the
    family. -/
def IsDirected (S : Set sys.Element) : Prop :=
  forall (x y : sys.Element), x in S -> y in S -> exists z in S, x <= z
    /\ y <= z

theorem IsDirected.of_IsChain {S : Set sys.Element} (h : sys.Chain S) : sys.IsDirected
    S := by
  intro x y hx hy
  rcases h.out x y hx hy with hxy | hyx
  * exact <y, hy, hxy, le_rfl>
  * exact <x, hx, le_rfl, hyx>

end InfoSys

end DomainTheory.InformationSystems
```

directedSup.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid44.IsDirected
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid44.finitelyConsistent_of_directed
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.familySup

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys
```

```
variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- **Factoid 4.4.** Union of a nonempty directed family is an element (as `familySup`)
    . -/
def directedSup (S : Set sys.Element) (hne : S.Nonempty) (hdir : sys.IsDirected S) :
    sys.Element :=
  sys.familySup S (sys.finitelyConsistent_of_directed S hne hdir)

/-- For directed families, the lub carrier equals the raw union (already deductively
    closed). -/
theorem directedSup.carrier_eq_union (S : Set sys.Element) (hne : S.Nonempty)
    (hdir : sys.IsDirected S) :
    (sys.directedSup S hne hdir).carrier = sys.familyUnionCarrier S := by
  let hU := sys.finitelyConsistent_of_directed S hne hdir
  refine Set.Subset.antisymm ?_ (sys.subset_deductiveClosure _ hU)
  intro a ha
  obtain <u, huU, hEnt> := (ha : a  in  (sys.deductiveClosure _ hU).carrier)
  obtain <z, hz, huZ> := sys.exists_mem_of_subset_directedUnion S hne hdir u huU
  exact <z, hz, z.closed u a huZ hEnt>

theorem directedSup.le (S : Set sys.Element) (hne : S.Nonempty) (hdir : sys.IsDirected
    S)
    {z : sys.Element} (h : forall  x  in  S, x  <=  z) : sys.directedSup S hne hdir
    <=  z :=
  familySup.le sys S (sys.finitelyConsistent_of_directed S hne hdir) h

end InfoSys

end DomainTheory.InformationSystems
```

Factoid44.lean

```
module

public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid44.directedSup

@[expose] public section

namespace DomainTheory.InformationSystems.InfoSys

set_option linter.unusedVariables false

theorem theorem_44.{u_1} : forall  {a : Type u_1} [inst : DecidableEq a] (sys :
    @DomainTheory.InformationSystems.InfoSys a inst)
  (S : Set (@DomainTheory.InformationSystems.InfoSys.Element a inst sys))
  (hne : @Set.Nonempty (@DomainTheory.InformationSystems.InfoSys.Element a inst sys) S
    )
  (hdir : @DomainTheory.InformationSystems.InfoSys.IsDirected a inst sys S),
  @Eq (Set a)
    (@DomainTheory.InformationSystems.InfoSys.Element.carrier a inst sys
      (@DomainTheory.InformationSystems.InfoSys.directedSup a inst sys S hne hdir))
    (@DomainTheory.InformationSystems.InfoSys.familyUnionCarrier a inst sys S) :=
  @DomainTheory.InformationSystems.InfoSys.directedSup.carrier_eq_union
```

```
end DomainTheory.InformationSystems.InfoSys
```

exists_mem_of_subset_directedUnion.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid44.IsDirected

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- Given `Y` whose tokens each lie in some member of a nonempty directed family, find
    a
single member containing all of `Y`. -/
theorem exists_mem_of_subset_directedUnion (S : Set sys.Element) (hne : S.Nonempty)
    (hdir : sys.IsDirected S) (Y : Finset a)
    (hY : (Y : Set a)  subseteq  sys.familyUnionCarrier S) :
    exists  z  in  S, (Y : Set a)  subseteq  z.carrier := by
  induction Y using Finset.induction_on with
  | empty =>
    obtain <z, hz> := hne
    exact <z, hz, fun _ ha => False.elim (Finset.notMem_empty _ (Finset.mem_coe.1 ha))
    >
  | insert a s _ha ih =>
    have ha' : exists  x  in  S, a  in  x.carrier :=
      hY (Finset.mem_coe.2 (Finset.mem_insert_self a s))
    have hs' : (s : Set a)  subseteq  sys.familyUnionCarrier S := by
      intro b hb
      exact hY (Finset.mem_coe.2 (Finset.mem_insert_of_mem (Finset.mem_coe.1 hb)))
    obtain <xa, hxa, haS> := ha'
    obtain <xs, hxs, hsS> := ih hs'
    obtain <z, hz, hxa_le, hxs_le> := hdir xa xs hxa hxs
    refine <z, hz, ?_>
    intro b hb
    rcases Finset.mem_insert.mp (Finset.mem_coe.1 hb) with rfl | hb'
    * exact hxa_le haS
    * exact hxs_le (hsS (Finset.mem_coe.2 hb'))

end InfoSys

end DomainTheory.InformationSystems
```

finitelyConsistent_of_directed.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid44.IsDirected
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid44.exists_mem_of_subset_directedUnion

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- Directed families are finitely consistent on their carrier union. -/
theorem finitelyConsistent_of_directed (S : Set sys.Element) (hne : S.Nonempty)
    (hdir : sys.IsDirected S) :
    sys.IsFinitelyConsistent (sys.familyUnionCarrier S) := by
  intro Y hY
  obtain <z, _hz, hYZ> := sys.exists_mem_of_subset_directedUnion S hne hdir Y hY
  exact z.consistent Y hYZ

end InfoSys

end DomainTheory.InformationSystems
```

chainSup.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid44.Chain
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid44.IsDirected
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid44.directedSup

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys
```

```
variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- Chain unions are elements. -/
def chainSup (S : Set sys.Element) (hne : S.Nonempty) (hchain : sys.Chain S) : sys.
    Element :=
  sys.directedSup S hne (IsDirected.of_IsChain sys hchain)

end InfoSys

end DomainTheory.InformationSystems
```

le_directedSup.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid44.IsDirected
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid44.Chain
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid44.directedSup
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid44.chainSup
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid44.exists_mem_of_subset_directedUnion
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid44.finitelyConsistent_of_directed

public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid43.le_familySup
@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

theorem le_directedSup (S : Set sys.Element) (hne : S.Nonempty) (hdir : sys.IsDirected
     S)
    {x : sys.Element} (hx : x  in  S) : x  <=  sys.directedSup S hne hdir :=
  sys.le_familySup S (sys.finitelyConsistent_of_directed S hne hdir) hx

end InfoSys
end DomainTheory.InformationSystems
```

botElement_eq_closure_empty.lean

```
module

public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid36
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid44
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid45.finiteApproximants

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- `False = {}`. -/
theorem botElement_eq_closure_empty :
    sys.botElement = sys.closure ({} : Finset a) sys.con_empty := by
  refine le_antisymm ?_ ?_
  * intro a ha
    -- ha : Ent {bot} a; show Ent {} a
    exact sys.ent_trans sys.con_empty (sys.con_sing sys.bot)
      (fun y hy => by
        have : y = sys.bot := Finset.mem_singleton.mp hy
        subst this
        exact sys.ent_bot sys.con_empty)
      ha
  * intro a ha
    -- ha : Ent {} a; show Ent {bot} a
    exact sys.ent_trans (sys.con_sing sys.bot) sys.con_empty
      (fun _ hy => False.elim (Finset.notMem_empty _ hy)) ha

end InfoSys
end DomainTheory.InformationSystems
```

compact_closure.lean

```
module

public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid36
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid44
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid45.finiteApproximants

public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid36.
    closure_le_element
@[expose] public section

namespace DomainTheory.InformationSystems
```

```
open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- Compactness of finite elements: if `u  <=   sqsup  S` for directed nonempty `S`,
    then
`u  <= ` some member of `S`. -/
theorem compact_closure (S : Set sys.Element) (hne : S.Nonempty) (hdir : sys.
    IsDirected S)
    {u : Finset a} (hu : u  in  sys.Con)
    (hle : sys.closure u hu  <=  sys.directedSup S hne hdir) :
    exists  z  in  S, sys.closure u hu  <=  z := by
  have huU : (u : Set a)  subseteq  sys.familyUnionCarrier S := by
    intro a ha
    have : a  in  (sys.directedSup S hne hdir).carrier :=
      hle (sys.subset_closure hu (Finset.mem_coe.2 ha))
    rwa [directedSup.carrier_eq_union sys S hne hdir] at this
  obtain <z, hz, huZ> := sys.exists_mem_of_subset_directedUnion S hne hdir u huU
  exact <z, hz, sys.closure_le_element z hu huZ>

end InfoSys
end DomainTheory.InformationSystems
```

directed_finiteApproximants.lean

```
module

public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid36
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid44
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid45.finiteApproximants

public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid36.
    closures_directed
@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

theorem directed_finiteApproximants (x : sys.Element) :
    sys.IsDirected (sys.finiteApproximants x) := by
  intro y_1 y_2 hy_1 hy_2
  obtain <u, hu, huX, rfl> := hy_1
  obtain <v, hv, hvX, rfl> := hy_2
  obtain <w, hw, hwX, huw, hvw> := sys.closures_directed x hu hv huX hvX
```

```
  exact <sys.closure w hw, <w, hw, hwX, rfl>, huw, hvw>

end InfoSys
end DomainTheory.InformationSystems
```

finiteApproximants.lean

```
module

public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid36
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid44

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- Finite approximations of `x`: the set `{u | u  in  Con, u  subseteq  x}`. -/
def finiteApproximants (x : sys.Element) : Set sys.Element :=
  {y | exists  (u : Finset a) (hu : u  in  sys.Con), u  subseteq  x.carrier  /\  y =
    sys.closure u hu}

end InfoSys

end DomainTheory.InformationSystems
```

nonempty_finiteApproximants.lean

```
module

public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid36
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid44
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid45.finiteApproximants
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid45.botElement_eq_closure_empty

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)
```

```
theorem nonempty_finiteApproximants (x : sys.Element) :
    (sys.finiteApproximants x).Nonempty := by
  refine <sys.botElement, {}, sys.con_empty, ?_, sys.botElement_eq_closure_empty>
  intro a ha
  exact False.elim (Finset.notMem_empty _ (Finset.mem_coe.1 ha))

end InfoSys
end DomainTheory.InformationSystems
```

eq_directedSup_finiteApproximants.lean

```
module

public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid36
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid44
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid45.finiteApproximants
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid45.nonempty_finiteApproximants
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid45.directed_finiteApproximants

public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid44.directedSup
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid36.
    closure_le_element
@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a] (sys : InfoSys a)

/-- **Factoid 4.5 / algebraicity.** `x` is the directed lub of its finite approximants.
    -/
theorem eq_directedSup_finiteApproximants (x : sys.Element) :
    x = sys.directedSup (sys.finiteApproximants x)
      (sys.nonempty_finiteApproximants x) (sys.directed_finiteApproximants x) := by
  refine le_antisymm ?_ ?_
  * -- x  <=  directedSup: every token of x lies in some u  subseteq  x
    intro a ha
    have : a  in  sys.approxUnion x := (sys.mem_element_iff_mem_closure x a).1 ha
    obtain <u, hu, huX, ha'> := this
    rw [directedSup.carrier_eq_union sys]
    exact <sys.closure u hu, <u, hu, huX, rfl>, ha'>
  * -- directedSup  <=  x: each approximant is  <=  x
    apply directedSup.le sys
    intro y hy
    obtain <u, hu, huX, rfl> := hy
```

```
    exact sys.closure_le_element x hu huX

end InfoSys
end DomainTheory.InformationSystems
```

Factoid45.lean

```
module

public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid45.eq_directedSup_finiteApproximants

@[expose] public section

namespace DomainTheory.InformationSystems.InfoSys

set_option linter.unusedVariables false

theorem theorem_45.{u_1} : forall  {a : Type u_1} [inst : DecidableEq a] (sys :
    @DomainTheory.InformationSystems.InfoSys a inst)
  (x : @DomainTheory.InformationSystems.InfoSys.Element a inst sys),
  @Eq (@DomainTheory.InformationSystems.InfoSys.Element a inst sys) x
    (@DomainTheory.InformationSystems.InfoSys.directedSup a inst sys
      (@DomainTheory.InformationSystems.InfoSys.finiteApproximants a inst sys x)
      (@DomainTheory.InformationSystems.InfoSys.nonempty_finiteApproximants a inst sys
     x)
      (@DomainTheory.InformationSystems.InfoSys.directed_finiteApproximants a inst sys
     x)) :=
  @DomainTheory.InformationSystems.InfoSys.eq_directedSup_finiteApproximants

end DomainTheory.InformationSystems.InfoSys
```

ScottContinuous.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid45

@[expose] public section

namespace DomainTheory.InformationSystems
namespace InfoSys

open DomainTheory.InformationSystems.Constructive

variable {a b : Type*} [DecidableEq a] [DecidableEq b]

/-- Image of a nonempty set under a function is nonempty. -/
theorem nonempty_image {X Y : Type*} {S : Set X} (hne : S.Nonempty) (f : X  ->  Y) :
    (f '' S).Nonempty :=
  hne.image f
```

```
/-- Monotone images preserve directedness. -/
theorem directed_image {A : InfoSys a} {B : InfoSys b} {S : Set A.Element}
    (hdir : A.IsDirected S) (f : A.Element  ->  B.Element)
    (hmono : forall  {x y}, x  <=  y  ->  f x  <=  f y) : B.IsDirected (f '' S) := by
  intro x y hx hy
  obtain <x_0, hx_0, rfl> := hx
  obtain <y_0, hy_0, rfl> := hy
  obtain <z_0, hz_0, hxz, hyz> := hdir x_0 y_0 hx_0 hy_0
  exact <f z_0, <z_0, hz_0, rfl>, hmono hxz, hmono hyz>

/-- Scott-continuous map of domains: monotone and preserves directed lubs. -/
structure ScottContinuous (A : InfoSys a) (B : InfoSys b) where
  toFun : A.Element  ->  B.Element
  mono' : forall  {x y : A.Element}, x  <=  y  ->  toFun x  <=  toFun y
  map_directedSup :
    forall  (S : Set A.Element) (hne : S.Nonempty) (hdir : A.IsDirected S),
      toFun (A.directedSup S hne hdir) =
        B.directedSup (toFun '' S) (nonempty_image hne toFun)
          (directed_image hdir toFun mono')

end InfoSys
end DomainTheory.InformationSystems
```

basicOpen.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid45
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid46.ScottContinuous

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a b : Type*} [DecidableEq a] [DecidableEq b]

/-- **Scott S4.** Basic open `[u] = {x  in  |A| | u  subseteq  x}`. -/
def basicOpen (A : InfoSys a) (u : Finset a) : Set A.Element :=
  {x | u  subseteq  x.carrier}

/-- Intersection of basic opens: `[u]  I  [v] = [u  U ' v]`. -/
theorem basicOpen.inter (A : InfoSys a) (u v : Finset a) :
    A.basicOpen u  I  A.basicOpen v = A.basicOpen (u  U ' v) := by
  ext x
  constructor
  * intro <hu, hv> a ha
```

```
    rcases mem_funion.mp (Finset.mem_coe.1 ha) with h | h
    * exact hu (Finset.mem_coe.2 h)
    * exact hv (Finset.mem_coe.2 h)
  * intro huv
    refine <?_, ?_>
    * intro a ha
      exact huv (Finset.mem_coe.2 (mem_funion.mpr (Or.inl (Finset.mem_coe.1 ha))))
    * intro a ha
      exact huv (Finset.mem_coe.2 (mem_funion.mpr (Or.inr (Finset.mem_coe.1 ha))))

end InfoSys

end DomainTheory.InformationSystems
```

eq_of_basicOpen_eq.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid45
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid46.ScottContinuous
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid46.basicOpen
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid46.toScottContinuous
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid46.ofScottContinuous
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid46.mono
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid46.toElement_directedSup

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a b : Type*} [DecidableEq a] [DecidableEq b]

/-- **T_0:** elements with the same basic-open neighborhoods are equal. -/
theorem eq_of_basicOpen_eq (A : InfoSys a) {x y : A.Element}
    (h : forall  u : Finset a, x  in  A.basicOpen u  <->  y  in  A.basicOpen u) : x =
    y := by
  refine le_antisymm ?_ ?_
  * intro a ha
    have : x  in  A.basicOpen {a} := by
      intro b hb
      have : b = a := Finset.mem_singleton.mp (Finset.mem_coe.1 hb)
```

```
      subst this
      exact ha
    exact ((h {a}).1 this) (Finset.mem_coe.2 (Finset.mem_singleton_self a))
  * intro a ha
    have : y  in  A.basicOpen {a} := by
      intro b hb
      have : b = a := Finset.mem_singleton.mp (Finset.mem_coe.1 hb)
      subst this
      exact ha
    exact ((h {a}).2 this) (Finset.mem_coe.2 (Finset.mem_singleton_self a))

end InfoSys
end DomainTheory.InformationSystems
```

mem_basicOpen.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid45
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid46.ScottContinuous
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid46.basicOpen
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid46.toScottContinuous
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid46.ofScottContinuous
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid46.mono
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid46.toElement_directedSup

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a b : Type*} [DecidableEq a] [DecidableEq b]

theorem mem_basicOpen {A : InfoSys a} {u : Finset a} {x : A.Element} :
    x  in  A.basicOpen u  <->  u  subseteq  x.carrier := Iff.rfl

end InfoSys
end DomainTheory.InformationSystems
```

mono.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid45
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid46.ScottContinuous

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a b : Type*} [DecidableEq a] [DecidableEq b]

namespace ScottContinuous

variable {A : InfoSys a} {B : InfoSys b}

theorem mono (f : ScottContinuous A B) {x y : A.Element} (h : x  <=  y) :
    f.toFun x  <=  f.toFun y :=
  f.mono' h

end ScottContinuous

end InfoSys

end DomainTheory.InformationSystems
```

ofScottContinuous.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid45
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid46.ScottContinuous
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid46.mono

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys
```

```
variable {a b : Type*} [DecidableEq a] [DecidableEq b]

namespace ApproximableMap

variable {A : InfoSys a} {B : InfoSys b}

/-- Relation recovered from a Scott-continuous map: `u f v  <->  v  subseteq  g(u)`. -/
def ofScottContinuous (g : ScottContinuous A B) : ApproximableMap A B where
  rel u v := exists  hu : u  in  A.Con, v  in  B.Con  /\  v  subseteq  (g.toFun (A.
    closure u hu)).carrier
  rel_dom := fun <hu, _, _> => hu
  rel_cod := fun <_, hv, _> => hv
  empty_rel := by
    refine <A.con_empty, B.con_empty, ?_>
    intro b hb
    exact False.elim (Finset.notMem_empty _ (Finset.mem_coe.1 hb))
  union_right := by
    intro u v v' <hu, hv, hsub> <_hu', hv', hsub'>
    refine <hu, ?_, ?_>
    * have hsubUV : (v  U ' v')  subseteq  (g.toFun (A.closure u hu)).carrier := by
        intro b hb
        rcases mem_funion.mp (Finset.mem_coe.1 hb) with h | h
        * exact hsub (Finset.mem_coe.2 h)
        * exact hsub' (Finset.mem_coe.2 h)
      exact (g.toFun (A.closure u hu)).consistent (v  U ' v') hsubUV
    * intro b hb
      rcases mem_funion.mp (Finset.mem_coe.1 hb) with h | h
      * exact hsub (Finset.mem_coe.2 h)
      * exact hsub' (Finset.mem_coe.2 h)
  mono := by
    intro u u' v v' <hu, hv, hsub> hEntUU' hEntVV' hu' hv'
    refine <hu', hv', ?_>
    have hclos : A.closure u hu  <=  A.closure u' hu' :=
      A.closure_le_of_entSet hu hu' hEntUU'
    intro b hb
    have hEntb : B.Ent v b := hEntVV' b hb
    have hb_in : b  in  (g.toFun (A.closure u hu)).carrier :=
      (g.toFun (A.closure u hu)).closed v b hsub hEntb
    exact g.mono hclos hb_in

end ApproximableMap

end InfoSys

end DomainTheory.InformationSystems
```

toElement_directedSup.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid44.le_directedSup
```

```
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid45
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid46.ScottContinuous

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a b : Type*} [DecidableEq a] [DecidableEq b]

namespace ApproximableMap

variable {A : InfoSys a} {B : InfoSys b}

/-- Approximable maps preserve directed lubs (Scott continuity on elements). -/
theorem toElement_directedSup (f : ApproximableMap A B)
    (S : Set A.Element) (hne : S.Nonempty) (hdir : A.IsDirected S) :
    f.toElement (A.directedSup S hne hdir) =
      B.directedSup (f.toElement '' S) (nonempty_image hne f.toElement)
        (directed_image hdir f.toElement f.toElement_mono) := by
  refine le_antisymm ?_ ?_
  * intro Y ⟨u, hu, hrel⟩
    have huU : (u : Set a)  subseteq  A.familyUnionCarrier S := by
      intro a ha
      have : a  in  (A.directedSup S hne hdir).carrier := hu ha
      rwa [directedSup.carrier_eq_union A S hne hdir] at this
    obtain ⟨z, hz, huZ⟩ := A.exists_mem_of_subset_directedUnion S hne hdir u huU
    have : Y  in  (f.toElement z).carrier := ⟨u, huZ, hrel⟩
    rw [directedSup.carrier_eq_union B]
    exact ⟨f.toElement z, ⟨z, hz, rfl⟩, this⟩
  * apply directedSup.le B
    intro y hy
    obtain ⟨x, hx, rfl⟩ := hy
    exact f.toElement_mono (A.le_directedSup S hne hdir hx)

end ApproximableMap

end InfoSys

end DomainTheory.InformationSystems
```

toElement_ofScottContinuous.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid45
```

```
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid46.ScottContinuous
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid46.basicOpen
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid46.toScottContinuous
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid46.ofScottContinuous
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid46.mono
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid46.toElement_directedSup

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a b : Type*} [DecidableEq a] [DecidableEq b]

namespace ApproximableMap

variable {A : InfoSys a} {B : InfoSys b}

/-- **Factoid 4.6 ( <- ).** Scott-continuous maps recover as `toElement` of the induced
approximable relation. -/
theorem toElement_ofScottContinuous (g : ScottContinuous A B) (x : A.Element) :
    (ofScottContinuous g).toElement x = g.toFun x := by
  refine le_antisymm ?_ ?_
  * intro Y <u, hu, <huCon, _, hsub>>
    have : A.closure u huCon  <=  x := A.closure_le_element x huCon hu
    exact g.mono this (hsub (Finset.mem_coe.2 (Finset.mem_singleton_self Y)))
  * intro Y hY
    let S := A.finiteApproximants x
    have hne := A.nonempty_finiteApproximants x
    have hdir := A.directed_finiteApproximants x
    have hx : x = A.directedSup S hne hdir := A.eq_directedSup_finiteApproximants x
    have hdirImg := directed_image hdir g.toFun g.mono'
    have hneImg := nonempty_image hne g.toFun
    have hg : g.toFun x = B.directedSup (g.toFun '' S) hneImg hdirImg := by
      have h1 : g.toFun x = g.toFun (A.directedSup S hne hdir) := congrArg g.toFun hx
      have h2 : g.toFun (A.directedSup S hne hdir) =
          B.directedSup (g.toFun '' S) hneImg hdirImg :=
        g.map_directedSup S hne hdir
      exact h1.trans h2
    have hY' : Y  in  (B.directedSup (g.toFun '' S) hneImg hdirImg).carrier := by
      rw [ <-  hg]; exact hY
    have hY'' : Y  in  B.familyUnionCarrier (g.toFun '' S) := by
      rwa [ <-  directedSup.carrier_eq_union B (g.toFun '' S) hneImg hdirImg]
    obtain <z, hz, hYz> := hY''
    obtain <y, hy, rfl> := hz
    obtain <u, hu, huX, rfl> := hy
    refine <u, huX, <hu, B.con_sing Y, ?_>>
```

```
    intro b hb
    have : b = Y := Finset.mem_singleton.mp (Finset.mem_coe.1 hb)
    subst this
    exact hYz

end ApproximableMap
end InfoSys

end DomainTheory.InformationSystems
```

Factoid46.lean

```
module

public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid46.toElement_ofScottContinuous

@[expose] public section

namespace DomainTheory.InformationSystems.InfoSys.ApproximableMap

set_option linter.unusedVariables false

theorem theorem_46.{u_1, u_2} : forall  {a : Type u_1} {b : Type u_2} [inst :
    DecidableEq a] [inst_1 : DecidableEq b]
  {A : @DomainTheory.InformationSystems.InfoSys a inst} {B : @DomainTheory.
    InformationSystems.InfoSys b inst_1}
  (g : @DomainTheory.InformationSystems.InfoSys.ScottContinuous a b inst inst_1 A B)
  (x : @DomainTheory.InformationSystems.InfoSys.Element a inst A),
  @Eq (@DomainTheory.InformationSystems.InfoSys.Element b inst_1 B)
    (@DomainTheory.InformationSystems.InfoSys.ApproximableMap.toElement a b inst
    inst_1 A B
      (@DomainTheory.InformationSystems.InfoSys.ApproximableMap.ofScottContinuous a b
    inst inst_1 A B g) x)
    (@DomainTheory.InformationSystems.InfoSys.ScottContinuous.toFun a b inst inst_1 A
    B g x) :=
  @DomainTheory.InformationSystems.InfoSys.ApproximableMap.toElement_ofScottContinuous

end DomainTheory.InformationSystems.InfoSys.ApproximableMap
```

toScottContinuous.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid45
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid46.ScottContinuous
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid46.toElement_directedSup

@[expose] public section
```

```
namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a b : Type*} [DecidableEq a] [DecidableEq b]

namespace ApproximableMap

variable {A : InfoSys a} {B : InfoSys b}

/-- **Factoid 4.6 ( -> ).** Every approximable map is Scott-continuous on elements. -/
def toScottContinuous (f : ApproximableMap A B) : ScottContinuous A B where
  toFun := f.toElement
  mono' := f.toElement_mono
  map_directedSup := f.toElement_directedSup

end ApproximableMap

end InfoSys

end DomainTheory.InformationSystems
```

4.5 Approximable mappings between domains

Dependencies. Inside ApproximableMappingsBetweenDomains, each box is one subfolder, or one Lean file sitting directly in the folder. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. The arrowhead points at the file or folder that imports, and the number is how many direct imports.



Figure 42: Approximable mappings between domains.

Once a general notion of set or domain has been defined mathematically, the next major issue is: How are the different domains to be related one to another? The answer to this important question can many times be given by defining an appropriate concept of mapping between domains. The answer need not be unique; the same collection of domains may support more than one idea of function depending on the special aspects of the domains that need to be brought out. In the case of the kinds of domains being studied here, there are two principal answers, one of which is to be explained in this section.

In order to understand an appropriate notion of a mapping or a function between domains constructed from information systems, we have to refer back to the way in which elements are introduced. As we have seen in Section 3, an element is a consistent, closed set of data objects. To generate an element, one has to generate more and more of the finite consistent subsets of the element. Note that the separate data objects have to be grouped into these finite consistent sets, because the entailment relation may require a finite set of arbitrary size on the left in making the necessary entailments for closure. It is this kind of passage from a finite set to its closure that is to be generalized in defining mappings.

Consider two domains. To map from one to another, some information about a possible element of the first is presented as input to the function f. Then as output the function f starts generating an element. If the input were u, a consistent set of the first domain, then part of the output in the second domain might be a consistent set v. We could say that there is an input/output relationship set up by f, and indicate by $u\ f\ v$ that this relation holds going from u in the first domain to v in the second. Of course to get the full effect of f, it is necessary to take all the v's related to a given u, because even a small finite amount of input may cause an infinite amount of output. But every element of a domain is just the sum total of its finite subsets (finite approximations), so it is sufficient to make the mapping relationship go between finite sets. Here is the formal definition with the exact conditions that the relation f must satisfy.

- Definition 5.1
- Definition 5.2
- Proposition 5.3
- Proposition 5.4
- Proposition 5.5
- Proposition 5.6

The proof is immediate, since we need only interpret the input/output relation $u\ (\mathrm{const}\ b)\ v$ as meaning $v \subseteq b$. We note that our notation is somewhat ambiguous since, for example, in 5.6(iii) we are using $\mathrm{const}\ b$ in two senses: once as a mapping from $\mathbf{A}$, and once as a mapping from $\mathbf{D}$. In fact, we should hang two subscripts on the thing to fix both the source and the target of the mapping. This is too painful, however, and we generally rely on context to make the meaning clear. (Formal rules of type checking can be given, so on a computer-based system all the necessary subscripts could be put back in. Thus, the kind of ambiguity we are dealing with here is not very serious.)

ApproximableMap.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35

@[expose] public section

universe u v

namespace DomainTheory.InformationSystems
namespace InfoSys

open DomainTheory.InformationSystems.Constructive

variable {a : Type u} {b : Type v} [DecidableEq a] [DecidableEq b]

/-- **Definition 5.1 (Scott 1982).** Approximable mapping between information systems.
    -/
structure ApproximableMap (A : InfoSys a) (B : InfoSys b) where
  rel : Finset a  ->  Finset b  ->  Prop
```

```
  rel_dom : forall  {u v}, rel u v  ->  u  in  A.Con
  rel_cod : forall  {u v}, rel u v  ->  v  in  B.Con
  empty_rel : rel {} {}
  union_right : forall  {u v v'}, rel u v  ->  rel u v'  ->  rel u (v  U ' v')
  mono : forall  {u u' v v'},
    rel u v  ->  A.EntSet u' u  ->  B.EntSet v v'  ->  u'  in  A.Con  ->  v'  in  B.
    Con  ->  rel u' v'

end InfoSys
end DomainTheory.InformationSystems
```

ext.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable.ApproximableMap

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v

variable {a : Type u} {b : Type v} [DecidableEq a] [DecidableEq b]

namespace ApproximableMap

variable {A : InfoSys a} {B : InfoSys b}

theorem ext {f g : ApproximableMap A B} (h : forall  u v, f.rel u v  <->  g.rel u v) :
     f = g := by
  obtain <rf, _, _, _, _, _> := f
  obtain <rg, _, _, _, _, _> := g
  have : rf = rg := by
    funext u v
    exact propext (h u v)
  subst this
  rfl

end ApproximableMap

end InfoSys

end DomainTheory.InformationSystems
```

singleton_funion_eq.lean

```
module
```

```
public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable.ApproximableMap
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable.ext

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v

variable {a : Type u} {b : Type v} [DecidableEq a] [DecidableEq b]

namespace ApproximableMap

variable {A : InfoSys a} {B : InfoSys b}

theorem singleton_funion_eq (a : b) (s : Finset b) :
    ({a} : Finset b)  U ' s = insert a s := by
  ext x
  constructor
  * intro hx
    rcases mem_funion.mp hx with ha | hs
    * exact Finset.mem_insert.mpr (Or.inl (Finset.mem_singleton.mp ha))
    * exact Finset.mem_insert_of_mem hs
  * intro hx
    rcases Finset.mem_insert.mp hx with ha | hs
    * exact mem_funion.mpr (Or.inl (Finset.mem_singleton.mpr ha))
    * exact mem_funion.mpr (Or.inr hs)

end ApproximableMap

end InfoSys

end DomainTheory.InformationSystems
```

exists_rel_of_subset_image.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable.ApproximableMap
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable.singleton_funion_eq

@[expose] public section
```

```
namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v

variable {a : Type u} {b : Type v} [DecidableEq a] [DecidableEq b]

namespace ApproximableMap

variable {A : InfoSys a} {B : InfoSys b}

/-- Given `Y  subseteq  f(x).carrier`, produce `u  subseteq  x` with `u f Y`. -/
theorem exists_rel_of_subset_image (f : ApproximableMap A B) (x : A.Element)
    (Y : Finset b)
    (hY : Y  subseteq  ({Y : b | exists  u : Finset a, u  subseteq  x.carrier  /\  f.
    rel u {Y}} : Set b)) :
    exists  u : Finset a, u  subseteq  x.carrier  /\  f.rel u Y := by
  induction Y using Finset.induction_on with
  | empty =>
    refine <{}, ?_, ?_>
    * intro a ha
      exact False.elim (Finset.notMem_empty a (Finset.mem_coe.1 ha))
    * exact f.mono f.empty_rel (A.entSet_empty {}) (B.entSet_empty {}) A.con_empty B.
    con_empty
  | insert a s _ha ih =>
    have ha' : exists  u : Finset a, u  subseteq  x.carrier  /\  f.rel u {a} :=
      hY (Finset.mem_coe.2 (Finset.mem_insert_self a s))
    have hs' :
        s  subseteq  ({Y : b | exists  u : Finset a, u  subseteq  x.carrier  /\  f.rel
     u {Y}} : Set b) := by
      intro b hb
      exact hY (Finset.mem_coe.2 (Finset.mem_insert_of_mem (Finset.mem_coe.1 hb)))
    obtain <ua, hua, hrela> := ha'
    obtain <us, hus, hrels> := ih hs'
    refine <ua  U ' us, ?_, ?_>
    * intro z hz
      have hz' : z  in  ua  U ' us := Finset.mem_coe.1 hz
      rcases mem_funion.mp hz' with h | h
      * exact hua (Finset.mem_coe.2 h)
      * exact hus (Finset.mem_coe.2 h)
    * have hUcon : ua  U ' us  in  A.Con :=
        x.consistent (ua  U ' us) fun z hz => by
          rcases mem_funion.mp hz with h | h
          * exact hua (Finset.mem_coe.2 h)
          * exact hus (Finset.mem_coe.2 h)
      have hEnt_ua : A.EntSet (ua  U ' us) ua :=
        fun y hy => A.ent_refl hUcon (subset_funion_left ua us hy)
      have hEnt_us : A.EntSet (ua  U ' us) us :=
        fun y hy => A.ent_refl hUcon (subset_funion_right ua us hy)
      have h1 : f.rel (ua  U ' us) {a} :=
        f.mono hrela hEnt_ua
          (fun z hz => by
```

```
            rw [Finset.mem_singleton] at hz
            rw [hz]
            exact B.ent_refl (B.con_sing a) (Finset.mem_singleton_self a))
          hUcon (B.con_sing a)
      have h2 : f.rel (ua  U ' us) s :=
        f.mono hrels hEnt_us (proposition_2_3_iii B (f.rel_cod hrels))
          hUcon (f.rel_cod hrels)
      have hU : f.rel (ua  U ' us) ({a}  U ' s) := f.union_right h1 h2
      simpa [singleton_funion_eq] using hU

end ApproximableMap

end InfoSys

end DomainTheory.InformationSystems
```

toElement.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable.ApproximableMap
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable.exists_rel_of_subset_image

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v

variable {a : Type u} {b : Type v} [DecidableEq a] [DecidableEq b]

namespace ApproximableMap

variable {A : InfoSys a} {B : InfoSys b}

/-- **Definition 5.2.** Image of an element under an approximable mapping. -/
def toElement (f : ApproximableMap A B) (x : A.Element) : B.Element where
  carrier := {Y | exists  u : Finset a, u  subseteq  x.carrier  /\  f.rel u {Y}}
  consistent := by
    intro Y hY
    obtain <u, _, hrel> := exists_rel_of_subset_image f x Y hY
    exact f.rel_cod hrel
  closed := by
    intro Y a hY hEnt
    obtain <u, hu, hrel> := exists_rel_of_subset_image f x Y hY
    have harel : f.rel u {a} :=
      f.mono hrel (proposition_2_3_iii A (f.rel_dom hrel))
```

```
        (fun z hz => by
          rw [Finset.mem_singleton] at hz
          rw [hz]
          exact hEnt)
        (f.rel_dom hrel) (B.con_sing a)
    exact <u, hu, harel>

end ApproximableMap

end InfoSys

end DomainTheory.InformationSystems
```

Basic.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable.ApproximableMap
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable.toElement
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable.ext
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable.singleton_funion_eq
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable.exists_rel_of_subset_image

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v

variable {a : Type u} {b : Type v} [DecidableEq a] [DecidableEq b]

namespace ApproximableMap

variable {A : InfoSys a} {B : InfoSys b}

@[simp] theorem mem_toElement (f : ApproximableMap A B) (x : A.Element) {Y : b} :
    Y  in  (f.toElement x).carrier  <->  exists  u : Finset a, u  subseteq  x.carrier
     /\  f.rel u {Y} :=
  Iff.rfl
end ApproximableMap
end InfoSys

end DomainTheory.InformationSystems
```

toElement_mono.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable.Basic

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v

variable {a : Type u} {b : Type v} [DecidableEq a] [DecidableEq b]

namespace ApproximableMap

variable {A : InfoSys a} {B : InfoSys b}

theorem toElement_mono (f : ApproximableMap A B) {x y : A.Element} (hxy : x  <=  y) :
    f.toElement x  <=  f.toElement y := by
  intro Y <u, hu, hrel>
  exact <u, fun z hz => hxy (hu hz), hrel>

end ApproximableMap
end InfoSys

end DomainTheory.InformationSystems
```

Approximable.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable.Basic
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable.toElement_mono

@[expose] public section
```

4.6 Products and sums of domains

Dependencies. Inside ProductsAndSumsOfDomains, each box is one subfolder, or one Lean file sitting directly in the folder. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. The arrowhead points at the file or folder that imports, and the number is how many direct imports.

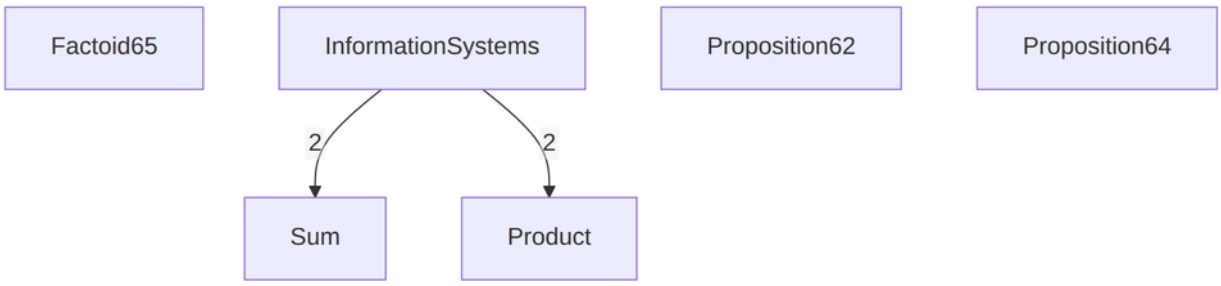


Figure 43: Products and sums of domains.

In the category of sets, the product (the cartesian product) of two sets is by definition just the set of ordered pairs of elements. We could use the same definition in the category of domains—provided we worked in terms of the elements of the domains. The disadvantage is that the determination of the product domain from data objects is lost, or at least pushed into the background. We shall therefore give a construction of products directly in terms of the given data objects, and then show how to prove that the product is just the one expected when we look at the elements.

The idea for the definition is a simple one. Think of two domains $\mathbf{A}$ and $\mathbf{B}$. A data object $X \in D_{\mathbf{A}}$ can be giving information about a possible element $x \in |\mathbf{A}|$, and a data object $Y \in D_{\mathbf{B}}$ can be giving information about a possible element $y \in |\mathbf{B}|$. How do we wish to give information about the pair (x, y)? The first piece of advice is not to say all that is known all at once. For example, if pressed, we can say we know X about the first coordinate of the pair; only later, if really pressed, we can say we also know Y about the second coordinate. The point is that data objects provide only partial information, and it does not matter if it takes several of them to give fairly complete information about the whole element. This attitude motivates, then, the particular form of our official definition.

- Definition 6.1
- Proposition 6.2
- Definition 6.3
- Proposition 6.4

It can also be shown that Propositions 6.2 and 6.4 uniquely characterize the domains $\mathbf{A} \times \mathbf{B}$ and $\mathbf{A} + \mathbf{B}$ up to isomorphism, and they give us the existence of additional mappings that are needed to show that product and sum are functors on the category of domains. We can also show from these results that the domain

$$\mathbf{BOOL} = \mathbf{1} + \mathbf{1}$$

has two elements true and false, such that any mapping on $\mathbf{BOOL}$ is uniquely determined by its action on true, false and $\perp_{\mathbf{BOOL}}$, and the values on the first two elements may be arbitrarily chosen.

`toElement_constMap_bot.lean`

```
module
```

```
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Proposition56
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid33
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Factoid65.
    unitSystem

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

namespace ApproximableMap

variable {a : Type*} [DecidableEq a] {A : InfoSys a}

/-- Elements of `|A|` are in bijection with approximable maps `1  ->  A` via `const`.
    -/
theorem toElement_constMap_bot (b : A.Element) :
    (constMap (A := unitSystem) b).toElement unitSystem.botElement = b :=
  constMap.toElement b unitSystem.botElement

end ApproximableMap
end InfoSys

end DomainTheory.InformationSystems
```

Factoid65.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Factoid65.
    toElement_constMap_bot

@[expose] public section

namespace DomainTheory.InformationSystems.InfoSys.ApproximableMap

set_option linter.unusedVariables false

theorem theorem_65.{u_1, u_2} : forall  {a : Type u_1} [inst : DecidableEq a] {A :
    @DomainTheory.InformationSystems.InfoSys a inst}
  (b : @DomainTheory.InformationSystems.InfoSys.Element a inst A),
  @Eq (@DomainTheory.InformationSystems.InfoSys.Element a inst A)
    (@DomainTheory.InformationSystems.InfoSys.ApproximableMap.toElement PUnit.{u_2 +
    1} a instDecidableEqPUnit inst
      DomainTheory.InformationSystems.InfoSys.unitSystem A
      (@DomainTheory.InformationSystems.InfoSys.ApproximableMap.constMap PUnit.{u_2 +
    1} a instDecidableEqPUnit inst
        DomainTheory.InformationSystems.InfoSys.unitSystem A b)
      (@DomainTheory.InformationSystems.InfoSys.botElement PUnit.{u_2 + 1}
    instDecidableEqPUnit
        DomainTheory.InformationSystems.InfoSys.unitSystem))
```

```
    b :=
  @DomainTheory.InformationSystems.InfoSys.ApproximableMap.toElement_constMap_bot

end DomainTheory.InformationSystems.InfoSys.ApproximableMap
```

unitElement_eq_bot.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Proposition56
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid33
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Factoid65.
    unitSystem

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

/-- The unique element of `|1|` is `False`. -/
theorem unitElement_eq_bot (x : unitSystem.Element) : x = unitSystem.botElement := by
  apply le_antisymm
  * intro a ha
    cases a
    exact trivial
  * exact botElement.le unitSystem x

end InfoSys
end DomainTheory.InformationSystems
```

approxMap_from_unit_eq_const.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Proposition56
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid33
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Factoid65.
    unitSystem
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Factoid65.
    unitElement_eq_bot

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Proposition56.constMap
@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys
```

```
namespace ApproximableMap

variable {a : Type*} [DecidableEq a] {A : InfoSys a}

/-- Every approximable map out of `1` is constant (Scott Factoid 6.5). -/
theorem approxMap_from_unit_eq_const (f : ApproximableMap unitSystem A) :
    f = constMap (A := unitSystem) (f.toElement unitSystem.botElement) :=
  constMap.unique (f.toElement unitSystem.botElement) f fun x => by
    rw [unitElement_eq_bot x]

end ApproximableMap
end InfoSys

end DomainTheory.InformationSystems
```

approxMap_to_unit_eq_const.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Proposition56
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid33
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Factoid65.
    unitSystem
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Factoid65.
    unitElement_eq_bot

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Proposition56.constMap
@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

namespace ApproximableMap

variable {a : Type*} [DecidableEq a] {A : InfoSys a}

/-- There is a unique approximable map into `1`, namely `const(False_1)`. -/
theorem approxMap_to_unit_eq_const (f : ApproximableMap A unitSystem) :
    f = constMap (A := A) unitSystem.botElement :=
  constMap.unique unitSystem.botElement f fun x => unitElement_eq_bot (f.toElement x)

end ApproximableMap
end InfoSys

end DomainTheory.InformationSystems
```

unitSystem.lean

```
module
```

```
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Proposition56
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid33

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

/-- **Factoid 6.5.** The unit information system on a singleton token type. -/
def unitSystem : InfoSys PUnit where
  bot := <>
  Con := Set.univ
  Ent := fun _ _ => True
  con_subset := by
    intro u v _ _
    exact trivial
  con_sing := by
    intro _
    exact trivial
  ent_con := by
    intro u a _
    exact trivial
  ent_bot := by
    intro u _
    exact trivial
  ent_refl := by
    intro u a _ _
    exact trivial
  ent_trans := by
    intro u v c _ _ _ _
    exact trivial

end InfoSys

end DomainTheory.InformationSystems
```

IsProdToken.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys
```

```
variable {a b : Type*} [DecidableEq a] [DecidableEq b]

/-- Scott 6.1(i): a product token is `(X, Delta_B)` or `(Delta_A, Y)`. -/
class IsProdToken (A : InfoSys a) (B : InfoSys b) (p : a  x  b) : Prop where
  out : p.1 = A.bot  \/  p.2 = B.bot

end InfoSys

end DomainTheory.InformationSystems
```

BasicInst5.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    IsProdToken

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a b : Type*} [DecidableEq a] [DecidableEq b]

instance (A : InfoSys a) (B : InfoSys b) (p : a  x  b) : Decidable (IsProdToken A B p)
     :=
  if h1 : p.1 = A.bot then isTrue <Or.inl h1>
  else if h2 : p.2 = B.bot then isTrue <Or.inr h2>
  else isFalse fun h => h.out.elim h1 h2

end InfoSys

end DomainTheory.InformationSystems
```

ProdToken.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    IsProdToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    BasicInst5

@[expose] public section

namespace DomainTheory.InformationSystems
```

```
open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a b : Type*} [DecidableEq a] [DecidableEq b]

/-- Token type of the product system `A  x  B`. -/
def ProdToken (A : InfoSys a) (B : InfoSys b) : Type _ :=
  {p : a  x  b // IsProdToken A B p}

end InfoSys

end DomainTheory.InformationSystems
```

BasicInst7.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    BasicInst5
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    ProdToken

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a b : Type*} [DecidableEq a] [DecidableEq b]

instance instDecidableEqProdToken (A : InfoSys a) (B : InfoSys b) :
    DecidableEq (ProdToken A B) :=
  Subtype.instDecidableEq

end InfoSys

end DomainTheory.InformationSystems
```

fstFinset.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    BasicInst5
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    ProdToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    BasicInst7
```

```
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    instLeftCommutativeFstInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    instLeftCommutativeSndInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    mem_foldr_fst

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a b : Type*} [DecidableEq a] [DecidableEq b]
variable (A : InfoSys a) (B : InfoSys b)

/-- Left projection of a finite set of product tokens (Scott 6.1). -/
def fstFinset (u : Finset (ProdToken A B)) : Finset a :=
  Multiset.foldr (fun p : ProdToken A B => insert p.val.1) ({} : Finset a)
    (u.filter (fun p => p.val.2 = B.bot)).1

theorem mem_fstFinset {u : Finset (ProdToken A B)} {x : a} :
    x  in  fstFinset A B u  <->  exists  p  in  u, p.val.2 = B.bot  /\  p.val.1 = x :=
     by
  unfold fstFinset
  rw [mem_foldr_fst]
  constructor
  * rintro <p, hp, hx>
    have hp' := Finset.mem_filter.mp hp
    exact <p, hp'.1, hp'.2, hx>
  * rintro <p, hp, hbot, hx>
    exact <p, Finset.mem_filter.mpr <hp, hbot>, hx>

theorem fstFinset.empty : fstFinset A B ({} : Finset (ProdToken A B)) = {} := by
  ext x
  constructor
  * intro hx
    rcases (mem_fstFinset A B).1 hx with <_, hp, _, _>
    exact False.elim (Finset.notMem_empty _ hp)
  * intro hx
    exact False.elim (Finset.notMem_empty x hx)

theorem fstFinset.funion (u v : Finset (ProdToken A B)) :
    fstFinset A B (u  U ' v) = fstFinset A B u  U ' fstFinset A B v := by
  ext x
  constructor
  * intro hx
    rcases (mem_fstFinset A B).1 hx with <p, hp, hbot, hx'>
    rcases mem_funion.mp hp with hp | hp
    * exact mem_funion.mpr (Or.inl ((mem_fstFinset A B).2 <p, hp, hbot, hx'>))
    * exact mem_funion.mpr (Or.inr ((mem_fstFinset A B).2 <p, hp, hbot, hx'>))
  * intro hx
    rcases mem_funion.mp hx with hx | hx
    * rcases (mem_fstFinset A B).1 hx with <p, hp, hbot, hx'>
```

```
      exact (mem_fstFinset A B).2 <p, mem_funion.mpr (Or.inl hp), hbot, hx'>
    * rcases (mem_fstFinset A B).1 hx with <p, hp, hbot, hx'>
      exact (mem_fstFinset A B).2 <p, mem_funion.mpr (Or.inr hp), hbot, hx'>

theorem fstFinset.insert_left (u : Finset (ProdToken A B)) (p : ProdToken A B)
    (hp : p.val.2 = B.bot) :
    fstFinset A B (insert p u) = insert p.val.1 (fstFinset A B u) := by
  ext x
  constructor
  * intro hx
    rcases (mem_fstFinset A B).1 hx with <q, hq, hbot, hx'>
    rcases Finset.mem_insert.mp hq with rfl | hq
    * exact Finset.mem_insert.mpr (Or.inl hx'.symm)
    * exact Finset.mem_insert.mpr (Or.inr ((mem_fstFinset A B).2 <q, hq, hbot, hx'>))
  * intro hx
    rcases Finset.mem_insert.mp hx with hx' | hx'
    * exact (mem_fstFinset A B).2 <p, Finset.mem_insert_self p u, hp, hx'.symm>
    * rcases (mem_fstFinset A B).1 hx' with <q, hq, hbot, hxq>
      exact (mem_fstFinset A B).2 <q, Finset.mem_insert_of_mem hq, hbot, hxq>

theorem fstFinset.insert_not_left (u : Finset (ProdToken A B)) (p : ProdToken A B)
    (hp : p.val.2  !=  B.bot) :
    fstFinset A B (insert p u) = fstFinset A B u := by
  ext x
  constructor
  * intro hx
    rcases (mem_fstFinset A B).1 hx with <q, hq, hbot, hx'>
    rcases Finset.mem_insert.mp hq with rfl | hq
    * exact False.elim (hp hbot)
    * exact (mem_fstFinset A B).2 <q, hq, hbot, hx'>
  * intro hx
    rcases (mem_fstFinset A B).1 hx with <q, hq, hbot, hx'>
    exact (mem_fstFinset A B).2 <q, Finset.mem_insert_of_mem hq, hbot, hx'>

theorem fstFinset.mono {u v : Finset (ProdToken A B)} (h : v  subseteq  u) :
    fstFinset A B v  subseteq  fstFinset A B u := by
  intro x hx
  rcases (mem_fstFinset A B).1 hx with <p, hp, hbot, hx'>
  exact (mem_fstFinset A B).2 <p, h hp, hbot, hx'>

theorem fstFinset.singleton_left (p : ProdToken A B) (hp : p.val.2 = B.bot) :
    fstFinset A B {p} = {p.val.1} := by
  rw [show ({p} : Finset _) = insert p {} from rfl, fstFinset.insert_left A B {} p hp,
    fstFinset.empty]
  rfl

end InfoSys

end DomainTheory.InformationSystems
```

instLeftCommutativeFstInsert.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
```

```
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    BasicInst5
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    ProdToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    BasicInst7

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a b : Type*} [DecidableEq a] [DecidableEq b]

variable (A : InfoSys a) (B : InfoSys b)

instance instLeftCommutativeFstInsert :
    LeftCommutative fun p : ProdToken A B => (insert p.val.1 : Finset a  ->  Finset a)
     :=
  <fun p q s => insert_comm' p.val.1 q.val.1 s>

end InfoSys

end DomainTheory.InformationSystems
```

instLeftCommutativeSndInsert.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    BasicInst5
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    ProdToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    BasicInst7
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    instLeftCommutativeFstInsert

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a b : Type*} [DecidableEq a] [DecidableEq b]

variable (A : InfoSys a) (B : InfoSys b)
```

```
instance instLeftCommutativeSndInsert :
    LeftCommutative fun p : ProdToken A B => (insert p.val.2 : Finset b  ->  Finset b)
     :=
  <fun p q s => insert_comm' p.val.2 q.val.2 s>

end InfoSys

end DomainTheory.InformationSystems
```

mem_foldr_fst.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    BasicInst5
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    ProdToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    BasicInst7
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    instLeftCommutativeFstInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    instLeftCommutativeSndInsert

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a b : Type*} [DecidableEq a] [DecidableEq b]

variable (A : InfoSys a) (B : InfoSys b)

theorem mem_foldr_fst (s : Multiset (ProdToken A B)) (x : a) :
    x  in  Multiset.foldr (fun p : ProdToken A B => insert p.val.1) ({} : Finset a) s
     <->
      exists  p  in  s, p.val.1 = x := by
  refine Multiset.induction_on s ?_ ?_
  * constructor
    * intro hx
      exact False.elim (Finset.notMem_empty x hx)
    * rintro <p, hp, _>
      exact False.elim (by cases hp)
  * intro p t ih
    simp only [Multiset.foldr_cons, Finset.mem_insert, ih, Multiset.mem_cons]
    constructor
    * rintro (hx | <q, hq, hq1>)
      * exact <p, Or.inl rfl, hx.symm>
      * exact <q, Or.inr hq, hq1>
```

```
    * rintro <q, hq, hq1>
      rcases hq with rfl | hq
      * exact Or.inl hq1.symm
      * exact Or.inr <q, hq, hq1>

end InfoSys

end DomainTheory.InformationSystems
```

mem_foldr_snd.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    BasicInst5
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    ProdToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    BasicInst7
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    instLeftCommutativeFstInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    instLeftCommutativeSndInsert

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a b : Type*} [DecidableEq a] [DecidableEq b]

variable (A : InfoSys a) (B : InfoSys b)

theorem mem_foldr_snd (s : Multiset (ProdToken A B)) (y : b) :
    y  in  Multiset.foldr (fun p : ProdToken A B => insert p.val.2) ({} : Finset b) s
     <->
      exists  p  in  s, p.val.2 = y := by
  refine Multiset.induction_on s ?_ ?_
  * constructor
    * intro hy
      exact False.elim (Finset.notMem_empty y hy)
    * rintro <p, hp, _>
      exact False.elim (by cases hp)
  * intro p t ih
    simp only [Multiset.foldr_cons, Finset.mem_insert, ih, Multiset.mem_cons]
    constructor
    * rintro (hy | <q, hq, hq2>)
      * exact <p, Or.inl rfl, hy.symm>
      * exact <q, Or.inr hq, hq2>
    * rintro <q, hq, hq2>
```

```
      rcases hq with rfl | hq
      * exact Or.inl hq2.symm
      * exact Or.inr <q, hq, hq2>

end InfoSys

end DomainTheory.InformationSystems
```

prodBot.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    BasicInst5
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    ProdToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    BasicInst7

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a b : Type*} [DecidableEq a] [DecidableEq b]

variable (A : InfoSys a) (B : InfoSys b)

/-- Product bottom `Delta_{A x B} = (Delta_A, Delta_B)`. -/
def prodBot : ProdToken A B :=
  <(A.bot, B.bot), <Or.inl rfl>>

end InfoSys

end DomainTheory.InformationSystems
```

sndFinset.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    BasicInst5
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    ProdToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    BasicInst7
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    instLeftCommutativeFstInsert
```

```
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    instLeftCommutativeSndInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    mem_foldr_snd

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a b : Type*} [DecidableEq a] [DecidableEq b]
variable (A : InfoSys a) (B : InfoSys b)

/-- Right projection of a finite set of product tokens (Scott 6.1). -/
def sndFinset (u : Finset (ProdToken A B)) : Finset b :=
  Multiset.foldr (fun p : ProdToken A B => insert p.val.2) ({} : Finset b)
    (u.filter (fun p => p.val.1 = A.bot)).1

theorem mem_sndFinset {u : Finset (ProdToken A B)} {y : b} :
    y  in  sndFinset A B u  <->  exists  p  in  u, p.val.1 = A.bot  /\  p.val.2 = y :=
     by
  unfold sndFinset
  rw [mem_foldr_snd]
  constructor
  * rintro <p, hp, hy>
    have hp' := Finset.mem_filter.mp hp
    exact <p, hp'.1, hp'.2, hy>
  * rintro <p, hp, hbot, hy>
    exact <p, Finset.mem_filter.mpr <hp, hbot>, hy>

theorem sndFinset.empty : sndFinset A B ({} : Finset (ProdToken A B)) = {} := by
  ext y
  constructor
  * intro hy
    rcases (mem_sndFinset A B).1 hy with <_, hp, _, _>
    exact False.elim (Finset.notMem_empty _ hp)
  * intro hy
    exact False.elim (Finset.notMem_empty y hy)

theorem sndFinset.funion (u v : Finset (ProdToken A B)) :
    sndFinset A B (u  U ' v) = sndFinset A B u  U ' sndFinset A B v := by
  ext y
  constructor
  * intro hy
    rcases (mem_sndFinset A B).1 hy with <p, hp, hbot, hy'>
    rcases mem_funion.mp hp with hp | hp
    * exact mem_funion.mpr (Or.inl ((mem_sndFinset A B).2 <p, hp, hbot, hy'>))
    * exact mem_funion.mpr (Or.inr ((mem_sndFinset A B).2 <p, hp, hbot, hy'>))
  * intro hy
    rcases mem_funion.mp hy with hy | hy
    * rcases (mem_sndFinset A B).1 hy with <p, hp, hbot, hy'>
      exact (mem_sndFinset A B).2 <p, mem_funion.mpr (Or.inl hp), hbot, hy'>
    * rcases (mem_sndFinset A B).1 hy with <p, hp, hbot, hy'>
```

```
      exact (mem_sndFinset A B).2 <p, mem_funion.mpr (Or.inr hp), hbot, hy'>

theorem sndFinset.insert_not_right (u : Finset (ProdToken A B)) (p : ProdToken A B)
    (hp : p.val.1  !=  A.bot) :
    sndFinset A B (insert p u) = sndFinset A B u := by
  ext y
  constructor
  * intro hy
    rcases (mem_sndFinset A B).1 hy with <q, hq, hbot, hy'>
    rcases Finset.mem_insert.mp hq with rfl | hq
    * exact False.elim (hp hbot)
    * exact (mem_sndFinset A B).2 <q, hq, hbot, hy'>
  * intro hy
    rcases (mem_sndFinset A B).1 hy with <q, hq, hbot, hy'>
    exact (mem_sndFinset A B).2 <q, Finset.mem_insert_of_mem hq, hbot, hy'>

theorem sndFinset.insert_right (u : Finset (ProdToken A B)) (p : ProdToken A B)
    (hp : p.val.1 = A.bot) :
    sndFinset A B (insert p u) = insert p.val.2 (sndFinset A B u) := by
  ext y
  constructor
  * intro hy
    rcases (mem_sndFinset A B).1 hy with <q, hq, hbot, hy'>
    rcases Finset.mem_insert.mp hq with rfl | hq
    * exact Finset.mem_insert.mpr (Or.inl hy'.symm)
    * exact Finset.mem_insert.mpr (Or.inr ((mem_sndFinset A B).2 <q, hq, hbot, hy'>))
  * intro hy
    rcases Finset.mem_insert.mp hy with hy' | hy'
    * exact (mem_sndFinset A B).2 <p, Finset.mem_insert_self p u, hp, hy'.symm>
    * rcases (mem_sndFinset A B).1 hy' with <q, hq, hbot, hyq>
      exact (mem_sndFinset A B).2 <q, Finset.mem_insert_of_mem hq, hbot, hyq>

theorem sndFinset.mono {u v : Finset (ProdToken A B)} (h : v  subseteq  u) :
    sndFinset A B v  subseteq  sndFinset A B u := by
  intro y hy
  rcases (mem_sndFinset A B).1 hy with <p, hp, hbot, hy'>
  exact (mem_sndFinset A B).2 <p, h hp, hbot, hy'>

theorem sndFinset.singleton_right (p : ProdToken A B) (hp : p.val.1 = A.bot) :
    sndFinset A B {p} = {p.val.2} := by
  rw [show ({p} : Finset _) = insert p {} from rfl, sndFinset.insert_right A B {} p hp
    ,
    sndFinset.empty]
  rfl

end InfoSys

end DomainTheory.InformationSystems
```

ProdCon.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
```

```
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    BasicInst5
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    ProdToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    BasicInst7
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    instLeftCommutativeFstInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    instLeftCommutativeSndInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    fstFinset
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    sndFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a b : Type*} [DecidableEq a] [DecidableEq b]

variable (A : InfoSys a) (B : InfoSys b)

/-- Consistency for the product system (Scott 6.1(iii)). -/
def ProdCon (u : Finset (ProdToken A B)) : Prop :=
  fstFinset A B u  in  A.Con  /\  sndFinset A B u  in  B.Con

end InfoSys

end DomainTheory.InformationSystems
```

ProdEnt.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    BasicInst5
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    ProdToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    BasicInst7
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    instLeftCommutativeFstInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    instLeftCommutativeSndInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    fstFinset
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    sndFinset
```

```
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.ProdCon

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a b : Type*} [DecidableEq a] [DecidableEq b]

variable (A : InfoSys a) (B : InfoSys b)

/-- Entailment for the product system (Scott 6.1(iv'), (iv'')).
Includes `ProdCon` so both projections stay available for `ent_con`. -/
def ProdEnt (u : Finset (ProdToken A B)) (p : ProdToken A B) : Prop :=
  ProdCon A B u  /\
    (p.val.2 = B.bot  ->  A.Ent (fstFinset A B u) p.val.1)  /\
      (p.val.1 = A.bot  ->  B.Ent (sndFinset A B u) p.val.2)

end InfoSys

end DomainTheory.InformationSystems
```

productSystem.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    BasicInst5
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    ProdToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    BasicInst7
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.prodBot
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    instLeftCommutativeFstInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    instLeftCommutativeSndInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    fstFinset
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    sndFinset
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.ProdCon
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.ProdEnt

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive
```

```
namespace InfoSys

variable {a b : Type*} [DecidableEq a] [DecidableEq b]

variable (A : InfoSys a) (B : InfoSys b)

/-- **Definition 6.1.** The product information system `A  x  B`. -/
def productSystem : InfoSys (ProdToken A B) where
  bot := prodBot A B
  Con := {u | ProdCon A B u}
  Ent := ProdEnt A B
  con_subset := by
    intro u v hu hv
    exact <A.con_subset hu.1 (fstFinset.mono A B hv),
      B.con_subset hu.2 (sndFinset.mono A B hv)>
  con_sing := by
    intro p
    refine <?_, ?_>
    * rcases p.property.out with h1 | h2
      * by_cases h2 : p.val.2 = B.bot
        * have hpeq : p = prodBot A B := Subtype.ext (Prod.ext h1 h2)
          subst hpeq
          rw [fstFinset.singleton_left A B (prodBot A B) rfl]
          exact A.con_sing A.bot
        * have hfst : fstFinset A B {p} = {} := by
            ext x
            constructor
            * intro hx
              rcases (mem_fstFinset A B).1 hx with <q, hq, hbot', _>
              have hq' : q = p := Finset.mem_singleton.mp hq
              subst hq'
              exact False.elim (h2 hbot')
            * intro hx
              exact False.elim (Finset.notMem_empty x hx)
          rw [hfst]
          exact A.con_empty
      * rw [fstFinset.singleton_left A B p h2]
        exact A.con_sing p.val.1
    * rcases p.property.out with h1 | h2
      * rw [sndFinset.singleton_right A B p h1]
        exact B.con_sing p.val.2
      * by_cases h1 : p.val.1 = A.bot
        * have hpeq : p = prodBot A B := Subtype.ext (Prod.ext h1 h2)
          subst hpeq
          rw [sndFinset.singleton_right A B (prodBot A B) rfl]
          exact B.con_sing B.bot
        * have hsnd : sndFinset A B {p} = {} := by
            ext y
            constructor
            * intro hy
              rcases (mem_sndFinset A B).1 hy with <q, hq, hbot', _>
              have hq' : q = p := Finset.mem_singleton.mp hq
              subst hq'
              exact False.elim (h1 hbot')
            * intro hy
              exact False.elim (Finset.notMem_empty y hy)
```

```
          rw [hsnd]
          exact B.con_empty
  ent_con := by
    intro u p <<hfst, hsnd>, hL, hR>
    refine <?_, ?_>
    * by_cases hp2 : p.val.2 = B.bot
      * rw [fstFinset.insert_left A B u p hp2]
        exact A.ent_con (hL hp2)
      * rw [fstFinset.insert_not_left A B u p hp2]
        exact hfst
    * by_cases hp1 : p.val.1 = A.bot
      * rw [sndFinset.insert_right A B u p hp1]
        exact B.ent_con (hR hp1)
      * rw [sndFinset.insert_not_right A B u p hp1]
        exact hsnd
  ent_bot := by
    intro u hu
    exact <hu, fun _ => A.ent_bot hu.1, fun _ => B.ent_bot hu.2>
  ent_refl := by
    intro u p hu hp
    refine <hu, ?_, ?_>
    * intro hbot
      exact A.ent_refl hu.1 ((mem_fstFinset A B).2 <p, hp, hbot, rfl>)
    * intro hbot
      exact B.ent_refl hu.2 ((mem_sndFinset A B).2 <p, hp, hbot, rfl>)
  ent_trans := by
    intro u v c hv hu hEnts hEntc
    refine <hv, ?_, ?_>
    * intro hbot
      have hfst : forall  y  in  fstFinset A B u, A.Ent (fstFinset A B v) y := by
        intro y hy
        rcases (mem_fstFinset A B).1 hy with <q, hq, hq2, rfl>
        exact (hEnts q hq).2.1 hq2
      exact A.ent_trans hv.1 hu.1 hfst (hEntc.2.1 hbot)
    * intro hbot
      have hsnd : forall  y  in  sndFinset A B u, B.Ent (sndFinset A B v) y := by
        intro y hy
        rcases (mem_sndFinset A B).1 hy with <q, hq, hq1, rfl>
        exact (hEnts q hq).2.2 hq1
      exact B.ent_trans hv.2 hu.2 hsnd (hEntc.2.2 hbot)

end InfoSys

end DomainTheory.InformationSystems
```

Product.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    IsProdToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    ProdToken
```

```
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.prodBot
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    fstFinset
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    sndFinset
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.ProdCon
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.ProdEnt
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    productSystem
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    BasicInst5
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    BasicInst7
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    instLeftCommutativeFstInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    instLeftCommutativeSndInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    mem_foldr_fst
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product.
    mem_foldr_snd

@[expose] public section
```

SumToken.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a b : Type*} [DecidableEq a] [DecidableEq b]

/-- Token type of the separated sum `A + B` (Scott 6.3(i)-(ii)).
`left X` is `(X, Delta)`, `right Y` is `(Delta, Y)`, and `bot` is `(Delta, Delta)`. -/
inductive SumToken (a b : Type*) where
  | left : a  ->  SumToken a b
  | right : b  ->  SumToken a b
  | bot : SumToken a b

end InfoSys
end DomainTheory.InformationSystems
```

instDecidableEqSumToken.lean

```
module
```

```
public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.SumToken

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a b : Type*} [DecidableEq a] [DecidableEq b]

instance instDecidableEqSumToken : DecidableEq (SumToken a b)
  | .left a, .left b =>
      if h : a = b then isTrue (h  |>  rfl)
      else isFalse fun h' => h (SumToken.left.inj h')
  | .right a, .right b =>
      if h : a = b then isTrue (h  |>  rfl)
      else isFalse fun h' => h (SumToken.right.inj h')
  | .bot, .bot => isTrue rfl
  | .left _, .right _ => isFalse fun h => nomatch h
  | .left _, .bot => isFalse fun h => nomatch h
  | .right _, .left _ => isFalse fun h => nomatch h
  | .right _, .bot => isFalse fun h => nomatch h
  | .bot, .left _ => isFalse fun h => nomatch h
  | .bot, .right _ => isFalse fun h => nomatch h

end InfoSys

end DomainTheory.InformationSystems
```

lftInsert.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.SumToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instDecidableEqSumToken

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a b : Type*} [DecidableEq a] [DecidableEq b]

variable (A : InfoSys a) (B : InfoSys b)
```

```
def lftInsert : SumToken a b  ->  Finset a  ->  Finset a
  | .left x => insert x
  | .right _ => id
  | .bot => id

end InfoSys

end DomainTheory.InformationSystems
```

instLeftCommutativeLftInsert.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.SumToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instDecidableEqSumToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.lftInsert

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a b : Type*} [DecidableEq a] [DecidableEq b]

variable (A : InfoSys a) (B : InfoSys b)

instance instLeftCommutativeLftInsert :
    LeftCommutative (lftInsert : SumToken a b  ->  Finset a  ->  Finset a) :=
  <fun p q s => by
    cases p <;> cases q <;> first | exact insert_comm' _ _ s | rfl>

end InfoSys

end DomainTheory.InformationSystems
```

SumCon_empty.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.SumToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.sumBot
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.lftInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.rhtInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.lftFinset
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.rhtFinset
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.SumCon
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.SumEnt
```

```
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.sumSystem
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instDecidableEqSumToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instLeftCommutativeLftInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instLeftCommutativeRhtInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    mem_foldr_lft
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    mem_foldr_rht

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a b : Type*} [DecidableEq a] [DecidableEq b]

variable (A : InfoSys a) (B : InfoSys b)

theorem SumCon_empty : SumCon A B ({} : Finset (SumToken a b)) :=
  Or.inl <by rw [lftFinset.empty]; exact A.con_empty, by rw [rhtFinset.empty]>

end InfoSys
end DomainTheory.InformationSystems
```

rhtInsert.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.SumToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instDecidableEqSumToken

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a b : Type*} [DecidableEq a] [DecidableEq b]

variable (A : InfoSys a) (B : InfoSys b)

def rhtInsert : SumToken a b  ->  Finset b  ->  Finset b
  | .right y => insert y
  | .left _ => id
```

```
  | .bot => id

end InfoSys

end DomainTheory.InformationSystems
```

instLeftCommutativeRhtInsert.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.SumToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instDecidableEqSumToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.rhtInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instLeftCommutativeLftInsert

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a b : Type*} [DecidableEq a] [DecidableEq b]

variable (A : InfoSys a) (B : InfoSys b)

instance instLeftCommutativeRhtInsert :
    LeftCommutative (rhtInsert : SumToken a b  ->  Finset b  ->  Finset b) :=
  <fun p q s => by
    cases p <;> cases q <;> first | exact insert_comm' _ _ s | rfl>

end InfoSys

end DomainTheory.InformationSystems
```

lftFinset.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.SumToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instDecidableEqSumToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.lftInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instLeftCommutativeLftInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instLeftCommutativeRhtInsert
```

```
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    mem_foldr_lft

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a b : Type*} [DecidableEq a] [DecidableEq b]
variable (A : InfoSys a) (B : InfoSys b)

/-- Left projection `lft u` (Scott 6.3). -/
def lftFinset (u : Finset (SumToken a b)) : Finset a :=
  Multiset.foldr lftInsert ({} : Finset a) u.1

omit [DecidableEq b] in

theorem mem_lftFinset {u : Finset (SumToken a b)} {x : a} :
    x  in  lftFinset u  <->  SumToken.left x  in  u := by
  unfold lftFinset
  rw [mem_foldr_lft]
  constructor
  * rintro <p, hp, hp'>
    rwa [ <-  hp']
  * intro hx
    exact <.left x, hx, rfl>

omit [DecidableEq b] in

theorem lftFinset.empty : lftFinset ({} : Finset (SumToken a b)) = {} := by
  ext x
  constructor
  * intro hx
    exact False.elim (Finset.notMem_empty _ ((mem_lftFinset).1 hx))
  * intro hx
    exact False.elim (Finset.notMem_empty x hx)

theorem lftFinset.funion (u v : Finset (SumToken a b)) :
    lftFinset (u  U ' v) = lftFinset u  U ' lftFinset v := by
  ext x
  constructor
  * intro hx
    have : SumToken.left x  in  u  U ' v := (mem_lftFinset).1 hx
    rcases mem_funion.mp this with h | h
    * exact mem_funion.mpr (Or.inl ((mem_lftFinset).2 h))
    * exact mem_funion.mpr (Or.inr ((mem_lftFinset).2 h))
  * intro hx
    rcases mem_funion.mp hx with h | h
    * exact (mem_lftFinset).2 (mem_funion.mpr (Or.inl ((mem_lftFinset).1 h)))
    * exact (mem_lftFinset).2 (mem_funion.mpr (Or.inr ((mem_lftFinset).1 h)))

theorem lftFinset.insert_bot (u : Finset (SumToken a b)) :
    lftFinset (insert (.bot : SumToken a b) u) = lftFinset u := by
```

```
  ext x
  constructor
  * intro hx
    have : SumToken.left x  in  insert .bot u := (mem_lftFinset).1 hx
    rcases Finset.mem_insert.mp this with h | h
    * exact False.elim (nomatch h)
    * exact (mem_lftFinset).2 h
  * intro hx
    exact (mem_lftFinset).2 (Finset.mem_insert.mpr (Or.inr ((mem_lftFinset).1 hx)))

theorem lftFinset.insert_left (u : Finset (SumToken a b)) (x : a) :
    lftFinset (insert (.left x) u) = insert x (lftFinset u) := by
  ext y
  constructor
  * intro hy
    have : SumToken.left y  in  insert (.left x) u := (mem_lftFinset).1 hy
    rcases Finset.mem_insert.mp this with h | h
    * injection h with hy'
      exact Finset.mem_insert.mpr (Or.inl hy')
    * exact Finset.mem_insert.mpr (Or.inr ((mem_lftFinset).2 h))
  * intro hy
    rcases Finset.mem_insert.mp hy with hy' | hy'
    * exact (mem_lftFinset).2 (Finset.mem_insert.mpr (Or.inl (congrArg SumToken.left
    hy')))
    * exact (mem_lftFinset).2 (Finset.mem_insert.mpr (Or.inr ((mem_lftFinset).1 hy')))

theorem lftFinset.insert_right (u : Finset (SumToken a b)) (y : b) :
    lftFinset (insert (.right y) u) = lftFinset u := by
  ext x
  constructor
  * intro hx
    have : SumToken.left x  in  insert (.right y) u := (mem_lftFinset).1 hx
    rcases Finset.mem_insert.mp this with h | h
    * exact False.elim (nomatch h)
    * exact (mem_lftFinset).2 h
  * intro hx
    exact (mem_lftFinset).2 (Finset.mem_insert.mpr (Or.inr ((mem_lftFinset).1 hx)))

omit [DecidableEq b] in

theorem lftFinset.mono {u v : Finset (SumToken a b)} (h : v  subseteq  u) :
    lftFinset v  subseteq  lftFinset u := by
  intro x hx
  exact (mem_lftFinset).2 (h ((mem_lftFinset).1 hx))

omit [DecidableEq b] in

theorem lftFinset.singleton_bot :
    lftFinset ({.bot} : Finset (SumToken a b)) = {} := by
  ext x
  constructor
  * intro hx
    have : SumToken.left x  in  ({.bot} : Finset _) := (mem_lftFinset).1 hx
    exact False.elim (nomatch Finset.mem_singleton.mp this)
  * intro hx
    exact False.elim (Finset.notMem_empty x hx)
```

```
omit [DecidableEq b] in

theorem lftFinset.singleton_left (x : a) :
    lftFinset ({.left x} : Finset (SumToken a b)) = {x} := by
  ext y
  simp only [mem_lftFinset, Finset.mem_singleton]
  exact <SumToken.left.inj, fun h => h  |>  rfl>

end InfoSys

end DomainTheory.InformationSystems
```

mem_foldr_lft.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.SumToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instDecidableEqSumToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.lftInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instLeftCommutativeLftInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instLeftCommutativeRhtInsert

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a b : Type*} [DecidableEq a] [DecidableEq b]

variable (A : InfoSys a) (B : InfoSys b)

omit [DecidableEq b] in

theorem mem_foldr_lft (s : Multiset (SumToken a b)) (x : a) :
    x  in  Multiset.foldr lftInsert ({} : Finset a) s  <->  exists  p  in  s, p = .
    left x := by
  refine Multiset.induction_on s ?_ ?_
  * constructor
    * intro hx
      exact False.elim (Finset.notMem_empty x hx)
    * rintro <_, hp, _>
      exact False.elim (by cases hp)
  * intro p t ih
    cases p with
    | left a =>
      simp only [Multiset.foldr_cons, lftInsert, Finset.mem_insert, ih, Multiset.
```

```
    mem_cons]
      constructor
      * rintro (hx | <q, hq, hq'>)
        * exact <.left a, Or.inl rfl, congrArg SumToken.left hx.symm>
        * exact <q, Or.inr hq, hq'>
      * rintro <q, hq, hq'>
        rcases hq with rfl | hq
        * injection hq' with hx
          exact Or.inl hx.symm
        * exact Or.inr <q, hq, hq'>
    | right b =>
      simp only [Multiset.foldr_cons, lftInsert, id_eq, ih, Multiset.mem_cons]
      constructor
      * rintro <q, hq, hq'>
        exact <q, Or.inr hq, hq'>
      * rintro <q, hq, hq'>
        rcases hq with rfl | hq
        * exact False.elim (nomatch hq')
        * exact <q, hq, hq'>
    | bot =>
      simp only [Multiset.foldr_cons, lftInsert, id_eq, ih, Multiset.mem_cons]
      constructor
      * rintro <q, hq, hq'>
        exact <q, Or.inr hq, hq'>
      * rintro <q, hq, hq'>
        rcases hq with rfl | hq
        * exact False.elim (nomatch hq')
        * exact <q, hq, hq'>

end InfoSys

end DomainTheory.InformationSystems
```

mem_foldr_rht.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.SumToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instDecidableEqSumToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.rhtInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instLeftCommutativeLftInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instLeftCommutativeRhtInsert

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys
```

```
variable {a b : Type*} [DecidableEq a] [DecidableEq b]

variable (A : InfoSys a) (B : InfoSys b)

omit [DecidableEq a] in

theorem mem_foldr_rht (s : Multiset (SumToken a b)) (y : b) :
    y  in  Multiset.foldr rhtInsert ({} : Finset b) s  <->  exists  p  in  s, p = .
    right y := by
  refine Multiset.induction_on s ?_ ?_
  * constructor
    * intro hy
      exact False.elim (Finset.notMem_empty y hy)
    * rintro <_, hp, _>
      exact False.elim (by cases hp)
  * intro p t ih
    cases p with
    | right b =>
      simp only [Multiset.foldr_cons, rhtInsert, Finset.mem_insert, ih, Multiset.
    mem_cons]
      constructor
      * rintro (hy | <q, hq, hq'>)
        * exact <.right b, Or.inl rfl, congrArg SumToken.right hy.symm>
        * exact <q, Or.inr hq, hq'>
      * rintro <q, hq, hq'>
        rcases hq with rfl | hq
        * injection hq' with hy
          exact Or.inl hy.symm
        * exact Or.inr <q, hq, hq'>
    | left a =>
      simp only [Multiset.foldr_cons, rhtInsert, id_eq, ih, Multiset.mem_cons]
      constructor
      * rintro <q, hq, hq'>
        exact <q, Or.inr hq, hq'>
      * rintro <q, hq, hq'>
        rcases hq with rfl | hq
        * exact False.elim (nomatch hq')
        * exact <q, hq, hq'>
    | bot =>
      simp only [Multiset.foldr_cons, rhtInsert, id_eq, ih, Multiset.mem_cons]
      constructor
      * rintro <q, hq, hq'>
        exact <q, Or.inr hq, hq'>
      * rintro <q, hq, hq'>
        rcases hq with rfl | hq
        * exact False.elim (nomatch hq')
        * exact <q, hq, hq'>

end InfoSys

end DomainTheory.InformationSystems
```

rhtFinset.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.SumToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instDecidableEqSumToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.rhtInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instLeftCommutativeLftInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instLeftCommutativeRhtInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    mem_foldr_rht

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a b : Type*} [DecidableEq a] [DecidableEq b]
variable (A : InfoSys a) (B : InfoSys b)

/-- Right projection `rht u` (Scott 6.3). -/
def rhtFinset (u : Finset (SumToken a b)) : Finset b :=
  Multiset.foldr rhtInsert ({} : Finset b) u.1

omit [DecidableEq a] in

theorem mem_rhtFinset {u : Finset (SumToken a b)} {y : b} :
    y  in  rhtFinset u  <->  SumToken.right y  in  u := by
  unfold rhtFinset
  rw [mem_foldr_rht]
  constructor
  * rintro <p, hp, hp'>
    rwa [ <-  hp']
  * intro hy
    exact <.right y, hy, rfl>

omit [DecidableEq a] in

theorem rhtFinset.empty : rhtFinset ({} : Finset (SumToken a b)) = {} := by
  ext y
  constructor
  * intro hy
    exact False.elim (Finset.notMem_empty _ ((mem_rhtFinset).1 hy))
  * intro hy
    exact False.elim (Finset.notMem_empty y hy)

theorem rhtFinset.funion (u v : Finset (SumToken a b)) :
    rhtFinset (u  U ' v) = rhtFinset u  U ' rhtFinset v := by
  ext y
  constructor
```

```
  * intro hy
    have : SumToken.right y  in  u  U ' v := (mem_rhtFinset).1 hy
    rcases mem_funion.mp this with h | h
    * exact mem_funion.mpr (Or.inl ((mem_rhtFinset).2 h))
    * exact mem_funion.mpr (Or.inr ((mem_rhtFinset).2 h))
  * intro hy
    rcases mem_funion.mp hy with h | h
    * exact (mem_rhtFinset).2 (mem_funion.mpr (Or.inl ((mem_rhtFinset).1 h)))
    * exact (mem_rhtFinset).2 (mem_funion.mpr (Or.inr ((mem_rhtFinset).1 h)))

theorem rhtFinset.insert_bot (u : Finset (SumToken a b)) :
    rhtFinset (insert (.bot : SumToken a b) u) = rhtFinset u := by
  ext y
  constructor
  * intro hy
    have : SumToken.right y  in  insert .bot u := (mem_rhtFinset).1 hy
    rcases Finset.mem_insert.mp this with h | h
    * exact False.elim (nomatch h)
    * exact (mem_rhtFinset).2 h
  * intro hy
    exact (mem_rhtFinset).2 (Finset.mem_insert.mpr (Or.inr ((mem_rhtFinset).1 hy)))

theorem rhtFinset.insert_left (u : Finset (SumToken a b)) (x : a) :
    rhtFinset (insert (.left x) u) = rhtFinset u := by
  ext y
  constructor
  * intro hy
    have : SumToken.right y  in  insert (.left x) u := (mem_rhtFinset).1 hy
    rcases Finset.mem_insert.mp this with h | h
    * exact False.elim (nomatch h)
    * exact (mem_rhtFinset).2 h
  * intro hy
    exact (mem_rhtFinset).2 (Finset.mem_insert.mpr (Or.inr ((mem_rhtFinset).1 hy)))

theorem rhtFinset.insert_right (u : Finset (SumToken a b)) (y : b) :
    rhtFinset (insert (.right y) u) = insert y (rhtFinset u) := by
  ext z
  constructor
  * intro hz
    have : SumToken.right z  in  insert (.right y) u := (mem_rhtFinset).1 hz
    rcases Finset.mem_insert.mp this with h | h
    * injection h with hz'
      exact Finset.mem_insert.mpr (Or.inl hz')
    * exact Finset.mem_insert.mpr (Or.inr ((mem_rhtFinset).2 h))
  * intro hz
    rcases Finset.mem_insert.mp hz with hz' | hz'
    * exact (mem_rhtFinset).2 (Finset.mem_insert.mpr (Or.inl (congrArg SumToken.right
    hz')))
    * exact (mem_rhtFinset).2 (Finset.mem_insert.mpr (Or.inr ((mem_rhtFinset).1 hz')))

omit [DecidableEq a] in

theorem rhtFinset.mono {u v : Finset (SumToken a b)} (h : v  subseteq  u) :
    rhtFinset v  subseteq  rhtFinset u := by
  intro y hy
  exact (mem_rhtFinset).2 (h ((mem_rhtFinset).1 hy))
```

```
omit [DecidableEq a] in

theorem rhtFinset.singleton_bot :
    rhtFinset ({.bot} : Finset (SumToken a b)) = {} := by
  ext y
  constructor
  * intro hy
    have : SumToken.right y  in  ({.bot} : Finset _) := (mem_rhtFinset).1 hy
    exact False.elim (nomatch Finset.mem_singleton.mp this)
  * intro hy
    exact False.elim (Finset.notMem_empty y hy)

omit [DecidableEq a] in

theorem rhtFinset.singleton_right (y : b) :
    rhtFinset ({.right y} : Finset (SumToken a b)) = {y} := by
  ext z
  simp only [mem_rhtFinset, Finset.mem_singleton]
  exact <SumToken.right.inj, fun h => h  |>  rfl>

end InfoSys

end DomainTheory.InformationSystems
```

SumCon.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.SumToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instDecidableEqSumToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instLeftCommutativeLftInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instLeftCommutativeRhtInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.lftFinset
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.rhtFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a b : Type*} [DecidableEq a] [DecidableEq b]
variable (A : InfoSys a) (B : InfoSys b)

/-- Consistency for the separated sum (Scott 6.3(iii)). -/
def SumCon (u : Finset (SumToken a b)) : Prop :=
  (lftFinset u  in  A.Con  /\  rhtFinset u = {})  \/
```

```
    (lftFinset u = {}  /\  rhtFinset u  in  B.Con)

theorem SumCon.lft_con_of_lft_nonempty {u : Finset (SumToken a b)}
    (hu : SumCon A B u) (hne : lftFinset u  !=  {}) : lftFinset u  in  A.Con := by
  rcases hu with <hl, _> | <hl, _>
  * exact hl
  * exact False.elim (hne hl)

theorem SumCon.lft_empty_of_rht_nonempty {u : Finset (SumToken a b)}
    (hu : SumCon A B u) (hne : rhtFinset u  !=  {}) : lftFinset u = {} := by
  rcases hu with <_, hr> | <hl, _>
  * exact False.elim (hne hr)
  * exact hl

theorem SumCon.rht_con_of_rht_nonempty {u : Finset (SumToken a b)}
    (hu : SumCon A B u) (hne : rhtFinset u  !=  {}) : rhtFinset u  in  B.Con := by
  rcases hu with <_, hr> | <_, hr>
  * exact False.elim (hne hr)
  * exact hr

theorem SumCon.rht_empty_of_lft_nonempty {u : Finset (SumToken a b)}
    (hu : SumCon A B u) (hne : lftFinset u  !=  {}) : rhtFinset u = {} := by
  rcases hu with <_, hr> | <hl, _>
  * exact hr
  * exact False.elim (hne hl)

end InfoSys

end DomainTheory.InformationSystems
```

SumEnt.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.SumToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instDecidableEqSumToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instLeftCommutativeLftInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instLeftCommutativeRhtInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.lftFinset
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.rhtFinset
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.SumCon

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys
```

```
variable {a b : Type*} [DecidableEq a] [DecidableEq b]

variable (A : InfoSys a) (B : InfoSys b)

/-- Entailment for the separated sum (Scott 6.3(iv')-(iv''')).
Includes `SumCon` so `ent_con` can update projections. -/
def SumEnt (u : Finset (SumToken a b)) (p : SumToken a b) : Prop :=
  SumCon A B u  /\
    match p with
    | .bot => True
    | .left x => lftFinset u  !=  {}  /\  A.Ent (lftFinset u) x
    | .right y => rhtFinset u  !=  {}  /\  B.Ent (rhtFinset u) y

end InfoSys

end DomainTheory.InformationSystems
```

sumBot.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.SumToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instDecidableEqSumToken

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a b : Type*} [DecidableEq a] [DecidableEq b]

variable (A : InfoSys a) (B : InfoSys b)

/-- Sum bottom `Delta_{A+B} = (Delta, Delta)`. -/
def sumBot : SumToken a b := .bot

end InfoSys

end DomainTheory.InformationSystems
```

sumSystem.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.SumToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instDecidableEqSumToken
```

```
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.sumBot
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instLeftCommutativeLftInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instLeftCommutativeRhtInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.lftFinset
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.rhtFinset
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.SumCon
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.SumEnt

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a b : Type*} [DecidableEq a] [DecidableEq b]

variable (A : InfoSys a) (B : InfoSys b)

/-- **Definition 6.3.** The separated sum information system `A + B`. -/
def sumSystem : InfoSys (SumToken a b) where
  bot := sumBot
  Con := {u | SumCon A B u}
  Ent := SumEnt A B
  con_subset := by
    intro u v hu hv
    rcases hu with <hl, hr> | <hl, hr>
    * exact Or.inl <A.con_subset hl (lftFinset.mono hv),
        Finset.Subset.antisymm (by
          intro y hy
          have : y  in  rhtFinset u := rhtFinset.mono hv hy
          rw [hr] at this
          exact False.elim (Finset.notMem_empty y this))
          (Finset.empty_subset _)>
    * exact Or.inr <Finset.Subset.antisymm (by
          intro x hx
          have : x  in  lftFinset u := lftFinset.mono hv hx
          rw [hl] at this
          exact False.elim (Finset.notMem_empty x this))
          (Finset.empty_subset _),
        B.con_subset hr (rhtFinset.mono hv)>
  con_sing := by
    intro p
    cases p with
    | left x =>
      exact Or.inl <by rw [lftFinset.singleton_left]; exact A.con_sing x,
        by
          ext y
          constructor
          * intro hy
            have : SumToken.right y  in  ({.left x} : Finset _) := (mem_rhtFinset).1
    hy
            exact False.elim (nomatch Finset.mem_singleton.mp this)
```

```
        * intro hy
          exact False.elim (Finset.notMem_empty y hy)>
  | right y =>
    exact Or.inr <by
        ext x
        constructor
        * intro hx
          have : SumToken.left x  in  ({.right y} : Finset _) := (mem_lftFinset).1
 hx
          exact False.elim (nomatch Finset.mem_singleton.mp this)
        * intro hx
          exact False.elim (Finset.notMem_empty x hx),
      by rw [rhtFinset.singleton_right]; exact B.con_sing y>
  | bot =>
    exact Or.inl <by rw [lftFinset.singleton_bot]; exact A.con_empty,
      rhtFinset.singleton_bot>
ent_con := by
  intro u p <hu, hEnt>
  cases p with
  | bot =>
    rcases hu with <hl, hr> | <hl, hr>
    * exact Or.inl <by rw [lftFinset.insert_bot]; exact hl,
        by rw [rhtFinset.insert_bot]; exact hr>
    * exact Or.inr <by rw [lftFinset.insert_bot]; exact hl,
        by rw [rhtFinset.insert_bot]; exact hr>
  | left x =>
    rcases hEnt with <hne, hA>
    have hr : rhtFinset u = {} := SumCon.rht_empty_of_lft_nonempty A B hu hne
    exact Or.inl <by
        rw [lftFinset.insert_left]
        exact A.ent_con hA,
      by rw [rhtFinset.insert_left]; exact hr>
  | right y =>
    rcases hEnt with <hne, hB>
    have hl : lftFinset u = {} := SumCon.lft_empty_of_rht_nonempty A B hu hne
    exact Or.inr <by rw [lftFinset.insert_right]; exact hl,
      by
        rw [rhtFinset.insert_right]
        exact B.ent_con hB>
ent_bot := by
  intro u hu
  exact <hu, trivial>
ent_refl := by
  intro u p hu hp
  refine <hu, ?_>
  cases p with
  | bot => exact trivial
  | left x =>
    have hx : x  in  lftFinset u := (mem_lftFinset).2 hp
    have hne : lftFinset u  !=  {} := Finset.ne_empty_of_mem hx
    exact <hne, A.ent_refl (SumCon.lft_con_of_lft_nonempty A B hu hne) hx>
  | right y =>
    have hy : y  in  rhtFinset u := (mem_rhtFinset).2 hp
    have hne : rhtFinset u  !=  {} := Finset.ne_empty_of_mem hy
    exact <hne, B.ent_refl (SumCon.rht_con_of_rht_nonempty A B hu hne) hy>
ent_trans := by
```

```
    intro u v c hv hu hEnts hEntc
    refine <hv, ?_>
    cases c with
    | bot => exact trivial
    | left x =>
      rcases hEntc with <_, <hne, hA>>
      have hlft : forall  y  in  lftFinset u, A.Ent (lftFinset v) y := by
        intro y hy
        have hp : SumToken.left y  in  u := (mem_lftFinset).1 hy
        have hEy : SumEnt A B v (.left y) := hEnts _ hp
        exact hEy.2.2
      have hne' : lftFinset v  !=  {} := by
        obtain <y, hy> := Finset.nonempty_of_ne_empty hne
        have hp : SumToken.left y  in  u := (mem_lftFinset).1 hy
        exact (hEnts _ hp).2.1
      refine <hne', ?_>
      exact A.ent_trans (SumCon.lft_con_of_lft_nonempty A B hv hne')
        (SumCon.lft_con_of_lft_nonempty A B hu hne) hlft hA
    | right y =>
      rcases hEntc with <_, <hne, hB>>
      have hrht : forall  z  in  rhtFinset u, B.Ent (rhtFinset v) z := by
        intro z hz
        have hp : SumToken.right z  in  u := (mem_rhtFinset).1 hz
        have hEz : SumEnt A B v (.right z) := hEnts _ hp
        exact hEz.2.2
      have hne' : rhtFinset v  !=  {} := by
        obtain <z, hz> := Finset.nonempty_of_ne_empty hne
        have hp : SumToken.right z  in  u := (mem_rhtFinset).1 hz
        exact (hEnts _ hp).2.1
      refine <hne', ?_>
      exact B.ent_trans (SumCon.rht_con_of_rht_nonempty A B hv hne')
        (SumCon.rht_con_of_rht_nonempty A B hu hne) hrht hB

end InfoSys

end DomainTheory.InformationSystems
```

Sum.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.SumToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.sumBot
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.lftInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.rhtInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.lftFinset
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.rhtFinset
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.SumCon
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.SumEnt
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.sumSystem
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instDecidableEqSumToken
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
```

```
    instLeftCommutativeLftInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    instLeftCommutativeRhtInsert
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    mem_foldr_lft
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    mem_foldr_rht
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum.
    SumCon_empty

@[expose] public section
```

4.7 The function space as a domain

Dependencies. Inside FunctionSpaceAsADomain, each box is one subfolder, or one Lean file sitting directly in the folder. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. The arrowhead points at the file or folder that imports, and the number is how many direct imports.

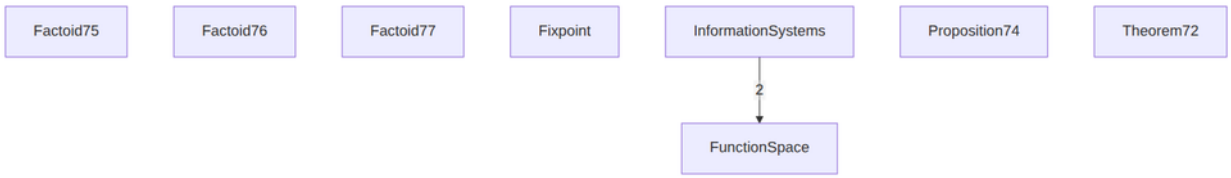


Figure 44: The function space as a domain.

Functions or mappings between domains are of basic importance for our theory, since it is through them that we most easily transform data and relate the structures into which the elements defined by the data objects enter. There are many possible functions, and large groups of them can be treated in a uniform manner. For instance, if the source and target domains match properly, any pair of functions can be composed—composition is an operation on functions of general significance. Now, if in the theory we could combine functions into domains themselves, then an operation like composition might become a mapping of the theory. Indeed, this is exactly what happens: suitably construed, composition is an approximable mapping of two arguments. Of course, for each configuration of linked source and target domains, there is a separate composition operation.

In order to make approximable mappings elements of a suitable domain, we have to discover first what their appropriate data objects are. In Section 5 this was hinted at already. To determine an approximable mapping $f : \mathbf{A} \to \mathbf{B}$, we have to say which pairs (u, v) with $u \in D_{\mathbf{A}}$ and $v \in D_{\mathbf{B}}$ stand in the mapping relation $u\ f\ v$. One such pair gives a certain (finite) amount of information about the possible functions that contain it, and an approximable mapping is completely determined by such pairs. Therefore, if there are appropriate notions of consistency and entailment for these pairs, we will be able to form a domain having functions as elements. Let us try out a formal definition first, and then look to an explanation of how it works.

- Definition 7.1
- Theorem 7.2

- Theorem 7.3
- Proposition 7.4

FunToken.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v

variable {a : Type u} {b : Type v} [DecidableEq a] [DecidableEq b]

/-- Token type of the function space `A  ->  B` (Scott 7.1(i)). -/
def FunToken (A : InfoSys a) (B : InfoSys b) : Type _ :=
  {p : Finset a  x  Finset b // p.1  in  A.Con  /\  p.2  in  B.Con}

end InfoSys

end DomainTheory.InformationSystems
```

BasicInst7.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    FunToken

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v

variable {a : Type u} {b : Type v} [DecidableEq a] [DecidableEq b]

instance (A : InfoSys a) (B : InfoSys b) : DecidableEq (FunToken A B) :=
```

```
  fun p q =>
    match decidableEq_finset p.val.1 q.val.1, decidableEq_finset p.val.2 q.val.2 with
    | isTrue h1, isTrue h2 => isTrue (Subtype.ext (Prod.ext h1 h2))
    | isFalse n1, _ =>
        isFalse fun h => n1 (congrArg (fun r : FunToken A B => r.val.1) h)
    | isTrue _, isFalse n2 =>
        isFalse fun h => n2 (congrArg (fun r : FunToken A B => r.val.2) h)

end InfoSys

end DomainTheory.InformationSystems
```

funBot.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    FunToken
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    BasicInst7

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type u} {b : Type v} [DecidableEq a] [DecidableEq b]
variable (A : InfoSys a) (B : InfoSys b)
universe u v

/-- The empty pair is a function-space token. -/
theorem funBot.property : ({} : Finset a)  in  A.Con  /\  ({} : Finset b)  in  B.Con
    :=
  <A.con_empty, B.con_empty>

/-- Function-space bottom `Delta_{A -> B} = ({}, {})` (Scott 7.1(ii)). -/
def funBot : FunToken A B :=
  <({}, {}), funBot.property A B>

end InfoSys

end DomainTheory.InformationSystems
```

funBot.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
```

```
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    FunToken
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    BasicInst7
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    instLeftCommutativeFunInput
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    instLeftCommutativeFunOutput
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    mem_foldr_funInput
public import DomainTheory.InformationSystems.InformationSystems.Proposition23.
    proposition_2_3_ii

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type u} {b : Type v} [DecidableEq a] [DecidableEq b]
variable (A : InfoSys a) (B : InfoSys b)
universe u v

/-- Union of input components of a finite set of function-space tokens. -/
def funInputUnion (s : Finset (FunToken A B)) : Finset a :=
  Multiset.foldr (fun p : FunToken A B => funion p.val.1) ({} : Finset a) s.1

theorem funInputUnion.empty : funInputUnion A B ({} : Finset (FunToken A B)) = {} :=
  rfl

theorem mem_funInputUnion {s : Finset (FunToken A B)} {x : a} :
    x  in  funInputUnion A B s  <->  exists  p  in  s, x  in  p.val.1 := by
  simpa [funInputUnion] using mem_foldr_funInput A B s.1 x

theorem entSet_inputUnion_of_ent {u' : Finset a} {s : Finset (FunToken A B)}
    (h : forall  q  in  s, A.EntSet u' q.val.1) :
    A.EntSet u' (funInputUnion A B s) := by
  intro x hx
  rcases (mem_funInputUnion A B).1 hx with <q, hq, hx'>
  exact h q hq x hx'

theorem funInputUnion.con_of_ent {u' : Finset a} {s : Finset (FunToken A B)}
    (hu' : u'  in  A.Con) (h : forall  q  in  s, A.EntSet u' q.val.1) :
    funInputUnion A B s  in  A.Con :=
  A.con_subset (proposition_2_3_ii A hu' (entSet_inputUnion_of_ent A B h))
    (subset_funion_right _ _)

theorem funInputUnion.funion (s t : Finset (FunToken A B)) :
    funInputUnion A B (s  U ' t) = funInputUnion A B s  U ' funInputUnion A B t := by
  ext x
  constructor
  * intro hx
    rcases (mem_funInputUnion A B).1 hx with <p, hp, hx'>
    rcases mem_funion.mp hp with hp | hp
```

```
    * exact mem_funion.mpr (Or.inl ((mem_funInputUnion A B).2 <p, hp, hx'>))
    * exact mem_funion.mpr (Or.inr ((mem_funInputUnion A B).2 <p, hp, hx'>))
  * intro hx
    rcases mem_funion.mp hx with hx | hx
    * rcases (mem_funInputUnion A B).1 hx with <p, hp, hx'>
      exact (mem_funInputUnion A B).2 <p, mem_funion.mpr (Or.inl hp), hx'>
    * rcases (mem_funInputUnion A B).1 hx with <p, hp, hx'>
      exact (mem_funInputUnion A B).2 <p, mem_funion.mpr (Or.inr hp), hx'>

theorem funInputUnion.singleton (p : FunToken A B) :
    funInputUnion A B {p} = p.val.1 := by
  ext x
  constructor
  * intro hx
    rcases (mem_funInputUnion A B).1 hx with <q, hq, hx'>
    have : q = p := Finset.mem_singleton.mp hq
    subst this
    exact hx'
  * intro hx
    exact (mem_funInputUnion A B).2 <p, Finset.mem_singleton_self p, hx>

theorem funInputUnion.insert (s : Finset (FunToken A B)) (p : FunToken A B) :
    funInputUnion A B (Insert.insert p s) = p.val.1  U ' funInputUnion A B s := by
  have h : Insert.insert p s = ({p} : Finset (FunToken A B))  U ' s := by
    ext q
    constructor
    * intro hq
      rcases Finset.mem_insert.mp hq with hqp | hq
      * exact mem_funion.mpr (Or.inl (hqp  |>  Finset.mem_singleton_self p))
      * exact mem_funion.mpr (Or.inr hq)
    * intro hq
      rcases mem_funion.mp hq with hq | hq
      * exact Finset.mem_insert.mpr (Or.inl (Finset.mem_singleton.mp hq))
      * exact Finset.mem_insert_of_mem hq
  rw [h, funInputUnion.funion, funInputUnion.singleton]

end InfoSys

end DomainTheory.InformationSystems
```

funOutputUnion.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    FunToken
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    BasicInst7
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    instLeftCommutativeFunInput
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    instLeftCommutativeFunOutput
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
```

```
    mem_foldr_funOutput

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type u} {b : Type v} [DecidableEq a] [DecidableEq b]
variable (A : InfoSys a) (B : InfoSys b)
universe u v

/-- Union of output components of a finite set of function-space tokens. -/
def funOutputUnion (s : Finset (FunToken A B)) : Finset b :=
  Multiset.foldr (fun p : FunToken A B => funion p.val.2) ({} : Finset b) s.1

theorem funOutputUnion.empty : funOutputUnion A B ({} : Finset (FunToken A B)) = {} :=
  rfl

theorem mem_funOutputUnion {s : Finset (FunToken A B)} {y : b} :
    y  in  funOutputUnion A B s  <->  exists  p  in  s, y  in  p.val.2 := by
  simpa [funOutputUnion] using mem_foldr_funOutput A B s.1 y

theorem funOutputUnion.funion (s t : Finset (FunToken A B)) :
    funOutputUnion A B (s  U ' t) = funOutputUnion A B s  U ' funOutputUnion A B t :=
    by
  ext y
  constructor
  * intro hy
    rcases (mem_funOutputUnion A B).1 hy with <p, hp, hy'>
    rcases mem_funion.mp hp with hp | hp
    * exact mem_funion.mpr (Or.inl ((mem_funOutputUnion A B).2 <p, hp, hy'>))
    * exact mem_funion.mpr (Or.inr ((mem_funOutputUnion A B).2 <p, hp, hy'>))
  * intro hy
    rcases mem_funion.mp hy with hy | hy
    * rcases (mem_funOutputUnion A B).1 hy with <p, hp, hy'>
      exact (mem_funOutputUnion A B).2 <p, mem_funion.mpr (Or.inl hp), hy'>
    * rcases (mem_funOutputUnion A B).1 hy with <p, hp, hy'>
      exact (mem_funOutputUnion A B).2 <p, mem_funion.mpr (Or.inr hp), hy'>

theorem funOutputUnion.singleton (p : FunToken A B) :
    funOutputUnion A B {p} = p.val.2 := by
  ext y
  constructor
  * intro hy
    rcases (mem_funOutputUnion A B).1 hy with <q, hq, hy'>
    have : q = p := Finset.mem_singleton.mp hq
    subst this
    exact hy'
  * intro hy
    exact (mem_funOutputUnion A B).2 <p, Finset.mem_singleton_self p, hy>

theorem funOutputUnion.insert (s : Finset (FunToken A B)) (p : FunToken A B) :
    funOutputUnion A B (Insert.insert p s) = p.val.2  U ' funOutputUnion A B s := by
```

```
  have h : Insert.insert p s = ({p} : Finset (FunToken A B))  U ' s := by
    ext q
    constructor
    * intro hq
      rcases Finset.mem_insert.mp hq with hqp | hq
      * exact mem_funion.mpr (Or.inl (hqp  |>  Finset.mem_singleton_self p))
      * exact mem_funion.mpr (Or.inr hq)
    * intro hq
      rcases mem_funion.mp hq with hq | hq
      * exact Finset.mem_insert.mpr (Or.inl (Finset.mem_singleton.mp hq))
      * exact Finset.mem_insert_of_mem hq
  rw [h, funOutputUnion.funion, funOutputUnion.singleton]

end InfoSys

end DomainTheory.InformationSystems
```

funion_assoc'.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    BasicInst7

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v

variable {a : Type u} {b : Type v} [DecidableEq a] [DecidableEq b]

variable (A : InfoSys a) (B : InfoSys b)

theorem funion_assoc' (a b c : Finset b) :
    (a  U ' b)  U ' c = a  U ' (b  U ' c) := by
  ext x
  constructor
  * intro hx
    rcases mem_funion.mp hx with h | h
    * rcases mem_funion.mp h with h | h
      * exact mem_funion.mpr (Or.inl h)
      * exact mem_funion.mpr (Or.inr (mem_funion.mpr (Or.inl h)))
    * exact mem_funion.mpr (Or.inr (mem_funion.mpr (Or.inr h)))
  * intro hx
    rcases mem_funion.mp hx with h | h
    * exact mem_funion.mpr (Or.inl (mem_funion.mpr (Or.inl h)))
    * rcases mem_funion.mp h with h | h
```

```
      * exact mem_funion.mpr (Or.inl (mem_funion.mpr (Or.inr h)))
      * exact mem_funion.mpr (Or.inr h)

end InfoSys

end DomainTheory.InformationSystems
```

funion_assoc.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    BasicInst7

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v

variable {a : Type u} {b : Type v} [DecidableEq a] [DecidableEq b]

variable (A : InfoSys a) (B : InfoSys b)

theorem funion_assoc (a b c : Finset a) :
    (a  U ' b)  U ' c = a  U ' (b  U ' c) := by
  ext x
  constructor
  * intro hx
    rcases mem_funion.mp hx with h | h
    * rcases mem_funion.mp h with h | h
      * exact mem_funion.mpr (Or.inl h)
      * exact mem_funion.mpr (Or.inr (mem_funion.mpr (Or.inl h)))
    * exact mem_funion.mpr (Or.inr (mem_funion.mpr (Or.inr h)))
  * intro hx
    rcases mem_funion.mp hx with h | h
    * exact mem_funion.mpr (Or.inl (mem_funion.mpr (Or.inl h)))
    * rcases mem_funion.mp h with h | h
      * exact mem_funion.mpr (Or.inl (mem_funion.mpr (Or.inr h)))
      * exact mem_funion.mpr (Or.inr h)

end InfoSys

end DomainTheory.InformationSystems
```

funion_comm'.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    BasicInst7

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v

variable {a : Type u} {b : Type v} [DecidableEq a] [DecidableEq b]

variable (A : InfoSys a) (B : InfoSys b)

theorem funion_comm' (a b : Finset b) : a  U ' b = b  U ' a := by
  ext x
  constructor
  * intro hx
    rcases mem_funion.mp hx with h | h
    * exact mem_funion.mpr (Or.inr h)
    * exact mem_funion.mpr (Or.inl h)
  * intro hx
    rcases mem_funion.mp hx with h | h
    * exact mem_funion.mpr (Or.inr h)
    * exact mem_funion.mpr (Or.inl h)

end InfoSys

end DomainTheory.InformationSystems
```

funion_comm.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    BasicInst7

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys
```

```
universe u v

variable {a : Type u} {b : Type v} [DecidableEq a] [DecidableEq b]

variable (A : InfoSys a) (B : InfoSys b)

theorem funion_comm (a b : Finset a) : a  U ' b = b  U ' a := by
  ext x
  constructor
  * intro hx
    rcases mem_funion.mp hx with h | h
    * exact mem_funion.mpr (Or.inr h)
    * exact mem_funion.mpr (Or.inl h)
  * intro hx
    rcases mem_funion.mp hx with h | h
    * exact mem_funion.mpr (Or.inr h)
    * exact mem_funion.mpr (Or.inl h)

end InfoSys

end DomainTheory.InformationSystems
```

funion_left_comm'.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    BasicInst7

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v

variable {a : Type u} {b : Type v} [DecidableEq a] [DecidableEq b]

variable (A : InfoSys a) (B : InfoSys b)

theorem funion_left_comm' (a b c : Finset b) :
    a  U ' (b  U ' c) = b  U ' (a  U ' c) := by
  ext x
  constructor
  * intro hx
    rcases mem_funion.mp hx with h | h
    * exact mem_funion.mpr (Or.inr (mem_funion.mpr (Or.inl h)))
    * rcases mem_funion.mp h with h | h
      * exact mem_funion.mpr (Or.inl h)
      * exact mem_funion.mpr (Or.inr (mem_funion.mpr (Or.inr h)))
```

```
  * intro hx
    rcases mem_funion.mp hx with h | h
    * exact mem_funion.mpr (Or.inr (mem_funion.mpr (Or.inl h)))
    * rcases mem_funion.mp h with h | h
      * exact mem_funion.mpr (Or.inl h)
      * exact mem_funion.mpr (Or.inr (mem_funion.mpr (Or.inr h)))

end InfoSys

end DomainTheory.InformationSystems
```

funion_left_comm.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    BasicInst7

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v

variable {a : Type u} {b : Type v} [DecidableEq a] [DecidableEq b]

variable (A : InfoSys a) (B : InfoSys b)

theorem funion_left_comm (a b c : Finset a) :
    a  U ' (b  U ' c) = b  U ' (a  U ' c) := by
  ext x
  constructor
  * intro hx
    rcases mem_funion.mp hx with h | h
    * exact mem_funion.mpr (Or.inr (mem_funion.mpr (Or.inl h)))
    * rcases mem_funion.mp h with h | h
      * exact mem_funion.mpr (Or.inl h)
      * exact mem_funion.mpr (Or.inr (mem_funion.mpr (Or.inr h)))
  * intro hx
    rcases mem_funion.mp hx with h | h
    * exact mem_funion.mpr (Or.inr (mem_funion.mpr (Or.inl h)))
    * rcases mem_funion.mp h with h | h
      * exact mem_funion.mpr (Or.inl h)
      * exact mem_funion.mpr (Or.inr (mem_funion.mpr (Or.inr h)))

end InfoSys

end DomainTheory.InformationSystems
```

instLeftCommutativeFunInput.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    FunToken
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    BasicInst7
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    funion_left_comm

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v

variable {a : Type u} {b : Type v} [DecidableEq a] [DecidableEq b]

variable (A : InfoSys a) (B : InfoSys b)

instance instLeftCommutativeFunInput :
    LeftCommutative fun p : FunToken A B => (funion p.val.1 : Finset a  ->  Finset a)
    :=
  <fun p q s => funion_left_comm p.val.1 q.val.1 s>

end InfoSys

end DomainTheory.InformationSystems
```

instLeftCommutativeFunOutput.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    FunToken
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    BasicInst7
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    funion_left_comm'
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    instLeftCommutativeFunInput

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive
```

```
namespace InfoSys

universe u v

variable {a : Type u} {b : Type v} [DecidableEq a] [DecidableEq b]

variable (A : InfoSys a) (B : InfoSys b)

instance instLeftCommutativeFunOutput :
    LeftCommutative fun p : FunToken A B => (funion p.val.2 : Finset b  ->  Finset b)
    :=
  <fun p q s => funion_left_comm' p.val.2 q.val.2 s>

end InfoSys

end DomainTheory.InformationSystems
```

filter_ne_subset_of_subset_insert.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    FunToken
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    BasicInst7
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    instLeftCommutativeFunInput
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    instLeftCommutativeFunOutput

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v

variable {a : Type u} {b : Type v} [DecidableEq a] [DecidableEq b]

variable (A : InfoSys a) (B : InfoSys b)

theorem filter_ne_subset_of_subset_insert {p : FunToken A B}
    {t w : Finset (FunToken A B)} (ht : t  subseteq  insert p w) :
    t.filter (*  !=  p)  subseteq  w := by
  intro q hq
  have <hqt, hne> := Finset.mem_filter.mp hq
  have : q  in  insert p w := ht hqt
  rcases Finset.mem_insert.mp this with rfl | hqW
```

```
  * exact False.elim (hne rfl)
  * exact hqW

end InfoSys

end DomainTheory.InformationSystems
```

FunCon.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    FunToken
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    BasicInst7
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    instLeftCommutativeFunInput
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    instLeftCommutativeFunOutput
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    funInputUnion
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    funOutputUnion

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v

variable {a : Type u} {b : Type v} [DecidableEq a] [DecidableEq b]

variable (A : InfoSys a) (B : InfoSys b)

/-- Consistency for the function space (Scott 7.1(iii)). -/
def FunCon (w : Finset (FunToken A B)) : Prop :=
  forall  s  subseteq  w, funInputUnion A B s  in  A.Con  ->  funOutputUnion A B s  in
      B.Con

end InfoSys

end DomainTheory.InformationSystems
```

FunEnt.lean

```
module
```

```
public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    FunToken
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    BasicInst7
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    instLeftCommutativeFunInput
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    instLeftCommutativeFunOutput
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    funOutputUnion
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    FunCon

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v

variable {a : Type u} {b : Type v} [DecidableEq a] [DecidableEq b]

variable (A : InfoSys a) (B : InfoSys b)

/-- Entailment for the function space (Scott 7.1(iv), constructive witness form).
Includes `FunCon` so `ent_con` has the ambient consistency hypothesis. -/
def FunEnt (w : Finset (FunToken A B)) (p : FunToken A B) : Prop :=
  FunCon A B w  /\
    exists  s  subseteq  w, (forall  q  in  s, A.EntSet p.val.1 q.val.1)  /\
      B.EntSet (funOutputUnion A B s) p.val.2

end InfoSys

end DomainTheory.InformationSystems
```

functionSystem.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    FunToken
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    BasicInst7
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    funBot
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    instLeftCommutativeFunInput
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
```

```
    instLeftCommutativeFunOutput
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    FunCon
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    FunEnt
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    exists_combined_witness
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    filter_ne_subset_of_subset_insert
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    funInputUnion
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    funOutputUnion
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    funion_assoc
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    funion_assoc'
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    funion_comm
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    funion_comm'
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    insert_filter_ne
public import DomainTheory.InformationSystems.InformationSystems.Proposition23.
    proposition_2_3_ii
public import DomainTheory.InformationSystems.InformationSystems.Proposition23.
    proposition_2_3_iii
public import DomainTheory.InformationSystems.InformationSystems.Proposition23.
    proposition_2_3_iv
public import DomainTheory.InformationSystems.InformationSystems.Proposition23.
    proposition_2_3_v

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type u} {b : Type v} [DecidableEq a] [DecidableEq b]
variable (A : InfoSys a) (B : InfoSys b)
universe u v

/-- Singletons are consistent in the function space. -/
theorem functionSystem.con_sing (p : FunToken A B) : FunCon A B {p} := by
  intro s hs _hin
  have hsub : funOutputUnion A B s  subseteq  p.val.2 := by
    intro y hy
    rcases (mem_funOutputUnion A B).1 hy with <q, hq, hy'>
    have hq' : q = p := Finset.mem_singleton.mp (hs hq)
    cases hq'
    exact hy'
  exact B.con_subset p.property.2 hsub

/-- Downward closure of function-space consistency. -/
```

```
theorem functionSystem.con_subset {w w' : Finset (FunToken A B)}
    (hw : FunCon A B w) (hw' : w'  subseteq  w) : FunCon A B w' :=
  fun s hs hin => hw s (fun _ hq => hw' (hs hq)) hin

/-- The function-space bottom is entailed by every consistent set. -/
theorem functionSystem.ent_bot {w : Finset (FunToken A B)} (hw : FunCon A B w) :
    FunEnt A B w (funBot A B) :=
  <hw, {}, Finset.empty_subset _,
    fun q hq => False.elim (Finset.notMem_empty q hq),
    fun y hy => False.elim (Finset.notMem_empty y hy)>

/-- Adding an entailed function-space token preserves consistency. -/
theorem functionSystem.ent_con {w : Finset (FunToken A B)} {p : FunToken A B}
    (hEnt : FunEnt A B w p) : FunCon A B (insert p w) := by
  obtain <hw, s, hs, hEntIn, hEntOut> := hEnt
  intro t ht hin
  if hp : p  in  t then
    let t' := t.filter (*  !=  p)
    have ht_eq : insert p t' = t := insert_filter_ne A B hp
    have ht'sub : t'  subseteq  w := filter_ne_subset_of_subset_insert A B ht
    have hin' : p.val.1  U ' funInputUnion A B t'  in  A.Con := by
      rwa [ <-  ht_eq, funInputUnion.insert] at hin
    have hsub : s  U ' t'  subseteq  w := funion_subset_iff.mpr <hs, ht'sub>
    have hEnt_in_s : A.EntSet p.val.1 (funInputUnion A B s) :=
      entSet_inputUnion_of_ent A B hEntIn
    have hEnt_big : A.EntSet (p.val.1  U ' funInputUnion A B t') (funInputUnion A B s)
     :=
      proposition_2_3_v A p.property.1 hin' (subset_funion_left _ _) hEnt_in_s
        (Finset.Subset.refl _)
    have hbig : (p.val.1  U ' funInputUnion A B t')  U ' funInputUnion A B s  in  A.
    Con :=
      proposition_2_3_ii A hin' hEnt_big
    have hin_st' : funInputUnion A B s  U ' funInputUnion A B t'  in  A.Con := by
      have heq :
          (p.val.1  U ' funInputUnion A B t')  U ' funInputUnion A B s =
            p.val.1  U ' (funInputUnion A B s  U ' funInputUnion A B t') := by
        rw [funion_assoc, funion_comm (funInputUnion A B t')]
      have hbig' : p.val.1  U ' (funInputUnion A B s  U ' funInputUnion A B t')  in  A
    .Con := by
        rwa [heq] at hbig
      exact A.con_subset hbig' (subset_funion_right _ _)
    have hin_union : funInputUnion A B (s  U ' t')  in  A.Con := by
      rwa [funInputUnion.funion]
    have hout_union : funOutputUnion A B s  U ' funOutputUnion A B t'  in  B.Con := by
      have : funOutputUnion A B (s  U ' t')  in  B.Con := hw _ hsub hin_union
      rwa [funOutputUnion.funion] at this
    have hout_s : funOutputUnion A B s  in  B.Con :=
      B.con_subset hout_union (subset_funion_left _ _)
    have hEnt_out' : B.EntSet (funOutputUnion A B s  U ' funOutputUnion A B t') p.val
    .2 :=
      proposition_2_3_v B hout_s hout_union (subset_funion_left _ _) hEntOut
        (Finset.Subset.refl _)
    have hfinal :
        (funOutputUnion A B s  U ' funOutputUnion A B t')  U ' p.val.2  in  B.Con :=
      proposition_2_3_ii B hout_union hEnt_out'
    have hout_t : p.val.2  U ' funOutputUnion A B t'  in  B.Con := by
```

```
      have heq :
          (funOutputUnion A B s  U ' funOutputUnion A B t')  U ' p.val.2 =
            funOutputUnion A B s  U ' (p.val.2  U ' funOutputUnion A B t') := by
        rw [funion_assoc', funion_comm' (funOutputUnion A B t')]
      have hfinal' :
          funOutputUnion A B s  U ' (p.val.2  U ' funOutputUnion A B t')  in  B.Con :=
     by
        rwa [heq] at hfinal
      exact B.con_subset hfinal' (subset_funion_right _ _)
    have : funOutputUnion A B t  in  B.Con := by
      rwa [ <-  ht_eq, funOutputUnion.insert]
    exact this
  else
    have ht' : t  subseteq  w := by
      intro q hq
      have : q  in  insert p w := ht hq
      rcases Finset.mem_insert.mp this with rfl | hqW
      * exact False.elim (hp hq)
      * exact hqW
    exact hw t ht' hin

/-- Function-space entailment is reflexive on members. -/
theorem functionSystem.ent_refl {w : Finset (FunToken A B)} {p : FunToken A B}
    (hw : FunCon A B w) (hp : p  in  w) : FunEnt A B w p :=
  <hw, {p},
    fun q hq => by
      cases Finset.mem_singleton.mp hq
      exact hp,
    fun q hq => by
      cases Finset.mem_singleton.mp hq
      exact proposition_2_3_iii A p.property.1,
    by
      rw [funOutputUnion.singleton]
      exact proposition_2_3_iii B p.property.2>

/-- Function-space entailment is transitive. -/
theorem functionSystem.ent_trans {u v : Finset (FunToken A B)} {c : FunToken A B}
    (hv : FunCon A B v) (hu : FunCon A B u)
    (hEnts : forall  y  in  u, FunEnt A B v y) (hEnt : FunEnt A B u c) :
    FunEnt A B v c := by
  obtain <_, s, hs, hEntIn, hEntOut> := hEnt
  refine <hv, ?_>
  have hWit : forall  q  in  s, exists  s_q  subseteq  v, (forall  r  in  s_q, A.
    EntSet q.val.1 r.val.1)  /\
      B.EntSet (funOutputUnion A B s_q) q.val.2 := by
    intro q hq
    exact (hEnts q (hs hq)).2
  obtain <S, hSsub, hSin, hSout> :=
    exists_combined_witness A B hv c.property.1 hEntIn hWit
  refine <S, hSsub, hSin, ?_>
  have hin_s : funInputUnion A B s  in  A.Con :=
    funInputUnion.con_of_ent A B c.property.1 hEntIn
  have hout_s : funOutputUnion A B s  in  B.Con := hu s hs hin_s
  have hin_S : funInputUnion A B S  in  A.Con :=
    funInputUnion.con_of_ent A B c.property.1 hSin
  have hout_S : funOutputUnion A B S  in  B.Con := hv S hSsub hin_S
```

```
  exact proposition_2_3_iv B hout_S hout_s hSout hEntOut

/-- **Definition 7.1.** The function-space information system `A  ->  B`. -/
def functionSystem : InfoSys (FunToken A B) where
  bot := funBot A B
  Con := {w | FunCon A B w}
  Ent := FunEnt A B
  con_subset := functionSystem.con_subset A B
  con_sing := functionSystem.con_sing A B
  ent_con := functionSystem.ent_con A B
  ent_bot := functionSystem.ent_bot A B
  ent_refl := functionSystem.ent_refl A B
  ent_trans := functionSystem.ent_trans A B

end InfoSys

end DomainTheory.InformationSystems
```

insert_filter_ne.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    FunToken
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    BasicInst7
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    instLeftCommutativeFunInput
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    instLeftCommutativeFunOutput

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v

variable {a : Type u} {b : Type v} [DecidableEq a] [DecidableEq b]

variable (A : InfoSys a) (B : InfoSys b)

theorem insert_filter_ne {p : FunToken A B} {t : Finset (FunToken A B)}
    (hp : p  in  t) : insert p (t.filter (*  !=  p)) = t := by
  ext q
  constructor
  * intro hq
    rcases Finset.mem_insert.mp hq with rfl | hq
    * exact hp
    * exact (Finset.mem_filter.mp hq).1
```

```
  * intro hq
    if h : q = p then
      exact Finset.mem_insert.mpr (Or.inl h)
    else
      exact Finset.mem_insert.mpr (Or.inr (Finset.mem_filter.mpr <hq, h>))

end InfoSys

end DomainTheory.InformationSystems
```

mem_foldr_funInput.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    FunToken
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    BasicInst7
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    instLeftCommutativeFunInput
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    instLeftCommutativeFunOutput

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v

variable {a : Type u} {b : Type v} [DecidableEq a] [DecidableEq b]

variable (A : InfoSys a) (B : InfoSys b)

theorem mem_foldr_funInput (t : Multiset (FunToken A B)) (x : a) :
    x  in  Multiset.foldr (fun p : FunToken A B => funion p.val.1) ({} : Finset a) t
    <->
      exists  p  in  t, x  in  p.val.1 := by
  refine Multiset.induction_on t ?_ ?_
  * constructor
    * intro hx
      exact False.elim (Finset.notMem_empty x hx)
    * rintro <_, hp, _>
      exact False.elim (by cases hp)
  * intro p t ih
    simp only [Multiset.foldr_cons, mem_funion, ih, Multiset.mem_cons]
    constructor
    * rintro (hx | <q, hq, hx>)
      * exact <p, Or.inl rfl, hx>
      * exact <q, Or.inr hq, hx>
```

```
    * rintro <q, hq, hx>
      rcases hq with rfl | hq
      * exact Or.inl hx
      * exact Or.inr <q, hq, hx>

end InfoSys

end DomainTheory.InformationSystems
```

mem_foldr_funOutput.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    FunToken
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    BasicInst7
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    instLeftCommutativeFunInput
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    instLeftCommutativeFunOutput

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v

variable {a : Type u} {b : Type v} [DecidableEq a] [DecidableEq b]

variable (A : InfoSys a) (B : InfoSys b)

theorem mem_foldr_funOutput (t : Multiset (FunToken A B)) (y : b) :
    y  in  Multiset.foldr (fun p : FunToken A B => funion p.val.2) ({} : Finset b) t
    <->
      exists  p  in  t, y  in  p.val.2 := by
  refine Multiset.induction_on t ?_ ?_
  * constructor
    * intro hy
      exact False.elim (Finset.notMem_empty y hy)
    * rintro <_, hp, _>
      exact False.elim (by cases hp)
  * intro p t ih
    simp only [Multiset.foldr_cons, mem_funion, ih, Multiset.mem_cons]
    constructor
    * rintro (hy | <q, hq, hy>)
      * exact <p, Or.inl rfl, hy>
      * exact <q, Or.inr hq, hy>
    * rintro <q, hq, hy>
```

```
      rcases hq with rfl | hq
      * exact Or.inl hy
      * exact Or.inr <q, hq, hy>

end InfoSys

end DomainTheory.InformationSystems
```

exists_combined_witness.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    FunToken
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    BasicInst7
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    instLeftCommutativeFunInput
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    instLeftCommutativeFunOutput
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    funInputUnion
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    funOutputUnion
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    FunCon

public import DomainTheory.InformationSystems.InformationSystems.Proposition23.
    proposition_2_3_ii
public import DomainTheory.InformationSystems.InformationSystems.Proposition23.
    proposition_2_3_iv
public import DomainTheory.InformationSystems.InformationSystems.Proposition23.
    proposition_2_3_v
public import DomainTheory.InformationSystems.InformationSystems.Proposition23.
    proposition_2_3_vi
@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u v

variable {a : Type u} {b : Type v} [DecidableEq a] [DecidableEq b]

variable (A : InfoSys a) (B : InfoSys b)

/-- Combine entailment witnesses along a finite set of tokens (for `ent_trans`). -/
theorem exists_combined_witness {u' : Finset a} {s : Finset (FunToken A B)}
    {v : Finset (FunToken A B)}
```

```
  (hv : FunCon A B v) (hu' : u'  in  A.Con)
  (hEntIn : forall  q  in  s, A.EntSet u' q.val.1)
  (hWit : forall  q  in  s, exists  s_q  subseteq  v, (forall  r  in  s_q, A.EntSet
 q.val.1 r.val.1)  /\
    B.EntSet (funOutputUnion A B s_q) q.val.2) :
  exists  S  subseteq  v, (forall  r  in  S, A.EntSet u' r.val.1)  /\
    B.EntSet (funOutputUnion A B S) (funOutputUnion A B s) := by
induction s using Finset.induction_on with
| empty =>
  refine <{}, Finset.empty_subset _, ?_, ?_>
  * intro _ hq
    exact False.elim (Finset.notMem_empty _ hq)
  * intro y hy
    exact False.elim (Finset.notMem_empty y hy)
| insert q s' hqnot ih =>
  have hEntIn' : forall  r  in  s', A.EntSet u' r.val.1 := fun r hr =>
    hEntIn r (Finset.mem_insert_of_mem hr)
  have hWit' : forall  r  in  s', exists  s_r  subseteq  v, (forall  t  in  s_r, A.
 EntSet r.val.1 t.val.1)  /\
      B.EntSet (funOutputUnion A B s_r) r.val.2 := fun r hr =>
    hWit r (Finset.mem_insert_of_mem hr)
  obtain <S', hS'sub, hS'in, hS'out> := ih hEntIn' hWit'
  obtain <Sq, hSqsub, hSqin, hSqout> := hWit q (Finset.mem_insert_self q s')
  refine <S'  U ' Sq, funion_subset_iff.mpr <hS'sub, hSqsub>, ?_, ?_>
  * intro r hr
    rcases mem_funion.mp hr with hr | hr
    * exact hS'in r hr
    * have hEnt_q : A.EntSet u' q.val.1 := hEntIn q (Finset.mem_insert_self q s')
      have hEnt_r : A.EntSet q.val.1 r.val.1 := hSqin r hr
      exact proposition_2_3_iv A hu' q.property.1 hEnt_q hEnt_r
  * rw [funOutputUnion.funion, funOutputUnion.insert]
    have hinS' : funInputUnion A B S'  in  A.Con :=
      funInputUnion.con_of_ent A B hu' hS'in
    have houtS' : funOutputUnion A B S'  in  B.Con := hv S' hS'sub hinS'
    have hinSq : funInputUnion A B Sq  in  A.Con :=
      funInputUnion.con_of_ent A B q.property.1 hSqin
    have houtSq : funOutputUnion A B Sq  in  B.Con := hv Sq hSqsub hinSq
    have hinUnion : funInputUnion A B (S'  U ' Sq)  in  A.Con := by
      rw [funInputUnion.funion]
      have hEnt : A.EntSet u' (funInputUnion A B S'  U ' funInputUnion A B Sq) :=
        proposition_2_3_vi A
          (entSet_inputUnion_of_ent A B hS'in)
          (proposition_2_3_iv A hu' q.property.1
            (hEntIn q (Finset.mem_insert_self q s'))
            (entSet_inputUnion_of_ent A B hSqin))
      exact A.con_subset (proposition_2_3_ii A hu' hEnt) (subset_funion_right _ _)
    have houtUnion : funOutputUnion A B S'  U ' funOutputUnion A B Sq  in  B.Con :=
 by
      have : funOutputUnion A B (S'  U ' Sq)  in  B.Con := hv _ (funion_subset_iff.
 mpr <hS'sub, hSqsub>) hinUnion
      rwa [funOutputUnion.funion] at this
    have hEnt1 : B.EntSet (funOutputUnion A B S'  U ' funOutputUnion A B Sq)
        (funOutputUnion A B s') :=
      proposition_2_3_v B houtS' houtUnion (subset_funion_left _ _) hS'out
        (Finset.Subset.refl _)
    have hEnt2 : B.EntSet (funOutputUnion A B S'  U ' funOutputUnion A B Sq) q.val.2
```

```
    :=
       proposition_2_3_v B houtSq houtUnion (subset_funion_right _ _) hSqout
         (Finset.Subset.refl _)
     exact proposition_2_3_vi B hEnt2 hEnt1

end InfoSys

end DomainTheory.InformationSystems
```

FunctionSpace.lean

```
module

public import DomainTheory.InformationSystems.InformationSystems.Constructive
public import DomainTheory.InformationSystems.InformationSystems.Proposition23
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    FunToken
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    funBot
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    funInputUnion
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    funOutputUnion
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    FunCon
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    FunEnt
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    functionSystem
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    BasicInst7
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    funion_left_comm
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    funion_left_comm'
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    funion_assoc
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    funion_comm
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    funion_assoc'
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    funion_comm'
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    instLeftCommutativeFunInput
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    instLeftCommutativeFunOutput
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    mem_foldr_funInput
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    mem_foldr_funOutput
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    insert_filter_ne
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
```

```
    filter_ne_subset_of_subset_insert
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace.
    exists_combined_witness

@[expose] public section
```

4.8 Some domain equations

Dependencies. Inside SomeDomainEquations, each box is one subfolder, or one Lean file sitting directly in the folder. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. The arrowhead points at the file or folder that imports, and the number is how many direct imports.

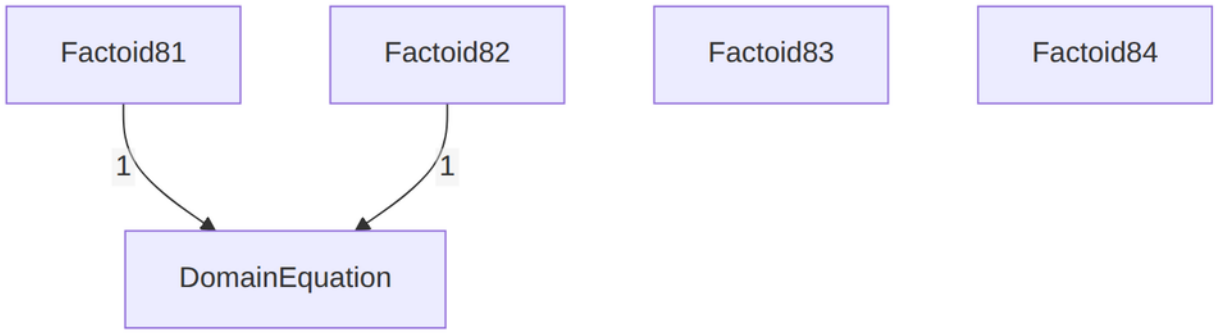


Figure 45: Some domain equations.

Having outlined the theory of several domain constructs, the final topic for this paper will be the discussion of the *iteration* of these constructs in giving *recursive*, rather than direct definitions of domains. These recursively defined systems have often been called "reflexive," because the domains generally contain copies of themselves as a part of their very structure. The way that this self-containment takes place is best expressed by the so-called *domain equations*, which are really isomorphisms that relate the domain as a whole to a combination of domains—usually with the main domain as a component. This description is rough, since recursion equations for domains can be as complex as recursion equations for functions. We will not enter into a full theory of domain equations now but will just review some preliminary examples to illustrate how the new presentation makes the constructions more explicit.

4.8.1 Factoid 8.1

A domain of trees or S-expressions. This is everyone's favorite example. And a very nice example it is, but we should not think that it contains all the meat of the theory of domain equations. Even if we generalize the kinds of equations to contain all iterations of the domain constructs $+$ and $\times$, the full power of the method has not been exploited. We will try to make this clear in the further examples.

Let a domain (information system) $\mathbf{A}$ be given. What we want to construct is a domain $\mathbf{T}$ of "trees" built up from elements of $\mathbf{A}$ as "atoms". For simplicity we consider unlabelled binary trees here, but more complex trees are easy to accommodate. The domain equation we want to "solve" is this one:

$$\mathbf{T} \cong \mathbf{A} + (\mathbf{T} \times \mathbf{T}).$$

If such a domain exists, then we can say that (up to isomorphism) the elements of the domain $\mathbf{T}$ are either bottom, or elements of the given domain $\mathbf{A}$, or pairs of elements from the domain $\mathbf{T}$ itself. And these are the only kinds of elements that $\mathbf{T}$ has.

To prove that such a domain exists it is only necessary to ask what information has to be given about a prospective element. The answer may involve us in a regress, but the running backwards need not be infinite—at least for the finite elements. As we shall see, the infinite elements of $\mathbf{T}$ can be self-replicating; but, to define a domain fully, all we have to do is to build up the finite elements out of the data objects in a systematic way. Fortunately, in order to satisfy the above equation, the required closure conditions on data objects are simple to achieve.

In the first place, we need copies of all the data objects of $\mathbf{A}$ to put into the sum. The easy way to do this is to take an object Δ not in $D_{\mathbf{A}}$, and to let, by definition,

$$\Delta_{\mathbf{T}} = (\Delta, \Delta).$$

That gives us one member of $D_{\mathbf{T}}$, the one we always have to have in any case. The copy of an $X \in D_{\mathbf{A}}$ is just going to be (X, Δ). The other members of $D_{\mathbf{T}}$ will be of the form (Δ, U), where U gives us information about the other kind of elements of $\mathbf{T}$. The point is that $\mathbf{T}$ has to be a sum, and we are using just the scheme of Definition 6.3 to set this up.

Next we have to think what kind of information the U above should contain. Because we want a product, we refer back to Definition 6.1 and imagine we have already defined $D_{\mathbf{T}}$. What 6.1(i) suggests is that we throw in a bunch of other data objects into $D_{\mathbf{T}}$. The only point that needs care is that the data objects for the product must be copied into the overall sum. With this in mind, the following clauses give us the inductive definition of $D_{\mathbf{T}}$:

(1) $\Delta_{\mathbf{T}} \in D_{\mathbf{T}}$;

(2) $(X, \Delta) \in D_{\mathbf{T}}$ whenever $X \in D_{\mathbf{A}}$; and

(3) $(\Delta, (Y, \Delta_{\mathbf{T}})) \in D_{\mathbf{T}}$, and $(\Delta, (\Delta_{\mathbf{T}}, Z)) \in D_{\mathbf{T}}$ whenever $Y, Z \in D_{\mathbf{T}}$.

Of course, when we say "inductive definition," we mean that $D_{\mathbf{T}}$ is the least class satisfying (1)–(3). By standard arguments it can be shown that $D_{\mathbf{T}}$ satisfies this set-theoretical equation:

$$D_{\mathbf{T}} = \{\Delta_{\mathbf{T}}\} \cup \{(X, \Delta) \mid X \in D_{\mathbf{A}}\} \cup \{(\Delta, (Y, \Delta_{\mathbf{T}})) \mid Y \in D_{\mathbf{T}}\} \cup \{(\Delta, (\Delta_{\mathbf{T}}, Z)) \mid Z \in D_{\mathbf{T}}\}.$$

In fact, with some very mild assumptions about ordered pairs in set theory, $D_{\mathbf{T}}$ is the only solution to the above equation.

Defining the data objects is but part of the story: the same data objects can enter into quite different information systems. Data objects are just "tokens" and are only given "meaning" when $\mathrm{Con}_{\mathbf{T}}$ and $\vdash_{\mathbf{T}}$ are defined. Let us consider the problem of consistency first. We already understand the notion as it applies to sum and product systems, so we must merely copy over the parts of the previous definitions in the right position for the definition of $\mathrm{Con}_{\mathbf{T}}$. There are two forms we could give this definition; perhaps the best is the inductive one. We have:

(4) $\emptyset \in \mathrm{Con}_{\mathbf{T}}$;

(5) $u \cup \{\Delta_{\mathbf{T}}\} \in \mathrm{Con}_{\mathbf{T}}$ whenever $u \in \mathrm{Con}_{\mathbf{T}}$;

(6) $\{(X, \Delta) \mid X \in w\} \in \mathrm{Con}_{\mathbf{T}}$ whenever $w \in \mathrm{Con}_{\mathbf{A}}$;

(7) $\{(\Delta, (Y, \Delta_{\mathbf{T}})) \mid Y \in u\} \cup \{(\Delta, (\Delta_{\mathbf{T}}, Z)) \mid Z \in v\} \in \mathrm{Con}_{\mathbf{T}}$ whenever $u, v \in \mathrm{Con}_{\mathbf{T}}$.

Conditions (4)–(7) certainly make the inductive character of $\mathrm{Con}_{\mathbf{T}}$ clear—again, let us emphasize, the set being specified is the least such. Also clear from the definition is the fact that a consistent set of $\mathbf{T}$—aside from containing $\Delta_{\mathbf{T}}$—is either a copy of a consistent set of $\mathbf{A}$ or a copy of a consistent set of $\mathbf{T} \times \mathbf{T}$. We could thus state a set-theoretical equation for $\mathrm{Con}_{\mathbf{T}}$ similar to the one for $D_{\mathbf{T}}$.

It remains to define entailment for $\mathbf{T}$. Here are the inductive clauses which are pretty much forced on us by our objective of solving the domain equation:

(8) $u \vdash_{\mathbf{T}} \Delta_{\mathbf{T}}$ always;

(9) $u \cup \{\Delta_{\mathbf{T}}\} \vdash_{\mathbf{T}} Y$ whenever $u \vdash_{\mathbf{T}} Y$;

(10) $\{(X, \Delta) \mid X \in w\} \vdash_{\mathbf{T}} (W, \Delta)$ whenever $w \vdash_{\mathbf{A}} W$;

(11) $\{(\Delta, (Y, \Delta_{\mathbf{T}})) \mid Y \in u\} \cup \{(\Delta, (\Delta_{\mathbf{T}}, Z)) \mid Z \in v\} \vdash_{\mathbf{T}} (\Delta, (X, \Delta_{\mathbf{T}}))$ whenever $u \vdash_{\mathbf{T}} X$ and $v \in \mathrm{Con}_{\mathbf{T}}$; and

(12) $\{(\Delta, (Y, \Delta_{\mathbf{T}})) \mid Y \in u\} \cup \{(\Delta, (\Delta_{\mathbf{T}}, Z)) \mid Z \in v\} \vdash_{\mathbf{T}} (\Delta, (\Delta_{\mathbf{T}}, X))$ whenever $v \vdash_{\mathbf{T}} X$ and $u \in \mathrm{Con}_{\mathbf{T}}$.

Inductive definitions engender inductive proofs. It now has to be checked that consistency and entailment for $\mathbf{T}$ satisfy the axioms of 2.1. The steps needed for this check are mechanical. (The proof may be aided by noting that the cases in (4)–(7) and in (8)–(12) are disjoint—except for a trivial overlap between (8) and (9). The cases get invoked typically by asking, when confronted with an entailment to prove, for the nature of the data object on the right of the turnstile.)

Having defined and verified that $\mathbf{T}$ is an information system, the validity of the domain equation for $\mathbf{T}$ is secured by forming the right-hand side and noting that $\mathbf{T}$ is *identical* to $\mathbf{A} + (\mathbf{T} \times \mathbf{T})$. The reason is that we carefully chose the notation to match the official definitions of sums and products. (In general, in solving domain equations some transformation might have to take place to "re-format" data objects if things are not set up to be literally the same.)

It should be remarked that the sense can be made precise in which $\mathbf{T}$ is the *least* solution of the given domain equation. (It is an initial algebra in a suitable category of algebras and algebra homomorphisms.) It is pretty obvious that it is minimal in some sense, because we put into it only what was strictly required by the problem and nothing more.

It is also fairly obvious that there are *many* solutions to this domain equation. A non-constructive way to obtain non-minimal solutions is to interpret the whole construction of $\mathbf{T}$ in a non-standard model of set theory. Though, in the definition of $D_{\mathbf{T}}$, it looks like we are only working with very finite objects, everything we did could be made abstract and could be carried out in some funny universe. The result would be a system of "finite" data objects having all the right formal properties but containing things not in the standard minimal system. We would then take the notions of consistency and entailment that also exist in the funny universe and restrict them to sets of data objects that are actually finite in the standard sense. It can be seen from the formal properties of the construction that the resulting notions satisfy our axioms for an information system and that the domain equation holds—BUT the system would have many different elements beyond what we put into the original $\mathbf{T}$. To make this construction work, by the way, we would

have to force $\mathbf{A}$ to be absolute in the model: if it is actually finite (say, $\mathbf{A} = \mathbf{BOOL}$), then there is no problem. (Constructive methods for introducing "nonstandard" data objects can also be given.)

Finally, we must remark on why we called $\mathbf{T}$ a domain of S-expressions. The answer becomes clear when we structure $\mathbf{T}$ as an algebra. First, there is an approximable mapping

$$\mathrm{atom} : \mathbf{A} \to \mathbf{T},$$

which injects $\mathbf{A}$ into $\mathbf{T}$ making the elements of $\mathbf{A}$ "atoms" of $\mathbf{T}$. Then there is a truth-valued predicate on $\mathbf{T}$ which decides whether an element is an atom:

$$\mathrm{isatom} : \mathbf{T} \to \mathbf{BOOL}.$$

Finally, since $\mathbf{T} \times \mathbf{T}$ is a part of $\mathbf{T}$, we can redefine the paring functions so that:

$$\mathrm{pair} : \mathbf{T} \times \mathbf{T} \to \mathbf{T}, \quad \mathrm{fst} : \mathbf{T} \to \mathbf{T}, \quad \text{and } \mathrm{snd} : \mathbf{T} \to \mathbf{T}.$$

In LISP terminology, these operations are the same as the familiar cons, car, and cdr. This makes $\mathbf{T}$ into an algebra where, starting from atoms, elements—expressions—can be built up by iterated pairing.

But why is our system different from the usual way of regarding S-expressions? The answer is that by including partial expressions (those involving $\Delta_{\mathbf{T}}$) and by completing the domain with limits, *infinite* expressions are introduced. For instance, if $a \in |\mathbf{T}|$, then we can solve the fixed-point equation:

$$x = \mathrm{pair}(\mathrm{atom}(a), x),$$

which is an infinite list of a's. This is but one example; the possibilities have been discussed in many papers too numerous to mention here.

As is common to remark, S-expressions can also be thought of as trees: the parse tree that gives the grammatical form of the expression. What we have added to the idea of a tree is the possibility of having infinite trees, and having all these trees as elements of a domain. ■

TreeToken.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

universe u

/-- Tokens of the tree system: sum tagging of atoms and product left/right copies. -/
inductive TreeToken (a : Type u) where
  | bot : TreeToken a
  | atom : a  ->  TreeToken a
  | pairL : TreeToken a  ->  TreeToken a
```

```
  | pairR : TreeToken a  ->  TreeToken a

instance instDecidableEqTreeToken {a : Type u} [DecidableEq a] :
    DecidableEq (TreeToken a)
  | .bot, .bot => isTrue rfl
  | .atom a, .atom b =>
      if h : a = b then isTrue (h  |>  rfl)
      else isFalse fun h' => h (TreeToken.atom.inj h')
  | .pairL a, .pairL b =>
      match instDecidableEqTreeToken a b with
      | isTrue h => isTrue (h  |>  rfl)
      | isFalse h => isFalse fun h' => h (TreeToken.pairL.inj h')
  | .pairR a, .pairR b =>
      match instDecidableEqTreeToken a b with
      | isTrue h => isTrue (h  |>  rfl)
      | isFalse h => isFalse fun h' => h (TreeToken.pairR.inj h')
  | .bot, .atom _ => isFalse fun h => nomatch h
  | .bot, .pairL _ => isFalse fun h => nomatch h
  | .bot, .pairR _ => isFalse fun h => nomatch h
  | .atom _, .bot => isFalse fun h => nomatch h
  | .atom _, .pairL _ => isFalse fun h => nomatch h
  | .atom _, .pairR _ => isFalse fun h => nomatch h
  | .pairL _, .bot => isFalse fun h => nomatch h
  | .pairL _, .atom _ => isFalse fun h => nomatch h
  | .pairL _, .pairR _ => isFalse fun h => nomatch h
  | .pairR _, .bot => isFalse fun h => nomatch h
  | .pairR _, .atom _ => isFalse fun h => nomatch h
  | .pairR _, .pairL _ => isFalse fun h => nomatch h

end InfoSys
end DomainTheory.InformationSystems
```

atomInsert.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

def atomInsert : TreeToken a  ->  Finset a  ->  Finset a
  | .atom x => insert x
  | .bot | .pairL _ | .pairR _ => id
```

```
end InfoSys

end DomainTheory.InformationSystems
```

atomFinset.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomInsert

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]
variable (A : InfoSys a)

instance : LeftCommutative (atomInsert : TreeToken a  ->  Finset a  ->  Finset a) :=
  <fun p q s => by cases p <;> cases q <;> first | exact insert_comm' _ _ s | rfl>

def atomFinset (u : Finset (TreeToken a)) : Finset a :=
  Multiset.foldr atomInsert ({} : Finset a) u.1

end InfoSys
end DomainTheory.InformationSystems
```

pairLFinset.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]
variable (A : InfoSys a)

private def pairLInsert : TreeToken a  ->  Finset (TreeToken a)  ->  Finset (TreeToken
    a)
```

```
  | .pairL t => insert t
  | .bot | .atom _ | .pairR _ => id

instance :
    LeftCommutative (pairLInsert : TreeToken a  ->  Finset (TreeToken a)  ->  Finset (
    TreeToken a)) :=
  <fun p q s => by cases p <;> cases q <;> first | exact insert_comm' _ _ s | rfl>

def pairLFinset (u : Finset (TreeToken a)) : Finset (TreeToken a) :=
  Multiset.foldr pairLInsert ({} : Finset (TreeToken a)) u.1

private theorem mem_foldr_pairL (s : Multiset (TreeToken a)) (t : TreeToken a) :
    t  in  Multiset.foldr pairLInsert ({} : Finset (TreeToken a)) s  <->
      exists  p  in  s, p = .pairL t := by
  refine Multiset.induction_on s ?_ ?_
  * constructor
    * intro ht; exact False.elim (Finset.notMem_empty t ht)
    * rintro <_, hp, _>; exact False.elim (by cases hp)
  * intro p rest ih
    cases p with
    | pairL a =>
      simp only [Multiset.foldr_cons, pairLInsert, Finset.mem_insert, ih, Multiset.
    mem_cons]
      constructor
      * rintro (ht | <q, hq, hq'>)
        * exact <.pairL a, Or.inl rfl, congrArg TreeToken.pairL ht.symm>
        * exact <q, Or.inr hq, hq'>
      * rintro <q, hq, hq'>
        rcases hq with rfl | hq
        * injection hq' with ht; exact Or.inl ht.symm
        * exact Or.inr <q, hq, hq'>
    | bot | atom _ | pairR _ =>
      simp only [Multiset.foldr_cons, pairLInsert, id_eq, ih, Multiset.mem_cons]
      constructor
      * rintro <q, hq, hq'>; exact <q, Or.inr hq, hq'>
      * rintro <q, hq, hq'>
        rcases hq with rfl | hq
        * exact False.elim (nomatch hq')
        * exact <q, hq, hq'>

theorem mem_pairLFinset {u : Finset (TreeToken a)} {t : TreeToken a} :
    t  in  pairLFinset u  <->  .pairL t  in  u := by
  constructor
  * intro ht
    rcases (mem_foldr_pairL u.1 t).1 ht with <p, hp, hp'>
    subst hp'; exact Finset.mem_def.2 hp
  * intro ht
    exact (mem_foldr_pairL u.1 t).2 <.pairL t, Finset.mem_def.1 ht, rfl>

end InfoSys
end DomainTheory.InformationSystems
```

mem_pairLFinset.lean

```
module
```

```
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
```

pairRFinset.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]
variable (A : InfoSys a)

private def pairRInsert : TreeToken a  ->  Finset (TreeToken a)  ->  Finset (TreeToken
     a)
  | .pairR t => insert t
  | .bot | .atom _ | .pairL _ => id

instance :
    LeftCommutative (pairRInsert : TreeToken a  ->  Finset (TreeToken a)  ->  Finset (
    TreeToken a)) :=
  <fun p q s => by cases p <;> cases q <;> first | exact insert_comm' _ _ s | rfl>

def pairRFinset (u : Finset (TreeToken a)) : Finset (TreeToken a) :=
  Multiset.foldr pairRInsert ({} : Finset (TreeToken a)) u.1

private theorem mem_foldr_pairR (s : Multiset (TreeToken a)) (t : TreeToken a) :
    t  in  Multiset.foldr pairRInsert ({} : Finset (TreeToken a)) s  <->
      exists  p  in  s, p = .pairR t := by
  refine Multiset.induction_on s ?_ ?_
  * constructor
    * intro ht; exact False.elim (Finset.notMem_empty t ht)
    * rintro <_, hp, _>; exact False.elim (by cases hp)
  * intro p rest ih
    cases p with
    | pairR a =>
      simp only [Multiset.foldr_cons, pairRInsert, Finset.mem_insert, ih, Multiset.
    mem_cons]
      constructor
      * rintro (ht | <q, hq, hq'>)
        * exact <.pairR a, Or.inl rfl, congrArg TreeToken.pairR ht.symm>
        * exact <q, Or.inr hq, hq'>
      * rintro <q, hq, hq'>
        rcases hq with rfl | hq
        * injection hq' with ht; exact Or.inl ht.symm
```

```
        * exact Or.inr <q, hq, hq'>
    | bot | atom _ | pairL _ =>
      simp only [Multiset.foldr_cons, pairRInsert, id_eq, ih, Multiset.mem_cons]
      constructor
      * rintro <q, hq, hq'>; exact <q, Or.inr hq, hq'>
      * rintro <q, hq, hq'>
        rcases hq with rfl | hq
        * exact False.elim (nomatch hq')
        * exact <q, hq, hq'>

theorem mem_pairRFinset {u : Finset (TreeToken a)} {t : TreeToken a} :
    t  in  pairRFinset u  <->  .pairR t  in  u := by
  constructor
  * intro ht
    rcases (mem_foldr_pairR u.1 t).1 ht with <p, hp, hp'>
    subst hp'; exact Finset.mem_def.2 hp
  * intro ht
    exact (mem_foldr_pairR u.1 t).2 <.pairR t, Finset.mem_def.1 ht, rfl>

end InfoSys
end DomainTheory.InformationSystems
```

TreeCon.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]
variable (A : InfoSys a)

/-- Consistency for trees (Scott S8 (4)-(7)). -/
inductive TreeCon : Finset (TreeToken a)  ->  Prop where
  | left (u : Finset (TreeToken a))
      (hA : atomFinset u  in  A.Con)
      (hL : pairLFinset u = {}) (hR : pairRFinset u = {}) : TreeCon u
  | right (u : Finset (TreeToken a))
      (hA : atomFinset u = {})
      (hL : TreeCon (pairLFinset u)) (hR : TreeCon (pairRFinset u)) : TreeCon u
```

```
theorem TreeCon.atom_con_of_atoms {u : Finset (TreeToken a)}
    (hu : TreeCon A u) (hne : atomFinset u  !=  {}) : atomFinset u  in  A.Con := by
  cases hu with
  | left _ hA _ _ => exact hA
  | right _ hA _ _ => exact False.elim (hne hA)

theorem TreeCon.atoms_empty_of_right {u : Finset (TreeToken a)}
    (hu : TreeCon A u) (hP : pairLFinset u  !=  {}  \/  pairRFinset u  !=  {}) :
    atomFinset u = {} := by
  cases hu with
  | left _ _ hL hR =>
    rcases hP with h | h
    * exact False.elim (h hL)
    * exact False.elim (h hR)
  | right _ hA _ _ => exact hA

theorem TreeCon.pairs_empty_of_left {u : Finset (TreeToken a)}
    (hu : TreeCon A u) (hA : atomFinset u  !=  {}) :
    pairLFinset u = {}  /\  pairRFinset u = {} := by
  cases hu with
  | left _ _ hL hR => exact <hL, hR>
  | right _ hAt _ _ => exact False.elim (hA hAt)

end InfoSys

end DomainTheory.InformationSystems
```

TreeCon_pairL_of_right.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    TreeCon_empty

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)
```

```
theorem TreeCon.pairL_of_right {u : Finset (TreeToken a)}
    (hu : TreeCon A u) (_hA : atomFinset u = {}) : TreeCon A (pairLFinset u) := by
  cases hu with
  | left _ _ hL _ => rw [hL]; exact TreeCon_empty A
  | right _ _ hL _ => exact hL

end InfoSys

end DomainTheory.InformationSystems
```

TreeCon_pairR_of_right.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    TreeCon_empty

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem TreeCon.pairR_of_right {u : Finset (TreeToken a)}
    (hu : TreeCon A u) (_hA : atomFinset u = {}) : TreeCon A (pairRFinset u) := by
  cases hu with
  | left _ _ _ hR => rw [hR]; exact TreeCon_empty A
  | right _ _ _ hR => exact hR

end InfoSys

end DomainTheory.InformationSystems
```

TreeEntPayload.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
```

```
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    mem_atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    mem_pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    mem_pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    TreeCon_pairL_of_right
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    TreeCon_pairR_of_right

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]
variable (A : InfoSys a)

/-- Payload of tree entailment (Scott S8 (8)-(12)), recursive on the token.

The extra disjuncts for `pairL bot` / `pairR bot` match official product
`ent_bot`: any nonempty pair-set entails the unique product bottom, which
`treeUnfold` identifies as both constructors. Without them, closed tree
elements could contain one encoding of `(Delta,Delta)` but not the other, and
`|T|` would fail to be order-isomorphic to `|A + (T  x  T)|`. -/
def TreeEntPayload (u : Finset (TreeToken a)) : TreeToken a  ->  Prop
  | .bot => True
  | .atom x => atomFinset u  !=  {}  /\  A.Ent (atomFinset u) x
  | .pairL t =>
      atomFinset u = {}  /\  TreeCon A (pairLFinset u)  /\  TreeCon A (pairRFinset u)
     /\
        ((pairLFinset u  !=  {}  /\  TreeEntPayload (pairLFinset u) t)  \/
          (t = .bot  /\  pairRFinset u  !=  {}))
  | .pairR t =>
      atomFinset u = {}  /\  TreeCon A (pairRFinset u)  /\  TreeCon A (pairLFinset u)
     /\
        ((pairRFinset u  !=  {}  /\  TreeEntPayload (pairRFinset u) t)  \/
          (t = .bot  /\  pairLFinset u  !=  {}))

theorem TreeEntPayload.of_mem {u : Finset (TreeToken a)} {p : TreeToken a}
    (hu : TreeCon A u) (hp : p  in  u) : TreeEntPayload A u p := by
  induction p generalizing u with
  | bot => exact trivial
  | atom x =>
    have hx : x  in  atomFinset u := mem_atomFinset.2 hp
    exact <Finset.ne_empty_of_mem hx,
```

```
      A.ent_refl (TreeCon.atom_con_of_atoms A hu (Finset.ne_empty_of_mem hx)) hx>
  | pairL t ih =>
    have ht : t  in  pairLFinset u := mem_pairLFinset.2 hp
    have hne := Finset.ne_empty_of_mem ht
    have hAt := TreeCon.atoms_empty_of_right A hu (Or.inl hne)
    exact <hAt, TreeCon.pairL_of_right A hu hAt, TreeCon.pairR_of_right A hu hAt,
      Or.inl <hne, ih (TreeCon.pairL_of_right A hu hAt) ht>>
  | pairR t ih =>
    have ht : t  in  pairRFinset u := mem_pairRFinset.2 hp
    have hne := Finset.ne_empty_of_mem ht
    have hAt := TreeCon.atoms_empty_of_right A hu (Or.inr hne)
    exact <hAt, TreeCon.pairR_of_right A hu hAt, TreeCon.pairL_of_right A hu hAt,
      Or.inl <hne, ih (TreeCon.pairR_of_right A hu hAt) ht>>

theorem TreeEntPayload.of_no_atoms_no_pairs {u : Finset (TreeToken a)} {t : TreeToken
    a}
    (hAt : atomFinset u = {}) (hL : pairLFinset u = {}) (hR : pairRFinset u = {})
    (h : TreeEntPayload A u t) : t = .bot := by
  cases t with
  | bot => rfl
  | atom x => exact False.elim (h.1 hAt)
  | pairL t =>
    rcases h with <_, _, _, hOr>
    rcases hOr with <hne, _> | <_, hRne>
    * exact False.elim (hne hL)
    * exact False.elim (hRne hR)
  | pairR t =>
    rcases h with <_, _, _, hOr>
    rcases hOr with <hne, _> | <_, hLne>
    * exact False.elim (hne hR)
    * exact False.elim (hLne hL)

end InfoSys

end DomainTheory.InformationSystems
```

TreeEnt.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    TreeEntPayload

@[expose] public section

namespace DomainTheory.InformationSystems
```

```
open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]
variable (A : InfoSys a)

/-- Entailment for trees. -/
def TreeEnt (u : Finset (TreeToken a)) (p : TreeToken a) : Prop :=
  TreeCon A u  /\  TreeEntPayload A u p

theorem TreeEnt.of_mem {u : Finset (TreeToken a)} {p : TreeToken a}
    (hu : TreeCon A u) (hp : p  in  u) : TreeEnt A u p :=
  <hu, TreeEntPayload.of_mem A hu hp>

end InfoSys

end DomainTheory.InformationSystems
```

atomFinset_empty.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

theorem atomFinset_empty : atomFinset ({} : Finset (TreeToken a)) = {} := rfl

end InfoSys

end DomainTheory.InformationSystems
```

eq_singleton_bot_of_subset.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
```

```
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

omit [DecidableEq a] in

theorem eq_singleton_bot_of_subset {s : Finset (TreeToken a)}
    (hne : s  !=  {}) (hsub : s  subseteq  {.bot}) : s = {.bot} := by
  ext y
  constructor
  * exact fun hy => hsub hy
  * intro hy
    have : y = .bot := Finset.mem_singleton.1 hy
    subst this
    obtain <z, hz> := Finset.nonempty_of_ne_empty hne
    have hzbot : z = .bot := Finset.mem_singleton.1 (hsub hz)
    exact hzbot  |>  hz

end InfoSys

end DomainTheory.InformationSystems
```

mem_foldr_atom.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset

@[expose] public section
```

```
namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

theorem mem_foldr_atom (s : Multiset (TreeToken a)) (x : a) :
    x  in  Multiset.foldr atomInsert ({} : Finset a) s  <->  exists  p  in  s, p = .
    atom x := by
  refine Multiset.induction_on s ?_ ?_
  * constructor
    * intro hx; exact False.elim (Finset.notMem_empty x hx)
    * rintro <_, hp, _>; exact False.elim (by cases hp)
  * intro p t ih
    cases p with
    | atom a =>
      simp only [Multiset.foldr_cons, atomInsert, Finset.mem_insert, ih, Multiset.
    mem_cons]
      constructor
      * rintro (hx | <q, hq, hq'>)
        * exact <.atom a, Or.inl rfl, congrArg TreeToken.atom hx.symm>
        * exact <q, Or.inr hq, hq'>
      * rintro <q, hq, hq'>
        rcases hq with rfl | hq
        * injection hq' with hx; exact Or.inl hx.symm
        * exact Or.inr <q, hq, hq'>
    | bot | pairL _ | pairR _ =>
      simp only [Multiset.foldr_cons, atomInsert, id_eq, ih, Multiset.mem_cons]
      constructor
      * rintro <q, hq, hq'>; exact <q, Or.inr hq, hq'>
      * rintro <q, hq, hq'>
        rcases hq with rfl | hq
        * exact False.elim (nomatch hq')
        * exact <q, hq, hq'>

end InfoSys

end DomainTheory.InformationSystems
```

mem_atomFinset.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
```

```
    mem_foldr_atom

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

theorem mem_atomFinset {u : Finset (TreeToken a)} {x : a} :
    x  in  atomFinset u  <->  .atom x  in  u := by
  constructor
  * intro hx
    rcases (mem_foldr_atom u.1 x).1 hx with <p, hp, hp'>
    subst hp'; exact Finset.mem_def.2 hp
  * intro hx
    exact (mem_foldr_atom u.1 x).2 <.atom x, Finset.mem_def.1 hx, rfl>

end InfoSys

end DomainTheory.InformationSystems
```

atomFinset_insert_atom.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    mem_atomFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

theorem atomFinset_insert_atom (u : Finset (TreeToken a)) (x : a) :
    atomFinset (insert (.atom x) u) = insert x (atomFinset u) := by
  ext y; simp only [Finset.mem_insert, mem_atomFinset]
  constructor
```

```
  * rintro (h | h)
    * exact Or.inl (TreeToken.atom.inj h)
    * exact Or.inr h
  * rintro (rfl | h)
    * exact Or.inl rfl
    * exact Or.inr h

end InfoSys

end DomainTheory.InformationSystems
```

atomFinset_insert_bot.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    mem_atomFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

theorem atomFinset_insert_bot (u : Finset (TreeToken a)) :
    atomFinset (insert (.bot : TreeToken a) u) = atomFinset u := by
  ext x; simp only [mem_atomFinset, Finset.mem_insert]
  constructor
  * rintro (h | h); exacts [False.elim (nomatch h), h]
  * exact Or.inr

end InfoSys

end DomainTheory.InformationSystems
```

atomFinset_insert_pairL.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
```

```
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    mem_atomFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

theorem atomFinset_insert_pairL (u : Finset (TreeToken a)) (t : TreeToken a) :
    atomFinset (insert (.pairL t) u) = atomFinset u := by
  ext x; simp only [mem_atomFinset, Finset.mem_insert]
  constructor
  * rintro (h | h); exacts [False.elim (nomatch h), h]
  * exact Or.inr

end InfoSys

end DomainTheory.InformationSystems
```

atomFinset_insert_pairR.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    mem_atomFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]
```

```
theorem atomFinset_insert_pairR (u : Finset (TreeToken a)) (t : TreeToken a) :
    atomFinset (insert (.pairR t) u) = atomFinset u := by
  ext x; simp only [mem_atomFinset, Finset.mem_insert]
  constructor
  * rintro (h | h); exacts [False.elim (nomatch h), h]
  * exact Or.inr

end InfoSys

end DomainTheory.InformationSystems
```

atomFinset_mono.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    mem_atomFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

theorem atomFinset_mono {u v : Finset (TreeToken a)} (h : v  subseteq  u) :
    atomFinset v  subseteq  atomFinset u := fun _ hx => mem_atomFinset.2 (h (
    mem_atomFinset.1 hx))

end InfoSys

end DomainTheory.InformationSystems
```

atomFinset_singleton_atom.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
```

```
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    mem_atomFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

theorem atomFinset_singleton_atom (x : a) :
    atomFinset ({.atom x} : Finset (TreeToken a)) = {x} := by
  ext y; simp only [mem_atomFinset, Finset.mem_singleton]
  exact <fun h => TreeToken.atom.inj h, fun h => congrArg TreeToken.atom h>

end InfoSys

end DomainTheory.InformationSystems
```

atomFinset_singleton_bot.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    mem_atomFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

theorem atomFinset_singleton_bot :
    atomFinset ({.bot} : Finset (TreeToken a)) = {} := by
  ext x
```

```
  constructor
  * intro hx
    exact False.elim (nomatch mem_atomFinset.1 hx)
  * intro hx
    exact False.elim (Finset.notMem_empty x hx)

end InfoSys

end DomainTheory.InformationSystems
```

atomFinset_singleton_pairL.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    mem_atomFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

theorem atomFinset_singleton_pairL (t : TreeToken a) :
    atomFinset ({.pairL t} : Finset (TreeToken a)) = {} := by
  ext x
  constructor
  * intro hx
    exact False.elim (nomatch mem_atomFinset.1 hx)
  * intro hx
    exact False.elim (Finset.notMem_empty x hx)

end InfoSys

end DomainTheory.InformationSystems
```

atomFinset_singleton_pairR.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
```

```
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    mem_atomFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

theorem atomFinset_singleton_pairR (t : TreeToken a) :
    atomFinset ({.pairR t} : Finset (TreeToken a)) = {} := by
  ext x
  constructor
  * intro hx
    exact False.elim (nomatch mem_atomFinset.1 hx)
  * intro hx
    exact False.elim (Finset.notMem_empty x hx)

end InfoSys

end DomainTheory.InformationSystems
```

mem_pairRFinset.lean

```
module

public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset
```

pairLFinset_empty.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset

@[expose] public section
```

```
namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

theorem pairLFinset_empty : pairLFinset ({} : Finset (TreeToken a)) = {} := rfl

end InfoSys

end DomainTheory.InformationSystems
```

pairLFinset_insert_atom.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    mem_pairLFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

theorem pairLFinset_insert_atom (u : Finset (TreeToken a)) (x : a) :
    pairLFinset (insert (.atom x) u) = pairLFinset u := by
  ext t; simp only [mem_pairLFinset, Finset.mem_insert]
  constructor
  * rintro (h | h); exacts [False.elim (nomatch h), h]
  * exact Or.inr

end InfoSys

end DomainTheory.InformationSystems
```

pairLFinset_insert_bot.lean

```
module
```

```
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    mem_pairLFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

theorem pairLFinset_insert_bot (u : Finset (TreeToken a)) :
    pairLFinset (insert (.bot : TreeToken a) u) = pairLFinset u := by
  ext t; simp only [mem_pairLFinset, Finset.mem_insert]
  constructor
  * rintro (h | h); exacts [False.elim (nomatch h), h]
  * exact Or.inr

end InfoSys

end DomainTheory.InformationSystems
```

pairLFinset_insert_pairL.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    mem_pairLFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive
```

```
namespace InfoSys

variable {a : Type*} [DecidableEq a]

theorem pairLFinset_insert_pairL (u : Finset (TreeToken a)) (t : TreeToken a) :
    pairLFinset (insert (.pairL t) u) = insert t (pairLFinset u) := by
  ext s; simp only [Finset.mem_insert, mem_pairLFinset]
  constructor
  * rintro (h | h)
    * exact Or.inl (TreeToken.pairL.inj h)
    * exact Or.inr h
  * rintro (rfl | h)
    * exact Or.inl rfl
    * exact Or.inr h

end InfoSys

end DomainTheory.InformationSystems
```

pairLFinset_insert_pairR.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    mem_pairLFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

theorem pairLFinset_insert_pairR (u : Finset (TreeToken a)) (t : TreeToken a) :
    pairLFinset (insert (.pairR t) u) = pairLFinset u := by
  ext s; simp only [mem_pairLFinset, Finset.mem_insert]
  constructor
  * rintro (h | h); exacts [False.elim (nomatch h), h]
  * exact Or.inr

end InfoSys
```

```
end DomainTheory.InformationSystems
```

pairLFinset_mono.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    mem_pairLFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

theorem pairLFinset_mono {u v : Finset (TreeToken a)} (h : v  subseteq  u) :
    pairLFinset v  subseteq  pairLFinset u := fun _ ht => mem_pairLFinset.2 (h (
    mem_pairLFinset.1 ht))

end InfoSys

end DomainTheory.InformationSystems
```

pairLFinset_singleton_atom.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    mem_pairLFinset

@[expose] public section

namespace DomainTheory.InformationSystems
```

```
open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

theorem pairLFinset_singleton_atom (x : a) :
    pairLFinset ({.atom x} : Finset (TreeToken a)) = {} := by
  ext t
  constructor
  * intro ht
    exact False.elim (nomatch mem_pairLFinset.1 ht)
  * intro ht
    exact False.elim (Finset.notMem_empty t ht)

end InfoSys

end DomainTheory.InformationSystems
```

pairLFinset_singleton_bot.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    mem_pairLFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

theorem pairLFinset_singleton_bot :
    pairLFinset ({.bot} : Finset (TreeToken a)) = {} := by
  ext t
  constructor
  * intro ht
    exact False.elim (nomatch mem_pairLFinset.1 ht)
  * intro ht
    exact False.elim (Finset.notMem_empty t ht)
```

```
end InfoSys

end DomainTheory.InformationSystems
```

pairLFinset_singleton_pairL.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    mem_pairLFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

theorem pairLFinset_singleton_pairL (t : TreeToken a) :
    pairLFinset ({.pairL t} : Finset (TreeToken a)) = {t} := by
  ext s; simp only [mem_pairLFinset, Finset.mem_singleton]
  exact <fun h => TreeToken.pairL.inj h, fun h => congrArg TreeToken.pairL h>

end InfoSys

end DomainTheory.InformationSystems
```

pairLFinset_singleton_pairR.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    mem_pairLFinset
```

```
@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

theorem pairLFinset_singleton_pairR (t : TreeToken a) :
    pairLFinset ({.pairR t} : Finset (TreeToken a)) = {} := by
  ext s
  constructor
  * intro hs
    exact False.elim (nomatch mem_pairLFinset.1 hs)
  * intro hs
    exact False.elim (Finset.notMem_empty s hs)

end InfoSys

end DomainTheory.InformationSystems
```

pairL_ne_or_subset_bot.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeEnt

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem pairL_ne_or_subset_bot {u v : Finset (TreeToken a)}
    (hEnts : forall  y  in  pairLFinset u, TreeEnt A v (.pairL y)) :
    pairLFinset v  !=  {}  \/  pairLFinset u  subseteq  {.bot} := by
  revert hEnts
  induction pairLFinset u using Finset.induction with
```

```
  | empty =>
    intro; exact Or.inr (Finset.empty_subset _)
  | insert y s hys ih =>
    intro hEnts
    have hy := hEnts y (Finset.mem_insert_self _ _)
    rcases hy.2.2.2.2 with <hneV, _> | <rfl, _>
    * exact Or.inl hneV
    * rcases ih (fun z hz => hEnts z (Finset.mem_insert_of_mem hz)) with h | hsub
      * exact Or.inl h
      * refine Or.inr ?_
        intro z hz
        rcases Finset.mem_insert.1 hz with rfl | hz
        * exact Finset.mem_singleton_self _
        * exact hsub hz

end InfoSys

end DomainTheory.InformationSystems
```

pairRFinset_empty.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

theorem pairRFinset_empty : pairRFinset ({} : Finset (TreeToken a)) = {} := rfl

end InfoSys

end DomainTheory.InformationSystems
```

TreeCon_empty.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
```

```
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    atomFinset_empty
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset_empty
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset_empty

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem TreeCon_empty : TreeCon A ({} : Finset (TreeToken a)) :=
  TreeCon.left _ (by rw [atomFinset_empty]; exact A.con_empty)
    pairLFinset_empty pairRFinset_empty

end InfoSys

end DomainTheory.InformationSystems
```

pairRFinset_insert_atom.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    mem_pairRFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive
```

```
namespace InfoSys

variable {a : Type*} [DecidableEq a]

theorem pairRFinset_insert_atom (u : Finset (TreeToken a)) (x : a) :
    pairRFinset (insert (.atom x) u) = pairRFinset u := by
  ext t; simp only [mem_pairRFinset, Finset.mem_insert]
  constructor
  * rintro (h | h); exacts [False.elim (nomatch h), h]
  * exact Or.inr

end InfoSys

end DomainTheory.InformationSystems
```

pairRFinset_insert_bot.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    mem_pairRFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

theorem pairRFinset_insert_bot (u : Finset (TreeToken a)) :
    pairRFinset (insert (.bot : TreeToken a) u) = pairRFinset u := by
  ext t; simp only [mem_pairRFinset, Finset.mem_insert]
  constructor
  * rintro (h | h); exacts [False.elim (nomatch h), h]
  * exact Or.inr

end InfoSys

end DomainTheory.InformationSystems
```

TreeCon_insert_bot.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    atomFinset_insert_bot
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset_insert_bot
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset_insert_bot

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem TreeCon_insert_bot {u : Finset (TreeToken a)} (hu : TreeCon A u) :
    TreeCon A (insert .bot u) := by
  cases hu with
  | left _ hA hL hR =>
    exact TreeCon.left _
      (by rw [atomFinset_insert_bot]; exact hA)
      (by rw [pairLFinset_insert_bot]; exact hL)
      (by rw [pairRFinset_insert_bot]; exact hR)
  | right _ hA hL hR =>
    exact TreeCon.right _
      (by rw [atomFinset_insert_bot]; exact hA)
      (by rw [pairLFinset_insert_bot]; exact hL)
      (by rw [pairRFinset_insert_bot]; exact hR)

end InfoSys

end DomainTheory.InformationSystems
```

pairRFinset_insert_pairL.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
```

```
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    mem_pairRFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

theorem pairRFinset_insert_pairL (u : Finset (TreeToken a)) (t : TreeToken a) :
    pairRFinset (insert (.pairL t) u) = pairRFinset u := by
  ext s; simp only [mem_pairRFinset, Finset.mem_insert]
  constructor
  * rintro (h | h); exacts [False.elim (nomatch h), h]
  * exact Or.inr

end InfoSys

end DomainTheory.InformationSystems
```

pairRFinset_insert_pairR.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    mem_pairRFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]
```

```
theorem pairRFinset_insert_pairR (u : Finset (TreeToken a)) (t : TreeToken a) :
    pairRFinset (insert (.pairR t) u) = insert t (pairRFinset u) := by
  ext s; simp only [Finset.mem_insert, mem_pairRFinset]
  constructor
  * rintro (h | h)
    * exact Or.inl (TreeToken.pairR.inj h)
    * exact Or.inr h
  * rintro (rfl | h)
    * exact Or.inl rfl
    * exact Or.inr h

end InfoSys

end DomainTheory.InformationSystems
```

TreeCon_insert_of_ent.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    atomFinset_insert_atom
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    atomFinset_insert_pairL
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    atomFinset_insert_pairR
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset_insert_pairL
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset_insert_atom
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset_insert_pairR
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset_insert_pairR
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset_insert_atom
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset_insert_pairL
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    TreeCon_insert_bot

@[expose] public section

namespace DomainTheory.InformationSystems
```

```
open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem TreeCon_insert_of_ent {u : Finset (TreeToken a)} {p : TreeToken a}
    (h : TreeEnt A u p) : TreeCon A (insert p u) := by
  induction p generalizing u with
  | bot =>
    exact TreeCon_insert_bot A h.1
  | atom x =>
    rcases h with <hu, hne, hA>
    have hLR := TreeCon.pairs_empty_of_left A hu hne
    exact TreeCon.left _
      (by rw [atomFinset_insert_atom]; exact A.ent_con hA)
      (by rw [pairLFinset_insert_atom]; exact hLR.1)
      (by rw [pairRFinset_insert_atom]; exact hLR.2)
  | pairL t ih =>
    rcases h with <_, hAt, hConL, hConR, hOr>
    refine TreeCon.right _
      (by rw [atomFinset_insert_pairL]; exact hAt) ?_
      (by rw [pairRFinset_insert_pairL]; exact hConR)
    rw [pairLFinset_insert_pairL]
    rcases hOr with <_, hPay> | <rfl, _>
    * exact ih <hConL, hPay>
    * exact TreeCon_insert_bot A hConL
  | pairR t ih =>
    rcases h with <_, hAt, hConR, hConL, hOr>
    refine TreeCon.right _
      (by rw [atomFinset_insert_pairR]; exact hAt)
      (by rw [pairLFinset_insert_pairR]; exact hConL) ?_
    rw [pairRFinset_insert_pairR]
    rcases hOr with <_, hPay> | <rfl, _>
    * exact ih <hConR, hPay>
    * exact TreeCon_insert_bot A hConR

end InfoSys

end DomainTheory.InformationSystems
```

pairRFinset_mono.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
```

```
    pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    mem_pairRFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

theorem pairRFinset_mono {u v : Finset (TreeToken a)} (h : v  subseteq  u) :
    pairRFinset v  subseteq  pairRFinset u := fun _ ht => mem_pairRFinset.2 (h (
    mem_pairRFinset.1 ht))

end InfoSys

end DomainTheory.InformationSystems
```

TreeCon_subset.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    atomFinset_mono
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset_mono
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset_mono

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem TreeCon_subset {u : Finset (TreeToken a)} (hu : TreeCon A u) :
```

```
    forall  {v}, v  subseteq  u  ->  TreeCon A v := by
  induction hu with
  | left u hA hL hR =>
    intro v hv
    refine TreeCon.left v (A.con_subset hA (atomFinset_mono hv)) ?_ ?_
    * exact Finset.Subset.antisymm
        (fun t ht => by
          have : t  in  pairLFinset u := pairLFinset_mono hv ht
          rw [hL] at this; exact False.elim (Finset.notMem_empty t this))
        (Finset.empty_subset _)
    * exact Finset.Subset.antisymm
        (fun t ht => by
          have : t  in  pairRFinset u := pairRFinset_mono hv ht
          rw [hR] at this; exact False.elim (Finset.notMem_empty t this))
        (Finset.empty_subset _)
  | right u hA hL hR ihL ihR =>
    intro v hv
    refine TreeCon.right v
      (Finset.Subset.antisymm
        (fun x hx => by
          have : x  in  atomFinset u := atomFinset_mono hv hx
          rw [hA] at this; exact False.elim (Finset.notMem_empty x this))
        (Finset.empty_subset _))
      (ihL (pairLFinset_mono hv)) (ihR (pairRFinset_mono hv))

end InfoSys

end DomainTheory.InformationSystems
```

pairRFinset_singleton_atom.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    mem_pairRFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]
```

```
theorem pairRFinset_singleton_atom (x : a) :
    pairRFinset ({.atom x} : Finset (TreeToken a)) = {} := by
  ext t
  constructor
  * intro ht
    exact False.elim (nomatch mem_pairRFinset.1 ht)
  * intro ht
    exact False.elim (Finset.notMem_empty t ht)

end InfoSys

end DomainTheory.InformationSystems
```

pairRFinset_singleton_bot.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    mem_pairRFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

theorem pairRFinset_singleton_bot :
    pairRFinset ({.bot} : Finset (TreeToken a)) = {} := by
  ext t
  constructor
  * intro ht
    exact False.elim (nomatch mem_pairRFinset.1 ht)
  * intro ht
    exact False.elim (Finset.notMem_empty t ht)

end InfoSys

end DomainTheory.InformationSystems
```

pairRFinset_singleton_pairL.lean

```
module
```

```
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    mem_pairRFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

theorem pairRFinset_singleton_pairL (t : TreeToken a) :
    pairRFinset ({.pairL t} : Finset (TreeToken a)) = {} := by
  ext s
  constructor
  * intro hs
    exact False.elim (nomatch mem_pairRFinset.1 hs)
  * intro hs
    exact False.elim (Finset.notMem_empty s hs)

end InfoSys

end DomainTheory.InformationSystems
```

pairRFinset_singleton_pairR.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    mem_pairRFinset

@[expose] public section

namespace DomainTheory.InformationSystems
```

```
open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

theorem pairRFinset_singleton_pairR (t : TreeToken a) :
    pairRFinset ({.pairR t} : Finset (TreeToken a)) = {t} := by
  ext s; simp only [mem_pairRFinset, Finset.mem_singleton]
  exact <fun h => TreeToken.pairR.inj h, fun h => congrArg TreeToken.pairR h>

end InfoSys

end DomainTheory.InformationSystems
```

TreeCon_singleton.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    atomFinset_singleton_atom
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset_singleton_pairL
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset_singleton_pairR
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    atomFinset_singleton_bot
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset_singleton_bot
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset_singleton_bot
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset_singleton_atom
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset_singleton_atom
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    atomFinset_singleton_pairL
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    atomFinset_singleton_pairR
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset_singleton_pairL
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset_singleton_pairR
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    TreeCon_empty
```

```
@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem TreeCon_singleton (p : TreeToken a) : TreeCon A {p} := by
  induction p with
  | bot =>
    exact TreeCon.left _
      (by rw [atomFinset_singleton_bot]; exact A.con_empty)
      pairLFinset_singleton_bot pairRFinset_singleton_bot
  | atom x =>
    exact TreeCon.left _
      (by rw [atomFinset_singleton_atom]; exact A.con_sing x)
      (pairLFinset_singleton_atom x) (pairRFinset_singleton_atom x)
  | pairL t ih =>
    exact TreeCon.right _
      (atomFinset_singleton_pairL t)
      (by rw [pairLFinset_singleton_pairL]; exact ih)
      (by rw [pairRFinset_singleton_pairL]; exact TreeCon_empty A)
  | pairR t ih =>
    exact TreeCon.right _
      (atomFinset_singleton_pairR t)
      (by rw [pairLFinset_singleton_pairR]; exact TreeCon_empty A)
      (by rw [pairRFinset_singleton_pairR]; exact ih)

end InfoSys

end DomainTheory.InformationSystems
```

pairR_ne_or_subset_bot.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeEnt

@[expose] public section
```

```
namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem pairR_ne_or_subset_bot {u v : Finset (TreeToken a)}
    (hEnts : forall  y  in  pairRFinset u, TreeEnt A v (.pairR y)) :
    pairRFinset v  !=  {}  \/  pairRFinset u  subseteq  {.bot} := by
  revert hEnts
  induction pairRFinset u using Finset.induction with
  | empty =>
    intro; exact Or.inr (Finset.empty_subset _)
  | insert y s hys ih =>
    intro hEnts
    have hy := hEnts y (Finset.mem_insert_self _ _)
    rcases hy.2.2.2.2 with <hneV, _> | <rfl, _>
    * exact Or.inl hneV
    * rcases ih (fun z hz => hEnts z (Finset.mem_insert_of_mem hz)) with h | hsub
      * exact Or.inl h
      * refine Or.inr ?_
        intro z hz
        rcases Finset.mem_insert.1 hz with rfl | hz
        * exact Finset.mem_singleton_self _
        * exact hsub hz

end InfoSys

end DomainTheory.InformationSystems
```

TreeEntPayload_trans.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    mem_atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    mem_pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    mem_pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    atomFinset_singleton_bot
```

```
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset_singleton_bot
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset_singleton_bot
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    TreeEntPayload
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairL_ne_or_subset_bot
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairR_ne_or_subset_bot
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    eq_singleton_bot_of_subset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    TreeCon_pairL_of_right
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    TreeCon_pairR_of_right

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem TreeEntPayload_trans {u v : Finset (TreeToken a)} {c : TreeToken a}
    (hv : TreeCon A v) (hu : TreeCon A u)
    (hEnts : forall  y  in  u, TreeEnt A v y) (hPay : TreeEntPayload A u c) :
    TreeEntPayload A v c := by
  induction c generalizing u v with
  | bot => exact trivial
  | atom x =>
    rcases hPay with <hne, hA>
    have hlft : forall  y  in  atomFinset u, A.Ent (atomFinset v) y := by
      intro y hy
      exact (hEnts _ (mem_atomFinset.1 hy)).2.2
    have hne' : atomFinset v  !=  {} := by
      obtain <y, hy> := Finset.nonempty_of_ne_empty hne
      exact (hEnts _ (mem_atomFinset.1 hy)).2.1
    exact <hne', A.ent_trans (TreeCon.atom_con_of_atoms A hv hne')
      (TreeCon.atom_con_of_atoms A hu hne) hlft hA>
  | pairL t ih =>
    rcases hPay with <hAt, hConL, hConR, hOr>
    rcases hOr with <hne, hPayL> | <rfl, hRne>
    * obtain <s, hs> := Finset.nonempty_of_ne_empty hne
      have hEy := hEnts _ (mem_pairLFinset.1 hs)
      have hAtv : atomFinset v = {} := hEy.2.1
      have hConLv : TreeCon A (pairLFinset v) := TreeCon.pairL_of_right A hv hAtv
      have hConRv : TreeCon A (pairRFinset v) := hEy.2.2.2.1
      have hEntsL : forall  y  in  pairLFinset u, TreeEnt A (pairLFinset v) y := by
        intro y hy
```

```
      have hEy' := hEnts _ (mem_pairLFinset.1 hy)
      rcases hEy'.2.2.2.2 with <_, hPay'> | <rfl, _>
      * exact <hEy'.2.2.1, hPay'>
      * exact <hConLv, trivial>
    refine <hAtv, hConLv, hConRv, ?_>
    have hEntsPair : forall  y  in  pairLFinset u, TreeEnt A v (.pairL y) :=
      fun y hy => hEnts _ (mem_pairLFinset.1 hy)
    rcases pairL_ne_or_subset_bot A hEntsPair with hneV | hsub
    * exact Or.inl <hneV, ih hConLv hConL hEntsL hPayL>
    * have huBot : pairLFinset u = {.bot} :=
        eq_singleton_bot_of_subset hne hsub
      have ht : t = .bot :=
        TreeEntPayload.of_no_atoms_no_pairs A
          atomFinset_singleton_bot pairLFinset_singleton_bot
          pairRFinset_singleton_bot (huBot  |>  hPayL)
      rcases hEy.2.2.2.2 with <hneV, _> | <_, hR>
      * exact Or.inl <hneV, ih hConLv hConL hEntsL hPayL>
      * exact Or.inr <ht, hR>
  * obtain <s, hs> := Finset.nonempty_of_ne_empty hRne
    have hEy := hEnts _ (mem_pairRFinset.1 hs)
    have hAtv : atomFinset v = {} := hEy.2.1
    refine <hAtv, TreeCon.pairL_of_right A hv hAtv, hEy.2.2.1, ?_>
    rcases hEy.2.2.2.2 with <hneR, _> | <_, hneL>
    * exact Or.inr <rfl, hneR>
    * exact Or.inl <hneL, trivial>
| pairR t ih =>
  rcases hPay with <hAt, hConR, hConL, hOr>
  rcases hOr with <hne, hPayR> | <rfl, hLne>
  * obtain <s, hs> := Finset.nonempty_of_ne_empty hne
    have hEy := hEnts _ (mem_pairRFinset.1 hs)
    have hAtv : atomFinset v = {} := hEy.2.1
    have hConRv : TreeCon A (pairRFinset v) := TreeCon.pairR_of_right A hv hAtv
    have hConLv : TreeCon A (pairLFinset v) := hEy.2.2.2.1
    have hEntsR : forall  y  in  pairRFinset u, TreeEnt A (pairRFinset v) y := by
      intro y hy
      have hEy' := hEnts _ (mem_pairRFinset.1 hy)
      rcases hEy'.2.2.2.2 with <_, hPay'> | <rfl, _>
      * exact <hEy'.2.2.1, hPay'>
      * exact <hConRv, trivial>
    refine <hAtv, hConRv, hConLv, ?_>
    have hEntsPair : forall  y  in  pairRFinset u, TreeEnt A v (.pairR y) :=
      fun y hy => hEnts _ (mem_pairRFinset.1 hy)
    rcases pairR_ne_or_subset_bot A hEntsPair with hneV | hsub
    * exact Or.inl <hneV, ih hConRv hConR hEntsR hPayR>
    * have huBot : pairRFinset u = {.bot} :=
        eq_singleton_bot_of_subset hne hsub
      have ht : t = .bot :=
        TreeEntPayload.of_no_atoms_no_pairs A
          atomFinset_singleton_bot pairLFinset_singleton_bot
          pairRFinset_singleton_bot (huBot  |>  hPayR)
      rcases hEy.2.2.2.2 with <hneV, _> | <_, hL>
      * exact Or.inl <hneV, ih hConRv hConR hEntsR hPayR>
      * exact Or.inr <ht, hL>
  * obtain <s, hs> := Finset.nonempty_of_ne_empty hLne
    have hEy := hEnts _ (mem_pairLFinset.1 hs)
    have hAtv : atomFinset v = {} := hEy.2.1
```

```
      refine <hAtv, TreeCon.pairR_of_right A hv hAtv, hEy.2.2.1, ?_>
      rcases hEy.2.2.2.2 with <hneL, _> | <_, hneR>
      * exact Or.inr <rfl, hneL>
      * exact Or.inl <hneR, trivial>

end InfoSys

end DomainTheory.InformationSystems
```

TreeEnt_trans.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    TreeEntPayload_trans

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem TreeEnt_trans {u v : Finset (TreeToken a)} {c : TreeToken a}
    (hv : TreeCon A v) (hu : TreeCon A u)
    (hEnts : forall  y  in  u, TreeEnt A v y) (hEntc : TreeEnt A u c) :
    TreeEnt A v c :=
  <hv, TreeEntPayload_trans A hv hu hEnts hEntc.2>

end InfoSys

end DomainTheory.InformationSystems
```

treeBot.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
```

```
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

/-- Tree bottom `Delta_T`. -/
def treeBot : TreeToken a := .bot

end InfoSys

end DomainTheory.InformationSystems
```

Factoid81.lean

```
module

public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.treeRhs

@[expose] public section

namespace DomainTheory.InformationSystems.InfoSys

set_option linter.unusedVariables false

theorem theorem_81.{u_1} : forall  {a : Type u_1} [inst : DecidableEq a] (A :
    @DomainTheory.InformationSystems.InfoSys a inst),
  @Eq
    (@DomainTheory.InformationSystems.InfoSys
      (DomainTheory.InformationSystems.InfoSys.SumToken a
        (@DomainTheory.InformationSystems.InfoSys.ProdToken (DomainTheory.
    InformationSystems.InfoSys.TreeToken a)
          (DomainTheory.InformationSystems.InfoSys.TreeToken a)
          (fun (a b : DomainTheory.InformationSystems.InfoSys.TreeToken a) =>
            @DomainTheory.InformationSystems.InfoSys.instDecidableEqTreeToken a inst a
     b)
          (fun (a b : DomainTheory.InformationSystems.InfoSys.TreeToken a) =>
            @DomainTheory.InformationSystems.InfoSys.instDecidableEqTreeToken a inst a
     b)
          (@DomainTheory.InformationSystems.InfoSys.treeSystem a inst A)
          (@DomainTheory.InformationSystems.InfoSys.treeSystem a inst A)))
      fun
        (a b :
          DomainTheory.InformationSystems.InfoSys.SumToken a
```

```
        (@DomainTheory.InformationSystems.InfoSys.ProdToken (DomainTheory.
InformationSystems.InfoSys.TreeToken a)
          (DomainTheory.InformationSystems.InfoSys.TreeToken a)
          (fun (a b : DomainTheory.InformationSystems.InfoSys.TreeToken a) =>
            @DomainTheory.InformationSystems.InfoSys.instDecidableEqTreeToken a
inst a b)
          (fun (a b : DomainTheory.InformationSystems.InfoSys.TreeToken a) =>
            @DomainTheory.InformationSystems.InfoSys.instDecidableEqTreeToken a
inst a b)
          (@DomainTheory.InformationSystems.InfoSys.treeSystem a inst A)
          (@DomainTheory.InformationSystems.InfoSys.treeSystem a inst A))) =>
  @DomainTheory.InformationSystems.InfoSys.instDecidableEqSumToken a
    (@DomainTheory.InformationSystems.InfoSys.ProdToken (DomainTheory.
InformationSystems.InfoSys.TreeToken a)
      (DomainTheory.InformationSystems.InfoSys.TreeToken a)
      (fun (a b : DomainTheory.InformationSystems.InfoSys.TreeToken a) =>
        @DomainTheory.InformationSystems.InfoSys.instDecidableEqTreeToken a inst a
 b)
      (fun (a b : DomainTheory.InformationSystems.InfoSys.TreeToken a) =>
        @DomainTheory.InformationSystems.InfoSys.instDecidableEqTreeToken a inst a
 b)
      (@DomainTheory.InformationSystems.InfoSys.treeSystem a inst A)
      (@DomainTheory.InformationSystems.InfoSys.treeSystem a inst A))
    inst
    (@DomainTheory.InformationSystems.InfoSys.instDecidableEqProdToken
      (DomainTheory.InformationSystems.InfoSys.TreeToken a) (DomainTheory.
InformationSystems.InfoSys.TreeToken a)
      (fun (a b : DomainTheory.InformationSystems.InfoSys.TreeToken a) =>
        @DomainTheory.InformationSystems.InfoSys.instDecidableEqTreeToken a inst a
 b)
      (fun (a b : DomainTheory.InformationSystems.InfoSys.TreeToken a) =>
        @DomainTheory.InformationSystems.InfoSys.instDecidableEqTreeToken a inst a
 b)
      (@DomainTheory.InformationSystems.InfoSys.treeSystem a inst A)
      (@DomainTheory.InformationSystems.InfoSys.treeSystem a inst A))
    a b)
(@DomainTheory.InformationSystems.InfoSys.treeRhs a inst A)
(@DomainTheory.InformationSystems.InfoSys.sumSystem a
  (@DomainTheory.InformationSystems.InfoSys.ProdToken (DomainTheory.
InformationSystems.InfoSys.TreeToken a)
    (DomainTheory.InformationSystems.InfoSys.TreeToken a)
    (fun (a b : DomainTheory.InformationSystems.InfoSys.TreeToken a) =>
      @DomainTheory.InformationSystems.InfoSys.instDecidableEqTreeToken a inst a b
)
    (fun (a b : DomainTheory.InformationSystems.InfoSys.TreeToken a) =>
      @DomainTheory.InformationSystems.InfoSys.instDecidableEqTreeToken a inst a b
)
    (@DomainTheory.InformationSystems.InfoSys.treeSystem a inst A)
    (@DomainTheory.InformationSystems.InfoSys.treeSystem a inst A))
  inst
  (@DomainTheory.InformationSystems.InfoSys.instDecidableEqProdToken
    (DomainTheory.InformationSystems.InfoSys.TreeToken a) (DomainTheory.
InformationSystems.InfoSys.TreeToken a)
    (fun (a b : DomainTheory.InformationSystems.InfoSys.TreeToken a) =>
      @DomainTheory.InformationSystems.InfoSys.instDecidableEqTreeToken a inst a b
)
```

```
        (fun (a b : DomainTheory.InformationSystems.InfoSys.TreeToken a) =>
          @DomainTheory.InformationSystems.InfoSys.instDecidableEqTreeToken a inst a b
    )
        (@DomainTheory.InformationSystems.InfoSys.treeSystem a inst A)
        (@DomainTheory.InformationSystems.InfoSys.treeSystem a inst A))
      A
      (@DomainTheory.InformationSystems.InfoSys.productSystem (DomainTheory.
    InformationSystems.InfoSys.TreeToken a)
        (DomainTheory.InformationSystems.InfoSys.TreeToken a)
        (fun (a b : DomainTheory.InformationSystems.InfoSys.TreeToken a) =>
          @DomainTheory.InformationSystems.InfoSys.instDecidableEqTreeToken a inst a b
    )
        (fun (a b : DomainTheory.InformationSystems.InfoSys.TreeToken a) =>
          @DomainTheory.InformationSystems.InfoSys.instDecidableEqTreeToken a inst a b
    )
        (@DomainTheory.InformationSystems.InfoSys.treeSystem a inst A)
        (@DomainTheory.InformationSystems.InfoSys.treeSystem a inst A))) :=
  @DomainTheory.InformationSystems.InfoSys.treeRhs.eq_sum_product

end DomainTheory.InformationSystems.InfoSys
```

treeSystem.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.treeBot
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    TreeCon_singleton
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    TreeCon_insert_of_ent
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    TreeEnt_trans
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    TreeCon_subset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]
```

```
variable (A : InfoSys a)

/-- **Factoid 8.1.** Tree information system solving `T  ~=  A + (T  x  T)` by
    construction. -/
def treeSystem : InfoSys (TreeToken a) where
  bot := treeBot
  Con := {u | TreeCon A u}
  Ent := TreeEnt A
  con_subset := fun hu hv => TreeCon_subset A hu hv
  con_sing := TreeCon_singleton A
  ent_con := fun h => TreeCon_insert_of_ent A h
  ent_bot := fun hu => <hu, trivial>
  ent_refl := fun hu hp => TreeEnt.of_mem A hu hp
  ent_trans := fun hv hu hEnts hEntc => TreeEnt_trans A hv hu hEnts hEntc

end InfoSys

end DomainTheory.InformationSystems
```

treeRhs.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.treeSystem

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]
variable (A : InfoSys a)

/-- The right-hand side of the domain equation is the official sum of products. -/
def treeRhs : InfoSys (SumToken a (ProdToken (treeSystem A) (treeSystem A))) :=
  sumSystem A (productSystem (treeSystem A) (treeSystem A))

theorem treeRhs.eq_sum_product :
    treeRhs A = sumSystem A (productSystem (treeSystem A) (treeSystem A)) :=
  rfl

end InfoSys
```

```
end DomainTheory.InformationSystems
```

treeUnfold.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.TreeCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.atomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.
    pairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.treeBot
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81.treeSystem

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]
variable (A : InfoSys a)

/-- Unfold a tree token into the sum-of-product carrier. -/
def treeUnfold (t : TreeToken a) :
    SumToken a (ProdToken (treeSystem A) (treeSystem A)) :=
  match t with
  | .bot => .bot
  | .atom x => .left x
  | .pairL t => .right <(t, treeBot), <Or.inr rfl>>
  | .pairR t => .right <(treeBot, t), <Or.inl rfl>>

theorem treeUnfold.atom (x : a) : treeUnfold A (.atom x) = SumToken.left x := rfl

theorem treeUnfold.bot : treeUnfold A treeBot = SumToken.bot := rfl

theorem treeUnfold.pairL (t : TreeToken a) :
    treeUnfold A (.pairL t) = .right <(t, treeBot), <Or.inr rfl>> := rfl

theorem treeUnfold.pairR (t : TreeToken a) :
    treeUnfold A (.pairR t) = .right <(treeBot, t), <Or.inl rfl>> := rfl

theorem treeUnfold.pairL_bot_eq_pairR_bot :
    treeUnfold A (.pairL treeBot) = treeUnfold A (.pairR treeBot) := rfl

end InfoSys

end DomainTheory.InformationSystems
```

4.8.2 Factoid 8.2

A domain for λ-claculus. A lengthy discussion with many references on λ-calculus models can be found in Longo [1982]. All we wish to remark on here is how the method of construction by solving a domain equation can be fit into the new presentation. What I have added to the previous ideas (that in any case came out of an analysis of finite elements of models) is the general view of information systems. In particular the models obtained this way are *not* lattices—hence, the need for the calculations with Con. I hope that the presentation here makes it clearer how "pure" λ-calculus models can be related to other domains having other types of structures—for instance, those needed in denotational semantics.

The domain equation we wish to solve is:

$$\mathbf{D} \cong \mathbf{A} + (\mathbf{D} \to \mathbf{D}).$$

We proceed in much the same way we did for $\mathbf{T}$, except we must now put in data objects appropriate to the function space. Here is construction, where again Δ is chosen outside $D_{\mathbf{A}}$ and $\Delta_{\mathbf{D}} = (\Delta, \Delta)$:

(1) $\Delta_{\mathbf{D}} \in D_{\mathbf{D}}$;

(2) $(X, \Delta) \in D_{\mathbf{D}}$ whenever $X \in D_{\mathbf{A}}$;

(3) $(\Delta, (u, v)) \in D_{\mathbf{D}}$ whenever $u, v \in \mathrm{Con}_{\mathbf{D}}$;

(4) $\emptyset \in \mathrm{Con}_{\mathbf{D}}$;

(5) $u \cup \{\Delta_{\mathbf{D}}\} \in \mathrm{Con}_{\mathbf{D}}$ whenever $u \in \mathrm{Con}_{\mathbf{D}}$;

(6) $\{(X, \Delta) \mid X \in w\} \in \mathrm{Con}_{\mathbf{D}}$ whenever $w \in \mathrm{Con}_{\mathbf{A}}$; and

(7) $\{(\Delta, (u_0, v_0)), \ldots, (\Delta, (u_{n-1}, v_{n-1}))\} \in \mathrm{Con}_{\mathbf{D}}$ provided $u_i, v_i \in \mathrm{Con}_{\mathbf{D}}$ for all $i < n$ and whenever $I \subseteq \{0, \ldots, n-1\}$ and $\bigcup\{u_i \mid i \in I\} \in \mathrm{Con}_{\mathbf{D}}$, then $\bigcup\{v_i \mid i \in I\} \in \mathrm{Con}_{\mathbf{D}}$.

What is different here from the definition of $\mathbf{T}$ is the fact that the concepts $D_{\mathbf{D}}$ and $\mathrm{Con}_{\mathbf{D}}$ are mutually recursive because the data objects are themselves built from consistent sets. The scheme is based on a combination of the sum construct and the function-space construct, but the mutual recursion allows "feedback" to occur.

To complete the definition we have to give the clauses for the inductive definition of entailment. They are:

(8) $u \vdash_{\mathbf{D}} \Delta_{\mathbf{D}}$ always;

(9) $u \cup \{\Delta_{\mathbf{D}}\} \vdash_{\mathbf{D}} Y$ whenever $u \vdash_{\mathbf{D}} Y$;

(10) $\{(X, \Delta) \mid X \in w\} \vdash_{\mathbf{D}} (W, \Delta)$ whenever $w \vdash_{\mathbf{A}} W$;

(11) $\{(\Delta, (u_0, v_0)), \ldots, (\Delta, (u_{n-1}, v_{n-1}))\} \vdash_{\mathbf{D}} (\Delta, (u', v'))$ whenever $\bigcup\{v_i \mid u' \vdash_{\mathbf{D}} u_i\} \vdash_{\mathbf{D}} v'$ and the set on the left is in $\mathrm{Con}_{\mathbf{D}}$.

Obviously these definitions are much shorter if we have a domain in which all sets are consistent, but there are many reasons for retaining the consistency concept throughout. The check that $\mathbf{D}$ is an information system and satisfies the domain equation is mechanical. We cannot detail here how this construction provides a λ-calculus model.

It is clear that these definitions are constructive, and that, with a suitable Gödel numbering of the data objects, the predicates for consistency and entailment are recursively enumerable. However, the recursion used builds up the predicates by going from less complicated data objects to more complicated ones; therefore, the predicates must be recursive, because, for a certain size data object, the derivation that puts it into the predicate is of a bounded length. This observation helps in the discussion of the computability of the operators defined on these domains—another topic we cannot discuss here. ■

RawLamToken.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

inductive RawLamToken (a : Type*) where
  | bot : RawLamToken a
  | atom : a  ->  RawLamToken a
  | funTok : List (RawLamToken a)  ->  List (RawLamToken a)  ->  RawLamToken a

mutual
  def rawLamBEq : RawLamToken a  ->  RawLamToken a  ->  Bool
    | .bot, .bot => true
    | .atom x, .atom y => decide (x = y)
    | .funTok u v, .funTok u' v' => rawLamListBEq u u' && rawLamListBEq v v'
    | _, _ => false
  def rawLamListBEq : List (RawLamToken a)  ->  List (RawLamToken a)  ->  Bool
    | [], [] => true
    | x :: xs, y :: ys => rawLamBEq x y && rawLamListBEq xs ys
    | _, _ => false
end

mutual
  theorem rawLamBEq_eq : forall  a b : RawLamToken a, rawLamBEq a b = true  <->  a = b
  | .bot, .bot => by simp [rawLamBEq]
  | .atom x, .atom y => by
      simp only [rawLamBEq, decide_eq_true_eq]
      exact <congrArg RawLamToken.atom, RawLamToken.atom.inj>
  | .funTok u v, .funTok u' v' => by
      simp only [rawLamBEq, Bool.and_eq_true]
      constructor
      * intro <hu, hv>
        rw [(rawLamListBEq_eq u u').1 hu, (rawLamListBEq_eq v v').1 hv]
      * intro h
        injection h with hu hv
```

```
        subst hu; subst hv
        exact <(rawLamListBEq_eq u u).2 rfl, (rawLamListBEq_eq v v).2 rfl>
  | .bot, .atom _ | .bot, .funTok _ _ | .atom _, .bot | .atom _, .funTok _ _
  | .funTok _ _, .bot | .funTok _ _, .atom _ => by
      constructor
      * intro h; simp [rawLamBEq] at h
      * intro h; cases h
  theorem rawLamListBEq_eq : forall  u v : List (RawLamToken a), rawLamListBEq u v =
    true  <->  u = v
  | [], [] => by simp [rawLamListBEq]
  | x :: xs, y :: ys => by
      simp only [rawLamListBEq, Bool.and_eq_true]
      constructor
      * intro <hx, hxs>
        rw [(rawLamBEq_eq x y).1 hx, (rawLamListBEq_eq xs ys).1 hxs]
      * intro h
        injection h with hx hxs
        subst hx; subst hxs
        exact <(rawLamBEq_eq x x).2 rfl, (rawLamListBEq_eq xs xs).2 rfl>
  | [], _ :: _ | _ :: _, [] => by
      constructor
      * intro h; simp [rawLamListBEq] at h
      * intro h; cases h
end

instance : DecidableEq (RawLamToken a) := fun a b =>
  if h : rawLamBEq a b = true then isTrue ((rawLamBEq_eq a b).1 h)
  else isFalse fun hab => h ((rawLamBEq_eq a b).2 hab)

end InfoSys
end DomainTheory.InformationSystems
```

lamAtomInsert.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

def lamAtomInsert : RawLamToken a  ->  Finset a  ->  Finset a
```

```
  | .atom x => insert x
  | .bot | .funTok _ _ => id

end InfoSys

end DomainTheory.InformationSystems
```

LamEntDepthInst9.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamAtomInsert

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

instance : LeftCommutative (lamAtomInsert : RawLamToken a  ->  Finset a  ->  Finset a)
     :=
  <fun p q s => by cases p <;> cases q <;> first | exact insert_comm' _ _ s | rfl>

end InfoSys

end DomainTheory.InformationSystems
```

lamFunInsert.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive
```

```
namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

def lamFunInsert :
    RawLamToken a  ->  Finset (List (RawLamToken a)  x  List (RawLamToken a))  ->
      Finset (List (RawLamToken a)  x  List (RawLamToken a))
  | .funTok u v => insert (u, v)
  | .bot | .atom _ => id

end InfoSys

end DomainTheory.InformationSystems
```

LamEntDepthInst10.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamFunInsert
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst9

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

instance : LeftCommutative
    (lamFunInsert : RawLamToken a  ->
      Finset (List (RawLamToken a)  x  List (RawLamToken a))  ->
        Finset (List (RawLamToken a)  x  List (RawLamToken a))) :=
  <fun p q s => by cases p <;> cases q <;> first | exact insert_comm' _ _ s | rfl>

end InfoSys

end DomainTheory.InformationSystems
```

lamAtomFinset.lean

```
module
```

```
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamAtomInsert
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst9
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst10

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

def lamAtomFinset (u : Finset (RawLamToken a)) : Finset a :=
  Multiset.foldr lamAtomInsert {} u.1

end InfoSys

end DomainTheory.InformationSystems
```

lamFunFinset.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamFunInsert
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst9
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst10

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys
```

```
variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

def lamFunFinset (u : Finset (RawLamToken a)) :
    Finset (List (RawLamToken a)  x  List (RawLamToken a)) :=
  Multiset.foldr lamFunInsert {} u.1

end InfoSys

end DomainTheory.InformationSystems
```

listToFinset.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

def listToFinset {b : Type*} [DecidableEq b] : List b  ->  Finset b
  | [] => {}
  | x :: xs => insert x (listToFinset xs)

end InfoSys

end DomainTheory.InformationSystems
```

lamInUnionInsert.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    listToFinset
```

```
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst9
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst10

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

def lamInUnionInsert [DecidableEq (RawLamToken a)] :
    List (RawLamToken a)  x  List (RawLamToken a)  ->  Finset (RawLamToken a)  ->
      Finset (RawLamToken a)
  | p, acc => listToFinset p.1  U ' acc

end InfoSys

end DomainTheory.InformationSystems
```

LamEntDepthInst17.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst9
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst10
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamInUnionInsert

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

instance [DecidableEq (RawLamToken a)] :
```

```
    LeftCommutative (lamInUnionInsert (a := a)) :=
  <fun p q s => by
    simp only [lamInUnionInsert]
    ext x; simp only [mem_funion]
    constructor <;> * rintro (h | h | h); exacts [Or.inr (Or.inl h), Or.inl h, Or.inr
    (Or.inr h)]>

end InfoSys

end DomainTheory.InformationSystems
```

lamOutUnionInsert.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    listToFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst9
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst10

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

def lamOutUnionInsert [DecidableEq (RawLamToken a)] :
    List (RawLamToken a)  x  List (RawLamToken a)  ->  Finset (RawLamToken a)  ->
      Finset (RawLamToken a)
  | p, acc => listToFinset p.2  U ' acc

end InfoSys

end DomainTheory.InformationSystems
```

LamEntDepthInst18.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
```

```
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst9
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst10
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamOutUnionInsert
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst17

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

instance [DecidableEq (RawLamToken a)] :
    LeftCommutative (lamOutUnionInsert (a := a)) :=
  <fun p q s => by
    simp only [lamOutUnionInsert]
    ext x; simp only [mem_funion]
    constructor <;> * rintro (h | h | h); exacts [Or.inr (Or.inl h), Or.inl h, Or.inr
    (Or.inr h)]>

end InfoSys

end DomainTheory.InformationSystems
```

lamInputUnion.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst9
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst10
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamInUnionInsert
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst17
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst18
```

```
@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

def lamInputUnion (s : Finset (List (RawLamToken a)  x  List (RawLamToken a))) :
    Finset (RawLamToken a) :=
  Multiset.foldr (lamInUnionInsert (a := a)) {} s.1

end InfoSys

end DomainTheory.InformationSystems
```

lamOutputUnion.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst9
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst10
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamOutUnionInsert
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst17
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst18

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

def lamOutputUnion (s : Finset (List (RawLamToken a)  x  List (RawLamToken a))) :
    Finset (RawLamToken a) :=
  Multiset.foldr (lamOutUnionInsert (a := a)) {} s.1
```

```
end InfoSys

end DomainTheory.InformationSystems
```

max_le_max_nat.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst9
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst10
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst17
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst18

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem max_le_max_nat {a b c d : Nat} (h1 : a  <=  c) (h2 : b  <=  d) :
    max a b  <=  max c d :=
  max_le (le_trans h1 (le_max_left _ _)) (le_trans h2 (le_max_right _ _))

end InfoSys

end DomainTheory.InformationSystems
```

mem_lamFunFinset.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamFunInsert
```

```
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst9
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst10
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamFunFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem mem_lamFunFinset {u : Finset (RawLamToken a)}
    {p : List (RawLamToken a)  x  List (RawLamToken a)} :
    p  in  lamFunFinset u  <->  .funTok p.1 p.2  in  u := by
  have aux (s : Multiset (RawLamToken a)) :
      p  in  Multiset.foldr lamFunInsert {} s  <->  exists  t  in  s, t = .funTok p.1
    p.2 := by
    refine Multiset.induction_on s ?_ ?_
    * constructor
      * intro hp; exact False.elim (Finset.notMem_empty p hp)
      * rintro <_, ht, _>; exact False.elim (by cases ht)
    * intro t rest ih
      cases t with
      | funTok u v =>
        simp only [Multiset.foldr_cons, lamFunInsert, Finset.mem_insert, ih, Multiset.
    mem_cons]
        constructor
        * rintro (hp | <q, hq, hq'>)
          * exact <.funTok u v, Or.inl rfl, by cases hp; rfl>
          * exact <q, Or.inr hq, hq'>
        * rintro <q, hq, hq'>
          rcases hq with rfl | hq
          * injection hq' with hu hv
            exact Or.inl (Prod.ext hu.symm hv.symm)
          * exact Or.inr <q, hq, hq'>
      | bot | atom _ =>
        simp only [Multiset.foldr_cons, lamFunInsert, id_eq, ih, Multiset.mem_cons]
        constructor
        * rintro <q, hq, hq'>; exact <q, Or.inr hq, hq'>
        * rintro <q, hq, hq'>
          rcases hq with rfl | hq
          * exact False.elim (nomatch hq')
          * exact <q, hq, hq'>
  constructor
  * intro hp
    rcases (aux u.1).1 hp with <t, ht, ht'>
    subst ht'; exact Finset.mem_def.2 ht
  * intro hp
```

```
    exact (aux u.1).2 <.funTok p.1 p.2, Finset.mem_def.1 hp, rfl>

end InfoSys

end DomainTheory.InformationSystems
```

mem_lamInputUnion.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    listToFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst9
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst10
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamInUnionInsert
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst17
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst18
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamInputUnion

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem mem_lamInputUnion {s : Finset (List (RawLamToken a)  x  List (RawLamToken a))}
    {t : RawLamToken a} :
    t  in  lamInputUnion s  <->  exists  p  in  s, t  in  listToFinset p.1 := by
  have aux (m : Multiset (List (RawLamToken a)  x  List (RawLamToken a))) :
      t  in  Multiset.foldr (lamInUnionInsert (a := a)) {} m  <->
        exists  p  in  m, t  in  listToFinset p.1 := by
    refine Multiset.induction_on m ?_ ?_
    * constructor
      * intro ht; exact False.elim (Finset.notMem_empty t ht)
      * rintro <_, hp, _>; exact False.elim (by cases hp)
    * intro p rest ih
      simp only [Multiset.foldr_cons, lamInUnionInsert, mem_funion, ih, Multiset.
    mem_cons]
```

```
      constructor
      * rintro (ht | <q, hq, ht>)
        * exact <p, Or.inl rfl, ht>
        * exact <q, Or.inr hq, ht>
      * rintro <q, hq, ht>
        rcases hq with rfl | hq
        * exact Or.inl ht
        * exact Or.inr <q, hq, ht>
  simpa [lamInputUnion] using aux s.1

end InfoSys

end DomainTheory.InformationSystems
```

mem_lamOutputUnion.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    listToFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst9
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst10
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamOutUnionInsert
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst17
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst18
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamOutputUnion

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem mem_lamOutputUnion {s : Finset (List (RawLamToken a)  x  List (RawLamToken a))
    }
    {t : RawLamToken a} :
    t  in  lamOutputUnion s  <->  exists  p  in  s, t  in  listToFinset p.2 := by
  have aux (m : Multiset (List (RawLamToken a)  x  List (RawLamToken a))) :
```

```
      t  in  Multiset.foldr (lamOutUnionInsert (a := a)) {} m  <->
        exists  p  in  m, t  in  listToFinset p.2 := by
    refine Multiset.induction_on m ?_ ?_
    * constructor
      * intro ht; exact False.elim (Finset.notMem_empty t ht)
      * rintro <_, hp, _>; exact False.elim (by cases hp)
    * intro p rest ih
      simp only [Multiset.foldr_cons, lamOutUnionInsert, mem_funion, ih, Multiset.
    mem_cons]
      constructor
      * rintro (ht | <q, hq, ht>)
        * exact <p, Or.inl rfl, ht>
        * exact <q, Or.inr hq, ht>
      * rintro <q, hq, ht>
        rcases hq with rfl | hq
        * exact Or.inl ht
        * exact Or.inr <q, hq, ht>
  simpa [lamOutputUnion] using aux s.1

end InfoSys

end DomainTheory.InformationSystems
```

mem_listToFinset.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    listToFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst9
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst10

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem mem_listToFinset {b : Type*} [DecidableEq b] {x : b} {xs : List b} :
    x  in  listToFinset xs  <->  x  in  xs := by
  induction xs with
  | nil =>
```

```
    constructor
    * intro h; exact False.elim (Finset.notMem_empty x h)
    * intro h; cases h
  | cons a as ih =>
    simp only [listToFinset, Finset.mem_insert, List.mem_cons, ih]

end InfoSys

end DomainTheory.InformationSystems
```

rawDepth.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst9
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst10
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst17
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst18

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

def rawDepth : RawLamToken a  ->  Nat
  | .bot | .atom _ => 0
  | .funTok u v =>
      1 + Nat.max
        (List.foldl (fun d t => Nat.max d (rawDepth t)) 0 u)
        (List.foldl (fun d t => Nat.max d (rawDepth t)) 0 v)

end InfoSys

end DomainTheory.InformationSystems
```

foldl_max_ge_init.lean

```
module
```

```
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst9
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst10
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst17
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst18
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.rawDepth

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

omit [DecidableEq a] in

theorem foldl_max_ge_init (u : List (RawLamToken a)) (init : Nat) :
    init  <=  List.foldl (fun d t => Nat.max d (rawDepth t)) init u := by
  induction u generalizing init with
  | nil => exact le_rfl
  | cons a as ih =>
    exact Nat.le_trans (Nat.le_max_left init (rawDepth a)) (ih _)

end InfoSys

end DomainTheory.InformationSystems
```

foldl_max_mono_init.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst9
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst10
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst17
```

```
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst18
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    max_le_max_nat
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.rawDepth

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

omit [DecidableEq a] in

theorem foldl_max_mono_init {u : List (RawLamToken a)} {i j : Nat} (hij : i  <=  j) :
    List.foldl (fun d t => Nat.max d (rawDepth t)) i u  <=
      List.foldl (fun d t => Nat.max d (rawDepth t)) j u := by
  induction u generalizing i j with
  | nil => exact hij
  | cons a as ih => exact ih (max_le_max_nat hij le_rfl)

end InfoSys

end DomainTheory.InformationSystems
```

foldl_list_le_of_mem.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst9
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst10
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst17
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst18
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.rawDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    foldl_max_mono_init
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    foldl_max_ge_init

@[expose] public section
```

```
namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

omit [DecidableEq a] in

theorem foldl_list_le_of_mem {t : RawLamToken a} {u : List (RawLamToken a)}
    (ht : t  in  u) :
    rawDepth t  <=  List.foldl (fun d t => Nat.max d (rawDepth t)) 0 u := by
  induction u with
  | nil => cases ht
  | cons a as ih =>
    simp only [List.mem_cons] at ht
    rcases ht with rfl | ht
    * exact Nat.le_trans (Nat.le_max_right 0 (rawDepth t))
        (foldl_max_ge_init as (Nat.max 0 (rawDepth t)))
    * exact Nat.le_trans (ih ht) (foldl_max_mono_init (Nat.zero_le _))

end InfoSys

end DomainTheory.InformationSystems
```

maxDepthInsert.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst9
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst10
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst17
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst18
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.rawDepth

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys
```

```
variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

def maxDepthInsert : RawLamToken a  ->  Nat  ->  Nat :=
  fun t d => Nat.max (rawDepth t) d

end InfoSys

end DomainTheory.InformationSystems
```

LamEntDepthInst31.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst9
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst10
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst17
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst18
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    maxDepthInsert

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

instance : LeftCommutative (maxDepthInsert (a := a)) :=
  <fun a b d => by simp only [maxDepthInsert]; exact Nat.max_left_comm _ _ _>

end InfoSys

end DomainTheory.InformationSystems
```

finsetMaxDepth.lean

```
module
```

```
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst9
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst10
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst17
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst18
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    maxDepthInsert
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst31
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    foldl_list_le_of_mem
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    foldr_max_le_of_forall
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    foldr_max_lt_of_forall
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamFunFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamInputUnion
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamOutputUnion
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    listToFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    mem_lamFunFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    mem_lamInputUnion
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    mem_lamOutputUnion
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    mem_listToFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.rawDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    rawDepth_funTok_gt_left
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    rawDepth_funTok_gt_right

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]
variable (A : InfoSys a)
```

```
def finsetMaxDepth (u : Finset (RawLamToken a)) : Nat :=
  Multiset.foldr (maxDepthInsert (a := a)) 0 u.1

theorem finsetMaxDepth.listToFinset_le (u : List (RawLamToken a)) :
    finsetMaxDepth (listToFinset u)  <=
      List.foldl (fun d t => Nat.max d (rawDepth t)) 0 u := by
  simp only [finsetMaxDepth]
  exact foldr_max_le_of_forall fun t ht =>
    foldl_list_le_of_mem ((mem_listToFinset).1 (Finset.mem_def.2 ht))

theorem finsetMaxDepth.list_lt_funTok (u v : List (RawLamToken a)) :
    finsetMaxDepth (listToFinset u) < rawDepth (.funTok u v) := by
  have h1 := finsetMaxDepth.listToFinset_le u
  have h2 : List.foldl (fun d t => Nat.max d (rawDepth t)) 0 u  <=
      Nat.max (List.foldl (fun d t => Nat.max d (rawDepth t)) 0 u)
        (List.foldl (fun d t => Nat.max d (rawDepth t)) 0 v) :=
    Nat.le_max_left _ _
  simp only [rawDepth]
  exact Nat.lt_of_le_of_lt (Nat.le_trans h1 h2) (Nat.lt_add_of_pos_left (Nat.succ_pos
    0))

omit [DecidableEq a] in

theorem le_finsetMaxDepth_of_mem {u : Finset (RawLamToken a)} {t : RawLamToken a}
    (ht : t  in  u) : rawDepth t  <=  finsetMaxDepth u := by
  simp only [finsetMaxDepth]
  have ht' : t  in  u.1 := Finset.mem_def.1 ht
  revert ht'
  refine Multiset.induction_on u.1 ?_ ?_
  * intro ht'; cases ht'
  * intro a s ih ht'
    simp only [Multiset.mem_cons] at ht'
    simp only [Multiset.foldr_cons, maxDepthInsert]
    rcases ht' with rfl | ht'
    * exact Nat.le_max_left _ _
    * exact Nat.le_trans (ih ht') (Nat.le_max_right _ _)

theorem finsetMaxDepth.inputUnion_lt {u : Finset (RawLamToken a)}
    {s : Finset (List (RawLamToken a)  x  List (RawLamToken a))}
    (hs : s  subseteq  lamFunFinset u) (hfun : lamFunFinset u  !=  {}) :
    finsetMaxDepth (lamInputUnion s) < finsetMaxDepth u := by
  obtain <p0, hp0> := Finset.nonempty_of_ne_empty hfun
  have htok0 : .funTok p0.1 p0.2  in  u := mem_lamFunFinset.1 hp0
  have hpos : 0 < finsetMaxDepth u := by
    have hraw : 0 < rawDepth (.funTok p0.1 p0.2) := by
      simp only [rawDepth]
      exact Nat.add_pos_left (Nat.succ_pos 0) _
    exact Nat.lt_of_lt_of_le hraw (le_finsetMaxDepth_of_mem htok0)
  refine foldr_max_lt_of_forall (s := (lamInputUnion s).1) hpos ?_
  intro t ht
  have ht' : t  in  lamInputUnion s := Finset.mem_def.2 ht
  rcases (mem_lamInputUnion).1 ht' with <p, hp, htp>
  have htok : .funTok p.1 p.2  in  u := mem_lamFunFinset.1 (hs hp)
  exact Nat.lt_of_lt_of_le
    (rawDepth_funTok_gt_left ((mem_listToFinset).1 htp))
    (le_finsetMaxDepth_of_mem htok)
```

```
theorem finsetMaxDepth.outputUnion_lt {u : Finset (RawLamToken a)}
    {s : Finset (List (RawLamToken a)  x  List (RawLamToken a))}
    (hs : s  subseteq  lamFunFinset u) (hfun : lamFunFinset u  !=  {}) :
    finsetMaxDepth (lamOutputUnion s) < finsetMaxDepth u := by
  obtain <p0, hp0> := Finset.nonempty_of_ne_empty hfun
  have htok0 : .funTok p0.1 p0.2  in  u := mem_lamFunFinset.1 hp0
  have hpos : 0 < finsetMaxDepth u := by
    have hraw : 0 < rawDepth (.funTok p0.1 p0.2) := by
      simp only [rawDepth]
      exact Nat.add_pos_left (Nat.succ_pos 0) _
    exact Nat.lt_of_lt_of_le hraw (le_finsetMaxDepth_of_mem htok0)
  refine foldr_max_lt_of_forall (s := (lamOutputUnion s).1) hpos ?_
  intro t ht
  have ht' : t  in  lamOutputUnion s := Finset.mem_def.2 ht
  rcases (mem_lamOutputUnion).1 ht' with <p, hp, htp>
  have htok : .funTok p.1 p.2  in  u := mem_lamFunFinset.1 (hs hp)
  exact Nat.lt_of_lt_of_le
    (rawDepth_funTok_gt_right ((mem_listToFinset).1 htp))
    (le_finsetMaxDepth_of_mem htok)

theorem finsetMaxDepth.list_lt_funTok_out (u v : List (RawLamToken a)) :
    finsetMaxDepth (listToFinset v) < rawDepth (.funTok u v) := by
  have h1 := finsetMaxDepth.listToFinset_le v
  have h2 : List.foldl (fun d t => Nat.max d (rawDepth t)) 0 v  <=
      Nat.max (List.foldl (fun d t => Nat.max d (rawDepth t)) 0 u)
        (List.foldl (fun d t => Nat.max d (rawDepth t)) 0 v) :=
    Nat.le_max_right _ _
  simp only [rawDepth]
  exact Nat.lt_of_le_of_lt (Nat.le_trans h1 h2) (Nat.lt_add_of_pos_left (Nat.succ_pos
    0))

theorem finsetMaxDepth.of_mem_funTok {u : Finset (RawLamToken a)}
    {xs ys : List (RawLamToken a)} (h : .funTok xs ys  in  u) :
    finsetMaxDepth (listToFinset xs) < finsetMaxDepth u  /\
      finsetMaxDepth (listToFinset ys) < finsetMaxDepth u :=
  <Nat.lt_of_lt_of_le (finsetMaxDepth.list_lt_funTok xs ys) (le_finsetMaxDepth_of_mem
    h),
    Nat.lt_of_lt_of_le (finsetMaxDepth.list_lt_funTok_out xs ys) (
    le_finsetMaxDepth_of_mem h)>

end InfoSys

end DomainTheory.InformationSystems
```

foldr_max_le_of_forall.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
```

```
    LamEntDepthInst9
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst10
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst17
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst18
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.rawDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    maxDepthInsert
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst31

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

omit [DecidableEq a] in

theorem foldr_max_le_of_forall {s : Multiset (RawLamToken a)} {b : Nat}
    (h : forall  t  in  s, rawDepth t  <=  b) :
    Multiset.foldr (maxDepthInsert (a := a)) 0 s  <=  b := by
  revert h
  refine Multiset.induction_on s ?_ ?_
  * intro; exact Nat.zero_le _
  * intro a s ih h
    simp only [Multiset.foldr_cons, maxDepthInsert]
    exact Nat.max_le.2 <h a (Multiset.mem_cons_self _ _),
      ih fun t ht => h t (Multiset.mem_cons_of_mem ht)>

end InfoSys

end DomainTheory.InformationSystems
```

foldr_max_lt_of_forall.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst9
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst10
```

```
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst17
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst18
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.rawDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    maxDepthInsert
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst31

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

omit [DecidableEq a] in

theorem foldr_max_lt_of_forall {s : Multiset (RawLamToken a)} {b : Nat}
    (hb : 0 < b) (h : forall  t  in  s, rawDepth t < b) :
    Multiset.foldr (maxDepthInsert (a := a)) 0 s < b := by
  revert h
  refine Multiset.induction_on s ?_ ?_
  * intro; exact hb
  * intro a s ih h
    simp only [Multiset.foldr_cons, maxDepthInsert]
    exact Nat.max_lt.2 <h a (Multiset.mem_cons_self _ _),
      ih fun t ht => h t (Multiset.mem_cons_of_mem ht)>

end InfoSys

end DomainTheory.InformationSystems
```

rawDepth_funTok_gt_left.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst9
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst10
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst17
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
```

```
    LamEntDepthInst18
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.rawDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    foldl_list_le_of_mem

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

omit [DecidableEq a] in

theorem rawDepth_funTok_gt_left {u v : List (RawLamToken a)} {t : RawLamToken a}
    (ht : t  in  u) : rawDepth t < rawDepth (.funTok u v) := by
  have h : rawDepth t  <=
      Nat.max (List.foldl (fun d t => Nat.max d (rawDepth t)) 0 u)
        (List.foldl (fun d t => Nat.max d (rawDepth t)) 0 v) :=
    Nat.le_trans (foldl_list_le_of_mem ht) (Nat.le_max_left _ _)
  simp only [rawDepth]
  exact Nat.lt_of_le_of_lt h (Nat.lt_add_of_pos_left (Nat.succ_pos 0))

end InfoSys

end DomainTheory.InformationSystems
```

rawDepth_funTok_gt_right.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst9
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst10
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst17
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst18
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.rawDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    foldl_list_le_of_mem

@[expose] public section
```

```
namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

omit [DecidableEq a] in

theorem rawDepth_funTok_gt_right {u v : List (RawLamToken a)} {t : RawLamToken a}
    (ht : t  in  v) : rawDepth t < rawDepth (.funTok u v) := by
  have h : rawDepth t  <=
      Nat.max (List.foldl (fun d t => Nat.max d (rawDepth t)) 0 u)
        (List.foldl (fun d t => Nat.max d (rawDepth t)) 0 v) :=
    Nat.le_trans (foldl_list_le_of_mem ht) (Nat.le_max_right _ _)
  simp only [rawDepth]
  exact Nat.lt_of_le_of_lt h (Nat.lt_add_of_pos_left (Nat.succ_pos 0))

end InfoSys

end DomainTheory.InformationSystems
```

LamConDepth.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    listToFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst9
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst10
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamAtomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamFunFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    mem_lamFunFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst17
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst18
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamInputUnion
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamOutputUnion
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
```

```
    LamEntDepthInst31
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    finsetMaxDepth

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

/-- Depth-based consistency: FunCon in the recursive clause (Scott (7)).
Uses `decidableEq_finset` so the empty-fun branch does not pull `Classical.choice`. -/
def LamConDepth (u : Finset (RawLamToken a)) : Prop :=
  haveI : DecidableEq (Finset (List (RawLamToken a)  x  List (RawLamToken a))) :=
    decidableEq_finset
  if _hF : lamFunFinset u = {} then
    lamAtomFinset u  in  A.Con
  else
    lamAtomFinset u = {}  /\
      (forall  p  in  lamFunFinset u,
        LamConDepth (listToFinset p.1)  /\  LamConDepth (listToFinset p.2))  /\
      (forall  s  subseteq  lamFunFinset u,
        LamConDepth (lamInputUnion s)  ->  LamConDepth (lamOutputUnion s))
termination_by finsetMaxDepth u
decreasing_by
  * exact (finsetMaxDepth.of_mem_funTok (mem_lamFunFinset.1 ?_?)).1
  * exact (finsetMaxDepth.of_mem_funTok (mem_lamFunFinset.1 ?_?)).2
  * exact finsetMaxDepth.inputUnion_lt ?_? ?_?
  * exact finsetMaxDepth.outputUnion_lt ?_? ?_?

end InfoSys

end DomainTheory.InformationSystems
```

LamEntDepth.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    listToFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamAtomInsert
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamFunInsert
```

```
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamAtomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamFunFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamInUnionInsert
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamOutUnionInsert
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamInputUnion
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamOutputUnion
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.rawDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    maxDepthInsert
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    finsetMaxDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst9
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst10
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    mem_listToFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    mem_lamFunFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst17
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst18
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    mem_lamInputUnion
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    mem_lamOutputUnion
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    max_le_max_nat
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    foldl_max_mono_init
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    foldl_max_ge_init
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    foldl_list_le_of_mem
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    rawDepth_funTok_gt_left
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    rawDepth_funTok_gt_right
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepthInst31
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    foldr_max_le_of_forall
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    foldr_max_lt_of_forall

@[expose] public section
```

```
namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]
variable (A : InfoSys a)

/-- Depth-based entailment as an inductive Prop (Scott (8)-(11)).
Premises are positive, so FunEnt-style recursion is allowed -- unlike FunCon
inside an inductive `LamCon`. -/
inductive LamEntDepth : Finset (RawLamToken a)  ->  RawLamToken a  ->  Prop where
  | bot {u : Finset (RawLamToken a)}
      (hu : LamConDepth A u) : LamEntDepth u .bot
  | atom {u : Finset (RawLamToken a)} {x : a}
      (hu : LamConDepth A u)
      (hF : lamFunFinset u = {})
      (hne : lamAtomFinset u  !=  {})
      (hEnt : A.Ent (lamAtomFinset u) x) : LamEntDepth u (.atom x)
  | funTok {u : Finset (RawLamToken a)} {xs ys : List (RawLamToken a)}
      {s : Finset (List (RawLamToken a)  x  List (RawLamToken a))}
      (hu : LamConDepth A u)
      (hA : lamAtomFinset u = {})
      (hConIn : LamConDepth A (listToFinset xs))
      (hConOut : LamConDepth A (listToFinset ys))
      (hs : s  subseteq  lamFunFinset u)
      /- Function tokens are only entailed in the function world (cf. `SumEnt` right).
    -/
      (hWorld : lamFunFinset u  !=  {})
      /- Atom/bot inputs only when `xs` is atom-world -- blocks mixed-world FunCon gaps
    . -/
      (hInCompat : lamFunFinset (listToFinset xs) = {}  ->
        lamFunFinset (lamInputUnion s) = {})
      (hEntIn : forall  p  in  s, forall  z  in  listToFinset p.1, LamEntDepth (
    listToFinset xs) z)
      (hEntOut : forall  z  in  listToFinset ys, LamEntDepth (lamOutputUnion s) z) :
      LamEntDepth u (.funTok xs ys)

theorem LamEntDepth_bot {u : Finset (RawLamToken a)} (hu : LamConDepth A u) :
    LamEntDepth A u .bot :=
  LamEntDepth.bot hu

theorem LamEntDepth.con {u : Finset (RawLamToken a)} {t : RawLamToken a}
    (ht : LamEntDepth A u t) : LamConDepth A u := by
  cases ht with
  | bot hu => exact hu
  | atom hu _ _ _ => exact hu
  | funTok hu _ _ _ _ _ _ _ _ => exact hu

end InfoSys

end DomainTheory.InformationSystems
```

LamConN.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

/-- Stage-`n` consistency (Scott (4)-(7), stratified). -/
def LamConN : Nat  ->  Finset (RawLamToken a)  ->  Prop
  | 0, u => lamAtomFinset u  in  A.Con  /\  lamFunFinset u = {}
  | n + 1, u =>
      LamConN n u  \/
        (lamAtomFinset u = {}  /\
          (forall  p  in  lamFunFinset u,
            LamConN n (listToFinset p.1)  /\  LamConN n (listToFinset p.2))  /\
            forall  s  subseteq  lamFunFinset u,
              LamConN n (lamInputUnion s)  ->  LamConN n (lamOutputUnion s))

end InfoSys

end DomainTheory.InformationSystems
```

LamCon.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamConN

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive
```

```
namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

def LamCon (u : Finset (RawLamToken a)) : Prop :=
  exists  n, LamConN A n u

end InfoSys

end DomainTheory.InformationSystems
```

IsLamToken.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamCon

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

/-- Well-formed tokens (Scott (1)-(3)). -/
class IsLamToken (t : RawLamToken a) : Prop where
  out : match t with
    | .bot | .atom _ => True
    | .funTok u v => LamCon A (listToFinset u)  /\  LamCon A (listToFinset v)

end InfoSys

end DomainTheory.InformationSystems
```

LamToken.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
```

```
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.IsLamToken

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

def LamToken : Type _ := { t : RawLamToken a // IsLamToken A t }

end InfoSys

end DomainTheory.InformationSystems
```

BasicInst23.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamToken

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

instance : DecidableEq (LamToken A) := Subtype.instDecidableEq

end InfoSys

end DomainTheory.InformationSystems
```

lamAtomFinset_empty.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

theorem lamAtomFinset_empty : lamAtomFinset ({} : Finset (RawLamToken a)) = {} := rfl

end InfoSys

end DomainTheory.InformationSystems
```

lamBot.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

def lamBot : LamToken A := <.bot, <trivial>>
```

```
end InfoSys

end DomainTheory.InformationSystems
```

lamForget.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

def lamForget (t : LamToken A) : RawLamToken a := t.val

end InfoSys

end DomainTheory.InformationSystems
```

lamForgetInsert.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.lamForget

@[expose] public section
```

```
namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

def lamForgetInsert :
    LamToken A  ->  Finset (RawLamToken a)  ->  Finset (RawLamToken a) :=
  fun t => insert (lamForget A t)

end InfoSys

end DomainTheory.InformationSystems
```

BasicInst27.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.lamForget
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamForgetInsert

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

instance : LeftCommutative (lamForgetInsert (A := A)) :=
  <fun p q s => insert_comm' (lamForget A p) (lamForget A q) s>

end InfoSys

end DomainTheory.InformationSystems
```

LamWF.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

/-- Well-formed tokens under depth Con (Scott (1)-(3) for the InfoSys instance).
Early staged `IsLamToken` / `LamToken` remain scaffolding only. -/
class LamWF (t : RawLamToken a) : Prop where
  out : match t with
    | .bot | .atom _ => True
    | .funTok xs ys => LamConDepth A (listToFinset xs)  /\  LamConDepth A (
    listToFinset ys)

end InfoSys

end DomainTheory.InformationSystems
```

LamConDepth_of_fun_empty.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27
```

```
@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem LamConDepth_of_fun_empty {u : Finset (RawLamToken a)}
    (hF : lamFunFinset u = {}) :
    LamConDepth A u  <->  lamAtomFinset u  in  A.Con := by
  have : DecidableEq (Finset (List (RawLamToken a)  x  List (RawLamToken a))) :=
    decidableEq_finset
  rw [LamConDepth.eq_1, dite_eq_left hF]

end InfoSys

end DomainTheory.InformationSystems
```

LamConDepth_of_fun_nonempty.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem LamConDepth_of_fun_nonempty {u : Finset (RawLamToken a)}
    (hF : lamFunFinset u  !=  {}) :
    LamConDepth A u  <->
      lamAtomFinset u = {}  /\
```

```
        (forall  p  in  lamFunFinset u,
          LamConDepth A (listToFinset p.1)  /\  LamConDepth A (listToFinset p.2))  /\
        (forall  s  subseteq  lamFunFinset u,
          LamConDepth A (lamInputUnion s)  ->  LamConDepth A (lamOutputUnion s)) := by
  have : DecidableEq (Finset (List (RawLamToken a)  x  List (RawLamToken a))) :=
    decidableEq_finset
  rw [LamConDepth.eq_1, dite_eq_right hF]

end InfoSys

end DomainTheory.InformationSystems
```

LamDToken.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamWF

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

/-- Carrier of `lambdaSystem`. -/
def LamDToken : Type _ := { t : RawLamToken a // LamWF A t }

end InfoSys

end DomainTheory.InformationSystems
```

BasicInst70.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
```

```
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamDToken

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

instance : DecidableEq (LamDToken A) := Subtype.instDecidableEq

end InfoSys

end DomainTheory.InformationSystems
```

LamEntN.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamConN
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]
```

```
variable (A : InfoSys a)

/-- Stage-`n` entailment on raw tokens (Scott (8)-(11), stratified). -/
def LamEntN : Nat  ->  Finset (RawLamToken a)  ->  RawLamToken a  ->  Prop
  | 0, u, .bot => LamConN A 0 u
  | 0, u, .atom x =>
      LamConN A 0 u  /\  lamAtomFinset u  !=  {}  /\  A.Ent (lamAtomFinset u) x
  | 0, _, .funTok _ _ => False
  | n + 1, u, t =>
      LamEntN n u t  \/
        match t with
        | .bot => LamConN A (n + 1) u
        | .atom x =>
            LamConN A (n + 1) u  /\  lamFunFinset u = {}  /\
              lamAtomFinset u  !=  {}  /\  A.Ent (lamAtomFinset u) x
        | .funTok u' v' =>
            LamConN A (n + 1) u  /\  lamAtomFinset u = {}  /\
              LamConN A n (listToFinset u')  /\  LamConN A n (listToFinset v')  /\
              exists  s  subseteq  lamFunFinset u,
                (forall  p  in  s, forall  t  in  listToFinset p.1, LamEntN n (
    listToFinset u') t)  /\
                  (forall  t  in  listToFinset v', LamEntN n (lamOutputUnion s) t)

end InfoSys

end DomainTheory.InformationSystems
```

LamEnt.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamEntN

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)
```

```
/-- Unstaged raw entailment. -/
def LamEnt (u : Finset (RawLamToken a)) (t : RawLamToken a) : Prop :=
  exists  n, LamEntN A n u t

end InfoSys

end DomainTheory.InformationSystems
```

filter_ne_subset_of_subset_insert_lam.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem filter_ne_subset_of_subset_insert_lam
    {p : List (RawLamToken a)  x  List (RawLamToken a)}
    {t w : Finset (List (RawLamToken a)  x  List (RawLamToken a))}
    (ht : t  subseteq  insert p w) : t.filter (*  !=  p)  subseteq  w := by
  intro q hq
  have <hqt, hne> := Finset.mem_filter.mp hq
  have : q  in  insert p w := ht hqt
  rcases Finset.mem_insert.mp this with rfl | hqW
  * exact False.elim (hne rfl)
  * exact hqW

end InfoSys

end DomainTheory.InformationSystems
```

finsetMaxDepth_funion_le.lean

```
module
```

```
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem finsetMaxDepth_funion_le (u v : Finset (RawLamToken a)) :
    finsetMaxDepth (u  U ' v)  <=  Nat.max (finsetMaxDepth u) (finsetMaxDepth v) := by
  refine foldr_max_le_of_forall ?_
  intro t ht
  have ht' : t  in  u  U ' v := Finset.mem_def.2 ht
  rcases mem_funion.1 ht' with hu | hv
  * exact Nat.le_trans (le_finsetMaxDepth_of_mem hu) (Nat.le_max_left _ _)
  * exact Nat.le_trans (le_finsetMaxDepth_of_mem hv) (Nat.le_max_right _ _)

end InfoSys

end DomainTheory.InformationSystems
```

finsetMaxDepth_insert_le.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27

@[expose] public section
```

```
namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem finsetMaxDepth_insert_le (u : Finset (RawLamToken a)) (a : RawLamToken a) :
    finsetMaxDepth (insert a u)  <=  Nat.max (rawDepth a) (finsetMaxDepth u) := by
  refine foldr_max_le_of_forall ?_
  intro x hx
  have hx' : x  in  insert a u := Finset.mem_def.2 hx
  rcases Finset.mem_insert.mp hx' with rfl | hu
  * exact Nat.le_max_left _ _
  * exact Nat.le_trans (le_finsetMaxDepth_of_mem hu) (Nat.le_max_right _ _)

end InfoSys

end DomainTheory.InformationSystems
```

funion_assoc_lam.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem funion_assoc_lam (a b c : Finset (RawLamToken a)) :
    (a  U ' b)  U ' c = a  U ' (b  U ' c) := by
  ext x
```

```
  constructor
  * intro hx
    rcases mem_funion.mp hx with h | h
    * rcases mem_funion.mp h with h | h
      * exact mem_funion.mpr (Or.inl h)
      * exact mem_funion.mpr (Or.inr (mem_funion.mpr (Or.inl h)))
    * exact mem_funion.mpr (Or.inr (mem_funion.mpr (Or.inr h)))
  * intro hx
    rcases mem_funion.mp hx with h | h
    * exact mem_funion.mpr (Or.inl (mem_funion.mpr (Or.inl h)))
    * rcases mem_funion.mp h with h | h
      * exact mem_funion.mpr (Or.inl (mem_funion.mpr (Or.inr h)))
      * exact mem_funion.mpr (Or.inr h)

end InfoSys

end DomainTheory.InformationSystems
```

funion_comm_lam.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem funion_comm_lam (a b : Finset (RawLamToken a)) : a  U ' b = b  U ' a := by
  ext x
  constructor
  * intro hx
    rcases mem_funion.mp hx with h | h
    * exact mem_funion.mpr (Or.inr h)
    * exact mem_funion.mpr (Or.inl h)
  * intro hx
    rcases mem_funion.mp hx with h | h
```

```
    * exact mem_funion.mpr (Or.inr h)
    * exact mem_funion.mpr (Or.inl h)

end InfoSys

end DomainTheory.InformationSystems
```

funion_empty_right_lam.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem funion_empty_right_lam (u : Finset (RawLamToken a)) :
    u  U ' ({} : Finset (RawLamToken a)) = u := by
  ext x; simp only [mem_funion, Finset.notMem_empty, or_false]

end InfoSys

end DomainTheory.InformationSystems
```

insert_filter_ne_lam.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
```

```
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem insert_filter_ne_lam
    {p : List (RawLamToken a)  x  List (RawLamToken a)}
    {t : Finset (List (RawLamToken a)  x  List (RawLamToken a))}
    (hp : p  in  t) : insert p (t.filter (*  !=  p)) = t := by
  ext q
  constructor
  * intro hq
    rcases Finset.mem_insert.mp hq with rfl | hq
    * exact hp
    * exact (Finset.mem_filter.mp hq).1
  * intro hq
    if h : q = p then
      exact Finset.mem_insert.mpr (Or.inl h)
    else
      exact Finset.mem_insert.mpr (Or.inr (Finset.mem_filter.mpr <hq, h>))

end InfoSys

end DomainTheory.InformationSystems
```

lamDBot.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamDToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst70
```

```
@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

def lamDBot : LamDToken A := <.bot, <trivial>>

end InfoSys

end DomainTheory.InformationSystems
```

lamDForget.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamDToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst70

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

def lamDForget (t : LamDToken A) : RawLamToken a := t.val

end InfoSys

end DomainTheory.InformationSystems
```

lamDForgetInsert.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamDToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst70
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.lamDForget

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

def lamDForgetInsert :
    LamDToken A  ->  Finset (RawLamToken a)  ->  Finset (RawLamToken a) :=
  fun t => insert (lamDForget A t)

end InfoSys

end DomainTheory.InformationSystems
```

BasicInst74.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27
```

```
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst70
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.lamDForget
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamDForgetInsert

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

instance : LeftCommutative (lamDForgetInsert (A := A)) :=
  <fun p q s => insert_comm' (lamDForget A p) (lamDForget A q) s>

end InfoSys

end DomainTheory.InformationSystems
```

IsLamWF_of_mem.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConDepth_of_fun_nonempty
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamWF
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst70
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst74

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys
```

```
variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem IsLamWF_of_mem {u : Finset (RawLamToken a)} (hu : LamConDepth A u)
    {t : RawLamToken a} (ht : t  in  u) : LamWF A t := by
  have : DecidableEq (Finset (List (RawLamToken a)  x  List (RawLamToken a))) :=
    decidableEq_finset
  match t with
  | .bot | .atom _ => exact <trivial>
  | .funTok xs ys =>
    have hmem : (xs, ys)  in  lamFunFinset u := mem_lamFunFinset.2 ht
    by_cases hF : lamFunFinset u = {}
    * exact False.elim (Finset.notMem_empty _ (hF  |>  hmem))
    * exact <((LamConDepth_of_fun_nonempty A hF).1 hu).2.1 _ hmem>

end InfoSys

end DomainTheory.InformationSystems
```

lamDForgetFinset.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamDToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst70
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamDForgetInsert
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst74
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    mem_foldr_lamDForget

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]
```

```
variable (A : InfoSys a)

def lamDForgetFinset (u : Finset (LamDToken A)) : Finset (RawLamToken a) :=
  Multiset.foldr (lamDForgetInsert (A := A)) {} u.1

theorem mem_lamDForgetFinset {u : Finset (LamDToken A)} {t : RawLamToken a} :
    t  in  lamDForgetFinset A u  <->  exists  p  in  u, p.val = t := by
  simpa [lamDForgetFinset] using mem_foldr_lamDForget A u.1 t

theorem lamDForgetFinset.insert (u : Finset (LamDToken A)) (p : LamDToken A) :
    lamDForgetFinset A (Insert.insert p u) = Insert.insert p.val (lamDForgetFinset A u
    ) := by
  ext t
  constructor
  * intro ht
    rcases (mem_lamDForgetFinset A).1 ht with <q, hq, rfl>
    rcases Finset.mem_insert.mp hq with rfl | hq
    * exact Finset.mem_insert_self _ _
    * exact Finset.mem_insert_of_mem ((mem_lamDForgetFinset A).2 <q, hq, rfl>)
  * intro ht
    rcases Finset.mem_insert.mp ht with rfl | ht
    * exact (mem_lamDForgetFinset A).2 <p, Finset.mem_insert_self p u, rfl>
    * rcases (mem_lamDForgetFinset A).1 ht with <q, hq, rfl>
      exact (mem_lamDForgetFinset A).2 <q, Finset.mem_insert_of_mem hq, rfl>

theorem lamDForgetFinset.mono {u v : Finset (LamDToken A)} (h : v  subseteq  u) :
    lamDForgetFinset A v  subseteq  lamDForgetFinset A u := by
  intro t ht
  rcases (mem_lamDForgetFinset A).1 ht with <p, hp, rfl>
  exact (mem_lamDForgetFinset A).2 <p, h hp, rfl>

theorem lamDForgetFinset.singleton (p : LamDToken A) :
    lamDForgetFinset A {p} = {p.val} := by
  ext t
  constructor
  * intro ht
    rcases (mem_lamDForgetFinset A).1 ht with <q, hq, hq'>
    have : q = p := Finset.mem_singleton.mp hq
    subst this; exact Finset.mem_singleton.mpr hq'.symm
  * intro ht
    exact (mem_lamDForgetFinset A).2
      <p, Finset.mem_singleton_self p, (Finset.mem_singleton.mp ht).symm>

end InfoSys

end DomainTheory.InformationSystems
```

LamDCon.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
```

```
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamDToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst70
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst74
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamDForgetFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConDepth_singleton_bot
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConDepth_singleton_atom
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConDepth_singleton_funTok
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    IsLamWF_of_mem
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    listToLamDFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamDForget_listToLamDFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConDepth_mono

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

/-- Bundled consistency on `LamDToken` finsets. -/
def LamDCon (u : Finset (LamDToken A)) : Prop :=
  LamConDepth A (lamDForgetFinset A u)

theorem LamDCon.singleton (p : LamDToken A) : LamDCon A {p} := by
  rw [LamDCon, lamDForgetFinset.singleton]
  match p.val, p.property with
  | .bot, _ => exact LamConDepth_singleton_bot A
  | .atom x, _ => exact LamConDepth_singleton_atom A x
  | .funTok xs ys, <<hxs, hys>> => exact LamConDepth_singleton_funTok A xs ys hxs hys

theorem LamDCon.listToLamDFinset (xs : List (RawLamToken a))
    (hCon : LamConDepth A (listToFinset xs)) :
    LamDCon A (listToLamDFinset A xs fun _t ht =>
      IsLamWF_of_mem A hCon ((mem_listToFinset).2 ht)) := by
```

```
  rw [LamDCon, lamDForget_listToLamDFinset]
  exact hCon

theorem LamDCon.subset {u v : Finset (LamDToken A)}
    (hu : LamDCon A u) (hv : v  subseteq  u) : LamDCon A v :=
  LamConDepth_mono A (lamDForgetFinset.mono A hv) hu

end InfoSys

end DomainTheory.InformationSystems
```

LamDEnt.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamDToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst70
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst74
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamDForgetFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.lamDBot
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamDCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth_of_mem
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth_trans

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

/-- Bundled entailment on `LamDToken`. -/
def LamDEnt (u : Finset (LamDToken A)) (p : LamDToken A) : Prop :=
  LamEntDepth A (lamDForgetFinset A u) p.val
```

```
theorem LamDEnt.bot {u : Finset (LamDToken A)} (hu : LamDCon A u) :
    LamDEnt A u (lamDBot A) :=
  LamEntDepth.bot hu

theorem LamDEnt.of_mem {u : Finset (LamDToken A)} {p : LamDToken A}
    (hu : LamDCon A u) (hp : p  in  u) : LamDEnt A u p :=
  LamEntDepth_of_mem A hu ((mem_lamDForgetFinset A).2 <p, hp, rfl>)

theorem LamDEnt.trans {u v : Finset (LamDToken A)} {c : LamDToken A}
    (hv : LamDCon A v) (hu : LamDCon A u)
    (hEnts : forall  t  in  u, LamDEnt A v t) (ht : LamDEnt A u c) :
    LamDEnt A v c :=
  LamEntDepth_trans A hv hu
    (fun t ht' => by
      rcases (mem_lamDForgetFinset A).1 ht' with <p, hp, rfl>
      exact hEnts p hp)
    ht

end InfoSys

end DomainTheory.InformationSystems
```

lamForgetFinset.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamForgetInsert
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

def lamForgetFinset (u : Finset (LamToken A)) : Finset (RawLamToken a) :=
```

```
  Multiset.foldr (lamForgetInsert (A := A)) {} u.1

end InfoSys

end DomainTheory.InformationSystems
```

LamTokenCon.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamForgetFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

/-- Bundled consistency on `LamToken` finsets. -/
def LamTokenCon (u : Finset (LamToken A)) : Prop :=
  LamCon A (lamForgetFinset A u)

end InfoSys

end DomainTheory.InformationSystems
```

LamTokenEnt.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
```

```
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamForgetFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamEnt

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

/-- Bundled entailment on `LamToken`. -/
def LamTokenEnt (u : Finset (LamToken A)) (p : LamToken A) : Prop :=
  LamEnt A (lamForgetFinset A u) p.val

end InfoSys

end DomainTheory.InformationSystems
```

lamFunFinset_empty.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

theorem lamFunFinset_empty : lamFunFinset ({} : Finset (RawLamToken a)) = {} := rfl
```

```
end InfoSys

end DomainTheory.InformationSystems
```

LamConDepth_empty.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamAtomFinset_empty
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamFunFinset_empty
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem LamConDepth_empty : LamConDepth A ({} : Finset (RawLamToken a)) := by
  have : DecidableEq (Finset (List (RawLamToken a)  x  List (RawLamToken a))) :=
    decidableEq_finset
  rw [LamConDepth.eq_1, dite_eq_left lamFunFinset_empty, lamAtomFinset_empty]
  exact A.con_empty

end InfoSys

end DomainTheory.InformationSystems
```

lamFunFinset_funion.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
```

```
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem lamFunFinset_funion (u v : Finset (RawLamToken a)) :
    lamFunFinset (u  U ' v) = lamFunFinset u  U ' lamFunFinset v := by
  ext p
  constructor
  * intro hp
    have : .funTok p.1 p.2  in  u  U ' v := mem_lamFunFinset.1 hp
    rcases mem_funion.1 this with hu | hv
    * exact mem_funion.2 (Or.inl (mem_lamFunFinset.2 hu))
    * exact mem_funion.2 (Or.inr (mem_lamFunFinset.2 hv))
  * intro hp
    rcases mem_funion.1 hp with hu | hv
    * exact mem_lamFunFinset.2 (mem_funion.2 (Or.inl (mem_lamFunFinset.1 hu)))
    * exact mem_lamFunFinset.2 (mem_funion.2 (Or.inr (mem_lamFunFinset.1 hv)))

end InfoSys

end DomainTheory.InformationSystems
```

lamFunFinset_insert_atom.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive
```

```
namespace InfoSys

variable {a : Type*} [DecidableEq a]

theorem lamFunFinset_insert_atom (u : Finset (RawLamToken a)) (x : a) :
    lamFunFinset (insert (.atom x) u) = lamFunFinset u := by
  ext p; simp only [mem_lamFunFinset, Finset.mem_insert]
  constructor
  * rintro (h | h); exacts [False.elim (nomatch h), h]
  * exact Or.inr

end InfoSys

end DomainTheory.InformationSystems
```

lamFunFinset_insert_bot.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

theorem lamFunFinset_insert_bot (u : Finset (RawLamToken a)) :
    lamFunFinset (insert (.bot : RawLamToken a) u) = lamFunFinset u := by
  ext p; simp only [mem_lamFunFinset, Finset.mem_insert]
  constructor
  * rintro (h | h); exacts [False.elim (nomatch h), h]
  * exact Or.inr

end InfoSys

end DomainTheory.InformationSystems
```

lamFunFinset_insert_funTok.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
```

```
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

theorem lamFunFinset_insert_funTok (u : Finset (RawLamToken a))
    (xs ys : List (RawLamToken a)) :
    lamFunFinset (insert (.funTok xs ys) u) = insert (xs, ys) (lamFunFinset u) := by
  ext p; simp only [Finset.mem_insert, mem_lamFunFinset]
  constructor
  * rintro (h | h)
    * exact Or.inl (Prod.ext (RawLamToken.funTok.inj h).1 (RawLamToken.funTok.inj h)
    .2)
    * exact Or.inr h
  * rintro (rfl | h)
    * exact Or.inl rfl
    * exact Or.inr h

end InfoSys

end DomainTheory.InformationSystems
```

lamFunFinset_mono.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

theorem lamFunFinset_mono {u v : Finset (RawLamToken a)} (h : v  subseteq  u) :
```

```
    lamFunFinset v  subseteq  lamFunFinset u :=
  fun _ hp => mem_lamFunFinset.2 (h (mem_lamFunFinset.1 hp))

end InfoSys

end DomainTheory.InformationSystems
```

lamFunFinset_singleton_funTok.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem lamFunFinset_singleton_funTok (u v : List (RawLamToken a)) :
    lamFunFinset ({.funTok u v} : Finset (RawLamToken a)) = {(u, v)} := by
  ext p
  simp only [mem_lamFunFinset, Finset.mem_singleton]
  constructor
  * intro h
    exact Prod.ext (RawLamToken.funTok.inj h).1 (RawLamToken.funTok.inj h).2
  * intro h
    cases h
    rfl

end InfoSys

end DomainTheory.InformationSystems
```

lamInputUnion_empty.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
```

```
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

theorem lamInputUnion_empty :
    lamInputUnion ({} : Finset (List (RawLamToken a)  x  List (RawLamToken a))) = {}
    := by
  rfl

end InfoSys

end DomainTheory.InformationSystems
```

lamInputUnion_funion.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem lamInputUnion_funion
```

```
    (s t : Finset (List (RawLamToken a)  x  List (RawLamToken a))) :
    lamInputUnion (s  U ' t) = lamInputUnion s  U ' lamInputUnion t := by
  ext x
  constructor
  * intro hx
    rcases (mem_lamInputUnion).1 hx with <p, hp, hx'>
    rcases mem_funion.1 hp with hp | hp
    * exact mem_funion.2 (Or.inl ((mem_lamInputUnion).2 <p, hp, hx'>))
    * exact mem_funion.2 (Or.inr ((mem_lamInputUnion).2 <p, hp, hx'>))
  * intro hx
    rcases mem_funion.1 hx with hx | hx
    * rcases (mem_lamInputUnion).1 hx with <p, hp, hx'>
      exact (mem_lamInputUnion).2 <p, mem_funion.2 (Or.inl hp), hx'>
    * rcases (mem_lamInputUnion).1 hx with <p, hp, hx'>
      exact (mem_lamInputUnion).2 <p, mem_funion.2 (Or.inr hp), hx'>

end InfoSys

end DomainTheory.InformationSystems
```

lamInputUnion_insert.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem lamInputUnion_insert
    (s : Finset (List (RawLamToken a)  x  List (RawLamToken a)))
    (p : List (RawLamToken a)  x  List (RawLamToken a)) :
    lamInputUnion (insert p s) = listToFinset p.1  U ' lamInputUnion s := by
  ext t
  constructor
  * intro ht
```

```
    rcases (mem_lamInputUnion).1 ht with <q, hq, ht'>
    rcases Finset.mem_insert.mp hq with rfl | hq
    * exact mem_funion.2 (Or.inl ht')
    * exact mem_funion.2 (Or.inr ((mem_lamInputUnion).2 <q, hq, ht'>))
  * intro ht
    rcases mem_funion.1 ht with ht | ht
    * exact (mem_lamInputUnion).2 <p, Finset.mem_insert_self p s, ht>
    * rcases (mem_lamInputUnion).1 ht with <q, hq, ht'>
      exact (mem_lamInputUnion).2 <q, Finset.mem_insert_of_mem hq, ht'>

end InfoSys

end DomainTheory.InformationSystems
```

lamInputUnion_singleton.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem lamInputUnion_singleton (p : List (RawLamToken a)  x  List (RawLamToken a)) :
    lamInputUnion ({p} : Finset (List (RawLamToken a)  x  List (RawLamToken a))) =
      listToFinset p.1 := by
  ext t
  change t  in  listToFinset p.1  U ' {}  <->  t  in  listToFinset p.1
  simp only [mem_funion, Finset.notMem_empty, or_false]

end InfoSys

end DomainTheory.InformationSystems
```

lamOutputUnion_empty.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

theorem lamOutputUnion_empty :
    lamOutputUnion ({} : Finset (List (RawLamToken a)  x  List (RawLamToken a))) = {}
    := by
  rfl

end InfoSys

end DomainTheory.InformationSystems
```

lamOutputUnion_funion.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]
```

```
variable (A : InfoSys a)

theorem lamOutputUnion_funion
    (s t : Finset (List (RawLamToken a)  x  List (RawLamToken a))) :
    lamOutputUnion (s  U ' t) = lamOutputUnion s  U ' lamOutputUnion t := by
  ext x
  constructor
  * intro hx
    rcases (mem_lamOutputUnion).1 hx with <p, hp, hx'>
    rcases mem_funion.1 hp with hp | hp
    * exact mem_funion.2 (Or.inl ((mem_lamOutputUnion).2 <p, hp, hx'>))
    * exact mem_funion.2 (Or.inr ((mem_lamOutputUnion).2 <p, hp, hx'>))
  * intro hx
    rcases mem_funion.1 hx with hx | hx
    * rcases (mem_lamOutputUnion).1 hx with <p, hp, hx'>
      exact (mem_lamOutputUnion).2 <p, mem_funion.2 (Or.inl hp), hx'>
    * rcases (mem_lamOutputUnion).1 hx with <p, hp, hx'>
      exact (mem_lamOutputUnion).2 <p, mem_funion.2 (Or.inr hp), hx'>

end InfoSys

end DomainTheory.InformationSystems
```

lamOutputUnion_insert.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem lamOutputUnion_insert
    (s : Finset (List (RawLamToken a)  x  List (RawLamToken a)))
    (p : List (RawLamToken a)  x  List (RawLamToken a)) :
    lamOutputUnion (insert p s) = listToFinset p.2  U ' lamOutputUnion s := by
```

```
  ext t
  constructor
  * intro ht
    rcases (mem_lamOutputUnion).1 ht with <q, hq, ht'>
    rcases Finset.mem_insert.mp hq with rfl | hq
    * exact mem_funion.2 (Or.inl ht')
    * exact mem_funion.2 (Or.inr ((mem_lamOutputUnion).2 <q, hq, ht'>))
  * intro ht
    rcases mem_funion.1 ht with ht | ht
    * exact (mem_lamOutputUnion).2 <p, Finset.mem_insert_self p s, ht>
    * rcases (mem_lamOutputUnion).1 ht with <q, hq, ht'>
      exact (mem_lamOutputUnion).2 <q, Finset.mem_insert_of_mem hq, ht'>

end InfoSys

end DomainTheory.InformationSystems
```

lamOutputUnion_singleton.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem lamOutputUnion_singleton (p : List (RawLamToken a)  x  List (RawLamToken a)) :
    lamOutputUnion ({p} : Finset (List (RawLamToken a)  x  List (RawLamToken a))) =
      listToFinset p.2 := by
  ext t
  change t  in  listToFinset p.2  U ' {}  <->  t  in  listToFinset p.2
  simp only [mem_funion, Finset.notMem_empty, or_false]

end InfoSys

end DomainTheory.InformationSystems
```

listToLamDFinset.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamWF
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamDToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst70
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst74

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

/-- Lift a raw Con set to a finset of depth-WF tokens. -/
def listToLamDFinset :
    (xs : List (RawLamToken a))  ->
      (forall  t  in  xs, LamWF A t)  ->  Finset (LamDToken A)
  | [], _ => {}
  | t :: ts, h =>
      insert <t, h t List.mem_cons_self>
        (listToLamDFinset ts fun w hw => h w (List.mem_cons_of_mem t hw))

end InfoSys

end DomainTheory.InformationSystems
```

mem_foldr_lamDForget.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
```

```
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamDToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst70
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamDForgetInsert
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst74

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem mem_foldr_lamDForget (s : Multiset (LamDToken A)) (t : RawLamToken a) :
    t  in  Multiset.foldr (lamDForgetInsert (A := A)) {} s  <->  exists  p  in  s, p.
    val = t := by
  refine Multiset.induction_on s ?_ ?_
  * constructor
    * intro ht; exact False.elim (Finset.notMem_empty t ht)
    * rintro <_, hp, _>; exact False.elim (by cases hp)
  * intro p rest ih
    simp only [Multiset.foldr_cons, lamDForgetInsert, Finset.mem_insert, ih, Multiset.
    mem_cons]
    constructor
    * rintro (rfl | <q, hq, hq'>)
      * exact <p, Or.inl rfl, rfl>
      * exact <q, Or.inr hq, hq'>
    * rintro <q, hq, hq'>
      rcases hq with rfl | hq
      * exact Or.inl hq'.symm
      * exact Or.inr <q, hq, hq'>

end InfoSys

end DomainTheory.InformationSystems
```

mem_lamAtomFinset.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
```

```
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

theorem mem_lamAtomFinset {u : Finset (RawLamToken a)} {x : a} :
    x  in  lamAtomFinset u  <->  .atom x  in  u := by
  have aux (s : Multiset (RawLamToken a)) :
      x  in  Multiset.foldr lamAtomInsert ({} : Finset a) s  <->  exists  p  in  s, p
    = .atom x := by
    refine Multiset.induction_on s ?_ ?_
    * constructor
      * intro hx; exact False.elim (Finset.notMem_empty x hx)
      * rintro <_, hp, _>; exact False.elim (by cases hp)
    * intro p t ih
      cases p with
      | atom a =>
        simp only [Multiset.foldr_cons, lamAtomInsert, Finset.mem_insert, ih, Multiset
    .mem_cons]
        constructor
        * rintro (hx | <q, hq, hq'>)
          * exact <.atom a, Or.inl rfl, congrArg RawLamToken.atom hx.symm>
          * exact <q, Or.inr hq, hq'>
        * rintro <q, hq, hq'>
          rcases hq with rfl | hq
          * injection hq' with hx; exact Or.inl hx.symm
          * exact Or.inr <q, hq, hq'>
      | bot | funTok _ _ =>
        simp only [Multiset.foldr_cons, lamAtomInsert, id_eq, ih, Multiset.mem_cons]
        constructor
        * rintro <q, hq, hq'>; exact <q, Or.inr hq, hq'>
        * rintro <q, hq, hq'>
          rcases hq with rfl | hq
          * exact False.elim (nomatch hq')
          * exact <q, hq, hq'>
  constructor
  * intro hx
    rcases (aux u.1).1 hx with <p, hp, hp'>
    subst hp'; exact Finset.mem_def.2 hp
  * intro hx
    exact (aux u.1).2 <.atom x, Finset.mem_def.1 hx, rfl>

end InfoSys

end DomainTheory.InformationSystems
```

LamConDepth_singleton_atom.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    mem_lamAtomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConDepth_of_fun_empty

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem LamConDepth_singleton_atom (x : a) :
    LamConDepth A ({.atom x} : Finset (RawLamToken a)) := by
  have : DecidableEq (Finset (List (RawLamToken a)  x  List (RawLamToken a))) :=
    decidableEq_finset
  have hF : lamFunFinset ({.atom x} : Finset (RawLamToken a)) = {} := by
    ext p; constructor
    * intro hp; exact False.elim (nomatch mem_lamFunFinset.1 hp)
    * intro hp; exact False.elim (Finset.notMem_empty p hp)
  exact (LamConDepth_of_fun_empty A hF).2 (by
    have h : lamAtomFinset ({.atom x} : Finset (RawLamToken a)) = {x} := by
      ext y; simp only [mem_lamAtomFinset, Finset.mem_singleton]
      exact <RawLamToken.atom.inj, congrArg RawLamToken.atom>
    rw [h]; exact A.con_sing x)

end InfoSys

end DomainTheory.InformationSystems
```

LamConDepth_singleton_bot.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
```

```
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    mem_lamAtomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConDepth_of_fun_empty

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem LamConDepth_singleton_bot :
    LamConDepth A ({.bot} : Finset (RawLamToken a)) := by
  have : DecidableEq (Finset (List (RawLamToken a)  x  List (RawLamToken a))) :=
    decidableEq_finset
  have hF : lamFunFinset ({.bot} : Finset (RawLamToken a)) = {} := by
    ext p; constructor
    * intro hp; exact False.elim (nomatch mem_lamFunFinset.1 hp)
    * intro hp; exact False.elim (Finset.notMem_empty p hp)
  exact (LamConDepth_of_fun_empty A hF).2 (by
    have h : lamAtomFinset ({.bot} : Finset (RawLamToken a)) = {} := by
      ext x; constructor
      * intro hx; exact False.elim (nomatch mem_lamAtomFinset.1 hx)
      * intro hx; exact False.elim (Finset.notMem_empty x hx)
    rw [h]; exact A.con_empty)

end InfoSys

end DomainTheory.InformationSystems
```

LamEntDepth_of_mem.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
```

```
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    mem_lamAtomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamInputUnion_singleton
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamOutputUnion_singleton
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConDepth_of_fun_empty
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConDepth_of_fun_nonempty

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem LamEntDepth_of_mem {u : Finset (RawLamToken a)} {t : RawLamToken a}
    (hu : LamConDepth A u) (ht : t  in  u) : LamEntDepth A u t := by
  suffices forall  M u t, Nat.max (finsetMaxDepth u) (rawDepth t) = M  ->
      LamConDepth A u  ->  t  in  u  ->  LamEntDepth A u t from
    this _ _ _ rfl hu ht
  intro M
  induction M using Nat.strong_induction_on with
  | h M ih =>
    intro u t hM hu ht
    match t with
    | .bot => exact LamEntDepth_bot A hu
    | .atom x =>
      cases decidableEq_finset (lamFunFinset u) {} with
      | isTrue hF =>
        have hx : x  in  lamAtomFinset u := mem_lamAtomFinset.2 ht
        exact LamEntDepth.atom hu hF (Finset.ne_empty_of_mem hx)
          (A.ent_refl ((LamConDepth_of_fun_empty A hF).1 hu) hx)
      | isFalse hF =>
        have hA := ((LamConDepth_of_fun_nonempty A hF).1 hu).1
        have hx : x  in  lamAtomFinset u := mem_lamAtomFinset.2 ht
        rw [hA] at hx; exact False.elim (Finset.notMem_empty x hx)
    | .funTok xs ys =>
      cases decidableEq_finset (lamFunFinset u) {} with
      | isTrue hF =>
        have : (xs, ys)  in  lamFunFinset u := mem_lamFunFinset.2 ht
        rw [hF] at this; exact False.elim (Finset.notMem_empty _ this)
      | isFalse hF =>
        have hu' := (LamConDepth_of_fun_nonempty A hF).1 hu
```

```
        rcases hu' with <hA, hWF, _hFun>
        have hmem : (xs, ys)  in  lamFunFinset u := mem_lamFunFinset.2 ht
        have <hConIn, hConOut> := hWF (xs, ys) hmem
        refine LamEntDepth.funTok (s := {(xs, ys)}) hu hA hConIn hConOut
          (fun p hp => by
            have hp' : p = (xs, ys) := Finset.mem_singleton.mp hp
            exact hp'  |>  hmem) hF ?_ ?_ ?_
        * intro hxsF
          have hIn : lamInputUnion {(xs, ys)} = listToFinset xs := by
            rw [lamInputUnion_singleton]
          rw [hIn, hxsF]
        * intro p hp z hz
          have hp' : p = (xs, ys) := Finset.mem_singleton.mp hp
          subst hp'
          refine ih (Nat.max (finsetMaxDepth (listToFinset xs)) (rawDepth z))
            ?_ (listToFinset xs) z rfl hConIn hz
          have htok : .funTok xs ys  in  u := mem_lamFunFinset.1 hmem
          have ht_lt : rawDepth z < finsetMaxDepth u :=
            Nat.lt_of_lt_of_le (rawDepth_funTok_gt_left ((mem_listToFinset).1 hz))
              (le_finsetMaxDepth_of_mem htok)
          have hxs_lt : finsetMaxDepth (listToFinset xs) < rawDepth (.funTok xs ys) :=
            finsetMaxDepth.list_lt_funTok xs ys
          rw [ <-  hM]
          refine Nat.max_lt.2 <
            Nat.lt_of_lt_of_le hxs_lt (Nat.le_max_right _ _),
            Nat.lt_of_lt_of_le ht_lt (Nat.le_max_left _ _)>
        * intro z hz
          rw [lamOutputUnion_singleton]
          refine ih (Nat.max (finsetMaxDepth (listToFinset ys)) (rawDepth z))
            ?_ (listToFinset ys) z rfl hConOut hz
          have ht_lt : rawDepth z < rawDepth (.funTok xs ys) :=
            rawDepth_funTok_gt_right ((mem_listToFinset).1 hz)
          have hys_lt : finsetMaxDepth (listToFinset ys) < rawDepth (.funTok xs ys) :=
            finsetMaxDepth.list_lt_funTok_out xs ys
          rw [ <-  hM]
          refine Nat.max_lt.2 <
            Nat.lt_of_lt_of_le hys_lt (Nat.le_max_right _ _),
            Nat.lt_of_lt_of_le ht_lt (Nat.le_max_right _ _)>

end InfoSys

end DomainTheory.InformationSystems
```

lamAtomFinset_insert_atom.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
```

```
    mem_lamAtomFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

theorem lamAtomFinset_insert_atom (u : Finset (RawLamToken a)) (x : a) :
    lamAtomFinset (insert (.atom x) u) = insert x (lamAtomFinset u) := by
  ext y; simp only [Finset.mem_insert, mem_lamAtomFinset]
  constructor
  * rintro (h | h)
    * exact Or.inl (RawLamToken.atom.inj h)
    * exact Or.inr h
  * rintro (rfl | h)
    * exact Or.inl rfl
    * exact Or.inr h

end InfoSys

end DomainTheory.InformationSystems
```

LamConDepth_insert_atom.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamAtomFinset_insert_atom
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamFunFinset_insert_atom
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConDepth_of_fun_empty

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive
```

```
namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

/-- Inserting an entailed atom preserves depth-Con. -/
theorem LamConDepth_insert_atom {u : Finset (RawLamToken a)} {x : a}
    (ht : LamEntDepth A u (.atom x)) :
    LamConDepth A (insert (.atom x) u) := by
  have : DecidableEq (Finset (List (RawLamToken a)  x  List (RawLamToken a))) :=
    decidableEq_finset
  cases ht with
  | atom hu hF _hne hEnt =>
    have hF' : lamFunFinset (insert (.atom x) u) = {} := by
      rw [lamFunFinset_insert_atom]; exact hF
    exact (LamConDepth_of_fun_empty A hF').2 (by
      rw [lamAtomFinset_insert_atom]
      exact A.ent_con hEnt)

end InfoSys

end DomainTheory.InformationSystems
```

lamAtomFinset_insert_bot.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    mem_lamAtomFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

theorem lamAtomFinset_insert_bot (u : Finset (RawLamToken a)) :
    lamAtomFinset (insert (.bot : RawLamToken a) u) = lamAtomFinset u := by
  ext x; simp only [mem_lamAtomFinset, Finset.mem_insert]
  constructor
  * rintro (h | h); exacts [False.elim (nomatch h), h]
  * exact Or.inr
```

```
end InfoSys

end DomainTheory.InformationSystems
```

LamConDepth_insert_bot.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamAtomFinset_insert_bot
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamFunFinset_insert_bot
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConDepth_of_fun_empty
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConDepth_of_fun_nonempty

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

/-- Inserting bot preserves depth-Con. -/
theorem LamConDepth_insert_bot {u : Finset (RawLamToken a)}
    (hu : LamConDepth A u) :
    LamConDepth A (insert (.bot : RawLamToken a) u) := by
  have : DecidableEq (Finset (List (RawLamToken a)  x  List (RawLamToken a))) :=
    decidableEq_finset
  by_cases hF : lamFunFinset u = {}
  * have hF' : lamFunFinset (insert (.bot : RawLamToken a) u) = {} := by
      rw [lamFunFinset_insert_bot]; exact hF
    exact (LamConDepth_of_fun_empty A hF').2 (by
      rw [lamAtomFinset_insert_bot]
      exact (LamConDepth_of_fun_empty A hF).1 hu)
  * have hu' := (LamConDepth_of_fun_nonempty A hF).1 hu
    rcases hu' with <hA, hWF, hFun>
    have hF' : lamFunFinset (insert (.bot : RawLamToken a) u)  !=  {} := by
```

```
      rw [lamFunFinset_insert_bot]; exact hF
    refine (LamConDepth_of_fun_nonempty A hF').2 <?_, ?_, ?_>
    * rw [lamAtomFinset_insert_bot]; exact hA
    * intro p hp
      rw [lamFunFinset_insert_bot] at hp
      exact hWF p hp
    * intro s hs hIn
      refine hFun s (by rw [ <-  lamFunFinset_insert_bot u]; exact hs) hIn

end InfoSys

end DomainTheory.InformationSystems
```

lamAtomFinset_insert_funTok.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    mem_lamAtomFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

theorem lamAtomFinset_insert_funTok (u : Finset (RawLamToken a))
    (xs ys : List (RawLamToken a)) :
    lamAtomFinset (insert (.funTok xs ys) u) = lamAtomFinset u := by
  ext x; simp only [mem_lamAtomFinset, Finset.mem_insert]
  constructor
  * rintro (h | h); exacts [False.elim (nomatch h), h]
  * exact Or.inr

end InfoSys

end DomainTheory.InformationSystems
```

lamAtomFinset_mono.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
```

```
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    mem_lamAtomFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

theorem lamAtomFinset_mono {u v : Finset (RawLamToken a)} (h : v  subseteq  u) :
    lamAtomFinset v  subseteq  lamAtomFinset u :=
  fun _ hx => mem_lamAtomFinset.2 (h (mem_lamAtomFinset.1 hx))

end InfoSys

end DomainTheory.InformationSystems
```

LamConDepth_mono.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamAtomFinset_mono
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamFunFinset_mono
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConDepth_of_fun_empty
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConDepth_of_fun_nonempty

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive
```

```
namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem LamConDepth_mono {u v : Finset (RawLamToken a)}
    (huv : u  subseteq  v) (hv : LamConDepth A v) : LamConDepth A u := by
  have : DecidableEq (Finset (List (RawLamToken a)  x  List (RawLamToken a))) :=
    decidableEq_finset
  have hFunMono := lamFunFinset_mono huv
  have hAtomMono := lamAtomFinset_mono huv
  by_cases hFv : lamFunFinset v = {}
  * have hFu : lamFunFinset u = {} :=
      Finset.Subset.antisymm
        (fun p hp => by
          have := hFunMono hp
          rw [hFv] at this; exact False.elim (Finset.notMem_empty p this))
        (Finset.empty_subset _)
    exact (LamConDepth_of_fun_empty A hFu).2
      (A.con_subset ((LamConDepth_of_fun_empty A hFv).1 hv) hAtomMono)
  * have hv' := (LamConDepth_of_fun_nonempty A hFv).1 hv
    rcases hv' with <hAv, hWF, hFun>
    by_cases hFu : lamFunFinset u = {}
    * have hAu : lamAtomFinset u = {} :=
        Finset.Subset.antisymm
          (fun x hx => by
            have := hAtomMono hx
            rw [hAv] at this; exact False.elim (Finset.notMem_empty x this))
          (Finset.empty_subset _)
      exact (LamConDepth_of_fun_empty A hFu).2 (by rw [hAu]; exact A.con_empty)
    * refine (LamConDepth_of_fun_nonempty A hFu).2 <?_, ?_, ?_>
      * exact Finset.Subset.antisymm
          (fun x hx => by
            have := hAtomMono hx
            rw [hAv] at this; exact False.elim (Finset.notMem_empty x this))
          (Finset.empty_subset _)
      * intro p hp; exact hWF p (hFunMono hp)
      * intro s hs hIn
        exact hFun s (fun p hp => hFunMono (hs hp)) hIn

end InfoSys

end DomainTheory.InformationSystems
```

LamEntDepth_weaken_atom.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
```

```
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamAtomFinset_mono
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConDepth_of_fun_empty

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

/-- Weaken atom-entailment when the larger set stays in the atom world (`fun = {}`). -/
theorem LamEntDepth_weaken_atom {u v : Finset (RawLamToken a)} {x : a}
    (huv : u  subseteq  v) (hv : LamConDepth A v) (hFv : lamFunFinset v = {})
    (ht : LamEntDepth A u (.atom x)) : LamEntDepth A v (.atom x) := by
  cases ht with
  | atom hu hF hne hEnt =>
    have hsubA := lamAtomFinset_mono huv
    have hne' : lamAtomFinset v  !=  {} := by
      intro hAv
      obtain <y, hy> := Finset.nonempty_of_ne_empty hne
      exact Finset.notMem_empty y (by have := hsubA hy; rwa [hAv] at this)
    exact LamEntDepth.atom hv hFv hne'
      (A.ent_trans ((LamConDepth_of_fun_empty A hFv).1 hv)
        ((LamConDepth_of_fun_empty A hF).1 hu)
        (fun y hy => A.ent_refl ((LamConDepth_of_fun_empty A hFv).1 hv) (hsubA hy))
        hEnt)

end InfoSys

end DomainTheory.InformationSystems
```

LamEntDepth_weaken_fun.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
```

```
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamAtomFinset_mono
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamFunFinset_mono
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

/-- Weaken entailment when the larger set stays in the function world (`atom = {}`). -/
theorem LamEntDepth_weaken_fun {u v : Finset (RawLamToken a)} {t : RawLamToken a}
    (huv : u  subseteq  v) (hv : LamConDepth A v) (hAv : lamAtomFinset v = {})
    (ht : LamEntDepth A u t) : LamEntDepth A v t := by
  cases ht with
  | bot _ => exact LamEntDepth.bot hv
  | atom _ _ hne _ =>
    obtain <x, hx> := Finset.nonempty_of_ne_empty hne
    exact False.elim (Finset.notMem_empty x (by
      have := lamAtomFinset_mono huv hx; rwa [hAv] at this))
  | funTok _ _ hConIn hConOut hs hWorld hInCompat hEntIn hEntOut =>
    exact LamEntDepth.funTok hv hAv hConIn hConOut
      (fun p hp => lamFunFinset_mono huv (hs hp))
      (fun hFv => hWorld (Finset.Subset.antisymm
          (fun p hp => by
            have := lamFunFinset_mono huv hp
            rw [hFv] at this; exact False.elim (Finset.notMem_empty p this))
          (Finset.empty_subset _)))
      hInCompat hEntIn hEntOut

end InfoSys

end DomainTheory.InformationSystems
```

LamEntDepth_weaken_insert.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
```

```
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamAtomFinset_mono
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamAtomFinset_insert_bot
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamFunFinset_insert_bot
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamFunFinset_insert_funTok
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConDepth_of_fun_nonempty
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth_weaken_fun
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth_weaken_atom

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

/-- Weaken Ent across inserting another token entailed by the same set. -/
theorem LamEntDepth_weaken_insert {u : Finset (RawLamToken a)}
    {a x : RawLamToken a}
    (hu' : LamConDepth A (insert a u))
    (hx : LamEntDepth A u x) (ha : LamEntDepth A u a) :
    LamEntDepth A (insert a u) x := by
  have : DecidableEq (Finset (List (RawLamToken a)  x  List (RawLamToken a))) :=
    decidableEq_finset
  have huv : u  subseteq  insert a u := Finset.subset_insert a u
  match x with
  | .bot => exact LamEntDepth.bot hu'
  | .atom z =>
    cases decidableEq_finset (lamFunFinset (insert a u)) {} with
    | isTrue hF =>
      exact LamEntDepth_weaken_atom A huv hu' hF hx
    | isFalse hF =>
      have hAt := ((LamConDepth_of_fun_nonempty A hF).1 hu').1
      cases hx with
      | atom _ _ hne _ =>
        obtain <y, hy> := Finset.nonempty_of_ne_empty hne
        exact False.elim (Finset.notMem_empty y (by
          have := lamAtomFinset_mono huv hy; rwa [hAt] at this))
  | .funTok xs ys =>
    cases decidableEq_finset (lamFunFinset (insert a u)) {} with
```

```
    | isFalse hF =>
      have hAv := ((LamConDepth_of_fun_nonempty A hF).1 hu').1
      exact LamEntDepth_weaken_fun A huv hu' hAv hx
    | isTrue hF =>
      cases ha with
      | bot _ =>
        cases hx with
        | funTok hu hA hConIn hConOut hs hWorld hInCompat hEntIn hEntOut =>
          have hAv : lamAtomFinset (insert (.bot : RawLamToken a) u) = {} := by
            rw [lamAtomFinset_insert_bot]; exact hA
          exact LamEntDepth.funTok hu' hAv hConIn hConOut
            (by rw [lamFunFinset_insert_bot]; exact hs)
            (by rw [lamFunFinset_insert_bot]; exact hWorld)
            hInCompat hEntIn hEntOut
      | atom _ _ hne _ =>
        cases hx with
        | funTok _ hA _ _ _ _ _ _ _ =>
          obtain <y, hy> := Finset.nonempty_of_ne_empty hne
          exact False.elim (Finset.notMem_empty y (by rwa [hA] at hy))
      | funTok _ _ _ _ _ _ _ _ _ =>
        -- `a` refined to funTok, so fun (insert a u)  !=  {}, contradicting hF
        simp only [lamFunFinset_insert_funTok] at hF
        exact False.elim (Finset.notMem_empty _ (hF  |>  Finset.mem_insert_self _ _))

end InfoSys

end DomainTheory.InformationSystems
```

LamConDepth_funion_of_entSet.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    funion_empty_right_lam
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth_weaken_insert
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    finsetMaxDepth_insert_le

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive
```

```
namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

/-- Prop 2.3(ii) via insert-first on `v`. -/
theorem LamConDepth_funion_of_entSet (M : Nat)
    (ihIns : forall  u t, Nat.max (finsetMaxDepth u) (rawDepth t) < M  ->
      LamEntDepth A u t  ->  LamConDepth A (insert t u))
    {u v : Finset (RawLamToken a)}
    (hu : LamConDepth A u)
    (hEnt : forall  t  in  v, LamEntDepth A u t)
    (hbound : forall  t  in  v, Nat.max (finsetMaxDepth u) (rawDepth t) < M) :
    LamConDepth A (u  U ' v) := by
  have : DecidableEq (Finset (List (RawLamToken a)  x  List (RawLamToken a))) :=
    decidableEq_finset
  suffices forall  (v u : Finset (RawLamToken a)),
      LamConDepth A u  ->
      (forall  t  in  v, LamEntDepth A u t)  ->
      (forall  t  in  v, Nat.max (finsetMaxDepth u) (rawDepth t) < M)  ->
      LamConDepth A (u  U ' v) from
    this v u hu hEnt hbound
  intro v
  induction v using Finset.induction_on with
  | empty =>
    intro u hu _ _
    simpa [funion_empty_right_lam] using hu
  | insert a t ha ih =>
    intro u hu hEnt hbound
    have mem_a : a  in  insert a t := Finset.mem_insert_self a t
    have han : Nat.max (finsetMaxDepth u) (rawDepth a) < M := hbound a mem_a
    have hEnt_a : LamEntDepth A u a := hEnt a mem_a
    have hu_ins : LamConDepth A (insert a u) := ihIns u a han hEnt_a
    have hEnt_t : forall  x  in  t, LamEntDepth A (insert a u) x := by
      intro x hx
      exact LamEntDepth_weaken_insert A hu_ins
        (hEnt x (Finset.mem_insert_of_mem hx)) hEnt_a
    have hbound_t : forall  x  in  t, Nat.max (finsetMaxDepth (insert a u)) (rawDepth
    x) < M := by
      intro x hx
      have hxu : Nat.max (finsetMaxDepth u) (rawDepth x) < M :=
        hbound x (Finset.mem_insert_of_mem hx)
      have hDi := finsetMaxDepth_insert_le u a
      have h1 : Nat.max (finsetMaxDepth (insert a u)) (rawDepth x)  <=
          Nat.max (Nat.max (rawDepth a) (finsetMaxDepth u)) (rawDepth x) :=
        max_le_max_nat hDi le_rfl
      refine Nat.lt_of_le_of_lt h1 ?_
      have h2 : Nat.max (Nat.max (rawDepth a) (finsetMaxDepth u)) (rawDepth x) =
          Nat.max (rawDepth a) (Nat.max (finsetMaxDepth u) (rawDepth x)) := by
        ac_rfl
      rw [h2]
      exact Nat.max_lt.2 <Nat.lt_of_le_of_lt (Nat.le_max_right _ _) han, hxu>
    have hunion : LamConDepth A (insert a u  U ' t) :=
      ih (insert a u) hu_ins hEnt_t hbound_t
```

```
    have heq : insert a u  U ' t = u  U ' insert a t := by
      ext x
      constructor
      * intro hx
        rcases mem_funion.1 hx with h | h
        * rcases Finset.mem_insert.mp h with rfl | hu'
          * exact mem_funion.2 (Or.inr (Finset.mem_insert_self _ _))
          * exact mem_funion.2 (Or.inl hu')
        * exact mem_funion.2 (Or.inr (Finset.mem_insert_of_mem h))
      * intro hx
        rcases mem_funion.1 hx with hu' | h
        * exact mem_funion.2 (Or.inl (Finset.mem_insert_of_mem hu'))
        * rcases Finset.mem_insert.mp h with rfl | ht'
          * exact mem_funion.2 (Or.inl (Finset.mem_insert_self _ _))
          * exact mem_funion.2 (Or.inr ht')
    rwa [heq] at hunion

end InfoSys

end DomainTheory.InformationSystems
```

LamConDepth_insert_of_ent.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamAtomFinset_insert_funTok
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamFunFinset_insert_funTok
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamInputUnion_empty
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamOutputUnion_empty
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamInputUnion_insert
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamOutputUnion_insert
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamInputUnion_funion
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamOutputUnion_funion
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConDepth_of_fun_nonempty
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
```

```
    LamConDepth_mono
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConDepth_insert_bot
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConDepth_insert_atom
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth_weaken_fun
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth_weaken_atom
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    insert_filter_ne_lam
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    filter_ne_subset_of_subset_insert_lam
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamFunFinset_funion
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConDepth_funion_of_entSet
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    funion_assoc_lam
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    funion_comm_lam
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    finsetMaxDepth_funion_le

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

/-- Prop 2.1(iii): inserting an entailed token preserves depth-Con. -/
theorem LamConDepth_insert_of_ent {u : Finset (RawLamToken a)} {t : RawLamToken a}
    (ht : LamEntDepth A u t) : LamConDepth A (insert t u) := by
  have : DecidableEq (Finset (List (RawLamToken a)  x  List (RawLamToken a))) :=
    decidableEq_finset
  suffices forall  M u t, Nat.max (finsetMaxDepth u) (rawDepth t) = M  ->
      LamEntDepth A u t  ->  LamConDepth A (insert t u) from
    this _ _ _ rfl ht
  intro M
  induction M using Nat.strong_induction_on with
  | h M ih =>
    intro u t hM ht
    have ihIns : forall  u' t', Nat.max (finsetMaxDepth u') (rawDepth t') < M  ->
        LamEntDepth A u' t'  ->  LamConDepth A (insert t' u') :=
      fun u' t' hlt hEnt => ih _ hlt u' t' rfl hEnt
    match t with
    | .bot => exact LamConDepth_insert_bot A (LamEntDepth.con A ht)
    | .atom x => exact LamConDepth_insert_atom A ht
    | .funTok xs ys =>
      cases ht with
```

```
  | @funTok u xs ys s hu hA hConIn hConOut hs hWorld hInCompat hEntIn hEntOut =>
    have hF' : lamFunFinset (insert (.funTok xs ys) u)  !=  {} := by
      rw [lamFunFinset_insert_funTok]
      exact Finset.insert_ne_empty _ _
    refine (LamConDepth_of_fun_nonempty A hF').2 <?_, ?_, ?_>
    * rw [lamAtomFinset_insert_funTok]; exact hA
    * intro p hp
      rw [lamFunFinset_insert_funTok] at hp
      rcases Finset.mem_insert.mp hp with rfl | hp
      * exact <hConIn, hConOut>
      * exact ((LamConDepth_of_fun_nonempty A hWorld).1 hu).2.1 p hp
    * intro sel hsel hIn
      if hp : (xs, ys)  in  sel then
        let t' := sel.filter (*  !=  (xs, ys))
        have ht_eq : insert (xs, ys) t' = sel := insert_filter_ne_lam hp
        have ht'sub : t'  subseteq  lamFunFinset u :=
          filter_ne_subset_of_subset_insert_lam (by
            intro q hq; have := hsel hq; rwa [lamFunFinset_insert_funTok] at this)
        have hin' : LamConDepth A (listToFinset xs  U ' lamInputUnion t') := by
          rwa [ <-  ht_eq, lamInputUnion_insert] at hIn
        have hsub : s  U ' t'  subseteq  lamFunFinset u :=
          funion_subset_iff.mpr <hs, ht'sub>
        have hEnt_in_s : forall  z  in  lamInputUnion s,
            LamEntDepth A (listToFinset xs) z := by
          intro z hz
          rcases (mem_lamInputUnion).1 hz with <p, hp', hz'>
          exact hEntIn p hp' z hz'
        have hEnt_big : forall  z  in  lamInputUnion s,
            LamEntDepth A (listToFinset xs  U ' lamInputUnion t') z := by
          intro z hz
          have hzx := hEnt_in_s z hz
          have hsubxs : listToFinset xs  subseteq  listToFinset xs  U '
lamInputUnion t' :=
            subset_funion_left _ _
          cases decidableEq_finset
              (lamFunFinset (listToFinset xs  U ' lamInputUnion t')) {} with
          | isFalse hFn =>
            have hAv := ((LamConDepth_of_fun_nonempty A hFn).1 hin').1
            exact LamEntDepth_weaken_fun A hsubxs hin' hAv hzx
          | isTrue hFhin =>
            match z with
            | .bot => exact LamEntDepth.bot hin'
            | .atom x =>
              exact LamEntDepth_weaken_atom A hsubxs hin' hFhin hzx
            | .funTok xs1 ys1 =>
              cases decidableEq_finset (lamFunFinset (listToFinset xs)) {} with
              | isTrue hxsF =>
                have hNoFun : lamFunFinset (lamInputUnion s) = {} := hInCompat
hxsF
                have hzmem : .funTok xs1 ys1  in  lamInputUnion s := hz
                have : (xs1, ys1)  in  lamFunFinset (lamInputUnion s) :=
                  mem_lamFunFinset.2 hzmem
                rw [hNoFun] at this
                exact False.elim (Finset.notMem_empty _ this)
              | isFalse hxsF =>
                have : lamFunFinset (listToFinset xs  U ' lamInputUnion t')  !=
```

```
{} := by
                  intro h
                  have : lamFunFinset (listToFinset xs) = {} :=
                    Finset.Subset.antisymm
                      (fun p hp => by
                        have hp' : p  in  lamFunFinset
                            (listToFinset xs  U ' lamInputUnion t') := by
                          rw [lamFunFinset_funion]
                          exact mem_funion.2 (Or.inl hp)
                        rw [h] at hp'; exact False.elim (Finset.notMem_empty p hp
'))
                      (Finset.empty_subset _)
                  exact hxsF this
                exact False.elim (this hFhin)
          have hDt : rawDepth (.funTok xs ys)  <=  M := by
            rw [ <-  hM]; exact Nat.le_max_right _ _
          have hDu : finsetMaxDepth u  <=  M := by
            rw [ <-  hM]; exact Nat.le_max_left _ _
          have hxs_lt : finsetMaxDepth (listToFinset xs) < M :=
            Nat.lt_of_lt_of_le (finsetMaxDepth.list_lt_funTok xs ys) hDt
          have hys_lt : finsetMaxDepth (listToFinset ys) < M :=
            Nat.lt_of_lt_of_le (finsetMaxDepth.list_lt_funTok_out xs ys) hDt
          have bound_in : forall  z  in  lamInputUnion s,
              Nat.max (finsetMaxDepth (listToFinset xs  U ' lamInputUnion t'))
                (rawDepth z) < M := by
            intro z hz
            have hDuBig : finsetMaxDepth (listToFinset xs  U ' lamInputUnion t') < M
 := by
              have hle := finsetMaxDepth_funion_le (listToFinset xs) (lamInputUnion
t')
              refine Nat.lt_of_le_of_lt hle (Nat.max_lt.2 <hxs_lt, ?_>)
              by_cases ht'e : t' = {}
              * rw [ht'e, lamInputUnion_empty]
                exact Nat.lt_of_le_of_lt (Nat.zero_le _) hxs_lt
              * obtain <q, hq> := Finset.nonempty_of_ne_empty ht'e
                exact Nat.lt_of_lt_of_le
                  (finsetMaxDepth.inputUnion_lt ht'sub hWorld) hDu
            have hz_lt : rawDepth z < M := by
              rcases (mem_lamInputUnion).1 hz with <p, hp', hz'>
              have htok : .funTok p.1 p.2  in  u := mem_lamFunFinset.1 (hs hp')
              exact Nat.lt_of_lt_of_le
                (Nat.lt_of_lt_of_le
                  (rawDepth_funTok_gt_left ((mem_listToFinset).1 hz'))
                  (le_finsetMaxDepth_of_mem htok))
                hDu
            exact Nat.max_lt.2 <hDuBig, hz_lt>
          have hbig : LamConDepth A
              ((listToFinset xs  U ' lamInputUnion t')  U ' lamInputUnion s) :=
            LamConDepth_funion_of_entSet A M ihIns hin' hEnt_big bound_in
          have hin_st' : LamConDepth A (lamInputUnion s  U ' lamInputUnion t') := by
            have heq :
                (listToFinset xs  U ' lamInputUnion t')  U ' lamInputUnion s =
                  listToFinset xs  U ' (lamInputUnion s  U ' lamInputUnion t') := by
              rw [funion_assoc_lam, funion_comm_lam (lamInputUnion t')]
            have hbig' : LamConDepth A
                (listToFinset xs  U ' (lamInputUnion s  U ' lamInputUnion t')) := by
```

```
            rwa [heq] at hbig
          exact LamConDepth_mono A (subset_funion_right _ _) hbig'
        have hin_union : LamConDepth A (lamInputUnion (s  U ' t')) := by
          rwa [lamInputUnion_funion]
        have hout_union : LamConDepth A
            (lamOutputUnion s  U ' lamOutputUnion t') := by
          have hout : LamConDepth A (lamOutputUnion (s  U ' t')) :=
            ((LamConDepth_of_fun_nonempty A hWorld).1 hu).2.2 (s  U ' t') hsub
hin_union
          rwa [lamOutputUnion_funion] at hout
        have hEnt_out' : forall  z  in  listToFinset ys,
            LamEntDepth A (lamOutputUnion s  U ' lamOutputUnion t') z := by
          intro z hz
          have hzx := hEntOut z hz
          have hsubO : lamOutputUnion s  subseteq
              lamOutputUnion s  U ' lamOutputUnion t' := subset_funion_left _ _
          cases decidableEq_finset
              (lamFunFinset (lamOutputUnion s  U ' lamOutputUnion t')) {} with
          | isFalse hFn =>
            have hAv := ((LamConDepth_of_fun_nonempty A hFn).1 hout_union).1
            exact LamEntDepth_weaken_fun A hsubO hout_union hAv hzx
          | isTrue hFo =>
            match z with
            | .bot => exact LamEntDepth.bot hout_union
            | .atom x =>
              exact LamEntDepth_weaken_atom A hsubO hout_union hFo hzx
            | .funTok xs1 ys1 =>
              -- Ent out_s funTok needs fun out_s  !=  {}, but out_s  subseteq
atom-world union
              cases hzx with
              | funTok _ _ _ _ hsO hWo _ _ _ =>
                have hOsF : lamFunFinset (lamOutputUnion s) = {} :=
                  Finset.Subset.antisymm
                    (fun p hp => by
                      have : p  in  lamFunFinset
                          (lamOutputUnion s  U ' lamOutputUnion t') := by
                        rw [lamFunFinset_funion]
                        exact mem_funion.2 (Or.inl hp)
                      rw [hFo] at this
                      exact False.elim (Finset.notMem_empty p this))
                    (Finset.empty_subset _)
                exact False.elim (hWo hOsF)
        have bound_out : forall  z  in  listToFinset ys,
            Nat.max (finsetMaxDepth (lamOutputUnion s  U ' lamOutputUnion t'))
              (rawDepth z) < M := by
          intro z hz
          have hDuOut : finsetMaxDepth
              (lamOutputUnion s  U ' lamOutputUnion t') < M := by
            have hle := finsetMaxDepth_funion_le (lamOutputUnion s) (
lamOutputUnion t')
            refine Nat.lt_of_le_of_lt hle (Nat.max_lt.2 <?_, ?_>)
            * by_cases hs0 : s = {}
              * rw [hs0, lamOutputUnion_empty]
                exact Nat.lt_of_le_of_lt (Nat.zero_le _) hys_lt
              * exact Nat.lt_of_lt_of_le
                  (finsetMaxDepth.outputUnion_lt hs hWorld) hDu
```

```
                * by_cases ht'e : t' = {}
                  * rw [ht'e, lamOutputUnion_empty]
                    exact Nat.lt_of_le_of_lt (Nat.zero_le _) hys_lt
                  * exact Nat.lt_of_lt_of_le
                      (finsetMaxDepth.outputUnion_lt ht'sub hWorld) hDu
              have hz_lt : rawDepth z < M :=
                Nat.lt_of_lt_of_le
                  (rawDepth_funTok_gt_right ((mem_listToFinset).1 hz)) hDt
              exact Nat.max_lt.2 <hDuOut, hz_lt>
            have hfinal : LamConDepth A
                ((lamOutputUnion s  U ' lamOutputUnion t')  U ' listToFinset ys) :=
              LamConDepth_funion_of_entSet A M ihIns hout_union hEnt_out' bound_out
            have hout_t : LamConDepth A
                (listToFinset ys  U ' lamOutputUnion t') := by
              have heq :
                  (lamOutputUnion s  U ' lamOutputUnion t')  U ' listToFinset ys =
                    lamOutputUnion s  U ' (listToFinset ys  U ' lamOutputUnion t') :=
    by
                rw [funion_assoc_lam, funion_comm_lam (lamOutputUnion t')]
              have hfinal' : LamConDepth A
                  (lamOutputUnion s  U ' (listToFinset ys  U ' lamOutputUnion t')) :=
    by
                rwa [heq] at hfinal
              exact LamConDepth_mono A (subset_funion_right _ _) hfinal'
            show LamConDepth A (lamOutputUnion sel)
            rwa [ <-  ht_eq, lamOutputUnion_insert]
          else
            have hsel' : sel  subseteq  lamFunFinset u := by
              intro p hp'
              have := hsel hp'
              rw [lamFunFinset_insert_funTok] at this
              rcases Finset.mem_insert.mp this with rfl | h
              * exact False.elim (hp hp')
              * exact h
            exact ((LamConDepth_of_fun_nonempty A hWorld).1 hu).2.2 sel hsel' hIn

end InfoSys

end DomainTheory.InformationSystems
```

LamEntDepth_trans.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    mem_lamAtomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamOutputUnion_empty
```

```
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamOutputUnion_insert
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamOutputUnion_funion
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConDepth_empty
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConDepth_of_fun_empty
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConDepth_of_fun_nonempty
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConDepth_mono
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth_weaken_fun
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth_weaken_atom
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamFunFinset_funion
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConDepth_funion_of_entSet
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConDepth_insert_of_ent

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

/-- Def 2.1 `ent_trans`: cases on the RHS token (Scott?s turnstile advice). -/
theorem LamEntDepth_trans {u v : Finset (RawLamToken a)} {c : RawLamToken a}
    (hv : LamConDepth A v) (hu : LamConDepth A u)
    (hEnts : forall  t  in  u, LamEntDepth A v t) (ht : LamEntDepth A u c) :
    LamEntDepth A v c := by
  have : DecidableEq (Finset (List (RawLamToken a)  x  List (RawLamToken a))) :=
    decidableEq_finset
  suffices forall  M u v c,
      Nat.max (finsetMaxDepth v) (Nat.max (finsetMaxDepth u) (rawDepth c)) = M  ->
      LamConDepth A v  ->  LamConDepth A u  ->
      (forall  t  in  u, LamEntDepth A v t)  ->  LamEntDepth A u c  ->  LamEntDepth A
    v c from
    this _ u v c rfl hv hu hEnts ht
  intro M
  induction M using Nat.strong_induction_on with
  | h M ih =>
    intro u v c hM hv hu hEnts ht
```

```
have hleV : finsetMaxDepth v  <=  M := by rw [ <-  hM]; exact Nat.le_max_left _ _
have hleU : finsetMaxDepth u  <=  M := by
  rw [ <-  hM]; exact Nat.le_trans (Nat.le_max_left _ _) (Nat.le_max_right _ _)
have hleC : rawDepth c  <=  M := by
  rw [ <-  hM]; exact Nat.le_trans (Nat.le_max_right _ _) (Nat.le_max_right _ _)
-- (8) bot
match c with
| .bot => exact LamEntDepth.bot hv
-- (10) atom
| .atom x =>
  cases ht with
  | @atom u x hu' hF hne hEnt =>
    obtain <y0, hy0> := Finset.nonempty_of_ne_empty hne
    have hEy0 := hEnts _ (mem_lamAtomFinset.1 hy0)
    have hFv : lamFunFinset v = {} := by
      cases hEy0 with
      | atom _ hFv' _ _ => exact hFv'
    have hneV : lamAtomFinset v  !=  {} := by
      cases hEy0 with
      | atom _ _ hneV _ => exact hneV
    have hAtomEnt : forall  y  in  lamAtomFinset u, A.Ent (lamAtomFinset v) y :=
by
      intro y hy
      have hEy := hEnts _ (mem_lamAtomFinset.1 hy)
      cases hEy with
      | atom _ _ _ hE => exact hE
    exact LamEntDepth.atom hv hFv hneV
      (A.ent_trans ((LamConDepth_of_fun_empty A hFv).1 hv)
        ((LamConDepth_of_fun_empty A hF).1 hu') hAtomEnt hEnt)
-- (11) funTok
| .funTok xs ys =>
  cases ht with
  | @funTok u xs ys s hu' hA hConIn hConOut hs hWorld hInCompat hEntIn hEntOut =>
    obtain <p0, hp0> := Finset.nonempty_of_ne_empty hWorld
    have hEp0 := hEnts _ (mem_lamFunFinset.1 hp0)
    have hAv : lamAtomFinset v = {} := by
      cases hEp0 with
      | funTok _ hAv' _ _ _ _ _ _ _ => exact hAv'
    have hFv : lamFunFinset v  !=  {} := by
      cases hEp0 with
      | funTok _ _ _ _ _ hFv' _ _ _ => exact hFv'
    have hWit : forall  p  in  s, exists  s_p  subseteq  lamFunFinset v,
        LamConDepth A (listToFinset p.1)  /\  LamConDepth A (listToFinset p.2)  /\
        (forall  r  in  s_p, forall  z  in  listToFinset r.1, LamEntDepth A (
listToFinset p.1) z)  /\
        (forall  z  in  listToFinset p.2, LamEntDepth A (lamOutputUnion s_p) z) :=
 by
      intro p hp
      have hEp := hEnts _ (mem_lamFunFinset.1 (hs hp))
      cases hEp with
      | @funTok _ _ _ s_p _ _ hCi hCo hs_p _ _ hEi hEo =>
        exact <s_p, hs_p, hCi, hCo, hEi, hEo>
    -- Combined witness by Finset.induction_on (choice-free; avoid erase/card)
    have hComb : forall  (s' : Finset (List (RawLamToken a)  x  List (RawLamToken
a))),
        s'  subseteq  s  ->
```

```
        exists  S  subseteq  lamFunFinset v,
          (forall  r  in  S, forall  z  in  listToFinset r.1, LamEntDepth A (
listToFinset xs) z)  /\
          (forall  z  in  lamOutputUnion s', LamEntDepth A (lamOutputUnion S) z)
/\
          LamConDepth A (lamOutputUnion S) := by
      intro s'
      induction s' using Finset.induction_on with
      | empty =>
        intro _
        refine <({} : Finset (List (RawLamToken a)  x  List (RawLamToken a))),
          Finset.empty_subset _, ?_, ?_, ?_>
        * intro _ hr; exact False.elim (Finset.notMem_empty _ hr)
        * intro z hz
          rw [lamOutputUnion_empty] at hz
          exact False.elim (Finset.notMem_empty z hz)
        * rw [lamOutputUnion_empty]; exact LamConDepth_empty A
      | insert q sR hq ihS =>
        intro hs'
        have hsR : sR  subseteq  s := fun p hp => hs' (Finset.mem_insert_of_mem hp
)
        have hqS : q  in  s := hs' (Finset.mem_insert_self q sR)
        obtain <S', hS'sub, hS'in, hS'out, hConS'> := ihS hsR
        obtain <Sq, hSqsub, hConQIn, hConQOut, hSqin, hSqout> := hWit q hqS
        have hSsub : S'  U ' Sq  subseteq  lamFunFinset v :=
          funion_subset_iff.mpr <hS'sub, hSqsub>
        have hEntAll : forall  t  in  lamInputUnion (S'  U ' Sq),
            LamEntDepth A (listToFinset xs) t := by
          intro t ht
          rcases (mem_lamInputUnion).1 ht with <r, hr, ht'>
          rcases mem_funion.1 hr with hr | hr
          * exact hS'in r hr t ht'
          * refine ih _ ?_ (listToFinset q.1) (listToFinset xs) t rfl
              hConIn hConQIn (fun w hw => hEntIn q hqS w hw) (hSqin r hr t ht')
            have htok : .funTok q.1 q.2  in  u := mem_lamFunFinset.1 (hs hqS)
            have htokR : .funTok r.1 r.2  in  v :=
              mem_lamFunFinset.1 (hSsub (mem_funion.2 (Or.inr hr)))
            have hzt := rawDepth_funTok_gt_left (v := r.2) ((mem_listToFinset).1
ht')
            have hDq := (finsetMaxDepth.of_mem_funTok htok).1
            have hDxs := finsetMaxDepth.list_lt_funTok xs ys
            exact Nat.max_lt.2 <Nat.lt_of_lt_of_le hDxs hleC,
              Nat.max_lt.2 <Nat.lt_of_lt_of_le hDq hleU,
                Nat.lt_of_lt_of_le
                  (Nat.lt_of_lt_of_le hzt (le_finsetMaxDepth_of_mem htokR))
                  hleV>>
        have hinU : LamConDepth A (lamInputUnion (S'  U ' Sq)) :=
          LamConDepth_mono A (subset_funion_right _ _)
            (LamConDepth_funion_of_entSet A M
              (fun _ _ _ hE => LamConDepth_insert_of_ent A hE)
              hConIn hEntAll (fun t ht => by
                rcases (mem_lamInputUnion).1 ht with <r, hr, ht'>
                have hDxs := finsetMaxDepth.list_lt_funTok xs ys
                have htok : .funTok r.1 r.2  in  v := mem_lamFunFinset.1 (hSsub hr
)
                exact Nat.max_lt.2 <Nat.lt_of_lt_of_le hDxs hleC,
```

```
                Nat.lt_of_lt_of_le
                  (Nat.lt_of_lt_of_le
                    (rawDepth_funTok_gt_left (v := r.2) ((mem_listToFinset).1 ht
'))
                    (le_finsetMaxDepth_of_mem htok)) hleV>))
        have hConU : LamConDepth A (lamOutputUnion (S'  U ' Sq)) :=
          ((LamConDepth_of_fun_nonempty A hFv).1 hv).2.2 (S'  U ' Sq) hSsub hinU
        have hConU' : LamConDepth A (lamOutputUnion S'  U ' lamOutputUnion Sq) :=
by
          rw [ <-  lamOutputUnion_funion]; exact hConU
        refine <S'  U ' Sq, hSsub, ?_, ?_, hConU>
        * intro r hr z hz
          rcases mem_funion.1 hr with hr | hr
          * exact hS'in r hr z hz
          * exact hEntAll _ ((mem_lamInputUnion).2 <r, mem_funion.2 (Or.inr hr),
hz>)
        * intro z hz
          rw [lamOutputUnion_funion]
          have hz'' : z  in  listToFinset q.2  U ' lamOutputUnion sR := by
            rwa [lamOutputUnion_insert] at hz
          rcases mem_funion.1 hz'' with hzQ | hzR
          * have hzx := hSqout z hzQ
            have hsub := subset_funion_right (lamOutputUnion S') (lamOutputUnion
Sq)
            cases decidableEq_finset
                (lamFunFinset (lamOutputUnion S'  U ' lamOutputUnion Sq)) {} with
            | isFalse hFn =>
              exact LamEntDepth_weaken_fun A hsub hConU'
                ((LamConDepth_of_fun_nonempty A hFn).1 hConU').1 hzx
            | isTrue hFo =>
              match z with
              | .bot => exact LamEntDepth.bot hConU'
              | .atom _ =>
                exact LamEntDepth_weaken_atom A hsub hConU' hFo hzx
              | .funTok _ _ =>
                cases hzx with
                | funTok _ _ _ _ _ hWo _ _ _ =>
                  exact False.elim (hWo (Finset.Subset.antisymm
                    (fun p hp => by
                      have : p  in  lamFunFinset
                          (lamOutputUnion S'  U ' lamOutputUnion Sq) := by
                        rw [lamFunFinset_funion]
                        exact mem_funion.2 (Or.inr hp)
                      exact False.elim (Finset.notMem_empty p (hFo  |>  this)))
                    (Finset.empty_subset _)))
          * have hzx := hS'out z hzR
            have hsub := subset_funion_left (lamOutputUnion S') (lamOutputUnion Sq
)
            cases decidableEq_finset
                (lamFunFinset (lamOutputUnion S'  U ' lamOutputUnion Sq)) {} with
            | isFalse hFn =>
              exact LamEntDepth_weaken_fun A hsub hConU'
                ((LamConDepth_of_fun_nonempty A hFn).1 hConU').1 hzx
            | isTrue hFo =>
              match z with
              | .bot => exact LamEntDepth.bot hConU'
```

```
            | .atom _ =>
              exact LamEntDepth_weaken_atom A hsub hConU' hFo hzx
            | .funTok _ _ =>
              cases hzx with
              | funTok _ _ _ _ _ hWo _ _ _ =>
                exact False.elim (hWo (Finset.Subset.antisymm
                  (fun p hp => by
                    have : p  in  lamFunFinset
                        (lamOutputUnion S'  U ' lamOutputUion Sq) := by
                      rw [lamFunFinset_funion]
                      exact mem_funion.2 (Or.inl hp)
                    exact False.elim (Finset.notMem_empty p (hFo  |>  this)))
                  (Finset.empty_subset _)))
    obtain <S, hSsub, hSin, hSoutS, hConS> :=
      hComb s (Finset.Subset.refl _)
    have hConOutS : LamConDepth A (lamOutputUnion s) :=
      ((LamConDepth_of_fun_nonempty A hWorld).1 hu').2.2 s hs
        (LamConDepth_mono A (subset_funion_right _ _)
          (LamConDepth_funion_of_entSet A M
            (fun _ _ _ hE => LamConDepth_insert_of_ent A hE)
            hConIn
            (fun t ht => by
              rcases (mem_lamInputUnion).1 ht with <p, hp, ht'>
              exact hEntIn p hp t ht')
            (fun t ht => by
              rcases (mem_lamInputUnion).1 ht with <p, hp, ht'>
              have hDxs := finsetMaxDepth.list_lt_funTok xs ys
              have htok : .funTok p.1 p.2  in  u := mem_lamFunFinset.1 (hs hp)
              exact Nat.max_lt.2 <Nat.lt_of_lt_of_le hDxs hleC,
                Nat.lt_of_lt_of_le
                  (Nat.lt_of_lt_of_le
                    (rawDepth_funTok_gt_left ((mem_listToFinset).1 ht'))
                    (le_finsetMaxDepth_of_mem htok)) hleU>)))
    have hSout : forall  z  in  listToFinset ys, LamEntDepth A (lamOutputUnion S)
z := by
      intro z hz
      refine ih _ ?_ (lamOutputUnion s) (lamOutputUnion S) z rfl
        hConS hConOutS (fun t ht => hSoutS t ht) (hEntOut z hz)
      exact Nat.max_lt.2 <
        Nat.lt_of_lt_of_le (finsetMaxDepth.outputUnion_lt hSsub hFv) hleV,
        Nat.max_lt.2 <
          Nat.lt_of_lt_of_le (finsetMaxDepth.outputUnion_lt hs hWorld) hleU,
          Nat.lt_of_lt_of_le
            (rawDepth_funTok_gt_right ((mem_listToFinset).1 hz)) hleC>>
    have hInCompat' : lamFunFinset (listToFinset xs) = {}  ->
        lamFunFinset (lamInputUnion S) = {} := by
      intro hxsF
      ext p
      constructor
      * intro hp
        have hmem : .funTok p.1 p.2  in  lamInputUnion S := mem_lamFunFinset.1 hp
        rcases (mem_lamInputUnion).1 hmem with <r, hr, hr'>
        cases hSin r hr _ hr' with
        | funTok _ _ _ _ _ hWx _ _ _ => exact False.elim (hWx hxsF)
      * intro hp; exact False.elim (Finset.notMem_empty _ hp)
    exact LamEntDepth.funTok hv hAv hConIn hConOut hSsub hFv hInCompat' hSin hSout
```

```
end InfoSys

end DomainTheory.InformationSystems
```

lamAtomFinset_singleton_funTok.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    mem_lamAtomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem lamAtomFinset_singleton_funTok (u v : List (RawLamToken a)) :
    lamAtomFinset ({.funTok u v} : Finset (RawLamToken a)) = {} := by
  ext x
  constructor
  * intro hx; exact False.elim (nomatch mem_lamAtomFinset.1 hx)
  * intro hx; exact False.elim (Finset.notMem_empty x hx)

end InfoSys

end DomainTheory.InformationSystems
```

LamConDepth_singleton_funTok.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
```

```
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamOutputUnion_empty
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamAtomFinset_singleton_funTok
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamFunFinset_singleton_funTok
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamOutputUnion_singleton
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConDepth_empty
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConDepth_of_fun_nonempty
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst70
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst74

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

/-- Singleton funTok is Con when components are (Scott (7) on a singleton). -/
theorem LamConDepth_singleton_funTok (xs ys : List (RawLamToken a))
    (hxs : LamConDepth A (listToFinset xs)) (hys : LamConDepth A (listToFinset ys)) :
    LamConDepth A ({.funTok xs ys} : Finset (RawLamToken a)) := by
  have : DecidableEq (Finset (List (RawLamToken a)  x  List (RawLamToken a))) :=
    decidableEq_finset
  have hF : lamFunFinset ({.funTok xs ys} : Finset (RawLamToken a)) = {(xs, ys)} :=
    lamFunFinset_singleton_funTok xs ys
  have hA : lamAtomFinset ({.funTok xs ys} : Finset (RawLamToken a)) = {} :=
    lamAtomFinset_singleton_funTok xs ys
  have hne : lamFunFinset ({.funTok xs ys} : Finset (RawLamToken a))  !=  {} := by
    rw [hF]; exact Finset.singleton_ne_empty _
  refine (LamConDepth_of_fun_nonempty A hne).2 <hA, ?_, ?_>
  * intro p hp
    have : p = (xs, ys) := Finset.mem_singleton.mp (hF  |>  hp)
    subst this; exact <hxs, hys>
  * intro s hs hIn
    rw [hF] at hs
    by_cases hmem : (xs, ys)  in  s
    * have hsin : s = {(xs, ys)} := by
        ext p
```

```
        constructor
        * intro hp
          exact Finset.mem_singleton.mpr (Finset.mem_singleton.mp (hs hp))
        * intro hp
          have : p = (xs, ys) := Finset.mem_singleton.mp hp
          subst this; exact hmem
      subst hsin
      rw [lamOutputUnion_singleton]
      exact hys
    * have hempty : s = {} := by
        ext p
        constructor
        * intro hp
          exact False.elim (hmem (by
            have : p = (xs, ys) := Finset.mem_singleton.mp (hs hp)
            subst this; exact hp))
        * intro hp; exact False.elim (Finset.notMem_empty p hp)
      subst hempty
      rw [lamOutputUnion_empty]
      exact LamConDepth_empty A

end InfoSys

end DomainTheory.InformationSystems
```

LamDCon_insert_of_ent.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConDepth_insert_of_ent
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamDToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst70
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst74
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamDForgetFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamDCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamDEnt

@[expose] public section

namespace DomainTheory.InformationSystems
```

```
open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem LamDCon_insert_of_ent {u : Finset (LamDToken A)} {p : LamDToken A}
    (ht : LamDEnt A u p) : LamDCon A (insert p u) := by
  rw [LamDCon, lamDForgetFinset.insert]
  exact LamConDepth_insert_of_ent A ht

end InfoSys

end DomainTheory.InformationSystems
```

lambdaSystem.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamDToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst70
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.lamDBot
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst74
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamDCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamDEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamDCon_insert_of_ent

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)
```

```
/-- **Definition 2.1 packaging.** The fun -calculus information system `D`. -/
def lambdaSystem : InfoSys (LamDToken A) where
  bot := lamDBot A
  Con := {u | LamDCon A u}
  Ent := LamDEnt A
  con_subset := fun hu hv => LamDCon.subset A hu hv
  con_sing := LamDCon.singleton A
  ent_con := fun h => LamDCon_insert_of_ent A h
  ent_bot := fun hu => LamDEnt.bot A hu
  ent_refl := fun hu hp => LamDEnt.of_mem A hu hp
  ent_trans := fun hv hu hEnts hEntc => LamDEnt.trans A hv hu hEnts hEntc

end InfoSys

end DomainTheory.InformationSystems
```

lamRhs.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst70
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst74
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lambdaSystem

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

/-- Right-hand side of the domain equation: `A + (D  ->  D)`. -/
def lamRhs : InfoSys (SumToken a (FunToken (lambdaSystem A) (lambdaSystem A))) :=
  sumSystem A (functionSystem (lambdaSystem A) (lambdaSystem A))
```

```
end InfoSys

end DomainTheory.InformationSystems
```

mem_listToLamDFinset.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamWF
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamDToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst70
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst74
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    listToLamDFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem mem_listToLamDFinset {xs : List (RawLamToken a)}
    {h : forall  t  in  xs, LamWF A t} {p : LamDToken A} :
    p  in  listToLamDFinset A xs h  <->  p.val  in  xs := by
  induction xs with
  | nil =>
    constructor
    * intro hp; exact False.elim (Finset.notMem_empty _ hp)
    * intro hp; cases hp
  | cons t ts ih =>
    constructor
    * intro hp
      rcases Finset.mem_insert.mp hp with hp | hp
      * subst hp; exact List.mem_cons_self
      * exact List.mem_cons_of_mem _ (ih.1 hp)
    * intro hp
```

```
      rcases List.mem_cons.mp hp with rfl | hp
      * exact Finset.mem_insert_self _ _
      * exact Finset.mem_insert_of_mem (ih.2 hp)

end InfoSys

end DomainTheory.InformationSystems
```

lamDForget_listToLamDFinset.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamWF
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamDToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst70
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst74
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamDForgetFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    listToLamDFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    mem_listToLamDFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem lamDForget_listToLamDFinset (xs : List (RawLamToken a))
    (h : forall  t  in  xs, LamWF A t) :
    lamDForgetFinset A (listToLamDFinset A xs h) = listToFinset xs := by
  ext t
  rw [mem_lamDForgetFinset, mem_listToFinset]
  constructor
  * rintro <p, hp, rfl>
```

```
    exact (mem_listToLamDFinset A).1 hp
  * intro ht
    let p : LamDToken A := <t, h t ht>
    exact <p, (mem_listToLamDFinset A).2 ht, rfl>

end InfoSys

end DomainTheory.InformationSystems
```

lamUnfold.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamDToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst70
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst74
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    IsLamWF_of_mem
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lambdaSystem
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    listToLamDFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamDCon

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

/-- Unfold a fun -token into the sum-of-function-space carrier (Scott?s matching). -/
def lamUnfold (t : LamDToken A) :
    SumToken a (FunToken (lambdaSystem A) (lambdaSystem A)) :=
  match t with
  | <.bot, _> => .bot
  | <.atom x, _> => .left x
```

```
  | <.funTok xs ys, <<hxs, hys>>> =>
      let u := listToLamDFinset A xs fun _w hw =>
        IsLamWF_of_mem A hxs ((mem_listToFinset).2 hw)
      let v := listToLamDFinset A ys fun _w hw =>
        IsLamWF_of_mem A hys ((mem_listToFinset).2 hw)
      .right <(u, v), <LamDCon.listToLamDFinset A xs hxs,
        LamDCon.listToLamDFinset A ys hys>>

end InfoSys

end DomainTheory.InformationSystems
```

Basic.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card

public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    RawLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamConN
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.IsLamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.lamBot
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.lamForget
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamForgetInsert
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamForgetFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamTokenCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamEntN
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamTokenEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamWF
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamDToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.lamDBot
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.lamDForget
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamDForgetInsert
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamDForgetFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamDCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamDEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lambdaSystem
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    listToLamDFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.lamUnfold
```

```
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.lamRhs
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    mem_lamAtomFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamAtomFinset_empty
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamFunFinset_empty
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamAtomFinset_mono
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamFunFinset_mono
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamAtomFinset_insert_bot
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamAtomFinset_insert_atom
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamAtomFinset_insert_funTok
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamFunFinset_insert_bot
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamFunFinset_insert_atom
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamFunFinset_insert_funTok
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamInputUnion_empty
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamOutputUnion_empty
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst23
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst27
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamAtomFinset_singleton_funTok
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamFunFinset_singleton_funTok
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamInputUnion_singleton
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamOutputUnion_singleton
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamInputUnion_insert
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamOutputUnion_insert
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamInputUnion_funion
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamOutputUnion_funion
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConDepth_empty
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConDepth_of_fun_empty
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConDepth_of_fun_nonempty
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConDepth_mono
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
```

```
    LamConDepth_singleton_bot
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConDepth_singleton_atom
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth_of_mem
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConDepth_insert_bot
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConDepth_insert_atom
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth_weaken_fun
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth_weaken_atom
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    funion_empty_right_lam
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    insert_filter_ne_lam
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    filter_ne_subset_of_subset_insert_lam
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamFunFinset_funion
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth_weaken_insert
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    finsetMaxDepth_insert_le
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConDepth_funion_of_entSet
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    funion_assoc_lam
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    funion_comm_lam
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    finsetMaxDepth_funion_le
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConDepth_insert_of_ent
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntDepth_trans
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst70
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    BasicInst74
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    mem_foldr_lamDForget
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConDepth_singleton_funTok
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    IsLamWF_of_mem
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamDCon_insert_of_ent
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    mem_listToLamDFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamDForget_listToLamDFinset

@[expose] public section
```

```
namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem mem_foldr_lamForget (s : Multiset (LamToken A)) (t : RawLamToken a) :
    t  in  Multiset.foldr (lamForgetInsert (A := A)) {} s  <->  exists  p  in  s, p.
    val = t := by
  refine Multiset.induction_on s ?_ ?_
  * constructor
    * intro ht; exact False.elim (Finset.notMem_empty t ht)
    * rintro <_, hp, _>; exact False.elim (by cases hp)
  * intro p rest ih
    simp only [Multiset.foldr_cons, lamForgetInsert, Finset.mem_insert, ih, Multiset.
    mem_cons]
    constructor
    * rintro (rfl | <q, hq, hq'>)
      * exact <p, Or.inl rfl, rfl>
      * exact <q, Or.inr hq, hq'>
    * rintro <q, hq, hq'>
      rcases hq with rfl | hq
      * exact Or.inl hq'.symm
      * exact Or.inr <q, hq, hq'>

theorem funion_insert_lam (u : Finset (RawLamToken a)) (a : RawLamToken a)
    (t : Finset (RawLamToken a)) : u  U ' insert a t = insert a (u  U ' t) := by
  ext x
  constructor
  * intro hx
    rcases mem_funion.mp hx with hu | hins
    * exact Finset.mem_insert_of_mem (mem_funion.mpr (Or.inl hu))
    * rcases Finset.mem_insert.mp hins with ha | ht
      * exact Finset.mem_insert.mpr (Or.inl ha)
      * exact Finset.mem_insert_of_mem (mem_funion.mpr (Or.inr ht))
  * intro hx
    rcases Finset.mem_insert.mp hx with ha | h
    * exact mem_funion.mpr (Or.inr (Finset.mem_insert.mpr (Or.inl ha)))
    * rcases mem_funion.mp h with hu | ht
      * exact mem_funion.mpr (Or.inl hu)
      * exact mem_funion.mpr (Or.inr (Finset.mem_insert.mpr (Or.inr ht)))

end InfoSys
end DomainTheory.InformationSystems
```

LamConN_atom_con.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
```

```
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.Basic

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem LamConN_atom_con {n : Nat} {u : Finset (RawLamToken a)}
    (hu : LamConN A n u) (hne : lamAtomFinset u  !=  {}) : lamAtomFinset u  in  A.Con
    := by
  induction n generalizing u with
  | zero => exact hu.1
  | succ n ih =>
    rcases hu with hu | <hA, _, _>
    * exact ih hu hne
    * exact False.elim (hne hA)

end InfoSys
end DomainTheory.InformationSystems
```

LamConN_atom_empty.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.Basic

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem LamConN_atom_empty {n : Nat} {u : Finset (RawLamToken a)}
    (hu : LamConN A n u) (hne : lamFunFinset u  !=  {}) : lamAtomFinset u = {} := by
  induction n generalizing u with
  | zero => exact False.elim (hne hu.2)
  | succ n ih =>
    rcases hu with hu | <hA, _, _>
```

```
    * exact ih hu hne
    * exact hA

end InfoSys
end DomainTheory.InformationSystems
```

LamConN_down_atom.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.Basic

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

/-- Atom/bot-only sets that are consistent at stage `n` are consistent at stage `0`. -/
theorem LamConN_down_atom {n : Nat} {u : Finset (RawLamToken a)}
    (hu : LamConN A n u) (hF : lamFunFinset u = {}) : LamConN A 0 u := by
  induction n generalizing u with
  | zero => exact hu
  | succ n ih =>
    rcases hu with hu | <hA, _, _>
    * exact ih hu hF
    * exact <by rw [hA]; exact A.con_empty, hF>

end InfoSys
end DomainTheory.InformationSystems
```

LamConN_empty.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.Basic

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive
```

```
namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem LamConN_empty (n : Nat) : LamConN A n ({} : Finset (RawLamToken a)) := by
  induction n with
  | zero =>
    change lamAtomFinset {}  in  A.Con  /\  lamFunFinset {} = {}
    rw [lamAtomFinset_empty, lamFunFinset_empty]
    exact <A.con_empty, rfl>
  | succ n ih => exact Or.inl ih

end InfoSys
end DomainTheory.InformationSystems
```

LamConN_exists_FunCon.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.Basic

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

/-- If `u` has function tokens at stage `n`, some earlier right branch supplies FunCon.
    -/
theorem LamConN_exists_FunCon {n : Nat} {u : Finset (RawLamToken a)}
    (hu : LamConN A n u) (hne : lamFunFinset u  !=  {}) :
    exists  k, k + 1  <=  n  /\  lamAtomFinset u = {}  /\
      (forall  p  in  lamFunFinset u,
        LamConN A k (listToFinset p.1)  /\  LamConN A k (listToFinset p.2))  /\
      (forall  s  subseteq  lamFunFinset u,
        LamConN A k (lamInputUnion s)  ->  LamConN A k (lamOutputUnion s)) := by
  induction n generalizing u with
  | zero => exact False.elim (hne hu.2)
  | succ n ih =>
    rcases hu with hu | <hA, hWF, hFun>
    * rcases ih hu hne with <k, hk, H>
      exact <k, Nat.le_succ_of_le hk, H>
    * exact <n, Nat.le_refl _, hA, hWF, hFun>
```

```
end InfoSys
end DomainTheory.InformationSystems
```

LamConN_fun_FunCon_right.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.Basic

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

/-- FunCon from an explicit right-branch witness at stage `n+1`. -/
theorem LamConN_fun_FunCon_right {n : Nat} {u : Finset (RawLamToken a)}
    (_hA : lamAtomFinset u = {})
    (_hWF : forall  p  in  lamFunFinset u,
      LamConN A n (listToFinset p.1)  /\  LamConN A n (listToFinset p.2))
    (hFun : forall  s  subseteq  lamFunFinset u,
      LamConN A n (lamInputUnion s)  ->  LamConN A n (lamOutputUnion s))
    {s : Finset (List (RawLamToken a)  x  List (RawLamToken a))}
    (hs : s  subseteq  lamFunFinset u) (hIn : LamConN A n (lamInputUnion s)) :
    LamConN A n (lamOutputUnion s) :=
  hFun s hs hIn

end InfoSys
end DomainTheory.InformationSystems
```

LamConN_fun_empty.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.Basic

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys
```

```
variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem LamConN_fun_empty {n : Nat} {u : Finset (RawLamToken a)}
    (hu : LamConN A n u) (hne : lamAtomFinset u  !=  {}) : lamFunFinset u = {} := by
  induction n generalizing u with
  | zero => exact hu.2
  | succ n ih =>
    rcases hu with hu | <hA, _, _>
    * exact ih hu hne
    * exact False.elim (hne hA)

end InfoSys
end DomainTheory.InformationSystems
```

LamConN_insert_bot.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.Basic

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem LamConN_insert_bot {n : Nat} {u : Finset (RawLamToken a)}
    (hu : LamConN A n u) : LamConN A n (insert (.bot : RawLamToken a) u) := by
  induction n generalizing u with
  | zero =>
    exact <by rw [lamAtomFinset_insert_bot]; exact hu.1,
      by rw [lamFunFinset_insert_bot]; exact hu.2>
  | succ n ih =>
    rcases hu with hu | <hA, hWF, hFun>
    * exact Or.inl (ih hu)
    * exact Or.inr <by rw [lamAtomFinset_insert_bot]; exact hA,
        fun p hp => hWF p (by rwa [lamFunFinset_insert_bot] at hp),
        fun s hs hIn =>
          hFun s (fun p hp => by rw [ <-  lamFunFinset_insert_bot u]; exact hs hp) hIn
    >

end InfoSys
end DomainTheory.InformationSystems
```

LamConN_mono.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.Basic

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem LamConN_mono {n m : Nat} (hmn : n  <=  m) {u : Finset (RawLamToken a)}
    (hu : LamConN A n u) : LamConN A m u := by
  induction hmn with
  | refl => exact hu
  | step _ ih => exact Or.inl ih

end InfoSys
end DomainTheory.InformationSystems
```

LamConN_FunCon_mono_atom.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.Basic
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConN_mono
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConN_down_atom

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)
```

```
/-- Apply a stage-`k` FunCon at stage `m  >=  k` when the input union has no function
    tokens. -/
theorem LamConN_FunCon_mono_atom {k m : Nat} (hkm : k  <=  m)
    {u : Finset (RawLamToken a)}
    (hFun : forall  s  subseteq  lamFunFinset u,
      LamConN A k (lamInputUnion s)  ->  LamConN A k (lamOutputUnion s))
    {s : Finset (List (RawLamToken a)  x  List (RawLamToken a))}
    (hs : s  subseteq  lamFunFinset u)
    (hIn : LamConN A m (lamInputUnion s))
    (hF : lamFunFinset (lamInputUnion s) = {}) :
    LamConN A m (lamOutputUnion s) := by
  have h0 := LamConN_down_atom A hIn hF
  have hk : LamConN A k (lamInputUnion s) := LamConN_mono A (Nat.zero_le k) h0
  exact LamConN_mono A hkm (hFun s hs hk)

end InfoSys
end DomainTheory.InformationSystems
```

LamConN_fun_FunCon_atom.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.Basic
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConN_empty
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConN_exists_FunCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConN_FunCon_mono_atom

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

/-- Full FunCon at stage `n+1` (uses `exists_FunCon` + atom-input lift).
Nested function tokens in the input union remain for the general `ent_con` argument. -/
theorem LamConN_fun_FunCon_atom {n : Nat} {u : Finset (RawLamToken a)}
    (hu : LamConN A (n + 1) u)
    {s : Finset (List (RawLamToken a)  x  List (RawLamToken a))}
    (hs : s  subseteq  lamFunFinset u) (hIn : LamConN A n (lamInputUnion s))
    (hF : lamFunFinset (lamInputUnion s) = {}) :
    LamConN A n (lamOutputUnion s) := by
  if hempty : s = {} then
    subst hempty
```

```
    simpa [lamOutputUnion_empty] using LamConN_empty A n
  else
    have hne : lamFunFinset u  !=  {} := by
      obtain <p, hp> := Finset.nonempty_of_ne_empty hempty
      exact Finset.ne_empty_of_mem (hs hp)
    rcases LamConN_exists_FunCon A hu hne with <k, hk, _, _, hFun>
    have hkn : k  <=  n := Nat.le_of_succ_le_succ hk
    exact LamConN_FunCon_mono_atom A hkn hFun hs hIn hF

end InfoSys
end DomainTheory.InformationSystems
```

LamConN_fun_wf.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.Basic
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConN_mono

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem LamConN_fun_wf {n : Nat} {u : Finset (RawLamToken a)}
    {p : List (RawLamToken a)  x  List (RawLamToken a)}
    (hu : LamConN A (n + 1) u) (hp : p  in  lamFunFinset u) :
    LamConN A n (listToFinset p.1)  /\  LamConN A n (listToFinset p.2) := by
  have aux {m : Nat} {w : Finset (RawLamToken a)}
      (hw : LamConN A m w) (hq : p  in  lamFunFinset w) :
      exists  k, k + 1  <=  m  /\
        LamConN A k (listToFinset p.1)  /\  LamConN A k (listToFinset p.2) := by
    induction m generalizing w with
    | zero =>
      exact False.elim (by
        have : p  in  ({} : Finset _) := by rw [ <-  hw.2]; exact hq
        exact Finset.notMem_empty p this)
    | succ m ih =>
      rcases hw with hw | <_, hWF, _>
      * rcases ih hw hq with <k, hk, hk'>
        exact <k, Nat.le_succ_of_le hk, hk'>
      * exact <m, Nat.le_refl _, hWF p hq>
  rcases aux hu hp with <k, hk, h1, h2>
  have hk' : k  <=  n := Nat.le_of_succ_le_succ hk
```

```
  exact <LamConN_mono A hk' h1, LamConN_mono A hk' h2>

end InfoSys
end DomainTheory.InformationSystems
```

LamConN_insert_of_ent_atom.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.Basic
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConN_mono

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

/-- Inserting an entailed bot or atom preserves stage-`n` consistency.
Function-token `ent_con` (FunCon on `insert`) remains. -/
theorem LamConN_insert_of_ent_atom {n : Nat} {u : Finset (RawLamToken a)} {x : a}
    (ht : LamEntN A n u (.atom x)) : LamConN A n (insert (.atom x) u) := by
  induction n generalizing u with
  | zero =>
    rcases ht with <<_, hF>, _, hEnt>
    exact <by rw [lamAtomFinset_insert_atom]; exact A.ent_con hEnt,
      by rw [lamFunFinset_insert_atom]; exact hF>
  | succ n ih =>
    rcases ht with ht | <_, hF, _, hEnt>
    * exact Or.inl (ih ht)
    * exact LamConN_mono A (Nat.zero_le (n + 1))
        <by rw [lamAtomFinset_insert_atom]; exact A.ent_con hEnt,
          by rw [lamFunFinset_insert_atom]; exact hF>

end InfoSys
end DomainTheory.InformationSystems
```

LamCon_empty.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.Basic
```

```
@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem LamCon_empty : LamCon A ({} : Finset (RawLamToken a)) :=
  <0, by
    change lamAtomFinset {}  in  A.Con  /\  lamFunFinset {} = {}
    rw [lamAtomFinset_empty, lamFunFinset_empty]
    exact <A.con_empty, rfl>>

end InfoSys
end DomainTheory.InformationSystems
```

LamCon_singleton_atom.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.Basic

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem LamCon_singleton_atom (x : a) :
    LamCon A ({.atom x} : Finset (RawLamToken a)) := by
  refine <0, ?_>
  change lamAtomFinset _  in  A.Con  /\  lamFunFinset _ = {}
  constructor
  * have h : lamAtomFinset ({.atom x} : Finset (RawLamToken a)) = {x} := by
      ext y
      simp only [mem_lamAtomFinset, Finset.mem_singleton]
      exact <RawLamToken.atom.inj, congrArg RawLamToken.atom>
    rw [h]; exact A.con_sing x
  * ext p
    constructor
```

```
    * intro hp; exact False.elim (nomatch mem_lamFunFinset.1 hp)
    * intro hp; exact False.elim (Finset.notMem_empty p hp)

end InfoSys
end DomainTheory.InformationSystems
```

LamCon_singleton_bot.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.Basic

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem LamCon_singleton_bot : LamCon A ({.bot} : Finset (RawLamToken a)) := by
  refine <0, ?_>
  change lamAtomFinset _  in  A.Con  /\  lamFunFinset _ = {}
  constructor
  * have h : lamAtomFinset ({.bot} : Finset (RawLamToken a)) = {} := by
      ext x
      constructor
      * intro hx; exact False.elim (nomatch mem_lamAtomFinset.1 hx)
      * intro hx; exact False.elim (Finset.notMem_empty x hx)
    rw [h]; exact A.con_empty
  * ext p
    constructor
    * intro hp; exact False.elim (nomatch mem_lamFunFinset.1 hp)
    * intro hp; exact False.elim (Finset.notMem_empty p hp)

end InfoSys
end DomainTheory.InformationSystems
```

LamCon_singleton_funTok.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.Basic
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConN_mono
```

```
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConN_empty

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem LamCon_singleton_funTok (u v : List (RawLamToken a))
    (hu : LamCon A (listToFinset u)) (hv : LamCon A (listToFinset v)) :
    LamCon A ({.funTok u v} : Finset (RawLamToken a)) := by
  rcases hu with <n, hu>
  rcases hv with <m, hv>
  let k := max n m
  refine <k + 1, Or.inr <?_, ?_, ?_>>
  * exact lamAtomFinset_singleton_funTok u v
  * intro p hp
    have : p = (u, v) := by
      have h := Finset.mem_singleton.mp (by
        rw [ <-  lamFunFinset_singleton_funTok u v]; exact hp)
      exact h
    subst this
    exact <LamConN_mono A (Nat.le_max_left n m) hu,
      LamConN_mono A (Nat.le_max_right n m) hv>
  * intro s hs hIn
    have hFun : lamFunFinset ({.funTok u v} : Finset (RawLamToken a)) = {(u, v)} :=
      lamFunFinset_singleton_funTok u v
    rw [hFun] at hs
    if hmem : (u, v)  in  s then
      have hsin : s = {(u, v)} := by
        ext p
        constructor
        * intro hp
          exact Finset.mem_singleton.mpr (Finset.mem_singleton.mp (hs hp))
        * intro hp
          have : p = (u, v) := Finset.mem_singleton.mp hp
          subst this; exact hmem
      subst hsin
      rw [lamOutputUnion_singleton]
      exact LamConN_mono A (Nat.le_max_right n m) hv
    else
      have hempty : s = {} := by
        ext p
        constructor
        * intro hp
          exact False.elim (hmem (by
            have : p = (u, v) := Finset.mem_singleton.mp (hs hp)
            subst this; exact hp))
        * intro hp
```

```
          exact False.elim (Finset.notMem_empty p hp)
      subst hempty
      exact LamConN_empty A k

end InfoSys
end DomainTheory.InformationSystems
```

LamCon_subset.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.Basic

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem LamCon_subset {u : Finset (RawLamToken a)} (hu : LamCon A u) :
    forall  {v}, v  subseteq  u  ->  LamCon A v := by
  rcases hu with <n, hu>
  induction n generalizing u with
  | zero =>
    intro v hv
    refine <0, ?_>
    change lamAtomFinset v  in  A.Con  /\  lamFunFinset v = {}
    rcases (hu : lamAtomFinset u  in  A.Con  /\  lamFunFinset u = {}) with <hA, hF>
    exact <A.con_subset hA (lamAtomFinset_mono hv),
      Finset.Subset.antisymm
        (fun p hp => by
          have : p  in  lamFunFinset u := lamFunFinset_mono hv hp
          rw [hF] at this; exact False.elim (Finset.notMem_empty p this))
        (Finset.empty_subset _)>
  | succ n ih =>
    intro v hv
    rcases hu with hu | <hA, hWF, hFun>
    * exact ih hu hv
    * refine <n + 1, Or.inr <?_, ?_, ?_>>
      * exact Finset.Subset.antisymm
          (fun x hx => by
            have : x  in  lamAtomFinset u := lamAtomFinset_mono hv hx
            rw [hA] at this; exact False.elim (Finset.notMem_empty x this))
          (Finset.empty_subset _)
      * intro p hp; exact hWF p (lamFunFinset_mono hv hp)
      * intro s hs hIn
```

```
        exact hFun s (fun p hp => lamFunFinset_mono hv (hs hp)) hIn

end InfoSys
end DomainTheory.InformationSystems
```

LamEntN_con.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.Basic

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem LamEntN_con {n : Nat} {u : Finset (RawLamToken a)} {t : RawLamToken a}
    (ht : LamEntN A n u t) : LamConN A n u := by
  match n, t with
  | 0, .bot => exact ht
  | 0, .atom _ => exact ht.1
  | 0, .funTok _ _ => exact False.elim ht
  | n + 1, t =>
    rcases ht with ht | ht
    * exact Or.inl (LamEntN_con (n := n) ht)
    * match t with
      | .bot => exact ht
      | .atom _ => exact ht.1
      | .funTok _ _ => exact ht.1

end InfoSys
end DomainTheory.InformationSystems
```

LamConN_insert_of_ent_bot.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.Basic
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntN_con
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConN_insert_bot
```

```
@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem LamConN_insert_of_ent_bot {n : Nat} {u : Finset (RawLamToken a)}
    (ht : LamEntN A n u .bot) : LamConN A n (insert (.bot : RawLamToken a) u) :=
  LamConN_insert_bot A (LamEntN_con A ht)

end InfoSys
end DomainTheory.InformationSystems
```

LamEntN_mono.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.Basic

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem LamEntN_mono {n m : Nat} (hmn : n  <=  m) {u : Finset (RawLamToken a)}
    {t : RawLamToken a} (ht : LamEntN A n u t) : LamEntN A m u t := by
  induction hmn with
  | refl => exact ht
  | step _ ih => exact Or.inl ih

end InfoSys
end DomainTheory.InformationSystems
```

LamEntN_of_mem.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
```

```
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.Basic
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamConN_fun_wf

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

/-- Membership entailment at a fixed stage (Scott (v) / ent_refl on raw tokens). -/
theorem LamEntN_of_mem {n : Nat} {u : Finset (RawLamToken a)} {t : RawLamToken a}
    (hu : LamConN A n u) (ht : t  in  u) : LamEntN A n u t := by
  induction n generalizing u t with
  | zero =>
    match t with
    | .bot => exact hu
    | .atom x =>
      exact <hu, Finset.ne_empty_of_mem (mem_lamAtomFinset.2 ht),
        A.ent_refl hu.1 (mem_lamAtomFinset.2 ht)>
    | .funTok _ _ =>
      exact False.elim (by
        have hp : (_, _)  in  lamFunFinset u := mem_lamFunFinset.2 ht
        rw [hu.2] at hp; exact Finset.notMem_empty _ hp)
  | succ n ih =>
    rcases hu with hu | hRight
    * exact Or.inl (ih hu ht)
    * have huFull : LamConN A (n + 1) u := Or.inr hRight
      match t with
      | .bot => exact Or.inr huFull
      | .atom x =>
        exact False.elim (by
          have : x  in  lamAtomFinset u := mem_lamAtomFinset.2 ht
          rw [hRight.1] at this; exact Finset.notMem_empty x this)
      | .funTok xs ys =>
        have hmem : (xs, ys)  in  lamFunFinset u := mem_lamFunFinset.2 ht
        have hWF := LamConN_fun_wf A huFull hmem
        refine Or.inr <huFull, hRight.1, hWF.1, hWF.2, <{(xs, ys)}, ?_, ?_, ?_>>
        * intro p hp
          have : p = (xs, ys) := Finset.mem_singleton.mp hp
          subst this
          exact hmem
        * intro p hp t ht'
          have : p = (xs, ys) := Finset.mem_singleton.mp hp
          subst this
          exact ih hWF.1 ht'
        * intro t ht'
          rw [lamOutputUnion_singleton (xs, ys)]
```

```
          exact ih hWF.2 ht'

end InfoSys
end DomainTheory.InformationSystems
```

LamEnt_bot.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.Basic

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem LamEnt_bot {u : Finset (RawLamToken a)} (hu : LamCon A u) :
    LamEnt A u .bot := by
  rcases hu with <n, hu>
  cases n with
  | zero => exact <0, hu>
  | succ n => exact <n + 1, Or.inr hu>

end InfoSys
end DomainTheory.InformationSystems
```

LamEnt_con.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.Basic
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntN_con

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys
```

```
variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem LamEnt_con {u : Finset (RawLamToken a)} {t : RawLamToken a}
    (ht : LamEnt A u t) : LamCon A u := by
  rcases ht with <n, ht>
  exact <n, LamEntN_con A ht>

end InfoSys
end DomainTheory.InformationSystems
```

LamEnt_of_mem.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.Basic
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEntN_of_mem

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem LamEnt_of_mem {u : Finset (RawLamToken a)} {t : RawLamToken a}
    (hu : LamCon A u) (ht : t  in  u) : LamEnt A u t := by
  rcases hu with <n, hu>
  exact <n, LamEntN_of_mem A hu ht>

end InfoSys
end DomainTheory.InformationSystems
```

LamTokenEnt_bot.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.Basic
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.LamEnt_bot

@[expose] public section
```

```
namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem LamTokenEnt_bot {u : Finset (LamToken A)} (hu : LamTokenCon A u) :
    LamTokenEnt A u (lamBot A) :=
  LamEnt_bot A hu

end InfoSys
end DomainTheory.InformationSystems
```

lamAtomFinset_funion.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.Basic

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem lamAtomFinset_funion (u v : Finset (RawLamToken a)) :
    lamAtomFinset (u  U ' v) = lamAtomFinset u  U ' lamAtomFinset v := by
  ext x
  constructor
  * intro hx
    have : .atom x  in  u  U ' v := mem_lamAtomFinset.1 hx
    rcases mem_funion.1 this with hu | hv
    * exact mem_funion.2 (Or.inl (mem_lamAtomFinset.2 hu))
    * exact mem_funion.2 (Or.inr (mem_lamAtomFinset.2 hv))
  * intro hx
    rcases mem_funion.1 hx with hu | hv
    * exact mem_lamAtomFinset.2 (mem_funion.2 (Or.inl (mem_lamAtomFinset.1 hu)))
    * exact mem_lamAtomFinset.2 (mem_funion.2 (Or.inr (mem_lamAtomFinset.1 hv)))

end InfoSys
end DomainTheory.InformationSystems
```

lamDForgetFinset_empty.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.Basic

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem lamDForgetFinset_empty : lamDForgetFinset A ({} : Finset (LamDToken A)) = {}
    :=
  rfl

end InfoSys
end DomainTheory.InformationSystems
```

LamDCon_empty.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.Basic
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamDForgetFinset_empty

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem LamDCon_empty : LamDCon A ({} : Finset (LamDToken A)) := by
  simpa [LamDCon, lamDForgetFinset_empty] using LamConDepth_empty A

end InfoSys
end DomainTheory.InformationSystems
```

lamForgetFinset_empty.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.Basic

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem lamForgetFinset_empty : lamForgetFinset A ({} : Finset (LamToken A)) = {} :=
  rfl

end InfoSys
end DomainTheory.InformationSystems
```

LamTokenCon_empty.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.Basic
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamCon_empty
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamForgetFinset_empty

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem LamTokenCon_empty : LamTokenCon A ({} : Finset (LamToken A)) := by
  simpa [LamTokenCon, lamForgetFinset_empty] using LamCon_empty A

end InfoSys
```

```
end DomainTheory.InformationSystems
```

lamRhs_eq_sum_function.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.Basic

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem lamRhs_eq_sum_function :
    lamRhs A = sumSystem A (functionSystem (lambdaSystem A) (lambdaSystem A)) :=
  rfl

end InfoSys
end DomainTheory.InformationSystems
```

Factoid82.lean

```
module

public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamRhs_eq_sum_function

@[expose] public section

namespace DomainTheory.InformationSystems.InfoSys

set_option linter.unusedVariables false

theorem theorem_82.{u_1} : forall  {a : Type u_1} [inst : DecidableEq a] (A :
    @DomainTheory.InformationSystems.InfoSys a inst),
  @Eq
    (@DomainTheory.InformationSystems.InfoSys
      (DomainTheory.InformationSystems.InfoSys.SumToken a
        (@DomainTheory.InformationSystems.InfoSys.FunToken (@DomainTheory.
    InformationSystems.InfoSys.LamDToken a inst A)
          (@DomainTheory.InformationSystems.InfoSys.LamDToken a inst A)
          (@DomainTheory.InformationSystems.InfoSys.instDecidableEqLamDToken a inst A)
          (@DomainTheory.InformationSystems.InfoSys.instDecidableEqLamDToken a inst A)
          (@DomainTheory.InformationSystems.InfoSys.lambdaSystem a inst A)
          (@DomainTheory.InformationSystems.InfoSys.lambdaSystem a inst A)))
```

```
    fun
      (a b :
        DomainTheory.InformationSystems.InfoSys.SumToken a
          (@DomainTheory.InformationSystems.InfoSys.FunToken
            (@DomainTheory.InformationSystems.InfoSys.LamDToken a inst A)
            (@DomainTheory.InformationSystems.InfoSys.LamDToken a inst A)
            (@DomainTheory.InformationSystems.InfoSys.instDecidableEqLamDToken a
  inst A)
            (@DomainTheory.InformationSystems.InfoSys.instDecidableEqLamDToken a
  inst A)
            (@DomainTheory.InformationSystems.InfoSys.lambdaSystem a inst A)
            (@DomainTheory.InformationSystems.InfoSys.lambdaSystem a inst A))) =>
      @DomainTheory.InformationSystems.InfoSys.instDecidableEqSumToken a
        (@DomainTheory.InformationSystems.InfoSys.FunToken (@DomainTheory.
  InformationSystems.InfoSys.LamDToken a inst A)
          (@DomainTheory.InformationSystems.InfoSys.LamDToken a inst A)
          (@DomainTheory.InformationSystems.InfoSys.instDecidableEqLamDToken a inst A)
          (@DomainTheory.InformationSystems.InfoSys.instDecidableEqLamDToken a inst A)
          (@DomainTheory.InformationSystems.InfoSys.lambdaSystem a inst A)
          (@DomainTheory.InformationSystems.InfoSys.lambdaSystem a inst A))
        inst
        (@DomainTheory.InformationSystems.InfoSys.instDecidableEqFunToken
          (@DomainTheory.InformationSystems.InfoSys.LamDToken a inst A)
          (@DomainTheory.InformationSystems.InfoSys.LamDToken a inst A)
          (@DomainTheory.InformationSystems.InfoSys.instDecidableEqLamDToken a inst A)
          (@DomainTheory.InformationSystems.InfoSys.instDecidableEqLamDToken a inst A)
          (@DomainTheory.InformationSystems.InfoSys.lambdaSystem a inst A)
          (@DomainTheory.InformationSystems.InfoSys.lambdaSystem a inst A))
        a b)
     (@DomainTheory.InformationSystems.InfoSys.lamRhs a inst A)
     (@DomainTheory.InformationSystems.InfoSys.sumSystem a
       (@DomainTheory.InformationSystems.InfoSys.FunToken (@DomainTheory.
  InformationSystems.InfoSys.LamDToken a inst A)
         (@DomainTheory.InformationSystems.InfoSys.LamDToken a inst A)
         (@DomainTheory.InformationSystems.InfoSys.instDecidableEqLamDToken a inst A)
         (@DomainTheory.InformationSystems.InfoSys.instDecidableEqLamDToken a inst A)
         (@DomainTheory.InformationSystems.InfoSys.lambdaSystem a inst A)
         (@DomainTheory.InformationSystems.InfoSys.lambdaSystem a inst A))
       inst
       (@DomainTheory.InformationSystems.InfoSys.instDecidableEqFunToken
         (@DomainTheory.InformationSystems.InfoSys.LamDToken a inst A)
         (@DomainTheory.InformationSystems.InfoSys.LamDToken a inst A)
         (@DomainTheory.InformationSystems.InfoSys.instDecidableEqLamDToken a inst A)
         (@DomainTheory.InformationSystems.InfoSys.instDecidableEqLamDToken a inst A)
         (@DomainTheory.InformationSystems.InfoSys.lambdaSystem a inst A)
         (@DomainTheory.InformationSystems.InfoSys.lambdaSystem a inst A))
       A
       (@DomainTheory.InformationSystems.InfoSys.functionSystem
         (@DomainTheory.InformationSystems.InfoSys.LamDToken a inst A)
         (@DomainTheory.InformationSystems.InfoSys.LamDToken a inst A)
         (@DomainTheory.InformationSystems.InfoSys.instDecidableEqLamDToken a inst A)
         (@DomainTheory.InformationSystems.InfoSys.instDecidableEqLamDToken a inst A)
         (@DomainTheory.InformationSystems.InfoSys.lambdaSystem a inst A)
         (@DomainTheory.InformationSystems.InfoSys.lambdaSystem a inst A))) :=
@DomainTheory.InformationSystems.InfoSys.lamRhs_eq_sum_function
```

```
end DomainTheory.InformationSystems.InfoSys
```

DomainEquation.lean

```
module

public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid81
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82

@[expose] public section

namespace DomainTheory.InformationSystems

namespace InfoSys

-- Factoid 8.1: see Factoid81.lean (`treeSystem`, `treeUnfold`, `treeRhs`)
-- Factoid 8.2: see Factoid82.lean (`lambdaSystem`, `lamUnfold`, `lamRhs`)
-- Factoid 8.3–8.4: see Factoid83.lean / Factoid84.lean (not re-exported here)

end InfoSys

end DomainTheory.InformationSystems
```

lamUnfold_atom.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.Basic

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem lamUnfold_atom (x : a) :
    lamUnfold A <.atom x, <trivial>> = SumToken.left x := rfl

end InfoSys
end DomainTheory.InformationSystems
```

lamUnfold_bot.lean

```
module
```

```
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.Basic

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem lamUnfold_bot : lamUnfold A (lamDBot A) = SumToken.bot := rfl

end InfoSys
end DomainTheory.InformationSystems
```

mem_lamForgetFinset.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.Basic

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem mem_lamForgetFinset {u : Finset (LamToken A)} {t : RawLamToken a} :
    t  in  lamForgetFinset A u  <->  exists  p  in  u, p.val = t := by
  simpa [lamForgetFinset] using mem_foldr_lamForget A u.1 t

end InfoSys
end DomainTheory.InformationSystems
```

LamTokenCon_subset.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
```

```
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.Basic
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamCon_subset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    mem_lamForgetFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem LamTokenCon_subset {u v : Finset (LamToken A)}
    (hu : LamTokenCon A u) (hv : v  subseteq  u) : LamTokenCon A v :=
  LamCon_subset A hu (by
    intro t ht
    rcases (mem_lamForgetFinset A).1 ht with <p, hp, rfl>
    exact (mem_lamForgetFinset A).2 <p, hv hp, rfl>)

end InfoSys
end DomainTheory.InformationSystems
```

LamTokenEnt_of_mem.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.Basic
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    mem_lamForgetFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamEnt_of_mem

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)
```

```
theorem LamTokenEnt_of_mem {u : Finset (LamToken A)} {p : LamToken A}
    (hu : LamTokenCon A u) (hp : p  in  u) : LamTokenEnt A u p :=
  LamEnt_of_mem A hu ((mem_lamForgetFinset A).2 <p, hp, rfl>)

end InfoSys
end DomainTheory.InformationSystems
```

lamForgetFinset_insert.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.Basic
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    mem_lamForgetFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem lamForgetFinset_insert (u : Finset (LamToken A)) (p : LamToken A) :
    lamForgetFinset A (insert p u) = insert p.val (lamForgetFinset A u) := by
  ext t
  constructor
  * intro ht
    rcases (mem_lamForgetFinset A).1 ht with <q, hq, rfl>
    rcases Finset.mem_insert.mp hq with rfl | hq
    * exact Finset.mem_insert_self _ _
    * exact Finset.mem_insert_of_mem ((mem_lamForgetFinset A).2 <q, hq, rfl>)
  * intro ht
    rcases Finset.mem_insert.mp ht with rfl | ht
    * exact (mem_lamForgetFinset A).2 <p, Finset.mem_insert_self p u, rfl>
    * rcases (mem_lamForgetFinset A).1 ht with <q, hq, rfl>
      exact (mem_lamForgetFinset A).2 <q, Finset.mem_insert_of_mem hq, rfl>

end InfoSys
end DomainTheory.InformationSystems
```

lamForgetFinset_singleton.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
```

```
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.Basic
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    mem_lamForgetFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem lamForgetFinset_singleton (p : LamToken A) :
    lamForgetFinset A {p} = {p.val} := by
  ext t
  constructor
  * intro ht
    rcases (mem_lamForgetFinset A).1 ht with <q, hq, hq'>
    have : q = p := Finset.mem_singleton.mp hq
    subst this; exact Finset.mem_singleton.mpr hq'.symm
  * intro ht
    exact (mem_lamForgetFinset A).2
      <p, Finset.mem_singleton_self p, (Finset.mem_singleton.mp ht).symm>

end InfoSys
end DomainTheory.InformationSystems
```

LamCon_of_forget_singleton.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.Basic
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamCon_singleton_bot
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamCon_singleton_atom
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    lamForgetFinset_singleton
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamCon_singleton_funTok

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys
```

```
variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem LamCon_of_forget_singleton (p : LamToken A) :
    LamCon A (lamForgetFinset A {p}) := by
  rw [lamForgetFinset_singleton]
  match p with
  | <.bot, _> => exact LamCon_singleton_bot A
  | <.atom x, _> => exact LamCon_singleton_atom A x
  | <.funTok u v, hWF> => exact LamCon_singleton_funTok A u v hWF.out.1 hWF.out.2

end InfoSys
end DomainTheory.InformationSystems
```

LamTokenCon_singleton.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.Basic
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamCon_of_forget_singleton

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

theorem LamTokenCon_singleton (p : LamToken A) : LamTokenCon A {p} :=
  LamCon_of_forget_singleton A p

end InfoSys
end DomainTheory.InformationSystems
```

lambdaSystem_con_axioms.lean

```
module

public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.FunctionSpace
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Sum
public import Mathlib.Data.Finset.Card
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.Basic
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamTokenCon_singleton
```

```
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamTokenCon_subset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamTokenEnt_bot
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid82.
    LamTokenEnt_of_mem

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

variable {a : Type*} [DecidableEq a]

variable (A : InfoSys a)

/-- Def 2.1 progress: `con_subset`, `con_sing`, `ent_bot`, `ent_refl`;
`ent_con` for bot/atom; `ent_con`/`ent_trans` on `funTok` remain. -/
theorem lambdaSystem_con_axioms :
    (forall  {u v : Finset (LamToken A)}, LamTokenCon A u  ->  v  subseteq  u  ->
    LamTokenCon A v)  /\
      (forall  p : LamToken A, LamTokenCon A {p})  /\
        (forall  {u : Finset (LamToken A)}, LamTokenCon A u  ->  LamTokenEnt A u (
    lamBot A))  /\
          (forall  {u : Finset (LamToken A)} {p : LamToken A},
            LamTokenCon A u  ->  p  in  u  ->  LamTokenEnt A u p) :=
  <fun hu hv => LamTokenCon_subset A hu hv,
    LamTokenCon_singleton A,
    fun hu => LamTokenEnt_bot A hu,
    fun hu hp => LamTokenEnt_of_mem A hu hp>

end InfoSys
end DomainTheory.InformationSystems
```

4.8.3 Factoid 8.3

A universal domain. As a final example of building up domains recursively, we give a construction of a "universal" domain U. (The reason for the name will be explained presently.) The best way to define U seems to be to define a domain V with a top element first, and then to remove the top.

The recursion for V is remarkably simple. We begin with two distinct objects Δ and ∇ that give information about the bottom and top of V, respectively. Thus, $\Delta_V = \Delta$ by definition. We assume that these two special data objects are "atomic" in the sense that they are not equal to any ordered pair of objects. For the definition of D_V we have these clauses:

(1) $\Delta, \nabla \in D_V$;

(2) $(X, \Delta) \in D_V$ and $(\Delta, Y) \in D_V$ whenever $X, Y \in D_V$.

In other words, we begin with two objects and close up under two flavors of copies of these objects. (A product result is involved here, so that is the reason for structuring the flavors the way we have.)

For V all subsets of D_V are consistent, so all we have left is to define entailment for this domain. The clauses are:

(3) $u \vdash_V \Delta$ always;

(4) $u \vdash_V \nabla$ whenever either $\nabla \in u$ or $\{X \mid (X, \Delta) \in u\} \vdash_V \nabla$ and $\{Y \mid (\Delta, Y) \in u\} \vdash_V \nabla$;

(5) $u \vdash_V (X', \Delta)$ whenever either $\nabla \in u$ or $\{X \mid (X, \Delta) \in u\} \vdash_V X'$; and

(6) $u \vdash_V (\Delta, Y')$ whenever either $\nabla \in u$ or $\{Y \mid (\Delta, Y) \in u\} \vdash_V Y'$.

The proof that V is an information system proceeds as before. Note that, under the above definition of entailment, the data objects Δ, (Δ, Δ), $((\Delta, \Delta), \Delta)$, etc. are all equivalent. There is, however, no other data object equivalent to ∇. The domain equation satisfied by V is:

$$V \cong V \times V.$$

Of course, there are an unlimited number of solutions to this equation, so the fact that V satisfies it tells us very little.

Because ∇ entails everything, we can regard it as a "rogue" object that ought to be banned from polite company: the only element of V it gives any information about is the top element, which is as unhelpful as any element could be. We should simply throw it out as being "inconsistent." What remains is the domain U. Formally we have:

(7) $D_U = D_V - \{\nabla\}$;

(8) $\Delta_U = \Delta_V$;

(9) $\mathrm{Con}_U = \{u \subseteq D_U \mid u \text{ finite and } u \nvdash_V \nabla\}$; and

(10) $u \vdash_U Y$ iff $u \in \mathrm{Con}_U$, $Y \in D_U$ and $u \vdash_V Y$. The same style of definition would work in any situation when an information system has a rogue data object that entails everything: there always is a system that results from eliminating all those objects that entail everything. Indeed, we could have always included such an object in any domain and altered the definition to take as elements those deductively closed sets of data objects that *do* not have the rogue object as a member. We did not do this for the reason that superfluous elements cause lots of exceptions in constructs such as product, where there is a temptation to let them enter into various combinations.

Now in U we *do* allow ∇ to enter into combinations—and this is part of the secret of the construction. The consequence is, however, that the domain equation which U satisfies is not too easy to state since it involves an unfamiliar functor. So it is not through such equations that we will understand its nature in a direct way. But it is possible to explain how it works by reference to the steps in the construction.

Imagine the full (infinite) binary tree. The data objects of U are giving information about possible paths in the tree. We think to the tree starting at the root node at the top of the page and growing down. The object Δ gives no information—so no paths are *excluded*. (If we would have allowed ∇, then the information it would have been giving is that all paths are excluded.) The data object (X, Δ) tells us about a path that *either* it is unrestricted on the right half of the tree, *or* on the left, when we start at the node directly below the root, the paths that are excluded from the subtree are those excluded according to X. This makes sense because the subtrees of the binary tree look exactly like the whole tree, so information can be relativized

or translated to other positions. With (Δ, Y) the rôles of right and left are interchanged. We could have introduced data objects of the form (X, Y) which tell us information about both halves of the tree at the same time, but the consistent set $\{(X, \Delta), (\Delta, Y)\}$ does the same job. In general consistent sets should be thought of as *conjunctions*; while, in this example, the comma in the ordered pair should be thought of as a *disjunction* when "reading" information objects.

We can now see that a single data object (if it contains ∇) looks down the tree along a finite path to some depth and then *excludes* the rest of the tree below that node. A consistent set of data objects leaves at least one hole, so at least one path is not excluded. The maximal consistent sets of information objects are those giving true information about one single (infinite) path—the total elements of the domain U correspond exactly to the infinite paths in the binary tree. The partial elements are harder to describe geometrically, however. In accumulating information into a consistent set, holes can be left all over the tree. A partial object is therefore of an indeterminate character, since the "path" we are describing might sneak through any one of the holes. (There is, by the way, a precise topological explanation of what is happening. The total elements of U form a well-known topological space, the so-called Cantor space, and the partially ordered set of elements of U is isomorphic to the lattice of *open* sets of the space—save that the whole space is not allowed.)

This is all very well, but what, we ask, is the good of this domain, and why is it called "universal". The proof cannot be given here, but the result is as follows. As a consequence of standard facts about countable Boolean algebras, it can be proved that every "countably based" domain is a subdomain of U. More specifically, if $\mathbf{A}$ is an information system, and if $D_{\mathbf{A}}$ is countable, then there exists a pair of approximable mappings

$$a : \mathbf{A} \to \mathbf{U} \text{ and } b : \mathbf{U} \to \mathbf{A},$$

such that

$$b \circ a = I_{\mathbf{A}} \text{ and } a \circ b \subseteq I_{\mathbf{U}}.$$

This makes $\mathbf{A}$ a special kind of retract of $\mathbf{U}$. The mappings a and b are far from unique, but at least there is one way to give a one-one embedding of the elements of $\mathbf{A}$ into the elements of $\mathbf{U}$.

The universal property of $\mathbf{U}$ can be applied quite widely. For example, since $(\mathbf{U} \to \mathbf{U})$ is a system with only countably many data objects (by explicit construction!), this system is a retract of $\mathbf{U}$. Fixing on one such retraction pair as above, makes $\mathbf{U}$ also into a model of the λ-calculus. Whether different retractions give essentially different models I do not know. But the point of the remark is to show that domains can contain their own function spaces for a variety of interesting reasons. ?

VToken.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys
```

```
/-- Data objects of `V`: bottom `Delta`, top `?`, and product left/right copies. -/
inductive VToken where
  | bot : VToken
  | top : VToken
  | pairL : VToken  ->  VToken
  | pairR : VToken  ->  VToken
  deriving DecidableEq

end InfoSys
end DomainTheory.InformationSystems
```

not_mem_uForget_top.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.vBot
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.vTop
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.vSystem
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.vUnfold
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.vRhs
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.IsUToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.UToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.uBot
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.uForget
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    uForgetInsert
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    uForgetFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.UCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.UEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.uSystem
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    mem_vPairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    mem_vPairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset_empty
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset_mono
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset_mono
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset_insert_pairL
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset_insert_pairR
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset_insert_bot
```

```
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset_insert_bot
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset_insert_pairR
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset_insert_pairL
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VEnt_mono
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    not_VEnt_empty_top
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    BasicInst33
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    BasicInst35
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    BasicInst39
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    mem_foldr_uForget
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    not_VEnt_singleton_ne_top
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    VEnt_insert_top_imp
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    not_VEnt_insert_top_of_ent
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    UCon_insert_of_ent

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

theorem not_mem_uForget_top (u : Finset UToken) : .top  notin  uForgetFinset u := by
  intro ht
  rcases (mem_uForgetFinset).1 ht with <p, _, hp>
  exact p.property.out hp

end InfoSys
end DomainTheory.InformationSystems
```

UCon_empty.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.vBot
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.vTop
```

```
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.vSystem
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.vUnfold
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.vRhs
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.IsUToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.UToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.uBot
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.uForget
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    uForgetInsert
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    uForgetFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.UCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.UEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.uSystem
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    mem_vPairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    mem_vPairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset_empty
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset_mono
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset_mono
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset_insert_pairL
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset_insert_pairR
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset_insert_bot
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset_insert_bot
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset_insert_pairR
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset_insert_pairL
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VEnt_mono
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    not_VEnt_empty_top
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    BasicInst33
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    BasicInst35
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    BasicInst39
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    mem_foldr_uForget
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    not_VEnt_singleton_ne_top
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    VEnt_insert_top_imp
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    not_VEnt_insert_top_of_ent
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    UCon_insert_of_ent
```

```
@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

theorem UCon_empty : UCon ({} : Finset UToken) := by
  simpa [UCon, uForgetFinset.empty] using not_VEnt_empty_top

end InfoSys
end DomainTheory.InformationSystems
```

Factoid83.lean

```
module

public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.UCon_empty

@[expose] public section

namespace DomainTheory.InformationSystems.InfoSys

set_option linter.unusedVariables false

theorem theorem_83 : DomainTheory.InformationSystems.InfoSys.UCon
  (@EmptyCollection.emptyCollection (Finset DomainTheory.InformationSystems.InfoSys.
    UToken)
    (@Finset.instEmptyCollection DomainTheory.InformationSystems.InfoSys.UToken)) :=
  DomainTheory.InformationSystems.InfoSys.UCon_empty

end DomainTheory.InformationSystems.InfoSys
```

vPairLFinset.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VToken

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys
private def pairLInsert : VToken  ->  Finset VToken  ->  Finset VToken
  | .pairL t => insert t
  | .bot | .top | .pairR _ => id

instance :
    LeftCommutative (pairLInsert : VToken  ->  Finset VToken  ->  Finset VToken) :=
```

```
  <fun p q s => by cases p <;> cases q <;> first | exact insert_comm' _ _ s | rfl>

/-- Left projection: `{X | (X, Delta)  in  u}`. -/
def vPairLFinset (u : Finset VToken) : Finset VToken :=
  Multiset.foldr pairLInsert ({} : Finset VToken) u.1

private theorem mem_foldr_pairL (s : Multiset VToken) (t : VToken) :
    t  in  Multiset.foldr pairLInsert ({} : Finset VToken) s  <->
      exists  p  in  s, p = .pairL t := by
  refine Multiset.induction_on s ?_ ?_
  * constructor
    * intro ht; exact False.elim (Finset.notMem_empty t ht)
    * rintro <_, hp, _>; exact False.elim (by cases hp)
  * intro p rest ih
    cases p with
    | pairL a =>
      simp only [Multiset.foldr_cons, pairLInsert, Finset.mem_insert, ih, Multiset.
    mem_cons]
      constructor
      * rintro (ht | <q, hq, hq'>)
        * exact <.pairL a, Or.inl rfl, congrArg VToken.pairL ht.symm>
        * exact <q, Or.inr hq, hq'>
      * rintro <q, hq, hq'>
        rcases hq with rfl | hq
        * injection hq' with ht; exact Or.inl ht.symm
        * exact Or.inr <q, hq, hq'>
    | bot | top | pairR _ =>
      simp only [Multiset.foldr_cons, pairLInsert, id_eq, ih, Multiset.mem_cons]
      constructor
      * rintro <q, hq, hq'>; exact <q, Or.inr hq, hq'>
      * rintro <q, hq, hq'>
        rcases hq with rfl | hq
        * exact False.elim (nomatch hq')
        * exact <q, hq, hq'>

theorem mem_vPairLFinset {u : Finset VToken} {t : VToken} :
    t  in  vPairLFinset u  <->  .pairL t  in  u := by
  simpa [vPairLFinset] using mem_foldr_pairL u.1 t

theorem vPairLFinset.insert_top (u : Finset VToken) :
    vPairLFinset (insert (.top : VToken) u) = vPairLFinset u := by
  ext t; simp only [mem_vPairLFinset, Finset.mem_insert]
  constructor
  * rintro (h | h); exacts [False.elim (nomatch h), h]
  * exact Or.inr

end InfoSys

end DomainTheory.InformationSystems
```

mem_vPairLFinset.lean

```
module

public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
```

vPairLFinset

vPairRFinset.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VToken

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys
private def pairRInsert : VToken  ->  Finset VToken  ->  Finset VToken
  | .pairR t => insert t
  | .bot | .top | .pairL _ => id

instance :
    LeftCommutative (pairRInsert : VToken  ->  Finset VToken  ->  Finset VToken) :=
  <fun p q s => by cases p <;> cases q <;> first | exact insert_comm' _ _ s | rfl>

/-- Right projection: `{Y | (Delta, Y)  in  u}`. -/
def vPairRFinset (u : Finset VToken) : Finset VToken :=
  Multiset.foldr pairRInsert ({} : Finset VToken) u.1

private theorem mem_foldr_pairR (s : Multiset VToken) (t : VToken) :
    t  in  Multiset.foldr pairRInsert ({} : Finset VToken) s  <->
      exists  p  in  s, p = .pairR t := by
  refine Multiset.induction_on s ?_ ?_
  * constructor
    * intro ht; exact False.elim (Finset.notMem_empty t ht)
    * rintro <_, hp, _>; exact False.elim (by cases hp)
  * intro p rest ih
    cases p with
    | pairR a =>
      simp only [Multiset.foldr_cons, pairRInsert, Finset.mem_insert, ih, Multiset.
    mem_cons]
      constructor
      * rintro (ht | <q, hq, hq'>)
        * exact <.pairR a, Or.inl rfl, congrArg VToken.pairR ht.symm>
        * exact <q, Or.inr hq, hq'>
      * rintro <q, hq, hq'>
        rcases hq with rfl | hq
        * injection hq' with ht; exact Or.inl ht.symm
        * exact Or.inr <q, hq, hq'>
    | bot | top | pairL _ =>
      simp only [Multiset.foldr_cons, pairRInsert, id_eq, ih, Multiset.mem_cons]
      constructor
      * rintro <q, hq, hq'>; exact <q, Or.inr hq, hq'>
      * rintro <q, hq, hq'>
        rcases hq with rfl | hq
        * exact False.elim (nomatch hq')
```

```
        * exact <q, hq, hq'>

theorem mem_vPairRFinset {u : Finset VToken} {t : VToken} :
    t  in  vPairRFinset u  <->  .pairR t  in  u := by
  simpa [vPairRFinset] using mem_foldr_pairR u.1 t

theorem vPairRFinset.empty : vPairRFinset ({} : Finset VToken) = {} := rfl

theorem vPairRFinset.insert_top (u : Finset VToken) :
    vPairRFinset (insert (.top : VToken) u) = vPairRFinset u := by
  ext t; simp only [mem_vPairRFinset, Finset.mem_insert]
  constructor
  * rintro (h | h); exacts [False.elim (nomatch h), h]
  * exact Or.inr

end InfoSys

end DomainTheory.InformationSystems
```

VEnt.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    mem_vPairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    mem_vPairRFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys
/-- Entailment on `V` (Scott (3)-(6)). -/
inductive VEnt : Finset VToken  ->  VToken  ->  Prop where
  | bot {u : Finset VToken} : VEnt u .bot
  | top_mem {u : Finset VToken} (h : .top  in  u) : VEnt u .top
  | top_both {u : Finset VToken}
      (hL : VEnt (vPairLFinset u) .top) (hR : VEnt (vPairRFinset u) .top) :
      VEnt u .top
  | pairL_top {u : Finset VToken} {x : VToken} (h : .top  in  u) : VEnt u (.pairL x)
  | pairL_proj {u : Finset VToken} {x : VToken}
      (h : VEnt (vPairLFinset u) x) : VEnt u (.pairL x)
  | pairR_top {u : Finset VToken} {y : VToken} (h : .top  in  u) : VEnt u (.pairR y)
  | pairR_proj {u : Finset VToken} {y : VToken}
      (h : VEnt (vPairRFinset u) y) : VEnt u (.pairR y)
```

```
/-- If `u  |-  ?`, then `u` entails every token (rogue top). -/
theorem VEnt.of_ent_top {u : Finset VToken} (h : VEnt u .top) (c : VToken) :
    VEnt u c := by
  match c with
  | .bot => exact VEnt.bot
  | .top => exact h
  | .pairL x =>
    cases h with
    | top_mem hm => exact VEnt.pairL_top hm
    | top_both hL _ => exact VEnt.pairL_proj (VEnt.of_ent_top hL x)
  | .pairR y =>
    cases h with
    | top_mem hm => exact VEnt.pairR_top hm
    | top_both _ hR => exact VEnt.pairR_proj (VEnt.of_ent_top hR y)

theorem VEnt.of_mem {u : Finset VToken} {c : VToken}
    (hc : c  in  u) : VEnt u c := by
  induction c generalizing u with
  | bot => exact VEnt.bot
  | top => exact VEnt.top_mem hc
  | pairL x ih =>
    exact VEnt.pairL_proj (ih (mem_vPairLFinset.2 hc))
  | pairR y ih =>
    exact VEnt.pairR_proj (ih (mem_vPairRFinset.2 hc))

/-- From `v  |-  (X, Delta)`, either `?  in  v` or the left projection entails `X`. -/
theorem VEnt.pairL_inv {v : Finset VToken} {x : VToken}
    (h : VEnt v (.pairL x)) : .top  in  v  \/  VEnt (vPairLFinset v) x := by
  cases h with
  | pairL_top hm => exact Or.inl hm
  | pairL_proj h' => exact Or.inr h'

/-- Push an `EntSet` through the left projection (or absorb via top). -/
theorem VEnt.pairLFinset_of_EntSet {u v : Finset VToken}
    (hEnts : forall  y  in  u, VEnt v y) :
    .top  in  v  \/  (forall  y  in  vPairLFinset u, VEnt (vPairLFinset v) y) := by
  by_cases htop : .top  in  v
  * exact Or.inl htop
  * refine Or.inr ?_
    intro y hy
    have hy' := hEnts _ (mem_vPairLFinset.1 hy)
    rcases VEnt.pairL_inv hy' with hm | hproj
    * exact False.elim (htop hm)
    * exact hproj

theorem VEnt.pairR_inv {v : Finset VToken} {y : VToken}
    (h : VEnt v (.pairR y)) : .top  in  v  \/  VEnt (vPairRFinset v) y := by
  cases h with
  | pairR_top hm => exact Or.inl hm
  | pairR_proj h' => exact Or.inr h'

theorem VEnt.pairRFinset_of_EntSet {u v : Finset VToken}
    (hEnts : forall  y  in  u, VEnt v y) :
    .top  in  v  \/  (forall  y  in  vPairRFinset u, VEnt (vPairRFinset v) y) := by
  by_cases htop : .top  in  v
  * exact Or.inl htop
```

```
  * refine Or.inr ?_
    intro y hy
    have hy' := hEnts _ (mem_vPairRFinset.1 hy)
    rcases VEnt.pairR_inv hy' with hm | hproj
    * exact False.elim (htop hm)
    * exact hproj

theorem VEnt.trans {u v : Finset VToken} {c : VToken}
    (hEnts : forall  y  in  u, VEnt v y) (h : VEnt u c) : VEnt v c := by
  induction h generalizing v with
  | bot => exact VEnt.bot
  | top_mem hm => exact hEnts _ hm
  | top_both _ _ ihL ihR =>
    rcases VEnt.pairLFinset_of_EntSet hEnts with hm | hL
    * exact VEnt.of_ent_top (VEnt.top_mem hm) .top
    * rcases VEnt.pairRFinset_of_EntSet hEnts with hm | hR
      * exact VEnt.of_ent_top (VEnt.top_mem hm) .top
      * exact VEnt.top_both (ihL hL) (ihR hR)
  | pairL_top hm => exact VEnt.of_ent_top (hEnts _ hm) _
  | pairL_proj _ ih =>
    rcases VEnt.pairLFinset_of_EntSet hEnts with hm | hL
    * exact VEnt.pairL_top hm
    * exact VEnt.pairL_proj (ih hL)
  | pairR_top hm => exact VEnt.of_ent_top (hEnts _ hm) _
  | pairR_proj _ ih =>
    rcases VEnt.pairRFinset_of_EntSet hEnts with hm | hR
    * exact VEnt.pairR_top hm
    * exact VEnt.pairR_proj (ih hR)

end InfoSys

end DomainTheory.InformationSystems
```

IsUToken.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

/-- Tokens of `U`: `D_V` without `?` (Scott (7)). -/
class IsUToken (t : VToken) : Prop where
```

```
  out : t  !=  .top

end InfoSys

end DomainTheory.InformationSystems
```

BasicInst33.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.IsUToken

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

instance : DecidablePred IsUToken := fun t =>
  if h : t = .top then isFalse fun ht => ht.out h
  else isTrue <h>

end InfoSys

end DomainTheory.InformationSystems
```

UToken.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.IsUToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    BasicInst33

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive
```

```
namespace InfoSys

def UToken : Type := { t : VToken // IsUToken t }

end InfoSys

end DomainTheory.InformationSystems
```

BasicInst35.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    BasicInst33
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.UToken

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

instance : DecidableEq UToken := Subtype.instDecidableEq

end InfoSys

end DomainTheory.InformationSystems
```

VCon.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive
```

```
namespace InfoSys
/-- Scott: all finite subsets of `D_V` are consistent. -/
def VCon (_u : Finset VToken) : Prop := True

theorem VCon.any (u : Finset VToken) : VCon u := trivial

end InfoSys

end DomainTheory.InformationSystems
```

mem_vPairRFinset.lean

```
module

public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset
```

uBot.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    BasicInst33
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.UToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    BasicInst35

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

/-- Bottom `Delta_U = Delta_V` (Scott (8)). -/
def uBot : UToken := <.bot, by decide>

end InfoSys

end DomainTheory.InformationSystems
```

uForget.lean

```
module
```

```
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    BasicInst33
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.UToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    BasicInst35

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

def uForget (t : UToken) : VToken := t.val

end InfoSys

end DomainTheory.InformationSystems
```

uForgetInsert.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    BasicInst33
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.UToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    BasicInst35
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.uForget

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

def uForgetInsert : UToken  ->  Finset VToken  ->  Finset VToken :=
  fun t => insert (uForget t)
```

```
end InfoSys

end DomainTheory.InformationSystems
```

BasicInst39.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    BasicInst33
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    BasicInst35
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.uForget
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    uForgetInsert

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

instance : LeftCommutative uForgetInsert :=
  <fun p q s => insert_comm' (uForget p) (uForget q) s>

end InfoSys

end DomainTheory.InformationSystems
```

mem_foldr_uForget.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    BasicInst33
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.UToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    BasicInst35
```

```
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    uForgetInsert
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    BasicInst39

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

theorem mem_foldr_uForget (s : Multiset UToken) (t : VToken) :
    t  in  Multiset.foldr uForgetInsert {} s  <->  exists  p  in  s, p.val = t := by
  refine Multiset.induction_on s ?_ ?_
  * constructor
    * intro ht; exact False.elim (Finset.notMem_empty t ht)
    * rintro <_, hp, _>; exact False.elim (by cases hp)
  * intro p rest ih
    simp only [Multiset.foldr_cons, uForgetInsert, Finset.mem_insert, ih, Multiset.
    mem_cons]
    constructor
    * rintro (rfl | <q, hq, hq'>)
      * exact <p, Or.inl rfl, rfl>
      * exact <q, Or.inr hq, hq'>
    * rintro <q, hq, hq'>
      rcases hq with rfl | hq
      * exact Or.inl hq'.symm
      * exact Or.inr <q, hq, hq'>

end InfoSys

end DomainTheory.InformationSystems
```

uForgetFinset.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    BasicInst33
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.UToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    BasicInst35
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    uForgetInsert
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    BasicInst39
```

```
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    mem_foldr_uForget

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys
def uForgetFinset (u : Finset UToken) : Finset VToken :=
  Multiset.foldr uForgetInsert {} u.1

theorem mem_uForgetFinset {u : Finset UToken} {t : VToken} :
    t  in  uForgetFinset u  <->  exists  p  in  u, p.val = t := by
  simpa [uForgetFinset] using mem_foldr_uForget u.1 t

theorem uForgetFinset.empty : uForgetFinset ({} : Finset UToken) = {} := rfl

theorem uForgetFinset.insert (u : Finset UToken) (p : UToken) :
    uForgetFinset (Insert.insert p u) = Insert.insert p.val (uForgetFinset u) := by
  ext t
  constructor
  * intro ht
    rcases (mem_uForgetFinset).1 ht with <q, hq, rfl>
    rcases Finset.mem_insert.mp hq with rfl | hq
    * exact Finset.mem_insert_self _ _
    * exact Finset.mem_insert_of_mem ((mem_uForgetFinset).2 <q, hq, rfl>)
  * intro ht
    rcases Finset.mem_insert.mp ht with rfl | ht
    * exact (mem_uForgetFinset).2 <p, Finset.mem_insert_self p u, rfl>
    * rcases (mem_uForgetFinset).1 ht with <q, hq, rfl>
      exact (mem_uForgetFinset).2 <q, Finset.mem_insert_of_mem hq, rfl>

theorem uForgetFinset.mono {u v : Finset UToken} (h : v  subseteq  u) :
    uForgetFinset v  subseteq  uForgetFinset u := by
  intro t ht
  rcases (mem_uForgetFinset).1 ht with <p, hp, rfl>
  exact (mem_uForgetFinset).2 <p, h hp, rfl>

theorem uForgetFinset.singleton (p : UToken) :
    uForgetFinset {p} = {p.val} := by
  ext t
  constructor
  * intro ht
    rcases (mem_uForgetFinset).1 ht with <q, hq, hq'>
    have : q = p := Finset.mem_singleton.mp hq
    subst this; exact Finset.mem_singleton.mpr hq'.symm
  * intro ht
    exact (mem_uForgetFinset).2
      <p, Finset.mem_singleton_self p, (Finset.mem_singleton.mp ht).symm>

end InfoSys

end DomainTheory.InformationSystems
```

UCon.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    BasicInst33
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.UToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    BasicInst35
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    BasicInst39
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    uForgetFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VEnt_mono
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    not_VEnt_singleton_ne_top

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys
/-- Consistency for `U`: finite `u  subseteq  D_U` with `u  |/- _V ?` (Scott (9)). -/
def UCon (u : Finset UToken) : Prop :=
  not  VEnt (uForgetFinset u) .top

theorem UCon.singleton (p : UToken) : UCon {p} := by
  rw [UCon, uForgetFinset.singleton]
  exact not_VEnt_singleton_ne_top p.property.out

theorem UCon.subset {u v : Finset UToken}
    (hu : UCon u) (hv : v  subseteq  u) : UCon v :=
  fun h => hu (VEnt_mono (uForgetFinset.mono hv) h)

end InfoSys

end DomainTheory.InformationSystems
```

UEnt.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset
```

```
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    BasicInst33
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.UToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    BasicInst35
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    BasicInst39
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    uForgetFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.UCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.uBot

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys
/-- Entailment for `U` (Scott (10)). -/
def UEnt (u : Finset UToken) (p : UToken) : Prop :=
  UCon u  /\  VEnt (uForgetFinset u) p.val

theorem UEnt.bot {u : Finset UToken} (hu : UCon u) : UEnt u uBot :=
  <hu, VEnt.bot>

theorem UEnt.of_mem {u : Finset UToken} {p : UToken}
    (hu : UCon u) (hp : p  in  u) : UEnt u p :=
  <hu, VEnt.of_mem ((mem_uForgetFinset).2 <p, hp, rfl>)>

theorem UEnt.trans {u v : Finset UToken} {c : UToken}
    (hv : UCon v) (_hu : UCon u)
    (hEnts : forall  y  in  u, UEnt v y) (h : UEnt u c) : UEnt v c :=
  <hv, VEnt.trans
    (fun y hy => by
      rcases (mem_uForgetFinset).1 hy with <p, hp, rfl>
      exact (hEnts p hp).2)
    h.2>

end InfoSys

end DomainTheory.InformationSystems
```

vBot.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
```

```
    vPairRFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

/-- Bottom `Delta_V`. -/
def vBot : VToken := .bot

end InfoSys

end DomainTheory.InformationSystems
```

vPairLFinset_empty.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

theorem vPairLFinset_empty : vPairLFinset ({} : Finset VToken) = {} := rfl

end InfoSys

end DomainTheory.InformationSystems
```

not_VEnt_empty_top.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset
```

```
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset_empty

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

/-- Empty set does not entail top (no derivation bottoms out). -/
theorem not_VEnt_empty_top : not  VEnt ({} : Finset VToken) .top := by
  have aux : forall  {u : Finset VToken} {c : VToken},
      VEnt u c  ->  c = .top  ->  u = {}  ->  False := by
    intro u c h
    induction h with
    | bot => intro hc; exact nomatch hc
    | top_mem hm =>
      intro _ hu; exact Finset.notMem_empty _ (hu  |>  hm)
    | top_both _ _ ihL _ =>
      intro _ hu
      exact ihL rfl (by rw [hu]; exact vPairLFinset_empty)
    | pairL_top => intro hc; exact nomatch hc
    | pairL_proj => intro hc; exact nomatch hc
    | pairR_top => intro hc; exact nomatch hc
    | pairR_proj => intro hc; exact nomatch hc
  exact fun h => aux h rfl rfl

end InfoSys

end DomainTheory.InformationSystems
```

not_VEnt_singleton_ne_top.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    mem_vPairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    mem_vPairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    not_VEnt_empty_top
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    BasicInst33
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    BasicInst35
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
```

```
    BasicInst39

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

/-- Singleton of a non-top token does not entail top. -/
theorem not_VEnt_singleton_ne_top {t : VToken} (ht : t  !=  .top) :
    not  VEnt ({t} : Finset VToken) .top := by
  intro h
  match t, h with
  | .bot, .top_mem hm =>
    exact False.elim (nomatch Finset.mem_singleton.mp hm)
  | .bot, .top_both hL _ =>
    have hL' : vPairLFinset ({.bot} : Finset VToken) = {} := by
      ext x; constructor
      * intro hx; exact False.elim (nomatch mem_vPairLFinset.1 hx)
      * intro hx; exact False.elim (Finset.notMem_empty x hx)
    exact not_VEnt_empty_top (by rwa [hL'] at hL)
  | .top, _ => exact ht rfl
  | .pairL x, .top_mem hm =>
    exact False.elim (nomatch Finset.mem_singleton.mp hm)
  | .pairL x, .top_both _ hR =>
    have hR' : vPairRFinset ({.pairL x} : Finset VToken) = {} := by
      ext y; constructor
      * intro hy; exact False.elim (nomatch mem_vPairRFinset.1 hy)
      * intro hy; exact False.elim (Finset.notMem_empty y hy)
    exact not_VEnt_empty_top (by rwa [hR'] at hR)
  | .pairR y, .top_mem hm =>
    exact False.elim (nomatch Finset.mem_singleton.mp hm)
  | .pairR y, .top_both hL _ =>
    have hL' : vPairLFinset ({.pairR y} : Finset VToken) = {} := by
      ext x; constructor
      * intro hx; exact False.elim (nomatch mem_vPairLFinset.1 hx)
      * intro hx; exact False.elim (Finset.notMem_empty x hx)
    exact not_VEnt_empty_top (by rwa [hL'] at hL)

end InfoSys

end DomainTheory.InformationSystems
```

vPairLFinset_insert_bot.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
```

```
    vPairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    mem_vPairLFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

theorem vPairLFinset_insert_bot (u : Finset VToken) :
    vPairLFinset (insert (.bot : VToken) u) = vPairLFinset u := by
  ext t; simp only [mem_vPairLFinset, Finset.mem_insert]
  constructor
  * rintro (h | h); exacts [False.elim (nomatch h), h]
  * exact Or.inr

end InfoSys

end DomainTheory.InformationSystems
```

vPairLFinset_insert_pairL.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    mem_vPairLFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

theorem vPairLFinset_insert_pairL (u : Finset VToken) (t : VToken) :
    vPairLFinset (insert (.pairL t) u) = insert t (vPairLFinset u) := by
  ext s; simp only [Finset.mem_insert, mem_vPairLFinset]
  constructor
  * rintro (h | h)
    * exact Or.inl (VToken.pairL.inj h)
    * exact Or.inr h
  * rintro (rfl | h)
    * exact Or.inl rfl
    * exact Or.inr h
```

```
end InfoSys

end DomainTheory.InformationSystems
```

vPairLFinset_insert_pairR.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    mem_vPairLFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

theorem vPairLFinset_insert_pairR (u : Finset VToken) (t : VToken) :
    vPairLFinset (insert (.pairR t) u) = vPairLFinset u := by
  ext s; simp only [mem_vPairLFinset, Finset.mem_insert]
  constructor
  * rintro (h | h); exacts [False.elim (nomatch h), h]
  * exact Or.inr

end InfoSys

end DomainTheory.InformationSystems
```

vPairLFinset_mono.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    mem_vPairLFinset

@[expose] public section

namespace DomainTheory.InformationSystems
```

```
open DomainTheory.InformationSystems.Constructive

namespace InfoSys

theorem vPairLFinset_mono {u v : Finset VToken} (h : u  subseteq  v) :
    vPairLFinset u  subseteq  vPairLFinset v := by
  intro t ht
  exact mem_vPairLFinset.2 (h (mem_vPairLFinset.1 ht))

end InfoSys

end DomainTheory.InformationSystems
```

vPairRFinset_insert_bot.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    mem_vPairRFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

theorem vPairRFinset_insert_bot (u : Finset VToken) :
    vPairRFinset (insert (.bot : VToken) u) = vPairRFinset u := by
  ext t; simp only [mem_vPairRFinset, Finset.mem_insert]
  constructor
  * rintro (h | h); exacts [False.elim (nomatch h), h]
  * exact Or.inr

end InfoSys

end DomainTheory.InformationSystems
```

vPairRFinset_insert_pairL.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VEnt
```

```
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    mem_vPairRFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

theorem vPairRFinset_insert_pairL (u : Finset VToken) (t : VToken) :
    vPairRFinset (insert (.pairL t) u) = vPairRFinset u := by
  ext s; simp only [mem_vPairRFinset, Finset.mem_insert]
  constructor
  * rintro (h | h); exacts [False.elim (nomatch h), h]
  * exact Or.inr

end InfoSys

end DomainTheory.InformationSystems
```

vPairRFinset_insert_pairR.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    mem_vPairRFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

theorem vPairRFinset_insert_pairR (u : Finset VToken) (t : VToken) :
    vPairRFinset (insert (.pairR t) u) = insert t (vPairRFinset u) := by
  ext s; simp only [Finset.mem_insert, mem_vPairRFinset]
  constructor
  * rintro (h | h)
    * exact Or.inl (VToken.pairR.inj h)
    * exact Or.inr h
```

```
  * rintro (rfl | h)
    * exact Or.inl rfl
    * exact Or.inr h

end InfoSys

end DomainTheory.InformationSystems
```

VEnt_insert_top_imp.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset_insert_pairL
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset_insert_pairR
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset_insert_bot
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset_insert_bot
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset_insert_pairR
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset_insert_pairL
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    BasicInst33
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    BasicInst35
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    BasicInst39

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

/-- If inserting a non-top `y` creates a top-entailment and `u  |-  y`, then already `u
      |-  ?`. -/
theorem VEnt_insert_top_imp {u : Finset VToken} {y : VToken}
    (hy : y  !=  .top) (hEnt : VEnt u y) (hIns : VEnt (insert y u) .top) :
    VEnt u .top := by
  induction y generalizing u with
  | bot =>
    cases hIns with
    | top_mem hm =>
```

```
      rcases Finset.mem_insert.mp hm with h | h
      * exact False.elim (nomatch h)
      * exact VEnt.top_mem h
    | top_both hL hR =>
      rw [vPairLFinset_insert_bot] at hL
      rw [vPairRFinset_insert_bot] at hR
      exact VEnt.top_both hL hR
  | top => exact False.elim (hy rfl)
  | pairL x ih =>
    cases hIns with
    | top_mem hm =>
      rcases Finset.mem_insert.mp hm with h | h
      * exact False.elim (nomatch h)
      * exact VEnt.top_mem h
    | top_both hL hR =>
      rw [vPairLFinset_insert_pairL] at hL
      rw [vPairRFinset_insert_pairL] at hR
      rcases VEnt.pairL_inv hEnt with htop | hx
      * exact VEnt.top_mem htop
      * by_cases hxTop : x = .top
        * subst hxTop; exact VEnt.top_both hx hR
        * exact VEnt.top_both (ih hxTop hx hL) hR
  | pairR z ih =>
    cases hIns with
    | top_mem hm =>
      rcases Finset.mem_insert.mp hm with h | h
      * exact False.elim (nomatch h)
      * exact VEnt.top_mem h
    | top_both hL hR =>
      rw [vPairLFinset_insert_pairR] at hL
      rw [vPairRFinset_insert_pairR] at hR
      rcases VEnt.pairR_inv hEnt with htop | hz
      * exact VEnt.top_mem htop
      * by_cases hzTop : z = .top
        * subst hzTop; exact VEnt.top_both hL hz
        * exact VEnt.top_both hL (ih hzTop hz hR)

end InfoSys

end DomainTheory.InformationSystems
```

not_VEnt_insert_top_of_ent.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    BasicInst33
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
```

```
    BasicInst35
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    BasicInst39
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    VEnt_insert_top_imp

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

/-- Adding an entailed non-top token cannot create a top-entailment. -/
theorem not_VEnt_insert_top_of_ent {u : Finset VToken} {y : VToken}
    (hy : y  !=  .top) (hEnt : VEnt u y) (hNot : not  VEnt u .top) :
    not  VEnt (insert y u) .top :=
  fun hIns => hNot (VEnt_insert_top_imp hy hEnt hIns)

end InfoSys

end DomainTheory.InformationSystems
```

UCon_insert_of_ent.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    BasicInst33
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.UToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    BasicInst35
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    BasicInst39
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    uForgetFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.UCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.UEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    not_VEnt_insert_top_of_ent

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive
```

```
namespace InfoSys

theorem UCon_insert_of_ent {u : Finset UToken} {p : UToken}
    (h : UEnt u p) : UCon (insert p u) := by
  rcases h with <hCon, hEnt>
  rw [UCon, uForgetFinset.insert]
  exact not_VEnt_insert_top_of_ent p.property.out hEnt hCon

end InfoSys

end DomainTheory.InformationSystems
```

vPairRFinset_mono.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    mem_vPairRFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

theorem vPairRFinset_mono {u v : Finset VToken} (h : u  subseteq  v) :
    vPairRFinset u  subseteq  vPairRFinset v := by
  intro t ht
  exact mem_vPairRFinset.2 (h (mem_vPairRFinset.1 ht))

end InfoSys

end DomainTheory.InformationSystems
```

VEnt_mono.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset
```

```
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset_mono
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset_mono

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

theorem VEnt_mono {u v : Finset VToken} {c : VToken}
    (huv : u  subseteq  v) (h : VEnt u c) : VEnt v c := by
  induction h generalizing v with
  | bot => exact VEnt.bot
  | top_mem hm => exact VEnt.top_mem (huv hm)
  | top_both _ _ ihL ihR =>
    exact VEnt.top_both (ihL (vPairLFinset_mono huv)) (ihR (vPairRFinset_mono huv))
  | pairL_top hm => exact VEnt.pairL_top (huv hm)
  | pairL_proj _ ih => exact VEnt.pairL_proj (ih (vPairLFinset_mono huv))
  | pairR_top hm => exact VEnt.pairR_top (huv hm)
  | pairR_proj _ ih => exact VEnt.pairR_proj (ih (vPairRFinset_mono huv))

end InfoSys

end DomainTheory.InformationSystems
```

uSystem.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    BasicInst33
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.UToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    BasicInst35
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.uBot
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    BasicInst39
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.UCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.UEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    UCon_insert_of_ent

@[expose] public section
```

```
namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

/-- **Scott (7)-(10).** The universal-domain information system `U`. -/
def uSystem : InfoSys UToken where
  bot := uBot
  Con := {u | UCon u}
  Ent := UEnt
  con_subset := fun hu hv => UCon.subset hu hv
  con_sing := UCon.singleton
  ent_con := fun h => UCon_insert_of_ent h
  ent_bot := fun hu => UEnt.bot hu
  ent_refl := fun hu hp => UEnt.of_mem hu hp
  ent_trans := fun hv hu hEnts h => UEnt.trans hv hu hEnts h

end InfoSys

end DomainTheory.InformationSystems
```

vSystem.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.vBot
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VCon

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys

/-- **Scott (1)-(6).** The information system `V` (all finite sets consistent). -/
def vSystem : InfoSys VToken where
  bot := vBot
  Con := {u | VCon u}
  Ent := VEnt
  con_subset := fun _ _ => trivial
  con_sing := fun _ => trivial
  ent_con := fun _ => trivial
  ent_bot := fun _ => VEnt.bot
  ent_refl := fun _ hp => VEnt.of_mem hp
  ent_trans := fun _ _ hEnts h => VEnt.trans hEnts h
```

```
end InfoSys

end DomainTheory.InformationSystems
```

vRhs.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.vSystem

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys
/-- Right-hand side of Scott?s equation `V  ~=  V  x  V`. -/
def vRhs : InfoSys (ProdToken vSystem vSystem) :=
  productSystem vSystem vSystem

theorem vRhs.eq_product : vRhs = productSystem vSystem vSystem := rfl

end InfoSys

end DomainTheory.InformationSystems
```

vTop.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys
```

```
/-- Top / rogue `?`. -/
def vTop : VToken := .top

end InfoSys

end DomainTheory.InformationSystems
```

vUnfold.lean

```
module

public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Product
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VToken
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.VEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairLFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.
    vPairRFinset
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid83.vSystem

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

namespace InfoSys
/-- Unfold product-shaped tokens into `V  x  V`. Top `?` has no single product-token
image (it generates the top *element*); we send it to `(?, Delta)` as a left witness.
    -/
def vUnfold (t : VToken) : ProdToken vSystem vSystem :=
  match t with
  | .bot => prodBot vSystem vSystem
  | .top => <(.top, .bot), <Or.inr rfl>>
  | .pairL x => <(x, .bot), <Or.inr rfl>>
  | .pairR y => <(.bot, y), <Or.inl rfl>>

theorem vUnfold.bot : vUnfold .bot = prodBot vSystem vSystem := rfl

theorem vUnfold.pairL (x : VToken) :
    vUnfold (.pairL x) = <(x, .bot), <Or.inr rfl>> := rfl

theorem vUnfold.pairR (y : VToken) :
    vUnfold (.pairR y) = <(.bot, y), <Or.inl rfl>> := rfl

end InfoSys

end DomainTheory.InformationSystems
```

4.8.4 Factoid 8.4

A domain of domains. Not many details can be presented here, but we would also like to remark that even domains can be made into a domain. One way of getting an idea of how this is possible is to note that since

subdomains of $\mathbf{U}$ correspond to certain kinds of functions on $\mathbf{U}$, and since the function space of $\mathbf{U}$ is also a subdomain of $\mathbf{U}$, it might be suspected that the subdomains of $\mathbf{U}$ form a single subdomain of $\mathbf{U}$.

That is a fairly sophisticated way of reaching the conclusion (and many details have to be worked out). A more elementary approach would be just to ask what it means to give a finite amount of information about a domain. For the sake of uniformity, suppose that the data objects of the possible domain are drawn from the non-negative integers, and that we conventionally use 0 for Δ. Then to give a finite amount of information about a domain is—roughly—to specify a finite part of Con and a finite part of $\vdash$. To make the formulation easier, we will reserve for 1 a rôle like the one recently played by ∇. What the specifications will boil down to is pairs (u, v) of finite sets of integers used as data objects to convey one piece of information about an entailment relation.

But hold, entailment relations are very closely connected to approximable mappings. Indeed, we remarked before that the identity function as an approximable mapping on a domain is just represented as the underlying entailment relation itself. Suppose we take as our domain the domain of all sets of integers. It is a powerset, so call it $\mathbf{P}$. That is to say, the integers are the data objects, all finite sets are consistent, and the entailment relation is the minimal possible one. (As far as elements go, an arbitrary set of integers is equal to the union of all its finite subsets, which means that the elements of the domain are in a one-one correspondence with the arbitrary sets of integers.) The question is: which approximable mappings on $\mathbf{P}$ into itself correspond to entailment relations on the integers as data objects?

The answer can be expressed most succinctly using our standard notation. If we think of $r : \mathbf{P} \to \mathbf{P}$ as a relation between finite sets in the usual way, then to say that r is reflexive is to say:

(1) $I_{\mathbf{P}} \subseteq r$.

To say that r is transitive is to say:

(2) $r \circ r = r$. To say that for r the object 0 plays at being Δ is to say:

$$(3) \quad \bar{0} \subseteq r(\bot),$$

where in general $\bar{n}$ is short for $\{n\}$ in the domain $\mathbf{P}$. Then, to say that 1 plays at being a rogue object is to say:

$$(4) \quad \top = r(\bar{1}).$$

Finally, to say that 1 is an inconsistent object that has to be excluded is to say:

$$(5) \quad \bar{1} \not\subseteq r(\bar{0}).$$

That's it. The collection of approximable mappings satisfying (1)–(5) gives us all the entailment relations we need. Condition (5) is a consistency condition, and for r the consistent finite sets u are those such that $\bar{1} \not\subseteq r(\bar{u})$. What we are asserting is that the totality of r satisfying (1)–(5) forms the elements of a domain—one that has been derived from $(\mathbf{P} \to \mathbf{P})$ in a way similar to the way we derived $\mathbf{U}$ from $\mathbf{V}$ above.

Having made domains into a domain, the next step is to see how constructs on domain (i.e. functors) can be made into approximable mappings. But the retelling and development of that story will have to wait for another publication along with the very interesting chapter on powerdomains. I only hope the ground covered here makes the theory of domains seem more elementary and more natural. ?

PEnt.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Proposition55
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35

@[expose] public section

namespace DomainTheory.InformationSystems

namespace Factoid84

open DomainTheory.InformationSystems.Constructive

open InfoSys

open ApproximableMap

/-- Minimal entailment on `Nat` with `0 = Delta`: `u  |-  n` iff `n = 0` or `n  in  u`.
    -/
def PEnt (u : Finset Nat) (n : Nat) : Prop :=
  n = 0  \/  n  in  u

end Factoid84

end DomainTheory.InformationSystems
```

P.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Proposition55
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.PEnt

@[expose] public section

namespace DomainTheory.InformationSystems
namespace Factoid84

open DomainTheory.InformationSystems.Constructive
open InfoSys
open ApproximableMap

/-- **Powerset information system.** Tokens `Nat`, bot `0`, every finite set
consistent, minimal entailment. -/
def P : InfoSys Nat where
  bot := 0
  Con := Set.univ
  Ent := PEnt
  con_subset := fun _ _ => trivial
  con_sing := fun _ => trivial
```

```
  ent_con := fun _ => trivial
  ent_bot := fun _ => Or.inl rfl
  ent_refl := fun _ ha => Or.inr ha
  ent_trans := by
    intro u v c _ _ hEnts hc
    rcases hc with rfl | hc
    * exact Or.inl rfl
    * exact hEnts c hc

theorem P.AllConsistent : P.AllConsistent := fun _ => trivial

end Factoid84
end DomainTheory.InformationSystems
```

IdemApprox.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Proposition55
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.P

@[expose] public section

namespace DomainTheory.InformationSystems

namespace Factoid84

open DomainTheory.InformationSystems.Constructive

open InfoSys

open ApproximableMap

/-- (2) `r  comp  r = r` (transitivity / idempotence). -/
def IdemApprox (r : ApproximableMap P P) : Prop :=
  ApproximableMap.comp r r = r

end Factoid84

end DomainTheory.InformationSystems
```

ReflApprox.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Proposition55
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.P

@[expose] public section
```

```
namespace DomainTheory.InformationSystems

namespace Factoid84

open DomainTheory.InformationSystems.Constructive

open InfoSys

open ApproximableMap

/-- (1) `I_P  subseteq  r` (reflexivity of the induced entailment). -/
def ReflApprox (r : ApproximableMap P P) : Prop :=
  forall  {u v : Finset Nat}, (idMap P).rel u v  ->  r.rel u v

end Factoid84

end DomainTheory.InformationSystems
```

bar.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Proposition55
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35

@[expose] public section

namespace DomainTheory.InformationSystems

namespace Factoid84

open DomainTheory.InformationSystems.Constructive

open InfoSys

open ApproximableMap

/-- Scott?s shorthand `\bar n = {n}` as a finite set of tokens. -/
def bar (n : Nat) : Finset Nat := {n}

end Factoid84

end DomainTheory.InformationSystems
```

barElement.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Proposition55
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
```

```
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.P
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.bar

@[expose] public section

namespace DomainTheory.InformationSystems

namespace Factoid84

open DomainTheory.InformationSystems.Constructive

open InfoSys

open ApproximableMap

/-- Finite element `\bar n` (entailment closure of `{n}`). -/
def barElement (n : Nat) : P.Element :=
  P.closure (bar n) trivial

end Factoid84

end DomainTheory.InformationSystems
```

ConsistentApprox.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Proposition55
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.P
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.barElement

@[expose] public section

namespace DomainTheory.InformationSystems

namespace Factoid84

open DomainTheory.InformationSystems.Constructive

open InfoSys

open ApproximableMap

/-- (5) `\bar 1 ? r(\bar 0)` -- `1` is excluded as inconsistent at the bottom. -/
def ConsistentApprox (r : ApproximableMap P P) : Prop :=
  not  ((barElement 1).carrier  subseteq  (r.toElement (barElement 0)).carrier)

end Factoid84

end DomainTheory.InformationSystems
```

botElement.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Proposition55
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.P

@[expose] public section

namespace DomainTheory.InformationSystems

namespace Factoid84

open DomainTheory.InformationSystems.Constructive

open InfoSys

open ApproximableMap

/-- Bottom element `False` of `|P|` (closure of `{}`). -/
def botElement : P.Element :=
  P.closure {} trivial

end Factoid84

end DomainTheory.InformationSystems
```

BotApprox.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Proposition55
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.P
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.barElement
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.botElement

@[expose] public section

namespace DomainTheory.InformationSystems

namespace Factoid84

open DomainTheory.InformationSystems.Constructive

open InfoSys

open ApproximableMap

/-- (3) `\bar 0  subseteq  r(False)` -- `0` plays the role of `Delta`. -/
def BotApprox (r : ApproximableMap P P) : Prop :=
  (barElement 0).carrier  subseteq  (r.toElement botElement).carrier
```

```
end Factoid84

end DomainTheory.InformationSystems
```

mem_botElement.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Proposition55
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.
    IsDomainApprox
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.
    DomainApprox
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.scottCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.flatRel
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.flatMap
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.
    flatDomainApprox
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.
    mem_barElement
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.
    mem_topElement
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.
    BotApprox_any

public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35.
    mem_closure_iff
@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive
open InfoSys
open ApproximableMap

namespace Factoid84

theorem mem_botElement (x : Nat) : x  in  botElement.carrier  <->  x = 0 := by
  constructor
  * intro hx
    have : PEnt {} x := (P.mem_closure_iff (by trivial : ({} : Finset Nat)  in  P.Con)
    ).1 hx
    rcases this with rfl | hx'
    * rfl
    * exact False.elim (Finset.notMem_empty _ hx')
  * rintro rfl
    exact (P.mem_closure_iff (by trivial : ({} : Finset Nat)  in  P.Con)).2 (Or.inl
    rfl)

end Factoid84
end DomainTheory.InformationSystems
```

barElement_zero_eq_bot.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Proposition55
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.
    IsDomainApprox
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.
    DomainApprox
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.scottCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.flatRel
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.flatMap
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.
    flatDomainApprox
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.
    mem_barElement
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.
    mem_topElement
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.
    BotApprox_any
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.
    mem_botElement

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive
open InfoSys
open ApproximableMap

namespace Factoid84

theorem barElement_zero_eq_bot : barElement 0 = botElement := by
  apply le_antisymm
  * intro x hx
    rcases (mem_barElement 0 x).1 hx with rfl | rfl
    * exact (mem_botElement 0).2 rfl
    * exact (mem_botElement 0).2 rfl
  * intro x hx
    have : x = 0 := (mem_botElement x).1 hx
    subst this
    exact (mem_barElement 0 0).2 (Or.inl rfl)

end Factoid84
end DomainTheory.InformationSystems
```

not_RogueApprox_idMap.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Proposition55
```

```
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.
    IsDomainApprox
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.
    DomainApprox
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.scottCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.flatRel
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.flatMap
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.
    flatDomainApprox
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.
    mem_barElement
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.
    mem_topElement
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.
    BotApprox_any

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive
open InfoSys
open ApproximableMap

namespace Factoid84

theorem not_RogueApprox_idMap : not  RogueApprox (idMap P) := by
  intro h
  have h2 : 2  in  ((idMap P).toElement (barElement 1)).carrier := by
    rw [h]; exact mem_topElement 2
  rcases (mem_toElement (idMap P) (barElement 1)).1 h2 with <u, hu, hrel>
  have hEnt : P.EntSet u {2} := hrel.2.2
  have h2u := hEnt 2 (Finset.mem_singleton_self 2)
  rcases h2u with h0 | h2u
  * exact absurd h0 (by decide)
  * have : 2  in  (barElement 1).carrier := hu (Finset.mem_coe.2 h2u)
    rcases (mem_barElement 1 2).1 this with h0 | h1'
    * exact absurd h0 (by decide)
    * exact absurd h1' (by decide)

end Factoid84
end DomainTheory.InformationSystems
```

Factoid84.lean

```
module

public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.
    not_RogueApprox_idMap

@[expose] public section

namespace DomainTheory.InformationSystems.Factoid84
```

```
set_option linter.unusedVariables false

theorem theorem_84 : Not
  (DomainTheory.InformationSystems.Factoid84.RogueApprox
    (@DomainTheory.InformationSystems.InfoSys.idMap Nat instDecidableEqNat
    DomainTheory.InformationSystems.Factoid84.P)) :=
  DomainTheory.InformationSystems.Factoid84.not_RogueApprox_idMap

end DomainTheory.InformationSystems.Factoid84
```

scottCon_empty_iff_consistent.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Proposition55
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.
    IsDomainApprox
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.
    DomainApprox
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.scottCon
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.flatRel
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.flatMap
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.
    flatDomainApprox
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.
    mem_barElement
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.
    mem_topElement
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.
    BotApprox_any
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.
    barElement_zero_eq_bot

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive
open InfoSys
open ApproximableMap

namespace Factoid84

theorem scottCon_empty_iff_consistent (r : ApproximableMap P P) :
    scottCon r {}  <->  ConsistentApprox r := by
  simp only [scottCon, ConsistentApprox, barElement_zero_eq_bot]
  rfl

end Factoid84
end DomainTheory.InformationSystems
```

topElement.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Proposition55
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.P

@[expose] public section

namespace DomainTheory.InformationSystems

namespace Factoid84

open DomainTheory.InformationSystems.Constructive

open InfoSys

open ApproximableMap

/-- Top element ` top ` of `|P|` (all tokens). -/
def topElement : P.Element :=
  P.topElement P.AllConsistent

end Factoid84

end DomainTheory.InformationSystems
```

RogueApprox.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Proposition55
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.P
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.barElement
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.topElement

@[expose] public section

namespace DomainTheory.InformationSystems

namespace Factoid84

open DomainTheory.InformationSystems.Constructive

open InfoSys

open ApproximableMap

/-- (4) ` top  = r(\bar 1)` -- `1` plays the role of the rogue `?`. -/
def RogueApprox (r : ApproximableMap P P) : Prop :=
```

```
  r.toElement (barElement 1) = topElement

end Factoid84

end DomainTheory.InformationSystems
```

IsDomainApprox.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Proposition55
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.PEnt
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.P
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.bar
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.barElement
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.botElement
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.topElement
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.ReflApprox
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.IdemApprox
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.BotApprox
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.
    RogueApprox
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.
    ConsistentApprox

@[expose] public section

namespace DomainTheory.InformationSystems
namespace Factoid84

open DomainTheory.InformationSystems.Constructive
open InfoSys
open ApproximableMap

/-- Scott (1)-(5): approximable maps that present entailment relations / domains. -/
structure IsDomainApprox (r : ApproximableMap P P) : Prop where
  refl : ReflApprox r
  idem : IdemApprox r
  bot : BotApprox r
  rogue : RogueApprox r
  consistent : ConsistentApprox r

end Factoid84
end DomainTheory.InformationSystems
```

DomainApprox.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Proposition55
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34
```

```
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.
    IsDomainApprox

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

open InfoSys

open ApproximableMap

namespace Factoid84

/-- Elements of the domain of domains (Scott?s assertion after (1)-(5)). -/
def DomainApprox : Type :=
  { r : ApproximableMap P P // IsDomainApprox r }

end Factoid84

end DomainTheory.InformationSystems
```

flatRel.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Proposition55
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.
    IsDomainApprox

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive
open InfoSys
open ApproximableMap

namespace Factoid84
/-- If `1  in  u`, accept any `v`; else require `v  subseteq  u  U  {0}` and `1  notin
     v`. -/
def flatRel (u v : Finset Nat) : Prop :=
  (1  in  u)  \/  ((forall  n  in  v, n = 0  \/  n  in  u)  /\  1  notin  v)

theorem flatRel.empty : flatRel {} {} :=
  Or.inr <fun _ hn => False.elim (Finset.notMem_empty _ hn), Finset.notMem_empty _>

theorem flatRel.of_mem_one {u v : Finset Nat} (h : 1  in  u) : flatRel u v :=
  Or.inl h
```

```
end Factoid84

end DomainTheory.InformationSystems
```

flatMap.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Proposition55
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.
    IsDomainApprox
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.flatRel
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.
    BotApprox_any
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.
    mem_barElement
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.
    mem_topElement

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive
open InfoSys
open ApproximableMap

namespace Factoid84
def flatMap : ApproximableMap P P where
  rel := flatRel
  rel_dom := fun _ => trivial
  rel_cod := fun _ => trivial
  empty_rel := flatRel.empty
  union_right := by
    intro u v v' hv hv'
    rcases hv with h1 | <hv, hv1>
    * exact Or.inl h1
    * rcases hv' with h1 | <hv', hv1'>
      * exact Or.inl h1
      * refine Or.inr <?_, ?_>
        * intro n hn
          rcases mem_funion.1 hn with h | h
          * exact hv n h
          * exact hv' n h
        * intro h1
          rcases mem_funion.1 h1 with h | h
          * exact hv1 h
          * exact hv1' h
  mono := by
    intro u u' v v' hrel hEntu hEntv _ _
    rcases hrel with h1 | <hv, hv1>
    * -- 1  in  u; show 1  in  u' from EntSet
```

```
      have h := hEntu 1 h1
      rcases h with h0 | h1'
      * exact False.elim (by cases h0)
      * exact Or.inl h1'
    * refine Or.inr <?_, ?_>
      * intro n hn
        have hn' := hEntv n hn
        rcases hn' with rfl | hn'
        * exact Or.inl rfl
        * have hn'' := hv n hn'
          rcases hn'' with rfl | hn''
          * exact Or.inl rfl
          * have hu' := hEntu n hn''
            rcases hu' with rfl | hu'
            * exact Or.inl rfl
            * exact Or.inr hu'
      * intro h1v
        have h := hEntv 1 h1v
        rcases h with h0 | h1v'
        * exact False.elim (by cases h0)
        * exact hv1 h1v'

theorem flatMap.consistent : ConsistentApprox flatMap := by
  intro hsub
  have h1 : 1  in  (barElement 1).carrier := (mem_barElement 1 1).2 (Or.inr rfl)
  have h1' : 1  in  (flatMap.toElement (barElement 0)).carrier := hsub h1
  rcases (mem_toElement flatMap (barElement 0)).1 h1' with <u, hu, hrel>
  rcases hrel with h1u | <_, hv1>
  * have : 1  in  (barElement 0).carrier := hu (Finset.mem_coe.2 h1u)
    rcases (mem_barElement 0 1).1 this with h | h
    * exact absurd h (by decide)
    * exact absurd h (by decide)
  * exact hv1 (Finset.mem_singleton_self 1)

theorem flatMap.idem : IdemApprox flatMap := by
  refine ApproximableMap.ext fun u v => ?_
  constructor
  * rintro <w, hf, hg>
    rcases hf with h1u | <hw, hw1>
    * exact Or.inl h1u
    * rcases hg with h1w | <hv, hv1>
      * exact False.elim (hw1 h1w)
      * refine Or.inr <?_, hv1>
        * intro n hn
          have hn' := hv n hn
          rcases hn' with rfl | hn'
          * exact Or.inl rfl
          * exact hw n hn'
  * intro h
    rcases h with h1u | <hv, hv1>
    * exact <insert 1 v, Or.inl h1u, Or.inl (Finset.mem_insert_self 1 v)>
    * exact <v, Or.inr <hv, hv1>, Or.inr <fun n hn => Or.inr hn, hv1>>

theorem flatMap.refl : ReflApprox flatMap := by
  intro u v <_, _, hEnt>
  by_cases h1 : 1  in  u
```

```
  * exact Or.inl h1
  * refine Or.inr <?_, ?_>
    * intro n hn; exact hEnt n hn
    * intro h1v
      have h := hEnt 1 h1v
      rcases h with h0 | h
      * exact absurd h0 (by decide)
      * exact h1 h

theorem flatMap.toElement_of_mem_one {x : P.Element} (h1 : 1  in  x.carrier) :
    flatMap.toElement x = topElement := by
  apply le_antisymm
  * intro _ _; exact mem_topElement _
  * intro n _
    refine <{1}, ?_, Or.inl (Finset.mem_singleton_self 1)>
    intro a ha
    have : a = 1 := Finset.mem_singleton.mp (Finset.mem_coe.1 ha)
    subst this; exact h1

theorem flatMap.rogue : RogueApprox flatMap :=
  flatMap.toElement_of_mem_one ((mem_barElement 1 1).2 (Or.inr rfl))

theorem flatMap.isDomain : IsDomainApprox flatMap where
  refl := flatMap.refl
  idem := flatMap.idem
  bot := BotApprox_any flatMap
  rogue := flatMap.rogue
  consistent := flatMap.consistent

end Factoid84

end DomainTheory.InformationSystems
```

mem_barElement.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Proposition55
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.
    IsDomainApprox

public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35.
    mem_closure_iff
@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

open InfoSys

open ApproximableMap
```

```
namespace Factoid84

theorem mem_barElement (n x : Nat) :
    x  in  (barElement n).carrier  <->  x = 0  \/  x = n := by
  constructor
  * intro hx
    have : PEnt (bar n) x := (P.mem_closure_iff (by trivial : bar n  in  P.Con)).1 hx
    rcases this with rfl | hx'
    * exact Or.inl rfl
    * exact Or.inr (Finset.mem_singleton.mp hx')
  * intro h
    rcases h with rfl | hn
    * exact (P.mem_closure_iff (by trivial : bar n  in  P.Con)).2 (Or.inl rfl)
    * rw [hn]
      exact (P.mem_closure_iff (by trivial : bar n  in  P.Con)).2
        (Or.inr (Finset.mem_singleton_self n))

end Factoid84

end DomainTheory.InformationSystems
```

BotApprox_any.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Proposition55
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.
    IsDomainApprox
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.
    mem_barElement

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

open InfoSys

open ApproximableMap

namespace Factoid84

theorem BotApprox_any (r : ApproximableMap P P) : BotApprox r := by
  intro x hx
  have hx0 : x = 0 := by
    rcases (mem_barElement 0 x).1 hx with rfl | rfl <;> rfl
  subst hx0
  refine <{}, fun a ha => False.elim (Finset.notMem_empty a (Finset.mem_coe.1 ha)), ?_
    >
  have hempty := r.empty_rel
```

```
  have hbot : P.EntSet {} ({0} : Finset Nat) := by
    intro z hz
    rw [Finset.mem_singleton] at hz; subst hz
    exact Or.inl rfl
  exact r.mono hempty (P.entSet_empty {}) hbot trivial

end Factoid84

end DomainTheory.InformationSystems
```

mem_topElement.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Proposition55
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.
    IsDomainApprox

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

open InfoSys

open ApproximableMap

namespace Factoid84

theorem mem_topElement (x : Nat) : x  in  topElement.carrier := trivial

end Factoid84

end DomainTheory.InformationSystems
```

flatDomainApprox.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Proposition55
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.
    IsDomainApprox
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.
    DomainApprox
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.flatMap

@[expose] public section
```

```
namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

open InfoSys

open ApproximableMap

namespace Factoid84

/-- Witness that Scott?s (1)-(5) are inhabited. -/
def flatDomainApprox : DomainApprox :=
  <flatMap, flatMap.isDomain>

end Factoid84

end DomainTheory.InformationSystems
```

scottCon.lean

```
module

public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Proposition55
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid34
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid35
public import DomainTheory.InformationSystems.SomeDomainEquations.Factoid84.
    IsDomainApprox

@[expose] public section

namespace DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

open InfoSys

open ApproximableMap

namespace Factoid84

/-- Scott?s consistency remark: for such an `r`, a finite `u` is ?consistent?
when `\bar 1 ? r(\bar u)`. -/
def scottCon (r : ApproximableMap P P) (u : Finset Nat) : Prop :=
  not  ((barElement 1).carrier  subseteq
      (r.toElement (P.closure u (by trivial))).carrier)

end Factoid84

end DomainTheory.InformationSystems
```

5 Bridge Theorems

Dana Scott developed three presentations of the same class of domains: **continuous lattices** (1972), **neighbourhood systems** (PRG-19 / 1980–81), and **information systems** (1982). In this chapter we supply **bridge theorems**: order isomorphisms and constructivity audits showing that the three presentations determine the same domains (up to isomorphism), including products, separated sums, and function spaces at the information-system level, with a transport of 1972 Scott maps along the round presentation.

The Lean development is sorry-free. The continuous-lattice identification `D ≃o RoundFilter` audits to `{propext, Quot.sound}`. Every InfoSys-facing declaration inherits `Classical.choice` from Mathlib 4.33's `Finset` instances, even when the proof term extracts no witness. Genuine classical frontiers remain Scott's 1972 topological definition of $\ll$ (algebraic $\Rightarrow$ continuous; ScottMap conjugation) and trichotomy for separated sums.

Dependencies. Each box is one folder. BridgeTheorems is the Lean files in that folder, not in a subfolder. The arrowhead points at the folder that imports, and the number is how many direct imports. A pair of arrows between the same boxes is a cycle.

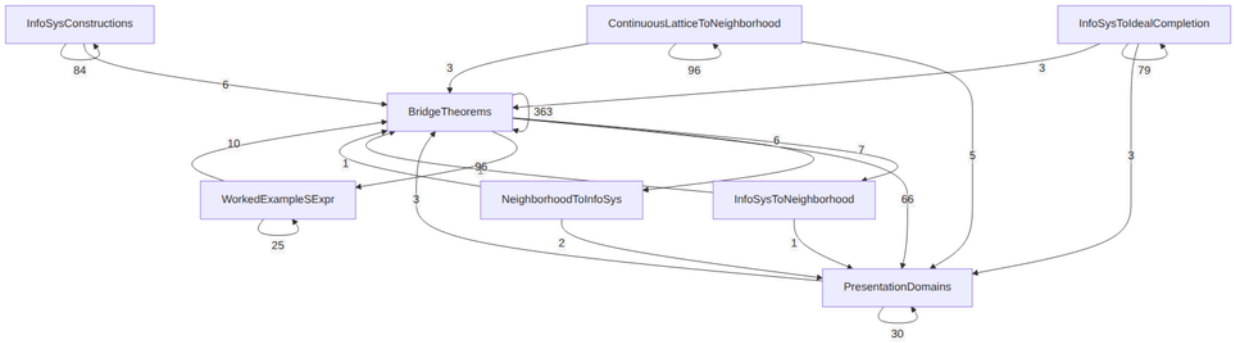


Figure 46: Bridge Theorems.

5.1 Introduction

Neighbourhood systems and information systems are equivalent in a precise sense. The mathematical folklore is stronger still: all three presentations carve out the same class of domains, related by ideal completion and the Scott topology. That equivalence is already known; this article is the write-up of a machine-checked proof of the named bridges, not a first presentation of the result. The three presentations were formalized in separate libraries; the maps between them are the content of this package.

We do **not** re-prove Scott's internal theorems; we import the finished sibling packages and build cross-presentation maps:

Presentation	Lean package	Characteristic object
Continuous lattices [1]	`scott1972`	`ContinuousLattice D`, way-below $\ll$, `ScottMap`

Presentation	Lean package	Characteristic object
Neighbourhood systems [2]	`scott1980`	`NeighborhoodSystem`, filters as domain elements
Information systems [3]	`scott1982`	`InfoSys`, elements as consistent closed token sets

The main subtlety is that a naïve reading of "domains = filters of neighbourhoods" overstates the continuous-lattice corner: for a continuous lattice `D`, the filter `{⇑a | a ≤ x}` has retract `x` but properly contains the principal round filter `{⇑a | a ≪ x}`. The correct identification is **`D ≃o RoundFilter`**, not raw $|\mathcal{D}|$.

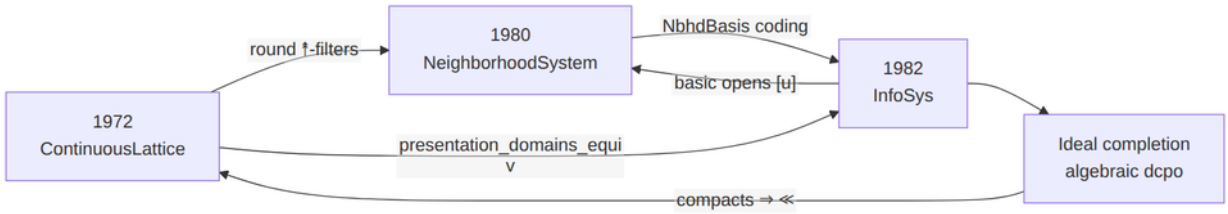


Figure 47: Introduction.

We prove **order isomorphisms of domains** (and of the named construction objects), not a 2-categorical equivalence of the full categories of continuous lattices / neighbourhood systems / information systems with all morphisms. Morphisms are linked where the sibling packages already supply them:

- approximable maps ↔ Scott-continuous maps on $|A|$ (Factoid 4.6);
- Scott maps of continuous lattices ↔ their conjugates on the round presentation.

A full functorial equivalence (preserving products, exponentials, and inverse limits simultaneously across all three presentations) would require additional coherence theorems.

5.2 Catalog of bridge theorems

Theorem	Direction	Lean module	Scope
`continuousLattice_to_neighborhoodSystem`	1972 → 1980	`ContinuousLatticeToNeighborhood.lean`	library
`neighborhoodSystem_to_infoSys`	1980 → 1982	`NeighborhoodToInfoSys.lean`	compared
`infoSys_to_neighborhoodSystem`	1982 → 1980	`InfoSysToNeighborhood.lean`	compared
`infoSys_to_idealCompletion`	1982 → algebraic	`InfoSysToIdealCompletion.lean`	library

Theorem	Direction	Lean module	Scope
`idealCompletion_to_continuousLattice`	algebraic → 1972	`IdealCompletionToContinuousLattice.lean`	library
`presentation_domains_equiv`	three-way	`PresentationDomains.lean`	compared
`infoSys_constructions_equiv`	constructions	`InfoSysConstructions.lean`, `ScottMapBridge.lean`	library
`sexNeighborhoodIso` / `sexIdealIso` / `sexDomainEquationIso`	S-expression instance	`WorkedExampleSExpr.lean`, `SexDomainEquation.lean`	compared

The sibling packages are **finished dependencies**, not work items of this paper: `scott1972`, `scott1980`, and `scott1982` (information systems through Factoid 8.4 / domain equations). They are vendored in `vendor/` at frozen SHAs (see `vendor/FROZEN.txt` / `lakefile.toml`), with the local modifications recorded there and in `formalization.yaml`; the remotes remain the per-paper homes. The compared family (Palomar `Challenge.lean` / `comparator.json`) is the 1980 ↔ 1982 coding, the round three-way identification, and the Factoid 8.1 instance; the other catalog rows are proved in `ScottModels/` but are not Comparator targets. This package was verified and registered in the Palomar registry as PALOMAR-2026-08-25-000003 v1 [4]. What Palomar verified is the claim:

> Order isomorphisms relating Scott's three presentations of domains — continuous lattices (1972), neighbourhood systems (PRG-19), and information systems (1982) — including the round identification `D ≃o RoundFilter ≃o RoundInfoSysElement` and the S-expression equation `T ≅ A + (T × T)` as `|T| ≃o |A + (T × T)|` (`SexSys.Element ≃o SexRhs.Element`).

This document (`view.pdf`) is the narrative account, not a formalization of the three papers. The module map below is left-to-right: sibling packages on the left, local `ScottModels/` modules on the right, with arrows for **direct** imports (transitive imports inside the siblings are omitted). `Equivalence` re-exports the bridges; only the `ScottMapBridge → Equivalence` edge is drawn to avoid clutter. Root `ScottModels.lean` also opens 1972 `ContinuousLattice.Specialization`, 1980 `Neighborhood.Basic`, and 1982 `InfoSys`.

5.3 Proof notes

5.3.1 Continuous lattices → neighbourhood systems

Claim. For a continuous lattice `D`, the sets `⇑a = {z | a ≪ z}` form a `NeighborhoodSystem` on token type `D`, and under `ContinuousLattice` one has an order isomorphism `D ≃o RoundFilter`.

Construction. `wayBelowUp a` is upward closed; `⇑⊥ = univ`; `⇑a ∩ ⇑b = ⇑(a ⊔ b)` (interpolation / directedness of way-below). Principal filters `toFilter x = {⇑a | a ≪ x}` are round: membership of `⇑a` yields, by interpolation in a continuous lattice, some `b` with `a ≪ b ≪ x`, hence `⇑b ∈ toFilter x`.

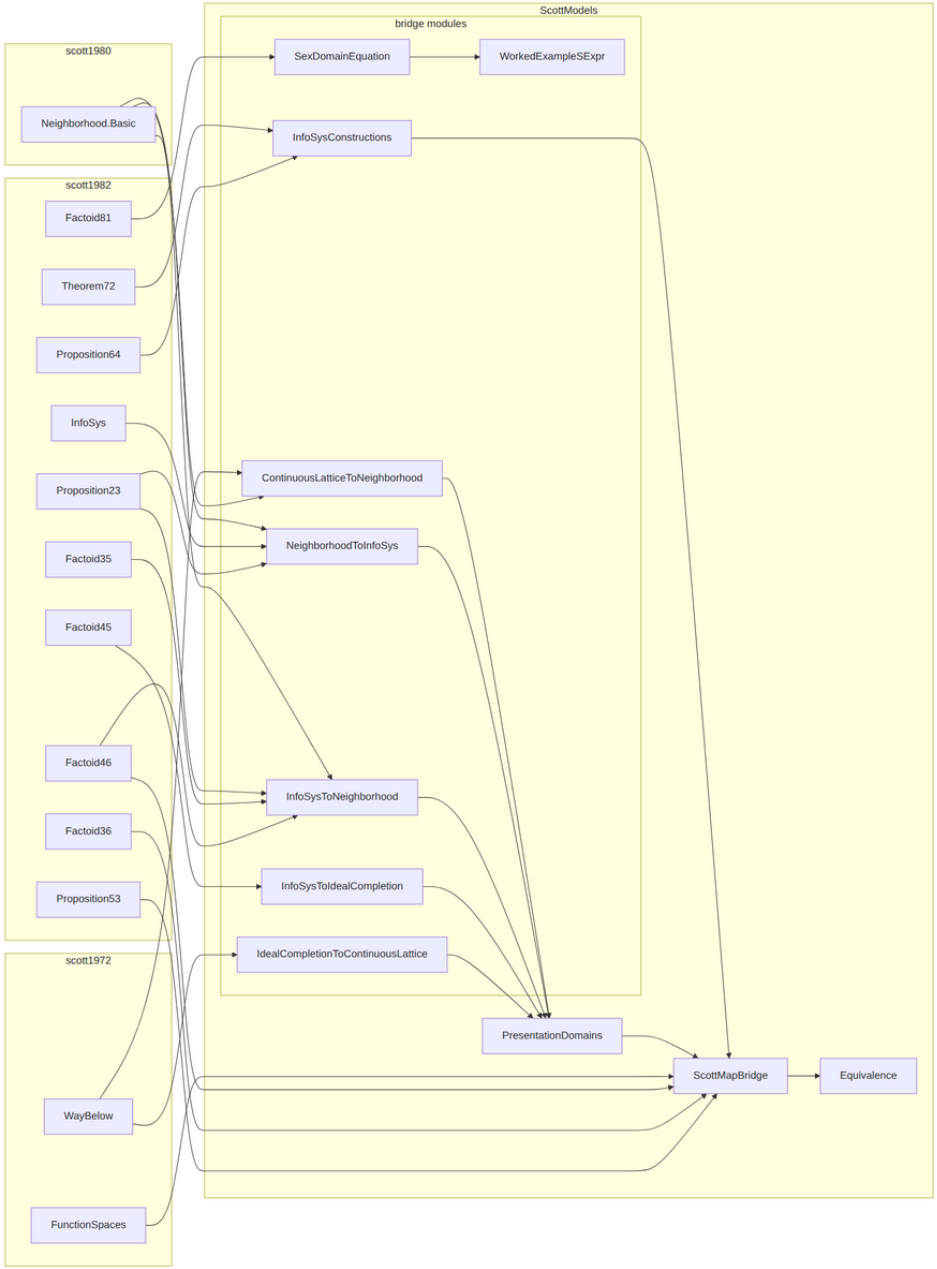


Figure 48: Lean module map.

Why raw $|\mathcal{D}|$ fails. The larger filter {⇑a | a ≤ x} still has ofFilter = x (supremum of codes), but is not equal to toFilter x whenever there are elements way-below strictly below x. Roundness (⇑a ∈ f ⇒ ∃ b, a ≪ b ∧ ⇑b ∈ f) cuts exactly to the image of toFilter.

Axioms. domainOrderIso : D ≃o RoundFilter audits to {propext, Quot.sound}.

5.3.2 Neighbourhood systems → information systems

Claim. A neighbourhood system equipped with a decidable exhaustive coding NbhdBasis ι α of its neighbourhood family $\mathcal{D}$ induces an InfoSys on tokens ι with $|\mathcal{D}|$ ≃o the InfoSys domain.

Construction. Tokens are neighbourhood indices. Consistency of a finite u ⊆ ι is membership of ⋂_ {i∈u} nbhd i in $\mathcal{D}$ (empty intersection = master set Δ). Entailment u ⊢ j means the intersection is a neighbourhood contained in nbhd j. Filters of the neighbourhood system correspond to InfoSys elements via the coding.

Constructivity. DecidableEq ι is required so InfoSys can use the choice-free Finset prelude from scott1982. The proof of ent_con avoids classical by_cases / em. The maps extract no witnesses; the kernel audit still reports inherited Classical.choice from the Mathlib 4.33 Finset instance frontier (§4).

5.3.3 Information systems → neighbourhood systems

Claim. For an information system A, the basic opens [u] = {x ∈ |A| | ↑u ⊆ x} (u ∈ Con) form a NeighborhoodSystem on |A|, and filters recover elements: |A| ≃o the filter domain.

Proof note. Scott's Factoid 4.6 supplies the basic-open vocabulary. The Lean proof initially pulled Classical.choice through simp on basicOpen_empty / finset unions; those simps were removed so the construction extracts no witnesses. The kernel audit still reports inherited Classical.choice through InfoSys/Finset (§4). Together with §3.2 this is the **1980 ↔ 1982** equivalence under coding.

5.3.4 Information systems → ideal completion

Claim. |A| ≃o Ideal (FiniteElement A), where finite elements are closures ū of consistent finsets (Factoid 3.5).

Construction. toIdeal x is the ideal of finite approximants of x; ofIdeal takes directed suprema of finite elements (1982 Factoids 4.4–4.5: directedSup, eq_directedSup_finiteApproximants, compact_closure). The maps extract no witnesses; the kernel audit inherits Classical.choice through InfoSys/Finset (§4).

5.3.5 Algebraic complete lattices → continuous lattices

Claim. If every element of a complete lattice is the directed supremum of compact elements below it (AlgebraicLattice), then ContinuousLattice holds.

Proof note. Order-theoretic compactness implies Set.Ici k is Scott-open, hence k ≪ y whenever k ≤ y. Algebraicity then yields y = ⊔{k compact | k ≤ y} ⊆ ⊔{x | x ≪ y}. This is the **classical frontier**: Scott's ≪ is defined topologically in scott1972, so the footprint inherits that classicality even though the order argument is elementary.

5.3.6 Three-presentation equivalence (`presentation_domains_equiv`)

Claim. Under `ContinuousLattice D` and `DecidableEq D`,

```
D ?o RoundFilter ?o RoundInfoSysElement
```

where `RoundInfoSysElement` is the subtype of elements of the InfoSys coded by `wayBelowNbhdBasis` (tokens = elements of `D`, neighbourhoods = $\Uparrow$`a`) that correspond to round filters. An extended form routes through the ideal-completion subtype.

Glue. `wayBelowNbhdBasis` packages the $\Uparrow$-system as an `NbhdBasis`; `roundFilter_infoSys_iso` transports roundness along `NbhdBasis.domainOrderIso`; `presentation_domains_equiv` is the composite with `continuousLattice_roundFilter_iso`.

What is not claimed. Raw `|`$\mathcal{D}$`|` and the full InfoSys domain `|A|` remain properly larger than `D`. The equivalence is on the **round** corner that matches continuous lattice points.

Axioms. Under Mathlib 4.33 the kernel audit includes `{propext, Classical.choice, Quot.sound}` through the inherited InfoSys/`Finset` instance frontier described in §4; the round-filter construction itself performs no choice-based witness extraction.

5.3.7 Constructions (`infoSys_constructions_equiv`)

Products `|A| × |B| ≃o |A × B|` via 1982 `pairElements` / `fstMap` / `sndMap` (Prop 6.2). The maps use explicit projections and assembly, with no witness selection.

Separated sums `WithBot (|A| ⊕ |B|) ≃o |A + B|` via `inl` / `inr` classify-and-assemble (Prop 6.4). Trichotomy on token polarity uses classical case-split (`Classical.choice` in the footprint).

Function spaces `ApproximableMap A B ≃o |A → B|` packages Theorem 7.2 (`approxMap_toElement` / `element_toApproxMap`) with pointwise `Le` as `PartialOrder`. No additional choice is used beyond the inherited InfoSys/`Finset` instance frontier.

Factoid 4.6 bridge `ApproximableMap A B ≃o ScottContinuous A B` via `toScottContinuous` / `ofScottContinuous`, using Prop 5.3(v) (`rel_iff_closure_le`) and `closure_le_element` for the round-trip on relations. The relation-level proof performs no witness extraction; its kernel audit inherits the InfoSys/`Finset` frontier in §4.

ScottMap conjugation A 1972 `ScottMap D E` conjugates along any pair of order isomorphisms ι`D : D ≃o D'`, ι`E : E ≃o E'` to a pointwise-ordered map `D' → E'`. Specializing to `continuousLattice_roundFilter_iso` / `presentation_domains_equiv` yields `scottMap_roundFilter_iso` and `scottMap_roundInfoSys_iso`. The conjugation itself is order-theoretic; the footprint includes `Classical.choice` because `ScottMap` is defined via Scott continuity in `scott1972`.

Out of scope (documented). Identifying `ApproximableMap` on the coded $\Uparrow$-InfoSys with `ScottMap D E` (needs roundness preservation of approximable maps), and relating `wayBelow (D × E)` to the InfoSys product of factors (cylinder basis).

`ConjScottMap.lean`

```
module
public import Mathlib.Order.BourbakiWitt
public import DomainTheory.ContinuousLattice.FunctionSpaces.ScottMap
```

```
public import Mathlib.Order.Hom.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.Basic
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottFunctionSubbasis_isOpen
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottFunctionTopology_eq_induced_pi
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    specializationLe_generateOpen
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottFunctionSpecializationLe
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_3_2
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSupMaps_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_apply_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_le_of_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.stepMap_pointwise_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.theorem_3_3
public import DomainTheory.ContinuousLattice.FunctionSpaces.wayBelow_finset_sup
public import DomainTheory.ContinuousLattice.FunctionSpaces.pointwiseSubbasic_isOpen
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottMapPointwiseTopology_eq_induced_pi
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    pointwiseSubbasic_scottOpen
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    wayBelow_le_finset_sup_step
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottFunctionSubbasis_isOpen_scott
public import DomainTheory.ContinuousLattice.FunctionSpaces.corollary_3_4_eval_on_C
public import DomainTheory.ContinuousLattice.FunctionSpaces.corollary_3_4
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    corollary_3_4_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    corollary_3_4_jointly_continuous
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    proposition_3_5_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_3_5
public import DomainTheory.ContinuousLattice.FunctionSpaces.retr_ambientSSup_eq_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_preservesDirectedSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    retr_wayBelow_of_wayBelow_incl
public import DomainTheory.ContinuousLattice.FunctionSpaces.sSup_image_retr_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    image_retr_wayBelow_nonempty
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    image_retr_wayBelow_directed
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_2_10_a
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_2_10_b
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_2_10
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_injective
public import DomainTheory.ContinuousLattice.FunctionSpaces.retr_bot
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_bot
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sup
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_finset_sup
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.incl_wayBelow
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_3_10_forward
```

```
public import DomainTheory.ContinuousLattice.FunctionSpaces.retr_eq_sSup
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_3_10_converse
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_le_of_mem
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    sInf_image_le_sInf_image_of_subset
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottMap_eq_sSup_openInfs
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_eq_of_comp
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtend_maximal
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono
public import DomainTheory.ContinuousLattice.FunctionSpaces.scottSubspaceExtend_mono
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_eq_of_ext
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtend_eq_of_ext
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono_retr
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono_incl
public import DomainTheory.ContinuousLattice.FunctionSpaces.
    scottSubspaceExtendInf_mono_incl_apply
public import DomainTheory.ContinuousLattice.FunctionSpaces.lemma_3_9_incl_inf
public import DomainTheory.ContinuousLattice.FunctionSpaces.lemma_3_9_retr_inf
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_3_12
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_3_13
public import DomainTheory.ContinuousLattice.FunctionSpaces.fix_unique
public import DomainTheory.ContinuousLattice.FunctionSpaces.proposition_3_14
public import DomainTheory.InformationSystems.DomainsAsLatticesAndAsTopologicalSpaces.
    Factoid46
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid36
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Proposition53
public import DomainTheory.BridgeTheorems.InfoSysConstructions
public import DomainTheory.BridgeTheorems.PresentationDomains

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems
open DomainTheory.InformationSystems.Constructive
open DomainTheory.InformationSystems.InfoSys
open DomainTheory.InformationSystems.InfoSys.ApproximableMap
open DomainTheory.ContinuousLattice
open ContinuousLatticeToNeighborhood

variable {a b : Type*} [DecidableEq a] [DecidableEq b]

variable {D E : Type*} [CompleteLattice D] [CompleteLattice E]

/-- Scott maps packaged as their conjugates along given presentation isos. -/
structure ConjScottMap {D' E' : Type*} [PartialOrder D'] [PartialOrder E']
    (?D : D ?o D') (?E : E ?o E') where
```

```
  scott : ScottMap D E

end DomainTheory.BridgeTheorems
```

wayBelowUp.lean

```
module
public import Mathlib.Order.Hom.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpenu2eisUpperSet
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_subset
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_Ici_subset
public import DomainTheory.ContinuousLattice.WayBelow.sInf_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.
    isContinuousLattice_iff_isLUB_sInf_nhds
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_23

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.ContinuousLattice

open DomainTheory.NeighbourhoodSystems

open scoped DomainTheory.ContinuousLattice

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

/-- `Up a = { z | a  <<  z }`. -/
def wayBelowUp (a : D) : Set D := {z | a  <<  z}

end ContinuousLatticeToNeighborhood

end DomainTheory.BridgeTheorems
```

wayBelowUp_anti.lean

```
module

public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.wayBelowUp
public import Mathlib.Order.Hom.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
```

```
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpenu2eisUpperSet
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_subset
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_Ici_subset
public import DomainTheory.ContinuousLattice.WayBelow.sInf_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.
    isContinuousLattice_iff_isLUB_sInf_nhds
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_23

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.ContinuousLattice

open DomainTheory.NeighbourhoodSystems

open scoped DomainTheory.ContinuousLattice

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

theorem wayBelowUp_anti {a b : D} (hab : a  <=  b) :
    wayBelowUp b  subseteq  wayBelowUp a :=
  fun _ h => WayBelow.le_trans hab h

end ContinuousLatticeToNeighborhood

end DomainTheory.BridgeTheorems
```

wayBelowUp_bot.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.wayBelowUp
public import Mathlib.Order.Hom.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpenu2eisUpperSet
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_subset
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_Ici_subset
public import DomainTheory.ContinuousLattice.WayBelow.sInf_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.
    isContinuousLattice_iff_isLUB_sInf_nhds
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_23
```

```
@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.ContinuousLattice

open DomainTheory.NeighbourhoodSystems

open scoped DomainTheory.ContinuousLattice

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

theorem wayBelowUp_bot : wayBelowUp (False : D) = (Set.univ : Set D) := by
  ext z
  exact <fun _ => trivial, fun _ => bot_wayBelow z>

end ContinuousLatticeToNeighborhood

end DomainTheory.BridgeTheorems
```

wayBelowUp_inter.lean

```
module

public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.wayBelowUp
public import Mathlib.Order.Hom.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpenu2eisUpperSet
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_subset
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_Ici_subset
public import DomainTheory.ContinuousLattice.WayBelow.sInf_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.
    isContinuousLattice_iff_isLUB_sInf_nhds
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_23

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.ContinuousLattice

open DomainTheory.NeighbourhoodSystems

open scoped DomainTheory.ContinuousLattice
```

```
namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

theorem wayBelowUp_inter (a b : D) :
    wayBelowUp a  I  wayBelowUp b = wayBelowUp (a  sqsup  b) := by
  ext z
  constructor
  * exact fun <ha, hb> => WayBelow.sup ha hb
  * intro h
    exact <WayBelow.le_trans le_sup_left h, WayBelow.le_trans le_sup_right h>

end ContinuousLatticeToNeighborhood

end DomainTheory.BridgeTheorems
```

toNeighborhoodSystem.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.wayBelowUp
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    wayBelowUp_inter
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    wayBelowUp_bot
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_23
public import Mathlib.Order.Hom.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpenu2eisUpperSet
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_subset
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_Ici_subset
public import DomainTheory.ContinuousLattice.WayBelow.sInf_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.
    isContinuousLattice_iff_isLUB_sInf_nhds

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.ContinuousLattice

open DomainTheory.NeighbourhoodSystems

open scoped DomainTheory.ContinuousLattice

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]
```

```
/-- Neighbourhood system of all ` Up a`. -/
def toNeighborhoodSystem : NeighborhoodSystem D where
  mem X := exists  a : D, X = wayBelowUp a
  master := Set.univ
  master_nonempty := <False, Set.mem_univ _>
  master_mem := <False, wayBelowUp_bot.symm>
  inter_mem := by
    intro X Y _Z hX hY _hZ _hZsub
    obtain <a, rfl> := hX
    obtain <b, rfl> := hY
    exact <a  sqsup  b, wayBelowUp_inter a b>
  sub_master := fun {_} _ => Set.subset_univ _

end ContinuousLatticeToNeighborhood

end DomainTheory.BridgeTheorems
```

Filter.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    toNeighborhoodSystem
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_23
public import Mathlib.Order.Hom.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpenu2eisUpperSet
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_subset
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_Ici_subset
public import DomainTheory.ContinuousLattice.WayBelow.sInf_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.
    isContinuousLattice_iff_isLUB_sInf_nhds

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.ContinuousLattice

open DomainTheory.NeighbourhoodSystems

open scoped DomainTheory.ContinuousLattice

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]
```

```
/-- Filter of the ` Up `-neighbourhood system on `D`. -/
abbrev Filter : Type _ :=
  (toNeighborhoodSystem : NeighborhoodSystem D).Element

end ContinuousLatticeToNeighborhood

end DomainTheory.BridgeTheorems
```

Round.lean

```
module

public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.Filter
public import Mathlib.Order.Hom.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpenu2eisUpperSet
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_subset
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_Ici_subset
public import DomainTheory.ContinuousLattice.WayBelow.sInf_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.
    isContinuousLattice_iff_isLUB_sInf_nhds
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_23
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.wayBelowUp

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.ContinuousLattice

open DomainTheory.NeighbourhoodSystems

open scoped DomainTheory.ContinuousLattice

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

/-- Roundness: ` Up a  in  f` is witnessed by a finer code `b` with `a  <<  b`. -/
class Round (f : Filter (D := D)) : Prop where
  out : forall  {a : D}, f.mem (wayBelowUp a)  ->  exists  b : D, a  <<  b  /\  f.mem
    (wayBelowUp b)

end ContinuousLatticeToNeighborhood

end DomainTheory.BridgeTheorems
```

RoundFilter.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.Round
public import Mathlib.Order.Hom.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpenu2eisUpperSet
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_subset
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_Ici_subset
public import DomainTheory.ContinuousLattice.WayBelow.sInf_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.
    isContinuousLattice_iff_isLUB_sInf_nhds
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_23
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.Filter

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.ContinuousLattice

open DomainTheory.NeighbourhoodSystems

open scoped DomainTheory.ContinuousLattice

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

section Continuous

variable (hD : ContinuousLattice D)

include hD

/-- Round filters of the ` Up `-system. -/
abbrev RoundFilter : Type _ :=
  { f : Filter (D := D) // Round f }

end Continuous

end ContinuousLatticeToNeighborhood

end DomainTheory.BridgeTheorems
```

codes.lean

```
module
```

```
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.Filter
public import Mathlib.Order.Hom.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpenu2eisUpperSet
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_subset
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_Ici_subset
public import DomainTheory.ContinuousLattice.WayBelow.sInf_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.
    isContinuousLattice_iff_isLUB_sInf_nhds
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_23
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.wayBelowUp

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.ContinuousLattice

open DomainTheory.NeighbourhoodSystems

open scoped DomainTheory.ContinuousLattice

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

/-- Codes whose ` Up `-neighbourhoods lie in the filter. -/
def codes (f : Filter (D := D)) : Set D :=
  {a : D | f.mem (wayBelowUp a)}

end ContinuousLatticeToNeighborhood

end DomainTheory.BridgeTheorems
```

bot_mem_codes.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.codes
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    wayBelowUp_bot
public import Mathlib.Order.Hom.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpenu2eisUpperSet
```

```
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_subset
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_Ici_subset
public import DomainTheory.ContinuousLattice.WayBelow.sInf_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.
    isContinuousLattice_iff_isLUB_sInf_nhds
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_23
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.wayBelowUp
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.Filter

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.ContinuousLattice

open DomainTheory.NeighbourhoodSystems

open scoped DomainTheory.ContinuousLattice

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

theorem bot_mem_codes (f : Filter (D := D)) : (False : D)  in  codes f := by
  have : f.mem (wayBelowUp (False : D)) := by
    rw [wayBelowUp_bot]
    exact f.master_mem
  exact this

end ContinuousLatticeToNeighborhood

end DomainTheory.BridgeTheorems
```

codes_directed.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.codes
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    wayBelowUp_inter
public import Mathlib.Order.Hom.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpenu2eisUpperSet
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_subset
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_Ici_subset
public import DomainTheory.ContinuousLattice.WayBelow.sInf_wayBelow
```

```
public import DomainTheory.ContinuousLattice.WayBelow.
    isContinuousLattice_iff_isLUB_sInf_nhds
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_23
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.wayBelowUp
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.Filter

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.ContinuousLattice

open DomainTheory.NeighbourhoodSystems

open scoped DomainTheory.ContinuousLattice

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

theorem codes_directed (f : Filter (D := D)) : DirectedOn (*  <=  *) (codes f) := by
  intro a ha b hb
  refine <a  sqsup  b, ?_, le_sup_left, le_sup_right>
  have : f.mem (wayBelowUp a  I  wayBelowUp b) := f.inter_mem ha hb
  rwa [wayBelowUp_inter] at this

end ContinuousLatticeToNeighborhood

end DomainTheory.BridgeTheorems
```

codes_nonempty.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.codes
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    bot_mem_codes
public import Mathlib.Order.Hom.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpenu2eisUpperSet
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_subset
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_Ici_subset
public import DomainTheory.ContinuousLattice.WayBelow.sInf_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.
    isContinuousLattice_iff_isLUB_sInf_nhds
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_23
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.Filter
```

```
@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.ContinuousLattice

open DomainTheory.NeighbourhoodSystems

open scoped DomainTheory.ContinuousLattice

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

theorem codes_nonempty (f : Filter (D := D)) : (codes f).Nonempty :=
  <False, bot_mem_codes f>

end ContinuousLatticeToNeighborhood

end DomainTheory.BridgeTheorems
```

ofFilter.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.codes
public import Mathlib.Order.Hom.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpenu2eisUpperSet
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_subset
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_Ici_subset
public import DomainTheory.ContinuousLattice.WayBelow.sInf_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.
    isContinuousLattice_iff_isLUB_sInf_nhds
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_23
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.Filter

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.ContinuousLattice

open DomainTheory.NeighbourhoodSystems

open scoped DomainTheory.ContinuousLattice
```

```
namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

/-- Retraction: ` sqsup ` of codes present in the filter. -/
noncomputable def ofFilter (f : Filter (D := D)) : D :=
  sSup (codes f)

end ContinuousLatticeToNeighborhood

end DomainTheory.BridgeTheorems
```

mem_wayBelowUp_ofFilter_of_round.lean

```
module

public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.Round
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.ofFilter
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    codes_directed
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    codes_nonempty
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    wayBelowUp_anti
public import Mathlib.Order.Hom.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpenu2eisUpperSet
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_subset
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_Ici_subset
public import DomainTheory.ContinuousLattice.WayBelow.sInf_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.
    isContinuousLattice_iff_isLUB_sInf_nhds
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_23
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.wayBelowUp
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.Filter

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.ContinuousLattice

open DomainTheory.NeighbourhoodSystems

open scoped DomainTheory.ContinuousLattice

namespace ContinuousLatticeToNeighborhood
```

```
variable {D : Type*} [CompleteLattice D]

theorem mem_wayBelowUp_ofFilter_of_round {f : Filter (D := D)} (hr : Round f) {a : D}
    :
    f.mem (wayBelowUp a)  <->  a  <<  ofFilter f := by
  constructor
  * intro ha
    obtain <b, hab, hb> := hr.out ha
    exact (WayBelow.sSup_iff (codes_nonempty f) (codes_directed f)).2 <b, hb, hab>
  * intro ha
    obtain <b, hbS, hab> := (WayBelow.sSup_iff (codes_nonempty f) (codes_directed f))
    .1 ha
    exact f.up_mem hbS <a, rfl> (wayBelowUp_anti hab.le)

end ContinuousLatticeToNeighborhood

end DomainTheory.BridgeTheorems
```

toFilter.lean

```
module

public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.Filter
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    wayBelowUp_inter
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    wayBelowUp_bot
public import Mathlib.Order.Hom.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpenu2eisUpperSet
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_subset
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_Ici_subset
public import DomainTheory.ContinuousLattice.WayBelow.sInf_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.
    isContinuousLattice_iff_isLUB_sInf_nhds
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_23
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.wayBelowUp

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.ContinuousLattice

open DomainTheory.NeighbourhoodSystems

open scoped DomainTheory.ContinuousLattice
```

```
namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

/-- Principal filter of ` Up `-neighbourhoods at `x`. -/
def toFilter (x : D) : Filter (D := D) where
  mem U := exists  a : D, U = wayBelowUp a  /\  a  <<  x
  sub := by
    intro U h
    obtain <a, <rfl, _>> := h
    exact <a, rfl>
  master_mem := <False, wayBelowUp_bot.symm, bot_wayBelow x>
  inter_mem := by
    intro U V hU hV
    obtain <a, <rfl, ha>> := hU
    obtain <b, <rfl, hb>> := hV
    exact <a  sqsup  b, <wayBelowUp_inter a b, WayBelow.sup ha hb>>
  up_mem := by
    intro U V hU hV hUV
    obtain <a, <rfl, ha>> := hU
    obtain <b, rfl> := hV
    exact <b, <rfl, hUV ha>>

end ContinuousLatticeToNeighborhood

end DomainTheory.BridgeTheorems
```

mem_wayBelowUp_toFilter.lean

```
module

public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.toFilter
public import Mathlib.Order.Hom.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpenu2eisUpperSet
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_subset
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_Ici_subset
public import DomainTheory.ContinuousLattice.WayBelow.sInf_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.
    isContinuousLattice_iff_isLUB_sInf_nhds
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_23
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.wayBelowUp

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.ContinuousLattice
```

```
open DomainTheory.NeighbourhoodSystems

open scoped DomainTheory.ContinuousLattice

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

theorem mem_wayBelowUp_toFilter {x a : D} :
    (toFilter x).mem (wayBelowUp a)  <->  a  <<  x := by
  constructor
  * intro h
    obtain <b, <heq, hb>> := h
    have : x  in  wayBelowUp b := hb
    rwa [ <-  heq] at this
  * intro ha
    exact <a, <rfl, ha>>

end ContinuousLatticeToNeighborhood

end DomainTheory.BridgeTheorems
```

ofFilter_toFilter.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.toFilter
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.ofFilter
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    mem_wayBelowUp_toFilter
public import Mathlib.Order.Hom.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpenu2eisUpperSet
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_subset
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_Ici_subset
public import DomainTheory.ContinuousLattice.WayBelow.sInf_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.
    isContinuousLattice_iff_isLUB_sInf_nhds
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_23
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.codes

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.ContinuousLattice
```

```
open DomainTheory.NeighbourhoodSystems

open scoped DomainTheory.ContinuousLattice

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

section Continuous

variable (hD : ContinuousLattice D)

include hD

theorem ofFilter_toFilter (x : D) : ofFilter (toFilter x) = x := by
  have hset : codes (toFilter x) = {a : D | a  <<  x} := by
    ext a
    exact mem_wayBelowUp_toFilter
  change sSup (codes (toFilter x)) = x
  rw [hset, hD.sSup_wayBelow x]

end Continuous

end ContinuousLatticeToNeighborhood

end DomainTheory.BridgeTheorems
```

toFilter_isRound.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.toFilter
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.Round
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    mem_wayBelowUp_toFilter
public import Mathlib.Order.Hom.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpenu2eisUpperSet
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_subset
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_Ici_subset
public import DomainTheory.ContinuousLattice.WayBelow.sInf_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.
    isContinuousLattice_iff_isLUB_sInf_nhds
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_23

@[expose] public section
```

```
namespace DomainTheory.BridgeTheorems

open DomainTheory.ContinuousLattice

open DomainTheory.NeighbourhoodSystems

open scoped DomainTheory.ContinuousLattice

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

section Continuous

variable (hD : ContinuousLattice D)

include hD

theorem toFilter_isRound (x : D) : Round (toFilter x) := by
  constructor
  intro a ha
  have ha' : a  <<  x := mem_wayBelowUp_toFilter.mp ha
  obtain <b, hab, hbx> := WayBelow.interpolate hD ha'
  exact <b, hab, mem_wayBelowUp_toFilter.mpr hbx>

end Continuous

end ContinuousLatticeToNeighborhood

end DomainTheory.BridgeTheorems
```

toFilter_mono.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.toFilter
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_23
public import Mathlib.Order.Hom.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpenu2eisUpperSet
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_subset
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_Ici_subset
public import DomainTheory.ContinuousLattice.WayBelow.sInf_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.
    isContinuousLattice_iff_isLUB_sInf_nhds

@[expose] public section
```

```
namespace DomainTheory.BridgeTheorems

open DomainTheory.ContinuousLattice

open DomainTheory.NeighbourhoodSystems

open scoped DomainTheory.ContinuousLattice

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

theorem toFilter_mono {x y : D} (hxy : x  <=  y) : toFilter x  <=  toFilter y := by
  intro U hU
  obtain <a, <rfl, ha>> := hU
  exact <a, <rfl, ha.trans_le hxy>>

end ContinuousLatticeToNeighborhood

end DomainTheory.BridgeTheorems
```

domainEmbedding.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.toFilter
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    ofFilter_toFilter
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    mem_wayBelowUp_toFilter
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    toFilter_mono
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_23
public import Mathlib.Order.Hom.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpenu2eisUpperSet
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_subset
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_Ici_subset
public import DomainTheory.ContinuousLattice.WayBelow.sInf_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.
    isContinuousLattice_iff_isLUB_sInf_nhds
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.wayBelowUp
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.Filter
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.ofFilter

@[expose] public section
```

```
namespace DomainTheory.BridgeTheorems

open DomainTheory.ContinuousLattice

open DomainTheory.NeighbourhoodSystems

open scoped DomainTheory.ContinuousLattice

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

section Continuous

variable (hD : ContinuousLattice D)

include hD

/-- Order-embedding into all filters (forgets roundness). -/
noncomputable def domainEmbedding :
    D ?o Filter (D := D) where
  toFun := toFilter
  inj' := by
    intro x y h
    rw [ <-  ofFilter_toFilter hD x,  <-  ofFilter_toFilter hD y, h]
  map_rel_iff' := by
    intro x y
    constructor
    * intro h
      have : ofFilter (toFilter x)  <=  ofFilter (toFilter y) := by
        refine sSup_le fun a ha => ?_
        have ha' : a  <<  x := mem_wayBelowUp_toFilter.mp ha
        exact le_sSup (h (wayBelowUp a) <a, <rfl, ha'>>)
      simpa [ofFilter_toFilter hD] using this
    * exact fun hxy => toFilter_mono hxy

end Continuous

end ContinuousLatticeToNeighborhood

end DomainTheory.BridgeTheorems
```

toFilter_ofFilter.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.toFilter
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.Round
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.ofFilter
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    mem_wayBelowUp_ofFilter_of_round
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_23
```

```
public import Mathlib.Order.Hom.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpenu2eisUpperSet
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_subset
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_Ici_subset
public import DomainTheory.ContinuousLattice.WayBelow.sInf_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.
    isContinuousLattice_iff_isLUB_sInf_nhds
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    toNeighborhoodSystem
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.Filter

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.ContinuousLattice

open DomainTheory.NeighbourhoodSystems

open scoped DomainTheory.ContinuousLattice

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

theorem toFilter_ofFilter {f : Filter (D := D)} (hr : Round f) :
    toFilter (ofFilter f) = f := by
  refine NeighborhoodSystem.Element.ext (V := toNeighborhoodSystem) fun U => ?_
  constructor
  * intro hU
    obtain <a, <rfl, ha>> := hU
    exact (mem_wayBelowUp_ofFilter_of_round hr).2 ha
  * intro hU
    obtain <a, rfl> := f.sub hU
    refine <a, <rfl, ?_>>
    exact (mem_wayBelowUp_ofFilter_of_round hr).1 hU

end ContinuousLatticeToNeighborhood

end DomainTheory.BridgeTheorems
```

domainOrderIso.lean

```
module

public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    toFilter_ofFilter
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    toFilter_isRound
```

```
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    ofFilter_toFilter
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.RoundFilter
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    mem_wayBelowUp_toFilter
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    toFilter_mono
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_23
public import Mathlib.Order.Hom.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpenu2eisUpperSet
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_subset
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_Ici_subset
public import DomainTheory.ContinuousLattice.WayBelow.sInf_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.
    isContinuousLattice_iff_isLUB_sInf_nhds
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.wayBelowUp
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.toFilter
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.ofFilter

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.ContinuousLattice

open DomainTheory.NeighbourhoodSystems

open scoped DomainTheory.ContinuousLattice

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

section Continuous

variable (hD : ContinuousLattice D)

include hD

/-- Order-isomorphism: continuous lattice  <->  round ` Up `-filters. -/
noncomputable def domainOrderIso : D ?o RoundFilter (D := D) where
  toFun x := <toFilter x, toFilter_isRound hD x>
  invFun f := ofFilter (D := D) f.1
  left_inv x := ofFilter_toFilter hD x
  right_inv f := Subtype.ext (toFilter_ofFilter f.2)
  map_rel_iff' := by
    intro x y
```

```
    constructor
    * intro h
      have : ofFilter (toFilter x)  <=  ofFilter (toFilter y) := by
        refine sSup_le fun a ha => ?_
        have ha' : a  <<  x := mem_wayBelowUp_toFilter.mp ha
        exact le_sSup (h (wayBelowUp a) <a, <rfl, ha'>>)
      simpa [ofFilter_toFilter hD] using this
    * intro hxy
      exact toFilter_mono hxy

end Continuous

end ContinuousLatticeToNeighborhood

end DomainTheory.BridgeTheorems
```

Basic.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.ContinuousLattice
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    toNeighborhoodSystem
public import Mathlib.Order.Hom.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpenu2eisUpperSet
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_subset
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_Ici_subset
public import DomainTheory.ContinuousLattice.WayBelow.sInf_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.
    isContinuousLattice_iff_isLUB_sInf_nhds
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_23
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.wayBelowUp
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.toFilter
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.Round
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.codes
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.ofFilter
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    domainOrderIso
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    domainEmbedding
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    wayBelowUp_bot
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    wayBelowUp_inter
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    wayBelowUp_anti
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.Filter
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
```

```
    toFilter_mono
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    mem_wayBelowUp_toFilter
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    bot_mem_codes
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    codes_nonempty
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    codes_directed
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    mem_wayBelowUp_ofFilter_of_round
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    toFilter_ofFilter
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    toFilter_isRound
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    ofFilter_toFilter
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.RoundFilter

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.ContinuousLattice
open DomainTheory.NeighbourhoodSystems
open scoped DomainTheory.ContinuousLattice

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

end ContinuousLatticeToNeighborhood

/-- Blueprint name. -/
abbrev continuousLattice_to_neighborhoodSystem {D : Type*} [CompleteLattice D]
    (_hD : ContinuousLattice D) : NeighborhoodSystem D :=
  ContinuousLatticeToNeighborhood.toNeighborhoodSystem
end DomainTheory.BridgeTheorems
```

codes_lower.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    wayBelowUp_anti
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.codes
public import Mathlib.Order.Hom.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpenu2eisUpperSet
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
```

```
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_subset
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_Ici_subset
public import DomainTheory.ContinuousLattice.WayBelow.sInf_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.
    isContinuousLattice_iff_isLUB_sInf_nhds
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_23
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.Basic

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.ContinuousLattice
open DomainTheory.NeighbourhoodSystems
open scoped DomainTheory.ContinuousLattice

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

theorem codes_lower (f : Filter (D := D)) {a b : D} (hba : b  <=  a)
    (ha : a  in  codes f) : b  in  codes f :=
  f.up_mem ha <b, rfl> (wayBelowUp_anti hba)

end ContinuousLatticeToNeighborhood
end DomainTheory.BridgeTheorems
```

mem_codes_iff.lean

```
module

public import DomainTheory.ContinuousLattice.WayBelow.WayBelow
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.codes
public import Mathlib.Order.Hom.Basic
public import Mathlib.Order.CompleteLattice.Basic
public import Mathlib.Order.UpperLower.Basic
public import Mathlib.Order.Directed
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpen
public import DomainTheory.ContinuousLattice.WayBelow.ScottOpenu2eisUpperSet
public import DomainTheory.ContinuousLattice.WayBelow.scottOpen_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.directedOn_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_subset
public import DomainTheory.ContinuousLattice.WayBelow.exists_wayBelow_Ici_subset
public import DomainTheory.ContinuousLattice.WayBelow.sInf_wayBelow
public import DomainTheory.ContinuousLattice.WayBelow.
    isContinuousLattice_iff_isLUB_sInf_nhds
public import DomainTheory.NeighbourhoodSystems.DomainsGivenByNeighbourhoods.
    Exercise_1_23
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.Basic

@[expose] public section

namespace DomainTheory.BridgeTheorems
```

```
open DomainTheory.ContinuousLattice
open DomainTheory.NeighbourhoodSystems
open scoped DomainTheory.ContinuousLattice

namespace ContinuousLatticeToNeighborhood

variable {D : Type*} [CompleteLattice D]

theorem mem_codes_iff {f : Filter (D := D)} {a : D} :
    a  in  codes f  <->  f.mem (wayBelowUp a) :=
  Iff.rfl

end ContinuousLatticeToNeighborhood
end DomainTheory.BridgeTheorems
```

ContinuousLatticeToNeighborhood.lean

```
module

public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.Basic
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.
    mem_codes_iff
public import DomainTheory.BridgeTheorems.ContinuousLatticeToNeighborhood.codes_lower

@[expose] public section
```

assembleSum.lean

```
module
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Proposition64.
    inrMap
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Proposition64.
    inlMap
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid33.botElement
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable.toElement
public import Mathlib.Order.TypeTags
public import Mathlib.Order.Hom.Basic
public import Mathlib.Order.Sum.Order
public import Mathlib.Order.WithBot
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.Theorem72
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Proposition64

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

open DomainTheory.InformationSystems.InfoSys

open DomainTheory.InformationSystems.InfoSys.ApproximableMap
```

```
namespace InfoSysConstructions

variable {a b : Type*} [DecidableEq a] [DecidableEq b] (A : InfoSys a) (B : InfoSys b)

/-- Assemble a sum-domain element from a separated-sum code. -/
def assembleSum : WithBot (A.Element ? B.Element)  ->  (sumSystem A B).Element
  | False => (sumSystem A B).botElement
  | some (.inl x) => (inlMap A B).toElement x
  | some (.inr y) => (inrMap A B).toElement y

end InfoSysConstructions

end DomainTheory.BridgeTheorems
```

assembleSum_mono.lean

```
module

public import Mathlib.Order.Sum.Order
public import DomainTheory.BridgeTheorems.InfoSysConstructions.assembleSum
public import DomainTheory.InformationSystems.ElementsOfASystem.Factoid33.botElement
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable.toElement_mono
public import DomainTheory.InformationSystems.InformationSystems.InfoSys.Basic
public import Mathlib.Order.Hom.Basic
public import Mathlib.Order.WithBot
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.Theorem72
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Proposition64

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

open DomainTheory.InformationSystems.InfoSys

open DomainTheory.InformationSystems.InfoSys.ApproximableMap

namespace InfoSysConstructions

variable {a b : Type*} [DecidableEq a] [DecidableEq b] (A : InfoSys a) (B : InfoSys b)

theorem assembleSum_mono {w_1 w_2 : WithBot (A.Element ? B.Element)} (h : w_1  <=  w_2
    ) :
    assembleSum A B w_1  <=  assembleSum A B w_2 := by
  cases w_1 with
  | bot => exact botElement.le _ _
  | coe s_1 =>
    cases w_2 with
    | bot => exact (WithBot.not_coe_le_bot _ h).elim
    | coe s_2 =>
      have hs : s_1  <=  s_2 := WithBot.coe_le_coe.1 h
```

```
      cases s_1 with
      | inl x =>
        cases s_2 with
        | inl x' => exact (inlMap A B).toElement_mono (Sum.inl_le_inl_iff.1 hs)
        | inr y' => exact (Sum.not_inl_le_inr hs).elim
      | inr y =>
        cases s_2 with
        | inl x' => exact (Sum.not_inr_le_inl hs).elim
        | inr y' => exact (inrMap A B).toElement_mono (Sum.inr_le_inr_iff.1 hs)

end InfoSysConstructions

end DomainTheory.BridgeTheorems
```

instPartialOrderApproximableMap.lean

```
module
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Proposition53.Le
public import DomainTheory.InformationSystems.ApproximableMappingsBetweenDomains.
    Approximable.ext
public import Mathlib.Order.Hom.Basic
public import Mathlib.Order.Sum.Order
public import Mathlib.Order.WithBot
public import DomainTheory.InformationSystems.FunctionSpaceAsADomain.Theorem72
public import DomainTheory.InformationSystems.ProductsAndSumsOfDomains.Proposition64

@[expose] public section

namespace DomainTheory.BridgeTheorems

open DomainTheory.InformationSystems

open DomainTheory.InformationSystems.Constructive

open DomainTheory.InformationSystems.InfoSys

open DomainTheory.InformationSystems.InfoSys.ApproximableMap

namespace InfoSysConstructions

variable {a b : Type*} [DecidableEq a] [DecidableEq b] (A : InfoSys a) (B : InfoSys b)

/-- Pointwise relation order on approximable maps (Prop 5.3 `Le`). -/
instance instPartialOrderApproximableMap : PartialOrder (ApproximableMap A B) where
  le := @Le _ _ _ _ A B
  le_refl _ _ _ h := h
  le_trans _ _ _ hfg hgh _ _ hf := hgh (hfg hf)
  le_antisymm _ _ hfg hgf := ApproximableMap.ext fun _ _ => <fun h => hfg h, fun h =>
    hgf h>

end InfoSysConstructions

end DomainTheory.BridgeTheorems
```

dcondSet.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.Comb
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.4).** Doubly strict conditional
`z ? x, y = z ? (z ? x,  top ), (z ?  top , y)`. -/
def dcondSet (z x y : Pomega) : Pomega :=
  condSet z (condSet z x topElem) (condSet z topElem y)

end DomainTheory.DataTypesAsLattices
```

0

dcondSet.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.Comb
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.4).** Doubly strict conditional
`z ? x, y = z ? (z ? x,  top ), (z ?  top , y)`. -/
def dcondSet (z x y : Pomega) : Pomega :=
  condSet z (condSet z x topElem) (condSet z topElem y)

end DomainTheory.DataTypesAsLattices
```

1

dcondSet.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.Comb
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.4).** Doubly strict conditional
`z ? x, y = z ? (z ? x,  top ), (z ?  top , y)`. -/
def dcondSet (z x y : Pomega) : Pomega :=
  condSet z (condSet z x topElem) (condSet z topElem y)

end DomainTheory.DataTypesAsLattices
```

2

dcondSet.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.Comb
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.4).** Doubly strict conditional
`z ? x, y = z ? (z ? x,  top ), (z ?  top , y)`. -/
def dcondSet (z x y : Pomega) : Pomega :=
  condSet z (condSet z x topElem) (condSet z topElem y)

end DomainTheory.DataTypesAsLattices
```

3

dcondSet.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.Comb
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.4).** Doubly strict conditional
`z ? x, y = z ? (z ? x,  top ), (z ?  top , y)`. -/
def dcondSet (z x y : Pomega) : Pomega :=
  condSet z (condSet z x topElem) (condSet z topElem y)

end DomainTheory.DataTypesAsLattices
```

4

dcondSet.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.Comb
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.4).** Doubly strict conditional
`z ? x, y = z ? (z ? x,  top ), (z ?  top , y)`. -/
def dcondSet (z x y : Pomega) : Pomega :=
  condSet z (condSet z x topElem) (condSet z topElem y)

end DomainTheory.DataTypesAsLattices
```

5

dcondSet.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.Comb
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.4).** Doubly strict conditional
`z ? x, y = z ? (z ? x,  top ), (z ?  top , y)`. -/
def dcondSet (z x y : Pomega) : Pomega :=
  condSet z (condSet z x topElem) (condSet z topElem y)

end DomainTheory.DataTypesAsLattices
```

6

dcondSet.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.Comb
```

```
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.4).** Doubly strict conditional
`z ? x, y = z ? (z ? x,  top ), (z ?  top , y)`. -/
def dcondSet (z x y : Pomega) : Pomega :=
  condSet z (condSet z x topElem) (condSet z topElem y)

end DomainTheory.DataTypesAsLattices
```

7

dcondSet.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.Comb
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.4).** Doubly strict conditional
`z ? x, y = z ? (z ? x,  top ), (z ?  top , y)`. -/
def dcondSet (z x y : Pomega) : Pomega :=
  condSet z (condSet z x topElem) (condSet z topElem y)

end DomainTheory.DataTypesAsLattices
```

8

dcondSet.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.Term
public import DomainTheory.DataTypesAsLattices.Lambda.IsCombinatory
public import DomainTheory.DataTypesAsLattices.Lambda.Comb
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (4.4).** Doubly strict conditional
`z ? x, y = z ? (z ? x,  top ), (z ?  top , y)`. -/
def dcondSet (z x y : Pomega) : Pomega :=
  condSet z (condSet z x topElem) (condSet z topElem y)
```

```
end DomainTheory.DataTypesAsLattices
```

9

e_one.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem e_one : e 1 = ({0} : Pomega) := by
  simpa using e_pow2 0

end DomainTheory.DataTypesAsLattices
```

0

e_one.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem e_one : e 1 = ({0} : Pomega) := by
  simpa using e_pow2 0

end DomainTheory.DataTypesAsLattices
```

1

e_one.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem e_one : e 1 = ({0} : Pomega) := by
  simpa using e_pow2 0

end DomainTheory.DataTypesAsLattices
```

2

e_one.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem e_one : e 1 = ({0} : Pomega) := by
  simpa using e_pow2 0

end DomainTheory.DataTypesAsLattices
```

3

e_one.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem e_one : e 1 = ({0} : Pomega) := by
  simpa using e_pow2 0

end DomainTheory.DataTypesAsLattices
```

4

e_one.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem e_one : e 1 = ({0} : Pomega) := by
  simpa using e_pow2 0

end DomainTheory.DataTypesAsLattices
```

5

e_one.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem e_one : e 1 = ({0} : Pomega) := by
  simpa using e_pow2 0

end DomainTheory.DataTypesAsLattices
```

6

e_one.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem e_one : e 1 = ({0} : Pomega) := by
  simpa using e_pow2 0

end DomainTheory.DataTypesAsLattices
```

7

e_one.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem e_one : e 1 = ({0} : Pomega) := by
  simpa using e_pow2 0

end DomainTheory.DataTypesAsLattices
```

8

e_one.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
```

```
theorem e_one : e 1 = ({0} : Pomega) := by
  simpa using e_pow2 0

end DomainTheory.DataTypesAsLattices
```

9

envSet.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
def envSet (? : Nat  ->  Pomega) (i : Nat) (v : Pomega) : Nat  ->  Pomega :=
  Function.update ? i v

namespace envSet

theorem comm {? : Nat  ->  Pomega} {i j : Nat} (hij : i  !=  j) (v w : Pomega) :
    envSet (envSet ? i v) j w = envSet (envSet ? j w) i v := by
  funext k
  unfold envSet
  by_cases hj : k = j
  * subst hj
    simp [Function.update, hij.symm]
  * by_cases hi : k = i
    * subst hi
      simp [Function.update, hij]
    * simp [Function.update, hi, hj]

end envSet

end DomainTheory.DataTypesAsLattices
```

0

envSet.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
def envSet (? : Nat  ->  Pomega) (i : Nat) (v : Pomega) : Nat  ->  Pomega :=
  Function.update ? i v

namespace envSet

theorem comm {? : Nat  ->  Pomega} {i j : Nat} (hij : i  !=  j) (v w : Pomega) :
    envSet (envSet ? i v) j w = envSet (envSet ? j w) i v := by
  funext k
```

```
  unfold envSet
  by_cases hj : k = j
  * subst hj
    simp [Function.update, hij.symm]
  * by_cases hi : k = i
    * subst hi
      simp [Function.update, hij]
    * simp [Function.update, hi, hj]

end envSet

end DomainTheory.DataTypesAsLattices
```

1

envSet.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
def envSet (? : Nat  ->  Pomega) (i : Nat) (v : Pomega) : Nat  ->  Pomega :=
  Function.update ? i v

namespace envSet

theorem comm {? : Nat  ->  Pomega} {i j : Nat} (hij : i  !=  j) (v w : Pomega) :
    envSet (envSet ? i v) j w = envSet (envSet ? j w) i v := by
  funext k
  unfold envSet
  by_cases hj : k = j
  * subst hj
    simp [Function.update, hij.symm]
  * by_cases hi : k = i
    * subst hi
      simp [Function.update, hij]
    * simp [Function.update, hi, hj]

end envSet

end DomainTheory.DataTypesAsLattices
```

2

envSet.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
```

```
def envSet (? : Nat  ->  Pomega) (i : Nat) (v : Pomega) : Nat  ->  Pomega :=
  Function.update ? i v

namespace envSet

theorem comm {? : Nat  ->  Pomega} {i j : Nat} (hij : i  !=  j) (v w : Pomega) :
    envSet (envSet ? i v) j w = envSet (envSet ? j w) i v := by
  funext k
  unfold envSet
  by_cases hj : k = j
  * subst hj
    simp [Function.update, hij.symm]
  * by_cases hi : k = i
    * subst hi
      simp [Function.update, hij]
    * simp [Function.update, hi, hj]

end envSet

end DomainTheory.DataTypesAsLattices
```

3

envSet.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
def envSet (? : Nat  ->  Pomega) (i : Nat) (v : Pomega) : Nat  ->  Pomega :=
  Function.update ? i v

namespace envSet

theorem comm {? : Nat  ->  Pomega} {i j : Nat} (hij : i  !=  j) (v w : Pomega) :
    envSet (envSet ? i v) j w = envSet (envSet ? j w) i v := by
  funext k
  unfold envSet
  by_cases hj : k = j
  * subst hj
    simp [Function.update, hij.symm]
  * by_cases hi : k = i
    * subst hi
      simp [Function.update, hij]
    * simp [Function.update, hi, hj]

end envSet

end DomainTheory.DataTypesAsLattices
```

4

envSet.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
def envSet (? : Nat  ->  Pomega) (i : Nat) (v : Pomega) : Nat  ->  Pomega :=
  Function.update ? i v

namespace envSet

theorem comm {? : Nat  ->  Pomega} {i j : Nat} (hij : i  !=  j) (v w : Pomega) :
    envSet (envSet ? i v) j w = envSet (envSet ? j w) i v := by
  funext k
  unfold envSet
  by_cases hj : k = j
  * subst hj
    simp [Function.update, hij.symm]
  * by_cases hi : k = i
    * subst hi
      simp [Function.update, hij]
    * simp [Function.update, hi, hj]

end envSet

end DomainTheory.DataTypesAsLattices
```

5

envSet.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
def envSet (? : Nat  ->  Pomega) (i : Nat) (v : Pomega) : Nat  ->  Pomega :=
  Function.update ? i v

namespace envSet

theorem comm {? : Nat  ->  Pomega} {i j : Nat} (hij : i  !=  j) (v w : Pomega) :
    envSet (envSet ? i v) j w = envSet (envSet ? j w) i v := by
  funext k
  unfold envSet
  by_cases hj : k = j
  * subst hj
    simp [Function.update, hij.symm]
  * by_cases hi : k = i
    * subst hi
      simp [Function.update, hij]
    * simp [Function.update, hi, hj]
```

```
end envSet

end DomainTheory.DataTypesAsLattices
```

6

envSet.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
def envSet (? : Nat  ->  Pomega) (i : Nat) (v : Pomega) : Nat  ->  Pomega :=
  Function.update ? i v

namespace envSet

theorem comm {? : Nat  ->  Pomega} {i j : Nat} (hij : i  !=  j) (v w : Pomega) :
    envSet (envSet ? i v) j w = envSet (envSet ? j w) i v := by
  funext k
  unfold envSet
  by_cases hj : k = j
  * subst hj
    simp [Function.update, hij.symm]
  * by_cases hi : k = i
    * subst hi
      simp [Function.update, hij]
    * simp [Function.update, hi, hj]

end envSet

end DomainTheory.DataTypesAsLattices
```

7

envSet.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
def envSet (? : Nat  ->  Pomega) (i : Nat) (v : Pomega) : Nat  ->  Pomega :=
  Function.update ? i v

namespace envSet

theorem comm {? : Nat  ->  Pomega} {i j : Nat} (hij : i  !=  j) (v w : Pomega) :
    envSet (envSet ? i v) j w = envSet (envSet ? j w) i v := by
  funext k
  unfold envSet
```

```
  by_cases hj : k = j
  * subst hj
    simp [Function.update, hij.symm]
  * by_cases hi : k = i
    * subst hi
      simp [Function.update, hij]
    * simp [Function.update, hi, hj]

end envSet

end DomainTheory.DataTypesAsLattices
```

8

envSet.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
def envSet (? : Nat  ->  Pomega) (i : Nat) (v : Pomega) : Nat  ->  Pomega :=
  Function.update ? i v

namespace envSet

theorem comm {? : Nat  ->  Pomega} {i j : Nat} (hij : i  !=  j) (v w : Pomega) :
    envSet (envSet ? i v) j w = envSet (envSet ? j w) i v := by
  funext k
  unfold envSet
  by_cases hj : k = j
  * subst hj
    simp [Function.update, hij.symm]
  * by_cases hi : k = i
    * subst hi
      simp [Function.update, hij]
    * simp [Function.update, hi, hj]

end envSet

end DomainTheory.DataTypesAsLattices
```

9

eq_2_1_else.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.maxExtend
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section
```

```
namespace DomainTheory.DataTypesAsLattices

theorem eq_2_1_else (p : Nat  ->  Pomega) {x : Pomega}
    (hne : x  !=  botElem) (hsing : forall  n, x  !=  ofNat n) :
    maxExtend p x = topElem := by
  ext k
  constructor
  * intro; trivial
  * intro
    exact Or.inr (Or.inr <hne, hsing>)

end DomainTheory.DataTypesAsLattices
```

0

eq_2_1_else.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.maxExtend
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_1_else (p : Nat  ->  Pomega) {x : Pomega}
    (hne : x  !=  botElem) (hsing : forall  n, x  !=  ofNat n) :
    maxExtend p x = topElem := by
  ext k
  constructor
  * intro; trivial
  * intro
    exact Or.inr (Or.inr <hne, hsing>)

end DomainTheory.DataTypesAsLattices
```

1

eq_2_1_else.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.maxExtend
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_1_else (p : Nat  ->  Pomega) {x : Pomega}
```

```
    (hne : x  !=  botElem) (hsing : forall  n, x  !=  ofNat n) :
    maxExtend p x = topElem := by
  ext k
  constructor
  * intro; trivial
  * intro
    exact Or.inr (Or.inr <hne, hsing>)

end DomainTheory.DataTypesAsLattices
```

2

eq_2_1_else.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.maxExtend
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_1_else (p : Nat  ->  Pomega) {x : Pomega}
    (hne : x  !=  botElem) (hsing : forall  n, x  !=  ofNat n) :
    maxExtend p x = topElem := by
  ext k
  constructor
  * intro; trivial
  * intro
    exact Or.inr (Or.inr <hne, hsing>)

end DomainTheory.DataTypesAsLattices
```

3

eq_2_1_else.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.maxExtend
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_1_else (p : Nat  ->  Pomega) {x : Pomega}
    (hne : x  !=  botElem) (hsing : forall  n, x  !=  ofNat n) :
    maxExtend p x = topElem := by
  ext k
  constructor
```

```
  * intro; trivial
  * intro
    exact Or.inr (Or.inr <hne, hsing>)

end DomainTheory.DataTypesAsLattices
```

4

eq_2_1_else.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.maxExtend
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_1_else (p : Nat  ->  Pomega) {x : Pomega}
    (hne : x  !=  botElem) (hsing : forall  n, x  !=  ofNat n) :
    maxExtend p x = topElem := by
  ext k
  constructor
  * intro; trivial
  * intro
    exact Or.inr (Or.inr <hne, hsing>)

end DomainTheory.DataTypesAsLattices
```

5

eq_2_1_else.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.maxExtend
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_1_else (p : Nat  ->  Pomega) {x : Pomega}
    (hne : x  !=  botElem) (hsing : forall  n, x  !=  ofNat n) :
    maxExtend p x = topElem := by
  ext k
  constructor
  * intro; trivial
  * intro
    exact Or.inr (Or.inr <hne, hsing>)
```

```
end DomainTheory.DataTypesAsLattices
```

6

eq_2_1_else.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.maxExtend
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_1_else (p : Nat  ->  Pomega) {x : Pomega}
    (hne : x  !=  botElem) (hsing : forall  n, x  !=  ofNat n) :
    maxExtend p x = topElem := by
  ext k
  constructor
  * intro; trivial
  * intro
    exact Or.inr (Or.inr <hne, hsing>)

end DomainTheory.DataTypesAsLattices
```

7

eq_2_1_else.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.maxExtend
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_1_else (p : Nat  ->  Pomega) {x : Pomega}
    (hne : x  !=  botElem) (hsing : forall  n, x  !=  ofNat n) :
    maxExtend p x = topElem := by
  ext k
  constructor
  * intro; trivial
  * intro
    exact Or.inr (Or.inr <hne, hsing>)

end DomainTheory.DataTypesAsLattices
```

8

eq_2_1_else.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.maxExtend
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_1_else (p : Nat  ->  Pomega) {x : Pomega}
    (hne : x  !=  botElem) (hsing : forall  n, x  !=  ofNat n) :
    maxExtend p x = topElem := by
  ext k
  constructor
  * intro; trivial
  * intro
    exact Or.inr (Or.inr <hne, hsing>)

end DomainTheory.DataTypesAsLattices
```

9

eq_2_1_singleton.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.maxExtend
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_1_singleton (p : Nat  ->  Pomega) (n : Nat) :
    maxExtend p (ofNat n) = p n := by
  ext k
  constructor
  * intro hk
    rcases hk with <he, _> | <m, hm, hkm> | <_, hall>
    * have hne : ofNat n  !=  {} := Set.Nonempty.ne_empty <n, (rfl : n  in  ofNat n)>
      exact (hne he).elim
    * have : n = m := by
        have hn : n  in  ofNat m := hm  |>  (rfl : n  in  ofNat n)
        simpa [ofNat] using hn
      subst this; exact hkm
    * exact (hall n rfl).elim
  * intro hk
    exact Or.inr (Or.inl <n, rfl, hk>)

end DomainTheory.DataTypesAsLattices
```

0

eq_2_1_singleton.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.maxExtend
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_1_singleton (p : Nat  ->  Pomega) (n : Nat) :
    maxExtend p (ofNat n) = p n := by
  ext k
  constructor
  * intro hk
    rcases hk with <he, _> | <m, hm, hkm> | <_, hall>
    * have hne : ofNat n  !=  {} := Set.Nonempty.ne_empty <n, (rfl : n  in  ofNat n)>
      exact (hne he).elim
    * have : n = m := by
        have hn : n  in  ofNat m := hm  |>  (rfl : n  in  ofNat n)
        simpa [ofNat] using hn
      subst this; exact hkm
    * exact (hall n rfl).elim
  * intro hk
    exact Or.inr (Or.inl <n, rfl, hk>)

end DomainTheory.DataTypesAsLattices
```

1

eq_2_1_singleton.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.maxExtend
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_1_singleton (p : Nat  ->  Pomega) (n : Nat) :
    maxExtend p (ofNat n) = p n := by
  ext k
  constructor
  * intro hk
    rcases hk with <he, _> | <m, hm, hkm> | <_, hall>
    * have hne : ofNat n  !=  {} := Set.Nonempty.ne_empty <n, (rfl : n  in  ofNat n)>
      exact (hne he).elim
```

```
    * have : n = m := by
        have hn : n  in  ofNat m := hm  |>  (rfl : n  in  ofNat n)
        simpa [ofNat] using hn
      subst this; exact hkm
    * exact (hall n rfl).elim
  * intro hk
    exact Or.inr (Or.inl <n, rfl, hk>)

end DomainTheory.DataTypesAsLattices
```

2

eq_2_1_singleton.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.maxExtend
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_1_singleton (p : Nat  ->  Pomega) (n : Nat) :
    maxExtend p (ofNat n) = p n := by
  ext k
  constructor
  * intro hk
    rcases hk with <he, _> | <m, hm, hkm> | <_, hall>
    * have hne : ofNat n  !=  {} := Set.Nonempty.ne_empty <n, (rfl : n  in  ofNat n)>
      exact (hne he).elim
    * have : n = m := by
        have hn : n  in  ofNat m := hm  |>  (rfl : n  in  ofNat n)
        simpa [ofNat] using hn
      subst this; exact hkm
    * exact (hall n rfl).elim
  * intro hk
    exact Or.inr (Or.inl <n, rfl, hk>)

end DomainTheory.DataTypesAsLattices
```

3

eq_2_1_singleton.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.maxExtend
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section
```

```
namespace DomainTheory.DataTypesAsLattices

theorem eq_2_1_singleton (p : Nat  ->  Pomega) (n : Nat) :
    maxExtend p (ofNat n) = p n := by
  ext k
  constructor
  * intro hk
    rcases hk with <he, _> | <m, hm, hkm> | <_, hall>
    * have hne : ofNat n  !=  {} := Set.Nonempty.ne_empty <n, (rfl : n  in  ofNat n)>
      exact (hne he).elim
    * have : n = m := by
        have hn : n  in  ofNat m := hm  |>  (rfl : n  in  ofNat n)
        simpa [ofNat] using hn
      subst this; exact hkm
    * exact (hall n rfl).elim
  * intro hk
    exact Or.inr (Or.inl <n, rfl, hk>)

end DomainTheory.DataTypesAsLattices
```

4

eq_2_1_singleton.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.maxExtend
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_1_singleton (p : Nat  ->  Pomega) (n : Nat) :
    maxExtend p (ofNat n) = p n := by
  ext k
  constructor
  * intro hk
    rcases hk with <he, _> | <m, hm, hkm> | <_, hall>
    * have hne : ofNat n  !=  {} := Set.Nonempty.ne_empty <n, (rfl : n  in  ofNat n)>
      exact (hne he).elim
    * have : n = m := by
        have hn : n  in  ofNat m := hm  |>  (rfl : n  in  ofNat n)
        simpa [ofNat] using hn
      subst this; exact hkm
    * exact (hall n rfl).elim
  * intro hk
    exact Or.inr (Or.inl <n, rfl, hk>)

end DomainTheory.DataTypesAsLattices
```

5

eq_2_1_singleton.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.maxExtend
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_1_singleton (p : Nat  ->  Pomega) (n : Nat) :
    maxExtend p (ofNat n) = p n := by
  ext k
  constructor
  * intro hk
    rcases hk with <he, _> | <m, hm, hkm> | <_, hall>
    * have hne : ofNat n  !=  {} := Set.Nonempty.ne_empty <n, (rfl : n  in  ofNat n)>
      exact (hne he).elim
    * have : n = m := by
        have hn : n  in  ofNat m := hm  |>  (rfl : n  in  ofNat n)
        simpa [ofNat] using hn
      subst this; exact hkm
    * exact (hall n rfl).elim
  * intro hk
    exact Or.inr (Or.inl <n, rfl, hk>)

end DomainTheory.DataTypesAsLattices
```

6

eq_2_1_singleton.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.maxExtend
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_1_singleton (p : Nat  ->  Pomega) (n : Nat) :
    maxExtend p (ofNat n) = p n := by
  ext k
  constructor
  * intro hk
    rcases hk with <he, _> | <m, hm, hkm> | <_, hall>
    * have hne : ofNat n  !=  {} := Set.Nonempty.ne_empty <n, (rfl : n  in  ofNat n)>
      exact (hne he).elim
    * have : n = m := by
        have hn : n  in  ofNat m := hm  |>  (rfl : n  in  ofNat n)
        simpa [ofNat] using hn
      subst this; exact hkm
```

```
    * exact (hall n rfl).elim
  * intro hk
    exact Or.inr (Or.inl <n, rfl, hk>)

end DomainTheory.DataTypesAsLattices
```

7

eq_2_1_singleton.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.maxExtend
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_1_singleton (p : Nat  ->  Pomega) (n : Nat) :
    maxExtend p (ofNat n) = p n := by
  ext k
  constructor
  * intro hk
    rcases hk with <he, _> | <m, hm, hkm> | <_, hall>
    * have hne : ofNat n  !=  {} := Set.Nonempty.ne_empty <n, (rfl : n  in  ofNat n)>
      exact (hne he).elim
    * have : n = m := by
        have hn : n  in  ofNat m := hm  |>  (rfl : n  in  ofNat n)
        simpa [ofNat] using hn
      subst this; exact hkm
    * exact (hall n rfl).elim
  * intro hk
    exact Or.inr (Or.inl <n, rfl, hk>)

end DomainTheory.DataTypesAsLattices
```

8

eq_2_1_singleton.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.maxExtend
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_1_singleton (p : Nat  ->  Pomega) (n : Nat) :
    maxExtend p (ofNat n) = p n := by
```

```
  ext k
  constructor
  * intro hk
    rcases hk with <he, _> | <m, hm, hkm> | <_, hall>
    * have hne : ofNat n  !=  {} := Set.Nonempty.ne_empty <n, (rfl : n  in  ofNat n)>
      exact (hne he).elim
    * have : n = m := by
        have hn : n  in  ofNat m := hm  |>  (rfl : n  in  ofNat n)
        simpa [ofNat] using hn
      subst this; exact hkm
    * exact (hall n rfl).elim
  * intro hk
    exact Or.inr (Or.inl <n, rfl, hk>)

end DomainTheory.DataTypesAsLattices
```

9

eq_2_5.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.succSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.5).** `(6  U  10) + 1 = 7  U  11`. -/
theorem eq_2_5 : succSet (ofNat 6  U  ofNat 10) = ofNat 7  U  ofNat 11 := by
  ext k
  constructor
  * intro <n, hn, hk>
    rcases hn with h6 | h10
    * simp [ofNat] at h6; subst h6; simp [ofNat, hk]
    * simp [ofNat] at h10; subst h10; simp [ofNat, hk]
  * intro hk
    rcases hk with h7 | h11
    * simp [ofNat] at h7; exact <6, Or.inl (by simp [ofNat]), by simp [h7]>
    * simp [ofNat] at h11; exact <10, Or.inr (by simp [ofNat]), by simp [h11]>

end DomainTheory.DataTypesAsLattices
```

0

eq_2_5.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.succSet

@[expose] public section
```

```
namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.5).** `(6 ∪ 10) + 1 = 7 ∪ 11`. -/
theorem eq_2_5 : succSet (ofNat 6 ∪ ofNat 10) = ofNat 7 ∪ ofNat 11 := by
  ext k
  constructor
  · intro ⟨n, hn, hk⟩
    rcases hn with h6 | h10
    · simp [ofNat] at h6; subst h6; simp [ofNat, hk]
    · simp [ofNat] at h10; subst h10; simp [ofNat, hk]
  · intro hk
    rcases hk with h7 | h11
    · simp [ofNat] at h7; exact ⟨6, Or.inl (by simp [ofNat]), by simp [h7]⟩
    · simp [ofNat] at h11; exact ⟨10, Or.inr (by simp [ofNat]), by simp [h11]⟩

end DomainTheory.DataTypesAsLattices
```

1

eq_2_5.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.succSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.5).** `(6 ∪ 10) + 1 = 7 ∪ 11`. -/
theorem eq_2_5 : succSet (ofNat 6 ∪ ofNat 10) = ofNat 7 ∪ ofNat 11 := by
  ext k
  constructor
  · intro ⟨n, hn, hk⟩
    rcases hn with h6 | h10
    · simp [ofNat] at h6; subst h6; simp [ofNat, hk]
    · simp [ofNat] at h10; subst h10; simp [ofNat, hk]
  · intro hk
    rcases hk with h7 | h11
    · simp [ofNat] at h7; exact ⟨6, Or.inl (by simp [ofNat]), by simp [h7]⟩
    · simp [ofNat] at h11; exact ⟨10, Or.inr (by simp [ofNat]), by simp [h11]⟩

end DomainTheory.DataTypesAsLattices
```

2

eq_2_5.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.succSet
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.5).** `(6  U  10) + 1 = 7  U  11`. -/
theorem eq_2_5 : succSet (ofNat 6  U  ofNat 10) = ofNat 7  U  ofNat 11 := by
  ext k
  constructor
  * intro <n, hn, hk>
    rcases hn with h6 | h10
    * simp [ofNat] at h6; subst h6; simp [ofNat, hk]
    * simp [ofNat] at h10; subst h10; simp [ofNat, hk]
  * intro hk
    rcases hk with h7 | h11
    * simp [ofNat] at h7; exact <6, Or.inl (by simp [ofNat]), by simp [h7]>
    * simp [ofNat] at h11; exact <10, Or.inr (by simp [ofNat]), by simp [h11]>

end DomainTheory.DataTypesAsLattices
```

3

eq_2_5.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.succSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.5).** `(6  U  10) + 1 = 7  U  11`. -/
theorem eq_2_5 : succSet (ofNat 6  U  ofNat 10) = ofNat 7  U  ofNat 11 := by
  ext k
  constructor
  * intro <n, hn, hk>
    rcases hn with h6 | h10
    * simp [ofNat] at h6; subst h6; simp [ofNat, hk]
    * simp [ofNat] at h10; subst h10; simp [ofNat, hk]
  * intro hk
    rcases hk with h7 | h11
    * simp [ofNat] at h7; exact <6, Or.inl (by simp [ofNat]), by simp [h7]>
    * simp [ofNat] at h11; exact <10, Or.inr (by simp [ofNat]), by simp [h11]>

end DomainTheory.DataTypesAsLattices
```

4

eq_2_5.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
```

```
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.succSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.5).** `(6  U  10) + 1 = 7  U  11`. -/
theorem eq_2_5 : succSet (ofNat 6  U  ofNat 10) = ofNat 7  U  ofNat 11 := by
  ext k
  constructor
  * intro <n, hn, hk>
    rcases hn with h6 | h10
    * simp [ofNat] at h6; subst h6; simp [ofNat, hk]
    * simp [ofNat] at h10; subst h10; simp [ofNat, hk]
  * intro hk
    rcases hk with h7 | h11
    * simp [ofNat] at h7; exact <6, Or.inl (by simp [ofNat]), by simp [h7]>
    * simp [ofNat] at h11; exact <10, Or.inr (by simp [ofNat]), by simp [h11]>

end DomainTheory.DataTypesAsLattices
```

5

eq_2_5.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.succSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.5).** `(6  U  10) + 1 = 7  U  11`. -/
theorem eq_2_5 : succSet (ofNat 6  U  ofNat 10) = ofNat 7  U  ofNat 11 := by
  ext k
  constructor
  * intro <n, hn, hk>
    rcases hn with h6 | h10
    * simp [ofNat] at h6; subst h6; simp [ofNat, hk]
    * simp [ofNat] at h10; subst h10; simp [ofNat, hk]
  * intro hk
    rcases hk with h7 | h11
    * simp [ofNat] at h7; exact <6, Or.inl (by simp [ofNat]), by simp [h7]>
    * simp [ofNat] at h11; exact <10, Or.inr (by simp [ofNat]), by simp [h11]>

end DomainTheory.DataTypesAsLattices
```

6

eq_2_5.lean

```
module
```

```
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.succSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.5).** `(6  U  10) + 1 = 7  U  11`. -/
theorem eq_2_5 : succSet (ofNat 6  U  ofNat 10) = ofNat 7  U  ofNat 11 := by
  ext k
  constructor
  * intro <n, hn, hk>
    rcases hn with h6 | h10
    * simp [ofNat] at h6; subst h6; simp [ofNat, hk]
    * simp [ofNat] at h10; subst h10; simp [ofNat, hk]
  * intro hk
    rcases hk with h7 | h11
    * simp [ofNat] at h7; exact <6, Or.inl (by simp [ofNat]), by simp [h7]>
    * simp [ofNat] at h11; exact <10, Or.inr (by simp [ofNat]), by simp [h11]>

end DomainTheory.DataTypesAsLattices
```

7

eq_2_5.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.succSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.5).** `(6  U  10) + 1 = 7  U  11`. -/
theorem eq_2_5 : succSet (ofNat 6  U  ofNat 10) = ofNat 7  U  ofNat 11 := by
  ext k
  constructor
  * intro <n, hn, hk>
    rcases hn with h6 | h10
    * simp [ofNat] at h6; subst h6; simp [ofNat, hk]
    * simp [ofNat] at h10; subst h10; simp [ofNat, hk]
  * intro hk
    rcases hk with h7 | h11
    * simp [ofNat] at h7; exact <6, Or.inl (by simp [ofNat]), by simp [h7]>
    * simp [ofNat] at h11; exact <10, Or.inr (by simp [ofNat]), by simp [h11]>

end DomainTheory.DataTypesAsLattices
```

8

eq_2_5.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.succSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.5).** `(6  U  10) + 1 = 7  U  11`. -/
theorem eq_2_5 : succSet (ofNat 6  U  ofNat 10) = ofNat 7  U  ofNat 11 := by
  ext k
  constructor
  * intro <n, hn, hk>
    rcases hn with h6 | h10
    * simp [ofNat] at h6; subst h6; simp [ofNat, hk]
    * simp [ofNat] at h10; subst h10; simp [ofNat, hk]
  * intro hk
    rcases hk with h7 | h11
    * simp [ofNat] at h7; exact <6, Or.inl (by simp [ofNat]), by simp [h7]>
    * simp [ofNat] at h11; exact <10, Or.inr (by simp [ofNat]), by simp [h11]>

end DomainTheory.DataTypesAsLattices
```

9

eq_2_6.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.predSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.6).** `0 ? 1 = False`. -/
theorem eq_2_6 : predSet (ofNat 0) = botElem := by
  ext k
  simp [predSet, ofNat, botElem]

end DomainTheory.DataTypesAsLattices
```

0

eq_2_6.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
```

```
public import DomainTheory.DataTypesAsLattices.Lambda.predSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.6).** `0 ? 1 = False`. -/
theorem eq_2_6 : predSet (ofNat 0) = botElem := by
  ext k
  simp [predSet, ofNat, botElem]

end DomainTheory.DataTypesAsLattices
```

1

eq_2_6.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.predSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.6).** `0 ? 1 = False`. -/
theorem eq_2_6 : predSet (ofNat 0) = botElem := by
  ext k
  simp [predSet, ofNat, botElem]

end DomainTheory.DataTypesAsLattices
```

2

eq_2_6.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.predSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.6).** `0 ? 1 = False`. -/
theorem eq_2_6 : predSet (ofNat 0) = botElem := by
  ext k
  simp [predSet, ofNat, botElem]

end DomainTheory.DataTypesAsLattices
```

3

eq_2_6.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.predSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.6).** `0 ? 1 = False`. -/
theorem eq_2_6 : predSet (ofNat 0) = botElem := by
  ext k
  simp [predSet, ofNat, botElem]

end DomainTheory.DataTypesAsLattices
```

4

eq_2_6.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.predSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.6).** `0 ? 1 = False`. -/
theorem eq_2_6 : predSet (ofNat 0) = botElem := by
  ext k
  simp [predSet, ofNat, botElem]

end DomainTheory.DataTypesAsLattices
```

5

eq_2_6.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.predSet

@[expose] public section
```

```
namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.6).** `0 ? 1 = False`. -/
theorem eq_2_6 : predSet (ofNat 0) = botElem := by
  ext k
  simp [predSet, ofNat, botElem]

end DomainTheory.DataTypesAsLattices
```

6

eq_2_6.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.predSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.6).** `0 ? 1 = False`. -/
theorem eq_2_6 : predSet (ofNat 0) = botElem := by
  ext k
  simp [predSet, ofNat, botElem]

end DomainTheory.DataTypesAsLattices
```

7

eq_2_6.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.predSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.6).** `0 ? 1 = False`. -/
theorem eq_2_6 : predSet (ofNat 0) = botElem := by
  ext k
  simp [predSet, ofNat, botElem]

end DomainTheory.DataTypesAsLattices
```

8

eq_2_6.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.predSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.6).** `0 ? 1 = False`. -/
theorem eq_2_6 : predSet (ofNat 0) = botElem := by
  ext k
  simp [predSet, ofNat, botElem]

end DomainTheory.DataTypesAsLattices
```

9

eq_2_7_bot.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.7).** Conditional cases. -/
theorem eq_2_7_bot (x y : Pomega) : condSet botElem x y = botElem := by
  ext n
  simp [condSet, botElem]

end DomainTheory.DataTypesAsLattices
```

0

eq_2_7_bot.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.7).** Conditional cases. -/
theorem eq_2_7_bot (x y : Pomega) : condSet botElem x y = botElem := by
  ext n
```

```
  simp [condSet, botElem]

end DomainTheory.DataTypesAsLattices
```

1

eq_2_7_bot.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.7).** Conditional cases. -/
theorem eq_2_7_bot (x y : Pomega) : condSet botElem x y = botElem := by
  ext n
  simp [condSet, botElem]

end DomainTheory.DataTypesAsLattices
```

2

eq_2_7_bot.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.7).** Conditional cases. -/
theorem eq_2_7_bot (x y : Pomega) : condSet botElem x y = botElem := by
  ext n
  simp [condSet, botElem]

end DomainTheory.DataTypesAsLattices
```

3

eq_2_7_bot.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
```

```
/-- **Scott 1976, (2.7).** Conditional cases. -/
theorem eq_2_7_bot (x y : Pomega) : condSet botElem x y = botElem := by
  ext n
  simp [condSet, botElem]

end DomainTheory.DataTypesAsLattices
```

4

eq_2_7_bot.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.7).** Conditional cases. -/
theorem eq_2_7_bot (x y : Pomega) : condSet botElem x y = botElem := by
  ext n
  simp [condSet, botElem]

end DomainTheory.DataTypesAsLattices
```

5

eq_2_7_bot.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.7).** Conditional cases. -/
theorem eq_2_7_bot (x y : Pomega) : condSet botElem x y = botElem := by
  ext n
  simp [condSet, botElem]

end DomainTheory.DataTypesAsLattices
```

6

eq_2_7_bot.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.7).** Conditional cases. -/
theorem eq_2_7_bot (x y : Pomega) : condSet botElem x y = botElem := by
  ext n
  simp [condSet, botElem]

end DomainTheory.DataTypesAsLattices
```

7

eq_2_7_bot.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.7).** Conditional cases. -/
theorem eq_2_7_bot (x y : Pomega) : condSet botElem x y = botElem := by
  ext n
  simp [condSet, botElem]

end DomainTheory.DataTypesAsLattices
```

8

eq_2_7_bot.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.7).** Conditional cases. -/
theorem eq_2_7_bot (x y : Pomega) : condSet botElem x y = botElem := by
  ext n
  simp [condSet, botElem]

end DomainTheory.DataTypesAsLattices
```

9

eq_2_7_mix.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_7_mix (x y z : Pomega) (h0 : 0  in  z) (hne : z  !=  ofNat 0) :
    condSet z x y = x  U  y := by
  have hpos : exists  t, t + 1  in  z := by
    by_contra h
    apply hne
    ext k
    constructor
    * intro hk
      have : k = 0 := by
        by_contra hk0
        exact h <k - 1, by
          have : k = (k - 1) + 1 := (Nat.succ_pred_eq_of_ne_zero hk0).symm
          rwa [ <-  this]>
      simp [this, ofNat]
    * intro hk
      simp [ofNat] at hk
      subst hk
      exact h0
  ext n
  constructor
  * intro hn
    rcases hn with <hnx, _> | <hny, _>
    * exact Or.inl hnx
    * exact Or.inr hny
  * intro hn
    rcases hn with hnx | hny
    * exact Or.inl <hnx, h0>
    * exact Or.inr <hny, hpos>

end DomainTheory.DataTypesAsLattices
```

0

eq_2_7_mix.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
```

```
theorem eq_2_7_mix (x y z : Pomega) (h0 : 0  in  z) (hne : z  !=  ofNat 0) :
    condSet z x y = x  U  y := by
  have hpos : exists  t, t + 1  in  z := by
    by_contra h
    apply hne
    ext k
    constructor
    * intro hk
      have : k = 0 := by
        by_contra hk0
        exact h <k - 1, by
          have : k = (k - 1) + 1 := (Nat.succ_pred_eq_of_ne_zero hk0).symm
          rwa [ <-  this]>
      simp [this, ofNat]
    * intro hk
      simp [ofNat] at hk
      subst hk
      exact h0
  ext n
  constructor
  * intro hn
    rcases hn with <hnx, _> | <hny, _>
    * exact Or.inl hnx
    * exact Or.inr hny
  * intro hn
    rcases hn with hnx | hny
    * exact Or.inl <hnx, h0>
    * exact Or.inr <hny, hpos>

end DomainTheory.DataTypesAsLattices
```

1

eq_2_7_mix.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_7_mix (x y z : Pomega) (h0 : 0  in  z) (hne : z  !=  ofNat 0) :
    condSet z x y = x  U  y := by
  have hpos : exists  t, t + 1  in  z := by
    by_contra h
    apply hne
    ext k
    constructor
    * intro hk
      have : k = 0 := by
```

```
        by_contra hk0
        exact h <k - 1, by
          have : k = (k - 1) + 1 := (Nat.succ_pred_eq_of_ne_zero hk0).symm
          rwa [ <-  this]>
      simp [this, ofNat]
    * intro hk
      simp [ofNat] at hk
      subst hk
      exact h0
  ext n
  constructor
  * intro hn
    rcases hn with <hnx, _> | <hny, _>
    * exact Or.inl hnx
    * exact Or.inr hny
  * intro hn
    rcases hn with hnx | hny
    * exact Or.inl <hnx, h0>
    * exact Or.inr <hny, hpos>

end DomainTheory.DataTypesAsLattices
```

2

eq_2_7_mix.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_7_mix (x y z : Pomega) (h0 : 0  in  z) (hne : z  !=  ofNat 0) :
    condSet z x y = x  U  y := by
  have hpos : exists  t, t + 1  in  z := by
    by_contra h
    apply hne
    ext k
    constructor
    * intro hk
      have : k = 0 := by
        by_contra hk0
        exact h <k - 1, by
          have : k = (k - 1) + 1 := (Nat.succ_pred_eq_of_ne_zero hk0).symm
          rwa [ <-  this]>
      simp [this, ofNat]
    * intro hk
      simp [ofNat] at hk
      subst hk
      exact h0
  ext n
```

```
  constructor
  * intro hn
    rcases hn with <hnx, _> | <hny, _>
    * exact Or.inl hnx
    * exact Or.inr hny
  * intro hn
    rcases hn with hnx | hny
    * exact Or.inl <hnx, h0>
    * exact Or.inr <hny, hpos>

end DomainTheory.DataTypesAsLattices
```

3

eq_2_7_mix.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_7_mix (x y z : Pomega) (h0 : 0  in  z) (hne : z  !=  ofNat 0) :
    condSet z x y = x  U  y := by
  have hpos : exists  t, t + 1  in  z := by
    by_contra h
    apply hne
    ext k
    constructor
    * intro hk
      have : k = 0 := by
        by_contra hk0
        exact h <k - 1, by
          have : k = (k - 1) + 1 := (Nat.succ_pred_eq_of_ne_zero hk0).symm
          rwa [ <-  this]>
      simp [this, ofNat]
    * intro hk
      simp [ofNat] at hk
      subst hk
      exact h0
  ext n
  constructor
  * intro hn
    rcases hn with <hnx, _> | <hny, _>
    * exact Or.inl hnx
    * exact Or.inr hny
  * intro hn
    rcases hn with hnx | hny
    * exact Or.inl <hnx, h0>
    * exact Or.inr <hny, hpos>
```

```
end DomainTheory.DataTypesAsLattices
```

4

eq_2_7_mix.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_7_mix (x y z : Pomega) (h0 : 0  in  z) (hne : z  !=  ofNat 0) :
    condSet z x y = x  U  y := by
  have hpos : exists  t, t + 1  in  z := by
    by_contra h
    apply hne
    ext k
    constructor
    * intro hk
      have : k = 0 := by
        by_contra hk0
        exact h <k - 1, by
          have : k = (k - 1) + 1 := (Nat.succ_pred_eq_of_ne_zero hk0).symm
          rwa [ <-  this]>
      simp [this, ofNat]
    * intro hk
      simp [ofNat] at hk
      subst hk
      exact h0
  ext n
  constructor
  * intro hn
    rcases hn with <hnx, _> | <hny, _>
    * exact Or.inl hnx
    * exact Or.inr hny
  * intro hn
    rcases hn with hnx | hny
    * exact Or.inl <hnx, h0>
    * exact Or.inr <hny, hpos>

end DomainTheory.DataTypesAsLattices
```

5

eq_2_7_mix.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
```

```
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_7_mix (x y z : Pomega) (h0 : 0  in  z) (hne : z  !=  ofNat 0) :
    condSet z x y = x  U  y := by
  have hpos : exists  t, t + 1  in  z := by
    by_contra h
    apply hne
    ext k
    constructor
    * intro hk
      have : k = 0 := by
        by_contra hk0
        exact h <k - 1, by
          have : k = (k - 1) + 1 := (Nat.succ_pred_eq_of_ne_zero hk0).symm
          rwa [ <-  this]>
      simp [this, ofNat]
    * intro hk
      simp [ofNat] at hk
      subst hk
      exact h0
  ext n
  constructor
  * intro hn
    rcases hn with <hnx, _> | <hny, _>
    * exact Or.inl hnx
    * exact Or.inr hny
  * intro hn
    rcases hn with hnx | hny
    * exact Or.inl <hnx, h0>
    * exact Or.inr <hny, hpos>

end DomainTheory.DataTypesAsLattices
```

6

eq_2_7_mix.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_7_mix (x y z : Pomega) (h0 : 0  in  z) (hne : z  !=  ofNat 0) :
    condSet z x y = x  U  y := by
  have hpos : exists  t, t + 1  in  z := by
```

```
    by_contra h
    apply hne
    ext k
    constructor
    * intro hk
      have : k = 0 := by
        by_contra hk0
        exact h <k - 1, by
          have : k = (k - 1) + 1 := (Nat.succ_pred_eq_of_ne_zero hk0).symm
          rwa [ <-  this]>
      simp [this, ofNat]
    * intro hk
      simp [ofNat] at hk
      subst hk
      exact h0
  ext n
  constructor
  * intro hn
    rcases hn with <hnx, _> | <hny, _>
    * exact Or.inl hnx
    * exact Or.inr hny
  * intro hn
    rcases hn with hnx | hny
    * exact Or.inl <hnx, h0>
    * exact Or.inr <hny, hpos>

end DomainTheory.DataTypesAsLattices
```

7

eq_2_7_mix.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_7_mix (x y z : Pomega) (h0 : 0  in  z) (hne : z  !=  ofNat 0) :
    condSet z x y = x  U  y := by
  have hpos : exists  t, t + 1  in  z := by
    by_contra h
    apply hne
    ext k
    constructor
    * intro hk
      have : k = 0 := by
        by_contra hk0
        exact h <k - 1, by
          have : k = (k - 1) + 1 := (Nat.succ_pred_eq_of_ne_zero hk0).symm
          rwa [ <-  this]>
```

```
      simp [this, ofNat]
    * intro hk
      simp [ofNat] at hk
      subst hk
      exact h0
  ext n
  constructor
  * intro hn
    rcases hn with <hnx, _> | <hny, _>
    * exact Or.inl hnx
    * exact Or.inr hny
  * intro hn
    rcases hn with hnx | hny
    * exact Or.inl <hnx, h0>
    * exact Or.inr <hny, hpos>

end DomainTheory.DataTypesAsLattices
```

8

eq_2_7_mix.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_7_mix (x y z : Pomega) (h0 : 0  in  z) (hne : z  !=  ofNat 0) :
    condSet z x y = x  U  y := by
  have hpos : exists  t, t + 1  in  z := by
    by_contra h
    apply hne
    ext k
    constructor
    * intro hk
      have : k = 0 := by
        by_contra hk0
        exact h <k - 1, by
          have : k = (k - 1) + 1 := (Nat.succ_pred_eq_of_ne_zero hk0).symm
          rwa [ <-  this]>
      simp [this, ofNat]
    * intro hk
      simp [ofNat] at hk
      subst hk
      exact h0
  ext n
  constructor
  * intro hn
    rcases hn with <hnx, _> | <hny, _>
    * exact Or.inl hnx
```

```
    * exact Or.inr hny
  * intro hn
    rcases hn with hnx | hny
    * exact Or.inl <hnx, h0>
    * exact Or.inr <hny, hpos>

end DomainTheory.DataTypesAsLattices
```

9

eq_2_7_pos.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.7), remaining cases.** -/
theorem eq_2_7_pos (x y z : Pomega) (h0 : 0  notin  z) (hne : z  !=  botElem) :
    condSet z x y = y := by
  have hpos : exists  t, t + 1  in  z := by
    obtain <k, hk> := Set.nonempty_iff_ne_empty.mpr hne
    have : k  !=  0 := fun h => h0 (h  |>  hk)
    exact <k - 1, by
      have : k = (k - 1) + 1 := (Nat.succ_pred_eq_of_ne_zero this).symm
      rwa [ <-  this]>
  ext n
  constructor
  * intro hn
    rcases hn with <_, hz> | <hny, _>
    * exact (h0 hz).elim
    * exact hny
  * intro hny
    exact Or.inr <hny, hpos>

end DomainTheory.DataTypesAsLattices
```

0

eq_2_7_pos.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.7), remaining cases.** -/
theorem eq_2_7_pos (x y z : Pomega) (h0 : 0  notin  z) (hne : z  !=  botElem) :
```

```
    condSet z x y = y := by
  have hpos : exists  t, t + 1  in  z := by
    obtain <k, hk> := Set.nonempty_iff_ne_empty.mpr hne
    have : k  !=  0 := fun h => h0 (h  |>  hk)
    exact <k - 1, by
      have : k = (k - 1) + 1 := (Nat.succ_pred_eq_of_ne_zero this).symm
      rwa [ <-  this]>
  ext n
  constructor
  * intro hn
    rcases hn with <_, hz> | <hny, _>
    * exact (h0 hz).elim
    * exact hny
  * intro hny
    exact Or.inr <hny, hpos>

end DomainTheory.DataTypesAsLattices
```

1

eq_2_7_pos.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.7), remaining cases.** -/
theorem eq_2_7_pos (x y z : Pomega) (h0 : 0  notin  z) (hne : z  !=  botElem) :
    condSet z x y = y := by
  have hpos : exists  t, t + 1  in  z := by
    obtain <k, hk> := Set.nonempty_iff_ne_empty.mpr hne
    have : k  !=  0 := fun h => h0 (h  |>  hk)
    exact <k - 1, by
      have : k = (k - 1) + 1 := (Nat.succ_pred_eq_of_ne_zero this).symm
      rwa [ <-  this]>
  ext n
  constructor
  * intro hn
    rcases hn with <_, hz> | <hny, _>
    * exact (h0 hz).elim
    * exact hny
  * intro hny
    exact Or.inr <hny, hpos>

end DomainTheory.DataTypesAsLattices
```

2

eq_2_7_pos.lean

```
module
```

```
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.7), remaining cases.** -/
theorem eq_2_7_pos (x y z : Pomega) (h0 : 0  notin  z) (hne : z  !=  botElem) :
    condSet z x y = y := by
  have hpos : exists  t, t + 1  in  z := by
    obtain <k, hk> := Set.nonempty_iff_ne_empty.mpr hne
    have : k  !=  0 := fun h => h0 (h  |>  hk)
    exact <k - 1, by
      have : k = (k - 1) + 1 := (Nat.succ_pred_eq_of_ne_zero this).symm
      rwa [ <-  this]>
  ext n
  constructor
  * intro hn
    rcases hn with <_, hz> | <hny, _>
    * exact (h0 hz).elim
    * exact hny
  * intro hny
    exact Or.inr <hny, hpos>

end DomainTheory.DataTypesAsLattices
```

3

eq_2_7_pos.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.7), remaining cases.** -/
theorem eq_2_7_pos (x y z : Pomega) (h0 : 0  notin  z) (hne : z  !=  botElem) :
    condSet z x y = y := by
  have hpos : exists  t, t + 1  in  z := by
    obtain <k, hk> := Set.nonempty_iff_ne_empty.mpr hne
    have : k  !=  0 := fun h => h0 (h  |>  hk)
    exact <k - 1, by
      have : k = (k - 1) + 1 := (Nat.succ_pred_eq_of_ne_zero this).symm
      rwa [ <-  this]>
  ext n
  constructor
  * intro hn
    rcases hn with <_, hz> | <hny, _>
    * exact (h0 hz).elim
    * exact hny
```

```
  * intro hny
    exact Or.inr <hny, hpos>

end DomainTheory.DataTypesAsLattices
```

4

eq_2_7_pos.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.7), remaining cases.** -/
theorem eq_2_7_pos (x y z : Pomega) (h0 : 0  notin  z) (hne : z  !=  botElem) :
    condSet z x y = y := by
  have hpos : exists  t, t + 1  in  z := by
    obtain <k, hk> := Set.nonempty_iff_ne_empty.mpr hne
    have : k  !=  0 := fun h => h0 (h  |>  hk)
    exact <k - 1, by
      have : k = (k - 1) + 1 := (Nat.succ_pred_eq_of_ne_zero this).symm
      rwa [ <-  this]>
  ext n
  constructor
  * intro hn
    rcases hn with <_, hz> | <hny, _>
    * exact (h0 hz).elim
    * exact hny
  * intro hny
    exact Or.inr <hny, hpos>

end DomainTheory.DataTypesAsLattices
```

5

eq_2_7_pos.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.7), remaining cases.** -/
theorem eq_2_7_pos (x y z : Pomega) (h0 : 0  notin  z) (hne : z  !=  botElem) :
    condSet z x y = y := by
  have hpos : exists  t, t + 1  in  z := by
    obtain <k, hk> := Set.nonempty_iff_ne_empty.mpr hne
```

```
    have : k  !=  0 := fun h => h0 (h  |>  hk)
    exact <k - 1, by
      have : k = (k - 1) + 1 := (Nat.succ_pred_eq_of_ne_zero this).symm
      rwa [ <-  this]>
  ext n
  constructor
  * intro hn
    rcases hn with <_, hz> | <hny, _>
    * exact (h0 hz).elim
    * exact hny
  * intro hny
    exact Or.inr <hny, hpos>

end DomainTheory.DataTypesAsLattices
```

6

eq_2_7_pos.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.7), remaining cases.** -/
theorem eq_2_7_pos (x y z : Pomega) (h0 : 0  notin  z) (hne : z  !=  botElem) :
    condSet z x y = y := by
  have hpos : exists  t, t + 1  in  z := by
    obtain <k, hk> := Set.nonempty_iff_ne_empty.mpr hne
    have : k  !=  0 := fun h => h0 (h  |>  hk)
    exact <k - 1, by
      have : k = (k - 1) + 1 := (Nat.succ_pred_eq_of_ne_zero this).symm
      rwa [ <-  this]>
  ext n
  constructor
  * intro hn
    rcases hn with <_, hz> | <hny, _>
    * exact (h0 hz).elim
    * exact hny
  * intro hny
    exact Or.inr <hny, hpos>

end DomainTheory.DataTypesAsLattices
```

7

eq_2_7_pos.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.7), remaining cases.** -/
theorem eq_2_7_pos (x y z : Pomega) (h0 : 0  notin  z) (hne : z  !=  botElem) :
    condSet z x y = y := by
  have hpos : exists  t, t + 1  in  z := by
    obtain <k, hk> := Set.nonempty_iff_ne_empty.mpr hne
    have : k  !=  0 := fun h => h0 (h  |>  hk)
    exact <k - 1, by
      have : k = (k - 1) + 1 := (Nat.succ_pred_eq_of_ne_zero this).symm
      rwa [ <-  this]>
  ext n
  constructor
  * intro hn
    rcases hn with <_, hz> | <hny, _>
    * exact (h0 hz).elim
    * exact hny
  * intro hny
    exact Or.inr <hny, hpos>

end DomainTheory.DataTypesAsLattices
```

8

eq_2_7_pos.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condSet

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- **Scott 1976, (2.7), remaining cases.** -/
theorem eq_2_7_pos (x y z : Pomega) (h0 : 0  notin  z) (hne : z  !=  botElem) :
    condSet z x y = y := by
  have hpos : exists  t, t + 1  in  z := by
    obtain <k, hk> := Set.nonempty_iff_ne_empty.mpr hne
    have : k  !=  0 := fun h => h0 (h  |>  hk)
    exact <k - 1, by
      have : k = (k - 1) + 1 := (Nat.succ_pred_eq_of_ne_zero this).symm
      rwa [ <-  this]>
  ext n
  constructor
  * intro hn
    rcases hn with <_, hz> | <hny, _>
    * exact (h0 hz).elim
    * exact hny
  * intro hny
    exact Or.inr <hny, hpos>
```

```
end DomainTheory.DataTypesAsLattices
```

9

eq_2_7_succ.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_7_succ (x y : Pomega) (k : Nat) :
    condSet (ofNat (k + 1)) x y = y := by
  ext n
  simp [condSet, ofNat]

end DomainTheory.DataTypesAsLattices
```

0

eq_2_7_succ.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_7_succ (x y : Pomega) (k : Nat) :
    condSet (ofNat (k + 1)) x y = y := by
  ext n
  simp [condSet, ofNat]

end DomainTheory.DataTypesAsLattices
```

1

eq_2_7_succ.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_7_succ (x y : Pomega) (k : Nat) :
    condSet (ofNat (k + 1)) x y = y := by
  ext n
  simp [condSet, ofNat]

end DomainTheory.DataTypesAsLattices
```

2

eq_2_7_succ.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_7_succ (x y : Pomega) (k : Nat) :
    condSet (ofNat (k + 1)) x y = y := by
  ext n
  simp [condSet, ofNat]

end DomainTheory.DataTypesAsLattices
```

3

eq_2_7_succ.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_7_succ (x y : Pomega) (k : Nat) :
    condSet (ofNat (k + 1)) x y = y := by
  ext n
  simp [condSet, ofNat]

end DomainTheory.DataTypesAsLattices
```

4

eq_2_7_succ.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_7_succ (x y : Pomega) (k : Nat) :
    condSet (ofNat (k + 1)) x y = y := by
  ext n
  simp [condSet, ofNat]

end DomainTheory.DataTypesAsLattices
```

5

eq_2_7_succ.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_7_succ (x y : Pomega) (k : Nat) :
    condSet (ofNat (k + 1)) x y = y := by
  ext n
  simp [condSet, ofNat]

end DomainTheory.DataTypesAsLattices
```

6

eq_2_7_succ.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section
```

```
namespace DomainTheory.DataTypesAsLattices

theorem eq_2_7_succ (x y : Pomega) (k : Nat) :
    condSet (ofNat (k + 1)) x y = y := by
  ext n
  simp [condSet, ofNat]

end DomainTheory.DataTypesAsLattices
```

7

eq_2_7_succ.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_7_succ (x y : Pomega) (k : Nat) :
    condSet (ofNat (k + 1)) x y = y := by
  ext n
  simp [condSet, ofNat]

end DomainTheory.DataTypesAsLattices
```

8

eq_2_7_succ.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_7_succ (x y : Pomega) (k : Nat) :
    condSet (ofNat (k + 1)) x y = y := by
  ext n
  simp [condSet, ofNat]

end DomainTheory.DataTypesAsLattices
```

9

eq_2_7_zero.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_7_zero (x y : Pomega) : condSet (ofNat 0) x y = x := by
  ext n
  simp [condSet, ofNat]

end DomainTheory.DataTypesAsLattices
```

0

eq_2_7_zero.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_7_zero (x y : Pomega) : condSet (ofNat 0) x y = x := by
  ext n
  simp [condSet, ofNat]

end DomainTheory.DataTypesAsLattices
```

1

eq_2_7_zero.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_7_zero (x y : Pomega) : condSet (ofNat 0) x y = x := by
```

```
  ext n
  simp [condSet, ofNat]

end DomainTheory.DataTypesAsLattices
```

2

eq_2_7_zero.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_7_zero (x y : Pomega) : condSet (ofNat 0) x y = x := by
  ext n
  simp [condSet, ofNat]

end DomainTheory.DataTypesAsLattices
```

3

eq_2_7_zero.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_7_zero (x y : Pomega) : condSet (ofNat 0) x y = x := by
  ext n
  simp [condSet, ofNat]

end DomainTheory.DataTypesAsLattices
```

4

eq_2_7_zero.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
```

```
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_7_zero (x y : Pomega) : condSet (ofNat 0) x y = x := by
  ext n
  simp [condSet, ofNat]

end DomainTheory.DataTypesAsLattices
```

5

eq_2_7_zero.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_7_zero (x y : Pomega) : condSet (ofNat 0) x y = x := by
  ext n
  simp [condSet, ofNat]

end DomainTheory.DataTypesAsLattices
```

6

eq_2_7_zero.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_7_zero (x y : Pomega) : condSet (ofNat 0) x y = x := by
  ext n
  simp [condSet, ofNat]

end DomainTheory.DataTypesAsLattices
```

7

eq_2_7_zero.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_7_zero (x y : Pomega) : condSet (ofNat 0) x y = x := by
  ext n
  simp [condSet, ofNat]

end DomainTheory.DataTypesAsLattices
```

8

eq_2_7_zero.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.mem_ofNat
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Lambda.ofNat

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

theorem eq_2_7_zero (x y : Pomega) : condSet (ofNat 0) x y = x := by
  ext n
  simp [condSet, ofNat]

end DomainTheory.DataTypesAsLattices
```

9

5.4 Constructivity summary

Bridge	Footprint	Notes
Nbhd ↔ InfoSys (both directions)	+ inherited `Classical.choice`	Decidable coding; Mathlib 4.33 `Finset` instance frontier
CL ↔ RoundFilter	`{propext, Quot.sound}`	Roundness is order-theoretic
InfoSys ↔ Ideal	+ inherited `Classical.choice`	Factoids 4.4–4.5; type-level `Finset` frontier
Algebraic ⇒ CL	classical	1972 topological $\ll$

Bridge	Footprint	Notes
Product / function space (1982)	+ inherited `Classical.choice`	No witness extraction in the maps
Separated sum (1982)	+ `Classical.choice`	Token polarity trichotomy
Factoid 4.6 ApproxMap ↔ ScottContinuous	+ inherited `Classical.choice`	Type-level `Finset` frontier
ScottMap conjugation	+ `Classical.choice`	Via 1972 `ScottMap`

Target discipline (from the 1982 package): prefer constructive proofs wherever Scott's 1982 text emphasizes constructivity; call out classical frontiers explicitly rather than hiding choice in automation.

Mathlib 4.33 makes `Finset.instSetLike` and `Finset.instPartialOrder` depend on `Classical.choice`. Consequently `#print axioms` reports choice for declarations whose types traverse `InfoSys`, even when their proof terms only manipulate explicit finite data. Here "no witness extraction" distinguishes that inherited library-instance footprint from genuine uses such as separated-sum polarity classification.

5.5 Worked example — S-expressions / trees

Dependencies. Inside WorkedExampleSExpr, each box is one Lean file. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. BridgeTheorems is the Lean files in that chapter folder, not in a subfolder. The arrowhead points at the file or folder that imports, and the number is how many direct imports.

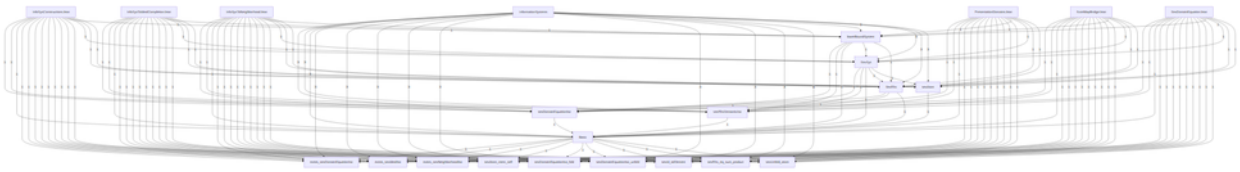


Figure 49: Worked example — S-expressions / trees.

Lean packaging: `WorkedExampleSExpr.lean`, `SexDomainEquation.lean`. The Palomar compared family includes `sexNeighborhoodIso`, `sexIdealIso`, and `sexDomainEquationIso` (Challenge / `comparator.json`). The compared domain-equation declaration is the order isomorphism `SexSys.Element ≃o SexRhs.Element` (`|T| ≃o |A + (T × T)|`), not the generic semantic factor `WithBot (|A| ⊕ (|T| × |T|)) ≃o |A + (T × T)|`.

5.5.1 Overview

The bridges of §§2–3 are abstract. This section fixes **one** concrete domain — Scott's S-expression / tree equation `T ≅ A + (T × T)` (**[Sco82]**, Factoid 8.1) over the $\mathbb{N}$ lower-bound atom system (Factoid 2.4) — and asks what that domain looks like in each paradigm:

Paradigm	What an element of `T` is
1982	a consistent, deductively closed set of tree tokens

Paradigm	What an element of `T` is
1980	a filter of basic opens `[u]` on that token domain
1972	a point of an algebraic continuous lattice (ideal of finite elements)

We build the domain natively as an information system (§5.2), then **read the same carrier** as a neighbourhood system (§5.3) and as a continuous lattice (§5.4). Section 5.5 exhibits the order isomorphisms for this instance — so that, on S-expressions,

`T_1982 ≃o T_1980 ≃o Ideal(K(T))` and `Ideal(K(T))` is a continuous lattice,

hence transitively `T_1982 ≃o T_1980` and (via the algebraic ⇒ continuous frontier) the 1972 presentation of the same domain. Order isomorphism is an equivalence relation, so the three presentations determine one domain up to `≃o`. Section 5.6 records two extras that live on top of the carriers: the domain equation at the level of domains, and identity as a morphism in both the approximable-map and Scott-continuous languages.

5.5.2 Information system (1982)

Atoms are propositions `n ≤ x` on ℕ (`lowerBoundSystem`). The tree system `SexSys := treeSystem lowerBoundSystem` has inductive tokens `TreeToken ℕ` (`bot` / `atom` / `pairL` / `pairR`). Consistency and entailment are Scott's sum×product clauses; Factoid 8.1 records that this is literally the information system of the right-hand side `A + (T × T)`:

- `SexRhs = sumSystem A (productSystem T T)` (`sexRhs_eq_sum_product`);
- token unfolding `treeUnfold` sends `atom n` to the left summand (`sexUnfold_atom`) and is a retraction onto the official sum-of-product carrier, with section `treeFold`;
- TreeEnt matches official product `ent_bot` on the two encodings of `(Δ, Δ)` (`pairL Δ` and `pairR Δ`), so closed elements are saturated for `ker(treeUnfold)`.

Finite elements include singleton atom closures `sexAtom n = ū` for `u = {atom n}`. Elements of `|T|` are the consistent closed sets of tokens — the 1982 presentation of the S-expression domain.

5.5.3 Neighbourhood system (1980)

From `|T|` one obtains a neighbourhood system whose basic opens are `[u] = {x ∈ |T| | ↑u ⊆ x}`. In this paradigm an element of the domain is a **filter** of such opens (not a raw set of tokens). Nothing new is invented: it is the image of `SexSys` under `InfoSysToNeighborhood`. The concrete iso of §5.5 below is exactly "read each token-element as its filter of basic opens."

5.5.4 Continuous lattice (1972)

Finite elements `K(T)` are the closures of consistent finsets. In the algebraic style that feeds Scott's 1972 theory, a domain element is an **ideal** of `K(T)` — the ideal completion of the compact basis. That poset of ideals is an algebraic complete lattice, hence a continuous lattice (`AlgebraicLattice ⇒ ContinuousLattice`, §3.5). This is the 1972 presentation of the same S-expression domain: points ordered by inclusion of ideals, with way-below recovered from the compact basis.

We do not claim a choice-free `D ≃o RoundFilter` story built from trees alone; the round ⇑-filter identification of §3.1 applies once one is already in the continuous-lattice setting.

5.5.5 Equivalence for this instance

Specializing the bridges of §3 to `SexSys` yields order isomorphisms of carriers:

- `sexNeighborhoodIso : |T| ≃o` filters of `[u] (InfoSysToNeighborhood.domainOrderIso)` — **1982 ≃o 1980**;
- `sexIdealIso : |T| ≃o Ideal (FiniteElement T) (InfoSysToIdealCompletion.domainOr` — **1982 ≃o algebraic / 1972 carrier**;
- `sexNeighborhoodIdealIso` (`neighborhood_ideal_iso`) — the constructive triangle closing **1980 ≃o Ideal(K(T))** without going back through tokens.

Composing any two edges recovers the third: isomorphism is transitive, so on this example the three presentations are pairwise equivalent. The 1972 corner uses the algebraic ⇒ continuous implication already flagged in §3.5 / §4. The neighbourhood ↔ information ↔ ideal triangle inherits `Classical.choice` only through the `Finset` instance frontier of §4.

5.5.6 Domain equation and morphisms

Beyond identifying the carriers, the same example exercises constructions and maps.

Domain equation. Factoid 8.1's token unfolding `treeUnfold` (section `treeFold`) lifts to an order isomorphism of the tree domain with the official RHS domain:

`sexDomainEquationIso : SexSys.Element ≃o SexRhs.Element`

(`treeDomainIso`; `|T| ≃o |A + (T × T)|`). The maps are verified unfolding: the carrier of `sexDomainEquationIso x` is `treeUnfold ″ x.carrier`, and the carrier of the inverse is the preimage `treeUnfold ⁻¹′ y.carrier` (`sexDomainEquationIso_unfold` / `sexDomainEquationIso_fold`). The only non-injectivity of `treeUnfold` is Scott's two encodings of the product bottom (`pairL ⊥` and `pairR ⊥`); after the `ent_bot` match, closed elements are saturated for that kernel, so image and preimage invert. The generic semantic factor `WithBot (|A| ⊕ (|T| × |T|)) ≃o SexRhs.Element` is the uncompared `sexRhsSemanticIso` (classical only through the 1982 sum trichotomy); it is **not** the compared fixed-point equation.

Morphisms. The identity approximable map `idMap T` (Prop 5.4) transports along Factoid 4.6 to a Scott-continuous endomap `sexIdScottContinuous` with `toFun = id` (`sexId_toElement`) — the same endomap, named once in each morphism language.

Axioms. Object-level neighbourhood / ideal isos, `sexDomainEquationIso`, and Factoid 4.6 for `idMap` audit to `{propext, Classical.choice, Quot.sound}` through the InfoSys/`Finset` frontier of §4. The uncompared semantic factor `sexRhsSemanticIso` additionally uses the 1982 sum trichotomy.

`exists_least_mem.lean`

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

lemma exists_least_mem {a : Pomega} (hex : exists  k, k  in  a) :
    exists  k, k  in  a  /\  forall  i < k, i  notin  a := by
  have : forall  n, n  in  a  ->  exists  k  <=  n, k  in  a  /\  forall  i < k, i
    notin  a := by
    intro n
    induction n using Nat.strong_induction_on with
    | h n ih =>
      intro hn
      rcases Classical.em (forall  i < n, i  notin  a) with hmin | hmin
      * exact <n, le_rfl, hn, hmin>
      * obtain <i, hi> := not_forall.mp hmin
        have hi' : i < n := (Classical.not_imp.mp hi).1
        have hia : i  in  a := not_not.mp (Classical.not_imp.mp hi).2
        obtain <k, hki, hk, hall> := ih i hi' hia
        exact <k, hki.trans hi'.le, hk, hall>
  obtain <n, hn> := hex
  obtain <k, _, hk, hall> := this n hn
  exact <k, hk, hall>

end DomainTheory.DataTypesAsLattices
```

0

exists_least_mem.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

lemma exists_least_mem {a : Pomega} (hex : exists  k, k  in  a) :
    exists  k, k  in  a  /\  forall  i < k, i  notin  a := by
  have : forall  n, n  in  a  ->  exists  k  <=  n, k  in  a  /\  forall  i < k, i
    notin  a := by
    intro n
    induction n using Nat.strong_induction_on with
    | h n ih =>
      intro hn
      rcases Classical.em (forall  i < n, i  notin  a) with hmin | hmin
      * exact <n, le_rfl, hn, hmin>
      * obtain <i, hi> := not_forall.mp hmin
        have hi' : i < n := (Classical.not_imp.mp hi).1
        have hia : i  in  a := not_not.mp (Classical.not_imp.mp hi).2
        obtain <k, hki, hk, hall> := ih i hi' hia
        exact <k, hki.trans hi'.le, hk, hall>
  obtain <n, hn> := hex
  obtain <k, _, hk, hall> := this n hn
  exact <k, hk, hall>
```

```
end DomainTheory.DataTypesAsLattices
```

1

exists_least_mem.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

lemma exists_least_mem {a : Pomega} (hex : exists  k, k  in  a) :
    exists  k, k  in  a  /\  forall  i < k, i  notin  a := by
  have : forall  n, n  in  a  ->  exists  k  <=  n, k  in  a  /\  forall  i < k, i
    notin  a := by
    intro n
    induction n using Nat.strong_induction_on with
    | h n ih =>
      intro hn
      rcases Classical.em (forall  i < n, i  notin  a) with hmin | hmin
      * exact <n, le_rfl, hn, hmin>
      * obtain <i, hi> := not_forall.mp hmin
        have hi' : i < n := (Classical.not_imp.mp hi).1
        have hia : i  in  a := not_not.mp (Classical.not_imp.mp hi).2
        obtain <k, hki, hk, hall> := ih i hi' hia
        exact <k, hki.trans hi'.le, hk, hall>
  obtain <n, hn> := hex
  obtain <k, _, hk, hall> := this n hn
  exact <k, hk, hall>

end DomainTheory.DataTypesAsLattices
```

2

exists_least_mem.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

lemma exists_least_mem {a : Pomega} (hex : exists  k, k  in  a) :
    exists  k, k  in  a  /\  forall  i < k, i  notin  a := by
  have : forall  n, n  in  a  ->  exists  k  <=  n, k  in  a  /\  forall  i < k, i
    notin  a := by
    intro n
    induction n using Nat.strong_induction_on with
    | h n ih =>
      intro hn
      rcases Classical.em (forall  i < n, i  notin  a) with hmin | hmin
```

```
      * exact <n, le_rfl, hn, hmin>
      * obtain <i, hi> := not_forall.mp hmin
        have hi' : i < n := (Classical.not_imp.mp hi).1
        have hia : i  in  a := not_not.mp (Classical.not_imp.mp hi).2
        obtain <k, hki, hk, hall> := ih i hi' hia
        exact <k, hki.trans hi'.le, hk, hall>
  obtain <n, hn> := hex
  obtain <k, _, hk, hall> := this n hn
  exact <k, hk, hall>

end DomainTheory.DataTypesAsLattices
```

3

exists_least_mem.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

lemma exists_least_mem {a : Pomega} (hex : exists  k, k  in  a) :
    exists  k, k  in  a  /\  forall  i < k, i  notin  a := by
  have : forall  n, n  in  a  ->  exists  k  <=  n, k  in  a  /\  forall  i < k, i
    notin  a := by
    intro n
    induction n using Nat.strong_induction_on with
    | h n ih =>
      intro hn
      rcases Classical.em (forall  i < n, i  notin  a) with hmin | hmin
      * exact <n, le_rfl, hn, hmin>
      * obtain <i, hi> := not_forall.mp hmin
        have hi' : i < n := (Classical.not_imp.mp hi).1
        have hia : i  in  a := not_not.mp (Classical.not_imp.mp hi).2
        obtain <k, hki, hk, hall> := ih i hi' hia
        exact <k, hki.trans hi'.le, hk, hall>
  obtain <n, hn> := hex
  obtain <k, _, hk, hall> := this n hn
  exact <k, hk, hall>

end DomainTheory.DataTypesAsLattices
```

4

exists_least_mem.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices
```

```
lemma exists_least_mem {a : Pomega} (hex : exists  k, k  in  a) :
    exists  k, k  in  a  /\  forall  i < k, i  notin  a := by
  have : forall  n, n  in  a  ->  exists  k  <=  n, k  in  a  /\  forall  i < k, i
    notin  a := by
    intro n
    induction n using Nat.strong_induction_on with
    | h n ih =>
      intro hn
      rcases Classical.em (forall  i < n, i  notin  a) with hmin | hmin
      * exact <n, le_rfl, hn, hmin>
      * obtain <i, hi> := not_forall.mp hmin
        have hi' : i < n := (Classical.not_imp.mp hi).1
        have hia : i  in  a := not_not.mp (Classical.not_imp.mp hi).2
        obtain <k, hki, hk, hall> := ih i hi' hia
        exact <k, hki.trans hi'.le, hk, hall>
  obtain <n, hn> := hex
  obtain <k, _, hk, hall> := this n hn
  exact <k, hk, hall>

end DomainTheory.DataTypesAsLattices
```

5

exists_least_mem.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

lemma exists_least_mem {a : Pomega} (hex : exists  k, k  in  a) :
    exists  k, k  in  a  /\  forall  i < k, i  notin  a := by
  have : forall  n, n  in  a  ->  exists  k  <=  n, k  in  a  /\  forall  i < k, i
    notin  a := by
    intro n
    induction n using Nat.strong_induction_on with
    | h n ih =>
      intro hn
      rcases Classical.em (forall  i < n, i  notin  a) with hmin | hmin
      * exact <n, le_rfl, hn, hmin>
      * obtain <i, hi> := not_forall.mp hmin
        have hi' : i < n := (Classical.not_imp.mp hi).1
        have hia : i  in  a := not_not.mp (Classical.not_imp.mp hi).2
        obtain <k, hki, hk, hall> := ih i hi' hia
        exact <k, hki.trans hi'.le, hk, hall>
  obtain <n, hn> := hex
  obtain <k, _, hk, hall> := this n hn
  exact <k, hk, hall>

end DomainTheory.DataTypesAsLattices
```

6

exists_least_mem.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

lemma exists_least_mem {a : Pomega} (hex : exists  k, k  in  a) :
    exists  k, k  in  a  /\  forall  i < k, i  notin  a := by
  have : forall  n, n  in  a  ->  exists  k  <=  n, k  in  a  /\  forall  i < k, i
    notin  a := by
    intro n
    induction n using Nat.strong_induction_on with
    | h n ih =>
      intro hn
      rcases Classical.em (forall  i < n, i  notin  a) with hmin | hmin
      * exact <n, le_rfl, hn, hmin>
      * obtain <i, hi> := not_forall.mp hmin
        have hi' : i < n := (Classical.not_imp.mp hi).1
        have hia : i  in  a := not_not.mp (Classical.not_imp.mp hi).2
        obtain <k, hki, hk, hall> := ih i hi' hia
        exact <k, hki.trans hi'.le, hk, hall>
  obtain <n, hn> := hex
  obtain <k, _, hk, hall> := this n hn
  exact <k, hk, hall>

end DomainTheory.DataTypesAsLattices
```

7

exists_least_mem.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

lemma exists_least_mem {a : Pomega} (hex : exists  k, k  in  a) :
    exists  k, k  in  a  /\  forall  i < k, i  notin  a := by
  have : forall  n, n  in  a  ->  exists  k  <=  n, k  in  a  /\  forall  i < k, i
    notin  a := by
    intro n
    induction n using Nat.strong_induction_on with
    | h n ih =>
      intro hn
      rcases Classical.em (forall  i < n, i  notin  a) with hmin | hmin
      * exact <n, le_rfl, hn, hmin>
      * obtain <i, hi> := not_forall.mp hmin
        have hi' : i < n := (Classical.not_imp.mp hi).1
        have hia : i  in  a := not_not.mp (Classical.not_imp.mp hi).2
```

```
        obtain <k, hki, hk, hall> := ih i hi' hia
        exact <k, hki.trans hi'.le, hk, hall>
  obtain <n, hn> := hex
  obtain <k, _, hk, hall> := this n hn
  exact <k, hk, hall>

end DomainTheory.DataTypesAsLattices
```

8

exists_least_mem.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

lemma exists_least_mem {a : Pomega} (hex : exists  k, k  in  a) :
    exists  k, k  in  a  /\  forall  i < k, i  notin  a := by
  have : forall  n, n  in  a  ->  exists  k  <=  n, k  in  a  /\  forall  i < k, i
    notin  a := by
    intro n
    induction n using Nat.strong_induction_on with
    | h n ih =>
      intro hn
      rcases Classical.em (forall  i < n, i  notin  a) with hmin | hmin
      * exact <n, le_rfl, hn, hmin>
      * obtain <i, hi> := not_forall.mp hmin
        have hi' : i < n := (Classical.not_imp.mp hi).1
        have hia : i  in  a := not_not.mp (Classical.not_imp.mp hi).2
        obtain <k, hki, hk, hall> := ih i hi' hia
        exact <k, hki.trans hi'.le, hk, hall>
  obtain <n, hn> := hex
  obtain <k, _, hk, hall> := this n hn
  exact <k, hk, hall>

end DomainTheory.DataTypesAsLattices
```

9

exists_least_not_mem.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

lemma exists_least_not_mem {a : Pomega} (hex : exists  k, k  notin  a) :
    exists  k, k  notin  a  /\  forall  i < k, i  in  a := by
```

```
  have : forall  n, n  notin  a  ->  exists  k  <=  n, k  notin  a  /\  forall  i < k,
     i  in  a := by
    intro n
    induction n using Nat.strong_induction_on with
    | h n ih =>
      intro hn
      rcases Classical.em (forall  i < n, i  in  a) with hmin | hmin
      * exact <n, le_rfl, hn, hmin>
      * obtain <i, hi> := not_forall.mp hmin
        have hi' : i < n := (Classical.not_imp.mp hi).1
        have hia : i  notin  a := (Classical.not_imp.mp hi).2
        obtain <k, hki, hk, hall> := ih i hi' hia
        exact <k, hki.trans hi'.le, hk, hall>
  obtain <n, hn> := hex
  obtain <k, _, hk, hall> := this n hn
  exact <k, hk, hall>

end DomainTheory.DataTypesAsLattices
```

0

exists_least_not_mem.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

lemma exists_least_not_mem {a : Pomega} (hex : exists  k, k  notin  a) :
    exists  k, k  notin  a  /\  forall  i < k, i  in  a := by
  have : forall  n, n  notin  a  ->  exists  k  <=  n, k  notin  a  /\  forall  i < k,
     i  in  a := by
    intro n
    induction n using Nat.strong_induction_on with
    | h n ih =>
      intro hn
      rcases Classical.em (forall  i < n, i  in  a) with hmin | hmin
      * exact <n, le_rfl, hn, hmin>
      * obtain <i, hi> := not_forall.mp hmin
        have hi' : i < n := (Classical.not_imp.mp hi).1
        have hia : i  notin  a := (Classical.not_imp.mp hi).2
        obtain <k, hki, hk, hall> := ih i hi' hia
        exact <k, hki.trans hi'.le, hk, hall>
  obtain <n, hn> := hex
  obtain <k, _, hk, hall> := this n hn
  exact <k, hk, hall>

end DomainTheory.DataTypesAsLattices
```

1

exists_least_not_mem.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

lemma exists_least_not_mem {a : Pomega} (hex : exists  k, k  notin  a) :
    exists  k, k  notin  a  /\  forall  i < k, i  in  a := by
  have : forall  n, n  notin  a  ->  exists  k  <=  n, k  notin  a  /\  forall  i < k,
     i  in  a := by
    intro n
    induction n using Nat.strong_induction_on with
    | h n ih =>
      intro hn
      rcases Classical.em (forall  i < n, i  in  a) with hmin | hmin
      * exact <n, le_rfl, hn, hmin>
      * obtain <i, hi> := not_forall.mp hmin
        have hi' : i < n := (Classical.not_imp.mp hi).1
        have hia : i  notin  a := (Classical.not_imp.mp hi).2
        obtain <k, hki, hk, hall> := ih i hi' hia
        exact <k, hki.trans hi'.le, hk, hall>
  obtain <n, hn> := hex
  obtain <k, _, hk, hall> := this n hn
  exact <k, hk, hall>

end DomainTheory.DataTypesAsLattices
```

2

exists_least_not_mem.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

lemma exists_least_not_mem {a : Pomega} (hex : exists  k, k  notin  a) :
    exists  k, k  notin  a  /\  forall  i < k, i  in  a := by
  have : forall  n, n  notin  a  ->  exists  k  <=  n, k  notin  a  /\  forall  i < k,
     i  in  a := by
    intro n
    induction n using Nat.strong_induction_on with
    | h n ih =>
      intro hn
      rcases Classical.em (forall  i < n, i  in  a) with hmin | hmin
      * exact <n, le_rfl, hn, hmin>
      * obtain <i, hi> := not_forall.mp hmin
        have hi' : i < n := (Classical.not_imp.mp hi).1
        have hia : i  notin  a := (Classical.not_imp.mp hi).2
        obtain <k, hki, hk, hall> := ih i hi' hia
        exact <k, hki.trans hi'.le, hk, hall>
```

```
  obtain <n, hn> := hex
  obtain <k, _, hk, hall> := this n hn
  exact <k, hk, hall>

end DomainTheory.DataTypesAsLattices
```

3

exists_least_not_mem.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

lemma exists_least_not_mem {a : Pomega} (hex : exists  k, k  notin  a) :
    exists  k, k  notin  a  /\  forall  i < k, i  in  a := by
  have : forall  n, n  notin  a  ->  exists  k  <=  n, k  notin  a  /\  forall  i < k,
     i  in  a := by
    intro n
    induction n using Nat.strong_induction_on with
    | h n ih =>
      intro hn
      rcases Classical.em (forall  i < n, i  in  a) with hmin | hmin
      * exact <n, le_rfl, hn, hmin>
      * obtain <i, hi> := not_forall.mp hmin
        have hi' : i < n := (Classical.not_imp.mp hi).1
        have hia : i  notin  a := (Classical.not_imp.mp hi).2
        obtain <k, hki, hk, hall> := ih i hi' hia
        exact <k, hki.trans hi'.le, hk, hall>
  obtain <n, hn> := hex
  obtain <k, _, hk, hall> := this n hn
  exact <k, hk, hall>

end DomainTheory.DataTypesAsLattices
```

4

exists_least_not_mem.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

lemma exists_least_not_mem {a : Pomega} (hex : exists  k, k  notin  a) :
    exists  k, k  notin  a  /\  forall  i < k, i  in  a := by
  have : forall  n, n  notin  a  ->  exists  k  <=  n, k  notin  a  /\  forall  i < k,
     i  in  a := by
    intro n
```

```
    induction n using Nat.strong_induction_on with
    | h n ih =>
      intro hn
      rcases Classical.em (forall  i < n, i  in  a) with hmin | hmin
      * exact <n, le_rfl, hn, hmin>
      * obtain <i, hi> := not_forall.mp hmin
        have hi' : i < n := (Classical.not_imp.mp hi).1
        have hia : i  notin  a := (Classical.not_imp.mp hi).2
        obtain <k, hki, hk, hall> := ih i hi' hia
        exact <k, hki.trans hi'.le, hk, hall>
  obtain <n, hn> := hex
  obtain <k, _, hk, hall> := this n hn
  exact <k, hk, hall>

end DomainTheory.DataTypesAsLattices
```

5

exists_least_not_mem.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

lemma exists_least_not_mem {a : Pomega} (hex : exists  k, k  notin  a) :
    exists  k, k  notin  a  /\  forall  i < k, i  in  a := by
  have : forall  n, n  notin  a  ->  exists  k  <=  n, k  notin  a  /\  forall  i < k,
     i  in  a := by
    intro n
    induction n using Nat.strong_induction_on with
    | h n ih =>
      intro hn
      rcases Classical.em (forall  i < n, i  in  a) with hmin | hmin
      * exact <n, le_rfl, hn, hmin>
      * obtain <i, hi> := not_forall.mp hmin
        have hi' : i < n := (Classical.not_imp.mp hi).1
        have hia : i  notin  a := (Classical.not_imp.mp hi).2
        obtain <k, hki, hk, hall> := ih i hi' hia
        exact <k, hki.trans hi'.le, hk, hall>
  obtain <n, hn> := hex
  obtain <k, _, hk, hall> := this n hn
  exact <k, hk, hall>

end DomainTheory.DataTypesAsLattices
```

6

exists_least_not_mem.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

lemma exists_least_not_mem {a : Pomega} (hex : exists  k, k  notin  a) :
    exists  k, k  notin  a  /\  forall  i < k, i  in  a := by
  have : forall  n, n  notin  a  ->  exists  k  <=  n, k  notin  a  /\  forall  i < k,
     i  in  a := by
    intro n
    induction n using Nat.strong_induction_on with
    | h n ih =>
      intro hn
      rcases Classical.em (forall  i < n, i  in  a) with hmin | hmin
      * exact <n, le_rfl, hn, hmin>
      * obtain <i, hi> := not_forall.mp hmin
        have hi' : i < n := (Classical.not_imp.mp hi).1
        have hia : i  notin  a := (Classical.not_imp.mp hi).2
        obtain <k, hki, hk, hall> := ih i hi' hia
        exact <k, hki.trans hi'.le, hk, hall>
  obtain <n, hn> := hex
  obtain <k, _, hk, hall> := this n hn
  exact <k, hk, hall>

end DomainTheory.DataTypesAsLattices
```

7

exists_least_not_mem.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

lemma exists_least_not_mem {a : Pomega} (hex : exists  k, k  notin  a) :
    exists  k, k  notin  a  /\  forall  i < k, i  in  a := by
  have : forall  n, n  notin  a  ->  exists  k  <=  n, k  notin  a  /\  forall  i < k,
     i  in  a := by
    intro n
    induction n using Nat.strong_induction_on with
    | h n ih =>
      intro hn
      rcases Classical.em (forall  i < n, i  in  a) with hmin | hmin
      * exact <n, le_rfl, hn, hmin>
      * obtain <i, hi> := not_forall.mp hmin
        have hi' : i < n := (Classical.not_imp.mp hi).1
        have hia : i  notin  a := (Classical.not_imp.mp hi).2
        obtain <k, hki, hk, hall> := ih i hi' hia
        exact <k, hki.trans hi'.le, hk, hall>
  obtain <n, hn> := hex
  obtain <k, _, hk, hall> := this n hn
  exact <k, hk, hall>
```

```
end DomainTheory.DataTypesAsLattices
```

8

exists_least_not_mem.lean

```
module

public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

lemma exists_least_not_mem {a : Pomega} (hex : exists  k, k  notin  a) :
    exists  k, k  notin  a  /\  forall  i < k, i  in  a := by
  have : forall  n, n  notin  a  ->  exists  k  <=  n, k  notin  a  /\  forall  i < k,
     i  in  a := by
    intro n
    induction n using Nat.strong_induction_on with
    | h n ih =>
      intro hn
      rcases Classical.em (forall  i < n, i  in  a) with hmin | hmin
      * exact <n, le_rfl, hn, hmin>
      * obtain <i, hi> := not_forall.mp hmin
        have hi' : i < n := (Classical.not_imp.mp hi).1
        have hia : i  notin  a := (Classical.not_imp.mp hi).2
        obtain <k, hki, hk, hall> := ih i hi' hia
        exact <k, hki.trans hi'.le, hk, hall>
  obtain <n, hn> := hex
  obtain <k, _, hk, hall> := this n hn
  exact <k, hk, hall>

end DomainTheory.DataTypesAsLattices
```

9

Scomb_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace Scomb

theorem inner_cont (u v : Pomega) :
    IsScottContinuous (fun x => funOf (funOf u x) (funOf v x)) :=
  theorem_1_3.tuple
    (fun y => funOf.isScottContinuous_left y)
    (fun t => funOf.isScottContinuous t)
    (funOf.isScottContinuous u)
    (funOf.isScottContinuous v)

theorem map_cont :
    IsScottContinuous (fun u => graph (fun v =>
      graph (fun x => funOf (funOf u x) (funOf v x)))) :=
  graph_const_isScottContinuous
    (fun u v => graph (fun x => funOf (funOf u x) (funOf v x)))
    (fun v => graph_const_isScottContinuous
      (fun u x => funOf (funOf u x) (funOf v x))
      (fun x => theorem_1_3
        (f := fun t => funOf t (funOf v x))
        (g := fun u => funOf u x)
        (funOf.isScottContinuous_left (funOf v x))
        (funOf.isScottContinuous_left x)))

theorem mid_cont (u : Pomega) :
    IsScottContinuous (fun v =>
      graph (fun x => funOf (funOf u x) (funOf v x))) :=
  graph_const_isScottContinuous
    (fun v x => funOf (funOf u x) (funOf v x))
    (fun x => theorem_1_3 (funOf.isScottContinuous (funOf u x))
      (funOf.isScottContinuous_left x))

theorem beta (f : Pomega) :
    funOf Scomb f =
      graph (fun v => graph (fun x => funOf (funOf f x) (funOf v x))) :=
  _root_.DomainTheory.DataTypesAsLattices.beta Scomb.map_cont f

theorem beta2 (f g : Pomega) :
    funOf (funOf Scomb f) g =
      graph (fun x => funOf (funOf f x) (funOf g x)) := by
  rw [Scomb.beta]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (Scomb.mid_cont f) g

theorem beta3 (f g x : Pomega) :
    funOf (funOf (funOf Scomb f) g) x = funOf (funOf f x) (funOf g x) := by
  rw [Scomb.beta2]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (Scomb.inner_cont f g) x

end Scomb

end DomainTheory.DataTypesAsLattices
```

0

Scomb_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace Scomb

theorem inner_cont (u v : Pomega) :
    IsScottContinuous (fun x => funOf (funOf u x) (funOf v x)) :=
  theorem_1_3.tuple
    (fun y => funOf.isScottContinuous_left y)
    (fun t => funOf.isScottContinuous t)
    (funOf.isScottContinuous u)
    (funOf.isScottContinuous v)

theorem map_cont :
    IsScottContinuous (fun u => graph (fun v =>
      graph (fun x => funOf (funOf u x) (funOf v x)))) :=
  graph_const_isScottContinuous
    (fun u v => graph (fun x => funOf (funOf u x) (funOf v x)))
    (fun v => graph_const_isScottContinuous
      (fun u x => funOf (funOf u x) (funOf v x))
      (fun x => theorem_1_3
        (f := fun t => funOf t (funOf v x))
        (g := fun u => funOf u x)
        (funOf.isScottContinuous_left (funOf v x))
        (funOf.isScottContinuous_left x)))

theorem mid_cont (u : Pomega) :
    IsScottContinuous (fun v =>
      graph (fun x => funOf (funOf u x) (funOf v x))) :=
  graph_const_isScottContinuous
    (fun v x => funOf (funOf u x) (funOf v x))
    (fun x => theorem_1_3 (funOf.isScottContinuous (funOf u x))
      (funOf.isScottContinuous_left x))

theorem beta (f : Pomega) :
    funOf Scomb f =
      graph (fun v => graph (fun x => funOf (funOf f x) (funOf v x))) :=
```

```
  _root_.DomainTheory.DataTypesAsLattices.beta Scomb.map_cont f

theorem beta2 (f g : Pomega) :
    funOf (funOf Scomb f) g =
      graph (fun x => funOf (funOf f x) (funOf g x)) := by
  rw [Scomb.beta]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (Scomb.mid_cont f) g

theorem beta3 (f g x : Pomega) :
    funOf (funOf (funOf Scomb f) g) x = funOf (funOf f x) (funOf g x) := by
  rw [Scomb.beta2]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (Scomb.inner_cont f g) x

end Scomb

end DomainTheory.DataTypesAsLattices
```

1

Scomb_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace Scomb

theorem inner_cont (u v : Pomega) :
    IsScottContinuous (fun x => funOf (funOf u x) (funOf v x)) :=
  theorem_1_3.tuple
    (fun y => funOf.isScottContinuous_left y)
    (fun t => funOf.isScottContinuous t)
    (funOf.isScottContinuous u)
    (funOf.isScottContinuous v)

theorem map_cont :
    IsScottContinuous (fun u => graph (fun v =>
      graph (fun x => funOf (funOf u x) (funOf v x)))) :=
  graph_const_isScottContinuous
    (fun u v => graph (fun x => funOf (funOf u x) (funOf v x)))
```

```
    (fun v => graph_const_isScottContinuous
      (fun u x => funOf (funOf u x) (funOf v x))
      (fun x => theorem_1_3
        (f := fun t => funOf t (funOf v x))
        (g := fun u => funOf u x)
        (funOf.isScottContinuous_left (funOf v x))
        (funOf.isScottContinuous_left x)))

theorem mid_cont (u : Pomega) :
    IsScottContinuous (fun v =>
      graph (fun x => funOf (funOf u x) (funOf v x))) :=
  graph_const_isScottContinuous
    (fun v x => funOf (funOf u x) (funOf v x))
    (fun x => theorem_1_3 (funOf.isScottContinuous (funOf u x))
      (funOf.isScottContinuous_left x))

theorem beta (f : Pomega) :
    funOf Scomb f =
      graph (fun v => graph (fun x => funOf (funOf f x) (funOf v x))) :=
  _root_.DomainTheory.DataTypesAsLattices.beta Scomb.map_cont f

theorem beta2 (f g : Pomega) :
    funOf (funOf Scomb f) g =
      graph (fun x => funOf (funOf f x) (funOf g x)) := by
  rw [Scomb.beta]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (Scomb.mid_cont f) g

theorem beta3 (f g x : Pomega) :
    funOf (funOf (funOf Scomb f) g) x = funOf (funOf f x) (funOf g x) := by
  rw [Scomb.beta2]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (Scomb.inner_cont f g) x

end Scomb

end DomainTheory.DataTypesAsLattices
```

2

Scomb_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace Scomb

theorem inner_cont (u v : Pomega) :
    IsScottContinuous (fun x => funOf (funOf u x) (funOf v x)) :=
  theorem_1_3.tuple
    (fun y => funOf.isScottContinuous_left y)
    (fun t => funOf.isScottContinuous t)
    (funOf.isScottContinuous u)
    (funOf.isScottContinuous v)

theorem map_cont :
    IsScottContinuous (fun u => graph (fun v =>
      graph (fun x => funOf (funOf u x) (funOf v x)))) :=
  graph_const_isScottContinuous
    (fun u v => graph (fun x => funOf (funOf u x) (funOf v x)))
    (fun v => graph_const_isScottContinuous
      (fun u x => funOf (funOf u x) (funOf v x))
      (fun x => theorem_1_3
        (f := fun t => funOf t (funOf v x))
        (g := fun u => funOf u x)
        (funOf.isScottContinuous_left (funOf v x))
        (funOf.isScottContinuous_left x)))

theorem mid_cont (u : Pomega) :
    IsScottContinuous (fun v =>
      graph (fun x => funOf (funOf u x) (funOf v x))) :=
  graph_const_isScottContinuous
    (fun v x => funOf (funOf u x) (funOf v x))
    (fun x => theorem_1_3 (funOf.isScottContinuous (funOf u x))
      (funOf.isScottContinuous_left x))

theorem beta (f : Pomega) :
    funOf Scomb f =
      graph (fun v => graph (fun x => funOf (funOf f x) (funOf v x))) :=
  _root_.DomainTheory.DataTypesAsLattices.beta Scomb.map_cont f

theorem beta2 (f g : Pomega) :
    funOf (funOf Scomb f) g =
      graph (fun x => funOf (funOf f x) (funOf g x)) := by
  rw [Scomb.beta]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (Scomb.mid_cont f) g

theorem beta3 (f g x : Pomega) :
    funOf (funOf (funOf Scomb f) g) x = funOf (funOf f x) (funOf g x) := by
  rw [Scomb.beta2]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (Scomb.inner_cont f g) x

end Scomb

end DomainTheory.DataTypesAsLattices
```

Scomb_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace Scomb

theorem inner_cont (u v : Pomega) :
    IsScottContinuous (fun x => funOf (funOf u x) (funOf v x)) :=
  theorem_1_3.tuple
    (fun y => funOf.isScottContinuous_left y)
    (fun t => funOf.isScottContinuous t)
    (funOf.isScottContinuous u)
    (funOf.isScottContinuous v)

theorem map_cont :
    IsScottContinuous (fun u => graph (fun v =>
      graph (fun x => funOf (funOf u x) (funOf v x)))) :=
  graph_const_isScottContinuous
    (fun u v => graph (fun x => funOf (funOf u x) (funOf v x)))
    (fun v => graph_const_isScottContinuous
      (fun u x => funOf (funOf u x) (funOf v x))
      (fun x => theorem_1_3
        (f := fun t => funOf t (funOf v x))
        (g := fun u => funOf u x)
        (funOf.isScottContinuous_left (funOf v x))
        (funOf.isScottContinuous_left x)))

theorem mid_cont (u : Pomega) :
    IsScottContinuous (fun v =>
      graph (fun x => funOf (funOf u x) (funOf v x))) :=
  graph_const_isScottContinuous
    (fun v x => funOf (funOf u x) (funOf v x))
    (fun x => theorem_1_3 (funOf.isScottContinuous (funOf u x))
      (funOf.isScottContinuous_left x))

theorem beta (f : Pomega) :
    funOf Scomb f =
      graph (fun v => graph (fun x => funOf (funOf f x) (funOf v x))) :=
```

```
  _root_.DomainTheory.DataTypesAsLattices.beta Scomb.map_cont f

theorem beta2 (f g : Pomega) :
    funOf (funOf Scomb f) g =
      graph (fun x => funOf (funOf f x) (funOf g x)) := by
  rw [Scomb.beta]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (Scomb.mid_cont f) g

theorem beta3 (f g x : Pomega) :
    funOf (funOf (funOf Scomb f) g) x = funOf (funOf f x) (funOf g x) := by
  rw [Scomb.beta2]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (Scomb.inner_cont f g) x

end Scomb

end DomainTheory.DataTypesAsLattices
```

4

Scomb_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace Scomb

theorem inner_cont (u v : Pomega) :
    IsScottContinuous (fun x => funOf (funOf u x) (funOf v x)) :=
  theorem_1_3.tuple
    (fun y => funOf.isScottContinuous_left y)
    (fun t => funOf.isScottContinuous t)
    (funOf.isScottContinuous u)
    (funOf.isScottContinuous v)

theorem map_cont :
    IsScottContinuous (fun u => graph (fun v =>
      graph (fun x => funOf (funOf u x) (funOf v x)))) :=
  graph_const_isScottContinuous
    (fun u v => graph (fun x => funOf (funOf u x) (funOf v x)))
```

```
    (fun v => graph_const_isScottContinuous
      (fun u x => funOf (funOf u x) (funOf v x))
      (fun x => theorem_1_3
        (f := fun t => funOf t (funOf v x))
        (g := fun u => funOf u x)
        (funOf.isScottContinuous_left (funOf v x))
        (funOf.isScottContinuous_left x)))

theorem mid_cont (u : Pomega) :
    IsScottContinuous (fun v =>
      graph (fun x => funOf (funOf u x) (funOf v x))) :=
  graph_const_isScottContinuous
    (fun v x => funOf (funOf u x) (funOf v x))
    (fun x => theorem_1_3 (funOf.isScottContinuous (funOf u x))
      (funOf.isScottContinuous_left x))

theorem beta (f : Pomega) :
    funOf Scomb f =
      graph (fun v => graph (fun x => funOf (funOf f x) (funOf v x))) :=
  _root_.DomainTheory.DataTypesAsLattices.beta Scomb.map_cont f

theorem beta2 (f g : Pomega) :
    funOf (funOf Scomb f) g =
      graph (fun x => funOf (funOf f x) (funOf g x)) := by
  rw [Scomb.beta]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (Scomb.mid_cont f) g

theorem beta3 (f g x : Pomega) :
    funOf (funOf (funOf Scomb f) g) x = funOf (funOf f x) (funOf g x) := by
  rw [Scomb.beta2]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (Scomb.inner_cont f g) x

end Scomb

end DomainTheory.DataTypesAsLattices
```

5

Scomb_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace Scomb

theorem inner_cont (u v : Pomega) :
    IsScottContinuous (fun x => funOf (funOf u x) (funOf v x)) :=
  theorem_1_3.tuple
    (fun y => funOf.isScottContinuous_left y)
    (fun t => funOf.isScottContinuous t)
    (funOf.isScottContinuous u)
    (funOf.isScottContinuous v)

theorem map_cont :
    IsScottContinuous (fun u => graph (fun v =>
      graph (fun x => funOf (funOf u x) (funOf v x)))) :=
  graph_const_isScottContinuous
    (fun u v => graph (fun x => funOf (funOf u x) (funOf v x)))
    (fun v => graph_const_isScottContinuous
      (fun u x => funOf (funOf u x) (funOf v x))
      (fun x => theorem_1_3
        (f := fun t => funOf t (funOf v x))
        (g := fun u => funOf u x)
        (funOf.isScottContinuous_left (funOf v x))
        (funOf.isScottContinuous_left x)))

theorem mid_cont (u : Pomega) :
    IsScottContinuous (fun v =>
      graph (fun x => funOf (funOf u x) (funOf v x))) :=
  graph_const_isScottContinuous
    (fun v x => funOf (funOf u x) (funOf v x))
    (fun x => theorem_1_3 (funOf.isScottContinuous (funOf u x))
      (funOf.isScottContinuous_left x))

theorem beta (f : Pomega) :
    funOf Scomb f =
      graph (fun v => graph (fun x => funOf (funOf f x) (funOf v x))) :=
  _root_.DomainTheory.DataTypesAsLattices.beta Scomb.map_cont f

theorem beta2 (f g : Pomega) :
    funOf (funOf Scomb f) g =
      graph (fun x => funOf (funOf f x) (funOf g x)) := by
  rw [Scomb.beta]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (Scomb.mid_cont f) g

theorem beta3 (f g x : Pomega) :
    funOf (funOf (funOf Scomb f) g) x = funOf (funOf f x) (funOf g x) := by
  rw [Scomb.beta2]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (Scomb.inner_cont f g) x

end Scomb

end DomainTheory.DataTypesAsLattices
```

Scomb_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace Scomb

theorem inner_cont (u v : Pomega) :
    IsScottContinuous (fun x => funOf (funOf u x) (funOf v x)) :=
  theorem_1_3.tuple
    (fun y => funOf.isScottContinuous_left y)
    (fun t => funOf.isScottContinuous t)
    (funOf.isScottContinuous u)
    (funOf.isScottContinuous v)

theorem map_cont :
    IsScottContinuous (fun u => graph (fun v =>
      graph (fun x => funOf (funOf u x) (funOf v x)))) :=
  graph_const_isScottContinuous
    (fun u v => graph (fun x => funOf (funOf u x) (funOf v x)))
    (fun v => graph_const_isScottContinuous
      (fun u x => funOf (funOf u x) (funOf v x))
      (fun x => theorem_1_3
        (f := fun t => funOf t (funOf v x))
        (g := fun u => funOf u x)
        (funOf.isScottContinuous_left (funOf v x))
        (funOf.isScottContinuous_left x)))

theorem mid_cont (u : Pomega) :
    IsScottContinuous (fun v =>
      graph (fun x => funOf (funOf u x) (funOf v x))) :=
  graph_const_isScottContinuous
    (fun v x => funOf (funOf u x) (funOf v x))
    (fun x => theorem_1_3 (funOf.isScottContinuous (funOf u x))
      (funOf.isScottContinuous_left x))

theorem beta (f : Pomega) :
    funOf Scomb f =
      graph (fun v => graph (fun x => funOf (funOf f x) (funOf v x))) :=
```

```
  _root_.DomainTheory.DataTypesAsLattices.beta Scomb.map_cont f

theorem beta2 (f g : Pomega) :
    funOf (funOf Scomb f) g =
      graph (fun x => funOf (funOf f x) (funOf g x)) := by
  rw [Scomb.beta]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (Scomb.mid_cont f) g

theorem beta3 (f g x : Pomega) :
    funOf (funOf (funOf Scomb f) g) x = funOf (funOf f x) (funOf g x) := by
  rw [Scomb.beta2]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (Scomb.inner_cont f g) x

end Scomb

end DomainTheory.DataTypesAsLattices
```

7

Scomb_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace Scomb

theorem inner_cont (u v : Pomega) :
    IsScottContinuous (fun x => funOf (funOf u x) (funOf v x)) :=
  theorem_1_3.tuple
    (fun y => funOf.isScottContinuous_left y)
    (fun t => funOf.isScottContinuous t)
    (funOf.isScottContinuous u)
    (funOf.isScottContinuous v)

theorem map_cont :
    IsScottContinuous (fun u => graph (fun v =>
      graph (fun x => funOf (funOf u x) (funOf v x)))) :=
  graph_const_isScottContinuous
    (fun u v => graph (fun x => funOf (funOf u x) (funOf v x)))
```

```
    (fun v => graph_const_isScottContinuous
      (fun u x => funOf (funOf u x) (funOf v x))
      (fun x => theorem_1_3
        (f := fun t => funOf t (funOf v x))
        (g := fun u => funOf u x)
        (funOf.isScottContinuous_left (funOf v x))
        (funOf.isScottContinuous_left x)))

theorem mid_cont (u : Pomega) :
    IsScottContinuous (fun v =>
      graph (fun x => funOf (funOf u x) (funOf v x))) :=
  graph_const_isScottContinuous
    (fun v x => funOf (funOf u x) (funOf v x))
    (fun x => theorem_1_3 (funOf.isScottContinuous (funOf u x))
      (funOf.isScottContinuous_left x))

theorem beta (f : Pomega) :
    funOf Scomb f =
      graph (fun v => graph (fun x => funOf (funOf f x) (funOf v x))) :=
  _root_.DomainTheory.DataTypesAsLattices.beta Scomb.map_cont f

theorem beta2 (f g : Pomega) :
    funOf (funOf Scomb f) g =
      graph (fun x => funOf (funOf f x) (funOf g x)) := by
  rw [Scomb.beta]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (Scomb.mid_cont f) g

theorem beta3 (f g x : Pomega) :
    funOf (funOf (funOf Scomb f) g) x = funOf (funOf f x) (funOf g x) := by
  rw [Scomb.beta2]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (Scomb.inner_cont f g) x

end Scomb

end DomainTheory.DataTypesAsLattices
```

8

Scomb_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.Scomb
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.FixedPoint.theorem_1_3
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
```

```
@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace Scomb

theorem inner_cont (u v : Pomega) :
    IsScottContinuous (fun x => funOf (funOf u x) (funOf v x)) :=
  theorem_1_3.tuple
    (fun y => funOf.isScottContinuous_left y)
    (fun t => funOf.isScottContinuous t)
    (funOf.isScottContinuous u)
    (funOf.isScottContinuous v)

theorem map_cont :
    IsScottContinuous (fun u => graph (fun v =>
      graph (fun x => funOf (funOf u x) (funOf v x)))) :=
  graph_const_isScottContinuous
    (fun u v => graph (fun x => funOf (funOf u x) (funOf v x)))
    (fun v => graph_const_isScottContinuous
      (fun u x => funOf (funOf u x) (funOf v x))
      (fun x => theorem_1_3
        (f := fun t => funOf t (funOf v x))
        (g := fun u => funOf u x)
        (funOf.isScottContinuous_left (funOf v x))
        (funOf.isScottContinuous_left x)))

theorem mid_cont (u : Pomega) :
    IsScottContinuous (fun v =>
      graph (fun x => funOf (funOf u x) (funOf v x))) :=
  graph_const_isScottContinuous
    (fun v x => funOf (funOf u x) (funOf v x))
    (fun x => theorem_1_3 (funOf.isScottContinuous (funOf u x))
      (funOf.isScottContinuous_left x))

theorem beta (f : Pomega) :
    funOf Scomb f =
      graph (fun v => graph (fun x => funOf (funOf f x) (funOf v x))) :=
  _root_.DomainTheory.DataTypesAsLattices.beta Scomb.map_cont f

theorem beta2 (f g : Pomega) :
    funOf (funOf Scomb f) g =
      graph (fun x => funOf (funOf f x) (funOf g x)) := by
  rw [Scomb.beta]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (Scomb.mid_cont f) g

theorem beta3 (f g x : Pomega) :
    funOf (funOf (funOf Scomb f) g) x = funOf (funOf f x) (funOf g x) := by
  rw [Scomb.beta2]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (Scomb.inner_cont f g) x

end Scomb

end DomainTheory.DataTypesAsLattices
```

graph_const_isScottContinuous.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Continuous.scottUnion
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- `graph (fun y => g x y)` is continuous in `x` when `g` is continuous in `x`. -/
theorem graph_const_isScottContinuous
    (g : Pomega  ->  Pomega  ->  Pomega)
    (hg : forall  y, IsScottContinuous (fun x => g x y)) :
    IsScottContinuous (fun x => graph (fun y => g x y)) := by
  constructor
  intro x
  ext k
  constructor
  * intro hk
    rcases hk with <n, m, rfl, hm>
    have : m  in  scottUnion (fun z => g z (e n)) x := by
      rw [ <-  (hg (e n)).out x]; exact hm
    obtain <p, hp, hmp> := mem_scottUnion.mp this
    exact mem_scottUnion.mpr <p, hp, n, m, rfl, hmp>
  * intro hk
    obtain <p, hp, n, m, rfl, hm> := mem_scottUnion.mp hk
    exact <n, m, rfl, IsScottContinuous.monotone (hg (e n)) hp hm>

end DomainTheory.DataTypesAsLattices
```

0

graph_const_isScottContinuous.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Continuous.scottUnion
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- `graph (fun y => g x y)` is continuous in `x` when `g` is continuous in `x`. -/
theorem graph_const_isScottContinuous
    (g : Pomega  ->  Pomega  ->  Pomega)
    (hg : forall  y, IsScottContinuous (fun x => g x y)) :
    IsScottContinuous (fun x => graph (fun y => g x y)) := by
```

```
  constructor
  intro x
  ext k
  constructor
  * intro hk
    rcases hk with <n, m, rfl, hm>
    have : m  in  scottUnion (fun z => g z (e n)) x := by
      rw [ <-  (hg (e n)).out x]; exact hm
    obtain <p, hp, hmp> := mem_scottUnion.mp this
    exact mem_scottUnion.mpr <p, hp, n, m, rfl, hmp>
  * intro hk
    obtain <p, hp, n, m, rfl, hm> := mem_scottUnion.mp hk
    exact <n, m, rfl, IsScottContinuous.monotone (hg (e n)) hp hm>

end DomainTheory.DataTypesAsLattices
```

1

graph_const_isScottContinuous.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Continuous.scottUnion
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- `graph (fun y => g x y)` is continuous in `x` when `g` is continuous in `x`. -/
theorem graph_const_isScottContinuous
    (g : Pomega  ->  Pomega  ->  Pomega)
    (hg : forall  y, IsScottContinuous (fun x => g x y)) :
    IsScottContinuous (fun x => graph (fun y => g x y)) := by
  constructor
  intro x
  ext k
  constructor
  * intro hk
    rcases hk with <n, m, rfl, hm>
    have : m  in  scottUnion (fun z => g z (e n)) x := by
      rw [ <-  (hg (e n)).out x]; exact hm
    obtain <p, hp, hmp> := mem_scottUnion.mp this
    exact mem_scottUnion.mpr <p, hp, n, m, rfl, hmp>
  * intro hk
    obtain <p, hp, n, m, rfl, hm> := mem_scottUnion.mp hk
    exact <n, m, rfl, IsScottContinuous.monotone (hg (e n)) hp hm>

end DomainTheory.DataTypesAsLattices
```

2

graph_const_isScottContinuous.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Continuous.scottUnion
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- `graph (fun y => g x y)` is continuous in `x` when `g` is continuous in `x`. -/
theorem graph_const_isScottContinuous
    (g : Pomega  ->  Pomega  ->  Pomega)
    (hg : forall  y, IsScottContinuous (fun x => g x y)) :
    IsScottContinuous (fun x => graph (fun y => g x y)) := by
  constructor
  intro x
  ext k
  constructor
  * intro hk
    rcases hk with <n, m, rfl, hm>
    have : m  in  scottUnion (fun z => g z (e n)) x := by
      rw [ <-  (hg (e n)).out x]; exact hm
    obtain <p, hp, hmp> := mem_scottUnion.mp this
    exact mem_scottUnion.mpr <p, hp, n, m, rfl, hmp>
  * intro hk
    obtain <p, hp, n, m, rfl, hm> := mem_scottUnion.mp hk
    exact <n, m, rfl, IsScottContinuous.monotone (hg (e n)) hp hm>

end DomainTheory.DataTypesAsLattices
```

3

graph_const_isScottContinuous.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Continuous.scottUnion
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- `graph (fun y => g x y)` is continuous in `x` when `g` is continuous in `x`. -/
theorem graph_const_isScottContinuous
    (g : Pomega  ->  Pomega  ->  Pomega)
    (hg : forall  y, IsScottContinuous (fun x => g x y)) :
    IsScottContinuous (fun x => graph (fun y => g x y)) := by
  constructor
  intro x
```

```
  ext k
  constructor
  * intro hk
    rcases hk with <n, m, rfl, hm>
    have : m  in  scottUnion (fun z => g z (e n)) x := by
      rw [ <-  (hg (e n)).out x]; exact hm
    obtain <p, hp, hmp> := mem_scottUnion.mp this
    exact mem_scottUnion.mpr <p, hp, n, m, rfl, hmp>
  * intro hk
    obtain <p, hp, n, m, rfl, hm> := mem_scottUnion.mp hk
    exact <n, m, rfl, IsScottContinuous.monotone (hg (e n)) hp hm>

end DomainTheory.DataTypesAsLattices
```

4

graph_const_isScottContinuous.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Continuous.scottUnion
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- `graph (fun y => g x y)` is continuous in `x` when `g` is continuous in `x`. -/
theorem graph_const_isScottContinuous
    (g : Pomega  ->  Pomega  ->  Pomega)
    (hg : forall  y, IsScottContinuous (fun x => g x y)) :
    IsScottContinuous (fun x => graph (fun y => g x y)) := by
  constructor
  intro x
  ext k
  constructor
  * intro hk
    rcases hk with <n, m, rfl, hm>
    have : m  in  scottUnion (fun z => g z (e n)) x := by
      rw [ <-  (hg (e n)).out x]; exact hm
    obtain <p, hp, hmp> := mem_scottUnion.mp this
    exact mem_scottUnion.mpr <p, hp, n, m, rfl, hmp>
  * intro hk
    obtain <p, hp, n, m, rfl, hm> := mem_scottUnion.mp hk
    exact <n, m, rfl, IsScottContinuous.monotone (hg (e n)) hp hm>

end DomainTheory.DataTypesAsLattices
```

5

graph_const_isScottContinuous.lean

```
module
```

```
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Continuous.scottUnion
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- `graph (fun y => g x y)` is continuous in `x` when `g` is continuous in `x`. -/
theorem graph_const_isScottContinuous
    (g : Pomega  ->  Pomega  ->  Pomega)
    (hg : forall  y, IsScottContinuous (fun x => g x y)) :
    IsScottContinuous (fun x => graph (fun y => g x y)) := by
  constructor
  intro x
  ext k
  constructor
  * intro hk
    rcases hk with <n, m, rfl, hm>
    have : m  in  scottUnion (fun z => g z (e n)) x := by
      rw [ <-  (hg (e n)).out x]; exact hm
    obtain <p, hp, hmp> := mem_scottUnion.mp this
    exact mem_scottUnion.mpr <p, hp, n, m, rfl, hmp>
  * intro hk
    obtain <p, hp, n, m, rfl, hm> := mem_scottUnion.mp hk
    exact <n, m, rfl, IsScottContinuous.monotone (hg (e n)) hp hm>

end DomainTheory.DataTypesAsLattices
```

6

graph_const_isScottContinuous.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Continuous.scottUnion
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- `graph (fun y => g x y)` is continuous in `x` when `g` is continuous in `x`. -/
theorem graph_const_isScottContinuous
    (g : Pomega  ->  Pomega  ->  Pomega)
    (hg : forall  y, IsScottContinuous (fun x => g x y)) :
    IsScottContinuous (fun x => graph (fun y => g x y)) := by
  constructor
  intro x
  ext k
```

```
  constructor
  * intro hk
    rcases hk with <n, m, rfl, hm>
    have : m  in  scottUnion (fun z => g z (e n)) x := by
      rw [ <-  (hg (e n)).out x]; exact hm
    obtain <p, hp, hmp> := mem_scottUnion.mp this
    exact mem_scottUnion.mpr <p, hp, n, m, rfl, hmp>
  * intro hk
    obtain <p, hp, n, m, rfl, hm> := mem_scottUnion.mp hk
    exact <n, m, rfl, IsScottContinuous.monotone (hg (e n)) hp hm>

end DomainTheory.DataTypesAsLattices
```

7

graph_const_isScottContinuous.lean

```
module

public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Continuous.scottUnion
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- `graph (fun y => g x y)` is continuous in `x` when `g` is continuous in `x`. -/
theorem graph_const_isScottContinuous
    (g : Pomega  ->  Pomega  ->  Pomega)
    (hg : forall  y, IsScottContinuous (fun x => g x y)) :
    IsScottContinuous (fun x => graph (fun y => g x y)) := by
  constructor
  intro x
  ext k
  constructor
  * intro hk
    rcases hk with <n, m, rfl, hm>
    have : m  in  scottUnion (fun z => g z (e n)) x := by
      rw [ <-  (hg (e n)).out x]; exact hm
    obtain <p, hp, hmp> := mem_scottUnion.mp this
    exact mem_scottUnion.mpr <p, hp, n, m, rfl, hmp>
  * intro hk
    obtain <p, hp, n, m, rfl, hm> := mem_scottUnion.mp hk
    exact <n, m, rfl, IsScottContinuous.monotone (hg (e n)) hp hm>

end DomainTheory.DataTypesAsLattices
```

8

graph_const_isScottContinuous.lean

```
module
```

```
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Continuous.mem_scottUnion
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous
public import DomainTheory.DataTypesAsLattices.Continuous.scottUnion
public import DomainTheory.DataTypesAsLattices.Pomega.e

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

/-- `graph (fun y => g x y)` is continuous in `x` when `g` is continuous in `x`. -/
theorem graph_const_isScottContinuous
    (g : Pomega  ->  Pomega  ->  Pomega)
    (hg : forall  y, IsScottContinuous (fun x => g x y)) :
    IsScottContinuous (fun x => graph (fun y => g x y)) := by
  constructor
  intro x
  ext k
  constructor
  * intro hk
    rcases hk with <n, m, rfl, hm>
    have : m  in  scottUnion (fun z => g z (e n)) x := by
      rw [ <-  (hg (e n)).out x]; exact hm
    obtain <p, hp, hmp> := mem_scottUnion.mp this
    exact mem_scottUnion.mpr <p, hp, n, m, rfl, hmp>
  * intro hk
    obtain <p, hp, n, m, rfl, hm> := mem_scottUnion.mp hk
    exact <n, m, rfl, IsScottContinuous.monotone (hg (e n)) hp hm>

end DomainTheory.DataTypesAsLattices
```

9

condC_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.condC
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace condC
```

```
theorem map_cont :
    IsScottContinuous (fun x => graph (fun y => graph (fun z => condSet z x y))) :=
  graph_const_isScottContinuous
    (fun x y => graph (fun z => condSet z x y))
    (fun y => graph_const_isScottContinuous (fun x z => condSet z x y)
      (fun z => condSet.isScottContinuous_mid z y))

theorem beta (x : Pomega) :
    funOf condC x = graph (fun y => graph (fun z => condSet z x y)) :=
  _root_.DomainTheory.DataTypesAsLattices.beta condC.map_cont x

theorem beta2 (x y : Pomega) :
    funOf (funOf condC x) y = graph (fun z => condSet z x y) := by
  rw [condC.beta]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (graph_const_isScottContinuous (
    fun y z => condSet z x y)
    (fun z => condSet.isScottContinuous_right z x)) y

theorem beta3 (x y z : Pomega) :
    funOf (funOf (funOf condC x) y) z = condSet z x y := by
  rw [condC.beta2]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (condSet.isScottContinuous_left x
    y) z

end condC

end DomainTheory.DataTypesAsLattices
```

0

condC_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.condC
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace condC

theorem map_cont :
```

```
    IsScottContinuous (fun x => graph (fun y => graph (fun z => condSet z x y))) :=
  graph_const_isScottContinuous
    (fun x y => graph (fun z => condSet z x y))
    (fun y => graph_const_isScottContinuous (fun x z => condSet z x y)
      (fun z => condSet.isScottContinuous_mid z y))

theorem beta (x : Pomega) :
    funOf condC x = graph (fun y => graph (fun z => condSet z x y)) :=
  _root_.DomainTheory.DataTypesAsLattices.beta condC.map_cont x

theorem beta2 (x y : Pomega) :
    funOf (funOf condC x) y = graph (fun z => condSet z x y) := by
  rw [condC.beta]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (graph_const_isScottContinuous (
    fun y z => condSet z x y)
    (fun z => condSet.isScottContinuous_right z x)) y

theorem beta3 (x y z : Pomega) :
    funOf (funOf (funOf condC x) y) z = condSet z x y := by
  rw [condC.beta2]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (condSet.isScottContinuous_left x
    y) z

end condC

end DomainTheory.DataTypesAsLattices
```

1

condC_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.condC
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace condC

theorem map_cont :
    IsScottContinuous (fun x => graph (fun y => graph (fun z => condSet z x y))) :=
  graph_const_isScottContinuous
```

```
    (fun x y => graph (fun z => condSet z x y))
    (fun y => graph_const_isScottContinuous (fun x z => condSet z x y)
      (fun z => condSet.isScottContinuous_mid z y))

theorem beta (x : Pomega) :
    funOf condC x = graph (fun y => graph (fun z => condSet z x y)) :=
  _root_.DomainTheory.DataTypesAsLattices.beta condC.map_cont x

theorem beta2 (x y : Pomega) :
    funOf (funOf condC x) y = graph (fun z => condSet z x y) := by
  rw [condC.beta]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (graph_const_isScottContinuous (
    fun y z => condSet z x y)
    (fun z => condSet.isScottContinuous_right z x)) y

theorem beta3 (x y z : Pomega) :
    funOf (funOf (funOf condC x) y) z = condSet z x y := by
  rw [condC.beta2]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (condSet.isScottContinuous_left x
    y) z

end condC

end DomainTheory.DataTypesAsLattices
```

2

condC_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.condC
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace condC

theorem map_cont :
    IsScottContinuous (fun x => graph (fun y => graph (fun z => condSet z x y))) :=
  graph_const_isScottContinuous
    (fun x y => graph (fun z => condSet z x y))
    (fun y => graph_const_isScottContinuous (fun x z => condSet z x y)
```

```
      (fun z => condSet.isScottContinuous_mid z y))

theorem beta (x : Pomega) :
    funOf condC x = graph (fun y => graph (fun z => condSet z x y)) :=
  _root_.DomainTheory.DataTypesAsLattices.beta condC.map_cont x

theorem beta2 (x y : Pomega) :
    funOf (funOf condC x) y = graph (fun z => condSet z x y) := by
  rw [condC.beta]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (graph_const_isScottContinuous (
    fun y z => condSet z x y)
    (fun z => condSet.isScottContinuous_right z x)) y

theorem beta3 (x y z : Pomega) :
    funOf (funOf (funOf condC x) y) z = condSet z x y := by
  rw [condC.beta2]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (condSet.isScottContinuous_left x
     y) z

end condC

end DomainTheory.DataTypesAsLattices
```

3

condC_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.condC
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace condC

theorem map_cont :
    IsScottContinuous (fun x => graph (fun y => graph (fun z => condSet z x y))) :=
  graph_const_isScottContinuous
    (fun x y => graph (fun z => condSet z x y))
    (fun y => graph_const_isScottContinuous (fun x z => condSet z x y)
      (fun z => condSet.isScottContinuous_mid z y))
```

```
theorem beta (x : Pomega) :
    funOf condC x = graph (fun y => graph (fun z => condSet z x y)) :=
  _root_.DomainTheory.DataTypesAsLattices.beta condC.map_cont x

theorem beta2 (x y : Pomega) :
    funOf (funOf condC x) y = graph (fun z => condSet z x y) := by
  rw [condC.beta]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (graph_const_isScottContinuous (
    fun y z => condSet z x y)
    (fun z => condSet.isScottContinuous_right z x)) y

theorem beta3 (x y z : Pomega) :
    funOf (funOf (funOf condC x) y) z = condSet z x y := by
  rw [condC.beta2]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (condSet.isScottContinuous_left x
    y) z

end condC

end DomainTheory.DataTypesAsLattices
```

4

condC_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.condC
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace condC

theorem map_cont :
    IsScottContinuous (fun x => graph (fun y => graph (fun z => condSet z x y))) :=
  graph_const_isScottContinuous
    (fun x y => graph (fun z => condSet z x y))
    (fun y => graph_const_isScottContinuous (fun x z => condSet z x y)
      (fun z => condSet.isScottContinuous_mid z y))

theorem beta (x : Pomega) :
    funOf condC x = graph (fun y => graph (fun z => condSet z x y)) :=
```

```
  _root_.DomainTheory.DataTypesAsLattices.beta condC.map_cont x

theorem beta2 (x y : Pomega) :
    funOf (funOf condC x) y = graph (fun z => condSet z x y) := by
  rw [condC.beta]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (graph_const_isScottContinuous (
    fun y z => condSet z x y)
    (fun z => condSet.isScottContinuous_right z x)) y

theorem beta3 (x y z : Pomega) :
    funOf (funOf (funOf condC x) y) z = condSet z x y := by
  rw [condC.beta2]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (condSet.isScottContinuous_left x
     y) z

end condC

end DomainTheory.DataTypesAsLattices
```

5

condC_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.condC
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace condC

theorem map_cont :
    IsScottContinuous (fun x => graph (fun y => graph (fun z => condSet z x y))) :=
  graph_const_isScottContinuous
    (fun x y => graph (fun z => condSet z x y))
    (fun y => graph_const_isScottContinuous (fun x z => condSet z x y)
      (fun z => condSet.isScottContinuous_mid z y))

theorem beta (x : Pomega) :
    funOf condC x = graph (fun y => graph (fun z => condSet z x y)) :=
  _root_.DomainTheory.DataTypesAsLattices.beta condC.map_cont x
```

```
theorem beta2 (x y : Pomega) :
    funOf (funOf condC x) y = graph (fun z => condSet z x y) := by
  rw [condC.beta]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (graph_const_isScottContinuous (
    fun y z => condSet z x y)
    (fun z => condSet.isScottContinuous_right z x)) y

theorem beta3 (x y z : Pomega) :
    funOf (funOf (funOf condC x) y) z = condSet z x y := by
  rw [condC.beta2]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (condSet.isScottContinuous_left x
    y) z

end condC

end DomainTheory.DataTypesAsLattices
```

6

condC_more.lean

```
module

public import DomainTheory.DataTypesAsLattices.Lambda.condC
public import DomainTheory.DataTypesAsLattices.Graph.graph
public import DomainTheory.DataTypesAsLattices.Pomega.e
public import DomainTheory.DataTypesAsLattices.Lambda.condSet
public import DomainTheory.DataTypesAsLattices.Graph.funOf
public import DomainTheory.DataTypesAsLattices.Graph.funOf_more
public import DomainTheory.DataTypesAsLattices.Graph.beta
public import DomainTheory.DataTypesAsLattices.Lambda.graph_const_isScottContinuous
public import Mathlib.Topology.Bases
public import Mathlib.Topology.Defs.Induced
public import Mathlib.Topology.Order
public import Mathlib.Topology.Separation.Basic
public import DomainTheory.DataTypesAsLattices.Continuous.IsScottContinuous

@[expose] public section

namespace DomainTheory.DataTypesAsLattices

namespace condC

theorem map_cont :
    IsScottContinuous (fun x => graph (fun y => graph (fun z => condSet z x y))) :=
  graph_const_isScottContinuous
    (fun x y => graph (fun z => condSet z x y))
    (fun y => graph_const_isScottContinuous (fun x z => condSet z x y)
      (fun z => condSet.isScottContinuous_mid z y))

theorem beta (x : Pomega) :
    funOf condC x = graph (fun y => graph (fun z => condSet z x y)) :=
  _root_.DomainTheory.DataTypesAsLattices.beta condC.map_cont x

theorem beta2 (x y : Pomega) :
    funOf (funOf condC x) y = graph (fun z => condSet z x y) := by
```

```
  rw [condC.beta]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (graph_const_isScottContinuous (
    fun y z => condSet z x y)
     (fun z => condSet.isScottContinuous_right z x)) y

theorem beta3 (x y z : Pomega) :
    funOf (funOf (funOf condC x) y) z = condSet z x y := by
  rw [condC.beta2]
  exact _root_.DomainTheory.DataTypesAsLattices.beta (condSet.isScottContinuous_left x
     y) z

end condC

end DomainTheory.DataTypesAsLattices
```

7

5.6 ContinuousLatticeToNeighborhood

Dependencies. Inside ContinuousLatticeToNeighborhood, each box is one Lean file. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. The arrowhead points at the file or folder that imports, and the number is how many direct imports.

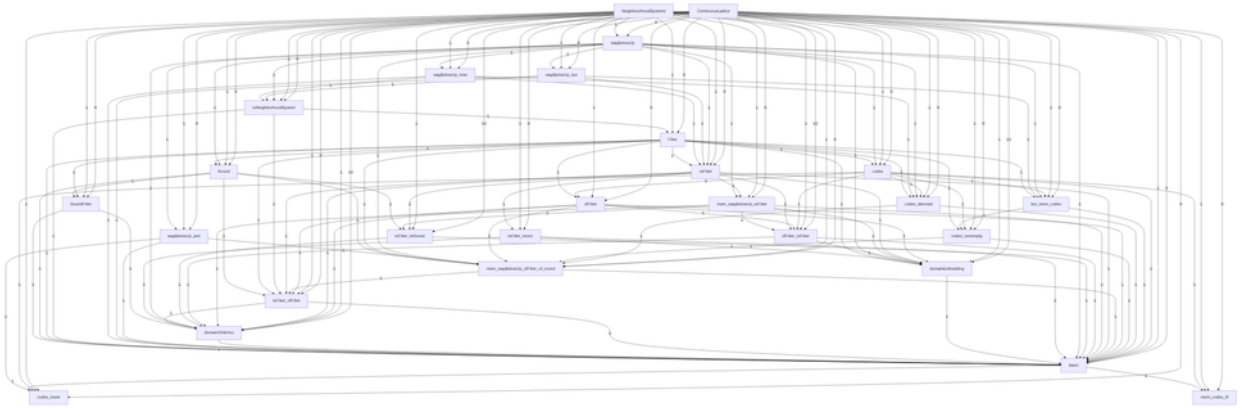


Figure 50: ContinuousLatticeToNeighborhood.

5.7 InfoSysConstructions

Dependencies. Inside InfoSysConstructions, each box is one Lean file. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. The arrowhead points at the file or folder that imports, and the number is how many direct imports.

5.8 InfoSysToIdealCompletion

Dependencies. Inside InfoSysToIdealCompletion, each box is one Lean file. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. The arrowhead points at the file or folder that imports, and the number is how many direct imports.



Figure 51: InfoSysConstructions.

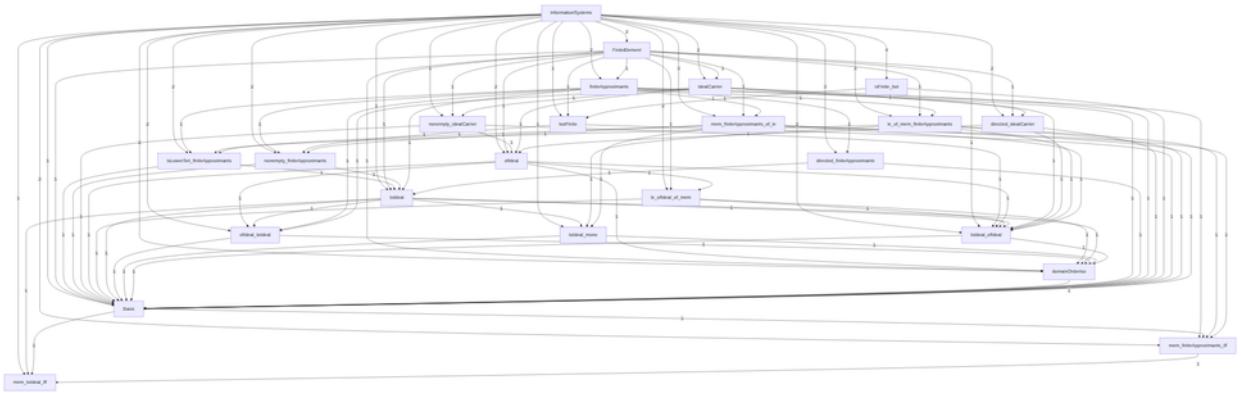


Figure 52: InfoSysToIdealCompletion.

5.9 InfoSysToNeighborhood

Dependencies. Inside InfoSysToNeighborhood, each box is one Lean file. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. BridgeTheorems is the Lean files in that chapter folder, not in a subfolder. The arrowhead points at the file or folder that imports, and the number is how many direct imports.

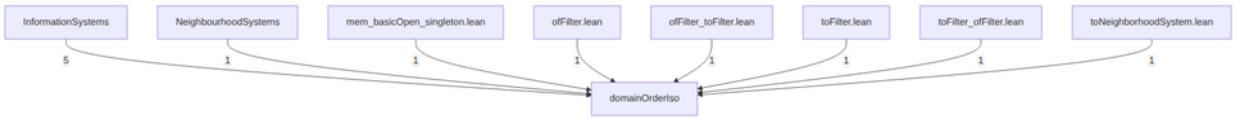


Figure 53: InfoSysToNeighborhood.

5.10 NeighborhoodToInfoSys

Dependencies. Inside NeighborhoodToInfoSys, each box is one Lean file. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. BridgeTheorems is the Lean files in that chapter folder, not in a subfolder. The arrowhead points at the file or folder that imports, and the number is how many direct imports.

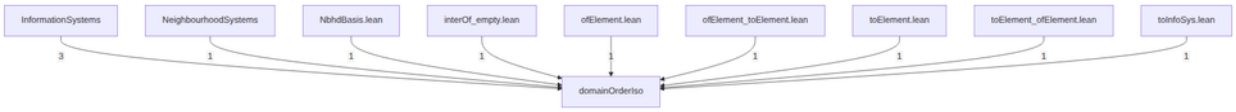


Figure 54: NeighborhoodToInfoSys.

5.11 PresentationDomains

Dependencies. Inside PresentationDomains, each box is one Lean file. A box named as another folder in this chapter is that whole folder. A box named as another chapter is that whole chapter. BridgeTheorems is the Lean files in that chapter folder, not in a subfolder. The arrowhead points at the file or folder that imports, and the number is how many direct imports.

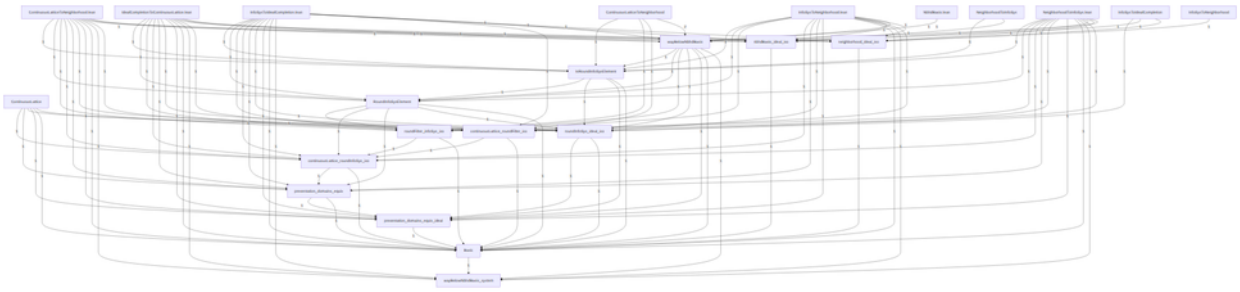


Figure 55: PresentationDomains.

6 Acknowledgments

6.1 AI-assisted development

The human author retains sole responsibility for the mathematical content, the choice of formalization route, and every formal claim in this work. Following standard publisher practice (e.g., COPE guidance on authorship and AI tools [5]), **no large language model is listed as a co-author** — authorship implies an accountability that automated systems cannot bear.

We gratefully acknowledge assistance from the following tools in this revision (auto-generated from `scripts/ai_model_cards.py` when building `arxiv.tex`). Source versions of the four Scott papers and the bridge theorems record their own model cards in their respective writeups under `vendor/`:

- **Cursor Composer 2.5** [6] — routine multi-step work: module scaffolding, dependency-ordered wiring of `DomainTheory/`, documentation and Mermaid blueprints, and medium proof obligations where the strategy was already fixed (`composer-2.5`).
- **Cursor Grok 4.7** [7] — primary agent model for the DomainTheory sessions: domain-theory constructions, inventory design in `arxiv.md`, constructivity audits, and adaptation of portable patterns from prior Scott formalizations. Used here via the Cursor agent environment.
- **Google Gemini 3.8** [8] — exploratory and support passes (typographic conventions, scope decisions, and cross-checks against the narrative inventory).

All definitions, constructivity audits, and final prose were reviewed by the human author, who takes full responsibility for them.

6.2 Artifact availability and reproducibility

The development is at `github.com/catskillsresearch/DomainTheory`. GitHub Actions CI runs `lake build` on `main`.

Locally:

```
lake exe cache get
lake build DomainTheory
```

Sibling packages `scott1972`, `scott1976`, `scott1980`, `scott1982`, and `scott_models` are vendored under `vendor/` (frozen revisions in `vendor/FROZEN.txt`); they are not Lake path dependencies. `arxiv.md` is the durable inventory and proof narrative. Axiom audits: `#print axioms` on the compared names in `comparator.json`.

Build the arXiv PDF / submission zip (Lean Code appendix = GitHub hyperlinks):

```
bash scripts/build_arxiv_tex.sh      # arxiv.md  ->  arxiv.tex + figures/
bash scripts/build_arxiv_pdf.sh      # compile PDF + package dist/arxiv_submit.zip
# or: bash scripts/package_arxiv_submit.sh
```

7 References

References

[1] D. Scott. Continuous lattices, 1972. in F. W. Lawvere (ed.), Toposes, Algebraic Geometry and Logic, Lecture Notes in Mathematics, vol. 274, Springer-Verlag, pp. 97–136. [Also: Tech. Monograph PRG-7, Oxford Univ. Computing Lab., Programming Research Group (1971).].

[2] D. Scott. Lectures on a mathematical theory of computation, 1981. Technical Monograph PRG-19, May 1981, Oxford University Computing Laboratory, Programming Research Group (neighbourhood systems).

[3] D. Scott. Domains for denotational semantics, 1982. ICALP 1982, LNCS 140, Springer, pp. 577–613.

[4] Palomar Registry. entry `PALOMAR-2026-08-25-000003`, version 1, 2026. Verified claim: order isomorphisms relating Scott's three presentations of domains — continuous lattices (1972), neighbourhood systems (PRG-19), and information systems (1982) — including the round identification `D ≃o RoundFilter ≃o RoundInfoSysElement` and the S-expression equation `T ≅ A + (T × T)` as `|T| ≃o |A + (T × T)|` (`SexSys.Element ≃o SexRhs.Element`). https://palomar-registry.org/entry?id=PALOMAR-2026-08-25-000003&version=1.

[5] Committee on Publication Ethics (COPE). Authorship and ai tools: Cope position statement, 2024. 2024. https://publicationethics.org/guidance/cope-position/authorship-and-ai-tools.

[6] Anysphere, Inc. Composer 2.5. Model announcement and documentation, https://cursor.com/blog/composer-2-5; model card as integrated in Cursor, https://cursor.com/docs/models, 2026. accessed 2026.

[7] xAI and Anysphere, Inc. Grok 4.7. Model documentation, https://docs.x.ai/docs/models; Cursor model catalog, https://cursor.com/docs/models, 2026. accessed 2026.

[8] Google DeepMind. Gemini 3.8. Technical documentation and model cards, https://ai.google.dev/gemini-api/docs/models, 2026. accessed 2026.

[9] ed J. C. Abbott. Trends in lattice theory, 1970. Van Nostrand Reinhold Mathematical Studies, vol. 31 (1970).

[10] S. Abramsky and A. Jung. Domain theory, 1994. In Handbook of Logic in Computer Science, Vol. 3, Oxford University Press, 1994.

[11] M. Aiello L. Aiello and R. W. Weyhrauch. The semantics of pascal in lcf, 1974. Artificial Intelligence Lab. Memo. AIM-221, Stanford Univ., Stanford, Calif.

[12] S. Alagić. Categorical theory of tree processing, 1974. Category Theory Applied to Computation and Control, E. Manes, ed., Univ. of Mass., pp. 80–99.

[13] P. Alexandroff and H. Hopf. Topologie i, 1935. Springer-Verlag, (1935).

[14] M. A. Arbib and E. G. Manes. Fuzzy morphisms in automata theory, 1974. 1974a; , Category Theory Applied to Computation and Control, E. Manes, ed., Univ. of Mass., pp. 98–105.

[15] M. A. Arbib and E. G. Manes. Categorists' view of automata and systems, 1974. 1974b; , Category Theory Applied to Computation and Control, E. Manes, ed., Univ. of Mass., pp. 62–79.

[16] M. A. Arbib and E. G. Manes. Basic concepts of category theory applicable to computation and control, 1974. 1974c; , Category Theory Applied to Computation and Control, E. Manes, ed., Univ. of Mass., pp. 2–41.

[17] J. W. de Bakker. Semantics of programming languages, 1969. Advances in Information Systems Science, vol. 2, Plenum Press, New York.

[18] J. W. de Bakker and D. Scott. A theory of programs, 1969. Unpublished notes, IBM Seminar, Vienna, 1969.

[19] J. W. de Bakker. Recursive procedures, 1971. 1971a; , Mathematical Centre Tracts, vol. 24, Amsterdam.

[20] J. W. de Bakker. Recursion, induction and symbol manipulation, 1971. 1971b; , Proc. MC-25 Informatica Symposium, Mathematical Centre Tracts, vol. 38, Amsterdam.

[21] J. W. de Bakker and W. P. de Roever. A calculus for recursive program schemes, 1972. Proc. IRIA Colloq. on Automata, North-Holland, Amsterdam.

[22] J. W. de Bakker and L. G. L. T. Meertens. On the completeness of the inductive assertion method, 1973. Mathematical Centre Rep. IW 12/72, Amsterdam.

[23] J. W. de Bakker and L. G. L. T. Meertens. Fixed points in programming theory, 1974. Foundations of Computer Science, J. W. de Bakker, ed., Mathematical Centre Tract, vol. 63, Amsterdam.

[24] H. P. Barendregt. Some extensional term models for combinatory logics and λ-calculi, 1971. Ph.D. thesis, Univ. of Utrecht.

[25] H. Bekić. Definable operations in general algebra, and the theory of automata and flowcharts, 1969. Rep., IBM Lab., Vienna.

[26] H. Bekić. Towards a mathematical theory of processes, 1971. Tech. Rep. TR 25.125, IBM Lab., Vienna.

[27] H. Bekić et al. A formal definition of a pl/i subset, parts i and ii, 1974. Tech. Rep. TR 25.139, IBM Lab., Vienna.

[28] E. Binz and H. H. Keller. Funktionenräume in der kategorie der limesräume, 1966. Annales Academiae Scientiarum Fennicae, Series A, I. Mathematica, no. 383 (1966), 21 pp.

[29] R. S. Bird. Unsolvability of the inclusion problem for dbs schemas, 1974. Rep. RCS22, Dept. of Computer Sci., Univ. of Reading.

[30] G. Birkhoff. Lattice theory, 1967. 3rd ed., American Mathematical Society Colloquium Publications, vol. 25, Providence, R.I.

[31] A. Blikle. Algorithmically definable functions: A contribution towards the semantics of programming languages, 1971. Dissertationes Math. Rozprawy Mat., 85.

[32] A. Blikle. Equational languages, 1972. Information Control, pp. 134–147.

[33] A. Blikle and A. Mazurkiewicz. An algebraic approach to the theory of programs, algorithms, languages and recursiveness, 1972. Proc. Internat. Sympos. and Summer School on the Mathematical Foundations of Computer Sci., Warsaw-Jabłonna.

[34] A. Blikle. An algebraic approach to programs and their computations, 1973. Proc. Symp. and Summer School on the Mathematical Foundations of Computer Sci., High Tatras, Czechoslovakia.

[35] A. Blikle. Proving programs by sets of computations, 1974. Proc. 3rd Symp. on the Mathematical Foundation of Computer Sci., Jadwisin, Poland; Lecture Notes in Computer Science, Springer-Verlag, Berlin, 1974.

[36] C. Böhm. The cuch as a formal and descriptive language, 1966. Formal Language Description Languages for Computer Programming, T. B. Steel, ed., North-Holland, Amsterdam, pp. 179–197.

[37] C. Böhm. Alcune proprietà delle forme β-η-normali nel λ-κ-calcolo, 1968. Consiglio Nazionale delle Ricerche: Pubblicazione 696 dell' Istituto per le Applicazioni del Calcolo, Roma.

[38] C. Böhm. λ-calculus and computer science theory, 1975. Proc. Rome Symp., March 1975, Lecture Notes in Computer Science, vol. 37, Springer-Verlag, Berlin.

[39] R. M. Burstall. Proving properties of programs by structural induction, 1969. Comput. J., 12, pp. 41–48.

[40] R. M. Burstall. Some techniques for proving the correctness of programs which alter data structures, 1972. 1972a; , Machine Intelligence vol. 7, B. Meltzer and D. Michie, eds., Edinburgh, pp. 23–50.

[41] R. M. Burstall. An algebraic description of programs with assertions, verification and simulation, 1972. 1972b; , Proc. ACM Conf. on Proving Assertions about Programs, Las Cruces, New Mexico, pp. 7–14.

[42] R. Burstall and J. W. Thatcher. The algebraic theory of recursive program schemes, 1974. Category Theory Applied to Computation and Control, E. Manes, ed., Univ. of Mass., pp. 154–160.

[43] J. M. Cadiou. Recursive definitions of partial functions and their computations, 1973. Ph.D. thesis, Computer Sci. Dept., Stanford Univ., Stanford, Calif.

[44] J. M. Cadiou and J. J. Levy. Mechanizable proofs about parallel processes, 1973. Presented at the 14th Ann. Symp. on Switching and Automata Theory.

[45] J. W. Case. Enumeration reducibility and the partial degrees, 1971. Ann. Math. Logic, 2, pp. 419–440.

[46] E. Čech. Topological spaces, 1966. (revised by Z. Frolík and M. Katětov), Prague.

[47] A. K. Chandra and Z. Manna. On the power of programming features, 1973. Artificial Intelligence Memo. AIM-185, Stanford Univ., Stanford, Calif.

[48] A. K. Chandra. Degrees of translatability and canonical forms in program schemas part i, 1974. IBM Res. Rep. RC4733, Yorktown Heights, N.Y.

[49] A. K. Chandra. The power of parallelism and nondeterminism in programming, 1974. 1974a; , IBM Res. Rep. RC4776, Yorktown Heights, N.Y.

[50] A. K. Chandra. Generalized program schemas, 1974. 1974b; , IBM Res. Rep. RC4827, Yorktown Heights, N.Y.

[51] A. Church. The calculi of λ-conversion, 1951. Annals of Mathematical Studies, vol. 6, Princeton University Press, Princeton, N.J.

[52] M. J. Clint. Program proving: Co-routines, 1972. Acta Informatica, 2, pp. 50–63.

[53] R. C. Constable and D. Gries. On classes of program schemata, 1972. this Journal, 1, pp. 66–118.

[54] D. C. Cooper. The equivalence of certain computations, 1966. Comput. J., 9, pp. 45–52.

[55] B. Courcelle and J. Vuillemin. Complétude d'un système formel pour des équations récursives simples, 1974. Compte-Rendu du Colloque de Paris.

[56] B. Courcelle and J. Vuillemin. Semantics and axiomatics of a simple recursive language, 1974. Rapport Laboria, vol. 60, IRIA; Also: 6th Ann. ACM Symp. on the Theory of Computing, Seattle, Washington.

[57] G. Kahn B. Courcelle and J. Vuillemin. Algorithmes d'équivalence pour des équations récursives simples, 1974. Proc. 2nd Colloq. on Automata Languages and Programming, Saarbrücken; Lecture Notes in Computer Science, Springer-Verlag, Berlin, pp. 213–220.

[58] H. B. Curry and R. Feys. Combinatory logic, 1958. vol. 1, North-Holland, Amsterdam.

[59] J. R. Hindley H. B. Curry and J. Seldin. Combinatory logic, 1971. vol. 2, North-Holland, Amsterdam.

[60] M. Davis. Computability and unsolvability, 1958. McGraw-Hill, New York.

[61] E. W. Dijkstra. Notes on structured programming, 1972. Structural Programming, C. A. R. Hoare, E. W. Dijkstra and O. J. Dahl, Academic Press, New York.

[62] E. W. Dijkstra. A simple axiomatic basis for programming language constructs, 1974. Indag. Math., 36, pp. 1–15.

[63] E. W. Dijkstra. A discipline of programming, 1976. Prentice-Hall, Englewood Cliffs, N.J., to appear.

[64] J. Donahue. Scottery child's guide no. 1, 1974. Dept. of Comput. Sci., Univ. of Toronto.

[65] B. N. Duong. A high level, variable free calculus for recursive programming, 1974. Comput. Sci. Dept. Rep. CS 74 03, Univ. of Waterloo.

[66] S. Eilenberg and C. C. Elgot. Recursiveness, 1970. Academic Press, New York.

[67] S. Eilenberg and C. C. Elgot. Automata, languages and machines, 1974. vol. A, Academic Press, New York (vol. B in press).

[68] A. Robinson C. C. Elgot and J. D. Rutledge. Multiple control computer models, 1966. IBM Res. Rep. RC 1622, Yorktown Heights, N.Y.

[69] C. C. Elgot. Abstract algorithms and diagram closure, 1967. IBM Res. Rep. RC 1750, Yorktown Heights, N.Y.

[70] C. C. Elgot. Algebraic theories and program schemes, 1971. Symposium on Semantics of Algorithmic Languages, E. Engeler, ed., Lecture Notes in Mathematics, vol. 188, Springer-Verlag, New York.

[71] M. H. van Emden and R. A. Kowalski. The semantics of predicate logic as a programming language, 1974. Memo. no. 83 (MIP-R-103), Dept of Machine Intelligence, Univ. of Edinburgh.

[72] E. Engeler. Algorithmic properties of structures, 1967. Mathematical Systems Theory, 1, pp. 183–195.

[73] E. Engeler. Formal languages: Automata and structures, 1968. Lectures in Advanced Mathematics, Markham.

[74] E. Engeler. Algorithmic logic, 1974. Foundations of Computer Science, J. W. de Bakker, ed., Mathematical Centre Tract MCT 63, Amsterdam.

[75] J. Engelfriet. Simple program schemes and formal languages, 1974. Springer Lecture Notes in Computer Science, vol. 20.

[76] J. Engelfriet and E. M. Schmidt. Io and oi, 1975. DAIMI PB-47, Matematisk Institut, Aarhus Universitet, Datalogisk Afdeling, Denmark.

[77] Ju. L. Ershov. Computable numerations of morphisms, 1971. Algebra and Logic, 10, pp. 247–308.

[78] Ju. L. Ershov. Computable functions of finite types, 1972. 1972a; Algebra and Logic, 11, pp. 367–437.

[79] Ju. L. Ershov. Everywhere defined (total) continuous functionals, 1972. 1972b; Algebra and Logic, 11, pp. 656–665.

[80] Ju. L. Ershov. Continuous lattices and a-spaces, 1972. 1972d; , Dokl. Akad. Nauk USSR, 207, pp. 523–526.

[81] Ju. L. Ershov. Theory of a-spaces, 1972. 1972e; , Algebra and Logic, 12, pp. 369–416.

[82] Ju. L. Ershov. Theorie der numerierungen, 1973. Veb Deutscher Verlag der Wissenschaften, Berlin.

[83] M. J. Fischer. λ-calculus schemata, 1972. Proc. ACM Conf. Proving Assertions about Programs, Las Cruces, N.M., pp. 104–109.

[84] A. E. Fischer and M. J. Fischer. Mode modules as representations of domains, 1973. Proc. ACM Symp. on Principles of Programming Languages, Boston, pp. 139–143.

[85] R. W. Floyd. Assigning meanings to programs, 1967. Proc. Symp. in Appl. Math., vol. 19, Mathematical Aspects of Computer Science, J. T. Schwartz, ed., pp. 33–41.

[86] M. M. Fokkinga. Inductive assertion patterns for recursive procedures, 1974. Compte-Rendu de Colloque de Paris.

[87] R. M. Friedberg and H. R. Rogers. Reducibility and completeness for sets of integers, 1959. Z. Math. Logik Grundlagen Math., 5, pp. 117–125.

[88] H. Friedman. Algorithmic procedures, generalized turing algorithms, and elementary recursion theory, 1971. Logic Colloquium 1969, North-Holland, Amsterdam, pp. 361–389.

[89] S. J. Garland and D. C. Luckham. Translating recursion schemes into program schemes, 1972. Proc. ACM Conf. on Proving Assertions about Programs, Las Cruces, N.M., pp. 83–96.

[90] H. Gericke. Lattice theory, 1966. Harrap.

[91] G. Gierz et al. Continuous lattices and domains, 2003. Cambridge University Press, 2003.

[92] J. A. Goguen. l-fuzzy sets, 1967. J. Math. Anal. Appl., 18, pp. 145–174.

[93] J. A. Goguen. Categories of l-sets, 1969. Bull. Amer. Math. Soc., 75, pp. 524–622.

[94] J. A. Goguen. On homomorphisms, simulations, correctness, subroutines and termination for programs and program schemes, 1972. Proc. 13th IEEE Conf. on Switching and Automata Theory, pp. 52–60.

[95] J. A. Goguen. Semantics of computation, 1974. Category Theory Applied to Computation and Control, E. Manes, ed., Univ. of Mass., pp. 234–249.

[96] J. A. Goguen et al. Initial algebra semantics, 1975. IBM Res. Rep. RC 5243, Yorktown Heights, N.Y.

[97] M. J. C. Gordon. Models of pure lisp, 1973. 1973a; , Experimental Programming Rep. 31, School of Artificial Intelligence, Univ. of Edinburgh. (Also Ph.D. thesis, Evaluation and denotation of pure LISP programs: A worked example in semantics.).

[98] M. J. C. Gordon. An extended abstract of "models of pure lisp", 1973. 1973b; , Memo. SAI-RM-7, School of Artificial Intelligence, Univ. of Edinburgh.

[99] M. J. C. Gordon. Operational reasoning and denotational semantics, 1975. 1975a; , Artificial Intelligence Memo. 264, Stanford Univ., Stanford, Calif.

[100] M. J. C. Gordon. Towards a semantic theory of dynamic binding, 1975. 1975b; , Artificial Intelligence Memo. 265, Stanford Univ., Stanford, Calif.

[101] G. Grätzer. Universal algebra, 1968. Van Nostrand, New York.

[102] I. Guessarian. Sur une réduction des schémas de programmes polyadiques à des schémas monadiques et ses applications, 1974. Compte-Rendu de Colloque de Paris.

[103] L. Gutteridge. Some results on enumeration reducibility, 1971. Ph.D. dissertation, Simon Fraser Univ., Burnaby, B.C., Canada.

[104] B. Lercher J. R. Hindley and J. P. Seldin. Introduction to combinatory logic, 1972. London Mathematical Society Lecture Note Series, vol. 7, Cambridge University Press, Cambridge, England.

[105] J. R. Hindley and G. Mitschke. Some remarks about the connections between combinatory logic and axiomatic recursion theory, 1975. Preprint 203, Fachbereich Mathematik, Technische Hochschule Darmstadt.

[106] P. Hitchcock and D. M. R. Park. Induction rules and proofs of termination, 1972. Proc. Colloques IRIA, Theorie des Automates des Languages et de la Programmation.

[107] P. Hitchcock. An approach to formal reasoning about programs, 1974. Ph.D. thesis, Univ. of Warwick, England.

[108] C. A. R. Hoare. An axiomatic basis of computer programming, 1969. Comm. ACM, 12, pp. 576–580, 583.

[109] C. A. R. Hoare. Proof of a program: Find, 1971. 1971a; , Comm. ACM, 14, pp. 39–45.

[110] C. A. R. Hoare. Procedures and parameters: An axiomatic approach, 1971. 1971b; , Symposium on Semantics of Programming Languages, E. Engeler, ed., Lecture Notes in Mathematics, vol. 188, Springer-Verlag, New York, pp. 102–116.

[111] C. A. R. Hoare. Notes on data structuring, 1972. Structured Programming, C. A. R. Hoare, E. W. Dijkstra and O. J. Dahl, Academic Press, New York.

[112] C. A. R. Hoare and N. Wirth. An axiomatic definition of the programming language pascal, 1972. Bericht der Fachgruppe Computer-Wissenschaften 6, Eidgenössische Technische Hochschule, Zürich.

[113] C. A. R. Hoare and P. Lauer. Consistent and complementary formal theories of the semantics of programming languages, 1974. Acta Informatica, 3, pp. 135–153.

[114] C. Hosono and M. Sato. A solution to scott's problem: "do the retracts in $p\omega$ form a continuous lattice?", 1975. Research Inst. for Mathematical Sci., Kyoto (preprint).

[115] G. Hotz. Eindeutigkeit und mehrdeutigkeit formaler sprachen, 1966. Electron. Informationsverarbeit. Kybernetik (Berlin), 2, pp. 235–246.

[116] J. M. E. Hyland. Recursion theory on the countable functionals, 1975. 1975a; , D. Phil. thesis, Oxford Univ., Oxford, England.

[117] J. M. E. Hyland. A survey of some useful partial order relations in terms of the lambda calculus, 1975. 1975b; , Proc. Conf. on λ-Calculus and Computer Science Theory, Rome, pp. 83–95.

[118] J. M. E. Hyland. A syntactic characterization of the equality in some models for the lambda calculus. to appear; , J. London Math. Soc.

[119] Y. I. Ianov. The logical scheme of algorithms, 1970. Problems in Cybernetics, vol. 1, Pergamon Press, New York.

[120] S. Igarashi. Admissibility of fixed point induction in first order logic of typed theories, 1972. Artificial Intelligence Memo. AIM-168, Computer Sci. Dept., Stanford Univ., Stanford, Calif.

[121] K. Indermark. Gleichungsdefinierbarkeit in relationalstrukturen, 1974. Habilitationsschrift, Mathematisch-Naturwissenschaftlichen Fakultät der Rheinischen Freidrich-Wilhelms-Universatat, Bonn, W. Germany.

[122] C. B. Jones. Mathematical semantics of goto: Exit formulation and its relation to continuations, 1974. preprint.

[123] G. Kahn. A preliminary theory of parallel programs, 1973. Rapport Laboria IRIA.

[124] D. M. Kaplan. Regular expressions and the equivalence of programs, 1969. J. Comput. System Sci., 3, pp. 361–385.

[125] R. M. Karp and R. E. Miller. Parallel program schemata, 1969. J. Comput. System Sci., 3, pp. 147–195.

[126] J. L. Kelley. General topology, 1955. Van Nostrand, (1955).

[127] D. J. Kfoury. Comparing algebraic structures up to algorithmic equivalence, 1972. Automata, Languages and Programming, M. Nivat, ed., North-Holland, Amsterdam, pp. 253–263.

[128] D. J. Kfoury. Translatability of schemas over restricted interpretations, 1974. J. Comput. System Sci., 8, pp. 387–408.

[129] S. C. Kleene. Introduction to metamathematics, 1950. Van Nostrand, New York.

[130] B. Knaster. Un théorème sur les fonctions d'ensembles, 1928. Ann. Soc. Polon. de Math., 8, pp. 133–134.

[131] P. J. Landin. The mechanical evaluation of expressions, 1964. Comput. J., 6, pp. 308–320.

[132] P. J. Landin. A correspondence between algol 60 and church's lambda notation, 1965. Comm. ACM, 8, pp. 89–101.

[133] P. J. Landin. A λ-calculus approach, 1966. 1966a; , Advances in Programming and Non-numerical Computation, L. Fox, ed., Pergamon Press, New York, pp. 97–141.

[134] P. J. Landin. A formal description of algol 60, 1966. 1966a; , Formal Language Description Languages for Computer Programming, T. B. Steel, ed., North-Holland, Amsterdam, pp. 266–294.

[135] P. J. Landin. The next 700 programming languages, 1966. 1966b; , Comm. ACM, 9, pp. 157–164.

[136] P. J. Landin. A program/machine symmetric automata theory, 1969. Machine Intelligence, vol. 5, B. Meltzer and D. Michie, eds., Edinburgh University Press, pp. 99–120.

[137] P. J. Landin and R. M. Burstall. Programs and their proofs: An algebraic approach, 1969. Machine Intelligence, vol. 4, American Elsevier, New York, pp. 17–44.

[138] J. Leszczykowski. A theorem on resolving equations in the space of languages, 1971. Bull. Acad. Polon. Sci. Sér. Sci. Math. Astronom. Phys., 19, pp. 967–970.

[139] C. H. Lewis and B. K. Rosen. Recursively defined data types, parts i and ii, 1973. 1973–74; IBM Res. Reps. RC 4429 (1973), RC 4713 (1974), Yorktown Heights, N.Y.

[140] B. Liskov and S. Zilles. An approach to abstraction, 1973. Group Memo. 88, Project MAC, Mass. Inst. of Tech., Cambridge, Mass.

[141] D. M. R. Park D. Luckham and M. Paterson. On formalized computer programs, 1970. J. Comput. System Sci., 4, pp. 220–249.

[142] S. Mac Lane. Categories for the working mathematician, 1972. Springer-Verlag, New York.

[143] E. Manes. Category theory applied to computation and control, 1974. Proc. 1st Internat. Symp. Math. Dept. and the Dept. of Computer and Information Sci., Univ. Mass., Amherst. Also: Lecture Notes in Computer Science, vol. 26, Springer-Verlag, New York.

[144] Z. Manna. The correctness of programs, 1969. J. Comput. System Sci., 3, pp. 119–127.

[145] Z. Manna and A. Pnueli. Formalization of properties of functional programs, 1970. J. Assoc. Comput. Mach., 17, pp. 555–569.

[146] Z. Manna and J. McCarthy. Properties of programs and partial function logic, 1970. Machine Intelligence 5, B. Meltzer and D. Michie, eds., Edinburgh University Press, pp. 27–38.

[147] Z. Manna and A. Pnueli. Axiomatic approach to total correctness of programs, 1972. Artificial Intelligence Lab. Memo. AIM-210, Stanford Univ., Stanford, Calif.

[148] Z. Manna and J. M. Cadiou. Recursive definitions of partial functions and their computations, 1972. Proc. ACM Conf. on Proving Assertions about Programs, Las Cruces, N.M., pp. 58–65.

[149] Z. Manna and J. Vuillemin. Fixpoint approach to the theory of computation, 1972. Comm. ACM, 15, pp. 528–536.

[150] Z. Ness Z. Manna and J. Vuillemin. Inductive methods for proving properties of programs, 1972. Proc. ACM Conf. on Proving Assertions about Programs, Las Cruces, N.M., pp. 27–50.

[151] Z. Manna. Mathematical theory of computation, 1974. McGraw-Hill, New York.

[152] G. Markowsky. Categories of chain-complete posets, 1974. RC 5100, Comput. Sci. Dept. IBM T. J. Watson Research Center, Yorktown Heights, N.Y.

[153] G. Markowsky. Chain-complete posets and directed sets with applications, 1974. RC 5024, IBM T. J. Watson Research Center, Yorktown Heights, N. Y.

[154] A. Mazurkiewicz. Proving algorithms by tail functions, 1971. Working Paper for IFIP WG2.2, Feb. 1970. Since published in: Information and Control, 18 (1971), pp. 220–226.

[155] A. Mazurkiewicz. Proving properties of processes, 1973. PRACE CO PAN-CC PAS Reports, vol. 134, Warsaw.

[156] J. McCarthy. The lisp 1.5 programmers' manual, 1962. MIT Press, Cambridge, Mass.

[157] J. McCarthy. A basis for a mathematical theory of computation, 1963. 1963a; , Computer Programming and Formal Systems, D. Braffort and D. Hirshberg, eds., North-Holland, Amsterdam, pp. 33–70.

[158] J. McCarthy. Towards a mathematical science of computation, 1963. 1963b; , Information Processing 1962, Proc. IFIP Congress 1962, C. M. Poppleworth, ed., North-Holland, Amsterdam, pp. 21–28.

[159] J. McCarthy. A formal description of a subset of algol, 1966. Formal Language Description Languages for Computer Programming, R. B. Steel, ed., North-Holland, Amsterdam, pp. 1–12.

[160] J. McCarthy and J. Painter. Correctness of a computer for arithmetic expressions, 1967. Mathematical Aspects of Computer Science, J. T. Schwartz, ed., Proc. of a Symposium in Applied Mathematics, vol. 19, pp. 33–41.

[161] E. Michael. Topologies on spaces of subsets, 1951. Transactions of the American Mathematical Society, vol. 77 (1951), pp. 152-182.

[162] R. E. Milne. The formal semantics of computer languages and their implementations, 1974. Ph.D. thesis, Cambridge Univ., Cambridge, England. [Also: Technical Monograph PRG-13, Oxford Univ. Computing Lab., Programming Research Group].

[163] R. Milner. Program schemes and recursive function theory, 1969. Machine Intelligence 5, B. Meltzer and D. Michie, eds., Edinburgh University Press, pp. 39–58.

[164] R. Milner. Algebraic theory of computable polyadic functions, 1970. 1970a; , Computer Sci. Memo., vol. 12, University College, Swansea.

[165] R. Milner. Equivalences on program schemes, 1970. 1970b; , J. Comput. System Sci., 4, pp. 205–219.

[166] R. Milner. Implementation and application of scott's logic for computable functions, 1972. Proc. ACM Conf., Las Cruces, N.M., pp. 1–6.

[167] R. Milner and R. Weyhrauch. Proving compiler correctness in a mechanized logic, 1972. Machine Intelligence, 7, pp. 51–72.

[168] R. Milner. Models of lcf, 1973. Artificial Intelligence Memo. AIM-186, Computer Sci. Dept., Stanford Univ., Stanford, Calif.

[169] R. Milner. An approach to the semantics of parallel programs, 1973. 1973a; , Proc. Convegno di Informatica Teorica, Instituto di Elaborazione delle Informazione, Pisa, Italy.

[170] R. Milner. Processes: A mathematical model of computing agents, 1973. 1973b; , Proc. Colloq in Mathematical Logic, Bristol, England.

[171] J. H. Morris. λ-calculus models of programming languages, 1968. Ph.D. thesis, Sloan School of Management, MIT MAC Reprint TR-57, Mass. Inst. of Tech., Cambridge, Mass.

[172] F. Lockwood Morris. The next 700 programming language descriptions, 1970. Computer Center, Univ. of Essex (typescript).

[173] J. H. Morris. Another recursion induction principle, 1971. Comm. ACM, 14, pp. 351–354.

[174] F. Lockwood Morris. Correctness of translations of programming languages, 1972. Computer Sci. Memo. CS 72-303, Stanford Univ., Stanford, Calif.

[175] J. H. Morris. A correctness proof using recursively defined functions, 1972. Formal Semantics of Programming Languages, R. Rustin, ed., Prentice-Hall, Englewood Cliffs, N.J., pp. 107–224.

[176] F. Lockwood Morris. Advice on structuring compilers and proving them correct, 1973. Proc. SIGACT/SIGPLAN Symp. on Principles of Programming Languages, Boston, pp. 144–152.

[177] J. H. Morris. Types are not sets, 1973. Proc. ACM Symp. on Principles of Programming Languages, Boston, pp. 120–124.

[178] J. Morris and R. Nakajima. Mechanical characterisation of the partial order in lattice models of λ-calculus, 1973. Tech. Rep. 18, Univ. of Calif., Berkeley.

[179] P. D. Mosses. The mathematical semantics of algol 60, 1974. Tech. Monograph PRG-12, Oxford Univ. Computing Lab., Programming Research Group.

[180] P. D. Mosses. The semantics of semantic equations, 1975. 1975a; , Mathematical Foundations of Computer Science, Goos and Hartmanis, eds., Lecture Notes in Computer Science, vol. 28, Springer-Verlag, New York, pp. 409–422.

[181] P. D. Mosses. Mathematical semantics and compiler generation, 1975. 1975b; , Thesis, Oxford Univ., Oxford, England.

[182] J. Myhill and J. C. Shepherdson. Effective operations on partial recursive functions, 1955. Z. Math. Logik Grundlagen Math., 1, pp. 310–317.

[183] R. Nakajima. Infinite normal forms for λ-calculus, 1975. λ-Calculus and Computer Science Theory, C. Böhm, ed., Springer-Verlag, Berlin, pp. 62–82.

[184] A. Nerode. Some stone spaces and recursion theory, 1959. Duke Mathematical Journal, vol. 26, pp. 397–406.

[185] M. Nivat. Langages algébriques sur le magma libre et sémantique des schémas de programmes, 1972. Proc. IRIA Symposium on Automata Formal Languages and Programming, North-Holland, Amsterdam.

[186] D. C. Oppen. On logic and program verification, 1975. Tech. Rep. 82, Dept. Computer Sci., Univ. of Toronto.

[187] D. M. R. Park. Fixpoint induction and proofs of program properties, 1970. 1970a; , Machine Intelligence, 5, B. Meltzer and D. Michie, eds., American Elsevier, New York, pp. 59–78.

[188] D. M. R. Park. Notes on a formalism for reasoning about schemes, 1970. 1970b; , Univ. of Warwick, unpublished notes.

[189] D. M. R. Park. The y combinator scott's lambda-calculus models, 1970. 1970c; , Univ. of Warwick, unpublished notes.

[190] D. M. R. Park. Finiteness is mu-ineffable, 1970. 1970d; , Theory of Computation Report, No. 3, Dept. of Computer Sci., Univ. of Warwick (1974).

[191] M. S. Paterson. Program schemata, 1963. Machine Intelligence, 3, D. Michie, ed., American Elsevier, New York, pp. 19–32.

[192] M. S. Paterson and C. S. Hewitt. Comparative schematology, 1970. Record of Project MAC Conf. on Concurrent Systems and Parallel Computation, Association for Computing Machinery, New York, pp. 119ff.

[193] R. Platek. New foundations for recursion theory, 1964. Thesis, Stanford Univ., Stanford, Calif., unpublished.

[194] G. D. Plotkin. A set-theoretical definition of application, 1972. Memo. MIP-R-95, School of Artificial Intelligence, University of Edinburgh.

[195] G. D. Plotkin. The λ-calculus is ω-incomplete, 1973. 1973a; , Res. Memo. SAI-RM-2, School of Artificial Intelligence, Univ. of Edinburgh.

[196] G. D. Plotkin. Lambda-definability and logical relations, 1973. 1973b; , Memo. SAI-RM-4, School of Artificial Intelligence, Univ. of Edinburgh.

[197] G. D. Plotkin. Call-by-name, call-by-value and the λ-calculus, 1973. 1973c; , J. Res. Memo. SAI-RM-6, School of Artificial Intelligence, Univ. of Edinburgh.

[198] G. D. Plotkin. A powerdomain construction, 1975. Dept. of Artificial Intelligence Res. Rep. 3, Univ. of Edinburgh.

[199] H. Rasiowa. On ω-1973.-valued algorithmic logic and related problems*, Supplement to Proc. Symp. and Summer School on Mathematical Foundations of Computer Sci., High Tatras, Czechoslovakia.

[200] P. Raulefs. The overtyped lambda calculus, 1975. 1975a; , Interner Bericht Nr. 2, Institut für Informatik, Universität Karlsruhe.

[201] P. Raulefs. Standard models of the overtyped lambda-calculus, 1975. 1975b; , Interner Bericht Nr. 3, Institut für Informatik, Universität Karlsruhe.

[202] R. R. Redziejowski. The theory of general events and its application to parallel programming, 1972. T.P. 18.220 IBM Nordic Laboratory, Sweden.

[203] J. C. Reynolds. Gedanken: A simple typeless language, 1969. Reprint 7621, Argonne National Laboratory, Argonne, Ill.

[204] J. C. Reynolds. Definitional interpreters for higher order programming languages, 1972. 1972a; , Proc. ACM 25th Nat. Conf., Boston, vol. 2, pp. 717–740.

[205] J. C. Reynolds. Notes on a lattice-theoretic approach to the theory of computation, 1972. 1972b; , Systems and Information Sci. Dept., Syracuse Univ., Syracuse, N.Y.

[206] J. C. Reynolds. On the relation between direct and continuation semantics, 1974. 1974a; , Proc. 2nd Colloq. on Automata, Languages and Programming, Saarbrücken, Lecture Notes in Computer Science, vol. 14, Springer-Verlag, Berlin, pp. 141–156.

[207] J. C. Reynolds. Semantics of the lattice of flow diagrams, 1974. 1974b; , Systems and Information Science Dept., Syracuse Univ., Syracuse, N.Y. (preprint).

[208] J. C. Reynolds. On the interpretations of scott's domains. to appear; Symposia Mathematica.

[209] J. C. Reynolds. Towards a theory of type structure. to appear; , Colloq. on Programming, Paris, April 1974, Lecture Notes in Computer Science, Springer-Verlag, Berlin.

[210] W. P. de Roever. Recursion and parameter mechanisms: An axiomatic approach, 1974. 1974a; Proc. 2nd Colloq. on Automata Languages and Programming, Saarbrucken, Lecture Notes in Computer Science, vol. 14, Springer-Verlag, Berlin, pp. 34–65.

[211] W. P. de Roever. Operational, mathematical and axiomatized semantics for recursive procedures and data structures, 1974. 1974b; , Mathematical Centre Report ID/1, Amsterdam.

[212] H. R. Rogers. Theory of recursive functions and effective computability, 1967. McGraw-Hill, New York.

[213] B. K. Rosen. Tree-manipulating systems and church-rosser theorems, 1973. J. Assoc. Comput. Mach., 20.

[214] J. D. Rutledge. Program schemata as automata, part i, 1970. 1970a; IBM Res. RC 3098. J. Comput and System Sci., 7 (1973), pp. 543–578.

[215] J. D. Rutledge. Parallel processes . . . schemata and transformation, 1970. 1970b; , IBM Res. Rep. RC 2912, Yorktown Heights, N.Y.

[216] A. Salwicki. Formalized algorithmic languages, 1970. 1970a; , Bull. Acad. Polon. Sci. Sér. Math. Astronom. Phys., 18, pp. 227–232.

[217] A. Salwicki. On the equivalence of fs-expressions and programs, 1970. 1970b; , Bull. Acad. Polon. Sci. Sér. Math. Astronom. Phys., 18, pp. 275–278.

[218] A. Salwicki. On the predicate calculi with iteration quantifiers, 1970. 1970c; , Bull. Acad. Polon. Sci. Sér. Math. Astronom. Phys., 18, pp. 279–286.

[219] J. G. Sanderson. The λ calculus, lattice theory and reflexive domains, 1973. Mathematical Institute Lecture Notes, Oxford, England.

[220] L. P. Sasso. Degrees of unsolvability of partial functions, 1971. Ph.D. dissertation, Univ. of Calif., Berkeley.

[221] L. P. Sasso. A minimal partial degree, 1973. Proc. Amer. Math. Soc., 38, pp. 388–392.

[222] L. P. Sasso. A survey of partial degrees, 1975. J. Symbolic Logic, 40, pp. 130–140.

[223] D. Scott. A type-theoretical alternative to iswim, cuch, owhy, 1969. Unpublished notes, Oxford, England.

[224] D. Scott. Outline of a mathematical theory of computation, 1970. Proceedings of the Fourth Annual Princeton Conference on Information Sciences and Systems, pp. 169–176. [Also: Tech. Monograph PRG-2, Oxford Univ. Computing Lab., Programming Research Group (1970).].

[225] D. Scott. Lattice theory, data types, and semantics, 1971. New York University Symposia in Areas of Current Interest in Computer Science (Randall Rustin ed.) (1971) to appear.

[226] D. Scott. The lattice of flow diagrams, 1971. Symposium on Semantics of Algorithmic Languages, E. Engeler, ed., Lecture Notes in Mathematics, vol. 188, Springer-Verlag, New York, pp. 311–366. [Also: Tech. Monograph PRG-3, Oxford Univ. Computing Lab., Programming Research Group (1970).].

[227] D. Scott and C. Strachey. Toward a mathematical semantics for computer languages, 1971. Proc. Symposium on Computers and Automata, Polytechnic Inst. of Brooklyn, vol. 21, pp. 19–46. [Also: Technical Monograph PRG-6, Oxford Univ. Computing Lab., Programming Research Group (1971).].

[228] D. Scott. Mathematical concepts in programming language semantics, 1972. 1972b; , AFIPS Conf. Proc., vol. 40, pp. 225–234.

[229] D. Scott. Data types as lattices, 1973. 1973a; , Unpublished lecture notes, Mathematical Centre, Amsterdam, 1973.

[230] D. Scott. Models for various type-free calculi, 1973. 1973b; , Proc. IVth Internat. Cong. for Logic, Methodology and the Philosophy of Science, Bucharest, P. Suppes et al., eds., North-Holland, Amsterdam, pp. 157–187.

[231] D. Scott. Lambda calculus and recursion theory, 1975. 1975a; , Proc. 3rd Scandinavian Logic Symposium, Stig Kanger, ed., North-Holland, Amsterdam, pp. 154–193.

[232] D. Scott. Combinators and classes, 1975. 1975b; , λ-Calculus and computer science theory, Proc. Rome Symp., March 1975; Lecture Notes in Computer Science, vol. 37, Springer-Verlag, Berlin, pp. 1–26.

[233] D. Scott. Some philosophical issues concerning theories of combinators, 1975. 1975c; , λ-Calculus and computer science theory, Proc. Rome Symp., March 1975; Lecture Notes in Computer Science, vol. 37, Springer-Verlag, Berlin, pp. 346–366.

[234] J. C. Shepherdson. Computation over abstract structures: Serial and parallel procedures and friedman's effective definitional schemes, 1975. Logic Colloquium 1973, H. E. Rose and J. C. Shepherdson, eds., North-Holland, Amsterdam, pp. 445–513.

[235] E. Spanier. Quasi-topologies, 1963. Duke Mathematical Journal, vol. 30 (1963) pp. 1-14.

[236] C. Strachey. Towards a formal semantics, 1966. Formal Language Description Languages for Computer Programming, T. B. Steel, ed., North-Holland, Amsterdam, pp. 198–220.

[237] C. Strachey. Fundamental concepts in programming languages, 1967. Unpublished lecture notes for the NATO Summer School, Copenhagen.

[238] C. Strachey. Varieties of programming languages, 1972. Proc. Internat. Computing Symp., Cini Foundation, Venice, pp. 222–233. [Also: Tech. Monograph PRG-10, Oxford Univ. Computing Lab., Programming Research Group (1972).].

[239] C. Strachey and C. Wadsworth. Continuations: A mathematical semantics for handling full jumps, 1974. Tech. Monograph PRG-11, Oxford Univ. Computing Lab., Programming Research Group.

[240] W. Strother. Fixed points, fixed sets, and m-retracts, 1955. Duke Mathematical Journal, vol. 22 (1955), pp. 551-556.

[241] A. Tang. Recursion theory and descriptive set theory in effectively given t_0 spaces, 1974. Ph.D. thesis, Princeton Univ., Princeton, N.J.

[242] A. Tang. Borel sets in $p\omega$, 1975. 1975a; , IRIA-Laboria, preprint.

[243] A. Tang. Notes on subsets on $p\omega$ with extra-finitary property, 1975. 1975b; , IRIA, preprint.

[244] A. Tang. A hierarchy of b_δ sets in $p\omega$, 1975. 1975c; , IRIA, preprint.

[245] A. Tang. Sets of the form $\{x \mid R(x) = T\}$ in $p\omega$, 1975. 1975c; , IRIA, preprint.

[246] A. Tang. $\leqq_e$ degrees in p, iria, preprint, 1975. 1975d.

[247] A. Tarski. On some fundamental concepts of mathematics, 1930. ; translated by J. H. Woodger in Logic, Semantics, Metamathematics, Cambridge, 1956, pp. 30–37.

[248] A. Tarski. A lattice-theoretical fixpoint theorem and its applications, 1955. Pacific J. Math., 5, pp. 285–309.

[249] R. D. Tennent. Mathematical semantics and design of programming languages, 1973. Ph.D. thesis, Univ. of Toronto. [Also: Tech. Rep. 59, Univ. of Toronto (1973).].

[250] R. D. Tennent. The mathematical semantics of programming languages, 1974. 1974a; , Dept. of Computing and Information Sci., Queen's Univ., Kingston, Ontario, Canada (preprint).

[251] R. D. Tennent. A contribution to the development of pascal-like languages, 1974. 1974b; , Tech. Rep. 74-25, Dept. of Computing and Information Sci., Queen's Univ., Kingston, Ontario, Canada.

[252] R. D. Tennent. Pasqual: A proposed generalization of pascal, 1975. Tech. Rep. 75-32, Dept. of Computing and Information Sci., Queen's Univ., Kingston, Ontario, Canada.

[253] J. Vuillemin. Proof techniques for recursive programs, 1973. 1973a; , IRIA—Laboria Rep.

[254] J. Vuillemin. Correct and optimal implementations of recursion in a simple programming language, 1973. 1973b; , IRIA—Laboria Rep. 24.

[255] C. P. Wadsworth. Semantics and pragmatics of the lambda-calculus, 1971. D. Phil. thesis, Oxford Univ., Oxford, England.

[256] C. P. Wadsworth. The relation between lambda-expressions and their denotation, 1975. 1975a; , Dept. of Systems and Information Sci., Syracuse Univ., Syracuse, N.Y. (preprint).

[257] C. P. Wadsworth. Approximate reduction and lambda-calculus models, 1975. 1975b; , Dept. of Systems and Information Sci., Syracuse Univ., Syracuse, N.Y.

[258] C. P. Wadsworth. Some unusual λ-calculus numeral systems, 1975. 1975c; , in preparation.

[259] C. P. Wadsworth. On the topological ordering in the d-model of the lambda-calculus. to appear; , in preparation.

[260] E. G. Wagner. Languages for defining sets in arbitrary algebras, 1971. 1971a; , Proc. of the 11th IEEE Conf. on Switching and Automata Theory, pp. 192–201.

[261] E. G. Wagner. An algebraic theory of recursive definitions and recursive languages, 1971. 1971b; , Proc. of the 3rd ACM Symp. on Theory of Computing.

[262] E. G. Wagner. Notes on categories, algebras and programming languages, 1974. Unpublished lecture notes, London.

[263] M. Wand. Mathematical foundations of formal language theory, 1973. MAC-TR-108, Project MAC, Mass. Inst. of Tech., Cambridge, Mass.

[264] M. Wand. An algebraic formulation of the chomsky hierarchy, 1974. 1974a; , Category Theory Applied to Computation and Control, E. Manes, ed., Univ. of Mass., pp. 216–221.

[265] M. Wand. On the recursive specification of data types, 1974. 1974b; , Category Theory Applied to Computation and Control, E. Manes, ed., Univ. of Mass., pp. 222–225.

[266] M. Wand. Realizing data structures as lattices, 1974. 1974c; , Tech. Rep. 11, Computer Sci. Dept., Indiana Univ.

[267] M. Wand. Fixed-point constructions in order-enriched categories, 1975. Tech. Rep. 23, Computer Sci. Dept., Indiana Univ.

[268] P. Wegner and D. Lehmann. Algebraic and topological prerequisites to scott's theory of computation, 1972. Tech. Rep. 72-2, Dept. of Computer Sci., Hebrew Univ. of Jerusalem.

[269] R. W. Weyrauch and R. Milner. Program correctness in a mechanized logic, 1972. Proc. of the 1st USA-JAPAN Computer Conf., pp. 384–390.

[270] J. B. Wright. Characterization of recursively enumerable sets, 1972. J. Symbolic Logic, 37, pp. 507–511.